METRIC
MIL-STD-1838A
6 April 1989

Superseding
DOD-STD-1838
9 October 1986

# MILITARY STANDARD

## COMMON ADA PROGRAMMING SUPPORT ENVIRONMENT (APSE) INTERFACE SET (CAIS)

(Revision A)

Volume II

NO DELIVERABLE DATA
REQUIRED BY THIS DOCUMENT

AREA MCCR/IPSC

Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

# CONTENTS


**1 SCOPE** 1
1.1 Purpose . . . 1
1.2 Application Guidance . . . 2

**2 REFERENCED DOCUMENTS** 3
2.1 Government Documents . . . 3
2.2 Other Publications . . . 3
2.3 Order of Precedence . . . 4
2.4 Source of Documents . . . 5

**3 DEFINITIONS** 7

**4 GENERAL REQUIREMENTS** 33
4.1 Introduction . . . 33
4.2 Method of Description . . . 37
4.2.1 Allowable differences . . . 37
4.2.2 Typographical conventions . . . 37
4.2.3 Semantic descriptions . . . 38
4.3 CAIS Node Model . . . 40
4.3.1 Nodes . . . 40
4.3.2 Relationships . . . 41
4.3.3 Attributes . . . 43
4.3.4 Processes . . . 43
4.3.5 Transactions . . . 44
4.3.6 Basic predefine definitions . . . 44
4.3.7 Names and identification . . . 50
4.3.8 Relationship Identification . . . 51
4.3.9 Pathnames . . . 53
4.3.10 Unique identifiers . . . 54
4.3.11 CAIS definitions . . . 55
4.3.12 Views . . . 59
4.3.13 Attribute monitors . . . 61
4.4 Access Control . . . 62
4.5 Input and Output . . . 64
4.5.1 Protocols . . . 64
4.5.2 Channels, client nodes, and PORT relationships . . . 65
4.5.3 Creating and using channels . . . 67
4.5.4 Compatibility checks . . . 69
4.5.5 Common IO . . . 69
4.6 Resource Control . . . 76
4.6.1 Resource model elements . . . 77
4.6.2 Resource allocatability . . . 80
4.6.3 Resource visibility . . . 80
4.6.4 Transparent resources . . . 82




Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

4.6.5 Resource access control . . . . . 82
4.6.6 Assigning resources . . . . . 82
4.7 Distribution . . . . . 85
4.7.1 Remote interprocess communications . . . . . 85

**5 CAIS NODE MANAGEMENT DETAILED REQUIREMENTS** **87**
5.1 Package CAIS_DEFINITIONS . . . . . 88
5.2 Package CAIS_NODE_MANAGEMENT . . . . . 95
5.2.1 Open operations . . . . . 95
5.2.2 Accesses without open operations . . . . . 96
5.2.3 Access delay s . . . . . 97
5.2.4 Opening a node handle . . . . . 103
5.2.5 Opening a node handle using a unique identifier . . . . . 106
5.2.6 Closing a node handle . . . . . 107
5.2.7 Changing the intent regarding node handle usage . . . . . 108
5.2.8 Examining the open status of a node handle . . . . . 109
5.2.9 Querying the intention of a node handle . . . . . 109
5.2.10 Querying the kind of anode . . . . . 110
5.2.11 Determining if one node kind is a specialization of another . . . . . 110
5.2.12 Determining if one relation is a specialization of another . . . . . 112
5.2.13 Determining if a node belongs to a node kind . . . . . 113
5.2.14 Determining if a relationship belongs to a relation . . . . . 114
5.2.15 Querying the number of open file handles on a CLIENT node . . . . . 116
5.2.16 Obtaining the unique identifier for a node . . . . . 116
5.2.17 Obtaining a pathname for the unique primary path . . . . . 117
5.2.18 Obtaining the relationship key list of a primary relationship . . . . . 118
5.2.19 Obtaining the relation of a primary relationship . . . . . 118
5.2.20 Obtaining the relationship key list of the last relationship traversed . . . 119
5.2.21 Obtaining the relation of the last relationship traversed . . . . . 120
5.2.22 Obtaining a partial pathname . . . . . 121
5.2.23 Obtaining the name of the last relation in a pathname . . . . . 122
5.2.24 Obtaining the key list of the last relationship in a pathname . . . . . 122
5.2.25 Querying the existence of a node . . . . . 123
5.2.26 Querying sameness . . . . . 124
5.2.27 Obtaining an index for a node handle . . . . . 124
5.2.28 Opening a node handle to the parent . . . . . 125
5.2.29 Copying a node . . . . . 126
5.2.30 Copying trees . . . . . 129
5.2.31 Renaming a relationship . . . . . 134
5.2.32 Deleting the primary relationship to a node . . . . . 138
5.2.33 Deleting the primary relationships of a tree . . . . . 139
5.2.34 Creating secondary relationships . . . . . 141
5.2.35 Creating bidirectional secondary relationships . . . . . 143
5.2.36 Deleting secondary relationships . . . . . 146
5.2.37 Obtaining a corresponding reverse relationship’s relation . . . . . 148
5.2.38 Obtaining a corresponding reverse relationship’s key . . . . . 150
5.2.39 Setting the inheritance property of a relationship . . . . . 151
5.2.40 Determining if a secondary relationship is inheritable . . . . . 153



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

5.2.41 Setting the current node relationship . . . 154
5.2.42 Opening a node handle to the current node . . . 155
5.2.43 Determining the creation time of a node . . . 155
5.2.44 Determining the last time a relationship was modified . . . 156
5.2.45 Determining the last time that node contents were written . . . 157
5.2.46 Determining the last time an attribute was modified . . . 157
5.2.47 Iteration predicate subtype . . . 158
5.2.48 Obtaining the default iteration predicate . . . 168
5.2.49 Node iteration types and subtypes . . . 169
5.2.50 Creating an iterator over nodes . . . 169
5.2.51 Determining node iteration status . . . 173
5.2.52 Determining the approximate size of the node iterator . . . 174
5.2.53 Getting the next node in a node iteration . . . 175
5.2.54 Skipping the next node in a node iteration . . . 176
5.2.55 Obtaining the path element for the next node in a node iteration . . . 177
5.2.56 Deleting anode iterator. . . . 178
5.2.57 Node kind instance iteration type . . . 178
5.2.58 Creating an iterator over instances of a node kind definition . . . 178
5.2.59 Determining node kind instance iteration status . . . 180
5.2.60 Determining the approximate size of the node kind instance iterator . . . 181
5.2.61 Getting the next node in a node kind instance iteration . . . 181
5.2.62 Skipping the next node in a node kind instance iteration . . . 183
5.2.63 Deleting a node kind instance iterator . . . 184
5.3 Package CAIS_ATTRIBUTE_MANAGEMENT . . . 185
5.3.1 Creating node attributes. . . . 186
5.3.2 Creating relationships attributes . . . 187
5.3.3 Deleting node attributes. . . . 189
5.3.4 Deleting relationship attributes . . . 190
5.3.5 Setting node attributes. . . . 192
5.3.6 Setting relationships attributes . . . 193
5.3.7 Getting node attributes. . . . 195
5.3.8 Getting relationship attributes . . . 196
5.3.9 Generic package CAIS_INTEGER_ATTRIBUTE . . . 198
5.3.10 Generic package CAIS_FLOAT_ATTRIBUTE . . . 207
5.3.11 Package CAIS_STRING_ATTRIBUTE . . . 217
5.3.12 Package CAIS_IDENTIFIER_ATTRIBUTE . . . 226
5.3.13 Generic package CAIS_ENUMERATION_ATTRIBUTE . . . 235
5.3.14 Package CAIS_BOOLEAN_ATTRIBUTE . . . 245
5.3.15 Attribute iteration types and subtypes . . . 245
5.3.16 Creating an iterator over node attributes . . . 246
5.3.17 Creating an iterator over relationship attributes . . . 247
5.3.18 Determining iteration status . . . 250
5.3.19 Determining the approximate size of the iterator . . . 250
5.3.20 Getting an identification for the next attribute . . . 251
5.3.21 Getting the next attribute value . . . 252
5.3.22 Skipping the next attribute in an iteration . . . 253
5.3.23 Deleting an attribute iterator . . . 253
5.4 Package CAIS_ACCESS_CONTROL_MANAGEMENT . . . 255



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

5.4.1 Node access . . . 255
5.4.2 Security labeling of CAIS nodes . . . 255
5.4.3 Groups and roles . . . 257
5.4.4 Granting access rights . . . 259
5.4.5 Denying access . . . 262
5.4.6 Access checking . . . 262
5.4.7 An example of CAIS access control . . . 263
5.4.8 Subtypes . . . 271
5.4.9 Value of all access rights . . . 271
5.4.10 Setting access control . . . 271
5.4.11 Deleting ACCESS relationships . . . 273
5.4.12 Obtaining the value of a GRANT attribute . . . 274
5.4.13 Denying access . . . 275
5.4.14 Deleting DENY relationships . . . 275
5.4.15 Examining access rights . . . 276
5.4.16 Adopting a rose . . . 277
5.4.17 Unlinking an adopted role . . . 278
5.5 Package CAIS_STRUCTURAL_NODE_MANAGEMENT . . . 280
5.5.1 Opening the predefine STRUCTURAL NKD . . . 280
5.5.2 Creating structural nodes . . . 280
5.6 Package CAIS_TRANSLATION_MANAGEMENT . . . 284
5.6.1 Subtypes . . . 284
5.6.2 Adopting a view . . . 284
5.6.3 Canceling a view adoption . . . 285
5.6.4 Adopting an NKI . . . 286
5.6.5 Cancelling an NKI adoption . . . 288
5.6.6 Opening a handle to an NKD by local name . . . 289
5.6.7 Opening a handle to an NKI by local name . . . 290
5.6.8 Opening a handle to an RD by local name . . . 291
5.6.9 Opening a handle to an RI by local name . . . 291
5.6.10 Opening a handle to an AD by local name . . . 292
5.6.11 Opening a handle to an AKD by local name . . . 293
5.6.12 Determining the name for a definition . . . 294
5.6.13 Determining the NKD for a node . . . 295
5.6.14 Determining the name for a relation . . . 296
5.6.15 Determining the RD for a relation . . . 297
5.6.16 Determining the name for a node attribute . . . 299
5.6.17 Determining the AD for a node attribute . . . 300
5.6.18 Determining the name for a relationship attribute . . . 301
5.6.19 Determining the AD for a relationship attribute . . . 302
5.7 Package CAIS_DEFINITION_MANAGEMENT . . . 304
5.7.1 Definition nodes . . . 304
5.7.2 Predefined entities . . . 342
5.7.3 Views . . . 347
5.7.4 Types and subtypes . . . 358
5.7.5 Opening the predefine VIEW NKD . . . 359
5.7.6 Opening the predefine NKI NKD . . . 359
5.7.7 Opening the predefine NKD NKD . . . 360



Source: https://assist.dla.mil -- Downloaded: 202610-26T07:54Z
Check the source to verify that this is the current version before use.

5.7.8 Opening the predefined RI NKD . . . . . 360
5.7.9 Opening the predefined RD NKD . . . . . 361
5.7.10 Opening the predefined AD NKD . . . . . 361
5.7.11 Opening the predefine named composite list AKD NKD . . . . . 362
5.7.12 Opening the predefine identifier AKD NKD . . . . . 362
5.7.13 Creating VIEW nodes.. . . . . 363
5.7.14 Deleting the primary relationships of a tree rooted at a VIEW node . . . 366
5.7.15 Creating Nods . . . . . 367
5.7.16 Deleting NKIs and NKDs . . . . . 372
5.7.17 Creating Skis . . . . . 373
5.7.18 Renaming NODE-KIND relationships in a view . . . . . 377
5.7.19 Specializing a node kind . . . . . 378
5.7.20 Generalizing node kinds. . . . . 380
5.7.21 Deleting SPECIALIZATION_OF relationships from NKDs . . . . . 382
5.7.22 Creating RDs . . . . . 384
5.7.23 Deleting RIs and RDs . . . . . 389
5.7.24 Creating Rib . . . . . 391
5.7.25 Renaming RELATION relationships in a view . . . . . 394
5.7.26 Specializing a relation . . . . . 395
5.7.27 Generalizing relations . . . . . 397
5.7.28 Deleting SPECIALIZATION_OF relationships from RDs . . . . . 400
5.7.29 Creating bidirectional RDs . . . . . 402
5.7.30 Deleting bidirectional RDs . . . . . 410
5.7.31 Creating Ads . . . . . 412
5.7.32 Deleting Ads . . . . . 415
5.7.33 Importing Ads . . . . . 417
5.7.34 Renaming ATTRIBUTE relationships in a view . . . . . 418
5.7.35 Redefining the AKD for an AD . . . . . 419
5.7.36 Creating Adds . . . . . 421
5.7.37 Creating identifier AKDs . . . . . 425
5.7.38 Deleting Adds . . . . . 428
5.7.39 Importing AKDs . . . . . 430
5.7.40 Renaming ATTRIBUTE_KIND relationships in a view . . . . . 431
5.7.41 Deleting the primary relationships of a tree rooted at a definition node . 432
5.7.42 Specifying source node kinds for a relation . . . . . 434
5.7.43 Deleting source node kinds for a relation . . . . . 436
5.7.44 Respecifying source node kinds for a relation . . . . . 438
5.7.45 Specifying key attributes for a relation . . . . . 439
5.7.46 Deleting key attributes for a relation . . . . . 442
5.7.47 Respecifying key attributes for a relation . . . . . 443
5.7.48 Specifying target node kinds for a relation . . . . . 445
5.7.49 Deleting target node kinds for a relation . . . . . 447
5.7.50 Specifying attributes for node kinds and relations . . . . . 448
5.7.51 Deleting attributes for node kinds and relations . . . . . 450
5.7.52 Respecifying attributes for node kinds and relations . . . . . 452
5.7.53 Specifying an attribute kind component . . . . . 453
5.7.54 Deleting an attribute kind component . . . . . 455
5.7.55 Specifying a variant attribute alternative . . . . . 457



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

5.7.56 Deleting a variant attribute alternative . . . 458
5.7.57 Setting definition node attributes . . . 460
5.7.58 Deleting definition node attributes . . . 462
5.7.59 Setting definition relationship attributes . . . 464
5.7.60 Delete definition relationship attributes . . . 467
5.7.61 Changing the NKD for a node . . . 469
5.7.62 Checking conformance of a node to an NKD . . . 471
5.7.63 Checking conformance of a relationship to an RD . . . 472
5.7.64 Checking conformance of a list to an AD or AKD . . . 475
5.8 Package CAIS_ATTRIBUTE_MONITOR_MANAGEMENT . . . 477
5.8.1 Types . . . 481
5.8.2 Opening the predefined MONITOR NKD . . . 481
5.8.3 Creating MONITOR nodes . . . 482
5.8.4 Deleting the primary relationship to a MONITOR node . . . 489
5.8.5 Associating an event handler process with a MONITOR node . . . 490
5.8.6 Specifying that a node attribute is to be monitored . . . 491
5.8.7 Specifying that a path attribute is to be monitored . . . 494
5.8.8 Canceling monitoring of designated attributes . . . 496
5.8.9 Determining a monitored attribute modification . . . 497
5.8.10 Awaiting a monitored attribute modification . . . 499
5.9 Package CAIS_RESOURCE_CONTROL . . . 503
5.9.1 Reassigning a node to a new medium resource . . . 503
5.9.2 Determining the medium resource to which a node is assigned . . . 504
5.9.3 Determining the processor resource to which a process is assigned . . . 505
5.9.4 Determining the current default medium resource . . . 506
5.9.5 Determining the current default processor resource . . . 506

**6 CAIS PROCESS MANAGEMENT DETAILED REQUIREMENTS 509**
6.1 Transactions . . . 514
6.1.1 Subtransactions and enclosing transactions . . . 514
6.1.2 Opening transactions . . . 514
6.1.3 Inclusion of nodes . . . 515
6.1.4 Inclusion of operations . . . 515
6.1.5 Visibility of effects . . . 516
6.1.6 Closing Transactions . . . 517
6.1.7 Enclosing processes in transactions . . . 518
6.1.8 Eligible effects of operations . . . 518
6.2 Package CAIS_PROCESS_DEFINITIONS . . . 520
6.3 Package CAIS_PROCESS_MANAGEMENT . . . 521
6.3.1 Determining the default priority . . . 524
6.3.2 Creating a dependent PROCESS node . . . 524
6.3.3 Creating a root PROCESS node . . . 530
6.3.4 Activating a process . . . 535
6.3.5 Awaiting termination of another process . . . 536
6.3.6 Activating a process and awaiting its termination . . . 537
6.3.7 Spawning a new process . . . 538
6.3.8 Invoking a new process . . . 540
6.3.9 Creating and activating a new job . . . 543



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

6.3.10 Deleting a job . . . 545
6.3.11 Appending results . . . 546
6.3.12 Overwriting results . . . 547
6.3.13 Getting results from a process . . . 547
6.3.14 Determining the status of a process . . . 548
6.3.15 Getting the parameter list . . . 549
6.3.16 Aborting a process . . . 550
6.3.17 Suspending a process . . . 551
6.3.18 Resurging a process . . . 552
6.3.19 Determining the lowest specifiable priority . . . 553
6.3.20 Determining the highest specifiable priority . . . 553
6.3.21 Determining if the current priority is modifiable . . . 554
6.3.22 Modifying the current priority . . . 554
6.3.23 Determining the number of open node handles . . . 555
6.3.24 Determining the number of input and output units used . . . 556
6.3.25 Determining the time of activation . . . 556
6.3.26 Determining the time of termination . . . 557
6.3.27 Determining the time a process has been active . . . 558
6.3.28 Determining the size of a process . . . 558
6.4 Package CAIS_TRANSACTION_MANAGEMENT . . . 560
6.4.1 Opening a transaction . . . 560
6.4.2 Including anode in a transaction . . . 561
6.4.3 Including a tree in a transaction . . . 562
6.4.4 Closing a transaction . . . 563
6.4.5 Examining the open status of a transaction . . . 564
6.4.6 Querying sameness for two transactions . . . 565
6.4.7 Determining whether a node is included in a transaction . . . 565

**7 INPUT AND OUTPUT DETAILED REQUIREMENTS 567**
7.1 Package CAIS_IO_DEFINITIONS . . . 569
7.2 Package CAIS_FILE_NODE_MANAGEMENT . . . 571
7.2.1 Opening the predefine SECONDARY-STORAGE NKD . . . 571
7.2.2 Creating secondary storage nodes . . . 571
7.3 Package CAIS_IO_CONNECTION . . . 575
7.3.1 Channel representation details . . . 575
7.3.2 Creating and deleting channels . . . 578
7.3.3 Opening protocols . . . 578
7.3.4 Compatibility rules . . . 578
7.3.5 Detailed protocol opening scenarios . . . 582
7.3.6 Discontinuing communication. . . . 587
7.3.7 Opening the predefined PORT RD . . . 589
7.3.8 Creating a channel . . . 589
7.3.9 Releasing a channel . . . 593
7.3.10 Deleting a CHANNEL node . . . 594
7.3.11 Waiting for notification of attempted opening . . . 595
7.3.12 Opening the opposite client node . . . 597
7.3.13 Checking PORT relationship kind compatibility . . . 598
7.3.14 Setting PORT relationship attributes . . . 599



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

7.4 Package CAIS_IMPORT_EXPORT . . . . . 602
7.4.1 Importing a file from the host file system . . . . . 602
7.4.2 Exporting a file to the host file system . . . . . 603
7.5 Package CAIS_COMMON_IO . . . . . 604
7.5.1 Types and exceptions . . . . . 604
7.5.2 Opening the predefine INTRODUCED-NODE NKD . . . . . 604
7.5.3 Opening the predefine INTRODUCED_NODE_RD . . . . . 605
7.5.4 Opening anode set handle . . . . . 605
7.5.5 Including anode in a node set . . . . . 606
7.5.6 Including a tree in a node set . . . . . 607
7.5.7 Including a structure of nodes in a node set . . . . . 609
7.5.8 Transmitting a node set in common external form . . . . . 610
7.5.9 Closing a node set . . . . . 611
7.5.10 Introducing nodes in common external form . . . . . 612
7.5.11 Introducing contents of introduced nodes . . . . . 616
7.5.12 Creating VIEW nodes with specific UIDs . . . . . 618
7.5.13 Creating NKDs with specific UIDs . . . . . 619
7.5.14 Creating NKIs with specific UIDs . . . . . 620
7.5.15 Creating RDs with specific UIDs . . . . . 621
7.5.16 Creating RIs with specific UIDs . . . . . 622
7.5.17 Creating bidirectional RDs with specific UIDs . . . . . 623
7.5.18 Creating ADs with specific UIDs . . . . . 625
7.5.19 Creating AKDs with specific UIDs . . . . . 626
7.5.20 Creating structural nodes with specific UIDs . . . . . 627
7.5.21 Creating a secondary storage node with a specific UID . . . . . 628

**8 DATA TRANSFER PROTOCOLS DETAILED REQUIREMENTS 629**
8.1 Input and Output Detailed Requirements . . . . . 629
8.2 Package CAIS_FRAME_IO . . . . . 630
8.2.1 Protocol parameters . . . . . 631
8.2.2 Types, subtypes, and exceptions . . . . . 632
8.2.3 Opening a frame file handle . . . . . 633
8.2.4 Closing a frame file handle . . . . . 634
8.2,5 Determining whether a frame file handle is open . . . . . 635
8.2.6 Determining the mode of a frame file handle . . . . . 635
8.2.7 Transmitting a flame . . . . . 636
8.2.8 Receiving a flame . . . . . 637
8.2.9 Resetting a frame file handle . . . . . 638
8.2.10 Determining if all messages have been read . . . . . 639
8.2.11 Determining if incoming queue is empty . . . . . 640
8.2.12 Determining if outgoing queue is full . . . . . 641
8.2.13 Determining the channel node associated with a file . . . . . 641
8.3 Package CAIS_MAGNETIC_TAPE_IO . . . . . 643
8.3.1 Protocol parameters . . . . . 643
8.3.2 Types, subtypes, and exceptions . . . . . 644
8.3.3 Opening a tape drive file handle . . . . . 646
8.3.4 Closing a tape drive file handle . . . . . 647
8.3.5 Determining whether a file handle is open . . . . . 648



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

8.3.6 Determining the mode of a magnetic tape drive . . . 649
8.3.7 Requesting the mounting of a tape . . . 649
8.3.8 Loading a gape . . . 650
8.3.9 Unloading a gape . . . 651
8.3.10 Requesting the dismounting of a tape . . . 651
8.3.11 Determining the position of the tape . . . 652
8.3.12 Rewinding the tape . . . 652
8.3.13 Skipping tape marks . . . 653
8.3.14 Writing a tape mark . . . 654
8.3.15 Determining the status of a magnetic tape drive . . . 654
8.3.16 Determining the recording method of a magnetic tape . . . 655
8.3.17 Determining whether a write ring is installed . . . 655
8.3.18 Skipping blocks in a magnetic tape file . . . 656
8.3.19 Reading a block from a magnetic tape file . . . 657
8.3.20 Writing a block to a magnetic tape file . . . 658
8.3.21 Resetting a magnetic tape file handle . . . 659
8.3.22 Determining the channel node associated with a file . . . 659
8.4 Package CAIS_DIRECT_IO . . . 661
8.4.1 Protocol parameters . . . 661
8.4.2 Definition of types . . . 662
8.4.3 Creating a direct file . . . 662
8.4.4 Opening a direct file handle . . . 667
8.4.5 Closing a direct file handle . . . 669
8.4.6 Resetting a direct file handle . . . 669
8.4.7 Synchronizing internal file with FILE node contents . . . 670
8.4.8 Determining the channel node associated with a file . . . 671
8.5 Package CAIS_SEQUENTIAL_IO . . . 672
8.5.1 Protocol parameters . . . 672
8.5.2 Definition of types . . . 673
8.5.3 Creating a sequential file . . . 673
8.5.4 Opening a sequential file handle . . . 678
8.5.5 Closing a sequential file handle . . . 680
8.5.6 Resetting a sequential file handle . . . 680
8.5.7 Synchronizing internal file with FILE node contents . . . 682
8.5.8 Determining the channel node associated with a file . . . 682
8.6 Package CAIS_TEXT_IO . . . 684
8.6.1 Protocol parameters . . . 685
8.6.2 Definition of types . . . 685
8.6.3 Creating a text file . . . 685
8.6.4 Opening a text file handle . . . 690
8.6.5 Closing a text file handle . . . 692
8.6.6 Resetting a text file handle . . . 693
8.6.7 Synchronizing internal file with FILE node contents . . . 694
8.6.8 Determining the channel node associated with a file . . . 695
8.6.9 Setting the maximum line length of a text file handle . . . 695
8.6.10 Setting the maximum page length of a text file handle . . . 696
8.7 Package CAIS_TERMINAL_IO . . . 698
8.7.1 Protocol parameters . . . 701



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

8.7.2 Types, Subtypes, Constants and Exceptions . . . 701
8.7.3 Opening a terminal file handle . . . 704
8.7.4 Closing a terminal file handle . . . 706
8.7.5 Determining whether a terminal file handle is open . . . 707
8.7.6 Determining the channel node associated with a terminal file handle . . . 707
8.7.7 Determining the physical display origin on the virtual display . . . 708
8.7.8 Re-orienting the physical display on the virtual display . . . 709
8.7.9 Determining if a terminal supports a set of graphic renditions . . . 709
8.7.10 Determining if a terminal supports a foreground color . . . 710
8.7.11 Determining if a terminal supports a background color . . . 710
8.7.12 Starting output buffering for a virtual display . . . 711
8.7.13 Ending output buffering for a virtual display . . . 711
8.7.14 Refreshing a virtual display . . . 712
8.7.15 Creating a screen . . . 712
8.7.16 Changing the active rendition for a screen . . . 713
8.7.17 Changing screen size . . . 714
8.7.18 Changing border characteristics . . . 714
8.7.19 Creating a copy of an existing screen . . . 715
8.7.20 Deleting a screen . . . 716
8.7.21 Determining the size of a screen . . . 716
8.7.22 Determining the active rendition of a screen . . . 716
8.7.23 Determining whether a screen has a border . . . 717
8.7.24 Determining the border rendition of a screen . . . 717
8.7.25 Querying sameness of screens . . . 718
8.7.26 Pasting a screen onto the virtual display . . . 719
8.7.27 Removing a screen from a virtual display . . . 719
8.7.28 Relocating a screen on a virtual display . . . 720
8.7.29 Move screen to front . . . 720
8.7.30 Move screen to back . . . 721
8.7.31 Determining paste position of a screen . . . 721
8.7.32 Determining whether a screen is occluded . . . 722
8.7.33 Moving the cursor position on a screen . . . 722
8.7.34 Determining the position of a cursor in a screen . . . 723
8.7.35 Writing a character to a screen . . . 723
8.7.36 Advancing the cursor to the next line . . . 724
8.7.37 Writing a line of text to a screen . . . 725
8.7.38 Inserting a character into a line . . . 725
8.7.39 Deleting characters from a line . . . 726
8.7.40 Inserting blank lines into a screen . . . 727
8.7.41 Deleting lines from a screen . . . 727
8.7.42 Erasing a region of a screen . . . 728
8.7.43 Changing the rendition for a region of a screen . . . 729
8.7.44 Determining a cell rendition . . . 729
8.7.45 Reading characters from a screen . . . 730
8.7.46 Sounding the terminal bell . . . 731
8.7.47 Starting output buffering for a screen . . . 731
8.7.48 Ending output buffering to a screen . . . 732
8.7.49 Enabling function key usage . . . 732



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

8.7.50 Determining function key usage . . . 733
8.7.51 Determining the number of function keys . . . 733
8.7.52 Determining the identification of a function key . . . 734
8.7.53 Enabling and disabling the locator . . . 734
8.7.54 Determining locator usage . . . 735
8.7.55 Determining the locator cursor position . . . 735
8.7.56 Determining the screen containing the locator cursor . . . 736
8.7.57 Determining locator cursor position relative to screen border . . . 737
8.7.58 Moving the locator cursor . . . 737
8.7.59 Determining the number of locator buttons . . . 738
8.7.60 Determining the identification of a locator button . . . 738
8.7.61 Reading a single input event . . . 739
8.7.62 Reading a string from a virtual keyboard . . . 740
8.7.63 Flushing the input event queue . . . 742
8.7.64 Determining if all input events have been read . . . 742
8.7.65 Determining if the input event queue is empty . . . 743

**9 CAIS LIST MANAGEMENT DETAILED REQUIREMENTS 745**
9.1 Package CAIS_LIST_MANAGEMENT . . . 751
9.1.1 Types, subtypes, constants and exceptions . . . 751
9.1.2 Copying a list . . . 752
9.1.3 Making a list empty . . . 753
9.1.4 Converting from text to list form . . . 753
9.1.5 Converting a list to its text representation . . . 754
9.1.6 Determining the equality of two lists . . . 755
9.1.7 Deleting an item from a linear list . . . 755
9.1.8 Determining the kind of list . . . 756
9.1.9 Determining the kind of list item . . . 757
9.1.10 Inserting a sequence of items into a linear list . . . 758
9.1.11 Concatenating two linear lists . . . 759
9.1.12 Extracting a sequence of items from a linear list . . . 759
9.1.13 Determining the length of a linear list . . . 760
9.1.14 Determining the position of the current linear list . . . 761
9.1.15 Determining whether the current linear list is outermost . . . 761
9.1.16 Making the next outer linear list current . . . 761
9.1.17 Making a nested sublist the current linear list . . . 762
9.1.18 Determining the length of the text form of a list or a list item . . . 763
9.1.19 Determining the name of a named item . . . 764
9.1.20 Determining the position of a named item . . . 765
9.1.21 Package CAIS_LIST_ITEM . . . 766
9.1.22 Package CAIS_IDENTIFIER_ITEM . . . 771
9.1.23 Generic package CAIS_ENUMERATION_ITEM . . . 780
9.1.24 Package CAIS_BOOLEAN_ITEM . . . 786
9.1.25 Generic package CAIS_INTEGER_ITEM . . . 787
9.1.26 Generic package CAIS_FLOAT_ITEM . . . 792
9.1.27 Package CAIS_STRING_ITEM . . . 798



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

**10 CAIS STANDARD DETAILED REQUIREMENTS** **805**
10.1 Package CAIS_STANDARD . . . . . 805

**11 CAIS CALENDAR DETAILED REQUIREMENTS** **807**
11.1 Package CAIS_CALENDAR . . . . . 807
11.1.1 Types, subtypes, and exceptions . . . . . 807
11.1.2 Getting the current time . . . . . 808
11.1.3 Getting the year part of the time . . . . . 808
11.1.4 Getting the month part of the time . . . . . 808
11.1.5 Getting the day part of the time . . . . . 809
11.1.6 Getting the seconds part of the time . . . . . 809
11.1.7 Splitting time into its components . . . . . 810
11.1.8 Combining components of time . . . . . 811
11.1.9 Adding time and duration . . . . . 811
11.1.10 Subtracting time and duration . . . . . 812
11.1.11 Comparing two values of time . . . . . 812

**12 CAIS PRAGMATIC DETAILED REQUIREMENTS** **816**
12.1 Package CAIS_PRAGMATICS . . . . . 815
12.1.1 Identifier and view pragmatics . . . . . 815
12.1.2 Node model pragmatics . . . . . 816
12.1.3 Handle and iterator pragmatics . . . . . 817
12.1.4 Integer and real number pragmatic . . . . . 819
12.1.5 Access control pragmatics . . . . . 819
12.1.6 File and device pragmatics . . . . . 820
12.1.7 List pragmatics . . . . . 822

**13 CAIS STATUS MANAGEMENT DETAILED REQUIREMENTS** **825**
13.1 Package CAIS_STATUS_DEFINITIONS . . . . . 845
13.2 Package CAIS_STATUS_MANAGEMENT . . . . . 857
13.2.1 Creating a status indicator . . . . . 857
13.2.2 Setting a status indicator . . . . . 857
13.2.3 Copying a status indicator . . . . . 858
13.2.4 Deleting a status indicator . . . . . 858
13.2.5 Obtaining a status indicator’s exception indication . . . . . 859
13.2.6 Obtaining status information from a status indicator . . . . . 859
13.2.7 Obtaining a descriptive message from a status indicator . . . . . 860
13.2.8 Determining if a status indicator is established . . . . . 860
13.3 Exception Conditions.. . . . . 862
13.3.1 Conditions for CAIS_DEFINITIONS exceptions . . . . . 862
13.3.2 Conditions for CAIS_PROCESS_DEFINITIONS exceptions . . . . . 908
13.3.3 Conditions for CAIS_10_DEFINITIONS exceptions . . . . . 908
13.3.4 Conditions for CAIS_FRAME-IO exceptions . . . . . 914
13.3.5 Conditions for CAIS_MAGNETIC_TAPE_IO exceptions . . . . . 915
13.3.6 Conditions for CAIS_TERMINAL_IO exceptions . . . . . 915
13.3.7 Conditions for CAIS_COMMON_IO exceptions . . . . . 918
13.3.8 Conditions for CAIS_LIST_MANAGEMENT exceptions . . . . . 918
13.3.9 Conditions for CAIS_CALENDAR exceptions . . . . . 924
13.3.10 Conditions for CAIS_PRAGMATICS exceptions . . . . . 925



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

13.3.11 Conditions for CAIS_FORM_TERMINAL_IO exceptions . . . . . 926

**14 GUIDANCE INFORMATION** **927**

14.1 Keywords . . . . . 927



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

# APPENDIXES

**Appendix A Predefined Entities** **929**
10 Predefined node description index . . . 929
20 Predefined Node Descriptions . . . 940
20.1 Top-level nodes . . . 943
20.2 Attribute definition nodes (Ads) . . . 960
20.3 Attribute kind definition nodes (AKDs) . . . 1000
20.4 Node kind definition nodes (NKDs) . . . 1027
20.5 Relation definition nodes (RDs) . . . 1050
20.6 Node kind interpretation nodes (NKIs) . . . 1114
20.7 Relation interpretation nodes (RIs) . . . 1130

**B CAIS Specification** **1167**

**C Compatibility with Pre-existing Standards** **1321**
10 Package CAIS_NODE_MANAGEMENT . . . 1321
10.1 Opening a node handle . . . 1321
10.2 Changing the intent regarding node handle usage . . . 1322
10.3 Querying the existence of anode . . . 1322
10.4 Querying sameness . . . 1322
10.5 Obtaining an open node handle to the parent . . . 1323
10.6 Copying a node . . . 1323
10.7 Copying trees . . . 1324
10.8 Renaming the primary relationship of a node . . . 1324
10.9 Deleting the primary relationship to a node . . . 1324
10.10 Deleting the primary relationships of a tree . . . 1325
10.11 Creating secondary relationships . . . 1325
10.12 Deleting secondary relationships . . . 1326
10.13 Setting inheritance property of a relationship . . . 1326
10.14 Determining if a secondary relationship is inheritable . . . 1326
10.15 Setting the current node relationship . . . 1327
10.16 Opening a node handle to the current node . . . 1327
10.17 Determining the creation time of a node . . . 1327
10.18 Determining the last time a relationship was modified . . . 1328
10.19 Determining the last time that node contents were written . . . 1328
10.20 Determining the last time an attribute was modified . . . 1328
10.21 Creating an iterator over nodes . . . 1329
10.22 Getting the next node in a node iteration . . . 1329
20 Package CAIS_ATTRIBUTE_MANAGEMENT . . . 1330
20.1 Creating node attributes . . . 1330
20.2 Creating path attributes . . . 1330
20.3 Deleting node attributes . . . 1330
20.4 Deleting path attributes . . . 1331
20.5 Setting node attributes . . . 1331
20.6 Setting path attributes . . . 1331
20.7 Getting node attributes . . . 1332



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

20.8 Getting path attributes . . . 1332
20.9 Creating an iterator over node attributes . . . 1333
20.10 Creating an iterator over relationship attributes . . . 1333
30 Package CAIS_ACCESS_CONTROL_MANAGEMENT . . . 1334
30.1 Subtypes . . . 1334
30.2 Setting access control . . . 1334
30.3 Deleting access relationships . . . 1334
30.4 Obtaining the value of a GRANT attribute . . . 1335
30.5 Examining access rights . . . 1335
30.6 Adopting a rose . . . 1335
40 Package CAIS_STRUCTURAL_NODE_MANAGEMENT . . . 1336
40.1 Creating structural nodes . . . 1336
50 Package CAIS_PROCESS_MANAGEMENT . . . 1337
50.1 Spawning a new process . . . 1337
50.2 Invoking a new process . . . 1338
50.3 Awaiting termination of another process . . . 1340
50.4 Deleting a job . . . 1340
50.5 Getting results from a process . . . 1340
50.6 Determining the status of a process . . . 1341
50.7 Aborting a process . . . 1341
50.8 Suspending a process . . . 1342
50.9 Resuming a process . . . 1342
50.10 Determining the number of open node handles . . . 1343
50.11 Determining the number of input and output units used . . . 1343
50.12 Determining the time of activation . . . 1344
50.13 Determining the time of termination . . . 1344
50.14 Determining the time a process has been active . . . 1344
50.15 Determining the size of a process . . . 1345
60 Package CAIS_IMPORT_EXPORT . . . 1345
60.1 Importing a file from the host file system . . . 1345
60.2 Exporting a file to the host file system . . . 1345
70 Package CAIS_FORM_TERMINAL_IO . . . 1346
70.1 Protocol parameters . . . 1347
70.2 Types, subtypes, and exceptions . . . 1347
70.3 Opening a form terminal file handle . . . 1348
70.4 Closing a form terminal file handle . . . 1350
70.5 Determining whether a file handle is open . . . 1350
70.6 Determining the channel node associated with a file . . . 1351
70.7 Determining the number of function keys . . . 1351
70.8 Determining intercepted input characters . . . 1352
70.9 Determining intercepted output characters . . . 1352
70.10 Creating a form . . . 1353
70.11 Deleting a form . . . 1354
70.12 Copying a form . . . 1354
70.13 Defining a qualified area . . . 1355
70.14 Removing an area qualifier . . . 1355
70.15 Changing the active position . . . 1356
70.16 Querying the active position . . . 1357



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

70.17 Advancing forward to quatified area . . . 1357
70.18 Writing to a form . . . 1358
70.19 Erasing a qualified area . . . 1359
70.20 Erasing a form . . . 1359
70.21 Activating a form on a terminal . . . 1360
70.22 Reading from a form . . . 1360
70.23 Determining changes to a form . . . 1361
70.24 Determining the identification of a function key . . . 1362
70.25 Determining the termination key . . . 1362
70.26 Determining the size of a form . . . 1363
70.27 Determining the size of the terminal . . . 1363
70.28 Determining if the area qualifier requires space in the form . . . 1364
70.29 Determining if the area qualifier requires space on a terminal . . . 1364
80 Package CAIS_PRAGMATICS . . . 1365

**D Syntax Summary 1367**
10 Pathname Syntax . . . 1368
20 GRANT Attribute Value Syntax . . . 1368
30 Classification Attribute Value Syntax . . . 1368
40 List External Representation Syntax . . . 1369
50 Iteration Predicate Syntax . . . 1370
60 External Node Set Syntax. . . . 1371
70 ATTRIBUTES. MONITORED Attribute Value Syntax . . . 1372
80 Constraints Attribute Value Syntax . . . 1373
90 Status Information List Syntax . . . 1374

**E Discretionary Access Control Conformance Guidelines 1381**
10 Class C1 . . . 1381
10.1 Capability a: user control of objects . . . 1381
10.2 Capability b: user ability to specify access modes . . . 1381
10.3 Capability c: user ability to specify individuals or groups . . . 1382
20 Class C2 . . . 1382
20.1 Capability a: user ability to control objects . . . 1383
20.2 Capability b: user ability to specify and control access modes . . . 1383
20.3 Capability c: user ability to specify individual users or groups of individual users . . . 1383
20.4 Capability d: user ability to include or exclude object access to the granularity of an individual . . . 1383
20.5 Capability e: assignment of access control by an authorized user . . . 1384
30 Class B3 . . . 1384
30.1 Capability a: user ability to control objects . . . 1385
30.2 Capability b: user ability to specify and control access modes . . . 1385
30.3 Capability c: access control list . . . 1385
30.4 Capability d: user ability to include or exclude object access to the granularity of an individual . . . 1385
30.5 Capability e: access permission to an object must be assigned by an authorized user . . . 1385
30.6 Capability f: ability to deny access to individuals or groups . . . 1385



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

**F Implementation Dependencies** **1387**

**G List Of Interfaces** **1389**

**Index** **1407**



Source: https://assist.dla.mil -- Downloaded: 202610-26T07:54Z
Check the source to verify that this is the current version before use.

# FIGURES


| | | | |
|---|---|---|---|
| **Figure** | 1 | CAIS system concept | 45 |
| | 2 | Tree formed by primary relationships | 47 |
| | 3 | Some predefined relations | 49 |
| | 4 | Example node kind definition and attribute definition | 56 |
| | 5 | Primary relationships to definition nodes | 57 |
| | 6 | VIEW node and emanating relationships | 60 |
| | 7 | Channels | 65 |
| | 8 | Multiple channels | 66 |
| | 9 | Channel state transition | 68 |
| | 10 | External node set | 72 |
| | 11 | Resource nodes example | 78 |
| | 12 | Resource node groups example | 79 |
| | 13 | Allocatability relationship example | 81 |
| | 14 | Resource visibility | 83 |
| | 15 | Remote interprocess communication | 86 |
| | 16 | COPY_ TREE example | 131 |
| | 17 | Discretionary access control | 264 |
| | 18 | Discretionary access control after process creation | 265 |
| | 19 | ACCESS relationships, Example 1 | 267 |
| | 20 | ACCESS relationships, Example 2 | 269 |
| | 21 | ACCESS relationships, Example 3 | 270 |
| | 22 | Inheritance of properties and constraints | 305 |
| | 23 | Example of an NKD lattice | 307 |
| | 24 | Node kind definition node (NOD) | 308 |
| | 25 | NKD and RD property representation | 309 |
| | 26 | Relation definition node (RD) | 311 |
| | 27 | Bidirectional RD | 312 |
| | 28 | Basic NKD and RD | 314 |
| | 29 | Attribute definition node (AD) and attribute kind definition node (AKD) | 315 |
| | 30 | Simple and variant Adds | 321 |
| | 31 | Example of a variant AKD | 323 |
| | 32 | List, uniform list and composite fist AKDs | 325 |
| | 33 | GRANT attribute kind representation | 326 |
| | 34 | TIME attribute kind representation | 327 |
| | 35 | Node kind interpretation node (NKI) | 347 |
| | 36 | Local names defined by an NKI | 348 |
| | 37 | Relation interpretation node (RI) | 349 |
| | 38 | Local names defined by an RI | 350 |
| | 39 | Adoption of views by a root process | 352 |
| | 40 | Adoption of views by a dependent process | 353 |
| | 41 | Allowable process status transitions | 511 |
| | 42 | Full channel representation | 575 |
| | 43 | Frame IO message queues | 630 |
| | 44 | Magnetic tape status transitions | 645 |




Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

45 Virtual display with screens . . . . . 699



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

# TABLES


| | | | |
|---|---|---|---|
| **Table** | I | Pathname BNF | 52 |
| | II | External Node Set BNF | 73 |
| | III | Resource Model Elements | 77 |
| | IV | Intents | 98 |
| | V | Matrix of Access Synchronization Constraints | 102 |
| | VI | Iteration Predicate BNF | 160 |
| | VII | Logical Operators in Iteration Predicates | 165 |
| | VIII | Applications of Attribute Functions | 165 |
| | IX | Classification Attribute Value BNF | 256 |
| | X | Predefined access rights | 260 |
| | XI | GRANT Attribute Value BNF | 262 |
| | XII | Predefined AKD Attributes | 319 |
| | XIII | Predefined Emanating Relations for AKDs | 320 |
| | XIV | Predefine Attributes of Relations Emanating from AKDs | 320 |
| | XV | Matrix of Composite List AKDs | 324 |
| | XVI | Predefine Relations that allow the CONSTRAINTS Attribute | 329 |
| | XVII | Constraints Attribute Value BNF | 330 |
| | XVIII | Applicable Constraint Items | 331 |
| | XIX | Inherited Properties and Constraints | 336 |
| | XX | Predefine Definition Node Attributes | 340 |
| | XXI | Predefine Definition Relationship Attributes | 341 |
| | XXII | ATTRIBUTES_MONITORED Attribute Value BNF | 478 |
| | XXIII | Relationships Created as a Result of CREATE_MONITOR_NODE | 480 |
| | XXIV | MONITOR Node Attributes Created by CREATE_MONITOR_NODE | 481 |
| | XXV | Events Causing Process Status Transition | 512 |
| | XXVI | Relationships Created for Root PROCESS Nodes | 522 |
| | XXVII | Relationships Inherited or Created for Dependent PROCESS Nodes | 523 |
| | XXVIII | DIRECTION Attribute Values.. | 576 |
| | XXIX | Channel State Transition | 577 |
| | XXX | DIRECTION Compatibility | 580 |
| | XXXI | Intent-DIRECTION Compatibility | 580 |
| | XXXII | Mode-DIRECTION Compatibility | 581 |
| | XXXIII | Intents for Protocol Opening | 581 |
| | XXXIV | Mode Compatibility | 581 |
| | XXXV | Detailed Protocol Opening Scenario | 583 |
| | XXXVI | Allowed Magnetic Tape Characters | 643 |
| | XXXVII | List External Representation BNF | 749 |
| | XXXVIII | Status Information List BNF | 827 |
| | XXXIX | Status Information List Description | 834 |
| | XL | Index To Predefined Node Descriptions | 929 |
| | XLI | Predefined Node Description Fields | 941 |




Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

# 5. CAIS NODE MANAGEMENT DETAILED REQUIREMENTS

This section describes the CAIS interfaces for the general manipulation of nodes, relationships and attributes. These interfaces are defined in the following CAIS packages:

a. CAIS_DEFINITIONS defines types, subtypes, exceptions, and constants used throughout the CAIS;

b. CAIS_NODE_MANAGEMENT defines interfaces for general operations on nodes and relationships;

c. CAIS_ATTRIBUTE_MANAGEMENT defines interfaces for general operations on attributes;

d. CAIS_ACCESS_CONTROL_MANAGEMENT defines interfaces for setting access rights and adopting roles;

e. CAIS_STRUCTURAL_NODE_MANAGEMENT defines interfaces for the creation of STRUCTURAL nodes;

f. CAIS_TRANSLATION_MANAGEMENT defines interfaces to control the translation of names and to translate between the two fundamental forms of identification (by name and by definition).

g. CAIS_DEFINITION_MANAGEMENT defines interfaces for the creation and maintenance of node kind, relation, attribute and attribute kind definitions and views of definitions;

h. CAIS_ATTRIBUTE_MONITOR_MANAGEMENT defines interfaces for creation of MONITOR nodes;

i. CAIS_RESOURCE_CONTROL defines interfaces for explicit resource control and monitoring of PROCESSOR and MEDIUM resources.

Specialized interfaces for the manipulation of PROCESS and FILE nodes and of their relationships and attributes are defined in Section 6 on page 509 and Section 7 on page 567, respectively.

To simplify manipulation by Ada programs, an Ada type NODE_TYPE is defined for values that represent an internal handle for a node, referred to as a node handle. Ada objects of this type can be associated with a node by means of CAIS procedures, causing an *open node handle* to be assigned to the object. While such an association is in effect, the node handle is said to be open; otherwise, the node handle is said to be *closed.* Most procedures expect either a parameter of type NODE_TYPE or a combination of a base node (specified by a parameter of type NODE_TYPE), and a path element relative to it, to identify a node.

An open node handle is guaranteed always to refer to the same node, regardless of any changes to relationships that could cause pathnames to become invalid or to refer to different nodes. This behavior is referred to as the *tracking* of nodes by open node handles.



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

## 5.1 Package CAIS_DEFINITIONS

This package defines the Ada type NODE_TYPE and its subtypes. This package also defines certain enumeration and string types, constants and exceptions useful for node manipulations. 1

```
type NODE_TYPE is limited private;
```

NODE_TYPE is the type for node handles. 2

```
type NODE_HANDLE_ ARRAY is array (CAIS_POSITIVE range <>) of NODE_TYPE;

type NODE_KIND is (FILE, STRUCTURAL, PROCESS);

type INTENT_SPECIFICATION is
    (NO_ACCESS, READ, WRITE, APPEND,
     READ_ATTRIBUTES, WRITE_ATTRIBUTES, APPEND_ATTRIBUTES,
     READ_RELATIONSHIPS, WRITE_RELATIONSHIPS, APPEND_RELATIONSHIPS,
     READ_CONTENTS, WRITE_CONTENTS, APPEND_CONTENTS,
     CONTROL, EXECUTE, CREATE_INSTANCES,
     EXCLUSIVE_READ, EXCLUSIVE_WRITE, EXCLUSIVE_APPEND,
     EXCLUSIVE_READ_ATTRIBUTES, EXCLUSIVE_WRITE_ATTRIBUTES,
     EXCLUSIVE_APPEND_ATTRIBUTES, EXCLUSIVE_READ_RELATIONSHIPS,
     EXCLUSIVE_WRITE_RELATIONSHIPS, EXCLUSIVE_APPEND_RELATIONSHIPS,
     EXCLUSIVE_READ_CONTENTS, EXCLUSIVE_WRITE_CONTENTS,
     EXCLUSIVE_APPEND_CONTENTS, EXCLUSIVE_CONTROL);

type INTENT_ARRAY is array (CAIS_POSITIVE range <>) of INTENT_SPECIFICATION;
```

NODE_HANDLE_ARRAY is the type for an array of node handles. NODE_KIND is the enumeration of the three basic kinds of nodes. INTENT_SPECIFICATION describes the usage of node handles and is further explained in Section 5.2 on page 95. INTENT_ARRAY is the type of the parameter INTENT of CAIS procedures which open or change the intent of anode handle, as further explained in Section 5.2 on page 95. (CAIS_POSITIVE is defined in Section 10 on page 805.) 3

```
subtype CLIENT_MODE_TYPE                    is NODE_TYPE;

    subtype FILE_NODE_TYPE                  is CLIENT_NODE_TYPE;
        subtype DEVICE_NODE_TYPE            is FILE_NODE_TYPE;
            subtype MAGNETIC_TAPE_NODE_TYPE is DEVICE-MODE-TYPE;
            subtype TERMINAL_NODE_TYPE      is DEVICE_NODE_TYPE;
        subtype GATEWAY_NODE_TYPE           is FILE_NODE_TYPE;
        subtype SECONDARY_STORAGE_NODE-TYPE is FILE_N0DE_TYPE;
            subtype EXECUTABLE_IMAGE_NODE_TYPE is SECONDARY_STORAGE_NODE_TYPE;
            subtype INTRODUCED_NODE_TYPE       is SECONDARY_STORAGE_NODE_TYPE;

    subtype PROCESS_NODE_TYPE               is CLIENT_NODE_TYPE;
```



Source: https://assist.dla.mil -- Downloaded: 202610-26T07:54Z
Check the source to verify that this is the current version before use.

```
subtype RESOURCE_NODE_TYPE                          is NODE_TYPE;

subtype STRUCTURAL_NODE_TYPE                        is NODE_TYPE;

    subtype CHANNEL_NODE_TYPE                       is STRUCTURAL_NODE_TYPE;
    subtype DEFINITIONS_NODE_TYPE                   is STRUCTURAL_NODE_TYPE;

    subtype DEFINITION_NODE-TYPE                    is STRUCTURAL_NODE_TYPE;
        subtype ATTRIBUTE_NODE_TYPE                 is DEFINITION_NODE_TYPE;
        subtype AD_TYPE                             is DEFINITION_NODE_TYPE;
            subtype FLOAT_AD_TYPE                   is AD_TYPE;
            subtype IDENTIFIER_AD_TYPE              is AD_TYPE;
                subtype ENUMERATION_AD_TYPE         is IDENTIFIER_AD_TYPE;
            subtype INTEGER_AD_TYPE                 is AD_TYPE;
            subtype STRING_AD-TYPE                  is AD_TYPE;
        subtype AKD_TYPE                            is DEFINITION_NODE_TYPE;
            subtype CLAKD_TYPE                      is AKD_TYPE;
                subtype NAMED_CLAKD_TYPE            is CLAKD_TYPE;
                    subtype NAMED_ORDERED_CLAKD_TYPE is NAMED_CLAKD_TYPE;
                subtype ORDERED_CLAKD_TYPE          is CLAKD_TYPE;
            subtype FAKD_TYPE                       is AKD_TYPE;
            subtype IDAKD_TYPE                      is AKD_TYPE;
            subtype IAKD_TYPE                       is AKD_TYPE;
            subtype LAKD_TYPE                       is AKD_TYPE;
            subtype SAKD_TYPE                       is AKD_TYPE;
            subtype ULAKD_TYPE                      is AKD_TYPE;
            subtype VAKD_TYPE                       is AKD_TYPE;
        subtype NKD_TYPE                            is DEFINITION_NODE_TYPE;
        subtype RD_TYPE                             is DEFINITION_NODE_TYPE;
            subtype PORT_RD_TYPE                    is RD_TYPE;

    subtype GROUP_NODE-TYPE                         is STRUCTURAL_NODE_TYPE;
    subtype INTERPRETATION_NODE_TYPE                is STRUCTURAL_NODE_TYPE;
        subtype NKI_TYPE                            is INTERPRETATION_NODE_TYPE;
        subtype RI_TYPE                             is INTERPRETATION_NODE_TYPE;
    subtype MEDIUM_NODE_TYPE                        is STRUCTURAL_NODE_TYPE;
    subtype MONITOR_NODE_TYPE                       is STRUCTURAL_NODE_TYPE;
    subtype NODE_KIND_NODE_TYPE                     is STRUCTURAL_NODE_TYPE;
    subtype PROCESSOR_NODE_TYPE                     is STRUCTURAL_NODE_TYPE;
    subtype SYSTEM_LEVEL_NODE_TYPE                  is STRUCTURAL-NODE_TYPE;
    subtype USER_NODE_TYPE                          is STRUCTURAL_NODE_TYPE;
    subtype VIEW_NODE_TYPE                          is STRUCTURAL_NODE_TYPE;
```

The subtypes of NODE_TYPE are used in the specifications of subprograms which perform operations on specific kinds of nodes. Input parameters of these subtypes must identify nodes of the kind indicated, or the exception NODE_KIND_ERROR is raised. 4



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
subtype ATTRIBUTE_NAME      is STRING;
subtype ATTRIBUTE_KIND_NAME is STRING;
subtype NODE_KIND_NAME      is STRING;
subtype RELATION_NAME       is STRING;
```

ATTRIBUTE_NAME, ATTRIBUTE_KIND_NAME, NODE_KIND_NAME, and RELATION_NAME are subtypes of string for the names of attributes, attribute kinds, node kinds and relations. Values of these subtypes must have the syntax of Ada identifiers; violations of this syntax causes exceptions to be raised. Both upper case and lowercase characters may be used in these names and case must be retained by the CAIS implementation. However, comparisons of names are insensitive to upper and lowercase.

```
subtype PATHNAME          is STRNG;
subtype RELATIONSHIP_KEY  is STRING;
```

PATHNAME and RELATIONSHIP_KEY are string subtypes for pathnames and relationship key lists. Values of these subtypes are subject to certain syntactic restrictions whose violation causes exceptions to be raised.

```
subtype UNIQUE_IDENTIFIER is STRING;
```

UNIQUE_IDENTIFIER is a string subtype for unique identifiers. Syntactically, a unique identifier consists of a prefix and a suffix separated by the character “.’’(period). The prefix and suffix may include any graphic characters other than “.” (period) or space. Otherwise, the exact form of these syntactic elements is implementation-defined. Unique identifiers provided as input parameters which do not meet these syntactic requirements cause an exception to be raised.

The CAIS imposes requirements on implementations with respect to the sizes of the components of unique identifiers. These requirements are specified as constants defined in package CAIS_PRAGMATICS. The lengths of prefixes and suffixes generated by an implementation must not exceed the constants CAIS_UNIQUE_IDENTIFIER_PREFIX_LENGTH and CAIS_UNIQUE_IDENTIFIER_SUFFIX_LENGTH respectively. A CAIS implementation must support the definition of prefixes as long as CAIS_UNIQUE_IDENTIFIER_PREFIX_LENGTH when an instance of that implementation is installed. A CAIS implementation must also be able to generate distinct suffixes for at least the number of unique identifiers specified by the constant CAIS_UNIQUE_IDENTIFIERS_PER_INSTANCE.

Input parameters to the CAIS interfaces of the string subtypes ATTRIBUTE_NAME, ATTRIBUTE_KIND_NAME, NODE_KIND_NAME, RELATION_NAME, PATHNAME, RELATIONSHIP_KEY and UNIQUE_IDENTIFIER may contain blanks, format effecters and/or other non-printing characters surrounding lexical elements.

```
subtype ATTRIBUTE_LIST               is CAIS_LIST_MANAGEMENT. LIST_TYPE;
subtype DISCRETIONARY_ACCESS_LIST is CAIS_LIST_MANAGEMENT. LIST_TYPE;
subtype MANDATORY_ACCESS_LIST        is CAIS_LIST_MANAGEMENT. LIST_TYPE;
subtype UNIQUE_IDENTIFIER_LIST       is CAIS_LIST_MANAGEMENT. LIST_TYPE;
```

ATTRIBUTE_LIST is the subtype for lists of attributes. A value of subtype ATTRIBUTE_LIST is an empty or named list, where each item is a list. The item names are attribute names. The



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

items are attribute values in list format to be given to the attributes named. (See Section 5.7.1.6 on page 315 for the description of attribute values in list format.) DISCRETIONARY. ACCESS_LIST and MANDATORY_ACCESS_LIST are the subtypes, respectively, of discretionary and mandatory access control information. A value of subtype DISCRETIONARY_ACCESS_LIST is the empty list or a GRANT attribute value. (See Table XI on page 262 for the format of GRANT attributes.) A value of subtype MANDATORY_ACCESS_LIST is the empty list or a SUBJECT_CLASSIFICATION or OBJECT_CLASSIFICATION attribute value. (See Table IX on page 256 for the format of subject and object classification attributes.) UNIQUE_IDENTIFIER_LIST is the subtype for a list of unique identifiers, each represented as a string item. Values of these LIST_TYPE subtypes are subject to certain syntactic restrictions whose violation causes exceptions to be raised. (LIST_TYPE is defined in Section 9 on page 745.)

```
NULL_NODE : constant NODE_TYPE;
```

NULL_NODE is a deferred constant node handle which is closed and therefore does not identify any node. However, NULL_NODE has a value distinguished from that of any other object of type NODE_-TYPE. The constant NULL_NODE is used in place of an open node handle to indicate that no node is identified. 11

```
CURRENT_USER      : constant PATHNAME              := "'CURRENT_USER";
CURRENT_NODE      : constant PATHNAME              := "'CURRENT_NODE";
CURRENT_PROCESS   : constant PATHNAME              := ";";
EMPTY_KEY_LIST    : constant RELATIONSHIP_KEY      := "";
LATEST_KEY        : constant RELATIONSHIP_KEY      := "#";
DEFAULT_RELATION  : constant RELATION_NAME         := "DOT";
```

CURRENT_USER, CURRENT_NODE, and CURRENT_PROCESS are standard pathnames for the current user's top-level node, current node (base node for pathnames beginning with a relationship key), and current PROCESS node, respectively. LATEST_KEY and DEFAULT_RELATION are standard names for the latest key and the default relation name, respectively. EMPTY_KEY_LIST is a relationship key list which specifies no key values. 12

```
LONG-DELAY : constant CAIS_DURATION : = CAIS_DURATION ' LAST;
```

LONG_DELAY is a constant of type CAIS_DURATION used for time limits. (CAIS_DURATION is defined in Section 10 on page 805. See also [1815A] 9.6 for type DURATION.) 13

```
ACCESS_VIOLATION          : exception;
AMBIGUOUS_VIEU_ERROR      : exception;
ATTRIBUTE_ERROR           : exception;
CONFORMANCE_ERROR         : exception;
DEFINITION_ERROR          : exception;
DEVICE_ERROR              : exception;
EXISTING_NODE_ERROR       : exception;
INTENT_VIOLATION          : exception;
ITERATOR_ERROR            : exception;
LOCK_ERROR                : exception;
```


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
NAME_ERROR                    : exception;
NODE_KIND_ERROR               : exception;
NOT_IN_VIEW_ERROR             : exception;
PATHNAME_SYNTAX_ERROR         : exception;
PREDEFINED_ATTRIBUTE_ERROR    : exception;
PREDEFINED_RELATION_ERROR     : exception;
RELATIONSHIP_ERROR            : exception;
RESOURCE_ALLOCATION_ERROR     : exception;
RESOURCE_AVAILABILITY_ERROR   : exception;
SECURITY_VIOLATION            : exception;
STATUS_ERROR                  : exception;
SYNTAX_ERROR                  : exception;
TRANSACTION_ERROR             : exception;
USE_ERROR                     : exception;
```

ACCESS_VIOLATION is raised if an operation is attempted which violates access right constraints other than knowledge of existence of the node. If an implementation adopts a security policy in which discretionary access rights are checked even when appropriate intents have been established, then operations may raise this exception even though the exception is not mentioned in the interface description. 14

AMBIGUOUS_VIEW_ERROR is raised by an operation which translates or forms a name for a property if the naming for the property visible in the composite view is ambiguous for translation or formation, respectively. AMBIGUOUS_VIEW_ERROR is also raised by an operation on a view which would define more than one naming for the same property in a single interpretation of a node kind or relation, and by an operation on a view which would define more than one naming for the same defmition in a single view. 15

ATTRIBUTE_ERROR is raised if an operation requires a given attribute and it does not exist, if an operation requires that an attribute does not exist and it does exist, or if a parameter containing a list of attribute values specifies a value for an attribute more than once. 16

CONFORMANCE_ERROR is raised if an operation would result in an attribute, node or relationship not conforming to its definition or the constraints in force on that definition. 17

DEFINITION_ERROR is raised if an operation constitutes an illegal modification of a definition. Any of the following are illegal definition modifications: an attempt to specify increased constraints on a property already defined for a definition and an instance of the property exists, an attempt to define an additional property for a definition which the definition cannot have given its current defined properties, an attempt to violate a restriction on the deletion of a definition node or a property definition relationship or attribute (see Section 5.7.1.9 on page 335). 18

DEVICE_ERROR is raised if a CAIS operation cannot be completed because of a malfunction of the underlying system. All operations may raise this exception, so it is not explicitly mentioned in the descriptions of the interfaces. 19

EXISTING_NODE_ERROR is raised if a node already exists with the identification given and an attempt is made to create a node with this identification. 20

INTENT_VIOLATION is raised if an operation is attempted on an open node handle which is in violation of the intent associated with the open node handle. 21

ITERATOR_ERROR is raised if an iterator is used that has not been set or is exhausted. 22



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

LOCK_ERROR is raised if access to a node is blocked due to the existence of a lock on this node, its attributes, relationships or contents. Such access may be an implicit action of a CAIS operation (see Section 5.2.2 on page 96). For example, DEFINITION nodes are implicitly read during conformance checking of writes to instances and when iterating over instances of a node kind. LOCK_ERROR is raised by any operation if there is an EXCLUSIVE_WRITE lock held by a transaction on any DEFINITION node to be referenced; these occurrences of LOCK_ERROR are not explicitly mentioned in the descriptions of interfaces. LOCK_ERROR may be raised prior to expiration of a specified timeout if the CAIS implementation can determine that a deadlock situation has occurred. 23

NAME_ERROR is raised if an attempt is made to access a node via a pathname or node handle while the node does not exist, is unobtainable, discretionary access control constraints for knowledge of existence of a node are violated or mandatory access controls for "read" operations are violated. If an implementation adopts a security policy in which discretionary access rights are checked even when appropriate intents have been established, then operations may raise this exception even though the exception is not mentioned in the interface description. This exception takes precedence over any exceptions which would divulge information about the existence or properties of the given node. 24

NODE_KIND_ERROR is raised if the kind of the node is incorrect for the attempted operation. 25

NOT_IN_VIEW_ERROR is raised by an operation which translates or forms a name in a given context, if the composite view of the current process does not provide a visible naming that applies in the given context. The required namings may be namings for properties of node kinds and relations, including attributes and emanating relations, or namings for definitions including attribute definitions, attribute kind definitions, node kind definitions and relation definitions. 26

PATHNAME_SYNTAX_ERROR is raised if pathname information is syntactically illegal. 27

PREDEFINED_ATTRIBUTE_ERROR is raised if an attempt is made to create or modify a CAIS-controlled attribute. 28

PREDEFINED_RELATION_ERROR is raised if an attempt is made to create or modify a CAIS-controlled relationship. 29

RELATIONSHIP_ERROR is raised if an operation requires a relationship identification that does not identify an existing relationship and such a relationship exists, if an operation requires a relationship identification that identifies exactly one relationship and either no such relationship exists or more than one such relationship exists, if a relationship key list specifies the value of a key more than once or if the relation of a relationship to be created does not have a unique key set. 30

RESOURCE_ALLOCATION_ERROR is raised if an operation is attempted which specifies the allocation of a resource which is not allocatable from the current processor. 31

RESOURCE_AVAILABILITY_ERROR is raised if an attempt is made to allocate an unavailable resource. 32

SECURITY_VIOLATION may be raised if an operation is attempted which violates mandatory access controls for "write" operations. SECURITY_VIOLATION is raised only if the conditions for other exceptions are not present. 33

STATUS_ERROR is raised if the status of a node handle, transaction handle, file handle, screen handle, status indicator or channel does not conform to expectations. 34

SYNTAX_ERROR is raised if the information given in certain parameters other than pathname parameters is syntactically illegal. 35

TRANSACTION_ERROR is raised if an operation is included in more than one transaction, if an 36



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

operation included in a transaction modifies a node which is not included in the same transaction or if an operation included in a transaction accesses a node using a file handle that was not opened inside the transaction.

USE_ERROR is raised if a restriction on the use of an interface is violated.

The CAIS does not prescribe a particular preference ordering among these exceptions in the presence of multiple error situations. However, some preference must be given where its lack would create inference paths violating security models. Actual preference orderings will have the following properties:

1. If access to information that is inaccessible according to the access control models is required to detect an additional error situation, the exception relating to the attempt to access inaccessible information takes precedence over the exception for the additional error.

2. Operations which traverse multiple relationships in a single path give preference to exceptions related to earlier relationships in the path over exceptions related to subsequent traversals.



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

## 5.2 Package CAIS_NODE_MANAGEMENT

This package defines the general primitives for manipulating, copying, renaming and deleting nodes and their relationships. The exceptions raised by all subprograms in this package are defined in the packages CAIS_DEFINITIONS and CAIS_PRAGMATICS.

The interfaces defined in this package are applicable to all nodes, relationships and attributes except where explicitly stated otherwise. These interfaces do not include the creation of nodes. Interfaces for creating nodes are defined in packages specific to each node kind. For example, the CREATE_NODE interface for creating STRUCTURAL nodes is defined in package CAIS_STRUCTURAL_NODE_MANAGEMENT; interfaces for creating DEFINITION nodes are defined in package CAIS_DEFINITION_MANAGEMENT.

Node handles which identify RDs and ADs are used in most of the interfaces described in this and other sections of this standard as a means of identifying relations and attributes. In the Additional Interfaces, relations and attributes are generally identified via relation names and attribute names. The Additional Interfaces have calls to the appropriate procedures within the package CAIS_TRANSLATION_MANAGEMENT to obtain node handles to RDs and ADs that are be used in calls to the primary interfaces. These procedures in CAIS_TRANSLATION_MANAGEMENT translate local names for DEFINITION nodes in the reamer described in Section 5.7.3.6 on page 355 using the composite view of the current process.

### 5.2.1 Open operations

Open node handles are central to node administration. An open node handle identifies a node, and has associated with it an intent which categorizes the operations intended to be performed on that node. (Intents are explained in Table IV on page 98.) Opening a node handle includes access control checks and establishes access synchronization:

1. The discretionary and mandatory access rights of the current process to the given node are checked as detailed in Section 5.4 on page 255. These checks establish that the process has the rights to perform the intended operations on the node.

2. The intents for the node handle to be opened are checked against outstanding locks on the node. A lock is established when an intent is granted, to ensure that subsequent operations on the node do not interfere with one another. Outstanding locks delay the establishment of additional locks as shown in Table V on page 102.

Three CAIS interfaces for manipulating node handles are OPEN (opens a node handle), CLOSE (closes a node handle), and CHANGE_INTENT (alters the specification of the intention of node handle usage). In addition, OPEN_VIA_UNIQUE_IDENTIFIER, OPEN_PARENT, GET_CURRENT_NODE, and GET_NEXT open node handles to existing nodes. These interfaces perform access synchronization in accordance with an intent specified by the parameter INTENT. One or more of the intents defined in Table IV can be expressed by the INTENT parameters.

In order to perform their function, many CAIS operations must access nodes not explicitly identified by open node handles passed to the interface. Except where stated otherwise, such accesses are subject to the access control checks and the access synchronization locks prescribed for opening a node handle. These operations are therefore understood to open node handles as part of their function, and to close



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

them before returning. Opening a node handle in this way is termed an “implicit open operation.” Examples of operations which include implicit open operations are the following:

1. In path traversal, each node traversed is opened with the intent to read relationships. The node at the end of the path is opened with an intent consistent with the operation to be performed on that node.

2. In operations applicable to all nodes in a tree (defined by primary relationships) or other structure, each node in the structure is opened with an intent consistent with the operation to be performed. Such operations include: COPY_TREE and DELETE_TREE in CAIS_NODE_-MANAGEMENT; DELETE_JOB, ABORT_PROCESS, SUSPEND_PROCESS and RESUME_ -PROCESS in CAIS_PROCESS_MANAGEMENT; INCLUDE_TREE in CAIS_TRANSACTION_-MANAGEMENT; DELETE_VIEW and DELETE_ DEFINITION_TREE in CAIS_DEFINITION_-MANAGEMENT; and INCLUDE_TREE and INCLUDE_STRUCTURE in CAIS_COMMON_-IO.

Subsequent uses of the phrase “open operation” refer to implicit open operations as well as the OPEN, OPEN_VIA_UNIQUE_IDENTIFIER, GET_CURRENT_NODE, OPEN_PARENT and GET_-NEXT operations which return open node handles.

### 5.2.2 Accesses without open operations

Some node accesses implicit in CAIS semantics do not include open operations. These accesses are not arbitrated by CAIS access controls nor are they delayed by the locks associated with open node handles. (Synchronization of these accesses with other accesses is provided by mechanisms internal to CAIS implementations to the extent required to avoid transient data. ) The accesses implicit in CAIS semantics which do not include open operations are the following:

1. The discretionary access rights of a group toward a given node are determined when that node is to be opened (and optionally at other times). These rights are described by ACCESS and DENY relationships emanating from the node and ADOPTED_ROLE relationships emanating from the current PROCESS node; the GROUP nodes at which those relationships terminate; and all parent nodes of the identified GROUP nodes, recursively. Access to that information during discretionary access control checking by an open operation does not itself constitute an open operation.

2. The current PROCESS node, the adopted roles of the current process and potential members of those adopted roles are accessed during role adoption. Such accesses, performed to determine potential membership, do not constitute open operations.

3. Access to subject and object classification labels during mandatory access control checks do not constitute open operations.

4. Accesses to definition nodes for purposes of determining definition information do not constitute open operations. Definition nodes are accessed when the instances they describe are accessed, for purposes of conformance checking. Definition nodes are accessed in determining whether one definition is a specialization of another, during identifications and during sub-kind queries. Definition information may be accessed to support an implementation of iterators.



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

5. Operations which create nodes do not constitute open operations on the new nodes. These operations may return open node handles with associated locks on the new nodes, but access rights toward the new node are not checked in producing those open handles. 6

### 5.2.3 Access delays

Open node handles can delay attempts to open other node handles to the same node or to change the intent of other node handles to that node, as explained in Table IV and summarized graphically in Table V. Many CAIS interfaces that may be delayed upon opening a node allow the specification of a time limit after which a call to such interfaces is allowed to terminate with an exception. 1

When a node has been included within a transaction (see Section 6.1 on page 514), an access synchronization lock is held by the transaction which may delay attempts which occur outside the transaction to obtain access to that node with any intent other than NO_ACCESS. Though some implicit accesses are not subject to locks associated with open node handles, all operations are subject to access synchronization with access synchronization locks held by a transaction, when they are performed outside the given transaction. Consequently, operations which entail implicit accesses of definition or access control information which is subject to a lock held by a transaction may also be delayed or fail with a LOCK_ERROR exception. 2

Open or CHANGE_INTENT operations performed by a process enclosed within a transaction, producing an open node handle with any intent other than NO_ACCESS, cause the transaction to acquire either an EXCLUSIVE_READ or EXCLUSIVE_WRITE access synchronization lock on the node. Open and CHANGE_INTENT operations can cause a transaction's EXCLUSIVE_READ access synchronization lock on a node to be upgraded to an EXCLUSIVE_WRITE access synchronization lock. This occurs when a process with operations in the transaction requests access to the node with write or control intent. 3

Consequently, open and CHANGE_INTENT operations by a process with operations in a transaction may delay the calling task, if the transaction must acquire or upgrade access synchronization locks on the nodes being opened. 4


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

TABLE IV: **Intents**

| Intent | Explanation |
|---|---|
| NO_ACCESS | The established access right for subsequent operations is to query properties of the node handle and obtainability of the node only. Locks on the node have no delaying effect. |
| READ<br>EXCLUSIVE_READ | Open and CHANGE_INTENT operations are delayed if the node, its contents, attributes or relationships are locked against read operations. The established access right for subsequent operations is to read node contents, attributes and relationships.<br>For EXCLUSIVE_READ, the node is locked against opens with any write or control intent as specified in Table V. Open and CHANGE_INTENT operations are additionally delayed if there are open node handles to the node with write or control intent. |
| WRITE<br>EXCLUSIVE_WRITE | Open and CHANGE_INTENT operations are delayed if the node, its contents, attributes or relationships are locked against write operations. The established access right for subsequent operations is to write, create or append to node contents, attributes and relationships.<br>For EXCLUSIVE_WRITE, the node is locked against opens with any read, write, append, execute or control intent as specified in Table V. Open and CHANGE_INTENT operations are additionally delayed if there are open node handles to the node with read, write, append, execute or control intent. |
| APPEND<br>EXCLUSIVE_APPEND | Open and CHANGE_INTENT operations are delayed if the node, its contents, attributes or relationships are locked against append operations. The established access right for subsequent operations is to append to the contents and to create attributes or relationships.<br>For EXCLUSIVE_APPEND, the node is locked against opens with write, append or execute intent as specified in Table V. Open and CHANGE_INTENT operations are additionally delayed if there are open node handles to the node with write, append or execute intent. |
| READ_CONTENTS<br>EXCLUSIVE_READ_CONTENTS | Open and CHANGE_INTENT operations are delayed if the node or its contents are locked against read operations. The established access right for subsequent operations is to read the node contents. |



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

TABLE IV: **Intents (continued)**

| Intent | Explanation |
|---|---|
| READ_CONTENTS (cont.)<br>EXCLUSIVE_READ_CONTENTS | For EXCLUSIVE_READ_CONTENTS, the node contents are locked against all opens with write intent as specified in Table V. Open and CHANGE_INTENT operations are additionally delayed if there are open node handles to the node with intent to write its contents. |
| WRITE_CONTENTS<br>EXCLUSIVE_WRITE_CONTENTS | Open and CHANGE_INTENT operations are delayed if the node or its contents are locked against write operations. The established access right for subsequent operations is to write or append to the node contents.<br>For EXCLUSIVE_WRITE_CONTENTS, the node contents are locked against opens with read, write, append or execute intent as specified in Table V. Open and CHANGE_INTENT operations are additionally delayed if there are open node handles to the node with intent to read, write, execute or append to its contents. |
| APPEND_CONTENTS<br>EXCLUSIVE_APPEND_CONTENTS | Open and CHANGE_INTENT operations are delayed if the node or its contents are locked against append operations. The established access right for subsequent operations is to append to the node contents or to assign the medium resource.<br>For EXCLUSIVE_APPEND_CONTENTS, the node contents are locked against opens with write, append, or execute intent as specified in Table V. Open and CHANGE_-INTENT operations are additionally delayed if there are open node handles to the node with intent to write, execute or append to its contents. |
| READ_ATTRIBUTES<br>EXCLUSIVE_READ_ATTRIBUTES | Open and CHANGE_INTENT operations are delayed if the node or its attributes are locked against read operations. The established access right for subsequent operations is to read node attributes.<br>For EXCLUSIVE_READ_ATTRIBUTES, the node is locked against opens with intent to write attributes as specified in Table V. Open and CHANGE_INTENT operations are additionally delayed if there are open node handles to the node with intent to write attributes. |
| WRITE_ATTRIBUTES<br>EXCLUSIVE_WRITE_ATTRIBUTES | Open and CHANGE_INTENT operations are delayed if the node or its attributes are locked against write operations. The established access right for subsequent operations is to modify and create node attributes. |



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

TABLE IV: **Intents (continued)**

| Intent | Explanation |
|---|---|
| WRITE_ATTRIBUTES (cont.)<br>EXCLUSIVE_WRITE_ATTRIBUTES | For EXCLUSIVE_WRITE_ATTRIBUTES, the node is locked against opens with intent to read, write or append attributes as specified in Table V. Open and CHANGE_-INTENT operations are additionally delayed if there are open node handles to the node with intent to read, write or append attributes. |
| APPEND_ATTRIBUTES<br>EXCLUSIVE_APPEND_ATTRIBUTES | Open and CHANGE_INTENT operations are delayed if the node or its attributes are locked against append operations. The established access right for subsequent operations is to create node attributes.<br>For EXCLUSIVE_APPEND_ATTRIBUTES, the node is locked against opens with intent to write or append attributes as specified in Table V. Open and CHANGE_-INTENT operations are additionally delayed if there are open node handles to the node with intent to write or append attributes. |
| READ_RELATIONSHIPS<br>EXCLUSIVE_READ_RELATIONSHIPS | Open and CHANGE_INTENT operations are delayed if the node or its relationships are locked against read operations. The established access right for subsequent operations is to read relationships, including their attributes, and (for VIEW nodes) to adopt the view represented by the node and NKIs within that view.<br>For EXCLUSIVE_READ_RELATIONSHIPS, the node is locked against opens with control intent or intent to write relationships as specified in Table V. Open and CHANGE_INTENT operations are additionally delayed if there are open node handles to the node with control intent or intent to write relationships. |
| WRITE_RELATIONSHIPS<br>EXCLUSIVE_WRITE_-<br>RELATIONSHIPS | Open and CHANGE_INTENT operations are delayed if the node or its relationships are locked against write operations. The established access right for subsequent operations is to write or create relationships, including their attributes.<br>For EXCLUSIVE_WRITE_RELATIONSHIPS, the node is locked against opens with control intent or intent to read, write or append relationships as specified in Table V. Open and CHANGE_INTENT operations are additionally delayed if there are open node handles to the node with control intent or intent to read, write or append relationships. |


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

TABLE IV: **Intents (continued)**

| Intent | Explanation |
|---|---|
| APPEND_RELATIONSHIPS<br>EXCLUSIVE_APPEND_-<br>RELATIONSHIPS | Open and CHANGE_INTENT operations are delayed if the node or its relationships are locked against append operations. The established access right for subsequent operations is to create relationships, including their attributes.<br>For EXCLUSIVE_APPEND_RELATIONSHIPS, the node is locked against opens with control intent or intent to write or append relationships as specified in Table V. Open and CHANGE_INTENT operations are additionally delayed if there are open node handles to the node with control intent or intent to write or append relationships. |
| CONTROL<br>EXCLUSIVE_CONTROL | Open and CHANGE_INTENT operations are delayed if the node or its relationships are locked against write, control or (exclusive or non-exclusive) READ and READ_-RELATIONSHIPS operations. The established access right for subsequent operations is to read, write or append access control information.<br>For EXCLUSIVE_CONTROL, the node is locked against opens to read, write or append relationships or to read, write or append access control information as specified in Table V. Open and CHANGE_INTENT operations are additionally delayed if there are open node handles to the node with intent to read, write or append relationships or to read, write or append access control information. |
| CREATE_INSTANCES | Open and CHANGE_INTENT operations are delayed only by EXCLUSIVE_WRITE access synchronization locks held by transactions on the node. The established access right for subsequent operations is to create nodes which are instances of the node kind definition represented by the node. The node is locked against operations which would result in an EXCLUSIVE_WRITE access synchronization lock on the node held by a transaction. |
| EXECUTE | Open and CHANGE_INTENT operations are delayed if the node contents are locked against read or append operations or its relationships are locked against read operations. The established access right for subsequent operations is the permission to initiate a process taking the node contents as its executable image or to assign the processor resource. |



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

TABLE V: Matrix of Access Synchronization Constraints

| I1 \ I2 | NON-EXCLUSIVE | | | | | | | | | | | | | | | | EXCLUSIVE | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | NO | R | W | A | RC | WC | AC | RA | WA | AA | RR | WR | AR | C | EX | CI | R | W | A | RC | WC | AC | RA | WA | AA | RR | WR | AR | C | |
| NO | . | . | . | . | . | . | . | . | . | . | . | . | . | . | . | . | . | . | . | . | . | . | . | . | . | . | . | . | . | . | NO |
| R | . | . | . | . | . | . | . | . | . | . | . | . | . | . | . | . | . | X | . | . | X | . | . | X | . | . | X | . | X | R |
| W | . | . | . | . | . | . | . | . | . | . | . | . | . | . | . | . | X | X | X | X | X | X | X | X | X | X | X | X | X | W |
| A | . | . | . | . | . | . | . | . | . | . | . | . | . | . | . | . | . | X | X | . | X | X | . | X | X | . | X | X | X | A |
| RC | . | . | . | . | . | . | . | . | . | . | . | . | . | . | . | . | . | X | . | . | X | . | . | . | . | . | . | . | . | RC |
| WC | . | . | . | . | . | . | . | . | . | . | . | . | . | . | . | . | X | X | X | X | X | X | . | . | . | . | . | . | . | WC |
| AC | . | . | . | . | . | . | . | . | . | . | . | . | . | . | . | . | . | X | X | . | X | X | . | . | . | . | . | . | . | AC |
| RA | . | . | . | . | . | . | . | . | . | . | . | . | . | . | . | . | . | X | . | . | . | . | . | X | . | . | . | . | . | RA |
| WA | . | . | . | . | . | . | . | . | . | . | . | . | . | . | . | . | X | X | X | . | . | . | X | X | X | . | . | . | . | WA |
| AA | . | . | . | . | . | . | . | . | . | . | . | . | . | . | . | . | . | X | X | . | . | . | . | X | X | . | . | . | . | AA |
| RR | . | . | . | . | . | . | . | . | . | . | . | . | . | . | . | . | . | X | . | . | . | . | . | . | . | . | X | . | X | RR |
| WR | . | . | . | . | . | . | . | . | . | . | . | . | . | . | . | . | X | X | X | . | . | . | . | . | . | X | X | X | X | WR |
| AR | . | . | . | . | . | . | . | . | . | . | . | . | . | . | . | . | . | X | X | . | . | . | . | . | . | . | X | X | X | AR |
| C | . | . | . | . | . | . | . | . | . | . | . | . | . | . | . | . | X | X | X | . | . | . | . | . | . | X | X | X | X | C |
| EX | . | . | . | . | . | . | . | . | . | . | . | . | . | . | . | . | . | X | X | . | X | X | . | . | . | . | X | . | X | EX |
| CI | . | . | . | . | . | . | . | . | . | . | . | . | . | . | . | . | . | . | . | . | . | . | . | . | . | . | . | . | . | CI |
| R | . | . | X | . | . | X | . | . | X | . | . | X | . | X | . | . | . | X | . | . | X | . | . | X | . | . | X | . | X | R |
| W | . | X | X | X | X | X | X | X | X | X | X | X | X | X | . | . | X | X | X | X | X | X | X | X | X | X | X | X | X | W |
| A | . | . | X | X | . | X | X | . | X | X | . | X | X | X | . | . | . | X | X | . | X | X | . | X | X | . | X | X | X | A |
| RC | . | . | X | . | . | X | . | . | . | . | . | . | . | . | . | . | . | X | . | . | X | . | . | . | . | . | . | . | . | RC |
| WC | . | X | X | X | X | X | X | . | . | . | . | . | . | . | X | . | X | X | X | X | X | X | . | . | . | . | . | . | . | WC |
| AC | . | . | X | X | . | X | X | . | . | . | . | . | . | . | X | . | . | X | X | . | X | X | . | . | . | . | . | . | . | AC |
| RA | . | . | X | . | . | . | . | . | X | . | . | . | . | . | . | . | . | X | . | . | . | . | . | X | . | . | . | . | . | RA |
| WA | . | X | X | X | . | . | . | X | X | X | . | . | . | . | . | . | X | X | X | . | . | . | X | X | X | . | . | . | . | WA |
| AA | . | . | X | X | . | . | . | . | X | X | . | . | . | . | . | . | . | X | X | . | . | . | . | X | X | . | . | . | . | AA |
| RR | . | . | X | . | . | . | . | . | . | . | . | X | . | X | . | . | . | X | . | . | . | . | . | . | . | . | X | . | X | RR |
| WR | . | X | X | X | . | . | . | . | . | . | X | X | X | X | X | . | X | X | X | . | . | . | . | . | . | X | X | X | X | WR |
| AR | . | . | X | X | . | . | . | . | . | . | . | X | X | X | . | . | . | X | X | . | . | . | . | . | . | . | X | X | X | AR |
| C | . | X | X | X | . | . | . | . | . | . | X | X | X | X | X | . | X | X | X | . | . | . | . | . | . | X | X | X | X | C |
| I1 / I2 | NO | R | W | A | RC | WC | AC | RA | WA | AA | RR | WR | AR | C | EX | CI | R | W | A | RC | WC | AC | RA | WA | AA | RR | WR | AR | C | |

X = Open with intent I2 is delayed if there are open handles opened with intent I1.

C = CONTROL
NO = NO_ACCESS CI = CREATE_INSTANCES EX = EXECUTE

R = READ W = WRITE A = APPEND
RC = READ_CONTENTS WC = WRITE_CONTENTS AC = APPEND_CONTENTS
RA = READ_ATTRIBUTES WA = WRITE_ATTRIBUTES AA = APPEND_ATTRIBUTES
RR = READ_RELATIONSHIPS WR = WRITE_RELATIONSHIPS AR = APPEND_RELATIONSHIPS



Source: https://assist.dla.mil -- Downloaded: 202610-26T07:54Z
Check the source to verify that this is the current version before use.

### 5.2.4 Opening a node handle

```
procedure OPEN
    (NODE       : in out NODE_TYPE;
     BASE       : in     NODE_TYPE;
     KEY        : in     RELATIONSHIP-KEY := EMPTY_KEY_LIST;
     RELATION   : in     RD_TYPE;
     INTENT     : in     INTENT_ARRAY     := (1 => READ);
     TIME_LIMIT : in     CAIS_DURATION    := LONG_DELAY;
     STATUS     : in     STATUS_INDICATOR := NO_INDICATION);

procedure OPEN
    (NODE       : in out NODE_TYPE;
     NAME       : in     PATHNAME;
     INTENT     : in     INTENT_ARRAY     := (1 => READ);
     TIME_LIM1T : in     CAIS_DURATION    := LONG_DELAY;
     STATUS     : in     STATUS_INDICATOR := NO_INDICATION);

procedure OPEN
    (NODE       : in out NODE_TYPE;
     HANDLE     : in     MODE_TYPE;
     INTENT     : in     INTENT_ARRAY     := (1 => READ);
     TIME_LIMIT : in     CAIS_DURATION    := LONG_DELAY;
     STATUS     : in     STATUS_INDICATOR := NO_INDICATION);
```

Purpose:

1 These procedures return an open node handle in NODE to the node identified by BASE, KEY and RELATION, to the node identified by the pathnarne NAME, or to the node identified by an open node handle HANDLE, respectively. INTENT specifies the access rights available for subsequent uses of the node handle; it also establishes access synchronization with other users of the node. TIME_LIMIT specifies a time limit for the delay imposed on OPEN by existing locks on the node. A delayed OPEN call completes after the node is unlocked or the specified time limit has elapsed. In the latter case, the exception LOCK_ERROR is raised.

2 When the node to be opened is identified by the parameter HANDLE, the key and relation of the last relationship traversed, if any, in opening the node handle HANDLE are associated with node handle NODE. See subprograms PATH_KEY (Section 5.2.20 on page 119) and PATH_-RELATION (Section 5.2.21 on page 120).

Parameters:

3 **NODE** is anode handle, initially closed, to be opened to the identified node.

4 **BASE** is an open node handle to a base node for node identification.

5 **KEY** is the relationship key list for node identification.

6 **RELATION** is an open node handle identifying the RD for node identification.

7 **NAME** is the pathname identifying the node to reopened.



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

**HANDLE** is the open node handle identifying the node to be opened.

**INTENT** is the intent of subsequent operations on the node; the actual parameter takes the form of an array aggregate.

**TIME_LIMIT** is the maximum time to wait for the unlocking of the identified node in accordance with the desired INTENT.

**STATUS** is an established status indicator.

Exceptions:

**PATHNAME_SYNTAX_ERROR** is raised if the NAME [1] or KEY [2] given is syntactically illegal (see Table I, page 52).

**NAME_ERROR** is raised if any traversed node in the path specified is unobtainable or inaccessible or if the node to which the handle is to be opened is inaccessible or unobtainable and the given INTENT includes any intent other than NO_ACCESS [1].

**NOT_IN_VIEW_ERROR** is raised if a key or relation name specified in KEY or NAME cannot be translated [1] or if KEY or a relationship key list specified in NAME consists of a single relationship key designator and no key of the corresponding relation is visible [5].

**AMBIGUOUS_VIEW_ERROR** is raised if translation of a key or relation name specified in KEY or NAME is ambiguous [1] or if KEY or a relationship key list specified in NAME consists of a single relationship key designator and more than one key of the corresponding relation is visible [4].

**USE_ERROR** is raised if the specified INTENT is an empty array [1].

**STATUS_ERROR** is raised if the node handle NODE is already open at the time of the call on OPEN [1], if BASE, RELATION, or HANDLE is not an open node handle [2] or if STATUS is not an established status indicator [3].

**NODE_KIND_ERROR** is raised if the open node handle RELATION does not identify an R.D [1].

**RELATIONSHIP_ERROR** is raised if KEY or any relationship key list in NAME specifies a value for the same key more than once [5], if BASE, KEY, and RELATION or any path element of NAME identifies more than one relationship [1] or if the relationship specified by BASE, KEY and RELATION or by any path element of NAME does not exist [2].

**TRANSACTION_ERROR** is raised if the operation is included in more than one transaction [1].

**LOCK_ERROR** is raised if the OPEN operation is delayed beyond the specified TIME_LIMIT due to the existence of locks in conflict with the specified INTENT [1]. This includes any delays caused by locks on nodes traversed on the path specified by the pathname NAME or locks on the node identified by BASE, preventing the reading of relationships emanating from these nodes. LOCK_ERROR may be raised prior to expiration of the time limit if the CAIS implementation can determine that a deadlock situation has occurred.

**INTENT_VIOLATION** is raised if BASE was not opened with an intent establishing the right to read relationships [1].

**ACCESS_VIOLATION** is raised if the discretionary access control rights of the current process are insufficient to traverse the path specified by NAME or by BASE, KEY and RELATION or to obtain access to the node consistent with the specified INTENT [1].


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

**SECURITY_VIOLATION** is raised if the attempt to obtain access to the node with the specified INTENT represents a violation of mandatory access controls. 24

Additional Interfaces:

```
procedure OPEN
    (NODE        : in out NODE_TYPE;
     BASE        : in     NODE_TYPE;
     KEY         : in     RELATIONSHIP_KEY  := EMPTY_KEY_LIST;
     RELATION    : in     RD_TYPE;
     INTENT      : in     INTENT_ARRAY      := (i => READ);
     KIND        : in     NKI_TYPE;
     TIME_LIMIT  : in     CAIS_DURATION     := LONG_DELAY;
     STATUS      : in     STATUS_INDICATOR  := NO_INDICATION)
is
begin
    OPEN (NODE, BASE, KEY, RELATION, INTENT, TIME_LIMIT, STATUS);
    begin
        ADOPT_INTERPRETATION (NODE, KIND, STATUS);
    exception
        when others =>
            CLOSE (NODE);
            raise;
    end;
end OPEN;


procedure OPEN
    (NODE        : in out NODE_TYPE;
     BASE        : in     NODE_TYPE;
     KEY         : in     RELATIONSHIP_KEY  := EMPTY_KEY_LIST;
     RELATION    : in     RELATION_NAME     := DEFAULT_RELATION;
     INTENT      : in     INTENT_ARRAY      :=    (1 => READ);
     TIME_LIMIT  : in     CAIS_DURATION     := LONG_DELAY;
     STATUS      : in     STATUS_INDICATOR  := NO_INDICATION)
is
    RELATION_DEFINITION: RD_TYPE;
begin
    DEFINITION_OF_RELATION (RELATION_DEFINITION, BASE, RELATION, STATUS);
    OPEN (NODE, BASE, KEY, RELATION_DEFINITION,
          INTENT, TIME_LIMIT, STATUS);
    CLOSE (RELATION_DEFINITION);
exception
    when others =>
        CLOSE (RELATION_DEFINITION);
        raise;
end OPEN;
```


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Notes:

An open node handle acts as if the handle forms an unnamed temporary secondary relationship to the node; this means that, if the node identified by the open node handle is renamed (potentially by another process), the open node handle tracks the renamed node. 25

It is possible to open a node handle to an unobtainable node or to an inaccessible node. The latter is consistent with the fact that the existence of a relationship emanating from an accessible node to which the user has READ_RELATIONSHIPS rights cannot be hidden from the user. 26

### 5.2.5 Opening a node handle using a unique identifier

```
procedure OPEN_VIA_UNIQUE_IDENTIFIER
    (NODE       : in out NODE_TYPE;
     IDENTIFIER : in     UNIQUE_IDENTIFIER;
     INTENT     : in     INTENT_ARRAY     := (1 => READ);
     TIME_LIMIT : in     CAIS_DURATION    := LONG_DELAY;
     STATUS     : in     STATUS_INDICATOR := NO_INDICATION);
```

Purpose:

This procedure returns an open node handle in NODE to the node identified by the unique identifier IDENTIFIER. INTENT specifies the access rights available for subsequent uses of the node handle; it also establishes access synchronization with other users of the node. TIME_-LIMIT specifies a time limit for the delay imposed on OPEN_VIA_UNIQUE_IDENTIFIER by existing locks on the node. A delayed OPEN_VIA_UNIQUE_IDENTIFIER call completes after the node is unlocked or the specified time limit has elapsed. In the latter case, the exception LOCK_ERROR is raised. 1

When a node is opened via its unique identifier, the node is opened without traversing any relationships or checking access rights to any other node; hence, the subprograms PATH_KEY (Section 5.2.20 on page 119) and PATH_RELATION (Section 5.2.21 on page 120) cannot be applied to the node handle NODE. 2

Parameters:

**NODE** is a node handle, initially closed, to be opened to the identified node. 3

**IDENTIFIER** is the unique identifier identifying the node to be opened. 4

**INTENT** is the intent of subsequent operations on the node; the actual parameter takes the form of an array aggregate. 5

**TIME_LIMIT** is the maximum time to wait for the unlocking of the identified node in accordance with the desired INTENT. 6

**STATUS** is an established status indicator. 7

Exceptions:

**SYNTAX_ERROR** is raised if the IDENTIFIER given is a syntactically illegal unique identifier (see Section 5.1 on page 88) [1]. 8


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

**NAME_ERROR** is raised if the node to which the handle is to be opened is non-existing, inaccessible or unobtainable [2].

**USE_ERROR** is raised if the specified INTENT is an empty array [1].

**STATUS_ERROR** is raised if the node handle NODE is already open at the time of the call on OPEN_VIA_UNIQUE_IDENTIFIER [1] or if STATUS is not an established status indicator [3].

**LOCK_ERROR** is raised if the OPEN_VIA_UNIQUE_IDENTIFIER operation is delayed beyond the specified TIME_LIMIT due to the existence of locks on the node in conflict with the specified INTENT [1]. LOCK_ERROR may be raised prior to expiration of the time limit if the CAIS implementation can determine that a deadlock situation has occurred.

**ACCESS_VIOLATION** is raised if the discretionary access control rights of the current process are insufficient to obtain access to the node consistent with the specified INTENT [1].

**SECURITY_VIOLATION** is raised if the attempt to obtain access to the node with the specified INTENT represents a violation of mandatory access controls.

Notes:

Using OPEN_VIA_UNIQUE_IDENTIFIER, it is not possible to open a node handle to a non-existing, unobtainable or inaccessible node. This is consistent with the fact that using the CAIS interfaces it is not possible to distinguish between non-existing, unobtainable and inaccessible nodes.

### 5.2.6 Closing a node handle

```
procedure CLOSE
    (NODE   : in out NODE_TYPE;
     STATUS : in     STATUS_INDICATOR := NO_INDICATION);
```

Purpose:

This procedure severs any association between the node handle NODE and the node and releases any associated lock on the node imposed by the intent of the node handle NODE. Closing an already closed node handle has no effect. If there are any open file handles associated with the node handle, the file handles are also closed.

Parameters:

**NODE** is a node handle, initially open, to be closed.

**STATUS** is an established status indicator.

Exceptions:

**STATUS_ERROR** is raised if STATUS is not an established status indicator [3].


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Notes:

A NODE_TYPE variable must be closed before another OPEN can be called using the same NODE_TYPE variable as an actual parameter to the formal NODE parameter of OPEN.

### 5.2.7 Changing the intent regarding node handle usage

```
procedure CHANGE_INTENT
    (NODE       : in out NODE_TYPE;
     INTENT     : in     INTENT_ARRAY;
     TIME_LIMIT : in     CAIS_DURATION     := LONG_DELAY;
     STATUS     : in     STATUS_INDICATOR  := NO_INDICATION);
```

Purpose:

This procedure changes the intent regarding use of the node handle NODE. It is semantically equivalent to (as a single operation) closing the node handle and reopening the node handle to the same node with the INTENT and TIME_LIMIT parameters of CHANGE_INTENT, except that CHANGE_INTENT guarantees to return an open node handle that refers to the same node as the node handle input in NODE (see the issue explained in the note below).

Parameters:

**NODE** is an open node handle.

**INTENT** is the intent of subsequent operations on the node; the actual parameter takes the form of an array aggregate.

**TIME_LIMIT** is the maximum time to wait for the unlocking of the node in accordance with the desired INTENT.

**STATUS** is an established status indicator.

Exceptions:

**NAME_ERROR** is raised if the node handle NODE identifies an unobtainable or inaccessible node and INTENT contains any intent specification other than NO_ACCESS [1].

**STATUS_ERROR** is raised if the node handle NODE is not an open node handle [2] or if STATUS is not an established status indicator [3].

**USE_ERROR** is raised if there are open file handles associated with the node and the new INTENT differs from the existing intent regarding the contents of the node identified by NODE [2] or if the specified INTENT is an empty array [I].

**LOCK_ERROR** is raised if the operation is delayed beyond the specified TIME_LIMIT due to the existence of locks on the node in conflict with the specified INTENT [1]. LOCK_-ERROR may be raised prior to expiration of the time limit if the CAIS implementation can determine that a deadlock situation has occurred.

**ACCESS_VIOLATION** is raised if the discretionary access control rights of the current process are insufficient to obtain access to the node consistent with the specified INTENT [1].


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

**SECURITY_VIOLATION** is raised if the attempt to obtain access consistent with the specified INTENT to the node specified by NODE represents a violation of mandatory access controls.

Notes:

Use of the sequence of a CLOSE and an OPEN operation instead of a CHANGE_INTENT operation cannot guarantee that the same node is opened, since relationships, and therefore the node identification, may have changed since the previous OPEN on the node.

### 5.2.8 Examining the open status of a node handle

```
function IS_OPEN
    (NODE   : in     NODE_TYPE;
     STATUS : in     STATUS_INDICATOR := NO_INDICATION)
    return BOOLEAN;
```

Purpose:

This function returns TRUE if the node handle NODE is open; otherwise, it returns FALSE.

Parameters:

**NODE** is a node handle.

**STATUS** is an established status indicator.

Exceptions:

**STATUS_ERROR** is raised if STATUS is not an established status indicator [3].

### 5.2.9 Querying the intention of a node handle

```
function INTENT
    (NODE   : in     NODE_TYPE;
     STATUS : in     STATUS_INDICATOR := NO_INDICATION)
    return INTENT_ARRAY;
```

Purpose:

This function returns the intent with which the node handle NODE is open.

Parameters:

**NODE** is an open node handle.


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

**STATUS** is an established status indicator.

Exceptions:

**STATUS_ERROR** is raised if the node handle NODE is not open [2] or if STATUS is not an established status indicator [3].

### 5.2.10 Querying the kind of a node

```
function KIND_OF_NODE
    (NODE   : in  NODE_TYPE;
     STATUS : in  STATUS_INDICATOR := NO_INDICATION)
    return NODE_KIND;
```

Purpose:

This function returns the kind (either FILE, PROCESS or STRUCTURAL) of the node identified by the open node handle NODE. This query indicates the value of the KIND attribute of the node. The query does not, however, constitute an access to the node and it is possible to query the kind of inaccessible and unobtainable nodes. The kind of the target node of any relationship is specified by the NODE_KIND attribute of the relationship. The kind of a node is also implicit in any open node handle to that node.

Parameters:

**NODE** is an open node handle.

**STATUS** is an established status indicator.

Exceptions:

**STATUS_ERROR** is raised if the node handle NODE is not open [2] or if STATUS is not an established status indicator [3].

### 5.2.11 Determining if one node kind is a specialization of another

```
function IS_SUB_KIND
    (KIND1  : in  NKD_TYPE;
     KIND2  : in  NKD_TYPE;
     STATUS : in  STATUS_INDICATOR := NO_INDICATION)
    return BOOLEAN;
```

Purpose:

This function returns TRUE if the NKD identified by KIND1 is the same NKD identified by KIND2 or a specialization of the NKD identified by KIND2. Otherwise, FALSE is returned.


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Parameters:

**KIND1, KIND2** are open node handles which identify NKDs.

**STATUS** is an established status indicator.

Exceptions:

**STATUS_ERROR** is raised if the node handle KIND1 or KIND2 is not open [2] or if STATUS is not an established status indicator [3]. 4

**NODE_KIND_ERROR** is raised if KIND1 or KIND2 does not identify an NKD [1]. 5

**TRANSACTION_ERROR** is raised if the operation is included in more than one transaction [1]. 6

**SECURITY_VIOLATION** is raised if the operation represents a violation of mandatory access controls. 7

Additional Interfaces:

```
function IS_SUB_KIND
    (KIND1 : in      NODE_KIND_NAME;
     KIND2 : in      NODE_KIND_NAME;
     STATUS : in     STATUS_ INDICATOR : = NO_INDICATION)
    return BOOLEAN
is
     KIND_DEFINITION1      : NKD_TYPE ;
     KIND_DEFINITION2      : NKD_TYPE;
     RESULT                : BOOLEAN;
begin
     0PEN_NODE_KIND_DEFINITION (KIND_DEFINITION1, KIND1, STATUS);
     OPEN_NODE_KIND_DEFINITION (KIND_DEFINITION2, KIND2, STATUS);

     RESULT := IS_SUB_KIND (KIND_DEFINITION1,
          KIND_DEFINITION2,
          STATUS);

     CLOSE (KIND_DEFINITION1);
     CLOSE (KIND_DEFINITION2);
     return RESULT;
exception
     when others =>
          CLOSE (KIND_DEFINITION1);
          CLOSE (KIND_DEFINITION2);
          raise;
end IS_SUB_KIND;
```


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

### 5.2.12 Determining if one relation is a specialization of another

```
function IS_SUB_RELATION
    (RELATION1 : in     RD_TYPE;
     RELATION2 : in     RD_TYPE;
     STATUS    : in     STATUS_INDICATOR := NO_INDICATION)
    return BOOLEAN;
```

Purpose:

This function returns TRUE if the RD identified by RELATION1 is the same RD identified by RELATION2 or a specialization of the RD identified by RELATION2. Otherwise, FALSE is returned.

Parameters:

**RELATION1, RELATION2** are an open node handles which identify RDs.

**STATUS** is an established status indicator.

Exceptions:

**STATUS_ERROR** is raised if the node handle RELATION1 or RELATION2 is not open [2] or if STATUS is not an established status indicator [3].

**NODE_KIND_ERROR** is raised if RELATION1 or RELATION2 does not identify an RD [1].

**TRANSACTION_ERROR** is raised if the operation is included in more than one transaction [1].

**SECURITY_VIOLATION** is raised if the operation represents a violation of mandatory access controls.

Additional Interfaces:

```
function IS_SUB_RELATION
    (RELATION1 : in     RELATION_NAME;
     RELATION2 : in     RELATION_NAME;
     STATUS    : in     STATUS_INDICATOR : = NO_INDICATION)
    return BOOLEAN
is
     RELATION_DEFINITION1     : RD_TYPE;
     RELATION_DEFINITION2     : RD-TYPE;
     RESULT : BOOLEAN;
begin
     OPEN_RELATION_DEFINITION (RELATION_DEFINITION1, RELATION1, STATUS);
     OPEN_RELATION_DEFINITION (RELATION_DEFINITION2, RELATION2, STATUS);

     RESULT := IS_SUB_RELATION
```


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

# 1. SCOPE

## 1.1 Purpose

This document provides specifications for a set of Ada [1] packages, with their intended semantics, which together form the set of common interfaces for Ada Programming Support Environments (APSES). This set of interfaces is known as the Common APSE Interface Set (CAIS) and is designed to promote the source-level portability of Ada programs, particularly Ada software development tools. 1

The goal of the CAIS is to promote interoperability and transportability of Ada software across all APSES, including those developed for the commercial marketplace, but primarily for those APSES developed for the Department of Defense (DoD). 2

*Interopembility* is defined as the ability of APSES to exchange database objects and their relationships in forms usable by tools and user programs without conversion. 3

*Transportability* of an APSE tool is defied as the ability of the tool to be installed on a different Kernel Ada Programming Support Environment (KAPSE); the tool must perform with the same functionality in both APSES. Transportability is measured in the degree to which this installation can be accomplished without reprogramming. Portability and transferability are commonly used synonyms. [2] 4

Those programs that are used in support of software and firmware development are referred to as took. This includes the spectrum of support software from project management through code development, configuration management and life cycle support. Tools are not restricted to only those software items normally associated with program generation, such as editors, compilers, debuggers, and linker-loaders. Groups of tools that are composed of a number of independent but interrelated programs (such as a debugger which is related to a specific compiler) are referred to as *tool sets.* [3] The CAIS establishes interface requirements for the transportability of Ada tool sets to be used in DoD APSES. Strict adherence to this interface set will ensure that Ada tools and tool sets will possess a high degree of transportability across conforming APSES. Where tools function as a set, the CAIS facilitates transportability of the tool set as a whole. Individual tools in this set might not be individually transportable because they depend on inputs from other tools in the set. 5

The scope of the CAIS includes interfaces to those services, traditionally provided by operating systems and database management systems, that affect tool transportability. The CAIS is intended to provide the transportability interfaces most often required by common software development tools and includes the following interface areas: 6

a. Node Model. This area presents a model for the CAIS in which contents, relationships and attributes of CAIS entities are defined and manipulated. Also included are facilities for access control, access synchronization, control over the allocation of certain resources, and support for distributed CAIS implementations. 7

[1] ANSI/MIL-STD-1815A 1983.
[2] KAPSE Interface Team: Public Report, Volume I, 1 April 1982; p. Cl.
[3] Requirement for Ada Programming Support Environment, STONEMAN; Department of Defense; Feb. 1980.



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

b. Processes. This area covers program invocation, program control and transactions. 8

c. Input and Output. This area covers file input and output, basic device input and output support, special device control facilities, and interprocess communication. Also included are facilities for the transfer of information between arbitrary CAIS instances in a common external form as well as support for interprocess communication between CAIS instances. 9

d. List Management. This area covers list operations useful for manipulation of parameters and attribute values. 10

e. Pragmatic. This area presents the pragmatic limitations that a conforming CAIS implementation is allowed to impose. 11

f. Status Management. This area covers operations that allow programs to obtain detailed exception and status information. 12

The CAIS as specified in this document is believed to provide the tool writer with significant advantages in tool portability, since the most crucial host dependencies are transparently encapsulated by the interfaces of the CAIS. 13

The CAIS is not a replacement for a host operating system. It standardizes those tool-to-host interfaces that are most crucial for tool portability. Other less frequently used or inherently host-dependent interfaces must complement the CAIS in order to provide a basis for the construction of an APSE. The degree of portability of tools will depend on the degree to which they can obtain the required functionality of host interfaces through the CAIS, rather than through host-dependent interfaces. 14

It is assumed that the reader of this document does not have a detailed knowledge of operating systems concepts, database management systems concepts and the Ada programming language, but is familiar enough with these topics to understand the concepts defined and used herein. 15

## 1.2 Application Guidance

The CAIS has potential application within all APSES, in particular those which are expected to become the primary environments that will support life-cycle software requirements for DoD Mission Critical Computer Systems (MCCS). This interface set is being issued as a military standard in order to encourage its application and utilization within government contracts. It is essential that initial applications focus on the issues of viability, feasibility, implementability, and usability of the interface set as a component of an APSE. Implementations of this interface set that provide sufficient metrics to document and support CAIS utilization will be critical in encouraging wide usage and acceptance of this standard and the CAIS concept in general as a component of APSE architectures. 1

Proper applications of this standard include the following: (1) implementations of the interface set focusing on the metrics and documentation that address vital concerns, e.g. performance, cost, usability, viability, etc., (2) tool implementations writ ten to demonstrate utilization of CAIS interfaces and portability y across two or more host systems, (3) implementations of the interface set within long-term DoD and commercial programming support environments utilized for development and life cycle maintenance of MCCS software, (4) implementations of the interface set that delve into applications beyond the scope of the developmental host application, e.g. run-time systems, bare machines, or new operating systems. 2



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

# 2. REFERENCED DOCUMENTS

## 2.1 Government Documents

Unless otherwise specified, the following specifications, standards, and handbooks of the issue listed in that issue of the Department of Defense Index of Specifications and Standards (DoDISS) specified in the solicitation form a part of this standard to the extent specified herein.

[1815A]: *Reference Manual for the Ada Programming Language,* ANSI/MIL-STD-1815A; United States Department of Defense; 22 January 1983.

[1838]: *Common Ada Programming Support Environment (APSE) Interface Set (CAIS), DOD-STD-*1838 United States Department of Defense; 9 October 1986.

[962B]: *Military Standards, Handbooks, and Bulletins, Preparation of, MIL-STD-962B* United States Department of Defense; 20 May 1988.

[DOD5200.28-STD]: *Department of Defense Trusted Computer System Evaluation Criteria,* United States Department of Defense Computer Security Center, March 1985. (Application for copies should be addressed to Department of Defense, Computer Security Center, Office of Standards and Products, Attention: Chief, Computer Security Standards, Fort George G. Meade, Maryland 20755,)

[RAC86]: *DoD Requirements and Design Criteria for the Common APSE Interface Set (CAIS);* United States Department of Defense; 4 October 1986.

[STONEMAN]: *Requirements for Ada Programming Support Environments, STONEMAN* United States Department of Defense; February 1980.

(Copies of specifications, standards, drawings, and publications required by contractors in connection with specific acquisition functions should be obtained from the contracting activity or as directed by the contracting officer.)

## 2.2 Other Publications

The following documents form a part of the standard to the extent specified herein. Unless otherwise specified, the issue of the documents which are DoD adopted shall be those listed in the DoDISS specified in the solicitation. The issue of documents that have not been adopted shall be those in effect on date of invitation for bids or request for proposal shall apply.

[ANSI 73a]: American National Standards Institute, *Recorded Magnetic Tape for* Information Inter-*change (800 CPI, NRZI (ANSI Standard x3.22-1973, adopted as FIPS PUB 3-1)).* (Application for copies should be addressed to American National Standards Institute, Inc., 1430 Broadway, New York, NY 10018.)

[ANSI 73b]: American National Standards Institute, *Recorded Magnetic Tape for Information In-terchange* (1600 *CPI, PE (ANSI Standard x3.39-1973, adopted as FIPS PUB* 25)). (Application for copies should be addressed to American National Standards Institute, Inc., 1430 Broadway, New York, NY 10018.)

[ANSI 76]: American National Standards Institute, *Recorded Magnetic Tape for Information Inter-change (6250 CPI, Group-coded Recording (ANSI Standard x3.54-1976, adopted as FIPS PUB 50)).*



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

(Application for copies should be addressed to American National Standards Institute, Inc., 1430 Broadway, New York, NY 10018.)

[ANSI 78]: American National Standards Institute, *Magnetic Tape Labels and File Structure for Information Interchange (ANSI Standard z3.27-1978, adopted as FIPS PUB 79).* (Application for copies should be addressed to American National Standards Institute, Inc., 1430 Broadway, New York, NY 10018.) 5

[ANSI 79]: American National Standards Institute, *Additional Controls for Use with American National Standard Code for Information Interchange (ANSI Standard 23,64-1979, adopted as FIPS PUB 86).* (Application for copies should be addressed to American National Standards Institute, Inc., 1430 Broadway, New York, NY 10018.) 6

[ANSI 86]: American National Standards Institute, *American National Standard Code for Information Interchange (ANSI Standard x3.4-1986, adopted as FIPS PUB 1-2).* (Application for copies should be addressed to American National Standards Institute, Inc., 1430 Broadway, New York, NY 10018.) 7

[DACS]: *DACS Glossary, a Bibliography of Software Engineering Terms, GLOS-1,* October 1979, Data and Analysis Center for Software. (Application for copies should be addressed to Data and Analysis Center for Software, RADC/ISISI, Griffiss AFB, NY 13441.) 8

[IEEE]: *IEEE Standard Glossary of Software Engineering Terminology, ANSI/IEEE Std. 729-1983.* (Application for copies should be addressed to Sales Department, American National Standards Institute, 1430 Broadway, New York, NY 10018.) 9

[ISO 76a]: *ISO* 1863, *Information Processing - 9 track, 12.7mm (O.5 in) wide magnetic tape for information interchange recorded at 32 rpmm (800 cpi).* (Application for copies should be addressed to international Standards Organization (ISO), 1 rue de Varembe, Case Postale 56, GH12111 Geneva 20, Switzerland.) 10

[ISO 76b]: *ISO* 3788, *Information Processing - 9 track, 12.7mm (O.5* in) wide *magnetic tape for information interchange recorded at 63 rpmm (1600 cpi) phase encoded.* (Application for copies should be addressed to international Standards Organization (ISO), 1 rue de Varembe, Case Postale 56, GH12111 Geneva 20, Switzerland.) 11

[ISO 84]: *ISO* 5652, *Information Processing - 9 track, 12.7mm (O.5 in)* wide *magnetic tape for information interchange - Format and recording using group coding at 246 cpmm (6250 cpi).* (Application for copies should be addressed to international Standards Organization (ISO), 1 rue de Varembe, Case Postale 56, GH12111 Geneva 20, Switzerland.) 12

[UK Ada Study]: *United Kingdom Ada Study Final Technical Report,* Volume I, London, Department of Industry, 1981. (Application for copies should be addressed to Scientific Information Office, British Defence Staff, British Embassy, 3100 Massachusetts Avenue, NW, Washington, D.C. 20008.) 13

[WEBS]: *Webster's New Collegiate Dictionary* Merriam-Webster Inc., Springfield, Massachusetts, 1985. 14

(Nongovernment standards are generally available for reference from libraries. They are also distributed among technical groups and using Federal agencies.) 15

## 2.3 Order of Precedence

In the event of a conflict between the text of this standard and the references cited herein, the text of this standard shall take precedence. 1



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

## 2.4 Source of Documents

Copies of listed military standards, specifications, and associated documents listed in the Department of Defense Index of Specifications and Standards, are available from the Department of Defense Single Stock Point, Commanding Officer, Naval Publications and Forms Center, 5801 Tabor Avenue, Philadelphia, PA 19120. Copies of industry association documents should be obtained from the sponsoring industry association. Copies of all other listed documents should be obtained from the contracting activity or as directed by the contracting officer.



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

2.4 SOURCE OF DOCUMENTS

REFERENCED DOCUMENTS



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

# 3. DEFINITIONS

The following is an alphabetical list of abbreviations and acronyms used in the description of the CAIS: 1

| | |
|---|---|
| AD | attribute definition node |
| AKD | attribute kind definition node |
| APSE | Ada Programming Support Environment |
| ASCII | American National Standard Code for Information Interchange [ANSI 86] |
| BNF | Backus-Naur Form |
| CAIS | Common APSE Interface Set |
| CEF | common external form |
| CLAKD | composite list attribute kind definition node |
| FAKD | floating point attribute kind definition node |
| FIFO | first-in-fist-out (as in a list or queue) |
| IAKD | integer attribute kind definition node |
| IDAKD | identifier attribute kind definition node |
| IO | input/output |
| KAPSE | Kernal Ada Programming Support Environment |
| LAKD | list attribute kind definition node |
| MCCS | DoD Mission Critical Computer System |
| MLS | multi-level secure |
| NKD | node kind definition node |
| NKI | node kind interpretation node |
| RD | relation definition node |
| RI | relation interpretation node |
| SAKD | string attribute kind definition node |
| TCS | trusted computer system |
| UID | unique identifier |
| ULAKD | uniform list attribute kind definition node |
| VAKD | variant attribute kind definition node |

The following is an alphabetical listing of terms which are used in the description of the CAIS. Where a document named in Section 2 was used to obtain the definition, the definition is preceded by a bracketed reference to that document. Portions of definitions that have been interpreted and tailored to fit the CAIS include the phrase “in the CAIS” as part of that definition. 2

**3.1 abort** To force the termination of a process. In the CAIS, this is done within the bounds of access control. When a process is aborted the value of its CURRENT_STATUS attribute is set to COMPLETED_ABNORMALLY. When an aborted process terminates its CURRENT_STATUS attribute is set to ABORTED. 1

Also, to close a transaction and discard its eligible effects. When a transaction is aborted, any open subtransactions of the transaction are also aborted and any processes created in or enclosed by the transaction which have not terminated are aborted. (See commit.) 2



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

**3.2 access** [DOD5200.28-STD] A specific type of interaction between a subject and an object that results in the flow of information from one to the other. 1

**3.3 access checking** The act of comparing the access rights of a process with those rights necessary to perform the intended operation on a node. Based on this comparison, the intended operation is either permitted or denied. 1

**3.4 access control** [DOD5200.28-STD] All aspects of controlling or synchronizing access to information. 1

**3.5 access rights** Values of GRANT attributes on ACCESS relationships that permit specific kinds of operations to be performed by subjects (processes) on objects. 1

**3.6 access to a node** Opening the node with any intent except NO_ACCESS, creating or deleting relationships emanating from the node, reading or writing the contents of the node, reading or writing of attributes of the node, reading or writing relationships emanating from the node or their attributes. 1

**3.7 accessible** A node is accessible if the current process, as a subject, has sufficient discretionary access rights to have knowledge of the existence of the node as an object and if mandatory access controls permit the current process, as a subject, to have knowledge of the existence of the node as an object. In the CAIS, a node is accessible if (1) the current process has (adopted a role which has) been granted at least the access right EXISTENCE to that node, (2) access has not been explicitly denied, and (3) mandatory access control rules permit the process to have knowledge of the existence of the node. 1

**3.8 activate** To start the execution of a process. When a process is activated, the CURRENT_STATUS attribute of the PROCESS node for the process is set to READY. 1

**3.9 active rendition** In the CAIS_TERMINAL_IO protocol, the rendition applied to all output operations to a given screen. 1

**3.10 active row** In the CAIS_TERMINAL_IO protocol, the row containing the cursor. 1

**3.11 Ada Programming Support Environment (APSE)** [UK Ada Study, STONEMAN] A set of hardware and software facilities whose purpose is to support the development and maintenance of Ada applications software throughout its life cycle with particular emphasis on software for embedded computer applications. The principal features are the database, the interfaces and the tool set. 1

**3.12 adopt a role** The action of a process to acquire the access rights which have been, or will be, granted to an object for adopters of the role associated with a group; in the CAIS, this is accomplished by establishing a secondary ADOPTED_ROLE relationship from a PROCESS node to the GROUP node representing the role. 1

**3.13 adopt a view** The action of a process which allows the process to use the local names defined in a view to translate or form names for definitions and properties of instances of definitions. 1



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

An adopted view is represented by a VIEW node which is the target node of a relationship of the predefine relation ADOPTED_VIEW.

**3.14 adopted node kind interpretation** A node kind interpretation within an adopted view that is associated with a node handle establishing the local names defined by that interpretation and interpretations for generalizations as the local names to be used when identifying properties of the node using that node handle. 1

**3.15 allocatability relationship** An ALLOCATABLE_PROCESSOR, ALLOCATABLE_-MEDIUM, ALLOCATABLE_DEVICE, or ALLOCATABLE_GATEWAY relationship. 1

**3.16 allocatable** The condition of a resource node for which there exists an allocatability relationship emanating from the current PROCESSOR node and terminating at the resource node. This signifies that the resource represented by the resource node can be allocated by processes executing on the processor represented by the current PROCESSOR node. 1

**3.17 alternative of a variant** One of several alternative attribute kind definitions defined by a variant attribute kind definition. 1

**3.18 ambiguous formation** The condition of a composite view where two or more namings are visible that associate the same property with different names. When this condition occurs CAIS operations fail and AMBIGUOUS_VIEW_ERROR is raised. 1

**3.19 ambiguous translation** The condition of a composite view where two or more namings are visible associating the same name for different properties. When this condition occurs CAIS operations fail and AMBIGUOUS_VIEW_ERROR is raised. 1

**3.20 applicable naming** A naming defined in a view is applicable in a context requiring a definition name if it names a definition of the required kind. A naming defied in a view is applicable in the context of a node or relationship if it names a property of that node or relationship. 1

**3.21 approved access rights** Access rights whose names appear in the resulting rights list of any grant item of the GRANT attribute values for which either (1) no necessary right is given or (2) the necessary right names an approved access right (under finite recursive application of this definition). 1

**3.22 assignment of resources** The provision of a resource to an entity for consumption. 1

Assigning a processor resource to a process permits the process to execute on that processor resource. 2

Assigning a medium resource to a node permits the representation of the node to reside on that medium resource. 3

Assigning a device or gateway resource to a process provides the process a channel to the resource. 4

**3.23 attribute** A property, of a node or relationship, which has an associated value. The value of an attribute provides information about the node or relationship such as the kind of the node or the invocation parameters in the case of a PROCESS node. An attribute is defied and identified by an attribute definition. The value of an attribute is modeled as a list. 1



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

**3.24 attribute definition** The definition of an attribute. The attribute defied by each attribute definition is a distinct property that may have an associated instance (an identified value) on certain nodes and relationships. An attribute definition identifies an attribute, specifies the attribute kind definition to which values of that attribute must conform, and places constraints on the use of that attribute.

**3.25 attribute definition node (AD)** A node representing an attribute definition.

**3.26 attribute iteration** The iteration over a set of attributes of a specified node or relation-

**3.27 attribute iterator** See iterator.

**3.28 attribute kind** A class of attribute values which are instances of the same attribute kind definition.

**3.20 attribute kind definition** A specification of the structure of an attribute value and constraints on the component parts of that structure.

**3.30 attribute kind definition node (AKD)** A node representing an attribute kind definition.

**3.31 attribute monitor** An entity (represented by a node) which detects operations that create, modify or delete specific attributes and notifies a process when such operations occur. The attributes monitored belong to a specific collection of nodes and their emanating relationships.

**3.32 attribute monitor event** A notification that a monitored attribute has been created, modified or deleted. An attribute monitor event includes information that identifies the affected attribute.

**3.33 attribute monitor event queue** A queue containing the attribute monitor events that have been posted to an event handler process associated with an attribute monitor but have not been retrieved by that process.

**3.34 available** The condition of a resource node which has the value TRUE for its AVAILABILITY attribute. This signifies that the resource represented by the node is available for use.

**3.35 base node** A node that serves as the starting point, usually for a path element or a pathname.

**3.36 basic node kind definition node** The node kind definition node representing the most general node kind definition. This node kind definition node is conceptually the top of the node kind definition lattice and is unique in that it is not a specialization of any other node kind definition.

**3.37 basic relation definition node** The relation definition node representing the most general relation definition. This relation definition node is conceptually the top of the relation definition lattice and is unique in that it is not a specialization of any other relation definition.



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

**3.38 bidirectional relationship** A relationship representing a two-way interrelation between two entities. In the CAIS node model, a bidirectional relationship is represented by a pair of unidirectional relationships that are created and deleted together, but otherwise are manipulated by the CAIS interfaces in the same manner as non-paired unidirectional relationships. The source node of one of the unidirectional relationships in the pair of unidirectional relationships that represents a bidirectional relationship is the target node of the other unidirectional relationship in the pair, and vice versa.

**3.39 CAIS implementation** The computer software and hardware used to realize the interfaces described in this document.

**3.40 CAIS instance** A collection of nodes conceptually rooted at a single system-level node.

**3.41 CAIS interface** A procedure or function defined within a CAIS package.

**3.42 CAIS-controlled definition** A definition, predefine by the CAIS, whose instances may not be created, modified or deleted through the general-purpose CAIS interfaces for manipulation of nodes, relationships and attributes; except where explicitly noted, special-purpose interfaces must be used to modify such instances. A definition node representing a CAIS-controlled definition has the value TRUE for the attribute CAIS_CONTROLLED.

**3.43 CAIS-controlled instance** An instance of a CAIS-controlled definition.

**3.44 CAIS-determined definition** A CAIS-controlled definition whose instances are manipulated only by the CAIS implementation and cannot be directly influenced via the CAIS interfaces. Constraints and default values cannot be applied to CAIS-determined definitions. A definition node representing a CAIS-determined definition has the value TRUE for the attributes CAIS_CONTROLLED and CAIS_DETERMINED.

**3.45 CAIS-determined instance** An instance of a CAIS-determined definition.

**3.40 cannot be formed** The condition that occurs when the name of a property is to be formed by the CAIS but there is no visible naming for that property in the composite view. When this condition occurs CAIS operations fail and the exception NOT_IN_VIEW_ERROR is raised.

**3.47 cannot be translated** The condition that occurs when the name of a property is to be translated by the CAIS but there is no visible naming for that property in the composite view. When this condition occurs CAIS operations fail and the exception NOT_IN_VIEW_ERROR is raised.

**3.48 canonical list text representation** The result of transforming a list representation to list text.

**3.49 channel** A path for data flow. In the CAIS, two PORT relationships and a CHANNEL node are used to represent a channel.

**3.50 channel creation** The act of creating the channel in the node model.

**3.51 channel deletion** The act of deleting the channel in the node model.



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

**3.52 channel pair** A pair of corresponding channels connected to corresponding nodes of a gateway pair. Data transmitted by a process connected to one of the channels can be received by a process connected to the other channel of the pair. 1

**3.53 character cell** In the CAIS_TERMINAL_IO protocol, a position on a screen capable of holding an ASCII character and an associated rendition. 1

**3.54 client node** A node which is or is prospectively transferring data using input and/or output mechanisms. 1

**3.55 client process** The process represented by a client PROCESS node. 1

**3.56 client PROCESS node** A PROCESS node which is a client node. 1

**3.57 close request** An event occurring when: 1

a. one of the client processes calls a protocol CLOSE interface, 2

b. a close request occurs on the opposite channel, where one of the client nodes is a GATEWAY node, or 3

c. a delete request occurs for a channel in the ESTABLISHED or CLOSED states. 4

**3.58 closed node handle** A node handle that is not associated with a particular node. A closed node handle cannot be used to access any node. 1

**3.59 closed protocol** The condition of a protocol after it has been disassociated from a channel. 1

**3.60 closed transaction handle** A transaction handle which does not identify any transaction. 1

**3.61 closing process** The client process which first causes a close request to be recognized for a channel in the ESTABLISHED state. 1

**3.62 commit** To close a transaction and retain its effects. If the transaction is a subtransaction, its effects, included nodes and locks are transferred to its parent transaction. (See abort.) 1

**3.63 common external form (CEF)** A machine-independent, external representation facilit sting the transfer of node model information between arbitrary CAIS instances. 1

**3.64 completed** The state of a process when its main program (in the sense of [1815A] 10.1) has terminated (in the sense of [1815A] 9.4) but not all processes spawned by that process have terminated. 1

**3.65 completed abnormally** The state of a process when it becomes completed by being aborted. The value of the CURRENT_STATUS attribute of such a process is set to COMPLETED_ABNORMALLY. 1



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

**3.66 completed normally** The state of a process when it becomes completed by normal termination of its main program or by abandonment. The value of the CURRENT_STATUS attribute of such a process is set to COMPLETED_NORMALLY. 1

**3.67 components** The list items of an attribute value that conforms to a uniform list or composite list attribute kind definition. 1

**3.68 composite list** A named or unnamed list of heterogeneous components which conforms to a composite list attribute kind definition. 1

**3.69 composite view** The collection of adopted views of a process and the precedence of those views determined by the order in which they were adopted. 1

**3.70 connected** When a channel has been created, the client nodes, the CHANNEL node, the PORT relationships, and, conceptually, the channel, are said to be connected to each other. 1

**3.71 constraint item** A named item in a constraints list that imposes a particular constraint on the attribute values allowed by an attribute kind definition. The name of a constraint item indicates the type of constraint imposed: 1

a. alternative constraint item – constrains or eliminates alternatives of variant attribute kinds. 2

b. component constraint item – constrains components of uniform or composite list attribute values. 3

c* naming constraint item – constrains the components of a uniform or composite list to be named or unnamed. 4

d. pat terns constraint item – constrains string and identifier attribute values to match lexical patterns, 5

e. range constraint item – constrains integer or float attribute values to fall within a specified bound. 6

f. uniform list size constraint item – constrains the number of components in a uniform list attribute value. 7

g. value constraint item – constrains an attribute to a specific value. 8

**3.72 constraints list** A value for the CONSTRAINTS attribute. A constraints list imposes further constraints on the attribute values allowed by an attribute kind definition. Constraints lists are composed of named constraint items. 1

**3.73 content descriptor** In an external node set, the named list at the start of a content section identifying the node whose contents is being transferred. 1

**3.74 content section** In an external node set, the second through last sections in the sequence of sections. Each content section is used to transmit the contents of a single text FILE node. 1



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

**3.75 contents** A file or process associated with a CAIS node. In the CAIS, a node is said to contain its contents.

**3.76 controlling protocol** The protocol which controls data transfer over a channel.

**3.77 corresponding definition node** The node kind definition node or relation definition node whose properties are named by a given node kind interpretation node or relation interpretation node, respectively. The corresponding definition node is the target of a DEFINITION relationship emanating from the interpretation node.

**3.78 corresponding reverse relationship** The opposite unidirectional relationship of a bidirectional relationship pair with respect to one of the unidirectional relationships comprising that pair.

**3.70 created** The state of a process which has been created but not necessarily activated. The value of the CURRENT_STATUS attribute of a process which has been created but not activated or aborted is CREATED.

**3.80 creating process** In the context of channel creation, the process that causes the channel to be created.

**3.81 current adopted interpretation** The node kind interpretation most recently adopted for a given node handle.

**3.82 current job** The PROCESS node tree containing the current PROCESS node; the root PROCESS node of this tree is the target node of secondary CURRENT_JOB relationships emanating from each PROCESS node in the tree.

**3.83 current linear list** A linear list, within a list structure, to which the linear list manipulations implicitly refer. Exactly one current linear list is associated with each LIST_TYPE value.

**3.84 current node** The node relative to which pathnames beginning with a relationship key designator are interpreted; this node is the target node of a secondary CURRENT_NODE relationship emanating from the current PROCESS node.

**3.85 current process** The currently executing process making the call to a CAIS interface. Pathnames are interpreted in the context of the current process and its current node.

**3.86 current PROCESS node** The PROCESS node representing the current process.

**3.87 current processor** The processor resource upon which the current process is executing, i.e., to which the current process is assigned. It is represented by the target node of a CURRENT_-PROCESSOR relationship emanating from the current PROCESS node.

**3.88 current PROCESSOR node** The PROCESSOR node to which the current process is assigned.



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

**3.89 current user** The user represented by the top-level USER node which is the target node of the secondary relationship of the predefine relation CURRENT_USER emanating from the current PROCESS node.

**3.90 current USER node** The USER node representing the current user.

**3.91 cursor position** In the CAIS_TERMINAL_IO protocol, the position on a screen at which the next operation will be performed if no explicit start position is given.

**3.92 data stream** See stream.

**3.93 data transfer protocol** A method defined by the CAIS for transmitting data between nodes.

**3.94 default GROUP node** The GROUP node representing a role that is automatically adopted when a job or process is created. A default GROUP node is the target of a secondary relationship of the predefine relation DEFAULT_ROLE from either (1) a top-level USER node or (2) a node that represents the executable image of a program. No node may have multiple default GROUP nodes.

**3.95 default rendition** In the CAIS_TERMINAL_IO protocol, a rendition guaranteed to be supported by a particular terminal device returned when a terminal file handle is opened.

**3.96 definition** A definition defines the common characteristics shared by all nodes, relationships, and attributes associated with that definition. In the CAIS node model, each node, relationship and attribute is associated with a definition. Definitions regulate the creation and manipulation of instances via the CAIS interfaces. All CAIS interfaces ensure that nodes, relationships, and attributes remain in conformance with their associated definitions.

**3.97 definition node** A node which represents a definition.

**3.98 delete request** A n event occurring when:

a. one of the client processes or the remote client process calls RELEASE_CHANNEL,

b. one of the client processes or the remote client process aborts or terminates, or

c. the channel must be deleted due to a problem identified in the underlying implementation.

**3.99 dependent process** A process other than a root process.

**9.100 descendant** For a process, a dependent process created by the process or a descendant of such a process. A process and its descendants form a tree connected by primary relationships.

For a transaction, a transaction's subtransaction or one of a subtransaction's descendants,

**3.101 device** [WEBS] A piece of equipment or a mechanism designed to serve a special purpose or perform a special function.


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

**3.102 device name** The key of a primary relationship of the predefined relation DEVICE. 1

**3.103 DEVICE node** A node representing a device resource, which may be used to identify, allocate and manipulate the device. 1

**3.104 device resource** An input and/or output device. 1

**3.105 direct operation** In the CAIS_TERMINAL_IO protocol, an operation to a screen where a cursor position is explicitly specified in the interface. 1

**3.106 direct scenario** The input and output connection scenario where the channel must exist prior to the beginning of the scenario. 1

**3.107 discretionary access control** [DOD5200.28-STD] A means of restricting access to objects based on the identity of subjects and/or groups to which they belong. The controls are discretionary in the sense that a subject with a certain access permission is capable of passing that permission (perhaps indirectly) onto any other subject. 1

**3.108 element** (of a file) A value of the generic data type with which an input and output package was instantiated (see [1815A] 14.1, 14.2, 14.2.2, and 14.2.4). 1

**3.109 eligible effects** Those effects of a CAIS operation which can be discarded as if the operation never took place. Only effects on nodes are eligible. Only the eligible effects of operations included in a transaction are discarded when a transaction is aborted. 1

**3.110 emanating relation** A property, defined by a node kind definition, that permits relationships of a given relation to emanate from nodes of the defined node kind. The emanating relations of a node kind definition are inherited by its specializations. 1

**3.111 empty list** A linear list that contains no items. It is not considered to be either a named list or an unnamed list. 1

**3.112 enclosing transaction** For a given process, a transaction that is associated with the process when the process is created. All transactions opened by the process are descendants of the enclosing transaction; all operations on nodes performed by the process which are not included in any transaction opened by the process are included in the enclosing transaction; and, all nodes opened by the process are included in the enclosing transaction. 1

**3.113 enumeration form** An external representation of the value of an identifier item in a list in the form of an Ada enumeration value whose image matches the identifier text form of the item under identifier equality. 1

**3.114 established status indicator** A status indicator which has been initialized to acquire status or has the implementation-defined value NO_INDICATION specifying that status information is not to be acquired. 1

**3.115 event handler process** A process associated with a MONITOR node which is notified 1


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

when a monitored attribute is created, modified or deleted by posting an event to its event queue.

**3.116 exportation** When used in the context of the package CAIS_IMPORT_EXPORT, exportation means the act of copying the contents of a SECONDARY_STORAGE node to a host system file. 1

When used in the context of the package CAIS_COMMON_IO, exportation means the act of describing a node set and transforming the information captured by the node set to the common external form. 2

**3.117 external file** (see [1815A] 14.1(1) based on the definition of an Ada external file) Values input from the external environment of the program, or output to the environment, are considered to occupy external files. An external file can be anything external to the program that can produce a value to be read or receive a value to be written, e.g., a device or a host system file. 1

**3.118 external node set** Node set information represented in the common external form. 1

**3.119 feasible CHANNEL NKD subset** The set of CHANNEL NKDs for which nodes of those kinds can be legal targets of given PORT relationships. 1

**3.120 file** See external file. 1

**3.121 file handle** An Ada object of type FILE_TYPE which is used to identify an internal file. 1

**3.122 FILE node** A node that represents an Ada external file. All such nodes are of the predefine FILE node kind. 1

**3.123 frame** In the CAIS_FRAME_IO protocol, a segment of a data stream. 1

**3.124 GATEWAY node** A node representing a gateway resource. 1

**3.125 gateway pair** Two GATEWAY nodes which reside in distinct CAIS instances, each representing the other node for the purpose of supporting remet e interprocess communication. 1

**3.126 gateway resource** A mechanism used to support communication with another CAIS instance. 1

**9.127 generalization** A definition which defines properties and constraints that are inherited by another definition. A definition is a generalization of another (more specialized) definition if there is a path, consisting of GENERALIZATION_OF relationships, which starts at the definition node representing the generalization and ends at the definition node representing the more specialized definition. 1

**3.128 group** A set of users; in the CAIS, a group is represented by a GROUP node. 1

**3.129 group name** The key of a primary GROUP relationship emanating from the system-level node or a secondary GROUP relationship emanating from a PROCESS node. 1

**3.130 GROUP node** A STRUCTURAL node representing a group. This node may have 1



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

emanating POTENTIAL_MEMBER and PERMANENT_MEMBER relationships which terminate at other GROUP nodes.

**3.131 header section** In an external node set, the first section. This header section contains a description of each node in a node set without explicitly describing file contents. 1

**3.132 identification of an attribute** A means to identify an attribute. In the CAIS, an attribute may be identified by the identification of a node or relationship together with either the local name of the attribute (in the context of that node or relationship) or an open handle to the definition node for the attribute. 1

**3.133 identification of a node** A means to identify a node. In the CAIS, a node may be identified by a open node handle to the node, by the unique identifier of the node, by a pathname or by an open handle to a base node and the identification of a relationship emanating from that base node which terminates at the node. 1

**3.134 identification of a relationship** A means to identify a relationship. In the CAIS, a relationship is identified by an open handle to the base node from which it emanates, either the relation name or an open node handle to the RD for the relation and a relationship key list. 1

**3.135 identifier equality** In package CAIS_LIST_MANAGEMENT, identifier equality holds among tokens and identifier text forms when the tokens and/or identifier text forms designate the same identifier value. 1

**3.136 identifier text form** An external representation of the value of an identifier item in a list in the form of a string. 1

**3.137 illegal identification** A node identification in which the unique identifier, the pathname, the relationship key list, or relation name is syntactically illegal. 1

**3.138 importation** When used in the context of the package CAIS_DEFINITION_MANAGEMENT, importation means the creation of relationships from a VIEW node which define local names for previously existing definition nodes. Creating such namings for NKDs and RDs includes the creation of corresponding NKIs and RIs. The definition nodes are said to have been imported into the view containing the new namings. 1

When used in the context of the package CAIS_IMPORT_EXPORT, importation means the act of copying a host system file into the contents of a SECONDARY_ STORAGE node. 2

When used in the context of the package CAIS_COMMON_IO, importation means the act of reading an external node set and recreating the node set information in the node model of a CAIS instance. Such importation takes place in two stages: first external nodes are introduced as nodes of an intermediate form and then the introduced nodes are integrated into the CAIS instance. 3

**3.139 inaccessible** A node is inaccessible if the current PROCESS node does not have sufficient discretionary access control rights to have knowledge of the node's existence or if mandatory access controls prevent information flow from the node to the current process. 1

**3.140 include** In the context of transactions, include means to designate a node as part of the 1


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

domain of a transaction. Operations subsequently performed on an included node may be added to the sequence of operations that comprise the transaction (unless they may be similarly added to a descendant of the transaction). Such operations are also said to be included within the transaction.

When used in the context of the package CAIS_COMMON_IO, include means to add a node to a node set.

**3.141 incoming queue** A queue, internal to the CAIS_FRAME_IO protocol, from which a process reads frames. The incoming queue of one client to a channel established for CAIS_FRAME_IO is the outgoing queue of the opposite client.

**3.142 inheritable** A property of a secondary relationship that describes whether or not it is copied when the node from which it emanates is copied. Also, a property of a secondary relationship emanating from a creating PROCESS node that describes whether or not it is copied, upon process creation, to emanate from a newly created PROCESS node. In the CAIS, the Boolean-valued INHERITABLE attribute on relationships establishes whether or not the relationships are inheritable.

**3.143 initiate** To place a program into execution; in the CAIS, this means a PROCESS node is created, a process is created as its contents, required resources are allocated to the process, and the process is activated.

**3.144 initiated process** The process whose program has been placed into execution.

**3.145 initiating process** The process placing a program into execution.

**3.146 input and output connection model** The CAIS mechanisms for controlling channels.

**3.147 input event** In the CAIS_TERMINAL_IO protocol, an action performed on a keyboard or locator.

**3.148 input event queue** In the CAIS_TERMINAL_IO protocol, a queue of input events performed on a particular terminal.

**3.149 instance** An entity created in conformance with a given definition. The entity is said to be an instance of the definition.

**3.150 instance iteration** The iteration over a set of entities that are instances of a given definition.

**3.151 integration** The second stage of importation. The information inherent in the introduced nodes is integrated properly into the node model of an importing CAIS instance.

**3.152 intent** A declaration of the intended access to a node using a given node handle. Intent parameters are provided for interfaces which open node handles or change the current intent associated with a node handle. Intents are used to establish the access rights required for intended operations on a node and to synchronize concurrent access to a node.



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

**3.153 interface** [DACS] A shared boundary.

**3.154 internal file** A file which is internal to a CAIS process. Such a file is identified by a file handle.

**3.155 interoperability** The abifity of APSEs to exchange database objects and their relationships in forms usable by tools and user programs without conversion.

**3.156 interpretation** A component of a view which provides names for properties of a given node kind or relation.

**3.157 interpretation node** A node representing an interpretation for a node kind or relation in a view. The names defined by an interpretation are represented by emanating relationships which terminate on definition nodes for properties.

**3.158 introduced attribute** An attribute on an introduced node or introduced relationship representing an attribute of a corresponding node or relationship of an exporting CAIS instance.

**3.159 introduced contents** The contents of an introduced node reflecting the contents of a corresponding node of an exporting CAIS instance.

**3.160 introduced node** A node in the importing CAIS instance whose properties represent the information inherent in an exported node represented in an external node set.

**3.161 introduced node set** A set of introduced nodes, introduced relationships, introduced attributes and introduced contents representing the information inherent in an external node set.

**3.162 introduced relationship** A relationship between introduced nodes simulating a relationship between two nodes of an exporting CAIS instance.

**3.163 introduction** The first stage of importation. Node set information written in the common external form is read in but the information is not fully integrated into the node model of the importing CAIS instance.

**3.164 item name** A name that may be associated with a list item.

**3.165 item value** A value associated with a list item. There are five kinds of values that an item can have: string, integer, floating point number, identifier and linear list.

**3.166 iteration** The sequential identification of the elements in a set of nodes, relationships or attributes.

**3.167 iteration predicate** A Boolean valued expression that is evaluated for each object (node or relationship) which is eligible for inclusion in an iteration. If the predicate evaluates to TRUE, the object is included in the iteration; otherwise, the object is omitted.

**3.168 iterator** A variable which provides the bookkeeping information necessary for iteration



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

over the relationships emanating from a base node (a node iterator), the attributes of a node or relationship (an attribute iterator) or the nodes of a given kind (a node kind instance iterator).

**3.169 job** A process node tree, spanned by primary relationship(s), which develops under a root PROCESS node as other (dependent) processes are initiated for the user.

**3.170 key** See relationship key.

**3.171 key value association** An element of a relationship key list that associates a key value with a given relationship key identified by a key name.

**3.172 latest key** The final part of a key value that is automatically assigned lexicographically following all previous key values for the same relation with the same initial character sequence emanating from a given node. Represented by the character "#".

**3.173 lattice** A hierarchical structure composed of definition nodes which establishes the inheritance, by a given definition, of properties defined by other definitions. In the CAIS node model, a lattice is formed by secondary SPECIALIZATION_OF relationships and corresponding reverse GENERALIZATION_OF relationships. The CAIS node model defines two definition lattices: one containing node kind definition nodes and another containing relation definition nodes.

**3.174 line** In the CAIS_TERMINAL_IO protocol, a single row of character cell positions.

**3.175 linear list** A linearly ordered set of data elements called list items.

**3.176 list** [IEEE] An ordered set of items of data. In the CAIS, a list is an entity of type LIST-TYPE whose value is a linearly ordered set of data elements which may be empty.

**3.177 list item** A data element in a list.

**3.178 local client node** The client node which is the main subject of discussion; the client node which is the current PROCESS node; the client node which is in the local CAIS instance.

**3.179 local name** A name defined by a view which can be used to identify a definition node or a property of a node or relationship. Local names are defined by the keys of ATTRIBUTE_KIND, ATTRIBUTE, NODE_KIND, RELATION, ATTRIBUTE_NAME, KEY_NAME and RELATION_NAME relationships in a view.

**3.180 local PORT relationship** The PORT relationship emanating from the local client node.

**3.181 mandatory access control** [DOD5200.28-STD] A means of restricting access to objects based on the sensitivity (as represented by a label) of the information contained in the objects and the formal authorization (i.e., clearance) of subjects to access information of such sensitivity y.

**3.182 maximum cell** In the CAIS_TERMINAL_IO protocol, the cell located in the maximum row and maximum column of a screen.



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

**3.183 maximum column** In the CAIS_TERMINAL_IO protocol, the right-most column of a screen. 1

**3.184 maximum row** In the CAIS_TERMINAL_IO protocol, the bottom-most row of a screen. 1

**3.185 MEDIUM node** A node representing a medium resource. 1

**3.186 medium resource** A secondary storage volume used to store nodes, relationships, attributes and FILE node contents. 1

**3.187 message** In the CAIS_FRAME_IO protocol, a frame with its associated priority. 1

**3.188 migration** The conversion of a node of a given node kind to another node kind or the conversion of a relationship of a given relation to another relation. 1

**3.189 minimum cell** In the CAIS_TERMINAL_IO protocol, the cell located in the minimum row and minimum column of a screen. 1

**3.190 minimum column** In the CAIS_TERMINAL_IO protocol, the left-most column of a screen. 1

**3.191 minimum row** In the CAIS_TERMINAL_IO protocol, the top-most row of a screen. 1

**3.192 MONITOR node** A node representing an attribute monitor. Nodes whose attributes are monitored are indicated by secondary MONITORS relationships which emanate from the MONITOR node. The ATTRIBUTES_MONITORED attribute of those relationships identifies the specific node and relationship attributes that are monitored. 1

**3.193 name** An identifier by which a thing is known. 1

**3.194 name formation** The use of a view to find a name for a definition or property. 1

**3.195 name translation** The use of a view to find the definition or property identified by a name. 1

**3.196 named item** A list item that has a name associated with it, i.e., a list item that has an item name. 1

**3.197 named list** A non-empty linear list that contains only named items. 1

**3.198 naming** An association, defined by a view, between a name and a definition or property. 1

**3.199 necessary right** An access right which is a necessary condition for the approval of associated resulting rights. A necessary right appears before a “=>” symbol in a GRANT attribute value and acts as a condition for approving the access rights represented by the resulting rights list that follows this symbol. 1



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

**3.200 nested list structure** A list together with all of its nested sublists (and all of their nested sublists, recursively including all the nested sublists).

**3.201 nested sublist** A linear list item of a linear list is called a nested sublist of the linear list containing the list item.

**3.202 node** A representation within the CAIS node model of an entity relevant to the APSE.

**3.203 node handle** An Ada object of type NODE_TYPE which is used to identify a CAIS node; it is internal to a process. A node may be accessed using an open node handle to the node.

**3.204 node iteration** The iteration over a set of relationships emanating from a specified base node.

**3.205 node iterator** See iterator.

**3.206 node kind** A class of nodes of a given kind. In the CAIS node model, a node kind definition node defines such a class of nodes.

**3.207 node kind definition** A specification of the properties of a particular kind of node.

**3.208 node kind definition lattice** The lattice formed by node kind definitions.

**3.209 node kind definition node (NKD)** A node representing a node kind definition.

**3.210 node kind instance iterator** See iterator.

**3.211 node kind interpretation** An interpretation in a view which provides local names for some or all of the attributes and relations that are properties defined by a node kind definition.

**3.212 node kind interpretation node (NKI)** A node representing a node kind interpretat ion.

**3.213 node model** The mechanisms defined by the CAIS for representing entities and their interrelationships. Also an instance of such mechanisms representing a particular set of entities and interrelationships.

**3.214 node set** A set of nodes whose properties constitute the information to be transferred via CEF between CAIS instances.

**3.215 node set handle** An Ada object of type NODE_SET_TYPE which is used to identify a node set.

**3.216 non-existing node** A node which has never been created.

**3.217 non-transparent resource** A resource of the underlying implementation that may be controlled by tools.


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

**3.218 null key** By convention, the value of a key which has no value. The null key is represented within a relationship key list by a key value association in which a key is named but no value is specified.

**3.219 object** [DOD5200.28-STD] A passive entity that contains or receives information. Access to an object potentially implies access to the information it contains. In the CAIS, an object is any node to be accessed.

**3.220 obtainable** A node is obtainable if it is the target node of a primary relationship.

**3.221 open file handle** A file handle that is associated with a particular internal file. A file handle that is not open cannot be used to access any internal file.

**3.222 open node handle** A node handle that is associated with a particular node. A node handle that is not open cannot be used to access any node.

**3.223 open node set handle** A node set handle that is associated with a particular node set. A node set handle that is not open cannot be used to access any node set.

**3.224 open screen handle** In the CAIS_TERMINAL_IO protocol, a screen handle uniquely associated with a particular screen. A screen handle that is not open cannot be used to access a screen.

**3.225 open transaction handle** A transaction handle which is associated with a particular transaction. A transaction handle which is not open cannot be used to identify a transaction.

**3.226 operation** A call to a CAIS interface and the effects of such a call as defined by the semantics of the interface.

**3.227 operation on a node** In the context of a transaction, any operation which accesses the node via an open node handle or opens the node with an intent to access the node; and, any operation which accesses the node but does not access any node via a handle or open any node with an intent to access it.

**3.228 opposite CAIS instance** The CAIS instance that contains the opposite GATEWAY node of a gateway pair.

**3.229 opposite channel** The corresponding channel connected to the opposite GATEWAY node of a gateway pair.

**3.230 opposite client node** The client node to which the local client node is connected.

**3.231 opposite GATEWAY node** For a given GATEWAY node of a gateway pair, the other GATEWAY node in that pair. The opposite GATEWAY node resides within the opposite CAIS instance.

**3.232 opposite PORT relationship** The PORT relationship emanating from the opposite client node.


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

**3.233 outgoing queue** A queue, internal to the CAIS_FRAME_IO protocol, to which a process writes frames. The outgoing queue of one client to a channel established for CAIS_FRAME_IO is the incoming queue of the opposite client.

**3.234 parent** The source node of the primary relationship to a given node; also the target node of the corresponding reverse relationship of the primary relationship to a node.

For a given subtransaction, the transaction which will acquire the effects, included nodes, and locks held by the subtransaction when the subtransaction is committed. The parent transaction for a subtransaction is designated when the subtransaction is opened.

**3.235 parent relationship** The corresponding reverse relationship of the primary relationship to a node. This relationship terminates at the parent node of its source node.

**3.236 pasting** In the CAIS_TERMINAL_IO protocol, the act of placing a screen on a virtual display.

**3.237 pasting order** In the CAIS_TERMINAL_IO protocol, the order of the pasted screens on a virtual display. The screen at the front of the pasting order is entirely visible. Screens further behind in the pasting order may be occluded.

**3.238 path** A sequence of relationships connecting one node to another. Starting from a given node, a path is followed by traversing a sequence of relationships until the desired node is reached.

**3.239 path element** A portion of a pathname representing the traversal of a single relationship; it consists of a relation name and relationship key list.

**3.240 pathname** A name for a path consisting of the concatenation of the local names of the traversed relationships in the path in the same order in which they are traversed.

**3.241 physical cursor** In the CAIS_TERMINAL_IO protocol, a cursor associated with a physical display generally manipulated by a locator.

**3.242 potential member** A group that may dynamically acquire membership in another group; in the CAIS, a GROUP node is termed a potential member of a group if it is reachable from the node representing the group by traversing only relationships of the predefined relations PERMANENT_MEMBER and POTENTIAL_MEMBER.

**3.243 pragmatic** Constraints, imposed by the CAIS or a CAIS implementation, on objects manipulated by the CAIS interfaces. CAIS-defined pragmatic specify minimum capacities required in an implementation. Implementation-defied pragmatic specify the actual capacities of an implementation.

**3.244 primary relationship** The initial relationship established from an existing node to a newly created node during its creation. A node is obtainable and may be accessed only if a primary relationship to the node exists.

**3.245 process** The execution of an Ada program, including all its tasks.



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

**3.246 PROCESS node** A node that contains a CAIS process. Such nodes are instances of the predefine node kind PROCESS or specializations thereof.

**3.247 process tree** For a given process, the set of processes consisting of the given process plus each process whose node's unique primary path traverses the node of the given process, i.e. the descendants of the process.

**3.248 PROCESSOR node** A node representing a processor resource.

**3.249 processor resource** A computer processing unit capable of executing CAIS processes.

**3.250 program** [1815A] A program is composed of a number of compilation units, one of which is a subprogram called the main program.

**3.251 protocol** See data transfer protocol.

**3.252 protocol CLOSE interface** A CLOSE interface found in a protocol package.

**3.253 protocol CLOSE operation** A call to a protocol CLOSE interface.

**3.254 protocol closing** The act of disassociating a protocol from a channel.

**3.255 protocol OPEN interface** An OPEN or CREATE interface found in a protocol package.

**3.256 protocol OPEN operation** A call to a protocol OPEN interface.

**3.257 protocol opening** The act of associating a protocol with a channel.

**3.258 protocol package** Any of the IO packages defined by CAIS which may be used to transfer data or any similar package defined by an implementation for extending CAIS capabilities.

**3.259 protocol parameter** A quantity which controls some aspect of a data transfer protocol.

**3.260 protocol parameter attribute** An attribute of CHANNEL nodes giving the value of a protocol parameter.

**3.261 protocol range attribute** An attribute of PORT relationships giving the range of allowable values of a protocol parameter.

**3.262 queue** [IEEE] A list that is accessed in a first-in, fist-out (FIFO) manner.

**3.263 range** For the CAIS_TERMINAL_IO protocol, logically contiguous character cells on a screen. A range is defined by a starting position and an ending position.

**3.264 ready** The state of a process which is ready to be executed. A process which is ready



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

is executed on a processor. The value of the CURRENT_STATUS attribute of a ready process is READY.

**3.265 relation** A class of relationships representing the same interrelation. In the CAIS node model, a relation definition node defines such a class of relationships.

**3.266 relation definition** A definition of a class of relationships specifying the kinds of nodes the relationships can connect and their attributes and keys.

**3.267 relation definition lattice** The lattice formed by relation definitions.

**3.268 relation definition node (RD)** A node representing a relation definition.

**3.269 relation interpretation** An interpretation of a relation definition in a view which provides local names for some or all of the attributes and keys that are properties defined by the relation definition.

**3.270 relation interpretation node (RI)** A node representing a relation interpretation.

**3.271 relation name** An identifier that identifies a relation.

**3.272 relationship** In the directed graph that forms the CAIS node model, an edge which emanates from a source node and terminates at a target node. A relationship is an instance of a relation. A relationship is either a primary relationship or a secondary relationship.

**3.273 relationship attribute** An attribute possessed by a relationship.

**3.274 relationship key** A relationship attribute that may be used to identify particular relationships of a given relation that emanate from the same node.

**3.275 relationship key designator** An element of a relationship key list which specifies a relationship key value.

**3.276 relationship key list** A relationship key designator or a sequence of key value associations which specifies relationship key values when creating or identifying relationships. Relationship key lists may be used in pathnames and in the base, relation, key method of identification.

**3.277 relationship key value** The value of a relationship key.

**3.278 remote client process** The client process connected to the opposite channel.

**3.279 remote interprocess communication** Data transfer between two processes executing in distinct CAIS instances.

**3.280 rendition** In the CAIS_TERMINAL_IO protocol, a displayed character cell's visual attributes, e.g., foreground and background colors, bold, blinking, underlined.


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

**3.281 reportable event** An attribute monitor event affecting an attribute of an obtainable node to which the event handler process has sufficient mandatory and discretionary access rights to read the attribute at the time that the event is requested from the attribute monitor event queue.

**3.282 resource** A device, gateway, medium, or processor resource.

**3.283 resource group** A node model structure consisting of a RESOURCE_GROUP node, the emanating PROCESSOR, MEDIUM, DEVICE, and GATEWAY relationships, and the resource nodes at which they terminate.

**3.284 resource model** The collection of nodes, attributes, and relationships representing resources of a CAIS instance.

**3.285 resource node** A DEVICE, GATEWAY, MEDIUM, or PROCESSOR node.

**3.286 resulting rights list** A list of approved access rights that can be acquired by a process conditionally upon acquisition of an associated necessary right.

**3.287 role** A role is associated with a GROUP node. It is the set of all ACCESS relationships and all DENY relationships emanating from nodes and terminating at the GROUP node or any of the GROUP nodes reachable recursively from the GROUP node by traversing parent relationships.

**3.288 root PROCESS node** The initial PROCESS node created when a user logs on to an APSE or when a new job is created via the CREATE_JOB interface.

**3.289 screen** In the CAIS_TERMINAL_IO protocol, a rectangular area composed of positions in which character cells are defined.

**3.290 screen handle** In the CAIS_TERMINAL_IO protocol, an Ada object of type SCREEN_TYPE which is used to identify a screen.

**3.291 secondary relationship** A relationship which is established between two existing nodes. (A primary relationship is established from an existing node to a newly created node.)

**3.292 secondary storage file** An external file that represents a disk or other random access storage file.

**3.293 security level** [DOD5200.28-STD] The combination of a hierarchical classification and a set of non-hierarchical categories that represents the sensitivity of information or authorization of a subject.

**3.294 sequential operation** In the CAIS_TERMINAL_IO protocol, an operation to a screen where an explicit starting position is not specified in the interface. During a sequential operation, the cursor position is implied.

**3.295 server scenario** The input and output connection scenario where the scenario may begin prior to the time the channel is created.



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

**3.296 simple attribute kind** A n attribute kind defied by an integer, float, string or identifier AKD.

**3.297 source node** The node from which a relationship emanates.

**3.298 specialization** A definition which inherits properties and constraints from other (more general) definitions. A given node is a specialization of another (more general) definition if there is a path, consisting of relationships of the predefined relation SPECIALIZATION_OF, which starts at the definition node representing the specialization and ends at the definition node representing the more general definition.

**3.299 status indicator** An Ada object of type STATUS_INDICATOR. These objects may be used to acquire status information resulting from CAIS operations.

**3.300 storage unit** The lowest level machine representation, typically an 8-bit byte.

**3.301 stream** In the CAIS_FRAME_IO protocol, an abstraction representing a sequence of storage units.

**3.302 stronger constraint** A constraint, imposed on instances of a definition, that is distinct from other (weaker) constraints and implies those other constraints.

**3.303 STRUCTURAL node** A node without contents. STRUCTURAL nodes are used strictly as holders of relationships and attributes.

**3.304 structured attribute kind** An attribute kind defined by a list, uniform list or corn- posite list AKD.

**3.305 subject** [DOD5200.28-STD] An active entity, generally in the form of a person, process, or device, that causes information to flow among objects or changes the system state. In the CAIS, a subject is any process (acting on behalf of a given user) performing an operation requiring access to an object.

**3.306 subtransaction** A transaction which is included within another transaction, called its parent transaction. When a subtransaction is committed, its effects, included nodes and locks are transferred to its parent transaction.

**3.307 suspended** The state of a process when its execution is stopped such that it can later be resumed. The value of the CURRENT_STATUS attribute of a suspended process is SUSPENDED.

**3.308 system-level node** The root of the CAIS primary relationship tree which spans the entire node structure.

**3.309 target node** The node at which a relationship terminates.

**3.310 task** [1815A] A task operates in parallel with other parts of the program.


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

**3.311 terminated** The state of a process when it has completed and all processes spawned by that process have terminated. When a process that has completed abnormally becomes terminated the value of its CURRENT_STATUS attribute is set to ABORTED. When a process that has completed normally becomes terminated the value of its CURRENT_STATUS attribute is set to TERMINATED. 1

**3.312 terminating relation** A property, defined by a node kind definition, that permits relationships of a given relation to terminate at nodes of the defined node kind. The terminating relations of a node kind definition are inherited by its specializations. 1

**3.313 token form** An external representation of the value of an identifier item in a list in the form of an object of the limited private type TOKEN_TYPE defined in package CAIS_LIST_MANAGEMENT. The object serves as a token for identifier text form of the item. 1

**3.314 tool** [IEEE - soft ware tool] A computer program used to help develop, test, analyze, or maintain another computer program or its documentation. Examples of tools are an automated design tool, compiler, test tool, and maintenance tool. 1

**3.315 tool sets** [STONEMAN] Groups of tools that are composed of a number of independent but interrelated programs (such as a debugger which is related to a special compiler). 1

**3.316 top-level node** A node whose parent is the system-level node. A top-level node may be a STRUCTURAL node representing a user, group, view, processor, gateway or medium, the STRUCTURAL node representing the collection of predefine definitions, or a FILE node representing a device. 1

**3.317 track** An open node handle or a relationship is always guaranteed to refer to the same node, regardless of any changes to (other) relationships that could cause pathnames to become illegal or refer to different nodes. An open node handle is said to track the node to which it refers. Similarly, relationships track their target nodes. 1

**3.318 transaction** A sequence of operations on designated nodes which has the appearance of an atomic operation whose effects can either be retained or discarded as if the sequence of operations never took place. 1

**3.319 transaction handle** An Ada object of type TRANSACTION_TYPE which is used to identify a transaction. A transaction handle may be used to designate a transaction when including nodes in a transaction, when enclosing a process in a transaction, when creating a subtransaction, and when querying or closing a transaction; it is internal to a process. 1

**3.320 transparent resource** A resource that is managed entirely by the underlying implement at ion. 1

**3.321 transportability** The ability of a tool to be installed on a different Kernel Ada Programming Support Environment (KAPSE); the tool must perform with the same functionality in both APSES. Transportability is measured in the degree to which this installation can be accomplished without reprogramming. 1


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

**3.322 traversal of a node** Traversal of a relationship emanating from the node.

**3.323 traversal of a relationship** The act of following a relationship from its source node to its target node.

**3.324 unadopt a role** The act of disowning an adopted role. A process may unadopt any role except the role of the current user and the default role of the current process's executable image.

**3.325 undefined token** A distinguished value of a variable of type TOKEN-TYPE that represents an undefined value for that variable.

**3.326 unidirectional relationship** A representation of a one-way interrelation between a source node and a target node.

**3.327 uniform list** A named or unnamed list in which all items are of a specified attribute kind. Uniform lists conform to a uniform list attribute kind definition.

**3.328 unique identifier** A string which identifies exactly one node within the CAIS instance, and is intended to identify exactly one node within the universe of CAIS instances.

**3.329 unique key set** The set of keys of an RD define a unique key set if the RD has the UNIQUE_KEYS attribute (which must have the value TRUE) and no generalization of the RD has that attribute. The values of the keys in a unique key set are sufficient to uniquely identify each relationship of the relation emanating from a given node. A unique key set is inherited by specializations of the RD which defines the set.

**3.330 unique primary path** The path from the system-level node to a given node traversing only primary relationships. Every node that is obtainable has a unique primary path.

**3.331 unnamed item** A list item that has no name associated with it, i.e., a list item that has no item name.

**3.332 unnamed list** A non-empty linear list that contains only unnamed items.

**3.333 unobtainable** A node is unobtainable if it is not the target node of a primary relationship. An unobtainable node cannot be accessed.

**3.334 unpasting** In the CAIS_TERMINAL_IO protocol, the act of removing a screen from a virtual display.

**3.335 user** An individual, a project, or other organizational entity. In the CAIS, each user is associated with a top-level node.

**3.336 user name** The key of a primary USER relationship.

**3.337 USER node** A top-level STRUCTURAL node which represents a user.



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

**3.338 view** A collection of nodes and relationships which defines names for certain definitions and the properties defined by those definitions.

**3.339 VIEW node** A STRUCTURAL node representing a view.

**3.340 virtual display** In the CAIS_TERMINAL_IO protocol, an abstraction representing a physical display.

**3.341 virtual keyboard** In the CAIS_TERMINAL_IO protocol, an abstraction representing a physical keyboard.

**3.342 virtual locator** In the CAIS_TERMINAL_IO protocol, an abstraction representing a physical locator device such as a mouse or light pen connected to a terminal.

**3.343 virtual terminal** In the CAIS_TERMINAL_IO protocol, an abstraction representing an interactive video terminal that consists of a virtual display, a virtual keyboard, and a virtual locator.

**3.344 visibility relationship** A RESOURCE_GROUP, CURRENT_PROCESSOR or DEVICE relationship emanating from a PROCESS node.

**3.345 visible group** A resource group for which there exists a visibility relationship emanating from the current PROCESS node.

**3.346 visible in a composite view** A naming in a view belonging to a composite view is visible if there is no visible naming for the same name in a higher precedence view.

**3.347 visible in a view** A naming in a view is visible if there is no higher precedence naming for the same name or the same property in the same view.

**3.348 visible resource** A resource represented by a resource node which is either the target of a visibility relationship emanating from the current PROCESS node or a member of a visible group. The current process is allowed to learn of the existence of the visible resource.

**3.349 weaker constraint** A constraint, imposed on instances of a definition, that is distinct from other (stronger) constraints but is implied by those other constraints.



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

# 4. GENERAL REQUIREMENTS

## 4.1 Introduction

The CAIS provides interfaces for data storage and retrieval, data transmission to and from external devices, and activation of programs and control of their execution. In order to achieve uniformity in the interfaces, a single model is used to describe data storage, devices and executing programs. This approach provides a single model for understanding the CAIS concepts; it provides a uniform understanding of and emphasis on data storage, program control and input and output; and it provides a consistent way of expressing interrelations both within and between data and executing programs. This unified model is called the *node model.* 1

Section 4.2 discusses how interfaces are described in the remainder of this document. Section 4.3 describes the node model. Section 4.4 describes the mandatory and discretionary access control model incorporated in the CAIS. Section 4.5 describes input and output. Section 4.6 describes resource control. Section 4.7 describes the provisions for distributed CAIS implementations. Sections 5 through 13 provide detailed descriptions of the interfaces. Section 14 provides information on the intended use of this document and relevant keywords for use by automated document retrieval systems. 2

The following is a list of the CAIS library packages, ordered by section number, and a general description of their contents: 3

**5.1 CAIS_DEFINITIONS** contains widely used definitions related to general node management, including type definitions for node handles, intents and other parameters; declarations of exceptions; declarations of constant pathnames and other constants used as default values for interface parameters. 4

**5.2 CAIS_NODE_MANAGEMENT** contains general purpose interfaces for the manipulation of nodes, relationships and pathnames; for iterations over nodes and emanating relationships; for opening and closing node handles; for copying and deleting nodes; and for creating, modifying and deleting relationships. 5

**5.3 CAIS_ATTRIBUTE_MANAGEMENT** contains interfaces for creating, setting, getting, deleting and iterating over node and relationship attributes. 6

**5.3.14 CAIS_BOOLEAN_ATTRIBUTE** contains interfaces for creating, setting, and getting identifier attributes in Boolean form. 7

**5.4 CAIS_ACCESS_CONTROL_MANAGEMENT** contains interfaces for granting, denying, deleting and examining discretionary access rights; and for adopting and unadopting roles. 8

**5.5 CAIS_STRUCTURAL_NODE_MANAGEMENT** contains interfaces for creating STRUCTURAL nodes. 9

**5.6 CAIS_TRANSLATION_MANAGEMENT** contains interfaces to control the translation of names and to translate between the two fundamental forms of identification (by name and by definition). 10

**5.7 CAIS_DEFINITION_MANAGEMENT** contains interfaces for the creating, modifying, deleting, and examining definition nodes and views. 11



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

**5.8 CAIS_ATTRIBUTE_MONITOR_MANAGEMENT** contains type definitions and interfaces for creating and deleting attribute monitors, for the specification of attributes to be monitored and for awaiting and getting events from monitor event queues. 12

**5.9 CAIS_RESOURCE_CONTROL** contains interfaces for reassigning nodes to different media; for examining the medium to which a node is assigned and the processor to which a process is assigned; and for determining the current default medium and processor resources. 13

**6.2 CAIS_PROCESS_DEFINITIONS** contains type definitions for process status, priority, parameters and results; and declarations of constants and exceptions related to process creation. 14

**6.3 CAIS_PROCESS_MANAGEMENT** contains interfaces for creating, activating, suspending, resuming and aborting processes; for awaiting process termination, managing process priorities, manipulating process results; and for examining various PROCESS node attributes. 15

**6.4 CAIS_TRANSACTION_MANAGEMENT** contains type definitions for transactions; interfaces for creating, committing, aborting and examining transactions; and interfaces for including nodes in a transaction. 16

**7.1 CAIS_IO_DEFINITIONS** contains type definitions for channel states and IO operation waits, constants for IO intents, exceptions related to establishing channels and exceptions common to various 10 protocol packages. 17

**7.2 CAIS_FILE_NODE_MANAGEMENT** contains interfaces for creating SECONDARY_STORAGE nodes. 18

**7.3 CAIS_IO_CONNECTION** contains interfaces which establish IO connections by creating channels between client nodes; and interfaces for releasing and deleting channels, for waiting on channels, for determining the opposite client and for testing the compatibility y of channel ports. 19

**7.4 CAIS_IMPORT_EXPORT** contains interfaces for importing host system files into the CAIS node model as FILE nodes and for exporting the contents of CAIS FILE nodes to the host system. 20

**7.5 CAIS_COMMON_IO** contains interfaces for producing and introducing selected portions of the node model in Common External Form (CEF), including interfaces for creating node sets, for including nodes in node sets and transmitting them as external node sets in CEF; and for introducing nodes from an external node set and integrating such introduced nodes into the node model. 21

**8.2 CAIS_FRAME_IO** is an IO protocol package which includes interfaces for transferring data between clients in buffers called frames; interfaces for opening and closing connections using the protocol; interfaces for reading and writing frames and for querying IO status; and type definitions for these interfaces. 22

**8.3 CAIS_MAGNETIC_TAPE_IO** is an IO protocol package which supports standard magnetic tape IO, including type definitions and interfaces for mounting, dismounting, loading, unloading, opening, closing, positioning, reading, writing and querying status. 23

**8.4 CAIS_DIRECT_IO** is a CAIS-integrated version of the Ada package DIRECT_IO ([1815A] 14.2.4) which includes an interface for the creation of SECONDARY_STORAGE nodes. 24



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

**8.5 CAIS_SEQUENTIAL_IO** is a CAIS-integrated version of the Ada package SEQUENTIAL_IO ([1815A] 14.2.3) which includes an interface for the creation of SECONDARY-STORAGE nodes. 25

**8.6 CAIS_TEXT_IO** is a CAIS-integrated version of the Ada package TEXT_IO ([1815A] 14.3) which includes an interface for the creation of SECONDARY_STORAGE nodes. 26

**8.7 CAIS_TERMINAL_IO** is an IO protocol package for communicating with interactive video terminals which supports multiple overlapping screens with borders, character renditions, foreground and background colors, function keys and an input event queue. 27

**9.1 CAIS_LIST_MANAGEMENT** contains type definitions and interfaces for the manipulation of CAIS lists, interfaces for the creating, deleting and comparing lists; for converting lists to and from text form; and for inserting and extracting list items using item names or positions. 28

**9.1.24 CAIS_BOOLEAN_ITEM** contains interfaces for manipulating identifier items in lists in Boolean form. 29

**10.1 CAIS_STANDARD** contains definitions of scalar types used throughout the CAIS including CAIS_INTEGER, CAIS_NATURAL, CAIS_POSITIVE and CAIS_DURATION. 30

**11.1 CAIS_CALENDAR** is a CAIS-integrated version of the Ada package CALENDAR ([1815A] 9.6) which uses the CAIS-defined types CAIS_INTEGER and CAIS_DURATION instead of the Ada types INTEGER and DURATION. 31

**1201 CAIS_PRAGMATICS** contains constant definitions specifying minimum capacities required in all CAIS implementations and the actual capacities of a particular CAIS implementation. 32

**13.1 CAIS_STATUS_DEFINITIONS** contains type definitions for status indicators, enumerated exception indications corresponding to CAIS exceptions, and integer constants which provide names for the conditions codes associated with exceptional conditions. 33

**13.2 CAIS_STATUS_MANAGEMENT** contains interfaces for creating, copying, setting, deleting and querying status indicators and for deriving error messages from status indicators. 34

**C.70 CAIS_FORM_TERMINAL_IO** is an IO protocol package for communicating with interactive form terminals. 35

The following appendixes provide additional information: 36

**Appendix A** provides descriptions of parts of the CAIS which are predefine. The material contained in this Appendix is a mandatory part of the standard. 37

**Appendix B** provides a set of the Ada package specifications which have been organized for compilation of the CAIS interfaces. The material contained in this Appendix is a mandatory part of the standard. 38

**Appendix C** discusses compatibility of this standard with previous revisions of the CAIS standard. The material contained in this Appendix is a mandatory part of the standard. 39

**Appendix D** summarizes the syntax descriptions given throughout this document. 40

**Appendix E** presents guidelines for CAIS implementations in regard to meeting the discretionary access control requirements of DOD5200.28-STD. 41



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

**Appendix F** describes how and where those aspects of a CAIS implementation which are implementation-dependent must be documented. The material contained in this Appendix is a mandatory part of the standard. 42

**Appendix G** provides a concise listing of all CAIS procedures and functions for quick reference. 43



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

# 4.2 Method of Description

The specifications of the CAIS interfaces are divided into two parts: 1

1. the syntax as defined by a canonical Ada package specification, and 2
2. the semantics as defined by the descriptions both of the general node model and of the particular packages and procedures. 3

The Ada package specifications as given in Appendix B of this document are termed canonical because they are representative of the form of the allowable actual Ada package specifications in any particular CAIS implementation. The packages which together provide an implementation of these specifications must have indistinguishable semantics from those stated herein. 4

## 4.2.1 Allowable differences

The packages which together provide a particular implementation of the CAIS must have the following properties: 1

1. The packages of a particular CAIS implementation are allowed to import additional library units; this may cause name conflicts with the names of library units required by otherwise legal and non-erroneous Ada programs. Barring such name conflicts, however, any Ada program that is legal and not erroneous in the presence of the canonical package specifications as library units must be legal and not erroneous if the canonical packages are replaced by the packages of a particular CAIS implementation. [Note: It is recommended, although not required, that any Ada program that is illegal in the presence of the canonical package specifications as library units is also illegal if the canonical packages are replaced by the packages of a particular CAIS implementation.] 2
2. The CAIS interfaces provided by the subprograms declared in the packages of a particular CAIS implementation must have semantics whose effects are indistinguishable from those in the standard. 3

The actual Ada package specifications of a particular implementation may differ from the canonical specifications as long as these properties are preserved. 4

## 4.2.2 Typographical conventions

This document follows the typographical conventions of [1815A] where these are not in conflict with those of [962 B]. In particular: 1

1. in the Ada specifications, UPPER CASE is used for Ada language identifiers, except for Ada reserved words which are rendered in lower case, 2
2. in the text, syntactic category names are written in normal typeface with any embedded underscores replaced by blanks, 3



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

3. in the text, where reference is made to a value of an Ada object, for example a value of a procedure parameter (see [1815A] 3.2), the Ada name of the object is used in normal typeface. However, where reference is made to an Ada object itself or to any other declared Ada entity, then the Ada name is given in upper case, including any embedded underscores. These two conventions are illustrated in the following example (from [1815A] 14.2.1 paragraphs 17, 18 and 19) by the occurrences of “mode”, “file”, and “STATUS_ERROR”: 4

    function MODE (FILE: in FILE_TYPE) return FILE-MODE; 5

    Returns the current mode of the given file. 6

    The exception STATUS_ERROR is raised if the file is not open. 7

4. at the place where a technical term is defied in the text, the term is given in an *italic* typeface. 8

## 4.2.3 Semantic descriptions

The interface semantics are described in most cases through narrative. These narratives are divided into as many as five paragraphs and appear in the following format. 1

Purpose:

This paragraph describes the semantics of the interface.

Parameters:

**PARAMETER** briefly describes each of the parameters. 3

Exceptions:

**EXCEPTION** briefly describes the conditions under which each exception is raised and the condition code that identifies each condition. The condition codes are represented as integer values or ranges in square brackets (e.g. [6], [3-8]). The condition code and other information concerning the cause of an exception are returned by an interface when an established status indicator parameter is provided in the call. A status indicator is an object of type STATUS_-INDICATOR (see Section 13 on page 825). 4

A general principle not restated for each CAIS interface is that the execution of a CAIS subprogram is atomic with respect to process data and node model data. Normal completion of a call to an interface results in all data modifications as stipulated by the semantics of the interface. Completion of a call through the raising of an exception results in no modification to process or node model data. Any exclusions to this principle are explicitly stated. Two notable exclusions are as follows: 5

1. Detailed status information can be collected for calls to CAIS interfaces, whether such calls complete normally or through an exception. 6
2. Calls to interfaces which suspend, resume or abort processes result in the exception ACCESS_VIOLATION if the calling process does not have sufficient access rights to control all of the designated processes (see Section 6.3 on page 521). However, the respective operation (suspension, resumption or abortion) is applied to those processes to which access can be gained. 7



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Ada language implementations are not required to update the values of parameters of modes OUT or IN OUT when execution of a subprogram is abandoned due to an exception. To ensure that status information is returned when an exception is raised, status indicators are objects which reference the status information collected (rather than objects containing the information) and the mode of status indicator parameters is IN. This indirection also allows status indicator parameters to be used in CAIS interfaces declared as Ada functions, which cannot have parameters of mode OUT or IN OUT. 8

Additional Interfaces:

```
-- In cases where an interface is overloaded and the additional versions
-- can be described in terms of the basic form of the interface and other
-- CAIS interfaces, these versions are described in this paragraph using
-- Ada.  This method of presenting the semantics of the Additional
-- Interfaces is a conceptual model.  It does not imply that the Additional
-- Interfaces must be implemented in terms of the existing ones exactly as
-- specified, merely that their behavior is equivalent to such an
-- implementation.  The semantics described in the Purpose, Parameters and
-- Exceptions paragraphs apply only to the principal interface; the
-- Additional Interfaces may have additional semantics as implied by the
-- given bodies.
```

**Notes:**

Any relevant information that does not fall under one of these four headings is included in this paragraph. 9



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

# 4.3 CAIS Node Model

The CAIS provides interfaces for administering entities relevant during the software life-cycle, such as files, processes, and devices. These entities have various properties and may have a variety of interrelations. The CAIS model uses the concept of a node as the carrier of information about an entity. It uses the concept of a relationship for representing an interrelation between two entities. It uses the concept of an attribute for representing a property of an entity or of an interrelation. 1

The model of the structure underlying the CAIS and reflecting the interrelations of entities is a directed graph of nodes, which form the vertices of the graph, and relationships, which form the edges of the graph. This model is a conceptual model. It does not imply that an implementation of the CAIS must use a directed graph to represent nodes and their relationships. 2

Both nodes and relationships possess attributes. Attributes of nodes describe properties of the entities represented by the nodes; attributes of relationships describe properties of the interrelations represented by the relationships. 3

Each node, relationship, and attribute is associated with a definition. The definitions define the basic properties of nodes, relationships, and attributes. They regulate the creation and manipulation of nodes, relationships, and attributes via the CAIS interfaces. All CAIS interfaces ensure that nodes, relationships, and attributes remain consistent with their definitions. Definitions are themselves represented by nodes, relationships and attributes. 4

Certain CAIS interfaces allow nodes, relationships, and attributes to be identified using names. The CAIS provides a mechanism, called views, that allows users of the node model (i.e., tools operating in a process on behalf of a CAIS user) to assign names that may be used to identify nodes, relationships, and their attributes. In addition, the CAIS allows direct identification of nodes via Ada objects called node handles. This method of identification bypasses the translation required when names are used. Node handles for definition nodes may be used to identify properties of nodes and relationships. 5

The CAIS predefine certain kinds of nodes, relationships, and attributes. Throughout this standard they are referred to by predefine names. The CAIS predefine a single view. The local names defined in this view correspond to the names of these predefined kinds which are used in the standard. These names are used as descriptive qualifiers to identify specific kinds of nodes, relationships, and attributes. When this is done the name is written in all capital letters. For example, a node of the predefine node kind STRUCTURAL is referred to as a STRUCTURAL node. A predefine relationship used to identify a USER is referred to as a USER relationship. The predefined attribute CURRENT_STATUS is referred to as the CURRENT_STATUS attribute. 6

## 4.3.1 Nodes

The CAIS supports various kinds of nodes which are defined by node kind definitions. A node kind definition defines a class of nodes called a node kind. The CAIS predefine certain node kinds as a basis for defining the semantics of the CAIS interfaces. Three predefine node kinds that are fundamental to the node model are STRUCTURAL nodes, FILE nodes and PROCESS nodes. The CAIS supports defining additional node kinds and modifying existing ones. Nodes can be created, renamed, accessed (as part of other operations), and deleted. 1

A node may have attributes, relationships and contents, as specified by its node kind definition. The contents can vary with the kind of node. A node is said “to contain” its contents. If a node is a FILE node, it contains an Ada external file. If a node is a PROCESS node, it contains a representation of 2



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

the execution of an Ada program. If a node is a *STRUCTURAL* node, it has no contents and the node is used strictly as a holder of relationships and attributes.

## 4.3.2 Relationships

The relationships of CAIS nodes form the edges of a directed graph. Relationships are directional and are said to emanate from a *source node* and to terminate at a *target node.* A relationship may have attributes describing the properties of the relationship or the kind of its target node. 1

A *relation* is a class of relationships representing the same kind of interrelation. In the CAIS node model, a relation definition (see Section 4.3.11 on page 55) defines such a class of relationships. Relationships are instances of relations. All of the relationships of a given relation share a common set of properties that are defined by a relation definition. 2

A relationship is identified by a base node, a relation, and one or more relationship key values. A relation is identified either by a name (which is interpreted relative to a given base node) or by its relation definition node. The *relationship keys* are special attributes of a relationship. Relationship key values distinguish between multiple relationships of the same relation emanating from a given node. 3

Nodes in the environment are attainable by following relationships. Interfaces are provided to *traverse a relationship,* that is, to follow a relationship from its source node to its target node. 4

Relationships can be formed that represent either a unidirectional (one-way) or bidirectional (two-way) interrelation of two nodes. A *unidirectional relationship* emanates from a source node, terminates at a target node and represents a unidirectional interrelation between those nodes. A *bidirectional relationship* represents a bidirectional interrelation between two nodes. In the CAIS node model, a bidirectional relationship is represented by a pair of unidirectional relationships that are created and deleted together, but otherwise are manipulated by the CAIS interfaces in the same manner as non-paired unidirectional relationships. The source node of one of the unidirectional relationships in the pair is the target node of the other and vice versa. Each of the unidirectional relationships in the pair is said to be the *corresponding reverse relationship* of the other. Interfaces are provided to identify a corresponding reverse relationship. 5

A unidirectional relationship is an instance of a unidirectional relation defined by a single relation definition. A bidirectional relationship is an instance of a bidirectional relation defined by a pair of relation definitions or a single relation definition if the unidirectional relationships comprising each pair belong to the same relation (see Section 5.7.1.4 on page 310). 6

### 4.3.2.1 Primary and secondary relationships

Relationships are categorized as either primary or secondary. When a node is created, an initial bidirectional relationship is established between some other node and the newly created node. Of the pair of unidirectional relationships that represent this bidirectional relationship, the one which terminates at the newly created node is called the *primary relationship* to the new node; it is the only primary relationship of which the new node is a target node. The source node of this primary relationship is called the *parent.* There is no requirement that all primary relationships emanating from a node be of the same relation. However, a primary relationship must be one of a pair of unidirectional relationships representing a bidirectional relationship and the ALLOWED_USAGE attribute of the relation definition associated with that relationship must conform to the requirements for primary relationships (see Section 5.7.1.4 on page 310). 1



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Primary relationships form a strictly hierarchical tree; that is, for every node (except the root) there is one and only one sequence of primary relationships leading to it from the node that is the root of the tree. No cycles can be created in this tree using only primary relationships. A node is *obtainable* if it has been created and is the target node of a primary relationship. 2

Primary relationships are deleted only by certain CAIS interfaces. After deletion of the primary relationship to a node, the node is said to be *unobtainable*. A *non-existing node* is one which has never been created. The CAIS provides an interface that renames a node by making the primary relationship to the node emanate from a different node which becomes the new parent of the node. The CAIS interfaces which create nodes, delete nodes, rename nodes and manipulate definitions are the only operations that manipulate primary relationships. They maintain a state in which each node has exactly one parent and a unique primary path (see Section 4.3.7 on page 50). 3

*Secondary relationships are* relationships established between two nodes. These may form an arbitrary directed graph. They are not limited to form strictly hierarchical trees. Secondary relationships can be unidirectional or bidirectional relationships. Unidirectional relationships are always secondary. Secondary relationships may exist to unobtainable nodes. A RENAME operation applied to a primary relationship has no effect on any secondary relationships which have the renamed node as a target node, i.e., the secondary relationships still track the renamed node (see Section 5.2.31 on page 134). It is a general underlying principle of the CAIS that CAIS-internal node identifications used to implement relationships are unique and persist over time, even if the respective nodes are deleted. 4

### 4.3.2.2 Inheritable relationships

Certain relationships are *inheritable.* Upon copying a node (see Section 5.2.29 on page 126), inheritable relationships emanating from the node are copied to emanate from the newly created node. Similarly, upon creation of a dependent PROCESS node (see Section 6 on page 509), inheritable relationships emanating from the node of the creating process are copied to emanate from the dependent PROCESS node. An inherited relationship is unaffected by any changes to the original relationship and its attributes made subsequent to the creation of the inherited relationship. 1

Bidirectional relationships are never inheritable. Unidirectional relationships of several relations predefined in the CAIS are never inheritable; these relations are COPY_OF, EXECUTABLE_IMAGE, DEFAULT_ROLE, CURRENT_PROCESSOR, STANDARD_INPUT, STANDARD_OUTPUT, STANDARD_ERROR and CURRENT_NODE. For the last four of these relations a mechanism equivalent to inheritance is provided by means of default parameters in the interfaces which create PROCESS nodes (see Section 6.3 on page 521). 2

Secondary relationships of two relations predefine in the CAIS are always inheritable; these relations are CURRENT_ JOB and CURRENT_USER. 3

Secondary relationships of the predefine relation ADOPTED_VIEW have their inheritance property set as detailed in Section 5.7.3.4 on page 351. 4

All other secondary relationships of relations predefine in the CAIS are created as inheritable relationships. Secondary relationships of user defined relations can be created as inheritable or not inheritable depending on the relation definition. However, after creation the inheritance property of these relationships can be changed by calling a special CAIS interface that sets the INHERITABLE attribute of a relationship. 5



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

### 4.3.3 Attributes

Both nodes and relationships may have attributes which provide information about the node or relationship. Each attribute is defined by an attribute definition. Attribute definitions define the basic properties of attributes and regulate their creation and manipulation. An attribute is identified either by a name, which is interpreted relative to the node or relationship having the attribute, or by its attribute definition node. 1

The value of an attribute must conform to an attribute kind definition. An attribute kind definition defines a class of attribute values called an *attribute kind.* Attribute kinds are defined in terms of the following basic and structured value types: 2

**INTEGER** - a signed integer value. 3

**FLOAT** – a signed floating point value. 4

**STRING** - a character string. 5

**IDENTIFIER** - a string having the syntax of an Ada identifier or a value of an enumeration type. 6

**VARIANT** - a value conforming to one of several alternative attribute kinds. 7

**LIST** - an unconstrained list of values. 8

**UNIFORM_LIST** - a list in which all values conform to a particular attribute kind. 9

**COMPOSITE_LIST** – a list of values conforming to various attribute kinds. 10

The CAIS predefine certain attributes which are discussed in Sections 5 through 13 and defined in Appendix A. 11

### 4.3.4 Processes

A *process* is the CAIS mechanism used to represent the execution of an Ada program. A process is represented as the contents of a PROCESS node. Taken together, the PROCESS node, its attributes, and relationships are used in the CAIS to manage the dynamics of the execution of a program. Each time execution of a program is *initiated,* a PROCESS node is created, the process is created, the resources necessary, to support the initial execution of the program are allocated to the process, and execution is started. The newly created process is called the *initiated process,* while the process which caused the creation of that process is called the *initiating process.* 1

The CAIS provides interfaces to perform process initiation with a single call to the CAIS, Additionally, the CAIS provides interfaces that allow process initiation to be done with multiple calls to the CAIS. This provides the initiating process the opportunist y to create the new PROCESS node and then complete the allocation of resources required by the new process (using multiple CAIS calls if necessary) prior to starting the execution. It is possible for a process to request additional resources after execution has begun. 2

A single CAIS process represents the execution of a single Ada program, even when that program includes multiple tasks. Within the process, Ada tasks execute in parallel (proceed independently) in accordance with the rules in [1815A] 9(5): 3

Parallel tasks (parallel logical processors) may be implemented on multicomputers, multiproces- 4



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

> sors, or with interleaved execution on a single physical processor. On the other hand, whenever an implementation can detect that the same effect can be guaranteed if parts of the actions of a given [Ada] task are executed by different physical processors acting in parallel, it may choose to execute them in this way; in such a case several physical processors implement a single logical processor.

When a task calls a CAIS interface to perform an operation which depends on the occurrence of some event, that task may be suspended pending the occurrence of that event. In this case, other tasks may continue to execute and call the CAIS interfaces, subject to the Ada tasking rules. The suspended task has no delaying effect on the other tasks' calls if the operations to be performed are unrelated. 5

Processes are analogous to Ada tasks in that they execute logically in parallel, have mechanisms for interprocess synchronization, and can exchange data with other processes. However, processes and Ada tasks are dissimilar in certain critical ways. Data, procedures or tasks in one process cannot be directly referenced from another process. Also, while tasks in a program are bound together prior to execution time, processes are not bound together except by cooperation using CAIS facilities at run time. 6

## 4.3.5 Transactions

An operation is a call to a CAIS interface; the effects of such calls are defined by the semantics of each interface. A sequence of operations can be included in a transaction. A *transaction* is a sequence of operations on designated nodes which has the appearance of an atomic operation whose effects can either be committed or discarded as if the sequence of operations never took place. The CAIS defines node access synchronization rules which restrict the visibility of the effects of a transaction on nodes prior to their being committed. Such synchronization ensures that the effects of a given set of operations performed within a transaction are the same whether or not the transaction occurs concurrently with other transactions. (Nonetheless, processes containing the transactions may themselves not be serializable, due to the effects of operations not included within the transaction.) Transactions can be nested within transactions. 1

An enclosing transaction for a process includes all operations on nodes performed by the process. An enclosing transaction can be designated for a process when the process is created. The enclosing transaction facilities of the CAIS provide a means to construct tools which operate as transactions by composing existing tools that were not necessarily designed to operate as transactions. 2

## 4.3.6 Basic predefine definitions

The CAIS predefine certain node kinds, relations, attributes, and attribute kinds. These definitions form the base upon which the CAIS interfaces and semantics are built. The CAIS imposes restrictions on the use of the CAIS interfaces to manipulate instances of certain predefined definitions in order to preserve the integrity of the node model. Such definitions are said to be *CAIS-controlled*. Most predefine node kinds, relations and attributes are CAIS-controlled (see Section 5.7.2 on page 342). The general-purpose interfaces provided for the manipulation of user-defined nodes, relationships and attributes may not be applied to instances of CAIS-controlled definitions except as explicitly noted in the interface descriptions. 1

All predefine definitions are specified in detail in Appendix A. 2



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

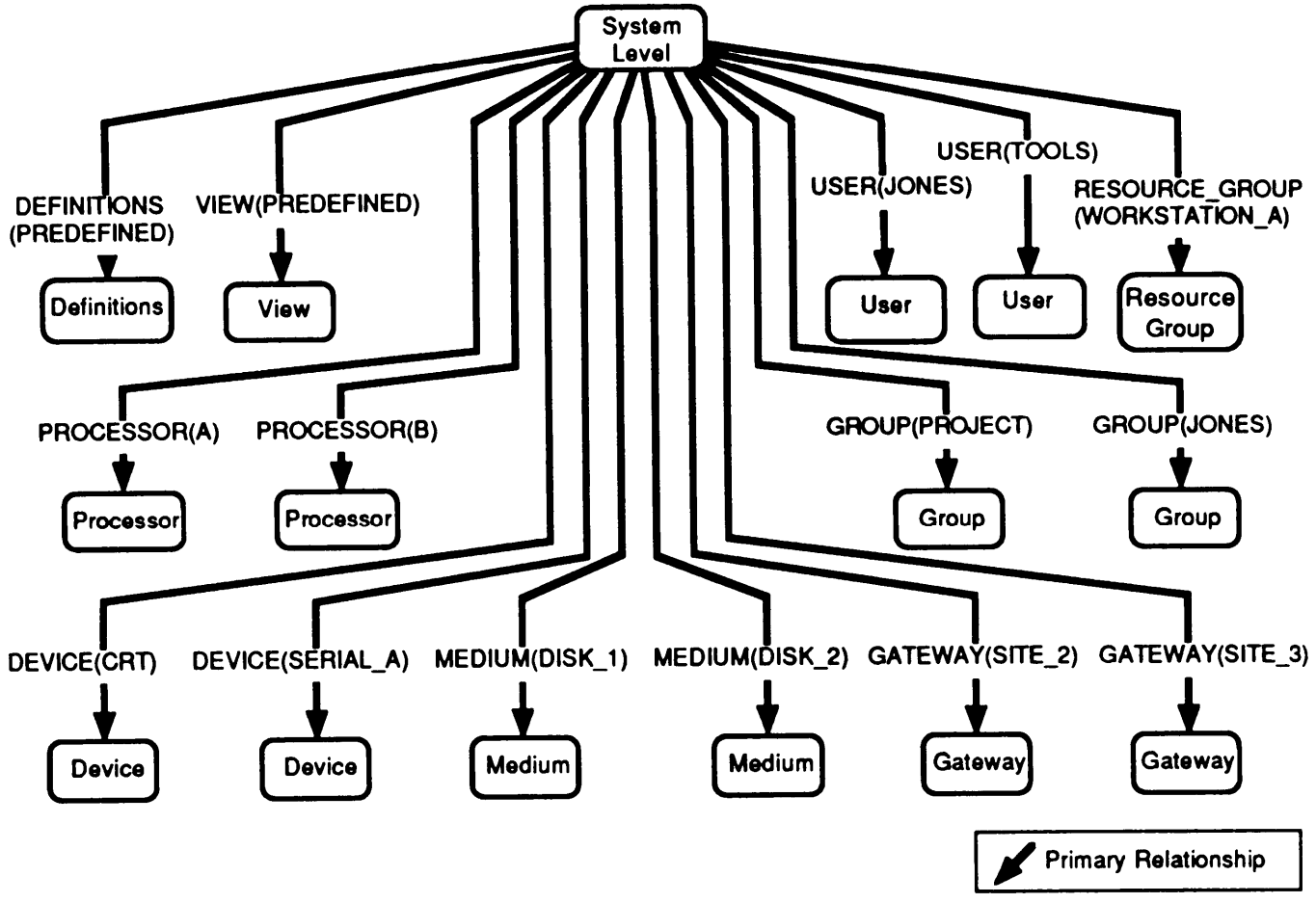


FIGURE 1. CAIS system concept

## 4.3.6.1 CAIS system concept

The CAIS node model incorporates the concept of a system (see the example in Figure 1). This concept 1
provides the means of administering all the entities represented within one CAIS implementation. This concept implies the existence of a *system-level node* which acts as the root of the CAIS primary relationship tree spanning the entire node structure. A CAIS *instance* is the collection of nodes in the primary relationship tree rooted at a given system-level node. The system-level node is defined as an unobtainable STRUCTURAL node. As such, operations performed on it through CAIS interfaces are limited to those that require only NO_ACCESS intents (see Table IV on page 98).

A *top-level node* is one whose parent is the system-level node. Top-level nodes cannot be created 2
with CAIS interfaces. There may be multiple, top-level USER nodes (see below), GROUP nodes (see Section 5.4.3 on page 257), DEFINITIONS nodes (see Section 4.3.11 on page 55), VIEW nodes (see Section 4.3.12 on page 59), RESOURCE_GROUP nodes, PROCESSOR nodes, DEVICE nodes,


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

MEDIUM nodes, and GATEWAY nodes (see Section 4.6 on page 76).

### 4.3.6.2 Basic predeflned relations and node kinds

This section explains the semantics of some predefine relations and node kinds, which are basic to the node model, and related CAIS concepts. 1

The CAIS node model incorporates the concept of a user. A *user* may be an individual, project, or other organizational entity; this concept is not equated with only an individual person. Each user has one top-level node called a *USER node.* This is a STRUCTURAL node which represents the user. From it the user can access other nodes of predefine or user-defined kinds. Each USER node is the target of a primary USER relationship emanating from the system-level node. The value of the key of a USER relationship is the *user name.* The CAIS does not define interfaces for creating nodes which represent users; such interfaces are to be provided outside the CAIS. 2

Figure 2 shows an example of a hierarchical tree formed by primary relationships. In the figure, DOT, SCRIPT, SYSTEM and TOOL are names of user-defined relations. The USER node that is the target node of the primary relationship ‘USER(JONES) has two relationships emanating from it. The relationship ‘SCRIPT(LOGIN) has a SECONDARY-STORAGE node as its target node and the relationship ‘DOT(TRACKER) has a STRUCTURAL node as its target node. The primary relationship ‘DOT(LANDING_SYSTEM) emanates from the STRUCTURAL node identified by ‘USER(JONES)’DOT(TRACKER) and has a STRUCTURAL node as its target node. The relationship of the user-defined relation WITH-UNIT with the relationship key value RADAR emanates from this STRUCTURAL node and has a SECONDARY-STORAGE node as its target node. The USER node identified by the relationship ‘USER(TOOLS) has several primary relationships emanating from it. The target nodes identified by two of these relationships, ‘TOOL(EDIT) and ‘TOOL(CLI), are EXECUTABLE_IMAGE nodes. These nodes might contain executable images of programs, CLI and EDIT. In addition, another STRUCTURAL node identified by ‘USER(TOOLS)’SYSTEM(TRACKER) is the target node of a primary relationship of the relation SYSTEM which emanates from the STRUCTURAL node identified by ‘USER(TOOLS). The EXECUTABLE_IMAGE node identified by ‘USER(TOOLS)-‘SYSTEM(TRACKER)’TOOL(SIMULATOR) may contain an executable image of a project-specific tool, i.e., specific to the TRACKER project. 3

The CAIS node model incorporates the concept of a job. When a user logs onto the APSE or calls the interface for creating a job, a *root PROCESS node* is created which often represents a command interpreter or other user-communication process. It is left to each CAIS implementation to provide a mechanism for logging onto the APSE and for enforcing any constraints that limit the top-level nodes at which users may log on. CAIS implementations must also provide a mechanism to control the selection of the executable image for the root PROCESS node created when a user logs onto the APSE. This mechanism and the execution environment of the process created must be specified in Appendix F. 4

After logging onto the APSE, the user will be regarded by the CAIS as the user associated with the USER node at which he logged on. A PROCESS node tree, spanned by primary relationships, develops from the root PROCESS node as other processes (called *dependent processes) are* initiated for the user. A particular user may have several root PROCESS nodes concurrently. Each corresponding PROCESS node tree is referred to as a *job.* 5

A JOB relationship emanates from each USER node and terminates at the root PROCESS node of each of the user’s jobs. The key of this relationship is assigned a value by the mechanism of interpreting the LATEST-KEY constant (see Section 4.3.7 on page 50) unless a value is explicitly specified when the job is created. 6



Source: https://assist.dla.mil -- Downloaded: 202610-26T07:54Z
Check the source to verify that this is the current version before use.

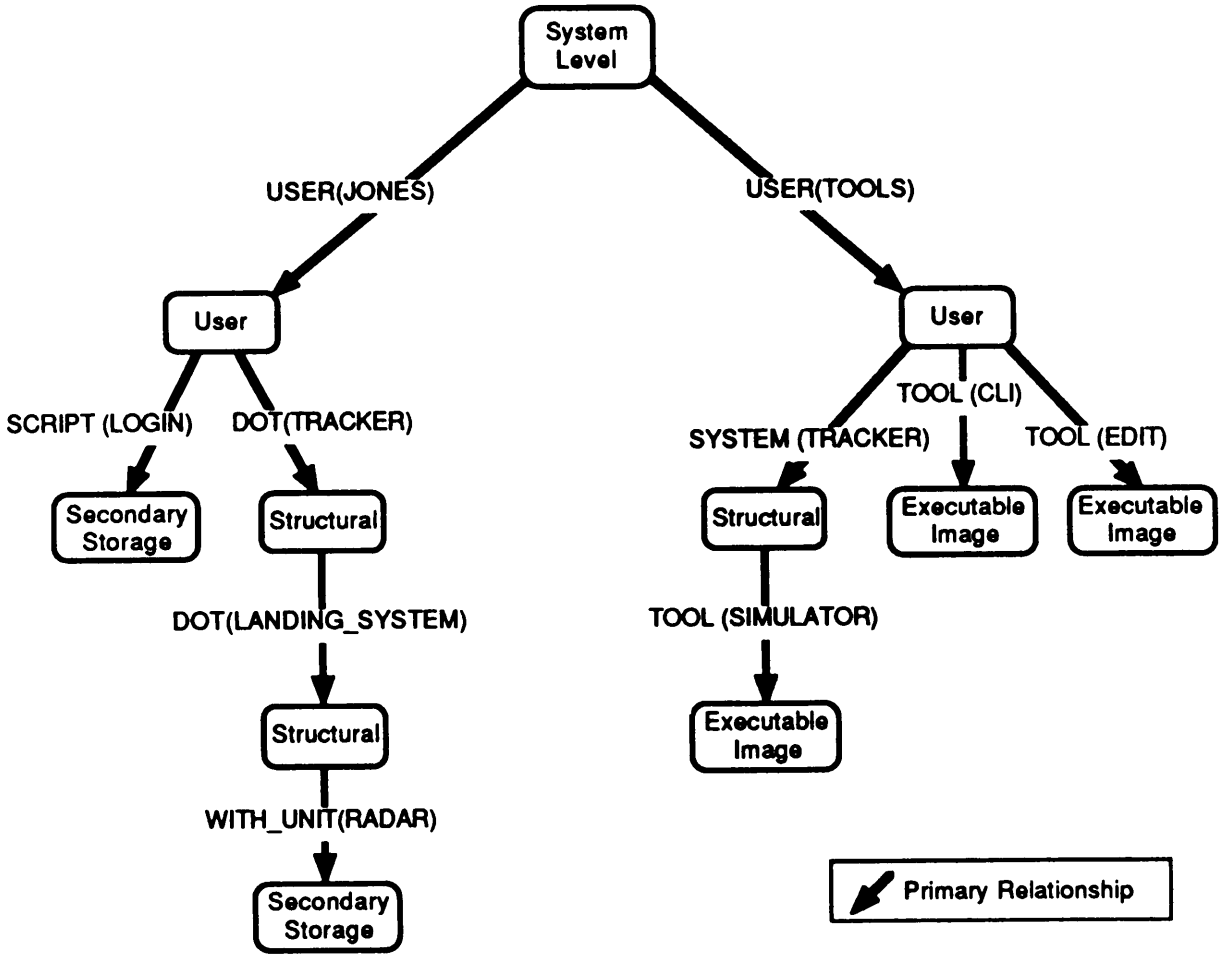


FIGURE 2. Tree formed by primary relationships



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

While the CAIS does not specify an interface for creating the initial root PROCESS node when a user logs onto the APSE, the effect is to be the same as a call to the interface which creates a job. The details of creating a job are described in Section 6 on page 509. The creation of a job involves establishing secondary relationships which emanate from the root PROCESS node. Dependent PROCESS nodes in the job inherit these relationships. They provide a means for reaching various top-level nodes from a PROCESS node. 7

When a CAIS interface is called by a process, the calling process is referred to as the *current process* for the CAIS operation performed by the call. This concept is implicit in all calls to CAIS interfaces. The current process for an operation defines the context in which the parameters to the CAIS interfaces are to be interpreted. In particular, pathnames are interpreted in the context of the current process. The term "current process" does not imply that only one processes can perform a CAIS operation at a time. 8

CURRENT_JOB, CURRENT_USER, and CURRENT_NODE are predefined relations which provide a convenient means for identifying other CAIS nodes. These relationships emanate from each PROCESS node. The CURRENT_JOB relationship always terminates at the root PROCESS node of a process's job, i.e., that PROCESS node's *current job*. The *current user* is the user on whose behalf the current process is executing. It is represented by the USER node which is the parent of the current job. The CURRENT_USER relationship always terminates at the USER node representing the current user. The CURRENT_NODE relationship can be used to terminate at a node called the *current node* which represents the current focus of the process or the context for its activities. The process can thus use the current node for a *base node*, i.e., a starting point, when specifying pathnames (see Section 4.3.7 on page 50). The CAIS requires that, when a root PROCESS node is created as a result of a user logging onto the APSE, a CURRENT_NODE relationship is also created terminating at the current USER node. 9

The CAIS provides mechanisms for associating the execution of a program with a specific kind of PROCESS node, with the EXECUTABLE_IMAGE node that contains the program, and with the processor resource on which the program is executed. A DEFAULT_PROCESS_NODE_KIND relationship may emanate from an EXECUTABLE_IMAGE node and terminate at a PROCESS node kind definition node. This relationship indicates the kind of PROCESS node used to represent the execution of the program, unless overridden in the call to a CAIS interface that creates the PROCESS node. The EXECUTABLE_IMAGE node for a process is identified by an EXECUTABLE_IMAGE relationship which emanates from the corresponding PROCESS node. The processor resource (see Section 4.6 on page 76) upon which a process is executing is identified by a CURRENT_PROCESSOR relationship which emanates from the corresponding PROCESS node and terminates at the PROCESSOR node representing that processor resource. 10

Figure 3 shows an example of some of these predefine relations. In the figure, DOT, PROCESS and TOOL are names of user-defined relations; the others are names of predefined relations. The user represented by the USER node that is the target node of the primary relationship 'USER(JONES) has logged on and the root PROCESS node that is the target of the primary relationship 'JOB(CLI) has been created. This root PROCESS node has several secondary relationships emanating from it. One of these relationships, 'EXECUTABLE_IMAGE(), terminates at the EXECUTABLE_IMAGE node identified by 'USER(TOOLS)'TOOL(CLI) which contains the executable image for the program being executed by the process. Another of these relationships, 'CURRENT_PROCESSOR(), terminates at the top-level PROCESSOR node 'PROCESSOR(A) which represents the processor resource upon which the process is executing. The relationship 'CURRENT_USER() identifies the USER node 'USER(JONES) as the current USER node. The relationship 'CURRENT_JOB() identifies the root PROCESS node for the user's job. The relationship 'CURRENT_NODE() identifies 11



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

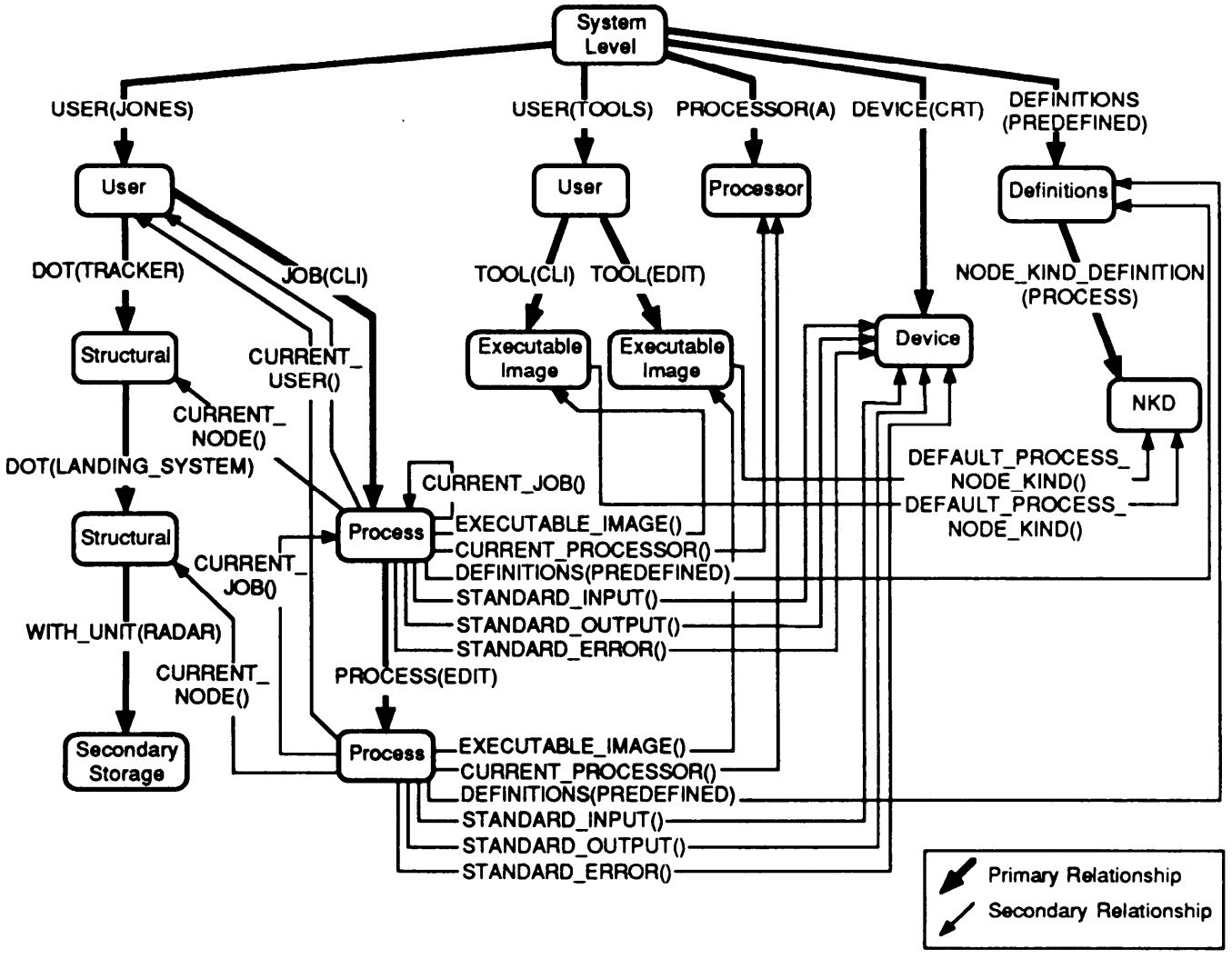


FIGURE 3. Some predefined relations



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

the STRUCTURAL node identfined by ‘USER(JONES)’DOT(TRACKER) as the current node for the process. The process represented by the root PROCESS node ‘JOB(CLI) has initiated a dependent process whose execution is represented by the PROCESS node which is the target of the primary relationship ‘PROCESS(EDIT). As depicted in Figure 3, the secondary relationships emanating from the PROCESS node ‘PROCESS(EDIT) are identical to those emanating from the PROCESS node ‘JOB(CLI) except that the relationship ‘EXECUTABLE_IMAGE() identifies the node identified by ‘USER(TOOLS)’TOOL(EDIT) as the executable image and the relationship ‘CURRENT_NODE() identfies the STRUCTURAL node identified by ‘USER(JONES)’DOT(TRACKER)’DOT(LANDING_SYSTEM) as the current node. Figure 3 also illustrates that each of the EXECUTABLE_IMAGE nodes, ‘USER(TOOLS)’TOOL(CLI) and ‘USER(TOOLS)’TOOL(EDIT), identifies the node kind definition node 'DEFINITIONS(PREDEFINED)'NODE_KIND_DEFINITION(PROCESS) as the default node kind definition node for PROCESS nodes representing the execution of those executable images via a secondary relationship ‘DEFAULT_PROCESS_NODE_KIND().

## 4.3.7 Names and identification

The attributes of a node or relationship, and the relations of relationships that emanate from a node are properties which may be identified by names. Definitions may also be identified by names. Names are assigned through the view mechanism (see Section 4.3.12 on page 59). The names are local to the type of property or type of definition which is named. Hence, there is no requirement, for example, that relation and attribute names be different from each other. Furthermore, the names of node attributes and emanating relations are local to a node kind and the names of relationship attributes are local to a relation. Names are used to form identifications of nodes, relationships and their attributes. Names may be used individually as parameters to CAIS interfaces and they may be included in pathnames (to identify nodes) and in relationship key lists (to identify relationships). 1

Names have the syntax of Ada identifiers and may include both lower case and upper case characters; however, all comparisons of names are insensitive to case. Interfaces which return names return alphabetic characters in their original case (as specified when the names were assigned). Predefined names use upper case characters; user-defied names may use both upper case and lower case. 2

Definitions and properties may also be identified by node handles which identify definition nodes, thus bypassing the translation of names. Identification of entities in the node model is summarized below. These identification methods are described in more detail in the sections which follow. 3

*Identification of a node* is provided by any of the following: 4

1. an open node handle to the node, 5
2. the unique identifier of the node, 6
3. a pathname, or 7
4. the identification of a relationship which terminates at the node. 8

An open node handle is an Ada object that identifies a node. A node handle may be opened by providing some other identification for the node such as a pathname. A unique identifier for a node is a string, automatically assigned by the CAIS implementation, which permanently identifies the node. A pathname identifies a node by identifying a sequence of relationships which lead to the node. 9

*Identification of a relationship* is provided by identifying the base node from which it emanates, 10



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

a relation to which the relationship belongs and values for keys of that relation in the form of a relationship key list. Such an identification is unique if the keys specified include a unique key set for the relation identified (see Section 5.7.1.4 on page 310). The relation may be identified by name or by an open node handle to a relation definition node.

*Identification of an attribute* is provided by identifying the node or relationship having the attribute (as described above) and either specifying the local name of the attribute (in the context of that node or relationship) or providing an open handle to the definition node for the attribute. 11

The phrase "to identify" means to provide an identification for an entity. An identification is considered an *illegal identification* if it contains a name which does not follow the syntax of an identifier, if it contains a unique identifier which is syntactically illegal with respect to the syntax described in Section 4.3.10 on page 54 or if it consists of a pathname or includes a relationship key list which is syntactically illegal with respect to the syntax defined in Table I. 12

## 4.3.8 Relationship Identification

Relationship key values distinguish between multiple relationships of a given relation emanating from a given node. Since relationship keys are attributes, their names have the syntax of Ada identifiers. Relationship key values also have the syntax of Ada identifiers. When a relationship key has no value, its value is said to be the *null key.* 1

When relationships are identified or created, relationship key values are designated by a *relationship key list.* Relationship key lists are passed individually to CAIS interfaces as parameters or are included in pathnames. Table I on page 52 shows the syntax for relationship key lists and pathnames in Backus-Naur Form (BNF). Blanks, format effecters and other non-printing characters are permitted surrounding lexical elements. 2

A relationship key list which is an empty key list does not designate any key values. Otherwise, a relationship key list may consist of a single relationship key designator or a list of key value associations. A *relationship key designator* designates the value of a key. A single relationship key designator maybe used to identify a relationship only when a single key of the given relation is visible (see Section 5.7.3.2 on page 349). A list of *key value associations* is used to designate multiple key values and to designate the null key as a key value. 3

Key value associations specify a particular key by name and a value required for that key. The value may be either the null key (represented by the absence of a value) or a relationship key designator. The null key indicates that the key must have no value. 4

Relationship key designators have two forms: an identifier or a string consisting of an optional identifier prefix followed by the character "#". The relationship key designator "#" is referred to as the *latest key.* 6

A relationship key list is specified together with a base node and a relation to identify existing relationships of that relation which emanate from that base node. If the relationship key list contains a single relationship key designator, it is equivalent to a relationship key list containing a single key value association in which the key named is the only visible key of the given relation. If the relationship key list is the empty key list, all relationships of the given relation are identified. Otherwise, the key value associations are interpreted in the order in which they appear in the list. 6



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

TABLE I: Pathname BNF

```
path_name                 ::= relationship_key_designator {path_element}
                            | path_element {path_element}
                            :
path_element              ::= '.relation_name [(relationship_key_list)]
                              .relationship_key_designator
relation_name             ::= identifier
relationship_key_list     ::= relationship_key_designat or
                              key_value_association
                                {,key_value_association}
                                empty_key_list
key_value_association ::= relationship_key_name =>
                              relationship_key_value
relationship_key_name     ::= identifier
relationship_key_value    ::= relationship_key_designat or
                              null_key
relationship_key_designator :: = identifier
                                 [identifier_prefix]#
identifier_prefix         ::= letter {[underline]letter_or_digit}[underline]
empty_key_list            ::=
null_key                  ::=
```

Notes:

1. See Appendix D for a description of the notation used.

As each key value association is interpreted, a further-restricted subset of the relationships belonging to the given relation is identified. Each key value association specifies a key name and optionally a relationship key designator. If the relationship key designator is omitted (i.e. the null key is used), the subset of identified relationships is restricted to those relationships which do not have a value for the key named. If the relationship key designator is an identifier, it represents a value for the key named. In this case, the relationships belonging to the identified subset are restricted to those relationships which have the specified value for the key named. Finally, if the relationship key designator contains "#", the identified subset of relationships is first restricted to those relationships (in the subset) for which the key named has a value that begins with the specified identifier prefix (if any). The identified subset of relationships is then further restricted to those relationships in the subset which have the lexicographically last value (within the identified subset) for the key named. 7

When identifying a single relationship, the subset of identified relationships that results from interpretation of all of the key value associations in the relationship key list must contain exactly one relationship. All relationship keys need not be included in the relationship key list so long as the specified keys identify a single relationship. 8

When creating a relationship, the relationship key list is interpreted in order in a similar manner. Each key value association further restricts the subset of existing relationships identified. A key value association in which the relationship key designator is an identifier restricts the identified subset to 9



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

those relationships having the specified value for that key. If the relationship key designator is omitted, the subset is restricted to relationships which do not have a value for the key named. As in relationship identification, a relationship key designator containing "#" restricts the subset of identified relationships to those relationships (in the subset) which have a value for the key that begins with the specified identifier prefix. However, when creating a relationship, the use of "#" causes a relationship key value to be generated for the new relationship. This relationship key value begins with the specified identifier prefix and lexicographically follows all existing values in the subset of identified relationships for the key named. Any keys of the given relation which are not included in the relationship key list but have a default value are added to the end of that list in an unspecified order. The value given to each key added to the list is its default value (see Section 5.7.1.8 on page 334). Any remaining keys which have no default value are not assigned a value.

When creating a new relationship, the resulting set of identified relationships must be empty. The set will be empty if "#" is used in any of the relationship key designators. Additional uniqueness constraints on the key values may also be imposed by unique key sets defined for the relation (see Section 5.7.1.4 on page 310). 10

Note that when "#" is not used in a relationship key list the order of interpretation of key value associations is not important. However, when "#" is used for one or more keys in a relationship key list (including any keys that are implicitly added to the list), the order of interpretation is important in determining which relationships are identified and, in the case of relationship creation, the key values generated. 11

## 4.3.9 Pathnames

Every accessible node may be reached by following a sequence of relationships; this sequence is called a *path* to the node. A path starts at a known (not necessarily top-level) node and follows a sequence of relationships to a desired node. The path from the system-level node to a given node that traverses only primary relationships is called the *unique primary path* to the given node. 1

Paths are specified by *pathnames*. When a pathname is supplied to or returned from a CAIS interface, it always refers to a path which starts at the current PROCESS node. Starting from this node, the path specified by the pathname is followed by traversing a sequence of relationships until the desired node is reached. The pathname for this path is made up of the concatenation of *path elements*, each of which identifies a traversed relationship, in the same order in which they are traversed 2

The syntax of a path element is an apostrophe (pronounced "tick") followed by a relation name and a parenthesized relationship key list. If the relationship key list is empty, the parentheses may be omitted. Thus, 'PARENT and 'PARENT() identify the same relationship. 3

The CAIS allows for an abbreviated form of path element when identifying relationships. If the relation name in a path element is DOT, then the path element may be represented simply by a dot (".") followed by a single relationship key designator. Thus, "'DOT(TRACKER)" is the same as ".TRACKER". The relation name DOT has no CAIS-specific semantics other than this abbreviation quality; there is no predefine relation named DOT. The abbreviated form of path element may be used for a relation named DOT in the identification of a relationship only if the relation has a single visible key (see Section 5.7.3.6 on page 355) and the relationship key value is not the null key. 4

A pathname may begin simply with a relationship key designator, not prefixed by either an apostrophe or ".". This is an abbreviation for a pathname obtained by prefixing the given pathname with "'CURRENT_NODE.", i.e., the interpretation follows the relationship of the predefined relation CURRENT_NODE and then the relationship of a relation identified by the relation name DOT 5



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

and the given relationship key designator. Thus LANDING. SYSTEM is the same as ‘CURRENT_-NODE.LANDING_SYSTEM.

A pathname may also consist of a colon (“:”). This refers to the current PROCESS node.

For example, in Figure 3 on page 49, all of the following are legal pathnames, within the context of the PROCESS node identified by ‘USER(JONES)’JOB(CLI). They all refer to the same node, since the CURRENT_NODE relationship points to the same node as ‘USER(JONES).TRACKER and the CURRENT_USER relationship points to the same node as ‘USER(JONES).

a. LANDING_SYSTEM’WITH_UNIT(RADAR)

b. 'USER(JONES).TRACKER.LANDING_SYSTEM'WITH_UNIT(RADAR)

c. 'CURRENT_USER.TRACKER.LANDING_SYSTEM'WITH_UNIT(RADAR)

Identification by pathname implies traversal of a node if a relationship emanating from the node is traversed; consequently all nodes on the path to a node are traversed, while the node at the end of the path is not traversed. An identification that would require traversal of an unobtainable or inaccessible (see Section 4.4 on page 62) node is treated as the identification for a non-existing node.

## 4.3.10 Unique identifiers

Every node created via the CAIS.interfaces is automatically assigned a permanent, unique identity called its unique identifier (UID). A unique identifier is a string-valued identification for a node which is unique to that node within the CAIS instance which created the node. In addition, this identification is intended to be unique among all nodes created by all CAIS instances. The unique identifier assigned to a node always identifies only that node, even if the node is deleted from a CAIS instance. A node may be archived on a storage medium in Common External Form (see Section 4.5.5 on page 69), using package CAIS_COMMON_IO. Such a node maybe deleted and subsequently recreated from the archive (using package CAIS_COMMON_IO) with its original unique identifier.

The CAIS provides interfaces for the identification of a node by its unique identifier. Each node has the predefined string-valued attribute UNIQUE_IDENTIFIER which contains the unique identifier assigned to the node. The CAIS also provides an interface to obtain the unique identifier of a node, given an open node handle to the node.

Unique identifiers are represented as strings consisting of two parts, a prefix and a suffix. The prefix identifies the CAIS instance in which the identified node was created. The prefix is fixed for a given CAIS instance and is assigned by the CAIS implementation whenever a CAIS instance is created. The prefix is intended to be unique among all CAIS instances. However, the administration of prefix assignments is not defied by the CAIS. The prefix and suffix are separated by a period (“.”). A distinct suffix uniquely identifies every node created by a given CAIS instance. A new suffix is created whenever a new node is created. See Section 5.1 on page 88 for a detailed description of the syntax of unique identifiers.

It is left to each CAIS implementation to provide interfaces for the specification of the unique identifier prefix upon installation.



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

## 4.3.11 CAIS definitions

In the CAIS, each node, relationship and attribute is an instance of a definition. A *definition* defines the common properties shared by all instances which are associated with and conform to that definition. A definition also imposes constraints on the properties defined. Definitions regulate the creation and manipulation of instances via the CAIS interfaces. All CAIS interfaces ensure that nodes, relationships, and attributes remain consistent with their associated definitions. 1

The CAIS node model represents definitions using nodes, relationships and attributes. The CAIS provides definition nodes for all predefine node kinds, relations, attributes and attribute kinds. (The predefine definition nodes are defined in Appendix A.) CAIS implementations may provide implementation-defined definition nodes. The CAIS interfaces may be used to create user-defined definition nodes. The definition nodes which define node kinds and relations occur in hierarchical structures called *lattices* through which properties and constraints are inherited. 2

### 4.3.11.1 Definition nodes

There are four kinds of definition nodes: node kind definition nodes (NKDs), relation definition nodes (RDs), attribute definition nodes (ADs) and attribute kind definition nodes (AKDs). An NKD represents a class of nodes called a node kind. Every node is an instance of some node kind. An RD represents a class of relationships called a relation. Every relationship is an instance of some relation. An AD represents an attribute which may occur on nodes and relationships. An AKD represents a class of attribute values called an attribute kind. Every attribute value is an instance of some attribute kind. Every AD is associated with an AKD that represents the allowed values of the attribute represented by that AD. 1

A *node kind definition* (represented by an NKD) specifies the emanating and terminating relations for a node kind, the number of relationships of each relation which may emanate from a node of that kind and the attributes allowed on a node of that kind. 2

A *relation definition* (represented by an RD) specifies the attributes which may occur on a relationship of the relation defined. Relation definitions also restrict the kinds of nodes which may be the source or target of a relationship. 3

An *attribute definition* (represented by an AD) defines an attribute, an identifiable property of certain nodes and relationships which may have an independent value on each of those nodes and relationships. An attribute definition identifies the attribute defined, specifies the attribute kind definition to which values of that attribute must conform, and places constraints on the use of that attribute. 4

An *attribute kind definition* (represented by an AKD) defines a set of attribute values. An attribute kind definition specifies the structure of values belonging to the set and imposes constraints on those values. Attribute kind definitions are used to specify the allowed values of attributes (which are defined by attribute definitions represented by ADs). Thus, an attribute definition defines an attribute while an attribute kind definition defines the set of values that may be assumed by certain attributes. 5

The properties and constraints defined by definitions are represented by attributes of definition nodes, by relationships which connect definition nodes and by attributes on those relationships. For example, an attribute of nodes of a given node kind is represented by an ATTRIBUTE relationship which emanates from the NKD representing that node kind and terminates at the AD representing the attribute. A CONSTRAINTS attribute on the ATTRIBUTE relationship can further constrain the defined node attribute in the context of nodes of that kind. In Figure 4 on page 56, an Officer node kind is defined. The ATTRIBUTE relationship specifies that Officer nodes may have the attribute 6



Source: https://assist.dla.mil -- Downloaded: 202610-26T07:54Z
Check the source to verify that this is the current version before use.

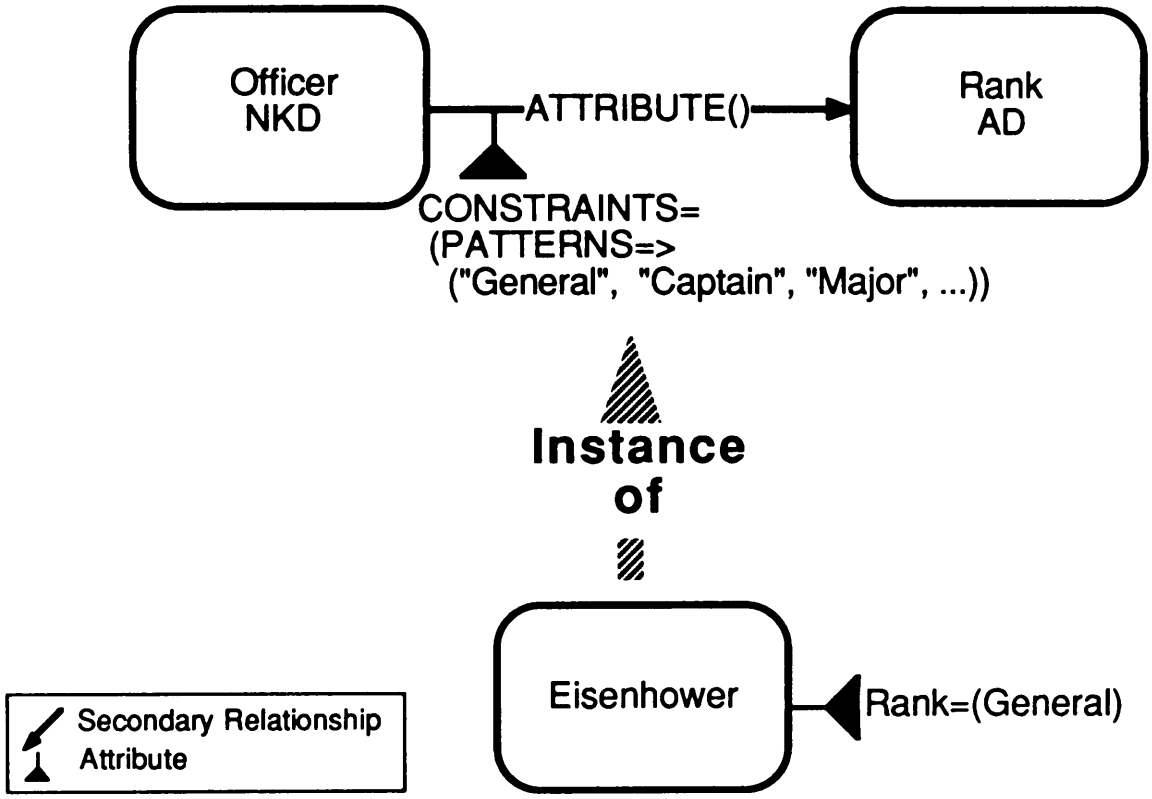


FIGURE 4. Example node kind definition and attribute definition

Rank. The CONSTRAINTS attribute on that relationship limits the ranks of Officers. The node labeled Eisenhower is an instance of the Officer node kind with the Rank attribute value General.

The detailed representation of definitions by definition nodes is described in Section 5.7.1 on page 304. 7

A top-level DEFINITIONS node serves as the parent of the definition nodes predefine by the CAIS. 8
The primary relationship to this node is 'DEFINITIONS(PREDEFINED). Implementations may also define top-level DEFINITIONS nodes for implementation-defined definitions. When a root PROCESS node is created, inheritable secondary DEFINITIONS relationships are created which emanate from the new PROCESS node and terminate at the predefine top-level DEFINITIONS node and an implementation-defined set of other top-level DEFINITIONS nodes. Figure 5 on page 57 illustrates such relationships.

The primary relationships to NKDs, RDs, ADs and AKDs are relationships of the relations NODE_- 9
KIND_DEFINITION, RELATION_DEFINITION, ATTRIBUTE_DEFINITION and ATTRIBUTE_-KIND_ DEFINITION, respectively. No special definitional semantics are associated with the primary relationship to a definition node; the normal semantics of primary relationships apply. All of these relationships can emanate from DEFINITIONS nodes. DEFINITIONS nodes can be created by users to serve as directories for user-defined definitions. Figure 5 on page 57 illustrates the predefine DEFINITIONS node 'DEFINITIONS(PREDEFINED), an implementation-defined DEFINITIONS node 'DEFINITIONS(IMPLEMENTATION), and a user-created DEFINITIONS node 'USER(JONES)'DEFINITIONS(JONES).


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

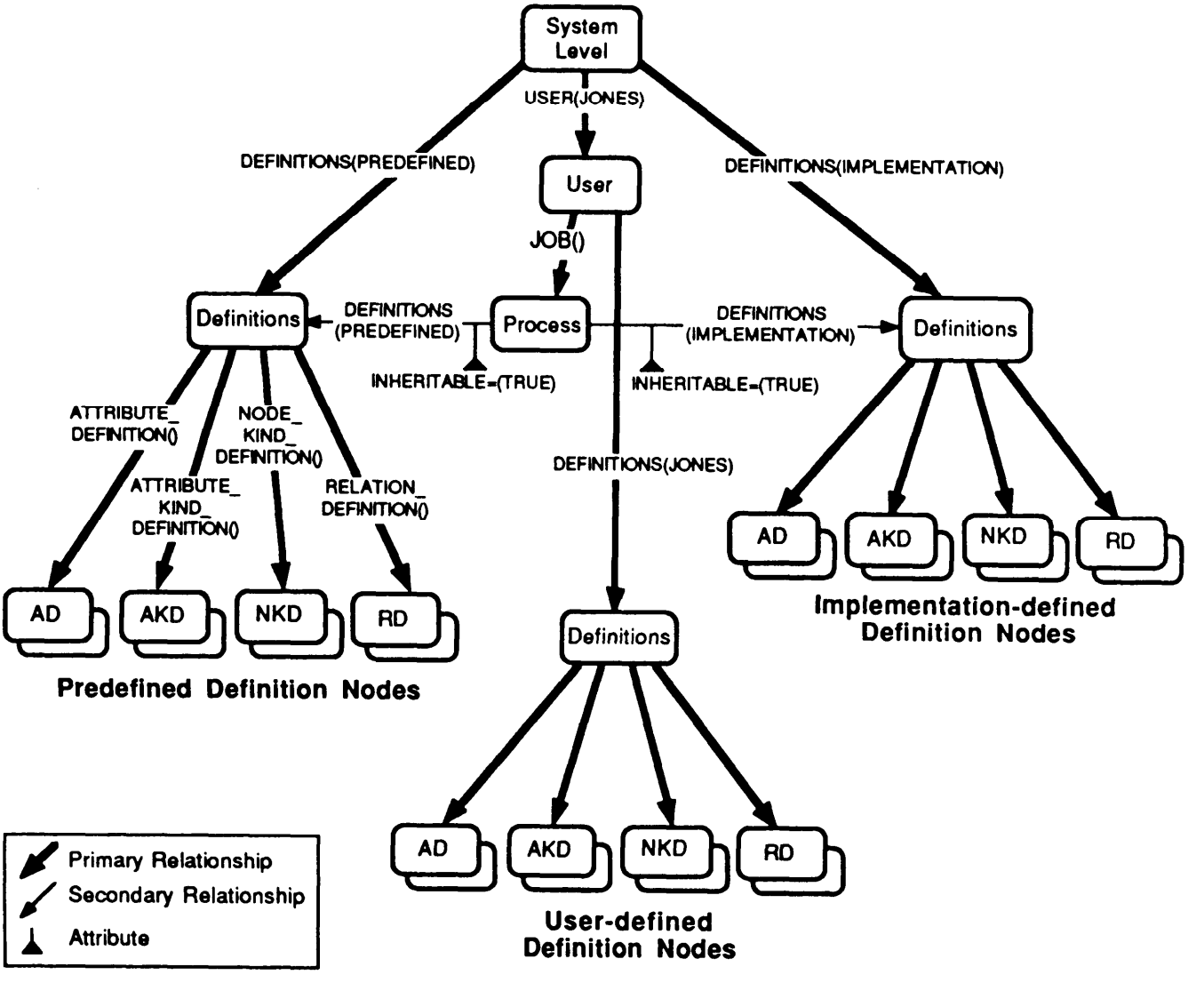


FIGURE 5. Primary relationships to definition nodes



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

## 4.3.11.2 Specialization and generalization

The CAIS supports the creation of definitions which inherit the properties and constraints defined by other definitions. Definitions which inherit properties and constraints are specializations of the definitions from which the properties and constraints are inherited. Conversely, definitions which define inherited properties and constraints are *generalizations* of the definitions which inherit them. 1

The CAIS supports the creation of specializations and generalizations of the definitions represented by NKDs and RDs. A given definition may be a specialization of one or more definitions and a generalization for zero or more definitions. A definition may not be a specialization of itself. 2

A definition which is a specialization of other definitions (generalizations for the specialization) inherits all of the properties and constraints defined by those generalizations. A specialization may also define properties and impose constraints which are not inherited from its generalizations. 3

Constraints which apply to a given property may be ordered by their strength. A given constraint is *stronger* than another constraint if the constraints are distinct and any instance which satisfies the stronger constraint also satisfies the other constraint. Conversely, the latter constraint is *weaker* than the stronger constraint. For example, the constraint on an integer N, N >= 1, is stronger than the constraint N >= O. 4

A specialization may impose stronger constraints on properties (such as narrower value ranges for an attribute) than those imposed by its generalizations. Weaker constraints may not be defined in a specialization. 5

The CAIS permits an instance defined by a given definition to be accessed as though it were an instance of a generalization of that definition. When this occurs, the instance is said to be accessed “through a generalization”. When an instance of a specialization is accessed through a corresponding generalization, only the properties defined by the generalization are visible; the constraints imposed on those properties by the generalization are satisfied since they are inherited by the specialization. When the instance is read, it appears to be an instance of the generalization; however, when the instance is modified through the generalization, the (possibly stronger) constraints defined by the specialization must prevail to preserve conformance of the instance to its definition. Thus, a CAIS operation which may be applied to instances of a given definition may also be applied to instances of specializations of that definition with the same effects so long as the constraints imposed by the specialization are not violated. 6

Generally, when a definition is used to identify instances, instances of specializations of that definition are treated as instances of the definition specified. Exceptions to this rule are explicitly noted in the interface descriptions. As an example, this rule applies to the identification of relationships. When relationships are identified by a relationship key list and a relation definition, relationships belonging to specializations of the relation definition specified are included as though they were instances of the specified relation definition. Thus, the relation defined by a relation definition includes the instances of that definition and the instances of all specializations of that definition as well. 7

Users may create specializations of the definition nodes that are predefine by the CAIS. Creation and modification of instances of such specializations are subject to the same restrictions that apply to instances of the predefined definitions themselves. 8

## 4.3.11.3 NKD and RD Lattices

The NKDs of a CAIS system, connected by relationships representing specialization, form an NKD 1



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

*lattice*. Similarly the RDs of a CAIS system, connected by relationships representing specialization, form an *RD lattice.* At the tops of these lattices are the most general definition nodes: the *basic NKD* and the *basic RD.* All other NKDs and RDs are specializations of these definitions. Thus, all NKDs and RDs, including those defied by users, are represented in the lattices. The definition lattices are acyclic, directed graphs.

Properties and constraints are explicitly represented by relationships and attributes of the definition node which defines them. Properties and constraints are inherited along the edges of the lattice toward more specialized definitions. Inherited properties and constraints are not explicitly represented at the specializations which inherit them, but rather are implicit in the lattice structure. 2

The predefine NKDs and RDs determine the overall structure of the lattices. Constraints imposed on instances by the predefine NKDs and RDs partition the lattices into disjoint sublattices. For example, at the highest level in the NKD lattice, NKDs are partitioned into the basic node kinds STRUCTURAL, PROCESS and FILE. All nodes are instances of these basic node kinds and their specializations. The STRUCTURAL and FILE node kinds are further partitioned into more specialized node kinds. A definition may not be a specialization of two or more node kinds belonging to different partitions. 3

The predefined NKD and RD lattices are defined in Appendix A. 4

### 4.3.11.4 Definition terminology

Phrases such as “node X is an instance of the node kind defined by the node kind definition represented by the node kind definition node Y“, while strictly correct, are cumbersome. Phrases such as this are usually shortened in this document to improve readability. For example the above phrase may be restated as “node X is an instance of Y or a specialization of Y“ or even “X is a Y node”, since there is little opportunity for confusion. Note that the statement “X is a Y node” includes the possibility that X is an instance of a specialization of Y rather than an instance of Y itself. This is consistent with the concept that instances of specializations of a definition may also be viewed as instances of that definition. 1

Similarly, phrases like “the node kind defined by the X NKD is a specialization of the node kind defined by the Y NKD” are avoided in favor of the shorter description “X is a specialization of Y“. 2

## 4.3.12 Views

The CAIS node model incorporates the concept of views. A *view* is a collection of nodes and relationships which defines names for certain definitions and the properties defied by those definitions. The names defined within a view are termed *local names.* Generally, names provided to the CAIS interfaces are translated and names returned by the CAIS interfaces are formed using views. *Name translation* is the mapping of a name to a particular definition node or property. Conversely, *name formation* is the mapping of a particular definition node or property to a name. 1

Each view consists of a VIEW node and a collection of node kind and relation interpretation nodes (see Figure 6). The primary relationship to a VIEW node is a VIEW relationship. Predefined and implementation- defined VIE W nodes are top-level nodes. The parent of a user-defined VIEW node can be any node. 2

Within a view, local names are specified as the key values of relationships which emanate from the VIEW node and its collection of interpretation nodes. An *interpretation node* provides local names for properties of nodes and relationships. A *node kind interpretation node* (NKI) provides local names 3



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

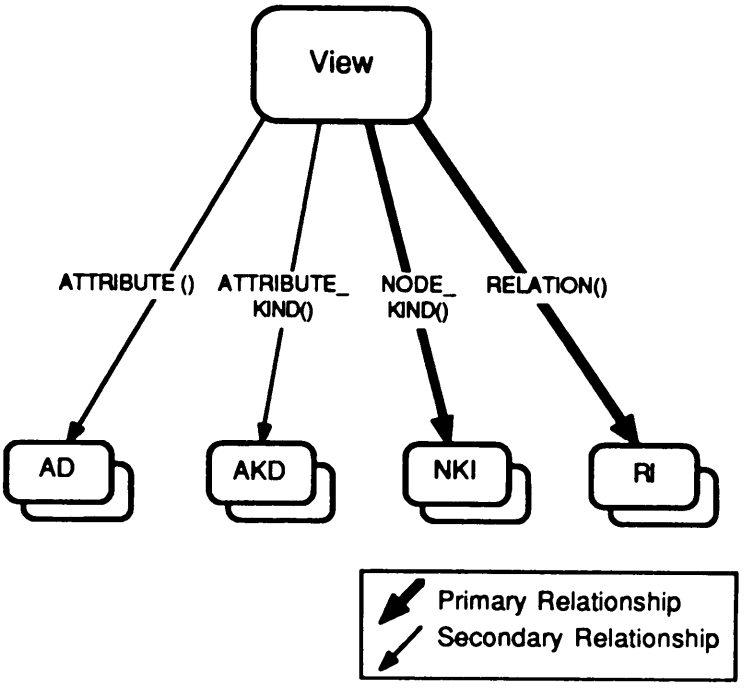


FIGURE 6. **VIEW node and emanating relationships**

for some or all of the attributes and relations that are properties defined by a particular NKD. The local names provided by an NKI are local to that interpretation and may differ from the local names given to the same attributes and relations by other NKIs. Similarly, a *relation interpretation node* (RI) provides local names for some or all of the relationship attributes that are properties defined by a particular RD. The local names defined by the key values of relationships emanating from a given RI are local to that interpretation and may differ from the local names assigned to the same attributes by other RIs.

The key values of the relationships which emanate from a VIEW node define local names for ADs, AKDs, NKDs and RDs. Primary NODE_KIND relationships emanate from VIEW nodes and terminate at NKIs. The key values of these relationships are local names defined by the view for the interpreted NKD. Similarly, primary RELATION relationships emanate from VIEW nodes and terminate at RIs. The key values of these relationships are the local names defined by the view for the interpreted RD. Secondary ATTRIBUTE and ATTRIBUTE_KIND relationships also emanate from VIEW nodes and terminate at ADs and AKDs respectively. The key values of these relationships are the local names defined by the VIEW for those ADs and AKDs. 4

A process must *adopt a view* in order to identify the definitions and the properties of instances of those definitions using the local names defined in that view. A process may adopt more than one view. Views may be adopted implicitly at the time a process is created or explicitly via CAIS interfaces. A process may also use the CAIS interfaces to explicitly cancel the adoption of a view. A given view can contain at most one interpretation for a given NKD or RD. However, a process can adopt more than one view which defines an interpretation for a given NKD or RD. A given view can define distinct interpretations for an NKD or RD and each of its specializations. 5

The collection of adopted views of a process is termed the *composite view* of the process. The order in 6



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

which views are adopted determines the precedence of the local names defined by the adopted views. Generally, the names defined by the most recently adopted view have the highest precedence. Alternatively, a process may designate that only a single, specific view is to be referenced when operations are performed using a given node handle. This is done by "adopting" an NKI within that view for the node handle. Section 5.7.3.5 on page 355 discusses adopted NKIs.

The detailed requirements for views are specified in Section 5.7.3 on page 347.

## 4.3.13 Attribute monitors

The CAIS node model incorporates the concept of an attribute monitor. An *attribute monitor* is a means whereby processes executing on the behalf of a specific user can monitor the creation, deletion and modification of specific node and relationship attributes on a collection of nodes.

Attribute monitors are represented by MONITOR nodes. The primary relationship to a MONITOR node is a relationship of the predefine relation ATTRIBUTE-MONITOR which emanates from the USER node representing the user on whose behalf the monitoring processes execute. Any number of separate MONITOR nodes may simultaneously monitor the attributes of a given node and attributes of relationships emanating from that node.

Whenever an attribute which is being monitored by a MONITOR node is created, deleted, or modified an attribute monitor event is inserted at the end of the attribute monitor event queue of one of the event handler processes associated with the MONITOR node. The CAIS provides interfaces which can be used by an attribute monitor event handler process to retrieve successive events from its attribute monitor event queue.

Details of attribute monitor operation are described in Section 5.8 on page 477.


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

# 4.4 Access Control

*Access control* in the CAIS refers to all aspects of controlling access to information.

> Access is a specific type of interaction between a subject and an object that results in the flow of information from one to the other. A subject is an active entity, generally in the form of a person, process, or device, that causes information to flow among objects or changes the system state. An object is a passive entity that contains or receives information. Access to an object potentially implies access to the information it contains. [DOD5200.28-STD]

The CAIS specifies mechanisms to support mandatory as well as discretionary access control.

> *Mandatory access control* provides access controls based directly on a comparison of the individual's clearance or authorization for the information and the classification or sensitivity designation of the information being sought. [DOD5200.28-STD]
>
> *Discretionary access control* is a means of restricting access to objects based on the identity of subjects and/or groups to which they belong. The controls are discretionary in the sense that a subject with a certain access permission is capable of passing that permission (perhaps indirectly) on to any other subject. [DOD5200.28-STD]

In the CAIS, an *object* is any node to be accessed and a *subject* is any process performing an operation requiring access to a node. *Access rights* are descriptions of the kinds of operations that processes are allowed to perform on nodes. Access control is used to limit access to nodes based on processes' access rights.

A node is *accessible* if the current process has sufficient discretionary access rights to have knowledge of the node's existence (see Section 5.4.4 on page 259 and Section 5.4.5 on page 262) and if mandatory access controls permit information flow from the node to the current process (see Section 5.4.6.2 on page 263). A node which is not accessible is *inaccessible.* Accessibility y is always relative to the access rights of the current process and is to be distinguished from obtainability, which is a property of the node alone.

The detailed requirements of CAIS access control have the following aspects.

a. Access control is relevant to those CAIS operations which constitute access to a node. See Section 5.4.1 on page 255 for a categorization of which operations do and do not constitute node access.

b. Nodes have security labels which categorize the authorization of individuals and the sensitivity of information, establishing access rights for purposes of mandatory controls. See Section 5.4.2 on page 255 for a description of the security labeling of nodes.

c. Users and tools are associated with groups. See Section 5.4.3 on page 257 for a description of groups.

d. Groups of users and tools can be granted or denied access rights to nodes on a discretionary basis. Section 5.4.4 on page 259 describes granting access and Section 5.4.5 on page 262 describes denying access.


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

e. Those operations which constitute node access first require access checking. *Access checking* is the act of determining access rights, comparing them against those required for the intended operation, and either permitting or denying the operation. This process is described in Section 5.4.6 on page 262. 13



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

# 4.5 Input and Output

CAIS input and output mechanisms unify support for traditional secondary storage file and device input and output as well as interprocess communication within a single CAIS instance and between CAIS instances. The main features of CAIS input and output are channels and protocols. Conceptually, data are viewed as flowing over channels between pairs of communicating entities. A *channel* is the node model representation of a path over which data may flow. A *data transfer protocol*, or just *protocol*, is a method for storing and transmitting information. The charnel representation can be used in combination with the node kind and relation definitions to ensure that information passed between communicating entities is, semantically, the information intended to be exchanged. Protocols are used to identify the different representational and procedural conventions for storing and transmitting data. The CAIS provides a few fundamental protocols and affords CAIS implementers the opportunity to supply any number of additional protocols. Few restrictions are placed upon the nature of added protocols. It is anticipated, for example, that sophisticated graphical user interfaces can be supplied as additional protocols which can be standards in their own right.

## 4.5.1 Protocols

Each protocol has a corresponding Ada package defining the interfaces used with that protocol. The protocols defined by this standard are:

a. **CAIS_FRAME_IO** supplies simple input and output methods. This protocol is the intended base for user-defined extensions to CAIS input and output interfaces.

b. **CAIS_MAGNETIC_TAPE_IO** supplies a standardized method for controlling magnetic tape drives.

c. **CAIS_DIRECT_IO** supplies a method essentially compatible with package DIRECT-IO defined in Chapter 14 of [1815A].

d. **CAIS_SEQUENTIAL_IO** supplies a method essentially compatible with package SEQUENTIAL_IO defined in Chapter 14 of [1815A].

e. **CAIS_TEXT_IO** supplies a method essentially compatible with package TEXT_IO defined in Chapter 14 of [1815A].

f. **CAIS_TERMINAL_IO** supplies a method for controlling character imaging display term.inals and high resolution terminals simulating character imaging terminals.

g. **CAIS_FORM_TERMINAL_IO** supplies a method for controlling form terminals.

Details of protocols “a” through “f” are described in Section 8 on page 629. Details of protocol “g” are described in Appendix C.

Every protocol defines OPEN and CLOSE interfaces which open and close that protocol. Some protocols also define a CREATE interface which creates a file, creates a channel between the new file and the calling process and opens a protocol on the channel. The term *protocol OPEN interface* means an OPEN or CREATE interface found in a protocol package. A *protocol package* is any of the protocol packages defined by the CAIS and listed above (Section 4.5.1 on page 64), or any protocol package defined by a CAIS implementer. The term *protocol CLOSE interface* means a CLOSE interface found in a protocol package. The terms *protocol OPEN operation* and *protocol CLOSE operation* mean calls to the respective interfaces.



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

## 4.5.2 Channels, client nodes, and PORT relationships

In the CAIS node model, the communicating entities are represented as nodes called client nodes. Only PROCESS nodes and FILE nodes (including instances of specializations of these kinds) may be client nodes. SECONDARY_STORAGE nodes, DEVICE nodes, and GATEWAY nodes are instances of predefine specializations of the FILE node kind and therefore may be client nodes. 1

At least one of the client nodes involved in a data transfer must be a PROCESS node. PROCESS nodes which are client nodes are termed *client PROCESS nodes*. The processes which these nodes represent are termed *client processes*. 2

Before data can be transferred between client nodes, a path for the data, called a channel, must be created. The CAIS mechanisms for controlling channels are collectively called the *input and output connection model*. Data transfer using input and output protocols can only take place between node pairs which have a channel in common. 3

Suppose a channel exists between two client nodes A and B. In the CAIS node model, this channel is represented as a CHANNEL node, a PORT relationship emanating from client node A and terminating at the CHANNEL node, and another PORT relationship emanating from client node B and terminating at the same CHANNEL node. Every channel is represented by two PORT relationships and one CHANNEL node. The two PORT relationships and the CHANNEL node are created in a single operation called *channel creation*. After a channel has been created, the client nodes are said to be connected. The client nodes can be viewed as being connected to each other, to the charnel, and to the CHANNEL node as well. Similarly, the client processes are said to be connected. 4

Figure 7 gives two examples of channel representation. A client process may transfer data to itself. This is illustrated by the lower diagram in Figure 7. Such a channel configuration may be useful in testing programs or for symmetry reasons. A single client node can be connected to many other client nodes. FILE nodes that allow multiple access may have channels to many PROCESS nodes. A single DEVICE node, representing a workstation, for example, may communicate over separate channels for the keyboard, CRT, and disk drive. Some of these situations are illustrated in Figure 8 on page 66. 5

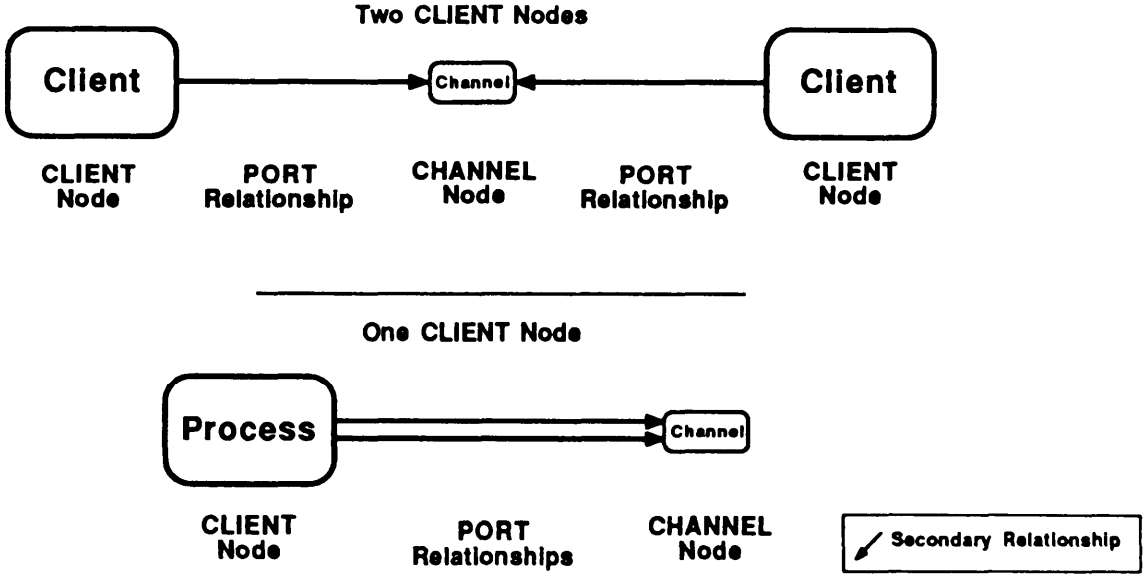


FIGURE 7. Channels


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

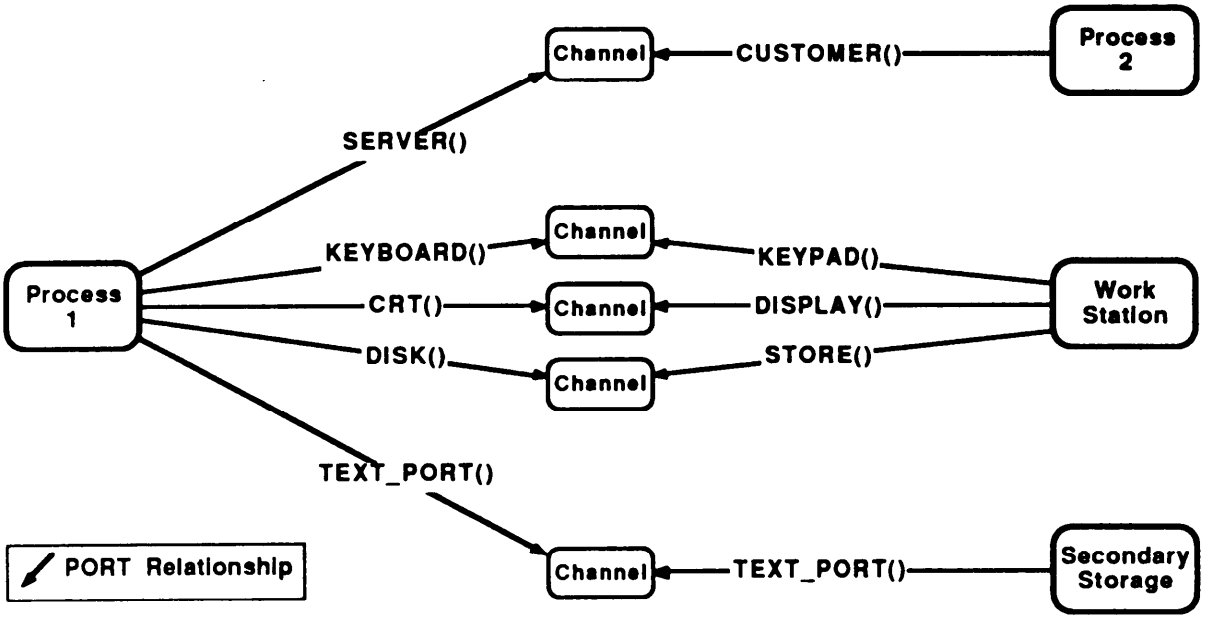


FIGURE 8. **Multiple channels**

In subsequent discussion of channels, the client node which is the main subject of the discussion will 6
be called the *local client node.* The other client node to which the channel is connected is called the *opposite client node.* Similarly, the PORT relationship emanating from the local client node is called the *local PORT relationship.* The PORT relationship emanating from the opposite client node is called the *opposite PORT relationship.*

CHANNEL nodes and PORT relationships are managed by the CAIS implementation as a by-product 7
of creating and deleting channels. Programs may not explicitly create or delete CHANNEL nodes or PORT relationships using other CAIS interfaces. Handles to the CHANNEL node are used to identify a particular channel. Users may create their own specializations of the predefine relation PORT and the predefine node kind CHANNEL.

Specializations of PORT relations are used to distinguish data streams which are to be treated as 8
semantically distinct. Such specialized relations are also used to describe the communication requirements and capabilities of client nodes, i.e., processes, secondary storage files, devices, and gateway communication facilities. By defining specializations of PROCESS, SECONDARY_STORAGE, DEVICE, and GATEWAY NKDs, and by constraining the PORT relationships which may emanate from instances of these nodes, users can limit the types of data streams semantically valid for a given client node kind. PORT RDs allow the communication characteristics of processes and files to be made explicit. This information can be preserved by transmitting the PORT RDs along with their associated FILE or EXECUTABLE_IMAGE nodes whenever such nodes are moved from one CAIS instance to another. Since PORT RDs are part of the structure of CAIS definitions, PORT RDs will normally follow any node instances that are transmitted with their definition information. Attributes of PORT relationships can be used to constrain the direction in which data may be transferred over a PORT relationship and which protocols may be used for data transfer.


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Specialized CHANNEL node kind definitions with constraints on terminating PORT relations are used to control semantic as well as functional compatibility of connected PORT relationships. Two PORT relationships are compatible for data transfer only if they may legitimately terminate at a single CHANNEL node. 9

For example, a tool which expects to read Ada source code should not be allowed to read from a file containing graphical images; a tool expecting graphical data should not be allowed to read Ada source code; but a tool, such as a print tool, expecting any form of text data, should be allowed to read either Ada source or graphics, provided both are represented in a textual format. Suppose specialized PORT relation A is used to represent Ada source code; specialized PORT relation T is used to represent any text information; and specialized PORT relation G is used for graphical images in a text file format. It is possible to define a sublattice of specialized CHANNEL NKDs that allows relationships of relations A and T to form a single channel, relationships of relations T and G to form a channel, but not relationships of relations A and G. This is the desired effect. 10

File contents are controlled by restricting the PORT relations allowed to emanate from nodes of a given FILE kind. Similarly, PROCESS nodes may be restricted to specific sets of data streams. 11

Programs capable of generalized communication, e.g., a general text copying program, will use generalized PORT RDs. Programs which depend on specialized data forms will use more specialized PORT RDs. It is the users' responsibility to construct lattices of PORT RDs and CHANNEL NKDs that give the desired constraints. When a program or file is moved between CAIS instances, or when a device is installed, the appropriate PORT RDs and CHANNEL NKDs must also be installed in the receiving CAIS instance. 12

PORT relationships and CHANNEL nodes possess predefine attributes used in controlling the flow of data. Specializations of PORT RDs and CHANNEL NKDs may have additional attributes defined for specific situations. All PORT relationships have a key attribute allowing multiple PORT relationships emanating from the same node to be differentiated. The values of the key attributes of all PORT relationships emanating from a single node must be distinct. PORT relationships also have a DIRECTION attribute which constrains the direction in which data may flow on the channel. In some cases, the DIRECTION attribute will be given an initial value by the PORT RD. In all cases, a value for the DIRECTION attribute will be established before data transfer is allowed. PORT relationships also possess the ALLOWED_PROTOCOLS attribute giving the names of protocols legal for that channel. CHANNEL nodes have a CONTROLLING_PROTOCOL attribute giving the name of the protocol in use and a STATE attribute giving the current state of the channel. 13

## 4.5.3 Creating and using channels

During its lifetime, a channel exists in several states. The states are changed by using CAIS interfaces. First a channel is created, at which time the node model representation of the channel is brought into existence in the CONNECTED state. The client nodes are now said to be connected. 1

*Protocol opening* is the means by which an internal file of an Ada program is associated with a channel and with the opposite client node representing a file or process. Protocol opening involves checking the compatibility of the protocol(s) used by the client process and/or the protocol(s) used to communicate with client FILE nodes. When protocol opening is completed successfully, the channel is placed in the ESTABLISHED state and a protocol is said to be established on the channel. Data can now be transmitted between the client nodes. 2

When communication with the established protocol comes to an end, a *protocol closing* operation is 3



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

performed, after which the channel will be in the CONNECTED state or the CLOSED state depending respectively upon whether there are one or two client processes. A channel in the CLOSED state is said to have a *closed* protocol. The process issuing the first protocol closing operation is called the *closing process.* When a protocol is closed, data buffered by the channel can still be read by the process opposite the closing process. When both client processes have issued protocol close operations, the channel enters the CONNECTED state. Once the channel has been returned to the CONNECTED state, a protocol, possibly a different protocol, can be opened again. Communicating processes in opposite CAIS instances, connected to channel pairs, act as if they were opposite client processes in the same CAIS instance (see Section 4.7.1 on page 85).

When the channel is no longer needed, a client process can delete the channel, at which time both PORT relationships are deleted from the node model and the channel enters the RETIRED state. Channels in the RETIRED state can no longer be used for data transfer. In order to provide for orderly shutdown, channels to be deleted which are initially in the ESTABLISHED or CLOSED states may pass through the CLOSED and CONNECTED states before reaching the RETIRED state. Channel states and state transitions are summarized in Figure 9.

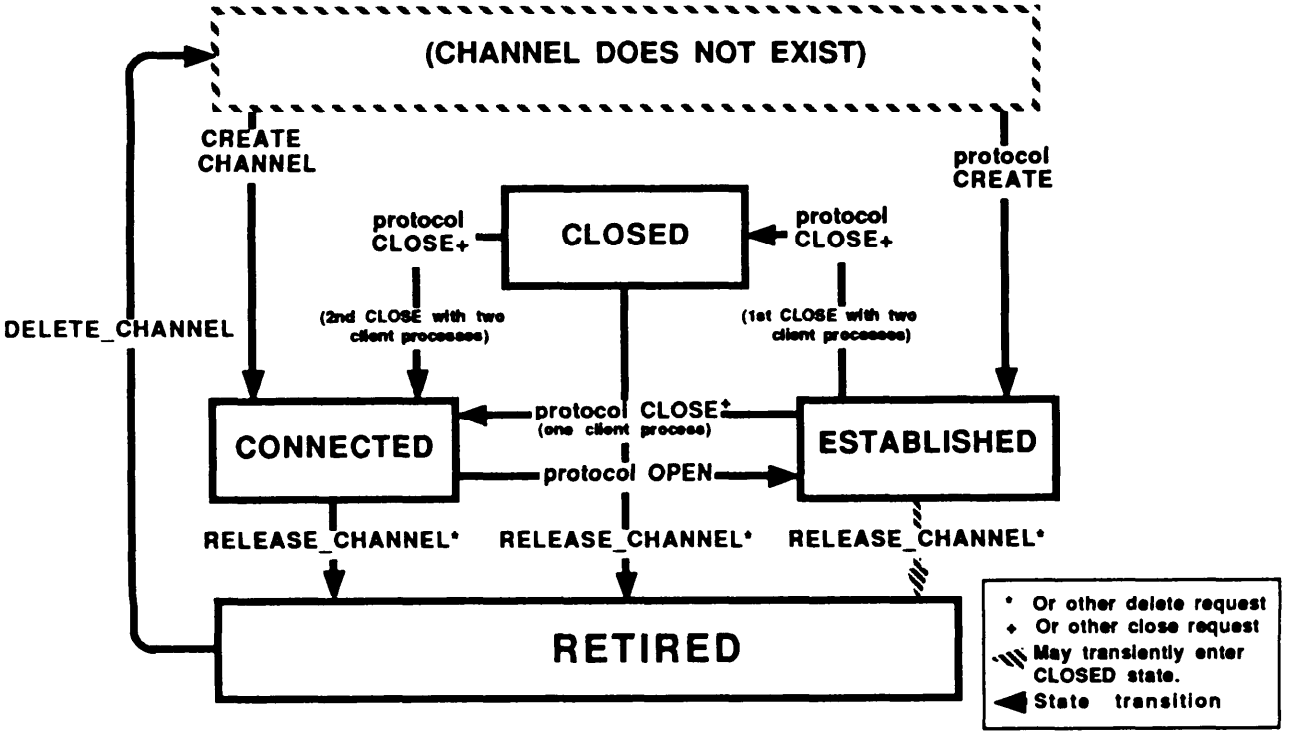


FIGURE 9. **Channel state transition**

A *creating process* is a process that causes a channel to be created. Note that for a given channel, the creating process is not necessarily either of the client processes being connected. For example, process A could create a charnel between processes B and C; moreover, this can happen prior to the time B or C are placed in execution.

The basic scenario for creating a channel and opening a protocol is:



Source: https://assist.dla.mil -- Downloaded: 202610-26T07:54Z
Check the source to verify that this is the current version before use.

1. The creating process creates the channel.
2. Each client process issues a protocol OPEN call.

The same scenario applies, for each process when two PROCESS nodes are connected, or for a PROCESS node connected to a SECONDARY_STORAGE, DEVICE, or GATEWAY node.

Some processes may be designed to act as servers, waiting until other processes connect channels to them. Interfaces are provided to accommodate this method of operation. Suppose process P is to connect to a server process S. The scenario for server operation is:

1. Process S issues a call to AWAIT_CHANNEL which blocks.
2. A process creates the channel. This may be any process with the necessary access rights, process P, process S, or a third process.
3. Process P issues a call to a protocol OPEN.
4. The call to AWAIT_CHANNEL in process S returns. Process S may then accept the open by issuing a call to a protocol OPEN, or it may reject the open by issuing a call to RELEASE_ - CHANNEL.
5. The OPEN call in process P will fail if S rejects the open; otherwise, both OPEN calls block until the compatibility checks for opening, Section 7.3.4 on page 578, are performed. If these checks succeed, both OPEN calls will complete with success, otherwise, both will fail.

One or both communicating processes may use the server scenario. The CAIS mechanisms also allow the server scenario to be used for communication with SECONDARY-STORAGE, DEVICE, and GATEWAY nodes. Detailed scenarios for all cases are given in Section 7.3.5 on page 582.

## 4.5.4 Compatibility checks

Checks to ensure compatibility between communicating processes and files are made when a channel is created and when a protocol is opened. When a channel is created, the PORT RDs are checked for compatibility and the DIRECTION attributes are checked for compatible values, if values are specified for both PORT relationships. The channel is not created if any check that is performed fails. Eight compatibility checks are made when a protocol is opened, DIRECTION attribute compatibility, compatibility of the DIRECTION attribute with the intent of the OPEN operation, compatibility of the DIRECTION attribute with the MODE parameter, compatibility of the MODE parameter with the intent, compatibility of the MODE parameters (if both clients are processes), compatibility of the protocols named by the client processes, mandatory access rights of the client nodes, and if one of the client nodes is a resource node, checks will be made for correct use of the resource (see Section 4.6 on page 76). If any of the performed checks fail, the protocol will not be opened. These checks are described in detail in Section 7.3.4 on page 578.

## 4.5.5 Common IO

To facilitate the transfer of information between arbitrary CAIS instances possibly running on heterogeneous machines, a common external representation for the information must be established. In the



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

CAIS, this representation is termed the *common external form* and is defined explicitly. The common external form encapsulates, in a linearized fashion, free of machine dependencies, the information inherent in that portion of the CAIS instance to be transferred. To maximize portability, the common external form is an ASCII encoding of the data.

Common IO is the collection of interfaces which allow one to denote the information to be transferred, to transform this information to the common external form, and, once transferred to another system in this form, to recreate the information within the node model of the receiving CAIS instance. 2

Conceptually, there exists a set of nodes, a *node set*, which comprises the information to be transferred to another CAIS instance. *Exportation* is the act of describing a node set and transforming the information included in the node set to the common external form. An *external node set* is node set information expressed in the common external form. *Importation* is the act of reading an external node set and recreating the node set information within the node model of a CAIS instance. 3

Although transferring information from one CAIS instance to another may be viewed as a batch transmission on magnetic tape, this does not preclude the use of an incremental approach, nor the use of another medium. 4

Common IO can be used for long-term archiving of CAIS instances. 5

## 4.5.5.1 Exporting data

The first step in exporting data to another CAIS instance is to describe the data to be exported by defining the appropriate node set. Interfaces are provided to create an initially empty node set and to include explicitly any node or logical structure of nodes in this set. 1

To *include* a node in a node set implies that all information about that node is to be transferred: its attributes, relationships, and contents. The information transferred for each relationship includes its attributes and an indication of its target node. In the common external form, the target of a relationship is denoted by the target node's UID. Contents are directly transferred for text FILE nodes only. Contents of a non-text FILE node can be transferred indirectly by encoding the contents into a text FILE node. If this text FILE node is then transported along with the original non-text FILE node, the non-text contents can be re-established in the importing CAIS instance by an appropriate decoding. Encoding and decoding is performed by application-specific tools. 2

Optionally, the node information transferred includes definition information as well. This implies that if a node is included, then each node in the definition lattice needed to interpret that node is included: its NKD, the RD of each of its relationships, the AD for each of its node attributes or relationship attributes, and the AKDs referenced by these ADs. The definition information for each node thusly included is similarly included. 3

Once the node set is fully described, the information inherent in this node set can be written in the common external form to a channel. Apart from node information, additional information describing the CAIS instance and various pragmatic aspects of the CAIS implementation are also written. All of the information writ ten comprises the external node set. 4

## 4.5.5.2 External node set

The common external form is used to represent externally the information inherent in a portion of a CAIS instance. It is a character-coded representation; all forms of data are converted to an ASCII 1



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

representation. The common external form uses the string-valued external representation of lists as defined by CAIS_LIST_MANAGEMENT (see Section 9 on page 745).

To represent anode set in the common external form, one creates an external node set. An external node set is a text file consisting of a number of sections. The first section, termed the *header section* describes all the nodes being exported without describing their contents. Each subsequent section, termed a *content section*, describes the contents of a single text FILE node. An external node set is depicted in Figure 10. The formats of the header and content sections are described more fully in subsequent paragraphs and by the BNF detailed in Table II. 2

The header section contains a named list in text form containing the name of the exporting CAIS instance, the value of the SMALL_ FOR_CAIS_DURATION pragmatic, the time that the external node set was created, a list of the nodes being exported, and a count of the number of text FILE node contents being exported. Each node is represented as a named list containing the UID of the node, the UID of its NKD, its attributes, and its relationships. Each attribute is represented as a named list containing the UID of the AD of the attribute and the value of the attribute. Each relationship is represented as a named list containing the UID of the RD of the relationship, an indication of whether it is a primary or secondary relationship, the UID of its target node, its key attributes, and its non-key attributes. If it is part of a bidirectional relationship, its reverse relationship is also included in the representation as a named list containing the UID of the RD of the reverse relationship and its key attributes. 3

The value of the SMALL_FOR_CAIS_DURATION pragmatic is included in order that the importing CAIS instance be able to interpret TIME attributes. The fourth component of a TIME attribute, termed the SECONDS component, is actually an integer value representing the SECONDS component of CAIS_CALENDAR.TIME in multiples of SMALL_FOR_CAIS_DURATION. 4

Each content section contains a named list, termed the *content descriptor* list, stating the UID of the node whose contents are contained therein, the UID of the NKD of this node, and a count of the number of lines following which hold the actual text contents. This descriptor list is contained in the first line or first few lines of the section. All remaining lines of the section contain the contents of the node. 5

Line breaks in a text file containing an external node set are sometimes significant. Sections of an external node set never share lines. The line containing the closing right parenthesis of a header section is the “closing line” of that section. The line after the closing line of the header section commences the first content section, if one exists. The line containing the closing right parenthesis of a content descriptor is also a closing line. Only spaces and format effecters may lie to the right of the closing right parenthesis on a closing line. The *n* lines (where *n* is given in the descriptor) after the closing line of the descriptor contain the actual text contents. The line following these n lines commences the next content sect ion, if one exists. Line breaks within the header section or within content descriptors are insignificant. 6



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

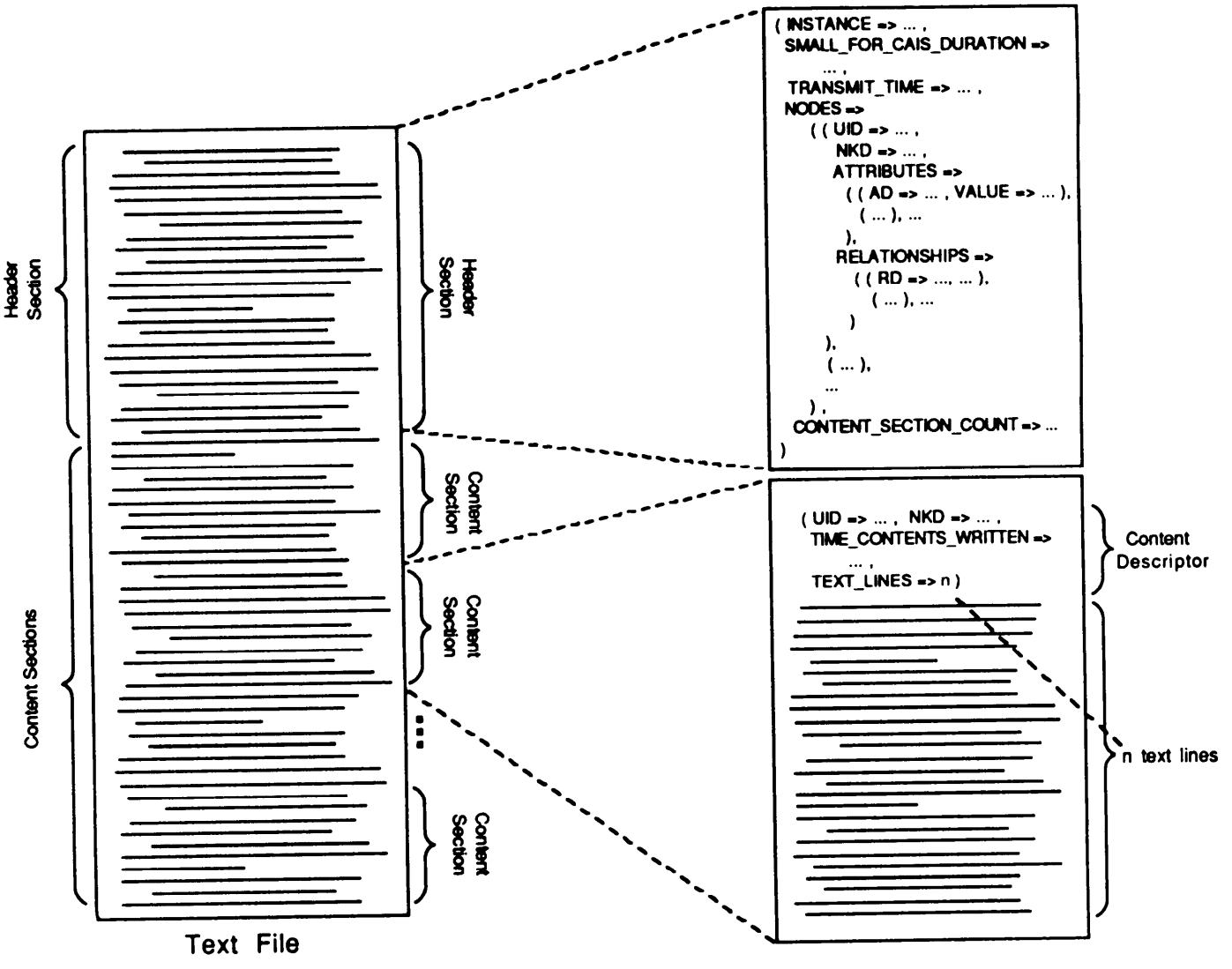


FIGURE 10. **External node set**



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

TABLE II: External Node Set BNF

```
external_node_set     ::= header_section { content_section }

header_section        ::= ( INSTANCE=> cais_instance_name,
                            SMALL_FOR_CAIS_DURATION => attribute_value,
                            TRANSMIT_TIME => attribute_value , NODES => node_s ,
                            CONTENT_ COUNT => integer )

node_s                ::= ([node { , node}])
attribute_s           ::= ( [ attribute{ , attribute} ] )
relationship_s        ::= ( [ relationship{, relationship } ] )

node                  ::= ( UID => node_uid , NKD => definition_uid ,
                            ATTRIBUTES => attribute_s , RELATIONSHIPS =>
                            relationships )
attribute             ::= (AD => definition_uid , VALUE => attribute_value )
relationship          ::= ( RD => definition_uid , KIND => relationship_kind ,
                            TARGET => target_uid , KEYS => attributers ,
                            ATTRIBUTES => attribute_s [ , REVERSE => reverse_link ] )
reverse_link          ::= ( RD => target_definition_uid , KEYS => attribute_s )

content_section       ::= content -descriptor
                          <text lines representing file node contents>

content_descriptor    ::= ( UID => node_uid , NKD => definition_uid ,
                            TEXT_LINES => integer )

cais_instance_name    ::= string

node_uid              ::= string
definition_uid        ::= string
target_uid            ::= string
target_definition_uid ::= string

attribute_value       ::= string

relationship_kind     ::== PRIMARY | SECONDARY
```

Notes:

1. See Appendix D for a description of the notation used.



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

#### 4.5.5.3 Importing data

An external node set can be imported by a CAIS instance, i.e., incorporated into the node model of an importing CAIS instance. Depending on the information transferred, the importation can require considerable human intervention. Much of the importation of a node set will be performed by tools rather than CAIS operations directly. Nevertheless, interfaces are provided to support the importation effort. 1

The act of importing a node set consists of two phases: introduction and integration. Introduction is provided by CAIS interfaces. Integration is to be provided by tools supported by CAIS interfaces. 2

*Introduction* is the act of converting a node set on an external medium to a form that can be easily manipulated by tools. This is the act of "reading in" or introducing a node set from an external file into the node model of the importing CAIS instance. Each node so brought in is termed an *introduced node,* each relationship is an *introduced relationship,* each attribute is an *introduced attribute,* and any node contents are *introduced contents.* The set of introduced nodes, relationships, attributes, and contents is termed the introduced *node set.* 3

After introduction, each exported node represented in the external node set has a corresponding introduced node in the importing CAIS instance. The introduced node is an instance of the predefine node kind INTRODUCED_NODE. The UID of the exported node is represented by the predefined attribute IMPORTED_UID on the corresponding introduced node. The UID of the NKD of the exported node is represented by the predefine attribute IMPORTED_DEFINITION_UID on the corresponding introduced node. 4

Each relationship emanating from an exported node is represented as a relationship of the predefine relation INTRODUCED_RELATIONSHIP on the corresponding introduced node; the latest key is used as the relationship key designator on the creation of these relationships. The target node of the exported relationship is represented by a UID stored in the predefine attribute IMPORTED_TARGET_UID on the introduced relationship. In addition, the introduced relationship will terminate at the introduced node corresponding to the exported target node. If the target node is not exported, the introduced relationship will terminate at an unobtainable node. The UID of the RD of the exported relationship is stored in the predefine attribute IMPORTED_DEFINITION_UID on the introduced relationship. An indication of whether the relationship is a primary or secondary relationship is recorded in the predefine identifier attribute IMPORTED_RELATIONSHIP_KIND. 5

All of the attributes of an exported node are represented within the single predefine list attribute INTRODUCED_NODE_ATTRIBUTES on the corresponding introduced node. This attribute is a list of the exported attributes each of which is represented as a 2-item unnamed list containing the UID of the AD for that attribute and a list representing the value of that attribute. Similarly, all of the key attributes of an exported relationship are represented within the single predefine list attribute INTRODUCED_KEY_ATTRIBUTES on its corresponding introduced node; all of the non-key attributes are represented within the single predefine list attribute INTRODUCED_RELATIONSHIP_-ATTRIBUTES. 6

Contents transferred for exported text FILE nodes are stored as contents of the corresponding introduced node. 7

*Integration* is the act of incorporating the information inherent in an introduced node set into non-introduced nodes of an importing CAIS instance. The desire is to convert the introduced information into the form it had when exported. 8

During integration a situation can exist where there is an introduced node and another node that 9



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

represent the same entity. Part of the integration process is to reconcile these duplicate representations.

Attributes containing UIDs aid in the reconciliation process. An introduced node may be reconciled strictly in terms of UIDs. By interpreting the introduced attributes of an introduced node, one may determine the UID of the node represented by this introduced node. This UID may then be used to search for a corresponding node. Predefine nodes especially might be reconciled in this manner. 10

Reconciliation may be more complicated than matching nodes solely on the basis of UIDs, however. Even with matching UIDs, the information inherent in the two nodes may be different; one node may represent a later or alternate state of the other. The differing information can be handled selectively by the process performing the importation, but the intent is to integrate the introduced information in the introduced node into its corresponding node. Once information is integrated, the introduced node may be deleted. 11

Certain Common IO interfaces allow the UID of a created node to be specified explicitly. The process performing integration may choose to create a node whose UID is the same as the IMPORTED_UID attribute of an introduced node. Under certain scenarios, future importing of nodes from the same exporting CAIS instance is facilitated if UIDs are equivalent in both the importing and exporting instances. 12

Since integration is only minimally supported by the CAIS, interactive tools may be necessary to perform integration. 13



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

# 4.6 Resource Control

The CAIS provides a means to name the physical or logical resources used to implement the CAIS. This allows a process to ascertain the structure of part or all of its operating environment and to explicitly allocate resources in that environment. Where allowed by an implementation, a process can control the assignment of computer processing units to processes, the assignment of storage media to nodes, the routing of data to devices, and the routing of data to other CAIS instances. The use of resource control features is always optional. Tool writers may choose not to influence the automatic assignment of resources, leaving the decisions to the CAIS implementation and the host operating system. 1

CAIS implementations will provide varying degrees of transparency to underlying resources. Resources over which CAIS processes have no control are called *transparent resources.* Transparent resources are managed individually by the CAIS implementation or the host operating system. Such resources cannot be individually named by CAIS processes for the purpose of resource allocation. Examples of transparent resources are pooled permanently mounted disks used for file storage, tightly coupled processors allocated on a millisecond basis by the host operating system, and fast memory. *Non-transparent resources are* resources which processes can name and over which they can exercise some control. Typical examples are specialized processors, printers, tape drives, special target computers, remountable disks, modems, and net work interfaces. In some situations, non-transparent resources are treated as transparent resources; moreover, resources transparent to one process may not be transparent to other processes. 2

The classes of resources recognized by CAIS are: 3

1. *Processor resource:* a computer processing unit capable of executing CAIS processes. A processor resource is a logical entity which may represent one or many physical processing units. 4
2. *Medium resource:* a secondary storage medium used to store nodes, relationships, attributes, and contents of SECONDARY_STORAGE nodes. The medium is a logical entity which may or may not correspond to a single physical volume. 5
3. Device *resource: an* input and/or output device. 6
4. *Gateway resource:* a mechanism used to support communication with another CAIS instance. 7

The functions provided by CAIS resource control are the *assignment of resources:* the assignment of processor resources to processes, the assignment of medium resources to nodes, the assignment of device resources to processes, and the assignment of gateway resources to processes. Assigning a processor resource to a process means that the process is to execute on that processor resource. The assignment takes place when the PROCESS node is created. Assigning a medium resource to a node means that all or part of the physical storage space needed to store the representation of that node, its attributes, its contents, its emanating relationships, and the attributes of its emanating relationships is to reside on that medium resource. The assignment takes place when the node is created or when reassigned. Since there are a variety of methods for implementing the CAIS node model, CAIS implementers are granted considerable latitude in determining which parts of the data representing a node are actually stored on the designated medium resource. This standard does require, however, that as a minimum, the contents of SECONDARY_STORAGE nodes be placed on the designated medium resource. Assigning a device or gateway resource to a process means that the process is allowed to perform input and output to the resource. Where the resource cannot be shared, this also means preventing the resource from being simultaneously assigned to another process. Assignment of a device or gateway resource takes place when a channel is created. 8



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

## 4.6.1 Resource model elements

In order to allow portable tools to adapt to the resources available at different installations, the properties of the resources are represented in the node model. The collection of nodes, attributes, and relationships representing the resource information is called the *resource model.* Resource models will vary from one CAIS instance to another. Table III summarizes the elements of resource models. The rows give the predefine node kinds for nodes from which relationships emanate. The columns of the table give predefine node kinds for nodes at which relationships terminate. The cells give the relations which may occur. The meaning of these is explained below.

**TABLE III: Resource Model Elements**

| SOURCE NODE KINDS | TARGET NODE KINDS | | | | |
|---|---|---|---|---|---|
| | RESOURCE _GROUP | PROCESSOR | MEDIUM | DEVICE | GATEWAY |
| SYSTEM _LEVEL | RESOURCE _GROUP P | PROCESSOR P | MEDIUM P | DEVICE() P | GATEWAY P |
| RESOURCE _GROUP | | PROCESSOR s | MEDIUM s | DEVICE s | GATEWAY s |
| PROCESSOR | | ALLOCATABLE _PROCESSOR s<br>DEFAULT _PROCESSOR s | ALLOCATABLE _MEDIUM s<br>DEFAULT _MEDIUM s | ALLOCATABLE _DEVICE s | ALLOCATABLE _GATEWAY s |
| PROCESS | RESOURCE _GROUP s | CURRENT _PROCESSOR s | | DEVICE so | |
| | P= PRIMARY S= SECONDARY O= OPTIONAL | | | | |

Resources are represented by PROCESSOR, MEDIUM, DEVICE, and GATEWAY nodes. Nodes of these predefined kinds are called *resource nodes.* Each resource node represents either a single physical processor, medium, device, or gateway resource or a group of physical resources to be treated as a single resource. Resource nodes are used to name specific resources. Primary relationships of resource nodes are of the relations PROCESSOR, MEDIUM, DEVICE, and GATEWAY, respectively. These relationships always emanate from the system-level node, forming a single-level tree structure composed of all resource nodes. Figure 11 on page 78, shows such a structure. The nodes in Figure 11 identified by the keys TRANSPARENT_PROCESSOR_RESOURCE and TRANSPARENT_MEDIUM_-RESOURCE are explained in Section 4.6.4 on page 82.

For convenience, resource nodes are grouped, using RESOURCE_GROUP nodes. Secondary PROCESSOR, MEDIUM, DEVICE, and GATEWAY relationships emanating from RESOURCE_GROUP nodes are used to form the group structures. The primary relationships for RESOURCE_GROUP nodes are RESOURCE_GROUP relationships emanating from the system-level node. The structures are a single-level tree of RESOURCE_GROUP nodes below the system-level node and single-level trees of resource nodes below each RESOURCE_GROUP node. The structure consisting of a RESOURCE_ -GROUP node, the emanating PROCESSOR, MEDIUM, DEVICE, and GATEWAY relationships, and



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

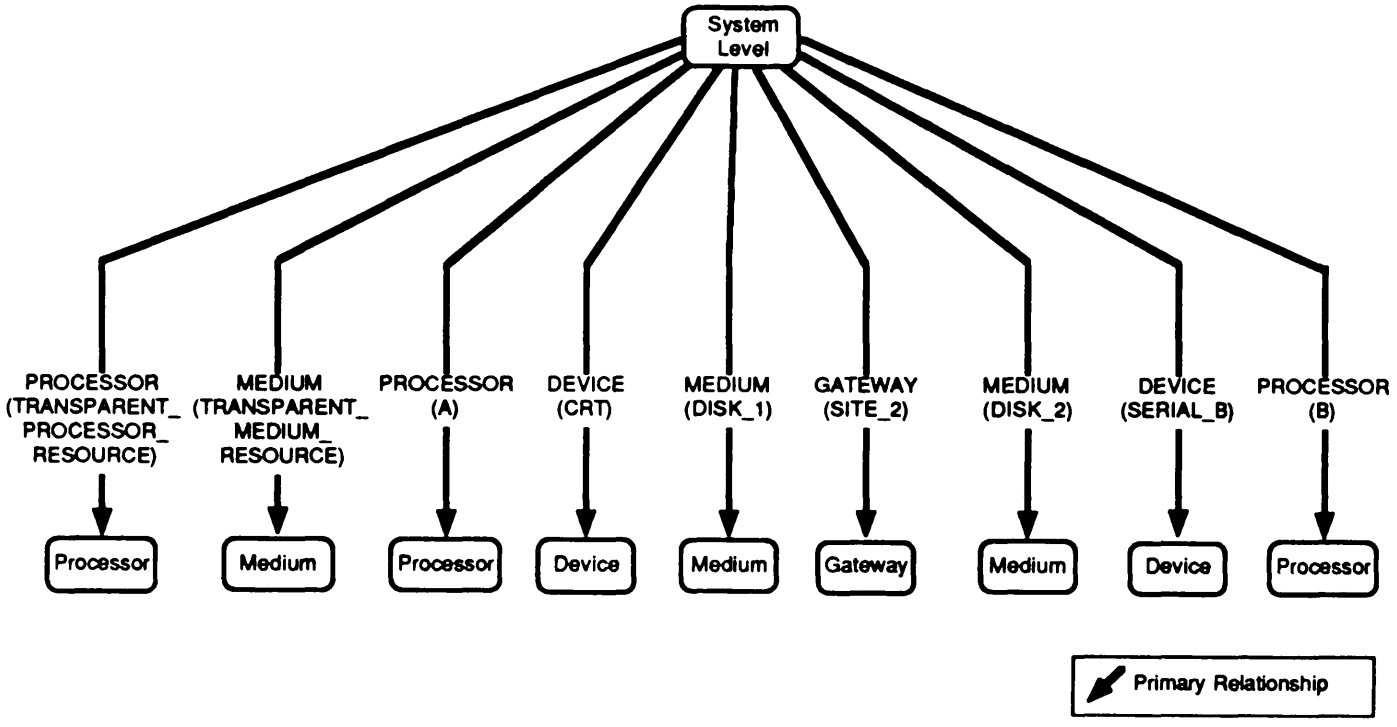


FIGURE 11. Resource nodes example

the resource nodes at which they terminate is called a *resource group*. A resource group can be used to represent the resources at a single workstation or server or a pool of resources available on a network or at a mainframe computer. A single resource node can be in several resource groups. Figure 12 on page 79 gives one example of resource grouping for the resources in Figure 11 on page 78. The example shows the predefined RESOURCE_GROUP node (see Section 4.6.4 on page 82) and RESOURCE_GROUP nodes grouping the resources which are visible from two workstations named WORKSTATION_A and WORKSTATION_B.

Nodes may be mid to be assigned to MEDIUM nodes. This means that the node is assigned to 4
the medium resource which the MEDIUM node represents. This extended terminology also applies to assigning processes or PROCESS nodes to PROCESSOR nodes, and to assigning DEVICE and GATEWAY nodes to PROCESS nodes and processes. The *current PROCESSOR node is* the PROCESSOR node to which the current process is assigned.

A remote computer upon which programs may be loaded and executed from the host computer is 5
usually treated as a device resource; however, if those programs are executed as processes of the CAIS instance under discussion, the computer is treated as a processor resource. If the remote computer supports a distinct CAIS instance, then one or more GATEWAY nodes can be used to support communication between processes in the CAIS instances.

Every resource node possesses the predefine AVAILABLE attribute. When the value of this attribute 6
is TRUE, the resource may be assigned. A resource may be unavailable because it is not functioning, it is not mounted, communication links are disrupted, it is already in use, no storage space is available, or for many other reasons. The value of this attribute is maintained by the underlying implementation, and can not be directly altered by CAIS processes.



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

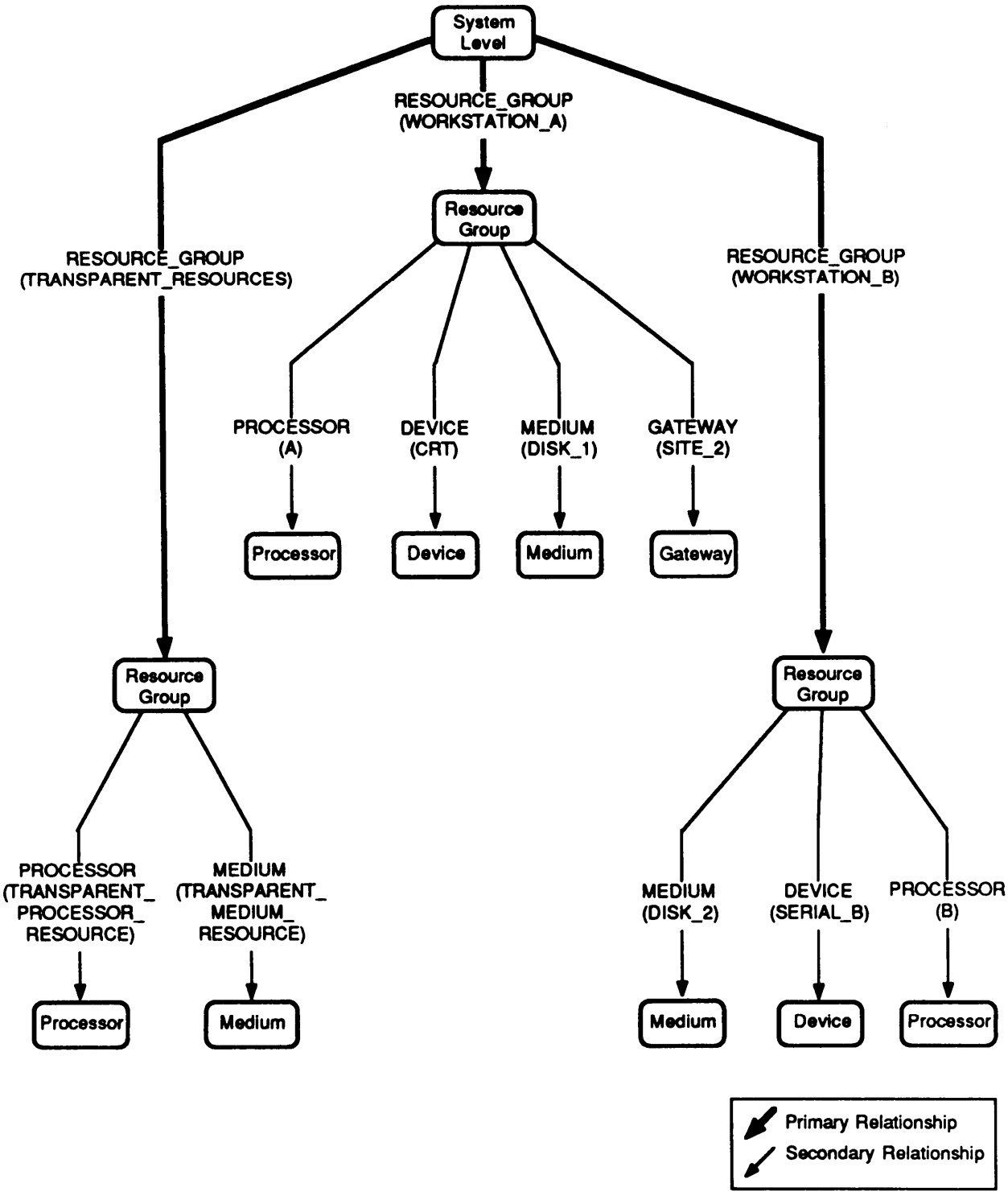


FIGURE 12. **Resource node groups example**


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

## 4.6.2 Resource allocatability

Each PROCESSOR node has emanating secondary relationships designating resources that may be assigned by processes assigned to that PROCESSOR node. These relationships, instances of the relations ALLOCATABLE_PROCESSOR, ALLOCATABLE_MEDIUM, ALLOCATABLE_DEVICE, and ALLOCATABLE_ GATEWAY, terminate at PROCESSOR, MEDIUM, DEVICE, and GATEWAY nodes, respectively. Relationships of these kinds are called *allocatability relationships.* These are illustrated in Figure 13 on page 81, with respect to the resource group example shown in Figure 12 on page 79. The figure exemplifies the situation where resources, identified by the relationships ‘MEDIUM(DISK_l) and ‘GATEWAY(SITE_2), are shared by both processor resources shown and therefore can be allocated by processes executing on either processor; moreover, processes executing on the processor identified by the relationship ‘PROCESSOR(A) can initiate processes assigned to the processor identified by the relationship ‘PROCESSOR(B), but not the reverse. The non-shared resources, i.e. those identified by the relationships ‘DEVICE(CRT), ‘MEDIUM(DISK_2), and ‘DEVICE(SERIAL-B), may be allocated only by the processor within the same resource group. One process may not initiate another process to execute on the same processor unless there is an ALLOCATABLE_PROCESSOR relationship emanating and terminating from the corresponding PROCESSOR node, as shown for both processor resources in Figure 13 on page 81. 1

Each PROCESSOR node may have, at most, one emanating DEFAULT_PROCESSOR relationship and, at most, one emanating DEFAULT_MEDIUM relationship. These secondary relationship terminate at PROCESSOR and MEDIUM nodes, respectively. Interfaces which create PROCESS nodes allow the calling process to optionally specify the PROCESSOR node to which the new process is to be assigned. If this node is not specified, the new process will be assigned the PROCESSOR node designated by the DEFAULT_PROCESSOR relationship emanating from the current PROCESSOR node. If no such DEFAULT_PROCESSOR relationship exists, then the new process will be assigned to a processor resource determined by the CAIS implementation. Interfaces which create nodes allow the calling process to optionally specify the MEDIUM node to which the new node will be assigned. If this node is not specified, the new node will be assigned to the MEDIUM node designated by the DEFAULT_MEDIUM relationship emanating from the current PROCESSOR node. If no such DEFAULT_ MEDIUM relationship exists, then the new node will be assigned to a medium resource determined by the CAIS implementation. 2

In Figure 13 on page 81, the relationships ‘DEFAULT_PROCESSOR() and ‘DEFAULT_MEDIUM() emanating from the PROCESSOR node ‘PROCESSOR(B) designate processor B and the medium resource DISK_2, respectively, as the default processor and default medium resources for processor B. The relationship ‘DEFAULT_MEDIUM() emanating from the PROCESSOR node ‘PROCESSOR(A) and terminating at the MEDIUM node ‘MEDIUM(DISK_l) indicates that the medium resource DISK_1 is the default medium for processor A. The default processor for processor A is implementation-defined due to the absence of a DEFAULT_PROCESSOR relationship emanating from the PROCESSOR node ‘PROCESSOR(A). 3

## 4.6.3 Resource visibility

In some situations, it is desirable to compartmentalize the resource model so that a given process can “see” only a subset of the resources of the entire CAIS instance. Processes gain visibility to resources via secondary RESOURCE_GROUP, DEVICE and CURRENT_PROCESSOR relationships emanating from PROCESS nodes and terminating at RESOURCE_GROUP nodes, DEVICE nodes and PROCESSOR nodes, respectively. These relationships, called *visibility relationships,* are created 1



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

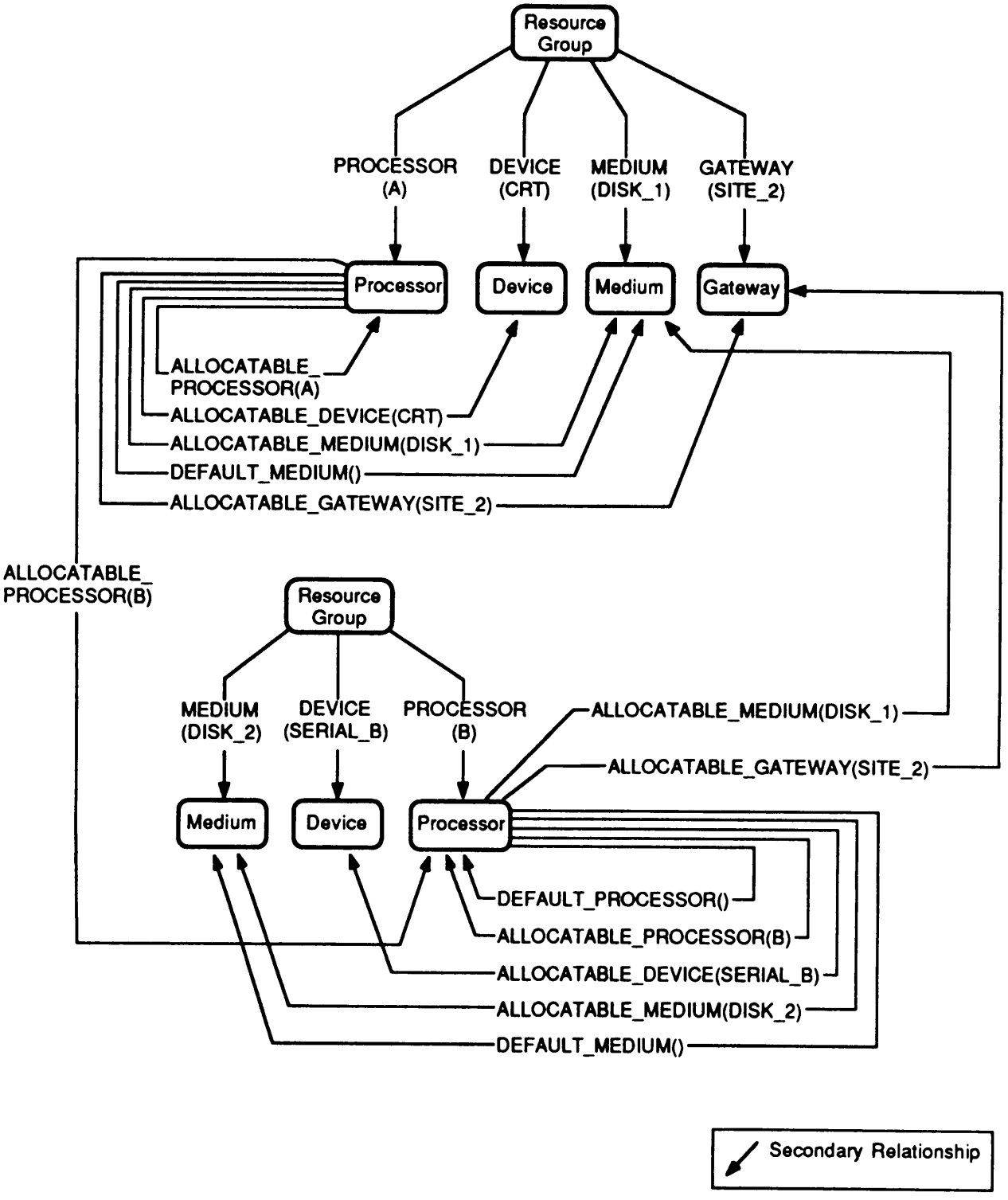


FIGURE 13. **Allocatability relationship example**

Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

by the CAIS implementation when the PROCESS node is created. The resource groups which have visibility relationships are called visible *groups* with respect to some given PROCESS node. The rules for determining which resource groups are made visible by this mechanism are implementation-defined.

When a PROCESS node is created, a CURRENT_PROCESSOR secondary relationship is created emanating from the current PROCESS node and terminating at the current PROCESSOR node. The CURRENT_PROCESSOR relationship is a visibility relationship, however, the implementation must guarantee that the current processor resource is also in a visible group. The resource visibility mechanisms are illustrated in Figure 14 on page 83.

## 4.6.4 Transparent resources

Every CAIS instance has two predefined resource nodes used to collectively represent all transparent resources. The TRANSPARENT_PROCESSOR_RESOURCE node is a PROCESSOR node which represents all processors that cannot be explicitly assigned by the current process. Similarly, the predefine TRANSPARENT_MEDIUM_RESOURCE node is a MEDIUM node which represents all media that cannot be explicitly assigned by the current process. When a node handle to one of these nodes is used to designate a resource, the underlying implementation will choose the resource to which the process or node will be assigned. Rules for making this assignment are implementation-dependent. The assignment may be made to either transparent or non-transparent resources. The two predefine nodes for transparent resources are implicitly allocatable by all processes. There is a predefine resource group for these nodes to which all PROCESS nodes have an emanating secondary relationship identified by ‘RESOURCE_GROUP(TRANSPARENT_RESOURCES), thus guaranteeing the visibility of the transparent resource nodes.

## 4.6.5 Resource access control

The right of a process to view and alter nodes of the resource model is orthogonal to the right of the process to allocate a resource. In other words, a process may be legitimately allowed to view all resources but may not be allowed to allocate all resources. Therefore, in addition to control of resource node visibility via resource groups and the normal access controls, additional controls are needed to limit the allocatability of resources.

For GATEWAY and DEVICE nodes where the resource is allocated by creating a channel, the allocating process must hold an open node handle with intent sufficient to append relationships to the resource node, as specified in Section 7.3.8 on page 589. In order to allocate a processor resource, the calling process must have sufficient access rights to obtain a node handle with EXECUTE intent to the PROCESSOR node. To allocate a medium resource, the calling process must have sufficient access rights to obtain a node handle with APPEND_CONTENTS intent to the MEDIUM node.

## 4.6.6 Assigning resources

Assignment of non-transparent resources can only take place if the resource node involved is available and allocatable. A resource node is *available* if and only if the value of its AVAILABLE attribute is TRUE. A resource node is *allocatable* if and only if there exists an allocatability relationship emanating from the current PROCESSOR node and terminating at the resource node and if the access control requirements described in the previous section are met. If these conditions do not hold, the operation of which the assignment is a part will fail.



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

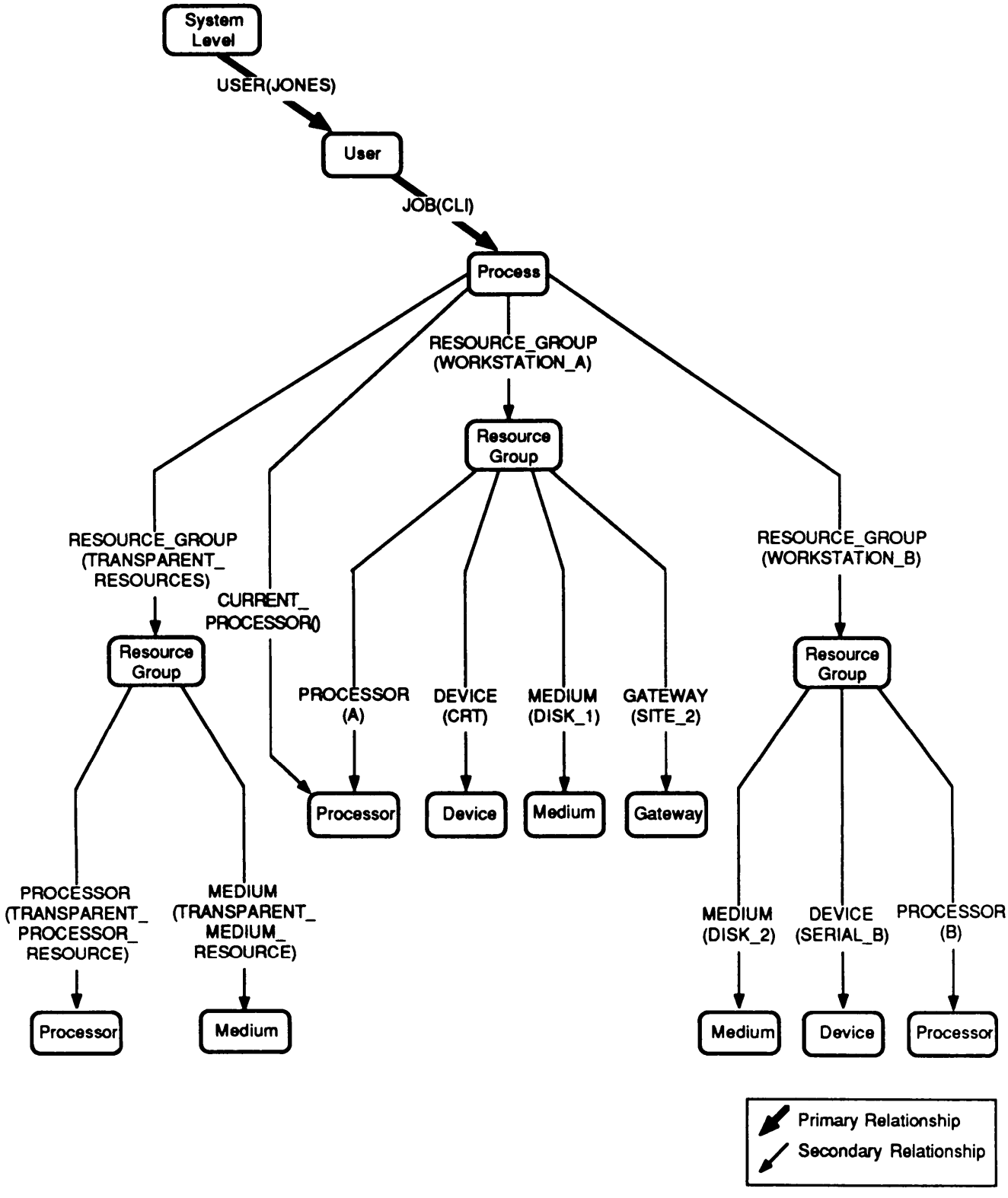


FIGURE 14. **Resource visibility**



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

DEFAULT_PROCESSOR and DEFAULT_MEDIUM relationships do not take the place of allocatability relationships. Therefore, resource nodes which are the targets of DEFAULT_PROCESSOR and DEFAULT_MEDIUM relationships must also be the targets of the appropriate allocatability relationships. 2



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

# 4.7 Distribution

The CAIS supports two methods of distributed implementation: distribution of the implementation of a single CAIS instance and the interconnection of many distinct CAIS instances. An example of the former is a CAIS implementation on a single processor workstation or grouping of tightly coupled processors. An example of the latter is a local area network of workstations where each workstation has interconnected yet distinct CAIS instances. 1

A single CAIS instance can be implemented so that many separate computers are used to support the operation of the CAIS. A process using CAIS interfaces may or may not be able to discern the various physical components which support a distributed implementation. The resource model (Section 4.6 on page 76) is used to make visible to processes those resources which the implementer wishes to allow processes to assign. If the details of the distribution are to be hidden, then the resource model will be sparse. If the details are to be visible, then the resource model will contain nodes representing the resources that processes are able to control. 2

In many situations, distribution means the interconnection of several distinct CAIS instances. To support this method, the CAIS supplies the GATEWAY node concept which facilitates the routine communication among a group of CAIS instances. 3

## 4.7.1 Remote interprocess communications

Several CAIS instances may exist on one system or network. However, since relationships cannot span CAIS instances, it is impossible for a process executing in one CAIS instance to name objects in another CAIS instance. Because of this limitation, the meaning of USER nodes cannot be shared between CAIS instances, and therefore a subject for a process cannot be established across a CAIS instance boundary. However, the GATEWAY node concept provides a method for supporting interprocess communication between CAIS instances. *Remote interprocess communication* is the data transfer between two processes executing in distinct CAIS instances. 1

A GATEWAY node is used to represent a corresponding GATEWAY node in another CAIS instance. The two nodes form a *gateway pair*. Each node is termed the *opposite GATEWAY node* of the other node in the gateway pair. The *opposite CAIS instance* is the CAIS instance that contains the opposite GATEWAY node. The CAIS implementations of the two CAIS instances are responsible for maintaining the correspondence between these nodes. The CAIS implementations are also responsible for ensuring a correspondence between channels connected to the nodes of a gateway pair. Corresponding channels form *channel pairs*, where one channel of each pair is connected to one of the GATEWAY nodes of a gateway pair and the other channel is connected to the other GATEWAY node of the gateway pair. Each channel of a channel pair is termed the *opposite channel* of the other. The CAIS implementations are responsible for ensuring that data flowing into a GATEWAY node from a channel flows out of the counterpart node on the corresponding channel. These concepts are illustrated in Figure 15, below. 2

Using this mechanism, a process connected to a GATEWAY node is able to exchange data with a process executing in another CAIS instance. The exact identity of the remote process is established by a mechanism specific to the CAIS implementations involved. 3

The normal CAIS input and output connection model is used for remote interprocess communication. In most cases, remote interprocess communication and interprocess communication within the same CAIS instance will be indistinguishable to processes. 4



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

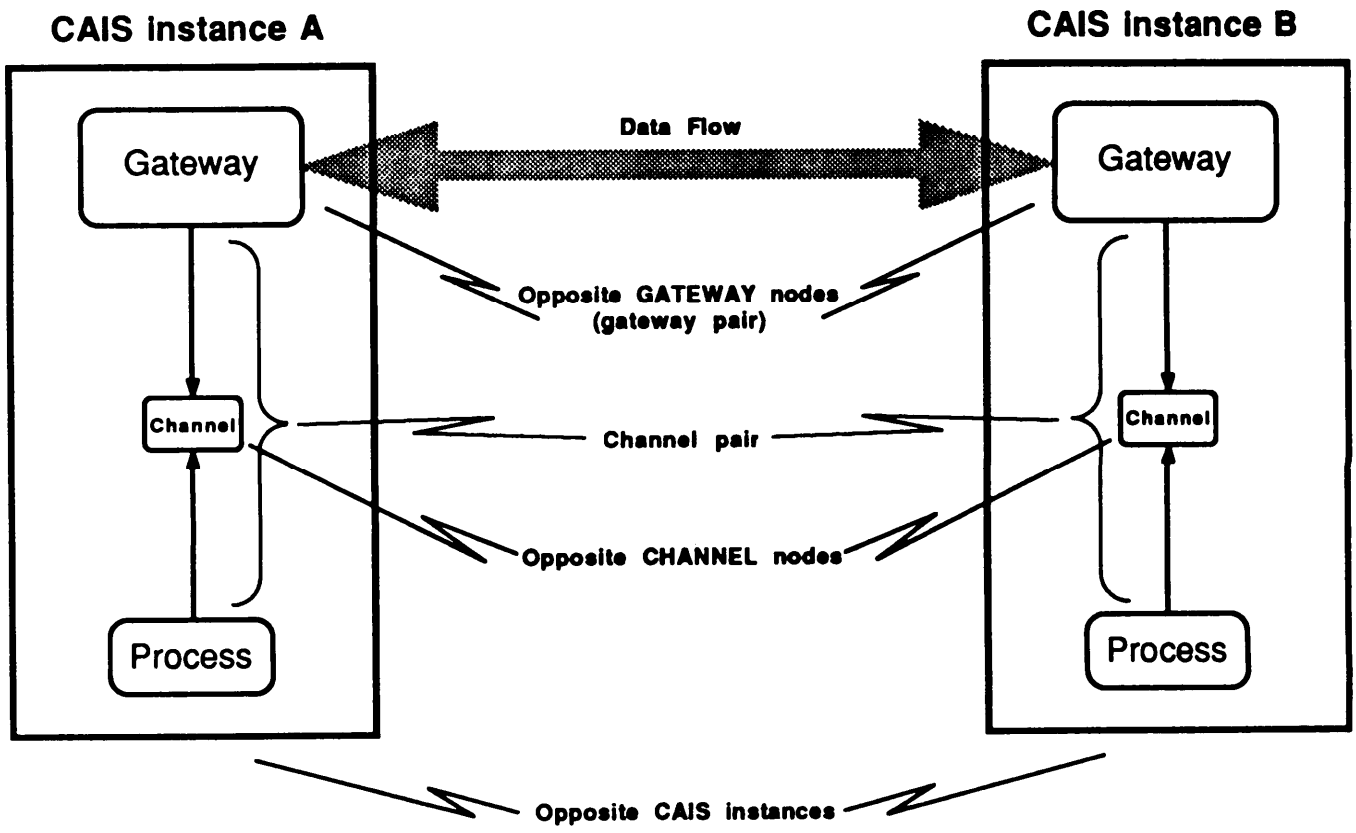


FIGURE 15. **Remote interprocess communication**



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

U.S. GOVERNMENT PRINTING OFFICE: 1990 704-034/30413

METRIC
MIL-STD-1838A
6 April 1989

Superseding
DOD-STD-1838
9 October 1986

# MILITARY STANDARD

## COMMON ADA PROGRAMMING SUPPORT ENVIRONMENT (APSE) INTERFACE SET (CAIS)

(Revision A)

Volume II

NO DELIVERABLE DATA
REQUIRED BY THIS DOCUMENT

AREA MCCR/IPSC

Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

# CONTENTS

**1 SCOPE** 1
1.1 Purpose . . . 1
1.2 Application Guidance . . . 2

**2 REFERENCED DOCUMENTS** 3
2.1 Government Documents . . . 3
2.2 Other Publications . . . 3
2.3 Order of Precedence . . . 4
2.4 Source of Documents . . . 5

**3 DEFINITIONS** 7

**4 GENERAL REQUIREMENTS** 33
4.1 Introduction . . . 33
4.2 Method of Description . . . 37
4.2.1 Allowable differences . . . 37
4.2.2 Typographical conventions . . . 37
4.2.3 Semantic descriptions . . . 38
4.3 CAIS Node Model . . . 40
4.3.1 Nodes . . . 40
4.3.2 Relationships . . . 41
4.3.3 Attributes . . . 43
4.3.4 Processes . . . 43
4.3.5 Transactions . . . 44
4.3.6 Basic predefine definitions . . . 44
4.3.7 Names and identification . . . 50
4.3.8 Relationship Identification . . . 51
4.3.9 Pathnames . . . 53
4.3.10 Unique identifiers . . . 54
4.3.11 CAIS definitions . . . 55
4.3.12 Views . . . 59
4.3.13 Attribute monitors . . . 61
4.4 Access Control . . . 62
4.5 Input and Output . . . 64
4.5.1 Protocols . . . 64
4.5.2 Channels, client nodes, and PORT relationships . . . 65
4.5.3 Creating and using channels . . . 67
4.5.4 Compatibility checks . . . 69
4.5.5 Common Id . . . 69
4.6 Resource Control . . . 76
4.6.1 Resource model elements . . . 77
4.6.2 Resource allocatability . . . 80
4.6.3 Resource visibility . . . 80
4.6.4 Transparent resources . . . 82



Source: https://assist.dla.mil -- Downloaded: 2026l0-26T07:54Z
Check the source to verify that this is the currentversion before use.

4.6.5 Resource access control.. . . . . 82
4.6.6 Assigning resouces . . . . . .82
4.7 Distribution . . . . . 85
4.7.1 Remote interprocess communications . . . . . 85

**5 CAIS NODE MANAGEMENT DETAILED REQUIREMENTS** 87
5.1 Package CAIS_DEFINITIONS. . . . . 88
5.2 Package CAIS_NODE_MANAGEMENT . . . . . 95
5.2.1 Open operations . . . . . 95
5.2.2 Accesses without open operations . . . . . 96
5.2.3 Access delay s..... . . . . 97
5.2.4 Opening a node handle. . . . . 103
5.2.5 Opening a node handle using a unique identifier . . . . . 106
5.2.6 Closing anode handle . . . . . 107
5.2.7 Changing the intent regarding node handle usage . . . . . 108
5.2.8 Examining the open status of a node handle . . . . . 109
5.2.9 Querying the intention of a node handle . . . . . 109
5.2.10 Querying the kind of a node . . . . . 110
5.2.11 Determining if one node kind is a specialization of another . . . . . 110
5.2.12 Determining if one relation is a specialization of another . . . . . 112
5.2.13 Determining if a node belongs to a node kind . . . . . 113
5.2.14 Determining if a relationship belongs to a relation . . . . . 114
5.2.15 Querying the number of open file handles on a CLIENT node . . . . . 116
5.2.16 Obtaining the unique identifier for a node . . . . . 116
5.2.17 Obtaining a pathname for the unique primary path . . . . . 117
5.2.18 Obtaining the relationship key list of a primary relationship . . . . . 118
5.2.19 Obtaining the relation of a primary relationship . . . . . 118
5.2.20 Obtaining the relationship key list of the last relationship traversed . . . 119
5.2.21 Obtaining the relation of the last relationship traversed . . . . . 120
5.2.22 Obtaining a partial pathname . . . . . 121
5.2.23 Obtaining the name of the last relation in a pathname . . . . . 122
5.2.24 Obtaining the key list of the last relationship in a pathname . . . . . 122
5.2.25 Querying the existence of a node . . . . . 123
5.2.26 Querying sameness . . . . . 124
5.2.27 Obtaining an index for a node handle . . . . . 124
5.2.28 Opening a node handle to the parent . . . . . 125
5.2.29 Copying a node. . . . . 126
5.2.30 Copying trees . . . . . 129
5.2.31 Renaming a relationship.. . . . . 134
5.2.32 Deleting the primary relationship to a node . . . . . 138
5.2.33 Deleting the primary relationships of a tree . . . . . 139
5.2.34 Creating secondary relationships . . . . . 141
5.2.35 Creating bidirectional secondary relationships . . . . . 143
5.2.36 Deleting secondary relationships . . . . . 146
5.2.37 Obtaining a corresponding reverse relationship’s relation . . . . . 148
5.2.38 Obtaining a corresponding reverse relationship’s key . . . . . 150
5.2.39 Setting the inheritance property of a relationship . . . . . 151
5.2.40 Determining if a secondary relationship is inheritable . . . . . 153



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

5.2.41 Setting the current node relationship . . . . . 154
5.2.42 Opening a node handle to the current node . . . . . 155
5.2.43 Determining the creation time of anode. . . . . 155
5.2.44 Determining the last time a relationship was modified . . . . . 156
5.2.45 Determining the last time that node contents were written . . . . . 157
5.2.46 Determining the last time an attribute was modified . . . . . 157
5.2.47 Iteration predicate subtype . . . . . 158
5.2.48 Obtaining the default iteration predicate . . . . . 168
5.2.49 Node iteration types and subtypes. . . . . 169
5.2.50 Creating an iterator over nodes.. . . . . 169
5.2.51 Determining node iteration status . . . . . 173
5.2.52 Determining the approximate size of the node iterator . . . . . 174
5.2.53 Getting the next node in a node iteration . . . . . 175
5.2.54 Skipping the next node in a node iteration . . . . . 176
5.2.55 Obtaining the path element for the next node in a node iteration . . . . . 177
5.2.56 Deleting anode iterator . . . . . 178
5.2.57 Node kind instance iteration type . . . . . 178
5.2.58 Creating an iterator over instances of a node kind definition . . . . . 178
5.2.59 Determining node kind instance iteration status . . . . . 180
5.2.60 Determining the approximate size of the node kind instance iterator . . . 181
5.2.61 Getting the next node in a node kind instance iteration . . . . . 181
5.2.62 Skipping the next node in a node kind instance iteration . . . . . 183
5.2.63 Deleting a node kind instance iterator . . . . . 184
5.3 Package CAIS_ATTRIBUTE_MANAGEMENT . . . . . 185
5.3.1 Creating node attributes . . . . . 186
5.3.2 Creating relationships attributes . . . . . 187
5.3.3 Deleting node attributes . . . . . 189
5.3.4 Deleting relationship attributes . . . . . 190
5.3.5 Setting node attributes . . . . . 192
5.3.6 Setting relationships attributes . . . . . 193
5.3.7 Getting node attributes . . . . . 195
5.3.8 Getting relationship attributes . . . . . 196
5.3.9 Generic package CAIS_INTEGER_ATTRIBUTE . . . . . 198
5.3.10 Generic package CAIS_FLOAT_ATTRIBUTE . . . . . 207
5.3.11 Package CAIS_STRING_ATTRIBUTE . . . . . 217
5.3.12 Package CAIS_IDENTIFIER_ATTRIBUTE . . . . . 226
5.3.13 Generic package CAIS_ENUMERATION_ATTRIBUTE . . . . . 235
5.3.14 Package CAIS_BOOLEAN_ATTRIBUTE . . . . . 245
5.3.15 Attribute iteration types and subtypes . . . . . 245
5.3.16 Creating an iterator over node attributes . . . . . 246
5.3.17 Creating an iterator over relationship attributes . . . . . 247
5.3.18 Determining iteration status. . . . . 250
5.3.19 Determining the approximate size of the iterator . . . . . 250
5.3.20 Getting an identification for the next attribute . . . . . 251
5.3.21 Getting the next attribute value . . . . . 252
5.3.22 Skipping the next attribute in an iteration . . . . . 253
5.3.23 Deleting an attribute iterator . . . . . 253
**5.4 Package CAIS_ACCESS_CONTROL_ MANAGEMENT** . . . . . 255



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

5.4.1 Node access. . . . . 255
5.4.2 Security labeling of CAIS nodes . . . . . 255
5.4.3 Groups and roles.... . . . . 257
5.4.4 Granting access rights. . . . . 259
5.4.5 Denying access. . . . . 262
5.4.6 Access checking . . . . . 262
5.4.7 An example of CAIS access control . . . . . 263
5.4.8 Subtypes . . . . . 271
5.4.9 Value of all access rights. . . . . 271
5.4.10 Setting access control.. . . . . 271
5.4.11 Deleting ACCESS relationships . . . . . 273
5.4.12 Obtaining the value of a GRANT attribute . . . . . 274
5.4.13 Denying access . . . . . 275
5.4.14 Deleting DENY relationships . . . . . 275
5.4.15 Examining access rights.. . . . . 276
5.4.16 Adopting a rose..... . . . . 277
5.4.17 Unlinking an adopted role . . . . . 278
5.5 Package CAIS_STRUCTURAL_NODE_MANAGEMENT . . . . . 280
5.5.1 Opening the predefine STRUCTURAL NKD . . . . . 280
5.5.2 Creating structural nodes . . . . . 280
5.6 Package CAIS_TRANSLATION_MANAGEMENT . . . . . 284
5.6.1 Subtypes . . . . . 284
5.6.2 Adopting a view . . . . . 284
5.6.3 Cancelling a view adoption . . . . . 285
5.6.4 Adopting an SKI..... . . . . 286
5.6.5 Cancelling an NKI adoption . . . . . 288
5.6.6 Opening a handle to an NKD by local name . . . . . 289
5.6.7 Opening a handle to an NKI by local name . . . . . 290
5.6.8 Opening a handle to an RD by local name . . . . . 291
5.6.9 Opening a handle to an RI by local name . . . . . 291
5.6.10 Opening a handle to an AD by local name . . . . . 292
5.6.11 Opening a handle to an AKD by local name . . . . . 293
5.6.12 Determining the name for a definition . . . . . 294
5.6.13 Determining the NKD for a node . . . . . 295
5.6.14 Determining the name for a relation . . . . . 296
5.6.15 Deterrnining the RD for a relation . . . . . 297
5.6.16 Determining the name for a node attribute . . . . . 299
5.6.17 Determining the AD for a node attribute . . . . . 300
5.6.18 Determining the name for a relationship attribute . . . . . 301
5.6.19 Determining the AD for a relationship attribute . . . . . 302
5.7 Package CAIS_DEFINITION_MANAGEMENT . . . . . 304
5.7.1 Definition nodes. . . . . 304
5.7.2 Predefined entities . . . . . 342
5.7.3 Views. . . . . 347
5.7.4 Types and subtypes . . . . . 358
5.7.5 Opening the predefined VIEW NKD. . . . . 359
5.7.6 Opening the predefined NKI NKD . . . . . 359
5.7.7 Opening the predefied NKD NKD . . . . . 360



Source: https://assist.dla.mil -- Downloaded: 2026l0-26T07:54Z
Check the source to verify that this is the currentversion before use.

5.7.8 Opening the predefined RI NKD. . . . . 360
5.7.9 Opening the predefined RD NKD . . . . . 361
5.7.10 Opening the predefined AD NKD. . . . . 361
5.7.11 Opening the predefined named composite list AKD NKD . . . . . 362
5.7.12 Opening the predefine identifier AKD NKD . . . . . 362
5.7.13 Creating VIEW nodes . . . . . 363
5.7.14 Deleting the primary relationships of a tree rooted at a VIEW node . . . 366
5.7.15 Creating Nods . . . . . 367
5.7.16 Deleting NKIs and Nods . . . . . 372
5.7.17 Creating Skis . . . . . 373
5.7.18 Renaming NODE-KIND relationships in a view . . . . . 377
5.7.19 Specializing a node kind . . . . . 378
5.7.20 Generalizing node kinds . . . . . 380
5.7.21 Deleting SPECIALIZATION_OF relationships from NKDs . . . . . 382
5.7.22 Creating RDs . . . . . 384
5.7.23 Deleting RIs and RDs . . . . . 389
5.7.24 Creating Rib . . . . . 391
5.7.25 Renaming RELATION relationships in a view . . . . . 394
5.7.26 Specializing a relation. . . . . 395
5.7.27 Generalizing relations. . . . . 397
5.7.28 Deleting SPECIALIZATION-OF relationships from RDs . . . . . 400
5.7.29 Creating bidirectional RDs . . . . . 402
5.7.30 Deleting bidirectional RDs . . . . . 410
5.7.31 Creating ADs . . . . . 412
5.7.32 Deleting ADs . . . . . 415
5.7.33 Importing ADs . . . . . 417
5.7.34 Renaming ATTRIBUTE relationships in a view . . . . . 418
5.7.35 Redefining the AKD for an AD. . . . . 419
5.7.36 Creating Adds . . . . . 421
5.7.37 Creating identifier Adds . . . . . 425
5.7.38 Deleting Adds . . . . . 428
5.7.39 Importing Adds . . . . . 430
5.7.40 Renaming ATTRIBUTE_KIND relationships in a view . . . . . 431
5.7.41 Deleting the primary relationships of a tree rooted at a definition node . 432
5.7.42 Specifying source node kinds for a relation . . . . . 434
5.7.43 Deleting source node kinds for a relation . . . . . 436
5.7.44 Respecifying source node kinds for a relation . . . . . 438
5.7.45 Specifying key attributes for a relation . . . . . 439
5.7.46 Deleting key attributes for a relation . . . . . 442
5.7.47 Respecifying key attributes for a relation . . . . . 443
5.7.48 Specifying target node kinds for a relation . . . . . 445
5.7.49 Deleting target node kinds for a relation . . . . . 447
5.7.50 Specifying attributes for node kinds and relations . . . . . 448
5.7.51 Deleting attributes for node kinds and relations . . . . . 450
5,7.52 Respecifying attributes for node kinds and relations . . . . . 452
5.7.53 Specifying an attribute kind component . . . . . 453
5.7.54 Deleting an attribute kind component . . . . . 455
5.7.55 Specifying a variant attribute alternative . . . . . 457



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

5.7.56 Deleting a variant attribute alternative . . . 458
5.7.57 Setting definition node attributes. . . . 460
5.7.58 Deleting definition node attributes . . . 462
5.7.59 Setting definition relationship attributes . . . 464
5.7.60 Delete definition relationship attributes . . . 467
5.7.61 Changing the NKD for anode... . . . 469
5.7.62 Checking conformance of a node to an NKD . . . 471
5.7.63 Checking conformance of a relationship to an RD . . . 472
5.7.64 Checking conformance of a list to an AD or AKD . . . 475
5.8 Package CAIS_ATTRIBUTE_MONITOR_MANAGEMENT . . . 477
5.8.1 Types . . . 481
5.8.2 Opening the predefine MONITOR NKD . . . 481
5.8.3 Creating MONITOR nodes . . . 482
5.8.4 Deleting the primary relationship to a MONITOR node . . . 489
5.8.5 Associating an event handler process with a MONITOR node . . . 490
5.8.6 Specifying that a node attribute is to be monitored . . . 491
5.8.7 Specifying that a path attribute is to be monitored . . . 494
5.8.8 Canceling monitoring of designated attributes . . . 496
5.8.9 Determining a monitored attribute modification . . . 497
5.8.10 Awaiting a monitored attribute modification . . . **499**
5.9 Package CAIS_RESOURCE_CONTROL . . . 503
5.9.1 Reassigning a node to a new medium resource . . . 503
5.9.2 Determining the medium resource to which a node is assigned . . . 504
5.9.3 Determining the processor resource to which a process is assigned . . . 505
5.9.4 Determining the current default medium resource . . . 506
5.9.5 Determining the current default processor resource . . . 506

**6 CAIS PROCESS MANAGEMENT DETAILED REQUIREMENTS 509**
6.1 Transactions . . . 514
6.1.1 Subtransactions and enclosing transactions . . . 514
6.1.2 Opening transactions. . . . 514
6.1.3 Inclusion of nodes.... . . . 515
6.1.4 Inclusion of operations.. . . . 515
6.1.5 Visibility of effects . . . 516
6.1.6 Closing Transactions . . . 517
6.1.7 Enclosing processes in transactions . . . 518
6.1.8 Eligible effects of operations . . . 518
6.2 Package CAIS_PROCESS_DEFINITIONS . . . 520
6.3 Package CAIS_PROCESS_MANAGEMENT . . . 521
6.3.1 Determining the default priority . . . 524
6.3.2 Creating a dependent PROCESS node . . . 524
6.3.3 Creating a root PROCESS node.. . . . 530
6.3.4 Activating a process . . . 535
6.3.5 Awaiting termination of another process . . . 536
6.3.6 Activating a process and awaiting its termination . . . 537
6.3.7 Spawning a new process. . . . 538
6.3.8 Invoking a new process. . . . 540
6.3.9 Creating and activating anew job . . . 543



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

6.3.10 Deleting a job. . . . . 545
6.3.11 Appending results . . . . . 546
6.3.12 Overwriting resets. . . . . 547
6.3.13 Getting results from a process . . . . . 547
6.3.14 Determining the status of a process . . . . . 548
6.3.15 Getting the parameter list. . . . . 549
6.3.16 Aborting a process. . . . . 550
6.3.17 Suspending a process . . . . . 551
6.3.18 Resuming a process . . . . . 552
6.3.19 Determining the lowest specifiable priority . . . . . 553
6.3.20 Determining the highest specifiable priority . . . . . 553
6.3.21 Determining if the current priority is modifiable. . . . . 554
6.3.22 Modifying the current priority. . . . . 554
6.3.23 Determining the number of open node handles . . . . . 555
6.3.24 Determining the number of input and output units used . . . . . 556
6.3.25 Determining the time of activation . . . . . 556
6.3.26 Determining the time of termination. . . . . 557
6.3.27 Determining the time a process has been active . . . . . 558
6.3.28 Determining the size of a process . . . . . 558
6.4 Package CAIS_TRANSACTION_MANAGEMENT . . . . . 560
6.4.1 Opening a transaction. . . . . 560
6.4.2 Including anode in a transaction . . . . . 561
6.4.3 Including a tree in a transaction . . . . . 562
6.4.4 Closing a transaction.. . . . . 563
6.4.5 Examining the open status of a transaction . . . . . 564
6.4.6 Querying sameness for two transactions . . . . . 565
6.4.7 Determining whether a node is included in a transaction . . . . . 565

**7 INPUT AND OUTPUT DETAILED REQUIREMENTS** 567

7.1 Package CAIS_IO_DEFINITIONS. . . . . 569
7.2 Package CAIS_FILE_NODE_MANAGEMENT . . . . . 571
7.2.1 Opening the predefine SECONDARY_STORAGE NKD . . . . . 571
7.2.2 Creating secondary storage nodes . . . . . 571
7.3 Package CAIS_IO_CONNECTION. . . . . 575
7.3.1 Channel representation details . . . . . 575
7.3.2 Creating and deleting channels . . . . . 578
7.3.3 Opening protocols . . . . . 578
7.3.4 Compatibility rules . . . . . 578
7.3.5 Detailed protocol opening scenarios . . . . . 582
7.3.6 Discontinuing communication. . . . . 587
7.3.7 Opening the predefined PORTED. . . . . 589
7.3.8 Creating a channel. . . . . 589
7.3.9 Releasing a channel . . . . . 593
7.3.10 Deleting a CHANNEL node. . . . . 594
7.3.11 Waiting for notification of attempted opening . . . . . 595
7.3.12 Opening the opposite client node. . . . . 597
7.3.13 Checking PORT relationship kind compatibility . . . . . 598
7.3.14 Setting PORT relationship attributes . . . . . 599



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

7.4 Package CAIS_IMPORT_EXPORT. . . . . 602
7.4.1 Importing a file from the host file system . . . . . 602
7.4.2 Exporting a file to the host file system . . . . . 603
7.5 Package CAIS_COMMON_I0. . . . . 604
7.5.1 Types and exceptions . . . . . 604
7.5.2 Opening the predefine INTRODUCED_NODE NKD . . . . . 604
7.5.3 Opening the predefine INTRODUCED_NODE_RD . . . . . 605
7.5.4 Opening anode set handle . . . . . 605
7.5.5 Including anode in a node set.. . . . . 606
7.5.6 Including a tree in a node set. . . . . 607
7.5.7 Including a structure of nodes in a node set . . . . . 609
7.5.8 Transmitting a node set in common external form . . . . . 610
7.5.9 Closing a node set . . . . . 611
7.5.10 Introducing nodes in common external form . . . . . 612
7.5.11 Introducing contents of introduced nodes . . . . . 616
7.5.12 Creating VIEW nodes with specific UIDs . . . . . 618
7.5.13 Creating NKDs with specific UIDs. . . . . 619
7.5.14 Creating NKIs with specific UIDs . . . . . 620
7.5.15 Creating RDs with specific UIDs. . . . . 621
7.5.16 Creating RIs with specific UIDs . . . . . 622
7.5.17 Creating bidirectional RDs with specific UIDs . . . . . 623
7.5.18 Creating ADs with specific UIDs. . . . . 625
7.5.19 Creating AKDs with specific UIDs. . . . . 626
7.5.20 Creating structural nodes with specific UIDs . . . . . 627
7.5.21 Creating a secondary storage node with a specific UID . . . . . 628

**8 DATA TRANSFER PROTOCOLS DETAILED REQUIREMENTS 629**
8.1 Input and Output Detailed Requirements . . . . . 629
8.2 Package CAIS_FRAME_IO . . . . . 630
8.2.1 Protocol parameters . . . . . 631
8.2.2 Types, subtypes, and exceptions . . . . . 632
8.2.3 Opening a frame file handle . . . . . 633
8.2.4 Closing a frame file handle . . . . . 634
8.2.5 Determining whether a frame file handle is open . . . . . 635
8.2.6 Determining the mode of a frame file handle . . . . . 635
8.2.7 Transmitting a frame. . . . . 636
8.2.8 Receiving a flame. . . . . 637
8.2.9 Resetting a frame file handle . . . . . 638
8.2.10 Determining if all messages have been read . . . . . 639
8.2.11 Determining if incoming queue is empty . . . . . 640
8.2.12 Determining if outgoing queue is full . . . . . 641
8.2.13 Determining the channel node associated with a file . . . . . 641
8.3 Package CAIS_MAGNETIC_TAPE_IO . . . . . 643
8.3.1 Protocol parameters . . . . . 643
8.3.2 Types, subtypes, and exceptions . . . . . 644
8.3.3 Opening a tape drive file handle . . . . . 646
8.3.4 Closing a tape drive file handle.. . . . . 647
8.3.5 Determining whether a file handle is open . . . . . 648



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

8.3.6 Determining the mode of a magnetic tape drive . . . 649
8.3.7 Requesting the mounting of a tape . . . 649
8.3.8 Loading a gape. . . . 650
8.3.9 Unloading a gape. . . . 651
8.3.10 Requesting the dismounting of a tape . . . 651
8.3.11 Determining the position of the tape . . . 652
8.3.12 Rewinding the tape . . . 652
8.3.13 Skipping tape marks . . . 653
8.3.14 Writing a tape mark . . . 654
8.3.15 Determining the status of a magnetic tape drive . . . 654
8.3.16 Determining the recording method of a magnetic tape . . . 655
8.3.17 Determining whether a write ring is installed . . . 655
8.3.18 Skipping blocks in a magnetic tape file . . . 656
8.3.19 Reading a block from a magnetic tape file . . . 657
8.3.20 Writing a block to a magnetic tape file . . . 658
8.3.21 Resetting a magnetic tape file handle . . . 659
8.3.22 Determining the channel node associated with a file . . . 659
8.4 Package CAIS_DIRECT_IO . . . 661
8.4.1 Protocol parameters . . . 661
8.4.2 Definition of types. . . . 662
8.4.3 Creating a direct file . . . 662
8.4.4 Opening a direct file handle . . . 667
8.4.5 Closing a direct file handle. . . . 669
8.4.6 Resetting a direct file handle... . . . 669
8.4.7 Synchronizing internal file with FILE node contents . . . 670
8.4.8 Determining the channel node associated with a file . . . 671
8.5 Package CAIS_SEQUENTIAL_IO . . . 672
8.5.1 Protocol parameters . . . 672
8.5.2 Definition of types . . . 673
8.5.3 Creating a sequential file. . . . 673
8.5.4 Opening a sequential file handle . . . 678
8.5.5 Closing a sequential file handle. . . . 680
8.5.6 Resetting a sequential file hamlle. . . . 680
8.5.7 Synchronizing internal file with FILE node contents . . . 682
8.5.8 Determining the channel node associated with a file . . . 682
8.6 Package CAIS_TEXT_IO . . . 684
8.6.1 Protocol parameters . . . 685
8.6.2 Definition of types . . . 685
8.6.3 Creating a text file . . . 685
8.6.4 Opening a text file handle . . . 690
8.6.5 Closing a text file handle. . . . 692
8.6.6 Resetting a text file handle . . . 693
8.6.7 Synchronizing internal file with FILE node contents . . . 694
8.6.8 Determining the channel node associated with a file . . . 695
8.6.9 Setting the maximum line length of a text file handle . . . 695
8.6.10 Setting the maximum page length of a text file handle . . . 696
8.7 Package CAIS_TERMINAL_IO . . . 698
8.7.1 Protocol parameters . . . 701



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

8.7.2 Types, Subtypes, Constants and Exceptions . . . 701
8.7.3 Opening a terminal file handle. . . . 704
8.7.4 Closing a terminal file handle. . . . 706
8.7.5 Determining whether a terminal file handle is open . . . 707
8.7.6 Determining the channel node associated with a terminal file handle . . . 707
8.7.7 Determining the physical display origin on the virtual display . . . 708
8.7.8 Re-orienting the physical display on the virtual display . . . 709
8.7.9 Determining if a terminal supports a set of graphic renditions . . . 709
8.7.10 Determining if a terminal supports a foreground color . . . 710
8.7.11 Determining if a terminal supports a background color . . . 710
8.7.12 Starting output buffering for a virtual display . . . 711
8.7.13 Ending output buffering for a virtual display . . . 711
8.7.14 Refreshing a virtual display . . . 712
8.7.15 Creating a screen . . . 712
8.7.16 Changing the active rendition for a screen . . . 713
8.7.17 Changing screen size . . . 714
8.7.18 Changing border characteristics . . . 714
8.7.19 Creating a copy of an existing screen . . . 715
8.7.20 Deleting a screen . . . 716
8.7.21 Determining the size of a screen . . . 716
8.7.22 Determining the active rendition of a screen . . . 716
8.7.23 Determining whether a screen has a border . . . 717
8.7.24 Determining the border rendition of a screen . . . 717
8.7.25 Querying sameness of screens . . . 718
8.7.26 Pasting a screen onto the virtual display . . . 719
8.7.27 Removing a screen from a virtual display . . . 719
8.7.28 Relocating a screen on a virtual display . . . 720
8.7.29 Move screen to front . . . 720
8.7.30 Move screen to back . . . 721
8.7.31 Determining paste position of a screen . . . 721
8.7.32 Determining whether a screen is occluded . . . 722
8.7.33 Moving the cursor position on a screen . . . 722
8.7.34 Determining the position of a cursor in a screen . . . 723
8.7.35 Writing a character to a screen. . . . 723
8.7.36 Advancing the cursor to the next line . . . 724
8.7.37 Writing a line of text to a screen . . . 725
8.7.38 Inserting a character into a line. . . . 725
8.7.39 Deleting characters from a line. . . . 726
8.7.40 Inserting blank lines into a screen . . . 727
8.7.41 Deleting lines from a screen . . . 727
8.7.42 Erasing a region of a screen . . . 728
8.7.43 Changing the rendition for a region of a screen . . . 729
8.7.44 Determining a cell rendition.. . . . 729
8.7.45 Reading characters from a screen . . . 730
8.7.46 Sounding the terminal bell. . . . 731
8.7.47 Starting output buffering for a screen . . . 731
8.7.48 Ending output buffering to a screen . . . 732
8.7.49 Enabling function key usage. . . . 732



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

8.7.50 Determining function key usage . . . . . 733
8.7.51 Determining the number of function keys . . . . . 733
8.7.52 Determining the identification of a function key . . . . . 734
8.7.53 Enabling and disabling the locator . . . . . 734
8.7.54 Determining locator usage . . . . . 735
8.7.55 Determining the locator cursor position . . . . . 735
8.7.56 Determining the screen containing the locator cursor . . . . . 736
8.7.57 Determining locator cursor position relative to screen border . . . . . 737
8.7.58 Moving the locator cursor. . . . . 737
8.7.59 Determining the number of locator buttons . . . . . 738
8.7.60 Determining the identification of a locator button . . . . . 738
8.7.61 Reading a single input event... . . . . 739
8.7.62 Reading a string from a virtual keyboard . . . . . 740
8.7.63 Flushing the input event queue.. . . . . 742
8.7.64 Determining if all input events have been read . . . . . 742
8.7.65 Determining if the input event queue is empty . . . . . 743

**9 CAIS LIST MANAGEMENT DETAILED REQUIREMENTS** 745

9.1 Package CAIS_LIST_MANAGEMENT . . . . . 751
9.1.1 Types, subtypes, constants and exceptions . . . . . 751
9.1.2 Copying a list . . . . . 752
9.1.3 Making a list empty . . . . . 753
9.1.4 Converting from text to list forrn. . . . . 753
9.1.5 Converting a list to its text representation . . . . . 754
9.1.6 Determining the equality of two fists . . . . . 755
9.1.7 Deleting an item from a linear fist . . . . . 755
9.1.8 Determining the kin do of list . . . . . 756
9.1.9 Determining the kind of list item. . . . . 757
9.1.10 Inserting a sequence of items into a linear list . . . . . 758
9.1.11 Concatenating two linear lists. . . . . 759
9.1.12 Extracting a sequence of items from a linear list . . . . . 759
9.1.13 Determining the length of alinear list . . . . . 760
9.1.14 Determining the position of the current linear list . . . . . 761
9.1.15 Determining whether the current linear list is outermost . . . . . 761
9.1.16 Making the next outer linear list current . . . . . 761
9.1.17 Making a nested sublist the current linear list . . . . . 762
9.1.18 Determining the length of the text form of a list or a list item . . . . . 763
9.1.19 Determining the name of a named item . . . . . 764
9.1.20 Determining the position of a named item . . . . . 765
9.1.21 Package CAIS_LIST_ITEM . . . . . 766
9.1.22 Package CAIS_IDENTIFIER_ITEM . . . . . 771
9.1.23 Generic package CAIS_ENUMERATION_ITEM . . . . . 780
9.1.24 Package CAIS_BOOLEAN_ITEM . . . . . 786
9.1.25 Generic package CAIS_INTEGER_ITEM . . . . . 787
9.1.26 Generic package CAIS_FLOAT_ITEM . . . . . 792
9.1.27 Package CAIS_STRING_ITEM . . . . . 798



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

**10 CAIS STANDARD DETAILED REQUIREMENTS** 805

10.l Package CAIS_STANDARD. . . . . 805

**11 CAIS CALENDAR DETAILED REQUIREMENTS** 807

ll.l Package CAIS_CALENDAR. . . . . 807

11.1.1 Types, subtypes, and exceptions . . . . . 807

11.1.2 Getting the current time . . . . . 808

11.1.3 Getting the year part of the time . . . . . 808

11.1.4 Getting the month part of the time . . . . . 808

11.1.5 Getting the day part of the time . . . . . 809

11.1.6 Getting the seconds part of the time . . . . . 809

11.1.7 Splitting time into its components . . . . . 810

11.1.8 Combining components of time. . . . . 811

11.1.9 Adding time and duration . . . . . 811

ll.l.l0 Subtracting time and duration . . . . . 812

ll.l.ll Comparing two values of time . . . . . 812

**12 CAIS PRAGMATIC DETAILED REQUIREMENTS** 815

12.1 Package CAIS_PRAGMATICS. . . . . 815

12.1.1 Identifier and view pragmatics . . . . . 815

12.1.2 Node model pragmatics . . . . . 816

12.1.3 Handle and iterator pragmatics. . . . . 817

12.1.4 Integer and real number pragmatics . . . . . 819

12.1.5 Access control pragmatics . . . . . 819

12.1.6 File and device pragmatics. . . . . 820

12.1.7 List pragmatics . . . . . 822

**13 CAIS STATUS MANAGEMENT DETAILED REQUIREMENTS** 825

13.1 Package CAIS_STATUS_DEFINITIONS . . . . . 845

13.2 Package CAIS_STATUS_MANAGEMENT . . . . . 857

13.2.1 Creating a status indicator . . . . . 857

13.2.2 Setting a status indicator . . . . . 857

13.2.3 Copying a status indicator . . . . . 858

13.2.4 Deleting a status indicator . . . . . 858

13.2.5 Obtaining a status ‘indicator’s exception indication . . . . . 859

13.2.6 Obtaining status information from a status indicator . . . . . 859

13.2.7 Obtaining a descriptive message from a status indicator . . . . . 860

13.2.8 Determining if a status indicator is established . . . . . 860

13.3 Exception Conditions . . . . . 862

13.3.1 Conditions for CAIS_DEFINITIONS exceptions . . . . . 862

13.3.2 Conditions for CAIS_PROCESS_DEFINITIONS exceptions . . . . . 908

13.3.3 Conditions for CAIS_IO_DEFINITIONS exceptions . . . . . 908

13.3.4 Conditions for CAIS_FRAME_IO exceptions . . . . . 914

13.3.5 Conditions for CAIS_MAGNETIC_TAPE_IO exceptions . . . . . 915

13.3.6 Conditions for CAIS_TERMINAL_IO exceptions . . . . . 915

13.3.7 Conditions for CAIS_COMMON_IO exceptions . . . . . 918

13.3.8 Conditions for CAIS_LIST_MANAGEMENT exceptions . . . . . 918

13.3.9 Conditions for CAIS_CALENDAR exceptions . . . . . 924

13.3.10 Conditions for CAIS_PRAGMATICS exceptions . . . . . 925




Source: https://assist.dla.mil -- Downloaded: 2026l0-26T07:54Z
Check the source to verify that this is the currentversion before use.

13.3.11 Conditions for CAIS_FORM_TERMINAL_IO exceptions . . . . . 926

**14 GUIDANCE INFORMATION** 927

14.1 Keywords. . . . . 927



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

# APPENDIXES


## Appendix A Predefined Entities 929

10 Predefine node description index . . . . . 929
20 Predefine Node Descriptions . . . . . 940
20.1 Top-level nodes . . . . . 943
20.2 Attribute definition nodes (ADs) . . . . . 960
20.3 Attribute kind definition nodes (AKDs) . . . . . 1000
20.4 Node kind definition nodes (NKDs) . . . . . 1027
20.5 Relation definition nodes (RDs) . . . . . 1050
20.6 Node kind interpretation nodes (NKIs) . . . . . 1114
20.7 Relation interpretation nodes (RIs) . . . . . 1130

## B CAIS Specification 1167

## C Compatibility with Pre-existing Standards 1321

10 Package CAIS_NODE_MANAGEMENT . . . . . 1321
10.1 Opening a node handle . . . . . 1321
10.2 Changing the intent regarding node handle usage . . . . . 1322
10.3 Querying the existence of a node . . . . . 1322
10.4 Querying sameness . . . . . 1322
10.5 Obtaining an open node handle to the parent . . . . . 1323
10.6 Copying a node . . . . . 1323
10.7 Copying trees. . . . . 1324
10.8 Renaming the primary relationship of a node . . . . . 1324
10.9 Deleting the primary relationship to a node . . . . . 1324
10.10 Deleting the primary relationships of a tree . . . . . 1325
10.11 Creating secondary relationships . . . . . 1325
10.12 Deleting secondary relationships . . . . . 1326
10.13 Setting inheritance property of a relationship . . . . . 1326
10.14 Determining if a secondary relationship is inheritable . . . . . 1326
10.15 Setting the current node relationship . . . . . 1327
10.16 Opening a node handle to the current node . . . . . 1327
10.17 Determining the creation time of a node . . . . . 1327
10.18 Determining the last time a relationship was modified . . . . . 1328
10.19 Determining the last time that node contents were written . . . . . 1328
10.20 Determining the last time an attribute was modified . . . . . 1328
10.21 Creating an iterator over nodes . . . . . 1329
10.22 Getting the next node in a node iteration . . . . . 1329
20 Package CAIS_ATTRIBUTE_MANAGEMENT . . . . . 1330
20.1 Creating node attributes . . . . . 1330
20.2 Creating path attributes . . . . . 1330
20.3 Deleting node attributes . . . . . 1330
20.4 Deleting path attributes . . . . . 1331
20.5 Setting node attributes. . . . . 1331
20.6 Setting path attributes . . . . . 1331
20.7 Getting node attributes . . . . . 1332




Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

20.8 Getting path attributes . . . 1332
20.9 Creating an iterator over node attributes . . . 1333
20.10 Creating an iterator over relationship attributes . . . 1333
30 Package CAIS_ACCESS_CONTROL_MANAGEMENT . . . 1334
30.1 Subtypes . . . 1334
30.2 Setting access control . . . 1334
30.3 Deleting access relationships . . . 1334
30.4 Obtaining the value of a GRANT attribute . . . 1335
30.5 Examining access rights . . . 1335
30.6 Adopting a rose . . . 1335
40 Package CAIS_STRUCTURAL_NODE_ MANAGEMENT . . . 1336
40.1 Creating structural nodes . . . 1336
50 Package CAIS_PROCESS_MANAGEMENT . . . 1337
50.1 Spawning a new process . . . 1337
50.2 Invoking a new process . . . 1338
50.3 Awaiting termination of another process . . . 1340
50.4 Deleting a job. . . . 1340
50.5 Getting results from a process . . . 1340
50.6 Determining the status of a process . . . 1341
50.7 Aborting a process . . . 1341
50.8 Suspending a process . . . 1342
50.9 Resuming a process . . . 1342
50.10 Determining the number of open node handles . . . 1343
50.11 Determining the number of input and output units used . . . 1343
50.12 Determining the time of activation . . . 1344
50.13 Determining the time of termination . . . 1344
50.14 Determining the time a process has been active . . . 1344
50.15 Determining the size of a process . . . 1345
60 Package CAIS_IMPORT_EXPORT. . . . 1345
60.1 Importing a fle from the host file systern . . . 1345
60.2 Exporting a file to the host file system . . . 1345
70 Package CAIS_FORM_TERMINAL_IO . . . 1346
70.1 Protocol parameters . . . 1347
70.2 Types, subtypes, and exceptions . . . 1347
70.3 Opening a form terminal file handle . . . 1348
70.4 Closing a form terminal file handle. . . . 1350
70.5 Determining whether a file handle is open . . . 1350
70.6 Determining the channel node associated with a file . . . 1351
70.7 Determining the number of function keys . . . 1351
70.8 Determining intercepted input characters . . . 1352
70.9 Determining intercepted output characters . . . 1352
70.10 Creating a form . . . 1353
70.11 Deleting a form . . . 1354
70.12 Copying a form . . . 1354
70.13 Defining a qualified area. . . . 1355
70.14 Removing an area qualifier . . . 1355
70.15 Changing the active position. . . . 1356
70.16 Querying the active position . . . 1357



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

70.17 Advancing forward to qualified area . . . 1357
70.18 Writing to a form . . . 1358
70.19 Erasing a qualified area . . . 1359
70.20 Erasing a form . . . 1359
70.21 Activating a form on a tertinal . . . 1360
70.22 Reading from a form . . . 1360
70.23 Determining changes to a form . . . 1361
70.24 Determining the identification of a function key . . . 1362
70.25 Determining the termination key . . . 1362
70.26 Determining the size of a form . . . 1363
70.27 Determining the size of the terminal . . . 1363
70.28 Determining if the area qualifier requires space in the form . . . 1364
70.29 Determining if the area qualifier requires space on a terminal . . . 1364
80 Package CAIS_PRAGMATICS . . . 1365

**D Syntax Summary** 1367
l0 Pathname Syntax. . . . 1368
20 GRANT Attribute Value Syntax. . . . 1368
30 Classification Attribute Value Syntax . . . 1368
40 List External Representation Syntax . . . 1369
50 Iteration Predicate Syntax. . . . 1370
60 External Node Set Syntax. . . . 1371
70 ATTRIBUTES-MONITORED Attribute Value Syntax . . . 1372
80 Constraints Attribute Value Syntax . . . 1373
90 Status Information List Syntax . . . 1374

**E Discretionary Access Control Conformance Guidelines** 1381
10 Class Cl . . . 1381
10.1 Capability a: user control of objects . . . 1381
10.2 Capability b: user ability to specify access modes . . . 1381
10.3 Capability c: user ability to specify individuals or groups . . . 1382
20 Class C2 . . . 1382
20.1 Capability a: user ability to control objects . . . 1383
20.2 Capability b: user ability to specify and control access modes . . . 1383
20.3 Capability c: user ability to specify individual users or groups of individual users . . . 1383
20.4 Capability d: user ability to include or exclude object access to the granularity of an individual . . . 1383
20.5 Capability e: assignment of access control by an authorized user . . . 1384
30 Class B3. . . . 1384
30.1 Capability a: user ability to control objects . . . 1385
30.2 Capability b: user ability to specify and control access modes . . . 1385
30.3 Capability c: access control list... . . . 1385
30.4 Capability d: user ability to include or exclude object access to the granularity of an individual . . . 1385
30.5 Capability e: access permission to an object must be assigned by an authorized user. . . . 1385
30.6 Capability f: ability to deny access to individuals or groups . . . 1385



Source: https://assist.dla.mil -- Downloaded: 2026l0-26T07:54Z
Check the source to verify that this is the current version before use.

**F Implementation Dependencies** 1387

**G List Of Interfaces** 1389

**Index** 1407



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

# FIGURES


Figure 1 CAIS system concept . . . . . 45
2 Tree formed by primary relationships . . . . . 47
3 Some predefined relations . . . . . 49
4 Example node kind definition and attribute definition . . . . . 56
5 Primary relationships to definition nodes . . . . . 57
6 VIEW node and emanating relationships . . . . . 60
7 Channels . . . . . 65
8 Multiple channels . . . . . 66
9 Channel state transition . . . . . 68
10 External node set . . . . . 72
11 Resource nodes example . . . . . 78
12 Resource node groups example . . . . . 79
13 Allocatability relationship example . . . . . 81
14 Resource visibility . . . . . 83
15 Remote interprocess communication . . . . . 86
16 COPY_ TREE example . . . . . 131
17 Discretionary access control . . . . . 264
18 Discretionary access control after process creation . . . . . 265
19 ACCESS relationships, Example l.. . . . . 267
20 ACCESS relationships, Example 2. . . . . 269
21 ACCESS relationships, Example 3. . . . . 270
22 Inheritance of properties and constraints . . . . . 305
23 Example of an NKD lattice . . . . . 307
24 Node kind definition node (NOD). . . . . 308
25 NKD and RD property representation . . . . . 309
26 Relation definition node (RD) . . . . . 311
27 Bidirectional RD. . . . . 312
28 Basic NKD and RD . . . . . 314
29 Attribute definition node (AD) and attribute kind definition node(AKD). . . . . 315
30 Simple and variant Adds . . . . . 321
31 Example of a variant AKD . . . . . 323
32 List, uniform list and composite list AKDs . . . . . 325
33 GRANT attribute kind representation . . . . . 326
34 TIME attribute kind representation . . . . . 327
35 Node kind interpretation node (NKI) . . . . . 347
36 Local names defined by an NKI . . . . . 348
37 Relation interpretation node (RI) . . . . . 349
38 Local names defined by an RI. . . . . 350
39 Adoption of views by a root process . . . . . 352
40 Adoption of views by a dependent process . . . . . 353
41 Allowable process status transitions . . . . . 511
42 Full channel representation . . . . . 575
43 Frame IO message queues . . . . . 630
44 Magnetic tape status transitions . . . . . 645




Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

45 Virtual display with screens . . . . . 699



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

# TABLES


| | | |
|---|---|---|
| **Table** | I | Pathname BNF. . . . . 52 |
| | II | External Node Set BNF . . . . . 73 |
| | III | Resource Model Elements . . . . . 77 |
| | IV | Intents. . . . . 98 |
| | V | Matrix of Access Synchronization Constraints. . . . . 102 |
| | VI | Iteration Predicate BNF . . . . . 160 |
| | VII | Logical Operators in Iteration Predicates . . . . . 165 |
| | VIII | Applications of Attribute Functions . . . . . 165 |
| | IX | Classification Attribute Value BNF. . . . . 256 |
| | X | Predefined access rights . . . . . 260 |
| | XI | GRANT Attribute Value BNF . . . . . 262 |
| | XII | Predefined AKD Attributes. . . . . 319 |
| | XIII | Predefined Emanating Relations for AKDs . . . . . 320 |
| | XIV | Predefine Attributes of Relations Emanating from AKDs . . . . . 320 |
| | XV | Matrix of Composite List AKDs. . . . . 324 |
| | XVI | Predefined Relations that allow the CONSTRAINTS Attribute . . . . . 329 |
| | XVII | Constraints Attribute Value BNF . . . . . 330 |
| | XVIII | Applicable Constraint Items. . . . . 331 |
| | XIX | Inherited Properties and Constraints . . . . . 336 |
| | XX | Predefined Definition Node Attributes . . . . . 340 |
| | XXI | Predefined Definition Relationship Attributes . . . . . 341 |
| | XXII | ATTRIBUTES_MONITORED Attribute Value BNF . . . . . 478 |
| | XXIII | Relationships Created as a Result of CREATE_MONITOR-NODE . . . . . 480 |
| | XXIV | MONITOR Node Attributes Created by CREATE_MONITOR-NODE . . . . . 481 |
| | XXV | Events Causing Process Status Transition . . . . . 512 |
| | XXVI | Relationships Created for Root PROCESS Nodes . . . . . 522 |
| | XXVII | Relationships Inherited or Created for Dependent PROCESS Nodes . . . . . 523 |
| | XXVIII | DIRECTION Attribute Values.. . . . . 576 |
| | XXIX | Channel State Transition . . . . . 577 |
| | XXX | DIRECTION Compatibility . . . . . 580 |
| | XXXI | Intent-DIRECTION Compatibility. . . . . 580 |
| | XXXII | Mode-DIRECTION Compatibility . . . . . 581 |
| | XXXIII | Intents for Protocol Opening . . . . . 581 |
| | XXXIV | Mode Compatibility . . . . . 581 |
| | XXXV | Detailed Protocol Opening Scenario . . . . . 583 |
| | XXXVI | Allowed Magnetic Tape Characters . . . . . 643 |
| | XXXVII | List External Representation BNF . . . . . 749 |
| | XXXVIII | Status Information List BNF . . . . . 827 |
| | XXXIX | Status Information List Description . . . . . 834 |
| | XL | Index To Predefined Node Descriptions . . . . . 929 |
| | XLI | Predefined Node Description Fields . . . . . 941 |




Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
        (RELATION_DEFINITION1, RELATION_DEFINITION2, STATUS);

    CLOSE (RELATION_DEFINITION1);
    CLOSE (RELATION_DEFINITION2);
    return RESULT;
exception
    when others =>
        CLOSE (RELATION_DEFINITION1);
        CLOSE (RELATION_DEFINITION2);
        raise;
end IS_SUB_RELATION;
```

### 5.2.13 Determining if a node belongs to a node kind

```
function IS_OF_KIND
    (NODE     : in    NODE_TYPE;
     KIND     : in    HKD_TYPE;
     STATUS   : in    STATUS_INDICATOR := NO_INDICATION)
    return BOOLEAN;
```

Purpose:

This function returns TRUE if the node identified by NODE belongs to the node kind defined by the NKD identified by KIND. That is, TRUE is returned if the NKD for NODE is the NKD identified by KIND or any specialization of the NKD identified by KIND. Otherwise, FALSE is returned. 1

Parameters:

**NODE** is an open node handle. 2

**KIND** is an open node handle identifying an NKD. 3

**STATUS** is an established status indicator. 4

Exceptions:

**STATUS_ERROR** is raised if the node handle NODE or KIND is not open [2] or if STATUS is not an established status indicator [3]. 5

**NODE_KIND_ERROR** is raised if KIND does not identify an NKD [1]. 6

**SECURITY_VIOLATION** is raised if the operation represents a violation of mandatory access controls. 7

Additional Interfaces:

```
function IS_OF_KIND
    (NODE     : in    NODE_TYPE;
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
        KIND        : in    NODE_KIND_NAME;
        STATUS      : in    STATUS_INDICATOR := NO_INDICATION)
    return BOOLEAN
is
    KIND_DEFINITION : NKD_TYPE;
    RESULT          : BOOLEAN;
begin
    OPEN_NODE_KIND_DEFINITION (KIND_DEFINITION, KIND, STATUS);

    RESULT := IS_OF_KIND (NODE, KIND_DEFINITION, STATUS);

    CLOSE (KIND_DEFINITION);
    return RESULT;
exception
    when others =>
        CLOSE (KIND_DEFINITION);
        raise;
end IS_OF_KIND;
```

### 5.2.14 Determining if a relationship belongs to a relation

```
function IS_OF_RELATION
    (BASE        : in    NODE_TYPE;
     KEY         : in    RELATIONSHIP_KEY := EMPTY_KEY_LIST;
     RELATION    : in    RD_TYPE;
     OF_RELATION : in    RD_TYPE;
     STATUS      : in    STATUS_INDICATOR := NO_INDICATION)
    return BOOLEAN;
```

Purpose:

This function returns TRUE if the relationship identified by BASE, KEY and RELATION belongs to the relation identified by OF_RELATION. That is, TRUE is returned if the RD for the relationship is the RD identified by OF_RELATION or any specialization of the RD identified by OF_RELATION. Otherwise, FALSE is returned.

Parameters:

**BASE** is an open node handle to the source node of the relationship.

**KEY** is a relationship key list which identifies the relationship in the relation identified by RELATION.

**RELATION** is an open node handle identifying an RD that defines are lation containing the relationship.

**OF_RELATION** is an open handle to the RD defining the relation for which inclusion is to be determined.

**STATUS** is an established status indicator.


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Exceptions:

**NOT_IN_VIEW_ERROR** is raised if a keyname specified in KEY cannot be translated [1] or if KEY consists of a single relationship key designator and no key of the specified relation is visible [5]. 7

**AMBIGUOUS_VIEW_ERROR** is raised if translation of a key name specified in KEY is ambiguous [1] or if KEY consists of a single relationship key designator and more than one key of the specified relation is visible [4]. 8

**RELATIONSHIP_ERROR** is raised if KEY specifies a value for the same key more than once [5], if BASE, KEY and RELATION identify more than one relationship [1] or if the relationship specified by BASE, KEY and RELATION does not exist [2]. 9

**STATUS_ERROR** is raised if the node handle BASE, RELATION or OF_RELATION is not open [2] or if STATUS is not an established status indicator [3]. 10

**NODE_KIND_ERROR** is raised if RELATION or OF_RELATION does not identify an RD [1]. 11

**INTENT_VIOLATION** is raised if BASE was not opened with an intent establishing the right to read relationships [1]. 12

**ACCESS_VIOLATION** is raised if the discretionary access rights of the current process are insufficient to allow access to the RD of the relationship identified by BASE, KEY and RELATION with intent READ_RELATIONSHIPS [1]. 13

**SECURITY_VIOLATION** is raised if the operation represents a violation of mandatory access controls. 14

Additional Interfaces:

```
function IS_OF_RELATION
    (BASE          : in    NODE_TYPE;
     KEY           : in    RELATIONSHIP_KEY  := EMPTY_KEY_LIST;
     RELATION      : in    RELATION_NAME;
     OF_RELATION   : in    RELATION_NAME;
     STATUS        : in    STATUS_INDICATOR  := NO_INDICATION)
    return BOOLEAN
is
     RELATION_DEFINITION       : RD_TYPE;
     OF_RELATION_DEFINITION    : RD_TYPE;
     RESULT                    : BOOLEAN;
begin
     DEFINITION_OF_RELATION (RELATION_DEFINITION, BASE, RELATION, STATUS);
     OPEN_RELATION_DEFINITION (OF_RELATION_DEFINITION, OF_RELATION, STATUS);

     RESULT := IS_OF_RELATION (BASE, KEY, RELATION_DEFINITION,
                               OF_RELATION_DEFINITION, STATUS);

     CLOSE (RELATION_DEFINITION);
     CLOSE (OF_RELATION_DEFINITION);
     return RESULT;
```


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
exception
    when others =>
        CLOSE (RELATION_DEFINITION);
        CLOSE (OF_RELATION_DEFINITION);
        raise;
end IS_OF_RELATION;
```

Notes:

In the additional interface, RELATION is anode-relative name, relative to BASE, while OF.-RELATION is a view-relative name in the composite view,

### 5.2.15 Querying the number of open file handles on a CLIENT node

```
function OPEN_FILE_HANDLE_COUNT
    (NODE   : in  CLIENT_NODE_TYPE;
     STATUS : in  STATUS_INDICATOR := NO_INDICATION)
    return CAIS_NATURAL;
```

Purpose:

This function returns the number of open file handles associated with the CLIENT node handle NODE.

Parameters:

**NODE** is an open node handle.

**STATUS** is an established status indicator.

Exceptions:

**STATUS_ERROR** is raised if the node handle NODE is not open [2] or if STATUS is not an established status indicator [3].

**NODE_KIND_ERROR** is raised if the node identified by NODE is not a CLIENT node [l].

### 5.2.16 Obtaining the unique identifier for a node

```
function UNIQUE_IDENTIFICATION
    (NODE   : in  NODE_TYPE;
     STATUS : in  STATUS_INDICATOR := NO_INDICATION)
    return UNIQUE_IDENTIFIER;
```

Purpose:

This function returns the unique identifier for the node identified by the open node handle NODE.


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Parameters:

**NODE** is an open node handle identifying the node whose unique identifier is to be returned. 2
**STATUS** is an established status indicator. 3

Exceptions:

**STATUS_ERROR** is raised if the node handle NODE is not open [2] or if STATUS is not an established status indicator [3]. 4

### 5.2.17 Obtaining a pathname for the unique primary path

```
function PRIMARY_NAME
    (NODE   : in  NODE_TYPE ;
     STATUS : in  STATUS_INDICATOR  := NO_INDICATION)
    return PATHNAME;
```

Purpose:

This function returns a pathname for the unique primary path to the node identified by NODE. Given the same composite view, the OPEN interface may be called with the pathname returned by this function to traverse the unique primary path to the node identified by NODE and open a handle to that node. Section 5.7.3.6 on page 355 describes pathname formation. 1

Parameters:

**NODE** is an open node handle identifying the node. 2
**STATUS** is an established status indicator. 3

Exceptions:

**NAME_ERROR** is raised if any node traversed on the primary path to the node is inaccessible [1]. 4
**NOT_IN_VIEW_ERROR** is raised if a path element of the pathname of the primary path to the node identified by NODE cannot be formed [3]. 5
**AMBIGUOUS_VIEW-ERROR** is raised if formation of a path element of the primary pathname to NODE is ambiguous [2] or if translation of a path element formed is ambiguous [1]. 6
**STATUS_ERROR** is raised if the node handle NODE is not open [2] or if STATUS is not an established status indicator [3]. 7
**TRANSACTION_ERROR** is raised if the operation is included in more than one transaction [1]. 8
**LOCK_ERROR** is raised if access consistent with intent READ_RELATIONSHIPS to any node traversed on the primary path cannot be obtained due to an existing lock on the node [1]. 9
**INTENT_VIOLATION** is raised if NODE was not opened with an intent establishing the right to read relationships [1]. 10
**ACCESS_VIOLATION** is raised if the discretionary access control rights of the current process are insufficient to traverse the node's primary path [1]. 11


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

### 5.2.18 Obtaining the relationship key list of a primary relationship

```
function PRIMARY_ KEY
    (NODE   : in    NODE_TYPE;
     STATUS : in    STATUS_INDICATOR  := NO_INDICATION)
    return RELATIONSHIP_KEY;
```

Purpose:

This function returns the relationship key list of the last path element of a pathname identifying the unique prirnary path to the node identified by NODE. Section 5.7.3.6 on page 355 describes the formation of path elements. 1

Parameters:

**NODE** is an open node handle identifying the node. 2

**STATUS** is an established status indicator. 3

Exceptions:

**NAME_ERROR** is raised if the parent of the node identified by NODE is inaccessible [l]. 4

**NOT_IN_VIEW_ERROR** is raised if the last path element of the pathname of the primary path to the node identified by NODE [3] cannot be formed. 5

**AMBIGUOUS_VIEW_ERROR** is raised if formation of the last path element of the primary pathname to the node identified by NODE is ambiguous [2] or if translation of the path element formed is ambiguous [1]. 6

**STATUS_ERROR** is raised if the node handle NODE is not open [2] or if STATUS is not an established status indicator [3]. 7

**TRANSACTION_ERROR** is raised if the operation is included in more than one transaction [1]. 6

**LOCK_ERROR** is raised if the parent is locked against reading relationships [1]. 9

**INTENT_VIOLATION** is raised if the node handle NODE was not opened with an intent establishing the right to read relationships [1]. 10

**ACCESS_VIOLATION** is raised if the discretionary access control rights of the current process are insufficient to obtain access to the node's parent consistent with intent READ_-RELATIONSHIPS [1]. 11

### 5.2.19 Obtaining the relation of a primary relationship

```
procedure PRIMARY_RELATION
    (RELATION :  in out RD_TYPE;
     NODE     :  in    NODE_TYPE;
     STATUS   :  in    STATUS_INDICATOR  : = NO_INDICATION);

function PRIMARY_RELATION
    (NODE     :  in    NODE_TYPE;
     STATUS   :  in    STATUS_INDICATOR  : = NO_INDICATION)
    return RELATION_NAME;
```


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Purpose:

This interface returns an identification of the relation for the incoming primary relationship to the node identified by NODE. In the first form, an open node handle to the RD for the relation with intent NO_ACCESS is returned in RELATION. In the second form, the relation name of the last path element of a pathname identifying the unique primary path to the node identified by NODE is returned as the result of the function. The formation of relation names is described in Section 5.7.3.6 on page 355.

No path is traversed in opening the node handle RELATION; hence, the subprograms PATH_-KEY (Section 5.2.20 on page 119) and PATH_RELATION (Section 5.2.21 on page 120) cannot be applied to the node handle RELATION.

Parameters:

**RELATION** is a node handle, initially closed, to be opened to the RD node for the relation for the primary relationship to the node identified by NODE.

**NODE** is an open node handle identifying the node.

**STATUS** is an established status indicator.

Exceptions:

**NAME_ERROR** is raised if the parent of the node identified by NODE is inaccessible [l].

**NOT_IN_VIEW_ERROR** is raised if a relation name is to be returned but no name for the relation can be formed [3].

**AMBIGUOUS_VIEW_ERROR** is raised if a relation name is to be returned and either formation of a name for the relation is ambiguous [2] or translation of the relation name formed is ambiguous [1].

**STATUS_ERROR** is raised if the node handle RELATION is already open at the time of the call on PRIMARY_RELATION [1], if NODE is not an open node handle [2] or if STATUS is not an established status indicator [3].

**TRANSACTION_ERROR** is raised if the operation is included in more than one transaction [1].

**LOCK_ERROR** is raised if the parent is locked against reading relationships [1].

**INTENT-VIOLATION** is raised if NODE was not opened with an intent establishing tile right to read relationships [1].

**ACCESS_VIOLATION** is raised if the discretionary access control rights of the current process are insufficient to obtain access to the node's parent consistent with intent to READ_-RELATIONSHIPS [1].

### 5.2.20 Obtaining the relationship key list of the last relationship traversed

```
function PATH-KEY
   (NODE   : in     NODE_TYPE;
    STATUS : in     STATUS_INDICATOR := NO_INDICATION)
   return RELATIONSHIP_KEY;
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Purpose:

This function returns the relationship key list used to identify the last relationship traversed in opening the node handle NODE. The relationship key list is returned even if the relationship has been deleted. The relationship key list is formed as described in Section 5.7.3.6 on page 355. However, the keys and values included are the same as those specified when the relationship was traversed, and the resulting specification may no longer uniquely identify the relationship. The key names are formed in the context of the relation for the traversed relationship.

Parameters:

**NODE** is an open node handle.

**STATUS** is an established status indicator.

Exceptions:

**NOT_IN_VIEW_ERROR** is raised if the RD for the relationship traversed or any of the ADs for the keys specified when the relationship was traversed have been deleted, or if the relationship key list used to traverse the last relationship cannot be formed [3].

**AMBIGUOUS_VIEW_ERROR** is raised if formation of the relationship key list used to traverse the last relationship is ambiguous [2] or if translation of the relationship key list formed is ambiguous [I].

**STATUS_ERROR** is raised if the node handle NODE is not open [2] or if STATUS is not an established status indicator [3].

**USE_ERROR** is raised if the node handle NODE was opened using the pathname ":" or if the node handle NODE was opened without traversing a relationship [3].

### 5.2.21 Obtaining the relation of the last relationship traversed

```
procedure PATH_RELATION
    (RELATION : in out RD_TYPE;
     NODE     : in     NODE_TYPE;
     STATUS   : in     STATUS_INDICATOR := NO_INDICATION);

function PATH_RELATION
    (NODE     : in     NODE_TYPE;
     STATUS   : in     STATUS_INDICATOR := NO_INDICATION)
    return RELATION_NAME;
```

Purpose:

This interface returns an identification of the relation for the last relationship traversed in opening the node handle NODE. The relation identification is returned even if the relationship has been deleted. In the first form, an open node handle to the RD used to identify the last relationship traversed is returned in RELATION with intent NO_ACCESS. In the second form, the relation



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

name used to identify the last relationship traversed is returned as the function result. The relation name is formed as described in Section 5.7.3.6 on page 355. The context for relation name formation is the NKD for the base node from which the traversed relationship emanated.

No path is traversed in opening the node handle RELATION; hence, the subprogram PATH_KEY (Section 5.2.20 on page 119) and this subprogram cannot be applied to the node handle RELATION.

Parameters:

**RELATION** is a node handle, initially closed, to be opened to the RD used to identify the last relationship traversed when opening the node handle NODE.

**NODE** is an open node handle.

**STATUS** is an established status indicator.

Exceptions:

**NOT_IN_VIEW_ERROR** is raised if the RD for the traversed relationship has been deleted or if the interface which returns a relation name is called and either a name cannot be formed for the RD in the context of the NKD for the base node or that NKD has been deleted [3].

**AMBIGUOUS_VIEW_ERROR** is raised if the interface which returns a relation name is called and formation of a name for the relation is ambiguous [2] or translation of the relation name formed is ambiguous [1].

**STATUS_ERROR** is raised if the node handle NODE is not open [2], if RELATION is already open [1] or if STATUS is not an established status indicator [3].

**USE_ERROR** is raised if the node handle NODE was opened using the pathname ":" or if the node handle NODE was opened without traversing a relationship [3].

### 5.2.22 Obtaining a partial pathname

```
function BASE_PATH
    (NAME   : in  PATHNAME;
     STATUS : in  STATUS_INDICATOR := NO_INDICATION)
    return PATHNAME;
```

Purpose:

This function returns the pathname obtained by deleting the last path element from NAME. It does not establish whether the pathname identifies an existing node; only the syntactic properties of the pathname are examined. This function also checks the syntactic legality of the pathname NAME. If the submitted name is an abbreviated name according to the rules of Section 4.3.7 on page 50, the function returns the corresponding portion of the fully expanded pathname. If the fully expanded pathname has only a single path element, ":" is returned.


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Parameters:

**NAME** is a pathname (not necessarily identifying a node).

**STATUS** is an established status indicator.

Exceptions:

**PATHNAME_SYNTAX_ERROR** is raised if NAME is a syntactically illegal pathname (see Table I, page 52) [1].

**STATUS_ERROR** is raised if STATUS is not an established status indicator [3].

**USE_ERROR** is raised if NAME has the value ":" [4].

### 5.2.23 Obtaining the name of the last relation in a pathname

```
function LAST_RELATION
    (NAME   : in    PATHNAME;
     STATUS : in    STATUS_INDICATOR := NO_INDICATION)
    return RELATION_NAME;
```

Purpose:

This function returns the name of the relation of the last path element of the pathname NAME. It does not establish whether the pathname identifies an existing node; only the syntactic properties of the pathname are examined. This function also checks the syntactic legality of the pathname NAME.

Parameters:

**NAME** is a pathname (not necessarily identifying a node).

**STATUS** is an established status indicator.

Exceptions:

**PATHNAME_SYNTAX_ERROR** is raised if NAME is a syntactically illegal pathname (see Table I, page 52) [1].

**STATUS_ERROR** is raised if STATUS is not an established status indicator [3].

**USE_ERROR** is raised if NAME has the value ":" [4].

### 5.2.24 Obtaining the key list of the last relationship in a pathname

```
function LAST_KEY
    (NAME   : in    PATHNAME;
     STATUS : in    STATUS_INDICATOR := NO_INDICATION)
    return RELATIONSHIP_KEY;
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Purpose:

This function returns the relationship key list of the last path element of the pathname NAME. The empty string is returned if the last path element of the pathname NAME has no relationship key list. This function does not establish whether the pathname identifies an existing node; only the syntactic properties of the pathname are examined. This function checks the syntactic legality of the pathname NAME.

Parameters:

**NAME** is a pathname (not necessarily identifying a node).

**STATUS** is an established status indicator.

Exceptions:

**PATHNAME_SYNTAX_ERROR** is raised if NAME is a syntactically illegal pathname (see Table I, page 52) [1].

**STATUS_ERROR** is raised if STATUS is not an established status indicator [3].

**USE_ERROR** is raised if NAME has the value ":" [4].

### 5.2.25 Querying the existence of a node

```
function IS_OBTAINABLE
    (NODE   : in  NODE_TYPE;
     STATUS : in  STATUS_INDICATOR := NO_INDICATION)
    return BOOLEAN;
```

Purpose:

This function returns FALSE if the node identified by NODE is unobtainable or inaccessible. It returns TRUE otherwise.

Parameters:

**NODE** is an open node handle identifying the node.

**STATUS** is an established status indicator.

Exceptions:

**STATUS_ERROR** is raised if NODE is not an open node handle [2] or if STATUS is not an established status indicator [3].



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Notes:

IS_OBTAINABLE can be used to determine whether a node identified via a secondary relationship has been made unobtainable or is inaccessible to the current process.

### 5.2.20 Querying sameness

```
function IS_SAME
    (NODE1  : in     NODE_TYPE;
     NODE2  : in     NODE_TYPE;
     STATUS : in     STATUS_INDICATOR  := NO_INDICATION)
    return BOOLEAN;
```

Purpose:

This function returns TRUE if the nodes identified by its arguments are the same node; otherwise it returns FALSE. If both nodes are unobtainable or inaccessible, IS_SAME returns its result as if the nodes were not unobtainable or inaccessible. If only one of the nodes is unobtainable or inaccessible, IS_SAME always returns FALSE.

Parameters:

**NODE1** is an open node handle to a node.

**NODE2** is an open node handle to a node.

**STATUS** is an established status indicator.

Exceptions:

**STATUS_ERROR** is raised if one or both of the node handles NODE1 and NODE2 is not open [2] or if STATUS is not an established status indicator [3].

Notes:

Sameness is not to be confused with equality of attribute values, relationships and contents of nodes, which is a necessary but not a sufficient criterion for sameness. Within a given CAIS system, existing nodes having the same UID are the same node.

Open node handles to unobtainable or inaccessible nodes can exist, if the intent under which these node handles were opened is NO_ACCESS only. No security violation arises in the IS_SAME interface, since the past existence and "sameness" of the nodes cannot be denied due to the visible existence of the relationships used in opening the node handles.

### 5.2.27 Obtaining an index for a node handle

```
function INDEX
    (NODE   : in     NODE_TYPE;
     MODULO : in     CAIS_POSITIVE;
     STATUS : in     STATUS_INDICATOR  := NO_INDICATION)
    return CAIS_NATURAL;
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Purpose:

This function returns an implementation-defined number in the range of 0 to MODULO-l. This number is guaranteed to be the same if the interface is called with the same MODULO value for two open node handles, N1 and N2, for which IS_SAME(N1,N2) is TRUE. Otherwise, the function result may but need not differ. 1

Parameters:

**NODE** is an open node handle. 2

**MODULO** is a number used to restrict the result to be in the range 0..MODULO-1. 3

**STATUS** is an established status indicator. 4

Exceptions:

**STATUS_ERROR** is raised if the node handle NODE is not an open node handle [2] or if STATUS is not an established status indicator [3]. 5

Notes:

This interface is intended to allow users to create hash tables for open node handles and fast checking for sameness. The fact that the type NODE_TYPE is limited private implies that the components of such hash tables need to be implemented in terms of access types, since the hash value can only be obtained after the node handle is opened. 6

An implementation of the interface should return numbers that are randomly distributed over the range 0..MODULO-1 for open node handles associated with different nodes. 7

### 5.2.28 Opening a node handle to the parent

```
procedure OPEN_PARENT
    (PARENT     : in out NODE_TYPE;
     NODE       :  in    NIODE_TYPE;
     INTENT     :  in    INTENT_ARRAY     := (1 => READ);
     TIME_LIMIT :  in    CAIS_DURATION    := LONG_DELAY;
     STATUS     :  in    STATUS_INDICATOR := NO_INDICATION);
```

Purpose:

This procedure returns an open node handle in PARENT which identifies the node which is the parent of the node identified by the open node handle NODE. The intent under which the node handle PARENT is opened is specified by INTENT. The parent node is the node which is the target of the reverse relationship of the primary relationship to the node identified by NODE. 1



Source: https://assist.dla.mil -- Downloaded: 2026l0-26T07:54Z
Check the source to verify that this is the currentversion before use.

Parameters:

**PARENT** is a node handle, initially closed, to be opened to the parent.

**NODE** is an open node handle identifying the node.

**INTENT** is the intent of subsequent operations on the node handle PARENT.

**TIME_LIMIT** is the maximum time to wait for the unlocking of the parent node in accordance with the desired INTENT.

**STATUS** is an established status indicator.

Exceptions:

**NAME_ERROR** is raised if the parent of the node identified by NODE is inaccessible and the given INTENT includes any intent other than NO_ACCESS [1].

**USE_ERROR** is raised if the specified INTENT is an empty array [1].

**STATUS_ERROR** is raised if the node handle PARENT is open at the time of the call [1], if the node handle NODE is not open [2] or if STATUS is not an established status indicator [3].

**TRANSACTION_ERROR** is raised if the operation is included in more than one transaction [1].

**LOCK_ERROR** is, raised if the opening of the parent is delayed beyond the specified TIME_-LIMIT due to the existence of locks in conflict with the specified INTENT [1]. LOCK_-ERROR may be raised prior to expiration of the time limit if the CAIS implementation can determine that a deadlock situation has occurred.

**INTENT_VIOLATION** is raised if NODE was not opened with an intent establishing the right to read relationships [1].

**ACCESS_VIOLATION** is raised if the discretionary access control rights of the current process are insufficient to obtain access to the parent with the specified INTENT [1].

**SECURITY_VIOLATION** is raised if the attempt to gain access to the parent with the specified INTENT represents a violation of mandatory access controls.

### 5.2.29 Copying a node

```
procedure COPY_NODE
    (FROM                  : in  NODE_TYPE;
     TO_BASE               : in  NODE_TYPE;
     TO_KEY                : in  RELATIONSHIP_KEY          := EMPTY_KEY_LIST;
     TO_RELATION           : in  RD_TYPE;
     DISCRETIONARY_ACCESS  : in  DISCRETIONARY_ACCESS_LIST := ALL_RIGHTS;
     MANDATORY_ACCESS      : in  MANDATORY_ACCESS_LIST     := EMPTY_LIST;
     STATUS                : in  STATUS_INDICATOR          := NO_INDICATION;
     MEDIUM_RESOURCE       : in  MEDIUM_NODE_TYPE          : = DEFAULT_MEDIUM) ;
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Purpose:

This procedure copies a SECONDARY_STORAGE or STRUCTURAL node which is not CAIS-controlled. The node identified by the node handle FROM is copied to a newly created node. The new node is identified by TO_BASE, TO_KEY and TO_RELATION. The newly created node is an instance of the same NKD as the node identified by FROM. If the node is a SECONDARY_-STORAGE node, its contents are also copied, i.e., a new copied file is created. A primary relationship of the relation TO_RELATION with the key values defined by the relationship key list TO_KEY is created from the TO_BASE node to the newly created node. The corresponding reverse relationship of the primary relationship is created from the newly created node to the node identified by TO_BASE. Regardless of any locks on the node identified by FROM, the newly created node is unlocked. 1

Emanating primary relationships are not copied. However, any inheritable secondary relationships emanating from the original node are recreated in the copy. If the target node of a relationship emanating from the original node is the node itself, then the copy has an analogous relationship to itself. Other secondary relationships whose target nodes are the original node are not copied. A secondary COPY_OF relationship is created from the newly created node to the node identified by FROM. 2

All attributes of the FROM node are also copied except TIME_CREATED, which is set to tile current time, and TIME_RELATIONSHIP_WRITTEN which is also set to the current time if any relationship emanating from the original node is not copied, excluding the parent relationship and the COPY_OF relationship, if any. Otherwise, TIME_RELATIONSHIP_WRITTEN is copied, 3

The access control information represented by inheritable ACCESS and DENY relationships applies to the newly created node. However, the DISCRETIONARY_ACCESS parameter may specify alternative access control information for the default role of the current user (see Section 5.4 on page 255). If the DISCRETIONARY_ACCESS parameter is an empty list, then the inheritable ACCESS relationship for the default role (if any) is copied. If the DISCRETIONARY_ACCESS parameter is not an empty list, then an ACCESS relationship for the default role is created, rather than copied, and the GRANT attribute is set to the value of that parameter. 4

The MANDATORY_ACCESS parameter specifies the object classification labels with which the new node is to be created. If its value is the empty list, the node inherits the subject classification of the creating process as its object classification. Otherwise, it must be an unnamed list consisting of an identifier item and optionally, an unnamed list of identifier items (see Table IX, page 256). 5

MEDIUM_RESOURCE specifies the medium resource on which the newly created node will reside. 6

Parameters:

**FROM** is an open node handle to the node to be copied. 7

**TO_BASE** is an open node handle to the source node of the primary relationship to the node to be created. 8

**TO_KEY** is a relationship key list for the identification of the primary relationship to the node to be created. 9

**TO_RELATION** is an open node handle to the RD of the primary relationship to be created to the new node. 10


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

**DISCRETIONARY_ACCESS** is alternative access control information for the default role of the current user to be associated with the created node instead of inherited access control information for that role; it is the value of the GRANT attribute of the ACCESS relationship to the user's default GROUP node (see Section 5.4.4 on page 259). 11

**MANDATORY_ACCESS** is a list defining the classification label for the created node (see Table IX, page 256). 12

**STATUS** is an established status indicator. 13

**MEDIUM_RESOURCE** is an open node handle identifying a MEDIUM node. 14

Exceptions:

**PATHNAME_SYNTAX_ERROR** is raised if the new node identification given by TO_KEY is syntactically illegal (see Table I, page 52) [2]. 15

**EXISTING_NODE_ERROR** is raised if a node already exists with the identification given for the new node [1]. 16

**SYNTAX_ERROR** is raised if DISCRETIONARY_ACCESS (see Section 5.4.4 on page 259) [10] or MANDATORY_ACCESS (see Table IX, page 256) [9] is syntactically illegal. 17

**NOT_IN_VIEW_ERROR** is raised if a key name specified in TO_KEY cannot be translated [1] or if TO_KEY consists of a single relationship key designator and no key of the specified relation is visible [5]. 18

**AMBIGUOUS_VIEW_ERROR** is raised if translation of a key name specified in TO_KEY is ambiguous [1] or if TO. KEY consists of a single relationship key designator and more than one key of the specified relation is visible [4]. 19

**PREDEFINED_RELATION_ERROR** is raised if TO_RELATION identifies a relation that is CAIS-controlled [1]. 20

**RELATIONSHIP_ERROR** is raised if TO_KEY specifies a value for the same key more than once [5] or if TO-RELATION or its corresponding reverse relation has no unique key set [6]. 21

**USE_ERROR** is raised if the value of MANDATORY_ACCESS [10] is semantically illegal. 22

**STATUS_ERROR** is raised if any of the node handles FROM, TO_BASE, TO_RELATION or MEDIUM_RESOURCE is not an open node handle [2] or if STATUS is not an established status indicator [3]. 23

**NODE_KIND_ERROR** is raised if the NKD of the node to be copied is CAIS-controlled [5], if TO_RELATION does not identify an RD or if MEDIUM_RESOURCE does not identify a MEDIUM node [1]. 24

**CONFORMANCE_ERROR** is raised if the operation would result in an attribute, node or relationship not conforming to its definition or the constraints in force on that definition [1, 2, 3, 4, 10, 11]. 25

**RESOURCE_ALLOCATION_ERROR** is raised if the medium resource identified through MEDIUM_RESOURCE is not allocatable [1]. 26

**RESOURCE_AVAILABILITY_ERROR** is raised if the medium resource identified through MEDIUM_RESOURCE is not available [1]. 27

**TRANSACTION_ERROR** is raised if the operation is included in more than one transaction [1] or if the operation is included in a transaction and the operation modifies a node which is not included in the same transaction [2]. 28



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

**INTENT_VIOLATION** is raised if FROM was not opened with an intent establishing the right to read contents, attributes, and relationships or if TO_BASE was not opened with an intent establishing the right to append relationships [1]. 29

**ACCESS_VIOLATION** is raised if the discretionary access control rights of the current process are insufficient to gain access with APPEND_CONTENTS intent to the MEDIUM node identified by MEDIUM_RESOURCE or to gain access to the NKD of the node to be copied with CREATE_INSTANCES intent [1]. 30

**SECURITY_VIOLATION** is raised if the operation represents a violation of mandatory access controls. 31

Additional Interfaces:

```
procedure COPY_NODE
    (FROM                  : in  NODE_TYPE;
     TO_BASE               : in  NODE_TYPE;
     TO_KEY                : in  RELATIONSHIP_KEY          := EMPTY_KEY_LIST ;
     TO_RELATION           : in  RELATION_NAME             := DEFAULT_RELATION;
     DISCRETIONARY_ACCESS  : in  DISCRETIONARY_ACCESS_LIST := ALL_RIGHTS;
     MANDATORY_ACCESS      : in  MANDATORY_ACCESS_LIST : = EMPTY_LIST;
     STATUS                : in  STATUS_INDICATOR          := NO_INDICATION;
     MEDIUM_RESOURCE       : in  MEDIUM_NODE_TYPE          := DEFAULT_MEDIUM)
is
     TO_RELATION_DEFINITION : RD_TYPE;
begin
     DEFINITION_OF_RELATION(TO_RELATION_DEFINITION, TO_BASE,
                            TO_RELATION, STATUS);
     COPY_NODE (FROM, TO_BASE, TO_KEY, TO_RELATION_DEFINITION,
                DISCRETIONARY_ACCESS, MANDATORY_ACCESS, STATUS,
                MEDIUM_RESOURCE) ;
     CLOSE (TO_RELATION_DEFINITION);
exception
     when others =>
          CLOSE (TO_RELATION_DEFINITION);
          raise;
end COPY_MODE;
```

Notes:

Since bidirectional relationships are not inheritable, they are not copied. 32

### 5.2.30 Copying trees

```
procedure COPY_TREE
    (FROM                  : in  NODE_TYPE;
     TO_BASE               : in  NODE_TYPE;
     TO_KEY                : in  RELATIONSHIP_KEY          := EMPTY_KEY_LIST;
     TO_RELATION           : in  RD_TYPE;
```


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
    DISCRETIONARY_ACCESS : in     DISCRETIONARY_ACCESS_LIST := ALL_RIGHTS;
    MANDATORY_ACCESS     : in     MANDATORY_ACCESS_LIST := EMPTY_LIST;
    STATUS               : in     STATUS_INDICATOR := NO_INDICATION;
    MEDIUM_RESOURCE      : in     MEDIUM_NODE_TYPE := DEFAULT_MEDIUM);
```

Purpose:

This procedure copies the tree of nodes which can be reached by traversing primary relationships beginning at the node identified by the open node handle FROM. The node identified by FROM is the root of the tree to be copied. The tree must contain only SECONDARY_STORAGE nodes and STRUCTURAL nodes which are not CAIS-controlled. Primary relationships are recreated between corresponding copied nodes. The root node of the newly created tree corresponding to the FROM node is identified by TO_BASE, TO_KEY and TO_RELATION. If an exception is raised by the procedure, none of the nodes is copied. If the node identified by TO_BASE is part of the tree to recopied, then the copy of the node identified by FROM will not be copied recursively.

Secondary relationships, attributes, and node contents are copied as described for COPY_NODE with the following additional rules:

a. Secondary relationships between two nodes which both are copied are recreated between the two copies (even when they are not inheritable).

b. Any inheritable secondary relationships emanating from a node which is copied, but which refer to nodes outside the tree being copied are copied so that they emanate from the copy, but still refer to the original target node.

c. Secondary relationships emanating from a node which is not copied, but which refer to nodes inside the tree being copied, are not copied.

A secondary COPY_OF relationship is created from each newly created node to the corresponding original node.

The access control information represented by inheritable ACCESS and DENY relationships applies to the newly created nodes. However, the DISCRETIONARY_ACCESS parameter may specify alternative access control information for the default role of the current user (see Section 5.4 on page 255). If the DISCRETIONARY_ACCESS parameter is an empty list, then the inheritable ACCESS relationships for the default role (if any) are copied. If the DISCRETIONARY_ACCESS parameter is not an empty list, then ACCESS relationships for the default role are created, rather than copied, and the GRANT attributes are set to the value of that parameter.

The MANDATORY_ACCESS parameter specifies the object classification labels with which the new nodes are to be created. If its value is the empty list, the nodes inherit the subject classification of the creating process as their object classification. Otherwise, it must be an unnamed list consisting of an identifier item and optionally, an unnamed list of identifier items (see Table IX, page 256).

MEDIUM_RESOURCE specifies the medium resource on which the newly created nodes will reside.

Figure 16, page 131, shows an example of copying a tree.



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

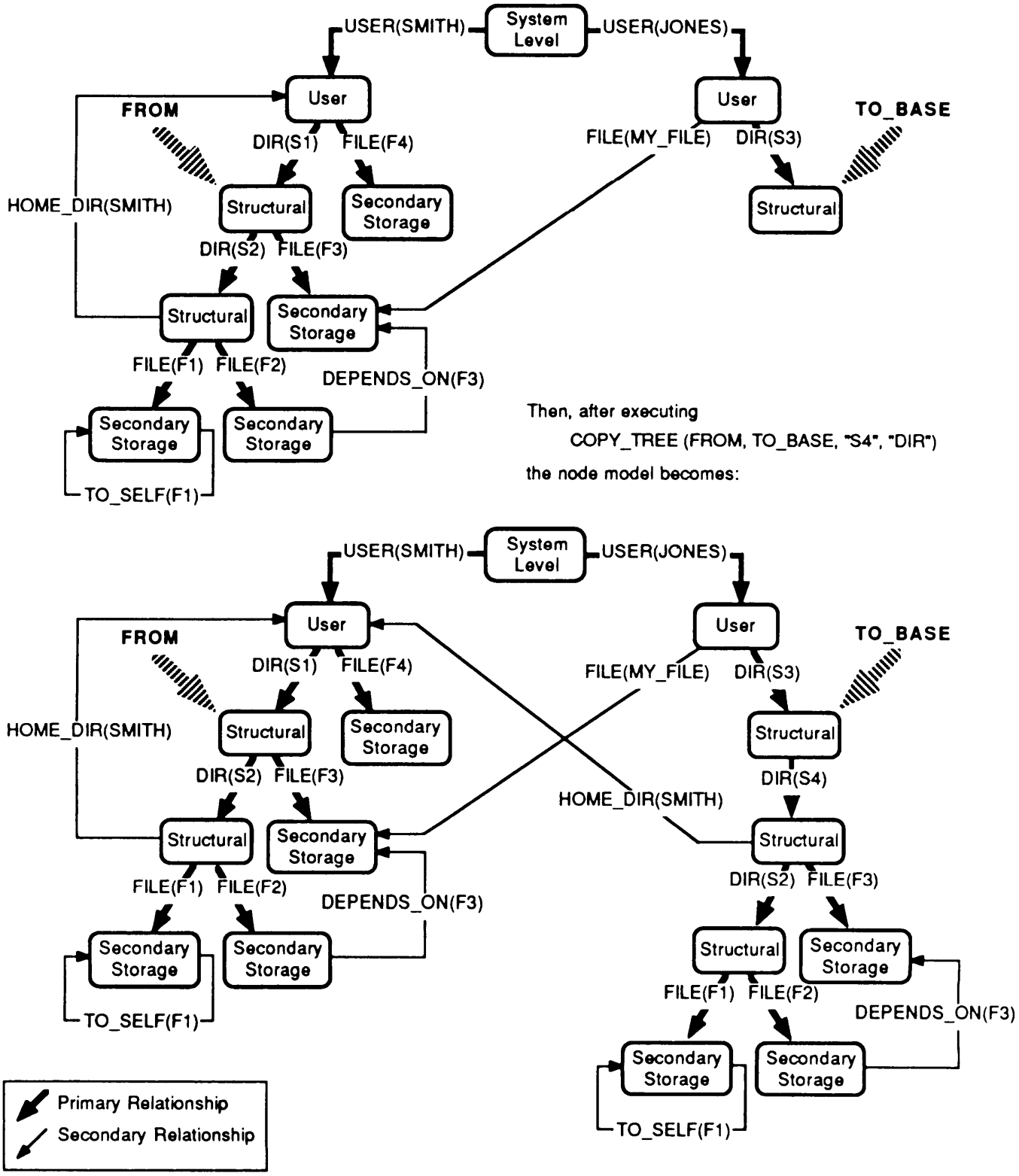


FIGURE 16. COPY_TREE example



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Parameters:

**FROM** is an open node handle to the root node of the tree to be copied.

**TO_BASE,** is an open node handle to the base node for the primary relationship to the node to be created as root of the new tree.

**TO_KEY** is a relationship key list for the identification of the primary relationship to the node to be created as the root of the new tree.

**TO_RELATION** is an open node handle to the RD of the primary relationship to be created to the node to be created as the root of the new tree.

**DISCRETIONARY_ACCESS** is alternative access control information for the default role of the current user to be associated with the created nodes instead of inherited access control information for that role; it is the value of the GRANT attributes of the ACCESS relationships to the user's default GROUP node (see Section 5.4.4 on page 259).

**MANDATORY_ACCESS** is a list defining the classification label for the created nodes (see Table IX, page 256).

**STATUS** is an established status indicator.

**MEDIUM_RESOURCE** is an open node handle identifying a MEDIUM node.

Exceptions:

**PATHNAME_SYNTAX_ERROR** is raised if the new node identification given by TO_KEY is syntactically illegal (see Table I, page 52) [2].

**EXISTING_NODE_ERROR** is raised if a node already exists with the identification given for the root node of the copied tree (given by TO_BASE, TO_KEY and TO_RELATION) [1].

**SYNTAX_ERROR** is raised if DISCRETIONARY_ACCESS (see Section 5.4.4 on page 259) [10] or MANDATORY_ACCESS (see Table IX, page 256) [9] is syntactically illegal.

**NOT_IN_VIEW_ERROR** is raised if a key name specified in TO_KEY cannot be translated [1] or if TO_KEY consists of a single relationship key designator and no key of the specified relation is visible [5].

**AMBIGUOUS_VIEW_ERROR** is raised if translation of a key name specified in TO_KEY is ambiguous [1] or if TO_KEY consists of a single relationship key designator and more than one key of the specified relation is visible [4].

**RELATIONSHIP_ERROR** is raised if TO_KEY specifies a value for the same key more than once [5] or if TO_RELATION or its corresponding reverse relation has no unique key set [6].

**PREDEFINED_RELATION_ERROR** is raised if TO_RELATION identifies a relation that is CAIS-controlled, or if the relation of any relationship to be copied is CAIS-controlled [1].

**USE_ERROR** is raised if the value of MANDATORY_ACCESS [10] is semantically illegal.

**STATUS_ERROR** is raised if any of the node handles FROM, TO_BASE, TO_RELATION or MEDIUM_RESOURCE is not open [2] or if STATUS is not an established status indicator [3].

**NODE_KIND_ERROR** is raised if the NKD of any node to be copied is CAIS-controlled [5], if TO_RELATION does not identify an RD or if MEDIUM_RESOURCE does not identify a MEDIUM node [1].


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

**CONFORMANCE_ERROR** is raised if the operation would result in an attribute, node or relationship not conforming to its definition or the constraints in force on that definition [1, 2, 3, 4, 10, 11]. 29

**TRANSACTION_ERROR** is raised if the operation is included in more than one transaction [1] or if the operation is included in a transaction and the operation modifies a node which is not included in the same transaction [2]. 30

**LOCK_ERROR** is raised if any node to be copied except the node identified by FROM is locked against read access to attributes, relationships or contents [1]. 31

**RESOURCE_ALLOCATION_ERROR** is raised if the medium resource identified through MEDIUM_RESOURCE is not allocatable [1]. 32

**RESOURCE_AVAILABILITY_ERROR** is raised if the medium resource identified through MEDIUM_RESOURCE is not available [1]. 33

**INTENT_VIOLATION** is raised if FROM was not opened with an intent establishing the right to read node contents, attributes and relationships or if TO_BASE was not opened with an intent establishing the right to append relationships [1]. 34

**ACCESS_VIOLATION** is raised if the discretionary access control rights of the current process are insufficient to obtain access to each node to be copied with READ intent, to obtain access to the NKD of each node to be copied with CREATE_INSTANCES intent or to obtain access with APPEND_CONTENTS intent to the MEDIUM node identified by MEDIUM_RESOURCE [1]. 35

**SECURITY_VIOLATION** is raised if the operation represents a violation of mandatory access controls. 36

Additional Interfaces:

```
procedure COPY_TREE
    (FROM                    : in    NODE_TYPE;
     TO_BASE                 : in    NODE_TYPE;
     TO_KEY                  : in    RELATIONSHIP_KEY          : = EMPTY_KEY_LIST;
     TO_RELATIOIV            : in    RELATION_NAME             : = DEFAULT_RELATION;
     DISCRETIONARY_ACCESS    : in    DISCRETIONARY_ACCESS_ LIST : = ALL_RIGHTS;
     MANDATORY_ACCESS        : in    MANDATORY_ACCESS_LIST     : = EMPTY_LIST;
     STATUS                  : in    STATUS_INDICATOR          : = NO_INDICATION;
     MEDIUM_RESOURCE         : in    MEDIUM_NODE_TYPE          : = DEFAULT_MEDIUM)
is
    TO_RELATION_DEFINITION : RD_TYPE;
begin
    DEFINITION_OF_RELATION (TO_RELATION_DEFININITION, TO_BASE,
                            TO_RELATIO, STATUS);
    COPY_TREE (FROM, TO_BASE, TO_KEY, TO_RELATION_DEFINITION,
               DISCRETIONARY_ACCESS, MANDATORY_ACCESS,
               STATUS, MEDIUM_RESOURCE);
    CLOSE (TO_RELATION_DEFINITION);

exception
    when others =>
```


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
            CLOSE (TO_RELATION_DEFINITION);
            raise;
    end COPY_TREE;
```

Notes:

Since bidirectional relationships are not inheritable, bidirectional relationships which emanate from copied nodes but do not terminate at copied nodes are not copied. 37

### 5.2.31 Renaming a relationship

```
procedure RENAME
    (BASE                      : in  NODE_TYPE;
     KEY                       : in  RELATIONSHIP_KEY := EMPTY_KEY_LIST;
     RELATION                  : in  RD_TYPE;

     NEW_BASE                  : in  NODE_TYPE        := NULL_NODE;
     NEW_KEY                   : in  RELATIONSHIP_KEY := EMPTY_KEY_LIST;
     NEW_RELATION              : in  RD_TYPE          := NULL_NODE;
     NEW_ATTRIBUTES            : in  ATTRIBUTE_LIST   := EMPTY_LIST;

     NEW_TARGET                : in  NODE_TYPE        := NULL_NODE;
     NEW_REVERSE_KEY           : in  RELATIONSHIP_KEY := EMPTY_KEY_LIST;
     NEW_REVERSE_ATTRIBUTES : in  ATTRIBUTE_LIST   := EMPTY_LIST;
     STATUS                    : in  STATUS_INDICATOR := NO_INDICATION);
```

Purpose:

This procedure deletes an existing relationship and creates a new relationship as an atomic operation. The type of new relationship (primary or secondary, unidirectional or bidirectional) is the same as the type of the existing relationship. Attributes of the existing relationship may be preserved. 1

The existing relationship is identified by the parameters BASE, KEY and RELATION. 2

If NEW_BASE has the value NULL_NODE (the default), the new relationship emanates from the same node as the existing relationship, the node identified by BASE. Otherwise, NEW_BASE is an open node handle identifying the base node for the new relationship. 3

If NEW_RELATION has the value NULL_NODE (the default), the relation for the new relationship is the same as the relation of the existing relationship. Otherwise, NEW_RELATION is an open node handle identifying the RD for the relation of the new relationship. 4

NEW_KEY has the form of a relationship key list and specifies values for keys of the new relationship. NEW_ATTRIBUTES provides values for attributes of the new relationship which are not keys. Attribute values of the existing relationship are copied for those attributes of the new relation that are not specified in the parameter NEW_KEY or NEW_ATTRIBUTES. Other attributes are assigned their default value (if any) in the context of the relation of the new relationship. 5

If NEW_TARGET has the value NULL_NODE (the default), the target node for the new rela- 6



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

tionship is the same as the target node of the existing relationship. Otherwise, NEW_TARGET is an open node handle identifying the target node of the new relationship.

The parameters NEW_REVERSE_KEY and NEW_REVERSE_ATTRIBUTES have the same functions as the parameters NEW_KEY and NEW_ATTRIBUTES, except that they apply to the reverse relationship of the new relationship, if any.

Attribute names in KEY are translated in the context of the relation identified by RELATION. Attribute names in NEW_KEY and NEW_ATTRIBUTES are translated in the context of the relation identified by NEW_RELATION or in the context of the relation identified by RELATION if NEW_RELATION has the value NULL_NODE. Attribute names in NEW_REVERSE_KEY and in NEW_REVERSE_ATTRIBUTES are translated in the context of the corresponding reverse relation.

Parameters:

**BASE** is an open node handle to the base node of the existing relationship.

**KEY** is a relationship key list identifying the existing relationship in the context of BASE and RELATION.

**RELATION** is an open node handle identifying a relation to which the existing relationship belongs.

**NEW_BASE** is an open node handle to the base node from which the new relationship is to emanate or the value NULL_NODE if the base node is to remain the same.

**NEW_KEY** is a relationship key list for the new relationship.

**NEW_RELATION** is an open node handle to the RD for the new relationship or the value NULL_NODE if the relation is to remain the same.

**NEW_ATTRIBUTES** is an empty list or a named list whose elements establish initial values for attributes of the new relationship; each named item specifies an attribute name and the value to be given to that attribute.

**NEW_TARGET** is an open node handle to the target node for the new relationship or the value NULL_NODE if the target node is not to be changed.

**NEW_REVERSE_KEY** is a relationship key list for the new reverse relationship.

**NEW_REVERSE_ATTRIBUTES** is an empty list or a named list whose elements establish initial values for attributes of the new reverse relationship; each named item specifies an attribute name and the value to be given to that attribute.

**STATUS** is an established status indicator.

Exceptions:

**PATHNAME_SYNTAX_ERROR** is raised if the relationship identification given by KEY, NEW_KEY or NEW_REVERSE_KEY is syntactically illegal (see Table I, page 52) [2].

**SYNTAX_ERROR** is raised if the NEW_ATTRIBUTES or NEW_REVERSE_ATTRIBUTES parameter is syntactically illegal [8].

**NOT_IN_VIEW_ERROR** is raised if an attribute name specified in KEY, NEW_KEY, NEW_ATTRIBUTES, NEW_REVERSE_KEY or NEW_REVERSE_ATTRIBUTES cannot be translated [I] or if KEY, NEW_KEY, or NEW_REVERSE_KEY consists of a single relationship key designator and no key of the corresponding relation is visible [5].


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

**AMBIGUOUS_VIEW_ERROR** is raised if translation of an attribute name specified in KEY, NEW_KEY, NEW_ATTRIBUTES, NEW_REVERSE_KEY or NEW_REVERSE_-ATTRIBUTES is ambiguous [1] or if KEY, NEW_KEY, or NEW_REVERSE_KEY consists of a single relationship key designator and more than one key of the corresponding relation is visible [4]. 23

**USE_ERROR** is raised if the change would cause more than than one primary relationship to terminate at the same node or would create a circular path consisting of primary relationships [5], if NEW_ATTRIBUTES or NEW_REVERSE_ATTRIBUTES names a key attribute of the corresponding relation [51] or if either the relation of the existing relationship or the relation designated by NEW_RELATION is a bidirectional relation and the other is not [53]. 24

**PREDEFINED_ATTRIBUTE_ERROR** is raised if NEW_KEY, NEW_ATTRIBUTES, NEW_REVERSE_KEY or NEW_REVERSE_ATTRIBUTES names a CAIS-controlled attribute [z]. 25

**ATTRIBUTE_ERROR** is raised if NEW_ATTRIBUTES or NEW_REVERSE_ATTRIBUTES specifies a value for the same attribute more than once [6]. 26

**PREDEFINED_RELATION_ERROR** is raised if changing the relation of the relationship to NEW_RELATION would change the set of predefine, CAIS-controlled relations to which the relationship belongs [3], or if the existing relationship is CAIS-controlled and the base node or target node is to be changed by the operation [1]. 27

**RELATIONSHIP_ERROR** is raised if KEY, NEW_KEY or NEW_REVERSE_KEY specifies a value for the same key more than once [5]; if BASE, KEY and RELATION identify more than one relationship [1] or no relationship [2]; if the new relationship or the new reverse relationship would have the same identification as an existing relationship [3]; or if NEW_-RELATION or its corresponding reverse relation has no unique key set [6]. 28

**STATUS_ERROR** is raised if the node handle BASE or RELATION is not open [2], if any of the node handles NEW_ BASE, NEW_RELATION or NEW_TARGET is not open and does not have the value NULL_NODE [2], or if STATUS is not an established status indicator [3]. 29

**NODE_KIND_ERROR** is raised if the relationship or its reverse relationship is a primary relationship to a PROCESS node [1], if the open node handle RELATION does not designate an RD [1] or if NEW_RELATION is an open node handle which does not designate an RD [1]. 30

**CONFORMANCE_ERROR** is raised if the operation would result in an attribute, node or relationship not conforming to its definition or the constraints in force on that definition [1, 2, 3, 4, 5, 7, 8, 9, 10, 11]. 31

**TRANSACTION_ERROR** is raised if the operation is included in more than one transaction [1] or if the operation is included in a transaction and the operation modifies a node which is not included in the same transaction [2]. 32

**LOCK_ERROR** is raised if access with intent WRITE_RELATIONSHIPS to a node to be modified cannot be obtained due to an existing lock [1]. 33

**INTENT_VIOLATION** is raised if BASE was not opened with an intent establishing the right to write relationships, if NEW_BASE is an open node handle which is not open with an intent establishing the right to append relationships [1], or if the existing relationship is bidirectional and TARGET_NODE is an open node handle which is not open with an intent establishing the right to append relationships. 34


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

**ACCESS_VIOLATION** is raised if the existing relationship is bidirectional and the current process does not have sufficient discretionary access control rights to obtain access to the target node of the existing relationship with intent WRITE_RELATIONSHIPS [1]. 35

**SECURITY_VIOLATION** is raised if the operation represents a violation of mandatory access controls. 36

Additional Interfaces:

```
procedure RENAME
    (NODE          : in  NODE_TYPE;
     NEW_BASE      : in  NODE_TYPE          := NULL_NODE;
     NEW_KEY       : in  RELATIONSHIP_KEY   := EMPTY_KEY_LIST;
     NEW_RELATION : in  RD_TYPE            := NULL_NODE;
     STATUS        : in  STATUS_INDICATOR   := NO_INDICATION)
is
     PARENT     : NODE_TYPE;
     RELATION : RD_TYPE;
begin

     OPEN_PARENT (PARENT, NODE, (1 => WRITE_RELATIONSHIPS),
                  0.0, STATUS);
     PRIMARY_RELATION (RELATION, NODE, STATUS);

     RENAME (BASE         => PARENT,
             KEY          => PRIMARY_KEY (NODE, STATUS),
             RELATION     => RELATION,
             NEW_BASE     => NEW_BASE,
             NEW_KEY      => NEW_KEY,
             NEW_RELATION => NEW_RELATION,
             STATUS       => STATUS);

     CLOSE (PARENT);
     CLOSE (RELATION);
exception
     when others =>
          CLOSE (PARENT);
          CLOSE (RELATION);
          raise;
end RENAME;

procedure RENAME
    (NODE          : in  NDE_TYPE;
     NEW_BASE      : in  MODE_TYPE;
     NEW_KEY       : in  RELATIONSHIP_KEY   := EMPTY_KEY_LIST;
     NEW_RELATION : in  RELATION_NAME;
     STATUS        : in  STATUS_INDICATOR   := NO_INDICATION)
is
     NEW_RELATION_DEFINITION : RD_TYPE;
```


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
begin
    DEFINITION_OF_RELATION(NEW_RELATION_DEFINITION, NEW_BASE,
        NEW_RELATION, STATUS);
    RENAME (NODE, NEW_BASE, NEW_KEY, NEW_RELATION_DEFINITION, STATUS);
    CLOSE (NEW_RELATION_DEFINITION);
exception
    when others =>
        CLOSE (NEW_RELATION_DEFINITION);
        raise;
end RENAME;
```

Notes:

Open node handles held by existing processes are not affected. 37

RENAME may be used to change the keys, source node, target node, attributes and the relation of a relationship as an atomic operation. 38

The additional interfaces rename the primary relationship to the node identified by NODE. 39

### 5.2.32 Deleting the primary relationship to anode

```
procedure DELETE_NODE
    (NODE         : in out NODE_TYPE;
     TIME_LIMIT   : in     CAIS_DURATION     := LONG_DELAY;
     STATUS       : in     STATUS_INDICATOR  := NO_INDICATION);
```

Purpose:

This procedure deletes the primary relationship to the node identified by NODE. The node must, be either a PROCESS node, or a SECONDARY_STORAGE or STRUCTURAL node which is not CAIS-controlled. The node becomes unobtainable. The node handle NODE is closed. If the node is a PROCESS node and the process is not yet terminated (see Section 6 on page 509), DELETE_NODE aborts the process. TIME_LIMIT specifies a time limit for the delay due to existing locks on the parent of the node. A delayed call completes after the node is unlocked or the specified time limit has elapsed. In the latter case, the exception LOCK_ERROR is raised. 1

Parameters:

**NODE** is an open node handle to the node which is the target node of the primary relationship to be deleted. 2

**TIME_LIMIT** is the maximum time to wait for the unlocking of the parent of NODE in accordance with the intent WRITE_RELATIONSHIPS. 3

**STATUS** is an established status indicator. 4


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Exceptions:

**NAME_ERROR** is raised if the parent node of the node identified by NODE is inaccessible [1]. 5

**USE_ERROR** is raised if any primary relationships emanate from the node to be deleted [7]. 6

**PREDEFINED_RELATION_ERROR** is raised if the relationship to be deleted belongs to a relation that is CAIS-controlled [1]. 7

**STATUS_ERROR** is raised if the node handle NODE is not open [2] or if STATUS is not an established status indicator [3]. 8

**NODE_KIND_ERROR** is raised if the node identified by NODE is CAIS-controlled and is not a PROCESS node [5]. 9

**CONFORMANCE_ERROR** is raised if the operation would result in a node not conforming to its definition or the constraints in force on that definition [3]. 10

**TRANSACTION_ERROR** is raised if the operation is included in more than one transaction [1] or if the operation is included in a transaction and the operation modifies a node which is not included in the same transaction [2]. 11

**LOCK_ERROR** is raised if access with intent WRITE_RELATIONSHIPS cannot be obtained to the parent of the node to be deleted within the specified TIME_LIMIT due to an existing lock on the node [1]. LOCK_ERROR may be raised prior to expiration of the time limit if the CAIS implementation can determine that a deadlock situation has occurred. 12

**INTENT_VIOLATION** is raised if the node handle NODE was not opened with an intent including EXCLUSIVE_WRITE and READ_RELATIONSHIPS [1]. 13

**ACCESS_VIOLATION** is raised if the current process does not have sufficient discretionary access control rights to obtain access to the parent of the node to be deleted with intent WRITE_RELATIONSHIPS [1]. 14

**SECURITY_VIOLATION** is raised if the operation represents a violation of mandatory access controls. 15

Notes:

The DELETE_NODE interface cannot be used to delete more than one primary relationship in a single operation. It is left to an implementation decision whether and when nodes whose primary relationships have been deleted are actually removed. However, secondary relationships to such nodes must remain until they are explicitly deleted using the DELETE_SECONDARY_-RELATIONSHIP procedure. 16

### 5.2.33 Deleting the primary relationships of a tree

```
procedure DELETE_TREE
    (NODE   : in out MODE_TYPE;
     STATUS : in     STATUS_INDICATOR := NO_INDICATION);
```



Source: https://assist.dla.mil -- Downloaded: 202610-26T07:54Z
Check the source to verify that this is the current version before use.

Purpose:

This procedure effectively performs the DELETE_NODE operation for a specified node and recursively applies DELETE_NODE to all nodes whose unique primary path traverses the designated node. The nodes whose primary relationships are to be deleted are opened with intent EXCLUSIVE_WRITE, thus locking them for other operations. The order in which the deletions of primary relationships is performed is not specified. If the DELETE_TREE operation raises an exception, none of the primary relationships is deleted. 1

Parameters:

**NODE** is an open node handle to the node at the root of the tree whose primary relationships are to be deleted. 2

**STATUS** is an established status indicator. 3

Exceptions:

**NAME_ERROR** is raised if the parent of the node identified by NODE or any target nodes of primary relationships to be deleted are inaccessible [1]. 4

**PREDEFINED_RELATION_ERROR** is raised if any primary relationship to be deleted belongs to a relation that is CAIS-controlled [1]. 5

**STATUS_ERROR** is raised if the node handle NODE is not open at the time of the call [2] or if STATUS is not an established status indicator [3]. 6

**NODE_KIND_ERROR** is raised if any node to be deleted is CAIS-controlled and is not a PROCESS node [5]. 7

**CONFORMANCE_ERROR** is raised if the operation would result in a node not conforming to its definition or the constraints in force on that definition [3]. 8

**TRANSACTION_ERROR** is raised if the operation is included in more than one transaction [1] or if the operation is included in a transaction and the operation modifies a node which is not included in the same transaction [2]. 9

**LOCK_ERROR** is raised if a node handle to the parent of the node specified by NODE cannot be opened with intent WRITE_RELATIONSHIPS or if a node handle identifying any node whose unique primary path traverses the node identified by NODE cannot be opened with intent EXCLUSIVE_WRITE [1]. 10

**INTENT_VIOLATION** is raised if the node handle NODE was not opened with an intent including EXCLUSIVE_WRITE and READ_RELATIONSHIPS [1]. 11

**ACCESS_VIOLATION** is raised if the current process does not have sufficient discretionary access control rights to obtain access to the parent of the node specified by NODE with intent WRITE_RELATIONSHIPS or to obtain access to any target node of a primary relationship to be deleted with intent EXCLUSIVE_WRITE [1]. 12

**SECURITY_VIOLATION** is raised if the operation represents a violation of mandatory access controls. 13



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Notes:

This interface can be used to delete more than one primary relationship in a single operation. 14

### 5.2.34 Creating secondary relationships

```
procedure CREATE_SECONDARY_RELATIONSHIP
    (TARGET_NODE     : in  NODE_TYPE;
     SOURCE_BASE     : in  NODE_TYPE;
     NEW_KEY         : in  RELATIONSHIP_KEY  := EMPTY_KEY_LIST;
     NEW_RELATION    : in  RD_TYPE;
     INHERITABLE     : in  BOOLEAN           := FALSE;
     NEW_ATTRIBUTES  : in  ATTRIBUTE_LIST    := EMPTY_LIST;
     STATUS          : in  STATUS_INDICATOR  := NO_INDICATION);
```

Purpose:

This procedure creates a secondary relationship between two existing nodes. The procedure takes an open node handle TARGET_NODE to the target node, an open node handle SOURCE_BASE to the source node, an explicit key list NEW_KEY, a explicit relationship attribute list NEW_-ATTRIBUTES and an open node handle NEW_RELATION to the RD for the relationship to be established from SOURCE-BASE to TARGET_NODE. 1

If NEW_RELATION identifies a bidirectional relation, a corresponding reverse relationship is created from TARGET_NODE to SOURCE_BASE. 2

The default values (if any) of the attributes defined by NEW_RELATION are used for any keys or other relationship attributes which are not named in NEW_KEY or in NEW_ATTRIBUTES. Any default values defined for attributes of the reverse relation associated with NEW_RELATION are used for the keys and attributes of the reverse relationship. In the absence of both an explicit value and a default value for an attribute (including a key attribute) the new relationship will not have a value for that attribute. 3

NEW_RELATION must identify a relation which is not CAIS-controlled.

Parameters:

**TARGET_NODE** is an open node handle to the node at which the new secondary relationship terminates. 5

**SOURCE_BASE** is an open node handle to the base node from which the new secondary relationship emanates. 6

**NEW_KEY** is the relationship key list for the new secondary relationship. 7

**NEW_RELATION** is an open node handle to the RD for the new secondary relationship. 8

**INHERITABLE** specifies the value of the INHERITABLE attribute of the newly created relationship. 9

**NEW_ATTRIBUTES** is an empty list or a named list whose elements establish initial values for relationship attributes of the new (forward) relationship; each named item specifies an attribute name and the value to be given to that attribute. 10

**STATUS** is an established status indicator. 11


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Exceptions:

**PATHNAME_SYNTAX_ERROR** is raised if the node identification given by NEW_KEY is syntactically illegal (see Table I, page 52) [2]. 12

**SYNTAX_ERROR** is raised if the NEW_ATTRIBUTES parameter (see description above) is syntactically illegal [8]. 13

**NOT_IN_VIEW_ERROR** is raised if an attribute name specified in NEW_KEY or NEW_-ATTRIBUTES cannot be translated [1] or if NEW_KEY consists of a single relationship key designator and no key of the relation NEW_RELATION is visible [5]. 14

**AMBIGUOUS_VIEW_ERROR** is raised if translation of an attribute name specified in NEW_KEY or in NEW_ATTRIBUTES is ambiguous [1] or if NEW_KEY consists of a single relationship key designator and more than one key of the relation NEW_RELATION is visible [4]. 15

**PREDEFINED_ATTRIBUTE_ERROR** is raised if NEW_KEY or NEW_ATTRIBUTES names a CAIS-controlled attribute [2]. 16

**ATTRIBUTE_ERROR** is raised if NEW_ATTRIBUTES specifies a value for the same attribute more than once [6]. 17

**PREDEFINED_RELATION_ERROR** is raised if NEW_RELATION identifies a CAIS-controlled relation [1]. 18

**RELATIONSHIP_ERROR** is raised if NEW_KEY specifies a value for the same key more than once [5]; if a relationship already exists with the identification given by SOURCE_-BASE, NEW_KEY and NEW_RELATION; if NEW_RELATION identifies a bidirectional relation and a relationship already exists with the identification given by TARGET_NODE, the reverse relation of NEW_RELATION and the default values of the keys for that relation [3]; or if NEW_RELATION or its corresponding reverse relation has no unique key set [6]. 19

**USE_ERROR** is raised if NEW_ATTRIBUTES names a key attribute of the relation identified by RELATION [51]. 20

**STATUS_ERROR** is raised if the node handle TARGET_NODE, SOURCE_BASE or NEW_-RELATION is not open [2] or if STATUS is not an established status indicator [3]. 21

**NODE_KIND_ERROR** is raised if the open node handle NEW_RELATION does not designate an RD [1]. 22

**CONFORMANCE_ERROR** is raised if the operation would result in an attribute, node or relationship not conforming to its definition or the constraints in force on that definition [1, 2, 3, 4, 10, 11]. 23

**TRANSACTION_ERROR** is raised if the operation is included in more than one transaction [1] or if the operation is included in a transaction and the operation modifies a node which is not included in the same transaction [2]. 24

**INTENT_VIOLATION** is raised if SOURCE_BASE was not opened with an intent establishing the right to create relationships or if NEW_RELATION identifies a bidirectional relation and TARGET_NODE was not opened with an intent establishing the right to create relationships [1]. 25

**SECURITY_VIOLATION** is raised if the operation represents a violation of mandatory access controls. 26


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Additional Interfaces:

```
procedure CREATE_SECONDARY_RELATIONSHIP
    (TARGET_NODE      : in    NODE_TYPE;
     SOURCE_BASE      : in    NODE_TYPE;
     NEW_KEY          : in    RELATIONSHIP_KEY  := EMPTY_KEY_LIST;
     NEW_RELATION     : in    RELATION_NAME     := DEFAULT_RELATION;
     INHERITABLE      : in    BOOLEAN           :=    FALSE;
     NEW_ATTRIBUTES : in      ATTRIBUTE_LIST    := EMPTY_LIST;
     STATUS           : in    STATUS_INDICATOR  := NO_INDICATION)
is
     RELATION_DEFINITION : RD_TYPE;
begin
     DEFINITION_OF_RELATION(RELATION_DEFINITION, SOURCE_BASE,
          NEW_RELATION, STATUS);
     CREATE_SECONDARY_RELATIONSHIP (TARGET_NODE, SOURCE_BASE,
                                    NEW_KEY,
                                    RELATION_DEFINITION,
                                    INHERITABLE,
                                    NEW_ATTRIBUTES,
                                    STATUS);
     CLOSE (RELATION_DEFINITION);
exception
     when others =>
          CLOSE (RELATION_DEFINITION);
          raise;
end CREATE_SECONDARY_RELATIONSHIP;
```

Notes:

CREATE_SECONDARY_RELATIONSHIP can be used to create unidirectional secondary relationships to nodes that are unobtainable or inaccessible. 27

The ALLOWED_USAGE attribute of the RD identified by NEW_RELATION must allow the use of the relation to create secondary relationships. 28

A bidirectional relationship cannot be inheritable. 29

### 5.2.35 Creating bidirectional secondary relationships

```
procedure    CREATE_BIDIRECTIONAL_SECONDARY_RELATIONSHIPS
    (TARGET_NODE              : in    NODE_TYPE;
     SOURCE_BASE              : in    NODE_TYPE;
     NEW_KEY                  : in    RELATIONSHIP_KEY  :=  EMPTY_LIST;
     NEW_RELATION             : in    RD_TYPE;
     NEW_ATTRIBUTES           : in    ATTRIBUTE_LIST    := EMPTY_LIST;
     NEW_REVERSE_KEY          : in    RELATIONSHIP_KEY  := EMPTY_KEY_LIST;
     NEW_REVERSE_ATTRIBUTES   : in    ATTRIBUTE_LIST    := EMPTY_LIST;
     STATUS                   : in    STATUS_INDICATOR  := NO_INDICATION);
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Purpose:

This procedure creates a bidirectional secondary relationship between two existing nodes. The procedure takes a node handle TARGET_NODE to the target node, a node handle SOURCE_BASE to the source node, an explicit key list NEW_KEY, an explicit relationship attribute list NEW_ATTRIBUTES and an open node handle NEW_RELATION to the relation definition for the relationship to be established from SOURCE_BASE to TARGET_NODE. It also takes an explicit key list NEW_REVERSE_KEY and an explicit relationship attribute list NEW_REVERSE_ATTRIBUTES for the reverse relationship to be established from TARGET_NODE to SOURCE_BASE. The RD for the reverse relationship is the RD for the reverse relation associated with the RD identified by NEW_RELATION.

The default value (if any) is used for any key or relationship attribute which is not explicitly named in NEW_KEY, NEW_ATTRIBUTES, NEW_REVERSE_KEY or NEW_REVERSE_ATTRIBUTES.

Parameters:

**TARGET_NODE** is an open node handle to the node at which the new secondary relationship of the relation identified by NEW_RELATION terminates.

**SOURCE_BASE** is an open node handle to the base node from which the new secondary relationship of the relation identified by NEW_RELATION emanates.

**NEW_KEY** is the relationship key list for the new secondary relationship emanating from SOURCE_BASE.

**NEW_RELATION** is an open node handle to the RD for the new secondary relationship emanating from SOURCE_BASE.

**NEW_ATTRIBUTES** is an empty list or a named list whose elements establish initial values for attributes of the new relationship; each named item specifies an attribute name and the value to be given to that attribute.

**NEW_REVERSE_KEY** is the relationship key list for the new reverse relationship emanating from TARGET_NODE.

**NEW_REVERSE_ATTRIBUTES** is an empty list or a named list whose elements establish initial values for attributes of the new reverse relationship; each named item specifies an attribute name and the value to be given to that attribute.

**STATUS** is an established status indicator.

Exceptions:

**PATHNAME_SYNTAX_ERROR** is raised if the node identification given by NEW_KEY or NEW_REVERSE_KEY is syntactically illegal (see Table I, page 52) [2].

**SYNTAX_ERROR** is raised if the NEW_ATTRIBUTES or NEW_REVERSE_ATTRIBUTES parameter (see descriptions above) is syntactically illegal [8].

**NOT_IN_VIEW_ERROR** is raised if an attribute name specified in NEW_KEY, NEW_REVERSE_KEY, NEW_ATTRIBUTES or NEW_REVERSE_ATTRIBUTES cannot be translated [I] or if NEW_KEY or NEW_REVERSE_KEY consists of a single relationship key designator and no key of the relation NEW_RELATION or its corresponding reverse relation is visible, respectively [5].


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

**AMBIGUOUS_VIEW_ERROR** is raised if translation of a key name specified in NEW_-KEY or in NEW_REVERSE_KEY or an attribute name in NEW_ATTRIBUTES or in NEW_REVERSE_ATTRIBUTES is ambiguous [1] or if NEW_KEY or NEW_REVERSE_-KEY consists of a single relationship key designator and more than one key of the relation NEW_RELATION or its corresponding reverse relation is visible, respectively [4]. 14

**PREDEFINED_ATTRIBUTE_ERROR** is raised if NEW_KEY, NEW_ATTRIBUTES, NEW_REVERSE. KEY or NEW_REVERSE_ATTRIBUTES names a CAIS-controlled attribute [z]. 15

**ATTRIBUTE_ERROR** is raised if NEW_ATTRIBUTES or NEW_REVERSE_ATTRIBUTES specifies a value for the same attribute more than once [6]. 16

**PREDEFINED_RELATION_ERROR** is raised if NEW_RELATION identifies a relation that is CAIS_controlled [1]. 17

**RELATIONSHIP_ERROR** is raised if NEW_KEY or NEW_REVERSE_KEY specifies a value for the same key more than once [5]; if either a primary or secondary relationship already exists with the identification given by SOURCE_BASE, NEW_KEY and NEW_RELATION or the identification given by TARGET_NODE, the reverse relation of NEW_RELATION and NEW_REVERSE_KEY for that relation [3]; or if NEW_RELATION or its corresponding reverse relation has no unique key set [6]. 18

**USE_ERROR** is raised if NEW_ATTRIBUTES or NEW_REVERSE_ATTRIBUTES names a key attribute of the relation identified by RELATION or the corresponding reverse relation, respectively [51]; or if the open node handle NEW_RELATION does not designate an RD for a bidirectional relation [52]. 19

**STATUS_ERROR** is raised if the node handle TARGET_NODE, SOURCE_BASE or NEW_-RELATION is not open [2] or if STATUS is not an established status indicator [3]. 20

**CONFORMANCE_ERROR** is raised if the operation would result in an attribute, node or relationship not conforming to its definition or the constraints in force on that definition [1, 2, 3, 4, 10, 11]. 21

**TRANSACTION_ERROR** is raised if the operation is included in more than one transaction [1] or if the operation is included in a transaction and the operation modifies a node which is not included in the same transaction [2]. 22

**INTENT_VIOLATION** is raised if SOURCE_BASE was not opened with an intent establishing the right to create relationships or if TARGET_NODE was not opened with an intent establishing the right to create relationships [1]. 23

**SECURITY_VIOLATION** is raised if the operation represents a violation of mandatory access controls. 24

Additional Interfaces:

```
procedure CREATE_BIDIRECTIONAL_SECONDARY_RELATIONSHIP
    (TARGET_NODE      : in  NODE_TYPE;
     SOURCE_BASE      : in  NODE_TYPE;
     NEW_KEY          : in  RELATIONSHIP_KEY  := EMPTY_KEY_LIST;
     NEW_RELATION     : in  RELATION_NAME     := DEFAULT_RELATION;
     NEW_ATTRIBUTES   : in  ATTRIBUTE_LIST    := EMPTY_LIST;
     NEW_REVERSE_KEY  : in  RELATIONSHIP_KEY  := EMPTY_KEY_LIST;
```


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
        NEW_REVERSE_ATTRIBUTES : in    ATTRIBUTE_LIST    := EMPTY_LIST;
        STATUS                 : in    STATUS_INDICATOR  := NO_INDICATION)
    is
        RELATION_DEFINITION : RD_TYPE;
    begin
        DEFINITION_OF_RELATION(RELATION_DEFINITION, SOURCE_BASE,
                               NEW_RELATION, STATUS);
        CREATE_BIDIRECTIONAL_SECONDARY_RELATIONSHIP
            (TARGET_NODE, SOURCE_BASE,
             NEW_KEY, RELATION_DEFINITION, NEW_ATTRIBUTES,
             NEW_REVERSE_KEY, NEW_REVERSE_ATTRIBUTES, STATUS);
        CLOSE (RELATION_DEFINITION);
    exception
        when others =>
            CLOSE (RELATION_DEFINITION);
            raise;
    end CREATE_BIDIRECTIONAL_SECONDARY_RELATIONSHIP;
```

Notes:

CREATE_SECONDARY_RELATIONSHIP can be used to create a bidirectional secondary relationship as well when the default values for keys are to be used for the reverse relationship. 25

The ALLOWED_USAGE attribute of the RD identified by NEW_RELATION must allow the use of the relation to create secondary relationships. 26

Bidirectional relationships cannot be inheritable. 27

### 5.2.36 Deleting secondary relationships

```
procedure DELETE_SECONDARY_RELATIONSHIP
    (BASE     : in    NODE_TYPE;
     KEY      : in    RELATIONSHIP_KEY  := EMPTY_KEY_LIST;
     RELATION : in    RD_TYPE;
     STATUS   : in    STATUS_INDICATOR  := NO_INDICATION);
```

Purpose:

This procedure deletes a secondary relationship identified by BASE, KEY and RELATION. If the relationship is bidirectional, the corresponding reverse relationship is also deleted. 1

Parameters:

**BASE** is an open node handle to the node from which the relationship to be deleted emanates. 2

**KEY** is the relationship key list identifying the relationship to be deleted. 3

**RELATION** is an open node handle to an RD of the relationship to be deleted. 4

**STATUS** is an established status indicator. 5


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Exceptions:

**PATHNAME_SYNTAX_ERROR** is raised if the node identification given by KEY is syntactically illegal (see Table I, page 52) [2]. 6

**NAME_ERROR** is raised if the relation identified by relation is bidirectional and the target node is inaccessible [1]. 7

**NOT_IN_VIEW_ERROR** is raised if a key name specified in KEY cannot be translated [1] or if KEY consists of a single relationship key designator and no key of the specified relation is visible [5]. 8

**AMBIGUOUS_VIEW_ERROR** is raised if translation of a key name specified in KEY is ambiguous [1] or if KEY consists of a single relationship key designator and more than one key of the specified relation is visible [4]. 9

**PREDEFINED_RELATION_ERROR** is raised if RELATION identifies a relation that is CAIS-controlled [1]. 10

**RELATIONSHIP_ERROR** is raised if KEY specifies a value for the same key more than once [5] or if the relationship identified by BASE, KEY and RELATION does not exist [2] or more than one such relationship exists [1]. 11

**STATUS_ERROR** is raised if either of the node handles BASE or RELATION is not open [2] or if STATUS is not an established status indicator [3]. 12

**NODE_KIND_ERROR** is raised if the open node handle RELATION does not designate an RD [1]. 13

**CONFORMANCE_ERROR** is raised if the operation would result in an attribute, node or relationship not conforming to its definition or the constraints in force on that definition [3]. 14

**TRANSACTION_ERROR** is raised if the operation is included in more than one transaction [1] or if the operation is included in a transaction and the operation modifies a node which is not included in the same transaction [2]. 15

**LOCK_ERROR** is raised if the relationship to be deleted is bidirectional and access with intent WRITE_RELATIONSHIPS to the target node cannot be obtained due to an existing lock on the target node [1]. 16

**INTENT_VIOLATION** is raised if BASE was not opened with an intent establishing the right to write relationships [1]. 17

**ACCESS_VIOLATION** is raised if RELATION identifies a bidirectional relation and the current process does not have sufficient discretionary access control rights to obtain access to the target node of the relationship to be deleted with intent WRITE_RELATIONSHIPS [1]. 18

**SECURITY_VIOLATION** is raised if the operation represents a violation of mandatory access controls. 19

Additional Interfaces:

```
procedure DELETE_SECONDARY_RELATIONSHIP
    (BASE     : in  NODE_TYPE;
     KEY      : in  RELATIONSHIP_KEY  := EMPTY_KEY_LIST;
     RELATION : in  RELATION_NAME     := DEFAULT_RELATION;
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
        STATUS      : in      STATUS_INDICATOR  := NO_INDICATION)
    is
        RELATION_DEFINITION : RD_TYPE;
    begin
        DEFINITION_OF_RELATION (RELATION_DEFINITION, BASE, RELATION, STATUS);
        DELETE_SECONDARY_RELATIONSHIP (BASE, KEY, RELATION_DEFINITION, STATUS);
        CLOSE (RELATION_DEFINITION);
    exception
        when others =>
            CLOSE (RELATION_DEFINITION);
            raise;
    end DELETE_SECONDARY_RELATIONSHIP;
```

Notes:

DELETE_SECONDARY_RELATIONSHIP can be used to delete secondary relationships to nodes that, have become unobtainable.

### 5.2.37 Obtaining a corresponding reverse relationship's relation

```
procedure GET_REVERSE_RELATION
    (REVERSE_RELATION : in out RD_TYPE;
     BASE             : in     NODE_TYPE;
     KEY              : in     RELATIONSHIP_KEY  := EMPTY_KEY_LIST;
     RELATION         : in     RD_TYPE;
     STATUS           : in     STATUS_INDICATOR  := NO_INDICATION);

function GET_REVERSE_RELATION
    (BASE             : in     NODE_TYPE;
     KEY              : in     RELATIONSHIP_KEY  := EMPTY_KEY_LIST;
     RELATION         : in     RD_TYPE;
     STATUS           : in     STATUS_INDICATOR  := NO_INDICATION)
    return RELATION_NAME;
```

Purpose:

This routine returns an identification of the specified relationship's corresponding reverse relation. BASE identifies the node from which the relationship specified by RELATION and KEY emanates. This relationship terminates at the source node for the reverse relationship.

An open node handle with NO_ACCESS intent to an RD is returned in REVERSE_RELATION for the first form of the interface. A relation name defined by a currently adopted view is returned in the second form.

No path is traversed in opening the node handle REVERSE_RELATION. Consequently, it cannot be passed to subprograms PATH_KEY and PATH_RELATION (see Section 5.2.20 on page 119 and Section 5.2.21 on page 120).


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Parameters:

**REVERSE_RELATION** is a closed node handle to be opened to the RD for the reverse relationship corresponding to the given relationship.

**BASE** is an open node handle.

**KEY** is a key list for one relationship of a bidirectional relationship.

**RELATION** is an open node handle identifying the RD for one relationship of a bidirectional relationship.

**STATUS** is an established status indicator.

Exceptions:

**PATHNAME_SYNTAX_ERROR** is raised if the KEY given is syntactically illegal [2].

**NAME-ERROR** is raised if the target node of the relationship specified by BASE, KEY and RELATION is inaccessible or unobtainable [1].

**RELATIONSHIP_ERROR** is raised if the relationship identified by BASE, KEY and RELATION does not exist [2] or if KEY specifies a value for the same key more than once [5].

**NOT_IN_VIEW_ERROR** is raised if a name specified in KEY cannot be translated [1], if the relation name to be returned cannot be formed [3] or if KEY consists of a single relationship key designator and no key of the corresponding relation is visible [5].

**AMBIGUOUS_VIEW_ERROR** is raised if a name specified in KEY has an ambiguous translation [I], if formation of the relation name to be returned is ambiguous [2] or if KEY consists of a single relationship key designator and more than one key of the corresponding relation is visible [4].

**USE_ERROR** is raised if the relationship specified by BASE, KEY and RELATION is not a bidirectional relationship [28].

**STATUS_ERROR** is raised if BASE or RELATION is not an open node handle [2], if REVERSE_RELATION is an open node handle [1] or if STATUS is not an established status indicator [3].

**NODE_KIND_ERROR** is raised if the open node handle RELATION does not identify an RD [1].

**TRANSACTION_ERROR** is raised if the operation is included in more than one transaction [1].

**LOCK_ERROR** is raised if a node handle to the target node specified by BASE, KEY and RELATION cannot be opened with intent READ_RELATIONSHIPS, due to the existence of locks in conflict with that intent [1].

**INTENT_VIOLATION** is raised if the node handle BASE was not opened with an intent establishing the right to read relationships [1].

**ACCESS_VIOLATION** is raised if the discretionary access control rights of the current process are insufficient to allow access to the target node specified by BASE, KEY and RELATION with intent READ_RELATIONSHIPS [1].

**SECURITY_VIOLATION** is raised if the operation represents a violation of mandatory access controls.


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

### 5.2.38 Obtaining a corresponding reverse relationship's key

```
function GET_REVERSE_KEY
    (BASE     : in     NODE_TYPE;
     KEY      : in     RELATIONSHIP_KEY  := EMPTY_KEY_LIST;
     RELATION : in     RD_TYPE;
     STATUS   : in     STATUS_INDICATOR  := NO_INDICATION)
    return RELATIONSHIP_KEY;
```

Purpose:

This function returns the key list of the specified relationship's corresponding reverse relationship. BASE identifies the node from which the relationship specified by RELATION and KEY emanates. This relationship terminates at the source node for the reverse relationship. 1

The key list returned can be used by the current process in identifying the reverse relationship. If the reverse relation has multiple keys, only values for keys with names specified in an adopted view of the current process are returned. If the reverse relation has only one key, it is returned as an unnamed key designator. 2

Parameters:

**BASE** is an open node handle. 3

**KEY** is a key list for one relationship of a bidirectional relationship. 4

**RELATION** is an open node handle identifying the RD for one relationship of a bidirectional relationship. 5

**STATUS** is an established status indicator. 6

Exceptions:

**PATHNAME_SYNTAX_ERROR** is raised if the KEY given is syntactically illegal [2]. 7

**NAME_ERROR** is raised if the target node of the relationship specified by BASE, KEY and RELATION is inaccessible or unobtainable [1]. 8

**RELATIONSHIP_ERROR** is raised if the relationship identified by BASE, KEY and RELATION does not exist [2] or if KEY specifies a value for the same key more than once [5]. 9

**NOT_IN_VIEW_ERROR** is raised if a name specified in KEY cannot be translated [1], if a path element for the reverse relationship cannot be formed [3] or if KEY consists of a single relationship key designator and no key of the corresponding relation is visible [5]. 10

**AMBIGUOUS_VIEW_ERROR** is raised if a name specified in KEY has an ambiguous translation [1], if formation of a path element for the reverse relationship is ambiguous [2] or if KEY consists of a single relationship key designator and more than one key of the corresponding relation is visible [4]. 11

**USE_ERROR** is raised if the relationship specified by BASE, KEY and RELATION is not a bidirectional relationship [28]. 12



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

**STATUS_ERROR** is raised if BASE or RELATION is not an open node handle [2] or if STATUS is not an established status indicator [3].

**TRANSACTION_ERROR** is raised if the operation is included in more than one transaction [1].

**LOCK_ERROR** is raised if a node handle to the target node specified by BASE, KEY and RELATION cannot be opened with intent READ_RELATIONSHIPS, due to the existence of locks in conflict with that intent [1].

**INTENT_VIOLATION** is raised if the node handle BASE was not opened with an intent establishing the right to read relationships [1].

**ACCESS_VIOLATION** is raised if the discretionary access control rights of the current process are insufficient to allow access to the target node specified by BASE, KEY and RELATION with intent READ_RELATIONSHIPS [1].

**SECURITY_VIOLATION** is raised if the operation represents a violation of mandatory access controls.

Notes:

The key specification returned for the reverse relationship might not uniquely identify a single relationship if the adopted views of the current process do not specify names for all keys of the reverse relationship.

### 5.2.39 Setting the inheritance property of a relationship

```
procedure SET.INHERITANCE
    (BASE        : in  NODE_TYPE;
     KEY         : in  RELATIONSHIP_KEY  : = EMPTY_KEY_LIST ;
     RELATION    : in  RD_TYPE;
     INHERITABLE : in  BOOLEAN;
     STATUS      : in  STATUS_INDICATOR  := NO_INDICATION);
```

Purpose:

This procedure sets the value of the INHERITABLE attribute of the relationship identified by BASE, KEY and RELATION to the value in INHERITABLE.

Parameters:

**BASE** is an open node handle to the node from which the relationship emanates.

**KEY** is the relationship key list of the affected relationship.

**RELATION** is an open node handle to an RD of the affected relationship.

**INHERITABLE** specifies the new value of the INHERITABLE attribute.

**STATUS** is an established status indicator.


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Exceptions:

**PATHNAME_SYNTAX_ERROR** is raised if the relationship identification given by KEY is syntactically illegal (see Table I, page 52) [2]. 7

**NOT_IN_VIEW_ERROR** is raised if a key name specified in KEY cannot retranslated [l] or if KEY consists of a single relationship key designator and no key of the specified relation is visible [5]. 8

**AMBIGUOUS_VIEW_ERROR** is raised if translation of a key name specified in KEY is ambiguous [1] or if KEY consists of a single relationship key designator and more than one key of the specified relation is visible [4]. 9

**PREDEFINED_RELATION_ERROR** is raised if RELATION identifies a predefine relation whose inheritance property cannot be altered by the user as specified in Section 4.3.2.2 on page 42 [2]. 10

**RELATIONSHIP_ERROR** is raised KEY specifies a value for the same key more than once [5], if the relationship identified by BASE, KEY and RELATION does not exist [2] or more than one such relationship exists [1]. 11

**CONFORMANCE_ERROR** is raised if the INHERITABLE attribute cannot be set to the specified value [IO]. 12

**STATUS_ERROR** is raised if the node handle BASE or RELATION is not open [2] or if STATUS is not an established status indicator [3]. 13

**NODE_KIND_ERROR** is raised if the open node handle RELATION does not designate an RD [1]. 14

**INTENT_VIOLATION** is raised if BASE was not opened with an intent establishing the right to write relationships [1]. 15

**SECURITY_VIOLATION** is raised if the operation represents a violation of mandatory access controls. 16

Additional Interfaces:

```
procedure SET_INHERITANCE
    (BASE          : in    NODE_TYPE;
     KEY           : in    RELATIONSHIP_KEY  := EMPTY_KEY_LIST;
     RELATION      : in    RELATION_NAME     := DEFAULT_RELATION;
     INHERITABLE   : in    BOOLEAN;
     STATUS        : in    STATUS_INDICATOR  := NO_INDICATION)
is
     RELATION_DEFINITION : RD_TYPE;
begin
     DEFINITION_OF_RELATION (RELATION_DEFINITION, BASE, RELATION, STATUS);
     SET_INHERITANCE( BASE, KEY, RELATION_DEFINITION,
                      INHERITABLE, STATUS);
     CLOSE (RELATION_DEFINITION);
exception
     when others =>
          CLOSE (RELATION_DEFINITION);
          raise;
end SET_INHERITANCE;
```


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

### 5.2.40 Determining if a secondary relationship is inheritable

```
function IS_INHERITABLE
    (BASE     : in  NODE_TYPE;
     KEY      : in  RELATIONSHIP_KEY  := EMPTY_KEY_LIST;
     RELATION : in  RD_TYPE;
     STATUS   : in  STATUS_INDICATOR  := NO_INDICATION)
    return BOOLEAN;
```

Purpose:

This function returns the value of the INHERITABLE attribute of the relationship identified by BASE, KEY and RELATION. For bidirectional relationships, this function always returns FALSE.

Parameters:

**BASE** is an open node handle to the node from which the relationship emanates.

**KEY** is the relationship key list identifying the affected relationship.

**RELATION** is an open node handle to an RD of the affected relationship.

**STATUS** is an established status indicator.

Exceptions:

**PATHNAME_SYNTAX_ERROR** is raised if the relationship identification given by KEY is syntactically illegal (see Table I, page 52) [2].

**NOT_IN_VIEW_ERROR** is raised if a key name specified in KEY cannot retranslated [l] or if KEY consists of a single relationship key designator and no key of the specified relation is visible [5].

**AMBIGUOUS_VIEW_ERROR** is raised if translation of a key name specified in KEY is ambiguous [1] or if KEY consists of a single relationship key designator and more than one key of the specified relation is visible [4].

**RELATIONSHIP_ERROR** is raised KEY specifies a value for the same key more than once [5] or if the relationship identified by BASE, KEY and RELATION does not exist [z] or more than one such relationship exists [1].

**STATUS_ERROR** is raised if the node handle BASE or RELATION is not open [2] or if STATUS is not an established status indicator [3].

**INTENT_VIOLATION** is raised if BASE was not opened with an intent establishing the right to read relationships [1].

Additional Interfaces:

```
function IS_INHERITABLE
    (BASE     : in  NODE_TYPE;
     KEY      : in  RELATIONSHIP_KEY  : = EMPTY_KEY_LIST;
```


Source: https://assist.dla.mil -- Downloaded: 2026l0-26T07:54Z
Check the source to verify that this is the currentversion before use.

```
        RELATION : in     RELATION_NAME     := DEFAULT_RELATION;
        STATUS   : in     STATUS_INDICATOR  := NO_INDICATION)
    return BOOLEAN
is
    RELATION_DEFINITION : RD_TYPE;
    RESULT  : BOOLEAN;
begin
    DEFINITION_OF_RELATION (RELATION_DEFINITION, BASE, RELATION, STATUS);
    RESULT  := IS_INHERITABLE(BASE, KEY, RELATION_DEFINITION, STATUS);
    CLOSE (RELATIOM_DEFINITION);
    return RESULT;
exception
    when others =>
        CLOSE (RELATION_DEFINITION);
        raise;
end IS_INHERITABLE;
```

### 5.2.41 Setting the current node relationship

```
procedure SET_CURRENT_NODE
    (NODE       : in     NODE_TYPE;
     TIME_LIMIT : in     CAIS_DURATION     := LONG_DELAY;
     STATUS     : in     STATUS_INDICATOR  := NO_INDICATION);
```

Purpose:

This procedure specifies the node identified by NODE as the current node. The CURRENT_NODE relationship of the current process is changed accordingly. 1

Parameters:

**NODE** is an open node handle to a node to be the new target node of tile CURRENT_NODE relationship emanating from the current PROCESS node. 2

**TIME_LIMIT** is the maximum time to wait for the unlocking of the current PROCESS node in accordance with the intent WRITE_RELATIONSHIPS. 3

**STATUS** is an established status indicator. 4

Exceptions:

**STATUS_ERROR** is raised if the node handle NODE is not open [2] or if STATUS is not an established status indicator [3]. 5

**LOCK_ERROR** is raised if access, with intent WRITE_RELATIONSHIPS, to the current PROCESS node cannot be obtained within the specified TIME_LIMIT due to an existing lock on the node [1]. LOCK_ERROR may be raised prior to expiration of the time limit if the CAIS implementation can determine that a deadlock situation has occurred. 6


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

### 5.2.42 Opening a node handle to the current node

```
procedure GET_CURRENT_NODE
    (NODE       : in out NODE_TYPE;
     INTENT     : in     INTENT_ARRAY     := (1 => NO_ACCESS);
     TIME_LIMIT : in     CAIS_DURATION    := LONG_DELAY;
     STATUS     : in     STATUS_INDICATOR := NO_INDICATION);
```

Purpose:

This procedure returns an open node handle to the current node of the current process in NODE; the intent with which the node handle is opened is specified by INTENT. 1

Parameters:

**NODE** is anode handle, initially closed, to be opened to the current node. 2

**INTENT** is the intent of subsequent operations on the node handle NODE. 3

**TIME_LIMIT** is the maximum time to wait for the unlocking of the current node in accordance with the desired INTENT. 4

**STATUS** is an established status indicator. 5

Exceptions:

**NAME_ERROR** is raised if the current node is inaccessible or unobtainable and the INTENT contains any intent specification other than NO_ACCESS [1]. 6

**USE_ERROR** is raised if INTENT is an empty array [l]. 7

**STATUS_ERROR** is raised if NODE is an open node handle at the time of the call [2] or if STATUS is not an established status indicator [3]. 8

**TRANSACTION_ERROR** is raised if the operation is included in more than one transaction [1]. 9

**LOCK_ERROR** is raised if access, with intent READ_RELATIONSHIPS, to the current PROCESS node cannot be obtained within the specified TIME_LIMIT due to an existing lock on the node [1]. LOCK_ERROR may be raised prior to expiration of the time limit if the CAIS implementation can determine that a deadlock situation has occurred. 10

**ACCESS_VIOLATION** is raised if the discretionary access rights of the current process are insufficient to obtain access to the current node with the specified INTENT. 11

**SECURITY_VIOLATION** is raised if the operation represents a violation of mandatory access controls. 12

### 5.2.43 Determining the creation time of a node

```
function TIME_CREATED
    (NODE   : in NODE_TYPE;
     STATUS : in STATUS_INDICATOR : = NO_INDICATION)
    return CAIS_CALENDAR. TIME;
```


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Purpose:

This function returns a value of type CAIS_CALENDAR.TIME representing the value of the TIME_CREATED attribute of the node identified by NODE. The value returned is the time at which the node was created. 1

Parameters:

**NODE** is an open node handle identifying the node whose attribute is being queried. 2

**STATUS** is an established status indicator. 3

Exceptions:

**STATUS_ERROR** is raised if NODE is not an open node handle [2] or if STATUS is not an established status indicator [3]. 4

**INTENT_VIOLATION** is raised if NODE was not opened with an intent to read attributes [1]. 5

### 5.2.44 Determining the last time a relationship was modified

```
function TIME_RELATIONSHIP_WRITTEN
    (NODE   : in    NODE_TYPE;
     STATUS : in    STATUS_INDICATOR  := NO_INDICATION)
    return CAIS_CALENDAR. TIME;
```

Purpose:

This function returns a value of type CAIS_CALENDAR.TIME representing the value of the TIME_RELATIONSHIP_WRITTEN attribute of the node identified by NODE. The value returned is the time at which any relationship emanating from the node identified by NODE was modified (i.e., a new relationship added or an existing relationship deleted) or at which any attributes of any relationship emanating from the node were modified (i.e., value of an attribute of the relationship changed, a new attribute of the relationship added or an existing attribute of the relationship deleted). 1

When a node is created (except through COPY or COPY_TREE), the value of the TIME_-RELATIONSHIP_WRITTEN attribute is set to the same value as the TIME_CREATED attribute. 2

The effects of any implicit operations performed by an implementation on implementation-defined properties (if any) must not be visible via the CAIS interfaces since such effects are not defined. In particular, such operations must not affect the value of TIME_RELATIONSHIP_WRITTEN. 3

Parameters:

**NODE** is an open node handle identifying the node whose attribute is being queried. 4

**STATUS** is an established status indicator. 5



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Exceptions:

**STATUS_ERROR** is raised if NODE is not an open node handle [2] or if STATUS is not an established status indicator [3]. 6

**INTENT_VIOLATION** is raised if NODE was not opened with an intent to read attributes [1]. 7

### 5.2.45 Determining the last time that node contents were written

```
function TIME_CONTENTS_WRITTEN
    (NODE   : in    FILE_MODE_TYPE;
     STATUS : in    STATUS_INDICATOR := NO_INDICATION)
    return CAIS_CALENDAR. TIME;
```

Purpose:

This function returns a value of type CAIS_CALENDAR.TIME representing the value of the TIME_CONTENTS_WRITTEN attribute of the FILE node identified by NODE. The value of this attribute is modified when a RESET, SYNCHRONIZE or CLOSE operation is performed by a protocol package if a write operation has been performed using the same open file handle subsequent to the last RESET or SYNCHRONIZE operation, or subsequent to opening the file handle if no RESET or SYNCHRONIZE operation has been performed using the file handle. 1

When a FILE node is created (except through COPY and COPY_TREE), the value of the TIME. CONTENTS_WRITTEN attribute is set to the same value as the TIME_CREATED attribute. 2

Parameters:

**NODE** is an open node handle identifying the node whose attribute is being queried. 3

**STATUS** is an established status indicator. 4

Exceptions:

**STATUS_ERROR** is raised if NODE is not an open node handle [2] or if STATUS is not an established status indicator [3]. 5

**NODE_KIND_ERROR** is raised if the node identified by NODE is not a FILE node [1]. 6

**INTENT_VIOLATION** is raised if NODE was not opened with an intent to read attributes [1]. 7

### 5.2.46 Determining the last time an attribute was modified

```
function TIME_ATTRIBUTE_WRITTEN
    (NODE   : in    NODE_TYPE;
     STATUS : in    STATUS_INDICATOR := NO_INDICATION)
    return CAIS_CALEHDAR. TIME;
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Purpose:

This function returns a value of type CAIS_CALENDAR.TIME representing the value of the TIME_ATTRIBUTE_WRITTEN attribute of the node identified by NODE. The value returned is the last time at which an attribute of the node identified by NODE was modified (i.e., an attribute value changed, a new attribute added or an existing attribute deleted) by a call on a CAIS interface.

When a node is created (except via COPY or COPY_TREE), the value of the TIME_ATTRIBUTE_WRITTEN attribute is set to the same value as the TIME_CREATED attribute.

Changes to the following predefine attributes do not cause the TIME_ATTRIBUTE_WRITTEN attribute to be modified:

| | |
|---|---|
| CURRENT_FILE_SIZE | HIGHEST_CLASSIFICATION |
| IO_UNIT_COUNT | LOWEST_CLASSIFICATION |
| MACHINE_TIME | OBJECT_CLASSIFICATION |
| OPEN_NODE_HANDLE_COUNT | PROCESS_SIZE |
| SUBJECT_CLASSIFICATION | TIME_ATTRIBUTE_WRITTEN |
| TIME_COMTENTS_WRITTEN | TIME_CREATED |
| TIME_FINISHED | TIME_RELATIONSHIP_WRITTEN |
| TIME-STARTED | |

The effects of any implicit operations performed by an implementation on implementation-defined properties (if any) must not be visible via the CAIS interfaces since such effects are not defined. in particular, such operations must not affect the value of TIME_ATTRIBUTE_WRITTEN.

Parameters:

**NODE** is an open node handle identifying the node whose attribute is being queried.

**STATUS** is an established status indicator.

Exceptions:

**STATUS_ERROR** is raised if NODE is not an open node handle [2] or if STATUS is not an established status indicator [3].

**INTENT_VIOLATION** is raised if NODE was not opened with an intent to read attributes [1].

### 5.2.47 Iteration predicate subtype

*Iteration* is the sequential identification of the elements in a set of nodes, relationships or attributes. The CAIS provides interfaces for several types of iteration. A *node iteration* is an iteration over the target nodes of a set of relationships emanating from a specified base node. A *node kind instance* iteration is an iteration over a set of nodes which are instances of a given NKD. An *attribute iteration* is an iteration over a set of attributes of a specified node or relationship (see Section 5.3.15 on page 245).

To perform an iteration, an iterator must be created. An *iterator* is an object of a limited private type which provides the bookkeeping information necessary to perform the iteration. The iterator may



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

be used to obtain an identification of each of the objects included in the iteration. Node iterators and node kind instance iterators permit the specification of an *iteration predicate.* This predicate is a Boolean valued expression. The predicate is evaluated for each node (and relationship) which is eligible for inclusion in the iteration. If the predicate evaluates to TRUE, the node is included in the iteration.

```
subtype ITERATION_PREDICATE is LIST_TYPE;
```

ITERATION_PREDICATE is a subtype of LIST_TYPE used to represent a predicate. 3

### 5.2.47.1 Predicate representation

Predicates are represented as CAIS lists; the general format for lists is defined by the package CAIS_-LIST_MANAGEMENT (Section 9.1 on page 751). The format of lists which represent iteration predicates is further restricted by the syntax equations in Table VI, on page 160. Predicates may be created and modified using the interfaces defined by the package CAIS_LIST_MANAGEMENT. 1

In a predicate, lists and list items are used to represent expressions. Simple items occurring in a predicate are treated as literal values, except for certain reserved identifiers (see Table VI on page 160). Sublists are interpreted as operations, function calls, literal items or lists comprised of expressions. When a reserved identifier or sublist is to be treated as a literal rather than evaluated as an expression, it must be represented as a literal item (see Section 5.2.47.11 on page 167). 2

Boolean literals (FALSE, TRUE) included in predicates can be either ordinary identifier items or enumerated identifier items. Boolean values resulting from the evaluation of expressions within predicates are enumerated identifier items. 3

The predicate for an iteration is specified when the iterator is created. A predicate which is syntactically incorrect, according to Table VI on page 160, causes the SYNTAX_ERROR exception to be raised when it is used to create an iterator. 4

### 5.2.47.2 Evaluation of predicates

During an iteration, the GET_NEXT interface (which returns the next node on the iterator) will raise an exception if evaluation of the predicate requires access to a property of the node and either the current process does not have the necessary access rights to read the property, the node is locked against reading of that property, or the iteration would be included in more than one transaction. 1

No other exceptions are raised by the evaluation of a predicate. However, the result of evaluating expressions in the predicate may be undefined. For example, the result of a NODE_ATTRIBUTE function call (which retrieves the value of a node attribute) is undefined if the current node in the iteration does not have a value for the specified attribute. 2



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

TABLE VI: **Iteration Predicate BNF**

```
predicate           ::= (expression)
expression          ::=   unary_operation
                        | binary_operation
                        | function_call
                        | item
                        | literal
unary_operation     ::= (unary_operator, expression)
binary_operation    ::= (expression, binary_operator, expression)
function_call       ::=   IS_PRIMARY
                        | (HAS_EMANATING, expression)
                        | (IS_DEFINED, expression)
                        | (NODE_ATTRIBUTE, expression)
                        | (RELATIONSHIP_ATTRIBUTE, expression)
                        | (EXTRACT_VALUE, expression [, expression])
item                ::=   list_item | string_item | integer_item
                        | float_item | identifier_item
literal             ::= (LITERAL, literal_item)
list_item           ::=   unnamed_list | named_list | empty_list
unnamed_list        ::= (expression {, expression})
named_list          ::= (identifier => expression
                          {, identifier => expression})
empty_list          ::= ()
unary_operator      ::= NOT
binary_operator     ::=   AND | AND_THEN | OR | OR_ELSE | XOR
                        | EQUAL | NOT_EQUAL | MATCHES
                        | GREATER | GREATER_EQUAL
                        | LESS | LESS_EQUAL
reserved_identifier ::=   unary_operator | binary_operator | EXTRACT_VALUE
                        | HAS_EMANATING | IS_DEFINED | IS_PRIMARY | LITERAL
                        | NODE_ATTRIBUTE | RELATIONSHIP_ATTRIBUTE
```

Notes:

1. See Appendix D for a description of the notation used.
2. An identifier_item may not be a reserved_identifier.
3. A literal_item is any item which may occur in a CAIS list (see Section 9 on page 745) including a list item. Since a literal_item is not an expression, it is not evaluated.
4. Integer items, float items, and string items are defined in Section 9 on page 745.



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

An undefined result is treated as a special value. When an operand in an operation or a parameter in a function call has this undefined value, the result of evaluation is undefined except in the case of certain logical operators (see Section 5.2.47.6 page 163) and the IS_DEFINED function (see Section 5.2.47.9 page 166). In addition, a list item is considered undefined if any item in the list is an expression whose result is undefined.

If the result of evaluation of the predicate is TRUE for a node eligible for inclusion in the iteration, or if the predicate cannot be evaluated due to an existing lock or due to insufficient access rights, that node is included in the iteration. Otherwise, the node is excluded. In particular, if the result of a successful evaluation is undefined, FALSE, or any value other than TRUE, the node is excluded.

### 5.2.47.3 Operations and function calls in predicates

Binary operations in predicates are represented as list items containing three items: An expression, followed by an identifier item naming a binary operator, followed by another expression. For example an AND operation is represented as the list:

```
(x, AND, y)
```

where *x* and *y* are the operand expressions.

Unary operations in predicates are represented as list items containing an identifier item naming a unary operator followed by an expression. For example, a NOT operation is represented as the list:

```
(NOT, x)
```

where *x* is the operand expression.

Since each operation and operand is represented by a separate list item, there is no precedence defined for operators. All expressions are fully parenthesized.

Operators are represented by reserved identifier items in the predicate.

Similarly, function calls are represented as lists in which the first item is a reserved identifier naming the function to be called and the remaining items are the parameters to the function. For example, the following function call returns the value of the KIND attribute:

```
(NODE_ATTRIBUTE, KIND)
```

### 5.2.47.4 Relational operators

Relational operators allowed in predicates are:

```
EQUAL       NOT_EQUAL
GREATER GREATER_EQUAL
LESS        LESS_EQUAL
```

The result of a relational operator is either TRUE, FALSE or undefined. When either operand of a relational operator is undefined, the result is undefined.


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

There are two types of relational operators: equality operators and ordering operators. Equality operators determine whether their operands have the same values. Ordering operators determine whether their operand values have a particular ordering.

The following relational operators are *equality operators:*

EQUAL NOT_EQUAL

Equality of items, including list items, is defined as specified in Section 9 on page 745. In particular, items which have different values, different types or different item names (under identifier equality) are not equal.

Users should be aware of the accuracy issues for equality operators between floating point types explained in [1815A], Section 4.5.7 in order to avoid erroneous assumptions about the equality of float items and lists involving such items.

The following relational operators are *ordering operators:*

GREATER GREATER_EQUAL
LESS LESS_EQUAL

Ordering operators may also be applied to all types of items including list items. The result is undefined for operands with different item types or different item names (under identifier equality).

The ordering operators for integer and float items are defined by the corresponding Ada relational operators for the type CAIS_INTEGER (see Section 10 on page 805) and for a floating point type which preserves at least CAIS_PRAGMATICS.LIST_MAXIMUM_DIGITS of accuracy, respectively.

The ordering operators for string items apply ASCII lexicographical order.

For identifier items, if either item is an ordinary identifier item, the ordering operator applies ASCII lexicographical order to the identifier text forms after raising lower case characters to upper case. If both items are enumerated identifier items, the ordering operator compares the enumeration positions. If the enumeration positions are the same but the images are different, the result of applying the ordering operator is undefined.

Ordering operators are evaluated as follows for list operands. If one list is named and the other is unnamed, the result is undefined. If both lists are empty, they are equal. If only one list is empty, it precedes (is less than) the other list. Otherwise, the first items in the lists are compared using the EQUAL operator. If first items are not equal, the result is determined by applying the ordering operator to those items (recursively). If the first items are equal, the result is determined by applying the ordering operator (recursively) to the lists formed by removing the first item from each list operand.

Examples of relational operations and their results:

Items:

(5, LESS, 10) => TRUE

(5.0, LESS, 10) => undefined

(ITEM, EQUAL, “ITEM”) => FALSE


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

(ITEM, GREATER, "ITEM") => undefined

(ABC, LESS_EQUAL, DEF) => TRUE

("CAIS", LESS, "CAIS-A") => TRUE

Lists:

( (1, 2, 3), EQUAL, (1, 2) ) => FALSE

( (1, 2, 3), GREATER, (1, 2) ) => TRUE

( (1, 2, 3), LESS_EQUAL, (1, 2, 4) ) => TRUE

( (COLOR => RED), GREATER_EQUAL, (HUE => RED) ) => undefined

( (YEAR => 1987, MONTH => 12, DAY=> 1),
GREATER,
(YEAR => 1987, MONTH => 11, DAY => 11) ) => TRUE

#### 5.2.47.5 MATCHES operator

The MATCHES operator compares a string or identifier item with a pattern. The pattern is represented as a string which may include the wild card characters “*” (representing zero or more characters) and “?” (representing any single character). The first operand of MATCHES is a string or identifier. The second operand is the string representing the pattern to be matched. The result of a MATCHES operation is undefined for other types of operands. The rules for matching are the same as those defined for conformance to the PATTERNS attribute in string and identifier AKDs (see Section 5.7.1.6.4011 page 318).

Examples of the MATCHES operator:

(AN_IDENTIFIER, MATCHES, "*IDENTIFIER") => TRUE

(“O-ZERO”, MATCHES, “O?ZERO”) => TRUE

(“STARTUP”, MATCHES, “START UP”) => FALSE

(5, MATCHES, “5”) => undefined

#### 5.2.47.6 Logical operators

The logical operators allowed in predicates are:

NOT XOR
AND AND_THEN
OR OR_ELSE



Source: https://assist.dla.mil -- Downloaded: 202610-26T07:54Z
Check the source to verify that this is the current version before use.

These operators have the same meanings as the corresponding Ada operators except that Boolean results are defined in certain cases when one of the operands is undefined. Boolean values are represented by the identifier items TRUE and FALSE. If an operand of a logical operation is evaluated and the resulting operand value is neither undefined, TRUE nor FALSE, then the result of the logical operation is undefined. 2

The second operand of an AND_THEN operation is evaluated only if the fist operand has the value TRUE. The second operand of an OR-ELSE operation is evaluated only if the first operand is FALSE. Table VII defines the results of these operations including the cases in which either or both operands are undefined. 3

### 5.2.47.7 EXTRACT_VALUE function

The EXTRACT_VALUE function is used to extract a value from a list. The first parameter must be an expression which evaluates to a list. If only the first parameter is provided, EXTRACT_VALUE returns the first item in the list. If two parameters are provided, the second parameter must be an expression which evaluates to an integer or an identifier. If it is an integer, the list item in the corresponding position is returned. If it is an identifier, the list must be named and the item having the specified name is returned. The result of EXTRACT_VALUE is undefined if the first parameter is not a list or the second parameter is not an integer or identifier which identifies an item in the list. 1

Examples:

(EXTRACT_VALUE, (5, 10, 15)) => 5

(EXTRACT_VALUE, (A, B, C), 2) => B

(EXTRACT_VALUE, (FIRST => ALBERT, SECOND => EINSTEIN), SECOND)
=> EINSTEIN

(EXTRACT_VALUE, (FIRST => ALBERT, SECOND => EINSTEIN), MIDDLE)
=> undefined

(EXTRACT_VALUE, 5) => undefined

(EXTRACT_VALUE, (1, 4, 3), 4) => undefined

(EXTRACT_VALUE, ( ) ) => undefined



Source: https://assist.dla.mil -- Downloaded: 202610-26T07:54Z
Check the source to verify that this is the current version before use.

| TABLE VII: **Logical Operators in Iteration Predicates** | | | | |
|---|---|---|---|---|
| **Binary Operators** | | | | |
| **Operator** | **Operand 1** | **Operand 2** | | |
| | | **TRUE** | **FALSE** | **undefined** |
| **AND** | **TRUE** | TRUE | FALSE | undefined |
| | **FALSE** | FALSE | FALSE | FALSE |
| | **undefined** | undefined | FALSE | undefined |
| **AND_THEN** | **TRUE** | TRUE | FALSE | undefined |
| | **FALSE** | FALSE | FALSE | FALSE |
| | **undefined** | undefined | undefined | undefined |
| **OR** | **TRUE** | TRUE | TRUE | TRUE |
| | **FALSE** | TRUE | FALSE | undefined |
| | **undefined** | TRUE | undefined | undefined |
| **OR_ELSE** | **TRUE** | TRUE | TRUE | TRUE |
| | **FALSE** | TRUE | FALSE | undefined |
| | **undefined** | undefined | undefined | undefined |
| **XOR** | **TRUE** | FALSE | TRUE | undefined |
| | **FALSE** | TRUE | FALSE | undefined |
| | **undefined** | undefined | undefined | undefined |
| **Unary Operators** | | | | |
| **Operator** | **Operand** | **Result** | | |
| **NOT** | **TRUE** | FALSE | | |
| | **FALSE** | TRUE | | |
| | **undefined** | undefined | | |

### 5.2.47.8 Attribute functions

Predicates may reference the attributes of nodes or relationships eligible for inclusion in an iteration. 1
Two functions which return the values of attributes are provided: NODE_ATTRIBUTE and RELATIONSHIP_ATTRIBUTE. The parameter to these functions must be the view_relative name of an AD; this is the name of an attribute definition node rather than a property name for the attribute applicable to a particular node kind or relation. The name is translated using the composite view of the current process when the iterator is created.

The applicability and interpretation of these functions depends on the type of iteration. For a node 2
iteration, RELATIONSHIP_ATTRIBUTE retrieves an attribute from each eligible relationship emanating from the specified base node, while NODE_ATTRIBUTE retrieves an attribute from the target node of each eligible relationship. For a node kind instance iteration, the function NODE_ATTRIBUTE refers to each node which is an instance of a specified NKD (see Table VIII). The RELATIONSHIP_-ATTRIBUTE function may not be used in a predicate for a node kind instance iterator.

| TABLE VIII: **Applications of Attribute Functions** | | |
|---|---|---|
| **Iteration Type** | **NODE_ATTRIBUTE** | **RELATIONSHIP_ATTRIBUTE** |
| Node | Target Node | Relationship to Target Node |
| Node Kind Instance | Instance of Node Kind | (invalid Predicate) |

Evaluation of an attribute function fails if the node is locked against the required access or if the 3



Source: https://assist.dla.mil -- Downloaded: 202610-26T07:54Z
Check the source to verify that this is the current version before use.

current process does not have the required read attributes access right to the node. In this case, the node is included in the iteration, but the GET_NEXT interface will raise an exception for that node.

The value of an attribute function is undefined for a node or relationship eligible for inclusion in the iteration if the attribute identified does not exist on the node or relationship.

Values returned by attribute functions are always lists. In particular, a simple item attribute is returned as an unnamed list containing a single item. The EXTRACT_VALUE function may be used to obtain the item itself.

Examples of attribute function calls:

```
(NODE_ATTRIBUTE, TIME_CREATED)  => value of TIME_CREATED attribute

(EXTRACT_VALUE, (NODE_ATTRIBUTE, TIME_CREATED), YEAR) =>
 YEAR component from TIME_CREATED attribute


(RELAT1ONSHIP_ATTRIBUTE, NODE_KIND)  => value of NODE_KIND attribute
```

#### 5.2.47.9 IS_DEFINED function

IS_DEFINED is a function which may be applied to any expression in a predicate. The resulting value is TRUE if the expression has a value and FALSE if the expression's value is undefined.

```
( IS_DEFINED, (85, EQUAL, THURSDAY) ) => FALSE

( IS_DEFINED, (85, EQUAL, 86) ) => TRUE

( IS_DEFINED, (NODE_ATTRIBUTE, STRUCTURAL_KIND) ) =>
                    TRUE for NKDs for certain STRUCTURAL
                    nodes, FALSE for other nodes.

( (NOT, (IS_DEFINED, (NODE_ATTRIBUTE, x))),
   OR_ELSE,
   ( (EXTRACT_VALUE, (NODE_ATTRIBUTE, X)), EQUAL, 10)
 ) =>               TRUE if the node attribute X
                    does not occur or its value is 10,
                    FALSE otherwise.
```

IS_DEFINED provides a means to test for the existence of an attribute of a node or relationship within a predicate.

#### 5.2.47.10 HAS_EMANATING function

Predicates may use the HAS_EMANATING function to test for the existence of relationships of specified relations emanating from nodes which are eligible for inclusion in the iteration. In the case of a node iterator, these nodes are the target nodes of relationships emanating from a single base node. In the case of a node kind instance iterator these nodes are nodes of the specified kind.



Source: https://assist.dla.mil -- Downloaded: 202610-26T07:54Z
Check the source to verify that this is the current version before use.

The HAS_EMANATING function may have one or two parameters. The first parameter is the name of an RD in the composite view of the current process. This name must be a view-relative name for a relation definition node rather than a property name for an emanating relation applicable to a particular node kind. The name is translated using the composite view of the current process when the iterator is created. 2

The second argument controls the inclusion of specializations of the relation. It must have the value TRUE or FALSE. If the second argument is TRUE, specializations of the specified relation are included. In this case, the HAS_EMANATING function returns TRUE if any relationship of the specified relation or of any specialization of that relation emanates from the eligible node. If the second argument is FALSE, specializations of the specified relation are excluded. In this case, the HAS_EMANATING function returns TRUE only if a relationship of the particular relation specified (excluding relationships of specializations) emanates from the eligible node. In all other cases, HAS_EMANATING returns FALSE. When the second argument is omitted, the value TRUE is assumed and specializations are included. 3

Evaluation of a HAS_EMANATING function fails if the node is locked against the required access or if the current process does not have the right to read relationships emanating from the node. In this case, the node is included in the iteration, but the GET_NEXT interface will raise an exception for the node. 4

The HAS_EMANATING function may be used in a node kind instance iteration to locate all relationships of a given relation that emanate from nodes of a given kind. All relationships of a given relation can be located if the iteration is over instances of the basic NKD. The HAS_EMANATING function is intended to provide an efficient means for locating relationships which avoids the additional overhead of creating an iterator for each included node to determine whether any emanating relationships of a given relation exist. 5

In the following examples, the HAS_EMANATING function is assumed to be used in an iteration over all definition nodes (all nodes of the predefine kind DEFINITION); ATTRIBUTE and KEY are view-relative names of RDs in the predefine view: 6

(HAS_EMANATING, ATTRIBUTE) => TRUE for all definition nodes which define or constrain attributes.

(HAS_EMANATING, ATTRIBUTE, TRUE) => same as above.

(HAS_EMANATING, KEY) => TRUE for all definition nodes which define or constrain key attributes.

(HAS_EMANATING, ATTRIBUTE, FALSE) => TRUE for all predefine definition nodes which define or constrain attributes that are not key attributes.

#### 5.2.47.11 Literal items

Reserved identifiers and lists in a predicate which must be interpreted as literal values can be specified as literal items. A literal item is represented by a list containing the reserved identifier LITERAL followed by any item which may occur in a CAIS list, including a list item. The item itself is the value of the literal item. The item is not evaluated. 1



Source: https://assist.dla.mil -- Downloaded: 202610-26T07:54Z
Check the source to verify that this is the current version before use.

Examples of the LITERAL operator: 2

(LITERAL, (5, GREATER, 10)) => (5, GREATER, 10)

(LITERAL, AND_THEN) => AND_THEN

### 5.2.47.12 IS_PRIMARY function

The implicit parameter to the IS_PRIMARY functions the current relationship in anode iteration. 1
The function is not allowed in a node kind instance iteration. The result is TRUE if the relationship is a primary relationship and FALSE otherwise.

Example:

```
( IS_PRIMARY, AND_THEN, ( (EXTRACT_VALUE (RELATIONSHIP_ATTRIBUTE, NAME) ),
        EQUAL , XYZ) )
                        => TRUE if the current relationship is primary
                           and the NAME attribute has the value XYZ,
                           FALSE otherwise.
```

## 5.2.48 Obtaining the default iteration predicate

```
function DEFAULT_PREDICATE
    (STATUS :  in STATUS_INDICATOR := NO_INDICATION)
    return  ITERATION_PREDICATE;
```

Purpose:

This function returns a CAIS list having the value: 1

(TRUE)

When used as an iteration predicate, the result of DEFAULT_PREDICATE causes all eligible instances to be included in the iteration. This function provides the default value for the PREDICATE parameter in the CREATE_ITERATOR interfaces. 2

Parameters:

**STATUS** is an established status indicator. 3

Exceptions:

**STATUS_ERROR** is raised if STATUS is not an established status indicator [3]. 4



Source: https://assist.dla.mil -- Downloaded: 202610-26T07:54Z
Check the source to verify that this is the current version before use.

### 5.2.49 Node iteration types and subtypes

The following types and subtypes are used in the interfaces for iterating over a set of nodes which are targets of relationships emanating from a base node. 1

```
type NODE_ITERATOR is limited private;
```

A node iterator is an Ada object of the type NODE_ITERATOR, which is a limited private type assumed to contain the bookkeeping information necessary for the implementation of the MORE, APPROXIMATE_SIZE, GET_NEXT, SKIP_NEXT and NEXT_NAME interfaces. The CREATE_-ITERATOR interface is used to create a node iterator over the target nodes of a collection of relationships emanating from a given base node. 2

```
subtype RELATION_NAME_PATTERN is RELATION_NAME;
```

Values of the subtype RELATION_NAME_PATTERN follow the syntax of relation names, except that the character "?" will match any single character and the character "*" will match any string of zero or more characters. 3

```
subtype RELATIOINSHIP_KEY_PATTERN is RELATIOINSHIP_KEY;
```

Values of the subtype RELATIONSHIP_KEY_PATTERN follow the syntax of a relationship key list except for the relationship key designators. The latest key character "#" may not appear in a relationship key designator. Also, a relationship key value may be specified in the form of an identifier pattern containing the characters "?" and "*". The character "?" will match any single character of a key value and the character "*" will match any string of zero or more characters of a key value. 4

```
type RELATIONSHIP_KIND is (PRIMARY, SECONDARY, BOTH);
type NODE_KIND_ARRAY    is array (CAIS_NATURAL range <>) of NODE_KIND;
```

RELATIONSHIP_KIND is an enumerated type which determines whether an iteration will be based on primary, secondary or both primary and secondary relationships. NODE_KIND_ARRAY is a type indicating the allowed values of the NODE_KIND attribute on relationships included in the iteration. 5

### 5.2.50 Creating an iterator over nodes

```
procedure CREATE_ITERATOR
    (ITERATOR              : in out NODE_ITERATOR;
     NODE                  :  in    NODE_TYPE;
     RELATION              :  in    RD_TYPE;
     PREDICATE             :  in    ITERATION_PREDICATE   := DEFAULT_PREDICATE ;
     INCLUDE_SPECIALIZATIONS
                           :  in    BOOLEAN               := TRUE;
     TIME_LIMIT            :  in    CAIS_DURATION         := LONG_DELAY;
     STATUS                :  in    STATUS_INDICATOR      := NO_INDICATION);
```



Source: https://assist.dla.mil -- Downloaded: 202610-26T07:54Z
Check the source to verify that this is the current version before use.

```
procedure CREATE. ITERATOR
    (ITERATOR           : in out NODE_ITERATOR;
     NODE               :  in    NODE_TYPE;
     KIND               :  in    NODE_KIND_ARRAY              := (FILE, STRUCTURAL);
     KEY                :  in    RELATIOINSHIP_KEY_PATTERN :=  EMPTY_KEY_LIST;
     RELATION           :  in    RD_TYPE;
     KIND_OF_RELATION   : in     RELATIONSHIP_KIND            := PRIMARY;
     STATUS             :  in    STATUS_INDICATOR             := NO_INDICATION);

procedure  CREATE_ITERATOR
    (ITERATOR           : in out NODE_ITERATOR;
     NODE               :  in    NODE_TYPE;
     KIND               :  in    NODE_KIND_ARRAY              := (FILE, STRUCTURAL);
     KEY                :  in    RELATIONSHIP_KEY_PATTERN := EMPTY_KEY_LIST;
     RELATION           :  in    RELATION_NAME_PATTERN        :=  DEFAULT_RELATION;
     KIND_OF_RELATION   : in     RELATIONSHIP_KIND            := PRIMARY;
     STATUS             :  in    STATUS_INDICATOR             := NO_INDICATION);
```

Purpose:

These procedures establish the node iterator ITERATOR over the relationships emanating from the base node identified by NODE. The iterator can then be processed by means of the node iteration interfaces MORE, APPROXIMATE_SIZE, GET_NEXT, SKIP_NEXT, NEXT_NAME and DELETE_ITERATOR described in the sections immediately following (Section 5.2.51 on page 173, Section 5.2.52 on page 174, Section 5.2.53 on page 175, Section 5,2.54 on page 176, Section 5.2.55 on page 177, and Section 5.2.56 on page 178). 1

These procedures provide control over the selection of emanating relationships included in the iteration. Relationships that do not match the selection criteria are omitted by subsequent calls to the node iteration interfaces using ITERATOR. The GET_NEXT or SKIP_NEXT interface maybe called exactly once for each relationship emanating from the base node that matches the selection criteria, Since more than one included relationship may terminate at the same target node, a given target node maybe returned more than once in anode iteration by the GET_NEXT interface. 2

A set of eligible relations is specified by RELATION. These relations must be emanating relations defined for the node kind of the base node NODE. Relationships of eligible relations which emanate from the base node NODE are eligible for inclusion in the iteration. 3

In the first and second procedures, RELATION is an open node handle to an RD. This RD defines the only eligible relation. In the third version of the CREATE_ITERATOR interface, RELATION is a relation name pattern. In this case, the eligible relations are those that maybe identified by names matching the relation name pattern. 4

In the first procedure, if the parameter INCLUDE_SPECIALIZATIONS is TRUE, all relationships which are instances of the relation identified by RELATION or instances of any specialization of that relation are eligible for inclusion in the iteration. If INCLUDE_SPECIALIZATIONS is FALSE, only relationships which are instances of the particular relation identified are eligible for inclusion; instances of specializations of that relation are not eligible. 5

In the first procedure, the parameter PREDICATE is an iteration predicate. An eligible relation- 6



Source: https://assist.dla.mil -- Downloaded: 202610-26T07:54Z
Check the source to verify that this is the current version before use.

ship is included in the iteration only if the predicate specified either evaluates to TRUE for that relationship or cannot be evaluated due to an existing lock on the target node, due to insufficient access rights toward the target node, or due to the conditions for TRANSACTION_ERROR which occur if the implicit opening of the target node results in the inclusion of the iteration in more than one transaction. The time at which the predicate is evaluated is implementation-defined; exceptions resulting from predicate evaluation are raised by the GET_NEXT interface.

The default value for the PREDICATE parameter, DEFAULT_PREDICATE, always evaluates 7
to TRUE. The predicate may reference the values of attributes of each relationship or target node and may test for the existence of relationships emanating from each target node (see Section 5.2.47 on page 158).

The ordering of the relationships included in the iteration is implementation-defined when the 8
first procedure is used.

For both the second and third procedures, the set of relationships included in the iteration is 9
restricted by KEY, KIND, and KIND_OF_RELATION as follows:

An included relationship must match the relationship key pattern specified by KEY in the context 10
of an eligible relation for that relationship. KEY must have the syntax of a relationship key list (see Table I on page 52), in which the relationship key values are identifier patterns which may have the wild card characters “?” and “*”. The latest key character “#” may not be used. The KEY may consist of a single relationship key designator or of one or more key value associations.

If KEY is an empty relationship key list (no key values are specified), then all eligible relationships 11
match. If KEY contains a single, unnamed key value pattern, then a relationship matches the KEY parameter in the context of an eligible relation if the relation has only one visible key and the value of that key matches the key value pattern.

If KEY contains one or more key value associations, then a relationship matches the KEY 12
parameter in the context of an eligible relation for that relationship if each key name in KEY unambiguously names a distinct key of that relation whose value matches the corresponding key value pattern. In this case, a key with no value matches a corresponding pattern for that key if the pattern is empty or if the pattern matches the empty string (“”). Hence, the key value pattern “*” matches the value of any key including a key with no value.

Only relationships whose target nodes are of a kind specified in KIND (STRUCTURAL, PRO- 13
CESS or FILE) are included in the iterator. Depending on the value of KIND_OF_RELATION (PRIMARY, SECONDARY, or BOTH), only primary, only secondary or both primary and secondary relationships are included in the iteration respectively.

For the second and third CREATE_ITERATOR procedures, the included relationships and 14
corresponding target nodes are returned by the interfaces in an order based on the identifications of the relationships (the same identifications formed by NEXT_NAME). All relationships and target nodes are returned in ASCII lexicographical order by relation name and then by key value. In this ordering, lower case alphabetic characters are treated as lexicographically equivalent to the corresponding upper case characters.

For relations having more than one key, the order in which keys are named in KEY establishes 15
a (possibly partial) lexicographical ordering of the included relationships by key value. In this lexicographical ordering, the values of each key increase according to ASCII lexicographical order. Furthermore, the values of keys named in KEY increase in the reverse order of their occurrence in KEY, i.e., the value of the last key named increases first. The order in which values of keys not named in KEY increase is implementation-defined, however, these keys increase in value before



Source: https://assist.dla.mil -- Downloaded: 202610-26T07:54Z
Check the source to verify that this is the current version before use.

the keys named in KEY. If a relation name for an included relationship cannot be formed or is ambiguous, the order of occurrence of that relationship in the iteration is implementation-defined.

For all forms of the interface, names are translated and formed as described in Section 5.7.3.6 on page 355. The context for formation of relation names matching the relation name pattern in RELATION, for translation of key names in KEY and for formation of the path elements returned by the NEXT_NAME interface is determined by the base node handle NODE.

If the base node handle NODE has a current adopted interpretation, that interpretation is adopted by the iterator and remains adopted until the iterator is deleted. Names are translated and formed using that adopted interpretation. Adoption of the interpretation by the iterator causes a lock with intent READ_RELATIONSHIPS to be acquired on the VIEW node of the view containing the interpretation. This lock is not released until the iterator is deleted. Changing the current adopted interpretation for the base node handle NODE after the creation of the iterator has no effect on the iteration.

If the base node handle NODE has no adopted interpretation, the composite view is used to translate and form names. In this case, changing the composite view of the current process after creation of the iterator has implementation-defined effects on the ordering of relationships on the iterator and on the formation of path elements returned by NEXT_NAME.

The effect on existing node iterators of the deletion of the base node, or the creation, deletion or conversion of relationships emanating from the base node is implementation-defined.

The TIME_LIMIT parameter applies to the implicit opening of an eligible target node which may be required to evaluate the specified PREDICATE. If the time limit expires while waiting for a lock on an eligible target node in conflict with the required access, the target node is included in the iteration and the GET_NEXT interface will raise LOCK_ERROR for that target node.

Parameters:

**ITERATOR** is an unestablished node iterator to be established.

**NODE** is an open node handle to a node whose emanating relationships form the basis for constructing the iterator.

**KIND** is a specification of the classes of target node kinds included in the iteration.

**KEY** is a relationship key list pattern specifying a pattern which the keys of included relationships must match.

**RELATION** is either an open node handle identifying an RD or a relation name pattern specifying the relations of the eligible relationships.

**KIND_OF_RELATION** is an enumeration value specifying whether primary, secondary or both types of relationships are included.

**PREDICATE** is an iteration predicate which must evaluate to TRUE for each relationship and associated target node included in the iteration.

**INCLUDE_SPECIALIZATIONS** is a Boolean value indication whether relationships belonging to specializations of the relation identified by open node handle RELATION are eligible for inclusion in the iteration.

**TIME_LIMIT** is the maximum time to wait for the unlocking of an eligible target node if the node must be opened to evaluate the iteration predicate and there are existing locks on the node with intents in conflict with the required access.

**STATUS** is an established status indicator.



Source: https://assist.dla.mil -- Downloaded: 202610-26T07:54Z
Check the source to verify that this is the current version before use.

Exceptions:

**ITERATOR_ERROR** is raised if ITERATOR is already established [2]. 31

**NODE_KIND_ERROR** is raised if the open node handle RELATION does not identify an RD [1]. 32

**NOT_IN_VIEW_ERROR** is raised if the iteration predicate specified by PREDICATE contains a name for an AD or an RD which cannot be translated [2], if a key name in the relationship key pattern specified by KEY cannot be translated in the context of an eligible relation [1] or if KEY consists of a pattern for a single relationship key designator and in the context of some eligible relation no key is visible [5]. 33

**AMBIGUOUS_VIEW_ERROR** is raised if translation of a key name in the relationship key pattern KEY is ambiguous in the context of an eligible relation [1] or if KEY consists of a single relationship key designator and in the context of some eligible relation more than one key is visible [4]. 34

**SYNTAX_ERROR** is raised if a pattern given in KEY or RELATION is syntactically illegal (see Table I, page 52 and Section 5.2.49 on page 169) [2] or if the PREDICATE is syntactically illegal (see Section 5.2.47 on page 158) [4]. 35

**STATUS_ERROR** is raised if NODE or RELATION is not an open node handle [2] or if STATUS is not an established status indicator [3]. 36

**RELATIONSHIP_ERROR** is raised if KEY specifies a value for the same key more than once [5] in the context of an eligible relation. 37

**INTENT_VIOLATION** is raised if the node handle NODE was not opened with an intent establishing the right to read relationships [1]. 38

Notes:

The functions PATH_KEY and PATH_RELATION may be used to determine the relationship which caused a node to be included in the iteration. The iteration interfaces can be used to identify relationships to inaccessible or unobtainable nodes. 39

### 5.2.51 Determining node iteration status

```
function MORE
    (ITERATOR : in     NODE_ITERATOR;
     STATUS   : in     STATUS_INDICATOR := NO_INDICATION)
    return BOOLEAN;
```

Purpose:

This function returns FALSE if all nodes contained on the node iterator have been retrieved with the GET_NEXT (see Section 5.2.53 on page 175) procedure or skipped over with the SKIP_NEXT (see Section 5.2.54 on page 176) procedure; otherwise it returns TRUE. 1



Source: https://assist.dla.mil -- Downloaded: 202610-26T07:54Z
Check the source to verify that this is the current version before use.

Parameters:

**ITERATOR** is a node iterator established by the procedure CREATE_ITERATOR (Section 5.2.50 on page 169). 2

**STATUS** is an established status indicator. 3

Exceptions:

**ITERATOR_ERROR** is raised if ITERATOR is not an established iterator (see procedure CREATE_ITERATOR Section 5.2.50 page 169) [1]. 4

**STATUS_ERROR** is raised if STATUS is not an established status indicator [3]. 5

### 5.2.52 Determining the approximate size of the node iterator

```
function APPROXIMATE_SIZE
    (ITERATOR : in     NODE_ITERATOR;
     STATUS   : in     STATUS_ INDICATOR := NO_INDICATION)
    return CAIS_NATURAL;
```

Purpose:

This function returns the approximate number of elements on the iterator at the moment of the call. Calls on GET_NEXT or SKIP_NEXT have no influence on the value returned by this function. 1

Parameters:

**ITERATOR** is a node iterator established by the procedure CREATE_ITERATOR (Section 5.2.50 on page 169). 2

**STATUS** is an established status indicator. 3

Exceptions:

**ITERATOR_ERROR** is raised if ITERATOR is not an established iterator (see procedure CREATE_ITERATOR Section 5.2.50 page 169) [1]. 4

**STATUS_ERROR** is raised if STATUS is not an established status indicator [3]. 5

Notes:

This interface should not be used in loops of the form: 6

```
for I in l . .  APPROXIMATE_SIZE (ITERATOR) loop
    GET_NEXT (ITERATOR, NEXT_NODE, (1 => READ));
end loop;
```

since the deletion of relationships may reduce the number of node handles returned by the repeated calls on GET_NEXT.



Source: https://assist.dla.mil -- Downloaded: 202610-26T07:54Z
Check the source to verify that this is the current version before use.

### 5.2.53 Getting the next node in a node iteration

```
procedure GET_NEXT
    (ITERATOR    : in out NODE_ITERATOR;
     NEXT_NODE   : in out MODE_TYPE;
     INTENT      : in     INTENT_ARRAY     := (1 => NO_ACCESS);
     TIME_LIMIT  : in     CAIS_DURATION    := LONG_DELAY;
     STATUS      : in     STATUS_INDICATOR := NO_INDICATION);
```

Purpose:

This procedure returns an open node handle in NEXT_NODE to the next target node on ITERATOR; the intent under which the node handle is opened is specified by INTENT. If NEXT_NODE is open prior to the call to GET_NEXT, it is closed prior to being opened to the next node. A time limit can be specified for the maximum delay permitted if the node to be opened is locked against access with the specified INTENT. If an exception is raised by the call on GET_NEXT, the next call on GET_NEXT for the same iterator will attempt to return an open node handle to the same node, i.e., the iterator is not advanced by a call resulting in an exception.

Parameters:

**ITERATOR** is a node iterator established by the procedure CREATE_ITERATOR (see Section 5.2.50 on page 169).

**NEXT_NODE** is a node handle to be opened to the next target node on ITERATOR.

**INTENT** is the intent of subsequent operations on the node handle NEXT_NODE.

**TIME_LIMIT** is the maximum time to wait for the unlocking of the next target node in accordance with the desired INTENT.

**STATUS** is an established status indicator.

Exceptions:

**NAME_ERROR** is raised if the iteration predicate could not be evaluated for the next node because access to that node was required and the node was unobtainable or inaccessible [3] or if the next node is unobtainable or inaccessible and the INTENT includes any intent other than NO_ACCESS [1].

**ITERATOR_ERROR** is raised if ITERATOR is not an established iterator (see procedure CREATE_ITERATOR Section 5.2.50) [1], or if ITERATOR is exhausted (i.e., the result of MORE(ITERATOR) is FALSE) [3].

**STATUS_ERROR** is raised if STATUS is not an established status indicator [3].

**USE_ERROR** is raised if INTENT is an empty array [1].

**TRANSACTION_ERROR** is raised if the operation is included in more than one transaction due to the inclusion of the base node, or another target node previously accessed to evaluate the iteration predicate, and the current target node in different transactions [1].


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

**LOCK_ERROR** is raised if the iteration predicate could not be evaluated for the next node because opening of the next node was delayed beyond the time limit specified at iterator creation due to existing locks in conflict with the required access [2] or if opening of the next node was delayed beyond the specified TIME_LIMIT due to existing locks in conflict with the specified INTENT [1]. LOCK_ERROR may be raised prior to expiration of the time limit if the CAIS implementation can determine that a deadlock situation has occurred. 12

**ACCESS_VIOLATION** is raised if the iteration predicate could not be evaluated for the next node because the discretionary access rights of the current process to that node were not sufficient for the required access [4] or if the discretionary access rights of the current process to the next node are not sufficient to obtain access to that node with the specified INTENT [1]. 13

**SECURITY_VIOLATION** is raised if the iteration predicate could not be evaluated for the next node because the required access to that node by the current process represented a violation of mandatory access controls or if the attempt by the current process to obtain access to the next node with the specified INTENT represents a violation of mandatory access controls. 14

Notes:

Since more than one included relationship may terminate at the same target node, a given target node may be returned more than once by GET_NEXT in a node iteration. 15

### 5.2.54 Skipping the next node in a node iteration

```
procedure SKIP_NEXT
    (ITERATOR : in out NODE_ITERATOR;
     STATUS   : in     STATUS_INDICATOR := NO_INDICATION);
```

Purpose:

This procedure advances the iterator to the next node on the iterator without returning an open handle to this node. 1

Parameters:

**ITERATOR** is a node iterator established by the procedure CREATE_ITERATOR (see Section 5.2.50 on page 169). 2

**STATUS** is an established status indicator. 3

Exceptions:

**ITERATOR_ERROR** is raised if ITERATOR not an established iterator (see procedure CREATE_ITERATOR Section 5.2.50 on page 169) [1], or if ITERATOR is exhausted (i.e., the result of MORE(ITERATOR) is FALSE) [3]. 4

**STATUS_ERROR** is raised if STATUS is not an established status indicator [3]. 5


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Notes:

This procedure can be used to advance the iterator across a node for which GET_NEXT resulted in an exception. 6

### 5.2.55 Obtaining the path element for the next node in a node iteration

```
function NEXT_NAME
    (ITERATOR : in     NODE_ITERATOR;
     STATUS   : in     STATUS_INDICATOR := NO_INDICATION)
    return PATHNAME;
```

Purpose:

This function returns a path element identifying the current relationship of the ITERATOR relative to the base node for the iteration. The returned value has the syntax of a path element (see Table I, page 52); it can be submitted to LAST_KEY and LAST_RELATION to obtain the constituent values. The path element identifies the relationship that is traversed by GET_NEXT to reach the next target node included in the iteration starting at the base node. 1

The iterator is not advanced by this call, i.e., it is possible to call GET_NEXT to obtain a node handle for this node or to call SKIP_NEXT to advance the iterator to the next node. 2

The path element is formed according to the semantics of name formation using views as described in Section 5.7.3.6 on page 355. If the base node had a current adopted interpretation at the time of iterator creation, the path element is formed in the context of that interpretation (see Section 5.2.50 on page 169). Otherwise, the path element is formed in the context of the base node using the composite view of the current process. In the latter case, changes to the composite view after the iterator is created have implementation-defined effects on the formation of path elements. 3

Parameters:

**ITERATOR** is a node iterator established by the procedure CREATE_ITERATOR (page 169). 4

**STATUS** is an established status indicator. 5

Exceptions:

**NOT_IN_VIEW_ERROR** is raised if a path element cannot be formed for the relationship to the next node on ITERATOR [3]. 6

**AMBIGUOUS_VIEW_ERROR** is raised if formation of the path element is ambiguous [2] or if translation of the path element formed is ambiguous [1]. 7

**ITERATOR_ERROR** is raised if ITERATOR is not an established iterator (see procedure CREATE_ITERATOR Section 5.2.50 on page 169) [1], or if ITERATOR is exhausted (i.e., the value of MORE(ITERATOR) is FALSE) [3]. 8

**STATUS_ERROR** is raised if STATUS is not an established status indicator [3]. 9


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

### 5.2.56 Deleting a node iterator

```
procedure DELETE_ITERATOR
    (ITERATOR : in out NODE_ITERATOR;
     STATUS   : in     STATUS_INDICATOR := NO_INDICATION);
```

Purpose:

This procedure deletes anode iterator. The value of ITERATOR after the call is unestablished, as if it were never established by CREATE_ITERATOR (see Section 5.2.50 on page 169). Deleting an iterator that is not established has no effect. 1

Parameters:

**ITERATOR** is anode iterator. 2

**STATUS** is an established status indicator. 3

Exceptions:

**STATUS_ERROR** is raised if STATUS is not unestablished status indicator [3].

### 5.2.57 Node kind instance iteration type

The following type is used in the interfaces for iterating over a set of nodes which are instances of a specified NKD. 1

```
type NODE_KIND_INSTANCE_ITERATOR is limited private;
```

A *node kind instance iterator* is an Ada object of the type NODE_KIND_INSTANCE_ITERATOR, which is a limited private type assumed to contain the bookkeeping information necessary for the implementation of the MORE, GET_NEXT, and SKIP_NEXT interfaces. The CREATE_ITERATOR interface is used to create a node kind instance iterator. 2

### 5.2.58 Creating an iterator over instances of a node kind definition

```
procedure CREATE_ITERATOR
    (ITERATOR                 : in out NODE_KIND_INSTANCE_ITERATOR;
     NODE_KIND                :  in    NKD_TYPE;
     PREDICATE                :  in    ITERATION_PREDICATE := DEFAULT_PREDICATE;
     IMCLUDE_SPECIALIZATIONS  : in     BOOLEAN             := TRUE;
     TIME_LIMIT               :  in    CAIS_DURATION       := LONG_DELAY;
     STATUS                   :  in    STATUS_INDICATOR    := NO_INDICATION);
```


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Purpose:

This procedure establishes the node kind instance iterator ITERATOR over the set of nodes that are instances of the NKD identified by NODE_KIND and (optionally) the instances of the specializations of that NKD. The iterator can then be processed by means of the node kind instance iteration interfaces MORE, APPROXIMATE_SIZE, GET_NEXT, SKIP_NEXT and DELETE_ITERATOR described in the sections immediately following this section (see Section 5.2.59 on page 180, Section 5.2.60 on page 181, Section 5.2.61 on page 181, and Section 5.2.62 on page 183, and Section 5.2.63 on page 184). 1

Instances of the NKD identified by NODE_KIND are eligible for inclusion in the iteration. If INCLUDE_SPECIALIZATIONS is TRUE, instances of all specializations of NODE_KIND are also eligible for inclusion. 2

Eligible instances (nodes) which are unobtainable or inaccessible to the current process are not included in the iteration. Otherwise, an eligible node is included in the iteration only if the iteration predicate specified by the PREDICATE parameter either evaluates to TRUE for the node or cannot be evaluated. Failure of predicate evaluation may occur due to an existing lock on the node, due to insufficient access rights toward the node or due to the conditions for TRANSACTION_ERROR. The time at which the predicate is evaluated is implementation-defined; an exception is raised by the GET_NEXT interface if the predicate cannot be evaluated for a node. 3

The default value for the PREDICATE parameter, DEFAULT_PREDICATE, always evaluates to TRUE. The predicate may reference the values of attributes of each node and may test for the existence of emanating relationships (see Section 5.2.47 on page 158). 4

The order in which instances (nodes) are returned by the node kind instance iteration interfaces (e.g., GET_NEXT) is implementation-defined. 5

The effect on existing node kind instance iterators of the creation, deletion or modifications of nodes of the specified node kind, or the conversion of nodes to or from the specified node kind is implementation-defined. 6

The TIME_LIMIT parameter applies to the implicit opening of an eligible node which may be required to evaluate the specified PREDICATE. If the time limit expires while waiting for a lock on an eligible node in conflict with the required access, the node is included in the iteration and the GET_NEXT interface will raise LOCK_ERROR for that node. 7

Parameters:

**ITERATOR** is an unestablished node kind instance iterator to be established. 8

**NODE_KIND** is an open node handle to an NKD which is the basis for constructing the iterator. 9

**PREDICATE** is an iteration predicate which must evaluate to TRUE for each node included in the iteration. 10

**INCLUDE_SPECIALIZATIONS** is a Boolean value indicating whether the instances of specializations of NODE_KIND are eligible for inclusion in the iteration. 11

**TIME_LIMIT** is the maximum time to wait for the unlocking of an eligible node if the node must be opened to evaluate the iteration predicate and there are existing locks with intents in conflict with the required access. 12

**STATUS** is an established status indicator. 13


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Exceptions:

**ITERATOR_ERROR** is raised if ITERATOR is already established [2].

**SYNTAX_ERROR** is raised if the PREDICATE is syntactically illegal [4] or uses the RELATIONSHIP_ATTRIBUTE function (see Section 5.2.47.8 on page 165 or the IS_PRIMARY function (see Section 5.2.47.12 on page 168) [5].

**STATUS_ERROR** is raised if NODE_KIND is not an open node handle or if STATUS is not an established status indicator [3].

**NODE_KIND_ERROR** is raised if NODE-KIND is not a node handle to an NKD [1].

**NOT_IN_VIEW_ERROR** is raised if the iteration predicate specified by PREDICATE contains a name for an AD or an RD which cannot be translated [2].

Notes:

The function HAS_EMANATING may be used in the iteration predicate to locate base nodes of the specified KIND which have emanating relationships belonging to a specified relation. In particular, if the specified KIND is the basic NKD, all base nodes for relationships of a given relation may be located. A nested node iteration may be applied to each base node in turn to iterate over these relationships.

### 5.2.59 Determining node kind instance iteration status

```
function MORE
    (ITERATOR : in     NODE_KIND_INSTANCE_ITERATOR;
     STATUS   : in     STATUS_INDICATOR := NO_INDICATION)
    return BOOLEAN;
```

Purpose:

This function returns FALSE if all instances included in the iteration identified by ITERATOR have been retrieved with the GET_NEXT (see Section 5.2.61 on page 181) procedure or skipped over with the SKIP_NEXT (see Section 5.2.62 on page 183) procedure; otherwise it returns TRUE.

Parameters:

**ITERATOR** is a node kind instance iterator established by the procedure CREATE_ITERATOR (Section 5.2.58 on page 178).

**STATUS** is an established status indicator.

Exceptions:

**ITERATOR_ERROR** is raised if ITERATOR is not an established iterator (see procedure CREATE_ITERATOR Section 5.2.58 on page 178) [1].

**STATUS_ERROR** is raised if STATUS is not an established status indicator [3].



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

### 5.2.60 Determining the approximate size of the node kind instance iterator

```
function APPROXIMATE_SIZE
    (ITERATOR : in     NODE_KIND_INSTANCE_ITERATOR;
     STATUS   : in     STATUS_INDICATOR := NO_INDICATION)
    return CAIS_NATURAL;
```

Purpose:

This function returns the approximate number of nodes on the iterator at the moment of the call. Calls on GET_NEXT or SKIP_NEXT have no influence on the value returned by this function.

Parameters:

**ITERATOR** is a node kind instance iterator established by the procedure CREATE_ITERATOR (see Section 5.2.58 on page 178).

**STATUS** is an established status indicator.

Exceptions:

**ITERATOR_ERROR** is raised if ITERATOR is not an established iterator (see procedure CREATE_ITERATOR Section 5.2.58 on page 178) [1].

**STATUS_ERROR** is raised if STATUS is not an established status indicator [3].

Notes:

This interface should not be used in loops of the form:

```
for I in 1 . . APPROXIMATE_SIZE (ITERATOR) loop
    GET_NEXT (ITERATOR, NEXT_NODE, (1 => READ));
end loop;
```

since the deletion of nodes may reduce the number of node handles returned by the repeated calls on GET_NEXT.

### 5.2.61 Getting the next node in a node kind instance iteration

```
procedure GET_NEXT
    (ITERATOR  : in out NODE_KIND_INSTANCE_ITERATOR;
     NEXT_MODE : in out NODE_TYPE;
     INTENT    : in     INTENT_ARRAY    := (1 => NO_ACCESS);
     TIME_LIMIT : in    CAIS_DURATION   := LONG_DELAY;
     STATUS    : in     STATUS_INDICATOR := NO_INDICATION);
```


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Purpose:

This procedure returns an open node handle in NEXT_NODE to the next node on ITERATOR; the intents under which the node handle is opened are specified by INTENT. If NEXT_NODE is open prior to the call to GET_NEXT, it is closed prior to being opened to the next node. A time limit can be specified for the maximum delay permitted if the node to be opened is locked against access with the specified INTENT. If an exception is raised by the call on GET_NEXT, the next call on GET_NEXT for the same iterator will attempt to return an open node handle to the same node, i.e., the iterator is not advanced by a call resulting in an exception.

Nodes which are unobtainable or inaccessible are not included in the iteration.

No path is traversed in opening the handle NEXT_NODE; hence, the subprograms PATH_KEY (Section 5.2.20 on page 119) and PATH_RELATION (Section 5.2.21 on page 120) cannot be applied to the node handle NEXT_NODE.

Parameters:

**ITERATOR** is a node kind instance iterator established by the procedure CREATE_ITERATOR (see Section 5.2.58 on page 178).

**NEXT_NODE** is a node handle to be opened to the next node on the ITERATOR.

**INTENT** is the intent of subsequent operations on the node handle NEXT_NODE.

**TIME_LIMIT** is the maximum time to wait for the unlocking of the next node in accordance with the desired INTENT.

**STATUS** is an established status indicator.

Exceptions:

**ITERATOR_ERROR** is raised if ITERATOR is not an established iterator (see procedure CREATE_ITERATOR Section 5.2.58 on page 178) [1], or if ITERATOR is exhausted (i.e., the result of MORE(ITERATOR) is FALSE) [3].

**STATUS_ERROR** is raised if STATUS is not an established status indicator [3].

**USE_ERROR** is raised if INTENT is an empty array [1].

**TRANSACTION_ERROR** is raised if the operation is included in more than one transaction due to the inclusion of an NKD on which the iteration is based, or another eligible node previously acessed to evaluate the iteration predicate, and the current eligible node in different transactions [1].

**LOCK_ERROR** is raised if the iteration predicate could not be evaluated for the next node because opening of the next node was delayed beyond the time limit specified at iterator creation due to existing locks in conflict with the required access [2]; if opening of the next node was delayed beyond the specified TIME_LIMIT due to existing locks in conflict with the specified INTENT [1]; or if an open transaction not containing this operation contains an NKD to which the implementation requires access in order iterate over its instances [3]. LOCK_ERROR may be raised prior to expiration of the time limit if the CAIS implementation can determine that a deadlock situation has occurred.


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

**ACCESS_VIOLATION** is raised if the iteration predicate could not be evaluated for the next node because the discretionary access rights of the current process to that node were not sufficient for the required access [4] or if the discretionary access rights of the current process to the next node are not sufficient to obtain access to that node with the specified INTENT [1]. ACCESS_VIOLATION is not raised for nodes which are unobtainable or inaccessible since such nodes are not included in the iteration.

**SECURITY_VIOLATION** is raised if the iteration predicate could not be evaluated for the next node because the required access to that node by the current process represented a violation of mandatory access controls or if the attempt by the current process to obtain access to the next node with the specified INTENT represents a violation of mandatory access controls.

Notes:

When a node is opened using GET_NEXT for a node kind instance iterator, the node is opened without traversing any relationships or checking of access control rights to any other node and no path is associated with the open node handle returned.

On a node kind instance iterator, GET_NEXT cannot be used to obtain handles to inaccessible or unobtainable nodes since these nodes are not included in the iteration.

### 5.2.62 Skipping the next node in a node kind instance iteration

```
procedure SKIP_NEXT
    (ITERATOR : in out NODE_KIND_INSTANCE_ITERATOR;
     STATUS   : in     STATUS_INDICATOR := NO_INDICATION);
```

Purpose:

This procedure advances ITERATOR to the next node on the iterator without returning an open handle to this node.

Parameters:

**ITERATOR** is a node kind instance iterator established by the procedure CREATE_ITERATOR (see Section 5.2.58 on page 178).

**STATUS** is an established status indicator.

Exceptions:

**ITERATOR_ERROR** is raised if ITERATOR is not an established iterator (see procedure CREATE_ITERATOR in Section 5.2.58 on page 178) [1], or if ITERATOR is exhausted (i.e., the result of MORE(ITERATOR) is FALSE) [3],

**STATUS_ERROR** is raised if STATUS is not an established status indicator [3].


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Notes:

This procedure can be used to advance the iterator across anode for which GET_NEXT resulted in an exception. 6

### 5.2.63 Deleting a node kind instance iterator

```
procedure DELETE_ITERATOR
    (ITERATOR : in out NODE_KIND_INSTANCE_ITERATOR;
     STATUS   : in     STATUS_INDICATOR := NO_INDICATION);
```

Purpose:

This procedure deletes anode kind instance iterator. The value of ITERATOR after the call is unestablished, as if it were never established by CREATE_ITERATOR (see Section 5.2.58 on page 178). Deleting an iterator that is not established has no effect. 1

Parameters:

**ITERATOR** is anode kind instance iterator. 2

**STATUS** is an established status indicator. 3

Exceptions:

**STATUS_ERROR** is raised if STATUS is not an established status indicator [3]. 4



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

## 5.3 Package CAIS_ATTRIBUTE_MANAGEMENT

This package supports the manipulation of attributes of nodes and relationships. The exceptions raised by all subprograms in this package are defined in the packages CAIS_DEFINITIONS, CAIS_-LIST_MANAGEMENT and CAIS_PRAGMATICS. 1

The name of an attribute follows the syntax of an Ada identifier. Upper and lower case distinctions are not significant when comparing attribute names. The value of an attribute must conform to the AKD associated with the AD for that attribute (see Section 5.7.1.6 on page 315) and any constraints that apply to that AKD in the context in which the attribute occurs. 2

The values of attributes maybe manipulated as lists; the format of lists is defined by the package CAIS_-LIST_MANAGEMENT (see Section 9.1 on page 751). The attribute kinds integer, float, string and identifier are called simple attribute kinds. When manipulated as a list, an attribute value conforming to a simple attribute kind is represented by a unnamed list consisting of a single item of the same kind. Other kinds of attributes, namely list, uniform list, and composite list, are called structured attribute kinds. Values conforming to structured attribute kinds are represented by named or unnamed lists which may contain zero or more items. The kind of a value conforming to a variant attribute kind is the attribute kind of the alternative to which the value conforms; this kind may be either simple or structured. See Section 5.7.1.6 on page 315 for a detailed description of attribute kinds and their list representations. 3

The values of structured attributes are always manipulated as lists. Package CAIS_ATTRIBUTE_-MANAGEMENT provides interfaces for manipulating any kind of attribute value in the form of a list. In addition, this package contains sub-packages which manipulate simple attribute values directly, without the use of lists. The sub-packages for manipulating integer values (CAIS_INTEGER_-ATTRIBUTE), floating values (CAIS_FLOAT_ATTRIBUTE) and identifier values as values of an Ada enumeration type (CAIS_ENUMERATION_ATTRIBUTE) are generic. An instantiation of CAIS_-ENUMERATION_ ATTRIBUTE for the Ada type BOOLEAN is provided as the library package CAIS_-BOOLEAN_ATTRIBUTE. The sub-packages CAIS_STRING_ATTRIBUTE and CAIS_IDENTIFIER_-ATTRIBUTE manipulate string values and identifier values in token form, respectively. The attribute values returned by an attribute iterator (see Section 5.3.15 on page 245) are always returned in the form of lists. 4

While there are special interfaces to retrieve the values of certain predefine attributes in the various CAIS packages, such values can also be retrieved by the general attribute manipulation interfaces defined in the package CAIS_ATTRIBUTE_MANAGEMENT. It can be reasonably assumed that it will not be possible to implement the latter retrievals as efficiently as the former ones. 5

Predefine attributes are defined in Appendix A. The values of predefine attributes conform to these definitions and are represented accordingly, as described in Section 5.7.1.6 on page 315. The values of attributes which are CAIS-controlled (see Section 5.7.2.1 on page 342) cannot be created, deleted or modified by the user using the interfaces defined in package CAIS_ATTRIBUTE_MANAGEMENT. 6

The operations defined for the manipulation of attributes identify the node to which an attribute belongs using an open node handle. They identify the relationship to which an attribute belongs by an open node handle to a base node, a relationship key list and an open node handle to an RD or a relation name. Attributes are identified by an attribute name or an open node handle to an AD. The operations may be used to create and modify the key attributes of relationships if the uniqueness constraints applying to the keys are satisfied by the new values (see Section 5.7.1.4 on page 310). Keys may also be manipulated using the RENAME interface (see Section 5.2.31 on page 134). 7


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

### 5.3.1 Creating node attributes

```
procedure CREATE_NODE_ATTRIBUTE
    (NODE      : in    NODE_TYPE;
     ATTRIBUTE : in    AD_TYPE;
     VALUE     : in    LIST_TYPE;
     STATUS    : in    STATUS_INDICATOR := NO_INDICATION);
```

Purpose:

This procedure creates the attribute identified by ATTRIBUTE of the node identified by NODE and sets its initial value to VALUE. 1

Parameters:

**NODE** is an open node handle to the node to receive the new attribute. 2

**ATTRIBUTE** is an open node handle to the AD which identifies the attribute to be created. 3

**VALUE** is the initial value of the attribute in the form of a list. 4

**STATUS** is an established status indicator. 5

Exceptions:

**PREDEFINED_ATTRIBUTE_ERROR** is raised if ATTRIBUTE identifies a CAIS-controlled node attribute [1]. 6

**ATTRIBUTE_ERROR** is raised if the node already has the specified attribute [l]. 7

**STATUS_ERROR** is raised if either of the node handles NODE or ATTRIBUTE is not open [2] or if STATUS is not an established status indicator [3]. 8

**NODE_KIND_ERROR** is raised if the open node handle ATTRIBUTE does not designate an AD [1]. 9

**CONFORMANCE_ERROR** is raised if the operation would result in an attribute, node or relationship not conforming to its definition or the constraints in force on that definition [4, 7, 8, 9, 10]. 10

**INTENT_VIOLATION** is raised if NODE was not opened with an intent establishing the right to create attributes [1]. 11

**SECURITY_VIOLATION** is raised if the operation represents a violation of mandatory access controls. SECURITY_VIOLATION is raised only if the conditions for other exceptions are not present. 12

Additional Interfaces:

```
procedure CREATE_NODE_ATTRIBUTE
    (NODE      : in    NODE_TYPE;
     ATTRIBUTE : in    ATTRIBUTE_NAME;
     VALUE     : in    LIST_TYPE;
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
        STATUS      : in      STATUS_INDICATOR  := NO_INDICATION)
    is
        ATTRIBUTE_DEFINITION : AD_TYPE;
    begin
        DEFINITION_OF_NODE_ATTRIBUTE (ATTRIBUTE_DEFINITION,
                                      NODE, ATTRIBUTE, STATUS);
        CREATE_NODE_ATTRIBUTE (NODE, ATTRIBUTE_DEFINITION, VALUE, STATUS);
        CLOSE  (ATTRIBUTE_DEFINITION);
    exception
        when  others  =>
            CLOSE  (ATTRIBUTE_DEFINITION);
            raise;
    end  CREATE_NODE_ATTRIBUTE;
```

### 5.3.2 Creating relationships attributes

```
procedure  CREATE_PATH_ATTRIBUTE
    (BASE       : in      NODE_TYPE;
     KEY        : in      RELATIONSHIP_KEY  := EMPTY_KEY_LIST;
     RELATION   : in      RD_TYPE;
     ATTRIBUTE  : in      AD_TYPE;
     VALUE      : in      LIST_TYPE;
     STATUS     : in      STATUS_INDICATOR  := NO_INDICATION);
```

Purpose:

This procedure creates the attribute identified by the ATTRIBUTE of the relationship identified by BASE, KEY and RELATION and sets its initial value to VALUE.

Parameters:

**BASE** is an open node handle to the node from which the relationship emanates.

**KEY** is the relationship key list of the affected relationship.

**RELATION** is an open node handle to an RD which identifies a relation of the affected relationship.

**ATTRIBUTE** is an open node handle to an AD which identifies the attribute to be created.

**VALUE** is the initial value of the attribute in the form of a list.

**STATUS** is an established status indicator.

Exceptions:

**PATHNAME_SYNTAX_ERROR** is raised if the relationship identification given by KEY is syntactically illegal (see Table I, page 52) [2].

**NOT_IN_VIEW_ERROR** is raised if a key name specified in KEY cannot be translated [1] or if KEY consists of a single relationship key designator and no key of the specified relation is visible [5].



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

**AMBIGUOUS_VIEW_ERROR** is raised if translation of a key name specified in KEY is ambiguous [1] or if KEY consists of a single relationship key designator and more than one key of the specified relation is visible [4]. 10

**RELATIONSHIP_ERROR** is raised if KEY specifies a value for the same key more than once [5], if a relationship identified by BASE, KEY and RELATION does not exist [2], if more than one such relationship exists [1] or if ATTRIBUTE identifies a key and another relationship is identified by the resulting key values [3]. 11

**PREDEFINED_ATTRIBUTE_ERROR** is raised if ATTRIBUTE identifies a CAIS-controlled relationship attribute [2]. 12

**ATTRIBUTE_ERROR** is raised if the relationship already has an attribute identified by ATTRIBUTE [2]. 13

**STATUS_ERROR** is raised if any of the node handles BASE, RELATION, or ATTRIBUTE is not open [2] or if STATUS is not an established status indicator [3]. 14

**NODE_KIND_ERROR** is raised if the open node handle RELATION does not designate an RD or if the open node handle ATTRIBUTE does not designate an AD [1]. 15

**CONFORMANCE_ERROR** is raised if the operation would result in an attribute, node or relationship not conforming to its definition or the constraints in force on that definition [4, 7, 8, 9, 10]. 16

**INTENT_VIOLATION** is raised if BASE was not opened with an intent establishing the right to write relationships [1]. 17

**SECURITY_VIOLATION** is raised if the operation represents a violation of mandatory access controls, SECURITY_VIOLATION is raised only if the conditions for other exceptions are not present. 18

Additional Interfaces:

```
procedure CREATE_PATH_ATTRIBUTE
    (BASE      : in  NODE_TYPE;
     KEY       : in  RELATIONSHIP_KEY := EMPTY_KEY_LIST;
     RELATION  : in  RELATION_NAME := DEFAULT_RELATION;
     ATTRIBUTE : in  ATTRIBUTE_NAME;
     VALUE     : in  LIST_TYPE;
     STATUS    : in  STATUS_INDICATOR := NO_INDICATION)
is
    RELATION_DEFINITION  : RD_TYPE;
    ATTRIBUTE_DEFINITION : AD_TYPE;
begin
    DEFINITION_OF_RELATION (RELATION_DEFINITION, BASE, RELATION, STATUS);
    DEFINITION_OF_PATH_ATTRIBUTE (ATTRIBUTE_DEFINITION,
                                  RELATION_DEFINITION,
                                  ATTRIBUTE, STATUS);
    CREATE_PATH_ATTRIBUTE (BASE, KEY, RELATION_DEFINITION,
                           ATTRIBUTE_DEFINITION, VALUE, STATUS);
    CLOSE (RELATION_DEFINITION);
    CLOSE (ATTRIBUTE_Definition);
exception
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
        when others =>
            CLOSE (RELATION_DEFINITION);
            CLOSE (ATTRIBUTE_DEFINITION);
            raise;
    end CREATE_PATH_ATTRIBUTE;
```

### 5.3.3 Deleting node attributes

```
procedure DELETE_NODE_ATTRIBUTE
    (NODE      : in  NODE_TYPE;
     ATTRIBUTE : in  AD_TYPE;
     STATUS    : in  STATUS_INDICATOR := NO_INDICATION);
```

Purpose:

This procedure deletes the attribute identified by ATTRIBUTE of the node identified by NODE. 1

Parameters:

**NODE** is an open node handle to the node whose attribute is to be deleted. 2

**ATTRIBUTE** is an open node handle to the AD which identifies the attribute to be deleted. 3

**STATUS** is an established status indicator. 4

Exceptions:

**PREDEFINED_ATTRIBUTE_ERROR** is raised if ATTRIBUTE identifies a CAIS-controlled node attribute [1]. 5

**ATTRIBUTE_ERROR** is raised if the node does not have the attribute identified by ATTRIBUTE [3]. 6

**CONFORMANCE_ERROR** is raised if ATTRIBUTE identifies an attribute which is not deletable [6]. 7

**STATUS_ERROR** is raised if the node handle NODE or ATTRIBUTE is not open [2] or if STATUS is not an established status indicator [3]. 8

**NODE_KIND_ERROR** is raised if the open node handle ATTRIBUTE does not designate an AD [1]. 9

**INTENT_VIOLATION** is raised if NODE was not opened with an intent establishing the right to write attributes [1]. 10

**SECURITY_VIOLATION** is raised if the operation represents a violation of mandatory access controls. SECURITY_VIOLATION is raised only if the conditions for other exceptions are not present. 11


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Additional Interfaces:

```
procedure DELETE_NODE_ATTRIBUTE
    (NODE      : in  NODE_TYPE;
     ATTRIBUTE : in  ATTRIBUTE_NAME;
     STATUS    : in  STATUS_INDICATOR := NO_INDICATION)
is
    ATTRIBUTE_DEFINITION : AD_TYPE;
begin
    DEFINITION_OF_NODE_ATTRIBUTE (ATTRIBUTE_DEFINITION,
                                  NODE, ATTRIBUTE, STATUS);
    DELETE_NODE_ATTRIBUTE (NODE, ATTRIBUTE_DEFINITION, STATUS);
    CLOSE (ATTRIBUTE_DEFINITION);
exception
    when others =>
        CLOSE (ATTRIBUTE_DEFINITION);
        raise;
end DELETE_NODE_ATTRIBUTE;
```

### 5.3.4 Deleting relationship attributes

```
procedure DELETE_PATH_ATTRIBUTE
    (BASE      : in  NODE_TYPE;
     KEY       : in  RELATIONSHIP_KEY := EMPTY_KEY_LIST;
     RELATION  : in  RD_TYPE;
     ATTRIBUTE : in  AD_TYPE;
     STATUS    : in  STATUS_INDICATOR := NO_INDICATION);
```

Purpose:

This procedure deletes the attribute identified by ATTRIBUTE of the relationship identified by BASE, KEY and RELATION. 1

Parameters:

**BASE** is an open node handle to the node from which the relationship emanates. 2

**KEY** is the relationship key list of the affected relationship. 3

**RELATION** is an open node handle to an RD which identifies the relation of the affected relationship. 4

**ATTRIBUTE** is an open node handle to the AD which identifies the attribute to be deleted. 5

**STATUS** is an established status indicator. 6

Exceptions:

**PATHNAME_SYNTAX_ERROR** is raised if the relationship identification given by KEY is syntactically illegal (see Table I, page 52) [2]. 7



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

**NOT_IN_VIEW_ERROR** is raised if a key name specified in KEY cannot be translated [1] or if KEY consists of a single relationship key designator and no key of the specified relation is visible [5]. 8

**AMBIGUOUS_VIEW_ERROR** is raised if translation of a key name specified in KEY is ambiguous [1] or if KEY consists of a single relationship key designator and more than one key of the specified relation is visible [4]. 9

**RELATIONSHIP_ERROR** is raised if KEY specifies a value for the same key more than once [5], if a relationship identified by BASE, KEY and RELATION does not exist [2], if more than one such relationship exists [1] or if ATTRIBUTE identifies a key and another relationship is identified by the resulting key values [3]. 10

**PREDEFINED_ATTRIBUTE_ERROR** is raised if ATTRIBUTE identifies a CAIS-controlled relationship attribute [2]. 11

**ATTRIBUTE_ERROR** is raised if the relationship does not have the attribute identified by ATTRIBUTE [4]. 12

**CONFORMANCE_ERROR** is raised if ATTRIBUTE identifies an attribute which is not deletable [6]. 13

**STATUS_ERROR** is raised if any of the node handles BASE, RELATION, or ATTRIBUTE is not open [2] or if STATUS is not an established status indicator [3]. 14

**NODE_KIND_ERROR** is raised if the open node handle RELATION does not designate an RD or if the open node handle ATTRIBUTE does not designate an AD [1]. 15

**INTENT_VIOLATION** is raised if BASE was not opened with an intent establishing the right to write relationships [1]. 16

**SECURITY_VIOLATION** is raised if the operation represents a violation of mandatory access controls. SECURITY_VIOLATION is raised only if the conditions for other exceptions are not present. 17

Additional Interfaces:

```
procedure DELETE. PATH. ATTRIBUTE
    (BASE      : in    NODE_TYPE;
     KEY       : in    RELATIONSHIP_KEY : = EMPTY_KEY_LIST;
     RELATION  : in    RELATION_NAME : = DEFAULT_RELATION;
     ATTRIBUTE : in    ATTRIBUTE_NAME;
     STATUS    : in    STATUS_INDICATOR : = NO_INDICATION)
is
     RELATION_DEFINITION : RD_TYPE;
     ATTRIBUTE_DEFINITION : AD_TYPE;
begin
     DEFINITION_OF_RELATION (RELATION_DEFINITION, BASE, RELATION, STATUS);
     DEFINITION_OF_PATH_ATTRIBUTE (ATTRIBUTE_DEFINITION,
                                   RELATION_DEFINITION,
                                   ATTRIBUTE, STATUS);
     DELETE_PATH_ATTRIBUTE (BASE , KEY, RELATION_DEFINITION,
                            ATTRIBUTE_DEFINITION, STATUS);
     CLOSE (RELATION_DEFINITION);
```


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
        CLOSE (ATTRIBUTE_DEFINITION);
    exception
        when others =>
            CLOSE (RELATION_DEFINITION);
            CLOSE (ATTRIBUTE_DEFINITION);
            raise;
    end DELETE_PATH_ATTRIBUTE;
```

### 5.3.5 Setting node attributes

```
procedure SET_NODE_ATTRIBUTE
    (NODE      : in  NODE_TYPE;
     ATTRIBUTE : in  AD_TYPE;
     VALUE     : in  LIST_TYPE;
     STATUS    : in  STATUS_INDICATOR := NO_INDICATION);
```

Purpose:

This procedure sets the value of the attribute identified by ATTRIBUTE of the node identified by NODE to the value given by VALUE.

Parameters:

**NODE** is an open node handle to the node whose attribute is to beset.

**ATTRIBUTE** is an open node handle to the AD which identifies the attribute to beset.

**VALUE** is the new value of the attribute in the form of a list.

**STATUS** is an established status indicator.

Exceptions:

**PREDEFINED_ATTRIBUTE_ERROR** is raised if ATTRIBUTE identifies a CAIS-controlled node attribute [1].

**ATTRIBUTE_ERROR** is raised if the node does not have the attribute identified by ATTRIBUTE [3].

**STATUS_ERROR** is raised if either of the node handles NODE or ATTRIBUTE is not open [2] or if STATUS is not an established status indicator [3].

**NODE_KIND_ERROR** is raised if the open node handle ATTRIBUTE does not designate an AD [1].

**CONFORMANCE_ERROR** is raised if the operation would result in an attribute, node or relationship not conforming to its definition or the constraints in force on that definition [4, 8, 9, 10].

**INTENT_VIOLATION** is raised if NODE was not opened with an intent establishing the right to write attributes [1].

**SECURITY_VIOLATION** is raised if the operation represents a violation of mandatory access controls. SECURITY_VIOLATION is raised only if the conditions for other exceptions are not present.



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Additional Interfaces:

```
procedure SET_NODE_ATTRIBUTE
    (NODE      : in  NODE_TYPE;
     ATTRIBUTE : in  ATTRIBUTE_NAME;
     VALUE     : in  LIST_TYPE;
     STATUS    : in  STATUS_INDICATOR := NO_INDICATION)
is
    ATTRIBUTE_DEFINITION : AD_TYPE;
begin
    DEFINITION_OF_NODE_ATTRIBUTE (ATTRIBUTE_DEFINITION,
                                  NODE, ATTRIBUTE, STATUS);
    SET_NODE_ATTRIBUTE (NODE, ATTRIBUTE_DEFINIT1ON, VALUE, STATUS);
    CLOSE (ATTRIBUTE_DEFINITION);
exception
    when others =>
        CLOSE (ATTRIBUTE_DEFINITION);
        raise;
end SET_NODE_ATTRIBUTE;
```

Notes:

The AD identified by ATTRIBUTE must have its MODIFIABLE attribute set to TRUE. 13

### 5.3.6 Setting relationships attributes

```
procedure SET_PATH_ATTRIBUTE
    (BASE      : in  NODE_TYPE;
     KEY       : in  RELATIONSHIP_KEY := EMPTY_KEY_LIST;
     RELATION  : in  RD_TYPE;
     ATTRIBUTE : in  AD_TYPE;
     VALUE     : in  LIST_TYPE;
     STATUS    : in  STATUS_INDICATOR := NO_INDICATION);
```

Purpose:

This procedure sets the value of the attribute identified by ATTRIBUTE of the relationship identified by BASE, KEY and RELATION to the value specified by VALUE. 1

Parameters:

**BASE** is an open node handle to the node from which the relationship emanates. 2

**KEY** is the relationship key list of the affected relationship. 3

**RELATION** is an open node handle to an RD for a relation of the affected relationship. 4

**ATTRIBUTE** is an open node handle to the AD of the attribute to beset. 5


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

**VALUE** is the new value of the attribute in the form of a list. 6

**STATUS** is an established status indicator. 7

Exceptions:

**PATHNAME_SYNTAX_ERROR** is raised if the relationship identification given by KEY is syntactically illegal (see Table I, page 52) [2]. 8

**RELATIONSHIP_ERROR** is raised if KEY specifies a value for the same key more than once [5], if a relationship identified by BASE, KEY and RELATION does not exist [2], if more than one such relationship exists [1] or if ATTRIBUTE identifies a key and another relationship is identified by the resulting key values [3]. 9

**NOT_IN_VIEW_ERROR** is raised if a key name specified in KEY cannot be translated [1] or if KEY consists of a single relationship key designator and no key of the specified relation is visible [5]. 10

**AMBIGUOUS_VIEW_ERROR** is raised if translation of a key name specified in KEY is ambiguous [1] or if KEY consists of a single relationship key designator and more than one key of the specified relation is visible [4]. 11

**PREDEFINED_ATTRIBUTE_ERROR** is raised if ATTRIBUTE identifies a CAIS-controlled relationship attribute [2]. 12

**ATTRIBUTE_ERROR** is raised if the relationship does not have the attribute identified by ATTRIBUTE [4]. 13

**STATUS_ERROR** is raised if any of the node handles BASE, RELATION or ATTRIBUTE is not open [2] or if STATUS is not an established status indicator [3]. 14

**NODE_KIND_ERROR** is raised if the open node handle RELATION does not designate an RD or if the open node handle ATTRIBUTE does not designate an AD [1]. 15

**CONFORMANCE_ERROR** is raised if the operation would result in an attribute, node or relationship not conforming to its definition or the constraints in force on that definition [4, 8, 9, 10]. 16

**INTENT_VIOLATION** is raised if BASE was not opened with an intent establishing the right to write relationships [1]. 17

**SECURITY_VIOLATION** is raised if the operation represents a violation of mandatory access controls. SECURITY_VIOLATION is raised only if the conditions for other exceptions are not present. 18

Additional Interfaces:

```
procedure SET_PATH_ATTRIBUTE
    (BASE      : in  NODE_TYPE ;
     KEY       : in  RELATIONSHIP_KEY  : = EMPTY_KEY_LIST;
     RELATION  : in  RELATION_NAME     : = DEFAULT_RELATION;
     ATTRIBUTE : in  ATTRIBUTE_NAME;
     VALUE     : in  LIST_TYPE;
     STATUS    : in  STATUS_INDICATOR  := NO_INDICATION)
is
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
        RELATION_DEFINITION     : RD_TYPE;
        ATTRIBUTE_DEFINITION : AD_TYPE;
    begin
        DEFINITION_OF_RELATION (RELATION_DEFINITION, BASE, RELATION STATUS);
        DEFINITION_OF_PATH_ATTRIBUTE (ATTRIBUTE_DEFINITION,
                                      RELATION_DEFINITION,
                                      ATTRIBUTE, STATUS);
        SET_PATH_ATTRIBUTE (BASE, KEY, RELATION_DEFINITION,
                            ATTRIBUTE_DEFINITION, VALUE, STATUS);
        CLOSE (RELATION_DEFINITION);
        CLOSE (ATTRIBUTE_DEFINITION);
    exception
        when others =>
            CLOSE (RELATION_DEFINITION);
            CLOSE (ATTRIBUTE_DEFINITION);
            raise;
    end SET_PATH_ATTRIBUTE;
```

Notes:

The AD identified by ATTRIBUTE must have its MODIFIABLE attribute set to TRUE. 19

### 5.3.7 Getting node attributes

```
procedure GET_NODE_ATTRIBUTE
    (NODE      : in      NODE_TYPE;
     ATTRIBUTE : in      AD_TYPE;
     VALUE     : in out  LIST_TYPE;
     STATUS    : in      STATUS_INDICATOR := NO-INDICATION);
```

Purpose:

This procedure returns the value of the attribute identified by ATTRIBUTE of the node identified by NODE. The value is returned in the form of a list in VALUE. 1

Parameters:

**NODE** is an open node handle to the node whose value of the attribute identified by ATTRIBUTE is to be retrieved. 2

**ATTRIBUTE** is an open node handle to the AD which identifies the attribute to be retrieved. 3

**VALUE** is the the value of the attribute, returned in the form of a list. 4

**STATUS** is an established status indicator. 5



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Exceptions:

**ATTRIBUTE_ERROR** is raised if the node does not have an attribute identified by ATTRIBUTE [3] or if ATTRIBUTE designates a predefine attribute for mandatory access control purposes which may not be read under the security policy adopted by the CAIS implementation [5]. 6

**STATUS_ERROR** is raised if either NODE or ATTRIBUTE is not an open node handle [2] or if STATUS is not an established status indicator [3]. 7

**NODE_KIND_ERROR** is raised if the open node handle ATTRIBUTE does not designate an AD [1]. 8

**INTENT_VIOLATION** is raised if NODE was not opened with an intent establishing the right to read attributes [1]. 9

Additional Interfaces:

```
procedure GET_NODE_ATTRIBUTE
    (NODE      : in     NODE_TYPE;
     ATTRIBUTE : in     ATTRIBUTE_NAME ;
     VALUE     : in out LIST_TYPE;
     STATUS    : in     STATUS_INDICATOR := NO_INDICATION)
is
     ATTRIBUTE_DEFINITION : AD_TYPE;
begin
     DEFINITION_OF_NODE_ATTRIBUTE (ATTRIBUTE_DEFINITION,
                                   NODE, ATTRIBUTE, STATUS);
     GET_NODE_ATTRIBUTE (NODE, ATTRIBUTE_DEFINITION, VALUE, STATUS);
     CLOSE (ATTRIBUTE_DEFINITION);
exception
     when others =>
          CLOSE (ATTRIBUTE_DEFINITION);
          raise;
end GET_NODE_ATTRIBUTE;
```

### 5.3.8 Getting relationship attributes

```
procedure GET_PATH_ATTRIBUTE
    (BASE      : in     NODE_TYPE;
     KEY       : in     RELATIONSHIP_KEY := EMPTY_KEY_LIST;
     RELATION  : in     RD_TYPE;
     ATTRIBUTE : in     AD_TYPE;
     VALUE     : in out LIST_TYPE;
     STATUS    : in     STATUS_INDICATOR := NO_INDICATION);
```

Purpose:

This procedure returns the value of the attribute identified by ATTRIBUTE of the relationship identified by BASE, KEY, and RELATION. The value is returned in VALUE in the form of a list. 1



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Parameters:

**BASE** is an open node handle to the node from which the relationship emanates.

**KEY** is the relationship key list of the affected relationship.

**RELATION** is an open node handle for the RD of a relation of the affected relationship.

**ATTRIBUTE** is an open node handle to the AD which identifies the attribute to be retrieved.

**VALUE** is the value of the attribute, returned in the form of a list.

**STATUS** is an established status indicator.

Exceptions:

**PATHNAME_SYNTAX_ERROR** is raised if the relationship identification given by KEY is syntactically illegal (see Table I, page 52) [2].

**RELATIONSHIP_ERROR** is raised if KEY specifies a value for the same key more than once [5], if a relationship identified by BASE, KEY and RELATION does not exist [2] or if more than one such relationship exists [1].

**NOT_IN_VIEW_ERROR** is raised if a key name specified in KEY cannot be translated [1] or if KEY consists of a single relationship key designator and no key of the specified relation is visible [5].

**AMBIGUOUS_VIEW_ERROR** is raised if translation of a key name specified in KEY is ambiguous [1] or if KEY consists of a single relationship key designator and more than one key of the specified relation is visible [4].

**ATTRIBUTE_ERROR** is raised if the relationship does not have the attribute identified by ATTRIBUTE [4].

**STATUS_ERROR** is raised if any of the node handles BASE, RELATION, or ATTRIBUTE is not open [2] or if STATUS is not an established status indicator [3].

**NODE_KIND_ERROR** is raised if the open node handle RELATION does not designate an RD or if the open node handle ATTRIBUTE does not designate an AD [1].

**INTENT_VIOLATION** is raised if BASE was not opened with an intent establishing the right to read relationships [1].

Additional Interfaces:

```
procedure GET_PATH_ATTRIBUTE
    (BASE      : in     NODE_TYPE;
     KEY       : in     RELATIONSHIP_KEY := EMPTY_KEY_LIST;
     RELATION  : in     RELATION_NAME := DEFAULT_RELATION;
     ATTRIBUTE : in     ATTRIBUTE_NAME;
     VALUE     : in out LIST_TYPE;
     STATUS    : in     STATUS_INDICATOR := NO_INDICATION)
is
    RELATION_DEFINITION  : RD_TYPE;
    ATTRIBUTE_DEFINITION : AD_TYPE;
begin
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
        DEFINITION_OF_RELATION (RELATION_DEFINITION, BASE, RELATION, STATUS);
        DEFINITION_OF_PATH_ATTRIBUTE (ATTRIBUTE_DEFINITION,
                                      RELATION_DEFINITION,
                                      ATTRIBUTE, STATUS);
        GET_PATH_ATTRIBUTE (BASE, KEY, RELATION_DEFINITION,
                            ATTRIBUTE_DEFINITION, VALUE, STATUS);
        CLOSE (RELATION_DEFINITION);
        CLOSE (ATTRIBUTE_DEFINITION);
    exception
        when others =>
            CLOSE (RELATION_DEFINITION);
            CLOSE (ATTRIBUTE_DEFINITION);
            raise;
    end GET_PATH_ATTRIBUTE;
```

### 5.3.9 Generic package CAIS_INTEGER_ATTRIBUTE

This generic package contains interfaces for manipulating the values of integer attributes. This package must be instantiated for the appropriate integer type (indicated by NUMBER in the specification). The exceptions raised by all subprograms in this package are defined in the packages CAIS_DEFINITIONS, CAIS_LIST_MANAGEMENT and CAIS_PRAGMATICS. 1

```
generic
    type NUMBER is range <>;
package CAIS_INTEGER_ATTRIBUTE is
    -- Specifications of subprograms for this generic package.
end CAIS_INTEGER_ATTRIBUTE;
```

#### 5.3.9.1 Creating integer node attributes

```
procedure CREATE_NODE_ATTRIBUTE
    (NODE      : in  NODE_TYPE;
     ATTRIBUTE : in  INTEGER_AD_TYPE ;
     VALUE     : in  NUMBER;
     STATUS    : in  STATUS_INDICATOR := NO_INDICATION);
```

Purpose:

This procedure creates the integer attribute identified by ATTRIBUTE of the node identified by NODE and sets its initial value to VALUE. 1

Parameters:

**NODE** is an open node handle to the node to receive the new attribute. 2

**ATTRIBUTE** is an open node handle to the AD which identifies the attribute to be created. 3

**VALUE** is the initial value of the attribute. 4

**STATUS** is an established status indicator. 5



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Exceptions:

**PREDEFINED_ATTRIBUTE_ERROR** is raised if ATTRIBUTE identifies a CAIS-controlled node attribute [1]. 6

**ATTRIBUTE_ERROR** is raised if the node already has the specified attribute [1]. 7

**STATUS_ERROR** is raised if either of the node handles NODE or ATTRIBUTE is not open [2] or if STATUS is not an established status indicator [3]. 8

**NODE_KIND_ERROR** is raised if the open node handle ATTRIBUTE does not designate an AD [1]. 9

**CONFORMANCE_ERROR** is raised if the operation would result in an attribute, node or relationship not conforming to its definition or to the constraints in force on that definition [4, 9, l0]. 10

**INTENT_VIOLATION** is raised if NODE was not opened with an intent establishing the right to create attributes [1]. 11

**SECURITY_VIOLATION** is raised if the operation represents a violation of mandatory access controls. SECURITY_VIOLATION is raised only if the conditions for other exceptions are not present. 12

Additional Interfaces:

```
procedure CREATE_NODE_ATTRIBUTE
    (NODE      : in    NODE_TYPE;
     ATTRIBUTE : in    ATTRIBUTE_NAME;
     VALUE     : in    NUMBER;
     STATUS    : in    STATUS_INDICATOR  : = NO_INDICATION)
is
     ATTRIBUTE_DEFINITION : INTEGER_AD_TYPE;
begin
     DEFINITION_OF_NODE_ATTRIBUTE (ATTRIBUTE_DEFINITION,
                                   NODE , ATTRIBUTE, STATUS);
     CREATE_NODE_ATTRIBUTE (NODE, ATRIBUTE_DEFINITION, VALUE, STATUS);
     CLOSE (ATTRIBUTE_DEFINITION);
exception
     when others =>
          CLOSE (ATTRIBUTE_DEFINITION);
          raise;
end CREATE_NODE_ATTRIBUTE;
```

### 5.3.9.2 Creating integer relationship attributes

```
procedure CREATE_ PATH_ ATTRIBUTE
    (BASE      : in    NODE_TYPE;
     KEY       : in    RELATIONSHIP_KEY  := EMPTY_KEY_LIST;
     RELATION  : in    RD_TYPE;
     ATTRIBUTE : in    IMTEGER_AD_TYPE;
     VALUE     : in    NUMBER;
     STATUS    : in    STATUS_INDICATOR  := NO_INDICATION);
```


Source: https://assist.dla.mil -- Downloaded: 2026l0-26T07:54Z
Check the source to verify that this is the currentversion before use.

Purpose:

This procedure creates the integer attribute identified by ATTRIBUTE of the relationship identified by BASE, KEY and RELATION and sets its initial value to VALUE.

Parameters:

**BASE** is an open node handle to the node from which the relationship emanates.

**KEY** is the relationship key list of the affected relationship.

**RELATION** is an open node handle to an RD which identifies a relation of the affected relationship.

**ATTRIBUTE** is an open node handle to the AD which identifies the attribute to be created.

**VALUE** is the initial value of the attribute.

**STATUS** is an established status indicator.

Exceptions:

**PATHNAME_SYNTAX_ERROR** is raised if the relationship identification given by KEY is syntactically illegal (see Table I, page 52) [2].

**NOT_IN_VIEW_ERROR** is raised if a key name specified in KEY cannot be translated [1] or if KEY consists of a single relationship key designator and no key of the specified relation is visible [5].

**AMBIGUOUS_VIEW_ERROR** is raised if translation of a key name specified in KEY is ambiguous [1] or if KEY consists of a single relationship key designator and more than one key of the specified relation is visible [4].

**RELATIONSHIP_ERROR** is raised if KEY specifies a value for the same key more than once [5], if a relationship identified by BASE, KEY and RELATION does not exist [2] or if more than one such relationship exists [1].

**PREDEFINED_ATTRIBUTE_ERROR** is raised if ATTRIBUTE identifies a CAIS-controlled relationship attribute [2].

**ATTRIBUTE_ERROR** is raised if the relationship already has an attribute identified by ATTRIBUTE [2].

**STATUS_ERROR** is raised if any of the node handles BASE, RELATION, or ATTRIBUTE is not open [2] or if STATUS is not an established status indicator [3].

**NODE_KIND_ERROR** is raised if the open node handle RELATION does not designate an RD or if the open node handle ATTRIBUTE does not designate an AD [1].

**CONFORMANCE_ERROR** is raised if the operation would result in an attribute, node or relationship not conforming to its definition or to the constraints in force on that definition [4, 9, lo].

**INTENT_VIOLATION** is raised if BASE was not opened with an intent establishing the right to write relationships [1].

**SECURITY_VIOLATION** is raised if the operation represents a violation of mandatory access controls. SECURITY_VIOLATION is raised only if the conditions for other exceptions are not present.


Source: https://assist.dla.mil -- Downloaded: 2026l0-26T07:54Z
Check the source to verify that this is the currentversion before use.

Additional Interfaces:

```
procedure CREATE_PATH_ATTRIBUTE
    (BASE      : in  NODE_TYPE;
     KEY       : in  RELATIONSHIP_KEY := EMPTY_KEY_LIST;
     RELATION  : in  RELATION_NAME := DEFAULT_RELATION;
     ATTRIBUTE : in  ATTRIBUTE_NAME;
     VALUE     : in  NUMBER;
     STATUS    : in  STATUS_INDICATOR := NO_INDICATION)
is
     RELATION_DEFINITION  : RD_TYPE;
     ATTRIBUTE_DEFINITION : INTEGER_AD_TYPE;
begin
     DEFINITION_OF_RELATION (RELATION_DEFINITION, BASE, RELATION, STATUS);
     DEFINITION_OF_PATH_ATTRIBUTE (ATTRIBUTE_DEFINITION,
                                   RELATION_DEFINITION,
                                   ATTRIBUTE, STATUS);
     CREATE_PATH_ATTRIBUTE (BASE, KEY, RELATION_DEFINITION,
                            ATTRIBUTE_DEFINITION, VALUE, STATUS);
     CLOSE (RELATION_DEFINITION);
     CLOSE (ATTRIBUTE_DEFINITION);
exception
     when others =>
          CLOSE (RELATION_DEFINITION);
          CLOSE (ATTRIBUTE_DEFINITION);
          raise;
end CREATE_PATH_ATTRIBUTE;
```

#### 5.3.9.3 Setting integer node attributes

```
procedure SET_NODE_ATTRIBUTE
    (NODE      : in  NODE_TYPE;
     ATTRIBUTE : in  INTEGER_AD_TYPE;
     VALUE     : in  NUMBER;
     STATUS    : in  STATUS_INDICATOR := NO_INDICATION);
```

Purpose:

This procedure sets the value of the integer attribute identified by ATTRIBUTE of the node identified by NODE to the value given by VALUE. 1

Parameters:

**NODE** is an open node handle to the node whose attribute is to be set. 2

**ATTRIBUTE** is an open node handle to the AD which identifies the attribute to be set. 3

**VALUE** is the new value of the attribute. 4

**STATUS** is an established status indicator. 5



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Exceptions:

**PREDEFINED_ATTRIBUTE_ERROR** is raised if ATTRIBUTE identifies a CAIS-controlled node attribute [1]. 6

**ATTRIBUTE_ERROR** is raised if the node does not have the attribute identified by ATTRIBUTE [3]. 7

**STATUS_ERROR** is raised if either of the node handles NODE or ATTRIBUTE is not open [2] or if STATUS is not an established status indicator [3]. 8

**NODE_KIND_ERROR** is raised if the open node handle ATTRIBUTE does not designate an AD [1]. 9

**CONFORMANCE_ERROR** is raised if the operation would result in an attribute, node or relationship not conforming to its definition or to the constraints in force on that definition [4, 9, 10] or if the attribute is not modifiable [5]. 10

**INTENT_VIOLATION** is raised if NODE was not opened with an intent establishing the right to write attributes [1]. 11

**SECURITY_VIOLATION** is raised if the operation represents a violation of mandatory access controls. SECURITY_VIOLATION is raised only if the conditions for other exceptions are not present. 12

Additional Interfaces:

```
procedure SET_NODE_ATTRIBUTE
    (NODE      : in    NODE_TYPE;
     ATTRIBUTE : in    ATTRIBUTE_NAME;
     VALUE     : in    NUMBER;
     STATUS    : in    STATUS_INDICATOR := NO_INDICATION)
is
     ATTRIBUTE_DEFINITION : INTEGER_AD_TYPE;
begin
     DEFINITION_OF_NODE_ATTRIBUTE (ATTRIBUTE_DEFINITION,
                                   NODE , ATTRIBUTE , STATUS);
     SET_NODE_ATTRIBUTE (NODE, ATTRIBUTE_DEFINITION, VALUE, STATUS);
     CLOSE (ATTRIBUTE_DEFINITION);
except ion
     when others =>
          CLOSE (ATTRIBUTE_DEFINITION);
          raise;
end SET_NODE_ATTRIBUTE;
```

Notes:

The AD identified by ATTRIBUTE must have its MODIFIABLE attribute set to TRUE. 13



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

#### 5.3.9.4 Setting integer relationship attributes

```
procedure SET_PATH_ATTRIBUTE
    (BASE      : in  NODE_TYPE;
     KEY       : in  RELATIONSHIP_KEY  := EMPTY_KEY_LIST;
     RELATION  : in  RD_TYPE;
     ATTRIBUTE : in  INTEGER_AD_TYPE;
     VALUE     : in  NUMBER;
     STATUS    : in  STATUS_INDICATOR  := NO_INDICATION);
```

Purpose:

This procedure sets the value of the integer attribute identified by ATTRIBUTE of the relationship identified by BASE, KEY and RELATION to the value specified by VALUE. 1

Parameters:

**BASE** is an open node handle to the node from which the relationship emanates. 2

**KEY** is the relationship key list of the affected relationship. 3

**RELATION** is an open node handle to an RD for a relation of the affected relationship. 4

**ATTRIBUTE** is an open node handle to the AD which identifies the attribute to be set. 5

**VALUE** is the new value of the attribute. 6

**STATUS** is an established status indicator. 7

Exceptions:

**PATHNAME_SYNTAX_ERROR** is raised if the relationship identification given by KEY is syntactically illegal (see Table I, page 52) [2]. 8

**RELATIONSHIP_ERROR** is raised if KEY specifies a value for the same key more than once [5], if a relationship identified by BASE, KEY and RELATION does not exist [2] or if more than one such relationship exists [1]. 9

**NOT_IN_VIEW_ERROR** is raised if a key name specified in KEY cannot be translated [1] or if KEY consists of a single relationship key designator and no key of the specified relation is visible [5]. 10

**AMBIGUOUS_VIEW_ERROR** is raised if translation of a key name specified in KEY is ambiguous [I] or if KEY consists of a single relationship key designator and more than one key of the specified relation is visible [4]. 11

**PREDEFINED_ATTRIBUTE_ERROR** is raised if ATTRIBUTE identifies a CAIS-controlled relationship attribute [2]. 12

**ATTRIBUTE_ERROR** is raised if the relationship does not have the attribute identified by ATTRIBUTE [4]. 13

**STATUS_ERROR** is raised if any of the node handles BASE, RELATION or ATTRIBUTE is not open [2] or if STATUS is not an established status indicator [3]. 14



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

**NODE_KIND_ERROR** is raised if the open node handle RELATION does not designate an RD or if the open node handle ATTRIBUTE does not designate an AD [1]. 15

**CONFORMANCE_ERROR** is raised if the operation would result in an attribute, node or relationship not conforming to its definition or to the constraints in force on that definition [4, 9, 10] or if the attribute is not modifiable [5]. 16

**INTENT_VIOLATION** is raised if BASE was not opened with an intent establishing the right to write relationships [1]. 17

**SECURITY_VIOLATION** is raised if the operation represents a violation of mandatory access controls. SECURITY_VIOLATION is raised only if the conditions for other exceptions are not present. 18

Additional Interfaces:

```
procedure SET_PATH_ATTRIBUTE
    (BASE      : in    NODE_TYPE;
     KEY       : in    RELATIONSHIP_KEY  : = EMPTY_KEY_LIST;
     RELATION  : in    RELATION_NAME     : = DEFAULT_RELATION;
     ATTRIBUTE : in    ATTRIBUTE_NAME ;
     VALUE     : in    NUMBER;
     STATUS    : in    STATUS_INDICATOR  := NO_INDICATION)
is
     RELATION_DEFINITION     : RD_TYPE;
     ATTRIBUTE_DEFINITION : INTEGER_AD_TYPE;
begin
     DEFINITION_OF_RELATION (RELATION_DEFINITION, BASE, RELATION, STATUS);
     DEFINITION_OF_PATH_ATTRIBUTE (ATTRIBUTE_DEFINITION,
                                   RELATION_DEFINITION,
                                   ATTRIBUTE,  STATUS);
     SET_PATH_ATTRIBUTE (BASE, KEY, RELATION_DEFINITION,
                         ATTRIBUTE_DEFINITION, VALUE, STATUS);
     CLOSE (RELATION_DEFINITION);
     CLOSE (ATTRIBUTE_DEFINITION);
exception
     when others =>
          CLOSE (RELATION_DEFINITION);
          CLOSE (ATTRIBUTE_DEFINITION);
          raise;
end SET_PATH_ATTRIBUTE;
```

Notes:

The AD identified by ATTRIBUTE must have its MODIFIABLE attribute set to TRUE. 19

#### 5.3.9.5 Getting integer node attributes

```
function NODE_ATTRIBUTE
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
(NODE      : in  NODE_TYPE;
 ATTRIBUTE : in  INTEGER_AD_TYPE;
 STATUS    : in  STATUS_INDICATOR := NO_INDICATION)
return NUMBER;
```

Purpose:

This function returns the value of the integer attribute identified by ATTRIBUTE of the node identified by NODE.

Parameters:

**NODE** is an open node handle to the node from which the attribute is to be retrieved.

**ATTRIBUTE** is an open node handle to the AD which identifies the attribute to be retrieved.

**STATUS** is an established status indicator.

Exceptions:

**ATTRIBUTE_ERROR** is raised if the node does not have an attribute identified by ATTRIBUTE [3].

**ITEM_KIND_ERROR** is raised if the value of the attribute is not a single integer item [1].

**STATUS_ERROR** is raised if either NODE or ATTRIBUTE is not an open node handle [2] or if STATUS is not an established status indicator [3].

**NODE_KIND_ERROR** is raised if the open node handle ATTRIBUTE does not designate an AD [1].

**INTENT_VIOLATION** is raised if NODE was not opened with an intent establishing the right to read attributes [1].

Additional Interfaces:

```
function NODE_ATTRIBUTE
    (NODE      : in  NODE_TYPE;
     ATTRIBUTE : in  ATTRIBUTE_NAME;
     STATUS    : in  STATUS_INDICATOR := NO_INDICATION)
    return NUMBER
is
    ATTRIBUTE_DEFINITION : INTEGER_AD_TYPE;
    VALUE                : NUMBER;
begin
    DEFINITION_OF_NODE_ATTRIBUTE (ATTRIBUTE_DEFINITION,
                                  NODE, ATTRIBUTE, STATUS);
    VALUE := NODE_ATTRIBUTE (NODE, ATTRIBUTE_DEFINITION, STATUS);
    CLOSE (ATTRIBUTE_DEFINITION);
    return (VALUE);
except ion
    when others =>
        CLOSE (ATTRIBUTE_DEFINITION);
        raise;
end NODE_ATTRIBUTE;
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

#### 5.3.9.6 Getting integer relationship attributes

```
function PATH_ATTRIBUTE
    (BASE      : in  NODE_TYPE;
     KEY       : in  RELATIONSHIP_KEY  := EMPTY_KEY_LIST;
     RELATION  : in  RD_TYPE;
     ATTRIBUTE : in  INTEGER_AD_TYPE;
     STATUS    : in  STATUS_INDICATOR  := NO_INDICATION)
    return NUMBER;
```

Purpose:

This function returns the value of the integer attribute identified by ATTRIBUTE of the relationship identified by BASE, KEY and RELATION. 1

Parameters:

**BASE** is an open node handle to the node from which the relationship emanates. 2

**KEY** is the relationship key list of the relationship. 3

**RELATION** is an open node handle to the RD of a relation of the relationship. 4

**ATTRIBUTE** is an open node handle to the AD which identifies the attribute to be retrieved. 5

**STATUS** is an established status indicator. 6

Exceptions:

**PATHNAME_SYNTAX_ERROR** is raised if the relationship identification given by KEY is syntactically illegal (see Table I, page 52) [2]. 7

**RELATIONSHIP_ERROR** is raised if KEY specifies a value for the same key more than once [5], if a relationship identified by BASE, KEY and RELATION does not exist [2] or if more than one such relationship exists [1]. 8

**NOT_IN_VIEW_ERROR** is raised if a key name specified in KEY cannot be translated [1] or if KEY consists of a single relationship key designator and no key of the specified relation is visible [5]. 9

**AMBIGUOUS_VIEW_ERROR** is raised if translation of a key name specified in KEY is ambiguous [1] or if KEY consists of a single relationship key designator and more than one key of the specified relation is visible [4]. 10

**ATTRIBUTE_ERROR** is raised if the relationship does not have the attribute identified by ATTRIBUTE [4]. 11

**ITEM._KIND_ERROR** is raised if the value of the attribute is not a single integer item [1]. 12

**STATUS_ERROR** is raised if any of the node handles BASE, RELATION, or ATTRIBUTE is not open [2] or if STATUS is not an established status indicator [3]. 13

**NODE_KIND_ERROR** is raised if the open node handle RELATION does not designate an RD or if the open node handle ATTRIBUTE does not designate an AD [1]. 14

**INTENT_VIOLATION** is raised if BASE was not opened with an intent establishing the right to read relationships [1]. 15



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Additional Interfaces:

```
function PATH_ATTRIBUTE
    (BASE      : in  NODE_TYPE;
     KEY       : in  RELATIOHSHIP_KEY  := EMPTY_KEY_LIST;
     RELATION  : in  RELATION_NAME     := DEFAULT_RELATION;
     ATTRIBUTE : in  ATTRIBUTE_NAME;
     STATUS    : in  STATUS_INDICATOR  := NO_INDICATION)
   return NUMBER
is
     RELATION_DEFINITION : RD_TYPE;
     ATTRIBUTE_DEFINITION : INTEGER_AD_TYPE;
     VALUE                : NUMBER;
begin
     DEFINITION_OF_RELATION (RELATION_DEFINITION, BASE, RELATION, STATUS);
     DEFINITION_OF_PATH_ATTRIBUTE (ATTRIBUTE_DEFINITION,
                                   RELATION_DEFINITION,
                                   ATTRIBUTE, STATUS);
     VALUE := PATH_ATTRIBUTE (BASE, KEY, RELATION_DEFINITION,
                              ATTRIBUTE_DEFINITION, STATUS);
     CLOSE (RELATION_DEFINITION);
     CLOSE (ATTRIBUTE_DEFINITION);
     return (VALUE);
exception
     when others =>
          CLOSE (RELATION_DEFINITION);
          CLOSE (ATTRIBUTE_DEFINITION);
          raise;
end PATH_ATTRIBUTE;
```

### 5.3.10 Generic package CAIS_FLOAT_ATTRIBUTE

This generic package contains interfaces for manipulating the values of floating point attributes. This package must be instantiated for the appropriate floating point type (indicated by NUMBER in the specification). The exceptions raised by all subprograms in this package are defined in the packages CAIS_DEFINITIONS, CAIS_LIST_MANAGEMENT and CAIS_PRAGMATICS. 1

See Section 9.1 on page 751 for issues regarding the accuracy of floating point values. 2

```
generic
    type NUMBER is digits <>;
package CAIS_FLOAT_ATTRIBUTE is
    -- Specifications of subprograms for this generic package.
end CAIS_FLOAT_ATTRIBUTE;
```

#### 5.3.10.l Creating floating point node attributes



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
procedure CREATE_NODE_ATTRIBUTE
    (NODE      : in  NODE_TYPE;
     ATTRIBUTE : in  FLOAT_AD_TYPE;
     VALUE     : in  NUMBER;
     STATUS    : in  STATUS_INDICATOR := NO_INDICATION);
```

Purpose:

This procedure creates the floating point attribute identified by ATTRIBUTE of the node identified by NODE and sets its initial value to VALUE. 1

Parameters:

**NODE** is an open node handle to the node to receive the new attribute. 2

**ATTRIBUTE** is an open node handle to the AD which identifies the attribute to be created. 3

**VALUE** is the initial value of the attribute. 4

**STATUS** is an established status indicator. 5

Exceptions:

**PREDEFINED_ATTRIBUTE_ERROR** is raised if ATTRIBUTE identifies a CAIS-controlled node attribute [1]. 6

**ATTRIBUTE_ERROR** is raised if the node already has the specified attribute [l]. 7

**STATUS_ERROR** is raised if either of the node handles NODE or ATTRIBUTE is not open [2] or if STATUS is not an established status indicator [3]. 8

**NODE_KIND_ERROR** is raised if the open node handle ATTRIBUTE does not designate an AD [1]. 9

**CONFORMANCE_ERROR** is raised if the operation would result in an attribute, node or relationship not conforming to its definition or to the constraints in force on that definition [4, 9, lo]. 10

**INTENT_VIOLATION** is raised if NODE was not opened with an intent establishing the right to create attributes [1]. 11

**SECURITY_VIOLATION** is raised if the operation represents a violation of mandatory access controls. SE PURITY. VIOLATION is raised only if the conditions for other exceptions are not present. 12

Additional Interfaces:

```
procedure CREATE_NODE_ATTRIBUTE
    (NODE      : in  NODE_TYPE;
     ATTRIBUTE : in  ATTRIBUTE_NAME;
     VALUE     : in  NUMBER;
     STATUS    : in  STATUS_INDICATOR : = NO_INDICATION)
is
```



Source: https://assist.dla.mil -- Downloaded: 2026l0-26T07:54Z
Check the source to verify that this is the currentversion before use.

```
        ATTRIBUTE_DEFINITION : FLOAT_AD_TYPE;
    begin
        DEFINITION_OF_NODE_ATTRIBUTE (ATTRIBUTE_DEFINITION,
                                      NODE, ATTRIBUTE, STATUS);
        CREATE_NODE_ATTRIBUTE (NODE, ATTRIBUTE_DEFINITION, VALUE, STATUS);
        CLOSE  (ATTRIBUTE_DEFINITION);
    exception
        when  others  =>
            CLOSE  (ATTRIBUTE_DEFINITION);
            raise;
    end  CREATE_NODE_ATTRIBUTE;
```

#### 5.3.10.2 Creating floating point relationship attributes

```
procedure  CREATE_PATH_ATTRIBUTE
    (BASE      :  in     NODE_TYPE;
     KEY       :  in     RELATIONSHIP_KEY  := EMPTY_KEY_LIST;
     RELATION  :  in     RD_TYPE;
     ATTRIBUTE :  in     FLOAT_AD_TYPE;
     VALUE     :  in     NUMBER;
     STATUS    :  in     STATUS_INDICATOR  := NO_INDICATION);
```

Purpose:

This procedure creates the floating point attribute identified by ATTRIBUTE of the relationship identified by BASE, KEY and RELATION and sets its initial value to VALUE. 1

Parameters:

**BASE** is an open node handle to the node from which the relationship emanates. 2

**KEY** is the relationship key list of the affected relationship. 3

**RELATION** is an open node handle to an RD which identifies a relation of the affected relationship. 4

**ATTRIBUTE** is an open node handle to the AD which identifies the attribute to be created. 5

**VALUE** is the initial value of the attribute. 6

**STATUS** is an established status indicator. 7

Exceptions:

**PATHNAME_SYNTAX_ERROR** is raised if the relationship identification given by KEY is syntactically illegal (see Table I, page 52) [2]. 8

**NOT_IN_VIEW_ERROR** is raised if a key name specified in KEY cannot be translated [l] or if KEY consists of a single relationship key designator and no key of the specified relation is visible [5]. 9


Source: https://assist.dla.mil -- Downloaded: 2026l0-26T07:54Z
Check the source to verify that this is the currentversion before use.

**AMBIGUOUS_VIEW_ERROR** is raised if translation of a key name specified in KEY is ambiguous [1] or if KEY consists of a single relationship key designator and more than one key of the specified relation is visible [4]. 10

**RELATIONSHIP_ERROR** is raised if KEY specifies a value for the same key more than once [5], if a relationship identified by BASE, KEY and RELATION does not exist [2] or if more than one such relationship exists [1]. 11

**PREDEFINED_ATTRIBUTE_ERROR** is raised if ATTRIBUTE identifies a CAIS-controlled relationship attribute [2]. 12

**ATTRIBUTE_ERROR** is raised if the relationship already has an attribute identified by ATTRIBUTE [2]. 13

**STATUS_ERROR** is raised if any of the node handles BASE, RELATION, or ATTRIBUTE is not open [2] or if STATUS is not an established status indicator [3]. 14

**NODE_KIND_ERROR** is raised if the open node handle RELATION does not designate an RD or if the open node handle ATTRIBUTE does not designate an AD [1]. 15

**CONFORMANCE_ERROR** is raised if the operation would result in an attribute, node or relationship not conforming to its definition or to the constraints in force on that definition [4, 9, lo]. 16

**INTENT_VIOLATION** is raised if BASE was not opened with an intent establishing the right to write relationships [1]. 17

**SECURITY_VIOLATION** is raised if the operation represents a violation of mandatory access controls. SECURITY_VIOLATION is raised only if the conditions for other exceptions are not present. 18

Additional Interfaces:

```
procedure CREATE_PATH_ATTRIBUTE
    (BASE      : in  NODE_TYPE;
     KEY       : in  RELATIONSHIP_KEY : = EMPTY_KEY_LIST;
     RELATION  : in  RELATION_NAME : = DEFAULT_RELATION;
     ATTRIBUTE : in  ATTRIBUTE_NAME;
     VALUE     : in  NUMBER;
     STATUS    : in  STATUS_INDICATOR : = NO_INDICATION)
is
     RELATION_DEFINITION : RD_TYPE;
     ATTRIBUTE_DEFINITION : FLOAT_AD_TYPE;
begin
     DEFINITION_OF_RELATION (RELATION_DEFINITION, BASE, RELATION, STATUS);
     DEFINITION_OF_PATH_ATTRIBUTE (ATTRIBUTE_DEFINITION,
                                   RELATION_DEFINITION,
                                   ATTRIBUTE , STATUS);
     CREATE_PATH_ATTRIBUTE (BASE, KEY, RELATION_DEFINITION,
                            ATTRIBUTE_DEFINITION, VALUE, STATUS);
     CLOSE (RELATION_DEFINITION);
     CLOSE (ATTRIBUTE_DEFINITION);
exception
     when others =>
```


Source: https://assist.dla.mil -- Downloaded: 2026l0-26T07:54Z
Check the source to verify that this is the currentversion before use.

```
            CLOSE (RELATION_DEFINITION);
            CLOSE (ATTRIBUTE_DEFINITION);
            raise;
    end CREATE_PATH_ATTRIBUTE;
```

#### 5.3.10.3 Setting floating point node attributes

```
procedure SET_NODE_ATTRIBUTE
    (NODE      : in  NODE_TYPE;
     ATTRIBUTE : in  FLOAT_AD_TYPE;
     VALUE     : in  NUMBER;
     STATUS    : in  STATUS_INDICATOR := NO_INDICATION);
```

Purpose:

This procedure sets the value of the floating point attribute identified by ATTRIBUTE of the node identified by NODE to the value given by VALUE. 1

Parameters:

**NODE** is an open node handle to the node whose attribute is to beset. 2

**ATTRIBUTE** is an open node handle to the AD which identifies the attribute to beset. 3

**VALUE** is the new value of the attribute. 4

**STATUS** is an established status indicator. 5

Exceptions:

**PREDEFINED_ATTRIBUTE_ERROR** is raised if ATTRIBUTE identifies a CAIS-controlled node attribute [1]. 6

**ATTRIBUTE_ERROR** is raised if the node does not have the attribute identified by ATTRIBUTE [3]. 7

**STATUS_ERROR** is raised if either of the node handles NODE or ATTRIBUTE is not open [2] or if STATUS is not an established status indicator [3]. 8

**NODE_KIND_ERROR** is raised if the open node handle ATTRIBUTE does not designate an AD [1]. 9

**CONFORMANCE_ERROR** is raised if the operation would result in an attribute, node or relationship not conforming to its definition or to the constraints in force on that definition [4, 9, 10] or if the attribute is not modifiable [5]. 10

**INTENT_VIOLATION** is raised if NODE was not opened with an intent establishing the right to write attributes [1]. 11

**SECURITY_VIOLATION** is raised if the operation represents a violation of mandatory access controls. SECURITY_VIOLATION is raised only if the conditions for other exceptions are not present. 12


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Additional Interfaces:

```
procedure SET_NODE_ATTRIBUTE
    (NODE      : in  NODE_TYPE;
     ATTRIBUTE : in  ATTRIBUTE_NAME;
     VALUE     : in  NUMBER;
     STATUS    : in  STATUS_INDICATOR := NO_INDICATION)
is
    ATTRIBUTE_DEFINITION : FLOAT_AD_TYPE;
begin
    DEFINITION_OF_NODE_ATTRIBUTE (ATTRIBUTE_DEFINITION,
                                  NODE, ATTRIBUTE, STATUS);
    SET_NODE_ATTRIBUTE (NODE, ATTRIBUTE_DEFINITION, VALUE, STATUS);
    CLOSE (ATTRIBUTE_DEFINITION);
exception
    when others =>
        CLOSE (ATTRIBUTE_DEFINITION);
        raise;
end SET_NODE_ATTRIBUTE;
```

Notes:

The AD identified by ATTRIBUTE must have its MODIFIABLE attribute set to TRUE. 13

### 5.3.10.4 Setting floating point relationship attributes

```
procedure SET_PATH_ATTRIBUTE
    (BASE      : in  NODE_TYPE;
     KEY       : in  RELATIONSHIP_KEY := EMPTY_KEY_LIST;
     RELATION  : in  RD_TYPE;
     ATTRIBUTE : in  FLOAT_AD_TYPE;
     VALUE     : in  NUMBER;
     STATUS    : in  STATUS_INDICATOR := NO_INDICATION);
```

Purpose:

This procedure sets the value of the floating point attribute identified by ATTRIBUTE of the relationship identified by BASE, KEY and RELATION to the value specified by VALUE. 1

Parameters:

**BASE** is an open node handle to the node from which the relationship emanates. 2

**KEY** is the relationship key list of the affected relationship. 3

**RELATION** is an open node handle to an RD for a relation of the affected relationship. 4

**ATTRIBUTE** is an open node handle to the AD which identifies the attribute to beset. 5

**VALUE** is the new value of the attribute. 6

**STATUS** is an established status indicator. 7



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Exceptions:

**PATHNAME_SYNTAX_ERROR** is raised if the relationship identification given by KEY is syntactically illegal (see Table I, page 52) [2]. 8

**RELATIONSHIP_ERROR** is raised if KEY specifies a value for the same key more than once [5], if a relationship identified by BASE, KEY and RELATION does not exist [2] or if more than one such relationship exists [1]. 9

**NOT_IN_VIEW_ERROR** is raised if a key name specified in KEY cannot be translated [1] or if KEY consists of a single relationship key designator and no key of the specified relation is visible [5]. 10

**AMBIGUOUS_VIEW_ERROR** is raised if translation of a key name specified in KEY is ambiguous [1] or if KEY consists of a single relationship key designator and more than one key of the specified relation is visible [4]. 11

**PREDEFINED_ATTRIBUTE_ERROR** is raised if ATTRIBUTE identifies a CAIS-controlled relationship attribute [2]. 12

**ATTRIBUTE_ERROR** is raised if the relationship does not have the attribute identified by ATTRIBUTE [4]. 13

**STATUS_ERROR** is raised if any of the node handles BASE, RELATION or ATTRIBUTE is not open [2] or if STATUS is not an established status indicator [3]. 14

**NODE_KIND_ERROR** is raised if the open node handle RELATION does not designate an RD or if the open node handle ATTRIBUTE does not designate an AD [1]. 15

**CONFORMANCE_ERROR** is raised if the operation would result in an attribute, node or relationship not conforming to its definition or to the constraints in force on that definition [4, 9, 10] or if the attribute is not modifiable [5]. 16

**INTENT_VIOLATION** is raised if BASE was not opened with an intent establishing the right to write relationships [1]. 17

**SECURITY_VIOLATION** is raised if the operation represents a violation of mandatory access controls. SECURITY_VIOLATION is raised only if the conditions for other exceptions are not present. 18

Additional Interfaces:

```
procedure SET_PATH_ ATTRIBUTE
    (BASE      : in    NODE_TYPE;
     KEY       : in    RELATIONSHIP_KEY  : = EMPTY_KEY_LIST;
     RELATION  : in    RELATION_NAME     : = DEFAULT_RELATION;
     ATTRIBUTE : in    ATTRIBUTE_NAME;
     VALUE     : in    NUMBER;
     STATUS    : in    STATUS_INDICATOR  : = NO_INDICATION)
is
     RELATION_DEFINITION : RD_TYPE;
     ATTRIBUTE_DEFINITION : FLOAT_AD_TYPE;
begin
     DEFINITION_OF_RELATION (RELATION_DEFINITION, BASE, RELATION, STATUS);
     DEFINITION_OF_PATH_ATTRIBUTE (ATTRIBUTE_DEFINITION,
```


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
                                        RELATION_DEFINITION,
                                        ATTRIBUTE, STATUS);
        SET_PATH_ATTRIBUTE (BASE, KEY, RELATION_DEFINITION,
                            ATTRIBUTE_DEFINITION, VALUE, STATUS);
        CLOSE (RELATION_DEFINITION);
        CLOSE (ATTRIBUTE_DEFINITION);
    exception
        when others =>
            CLOSE (RELATION_DEFINITION);
            CLOSE (ATTRIBUTE_DEFINITION);
            raise;
    end SET_PATH_ATTRIBUTE;
```

Notes:

The AD identified by ATTRIBUTE must have its MODIFIABLE attribute set to TRUE.

### 5.3.10.5 Getting floating point node attributes

```
function NODE_ATTRIBUTE
    (NODE      : in  NODE_TYPE;
     ATTRIBUTE : in  FLOAT_AD_TYPE;
     STATUS    : in  STATUS_INDICATOR := NO_INDICATION)
    return NUMBER;
```

Purpose:

This function returns the value of the floating point attribute identified by ATTRIBUTE of the node identified by NODE.

Parameters:

**NODE** is an open node handle to the node from which the attribute is to be retrieved.

**ATTRIBUTE** is an open node handle to the AD which identifies the attribute to be retrieved.

**STATUS** is an established status indicator.

Exceptions:

**ATTRIBUTE_ERROR** is raised if the node does not have an attribute identified by ATTRIBUTE [3].

**ITEM_KIND_ERROR** is raised if the value of the attribute is not a single floating point item [1].

**STATUS_ERROR** is raised if either NODE or ATTRIBUTE is not an open node handle [2] or if STATUS is not an established status indicator [3].

**NODE_KIND_ERROR** is raised if the open node handle ATTRIBUTE does not designate an AD [1].

**INTENT_VIOLATION** is raised if NODE was not opened with an intent establishing the right to read attributes [1].


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Additional Interfaces:

```
function NODE_ATTRIBUTE
    (NODE      : in    NODE_TYPE;
     ATTRIBUTE : in    ATTRIBUTE_NAME;
     STATUS    : in    STATUS_INDICATOR := NO_INDICATION)
    return NUMBER
is
     ATTRIBUTE_DEFINITION : FLOAT_AD_TYPE;
     VALUE                : NUMBER;
begin
     DEFINITION_OF_NODE_ATTRIBUTE (ATTRIBUTE_DEFINITION,
                                   NODE, ATTRIBUTE, STATUS);
     VALUE := NODE_ATTRIBUTE (NODE, ATTRIBUTE_DEFINITION, STATUS);
     CLOSE (ATTRIBUTE_DEFINITION);
     return (VALUE);
exception
     when others =>
          CLOSE (ATTRIBUTE_DEFINITION);
          raise;
end NODE_ATTRIBUTE;
```

### 5.3.10.6 Getting floating point relationship attributes

```
function PATH_ATTRIBUTE
    (BASE      : in    NODE_TYPE;
     KEY       : in    RELATIONSHIP_KEY := EMPTY_KEY_LIST;
     RELATION  : in    RD_TYPE;
     ATTRIBUTE : in    FLOAT_AD_TYPE;
     STATUS    : in    STATUS_INDICATOR := NO_INDICATION)
    return NUMBER;
```

Purpose:

This function returns the value of the floating point attribute identified by ATTRIBUTE of the relationship identified by BASE, KEY and RELATION.

Parameters:

**BASE** is an open node handle to the node from which the relationship emanates.

**KEY** is the relationship key list of the relationship.

**RELATION** is an open node handle to the RD of a relation of the relationship.

**ATTRIBUTE** is an open node handle to the AD which identifies the attribute to be retrieved.

**STATUS** is an established status indicator.



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Exceptions:

**PATHNAME_SYNTAX_ERROR** is raised if the relationship identification given by KEY is syntactically illegal (see Table I, page 52) [2]. 7

**RELATIONSHIP_ERROR** is raised if KEY specifies a value for the same key more than once [5], if a relationship identified by BASE, KEY and RELATION does not exist [2] or if more than one such relationship exists [1]. 8

**NOT_IN_VIEW_ERROR** is raised if a key name specified in KEY cannot be translated [1] or if KEY consists of a single relationship key designator and no key of the specified relation is visible [5]. 9

**AMBIGUOUS_VIEW_ERROR** is raised if translation of a key name specified in KEY is ambiguous [1] or if KEY consists of a single relationship key designator and more than one key of the specified relation is visible [4]. 10

**ATTRIBUTE_ERROR** is raised if the relationship does not have the attribute identified by ATTRIBUTE [4]. 11

**ITEM_KIND_ERROR** is raised if the value of the attribute is not a single floating point item [1]. 12

**STATUS_ERROR** is raised if any of the node handles BASE, RELATION, or ATTRIBUTE is not open [2] or if STATUS is not an established status indicator [3]. 13

**NODE_KIND_ERROR** is raised if the open node handle RELATION does not designate an RD or if the open node handle ATTRIBUTE does not designate an AD [1]. 14

**INTENT_VIOLATION** is raised if BASE was not opened with an intent establishing the right to read relationships [1]. 15

Additional Interfaces:

```
function PATH_ATTRIBUTE
    (BASE      : in  NODE_TYPE;
     KEY       : in  RELATIONSHIP_KEY := EMPTY_KEY_LIST;
     RELATION  : in  RELATION_NAME := DEFAULT_RELATION;
     ATTRIBUTE : in  ATTRIBUTE_NAME;
     STATUS    : in  STATUS_INDICATOR := NO_INDICATION)
    return NUMBER
is
    RELATION_DEFINITION  : RD_TYPE;
    ATTRIBUTE_DEFINITION : FLOAT_AD_TYPE;
    VALUE                : NUMBER;
begin
    DEFINITION_OF_RELATION (RELATION_DEFINITION, BASE, RELATION, STATUS);
    DEFINITION_OF_PATH_ATTRIBUTE (ATTRIBUTE_DEFINITION,
                                  RELATION_DEFINITION,
                                  ATTRIBUTE , STATUS);
    VALUE := PATH_ATTRIBUTE (BASE, KEY, RELATION_DEFINITION,
                             ATTRIBUTE_DEFINITION, STATUS);
    CLOSE (RELATION_DEFINITION);
    CLOSE (ATTRIBUTE_DEFINITION);
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
        return (VALUE);
    exception
        when others =>
            CLOSE (RELATION_DEFINITION);
            CLOSE (ATTRIBUTE_DEFINITION);
            raise;
    end PATH_ATTRIBUTE;
```

### 5.3.11 Package CAIS_STRING_ATTRIBUTE

This package contains interfaces for manipulating the values of string attributes. The exceptions raised by all subprograms in this package are defined in the packages CAIS_DEFINITIONS, CAIS_LIST_-MANAGEMENT and CAIS_PRAGMATICS.

#### 5.3.11.1 Creating string node attributes

```
procedure CREATE_NODE_ATTRIBUTE
    (NODE      : in  NODE_TYPE;
     ATTRIBUTE : in  STRING_AD_TYPE;
     VALUE     : in  STRING;
     STATUS    : in  STATUS_INDICATOR := NO_INDICATION);
```

Purpose:

This procedure creates the string attribute identified by ATTRIBUTE, of the node identified by NODE and sets its initial value to VALUE.

Parameters:

**NODE** is an open node handle to the node to receive the new attribute.

**ATTRIBUTE** is an open node handle to the AD which identifies the attribute to be created.

**VALUE** is the initial value of the attribute.

**STATUS** is an established status indicator.

Exceptions:

**PREDEFINED_ATTRIBUTE_ERROR** is raised if ATTRIBUTE identifies a CAIS-controlled node attribute [1].

**ATTRIBUTE_ERROR** is raised if the node already has the specified attribute [l].

**STATUS_ERROR** is raised if either of the node handles NODE or ATTRIBUTE is not open [2] or if STATUS is not an established status indicator [3].

**NODE_KIND_ERROR** is raised if the open node handle ATTRIBUTE does not designate an AD [1].


Source: https://assist.dla.mil -- Downloaded: 2026l0-26T07:54Z
Check the source to verify that this is the currentversion before use.

**CONFORMANCE_ERROR** is raised if the operation would result in an attribute, node or relationship not conforming to its definition or to the constraints in force on that definition [4, 9, 10]. 10

**INTENT_VIOLATION** is raised if NODE was not opened with an intent establishing the right to create attributes [1]. 11

**SECURITY_VIOLATION** is raised if the operation represents a violation of mandatory access controls. SECURITY_VIOLATION is raised only if the conditions for other exceptions are not present. 12

Additional Interfaces:

```
procedure CREATE_NODE_ATTRIBUTE
    (NODE      : in    NODE_TYPE;
     ATTRIBUTE : in    ATTRIBUTE_NAME;
     VALUE     : in    STRING;
     STATUS    : in    STATUS_INDICATOR  := NO_INDICATION)
is
     ATTRIBUTE_DEFINITION : STRING_AD_TYPE;
begin
     DEFINITION_OF_NODE_ATTRIBUTE (ATTRIBUTE_DEFINITION,
                                   NODE, ATTRIBUTE, STATUS);
     CREATE_NODE_ATTRIBUTE (NODE, ATTRIBUTE_DEFINITION, VALUE, STATUS);
     CLOSE  (ATTRIBUTE_DEFINITION);
exception
     when others =>
          CLOSE (ATTRIBUTE_DEFINITION);
          raise;
end CREATE_NODE_ATTRIBUTE;
```

### 5.3.11.2 Creating string relationship attributes

```
procedure CREATE_PATH_ATTRIBUTE
    (BASE      : in    NODE_TYPE;
     KEY       : in    RELATIONSHIP_KEY := EMPTY_KEY_LIST;
     RELATION  : in    RD_TYPE;
     ATTRIBUTE : in    STRING_AD_TYPE;
     VALUE     : in    STRING;
     STATUS    : in    STATUS_INDICATOR := NO_INDICATION);
```

Purpose:

This procedure creates the string attribute identified by ATTRIBUTE of the relationship identified by BASE, KEY and RELATION and sets its initial value to VALUE. 1



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Parameters:

**BASE** is an open node handle to the node from which the relationship emanates.

**KEY** is the relationship key list of the relationship.

**RELATION** is an open node handle to an RD which identifies a relation of the relationship.

**ATTRIBUTE** is an open node handle to the AD which identifies the attribute to be created.

**VALUE** is the initial value of the attribute.

**STATUS** is an established status indicator.

Exceptions:

**PATHNAME_SYNTAX_ERROR** is raised if the relationship identification given by KEY is syntactically illegal (see Table I, page 52) [2].

**NOT_IN_VIEW_ERROR** is raised if a key name specified in KEY cannot be translated [1] or if KEY consists of a single relationship key designator and no key of the specified relation is visible [5].

**AMBIGUOUS_VIEW_ERROR** is raised if translation of a key name specified in KEY is ambiguous [1] or if KEY consists of a single relationship key designator and more than one key of the specified relation is visible [4].

**RELATIONSHIP_ERROR** is raised if KEY specifies a value for the same key more than once [5], if a relationship identified by BASE, KEY and RELATION does not exist [2] or if more than one such relationship exists [I].

**PREDEFINED_ATTRIBUTE_ERROR** is raised if ATTRIBUTE identifies a CAIS-controlled relationship attribute [2].

**ATTRIBUTE_ERROR** is raised if the relationship already has an attribute identified by ATTRIBUTE [2].

**STATUS_ERROR** is raised if any of the node handles BASE, RELATION, or ATTRIBUTE is not open [2] or if STATUS is not an established status indicator [3].

**NODE_KIND_ERROR** is raised if the open node handle RELATION does not designate an RD or if the open node handle ATTRIBUTE does not designate an AD [1].

**CONFORMANCE_ERROR** is raised if the operation would result in an attribute, node or relationship not conforming to its definition or to the constraints in force on that definition [4, 9, 10].

**INTENT_VIOLATION** is raised if BASE was not opened with an intent establishing the right to write relationships [I].

**SECURITY_VIOLATION** is raised if the operation represents a violation of mandatory access controls. SECURITY_VIOLATION is raised only if the conditions for other exceptions are not present.

Additional Interfaces:

```
procedure CREATE_PATH_ATTRIBUTE
    (BASE      : in      NODE_TYPE;
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
        KEY          : in    RELATIONSHIP_KEY  := EMPTY_KEY_LIST;
        RELATION     : in    RELATION_NAME     := DEFAULT_RELATION;
        ATTRIBUTE    : in    ATTRIBUTE_NAME;
        VALUE        : in    STRING;
        STATUS       : in    STATUS_INDICATOR  := NO_INDICATION)
    is
        RELATION_DEFINITION : RD_TYPE;
        ATTRIBUTE_DEFINITION : STRING_AD_TYPE;
    begin
        DEFINITION_OF_RELATION (RELATION_DEFINITION, BASE, RELATION, STATUS);
        DEFINITION_OF_PATH_ATTRIBUTE (ATTRIBUTE_DEFINITION,
                                      RELATION_DEFINITION,
                                      ATTRIBUTE, STATUS);
        CREATE_PATH_ATTRIBUTE (BASE, KEY, RELATION_DEFINITION,
                               ATTRIBUTE_DEFINITION, VALUE, STATUS);
        CLOSE (RELATION_DEFINITION);
        CLOSE (ATTRIBUTE_DEFINITION);
    exception
        when others =>
            CLOSE (RELATION_DEFINITION);
            CLOSE (ATTRIBUTE_DEFINITION);
            raise;
    end CREATE_PATH_ATTRIBUTE;
```

#### 5.3.11.3 Setting string node attributes

```
    procedure SET_NODE_ATTRIBUTE
        (NODE        : in    NODE_TYPE;
         ATTRIBUTE   : in    STRING_AD_TYPE;
         VALUE       : in    STRING;
         STATUS      : in    STATUS_INDICATOR  := NO_INDICATION);
```

Purpose:

This procedure sets the value of the string attribute identified by ATTRIBUTE of the node identified by NODE to the value specified by VALUE. 1

Parameters:

**NODE** is an open node handle to the node whose attribute is to be set. 2

**ATTRIBUTE** is an open node handle to the AD which identifies the attribute to be set. 3

**VALUE** is the new value of the attribute. 4

**STATUS** is an established status indicator. 5



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Except ions:

**PREDEFINED_ATTRIBUTE_ERROR** is raised if ATTRIBUTE identifies a CAIS-controlled node attribute [1]. 6

**ATTRIBUTE_ERROR** is raised if the node does not have the attribute identified by ATTRIBUTE [3]. 7

**STATUS_ERROR** is raised if either of the node handles NODE or ATTRIBUTE is not open [2] or if STATUS is not an established status indicator [3]. 8

**NODE_KIND_ERROR** is raised if the open node handle ATTRIBUTE does not designate an AD [1]. 9

**CONFORMANCE_ERROR** is raised if the operation would result in an attribute, node or relationship not conforming to its definition or to the constraints in force on that definition [4, 9, 10] or if the attribute is not modifiable [5]. 10

**INTENT_VIOLATION** is raised if NODE was not opened with an intent establishing the right to write attributes [I]. 11

**SECURITY_VIOLATION** is raised if the operation represents a violation of mandatory access controls. SECURITY_VIOLATION is raised only if the conditions for other exceptions are not present. 12

Additional Interfaces:

```
procedure SET_NODE_ATTRIBUTE
    (NODE      : in    NODE_TYPE;
     ATTRIBUTE : in    ATTRIBUTE_NAME;
     VALUE     : in    STRING;
     STATUS    : in    STATUS_INDICATOR := NO_INDICATION)
is
     ATTRIBUTE_DEFINITION : STRING_AD_TYPE;
begin
     DEFINITION_OF_NODE_ATTRIBUTE (ATTRIBUTE_DEFINITION,
                                   NODE, ATTRIBUTE, STATUS);
     SET_NODE_ATTRIBUTE (NODE, ATTRIBUTE_DEFINITION, VALUE, STATUS);
     CLOSE (ATTRIBUTE_DEFINITION);
exception
     when others =>
          CLOSE (ATTRIBUTE_DEFINITION);
          raise;
end SET_NODE_ATTRIBUTE;
```

Notes:

The AD identified by ATTRIBUTE must have its MODIFIABLE attribute set to TRUE. 13



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

#### 5.3.11.4 Setting string relationship attributes

```
procedure SET_PATH_ATTRIBUTE
    (BASE      : in  NODE_TYPE;
     KEY       : in  RELATIONSHIP_KEY  := EMPTY_KEY_LIST;
     RELATION  : in  RD_TYPE;
     ATTRIBUTE : in  STRING_AD_TYPE;
     VALUE     : in  STRING;
     STATUS    : in  STATUS_INDICATOR  := NO_INDICATION);
```

Purpose:

This procedure sets the value of the string attribute identified by ATTRIBUTE of the relationship identified by BASE, KEY and RELATION to the value specified by VALUE. 1

Parameters:

**BASE** is an open node handle to the node from which the relationship emanates. 2

**KEY** is the relationship key list of the relationship. 3

**RELATION** is an open node handle to an RD for a relation of the affected relationship. 4

**ATTRIBUTE** is an open node handle to the AD which identifies the attribute to be set. 5

**VALUE** is the new value of the attribute. 6

**STATUS** is an established status indicator. 7

Exceptions:

**PATHNAME_SYNTAX_ERROR** is raised if the relationship identification given by KEY is syntactically illegal (see Table I, page 52) [2]. 8

**RELATIONSHIP_ERROR** is raised if KEY specifies a value for the same key more than once [5], if a relationship identified by BASE, KEY and RELATION does not exist [2] or if more than one such relationship exists [1]. 9

**NOT_IN_VIEW_ERROR** is raised if a key name specified in KEY cannot be translated [1] or if KEY consists of a single relationship key designator and no key of the specified relation is visible [5]. 10

**AMBIGUOUS_VIEW_ERROR** is raised if translation of a key name specified in KEY is ambiguous [1] or if KEY consists of a single relationship key designator and more than one key of the specified relation is visible [4]. 11

**PREDEFINED_ATTRIBUTE_ERROR** is raised if ATTRIBUTE identifies a CAIS-controlled relationship attribute [2]. 12

**ATTRIBUTE_ERROR** is raised if the relationship does not have the attribute identified by ATTRIBUTE [4]. 13

**STATUS_ERROR** is raised if any of the node handles BASE, RELATION or ATTRIBUTE is not open [2] or if STATUS is not an established status indicator [3]. 14



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

**NODE_KIND_ERROR** is raised if the open node handle RELATION does not designate an RD or if the open node handle ATTRIBUTE does not designate an AD [1]. 15

**CONFORMANCE_ERROR** is raised if the operation would result in an attribute, node or relationship not conforming to its definition or to the constraints in force on that definition [4, 9, 10] or if the attribute is not modifiable [5]. 16

**INTENT_VIOLATION** is raised if BASE was not opened with an intent establishing the right to write relationships [1]. 17

**SECURITY_VIOLATION** is raised if the operation represents a violation of mandatory access controls. SECURITY_VIOLATION is raised only if the conditions for other exceptions are not present. 18

Additional Interfaces:

```
procedure SET_ PATH_ ATTRIBUTE
    (BASE      : in    NODE_TYPE;
     KEY       : in    RELATIONSHIP_KEY  : = EMPTY_KEY_LIST;
     RELATION  : in    RELATION_NAME     : = DEFAULT_RELATION;
     ATTRIBUTE : in    ATTRIBUTE_NAME;
     VALUE     : in    STRING;
     STATUS    : in    STATUS_INDICATOR  : = NO_INDICATION)
is
     RELATION_DEFINITION : RD_TYPE;
     ATTRIBUTE_DEFINITION : STRING_AD_TYPE;
begin
     DEFINITION_OF_RELATION (RELATION_DEFINITION, BASE, RELATION, STATUS);
     DEFINITION_OF_PATH_ATTRIBUTE (ATTRIBUTE_DEFINITION,
                                   RELATION_DEFINITION,
                                   ATTRIBUTE,  STATUS);
     SET_PATH_ATTRIBUTE (BASE, KEY, RELATION_DEFINITION,
                         ATTRIBUTE_DEFINITION, VALUE, STATUS);
     CLOSE  (RELATION_DEFINITION);
     CLOSE  (ATTRIBUTE_DEFINITION);
exception
     when  others  =>
          CLOSE  (RELATION_DEFINITION);
          CLOSE  (ATTRIBUTE_DEFINITION);
          raise;
end SET_PATH_ATTRIBUTE;
```

Notes:

The AD identified by ATTRIBUTE must have its MODIFIABLE attribute set to TRUE. 19

### 5.3.11.5 Getting string node attributes

```
function  NODE_ATTRIBUTE
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
(NODE      : in  NODE_TYPE;
 ATTRIBUTE : in  STRING_AD_TYPE;
 STATUS    : in  STATUS_INDICATOR := NO_INDICATION)
return STRING;

function NODE_ATTRIBUTE
(NODE      : in  NODE_TYPE;
 ATTRIBUTE : in  ATTRIBUTE_NAME;
 STATUS    : in  STATUS_INDICATOR := NO_INDICATION)
return STRING;
```

Purpose:

These functions return the value of the string attribute identified by ATTRIBUTE of the node identified by the open node handle NODE. In the first function, ATTRIBUTE is an open node handle to the AD which identifies the attribute and, in the second function, it is the local name of the attribute in the context of the node handle NODE.

Parameters:

**NODE** is an open node handle to the node having the attribute to be retrieved.

**ATTRIBUTE** is an open node handle to the AD or the local name in the context of the node handle NODE which identifies the attribute to be retrieved.

**STATUS** is an established status indicator.

Exceptions:

**NOT_IN_VIEW_ERROR** is raised if the attribute name ATTRIBUTE cannot retranslated [1].

**AMBIGUOUS_VIEW_ERROR** is raised if translation of the attribute name ATTRIBUTE is ambiguous [1].

**ATTRIBUTE_ERROR** is raised if the node does not have an attribute identified by ATTRIBUTE [3].

**ITEM_KIND_ERROR** is raised if the value of the attribute is not a single string item [1].

**STATUS_ERROR** is raised if either NODE or ATTRIBUTE is not an open node handle [2] or if STATUS is not an established status indicator [3].

**NODE_KIND_ERROR** is raised if the open node handle ATTRIBUTE does not designate an AD [1].

**INTENT_VIOLATION** is raised if NODE was not opened with an intent establishing the right to read attributes [1].



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

#### 5.3.11.6 Getting string relationship attributes

```
function PATH_ATTRIBUTE
    (BASE      : in  NODE_TYPE;
     KEY       : in  RELATIONSHIP_KEY  := EMPTY_KEY_LIST;
     RELATION  : in  RD_TYPE;
     ATTRIBUTE : in  STRING_AD_TYPE;
     STATUS    : in  STATUS_INDICATOR  := NO_INDICATION)
    return STRING;

function PATH_ATTRIBUTE
    (BASE      : in  NODE_TYPE;
     KEY       : in  RELATIONSHIP_KEY  := EMPTY_KEY_LIST;
     RELATION  : in  RELATION_NAME     := DEFAULT_RELATION;
     ATTRIBUTE : in  ATTRIBUTE_NAME;
     STATUS    : in  STATUS_INDICATOR  := NO_INDICATION)
    return STRING;
```

Purpose:

These functions return the value of the string attribute identified by ATTRIBUTE of the relationship identified by BASE, KEY and RELATION. In the first interface, ATTRIBUTE is an open node handle to the AD for the attribute and RELATION is an open node handle to an RD for a relation of the relationship. In the second interface, ATTRIBUTE is the local name for the attribute in the context of the identified relation and RELATION is the local name for a relation of the relationship in the context of the base node handle BASE.

Parameters:

**BASE** is an open node handle to the node from which the relationship emanates.

**KEY** is the relationship key list of the relationship containing the attribute.

**RELATION** is an open node handle to an RD for a relation of the relationship or the local name of an emanating relation for the relationship in the context of the base node handle BASE.

**ATTRIBUTE** is an open node handle to the AD which identifies the attribute or the local name for the attribute in the context of the relation identified by RELATION.

**STATUS** is an established status indicator.

Exceptions:

**PATHNAME_SYNTAX_ERROR** is raised if the relationship identification given by KEY is syntactically illegal (see Table I, page 52) [2].

**RELATIONSHIP_ERROR** is raised if if KEY specifies a value for the same key more than once [5], a relationship identified by BASE, KEY and RELATION does not exist [2] or if more than one such relationship exists [1].


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

**NOT_IN_VIEW_ERROR** is raised if a key name specified in KEY, the attribute name ATTRIBUTE or the relation name RELATION cannot be translated [1] or if KEY consists of a single relationship key designator and no key of the specified relation is visible [5].

**AMBIGUOUS_VIEW_ERROR** is raised if translation of a key name specified in KEY, the attribute name ATTRIBUTE, or the relation name RELATION is ambiguous [1] or if KEY consists of a single relationship key designator and more than one key of the specified relation is visible [4].

**ATTRIBUTE_ERROR** is raised if the relationship does not have the attribute identified by ATTRIBUTE [4].

**ITEM_KIND_ERROR** is raised if the value of the attribute is not a single string item [1].

**STATUS_ERROR** is raised if any of the node handles BASE, RELATION, or ATTRIBUTE is not open [2] or if STATUS is not an established status indicator [3].

**NODE_KIND_ERROR** is raised if the open node handle RELATION does not designate an RD or if the open node handle ATTRIBUTE does not designate an AD [1].

**INTENT_VIOLATION** is raised if BASE was not opened with an intent establishing the right to read relationships [1].

### 5.3.12 Package CAIS_IDENTIFIER_ATTRIBUTE

This package contains interfaces for manipulating the values of identifier attributes (including enumerated identifier attributes) in the form of values of type TOKEN_TYPE (see package CAIS_LIST_MANAGEMENT Section 9 on page 745). The exceptions raised by all subprograms in this package are defined in the packages CAIS_DEFINITIONS, CAIS_LIST_MANAGEMENT and CAIS_PRAGMATIC.

#### 5.3.12.1 Creating identifier node attributes

```
procedure CREATE_NODE_ATTRIBUTE
    (NODE      : in  NODE_TYPE;
     ATTRIBUTE : in  IDENTIFIER_AD_TYPE;
     VALUE     : in  TOKEN_TYPE;
     STATUS    : in  STATUS_INDICATOR := NO_INDICATION);
```

Purpose:

This procedure creates the identifier attribute identified by ATTRIBUTE of the node identified by NODE and sets its initial value to VALUE.

Parameters:

**NODE** is an open node handle to the node to receive the new attribute.

**ATTRIBUTE** is an open node handle to the AD which identifies the attribute to be created.

**VALUE** is the initial value of the attribute in token form.

**STATUS** is an established status indicator.



Source: https://assist.dla.mil -- Downloaded: 202610-26T07:54Z
Check the source to verify that this is the current version before use.

Exceptions:

**PREDEFINED_ATTRIBUTE_ERROR** is raised if ATTRIBUTE identifies a CAIS-controlled node attribute [1]. 6

**ATTRIBUTE_ERROR** is raised if the node already has the specified attribute [1]. 7

**STATUS_ERROR** is raised if either of the node handles NODE or ATTRIBUTE is not open [2] or if STATUS is not an established status indicator [3]. 8

**NODE_KIND_ERROR** is raised if the open node handle ATTRIBUTE does not designate an AD [1]. 9

**CONFORMANCE_ERROR** is raised if the operation would result in an attribute, node or relationship not conforming to its definition or to the constraints in force on that definition [4, 9, l0]. 10

**INTENT_VIOLATION** is raised if NODE was not opened with an intent establishing the right to create attributes [1]. 11

**SECURITY_VIOLATION** is raised if the operation represents a violation of mandatory access controls. SECURITY_VIOLATION is raised only if the conditions for other exceptions are not present. 12

Additional Interfaces:

```
procedure CREATE_NODE_ATTRIBUTE
    (NODE      : in    NODE_TYPE;
     ATTRIBUTE : in    ATTRIBUTE_NAME;
     VALUE     : in    TOKEN_TYPE;
     STATUS    : in    STATUS_INDICATOR := NO_INDICATION)
is
     ATTRIBUTE_DEFINITION : IDENTIFIER_AD_TYPE;
begin
     DEFINITION_OF_NODE_ATTRIBUTE (ATTRIBUTE_DEFINITION,
                                   NODE, ATTRIBUTE, STATUS);
     CREATE_NODE_ATTRIBUTE (NODE, ATTRIBUTE_DEFINITION, VALUE, STATUS);
     CLOSE (ATTRIBUTE_DEFINITION);
exception
     when others =>
          CLOSE (ATTRIBUTE_DEFINITION);
          raise;
end CREATE_NODE_ATTRIBUTE;
```

### 5.3.12.2 Creating identifier relationship attributes

```
procedure CREATE_PATH_ATTRIBUTE
    (BASE      : in    NODE_TYPE;
     KEY       : in    RELATIONSHIP_KEY := EMPTY_KEY_LIST;
     RELATION  : in    RD_TYPE;
     ATTRIBUTE : in    IDENTIFIER_AD_TYPE;
     VALUE     : in    TOKEN_TYPE;
     STATUS    : in    STATUS_INDICATOR := NO_INDICATION);
```


Source: https://assist.dla.mil -- Downloaded: 2026l0-26T07:54Z
Check the source to verify that this is the current version before use.

Purpose:

This procedure creates the identifier attribute identified by ATTRIBUTE of the relationship identified by BASE, KEY and RELATION and sets its initial value to VALUE. 1

Parameters:

**BASE** is an open node handle to the node from which the relationship emanates. 2

**KEY** is the relationship key list of the affected relationship. 3

**RELATION** is an open node handle to an RD which identifies a relation of the affected relationship. 4

**ATTRIBUTE** is an open node handle to the AD which identifies the attribute to be created. 5

**VALUE** is the initial value of the attribute in token form. 6

**STATUS** is an established status indicator. 7

Exceptions:

**PATHNAME_SYNTAX_ERROR** is raised if the relationship identification given by KEY is syntactically illegal (see Table I, page 52) [2]. 8

**NOT_IN_VIEW_ERROR** is raised if a key name specified in KEY cannot be translated [1] or if KEY consists of a single relationship key designator and no key of the specified relation is visible [5]. 9

**AMBIGUOUS_VIEW_ERROR** is raised if translation of a key name specified in KEY is ambiguous [1] or if KEY consists of a single relationship key designator and more than one key of the specified relation is visible [4]. 10

**RELATIONSHIP_ERROR** is raised if KEY specifies a value for the same key more than once [5], if a relationship identified by BASE, KEY and RELATION does not exist [2], if more than one such relationship exists [1] or if ATTRIBUTE identifies a key and another relationship is identified by the resulting key values [3]. 11

**PREDEFINED_ATTRIBUTE_ERROR** is raised if ATTRIBUTE identifies a CAIS-controlled relationship attribute [2]. 12

**ATTRIBUTE_ERROR** is raised if the relationship already has an attribute identified by ATTRIBUTE [2]. 13

**STATUS_ERROR** is raised if any of the node handles BASE, RELATION, or ATTRIBUTE is not open [2] or if STATUS is not an established status indicator [3]. 14

**NODE_KIND_ERROR** is raised if the open node handle RELATION does not designate an RD or if the open node handle ATTRIBUTE does not designate an AD [1]. 15

**CONFORMANCE_ERROR** is raised if the operation would result in an attribute, node or relationship not conforming to its definition or to the constraints in force on that definition [4, 9, 10]. 16

**INTENT_VIOLATION** is raised if BASE was not opened with an intent establishing the right to write relationships [1]. 17

**SECURITY_VIOLATION** is raised if the operation represents a violation of mandatory access controls. SECURITY_VIOLATION is raised only if the conditions for other exceptions are not present. 18


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Additional Interfaces:

```
procedure CREATE_PATH_ATTRIBUTE
    (BASE      : in  NODE_TYPE;
     KEY       : in  RELATIONSHIP_KEY  := EMPTY_KEY_LIST;
     RELATION  : in  RELATION_NAME     := DEFAULT_RELATION;
     ATTRIBUTE : in  ATTRIBUTE_NAME;
     VALUE     : in  TOKEN_TYPE;
     STATUS    : in  STATUS_INDICATOR  := NO_INDICATION)
is
     RELATION_DEFINITION : RD_TYPE;
     ATTRIBUTE_DEFINITION : IDENTIFIER_AD_TYPE;
begin
     DEFINITION_OF_RELATION (RELATION_DEFINITION, BASE, RELATION, STATUS);
     DEFINITION_OF_PATH_ATTRIBUTE (ATTRIBUTE_DEFINITION,
                                   RELATION_DEFINITION,
                                   ATTRIBUTE, STATUS);
     CREATE_PATH_ATTRIBUTE (BASE, KEY, RELATION_DEFINITION,
                            ATTRIBUTE-DEFINITION, VALUE, STATUS);
     CLOSE (RELATION_DEFINITION);
     CLOSE (ATTRIBUTE_DEFINITION);
exception
     when others =>
          CLOSE (RELATION_DEFINITION);
          CLOSE (ATTRIBUTE_DEFINITION);
          raise;
end CREATE_PATH_ATTRIBUTE;
```

### 5.3.12.3 Setting identifier node attributes

```
procedure SET_NODE_ATTRIBUTE
    (NODE      : in  NODE_TYPE;
     ATTRIBUTE : in  IDENTIFIER_AD_TYPE;
     VALUE     : in  TOKEN_TYPE;
     STATUS    : in  STATUS_INDICATOR  := NO_INDICATION);
```

Purpose:

This procedure sets the value of the identifier attribute identified by ATTRIBUTE of the node identified by NODE to the value given by VALUE.

Parameters:

**NODE** is an open node handle to the node whose attribute is to beset.

**ATTRIBUTE** is an open node handle to the AD which identifies the attribute to be set.

**VALUE** is the new value of the attribute in token form.

**STATUS** is an established status indicator.



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Exceptions:

**PREDEFINED_ATTRIBUTE_ERROR** is raised if ATTRIBUTE identifies a CAIS-controlled node attribute [1]. 6

**ATTRIBUTE_ERROR** is raised if the node does not have the attribute identified by ATTRIBUTE [3]. 7

**STATUS_ERROR** is raised if either of the node handles NODE or ATTRIBUTE is not open [2] or if STATUS is not an established status indicator [3]. 8

**NODE_KIND_ERROR** is raised if the open node handle ATTRIBUTE does not designate an AD [1]. 9

**CONFORMANCE_ERROR** is raised if the operation would result in an attribute, node or relationship not conforming to its definition or to the constraints in force on that definition [4, 9, 10] or if the attribute is not modifiable [5]. 10

**INTENT_VIOLATION** is raised if NODE was not opened with an intent establishing the right to write attributes [1]. 11

**SECURITY_VIOLATION** is raised if the operation represents a violation of mandatory access controls. SECURITY_VIOLATION is raised only if the conditions for other exceptions are not present. 12

Additional Interfaces:

```
procedure SET_NODE_ATTRIBUTE
    (NODE      : in    NODE_TYPE;
     ATTRIBUTE : in    ATTRIBUTE_NAME;
     VALUE     : in    TOKEN_TYPE;
     STATUS    : in    STATUS_INDICATOR  : = NO_INDICATION)
is
     ATTRIBUTE_DEFINITION : IDENTIFIER_AD_TYPE;
begin
     DEFINITION_OF_NODE_ATTRIBUTE  (ATTRIBUTE_DEFINITION,  ,
                                    NODE, ATTRIBUTE, STATUS);
     SET_NODE_ATTRIBUTE (NODE, ATTRIBUTE_DEFINITION, VALUE, STATUS);
     CLOSE  (ATTRIBUTE_DEFINITION);
exception
     when  others  =>
          CLOSE  (ATTRIBUTE_DEFINITION);
          raise;
end SET_NODE_ATTRIBUTE;
```

Notes:

The AD identified by ATTRIBUTE must have its MODIFIABLE attribute set to TRUE. 13


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

#### 5.3.12.4 Setting identifier relationship attributes

```
procedure SET_PATH_ATTRIBUTE
    (BASE      : in  NODE_TYPE;
     KEY       : in  RELATIONSHIP_KEY  := EMPTY_KEY_LIST;
     RELATION  : in  RD_TYPE;
     ATTRIBUTE : in  IDENTIFIER_AD_TYPE;
     VALUE     : in  TOKEN_TYPE;
     STATUS    : in  STATUS_INDICATOR  := NO_INDICATION);
```

Purpose:

This procedure sets the value of the identifier attribute identified by ATTRIBUTE of the relationship identified by BASE, KEY and RELATION to the value specified by VALUE.

Parameters:

**BASE** is an open node handle to the node from which the relationship emanates.

**KEY** is the relationship key list of the affected relationship.

**RELATION** is an open node handle to an RD for a relation of the affected relationship.

**ATTRIBUTE** is an open node handle to the AD which identifies the attribute to be set.

**VALUE** is the new value of the attribute in token form.

**STATUS** is an established status indicator.

Exceptions:

**PATHNAME_SYNTAX_ERROR** is raised if the relationship identification given by KEY is syntactically illegal (see Table I, page 52) [2].

**RELATIONSHIP_ERROR** is raised if KEY specifies a value for the same key more than once [5], if a relationship identified by BASE, KEY and RELATION does not exist [2], if more than one such relationship exists [I] or if ATTRIBUTE identifies a key and another relationship is identified by the resulting key values [3].

**NOT_IN_VIEW_ERROR** is raised if a key name specified in KEY cannot be translated [1] or if KEY consists of a single relationship key designator and no key of the specified relation is visible [5].

**AMBIGUOUS_VIEW_ERROR** is raised if translation of a key name specified in KEY is ambiguous [1] or if KEY consists of a single relationship key designator and more than one key of the specified relation is visible [4].

**PREDEFINED_ATTRIBUTE_ERROR** is raised if ATTRIBUTE identifies a CAIS-controlled relationship attribute [2].

**ATTRIBUTE_ERROR** is raised if the relationship does not have the attribute identified by ATTRIBUTE [4].

**STATUS_ERROR** is raised if any of the node handles BASE, RELATION or ATTRIBUTE is not open [2] or if STATUS is not an established status indicator [3].


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

**NODE_KIND_ERROR** is raised if the open node handle RELATION does not designate an RD or if the open node handle ATTRIBUTE does not designate an AD [1]. 15

**CONFORMANCE_ERROR** is raised if the operation would result in an attribute, node or relationship not conforming to its definition or to the constraints in force on that definition [4, 9, 10] or if the attribute is not modifiable [5]. 16

**INTENT_VIOLATION** is raised if BASE was not opened with an intent establishing the right to write relationships [1]. 17

**SECURITY_VIOLATION** is raised if the operation represents a violation of mandatory access controls. SECURITY_VIOLATION is raised only if the conditions for other exceptions are not present. 18

Additional Interfaces:

```
procedure SET_PATH_ATTRIBUTE
    (BASE      : in  NODE_TYPE;
     KEY       : in  RELATIONSHIP_KEY  := EMPTY_KEY_LIST;
     RELATION  : in  RELATION_NAME     := DEFAULT_RELATION;
     ATTRIBUTE : in  ATTRIBUTE_NAME;
     VALUE     : in  TOKEN_TYPE;
     STATUS    : in  STATUS_INDICATOR  := NO_INDICATION)
is
     RELATION_DEFINITION     : RD_TYPE;
     ATTRIBUTE_DEFINITION : IDENTIFIER_AD_TYPE;
begin
     DEFINITION_OF_RELATION (RELATION_DEFINITION, BASE, RELATION, STATUS);
     DEFINITION_OF_PATH_ATTRIBUTE (ATTRIBUTE_DEFINITION,
                                   RELATION_DEFINITION,
                                   ATTRIBUTE, STATUS);
     SET_PATH_ATTRIBUTE (BASE, KEY, RELATION_DEFINITION,
                         ATTRIBUTE_DEFINITION, VALUE, STATUS);
     CLOSE (RELATION_DEFINITION);
     CLOSE (ATTRIBUTE_DEFINITION);
exception
     when others =>
          CLOSE (RELATION_DEFINITION);
          CLOSE (ATTRIBUTE_DEFINITION);
          raise;
end SET_PATH_ATTRIBUTE
```

Notes:

The AD identified by ATTRIBUTE must have its MODIFIABLE attribute set to TRUE. 19

### 5.3.12.6 Getting identifier node attributes

```
procedure GET_NODE_ATTRIBUTE
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
(NODE      : in      NODE_TYPE;
 ATTRIBUTE : in      IDENTIFIER_AD_TYPE;
 VALUE     : in out  TOKEN_TYPE;
 STATUS    : in      STATUS_INDICATOR := NO_INDICATION);
```

Purpose:

This procedure returns the value of the identifier attribute identified by ATTRIBUTE of the node identified by NODE in VALUE.

Parameters:

**NODE** is an open node handle to the node from which the attribute is to be retrieved.

**ATTRIBUTE** is an open node handle to the AD which identifies the attribute to be retrieved.

**VALUE** is the value of the attribute returned (in token form).

**STATUS** is an established status indicator.

Exceptions:

**ATTRIBUTE_ERROR** is raised if the node does not have an attribute identified by ATTRIBUTE [3].

**ITEM_KIND_ERROR** is raised if the value of the attribute is not a single identifier item [1].

**STATUS_ERROR** is raised if either NODE or ATTRIBUTE is not an open node handle [2] or if STATUS is not an established status indicator [3].

**NODE_KIND_ERROR** is raised if the open node handle ATTRIBUTE does not designate an AD [1].

**INTENT_VIOLATION** is raised if NODE was not opened with an intent establishing the right to read attributes [1].

Additional Interfaces:

```
procedure GET_NODE_ATTRIBUTE
    (NODE      : in      NODE_TYPE;
     ATTRIBUTE : in      ATTRIBUTE_NAME;
     VALUE     : in out  TOKEN_TYPE;
     STATUS    : in      STATUS_INDICATOR := NO_INDICATION)
is
     ATTRIBUTE_DEFINITION : IDENTIFIER_AD_TYPE;
begin
     DEFINITION_OF_NODE_ATTRIBUTE (ATTRIBUTE_DEFINITION,
                                   NODE, ATTRIBUTE, STATUS);
     GET_NODE_ATTRIBUTE (NODE, ATTRIBUTE_DEFINITION, VALUE, STATUS);
     CLOSE (ATTRIBUTE_DEFINITION);
exception
     when others =>
          CLOSE (ATTRIBUTE_DEFINITION);
          raise;
end GET_NODE_ATTRIBUTE;
```


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

#### 5.3.12.6 Getting identifier relationship attributes

```
procedure GET_PATH_ATTRIBUTE
    (BASE      : in     NODE_TYPE;
     KEY       : in     RELATIONSHIP_KEY  := EMPTY_KEY_LIST;
     RELATION  : in     RD_TYPE;
     ATTRIBUTE : in     IDENTIFIER_AD_TYPE;
     VALUE     : in out TOKEN_TYPE;
     STATUS    : in     STATUS_INDICATOR  := NO_INDICATION);
```

Purpose:

This procedure returns the value of the identifier attribute identified by ATTRIBUTE of the relationship identified by BASE, KEY and RELATION in VALUE.

Parameters:

**BASE** is an open node handle to the node from which the relationship emanates.

**KEY** is the relationship key list of the relationship.

**RELATION** is an open node handle to the RD of a relation of the relationship.

**ATTRIBUTE** is an open node handle to the AD which identifies the attribute to be retrieved.

**VALUE** is the value of the attribute returned (in token form).

**STATUS** is an established status indicator.

Exceptions:

**PATHNAME_SYNTAX_ERROR** is raised if the relationship identification given by KEY is syntactically illegal (see Table I, page 52) [2].

**RELATIONSHIP_ERROR** is raised if KEY specifies a value for the same key more than once [5], if a relationship identified by BASE, KEY and RELATION does not exist [2] or if more than one such relationship exists [1].

**NOT_IN_VIEW_ERROR** is raised if a key name specified in KEY cannot be translated [1] or if KEY consists of a single relationship key designator and no key of the specified relation is visible [5].

**AMBIGUOUS_VIEW_ERROR** is raised if translation of a key name specified in KEY is ambiguous [1] or if KEY consists of a single relationship key designator and more than one key of the specified relation is visible [4].

**ATTRIBUTE_ERROR** is raised if the relationship does not have the attribute identified by ATTRIBUTE [4].

**ITEM_KIND_ERROR** is raised if the value of the attribute is not a single identifier item [1].

**STATUS_ERROR** is raised if any of the node handles BASE, RELATION, or ATTRIBUTE is not open [2] or if STATUS is not an established status indicator [3].

**NODE_KIND_ERROR** is raised if the open node handle RELATION does not designate an RD or if the open node handle ATTRIBUTE does not designate an AD [1].

**INTENT_VIOLATION** is raised if BASE was not opened with an intent establishing the right to read relationships [1].


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Additional Interfaces:

```
procedure GET_ PATH_ ATTRIBUTE
    (BASE      : in     NODE_TYPE;
     KEY       : in     RELATIONSHIP_KEY := EMPTY_KEY_LIST;
     RELATION  : in     RELATION_NAME  := DEFAULT_RELATION;
     ATTRIBUTE : in     ATTRIBUTE_NAME;
     VALUE     : in out TOKEN_TYPE;
     STATUS    : in     STATUS_INDICATOR := NO_INDICATION)
is
     RELATION_DEFINITION     : RD_TYPE;
     ATTRIBUTE_DEFINITION : IDENTIFIER_AD_TYPE;
begin
     DEFINITION_OF_RELATION (RELATION_DEFINITION, BASE, RELATION, STATUS);
     DEFINITION_OF_PATH_ATTRIBUTE (ATTRIBUTE_DEFINITION,
                                   RELATION_DEFINITION,
                                   ATTRIBUTE, STATUS);
     GET_PATH_ATTRIBUTE (BASE, KEY, RELATION_DEFINITI0N,
                         ATTRIBUTE_DEFINITION, VALUE, STATUS);
     CLOSE (RELATION_DEFINITION);
     CLOSE (ATTRIBUTE_DEFINITION);
exception
     when others =>
          CLOSE (RELATION_DEFINITION);
          CLOSE (ATTRIBUTE_DEFINITION);
          raise;
end GET_PATH_ATTRIBUTE;
```

### 5.3.13 Generic package CAIS_ENUMERATION_ATTRIBUTE

This generic package contains interfaces for manipulating the values of identifier attributes in the form of values of an Ada enumeration type. The package must be instantiated for the Ada enumeration type (indicated by ENUMERATION in the specification). The package may be used to access both ordinary and enumerated identifier attributes. The exceptions raised by all subprograms in this package are defined in the packages CAIS_DEFINITIONS, CAIS_LIST_MANAGEMENT and CAIS_PRAGMATICS. 1

The enumeration form of an identifier attribute value is the value of the Ada type ENUMERATION whose image matches the text form of the identifier value under identifier equality. When an attribute conforming to an enumerated identifier AKD is accessed using this package an exception is raised if the enurneration position of the identifier value (which is the position of the matching pattern in the PATTERNS attribute of the AKD, minus one) is not the same as the enumeration position of the corresponding value of the Ada type ENUMERATION which has the same image (see Section 5.7.l.6.5 on page 320); CONFORMANCE_ERROR is raised for create and set operations while ITEM_KIND_-ERROR is raised for get operations. No position checking is performed in the case of an ordinary identifier AKD. 2

```
generic
```


Source: https://assist.dla.mil -- Downloaded: 2026l0-26T07:54Z
Check the source to verify that this is the currentversion before use.

```
    type ENUMERATION is (<>);
package CAIS_ENUMERATION_ATTRIBUTE is
    -- Specifications of subprograms for this generic package.
end CAIS_ENUMERATION_ATTRIBUTE;
```

#### 5.3.13.1 Creating identifier node attributes

```
procedure CREATE_NODE_ATTRIBUTE
    (NODE      : in  NODE_TYPE;
     ATTRIBUTE : in  IDENTIFIER_AD_TYPE;
     VALUE     : in  ENUMERATION;
     STATUS    : in  STATUS_INDICATOR := NO_INDICATION);
```

Purpose:

This procedure creates the identifier attribute identified by ATTRIBUTE of the node identified by NODE and sets its initial value to VALUE. 1

Parameters:

**NODE** is an open node handle to the node to receive the new attribute. 2

**ATTRIBUTE** is an open node handle to the AD which identifies the attribute to be created. 3

**VALUE** is the initial value of the attribute. 4

**STATUS** is an established status indicator. 5

Exceptions:

**PREDEFINED_ATTRIBUTE_ERROR** is raised if ATTRIBUTE identifies a CAIS-controlled node attribute [1]. 6

**ATTRIBUTE_ERROR** is raised if the node already has the specified attribute [l]. 7

**STATUS_ERROR** is raised if either of the node handles NODE or ATTRIBUTE is not open [2] or if STATUS is not an established status indicator [3]. 8

**NODE_KIND_ERROR** is raised if the open node handle ATTRIBUTE does not designate an AD [1]. 9

**CONFORMANCE_ERROR** is raised if the operation would result in an attribute, node or relationship not conforming to its definition or to the constraints in force on that definition [4, 9, lo]. 10

**INTENT_VIOLATION** is raised if NODE was not opened with an intent establishing the right to create attributes [1]. 11

**SECURITY_VIOLATION** is raised if the operation represents a violation of mandatory access controls. SECURITY_VIOLATION is raised only if the conditions for other exceptions are not present. 12



Source: https://assist.dla.mil -- Downloaded: 2026l0-26T07:54Z
Check the source to verify that this is the currentversion before use.

Additional Interfaces:

```
procedure CREATE_NODE_ATTRIBUTE
    (NODE      : in  NODE_TYPE;
     ATTRIBUTE : in  ATTRIBUTE_NAME;
     VALUE     : in  ENUMERATION;
     STATUS    : in  STATUS_INDICATOR := NO_INDICATION)
is.
     ATTRIBUTE_DEFINITION : IDENTIFIER_AD_TYPE;
begin
     DEFINITION_OF_NODE_ATTRIBUTE (ATTRIBUTE_DEFINITION,
                                   NODE, ATTRIBUTE, STATUS);
     CREATE_NODE_ATTRIBUTE (NODE, ATTRIBUTE_DEFINITION, VALUE, STATUS);
     CLOSE (ATTRIBUTE_DEFINITION);
exception
     when others =>
          CLOSE (ATTRIBUTE_DEFINITION);
          raise;
end CREATE_NODE_ATTRIBUTE;
```

### 5.3.13.2 Creating identifier relationship attributes

```
procedure CREATE_PATH_ATTRIBUTE
    (BASE      : in  NODE_TYPE;
     KEY       : in  RELATIONSHIP_KEY := EMPTY_KEY_LIST;
     RELATION  : in  RD_TYPE;
     ATTRIBUTE : in  IDENTIFIER_AD_TYPE;
     VALUE     : in  ENUMERATION;
     STATUS    : in  STATUS_INDICATOR := NO_INDICATION);
```

Purpose:

This procedure creates the identifier attribute identified by ATTRIBUTE of the relationship identified by BASE, KEY and RELATION and sets its initial value to VALUE.

Parameters:

**BASE** is an open node handle to the node from which the relationship emanates.

**KEY** is the relationship key list of the affected relationship.

**RELATION** is an open node handle to an RD which identifies a relation of the affected relationship.

**ATTRIBUTE** is an open node handle to the AD which identifies the attribute to be created.

**VALUE** is the initial value of the attribute.

**STATUS** is an established status indicator.


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Exceptions:

**PATHNAME_SYNTAX_ERROR** is raised if the relationship identification given by KEY is syntactically illegal (see Table I, page 52) [2]. 8

**NOT_IN_VIEW_ERROR** is raised if a key name specified in KEY cannot be translated [1] or if KEY consists of a single relationship key designator and no key of the specified relation is visible [5]. 9

**AMBIGUOUS_VIEW_ERROR** is raised if translation of a key name specified in KEY is ambiguous [1] or if KEY consists of a single relationship key designator and more than one key of the specified relation is visible [4]. 10

**RELATIONSHIP_ERROR** is raised if KEY specifies a value for the same key more than once [5], if a relationship identified by BASE, KEY and RELATION does not exist [2], if more than one such relationship exists [1] or if ATTRIBUTE identifies a key and another relationship is identified by the resulting key values [3]. 11

**PREDEFINED_ATTRIBUTE_ERROR** is raised if ATTRIBUTE identifies a CAIS-controlled relationship attribute [2]. 12

**ATTRIBUTE_ERROR** is raised if the relationship already has an attribute identified by ATTRIBUTE [2]. 13

**STATUS_ERROR** is raised if any of the node handles BASE, RELATION, or ATTRIBUTE is not open [2] or if STATUS is not an established status indicator [3]. 14

**NODE_KIND_ERROR** is raised if the open node handle RELATION does not designate an RD or if the open node handle ATTRIBUTE does not designate an AD [1]. 15

**CONFORMANCE_ERROR** is raised if the operation would result in an attribute, node or relationship not conforming to its definition or to the constraints in force on that definition [4, 9, 10]. 16

**INTENT_VIOLATION** is raised if BASE was not opened with an intent establishing the right to write relationships [1]. 17

**SECURITY_VIOLATION** is raised if the operation represents a violation of mandatory access controls. SECURITY_VIOLATION is raised only if the conditions for other exceptions are not present. 18

Additional Interfaces:

```
procedure CREATE_PATH_ATTRIBUTE
    (BASE      : in  NODE_TYPE;
     KEY       : in  RELATIONSHIP_KEY  := EMPTY_KEY_LIST;
     RELATION  : in  RELATION_NAME     := DEFAULT_RELATION;
     ATTRIBUTE : in  ATTRIBUTE_NAME;
     VALUE     : in  ENUMERATION;
     STATUS    : in  STATUS_INDICATOR  := NO_INDICATION)
is
    RELATION_DEFINITION : RD_TYPE;
    ATTRIBUTE_DEFINITION : IDENTIFIER_AD_TYPE;
begin
    DEFINITION_OF_RELATION (RELATION_DEFINITION, BASE, RELATION, STATUS) ;
```


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
        DEFINITION_OF_PATH_ATTRIBUTE (ATTRIBUTE_DEFINITION,
                                      RELATION_DEFINITION,
                                      ATTRIBUTE, STATUS);
        CREATE_PATH_ATTRIBUTE (BASE, KEY, RELATION_DEFINITION
                               ATTRIBUTE_DEFINITION, VALUE, STATUS);
        CLOSE (RELATION_DEFINITION);
        CLOSE (ATTRIBUTE_DEFINITION);
    exception
        when others =>
            CLOSE (RELATION_DEFINITION);
            CLOSE (ATTRIBUTE_DEFINITION);
            raise;
    end CREATE_PATH_ATTRIBUTE;
```

#### 5.3.13.3 Setting identifier node attributes

```
procedure SET_NODE_ATTRIBUTE
    (NODE      : in  NODE_TYPE;
     ATTRIBUTE : in  IDENTIFIER_AD_TYPE;
     VALUE     : in  ENUMERATION;
     STATUS    : in  STATUS_INDICATOR := NO_INDICATION);
```

Purpose:

This procedure sets the value of the identifier attribute identified by ATTRIBUTE of the node identified by NODE to the value given by VALUE. 1

Parameters:

**NODE** is an open node handle to the node whose attribute is to beset. 2

**ATTRIBUTE** is an open node handle to the AD which identifies the attribute to be set. 3

**VALUE** is the new value of the attribute. 4

**STATUS** is an established status indicator. 5

Exceptions:

**PREDEFINED_ATTRIBUTE_ERROR** is raised if ATTRIBUTE identifies a CAIS-controlled node attribute [1]. 6

**ATTRIBUTE_ERROR** is raised if the node does not have the attribute identified by ATTRIBUTE [3]. 7

**STATUS_ERROR** is raised if either of the node handles NODE or ATTRIBUTE is not open [2] or if STATUS is not an established status indicator [3]. 8

**NODE_KIND_ERROR** is raised if the open node handle ATTRIBUTE does not designate an AD [1]. 9


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

**CONFORMANCE_ERROR** is raised if the operation would result in an attribute, node or relationship not conforming to its definition or to the constraints in force on that definition [4, 9, 10] or if the attribute is not modifiable [5]. 10

**INTENT_VIOLATION** is raised if NODE was not opened with an intent establishing the right to write attributes [1]. 11

**SECURITY_VIOLATION** is raised if the operation represents a violation of mandatory access controls. SECURITY-VIOLATION is raised only if the conditions for other exceptions are not present. 12

Additional Interfaces:

```
procedure SET_NODE_ATTRIBUTE
    (NODE      : in    NODE_TYPE;
     ATTRIBUTE : in    ATTRIBUTE_NAME;
     VALUE     : in    ENUMERATION;
     STATUS    : in    STATUS_INDICATOR := NO_INDICATION)
is
     ATTRIBUTE_DEFINITION : IDEDITIFIER_AD_TYPE;
begin
     DEFINITION_OF_NODE_ATTRIBUTE (ATTRIBUTE_DEFINITION,
                                   NODE, ATTRIBUTE, STATUS);
     SET_NODE_ATTRIBUTE (NODE, ATTRIBUTE_DEFINITION, VALUE, STATUS);
     CLOSE (ATTRIBUTE_DEFINITION);
exception
     when others =>
          CLOSE (ATTRIBUTE_DEFINITION);
          raise;
end SET_NODE_ATTRIBUTE;
```

Notes:

The AD identified by ATTRIBUTE must have its MODIFIABLE attribute set to TRUE. 13

### 5.3.13.4 Setting identifier relationship attributes

```
procedure SET_PATH_ATTRIBUTE
    (BASE      : in    NODE_TYPE;
     KEY       : in    RELATIONSHIP_KEY := EMPTY_KEY_LIST;
     RELATION  : in    RD_TYPE;
     ATTRIBUTE : in    IDENTIFIER_AD_TYPE;
     VALUE     : in    ENUMERATION;
     STATUS    : in    STATUS_INDICATOR := NO_INDICATION);
```

Purpose:

This procedure sets the value of the identifier attribute identified by ATTRIBUTE of the relationship identified by BASE, KEY and RELATION to the value specified by VALUE. 1


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Parameters:

**BASE** is an open node handle to the node from which the relationship emanates.

**KEY** is the relationship key list of the affected relationship.

**RELATION** is an open node handle to an RD for a relation of the affected relationship.

**ATTRIBUTE** is an open node handle to the AD which identifies the attribute to be set.

**VALUE** is the new value of the attribute.

**STATUS** is an established status indicator.

Exceptions:

**PATHNAME_SYNTAX_ERROR** is raised if the relationship identification given by KEY is syntactically illegal (see Table I, page 52) [2].

**RELATIONSHIP_ERROR** is raised if KEY specifies a value for the same key more than once [5], if a relationship identified by BASE, KEY and RELATION does not exist [2], if more than one such relationship exists [1] or if ATTRIBUTE identifies a key and another relationship is identified by the resulting key values [3].

**NOT_IN_VIEW_ERROR** is raised if a key name specified in KEY cannot be translated [1] or if KEY consists of a single relationship key designator and no key of the specified relation is visible [5].

**AMBIGUOUS_VIEW_ERROR** is raised if translation of a key name specified in KEY is ambiguous [1] or if KEY consists of a single relationship key designator and more than one key of the specified relation is visible [4].

**PREDEFINED_ATTRIBUTE_ERROR** is raised if ATTRIBUTE identifies a CAIS-controlled relationship attribute [2].

**ATTRIBUTE_ERROR** is raised if the relationship does not have the attribute identified by ATTRIBUTE [4].

**STATUS_ERROR** is raised if any of the node handles BASE, RELATION or ATTRIBUTE is not open [2] or if STATUS is not an established status indicator [3].

**NODE_KIND_ERROR** is raised if the open node handle RELATION does not designate an RD or if the open node handle ATTRIBUTE does not designate an AD [1].

**CONFORMANCE_ERROR** is raised if the operation would result in an attribute, node or relationship not conforming to its definition or to the constraints in force on that definition [4, 9, 10] or if the attribute is not modifiable [5].

**INTENT_VIOLATION** is raised if BASE was not opened with an intent establishing the right to write relationships [1].

**SECURITY_VIOLATION** is raised if the operation represents a violation of mandatory access controls. SECURITY_VIOLATION is raised only if the conditions for other exceptions are not present.

Additional Interfaces:


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
procedure SET_PATH_ATTRIBUTE
    (BASE      : in  NODE_TYPE;
     KEY       : in  RELATIONSHIP_KEY  := EMPTY_KEY_LIST;
     RELATION  : in  RELATION_NAME     := DEFAULT_RELATION;
     ATTRIBUTE : in  ATTRIBUTE_NAME;
     VALUE     : in  ENUMERATION;
     STATUS    : in  STATUS_INDICATOR  := NO_INDICATION)
is
     RELATION_DEFINITION     : RD_TYPE;
     ATTRIBUTE_DEFINITION : IDENTIFIER_AD_TYPE;
begin
     DEFINITION_OF_RELATION (RELATION_DEFINITION, BASE, RELATION, STATUS);
     DEFINITION_OF_PATH_ATTRIBUTE (ATTRIBUTE_DEFINITION,
                                   RELATION_DEFINITION,
                                   ATTRIBUTE, STATUS);
     SET_PATH_ATTRIBUTE (BASE, KEY, RELATION_DEFINITION,
                         ATTRIBUTE_DEFINITION, VALUE, STATUS);
     CLOSE (RELATION_DEFINITION);
     CLOSE (ATTRIBUTE_DEFINITION);
exception
     when others =>
          CLOSE (RELATION_DEFINITION);
          CLOSE (ATTRIBUTE_DEFINITION);
          raise;
end SET_PATH_ATTRIBUTE;
```

Notes:

The AD identified by ATTRIBUTE must have its MODIFIABLE attribute set to TRUE. 19

### 5.3.13.5 Getting identifier node attributes

```
function NODE_ATTRIBUTE
    (NODE      : in  NODE_TYPE;
     ATTRIBUTE : in  IDENTIFIER_AD_TYPE;
     STATUS    : in  STATUS_INDICATOR  := NO_INDICATION)
    return ENUMERATION;
```

Purpose:

This function returns the value of the identifier attribute identified by ATTRIBUTE of the node identified by NODE. 1

Parameters:

**NODE** is an open node handle to the node from which the attribute is to be retrieved. 2

**ATTRIBUTE** is an open node handle to the AD which identifies the attribute to be retrieved. 3

**STATUS** is an established status indicator. 4


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Exceptions:

**ATTRIBUTE_ERROR** is raised if the node does not have an attribute identified by ATTRIBUTE [3]. 5

**ITEM_KIND_ERROR** is raised if the value of the attribute is not a single identifier item [1], if there is no enumeration value of the type ENUMERATION corresponding (by image) to the identifier value [2] or if there is such a corresponding enumeration value and the AKD for the attribute is an enumerated identifier AKD but the position of the enumeration value in the base type of ENUMERATION is not the same as the enumeration position of the identifier value determined by the PATTERNS attribute of the AKD [3]. 6

**STATUS_ERROR** is raised if either NODE or ATTRIBUTE is not an open node handle [2] or if STATUS is not an established status indicator [3]. 7

**NODE_KIND_ERROR** is raised if the open node handle ATTRIBUTE does not designate an AD [1]. 8

**INTENT_VIOLATION** is raised if NODE was not opened with an intent establishing the right to read attributes [1]. 9

Additional Interfaces:

```
function NODE_ATTRIBUTE
    (NODE      : in  NODE_TYPE;
     ATTRIBUTE : in  ATTRIBUTE_NAME;
     STATUS    : in  STATUS_INDICATOR := NO_INDICATION)
    return ENUMERATION
is
     ATTRIBUTE_DEFINITION : IDENTIFIER_AD_TYPE;
     VALUE                : ENUMERATION;
begin
     DEFINITION_OF_NODE_ATTRIBUTE (ATTRIBUTE_DEFINITION,
                                   NODE, ATTRIBUTE, STATUS);
     VALUE := NODE_ATTRIBUTE (NODE, ATTRIBUTE_DEFINITION, STATUS);
     CLOSE (ATTRIBUTE_DEFINITION);
     return (VALUE);
exception
     when others =>
          CLOSE (ATTRIBUTE_DEFINITION);
          raise;
end NODE_ATTRIBUTE;
```

### 5.3.13.6 Getting identifier relationship attributes

```
function PATH_ ATTRIBUTE
    (BASE      : in  NODE_TYPE;
     KEY       : in  RELATIONSHIP_KEY := EMPTY_KEY_LIST;
     RELATION  : in  RD_TYPE;
     ATTRIBUTE : in  IDENTIFIER_AD_TYPE;
     STATUS    : in  STATUS_INDICATOR := NO_INDICATION)
    return ENUMERATION;
```


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Purpose:

This function returns the value of the identifier attribute identified by ATTRIBUTE of the relationship identified by BASE, KEY and RELATION.

Parameters:

**BASE** is an open node handle to the node from which the relationship emanates.

**KEY** is the relationship key list of the relationship.

**RELATION** is an open node handle to the RD of a relation of the relationship.

**ATTRIBUTE** is an open node handle to the AD which identifies the attribute to be retrieved.

**STATUS** is an established status indicator.

Exceptions:

**PATHNAME_SYNTAX_ERROR** is raised if the relationship identification given by KEY is syntactically illegal (see Table 1, page 52) [2].

**RELATIONSHIP_ERROR** is raised if KEY specifies a value for the same key more than once [5], if a relationship identified by BASE, KEY and RELATION does not exist [2] or if more than one such relationship exists [1].

**NOT_IN_VIEW_ERROR** is raised if a key name specified in KEY cannot be translated [1] or if KEY consists of a single relationship key designator and no key of the specified relation is visible [5].

**AMBIGUOUS_VIEW_ERROR** is raised if translation of a key name specified in KEY is ambiguous [1] or if KEY consists of a single relationship key designator and more than one key of the specified relation is visible [4].

**ATTRIBUTE_ERROR** is raised if the relationship does not have the attribute identified by ATTRIBUTE [4].

**ITEM_KIND_ERROR** is raised if the value of the attribute is not a single identifier item [1], if there is no enumeration value of the type ENUMERATION corresponding (by image) to the identifier value [2] or if there is such a corresponding enumeration value and the AKD for the attribute is an enumerated identifier AKD but the position of the enumeration value in the base type of ENUMERATION is not the same as the enumeration position of the identifier value determined by the PATTERNS attribute of the AKD [3].

**STATUS_ERROR** is raised if any of the node handles BASE, RELATION, or ATTRIBUTE is not open [2] or if STATUS is not an established status indicator [3].

**NODE_KIND_ERROR** is raised if the open node handle RELATION does not designate an RD or if the open node handle ATTRIBUTE does not designate an AD [1].

**INTENT_VIOLATION** is raised if BASE was not opened with an intent establishing the right to read relationships [1].


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Additional Interfaces:

```
function PATH. ATTRIBUTE
    (BASE      : in    NODE_TYPE;
     KEY       : in    RELATIONSHIP_KEY  := EMPTY_KEY_LIST;
     RELATION  : in    RELATION_NAME     := DEFAULT_RELATION;
     ATTRIBUTE : in    ATTRIBUTE_NAME;
     STATUS    : in    STATUS_INDICATOR  := NO_INDICATION)
   return ENUMERATION
is
     RELATION_DEFINITION    : RD_TYPE;
     ATTRIBUTE_DEFINITION   : IDENTIFIER_AD_TYPE;
     VALUE                  : ENUMERATION;
begin
     DEFINITION_OF_RELATION (RELATION_DEFINITION, BASE, RELATION, STATUS);
     DEFINITION_OF_PATH_ATTRIBUTE (ATTRIBUTE_DEFINITION,
                                   RELATION_DEFINITION,
                                   ATTRIBUTE, STATUS);
     VALUE := PATH_ATTRIBUTE (BASE, KEY, RELATION_DEFINITION,
                              ATTRIBUTE_DEFINITION, STATUS);
     CLOSE (RELATION_DEFINITION);
     CLOSE (ATTRIBUTE_DEFINITION);
     return (VALUE);
exception
     when others =>
          CLOSE (RELATION_DEFINITION);
          CLOSE (ATTRIBUTE_DEFINITION);
          raise;
end PATH_ATTRIBUTE;
```

### 5.3.14 Package CAIS_BOOLEAN_ATTRIBUTE

This library package contains interfaces for manipulating the values of Boolean attributes (attributes conforming to the predefined BOOLEAN AKD). This package is defined by the following instantiation of package CAIS_ENUMERATION_ATTRIBUTE, however it need not be implemented using this instantiation so long as the package specification and semantics are the same as those implied by the instantiation: 1

```
package CAIS_BOOLEAN_ATTRIBUTE is
   new CAIS_ENUMERATION_ATTRIBUTE (ENUMERATION => BOOLEAN);
```

Note that CAIS_BOOLEAN_ATTRIBUTE is instantiated as a library package rather than as a sub-package of CAIS_ATTRIBUTE_MANAGEMENT. 2

### 5.3.15 Attribute iteration types and subtypes

These types and subtypes are used in the following interfaces for iteration over a set of attributes of 1


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

nodes or relationships.

```
type ATTRIBUTE_ITERATOR is limited private;
```

An *attribute iterator* is an Ada object of the type ATTRIBUTE_ITERATOR, which is a limited private type assumed to contain the bookkeeping information necessary for the implementation of the MORE, NEXT_NAME, GET_NEXT_VALUE and SKIP_NEXT interfaces. 2

```
subtype ATTRIBUTE_NAME_PATTERN is STRING;
```

An ATTRIBUTE_NAME_PATTERN has the same syntax as an ATTRIBUTE_NAME, except that "?" will match any single character and "*" will match any string of zero or more characters. 3

```
type ATTRIBUTE_ITERATOR_SCOPE is (VISIBLE_ATTRIBUTES, ALL_ATTRIBUTES);
```

An ATTRIBUTE_ITERATOR_SCOPE determines the set of attributes eligible for inclusion in an attribute iterator. The value VISIBLE_ATTRIBUTES indicates that only visible attributes are to be included, while the value ALL_ATTRIBUTES indicates that all attributes are to be included. 4

### 5.3.16 Creating an iterator over node attributes

```
procedure CREATE_NODE_ATTRIBUTE_ITERATOR
    (ITERATOR : in out ATTRIBUTE_ITERATOR;
     NODE     : in     NODE_TYPE;
     PATTERN  : in     ATTRIBUTE_NAME_PATTERN : = "*";
     SCOPE    : in     ATTRIBUTE_ITERATOR_SCOPE : = VISIBLE_ATTRIBUTES ;
     STATUS   : in     STATUS_INDICATOR         := NO_INDICATION);
```

Purpose:

This procedure establishes the parameter ITERATOR as an attribute iterator over attributes of the node identified by the open node handle NODE. The iterator can then be processed by means of the MORE, NEXT_NAME, GET_NEXT_VALUE and SKIP_NEXT interfaces. 1

Attributes eligible for inclusion in the iteration are selected according to the SCOPE parameter. If the value of SCOPE is VISIBLE_ATTRIBUTES, only visible attributes are eligible for inclusion in the iteration. In this case, an eligible attribute is included only if its name matches the pattern specified in PATTERN; the included attributes are returned by the iteration interfaces in ASCII lexicographical order by attribute name. If formation of a name for an included attribute is ambiguous, the position of that attribute in the iteration ordering is implementation-defined, 2

If the value of the SCOPE parameter is ALL_ATTRIBUTES, all attributes of the node are included, whether or not they are visible. In this case, the value of the PATTERN parameter has no effect on the iteration; the ordering of the included attributes is implementation-defhed. 3

Each attribute is included at most once in the iteration, Predefine attributes for mandatory access control labels are never included in an attribute iteration. 4


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Attribute names are formed as described in Section 5.7.3.6 on page 355. The context for formation of attribute names is determined by the node handle NODE.

If the node handle NODE has a current adopted interpretation, that interpretation is adopted by the iterator and remains adopted until the iterator is deleted. Attribute names are formed using that adopted interpretation. Adoption of the interpretation by the iterator causes a lock with intent READ_RELATIONSHIPS to be acquired on the VIEW node of the view containing the interpretation. This lock is not released until the iterator is deleted. Changing the current adopted interpretation for the node handle NODE after the creation of the iterator has no effect on the iteration.

If the node handle NODE has no adopted interpretation, the composite view is used to form names in the context of the NKD for the node. In this case, changing the composite view of the current process after creation of the iterator has implementation-defined effects on the order and content of the iteration, if the SCOPE is VISIBLE_ATTRIBUTES, and on the formation of names returned by NEXT_NAME.

The effect on existing attribute iterators of the deletion of the node, or the creation, modification or deletion of attributes of the node is implementation-defined.

Parameters:

**ITERATOR** is an unestablished attribute iterator to be established.

**NODE** is an open node handle to a node over whose attributes the iterator is to be established.

**PATTERN** is a pattern for attribute names as described in Section 5.3.15 on page 245,

**SCOPE** is an enumeration value that determines the scope of the resultant attribute iterator.

**STATUS** is an established status indicator.

Exceptions:

**ITERATOR_ERROR** is raised if ITERATOR is already established [2].

**SYNTAX_ERROR** is raised if the PATTERN is syntactically illegal (see Section 4.3.3 on page 43 and Section 5.3.15 on page 245) [6].

**STATUS_ERROR** is raised if NODE is not an open node handle [2] or if STATUS is not an established status indicator [3].

**INTENT_VIOLATION** is raised if NODE was not opened with an intent establishing the right to read attributes [1].

### 5.3.17 Creating an iterator over relationship attributes

```
procedure CREATE_PATH_ATTRIBUTE_ITERATOR
    (ITERATOR : in out ATTRIBUTE_ITERATOR;
     BASE     : in     NODE_TYPE;
     KEY      : in     RELATIONSHIP_KEY           := EMPTY_KEY_LIST;
     RELATION : in     RD_TYPE;
     PATTERN  : in     ATTRIBUTE_NAME_PATTERN : = "*";
     SCOPE    : in     ATTRIBUTE_ITERATOR_SCOPE : = VISIBLE_ATTRIBUTES;
     STATUS   : in     STATUS_INDICATOR           := NO_INDICATION);
```


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Purpose:

This procedure establishes the parameter ITERATOR as an attribute iterator over attributes of the relationship identified by the parameters BASE, KEY and RELATION. The iterator can then be processed by means of the MORE, NEXT_NAME, GET_NEXT_VALUE and SKIP_NEXT interfaces. 1

Attributes eligible for inclusion in the iteration are selected according to the SCOPE parameter. If the value of SCOPE is VISIBLE_ATTRIBUTES, only visible attributes are eligible for inclusion in the iteration. In this case, an eligible attribute is included only if its name matches the pattern specified in PATTERN; the included attributes are returned by the iteration interfaces in ASCII lexicographical order by attribute name. If formation of a name for an included attribute is ambiguous, the position of that attribute in the iteration ordering is implementation-defined. 2

If the value of the SCOPE parameter is ALL_ATTRIBUTES, all attributes of the relationship are included, whether or not they are visible. In this case, the value of the PATTERN parameter has no effect on the iteration; the order of the attributes included in the iteration is implementation-defined. 3

Each attribute is included at most once in the iteration. 4

Attribute names are formed as described in Section 5.7.3.6 on page 355. The context for formation of attribute names is determined by the node handle NODE. 5

If the base node handle BASE has a current adopted interpretation, that interpretation is adopted by the iterator and remains adopted until the iterator is deleted. Names are formed using that adopted interpretation. Adoption of the interpretation by the iterator causes a lock with intent READ_RELATIONSHIPS to be acquired on the VIEW node of the view containing the interpretation. This lock is not released until the iterator is deleted. Changing the current adopted interpretation for the base node handle BASE after the creation of the iterator has no effect on the iteration. 6

If the base node handle BASE has no adopted interpretation, the composite view is used to form names in the context of the NKD for the base node. In this case, changing the composite view of the current process after creation of the iterator has implementation-defined effects on the order and content of the iteration, if the SCOPE is VISIBLE_ATTRIBUTES, and on the formation of names returned by NEXT_NAME. 7

The effect on existing attribute iterators of the deletion of the base node, deletion of the relationship, or the creation, modification or deletion of attributes of the relationship is implementation-defined. 8

Parameters:

**ITERATOR** is an unestablished attribute iterator to be established. 9

**BASE** is an open node handle to the node from which the relationship emanates. 10

**KEY** is the relationship key list of the relationship. 11

**RELATION** is an open node handle for an RD which identifies a relation of the relationship. 12

**PATTERN** is a pattern for attribute names (see Section 5.3.15 on page 245). 13

**SCOPE** is an enumeration value that determines the scope of the resultant attribute iterator. 14

**STATUS** is an established status indicator. 15



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Exceptions:

**ITERATOR_ERROR** is raised if ITERATOR is already established [z]. 16

**PATHNAME_SYNTAX_ERROR** is raised if the relationship identification given by KEY is syntactically illegal (see Table I, page 52) [2]. 17

**NOT_IN_VIEW_ERROR** is raised if a key name specified in KEY cannot be translated [I] or if KEY consists of a single relationship key designator and no key of the specified relation is visible [5]. 18

**AMBIGUOUS_VIEW_ERROR** is raised if translation of a key name specified in KEY is ambiguous [1] or if KEY consists of a single relationship key designator and more than one key of the specified relation is visible [4]. 19

**RELATIONSHIP_ERROR** is raised if KEY specifies a value for the same key more than once [5], if a relationship identified by BASE, KEY and RELATION does not exist [2] or if more than one such relationship exists [1]. 20

**SYNTAX_ERROR** is raised if PATTERN is syntactically illegal (see Section 4.3.3 on page 43 and Section 5.3.15 on page 245) [6]. 21

**STATUS_ERROR** is raised if BASE or RELATION is not an open node handle [2] or if STATUS is not an established status indicator [3]. 22

**NODE_KIND_ERROR** is raised if the open node handle RELATION does not designate an RD [1]. 23

**INTENT_VIOLATION** is raised if BASE was not opened with an intent establishing the right to read relationships [1]. 24

Additional Interfaces:

```
procedure CREATE_PATH_ATTRIBUTE_ITERATOR
    (ITERATOR : in out ATTRIBUTE_ITERATOR;
     BASE     : in     NODE_TYPE;
     KEY      : in     RELATIONSHIP_KEY           : = EMPTY_KEY_LIST;
     RELATION : in     RELATION_NAME              : = DEFAULT_RELATION;
     PATTERN  : in     ATTRIBUTE_NAME_PATTERN     := "*":
     SCOPE    : in     ATTRIBUTE_ITERATOR_SCOPE : = VISIBLE_ATTRIBUTES;
     STATUS   : in     STATUS_INDICATOR           : = NO_INDICATION)
is
     RELATION_DEFINITION : RD_TYPE;
begin
     DEFINITION_OF_RELATION (RELATION_DEFINITION, BASE, RELATION, STATUS);
     CREATE_PATH_ATTRIBUTE_ITERATOR (ITERATOR, BASE, KEY,
                                     RELATION_DEFINITION, PATTERN,
                                     SCOPE, STATUS) ;
     CLOSE (RELATION_DEFINITION);
exception
     when others =>
          CLOSE (RELATION_DEFINITION);
          raise;
end CREATE_PATH_ATTRIBUTE_ITERATOR;
```


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

### 5.3.18 Determining iteration status

```
function MORE
    (ITERATOR : in     ATTRIBUTE_ITERATOR;
     STATUS   : in     STATUS_INDICATOR := NO_INDICATION)
    return BOOLEAN;
```

Purpose:

This function returns FALSE if all attributes contained on the attribute iterator have been retrieved with the subprograms SKIP_NEXT and GET_NEXT_VALUE; otherwise, it returns TRUE. 1

Parameters:

**ITERATOR** is an established attribute iterator. 2

**STATUS** is an established status indicator. 3

Exceptions:

**ITERATOR_ERROR** is raised if ITERATOR is not established (see procedures CREATE_NODE_ATTRIBUTE_ITERATOR Section 5.3.16 on page 246 and CREATE_PATH_ATTRIBUTE_ITERATOR Section 5.3.17 on page 247) [1]. 4

**STATUS_ERROR** is raised if STATUS is not an established status indicator [3]. 5

### 5.3.19 Determining the approximate size of the iterator

```
function APPROXIMATE_SIZE
    (ITERATOR : in     ATTRIBUTE_ITERATOR;
     STATUS   : in     STATUS_INDICATOR := NO_INDICATION)
    return CAIS_NATURAL;
```

Purpose:

This function returns the approximate number of elements on the attribute iterator at the moment of the call. Calls on NEXT_NAME, GET_NEXT_VALUE or SKIP_NEXT have no influence on the value returned by this function. 1

Parameters:

**ITERATOR** is an established attribute iterator. 2

**STATUS** is an established status indicator. 3



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Exceptions:

**ITERATOR_ERROR** is raised if ITERATOR is not established (see procedures CREATE_NODE_ATTRIBUTE_ITERATOR Section 5.3.16 on page 246 and CREATE_PATH_ATTRIBUTE_ITERATOR Section 5.3.17 on page 247) [1]. 4

**STATUS_ERROR** is raised if STATUS is not an established status indicator [3].

Notes:

This interface should not be used in loops of the form:

```
for I in 1 . . APPROXIMATE_SIZE (ITERATOR) loop
    NEXT_ATTRIBUTE_NAME : = NEXT_NAME (ITERATOR);
    GET_NEXT_VALUE (ITERATOR, NEXT_ATTRIBUTE_VALUE);
end loop;
```

since the deletion of attributes may reduce the number of attributes returned by the repeated calls on NEXT_NAME and GET. NEXT_VALUE.

### 5.3.20 Getting an identification for the next attribute

```
procedure NEXT_NAME
    (ATTRIBUTE : in out AD_TYPE;
     ITERATOR : in ATTRIBUTE_ITERATOR;
     STATUS : in STATUS_INDICATOR := NO_INDICATION);

function NEXT_NAME
    (ITERATOR : in ATTRIBUTE_ITERATOR;
     STATUS : in STATUS_INDICATOR := NO_INDICATION)
    return ATTRIBUTE_NAME;
```

Purpose:

This interface returns an identification of the next attribute on the attribute iterator without advancing the iterator. In the first form of this interface, an open node handle with intent NO_ACCESS identifying the AD for the attribute is returned in ATTRIBUTE. IN the second form of this interface, an attribute name identifying the next attribute is returned as the result of the function. 1

For the first interface, if the node handle ATTRIBUTE is already open when NEXT_NAME is called, it is closed and then opened to the AD for the next attribute. The handle is opened with intent NO_ACCESS. No path is traversed in opening the handle; hence, the subprograms PATH_KEY (Section 5.2.20 on page 119) and PATH_RELATION (Section 5.2.21 on page 120 cannot be applied to the node handle ATTRIBUTE. 2

The attribute name returned by the second interface is formed according to the semantics of name formation using fews as described in Section 5.7.3.6 on page 355. If the (base) node handle had an adopted interpretation at the time of iterator creation, the attribute name is formed in 3



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

the context of that adopted interpretation (see procedure CREATE_NODE_ATTRIBUTE_ITERATOR Section 5.3.16 on page 246 and procedure CREATE. PATH_ATTRIBUTE_ITERATOR Section 5.3.17 on page 247) Otherwise, the attribute name is formed in the context of the node kind or relation using the composite view of the current process. In the latter case, changes to the composite view after the iterator is created have implementation-defined effects on the formation of attribute names.

Parameters:

**ATTRIBUTE** is a node handle to be opened to the AD for the next attribute on the iterator ITERATOR.

**ITERATOR** is an established attribute iterator.

**STATUS** is an established status indicator.

Exceptions:

**NOT_IN_VIEW_ERROR** is raised if an attribute name is to be returned and such a name cannot be formed [3].

**AMBIGUOUS_VIEW_ERROR** is raised if an attribute name is to be returned and formation of an attribute name is ambiguous [2] or if translation of the attribute name formed is ambiguous [3].

**ITERATOR_ERROR** is raised if ITERATOR is not established (see procedures CREATE_NODE_ATTRIBUTE_ITERATOR Section 5.3.16 on page 246 and CREATE_PATH_ATTRIBUTE_ITERATOR Section 5.3.17) page 247) [1], or if ITERATOR is exhausted [3].

**STATUS_ERROR** is raised if STATUS is not an established status indicator [3].

### 5.3.21 Getting the next attribute value

```
procedure GET_NEXT_VALUE
    (ITERATOR : in out ATTRIBUTE_ITERATOR;
     VALUE    : in out LIST_TYPE;
     STATUS   : in     STATUS_INDICATOR := NO_INDICATION);
```

Purpose:

This procedure returns the value of the next attribute on the iterator in VALUE in the form of a list and in accordance with the rules given in Section 5.3 on page 185, and then advances the iterator to the next attribute on the iterator (i.e., the one corresponding to the returned value).

Parameters:

**ITERATOR** is an established attribute iterator.

**VALUE** is the value of the next attribute on the iterator in the form of a list.

**STATUS** is an established status indicator.



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Exceptions:

**ITERATOR_ERROR** is raised if ITERATOR is not established (see procedures CREATE_-NODE_ATTRIBUTE_ITERATOR Section 5.3.16 on page 246 and CREATE_PATH_ATTRIBUTE_ITERATOR Section 5.3.17 on page 247) [1], or if ITERATOR is exhausted [3].

**STATUS_ERROR** is raised if STATUS is not an established status indicator [3].

### 5.3.22 Skipping the next attribute in an iteration

```
procedure SKIP_NEXT
    (ITERATOR : in out ATTRIBUTE_ITERATOR;
     STATUS   : in     STATUS_INDICATOR := NO_INDICATION);
```

Purpose:

This procedure advances the iterator to the next attribute on the iterator.

Parameters:

**ITERATOR** is an established attribute iterator.

**STATUS** is an established status indicator.

Exceptions:

**ITERATOR_ERROR** is raised if ITERATOR is not established (see procedures CREATE_-NODE_ATTRIBUTE_ITERATOR Section 5.3.16 on page 246 and CREATE_PATH_ATTRIBUTE_ITERATOR Section 5.3.17 on page 247) [1], or if ITERATOR is exhausted [3].

**STATUS_ERROR** is raised if STATUS is not an established status indicator [3]

### 5.3.23 Deleting an attribute iterator

```
procedure DELETE_ITERATOR
    (ITERATOR : in out ATTRIBUTE_ITERATOR;
     STATUS   : in     STATUS_INDICATOR : = NO_INDICATION);
```

Purpose:

This procedure deletes an attribute iterator. The value of the ITERATOR parameter after the call is unestablished, as if it were never established by the procedure CREATE_NODE. -ATTRIBUTE_ITERATOR (Section 5.3.16 on page 246) or CREATE_PATH_ATTRIBUTE_-ITERATOR (Section 5.3.17 on page 247). Deleting an iterator that is unestablished has no effect.


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Parameters:

**ITERATOR** is an attribute iterator. 2

**STATUS** is an established status indicator. 3

Exceptions:

**STATUS_ERROR** is raised if STATUS is not an established status indicator [3]. 4



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

# 5.4 Package CAIS_ACCESS_CONTROL_MANAGEMENT

This package provides primitives for manipulating access control information for nodes. In addition, certain subprograms declared elsewhere (e.g., the node creation interfaces and the node open interfaces) allow the specification of initial access control information. The exceptions raised by all subprograms in this package are defined in the packages CAIS_DEFINITIONS and CAIS_PRAGMATICS.

## 5.4.1 Node access

In the CAIS, the following classes of operations constitute *access to a node:*

a. opening a node with any intent except NO_ACCESS,

b. reading or writing the contents of a node,

c. reading or writing attributes of a node, and

d. reading or writing relationships emanating from a node (including reading or writing attributes of such relationships).

In the CAIS, the following classes of operations do not constitute access to a node:

a. opening a node with intent NO_ACCESS,

b. querying the kind of a node,

c. reading or writing relationships for which the node in question is a target (including reading or writing attributes of such relationships),

d. querying the status of node handles to the node, and

e. closing node handles to the node.

The phrase “to obtain access” to a node means to be permitted by access checking to perform certain operations on the node. Access to a node by means of a pathname can only be achieved if the current process has the respective access rights to the node as well as the right to read the relationships emanating from nodes along the path to the node.

## 5.4.2 Security labeling of CAIS nodes

The CAIS provides interfaces to allow tools to operate within a Trusted Computer System (TCS) that meets the B3 criteria as defined in [DOD5200.28-STD]. All CAIS nodes must be explicitly or implicitly associated with a security label. Labels exported from any CAIS implementation "must be accurate representations of the corresponding internal sensitivity labels being exported." [DOD5200.28-STD]

When implemented within a TCS, CAIS supports operations in a multi-level secure (MLS) environment. Nodes in the MLS environment can be associated with different security levels. Each *security level* represents the combination of a hierarchical classification and a set of non-hierarchical categories that represents the information’s sensitivity. Explicit security labels on nodes must correctly represent these levels.



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

The labeling of nodes is provided through node attributes. The attribute SUBJECT_CLASSIFICATION is assigned to each process node and is the process' classification as a subject. The attribute OBJECT_CLASSIFICATION is assigned to each node and is the node's classification as an object. These attributes are initially set at node creation using CAIS interfaces. The capability to read, write, or modify these attributes is under the control of the CAIS implementation according to its security policies. 3

The value of these attributes is a list containing one or two items: the hierarchical classification level and, optionally, the non-hierarchical category list. The hierarchical classification is a keyword member of the ordered set of hierarchical classification keywords. The non-hierarchical category list is a list of one or more keyword members of the set of non-hierarchical categories. Both the hierarchical classification level set and the non-hierarchical category set are implementation-defined. The BNF for the value of a classification attribute (and of the parameter which provides it at node creation) is given in Table IX. 4

TABLE IX: **Classification Attribute Value BNF**

| | |
|---|---|
| object_classification | ::= classification |
| subject_classification | ::= classification |
| classification | ::= ( hierarchical. classification [, non_hierarchical_categories] ) |
| hierarchical_classification | ::= keyword |
| non_hierarchical_categories | ::= ( keyword{, keyword} ) |
| keyword | ::= identifier |

Notes:

1. See Appendix D for a description of the notation used.

Labels on nodes are relevant to any access checking involving the comparison of hierarchical or non-hierarchical attribute values. Mandatory access checking always involves label comparisons. 5

When not implemented within a TCS, those CAIS interfaces sensitive to security operate as on a dedicated system. All nodes are explicitly or implicitly labeled at a single level, and all users are cleared to at least that level. 6

Specification of labeling in the CAIS is as follows. 7

a. Labeling of root PROCESS Nodes at Logon: When a root PROCESS node is created at logon, it is assigned subject and object classification labels. The method by which these initial labels are assigned is not specified; however, the labels "shall accurately represent security levels of the specific [users] with which they are associated." [DOD5200.28-STD] In addition, if a user has the capability to function at several different classification levels, the CAIS implementation is responsible for maintaining the isolation and integrity of individual sessions. 8

b. Labeling of root PROCESS Nodes Created through the CAIS: When a PROCESS node is created through the CAIS interfaces, the creating process may specify the values for the 9



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

created PROCESS node's classification attributes. The resulting classification must adhere to the requirements for mandatory access control over write operations. If no classification is specified, the classification attributes of the root process created are inherited (copied) from the creating process. When a dependent PROCESS node is created, the classification attributes of the created process are inherited from the root process of the current job.

c. Labeling of Non-PROCESS Nodes: When a non-PROCESS node is created, it is assigned an object classification label. This label must adhere to the requirements for mandatory access control over write operations. The classification label may be specified in the create operation, or it may be inherited from the creating process. 10

d. Labeling of Nodes for Devices: DEVICE nodes may have a range of classification levels. The range of classification levels is specified by two node attributes. The attribute HIGHEST_CLASSIFICATION defines the highest allowable OBJECT_CLASSIFICATION label that may be assigned to the device. The attribute LOWEST_CLASSIFICATION defines the lowest allowable OBJECT_CLASSIFICATION label that may be assigned to the device. When the DEVICE node is opened, it inherits its OBJECT_CLASSIFICATION label from the first process performing the open operation. If it is not possible to label the DEVICE node within the bounds of the attributes HIGHEST_CLASSIFICATION and LOWEST_CLASSIFICATION, the operation fails. Only when all open node handles to the node have been closed can a new classification label be assigned to the node. The capability to read, write, or modify the attributes HIGHEST_CLASSIFICATION and LOWEST_CLASSIFICATION is under the control of the CAIS implementation according to its security policies. 11

### 5.4.3 Groups and roles

The CAIS node model incorporates the concepts of groups and roles. A group is a set of users and/or tools that is represented by a GROUP node. Group membership of a subject determines the set of access rights it has or can acquire with respect to objects. GROUP nodes are organized in GROUP node trees. Each top-level GROUP node is reachable from the system-level node via a primary GROUP relationship. The key of this relationship is the group name. The primary relationship of a GROUP node other than a top-level GROUP node must be of the relation PERMANENT_MEMBER emanating from another GROUP node. Upon creation of a root PROCESS node, secondary GROUP relationships are created emanating from the PROCESS node and terminating at an implementation-defined subset of top-level GROUP nodes. The keys of these relationships are the respective group names. Dependent PROCESS nodes can inherit these relationships. 1

GROUP nodes, their attributes and their emanating relationships cannot be modified by means of CAIS interfaces. It is left to each CAIS implementation to provide interfaces for the creation, modification, or deletion of GROUP nodes. The effects of the deletion of GROUP nodes and of alterations of group memberships on concurrently executing processes are implementation-defined. 2

The CAIS associates a role with each GROUP node. The *role* associated with a GROUP node is the set of all ACCESS relationships and all DENY relationships which emanate from nodes and terminate at this GROUP node or any of the GROUP nodes on its primary path. ACCESS and DENY relationships provide information from which access rights of a subject to an object are determined as further described in Section 5.4.6.1 on page 262. 3



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

#### 5.4.3.1 Default roles

Secondary DEFAULT_ROLE relationships are used to associate GROUP nodes with all USER nodes and some nodes containing the executable images of programs. Each USER node has a DEFAULT_-ROLE relationship emanating from it and terminating at a GROUP node. An executable image of a program may also be associated with a GROUP node by means of a DEFAULT_ROLE relationship emanating from its node and terminating at the GROUP node. This GROUP node is termed the *default GROUP node* of the user or program, respectively. No node may have multiple default GROUP nodes. It is left to each CAIS implementation to provide interfaces for the creation, modification, or deletion of DEFAULT_ROLE relationships. The effects of the deletion or alteration of DEFAULT_-ROLE relationships on concurrently executing processes are implementation-defined.

#### 5.4.3.2 Adopting a role

A process is said to execute “under the authority of its adopted roles”. When a process adopts a particular role, that process acquires the access rights which have been or will be granted to adopters of that role. An ADOPTED_ROLE relationship is created from the PROCESS node to a GROUP node, allowing the process to act under the authority of the role associated with that group. The GROUP node and the role associated with it are therefore said to be adopted by that process.

GROUP nodes can have secondary POTENTIAL_MEMBER relationships emanating from them to other GROUP nodes. A given GROUP node is a *potential member* of another group if the given GROUP node is reachable from the other by traversing only PERMANENT_MEMBER and POTENTIAL_-MEMBER relationships. A process can adopt any group of which an already adopted group of the process is a potential member.

Roles are adopted either implicitly at creation of the PROCESS node or explicitly using CAIS interfaces.

a. implicit Role Adoption: When a root PROCESS node is created, it implicitly adopts the role associated with the default GROUP node of the current user. It also implicitly adopts the role associated with the default GROUP node of its executable image, if such a default GROUP node exists. When any other PROCESS node is created, it implicitly adopts those adopted roles of its creating process which are identified by inheritable ADOPTED_-ROLE relationships and the role associated with the default GROUP node of its executable image, if such a default GROUP node exists. The default role of the current user is always inheritable.

b. Explicit Role Adoption: An executing process may explicitly adopt roles associated with certain GROUP nodes in addition to its existing roles, using the ADOPT_ROLE interface (see Section 5.4.16 on page 277). For a process to adopt a role associated with a given GROUP node, some GROUP node already adopted by the process must be a potential member of the GROUP.

There may be multiple ADOPTED_ROLE relationships emanating from a PROCESS node. Keys of ADOPTED_ROLE relationships are implementation-defined when the relationship is created implicitly or are specified by the user as a parameter of the ADOPT_ROLE procedure when the relationship is created explicitly.

A process may disown any of its adopted roles, except the default roles of the current user and the



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

executable image; the process is said to *unadopt a role*. When the process disowns a role by using the UNADOPT_ROLE interface (see Section 5.4.17 on page 278), the respective ADOPTED_ROLE relationship is deleted. The process then executes under the authority of its remaining adopted roles. When a process terminates or aborts, it implicitly unadopts all of its adopted roles, but the respective ADOPTED_ROLE relationships emanating from the PROCESS node are not deleted.

### 5.4.4 Granting access rights

A node may be the source of zero or more secondary ACCESS relationships terminating at GROUP nodes. Each ACCESS relationship has a GRANT attribute that specifies which access rights to the node are granted to processes executing under the authority of roles that include this relationship.

ACCESS relationships and their GRANT attributes are established for nodes either at node creation or by using the interfaces provided in this package:

a. At node creation, the DISCRETIONARY_ACCESS parameter of the node creation interface provides a set of access rights to be granted to the current user. An ACCESS relationship with a GRANT attribute value given by the DISCRETIONARY_ACCESS parameter is established to the GROUP node associated with the current user; i.e., the node identified by the path 'CURRENT_USER’DEFAULT_ROLE.

b. The SET_GRANTED_RIGHTS procedure (see Section 5.4.10 on page 271) can be used by a process to establish an ACCESS relationship between a node and a GROUP node and to set the value of the GRANT attribute. This procedure can also be used to change the value of the GRANT attribute of an existing ACCESS relationship.

Access rights may be user-defined, but certain access rights have special significance to CAIS operations. In particular, the CAIS recognizes the access rights listed in Table X on page 260. This table also lists the kind of access for which specific access rights are necessary or sufficient.

The value of the GRANT attribute on ACCESS relationships must conform to the syntax of Table XI, which corresponds to the syntax of named aggregates in Ada. The syntax is consistent with that given in Section 9.1 on page 751. The list management interfaces in Section 9.1 can be used to construct and manipulate values of the GRANT attribute. Note that secure CAIS implementations may have special constraints on the construction and manipulation of values of the GRANT attribute (refer to Appendix E).

The syntax allows for conditional granting of access rights. In order for a process to acquire the access rights specified in a *resulting rights list*, the process must meet the condition specified by a named *necessary right*. The necessary right acts as a condition for having the access rights specified by the resulting rights list. A necessary right exists before a “=>” symbol, which is then followed by the resulting rights list.

In order to limit the set of GROUP nodes to which ACCESS relationships can be established, a CAIS implementation is allowed to restrict the set of GROUP nodes which may be target nodes of ACCESS relationships to an implementation-defined subset which may depend on the process and its acquired roles. A violation of such restrictions causes the exception ACCESS_VIOLATION to be raised by the CAIS interfaces attempting to create the ACCESS relationship. CAIS implementations may not restrict the discretionary access rights granted to the default role of the current user when a node is created.



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

TABLE X: Predefined access rights

| Access Right | Explanation |
|---|---|
| EXISTENCE | The minimum access rights without which the object is inaccessible to the subject. Without additional access rights the subject may neither read nor write attributes, relationships or contents of the object. This access right is not necessary to open the object with the intent NO_ACCESS. |
| READ_RELATIONSHIPS | The subject may read attributes of relationships emanating from the object or use it for traversal to another node; for objects which are VIEW nodes, the subject may adopt the corresponding view and NKIs within the view; the access right EXISTENCE is implicitly granted. This access right is necessary to open the object with (exclusive or non-exclusive) intent READ_RELATIONSHIPS. |
| WRITE_RELATIONSHIPS | The subject may create or delete relationships emanating from the object or may create, delete, or modify attributes of these relationships; the access right EXISTENCE is implicitly granted. This access right is necessary to open the object with (exclusive or non-exclusive) intent WRITE_RELATIONSHIPS. This access right is sufficient to open the object with (exclusive or non-exclusive) intent APPEND_RELATIONSHIPS. |
| APPEND_RELATIONSHIPS | The subject may create relationships emanating from the object and attributes of these relationships; the access right EXISTENCE is implicitly granted. This access right is necessary to open the object with (exclusive or non-exclusive) intent APPEND_RELATIONSHIPS. |
| READ_ATTRIBUTES | The subject may read attributes of the object; the access right EXISTENCE is implicitly granted. This access right is necessary to open the object with (exclusive or non-exclusive) intent READ_ATTRIBUTE. |
| WRITE_ATTRIBUTES | The subject may create, write, or delete attributes of the object; the access right EXISTENCE is implicitly granted. This access right is necessary to open the object with (exclusive or non-exclusive) intent WRITE_ATTRIBUTES. This access right is sufficient to open the object with (exclusive or non-exclusive) intent APPEND_ATTRIBUTES. |
| APPEND_ATTRIBUTES | The subject may create attributes of the object; the access right EXISTENCE is implicitly granted. This access right is necessary to open the object with (exclusive or non-exclusive) intent APPEND_ATTRIBUTES. |
| READ_CONTENTS | The subject may read contents of the object; the access right EXISTENCE is implicitly granted. This access right is necessary to open the object with (exclusive or non-exclusive) intent READ_CONTENTS. |



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

**TABLE X (continued): Predefined access rights**

| Access Right | Explanation |
|---|---|
| WRITE_CONTENTS | The subject may create, write, or delete contents of the object; the access right EXISTENCE is implicitly granted. This access right is necessary to open the object with (exclusive or non-exclusive) intent WRITE_CONTENTS. This access right is sufficient to open the object with (exclusive or non-exclusive) intent APPEND_CONTENTS. |
| APPEND_CONTENTS | The subject may append contents of the object or assign the medium resource; the access right EXISTENCE is implicitly granted. This access right is necessary to open the object with (exclusive or non-exclusive) intent APPEND_CONTENTS. |
| READ | This is the union of READ_RELATIONSHIPS, READ_ATTRIBUTES, READ_CONTENTS and EXISTENCE access rights. This access right is necessary to open the object with (exclusive or non-exclusive) intent READ. It is sufficient to open the object with (exclusive or non-exclusive) intent READ_RELATIONSHIPS, READ_ATTRIBUTES or READ_CONTENTS. |
| WRITE | This is the union of WRITE_RELATIONSHIPS, WRITE_ATTRIBUTES, WRITE_CONTENTS and EXISTENCE access rights. This access right is necessary to open the object with (exclusive or non-exclusive) intent WRITE. It is sufficient to open the object with (exclusive or non-exclusive) intent WRITE_RELATIONSHIPS, WRITE_ATTRIBUTES, WRITE_CONTENTS, APPEND_RELATIONSHIPS, APPEND_ATTRIBUTES and APPEND_CONTENTS. |
| APPEND | This is the union of APPEND_RELATIONSHIPS, APPEND_ATTRIBUTES, APPEND_CONTENTS and EXISTENCE access rights. This access right is necessary to open the object with (exclusive or non-exclusive) intent APPEND. It is sufficient to open the object with (exclusive or non-exclusive) intent APPEND_RELATIONSHIPS, APPEND_ATTRIBUTES or APPEND_CONTENTS. |
| EXECUTE | The subject may create a process that takes the contents of the object as its executable image or assign the processor resource; the access right EXISTENCE is implicitly granted. This access right is necessary to open the object with intent EXECUTE. |
| CREATE_INSTANCES | The subject with this right toward an NKD may create nodes for which the object is the definition; the access right EXISTENCE is implicitly granted. This access right is necessary to open the object with intent CREATE_INSTANCES. |
| CONTROL | The subject may modify access control information of the object; the access right EXISTENCE is implicitly granted. This access right is necessary to open the object with (exclusive or non-exclusive) intent CONTROL. |
| ALL_RIGHTS | This access right is the union of all other predefine access rights. All access rights in this table are granted to the subject. |


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

TABLE XI: **GRANT Attribute Value BNF**

```
grant_attribute_value    ::=  ( [ grant-item{, grant-item} ] )
grant_item               ::=  ( [ necessary_right = > ] resulting_rights_list )
necessary_right          ::=  identifier
resulting_rights_list    ::=  identifier | ( identifier {, identifier} )
```

Notes:

1. See Appendix D for a description of the notation used.

### 5.4.5 Denying access

Zero or more DENY relationships may emanate from a node and terminate at GROUP nodes. Each DENY relationship specifies the denial of access to any processes executing under the authority of roles that include a DENY relationship from the node being accessed.

The DENY_ACCESS procedure (see Section 5.4.13 on page 275) is used to create a DENY relationship. In the event that the adopted roles of a process contain both an ACCESS relationship and a DENY relationship emanating from the same node, the effect of the DENY relationship takes precedence over any rights conferred by the ACCESS relationship, and the access to the node is denied to the process. The DELETE_DENY_RELATIONSHIP interface can be used to delete a DENY relationship.

### 5.4.6 Access checking

Access checking requires checking the mandatory and discretionary access rights of the current process. Both must be sufficient for the intended operation to be permitted.

#### 5.4.6.1 Discretionary access checking

Checking the ability of a process to access a node under the discretionary access rules of the CAIS involves the following steps:

a. Determining all adopted roles (see Section 5.4.3 on page 257) of the current process.

b. Checking for the existence of a DENY relationship emanating from the node and ternti-nating at the GROUP nodes associated with the adopted roles. If such a relationship does exist, the intended operation is disallowed.

c. Evaluating the values of the GRANT attributes of the ACCESS relationships emanating from the node and terminating at the GROUP nodes associated with each adopted role. This evaluation determines the set of approved access rights. *Approved access rights* are access rights whose names appear in the resulting rights list of any grant item of the GRANT attribute values for which either (1) no necessary right is given or (2) the necessary right names an approved access right (under finite recursive application of this definition).


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

d. Comparing the approved access rights against the rights required to perform the intended operation.

The CAIS model requires that discretionary access checking be performed at the time a node handle is opened (see Section 5.2.4 on page 103). At this point access rights implied by the INTENT parameter of the open operation must be a subset of the approved access rights. If this is not the case, the open operation is terminated and an exception is raised. If the access rights implied by the INTENT parameter are a subset of the approved access rights, then for subsequent access using the open node handle the access rights required may be compared to the rights implied by the intent rather than the approved access rights.

#### 5.4.6.2 Mandatory access checking

Mandatory access controls are relevant to at least those secure CAIS implementations that must meet DOD5200.28-STD requirements for levels B1 and above. If the mandatory access controls specified herein are not satisfied, the operation terminates by raising the exception SECURITY_VIOLATION, except where the indication of failure constitutes violation of mandatory access control rules for read operations, in which case NAME_ERROR may be raised.

CAIS operations which access nodes are classified as read, write, or read/write operations. A process must satisfy both hierarchical and non-hierarchical access rights rules governing the type of operation intended to be performed to obtain access to a node. Access checking is performed by comparing the classification of the process with that of the node.

A process can satisfy hierarchical access rights rules to read a node if the hierarchical classification of the process is greater than or equal to that of the node. To obtain write access to the node, the process's hierarchical classification must be less than or equal to the hierarchical classification of the node. To obtain read/write access to a node, the hierarchical classification of both the process and the node must be the same.

A process can satisfy non-hierarchical access rights rules for read operations to a node if the set of non-hierarchical categories assigned to the node is some subset of the categories assigned to the process. A process can satisfy non-hierarchical access rights rules for write operations to a node if the set of non-hierarchical categories assigned to the process is some subset of the categories assigned to the node. A process can satisfy non-hierarchical access rights rules for read/write operations to a node if the set of non-hierarchical categories assigned to the node is the same as the set of categories assigned to the process.

The CAIS model requires mandatory access checking to be performed at the time a node handle is opened (see Section 5.2.4 on page 103) with respect to the intents expressed in the INTENT parameter. For subsequent access using a successfully opened node handle, mandatory access checking may, but need not, be repeated. The effects of alterations of classifications of nodes on concurrently executing processes are implementation-defined.

### 5.4.7 An example of CAIS access control

The following mechanisms relevant to access control are illustrated in Figures 17 through 21:

a. an example of a node model structure (Figure 17),



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

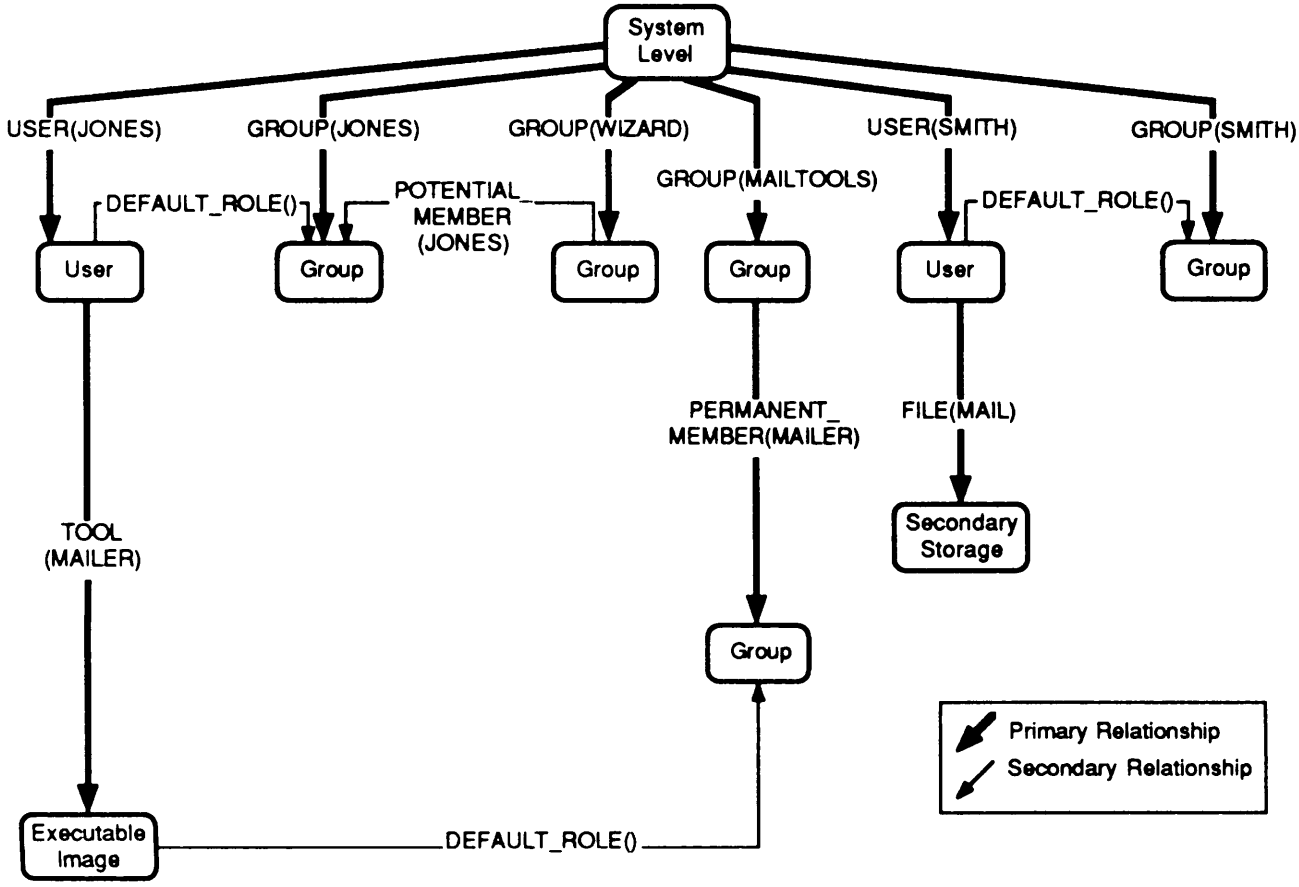


FIGURE 17. Discretionary access control



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

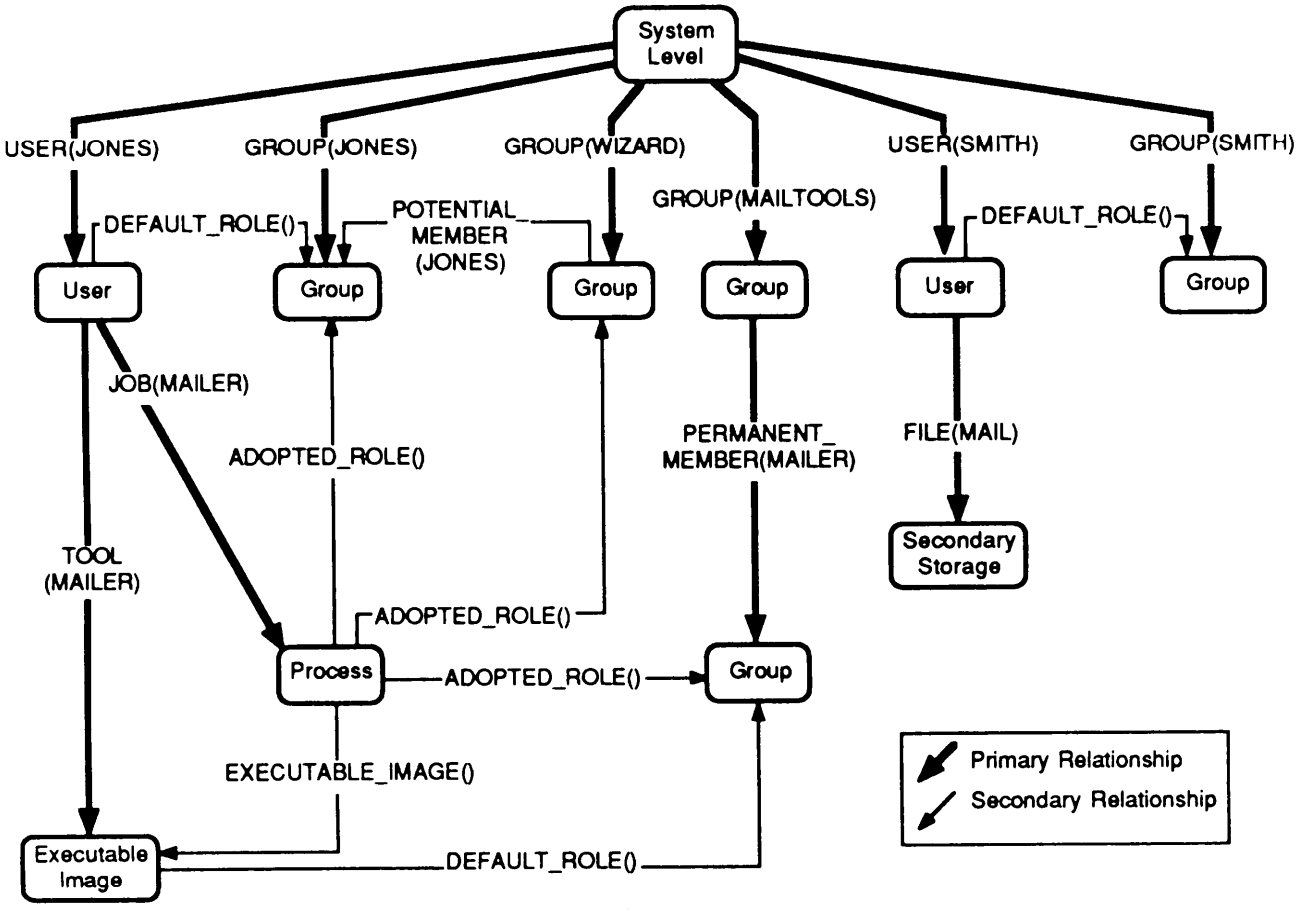


FIGURE 18. Discretionary access control after process creation


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

b. an example of the node model structure after the creation of a PROCESS node by user JONES (Figure 18), and

c. an example of the relationships relevant to the determination of approved access rights. Figures 19, 20, and 21 are derived from Figure 18. In Figure 19, the process 'USER(JONES)-'JOB(MAILER) has implicitly adopted the roles 'GROUP(JONES) and 'GROUP(MAILTOOLS).MAILER. Because the latter is reachable via a PERMANENT_MEMBER relationship from the group MAILTOOLS, any ACCESS relationships to ‘GROUP(MAILTOOLS) are part of the role 'GROUP(MAILTOOLS).MAILER. In addition, this process has adopted the role 'GROUP(WIZARD) explicitly.

These figures show a PROCESS node 'USER(JONES)'JOB(MAILER), a SECONDARY_STORAGE node 'USER(SMITH)'FILE(MAIL), and five GROUP nodes: 'GROUP(JONES), 'GROUP(WIZARD), 'GROUP(MAILTOOLS), 'GROUP(MAILTOOLS).MAILER and 'GROUP(SMITH) representing roles (where 'GROUP(WIZARD) was adopted explicitly). The relationship keys on the ADOPTED_ROLE relationships are omitted, although required by the CAIS node model, since these keys are not relevant to the discussion in this section. Figure 19 depicts the following relationships that might exist, given the above nodes:

a. an ACCESS relationship from the node 'FILE(MAIL) to the GROUP node 'GROUP(JONES) with a GRANT attribute value of ((NEWMAIL)),

b. an ACCESS relationship from the node 'FILE(MAIL) to the GROUP node 'GROUP-(MAILTOOLS) with a GRANT attribute value of ((NEWMAIL=>APPEND), (READ-MAIL=>READ)),

c. an ACCESS relationship from the node 'FILE(MAIL) to the GROUP node 'GROUP(SMITH) with a GRANT attribute value of (((NEWMAIL,READMAIL,CONTROL))),

d. an ACCESS relationship from the node 'FILE(MAIL) to the GROUP node 'GROUP(WIZARD) with a GRANT attribute value of (((READ,WRITE))),

e. an ADOPTED_ROLE relationship from the PROCESS node 'JOB(MAILER) to the GROUP node ‘GROUP(JONES), by implicit adoption through the default GROUP node of the current user,

f. an ADOPTED_ROLE relationship from the PROCESS node 'JOB(MAILER) to the GROUP node ‘GROUP(WIZARD), by explicit adoption, and

g. an ADOPTED_ROLE relationship from the PROCESS node 'JOB(MAILER) to the GROUP node 'GROUP(MAILTOOLS).MAILER, by implicit adoption through the default role of the executable image.

The values of the GRANT attributes of the ACCESS relationships are used to determine the set of approved access rights of the process 'JOB(MAILER); in the example shown in Figure 19, the values of the GRANT attributes for this job to FILE(MAIL) are (NEWMAIL), (( READ, WRITE)), (NEWMAIL=>APPEND) and (READMAIL=>READ).

Approved access rights are access rights whose names appear in the resulting rights list of any grant item of these GRANT attribute values for which (1) no necessary right is given or (2) the necessary right names an approved access right (under finite recursive application of the definition of approved


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

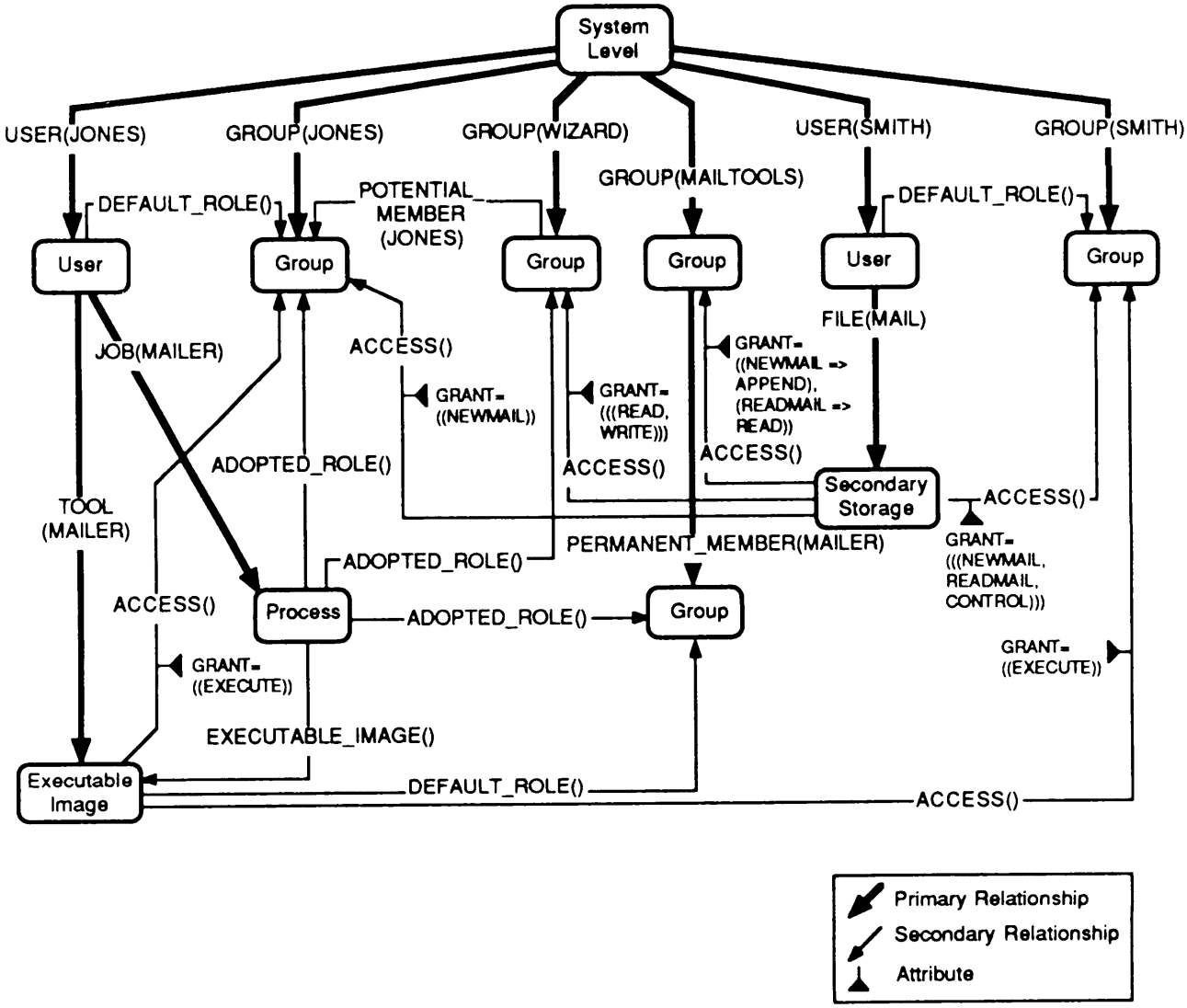


FIGURE 19. ACCESS relationships, Example 1



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

access rights). In this example, the access rights NEWMAIL, READ and WRITE have no necessary right given, thus the set of approved access rights under rule (1) contains NEWMAIL, READ and WRITE.

The access right APPEND can be added to the set of approved access rights under rule (2) because APPEND appears in the resulting rights list of the grant item whose necessary right name, NEWMAIL, matches an access right name already in the set. The set of approved access rights now consists of NEWMAIL, READ, WRITE and APPEND. 15

If the process had not explicitly adopted 'GROUP(WIZARD), the access rights would be determined by the GRANT attribute values ((NEWMAIL)) and ((NEWMAIL=>APPEND),(READMAIL=> READ)), which results in the set of approved access rights NEWMAIL and APPEND. 16

If 'USER(SMITH) were to run 'USER(JONES)'TOOL(MAILER), as shown in Figure 20, the job's access rights to FILE(MAIL) would be determined by the GRANT attribute values (((NEWMAIL, READMAIL,CONTROL))) and ((NEWMAIL=>APPEND),(READMAIL=>READ)), which results in the approved access rights NEWMAIL, READMAIL, CONTROL, APPEND, and READ. Note that 'USER(SMITH) cannot read or append the contents of the secondary storage file 'FILE(MAIL) except through the services of a tool whose default role is reachable via the primary PERMANENT_MEMBER relationship from 'GROUP(MAILTOOLS) or has the capability as a potential member to adopt the role 'GROUP(MAILTOOLS). By having access right CONTROL, the process may change the access rights to FILE(MAIL) at will. Of course, if 'USER(SMITH) could adopt the role WIZARD or could use a tool that adopts this role, he would not need to use a tool in the family of MAILTOOLS to read or write the contents of the file 'FILE(MAIL). It is a matter of policy in setting up the access control structures to determine whether highly privileged roles, such as the role WIZARD, are adoptable by specific users or tools. 17

While the access rights READ, WRITE, APPEND, and CONTROL are access rights which carry meaning to the CAIS interfaces that open node handles, NEWMAIL and READMAIL are user-defined access rights without CAIS-specific semantics. An approved user-defined access right does not influence the checking of access rights; it only influences the determination of approved access rights. 18

In each of these examples, users SMITH and JONES were allowed to run the MAILER program because ACCESS relationships with a GRANT attribute value of ((EXECUTE)) were established from the EXECUTABLE_IMAGE node containing the MAILER program to the GROUP nodes which are their respective default roles. 19

Figure 19 and Figure 20 show the use of GROUP nodes for assigning access rights to tools or groups of tools; Figure 21 exhibits an example in which access rights for users are grouped in a similar manner. The group 'GROUP(ALL_USERS) is introduced as a top-level GROUP node with emanating PERMANENT_MEMBER relationships to the GROUP nodes associated with the users SMITH and JONES. The latter are no longer top-level GROUP nodes. In this new example, the ACCESS relationship with a GRANT attribute of ((EXECUTE)) that was targeted at 'GROUP(JONES) in Figure 20 is moved so that it terminates at 'GROUP(ALL_USERS). All users are now allowed to run the MAILER program since the EXECUTE access right has been granted to 'GROUP(ALL_USERS). Note also that the NEWMAIL access right that was associated with GROUP(SMITH) has been deleted. Now, mail can be sent to SMITH by all users, but he can read it only via the MAILER program. WIZARDS can still read or edit the file 'FILE(MAIL) without using a program whose DEFAULT_ROLE is a member of the MAILTOOLS group. 20

A change to either of the above scenarios so that all access right changes regarding the file 'FILE(MAIL) have to be accomplished through a tool in 'GROUP(MAILTOOLS) can be done by simply moving the 21



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

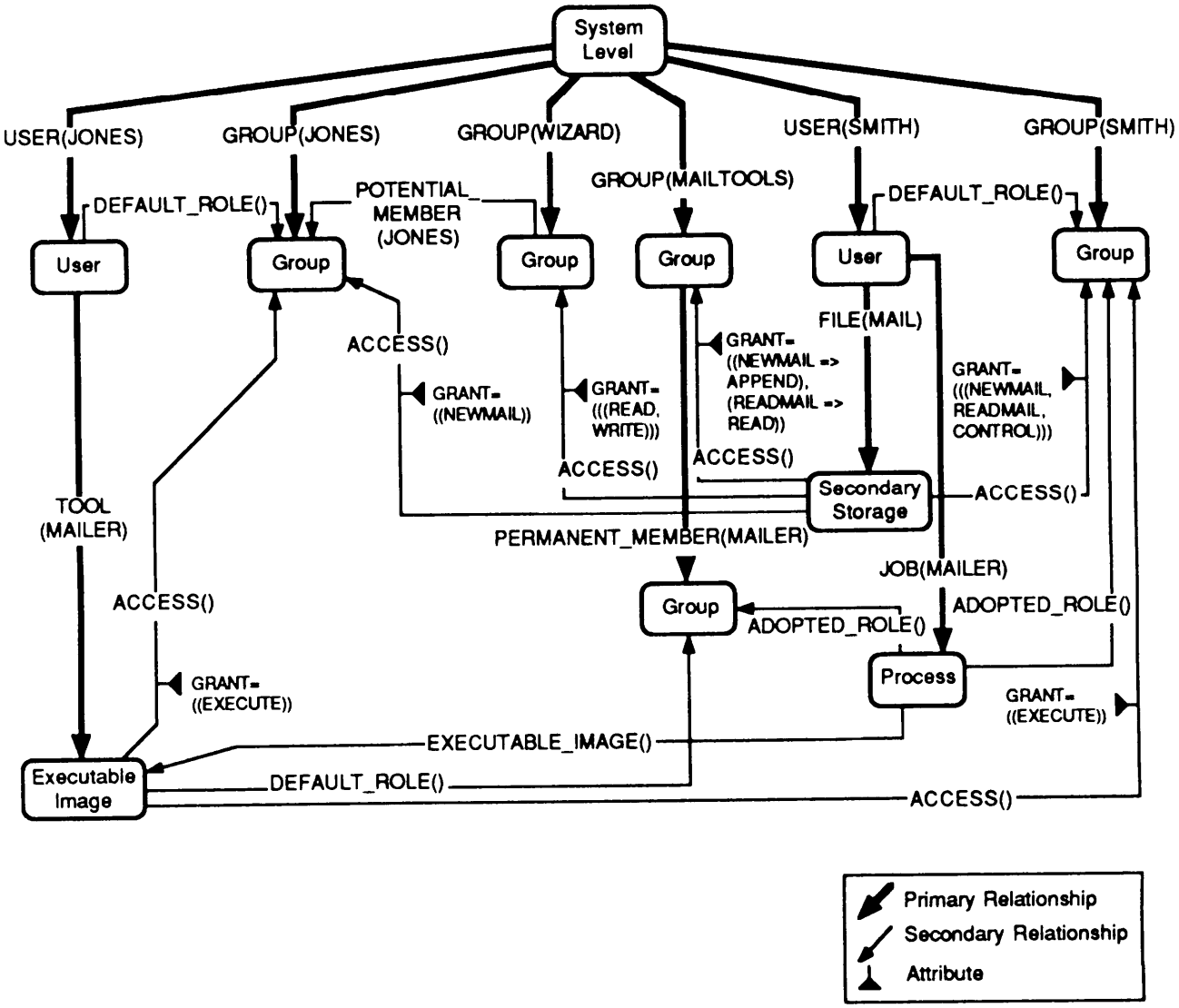


FIGURE 20. ACCESS relationships, Example 2


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

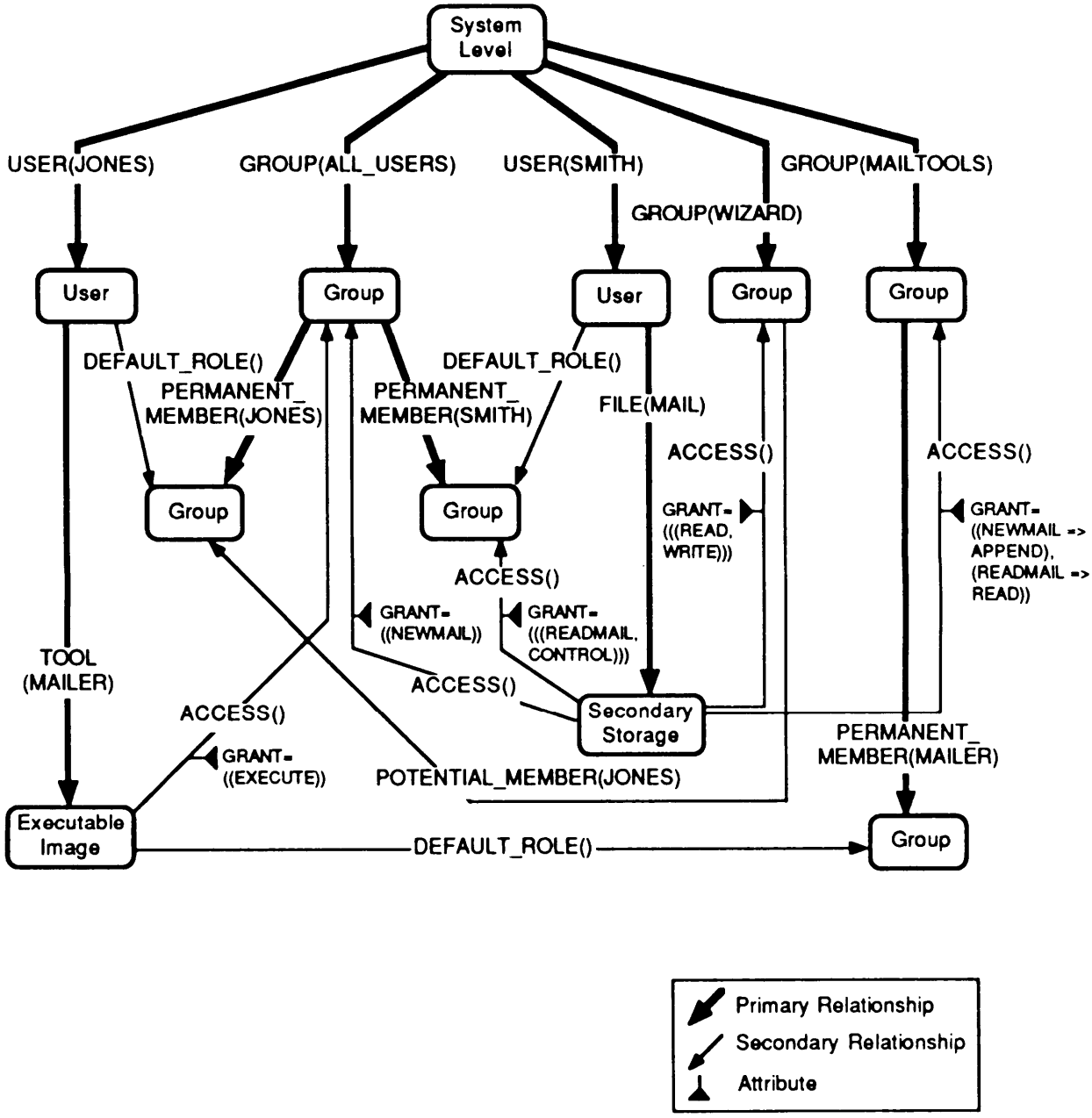


FIGURE 21. ACCESS relationships, Example 3


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

GRANT attribute value CONTROL to the ACCESS relationship targeted at 'GROUP(MAILTOOLS).

If this example occurred on a CAIS implementation requiring mandatory access controls, mandatory access checks would be performed at least each time a node handle is opened.

### 5.4.8 Subtypes

```
subtype ACCESS_RIGHTS is STRING;
```

ACCESS_RIGHTS is a subtype for values of access rights. The value is an access right identifier defined in Table X on page 260. Blanks, format effecters and/or non-printing characters are permitted surrounding the access right identifier.

### 5.4.9 Value of all access rights

```
function ALL_RIGHTS
    (STATUS : in STATUS_INDICATOR := NO_INDICATION)
    return DISCRETIONARY_ACCESS_LIST;
```

Purpose:

This function returns a value of type DISCRETIONARY_ACCESS_LIST, which, when installed as the value of the GRANT attribute on an ACCESS relationship, grants all predefine discretionary access rights. The list value consists of a single grant item (see Table XI, page 262) without necessary right and a resulting rights list consisting of the identifier ALL_RIGHTS (see Table X, page 260).

Parameters:

**STATUS** is an established status indicator.

Exceptions:

**STATUS_ERROR** is raised if STATUS is not an established status indicator [3].

### 5.4.10 Setting access control

```
procedure SET_GRANTED_RIGHTS
    (NODE       : in  NODE_TYPE;
     GROUP_NODE : in  GROUP_NODE_TYPE;
     GRANT      : in  DISCRETIONARY_ACCESS_LIST;
     STATUS     : in  STATUS_INDICATOR := NO_INDICATION);
```

Purpose:



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

This procedure sets access control information for a given node. If an ACCESS relationship does not exist from the node identified by NODE to the node identified by GROUP_NODE, such a relationship with an implementation-defined key is created. If necessary, a GRANT attribute is created on this relationship. The value of the GRANT attribute is set to the value of the GRANT parameter (see Table XI, page 262, for the syntax).



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Parameters:

**NODE** is an open node handle to the node whose access control information is to be set.

**GROUP_NODE** is an open node handle to a GROUP node.

**GRANT** is a list describing what access rights can be granted.

**STATUS** is an established status indicator.

Except ions:

**SYNTAX_ERROR** is raised if the value specified for the parameter GRANT is syntactically illegal (see Table XI, page 262) [10].

**NODE_KIND_ERROR** is raised if the open node handle GROUP_NODE does not identify a GROUP node [1].

**STATUS_ERROR** is raised if NODE and GROUP_NODE are not both open node handles [2] or if STATUS is not an established status indicator [3].

**CONFORMANCE_ERROR** is raised if the operation would result in an attribute, node or relationship not conforming to its definition or the constraints in force on that definition [3,10].

**INTENT_VIOLATION** is raised if NODE was not opened with intent CONTROL [1].

**ACCESS_VIOLATION** is raised if the executing process is not allowed to establish or alter an ACCESS relationship to the given GROUP node according to implementationI-defined criteria [2].

**SECURITY_VIOLATION** is raised if the operation represents a violation of mandatory access controls. SECURITY_VIOLATION is raised only if the conditions for other exceptions are not present.

### 5.4.11 Deleting ACCESS relationships

```
procedure DELETE_GRANTED_RIGHTS
   (NODE       : in  NODE_TYPE;
    GROUP_NODE : in  GROUP_NODE_TYPE;
    STATUS     : in  STATUS_INDICATOR := NO_INDICATION);
```

Purpose:

This procedure deletes access control information for a given node. If an ACCESS relationship exists from the node identified by NODE to the node identified by GROUP_NODE, it is deleted. If no such ACCESS relationship exists, this interface has no effect, and no error indication is given.

Parameters:

**NODE** is an open node handle to the node whose access control information is to be deleted.

**GROUP_NODE** is an open node handle to a GROUP node.

**STATUS** is an established status indicator.



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Exceptions:

**NODE_KIND_ERROR** is raised if the open node handle GROUP_NODE does not identify a GROUP node [1].

**STATUS_ERROR** is raised if NODE and GROUP_NODE are not both open node handles [2] or if STATUS is not an established status indicator [3].

**CONFORMANCE_ERROR** is raised if the operation would result in an attribute, node or relationship not conforming to its definition or the constraints in force on that definition [3].

**INTENT_VIOLATION** is raised if NODE was not opened with intent CONTROL [1].

**ACCESS_VIOLATION** is raised if the executing process is not allowed to establish or alter an ACCESS relationship to the given GROUP node according to implementation-defined criteria [2].

**SECURITY_VIOLATION** is raised if the operation represents a violation of mandatory access controls. SECURITY_VIOLATION is raised only if the conditions for other exceptions are not present.

### 5.4.12 Obtaining the value of a GRANT attribute

```
procedure GET_GRAIVTED_RIGHTS
   (NODE       : in     NODE_TYPE;
    GROUP_NODE : in     GROUP_NODE_TYPE;
    GRANT      : in out DISCRETIONARY_ACCESS_LIST;
    STATUS     : in     STATUS_INDICATOR  : = NO_INDICATION);
```

Purpose:

This procedure retrieves the value of the GRANT attribute of the ACCESS relationship between the object NODE and the GROUP node GROUP_NODE. It returns the empty list if no such relationship exists.

Parameters:

**NODE** is an open node handle to the node from which the ACCESS relationship emanates.

**GROUP_NODE** is an open node handle to a GROUP node.

**GRANT** is, upon return, a list describing what access rights are granted to the group.

**STATUS** is an established status indicator.

Exceptions:

**NODE_KIND_ERROR** is raised if the open node handle GROUP_NODE does not identify a GROUP node [1].

**STATUS_ERROR** is raised if NODE and GROUP_NODE are not both open node handles [2] or if STATUS is not an established status indicator [3].

**INTENT_VIOLATION** is raised if NODE was not opened with an intent establishing the right to read relationships or to read access control information [1].



Source: https://assist.dla.mil -- Downloaded: 2026l0-26T07:54Z
Check the source to verify that this is the currentversion before use.

### 5.4.13 Denying access

```
procedure DENY_ACCESS
    (NODE       : in  NODE_TYPE;
     GROUP_NODE : in  GROUP_NODE_TYPE;
     STATUS     : in  STATUS_INDICATOR := NO_INDICATION);
```

Purpose:

This procedure sets access control information for a given node. If a DENY relationship does not exist from the node identified by NODE to the node identified by GROUP_NODE, such a relationship with an implementation-defined key is created.

Parameters:

**NODE** is an open handle to the node to which access is to be denied.

**GROUP_NODE** is an open node handle to a GROUP node.

**STATUS** is unestablished status indicator.

Exceptions:

**NODE_KIND_ERROR** is raised if the open node handle GROUP-NODE does not identify a GROUP node [1].

**STATUS_ERROR** is raised if NODE and GROUP_NODE are not both open node handles [2] or if STATUS is not an established status indicator [3].

**CONFORMANCE_ERROR** is raised if the operation would result in an attribute, node or relationship not conforming to its definition or the constraints in force on that definition [3].

**INTENT_VIOLATION** is raised if NODE was not opened with intent CONTROL [1].

**ACCESS_VIOLATION** is raised if the executing process is not allowed to establish or alter a DENY relationship to the given GROUP node according to implementation-defined criteria [3].

**SECURITY_VIOLATION** is raised if the operation represents a violation of mandatory access control. SECURITY_VIOLATION is raised only if the conditions for other exceptions are not present.

### 5.4.14 Deleting DENY relationships

```
procedure DELETE_DENY_RELATIONSHIP
    (NODE       : in  NODE_TYPE;
     GROUP_NODE : in  GROUP_NODE_TYPE;
     STATUS     : in  STATUS_INDICATOR := NO_INDICATION);
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Purpose:

This procedure deletes discretionary access control information for a given node. If a DENY relationship exists from the node identified by NODE to the node identified by GROUP_NODE, it is deleted. If no such DENY relationship exists, this interface has no effect, and no error indication is given. 1

Parameters:

**NODE** is an open node handle to the node whose discretionary access control information is to be deleted. 2

**GROUP_NODE** is an open node handle to a GROUP node. 3

**STATUS** is an established status indicator. 4

Exceptions:

**NODE_KIND_ERROR** is raised if the open node handle GROUP_NODE does not identify a GROUP node [1]. 5

**STATUS_ERROR** is raised if NODE and GROUP_NODE are not both open node handles [2] or if STATUS is not an established status indicator [3]. 6

**CONFORMANCE_ERROR** is raised if the operation would result in an attribute, node or relationship not conforming to its definition or the constraints in force on that definition [3]. 7

**INTENT_VIOLATION** is raised if NODE was not opened with intent CONTROL [1]. 8

**ACCESS_VIOLATION** is raised if the executing process is not allowed to establish or alter a DENY relationship to the given GROUP node according to implementation-defined criteria [3]. 9

**SECURITY_VIOLATION** is raised if the operation represents a violation of mandatory access controls. SECURITY_VIOLATION is raised only if the conditions for other exceptions are not present. 10

### 5.4.15 Examining access rights

```
function IS_APPROVED
    (OBJECT_NODE  : in     NODE_TYPE;
     ACCESS_RIGHT : in     ACCESS_RIGHTS;
     STATUS       : in     STATUS_INDICATOR  := NO_INDICATION)
    return BOOLEAN;
```

Purpose:

This function returns TRUE if the current process as a subject has an approved access right ACCESS_RIGHT to the OBJECT_NODE as an object and has not been denied access to the node identified by OBJECT_NODE. Otherwise it returns FALSE. 1



Source: https://assist.dla.mil -- Downloaded: 2026l0-26T07:54Z
Check the source to verify that this is the currentversion before use.

Parameters:

**OBJECT_NODE** is an open node handle to the object node.

**ACCESS_RIGHT** is the name of a predefine or user-defined access right.

**STATUS** is an established status indicator.

Exceptions:

**SYNTAX_ERROR** is raised if the parameter ACCESS_RIGHT is not a valid Ada identifier [7].

**STATUS_ERROR** is raised if OBJECT_NODE is not an open node handle [2] or if STATUS is not an established status indicator [3].

**INTENT_VIOLATION** is raised if OBJECT_NODE was not opened with an intent establishing the right to read relationships or to read access control information [1].

### 5.4.16 Adopting a role

```
procedure ADOPT_ROLE
    (GROUP_NODE : in  GROUP_NODE_TYPE;
     KEY        : in  RELATIONSHIP_KEY  := EMPTY_KEY_LIST;
     INHERITABLE : in  BOOLEAN          := TRUE;
     STATUS     : in  STATUS_INDICATOR  := NO_INDICATION);
```

Purpose:

This procedure causes the current process to adopt the role associated with the GROUP_NODE. An ADOPTED_ROLE relationship with a key designated by KEY is created from the calling PROCESS node to the GROUP node identified by GROUP_NODE. In order for the current process to adopt the role, some adopted group of the current process must be a potential member of the group.

Parameters:

**GROUP_NODE** is an open node handle to a GROUP node.

**KEY** is a relationship key list to be used in creating the relationship.

**INHERITABLE** specifies the value of the INHERITABLE attribute of the newly created relationship.

**STATUS** is an established status indicator.

Except ions:

**PATHNAME_SYNTAX_ERROR** is raised if KEY is syntactically illegal (see Table I, page 52) [2].



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

**NODE_KIND_ERROR** is raised if the open node handle GROUP_NODE does not identify a GROUP node [1].

**NOT_IN_VIEW_ERROR** is raised if a key name specified in KEY cannot be translated [1] or if KEY consists of a single relationship key designator and no key of the relation is visible [5].

**AMBIGUOUS_VIEW_ERROR** is raised if a key name specified in KEY has an ambiguous translation [1] or if KEY consists of a single relationship key designator and more than one key of the relation is visible [4].

**RELATIONSHIP_ERROR** is raised if KEY specifies a value for the same key more than once [5] or if there already exists an ADOPTED_ROLE relationship with key list KEY emanating from the current PROCESS node [3].

**USE_ERROR** is raised if there is no adopted role of the current process that is a potential member of the group [48].

**STATUS_ERROR** is raised if GROUP_NODE is not an open node handle [2] or if STATUS is not an established status indicator [3].

**CONFORMANCE_ERROR** is raised if the operation would result in an attribute, node or relationship not conforming to its definition or the constraints in force on that definition [3,10].

**LOCK_ERROR** is raised if access with intent APPEND_RELATIONSHIPS to the current PROCESS node cannot be obtained due to an existing lock on the node [1].

**SECURITY_VIOLATION** is raised if the operation represents a violation of mandatory access controls. SECURITY_VIOLATION is raised only if the conditions for other exceptions are not present.

### 5.4.17 Unlinking an adopted role

```
procedure UNADOPT_ROLE
    (KEY    : in    RELATIONSHIP_KEY;
     STATUS : in    STATUS_INDICATOR  : = NO_INDICATION);
```

Purpose:

This procedure deletes the ADOPTED_ROLE relationship with key list designated by KEY emanating from the current PROCESS node. If there is no such relationship, the procedure has no effect.

Parameters:

**KEY** is the relationship key list of the relation ADOPTED_ROLE.

**STATUS** is an established status indicator.

Exceptions:

**PATHNAME_SYNTAX_ERROR** is raised if KEY is syntactically illegal (see Table I, page 52) [2].



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

**STATUS_ERROR** is raised if STATUS is not an established status indicator [3].

**NOT_IN_VIEW_ERROR** is raised if a key name specified in KEY cannot be translated [1] or if KEY consists of a single relationship key designator and no key of the relation is visible [5].

**AMBIGUOUS_VIEW_ERROR** is raised if a key name specified in KEY has an ambiguous translation [1] or if KEY consists of a single relationship key designator and more than one key of the relation is visible [4].

**RELATIONSHIP_ERROR** is raised if KEY specifies a value for the same key more than once [5], if the KEY parameter identifies more than one existing ADOPTED_ROLE relationship emanating from the current PROCESS node [1].

**USE_ERROR** is raised if the adopted role is a default role of the current user or of the current process' executable image [49].

**CONFORMANCE_ERROR** is raised if the operation would result in an attribute, node or relationship not conforming to its definition or the constraints in force on that definition [3].

**LOCK_ERROR** is raised if access to the current PROCESS node with intent WRITE_RELATIONSHIPS cannot be obtained due to an existing lock on the node [1],


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

# 5.5 Package CAIS_STRUCTURAL_NODE_MANAGEMENT

STRUCTURAL nodes are special nodes in the sense that they do not have contents as the other nodes of the CAIS model do. 1

The package CAIS_STRUCTURAL_NODE_MANAGEMENT defines the primitive operations for cresting STRUCTURAL nodes. The exceptions raised by all subprograms in this package are defined in the packages CAIS_DEFINITIONS and CAIS_PRAGMATICS. 2

## 5.5.1 Opening the predefine STRUCTURAL NKD

```
function STRUCTURAL_NKD
    (STATUS : in      STATUS_INDICATOR  : = NO_INDICATION)
    return NKD_TYPE;
```

Purpose:

This function returns an open node handle with NO_ACCESS intent to the predefine NKD for STRUCTURAL nodes. 1

The node kind definition name STRUCTURAL and the relation NODE_KIND_DEFINITION are associated with the node handle returned as the key and relation used in opening the handle. See subprograms PATH_KEY (Section 5.2.20 on page 119) and PATH_RELATION (Section 5.2.21 on page 120). 2

Parameters:

**STATUS** is an established status indicator. 3

Exceptions:

**STATUS_ERROR** is raised if STATUS is not an established status indicator [3]. 4

## 5.5.2 Creating structural nodes

```
procedure CREATE_NODE
    (NODE                  : in out STRUCTURAL_NODE_TYPE;
     BASE                  :  in    NODE_TYPE;
     KEY                   :  in    RELATIONSHIP_KEY          : = EMPTY_KEY_LIST;
     RELATION              :  in    RD_TYPE;
     INTENT                :  in    INTENT_ARRAY              : = (l=> WRITE) ;
     ATTRIBUTES            :  in    ATTRIBUTE_LIST            : = EMPTY_LIST;
     DISCRETIONARY_ACCESS : in     DISCRETIONARY_ACCESS_ LIST : = ALL_ RIGHTS;
     MANDATORY_ACCESS      :  in    MANDATORY_ACCESS_ LIST : = EMPTY_LIST;
     KIND                  :  in    NODE_KIND_NODE_TYPE       : = STRUCTURAL_NKD;
     STATUS                :  in    STATUS_INDICATOR          : = NO_INDICATION;
     MEDIUM_RESOURCE       :  in    MEDIUM_NODE_TYPE          : = DEFAULT_MEDIUM);
```


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Purpose:

This procedure creates a STRUCTURAL node and installs the primary relationship to it as well as the corresponding reverse relationship to the parent node identified by BASE. The relation definition identification and relationship key list of the primary relationship to the node are given by the parameters RELATION and KEY, respectively; the base node from which the primary relationship emanates is given by the parameter BASE. An open node handle to the newly created node with intent as specified by the INTENT parameter is returned in NODE.

The ATTRIBUTES parameter provides initial values for attributes of the node.

The DISCRETIONARY. ACCESS parameter specifies the initial access control information to be established between the created node and the default GROUP node of the current user (see Section 5.4.4 on page 259).

The MANDATORY_ACCESS parameter specifies the object classification labels with which the node is to be created. If its value is the empty list, the node inherits the subject classification of the creating process as its object classification. Otherwise, it must be an unnamed list consisting of an identifier item and, optionally, an unnamed list of identifier items (see Table IX, page 256).

The MEDIUM_RESOURCE parameter specifies the medium resource on which the newly created node will reside.

KIND is an open node handle identifying the node kind for the new node. It must identify a STRUCTURAL node kind. It may specify either an NKD or an NKI. If KIND identifies an NKI, the NKI is made the current adopted interpretation for the node handle NODE.

Parameters:

**NODE** is a node handle, initially closed, to be opened to the newly created node.

**BASE** is an open node handle to the node from which the primary relationship to the new node is to emanate.

**KEY** is the relationship key list of the primary relationship to be created.

**RELATION** is an open node handle to the relation definition of the primary relationship to be created.

**INTENT** is the intent of subsequent operations on the node; the actual parameter takes the form of an array aggregate.

**ATTRIBUTES** is an empty or named list (see Section 9.1 on page 751) whose elements are used to establish initial values for attributes of the newly created node; each named item of the list specifies an attribute name and the value to be given to that attribute.

**DISCRETIONARY_ACCESS** is the initial access control information associated with the created node; it is the value of the GRANT attribute of the ACCESS relationship to the user's default GROUP node (see Section 5.4.4 on page 259).

**MANDATORY_ACCESS** is a list defining the classification label for the created node (see Table IX, page 256).

**KIND** is an open node handle identifying the node kind for the new node.

**STATUS** is an established status indicator.

**MEDIUM_RESOURCE** is an open node handle identifying a MEDIUM node.


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Exceptions:

**PATHNAME_SYNTAX_ERROR** is raised if the node identification given by KEY is syntactically illegal (see Table I, page 52) [2]. 18

**NAME_ERROR** is raised if the parent of the NKI identified by KIND is inaccessible [1]. 19

**EXISTING_NODE_ERROR** is raised if a node already exists with the identification given [1]. 20

**SYNTAX_ERROR** is raised if the ATTRIBUTES parameter (see description above) [8], the DISCRETIONARY_ACCESS parameter (see Section 5.4.4 on page 259) [10] or the MANDATORY_ACCESS parameter (see Table IX, page 256) [9] is syntactically illegal. 21

**NOT_IN_VIEW_ERROR** is raised if a key name specified in KEY or an attribute name specified in ATTRIBUTES cannot be translated [1] or if KEY consists of a single relationship key designator and no key of the specified relation is visible [5]. 22

**AMBIGUOUS_VIEW_ERROR** is raised if translation of a key name specified in KEY or an attribute name specified in ATTRIBUTES is ambiguous [1] or if KEY consists of a single relationship key designator and more than one key of the specified relation is visible [4]. 23

**PREDEFINED_RELATION_ERROR** is raised if RELATION identifies a relation that is CAIS-controlled [1]. 24

**RELATIONSHIP_ERROR** is raised if KEY specifies a value for the same key more than once [5] or if RELATION or its corresponding reverse relation has no unique key set [6]. 25

**PREDEFINED_ATTRIBUTE_ERROR** is raised if an attribute name specified within the ATTRIBUTES parameter identifies an attribute that is CAIS-controlled [1]. 26

**ATTRIBUTE_ERROR** is raised if ATTRIBUTES specifies a value for the same attribute more than once [6]. 27

**USE_ERROR** is raised if the value for the MANDATORY_ACCESS parameter is semantically illegal [10] or if the open node handle RELATION does not designate an RD for a bidirectional relation [52]. 28

**STATUS_ERROR** is raised if the node handle BASE, RELATION, KIND or MEDIUM_RESOURCE is not open [2], if NODE is an open node handle at the time of the call [1] or if STATUS is not an established status indicator [3]. 29

**NODE_KIND_ERROR** is raised if the open node handle RELATION does not identify an RD [1], if the open node handle KIND does not identify a STRUCTURAL node kind or a specialization of that node kind [2], if the open node handle KIND identifies a kind that is CAIS-controlled [4], or if the open node handle MEDIUM_RESOURCE does not identify a MEDIUM node [1]. 30

**CONFORMANCE_ERROR** is raised if the operation would result in an attribute, node or relationship not conforming to its definition or the constraints in force on that definition [1, 2, 3, 4, 7, 8, 9, 10, 11]. 31

**RESOURCE_ALLOCATION_ERROR** is raised if the medium resource identified through MEDIUM_RESOURCE is not allocatable [1]. 32

**RESOURCE_AVAILABILITY_ERROR** is raised if the medium resource identified through MEDIUM_RESOURCE is not available [1]. 33

**TRANSACTION_ERROR** is raised if the operation is included in more than one transaction [1] or if the operation is included in a transaction and the operation modifies a node which is not included in the same transaction [2]. 34



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

**LOCK_ERROR** is raised if a node handle to the parent of the NKI specified by KIND cannot be opened with intent READ_RELATIONSHIPS due to outstanding locks in conflict with that intent [1]. 35

**INTENT_VIOLATION** is raised if BASE was not opened with an intent establishing the right to create relationships [1]. 36

**ACCESS_VIOLATION** is raised if the discretionary access control rights of the current process are insufficient to gain access with CREATE_INSTANCES intent to the NKD identified through KIND or with READ_RELATIONSHIPS intent to the parent of the NKI specified by KIND or with APPEND_CON_TENTS intent to the MEDIUM node identified by MEDIUM_RESOURCE [1]. 37

**SECURITY_VIOLATION** is raised if the operation represents a violation of mandatory access controls. SECURITY_VIOLATION is raised only if the conditions for other exceptions are not present. 38

Additional Interfaces:

```
procedure CREATE_NODE
    (NODE                    : in out STRUCTURAL_NODE_TYPE;
     BASE                    :  in    NODE_TYPE;
     KEY                     :  in    RELATIONSHIP_KEY          := EMPTY_KEY_LIST;
     RELATION                :  in    RELATION_NAME             : = DEFAULT_RELATION;
     INTENT                  :  in    INTENT_ARRAY              := (i=> WRITE) ;
     ATTRIBUTES              :  in    ATTRIBUTE_LIST            : = EMPTY_LIST;
     DISCRETIONARY_ACCESS : in    DISCRETIONARY_ACCESS_LIST : = ALL_RIGHTS;
     MANDATORY_ACCESS        :  in    MANDATORY_ACCESS_LIST : = EMPTY_LIST;
     KIND                    :  in    NODE_KIND_NAME            : = "STRUCTURAL";
     STATUS                  :  in    STATUS_INDICATOR          := NO_INDICATION;
     MEDIUM_RESOURCE         :  in    MEDIUM_NODE_TYPE          := DEFAULT_MEDIUM)
is
     RELATION_DEFINITION : RD_TYPE;
     KIND_DEFINITION     : NKI_TYPE;
begin
     DEFINITION_OF_RELATION (RELATION_DEFINITION, BASE, RELATION, STATUS);
     OPEN_NODE_KIND (KIND_DEFINITION, KIND, STATUS);
     CREATE_NODE
         (NODE, BASE, KEY, RELATION_DEFINITION, INTENT, ATTRIBUTES,
          DISCRETIONARY_ACCESS, MANDATORY_ACCESS, KIND_DEFINITION,
          STATUS, MEDIUM_RESOURCE);
exception
     when others =>
          CLOSE (RELATION_DEFINITION);
          CLOSE (KIND_DEFINITION);
          raise;
end CREATE_NODE;
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

# 5.6 Package CAIS_TRANSLATION_MANAGEMENT

The package CAIS_TRANSLATION_MANAGEMENT defines operations to control the translation of names and to translate between the two fundamental forms of identification (by name and by definition).

Operations are provided to adopt and to cancel the adoption of views and NKIs, in order to control the namings that are available to a process.

Operations are also provided to open handles to definition nodes given a name or an identification of an instance, and to determine names given a handle to a definition node or an identification of an instance. This second class of operations allows translation between the names by which instances may be known to users of the CAIS, and the definitions by which they are ultimately identified.

See Section 5.7.3 on page 347 for details on the operation of views, and the interfaces of Section 5.7 for view and definition maintenance operations.

## 5.6.1 Subtypes

```
subtype DEFINITION_NAME is STRING;
```

Values of DEFINITION_NAME objects are names for definitions and must conform to identifier syntax.

## 5.6.2 Adopting a view

```
procedure ADOPT_VIEW
    (NODE        : in  VIEW_NODE_TYPE;
     INHERITABLE : in  BOOLEAN          := FALSE;
     STATUS      : in  STATUS_INDICATOR := NO_INDICATION);
```

Purpose:

This procedure adopts a view for the current process. An ADOPTED_VIEW relationship with key LATEST_KEY is created from the current PROCESS node to the VIEW node identified by the open node handle NODE. The view becomes the most recently adopted (and therefore the highest precedence) view. If the view is already an adopted view of the current process, its precedence in the composite view might be changed by this operation. However, this operation has no effect on existing ADOPTED_VIEW relationships.

The node handle NODE must have been opened with an intent granting the right to read relationships. The current process acquires a read relationships access synchronization lock on the VIEW node for the duration of the adoption. (See Section 5.7.3.3 on page 351).

The same view may be adopted concurrently, by multiple processes or the same process.

Parameters:

**NODE** is an open node handle to the VIEW node representing the view to be adopted.



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

**INHERITABLE** specifies the value of the INHERITABLE attribute of the newly created relationship.

**STATUS** is an established status indicator.

Except ions:

**NODE_KIND_ERROR** is raised if the node identified by NODE is not a VIEW node [1].

**STATUS_ERROR** is raised if the node handle NODE is not open [2] or if STATUS is not an established status indicator [3].

**CONFORMANCE_ERROR** is raised if the operation would result in an attribute, node or relationship not conforming to its definition or the constraints in force on that definition [3,10].

**TRANSACTION_ERROR** is raised if the operation is included in more than one transaction [1] or if the operation is included in a transaction and the operation modifies a node not included in the same transaction [2].

**INTENT_VIOLATION** is raised if the node handle NODE was not opened with an intent including READ_RELATIONSHIPS [1].

**SECURITY_VIOLATION** is raised if the operation represents a violation of mandatory access controls. SECURITY_VIOLATION is raised only if the conditions for other exceptions are not present.

Notes:

The access right READ_RELATIONSHIPS toward a VIEW node as object is sufficient for a subject to adopt the corresponding view.

### 5.6.3 Canceling a view adoption

```
procedure UNADOPT_VIEW
    (NODE   : in    VIEW_NODE_TYPE;
     STATUS : in    STATUS_INDICATOR := NO_INDICATION);
```

Purpose:

This procedure cancels a view adoption of the current process. An ADOPTED_VIEW relationship emanating from the current PROCESS node and terminating at the VIEW node identified by the open node handle NODE is deleted. If multiple such relationships exist, the one deleted is the last one crested. If there is no such relationship, the procedure has no effect.

The read relationships access synchronization lock on the VIEW node due to the view adoption being cancelled is also cancelled. (See Section 5.7.3.3 on page 351).

Parameters:

**NODE** is an open node handle to a VIEW node.

**STATUS** is an established status indicator.


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Exceptions:

**NODE_KIND_ERROR** is raised if the node identified by NODE is not a VIEW node [1].

**STATUS_ERROR** is raised if the node handle NODE is not open [2] or if STATUS is not an established status indicator [3].

**CONFORMANCE_ERROR** is raised if the operation would result in an attribute, node or relationship not conforming to its definition or the constraints in force on that definition [3].

**SECURITY_VIOLATION** is raised if the operation represents a violation of mandatory access controls. SECURITY_VIOLATION is raised only if the conditions for other exceptions are not present.

Notes:

Successful completion of this operation does not necessarily indicate that the specified view is no longer an adopted view, as the view might have been adopted more than once. Only the most recently created ADOPTED_VIEW relationship emanating from the current PROCESS node and terminating at the specified VIEW node is deleted upon a single call to this interface.

### 5.6.4 Adopting an NKI

```
procedure ADOPT_INTERPRETATION
    (NODE           : in out NODE_TYPE;
     INTERPRETATION : in     NKI_TYPE;
     STATUS         : in     STATUS_INDICATOR := NO_INDICATION);
```

Purpose:

This procedure adds an NKI to the set of adopted interpretations for a node handle as the current adopted interpretation. INTERPRETATION identifies the NKI.

The NKI becomes the most recently adopted NKI for NODE. It and NKIs in the same view for any generalizations of the specified kind, exclusively, will apply to uses of the handle NODE.

The NKD identified by the DEFINITION relationship emanating from the NKI identified by INTERPRETATION must be either the NKD for NODE or a generalization of that kind.

The current process acquires a read relationships access synchronization lock on the parent of the NKI (a VIEW node) for the duration of the adoption. (See Section 5.7.3.3 on page 351).

Parameters:

**NODE** is an open node handle.

**INTERPRETATION** is an NKI applicable to the node identified by NODE and in a view not undergoing modification.

**STATUS** is an established status indicator.



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Exceptions:

**NAME_ERROR** is raised if the parent of the node identified by INTERPRETATION is inaccessible [1]. 8

**USE_ERROR** is raised if the NKI does not apply to the node identified by NODE [13]. 9

**STATUS_ERROR** is raised if NODE or INTERPRETATION is not an open node handle [2] or if STATUS is not an established status indicator [3]. 10

**TRANSACTION_ERROR** is raised if the operation is included in more than one transaction [1] due to the inclusion of the node identified by NODE and the VIEW node containing the node identified by INTERPRETATION in different transactions. 11

**LOCK_ERROR** is raised if a node handle to the parent of the node specified by INTERPRETATION cannot be opened with intent READ_RELATIONSHIPS due to outstanding locks in conflict with that intent [1]. 12

**ACCESS_VIOLATION** is raised if the current process does not have sufficient discretionary access control rights to obtain access to the parent of the node specified by INTERPRETATION with intent READ_RELATIONSHIPS [1]. 13

**SECURITY_VIOLATION** is raised if the operation represents a violation of mandatory access controls. SECURITY_VIOLATION is raised only if the conditions for other exceptions are not present. 14

Additional Interfaces:

```
procedure ADOPT_INTERPRETATION
    (NODE           : in out NODE_TYPE;
     INTERPRETATION : in     NODE_KIND_NAME;
     STATUS         : in     STATUS_INDICATOR := NO_INDICATION)
is
   NKI : NKI_TYPE;
begin
   OPEN_NODE_KIND (NKI, INTERPRETATION, STATUS);
   ADOPT_INTERPRETATION (NODE, NKI, STATUS);
   CLOSE (NKI);
exception
   when others =>
      CLOSE (NKI);
      raise;
end ADOPT_INTERPRETATION;
```

Notes:

While an interpretation is adopted for a node handle, views not containing the interpretation (including high precedence views and views subsequently adopted) have no effect on accesses to the node using the handle NODE. 15

The access right READ_RELATIONSHIPS toward a VIEW node as object is sufficient for a subject to adopt interpretations in the corresponding view. 16



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

### 5.6.5 Canceling an NKI adoption

```
procedure UNADOPT_INTERPRETATION
    (NODE           : in out BODE_TYPE;
     INTERPRETATION : in     NKI_TYPE           := NULL_NODE;
     STATUS         : in     STATUS_INDICATOR   := NO_INDICATION);
```

Purpose:

This procedure cancels the current adopted interpretation for anode handle. The most recent active association of an NKI with an open node handle which was made by a single call to ADOPT_INTERPRETATION is canceled. If at the time of that call there was a current adopted interpretation for the node handle, it is restored as the current adopted interpretation. 1

If INTERPRETATION has the value NULL_NODE, it has no effect. Otherwise, INTERPRETATION identifies the NKI most recently bound to the handle. When multiple interpretations may be adopted for a node handle, the INTERPRETATION parameter provides the calling subprogram the opportunity to check that the current adopted interpretation is the one expected. 2

The read relationships access synchronization lock on the VIEW node due to the interpretation adoption being cancelled is also cancelled. (See Section 5.7.3.3 on page 351). 3

Parameters:

**NODE** is an open node handle. 4

**INTERPRETATION** is an open node handle identifying an NKI, or has the value NULL_NODE. 5

**STATUS** is an established status indicator. 6

Exceptions:

**USE_ERROR** is raised if the open node handle NODE does not have an active adoption [15] or if the active adoption for the open node handle NODE is of a node kind interpretation other than that identified by the open node handle INTERPRETATION [14]. 7

**NODE_KIND_ERROR** is raised if the open node handle INTERPRETATION does not identify an NKI [2]. 8

**STATUS_ERROR** is raised if NODE is not an open node handle [2], if INTERPRETATION is neither an open node handle nor NULL_NODE [2] or if STATUS is not an established status indicator [3]. 9

**SECURITY_VIOLATION** is raised if the operation represents a violation of mandatory access controls. SECURITY_VIOLATION is raised only if the conditions for other exceptions are not present. 10

Additional Interfaces:



Source: https://assist.dla.mil -- Downloaded: 202610-26T07:54Z
Check the source to verify that this is the current version before use.

```
procedure UNADOPT_INTERPRETATION
    (NODE           : in out NODE_TYPE;
     INTERPRETATION : in      NODE_KIND_NAME;
     STATUS         : in      STATUS_INDICATOR := NO_INDICATION)
is
    NKI : NKI_TYPE;
begin
    OPEN_NODE_KIND (NKI, INTERPRETATION, STATUS);
    UNADOPT_INTERPRETATION (NODE, NKI, STATUS);
    CLOSE (NKI);
exception
    when others =>
        CLOSE (NKI);
        raise;
end UNADOPT_INTERPRETATION;
```

### 5.6.6 Opening a handle to an NKD by local name

```
procedure OPEN_NODE_KIND_DEFINITION
    (NODE   : in out NKD_TYPE;
     KIND   : in     NODE_KIND_NAME;
     STATUS : in     STATUS_INDICATOR := NO_INDICATION);
```

Purpose:

This procedure returns an open node handle in NODE to tile NKD named. An NKI is identified by a NODE_KIND relationship with the NAME key specified by KIND emanating from a VIEW node for a view adopted by the current process. The corresponding NKD is the target of the DEFINITION relationship emanating from the NKI. The node handle is returned with NO_-ACCESS intent. 1

The node kind name specified by KIND and the relation NODE_KIND are associated with NODE as the key and relation used in opening the hande. See subprograms PATH_KEY (Section 5.2.20 on page 119) and PATH_RELATION (Section 5.2.21 on page 120). 2

Parameters:

**NODE** is a closed node handle to be opened to the NKD. 3

**KIND** is anode kind name identifying an NKI in a view adopted by the current process. 4

**STATUS** is an established status indicator. 5

Exceptions:

**SYNTAX_ERROR is** raised if the KIND parameter is syntactically illegal as a definition name [14]. 6

**NOT_IN_VIEW_ERROR** is raised if KIND cannot retranslated [2]. 7


Source: https://assist.dla.mil -- Downloaded: 2026l0-26T07:54Z
Check the source to verify that this is the currentversion before use.

**STATUS_ERROR** is raised if NODE is an open node handle [1] or if STATUS is not an established status indicator [3].

**TRANSACTION_ERROR** is raised if the operation is included in more than one transaction [1].

**SECURITY_VIOLATION** is raised if the operation represents a violation of mandatory access controls. SECURITY_VIOLATION is raised only if the conditions for other exceptions are not present.

### 5.6.7 Opening a handle to an NKI by local name

```
procedure OPEN_NODE_KIND
    (NODE   : in out NKI_TYPE;
     KIND   : in     NODE_KIND_NAME;
     STATUS : in     STATUS_INDICATOR := NO_INDICATION);
```

Purpose:

This procedure returns an open node handle in NODE to the NKI named. The NKI is identified by a NODE_KIND relationship with the NAME key specified by KIND emanating from a VIEW node for a view adopted by the current process. The node handle is returned with NO_ACCESS intent.

The node kind name specified by KIND and the relation NODE_KIND are associated with NODE as the key and relation used in opening the handle. See subprograms PATH_KEY (Section 5.2.20 on page 119) and PATH_RELATION (Section 5.2.21 on page 120).

Parameters:

**NODE** is a closed node handle to be opened to the NKI.

**KIND** is a node kind name identifying the NKI in a view adopted by the current process.

**STATUS** is an established status indicator.

Exceptions:

**SYNTAX_ERROR** is raised if the KIND parameter is syntactically illegal as a definition name [14].

**NOT_IN_VIEW_ERROR** is raised if KIND cannot be translated [2].

**STATUS_ERROR** is raised if NODE is an open node handle [1] or if STATUS is not an established status indicator [3].

**TRANSACTION_ERROR** is raised if the operation is included in more than one transaction [1].

**SECURITY_VIOLATION** is raised if the operation represents a violation of mandatory access controls. SECURITY_VIOLATION is raised only if the conditions for other exceptions are not present.



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

### 5.6.8 Opening a handle to an RD by local name

```
procedure OPEN_RELATION_DEFINITION
    (NODE   : in out RD_TYPE;
     KIND   : in     RELATION_NAME;
     STATUS : in     STATUS_INDICATOR := NO_INDICATION);
```

Purpose:

This procedure returns an open node handle in NODE to the RD named. An RI is identified by a RELATION relationship with the NAME key specified by KIND emanating from a VIEW node for a view adopted by the current process. The corresponding RD is the target of the DEFINITION relationship emanating from the RI. The node handle is returned with NO_ACCESS intent.

The relation definition name specified by KIND and the relation RELATION are associated with NODE as the key and relation used in opening the handle. See subprograms PATH_KEY (Section 5.2.20 on page 119) and PATH_RELATION (Section 5.2.21 on page 120).

Parameters:

**NODE** is a closed node handle to be opened to the definition node.

**KIND** is a relation definition name identifying an RI in a view adopted by the current process.

**STATUS** is an established status indicator.

Exceptions:

**SYNTAX_ERROR** is raised if the KIND parameter is syntactically illegal as a definition name [14].

**NOT_IN_VIEW_ERROR** is raised if KIND cannot be translated [2].

**STATUS_ERROR** is raised if NODE is an open node handle [1] or if STATUS is not an established status indicator [3].

**TRANSACTION_ERROR** is raised if the operation is included in more than one transaction [1].

**SECURITY_VIOLATION** is raised if the operation represents a violation of mandatory access controls. SECURITY_VIOLATION is raised only if the conditions for other exceptions are not present.

### 5.6.9 Opening a handle to an RI by local name

```
procedure OPEN_RELATION
    (NODE   : in out RI_TYPE;
     KIND   : in     RELATION_NAME;
     STATUS : in     STATUS_INDICATOR := NO_INDICATION);
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Purpose:

This procedure returns an open node handle in NODE to the RI named. The RI is identified by a RELATION relationship with the NAME key specified by KIND emanating from a VIEW node for a view adopted by the current process. The node handle is returned with NO_ACCESS intent. 1

The relation interpretation name specified by KIND and the relation RELATION are associated with NODE as the key and relation used in opening the handle. See subprograms PATH_KEY (Section 5.2.20 on page 119) and PATH_RELATION (Section 5.2.21 on page 120). 2

Parameters:

**NODE** is a closed node handle to be opened to the interpretation node. 3

**KIND** is a relation definition name identifying the RI in a view adopted by the current process. 4

**STATUS** is an established status indicator. 5

Exceptions:

**SYNTAX_ERROR** is raised if the KIND parameter is syntactically illegal as a definition name [14]. 6

**NOT_IN_VIEW_ERROR** is raised if KIND cannot be translated [2]. 7

**STATUS_ERROR** is raised if NODE is an open node handle [1] or if STATUS is not an established status indicator [3]. 8

**TRANSACTION_ERROR** is raised if the operation is included in more than one transaction [1]. 9

**SECURITY_VIOLATION** is raised if the operation represents a violation of mandatory access controls. SECURITY_VIOLATION is raised only if the conditions for other exceptions are not present. 10

### 5.6.10 Opening a handle to an AD by local name

```
procedure OPEN_ATTRIBUTE_DEFINITION
   (NODE   : in out AD_TYPE;
    NAME   : in     ATTRIBUTE_NAME;
    STATUS : in     STATUS_INDICATOR := NO_INDICATION);
```

Purpose:

This procedure returns an open node handle in NODE to the AD named. The AD is identified by an ATTRIBUTE relationship with the NAME key specified by KIND emanating from a VIEW node for a view adopted by the current process. The node handle is returned with NO_ACCESS intent. 1

The attribute definition name specified by KIND and the relation ATTRIBUTE are associated with NODE as the key and relation used in opening the handle. See subprograms PATH_KEY (Section 5.2.20 on page 119) and PATH_RELATION (Section 5.2.21 on page 120). 2


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Parameters:

**NODE** is a closed node handle to be opened to the AD.

**NAME** is an attribute name identifying an AD in a view adopted by the current process.

**STATUS** is an established status indicator.

Exceptions:

**SYNTAX_ERROR** is raised if the KIND parameter is syntactically illegal as a definition name [14].

**NOT_IN_VIEW_ERROR** is raised if NAME cannot be translated [2].

**STATUS_ERROR** is raised if NODE is an open node handle [1] or if STATUS is not an established status indicator [3].

**TRANSACTION_ERROR** is raised if the operation is included in more than one transaction [1].

**SECURITY_VIOLATION** is raised if the operation represents a violation of mandatory access controls. SECURITY_VIOLATION is raised only if the conditions for other exceptions are not present.

### 5.6.11 Opening a handle to an AKD by local name

```
procedure OPEN_ATTRIBUTE_KIND_DEFINITION
    (NODE   : in out AKD_TYPE;
     KIND   : in     ATTRIBUTE_KIND_NAME;
     STATUS : in     STATUS_INDICATOR := NO_INDICATION);
```

Purpose:

This procedure returns an open node handle in NODE to the AKD named. The definition node is identified by an ATTRIBUTE_KIND relationship with the NAME key specified by KIND emanating from a VIEW node for a view adopted by the current process. The node handle is returned with NO_ACCESS intent.

The attribute kind definition name specified by KIND and the relation ATTRIBUTE_KIND are associated with NODE as the key and relation used in opening the handle. See subprograms PATH_KEY (Section 5.2.20 on page 119) and PATH_RELATION (Section 5.2.21 on page 120).

Parameters:

**NODE** is a closed node handle to be opened to the AKD.

**KIND** is an attribute kind name identifying an AKD in a view adopted by the current process.

**STATUS** is an established status indicator.


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Exceptions:

**SYNTAX_ERROR** is raised if the KIND parameter is syntactically illegal as a definition name [14].

**NOT_IN_VIEW_ERROR** is raised if KIND cannot be translated [2].

**STATUS_ERROR** is raised if NODE is an open node handle [1] or if STATUS is not an established status indicator [3].

**TRANSACTION_ERROR** is raised if the operation is included in more than one transaction [1].

**SECURITY_VIOLATION** is raised if the operation represents a violation of mandatory access controls. SECURITY_VIOLATION is raised only if the conditions for other exceptions are not present.

### 5.6.12 Determining the name for a definition

```
function NAME_OF_DEFINITION
    (DEFINIT1ON : in     DEFINITION_NODE_TYPE;
     STATUS     : in     STATUS_INDICATOR := NO_INDICATION)
    return DEFINITION_NAME;
```

This function returns the name of the definition identified by the open node handle DEFINITION.

DEFINITION identifies an AD, AKD, NKD or RD. The applicable name defined by the highest precedence view is returned. This name is the value of the NAME key on an ATTRIBUTE, ATTRIBUTE_KIND, NODE_KIND or RELATION relationship. The relationship emanates from a VIEW node and terminates at the definition node in the case of an AD or AKD. The relationship emanates from a VIEW node and terminates at the highest precedence interpretation for the definition or a generalization thereof in the case of an NKD or RD.

Parameters:

**DEFINITION** is an open node handle to a definition node.

**STATUS** is an established status indicator.

Exceptions:

**NOT_IN_VIEW_ERROR** is raised if the name to be returned cannot be formed [3].

**AMBIGUOUS_VIEW_ERROR** is raised if formation of the name to be returned is ambiguous [2].

**STATUS_ERROR** is raised if DEFINITION is not an open node handle [2] or if STATUS is not an established status indicator [3].

**NODE_KIND_ERROR** is raised if the open node handle DEFINITION does not identify a definition node [1].

**TRANSACTION_ERROR** is raised if the operation is included in more than one transaction [1].

**SECURITY_VIOLATION** is raised if the operation represents a violation of mandatory access controls. SECURITY_VIOLATION is raised only if the conditions for other exceptions are not present.



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Notes:

AD and AKD names returned by this interface and passed to other interfaces (e.g. OPEN_ATTRIBUTE_DEFINITION or OPEN_ATTRIBUTE_KIND_DEFINITION, respectively) will identify the same AD or AKD as identified to this interface. 11

NKD and RD names returned by this interface and passed to other interfaces (e.g. OPEN_NODE_KIND_DEFINITION or OPEN_RELATION_DEFINITION, respectively) will identify either the same NKD or RD as identified to this interface or a generalization thereof. A generalization is identified if the applicable interpretations contained in the highest precedence view having an applicable interpretation correspond to generalizations rather than to the definition itself. 12

### 5.6.13 Determining the NKD for a node

```
procedure DEFINITION_OF_NODE
    (DEFINITION : in out NKD_TYPE;
     NODE       : in     NODE_TYPE;
     STATUS     : in     STATUS_INDICATOR := NO_INDICATION);
```

Purpose:

This procedure returns an open node handle in DEFINITION which identifies the NKD for the kind of the node identified by the open node handle NODE. The handle is opened with intent NO_ACCESS. 1

No path is traversed in opening the node handle DEFINITION. Consequently, it cannot be passed to subprograms PATH_KEY and PATH_RELATION (see Section 5.2.20 on page 119 and Section 5.2.21 on page 120). 2

Parameters:

**DEFINITION** is a closed node handle to be opened to the definition node. 3

**NODE** is an open node handle. 4

**STATUS** is an established status indicator. 5

Exceptions:

**STATUS_ERROR** is raised if NODE is not an open node handle, if DEFINITION is an open node handle [1] or if STATUS is not an established status indicator [3]. 6

**TRANSACTION_ERROR** is raised if the operation is included in more than one transaction [1]. 7

**SECURITY_VIOLATION** is raised if the operation represents a violation of mandatory access controls. SECURITY_VIOLATION is raised only if the conditions for other exceptions are not present. 8


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

### 5.6.14 Determining the name for a relation

```
function NAME_OF_RELATION
    (BASE     : in     NODE_TYPE;
     RELATION : in     RD_TYPE;
     STATUS   : in     STATUS_INDICATOR := NO_INDICATION)
    return RELATION_NAME;

function NAME_OF_RELATION
    (BASE     : in     NODE_TYPE;
     KEY      : in     RELATIONSHIP_KEY;
     RELATION : in     RD_TYPE;
     STATUS   : in     STATUS_INDICATOR := NO_INDICATION)
    return RELATION_NAME;
```

These functions return the name of the specified relation.

RELATION identifies an RD for which relationships can emanate from the node identified by BASE. An applicable NKI for the base node must define a name for the relation.

In the first form of the interface, the name returned is defined in a naming for the specific relation identified or a generalization of that relation.

In the second form of the interface, a specific relationship is identified. The name returned is defined in a naming for that relationship's relation or a generalization thereof. The relationship's relation may be specialization of the relation identified by RELATION. That is, instances of specializations can be identified through a given relation. Consequently the results of the two interfaces may be different for the same actual parameter RELATION: the result of the second form is more specific.

Parameters:

**BASE** is an open node handle.

**KEY** is a key list for relationship identification.

**RELATION** identifies a relation for which relationships can emanate from the node identified by BASE. If a key list is also provided, BASE, RELATION and KEY are used to determine the relation for a specific relationship for which a name is to be formed. Otherwise, a name is formed for the relation identified by RELATION itself.

**STATUS** is an established status indicator.

Exceptions:

**PATHNAME_SYNTAX_ERROR** is raised if the relationship key list given is syntactically illegal [2].

**RELATIONSHIP_ERROR** is raised if the specified relationship does not exist [2] or if the specification identifies multiple relationships of different relations [1] or if the key list specifies a value for the same key more than once [5].


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

**NOT_IN_VIEW_ERROR** is raised if a key name specified cannot be translated [1], if the key list consists of a single relationship key designator and no key of the corresponding relation is visible [5] or if the name to be returned cannot be formed [3]. 11

**AMBIGUOUS_VIEW_ERROR** is raised if a key name specified has an ambiguous translation [1], if a key list consists of a single relationship key designator and more than one key of the corresponding relation is visible [4] or if formation of the name to be returned is ambiguous [2]. 12

**STATUS_ERROR** is raised if BASE or RELATION is not an open node handle [2] or if STATUS is not an established status indicator [3]. 13

**NODE_KIND_ERROR** is raised if the open node handle RELATION does not identify an RD [1]. 14

**TRANSACTION_ERROR** is raised if the operation is included in more than one transaction [1]. 15

**INTENT_VIOLATION** is raised by the second form if the node handle BASE was not opened with an intent establishing the right to read relationships [1]. 16

**SECURITY_VIOLATION** is raised if the operation represents a violation of mandatory access controls. SECURITY_VIOLATION is raised only if the conditions for other exceptions are not present. 17

### 5.6.15 Determining the RD for a relation

```
procedure DEFINITION_OF_RELATION
    (DEFINITION : in out RD_TYPE;
     BASE       : in     NODE_TYPE;
     RELATION   : in     RELATION_NAME;
     STATUS     : in     STATUS_INDICATOR := NO_INDICATION);

procedure DEFINITION_OF_RELATION
    (DEFINITION : in out RD_TYPE;
     BASE       : in     NODE_TYPE;
     KEY        : in     RELATIONSHIP_KEY;
     RELATION   : in     RD_TYPE;
     STATUS     : in     STATUS_INDICATOR := NO_INDICATION) ;
```

Purpose:

These procedures return in DEFINITION an open node handle which identifies the RD for the specified relation. 1

In the first form of the interface, the relation is specified by the name RELATION, which must be defined by an applicable NKI for the node identified by BASE. This form does not require that there be any relationships of the specified relation emanating from the base node, nor that the specified node be obtainable and accessible. This form relies only on the kind of the node and an applicable NKI providing a naming. 2

In the second form, the relation is for a relationship which does exist. The relationship emanates from the base node and is identified by the open RD handle RELATION and the key list KEY. 3


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

The RD identified by DEFINITION will be either the same as the RD identified by RELATION or a specialization of that RD.

No path is traversed in opening the node handle DEFINITION. Consequently, it cannot be passed to subprograms PATH_KEY and PATH_RELATION (see Section 5.2.20 on page 119 and Section 5.2.21 on page 120).

Parameters:

**DEFINITION** is a closed node handle to be opened to the definition node.

**BASE** is an open node handle.

**KEY** is a key list for relationship identification.

**RELATION** identifies a relation for which relationships can emanate from the node identified by BASE. If it is a name, it is translated through an applicable NKI to determine the relation. If it is a handle to an RD, it is used to identify an instance whose relation is determined.

**STATUS** is an established status indicator.

Exceptions:

**PATHNAME_SYNTAX_ERROR** is raised if the RELATION name or relationship key list given is syntactically illegal [2].

**RELATIONSHIP_ERROR** is raised if the specified relationship does not exist [2] or if the specification identifies multiple relationships of different relations [1] or if a key list specifies a value for the same key more than once [5].

**NOT_IN_VIEW_ERROR** is raised if a relation or key name specified cannot be translated [I] or if a key list consists of a single relationship key designator and no key of the corresponding relation is visible [5].

**AMBIGUOUS_VIEW_ERROR** is raised if a relation or key name specified has an ambiguous translation [1] or if a key list consists of a single relationship key designator and more than one key of the corresponding relation is visible [4].

**STATUS_ERROR** is raised if the node handle BASE or RELATION is not open [2], if DEFINITION is an open node handle [1] or if STATUS is not an established status indicator [3].

**NODE_KIND_ERROR** is raised if the open node handle RELATION does not identify an RD [1].

**TRANSACTION_ERROR** is raised if the operation is included in more than one transaction [1].

**INTENT_VIOLATION** is raised by the second form if the node handle BASE was not opened with an intent establishing the right to read relationships [1].

**SECURITY_VIOLATION** is raised if the operation represents a violation of mandatory access controls. SECURITY_VIOLATION is raised only if the conditions for other exceptions are not present.


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Additional Interfaces:

```
procedure DEFINITION_OF_RELATION
    (DEFINITION : in out RD_TYPE;
     BASE       : in     NODE_TYPE;
     KEY        : in     RELATIONSHIP_KEY;
     RELATION   : in     RELATION_NAME;
     STATUS     : in     STATUS_INDICATOR := NO_INDICATION)
is
    GENERAL_RELATION : RD_TYPE;
begin
    DEFINITION_OF_RELATION
        (GENERAL_RELATION,
         BASE ,
         RELATION,
         STATUS);
    DEFINITION_OF_RELATION
        (DEFINITION,
         BASE,
         KEY,
         GENERAL_RELATION,
         STATUS);
    CLOSE (GENERAL_RELATION);
exception
    when others =>
        CLOSE (GENERAL_RELATION);
        raise;
end DEFINITION_OF_RELATION;
```

### 5.6.16 Determining the name for anode attribute

```
function NAME_OF_NODE_ATTRIBUTE
    (NODE      : in NODE_TYPE;
     ATTRIBUTE : in AD_TYPE;
     STATUS    : in STATUS_INDICATOR := NO_INDICATION)
    return ATTRIBUTE_NAME;
```

This function returns the name of the specified node attribute.

ATTRIBUTE identifies an AD for which an attribute can exist on the node identified by NODE. An applicable NKI for the base node must define a name for the attribute. The highest precedence name is returned.

Parameters:

**NODE** is an open node handle.

**ATTRIBUTE** is an open node handle identifying of an attribute which can be an attribute of the node identified by NODE.

**STATUS** is an established status indicator.


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Exceptions:

**NOT_IN_VIEW_ERROR** is raised if if the name to be returned cannot be formed [3].

**AMBIGUOUS_VIEW_ERROR** is raised if formation of the name to be returned is ambiguOus [2].

**STATUS_ERROR** is raised if NODE or ATTRIBUTE is not an open node handle [2] or if STATUS is not an established status indicator [3].

**NODE_KIND_ERROR** is raised if the open node handle ATTRIBUTE does not identify an AD [1].

**TRANSACTION_ERROR** is raised if the operation is included in more than one transaction [1].

**SECURITY_VIOLATION** is raised if the operation represents a violation of mandatory access controls. SECURITY_VIOLATION is raised only if the conditions for other exceptions are not present.

### 5.6.17 Determining the AD for a node attribute

```
procedure DEFINITION_OF_NODE_ATTRIBUTE
    (DEFINITION : in out AD_TYPE;
     NODE       : in     NODE_TYPE;
     ATTRIBUTE  : in     ATTRIBUTE_NAME;
     STATUS     : in     STATUS_INDICATOR := NO_INDICATION);
```

Purpose:

This procedure returns an open node handle in DEFINITION which identifies the AD for the attribute named in ATTRIBUTE on the node identified by the open node handle NODE. The local name specified in ATTRIBUTE must be defined by an applicable NKI. The handle is opened with intent NO_ACCESS.

No path is traversed in opening the node handle DEFINITION. Consequently, it cannot be passed to subprograms PATH_KEY and PATH_RELATION (see Section 5.2.20 on page 119 and Section 5.2.21 on page 120).

Parameters:

**DEFINITION** is a closed node handle to be opened to the definition node.

**NODE** is an open node handle.

**ATTRIBUTE** is the name of an attribute which can be an attribute of the node identified by NODE.

**STATUS** is an established status indicator.

Exceptions:

**SYNTAX_ERROR** is raised if the attribute name in ATTRIBUTE given is syntactically illegal [11].



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

**NOT_IN_VIEW_ERROR** is raised if the name specified in ATTRIBUTE cannot be translated [1].

**AMBIGUOUS_VIEW_ERROR** is raised if the name specified in ATTRIBUTE has an ambiguous translation [1].

**STATUS_ERROR** is raised if NODE is not an open node handle [2], if DEFINITION is an open node handle [1] or if STATUS is not an established status indicator [3].

**TRANSACTION_ERROR** is raised if the operation is included in more than one transaction [1].

**SECURITY_VIOLATION** is raised if the operation represents a violation of mandatory access controls. SECURITY_VIOLATION is raised only if the conditions for other exceptions are not present.

Notes:

This operation does not require that the node identified by NODE have the specified attribute, only that such an attribute can exist.

This operation does not require that the specified node be obtainable and accessible.

### 5.6.18 Determining the name for a relationship attribute

```
function NAME_OF_PATH_ATTRIBUTE
    (RELATION    : in    RD_TYPE;
     ATTRIBUTE   : in    AD_TYPE ;
     STATUS      : in    STATUS_INDICATOR  : = NO_INDICATION)
   return ATTRIBUTE_NAME;
```

Purpose:

This function returns the name of the specified relationship attribute.

ATTRIBUTE identifies an AD for which an attribute can exist on relationships of the relation identified by RELATION. An applicable RI for the relation must define a name for the attribute. The highest precedence name is returned.

Parameters:

**RELATION** is an open node handle to an RD.

**ATTRIBUTE** is an open node handle identifying of an attribute for the given relation.

**STATUS** is an established status indicator.

Exceptions:

**NOT_IN_VIEW_ERROR** is raised if if the name to be returned cannot be formed [3].

**AMBIGUOUS_VIEW_ERROR** is raised if formation of the name to be returned is ambiguous [2].


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

**STATUS_ERROR** is raised if RELATION or ATTRIBUTE is not an open node handle [2] or if STATUS is not an established status indicator [3].

**NODE_KIND_ERROR** is raised if the open node handle RELATION does not identify an RD [1] or if the open node handle ATTRIBUTE does not identify an AD [1].

**TRANSACTION_ERROR** is raised if the operation is included in more than one transaction [1].

**SECURITY_VIOLATION** is raised if the operation represents a violation of mandatory access controls. SECURITY_VIOLATION is raised only if the conditions for other exceptions are not present.

### 5.6.19 Determining the AD for a relationship attribute

```
procedure DEFINITION_OF_PATH_ATTRIBUTE
    (DEFINITION : in out AD_TYPE;
     RELATION   : in     RD_TYPE;
     ATTRIBUTE  : in     ATTRIBUTE_NAME;
     STATUS     : in     STATUS_INDICATOR := NO_INDICATION);
```

Purpose:

This procedure returns an open node handle which identifies the AD for the attribute identified by the attribute name ATTRIBUTE on relationships of the relation identified by RELATION, The local name specified in ATTRIBUTE must be defined by an applicable relation interpretation. The handle is opened with NO_ACCESS intent.

No path is traversed in opening the node handle DEFINITION. Consequently, it cannot be passed to subprograms PATH_KEY and PATH_RELATION (see Section 5.2.20 on page 119 and Section 5.2.21 on page 120).

Parameters:

**DEFINITION** is a closed node handle to be opened to the definition node.

**RELATION** is an open node handle identifying an RD.

**ATTRIBUTE** is the name of an attribute which can be an attribute of relationships of the relation identified by RELATION.

**STATUS** is an established status indicator.

Exceptions:

**SYNTAX_ERROR** is raised if the attribute name given is syntactically illegal [11].

**NOT_IN_VIEW_ERROR** is raised if a name specified in ATTRIBUTE cannot be translated [1].

**AMBIGUOUS_VIEW_ERROR** is raised if a name specified in ATTRIBUTE has an ambiguous translation [1].


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

**STATUS_ERROR** is raised if RELATION is not an open node handle [2], if DEFINITION is an open node handle [1] or if STATUS is not an established status indicator [3]. 10

**NODE_KIND_ERROR** is raised if the open node handle RELATION does not identify an RD [1]. 11

**TRANSACTION_ERROR** is raised if the operation is included in more than one transaction [1]. 12

**SECURITY_VIOLATION** is raised if the operation represents a violation of mandatory access controls. SECURITY_VIOLATION is raised only if the conditions for other exceptions are not present. 13

Additional Interfaces:

```
procedure DEFINITION_OF_PATH_ATTRIBUTE
    (DEFINITION : in out AD_TYPE;
     BASE       : in     NODE_TYPE;
     RELATION   : in     RELATION_NAME;
     ATTRIBUTE  : in     ATTRIBUTE_NAME;
     STATUS     : in     STATUS_INDICATOR := NO_INDICATION)
is
    RELATION_DEFINITION : RD_TYPE;
begin
    DEFINITION_OF_RELATION (RELATION_DEFINITION, BASE, RELATION, STATUS);
    DEFINITION_OF_PATH_ATTRIBUTE
        (DEFINITION,
         RELATION_DEFINITION,
         ATTRIBUTE ,
         STATUS) ;
    CLOSE (RELATION_DEFINITION);
exception
    when others =>
        CLOSE (RELATION_DEFINITION);
        raise;
end DEFINITION_OF_PATH_ATTRIBUTE;
```


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

# 5.7 Package CAIS_DEFINITION_MANAGEMENT

The package CAIS_DEFINITION_MANAGEMENT defines operations for creating, manipulating, and deleting definitions and views. The exceptions raised by the subprogram in this package are defined in the packages CAIS_DEFINITIONS and CAIS_PRAGMATICS.

## 5.7.1 Definition nodes

In the CAIS, each node, relationship and attribute is an instance of a definition represented by a definition node. Certain attributes and relationships represent constraints imposed by definitions and properties defined by definitions.

There are four kinds of definition nodes: node kind definition nodes (NKDs), relation definition nodes (RDs), attribute definitions nodes (ADs) and attribute kind definition nodes (AKDs) (see Section 4.3.11.1 on page 55).

All definition nodes may have the predefine list attribute ANNOTATION which is intended to convey information concerning the purpose and significance of the definition. The CAIS makes no assumptions about the structure or meaning of values of the ANNOTATION attribute.

### 6.7.1.1 Representation of specialization and generalization

Specializations and generalizations are defined in Section 4.3.11.2 on page 58. In the node model, specializations are represented by bidirectional SPECIALIZATION_OF relationships with reverse GENERALIZATION_OF relationships. A definition node represents a specialization of another definition node if there is a path, consisting of SPECIALIZATION_OF relationships, which starts at the definition node representing the specialization and ends at the other definition node which represents a generalization for the specialization. This path constitutes an inheritance path, whereby the specialization inherits the properties and constraints defined by the generalization. The SPECIALIZATION_OF relationships are the only representation of such inheritance: the relationships and attributes which explicitly represent properties and constraints at generalizations are not copied to other definitions by virtue of their being specializations.

A given definition may be a specialization of some definitions and a generalization of other definitions. A definition may not be a specialization of itself.

In the example in Figure 22 on page 305, four NKDs are related by SPECIALIZATION_OF relationships. NKD-4 is a specialization of NKD-1, NKD-2 and NKD-3. Conversely, NKD-1, NKD-2 and NKD-3 are generalizations of NKD-4. Also, NKD-2 and NKD-3 are specializations of NKD-1, and conversely, NKD-1 is a generalization of NKD-2 and NKD-3.

Figure 22 illustrates the inheritance of properties and constraints by specializations of more general definitions. In the figure, attribute X defined on NKD-1 is inherited by NKD-2, NKD-3 and NKD-4. The attribute Y defined on NKD-2 is inherited by NKD-4. There are also various constraints illustrated which constrain the range of values for attributes X and Y in the context of each NKD. The constraints on the attribute X (LOWER_BOUND and UPPER_BOUND) imposed by each NKD are

```
NKD-1 -          X <= 10
NKD-2 -          X <=  6
NKD-3 -     2<= X <=  7
```


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

The constraints imposed by NKD-2 and NKD-3 are stronger than those imposed by NKD-1. NKD-4 inherits all of these constraints, so the effective constraints for X in NKD-4 are: 5

NKD-4 - 2 <= x <= 6.

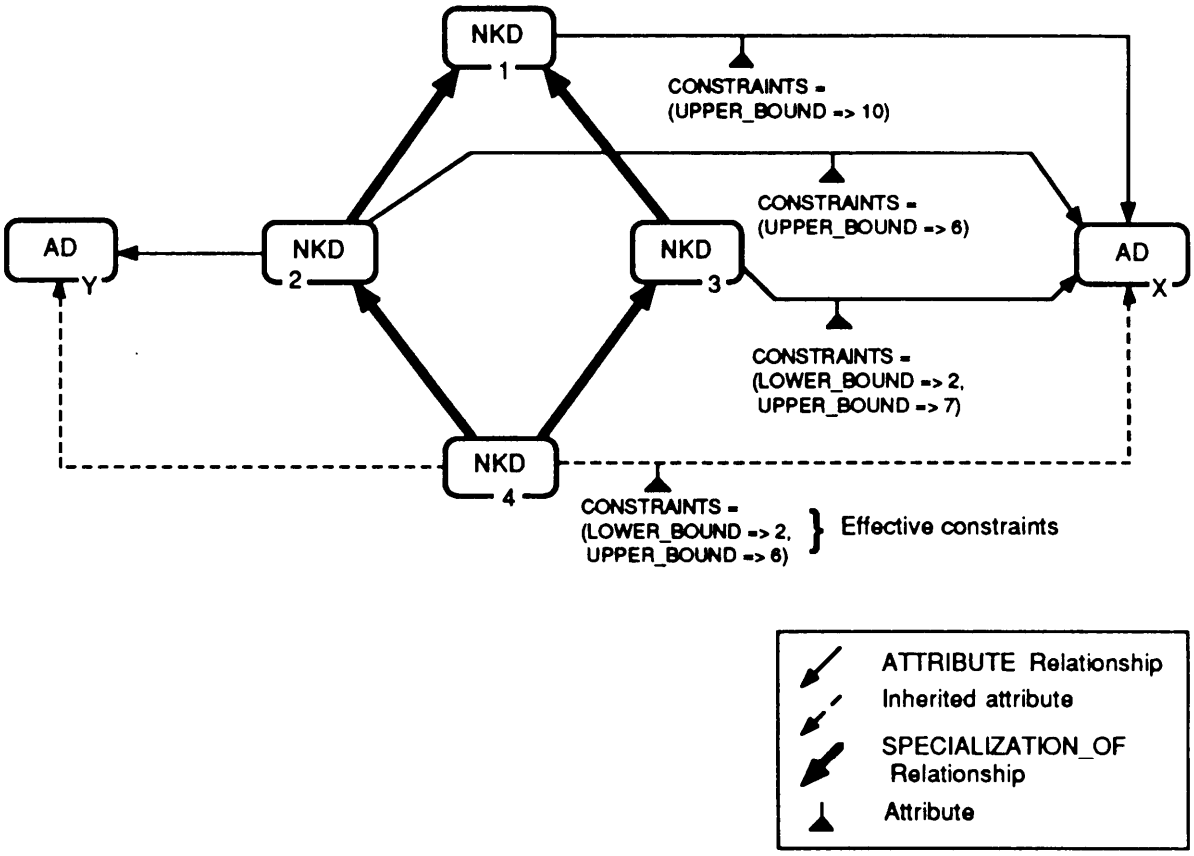


FIGURE 22. Inheritance of properties and constraints

A node which is an instance of NKD-4 may also be accessed through the generalizations NKD-1, NKD-2, or NKD-3. However, the attribute X of a node which is an instance of NKD-4 may not be set to 9 (even when accessed through NKD-1) since this violates the UPPER-BOUND constraint 6 which applies to X in all instances of NKD-4. 6

### 5.7.1.2 NKD and RD Lattices

The NKDs of a CAIS instance, connected by SPECIALIZATION_OF relationships, form an NKD 1



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

lattice. The RDs of a CAIS instance, also connected by SPECIALIZATION_OF relationships, form an RD lattice. At the tops of these lattices are the most general definition nodes: the basic NKD and the basic RD. Figure 23 illustrates a portion of an NKD lattice which includes some of the NKDs predefine by the CAIS. The basic NKD is at the top. The primary kinds of nodes, STRUCTURAL, PROCESS and FILE are predefine specializations of the basic NKD.

Constraints are used to partition the NKD lattice. The identifier-valued attribute KIND is predefine for the basic NKD at the top of the NKD lattice (see Figure 23 on page 307). The KIND attribute is therefore inherited by all NKDs, both predefine and user-defined. The allowed values of KIND are STRUCTURAL, PROCESS, and FILE, These values correspond to the basic kinds of nodes supported by the CAIS. The predefine NKD lattice includes an individual generalization for each of these CAIS node kinds. In each generalization, the value of KIND is constrained using a VALUE constraint item, (see Section 5.7.1.7 on page 328) to allow only the appropriate value for that node kind. Hence, in the predefine generalization for FILE nodes, the value of KIND is constrained to be FILE, while in the predefine generalization for PROCESS nodes, the value of KIND is constrained to be PROCESS. Since constraints are inherited and these constraints are conflicting, it is impossible to define a specialization which is both a PROCESS node kind and a FILE node kind. Figure 23 on page 307 illustrates such an illegal specialization. 2

Similarly, the STRUCTURAL node kinds are further partitioned by constraints on the STRUCTURAL_KIND attribute. FILE node kinds are further partitioned as well. In general, no specializations of different classes of CAIS predefined nodes may be defined. The constraints imposed on KIND, STRUCTURAL_KIND and other similar attributes partition the NKD lattice into disjoint sub-lattices below each of the predefine generalizations. The predefine NKD and RD lattices are defined in Appendix A. 3

### 5.7.1.3 Node kind definition nodes (NKDs)

Every node is an instance of a node kind definition, represented by an NKD. An NKD defines and constrains the properties of all nodes of the node kind represented. Figure 24 on page 308 illustrates the attributes and emanating relations of an NKD. Bidirectional, secondary ATTRIBUTE, EMANATES and TERMINATES relationships may emanate from NKDs. These relationships define the properties of the node kind. The corresponding reverse relations are ATTRIBUTE_OF, EMANATES_FROM, and TERMINATES_AT. Attributes on these relationships may further constrain the properties defined. 1

Emanating ATTRIBUTE relationships specify the set of attributes associated with a node kind. Each ATTRIBUTE relationship emanating from an NKD and terminating at an AD defines an attribute of the node kind and specifies properties of that attribute (see Figure 25 on page 309). The AD which is the target of the ATTRIBUTE relationship defines a particular attribute of the nodes defined by the NKD. Since a given attribute may have only one description and one instance (value) for a given node, only one ATTRIBUTE relationship emanating from a given NKD may terminate at a given AD. 2

ATTRIBUTE relationships emanating from a given NKD define properties which are inherited by specializations of that NKD. ATTRIBUTE relationships may have the CONSTRAINTS attribute which defines additional constraints on the attribute applicable to nodes conforming to the source NKD. These constraints are inherited by specializations. They are described in detail in Section 5.7.1.7 on page 328. 3

Attribute values may be specified explicitly when a node is created. In addition, ATTRIBUTE relationships may have the DEFAULT_VALUE attribute which specifies a value for the attribute if no value is explicitly assigned when the node or relationship is created. In the absence of both an explicit value and a DEFAULT_VALUE attribute which applies to an attribute, a node or relationship 4


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

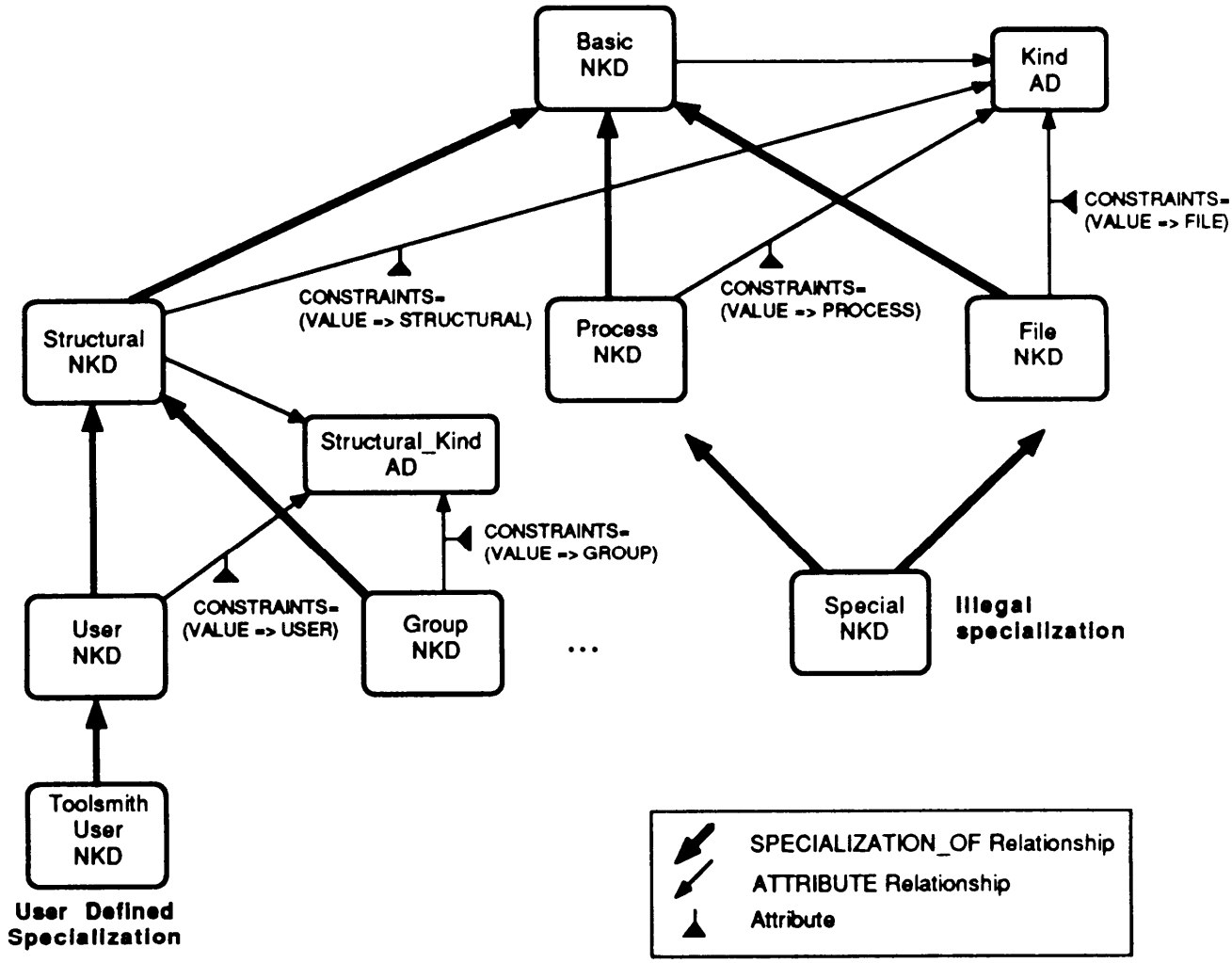


FIGURE 23. Example of an NKD lattice



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

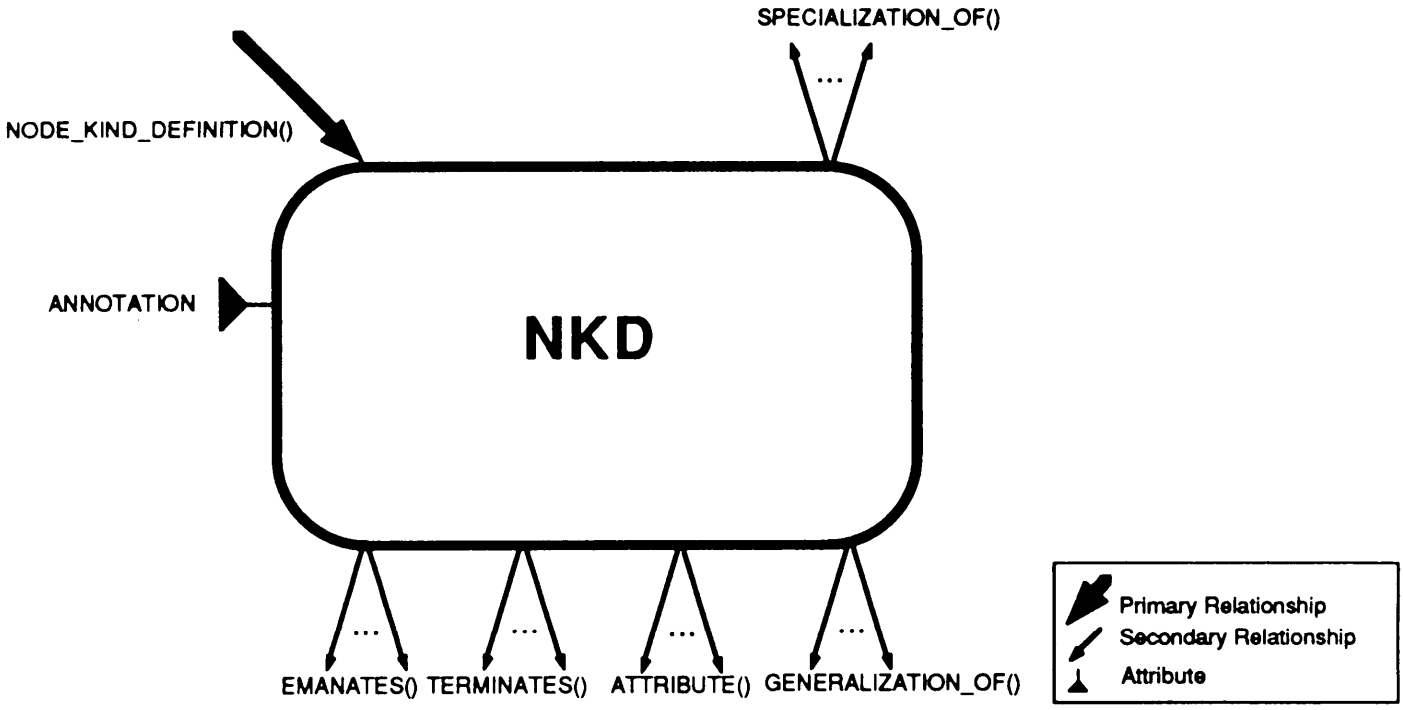


FIGURE 24. Node kind definition node (NKD)

is created with no value for the attribute. DEFAULT_VALUE specifications for attribute properties are inherited by specializations along with the attribute property. They are described in detail in Section 5.7.1.8 on page 334.

EMANATES and TERMINATES relationships emanate from NKDs and terminate at RDs (see Figure 25 on page 309). An EMANATES (TERMINATES) relationship specifies that relationships of the relation defined by the target RD may emanate from (terminate at) nodes defined by the NKD which is the source of that relationship. EMANATES and TERMINATES relationships define the emanating relations and *terminating relations* for an NKD. Emanating relations and terminating relations are properties defined by an NKD and are inherited by specializations of that NKD. 5

EMANATES relationships may have the attributes MINIMUM_INSTANCES and MAXIMUM_INSTANCES. These attributes constrain the number of relationships conforming to the target RD of the EMANATES relationship which may emanate from a node defined by the NKD which is the source of the EMANATES relationship. The collection of relationships constrained includes all relationships of specializations of that relation. The MAXIMUM_INSTANCES attribute specifies the maximum number of relationships allowed, and the MINIMUM_INSTANCES attribute specifies the minimum number of relationships which must be retained. The MINIMUM_INSTANCES constraint is enforced only when relationships are deleted. 6

The constraints imposed by the MINIMUM_INSTANCES and MAXIMUM_INSTANCES attributes of an EMANATES relationship are inherited by specializations of the source NKD. However, narrower bounds may be defied for a specialization by another EMANATES relationship which terminates at 7


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

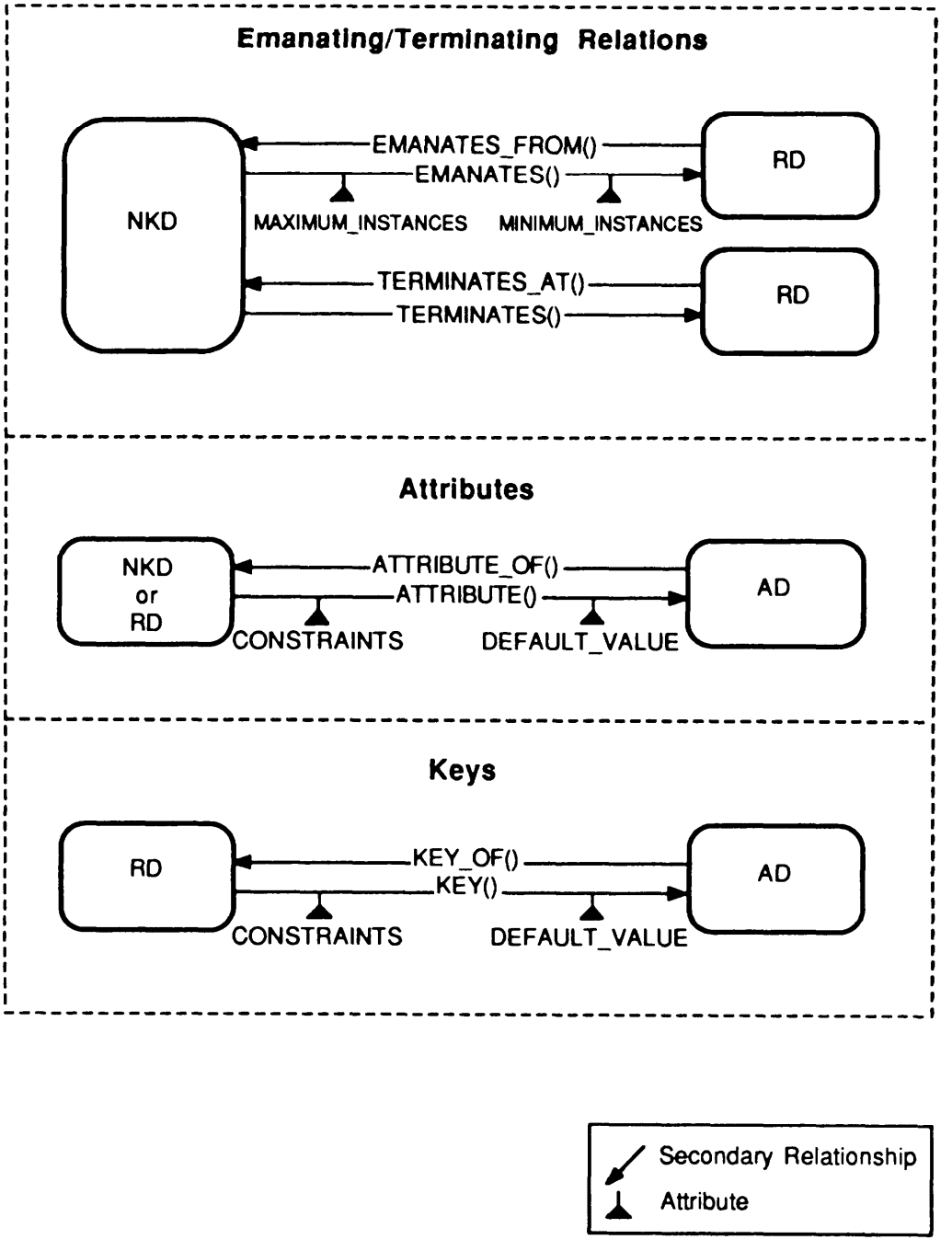


FIGURE 25. NKD and RD property representation


Source: https://assist.dla.mil -- Downloaded: 202610-26T07:54Z
Check the source to verify that this is the current version before use.

the same RD. In this case, the value of MINIMUM_INSTANCES on the EMANATES relationship emanating from the specialization must be greater than or equal to the inherited value, and the value of MAXIMUM_INSTANCES for the specialization must be less than or equal to the inherited value.

The structure of the RD lattice imposes restrictions on the terminating relation properties of an NKD (see Section 5.7.1.4).

#### 5.7.1.4 Relation definition nodes (RDs)

Every relationship is an instance of a relation definition represented by an RD. An RD defines and constrains the properties of all relationships of the relation represented. An RD may have the predefine attributes ALLOWED_USAGE (bidirectional relations only) and UNIQUE_KEYS (see Figure 26). These attributes constrain individual relationships or sets of relationships, emanating from a given node, which are instances of the relation defined or its specializations. Bidirectional ATTRIBUTE and KEY relationships may emanate from an RD. These relationships define properties of the relationships conforming to that RD. The corresponding reverse relations are ATTRIBUTE_OF and KEY_OF. Bidirectional REVERSE_LINK relationships may emanate from an RD. These relationships constrain relationships conforming to that RD to have reverse relationships of the relation represented by the target RD of the REVERSE_LINK relationship. The corresponding reverse relation is REVERSE_-LINK.

Emanating ATTRIBUTE relationships specify the set of attributes defined by an RD. Those attributes that are also keys are specified by KEY relationships. Since keys are specialized attributes, the KEY relation is defined as a specialization of the ATTRIBUTE relation in the predefine RD lattice (see Appendix A). A key may be referenced as an attribute by interfaces which manipulate or iterate over the attributes of a relationship. However, the uniqueness constraints (described below) which apply to keys must not be violated.

Each ATTRIBUTE relationship emanating from an RD terminates at an AD. The AD identifies the attribute and specifies its properties (see Figure 25 on page 309). From a given RD, only one emanating ATTRIBUTE relationship may terminate at a given AD, since a given attribute can have only one value for a given relationship. Each AD defining a relationship attribute has an associated AKD. Those ATTRIBUTE relationships which are also KEY relationships may only terminate at ADs associated with identifier AKDs (see Section 5.7.1.6.5 on page 320). ATTRIBUTE relationships which are not KEY relationships may terminate at ADs associated with any type of AKD.

Properties represented by ATTRIBUTE relationships emanating from a given RD are inherited by specializations of that RD. ATTRIBUTE relationships (and therefore KEY relationships as well) may have the attributes CONSTRAINTS and DEFAULT_VALUE. The CONSTRAINTS attribute defines additional constraints on the attribute applicable to relationships of the relation defined by the RD. These constraints are also inherited by specializations. They are described in detail in Section 5.7.1.7 on page 328. The DEFAULT_VALUE attribute provides a value for an attribute which is not explicitly assigned a value when the relationship is created. Default values are inherited. They are described in detail in Section 5.7.1.8 on page 334.

All relationships emanating from a given node must be uniquely identifiable. Thus, any RD which is used to create relationships must define or inherit a set of keys called a unique key set. The keys of an RD define a *unique key set* if the RD has the UNIQUE_KEYS attribute (which must have the value TRUE) and no generalization of the RD has that attribute. The unique key set defined by such an RD is inherited by specializations of the RD. When a new relationship is created, the key-value associations specified for the keys in a unique key set must be distinct from the corresponding key-value associations


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

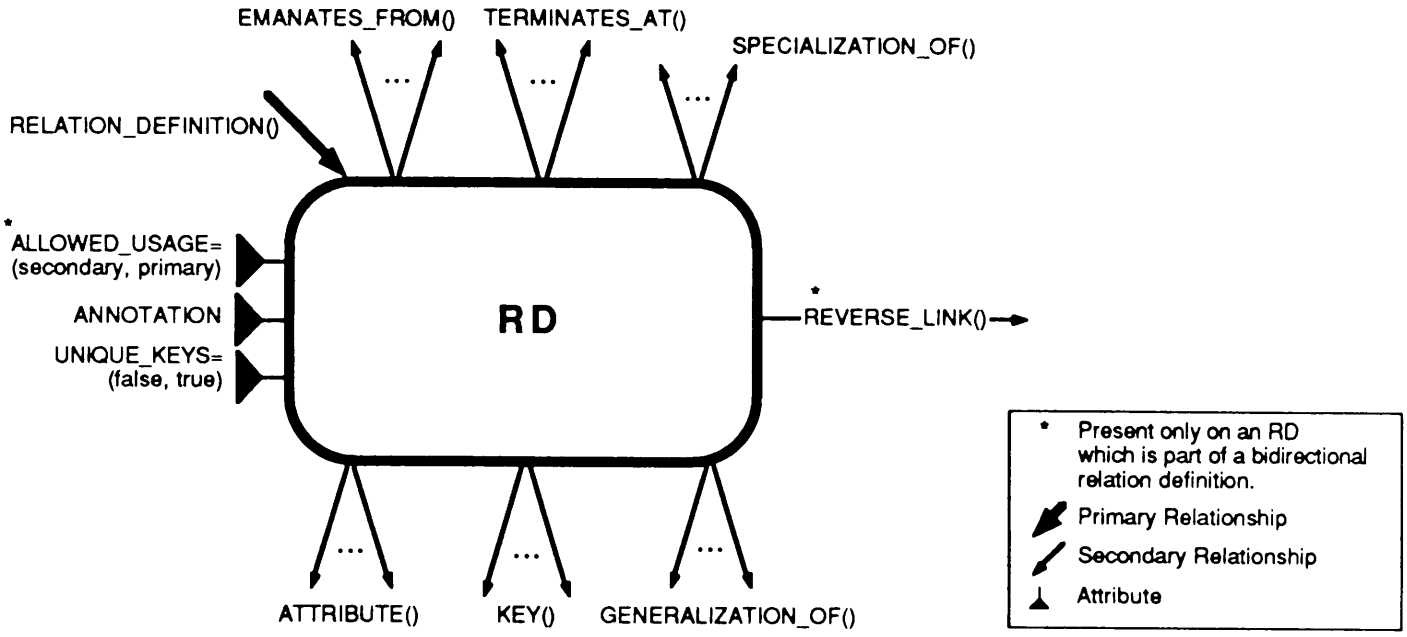


FIGURE 26. Relation definition node (RD)


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

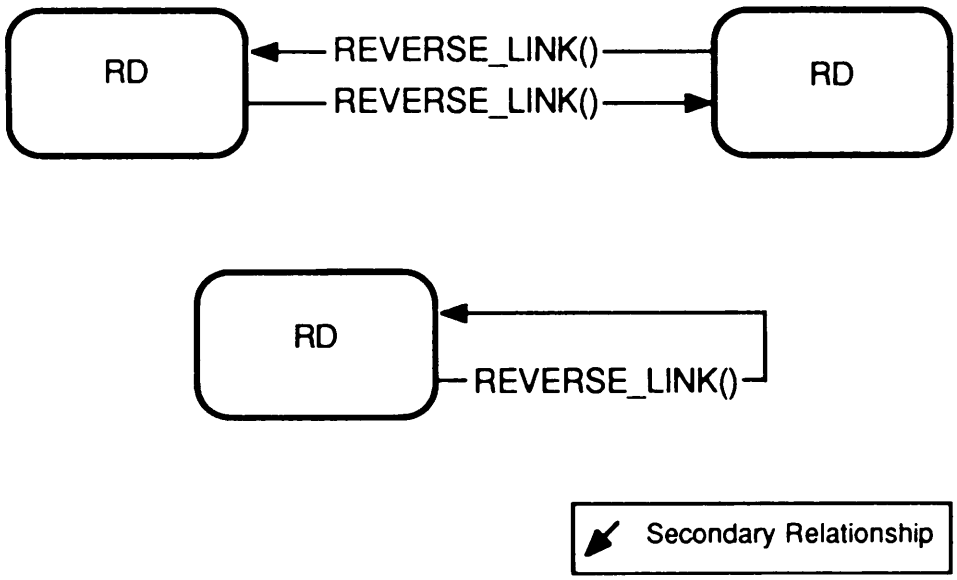


FIGURE 27. Bidirectional RD

of every existing relationship, of the relation that defines the unique key set, which emanates from the same node. For purposes of key value comparison, a key which has no value is considered to have a special value (the null key) which is distinct from any actual value of that key.

Instances of an RD may be created only if the RD defines or inherits a unique key set. Such an RD is said to have a UNIQUE_KEYS constraint. If the UNIQUE_KEYS attribute occurs on an RD which has no keys, the unique key set defined is the empty set. In this case, at most one relationship of the relation defined by the RD may emanate from the same node, since there are no key values to distinguish multiple relationships of the relation. An RD may inherit more than one unique key set. Emanating relationships of the relation defied by such an RD have more than one unique identification. 6

A unidirectional relation definition is represented by a single RD. A bidirectional relation definition may be represented by a pair of distinct RDs which are related by a bidirectional REVERSE_LINK relationship emanating from each RD of the pair and terminating at the other (see Figure 27 on page 312). Alternatively, a bidirectional relation may be represented by a single RD and a unidirectional REVERSE_LINK relationship emanating from and terminating at the same RD. Whenever a relationship is created using an RD that is part of a bidirectional relation definition, a corresponding reverse relationship is created using the RD identified by the REVERSE_LINK relationship. Similarly, when one of the relationships comprising a bidirectional relationship is deleted, the corresponding reverse relationship is also deleted. When a bidirectional relation definition consists of a single RD, the pair of relationships comprising an instance of the definition are instances of the same relation. 7

RDs belonging to a bidirectional relation definition may have the identifier-valued attribute ALLOWED_USAGE which constrains the relationships created using those RDs or any specialization of those RDs. This attribute may have the value PRIMARY or SECONDARY. The value PRIMARY requires relationships of the relation to be primary relationships while the value SECONDARY requires those relationships to be secondary. In the absence of the ALLOWED_USAGE attribute, an RD which belongs to a bidirectional relation definition may be used to create either a primary relationship or a secondary relationship. ALLOWED_USAGE may not have the value PRIMARY on both RDs be- 8


Source: https://assist.dla.mil - Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

longing to a bidirectional relation definition. The constraints represented by the ALLOWED_USAGE attributes are inherited by specializations.

Specializations of RDs which belong to a bidirectional relation definition must also be bidirectional. The specializations of a particular bidirectional relation must have reverse relations which are specializations of the given bidirectional relation's reverse relation. (Consequently, all bidirectional relationships identified through a given bidirectional relation will have reverse relationships identifiable through that relation's reverse relation.) 9

RDs may be targets of EMANATES and TERMINATES relationships which emanate from NKDs. These EMANATES and TERMINATES relationships define the kinds of nodes from which instances of the RD may emanate and the kinds of nodes at which those instances may terminate. 10

The RD lattice imposes restrictions on the target node kinds for relations. In particular, a relationship of a given relation may terminate at a node of a given node kind only if relationships of all generalizations of that relation may terminate at nodes of that kind. Thus, a given NKD may be the source of a TERMINATES relationship which terminates at a given RD only if every generalization of that RD is the target of a TERMINATES relationship emanating from the same NKD or a generalization of the NKD. 11

The topmost RD in the RD lattice, the basic RD, is the target of an EMANATES relationship and a TERMINATES relationship emanating from the basic NKD at the top of the NKD lattice (see Figure 28). These relationships allow all nodes and their relationships to be viewed through the basic NKD and the basic RD. However, instances of the basic RD cannot be created because the basic RD does not define a unique key set. 12

The basic RD defines the attributes INHERITABLE and NODE_KIND, which have values for all relationships. The NODE_KIND attribute is an identifier-valued attribute which specifies the general node kind, STRUCTURAL, PROCESS or FILE, of the target node of a relationship. The INHERITABLE attribute is a Boolean attribute (see Section 5.7.1.6.5 on page 320). The INHERITABLE attribute of a secondary relationship emanating from a node specifies whether the relationship is inheritable (see Section 4.3.2.2). The INHERITABLE attribute is always FALSE for relationships which are part of a bidirectional relationship. 13

### 5.7.1.5 Attribute definition nodes (ADs)

Every attribute is associated with an attribute definition represented by an AD. Each AD defines a distinct attribute. Instances of an AD are attribute values which convey information about the nodes and relationships on which they occur. Instances of the same AD may occur on nodes of various kinds and on relationships of various relations. Each node or relationship may have at most one instance of a given AD, that is, at most one value for a given attribute. The AD for an attribute is used to identify the value of that attribute in the context of a particular node or relationship. The AD for an attribute indicates the set of allowed values for that attribute and may restrict the use of that attribute (see Figure 29). 1

Each attribute which may occur on an instance of an NKD or RD is represented by an ATTRIBUTE relationship which emanates from that NKD or RD (or from a generalization of that NKD or RD) and terminates at an AD (see Figure 25 on page 309). 2

A bidirectional ATTRIBUTE_KIND relationship emanates from each AD and terminates at an AKD, The target AKD specifies the set of allowed attribute values for the attribute defined by the AD. The ATTRIBUTE_KIND relationship may have the CONSTRAINTS attribute, which further restricts 3



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

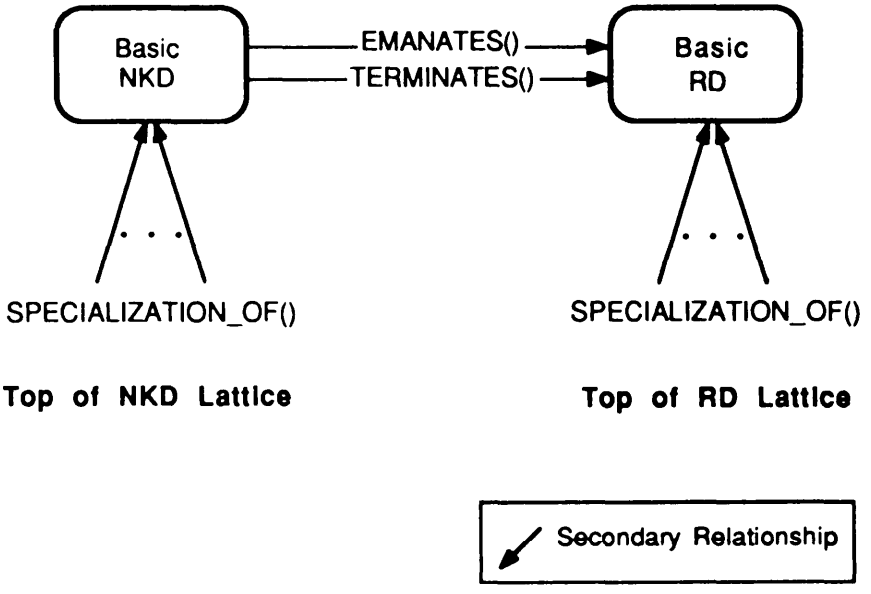


FIGURE 28. Basic NKD and RD


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

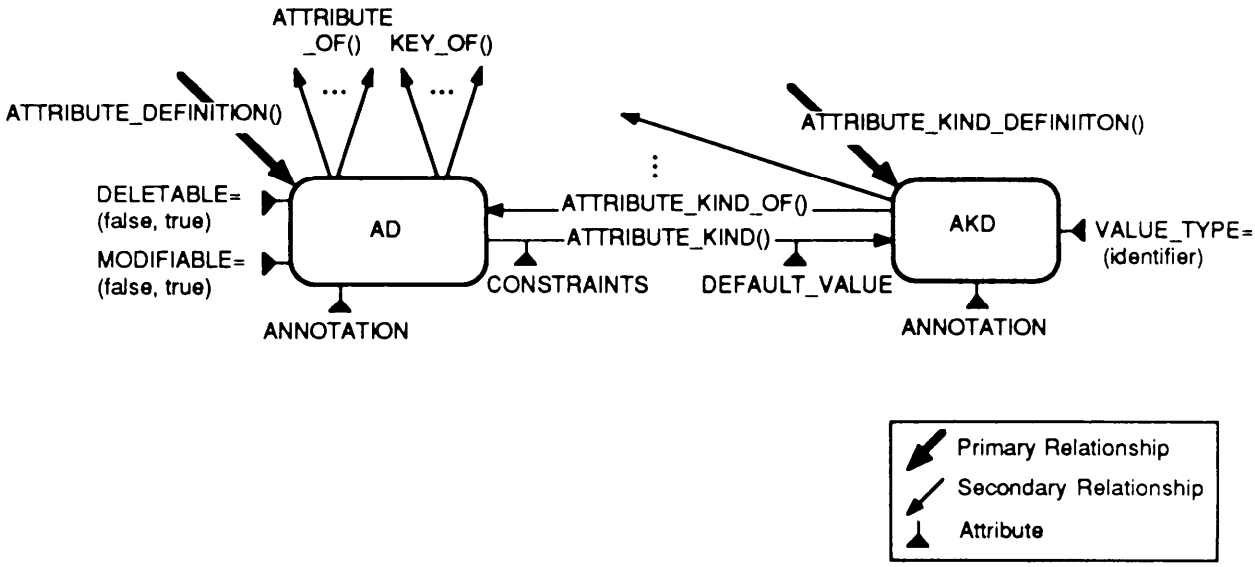


FIGURE 29. Attribute definition node (AD) and attribute kind definition node (AKD)

the allowed values, and the DEFAULT_VALUE attribute which defines a default value for the attribute. Any value assigned to an attribute defined by the AD must conform to this AKD and the CONSTRAINTS attribute (if any). Default values for attributes are described in Section 5.7.1.8 on page 334. The corresponding reverse relation is ATTRIBUTE_KIND_OF. An attribute defined by an AD associated with a particular AKD is said to be an attribute of that particular kind.

All ADs also have the following predefined, Boolean-valued attributes:

DELETABLE, which is TRUE if an attribute value may be deleted after it has been created and FALSE otherwise, and

MODIFIABLE, which is TRUE if an attribute value may be modified after it has been created and FALSE otherwise.

MODIFIABLE must be true if DELETABLE is.

### 5.7.1.6 Attribute kind definition nodes (AKDs)

In the CAIS, every attribute value must conform to an attribute kind definition represented by an AKD. AKDs constrain the values of attributes (instances of ADs). In particular, an AKD specifies the set of allowed values for every AD which is the source of an ATTRIBUTE_KIND relationship that terminates at that AKD (see Figure 29 on page 315).

Attribute values are modeled as CAIS lists. Lists are represented at the CAIS interfaces as ob-


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

jects of the implementation-defined, limited private type LIST_TYPE in the package CAIS_LIST_-MANAGEMENT. Lists are manipulated using the CAIS interfaces defined in that package. A full understanding of AKDs requires knowledge of the general structure of CAIS lists which is presented in the beginning of the section on CAIS LIST MANAGEMENT (Section 9 on page 745).

The CAIS defines several types of attribute values. The value type defined by an AKD is represented by its VALUE_TYPE attribute (see Figure 29 on page 315). The possible values for VALUE_TYPE and the types of values represented are:

INTEGER - a signed integer value.

FLOAT - a signed floating point value.

STRING - a literal character string.

IDENTIFIER - a string having the syntax of an Ada identifier, a token representing such a string, or a value of an Ada enumeration type.

VARIANT - a value conforming to one of several alternative attribute kinds.

LIST - a named or unnamed CAIS list in which all items are of unspecified kinds.

UNIFORM_LIST - a named or unnamed CAIS list in which all items are of a specified attribute kind.

COMPOSITE_LIST - a named or unnamed CAIS list with items of various attribute kinds.

A different kind of AKD is used to represent each of the above value types. The kind of an AKD is denoted by the value of its VALUE_TYPE attribute (printed in lowercase) followed by “AKD”. For example, an “integer AKD” is the kind of AKD which has INTEGER as the value of its VALUE_TYPE attribute. The following abbreviations are used to identify the kinds of AKDs:

AKD - Any kind of AKD

IAKD - Integer AKD

FAKD - Float AKD

SAKD - String AKD

IDAKD - Identifier AKD

VAKD - Variant AKD

LAKD - List AKD

ULAKD - Uniform List AKD

CLAKD - Composite List AKD

Integer, float, string and identifier AKDs define simple, unstructured values which are modeled as CAIS list items of the corresponding item type. The attribute kinds defined by these AKDs are called simple attribute kinds.

Variant AKDs define values which must conform to any one of several different AKDs. For example, a variant AK D might define values which represent terms in an expression. Such a variant AKD could allow a term to be either a floating point value defined by a float AKD, an integer value defined by an integer AKD or an identifier value defined by an identifier AKD (e.g., the name of a mathematical or



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

physical constant such as PI, E, C, R). The interfaces in package CAIS_LIST_MANAGEMENT can be used to determine the particular value type of an item which conforms to a variant AKD.

List, composite list and uniform list AKDs define *structured attribute kinds*. The values of structured attribute kinds are modeled as lists which may have multiple items. The items are called components. Lists conforming to a list, uniform list, or composite list AKD are referred to as *lists, uniform lists* or *composite lists,* respectively. Additional, related AKDs are used to define the allowed values for the components of uniform list and composite list attribute kinds.

List AKDs define list structures which may be constrained to have either named or unnamed components but are otherwise unconstrained. These lists may have any number of components (list items) of any kinds.

Uniform lists consist of components which are all of the same kind. Uniform lists may be named and accessed by name as well as by position. The number of items in uniform list values may be constrained or unconstrained.

Composite lists are aggregates in which each component has a particular and possibly distinct kind. The components of a composite list may be accessed by name or by position. Variant AKDs can be used in conjunction with composite list AKDs to define list structures which are similar to Ada records with variant parts.

Attributes of all value types may be manipulated in the form of lists using interfaces defined in packages CAIS_ATTRIBUTE_MANAGEMENT and CAIS_LIST_MANAGEMENT. The list form of an attribute value conforming to a simple attribute kind (integer, float, string or identifier) is an unnamed list containing a single item of the appropriate kind. Package CAIS_ATTRIBUTE_MANAGEMENT provides interfaces for accessing such simple attribute values directly without using the list form. Attributes conforming to a structured attribute kind are always manipulated as lists.

Note:

When an AKD for a simple attribute kind is used to define a component of a uniform list or composite list attribute, the corresponding component value is represented as a single item, rather than a list containing a single item. For example, an integer attribute with the value 10 is represented in list form as the list

(10).

However, a uniform list having two components with the integer values 5 and 9 is represented by the list

(5, 9)

rather than

((5), (9)).

**5.7.1.6.1 Structure and Semantics of AKDs** Each kind of AKD has certain predefine attributes and may have emanating relationships which terminate at other AKDs, The predefine attributes applicable to various kinds of AKDs are described in Table XII on page 319. Most of these attributes represent constraints on the values which conform to the AKD. In checking conformance to constraints, CAIS implementations must perform various relational operations on the list items that



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

represent attribute values. Relational operations on integer items are defined according to the CAIS type CAIS_INTEGER. Relation operations on floating point items are defined according to a floating point type with CAIS_PRAGMATICS.LIST_MAXIMUM_DIGITS accuracy. All other relational operators required are equality operators defined in Section 9 on page 745.

Certain kinds of AKDs define components or alternatives via relationships which emanate from that AKD and terminate at other AKDs (see Table XIV on page 320). These relationships may have the attributes specified in Table XIV on page 320.

The sections which follow present semantic details of the various kinds of AKDs. The AKDs for simple items and variants are illustrated in Figure 30 on page 321. The various list AKDs are illustrated in Figure 32 on page 325.

**5.7.1.6.2 Integer AKDs** The LOWER_BOUND and UPPER_BOUND attributes may be used to constrain the range of integer values which conform to an integer AKD. When both the LOWER_BOUND and UPPER_BOUND attributes are specified, the UPPER_BOUND value must be greater than or equal to the LOWER_BOUND value.

All integer values must fall in the range defined by the constants MINIMUM_INTEGER and MAXIMUM_INTEGER defined in the package CAIS_PRAGMATICS (see Section 12 on page 815).

**5.7.1.6.3 Float AKDs** Float AKDs have the integer attribute DIGITS which specifies the required precision for conforming floating point values. The value of the DIGITS attribute must not exceed the constant LIST_MAXIMUM_DIGITS defined in the package CAIS_PRAGMATICS (see Section 12 on page 815).

The LOWER_BOUND and UPPER_BOUND attributes may be used to constrain the range of floating point values which conform to the AKD. When both the LOWER_BOUND and UPPER_BOUND attributes are specified, the UPPER_BOUND value must be greater than or equal to the LOWER_BOUND value.

**5.7.1.6.4 String AKDs** The strings which conform to a string AKD are constrained by the PATTERNS attribute. The PATTERNS attribute is an unnamed list of quoted string items, Each quoted string item is a pattern representing allowed values of the string attribute kind. A pattern may contain the wild card characters “*” and “?”. The “*” character matches zero or more characters in a string attribute value, and the “?” character matches any single character. All other characters in the pat tern match themselves. Uppercase and lowercase characters are distinct.

A string conforms to a string AKD if the string matches any of the patterns specified in the PATTERNS attribute.

The PATTERNS attribute value

PATTERNS = ("John *", "Bob S*", "Mary ?. *")

matches the strings

"John Doe" "Bob Smith" "Mary K. Lee"

but does not match the strings

"John" "Bob Hines" "Mary TI. Lee".



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

TABLE XII: **Predefined AKD Attributes**

| **Attribute Name** | **Attribute Kind** | **Applicable AKDs** | **Description** |
|---|---|---|---|
| VALUE_TYPE | identifier | any | Type of value. |
| ANNOTATION | list | any | User's description of AKD. |
| LOWER_BOUND | integer<br>float | INTEGER<br>FLOAT | Lowest allowed value. |
| UPPER_BOUND | integer<br>float | INTEGER<br>FLOAT | Highest allowed value. |
| DIGITS | integer | FLOAT | Precision of values. |
| PATTERNS | uniform list of strings | STRING<br>IDENTIFIER | Patterns to match. |
| ENUMERATED | Boolean | IDENTIFIER | Type of identifier AKD:<br>TRUE => Enumerated identifiers.<br>FALSE => Ordinary identifiers. |
| MINIMUM_SIZE | integer | UNIFORM_LIST | Least number of list items. |
| MAXIMUM_SIZE | integer | UNIFORM_LIST | Greatest number of items. |
| NAMING | identifier<br>(NAMED,<br>UNNAMED) | LIST<br>UNIFORM_LIST<br>COMPOSITE_LIST | Type of list:<br>NAMED => Named list only.<br>UNNAMED => Unnamed list only.<br>omitted => Named or Unnamed. |
| ORDERED | Boolean | COMPOSITE_LIST | Ordering of list:<br>TRUE => Ordered components,<br>FALSE => Unordered components. |

Notes:

1. Only the ANNOTATION attribute may be created and modified via the interfaces in package CAIS_ATTRIBUTE_MANAGEMENT (see Section 5.3 on page 185). All the other attributes must be manipulated via the interfaces defined in package CAIS_DEFINITION_MANAGEMENT.

2. The attributes VALUE_TYPE, DIGITS, PATTERNS, ENUMERATED and ORDERED are required on the kinds of AKDs to which they apply. NAMING is required for composite list AKDs. The remaining attributes are optional.

3. The attributes MINIMUM_SIZE and MAXIMUM_SIZE must be non-negative integer values less than or equal to the integer constant LIST_LENGTH defined in the package CAIS_PRAGMATICS (see Section 12 on page 815).



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

TABLE XIII: **Predefined Emanating Relations for AKDs**

| Relation Name | Source AKD Kind | Target AKD Kind | Description of Relationship |
|---|---|---|---|
| COMPONENT | uniform list<br>composite list | any AKD | Defines all list components.<br>Defines a list component. |
| ALTERNATIVE | variant | any AKD | Defines an alternative of a variant AKD. |

Notes:

1. Reverse links for terminating relations are not included in this table.

2. The relations in this table are all bidirectional with corresponding reverse relations COMPONENT_OF and ALTERNATIVE_OF.

3. A variant AKD must have at least one emanating ALTERNATIVE relationship.

TABLE XIV: **Predefined Attributes of Relations Emanating from AKDs**

| Attribute Name | Attribute Kind | Relations | Description of Attribute |
|---|---|---|---|
| NAME (key) | identifier | COMPONENT<br>ALTERNATIVE | Defines component names in composite lists.<br>Identifies alternatives of a variant . |
| POSITION (key) | identifier | COMPONENT | Orders the components of a composite list. |
| OPTIONAL | Boolean | COMPONENT | TRUE if the component is optional, FALSE if required. |
| CONSTRAINTS | composite list | COMPONENT<br>ALTERNATIVE | Imposes constraints on the target AKD. |

Notes:

1. The NAME key is required for named composite lists. The POSITION key is required for ordered composite lists. OPTIONAL is required. CONSTRAINTS is optional.

2. The NAME and POSITION key values on all COMPONENT relationships emanating from a composite list AKD must be distinct.

3. Values of the CONSTRAINTS attributes must be applicable to the target AKD (see Section 5.7.1.7 on page 328).

**5.7.1.6.5 Identifier AKDs and Enumerated Identifier AKDs** Identifier values may be represented in three different external forms: identifier text form, token form and enumeration form. Section 9 on page 745 describes these forms in detail. Briefly, the identifier text form is a string which has the syntax of a Ada identifier. The token form is a limited private object (of type TOKEN_TYPE defined in package CAIS_LIST_MANAGEMENT) which serves as a token for the identifier text form. The enumeration form is a value of an Ada enumeration type whose image is equal to the identifier text form under identifier equality.


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

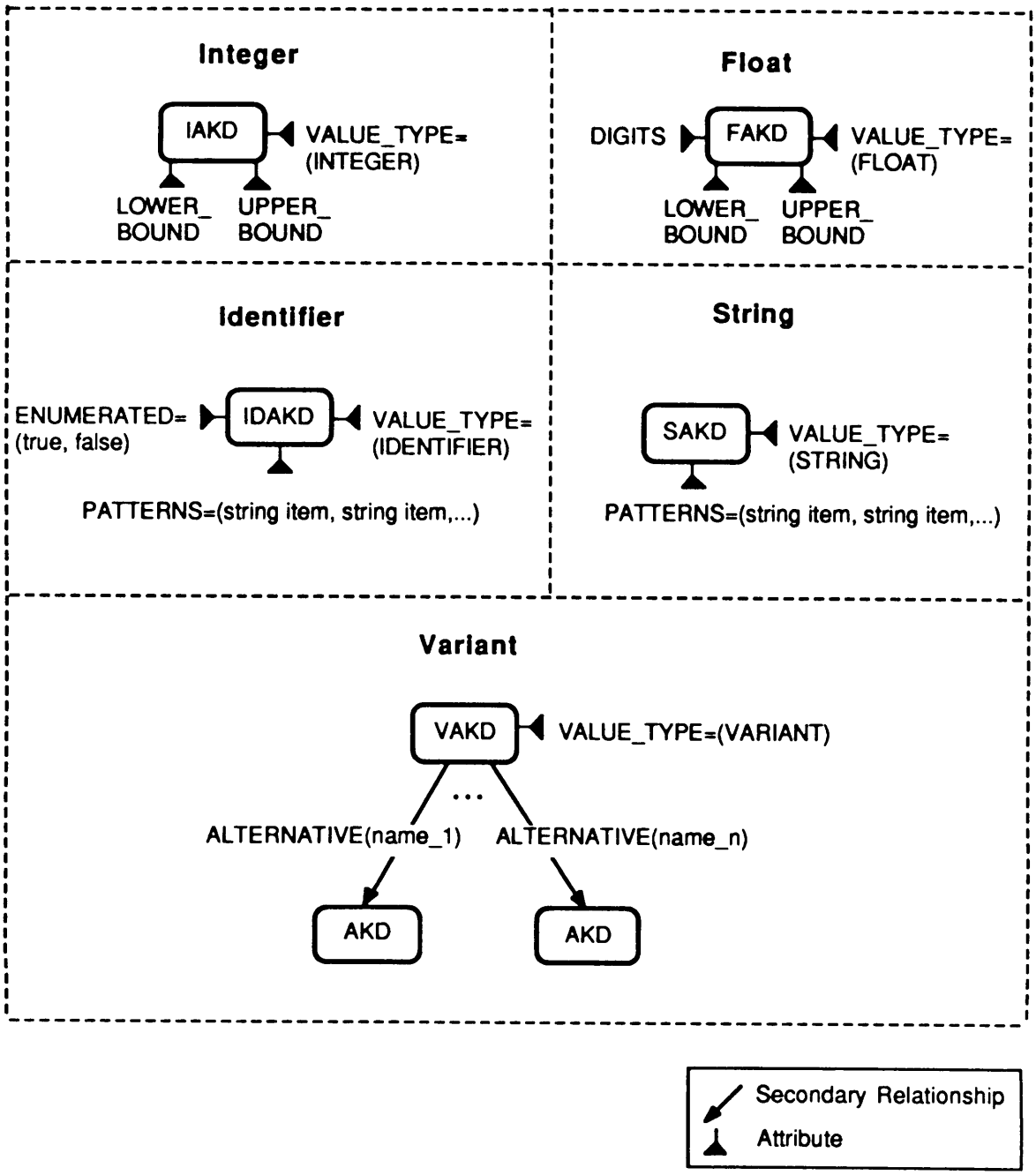


FIGURE 30. Simple and variant AKDs



Source: https://assist.dla.mil - Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Identifier AKDs are similar to string AKDs. Identifier AKDs have the PATTERNS attribute. The identifier text form of the identifier values which conform to an identifier AKD are constrained by the PATTERNS attribute in the same manner as strings which conform to a string AKD (see Section 5.7.1.6.4 on page 318). However, pattern matching for identifier values is not case sensitive; identifier equality applies. 2

Identifier AKDs have the Boolean-valued attribute ENUMERATED. If ENUMERATED has the value TRUE, the identifier AKD is called an enumerated identifier AKD or an enumeration AKD. If ENUMERATED is FALSE, the identifier AKD is called an ordinary identifier AKD. The PATTERNS attribute of an enumerated identifier AKD must contain only patterns which have the syntax of Ada identifiers; wild card characters are not allowed. The patterns enumerate all conforming identifier values. The order of the patterns is significant. Each conforming identifier value is associated with an enumeration position which is equal to the position of the corresponding pattern in the PATTERNS attribute minus one. 3

The generic package CAIS_ENUMERATION_ATTRIBUTE (see Section 5.3.13 on page 235) may be instantiated with an Ada enumeration type in order to access the values of attributes corn-forming to an identifier AKD as enumeration values of the type. When the interfaces in that package are used to access the value of an attribute conforming to an enumerated identifier AKD, the enumeration position of the identifier value (determined by the PATTERNS attribute) must be the same as the position of the corresponding Ada enumeration value which has the same image (see [1815A] 3.5.1 paragraph 4). 4

If an attribute value is retrieved in list form and an item in the list is an instance of an enumerated identifier AKD, the item is represented as an enumerated identifier item rather than an ordinary identifier item (see Section 9 on page 745). The enumerated identifier item includes the enumeration position of the identifier value, determined by the PATTERNS attribute. When a list containing an enumerated identifier item is used to set the value of an attribute and the item must conform to an enumerated identifier AKD, the enumeration position of the item must be the same as the enumeration position of the identifier determined by the PATTERNS attribute. The generic package CAIS_ENUMERATION_ITEM (see Section 9.1.23 on page 780) may be instantiated with an Ada enumeration type to access identifier items in lists as values that Ada enumeration type. 5

Note that both the identifier and enumeration forms may be used to access values conforming to ordinary and enumerated identifier AKDs; however, position checking occurs only when the enumeration form is used to access a value conforming to an enumerated identifier AKD. 6

The predefine BOOLEAN AKD is an enumerated identifier AKD used to define Boolean-valued attributes and components. The BOOLEAN AKD has the following PATTERNS attribute: 7

```
PATTERNS = ("FALSE", "TRUE").
```

The CAIS provides the packages CAIS_BOOLEAN_ATTRIBUTE (an instantiation of CAIS_ENUMERATION_ATTRIBUTE, see Section 5.3.14 on page 245) and CAIS_BOOLEAN_ITEM (an instantiation of CAIS_ENUMERATION_ITEM, see Section 9.1.24 on page 786) for the manipulation of Boolean attribute and component values. 8

**5.7.1.6.6 Variant AKDs** A variant AKD defines a set of alternative AKDs. A conforming value must conform to one of these AKDs. The alternatives are represented by ALTERNATIVE relationships which emanate from the variant AKD and terminate at other AKDs. The alternatives may be AKDs of any kinds, including other variant AKDs. ALTERNATIVE relationships have the NAME key. The NAME key value may be used to identify an alternative in an alternative constraint item occurring in 1



Source: https://assist.dla.mil -- Downloaded: 202610-26T07:54Z
Check the source to verify that this is the current version before use.

a CONSTRAINTS attribute (see Section 5.7.1.7 on page 328). A variant AKD must define at least one alternative.

A value conforms to a variant AKD if it conforms to any of the alternative AKDs and to the CONSTRAINTS attribute (if any) on the corresponding ALTERNATIVE relationship.

Figure 31 illustrates a variant AKD that represents values which are terms in an equation. Any integer, float or identifier item conforms to this AKD.

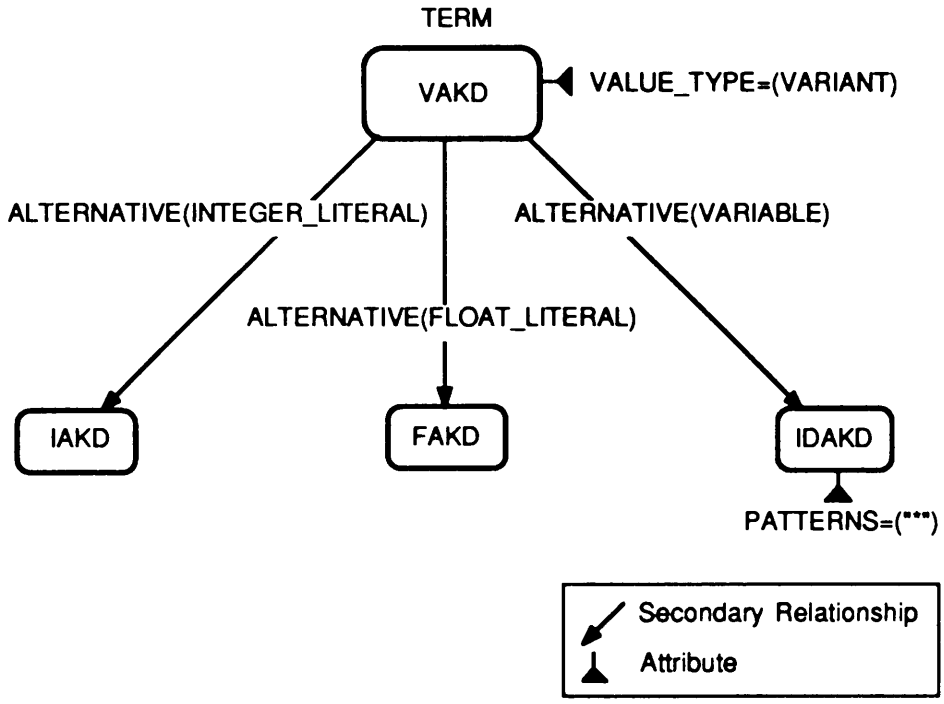


FIGURE 31. Example of a variant AKD

**5.7.1.6.7 List AKDs** A list AKD may have the identifier attribute NAMING with the value NAMED or UNNAMED (see Figure 32). If NAMED is specified, a conforming list must be named. If UNNAMED is specified, a conforming list must not be named. In the absence of the NAMING attribute, any list conforms to a list AKD.

**5.7.1.6.8 Uniform list AKDs** A uniform list AKD has an emanating COMPONENT relationship which terminates at another AKD (see Figure 32 on page 325). Each component of a conforming list must conform to the AKD which is the target of the COMPONENT relationship and to the CONSTRAINTS attribute (if any) on that relationship.

A uniform list AKD may have the identifier attribute NAMING with the value NAMED or UNNAMED. If NAMED is specified, a conforming list must be named. If UNNAMED is specified, a conforming list must not be named. In the absence of the NAMING attribute, a conforming list may


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

be either named or unnamed. The components of a uniform list may be referenced by their positions in the list and by their names if the list is named. The components of a named uniform list may have any names.

A uniform list AKD may have the integer attributes MINIMUM_SIZE and MAXIMUM_SIZE. These attributes constrain the number of items in conforming lists. When the MINIMUM_SIZE attribute is absent, the value O is assumed. When the MAXIMUM_SIZE attribute is absent, the value of the constant LIST_LENGTH in package CAIS_PRAGMATICS is assumed. When both attributes are specified, the value of MINIMUM_SIZE must be less than or equal to the value of MAXIMUM_SIZE.

Figure 33 illustrates the AKD structure which represents the predefine GRANT attribute. The GRANT attribute is related to access control (see Section 5.4.4 on page 259). The syntax for lists conforming to the AKD for the GRANT attribute is specified using the BNF equations which appear in the figure (see Appendices A and D). The AKD for the GRANT attribute is a uniform list AKD which is unnamed and may have any number of components (grant_items). The grant-items, in turn, are represented by uniform lists which have exactly one element and may be named or unnamed. The components of these lists are resulting-rights-lists which are represented by a variant AKD. The alternatives of the variant are: 1) an unnamed, uniform list of identifiers containing at least one identifier and 2) a single identifier. An example of a conforming GRANT attribute value is:

```
( (UPDATE => (READ, WRITE)), (APPEND) )
```

**5.7.1.6.9 Composite list AKDs** Composite list AKDs have the identifier attribute NAMING which must have the value NAMED or UNNAMED. If the value is NAMED, the composite list AKD defines a named composite list AKD and conforming lists must be named. If the value is UNNAMED, conforming lists must be unnamed.

Composite list AKDs also have the Boolean attribute ORDERED. When the ORDERED attribute is TRUE, the composite list AKD defines an ordered composite list AKD and the components of a conforming list must be in a specified order. A composite list AKD must be either named, ordered or both named and ordered (see Table XV).

TABLE XV: **Matrix of Composite List AKDs**

| **NAMING Attribute** | **ORDERED Attribute** | |
|---|---|---|
| | **FALSE** | **TRUE** |
| NAMED | named | named and ordered |
| UNNAMED | illegal | ordered |

A composite list AKD defines the components which may comprise a conforming list. Figure 32 illustrates the structure of a composite list AKD. Each component is defined by a COMPONENT relationship which emanates from the composite list AKD and terminates at another AKD (of any kind). COMPONENT relationships may have the CONSTRAINTS attribute which applies to the target AKD. A component of a conforming list must conform to the target AKD of a corresponding COMPONENT relationship and to the CONSTRAINTS attribute (if any) on that relationship.

COMPONENT relationships have the Boolean attribute OPTIONAL which must have the value TRUE or FALSE. A conforming list must include a component conforming to each COMPONENT relationship for which the OPTIONAL attribute is FALSE.

For a named composite list AKD, each COMPONENT relationship has a NAME key value which



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

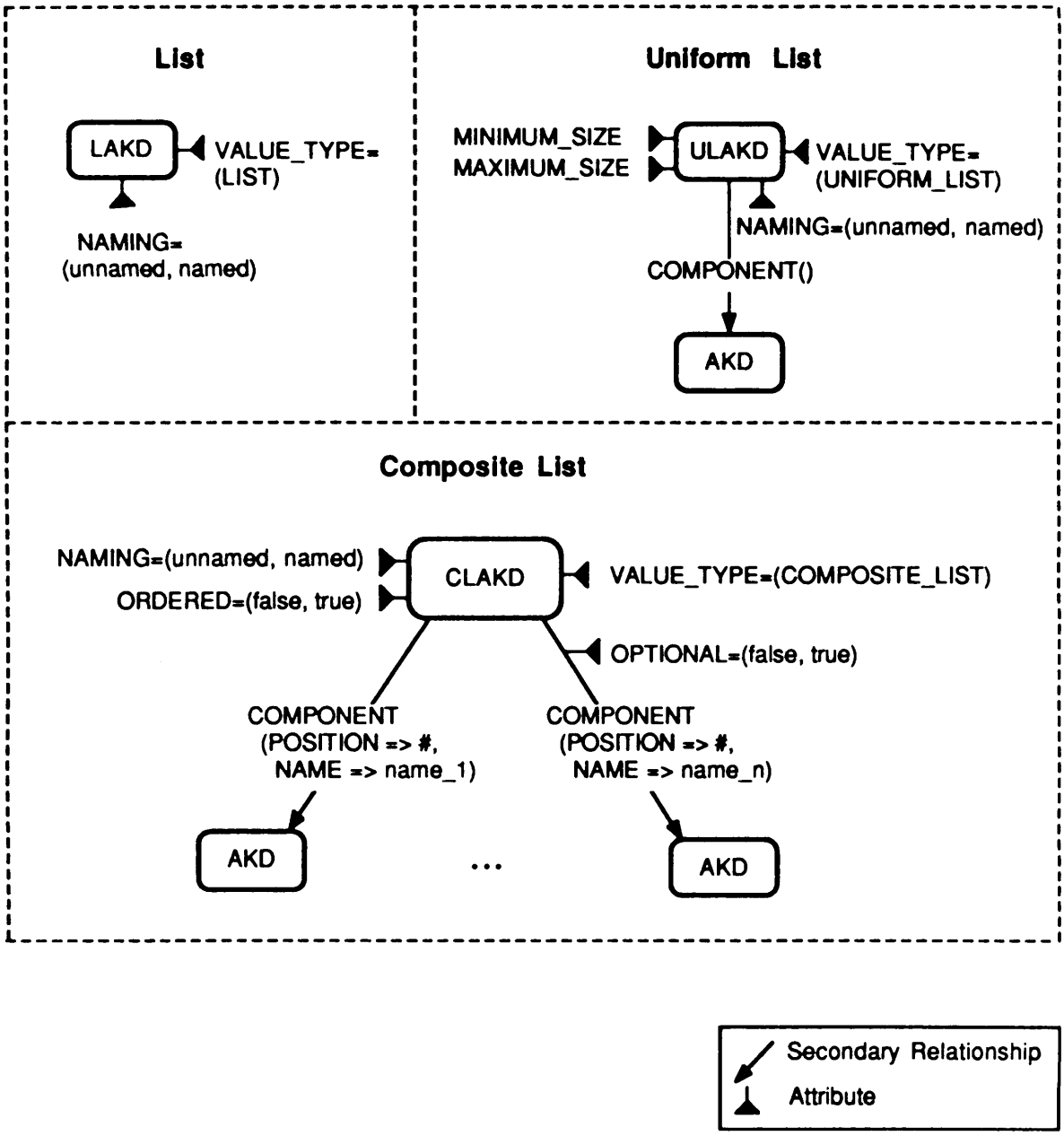


FIGURE 32. List, uniform list and composite list AKDs



Source: https://assist.dla.mil - Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

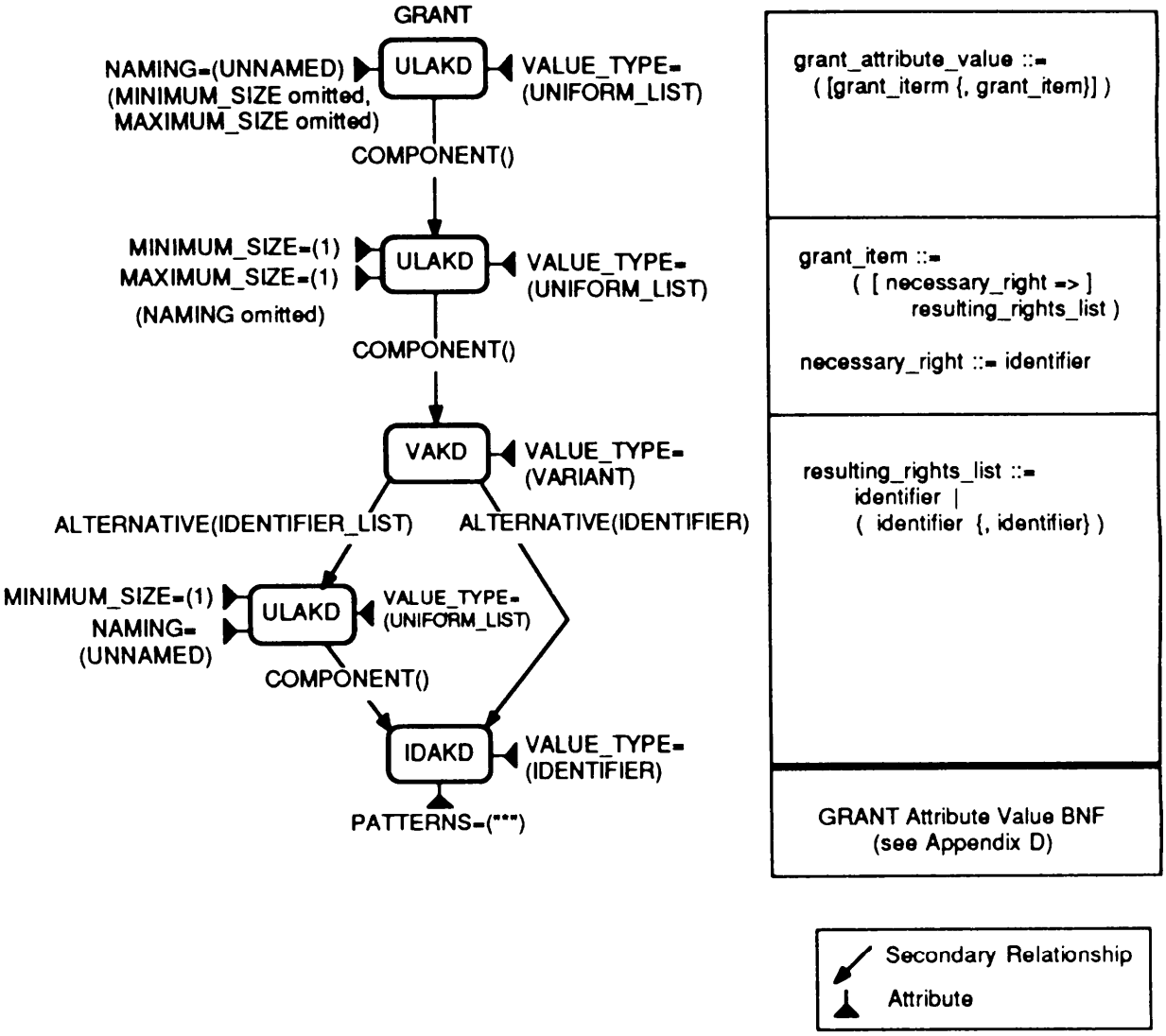


FIGURE 33. GRANT attribute kind representation



Source: https://assist.dla.mil - Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

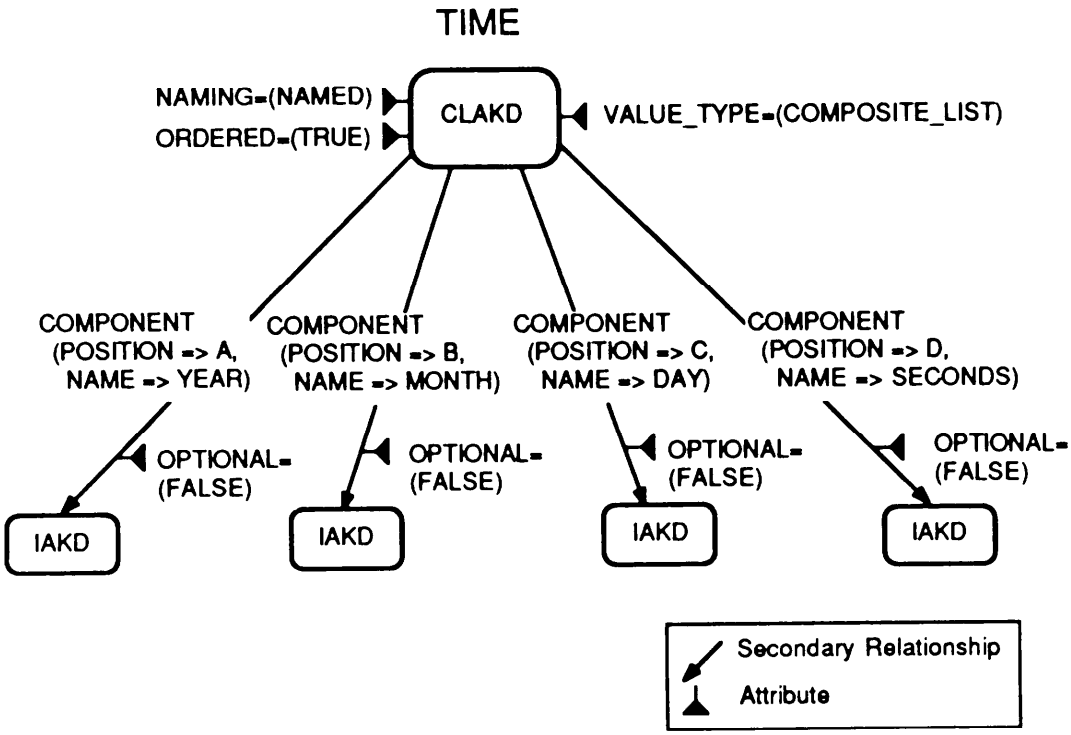


FIGURE 34. TIME attribute kind representation

defines the name of the component. Each named component of a conforming list must conform to a COMPONENT relationship with the corresponding NAME key value.

For an ordered composite list AKD, each COMPONENT relationship has a POSITION key value. 6
Each component of a conforming list must conform to the corresponding COMPONENT relationship in the lexicographical ordering determined by the POSITION key values. In an ordered composite list, only trailing optional components may be omitted.

The COMPONENT relationships of a composite list AKD which is both named and ordered have 7
both NAME and POSITION key values. Components of a list conforming to a named and ordered composite list definition must conform both by name and by position.

Figure 34 illustrates the composite list AKD for the predefined TIME attribute kind (defined in 8
Appendix A). This composite list is both named and ordered. The components are named YEAR, MONTH, DAY, and SECONDS. They are placed in that order by the POSITION key values. An integer AKD with appropriate LOWER_BOUND and UPPER_BOUND attributes (not shown) defines each component.

A variant composite list can be defined as a variant AKD in which each alternative is a composite 9
list AKD. These alternative composite list AKDs may have common components. A component which is shared by the alternatives may be used as a discriminant by placing a different CONSTRAINTS attribute on the COMPONENT relationship in each alternative.


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

#### 5.7.1.7 CONSTRAINTS attributes

Relationships which define certain properties (attributes, alternatives and components) may have the CONSTRAINTS attribute (see Table XVI). A CONSTRAINTS attribute limits the set of values for a property in the context of the definition node which is the source of the relationship defining the property. The relationship terminates at an AKD or AD. The constraints imposed by a CONSTRAINTS attribute augment the constraints specified for that AKD or AD. CONSTRAINTS attributes provide a mechanism for defining sub-kinds of attribute kinds. 1

A CONSTRAINTS attribute always applies to an AKD, directly or indirectly. If the property definition relationship terminates at an AKD, the CONSTRAINTS attribute applies to that AKD. Otherwise, the relationship terminates at an AD and the CONSTRAINTS attribute applies to the AKD associated with that AD, i.e., the AKD which is the target of the ATTRIBUTE_KIND relationship emanating from the AD. 2

CONSTRAINTS attributes on ATTRIBUTE (and therefore KEY) relationships emanating from an NKD or RD are inherited by the specializations of that definition in the NKD or RD lattice. CONSTRAINTS attributes on these relationships may further constrain attributes inherited from generalizations. Figure 22 on page 305, illustrates inheritance of CONSTRAINTS attributes in the NKD lattice. 3

In the context of a given definition, multiple constraints may apply to a property. The constraints imposed by a CONSTRAINTS attribute of a property definition relationship may not be weaker than any constraint imposed by the target AKD or AD or by an inherited CONSTRAINTS attribute for the property. When multiple CONSTRAINTS attributes apply to the same AKD in a given context, they are always equivalent to a single, effective CONSTRAINTS attribute. At least one value must conform to the effective constraints imposed on an AKD; otherwise the constraints are incompatible. 4


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

TABLE XVI: **Predefined Relations that allow the CONSTRAINTS Attribute**

| Predefined Relation | Source Node Kind | Target Node Kind |
|---|---|---|
| ATTRIBUTE | NKD<br>RD | AD<br>AD |
| KEY | RD | AD |
| ATTRIBUTE_KIND | AD | any AKD |
| ALTERNATIVE | variant AKD | any AKD |
| COMPONENT | uniform list AKD<br>composite list AKD | any AKD<br>any AKD |

Values for the CONSTRAINTS attribute are constraints lists. The syntax for constraints lists is specified in Table XVII. A *constraints list* is a composite list consisting of optional named constraint items. Each named constraint item imposes a particular constraint on the values of the constrained property. An empty constraints list is allowed but imposes no (additional) constraints. (An empty constraints list may serve as a place holder in an ordered list of constraints lists.)

There are several kinds of constraint items. The kinds of constraint items which may be applied to a property are determined by the VALUE_TYPE attribute of the AKD for that property (see Table XVIII). A constraint item which may be applied to a property is said to be applicable to the AKD for that property. Some constraint items mirror the AKD attributes with the same names and have the same semantics. Other constraint items have the same names as the properties of AKDs which they constrain. For example, an *alternative* constraint item constrains the alternatives of a variant AKD.

The various kinds of constraint items are described below:

**1) Value Constraint Items:**

*Value constraint items* restrict a property to a single value. The property may have any value type. When a value constraint item is used in a constraints list, it must be the only constraint item in the list. The value specified in a value constraint item in a given context must conform to all other constraint items which are applicable in that context. If a value constraint item applies in a given context, only another value constraint item with the same value may be applied in that context. The effective constraint item that results from combining a value constraint item with any other constraint item is the same as the value constraint item.

A value conforms to a value constraint item if it is equal to the specified value.

Example constraints lists:

(VALUE => 10)

(VALUE => “Washington”)


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

TABLE XVII: **Constraints Attribute Value BNF**

```
constraints_list                  ::=  ( constraint_item {, constraint_item} )
                                     | ()
constraint_item                   ::=  value_constraint_item
                                     | range_constraint_item
                                     | pattern_constraint_item
                                     | alternative_constraint_item
                                     | uniform_list_size_constraint_item
                                     | component_constraint_item
                                     | naming_constraint_item
value_constraint_item             ::= VALUE => item_value
range_constraint_item             ::=  LOWER_BOUND => numeric_item
                                     | UPPER_BOUND => numeric_item
patterns_constraint_item          ::= PATTERNS => ( string_item {, string_item} )
alternative_constraint item       ::=  ALTERNATIVE =>
                                     ( alternative_name => constraints_list
                                     {, alternative_name=> constrained_list} )
uniform_list_size_constraint_item ::=  MINIMUM_SIZE => integer_item
                                     | MAXIMUM_SIZE => integer_item
component_constraint_item         ::=  COMPONENT => constraints_list
                                     | COMPONENT =>
                                     ( constraints_list {, constraints_list} )
                                     | COMPONENT =>
                                     ( component_name = > constraints_list
                                     {, component_name = > constraints_list} )
naming_constraint_item            ::=  NAMING => NAMED
                                     | NAMING => UNNAMED
numeric_item                      ::=  integer_item | float_item
alternative_name                  ::=  identifier
component_name                    ::=  identifier
```

Notes:

1. See Appendix D for a description of the notation used.
2. Items, integer items, float items, string items and identifiers are defined in Section 9, CAIS LIST MANAGEMENT.



Source: https://assist.dla.mil -- Downloaded: 202610-26T07:54Z
Check the source to verify that this is the current version before use.

TABLE XVIII: **Applicable Constraint Items**

| AKD VALUE_TYPE | Applicable Constraint Items | Constraint Item Names |
|---|---|---|
| Any | value_constraint_item | VALUE |
| INTEGER<br>FLOAT | range_constraint_item | LOWER_BOUND<br>UPPER_BOUND |
| STRING<br>IDENTIFIER | patterns_constraint_item | PATTERNS |
| VARIANT | alternative_constraint_item | ALTERNATIVE |
| LIST | naming_constraint_item | NAMING |
| UNIFORM_LIST | naming_constraint_item<br>component_constraint_item<br>size_constraint_item | NAMING<br>COMPONENT<br>MINIMUM_SIZE<br>MAXIMUM_SIZE |
| COMPOSITE_LIST | component_constraint_item | COMPONENT |

**2) Range Constraint Items:**

Range constraint items restrict the range of values for integer and float properties. A range constraint item must have the name LOWER_BOUND or UPPER_BOUND. When both constraint items are specified, the LOWER_BOUND value must be less than or equal to the UPPER_BOUND value. 11

A value conforms to an UPPER_BOUND range constraint item if it is less than or equal to the specified UPPER_BOUND value. A value conforms to a LOWER_BOUND range constraint if it is greater than or equal to the specified LOWER_BOUND value. 12

For the following constraints lists 13

```
(UPPER_BOUND => 5.0)

(LOWER_BOUND => 2, UPPER_BOUND => 7)

(LOWER_BOUND => 1, UPPER_BOUND => 1)
```

the values 4.9, 2 and 1 conform respectively.

**3) Patterns Constraint Items:**

Patterns constraint items restrict the values of string or identifier properties. A patterns constraint item is an unnamed list of quoted strings which represent patterns as in the PATTERNS attribute of a string or identifier AKD. Each pattern which contains a wild card character ("*" or "?") must exactly match a pattern in the PATTERNS attribute of the AKD for the constrained property and a pattern in each of the other patterns constraint items that apply to the property in the context of the CONSTRAINTS attribute. Thus, new wild card patterns for the constrained property may not be defined in a patterns constraint item but the set of wild card patterns may be reduced. All patterns which do not contain wild card characters must conform to the AKD for the property and to any other patterns constraint items that apply in the context. 14

When two or more patterns constraint items apply in a given context, the effective patterns constraint 15


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

item consists of those patterns which are common to all of the constraint items plus those patterns, without wild card characters, that conform to all of the constraint items.

A string or identifier conforms to a patterns constraint item if the string or the identifier text form of the identifier, respectively, matches any of the specified patterns and, in the case of an identifier, if the text form also has the syntax of an Ada identifier. For a string property, uppercase and lowercase characters are distinguished in pattern matching. For an identifier property, matching is insensitive to uppercase and lowercase. 16

Example constraints list: 17

```
(PATTERNS => ("*.library", "*.source", "system.object"))

(PATTERNS => ("Ada.library, "*.source", "*.object", "*.symbol")
```

The equivalent constraint item for combination of the above constraints is: 18

```
(PATTERNS => ("Ada.library, "*.source, "system.object")
```

**4) Alternative Constraint Items:**

*Alternative constraint items* constrain variant properties by restricting the set of allowed alternatives and the values allowed for those alternatives. An alternative constraint item consists of a named list of constraints lists. Each constraints list applies to the alternative having the same name. (The alternative names for a variant AKD are defied by the NAME key values on the emanating ALTERNATIVE relationships.) Each constraints list must be applicable to the AKD which is the target of the corresponding ALTERNATIVE relationship. Only the alternatives named in the list are allowed for the constrained property. An alternative constraint item must contain at least one named constraints list. The constraints list applying to an alternative may be empty, indicating that the AKD which defines the alternative is not further constrained. 19

Each alternative specified in an alternative constraint item in a given context must be an alternative specified in every other alternative constraint item that applies in that context. When two or more alternative constraint items apply in a given context, the equivalent constraint item consists of those alternatives which are common to all of the constraint items; the constraints list for each of these alternatives is the effective constraints list that results from combining all the constraint lists applied to that alternative. 20

A value conforms to an alternative constraint item if it conforms to the constraints list for an alternative named in the alternative constraint item. 21

The constraints list 22

```
(ALTERNATIVE => (INTEGER_LITERAL => (UPPER_BOUND => 10),
                 FLOAT_LITERAL   => ()))
```

is applicable to the example variant AKD in Figure 31 on page 323. This constraints list limits the allowed alternatives to the alternatives named INTEGER_LITERAL and FLOAT_LITERAL; VARIABLE is excluded. The values allowed by the INTEGER_LITERAL alternative are further constrained to be less than or equal to 10. No additional constraints are applied to the FLOAT_LITERAL alternative.


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

**5) Uniform List Size Constraint Items:**

*Uniform list size constraint items* constrain the number of list items allowed for a uniform list property. A uniform list size constraint item must be named MINIMUM_SIZE or MAXIMUM_SIZE. The values must be integers in the range 0 to the constant LIST_LENGTH defined in the package CAIS_PRAGMATICS. When both MINIMUM_SIZE and MAXIMUM_SIZE constraint items are specified, MINIMUM_SIZE must be less than or equal to MAXIMUM_SIZE. 23

A uniform list value conforms to a MINIMUM_SIZE constraint item if it contains at least the specified number of items. A uniform list value conforms to a MAXIMUM_SIZE constraint item if it has no more than the specified number of items. 24

Example constraints lists: 25

```
(MAXIMUM_SIZE => 200)

(MINIMUM_SIZE => 1, MAXIMUM_SIZE => 10)
```

**6) Component Constraint Items:**

Component constraint items restrict the values allowed for the components of uniform lists and composite lists. For a uniform list, a component constraint item is a constraints list named COMPONENT. The constraints list must be applicable to the AKD which is the target of the COMPONENT relationship emanating from the uniform list AKD for the property. The constraint applies to all the items of a uniform list. 26

A uniform list conforms to a component constraint item if each list item conforms to the specified constraints list. The constraints list 27

```
(COMPONENT => (UPPER_BOUND => 5))
```

is applicable to a uniform list AKD with integer components.

For a composite list property, a component constraint item also has the name COMPONENT. The value of the component constraint item must be a list comprised of constraints lists. Each constraints list must correspond to a COMPONENT relationship emanating from the composite list AKD for the property. The list of constraints lists may be named or unnamed. If the component constraint item is a named list, the constrained composite list AKD must be named. Each constraints list applies to the COMPONENT relationship having the same NAME key value. If the component constraint item is an unnamed list, then the constrained composite list AKD must be ordered. The unnamed constraints lists correspond to the composite list components in the lexicographical order determined by the POSITION key values on the COMPONENT relationships. Each constraints list in a component constraint item must be applicable to the target AKD of the COMPONENT relationship. 28

A composite list conforms to a component constraint item if each component in the list conforms to the corresponding constraints list (if any) in that component constraint item. 29

The constraints lists 30

```
(COMPONENT => (YEAR => (VALUE => 1987),
                  MONTH => (UPPER_BOUND => 10))
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

and

```
(COMPONENT => ( (VALUE => 1987),
                (UPPER_BOUND => 10) )
```

are applicable to the TIME AKD in Figure 34 on page 327. They impose equivalent constraints (the second list is positional rather than named).

**7) Naming Constraint Items:**

*Naming constraint items* restrict the types of lists which conform to a list or uniform list property. If the value of the NAMING constraint is NAMED, then a conforming list must be named. If the value of the NAMING constraint is UNNAMED, then a conforming list must be unnamed.

Examples of naming constraints lists:

```
(NAMING => NAMED)

(NAMING => UNNAMED)
```

### 5.7.1.8 DEFAULT_VALUE attributes

ATTRIBUTE_KIND relationships emanating from ADs and ATTRIBUTE relationships (including KEY relationships) emanating from NKDs and RDs may have the DEFAULT_VALUE attribute (see Figure 29 on page 315 and Figure 25 on page 309). The DEFAULT_VALUE attribute provides a default value for an attribute. When a DEFAULT_VALUE attribute occurs on an ATTRIBUTE relationship, the default value is applicable to the attribute defined in the context of the NKD or RD which is the source of the ATTRIBUTE relationship and in the context of specializations of that NKD or RD. When a DEFAULT_VALUE is applied to an ATTRIBUTE_KIND relationship the default value is applicable to the attribute defined by the AD which is the source of the relationship in the context of any NKD or RD which does not define (or inherit) a default value specified on an ATTRIBUTE relationship.

When a node or relationship is created and no value is specified for an attribute which has an applicable default value in the context of the definition for the new node or relationship, that attribute is created with an applicable default value. In the absence of both an explicit attribute value and an applicable default value, the attribute is not created and the new node or relationship will have no value for that attribute.

For an attribute which is not a key, a default value must conform to the constraints which apply to the attribute in the context of the AD for the attribute and in the context of the NKDs or RDs to which the default value applies.

For a key, a default value must be a string having the syntax of a relationship key designator (see Table I on page 52). If the default value for a key ends with “#” (the latest key designator), it must match a pattern for the key which ends with the wild card character “*”.

Since a definition may have generalizations, there can be more than one default value that is applicable to an attribute in the context of a definition. The definition lattices establish a partial precedence for default values. For a given attribute, a default specified in a definition A has higher precedence that a default value specified in a definition B if and only if definition A is a specialization of definition B.

Whenever there are distinct default values applicable to an attribute in the context of a given definition


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

the highest precedence applicable default value is used. The set of default values which are applicable to an attribute in the context of a definition must always include a highest-precedence default value to ensure that the default value to be used is not ambiguous. A highest precedence applicable default value can always be defined for a definition since a default value defined in the definition itself has higher precedence than any other default value that is applicable in the context of that definition.

When a node or relationship is created and an applicable default value is used, that value must conform to the attribute definition to which it is applied and to any constraints applicable to that attribute in the context of the definition for the node or relationship created. To ensure this, an NKD or RD cannot define an attribute which has a DEFAULT_VALUE, specified on the ATTRIBUTE_KIND relationship emanating from the AD for the attribute if that value does not conform to the constraints imposed on the attribute, unless the NKD or RD also defines a DEFAULT_VALUE for the attribute which does conform. Similarly, a specialization cannot inherit a DEFAULT_VALUE which does not conform to constraints in force on the defaulted attribute at the specialization. Just as inherited DEFAULT_VALUES must sometimes be overridden by respecifying them at specializations to ensure one of highest precedence, so must they be explicitly specified to ensure that the DEFAULT_VALUE conforms to any constraints to be placed on the defaulted attribute. 7

The creation, modification or deletion of a DEFAULT_VALUE attribute has no effect on existing instances. 8

#### 5.7.1.9 Evolution of definitions

The CAIS provides interfaces for adding new definitions, for changing existing definition nodes and for deleting definitions. These interfaces are defined in package CAIS_DEFINITION_MANAGEMENT. 1

When an NKD or RD is created, it is added to the corresponding definition lattice as a specialization of some existing definition (a generalization). The predefine NKD lattice contains the basic NKD (the top of the lattice) and a specialization of that NKD for each of the predefined node kinds (STRUCTURAL, DEVICE, FILE, GROUP, PROCESS, NODE_KIND_DEFINITION, etc.). Implementation-defined and user-defined NKDs may be added as specializations of these predefine NKDs or as specializations of other NKDs which have already been added to the lattice. Similarly, the predefine RD lattice contains the basic RD (the top of the lattice) and a specialization of that definition for each of the predefine relations (ACCESS, CURRENT_JOB, GROUP, USER, EMANATES, etc.). Implementation-defined and user-defined RDs may be added as specializations of the predefine RDs or as specializations of other RDs which have already been added. There is no definition lattice for ADs or AKDs. 2

A new NKD or RD automatically inherits all the properties and constraints of the specified generalization. A bidirectional SPECIALIZATION_OF relationship, emanating from the new specialization and terminating at the generalization, is created. The corresponding reverse GENERALIZATION_OF relationship, emanating from the generalization and terminating at the new specialization, is also created. 3

The properties and constraints which are inherited by specializations in the NKD and RD lattices are represented by the relationships and attributes listed in Table XIX. 4


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

TABLE XIX: **Inherited Properties and Constraints**

| **Definition Node Kind** | **Property Definition Relationship** | **Inherited Attributes** |
|---|---|---|
| Node Kind Definition (NKD) | ATTRIBUTE | CONSTRAINTS<br>DEFAULT_VALUE |
| | EMANATES | MINIMUM_INSTANCES<br>MAXIMUM_INSTANCES |
| | TERMINATES | none |
| Relation Definition (RD) | ATTRIBUTE | CONSTRAINTS<br>DEFAULT_VALUE |
| | KEY | CONSTRAINTS<br>DEFAULT_VALUE |
| | REVERSE_LINK | none |
| | none | ALLOWED_USAGE |
| | none | UNIQUE_KEYS |

The CAIS provides interfaces to make an existing NKD or RD a specialization of another existing NKD or RD (in addition to the initial generalization of that NKD or RD and any other generalizations that may already exist). Thus, a definition in a lattice may become a specialization of any number of other definitions. A definition inherits the properties and constraints of all those generalizations.

The CAIS also provides interfaces to make an existing definition become a generalization for other existing definitions. An array of definition nodes specifies the set of existing definitions which are to be specializations. The existing definition which is to be the corresponding generalization is also specified. Those definition nodes in the array which are not already specializations of the generalization become specializations. Common properties, shared by all of the specializations in the array, which are not already properties of the generalization become new properties defined by the generalization. No CONSTRAINTS attributes are specified for those new properties of the generalization.

Existing definitions can be modified. Properties can be added or deleted, constraints can be strengthened, weakened, added or deleted, and SPECIALIZATION_OF relationships can be added or deleted so long as the following general rules are not violated:

1. A constraint cannot be added or strengthened if there are existing instances of the constrained property.
2. A definition may not impose a constraint which is stronger than those imposed by any specialization of that definition.
3. A property or definition may not be deleted if there are instances having that property or other definitions which are dependent on that property or definition.

When it is necessary to change a definition and there are existing instances or specializations which would cause one of the above rules to be violated, a new definition can be created with the required properties and constraints. The existing instances can then be migrated to the new definition.

The CAIS defines interfaces that support the migration of instances of a given NKD to a new NKD (see CONVERT_NODE in Section 5.7.61 on page 469). Migration of relationships can be achieved through the RENAME interface (see Section 5.2.31 on page 134). Properties of the instance which are also defined by the new definition may be retained, however they must confrom to the new definition.



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

When a node is migrated, the new NKD must have all of the terminating relation properties of the original NKD.

When a node or relationship is migrated the values of attributes of the new definition may be specified explicitly, overriding existing values. Any attribute defined by the new definition which is not a attribute defined by the original definition and is not explicitly assigned a value is assigned the default value for the new attribute. In the absence of a default value, the new attribute will have no value. The rules related to key value uniqueness for a new RD must be satisfied by a migrated relationship. The NKD for the node from which a relationship emanates (at which a relationship terminates) must have an emanating (a terminating) property for the new RD.

**5.7.1.9.1 Definition node deletion** Definition nodes cannot be deleted if their deletion would result in incomplete or inconsistent definitions. The following rules apply to the deletion of definition nodes. These rules imply a possible ordering for the deletion of a set of interrelated NKDs, RDs, ADs, AKDs and their instances; that ordering is: node kind instances, NKDs, RDs, ADs and AKDs.

An NKD cannot be deleted if:

a. it is the target of a SPECIALIZATION_OF relationship emanating from an obtainable NKD;

b. it is the target of more than one DEFINITION relationship emanating from obtainable NKIs;

c. there is an obtainable node defined by the NKD.

An RD cannot be deleted if:

a. it is the target of a SPECIALIZATION_OF relationship emanating from an obtainable RD;

b. it is the target of more than one DEFINITION relationship emanating from obtainable RIs;

c. it is the target of an EMANATES relationship emanating from an obtainable NKD;

d. it is the target of a TERMINATES relationship emanating from an obtainable NKD.

An AD cannot be deleted if:

a. it is the target of more than one ATTRIBUTE relationship emanating from obtainable VIEW nodes;

b. it is the target of an ATTRIBUTE relationship emanating from an obtainable NKD or RD.

An AKD cannot be deleted if:

a. it is the target of more than one ATTRIBUTE_KIND relationship emanating from obtainable VIEW nodes;


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

b. it is the target of an ATTRIBUTE_KIND relationship emanating from an obtainable AD;

c. it is the target of an ALTERNATIVE relationship emanating from an obtainable variant AKD;

d. it is the target of a COMPONENT relationship emanating from an obtainable uniform or composite list AKD.

In no case can a definition node be deleted if it is interpreted through a currently adopted view or if it is predefine.

**5.7.1.9.2 Definition relationship deletion** Property definition relationships cannot be deleted if their deletion would result in incomplete or inconsistent definitions. The following rules apply to the deletion of property definition relationships.

An EMANATES relationship emanating from an obtainable NKD and terminating at an obtainable RD cannot be deleted if:

a. there exists an instance of the property (i.e., there exists a node defined by the NKD or a specialization of that NKD and the node has an emanating relationship whose relation is defined by the RD);

b. more than one view contains a RELATION_NAME relationship which corresponds to the relationship (i.e., the RELATION_NAME relationship emanates from an NKI corresponding to the NKD or a specialization thereof and terminates at an RI corresponding to the RD).

A TERMINATES relationship emanating from an obtainable NKD and terminating at an obtainable RD cannot be deleted if:

a. there exists an instance of the property (i.e., there exists a relationship defined by the RD which terminates at a node defined by the NKD or a specialization of that NKD );

b. the NKD or a specialization thereof has an emanating TERMINATES relationship which terminates at a specialization of the RD and would be illegal without the given TERMINATES relationship.

An ATTRIBUTE relationship emanating from an obtainable NKD or RD and terminating at an obtainable AD cannot be deleted if:

a. there exists an instance of the property (i.e., there exists a node or relationship defined by the definition node or a specialization thereof and the node or relationship has the attribute defined by the AD);

b. more than one view contains an ATTRIBUTE_NAME relationship which corresponds to the relationship (i.e., the naming relationship emanates from an interpretation node corresponding to the NKD or RD or a specialization thereof and terminates at the AD).


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

An ATTRIBUTE_KIND relationship emanating from an obtainable AD and terminating at an obtainable AKD can only be deleted by an implementation when the AD which is the source of the relationship is deleted or when a new ATTRIBUTE_KIND relationship is to be created for that AD.

An ALTERNATIVE or COMPONENT relationship emanating from an obtainable AKD and terminating at an obtainable AKD cannot be deleted if:

a. there exists an instance of the property (i.e., there exists a node or relationship which has an attribute defined by the source AKD of the definition relationship or which has that source AKD as an alternative or component, recursively);

b. more than one view contains an ATTRIBUTE_KIND relationship which depends on the relationship (i.e., the ATTRIBUTE_KIND relationships terminate at the source node of the definition relationship or at an AKD which has that source as a component).

A SPECIALIZATION_OF relationship emanating from and terminating at NKDs or RDs can be deleted if the properties which it causes to be inherited and which are not otherwise defined could be deleted relative to the source node.

All property definition relationships are also deleted when their source node is deleted. That is the only time that a DEFINITION relationship emanating from an NKI or RI and terminating at the corresponding NKD or RD is deleted.

In no case can a property definition relationship be deleted if it is interpreted through a currently adopted view or if it is predefine.

**5.7.1.9.3 Definition attributes** The predefine node attributes specific to definition nodes are summarized in Table XX.

The predefined relationship attributes specific to property definition relationships are summarized in Table XXI.

The predefined definition node and relationship attributes cannot be modified after creation if

a. The modification would result in a less restrictive constraint being defined for a property in a definition than is defined for a generalization of the definition.

b. The modification would result in a more restrictive constraint being defined for a property in a definition than is defined for a specialization of the definition. (This is equivalent to the preceding condition.)

c. The modification would result in a more restrictive constraint on a property of which instances exist.

### 5.7.1.10 Access to definition nodes

All modifications of a node, attribute or relationship must be done in accordance with the applicable definitions. This implies that the CAIS implementation reads definition nodes prior to any such modification. In particular, conformance checking of each modification to a definition node is performed using the definitions as represented before the modification.


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

TABLE XX: **Predefine Definition Node Attributes**

| **Attribute** | **Definition Node** | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|
| | NKD | RD | AD | IAKD | FAKD | IDAKD | SAKD | LAKD | ULAKD | VAKD | CLAKD |
| ANNOTATION [1] | x | x | x | x | x | x | x | x | x | x | x |
| ALLOWED_-USAGE | | x | | | | | | | | | |
| DELETABLE [2] | | | x | | | | | | | | |
| MODIFIABLE [2] | | | x | | | | | | | | |
| VALUE_-TYPE [2] | | | | x | x | x | x | x | x | x | x |
| LOWER_-BOUND [3] | | | | x | x | | | | | | |
| UPPER_-BOUND [3] | | | | x | x | | | | | | |
| DIGITS [2] | | | | | x | | | | | | |
| PATTERNS [2] | | | | | | x | x | | | | |
| Enumerated [2] | | | | | | x | | | | | |
| MINIMUM_-SIZE | | | | | | | | | x | | |
| MAXIMUM_-SIZE | | | | | | | | | x | | |
| FAMING | | | | | | | | x | x | | x |
| ORDERED [2] | | | | | | | | | | | x |
| UNIQUE_-KEYS | | x | | | | | | | | | |

Footnotes:

1. ANNOTATION can be modified through CAIS_ATTRIBUTE_MANAGEMENT interfaces; all other attributes can only be created, modified or deleted through the CAIS_DEFINITION_MANAGEMENT interfaces.

2. These definition node attributes are required to be on all definition nodes to which they apply. They are set at definition node creation. Additionally, NAMING is required on CLAKDS.

3. A LOWER_BOUND attribute must have a value which is no greater than the value of the corresponding UPPER_BOUND attribute value, when both are specified.

Legend:

| | | |
|---|---|---|
| NKD | = | Node Kind Definition Node |
| RD | = | Relation Definition Node |
| AD | = | Attribute Definition Node |
| IAKD | = | Integer Attribute Kind Definition Node |
| FAKD | = | Float Attribute Kind Definition Node |
| IDAKD | = | Identifier Attribute Kind Definition Node |
| SAKD | = | String Attribute Kind Definition Node |
| LAKD | = | List Attribute Kind Definition Node |
| ULAKD | = | Uniform List Attribute Kind Definition Node |
| VAKD | = | Variant Attribute Kind Definition Node |
| CLAKD | = | Composite List Attribute Kind Definition Node |


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

TABLE XXI: **Predefine Definition Relationship Attributes**

| Attribute | Relation | | | | |
|---|---|---|---|---|---|
| | ATTRIBUTE_KIND | ALTERNATIVE | ATTRIBUTE, KEY | COMPONENT | EMANATES |
| CONSTRAINTS | x | x | x | x | |
| DEFAULT_-VALUE | x | | x | | |
| NAME | | x | | x | |
| OPTIONAL | | | | x | |
| POSITION | | | | x | |
| MINIMUM_-INSTANCES | | | | | x |
| MAxIMUM_-INSTANCES | | | | | x |

Notes:

1. All of these attributes can only be created, modified or deleted through the Definition Management interfaces.

2. NAME and POSITION are key attributes.

3. The attribute OPTIONAL is required on the definition relationships to which it applies. It is set at relationship creation. All other attributes in this table are optional.

4. DEFAULT_VALUE can be modified after instances of the corresponding attribute exist. The other attributes in the table can be modified only to weaken constraints after instances of the property exist.

5. A MINIMUM_INSTANCES attribute must have a value which is no greater than the value of the corresponding MAXIMUM_INSTANCES attribute value, when both are specified.

NKDs applicable to a prospective target node are accessed during the conformance checking associated with creating a relationship, even though creating a unidirectional relationship generally does not constitute access to the target node itself. Once an NKD becomes unobtainable, it is no longer possible to create unidirectional relationships to its unobtainable instances. To attempt to do so results in a CONFORMANCE_ERROR exception, since an unobtainable definition defines no properties. 2

Implicit accesses of definition nodes performed by the CAIS implementation are not subject to access synchronization with locks which are due to the intents associated with any open node handles on those definition nodes. However, all operations on definition nodes are subject to access synchronization with access synchronization locks held by a transaction, when they are performed outside the given transaction. Consequently, these operations (which include every write operation, as these must provide conformance checks) may block the calling task or fail with a LOCK_ERROR exception. See Section 5.2.2 and Section 5.2.3 beginning on page 96 for a discussion of implicit accesses and delayed accesses. 3

Definition nodes are subject to the same access control rules as other nodes. That is, read access rights are required to directly examine definition nodes and write access rights are required to modify definition nodes. However, granting any write access right to an instance of a given definition also implicitly grants access to the definition node for the purpose of conformance checking. This implicit access right extends to other definition nodes that define properties or generalizations of the given definition node, since those other definition nodes must be accessed during conformance checking, Even granting only read access to an instance allows some inferences to be made about the definition 4


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

of that instance. Therefore, if inferences about definitions are to be limited, access to instances must be limited. In order to create a node (an instance of an NKD), the CREATE_INSTANCES access right to the NKD is required. This access right provides a means to limit inferences about definitions through instances since only authorized subjects can create instances and make them accessible to other subjects.

When an instance of a given definition node is created, modified, or deleted, the definition nodes that may be implicitly accessed for conformance checking include those which are the target of a SPECIALIZATION_OF, EMANATES, ATTRIBUTE, ATTRIBUTE_KIND, COMPONENT or ALTERNATIVE relationship that emanates from the given definition node or from snot her definition node which is implicitly accessed. In addition, when a relationship is created or modified, the NKD for the target node and its generalizations may be accessed, but only to determine whether the target node belongs to a node kind at which the relationship may terminate,

Granting any write access right to a definition also grants the right to have implicit reads of specializetions and generalizations done on the subject's behalf. These reads are implied by the rules governing modifications of inheritable properties and constraints.

### 5.7.2 Predefine entities

The CAIS node model predefine many nodes and associated relationships and attributes. Most of these nodes are definition nodes, Others include the predefine system-level node, the DEFINITIONS node for the predefine definitions, the VIEW node for the predefine view and the interpretation nodes of the predefine view. All predefine nodes and their predefine properties are specified in Appendix A (Predefine Entities).

The CAIS imposes restrictions on the use of CAIS interfaces to create, modify or delete instances of certain predefine definitions and instances of user-defined specializations of those definitions. Additional restrictions apply to the modification of most predefined instances as well. The restrictions are described in the following subsections.

#### 5.7.2.1 CAIS-controlled entities

Most predefine definitions define instances which must be controlled by the CAIS implementation to preserve their semantic integrity. Therefore, instances of such definitions generally cannot be created, modified and deleted directly by users through the general-purpose interfaces such as those defined in the packages CAIS_NODE_MANAGEMENT, CAIS_ATTRIBUTE_MANAGEMENT, CAIS_-STRUCTURAL_NODE_MANAGEMENT and CAIS_FILE_NODE_MANAGEMENT. Instead, instances of these definitions are created and modified through special-purpose interfaces defined in other CAIS packages. For example, definition nodes must be created and deleted using the special-purpose interfaces in package CAIS_DEFINITION_MANAGEMENT.

The definition nodes whose instances are controlled by the CAIS are identified by the Boolean-valued attribute CAIS_CONTROLLED. Definitions for which the CAIS_CONTROLLED attribute is TRUE are called *CAIS-controlled definitions.* All specializations of CAIS-controlled definitions are also CAIS-controlled. Instances of CAIS-controlled definitions are called *CAIS-controlled instances.*

Most of the predefine definitions are CAIS-controlled. AKDs are not CAIS-controlled. The following are the pathnames (in the predefine view) of all the predefine definitions (excepting the AKDs) which are not CAIS-controlled:


Source: https://assist.dla.mil -- Downloaded: 2026l0-26T07:54Z
Check the source to verify that this is the currentversion before use.

Definitions with CAIS_CONTROLLED = FALSE

'DEFINITIONS ( PREDEFINED ) 'ATTRIBUTE_DEFINITION ( ANNOTATION )
'DEFINITIONS ( PREDEFINED ) 'ATTRIBUTE_DEFINITION ( GENERIC_KEY )
'DEFINITIONS ( PREDEFINED ) 'ATTRIBUTE_DEFINITION ( PARAMETERS )

'DEFINITIONS ( PREDEFINED ) 'NODE_KIND_DEFINITION ( BASIC )
'DEFINITIONS ( PREDEFINED ) 'NODE_KIND_DEFINITION ( CLIENT )
'DEFINITIONS ( PREDEFINED ) 'NODE_KIND_DEFINITION ( DEFINITIONS )
'DEFINITIONS ( PREDEFINED ) 'NODE_KIND_DEFINITION ( EXECUTABLE_IMAGE )
'DEFINITIONS ( PREDEFINED ) 'NODE_KIND_DEFINITION ( FILE )
'DEFINITIONS ( PREDEFINED ) 'NODE_KIND_DEFINITION SECONDARY_STORAGE )
'DEFINITIONS ( PREDEFINED ) 'NODE_KIND_DEFINITION ( STRUCTURAL )

'DEFINITIONS ( PREDEFINED ) 'RELATION_DEFINITION ( ATTRIBUTE_DEFINITION )
'DEFINITIONS ( PREDEFINED ) 'RELATION_DEFINITION ( ATTRIBUTE_DEFINITION_PARENT )
'DEFINITIONS ( PREDEFINED ) 'RELATION_DEFINITION ( ATTRIBUTE_KIND_DEFINITION )
'DEFINITIONS ( PREDEFINED ) 'RELATION_DEFINITION ( ATTRIBUTE_KIND_DEFINITION_PARENT )
'DEFINITIONS ( PREDEFINED ) 'RELATION_DEFINITION BASIC )
'DEFINITIONS ( PREDEFINED ) 'RELATION_DEFINITION ( CHANNEL )
'DEFINITIONS ( PREDEFINED ) 'RELATION_DEFINITION ( CHANNEL_PARENT )
'DEFINITIONS ( PREDEFINED ) 'RELATION_DEFINITION ( COPY_OF )
'DEFINITIONS ( PREDEFINED ) 'RELATION_DEFINITION ( DEFAULT_PROCESS_NODE_KIND )
'DEFINITIONS ( PREDEFINED ) 'RELATION_DEFINITION ( DEFINITIONS_PARENT )
'DEFINITIONS ( PREDEFINED ) 'RELATION_DEFINITION ( DEFINITIONS_PRIMARY )
'DEFINITIONS ( PREDEFINED ) 'RELATION_DEFINITION ( DEFINITIONS_SECONDARY )
'DEFINITIONS ( PREDEFINED ) 'RELATION_DEFINITION ( INTRODUCED_NODE )
'DEFINITIONS ( PREDEFINED ) 'RELATION_DEFINITION ( INTRODUCED_NODE_PARENT )
'DEFINITIONS ( PREDEFINED ) 'RELATION_DEFINITION ( NODE_KIND_DEFINITION )
'DEFINITIONS ( PREDEFINE ) 'RELATION_DEFINITION ( NODE_KIND_DEFINITION_PARENT )
'DEFINITIONS ( PREDEFINED ) 'RELATION_DEFINITION ( PARENT )
'DEFINITIONS ( PREDEFINED ) 'RELATION_DEFINITION ( RELATION_DEFINITION )
'DEFINITIONS ( PREDEFINED ) 'RELATION_DEFINITION ( RELATION_DEFINITION_PARENT )
'DEFINITIONS ( PREDEFINED ) 'RELATION_DEFINITION ( RESOURCE_GROUP_SECONDARY )
'DEFINITIONS ( PREDEFINED ) 'RELATION_DEFINITION ( VIEW_PARENT )
'DEFINITIONS ( PREDEFINED ) 'RELATION_DEFINITION ( VIEW_PRIMARY )
'DEFINITIONS ( PREDEFINED ) 'RELATION_DEFINITION ( VIEW_SECONDARY )

The following restrictions apply to the direct manipulation of CAIS-controlled instances except where explicitly noted in the interface descriptions: 4

a. Instances of CAIS-controlled definitions may not be created, copied or deleted using general-purpose CAIS interfaces. 5

b. The NKD of a node cannot be changed (through conversion) so that the set of predefined, CAIS-controlled node kinds to which the node belongs is changed. 6

c. The RD of a relationship cannot be changed (through renaming) so that the set ofprede- 7


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

fined, CAIS-controlled relations to which the relationship belongs is changed.

d. Relationships of CAIS-controlled relations cannot have their source or target node changed using general-purpose CAIS interfaces. 8

e. Values of CAIS-controlled attributes cannot be set using general-purpose CAIS interfaces. 9

The descriptions of interfaces which permit the manipulation of instances of CAIS-controlled definitions identify those definitions. Many of the special-purpose interfaces that create CAIS-controlled nodes or relationships also permit the explicit creation of certain CAIS-controlled attributes. 10

The following restrictions apply to the use of CAIS-controlled attribute and relation definitions to define properties: 11

a. A CAIS-controlled attribute definition may not be used to define a new attribute of a node kind or relation. CAIS-controlled attributes must be inherited from predefined node kinds and relations. 12

b. A predefine, CAIS-controlled relation may not be used to define a new emanating relation of a node kind. Predefine, CAIS-controlled emanating relations must be inherited from predefine node kinds. 13

Modifications to those CAIS-controlled instances which are predefine by the CAIS are further restricted. These CAIS-controlled instances include all of the predefine definitions and the predefine view. The following restrictions apply to all predefine, CAIS-controlled instances when using any CAIS interface (including special-purpose interfaces): 14

a. Predefine, CAIS-controlled instances may not be deleted or converted. 15

b. Predefine, CAIS-controlled relationships may not have their source or target node changed. 16

c. Predefine, CAIS-controlled attribute values may not be set. 17

In addition, the following restriction applies to the modification of those predefine, CAIS-controlled instances which represent definitions (the predefined definitions): 18

The effective constraints imposed on a predefine property may not be modified in the context of a predefine definition. 19

However, constraints on predefine properties which are not CAIS-determined (see Section 5.7.2.2 below) may be strengthened in the context of user-defined specializations of predefine definitions. A consequence of the above restriction is that keys cannot be added to a predefined relation definition which defines a minimal unique key set (see Section 5.7.1.4 page 310) since that would weaken the predefine uniqueness constraint on the keys in that set. 20

Finally, the following restriction applies to all predefine definitions and to all user-defined specializations of these definitions: 21


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

A predefined definition (or any user-defined specialization thereof) may not be made a specialization or generalization of any other predefine definition unless it is already a specialization or generalization, respectively, of that predefine definition.

These restrictions preserve the consistency of the predefine portions of the node model across all CAIS instances. Note that the restrictions do not prohibit the addition of user-defined properties to predefine definitions nor do they prohibit the creation, modification and deletion of instances of such user-defined properties on predefine instances.

#### 5.7.2.2 CAIS-determined entities

Instances of some definitions are manipulated exclusively by the CAIS implementation. That is, the values of instances are determined entirely by the CAIS rather than the user. These definition nodes are identified by the predefine Boolean-valued attribute CAIS_DETERMINED. When CAIS_DETERMINED is TRUE, instances of the definition are created, modified and deleted entirely by the CAIS. Such definitions and their instances are said to be *CAIS-determined*.

The predefine definitions with the following pathnames are CAIS-determined:

Definitions with CAIS_DETERMINED = TRUE

'DEFINITIONS ( PREDEFINED ) 'ATTRIBUTE_DEFINITION ( AVAILABLE )
'DEFINITIONS ( PREDEFINED ) 'ATTRIBUTE_DEFINITION ( CAIS_CONTROLLED )
'DEFINITIONS ( PREDEFINED ) 'ATTRIBUTE_DEFINITION ( CAIS_DETERMINED )
'DEFINITIONS ( PREDEFINED ) 'ATTRIBUTE_DEFINITION ( CAIS_KEY )
'DEFINITIONS ( PREDEFINED ) 'ATTRIBUTE_DEFINITION ( CONTROLLING_PROTOCOL )
'DEFINITION ( PREDEFINED ) 'ATTRIBUTE_DEFINITION ( CURRENT_FILE_SIZE )
'DEFINITIONS ( PREDEFINED ) 'ATTRIBUTE_DEFINITION ( CURRENT_STATUS )
'DEFINITIONS ( PREDEFINED ) 'ATTRIBUTE_DEFINITION ( IMPORTED_DEFINITION_UID )
'DEFINITIONS ( PREDEFINED ) 'ATTRIBUTE_DEFINITION ( IMPORTED_NAME )
'DEFINITIONS ( PREDEFINED ) 'ATTRIBUTE_DEFINITION ( IMPORTED_RELATIONSHIP_KIND )
'DEFINITIONS ( PREDEFINED ) 'ATTRIBUTE_DEFINITION ( IMPORTED_TARGET_UID )
'DEFINITIONS ( PREDEFINED ) 'ATTRIBUTE_DEFINITION ( IMPORTED_UID )
'DEFINITIONS ( PREDEFINED ) 'ATTRIBUTE_DEFINITION ( INTRODUCED_KEY_ATTRIBUTES )
'DEFINITIONS ( PREDEFINED ) 'ATTRIBUTE_DEFINITION ( INTRODUCED_NODE_ATTRIBUTES )
'DEFINITIONS ( PREDEFINED ) 'ATTRIBUTE_DEFINITION ( INTRODUCED_RELATIOHSHIP_ATTRIBUTES )
'DEFINITIONS ( PREDEFINED ) 'ATTRIBUTE_DEFINITION ( IO_UNIT_COUNT )
'DEFINITIONS ( PREDEFINED ) 'ATTRIBUTE_DEFINITION ( MACHINE_TIME )
'DEFINITIONS ( PREDEFINED ) 'ATTRIBUTE_DEFINITION ( NODE_KIND )
'DEFINITIONS ( PREDEFINED ) 'ATTRIBUTE_DEFINITION ( OPEN_NODE_HANDLE_COUNT )
'DEFINITIONS ( PREDEFINED ) 'ATTRIBUTE_DEFINITION ( PROCESS_SIZE )
'DEFINITIONS ( PREDEFINED ) 'ATTRIBUTE_DEFINITION ( STATE )
'DEFINITIONS ( PREDEFINED ) 'ATTRIBUTE_DEFINITION ( TIME_ATTRIBUTE_WRITTEN )
'DEFINITIONS ( PREDEFINED ) 'ATTRIBUTE_DEFINITION ( TIME_CONTENTS_WRITTEN )
'DEFINITIONS ( PREDEFINED ) 'ATTRIBUTE_DEFINITION ( TIME_CREATED )
'DEFINITIONS ( PREDEFINED ) 'ATTRIBUTE_DEFINITION ( TIME_FINISHED )
'DEFINITIONS ( PREDEFINED ) 'ATTRIBUTE_DEFINITION ( TIME_RELATIONSHIP_WRITTEN )
'DEFINTIONS ( PREDEFINED ) 'ATTRIBUTE_DEFINITION ( TIME_STARTED )


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

'DEFINITIONS ( PREDEFINED ) 'ATTRIBUTE_DEFINITION ( UNIQUE_IDENTIFIER )

'DEFINITIONS ( PREDEFINED ) 'RELATION_DEFINITION ( CURRENT_JOB )
'DEFINITIONS ( PREDEFINED ) 'RELATION_DEFINITION ( CURRENT_PROCESSOR )
'DEFINITIONS ( PREDEFINED ) 'RELATION_DEFINITION ( CURRENT_USER )
'DEFINITIONS ( PREDEFINE ) 'RELATION_DEFINITION ( EXECUTABLE_IMAGE )
'DEFINITIONS ( PREDEFINED ) 'RELATION_DEFINITION HANDLER_PROCESS )
'DEFINITIONS ( PREDEFINED ) 'RELATION_DEFINITION ( INTRODUCED_RELATION )

All CAIS-determined definitions are also CAIS-controlled. That is, if CAIS_DETERMINED is TRUE, then CAIS_CONTROLLED is also TRUE. However, when a definition is CAIS-deterrnined, the following additional restrictions apply to the use of the definition: 3

a. Constraints may not be applied to a CAIS-determined definition. 4

b. A default value may not reapplied to a CAIS-determined definition. 5

#### 5.7.2.3 Extensions to the predefined entities

Extensions to the CAIS, such as new IO protocol packages which are incorporated into CAIS implementations, may manipulate instances of CAIS-controlled and CAIS-determined definitions in a manner consistent with the CAIS-defined semantics of those definitions. 1

CAIS implementations may add, and provide the means to add, new CAIS-controlled and CAIS-determined definitions. The restrictions described above should also apply to such definitions. CAIS implementations may also extend, and provide the means to extend, the definitions predefine by the CAIS. For example, an implementation may extend the set of values allowed by the PROTOCOL AKD in order to add anew IO protocol. Such extensions must not further constrain a predefined definition. The means provided by an implementation to add or extend predefined definitions should be restricted to privileged administrators. 2



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

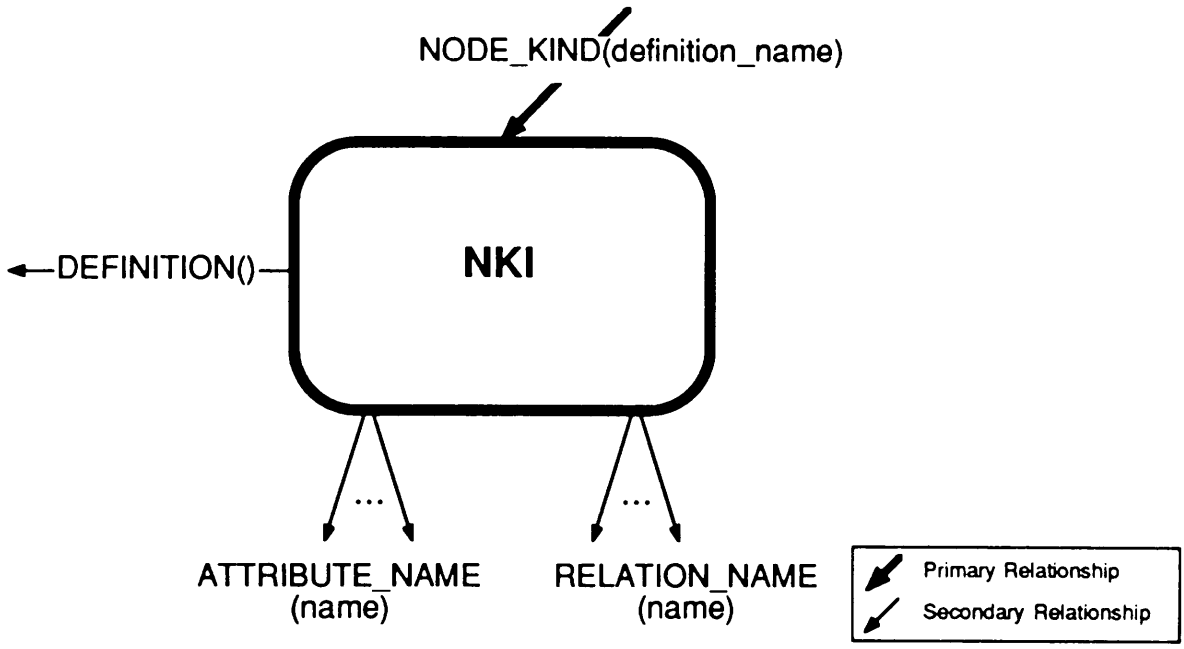


FIGURE 35. Node kind interpretation node (NKI)

### 5.7.3 Views

This package includes interfaces for the maintenance and utilization of NKIs, RIs, and VIEW nodes.

#### 5.7.3.1 Node kind interpretations

Each NKI is an interpretation for a specific NKD. The NKD interpreted by an NKI is the target of the DEFINITION relationship which emanates from that NKI. This NKD is termed the definition node *corresponding to* the NKI (see Figure 35).

An NKI can define local names for some or all of the properties defined by the corresponding NKD (including inherited properties). The local rimes defined by the NKI for specific attribute and emanating relation properties defined by the corresponding NKD are specified by the NAME key values of RELATION_NAME and ATTRIBUTE_NAME relationships emanating from the NKI. A given NKI can define at most one local name for a given property of the corresponding NKD.

An NKI defines a local name for an attribute via a secondary ATTRIBUTE_NAME relationship which emanates from the NKI and terminates at an AD (see Figure 36). The target AD must be an attribute property defined by the corresponding NKD. The NAME key value of this relationship is the local name for the attribute. Subject to the precedence ordering of the adopted views within a process' composite view, such a local name for an attribute may be used to identify that particular attribute in the context of nodes which are instances defined by the corresponding NKD or its specializations.


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

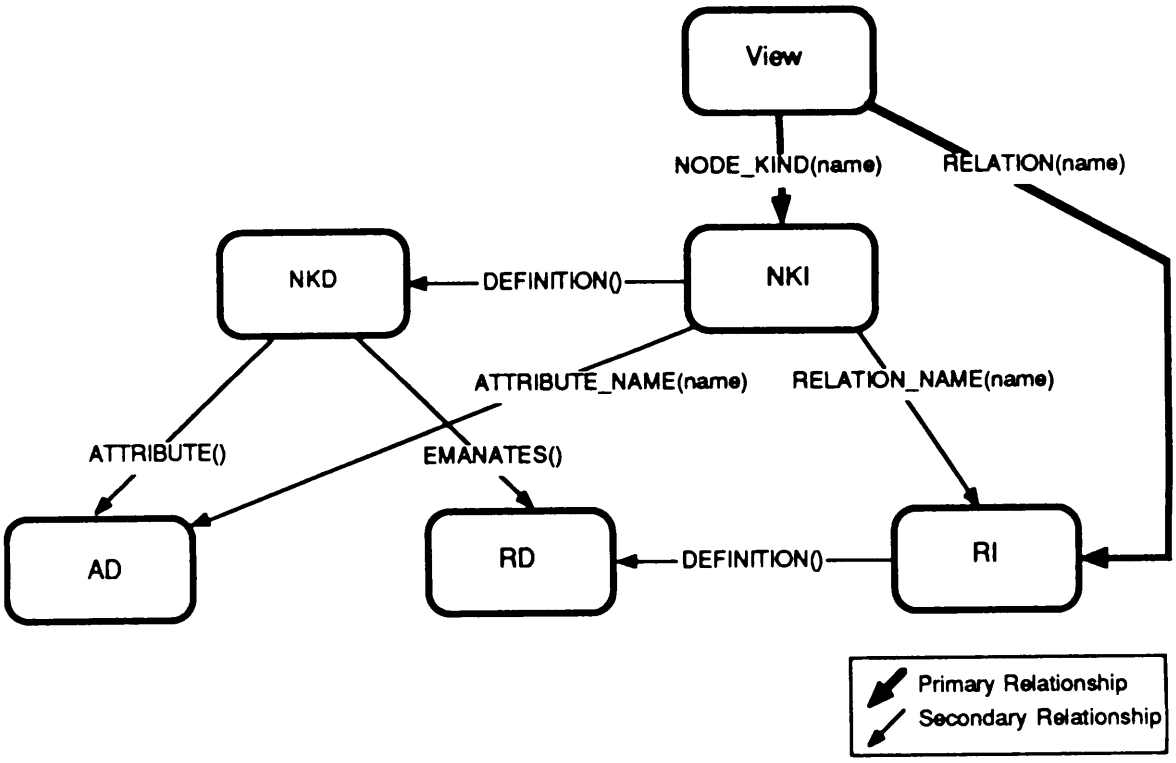


FIGURE 36. Local names defined by an NKI


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

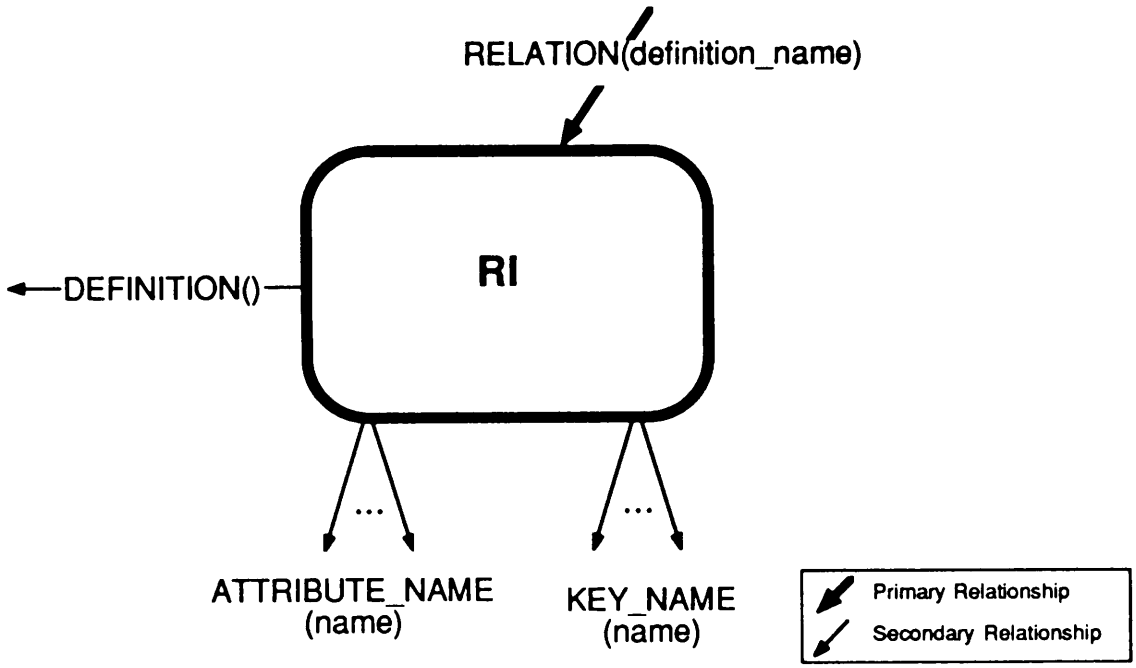


FIGURE 37. Relation interpretation node (RI)

An NKI defines a local name for a relation via a secondary RELATION_NAME relationship which emanates from the NKI and terminates at an RI. The primary relationships to the NKI and the RI related by a RELATION_NAME relationship must emanate from the same VIEW node. The RD corresponding to the RI which is the target of a RELATION_NAME relationship must be an emanating relation property defined by the NKD corresponding to the NKI. The NAME key value of a RELATION_NAME relationship is the local name defined by the NKI for the emanating relation (see Figure 36). Subject to the precedence ordering of the adopted views within a process' composite view, such a local name for a relation may be used in the identification of relationships emanating from nodes which are instances defined by the corresponding NKD or its specializations. In the context of such a node, the local relation name may be used to identify relationships which are instances defined by either the RD corresponding to the RI or its specializations.

### 5.7.3.2 Relation interpretations

Each RI is an interpretation of a specific RD. The RD interpreted by an RI is the target of the secondary DEFINITION relationship which emanates from that RI (see Figure 37). This RD is termed the RD *corresponding to* the RI.

An RI can define local names for some or all of the attributes which are properties of relationships defined by the corresponding RD (including inherited properties). The local names defined by the RI for properties defined by the corresponding RD are specified by the NAME key values of ATTRIBUTE_-



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

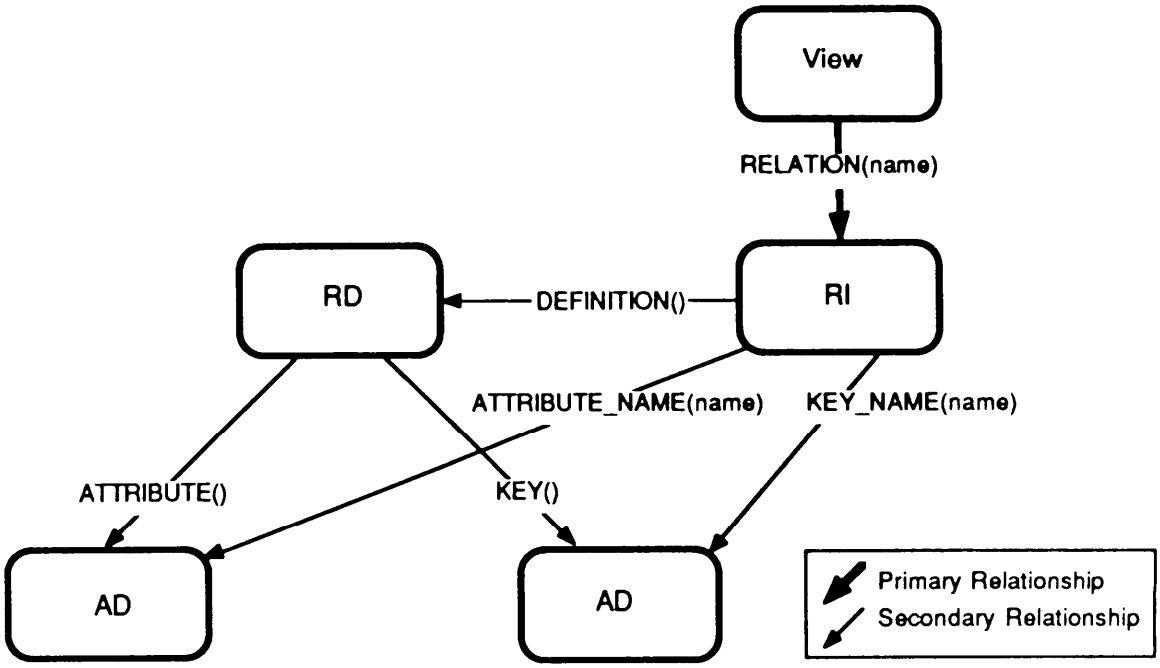


FIGURE 38. Local names defined by an RI

NAME and KEY_NAME relationships emanating from the RI. A given RI can define at most one local name for a given attribute of the corresponding RD.

An RI defines a local name for a relationship attribute which will not be used as a key via a secondary ATTRIBUTE_NAME relationship which emanates from the RI and terminates at an AD (see Figure 38). The target AD must be an attribute property defined by the corresponding RD. The NAME key value of this relationship is the local name defied by the RI for the relationship attribute. Subject to the precedence ordering of the adopted views, a local name for a relationship attribute may be used to identify that attribute in the context of relationships which are instances of the corresponding RD or its specializations. 3

An RI defies a local name for a key via a secondary KEY_NAME relationship which emanates from the RI and terminates at an AD (see Figure 38). The target AD must be a key defined by the corresponding RD. The NAME key value of this relationship is the local name defined by the RI for the key property. Subject to the precedence ordering of the adopted views, a local name for a relationship key may be used to associate a key value with the key in a key value association. Associations of key values with a relationship key using a local key name may be done in the context of relationships which are instances defined by the corresponding RD or its specializations. 4

The KEY_NAME relation is predefined as a specialization of the ATTRIBUTE_NAME relation. These relations have the same key NAME. Since the key values must be distinct, the local names assigned to attributes and keys by an RI must be distinct. A key may always be referenced as an attribute by interfaces which read or iterate over attributes of a relationship. 5


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

#### 5.7.3.3 Creating views

To create a view, a VIEW node is fist created. The view is then developed by adding definitions. Whenever a new definition node is created, it is also imported to some user-specified view. When a new NKD or RD is created, a corresponding NKI or RI is created in a specified view. When a new AD or AKD is created, a corresponding relationship is created from a specified VIEW node to the new definition node. When a new property is added to an existing NKD or RD, a corresponding relationship is created from a corresponding NKI or RI in a specified view to the definition node for the property. In addition, interpretations may be added to a view by importing existing NKDs and RDs. Existing ADs and AKDs may also be imported.

A read relationships access synchronization lock is placed on VIEW nodes while the view or an NKI within the view is adopted. Attempts to open the node for exclusive append or write relationships intent fail with the LOCK_ERROR exception under these circumstances. Operations which modify a view require opening a VIEW node with an exclusive append or write relationships intent. Therefore operations which modify a view cannot be performed while the view or an NKI within the view is adopted.

Operations which modify views are the following:

a. those which make VIEW nodes unobtainable;

b. those which create or write ATTRIBUTE, ATTRIBUTE_KIND, NODE_KIND or RELATION relationships emanating from a VIEW node; and

c. those which create or write ATTRIBUTE_NAME, KEY_NAME or RELATION_NAME relationships emanating from an interpretation node.

#### 5.7.3.4 Adopting views

When a view is adopted by a CAIS process, a secondary ADOPTED_VIEW relationship with key designator LATEST_KEY is created which emanates from the PROCESS node for the adopting process and terminates at the VIEW node of the view being adopted. The view represented by that VIEW node becomes the most recently adopted view within the composite view of the adopting process. If a process adopts a view which it has already adopted, an additional ADOPTED_VIEW relationship is created. The lexicographical ordering of the key values on the ADOPTED_VIEW relationships which emanate from any given PROCESS node specifies the order in which the views were adopted by the process and the precedence of the views within the composite view of the process. The local names defined by the most recently adopted view have a higher precedence than the local names defined by any previously adopted view within the composite view of the process.

Figures 39 and 40 illustrate the ADOPTED_VIEW relationships which emanate from processes which have adopted views. In Figure 39, a process has adopted the predefine view, an implementation-defined view and a user-defined view, in that order. The respective key values on the ADOPTED_-VIEW relationships are A, B and C. The user-defied view is the most recently adopted view and has the highest precedence. The implementation-defined and predefined are respectively lower precedence views. The lexicographical order of the key values A, B and C is consistent with the precedence of the views in the composite view of the process.

Once a view has been adopted by a process, it remains an adopted view in the composite view of the process until the process either explicitly or implicitly cancels all of its adoptions of that view. All



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

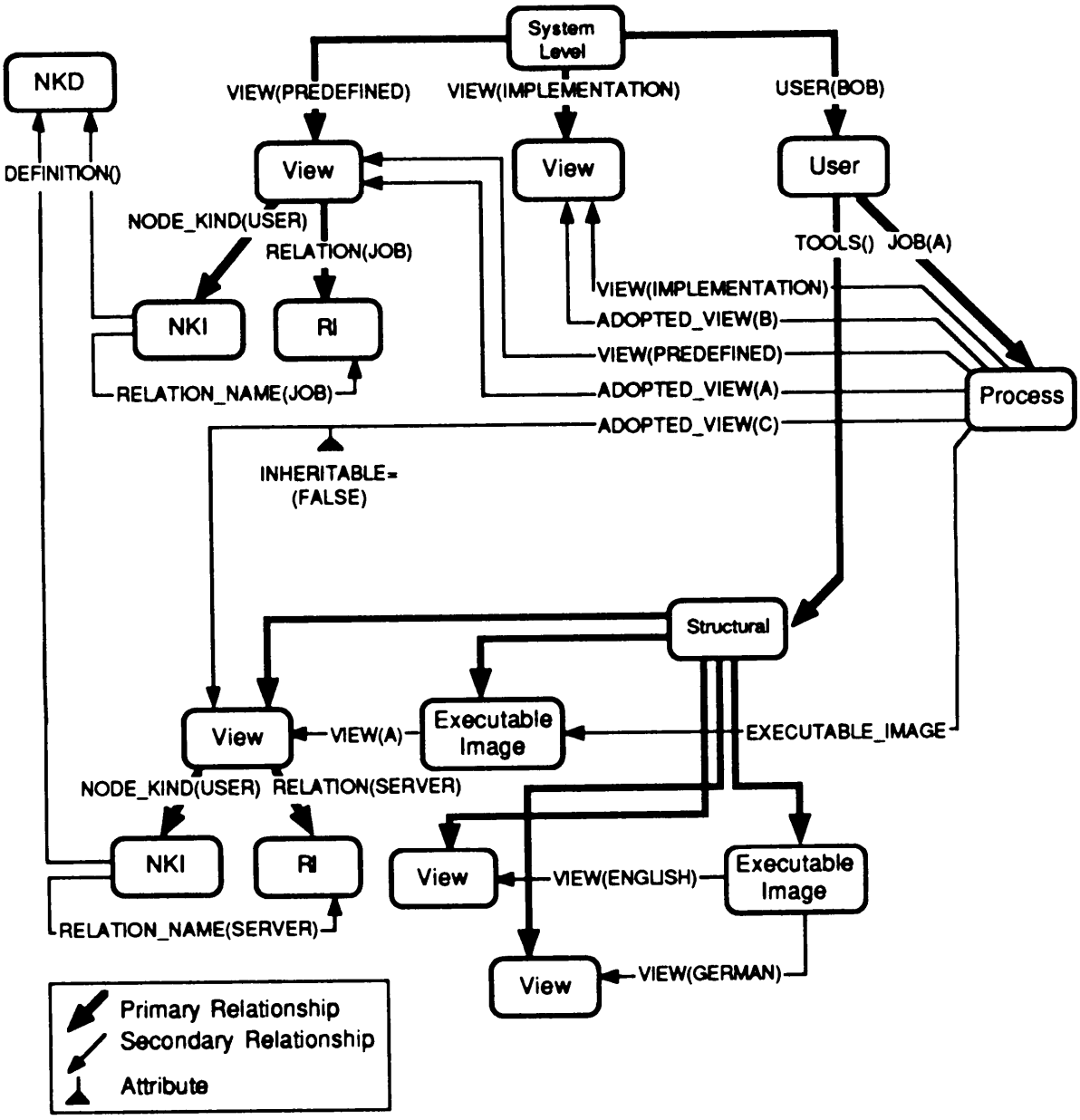


FIGURE 39. Adoption of views by a root process



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

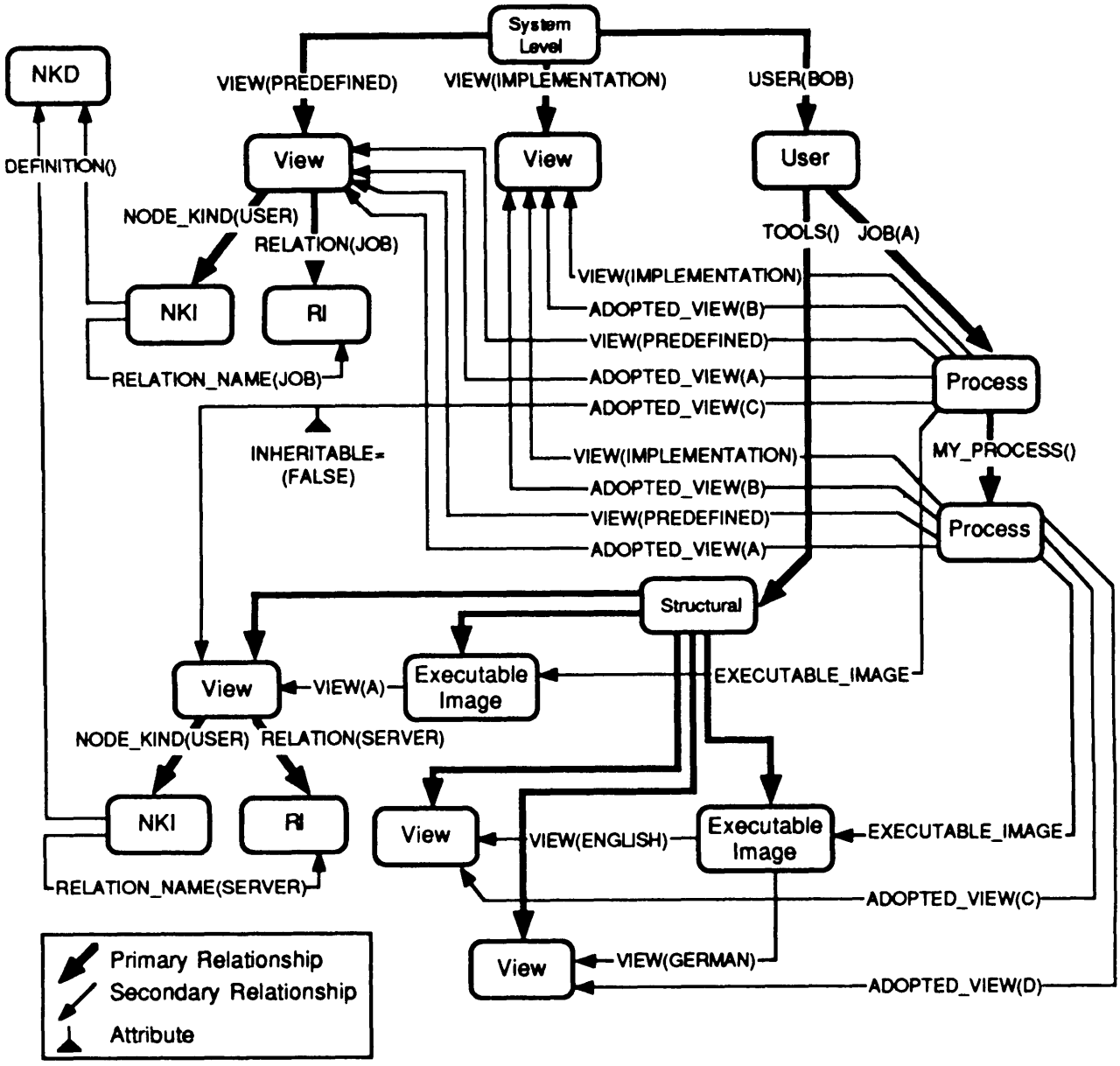


FIGURE 40. Adoption of views by a dependent process


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

view adoptions of a process are implicitly canceled when the process terminates. The cancellation of a view adoption removes the highest precedence (most recent) adoption of that view from the composite view of the process. When a view adoption is explicitly canceled by a process, an ADOPTED_VIEW relationship emanating from the PROCESS node of that process and terminating at the VIEW node for the adopted view is deleted. If there is more than one such relationship, the most recently created relationship is deleted. When a view adoption is implicitly canceled (due to termination of the process), the corresponding ADOPTED_VIEW relationship is not deleted.

The CAIS predefine a top-level VIEW node, which is the target of a primary VIEW relationship 4
emanating from the system-level node with the key value PREDEFINE. The local names defined in this predefine view are used in this standard to identify the predefine node kinds, relations, attributes and attributes kinds. The predefine VIEW node is defined in Appendix A. In addition to the predefine VIEW node, there may also be implementation-defined, top-level VIEW nodes which are the targets of primary VIEW relationships with implementation-defined key values emanating from the system-level node.

When a root process is created, secondary VIEW relationships are created which emanate from the 5
new root PROCESS node and terminate at the predefine VIEW node and an implementation-defined set of top-level VIEW nodes. The key values on these relationships are identical to the key values on the corresponding primary relationships to these nodes which emanate from the system-level node. Dependent processes in the job inherit these relationships (see Figure 39 and Figure 40).

Views can be associated with an EXECUTABLE_IMAGE node. When a CAIS process is created, it 6
implicitly adopts the views identified by the VIEW nodes which are the targets of user-created primary or secondary VIEW relationships emanating from the node that contains the executable image for the process. The key values of these relationships are user-defined. In Figure 39, a VIEW relationship emanates from the EXECUTABLE_IMAGE node for the executable image for the PROCESS node and terminates at a user-defined VIEW node.

When a root process is created, the predefine view is implicitly adopted first (see Figure 39). Next, 7
an implementation-defined set of views represented by top-level VIEW nodes is implicitly adopted. The order in which these implementation-defined views are adopted is implementation-defined. The INHERITABLE attributes of the ADOPTED_VIEW relationships representing these adoptions are set to TRUE. Finally, the views associated with the executable image for the process are implicitly adopted. The order in which these views are adopted is determined by the lexicographical order of the key values of the VIEW relationships emanating from the EXECUTABLE_IMAGE node. The view associated with the lexicographically first key value is adopted first. The INHERITABLE attributes of the ADOPTED_VIEW relationships representing these final implicit adoptions are set to FALSE.

When a dependent process is created, it inherits any inheritable ADOPTED_VIEW relationships 8
which emanate from the PROCESS node of its parent process. In Figure 40, the dependent process (represented by the lower PROCESS node) inherits the view adoptions for the predefine and implementation-defined views. The user-defined view is not inherited. The effect of inheriting these relationships is that the dependent process implicitly adopts the views identified by the VIEW nodes which are the targets of the relationships. These implicit adoptions are performed in the lexicographical order of the key values of the inherited ADOPTED_VIEW relationships. Since the keys of these relationships were created using the key designator LATEST_KEY, the order of adoption is the same as in the parent process and therefore the relative precedence of the inherited views in the composite view is also the same as in the parent process.

Finally, the dependent process implicitly adopts the views that are associated with the executable 9
image for the process. The order in which these views are implicitly adopted is determined by the



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

lexicographical order of the key values on the VIEW relationships emanating from the EXECUTABLE_-IMAGE node. The view associated with the lexicographically first key value is adopted first. The INHERITABLE attributes of the ADOPTED_VIEW relationships representing these last implicit view adoptions are set to FALSE.

The views associated with executable images are adopted last so that names defined by these views will have a higher precedence than names defined by other implicitly adopted views.

#### 5.7.3.5 Adopting interpretations

The CAIS provides interfaces which allow a process to designate the local names it wishes to use when accessing the properties of a node by associating specific NKIs with an open node handle. The NKIs associated with a given open node handle are termed the *adopted NKIs* of the node handle. When a process establishes a given NKI as an adopted interpretation of the node handle, it is said to "adopt" the NKI for the node handle. A process can adopt a given NKI for a node handle if the node identified is an instance of either the NKD corresponding to that NKI or a specialization of that NKD.

When a process adopts an NKI for a given node handle, that NKI becomes the *current adopted interpretation* for the node handle and supersedes any previously adopted NKIs of the given node handle. A process may adopt a given NKI more than once for a given node handle without affecting earlier adoptions of the same NKI for the node handle. For a given node handle, a process may cancel the most recent adoption of the current adopted interpretation, thereby restoring the previous current adopted interpretation (if one exists) for the node handle. When a node handle with adopted NKIs is closed, all of its NKI adoptions are canceled. A view which contains an NKI that is currently adopted cannot be modified until the adoptions are either explicitly cancelled or the handles are closed. (See Section 5.7.3.3 on page 351 for operations which modify views.) All open node handles are closed when a process terminates.

When a node handle with adopted NKIs is passed to a CAIS interface in which attributes of the identified node, emanating relationships of the identified node, or attributes of those relationships are to be identified by name, the precedence ordering of the adopted views within the process' composite view has no influence on the interpretation of the identification. Instead, local names used in such identifications are interpreted according to the local names defied within the particular view containing the current adopted interpretation for the node handle.

When a node handle with adopted NKIs is passed to a CAIS interface which creates an attribute or node iterator, the iterator adopts the NKI. Local names appearing in iteration predicates and name patterns, and returned by GET_NEXT operations, are translated or formed using the view containing the adopted interpretation. Adoption of the NKI is cancelled when the iterator is deleted.

#### 5.7.3.6 Identification using views

The association in a view of a name with a definition node or a property is a naming. Definition namings are defined by relationships emanating from view nodes. Property namings are defined by relationships emanating from interpretation nodes (NKIs and RIs).

A naming is applicable in a given context if it names a definition or property which may occur in that context. Definition names are applicable in contexts requiring the identification of a specific kind of definition node (NKD, RD, AD, or AKD), without reference to an instance. For example, interfaces which create nodes require the identification of the NKD for the node to be created.



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Property namings are applicable in contexts requiring identification of a property (attribute or ems- 3
nating relation) of a particular instance (node or relationship). For example, interfaces which crest e
node attributes require the identification of the AD for the attribute to be created. That identification
is relative to the kind of the given node; the context is the NKD for the node and all generalizations
thereof.

Similarly, a relation naming is applicable in the context of an NKD and a given relation if (1) the 4
naming is defined by an NKI for the NKD or a generalization thereof and (2) the naming names the
given relation or any generalization of that relation.

**5.7.3.6.1 Visibility of namings within a single view** Applicable namings for properties of a 1
node are specified by interpretations for a particular NKD and by interpretations for all generalizations
thereof. This NKD is the NKD corresponding to the adopted interpretation for the handle identifying
the node, if any. If there is no current adopted NKI, this NKD is the one for the node. Applicable
namings for properties of a relationship are specified by interpretations for the relationship's RD and by
interpretations for all generalizations thereof. These definitions are the context in which the namings
are applicable.

There can be more than one applicable naming for a given property or name in the context of a 2
definition. The lattices establish a partial precedence for namings within a view. An applicable naming
NA defined by an interpretation A has a higher precedence than a naming NB defined by interpretation
B if the definition corresponding to A is a specialization of the definition corresponding to B, or if A
and B are the same interpretation and NA names a specialization of the relation named by NB.

A property naming is *visible in a view* for translation (formation), in the context of a definition, if the 3
naming is applicable in that context and there is no higher precedence applicable naming for the same
name (property) defined in that view.

Two different namings for the same name or property can be visible in the context of a given definition, 4
if they are specified by interpretations for generalizations. When two or more namings associating the
same name with different properties are visible in a given context, the view is said to be "ambiguous
for translation of the name". When two or more namings associating the same property with different
names are visible in a given context, the view is said to be "ambiguous for formation of a name" for
that property. For purposes of translation, two names are the same if they are equal by identifier
comparison. For purposes of formation, two names are the same only if they are equal by string
comparison.

**5.7.3.6.2 Visibility of namings within a composite view** For a name to be used in translation 1
or formation, it must be *visible in the composite view*. The namings visible in the composite view for
operations using node handles with a current adopted interpretation are only those which are visible in
the view containing that interpretation. For translation operations without an adopted interpretation,
a naming visible in an adopted view is visible in the composite view if no higher precedence view has
a visible naming for the same name. For formation operations without an adopted interpretation, a
naming visible in an adopted view is visible in the composite view if no higher precedence view has a
visible naming for the same property.

When a name is translated or formed, only namings visible in the composite view of the current process 2
are applied. Such namings must be unambiguous within the defining view. Hence, two problems can
arise during name translation or formation:



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

a. The view containing a visible property naming in the composite view may be ambiguous for translation or formation of the given name. These conditions are referred to as *ambiguous translation* and *ambiguous formation.* Under these conditions, the intended operation fails and AMBIGUOUS_VIEW_ERROR is raised. 3

b. If there is no visible naming in the composite view for name translation or formation, the intended operation fails and NOT_IN_VIEW_ERROR is raised. It is said that the given name *cannot be translated* or *cannot be formed.* 4

Identifications in general, either accepted or returned by the CAIS, must be sufficient to uniquely and unambiguously identify the given entity. This means, for instance, that a name formation operation will fail if the name is ambiguous for either translation or formation, or if it cannot be formed. 5

Properties of a node are identified using attribute and relation namings visible in the context of that node's kind. Properties of a relationship are identified using attribute namings visible in the context of that relationship's relation. 6

When an identification of a relationship consisting of a relation name and a relationship key list is translated in the context of a base node, the relation name is translated using a relation naming which is visible in the context of the base node's NKD. Each key name specified in the relationship key list is translated using key namings visible in the context of the identified RD; these namings may be from any view unless the base node is identified using a node handle with an adopted NKI in which case only visible namings in the view containing that NKI are used. Names for other attributes of the relationship are also translated in the context of the relation. 7

When an identification of a relationship which emanates from a given base node is formed, a relation name is formed using a relation naming which is visible in the context of the base node's NKD and the RD of the relation. Key names returned in the relationship key list and names for other attributes of the relationship are formed using key and attribute namings visible in the context of the RD of the relation. 8

Key lists returned by the CAIS contain values for keys which are visible in the composite view in the context of the relation. Values are returned for all visible keys. If multiple keys are visible, the key values are specified with named association. Otherwise, when zero or one key is visible, an unnamed key designator is returned. The resulting key list (together with base node and relation) must be sufficient to uniquely identify the relationship; otherwise, the intended operation fails and NOT_IN_-VIEW_ERROR is raised. 9

Translation and formation of pathnames are performed sequentially, using each node on the path as a base node. Each path element is translated or formed as for base node, relation name and relationship key list specifications. 10


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

### 5.7.4 Types and subtypes

Package CAIS_DEFINITION_MANAGEMENT provides operations on definition, interpretation and VIEW nodes. The predefine names for these node kinds and their specializations are as follows:

a. ATTRIBUTE_ DEFINITION
b. FLOAT_ATTRIBUTE_KIND_DEFINITION
c. IDENTIFIER_ATTRIBUTE_KIND_DEFINITION
d. INTEGER_ATTRIBUTE_KIND_DEFINITION
e. LIST_ATTRIBUTE_KIND_DEFINITION
f. NAMED_COMPOSITE_LIST_AKD
g. NAMED_ORDERED_COMPOSITE_LIST_AKD
h. ORDERED_COMPOSITE_LIST_AKD
i. STRING_ATTRIBUTE_KIND_DEFINITION
j. UNIFORM_LIST_ATTRIBUTE_KIND_DEFINITION
k. VARIANT_ATTRIBUTE_KIND_DEFINITION
l. NODE_KIND_DEFINITION
m. NODE_KIND_INTERPRETATION
n. RELATION_DEFINITION
o. RELATION_INTERPRETATION
p. VIEW

The following types are defined in package CAIS_DEFINITION_MANAGEMENT. Some are used in the interfaces, while others are provided for use in accessing definition node attributes:

```
type RELATIONSHIP_KIND is (PRIMARY, SECONDARY);
```

The literals of the type RELATIONSHIP_KIND are possible values for the ALLOWED_USAGE attribute on RDs which comprise bidirectional relation definitions. This attribute can specify that all relationships of a given relation and its specializations must be exclusively primary or exclusively secondary relationships. If an RD comprising part of a bidirectional relation definition does not have this attribute then the relationships of that relation are not constrained to either usage.

```
type VALUE_TYPE is ( INTEGER,
                     FLOAT,
                     STRING,
                     IDENTIFIER,
                     VARIANT,
                     LIST,
                     UNIFORM_LIST,
                     COMPOSITE_LIST);
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

The elements of the enumeration type VALUE_TYPE name the kinds of attribute values and so enumerate the structures which lists containing attribute values can have.

### 5.7.5 Opening the predefine VIEW NKD

```
function VIEW_NKD
    (STATUS : in      STATUS_INDICATOR := NO_INDICATION)
    return NKD_TYPE;
```

Purpose:

This function returns an open node handle with NO_ACCESS intent to the predefine NKD for VIEW nodes.

The node kind name VIEW and the relation NODE_KIND_DEFINITION are associated with NODE as the key and relation used in opening the handle. See subprograms PATH_KEY (Section 5.2.20 on page 119) and PATH_RELATION (Section 5.2.21 on page 120).

Parameters:

**STATUS** is an established status indicator.

Exceptions:

**STATUS_ERROR** is raised if STATUS is not an established status indicator [3].

### 5.7.6 Opening the predefine NKI NKD

```
function NODE_KIND_INTERPRETATION_NKD
    (STATUS : in      STATUS_INDICATOR := NO_INDICATION)
    return NKD_TYPE;
```

Purpose:

This function returns an open node handle with NO_ACCESS intent to the predefined NKD for NKIs.

The node kind name NODE_KIND_INTERPRETATION and the relation NODE_KIND_DEFINITION are associated with NODE as the key and relation used in opening the handle. See subprograms PATH_KEY (Section 5.2.20 on page 119) and PATH_RELATION (Section 5.2.21 on page 120).

Parameters:

**STATUS** is an established status indicator.



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Exceptions:

**STATUS_ERROR** is raised if STATUS is not an established status indicator [3]. 4

### 5.7.7 Opening the predefine NKD NKD

```
function NODE_KIND_DEFINITION_NKD
    (STATUS : in      STATUS_INDICATOR := NO_INDICATION)
    return NKD_TYPE;
```

Purpose:

This function returns an open node handle with NO_ACCESS intent to the predefined NKD for NKDs. 1

The node kind name NODE_KIND_DEFINITION and the relation NODE_KIND_DEFINITION are associated with NODE as the key and relation used in opening the handle. See subprograms PATH_KEY (Section 5.2.20 on page 119) and PATH_RELATION (Section 5.2.21 on page 120). 2

Parameters:

**STATUS** is an established status indicator. 3

Exceptions:

**STATUS_ERROR** is raised if STATUS is not an established status indicator [3]. 4

### 5.7.8 Opening the predefine RI NKD

```
function RELATION_INTERPRETATION_NKD
    (STATUS : in      STATUS_INDICATOR : = NO_INDICATION)
    return NKD_TYPE;
```

Purpose:

This function returns an open node handle with NO. ACCESS intent to the predefine NKD for RIs. 1

The node kind name RELATION_INTERPRETATION and the relation NODE_KIND_DEFINITION are associated with NODE as the key and relation used in opening the handle. See subprograms PATH_KEY (Section 5.2.20 on page 119) and PATH_RELATION (Section 5.2.21 on page 120). 2

Parameters:

**STATUS** is an established status indicator. 3


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Exceptions:

**STATUS_ERROR** is raised if STATUS is not an established status indicator [3]. 4

### 5.7.9 Opening the predefine RD NKD

```
function RELATION_DEFINITION_NKD
    (STATUS : in STATUS_INDICATOR := NO_INDICATION)
    return NKD_TYPE;
```

Purpose:

This function returns an open node handle with NO_ACCESS intent to the predefined NKD for RDs. 1

The node kind name RELATION_DEFINITION and the relation NODE_KIND_DEFINITION are associated with NODE as the key and relation used in opening the handle. See subprograms PATH_KEY (Section 5.2.20 on page 119) and PATH_RELATION (Section 5.2.21 on page 120). 2

Parameters:

**STATUS** is an established status indicator. 3

Exceptions:

**STATUS_ERROR** is raised if STATUS is not an established status indicator [3]. 4

### 5.7.10 Opening the predefine AD NKD

```
function ATTRIBUTE_DEFINITION_NKD
    (STATUS : in STATUS_INDICATOR : = NO_INDICATION)
    return NKD_TYPE;
```

Purpose:

This function returns an open node handle with NO_ACCESS intent to the predefine NKD for ADs. 1

The node kind name ATTRIBUTE_DEFINITION and the relation NODE_KIND_DEFINITION are associated with NODE as the key and relation used in opening the handle. See subprograms PATH_KEY (Section 5.2.20 on page 119) and PATH_RELATION (Section 5.2.21 on page 120). 2

Parameters:

**STATUS** is an established status indicator. 3



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Exceptions:

**STATUS_ERROR** is raised if STATUS is not an established status indicator [3]. 4

### 5.7.11 Opening the predefine named composite list AKD NKD

```
function NAMED_COMPOSITE_LIST_AKD_NKD
    (STATUS : in STATUS_INDICATOR := NO_INDICATION)
    return NKD_TYPE;
```

Purpose:

This function returns an open node handle with NO_ACCESS intent to the predefined NKD for named composite list AKDs. 1

The node kind name NAMED_COMPOSITE_LIST_AKD and the relation NODE_KIND_DEFINITION are associated with NODE as the key and relation used in opening the handle. See subprograms PATH_KEY (Section 5.2.20 on page 119) and PATH_RELATION (Section 5.2.21 on page 120). 2

Parameters:

**STATUS** is an established status indicator. 3

Exceptions:

**STATUS_ERROR** is raised if STATUS is not an established status indicator [3]. 4

### 5.7.12 Opening the predefine identifier AKD NKD

```
function IDENTIFIER_ATTRIBUTE_KIND_DEFINITION_NKD
    (STATUS : in STATUS_INDICATOR : = NO_INDICATION)
    return NKD_TYPE;
```

Purpose:

This function returns an open node handle with NO_ACCESS intent to the predefine NKD for identifier AKDs. 1

The node kind name IDENTIFIER_ATTRIBUTE_KIND_DEFINITION and the relation NODE_KIND_DEFINITION are associated with NODE as the key and relation used in opening the handle. See subprograms PATH_KEY (Section 5.2.20 on page 119) and PATH_RELATION (Section 5.2.21 on page 120). 2


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Parameters:

**STATUS** is an established status indicator. 3

Exceptions:

**STATUS_ERROR** is raised if STATUS is not an established status indicator [3]. 4

### 5.7.13 Creating VIEW nodes

```
procedure CREATE_VIEW
    (VIEW                 : in out VIEW_NODE_TYPE;
     BASE                 :  in    NODE_TYPE;
     KEY                  :  in    RELATIONSHIP_KEY           := EMPTY_KEY_LIST;
     ATTRIBUTES           :  in    ATTRIBUTE_LIST             := EMPTY_LIST;
     DISCRETIONARY_ACCESS :  in    DISCRETIONARY_ACCESS_LIST  := ALL_RIGHTS;
     MANDATORY_ACCESS     :  in    MANDATORY_ACCESS_LIST      := EMPTY_LIST;
     KIND                 :  in    NODE_KIND_NODE_TYPE        := VIEW_NKD;
     STATUS               :  in    STATUS_INDICATOR           := NO_INDICATION;
     MEDIUM_RESOURCE      :  in    MEDIUM_NODE_TYPE           := DEFAULT_MEDIUM);
```

Purpose:

This procedure creates an empty view. A VIEW node and the primary VIEW relationship to it are created, as well as the corresponding reverse relationship to the node identified by BASE. The primary relationship key list is given by KEY. An open node handle to the newly created node with EXCLUSIVE_WRITE intent is returned in VIEW. 1

KIND is an open node handle identifying the node kind for the new node. It must identify a VIEW node kind. It may specify either an NKD or an NKI. If KIND identifies an NKI, the NKI is made the current adopted interpretation for the node handle VIEW. 2

The MANDATORY_ACCESS parameter specifies the object classification labels with which the node is to be created. If its value is the empty list, the node inherits the subject classification of the creating process as its object classification. (See Table IX on page 256.) 3

The MEDIUM_RESOURCE parameter specifies the medium resource on which the newly created node will reside. 4

Parameters:

**VIEW** is a closed node handle to be opened to the newly created node. 5

**BASE** is an open node handle to the node from which the primary relationship is to emanate. 6

**KEY** is the relationship key list for the primary relationship to be created. 7

**ATTRIBUTES** is an empty or named list whose elements are used to establish initial values for attributes of the newly created node; each named item of the list specifies an attribute name and the value to be given to that attribute. 8



Source: https://assist.dla.mil -- Downloaded: 202610-26T07:54Z
Check the source to verify that this is the current version before use.

**DISCRETIONARY_ACCESS** is the initial access control information associated with the created node; it is the value of the GRANT attribute of the ACCESS relationship to the user's default GROUP node.

**MANDATORY_ACCESS** is a list defining the classification label for the created node.

**KIND** is an open node handle identifying the node kind for the new node.

**STATUS** is an established status indicator.

**MEDIUM_RESOURCE** is an open node handle identifying a MEDIUM node.

Exceptions:

**PATHNAME_SYNTAX_ERROR** is raised if the node identification given by KEY is syntactically illegal [2].

**NAME_ERROR** is raised if the parent of the NKI identified by KIND is inaccessible [1].

**EXISTING_NODE_ERROR** is raised if a node already exists with the identification given [1].

**NODE_KIND_ERROR** is raised if the open node handle KIND does not identify a VIEW node kind [2] or if the open node handle MEDIUM_RESOURCE does not identify a MEDIUM node [1].

**RELATIONSHIP_ERROR** is raised if KEY specifies a value for the same key more than once [5].

**ATTRIBUTE_ERROR** is raised if the attribute list specifies a value for the same attribute more than once [6].

**NOT_IN_VIEW_ERROR** is raised if an attribute name in KEY or ATTRIBUTES cannot be translated [1] or if KEY consists of a single relationship key designator and no key of the corresponding relation is visible [5].

**AMBIGUOUS_VIEW_ERROR** is raised if an attribute name in KEY or ATTRIBUTES has an ambiguous translation [1] or if KEY consists of a single relationship key designator and more than one key of the corresponding relation is visible [4].

**SYNTAX_ERROR** is raised if the ATTRIBUTES parameter, the DISCRETIONARY_ACCESS parameter or the MANDATORY_ACCESS parameter is syntactically illegal [8-10].

**PREDEFINED_ATTRIBUTE_ERROR** is raised if any attribute named in the ATTRIBUTES parameter is a CAIS-controlled attribute [1].

**USE_ERROR** is raised if the value for the MANDATORY_ACCESS parameter is semantically illegal [10].

**STATUS_ERROR** is raised if BASE, KIND or MEDIUM_RESOURCE is not an open node handle [2], if VIEW is an open node handle prior to the call [1] or if STATUS is not an established status indicator [3].

**CONFORMANCE_ERROR** is raised if the operation would result in an attribute, node or relationship not conforming to its definition or the constraints in force on that definition [3,7,8,9,10].

**TRANSACTION_ERROR** is raised if the operation is included in more than one transaction [1] or if the operation is included in a transaction and the operation modifies a node which is not included in the same transaction [2].


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

**INTENT_VIOLATION** is raised if BASE was not opened with an intent establishing the right to create relationships [1]. 28

**LOCK_ERROR** is raised if a node handle to the parent of the NKI specified by KIND cannot be opened with intent READ_RELATIONSHIPS due to outstanding locks in conflict with that intent [1]. 29

**ACCESS_VIOLATION** is raised if the discretionary access control rights of the current process are insufficient to allow access with CREATE_INSTANCES intents to the NKD identified through KIND or with APPEND_CONTENTS intents to the MEDIUM node identified by MEDIUM_RESOURCE [1]. 30

**RESOURCE_ALLOCATION_ERROR** is raised if the medium resource is not allocatable [1]. 31

**RESOURCE_AVAILABILITY_ERROR** is raised if the medium resource is not available [1]. 32

**SECURITY_VIOLATION** is raised if the operation represents a violation of mandatory access controls. SECURITY_VIOLATION is raised only if the conditions for other exceptions are not present. 33

Additional Interfaces:

```
procedure CREATE_VIEW
    (VIEW                 : in out VIEW_NODE_TYPE;
     BASE                 :  in    NODE_TYPE;
     KEY                  :  in    RELATIONSHIP_KEY           : = EMPTY_KEY_LIST;
     ATTRIBUTES           :  in    ATTRIBUTE_LIST             := EMPTY_LIST;
     DISCRETIONARY_ACCESS : in    DISCRETIONARY_ACCESS_LIST  := ALL_RIGHTS;
     MANDATORY_ACCESS     :  in    MANDATORY_ACCESS_LIST      : = EMPTY_LIST;
     KIND                 :  in    NODE_KIND_NAME;
     STATUS               :  in    STATUS_INDICATOR           : = NO_INDICATION;
     MEDIUM_RESOURCE      :  in    MEDIUM_NODE_TYPE           : = DEFAULT_MEDIUM)
is
    NODE_KIND : NKI_TYPE; -- "VIEW" or a specialization.
begin
    OPEN_NODE_KIND (NODE_KIND, KIND, STATUS);
    CREATE_VIEW
        (VIEW,
         BASE,
         KEY,
         ATTRIBUTES,
         DISCRETIONARY_ACCESS,
         MANDATORY_ACCESS,
         NODE_KIND,
         STATUS,
         MEDIUM_RESOURCE);
    CLOSE (NODE_KIND);
exception
    when others =>
        CLOSE (NODE_KIND);
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
        raise;
    end CREATE_VIEW;
```

### 5.7.14 Deleting the primary relationships of a tree rooted at a VIEW node

```
procedure DELETE_VIEW
    (NODE   : in out VIEW_NODE_TYPE;
     STATUS : in     STATUS_INDICATOR := NO_INDICATION);
```

Purpose:

This procedure deletes a view. Effectively, the DELETE_TREE operation is performed for the VIEW node specified by the open node handle NODE. The primary relationship to the VIEW node and all nodes reachable by a unique primary path from the VIEW node are deleted in an unspecified order. The nodes reachable from the VIEW node by unique primary paths are opened with intent EXCLUSIVE_WRITE. If the DELETE_VIEW operation raises an exception, none of the primary relationships is deleted. 1

Parameters:

**NODE** is an open node handle to the VIEW node at the root of the tree whose primary relationships are to be deleted. 2

**STATUS** is an established status indicator. 3

Exceptions:

**NAME_ERROR** is raised if the parent of the node identified by NODE or any of the target nodes of primary relationships to be deleted are inaccessible [1]. 4

**STATUS_ERROR** is raised if the node handle NODE is not open prior to the call [2] or if STATUS is not an established status indicator [3]. 5

**NODE_KIND_ERROR** is raised if NODE does not identify a VIEW node [1] or if any node to be deleted is CAIS-controlled and is not a VIEW or interpretation node [5]. 6

**USE_ERROR** is raised if a node to be deleted is predefine [62]. 7

**CONFORMANCE_ERROR** is raised if the operation would result in an attribute, node or relationship not conforming to its definition or the constraints in force on that definition [3]. 8

**LOCK_ERROR** is raised if a node handle to the parent of the node specified by NODE cannot be opened with intent WRITE_RELATIONSHIPS or if a node handle identifying any node whose unique primary path traverses the node identified by NODE cannot be opened with intent EXCLUSIVE_WRITE due to outstanding locks [1]. 9

**TRANSACTION_ERROR** is raised if the operation is included in more than one transaction [1] or if the operation is included in a transaction and the operation modifies a node which is not included in the same transaction [2]. 10

**INTENT_VIOLATION** is raised if the node handle NODE was not opened with an intent including EXCLUSIVE_WRITE and READ_RELATIONSHIPS [1]. 11


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

**ACCESS_VIOLATION** is raised if the current process does not have sufficient discretionary access control rights to obtain access to the parent of the node specified by NODE with intent WRITE_RELATIONSHIPS or to obtain access to any target node of a primary relationship to be deleted with intent EXCLUSIVE_WRITE [1]. 12

**SECURITY_VIOLATION** is raised if the operation represents a violation of mandatory access controls. SECURITY_VIOLATION is raised only if the conditions for other exceptions are not present. 13

### 5.7.15 Creating NKDs

```
procedure CREATE_NODE_KIND
    (NKI                          : in out NKI_TYPE;
     NKD                          : in out NKD_TYPE;
     VIEW                         : in     VIEW_NODE_TYPE;
     SPECIALIZATION_OF            : in     NKD_TYPE;
     BASE                         : in     NODE_TYPE;
     LOCAL_NAME                   : in     RELATIONSHIP_KEY;
     KEY                          : in     RELATIONSHIP_KEY          := EMPTY_KEY_LIST;
     SPECIALIZATION_OF_KEY        : in     RELATIONSHIP_KEY          := EMPTY_KEY_LIST;
     GENERALIZATION_OF_KEY        : in     RELATIONSHIP_KEY          := EMPTY_KEY_LIST;
     INTERPRETATION_ATTRIBUTES : in        ATTRIBUTE_LIST            := EMPTY_LIST;
     DEFINITION_ATTRIBUTES        : in     ATTRIBUTE_LIST            := EMPTY_LIST;
     DISCRETIONARY_ACCESS         : in     DISCRETIONARY_ACCESS_LIST := ALL_RIGHTS;
     MANDATORY_ACCESS             : in     MANDATORY_ACCESS_LIST := EMPTY_LIST;
     INTERPRETATION_KIND          : in     NODE_KIND_NODE_TYPE :=
                                               NODE_KIND_INTERPRETATION_NKD;
     DEFINITION_KIND              : in     NODE_KIND_NODE_TYPE :=
                                               NODE_KIND_DEFINITION_NKD;
     STATUS                       : in     STATUS_INDICATOR          := NO_INDICATION;
     INTERPRETATION_MEDIUM        : in     MEDIUM_NODE_TYPE          := DEFAULT_MEDIUM;
     DEFINITION_MEDIUM            : in     MEDIUM_NODE_TYPE          := DEFAULT_MEDIUM);
```

Purpose:

This procedure creates a new node kind. An NKD and the primary NODE_KIND_DEFINITION relationship to it are created, as well as the corresponding reverse relationship to the node identified by BASE. The primary relationship key list is given by KEY. An open node handle to the newly created NKD with WRITE intent is returned in NKD. 1

A SPECIALIZATION_OF relationship is created from the new NKD to the NKD identified by SPECIALIZATION_OF. SPECIALIZATION_OF_KEY and GENERALIZATION_OF_KEY provide the relationship key lists for this relationship and the corresponding reverse relationship. 2

This procedure also creates an NKI and installs the primary NODE_KIND relationship to it, as well as the corresponding reverse relationship to the VIEW node identified by VIEW. The primary relationship key list is given by LOCAL_NAME. An open node handle to the newly created NKI with WRITE intent is returned in NKI. 3

The INTERPRETATION_ATTRIBUTES and DEFINITION_ATTRIBUTES parameters pro- 4


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

vide initial values for attributes of the new NKI and NKD, respectively. They cannot specify values for any CAIS-controlled attributes.

The MANDATORY_ACCESS parameter specifies the object classification labels with which the nodes are to be created. If its value is the empty list, the nodes inherit the subject classification of the creating process as their object classification. (See Table IX on page 256.)

INTERPRETATION_KIND and DEFINITION_KIND are open node handles identifying the node kinds for the new nodes. They must identify an NKI and NKD kind, respectively. Each may specify a kind either through an NKD or an NKI. If INTERPRETATION_KIND identifies an NKI, the NKI is made the current adopted interpretation for the node handle NKI. If DEFINITION_-KIND identifies an NKI, the NKI is made the current adopted interpretation for the node handle NKD.

The INTERPRETATION_MEDIUM parameter specifies the medium resource on which the new NKI will reside. The DEFINITION_MEDIUM parameter specifies the medium resource on which the new NKD will reside.

Parameters:

**NKI** is a closed node handle to be opened to the new NKI.

**NKD** is a closed node handle to be opened to the new NKD.

**VIEW** is an open node handle to the VIEW node from which the primary relationship to the NKI is to emanate.

**SPECIALIZATION_OF** is an open node handle identifying the NKD of which the new definition is to be a specialization.

**BASE** is an open node handle to the node from which the primary relationship to the new NKD is to emanate.

**LOCAL_NAME** is the key list of the primary NODE_KIND relationship to be created to the NKI.

**KEY** is the key list of the primary NODE_KIND_DEFINITION relationship to be created to the NKD.

**SPECIALIZATION_OF_KEY** is the key list of the SPECIALIZATION_OF relationship to be created from the NKD.

**GENERALIZATION_OF_KEY** is the key list of the GENERALIZATION_OF relationship to be created to the NKD.

**INTERPRETATION_ATTRIBUTES** is an empty or named list whose elements are used to establish initial values for attributes of the new NKI; each named item of the list specifies an attribute name and the value to be given to that attribute.

**DEFINITION_ATTRIBUTES** is an empty or named list whose elements are used to establish initial values for attributes of the new NKD; each named item of the list specifies an attribute name and the value to be given to that attribute.

**DISCRETIONARY_ACCESS** is the initial access control information associated with the created nodes; it is the value of the GRANT attributes of the ACCESS relationships to the user's default GROUP node.

**MANDATORY_ACCESS** is a list defining the classification labels for the created nodes.


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

**INTERPRETATION_KIND** is an open node handle identifying the node kind for the NKI to be created.

**DEFINITION_KIND** is an open node handle identifying the node kind for the NKD to be created.

**STATUS** is an established status indicator.

**INTERPRETATION_MEDIUM** is an open node handle identifying a MEDIUM node.

**DEFINITION_MEDIUM** is an open node handle identifying a MEDIUM node.

Exceptions:

**PATHNAME_SYNTAX_ERROR** is raised if the node identification given by LOCAL_NAME, KEY, SPECIALIZATION_OF_KEY or GENERALIZATION_OF_KEY is syntactically illegal [2].

**NAME_ERROR** is raised if the parent of the NKI identified by INTERPRETATION_KIND or DEFINITION_KIND is inaccessible [1].

**EXISTING_NODE_ERROR** is raised if a primary relationship already exists with the identification given [1].

**RELATIONSHIP_ERROR** is raised if a secondary relationship already exists with the identification given [3] or if a key list specifies a value for the same key more than once [5].

**ATTRIBUTE_ERROR** is raised if an attribute list specifies a value for the same attribute more than once [6].

**NODE_KIND_ERROR** is raised if the open node handle INTERPRETATION_KIND does not identify an NKI kind [2], if the open node handle DEFINITION_KIND does not identify an NKD kind [2], if the open handle VIEW does not identify a VIEW node [1], if the open node handle SPECIALIZATION_OF does not identify an NKD [1] or if the open node handle INTERPRETATION_MEDIUM or DEFINITION_MEDIUM does not identify a MEDIUM node [1].

**NOT_IN_VIEW_ERROR** is raised if an attribute name in LOCAL_NAME, KEY, SPECIALIZATION_OF_KEY, GENERALIZATION_OF_KEY, INTERPRETATION_ATTRIBUTES or DEFINITION_ATTRIBUTES cannot be translated [1] or if LOCAL_NAME, KEY, SPECIALIZATION_OF_KEY or GENERALIZATION_OF_KEY consists of a single relationship key designator and no key of the corresponding relation is visible [5].

**AMBIGUOUS_VIEW_ERROR** is raised if an attribute name in LOCAL_NAME, KEY, SPECIALIZATION_OF_KEY, GENERALIZATION_OF_KEY, INTERPRETATION_ATTRIBUTES or DEFINITION_ATTRIBUTES has an ambiguous translation [1] or if LOCAL_NAME, KEY, SPECIALIZATION_OF_KEY or GENERALIZATION_OF_KEY consists of a single relationship key designator and more than one key of the corresponding relation is visible [4].

**SYNTAX_ERROR** is raised if the INTERPRETATION_ATTRIBUTES, DEFINITION_ATTRIBUTES, DISCRETIONARY_ACCESS or MANDATORY_ACCESS parameter is syntactically illegal [8-10].

**PREDEFINED_ATTRIBUTE_ERROR** is raised if any attribute named in the INTERPRETATION_ATTRIBUTES or DEFINITION_ATTRIBUTES parameters is a CAIS-controlled attribute [1].


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

**USE_ERROR** is raised if the value for MANDATORY_ACCESS is semantically illegal [10]. 36

**STATUS_ERROR** is raised if VIEW, SPECIALIZATION_OF, BASE, INTERPRETATION_KIND, DEFINITION_KIND, INTERPRETATION_MEDIUM or DEFINITION_MEDIUM is not an open node handle [2], if NKI or NKD is an open node handle prior to the call [1] or if STATUS is not an established status indicator [3]. 37

**CONFORMANCE_ERROR** is raised if the operation would result in an attribute, node or relationship not conforming to its definition or the constraints in force on that definition [1,3,7,8,9,10]. 38

**LOCK_ERROR** is raised if a node handle to the parent of the NKI specified by INTERPRETATION_KIND or DEFINITION_KIND cannot be opened with intent READ_RELATIONSHIPS due to outstanding locks in conflict with that intent [1]. 39

**TRANSACTION_ERROR** is raised if the operation is included in more than one transaction [1] or if the operation is included in a transaction and the operation modifies a node which is not included in the same transaction [2]. 40

**INTENT_VIOLATION** is raised if SPECIALIZATION_OF and BASE were not opened with an intent establishing the right to create relationships [1] or if VIEW was not opened with an exclusive intent establishing the right to create relationships [1]. 41

**ACCESS_VIOLATION** is raised if the discretionary access control rights of the current process are insufficient to allow access with CREATE_INSTANCES intents to the NKDs identified by INTERPRETATION_KIND or DEFINITION_KIND or with READ_RELATIONSHIPS intents to the parent of an NKI specified by INTERPRETATION_KIND or DEFINITION_KIND or with APPEND_CONTENTS intents to the MEDIUM nodes identified by INTERPRETATION_MEDIUM or DEFINITION_MEDIUM [1]. 42

**RESOURCE_ALLOCATION_ERROR** is raised if either medium resource is not allocatable [1]. 43

**RESOURCE_AVAILABILITY_ERROR** is raised if either medium resource is not available [1]. 44

**SECURITY_VIOLATION** is raised if the operation represents a violation of mandatory access controls. SECURITY_VIOLATION is raised only if the conditions for other exceptions are not present. 45

Additional Interfaces:

```
procedure CREATE_NODE_KIND
    (NKI                          : in out NKI_TYPE;
     NKD                          : in out NKD_TYPE;
     VIEW                         : in     VIEW_NODE_TYPE;
     SPECIALIZATION_OF            : in     NKD_TYPE;
     BASE                         : in     NODE_TYPE;
     LOCAL_NAME                   : in     RELATIONSHIP_KEY;
     KEY                          : in     RELATIONSHIP_KEY      := EMPTY_KEY_LIST;
     SPECIALIZATION_OF_KEY        : in     RELATIONSHIP_KEY      := EMPTY_KEY_LIST;
     GENERALIZATION_OF_KEY        : in     RELATIONSHIP_KEY      := EMPTY_KEY_LIST;
     INTERPRETATION_ATTRIBUTES : in     ATTRIBUTE_LIST        := EMPTY_LIST;
     DEFINITION_ATTRIBUTES        : in     ATTRIBUTE_LIST        := EMPTY_LIST;
```


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
        DISCRETIONARY_ACCESS      : in  DISCRETIONARY_ACCESS_LIST := ALL_RIGHTS;
        MANDATORY_ACCESS          : in  MANDATORY_ACCESS_LIST := EMPTY_LIST;
        INTERPRETATION_KIND       : in  NODE_KIND_NAME;
        DEFINITION_KIND           : in  NODE_KIND_NAME :=
                                          "NODE_KIND_DEFINITION";
        STATUS                    : in  STATUS_INDICATOR := NO_INDICATION;
        INTERPRETATION_MEDIUM     : in  MEDIUM_NODE_TYPE := DEFAULT_MEDIUM;
        DEFINITION_MEDIUM         : in  MEDIUM_NODE_TYPE := DEFAULT_MEDIUM)
    is
        INTERPRETATION_NODE_KIND : NKI_TYPE;
        -- "NODE_KIND_INTERPRETATION" or a specialization.
        DEFINITION_NODE_KIND      : NKI_TYPE;

    begin
        OPEN_NODE_KIND (INTERPRETATION_NODE_KIND, INTERPRETATION_KIND, STATUS);
        OPEN_NODE_KIND (DEFINITION_NODE_KIND, DEFINITION_KIND, STATUS);
        CREATE_NODE_KIND
            (NKI,
             NKD,
             VIEW,
             SPECIALIZATION_OF,
             BASE,
             LOCAL_NAME,
             KEY,
             SPECIALIZATION_OF_KEY,
             GENERALIZATION_OF_KEY,
             INTERPRETATION_ATTRIBUTES,
             DEFINITION_ATTRIBUTES,
             DISCRETIONARY_ACCESS,
             MANDATORY_ACCESS,
             INTERPRETATION_NODE_KIND,
             DEFINITION_NODE_KIND,
             STATUS,
             INTERPRETATION_MEDIUM,
             DEFINITION_MEDIUM);
        CLOSE (INTERPRETATION_NODE_KIND);
        CLOSE (DEFINITION_NODE_KIND);
    exception
        when others =>
            CLOSE (INTERPRETATION_NODE_KIND);
            CLOSE (DEFINITION_NODE_KIND);
            raise;
    end CREATE_NODE_KIND;
```

Notes:

This operation will fail if the mandatory access control classification specified for the new NKD is insufficient to permit property inheritance. 46


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

### 5.7.16 Deleting NKIs and NKDs

```
procedure DELETE_NODE_KIND
    (NKD                 : in out NKD_TYPE;
     VIEW                :  in    VIEW_NODE_TYPE;
     DELETE_DEFINITION   : in     BOOLEAN          := FALSE;
     TIME_LIMIT          :  in    CAIS_DURATION    := LONG_DELAY;
     STATUS              :  in    STATUS_INICATOR  := NO_INDICATION);
```

Purpose:

This procedure deletes anode kind interpretation from a view, and optionally deletes the corresponding node kind definition. 1

Effectively, the DELETE_NODE operation is performed for the NKI in the view identified by the open node hande VIEW and corresponding to the NKD identified by the open handle NKD. The NKI becomes unobtainable. All ATTRIBUTE_NAME, DEFINITION and Relation_-NAME relationships emanating from the NKI are deleted. The TIME_LIMIT parameter allows specification of a time limit for the delay imposed on the DELETE_NODE_KIND operation by the existence of locks on any node to be accessed. A delayed call completes after the node is unlocked or the specified time limit has elapsed. In the latter case, the exception LOCK_ERROR is raised. 2

If the value of DELETE_DEFINITION is TRUE, then this procedure also effectively performs the DELETE_NODE operation on the NKD. The restrictions in Section 5.7.1.9 on page 335 on deletion of an NKD apply. 3

If the DELETE_NODE_KIND operation raises an exception, no relationship is deleted. 4

Parameters:

**NKD** is an open node handle identifying the NKD whose corresponding NKI is to be deleted. 5

**VIEW** is an open node handle to the VIEW node from which the primary NODE_KIND relationship to be deleted emanates. 6

**DELETE_DEFINITION** is a Boolean. If TRUE, the primary relationship to the NKD is deleted. Otherwise, this operation has no effect on the NKD. 7

**TIME_LIMIT** is a time limit for the delay on waiting for the unlocking of a node. 8

**STATUS** is an established status indicator. 9

Exceptions:

**NAME_ERROR** is raised if the NKI or the parent of the NKD whose primary relationship is to be deleted or any source node of a secondary relationship to be deleted is inaccessible [1]. 10

**RELATIONSHIP_ERROR** is raised if the view does not contain an NKI corresponding to the NKD [2]. 11

**USE_ERROR** is raised if any primary relationships emanate from a node to be deleted [7], if a node to be deleted is predefine [62]. 12


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

**STATUS_ERROR** is raised if the node handle NKD or VIEW is not open prior to the call [2] or if STATUS is not an established status indicator [3]. 13

**NODE_KIND_ERROR** is raised if NKD does not identify an NKD [1] or if VIEW does not identify a VIEW node [1]. 14

**CONFORMANCE_ERROR** is raised if the operation would result in an attribute, node or relationship not conforming to its definition or the constraints in force on that definition [3]. 15

**DEFINITION_ERROR** is raised if the value of DELETE_DEFINITION is TRUE and a restriction on the deletion of an NKD would be violated [1] or if the value of DELETE_-DEFINITION is FALSE and the VIEW contains the last interpretation node corresponding to the definition [6]. 16

**LOCK_ERROR** is raised if a node handle identifying the source node of a relationship to be deleted cannot be opened with intents WRITE_RELATIONSHIPS [1], if a node handle identifying the NKI cannot be opened with intent EXCLUSIVE_WRITE due to outstanding locks [1] or if an open transaction not containing this operation contains other eligible operations whose effects determine whether the conditions for DEFINITION_ERROR apply [3]. 17

**TRANSACTION_ERROR** is raised if the operation is included in more than one transaction [1] or if the operation is included in a transaction and the operation modifies a node which is not included in the same transaction [2]. 18

**INTENT_VIOLATION** is raised if the node handle VIEW was not opened with an intent establishing the right to READ_RELATIONSHIPS and an exclusive intent establishing the right to WRITE_RELATIONSHIPS [1] or if DELETE_DEFINITION is TRUE and NKD was not opened with an intent including READ_RELATIONSHIPS and EXCLUSIVE_-WRITE [1]. 19

**ACCESS_VIOLATION** is raised if the current process does not have sufficient discretionary access control rights to obtain access to the source node of a relationship to be deleted with intents WRITE_RELATIONSHIPS or to obtain access to the NKI with intents EXCLU-SIVE_WRITE and READ_RELATIONSHIPS [1]. 20

**SECURITY_VIOLATION** is raised if the operation represents a violation of mandatory access controls. SECURITY_VIOLATION is raised only if the conditions for other exceptions are not present. 21

### 5.7.17 Creating NKIs

```
procedure IMPORT_NODE_KIND
    (NKI                    : in out NKI_TYPE;
     VIEW                   : in     VIEW_NODE_TYPE;
     NKD                    : in     NKD_TYPE;
     LOCAL_NAME             : in     RELATIONSHIP_KEY;
     ATTRIBUTES             : in     ATTRIBUTE_LIST          := EMPTY_LIST;
     DISCRETIONARY_ACCESS   : in     DISCRETIONARY_ACCESS_LIST : = ALL_RIGHTS;
     MANDATORY_ACCESS       : in     MANDATORY_ACCESS_LIST : = EMPTY_LIST;
     INTERPRETATION_KIND    : in     NODE_KIND_NODE_TYPE : =
                                        NODE_KIND_INTERPRETATION_NKD;
     STATUS                 : in     STATUS_INDICATOR        : = NO_INDICATION;
     MEDIUM_RESOURCE        : in     MEDIUM_NODE_TYPE        : = DEFAULT_MEDIUM);
```


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Purpose:

This procedure creates an interpretation for an existing node kind.

An NKI and the primary NODE_KIND relationship to it are created, as well as the corresponding reverse relationship to the VIEW node identified by VIEW. The primary relationship key list is given by LOCAL_NAME. An open node handle to the newly created NKI with WRITE intent is returned in NKI.

The corresponding NKD for the NKI is identified by the open node handle NKD. A DEFINITION relationship is created from the new NKI to the NKD.

The MANDATORY_ACCESS parameter specifies the object classification label with which the node is to be created. If its value is the empty list, the node inherits the subject classification of the creating process as its object classification.

INTERPRETATION_KIND is an open node handle identifying the node kind for the new node. It must identify an NKI kind. It may specify either an NKD or an NKI. If INTERPRETATION_-KIND identifies an NKI, the NKI is made the current adopted interpretation for the node handle NKI.

The MEDIUM_RESOURCE parameter specifies the medium resource on which the newly created node will reside.

Parameters:

**NKI** is a closed node handle to be opened to the newly created NKI.

**VIEW** is an open node handle to the VIEW node from which the primary relationship to the new NKI is to emanate.

**NKD** is an open node handle to the corresponding NKD for the new NKI.

**LOCAL_NAME** is the key list of the primary NODE_KIND relationship to be created to the NKI.

**ATTRIBUTES** is an empty or named list whose elements are used to establish initial values for attributes of the newly created NKI; each named item of the list specifies an attribute name and the value to be given to that attribute.

**DISCRETIONARY_ACCESS** is the initial access control information associated with the created node; it is the value of the GRANT attribute of the ACCESS relationship to the user's default GROUP node.

**MANDATORY_ACCESS** is a list defining the classification label for the created node.

**INTERPRETATION_KIND** is an open node handle identifying the node kind for the NKI to be created.

**STATUS** is an established status indicator.

**MEDIUM_RESOURCE** is an open node handle identifying a MEDIUM node.

Exceptions:

**PATHNAME_SYNTAX_ERROR** is raised if the node identification given by LOCAL_-NAME is syntactically illegal [2].


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

**NAME_ERROR** is raised if the node identified by NKD is inaccessible or unobtainable [1] or if the parent of the NKI identified by INTERPRETATION_KIND is inaccessible [1].

**EXISTING_NODE_ERROR** is raised if a node already exists with the identification given [1].

**RELATIONSHIP_ERROR** is raised if a key list specifies a value for the same key more than once [5].

**ATTRIBUTE_ERROR** is raised if an attribute list specifies a value for the same attribute more than once [6].

**NODE_KIND_ERROR** is raised if the open node handle VIEW does not identify a VIEW node [1], if the open node handle NKD does not identify an NKD [1], if the open node handle INTERPRETATION_KIND does not identify an NKI kind [2] or if the open node handle MEDIUM_RESOURCE does not identify a MEDIUM node [1].

**NOT_IN_VIEW_ERROR** is raised if an attribute name specified in LOCAL_NAME or ATTRIBUTES cannot be translated [1] or if LOCAL_NAME consists of a single relationship key designator and no key of the corresponding relation is visible [5].

**AMBIGUOUS_VIEW_ERROR** is raised if an attribute name specified in LOCAL_NAME or ATTRIBUTES has an ambiguous translation [1] or if LOCAL_NAME consists of a single relationship key designator and more than one key of the corresponding relation is visible [4].

**SYNTAX_ERROR** is raised if the ATTRIBUTES, DISCRETIONARY_ACCESS or MANDATORY_ACCESS parameter is syntactically illegal [8-10].

**PREDEFINED_ATTRIBUTE_ERROR** is raised if any attribute named in the ATTRIBUTES parameter is a CAIS-controlled attribute [1].

**USE_ERROR** is raised if the value for the MANDATORY_ACCESS parameter is semantically illegal [10] or if the view already includes an interpretation for the definition node identified through NKD [16].

**STATUS_ERROR** is raised if VIEW, NKD, INTERPRETATION_KIND or MEDIUM_RESOURCE is not an open node handle [2], if NKI is an open node handle prior to the call [1] or if STATUS is not an established status indicator [3].

**CONFORMANCE_ERROR** is raised if the operation would result in an attribute, node or relationship not conforming to its definition or the constraints in force on that definition [3,7,8,9,10].

**LOCK_ERROR** is raised if a node handle to the parent of the NKI specified by INTERPRETATION_KIND cannot be opened with intent READ_RELATIONSHIPS due to outstanding locks in conflict with that intent [1].

**TRANSACTION_ERROR** is raised if the operation is included in more than one transaction [1] or if the operation is included in a transaction and the operation modifies a node which is not included in the same transaction [2].

**INTENT_VIOLATION** is raised if VIEW was not opened with an intent establishing the right to read relationships and an exclusive intent establishing the right to write relationships [1].

**ACCESS_VIOLATION** is raised if the discretionary access control rights of the current process are insufficient to allow access with CREATE_INSTANCES intents to the NKD identified by INTERPRETATION_KIND or with READ_RELATIONSHIPS intents to the parent of the NKI specified by INTERPRETATION_KIND or with APPEND_CONTENTS intents to the MEDIUM node identified by MEDIUM_RESOURCE [1].


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

**RESOURCE_ALLOCATION_ERROR** is raised if the medium resource is not allocatable [1]. 34

**RESOURCE_AVAILABILITY_ERROR** is raised if the medium resource is not available [1]. 35

**SECURITY_VIOLATION** is raised if the operation represents a violation of mandatory access controls. SECURITY_VIOLATION is raised only if the conditions for other exceptions are not present. 36

Additional Interfaces:

```
procedure IMPORT_NODE_KIND
    (NKI                    : in out NKI_TYPE;
     VIEW                   : in     VIEW_NODE_TYPE;
     NKD                    : in     NKD_TYPE;
     LOCAL_NAME             : in     RELATIONSHIP_KEY;
     ATTRIBUTES             : in     ATTRIBUTE_LIST             : = EMPTY_LIST;
     DISCRETIONARY_ACCESS : in     DISCRETIONARY_ACCESS_LIST : = ALL_RIGHTS;
     MANDATORY_ACCESS       : in     MANDATORY_ACCESS_LIST : = EMPTY_LIST;
     INTERPRETATION_KIND    : in     NODE_KIND_NAME;
     STATUS                 : in     STATUS_INDICATOR           := NO_INDICATION;
     MEDIUM_RESOURCE        : in     MEDIUM_NODE_TYPE           := DEFAULT_MEDIUM)
is
    NODE_KIND : NKI_TYPE;
    -- "NODE_KIND_INTERPRETATION" or a specialization.
begin
    OPEN_NODE_KIND (NODE_KIND, INTERPRETATION_KIND, STATUS);
    IMPORT_NODE_KIND
        (NKI,
         VIEW,
         NKD,
         LOCAL_NAME,
         ATTRIBUTES,
         DISCRETIONARY_ACCESS,
         MANDATORY_ACCESS,
         NODE_KIND,
         STATUS ,
         MEDIUM_RESOURCE);
    CLOSE (NODE_KIND);
exception
    when others =>
        CLOSE (NODE_KIND);
        raise;
end IMPORT_NODE_KIND;
```

Notes:

An interpretation node may have a different mandatory access control classification than that of the corresponding definition node. 37


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

### 5.7.18 Renaming NODE_KIND relationships in a view

```
procedure RENAME_NODE_KIND
    (VIEW     : in     VIEW_NODE_TYPE;
     OLD_NAME : in     RELATIONSHIP_KEY;
     NEW_NAME : in     RELATIONSHIP_KEY;
     STATUS   : in     STATUS_INDICATOR := NO_INDICATION);
```

Purpose:

This procedure changes a name for an existing node kind. 1

The key designators for a primary NODE_KIND relationship emanating from the VIEW node identified by VIEW are changed. The relationship affected is identified by the key list parameter OLD_NAME. Its new identification is specified by the key list parameter NEW_NAME. 2

The primary relationship is not modified if this operation raises an exception. 3

Parameters:

**VIEW** is an open node handle to the VIEW node from which the primary relationship to be renamed emanates. 4

**OLD_NAME** is a key list for a NODE_KIND relationship which emanates from the VIEW node. 5

**NEW_NAME** is a new key list for the relationship identified by OLD_NAME. 6

**STATUS** is an established status indicator. 7

Exceptions:

**PATHNAME_SYNTAX_ERROR** is raised if the old or new node identification is syntactically illegal [2]. 8

**EXISTING_NODE_ERROR** is raised if another node exists with the new identification given [1]. 9

**RELATIONSHIP_ERROR** is raised if a relationship with the identification OLD_NAME does not exist [2] or if a key list specifies a value for the same key more than once [5]. 10

**NOT_IN_VIEW_ERROR** is raised if an attribute name in OLD_NAME or NEW_NAME cannot be translated [1] or if OLD_NAME or NEW_NAME consists of a single relationship key designator and no key of the corresponding relation is visible [5]. 11

**AMBIGUOUS_VIEW_ERROR** is raised if an attribute name in OLD_NAME or NEW_NAME has an ambiguous translation [1] or if OLD_NAME or NEW_NAME consists of a single relationship key designator and more than one key of the corresponding relation is visible [4]. 12

**NODE_KIND_ERROR** is raised if VIEW does not identify a VIEW node [1]. 13

**STATUS_ERROR** is raised if the node handle VIEW is not open [2] or if STATUS is not an established status indicator [3]. 14

**USE_ERROR** is raised if the relationship is predefine [61]. 15



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

**CONFORMANCE_ERROR** is raised if the operation would result in an attribute, node or relationship not conforming to its definition or the constraints in force on that definition [5,6,10]. 16

**TRANSACTION_ERROR** is raised if the operation is included in more than one transaction [1] or if the operation is included in a transaction and the operation modifies a node which is not included in the same transaction [2]. 17

**INTENT_VIOLATION** is raised if VIEW was not opened with an intent establishing the right to read relationships and an exclusive intent establishing the right to write relationships [1]. 18

**SECURITY_VIOLATION** is raised if the operation represents a violation of mandatory access controls. SECURITY_VIOLATION is raised only if the conditions for other exceptions are not present. 19

### 5.7.19 Specializing a node kind

```
procedure SPECIALIZE_NODE_KIND
    (NKD          : in  NKD_TYPE;
     ANCESTOR     : in  NKD_TYPE;
     KEY          : in  RELATIONSHIP_KEY  := EMPTY_KEY_LIST;
     REVERSE_ KEY : in  RELATIONSHIP_KEY  := EMPTY_KEY_LIST;
     STATUS       : in  STATUS_INDICATOR  := NO_INDICATION) ;
```

Purpose:

This procedure designates an existing node kind to be a specialization of another existing node kind. 1

A SPECIALIZATION_OF relationship between two NKDs is created. The source node is the NKD identified by the open node handle NKD. The target node is the NKD identified by the open node handle ANCESTOR. This procedure has no effect if such a SPECIALIZATION_OF relationship already exists. The NKDs cannot be the same node, nor can ANCESTOR identify a definition which is a specialization of the definition identified by NKD. The key lists used in creating the relationship and the corresponding reverse relationship are given by KEY and REVERSE_KEY. 2

An effect of this specialization is to make inheritable properties and constraints on those properties which are defined for the ancestor definition to be inherited by the definition identified by NKD (and its specializations). 3

If the specialization operation results in the inheritance of more restrictive constraints on a property, the semantics for deletion of a property apply. In particular, additional constraints on a property are not permitted if instances of that property exist on instances of the definition identified by NKD or specializations thereof. (See Section 5.7 on page 304 for the definition relationship attributes which define additional constraints, and for the deletion semantics which apply to property definition relationships.) 4

Properties inherited by a definition cannot take precedence over properties defined through property definition relationships emanating from the definition node. In particular, an ancestor cannot define more restrictive constraints for a property than a specialization does. 5

A further effect of this operation is that interpretations applicable to the ancestor kind become indirect applicable interpretations for the specialization. 6



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Parameters:

**NKD** is an open node handle to the definition which is to become a specialization.

**ANCESTOR** is an open node handle to the definition which is being specialized.

**KEY** is the key list of the SPECIALIZATION_OF relationship to be created from the node identified by NKD.

**REVERSE_KEY** is the key list of the GENERALIZATION_OF relationship to be created from the node identified by ANCESTOR.

**STATUS** is an established status indicator.

Exceptions:

**PATHNAME_SYNTAX_ERROR** is raised if the KEY or REVERSE_KEY parameter is syntactically illegal [2].

**RELATIONSHIP_ERROR** is raised if a SPECIALIZATION_OF or GENERALIZATION_-OF relationship with the identification given already exists [3] or if a key list specifies a value for the same key more than once [5].

**NOT_IN_VIEW_ERROR** is raised if an attribute name in KEY or REVERSE_KEY cannot be translated [1] or if KEY or REVERSE_KEY consists of a single relationship key designator and no key of the corresponding relation is visible [5].

**AMBIGUOUS_VIEW_ERROR** is raised if an attribute name in KEY or REVERSE_KEY has an ambiguous translation [1] or if KEY or REVERSE_KEY consists of a single relationship key designator and more than one key of the corresponding relation is visible [4].

**NODE_KIND_ERROR** is raised if NKD and ANCESTOR do not identify NKDs [1].

**DEFINITION_ERROR** is raised if this operation is used to specify properties or constraints which are mutually exclusive with those already defined or to specify more restrictive constraints for an existing property and an instance of the property already exists for an instance of the definition identified through NKD or a specialization thereof [3], if constraints would be modified on a predefine property of a predefine definition [9], or if one predefine definition would become a specialization of another and is not already [10].

**USE_ERROR** is raised if this operation would result in a cyclic path composed of SPECIALIZATION_OF relationships [17].

**STATUS_ERROR** is raised if NKD or ANCESTOR is not an open node handle [2] or if STATUS is not an established status indicator [3].

**CONFORMANCE_ERROR** is raised if the operation would result in an attribute, node or relationship not conforming to its definition or the constraints in force on that definition [3,10].

**LOCK_ERROR** is raised if an open transaction not containing this operation contains other eligible operations whose effects determine whether the conditions for DEFINITION_ERROR apply [3].

**TRANSACTION_ERROR** is raised if the operation is included in more than one transaction [1] or if the operation is included in a transaction and the operation modifies a node which is not included in the same transaction [2].



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

**INTENT_VIOLATION** is raised if NKD or ANCESTOR was not opened with an intent establishing the right to read and create relationships [1]. 23

**SECURITY_VIOLATION** is raised if the operation represents a violation of mandatory access controls. SECURITY_VIOLATION is raised only if the conditions for other exceptions are not present. 24

### 5.7.20 Generalizing node kinds

```
procedure GENERALIZE_NODE_KIND
   (NKD                      : in  NKD_TYPE;
    DESCENDANTS              : in  NODE_HANDLE_ARRAY;
    KEY_GENERATOR            : in  RELATIONSHIP_KEY  := EMPTY_KEY_LIST;
    REVERSE_KEY_GENERATOR    : in  RELATIONSHIP_KEY  := EMPTY_KEY_LIST;
    VIEW                     : in  VIEW_NODE_TYPE    := NULL_NODE;
    STATUS                   : in  STATUS_INDICATOR  := NO_INDICATION);
```

Purpose:

This procedure designates an existing node kind to be a generalization of a set of other existing node kinds. 1

SPECIALIZATION_OF relationships are created from a set of NKDs to a single NKD. The target node is the NKD identified by the open node handle NKD. The source nodes are the NKDs identified by the open node handles in DESCENDANTS which are not already specializations of the target. (Existing specializations can be specified. ) A node can be specified only once by the handles in NKD and DESCENDANTS, and NKD cannot identify a definition which is a specialization of a definition identified in DESCENDANTS. The effect is equivalent to multiple calls to the SPECIALIZE_NODE_KIND interface, except that no relationship is created if this operation raises an exception. 2

The key lists used in creating the SPECIALIZATION_OF relationships and the corresponding reverse relationships are given by KEY_GENERATOR and REVERSE_KEY_GENERATOR. If multiple relationships are to be created, designators specified for keys which must be unique must incorporate LATEST_KEY. 3

A further effect of this generalization is to make inheritable properties which are defined for all of the descendant definitions to be defined for the definition corresponding to the node identified by NKD, when the definition does not already have such properties. This is done by creating ATTRIBUTE, EMANATES or TERMINATES relationships to the definitions of such properties. The key designator used in creating such relationships is LATEST_KEY. MINIMUM_INSTANCES, MAXIMUM_INSTANCES, CONSTRAINTS and DEFAULT_VALUE attributes are not defined for these new relationships. 4

If VIEW has the value NULL_NODE, it has no effect. Otherwise, VIEW identifies a VIEW node for a view containing NKIs corresponding to the generalization NKD and zero or more descendant NKDs. For each property added to the generalization, a naming will be defined for the corresponding NKI if the NKI does not already define a naming for the same name or property and if the NKIs corresponding to the descendants define a common naming. This is done by creating ATTRIBUTE_NAME or RELATION_NAME relationships using the common key lists. 5



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Parameters:

**NKD** is an open node handle to the definition node which is to become a generalization.

**DESCENDANTS** is an array of open node handles to the NKDs from which the node identified by NKD is to be generalized.

**KEY_GENERATOR** is the key list for the SPECIALIZATION_OF relationships to be created from the node identified by NKD.

**REVERSE_KEY_GENERATOR** is the key list for the GENERALIZATION_OF relationships to be created from the node identified by DESCENDANTS.

**VIEW** is an open node handle identifying a VIEW node, or has the value NULL_NODE.

**STATUS** is an established status indicator.

Exceptions:

**PATHNAME_SYNTAX_ERROR** is raised if the KEY_GENERATOR or REVERSE_KEY_GENERATOR parameter is syntactically illegal [2].

**NAME_ERROR** is raised if an NKI in the specified view and corresponding to one of the NKDs specified is inaccessible [1].

**RELATIONSHIP_ERROR** is raised if the KEY_GENERATOR or REVERSE_KEY_GENERATOR parameter would result in creation of a relationship with the same identification as one which already exists [3] or if a key list specifies a value for the same key more than once [5].

**NOT_IN_VIEW_ERROR** is raised if an attribute name in KEY_GENERATOR or REVERSE_KEY_GENERATOR cannot be translated [1] or if KEY_GENERATOR or REVERSE_KEY_GENERATOR consists of a single relationship key designator and no key of the corresponding relation is visible [5].

**AMBIGUOUS_VIEW_ERROR** is raised if an attribute name in KEY_GENERATOR or REVERSE_KEY_GENERATOR has an ambiguous translation [1] or if KEY_GENERATOR or REVERSE_KEY_GENERATOR consists of a single relationship key designator and more than one key of the corresponding relation is visible [4].

**NODE_KIND_ERROR** is raised if the open node handles NKD and in DESCENDANTS do not identify NKDs [1] or if the open node handle VIEW does not identify a VIEW node [1].

**DEFINITION_ERROR** is raised if this operation is used to specify properties or constraints which are mutually exclusive with those already defined or to specify more restrictive constraints for an existing property and an instance of the property already exists for an instance of the definition identified through NKD or a specialization thereof [3], if constraints would be modified on a predefine property of a predefine definition [9], or if one predefine definition would become a specialization of another and is not already [10].

**USE_ERROR** is raised if this operation would result in a cyclic path composed of SPECIALIZATION_OF relationships [17], if a view is specified which does not contain an NKI corresponding to the generalization NKD [18], if a node is specified more than once by the handles NKD and in DESCENDANTS [25].

**STATUS_ERROR** is raised if the node handles NKD and in DESCENDANTS are not all open [2], if VIEW is neither an open node handle nor NULL_NODE [2] or if STATUS is not an established status indicator [3].


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

**CONFORMANCE_ERROR** is raised if the operation would result in an attribute, node or relationship not conforming to its definition or the constraints in force on that definition [3,10]. 21

**LOCK_ERROR** is raised if a node handle identifying the NKIs corresponding to the descendant definition nodes cannot be opened with intent READ_RELATIONSHIPS, if a node handle identifying the NKI corresponding to the generalization definition node cannot be opened with intents READ_RELATIONSHIPS and APPEND_RELATIONSHIPS due to outstanding locks [1] or if an open transaction not containing this operation contains other eligible operations whose effects determine whether the conditions for DEFINITION_ERROR apply [3]. 22

**TRANSACTION_ERROR** is raised if the operation is included in more than one transaction [1] or if the operation is included in a transaction and the operation modifies a node which is not included in the same transaction [2]. 23

**INTENT_VIOLATION** is raised if the node handles NKD and in DESCENDANTS were not all opened with an intent establishing the rights to read and create relationships [1] or if the open node handle VIEW was not opened with an intent establishing the right to read relationships and an exclusive intent establishing the right to write relationships [1]. 24

**ACCESS_VIOLATION** is raised if the discretionary access control rights of the current process are insufficient to gain access with READ_RELATIONSHIPS intent to the NKIs corresponding to the descendant definition nodes [1] or to gain access with READ and APPEND_RELATIONSHIPS intents to the NKI corresponding to the generalization definition node [1]. 25

**SECURITY_VIOLATION** is raised if the operation represents a violation of mandatory access controls. SECURITY_VIOLATION is raised only if the conditions for other exceptions are not present. 26

### 5.7.21 Deleting SPECIALIZATION_OF relationships from NKDs

```
procedure DELETE_NODE_KIND_SPECIALIZATION
    (DESCENDANT : in     NKD_TYPE;
     ANCESTOR   : in     NKD_TYPE;
     STATUS     : in     STATUS_INDICATOR := NO_INDICATION);
```

Purpose:

This procedure severs a specialization association between two node kinds. 1

A SPECIALIZATION_OF relationship between two NKDs is deleted. The source node is the NKD identified by the open node handle DESCENDANT. The target node is the NKD identified by the open node handle ANCESTOR. 2

There can be only one SPECIALIZATION_OF relationship directly connecting the two NKDs. However, independent of this relationship, there may also be a path composed of multiple SPECIALIZATION_OF relationships indirectly connecting the two nodes through one or more other NKDs. In this case, the descendant will still be a specialization of the ancestor upon completion of this operation, and there are no restrictions on the use of this procedure. 3

There are also no restrictions on the use of this procedure if there are no instances of the 4



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

descendant NKD and its specializations. However, if instances of the descendants exist, and the descendant would not be a specialization of the ancestor upon completion of this operation, certain restrictions apply. The restrictions are those governing the deletion of properties from a definition, as this procedure may effectively delete inherited properties.

The first restriction is that this procedure cannot result in the inheritance of multiple, conflicting default values for attributes. This would occur if a highest precedence default value was being inherited through the ancestor, and upon completion of this operation other generalizations of the descendant would provide two or more differing default values with none being highest precedence. This restriction applies at the NKD identified by DESCENDANT, without reference to the descendant's specializations.

The second restriction is that this procedure cannot result in the deletion of a property of which instances exist. This restriction applies as follows:

1. There is a set of properties defined for the descendant NKD by the SPECIALIZATION_OF relationship to be deleted. This set consists of those properties not defined by the descendant itself and not inherited through generalizations other than that identified by ANCESTOR. If there are instances of the NKD identified by DESCENDANT, they must have none of the properties in the given set.
2. For each of the specializations of the descendant, the set of properties specified above is reduced by any properties defined by the specialized definition. If there are instances of that definition, they must have none of the properties in the given set.
3. The above rule applies recursively, further reducing the set of properties under consideration for all specializations.

Eliminating an ancestor may eliminate some inherited constraints on remaining properties. There are no restrictions on weakening constraints, and so there are no restrictions on the use of this procedure relating to inherited constraints.

Parameters:

**DESCENDANT** is an open node handle identifying the specialization from which the SPECIALIZATION_OF relationship to be deleted emanates.

**ANCESTOR** is an open node handle identifying the generalization at which the SPECIALIZATION_OF relationship to be deleted terminates.

**STATUS** is an established status indicator.

Exceptions:

**RELATIONSHIP_ERROR** is raised if there is no SPECIALIZATION_OF relationship connetting the specified NKDs [4].

**USE_ERROR** is raised if the relationship to be deleted is predefine [63], if the SPECIALIZATION_OF relationship to be deleted is the only one emanating from the descendant [21].

**STATUS_ERROR** is raised if the node handle DESCENDANT or ANCESTOR is not open [2] or if STATUS is not an established status indicator [3].


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

**NODE_KIND_ERROR** is raised if DESCENDANT or ANCESTOR does not identify an NKD [1]. 16

**CONFORMANCE_ERROR** is raised if the operation would result in an attribute, node or relationship not conforming to its definition [3]. 17

**DEFINITION_ERROR** is raised if a restriction on the deletion of an inherited property would be violated [2] or if this operation results in the inheritance of differing default values not resolved by precedence [3]. 18

**LOCK_ERROR** is raised if an open transaction not containing this operation contains other eligible operations whose effects determine whether the conditions for DEFINITION_ERROR apply [3]. 19

**TRANSACTION_ERROR** is raised if the operation is included in more than one transaction [1] or if the operation is included in a transaction and the operation modifies a node which is not included in the same transaction [2]. 20

**INTENT_VIOLATION** is raised if the node handle DESCENDANT or ANCESTOR was not opened with an intent establishing the rights to read and write relationships [1]. 21

**SECURITY_VIOLATION** is raised if the operation represents a violation of mandatory access controls. SECURITY_VIOLATION is raised only if the conditions for other exceptions are not present. 22

### 5.7.22 Creating RDs

```
procedure CREATE_RELATION
    (RI                          : in out RI_TYPE;
     RD                          : in out RD_TYPE;
     VIEW                        : in     VIEW_NODE_TYPE;
     SPECIALIZATION_OF           : in     RD_TYPE;
     BASE                        : in     NODE_TYPE;
     LOCAL_NAME                  : in     RELATIONSHIP_KEY;
     KEY                         : in     RELATIONSHIP_KEY       : = EMPTY_KEY_LIST;
     SPECIALIZATION_OF_KEY       : in     RELATIONSHIP_KEY       : = EMPTY_KEY_LIST;
     GENERALIZATION_OF_KEY       : in     RELATIONSHIP_KEY       : = EMPTY_KEY_LIST;
     INTERPRETATION_ATTRIBUTES : in     ATTRIBUTE_LIST         : = EMPTY_LIST;
     DEFINITION_ATTRIBUTES       : in     ATTRIBUTE_LIST         := EMPTY_LIST;
     DISCRETIONARY_ACCESS        : in     DISCRETIONARY_ACCESS_LIST := ALL_RIGHTS;
     MANDATORY_ACCESS            : in     MANDATORY_ACCESS_LIST  := EMPTY_LIST;
     INTERPRETATION_KIND         : in     NODE_KIND_NODE_TYPE :=
                                              RELATION_INTERPRETATION_NKD;
     DEFINITION_KIND             : in     NODE_KIND_NODE_TYPE :=
                                              RELATION_DEFINITION_NKD;
     STATUS                      : in     STATUS_INDICATOR       := NO_INDICATION;
     INTERPRETATION_MEDIUM       : in     MEDIUM_NODE_TYPE       := DEFAULT_MEDIUM;
     DEFINITION_MEDIUM           : in     MEDIUM_NODE_TYPE       := DEFAULT_MEDIUM);
```

Purpose:

This procedure creates anew unidirectional relation. An RD and the primary RELATION_- 1



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

DEFINITION relationship to it are created, as well as the corresponding reverse relationship to the node identified by BASE. The primary relationship key list is given by KEY. An open node handle to the newly created RD with WRITE intent is returned in RD.

A SPECIALIZATION_OF relationship is created from the new RD to the unidirectional RD identified by SPECIALIZATION_OF. SPECIALIZATION_OF_KEY and GENERALIZATION_-OF_KEY provide the relationship key lists for this relationship and the corresponding reverse relationship.

This procedure also creates an RI and installs the primary RELATION relationship to it, as well as the corresponding reverse relationship to the VIEW node identified by VIEW. The primary relationship key list is given by LOCAL_NAME. An open node handle to the newly created RI with WRITE intent is returned in RI.

The INTERPRETATION_ATTRIBUTES and DEFINITION_ATTRIBUTES parameters provide initial values for attributes of the new RI and RD, respectively. INTERPRETATION_ATTRIBUTES cannot specify values for any CAIS-controlled attributes. DEFINITION_ATTRIBUTES may include values for the optional, CAIS-controlled RD attributes ALLOWED_USAGE and UNIQUE_KEYS.

The MANDATORY_ACCESS parameter specifies the object classification labels with which the nodes are to be created. If its value is the empty list, the nodes inherit the subject classification of the creating process as their object classification. (See Table IX on page 256.)

INTERPRETATION_KIND and DEFINITION_KIND are open node handles identifying the node kinds for the new nodes. They must identify an RI and RD kind, respectively. Each may specify a kind either through an NKD or an NKI. If INTERPRETATION_KIND identifies an NKI, the NKI is made the current adopted interpretation for the node handle RI. If DEFINITION_-KIND identifies an NKI, the NKI is made the current adopted interpretation for the node handle RD.

The INTERPRETATION_MEDIUM parameter specifies the medium resource on which the new RI will reside. DEFINITION_MEDIUM parameter specifies the medium resource on which the new RD will reside.

**Parameters:**

**RI** is a closed node handle be opened to the new RI.

**RD** is a closed node handle be opened to the new RD.

**VIEW** is an open node handle to the VIEW node from which the primary relationship to the new RI is to emanate.

**SPECIALIZATION_OF** is an open node handle identifying the RD of which the new definition is to be a specialization.

**BASE** is an open node handle to the node from which the primary relationship to the new RD is to emanate.

**LOCAL_NAME** is the relationship key list of the primary RELATION relationship to be created to the RI.

**KEY** is the relationship key list of the primary RELATION_DEFINITION relationship to be created to the RD.

**SPECIALIZATION_OF_KEY** is the key list of the SPECIALIZATION_OF relationship to be created from the RD.



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

**GENERALIZATION_OF_KEY** is the key list of the GENERALIZATION_OF relationship to be created to the RD.

**INTERPRETATION_ATTRIBUTES** is an empty or named list whose elements are used to establish initial values for attributes of the new RI; each named item of the list specifies an attribute name and the value to be given to that attribute.

**DEFINITION_ATTRIBUTES** is an empty or named list whose elements are used to establish initial values for attributes of the new RD; each named item of the list specifies an attribute name and the value to be given to that attribute.

**DISCRETIONARY_ACCESS** is the initial access control information associated with the created nodes; it is the value of the GRANT attributes of the ACCESS relationships to the user's default GROUP node.

**MANDATORY_ACCESS** is a list defining the classification labels for the created nodes.

**INTERPRETATION_KIND** is an open node handle identifying the node kind for the RI to be created.

**DEFINITION_KIND** is an open node handle identifying the node kind for the RD to be created.

**STATUS** is an established status indicator.

**INTERPRETATION_MEDIUM** is an open node handle identifying a MEDIUM node.

**DEFINITION_MEDIUM** is an open node handle identifying a MEDIUM node.

Exceptions:

**PATHNAME_SYNTAX_ERROR** is raised if the node identification given by LOCAL_NAME, KEY, SPECIALIZATION_OF_KEY or GENERALIZATION_OF_KEY is syntactically illegal [2].

**NAME_ERROR** is raised if the parent of the NKI identified by INTERPRETATION_KIND or DEFINITION_KIND is inaccessible [1].

**EXISTING_NODE_ERROR** is raised if a primary relationship already exists with the identification given [1].

**RELATIONSHIP_ERROR** is raised if a secondary relationship already exists with the identification given [3] or if a key list specifies a value for the same key more than once [5].

**ATTRIBUTE_ERROR** is raised if an attribute list specifies a value for the same attribute more than once [6].

**NODE_KIND_ERROR** is raised if the open node handle INTERPRETATION_KIND does not identify an RI kind [2], if the open node handle DEFINITION_KIND does not identify an RD kind [2], if the open handle VIEW does not identify a VIEW node [1], if the open handle SPECIALIZATION_OF does not identify an RD [1] or if the open handle INTERPRETATION_MEDIUM or DEFINITION_MEDIUM does not identify a MEDIUM node [1].

**NOT_IN_VIEW_ERROR** is raised if an attribute name in LOCAL_NAME, KEY, SPECIALIZATION_OF_KEY, GENERALIZATION_OF_KEY, INTERPRETATION_ATTRIBUTES or DEFINITION_ATTRIBUTES cannot be translated [1] or if LOCAL_NAME, KEY, SPECIALIZATION_OF_KEY or GENERALIZATION_OF_KEY consists of a single relationship key designator and no key of the corresponding relation is visible [5].



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

**AMBIGUOUS_VIEW_ERROR** is raised if an attribute name in LOCAL_NAME, KEY, SPECIALIZATION_OF_KEY, GENERALIZATION_OF_KEY, INTERPRETATION_ATTRIBUTES or DEFINITION_ATTRIBUTES has an ambiguous translation [1] or if LOCAL_NAME, KEY, SPECIALIZATION_OF_KEY or GENERALIZATION_OF_KEY consists of a single relationship key designator and more than one key of the corresponding relation is visible [4]. 33

**SYNTAX_ERROR** is raised if the INTERPRETATION_ATTRIBUTES, DEFINITION_ATTRIBUTES, DISCRETIONARY_ACCESS or MANDATORY_ACCESS parameter is syntactically illegal [8-10]. 34

**PREDEFINED_ATTRIBUTE_ERROR** is raised if any attribute named in the INTERPRETATION_ATTRIBUTES or DEFINITION_ATTRIBUTES parameters is a CAIS-controlled attribute that is not identified in the Purpose section of this interface as an attribute which may be specified in that parameter [1]. 35

**USE_ERROR** is raised if the value for the MANDATORY_ACCESS parameter is semantically illegal [10] or if the generalization identified is bidirectional [19]. 36

**STATUS_ERROR** is raised if VIEW, SPECIALIZATION_OF, BASE, INTERPRETATION_KIND, DEFINITION_KIND, INTERPRETATION_MEDIUM or DEFINITION_MEDIUM is not an open node handle [2], if RI or RD is an open node handle prior to the call [1] or if STATUS is not an established status indicator [3]. 37

**CONFORMANCE_ERROR** is raised if the operation would result in an attribute, node or relationship not conforming to its definition or the constraints in force on that definition [1,3,7,8,9,10]. 38

**LOCK_ERROR** is raised if a node handle to the parent of the NKI specified by INTERPRETATION_KIND or DEFINITION_KIND cannot be opened with intent READ_ RELATIONSHIPS due to outstanding locks in conflict with that intent [1]. 39

**TRANSACTION_ERROR** is raised if the operation is included in more than one transaction [1] or if the operation is included in a transaction and the operation modifies a node which is not included in the same transaction [2]. 40

**INTENT_VIOLATION** is raised if SPECIALIZATION_OF and BASE were not opened with an intent establishing the right to create relationships [1] or if VIEW was not opened with an exclusive intent establishing the right to create relationships [1]. 41

**ACCESS_VIOLATION** is raised if the discretionary access control rights of the current process are insufficient to allow access with CREATE_INSTANCES intents to the NKDs identified by INTERPRETATION_KIND or DEFINITION_KIND or with READ_RELATIONSHIPS intents to the parent of an NKI specified by INTERPRETATION_KIND or DEFINITION_KIND or with APPEND_CONTENTS intents to the MEDIUM nodes identified by INTERPRETATION_MEDIUM or DEFINITION_MEDIUM [1]. 42

**RESOURCE_ALLOCATION_ERROR** is raised if either medium resource is not allocatable [1]. 43

**RESOURCE_AVAILABILITY_ERROR** is raised if either medium resource is not available [1]. 44

**SECURITY_VIOLATION** is raised if the operation represents a violation of mandatory access controls. SECURITY_VIOLATION is raised only if the conditions for other exceptions are not present. 45


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Additional Interfaces:

```
procedure CREATE_RELATION
    (RI                              : in out RI_TYPE;
     RD                              : in out RD_TYPE;
     VIEW                            : in     VIEW_NODE_TYPE;
     SPECIALIZATION_OF               : in     RD_TYPE;
     BASE                            : in     NODE_TYPE;
     LOCAL_NAME                      : in     RELATIONSHIP_KEY;
     KEY                             : in     RELATIONSHIP_KEY       := EMPTY_KEY_LIST;
     SPECIALIZATION_OF_KEY           : in     RELATIONSHIP_KEY       := EMPTY_KEY_LIST;
     GENERALIZATION_OF_KEY           : in     RELATIONSHIP_KEY       := EMPTY_KEY_LIST;
     INTERPRETATION_ATTRIBUTES : in   ATTRIBUTE_LIST         := EMPTY_LIST;
     DEFINITION_ATTRIBUTES           : in     ATTRIBUTE_LIST         := EMPTY_LIST;
     DISCRETIONARY_ACCESS            : in     DISCRETIONARY_ACCESS_LIST := ALL_RIGHTS;
     MANDATORY_ACCESS                : in     MANDATORY_ACCESS_LIST := EMPTY_LIST;
     INTERPRETATION_KIND             : in     NODE_KIND_NAME;
     DEFINITION_KIND                 : in     NODE_KIND_NAME         :=
                                                 "RELATION_DEFINITION";
     STATUS                          : in     STATUS_INDICATOR       := NO_INDICATION;
     INTERPRETATION_MEDIUM           : in     MEDIUM_NODE_TYPE       := DEFAULT_MEDIUM;
     DEFINITION_MEDIUM               : in     MEDIUM_NODE_TYPE       := DEFAULT_MEDIUM)
is
    INTERPRETATION_NODE_KIND : NKI_TYPE;
    -- "RELATION_INTERPRETATION" or a specialization.
    DEFINITION_NODE_KIND       : NKI_TYPE;
begin
    OPEN_NODE_KIND (INTERPRETATION_NODE_KIND, INTERPRETATION_KIND, STATUS);
    OPEN_NODE_KIND (DEFINITION_NODE_KIND, DEFINITION_KIND, STATUS);
    CREATE_RELATION
        (RI,
         RD,
         VIEW,
         SPECIALIZATION_OF,
         BASE,
         LOCAL_NAME,
         KEY,
         SPECIALIZATION_OF_KEY,
         GENERALIZATION_OF_KEY,
         INTERPRETATION_ATTRIBUTES,
         DEFINITION_ATTRIBUTES,
         DISCRETIONARY_ACCESS,
         MANDATORY_ACCESS,
         INTERPRETATION_NODE_KIND,
         DEFINITION_NODE_KIND,
         STATUS,
         INTERPRETATION_MEDIUM,
         DEFINITION_MEDIUM);
```


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
        CLOSE (INTERPRETATION_NODE_KIND);
        CLOSE (DEFINITION_NODE_KIND);
    exception
        when others =>
            CLOSE (INTERPRETATION_NODE_KIND);
            CLOSE (DEFINITION_NODE_KIND);
            raise;
    end CREATE_RELATION;
```

Notes:

Upon successful completion of this operation, the attribute properties defined for the new relation are identical to those defied for the relation identified by SPECIALIZATION_OF. However, EMANATES_FROM and TERMINATES_AT properties are not defined for RDs through inheritance. 46

This operation will fail if the mandatory access control classification specified for the new RD is insufficient to permit property inheritance. 47

### 5.7.23 Deleting RIs and RDs

```
procedure DELETE_RELATION
    (RD                 : in out RD_TYPE;
     VIEW               : in     VIEW_NODE_TYPE;
     DELETE_DEFINITION  : in     BOOLEAN          := FALSE;
     TIME_LIMIT         : in     CAIS_DURATION    := LONG_DELAY;
     STATUS             : in     STATUS_INDICATOR := NO_INDICATION);
```

Purpose:

This procedure deletes a relation interpretation for either unidirectional or bidirectional relations, and optionally deletes a unidirectional relation definition. 1

Effectively, the DELETE_NODE operation is performed for the RI in the view identified by the open node handle VIEW and corresponding to the RD identified by the open handle RD. The RI becomes unobtainable. All ATTRIBUTE_NAME, KEY_NAME and DEFINITION relationships emanating from the RI are deleted; all RELATION_NAME relationships terminating at the RI are deleted. The TIME_LIMIT parameter allows specification of a time limit for the delay imposed on the DELETE_RELATION operation by the existence of locks on any node to be accessed. A delayed call completes after the node is unlocked or the specified time limit has elapsed. In the latter case, the exception LOCK_ERROR is raised. 2

If the value of DELETE_DEFINITION is TRUE, then this procedure also effectively performs the DELETE_NODE operation on the RD identified by the open node handle RD. This RD must be for unidirectional relations. The restrictions in Section 5.7.1.9 on page 335 on deletion of RDs apply. 3

If the DELETE_RELATION operation raises an exception, no relationship is deleted. 4


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Parameters:

**RD** is an open node handle identifying an RD whose corresponding RI is to be deleted.

**VIEW** is an open node handle to the VIEW node from which the primary RELATION relationship to be deleted emanates.

**DELETE_DEFINITION** is a Boolean. If TRUE, the primary relationship to the RD is to be deleted. Otherwise, this operation has no effect on the RD.

**TIME_LIMIT** is a time limit for the delay on waiting for the unlocking of a node.

**STATUS** is an established status indicator.

Exceptions:

**NAME_ERROR** is raised if the RI or the parent of the RD whose primary relationship is to be deleted or the source node of a secondary relationship to be deleted is inaccessible [1].

**RELATIONSHIP_ERROR** is raised if the view does not contain an RI corresponding to the RD [2].

**USE_ERROR** is raised if any primary relationships emanate from a node to be deleted [7], if a node to be deleted is predefined [62] or if DELETE_DEFINITION is TRUE and the relation identified is bidirectional [12].

**STATUS_ERROR** is raised if the node handle RD or VIEW is not open prior to the call [2] or if STATUS is not an established status indicator [3].

**NODE_KIND_ERROR** is raised if RD does not identify an RD [1] or if VIEW does not identify a VIEW node [1].

**CONFORMANCE_ERROR** is raised if the operation would result in an attribute, node or relationship not conforming to its definition or the constraints in force on that definition [3].

**DEFINITION_ERROR** is raised if the value of DELETE_DEFINITION is TRUE and a restriction on the deletion of an RD would be violated [1] or if the value of DELETE_DEFINITION is FALSE and view contains the last interpretation node corresponding to the definition [6].

**LOCK_ERROR** is raised if a node handle identifying the source node of a relationship to be deleted cannot be opened with intents WRITE_RELATIONSHIPS, if a node handle identifying the RI cannot be opened with intent EXCLUSIVE_WRITE due to outstanding locks [1] or if an open transaction not containing this operation contains other eligible operations whose effects determine whether the conditions for DEFINITION_ERROR apply [3].

**TRANSACTION_ERROR** is raised if the operation is included in more than one transaction [1] or if the operation is included in a transaction and the operation modifies a node which is not included in the same transaction [2].

**INTENT_VIOLATION** is raised if the node handle VIEW was not opened with an intent establishing the right to READ_RELATIONSHIPS and an exclusive intent establishing the right to WRITE_RELATIONSHIPS [1] or if DELETE_DEFINITION is TRUE and RD was not opened with intents READ_RELATIONSHIPS and EXCLUSIVE_WRITE [1].


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

**ACCESS_VIOLATION** is raised if the current process does not have sufficient discretionary access control rights to obtain access to the source node of a relationship to be deleted with intents WRITE_RELATIONSHIPS or to obtain access to the RI with intent EXCLUSIVE_WRITE or to obtain access to an NKI in the view with intents READ_RELATIONSHIPS and WRITE_RELATIONSHIPS [1]. 20

**SECURITY_VIOLATION** is raised if the operation represents a violation of mandatory access controls. SECURITY_VIOLATION is raised only if the conditions for other exceptions are not present. 21

Notes:

This operation cannot be used to delete the primary relationship to RDs which were created by the CREATE_BIDIRECTIONAL_RELATION operation. 22

### 5.7.24 Creating RIs

```
procedure IMPORT_RELATION
    (RI                    : in out RI_TYPE;
     VIEW                  : in     VIEW_NODE_TYPE;
     RD                    : in     RD_TYPE;
     LOCAL_NAME            : in     RELATIONSHIP_KEY;
     ATTRIBUTES            : in     ATTRIBUTE_LIST          := EMPTY_LIST;
     DISCRETIONARY_ACCESS : in     DISCRETIONARY_ACCESS_LIST : = ALL_RIGHTS;
     MANDATORY_ACCESS      : in     MANDATORY_ACCESS_LIST : = EMPTY_LIST;
     INTERPRETATION_KIND : in     NODE_KIND_NODE_TYPE : =
                                      RELATION_INTERPRETATION_NKD;
     STATUS                : in     STATUS_INDICATOR        := NO_INDICATION;
     MEDIUM_RESOURCE       : in     MEDIUM_NODE_TYPE        := DEFAULT_MEDIUM);
```

Purpose:

This procedure creates an interpretation for an existing relation. 1

An RI and the primary RELATION relationship to it are created, as well as the corresponding reverse relationship to the VIEW node identified by VIEW. The primary relationship key list is given by LOCAL_NAME. An open node handle to the new RI with WRITE intent is returned in RI. 2

The corresponding RD for the RI is identified by the open node handle RD. A DEFINITION relationship is created from the new RI to the RD. 3

The MANDATORY_ACCESS parameter specifies the object classification label with which the node is to be created. If its value is the empty list, the node inherits the subject classification of the creating process as its object classification. 4

INTERPRETATION_KIND is an open node handle identifying the node kind for the new node. It must identify RI kind. It may specify either an NKD or an NKI. If INTERPRETATION_KIND identifies an NKI, the NKI is made the current adopted interpretation for the node handle RI. 5

The MEDIUM_RESOURCE parameter specifies the medium resource on which the new RI will reside. 6


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Parameters:

**RI** is a closed node handle to be opened to the newly created RI.

**VIEW** is an open node handle to the view node from which the primary relationship to the new RI is to emanate.

**RD** is an open node handle to the corresponding RD for the new RI.

**LOCAL_NAME** is the relationship key list of the primary RELATION relationship to be created to the RI.

**ATTRIBUTES** is an empty or named list whose elements are used to establish initial values for attributes of the newly created RI; each named item of the list specifies an attribute name and the value to be given to that attribute.

**DISCRETIONARY_ACCESS** is the initial access control information associated with the created node; it is the value of the GRANT attribute of the ACCESS relationship to the user's default GROUP node.

**MANDATORY_ACCESS** is a list defining the classification label for the created node.

**INTERPRETATION_KIND** is an open node handle identifying the node kind for the RI to be created.

**STATUS** is an established status indicator.

**MEDIUM_RESOURCE** is an open node handle identifying a MEDIUM node.

Exceptions:

**PATHNAME_SYNTAX_ERROR** is raised if the node identification given by LOCAL_NAME is syntactically illegal [2].

**NAME_ERROR** is raised if the node identified by RD is inaccessible or unobtainable [1] or if the parent of the NKI identified by INTERPRETATION_KIND is inaccessible [1].

**EXISTING_NODE_ERROR** is raised if a node already exists with the identification given [1].

**RELATIONSHIP_ERROR** is raised if a key list specifies a value for the same key more than once [5].

**ATTRIBUTE_ERROR** is raised if an attribute list specifies a value for the same attribute more than once [6].

**NODE_KIND_ERROR** is raised if the open node handle VIEW does not identify a VIEW node [1], if the open node handle RD does not identify an RD [1], if the open node handle INTERPRETATION_KIND does not identify an RI kind [2] or if the open node handle MEDIUM_RESOURCE does not identify a MEDIUM node [1].

**NOT_IN_VIEW_ERROR** is raised if an attribute name specified in LOCAL_NAME or ATTRIBUTES cannot be translated [1] or if LOCAL_NAME consists of a single relationship key designator and no key of the corresponding relation is visible [5].

**AMBIGUOUS_VIEW_ERROR** is raised if an attribute name specified in LOCAL_NAME or ATTRIBUTES has an ambiguous translation [1] or if LOCAL_NAME consists of a single relationship key designator and more than one key of the corresponding relation is visible [4].



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

**SYNTAX_ERROR** is raised if the ATTRIBUTES, DISCRETIONARY_ACCESS or MANDATORY_ACCESS parameter is syntactically illegal [8-10]. 25

**PREDEFINED_ATTRIBUTE_ERROR** is raised if any attribute named in the ATTRIBUTES parameter is a CAIS-controlled attribute [1]. 26

**USE_ERROR** is raised if the value for the MANDATORY_ACCESS parameter is semantically illegal [10] or if the view already includes an interpretation for the definition node identified through RD [16]. 27

**STATUS_ERROR** is raised if VIEW, RD, INTERPRETATION_KIND or MEDIUM_RESOURCE is not an open node handle [2], if RI is an open node handle prior to the call [1] or if STATUS is not an established status indicator [3]. 28

**CONFORMANCE_ERROR** is raised if the operation would result in an attribute, node or relationship not conforming to its definition or the constraints in force on that definition [3,7,8,9,10]. 29

**LOCK_ERROR** is raised if a node handle to the parent of the NKI specified by INTERPRETATION_KIND cannot be opened with intent READ_RELATIONSHIPS due to outstanding locks in conflict with that intent [1]. 30

**TRANSACTION_ERROR** is raised if the operation is included in more than one transaction [1] or if the operation is included in a transaction and the operation modifies a node which is not included in the same transaction [2]. 31

**INTENT_VIOLATION** is raised if VIEW was not opened with an intent establishing the right to read relationships and an exclusive intent establishing the right to write relationships [1]. 32

**ACCESS_VIOLATION** is raised if the discretionary access control rights of the current process are insufficient to allow access with CREATE_INSTANCES intents to the NKD identified through INTERPRETATION_KIND or with READ_RELATIONSHIPS intents to the parent of the NKI specified by INTERPRETATION_KIND or with APPEND_CONTENTS intents to the MEDIUM node identified by MEDIUM_RESOURCE [1]. 33

**RESOURCE_ALLOCATION_ERROR** is raised if the medium resource is not allocatable [1]. 34

**RESOURCE_AVAILABILITY_ERROR** is raised if the medium resource is not available [1]. 35

**SECURITY_VIOLATION** is raised if the operation represents a violation of mandatory access controls. SECURITY_VIOLATION is raised only if the conditions for other exceptions are not present. 36

Additional Interfaces:

```
procedure IMPORT_RELATION
    (RI                    : in out RI_TYPE;
     VIEW                  : in     VIEW_NODE_TYPE;
     RD                    : in     RD_TYPE;
     LOCAL_NAME            : in     RELATIONSHIP_KEY;
     ATTRIBUTES            : in     ATTRIBUTE_LIST           := EMPTY_LIST;
     DISCRETIONARY_ACCESS  : in     DISCRETIONARY_ACCESS_LIST := ALL_RIGHTS;
     MANDATORY_ACCESS      : in     MANDATORY_ACCESS_LIST := EMPTY_LIST;
     INTERPRETATION_KIND   : in     NODE_KIND_NAME;
```


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
        STATUS              : in  STATUS_INDICATOR  := NO_INDICATION;
        MEDIUM_RESOURCE     : in  MEDIUM_NODE_TYPE  := DEFAULT_MEDIUM)
    is
        NODE_KIND : NKI_TYPE; -- "RELATION_INTERPRETATION" or a specialization.
    begin
        OPEN_NODE_KIND (NODE_KIND, INTERPRETATION_KIND, STATUS);
        IMPORT_RELATION
            (RI,
             VIEW,
             RD,
             LOCAL_NAME,
             ATTRIBUTES,
             DISCRETIONARY_ACCESS,
             MANDATORY_ACCESS,
             NODE_KIND,
             STATUS,
             MEDIUM_RESOURCE);
        CLOSE (NODE_KIND);
    exception
        when others =>
            CLOSE (NODE_KIND);
            raise;
    end IMPORT_RELATION;
```

Notes:

An interpretation node may have a different mandatory access control classification than that of the corresponding definition node. 37

### 5.7.25 Renaming RELATION relationships in a view

```
procedure RENAME_RELATION
    (VIEW     : in  VIEW_NODE_TYPE;
     OLD_NAME : in  RELATIONSHIP_KEY;
     NEW_NAME : in  RELATIONSHIP_KEY;
     STATUS   : in  STATUS_INDICATOR := NO_INDICATION);
```

Purpose:

This procedure changes a name for an existing relation definition. 1

The key designators for a primary RELATION relationship emanating from the VIEW node identified by VIEW are changed. The relationship affected is identified by the key list parameter OLD_NAME. Its new identification is specified by the key list parameter NEW_NAME. 2

The primary relationship is not modified if this operation raises an exception. 3


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Parameters:

**VIEW** is an open node handle to the VIEW node from which the primary relationship to be renamed emanates.

**OLD_NAME** is a key list for a RELATION relationship which emanates from the view node.

**NEW_NAME** is a new key list for the relationship identified by OLD_NAME.

**STATUS** is an established status indicator.

Exceptions:

**PATHNAME_SYNTAX_ERROR** is raised if the old or new node identification is syntactically illegal [2].

**EXISTING_NODE_ERROR** is raised if another node exists with the new identification given [1].

**RELATIONSHIP_ERROR** is raised if a relationship with the identification OLD_NAME does not exist [2] or if a key list specifies a value for the same key more than once [5].

**NOT_IN_VIEW_ERROR** is raised if an attribute name in OLD_NAME or NEW_NAME cannot be translated [1] or if OLD_NAME or NEW_NAME consists of a single relationship key designator and no key of the corresponding relation is visible [5].

**AMBIGUOUS_VIEW_ERROR** is raised if an attribute name in OLD_NAME or NEW_NAME has an ambiguous translation [1] or if OLD_NAME or NEW_NAME consists of a single relationship key designator and more than one key of the corresponding relation is visible [4].

**NODE_KIND_ERROR** is raised if VIEW does not identify a VIEW node [1].

**STATUS_ERROR** is raised if the node handle VIEW is not open [2] or if STATUS is not an established status indicator [3].

**USE_ERROR** is raised if the relationship is predefine [61].

**CONFORMANCE_ERROR** is raised if the operation would result in an attribute, node or relationship not conforming to its definition or the constraints in force on that definition [5,6,10].

**TRANSACTION_ERROR** is raised if the operation is included in more than one transaction [1] or if the operation is included in a transaction and the operation modifies a node which is not included in the same transaction [2].

**INTENT_VIOLATION** is raised if VIEW was not opened with an intent establishing the right to read relationships and an exclusive intent establishing the right to write relationships [1].

**SECURITY_VIOLATION** is raised if the operation represents a violation of mandatory access controls. SECURITY_VIOLATION is raised only if the conditions for other exceptions are not present.

### 5.7.26 Specializing a relation

```
procedure SPECIALIZE_RELATION
    (RD              : in      RD_TYPE;
```


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
    ANCESTOR      : in    RD_TYPE;
    KEY           : in    RELATIONSHIP_KEY    := EMPTY_KEY_LIST;
    REVERSE_KEY   : in    RELATIONSHIP_KEY    := EMPTY_KEY_LIST;
    STATUS        : in    STATUS_INDICATOR    := NO_INDICATION);
```

Purpose:

This procedure designates an existing relation to be a specialization of another existing relation. 1

A SPECIALIZATION_OF relationship between two RDs is created. The source node is the RD identified by the open node handle RD. The target node is the RD identified by the open node handle ANCESTOR. This procedure has no effect if such a SPECIALIZATION_OF relationship already exists. The RDs cannot be the same node, nor can ANCESTOR identify a definition which is a specialization of the definition identified by RD. The key lists used in creating the relationship and the corresponding reverse relationship are given by KEY and REVERSE_KEY. 2

If the ancestor definition is part of a bidirectional RD, the specialization must be as well. In this case, the specialization RD's reverse RD is made a specialization of the ancestor definition's reverse RD. 3

An effect of this specialization is to make inheritable properties and constraints on those properties which are defined for the ancestor definition to be inherited by the definition identified by RD (and its specializations). 4

If the specialization operation results in the inheritance of more restrictive constraints on a property, the semantics for deletion of a property apply. In particular, additional constraints on a property are not permitted if instances of that property exist on instances of the definition identified by RD or specializations thereof. (See Section 5.7 on page 304 for the definition relationship attributes which define additional constraints, and for the deletion semantics which apply to property definition relationships.) 5

Properties inherited by a definition cannot take precedence over properties defined through property definition relationships emanating from the definition node. In particular, an ancestor cannot define more restrictive constraints for a property than a specialization does. 6

A further effect of this operation is that interpretations applicable to the ancestor kind become indirect applicable interpretations for the specialization. 7

Parameters:

**RD** is an open node handle to the definition which is to become a specialization. 8

**ANCESTOR** is an open node handle to the definition which is being specialized. 9

**KEY** is the key list of the SPECIALIZATION_OF relationship to be created from the node identified by RD. 10

**REVERSE_KEY** is the key list of the GENERALIZATION_OF relationship to be created from the node identified by ANCESTOR. 11

**STATUS** is an established status indicator. 12


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Exceptions:

**PATHNAME_SYNTAX_ERROR is** raised if the KEY or REVERSE_KEY parameter is syntactically illegal [2]. 13

**RELATIONSHIP_ERROR** is raised if a SPECIALIZATION_OF or GENERALIZATION_-OF relationship with the identification given already exists [3] or if a key list specifies a value for the same key more than once [5]. 14

**NOT_IN_VIEW_ERROR** is raised if an attribute name in KEY or REVERSE_KEY cannot be translated [1] or if KEY or REVERSE_KEY consists of a single relationship key designator and no key of the corresponding relation is visible [5]. 15

**AMBIGUOUS_VIEW_ERROR** is raised if an attribute name in KEY or REVERSE_KEY has an ambiguous translation [1] or if KEY or REVERSE_KEY consists of a single relationship key designator and more than one key of the corresponding relation is visible [4]. 16

**NODE_KIND_ERROR** is raised if RD and ANCESTOR do not identify RDs [1]. 17

**DEFINITION_ERROR** is raised if this operation is used to specify properties or constraints which are mutually exclusive with those already defined or to specify more restrictive constraints for an existing property and an instance of the property already exists for an instance of the definition identified through RD or a specialization thereof [3], if constraints would be modified on a predefine property of a predefine definition [9], or if one predefine definition would become a specialization of another and is not already [10]. 18

**USE_ERROR** is raised if this operation would result in a cyclic path composed of SPECIALIZATION_OF relationships [17] or if the ancestor definition is a bidirectional RD and the definition identified through RD is not [19]. 19

**STATUS_ERROR** is raised if RD or ANCESTOR is not an open node handle [2] or if STATUS is not an established status indicator [3]. 20

**CONFORMANCE_ERROR** is raised if the operation would result in an attribute, node or relationship not conforming to its definition or the constraints in force on that definition [3,10]. 21

**LOCK_ERROR** is raised if an open transaction not containing this operation contains other eligible operations whose effects determine whether the conditions for DEFINITION_ERROR apply [3]. 22

**TRANSACTION_ERROR** is raised if the operation is included in more than one transaction [1] or if the operation is included in a transaction and the operation modifies a node which is not included in the same transaction [2]. 23

**INTENT_VIOLATION** is raised if RD or ANCESTOR was not opened with an intent establishing the right to read attributes and to read and create relationships [1]. 24

**SECURITY_VIOLATION** is raised if the operation represents a violation of mandatory access controls. SECURITY_VIOLATION is raised only if the conditions for other exceptions are not present. 25

### 5.7.27 Generalizing relations

```
procedure GENERALIZE_RELATION
    (RD                    : in     RD_TYPE;
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
    DESCENDANTS              : in    NODE_HANDLE_ARRAY;
    KEY_GENERATOR            : in    RELATIONSHIP_KEY   := EMPTY_KEY_LIST;
    REVERSE_KEY_GENERATOR    : in    RELATIONSHIP_KEY   := EMPTY_KEY_LIST;
    VIEW                     : in    VIEW_NODE_TYPE     := NULL_NODE;
    STATUS                   : in    STATUS_INDICATOR   := NO_INDICATION);
```

Purpose:

This procedure designates an existing relation to be a generalization of a set of other existing relations. 1

SPECIALIZATION_OF relationships are created from a set of RDs to a single RD. The target node is the RD identified by the open node handle RD. The source nodes are the RDs identified by the open node handles in DESCENDANTS which are not already specializations of the target. (Existing specializations can be specified.) A node can be specified only once by the handles in RD and DESCENDANTS, and RD cannot identify a definition which is a specialization of a definition identified in DESCENDANTS. The effect is equivalent to multiple calls to the SPECIALIZE_-RELATION interface, except that no relationship is created if this operation raises an exception. 2

The key lists used in creating the SPECIALIZATION_OF relationships and the corresponding reverse relationships are given by KEY_GENERATOR and REVERSE_KEY_GENERATOR. If multiple relationships are to be created, designators specified for keys which must be unique must incorporate LATEST_KEY. 3

If the intended generalization is part of a bidirectional RD, the descendants must be as well. In this case, the operation is applied to the reverse relations as well. 4

A further effect of this generalization is to make inheritable properties which are defined for all of the descendant definitions to be defined for the definition corresponding to the node identified by RD, when the definition does not already have such properties. This is done by creating ATTRIBUTE, KEY, EMANATES_FROM or TERMINATES_AT relationships to the definitions of such properties. The key designator used in creating such relationships is LATEST_KEY_MINIMUM_INSTANCES, MAXIMUM_INSTANCES, CONSTRAINTS and DEFAULT_VALUE attributes are not defined for these new relationships. 5

If VIEW has the value NULL_NODE, it has no effect. Otherwise, VIEW identifies a VIEW node for a view containing RIs corresponding to the generalization RD and zero or more descendant RDs. For each property added to the generalization, a naming will be defined for the corresponding RI if the RI does not already define a naming for the same name or property and if the RIs corresponding to the descendants define a common naming. This is done by creating ATTRIBUTE_NAME, KEY_NAME or RELATION_NAME relationships using the common key lists. 6

Parameters:

**RD** is an open node handle to the RD which is to become a generalization. 7

**DESCENDANTS** is an array of open node handles to the RDs from which the node identified by RD is to be generalized. 8

**VIEW** is an open node handle identifying a VIEW node, or has the value NULL_NODE. 9

**KEY_GENERATOR** is the key list for the SPECIALIZATION_OF relationships to be created from the node identified by RD. 10


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

**REVERSE_KEY_GENERATOR** is the key list for the GENERALIZATION_OF relationships to be created from the node identified by DESCENDANTS. 11

**STATUS** is an established status indicator. 12

Exceptions:

**PATHNAME_SYNTAX_ERROR** is raised if the KEY_GENERATOR or REVERSE_KEY_GENERATOR parameter is syntactically illegal [2]. 13

**NAME_ERROR** is raised if an RI in the specified view and corresponding to one of the RDs specified is inaccessible [1]. 14

**RELATIONSHIP_ERROR** is raised if the KEY_GENERATOR or REVERSE_KEY_GENERATOR parameter would result in creation of a relationship with the same identification as one which already exists [3] or if a key list specifies a value for the same key more than once [5]. 15

**NOT_IN_VIEW_ERROR** is raised if an attribute name in KEY_GENERATOR or REVERSE_KEY_GENERATOR cannot be translated [1] or if KEY_GENERATOR or REVERSE_KEY_GENERATOR consists of a single relationship key designator and no key of the corresponding relation is visible [5]. 16

**AMBIGUOUS_VIEW_ERROR** is raised if an attribute name in KEY_GENERATOR or REVERSE_KEY_GENERATOR has an ambiguous translation [1] or if KEY_GENERATOR or REVERSE_KEY_GENERATOR consists of a single relationship key designator and more than one key of the corresponding relation is visible [4]. 17

**NODE_KIND_ERROR** is raised if the open node handles RD and in DESCENDANTS do not identify RDs [1] or if the open node handle VIEW does not identify a VIEW node [1]. 18

**DEFINITION_ERROR** is raised if this operation is used to specify properties or constraints which are mutually exclusive with those already defied or to specify more restrictive constraints for an existing property and an instance of the property already exists for an instance of the definition identified through RD or a specialization thereof [3], if constraints would be modified on a predefined property of a predefine definition [9], or if one predefine definition would become a specialization of another and is not already [10]. 19

**USE_ERROR** is raised if this operation would result in a cyclic path composed of SPECIALIZATION_OF relationships [17], if a view is specified which does not contain an RI corresponding to the generalization RD [18], if the definition identified through RD is a bidirectional RD and any of those identified through DESCENDANTS is not [19] or if a node is specified more than once by the handles RD and in DESCENDANTS [25]. 20

**STATUS_ERROR** is raised if the node handles RD and in DESCENDANTS are not all open [2], if VIEW is neither an open node handle nor NULL_NODE [2] or if STATUS is not an established status indicator [3]. 21

**CONFORMANCE_ERROR** is raised if the operation would result in an attribute, node or relationship not conforming to its definition or the constraints in force on that definition [3,10]. 22

**LOCK_ERROR** is raised if a node handle identifying the RIs corresponding to the descendant definition nodes cannot be opened with intent READ_RELATIONSHIPS, if a node handle identifying the RI corresponding to the generalization definition node cannot be opened with intents READ_RELATIONSHIPS and APPEND_RELATIONSHIPS due to outstanding 23


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

locks [1] or if an open transaction not containing this operation contains other eligible operations whose effects determine whether the conditions for DEFINITION_ERROR apply [3].

**TRANSACTION_ERROR** is raised if the operation is included in more than one transaction [1] or if the operation is included in a transaction and the operation modifies a node which is not included in the same transaction [2].

**INTENT_VIOLATION** is raised if the node handles RD and in DESCENDANTS were not all opened with an intent establishing the rights to read attributes and to read and create relationships [1] or if the open node handle VIEW was not opened with an intent establishing the right to read relationships and an exclusive intent establishing the right to write relationships [1].

**ACCESS_VIOLATION** is raised if the discretionary access control rights of the current process are insufficient to gain access with READ_RELATIONSHIPS intent to the RIs corresponding to the descendant definition nodes [1] or to gain access with READ and APPEND_RELATIONSHIPS intents to the RI corresponding to the generalization definition node [1].

**SECURITY_VIOLATION** is raised if the operation represents a violation of mandatory access controls. SECURITY_VIOLATION is raised only if the conditions for other exceptions are not present.

### 5.7.28 Deleting SPECIALIZATION_OF relationships from RDs

```
procedure DELETE_RELATION_SPECIALIZATION
    (DESCENDANT : in     RD_TYPE;
     ANCESTOR   : in     RD_TYPE;
     STATUS     : in     STATUS_INDICATOR  := NO_INDICATION);
```

Purpose:

This procedure severs a specialization association between two relations.

A SPECIALIZATION_OF relationship between two RDs is deleted. The source node is the RD identified by the open node handle DESCENDANT. The target node is the RD identified by the open node handle ANCESTOR. If the ancestor is bidirectional, the descendant will be as well, and there will be a SPECIALIZATION_OF relationship between the two reverse relations; that SPECIALIZATION_OF relationship is also deleted.

There can be only one SPECIALIZATION_OF relationship directly connecting the two RDs. However, independent of this relationship, there may also be a path composed of multiple SPECIALIZATION_OF relationships indirectly connecting the two nodes through one or more other RDs. In this case, the descendant will still be a specialization of the ancestor upon completion, of this operation, and there are no restrictions on the use of this procedure.

There are also no restrictions on the use of this procedure if there are no instances of the descendant RD and its specializations. However, if instances of the descendants exist, and the descendant would not be a specialization of the ancestor upon completion of this operation, certain restrictions apply. The restrictions are those governing the deletion of properties from a definition, as this procedure may effectively delete inherited properties.


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

The first restriction is that this procedure cannot result in the inheritance of multiple, conflicting default values for attributes. This would occur if a highest precedence default value was being inherited through the ancestor, and upon completion of this operation other generalizations of the descendant would provide two or more differing default values with none being highest precedence. This restriction applies at the RD identified by DESCENDANT, without reference to the descendant's specializations.

The second restriction is that this procedure cannot result in the deletion of a property of which instances exist. This restriction applies as follows:

1. There is a set of properties defined for the descendant RD by the SPECIALIZATION_OF relationship to be deleted. This set consists of those properties not defined by the descendant itself and not inherited through generalizations other than that identified by ANCESTOR. If there are instances of the RD identified by DESCENDANT, they must have none of the properties in the given set.
2. For each of the specializations of the descendant, the set of properties specified above is reduced by any properties defined by the specialized definition. If there are instances of that definition, they must have none of the properties in the given set.
3. The above rule applies recursively, further reducing the set of properties under consideration for all specializations.

Eliminating an ancestor may eliminate some inherited constraints on remaining properties. There are no restrictions on weakening constraints, and so there are no restrictions on the use of this procedure relating to inherited constraints.

Parameters:

**DESCENDANT** is an open node handle identifying the specialization from which the SPECIALIZATION_OF relationship to be deleted emanates.

**ANCESTOR** is an open node handle identifying the generalization at which the SPECIALIZATION_OF relationship to be deleted terminates.

**STATUS** is an established status indicator.

Exceptions:

**RELATIONSHIP_ERROR** is raised if there is no SPECIALIZATION_OF relationship connecting the specified RDs [4].

**USE_ERROR** is raised if the relationship to be deleted is predefine [63] or if a SPECIALIZATION_OF relationship to be deleted is the only one emanating from the source node [21].

**STATUS_ERROR** is raised if the node handle DESCENDANT or ANCESTOR is not open [2] or if STATUS is not an established status indicator [3].

**NODE_KIND_ERROR** is raised if DESCENDANT or ANCESTOR does not identify an RD [1].

**CONFORMANCE_ERROR** is raised if the operation would result in an attribute, node or relationship not conforming to its definition [3].


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

**DEFINITION_ERROR** is raised if a restriction on the deletion of an inherited property would be violated [2] or if this operation results in the inheritance of differing default values not resolved by precedence [3]. 19

**LOCK_ERROR** is raised if a node handle to the reverse RD for either RD (if any) cannot be opened with intent WRITE_RELATIONSHIPS due to outstanding locks in conflict with that intent [1]. 20

**TRANSACTION_ERROR** is raised if the operation is included in more than one transaction [1] or if the operation is included in a transaction and the operation modifies a node which is not included in the same transaction [2]. 21

**INTENT_VIOLATION** is raised if the node handle DESCENDANT or ANCESTOR was not opened with an intent establishing the rights to read and write relationships [1]. 22

**ACCESS_VIOLATION** is raised if the discretionary access control rights of the current process are insufficient to allow access with READ_RELATIONSHIPS and WRITE_RELATIONSHIPS intents to either RD's reverse RD (if any) [1]. 23

**LOCK_ERROR** is raised if an open transaction not containing this operation contains other eligible operations whose effects determine whether the conditions for DEFINITION_ERROR apply [3]. 24

**SECURITY_VIOLATION** is raised if the operation represents a violation of mandatory access controls. SECURITY_VIOLATION is raised only if the conditions for other exceptions are not present. 25

### 5.7.29 Creating bidirectional RDs

```
procedure   CREATE_BIDIRECTIONAL_RELATION
    (RI1                              : in out  RI_TYPE;
     RI2                              : in out  RI_TYPE;
     RD1                              : in out  RD_TYPE;
     RD2                              : in out  RD_TYPE;
     VIEW                             : in      VIEW_NODE_TYPE;
     SPECIALIZATION_OF1               : in      RD_TYPE;
     SPECIALIZATION_OF2               : in      RD_TYPE;
     BASE1                            : in      NODE_TYPE;
     BASE2                            : in      NODE_TYPE;
     LOCAL_NAME1                      : in      RELATIONSHIP_KEY;
     LOCAL_NAME2                      : in      RELATIONSHIP_KEY;
     KEY1                             : in      RELATIONSHIP_KEY   := EMPTY_KEY_LIST;
     KEY2                             : in      RELATIONSHIP_KEY   : = EMPTY_KEY_LIST;
     SPECIALIZATION_OF1_KEY           : in      RELATIONSHIP_KEY   : = EMPTY_KEY_LIST;
     SPECIALIZATION_OF2_KEY           : in      RELATIONSHIP_KEY   : = EMPTY_KEY_LIST;
     GENERALIZATION_OF1_KEY           : in      RELATIONSHIP_KEY   : = EMPTY_KEY_LIST;
     GENERALIZATION_OF2_KEY           : in      RELATIONSHIP_KEY   := EMPTY_KEY_LIST;
     INTERPRETATION_ATTRIBUTES1 : in    ATTRIBUTE_LIST     := EMPTY_LIST;
     INTERPRETATION_ATTRIBUTES2 : in    ATTRIBUTE_LIST     := EMPTY_LIST;
     DEFINITION_ATTRIBUTES1           : in      ATTRIBUTE_LIST     := EMPTY_LIST;
     DEFINITION_ATTRIBUTES2           : in      ATTRIBUTE_LIST     := EMPTY_LIST;
     DISCRETIONARY_ACCESS             : in      DISCRETIONARY_ACCESS_LIST := ALL_RIGHTS;
```


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
        MANDATORY_ACCESS          : in    MANDATORY_ACCESS_LIST      := EMPTY_LIST;
        INTERPRETATION_KIND1      : in    NODE_KIND_NODE_TYPE        :=
                                              RELATION_INTERPRETATION_NKD;
        INTERPRETATION_KIND2      : in    NODE_KIND_NODE_TYPE        :=
                                              RELATION_INTERPRETATION_NKD;
        DEFINITION_KIND1          : in    NODE_KIND_NODE_TYPE        :=
                                              RELATION_DEFINITION_NKD;
        DEFINITION_KIND2          : in    NODE_KIND_NODE_TYPE        :=
                                              RELATION_DEFINITION_NKD;
        STATUS                    : in    STATUS_INDICATOR := NO_INDICATION;
        INTERPRETATION_MEDIUM     : in    MEDIUM_NODE_TYPE   := DEFAULT_MEDIUM;
        DEFINITION_MEDIUM         : in    MEDIUM_NODE_TYPE   := DEFAULT_MEDIUM);

    procedure  CREATE_BIDIRECTIONAL_RELATION
        (RI                       : in out RI_TYPE;
         RD                       : in out RD_TYPE;
         VIEW                     : in    VIEW_NODE_TYPE;
         SPECIALIZATION_OF        : in    RD_TYPE;
         BASE                     : in    NODE_TYPE;
         LOCAL_NAME               : in    RELATIONSHIP_KEY;
         KEY                      : in    RELATIONSHIP_KEY   := EMPTY_KEY_LIST;
         SPECIALIZATION_OF_KEY    : in    RELATIONSHIP_KEY   := EMPTY_KEY_LIST;
         GENERALIZATION_OF_KEY    : in    RELATIONSHIP_KEY   := EMPTY_KEY_LIST;
         INTERPRETATION_ATTRIBUTES : in   ATTRIBUTE_LIST     := EMPTY_LIST;
         DEFINITION_ATTRIBUTES    : in    ATTRIBUTE_LIST     := EMPTY_LIST;
         DISCRETIONARY_ACCESS     : in    DISCRETIONARY_ACCESS_LIST := ALL_RIGHTS;
         MANDATORY_ACCESS         : in    MANDATORY_ACCESS_LIST      := EMPTY_LIST;
         INTERPRETATION_KIND      : in    NODE_KIND_NODE_TYPE        :=
                                              RELATION_INTERPRETATION_NKD;
         DEFINITION_KIND          : in    NODE_KIND_NODE_TYPE        :=
                                              RELATION_DEFINITION_NKD;
         STATUS                   : in    STATUS_INDICATOR := NO_INDICATION;
         INTERPRETATION_MEDIUM    : in    MEDIUM_NODE_TYPE   := DEFAULT_MEDIUM;
         DEFINITION_MEDIUM        : in    MEDIUM_NODE_TYPE   := DEFAULT_MEDIUM);
```

Purpose:

This procedure creates a new bidirectional relation. New definition and interpretation nodes for bidirectional relationships are created. The first form creates a bidirectional relation definition represented by two distinct RDs, for bidirectional relationships whose two unidirectional components are semantically distinct. The second form creates a bidirectional relation definition represented by a single RD, for bidirectional relationships whose two unidirectional components are semantically identical. 1

These procedures create RDs and install the primary RELATION_DEFINITION relationships to them, as well as the corresponding reverse relationships to the BASE nodes. The primary relationship key lists are given by the KEY parameters. Open node handles to the newly created RDs with WRITE intent are returned in the RD parameters. 2


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

In the first form, a bidirectional REVERSE_LINK relationship is created between the new RDs, indicating that together they comprise a bidirectional relation definition.

In the second form, a unidirectional REVERSE_LINK relationship is created emanating from and terminating at the one new RD, indicating that it alone comprises a bidirectional relation definition.

SPECIALIZATION_OF relationships are created from the new definition nodes to the definition nodes identified by the SPECIALIZATION_OF parameters. The SPECIALIZATION_OF_KEY and GENERALIZATION_OF_KEY parameters provide the relationship key lists for this relationship and the corresponding reverse relationship.

ATTRIBUTE relationships with key values INHERITABLE are created from the RDs to the INHERITABLE AD. Default value (FALSE) and constraints (VALUE=>FALSE) attributes are created on the relationship, designating that instances of the new bidirectional relation are never inherit able.

These procedures also create RIs and install the primary RELATION relationships to them, as well as the corresponding reverse relationships to the VIEW node identified by VIEW. The primary relationship key lists are given by the LOCAL_NAME parameters.

The INTERPRETATION_ATTRIBUTES and DEFINITION_ATTRIBUTES parameters provide initial values for attributes of the new RIs and RDs, respectively. The INTERPRETATION_ATTRIBUTES parameters cannot specify values for any CAIS-controlled attributes. The DEFINITION_ATTRIBUTES parameters may include values for the optional, CAIS-controlled RD attributes ALLOWED_USAGE and UNIQUE_KEYS.

The MANDATORY_ACCESS parameter specifies the object classification labels with which the nodes are to be created. If its value is the empty list, the nodes inherit the subject classification of the creating process as their object classification (see Table IX on page 256).

The INTERPRETATION_KIND and DEFINITION_KIND parameters are open node handles identifying the node kinds for the new nodes. They must identify RI and RD kinds, respectively. Each may specify a kind either through an NKD or an NKI. If an INTERPRETATION_KIND parameter identifies an NKI, the NKI is made the current adopted interpretation for the corresponding RI node handle. If a DEFINITION_KIND parameter identifies an NKI, the NKI is made the current adopted interpretation for the corresponding RD node handle.

The INTERPRETATION_MEDIUM parameter specifies the medium resource on which the new RIs will reside. The DEFINITION_MEDIUM parameter specifies the medium resource on which the new RDs will reside.

Parameters:

**RI** parameters are closed node handles to be opened to the newly created RIs.

**RD** parameters are closed node handles to be opened to the newly created RDs.

**VIEW** is an open node handle to the VIEW node from which the primary relationships to the new RIs are to emanate.

**SPECIALIZATION_OF** parameters are open node handles identifying the RDs of which the new definitions are to be specializations.

**BASE** parameters are open node handles to the nodes from which the primary relationships to the new RDs are to emanate.


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

**LOCAL_NAME** parameters are key lists for the primary RELATION relationships to be created to the new RIs.

**KEY** parameters are key lists for the primary RELATION_DEFINITION relationships to be created to the new RDs.

**SPECIALIZATION_OF_KEY** parameters are key lists of the SPECIALIZATION_OF relationships to be created from the RDs.

**GENERALIZATION_OF_KEY** parameters are key lists of the GENERALIZATION_OF relationships to be created to the RDs.

**INTERPRETATION_ATTRIBUTES** parameters are empty or named lists whose elements are used to establish initial values for attributes of the newly created RIs; each named item of the list specifies an attribute name and the value to be given to that attribute.

**DEFINITION_ATTRIBUTES** parameters are empty or named lists whose elements are used to establish initial values for attributes of the the newly created RDs; each named item of the list specifies an attribute name and the value to be given to that attribute.

**DISCRETIONARY_ACCESS** is the initial access control information associated with the created nodes; it is the value of the GRANT attributes of the ACCESS relationships to the user's default GROUP node.

**MANDATORY_ACCESS** is a list defining the classification labels for the created nodes.

**INTERPRETATION_KIND** parameters are open node handles identifying the node kind for the RIs to be created.

**DEFINITION_KIND** parameters are open node handles identifying the node kind for the RDs to be created.

**STATUS** is an established status indicator.

**INTERPRETATION_MEDIUM** is an open node handle identifying a MEDIUM node.

**DEFINITION_MEDIUM** is an open node handle identifying a MEDIUM node.

Exceptions:

**PATHNAME_SYNTAX_ERROR** is raised if the node identification given by a LOCAL_NAME, KEY, SPECIALIZATION_OF_KEY or GENERALIZATION_OF_KEY parameter is syntactically illegal [2].

**NAME_ERROR** is raised if the parent of the NKI identified by an INTERPRETATION_KIND or DEFINITION_KIND parameter is inaccessible [1].

**EXISTING_NODE_ERROR** is raised if a primary relationship already exists with an identification given [1].

**RELATIONSHIP_ERROR** is raised if a secondary relationship already exists with an identification given [3] or if a key list specifies a value for the same key more than once [5].

**ATTRIBUTE_ERROR** is raised if an attribute list specifies a value for the same attribute more than once [6].

**NODE_KIND_ERROR** is raised if an open INTERPRETATION_KIND node handle does not identify an RI kind [2], if an open DEFINITION_KIND node handle does not identify an RD kind [2], if the open node handle VIEW does not identify a VIEW node [1], if an open SPECIALIZATION_OF node handle does not identify an RD [1] or if the open node handle INTERPRETATION_MEDIUM or DEFINITION_MEDIUM does not identify a MEDIUM node [1].



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

**NOT_IN_VIEW_ERROR** is raised if an attribute name in a LOCAL_NAME, KEY, SPECIALIZATION_OF_KEY, GENERALIZATION_OF_KEY, INTERPRETATION_ATTRIBUTES or DEFINITION_ATTRIBUTES parameter cannot be translated [1] or if LOCAL_NAME, KEY, SPECIALIZATION_OF_KEY or GENERALIZATION_OF_KEY consists of a single relationship key designator and no key of the corresponding relation is visible [5]. 36

**AMBIGUOUS_VIEW_ERROR** is raised if an attribute name in a LOCAL_NAME, KEY, SPECIALIZATION_OF_KEY, GENERALIZATION_OF_KEY, INTERPRETATION_ATTRIBUTES or DEFINITION_ATTRIBUTES parameter has an ambiguous translation [1] or if LOCAL_NAME, KEY, SPECIALIZATION_OF_KEY or GENERALIZATION_OF_KEY consists of a single relationship key designator and more than one key of the corresponding relation is visible [4]. 37

**SYNTAX_ERROR** is raised if an INTERPRETATION_ATTRIBUTES, DEFINITION_ATTRIBUTES, DISCRETIONARY_ACCESS or MANDATORY_ACCESS parameter is syntactically illegal [8-10]. 38

**PREDEFINED_ATTRIBUTE_ERROR** is raised if any attribute named in the INTERPRETATION_ATTRIBUTES or DEFINITION_ATTRIBUTES parameters is a CAIS-controlled attribute that is not identified in the Purpose section of this interface as an attribute which may be specified in that parameter [1]. 39

**USE_ERROR** is raised if the value for the MANDATORY_ACCESS parameter is semantically illegal [10] or if one of two generalizations specified is bidirectional and the other is not its reverse relation [19]. 40

**STATUS_ERROR** is raised if a VIEW, SPECIALIZATION_OF, BASE, INTERPRETATION_KIND, DEFINITION_KIND, INTERPRETATION_MEDIUM or DEFINITION_MEDIUM node handle is not open [2], if an RI or RD node handle is open prior to the call [1] or if STATUS is not an established status indicator [3]. 41

**CONFORMANCE_ERROR** is raised if the operation would result in an attribute, node or relationship not conforming to its definition or the constraints in force on that definition [1,3,7,8,9,10]. 42

**LOCK_ERROR** is raised if a node handle to the parent of an NKI specified by an INTERPRETATION_KIND or DEFINITION_KIND parameter cannot be opened with intent READ_RELATIONSHIPS due to outstanding locks in conflict with that intent [1]. 43

**TRANSACTION_ERROR** is raised if the operation is included in more than one transaction [1] or if the operation is included in a transaction and the operation modifies a node which is not included in the same transaction [2]. 44

**INTENT_VIOLATION** is raised if the SPECIALIZATION_OF and BASE node handles were not opened with an intent establishing the right to create relationships [1] or if the VIEW node handle was not opened with an exclusive intent establishing the right to create relationships [1]. 45

**ACCESS_VIOLATION** is raised if the discretionary access control rights of the current process are insufficient to allow access with CREATE_INSTANCES intents to one of the NKDs for the kinds of nodes to be created or with READ_RELATIONSHIPS intents to the parent of an NKI specified by an INTERPRETATION_KIND or DEFINITION_KIND parameter or with APPEND_CONTENTS intents to either MEDIUM node [1]. 46

**RESOURCE_ALLOCATION_ERROR** is raised if either medium resource is not allocatable [1]. 47


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

**RESOURCE_AVAILABILITY_ERROR** is raised if either medium resource is not available [1]. 48

**SECURITY_VIOLATION** is raised if the operation represents a violation of mandatory access controls. SECURITY_VIOLATION is raised only if the conditions for other exceptions are not present. 49

Additional Interfaces:

```
procedure CREATE_BIDIRECTIONAL_RELATION
    (RI1                          : in out RI_TYPE;
     RI2                          : in out RI_TYPE;
     RD1                          : in out RD_TYPE;
     RD2                          : in out RD_TYPE;
     VIEW                         : in     VIEW_NODE_TYPE;
     SPECIALIZATION_OF1           : in     RD_TYPE;
     SPECIALIZATION_OF2           : in     RD_TYPE;
     BASE1                        : in     NODE_TYPE;
     BASE2                        : in     NODE_TYPE;
     LOCAL_NAME1                  : in     RELATIONSHIP_KEY;
     LOCAL_NAME2                  : in     RELATIONSHIP_KEY;
     KEY1                         : in     RELATIONSHIP_KEY      := EMPTY_KEY_LIST;
     KEY2                         : in     RELATIONSHIP_KEY      := EMPTY_KEY_LIST;
     SPECIALIZATION_OF1_KEY       : in     RELATIONSHIP_KEY      := EMPTY_KEY_LIST;
     SPECIALIZATION_OF2_KEY       : in     RELATIONSHIP_KEY      := EMPTY_KEY_LIST;
     GENERALIZATION_OF1_KEY       : in     RELATIONSHIP_KEY      := EMPTY_KEY_LIST;
     GENERALIZATION_OF2_KEY       : in     RELATIONSHIP_KEY      := EMPTY_KEY_LIST;
     INTERPRETATION_ATTRIBUTES1 : in     ATTRIBUTE_LIST        := EMPTY_LIST;
     INTERPRETATION_ATTRIBUTES2 : in     ATTRIBUTE_LIST        := EMPTY_LIST;
     DEFINITION_ATTRIBUTES1       : in     ATTRIBUTE_LIST        := EMPTY_LIST;
     DEFINITION_ATTRIBUTES2       : in     ATTRIBUTE_LIST        := EMPTY_LIST;
     DISCRETIONARY_ACCESS         : in     DISCRETIONARY_ACCESS_LIST := ALL_RIGHTS;
     MANDATORY_ACCESS             : in     MANDATORY_ACCESS_LIST := EMPTY_LIST;
     INTERPRETATION_KIND1         : in     NODE_KIND_NAME;
     INTERPRETATION_KIND2         : in     NODE_KIND_NAME        :=
                                               "RELATION_INTERPRETATION";
     DEFINITION_KIND1             : in     NODE_KIND_NAME        :=
                                               "RELATION_DEFINITION";
     DEFINITION_KIND2             : in     NODE_KIND_NAME        :=
                                               "RELATION_DEFINITION";
     STATUS                       : in     STATUS_INDICATOR   := NO_INDICATION;
     INTERPRETATION_MEDIUM        : in     MEDIUM_NODE_TYPE   := DEFAULT_MEDIUM;
     DEFINITION_MEDIUM            : in     MEDIUM_NODE_TYPE   := DEFAULT_MEDIUM)
is
    INTERPRETATION_NODE_KIND1 : NKI_TYPE;
    -- "RELATION_INTERPRETATION" or a specialization.
    INTERPRETATION_NODE_KIND2 : NKI_TYPE;
    DEFINITION_NODE_KIND1  : NKI_TYPE;
    DEFINITION_NODE_KIND2  : NKI_TYPE;
```


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
begin
    OPEN_NODE_KIND (INTERPRETATION_NODE_KIND1, INTERPRETATION_KIND1, STATUS);
    OPEN_NODE_KIND (INTERPRETATION_NODE_KIND2, INTERPRETATION_KIND2, STATUS);
    OPEN_NODE_KIND  (DEFINITION_NODE_KIND1,  DEFINITION_KIND1,  STATUS);
    OPEN_NODE_KIND  (DEFINITION_NODE_KIND2,  DEFINITION_KIND2,  STATUS);
    CREATE_BIDIRECTIONAL_RELATION
        (RI1,
         RI2,
         RD1,
         RD2,
         VIEW,
         SPECIALIZATION_OF1,
         SPECIALIZATION_OF2,
         BASE1,
         BASE2,
         LOCAL_NAME1,
         LOCAL_NAME2,
         KEY1,
         KEY2,
         SPECIALIZATION_OF1_KEY,
         SPECIALIZATION_OF2_KEY,
         GENERALIZATION_OF1_KEY,
         GENERALIZATION_OF2_KEY,
         INTERPRETATION_ATTRIBUTES1,
         INTERPRETATION_ATTRIBUTES2,
         DEFINITION_ATTRIBUTES1,
         DEFINITION_ATTRIBUTES2,
         DISCRETIONARY_ACCESS,
         MANDATORY_ACCESS,
         INTERPRETATION_NODE_KIND1,
         INTERPRETATION_NODE_KIND2,
         DEFINITION_NODE_KIND1,
         DEFINITION_NODE_KIND2,
         STATUS,
         INTERPRETATION_MEDIUM,
         DEFINITION_MEDIUM);
    CLOSE (INTERPRETATION_NODE_KIND1);
    CLOSE (INTERPRETATION_NODE_KIND2);
    CLOSE  (DEFINITION_NODE_KIND1);
    CLOSE  (DEFINITION_NODE_KIND2);
exception
    when  others  =>
        CLOSE (INTERPRETATION_NODE_KIND1);
        CLOSE (INTERPRETATION_NODE_KIND2);
        CLOSE  (DEFINITION_NODE_KIND1);
        CLOSE  (DEFINITION_NODE_KIND2);
        raise;
end CREATE_BIDIRECTIONAL_RELATION;
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
procedure   CREATE_BIDIRECTIONAL_RELATION
    (RI                          : in out RI_TYPE;
     RD                          : in out RD_TYPE;
     VIEW                        : in     VIEW_NODE_TYPE;
     SPECIALIZATION_OF           : in     RD_TYPE;
     BASE                        : in     NODE_TYPE;
     LOCAL_NAME                  : in     RELATIONSHIP_KEY;
     KEY                         : in     RELATIONSHIP_KEY        := EMPTY_KEY_LIST;
     SPECIALIZATION_OF_KEY       : in     RELATIONSHIP_KEY        := EMPTY_KEY_LIST;
     GENERALIZATION_OF_KEY       : in     RELATIONSHIP_KEY        := EMPTY_KEY_LIST;
     INTERPRETATION_ATTRIBUTES   : in     ATTRIBUTE_LIST          := EMPTY_LIST;
     DEFINITION_ATTRIBUTES       : in     ATTRIBUTE_LIST          := EMPTY_LIST;
     DISCRETIONARY_ACCESS        : in     DISCRETIONARY_ACCESS_LIST := ALL_RIGHTS;
     MANDATORY_ACCESS            : in     MANDATORY_ACCESS_LIST := EMPTY_LIST;
     INTERPRETATION_KIND         : in     NODE_KIND_NAME;
     DEFINITION_KIND             : in     NODE_KIND_NAME          :=
                                              "RELATION_DEFINITION";
     STATUS                      : in     STATUS_INDICATOR        := NO_INDICATION;
     INTERPRETATION_MEDIUM       : in     MEDIUM_NODE_TYPE        := DEFAULT_MEDIUM;
     DEFINITION_MEDIUM           : in     MEDIUM_NODE_TYPE        := DEFAULT_MEDIUM)
is
    INTERPRETATION_NODE_KIND : NKI_TYPE;
    -- "RELATION_INTERPRETATION" or a specialization.
    DEFINITION_NODE_KIND : NKI_TYPE;


begin
    OPEN_NODE_KIND (INTERPRETATION_NODE_KIND, INTERPRETATION_KIND, STATUS);
    OPEN_NODE_KIND (DEFINITION_NODE_KIND, DEFINITION_KIND, STATUS);
    CREATE_BIDIRECTIONAL_RELATION
        (RI,
         RD,
         VIEW,
         SPECIALIZATION_OF,
         BASE,
         LOCAL_NAME,
         KEY,
         SPECIALIZATION_OF_KEY,
         GENERALIZATION_OF_KEY,
         INTTERPRETATION_ATTRIBUTES,
         DEFINITION_ATTRIBUTES,
         DISCRETIONARY_ACCESS,
         MANDATORY_ACCESS,
         INTERPRETATION_NODE_KIND,
         DEFINITION_NODE_KIND,
         STATUS,
         INTERPRETATION_MEDIUM,
         DEFINITION_MEDIUM);
    CLOSE (INTERPRETATION_NODE_KIND);
```


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
    CLOSE (DEFINITION_NODE_KIND);
exception
    when others =>
        CLOSE (INTERPRETATION_NODE_KIND);
        CLOSE (DEFINITION_NODE_KIND);
        raise;
end CREATE_BIDIRECTIONAL_RELATION;
```

### 5.7.30 Deleting bidirectional RDs

```
procedure DELETE_BIDIRECTIONAL_RELATION
    (RD                  : in out RD_TYPE;
     VIEW                : in     VIEW_NODE_TYPE;
     DELETE_DEFINITION   : in     BOOLEAN             := FALSE;
     TIME_LIMIT          : in     CAIS_DURATION       := LONG_DELAY;
     STATUS              : in     STATUS_INDICATOR    := NO_INDICATION);
```

Purpose:

This procedure deletes a relation interpretation for a bidirectional relation from a view, and optionally deletes the corresponding bidirectional relation definition. 1

Effectively, the DELETE_NODE operation is performed for the RI in the view identified by the open node handle VIEW and corresponding to the RD identified by the open handle RD. If there is an RI in the view for the RD's reverse relation, it is deleted as well. The RIs become unobtainable. All ATTRIBUTE_NAME, KEY_NAMEand DEFINITION relationships emanating from the RIs are deleted; all RELATION_NAME relationships terminating at the RIs are deleted. The TIME_-LIMIT parameter allows specification of a time limit for the delay imposed on the DELETE_BI-DIRECTIONAL_RELATION operation by the existence of locks on any node to be accessed. A delayed call completes after the node is unlocked or the specified time limit has elapsed. In the latter case, the exception LOCK_ERROR is raised. 2

If the value of DELETE_DEFINITION is TRUE, then this procedure also effectively performs the DELETE_NODE operation on the RDs. One RD is identified by the open node handle RD. The other (if distinct) is identified by the REVERSE_LINK relationship emanating from the fist. The restrictions in Section 5.7.1.9 on page 335 on deletion of RDs apply. 3

If the DELETE_BIDIRECTIONAL_RELATION operation raises an exception, no relationship is deleted. 4

Parameters:

**RD** is an open node handle identifying an RD which was created by CREATE_BIDIRECTI-ONAL_RELATION whose corresponding RI is to be deleted. 5

**VIEW** is an open node handle to the VIEW node from which the primary RELATION rela-tionships to be deleted emanate. 6

**DELETE_DEFINITION** is a Boolean. If TRUE, the primary relationships to the RDs are to be deleted. Otherwise, this operation has no effect on the RDs. 7


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

**TIME_LIMIT** is a time limit for the delay on waiting for the unlocking of a node. 8

**STATUS** is an established status indicator. 9

Exceptions:

**NAME_ERROR** is raised if either RI or either parent of an RD whose primary relationship is to be deleted or the source node of a secondary relationship to be deleted is inaccessible [1]. 10

**RELATIONSHIP_ERROR** is raised if the view does not contain an RI corresponding to the RD [2]. 11

**USE_ERROR** is raised if any primary relationships emanate from the node to be deleted [7], if a node to be deleted is predefine [62] or if the relation identified is unidirectional [11]. 12

**STATUS_ERROR** is raised if the node handle RD or VIEW is not open prior to the call [2] or if STATUS is not an established status indicator [3]. 13

**NODE_KIND_ERROR** is raised if RD does not identify an RD [1] or if VIEW does not identify a VIEW node [1]. 14

**CONFORMANCE_ERROR** is raised if the operation would result in an attribute, node or relationship not conforming to its definition or the constraints in force on that definition [3]. 15

**DEFINITION_ERROR** is raised if the value of DELETE_DEFINITION is TRUE and a restriction on the deletion of an RD would be violated [1] or if the value of DELETE_DEFINITION is FALSE and view contains the last interpretation node corresponding to the definition or its reverse relation [6]. 16

**LOCK_ERROR** is raised if a node handle identifying the source node of a relationship to be deleted cannot be opened with intents WRITE_RELATIONSHIPS [1], if DELETE_DEFINITION is TRUE and a node handle identifying the reverse RD cannot be opened with intent EXCLUSIVE_WRITE [1], if a node handle identifying either RI cannot be opened with intent EXCLUSIVE_WRITE due to outstanding locks [1] or if an open transaction not containing this operation contains other eligible operations whose effects determine whether the conditions for DEFINITION_ERROR apply [3]. 17

**TRANSACTION_ERROR** is raised if the operation is included in more than one transaction [1] or if the operation is included in a transaction and the operation modifies a node which is not included in the same transaction [2]. 18

**INTENT_VIOLATION** is raised if the node handle VIEW was not opened with an intent establishing the right to READ_RELATIONSHIPS and an exclusive intent establishing the right to WRITE_RELATIONSHIPS [1] or if DELETE_DEFINITION is TRUE and RD was not opened with intents READ_RELATIONSHIPS and EXCLUSIVE_WRITE [1]. 19

**ACCESS_VIOLATION** is raised if the current process does not have sufficient discretionary access control rights to obtain access to the source node of a relationship to be deleted with intents WRITE_RELATIONSHIPS or to obtain access to the target node of a primary relationship to be deleted with intent EXCLUSIVE_WRITE or to obtain access to an NKI whose parent is the VIEW node with intents READ_RELATIONSHIPS and WRITE_RELATIONSHIPS [1]. 20

**SECURITY_VIOLATION** is raised if the operation represents a violation of mandatory access controls. SECURITY_VIOLATION is raised only if the conditions for other exceptions are not present. 21


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

### 5.7.31 Creating ADs

```
procedure CREATE_ATTRIBUTE_DEFINITION
    (AD                    : in out AD_TYPE;
     VIEW                  : in     VIEW_NODE_TYPE;
     BASE                  : in     NODE_TYPE;
     ATTRIBUTE_KIND        : in     AKD_TYPE;
     LOCAL_NAME            : in     RELATIONSHIP_KEY;
     KEY                   : in     RELATIONSHIP_KEY          := EMPTY_KEY_LIST;
     ATTRIBUTE_KIND_KEY    : in     RELATIONSHIP_XEY          := EMPTY_KEY_LIST;
     ATTRIBUTES            : in     ATTRIBUTE_LIST            := EMPTY-LIST;
     DISCRETIONARY_ACCESS : in     DISCRETIONARY_ACCESS_LIST := ALL_RIGHTS;
     MANDATORY_ACCESS      : in     MANDATORY_ACCESS_LIST := EMPTY_LIST;
     KIND                  : in     NODE_KIND_NODE_TYPE :=
                                        ATTRIBUTE_DEFINITION_NKD;
     STATUS                : in     STATUS_INDICATOR          := NO_INDICATION;
     MEDIUM_RESOURCE       : in     MEDIUM_NODE_TYPE          := DEFAULT_MEDIUM);
```

Purpose:

This procedure creates a new attribute definition and specifies the AKD which defines the allowed values of attributes conforming to the new AD. The primary ATTRIBUTE_DEFINITION relationship to the AD is installed, as well as the corresponding reverse relationship to the node identified by BASE. KEY specifies the primary relationship key list. An open node handle to the new AD with WRITE intent is returned in AD. 1

This procedure installs a bidirectional ATTRIBUTE_KIND relationship from the new AD to the AKD identified by ATTRIBUTE_KIND_ATTRIBUTE_KIND_KEY specifies the key list used in creating the corresponding reverse relationship. 2

This procedure also installs an ATTRIBUTE relationship from the VIEW node identified by VIEW to the new AD. The relationship key list is given by LOCAL_NAME. 3

The ATTRIBUTES parameter provides initial values for attributes of the new node. ATTRIBUTES may include values for the required, CAIS-controlled AD attributes DELETABLE and MODIFIABLE. 4

The MANDATORY_ACCESS parameter specifies the object classification label with which the AD is to be created. If its value is the empty list, the node inherits the subject classification of the creating process as its object classification. 5

KIND is an open node handle identifying the node kind for the new node. It must identify an AD kind. It may specify either an NKD or an NKI. If KIND identifies an NKI, the NKI is made the current adopted interpretation for the node handle AD. 6

The MEDIUM_RESOURCE parameter specifies the medium resource on which the new AD will reside. 7

Parameters:

AD is a closed node handle to be opened to the new AD. 8



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

**VIEW** is an open node handle to the VIEW node from which the secondary ATTRIBUTE_-DEFINITION relationship to the new AD is to emanate.

**BASE** is an open node handle to the source node of the primary ATTRIBUTE_DEFINITION relationship to the new AD.

**ATTRIBUTE_KIND** is an open node handle identifying the AKD which defines the allowed values for the attribute defined by the new AD.

**LOCAL_NAME** is the key list of the secondary ATTRIBUTE relationship from the VIEW node to the new AD.

**KEY** is the key list of the primary ATTRIBUTE_DEFINITION relationship to the new AD.

**ATTRIBUTE_KIND_KEY** is the key list of the ATTRIBUTE_KIND_OF relationship from the AKD to the new AD.

**ATTRIBUTES** is an empty or named list whose elements are used to establish the initial values for attributes of the new AD; each named item of the list specifies an attribute name and the value to be given to that attribute.

**DISCRETIONARY_ACCESS** is the initial access control information associated with the new AD; it is the value of the GRANT attribute of the ACCESS relationship to the user's default GROUP node.

**MANDATORY_ACCESS** is a list defining the classification label for the new AD.

**KIND** is an open node handle identifying the node kind for the new AD.

**STATUS** is an established status indicator.

**MEDIUM_RESOURCE** is an open node handle identifying a MEDIUM node.

Exceptions:

**PATHNAME_SYNTAX_ERROR** is raised if the node identification given by LOCAL_-NAME, KEY or ATTRIBUTE_KIND_KEY is syntactically illegal [2].

**NAME_ERROR** is raised if the parent of the NKI identified by KIND is inaccessible [1].

**EXISTING_NODE_ERROR** is raised if a primary relationship already exists with the identification given [1].

**RELATIONSHIP_ERROR** is raised if' a secondary relationship already exists with an identification given [3] or if a key list specifies a value for the same key more than once [5].

**ATTRIBUTE_ERROR** is raised if an attribute list specifies a value for the same attribute more than once [6].

**NODE_KIND_ERROR** is raised if the open node handle KIND does not identify an AD kind [2], if the open node handle VIEW does not identify a VIEW node [1], if the open handle ATTRIBUTE_KIND does not identify an AKD [1] or if the open node handle MEDIUM_-RESOURCE does not identify a MEDIUM node [2].

**NOT_IN_VIEW_ERROR** is raised if an attribute name specified in LOCAL_NAME, KEY, ATTRIBUTE_ KIND_ KEY or ATTRIBUTES cannot be translated [1] or if LOCAL_NAME, KEY or ATTRIBUTE_KIND_KEY consists of a single relationship key designator and no key of the corresponding relation is visible [5].

**AMBIGUOUS_VIEW_ERROR** is raised if an attribute name specified in LOCAL_NAME, KEY, ATTRIBUTE_KIND_KEY or ATTRIBUTES has an ambiguous translation [1] or if LOCAL_NAME, KEY or ATTRIBUTE_KIND_KEY consists of a single relationship key designator and more than one key of the corresponding relation is visible [4].



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

**SYNTAX_ERROR** is raised if the ATTRIBUTES parameter, the DISCRETIONARY_ACCESS parameter or the MANDATORY_ACCESS parameter is syntactically illegal [8-10]. 29

**PREDEFINED_ATTRIBUTE_ERROR** is raised if any attribute named in the ATTRIBUTES parameter is a CAIS-controlled attribute that is not identified in the Purpose section of this interface as an attribute which may be specified in that parameter [1]. 30

**USE_ERROR** is raised if the value for the MANDATORY_ACCESS parameter is semantically illegal [10]. 31

**STATUS_ERROR** is raised if VIEW, ATTRIBUTE_KIND, BASE, KIND or MEDIUM_RESOURCE is not an open node handle [2], if AD is an open node handle prior to the call [1] or if STATUS is not an established status indicator [3]. 32

**CONFORMANCE_ERROR** is raised if the operation would result in an attribute, node or relationship not conforming to its definition or the constraints in force on that definition [1,3,7,8,9,10]. 33

**LOCK_ERROR** is raised if a node handle to the parent of the NKI specified by KIND cannot be opened with intent READ_RELATIONSHIPS due to outstanding locks in conflict with that intent [1]. 34

**TRANSACTION_ERROR** is raised if the operation is included in more than one transaction [1] or if the operation is included in a transaction and the operation modifies a node which is not included in the same transaction [2]. 35

**INTENT_VIOLATION** is raised if ATTRIBUTE_KIND and BASE were not opened with an intent establishing the right to create relationships [1] or if VIEW was not opened with an exclusive intent establishing the right to write relationships [1]. 36

**ACCESS_VIOLATION** is raised if the discretionary access control rights of the current process are insufficient to allow access with CREATE_INSTANCES intents to the NKD identified by KIND or with READ_RELATIONSHIPS intents to the parent of the NKI specified by KIND or with APPEND_CONTENTS intents to the MEDIUM node identified by MEDIUM_RESOURCE [1]. 37

**RESOURCE_ALLOCATION_ERROR** is raised if the medium resource is not allocatable [1]. 38

**RESOURCE_AVAILABILITY_ERROR** is raised if the medium resource is not available [1]. 39

**SECURITY_VIOLATION** is raised if the operation represents a violation of mandatory access controls. SECURITY_VIOLATION is raised only if the conditions for other exceptions are not present. 40

Additional Interfaces:

```
procedure CREATE_ATTRIBUTE_DEFINITION
    (AD                    : in out AD_TYPE;
     VIEW                  : in     VIEW_NODE_TYPE;
     BASE                  : in     NODE_TYPE;
     ATTRIBUTE_KIND        : in     ATTRIBUTE_KIND_NAME;
     LOCAL_NAME            : in     RELATIONSHIP_KEY;
     KEY                   : in     RELATIONSHIP_KEY       := EMPTY_KEY_LIST;
     ATTRIBUTE_KIND_KEY    : in     RELATIONSHIP_KEY       := EMPTY_KEY_LIST;
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
        ATTRIBUTES              : in    ATTRIBUTE_LIST          := EMPTY_LIST;
        DISCRETIONARY_ACCESS    : in    DISCRETIONARY_ACCESS_LIST := ALL_RIGHTS;
        MANDATORY_ACCESS        : in    MANDATORY_ACCESS_LIST   := EMPTY_LIST;
        KIND                    : in    NODE_KIND_NAME          := "ATTRIBUTE_DEFINITION";
        STATUS                  : in    STATUS_INDICATOR        := NO_INDICATION;
        MEDIUM_RESOURCE         : in    MEDIUM_NODE_TYPE        := DEFAULT_MEDIUM)
    is
        AKD         : AKD_TYPE;
        NODE_KIND   : NKI_TYPE;
    begin
        OPEN_ATTRIBUTE_KIND_DEFINITION (AKD, ATTRIBUTE_KIND, STATUS);
        OPEN_NODE_KIND (NODE_KIND, KIND, STATUS);

        CREATE_ATTRIBUTE_DEFINITION
            (AD,                VIEW,
             BASE,              AKD,
             LOCAL_NAME, KEY,
             ATTRIBUTE_KIND_KEY,
             ATTRIBUTES,
             DISCRETIONARY_ACCESS,
             MANDATORY_ACCESS,
             NODE_KIND,
             STATUS,
             MEDIUM_RESOURCE);
        CLOSE (AKD);
        CLOSE (NODE_KIND);
    exception
        when others =>
            CLOSE (AKD);
            CLOSE (NODE_KIND);
            raise;
    end CREATE_ATTRIBUTE_DEFINITION;
```

### 5.7.32 Deleting ADs

```
procedure DELETE_ATTRIBUTE_DEFINITION
    (AD                     : in out AD_TYPE;
     VIEW                   : in     VIEW_NODE_TYPE;
     DELETE_DEFINITION      : in     BOOLEAN             := FALSE;
     TIME_LIMIT             : in     CAIS_DURATION       := LONG_DELAY;
     STATUS                 : in     STATUS_INDICATOR    := NO_INDICATION);
```

Purpose:

This procedure deletes a relationship which defines a local name for an attribute definition, and optionally deletes the corresponding attribute definition. 1

A secondary ATTRIBUTE relationship emanating from a VIEW node and terminating at an 2



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

AD is deleted. These nodes are identified by the parameters AD and VIEW.

All ATTRIBUTE_NAME and KEY_NAME relationships which terminate at the AD and emanate from NKIs or RIs whose parent is the VIEW node identified by VIEW are deleted.

If the value of DELETE_DEFINITION is TRUE, then this procedure also effectively performs the DELETE_NODE operation on the AD. The restrictions in Section 5.7.1.9 on page 335 on deletion of an AD apply.

The TIME_LIMIT parameter allows specification of a time limit for the delay imposed on the DELETE_ATTRIBUTE_DEFINITION operation by the existence of locks on any node to be accessed. A delayed call completes after the node is unlocked or the specified time limit has elapsed. In the latter case, the exception LOCK_ERROR is raised.

If the DELETE_ATTRIBUTE_DEFINITION operation raises an exception, no relationship is deleted.

Parameters:

**AD** is is an open node handle to the AD at which the secondary ATTRIBUTE_DEFINITION relationship to be deleted terminates.

**VIEW** is an open node handle to the VIEW node from which the secondary ATTRIBUTE relationship to be deleted emanates.

**DELETE_DEFINITION** is a Boolean. If TRUE, the primary relationship to the AD is also to be deleted. Otherwise, this operation has no effect on this AD.

**TIME_LIMIT** is a time limit for the delay on waiting for the unlocking of a node.

**STATUS** is an established status indicator.

Exceptions:

**NAME_ERROR** is raised if an interpretation node in the view is inaccessible or DELETE_DEFINITION is TRUE and the parent of the AD or the target of the ATTRIBUTE_KIND relationship emanating from the AD is inaccessible [1].

**RELATIONSHIP_ERROR** is raised if the view does not contain the AD [2].

**USE_ERROR** is raised if a primary [62] or secondary [63] relationship to be deleted is predefined or if any primary relationships emanate from the node to be deleted [7].

**STATUS_ERROR** is raised if the node handle AD or VIEW is not open prior to the call [2] or if STATUS is not an established status indicator [3].

**NODE_KIND_ERROR** is raised if AD does not identify an AD [1] or if VIEW does not identify a VIEW node [1].

**CONFORMANCE_ERROR** is raised if the operation would result in an attribute, node or relationship not conforming to its definition or the constraints in force on that definition [3].

**DEFINITION_ERROR** is raised if the value of DELETE_DEFINITION is TRUE and a restriction on the deletion of an AD would be violated [1] or if the value of DELETE_DEFINITION is FALSE and VIEW identifies the last view naming the definition [7].

**LOCK_ERROR** is raised if a node handle to the source node of a relationship to be deleted cannot be opened with intents WRITE_RELATIONSHIPS due to outstanding locks [1] or if an open transaction not containing this operation contains other eligible operations whose effects determine whether the conditions for DEFINITION_ERROR apply [3].



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

**TRANSACTION_ERROR** is raised if the operation is included in more than one transaction [1] or if the operation is included in a transaction and the operation modifies a node which is not included in the same transaction [2]. 20

**INTENT_VIOLATION** is raised if the node handle VIEW was not opened with an intent establishing the right to read relationships and an exclusive intent establishing the right to write relationships [1] or if DELETE_DEFINITION is TRUE and the node handle AD was not opened with an intent including EXCLUSIVE_WRITE and READ_RELATIONSHIPS [1]. 21

**ACCESS_VIOLATION** is raised if the current process does not have sufficient discretionary access control rights to obtain access to the source node of a relationship to be deleted with intents WRITE_RELATIONSHIPS or to obtain access to an interpretation node whose parent is the VIEW node with intents READ_RELATIONSHIPS and WRITE_RELATIONSHIPS [1]. 22

**SECURITY_VIOLATION** is raised if the operation represents a violation of mandatory access controls. SECURITY_VIOLATION is raised only if the conditions for other exceptions are not present. 23

### 5.7.33 Importing ADs

```
procedure IMPORT_ATTRIBUTE_DEFINITION
    (VIEW       : in  VIEW_NODE_TYPE;
     AD         : in  AD_TYPE;
     LOCAL_NAME : in  RELATIONSHIP_KEY;
     STATUS     : in  STATUS_INDICATOR := NO_INDICATION);
```

Purpose:

This procedure defines a name for an existing attribute definition. 1

A secondary ATTRIBUTE relationship with the key list LOCAL_NAME is created. The source node is the VIEW node identified by the open node handle VIEW; the target node is the AD identified by the open node handle AD. 2

Parameters:

**VIEW** is an open node handle to the VIEW node from which the new secondary relationship is to emanate. 3

**AD** is an open node handle to the AD at which the new secondary relationship is to terminate. 4

**LOCAL_NAME** is the key list for the new secondary relationship to be created. 5

**STATUS** is an established status indicator. 6

Exceptions:

**PATHNAME_SYNTAX_ERROR** is raised if the node identification given by LOCAL_NAME is syntactically illegal [2]. 7



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

**NAME_ERROR** is raised if the node identified by AD is inaccessible or unobtainable [1]. 8

**RELATIONSHIP_ERROR** is raised if an ATTRIBUTE relationship with the identification LOCAL_NAME already exists [3] or if LOCAL_NAME specifies a value for the same key more than once [5]. 9

**NODE_KIND_ERROR** is raised if the open node handle VIEW does not identify a VIEW node [1] or if the open node handle AD does not identify an AD [1]. 10

**NOT_IN_VIEW_ERROR** is raised if an attribute name in LOCAL_NAME cannot be translated [1] or if LOCAL_NAME consists of a single relationship key designator and no key of the corresponding relation is visible [5]. 11

**AMBIGUOUS_VIEW_ERROR** is raised if an attribute name in LOCAL_NAME has an ambiguous translation [1] or if LOCAL_NAME consists of a single relationship key designator and more than one key of the corresponding relation is visible [4]. 12

**USE_ERROR** is raised if the identified view already includes the specified AD [16]. 13

**STATUS_ERROR** is raised if VIEW or AD is not an open node handle [2] or if STATUS is not an established status indicator [3]. 14

**CONFORMANCE_ERROR** is raised if the operation would result in an attribute, node or relationship not conforming to its definition or the constraints in force on that definition [3]. 15

**TRANSACTION_ERROR** is raised if the operation is included in more than one transaction [1] or if the operation is included in a transaction and the operation modifies a node which is not included in the same transaction [2]. 16

**INTENT_VIOLATION** is raised if VIEW was not opened with an intent establishing the right to read relationships and an exclusive intent establishing the right to write relationships [1]. 17

**SECURITY_VIOLATION** is raised if the operation represents a violation of mandatory access controls. SECURITY_VIOLATION is raised only if the conditions for other exceptions are not present. 18

### 5.7.34 Renaming ATTRIBUTE relationships in a view

```
procedure  RENAME_ATTRIBUTE_DEFINITION
    (VIEW       : in      VIEU_NODE_TYPE;
     OLD_NAME : in      RELATIONSHIP_KEY;
     NEW_NAME : in      RELATIONSHIP_KEY;
     STATUS     : in      STATUS_INDICATOR  : = NO_INDICATION);
```

Purpose:

This procedure changes a name for an existing attribute definition. 1

The key designators for a secondary ATTRIBUTE relationship are changed. VIEW is an open node handle to the VIEW node from which the relationship emanates. The relationship affected is identified by the key list parameter OLD_NAME. Its new identification is specified by the key list parameter NEW_NAME. 2

The secondary relationship is not modified if this operation raises an exception. 3



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Parameters:

**VIEW** is an open node handle to the VIEW node from which the secondary relationship to be renamed emanates. 4

**OLD_NAME** is a key list for an ATTRIBUTE relationship which emanates from the VIEW node. 5

**NEW_NAME** is a new key list for the relationship identified by OLD_NAME. 6

**STATUS** is an established status indicator. 7

Exceptions:

**PATHNAME_SYNTAX_ERROR** is raised if the old or new node identification is syntactically illegal [2]. 8

**RELATIONSHIP_ERROR** is raised if an ATTRIBUTE relationship with the identification OLD_NAME does not exist [2] or an ATTRIBUTE relationship with the identification NEW_NAME already exists [3] or if a key list specifies a value for the same key more than once [5]. 9

**NOT_IN_VIEW_ERROR** is raised if an attribute name in OLD_NAME or NEW_NAME cannot be translated [1] or if OLD_NAME or NEW_NAME consists of a single relationship key designator and no key of the corresponding relation is visible [5]. 10

**AMBIGUOUS_VIEW_ERROR** is raised if an attribute name in OLD_NAME or NEW_NAME has an ambiguous translation [1] or if OLD_NAME or NEW_NAME consists of a single relationship key designator and more than one key of the corresponding relation is visible [4]. 11

**NODE_KIND_ERROR** is raised if VIEW does not identify a VIEW node [1]. 12

**STATUS_ERROR** is raised if the node handle VIEW is not open [2] or if STATUS is not an established status indicator [3]. 13

**USE_ERROR** is raised if the relationship is predefined [61]. 14

**CONFORMANCE_ERROR** is raised if the operation would result in an attribute, node or relationship not conforming to its definition or the constraints in force on that definition [5,6,10]. 15

**TRANSACTION_ERROR** is raised if the operation is included in more than one transaction [1] or if the operation is included in a transaction and the operation modifies a node which is not included in the same transaction [2]. 16

**INTENT_VIOLATION** is raised if VIEW was not opened with an intent establishing the right to read relationships and an exclusive intent establishing the right to write relationships [1]. 17

**SECURITY_VIOLATION** is raised if the operation represents a violation of mandatory access controls. SECURITY_VIOLATION is raised only if the conditions for other exceptions are not present. 18

### 5.7.35 Redefining the AKD for an AD

```
procedure REDEFINE_ATTRIBUTE_KIND
    (AD                 : in      AD_TYPE;
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
    AKD           : in    AKD_TYPE;
    REVERSE_KEY   : in    RELATIONSHIP_KEY := EMPTY_KEY_LIST;
    TIME_LIMIT    : in    CAIS_DURATION    := LONG_DELAY;
    STATUS        : in    STATUS_INDICATOR := NO_INDICATION);
```

Purpose:

This procedure changes the value type for attributes defined by the given attribute definition. 1

The bidirectional, secondary ATTRIBUTE_KIND relationship emanating from the AD identified by the open handle AD is set to have the new target AKD identified by the open handle AKD. The key list for the corresponding reverse relationship is provided by REVERSE_KEY. No other relationship attribute is modified. 2

All constraints and default values applied through NKDs and RDs to the AD, and all constraints applied through the AD to the AKD, must reapplicable to the new AKD. If instances of the AD exist, the new AKD must not place stronger constraints on value ranges than the current one. The secondary relationship is not modified if this operation raises an exception. 3

The TIME_LIMIT parameter allows specification of a time limit for the delay imposed on the REDEFINE_ATTRIBUTE_KIND operation by the existence of locks on the AD's current AKD. A delayed call completes after the node is unlocked or the specified time limit has elapsed. In the latter case, the exception LOCK_ERROR is raised. 4

Parameters:

**AD** is an open node handle to the AD from which the ATTRIBUTE_KIND relationship emanates. 5

**AKD** is the new target AKD for the ATTRIBUTE_KIND relationship emanating from NODE. 6

**REVERSE_KEY** is the relationship key list for the reverse relationship emanating from the new target node. 7

**TIME_LIMIT** is a time limit for the delay on waiting for the unlocking of a node. 8

**STATUS** is an established status indicator. 9

Exceptions:

**PATHNAME_SYNTAX_ERROR** is raised if the REVERSE_KEY parameter is syntacticaliy illegal [2]. 10

**RELATIONSHIP_ERROR** is raised if REVERSE_KEY identifies an existing ATTRIBUTE_KIND_OF relationship other than the one to be modified [3] or if REVERSE_KEY specifies a value for the same key more than once [5]. 11

**NOT_IN_VIEW_ERROR** is raised if an attribute name in REVERSE_KEY cannot be translated [1] or if REVERSE_KEY consists of a single relationship key designator and no key of the corresponding relation is visible [5]. 12

**AMBIGUOUS_VIEW_ERROR** is raised if an attribute name in REVERSE_KEY has an ambiguous translation [1] or if REVERSE_KEY consists of a single relationship key designator and more than one key of the corresponding relation is visible [4]. 13



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

**NODE_KIND_ERROR** is raised if AD does not identify an AD [1] or if AKD does not identify an AKD [1]. 14

**STATUS_ERROR** is raised if the node handle AD or AKD is not open [2] or if STATUS is not an established status indicator [3]. 15

**USE_ERROR** is raised if the relationship is predefine [61]. 16

**CONFORMANCE_ERROR** is raised if the operation would result in an attribute, node or relationship not conforming to its definition or the constraints in force on that definition [3,10]. 17

**DEFINITION_ERROR** is raised if a restriction on the modification of an AD would be violated [4]. 18

**LOCK_ERROR** is raised if a node handle to the AD's current AKD cannot be opened with intent WRITE_RELATIONSHIPS due to outstanding locks [1] or if an open transaction not containing this operation contains other eligible operations whose effects determine whether the conditions for DEFINITION_ERROR apply [3]. 19

**TRANSACTION_ERROR** is raised if the operation is included in more than one transaction [1] or if the operation is included in a transaction and the operation modifies a node which is not included in the same transaction [2]. 20

**INTENT_VIOLATION** is raised if AD was not opened with an intent establishing the right to read and write relationships [1] or if AKD was not opened with an intent establishing the right to read attributes and to read and write relationships [1]. 21

**ACCESS_VIOLATION** is raised if the discretionary access control rights of the current process are insufficient to allow access to the AD's current AKD with intent to WRITE_RELATIONSHIPS [1]. 22

**SECURITY_VIOLATION** is raised if the operation represents a violation of mandatory access controls. SECURITY_VIOLATION is raised only if the conditions for other exceptions are not present. 23

### 5.7.36 Creating AKDs

```
procedure CREATE_ATTRIBUTE_KIND
    (AKD                    : in out AKD_TYPE;
     VIEW                   : in     VIEW_NODE_TYPE;
     BASE                   : in     NODE_TYPE;
     LOCAL_NAME             : in     RELATIONSHIP_KEY;
     KEY                    : in     RELATIONSHIP_KEY          := EMPTY_KEY_LIST;
     ATTRIBUTES             : in     ATTRIBUTE_LIST            := EMPTY_LIST;
     DISCRETIONARY_ACCESS   : in     DISCRETIONARY_ACCESS_LIST := ALL_RIGHTS;
     MANDATORY_ACCESS       : in     MANDATORY_ACCESS_LIST := EMPTY_LIST;
     KIND                   : in     NODE_KIND_NODE_TYPE: =
                                         NAMED_COMPOSITE_LIST_AKD_NKD;
     STATUS                 : in     STATUS_INDICATOR          := NO_INDICATION;
     MEDIUM_RESOURCE        : in     MEDIUM_NODE_TYPE          := DEFAULT_MEDIUM) ;
```


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Purpose:

This procedure creates a new attribute kind. An AKD and the primary ATTRIBUTE_KIND_DEFINITION relationship to it are created, as well as the corresponding reverse relationship to the node identified by BASE. The primary relationship key list is given by KEY. An open node handle to the newly created AKD with WRITE intent is returned in AKD.

A secondary ATTRIBUTE_KIND relationship is created from the VIEW node identified by VIEW to the new AKD. The relationship key list is given by the parameter LOCAL_NAME.

The ATTRIBUTES parameter provides initial values for attributes of the new node. ATTRIBUTES may include values for any of the following CAIS-controlled AKD attributes that are defined for the AKD being created: LOWER_BOUND and UPPER_BOUND (IAKDs and FAKDs); DIGITS (FAKDs); PATTERNS (IDAKDs and SAKDs); ENUMERATED (IDAKDs); MINIMUM_SIZE and MAXIMUM_SIZE (ULAKDs); and NAMING and ORDERED (CLAKDs).

The MANDATORY_ACCESS parameter specifies the object classification label with which the node is to be created. If its value is the empty list, the node inherits the subject classification of the creating process as its object classification.

KIND is an open node handle identifying the node kind for the new node. It must identify an AKD kind. It may specify either an NKD or an NKI. If KIND identifies an NKI, the NKI is made the current adopted interpretation for the node handle AKD.

The MEDIUM_RESOURCE parameter specifies the medium resource on which the newly created node will reside.

Parameters:

**AKD** is a closed node handle to be opened to the newly created AKD.

**VIEW** is an open node handle to the VIEW node from which the secondary ATTRIBUTE_KIND relationship to the new node is to emanate.

**BASE** is an open node handle to the node from which the primary ATTRIBUTE_KIND_DEFINITION relationship to the new node is to emanate.

**LOCAL_NAME** is the key list of the ATTRIBUTE_KIND relationship to be created to the newly created node.

**KEY** is the key list of the ATTRIBUTE_KIND_DEFINITION relationship to be created.

**ATTRIBUTES** is an empty or named list whose elements are used to establish initial values for attributes of the newly created node; each named item of the list specifies an attribute name and the value to be given to that attribute.

**DISCRETIONARY_ACCESS** is the initial access control information associated with the node; it is the value of the GRANT attribute of the ACCESS relationship to the user's default GROUP node.

**MANDATORY_ACCESS** is a list defining the classification label for the created node.

**KIND** is an open node handle identifying the AKD kind for the node to be created.

**STATUS** is an established status indicator.

**MEDIUM_RESOURCE** is an open node handle identifying a MEDIUM node.



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Except ions:

**PATHNAME_SYNTAX_ERROR** is raised if the node identification given by LOCAL_-NAME or KEY is syntactically illegal [2].

**NAME_ERROR** is raised if the parent of the NKI identified by KIND is inaccessible [1].

**EXISTING_NODE_ERROR** is raised if a primary relationship already exists with the identification given [1].

**RELATIONSHIP_ERROR** is raised if a secondary relationship already exists with the identification given [3].

**NODE_KIND_ERROR** is raised if the open node handle KIND does not identify an AKD kind [2], if the open node handle VIEW does not identify a VIEW node [1] or if the open node handle MEDIUM_RESOURCE does not identify a MEDIUM node [1].

**RELATIONSHIP_ERROR** is raised if an ATTRIBUTE_KIND relationship with the identification LOCAL_NAME already exists [3] or if a key list specifies a value for the same key more than once [5].

**ATTRIBUTE_ERROR** is raised if an attribute list specifies a value for the same attribute more than once [6].

**NOT_IN_VIEW_ERROR** is raised if an attribute name specified in LOCAL_NAME, KEY or ATTRIBUTES cannot be translated [1] or if LOCAL_NAME or KEY consists of a single relationship key designator and no key of the corresponding relation is visible [5].

**AMBIGUOUS_VIEW_ERROR** is raised if an attribute name specified in LOCAL_NAME, KEY or ATTRIBUTES has an ambiguous translation [1] or if KEY or LOCAL_NAME consists of a single relationship key designator and more than one key of the corresponding relation is visible [4].

**SYNTAX_ERROR** is raised if the ATTRIBUTES parameter, the DISCRETIONARY_ACCESS parameter or the MANDATORY_ACCESS parameter is syntactically illegal [8-10].

**PREDEFINED_ATTRIBUTE_ERROR** is raised if any attribute named in the ATTRIBUTES parameter is a CAIS-controlled attribute that is not identified in the Purpose section of this interface as an attribute which may be specified in that parameter [1].

**USE_ERROR** is raised if the value for MANDATORY_ACCESS is semantically illegal [10].

**STATUS_ERROR** is raised if VIEW, BASE, KIND or MEDIUM_RESOURCE is not an open node handle [2], if AKD is an open node handle prior to the call [1] or if STATUS is not an established status indicator [3].

**CONFORMANCE_ERROR** is raised if the operation would result in an attribute, node or relationship not conforming to its definition or the constraints in force on that definition [1,3,7,8,9,10].

**DEFINITION_ERROR** is raised if the operation would result in a composite list AKD that is not constrained to be either named or ordered (or both) [5].

**LOCK_ERROR** is raised if a node handle to the parent of the NKI specified by KIND cannot be opened with intent READ_RELATIONSHIPS due to outstanding locks in conflict with that intent [1].

**TRANSACTION_ERROR** is raised if the operation is included in more than one transaction [1] or if the operation is included in a transaction and the operation modifies a node which is not included in the same transaction [2].


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

**INTENT_VIOLATION** is raised if BASE was not opened with an intent establishing the right to create relationships [1] or if VIEW was not opened with an exclusive intent establishing the right to create relationships [1]. 35

**ACCESS_VIOLATION** is raised if the discretionary access control rights of the current process are insufficient to allow access with CREATE_INSTANCES intents to the NKD identified by KIND or with READ_RELATIONSHIPS intents to the parent of the NKI specified by KIND or with APPEND_CONTENTS intents to the MEDIUM node identified by MEDIUM_RESOURCE [1]. 36

**RESOURCE_ALLOCATION_ERROR** is raised if the medium resource is not allocatable [1]. 37

**RESOURCE_AVAILABILITY_ERROR** is raised if the medium resource is not available [1]. 38

**SECURITY_VIOLATION** is raised if the operation represents a violation of mandatory access controls. SECURITY_VIOLATION is raised only if the conditions for other exceptions are not present. 39

Additional Interfaces:

```
procedure CREATE_ATTRIBUTE_KIND
    (AKD                    : in out AKD_TYPE;
     VIEW                   : in     VIEW_NODE_TYPE;
     BASE                   : in     NODE_TYPE;
     LOCAL_NAME             : in     RELATIONSHIP_KEY;
     KEY                    : in     RELATIONSHIP_KEY          := EMPTY_KEY_LIST;
     ATTRIBUTES             : in     ATTRIBUTE_LIST            := EMPTY_LIST;
     DISCRETIONARY_ACCESS : in     DISCRETIONARY_ACCESS_LIST := ALL_RIGHTS;
     MANDATORY_ACCESS       : in     MANDATORY_ACCESS_LIST := EMPTY_LIST;
     KIND                   : in     NODE_KIND_NAME;
     STATUS                 : in     STATUS_INDICATOR          := NO_INDICATION;
     MEDIUM_RESOURCE        : in     MEDIUM_NODE_TYPE          := DEFAULT_MEDIUM)
is
    NODE_KIND : NKI_TYPE;
begin
    OPEN_NODE_KIND (NODE_KIND, KIND, STATUS);
    CREATE_ATTRIBUTE_KIND
        (AKD,
         VIEW,
         BASE,
         LOCAL NAME,
         KEY,
         ATTRIBUTES,
         DISCRETIONARY_ACCESS,
         MANDATORY_ACCESS,
         NODE_KIND,
         STATUS,
         MEDIUM_RESOURCE);
    CLOSE (NODE_KIND);
```


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
except ion
    when others =>
        CLOSE (NODE_KIND);
        raise;
end CREATE_ATTRIBUTE_KIND;
```

### 5.7.37 Creating identifier AKDs

```
generic
    type ENUMERATION is (<>);
procedure CREATE_IDENTIFIER_ATTRIBUTE_KIND
    (AKD                    : in out AKD_TYPE;
     VIEW                   : in     VIEW_NODE_TYPE;
     BASE                   : in     NODE_TYPE;
     LOCAL_NAME             : in     RELATIONSHIP_KEY;
     KEY                    : in     RELATIONSHIP_KEY          := EMPTY_KEY_LIST;
     ATTRIBUTES             : in     ATTRIBUTE_LIST            := EMPTY_LIST;
     DISCRETIONARY_ACCESS   : in     DISCRETIONARY_ACCESS_LIST := ALL_RIGHTS;
     MANDATORY_ACCESS       : in     MANDATORY_ACCESS_LIST  := EMPTY_LIST;
     KIND                   : in     NODE_KIND_NODE_TYPE :=
                                         IDENTIFIER_ATTRIBUTE_KIND_DEFINITION_NKD;
     STATUS                 : in     STATUS_INDICATOR          := NO_INDICATION;
     MEDIUM_RESOURCE        : in     MEDIUM_NODE_TYPE          := DEFAULT_MEDIUM);
```

Purpose:

An instantiation of this generic procedure creates anew identifier attribute kind corresponding to the base type of the Ada enumeration type ENUMERATION. An identifier AKD and the primary ATTRIBUTE_KIND_DEFINITION relationship to it are created, as well as the corresponding reverse relationship to the node identified by BASE. The primary relationship key list is given by KEY. An open node handle to the newly created identifier AKD with WRITE intent is returned in AKD. 1

A secondary ATTRIBUTE_KIND relationship is created from the VIEW node identified by VIEW to the new AKD. The relationship key list is given by the parameter LOCAL_NAME. 2

A PATTERNS attribute for the AKD is created. The patterns are strings containing the images of the enumeration values of the base type for ENUMERATION in the enumeration order for that base type. The images are those as defined by 'IMAGE (see [1815A] 3.5.5, paragraphs 10-11). CONFORMANCE_ERROR is raised if the image of any enumeration value is not an identifier. 3

The ATTRIBUTES parameter provides initial values for attributes of the new node. ATTRIBUTES may include a value for the CAIS-controlled AKD attribute ENUMERATED; hence, the AKD created may be either an ordinary identifier AKD or an enumerated identifier AKD. 4

The MANDATORY_ACCESS parameter specifies the object classification label with which the node is to be created. If its value is the empty list, the node inherits the subject classification of the creating process as its object classification. 5

KIND is an open node handle identifying the node kind for the new node. It must identify an 6


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

identifier AKD kind. It may specify either an NKD or an NKI. If KIND identifies an NKI, the NKI is made the current adopted interpretation for the node handle AKD.

The MEDIUM_RESOURCE parameter specifies the medium resource on which the newly created node will reside.

Parameters:

**AKD** is a closed node handle to be opened to the newly created identifier AKD.

**VIEW** is an open node handle to the VIEW node from which the secondary ATTRIBUTE_KIND relationship to the new node is to emanate.

**BASE** is an open node handle to the node from which the primary ATTRIBUTE_KIND_DEFINITION relationship to the new node is to emanate.

**LOCAL_NAME** is the key list of the ATTRIBUTE_KIND relationship to be created to the newly created node.

**KEY** is the key list of the ATTRIBUTE_KIND_DEFINITION relationship to be created.

**ATTRIBUTES** is an empty or named list whose elements are used to establish initial values for attributes of the newly created node; each named item of the list specifies an attribute name and the value to be given to that attribute.

**DISCRETIONARY_ACCESS** is the initial access control information associated with the node; it is the value of the GRANT attribute of the ACCESS relationship to the user's default GROUP node.

**MANDATORY_ACCESS** is a list defining the classification label for the created node.

**KIND** is an open node handle identifying the identifier AKD kind for the node to be created.

**STATUS** is an established status indicator.

**MEDIUM_RESOURCE** is an open node handle identifying a MEDIUM node.

Exceptions:

**PATHNAME_SYNTAX_ERROR** is raised if the node identification given by LOCAL_NAME or KEY is syntactically illegal [2].

**NAME_ERROR** is raised if the parent of the NKI identified by KIND is inaccessible [1].

**EXISTING_NODE_ERROR** is raised if a primary relationship already exists with the identification given [1].

**RELATIONSHIP_ERROR** is raised if a secondary relationship already exists with the identification given [3].

**NODE_KIND_ERROR** is raised if the open node handle KIND does not identify an identifier AKD kind [2], if the open node handle VIEW does not identify a VIEW node [1] or if the open node handle MEDIUM_RESOURCE does not identify a MEDIUM node [1].

**RELATIONSHIP_ERROR** is raised if an ATTRIBUTE_KIND relationship with the identification LOCAL_NAME already exists [3] or if a key list specifies a value for the same key more than once [5].

**ATTRIBUTE_ERROR** is raised if an attribute list specifies a value for the same attribute more than once [6].


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

**NOT_IN_VIEW_ERROR** is raised if an attribute name specified in LOCAL_NAME, KEY or ATTRIBUTES cannot be translated [1] or if LOCAL_NAME or KEY consists of a single relationship key designator and no key of the corresponding relation is visible [5]. 26

**AMBIGUOUS_VIEW_ERROR** is raised if an attribute name specified in LOCAL_NAME, KEY or ATTRIBUTES has an ambiguous translation [1] or if KEY or LOCAL_NAME consists of a single relationship key designator and more than one key of the corresponding relation is visible [4]. 27

**SYNTAX_ERROR** is raised if the ATTRIBUTES parameter, the DISCRETIONARY_ACCESS parameter or the MANDATORY_ACCESS parameter is syntactically illegal [8-10]. 28

**PREDEFINED_ATTRIBUTE_ERROR** is raised if any attribute named in the ATTRIBUTES parameter is a CAIS-controlled attribute that is not identified in the Purpose section of this interface as an attribute which may be specified in that parameter [1]. 29

**USE_ERROR** is raised if the value for MANDATORY_ACCESS is semantically illegal [10]. 30

**STATUS_ERROR** is raised if VIEW, BASE, KIND or MEDIUM_RESOURCE is not an open node handle [2], if AKD is an open node handle prior to the call [1] or if STATUS is not an established status indicator [3]. 31

**CONFORMANCE_ERROR** is raised if the operation would result in an attribute, node or relationship not conforming to its definition or the constraints in force on that definition [1,3,7,8,9,10]. 32

**LOCK_ERROR** is raised if a node handle to the parent of the NKI specified by KIND cannot be opened with intent READ_RELATIONSHIPS due to outstanding locks in conflict with that intent [1]. 33

**TRANSACTION_ERROR** is raised if the operation is included in more than one transaction [1] or if the operation is included in a transaction and the operation modifies a node which is not included in the same transaction [2]. 34

**INTENT_VIOLATION** is raised if BASE was not opened with an intent establishing the right to create relationships [1] or if VIEW was not opened with an exclusive intent establishing the right to create relationships [1]. 35

**ACCESS_VIOLATION** is raised if the discretionary access control rights of the current process are insufficient to allow access with CREATE_INSTANCES intents to the NKD identified by KIND or with READ_RELATIONSHIPS intents to the parent of the NKI specified by KIND or with APPEND_CONTENTS intents to the MEDIUM node identified by MEDIUM_RESOURCE [1]. 36

**RESOURCE_ALLOCATION_ERROR** is raised if the medium resource is not allocatable [1]. 37

**RESOURCE_AVAILABILITY_ERROR** is raised if the medium resource is not available [1]. 38

**SECURITY_VIOLATION** is raised if the operation represents a violation of mandatory access controls. SECURITY_VIOLATION is raised only if the conditions for other exceptions are not present. 39

Additional Interfaces:

generic


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
    type ENUMERATION is (<>);
procedure CREATE_IDENTIFIER_ATTRIBUTE_KIND_BY_NAME
    (AKD                    : in out AKD_TYPE;
     VIEW                   : in     VIEW_NODE_TYPE;
     BASE                   : in     NODE_TYPE;
     LOCAL_NAME             : in     RELATIONSHIP_KEY;
     KEY                    : in     RELATIONSHIP_KEY          := EMPTY_KEY_LIST;
     ATTRIBUTES             : in     ATTRIBUTE_LIST            := EMPTY_LIST;
     DISCRETIONARY_ACCESS   : in     DISCRETIONARY_ACCESS_LIST := ALL_RIGHTS;
     MANDATORY_ACCESS       : in     MANDATORY_ACCESS_LIST     := EMPTY_LIST;
     KIND                   : in     NODE_KIND_NAME;
     STATUS                 : in     STATUS_INDICATOR          := NO_INDICATION;
     MEDIUM_RESOURCE        : in     MEDIUM_NODE_TYPE          := DEFAULT_MEDIUM)
is
    NODE_KIND : NKI_TYPE;
    procedure CREATE_IDENTIFIER_ATTRIBUTE_KIND_BY_HANDLE is
        new CREATE_IDENTIFIER_ATTRIBUTE_KIND (ENUMERATION);
begin
    OPEN_NODE_KIND (NODE_KIND, KIND, STATUS);
    CREATE_IDENTIFIER_ATTRIBUTE_KIND_BY_HANDLE
        (AKD,
         VIEW,
         BASE,
         LOCAL_NAME,
         KEY,
         ATTRIBUTES,
         DISCRETIONARY_ACCESS,
         MANDATORY_ACCESS,
         NODE_KIND,
         STATUS,
         MEDIUM_RESOURCE);
    CLOSE (NODE_KIND);
exception
    when others =>
        CLOSE (NODE_KIND);
        raise;
end  CREATE_IDENTIFIER_ATTRIBUTE_KIND_BY_NAME;
```

### 5.7.38 Deleting AKDs

```
procedure DELETE_ATTRIBUTE_KIND
    (AKD                    : in out AKD_TYPE;
     VIEW                   : in     VIEW_NODE_TYPE;
     DELETE_DEFINITION      : in     BOOLEAN          := FALSE;
     TIME_LIMIT             : in     CAIS_DURATION    := LONG_DELAY;
     STATUS                 : in     STATUS_INDICATOR := NO_INDICATION);
```


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Purpose:

This procedure deletes a relationship which defines a local name for an attribute kind definition, and optionally deletes the corresponding attribute kind definition. 1

A secondary ATTRIBUTE_KIND relationship emanating from a VIEW node and terminating at an AKD is deleted. These nodes are identified by the parameters AKD and VIEW. 2

If the value of DELETE_DEFINITION is TRUE, then this procedure also effectively performs the DELETE_NODE operation on the AKD. The restrictions in Section 5.7.1.9 on page 335 on deletion of an AKD apply. 3

The TIME_LIMIT parameter allows specification of a time limit for the delay imposed on the DELETE_ATTRIBUTE_KIND operation by the existence of locks on any node to be accessed. A delayed call completes after the node is unlocked or the specified time limit has elapsed. In the latter case, the exception LOCK_ERROR is raised. 4

If the DELETE_ATTRIBUTE_KIND operation raises an exception, no relationship is deleted. 5

Parameters:

**AKD** is an open node handle to the AKD at which the secondary ATTRIBUTE_KIND relationship to be deleted terminates. 6

**VIEW** is an open node handle to the VIEW node from which the ATTRIBUTE_KIND relationship to be deleted emanates. 7

**DELETE_DEFINITION** is a Boolean. If TRUE, the primary relationship to the AKD is also to be deleted. Otherwise, this operation has no effect on the AKD. 8

**TIME_LIMIT** is a time limit for the delay on waiting for the unlocking of a node. 9

**STATUS** is an established status indicator. 10

Exceptions:

**NAME_ERROR** is raised if the parent of the AKD whose primary relationship is to be deleted is inaccessible [1]. 11

**RELATIONSHIP_ERROR** is raised if the view does not contain the AKD [2]. 12

**USE_ERROR** is raised if any primary relationships emanate from the node to be deleted [7] or if a primary [62] or secondary [63] relationship to be deleted is predefine. 13

**STATUS_ERROR** is raised if the node handle AKD or VIEW is not open [2] or if STATUS is not an established status indicator [3]. 14

**NODE_KIND_ERROR** is raised if AKD does not identify an AKD or if VIEW does not identify a VIEW node [1]. 15

**CONFORMANCE_ERROR** is raised if the operation would result in an attribute, node or relationship not conforming to its definition or the constraints in force on that definition [3]. 16

**DEFINITION_ERROR** is raised if the value of DELETE_DEFINITION is TRUE and a restriction on the deletion of an AKD would be violated [1] or if the value of DELETE_-DEFINITION is FALSE and VIEW identifies the last view naming the definition [7]. 17


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

**LOCK_ERROR** is raised if a node handle to the source node of a relationship to be deleted cannot be opened with intent WRITE_RELATIONSHIPS due to outstanding locks [1] or if an open transaction not containing this operation contains other eligible operations whose effects determine whether the conditions for DEFINITION_ERROR apply [3]. 18

**TRANSACTION_ERROR** is raised if the operation is included in more than one transaction [1] or if the operation is included in a transaction and the operation modifies a node which is not included in the same transaction [2]. 19

**INTENT_VIOLATION** is raised if the node handle VIEW was not opened with an intent establishing the right to read relationships and an exclusive intent establishing the right to write relationships [1] or if DELETE_DEFINITION is TRUE and the node handle AD was not opened with an intent including EXCLUSIVE_WRITE and READ_RELATIONSHIPS [1]. 20

**ACCESS_VIOLATION** is raised if DELETE_DEFINITION is true and the current process does not have sufficient discretionary access control rights to obtain access to the source node of a relationship to be deleted with intent WRITE_RELATIONSHIPS [1]. 21

**SECURITY_VIOLATION** is raised if the operation represents a violation of mandatory access controls. SECURITY_VIOLATION is raised only if the conditions for other exceptions are not present. 22

### 5.7.39 Importing AKDs

```
procedure IMPORT_ATTRIBUTE_KIND
    (VIEW          : in    VIEW_NODE_TYPE;
     AKD           : in    AKD_TYPE;
     LOCAL_NAME : in    RELATIONSHIP_KEY;
     STATUS        : in    STATUS_INDICATOR  : = NO_INDICATION);
```

Purpose:

This procedure defines a name for an existing attribute kind. 1

A secondary ATTRIBUTE_KIND relationship with key list LOCAL_NAME is created. The source node is the VIEW node identified by the open node handle VIEW; the target node is the AKD identified by the open node handle AKD. 2

Parameters:

**VIEW** is an open node handle to the VIEW node from which the new secondary relationship is to emanate. 3

**AKD** is an open node handle to the AKD at which the new secondary relationship is to terminate. 4

**LOCAL_NAME** is the key list for the new secondary relationship to be created. 5

**STATUS** is an established status indicator. 6



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Exceptions:

**PATHNAME_SYNTAX_ERROR** is raised if the node identification given by LOCAL_NAME is syntactically illegal [2].

**NAME_ERROR** is raised if the node identified by AKD is inaccessible or unobtainable [1].

**RELATIONSHIP_ERROR** is raised if an ATTRIBUTE_KIND relationship with the identification LOCAL_NAME already exists [3] or if LOCAL_NAME specifies a value for the same key more than once [5].

**NODE_KIND_ERROR** is raised if the open node handle VIEW does not identify a VIEW node [1] or if the open node handle AKD does not identify an AKD [1].

**NOT_IN_VIEW_ERROR** is raised if an attribute name in LOCAL_NAME cannot be translated [1] or if LOCAL_NAME consists of a single relationship key designator and no key of the corresponding relation is visible [5].

**AMBIGUOUS_VIEW_ERROR** is raised if an attribute name in LOCAL_NAME has an ambiguous translation [1] or if LOCAL_NAME consists of a single relationship key designator and more than one key of the corresponding relation is visible [4].

**USE_ERROR** is raised if the view already includes the specified attribute kind [16].

**STATUS_ERROR** is raised if VIEW or AKD is not an open node handle [2] or if STATUS is not an established status indicator [3].

**CONFORMANCE_ERROR** is raised if the operation would result in an attribute, node or relationship not conforming to its definition or the constraints in force on that definition [3].

**TRANSACTION_ERROR** is raised if the operation is included in more than one transaction [1] or if the operation is included in a transaction and the operation modifies a node which is not included in the same transaction [2].

**INTENT_VIOLATION** is raised if VIEW was not opened with an intent establishing the right to read relationships and an exclusive intent establishing the right to write relationships [1].

**SECURITY_VIOLATION** is raised if the operation represents a violation of mandatory access controls. SECURITY_VIOLATION is raised only if the conditions for other exceptions are not present.

### 5.7.40 Renaming ATTRIBUTE_KIND relationships in a view

```
procedure RENAME_ATTRIBUTE_KIND
   (VIEW      : in     VIEW_NODE_TYPE;
    OLD_NAME  : in     RELATIONSHIP_KEY;
    NEW_NAME  : in     RELATIONSHIP_KEY;
    STATUS    : in     STATUS_INDICATOR  : = NO_INDICATION);
```

Purpose:

This procedure changes a name for an existing attribute kind.

The key designators for a secondary ATTRIBUTE_KIND relationship are changed. VIEW is an open node handle to a VIEW node from which the relationship emanates. The relationship affected is identified by the key list parameter OLD_NAME. Its new identification is specified by the key list parameter NEW_NAME.


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

The secondary relationship is not modified if this operation raises an exception. 3

Parameters:

**VIEW** is an open node handle to the VIEW node from which the secondary relationship to be renamed emanates. 4

**OLD_NAME** is a key list for an ATTRIBUTE_KIND relationship which emanates from the VIEW node. 5

**NEW_NAME** is a new key list for the relationship identified by OLD_NAME. 6

**STATUS** is an established status indicator. 7

Exceptions:

**PATHNAME_SYNTAX_ERROR** is raised if the old or new node identification is syntactically illegal [2]. 8

**RELATIONSHIP_ERROR** is raised if an ATTRIBUTE_KIND relationship does not exist with the identification OLD_NAME [2] or does exist with the identification NEW_NAME [3] or if a key list specifies a value for the same key more than once [5]. 9

**NOT_IN_VIEW_ERROR** is raised if an attribute name in OLD_NAME or NEW_NAME cannot be translated [1] or if OLD_NAME or NEW_NAME consists of a single relationship key designator and no key of the corresponding relation is visible [5]. 10

**AMBIGUOUS_VIEW_ERROR** is raised if an attribute name in OLD_NAME or NEW_NAME has an ambiguous translation [1] or if OLD_NAME or NEW_NAME consists of a single relationship key designator and more than one key of the corresponding relation is visible [4]. 11

**NODE_KIND_ERROR** is raised if VIEW does not identify a VIEW node [1]. 12

**STATUS_ERROR** is raised if the node handle VIEW is not open [2] or if STATUS is not an established status indicator [3]. 13

**USE_ERROR** is raised if the relationship is predefine [61]. 14

**CONFORMANCE_ERROR** is raised if the operation would result in an attribute, node or relationship not conforming to its definition or the constraints in force on that definition [5,6,10]. 15

**TRANSACTION_ERROR** is raised if the operation is included in more than one transaction [1] or if the operation is included in a transaction and the operation modifies a node which is not included in the same transaction [2]. 16

**INTENT_VIOLATION** is raised if VIEW was not opened with an intent establishing the right to read relationships and an exclusive intent establishing the right to write relationships [1]. 17

**SECURITY_VIOLATION** is raised if the operation represents a violation of mandatory access controls. SECURITY_VIOLATION is raised only if the conditions for other exceptions are not present. 18

### 5.7.41 Deleting the primary relationships of a tree rooted at a definition node

```
procedure DELETE_DEFINITION_TREE
   (NODE   : in out DEFINITION_NODE_TYPE;
    STATUS : in     STATUS_INDICATOR := NO_INDICATION);
```


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Purpose:

This procedure deletes a tree of definitions.

Effectively, the DELETE_TREE operation is performed for the NKD, RD, AD or AKD specified by the open node handle NODE. The primary relationship to the definition node and all nodes reachable by a unique primary path from the definition node are deleted. The nodes reachable from the definition node by unique primary paths are opened with intent EXCLUSIVE_WRITE. If the DELETE_DEFINITION_TREE operation raises an exception, no relationships are deleted.

The restrictions in Section 5.7.1.9 on page 335 on deletion of definition nodes apply. Effectively, the primary relationships to definition nodes reachable from the specified definition node cannot be deleted by the DELETE_DEFINITION_TREE operation unless they can be deleted by the DELETE_NODE_KIND, DELETE_RELATION, DELETE_ATTRIBUTE_DEFINITION or DELETE_ATTRIBUTE_KIND operations.

The order of deletion is unspecified. However, the existence of a definition node in the tree will not constitute a violation of the restrictions on deletion of another definition node in the tree.

This operation can delete primary relationships to nodes other than definition nodes, but NODE must identify a definition node.

Parameters:

**NODE** is an open node handle to the definition node at the root of the tree whose primary relationships are to be deleted.

**STATUS** is an established status indicator.

Exceptions:

**NAME_ERROR** is raised if the parent of the definition node at the root of the tree to be deleted or any of the target nodes of primary relationships to be deleted are inaccessible [1].

**STATUS_ERROR** is raised if the node handle NODE is not open prior to the call [2] or if STATUS is not an established status indicator [3].

**NODE_KIND_ERROR** is raised if NODE does not identify a definition node [1] or if any node to be deleted is CAIS-controlled and is not a definition node [5].

**USE_ERROR** is raised if any node to be deleted is predefine [62].

**CONFORMANCE_ERROR** is raised if the operation would result in an attribute, node or relationship not conforming to its definition or the constraints in force on that definition [3].

**DEFINITION_ERROR** is raised if a restriction on the deletion of a definition node would be violated [1].

**LOCK_ERROR** is raised if a node handle to the parent of the definition node specified by NODE cannot be opened with intent WRITE_RELATIONSHIPS, if a node handle identifying any node whose unique primary path traverses the definition node cannot be opened with intent EXCLUSIVE_WRITE due to outstanding locks [1] or if an open transaction not containing this operation contains other eligible operations whose effects determine whether the conditions for DEFINITION_ERROR apply [3].


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

**TRANSACTION_ERROR** is raised if the operation is included in more than one transaction [1] or if the operation is included in a transaction and the operation modifies a node which is not included in the same transaction [2]. 15

**INTENT_VIOLATION** is raised if the node handle NODE was not opened with intent including EXCLUSIVE_WRITE and READ_RELATIONSHIPS [1]. 16

**ACCESS_VIOLATION** is raised if the current process does not have sufficient discretionary access control rights to obtain access to the parent of the definition node specified by NODE with intent WRITE_RELATIONSHIPS or to obtain access to any target node of a primary relationship to be deleted with intent EXCLUSIVE_WRITE [1]. 17

**SECURITY_VIOLATION** is raised if the operation represents a violation of mandatory access controls. SECURITY_VIOLATION is raised only if the conditions for other exceptions are not present. 18

### 5.7.42 Specifying source node' kinds for a relation

```
procedure DEFINE_SOURCE
   (VIEW          : in   VIEW_NODE_TYPE;
    NKD           : in   NKD_TYPE;
    RD            : in   RD_TYPE;
    LOCAL_NAME    : in   RELATIONSHIP_KEY;
    KEY           : in   RELATIONSHIP_KEY   := EMPTY_KEY_LIST;
    REVERSE_KEY   : in   RELATIONSHIP_KEY   : = EMPTY_KEY_LIST;
    ATTRIBUTES    : in   ATTRIBUTE_LIST     : = EMPTY_LIST;
    STATUS        : in   STATUS_INDICATOR   := NO_INDICATION);
```

Purpose:

This procedure defines that relationships of the specified relation can emanate from nodes of the specified kind. 1

An EMANATES relationship with key list KEY is created. The source node is the NKD identified by the open node handle NKD. The target node is the RD identified by the open node handle RD_REVERSE_KEY provides the key list for the corresponding reverse relationship. 2

The ATTRIBUTES parameter provides initial values for attributes of the new relationship. ATTRIBUTES may include values for the optional, CAIS-controlled EMANATES relationship attributes MINIMUM_INSTANCES and MAXIMUM_INSTANCES. 3

VIEW is an open node handle to a VIEW node for a view containing an NKI and RI corresponding to the NKD and RD. A RELATION_NAME relationship with key list LOCAL_NAME is created which emanates from the NKI and terminates at the RI. This relationship establishes the resulting NAME key value as a relation name for relationships of the given relation emanating from nodes interpreted through the NKI. 4

Parameters:

VIEW is an open node handle to a VIEW node. 5

NKD is an open node handle to an NKD. 6


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

**RD** is an open node handle to an RD.

**LOCAL_NAME** is the key list for the RELATION_NAME relationship to be created.

**KEY** is the key list for the EMANATES relationship to be created.

**REVERSE_KEY** is the key list for the EMANATES_FROM relationship to be created.

**ATTRIBUTES** is an empty or named list used to establish initial values for attributes of the newly created EMANATES relationship; each named item of the list specifies an attribute name and the value to be given to that attribute.

**STATUS** is an established status indicator.

Exceptions:

**PATHNAME_SYNTAX_ERROR** is raised if the LOCAL_NAME, KEY or REVERSE_KEY given is syntactically illegal [2].

**NAME_ERROR** is raised if the NKI or RI to be accessed is inaccessible [1].

**RELATIONSHIP_ERROR** is raised if an EMANATES, EMANATES_FROM or RELATION_NAME relationship already exists with the identification given [3] or if a key list specifies a value for the same key more than once [5].

**ATTRIBUTE_ERROR** is raised if an attribute list specifies a value for the same attribute more than once [6].

**SYNTAX_ERROR** is raised if the ATTRIBUTES parameter is syntactically illegal [8].

**NOT_IN_VIEW_ERROR** is raised if an attribute name specified in ATTRIBUTES, LOCAL_NAME, KEY or REVERSE_KEY cannot be translated [1] or if LOCAL_NAME, KEY or REVERSE_ KEY consists of a single relationship key designator and no key of the corresponding relation is visible [5].

**AMBIGUOUS_VIEW_ERROR** is raised if an attribute name specified in ATTRIBUTES, LOCAL_NAME, KEY or REVERSE_KEY has an ambiguous translation [1], if a KEY_NAME relationship already connects the specified NKI and RI [3] or if LOCAL_NAME, KEY or REVERSE_KEY consists of a single relationship key designator and more than one key of the corresponding relation is visible [4].

**PREDEFINED_RELATION_ERROR** is raised if the RD is predefine and CAIS-controlled and the emanates property would be a new one for the node kind [5].

**PREDEFINED_ATTRIBUTE_ERROR** is raised if any attribute named in the ATTRIBUTES parameter is a CAIS-controlled attribute that is not identified in the Purpose section of this interface as an attribute which may be specified in that parameter [2].

**USE_ERROR** is raised if the view identified by VIEW does not contain an interpretation for the NKD or RD [20] or if an EMANATES relationship already emanates from the NKD and terminates at the RD [22].

**STATUS_ERROR** is raised if VIEW, NKD or RD is not an open node handle [2] or if STATUS is not an established status indicator [3].

**NODE_KIND_ERROR** is raised if the open node handle VIEW does not identify a VIEW node [1], if the open node handle NKD does not identify an NKD [1] or if the open node handle RD does not identify an RD [1].


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

**CONFORMANCE_ERROR** is raised if the operation would result in an attribute, node or relationship not conforming to its definition or the constraints in force on that definition [3,7,8,9,10]. 25

**DEFINITION_ERROR** is raised if this operation is used to specify properties or constraints which conflict with those already defined for the relation and its generalizations and specializations [3]. 26

**LOCK_ERROR** is raised if a node handle to the NKI or RI cannot be opened with intents to read and create relationships due to outstanding locks [1] or if an open transaction not containing this operation contains other eligible operations whose effects determine whether the conditions for DEFINITION_ERROR apply [3]. 27

**TRANSACTION_ERROR** is raised if the operation is included in more than one transaction [1] or if the operation is included in a transaction and the operation modifies a node which is not included in the same transaction [2]. 28

**INTENT_VIOLATION** is raised if the node handle NKD or RD was not opened with an intent establishing the rights to read and append relationships [1] or if the node handle VIEW was not opened with an intent establishing the right to read relationships and an exclusive intent establishing the right to write relationships [1]. 29

**ACCESS_VIOLATION** is raised if the discretionary access control rights of the current process are insufficient to allow access to the NKI or RI with intents to read and append relationships [1]. 30

**SECURITY_VIOLATION** is raised if the operation represents a violation of mandatory access controls. SECURITY_VIOLATION is raised only if the conditions for other exceptions are not present. 31

### 5.7.43 Deleting source node kinds for a relation

```
procedure DELETE_SOURCE
   (VIEW              : in   VIEW_NODE_TYPE;
    NKD               : in   NKD_TYPE;
    RD                : in   RD_TYPE;
    DELETE_DEFINITION : in   BOOLEAN          := FALSE;
    TIME_LIMIT        : in   CAIS_DURATION    := LONG_DELAY;
    STATUS            : in   STATUS_INDICATOR := NO_INDICATION);
```

Purpose:

This procedure deletes a relationship which defines a local name for a relation. This procedure optionally deletes a relationship defining that relationships of the specified relation can emanate from nodes of the specified kind. 1

A RELATION_NAME relationship in the view identified by the open node handle VIEW is deleted. The relationship emanates from the NKI corresponding to the NKD identified by the open node handle NKD. The relationship terminates at the RI corresponding to the RD identified by the open node handle RD. 2

If DELETE_DEFINITION is true, the corresponding EMANATES relationship connecting the NKD and RD is deleted. The restrictions in Section 5.7.1.9 on page 335 on deletion of an EMANATES relationship apply. 3


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

The TIME_LIMIT parameter allows the specification of a time limit for the delay imposed on the DELETE_RELATION operation by the existence of locks on any node to be accessed.

Parameters:

**VIEW** is an open node handle identifying the VIEW node for the view from which the RELATION_NAME relationship is to be deleted.

**NKD** is an open node handle to an NKD with a corresponding NKI in the view.

**RD** is an open node handle to an RD with a corresponding RI in the view.

**DELETE_DEFINITION** is a Boolean indicating whether the corresponding EMANATES relationship is to be deleted.

**TIME_LIMIT** is a time limit on waiting for the unlocking of a node.

**STATUS** is an established status indicator.

Exceptions:

**NAME_ERROR** is raised if the NKI or RI is inaccessible [1].

**RELATIONSHIP_ERROR** is raised if the view does not contain an NKI or RI corresponding to the NKD or RD specified [2], if there is no RELATION_NAME relationship connecting the NKI and RI [2], or if the value of DELETE_DEFINITION is TRUE and there is no EMANATES relationship connecting the NKD and RD (the emanates property is inherited) [2].

**USE_ERROR** is raised a relationship to be deleted is predefine [63].

**STATUS_ERROR** is raised if VIEW, NKD or RD is not an open node handle [2] or if STATUS is not an established status indicator [3].

**NODE_KIND_ERROR** is raised if the open node handle VIEW does not identify a VIEW node [1], if the open node handle NKD does not identify an NKD [1] or if the open node handle RD does not identify an RD [1].

**CONFORMANCE_ERROR** is raised if the operation would result in an attribute, node or relationship not conforming to its definition or the constraints in force on that definition [3].

**DEFINITION_ERROR** is raised if the value of DELETE_DEFINITION is TRUE and a restriction on the deletion of an EMANATES relationship would be violated [2] or if the value of DELETE_DEFINITION is FALSE and the RELATION_NAME relationship to be deleted is the last one for the corresponding EMANATES relationship [8].

**LOCK_ERROR** is raised if a node handle to the NKI cannot be opened with intents WRITE. - RELATIONSHIPS due to outstanding locks [1] or if an open transaction not containing this operation contains other eligible operations whose effects determine whether the conditions for DEFINITION_ERROR apply [3].

**TRANSACTION_ERROR** is raised if the operation is included in more than one transaction [1] or if the operation is included in a transaction and the operation modifies a node which is not included in the same transaction [2].

**INTENT_VIOLATION** is raised if the node handle VIEW was not opened with an intent establishing the right to read relationships and an exclusive intent establishing the right to write relationships [1] or if the value of DELETE_DEFINITION is TRUE and the node handle NKD or RD was not opened with an intent establishing the rights to read and write relationships [1].


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

**ACCESS_VIOLATION** is raised if the value of the discretionary access control rights of the current process are insufficient to allow access to the NKI with intent WRITE_RELATIONSHIPS [1]. 21

**SECURITY_VIOLATION** is raised if the operation represents a violation of mandatory access controls. SECURITY_VIOLATION is raised only if the conditions for other exceptions are not present. 22

### 5.7.44 Respecifying source node kinds for a relation

```
procedure NAME_SOURCE
    (VIEW        : in    VIEW_NODE_TYPE;
     NKD         : in    BKD_TYPE;
     RD          : in    RD_TYPE;
     LOCAL_NAME  : in    RELATIONSHIP_KEY;
     STATUS      : in    STATUS_INDICATOR  : = NO_INDICATION);
```

Purpose:

This procedure defines for the specified node kind the local name of the specified relation. 1

VIEW is an open node handle to a VIEW node for a view containing an NKI and RI corresponding to the NKD and RD identified by the open node handles NKD and RD. A RELATION_NAME relationship with key list LOCAL_NAME is created which emanates from the NKI and terminates at the RI. This relationship establishes the resulting NAME key value as a relation name for relationships of the given relation emanating from nodes interpreted through the NKI. 2

The RD must be the target of an EMANATES relationship emanating from the NKD or a generalization thereof. This procedure does not modify any definition nodes. 3

Parameters:

**VIEW** is an open node handle to a VIEW node. 4

**NKD** is an open node handle to an NKD. 5

**RD** is an open node handle to an RD. 6

**LOCAL_NAME** is the key list for the RELATION_NAME relationship to be created. 7

**STATUS** is an established status indicator. 8

Exceptions:

**PATHNAME_SYNTAX_ERROR** is raised if the LOCAL_NAME given is syntactically illegal [2]. 9

**NAME_ERROR** is raised if the NKI or RI to be accessed is inaccessible [1]. 10

**RELATIONSHIP_ERROR** is raised if a RELATION_NAME relationship already exists with the identification given [3] or if LOCAL_NAME specifies a value for the same key more than once [5]. 11


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

**NOT_IN_VIEW_ERROR** is raised if a key name specified in LOCAL_NAME cannot be translated [1] or if LOCAL_NAME consists of a single relationship key designator and no key of the corresponding relation is visible [5]. 12

**AMBIGUOUS_VIEW_ERROR** is raised if a key name specified in LOCAL_NAME has an ambiguous translation [1], if a RELATION_NAME relationship already connects the NKI and RI [3] or if LOCAL_NAME consists of a single relationship key designator and more than one key of the corresponding relation is visible [4]. 13

**USE_ERROR** is raised if the view identified by VIEW does not contain an interpretation for the NKD or RD [20] or if an EMANATES relationship does not connect the NKD and RD [23]. 14

**STATUS_ERROR** is raised if VIEW, NKD or RD is not an open node handle [2] or if STATUS is not an established status indicator [3]. 15

**NODE_KIND_ERROR** is raised if the open node handle VIEW does not identify a VIEW node [1], if the open node handle NKD does not identify an NKD [1] or if the open node handle RD does not identify an RD [1]. 16

**CONFORMANCE_ERROR** is raised if the operation would result in an attribute, node or relationship not conforming to its definition or the constraints in force on that definition [3,10]. 17

**LOCK_ERROR** is raised if a node handle to the NKI or RI cannot be opened with intents to read and create relationships due to outstanding locks [1]. 18

**TRANSACTION_ERROR** is raised if the operation is included in more than one transaction [1] or if the operation is included in a transaction and the operation modifies a node which is not included in the same transaction [2]. 19

**INTENT_VIOLATION** is raised if the node handle NKD or RD was not opened with an intent establishing the rights to read relationships [1] or if the node handle VIEW was not opened with an intent establishing the right to read relationships and an exclusive intent establishing the right to write relationships [1]. 20

**ACCESS_VIOLATION** is raised if the discretionary access control rights of the current process are insufficient to allow access to the NKI or RI with intents to read and append relationships [1]. 21

**SECURITY_VIOLATION** is raised if the operation represents a violation of mandatory access controls. SECURITY_VIOLATION is raised only if the conditions for other exceptions are not present. 22

### 5.7.45 Specifying key attributes for a relation

```
procedure DEFINE_KEY
   (VIEW           : in    VIEW_NODE_TYPE;
    RD             : in    RD_TYPE;
    AD             : in    IDENTIFIER_AD_TYPE;
    LOCAL_NAME     : in    RELATIONSHIP_KEY;
    KEY            : in    RELATIONSHIP_KEY := EMPTY_KEY_LIST;
    REVERSE_KEY    : in    RELATIONSHIP_KEY := EMPTY_KEY_LIST;
    ATTRIBUTES     : in    ATTRIBUTE_LIST := EMPTY_LIST;
    STATUS         : in    STATUS_INDICATOR := NO_INDICATION);
```


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Purpose:

This procedure defines a new key for the specified relation.

A KEY relationship with key list KEY is created. The relationship emanates from the RD identified by the open node handle RD and terminates at the identifier AD identified by the open node handle AD. REVERSE_KEY provides the key list for the corresponding reverse relationship.

If there exists a KEY relationship emanating from a generalization of the RD and terminating at the specified AD, then the new relationship does not define a new key for the specified relation. (The key was already a property of the relation.) The new relationship can be used in specifying a more constrained value for the CONSTRAINTS attribute or a value for the DEFAULT_VALUE attribute.

If there exists a KEY relationship emanating from a specialization of the RD and terminating at the specified AD, then the new relationship does not define a new key for that specialized relation.

The ATTRIBUTES parameter provides initial values for attributes of the new relationship. ATTRIBUTES may include values for the optional, CAIS-controlled KEY relationship attributes DEFAULT_VALUE and CONSTRAINTS.

VIEW is an open node handle to a VIEW node for a view containing the AD and an RI corresponding to the RD. A KEY_NAME relationship with key list LOCAL_NAME is created which emanates from the RI and terminates at the AD. This relationship establishes the resulting NAME key value as a name for the given key on relationships interpreted through the RI.

Parameters:

**VIEW** is an open node handle to a VIEW node.

**RD** is an open node handle to an RD.

**AD** is an open node handle to an identifier AD.

**LOCAL_NAME** is the key list for the KEY_NAME relationship to be created emanating from the RI.

**KEY** is the key list for the KEY relationship to be created emanating from the RD.

**REVERSE_KEY** is the key list for the KEY_OF relationship to be created.

**ATTRIBUTES** is an empty or named list whose elements are used to establish initial values for attributes of the newly created KEY relationship emanating from the RD; each named item of the list specifies an attribute name and the value to be given to that attribute.

**STATUS** is an established status indicator.

Exceptions:

**PATHNAME_SYNTAX_ERROR** is raised if the LOCAL_NAME, KEY or REVERSE_KEY given is syntactically illegal [2].

**NAME_ERROR** is raised if the RI is inaccessible [1].

**RELATIONSHIP_ERROR** is raised if an ATTRIBUTE, KEY, ATTRIBUTE_OF, KEY_OF, ATTRIBUTE_NAME or KEY_NAME relationship already exists with the identification given [3] or if a key list specifies a value for the same key more than once [5].



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

**ATTRIBUTE_ERROR** is raised if an attribute list specifies a value for the same attribute more than once [6]. 18

**SYNTAX_ERROR** is raised if the ATTRIBUTES parameter is syntactically illegal [8]. 19

**NOT_IN_VIEW_ERROR** is raised if an attribute name specified in ATTRIBUTES, LOCAL_NAME, KEY or REVERSE_KEY cannot be translated [1] or if LOCAL_NAME, KEY or REVERSE_KEY consists of a single relationship key designator and no key of the corresponding relation is visible [5]. 20

**AMBIGUOUS_VIEW_ERROR** is raised if an attribute name specified in ATTRIBUTES, LOCAL_NAME, KEY or REVERSE_KEY has an ambiguous translation [1], if a KEY_NAME relationship already connects the specified RI and AD [3] or if KEY or REVERSE_KEY consists of a single relationship key designator and more than one key of the corresponding relation is visible [4]. 21

**PREDEFINED_ATTRIBUTE_ERROR** is raised if any attribute named in the ATTRIBUTES parameter is a CAIS-controlled attribute that is not identified in the Purpose section of this interface as an attribute which may be specified in that parameter [2] or if the AD is predefine and CAIS-controlled and the key property would be a new one for the relation [5]. 22

**USE_ERROR** is raised if the view identified by VIEW does not contain the AD or an interpretation for the RD [20] or if an ATTRIBUTE or KEY relationship already emanates from the RD and terminates at the AD [22]. 23

**STATUS_ERROR** is raised if VIEW, RD or AD is not an open node handle [2] or if STATUS is not an established status indicator [3]. 24

**NODE_KIND_ERROR** is raised if the open node handle VIEW does not identify a VIEW node [1], if the open node handle RD does not identify an RD [1] or if the open node handle AD does not identify an identifier AD [1]. 25

**CONFORMANCE_ERROR** is raised if the operation would result in an attribute, node or relationship not conforming to its definition or the constraints in force on that definition [3,7,8,9,10]. 26

**DEFINITION_ERROR** is raised if this operation is used to specify properties or constraints which conflict with those already defined for the attribute or the relation and its generalizations and specializations [3]. 27

**LOCK_ERROR** is raised if a node handle to the RI cannot be opened with intents to read and create relationships due to outstanding locks [1] or if an open transaction not containing this operation contains other eligible operations whose effects determine whether the conditions for DEFINITION_ERROR apply [3]. 28

**TRANSACTION_ERROR** is raised if the operation is included in more than one transaction [1] or if the operation is included in a transaction and the operation modifies a node which is not included in the same transaction [2]. 29

**INTENT_VIOLATION** is raised if the node handle RD or AD was not opened with an intent establishing the rights to read and append relationships [1] or if the node handle VIEW was not opened with an intent establishing the right to read relationships and an exclusive intent establishing the right to write relationships [1]. 30

**ACCESS_VIOLATION** is raised if the discretionary access control rights of the current process are insufficient to allow access to the RI with intents to read and append relationships [1]. 31


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

**SECURITY_VIOLATION** is raised if the operation represents a violation of mandatory access controls. SECURITY_VIOLATION is raised only if the conditions for other exceptions are not present. 32

### 5.7.46 Deleting key attributes for a relation

```
procedure DELETE_KEY
   (VIEW              : in  VIEW_NODE_TYPE;
    RD                : in  RD_TYPE;
    AD                : in  IDENTIFIER_AD_TYPE;
    DELETE_DEFINITION : in  BOOLEAN          := FALSE;
    TIME_LIMIT        : in  CAIS_DURATION    := LONG_DELAY;
    STATUS            : in  STATUS_INDICATOR := NO_INDICATION);
```

Purpose:

This procedure deletes a relationship which defies a local name for a key. This procedure optionally deletes a relationship defining the specified key for the specified relation. 1

A KEY_NAME relationship in the view identified by the open node handle VIEW is deleted. The relationship emanates from the RI corresponding to the RD identified by the open node handle RD. The relationship terminates at the AD identified by the open node handle AD. 2

If DELETE_DEFINITION is true, the corresponding KEY relationship connecting the RD and AD is deleted. The restrictions in Section 5.7.1.9 on page 335 on deletion of a KEY relationship apply. 3

The TIME_LIMIT parameter allows the specification of a time limit for the delay imposed on the DELETE_KEY operation by the existence of locks on any node to be accessed. 4

Parameters:

**VIEW** is an open node handle identifying the VIEW node for the view from which the KEY_NAME relationship is to be deleted. 5

**RD** is an open node handle to an RD with a corresponding RI in the view. 6

**AD** is an open node handle to an AD in the view. 7

**DELETE_DEFINITION** is a Boolean indicating whether the corresponding KEY relationship emanating from the RD is to be deleted. 8

**TIME_LIMIT** is a time limit on waiting for the unlocking of a node. 9

**STATUS** is an established status indicator. 10

Exceptions:

**NAME_ERROR** is raised if the RI is inaccessible. 11

**RELATIONSHIP_ERROR** is raised if the view does not contain an RI corresponding to the RD specified [2], if there is no KEY_NAME relationship connecting the RI and AD [2], or if the value of DELETE_DEFINITION is TRUE and there is no KEY relationship connecting the RD and AD (the key property is inherited) [2]. 12



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

**USE_ERROR** is raised a relationship to be deleted is predefine [63].

**STATUS_ERROR** is raised if VIEW, RD or AD is not an open node handle [2] or if STATUS is not an established status indicator [3].

**NODE_KIND_ERROR** is raised if the open node handle VIEW does not identify a VIEW node [1], if the open node handle RD does not identify an RD [1] or if the open node handle AD does not identify an AD [1].

**CONFORMANCE_ERROR** is raised if the operation would result in an attribute, node or relationship not conforming to its definition or the constraints in force on that definition [3].

**DEFINITION_ERROR** is raised if the value of DELETE_DEFINITION is TRUE and a restriction on the deletion of a KEY relationship would be violated [2] or if the value of DELETE_DEFINITION is FALSE and the KEY_NAME relationship to be deleted is the last one for the corresponding KEY relationship [8].

**LOCK_ERROR** is raised if a node handle to the RI cannot be opened with intents WRITE_-RELATIONSHIPS due to the existence of locks in conflict with that intent [1] or if an open transaction not containing this operation contains other eligible operations whose effects determine whether the conditions for DEFINITION_ERROR apply [3].

**TRANSACTION_ERROR** is raised if the operation is included in more than one transaction [1] or if the operation is included in a transaction and the operation modifies a node which is not included in the same transaction [2].

**INTENT_VIOLATION** is raised if the node handle VIEW was not opened with an intent establishing the right to read relationships and an exclusive intent establishing the right to write relationships [1] or if the value of DELETE_DEFINITION is TRUE and the node handle RD or AD was not opened with an intent establishing the rights to read and write relationships [1].

**ACCESS_VIOLATION** is raised if the value of the discretionary access control rights of the current process are insufficient to allow access to the RI with intent WRITE_RELATION-SHIPS [1].

**SECURITY_VIOLATION** is raised if the operation represents a violation of mandatory access controls. SECURITY_VIOLATION is raised only if the conditions for other exceptions are not present.

### 5.7.47 Respecifying key attributes for a relation

```
procedure NAME_KEY
    (VIEW        : in    VIEW_NODE_TYPE;
     RD          : in    RD_TYPE;
     AD          : in    IDENTIFIER_AD_TYPE;
     LOCAL_NAME  : in    RELATIONSHIP_KEY;
     STATUS      : in    STATUS_INDICATOR := NO_INDICATION);
```

Purpose:

This procedure defines for the specified relation the local name of the specified key.

VIEW is an open node handle to a VIEW node for a view containing an RI corresponding to the RD identified by the open node handle RD and the AD identified by the open node handle



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

AD. A KEY_NAME relationship with key list LOCAL_NAME is created which emanates from the RI and terminates at the AD. This relationship establishes the resulting NAME key value as a name for the given key on relationships interpreted through the RI.

The identifier AD identified by AD must be the target of a KEY relationship emanating from the RD identified by RD or a generalization of that relation. This procedure does not modify any definition nodes.

Parameters:

**VIEW** is an open node handle to a VIEW node.

**RD** is an open node handle to an RD.

**AD** is an open node handle to an AD.

**LOCAL_NAME** is the key list for the KEY_NAME relationship to be created.

**STATUS** is an established status indicator.

Exceptions:

**PATHNAME_SYNTAX_ERROR** is raised if the LOCAL_NAME given is syntactically illegal [2].

**NAME_ERROR** is raised if the RI to be accessed is inaccessible [1].

**RELATIONSHIP_ERROR** is raised if a KEY_NAME relationship already exists with the identification given [3] or if LOCAL_NAME specifies a value for the same key more than once [5].

**NOT_IN_VIEW_ERROR** is raised if a key name specified in LOCAL_NAME cannot be translated [1] or if LOCAL_NAME consists of a single relationship key designator and no key of the corresponding relation is visible [5].

**AMBIGUOUS_VIEW_ERROR** is raised if a key name specified in LOCAL_NAME has an ambiguous translation [1], if a KEY_NAME relationship already connects the specified RI and AD [3] or if LOCAL_NAME consists of a single relationship key designator and more than one key of the corresponding relation is visible [4].

**USE_ERROR** is raised if the view identified by VIEW does not contain the AD and an interpretation for the RD [20] or if a KEY relationship does not connect the RD and AD [23].

**STATUS_ERROR** is raised if VIEW, RD or AD is not an open node handle [2] or if STATUS is not an established status indicator [3].

**NODE_KIND_ERROR** is raised if the open node handle VIEW does not identify a VIEW node [1], if the open node handle RD does not identify an RD [1] or if the open node handle AD does not identify an identifier AD [1].

**CONFORMANCE_ERROR** is raised if the operation would result in an attribute, node or relationship not conforming to its definition or the constraints in force on that definition [3,10].

**LOCK_ERROR** is raised if a node handle to the RI cannot be opened with intents to read and create relationships due to the existence of locks in conflict with that intent [1].



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

**TRANSACTION_ERROR** is raised if the operation is included in more than one transaction [1] or if the operation is included in a transaction and the operation modifies a node which is not included in the same transaction [2].

**INTENT_VIOLATION** is raised if the node handle RD or AD was not opened with an intent establishing the rights to read relationships [1] or if the node handle VIEW was not opened with an intent establishing the right to read relationships and an exclusive intent establishing the right to write relationships [1].

**ACCESS_VIOLATION** is raised if the discretionary access control rights of the current process are insufficient to allow access to the RI with intents to read and append relationships [1].

**SECURITY_VIOLATION** is raised if the operation represents a violation of mandatory access controls. SECURITY_VIOLATION is raised only if the conditions for other exceptions are not present.

### 5.7.48 Specifying target node kinds for a relation

```
procedure DEFINE_TARGET
    (NKD           : in  NKD_TYPE;
     RD            : in  RD_TYPE;
     KEY           : in  RELATIONSHIP_KEY  := EMPTY_KEY_LIST;
     REVERSE_KEY   : in  RELATIONSHIP_KEY  := EMPTY_KEY_LIST;
     ATTRIBUTES    : in  ATTRIBUTE_LIST    := EMPTY_LIST;
     STATUS        : in  STATUS_INDICATOR  := NO_INDICATION);
```

Purpose:

This procedure defines that relationships of the specified relation can terminate at nodes of the specified kind.

A TERMINATES relationship with key list KEY is created. The relationship emanates from the NKD specified by the open node handle NKD and terminates at the RD specified by the open node handle RD. REVERSE_KEY provides the key list for the corresponding reverse relationship.

This operation requires that there be a TERMINATES relationship terminating at each of the RD's immediate generalizations and emanating from the NKD or one of its generalizations.

The ATTRIBUTES parameter provides initial values for attributes of the relationship. ATTRIBUTES cannot specify values for any CAIS-controlled attributes.

Parameters:

**NKD** is an open node handle to an NKD.

**RD** is an open node handle to an RD.

**KEY** is a key list for the TERMINATES relationship to be created.

**REVERSE_KEY** is the key list for the TERMINATES_AT relationship to be created.

**ATTRIBUTES** is an empty or named list whose elements are used to establish initial values for attributes of the newly created TERMINATES relationship; each named item of the list specifies an attribute name and the value to be given to that attribute.


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

**STATUS** is an established status indicator.

Exceptions:

**PATHNAME_SYNTAX_ERROR** is raised if the KEY or REVERSE_KEY given is syntactically illegal [2].

**RELATIONSHIP_ERROR** is raised if a TERMINATES or TERMINATES_AT relationship already exists with the identification given [3] or if a key list specifies a value for the same key more than once [5].

**ATTRIBUTE_ERROR** is raised if an attribute list specifies a value for the same attribute more than once [6].

**SYNTAX_ERROR** is raised if the ATTRIBUTES parameter is syntactically illegal [8].

**NOT_IN_VIEW_ERROR** is raised if an attribute name specified in ATTRIBUTES, KEY or REVERSE_KEY cannot be translated [1] or if KEY or REVERSE_KEY consists of a single relationship key designator and no key of the corresponding relation is visible [5].

**AMBIGUOUS_VIEW_ERROR** is raised if an attribute name specified in ATTRIBUTES, KEY or REVERSE_KEY has an ambiguous translation [1] or if KEY or REVERSE_KEY consists of a single relationship key designator and more than one key of the corresponding relation is visible [4].

**PREDEFINED_ATTRIBUTE_ERROR** is raised if any attribute named in the ATTRIBUTES parameter is a CAIS-controlled attribute [2].

**USE_ERROR** is raised if a TERMINATES relationship already emanates from the NKD and terminates at the RD [22].

**STATUS_ERROR** is raised if NKD or RD is not an open node handle [2] or if STATUS is not an established status indicator [3].

**NODE_KIND_ERROR** is raised if the open node handle NKD does not identify an NKD [1] or if the open node handle RD does not identify an RD [1].

**CONFORMANCE_ERROR** is raised if the operation would result in an attribute, node or relationship not conforming to its definition or the constraints in force on that definition [3,7,8,9,10].

**DEFINITION_ERROR** is raised if this operation is used to specify properties or constraints which conflict with those already defied for the relation and its generalizations and specializations [3].

**LOCK_ERROR** is raised if an open transaction not containing this operation contains other eligible operations whose effects determine whether the conditions for DEFINITION_ERROR apply [3].

**TRANSACTION_ERROR** is raised if the operation is included in more than one transaction [1] or if the operation is included in a transaction and the operation modifies a node which is not included in the same transaction [2].

**INTENT_VIOLATION** is raised if the node handle NKD or RD was not opened with an intent establishing the rights to read and append relationships [1].

**SECURITY_VIOLATION** is raised if the operation represents a violation of mandatory access controls. SECURITY_VIOLATION is raised only if the conditions for other exceptions are not present.



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

### 5.7.49 Deleting target node kinds for a relation

```
procedure DELETE_TARGET
    (NKD          : in  NKD_TYPE;
     RD           : in  RD_TYPE;
     TIME_LIMIT   : in  CAIS_DURATION       := LONG_DELAY;
     STATUS       : in  STATUS_INDICATOR := NO_INDICATION);
```

Purpose:

This procedure deletes a relationship defining that relationships of the specified relation can terminate at nodes of the specified kind.

A TERMINATES relationship is deleted. This relationship emanates from the NKD identified by the open node handle NKD and terminates at the RD identified by the open node handle RD. The restrictions in Section 5.7.1.9 on page 335 on deletion of a TERMINATES relationship apply.

The TIME_LIMIT parameter allows the specification of a time limit for the delay imposed on the DELETE_TARGET operation by the existence of locks on any node to be accessed.

Parameters:

**NKD** is an open node handle to an NKD.

**RD** is an open node handle to an RD.

**TIME_LIMIT** is a time limit on waiting for the unlocking of a node.

**STATUS** is an established status indicator.

Exceptions:

**RELATIONSHIP_ERROR** is raised if a TERMINATES relationship does not exist with the source and target nodes specified [2].

**USE_ERROR** is raised the relationship to be deleted is predefine [63].

**STATUS_ERROR** is raised if NKD or RD is not an open node handle [2] or if STATUS is not an established status indicator [3].

**NODE_KIND_ERROR** is raised if the open node handle NKD does not identify an NKD [1] or if the open node handle RD does not identify an RD [1].

**CONFORMANCE_ERROR** is raised if the operation would result in an attribute, node or relationship not conforming to its definition or the constraints in force on that definition [3].

**DEFINITION_ERROR** is raised if a restriction on the deletion of a TERMINATES relationship would be violated [2].

**LOCK_ERROR** is raised if an open transaction not containing this operation contains other eligible operations whose effects determine whether the conditions for DEFINITION_ERROR apply [3].

**TRANSACTION_ERROR** is raised if the operation is included in more than one transaction [1] or if the operation is included in a transaction and the operation modifies a node which is not included in the same transaction [2].


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

**INTENT_VIOLATION** is raised if the node handles NKD and RD were not opened with intents establishing the rights to read and write relationships [1]. 16

**SECURITY_VIOLATION** is raised if the operation represents a violation of mandatory access controls. SECURITY_VIOLATION is raised only if the conditions for other exceptions are not present. 17

### 5.7.50 Specifying attributes for node kinds and relations

```
procedure DEFINE_ATTRIBUTE
   (VIEW          : in    VIEW_NODE_TYPE;
    NODE          : in    DEFINITION_NODE_TYPE;
    AD            : in    AD_TYPE;
    LOCAL_NAME    : in    RELATIONSHIP_KEY;
    KEY           : in    RELATIONSHIP_KEY := EMPTY_KEY_LIST;
    REVERSE_KEY   : in    RELATIONSHIP_KEY := EMPTY_KEY_LIST;
    ATTRIBUTES    : in    ATTRIBUTE_LIST   := EMPTY_LIST;
    STATUS        : in    STATUS_INDICATOR := NO_INDICATION);
```

Purpose:

This procedure defines a new attribute for the specified relation or node kind. 1

An ATTRIBUTE relationship with key list KEY is created. The source node is the NKD or RD identified by the open node handle NODE. The target node is the AD identified by the open node handle AD. REVERSE_KEY provides the key list for the corresponding reverse relationship. 2

If there exists an ATTRIBUTE relationship emanating from a generalization of the definition node and terminating at the specified AD, then the new relationship does not define a new attribute for the specified definition. (The attribute was already a property.) The new relationship can be used in specifying a more constrained value for the CONSTRAINTS attribute or a value for the DEFAULT_VALUE attribute. 3

If there exists an ATTRIBUTE relationship emanating from a specialization of the definition node and terminating at the specified AD, then the new relationship does not define a new attribute for that specialized node kind or relation. 4

The ATTRIBUTES parameter provides initial values for attributes of the new relationship. ATTRIBUTES may include values for the optional, CAIS-controlled ATTRIBUTE relationship attributes DEFAULT_VALUE and CONSTRAINTS. 5

VIEW is an open node handle to a VIEW node for a view containing the AD and an NKI or RI corresponding to the NKD or RD. An ATTRIBUTE_NAME relationship with key list LOCAL_NAME is created which emanates from the interpretation node and terminates at the AD. This relationship establishes the resulting NAME key value as a name for instances of the given attribute on instances interpreted through the interpretation. 6

Parameters:

**VIEW** is an open node handle to a VIEW node. 7

**NODE** is an open node handle to an NKD or RD. 8



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

**AD** is an open node handle to an AD.

**LOCAL_NAME** is the key list for the ATTRIBUTE_NAME relationship to be created.

**KEY** is a key list for the ATTRIBUTE relationship to be created.

**REVERSE_KEY** is the key list for the ATTRIBUTE_OF relationship to be created.

**ATTRIBUTES** is an empty or named list used to establish initial values for attributes of the newly created ATTRIBUTE relationship; each named item of the list specifies an attribute name and the value to be given to that attribute.

**STATUS** is an established status indicator.

Except ions:

**PATHNAME_SYNTAX_ERROR** is raised if the LOCAL_NAME, KEY or REVERSE_-KEY given is syntactically illegal [2].

**NAME_ERROR** is raised if the NKI or RI to be accessed is inaccessible [1].

**RELATIONSHIP_ERROR** is raised if an ATTRIBUTE, ATTRIBUTE_OF or ATTRIBUTE_-NAME relationship already exists with the identification given [3] or if a key list specifies a value for the same key more than once [5].

**ATTRIBUTE_ERROR** is raised if an attribute list specifies a value for the same attribute more than once [6].

**SYNTAX_ERROR** is raised if the ATTRIBUTES parameter is syntactically illegal [8].

**NOT_IN_VIEW_ERROR** is raised if an attribute name specified in ATTRIBUTES, LO-CAL_NAME, KEY or REVERSE_KEY cannot be translated [1] or if LOCAL_NAME, KEY or REVERSE_KEY consists of a single relationship key designator and no key of the corresponding relation is visible [5].

**AMBIGUOUS_VIEW_ERROR** is raised if an attribute name specified in ATTRIBUTES, LOCAL_NAME, KEY or REVERSE_KEY has an ambiguous translation [1], if an AT-TRIBUTE_NAME relationship already connects the specified interpretation and AD [3] or if LOCAL_NAME, KEY or REVERSE_KEY consists of a single relationship key designator and more than one key of the corresponding relation is visible [4].

**PREDEFINED_ATTRIBUTE_ERROR** is raised if any attribute named in the ATTRI-BUTES parameter is a CAIS-controlled attribute that is not identified in the Purpose section of this interface as an attribute which may be specified in that parameter [2] or if the AD is predefined and CAIS-controlled and the attribute property would be a new one for the definition [5].

**USE_ERROR** is raised if the view identified by VIEW does not contain the AD and an interpretation for the NKD or RD [20] or if an ATTRIBUTE relationship already emanates from the NKD or RD and terminates at the AD [22].

**STATUS_ERROR** is raised if VIEW, NODE or AD is not an open node handle [2] or if STATUS is not an established status indicator [3].

**NODE_KIND_ERROR** is raised if the open node handle VIEW does not identify a VIEW node [1], if the open node handle NODE does not identify an NKD or RD [1] or if the open node handle AD does not identify an AD [1].

**CONFORMANCE_ERROR** is raised if the operation would result in an attribute, node or relationship not conforming to its definition or the constraints in force on that definition [3,7,8,9,10].



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

**DEFINITION_ERROR** is raised if this operation is used to specify properties or constraints which conflict with those already defined for an attribute, the attribute kind or the node kind or relation [3]. 27

**LOCK_ERROR** is raised if a node handle to the NKI or RI cannot be opened with intents to read and create relationships due to the existence of locks in conflict with that intent [1] or if an open transaction not containing this operation contains other eligible operations whose effects determine whether the conditions for DEFINITION_ERROR apply [3]. 28

**TRANSACTION_ERROR** is raised if the operation is included in more than one transaction [1] or if the operation is included in a transaction and the operation modifies a node which is not included in the same transaction [2]. 29

**INTENT_VIOLATION** is raised if the node handle NODE or AD was not opened with an intent establishing the rights to read and append relationships [1] or if the node handle VIEW was not opened with an intent establishing the right to read relationships and an exclusive intent establishing the right to write relationships [1]. 30

**ACCESS_VIOLATION** is raised if the discretionary access control rights of the current process are insufficient to allow access to the NKI or RI with intents to read and append relationships [1]. 31

**SECURITY_VIOLATION** is raised if the operation represents a violation of mandatory access controls. SECURITY_VIOLATION is raised only if the conditions for other exceptions are not present. 32

### 5.7.51 Deleting attributes for node kinds and relations

```
procedure DELETE_ATTRIBUTE
   (VIEW                : in  VIEW_NODE_TYPE;
    NODE                : in  DEFINITION_NODE_TYPE;
    AD                  : in  AD_TYPE;
    DELETE_DEFINITION   : in  BOOLEAN           := FALSE;
    TIME_LIMIT          : in  CAIS_DURATION     := LONG_DELAY;
    STATUS              : in  STATUS_INDICATOR  := NO_INDICATION);
```

Purpose:

This procedure deletes a relationship which defines a local name for an attribute. This procedure optionally deletes a relationship defining the attribute for the specified definition. 1

An ATTRIBUTE_NAME relationship in the view identified by the open node handle VIEW is deleted. The relationship emanates from the NKI or RI corresponding to the NKD or RD identified by the open node handle NODE. The relationship terminates at the AD identified by the open node handle AD. 2

If DELETE_DEFINITION is true, the corresponding ATTRIBUTE relationship emanating from the NKD or RD identified by NODE and terminating at the AD is also deleted. The restrictions in Section 5.7.1.9 on page 335 on deletion of an ATTRIBUTE relationship apply. 3

The TIME_LIMIT parameter allows the specification of a time limit for the delay imposed on the DELETE_ATTRIBUTE operation by the existence of locks on any node to be accessed. 4



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Parameters:

**VIEW** is an open node handle identifying the VIEW node for the view from which the ATTRIBUTE_NAME relationship is to be deleted.

**NODE** is an open handle to an NKD or RD.

**AD** is an open node handle to an AD in the view.

**DELETE_DEFINITION** is a Boolean indicating whether the corresponding ATTRIBUTE relationship emanating from the definition node is to be deleted.

**TIME_LIMIT** is a time limit on waiting for the unlocking of a node.

**STATUS** is an established status indicator.

Exceptions:

**NAME_ERROR** is raised if the interpretation node corresponding to the definition node identified by NODE is inaccessible [1].

**RELATIONSHIP_ERROR** is raised if the view does not contain an interpretation corresponding to the definition specified by NODE [2], if there is no ATTRIBUTE_NAME relationship connecting the interpretation and AD [2], or if the value of DELETE_DEFINITION is TRUE and there is no ATTRIBUTE relationship connecting the definition and AD (the attribute property is inherited) [2].

**USE_ERROR** is raised a relationship to be deleted is predefine [63].

**STATUS_ERROR** is raised if VIEW, NODE or AD is not an open node handle [2] or if STATUS is not an established status indicator [3].

**NODE_KIND_ERROR** is raised if the open node handle VIEW does not identify a VIEW node [1], if the open node handle NODE does not identify an NKD or RD [1] or if the open node handle AD does not identify an AD [1].

**CONFORMANCE_ERROR** is raised if the operation would result in an attribute, node or relationship not conforming to its definition or the constraints in force on that definition [3].

**DEFINITION_ERROR** is raised if the value of DELETE_DEFINITION is TRUE and a restriction on the deletion of an ATTRIBUTE relationship would be violated [2] or if the value of DELETE_DEFINITION is FALSE and the ATTRIBUTE_NAME relationship to be deleted is the last one for the corresponding ATTRIBUTE relationship [8].

**LOCK_ERROR** is raised if a node handle to the interpretation node cannot be opened with intents WRITE_RELATIONSHIPS due to the existence of locks in conflict with that intent [1] or if an open transaction not containing this operation contains other eligible operations whose effects determine whether the conditions for DEFINITION_ERROR apply [3].

**TRANSACTION_ERROR** is raised if the operation is included in more than one transaction [1] or if the operation is included in a transaction and the operation modifies a node which is not included in the same transaction [2].

**INTENT_VIOLATION** is raised if the node handle VIEW was not opened with an intent establishing the right to read relationships and an exclusive intent establishing the right to write relationships [1] or if the value of DELETE_DEFINITION is TRUE and the node handle NODE or AD was not opened with an intent establishing the rights to read and write relationships [1].



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

**ACCESS_VIOLATION** is raised if the value of the discretionary access control rights of the current process are insufficient to allow access to the interpretation node with intent WRITE_RELATIONSHIPS [1]. 21

**SECURITY_VIOLATION** is raised if the operation represents a violation of mandatory access controls. SECURITY_VIOLATION is raised only if the conditions for other exceptions are not present. 22

### 5.7.52 Respecifying attributes for node kinds and relations

```
procedure NAME_ATTRIBUTE
    (VIEW          : in  VIEW_NODE_TYPE;
     DEFINITION    : in  DEFINITION_NODE_TYPE;
     AD            : in  AD_TYPE;
     LOCAL_NAME    : in  RELATIONSHIP_KEY;
     STATUS        : in  STATUS_INDICATOR := NO_INDICATION);
```

Purpose:

This procedure defines for the specified relation or node kind the local name of the specified attribute. 1

VIEW is an open node handle to a VIEW node for a view containing an interpretation corresponding to the NKD or RD identified by the open node handle DEFINITION and the AD identified by the open node handle AD. An ATTRIBUTE_NAME relationship with key list LOCAL_NAME is created which emanates from the interpretation node and terminates at the AD. This relationship establishes the resulting NAME key value as a name for the given attribute on instances of the definition. 2

The AD identified by AD must be the target of an ATTRIBUTE relationship which emanates from the definition node identified by DEFINITION or a generalization thereof. This procedure does not modify any definition nodes. 3

Parameters:

**VIEW** is an open node handle to a VIEW node. 4

**DEFINITION** is an open node handle to an NKD or RD. 5

**AD** is an open node handle to an AD. 6

**LOCAL_NAME** is the key list for the ATTRIBUTE_NAME relationship to be created. 7

**STATUS** is an established status indicator. 8

Exceptions:

**PATHNAME_SYNTAX_ERROR** is raised if the LOCAL_NAME given is syntactically illegal [2]. 9

**NAME_ERROR** is raised if an interpretation node to be accessed is inaccessible [1]. 10


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

**RELATIONSHIP_ERROR** is raised if an ATTRIBUTE_NAME relationship already exists with the identification given [3] or if LOCAL_NAME specifies a value for the same key more than once [5]. 11

**NOT_IN_VIEW_ERROR** is raised if a key name specified in LOCAL_NAME cannot be translated [1] or if LOCAL_NAME consists of a single relationship key designator and no key of the corresponding relation is visible [5]. 12

**AMBIGUOUS_VIEW_ERROR** is raised if a key name specified in LOCAL_NAME has an ambiguous translation [1], if an ATTRIBUTE_NAME relationship already connects the specified interpretation and AD [3] or if LOCAL_NAME consists of a single relationship key designator and more than one key of the corresponding relation is visible [4]. 13

**USE_ERROR** is raised if the view identified by VIEW does not contain the AD and an interpretation for the definition [20] or if an ATTRIBUTE relationship does not connect the definition and AD [23]. 14

**STATUS_ERROR** is raised if VIEW, DEFINITION or AD is not an open node handle [2] or if STATUS is not an established status indicator [3]. 15

**NODE_KIND_ERROR** is raised if the open node handle VIEW does not identify a VIEW node [1], if the open node handle DEFINITION does not identify an NKD or RD [1] or if the open node handle AD does not identify an AD [1]. 16

**CONFORMANCE_ERROR** is raised if the operation would result in an attribute, node or relationship not conforming to its definition or the constraints in force on that definition [3,10]. 17

**LOCK_ERROR** is raised if a node handle to the interpretation cannot be opened with intents to read and create relationships due to the existence of locks in conflict with that intent [1]. 18

**TRANSACTION_ERROR** is raised if the operation is included in more than one transaction [1] or if the operation is included in a transaction and the operation modifies a node which is not included in the same transaction [2]. 19

**INTENT_VIOLATION** is raised if the node handle DEFINITION or AD was not opened with an intent establishing the rights to read relationships [1] or if the node handle VIEW was not opened with an intent establishing the right to read relationships and an exclusive intent establishing the right to write relationships [1]. 20

**ACCESS_VIOLATION** is raised if the discretionary access control rights of the current process are insufficient to allow access to the interpretation node with intents to read and append relationships [1]. 21

**SECURITY_VIOLATION** is raised if the operation represents a violation of mandatory access controls. SECURITY_VIOLATION is raised only if the conditions for other exceptions are not present. 22

### 5.7.53 Specifying an attribute kind component

```
procedure DEFINE_COMPONENT
    (STRUCTURED_AKD : in     AKD_TYPE;
     COMPONENT_AKD  : in     AKD_TYPE;
     KEY            : in     RELATIONSHIP_KEY  := EMPTY_KEY_LIST;
     REVERSE_KEY    : in     RELATIONSHIP_KEY  := EMPTY_KEY_LIST;
     ATTRIBUTES     : in     ATTRIBUTE_LIST    := EMPTY_LIST;
     STATUS         : in     STATUS_INDICATOR  := NO_INDICATION);
```


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Purpose:

This procedure defines a new component for the specified uniform or composite list attribute kind (see Section 5.7.1.6.8 on page 323 and Section 5.7.1.6.9 on page 324).

A COMPONENT relationship with key list KEY is created. STRUCTURED_AKD is an open node handle identifying the source node, which must be a uniform list AKD with no emanating COMPONENT relationships or a composite list AKD. COMPONENT_AKD is an open node handle identifying the target node, which must be an AKD. REVERSE_KEY provides the key list for the corresponding reverse relationship.

If the composite list AKD is an ordered composite list AKD, KEY can include a designator for the key POSITION. If the composite list AKD is a named composite list AKD, KEY can include a designator for the key NAME.

If STRUCTURED_AKD identifies an AKD which is an alternative or a component of another AKD, this operation must be a legal modification of that other attribute kind. It must also be a legal modification of any AD which is the source of an ATTRIBUTE_KIND relationship terminating at the AKD.

The ATTRIBUTES parameter provides initial values for attributes of the new relationship. ATTRIBUTES may include values for the optional, CAIS-controlled COMPONENT relationship attribute CONSTRAINTS and the required attribute OPTIONAL.

Parameters:

**STRUCTURED_AKD** is an open node handle identifying a uniform list or composite list AKD.

**COMPONENT_AKD** is an open node handle identifying an AKD.

**KEY** is a key list for the COMPONENT relationship to be created.

**REVERSE_KEY** is a key list for the COMPONENT_OF relationship to be created.

**ATTRIBUTES** is an empty or named list whose elements are used to establish initial values for attributes of the newly created COMPONENT relationship; each named item of the list specifies an attribute name and the value to be given to that attribute.

**STATUS** is an established status indicator.

Exceptions:

**PATHNAME_SYNTAX_ERROR** is raised if the KEY or REVERSE_KEY given is syntactically illegal [2].

**RELATIONSHIP_ERROR** is raised if a COMPONENT or COMPONENT_OF relationship already exists with the identification given [3] or if a key list specifies a value for the same key more than once [5].

**ATTRIBUTE_ERROR** is raised if an attribute list specifies a value for the same attribute more than once [6].

**SYNTAX_ERROR** is raised if the ATTRIBUTES parameter is syntactically illegal [8].

**NOT_IN_VIEW_ERROR** is raised if an attribute name specified in ATTRIBUTES, KEY or REVERSE_KEY cannot be translated [1] or if KEY or REVERSE_KEY consists of a single relationship key designator and no key of the corresponding relation is visible [5].


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

**AMBIGUOUS_VIEW_ERROR** is raised if an attribute name specified in ATTRIBUTES, KEY or REVERSE_KEY has an ambiguous translation [1] or if KEY or REVERSE_KEY consists of a single relationship key designator and more than one key of the corresponding relation is visible [4]. 17

**PREDEFINED_ATTRIBUTE_ERROR** is raised if any attribute named in the ATTRIBUTES parameter is a CAIS-controlled attribute that is not identified in the Purpose section of this interface as an attribute which may be specified in that parameter [2]. 18

**USE_ERROR** is raised if any key named in the KEY parameter is a CAIS-controlled key that is not identified in the Purpose section of this interface as a key which may be specified in that parameter for the given COMPONENT relation [24]. 19

**STATUS_ERROR** is raised if STRUCTURED_AKD or COMPONENT_AKD is not an open node handle [2] or if STATUS is not an established status indicator [3]. 20

**NODE_KIND_ERROR** is raised if the open node handle STRUCTURED_AKD does not identify a uniform or composite list AKD [1] or if the open node handle COMPONENT_AKD does not identify an AKD [1]. 21

**CONFORMANCE_ERROR** is raised if the operation would result in an attribute, node or relationship not conforming to its definition or the constraints in force on that definition [3,7,8,9,10]. 22

**DEFINITION_ERROR** is raised if this operation is used to specify properties or constraints which conflict with those already defined for the attribute or the component [3]. 23

**LOCK_ERROR** is raised if an open transaction not containing this operation contains other eligible operations whose effects determine whether the conditions for DEFINITION_ERROR apply [3]. 24

**TRANSACTION_ERROR** is raised if the operation is included in more than one transaction [1] or if the operation is included in a transaction and the operation modifies a node which is not included in the same transaction [2]. 25

**INTENT_VIOLATION** is raised if the node handle STRUCTURED_AKD was not opened with an intent establishing the rights to read and append relationships [1] or if COMPONENT_AKD was not opened with an intent establishing the rights to read attributes and to read and append relationships [1]. 26

**SECURITY_VIOLATION** is raised if the operation represents a violation of mandatory access controls. SECURITY_VIOLATION is raised only if the conditions for other exceptions are not present. 27

### 5.7.54 Deleting an attribute kind component

```
procedure DELETE_COMPONENT
    (STRUCTURED_AKD : in     AKD_TYPE;
     COMPONENT_AKD  : in     AKD_TYPE;
     TIME_LIMIT     : in     CAIS_DURATION     := LONG_DELAY;
     STATUS         : in     STATUS_INDICATOR  := NO_INDICATION);
```

Purpose:

This procedure deletes a relationship which defines a component for an attribute kind. 1



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

A COMPONENT relationship is deleted. The relationship emanates from the AKD identified by the open node handle STRUCTURED_AKD and terminates at the AKD identified by the open node handle COMPONENT_AKD. The restrictions in Section 5.7.1.9 on page 335 on deletion of a COMPONENT relationship apply.

The TIME_LIMIT parameter allows the specification of a time limit for the delay imposed on the DELETE_COMPONENT operation by the existence of locks on any node to be accessed.

Parameters:

**STRUCTURED_AKD** is an open handle to a composite or uniform list AKD.

**COMPONENT_AKD** is an open handle to the target AKD for the COMPONENT relationship to be deleted.

**TIME_LIMIT** is a time limit on waiting for the unlocking of a node.

**STATUS** is an established status indicator.

Exceptions:

**RELATIONSHIP_ERROR** is raised if a COMPONENT relationship does not exist with the specified source and target nodes [2].

**USE_ERROR** is raised the relationship to be deleted is predefine [63].

**STATUS_ERROR** is raised if STRUCTURED_AKD or COMPONENT_AKD is not an open node handle [2] or if STATUS is not an established status indicator [3].

**NODE_KIND_ERROR** is raised if STRUCTURED_AKD does not identify a composite or uniform list AKD [1] or if the open node handle COMPONENT_AKD does not identify an AKD [1].

**CONFORMANCE_ERROR** is raised if the operation would result in an attribute, node or relationship not conforming to its definition or the constraints in force on that definition [3].

**DEFINITION_ERROR** is raised if a restriction on the deletion of a COMPONENT relationship would be violated [2].

**LOCK_ERROR** is raised if an open transaction not containing this operation contains other eligible operations whose effects determine whether the conditions for DEFINITION_ERROR apply [3].

**TRANSACTION_ERROR** is raised if the operation is included in more than one transaction [1] or if the operation is included in a transaction and the operation modifies a node which is not included in the same transaction [2].

**INTENT_VIOLATION** is raised if the node handle STRUCTURED_AKD or COMPONENT_AKD was not opened with an intent establishing the rights to read and write relationships [1].

**SECURITY_VIOLATION** is raised if the operation represents a violation of mandatory access controls. SECURITY_VIOLATION is raised only if the conditions for other exceptions are not present.



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

### 5.7.55 Specifying a variant attribute alternative

```
procedure DEFINE_ALTERNATIVE
    (VAKD                : in  VAKD_TYPE;
     ALTERNATIVE_AKD     : in  AKD_TYPE;
     KEY                 : in  RELATIONSHIP_KEY   := EMPTY_KEY_LIST;
     REVERSE_KEY         : in  RELATIONSHIP_KEY   := EMPTY_KEY_LIST;
     ATTRIBUTES          : in  ATTRIBUTE_LIST     := EMPTY_LIST;
     STATUS              : in  STATUS_INDICATOR   := NO_INDICATION);
```

Purpose:

This procedure defies an alternative for the specified variant attribute kind (see Section 5.7.1.6.6 on page 322).

An ALTERNATIVE relationship with key list KEY is created. VAKD is an open node handle identifying the source node, which must be a variant AKD. ALTERNATIVE_AKD is an open node handle identifying the target node, which must be an AKD. REVERSE_KEY provides the key list for the corresponding reverse relationship.

If VAKD identifies an AKD which is an alternative or a component of another, this operation must be a legal modification of that other attribute kind. It must also be a legal modification of any attribute definition which is the source of an ATTRIBUTE_KIND relationship terminating at the AKD.

The ATTRIBUTES parameter provides initial values for attributes of the new relationship. ATTRIBUTES may include values for the optional, CAIS-controlled ALTERNATIVE relationship attribute CONSTRAINTS.

Parameters:

**VAKD** is an open node handle identifying a variant AKD.

**ALTERNATIVE_AKD** is an open node handle identifying an AKD.

**KEY** is a key list for the ALTERNATIVE relationship to be created.

**REVERSE_KEY** is a key list for the ALTERNATIVE_OF relationship to be created.

**ATTRIBUTES** is an empty or named list whose elements are used to establish initial values for attributes of the newly created ALTERNATIVE relationship; each named item of the list specifies an attribute name and the value to be given to that attribute.

**STATUS** is an established status indicator.

Exceptions:

**PATHNAME_SYNTAX_ERROR** is raised if the KEY or REVERSE_KEY given is syntactically illegal [2].

**RELATIONSHIP_ERROR** is raised if an ALTERNATIVE or ALTERNATIVE_OF relationship already exists with the identification given [3] or if a key list specifies a value for the same key more than once [5].


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

**ATTRIBUTE_ERROR** is raised if an attribute list specifies a value for the same attribute more than once [6]. 13

**SYNTAX_ERROR** is raised if the ATTRIBUTES parameter is syntactically illegal [8]. 14

**NOT_IN_VIEW_ERROR** is raised if an attribute name specified in ATTRIBUTES, KEY or REVERSE_KEY cannot be translated [1] or if KEY or REVERSE_KEY consists of a single relationship key designator and no key of the corresponding relation is visible [5]. 15

**AMBIGUOUS_VIEW_ERROR** is raised if an attribute name specified in ATTRIBUTES, KEY or REVERSE_KEY has an ambiguous translation [1] or if KEY or REVERSE_KEY consists of a single relationship key designator and more than one key of the corresponding relation is visible [4]. 16

**PREDEFINED_ATTRIBUTE_ERROR** is raised if any attribute named in the ATTRIBUTES parameter is a CAIS-controlled attribute that is not identified in the Purpose section of this interface as an attribute which may be specified in that parameter [2]. 17

**STATUS_ERROR** is raised if VAKD or ALTERNATIVE_AKD is not an open node handle [2] or if STATUS is not an established status indicator [3]. 18

**NODE_KIND_ERROR** is raised if the open node handle VAKD does not identify a variant AKD [1] or if the open node handle ALTERNATIVE_AKD does not identify an AKD [1]. 19

**CONFORMANCE_ERROR** is raised if the operation would result in an attribute, node or relationship not conforming to its definition or the constraints in force on that definition [3,7,8,9,10]. 20

**DEFINITION_ERROR** is raised if this operation is used to specify properties or constraints which conflict with those already defined for the attribute or the alternative [3]. 21

**LOCK_ERROR** is raised if an open transaction not containing this operation contains other eligible operations whose effects determine whether the conditions for DEFINITION_ERROR apply [3]. 22

**TRANSACTION_ERROR** is raised if the operation is included in more than one transaction [1] or if the operation is included in a transaction and the operation modifies a node which is not included in the same transaction [2]. 23

**INTENT_VIOLATION** is raised if the node handle VAKD was not opened with an intent establishing the rights to read and append relationships [1] or if ALTERNATIVE_AKD was not opened with an intent establishing the rights to read attributes and to read and append relationships [1]. 24

**SECURITY_VIOLATION** is raised if the operation represents a violation of mandatory access controls. SECURITY_VIOLATION is raised only if the conditions for other exceptions are not present. 25

### 5.7.56 Deleting a variant attribute alternative

```
procedure DELETE.ALTERNATIVE
    (VAKD                : in    VAKD_TYPE;
     ALTERNATIVE_AKD : in    AKD_TYPE;
     TINE_LIMIT          : in    CAIS_DURATION       : = LONG_DELAY;
     STATUS              : in    STATUS_INDICATOR    : = NO_INDICATION);
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Purpose:

This procedure deletes a relationship which defines an alternative for a variant attribute kind.

An ALTERNATIVE relationship is deleted. The relationship emanates from the VAKD identified by the open node handle VAKD and terminates at the AKD identified by the open node handle ALTERNATIVE_AKD. The restrictions in Section 5.7.1.9 on page 335 on deletion of an ALTERNATIVE relationship apply.

The TIME_LIMIT parameter allows the specification of a time limit for the delay imposed on the DELETE_ALTERNATIVE operation by the existence of locks on any node to be accessed.

Parameters:

**VAKD** is an open handle to a variant AKD.

**ALTERNATIVE_AKD** is an open handle to the target AKD for the ALTERNATIVE relationship to be deleted.

**TIME_LIMIT** is a time limit on waiting for the unlocking of a node.

**STATUS** is an established status indicator.

Exceptions:

**RELATIONSHIP_ERROR** is raised if an ALTERNATIVE relationship does not exist with the specified source and target nodes [2].

**USE_ERROR** is raised if the relationship to be deleted is predefine [63].

**STATUS_ERROR** is raised if VAKD or ALTERNATIVE_AKD is not an open node handle [2] or if STATUS is not an established status indicator [3].

**NODE_KIND_ERROR** is raised if the open node handle VAKD does not identify a variant AKD [1] or if the open node handle ALTERNATIVE_AKD does not identify an AKD [1].

**CONFORMANCE_ERROR** is raised if the operation would result in an attribute, node or relationship not conforming to its definition or the constraints in force on that definition [3].

**DEFINITION_ERROR** is raised if a restriction on the deletion of an ALTERNATIVE relationship would be violated [2].

**LOCK_ERROR** is raised if an open transaction not containing this operation contains other eligible operations whose effects determine whether the conditions for DEFINITION_ERROR apply [3].

**TRANSACTION_ERROR** is raised if the operation is included in more than one transaction [1] or if the operation is included in a transaction and the operation modifies a node which is not included in the same transaction [2].

**INTENT_VIOLATION** is raised if the node handle VAKD or ALTERNATIVE_AKD was not opened with an intent establishing the rights to read and write relationships [1].

**SECURITY_VIOLATION** is raised if the operation represents a violation of mandatory access controls. SECURITY_VIOLATION is raised only if the conditions for other exceptions are not present.


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

### 5.7.57 Setting definition node attributes

```
procedure SET_DEFINITION_ATTRIBUTE
    (NODE      : in  DEFINITION_NODE_TYPE;
     ATTRIBUTE : in  AD_TYPE;
     VALUE     : in  LIST_TYPE;
     STATUS    : in  STATUS_INDICATOR := NO_INDICATION);
```

Purpose:

This procedure sets the value of a CAIS-controlled, definition node attribute to the value given by VALUE. The node is an AD, AKD or RD identified by the open node handle NODE. The attribute is identified by the open node handle ATTRIBUTE. If it is an optional attribute which does not yet exist on the node, it is created. 1

The definition identified by NODE may be an instance of a specialization of one of the predefined NKDs for ADs, RDs or AKDs. That specialization may impose constraints on the attribute beyond those specified by the CAIS. This procedure observes such constraints and raises the exception CONFORMANCE_ERROR if VALUE does not conform to them. 2

If instances of the definition identified by NODE exist, the new value specified for the attribute cannot impose a stronger constraint than the current value. 3

If NODE identifies an RD, ATTRIBUTE can identify ALLOWED_USAGE or UNIQUE_KEYS. The value specified for the attribute must be at least as constraining as that specified by any generalization of the RD. However, the value must be at most as constraining as that specified by any specialization of the RD. The attribute ALLOWED_USAGE can only be deleted from definition nodes for which instances exist. The attribute UNIQUE_KEYS cannot be modified when instances of the relation exist. 4

If NODE identifies an AD, ATTRIBUTE can identify DELETABLE or MODIFIABLE_VALUE can specify either FALSE or TRUE, regardless of whether instances of the AD exist, so long as MODIFIABLE is TRUE when DELETABLE is TRUE. 5

If NODE identifies an AKD, ATTRIBUTE can identify LOWER_BOUND, UPPER_BOUND, DIGITS, PATTERNS, ENUMERATED, MINIMUM_SIZE, MAXIMUM_SIZE, NAMING, or ORDERED. The value specified must be compatible with any existing CONSTRAINTS or DEFAULT_VALUE attributes which apply to the AKD or the ADs for which it is the attribute kind. 6

If instances of the AKD identified by NODE exist, the following modifications are permitted: 7

a. DIGITS can be decreased; 8

b. LOWER_BOUND and MINIMUM_SIZE can be decreased (or deleted); 9

c. UPPER_BOUND and MAXIMUM_SIZE can be increased (or deleted); 10

d. PATTERNS can be extended; 11

e. ENUMERATED can be set to FALSE; 12

f. ORDERED can be set to FALSE (NAMING must then be NAMED); 13

g. NAMING can only be deleted (if the AKD is not a CLAKD). 14



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Parameters:

**NODE** is an open node handle identifying an AD, AKD or RD for which a predefine attribute is to be set.

**ATTRIBUTE** is an open node handle identifying the AD of the attribute to be set

**VALUE** is the value for the attribute.

**STATUS** is an established status indicator.

Except ions:

**NAME_ERROR** is raised if a definition node to be accessed is inaccessible [1].

**PREDEFINED_ATTRIBUTE_ERROR** is raised if the attribute identified by the ATTRIBUTE parameter is a predefine attribute that is not identified in the Purpose section of this interface as an attribute which may be specified in that parameter [1].

**USE_ERROR** is raised if ATTRIBUTE does not identify a predefine AD [26] or if the attribute identified is a predefine instance [60].

**STATUS_ERROR** is raised if NODE or ATTRIBUTE is not an open node handle [2] or if STATUS is not an established status indicator [3].

**NODE_KIND_ERROR** is raised if the open node handle NODE does not identify a AD, AKD or RD [1] or if the open node handle ATTRIBUTE does not identify an AD [1].

**CONFORMANCE_ERROR** is raised if the operation would result in an attribute, node or relationship not conforming to its definition or the constraints in force on that definition [4,7,8,9,10].

**DEFINITION_ERROR** is raised if this operation is used to specify properties or constraints which conflict with those already defined for a definition [3], if the operation would result in a composite list AKD that is not constrained to be either named or ordered (or both) [5] or if constraints would be modified on a predefined property of a predefine definition [9].

**LOCK_ERROR** is raised if an open transaction not containing this operation contains other eligible operations whose effects determine whether the conditions for DEFINITION_ERROR apply [3].

**INTENT_VIOLATION** is raised if the node handle NODE was not opened with an intent establishing the right to write attributes and to read [1].

**SECURITY_VIOLATION** is raised if the operation represents a violation of mandatory access controls. SECURITY_VIOLATION is raised only if the conditions for other exceptions are not present.

Additional Interfaces:

```
procedure SET_DEFINITION_ATTRIBUTE
    (NODE      : in  DEFINITION_NODE_TYPE;
     ATTRIBUTE : in  ATTRIBUTE_NAME;
     VALUE     : in  LIST_TYPE;
     STATUS    : in  STATUS_INDICATOR := NO_INDICATION)
is
```



Source: https://assist.dla.mil -- Downloaded: 202610-26T07:54Z
Check the source to verify that this is the current version before use.

```
    ATTRIBUTE_DEFINITION : AD_TYPE;
begin
    DEFINITION_OF_NODE_ATTRIBUTE
        (ATTRIBUTE_DEFINITION,
         NODE,
         ATTRIBUTE,
         STATUS);
    SET_DEFINITION_ATTRIBUTE
        (NODE,
         ATTRIBUTE_DEFINITION,
         VALUE,
         STATUS);
    CLOSE (ATTRIBUTE_DEFINITION);
exception
    when others =>
        CLOSE (ATTRIBUTE_DEFINITION);
        raise;
end SET_DEFINITION_ATTRIBUTE;
```

### 5.7.58 Deleting definition node attributes

```
procedure DELETE_DEFINITION_ATTRIBUTE
    (NODE      : in  DEFINITION_NODE_TYPE;
     ATTRIBUTE : in  AD_TYPE;
     STATUS    : in  STATUS_INDICATOR := NO_INDICATION);
```

Purpose:

This procedure deletes a CAIS-controlled, definition node attribute. The node is an AKD or RD identified by the open node handle NODE. The attribute is identified by the open node handle ATTRIBUTE, It must bean optional attribute which exists on the node. 1

The definition identified by NODE may be an instance of a specialization of one of the predefined NKDs for AKDs or RDs. That specialization may impose constraints on the attribute beyond those specified by the CAIS. This procedure observes such constraints and raises the exception CONFORMANCE_ERROR if VALUE does not conform to them. 2

If NODE identifies an RD, ATTRIBUTE can identify ALLOWED_USAGE or UNIQUE_KEYS. The attribute UNIQUE_KEYS cannot be deleted when instances of the relation exist. 3

If NODE identifies an AKD, ATTRIBUTE can identify LOWER_BOUND, UPPER_BOUND, MINIMUM_SIZE, MAXIMUM_SIZE, or NAMING (if the AKD is not a CLAKD). 4

Parameters:

**NODE** is an open node handle identifying an AKD or RD for which a predefined attribute is to be deleted. 5

**ATTRIBUTE** is an open node handle identifying the AD of the attribute to be deleted. 6

**STATUS** is an established status indicator. 7



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Except ions:

**ATTRIBUTE_ERROR** is raised if the node does not have the attribute identified by ATTRIBUTE [3]. 8

**PREDEFINED_ATTRIBUTE_ERROR** is raised if the attribute identified by the ATTRIBUTE parameter is a predefine attribute that is not identified in the Purpose section of this interface as an attribute which may be specified in that parameter [1]. 9

**USE_ERROR** is raised if ATTRIBUTE does not identify a predefine AD [26] or if the attribute identified is a predefine instance [60]. 10

**STATUS_ERROR** is raised if NODE or ATTRIBUTE is not an open node handle [2] or if STATUS is not an established status indicator [3]. 11

**NODE_KIND_ERROR** is raised if the open node handle NODE does not identify an AKD or RD [1] or if the open node handle ATTRIBUTE does not identify an AD [1]. 12

**DEFINITION_ERROR** is raised if this operation is used to modify properties or constraints which conflict with those already defined for a definition [3] or if constraints would be modified on a predefine property of a predefine definition [9]. 13

**LOCK_ERROR** is raised if an open transaction not containing this operation contains other eligible operations whose effects determine whether the conditions for DEFINITION_ERROR apply [3]. 14

**INTENT_VIOLATION** is raised if the node handle NODE was not opened with an intent establishing the right to write attributes and to read [1]. 15

**SECURITY_VIOLATION** is raised if the operation represents a violation of mandatory access controls. SECURITY_VIOLATION is raised only if the conditions for other exceptions are not present. 16

Additional Interfaces:

```
procedure DELETE_DEFINITION_ATTRIBUTE
    (NODE      : in    DEFINITION_NODE_TYPE;
     ATTRIBUTE : in    ATTRIBUTE_NAME;
     STATUS    : in    STATUS_INDICATOR := NO_INDICATION)
is
    ATTRIBUTE_DEFINITION : AD_TYPE;
begin
    DEFINITION_OF_NODE_ATTRIBUTE
        (ATTRIBUTE_DEFINITION
         NODE,
         ATTRIBUTE,
         STATUS);
    DELETE_DEFINITION_ATTRIBUTE
        (NODE,
         ATTRIBUTE_DEFINITION,
         STATUS);
    CLOSE (ATTRIBUTE_DEFINITION);
exception
    when others =>
```


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
        CLOSE (ATTRIBUTE_DEFINITION);
        raise;
end DELETE_DEFINITION_ATTRIBUTE;
```

### 5.7.59 Setting definition relationship attributes

```
procedure SET_PROPERTY_ATTRIBUTE
    (BASE      : in  DEFINITION_NODE_TYPE;
     KEY       : in  RELATIONSHIP_KEY;
     RELATION  : in  RD_TYPE;
     ATTRIBUTE : in  AD_TYPE;
     VALUE     : in  LIST_TYPE;
     STATUS    : in  STATUS_INDICATOR := NO_INDICATION);
```

Purpose:

This procedure sets the value of a CAIS-controlled, property definition relationship attribute to the value given by VALUE. The source node is an AD, AKD, NKD or RD identified by the open node handle BASE. The relationship is of the relation identified by the open node handle RELATION with key list KEY. RELATION must identify an ATTRIBUTE_KIND, ALTERNATIVE, ATTRIBUTE, KEY, COMPONENT or EMANATES relation. 1

The attribute is identified by the open node handle ATTRIBUTE. It must be a CONSTRAINTS, DEFAULT_VALUE, NAME, OPTIONAL, POSITION, MINIMUM_INSTANCES or MAXI_ MUM_INSTANCES attribute. If it is an optional attribute which does not yet exist on the relationship, it is created. 2

If any instance of the definition identified by BASE has an instance of the property defined by RELATION and KEY, the new value specified for the attribute cannot impose a stronger constraint than the current value. 3

If BASE, KEY and RELATION identify an ATTRIBUTE_KIND relationship emanating from an AD, ATTRIBUTE can identify a CONSTRAINTS or DEFAULT_VALUE attribute. A CON-STRAINTS value specified must be compatible with any other existing constraints and default values which apply to the attribute identified by the AD. A DEFAULT_VALUE value specified must be compatible with any constraints which apply to the attribute identified by the AD for those node kinds and relations not already having a default value for the attribute. 4

If BASE, KEY and RELATION identify an ALTERNATIVE relationship emanating from a VAKD, ATTRIBUTE can identify a CONSTRAINTS or NAME attribute. Any constraints specified must be compatible with any other existing constraints or default values which apply to the VAKD or the ADs for which it is the attribute kind. Any new name must be distinct from others defined for alternatives of the VAKD, and the current name (if any) must not be referenced in constraints which apply to the VAKD. 5

If BASE, KEY and RELATION identify a COMPONENT relationship emanating from a ULAKD, ATTRIBUTE can identify a CONSTRAINTS attribute. Any constraints specified must be compatible with any other existing constraints or default values which apply to the AKD or the ADs for which it is the attribute kind. 6

If BASE, KEY and RELATION identify a COMPONENT relationship emanating from a CLAKD, ATTRIBUTE can identify a CONSTRAINTS, NAME, OPTIONAL or POSITION 7



Source: https://assist.dla.mil -- Downloaded: 202610-26T07:54Z
Check the source to verify that this is the current version before use.

attribute. Any constraints specified must be compatible with any other existing constraints or default values which apply to the AKD or the ADs for which it is the attribute kind. Names must be distinct from those defined for other components of the CLAKD, and the current name must not be referenced in constraints which apply to the VAKD. Names cannot be redefined after instances of the AKD exist. OPTIONAL can only be set TRUE if instances of the AKD exist. Positions must be distinct from those defined for other components of the CLAKD. Relative positions cannot be modified if any CONSTRAINTS attributes which apply to the CLAKD rely on positional notation or if instances of the AKD exist.

If BASE, KEY and RELATION identify an ATTRIBUTE or KEY relationship emanating from an NKD or RD, ATTRIBUTE can identify a CONSTRAINTS or DEFAULT_VALUE attribute. Any constraints specified must be no weaker than any specified by generalizations of the base node and no stronger than any specified by specializations. Any default value specified must not result in a specialization inheriting distinct default values (unresolved through the precedence imposed by the lattice). 8

If BASE, KEY and RELATION identify an EMANATES relationship emanating from an NKD, ATTRIBUTE can identify a MINIMUM_INSTANCES or MAXIMUM_INSTANCES attribute. Such constraints must be no weaker than any specified by generalizations of the NKD and no stonger than any specified by specializations. 9

Parameters:

**BASE** is an open node handle identifying the AD, AKD, NKD or RD from which the relationship whose attribute is to be set emanates. 10

**RELATION** is an open node handle identifying the RD for the relationship. 11

**KEY** is a key list for the relationship. 12

**ATTRIBUTE** is an open node handle identifying the AD for the attribute to be set. 13

**VALUE** is the value for the attribute. 14

**STATUS** is an established status indicator. 15

Exceptions:

**PATHNAME_SYNTAX_ERROR** is raised if the KEY given is syntactically illegal [2]. 16

**NAME_ERROR** is raised if a definition node to be accessed is inaccessible [1]. 17

**RELATIONSHIP_ERROR** is raised if a relationship with the identification KEY does not exist [2] or if KEY specifies a value for the same key more than once [5]. 18

**NOT_IN_VIEW_ERROR** is raised if a key name specified in KEY cannot be translated [1] or if KEY consists of a single relationship key designator and no key of the corresponding relation is visible [5]. 19

**AMBIGUOUS_VIEW_ERROR** is raised if a key name specified in KEY has an ambiguous translation [1] or if KEY consists of a single relationship key designator and more than one key of the corresponding relation is visible [4]. 20

**PREDEFINED_ATTRIBUTE_ERROR** is raised if ATTRIBUTE is a CAIS-controlled attribute that is not identified in the Purpose section of this interface as an attribute which may be specified in that parameter [2]. 21


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

**USE_ERROR** is raised if ATTRIBUTE does not identify a predefine AD [26] or if the attribute identified is a predefine instance [61]. 22

**STATUS_ERROR** is raised if BASE, RELATION or ATTRIBUTE is not an open node handle [2] or if STATUS is not an established status indicator [3]. 23

**NODE_KIND_ERROR** is raised if the open node handle BASE does not identify a definition node [1], if the open node handle RELATION does not identify an RD [1] or if the open node handle ATTRIBUTE does not identify an AD [1]. 24

**CONFORMANCE_ERROR** is raised if the operation would result in an attribute, node or relationship not conforming to its definition or the constraints in force on that definition [4,7,8,9,10]. 25

**DEFINITION_ERROR** is raised if this operation is used to specify properties or constraints which conflict with those already defined for a definition [3] or if constraints would be modified on a predefine property of a predefine definition [9]. 26

**LOCK_ERROR** is raised if an open transaction not containing this operation contains other eligible operations whose effects determine whether the conditions for DEFINITION_ERROR apply [3]. 27

**INTENT_VIOLATION** is raised if the node handle BASE was not opened with an intent establishing the right to write relationships and to read [1]. 28

**SECURITY_VIOLATION** is raised if the operation represents a violation of mandatory access controls. SECURITY_VIOLATION is raised only if the conditions for other exceptions are not present. 29

Additional Interfaces:

```
procedure SET_PROPERTY_ATTRIBUTE
    (BASE      : in    DEFINITION_NODE_TYPE;
     KEY       : in    RELATIONSHIP_KEY;
     RELATION  : in    RELATION_NAME;
     ATTRIBUTE : in    ATTRIBUTE_NAME;
     VALUE     : in    LIST_TYPE;
     STATUS    : in    STATUS_INDICATOR  := NO_INDICATION)
is
    ATTRIBUTE_DEFINITION : AD_TYPE;
    RELATION_DEFINITION : RD_TYPE;
begin
    DEFINITION_OF_RELATION (RELATION_DEFINITION, BASE, RELATION, STATUS);
    DEFINITION_OF_PATH_ATTRIBUTE
        (ATTRIBUTE_DEFINITION,
         RELATION_DEFINITION,
         ATTRIBUTE,
         STATUS);
    SET_PROPERTY_ATTRIBUTE
        (BASE,
         KEY,
         RELATION_DEFINITION,
         ATTRIBUTE_DEFINITION,
```


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
            VALUE,
            STATUS);
        CLOSE (RELATION_DEFINITION);
        CLOSE (ATTRIBUTE_DEFINITION);
    exception
        when others =>
            CLOSE (RELATION_DEFINITION);
            CLOSE (ATTRIBUTE_DEFINITION);
            raise;
    end SET_PROPERTY_ATTRIBUTE;
```

### 5.7.60 Delete definition relationship attributes

```
procedure DELETE_PROPERTY_ATTRIBUTE
    (BASE      : in  DEFINITION_NODE_TYPE;
     KEY       : in  RELATIONSHIP_KEY;
     RELATION  : in  RD_TYPE;
     ATTRIBUTE : in  AD_TYPE;
     STATUS    : in  STATUS_INDICATOR := NO_INDICATION);
```

Purpose:

This procedure deletes the value of a CAIS-controlled, property definition relationship attribute. The source node is an AD, AKD, NKD or RD identified by the open node handle BASE. The relationship is of the relation identified by the open node handle RELATION with key list KEY. RELATION must identify an ATTRIBUTE_KIND, ALTERNATIVE, ATTRIBUTE, KEY, COMPONENT or EMANATES relation. 1

The attribute is identified by the open node handle ATTRIBUTE. It must be a CONSTRAINTS, DEFAULT_VALUE, MINIMUM_INSTANCES or MAXIMUM_INSTANCES attribute. It must be an optional attribute which exists on the relationship. 2

If BASE, KEY and RELATION identify an ATTRIBUTE_KIND, ALTERNATIVE or COMPONENT relationship, ATTRIBUTE can identify a CONSTRAINTS attribute. 3

If BASE, KEY and RELATION identify an ATTRIBUTE, ATTRIBUTE_KIND or KEY relationship, ATTRIBUTE can identify a CONSTRAINTS or DEFAULT_VALUE attribute. For ATTRIBUTE and KEY relationships, deletion of the default value must not result in a specialization inheriting distinct default values (unresolved through the precedence imposed by the lattice). 4

If BASE, KEY and RELATION identify an EMANATES relationship, ATTRIBUTE can identify a MINIMUM_INSTANCES or MAXIMUM_INSTANCES attribute. 5

Parameters:

**BASE** is an open node handle identifying the AD, AKD, NKD or RD from which the relationship whose attribute is to be deleted emanates. 6

**RELATION** is an open node handle identifying the RD for the relationship. 7


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

**KEY** is a key list for the relationship. 8

**ATTRIBUTE** is an open node handle identifying the AD for the attribute to be deleted. 9

**STATUS** is an established status indicator. 10

Exceptions:

**PATHNAME_SYNTAX_ERROR** is raised if the KEY given is syntactically illegal [2]. 11

**ATTRIBUTE_ERROR** is raised if the relationship does not have the attribute identified by ATTRIBUTE [3]. 12

**RELATIONSHIP_ERROR** is raised if the specified relationship does not exist [2] or if KEY specifies a value for the same key more than once [5]. 13

**NOT_IN_VIEW_ERROR** is raised if a key name specified in KEY cannot be translated [1] or if KEY consists of a single relationship key designator and no key of the corresponding relation is visible [5]. 14

**AMBIGUOUS_VIEW_ERROR** is raised if a key name specified in KEY has an ambiguous translation [1] or if KEY consists of a single relationship key designator and more than one key of the corresponding relation is visible [4]. 15

**PREDEFINED_ATTRIBUTE_ERROR** is raised if ATTRIBUTE is a predefine attribute that is not identified in the Purpose section of this interface as an attribute which may be specified in that parameter [2]. 16

**USE_ERROR** is raised if ATTRIBUTE does not identify a predefine AD [26] or if the attribute identified is a predefine instance [61]. 17

**STATUS_ERROR** is raised if BASE, RELATION or ATTRIBUTE is not an open node handle [2] or if STATUS is not an established status indicator [3]. 18

**NODE_KIND_ERROR** is raised if the open node handle BASE does not identify a definition node [1], if the open node handle RELATION does not identify an RD [1] or if the open node handle ATTRIBUTE does not identify an AD [1]. 19

**DEFINITION_ERROR** is raised if this operation is used to modify properties or constraints which conflict with those already defined for a definition [3] or if constraints would be modified on a predefine property of a predefine definition [9]. 20

**LOCK_ERROR** is raised if an open transaction not containing this operation contains other eligible operations whose effects determine whether the conditions for DEFINITION_ERROR apply [3]. 21

**INTENT_VIOLATION** is raised if the node handle BASE was not opened with an intent establishing the right to write relationships and to read [1]. 22

**SECURITY_VIOLATION** is raised if the operation represents a violation of mandatory access controls. SECURITY_VIOLATION is raised only if the conditions for other exceptions are not present. 23

Additional Interfaces:

```
procedure DELETE_PROPERTY_ATTRIBUTE
    (BASE      : in    DEFINITION_NODE_TYPE;
     KEY       : in    RELATIONSHIP_KEY;
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
        RELATION  : in     RELATION_NAME;
        ATTRIBUTE : in     ATTRIBUTE_NAME;
        STATUS    : in     STATUS_INDICATOR := NO_INDICATION)
    is
        ATTRIBUTE_DEFINITION : AD_TYPE;
        RELATION_DEFINITION : RD_TYPE;
    begin
        DEFINITION_OF_RELATION (RELATION_DEFINITION, BASE, RELATION, STATUS);
        DEFINITION_OF_PATH_ATTRIBUTE
            (ATTRIBUTE_DEFINITION,
             RELATION_DEFINITION,
             ATTRIBUTE,
             STATUS);
        DELETE_PROPERTY_ATTRIBUTE
            (BASE,
             KEY ,
             RELATION_DEFINITION,
             ATTRIBUTE_DEFINITION,
             STATUS);
        CLOSE (RELATION_DEFINITION);
        CLOSE (ATTRIBUTE_DEFINITION);
    exception
        when others =>
            CLOSE (RELATION_DEFINITION);
            CLOSE (ATTRIBUTE_DEFINITION);
            raise;
    end DELETE_PROPERTY_ATTRIBUTE;
```

### 5.7.61 Changing the NKD for a node

```
procedure CONVERT_NODE
    (NODE           : in     NODE_TYPE;
     KIND           : in     NKD_TYPE;
     NEW_ATTRIBUTES : in     ATTRIBUTE_LIST     := EMPTY_LIST;
     STATUS         : in     STATUS_INDICATOR   := NO_INDICATION);
```

Purpose:

This procedure changes the node identified by the open node handle NODE to be an instance of the node kind identified by KIND. 1

NEW_ATTRIBUTES provides values for attributes to be given to the converted node. Any existing attributes not specified in NEW_ATTRIBUTES are retained. Any attributes having default values defined by the new node kind and nonexisting on the node nor specified in NEW_-ATTRIBUTES are created with those default values. 2

The node, as modified for new and defaulted attributes, must not violate any constraints defined by the kind nor have properties not defined by the kind. 3


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Parameters:

**NODE** is an open node handle.

**KIND** is an open node handle identifying the new NKD for the given node.

**NEW_ATTRIBUTES** is an empty list or a named list whose elements establish initial values for attributes of the converted node; each named item specifies an attribute name and the value to be given to that attribute.

**STATUS** is an established status indicator.

Exceptions:

**SYNTAX_ERROR** is raised if the NEW_ATTRIBUTES parameter is syntactically illegal [8].

**ATTRIBUTE_ERROR** is raised if an attribute list specifies a value for the same attribute more than once [6].

**NOT_IN_VIEW_ERROR** is raised if an attribute name NEW_ATTRIBUTES cannot be translated [1].

**AMBIGUOUS_VIEW_ERROR** is raised if translation of an attribute name specified in NEW_ATTRIBUTES is ambiguous [1].

**STATUS_ERROR** is raised if NODE or KIND is not an open node handle [2] or if STATUS is not an established status indicator [3].

**NODE_KIND_ERROR** is raised if KIND does not identify an NKD [1] or if the conversion would change the set of predefine, CAIS-controlled node kinds to which the node belongs [6].

**PREDEFINED_ATTRIBUTE_ERROR** is raised if NEW_ATTRIBUTES names a CAIS-controlled attribute [2].

**CONFORMANCE_ERROR** is raised if the operation would result in an attribute, node or relationship not conforming to its definition or the constraints in force on that definition [1,2,3,4,10].

**INTENT_VIOLATION** is raised if the node handle NODE was not opened with intents establishing the rights to write and read [1].

**ACCESS_VIOLATION** is raised if the discretionary access control rights of the current process are insufficient to allow access to the NKD identified through KIND with intent CREATE_INSTANCES [1].

**SECURITY_VIOLATION** is if the operation represents a violation of mandatory access controls. SECURITY_VIOLATION is raised only if the conditions for other exceptions are not present.

Additional Interfaces:

```
procedure CONVERT_NODE
    (NODE           : in  NODE_TYPE;
     KIND           : in  NODE_KIND_NAME;
     NEW_ATTRIBUTES : in  ATTRIBUTE_LIST     := EMPTY_LIST;
     STATUS         : in  STATUS_INDICATOR   := NO_INDICATION)
```


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
is
    NODE_KIND : NKD_TYPE;
begin
    0PEN_NODE_KIND_DEFINITION (NODE_KIND, KIND, STATUS);
    COMVERT_NODE (NODE, NODE_KIND, NEW_ATTRIBUTES, STATUS);
    CLOSE (NODE_KIND);
exception
    when others =>
        CLOSE (NODE_KIND);
        raise;
end CONVERT_NODE;
```

### 5.7.62 Checking conformance of a node to an NKD

```
function CONFORMS
    (NODE                      : in     NODE_TYPE;
     NKD                       : in     NKD_TYPE;
     CONFORMANCE_ERROR_INFO    : in     STATUS_INDICATOR := NO_INDICATION;
     STATUS                    : in     STATUS_INDICATOR := NO_INDICATION)
    return BOOLEAN;
```

Purpose:

This function tests the conformance of the node identified by the open node handle NODE to the node kind identified by the open node handle NKD.

TRUE is returned if the node does not violate any constraints defined by the NKD nor have properties not defined by the NKD. The node need not bean instance of the specified kind.

FALSE is returned if the node has non-conforming properties.

If CONFORMANCE_ERROR_INFO and STATUS are collecting status information, their status information is set to indicate a call on this function. Any violations on the preconditions of this interface resulting in an exception are indicated through STATUS, as usual. Alternatively, if this function returns false, CONFORMANCE_ERROR_INFO will indicate an exception and condition detected during the conformance check. The condition maybe a CONFORMANCE_-ERROR condition [1,2,3,4,10], or NODE_KIND_ERROR [6] is indicated if the node's kind and the kind identified by NKD belong to different sets of predefined, CAIS-controlled node kinds. NODE_KIND_ERROR [7] is indicated only if no conditions for CONFORMANCE_ERROR are detected.

Parameters:

**NODE** is an open node handle identifying the node on which to perform the conformance check.

**NKD** is an open node handle identifying the NKD against which to check conformance.

**CONFORMANCE_ERROR_INFO** is an established status indicator. If it is collecting status information, that information will indicate what condition, if any, was detected during the conformance check.

**STATUS** is an established status indicator. If it is collecting status information, that informs-tion will indicate whether or not the preconditions of this interface have been satisfied.



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Exceptions:

**STATUS_ERROR** is raised if NODE or NKD is not an open node handle [2] or if CONFORMANCE_ERROR_INFO or STATUS is not an established status indicator [3]. 9

**NODE_KIND_ERROR** is raised if NKD does not identify an NKD [1]. 10

**INTENT_VIOLATION** is raised if the node handle NODE was not opened with intents establishing the rights to read [1]. 11

**SECURITY_VIOLATION** is if the operation represents a violation of mandatory access controls. SECURITY_VIOLATION is raised only if the conditions for other exceptions are not present. 12

Additional Interfaces:

```
function CONFORMS
    (NODE                       : in    NODE_TYPE;
     NKD                        : in    NODE_KIND_NAME;
     CONFORMANCE_ERROR_INFO     : in    STATUS_INDICATOR  := NO_INDICATION;
     STATUS                     : in    STATUS_INDICATOR  := NO_INDICATION)
    return BOOLEAN
is
    NODE_KIND  : NKD_TYPE;
    RESULT     : BOOLEAN;
begin
    OPEN_NODE_KIND_DEFINITION (NODE_KIND, NKD, STATUS);
    begin
        RESULT  := CONFORMS (NODE, NODE_KIND, CONFORMANCE_ERROR_INFO, STATUS);
    except ion
        when others =>
            CLOSE (NODE_KIND);
            raise;
    end;
    CLOSE (NODE_KIND);
    return  RESULT;
end CONFORM;
```

Notes:

This interface will not raise CONFORMANCE_ERROR, nor will it raise NODE_KIND_ERROR for condition [6]. 13

### 5.7.63 Checking conformance of a relationship to an RD

```
function CONFORMS
    (BASE                       : in    NODE_TYPE;
     KEY                        : in    RELATIONSHIP_KEY  := EMPTY_KEY_LIST;
     RELATION                   : in    RD_TYPE;
```


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
    RD                        : in     RD_TYPE;
    CONFORMANCE_ERROR_INFO    : in     STATUS_INDICATOR := NO_INDICATION;
    STATUS                    : in     STATUS_INDICATOR  := NO_INDICATION)
  return BOOLEAN;
```

Purpose:

This function tests the conformance of the relationship identified by the specified base node, relation and key list to the relation identified by RD. 1

TRUE is returned if the relationship does not violate any constraints defined by the RD nor have properties not defined by the RD. The relationship need not be an instance of the relation specified by RD. 2

FALSE is returned the relationship has non-conforming properties. 3

If CONFORMANCE_ERROR_INFO and STATUS are collecting status information, their status information is set to indicate a call on this function. Any violations on the preconditions of this interface resulting in an exception are indicated through STATUS, as usual. Alternatively, if this function returns false, CONFORMANCE_ERROR_INFO will indicate an exception and condition detected during the conformance check. The condition may be a CONFORMANCE_ERROR condition [1,2,4,10], or PREDEFINED_RELATION_ERROR [3] is indicated if RELATION and RD identify relations belonging to different sets of predefine, CAIS-controlled relations. PREDEFINED_RELATION_ERROR is indicated only if no conditions for CONFORMANCE_ERROR are detected. 4

Parameters:

**BASE** is an open node handle identifying the base node for the relationship on which to perform the conformance check. 5

**KEY** is the key list for the relationship to be checked. 6

**RELATION** is an open node handle identifying the RD for the relationship to be checked. 7

**RD** is an open node handle identifying the RD against which to check conformance. 8

**CONFORMANCE_ERROR_INFO** is an established status indicator. If it is collecting status information, that information will indicate what condition, if any, was detected during the conformance check. 9

**STATUS** is an established status indicator. If it is collecting status information, that informstion will indicate whether or not the preconditions of this interface have been satisfied. 10

Exceptions:

**PATHNAME_SYNTAX_ERROR** is raised if the KEY given is syntactically illegal [2]. 11

**STATUS_ERROR** is raised if BASE, RELATION or RD is not an open node handle [2] or if CONFORMANCE_ERROR_INFO or STATUS is not an established status indicator [3]. 12

**NODE_KIND_ERROR** is raised if RELATION or RD does not identify an RD [1]. 13

**RELATIONSHIP_ERROR** is raised if the relationship identified by BASE, KEY and RELATION does not exist [2] or if KEY specifies a value for the same key more than once [5]. 14



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

**NOT_IN_VIEW_ERROR** is raised if a key name specified in KEY cannot be translated [1] or if KEY consists of a single relationship key designator and no key of the corresponding relation is visible [5]. 15

**AMBIGUOUS_VIEW_ERROR** is raised if a key name specified in KEY has an ambiguous translation [1] or if KEY consists of a single relationship key designator and more than one key of the corresponding relation is visible [4]. 16

**INTENT_VIOLATION** is raised if the node handle BASE was not opened with intents establishing the rights to read relationships [1]. 17

**SECURITY_VIOLATION** is if the operation represents a violation of mandatory access controls, SECURITY_VIOLATION is raised only if the conditions for other exceptions are not present. 18

Additional Interfaces:

```
function CONFORMS
    (BASE                    : in    NODE_TYPE;
     KEY                     : in    RELATIONSHIP_KEY  := EMPTY_KEY_LIST;
     RELATION                : in    RELATION_NAME;
     RD                      : in    RELATION_NAME;
     CONFORMANCE_ERROR_INFO  : in    STATUS_INDICATOR  := NO_INDICATION;
     STATUS                  : in    STATUS_INDICATOR  := NO_INDICATION)
    return BOOLEAN
is
    INSTANCE_RD : RD_TYPE;
    CHECK_RD    : RD_TYPE;
    RESULT      : BOOLEAN;
begin
    DEFINITION_OF_RELATION (INSTANCE_RD, BASE, RELATION, STATUS);
    OPEN_RELATION_DEFINITION (CHECK_RD, RD, STATUS);
    RESULT := CONFORMS
        (BASE, KEY, INSTANCE_RD, CHECK_RD, CONFORMANCE_ERROR_INFO, STATUS);
    CLOSE (INSTANCE_RD);
    CLOSE (CHECK_RD);
    return RESULT;
exception
    when others =>
        CLOSE (INSTANCE_RD);
        CLOSE (CHECK_RD);
        raise;
end CONFORMS;
```

Notes:

This interface raises neither CONFORMANCE_ERROR nor PREDEFINED_RELATION_ERROR. 19



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

### 5.7.64 Checking conformance of a list to an AD or AKD

```
function CONFORMS
    (LIST                    : in    LIST_TYPE;
     DEFINITION              : in    ATTRIBUTE_NODE_TYPE;
     CONFORMANCE_ERROR_INFO  : in    STATUS_INDICATOR := NO_INDICATION;
     STATUS                  : in    STATUS_INDICATOR  := NO_INDICATION)
    return BOOLEAN;
```

Purpose:

This function tests the conformance of the list in LIST to the AD or AKD identified by the open node handle DEFINITION. If DEFINITION identifies an AD, the check is against the AD's AKD, but also takes into account any constraints placed on the AKD by the AD. 1

TRUE is returned if the list conforms to the AKD and does not violate any constraints defined by the AD. Otherwise, FALSE is returned. 2

If CONFORMANCE_ERROR_INFO and STATUS are collecting status information, their status information is set to indicate a call on this function. Any violations on the preconditions of this interface resulting in an exception are indicated through STATUS, as usual. Alternatively, if this function returns false, CONFORMANCE_ERROR_INFO will indicate a CONFORMANCE_-ERROR exception and condition [7,8,9,10] detected during the conformance check. 3

Parameters:

**LIST** is a list on which to perform the conformance check. 4

**DEFINITION** is an open node handle identifying the AD or AKD against which to check conformance. 5

**CONFORMANCE_ERROR_INFO** is an established status indicator. If it is collecting status information, that information will indicate what condition, if any, was detected during the conformance check. 6

**STATUS** is an established status indicator. If it is collecting status information, that information will indicate whether or not the preconditions of this interface have been satisfied. 7

Exceptions:

**STATUS_ERROR** is raised if DEFINITION is not an open node handle [2] or if CONFORMANCE_ERROR_INFO or STATUS is not an established status indicator [3]. 8

**NODE_KIND_ERROR** is raised if DEFINITION does not identify an AD or AKD [1]. 9

**SECURITY_VIOLATION** is if the operation represents a violation of mandatory access controls. SECURITY_VIOLATION is raised only if the conditions for other exceptions are not present. 10



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Additional Interfaces:

```
function CONFORMS
    (LIST                       : in     LIST_TYPE;
     DEFINITION                 : in     ATTRIBUTE_NAME;
     CONFORMANCE_ERROR_INFO     : in     STATUS_INDICATOR := NO_INDICATION;
     STATUS                     : in     STATUS_INDICATOR := NO_INDICATION)
    return BOOLEAN
is
    AD      : AD_TYPE;
    RESULT  : BOOLEAN;
begin
    OPEN_ATTRIBUTE_DEFINITION (AD, DEFINITION, STATUS);
    begin
        RESULT := CONFORMS
            (LIST, AD, CONFORMANCE_ERROR_INFO, STATUS);
    exception
        when others =>
            CLOSE (AD);
            raise;
    end;
    CLOSE (AD);
    return RESULT;
end CONFORMS;
```

Notes:

This interface does not raise CONFORMANCE_ERROR.



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

# 5.8 Package CAIS_ATTRIBUTE_MONITOR_MANAGEMENT

This package supports the creation, manipulation, and deletion of MONITOR nodes. The exceptions raised by all subprograms in this package are defined in the packages CAIS_DEFINITIONS and CAIS_-PRAGMATICS. 1

MONITOR nodes can be used to monitor modifications to specific node and relationship attributes. A given MONITOR node can monitor modifications to node attributes of a collection of nodes and relationship attributes on relationships which emanate from those nodes. Whenever an attribute which is being monitored by a MONITOR node is created, modified or deleted, an attribute monitor event is inserted at the end of the attribute monitor event queue of an attribute monitor event handler process associated with the MONITOR node. An *event handler process* is a process associated with a MONITOR node which is notified when a monitored attribute is created, modified or deleted by posting an event to its event queue. An *attribute monitor event* is a notification that a monitored attribute has been created, modified or deleted. An attribute monitor event includes information that identifies the affected attribute. The attribute monitor event queue of an attribute monitor event handler process is a queue containing the attribute monitor events that have been posted to the event handler process but not yet retrieved by that process. 2

A MONITOR node is created by a call to the procedure CREATE_MONITOR_NODE. The primary relationship to a MONITOR node is an ATTRIBUTE_MONITOR relationship and always emanates from the current USER node. 3

Specific node and relationship attributes are added to the set of attributes which are monitored by a MONITOR node by calling the procedures MONITOR_NODE_ATTRIBUTE and MONITOR_-PATH_ATTRIBUTE. These procedures cause the creation of MONITORS relationships which emanate from MONITOR nodes and terminate at the nodes whose node or relationship attributes are being monitored. The ATTRIBUTES_MONITORED attribute on each MONITORS relationship identifies the specific node and relationship attributes which are being monitored. MONITORS relationships are bidirectional relationships with corresponding reverse relationships of the predefine relation MONI-TORED_BY. 4

Specific attributes may be removed from this set by calling the procedure CANCEL_ATTRIBUTE_-MONITORING. This procedure causes the deletion of a MONITORS relationship if its ATTRIBUTES_-MONITORED attribute would indicate an empty collection of monitored attributes. 5

Table XXII specifies the syntax for ATTRIBUTES_MONITORED attributes. An ATTRIBUTES_-MONITORED attribute is a named uniform list of attribute designators. Each named attribute designator represents a node attribute or a set of relationship attributes being monitored. A designator for a node attribute identifies a single attribute of the monitored node. The designator for a node attribute is a string item containing the unique identifier (UID) of the AD for the attribute. 6

A relationship attribute designator identifies a relationship attribute and a set of relationships for which the attribute is being monitored. A relationship attribute designator is an unordered, named composite list consisting of a string item containing the UID of an RD, a key value list and a string item containing the UID of the AD for the monitored attributes. The RD and the key value list identify the set of relationships for which the attribute is being monitored. A key value list is an unnamed uniform list of key value designators which specify values of keys of the monitored relationships. Each key value designator identifies a key by the UID of the AD for the key and optionally specifies a value for the key using an identifier item. If a value is specified for the key, only relationships for which the key has the specified value are monitored. If no value is specified for the key, only relationships with no value for the key are included in the set of monitored relationships. 7



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

TABLE XXII: **ATTRIBUTES_MONITORED Attribute Value BNF**

```
attributes_monitored_value        ::= (name => attribute_designator
                                       {, name => attribute_designator})
attribute_designator              ::= AD_unique_identifier
                                    | relationship_attribute_designator
relationship_attribute_designator ::= (RD => RD_unique_identifier, KEYS => key_value_list,
                                       AD => AD_unique_identifier)
key_value_list                    ::= (key_value_designator {, key_value_designator})
                                    | ()
key_value_designator              ::= (AD => AD_unique_identifier [, VALUE => key_value ])
name                              ::= identifier
key_value                         ::= identifier
AD_unique_identifier              ::= unique_identifier
RD_unique_identifier              ::= unique_identifier
unique_identifier                 ::= quoted_string
```

Notes:

1, See Appendix D for a description of the notation used.
2. See Section 9 for a description of identifiers and quoted strings.

Attribute monitor events are retrieved by an event handler process by calling the procedures GET_MONITOR_EVENT and AWAIT_MONITOR_EVENT. 8

An event handler process is associated with a MONITOR node by a secondary HANDLER_PROCESS relationship which emanates from the MONITOR node and terminates at the PROCESS node representing the execution of the process. A HANDLER_PROCESS relationship is a bidirectional relationship with a corresponding reverse relationship of the relation HANDLER_PROCESS_OF. A given MONITOR node may have more than one associated event handler process, however the mandatory access control subject classification labels of the PROCESS nodes (see Section 5.4.2 on page 255) representing the execution of those processes must be distinct. Processes executing on behalf of the user represented by the USER node which is the parent of a MONITOR node can be explicitly associated with that MONITOR node as event handler processes. Event handler processes with specific mandatory access control subject classifications may also be implicitly created and associated with a MONITOR node by the CAIS implementation as required by the semantics of attribute monitor event handler process selection. Associated with each MONITOR node is an EXECUTABLE_IMAGE node whose content is the executable image for the program to be executed by implicitly created attribute monitor event handler processes for that MONITOR node. This association is represented by a secondary relationship of the predefine relation HANDLER_IMAGE which emanates from a MONITOR node and terminates at the associated EXECUTABLE_IMAGE node. Each implicitly created event handler process is created as if an implicit call is made by the CAIS implementation to the procedure CREATE_JOB on behalf of the user represented by the USER node which is the parent of the MONITOR node. The primary relationship to the root PROCESS node for an event handler process is a JOB relationship that emanates from the USER node which is the parent of the MONITOR node. 9

The selection of the attribute monitor event handler process to which an attribute monitor event is 10



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

posted is determined by the mandatory access control labeling of the modified node. The SUBJECT_-CLASSIFICATION attribute on the PROCESS node of the selected handler process must specify a mandatory access control subject label whose hierarchical classification level is the maximum of the hierarchical classification level of the object label on the modified node and the classification level specified in the HANDLER_MANDATORY_ACCESS attribute of the MONITOR node. Furthermore, the set of non-hierarchical categories specified in the SUBJECT_CLASSIFICATION attribute on the PROCESS node of the selected event handler process must contain each of the non-hierarchical categories specified in the OBJECT_CLASSIFICATION attribute of the modified node and in the HANDLER_-MANDATORY_ACCESS attribute of the MONITOR node. These rules regarding the selection of the event handler process insure that the flow of information to the event handler process in the form of the attribute monitor event does not constitute a violation of the CAIS mandatory access controls and that the selected event handler process has sufficient mandatory access control rights to read the modified node. When necessary, a CAIS implementation must implicitly create an attribute monitor event handler process with the appropriate mandatory access control subject classification label. An additional handler process is not created if at the time of the posting of an attribute monitor event there is an associated handler PROCESS node whose subject classification label meets these criteria and whose CURRENT_STATUS attribute is equal to READY or SUSPENDED. The PROCESS node for an implicitly created attribute monitor event handler process is created by an operation whose effects are semantically identical to an implicit call to the procedure CREATE_JOB in package CAIS_-PROCESS_MANAGEMENT. Processes can be explicitly designated as attribute monitor event handler processes associated with a MONITOR node by calling the procedure ASSOCIATE_EVENT_HANDLER_PROCESS.

11 An attribute monitor event is *reportable* to an event handler process if the node affected is obtainable and the event handler process has sufficient mandatory and discretionary access rights to be given knowledge of the event's occurrence at the time that the event is requested from the attribute monitor event queue. The access rights required are to read the affected attribute. Events which are not reportable are skipped over and discarded when the event handler process attempts to read them from the event queue.

12 Any number of separate MONITOR nodes may simultaneously monitor the attributes of a given node and attributes of relationships emanating from that node. In cases where more than one MONITOR node is monitoring a particular attribute, an attribute monitor event is added to the event queue of an event handler process associated with each MONITOR node whenever a monitored attribute is created, deleted or otherwise modified. However, in such cases, the order in which the attribute monitor events are posted to the attribute monitor event queues of the respective event handler processes is implementation-defied. The creation or deletion of a relationship results in an attribute monitor event being posted for each monitored attribute on the relationship that is created or deleted.

13 As part of the creation of a MONITOR node, the procedure CREATE_MONITOR_NODE also creates several secondary relationships emanating from the newly created MONITOR node as described in Table XXIII, page 480 and several predefine attributes on the newly created MONITOR node as described in Table XXIV, page 481.



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

| TABLE XXIII: **Relationships Created as a Result of CREATE_MONITOR_NODE** | |
|---|---|
| ACCESS | Created to the current user's default GROUP node. The GRANT attribute on the relationship is set to the value of the interface parameter DISCRETIONARY_ACCESS. |
| HANDLER_ATTRIBUTE | Created to the ADs for the attributes identified by attribute names specified in the interface parameter HANDLER_ATTRIBUTES. The ATTRIBUTE_VALUE attribute on each relationship contains the value for the indicated attribute as specified in the interface parameter HANDLER_ATTRIBUTES. The HANDLER_ATTRIBUTES relationships correspond to the ATTRIBUTES parameter of the CREATE_JOB interface. |
| HANDLER_ERROR_FILE | Created to the node identified by the interface parameter HANDLER_ERROR_FILE. This relationship corresponds to the ERROR_FILE parameter of the CREATE_JOB interface. |
| HANDLER_ENVIRONMENT_NODE | Created to the node identified by the interface parameter HANDLER_ENVIRONMENT_NODE. This relationship corresponds to the ENVIRONMENT_NODE parameter of the CREATE_JOB interface. |
| HANDLER_IMAGE | Created to the node identified by the interface parameter HANDLER_FILE_NODE. This relationship corresponds to the FILE_NODE parameter of the CREATE_JOB interface. |
| HANDLER_INPUT_FILE | Created to the node identified by the interface parameter HANDLER_INPUT_FILE. This relationship corresponds to the INPUT_FILE parameter of the CREATE_JOB interface. |
| HANDLER_MEDIUM_RESOURCE | Created to the node identified by the interface parameter HANDLER_MEDIUM_RESOURCE. This relationship corresponds to the MEDIUM_RESOURCE parameter of the CREATE_JOB interface. |
| HANDLER_OUTPUT_FILE | Created to the node identified by the interface parameter HANDLER_OUTPUT_FILE. This relationship corresponds to the OUTPUT_FILE parameter of the CREATE_JOB interface. |
| HANDLER_PROCESS_NODE_KIND | Created to the PROCESS NKD derived from the in_terface parameter HANDLER_PROCESS_NODE_KIND. This relationship corresponds to the PROCESS_ NODE_KIND parameter of the CREATE_JOB interface. |
| HANDLER_PROCESSOR_RESOURCE | Created to the node identified by the interface parameter HANDLER_PROCESSOR_RESOURCE. This relationship corresponds to the PROCESSOR_RESOURCE parameter of the CREATE_JOB interface. |



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

| TABLE XXIV: **MONITOR node attributes created by CREATE_MONITOR_NODE** | |
|---|---|
| Predefine Attribute (set to the value of the interface parameter of the same name): | Corresponding to the CREATE_JOB interface parameter: |
| HANDLER_AS_A_TRANSACTION | AS_A_TRANSACTION |
| HANDLER_DELETE_WHEN_-TERMINATED | DELETE_WHEN_TERMINATED |
| HANDLER_DISCRETIONARY_-ACCESS | DISCRETIONARY_ ACCESS |
| HANDLER_INPUT_PARAMETERS | INPUT_PARAMETERS |
| HANDLER_KEY | KEY |
| HANDLER_MANDATORY_-ACCESS | MANDATORY_ACCESS; the actual parameter value may specify a higher hierarchical classification level and a larger set of non-hierarchical categories. |
| HANDLER_PRIORITY | PRIORITY |

When the PROCESS node for an attribute monitor event handler process is implicitly created, the value of each of the MONITOR node attributes listed in Table XXIV is used as if it was the value for the indicated formal parameter to the procedure CREATE_JOB. Similarly, anonymous open node handles to the target nodes of the secondary relationships described in Table XXIII on page 480 are used in the implicit creation of such a PROCESS node as if those node handles were passed as the actual parameters corresponding to the formal parameters to CREATE_JOB as indicated in the table.

All of the exceptional conditions specified in the semantics of the CREATE_JOB interface apply to the implicit creation of event handler processes. The detection of an exceptional condition by the CAIS implementation during an attempt to implicitly create an event handler process for a given MONITOR node results in the failure of the attempt to post an attribute monitor event to an event handler process associated with that MONITOR node. Whether such failures are recorded and the method by which they are recorded is implementation-dependent.

### 5.8.1 Types

The following type is used in the interfaces:

```
type MONITOR_EVENT_KIND is
    (NO_EVENT, NODE_ATTRIBUTE_EVENT, PATH_ATTRIBUTE_EVENT);
```

MONITOR_EVENT_KIND is the enumeration of the kinds of attribute monitor event descriptions returned by the procedures GET_MONITOR_EVENT and AWAIT_MONITOR_EVENT.

### 5.8.2 Opening the predefine MONITOR NKD

```
function MONITOR_NKD
    (STATUS : in STATUS_INDICATOR := NO_INDICATION)
    return NKD_TYPE;
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Purpose:

This function returns an open node handle with NO_ACCESS intent to the predefine NKD for MONITOR nodes. 1

The node kind definition name MONITOR and the relation NODE_KIND_DEFINITION are associated with the node handle returned as the key and relation used in opening the handle. See subprograms PATH_KEY (Section 5.2.20 on page 119) and PATH_RELATION (Section 5.2.21 on page 120). 2

Parameters:

**STATUS** is an established status indicator. 3

Exceptions:

**STATUS_ERROR** is raised if STATUS is not an established status indicator [3]. 4

### 5.8.3 Creating MONITOR nodes

```
procedure CREATE_MONITOR_NODE
    (NODE                          : in out MONITOR_NODE_TYPE;
     HANDLER_FILE_NODE             : in     EXECUTABLE_IMAGE_NODE_TYPE;
     HANDLER_PROCESS_NODE_KIND     : in     NODE_KIND_NODE_TYPE     := NULL_NODE;
     HANDLER_INPUT_PARAMETERS      : in     PARAMETER_LIST          := EMPTY_LIST;
     HANDLER_KEY                   : in     RELATIOHSHIP_KEY        := EMPTY_KEY_LIST;
     HANDLER_DISCRETIONARY_ACCESS  : in     DISCRETIONARY_ACCESS_LIST := ALL_RIGHTS;
     HANDLER_MANDATORY_ACCESS      : in     MANDATORY_ACCESS_LIST := EMPTY_LIST;
     HANDLER_ATTRIBUTES            : in     ATTRIBUTE_LIST          := EMPTY_LIST;
     HANDLER_INPUT_FILE            : in     PATHNAME                :=  STANDARD_INPUT;
     HANDLER_OUTPUT_FILE           : in     PATHNAME                := STANDARD_OUTPUT;
     HANDLER_ERROR_FILE            : in     PATHNAME                := STANDARD_ERROR;
     HANDLER_ENVIRONMENT_NODE      : in     PATHNAME                :=  CURRENT_USER;
     HANDLER_DELETE_WHEN_TERMINATED : in    BOOLEAN                 := TRUE;
     HANDLER_AS_A_TRANSACTION      : in     BOOLEAN                 := FALSE;
     HANDLER_PROCESSOR_RESOURCE    : in     PROCESSOR_NODE_TYPE     :=  DEFAULT_PROCESSOR;
     HANDLER_PRIORITY              : in     PROCESS_PRIORITY        :=  DEFAULT_PRIORITY;
     HANDLER_MEDIUM_RESOURCE       : in     MEDIUM_NODE_TYPE        := DEFAULT_MEDIUM;
     KEY                           : in     RELATIONSHIP_KEY        := EMPTY_KEY_LIST;
     ATTRIBUTES                    : in     ATTRIBUTE_LIST          := EMPTY_LIST;
     DISCRETIONARY_ACCESS          : in     DISCRETIONARY_ACCESS_LIST :=  ALL_RIGHTS;
     MANDATORY_ACCESS              : in     MANDATORY_ACCESS_LIST := EMPTY_LIST;
     KIND                          : in     NODE_KIND_NODE_TYPE     := MONITOR_NKD;
     STATUS                        : in     STATUS_INDICATOR        :=  NO_INDICATION;
     MEDIUM_RESOURCE               : in     MEDIUM_NODE_TYPE        :=  DEFAULT_MEDIUM);
```

Purpose:

This procedure creates a MONITOR node and installs the primary relationship to it as well as the corresponding reverse relationship to the parent of the newly created node. The primary 1



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

relationship to the newly created node is an ATTRIBUTE_MONITOR relationship with the key given by the relationship key list KEY. The parent of the newly created node is the current USER node. The current process must have sufficient rights to append relationships to the current USER node. An open node handle to the newly created MONITOR node with WRITE intent is returned in NODE. A newly created MONITOR node does not monitor any attributes and does not have any attribute monitor event handler processes associated with it.

Secondary relationships emanating from the new MONITOR node are created as described in Table XXIII, page 480. 2

Predefine attributes are created on the new MONITOR node as described in Table XXIV, page 481. 3

The parameters prefixed by "HANDLER_" are parameters corresponding to those specified for CREATE_JOB (see Section 6.3.9 page 543). These parameters determine characteristics of implicitly created handler processes. In particular, the NKD for the handler PROCESS nodes created is determined by the open node handle parameter HANDLER_PROCESS_NODE_KIND. If HANDLER_PROCESS_NODE_KIND has the value NULL_NODE (the default), then the PROCESS NKD which is the target of the DEFAULT_PROCESS_NODE_KIND relationship emanating from the EXECUTABLE_IMAGE node is used or, in the absence of that relationship, the predefine PROCESS NKD is used. 4

If HANDLER_PROCESS_NODE_KIND does not have the value NULL_NODE, it must identify the PROCESS node kind to be used. In this case, if the DEFAULT_PROCESS_NODE_KIND relationship exists, the HANDLER_PROCESS_NODE_KIND parameter must identify the NKD that is the target of that relationship or a specialization of that NKD. It may specify either an NKD or an NKI (through which an NKD is determined). 5

The value of the ATTRIBUTES parameter is a named list containing the initial values for attributes to be created on the new MONITOR node. 6

KIND is an open node handle identifying the node kind for the new MONITOR node. It must identify a MONITOR node kind. It may specify either an NKD or an NKI. If KIND identifies an NKI, the NKI is made the current adopted interpretation for the node handle NODE. 7

The DISCRETIONARY_ACCESS parameter specifies the initial access control information to be established between the created MONITOR node and the default GROUP node of the current user (see Section 5.4.4 on page 259). 8

The MANDATORY_ACCESS parameter specifies the object classification labels with which the MONITOR node is to be created. If its value is the empty list, the node inherits the subject classification of the creating process as its object classification. Otherwise, it must be an unnamed list consisting of an identifier item and an empty or unnamed list of identifier items (see Table IX, page 256). 9

The MEDIUM_RESOURCE parameter specifies the medium resource on which the newly created MONITOR node will reside. 10

Parameters:

**NODE** is a node handle, initially closed, to be opened to the newly created MONITOR node. 11

**HANDLER_FILE_NODE** is an open node handle to the EXECUTABLE_IMAGE node containing the executable image whose execution will be represented by attribute monitor event handler processes implicitly associated with the newly created MONITOR node. 12



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

**HANDLER_PROCESS_NODE_KIND** is an open node handle which determines the PROCESS NKD for the PROCESS nodes of implicitly created event handler processes.

**HANDLER_INPUT_PARAMETERS** is a list containing parameter information to be stored in the INPUT_PARAMETERS attribute of the PROCESS node for each implicitly created attribute monitor event handler process.

**HANDLER_KEY** is the relationship key list used in the creation of the primary JOB relationship which emanates from the current USER node and terminates at the PROCESS node for each implicitly created attribute monitor event handler process.

**HANDLER_DISCRETIONARY_ACCESS** is the initial access control information associated with the PROCESS node for each implicitly created attribute monitor event handler process; it is the value of the GRANT attribute of the ACCESS relationship to the user's default GROUP node (see Section 5.4.4 on page 259).

**HANDLER_MANDATORY_ACCESS** is a list defining the minimum classification label that is to be given to the PROCESS node for implicitly created attribute monitor event handler processes (see Table IX, page 256).

**HANDLER_ATTRIBUTES** is a named list (see Section 9.1 on page 751) whose elements are used to establish initial values for attributes of the PROCESS node for each implicitly created attribute monitor event handler process. Each named item of the list specifies an attribute name and the value to be given to that attribute.

**HANDLER_INPUT_FILE** is a pathname to a FILE node that will be the target of the secondary STANDARD_INPUT relationship emanating from the PROCESS node for each implicitly created attribute monitor event handler process.

**HANDLER_OUTPUT_FILE** is a pathname to a FILE node that will be the target of the secondary STANDARD_OUTPUT relationship emanating from the PROCESS node for each implicitly created attribute monitor event handler process.

**HANDLER_ERROR_FILE** is a pathname to a FILE node that will be the target of the secondary STANDARD_ERROR relationship emanating from the PROCESS node for each implicitly created attribute monitor event handler process.

**HANDLER_ENVIRONMENT_NODE** is a pathname to a node each implicitly created attribute monitor event handler process will have as its initial current node.

**HANDLER_DELETE_WHEN_TERMINATED** is a Boolean; if TRUE, the PROCESS node for each implicitly created attribute monitor event handler process will be deleted when the job terminates; if FALSE, the PROCESS node will be kept when the job terminates. Note that in the latter case, an explicit call to the DELETE_JOB interface must be made in order to delete the PROCESS node.

**HANDLER_AS_A_TRANSACTION** is a Boolean; if TRUE, each implicitly created attribute monitor event handler process is enclosed in a transaction; if FALSE none are to be enclosed in a transaction.

**HANDLER_PROCESSOR_RESOURCE** is an open node handle identifying the PROCESSOR node for the processor resource on which any implicitly created attribute monitor event handler processes will execute.

**HANDLER_PRIORITY** is the priority for any implicitly created attribute monitor event handler processes.

**HANDLER_MEDIUM_RESOURCE** is an open node handle identifying the MEDIUM node for the medium resource on which the PROCESS nodes for implicitly created attribute monitor event handler processes will reside.



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

**KEY** is the relationship key list of the primary relationship to be created.

**ATTRIBUTES** is a named list (see Section 9.1 on page 751) whose elements are used to establish initial values for attributes of the newly created node. Each named item of the list specifies an attribute name and the value to be given to that attribute.

**DISCRETIONARY_ACCESS** is the initial access control information associated with the created MONITOR node; it is the value of the GRANT attribute of the ACCESS relationship to the user's default GROUP node (see Section 5.4.4 on page 259).

**MANDATORY_ACCESS** is a list defining the classification label for the created MONITOR node (see Table IX on page 256).

**KIND** is an open node handle identifying the node kind for the new node.

**STATUS** is an established status indicator.

**MEDIUM_RESOURCE** is an open node handle identifying the MEDIUM node for the medium resource on which the newly created MONITOR node will reside.

Exceptions:

**PATHNAME_SYNTAX_ERROR** is raised if the relationship key list given in KEY or HANDLER_KEY is syntactically illegal [2] or the pathname given in HANDLER_INPUT_FILE, HANDLER_OUTPUT_FILE, HANDLER_ERROR_FILE or HANDLER_ ENVIRONMENT_NODE is syntactically illegal (see Table I on page 52) [1].

**NAME_ERROR** is raised if the parent of the NKI identified by KIND or HANDLER_PROCESS_NODE_KIND is inaccessible [1].

**RELATIONSHIP_ERROR** is raised if HANDLER_KEY, KEY or any relationship key list in HANDLER_INPUT_FILE, HANDLER_OUTPUT_FILE, HANDLER_ERROR_FILE or HANDLER_ENVIRONMENT_NODE specifies a value for the same key more than once [5] or if any of the nodes identified by HANDLER_INPUT_FILE, HANDLER_OUTPUT_FILE, HANDLER_ERROR_FILE or HANDLER_ENVIRONMENT_NODE does not exist [2].

**EXISTING_NODE_ERROR** is raised if an ATTRIBUTE_MONITOR relationship with relationship key list KEY already exists [1].

**SYNTAX_ERROR** is raised if any of the parameters HANDLER_ATTRIBUTES, ATTRIBUTES [8], HANDLER_DISCRETIONARY_ACCESS, DISCRETIONARY_ACCESS, HANDLER_MANDATORY_ACCESS, MANDATORY_ACCESS is syntactically illegal [9].

**NOT_IN_VIEW_ERROR** is raised if a name specified in HANDLER_ATTRIBUTES, ATTRIBUTES, HANDLER_KEY, KEY, HANDLER_INPUT_FILE, HANDLER_OUTPUT_FILE, HANDLER_ERROR_FILE or HANDLER_ENVIRONMENT_NODE cannot be translated [1] or if HANDLER_KEY, KEY or any relationship key list in HANDLER_INPUT_FILE, HANDLER_OUTPUT_FILE, HANDLER_ERROR_FILE or HANDLER_ENVIRONMENT_NODE consists of a single relationship key designator and no key of the corresponding relation is visible [5].

**AMBIGUOUS_VIEW_ERROR** is raised if a name specified in ATTRIBUTE, KEY, HANDLER_KEY, HANDLER_INPUT_FILE, HANDLER_OUTPUT_FILE, HANDLER_ERROR_FILE or HANDLER_ENVIRONMENT_NODE has an ambiguous translation [1] or if HANDLER_KEY, KEY or any relationship key list in HANDLER_INPUT_FILE, HANDLER_OUTPUT_FILE, HANDLER_ERROR_FILE or HANDLER_ENVIRONMENT_NODE consists of a single relationship key designator and more than one key of the corresponding relation is visible [4].



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

**PREDEFINED_ATTRIBUTE_ERROR** is raised if any attribute name given within the HANDLER_ATTRIBUTES or ATTRIBUTES parameters identifies a CAIS-controlled attribute [1]. 42

**ATTRIBUTE_ERROR** is raised if HANDLER_ATTRIBUTES or ATTRIBUTES specifies a value for the same attribute more than once [6]. 43

**EXECUTABLE_IMAGE_ERROR** is raised if it can be determined that the node identified by HANDLER_FILE_NODE does not contain an executable image [1]. 44

**USE_ERROR** is raised if the value for the HANDLER_MANDATORY_ACCESS [10], HANDLER_PRIORITY [38], or MANDATORY_ACCESS [10] parameter is semantically illegal. 45

**STATUS_ERROR** is raised if KIND, HANDLER_FILE_NODE, HANDLER_PROCESSOR_RESOURCE, HANDLER_MEDIUM_RESOURCE or MEDIUM_RESOURCE is not an open node handle [2], if HANDLER_PROCESS_NODE_KIND does not have the value NULL_NODE and is not open an open node handle [2], if NODE is an open node handle prior to the call [1] or if STATUS is not an established status indicator [3]. 46

**NODE_KIND_ERROR** is raised if KIND does not identify a MONITOR node kind [2], if HANDLER_FILE_NODE does not identify an EXECUTABLE_IMAGE node [1], if HANDLER_PROCESS_NODE_KIND does not have the value NULL_NODE and does not identify the default PROCESS node kind for the node identified by HANDLER_FILE_NODE or a specialization thereof [2], if HANDLER_PROCESSOR_RESOURCE does not identify a PROCESSOR node [1] or if HANDLER_MEDIUM_RESOURCE or MEDIUM_RESOURCE does not identify a MEDIUM node [1]. 47

**CONFORMANCE_ERROR** is raised if the operation would result in an attribute, node or relationship not conforming to its definition or the constraints in force on that definition [1-5,7-10]. 48

**RESOURCE_ALLOCATION_ERROR** is raised if the medium resource identified through MEDIUM_RESOURCE or the processor resource identified through HANDLER_PROCESSOR_RESOURCE is not allocatable [1] or if the medium resource identified through HANDLER_MEDIUM_RESOURCE or MEDIUM_RESOURCE is not allocatable by a process executing on the processor resource identified through HANDLER_PROCESSOR_RESOURCE [2] 49

**RESOURCE_AVAILABILITY_ERROR** is raised if the the medium resource identified through HANDLER_MEDIUM_RESOURCE or MEDIUM_RESOURCE is unavailable or if the processor resource identified through HANDLER_PROCESSOR_RESOURCE is unavailable [1]. 50

**TRANSACTION_ERROR** is raised if the operation is included in more than one transaction [1] or if the operation is included in a transaction and the operation modifies a node which is not included in the same transaction [2]. 51

**LOCK_ERROR** is raised if access to the current USER node with intent APPEND_RELATIONSHIPS or access to the parent of the NKI specified by KIND with intent READ_RELATIONSHIPS cannot be obtained due to an existing lock on the node [1]. 52

**INTENT_VIOLATION** is raised if the node handle HANDLER_FILE_NODE was not opened with intents establishing the right to execute its contents [1]. 53

**ACCESS_VIOLATION** is raised if the discretionary access rights of the current process are insufficient to gain access with APPEND_RELATIONSHIPS intent to the current USER node or with CREATE_INSTANCES intent to the NKD identified by KIND and HANDLER_PROCESS_NODE_KIND or with READ_RELATIONSHIPS intent to the parent of the NKI 54



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

specified by KIND or with EXECUTE intent to the PROCESSOR node identified by HANDLER_PROCESSOR_RESOURCE or with APPEND_CONTENTS intent to the MEDIUM node identified by HANDLER_MEDIUM_RESOURCE or MEDIUM_RESOURCE [1].

**SECURITY_VIOLATION** is raised if the operation represents a violation of mandatory access controls. SECURITY_VIOLATION is raised only if the conditions for other exceptions are not present.

Additional Interfaces:

```
procedure CREATE_MONITOR_NODE
    (NODE                              : in out MONITOR_NODE_TYPE;
     HANDLER_FILE_NODE                 : in     EXECUTABLE_IMAGE_NODE_TYPE;
     HANDLER_PROCESS_NODE_KIND         : in     NODE_KIND_NAME;
     HANDLER_INPUT_PARAMETERS          : in     PARAMETER_LIST           := EMPTY_LIST;
     HANDLER_KEY                       : in     RELATIONSHIP_KEY         := EMPTY_KEY_LIST;
     HANDLER_DISCRETIONARY_ACCESS      : in     DISCRETIONARY_ACCESS_LIST := ALL_RIGHTS;
     HANDLER_MANDATORY_ACCESS          : in     MANDATORY_ACCESS_LIST    := EMPTY_LIST;
     HANDLER_ATTRIBUTES                : in     ATTRIBUTE_LIST           := EMPTY_LIST;
     HANDLER_INPUT_FILE                : in     PATHNAME                 := STANDARD_INPUT;
     HANDLER_OUTPUT_FILE               : in     PATHNAME                 := STANDARD_OUTPUT;
     HANDLER_ERROR_FILE                : in     PATHNAME                 := STANDARD_ERROR;
     HANDLER_ENVIRONMENT_NODE          : in     PATHNAME                 := CURRENT_USER;
     HANDLER_DELETE_WHEN_TERMINATED    : in     BOOLEAN                  := TRUE;
     HANDLER_AS_A_TRANSACTION          : in     BOOLEAN                  := FALSE;
     HANDLER_PROCESSOR_RESOURCE        : in     PROCESSOR_NODE_TYPE      := DEFAULT_PROCESSOR;
     HANDLER_PRIORITY                  : in     PROCESSOR_PRIORITY       := DEFAULT_PRIORITY;
     HANDLER_MEDIUM_RESOURCE           : in     MEDIUM_NODE_TYPE         := DEFAULT_MEDIUM;
     KEY                               : in     RELATIONSHIP_KEY         := EMPTY_KEY_LIST;
     ATTRIBUTES                        : in     ATTRIBUTE_LIST           := EMPTY_LIST;
     DISCRETIONARY_ACCESS              : in     DISCRETIONARY_ACCESS_LIST := ALL_RIGHTS;
     MANDATORY_ACCESS                  : in     MANDATORY_ACCESS_LIST    := EMPTY_LIST;
     KIND                              : in     NODE_KIND_NAME           := "MONITOR";
     STATUS                            : in     STATUS_INDICATOR         := NO_INDICATION;
     MEDIUM_RESOURCE                   : in     MEDIUM_NODE_TYPE         := DEFAULT_MEDIUM)
is
     KIND_DEFINITION                : NKI_TYPE;
     HANDLER_KIND_DEFINITION        : NKI_TYPE;


     OPEN_NODE_KIND (KIND_DEFINITION, KIND, STATUS);

     if HANDLER_PROCESS_NODE_KIND = "" then
         CREATE_MONITOR_NODE (NODE,
                              HANDLER_FILE_NODE,
                              NULL_NODE,
                              HANDLER_INPUT_PARAMETERS,
                              HANDLER_KEY,
                              HANDLER_DISCRETIONARY_ACCESS,
                              HANDLER_MANDATORY_ACCESS,
                              HANDLER_ATTRIBUTES,
                              HANDLER_INPUT_FILE,
                              HANDLER_OUTPUT_FILE,
                              HANDLER_ERROR_FILE,
```


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
                                    HANDLER_ENVIRONMENT_NODE,
                                    HANDLER_DELETE_WHEN_TERMINATED,
                                    HANDLER_AS_A_TRANSACTION,
                                    HANDLER_PROCESSOR_RESOURCE,
                                    HANDLER_PRIORITY,
                                    HANDLER_MEDIUM_RESOURCE,
                                    KEY,
                                    ATTRIBUTES,
                                    DISCRETIONARY_ACCESS,
                                    MANDATORY_ACCESS,
                                    KIND_DEFINITION,
                                    STATUS,
                                    MEDIUM_RESOURCE);
        else
            OPEN_NODE_KIND   (HANDLER_KIND_DEFINITION,
                              HANDLER_PROCESS_NODE_KIND,   STATUS);

            CREATE_MONITOR_NODE (NODE,
                                    HANDLER_FILE_NODE,
                                    HANDLER_KIND_DEFINITION,
                                    HANDLER_INPUT_PARAMETERS,
                                    HANDLER_KEY,
                                    HANDLER_DISCRETIONARY_ACCESS,
                                    HANDLER_MANDATORY_ACCESS,
                                    HANDLER_ATTRIBUTES,
                                    HANDLER_INPUT_FILE,
                                    HANDLER_OUTPUT_FILE,
                                    HANDLER_ERROR_FILE,
                                    HANDLER_ENVIRONMENT_NODE,
                                    HANDLER_DELETE_WHEN_TERMINATED,
                                    HANDLER_AS_A_TRANSACTION,
                                    HANDLER_PROCESSOR_RESOURCE,
                                    HANDLER_PRIORITY,
                                    HANDLER_MEDIUM_RESOURCE,
                                    KEY,
                                    ATTRIBUTES,
                                    DISCRETIONARY_ACCESS,
                                    MANDATORY_ACCESS,
                                    KIND_DEFINITION,
                                    STATUS,
                                    MEDIUM_RESOURCE);
        end if;

        CLOSE (KIND_DEFINITION);
        CLOSE (HANDLER_KIND_DEFINITION);
    exception
        when others =>
            CLOSE (KIND_DEFINITION);
            CLOSE (HANDLER_KIND_DEFINITION);
            raise;
    end CREATE_MONITOR_NODE;
```


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

### 5.8.4 Deleting the primary relationship to a MONITOR node

```
procedure DELETE_MONITOR_NODE
    (NODE       : in  MONITOR_NODE_TYPE;
     TIME_LIMIT : in  CAIS_DURATION       := LONG_DELAY;
     STATUS     : in  STATUS_INDICATOR    := NO_INDICATION);
```

Purpose:

This procedure deletes the primary relationship to the MONITOR node identified by NODE. The node becomes unobtainable. The node handle NODE is closed. Any MONITORS relationships emanating from the MONITOR node are deleted. Any attributes being monitored by the identified MONITOR node cease to be monitored by that node. The TIME_LIMIT parameter allows the specification of a time limit for the delay imposed by the existence of locks on the parent of the MONITOR node or the target nodes of MONITORS relationships emanating from the MONITOR node. A delayed call completes after the implicitly accessed node is unlocked or the specified time Limit has elapsed. In the latter case, the exception LOCK_ERROR is raised. 1

This procedure has no direct effect on the execution of attribute monitor event handler processes associated with the deleted MONITOR node. 2

Parameters:

**NODE** is an open node handle to the MONITOR node which is the target node of the primary relationship to be deleted. 3

**TIME_LIMIT** is a time limit for the delay on waiting for the unlocking of a node in accordance with the intent WRITE_RELATIONSHIPS. 4

**STATUS** is an established status indicator. 5

Exceptions:

**NAME_ERROR** is raised if the parent of the node identified by NODE is inaccessible [1]. 6

**USE_ERROR** is raised if any primary relationships emanate from the MONITOR node to be deleted [7]. 7

**STATUS_ERROR** is raised if the node handle NODE is not open prior to the call [2] or if STATUS is not an established status indicator [3]. 8

**CONFORMANCE_ERROR** is raised if the operation would result in an attribute, node or relationship not conforming to its definition or the constraints in force on that definition [3]. 9

**NODE_KIND_ERROR** is raised if the open node handle NODE does not identify a MONITOR node [1]. 10

**TRANSACTION_ERROR** is raised if the operation is included in more than one transaction [1] or if the operation is included in a transaction and the operation modifies a node which is not included in the same transaction [2]. 11

**LOCK_ERROR** is raised if access with intent WRITE_RELATIONSHIPS, to every node which is the target of a bidirectional MONITORS relationship emanating from the node to be deleted, cannot be obtained due to existing locks on those nodes [1]. LOCK_ERROR may be raised prior to expiration of the time limit if the CAIS implementation can determine that a deadlock situation has occurred. 12


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

**INTENT_VIOLATION** is raised if the node handle NODE was not opened with an intent including EXCLUSIVE_WRITE and READ_RELATIONSHIPS [1]. 13

**ACCESS_VIOLATION** is raised if the the current process does not have sufficient discretionary access control rights to obtain access to the parent of the node to be deleted with intent WRITE_RELATIONSHIPS and the conditions for NAME_ERROR are not present [1]. 14

**SECURITY_VIOLATION** is raised if the operation represents a violation of mandatory access controls. SECURITY_VIOLATION is raised only if the conditions for other exceptions are not present. 15

### 5.8.5 Associating an event handler process with a MONITOR node

```
procedure ASSOCIATE_EVENT_HANDLER_PROCESS
    (MONITOR : in     MONITOR_NODE_TYPE;
     NODE    : in     PROCESS_NODE_TYPE;
     KEY     : in     RELATIONSHIP_KEY   := EMPTY_KEY_LIST;
     STATUS  : in     STATUS_INDICATOR   := NO_INDICATION);
```

Purpose:

This procedure associates the event handler process whose execution is represented by the PROCESS node identified by the open node handle NODE with the MONITOR node identified by the open node handle MONITOR. The association is represented by creating a HANDLER_PROCESS relationship with key given by the relationship key list KEY which emanates from the MONITOR node identified by MONITOR and terminates at the PROCESS node identified by NODE. A corresponding reverse relationship of the relation HANDLER_PROCESS_OF is also created which emanates from the PROCESS node identified by NODE and terminates at the MONITOR node identified by MONITOR. The attribute monitor event queue of the process is initialized to the empty state. 1

Parameters:

**MONITOR** is an open node handle identifying a MONITOR node. 2

**NODE** is an open node handle identifying a PROCESS node. 3

**KEY** is a relationship key list for the identification of the newly created secondary HANDLER_PROCESS relationship. 4

**STATUS** is an established status indicator. 5

Exceptions:

**PATHNAME_SYNTAX_ERROR** is raised if the KEY parameter given is syntactically illegal [2]. 6

**RELATIONSHIP_ERROR** is raised KEY specifies a value for the same key more than once [5] or if a secondary HANDLER_PROCESS relationship with the identification given by KEY already emanates from the MONITOR node identified by MONITOR [3]. 7


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

**NOT_IN_VIEW_ERROR** is raised if a key name specified in KEY cannot be translated [1] or if KEY consists of a single relationship key designator and no key of the corresponding relation is visible [5]. 8

**AMBIGUOUS_VIEW_ERROR** is raised if a key name specified in KEY has an ambiguous translation [1] or if KEY consists of a single relationship key designator and more than one key of the corresponding relation is visible [4]. 9

**USE_ERROR** is raised if the value of the predefine attribute STATUS of the PROCESS node identified by NODE is TERMINATED or ABORTED [29], if the PROCESS node identified by NODE is already the target of a HANDLER_PROCESS relationship [30], if there is already an attribute monitor event handler process associated with the MONITOR node identified by MONITOR whose mandatory access control subject classification label is the same as the mandatory access control label on the PROCESS node identified by NODE [31], or if the USER node which is the parent of the MONITOR node identified by MONITOR is not also the target of the CURRENT_USER relationship which emanates from the PROCESS node identified by NODE [32]. 10

**STATUS_ERROR** is raised if MONITOR or NODE is not an open node handle [2] or if STATUS is not an established status indicator [3]. 11

**NODE_KIND_ERROR** is raised if MONITOR does not identify a MONITOR node or if NODE does not identify a PROCESS node [1]. 12

**CONFORMANCE_ERROR** is raised if the operation would result in an attribute, node or relationship not conforming to its definition or the constraints in force on that definition [1,2,3]. 13

**TRANSACTION_ERROR** is raised if the operation is included in more than one transaction [1] or if the operation is included in a transaction and the operation modifies a node which is not included in the same transaction [2]. 14

**INTENT_VIOLATION** is raised if the node handle MONITOR was not opened with an intent establishing the right to append relationships [1]. 16

**SECURITY_VIOLATION** is raised if the operation represents a violation of mandatory access controls. SECURITY_VIOLATION is raised only if the conditions for other exceptions are not present. 16

Notes:

An attribute monitor event handler process remains associated with the MONITOR node until it terminates or aborts or until the MONITOR node itself is deleted. When an attribute monitor event handler process terminates or aborts the HANDLER_PROCESS relationship representing its association with the MONITOR node is not deleted. 17

### 5.8.6 Specifying that a node attribute is to be monitored

```
procedure MONITOR_NODE_ATTRIBUTE
    (MONITOR   : in  MONITOR_NODE_TYPE;
     NODE      : in  NODE_TYPE;
     ATTRIBUTE : in  AD_TYPE;
     NAME      : in  IDENTIFIER_TEXT;
     STATUS    : in  STATUS_INDICATOR := NO_INDICATION);
```


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Purpose:

This procedure designates the node attribute identified by ATTRIBUTE on the node identified by the open node handle NODE to be an attribute monitored by the MONITOR node identified by the open node handle MONITOR. If a secondary MONITORS relationship which emanates from the MONITOR node and terminates at the node identified by NODE does not exist, such a secondary relationship with key LATEST_KEY is created along with a corresponding reverse relationship of the relation MONITORED_BY with key LATEST_KEY. An attribute designator designating the attribute identified by ATTRIBUTE is appended to the end of the current list of attribute designators stored in the ATTRIBUTES_MONITORED attribute on the MONITORS relationship with the item name specified by NAME. The value specified in NAME may be subsequently used to cancel the monitoring implied by the attribute designator.

The following predefined node attributes cannot be monitored:

| | |
|---|---|
| CURRENT_FILE_SIZE | HIGHEST_CLASSIFICATION |
| IO_UNIT_COUNT | LOWEST_CLASSIFICATION |
| MACHINE_TIME | OBJECT_CLASSIFICATION |
| OPEN_NODE_HANDLE_COUNT | PROCESS_SIZE |
| SUBJECT_CLASSIFICATION | |

Parameters:

**MONITOR** is an open node handle identifying a MONITOR node.

**NODE** is an open node handle identifying the node with the attribute to be monitored.

**ATTRIBUTE** is an open node handle identifying the AD for the attribute which is to be monitored.

**NAME** is the name assigned to the attribute designator item appended to the ATTRIBUTES_-MONITORED attribute value.

**STATUS** is an established status indicator.

Exceptions:

**ATTRIBUTE_ERROR** is raised if ATTRIBUTE designates a predefine attribute used for mandatory access control purposes [5].

**USE_ERROR** is raised if the node attribute identified by NODE and ATTRIBUTE is already being monitored by the MONITOR node identified by MONITOR [33] or if ATTRIBUTE identifies a predefine attribute that cannot be monitored [35].

**SYNTAX_ERROR** is raised if the value of the parameter NAME does not conform to the syntax of an Ada identifier [2].

**NAMED_LIST_ERROR** is raised if another attribute designator already exists with the name specified by NAME [4].

**STATUS_ERROR** is raised if MONITOR, NODE or ATTRIBUTE is not an open node handle [2] or if STATUS is not an established status indicator [3].

**NODE_KIND_ERROR** is raised if MONITOR does not identify a MONITOR node or if ATTRIBUTE does not identify an AD [1].



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

**CONFORMANCE_ERROR** is raised if the attribute identified by ATTRIBUTE is not a defined property of the node identified by NODE [4] or if the operation would result in an attribute, node or relationship not conforming to its definition or the constraints in force on that definition [1,2,3]. 13

**TRANSACTION_ERROR** is raised if the operation is included in more than one transaction [1] or if the operation is included in a transaction and the operation modifies a node which is not included in the same transaction [2]. 14

**INTENT_VIOLATION** is raised if the node handle MONITOR or NODE was not opened with an intent establishing the rights to read and write relationships [1]. 15

**SECURITY_VIOLATION** is raised if the operation represents a violation of mandatory access controls. SECURITY_VIOLATION is raised only if the conditions for other exceptions are not present. 16

Additional Interfaces:

```
procedure MONITOR_NODE_ATTRIBUTE
    (MONITOR   : in    MONITOR_NODE_TYPE;
     NODE      : in    NODE_TYPE;
     ATTRIBUTE : in    ATTRIBUTE_NAME;
     NAME      : in    IDENTIFIER_TEXT := "";
     STATUS    : in    STATUS_INDICATOR := NO_INDICATION)
is
    ATTRIBUTE_DEFINITION : AD_TYPE;
begin
    DEFINITION_OF_NODE_ATTRIBUTE (ATTRIBUTE_DEFINITION,
                                  NODE, ATTRIBUTE, STATUS);

    if NAME = "" then
        MONITOR_NODE_ATTRIBUTE (MONITOR, NODE, ATTRIBUTE_DEFINITION,
                                ATTRIBUTE, STATUS);
    else
        MONITOR_NODE_ATTRIBUTE (MONITOR, NODE, ATTRIBUTE_DEFINITION,
                                NAME, STATUS);
    end if;

    CLOSE (ATTRIBUTE_DEFINITION);
exception
    when others =>
        CLOSE (ATTRIBUTE_DEFINITION);
        raise;
end MONITOR_NODE_ATTRIBUTE;
```

Notes:

The additional interface uses the attribute name as the item name of the attribute designator appended if the NAME parameter is omitted. 17

The predefine attributes TIME_ATTRIBUTE_WRITTEN, TIME_RELATIONSHIP_WRIT- 18



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

TEN, and TIME_CONTENTS_WRITTEN can be monitored to detect most modifications to a node.

### 5.8.7 Specifying that a path attribute is to be monitored

```
procedure MONITOR_PATH_ATTRIBUTE
    (MONITOR   : in  MONITOR_NODE_TYPE ;
     NODE      : in  NODE_TYPE;
     KEY       : in  RELATIONSHIP_KEY  := EMPTY_KEY_LIST;
     RELATION  : in  RD_TYPE;
     ATTRIBUTE : in  AD_TYPE;
     NAME      : in  IDENTIFIER_TEXT;
     STATUS    : in  STATUS_INDICATOR  := NO_INDICATION);
```

Purpose:

This procedure designates all relationship attributes, which are instances of the AD identified by ATTRIBUTE on relationships of the relation identified by RELATION with key values specified by KEY emanating from the node identified by the open node handle NODE, as attributes monitored by the MONITOR node identified by the open node handle MONITOR. If a secondary MONITORS relationship which emanates from the MONITOR node and terminates at the node identified by NODE does not exist, such a secondary relationship with key LATEST_KEY is created along with a corresponding reverse relationship of the relation MONITORED_BY with key LATEST_KEY. A relationship attribute designator designating the relationship attributes indicated by ATTRIBUTE, RELATION and KEY is appended to the end of the current list of attribute designators stored in the ATTRIBUTES_MONITORED attribute on the MONITORS relationship. The item name is specified by NAME. The value specified in NAME may be subsequently used to cancel the monitoring of relationship attributes implied by the relationship attribute designator.

Parameters:

**MONITOR** is an open node handle identifying a MONITOR node.

**NODE** is an open node handle identifying the node from which the relationship emanates whose attribute is to be monitored.

**KEY** is the relationship key list identifying relationships whose attribute is to be monitored; the identified relationships are not required to exist.

**RELATION** is an open node handle identifying an RD for relationships which may emanate from the node identified by NODE.

**ATTRIBUTE** is an open node handle identifying the AD for the attribute to be monitored.

**NAME** is the name assigned to the attribute designator item appended to the ATTRIBUTES_-MONITORED attribute value.

**STATUS** is an established status indicator.



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Exceptions:

**PATHNAME_SYNTAX_ERROR** is raised if the KEY given is syntactically illegal [2]. 9

**SYNTAX_ERROR** is raised if the value of the parameter NAME does not conform to the syntax of an Ada identifier [2]. 10

**NOT_IN_VIEW_ERROR** is raised if a key name specified in KEY cannot be translated [1] or if KEY consists of a single relationship key designator and no key of the specified relation is visible [5]. 11

**AMBIGUOUS_VIEW_ERROR** is raised if a key name specified in KEY has an ambiguous translation [1] or if KEY consists of a single relationship key designator and more than one key of the specified relation is visible [4]. 12

**RELATIONSHIP_ERROR** is raised if KEY specifies a value for the same key more than once [5]. 13

**NAMED_LIST_ERROR** is raised if another attribute designator already exists with the name specified by NAME [4]. 14

**STATUS_ERROR** is raised if MONITOR, NODE, ATTRIBUTE or RELATION is not an open node handle [2] or if STATUS is not an established status indicator [3]. 15

**NODE_KIND_ERROR** is raised if MONITOR does not identify a MONITOR node or if ATTRIBUTE does not identify an AD or if RELATION does not identify an RD [1]. 16

**CONFORMANCE_ERROR** is raised if relationships of the relation identified by RELATION cannot emanate from the node identified by NODE [1] or if the attribute identified by ATTRIBUTE or any key identified in KEY is not a defined key of the relation identified by RELATION [4] or if the operation would result in an attribute, node or relationship not conforming to its definition or the constraints in force on that definition [1,2,3]. 17

**TRANSACTION_ERROR** is raised if the operation is included in more than one transaction [1] or if the operation is included in a transaction and the operation modifies a node which is not included in the same transaction [2]. 18

**INTENT_VIOLATION** is raised if the node handle MONITOR or NODE was not opened with an intent establishing the rights to read and write relationships [1]. 19

**SECURITY_VIOLATION** is raised if the operation represents a violation of mandatory access controls. SECURITY_VIOLATION is raised only if the conditions for other exceptions are not present. 20

Additional Interfaces:

```
procedure MONITOR_PATH_ATTRIBUTE
    (MONITOR   : in  MONITOR_NODE_TYPE;
     NODE      : in  NODE_TYPE;
     KEY       : in  RELATIONSHIP_KEY  := EMPTY_KEY_LIST;
     RELATION  : in  RELATION_NAME;
     ATTRIBUTE : in  ATTRIBUTE_NAME;
     NAME      : in  IDENTIFIER_TEXT := "";
     STATUS    : in  STATUS_INDICATOR := NO_INDICATION)
is
     RELATION_DEFINITION : RD_TYPE;
```


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
        ATTRIBUTE_DEFINITION : AD_TYPE;
    begin
      DEFINITION_OF_RELATION (RELATION_DEFINITION, NODE, RELATION, STATUS);
      DEFINITION_OF_NODE_ATTRIBUTE (ATTRIBUTE_DEFINITION,
                                    NODE, ATTRIBUTE, STATUS);
      if NAME = "" then
          MONITOR_PATH_ATTRIBUTE (MONITOR, NODE, KEY, RELATION_DEFINITION,
                                  ATTRIBUTE_DEFINITION, ATTRIBUTE, STATUS);
      else
          MONITOR_PATH_ATTRIBUTE (MONITOR, NODE, KEY, RELATION_DEFINITION,
                                  ATTRIBUTE_DEFINITION, NAME, STATUS);
      end if;

      CLOSE (RELATION_DEFINITION);
      CLOSE (ATTRIBUTE_DEFINITION);
    exception
        when others =>
            CLOSE (RELATION_DEFINITION);
            CLOSE (ATTRIBUTE_DEFINITION);
            raise;
    end MONITOR_PATH_ATTRIBUTE;
```

Notes:

The additional interface uses the attribute name as the item name of the relationship attribute designator appended if the NAME parameter is omitted.

### 5.8.8 Canceling monitoring of designated attributes

```
procedure CANCEL_ATTRIBUTE_MONITORING
    (MONITOR   : in  MONITOR_NODE_TYPE;
     NODE      : in  NODE_TYPE;
     NAME      : in  IDENTIFIER_TEXT;
     STATUS    : in  STATUS_INDICATOR := NO_INDICATION);
```

Purpose:

This procedure deletes the attribute designator identified by NAME associated with the node identified by the open node handle NODE and the MONITOR node identified by the open node handle MONITOR. In the absence of other attribute designators monitoring the same attributes, the effect is to stop monitoring of the designated attributes by the specified MONITOR node. The MONITORS relationship is deleted if the resultant value of the ATTRIBUTES_MONITORED attribute is the empty fist. Any unhandled attribute motitor events which were previously posted to the attribute motitor event queues of attribute monitor event handler processes associated with the MONITOR node identified by MONITOR are unaffected by a call to this procedure.


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Parameters:

**MONITOR** is an open node handle identifying a MONITOR node.

**NODE** is an open node handle identifying the node having an attribute monitored by the MONITOR node.

**NAME** is the item name of the attribute designator to be deleted from the value of the ATTRIBUTES_MONITORED attribute of the MONITORS relationship emanating from the node identified by MONITOR and terminating at the node identified by NODE.

**STATUS** is an established status indicator.

Exceptions:

**SYNTAX_ERROR** is raised if the value of the parameter NAME does not conform to the syntax of an Ada identifier [2].

**USE_ERROR** is raised if attributes of the specified node are not being monitored through the specified MONITOR node [34].

**SEARCH_ERROR** is raised if NAME does not identify an attribute designator for the node identified by NODE and the monitor identified by MONITOR [1].

**STATUS_ERROR** is raised if MONITOR, NODE or ATTRIBUTE is not an open node handle [2] or if STATUS is not an established status indicator [3].

**NODE_KIND_ERROR** is raised if MONITOR does not identify a MONITOR node or if ATTRIBUTE does not identify an AD [1].

**CONFORMANCE_ERROR** is raised if the operation would result in an attribute, node or relationship not conforming to its definition or the constraints in force on that definition [3].

**TRANSACTION_ERROR** is raised if the operation is included in more than one transaction [1] or if the operation is included in a transaction and the operation modifies a node which is not included in the same transaction [2].

**INTENT_VIOLATION** is raised if the node handle MONITOR or NODE was not opened with an intent establishing the rights to read and write relationships [1].

**SECURITY_VIOLATION** is raised if the operation represents a violation of mandatory access controls. SECURITY_VIOLATION is raised only if the conditions for other exceptions are not present.

### 5.8.9 Determining a monitored attribute modification

```
procedure GET_MONITOR_EVENT
    (NODE          : in out NODE_TYPE;
     ATTRIBUTE     : in out AD_TYPE;
     RELATION      : in out RD_TYPE;
     KEY           :    out RELATIONHIP_KEY;
     LAST          :    out CAIS_NATURAL;
     EVENT_KIND    :    out MONITOR_EVENT_KIND;
     TIME_LIMIT    : in     CAIS_DURATION     := LONG_DELAY;
     STATUS        : in     STATUS_INDICATOR  := NO_INDICATION);
```


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Purpose:

This procedure removes successive events from the head of the attribute monitor event queue of the current process until the event queue is emptied or until a reportable event is removed from the queue. If at the time of the call the attribute monitor event queue of the current process is empty or the queue becomes empty before a reportable event is encountered, the value NO_EVENT is returned in EVENT_KIND and no description of an attribute monitor event is returned. (That is, zero is returned in LAST, the node handles NODE, ATTRIBUTE and RELATION remain closed, and the actual parameter corresponding to the parameter KEY is not affected by the call.) Otherwise, if a reportable event is encountered before the attribute queue is emptied, a description of that attribute monitor event is returned in the parameters EVENT_KIND, NODE, ATTRIBUTE, RELATION, KEY and LAST. 1

If an event removed from the attribute monitor event queue is an event resulting from the creation, modification or deletion of a node attribute, an attempt is made to open the node handle NODE with READ_ATTRIBUTES intent to the node having the monitored attribute. In the case where an event removed from the queue is an event resulting from the creation, modification or deletion of a relationship attribute, an attempt is made to open the node handle NODE with READ_RELATIONSHIPS intent to the node from which the relationship having the monitored attribute emanates. In both cases, the opening of the node handle NODE fails if the node to be opened is unobtainable or inaccessible, or if the discretionary access rights of the current process are insufficient to gain access to the node with the particular intent, or if the open operation represents a violation of mandatory access controls, If the opening of the node handle NODE is successful for a given event, the event is a reportable event and a description of the event is returned. 2

If after the removal of a reportable event from the event queue the CAIS implementation de-ten-nines that the conditions for the raising of the exception LOCK_ERROR are present, the reportable event is re-inserted into the event queue at the head of the event queue. 3

Descriptions of reportable events are returned as follows. An open node handle with NO_ACCESS intent to the AD for the monitored attribute is returned in ATTRIBUTE. In the case of an event resulting from the monitoring of a node attribute, the value NODE_ATTRIBUTE_EVENT is returned in EVENT_KIND, zero is returned in LAST, the node handle RELATION remains closed, and the actual parameter corresponding to the parameter KEY is not affected by the call. 4

In the case of an event resulting from the creation, modification, or deletion of a relationship attribute, the value PATH_ATTRIBUTE_EVENT is returned in EVENT_KIND. An open node handle with intent NO_ACCESS is returned in RELATION which identifies the RD for the relationship. A relationship key list identifying the relationship is returned in KEY. The index of the last character of the relationship key list returned in KEY is returned in LAST. The index of the first character returned in KEY is KEY'FIRST. 5

The relationship key list returned in KEY is formed using relationship key names defined within the current composite view of the current process according to the semantics of identification formation using views described in Section 5.7.3.6 on page 355. If there is only one relationship key value in the relationship key list, the key value is expressed as a relationship key designator. Otherwise, the relationship key value is expressed as key value associations. 6

The MONITOR relationship from the MONITOR node (associated with the event handler process) to the monitored node is the last relationship traversed in opening the node handle 7


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

NODE. No path is traversed to open the handles ATTRIBUTE and RELATION; hence, the subprograms PATH_KEY (Section 5.2.20 on page 119) and PATH_RELATION (Section 5.2.21 on page 120) cannot be applied to the node handles ATTRIBUTE and RELATION.

Parameters:

**NODE** is a node handle, initially closed, to be opened when an event description is returned to the node having the monitored attribute or the emanating relationship with the monitored attribute. 8

**ATTRIBUTE** is a node handle, initially closed, to be opened with NO_ACCESS intent to the AD for the monitored attribute. 9

**RELATION** is a node handle, initially closed, to be opened with NO_ACCESS intent to the RD for the relationship having the monitored attribute. 10

**KEY** is a relationship key list identifying the relationship having the monitored attribute. 11

**LAST** is the index of the last character of the relationship key list placed into KEY, 12

**EVENT_KIND** is indicates the kind of attribute monitor event description being returned. 13

**TIME_LIMIT** is a time limit for the delay encountered during the processing of a given attribute monitor event removed from the attribute monitor event queue of the current process while waiting for the unlocking of a monitored node in accordance with the intent (either READ_ATTRIBUTES or READ_RELATIONSHIPS) with which the node handle NODE is to be opened. 14

**STATUS** is an established status indicator. 15

Exceptions:

**NOT_IN_VIEW_ERROR** is raised if a key name to be returned in KEY cannot be formed [3]. 16

**AMBIGUOUS_VIEW_ERROR** is raised if an ambiguous view is searched for a key name during the formation of a key value association to be returned in KEY [2]. 17

**USE_ERROR** is raised if the current process is not an attribute monitor event handler process [37]. 16

**LOCK_ERROR** is raised if for a given attribute monitor event removed from the event queue the opening of the node handle NODE is delayed beyond the specified time limit due to the existence of locks in conflict with the particular intent (READ_ATTRIBUTES or READ_-RELATIONSHIPS) [1]. LOCK_ERROR may also be raised prior to the expiration of the time limit if the CAIS implementation can determine that a deadlock situation has occurred. 19

**STATUS_ERROR** is raised if NODE, ATTRIBUTE or RELATION is an open node handle prior to the call [1] or if STATUS is not an established status indicator [3]. 20

### 5.8.10 Awaiting a monitored attribute modification

```
procedure AWAIT_MONITOR_EVENT
    (NODE                : in out NODE_TYPE;
     ATTRIBUTE           : in out AD_TYPE;
```


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
RELATION              : in out RD_TYPE;
KEY                   :    out RELATIONSHIP_KEY;
LAST                  :    out CAIS_NATURAL;
EVENT_KIND            :    out MONITOR_EVENT_KIND;
ARRIVAL_TIME_LIMIT : in      CAIS_DURATION      := LONG_DELAY;
OPEN_TIME_LIMIT       : in   CAIS_DURATION      := LONG_DELAY;
STATUS                : in   STATUS_INDICATOR   := NO_INDICATION);
```

Purpose:

This procedure removes successive attribute monitor events from the attribute monitor event queue of the current process until a reportable event is removed from the queue. A description of the reportable event is returned in the parameters EVENT_KIND, NODE, ATTRIBUTE, RELATION, KEY and LAST. If the attribute monitor event queue of the current process becomes empty prior to the removal of a reportable event, this procedure suspends the calling task and waits for the posting of another attribute monitor event to the attribute monitor event queue. The calling task is suspended until an attribute monitor event is posted to the process' attribute monitor event queue or until the time limit specified in the parameter ARRIVAL_TIME_LIMIT is exceeded. This procedure may return prior to the expiration of the time limit specified in the ARRIVAL_TIME_LIMIT parameter if the CAIS implementation can determine that the posting of additional attribute monitor events to the process' attribute monitor event queue is not possible due to the associated MONITOR node having become unobtainable. When this procedure returns prior to the removal of a reportable event from the event queue, the value NO_EVENT is returned in the EVENT_KIND parameter, zero is returned in LAST, the node handles NODE, ATTRIBUTE and RELATION remain closed and actual parameter corresponding to the parameter KEY is not affected by the call. 1

If after the removal of a reportable event from the event queue the CAIS implementation determines that the conditions for the raising of the exception LOCK_ERROR are present, the reportable event is re-inserted into the event queue at the head of the event queue. 2

If an event removed from the attribute monitor event queue is an event resulting from the creation, modification or deletion of a node attribute, an attempt is made to open the node handle NODE with READ_ATTRIBUTES intent to the node having the monitored attribute. In the case where an event removed from the queue is an event resulting from the creation, modification or deletion of a relationship attribute, an attempt is made to open the node handle NODE with READ_RELATIONSHIPS intent to the node from which the relationship having the monitored attribute emanates. In both cases, the opening of the node handle NODE fails if the node to be opened is unobtainable or inaccessible, or if the discretionary access rights of the current process are insufficient to gain access to the node with the particular intent, or if the open operation represents a violation of mandatory access controls. If the opening of the node handle NODE is successful for a given event, the event is a reportable event and a description of the event is returned. 3

Descriptions of reportable events are returned as follows. An open node handle with NO_ACCESS intent to the AD for the monitored attribute is returned in ATTRIBUTE. In the case of an event resulting from the monitoring of a node attribute, the value NODE_ATTRIBUTE_EVENT is returned in EVENT_KIND, zero is returned in LAST, the node handle RELATION remains closed, and the actual parameter corresponding to the parameter KEY is not affected by the call. 4



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

In the case of an event resulting from the creation, modification, or deletion of a relationship attribute, the value PATH_ATTRIBUTE_EVENT is returned in EVENT_KIND. An open node handle with intent NO_ACCESS is returned in RELATION which identifies the RD for the relationship. A relationship key list identifying the relationship is returned in KEY. The index of the last character of the relationship key list returned in KEY is returned in LAST. The index of the first character returned in KEY is KEY'FIRST.

The relationship key list returned in KEY is formed using relationship key names defined within the current composite view of the current process according to the semantics of identification formation using views described in Section 5.7.3.6 on page 355. If there is only one relationship key value in the relationship key list, the key value is expressed as a relationship key designator. Otherwise, the relationship key value is expressed as key value associations.

The MONITOR relationship from the MONITOR node (associated with the event handler process) to the monitored node is the last relationship traversed in opening the node handle NODE. No path is traversed to open the handles ATTRIBUTE and RELATION; hence, subprograms PATH_KEY (Section 5.2.20 on page 119) and PATH_RELATION (Section 5.2.21 on page 120) cannot be applied to the node handles ATTRIBUTE and RELATION.

Parameters:

**NODE** is a node handle, initially closed, to be opened when an event description is returned to the node having the monitored attribute or the emanating relationship with the monitored attribute.

**ATTRIBUTE** is a node handle, initially closed, to be opened with NO_ACCESS intent to the AD for the monitored attribute.

**RELATION** is a node handle, initially closed, to be opened with NO_ACCESS intent to the RD for the relationship having the monitored attribute.

**KEY** is a relationship key list identifying the relationship having the monitored attribute.

**LAST** is the index of the last character of the relationship key list placed into KEY.

**EVENT_KIND** is indicates the kind of attribute monitor event description being returned.

**ARRIVAL_TIME_LIMIT** is the maximum time to wait for the posting of an attribute monitor event to the event queue of the current process.

**OPEN_TIME_LIMIT** is a time limit for the delay encountered during the processing of a given attribute monitor event removed from the attribute monitor event queue of the current process while waiting for the unlocking of a monitored node in accordance with the intent (either READ_ATTRIBUTES or READ_RELATIONSHIPS) with which the node handle NODE is to be opened.

**STATUS** is an established status indicator.

Exceptions:

**NOT_IN_VIEW_ERROR** is raised if a key name to be returned in KEY cannot be formed [3].

**AMBIGUOUS_VIEW_ERROR** is raised if an ambiguous view is searched for a key name during the formation of a key value association to be returned in KEY [2].



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

**USE_ERROR** is raised if the current process is not an attribute monitor event handler process [37]. 19

**LOCK_ERROR** is raised if for a given attribute monitor event removed from the event queue the opening of the node handle NODE is delayed beyond the specified time limit due to the existence of locks in conflict with the particular intent (READ_ATTRIBUTES or READ_-RELATIONSHIPS) [1]. LOCK_ERROR may also be raised prior to the expiration of the time limit if the CAIS implementation can determine that a deadlock situation has occurred. 20

**STATUS_ERROR** is raised if NODE, ATTRIBUTE or RELATION is an open node handle prior to the call [1] or if STATUS is not an established status indicator [3]. 21



Source: https://assist.dla.mil -- Downloaded: 202610-26T07:54Z
Check the source to verify that this is the current version before use.

# 5.9 Package CAIS_RESOURCE_CONTROL

This package provides facilities for resource control. Services are provided for reassigning nodes to particular medium resources, determining the medium resource to which a node has been assigned, and providing node handles to default PROCESSOR and MEDIUM nodes. 1

## 5.9.1 Reassigning a node to a new medium resource

```
procedure REASSIGN_NODE
    (NODE            : in  NODE_TYPE;
     MEDIUM_RESOURCE : in  MEDIUM_NODE_TYPE;
     STATUS          : in  STATUS_INDICATOR := NO_INDICATION);
```

Purpose:

This procedure reassigns a node to a new medium resource. Implementations are not obliged to support the reassignment of all nodes. Reassignment of a node to the MEDIUM node to which it is already assigned has no effect. If MEDIUM_RESOURCE identifies the TRANSPARENT_MEDIUM_RESOURCE node, then, where possible, the node will be reassigned by the underlying implementation to a MEDIUM node other than the MEDIUM node to which it is currently assigned. 1

Parameters:

**NODE** is an open node handle to the node to be reassigned. 2

**MEDIUM_RESOURCE** is an open node handle to the MEDIUM node to which the node specified by NODE is to be reassigned. 3

**STATUS** is an established status indicator. 4

Exceptions:

**NODE_KIND_ERROR** is raised if the MEDIUM_RESOURCE parameter does not identify a MEDIUM node [1]. 5

**STATUS_ERROR** is raised if NODE or MEDIUM_RESOURCE is not an open node handle [2], or STATUS is not an established status indicator [3]. 6

**RESOURCE_ERROR** is is raised if the implementation does not support reassignment of the node identified by NODE [1]. 7

**TRANSACTION_ERROR** is raised if the operation is included in more than one transaction [1] or if the operation is included in a transaction and the operation modifies a node which is not included in the same transaction [2]. 8

**INTENT_VIOLATION** is raised if the node identified by the NODE parameter was not opened with an intent establishing EXCLUSIVE_WRITE access [1]. 9

**ACCESS_VIOLATION** is raised if discretionary access control rights of the current process are insufficient to obtain access to the node identified by MEDIUM_RESOURCE consistent with the implied intent APPEND_CONTENTS [1]. 10


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

**RESOURCE_ALLOCATION_ERROR** is raised if the medium resource identified through MEDIUM_RESOURCE is not allocatable [1]. 11

**RESOURCE_AVAILABILITY_ERROR** is raised if the medium resource identified through MEDIUM_RESOURCE is not available [1]. 12

**SECURITY_VIOLATION** is raised if the operation represents a violation of mandatory access controls and the conditions for other exceptions are not present. 13

### 5.9.2 Determining the medium resource to which a node is assigned

```
procedure GET_MEDIUM_RESOURCE
    (MEDIUM_RESOURCE : in out MEDIUM_NODE_TYPE;
     NODE            : in     NODE_TYPE;
     STATUS          : in     STATUS_INDICATOR := NO_INDICATION);
```

Purpose:

This procedure returns a node handle to the MEDIUM node to which the node identified by NODE is assigned. The returned node handle is opened with NO_ACCESS intent. If the node is assigned to a medium resource which is not in a visible group of the current process (see Section 4.6.6 on page 82), then a handle to the predefined TRANSPARENT_MEDIUM_RESOURCE node will be returned. 1

The MEDIUM node is opened without traversing any relationships; hence, the subprograms PATH_KEY (Section 5.2.20 on page 119) and PATH_RELATION (Section 5.2.21 on page 120) cannot be applied to the node handle MEDIUM_RESOURCE. 2

Parameters:

**MEDIUM_RESOURCE** is a node handle, initially closed, to be opened to the MEDIUM node to which the node is assigned. 3

**NODE** is an open node handle to the node. 4

**STATUS** is an established status indicator. 5

Exceptions:

**INTENT_VIOLATION** is raised if the node identified by the NODE parameter was not opened with an intent establishing READ_RELATIONSHIPS access [1]. 6

**STATUS_ERROR** is raised if NODE is not an open node handle [2], if MEDIUM_RESOURCE is open at the time this procedure is called [1], or if STATUS is not an established status indicator [3]. 7

**SECURITY_VIOLATION** is raised if the operation represents a violation of mandatory access controls and the conditions for other exceptions are not present. 8



Source: https://assist.dla.mil - Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Notes:

If the node is assigned to a medium resource which has no MEDIUM node, then a handle to the TRANSPARENT_MEDIUM_RESOURCE node will be returned.

### 5.9.3 Determining the processor resource to which a process is assigned

```
procedure GET_PROCESSOR_RESOURCE
    (PROCESSOR_RESOURCE : in out PROCESSOR_NODE_TYPE;
     PROCESS_NODE       : in     PROCESS_NODE_TYPE;
     STATUS             : in     STATUS_INDICATOR := NO_INDICATION);
```

Purpose:

This procedure returns a handle to the PROCESSOR node to which the PROCESS node specified by PROCESS_NODE is assigned. The returned node handle is opened with NO_ACCESS intent. If the node is assigned to a processor resource which is not in a visible group of the current process (see Section 4.6.6 on page 82), then a handle to the predefine TRANSPARENT_PROCESSOR_-RESOURCE node will be returned.

The PROCESSOR node is opened without traversing any relationships; hence, the subprograms PATH_KEY (Section 5.2.20 on page 119) and PATH_RELATION (Section 5.2.21 on page 120) cannot be applied to the node handle PROCESS OR_RESOURCE.

Parameters:

**PROCESSOR_RESOURCE** is a node handle, initially closed, to be opened to the PRO-CESSOR node to which the PROCESS node is assigned.

**PROCESS_NODE** is an open node handle to the PROCESS node of interest.

**STATUS** is an established status indicator.

Exceptions:

**NODE_KIND_ERROR** is raised if the PROCESS_NODE parameter does not identify a PROCESS node.

**INTENT_VIOLATION** is raised if the node identified by PROCESS_NODE was not opened with an intent establishing READ_RELATIONSHIPS access [1].

**STATUS_ERROR** is raised if PROCESS_NODE is not an open node handle [2], if PROCES-SOR_RESOURCE is open at the time this procedure is called [1], or if STATUS is not an established status indicator [3].

**SECURITY_VIOLATION** is raised if the operation represents a violation of mandatory access controls and the conditions for other exceptions are not present.



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Notes:

If the process is assigned to a processor resource which has no PROCESSOR node, then a handle to the TRANSPARENT_PROCESSOR_RESOURCE node will be returned.

### 5.9.4 Determining the current default medium resource

```
function DEFAULT_MEDIUM
    (STATUS :  in STATUS_INDICATOR : = NO_INDICATION)
    return MEDIUM_NODE_ TYPE;
```

Purpose:

This function returns a node handle to the MEDIUM node which is the target of the DEFAULT_-MEDIUM relationship emanating from the current PROCESSOR node. The returned node handle bears the implied intent NO_ACCESS. If a DEFAULT_MEDIUM relationship does not emanate from the current PROCESSOR node, an open node handle to the TRANSPARENT_-MEDIUM_RESOURCE node will be returned. If the MEDIUM node is not in a visible group of the current process, then an open node handle to the TRANSPARENT_MEDIUM_RESOURCE node will be returned instead.

In the case that an open handle to the TRANSPARENT_MEDIUM_RESOURCE node is returned, the key value TRANSPARENT_MEDIUM_RES OURCE and the secondary relation MEDIUM are associated with the node handle returned as the key and relation used in opening the handle. See subprograms PATH_KEY (Section 5.2.20 on page 119) and PATH_RELATION (Section 5.2.21 on page 120).

Parameters:

**STATUS** is an established status indicator.

Exceptions:

**ACCESS_VIOLATION** is raised if discretionary access control rights of the current process are insufficient to obtain access to the current PROCESSOR node consistent with the implied intent READ_RELATIONSHIPS [1].

**STATUS_ERROR** is raised if STATUS is not an established status indicator [3].

**TRANSACTION_ERROR** is raised if the operation is included in more than one transaction [1].

**SECURITY_VIOLATION** is raised if the operation represents a violation of mandatory access controls and the conditions for other exceptions are not present.

### 5.9.5 Determining the current default processor resource

```
function DEFAULT_PROCESSOR
    (STATUS :  in STATUS_INDICATOR : = NO_INDICATION)
    return  PROCESSOR_NODE_TYPE;
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Purpose:

This function returns a node handle to the PROCESSOR node which is the target of the DEFAULT_PROCESSOR relationship emanating from the current PROCESSOR node. The returned node handle is opened with NO_ACCESS intent. If the default PROCESSOR is not in a visible group of the current process, an open node handle to the TRANSPARENT_-PROCESSOR_RESOURCE node will be returned.

In the case that an open handle to the TRANSPARENT_PROCESSOR_RESOURCE node is returned, the key value TRANSPARENT_PROCESSOR_RESOURCE and the secondary relation PROCESSOR are associated with the node handle returned as the key and relation used in opening the handle. See subprograms PATH_KEY (Section 5.2.20 on page 119) and PATH_-RELATION (Section 5.2.21 on page 120).

Parameters:

**STATUS** is an established status indicator.

Except ions:

**ACCESS_VIOLATION** is raised if discretionary access control rights of the current process are insufficient to obtain access to the current PROCESSOR node consistent with the implied intent READ_RELATIONSHIPS [1].

**STATUS_ERROR** is raised if STATUS is not an established status indicator [3].

**TRANSACTION_ERROR** is raised if the operation is included in more than one transaction [1].

**SECURITY_VIOLATION** is raised if the operation represents a violation of mandatory access controls and the conditions for other exceptions are not present.



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

# 6. CAIS PROCESS MANAGEMENT DETAILED REQUIREMENTS

This section describes the semantics of the execution of Ada programs, represented as CAIS processes, and the facilities provided by the CAIS for initiating and controlling processes. The major stages in the life of a process are creation, activation, completion, and termination or abortion. Subsequent to activation, suspension and resumption may also occur. The CAIS defines facilities to control and coordinate the transition of a process through these stages in its life. 1

The CAIS further defines facilities to open and close transactions (see Section 6.1 on page 514) which allow a designated sequence of operations to have the appearance of an atomic operation, including operations performed by multiple processes. 2

Each CAIS process has a current status associated with it which changes to reflect a process's transition through its stages of execution. The attribute CURRENT_STATUS, which exists on all PROCESS nodes, contains the current status of a process. 3

Process creation consists of creation of a PROCESS node, creation of a process, allocation of resources necessary for the initial execution of the new process and implicit adoption of roles and views for the new process. A process which has been created is not executed until it is activated. The ability to create a process without activating it provides the creator an opportunity to complete the allocation of resources and establish the environment required by the new process before execution is begun. 4

The mechanisms for creating a new process without activating it are: 5

a. Create dependent process — the procedure CREATE_PROCESS_NODE creates a new dependent process for a specified program and returns without activating the new process. The parent of the new PROCESS node is the PROCESS node for the current process. An open node handle identifying the newly created PROCESS node is returned. 6

b. Create root process — the procedure CREATE_JOB_NODE creates a new root process for a specified program and returns without activating the new process. The parent of the new PROCESS node is the USER node for the current user. An open node handle identifying the newly created PROCESS node is returned. 7

In each of the above mechanisms, the CURRENT_STATUS attribute of the newly created process is set to CREATED. 8

The mechanisms for activating a process that has the CREATED status are: 9

a. Activate — the procedure ACTIVATE_PROCESS activates a process that has the CREATED status. Activation indicates that the process is ready for execution. Control is returned to the calling task after the process has been activated. The calling process and the activated process execute in parallel, and within each of them, their tasks may execute in parallel. 10

b. Activate and await termination — the procedure ACTIVATE_AND_AWAIT_TERMINATION activates a process that has the CREATED status. Control returns to the calling task after the activated process has terminated (or a time limit is reached). While the calling task is blocked waiting for termination of the process, other tasks in the calling process may execute in parallel with the created process and its tasks. 11



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

When a process is activated, the CURRENT_STATUS attribute of the activated process is set to READY, indicating that the process is ready to be executed. 12

The CAIS also defines the following facilities to create a process and activate it immediately: 13

a. Spawn — the procedure SPAWN_PROCESS returns after creating a process for a specified program and activating it. An open node handle identifying the newly created node is returned. 14

b. Invoke — the procedure INVOKE_PROCESS creates and activates a process for a specified program but does not return control to the calling task until the new process has terminated (or a time limit is reached). An open node handle identifying the created node is returned. 15

c. Create job — the procedure CREATE_JOB returns after creating and activating a root process for a specified program. 16

A process is said to be *created* after the PROCESS node has been created. The value of the CURRENT_STATUS attribute of a created process which has not been activated or aborted is CREATED. 17

A process is said to be *ready* when it is ready for execution. A process which is ready is executed. A process initially becomes ready when it is activated. The value of the CURRENT_STATUS attribute of a ready process is READY. 18

A process is said to be *suspended* when its execution is stopped such that it can later be resumed. A process may be suspended either by itself or by another process. If an Ada program is suspended, all tasks are suspended and no tasks may be activated until the process is resumed. A process may be resumed even if its parent process is not resumed. The value of the CURRENT_STATUS attribute of a suspended process is SUSPENDED. 19

A process is said to be *completed* when its main program (in the sense of [1815A] 10.1) has terminated (in the sense of [1815A] 9.4) but not all processes spawned by this process have terminated. See also the notes in [1815A] 9.4. A process can become completed by normal termination of its main program, through an unhandled exception (when the main program is abandoned in the sense of [1815A] 11.4.1), or by being aborted. When the main program of a process terminates normally or is abandoned, the process is said to have *completed normally* and the value of its CURRENT_STATUS attribute is set to COMPLETED_NORMALLY. When a process becomes completed by being aborted it is said to have *completed abnormally* and the value of its CURRENT_STATUS attribute is set to COMPLETED_ABNORMALLY. An exception handler, with an OTHERS clause that calls ABORT_PROCESS, may be included in the main program to force abnormal completion when an otherwise unhandled exception occurs during execution of the main program. 20

A process is said to be *aborted* when its execution is stopped, via a call to the ABORT_PROCESS interface or as a result of PROCESS node deletion, prior to termination of the process. A process may be aborted either by itself or by another process. A process maybe aborted before it has been activated. When a process is aborted the value of its CURRENT_STATUS attribute is set to COMPLETED_ABNORMALLY. 21

A process is said to be *terminated* when it has completed and all processes spawned by the process have terminated. When a process that has completed abnormally becomes terminated the value of its CURRENT_STATUS attribute is set to ABORTED. When a process that has completed normally becomes terminated the value of its CURRENT_STATUS attribute is set to TERMINATED. 22

Figure 41 shows the allowable transitions of the current status of a process. 23



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

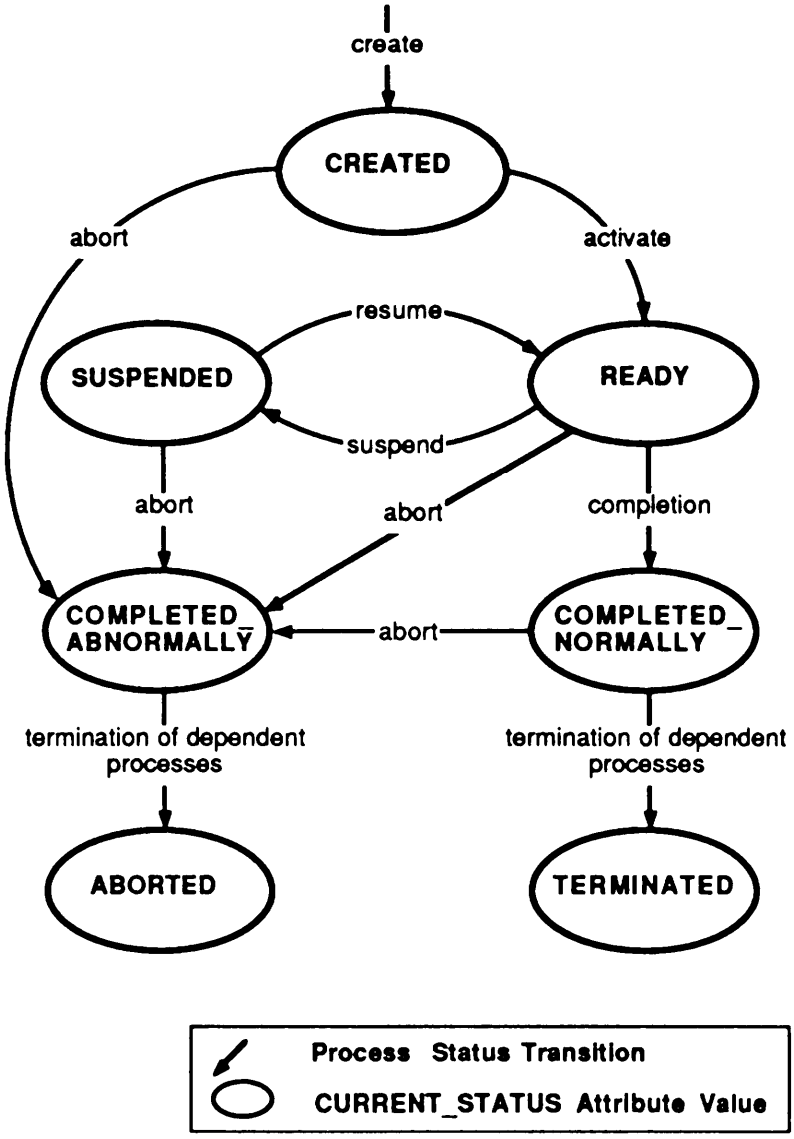


FIGURE 41. Allowable process status transitions

Table XXV identifies the events that cause the current status of a process to change and states how it is changed by the event. 24

For a given process, a process tree is the set of processes consisting of the given process plus each process whose node's unique primary path traverses the node of the given process. When a process is aborted, suspended or resumed, all of the processes in its process tree are likewise aborted, suspended or resumed, subject to discretionary access control; in any case their PROCESS nodes remain until deleted. 25

When a process terminates the following actions are performed on its behalf: 26

a. All open node handles of the process are closed. 27

b. All open transaction handles of the process are closed. The identified transactions are committed in the case of normal termination and aborted otherwise. 28



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

TABLE XXV: **Events Causing Process Status Transition**

| Event | Current Status | Current Status After the Event |
|---|---|---|
| Process Creation | Non-existent | CREATED |
| | All Other States | Not Applicable |
| Process Activation | CREATED | READY |
| | All other states | Not Applicable |
| Invocation of Interface SUSPEND. PROCESS | READY | SUSPENDED |
| | All Other States | No Effect |
| Invocation of Interface RESUME-PROCESS | SUSPENDED | READY |
| | All Other States | No Effect |
| Completion or Abandonment of Main Program | READY | COMPLETED-NORMALLY |
| | All Other States | Not Applicable |
| Invocation of Interface ABORT_ PROCESS | CREATED | COMPLETED. ABNORMALLY |
| | READY | COMPLETED-ABNORMALLY |
| | SUSPENDED | COMPLETED_ ABNORMALLY |
| | COMPLETED_ NORMALLY | COMPLETED-ABNORMALLY |
| | All Other States | No Effect |
| Termination of the Process | COMPLETED-NORMALLY | TERMINATED |
| | COMPLETED_ ABNORMALLY | ABORTED |
| | All Other States | Not Applicable |

c. A delete request event is generated (through an implicit call to the RELEASE-CHANNEL interface) for each of the channels for which the process is a client that is not in the RETIRED state. 29

d. All open node set handles of the process are closed. 30

e. All adopted roles of the process are unadopted. 31

f. All view adoptions of the process are cancelled. 32

g. All NKI adoptions of the process are cancelled. 33

h. All established iterators are deleted. 34



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

The operations which create PROCESS nodes determine the priority at which the process will execute. Among ready processes, higher priority processes have preferred status in the allocation of processor resources. However, a CAIS implementation may interpret process priorities as a recommendation for the allocation of processor resources as opposed to a strict requirement for such allocation. Thus a CAIS implementation's process scheduling may interleave the execution of higher and lower priority processes. Therefore, process priorities should not be used for process synchronization. The functions LOWEST_USER_PRIORITY and HIGHEST_USER_PRIORITY (Section 6.3.19 on page 553) return the lowest and highest priorities which may be assigned to a process created by the current user for execution on a specified processor. These priorities will be equal for CAIS implementations in which all processes have equal status in the allocation of processor resources. 35

There is no predefine effect on process priorities when two processes engaged in interprocess communication have different priorities. The interface SET_PRIORITY can be used to explicitly modify a process' priority, if supported by the CAIS implementation. 36

Every PROCESS node has several attributes. Four of these are: RESULTS, which can be used to store multiple user_defined strings giving (intermediate) results of the process; PARAMETERS, which contains the parameters with which the process was initiated; CURRENT_STATUS, which gives the current status of the process (see Table XXV, page 512); and PRIORITY, whose value is the most recent priority of the process. The PRIORITY attribute is set at process initiation. Some implementations may allow it to be modified through the SET_PRIORITY interface. 37

In addition, every PROCESS node has several attributes which provide information for standardized debugging and performance measurement of processes within the CAIS implementation. One of these attributes, OPEN_NODE_HANDLE_COUNT, gives the number of node handles the process currently has open. The following attributes have implementation-dependent values and should not be used for comparison with values from other CAIS implementations. TIME_STARTED and TIME_FINISHED give the time of activation and the time of termination of the process. MACHINE_TIME gives the length of time the process has been active on the logical processor. IO_UNIT_COUNT gives the number of input and output operations that have been performed by the process. PROCESS_SIZE gives the amount of memory currently in use by the process. 38

The CURRENT_STATUS, OPEN_NODE_HANDLE_COUNT, TIME_STARTED, TIME_FINISHED, MACHINE_TIME, and IO_UNIT_COUNT attributes are maintained by the implementation and cannot be set using CAIS interfaces. 39

When a process has terminated, the final status recorded in the PROCESS node attribute CURRENT_STATUS, will persist as long as the PROCESS node exists. CURRENT_STATUS may be examined by the CAIS interfaces CURRENT_STATUS and GET_RESULTS. 40

For purposes of input and output, every PROCESS node may have one relationship of each of the following relations: STANDARD_INPUT, STANDARD_OUTPUT and STANDARD_ERROR. STANDARD_INPUT, STANDARD_OUTPUT and STANDARD_ERROR relationships are optionally established at process creation. The STANDARD_INPUT and STANDARD_OUTPUT files conform to the semantics given for these in [1815A] 14.3.2, except that these files are not automatically open upon initiation of process execution. 41


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

# 6.1 Transactions

A transaction is a sequence of operations on designated nodes which has the appearance of anatomic operation whose effects can either be retained or discarded as if the operations never took place. The CAIS defines access synchronization rules for the nodes included in a transaction which restrict the visibility of the effects of a transaction while that transaction is in progress. Such synchronization ensures that the effects of operations performed within a transaction are the same whether or not the transaction occurs concurrently with other transactions. (However, processes containing the transactions may themselves not be serializable, due to effects which are not included within transactions.) 1

When a transaction is in progress, it is said to be open. Once a transaction has been opened, nodes may be included in the transaction (see Section 6.1.3 on page 515). The operations included in a transaction are operations on included nodes (see Section 6.1.4 on page 515). Eligible effects of operations are effects on nodes which can be undone as if the operation never took place (see Section 6.1.8 on page 518). The eligible effects of operations included in a transaction are said to be the eligible effects of the transaction. 2

A transaction remains open until it is closed, at which time the transaction is either committed or aborted. If a transaction is *committed,* all effects of the transaction are retained; if a transaction is *aborted,* the eligible effects are discarded (see Section 6.1.6 on page 517). 3

### 6.1.1 Subtransactions and enclosing transactions

When a transaction is opened, it can be designated as being nested within another transaction, called its *parent transaction.* A nested transaction is called a *subtransaction.* The *descendants* of a transaction are the subtransactions of the transaction and the descendants of those subtransactions, transitively. When a subtransaction is committed, its included nodes and effects are transferred to its parent transaction (see Section 6.1.6 on page 517). 1

An operation which is included in a transaction or one of its descendant transactions is said to be performed inside the transaction. Conversely, an operation which is not included in a transaction or a descendant of that transaction is said to be performed outside the transaction. 2

Transactions may cross process boundaries. An *enclosing transaction* for a process is a transaction that is associated with the process when the process is created. All transactions opened by the process are descendants of the enclosing transaction; all operations on nodes performed by the process which are not included in any transaction opened by the process are included in the enclosing transaction; and, all nodes opened by the process are included in the enclosing transaction. A process which has an enclosing transaction is said to be enclosed by the transaction. Any dependent process created by an enclosed process is either enclosed by the same transaction or a descendant of that transaction. As a consequence, all operations on nodes performed by processes in the process tree of an enclosed process are performed inside the enclosing transaction (see Section 6.1.7 on page 518). 3

### 6.1.2 Opening transactions

A transaction may be opened by a call to the procedure OPEN_TRANSACTION. This procedure returns an *open transaction handle* which identifies the newly opened transaction. A parent transaction may be designated for the new transaction by the PARENT parameter of OPEN_TRANSACTION. If no parent transaction is designated, the enclosing transaction for the current process (if any) is the parent; otherwise the newly opened transaction has no parent. A process may have multiple transaction 1



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

handles opened concurrently. However, an enclosed process does not have an open transaction handle which identifies its enclosing transaction; thus a process cannot perform explicit operations on its enclosing transaction.

### 6.1.3 Inclusion of nodes

The operations comprising a transaction are operations on designated nodes. These designated nodes are said to be *included* in the transaction. Nodes may be explicitly or implicitly included in an open transaction. An included node remains included until the transaction is closed. A node maybe included with either read or write intent. The inclusion of a node with read intent has no effect if the node is already included with either read or write intent. The inclusion of a node with write intent has no effect if the node is already included with write intent. The function IS_INCLUDED may be called to determine whether a node is included in a transaction.

Nodes are explicitly included in a transaction by calling the procedures INCLUDE and INCLUDE_TREE. In calls to these procedures, the transaction is identified by an open transaction handle and a node to be included is identified by an open node handle. The node handle must be open with an intent to access the node. When INCLUDE_TREE is called, all nodes in the primary tree rooted at the identified node are included in the transaction. The nodes are included with write intent if the node handle is open with any write or control intent. Otherwise, the nodes are included with read intent.

Operations included in a transaction may cause nodes to be implicitly included in a transaction:

1. If an operation on an included node opens the node with any write or control intent, the node is implicitly included with write intent.
2. If an operation creates a node, the created node is implicitly included with write intent.
3. If an operation in an enclosed process opens a node with an intent to access the node, the node is implicitly included in the enclosing transaction of the process. The node is included with write intent if it is opened with any write or control intent. Otherwise, the node is included with read intent.

Note that an operation may open a node explicitly, e.g. via OPEN or CHANGE_INTENT, or implicitly (see Section 5.2.1 on page 95). Some operations may implicitly access a node without opening the node (see Section 5.2.2 on page 96).

### 6.1.4 Inclusion of operations

An *operation on a node* is any operation which accesses the node via an open node handle or opens the node with an intent to access the node; and, any operation which accesses the node but does not access any node via a handle or open any node with an intent to access it (as described in Section 5.2.2 on page 96).

An operation performed by a process is *included* in a transaction opened by the process or in the enclosing transaction for the process as follows:

1. An operation which explicitly includes a node in a transaction is included in the transaction; otherwise,



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

2. an operation on a node included in a transaction is included in the transaction if it is not an operation on any node included in a descendant of the transaction; and

3. an operation on nodes not included in any transaction opened by the process is included in the enclosing transaction, if any.

TRANSACTION_ERROR is raised if an operation is included in more than one transaction, if an operation included in a transaction modifies a node which is not included in the same transaction or if a write operation included in a transaction accesses a SECONDARY_STORAGE or CHANNEL node using a file handle that was not opened inside the transaction.

Some consequences of the inclusion rules for nodes and operations are:

1. If an operation included in a transaction creates or deletes a bidirectional relationship, both nodes involved must be included in the transaction.

2. A created node and the corresponding creation operation can be included in a transaction only if the parent node is included in the transaction, prior to creation.

3. If a definition node is modified in a transaction, a node that conforms to the definition node as modified can be created in the transaction only if the parent of the created node is included in the transaction.

4. When a node is created whose parent is not included in a transaction, the creation of the node cannot be included in the transaction. However, subsequent operations on the created node can be included in the transaction if the node is included in the transaction after creation.

5. If operations performed using a file handle are to be included in a transaction, the operation which opens the file handle must be performed inside the transaction.

### 6.1.5 Visibility of effects

Prior to commitment, the effects of a transaction are visible only to other operations performed inside that transaction. Thus, the effects of a subtransaction are not visible to operations performed inside the parent transaction (but outside the subtransaction) until the subtransaction is committed.

To restrict the visibility of effects, a transaction establishes an access synchronization lock on each included node. These locks are held by the transaction until it is closed. When a node is included with read or write intent, a corresponding EXCLUSIVE_READ or EXCLUSIVE_WRITE lock is acquired by the transaction. The lock does not grant access rights; however, the synchronizing effect of the lock is the same as that which occurs when an open node handle is created with the intent having the same name, with the following exceptions:

1. The lock only blocks operations performed outside the transaction. It has no effect on operations performed inside the transaction.

2. The lock also blocks any operation performed outside the transaction which has the effect of including the locked node in another transaction.

Thus, the effect of the lock on an included node is to block the following operations when they are performed outside the transaction:



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

1. Operations which access or open the node with a write or control intent.

2. Operations which access the node or open the node with any intent to access the node, if the node has been included with a write intent.

3. Operations which would include the node in another transaction.

An EXCLUSIVE_READ lock on a node held by a transaction is upgraded to an EXCLUSIVE_WRITE lock if an operation on the node causes the node to be included with write intent. Upgrading the lock may block the operation until other locks on the node are released. An EXCLUSIVE_WRITE lock held by a transaction is never downgraded, e.g. as a result of changing intents or transferring the lock to its parent transaction.

Note that operations which create, modify or delete nodes, and operations which iterate over instances of a definition implicitly access definition nodes. Such operations may therefore be blocked when performed outside a transaction which includes a definition node implicitly accessed by the operation.

### 6.1.6 Closing Transactions

When a transaction is closed, it is always either aborted or committed. Thus, all eligible effects of the transaction are either retained or discarded.

When a transaction which is not a subtransaction is committed, the synchronization locks held by the transaction are released; all effects are retained and become visible to all operations.

When a subtransaction is committed, the effects, the included nodes, and the locks held by that sub-transaction are transferred to its parent transaction as if all operations included in the subtransaction were included in the parent transaction. Thus, all nodes included in the subtransaction are included in the parent transaction. The transfer of locks causes the eligible effects of the closed subtransaction to become visible to all operations performed inside the parent transaction.

When a transaction is aborted, the eligible effects of the transaction are discarded and the locks held by the transaction are released. File handles opened inside the transaction are closed. Nodes created by included operations become unobtainable and all open handles to such nodes are closed. Since an operation which creates a relationship is an operation on the source node of the relationship but not necessarily an operation on the target node, it is possible for an operation performed outside a transaction to create a relationship which terminates at a node created by an operation performed inside the transaction. If, after the creation of the relationship, the transaction is aborted, the created relationship terminates at an unobtainable node.

When a process terminates, all open transactions opened by the process are implicitly closed. If the process terminates normally, each implicitly closed transaction is committed. If the process terminates abnormally (because the process was aborted), these transactions are aborted.

A process may call CLOSE_TRANSACTION to close a transaction opened by that process. The value of the Boolean parameter COMMIT_MODIFICATIONS determines whether the transaction is to be committed or aborted. The transaction may be committed only if there are no open subtransactions of the transaction and all dependent processes enclosed by the transaction have terminated, If the transaction is to be aborted, the operation is applied recursively to all open subtransactions. The operation also attempts to abort each process created in or enclosed by the transaction that has not terminated. If the access rights of the current process do not permit the abortion of such a process,



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

that process is not aborted, the transaction remains open and an exception is raised; otherwise, the transaction is aborted.

An open transaction is aborted if a system failure occurs prior to closure of the transaction. For a system failure occurring during the closure of a transaction that is to decommitted, it is implementation-dependent whether the transaction is committed or aborted.

When a transaction is closed, any open transaction handle that identifies the transaction is also closed. A *closed transaction handle* does not identify a transaction. The function IS_OPEN maybe used to determine the status of a transaction. The function IS_SAME may be used to compare transaction handles.

### 6.1.7 Enclosing processes in transactions

The enclosing transaction (if any) for a dependent process is determined by the ENCLOSING_TRANSACTION parameter of the interfaces CREATE_PROCESS_NODE, SPAWN_PROCESS or INVOKE_PROCESS in package CAIS_PROCESS_MANAGEMENT when the process is created. This parameter may be an open transaction handle which identifies the enclosing transaction. However, if NULL_TRANSACTION is specified as the value of ENCLOSING_TRANSACTION, the enclosing transaction for the process created is the same as that of the creating process, if any. By designating an enclosing transaction for a dependent process, the creating process is able to conditionally discard the eligible effects of all operations performed by that dependent process and by the descendants of that process.

An enclosing transaction is implicitly opened for a root PROCESS node if TRUE is passed as the value of the AS_A_TRANSACTION parameter to the procedure CREATE_JOB_NODE or CREATE_JOB when the root PROCESS node is created. The transaction is implicitly closed when the process terminates. If termination is normal, the transaction is committed. If termination is abnormal (due to abortion of the process), the transaction is also aborted. The effect of the transaction is otherwise the same as the effect of an enclosing transaction for a dependent process. Hence, if the job represented by the root PROCESS node is aborted the eligible effects of all operations performed by processes in the job are discarded.

### 6.1.8 Eligible effects of operations

The *eligible effects* of a CAIS operation are modifications to nodes caused by the operation which may be undone as though the operation had not occurred. All effects of CAIS operations on nodes are eligible except the following:

1. Effects on the PROCESS node attributes CURRENT_STATUS, IO_UNIT_COUNT, MACHINE_TIME, OPEN_NODE_HANDLE_COUNT, PRIORITY, PROCESS_SIZE, RESULTS, TIME_FINISHED, and TIME_STARTED.
2. Effects on the contents of a PROCESS node, i.e. effects on the execution state of a process.
3. Effects on the contents of a FILE node other than a SECONDARY_STORAGE node.

Excluding such effects, the occurrence of an operation performed inside a transaction which is aborted is not discernible based upon an inspection of nodes modified by the operation.



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Some operations on nodes have no eligible effects. For example, since effects on PROCESS node contents and the PROCESS node attributes listed above are ineligible, the effects of aborting the execution of a process are not eligible. Similarly, the effects of opening a node handle are not eligible. Since effects on the contents of DEVICE nodes are not eligible, the effects of writing data to a terminal are not eligible. 6



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

## 6.2 Package CAIS_PROCESS_DEFINITIONS

This package defines the types and exceptions associated with PROCESS nodes. 1

```
type PROCESS_STATUS_KIND is (CREATED, READY, SUSPENDED,
                             COMPLETED_ABNORMALLY, COMPLETED_NORMALLY,
                             ABORTED, TERMINATED);
```

An object of type PROCESS_STATUS_KIND is the status of a process. 2

```
subtype PROCESS_PRIORITY is CAIS_INTEGER;
```

An object of subtype PROCESS_PRIORITY is the priority of a process. 3

```
subtype RESULTS_LIST    is CAIS_LIST_MANAGEMENT_LIST_TYPE;
subtype RESULTS_STRING is STRING;
subtype PARAMETER_LIST is CAIS_LIST_MANAGEMENT_LIST_TYPE;
```

An object of type RESULTS_LIST is a list of results from a process. The elements of this list are of type RESULTS_STRING. An object of type PARAMETER_LIST is a list containing process parameter information. 4

```
ROOT_PROCESS     : constant PATHNAME := "'CURRENT_JOB";
STANDARD_INPUT   : constant PATHNAME := "'STANDARD_INPUT";
STANDARD_OUTPUT  : constant PATHNAME := "'STANDARD_OUTPUT";
STANDARD_ERROR   : constant PATHNAME := "'STANDARD_ERROR";
```

ROOT_PROCESS is a standard pathname for the root PROCESS node of the current job. STANDARD_INPUT, STANDARD_OUTPUT and STANDARD_ERROR are pathnames for the standard input, output and error files, respectively, of the current process. 5

```
EXECUTABLE_IMAGE_ERROR :  exception;
```

EXECUTABLE_IMAGE_ERROR is raised if the contents of an EXECUTABLE_IMAGE node cannot be executed. 6



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

## 6.3 Package CAIS_PROCESS_MANAGEMENT

This package specifies interfaces for the creation, activation and termination of processes, and the examination and modification of PROCESS node attributes. The exceptions raised by all subprograms in this package are defined in the packages CAIS_DEFINITIONS, CAIS_PROCESS_DEFINITIONS, and CAIS_PRAGMATICS. 1

As part of the creation of root PROCESS nodes (jobs), new secondary relationships emanating from the newly created PROCESS node are created as described in Table XXVI. 2

As part of the creation of dependent PROCESS nodes, several secondary relationships of predefine relations are created, emanating from the newly created PROCESS node. In addition, the newly created PROCESS node inherits all secondary relationships from the PROCESS node of the creating process for which IS_INHERITABLE (see Section 4.3.2.2 on page 42) is TRUE. 3

Table XXVII summarizes the inheritance or creation of all predefined relationships which emanate from the newly created dependent PROCESS node. 4



Source: https://assist.dla.mil -- Downloaded: 202610-26T07:54Z
Check the source to verify that this is the current version before use.

TABLE XXVI: **Relationships Created for Root PROCESS Nodes**

| **A Secondary Relationship of the Predefined Relation:** | **Is Created to the Node Identified by the:** |
|---|---|
| ACCESS | Pathname 'CURRENT_-USER'DEFAULT_ROLE. (The GRANT attribute is set by the interface parameter DISCRETIONARY_ACCESS.) |
| ADOPTED_ROLE | Pathname 'CURRENT_USER'DEFAULT_-ROLE. This secondary relationship is also created to the default GROUP node of the node containing the executable image of the program, if such a GROUP node exists. |
| ADOPTED_VIEW | Predefine VIEW node, implementation-defined subset of top-level VIEW nodes and VIEW nodes associated with the EXECUTABLE_-IMAGE node containing the executable image. |
| CURRENT_JOB | Newly-created root PROCESS node. |
| CURRENT_NODE | Interface parameter ENVIRONMENT_NODE. |
| CURRENT_PROCESSOR | Interface parameter PROCESSOR_RESOURCE. |
| CURRENT_USER | User's USER node. |
| DEFINITIONS | Predefined DEFINITIONS node, which is the root of the tree of predefine definitions, and an implementation-defined subset of top-level DEFINITIONS nodes. |
| DEVICE | Implementation-defined subset of DEVICE nodes. |
| EXECUTABLE_IMAGE | Interface parameter FILE_NODE. |
| GROUP | Implementation-defined subset of GROUP nodes. |
| PARENT | Constant CURRENT_USER. |
| RESOURCE_GROUP | Implementation-defined subset of RESOURCE_GROUP nodes. |
| STANDARD_ERROR | Interface parameter ERROR_FILE. |
| STANDARD_INPUT | interface parameter INPUT_FILE. |
| STANDARD_OUTPUT | Interface parameter OUTPUT_FILE. |
| USER | Implementation-defined subset of USER nodes. |
| VIEW | Predefined VIEW node and an implementation-defined subset of top-level VIEW nodes. |



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

TABLE XXVII: **Relationships Inherited or Created for Dependent PROCESS Nodes**

| **A Secondary Relationship of the Predefined Relation:** | **Is:** |
|---|---|
| ACCESS | Created to the node identified by the pathname 'CURRENT_USER'DEFAULT_ROLE. (The GRANT attribute is set by the interface parameter DISCRETIONARY_ACCESS.) |
| ADOPTED_ROLE | Created to the default GROUP node of the node containing the executable image of the program, if such a GROUP node exists; in addition, the PROCESS node inherits the ADOPTED_ROLE relationships from its creating process. |
| ADOPTED_VIEW | Inherited from the creating process. Also created to VIEW nodes associated with EXECUTABLE_IMAGE node containing the executable image. |
| CURRENT_JOB | Created to the root PROCESS node of a PROCESS node's job. |
| CURRENT_NODE | Created to the node identified by the interface parameter ENVIRONMENT_NODE. |
| CURRENT_PROCESSOR | Created to the node identified by the interface parameter PROCESSOR_RESOURCE. |
| CURRENT_USER | Inherited from the creating process. |
| DEFINITIONS | Inherited from the creating process. |
| DEVICE | Inherited from the creating process. |
| EXECUTABLE_IMAGE | Created to the node identified by the interface parameter FILE. NODE. |
| GROUP | Inherited from the creating process. |
| PARENT | Created to the PROCESS node of the creating process. |
| RESOURCE_GROUP | Inherited from the creating process. |
| STANDARD_ERROR | Created to the node identified by the interface parameter ERROR_FILE. |
| STANDARD_INPUT | Created to the node identified by the interface parameter INPUT_FILE. |
| STANDARD_OUTPUT | Created to the node identified by the interface parameter OUTPUT_FILE. |
| USER | Inherited from the creating process. |
| VIEW | Inherited from the creating process. |



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

### 6.3.1 Determining the default priority

```
function DEFAULT_PRIORITY
    (PROCESSOR : in     PROCESSOR_NODE_TYPE  := DEFAULT_PROCESSOR;
     STATUS    : in     STATUS_INDICATOR     := NO_INDICATION)
    return PROCESS_PRIORITY;
```

Purpose:

This function returns the priority at which the current process will, by default, initiate a new process on the processor identified through PROCESSOR.

Parameters:

**PROCESSOR** is an open node handle identifying a PROCESSOR node.

**STATUS** is an established status indicator.

Exceptions:

**NODE_KIND_ERROR** is raised if the open node handle PROCESSOR does not identify a PROCESSOR node [1].

**USE_ERROR** is raised if the current process cannot initiate processes on the processor specified [39].

**STATUS_ERROR** is raised if PROCESSOR is not an open node handle prior to the call [2], or if STATUS is not an established status indicator [3].

### 6.3.2 Creating a dependent PROCESS node

```
procedure CREATE_PROCESS_NODE
    (NODE                    : in out PROCESS_NODE_TYPE;
     FILE_NODE               :  in    EXECUTABLE_IMAGE_NODE_TYPE;
     INTENT                  :  in    INTENT_ARRAY            := (READ, WRITE);
     IMPUT_PARAMETERS        :  in    PARAMETER_LIST          := EMPTY_LIST;
     KEY                     :  in    RELATIONSHIP_KEY        := EMPTY_KEY_LIST;
     RELATION                :  in    RD_TYPE;
     DISCRETIONARY_ACCESS    :  in    DISCRETIONARY_ACCESS_LIST := ALL_RIGHTS;
     ATTRIBUTES              :  in    ATTRIBUTE_LIST          := EMPTY_LIST;
     INPUT_FILE              :  in    PATHNAME                := "";
     OUTPUT_FILE             :  in    PATHNAME                := "";
     ERROR_FILE              :  in    PATHNAME                := "";
     ENVIRONMENT_NODE        :  in    PATHNAME                := CURRENT_NODE;
     PROCESS_NODE_KIND       :  in    NODE_KIND_NODE_TYPE     := NULL_NODE;
     ENCLOSING_TRANSACTION   :  in    TRANSACTION_TYPE        := NULL_TRANSACTION;
     STATUS                  :  in    STATUS_INDICATOR        := NO_INDICATION;
     PRIORITY                :  in    PROCESS_PRIORITY        := DEFAULT_PRIORITY;
     PROCESSOR_RESOURCE      :  in    PROCESSOR_NODE_TYPE     := DEFAULT_PROCESSOR;
     MEDIUM_RESOURCE         :  in    MEDIUM_NODE_TYPE        := DEFAULT_MEDIUM);
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Purpose:

This procedure creates a new dependent PROCESS node whose contents represent the execution of the program contained in the EXECUTABLE_IMAGE node identified by FILE_NODE. The primary relationship to the newly created PROCESS node emanates from the current PROCESS node. The relationship belongs to the relation identified by RELATION and has the relationship keys specified in KEY. The reverse link of the primary relationship is automatically created. The process is not activated; the CURRENT_STATUS attribute of its node is set to CREATED. The PROCESS node containing the calling task must have execution rights for the node identified by FILE_NODE. An open node handle NODE to the new node is returned, with an intent as specified by the INTENT parameter. The new process, as a subject, has all discretionary access rights to its own PROCESS node (the object). The subject and object classifications labels for the new PROCESS node are inherited from the root PROCESS node of the current job. 1

Secondary relationships emanating from the new PROCESS node are created and inherited as described in Table XXVII, page 523. If the parameters INPUT_FILE, OUTPUT_FILE, or ERROR_FILE have the null string as their value, then the secondary relationship corresponding to that parameter is not created. 2

The DISCRETIONARY_ACCESS parameter specifies the initial access control information to be established between the created node and the default GROUP node of the current user (see Section 5.4 on page 255). 3

The NKD for the PROCESS node created is determined by the node handle parameter PROCESS_NODE_KIND. If PROCESS_NODE_KIND has the value NULL_NODE (the default), then the PROCESS NKD which is the target of the DEFAULT_PROCESS_NODE_KIND relationship emanating from the EXECUTABLE_IMAGE node is used or, in the absence of that relationship, the predefine PROCESS NKD is used. 4

If PROCESS_NODE_KIND does not have the value NULL_NODE, it must identify the PROCESS node kind to be used. It may specify either an NKD or an NKI (from which an NKD is determined). If PROCESS_NODE_KIND identifies an NKI, the NKI is made the current adopted interpretation for the node handle NODE. If the DEFAULT_PROCESS_NODE_KIND relationship exists, the NKD identified through the PROCESS_NODE_KIND parameter must be the NKD which is the target of that relationship or a specialization of that NKD. 5

The parameter ENCLOSING_TRANSACTION is a transaction handle which determines the enclosing transaction for the new process. If ENCLOSING_TRANSACTION has the value NULL_TRANSACTION, the enclosing transaction is the same as the enclosing transaction for the current process, if any. Otherwise, the handle identifies an open transaction (opened by the current process) as the enclosing transaction for the new process. All operations on nodes performed by the new process and its descendants are performed inside the enclosing transaction, if any. The enclosing transaction cannot be committed until the new process terminates (see CLOSE_TRANSACTION Section 6.1.6, page 517). All transactions opened by the new process and its descendants will be descendants of the enclosing transaction. 6

Parameters:

**NODE** is a node handle, initially closed, to be opened to the newly created PROCESS node. 7

**FILE_NODE** is an open node handle to the EXECUTABLE_IMAGE node containing the executable image whose execution will be represented by the new process. 8



Source: https://assist.dla.mil -- Downloaded: 202610-26T07:54Z
Check the source to verify that this is the current version before use.

**INTENT** is the intent of subsequent operations on the node; the actual parameter takes the form of an array aggregate.

**INPUT_PARAMETERS** is a list containing process parameter information. The list is constructed and parsed using the tools provided in CAIS_LIST_MANAGEMENT (see Section 9 on page 745). The value of INPUT_PARAMETERS is stored in the PARAMETERS attribute of the new node.

**KEY** is the relationship key list of the primary relationship from the current PROCESS node to the new PROCESS node.

**RELATION** is an open node handle identifying the RD for the primary relationship from the current PROCESS node to the new PROCESS node.

**DISCRETIONARY_ACCESS** is the initial access control information associated with the created node; it is the value of the GRANT attribute of the ACCESS relationship created to the user's default GROUP node (see Section 5.4.6.1 on page 262).

**ATTRIBUTES** is an empty or named list (see Section 9 on page 745) whose elements are used to establish initial values for attributes of the newly created node; each named item of the list specifies an attribute name and the value to be given to that attribute.

**INPUT_FILE** is a pathname to a FILE node that will be the target node of the secondary STANDARD_INPUT relationship.

**OUTPUT_FILE** is a pathname to a FILE node that will be the target node of the secondary STANDARD_OUTPUT relationship.

**ERROR_FILE** is a pathname to a FILE node that will be the target node of the secondary STANDARD_ERROR relationship.

**ENVIRONMENT_NODE** is a pathname to a node the new process will have as its initial current node.

**PROCESS_NODE_KIND** is a node handle which determines the PROCESS NKD for the PROCESS node created. If it has the value NULL_NODE, a default PROCESS NKD is used, otherwise it identifies the PROCESS node kind to be used.

**ENCLOSING_TRANSACTION** is a transaction handle that designates an enclosing transaction for the new process. If it has the value NULL_TRANSACTION, it designates the enclosing transaction of the calling process, if any. Otherwise, it is an open transaction handle that identifies the enclosing transaction.

**STATUS** is an established status indicator.

**PRIORITY** is the priority for the new process.

**PROCESSOR_RESOURCE** is an open node handle identifying a PROCESSOR node. It specifies the processor resource on which the process will execute.

**MEDIUM_RESOURCE** is an open node handle identifying a MEDIUM node. It specifies the medium resource on which the newly created node will reside.

Exceptions:

**PATHNAME_SYNTAX_ERROR** is raised if the relationship key list given in KEY [2] is syntactically illegal or the pathname given in INPUT_FILE, OUTPUT_FILE, ERROR_FILE or ENVIRONMENT_NODE is syntactically illegal [1].



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

**NAME_ERROR** is raised if the parent of the NKI identified by PROCESS_NODE_KIND is inaccessible [1]. 26

**RELATIONSHIP_ERROR** is raised if any of the nodes identified by INPUT_FILE, OUTPUT_FILE, ERROR_FILE, or ENVIRONMENT_NODE does not exist [2]; or if KEY or any relationship key list in INPUT_FILE, OUTPUT_FILE, ERROR_FILE, or ENVIRONMENT_NODE specifies a value for the same key more than once [5]. 27

**EXISTING_NODE_ERROR** is raised if a relationship of the relation RELATION with the relationship key list KEY already exists [1]. 28

**NODE_KIND_ERROR** is raised if the open node handle FILE_NODE does not identify an EXECUTABLE_IMAGE node [1], if the open node handle PROCESSOR_RESOURCE does not identify a PROCESSOR node [1], if the open node handle MEDIUM_RESOURCE does not identify a MEDIUM node [1], if the node handle PROCESS_NODE_KIND does not have the value NULL_NODE and does not identify a PROCESS node kind which is the executable image's default PROCESS node kind or a specialization of it [2], or if the open node handle RELATION does not identify an RD [1]. 29

**CONFORMANCE_ERROR** is raised if the operation would result in an attribute, node or relationship not conforming to its definition or the constraints in force on that definition [1-5,7-11]. 30

**NOT_IN_VIEW_ERROR** is raised if an attribute name specified in ATTRIBUTES, a key name specified in KEY, or a key name or relation name specified in the pathname INPUT_FILE, OUTPUT_FILE, ERROR_FILE, or ENVIRONMENT_NODE cannot be translated [1]; or if KEY or a relationship key list in INPUT_FILE, OUTPUT_FILE, ERROR_FILE, or ENVIRONMENT_NODE consists of a single relationship key designator and no key of the corresponding relation is visible [5]. 31

**AMBIGUOUS_VIEW_ERROR** is raised if an attribute name specified in ATTRIBUTES, a key name specified in KEY, or a key name or relation name specified in the pathname INPUT_FILE, OUTPUT_FILE, ERROR_FILE, or ENVIRONMENT_NODE has an ambiguous translation [1]; or if KEY or a relationship key list in INPUT_FILE, OUTPUT_FILE, ERROR_FILE, or ENVIRONMENT_NODE consists of a single relationship key designator and more than one key of the corresponding relation is visible [4]. 32

**SYNTAX_ERROR** is raised if the parameter DISCRETIONARY_ACCESS (see Section 5.4.6.1 on page 262) [10] or ATTRIBUTES (see description above) [8] is syntactically illegal. 33

**PREDEFINED_RELATION_ERROR** is raised if RELATION identifies a CAIS-controlled relation [1]. 34

**PREDEFINED_ATTRIBUTE_ERROR** is raised if any attribute name given by the ATTRIBUTES parameter identifies a CAIS-controlled attribute [1]. 35

**ATTRIBUTE_ERROR** is raised if the ATTRIBUTES parameter specifies a value for the same attribute more than once [6]. 36

**EXECUTABLE_IMAGE_ERROR** is raised if the contents of the EXECUTABLE_IMAGE node identified by FILE_NODE cannot be executed [1]. 37

**USE_ERROR** is raised if the parameter PRIORITY [38] is semantically illegal. 38

**STATUS_ERROR** is raised if NODE is an open node handle prior to the call [1], if STATUS is not an established status indicator [3], if FILE_NODE, RELATION, PROCESSOR_RESOURCE, or MEDIUM_RESOURCE is not an open node handle [2], if PROCESS_NODE_KIND does not have the value NULL_NODE and is not an open node handle [2] or 39



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

if ENCLOSING_TRANSACTION does not have the value NULL_TRANSACTION and is not an open transaction handle [6].

**TRANSACTION_ERROR** is raised if the operation is included in more than one transaction [1] or if the operation is included in a transaction and the operation modifies a node which is not included in the same transaction [2]. 40

**LOCK_ERROR** is raised if access with intent APPEND_RELATIONSHIPS to the current PROCESS node cannot be obtained or access with intent READ_RELATIONSHIPS to the parent of the NKI specified by KIND cannot be obtained due to an existing lock [1]. 41

**INTENT_VIOLATION** is raised if the node designated by FILE_NODE was not opened with an intent establishing the right to execute its contents [1]. 42

**RESOURCE_ALLOCATION_ERROR** is raised if the medium resource identified through MEDIUM_RESOURCE or the processor resource identified through PROCESSOR_RESOURCE is not allocatable [1] or if the medium resource identified through MEDIUM_RESOURCE is not allocatable by a process executing on the processor resource identified through PROCESSOR_RESOURCE [2]. 43

**RESOURCE_AVAILABILITY_ERROR** is raised if the medium resource identified through MEDIUM_RESOURCE is unavailable or if the processor resource identified through PROCESSOR_RESOURCE is unavailable [1]. 44

**ACCESS_VIOLATION** is raised if the discretionary access control rights of the current process are insufficient to allow access with CREATE_INSTANCES intent to the NKD identified through PROCESS_NODE_KIND or with READ_RELATIONSHIPS intent to the parent of the NKI specified by PROCESS_NODE_KIND or with APPEND_CONTENTS intent to the MEDIUM node identified by MEDIUM_RESOURCE or with EXECUTE intent to the PROCESSOR node identified by PROCESSOR_RESOURCE [1]. ACCESS_VIOLATION is also raised if the default discretionary access control rights of the new process would be insufficient to obtain access with intent READ_RELATIONSHIPS to any VIEW node for the views to be implicitly adopted [5]. 45

**SECURITY_VIOLATION** may be raised if the attempt to obtain access to the node identified by NODE for the specified intent represents a violation of mandatory access controls. SECURITY_VIOLATION is raised only if the conditions for other exceptions are not present. 46

Additional Interfaces:

```
procedure CREATE_PROCESS_NODE
    (NODE                    : in out PROCESS_NODE_TYPE;
     FILE_NODE               : in     EXECUTABLE_IMAGE_NODE_TYPE;
     INTENT                  : in     INTENT_ARRAY           := (READ, WRITE);
     INPUT_PARAMETERS        : in     PARAMETER_LIST         : = EMPTY_LIST;
     KEY                     : in     RELATIONSHIP_KEY       : = EMPTY_KEY_LIST;
     RELATION                : in     RELATION_NAME          : = DEFAULT_RELATION;
     DISCRETIONARY_ACCESS    : in     DISCRETIONARY_ACCESS_LIST : = ALL_RIGHTS;
     ATTRIBUTES              : in     ATTRIBUTE_LIST         : = EMPTY_LIST;
     INPUT_FILE              : in     PATHNAME               : = "";
     OUTPUT_FILE             : in     PATHNAME               : = "";
     ERROR_FILE              : in     PATHNAME               : = "";
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
        ENVIRONMENT_NODE          : in  PATHNAME                := CURRENT_NODE;
        PROCESS_NODE_KIND         : in  NODE_KIND_NAME          := "";
        ENCLOSING_TRANSACTION     : in  TRANSACTION_TYPE        := NULL_TRANSACTION;
        STATUS                    : in  STATUS_INDICATOR        := NO_INDICATION;
        PRIORITY                  : in  PROCESS_PRIORITY        := DEFAULT_PRIORITY;
        PROCESSOR_RESOURCE        : in  PROCESSOR_NODE_TYPE     := DEFAULT_PROCESSOR;
        MEDIUM_RESOURCE           : in  MEDIUM_NODE_TYPE        := DEFAULT_MEDIUM)
    is
        CALLER                    : PROCESS_NODE_TYPE;
        RELATION_DEFINITION : RD_TYPE;
        PROCESS_DEFINITION : NKI_TYPE;
    begin

        OPEN (CALLER, CURRENT_PROCESS, (1 => NO_ACCESS), STATUS => STATUS);

        DEFINITION_OF_RELATION (RELATION_DEFINITION, CALLER, RELATION, STATUS);

        if PROCESS_NODE_KIND = "" then
            CREATE_PROCESS_NODE
                (NODE, FILE_NODE, INTENT, INPUT_PARAMETERS,
                 KEY, RELATION_DEFINITION, DISCRETIONARY_ACCESS, ATTRIBUTES,
                 INPUT_FILE, OUTPUT_FILE, ERROR_FILE, ENVIRONMEMT_NODE,
                 NULL_NODE, EMCLOSING_TRANSACTION,
                 STATUS, PRIORITY, PROCESSOR_RESOURCE, MEDIUM_RESOURCE);
        else
            OPEN_NODE_KIND (PROCESS_DEFINITION, PROCESS_NODE_KIND, STATUS);

            CREATE_PROCESS_NODE
                (NODE, FILE_NODE, INTENT, INPUT_PARAMETERS,
                 KEY, RELATION_DEFINITION, DISCRETIONARY_ACCESS, ATTRIBUTES,
                 INPUT_FILE, OUTPUT_FILE, ERROR_FILE, ENVIRONMENT_NODE,
                 PROCESS_DEFINITION, ENCLOSING_TRANSACTION,
                 STATUS, PRIORITY, PROCESSOR_RESOURCE, MEDIUM_RESOURCE);
        end if;

        CLOSE (CALLER);
        CLOSE (RELATION_DEFINITION);
        CLOSE (PROCESS_DEFINITION);
    exception
        when others =>
            CLOSE (CALLER);
            CLOSE (RELATION_DEFINITION);
            CLOSE (PROCESS_DEFINITION);
            raise;
    end CREATE_PROCESS_NODE;
```


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Notes:

The value of the parameter ENCLOSING_TRANSACTION is not available to the newly created process; therefore the newly created process cannot perform explicit operations on its enclosing transaction. 47

### 6.3.3 Creating a root PROCESS node

```
procedure CREATE_JOB_NODE
    (NODE                        : in out PROCESS_NODE_TYPE;
     FILE_NODE                   : in     EXECUTABLE_IMAGE_NODE_TYPE;
     INTENT                      : in     INTENT_ARRAY              := (READ, WRITE);
     INPUT_PARAMETERS            : in     PARAMETER_LIST            := EMPTY_LIST;
     KEY                         : in     RELATIONSHIP_KEY          := EMPTY_KEY_LIST;
     DISCRETIONARY_ACCESS        : in     DISCRETIONARY_ACCESS_LIST := ALL_RIGHTS;
     MANDATORY_ACCESS            : in     MANDATORY_ACCESS_LIST     := EMPTY_LIST;
     ATTRIBUTES                  : in     ATTRIBUTE_LIST            := EMPTY_LIST;
     INPUT_FILE                  : in     PATHNAME                  := "";
     OUTPUT_FILE                 : in     PATHNAME                  := "";
     ERROR_FILE                  : in     PATHNAME                  := "";
     ENVIRONMENT_NODE            : in     PATHNAME                  := CURRENT_USER;
     DELETE_WHEN_TERMINATED      : in     BOOLEAN                   := TRUE;
     PROCESS_NODE_KIND           : in     NODE_KIND_NODE_TYPE       := NULL_NODE;
     AS_A_TRANSACTION            : in     BOOLEAN                   := FALSE;
     STATUS                      : in     STATUS_INDICATOR          := NO_INDICATION;
     PRIORITY                    : in     PROCESS_PRIORITY          := DEFAULT_PRIORITY;
     PROCESSOR_RESOURCE          : in     PROCESSOR_NODE_TYPE       := DEFAULT_PROCESSOR;
     MEDIUM_RESOURCE             : in     MEDIUM_NODE_TYPE          := DEFAULT_MEDIUM);
```

Purpose:

This procedure creates a new root PROCESS node whose contents represent the execution of the program contained in the EXECUTABLE_IMAGE node identified by FILE_NODE. The process is not activated; the CURRENT_STATUS attribute of its node is set to CREATED. The PROCESS node containing the calling task must have execution rights for the node identified by FILE_NODE and sufficient rights to append relationships to the node identified by 'CURRENT_-USER. Anew primary relationship of the relation JOB is established from the current USER node to the root PROCESS node of the new job. The new process as a subject has all discretionary access rights to its own PROCESS node (the object). 1

Secondary relationships emanating from the new root PROCESS node are created as described in Table XXVI, page 522. If the parameters INPUT_FILE, OUTPUT_FILE, or ERROR_FILE have the null string as their value, then the secondary relationship corresponding to that parameter is not created. 2

The DISCRETIONARY_ACCESS parameter specifies the initial access control information to be established between the created node and the default GROUP node of the current user (see Section 5.4 on page 255). 3

The MANDATORY_ACCESS parameter specifies the object classification labels with which the 4


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

node is to be created. If its value is the empty list, the node inherits the subject classification of the creating process as its object classification. Otherwise, it must be an unnamed list consisting of an identifier item and an empty or unnamed list of identifier items (see Table IX, page 256). Object and subject classification labels of a process are the same.

The NKD for the PROCESS node created is determined by the node handle parameter PROCESS_NODE_KIND. If PROCESS_NODE_KIND has the value NULL_NODE (the default), then the PROCESS NKD which is the target of the DEFAULT_PROCESS_NODE_KIND relationship emanating from the EXECUTABLE_IMAGE node is used or, in the absence of that relationship, the predefine PROCESS NKD is used.

If PROCESS_NODE_KIND does not have the value NULL_NODE, it must identify the PROCESS node kind to be used. It may specify either an NKD or an NKI (from which an NKD is determined). If PROCESS_NODE_KIND identifies an NKI, the NKI is made the current adopted interpretation for the node handle NODE. In this case, if the DEFAULT_PROCESS_NODE_KIND relationship exists, the NKD identified through the PROCESS_NODE_KIND parameter must be the NKD which is the target of that relationship or a specialization of that NKD.

The parameter AS_A_TRANSACTION specifies whether the new process is to be enclosed in a transaction. If AS_A_TRANSACTION is TRUE, an enclosing transaction is opened for the new process; this transaction is committed if the new process terminates normally and aborted if the process is aborted. All transactions opened by the new job will be descendants of this enclosing transaction and all operations on nodes performed by the job are performed inside this enclosing transaction. If AS_A_TRANSACTION is false, the new process is not enclosed by any transaction.

Parameters:

**NODE** is a node handle, initially closed, to be opened to the newly created PROCESS node.

**FILE_NODE** is an open node handle to the EXECUTABLE_IMAGE node containing the executable image whose execution will be represented by the new process.

**INTENT** is the intent of subsequent operations on the node; the actual parameter takes the form of an array aggregate.

**INPUT_PARAMETERS** is a list containing process parameter information. The list is constructed and parsed using the tools provided in CAIS_LIST_MANAGEMENT. INPUT_PARAMETERS is stored in the PARAMETERS attribute of the new node.

**KEY** is the relationship key list of the primary JOB relationship from the current USER node to the new PROCESS node.

**DISCRETIONARY_ACCESS** is the initial access control information associated with the created node; it is a list of access rights to be put on the ACCESS relationship to the user's default GROUP node.

**MANDATORY_ACCESS** is a list defining the classification label for the created node (see Table IX, page 256).

**ATTRIBUTES** is an empty or named list (see Section 9 on page 745) whose elements are used to establish initial values for attributes of the newly created node; each named item of the list specifies an attribute name and the value to be given to that attribute.


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

**INPUT_FILE** is a pathname to a FILE node that will be the target node of the secondary STANDARD_INPUT relationship.

**OUTPUT_FILE** is a pathname to a FILE node that will be the target node of the secondary STANDARD_OUTPUT relationship.

**ERROR_FILE** is a pathname to a FILE node that will be the target node of the secondary STANDARD_ERROR relationship.

**ENVIRONMENT_NODE** is a pathname to a node the new process will have as its initial current node.

**DELETE_WHEN_TERMINATED** is a Boolean. If DELETE_WHEN_TERMINATED has the value TRUE, an attempt will be made to delete the tree rooted at the PROCESS node when the job terminates. The delete attempt may not succeed; for example, processes outside the job may hold open node handles to nodes within the tree. If DELETE_WHEN_-TERMINATED has the value FALSE, the PROCESS node will be kept when the job terminates. Note that in the latter case, an explicit call to the DELETE_JOB interface (see Section 6.3.10 on page 545) must be made in order to delete the PROCESS node.

**PROCESS_NODE_KIND** is an node handle which determines the PROCESS NKD for the PROCESS node created. If it has the value NULL_NODE, a default PROCESS NKD is used, otherwise it identifies the PROCESS node kind to be used.

**AS_A_TRANSACTION** is a Boolean; if TRUE, an enclosing transaction is created for the new process; if FALSE, the new process is not enclosed in any transaction.

**STATUS** is an established status indicator.

**PRIORITY** is the priority for the new process.

**PROCESSOR_RESOURCE** is an open node handle identifying a PROCESSOR node. It specifies the processor resource on which the process will execute.

**MEDIUM_RESOURCE** is an open node handle identifying a MEDIUM node. It specifies the medium resource on which the newly created node will reside.

Exceptions:

**PATHNAME_SYNTAX_ERROR** is raised if the relationship key list given in KEY [2] is syntactically illegal or the pathname given in INPUT_FILE, OUTPUT_FILE, ERROR_-FILE or ENVIRONMENT_NODE is syntactically illegal [1].

**NAME_ERROR** is raised if the parent of the NKI identified by PROCESS_NODE_KIND is inaccessible [1].

**RELATIONSHIP_ERROR** is raised if any of the nodes identified by INPUT_FILE, OUT-PUT_FILE, ERROR_FILE, or ENVIRONMENT_NODE does not exist [2] or if KEY or any relationship key list in INPUT_FILE, OUTPUT_FILE, ERROR_FILE, or ENVIRON-MENT_NODE specifies a value for the same key more than once [5].

**EXISTING_NODE_ERROR** is raised if a relationship of the relation JOB with relationship key list KEY already exists [1].

**NODE_KIND_ERROR** is raised if the open node handle FILE_NODE does not identify an EXECUTABLE_IMAGE node [1], if the open node handle PROCESSOR_RESOURCE does not identify a PROCESSOR node [1], if the open node handle MEDIUM_RESOURCE does not identify a MEDIUM node [1] or if the node handle PROCESS_NODE_KIND does not have the value NULL_NODE and does not identify the executable image's default PROCESS node kind or a specialization of it [2].



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

**CONFORMANCE_ERROR** is raised if the operation would result in an attribute, node or relationship not conforming to its definition or the constraints in force on that definition [1-5,7-10]. 32

**NOT_IN_VIEW_ERROR** is raised if an attribute name specified in ATTRIBUTES, a key name specified in KEY, or a key name or relation name specified in the pathname INPUT_FILE, OUTPUT_FILE, ERROR_FILE, or ENVIRONMENT_NODE cannot be translated [1] or if KEY or a relationship key list in INPUT_FILE, OUTPUT_FILE, ERROR_FILE or ENVIRONMENT_NODE consists of a single relationship key designator and no key of the corresponding relation is visible [5]. 33

**AMBIGUOUS_VIEW_ERROR** is raised if an attribute name specified in ATTRIBUTES, a key name specified in KEY, or a key name or relation name specified in the pathname INPUT_FILE, OUTPUT_FILE, ERROR_FILE, or ENVIRONMENT_NODE has an ambiguous translation [1] or if KEY or a relationship key list in INPUT_FILE, OUTPUT_FILE, ERROR_FILE, or ENVIRONMENT_NODE consists of a single relationship key designator and more than one key of the corresponding relation is visible [4]. 34

**SYNTAX_ERROR** is raised if the parameter MANDATORY_ACCESS (see Table IX, page 256) [9], DISCRETIONARY_ACCESS (see Section 5.4.6.1 on page 262) [10] or ATTRIBUTES (see description above) [8] is syntactically illegal. 35

**PREDEFINED_ATTRIBUTE_ERROR** is raised if any attribute name given by the ATTRIBUTES parameter identifies a CAIS_controlled attribute [1]. 36

**ATTRIBUTE_ERROR** is raised if the ATTRIBUTES parameter specifies a value for the same attribute more than once [6]. 37

**EXECUTABLE_IMAGE_ERROR** is raised if the contents of the EXECUTABLE_IMAGE node identified by FILE_NODE cannot be executed [1]. 38

**USE_ERROR** is raised if either of the parameters MANDATORY_ACCESS [10] or PRIORITY [38] is semantically illegal. 39

**STATUS_ERROR** is raised if NODE is an open node handle prior to the call [1], if STATUS is not an established status indicator [3], if FILE_NODE, RELATION, PROCESSOR_RESOURCE or MEDIUM_RESOURCE is not an open node handle [2] or if PROCESS_NODE_KIND does not have the value NULL_NODE and is not an open node handle [2]. 40

**TRANSACTION_ERROR** is raised if the operation is included in more than one transaction [1] or if the operation is included in a transaction and the operation modifies a node which is not included in the same transaction [2]. 41

**LOCK_ERROR** is raised if access to the current USER node with intent APPEND_RELATIONSHIPS cannot be obtained or access to the parent of the NKI specified by KIND with intent READ_RELATIONSHIPS cannot be obtained due to an existing lock [1]. 42

**INTENT_VIOLATION** is raised if the node designated by FILE_NODE was not opened with an intent establishing the right to execute its contents [1]. 43

**RESOURCE_ALLOCATION_ERROR** is raised if the medium resource identified through MEDIUM_RESOURCE or the processor resource identified through PROCESSOR_RESOURCE is not allocatable [1] or if the medium resource identified through MEDIUM_RESOURCE is not allocatable by a process executing on the processor resource identified through PROCESSOR_RESOURCE [2]. 44

**RESOURCE_AVAILABILITY_ERROR** is raised if the medium resource identified through MEDIUM_RESOURCE is unavailable or if the processor resource identified through PROCESSOR_RESOURCE is unavailable [1]. 45



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

**ACCESS_VIOLATION** is raised if the discretionary access control rights of the current process are insufficient to allow access with APPEND_RELATIONSHIPS intent to the current USER node or with CREATE_INSTANCES intent to the NKD identified by PROCESS_NODE_KIND or with READ_RELATIONSHIPS intent to the parent of the NKI specified by KIND or with APPEND_CONTENTS intent to the MEDIUM node identified by MEDIUM_RESOURCE or with EXECUTE intent to the PROCESSOR node identified by PROCESSOR_RESOURCE [1]. ACCESS_VIOLATION is also raised if the default discretionary access control rights of the new process would be insufficient to obtain access with intent READ_RELATIONSIHPS to any VIEW node for the views to be implicitly adopted [5]. 46

**SECURITY_VIOLATION** is raised if the attempt to obtain access to the node identified by CURRENT_USER represents a violation of mandatory access controls. SECURITY_VIOLATION may be raised if an attempt to obtain access to the node identified by NODE for the specified intent represents a violation of mandatory access controls. SECURITY_VIOLATION is raised only if the conditions for other exceptions are not present. 47

Additional Interfaces:

```
procedure CREATE_JOB_NODE
    (NODE                        : in out PROCESS_NODE_TYPE;
     FILE_NODE                   : in     EXECUTABLE_IMAGE_NODE_TYPE;
     INTENT                      : in     INTENT_ARRAY              := (READ, WRITE);
     INPUT_PARAMETERS            : in     PARAMETER_LIST            : = EMPTY_LIST;
     KEY                         : in     RELATIONSHIP_KEY          : = EMPTY_KEY_LIST;
     DISCRETIONARY_ACCESS        : in     DISCRETIONARY_ACCESS_LIST : = ALL_RIGHTS;
     MANDATORY_ACCESS            : in     MANDATORY_ACCESS_LIST : = EMPTY_LIST;
     ATTRIBUTES                  : in     ATTRIBUTE_LIST            : = EMPTY_LIST;
     INPUT_FILE                  : in     PATHNAME                  : = "";
     OUTPUT_FILE                 : in     PATHNAME                  : = "";
     ERROR_FILE                  : in     PATHNAME                  : = "";
     ENVIRONMENT_NODE            : in     PATHNAME                  : = CURRENT_NODE;
     DELETE_WHEN_TERMINATED : in     BOOLEAN                   : = TRUE;
     PROCESS_NODE_KIND           : in     NODE_KIND_NAME;
     AS_A_TRANSACTION            : in     BOOLEAN                   : = FALSE;
     STATUS                      : in     STATUS_INDICATOR          := NO_INDICATION;
     PRIORITY                    : in     PROCESS_PRIORITY          := DEFAULT_PRIORITY;
     PROCESSOR_ RESOURCE         : in     PROCESSOR_NODE_TYPE       := DEFAULT_PROCESSOR;
     MEDIUM_ RESOURCE            : in     MEDIUM_NODE_TYPE          := DEFAULT_MEDIUM)
is
    PROCESS_DEFINITION : NKI_TYPE;
begin

    if PROCESS_NODE_KIND = "" then
        CREATE_JOB_NODE
            (NODE, FILE_NODE, INTENT, INPUT_PARAMETERS,
             KEY, DISCRETIONARY_ACCESS, MANDATORY_ACCESS, ATTRIBUTES,
             INPUT_FILE, OUTPUT_FILE, ERROR_FILE, ENVIRONMENT_NODE,
             DELETE_WHEN_TERMINATED, NULL_NODE, AS_A_TRANSACTION,
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
            STATUS, PRIORITY, PROCESSOR_RESOURCE, MEDIUM_RESOURCE);
    else
        OPEN_NODE_KIND (PROCESS_DEFINITION, PROCESS_NODE_KIND, STATUS);

        CREATE_JOB_NODE
            (NODE, FILE_NODE, INTENT, INPUT_PARAMETERS,
             KEY, DISCRETIONARY_ACCESS, MANDATORY_ACCESS, ATTRIBUTES,
             INPUT_FILE, OUTPUT_FILE, ERROR_FILE, ENVIRONMENT_NODE,
             DELETE_WHEN_TERMINATED, PROCESS_DEFINITION, AS_A_TRANSACTION,
             STATUS, PRIORITY, PROCESSOR_RESOURCE, MEDIUM_RESOURCE);
    end if;

    CLOSE (PROCESS_DEFINITION);
exception
    when others =>
        CLOSE (PROCESS_DEFINITION);
        raise;
end CREATE_JOB_NODE;
```

### 6.3.4 Activating a process

```
procedure ACTIVATE_PROCESS
    (NODE    : in  PROCESS_NODE_TYPE;
     STATUS  : in  STATUS_INDICATOR := NO_INDICATION);
```

Purpose:

This procedure specifies that the process identified by NODE is ready for execution. The value of the CURRENT_STATUS attribute for the PROCESS node identified by NODE must be CREATED when this procedure is called (see Table XXV, page 512). The value of the CURRENT_STATUS attribute is changed to READY. Control returns to the calling task after activating the process.

Parameters:

**NODE** is an open node handle identifying the PROCESS node of the process to deactivated,
**STATUS** is an established status indicator.

Exceptions:

**USE_ERROR** is raised if the process identified by NODE does not have the CREATED status [57].

**NODE_KIND_ERROR** is raised if the node identified by NODE is not a PROCESS node [1].

**STATUS_ERROR** is raised if NODE is not an open node handle [2] or if STATUS is not an established status indicator [3].

**INTENT_VIOLATION** is raised if the node handle NODE was not opened with an intent establishing rights to write attributes and contents [1].



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

**SECURITY_VIOLATION** is raised if the attempt to activate the process represents a violation of the mandatory access controls for the CAIS_SECURITY_VIOLATION is raised only if the conditions for other exceptions are not present. 8

### 6.3.5 Awaiting termination of another process

```
procedure AWAIT_PROCESS_TERMINATION
    (NODE         : in     PROCESS_NODE_TYPE;
     TIME_LIMIT   : in     CAIS_DURATION      := LONG_DELAY;
     STATUS       : in     STATUS_INDICATOR   := NO_INDICATION);
```

Purpose:

This procedure suspends the calling task and waits for the process identified by NODE to terminate. The calling task is suspended until the identified process terminates or until the time limit is exceeded. 1

Parameters:

**NODE** is an open node handle identifying the process to be awaited. 2

**TIME_LIMIT** is the limit on the time that the calling task will be suspended awaiting the process. When the limit is exceeded the calling task resumes execution. 3

**STATUS** is an established status indicator. 4

Exceptions:

**NODE_KIND_ERROR** is raised if NODE does not identify a PROCESS node [1]. 5

**STATUS_ERROR** is raised if NODE is not an open node handle [2] or if STATUS is not an established status indicator [3]. 6

**INTENT_VIOLATION** is raised if NODE was not opened with an intent establishing the right to read attributes [1]. 7

**SECURITY_VIOLATION** may be raised if the attempt to wait for termination of the process represents a violation of the mandatory access controls of the CAIS implementation. SECURITY_VIOLATION is raised only if the conditions for other exceptions are not present. 8

Additional Interfaces:

```
procedure AWAIT_PROCESS_TERMINATION
    (NODE              : in      PROCESS_NODE_TYPE;
     RESULTS_RETURNED  : in out  RESULTS_LIST;
     STATUS            :    out  PROCESS_STATUS_KIND;
     TIME_LIMIT        : in      CAIS_DURATION      := LONG_DELAY;
     AWAIT_STATUS      : in      STATUS_INDICATOR   := NO_INDICATION)
is
begin
    AWAIT_PROCESS_TERMINATION (NODE, TIME_LIMIT, AWAIT_STATUS);
    GET_RESULTS (NODE, RESULTS_RETURNED, AWAIT_STATUS);
    STATUS := CURRENT_STATUS (NODE, AWAIT_STATUS);
end AWAIT_PROCESS_TERMINATION;
```



Source: https://assist.dla.mil -- Downloaded: 202610-26T07:54Z
Check the source to verify that this is the current version before use.

Notes:

See also the description of the interface GET_RESULTS on page 547.

### 6.3.6 Activating a process and awaiting its termination

```
procedure ACTIVATE_AND_AWAIT_TERMINATION
    (NODE           : in  PROCESS_NODE_TYPE;
     TIME_LIMIT     : in  CAIS_DURATION     := LONG_DELAY;
     STATUS         : in  STATUS_INDICATOR  := NO_INDICATION);
```

Purpose:

This procedure activates the process identified by NODE and waits for its termination. The body of ACTIVATE_AND_AWAIT_TERMINATION is equivalent to the following code fragment:

```
ACTIVATE_PROCESS (NODE, STATUS);
AWAIT_PROCESS_TERMINATION (NODE, TIME_LIMIT, STATUS);
```

Parameters:

See ACTIVATE_PROCESS and AWAIT_PROCESS_TERMINATION.

Exceptions:

See ACTIVATE_PROCESS and AWAIT_PROCESS_TERMINATION.

Additional Interfaces:

```
procedure ACTIVATE_AND_AWAIT_TERMINATION
    (NODE           : in  PROCESS_NODE_TYPE;
     RESULTS_RETURNED : in out RESULTS_LIST;
     STATUS         :     out PROCESS_STATUS_KIND;
     TIME_LIMIT     : in  CAIS_DURATION     := LONG_DELAY;
     AWAIT_STATUS   : in  STATUS_INDICATOR  := NO_INDICATION)
is
begin
    ACTIVATE_AND_AWAIT_TERMINATION (NODE, TIME_LIMIT, AWAIT_STATUS);
    GET_RESULTS (NODE, RESULTS_RETURNED, AWAIT_STATUS);
    STATUS := CURRENT_STATUS (NODE, AWAIT_STATUS);
end ACTIVATE_AND_AWAIT_TERMINATION;
```


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

### 6.3.7 Spawning a new process

```
procedure SPAW_PROCESS
    (NODE                    : in out PROCESS_NODE_TYPE;
     FILE_NODE               : in     EXECUTABLE_IMAGE_NODE_TYPE;
     INTENT                  : in     INTENT_ARRAY           := (1 => READ);
     INPUT_PARAMETERS        : in     PARAMETER_LIST         := EMPTY_LIST;
     KEY                     : in     RELATIONSHIP_KEY       := EMPTY_KEY_LIST;
     RELATION                : in     RD_TYPE;
     DISCRETIONARY_ACCESS    : in     DISCRETIONARY_ACCESS_LIST := ALL_RIGHTS;
     ATTRIBUTES              : in     ATTRIBUTE_LIST         := EMPTY_LIST;
     INPUT_FILE              : in     PATHNAME               := STANDARD_INPUT;
     OUTPUT_FILE             : in     PATHNAME               := STANDARD_OUTPUT;
     ERROR_FILE              : in     PATHNAME               := STANDARD_ERROR;
     ENVIRONMENT_NODE        : in     PATHNAME               := CURRENT_NODE;
     PROCESS_NODE_KIND       : in     NODE_KIND_NODE_TYPE    := NULL_NODE;
     ENCLOSING_TRANSACTION   : in     TRANSACTION_TYPE       := NULL_TRANSACTION;
     STATUS                  : in     STATUS_INDICATOR       := NO_INDICATION;
     PRIORITY                : in     PROCESS_PRIORITY       := DEFAULT_PRIORITY;
     PROCESSOR_RESOURCE      : in     PROCESSOR_NODE_TYPE    := DEFAULT_PROCESSOR;
     MEDIUM_RESOURCE         : in     MEDIUM_NODE_TYPE       := DEFAULT_MEDIUM);
```

Purpose:

This procedure creates a new PROCESS node whose contents represent the execution of the program contained in the EXECUTABLE_IMAGE node identified by FILE_NODE. The process is then activated, i.e., its status is set to READY. Control returns to the calling task after the new node is created and the process is activated. The calling task and the new process then execute in parallel. 1

The body of procedure SPAWN_PROCESS is equivalent to the following code fragment except that INTENT_ERROR is not raised by the call to ACTIVATE_PROCESS: 2

```
CREATE_PROCESS_NODE
    (NODE, FILE_NODE, INTENT, INPUT_PARAMETERS, KEY, RELATION,
     DISCRETIONARY_ACCESS, ATTRIBUTES, INPUT_FILE, OUTPUT_FILE,
     ERROR_FILE, ENVIRONMENT_NODE, PROCESS_NODE_KIND,
     ENCLOSING_TRANSACTION, STATUS, PRIORITY,
     PROCESSOR_RESOURCE, MEDIUM_RESOURCE);

ACTIVATE_PROCESS (NODE, STATUS);
```

Parameters:

See the parameters of CREATE_PROCESS_NODE in Section 6.3.2 page 524. Note that the default values of some SPAWN_PROCESS parameters are not the same as those of CREATE_-PROCESS_NODE. 3


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Exceptions:

See the exceptions raised by CREATE_PROCESS_NODE in Section 6.3.2 page 524,

Additional Interfaces:

```
procedure SPAWN_PROCESS
    (NODE                    : in out PROCESS_NODE_TYPE;
     FILE_NODE               : in     EXECUTABLE_IMAGE_NODE_TYPE;
     INTENT                  : in     INTENT_ARRAY           := (1 => READ);
     INPUT_PARAMETERS        : in     PARAMETER_LIST         := EMPTY_LIST;
     KEY                     : in     RELATIONSHIP_KEY       := EMPTY_KEY_LIST;
     RELATION                : in     RELATION_NAME          := DEFAULT_RELATION;
     DISCRETIONARY_ACCESS    : in     DISCRETIONARY_ACCESS_LIST := ALL_RIGHTS;
     ATTRIBUTES              : in     ATTRIBUTE_LIST         := EMPTY_LIST;
     INPUT_FILE              : in     PATHNAME               := STANDARD_INPUT;
     OUTPUT_FILE             : in     PATHNAME               := STANDARD_OUTPUT;
     ERROR_FILE              : in     PATHNAME               := STANDARD_ERROR;
     ENVIRONMENT_NODE        : in     PATHNAME               := CURRENT_NODE;
     PROCESS_NODE_KIND       : in     NODE_KIND_NAME         := "";
     ENCLOSING_TRANSACTION   : in     TRANSACTION_TYPE       := NULL_TRANSACTION;
     STATUS                  : in     STATUS_INDICATOR       := NO_INDICATION;
     PRIORITY                : in     PROCESS_PRIORITY       := DEFAULT_PRIORITY;
     PROCESSOR_RESOURCE      : in     PROCESSOR_NODE_TYPE    := DEFAULT_PROCESSOR;
     MEDIUM_RESOURCE         : in     MEDIUM_NODE_TYPE       := DEFAULT_MEDIUM)
is
    CALLER                   : PROCESS_NODE_TYPE;
    RELATION_DEFINITION      : RD_TYPE;
    PROCESS_DEFINITION       : NKI_TYPE;
begin

    OPEN (CALLER, CURRENT_PROCESS, (1 => NO_ACCESS), STATUS => STATUS);

    DEFINITION_OF_RELATION (RELATION_DEFINITION, CALLER, RELATION, STATUS);

    if PROCESS_NODE_KIND = "" then
        SPAWN_PROCESS
            (NODE, FILE_NODE, INTENT, INPUT_PARAMETERS, KEY,
             RELATION_DEFINITION, DISCRETIONARY_ACCESS,
             ATTRIBUTES, INPUT_FILE, OUTPUT_FILE, ERROR_FILE,
             ENVIRONMENT_NODE, NULL_NODE,
             ENCLOSING_TRANSACTION, STATUS, PRIORITY,
             PROCESSOR_RESOURCE, MEDIUM_RESOURCE);
    else
        OPEN_NODE_KIND (PROCESS_DEFINITION, PROCESS_NODE_KIND, STATUS);

        SPAWN_PROCESS
            (NODE, FILE_NODE, INTENT, INPUT_PARAMETERS, KEY,
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
            RELATION_DEFINITION, DISCRETIONARY_ACCESS,
            ATTRIBUTES, INPUT_FILE, OUTPUT_FILE, ERROR_FILE,
            ENVIRONMENT_NODE, PROCESS_DEFINITION,
            ENCLOSING_TRANSACTION, STATUS, PRIORITY,
            PROCESSOR_RESOURCE, MEDIUM_RESOURCE);
    end if;

    CLOSE (CALLER);
    CLOSE (RELATION_DEFINITION);
    CLOSE (PROCESS_DEFINITION);
exception
    when others =>
        CLOSE (CALLER);
        CLOSE (RELATION_DEFINITION);
        CLOSE (PROCESS_DEFINITION);
        raise;
end SPAWN_PROCESS;
```

### 6.3.8 Invoking a new process

```
procedure INVOKE_PROCESS
    (NODE                    : in out PROCESS_NODE_TYPE;
     FILE_NODE               : in     EXECUTABLE_IMAGE_NODE_TYPE;
     INTENT                  : in     INTENT_ARRAY          := (1 => READ);
     RESULTS_RETURNED        : in out RESULTS_LIST;
     STATUS                  :    out PROCESS_STATUS_KIND;
     INPUT_PARAMETERS        : in     PARAMETER_LIST        := EMPTY_LIST;
     KEY                     : in     RELATIONSHIP_KEY      := EMPTY_KEY_LIST;
     RELATION                : in     RD_TYPE;
     DISCRETIONARY_ACCESS    : in     DISCRETIONARY_ACCESS_LIST := ALL_RIGHTS;
     ATTRIBUTES              : in     ATTRIBUTE_LIST        := EMPTY_LIST;
     INPUT_FILE              : in     PATHNAME              := STANDARD_INPUT;
     OUTPUT_FILE             : in     PATHNAME              := STANDARD_OUTPUT;
     ERROR_FILE              : in     PATHNAME              := STANDARD_ERROR;
     ENVIRONMENT_NODE        : in     PATHNAME              := CURRENT_NODE;
     TIME_LIMIT              : in     CAIS_DURATION         := LONG_DELAY;
     PROCESS_NODE_KIND       : in     NODE_KIND_NODE_TYPE   := NULL_NODE;
     ENCLOSING_TRANSACTION   : in     TRANSACTION_TYPE      := NULL_TRANSACTION;
     INVOKE_PROCESS_STATUS   : in     STATUS_INDICATOR      := NO_INDICATION;
     PRIORITY                : in     PROCESS_PRIORITY      := DEFAULT_PRIORITY;
     PROCESSOR_RESOURCE      : in     PROCESSOR_NODE_TYPE   := DEFAULT_PROCESSOR;
     MEDIUM_RESOURCE         : in     MEDIUM_NODE_TYPE      := DEFAULT_MEDIUM);
```

Purpose:

This procedure creates a new PROCESS node whose contents represent the execution of the program contained in the EXECUTABLE_IMAGE node identified by FILE_NODE. The process is then activated, i.e., its status is set to READY. Control returns to the calling task after the



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

newly created process terminates or the amount of time specified by the parameter TIME_LIMIT expires.

The body of procedure INVOKE_PROCESS is equivalent to the following code fragment except that INTENT_ERROR is not raised by the call to ACTIVATE_AND_AWAIT_TERMINATION:

```
CREATE_PROCESS_NODE
    (NODE, FILE_NODE, INTENT, INPUT_PARAMETERS, KEY, RELATION,
     DISCRETIONARY_ACCESS, ATTRIBUTES, INPUT_FILE, OUTPUT_FILE,
     ERROR_FILE, ENVIRONMENT_NODE, PROCESS_NODE_KIND,
     ENCLOSING_TRANSACTION, INVOKE_PROCESS_STATUS,
     PRIORITY, PROCESSOR_RESOURCE, MEDIUM_RESOURCE);

ACTIVATE_AND_AWAIT_TERMINATION
    (NODE, RESULTS_RETURNED, STATUS, TIME_LIMIT, INVOKE_PROCESS_STATUS);
```

Parameters:

All parameters of INVOKE_PROCESS, except those which follow, are the same as the corresponding CREATE_PROCESS_NODE parameters with the same names (see Section 6.3.2 page 524). Note that the default values are not the same as those of CREATE_PROCESS_NODE.

**RESULTS_RETURNED** is a list of results from the new process, which are represented by strings. The individual results may be extracted from the list using the subprograms of CAIS_LIST_MANAGEMENT.

**STATUS** is the current status of the new process at the time when control is returned to the calling task. If the new process completes or terminates within the specified time limit, STATUS will have one of the values COMPLETED_NORMALLY, COMPLETED_ABNORMALLY, ABORTED or TERMINATED. If the new process does not complete within the time limit, STATUS will have the value READY or SUSPENDED.

**TIME_LIMIT** is the limit on the time that the calling task will be suspended awaiting the new process. When the limit is exceeded, the calling task resumes execution.

**INVOKE_PROCESS_STATUS** is an established status indicator.

Exceptions:

Exceptions are the same as those raised by CREATE_PROCESS_NODE (see Section 6.3.2 page 524).

Additional Interfaces:

```
procedure INVOKE_PROCESS
    (NODE                : in out PROCESS_NODE_TYPE;
     FILE_NODE           : in    EXECUTABLE_IMAGE_NODE_TYPE;
     INTENT              : in    INTENT_ARRAY          := (1 => READ);
     RESULTS_RETURNED    : in out RESULTS_LIST;
```


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
        STATUS                      :     out PROCESS_STATUS_KIMD;
        INPUT_PARAMETERS            : in      PARAMETER_LIST          := EMPTY_LIST;
        KEY                         : in      RELATIONSHIP_KEY        := EMPTY_KEY_LIST;
        RELATION                    : in      RELATION_NAME           := DEFAULT_RELATION;
        DISCRETIONARY_ACCESS : in             DISCRETIONARY_ACCESS_LIST := ALL_RIGHTS;
        ATTRIBUTES                  : in      ATTRIBUTE_LIST          := EMPTY_LIST;
        INPUT_FILE                  : in      PATHNAME                := STANDARD_INPUT;
        OUTPUT_FILE                 : in      PATHNAME                := STANDARD_OUTPUT;
        ERROR_FILE                  : in      PATHNAME                := STANDARD_ERROR;
        ENVIRONMENT_NODE            : in      PATHNAME                := CURRENT_NODE;
        TIME_LIMIT                  : in      CAIS_DURATION           := LONG_DELAY;
        PROCESS_NODE_KIND           : in      NODE_KIND_NAME          := "";
        ENCLOSING_TRANSACTION : in            TRANSACTION_TYPE        := NULL_TRANSACTION;
        INVOKE_PROCESS_STATUS : in            STATUS_INDICATOR        := NO_INDICATION;
        PRIORITY                    : in      PROCESS_PRIORITY        := DEFAULT_PRIORITY;
        PROCESSOR_RESOURCE          : in      PROCESSOR_NODE_TYPE     := DEFAULT_PROCESSOR;
        MEDIUM_RESOURCE             : in      MEDIUM_NODE_TYPE        := DEFAULT_MEDIUM)
is
    CALLER                      : PROCESS_NODE_TYPE;
    RELATION_DEFINITION : RD_TYPE;
    PROCESS_DEFINITION : NKI_TYPE;
begin

    OPEN (CALLER, CURRENT_PROCESS, (1 => NO_ACCESS)
          STATUS => INVOKE_PROCESS_STATUS);

    DEFINITION_OF_RELATION   (RELATION_DEFINITION, CALLER, RELATION,
                              INVOKE_PROCESS_STATUS);


    if PROCESS_NODE_KIND = "" then
        INVOKE_PROCESS
            (NODE, FILE_NODE, INTENT, RESULTS_RETURNED,
             STATUS, INPUT_PARAMETERS, KEY, RELATION_DEFINITION,
             DISCRETIONARY_ACCESS, ATTRIBUTES, INPUT_FILE,
             OUTPUT_FILE, ERROR_FILE, ENVIRONMENT_NODE, TIME_LIMIT,
             NULL_NODE, ENCLOSING_TRANSACTION,
             INVOKE_PROCESS_STATUS, PRIORITY, PROCESSOR_RESOURCE,
             MEDIUM_RESOURCE);
    else
        OPEN_NODE_KIND (PROCESS_DEFINITION, PROCESS_NODE_KIND,
                        INVOKE_PROCESS_STATUS);

        INVOKE_PROCESS
            (NODE, FILE_NODE, INTENT, RESULTS_RETURNED,
             STATUS, INPUT_PARAMETERS, KEY, RELATION_DEFINITION,
             DISCRETIONARY_ACCESS, ATTRIBUTES, INPUT_FILE,
             OUTPUT_FILE, ERROR_FILE, ENVIRONMENT_NODE, TIME_LIMIT,
             PROCESS_DEFINITION,   ENCLOSING_TRANSACTION,
```


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
            INVOKE_PROCESS_STATUS, PRIORITY, PROCESSOR_RESOURCE,
            MEDIUM_RESOURCE);
    and if;

    CLOSE (CALLER);
    CLOSE (RELATION_DEFINITION);
    CLOSE (PROCESS_DEFINITION);
exception
    when others =>
        CLOSE (CALLER);
        CLOSE (RELATION_DEFINITION);
        CLOSE (PROCESS_DEFINITION);
        raise;
end INVOKE_PROCESS;
```

Notes:

Both control and data (results and process status) are returned to the calling task upon termination of the invoked process or when the TIME_LIMIT is exceeded.

See also the notes to the interface CREATE_PROCESS_NODE in Section 6.3.2 on page 524.

### 6.3.9 Creating and activating a new job

```
procedure CREATE_JOB
    (FILE_NODE                  : in  EXECUTABLE_IMAGE_NODE_TYPE;
     INPUT_PARAMETERS           : in  PARAMETER_LIST           := EMPTY_LIST;
     KEY                        : in  RELATIONSHIP_KEY         := EMPTY_KEY_LIST;
     DISCRETIONARY_ACCESS       : in  DISCRETIONARY_ACCESS_LIST := ALL_RIGHTS;
     MANDATORY_ACCESS           : in  MANDATORY_ACCESS_LIST := EMPTY_LIST;
     ATTRIBUTES                 : in  ATTRIBUTE_LIST           := EMPTY_LIST;
     INPUT_FILE                 : in  PATHNAME                 := STANDARD_INPUT;
     OUTPUT_FILE                : in  PATHNAME                 := STANDARD_OUTPUT;
     ERROR_FILE                 : in  PATHNAME                 := STANDARD_ERROR;
     ENVIRONMENT_NODE           : in  PATHNAME                 := CURRENT_USER;
     DELETE_WHEN_TERMINATED : in  BOOLEAN                  := TRUE;
     PROCESS_NODE_KIND          : in  NODE_KIND_NODE_TYPE := NULL_NODE;
     AS_A_TRANSACTION           : in  BOOLEAN                  := FALSE;
     STATUS                     : in  STATUS_INDICATOR  := NO_INDICATION;
     PRIORITY                   : in  PROCESS_PRIORITY  := DEFAULT_PRIORITY;
     PROCESSOR_RESOURCE         : in  PROCESSOR_NODE_TYPE := DEFAULT_PROCESSOR;
     MEDIUM_RESOURCE            : in  MEDIUM_NODE_TYPE  := DEFAULT_MEDIUM);
```

Purpose:

This procedure creates anew root PROCESS node whose contents represent the execution of the program contained in the EXECUTABLE_IMAGE node identified by FILE_NODE. The process



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

is then activated, i.e., its status is set to READY. Control returns to the calling task after the new job is created.

The body of CREATE_JOB is equivalent to the following code fragment (where NODE is a local variable of type PROCESS_NODE_TYPE), except that STATUS_ERROR is not raised by the call to ACTIVATE_PROCESS:

```
CREATE_JOB_NODE
    (NODE, FILE_NODE, INPUT_PARAMETERS, KEY,
     DISCRETIONARY_ACCESS, MANDATORY_ACCESS, ATTRIBUTES,
     INPUT_FILE, OUTPUT_FILE, ERROR_FILE, ENVIRONMENT_NODE,
     DELETE_WHEN_TERMINATED, PROCESS_NODE_KIND,
     AS_A_TRANSACTION, STATUS, PRIORITY,
     PROCESSOR_RESOURCE, MEDIUM_RESOURCE);

ACTIVATE_PROCESS (NODE, STATUS);

CLOSE (NODE);
```

Parameters:

See the parameters of the CREATE_JOB_NODE interface (Section 6.3.3 page 530).

Exceptions:

See the exceptions raised by CREATE_JOB_NODE (Section 6.3.3 page 530).

Additional Interfaces:

```
procedure CREATE_JOB
    (FILE_NODE                : in  EXECUTABLE_IMAGE_NODE_TYPE;
     INPUT_PARAMETERS         : in  PARAMETER_LIST          := EMPTY_LIST;
     KEY                      : in  RELATIONSHIP_KEY        := EMPTY_KEY_LIST;
     DISCRETIONARY_ACCESS     : in  DISCRETIONARY_ACCESS_LIST := ALL_RIGHTS;
     MANDATORY_ACCESS         : in  MANDATORY_ACCESS_LIST := EMPTY_LIST;
     ATTRIBUTES               : in  ATTRIBUTE_LIST          := EMPTY_LIST;
     INPUT_FILE               : in  PATHNAME                := STANDARD_INPUT;
     OUTPUT_FILE              : in  PATHNAME                := STANDARD_OUTPUT;
     ERROR_FILE               : in  PATHNAME                := STANDARD_ERROR;
     ENVIRONMENT_NODE         : in  PATHNAME                := CURRENT_NODE;
     DELETE_WHEN_TERMINATED   : in  BOOLEAN                 := TRUE;
     PROCESS_NODE_KIND        : in  NODE_KIND_NAME;
     AS_A_TRANSACTION         : in  BOOLEAN                 := FALSE;
     STATUS                   : in  STATUS_INDICATOR        := NO_INDICATION;
     PRIORITY                 : in  PROCESS_PRIORITY        := DEFAULT_PRIORITY;
     PROCESSOR_RESOURCE       : in  PROCESSOR_NODE_TYPE     := DEFAULT_PROCESSOR;
     MEDIUM_RESOURCE          : in  MEDIUM_NODE_TYPE        := DEFAULT_MEDIUM)
is
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
    PROCESS_DEFINITION : NKI_TYPE;
begin

    if PROCESS_NODE_KIND = "" then
        CREATE_JOB
            (FILE_NODE, INPUT_PARAMETERS, KEY,
             DISCRETIONARY_ACCESS, MANDATORY_ACCESS, ATTRIBUTES,
             INPUT_FILE, OUTPUT_FILE, ERROR_FILE, ENVIRONMENT_NODE,
             DELETE_WHEN_TERMINATED, NULL_NODE, AS_A_TRANSACTION,
             STATUS, PRIORITY, PROCESSOR_RESOURCE, MEDIUM_RESOURCE);
    else
        OPEN_NODE_KIND (PROCESS_DEFINITION, PROCESS_NODE_KIND, STATUS);

        CREATE_JOB
            (FILE_NODE, IMPUT_PARAMETERS, KEY,
             DISCRETIONARY_ACCESS, MANDATORY_ACCESS, ATTRIBUTES,
             INPUT_FILE, OUTPUT_FILE, ERROR_FILE, ENVIRONMENT_NODE,
             DELETE_WHEN_TERMINATED, PROCESS_DEFINITION, AS_A_TRANSACTION,
             STATUS, PRIORITY, PROCESSOR_RESOURCE, MEDIUM_RESOURCE);
    end if;

    CLOSE (PROCESS_DEFINITION);
exception
    when others =>
        CLOSE (PROCESS_DEFINITION);
        raise;
end CREATE_JOB;
```

### 6.3.10 Deleting a job

```
procedure DELETE_JOB
    (NODE   : in out PROCESS_NODE_TYPE;
     STATUS : in     STATUS_INDICATOR := NO_INDICATION);
```

Purpose:

This procedure effectively performs the DELETE_NODE operation for a specified root PROCESS node and recursively applies DELETE_NODE to all nodes reachable by a unique primary path from the designated node. The nodes whose primary relationships are to be deleted are opened with intent EXCLUSIVE_WRITE, thus locking them for other operations. The order in which the deletions of primary relationships is performed is not specified. If the DELETE_JOB operation raises an exception, none of the primary relationships is deleted. 1

Parameters:

**NODE** is an open node handle to the root PROCESS node of the job to be deleted. 2

**STATUS** is an established status indicator. 3



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Exceptions:

**NAME_ERROR** is raised if the parent of the node identified by NODE or any of the target nodes of primary relationships to be deleted are inaccessible [1].

**PREDEFINED_RELATION_ERROR** is raised if the primary relationship to the node identified by NODE is not a JOB relationship [4].

**STATUS_ERROR** is raised if the node handle NODE is not open prior to the call [2] or if STATUS is not an established status indicator [3].

**CONFORMANCE_ERROR** is raised if the operation would result in a node not conforming to its definition or the constraints in force on that definition [3].

**TRANSACTION_ERROR** is raised if the operation is included in more than one transaction [1] or if the operation is included in a transaction and the operation modifies a node which is not included in the same transaction [2].

**LOCK_ERROR** is raised if a node handle to the parent of the node specified by NODE cannot be opened with intent WRITE_RELATIONSHIPS or if a node handle identifying any node whose unique primary path traverses the node identified by NODE cannot be opened with intent EXCLUSIVE_WRITE [1].

**INTENT_VIOLATION** is raised if the node handle NODE was not opened with an intent including EXCLUSIVE_WRITE and READ_RELATIONSHIPS [1].

**ACCESS_VIOLATION** is raised if the current process does not have sufficient discretionary access rights to obtain access to the parent of the node specified by NODE with intent WRITE_RELATIONSHIPS or to obtain access to any target node of a primary relationship to be deleted with intent EXCLUSIVE_WRITE and the conditions for NAME_ERROR are not present [1].

**SECURITY_VIOLATION** is raised if the operation represents a violation of mandatory access controls. SECURITY_VIOLATION is raised only if the conditions for other exceptions are not present.

Notes:

This operation can be used to delete more than one primary relationship in a single operation.

The DELETE_TREE operation (see Section 5.2.33 on page 139) cannot be applied to jobs, because the JOB relation is CAIS-controlled.

### 6.3.11 Appending results

```
procedure APPEND_RESULTS
    (RESULTS : in     RESULTS_STRING;
     STATUS  : in     STATUS_INDICATOR := NO_INDICATION);
```

Purpose:

This procedure inserts the value of its RESULTS parameter as the last item in the list which is the value of the RESULTS attribute of the current PROCESS node.



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Parameters:

**RESULTS** is a string to be appended to the RESULTS attribute value of the current PROCESS node. 2

**STATUS** is an established status indicator. 3

Exceptions:

**LOCK_ERROR** is raised if access with intent WRITE_ATTRIBUTES to the current PROCESS node cannot be obtained due to an existing lock on the node [1]. 4

**STATUS_ERROR** is raised if STATUS is not an established status indicator [3]. 5

**CONFORMANCE_ERROR** is raised if the operation would result in an attribute not conforming to its definition or the constraints in force on that definition [7-10]. 6

### 6.3.12 Overwriting results

```
procedure WRITE_RESULTS
    (RESULTS : in     RESULTS_STRING;
     STATUS  : in     STATUS_INDICATOR := NO_INDICATION);
```

Purpose:

This procedure replaces the value of the RESULTS attribute of the current PROCESS node with a list containing a single item which is the value of the parameter RESULTS. 1

Parameters:

**RESULTS** is a string to be stored in the RESULTS attribute value list of the current PROCESS node. 2

**STATUS** is an established status indicator. 3

Exceptions:

**LOCK_ERROR** is raised if access with intent WRITE_ATTRIBUTES to the current PROCESS node cannot be obtained due to an existing lock on the node [1]. 4

**STATUS_ERROR** is raised if STATUS is not an established status indicator [3]. 5

**CONFORMANCE_ERROR** is raised if the operation would result in an attribute not conforming to its definition or the constraints in force on that definition [7-10]. 6

### 6.3.13 Getting results from a process

```
procedure GET_ RESULTS
    (NODE    : in     PROCESS_NODE_TYPE;
     RESULTS : in out RESULTS_LIST;
     STATUS  : in     STATUS_INDICATOR := NO_INDICATION);
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Purpose:

This procedure returns the value of the attribute RESULTS of the PROCESS node identified by NODE. The process need not have terminated. The empty list is returned in RESULTS if WRITE_RESULTS or APPEND_RESULTS has not been called by the process contained in the node identified by NODE.

Parameters:

**NODE** is an open node handle to a PROCESS node.

**RESULTS** is an unnamed list of strings giving the value of the RESULTS attribute of the PROCESS node identified by NODE. The individual strings may be extracted from the list using the subprograms of CAIS_LIST_MANAGEMENT (see Section 9 on page 745).

**STATUS** is an established status indicator.

Exceptions:

**NODE_KIND_ERROR** is raised if the node identified by NODE is not a PROCESS node [1].

**STATUS_ERROR** is raised if NODE is not an open node handle [2] or if STATUS is not an established status indicator [3].

**INTENT_VIOLATION** is raised if the NODE was not opened with an intent establishing the right to read attributes [1].

Additional Interfaces:

```
procedure GET_RESULTS
    (NODE                 : in     PROCESS_NODE_TYPE;
     RESULTS              : in out RESULTS_LIST;
     STATUS               :    out PROCESS_STATUS_KIND;
     GET_RESULTS_STATUS : in     STATUS_INDICATOR := NO_INDICATION)
is
begin
    GET_RESULTS (NODE, RESULTS, GET_RESULTS_STATUS);
    STATUS := CURRENT_STATUS (NODE, GET_RESULTS_STATUS);
end GET_RESULTS;
```

Notes:

The STATUS parameter in the Additional Interfaces performs in a manner similar to the function CURRENT_STATUS (Section 6.3.14 on page 548).

### 6.3.14 Determining the status of a process

```
function CURRENT_STATUS
    (NODE   : in PROCESS_NODE_TYPE;
     STATUS : in STATUS_INDICATOR := NO_INDICATION)
    return PROCESS_STATUS_KIND;
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Purpose:

This function returns the value of the attribute CURRENT_STATUS associated with the PROCESS node identified by NODE.

Parameters:

**NODE** is an open node handle identifying the node of the process whose status is to be queried.

**STATUS** is an established status indicator.

Exceptions:

**NODE_KIND_ERROR** is raised if the node identified by NODE is not a PROCESS node [1].

**STATUS_ERROR** is raised if NODE is not an open node handle [2] or if STATUS is not an established status indicator [3].

**INTENT_VIOLATION** is raised if the node handle NODE was not opened with an intent establishing the right to read attributes [1].

### 6.3.15 Getting the parameter list

```
procedure GET_PARAMETERS
    (PARAMETERS : in out PARAMETER_LIST;
     STATUS     : in     STATUS_INDICATOR := NO_INDICATION);
```

Purpose:

This procedure returns the value of the attribute PARAMETERS of the current PROCESS node.

Parameters:

**PARAMETERS** is a list containing parameter information. The list is constructed and can be manipulated using the subprograms provided in CAIS_LIST_MANAGEMENT (see Section 9 on page 745).

**STATUS** is an established status indicator.

Exceptions:

**LOCK_ERROR** is raised if access with intent READ_ATTRIBUTES to the current PROCESS node cannot be obtained due to an existing lock on the node [1].

**STATUS_ERROR** is raised if STATUS is not an established status indicator [3].



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Notes:

The value of the attribute PARAMETERS is set during PROCESS node creation; see the interfaces SPAWN_PROCESS (page 538), INVOKE_PROCESS (page 540), CREATE_JOB (page 543), CREATE_PROCESS_NODE (page 524) and CREATE_JOB_NODE (page 530). 6

### 6.3.16 Aborting a process

```
procedure ABORT_PROCESS
    (NODE    : in    PROCESS_NODE_TYPE;
     RESULTS : in    RESULTS_STRING;
     STATUS  : in    STATUS_INDICATOR := NO_INDICATION);
```

Purpose:

This procedure aborts the process represented by NODE (see Table XXV, page 512). It also aborts any process in its process tree for which the current process has sufficient discretionary access rights to (1) traverse the primary relationships to the node representing the process and (2) obtain access to the node representing the process with intent WRITE_ATTRIBUTES and WRITE_CONTENTS. If the current process does not have such access rights to the node of any process in the process tree, the respective node's process is not aborted, and after all processes that can be aborted are aborted, ABORT_PROCESS returns by raising the exception ACCESS_VIOLATION. The order in which processes are aborted is not specified. After return of ABORT_PROCESS the value of the CURRENT_STATUS attribute of NODE will be COMPLETED_ABNORMALLY, ABORTED or TERMINATED. It will be COMPLETED_ABNORMALLY if some processes in the process tree could not be aborted and have not terminated. It will be ABORTED if all processes in the process tree have terminated. It will be TERMINATED only if that was its value before ABORT_PROCESS took effect. If NODE_KIND_ERROR, STATUS_ERROR or INTENT_VIOLATION is raised, none of the processes in the process tree is aborted. 1

Parameters:

**NODE** is an open node handle for the node of the process to be aborted. 2

**RESULTS** is a string to be appended to the RESULTS attribute of the node represented by NODE. 3

**STATUS** is an established status indicator. 4

Exceptions:

**NODE_KIND_ERROR** is raised if the node identified by NODE is not a PROCESS node [1]. 5

**STATUS_ERROR** is raised if NODE is not an open node handle [2] or if STATUS is not an established status indicator [3]. 6

**INTENT_VIOLATION** is raised if the node was not opened with an intent establishing rights to read relationships and to write attributes and contents [1]. 7

**ACCESS_VIOLATION** is raised if the current process does not have sufficient discretionary access rights to obtain access to any node of a process to be aborted with intent including READ_RELATIONSHIPS, WRITE_ATTRIBUTES and WRITE_CONTENTS [1]. 8


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Notes:

ABORT_PROCESS can be used by a task to abort the process that contains it.

ABORT_PROCESS can be used to abort a process whose CURRENT_STATUS attribute has the value CREATED, i.e. a process which has not been activated.

It is intentional that LOCK_ERROR will not be raised by this procedure.

An enclosing transaction for a root process is closed by either normal termination or abortion of the process. If the root process is aborted, the eligible effects of all operations performed are discarded.

### 6.3.17 Suspending a process

```
procedure SUSPEND_PROCESS
    (NODE   : in  PROCESS_NODE_TYPE;
     STATUS : in  STATUS_INDICATOR := NO_INDICATION);
```

Purpose:

This procedure suspends the process represented by NODE (see Table XXV, page 512). It also suspends any process in its process tree for which the current process has sufficient discretionary access rights to (1) traverse the primary relationships to the node representing the process and (2) obtain access to the node representing the process with intent WRITE_ATTRIBUTES and WRITE_CONTENTS. If the current process does not have such access rights to the node of any process in the process tree, the respective node's process is not suspended, and after all processes that can be suspended are suspended, SUSPEND_PROCESS returns by raising the exception ACCESS_VIOLATION. The order in which processes are suspended is not specified. If the value of the CURRENT_STATUS attribute of a process to be suspended is READY at the time of suspension, it is changed to SUSPENDED; otherwise, the process is not affected. If NODE_-KIND_ERROR, STATUS_ERROR or INTENT_VIOLATION is raised, none of the processes in the process tree is suspended.

Parameters:

**NODE** is an open node handle identifying the node of the process to be suspended.

**STATUS** is an established status indicator.

Exceptions:

**NODE_KIND_ERROR** is raised if the node identified by NODE is not a PROCESS node [1].

**STATUS_ERROR** is raised if NODE is not an open node handle [2] or if STATUS is not an established status indicator [3].

**INTENT_VIOLATION** is raised if the node handle NODE was not opened with an intent establishing rights to read relationships and to write attributes and contents [1].

**ACCESS_VIOLATION** is raised if the current process does not have sufficient discretionary access rights to obtain access to any node of a process to be suspended with intents including READ_RELATIONSHIPS, WRITE_ATTRIBUTES and WRITE_CONTENTS [1].


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Notes:

SUSPEND_PROCESS can be used by a task to suspend the process that contains it. 8

Processes in the process tree of the process represented by NODE may be resumed by other CAIS calls prior to the completion of the call on SUSPEND_PROCESS. 9

### 6.3.18 Resuming a process

```
procedure RESUME_PROCESS
    (NODE   : in    PROCESS_NODE_TYPE;
     STATUS : in    STATUS_INDICATOR := NO_INDICATION);
```

Purpose:

This procedure resumes the process represented by NODE (see Table XXV, page 512). It also resumes any process in its process tree for which the current process has sufficient discretionary access rights to (1) traverse the primary relationships to the node representing the process and (2) obtain access to the node representing the process with intent WRITE_ATTRIBUTES and WRITE_CONTENTS. If the current process does not have such access rights to the node of any process in the process tree, the respective node's process is not resumed, and after all processes that can be resumed are resumed, RESUME_PROCESS returns by raising the exception ACCESS_VIOLATION. The order in which processes are resumed is not specified. If the value of the CURRENT_STATUS attribute of a process to be resumed is SUSPENDED at the time of resumption, the value is changed to READY; otherwise, the process is not affected. If NODE_KIND_ERROR, STATUS_ERROR or INTENT_VIOLATION is raised, none of the processes in the process tree is resumed. 1

Parameters:

**NODE** is an open node handle identifying the node of the process to be resumed. 2

**STATUS** is an established status indicator. 3

Exceptions:

**NODE_KIND_ERROR** is raised if the node identified by NODE is not a PROCESS node [1]. 4

**STATUS_ERROR** is raised if NODE is not an open node handle [2] or if STATUS is not an established status indicator [3]. 5

**INTENT_VIOLATION** is raised if the node handle NODE was not opened with an intent establishing rights to read relationships and to write attributes and contents [1]. 6

**ACCESS_VIOLATION** is raised if the current process does not have sufficient discretionary access rights to obtain access to any node of a process to be resumed with intent including READ_RELATIONSHIPS, WRITE_ATTRIBUTES and WRITE_CONTENTS [1]. 7


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Notes:

Processes in the process tree of the process represented by NODE maybe suspended by other CAIS calls prior to the completion of the call on RESUME_PROCESS. 8

### 6.3.19 Determining the lowest specifiable priority

```
function LOWEST_USER_PRIORITY
    (PROCESSOR : in     PROCESSOR_NODE_TYPE  := DEFAULT_PROCESSOR;
     STATUS    : in     STATUS_INDICATOR     := NO_INDICATION)
    return PROCESS_PRIORITY;
```

Purpose:

This function returns the lowest priority at which the current process can initiate a new process on the processor identified by PROCESSOR. 1

Parameters:

**PROCESSOR** is an open node handle identifying a PROCESSOR node. 2

**STATUS** is an established status indicator. 3

Exceptions:

**NODE_KIND_ERROR** is raised if the open node handle PROCESSOR does not identify a PROCESSOR node [1]. 4

**USE_ERROR** is raised if the current process cannot initiate processes on the processor specified [39]. 5

**STATUS_ERROR** is raised if PROCESSOR is not an open node handle prior to the call [2] or if STATUS is not an established status indicator [3]. 6

### 6.3.20 Determining the highest specifiable priority

```
function HIGHEST_USER_PRIORITY
    (PROCESSOR : in     PROCESSOR_NODE_TYPE : = DEFAULT_PROCESSOR;
     STATUS    : in     STATUS_INDICATOR : = NO_INDICATION)
    return PROCESS_PRIORITY;
```

Purpose:

This function returns the highest priority at which the current process can initiate a new process on the processor identified by PROCESSOR. 1


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Parameters:

**PROCESSOR** is an open node handle identifying a PROCESSOR node.

**STATUS** is an established status indicator.

Exceptions:

**NODE_KIND_ERROR** is raised if the open node handle PROCESSOR does not identify a PROCESSOR node [1].

**USE_ERROR** is raised if the current process cannot initiate processes on the processor specified [39].

**STATUS_ERROR** is raised if PROCESSOR is not an open node handle prior to the call [2] or if STATUS is not an established status indicator [3].

### 6.3.21 Determining if the current priority is modifiable

```
function PRIORITIES_ARE_MODIFIABLE
    (STATUS : in STATUS_INDICATOR := NO_INDICATION)
    return BOOLEAN;
```

Purpose:

This function returns TRUE if the current process can modify its priority; otherwise, FALSE is returned.

Parameters:

**STATUS** is an established status indicator.

Exceptions:

**STATUS_ERROR** is raised if STATUS is not an established status indicator [3].

### 6.3.22 Modifying the current priority

```
procedure SET_PRIORITY
    (PRIORITY : in PROCESS_PRIORITY;
     STATUS   : in STATUS_INDICATOR := NO_INDICATION);
```

Purpose:

This procedure sets the priority of the current process to PRIORITY.



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Parameters:

**PRIORITY** is a process priority.

**STATUS** is an established status indicator.

Exceptions:

**USE_ERROR** is raised if the current process cannot set its priority to PRIORITY [40].

**STATUS_ERROR** is raised if STATUS is not an established status indicator [3].

Notes:

The PRIORITIES_ARE_MODIFIABLE operation indicates whether the SET_PRIORITY operation can be successfully performed.

The LOWEST_USER_PRIORITY and HIGHEST_USER_PRIORITY operations indicate the legal range of values which may be passed through the PRIORITY parameter.

### 6.3.23 Determining the number of open node handles

```
function OPEN_HANDLE_COUNT
    (NODE   : in  PROCESS_NODE_TYPE;
     STATUS : in  STATUS_INDICATOR := NO_INDICATION)
    return CAIS_NATURAL;
```

Purpose:

This function returns a natural number representing the value of the attribute OPEN_NODE_HANDLE_COUNT of the PROCESS node identified by NODE. This value is the number of currently open node handles of the current process.

Neither the handles returned by CAIS functions nor any other handles declared and opened implicitly by a CAIS implementation are included in the value of the OPEN_NODE_HANDLE_COUNT attribute. These handles are also not counted in the pragmatic limits imposed by the constants CAIS_NODE_HANDLES_PER_PROCESS and NODE_HANDLES_PER_PROCESS in package CAIS_PRAGMATICS.

Parameters:

**NODE** is an open node handle identifying the PROCESS node whose attribute is being queried.

**STATUS** is an established status indicator.

Exceptions:

**NODE_KIND_ERROR** is raised if the node identified by NODE is not a PROCESS node [1].

**STATUS_ERROR** is raised if NODE is not an open node handle [2] or if STATUS is not an established status indicator [3].


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

**INTENT_VIOLATION** is raised if the node handle NODE was not opened with an intent establishing the right to read attributes [1]. 7

### 6.3.24 Determining the number of input and output units used

```
function IO_UNIT_COUNT
    (NODE   : in  PROCESS_NODE_TYPE;
     STATUS : in  STATUS_INDICATOR := NO_INDICATION)
    return CAIS_NATURAL;
```

Purpose:

This function returns a natural number representing the value of the attribute IO_UNIT_COUNT of the PROCESS node identified by NODE. This value is the number of input and output operations that have been performed by the process identified by NODE. 1

Parameters:

**NODE** is an open node handle identifying the PROCESS node whose attribute is being queried. 2

**STATUS** is an established status indicator. 3

Exceptions:

**NODE_KIND_ERROR** is raised if the node identified by NODE is not a PROCESS node [1]. 4

**STATUS_ERROR** is raised if NODE is not an open node handle [2] or if STATUS is not an established status indicator [3]. 5

**INTENT_VIOLATION** is raised if the node handle was not opened with an intent establishing the right to read attributes [1]. 6

### 6.3.25 Determining the time of activation

```
function TIME_STARTED
    (NODE   : in  PROCESS_NODE_TYPE;
     STATUS : in  STATUS_INDICATOR := NO_INDICATION)
    return CAIS_CALENDAR_TIME;
```

Purpose:

This function returns a value of type CAIS_CALENDAR_TIME representing the value of the attribute TIME_STARTED of the PROCESS node identified by NODE. This value is the time of process activation, i.e. the time at which the value of the CURRENT_STATUS attribute of the PROCESS node was fist set to READY. 1


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Parameters:

**NODE** is an open node handle identifying the PROCESS node whose attribute is being queried.
**STATUS** is an established status indicator.

Exceptions:

**USE_ERROR** is raised if the process identified by NODE has has not been activated [56].
**NODE_KIND_ERROR** is raised if the node identified by NODE is not a PROCESS node [1].
**STATUS_ERROR** is raised if NODE is not an open node handle [2] or if STATUS is not an established status indicator [3].
**INTENT_VIOLATION** is raised if the node handle NODE was not opened with an intent establishing the right to read attributes [1].

Notes:

The TIME_STARTED attribute of a PROCESS node has no value until the process is activated.

### 6.3.26 Determining the time of termination

```
function TIME_FINISHED
    (NODE   : in  PROCESS_NODE_TYPE;
     STATUS : in  STATUS_INDICATOR := NO_INDICATION)
    return CAIS_CALENDAR_TIME;
```

Purpose:

This function returns a value of type CAIS_CALENDAR.TIME representing the value of the attribute TIME_FINISHED of the PROCESS node identified by NODE. This value is the time at which the value of the CURRENT_STATUS attribute of the PROCESS node was set to TERMINATED or ABORTED.

Parameters:

**NODE** is an open node handle identifying the PROCESS node whose attribute is being queried.
**STATUS** is an established status indicator.

Exceptions:

**NODE_KIND_ERROR** is raised if the node identified by NODE is not a PROCESS node [1].
**STATUS_ERROR** is raised if NODE is not an open node handle [2] or if STATUS is not an established status indicator [3].
**INTENT_VIOLATION** is raised if the node handle NODE was not opened with an intent establishing the right to read attributes [1].
**USE_ERROR** is raised if the process is not yet terminated [50].


Source: https://assist.dla.mil - Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Notes:

The TIME_FINISHED attribute of a PROCESS node has no value until the process is terminated. 8

### 6.3.27 Determining the time a process has been active

```
function MACHINE_TIME
    (NODE   : in    PROCESS_NODE_TYPE;
     STATUS : in    STATUS_INDICATOR := NO_INDICATION)
    return CAIS_DURATION;
```

Purpose:

This function returns a value of type CAIS_DURATION representing the value of the attribute MACHINE_TIME ofthe PROCESS node identified by NODE. This valueis the length of time the process has been executing on the logical processor. 1

Parameters:

**NODE** is an open node hande identifying the PROCESS node whose attribute is being queried, 2

**STATUS** is an established status indicator. 3

Exceptions:

**NODE_KIND_ERROR** is raised if the node identified by NODE is not a PROCESS node [1]. 4

**STATUS_ERROR** is raised if NODE is not an open node handle [2] or if STATUS is not an established status indicator [3]. 5

**INTENT_VIOLATION** is raised if the node handle NODE was not opened with an intent establishing the right to read attributes [1]. 6

### 6.3.28 Determining the size of a process

```
function PROCESS_SIZE
    (NODE   : in    PROCESS_NODE_TYPE;
     STATUS : in    STATUS_INDICATOR : = NO_INDICATION)
    return CAIS_NATURAL;
```

Purpose:

This function returns a value of type CAIS_NATURAL representing the value of the attribute PROCESS_SIZE of the PROCESS node identified by NODE. The attribute PROCESS_SIZE designates the amount of memory currently in use by the process. If the process is completed, the value returned is the amount of memory in use by the process at the time that the process was completed. The result of this function indicates the size of the process in memory storage units. The size of a memory storage unit is defined by the constant CAIS_PRAGMATICS.MEMORY_-STORAGE_UNIT_SIZE. 1


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Parameters:

**NODE** is an open node handle identifying the PROCESS node whose attribute is being queried. 2

**STATUS** is an established status indicator. 3

Exceptions:

**NODE_KIND_ERROR** is raised if the node identified by NODE is not a PROCESS node [1]. 4

**STATUS_ERROR** is raised if NODE is not an open node handle [2] or if STATUS is not an established status indicator [3]. 5

**INTENT_VIOLATION** is raised if the node handle NODE was not opened with an intent establishing the right to read attributes [1]. 6


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

# 6.4 Package CAIS_TRANSACTION_MANAGEMENT

This package specifies interfaces for opening, closing, committing, aborting, examining and including nodes in transactions. This package also defines the type for transaction handles. The exceptions raised by all subprograms in this package are defined in the packages CAIS_DEFINITIONS, CAIS_-PROCESS_DEFINITIONS, and CAIS_PRAGMATICS.

```
type TRANSACTION_TYPE is limited private;

NULL_TRANSACTION : constant TRANSACTION_TYPE;
```

An object of the limited private type TRANSACTION_TYPE is called a transaction handle. An open transaction handle is used by a process to identify a transaction opened by that process. The implementation_defined deferred constant NULL_TRANSACTION is a closed transaction handle which has a value distinct from the value of any other transaction handle. The NULL_TRANSACTION value is used for a transaction handle parameter to indicate that no transaction is identified.

## 6.4.1 Opening a transaction

```
procedure OPEN_TRANSACTION
    (TRANSACTION : in out TRANSACTION_TYPE;
     PARENT      : in     TRANSACTION_TYPE   := NULL_TRANSACTION;
     STATUS      : in     STATUS_INDICATOR   := NO_INDICATION);
```

Purpose:

This procedure returns an open transaction handle in TRANSACTION identifying a new transaction. PARENT is a transaction handle that determines the parent transaction. If PARENT has the value NULL_TRANSACTION, then the new transaction is a subtransaction of the enclosing transaction for the current process, if any. Otherwise, the new transaction is a subtransaction of the open transaction identified by PARENT.

Parameters:

**TRANSACTION** is a transaction handle, initially closed, to be opened.

**PARENT** is a transaction handle which determines the parent transaction for the new subtransaction.

**STATUS** is an established status indicator.

Exceptions:

**STATUS_ERROR** is raised if the transaction handle TRANSACTION is an open transaction handle [5], if the transaction handle PARENT is not open and does not have the value NULL_TRANSACTION [6], or if STATUS is not an established status indicator [3].


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

### 6.4.2 Including a node in a transaction

```
procedure INCLUDE
    (TRANSACTION : in     TRANSACTION_TYPE;
     NODE        : in     NODE_TYPE;
     TIME_LIMIT  : in     CAIS_DURATION     := LONG_DELAY;
     STATUS      : in     STATUS_INDICATOR  := NO_INDICATION);
```

Purpose:

This procedure includes the node identified by the open node handle NODE in the transaction identified by the open transaction handle TRANSACTION. This causes operations subsequently performed on the node by the current process to be included in the transaction, excepting operations similarly included in a descendant of the transaction, until the transactions closed. 1

This procedure also establishes either an EXCLUSIVE_READ or an EXCLUSIVE_WRITE access synchronization lock on the node which locks the node against operations not included within the transaction for the duration of the transaction. An EXCLUSIVE_WRITE access synchronization lock is established if the node handle NODE was opened with any write or control intent; otherwise an EXCLUSIVE_READ access synchronization lock is established. The intent regarding use of the node handle NODE is not changed. 2

If the specified node is already included in another open transaction and the transaction identified by TRANSACTION is not a descendant of that open transaction, the INCLUDE operation is delayed. INCLUDE has no effect if the specified node is already included in the specified transaction with the same intent or with write intent. 3

The TIME_LIMIT parameter allows the specification of a time limit for the delay imposed on INCLUDE by other transactions or by the existence of locks on the node to be included. A delayed INCLUDE call completes after any such transactions are closed and the node to be included is unlocked, or the specified time limit has elapsed. In the latter case, the exception LOCK_ERROR is raised. 4

Parameters:

**TRANSACTION** is an open transaction handle. 5

**NODE** is an open node handle. 6

**TIME_LIMIT** is a time limit for the delay on waiting for the unlocking of a node in accordance with the access synchronization which is to be established. 7

**STATUS** is an established status indicator. 8

Exceptions:

**USE_ERROR** is raised if the open node handle NODE has no access intent [59]. 9

**STATUS_ERROR** is raised if the node handle NODE is not an open node handle [2], if the transaction handle TRANSACTION is not an open transaction handle [6] or if STATUS is not an established status indicator [3]. 10


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

**LOCK_ERROR** is raised if the INCLUDE operation is delayed beyond the specified time limit due to the existence of locks in conflict with the access synchronization lock which is to be established [1]. LOCK_ERROR may be raised prior to expiration of the time limit if the CAIS implementation can determine that a deadlock situation has occurred. 11

**SECURITY_VIOLATION** may be raised if the attempt to include operations on the specified node represents a violation of the mandatory access controls of the CAIS implementation. SECURITY_VIOLATION is raised only if the conditions for other exceptions are not present. 12

Notes:

A call to INCLUDE may be blocked if the node is already included in another open transaction, regardless of whether that transaction is part of the current process. This occurs when the specified transaction is not a descendant of the other, open transaction. 13

### 6.4.3 Including a tree in a transaction

```
procedure INCLUDE_TREE
    (TRANSACTION : in     TRANSACTION_TYPE;
     NODE        : in     NODE_TYPE;
     TIME_LIMIT  : in     CAIS_DURATION     := LONG_DELAY;
     STATUS      : in     STATUS_INDICATOR  := NO_INDICATION);
```

Purpose:

This procedure includes the nodes in a designated primary tree in the transaction identified by TRANSACTION. It effectively performs the INCLUDE operation on the node identified by NODE and recursively applies INCLUDE to all nodes whose unique primary path traverses the designated node. The intent of the node handle NODE is used in applying INCLUDE to each node in the tree. If any of the specified nodes are already included in another open transaction and the transaction identified by TRANSACTION is not a descendant of that open transaction, the INCLUDE_TREE operation is delayed. 1

The TIME_LIMIT parameter allows the specification of a time limit for the delay imposed on INCLUDE_TREE by other transactions or by the existence of locks on the nodes to be included. A delayed INCLUDE_TREE call completes after any such transactions are closed and the nodes to be included are unlocked, or the specified time limit has elapsed. In the latter case, the exception LOCK_ERROR is raised. If INCLUDE_TREE raises an exception, no operations are included in the transaction. 2

Parameters:

**TRANSACTION** is an open transaction handle. 3

**NODE** is an open node handle. 4

**TIME_LIMIT** is a time limit for the delay on waiting for the unlocking of a node in accordance with the access synchronization which is to be established. 5


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

**STATUS** is unestablished status indicator. 6

Exceptions:

**STATUS_ERROR** is raised if the node handle NODE is not an open node handle [2], if the transaction handle TRANSACTION is not an open transaction handle [6] or if STATUS is not an established status indicator [3]. 7

**LOCK_ERROR** is raised if the INCLUDE_TREE operation is delayed beyond the specified time limit due to the existence of locks in conflict with the access synchronization which is to be established [1]. LOCK_ERROR may be raised prior to expiration of the timeout if the CAIS implementation can determine that a deadlock situation has occurred. 8

**SECURITY_VIOLATION** may be raised if the attempt to include operations on the specified node represents a violation of the mandatory access controls of the CAIS implementation. SECURITY_VIOLATION is raised only if the conditions for other exceptions are not present. 9

Notes:

A call to INCLUDE_TREE may be blocked if any of the specified nodes are already included in another open transaction, regardless of whether that transaction is part of the current process. This results when the specified transaction is not a descendant of the other, open transaction. 10

### 6.4.4 Closing a transaction

```
procedure CLOSE_TRANSACTION
    (TRANSACTION             : in out TRANSACTION_TYPE;
     COMMIT_MODIFICATIONS : in     BOOLEAN          := TRUE;
     STATUS               : in     STATUS_INDICATOR := NO_INDICATION);
```

Purpose:

This procedure closes the transaction identified by TRANSACTION, if any. 1

If COMMIT_MODIFICATIONS is TRUE, the transaction is committed. In this case, TRANSACTION must identify an open transaction; the transaction identified must not have any open subtransactions and must not enclose any process which has not terminated. 2

If COMMIT_MODIFICATIONS is FALSE, the transaction is to be aborted. In this case, if TRANSACTION is already closed, this procedure has no effect. If TRANSACTION is an open transaction handle, the operation is applied recursively to all open subtransactions of the identified transaction which were opened by the current process. All processes created in or enclosed by the identified transaction that have not terminated but can be aborted are aborted. A process can be aborted if the current process has sufficient access rights to traverse the primary relationships to the PROCESS nodes for the process, beginning at the current PROCESS node, and sufficient access rights to write attributes and contents of the PROCESS node of that process. If all of the processes can be aborted, the transaction is closed and its eligible effects are discarded. Otherwise, the transaction identified remains open and ACCESS_VIOLATION is raised. 3



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Parameters:

**TRANSACTION** is a transaction handle.

**COMMIT_MODIFICATIONS** is a Boolean; if TRUE, the effects of operations performed inside the specified transaction will be committed; if FALSE, the eligible effects of those operations will be discarded.

**STATUS** is an established status indicator.

Exceptions:

**STATUS_ERROR** is raised if COMMIT_MODIFICATIONS is TRUE and the transaction handle TRANSACTION is not open [6], or if STATUS is not an established status indicator [3].

**USE_ERROR** is raised if COMMIT_MODIFICATIONS is TRUE and either the transaction has open subtransactions or a process enclosed by the transaction has not terminated [58].

**ACCESS_VIOLATION** is raised if COMMIT_MODIFICATIONS is FALSE and the current process does not have sufficient discretionary access rights to obtain access to the PROCESS node of any process enclosed by the transaction with an intent including READ_RELATIONSHIPS, WRITE_RELATIONSHIPS and WRITE_CONTENTS [1].

Notes:

Due to insufficient access rights, CLOSE_TRANSACTION may abort some of the processes enclosed by a transaction without aborting others. Also, some subtransactions may also be aborted without aborting others. In these cases, the identified transaction is not closed and ACCESS_VIOLATION is raised.

### 6.4.5 Examining the open status of a transaction

```
function IS_OPEN
    (TRANSACTION : in     TRANSACTION_TYPE;
     STATUS      : in     STATUS_INDICATOR := NO_INDICATION)
    return BOOLEAN;
```

Purpose:

This function returns TRUE if the transaction identified by TRANSACTION is open; otherwise it returns FALSE.

Parameters:

**TRANSACTION** is a transaction handle.

**STATUS** is an established status indicator.



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Exceptions:

**STATUS_ERROR** is raised if STATUS is not an established status indicator [3]. 4

Notes:

If a transaction handle is open, the transaction identified by that handle is also open. When a transaction is closed, any handle identifying the transaction is closed. 5

### 6.4.6 Querying sameness for two transactions

```
function IS_SAME
    (TRANSACTION1 : in     TRANSACTION_TYPE;
     TRANSACTION2 : in     TRANSACTION_TYPE;
     STATUS       : in     STATUS_INDICATOR := NO_INDICATION)
    return BOOLEAN;
```

Purpose:

This function returns TRUE if the transaction handles TRANSACTION1 and TRANSACTION2 are open transactions handles which identify the same transaction or if both handles have the value NULL_TRANSACTION. 1

Parameters:

**TRANSACTION1** is a transaction handle which is open or has the value NULL_TRANSACTION. 2

**TRANSACTION2** is a transaction handle which is open or has the value NULL_TRANSACTION. 3

**STATUS** is an established status indicator. 4

Exceptions:

**STATUS_ERROR** is raised if at least one of the transaction handles, TRANSACTION1 and TRANSACTION2, is not an open transaction handle and does not have the value NULL_TRANSACTION [6] or if STATUS is not an established status indicator [3]. 5

### 6.4.7 Determining whether a node is included in a transaction

```
function IS_INCLUDED
    (NODE        : in     NODE_TYPE;
     TRANSACTION : in     TRANSACTION_TYPE   := NULL_TRANSACTION;
     STATUS      : in     STATUS_INDICATOR   := NO_INDICATION)
    return BOOLEAN;
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Purpose:

When TRANSACTION has the value NULL_TRANSACTION, this function returns TRUE if the node identified by NODE is included in any transaction and FALSE otherwise. 1

When TRANSACTION is an open transaction handle, this function returns TRUE if the node identified by NODE is included in the transaction identified by TRANSACTION and FALSE otherwise. 2

Parameters:

**NODE** is an open node handle. 3

**TRANSACTION** is a transaction handle which is open or has the value NULL_TRANSACTION. 4

**STATUS** is an established status indicator. 5

Exceptions:

**STATUS_ERROR** is raised if NODE is not an open node handle [2], if TRANSACTION is not an open transaction handle and does not have the value NULL_TRANSACTION [6], or if STATUS is not an established status indicator [3]. 6



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

*U.S. GOVERNMENT PRINTING OFFICE: 1990 704-034/30413

24 January 1990

Memorandum for Recipients, MIL-STD-1838, revision A (CAIS-A)

Subject: MIL-STD-1838A

Encl: MIL-STD-1838A, 6 April 1989

1. The Common APSE Interface Set (CAIS), Revision A (MIL-STD-1838A) has been under preparation since late CY 1985. Sponsored by the Ada Joint Program Office (AJPO), the following major milestones have been completed:

- o 12/85: CAIS-A Standard Development Contract Awarded to SofTech
- o 12/86: Initial DOD-STD-1838A Draft released
- o 10/87: First CAIS-A Public Review
- o 1/88: Second CAIS-A Public Review
- o 5/88: Final Draft DOD-STD-1838A delivered to AJPO
- o 8/88: CAIS-A selected for use by Boeing and Unisys in STARS development work
- o 11/88: Initial CAIS-A prototype delivered
- o 3/89: Final Government Review Team (GRT) Meeting
- o 3/89: SWG on APSE project approves use of CAIS-A
- o 4/89: After responding to 446 public review comments, CAIS-A standard approved
- o 10/89: CAIS-A standard approved for public release

2. Enclosed is your copy of the CAIS-A standard, approved 6 April 1989 and approved for public release on 3 October 1989.

Duston Hayward

D. Hayward
CAIS Project Manager

Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

METRIC
MIL-STD-1838A
6 April 1989

Superceding
DOD-STD-1838
9 October 1986

# MILITARY STANDARD

## COMMON ADA PROGRAMMING SUPPORT ENVIRONMENT (APSE) INTERFACE SET (CAIS)

(Revision A)

Volume III

NO DELIVERABLE DATA
REQUIRED BY THIS DOCUMENT

AREA MCCR/IPSC

Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

# CONTENTS


1 SCOPE 1
1.1 Purpose . . . 1
1.2 Application Guidance . . . 2

2 REFERENCED DOCUMENTS 3
2.1 Government Documents . . . 3
2.2 Other Publications . . . 3
2.3 Order of Precedence . . . 4
2.4 Source of Documents . . . 5

3 DEFINITIONS 7

4 GENERAL REQUIREMENTS 33
4.1 Introduction . . . 33
4.2 Method of Description . . . 37
4.2.1 Allowable differences . . . 37
4.2.2 Typographical conventions . . . 37
4.2.3 Semantic descriptions . . . 38
4.3 CAIS Node Model . . . 40
4.3.1 Nodes . . . 40
4.3.2 Relationships . . . 41
4.3.3 Attributes . . . 43
4.3.4 Processes . . . 43
4.3.5 Transactions . . . 44
4.3.6 Basic predefined definitions . . . 44
4.3.7 Names and identification . . . 50
4.3.8 Relationship Identification . . . 51
4.3.9 Pathnames . . . 53
4.3.10 Unique identifiers . . . 54
4.3.11 CAIS definitions . . . 55
4.3.12 Views . . . 59
4.3.13 Attribute monitors . . . 61
4.4 Access Control . . . 62
4.5 Input and Output . . . 64
4.5.1 Protocols . . . 64
4.5.2 Channels, client nodes, and PORT relationships . . . 65
4.5.3 Creating and using channels . . . 67
4.5.4 Compatibility checks . . . 69
4.5.5 Common Id . . . 69
4.6 Resource Control . . . 76
4.6.1 Resource model elements . . . 77
4.6.2 Resource allocatability . . . 80
4.6.3 Resource visibility . . . 80
4.6.4 Transparent resources . . . 82




Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

4.6.5 Resource access control. . . . . 82
4.6.6 Assigning resouces. . . . . 82
4.7 Distribution . . . . . 85
4.7. I Remote interprocess communications . . . . . 85

**5 CAIS NODE MANAGEMENT DETAILED REQUIREMENTS 87**
5.1 Package CAIS_DEFINITIONS . . . . . 88
5.2 Package CAIS_NODE_MANAGEMENT . . . . . 98
5.2.1 Open operations . . . . . 95
5.2.2 Accesses without open operations . . . . . 96
5.2.3 Access delays . . . . . 97
5.2.4 Opening a node handle.. . . . . 103
5.2.5 Opening a node handle using a unique identifier . . . . . 106
5.2.6 Closing a node handle . . . . . 107
5.2.7 Changing the intent regarding node handle usage . . . . . 108
5.2.8 Examining the open status of a node handle . . . . . 109
5.2.9 Querying the intention of a node handle . . . . . 109
5.2.10 Querying the kind of a node . . . . . 110
5.2.11 Determining if one node kind is a specialization of another . . . . . 110
5.2.12 Determining if one relaticn is a specialization of another . . . . . 112
5.2.13 Determining if a node belongs to a node kind . . . . . 113
5.2.14 Determining if a relationship belongs to a relation . . . . . 114
5.2.15 Querying the number of open file handles on a CLIENT node . . . . . 116
5.2.16 Obtaining the unique identifier for a node . . . . . 116
5.2.17 Obtaining a pathname for the unique primary path . . . . . 117
5.2.18 Obtaining the relationship key list of a primary relationship . . . . . 118
5.2.19 Obtaining the relation of a primary relationship . . . . . 118
5.2.20 Obtaining the relationship key list of the last relationship traversed . . . 119
5.2.21 Obtaining the relation of the last relationship traversed . . . . . 120
5.2.22 Obtaining a partial pathname . . . . . 121
5.2.23 Obtaining the name of the last relation in a pathname . . . . . 122
5.2.24 Obtaining the key list of the last relationship in a pathname . . . . . 122
5.2.25 Querying the existence of a node . . . . . 123
5.2.26 Querying sameness . . . . . 124
5.2.27 Obtaining an index for a node handle . . . . . 124
5.2.28 Opening a node handle to the parent . . . . . 125
5.2.29 Copying a node . . . . . 126
5.2.30 Copying trees . . . . . 129
5.2.31 Renaming a relationship . . . . . 134
5.2.32 Deleting the primary relationship to a node . . . . . 138
5.2.33 Deleting the primary relationships of a tree . . . . . 139
5.2.34 Creating secondary relationships . . . . . 141
5.2.35 Creating bidirectional secondary relationships . . . . . 143
5.2.36 Deleting secondary relationships . . . . . 146
5.2.37 Obtaining a corresponding reverse relationship’s relation . . . . . 148
5.2.38 Obtaining a corresponding reverse relationship’s key . . . . . 150
5.2.39 Setting the inheritance property of a relationship . . . . . 151
5.2.40 Determining if a secondary relationship is inheritable . . . . . 153



Source: https://assist.dla.mil -- Downloaded: 202610-26T07:54Z
Check the source to verify that this is the current version before use.

5.2.41 Setting the current node relationship . . . 154
5.2.42 Opening a node handle to the current node . . . 155
5.2.43 Determining the creation time of a node . . . 155
5.2.44 Determining the last time a relationship was modified . . . 156
5.2.45 Determining the last time that node contents were written . . . 157
5.2.46 Determining the last time an attribute was modified . . . 157
5.2.47 Iteration predicate subtype . . . 158
5.2.48 Obtaining the default iteration predicate . . . 168
5.2.49 Node iteration types and subtypes . . . 169
5.2.50 Creating an iterator over nodes. . . . 169
5.2.51 Determining node iteration status . . . 173
5.2.52 Determining the approximate size of the node iterator . . . 174
5.2.53 Getting the next node in a node iteration . . . 175
5.2.54 Skipping the next node in a node iteration . . . 176
5.2.55 Obtaining the path element for the next node in a node iteration . . . 177
5.2.56 Deleting anode iterator. . . . 178
5.2.57 Node kind instance iteration type . . . 178
5.2.58 Creating an iterator over instances of a node kind definition . . . 178
5.2.59 Determining node kind instance iteration status . . . 190
5.2.60 Determining the approximate size of the node kind instance iterator . . . 181
5.2.61 Getting the next node in a node kind instance iteration . . . 181
5.2.62 Skipping the next node in a node kind instance iteration . . . 183
5.2.63 Deleting a node kind instance iterator . . . 184
5.3 Package CAIS_ATTRIBUTE_MANAGEMENT . . . 185
5.3.1 Creating node attributes . . . 186
5.3.2 Creating relationships attributes . . . 187
5.3.3 Deleting node attributes . . . 189
5.3.4 Deleting relationship attributes . . . 190
5.3.5 Setting node attributes. . . . 192
5.3.6 Setting relationships attributes . . . 193
5.3.7 Getting node attributes. . . . 195
5.3.8 Getting relationship attributes . . . 196
5.3.9 Generic package CAIS_INTEGER_ATTRIBUTE . . . 198
5.3.10 Generic package CAIS_FLOAT_ATTRIBUTE . . . 207
5.3.11 Package CAIS_STRING_ATTRIBUTE . . . 217
5.3.12 Package CAIS_IDENTIFIER_ATTRIBUTE . . . 226
5.3.13 Generic package CAIS_ENUMERATION_ATTRIBUTE . . . 235
5.3.14 Package CAIS_BOOLEAN_ATTRIBUTE . . . 245
5.3,15 Attribute iteration types and subtypes . . . 245
5.3.16 Creating an iterator over node attributes . . . 246
5.3.17 Creating an iterator over relationship attributes . . . 247
5.3.18 Determining iteration status . . . 250
5.3.19 Determining the approximate size of the iterator . . . 250
5.3.20 Getting an identification for the next attribute . . . 251
5.3.21 Getting the next attribute value . . . 252
5.3.22 Skipping the next attribute in an iteration . . . 253
5.3.23 Deleting an attribute iterator . . . 253
5.4 Package CAIS_ACCESS_CONTROL_MANAGENIENT . . . 255



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

5.4.1 Node access . . . 255
5.4.2 Security labeling of CAIS nodes . . . 255
5.4.3 Groups and roles . . . 257
5.4.4 Granting access rights . . . 259
5.4.5 Denying access . . . 262
5.4.6 Access checking . . . 262
5.4.7 An example of CAIS access control . . . 263
5.4.8 Subtypes . . . 271
5.4.9 Value of all access rights . . . 271
5.4.10 Setting access control . . . 271
5.4.11 Deleting ACCESS relationships . . . 273
5.4.12 Obtaining the value of a GRANT attribute . . . 274
5.4.13 Denying access . . . 275
5.4.14 Deleting DENY relationships . . . 275
5.4.15 Examining access rights . . . 276
5.4.16 Adopting a role . . . 277
5.4.17 Unlinking an adopted role . . . 278
5.5 Package CAIS_STRUCTURAL_NODE_MANAGEMENT . . . 280
5.5.1 Opening the predefined STRUCTURAL NKD . . . 280
5.5.2 Creating structural nodes . . . 280
5.6 Package CAIS_TRANSLATION_MANAGEMENT . . . 284
5.6.1 Subtypes . . . 284
5.6.2 Adopting a view . . . 284
5.6.3 Cancelling a view adoption. . . . 285
5.6.4 Adopting an NKI . . . 286
5.6.5 Cancelling an NKI adoption . . . 288
5.6,6 Opening a handle to an NKD by local name . . . 289
5.6.7 Opening a handle to an NKI by local name . . . 290
5.6.8 Opening a handle to an RD by local name . . . 291
5.6.9 Opening a handle to an RI by local name . . . 291
5.6.10 Opening a handle to an AD by local name . . . 292
5.6.11 Opening a handle to an AKD by local name . . . 293
5.6.12 Determining the name for a definition . . . 294
5.6.13 Determining the NKD for a node . . . 295
5.6.14 Determining the narne for a relation . . . 296
5.6.15 Determining the RD for a relation . . . 297
5.6.16 Determining the name for a node attribute . . . 299
5.6.17 Determining the AD for a node attribute . . . 300
5.6.18 Determining the name for a relationship attribute . . . 301
5.6.19 Determining the AD for a relationship attribute . . . 302
5.7 Package CAIS_DEFINITION_MANAGEMENT . . . 304
5.7.1 Definition nodes . . . 304
5.7.2 Predefined entities . . . 342
5.7.3 Views . . . 347
5.7.4 Types and subtypes . . . 358
5.7.5 Opening the predefine VIEW NKD . . . 359
5.7.6 Opening the predefined NKI NKD . . . 359
5.7.7 Opening the predefine NKD NKD . . . 360



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

5.7.8 Opening the predefined RI NKD . . . . . 360
5.7.9 Opening the predefined RD NKD . . . . . 361
5.7.10 Opening the predefined AD NKD . . . . . 361
5.7.11 Opening the predefine named composite List AKD NKD . . . . . 362
5.7.12 Opening the predefine identifier AKD NKD . . . . . 362
5.7.13 Creating VIEW nodes . . . . . 363
5.7.14 Deleting the primary relationships of a tree rooted at a VIEW node . . . 366
5.7.15 Creating Nods . . . . . 367
5.7.16 Deleting NKIs and NKDs . . . . . 372
5.7.17 Creating NKIs . . . . . 373
5.7.18 Renaming NODE_KIND relationships in a view . . . . . 377
5.7.19 Specializing a node kind . . . . . 378
5.7.20 Generalizing node kinds . . . . . 380
5.7.21 Deleting SPECIALIZATION_OF relationships from NKDs . . . . . 382
5.7.22 Creating RDs . . . . . 384
5.7.23 Deleting RIs and RDs . . . . . 389
5,7.24 Creating Rib . . . . . 391
5.7.25 Renaming RELATION relationships in a view . . . . . 394
5.7.26 Specializing a relation . . . . . 395
5.7.27 Generalizing relations . . . . . 397
5.7.28 Deleting SPECIALIZATION_OF relationships from RDs . . . . . 400
5.7.29 Creating bidirectional RDs . . . . . 402
5.7.30 Deleting bidirectional RDs . . . . . 410
5.7.31 Creating ADs . . . . . 412
5.7.32 Deleting ADs . . . . . 415
5.7.33 Importing ADs . . . . . 417
5.7.34 Renaming ATTRIBUTE relationships in a view . . . . . 418
5.7.35 Redefining the AKD for an AD . . . . . 419
5.7.36 Creating Adds . . . . . 421
5.7.37 Creating identifier Adds . . . . . 425
5.7.38 Deleting Adds . . . . . 428
5.7.39 Importing Adds . . . . . 430
5.7.40 Renaming ATTRIBUTE-KIND relationships in a view . . . . . 431
5.7.41 Deleting the primary relationships of a tree rooted at a definition node . 432
5.7.42 Specifying source node kinds for a relation . . . . . 434
5.7.43 Deleting source node kinds for a relation . . . . . 436
5.7.44 Respecifying source node kinds for a relation . . . . . 438
5.7.45 Specifying key attributes for a relation . . . . . 439
5.7.46 Deleting key attributes for a relation . . . . . 442
5.7.47 Respecifying key attributes for a relation . . . . . 443
5.7.48 Specifying target node kinds for a relation . . . . . 445
5.7.49 Deleting target node kinds for a relation . . . . . 447
5.7.50 Specifying attributes for node kinds and relations . . . . . 448
5.7.51 Deleting attributes for node kinds and relations . . . . . 450
5.7.52 Respecifying attributes for node kinds and relations . . . . . 452
5.7.53 Specifying an attribute kind component . . . . . 453
5.7.54 Deleting an attribute kind component . . . . . 455
5.7.55 Specifying a variant attribute alternative . . . . . 457



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

5.7.56 Deleting a variant attribute alternative . . . 458
5.7.57 Setting definition node attributes . . . 460
5.7.58 Deleting definition node attributes . . . 462
5.7.59 Setting definition relationship attributes. . . . 464
5.7.60 Delete definition relations attributes . . . 467
5.7.61 Changing the NKD for a node . . . 469
5.7.62 Checking conformance of a node to an NKD . . . 471
5.7.63 Checking conformance of a relationship to an RD . . . 472
5.7.64 Checking conformance of a list to an AD or AKD . . . 475
5.8 Package CAIS_ATTRIBUTE_MONITOR_MANAGEMENT . . . 477
5.8.1 Types . . . 481
5.8.2 Opening the predefined MONITOR NKD . . . 481
5.8.3 Creating MONITOR nodes . . . 482
5.8.4 Deleting the primary relationship to a MONITOR node . . . 489
5.8.5 Associating an event handler process with a MONITOR node . . . 490
5.8.6 Specifying that a node attribute is to be monitored . . . 491
5.8.7 Specifying that a path attribute is to be monitored . . . 494
5.8.8 Canceling monitoring of designated attributes . . . 496
5.8.9 Determining a monitored attribute modification . . . 497
5.8.10 Awaiting a monitored attribute modification . . . 499
5.9 Package CAIS_RESOURCE_CONTROL . . . 503
5.9.1 Reassigning a node to a new medium resource . . . 503
5.9.2 Determining the medium resource to which a node is assigned . . . 504
5.9.3 Determining the processor resource to which a process is assigned . . . 505
5.9.4 Determining the current default medium resource . . . 506
5.9.5 Determining the current default processor resource . . . 506

**6 CAIS PROCESS MANAGEMENT DETAILED REQUIREMENTS 509**
6.1 Transactions . . . 514
6.1.1 Subtransactions and enclosing transactions . . . 514
6.1.2 Opening transactions . . . 514
6.1.3 Inclusion of nodes . . . 515
6.1.4 Inclusion of operations . . . 515
6.1.5 Visibility of effects . . . 516
6.1.6 Closing Transactions . . . 517
6.1.7 Enclosing processes in transactions . . . 518
6.1.8 Eligible effects of operations . . . 518
6.2 Package CAIS_PROCESS_DEFINITIONS . . . 520
6.3 Package CAIS_PROCESS_MANAGENIENT . . . 521
6.3,1 Determining the default priority . . . 524
6.3.2 Creating a dependent PROCESS node . . . 524
6.3.3 Creating a root PROCESS node. . . . 530
6.3.4 Activating a process . . . 535
6.3.5 Awaiting termination of another process . . . 536
6.3.6 Activating a process and awaiting its termination . . . 537
6.3.7 Spawning a new process . . . 538
6.3.8 Invoking a new process . . . 540
6.3.9 Creating and activating a new job . . . 543



Source: https://assist.dla.mil -- Downloaded: 202610-26T07:54Z
Check the source to verify that this is the current version before use.

6.3.10 Deleting a job . . . 545
6.3.11 Appending results . . . 546
6.3.12 Overwriting resets . . . 547
6.3.13 Getting results from a process . . . 547
6.3.14 Determining the status of a process . . . 548
6.3.15 Getting the parameter list . . . 549
6.3.16 Aborting a process . . . 550
6.3.17 Suspending a process . . . 551
6.3.18 Resuming a process . . . 552
6.3.19 Deterrnining the lowest specifiable priority . . . 553
6.3.20 Determining the highest specifiable priority . . . 553
6.3.21 Determining if the current priority is modifiable . . . 554
6.3.22 Modifying the current priority . . . 554
6.3.23 Determining the number of open node handles . . . 555
6.3.24 Determining the number of input and output units used . . . 556
6.3.25 Determining the time of activation . . . 556
6.3.26 Determining the time of termination . . . 557
6.3.27 Determining the time a process has been active . . . 558
6.3.28 Determining the size of a process . . . 558
6.4 Package CAIS_TRANSACTION_MANAGEMENT . . . 560
6.4.1 Opening a transaction . . . 560
6.4.2 Including a node in a transaction . . . 561
6.4.3 Including a tree in a transaction . . . 562
6.4.4 Closing a transaction. . . . 563
6.4.5 Examining the open status of a transaction . . . 564
6.4.6 Querying sameness for two transactions . . . 565
6.4.7 Determining whether a node is included in a transaction . . . 565

**7 INPUT AND OUTPUT DETAILED REQUIREMENTS 567**
7.1 Package CAIS_IO_DEFINITIONS . . . 569
7.2 Package CAIS_FILE_NODE_MANAGELIENT . . . 571
7.2.1 Opening the predefined SECONDARY_STORAGE NKD . . . 571
7.2.2 Creating secondary storage nodes . . . 571
7.3 Package CAIS_IO_CONNECTION . . . 575
7.3.1 Channel representation details . . . 575
7.3.2 Creating and deleting channels.. . . . 578
7.3.3 Opening protocols . . . 578
7.3.4 Compatibility rules . . . 578
7.3.5 Detailed protocol opening scenarios . . . 582
7.3.6 Discontinuing communication . . . 587
7.3.7 Opening the predefined PORT RD . . . 589
7.3.8 Creating a channel . . . 589
7.3.9 Releasing a channel . . . 593
7.3.10 Deleting a CHANNEL node . . . 594
7.3.11 Waiting for notification of attempted opening . . . 595
7.3.12 Opening the opposite client node . . . 597
7.3.13 Checking PORT relationship kind compatibility . . . 598
7.3.14 Setting PORT relationship attributes . . . 599



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

7.4 Package CAIS_IMPORT_EXPORT . . . . . 602
7.4.1 Importing a file from the host file system . . . . . 602
7.4.2 Exporting a file to the host file system . . . . . 603
7.5 Package CAIS_COMMON_IO . . . . . 604
7.5.1 Types and exceptions . . . . . 604
7.5.2 Opening the predefined INTRODUCED_NODE NKD . . . . . 604
7.5.3 Opening the predefined INTRODUCED_NODE_RD . . . . . 605
7.5.4 Opening anode set handle . . . . . 605
7.5.5 Including anode in anode set . . . . . 606
7.5.6 Including a tree in a node set . . . . . 607
7.5.7 Including a structure of nodes in a node set . . . . . 609
7.5.8 Transmitting a node set in common external form . . . . . 610
7.5.9 Closing a node set . . . . . 611
7.5.10 Introducing nodes in common external form . . . . . 612
7.5.11 Introducing contents of introduced nodes . . . . . 616
7.5.12 Creating VIEW nodes with specific UIDs . . . . . 618
7.5.13 Creating NKDs with specific UIDs . . . . . 619
7.5.14 Creating NKIs with specific UIDs . . . . . 620
7.5.15 Creating RDs with specific UIDs . . . . . 621
7.5.16 Creating RIs with specific UIDs . . . . . 622
7.5.17 Creating bidirectional RDs with specific UIDs . . . . . 623
7.5.18 Creating ADs with specific UIDs . . . . . 625
7.5.19 Creating AKDs with specific UIDs . . . . . 626
7.5.20 Creating structural nodes with specific UIDs . . . . . 627
7.5.21 Creating a secondary storage node with a specific UID . . . . . 628

**8 DATA TRANSFER PROTOCOLS DETAILED REQUIREMENTS 629**
8.1 Input and Output Detailed Requirements . . . . . 629
8.2 Package CAIS_FRAME_IO . . . . . 630
8.2.1 Protocol parameters . . . . . 631
8.2.2 Types, subtypes, and exceptions . . . . . 632
8.2.3 Opening a frame file handle . . . . . 633
8.2.4 Closing a frame file handle . . . . . 634
8.2.5 Determining whether a frame file handle is open . . . . . 635
8.2.6 Determining the mode of a frame file handle . . . . . 635
8.2.7 Transmitting a frame . . . . . 636
8.2.8 Receiving a frame . . . . . 637
8.2.9 Resetting a frame file handle . . . . . 638
8.2.10 Determining if all messages have been read . . . . . 639
8.2.11 Determining if incoming queue is empty . . . . . 640
3.2.12 Determining if outgoing queue is full . . . . . 641
8.2.13 Determining the channel node associated with a file . . . . . 641
8.3 Package CAIS_MAGNETIC_TAPE_IO . . . . . 643
8.3.1 Protocol parameters . . . . . 643
8.3.2 Types, subtypes, and exceptions . . . . . 644
8.3.3 Opening a tape drive file handle . . . . . 646
8.3.4 Closing a tape drive file handle . . . . . 647
8.3.5 Determining whether a file handle is open . . . . . 648



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

8.3.6 Determining the mode of a magnetic tape drive . . . 649
8.3.7 Requesting the mounting of a tape . . . 649
8.3.8 Loading a tape . . . 650
8.3.9 Unloading a tape . . . 651
8.3.10 Requesting the dismounting of a tape . . . 651
8.3.11 Determining the position of the tape . . . 652
8.3.12 Rewinding the tape . . . 652
8.3.13 Skipping tape marks . . . 653
8.3.14 Writing a tape mark . . . 654
8.3.15 Determining the status of a magnetic tape drive . . . 654
8.3.16 Determining the recording method of a magnetic tape . . . 655
8.3.17 Determining whether a write ring is installed . . . 655
8.3.18 Skipping blocks in a magnetic tape file . . . 656
8.3.19 Reading a block from a magnetic tape file . . . 657
8.3,20 Writing a block to a magnetic tape file . . . 658
8.3.21 Resetting a magnetic tape file handle . . . 659
8.3.22 Determining the channel node associated with a file . . . 659
8.4 Package CAIS_DIRECT_IO . . . 661
8.4.1 Protocol parameters . . . 661
8.4.2 Definition of types . . . 662
8.4.3 Creating a direct file . . . 662
8.4.4 Opening a direct file handle . . . 667
8.4.5 Closing a direct file handle . . . 669
8.4.6 Resetting a direct file handle . . . 669
8.4.7 Synchronizing internal file with FILE node contents . . . 670
8.4.8 Determining the channel node associated with a file . . . 671
8.5 Package CAIS_SEQUENTIAL_IO . . . 672
8.5.1 Protocol parameters . . . 672
8.5.2 Definition of types . . . 673
8.5.3 Creating a sequential file . . . 673
8.5.4 Opening a sequential file handle . . . 678
8.5.5 Closing a sequential file handle . . . 680
8.5.6 Resetting a sequential file handle . . . 680
8.5.7 Synchronizing internal file with FILE node contents . . . 682
8.5.8 Determining the channel node associated with a file . . . 682
8.6 Package CAIS_TEXT_IO . . . 684
8.6.1 Protocol parameters . . . 685
8.6.2 Definition of types . . . 685
8.6.3 Creating a text file . . . 685
8.6.4 Opening a text file handle . . . 690
8.6.5 Closing a text file handle. . . . 692
8.6.6 Resetting a text file handle . . . 693
8.6.7 Synchronizing internal file with FILE node contents . . . 694
8.6.8 Determining the channel node associated with a file . . . 695
8.6.9 Setting the maximum line length of a text file handle . . . 695
8.6.10 Setting the maximum page length of a text file handle . . . 696
8.7 Package CAIS_TERMINAL_IO . . . 698
8.7.1 Protocol parameters . . . 701



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

8.7.2 Types, Subtypes, Constants and Exceptions . . . . . 701
8.7.3 Opening a terminal file handle . . . . . 704
8.7.4 Closing a terminal file handle . . . . . 706
8.7.5 Determining whether a terminal file handle is open . . . . . 707
8.7.6 Determining the channel node associated with a terminal file handle . . . 707
8.7.7 Determining the physical display origin on the virtual display . . . . . 708
8.7.8 Re-orienting the physical display on the virtual display . . . . . 709
8.7.9 Determining if a terminal supports a set of graphic renditions . . . . . 709
8.7.10 Determining if a terminal supports a foreground color . . . . . 710
8.7.11 Determining if a terminal supports a background color . . . . . 710
8.7.12 Starting output buffering for a virtual display . . . . . 711
8.7.13 Ending output buffering for a virtual display . . . . . 711
8.7.14 Refreshing a virtual display . . . . . 712
8.7.15 Creating a screen . . . . . 712
8.7.16 Changing the active rendition for a screen . . . . . 713
8.7.17 Changing screen size . . . . . 714
8.7.18 Changing border characteristics . . . . . 714
8.7.19 creating a copy of an existing screen . . . . . 715
8.7.20 Deleting a screen . . . . . 716
8.7.21 Determining the size of a screen . . . . . 716
8.7.22 Determining the active rendition of a screen . . . . . 716
8.7.23 Determining whether a screen has a border . . . . . 717
8.7.24 Determining the border rendition of a screen . . . . . 717
8.7.25 Querying sameness of screens . . . . . 718
8.7.26 Pasting a screen onto the virtual display . . . . . 719
8.7.27 Removing a screen from a virtual display . . . . . 719
8.7.28 Relocating a screen on a virtual display . . . . . 720
8.7.29 Move screen to front . . . . . 720
8.7.30 Move screen to back . . . . . 721
8.7.31 Determining paste position of a screen . . . . . 721
8.7.32 Determining whether a screen is occluded . . . . . 722
8.7.33 Moving the cursor position on a screen , . . . . . 722
8.7.34 Determining the position of a cursor in a screen . . . . . 723
8.7.35 Writing a character to a screen . . . . . 723
8.7.36 Advancing the cursor to the next line . . . . . 724
8.7.37 Writing a line of text to a screen . . . . . 725
8.7.38 Inserting a character into a line . . . . . 725
8.7.39 Deleting characters from a like . . . . . 726
8.7.40 Inserting blank lines into a screen . . . . . 727
8.7.41 Deleting lines from a screen . . . . . 727
8.7.42 Erasing a region of a screen . . . . . 728
8.7.43 Changing the rendition for a region of a screen . . . . . 729
8.7.44 Determining a cell rendition . . . . . 729
8.7.45 Reading characters from a screen . . . . . 730
8.7.46 Sounding the terminal bell . . . . . 731
8.7.47 Starting output buffering for a screen . . . . . 731
8.7.48 Ending output buffering to a screen . . . . . 732
8.7.49 Enabling function key usage . . . . . 732



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

8.7.50 Determining function key usage . . . 733
8.7.51 Determining the number of function keys . . . 733
8.7.52 Determining the identification of a function key . . . 734
8.7.53 Enabling and disabling the locator . . . 734
8.7.54 Determining locator usage . . . 735
8.7.55 Determining the locator cursor position . . . 735
8.7.56 Determining the screen containing the locator cursor . . . 736
8.7.57 Determining locator cursor position relative to screen border . . . 737
8.7.58 Moving the locator cursor . . . 737
8.7.59 Determining the number of locator buttons . . . 738
8.7.60 Determining the identification of a locator button . . . 738
8.7.61 Reading a single input event . . . 739
8.7.62 Reading a string from a virtual keyboard . . . 740
8.7.63 Flushing the input event queue . . . 742
8.7.64 Determining if all input events have been read . . . 742
8.7.65 Determining if the input event queue is empty . . . 743

**9 CAIS LIST MANAGEMENT DETAILED REQUIREMENTS 745**
9.1 Package CAIS_LIST_MANAGEMENT . . . 751
9.1.1 Types, subtypes, constants and exceptions . . . 751
9.1.2 Copying a list . . . 752
9.1.3 Making a list empty . . . 753
9.1.4 Converting from text to list form . . . 753
9.1.5 Converting a list to its text representation . . . 75-4
9.1.6 Determining the equality of two lists . . . 755
9.1.7 Deleting an item from a linear list . . . 755
9.1.8 Determining the kind of list . . . 756
9.1.9 Determining the kind of list itern. . . . 757
9.1.10 Inserting a sequence of items into a linear list . . . 758
9.1.11 Concatenating two linear lists . . . 759
9.1.12 Extracting a sequence of items from a linear list . . . 759
9.1.13 Determining the length of a linear list . . . 760
9.1.14 Determining the position of the current linear list . . . 761
9.1.15 Determining whether the current linear list is outermost . . . 761
9.1.16 Making the next outer linear list current . . . 761
9.1.17 Making a nested sublist the current linear list . . . 762
9.1.18 Determining the length of the text form of a list or a list item . . . 763
9.1.19 Determining the name of a named item . . . 764
9.1.20 Determining the position of a named item . . . 765
9.1.21 Package CAIS_LIST_ITEM . . . 766
9.1.22 Package CAIS_IDENTIF1ER_ITEM . . . 771
9.1.23 Generic package CAIS_ENUMERATION_ITEM . . . 780
9.1.24 Package CAIS_BOOLEAN_ITEM . . . 786
9.1.25 Generic package CAIS_INTEGER_ITEM . . . 787
9.1.26 Generic package CAIS_FLOAT_ITEM . . . 792
9.1.27 Package CAIS_STRING_ITEM . . . 798



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

**10 CAIS STANDARD DETAILED REQUIREMENTS** **805**
10.1 Package CAIS_STANDARD . . . . . 805

**11 CAIS CALENDAR DETAILED REQUIREMENTS** **807**
11.1 Package CAIS_CALENDAR . . . . . 807
11.1.1 Types, subtypes, and exceptions . . . . . 807
11.1.2 Getting the current time . . . . . 808
11.1.3 Getting the year part of the time . . . . . 808
11.1.4 Getting the month part of the time . . . . . 808
11.1.5 Getting the day part of the time . . . . . 809
11.1.6 Getting the seconds part of the time . . . . . 809
11.1.7 Splitting time into its components . . . . . 810
11.1.8 Combining components of time . . . . . 811
11.1.9 Adding time and duration . . . . . 811
11.1.10 Subtracting time and duration . . . . . 812
11.1.11 Comparing two values of time . . . . . 812

**12 CAIS PRAGMATIC DETAILED REQUIREMENTS** **815**
12.1 Package CAIS_PRAGMATICS . . . . . 815
12.1.1 Identifier and view pragmatics . . . . . 815
12.1.2 Node model pragmatics . . . . . 816
12.1.3 Handle and iterator pragmatics . . . . . 817
12.1.4 Integer and real number pragmatics . . . . . 819
12.1.5 Access control pragmatics . . . . . 819
12.1.6 File and device pragmatics . . . . . 820
12.1.7 List pragmatics . . . . . 822

**13 CAIS STATUS MANAGEMENT DETAILED REQUIREMENTS** **825**
13.1 Package CAIS_STATUS_DEFINITIONS . . . . . 845
13.2 Package CAIS_STATUS_MANAGEMENT . . . . . 857
13.2.1 Creating a status indicator . . . . . 857
13.2.2 Setting a status indicator . . . . . 857
13.2.3 Copying a status indicator . . . . . 858
13.2.4 Deleting a status indicator . . . . . 858
13.2.5 Obtaining a status “indicator’s exception indication . . . . . 859
13.2.6 Obtaining status information from a status indicator . . . . . 859
13.2.7 Obtaining a descriptive message from a status indicator . . . . . 860
13.2.8 Determining if a status indicator is established . . . . . 860
13.3 Exception Conditions.. . . . . 862
13.3.1 Conditions for CAIS_DEFINITIONS exceptions . . . . . 862
13.3.2 Conditions for CAIS_PROCESS_DEFINITIONS exceptions . . . . . 908
13.3.3 Conditions for CAIS_IO_DEFINITIONS exceptions . . . . . 908
13.3.4 Conditions for CAIS_FRAME_IO exceptions . . . . . 914
13.3.5 Conditions for CAIS_MAGNETIC_TAPE_IO exceptions . . . . . 915
13.3.6 Conditions for CAIS_TERMINAL_IO exceptions . . . . . 915
13.3.7 Conditions for CAIS_COMMON_IO exceptions . . . . . 918
13.3.8 Conditions for CAIS_LIST_MANAGEMENT exceptions . . . . . 918
13.3.9 Conditions for CAIS_CALENDAR exceptions . . . . . 924
13.3.10 Conditions for CAIS_PRAGMATICS exceptions . . . . . 925



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

13.3.11 Conditions for CAIS_FORM_TERMINAL_IO exceptions . . . . . 926

**14 GUIDANCE INFORMATION** **927**

14.1 Keywords . . . . . 927



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

# APPENDIXES

**Appendix A Predeflned Entities** **929**
10 Predefined node description index . . . . . 929
20 Predefined Node Descriptions . . . . . 940
20.1 Top-level nodes . . . . . 943
20.2 Attribute definition nodes (ADs) . . . . . 960
20.3 Attribute kind definition nodes (AKDs) . . . . . 1000
20.4 Node kind definition nodes (NKDs) . . . . . 1027
20.5 Relation definition nodes (RDs) . . . . . 1050
20.6 Node kind interpretation nodes (NKIs) . . . . . 1114
20.7 Relation interpretation nodes (RIs) . . . . . 1130

**B CAIS Specification** **1167**

**C Compatibility with Pre-existing Standards** **1321**
10 Package CAIS_NODE_MANAGEMENT . . . . . 1321
10.1 Opening a node handle . . . . . 1321
10.2 Changing the intent regarding node handle usage . . . . . 1322
10.3 Querying the existence of a node . . . . . 1322
10.4 Querying sameness . . . . . 1322
10.5 Obtaining an open node handle to the parent . . . . . 1323
10.6 Copying a node . . . . . 1323
10.7 Copying trees . . . . . 1324
10.8 Renaming the primary relationship of a node . . . . . 1324
10.9 Deleting the primary relationship to a node . . . . . 1324
10.10 Deleting the primary relationships of a tree . . . . . 1325
10.11 Creating secondary relationships . . . . . 1325
10.12 Deleting secondary relationships . . . . . 1326
10.13 Setting inheritance property of a relationship . . . . . 1326
10.14 Determining if a secondary relationship is inheritable . . . . . 1326
10.15 Setting the current node relationship . . . . . 1327
10.16 Opening anode handle to the current node . . . . . 1327
10.17 Determining the creation time of a node. . . . . 1327
10.18 Determining the last time a relationship was modified . . . . . 1328
10.19 Determining the last time that node contents were written . . . . . 1328
10.20 Determining the last time an attribute was modified . . . . . 1328
10.21 Creating an iterator over nodes . . . . . 1329
10.22 Getting the next node in a node iteration . . . . . 1329
20 Package CAIS_ATTRIBUTE_MANAGEMENT . . . . . 1330
20.1 Creating node attributes . . . . . 1330
20.2 Creating path attributes . . . . . 1330
20.3 Deleting node attributes . . . . . 1330
20.4 Deleting path attributes . . . . . 1331
20.5 Setting node attributes . . . . . 1331
20.6 Setting path attributes . . . . . 1331
20.7 Getting node attributes . . . . . 1332



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

20.8 Getting path attributes . . . 1332
20.9 Creating an iterator over node attributes . . . 1333
20.10 Creating an iterator over relationship attributes . . . 1333
30 Package CAIS_ACCESS_CONTROL_MANAGEMENT . . . 1334
30.1 Subtypes . . . 1334
30.2 Setting access control . . . 1334
30.3 Deleting access relationships . . . 1334
30.4 Obtaining the value of a GRANT attribute . . . 1335
30.5 Examining access rights . . . 1335
30.6 Adopting a rose . . . 1335
40 Package CAIS_STRUCTURAL_NODE_MANAGEMENT . . . 1336
40.1 Creating structural nodes . . . 1336
50 Package CAIS_PROCESS_MANAGEMENT . . . 1337
50.1 Spawning a new process . . . 1337
50.2 Invoking a new process . . . 1338
50.3 Awaiting termination of another process . . . 1340
50.4 Deleting a job . . . 1340
50.5 Getting results from a process . . . 1340
50.6 Determining the status of a process . . . 1341
50.7 Aborting a process . . . 1341
50.8 Suspending a process . . . 1342
50.9 Resuming a process . . . 1342
50.10 Determining the number of open node handles . . . 1343
50.11 Determining the number of input and output units used . . . 1343
50.12 Determining the time of activation . . . 1344
50.13 Determining the time of termination . . . 1344
50.14 Determining the time a process has been active . . . 1344
50.15 Determining the size of a process . . . 1345
60 Package CAIS_IMPORT_EXPORT . . . 1345
60.1 Importing a file from the host file system . . . 1345
60.2 Exporting a file to the host file system . . . 1345
70 Package CAIS_FORM_TERMINAL_IO . . . 1346
70.1 Protocol parameters . . . 1347
70.2 Types, subtypes, and exceptions . . . 1347
70.3 Opening a form terminal file handle . . . 1348
70.4 Closing a form terminal file handle . . . 1350
70.5 Determining whether a file handle is open . . . 1350
70.6 Determining the channel node associated with a file . . . 1351
70.7 Determining the number of function keys . . . 1351
70.8 Determining intercepted input characters . . . 1352
70.9 Determining intercepted output characters . . . 1352
70.10 Creating a form . . . 1353
70.11 Deleting a form . . . 1354
70.12 Copying a form . . . 1354
70.13 Defining a qualified area. . . . 1355
70.14 Removing an area qualifier . . . 1355
70.15 Changing the active position . . . 1356
70.16 Querying the active position . . . 1357



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

70.17 Advancing forward to qualified area . . . 1357
70.18 Writing to a form . . . 1358
70.19 Erasing a qualified area . . . 1359
70.20 Erasing a form . . . 1359
70.21 Activating a form on a terminal . . . 1360
70.22 Reading from a form . . . 1360
70.23 Determining changes to a form . . . 1361
70.24 Determining the identification of a function key . . . 1362
70.25 Determining the termination key . . . 1362
70.26 Determining the size of a form . . . 1363
70.27 Determining the size of the terminal . . . 1363
70.28 Determining if the area qualifier requires space in the form . . . 1364
70.29 Determining if the area qualifier requires space on a terminal . . . 1364
80 Package CAIS_PRAGMATICS . . . 1365

**D Syntax Summary** **1367**
10 Pathname Syntax . . . 1368
20 GRANT Attribute Value Syntax . . . 1368
30 Classification Attribute Value Syntax . . . 1368
40 List External Representation Syntax . . . 1369
50 Iteration Predicate Syntax . . . 1370
60 External Node Set Syntax . . . 1371
70 ATTRIBUTES_MONITORED Attribute Value Syntax . . . 1372
80 Constraints Attribute Value Syntax . . . 1373
90 Status Information List Syntax . . . 1374

**E Discretionary Access Control Conformance Guidelines** **1381**
10 Class C1 . . . l38l
10.1 Capability a: user control of objects . . . 1381
10.2 Capability b: user ability to specify access modes . . . 1381
10.3 Capability c: user ability to specify individuals or groups . . . 1382
20 Class C2 . . . 1382
20.1 Capability a: user ability to control objects . . . 1383
20.2 Capability b: user ability to specify and control access modes . . . 1383
20.3 Capability c: user ability to specify individual users or groups of individual users . . . 1383
20.4 Capability d: user ability to include or exclude object access to the granularity of an individual . . . 1383
20.5 Capability e: assignment of access control by an authorized user . . . 1384
30 Class B3 . . . 1384
30.1 Capability a: user ability to control objects . . . 1385
30.2 Capability b: user ability to specify and control access modes . . . 1385
30.3 Capability c: access control list . . . 1385
30.4 Capability d: user ability to include or exclude object access to the granularity of an individual . . . 1385
30.5 Capability e: access permission to an object must be assigned by an authorized user . . . 1385
30.6 Capability f ability to deny access to individuals or groups . . . 1385



Source: https://assist.dla.mil -- Downloaded: 2026l0-26T07:54Z
Check the source to verify that this is the current version before use.

**F Implementation Dependencies** **1387**

**G List Of Interfaces** **1389**

**Index** **1407**



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

# FIGURES


**Figure** 1 CAIS system concept . . . 45
2 Tree formed by primary relationships . . . 47
3 Some predefined relations . . . 49
4 Example node kind definition and attribute definition . . . 56
5 Primary relationships to definition nodes . . . 57
6 VIEW node and emanating relationships . . . 60
7 Channels . . . 65
8 Multiple channels . . . 66
9 Channel state transition . . . 68
10 External node set . . . 72
11 Resource nodes example . . . 78
12 Resource node groups example . . . 79
13 Allocatability relationship example . . . 81
14 Resource visibility . . . 83
15 Remote interprocess communication . . . 86
16 COPY_ TREE example . . . 131
17 Discretionary access control . . . 264
18 Discretionary access control after process creation . . . 265
19 ACCESS relationships, Example 1 . . . 267
20 ACCESS relationships, Example 2 . . . 269
21 ACCESS relationships, Example 3 . . . 270
22 Inheritance of properties and constraints . . . 305
23 Example of an NKD lattice . . . 307
24 Node kind definition node (NOD) . . . 308
25 NKD and RD property representation . . . 309
26 Relation definition node (RD) . . . 311
27 Bidirectional RD . . . 312
28 Basic NKD and RD . . . 314
29 Attribute definition node (AD) and attribute kind definition node (AKD) . . . 315
30 Simple and variant AKDs . . . 321
31 Example of a variant AKD . . . 323
32 List, uniform list and composite list AKDs . . . 325
33 GRANT attribute kind representation . . . 326
34 TIME attribute kind representation . . . 327
35 Node kind interpretation node (NKI) . . . 347
36 Local names defined by an NKI . . . 348
37 Relation interpretation node (RI) . . . 349
38 Local names defined by an RI . . . 350
39 Adoption of views by a root process . . . 352
40 Adoption of views by a dependent process . . . 353
41 Allowable process status transitions . . . 511
42 Full channel representation . . . 575
43 Frame IO message queues . . . 630
44 Magnetic tape status transitions . . . 645




Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

45 Virtual display with screens . . . . . 699



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

# TABLES


| | | | |
|---|---|---|---|
| **Table** | I | Pathname BNF | 52 |
| | II | External Node Set BNF | 73 |
| | III | Resource Model Elements | 77 |
| | IV | Intents | 98 |
| | V | Matrix of Access Synchronization Constraints | 102 |
| | VI | Iteration Predicate BNF | 160 |
| | VII | Logical Operators in Iteration Predicates | 165 |
| | VIII | Applications of Attribute Functions | 165 |
| | IX | Classification Attribute Value BNF | 256 |
| | X | Predefined access rights | 260 |
| | XI | GRANT Attribute Value BNF | 262 |
| | XII | Predefined AKD Attributes | 319 |
| | XIII | Predefined Emanating Relations for AKDs | 320 |
| | XIV | Predefined Attributes of Relations Emanating from AKDs | 320 |
| | XV | Matrix of Composite List AKDs | 324 |
| | XVI | Predefined Relations that allow the CONSTRAINTS Attribute | 329 |
| | XVII | Constraints Attribute Value BNF | 330 |
| | XVIII | Applicable Constraint Items | 331 |
| | XIX | Inherited Properties and Constraints | 336 |
| | XX | Predefined Definition Node Attributes | 340 |
| | XXI | Predefine Definition Relationship Attributes | 341 |
| | XXII | ATTRIBUTES_MONITORED Attribute Value BNF | 478 |
| | XXIII | Relationships Created as a Result of CREATE. MONITOR. NODE | 480 |
| | XXIV | MONITOR Node Attributes Created by CREATE_MONITOR_NODE | 481 |
| | XXV | Events Causing Process Status Transition | 512 |
| | XXVI | Relationships Created for Root PROCESS Nodes | 522 |
| | XXVII | Relationships Inherited or Created for Dependent PROCESS Nodes | 523 |
| | XXVIII | DIRECTION Attribute Values.. | 576 |
| | XXIX | Channel State Transition | 577 |
| | XXX | DIRECTION Compatibility | 580 |
| | XXXI | Intent-DIRECTION Compatibility | 580 |
| | XXXII | Mode-DIRECTION Compatibility | 581 |
| | XXXIII | Intents for Protocol Opening | 581 |
| | XXXIV | Mode Compatibility | 581 |
| | XXXV | Detailed Protocol Opening Scenario | 583 |
| | XXXVI | Allowed Magnetic Tape Characters | 643 |
| | XXXVII | List External Representation BNF | 749 |
| | XXXVIII | Status Information List BNF | 827 |
| | XXXIX | Status Information List Description | 834 |
| | XL | Index To Predefined Node Descriptions | 929 |
| | XLI | Predefined Node Description Fields | 941 |




Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

# 7. INPUT AND OUTPUT DETAILED REQUIREMENTS

This section describes some of the CAIS interfaces for the transfer of data among PROCESS and FILE nodes. These interfaces are defined in CAIS packages:

a. CAIS_IO_DEFINITIONS defines types, subtypes, exceptions, and constants used in the other CAIS packages described in this section;

b. CAIS_FILE_NODE_MANAGEMENT defines interfaces for creating SECONDARY_STORAGE nodes.

c. CAIS_IO_CONNECTION defines interfaces for controlling channels;

d. CAIS_IMPORT_EXPORT defines interfaces for implementation-dependent transfer of file contents to and from the host operating system;

e. CAIS_COMMON_IO defines interfaces for transferring the structure and contents of a set of CAIS nodes.

Data transfer protocol packages are defined in Section 8 on page 629.

Throughout this document, the word "file" is used to mean an Ada external file. In the CAIS, an Ada external file can be the contents of a FILE node or data exchanged with a process by input and output methods. In [1815A] the word “file” is used to mean an internal file. In some of the packages in this section, input and output operations are expressed as operations on objects of some file type, rather than directly in terms of the contents of a node. These objects are internal to a CAIS process *(internal files).* Internal files may be identified by *file handles.* An Ada type FILE_TYPE is defined for values that represent file handles. Ada objects of this type can be associated with a file by means of CAIS interfaces causing an *open file handle* to be assigned to the object. While such an association is in effect, the file handle is said to be open. In this section, a parameter or variable of type FILE_TYPE identifies a file.

File node kinds include the specialized node kinds DEVICE, SECONDARY_STORAGE and GATEWAY. This means that input and output data transfers are allowed between one PROCESS node and another as well as between a PROCESS node and a general FILE node, a DEVICE node, a SECONDARY_STORAGE node, a GATEWAY node, or specializations of these kinds.

In this document, access rights for node contents, i.e., READ_CONTENTS, WRITE_CONTENTS, APPEND_CONTENTS, and the corresponding EXCLUSIVE rights control the ability of subjects to perform input and output operations on a node, irrespective of whether nodes of that kind possess contents or whether the contents are the data being transmitted.

An open file handle is always associated with an open node handle that was used in opening the file handle or returned along with the open file handle by a node creating interface. An open node handle may be associated with multiple open file handles. File handles may be closed without affecting the status of the associated node handle. However, closing the node handle also closes all handles associated with that FILE node handle (see Section 5.2.6 on page 107).

The locking semantics of intents provided upon opening node handles applies only to nodes (see Table IV on page 98). Thus, it is possible to open multiple file handles associated with a given node


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

handle opened under an exclusive intent on the contents of the node. Consequently, a single Ada program can obtain exclusive access to an external file and yet open and operate upon multiple file handles to this file without relinquishing exclusive access for obtaining additional file handles to the file. The opening of a second node handle to this node by the same or any other process will however be delayed as specified in Table V on page 102. It is not possible to change the intent relating to the contents of an open node handle if there are open file handles associated with the node. The interaction of operations on two or more file handles associated with the same open node handle is implementation-dependent.

13 A SECONDARY_STORAGE node represents a disk or other random access storage file. Each SECONDARY_STORAGE node has two predefine attributes related to the number of file storage units allocated to the contents of the node. The predefine attribute MAXIMUM_FILE_SIZE indicates the maximum number of file storage units that may be allocated to the contents of the SECONDARY_STORAGE node. A value of zero indicates that the number of file storage units is unrestricted. The predefined attribute CURRENT_FILE_SIZE indicates the number of file storage units that are allocated to the contents of the SECONDARY_STORAGE node. The exception CAPACITY_ERROR is raised when an operation on the contents of a SECONDARY_STORAGE node (the result of a write or close operation) causes the maximum permitted number of file storage units to be exceeded. The size of a file storage unit is defined by the implementation-defied constant FILE_STORAGE_UNIT_SIZE declared within the package CAIS_PRAGMATICS (see Section 12 on page 815).

14 CAIS predefine the NKDs and RDs to be used as the most general definitions of client node kinds, PORT relations, and CHANNEL node kinds. The predefine NKDs PROCESS, DEVICE, GATEWAY, and SECONDARY_STORAGE are the most general client node kinds. DEVICE, GATEWAY, and SECONDARY_STORAGE NKDs possess the predefine common generalization FILE. The DEVICE NKDs have two predefined specializations MAGNETIC_TAPE and TERMINAL. The predefine PROCESS NKD is the most general definition of all client PROCESS nodes. The predefine PORT RD is the most general definition of all PORT relations. The predefine CLIENT RD is the most general definition of the reverse link of PORT relations. The predefine CHANNEL NKD is the most general definition of all CHANNEL node kinds. Since the PORT RD does not constrain the protocols which can be used, tools which use the FILE, PORT, PROCESS, and CHANNEL definitions can perform input and output with no CAIS-imposed constraints on the semantic nature of the data transmitted. Tool authors wishing to use the semantic checking afforded by the CAIS will utilize specializations of the predefined definitions.


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

## 7.1 Package CAIS_IO_DEFINITIONS

This package defies types, constants, and exceptions used throughout the other packages defined in Section 7. 1

```
type CHANNEL_STATE is (CONNECTED, ESTABLISHED, CLOSED, RETIRED);
```

CHANNEL_STATE is the enumeration of channel states. 2

```
typo CAIS_IO_DURATION is now CAIS_DURATION;

INFINITE_IO_DELAY  : constant CAIS_IO_DURATION := CAIS_IO_DURATION'LAST;
LONG_IO_DELAY      : constant CAIS_IO_DURATION := CAIS_IO_DURATION'LAST -
                                                  CAIS_PRAGMATICS.SMALL_FOR_CAIS_DURATION;
```

CAIS_IO_DURATION is the type for time limits on input and output operations. INFINITE_IO_-DELAY and LONG_IO_DELAY are two such limits. The value INFINITE_IO_DELAY is special in that it really intends an infinite delay. If an operation specifies an INFINITE_IO_DELAY time limit, the limit will never be exceeded. Hence, the operation need never be timed. All other values of CAIS_IO_DURATION are timed delays. LONG_IO_DELAY is the longest specifiable delay that can be exceeded. 3

```
IN_INTENT       : constant INTENT_ARRAY := (1 => READ_CONTENTS);
INOUT_INTENT    : constant INTENT_ARRAY := (READ_CONTENTS, WRITE_CONTENTS);
OUT_INTENT      : constant INTENT_ARRAY := (1 => WRITE_CONTENTS);
APPEND_INTENT : constant INTENT_ARRAY := (1 => APPEND_CONTENTS);
```

The intents defined above correspond to the intents required for each of the modes associated with file nodes. 4

```
UMBOUNDED_FILE_SIZE :  constant CAIS_NATURAL := O;
```

UNBOUNDED_FILE_SIZE indicates, upon creation of a file, that the size of the file is to have an unrestricted limit. 5

```
CLIENT_ERROR     : exception;
DATA_ERROR       :    exception;
END_ERROR        :    exception;
LAYOUT_ERROR     :    exception;
MODE_ERROR       : exception;
OPEN_DENIED      :    exception;
PORT_ERROR       :    exception;
PROTOCOL_ERROR : exception;
TIME_EXPIRED     :    exception;
```


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

CLIENT_ERROR is raised if a prospective client node is not a FILE, DEVICE, PROCESS, or GATEWAY node or if it is a PROCESS node representing a completed process. This exception is also raised if a process attempting a protocol OPEN is not a client node of the specified channel. 6

DATA_ERROR is raised on input if the data read cannot be properly interpreted due to syntax or type errors. 7

END_ERROR is raised if an attempt is made to skip or to read past the end of a file. 8

LAYOUT_ERROR is raised if an operation attempts to exceed various limits on output or if an interface returns a value which exceeds the range of its subtype. 9

MODE_ERROR is raised if an input or output operation is attempted that is in conflict with the mode specified upon opening or resetting the respective file handle. 10

OPEN_DENIED is raised if an attempt to open a channel is denied by the opposite client process. 11

PORT_ERROR is raised if PORT relationships are incompatible or incorrectly specified. Port compatibility does not include mandatory or discretionary access checks. 12

PROTOCOL. ERROR is raised by varying conditions specific to each protocol. 13

TIME_EXPIRED is raised if an operation is delayed beyond a specified time limit for reasons other than a lock conflict. 14

The exception CAPACITY_ERROR defined in package CAIS_PRAGMATICS (see Section 12 on page 815) is raised if MAXIMUM_FILE_SIZE is exceeded. This may occur either during a write operation, a synchronize operation, or a close operation. 15


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

# 7.2 Package CAIS_FILE_NODE_MANAGEMENT

The package CAIS_FILE_NODE_MANAGEMENT defines primitive operations for creating SECONDARY_STORAGE nodes. The exceptions raised by the subprograms in this package are defined in the packages CAIS_DEFINITIONS and CAIS_PRAGMATICS. 1

## 7.2.1 Opening the predefine SECONDARY_STORAGE NKD

```
function SECONDARY_STORAGE_NKD
    (STATUS : in STATUS_INDICATOR := NO_INDICATION)
    return NKD_TYPE;
```

Purpose:

This function returns an open node handle with NO_ACCESS intent to the predefined NKD for SECONDARY_STORAGE nodes. 1

The node kind definition name SECONDARY_STORAGE and the relation NODE_KIND_DEFINITION are associated with the node handle returned as the key and relation used in opening the handle. See subprograms PATH_KEY (Section 5.2.20 on page 119) and PATH_RELATION (Section 5.2.21 on page 120). 2

Parameters:

**STATUS** is an established status indicator. 3

Exceptions:

**STATUS_ERROR** is raised if STATUS is not an established status indicator [3]. 4

## 7.2.2 Creating secondary storage nodes

```
procedure CREATE_SECONDARY_STORAGE_NODE
    (NODE                  : in out SECONDARY_STORAGE_NODE_TYPE;
     BASE                  : in     NODE_TYPE;
     KEY                   : in     RELATIONSHIP_KEY          := EMPTY_KEY_LIST;
     RELATION              : in     RD_TYPE;
     INTENT                : in     INTENT_ARRAY              := (1 => WRITE);
     ATTRIBUTES            : in     ATTRIBUTE_LIST            := EMPTY_LIST;
     DISCRETIONARY_ACCESS  : in     DISCRETIONARY_ACCESS_LIST := ALL_RIGHTS;
     MANDATORY_ACCESS      : in     MANDATORY_ACCESS_LIST     := EMPTY_LIST;
     KIND                  : in     NODE_KIND_NODE_TYPE       := SECONDARY_STORAGE_NKD;
     STATUS                : in     STATUS_INDICATOR          := NO_INDICATION;
     MEDIUM_RESOURCE       : in     MEDIUM_NODE_TYPE          := DEFAULT_MEDIUM);
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Purpose:

This procedure creates a SECONDARY_STORAGE node and installs the primary relationship to it as well as the corresponding reverse relationship to the parent identified by BASE. The relation definition identification and relationship key list of the primary relationship to the node are given by the parameters RELATION and KEY, respectively; the base node from which the primary relationship emanates is given by the parameter BASE. An open node handle to the newly created node with intents as specified by the INTENT parameter is returned in NODE.

The ATTRIBUTES parameter provides initial values for attributes of the node.

The DISCRETIONARY_ACCESS parameter specifies the initial access control information to be established between the created node and the default group node of the current user (see Section 5.4 on page 255).

The MANDATORY_ACCESS parameter specifies the object classification labels with which the node is to be created. If its value is the empty list, the node inherits the subject classification of the creating process as its object classification. Otherwise, it must be an unnamed list consisting of an identifier item and, optionally, an unnamed list of identifier items (see Table IX, page 256).

The MEDIUM_RESOURCE parameter specifies the medium resource on which the newly created node will reside.

KIND is an open node handle identifying the node kind for the new node. It must identify a SECONDARY_STORAGE node kind. It may specify either an NKD or an NKI. If KIND identifies an NKI, the NKI is made the current adopted interpretation for the node handle VIEW. This interface may not be used to create nodes of the predefine kind INTRODUCED_NODE.

Parameters:

**NODE** is a node handle, initially closed, to be opened to the newly created node.

**BASE** is an open node handle to the node from which the primary relationship to the new node is to emanate.

**KEY** is the relationship key list of the primary relationship to be created.

**RELATION** is an open node handle to the relation definition of the primary relationship to be created.

**INTENT** is the intent of subsequent operations on the node; the actual parameter takes the form of an array aggregate.

**ATTRIBUTES** is an empty or named list (see Section 9.1 on page 751) whose elements are used to establish initial values for attributes of the newly created node; each named item of the list specifies an attribute name and the value to be given to that attribute.

**DISCRETIONARY_ACCESS** is the initial access control information associated with the created node; it is the value of the GRANT attribute of the ACCESS relationship to the user's default GROUP node (see Section 5.4.4 on page 259).

**MANDATORY_ACCESS** is a list defining the classification label for the created node.

**KIND** is an open node handle identifying the node kind for the new node.

**STATUS** is an established status indicator.

**MEDIUM_RESOURCE** is an open node handle identifying a MEDIUM node.



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Exceptions:

**PATHNAME_SYNTAX_ERROR** is raised if the node identification given by KEY is syntactically illegal (see Table I, page 52) [2].

**NAME_ERROR** is raised if the parent of the NKI identified by KIND is inaccessible [1].

**EXISTING_NODE_ERROR** is raised if a node already exists with the identification given [1].

**SYNTAX_ERROR** is raised if the ATTRIBUTES parameter [8], the DISCRETIONARY_ACCESS parameter [10] or the MANDATORY_ACCESS parameter [9] is syntactically illegal.

**NOT_IN_VIEW_ERROR** is raised if a key name specified in KEY or an attribute name specified in ATTRIBUTES cannot be translated [1] or if KEY consists of a single relationship key designator and no key of the specified relation is visible [5].

**AMBIGUOUS_VIEW_ERROR** is raised if translation of a key name specified in KEY or an attribute name specified in ATTRIBUTES is ambiguous [1] or if KEY consists of a single relationship key designator and more than one key of the specified relation is visible [4].

**PREDEFINED_RELATION_ERROR** is raised if RELATION identifies a relation that is CAIS-controlled [1].

**RELATIONSHIP_ERROR** is raised if KEY specifies a value for the same key more than once [5] or if RELATION or its corresponding reverse relation has no unique key set [6].

**PREDEFINED_ATTRIBUTE_ERROR** is raised if an attribute name within the ATTRIBUTES parameter identifies an attribute that is CAIS-controlled [1].

**ATTRIBUTE_ERROR** is raised if ATTRIBUTES specifies a value for the same attribute more than once [6].

**USE_ERROR** is raised if the value for the MANDATORY_ACCESS [10] parameter is semantically illegal.

**STATUS_ERROR** is raised if the node handle BASE, RELATION, KIND or MEDIUM_RESOURCE is not open [2], if NODE is an open node handle at the time of the call [1] or if STATUS is not an established status indicator [3].

**NODE_KIND_ERROR** is raised if the open node handle RELATION does not identify an RD [1], if the open node handle KIND does not identify a SECONDARY_STORAGE node kind [2], if the open node handle MEDIUM_RESOURCE does not identify a MEDIUM node [1], or if the open node handle KIND identifies a node kind that is CAIS-controlled [4].

**CONFORMANCE_ERROR** is raised if the operation would result in an attribute, node or relationship not conforming to its definition or the constraints in force on that definition [1-4, 7-10, 11].

**RESOURCE_ALLOCATION_ERROR** is raised if the medium resource identified through MEDIUM_RESOURCE is not allocatable [1].

**RESOURCE_AVAILABILITY_ERROR** is raised if the medium resource identified through MEDIUM_RESOURCE is not available [1].

**TRANSACTION_ERROR** is raised if the operation is included in more than one transaction [1] or if the operation is included in a transaction and the operation modifies a node which is not included in the same transaction [2].



Source: https://assist.dla.mil -- Downloaded: 202610-26T07:54Z
Check the source to verify that this is the current version before use.

**LOCK_ERROR** is raised if a node handle to the parent of the NKI specified by KIND cannot be opened with intent READ_RELATIONSHIPS due to outstanding locks in conflict with that intent [1]. 35

**INTENT_VIOLATION** is raised if BASE was not opened with an intent establishing the right to create relationships [1]. 36

**ACCESS_VIOLATION** is raised if the discretionary access control rights of the current process are insufficient to gain access with CREATE_INSTANCES intents to the NKD identified through KIND [1] or with READ_RELATIONSHIPS intents to the parent of the NKI specified by KIND or with APPEND_CONTENTS intent to the MEDIUM node identified by MEDIUM_RESOURCE [1]. 37

**SECURITY_VIOLATION** is raised if the operation represents a violation of mandatory access controls. SECURITY_VIOLATION is raised only if the conditions for other exceptions are not present. 38

Additional Interfaces:

```
procedure CREATE_SECONDARY_STORAGE_NODE
    (NODE                  : in out SECONDARY_STORAGE_NODE_TYPE;
     BASE                  : in     NODE_TYPE;
     KEY                   : in     RELATIONSHIP_KEY       := EMPTY_KEY_LIST;
     RELATION              : in     RELATION_NAME          := DEFAULT_RELATION;
     INTENT                : in     INTENT_ARRAY           := (1 => WRITE);
     ATTRIBUTES            : in     ATTRIBUTE_LIST         := EMPTY_LIST;
     DISCRETIONARY_ACCESS  : in     DISCRETIONARY_ACCESS_LIST := ALL_RIGHTS;
     MANDATORY_ACCESS      : in     MANDATORY_ACCESS_LIST  := EMPTY_LIST;
     KIND                  : in     NODE_KIND_NAME         := "SECONDARY_STORAGE";
     STATUS                : in     STATUS_INDICATOR       := NO_INDICATION;
     MEDIUM_RESOURCE       : in     MEDIUM_NODE_TYPE       := DEFAULT_MEDIUM)
is
    RELATION_DEFINITION : RD_TYPE;
    KIND_DEFINITION     : NKI_TYPE;
begin
    DEFINITION_OF_RELATION (RELATION_DEFINITION, BASE, RELATION, STATUS);
    OPEN_NODE_KIND (KIND_DEFINITION, KIND, STATUS);
    CREATE_SECONDARY_STORAGE_NODE
       (NODE, BASE, KEY, RELATION_DEFINITION, INTENT, ATTRIBUTES,
        DISCRETIONARY_ACCESS, MANDATORY_ACCESS, KIND_DEFINITION,
        STATUS, MEDIUM_RESOURCE);
    CLOSE (RELATION_DEFINITION);
    CLOSE (KIND_DEFINITION);
exception
    when, others =>
        CLOSE (RELATION_DEFINITION);
        CLOSE (KIND_DEFINITION);
        raise;
end CREATE_SECONDARY_STORAGE_NODE;
```


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

## 7.3 Package CAIS_IO_CONNECTION

This package provides facilities for creating and deleting channels. The exceptions raised by the subprograms in this package are defined in the packages CAIS_DEFINITIONS, CAIS_IO_DEFINITIONS and CAIS_PRAGMATICS.

### 7.3.1 Channel representation details

Figure 42 shows the full channel representation, giving those features which are common to most channels.

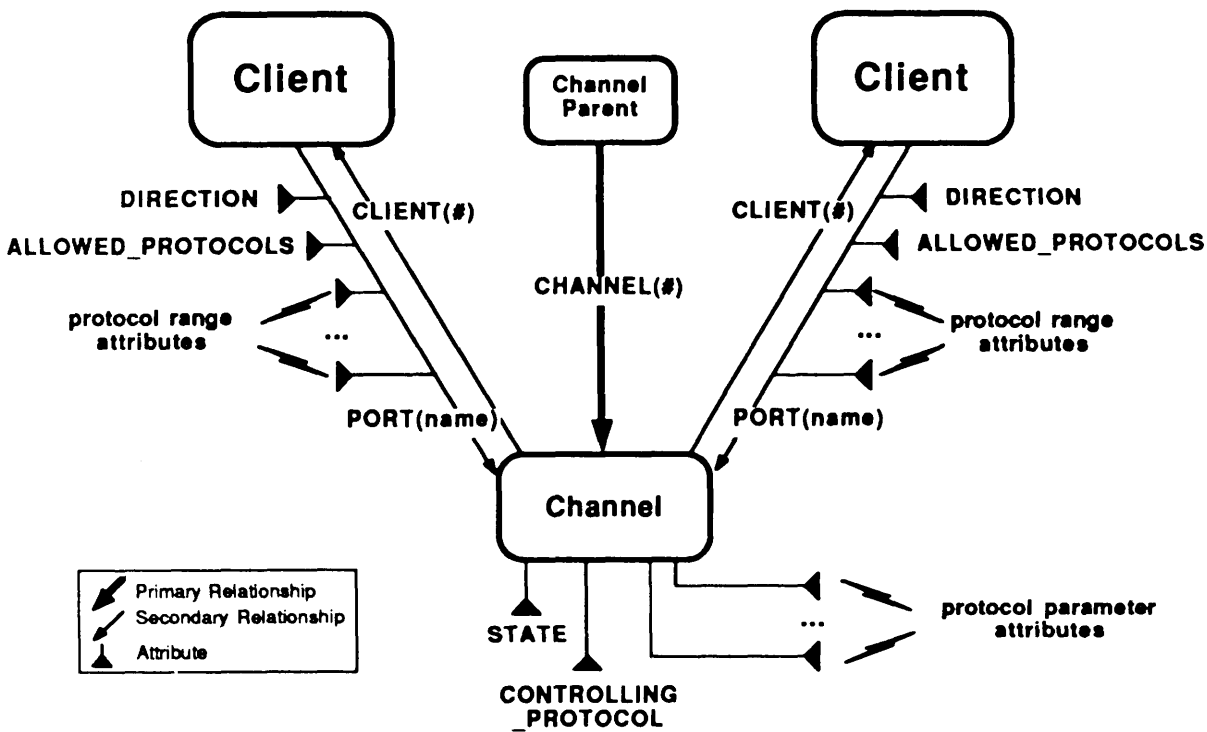


FIGURE 42. Full channel representation

A channel is represented in the CAIS node model by a CHANNEL node and a pair of PORT relationships which emanate from the nodes representing the clients of the channel and terminate at the CHANNEL node. As depicted in Figure 42, PORT relationships possess the DIRECTION attribute, the ALLOWED_PROTOCOLS attribute, a reverse link of the CLIENT relation, and the appropriate protocol range attributes for the protocols which may be used. These PORT relationship attributes may be created at the time a channel is created or they may be given values subsequent to the initial creation of a channel. All obtainable CHANNEL nodes possess a STATE attribute, a CONTROLLING_PROTOCOL attribute, and a terminating primary CHANNEL relationship. Depending upon



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

the protocol, CHANNEL nodes in the ESTABLISHED state may also possess copies of some or all of the protocol parameter attributes expressed on the PORT relationships which indicate the protocol parameter values selected during the opening of the protocol.

The predefine definitions allow PORT relationships to emanate from PROCESS, DEVICE, FILE, and GATEWAY nodes and to terminate at CHANNEL nodes. User-defined specializations of the predefine PORT RD can be defined such that PORT relationships which are instances of those RDs are constrained to emanate from specific kinds of nodes and terminate only a CHANNEL node which are instances of certain specializations of the predefine NKD for CHANNEL nodes.

#### 7.3.1.1 DIRECTION attribute

The DIRECTION attribute is an enumeration allowed to assume values described in Table XXVIII.

TABLE XXVIII: **DIRECTION Attribute Values**

| Attribute | Purpose |
|---|---|
| IN | data may only flow into the client node, |
| OUT | data may only flow out of the client node, |
| IN_AND_OUT | data will flow both ways, |
| IN_OR_OUT | data may or may not flow both ways, depending upon conditions which cannot be determined prior to protocol opening. |

#### 7.3.1.2 ALLOWED_PROTOCOLS attribute

The ALLOWED_PROTOCOLS attribute is a list. Each item in the list is an identifier giving the name of a protocol which may be used with the PORT relationship. An *allowed protocol* of a PORT relationship is a protocol that is named in the value of the ALLOWED_PROTOCOLS attribute of the PORT relationship. A protocol cannot be successfully opened unless it is an allowed protocol of both PORT relationships of a channel. If a PORT relationship does not have an ALLOWED_PROTOCOLS attribute, any attempt to perform a protocol open operation on the channel proceeds as if the PORT relationship had an ALLOWED_PROTOCOLS attribute with the value (NO_PROTOCOL).

#### 7.3.1.3 Protocol range attributes and protocol attributes

Each protocol has a number of associated controlling variables, called *protocol parameters*, that determine the operation of that protocol. (Protocol parameters have no direct relationship to the parameters occurring in subprogram and function calls.) Protocol parameters determine such quantities as buffer sizes, page sizes, and queue lengths. PORT relationships may possess attributes, called *protocol range* attributes, giving the allowed ranges for the protocol parameters applicable to the protocols used with a given PORT relationship. When a protocol is opened, values for the protocol parameters are determined by applying protocol-specific rules involving the ranges given by the protocol range parameters. The protocol attributes of the CHANNEL node are set to values chosen for the corresponding protocol parameters. Processes can subsequently obtain these values as normal attribute values.



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

#### 7.3.1.4 CONTROLLING. PROTOCOL attribute

Every channel in the ESTABLISHED state has a controlling protocol (see Section 7.3.4.2.6 on page 581). During protocol opening operations, the name of the protocol being opened becomes the value of the CONTROLLING_PROTOCOL attribute of the CHANNEL node. When a channel is in the CONNECTED and RETIRED states, the value of this attribute is NO_PROTOCOL. The value of the CONTROLLING_PROTOCOL attribute is managed automatically by the CAIS implementation.

#### 7.3.1.6 CHANNEL relation

The primary relationships to CHANNEL nodes are CHANNEL relationships. They emanate from a node designated by the process which creates the channel and terminates at the CHANNEL node created when the channel is created. Instances of this relation are created and deleted automatically as a by-product of channel creation and CHANNEL node deletion.

#### 7.3.1.6 STATE attribute

CHANNEL nodes have a STATE attribute with an IDENTIFIER value type. The value of this attribute is the state of the channel. This value is kept current by the CAIS implementation. The channel states and transitions are summarized in Table XXIX below and Figure 9 on page 68. A process can ascertain the state of a channel by obtaining the value of the STATE attribute.

TABLE XXIX: **Channel State Transition**

| State Before Operation | Operation | State After Operation |
|---|---|---|
| (Non-existent) | CREATE_CHANNEL | CONNECTED |
| CONNECTED | protocol OPEN | ESTABLISHED |
| (Non-existent) | protocol CREATE | ESTABLISHED |
| ESTABLISHED | protocol CLOSE[1] | CLOSED[3] |
| CLOSED | protocol CLOSE[1] | CONNECTED[4] |
| ESTABLISHED | protocol CLOSE[1] | CONNECTED[5] |
| ESTABLISHED | RELEASE_CHANNEL[2] | RETIRED[6] |
| CLOSED | RELEASE_CHANNEL[2] | RETIRED |
| CONNECTED | RELEASE_CHANNEL[2] | RETIRED |
| RETIRED | DELETE_CHANNEL_NODE | (Non-existent) |

[1] A close request may also be generated by the underlying implementation, see Section 7.3.6 on page 587.
[2] A delete request may also be generated by the underlying implementation, see Section 7.3.6 on page 587.
[3] The first close request when both client nodes are PROCESS nodes or one client node is a GATEWAY node.
[4] The second close request when both client nodes are PROCESS nodes or one client node is a GATEWAY node,
[5] When one client node is a FILE or DEVICE node.
[6] The channel may pass through the CLOSED state where orderly shutdown is possible,



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

### 7.3.2 Creating and deleting channels

Channels are created by the CAIS implementation as a result of calling CREATE_CHANNEL in this package or interfaces in some protocol packages. Channels are deleted by calling the RELEASE_-CHANNEL procedure to eliminate the PORT relationships and the DELETE_CHANNEL_NODE procedure to delete the CHANNEL node. CHANNEL nodes, CHANNEL relationships, and PORT relationships are created and deleted by the CAIS implementation as a result of creating and deleting channels; they may not be explicitly created or deleted by interfaces outside the CAIS_IO_CONNECTION package or protocol packages. 1

When a CHANNEL node is created, an ACCESS relationship is created which emanates from the CHANNEL node and terminates at the default GROUP node of the process calling the CAIS interface which resulted in the creation of the CHANNEL node. The GRANT attribute on this ACCESS relationship is given the value ((ALL_RIGHTS)) unless a different value for it is specified in the CAIS interface call. Additionally, the CAIS implementation automatically creates the necessary ACCES S relationships to allow the client processes of the channel to secure READ_ATTRIBUTES and READ_-RELATIONSHIPS access rights to the node, except where such rights would constitute a violation of the mandatory access controls for the CAIS. Specifically, for each client node that is a PROCESS node, an ACCESS relationship is created emanating from the CHANNEL node and terminating at the default GROUP node of the current user. The GRANT attribute of such relationships is given the value (((READ_ATTRIBUTES, READ_RELATIONSHIPS))). 2

### 7.3.3 Opening protocols

The interfaces provide two methods for a client process to open a protocol: 1

a. When the channel exists before the protocol OPEN is called, the *direct scenario* can be used. In this method, the client process simply calls the protocol OPEN interface. If the opposite client node is a SECONDARY_STORAGE or DEVICE node, the protocol is opened with no delay other than the normal delay for release of locks. If the opposite client node is a PROCESS or GATEWAY node, the operation is delayed until the opposite client process or the remote client process has also started a protocol OPEN operation. A task may be deployed to wait for protocol opening while other tasks continue to execute. 2

b. If the channel does not exist and the process is a server which waits for another process to create the charnel, the *server scenario* is used. By using the AWAIT_CHANNEL interface, a client process may obtain notification when channels become available for protocol opening as a result of actions by other processes. The client process may then issue a protocol OPEN to complete the protocol opening or a RELEASE_CHANNEL to suppress the attempt. 3

### 7.3.4 Compatibility rules

Checks to ensure compatibility between communicating processes, files, and devices are made when a channel is created and when a protocol is opened. 1

#### 7.3.4.1 Checks at channel creation

When a channel is created, a PORT relation compatibility y check is performed. If that check succeeds, 1



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

then a DIRECTION attribute compatibility check is performed if values for the DIRECTION attributes of both PORT relationships are available. The channel is created if the first check succeeds and the second check is not performed or if both checks succeed.

**7.3.4.1.1 PORT relation compatibility** A channel cannot be successfully created unless the normal rules for emanating and terminating relationships are followed. Users must ensure that the lattices of definition nodes correctly specify which PORT relations may be used with which client nodes and which PORT relations may be used in combination. Legal combinations are controlled by defining appropriate specializations of CHANNEL NKDs.

When creating a channel, two PORT RDs are specified, one for each of the PORT relationships to be created. For these two PORT RDs, there exists a set of CHANNEL NKDs, called the feasible CHANNEL NKD subset. The *feasible CHANNEL NKD subset* is that set of CHANNEL NKDs for which nodes of those kinds can be legal targets of relationships of the PORT relations specified in the call to a CAIS interface which creates a channel and for which the process calling the interface has sufficient discretionary access rights to gain access with CREATE_INSTANCES intent. The CAIS implementation is responsible for identifying the feasible CHANNEL NKD subset. If the feasible CHANNEL NKD subset is empty, the channel is not created. If the feasible CHANNEL NKD subset has more than one NKD, the CAIS implementation selects one in an implementation-defied fashion.

**7.3.4.1.2 DIRECTION attribute compatibility** The DIRECTION attributes of the two PORT relationships must have compatible values. Values of DIRECTION attributes may be specified as initial values from PORT RDs, may be specified in the call to the CREATE_CHANNEL interface, or may be set using the SET_PORT_ATTRIBUTE interface. If values for the DIRECTION attributes of both PORT relationships have been established by either or both of the first two methods above, then these values are checked for compatibility at the time the channel is created. The compatibility rules are given in Table XXX below. In this table, one PORT relationship is arbitrarily designated as A and the opposite PORT relationship as B.

### 7.3.4.2 Checks at protocol open

Eight compatibility checks are made when a protocol is opened. If any of these checks fail, the open fails. The checks are:

a. DIRECTION attribute compatibility,

b. intent-DIRECTION compatibility,

c. mode-DIRECTION compatibility,

d. mode-intent compatibility,

e. mode compatibility,

f. protocol compatibility,

g. client-to-client mandatory access rights, and

h. resource checks.



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

**7.3.4.2.1 DIRECTION attribute compatibility** Values for both DIRECTION attributes of the PORT relationships must be established and checked before protocol opening can be completed. Since the values of these attributes may be changed between channel creation and protocol opening, the checks are performed again, even if they were carried out at channel creation. (The checks can be omitted if they succeeded at channel creation and the implementation can verify that the values of the DIRECTION attributes were not altered.) Table XXX, below, gives the compatible pairs of DIRECTION attribute values.

TABLE XXX: **DIRECTION Compatibility**

| PORT B DIRECTION ATTRIBUTE VALUE | PORT A DIRECTION ATTRIBUTE VALUE | | | |
|---|---|---|---|---|
| | IN | OUT | IN_AND_OUT | IN_OR_OUT |
| IN | X | OK | X | OK |
| OUT | OK | X | X | OK |
| IN_AND_OUT | X | X | OK | OK |
| IN_-OR_OUT | OK | OK | OK | OK |

Legend: OK = allowed combination; X = invalid combination

**7.3.4.2.2 Intent-DIRECTION compatibility** All protocol OPEN operations have associated intents, derived from the open node handle of the opposite client node. These intents must be compatible with the value of the DIRECTION attribute of the local PORT relationship. For a given value of the DIRECTION attribute, Table XXXI, below, gives the intents allowed. If an intent is allowed, the corresponding exclusive intent is also allowed.

TABLE XXXI: **Intent-DIRECTION Compatibility**

| DIRECTION | ALLOWED INTENTS | | |
|---|---|---|---|
| | READ_CONTENTS | WRITE_CONTENTS | APPEND_CONTENTS |
| IN | OK | X | X |
| OUT | X | OK | OK |
| IN_AND_OUT | OK | OK | OK |
| IN_OR_OUT | OK | OK | OK |

Legend: OK = allowed combination; X = invalid combination

**7.3.4.2.3 Mode-DIRECTION compatibility** Many protocol Open interfaces have a MODE parameter which indicates the mode under which a file handle is to be opened. For protocol OPEN interfaces which do not have a MODE parameter, the mode under which the file handle is opened is implicit and defined by the respective protocols. In all cases, the mode under which a file handle is opened must be must be compatible with the value of the DIRECTION attribute of the PORT relationship emanating from the PROCESS node of the process issuing the call. The allowable combinations of DIRECTION attribute values and file handle modes are as specified in Table XXXII, below.


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

| TABLE XXXII: **Mode-DIRECTION Compatibility** | | | | |
|---|---|---|---|---|
| **DIRECTION** | **ALLOWED MODE VALUE** | | | |
| | **IN_FILE** | **INOUT_FILE** | **OUT_FILE** | **APPEND_FILE** |
| **IN** | OK | X | X | X |
| **OUT** | X | X | OK | OK |
| **IN_AN_OUT** | OK | OK | OK | OK |
| **IN_OR_OUT** | OK | OK | OK | OK |
| Legend: OK = allowed combination; X = invalid combination | | | | |

**7.3.4.2.4 Mode-intent compatibility** When a call to a protocol OPEN interface is made, the mode specified in the MODE parameter of the interface or the mode implied by the operation must be compatible with the intents of the opposite client node handle. Table XXXIII gives the opposite client node intents required to open a protocol with a given mode. For this purpose, an exclusive intent is considered equivalent to the corresponding non-exclusive intent.

| TABLE XXXIII: **Mode-Intent Compatibility** | |
|---|---|
| **MODE** | **Intents Required** |
| **IN_FILE** | READ_CONTENTS |
| **INOUT_FILE** | READ_CONTENTS and WRITE_CONTENTS |
| **OUT_FILE** | WRITE_CONTENTS |
| **APPEND-FILE** | APPEND_CONTENTS |

**7.3.4.2.5 Mode compatibility** When both clients of a channel are processes, the modes under which they open their file handles for data exchange over the channel must be compatible. The first client calling a protocol OPEN interface has that operation delayed until the second client also attempts an OPEN operation on the same channel (or calls RELEASE_CHANNEL). Once both clients have called protocol OPEN interfaces, the MODE parameters specified or implied on the calls are compared to ensure that each permits the data transfers implied by the other. Table XXXIV, below, gives the compatible pairs of file handle modes.

| TABLE XXXIV: **Mode Compatibility** | | | | |
|---|---|---|---|---|
| **CLIENT B MODE** | **CLIENT A MODE** | | | |
| | **IN_FILE** | **INOUT_FILE** | **OUT_FILE** | **APPEND_FILE** |
| **IN_FILE** | X | X | OK | OK |
| **INOUT_FILE** | X | OK | X | X |
| **OUT_FILE** | OK | X | X | X |
| **APPEND_FILE** | OK | X | X | X |
| Legend: OK = allowed combination; X = invalid combination | | | | |

**7.3.4.2.6 Protocol compatibility Two client nodes cannot transfer data unless:**

a. The controlling protocol is named in the values of the ALLOWED_PROTOCOLS attributes of both PORT relationships,

b. when there are two client processes, both are using the same protocol package for input and output on the channel, and



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

c. it is possible to adopt values for the protocol parameters which fall within the values of the applicable protocol range attributes of both PORT relationships.

Therefore, before a protocol can be opened for data transfer, it is necessary to name a protocol to be used and to adopt values for the protocol parameters which govern that protocol. The *controlling protocol* is the protocol which is to be used for data transfer. Names of the predefine protocols are the names of the protocol packages given in Section 4.5.1 on page 64. Names of protocol packages added by implementors are also protocol names. These must be documented in Appendix F of the reference manual for specific CAIS implementations. If there is only one client process, then the name of the controlling protocol is determined by the the protocol package to which the protocol OPEN interface, called by the process, belongs. The values of the ALLOWED_PROTOCOLS attributes on both the PORT relationships must include an identifier which is same as the name of the controlling protocol. If there are two client processes, then both OPEN operations must use the same protocol OPEN interface. The value of the CONTROLLING_PROTOCOL attribute of the CHANNEL node assumes the name of the controlling protocol.

Once the controlling protocol is determined, it is then possible to adopt values for the protocol parameters. The attributes on the PORT relationships give the ranges of protocol parameter values acceptable to the respective client nodes. The ranges must overlap so that a common value can be established. The semantics of each protocol specify the applicable protocol parameters, how the ranges are to be specified, and how an acceptable value is to be chosen (see the rules outlined for each protocol in Section 8 on page 629). The values of the protocol parameters adopted become the values of attributes of the CHANNEL node, called *protocol parameter attributes.*

For example, a protocol may have a parameter to control the maximum message length in characters. One PORT relationship may specify that the maximum value must be at least 50 characters but no more that 250 characters. If the opposite PORT relationship specifies that the maximum message length must be at least 100 characters but no more that 500 characters, an acceptable value in the range of 100 to 250 characters can be established. If the protocol semantics state that the message length limit should be maximized, the value of 250 characters is adopted and becomes the value of the MESSAGE_LENGTH_LIMIT attribute of the CHANNEL node.

**7.3.4.2.7 Client-to-client mandatory access rights** If a client node is a PROCESS node, it is a subject and must have mandatory access rights, relative to the opposite client node, consistent with the direction in which data are to be transmitted, as defined by the value of the MODE parameter of the protocol OPEN interface used. In a multilevel secure CAIS implementation, the implementation-defined mandatory access controls apply between the client nodes.

**7.3.4.2.8 Resource checks** If the opposite client node is a DEVICE or GATEWAY node, it is also a RESOURCE node. In this case, the protocol opening also constitutes the assignment of that RESOURCE node to the current process. This is successful only if the RESOURCE node is available and allocatable. These criteria are described in Section 4.6.6 on page 82.

### 7.3.5 Detailed protocol opening scenarios

The CAIS implementation must control the synchronization of events which coordinates the client processes during protocol opening. When a protocol is to be opened between two client nodes, one of the client nodes must always be a PROCESS node. This node is arbitrarily called client node A



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

and the opposite client node is called client node B. Client process A may use the direct or server scenario. Client node B can be a FILE or DEVICE node, a PROCESS node using the direct scenario, a PROCESS node using the server scenario, a GATEWAY node, or the same PROCESS node as client node A. Each of these situations is assigned a case number in Table XXXV, below. The detailed sequence of events for each case are described in the remainder of this section. However, some general principles are discussed first.

TABLE XXXV: **Detailed Protocol Opening Scenario**

| **CLIENT NODE B** | **CLIENT PROCESS A** | |
|---|---|---|
| | **Direct Scenario** | **Server Scenario** |
| FILE or DEVICE node | CASE 1 | CASE 6 |
| PROCESS using Direct Scenario | CASE 2 | CASE 7 |
| PROCESS using Server Scenario | CASE 3 | CASE 8 |
| GATEWAY node | CASE 4 | CASE 9 |
| Same PROCESS node as Client A | CASE 5 | CASE 5 |

Since certain CAIS-controlled attributes of PORT relationships are used to control the compatibility checking and the protocol used on a channel, the CAIS implementation must ensure that values of these attributes cannot be altered from the time both protocol OPEN calls have been issued until the protocol OPEN operation fails or the channel reenters the CONNECTED state from the ESTABLISHED state. These CAIS-controlled PORT relationship attributes are DIRECTION, ALLOWED_PROTOCOLS, and the predefine protocol parameter attributes.

When client node B is a GATEWAY node, there will eventually exist in the opposite CAIS instance, a client process, called the remote *client process*, which is connected to the opposite channel. The protocol opening semantics are engineered so that from the viewpoint of a process, connection to a remote client process is indistinguishable from connection to a process in the same CAIS instance. In the remote situation, the checks performed at channel creation and protocol opening are done relative to the local GATEWAY node, local PORT RDs, and local CHANNEL NKDs, not relative to any objects in the opposite CAIS instance. It is the responsibility of the CAIS implementations and the administrators of the installations to ensure that the correspondence between gateway pairs and channel pairs is maintained and that access constraints to GATEWAY nodes are such as to prevent unauthorized access to foreign CAIS instances.

The sequence of events for case 1 is:

a. the channel reaches the CONNECTED state;

b. client process A issues a call to a protocol OPEN interface which blocks;

c. if the time limit expires on the protocol OPEN operation, the scenario ends at this point;

d. the CONTROLLING_PROTOCOL attribute of the CHANNEL node assumes the name of the protocol used by process A;

e. the CAIS implementation must now prevent modification of attribute values of both PORT relationships;

f. the protocol opening compatibility checks (described in Section 7.3.4.2 on page 579) are performed. If these checks succeed, the channel is placed in the ESTABLISHED state and the pending OPEN operation in client process A completes successfully. Otherwise



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

the OPEN operation in client process A completes with failure, the CONTROLLING_-PROTOCOL attribute assumes the value NO_PROTOCOL, and the scenario ends;

g. if the OPEN operation succeeds, the channel may be used for data transmission.

The sequence of events for case 2 is:

a. the channel reaches the CONNECTED state;

b. client processes A and B issue calls to protocol OPEN interfaces which both block. These calls may be issued in any order. After the first OPEN is issued, the CONTROLLING_-PROTOCOL attribute assumes the name of the protocol. At this point, each client process has a pending protocol OPEN operation;

c. if the time limit on a pending protocol OPEN operation expires before the second protocol OPEN is issued, the scenario ends at this point;

d. the CAIS implementation must now prevent modification of attribute values of both PORT relationships;

e. the protocol opening compatibility checks (described in Section 7.3.4.2 on page 579) are performed. If these checks succeed, the channel is placed in the ESTABLISHED state and both pending OPEN operations complete successfully. Otherwise both pending OPEN operations complete with failure, the CONTROLLING_PROTOCOL attribute assumes the value NO_PROTOCOL, and the scenario ends;

f. if both OPEN operations succeed, the channel may now be used for data transmission.

The sequence of events for case 3 is:

a. the channel reaches the CONNECTED state and client process B issues a call to AWAIT_-CHANNEL which blocks. These events may occur in either order;

b. if the time limit of the AWAIT_CHANNEL operation expires, the scenario ends at this point ;

c. client process A issues a call to a protocol OPEN interface which blocks. Process A's OPEN may occur prior to client process B's AWAIT_CHANNEL;

d. if the time limit on process A's protocol operation expires before process B issues AWAIT_-CHANNEL, the scenario ends at this point;

e. the CONTROLLING_PROTOCOL attribute of the CHANNEL node assumes the name of the protocol used by process A;

f. the pending AWAIT_CHANNEL operation in client process B completes successfully. Process B then determines if the client process A's protocol open is to be accepted or not. If the open is to be rejected, then client process B issues a call to RELEASE_CHANNEL. The pending OPEN operation in client process A then completes with failure, the CONTROLLING_PROTOCOL attribute assumes the value NO_PROTOCOL, and the scenario ends. If the open is to be accepted, client process B issues a call to a protocol OPEN interface, which then blocks. At this point, each client process has a pending protocol OPEN operation;


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

g. the CAIS implementation must now prevent modification of attribute values of both PORT relationships;

h. the protocol opening compatibility checks (described in Section 7.3.4.2 on page 579) are performed. If these checks succeed, the channel is placed in the ESTABLISHED state and both pending OPEN operations complete successfully. Otherwise both pending OPEN operations complete with failure, the CONTROLLING_PROTOCOL attribute assumes the value NO_PROTOCOL, and the scenario ends;

i. if both OPEN operations succeed, the channel may be used for data transmission.

The sequence of events for case 4 is:

a. the channel reaches the CONNECTED state;

b. client process A issues a call to a protocol OPEN interface which blocks;

c. the CONTROLLING_PROTOCOL attribute of the CHANNEL node assumes the name of the protocol used by process A;

d. if the remote client process issues or has a pending AWAIT_CHANNEL, then AWAIT_-CHANNEL completes. If the remote client process issues a RELEASE_CHANNEL, then the pending OPEN in client process A completes with failure and the scenario ends. If the remote client process has a pending protocol OPEN, it remains blocked. If the remote client process issues a protocol OPEN call, then it blocks;

e. the CAIS implementation must now prevent processes from modifying attribute values of all four PORT relationships;

f. The CAIS implementations can now perform additional implementation-dependent checks. Such checks might be to validate the subjects, to check that protocols used on each channel are identical or at least compatible, or to check that the details of the low-level communication link are in order. If these checks fail, then both pending OPEN operations complete with failure, both CONTROLLING_PROTOCOL attributes assume the value NO_PROTOCOL, and the scenario ends;

g. the protocol opening compatibility y checks (described in Section 7.3.4.2 on page 579) are performed twice, once in each CAIS instance between the client process node and the GATEWAY node in the same CAIS instance. If these checks succeed, both channels are placed in the ESTABLISHED state and both pending OPEN operations complete successfully. Otherwise both pending OPEN operations complete with failure, both CONTROLLING_-PROTOCOL attributes assume the value NO_PROTOCOL, and the scenario ends;

h. if both OPEN operations succeed, the channels may be used for data transmission between CAIS instances.

In case 5, where both client nodes are the same PROCESS node, each end of the channel is treated as if it were connected to a distinct PROCESS node. Therefore, case 2, 3, or 8 applies. In case 5, two distinct CHANNEL node handles must be obtained to differentiate the ends of the channel; moreover, processing for each end of the channel must be performed by distinct Ada tasks.


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

The sequence of events for case 6 is:

a. client process A issues a call to AWAIT_CHANNEL, This call blocks until a channel satisfying the key pattern parameter in the call enters the CONNECTED state. This can happen as a result of closing a protocol on an existing channel or the creation of a new channel. The call does not block if there already exists a channel in the CONNECTED state satisfying the key pattern parameter. From this point on, this scenario is identical to case 1, beginning at step b.

Case 7 is the same as case 3 except that the roles of client nodes A and B are reversed.

The sequence of events for case 8 is:

a. client process A issues a call to AWAIT_CHANNEL which blocks, client process B issues a call to AWAIT_CHANNEL which blocks, and the channel reaches the CONNECTED state. These three events may occur in any order;

b. both pending AWAIT_CHANNEL calls complete;

c. either client process may issue a RELEASE_CHANNEL, thereby ending the scenario. If the opposite client process has a pending protocol OPEN, then that call completes with failure;

d. if the channel is not deleted, from this point on, the scenario is equivalent to case 2, beginning at step b.

The sequence of events for case 9 is:

a. client process A issues a call to AWAIT_CHANNEL. This call blocks until a channel satisfying the key pattern parameter in the call enters the CONNECTED state, the opposite channel reaches the CONNECTED state, and the remote client process has a pending AWAIT_CHANNEL or protocol OPEN operation;

b. if the remote client process has a pending AWAIT_CHANNEL operation, then both AWAIT_CHANNEL operations complete successfully. If the remote client process has a pending protocol OPEN operation, then the local AWAIT_CHANNEL operation completes successfully;

c. client process A issues a protocol OPEN operation which blocks;

d. the CONTROLLING_PROTOCOL attribute of the CHANNEL node assumes the name of the protocol used by process A;

e. if the remote client process issues a RELEASE_CHANNEL, then the pending OPEN in client process A completes with failure, the CONTROLLING_PROTOCOL attribute assumes the value NO_PROTOCOL, and the scenario ends. If the remote client process has a pending protocol OPEN, it remains blocked. If the remote client process issues a protocol OPEN call, then it blocks;

f. AWAIT_CHANNEL completes. At this point, this scenario is identical to case 4, beginning at step e.


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

### 7.3.6 Discontinuing communication

Shutdown of an ESTABLISHED channel is done in two steps, protocol closing and channel deletion. Protocol closing allows communication to be discontinued in an orderly fashion. When a protocol is closed without deleting the channel, the channel can be reused in another protocol open. Channel deletion causes the channel to be eliminated with no possibility of reuse. If a protocol is open on a channel to be deleted, the channel deletion includes protocol closing as a first step. Protocol closing and channel deletion are marked by events called close *request* and *delete request*, respectively.

A close request is recognized for a channel when:

a. one of the client processes calls a protocol CLOSE interface,

b. a close request occurs on the opposite channel where one of the client nodes is a GATEWAY node, or

c. a delete request is recognized for a channel in the ESTABLISHED or CLOSED states.

A close request is deemed to have been issued by the client process which issues the protocol CLOSE operation. If a close request originates at a remote client process, the local GATEWAY node is deemed to have issued the close request.

A delete request is recognized for a channel when:

a. one of the client processes, or the remote client process calls RELEASE_CHANNEL,

b. one of the client processes, or the remote client process aborts or terminates, or

c. the channel is to be deleted due to a problem identified by the underlying implementation.

When a close request results from a delete request, the close request is deemed to have been issued by the terminating process or the process issuing the RELEASE_CHANNEL. If the delete request is originated by the underlying implementation, then the implementation must determine if a close request is deemed to have been issued by one or both of the client nodes. If no buffered data can be recovered and both client nodes are processes, the implementation may generate two close requests, one from each client process.

#### 7.3.6.1 Closing protocols

When a close request is recognized, the channel enters the CLOSED or CONNECTED state depending upon how the close request originated and the kinds of client nodes.

If a client process issues a close request to a channel in the ESTABLISHED state, then:

a. if the opposite client node is a FILE or DEVICE node, the channel immediately enters the CONNECTED state and the CONTROLLING_PROTOCOL attribute assumes the value NO_PROTOCOL.

b. if the opposite client node is a PROCESS node, then the channel enters the CLOSED state. The CONTROLLING_PROTOCOL attribute value is unchanged.


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

c. if the opposite client node is a GATEWAY node, then both the local and opposite channels immediately enters the CLOSED state. The CONTROLLING_PROTOCOL attribute values of both CHANNEL nodes is unchanged.

To provide for orderly shutdown, some input and output operations are still valid for channels in the CLOSED state. The following are true for channels in the CLOSED state:

a. any write or put operation to the channel fails.

b. the client process which issues a protocol CLOSE operation may no longer read the channel using the file handle specified in the call to CLOSE.

c. any protocol input or output operation on a channel for which the same process has issued a RELEASE_CHANNEL fails.

d. any protocol input or output operation on a channel for which the underlying implementation has generated a close request for the same process fails with an exception.

e. read or get operations by a client process which has not issued a close request to the channel proceeds normally until any buffered data are exhausted, at which time some end-of-file indication is generated. The exact behavior is protocol-dependent.

f. any protocol OPEN operation to the CHANNEL node fails.

g. modifications to CAIS-controlled PORT relationship attributes may be made using the SET_PORT_ATTRIBUTE interface.

If a client node issues a close request to a channel in the CLOSED state, then:

a. if the opposite client node is a PROCESS node, then the opposite client process has already issued a close request to this channel. The channel immediately enters the CONNECTED state and the CONTROLLING_PROTOCOL attribute assumes the value NO_PROTOCOL.

b. if the opposite client node is a GATEWAY node, then both the local and opposite channels immediately enters the CONNECTED state and the CONTROLLING_PROTOCOL attribute values of both CHANNEL nodes assumes the value NO_PROTOCOL.

#### 7.3.6.2 Deleting channels

If a channel is in the CONNECTED state when a delete request is recognized, the PORT relationships are deleted and the channel enters the RETIRED state.

If a delete request is recognized for a channel in the ESTABLISHED state, the channel may not enter the RETIRED state immediately. The channel may pass through the CLOSED state in order to provide an opportunity for orderly shutdown. Similarly, if a delete request is recognized for a channel in the CLOSED state, the channel may remain in the CLOSED state long enough to allow orderly shutdown before entering the RETIRED state. Once the protocol is closed, the PORT relationships are deleted and the channel enters the RETIRED state.

Once in the RETIRED state, a channel may not return to the CONNECTED state, nor may a


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

protocol be opened on the channel. A delete request recognized for a channel in the RETIRED state has no effect. The CHANNEL node of a channel in the RETIRED state can be deleted by calling DELETE_CHANNEL_NODE.

### 7.3.7 Opening the predefine PORT RD

```
function PORT_RD
    (STATUS :  in STATUS_INDICATOR : = NO_INDICATION)
    return PORT_RD_TYPE;
```

Purpose:

This function returns an open node handle with NO_ACCESS intent to the predefine PORT RD node. 1

The relation definition name PORT and the relation RELATION_DEFINITION are associated with the node handle returned as the key and relation used in opening the handle. See sub-programs PATH_KEY (Section 5.2.20 on page 119) and PATH_RELATION (Section 5.2.21 on page 120). 2

Parameters:

**STATUS** is an established status indicator. 3

Exceptions:

**STATUS_ERROR** is raised if STATUS is not an established status indicator [3]. 4

### 7.3.8 Creating a channel

```
procedure CREATE_CHAIU!lEL
    (CHANNEL                : in out CHANNEL_NODE_TYPE;
     CLIENT_A               : in     CLIENT_NODE_TYPE;
     PORT_KEY_A             : in     RELATIONSHIP_KEY  := EMPTY_KEY_LIST;
     PORT_RELATION_A        : in     PORT_RD_TYPE      := PORT_RD;
     CLIENT_B               : in     CLIENT_NODE_TYPE;
     PORT_KEY_B             : in     RELATIONSHIP_KEY  := EMPTY_KEY_LIST;
     PORT_RELATION_B        : in     PORT_RD_TYPE      := PORT_RD;
     ATTRIBUTES_A           : in     ATTRIBUTE_LIST    := EMPTY_LIST;
     ATTRIBUTES_B           : in     ATTRIBUTE_LIST    := EMPTY_LIST;
     CHANNEL_PARENT         : in     PATHNAME          := CURRENT_PROCESS;
     DISCRETIONARY_ACCESS : in     DISCRETIONARY_ACCESS_LIST := ALL_RIGHTS;
     MANDATORY_ACCESS       : in     MANDATORY_ACCESS_LIST := EMPTY_LIST;
     TIME_LIMIT             : in     CAIS_DURATION     := LONG_DELAY;
     STATUS                 : in     STATUS_INDICATOR := NO_INDICATION;
     MEDIUM_RESOURCE        : in     MEDIUM_NODE_TYPE  := DEFAULT_MEDIUM);
```


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Purpose:

This procedure creates a channel between two client nodes. The client nodes are those identified by the open node handles CLIENT_A and CLIENT_B. The primary relationship to the newly created CHANNEL node is a CHANNEL relationship with a key equal to LATEST_KEY which emanates from the node identified by the pathname specified in CHANNEL_PARENT. The newly created CHANNEL node is an instance of the NKD selected by the CAIS implementation from the feasible CHANNEL NKD subset (see Section 7.3.4.1.1 on page 579) for the PORT RDs identified by the PORT_RELATION_A and PORT_RELATION_B parameters. An open node handle identifying the newly created CHANNEL node is returned in the CHANNEL parameter. This node handle CHANNEL is opened with the intent NO_ACCESS without traversing any relationships. The STATE attribute of the CHANNEL node is set to CONNECTED. 1

PORT relationships are created which emanate from the respective client nodes and which terminate at the newly created CHANNEL node. The PORT_RELATION_A and PORT_-RELATION_B parameters identify the PORT RDs to be used in the construction of the PORT relationships to be created from the node identified by CLIENT_A and CLIENT_B respectively. The keys on the PORT relationships are given by the PORT_KEY_A and PORT_KEY_B parameters. Unique values for the keys of the corresponding reverse relationships (the CLIENT relationships) are obtained using the constant LATEST_KEY. 2

The ATTRIBUTES_A and ATTRIBUTES_B parameters can be used to specify attribute values on the PORT relationships of the channel. Values for the predefined PORT relationship attributes DIRECTION and ALLOWED_PROTOCOLS as well as values for protocol-specific range parameter attributes can be specified. The predefined attributes DIRECTION and ALLOWED_PROTOCOLS are not created on the respective PORT relationships unless values for them are specified in ATTRIBUTES_A and ATTRIBUTES_B or default values have been specified for those attributes by the PORT RDs identified by PORT_RELATION_A and PORT_-RELATION_B. The absence of an ALLOWED_PROTOCOL attribute on a PORT relationship is semantically equivalent to the PORT relationship having an ALLOWED_PROTOCOLS attribute with the value (NO_PROTOCOL). 3

The DISCRETIONARY_ACCESS parameter specifies the value of the GRANT attribute of the ACCESS relationship to the current user's default GROUP node (see Section 5.4 on page 255). Additional ACCESS relationships emanating from the newly created CHANNEL node are created as specified in Section 7.3.2 on page 578. 4

The MANDATORY_ACCESS parameter specifies the object classification label to be given to the newly created CHANNEL node. If its value is the empty list, the node inherits the subject classification of the creating process as its object classification. Otherwise, it must be an unnamed list consisting of an identifier item and, optionally, an unnamed list of identifier items (see Table IX, page 256). 5

The MEDIUM_RESOURCE parameter specifies the medium resource on which the newly created CHANNEL node will reside. 6

Parameters:

CHANNEL is a node handle, initially closed, to be opened to the newly created CHANNEL node. 7

CLIENT_A is an open node handle to one of the client nodes. 8



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

**PORT_KEY_A** is the key value PORT relationship A is to have.

**PORT_RELATION_A** is an open node handle to an RD defining the relation to be used for PORT relationship A.

**CLIENT_B** is an open node handle to the other client node.

**PORT_KEY_B** is the key value PORT relationship B is to have.

**PORT_RELATION_B** is an open node handle to an RD defining the relation to be used for PORT relationship B.

**ATTRIBUTES_A** is an empty or named list (see Section 9.1 on page 751) whose elements are used to establish initial values for attributes of PORT relationship A; each named item of the list specifies an attribute name and the value to be given to that attribute.

**ATTRIBUTES_B** is an empty or named list (see Section 9.1 on page 751) whose elements are used to establish initial values for attributes of PORT relationship B; each named item of the list specifies an attribute name and the value to be given to that attribute.

**CHANNEL_PARENT** is the pathname which identifies the node from which the CHANNEL relationship will emanate.

**TIME_LIMIT** is a time limit for the delay on waiting for the unlocking of CHANNEL_-PARENT in accordance with the implied intent APPEND_RELATIONSHIPS.

**DISCRETIONARY_ACCESS** is the initial access control information associated with the created node; it is the value of the GRANT attribute of the ACCESS relationship to the user's default GROUP node (see Section 5.4.4 on page 259).

**MANDATORY_ACCESS** is a list defining the classification label for the created node (see Table IX, page 256).

**STATUS** is an established status indicator.

**MEDIUM_RESOURCE** is an open node handle identifying a MEDIUM node.

Exceptions:

**PATHNAME_SYNTAX_ERROR** is raised if the relationship key list given in PORT_KEY_-A or PORT_ KEY_B isy syntactically illegal [2], or if the pathname given in CHANNEL_-PARENT is syntactically illegal [1] (see Table I, page 52).

**CLIENT_ERROR** is raised if either of the client nodes identified by CLIENT_A or CLIENT_-B is not a FILE or PROCESS node [2], or if at least one of the client nodes identified by CLIENT_A or CLIENT_B is not a PROCESS node [4], or if either of the client nodes identified by CLIENT_A or CLIENT_B is a PROCESS node and the CURRENT_STATUS attribute of the PROCESS node indicates that the process has completed or terminated [3].

**NOT_IN_VIEW_ERROR** is raised if an attribute name specified in ATTRIBUTES_A or AT-TRIBUTES_B or a key name specified in PORT_KEY_A or PORT_KEY_B cannot be trans-lated [1] or if PORT_KEY_A or PORT_KEY_B consists of a single relationship key designa-tor and no key of the relation identified by PORT_RELATION_A or PORT_RELATION_B is visible, respectively [5].

**AMBIGUOUS_VIEW_ERROR** is raised if translation of an attribute name specified in ATTRIBUTES_A or ATTRIBUTES_B is ambiguous, if a key name specified in PORT_-KEY_A or PORT_KEY_B is ambiguous [1] or if PORT_KEY_A or PORT_KEY_B consists of a single relationship designator and more than one key of the relation identified by PORT_-RELATION_A or PORT_RELATION_B is visible, respectively [4].


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

**NAME_ERROR** is raised if the node handle PORT_RELATION_A or PORT_RELATION_B refers to an unobtainable or inaccessible node [1] or if the node identified by CHANNEL_-PARENT or any node traversed on that path is unobtainable or inaccessible [1].

**NODE_KIND_ERROR** is raised if the open node handle PORT_RELATION_A or PORT_-RELATION_B does not identify an RD [1] or if the RD identified by PORT_RELATION_A or PORT_RELATION_B is not an RD for a PORT relation [3].

**PORT_ERROR** is raised if an NKD for a CHANNEL node cannot be found that permits a node of that kind to be the target of relationships of both of the specified PORT relations [1], or if an attempt is made to create incompatible DIRECTION attributes on the respective PORT relationships [2].

**SYNTAX_ERROR** is raised if ATTRIBUTES_A or ATTRIBUTES_B is syntactically illegal [8], if the DISCRETIONARY_ACCESS parameter (see Section 5.4.4 on page 259) [10] or the MANDATORY_ACCESS parameter (see Table IX, page 256) [9] is syntactically illegal.

**RELATIONSHIP_ERROR** is raised if the combination of PORT_RELATION_A and PORT_-KEY_A or PORT_RELATION_B and PORT_KEY_B matches an existing PORT relationship emanating from the respective client node [3] or if any path element of CHANNEL_-PARENT identifies more than one relationship [1] or if the relationship specified by any path element of CHANNEL_PARENT does not exist [2] or if PORT_KEY_A or PORT_KEY_B specifies a value for the same key more than once [5].

**PREDEFINED_ATTRIBUTE_ERROR** is raised if an attribute name specified within the ATTRIBUTES_A or ATTRIBUTES_B parameter identifies a CAIS-controlled attribute other than DIRECTION, ALLOWED_PROTOCOLS or any of the predefine protocol parameter attributes. [1]

**ATTRIBUTE_ERROR** is raised if ATTRIBUTES_A or ATTRIBUTES_B specifies a value for the same attribute more than once [6].

CONFORMANCE_ERROR is raised if the operation would result in an attribute or relationship not conforming to its definition or the constraints in force on that definition [1, 3-5, 7-10].

**RESOURCE_ALLOCATION_ERROR** is raised if one of the client nodes is a DEVICE or GATEWAY node which is not allocatable, or if the medium resource identified through MEDIUM_RESOURCE is not allocatable [1].

**RESOURCE_AVAILABILITY_ERROR** is raised if one of the client nodes is a DEVICE or GATEWAY node which is not available, or if the medium resource identified through MEDIUM_RESOURCE is not available [1].

**STATUS_ERROR** is raised if the node handle CLIENT_A, PORT_RELATION_A, CLIENT_-B, PORT_RELATION_B, or MEDIUM_RESOURCE is not open [2], or if CHANNEL is an open node handle at the time of the call [1], or if STATUS is not an established status indicator [3].

**TRANSACTION_ERROR** is raised if the operation is included in more than one transaction [1] or if the operation is included in a transaction and the operation modifies a node which is not included in the same transaction [2].

**INTENT_VIOLATION** is raised if CLIENT_A or CLIENT_B was not opened with an intent establishing the right to append relationships [1].

**ACCESS_VIOLATION** is raised if the discretionary access control rights of the current process are insufficient to gain access with APPEND_RELATIONSHIPS intent to the node


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

specified by CHANNEL_PARENT or are insufficient to traverse the path specified by CHANNEL_PARENT or are insufficient to gain access with APPEND_CONTENTS intent to the MEDIUM node identified by MEDIUM_RESOURCE [1].

**LOCK_ERROR** is raised if the CREATE_CHANNEL operation is delayed beyond the specified TIME_LIMIT due to the existence of locks on nodes traversed by the path specified by CHANNEL_PARENT or locks in conflict with the implied intent APPEND_RELATIONSHIPS to the node specified by CHANNEL_PARENT [1]. LOCK_ERROR may be raised prior to the expiration of the timeout if the CAIS implementation can determine that a deadlock situation has occurred.

**SECURITY_VIOLATION** is raised if the operation represents a violation of mandatory access controls. SECURITY_VIOLATION is raised only if the conditions for other exceptions are not present.

### 7.3.9 Releasing a channel

```
procedure RELEASE_CHANNEL
    (CHANNEL          : in CHANNEL_NODE_TYPE;
     STATUS           : in STATUS_INDICATOR : = NO_INDICATION);
```

Purpose:

This procedure causes a delete request event to occur for the channel identified by CHANNEL. This will cause the PORT relationships of the channel to be deleted and the channel will enter the RETIRED state. If the channel is in the ESTABLISHED or CLOSED state at the time this interface is called, the interface may return before the channel reaches the RETIRED state; however, the CAIS implementation must ensure that the channel eventually reaches the RETIRED state after orderly shutdown is achieved. Releasing a channel in the RETIRED state has no effect.

Parameters:

**CHANNEL** is an open node handle to the CHANNEL node of the channel to be released.

**STATUS** is an established status indicator.

Exceptions:

**NODE_KIND_ERROR** is raised if the open node handle CHANNEL does not identify a CHANNEL node [1].

**STATUS_ERROR** is raised if the node handle CHANNEL is not open [2] or if STATUS is not an established status indicator [3].

**CLIENT_ERROR** is raised if the current PROCESS node is not a client node of the channel [1].

**CONFORMANCE_ERROR** is raised if the operation would result in a node not conforming to its definition or the constraints in force on that definition [3].


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

**TRANSACTION_ERROR** is raised if the operation is included in more than one transaction [1] or if the operation is included in a transaction and the operation modifies a node which is not included in the same transaction [2].

**SECURITY_VIOLATION** is raised if the operation represents a violation of mandatory access controls. SECURITY_VIOLATION is raised only if the conditions for other exceptions are not present.

Notes:

CHANNEL remains open so that it may be used in a subsequent DELETE_CHANNEL_NODE operation.

### 7.3.10 Deleting a CHANNEL node

```
procedure DELETE_CHANNEL_NODE
    (CHANNEL      : in out CHANNEL_NODE_TYPE;
     TIME_LIMIT   : in     CAIS_DURATION     := LONG_DELAY;
     STATUS       : in     STATUS_INDICATOR  := NO_INDICATION);
```

Purpose:

This procedure deletes the CHANNEL relationship to a CHANNEL node. The CHANNEL node becomes unobtainable and the node handle CHANNEL is closed. TIME_LIMIT specifies a time limit for the delay due to existing locks on the parent of the CHANNEL node. A delayed call completes after the node is unlocked or the specified time limit has elapsed. In the latter case, the exception LOCK_ERROR is raised.

Parameters:

**CHANNEL** is an open node handle to the CHANNEL node to be deleted.

**TIME_LIMIT** is the maximum time to wait for the unlocking of the parent of CHANNEL node in accordance with the intent WRITE_RELATIONSHIPS.

**STATUS** is an established status indicator.

Exceptions:

**NODE_KIND_ERROR** is raised if the open node handle CHANNEL does not identify a CHANNEL node [1].

**STATUS_ERROR** is raised if the node handle CHANNEL is not open [2], if STATUS is not an established status indicator [3], or if the channel is not in the RETIRED state [9].

**CONFORMANCE_ERROR** is raised if the operation would result in a node not conforming to its definition or the constraints in force on that definition [3].

**LOCK_ERROR** is raised if access, with intent WRITE_RELATIONSHIPS, to the parent of the CHANNEL node cannot be obtained within the specified TIME_LIMIT due to an existing lock on the node [1]. LOCK_ERROR may be raised prior to expiration of the time limit if the CAIS implementation can determine that a deadlock situation has occurred.


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

**TRANSACTION_ERROR** is raised if the operation is included in more than one transaction [1] or if the operation is included in a transaction and the operation modifies a node which is not included in the same transaction [2]. 9

**INTENT_VIOLATION** is raised if the CHANNEL was not opened with an intent including EXCLUSIVE_WRITE and READ_RELATIONSHIPS [1]. 10

**ACCESS_VIOLATION** is raised if the current process does not have sufficient discretionary access rights to obtain access to the parent of the node to be deleted with intent WRITE_-RELATIONSHIPS [1]. 11

**SECURITY_VIOLATION** is raised if the operation represents a violation of mandatory access controls. SECURITY_VIOLATION is raised only if the conditions for other exceptions are not present. 12

### 7.3.11 Waiting for notification of attempted opening

```
procedure AWAIT_CHANNEL
    (CHANNEL          : in out CHANNEL_NODE_TYPE;
     PORT_KEY         : in     RELATIONSHIP_KEY_PATTERN : = EMPTY_KEY_LIST;
     PORT_RELATION    : in     PORT_RD_TYPE;
     TIME_LIMIT       : in     CAIS_IO_DURATION          := LONG_IO_DELAY;
     STATUS           : in     STATUS_INDICATOR          := NO_INDICATION);

procedure AWAIT_CHANNEL
    (CHANNEL               : in out CHANNEL_NODE_TYPE;
     PORT_KEY              : in     RELATIONSHIP_KEY_PATTERN := EMPTY_KEY_LIST;
     PORT_RELATION_NAME    : in     RELATION_NAME_PATTERN    := "PORT";
     TIME_LIMIT            : in     CAIS_IO_DURATION         := LONG_IO_DELAY;
     STATUS                : in     STATUS_INDICATOR         := NO_INDICATION);
```

Purpose:

This procedure allows a task to await connections. The parameters PORT_KEY and PORT_-RELATION or PORT_RELATION_NAME identify the subset of channels over which connections are awaited. The channels may already exist or may subsequently be created. 1

When a call to AWAIT_CHANNEL completes without raising an exception, the node handle CHANNEL is opened with NO_ACCESS intent to the CHANNEL node for the channel selected by the call. The calling process may then issue a protocol open on the selected channel or delete the selected channel. The CHANNEL node identified by CHANNEL will be in the CONNECTED state and may or may not have existed at the time AWAIT_CHANNEL was called. 2

AWAIT_CHANNEL will return when both of the following conditions are met: 3

a. A channel exists in the CONNECTED state for which the current process is one of the clients;

b. The channel satisfies the selection criteria expressed in the parameters PORT_KEY and PORT_RELATION or PORT_RELATION_NAME;

and when one of the following conditions is met:


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

a. The opposite client node is a DEVICE or SECONDARY_STORAGE node and the channel has not been previously opened;

b. The opposite client node is a PROCESS or GATEWAY node and the opposite process has called either the AWAIT_CHANNEL or a protocol OPEN interface on the channel.

The semantics for the matching of formed PORT relationship identifications to the selection criteria expressed in the PORT_KEY, PORT_RELATION, and PORT_RELATION_NAME patterns are identical to those specified for node iterator creation in section 5.2.49 on page 169. However, the candidate relation names are evaluated at the time the interface is called. Subsequent ADOPT_VIEW or UNADOPT_VIEW operations have no effect on this list.

If a process has many pending AWAIT_CHANNEL calls which could be satisfied by a single event, then one of these pending calls will return. The CAIS does not specify which pending AWAIT_CHANNEL call will complete.

The last relationship traversed in opening the node handle CHANNEL is the local PORT relationship of the selected channel emanating from the PROCESS node of the current process. See subprograms PATH_KEY (Section 5.2.20 on page 119) and PATH_RELATION (Section 5.2.21 on page 120).

Parameters:

**CHANNEL** is an node handle, initially closed, to be opened to the CHANNEL node of the selected channel. This handle is opened with the intents READ_ATTRIBUTES and READ_-RELATIONSHIPS.

**PORT_KEY** is a relationship key list pattern specifying a pattern which the keys of the local PORT relationship of the selected channel must match.

**PORT_RELATION** is an open node handle to the RD for the PORT relationship or prospective PORT relationship emanating from the current PROCESS node and terminating at the node identified by CHANNEL.

**PORT_RELATION_NAME** is a pattern to match the relation name of the PORT relationship of the channel(s) of interest determined with respect to the composite view of the calling process.

**TIME_LIMIT** is a time limit for the delay on waiting for this operation to complete.

**STATUS** is an established status indicator.

Exceptions:

**SYNTAX_ERROR** is raised if PORT_KEY [3] or PORT_RELATION_NAME [2] is syntactically illegal.

**NOT_IN_VIEW_ERROR** is raised if the composite view of the current process at the time of the call does not define at least one name for a PORT relation matching the pattern specified by PORT_RELATION_NAME [4], if a key name in PORT_KEY cannot be translated in the context of the relation identified by PORT_RELATION or in the context of a PORT relation whose name matches PORT_RELATION_NAME [1], or if PORT_KEY consists of a pattern for a single relationship key descriptor and no key is visible in the context of the relation identified by PORT_RELATION or in the context of a PORT relation whose name matches PORT_RELATION_NAME [5].



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

**AMBIGUOUS_VIEW_ERROR** is raised if translation of a key name in PORT_KEY is ambiguous in the context of the relation identified by PORT_RELATION or in the context of a PORT relation whose name matches PORT_RELATION_NAME [1], or if PORT_KEY consists of a pattern for a single relationship key descriptor and more than one key is visible in the context of the relation identified by PORT_RELATION or in the context of a PORT relation whose name matches PORT_RELATION_NAME [4]. 15

**RELATIONSHIP_ERROR** is raised if PORT_KEY specifies a value for the same key more than once [5]. 16

**NODE_KIND_ERROR** is raised if the open node handle PORT_RELATION does not identify an RD [1] or if the RD identified by PORT_RELATION is not an RD for a PORT relation [3]. 17

**TIME_EXPIRED** is raised if the AWAIT_CHANNEL operation is delayed beyond the specified TIME_LIMIT [1]. 18

**STATUS_ERROR** is raised if the node handle CHANNEL is already open at the time of the call [1] or if the node handle PORT_RELATION is not open [2] or if STATUS is not an established status indicator [3]. 19

**SECURITY_VIOLATION** is raised if the operation represents a violation of mandatory access controls. SECURITY_VIOLATION is raised only if the conditions for other exceptions are not present. 20

### 7.3.12 Opening the opposite client node

```
procedure 0PEN_OPPOSITE_CLIENT
    (OPPOSITE_CLIENT :  in out CLIENT_NODE_TYPE;
     CHANNEL         :  in     CHANNEL_NODE_TYPE;
     INTENT          :  in     INTENT_ARRAY      : = (1 => NO_ACCESS) ;
     TIME_LIMIT      :  in     CAIS_DURATION     : = LONG_DELAY ;
     STATUS          :  in     STATUS_INDICATOR  : = NO_INDICATION);
```

Purpose:

Given a handle to a CHANNEL node, this procedure returns a node handle to the opposite client node. 1

The last relationship traversed in opening the node handle OPPOSITE_CLIENT is the CLIENT relationship from the node identified by CHANNEL to the opposite client node. See subprograms PATH_KEY (Section 5.2.20 on page 119) and PATH_RELATION (Section 5.2.21 on page 120). 2

Parameters:

**OPPOSITE_CLIENT** is a node handle, initially closed, to be opened to the opposite client node. 3

**CHANNEL** is an open node handle to the CHANNEL node of the channel for which the opposite client node is desired. 4

**INTENT** is the intent of subsequent operations on OPPOSITE_CLIENT. 5

**TIME_LIMIT** is a time limit for the delay on waiting for this operation to complete. 6

**STATUS** is an established status indicator. 7


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Exceptions:

CLIENT. ERROR is raised if the current PROCESS node is not a client node of the channel identified by CHANNEL [1]. 8

NODE_ KIND_ ERROR is raised if CHANNEL does not identify a CHANNEL node [1]. 9

STATUS_ ERROR is raised if CHANNEL is not open [2], if OPPOSITE--CLIENT is already open [1], if the channel is in the RETIRED state [9], or if STATUS is not an established status indicator [3]. 10

TRANSACTION. ERROR is raised if the operation is included in more than one transaction [1], 11

LOCK_ ERROR is raised if access, with the specified INTENT, to the opposite client node cannot be obtained within the specified TIME-LIMIT due to an existing lock on the node [1]. LOCK_ ERROR maybe raised prior to expiration of the time limit if the CAIS implementation can determine that a deadlock situation has occurred. 12

ACCESS-VIOLATION is raised if the discretionary access rights of the current process are insufficient to obtain access to opposite client node with the specified INTENT [1]. 13

SECURITY_VIOLATION is raised if the operation represents a violation of mandatory access controls. SECURITY-VIOLATION is raised only if the conditions for other exceptions are not present. 14

### 7.3.13 Checking PORT relationship kind compatibility

```
function COMPATIBLE_PORT-KINDS
    (PORT_RELATION_A : in      PORT_RD_TYPE;
     PORT-RELATION-B : in      PORT-RD-TYPE;
     STATUS          : in      STATUS_INDICATOR  := NO-INDICATION)
    return BOOLEAN;
```

Purpose:

This function provides a determination of whether two PORT RDs are compatible. The value TRUE is returned if the feasible CHANNEL NKD subset given the PORT RDs identified by the open node handles PORT-RELATION. A and PORT_RELATION_B is non-empty; otherwise, the value FALSE is returned. 1

Parameters:

PORT_RELATION_A is the node handle for an RD defining one of the PORT relations to be checked. 2

PORT-RELATION-B is the node handle for an RD defining the other PORT relation to be checked. 3

STATUS is an established status indicator. 4


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Exceptions:

**NAME_ERROR** is raised if the node handle PORT_RELATION_A or PORT_RELATION_B refers to an unobtainable or inaccessible node [1].

**NODE_KIND_ERROR** is raised if the open node handle PORT_RELATION_A or PORT_RELATION_B does not identify an RD [1] or if the RD identified by PORT_RELATION_A or PORT_RELATION_B is not an RD for a PORT relation [3].

**STATUS_ERROR** is raised if the node handle PORT_RELATION_A or PORT_RELATION_B is not open [2] or if STATUS is not an established status indicator [3].

**TRANSACTION_ERROR** is raised if the operation is included in more than one transaction [1].

item[SECURITY_VIOLATION] is raised if the operation represents a violation of mandatory access controls. SECURITY_VIOLATION is raised only if the conditions for other exceptions are not present.

### 7.3.14 Setting PORT relationship attributes

```
procedure SET_PORT_ATTRIBUTE
    (CLIENT          : in    CLIENT_NODE_TYPE;
     KEY             : in    RELATIONSHIP_KEY;
     PORT_RELATION   : in    PORT_RD_TYPE       := PORT_RD;
     ATTRIBUTE       : in    AD_TYPE;
     VALUE           : in    LIST_TYPE;
     STATUS          : in    STATUS_INDICATOR   := NO_INDICATION);
```

Purpose:

This procedure sets the value of an attribute on a PORT relationship to the value given by VALUE. The PORT relationship to be modified is the relationship of the relation identified by the open node handle PORT_RELATION with key list KEY which emanates from the node identified by the open node handle CLIENT. The attribute to be created or modified is identified by the open node handle ATTRIBUTE. If the identified attribute does not exist on the relationship, it is created. The attribute identified by ATTRIBUTE may be any PORT relationship attribute which is not CAIS-controlled, the predefine CAIS-controlled attributes DIRECTION and ALLOWED_PROTOCOLS, or any of the CAIS-controlled predefine protocol parameter attributes.

Parameters:

**CLIENT** is an open node handle identifying the client node from which the PORT relationship whose attribute is to be set emanates.

**KEY** is the key list identifying the PORT relationship to be modified.

**PORT_RELATION** is an open node handle identifying the RD for the PORT relationship.

**ATTRIBUTE** is an open node handle identifying the AD for the attribute to be set.

**VALUE** is the value for the attribute.

**STATUS** is an established status indicator.



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Exceptions:

**PATHNAME_SYNTAX_ERROR** is raised if the KEY given is syntactically illegal [2]. 8

**NOT_IN_VIEW_ERROR** is raised if a key name specified in KEY cannot be translated [1] or if KEY consists of a single relationship key designator and no key of the specified relation is visible [5]. 9

**AMBIGUOUS_VIEW_ERROR** is raised if a key name specified in KEY has an ambiguous translation [1] or if KEY consists of a single relationship key designator and more than one key of the specified relation is visible [4]. 10

**PREDEFINED_ATTRIBUTE_ERROR** is raised if ATTRIBUTE identifies either a CAIS-controlled attribute other than those enumerated in the Purpose section above [2]. 11

**RELATIONSHIP_ERROR** is raised if the specified PORT relationship does not exist [2] or if KEY specifies a value for the same key more than once [5]. 12

**USE_ERROR** is raised if the protocol compatibility checking has been initiated by one or both clients of the channel and this checking is still in progress [6]. 13

**STATUS_ERROR** is raised if CLIENT, PORT_RELATION, or ATTRIBUTE is not an open node handle [2], or if the channel identified by the CLIENT, KEY and PORT_RELATION parameters is not in the CONNECTED state [9] or if STATUS is not an established status indicator [3]. 14

**NODE_KIND_ERROR** is raised if the open node handle PORT_RELATION does not identify an RD [1] or if the RD identified by PORT_RELATION is not an RD for a PORT relationship [3] or if the open node handle ATTRIBUTE does not identify an AD [1]. 15

**CONFORMANCE_ERROR** is raised if the operation would result in an attribute or relationship not conforming to its definition or the constraints in force on that definition [4, 5, 7-10]. 16

**INTENT_VIOLATION** is raised if the node handle CLIENT was not opened with an intent establishing the right to write relationships [1]. 17

**ACCESS_VIOLATION** is raised if the current process is not a client of the channel and the discretionary access control rights of the process are insufficient to obtain access to the CHANNEL node which is the target of the identified PORT relationship with intent READ_ATTRIBUTES [1]. 18

**SECURITY_VIOLATION** is raised if the operation represents a violation of mandatory access controls. SECURITY_VIOLATION is raised only if the conditions for other exceptions are not present. 19

Additional Interfaces:

```
procedure SET_PORT_ATTRIBUTE
    (CLIENT        : in    CLIENT_NODE_TYPE;
     KEY           : in    RELATIONSHIP_KEY;
     PORT_RELATION : in    RELATION_NAME      := "PORT";
     ATTRIBUTE     : in    ATTRIBUTE_NAME;
     VALUE         : in    LIST_TYPE;
     STATUS        : in    STATUS_INDICATOR   := NO_INDICATION)
is
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
        RELATION_DEFINITION : PORT_RD_TYPE;
        ATTRIBUTE_DEFINITION : AD_TYPE;
    begin
        DEFINITION_OF_RELATION (RELATION_DEFINITION, CLIENT,
                                PORT_RELATION, STATUS);
        DEFINITION_OF_PATH_ATTRIBUTE (ATTRIBUTE_DEFINITION,
                                      RELATION_DEFINITION,
                                      ATTRIBUTE, STATUS);
        SET_PORT_ATTRIBUTE (CLIENT, KEY, RELATION_DEFINITION,
                            ATTRIBUTE_DEFINITION, VALUE, STATUS);
        CLOSE (RELATION_DEFINITION);
        CLOSE (ATTRIBUTE_DEFINITION);
    exception
        when others =>
            CLOSE (RELATION_DEFINITION);
            CLOSE (ATTRIBUTE_DEFINITION);
            raise;
    end SET_PORT_ATTRIBUTE;
```


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

# 7.4 Package CAIS_IMPORT_EXPORT

A particular CAIS implementation is allowed to maintain files separately from files maintained by the host file system. This package provides the capability to transfer files between a CAIS instance and its host system. The exceptions raised by all subprograms in this package are defied in the package CAIS_DEFINITIONS. 1

## 7.4.1 Importing a file from the host file system

```
procedure IMPORT_CONTENTS
    (FROM            : in  STRING;
     TO              : in  SECONDARY_STORAGE_NODE_TYPE;
     CHARACTERISTICS : in  LIST_TYPE            := EMPTY_LIST;
     STATUS          : in  STATUS_INDICATOR     := NO_INDICATION);
```

Purpose:

This procedure copies the file identified by FROM in the host file system into the contents of the file node identified by TO. The contents of the file FROM in the host file system replace the contents of the file node TO. The CHARACTERISTICS parameter provides implementation-dependent information for copying the file node contents. The CURRENT_FILE_SIZE attribute on the node identified by TO is set to the size of the node's new contents. 1

Parameters:

**FROM** is the name of the host file to be imported. 2

**TO** is an open node handle to the node receiving the contents. 3

**CHARACTERISTICS** is a list of implementation-dependent information to be used for copying the host file. 4

**STATUS** is an established status indicator. 5

Exceptions:

**STATUS_ERROR** is raised if TO is not an open node handle [2] or if STATUS is not an established status indicator [3]. 6

**NODE_KIND_ERROR** is raised if the node identified by TO is not a SECONDARY_STORAGE node [1]. 7

**USE_ERROR** is raised if there is no underlying host system [45] or if the parameters FROM [46] or CHARACTERISTICS [47] have (implementation-dependent) values that do not permit copying the host file or if there is one or more open file handles associated with the open node handle TO [9]. 8

**INTENT_VIOLATION** is raised if TO was not opened with an intent including EXCLUSIVE_WRITE_CONTENTS [1]. 9

**CAPACITY_ERROR** is raised if the size of the host file to be copied into the contents of the node identified by TO exceeds the maximum size established for the node's contents as indicated by the node's MAXIMUM_FILE_SIZE attribute [2]. 10

**SECURITY_VIOLATION** is raised if the operation represents a violation of mandatory access controls and the conditions for other exceptions are not present. 11



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

### 7.4.2 Exporting a file to the host file system

```
procedure EXPORT_CONTENTS
    (FROM            : in  SECONDARY_STORAGE_NODE_TYPE;
     TO              : in  STRING;
     CHARACTERISTICS : in  LIST_TYPE           := EMPTY_LIST;
     STATUS          : in  STATUS_INDICATOR    := NO_INDICATION);
```

Purpose:

This procedure copies the contents of the file node identified by FROM to the file identified by TO in the host file system. The CHARACTERISTICS parameter provides implementation-dependent information for copying the file node contents. 1

Parameters:

**FROM** is an open node handle to the node whose contents are to reexported. 2

**TO** is the name of the host file receiving the contents. 3

**CHARACTERISTICS** is a list of implementation-dependent information to be used for copying the contents. 4

**STATUS** is an established status indicator. 5

Exceptions:

**STATUS_ERROR** is raised if FROM is not an open node handle [2] or if STATUS is not an established status indicator [3]. 6

**NODE_KIND_ERROR** is raised if the node identified by FROM is not a SECONDARY_STORAGE node [1]. 7

**USE_ERROR** is raised if there is no underlying host system [45] or if the parameters TO [46] or CHARACTERISTICS [47] have (implementation-dependent) values that do not permit copying the contents or if the structure of the contents of the node identified by FROM cannot be determined by the implementation given the manner in which node was crested [36]. 8

**INTENT_VIOLATION** is raised if FROM was not opened with an intent establishing the right to read attributes and contents [1]. 9

**SECURITY_VIOLATION** is raised if the operation represents a violation of mandatory access controls and the conditions for other exceptions are not present. 10



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

# 7.5 Package CAIS_COMMON_IO

This package defines services necessary to describe a node set, to create a corresponding external node set, and to introduce an external node set into an importing CAIS instance. These concepts are discussed in Section 4.5.5 on page 69. 1

Also provided are special node creation interfaces that allow the specification of the unique identifier for the nodes to be created. These interfaces aid in incorporating introduced information into an importing CAIS instance. They parallel the ordinary node creation interfaces but include parameters identifying introduced nodes. The IMPORTED_UID attribute of these introduced nodes will be used as UIDs for the nodes created via these special interfaces. 2

## 7.5.1 Types and exceptions

```
type NODE_SET_TYPE is limited private;
```

The type NODE_SET_TYPE is an Ada type which must be implemented to carry the semantics of node sets as defined by the interfaces in this package. An object of type NODE_SET_TYPE is termed a *node set handle* and identifies a node set. A node set handle has two states: open and closed. Each newly-declared node set handle is initially closed but may be opened explicitly. Nodes may be included only within node sets identified by open node set handles. An open node set holds a READ or EXCLUSIVE_READ lock on each included node. 1

```
UID_ERROR : exception;
```

The exception UID_ERROR is raised when a node being created is to be given a UID which is not unique within the CAIS instance. 2

All other exceptions raised by the subprograms in CAIS_COMMON_IO are defined in the packages CAIS_DEFINITIONS and CAIS_IO_DEFINITIONS. 3

## 7.5.2 Opening the predefine INTRODUCED_NODE NKD

```
function INTRODUCED_NODE_NKD
    (STATUS : in STATUS_INDICATOR := NO_INDICATION)
    return NKD_TYPE;
```

Purpose:

This function returns an open node handle with NO_ACCESS intent to the predefine NKD for INTRODUCED_NODE nodes. 1

The node kind definition name INTRODUCED_NODE and the relation NODE_KIND_DEFINITION are associated with the node handle returned as the key and relation used in opening the handle. See subprograms PATH_KEY (Section 5.2.20 on page 119) and PATH_RELATION (Section 5.2.21 on page 120). 2



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Parameters:

**STATUS** is an established status indicator. 3

Exceptions:

**STATUS_ERROR** is raised if STATUS is not an established status indicator [3]. 4

### 7.5.3 Opening the predefine INTRODUCED_NODE_RD

```
function INTRODUCED_NODE_RD
    (STATUS : in STATUS_INDICATOR := NO_INDICATION)
    return RD_TYPE;
```

Purpose:

This function returns an open node handle with NO_ACCESS intent to the predefine RD for INTRODUCED_NODE relationships. 1

The relation definition name INTRODUCED_NODE and the relation RELATION_DEFINITION are associated with the node handle returned as the key and relation used in opening the handle. See subprograms PATH_KEY (Section 5.2.20 on page 119) and PATH_RELATION (Section 5.2.21 on page 120). 2

Parameters:

**STATUS** is an established status indicator. 3

Exceptions:

**STATUS_ERROR** is raised if STATUS is not an established status indicator [3]. 4

### 7.5.4 Opening a node set handle

```
procedure OPEN_NODE_SET
    (NODE_SET : in out NODE_SET_TYPE;
     STATUS   : in     STATUS_INDICATOR := NO_INDICATION);
```

Purpose:

This procedure opens a node set handle to an empty node set. Elements can be added to this set through INCLUDE_NODE, INCLUDE_TREE, and INCLUDE_STRUCTURE operations. 1



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Parameters:

**NODE_SET** is a node set handle, initially closed, to be opened. 2

**STATUS** is an established status indicator. 3

Exceptions:

**STATUS_ERROR** is raised if NODE_SET is an open node set handle [12] or if STATUS is not an established status indicator [3]. 4

### 7.5.5 Including a node in a node set

```
procedure INCLUDE_NODE
   (MODE_SET                    : in out DIODE_SET_TYPE;
    If ODE                      : in     BODE_TYPE;
    INCLUDE_DEFINITION_NODES : in     BOOLEAN          : = TRUE;
    EXCLUSIVE_READ_INTENT    : in     BOOLEAN          : = TRUE;
    TIME_LIMIT               : in     CAIS_DURATION    : = LONG_DELAY;
    STATUS                   : in     STATUS_ INDICATOR := NO_INDICATION);
```

Purpose:

This procedure explicitly includes the node identified by NODE in the node set identified by NODE_SET. If INCLUDE_DEFINITION_NODES is TRUE, then the definition nodes that define the node identified by NODE are also included. Each included node is implicitly opened with the intent EXCLUSIVE_READ, if the parameter EXCLUSIVE_READ_INTENT is TRUE, and the intent READ otherwise. The locks established are held by the node set until the node set is closed. TIME_LIMIT specifies a time limit for the delay imposed by existing locks on included nodes. 1

Parameters:

**NODE_SET** is an open node set handle identifying the node set within which a node is to be included. 2

**NODE** is an open node handle identifying a node to be included in the node set. 3

**INCLUDE_DEFINITION_NODES** determines whether definition nodes are added to the node set. If set to TRUE, the included nodes are the NKD of each included node, the RD of each relation of an included NKD, the generalizations of each included NKD and RD, the AD of each node attribute of each included NKD and of each relationship attribute of each included RD, the AKDs referenced by these ADs and the additional AKDs which define included AKDs. This rule is applied recursively; i.e., for each node so included, the definition nodes for that node are also included. 4

**EXCLUSIVE_READ_INTENT** determines the type of lock acquired by the node set on the included nodes. When TRUE, included nodes are opened with the intent EXCLUSIVE_-READ, otherwise they are opened with the intent READ. 5

**TIME_LIMIT** is the maximum time to wait for the unlocking of the nodes to be included in accordance with the specified intent. 6

**STATUS** is an established status indicator. 7



Source: https://assist.dla.mil -- Downloaded: 202610-26T07:54Z
Check the source to verify that this is the current version before use.

Exceptions:

**STATUS_ERROR** is raised if NODE_SET is not an open node set handle [13], if NODE is not an open node handle [2], or if STATUS is not an established status indicator [3].

**NAME_ERROR** is raised if any node to be included is unobtainable or inaccessible [1].

**TRANSACTION_ERROR** is raised if the operation is included in more than one transaction due to the inclusion of a node, already in the node set, and the node identified by NODE in different transactions [1].

**LOCK_ERROR** is raised if access, with the specified intent (READ or EXCLUSIVE_READ), to all nodes to be included cannot be obtained within the specified TIME_LIMIT due to existing locks.

**INTENT_VIOLATION** is raised if NODE was not opened with an intent including READ [1].

**ACCESS_VIOLATION** is raised if the discretionary access control rights of the current process are insufficient to allow access to any node to be included with the intent READ [1]. ACCESS_VIOLATION is only raised if the conditions for NAME_ERROR are not present.

**SECURITY_VIOLATION** is raised if an attempt by the current process to obtain access to an included node with READ intent represents a violation of mandatory access controls. SECURITY_VIOLATION is raised only if the conditions for other exceptions are not present.

Notes:

A node cannot be deleted while included within an open node set since included nodes are opened with at least READ intent.

### 7.5.6 Including a tree in a node set

```
procedure INCLUDE_TREE
    (NODE_SET                    : in out NODE_SET_TYPE;
     NODE                        : in     NODE_TYPE;
     INCLUDE_DEFINITION_NODES    : in     BOOLEAN          : = TRUE;
     EXCLUSIVE_READ_INTENT       : in     BOOLEAN          : = TRUE;
     TIME_LIMIT                  : in     CAIS_DURATION    : = LONG_DELAY;
     STATUS                      : in     STATUS_INDICATOR : = NO_INDICATION);
```

Purpose:

This procedure effectively performs the INCLUDE_NODE operation on the node identified by the open node handle NODE and on all nodes whose unique primary path traverses the node identified by NODE. If INCLUDE_DEFINITION_NODES is TRUE, then the definition nodes that define the included nodes are also included. Each included node is implicitly opened with the intent EXCLUSIVE_READ if the parameter EXCLUSIVE_READ_INTENT is TRUE and with the intent READ otherwise. The locks established are held by the node set until the node set is closed. TIME_LIMIT specifies a time limit for the delay imposed by existing locks on included nodes.



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Parameters:

**NODE_SET** is an open node set handle identifying the node set within which the nodes are to be included.

**NODE** is an open node handle identifying the root node of the tree to be included in the node set.

**INCLUDE_DEFINITION_NODES** determines whether definition nodes are added to the node set. If set to TRUE, the included nodes are the NKD of each included node, the RD of each relation of an included NKD, the generalizations of each included NKD and RD, the AD of each node attribute of each included NKD and of each relationship attribute of each included RD, the AKDs referenced by these ADs and the additional AKDs which define included AKDs. This rule is applied recursively; i.e., for each node so included, the definition nodes for that node are also included.

**EXCLUSIVE_READ_INTENT** determines the type of lock acquired by the node set on the included nodes. When TRUE, included nodes are opened with the intent EXCLUSIVE_-READ, otherwise they are opened with the intent READ.

**TIME_LIMIT** is the maximum time to wait for the unlocking of the nodes to be included in accordance with the specified intent.

**STATUS** is an established status indicator.

Exceptions:

**STATUS_ERROR** is raised if NODE_SET is not an open node set handle [13], if NODE is not an open node handle [2], or if STATUS is not an established status indicator [3].

**NAME_ERROR** is raised if any node to be included is unobtainable or inaccessible [1].

**TRANSACTION_ERROR** is raised if the operation is included in more than one transaction due to the inclusion of nodes in the node set in different transactions [1].

**LOCK_ERROR** is raised if access, with the specified intent (READ or EXCLUSIVE_READ), to all nodes to be included cannot be obtained within the specified TIME_LIMIT due to existing locks.

**INTENT_VIOLATION** is raised if NODE was not opened with an intent including READ [1].

**ACCESS_VIOLATION** is raised if the discretionary access control rights of the current process are insufficient to allow access to any node to be included with the intent READ [1]. ACCESS_VIOLATION is only raised if the conditions for NAME_ERROR are not present.

**SECURITY_VIOLATION** is raised if an attempt by the current process to obtain access to an included node with READ intent represents a violation of mandatory access controls. SECURITY_VIOLATION is raised only if the conditions for other exceptions are not present.

Notes:

A node cannot be deleted while included within an open node set since included nodes are opened with at least READ intent.



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

### 7.5.7 Including a structure of nodes in a node set

```
procedure INCLUDE_STRUCTURE
    (NODE_SET                   : in out NODE_SET_TYPE;
     NODE                       : in     NODE_TYPE;
     RELATIONS                  : in     NODE_HANDLE_ARRAY;
     INCLUDE_DEFINITION_NODES   : in     BOOLEAN          := TRUE;
     EXCLUSIVE_READ_INTENT      : in     BOOLEAN          := TRUE;
     TIME_LIMIT                 : in     CAIS_DURATION    := LONG_DELAY;
     STATUS                     : in     STATUS_INDICATOR := NO_INDICATION);
```

Purpose:

This procedure effectively performs the INCLUDE_NODE operation on the node identified by the open node handle NODE and on all nodes which can be reached beginning at that node by following relationships of the relations identified by open node handles in the array RELATIONS. If INCLUDE_DEFINITION_NODES is TRUE, then the definition nodes that define the included nodes are also included. Each included node is implicitly opened with the intent EXCLUSIVE_-READ if the parameter EXCLUSIVE_READ_INTENT is TRUE and the intent READ otherwise. The locks established are held by the node set until the node set is closed. TIME_LIMIT specifies a time limit for the delay imposed by existing locks on included nodes.

Parameters:

**NODE_SET** is an open node set handle identifying the node set within which the nodes are to be included.

**NODE** is an open node handle identifying the root node of the structure to be included in the node set.

**RELATIONS** is an array of open node handles identifying the RDs of the relations whose relationships are to be followed to determine which nodes are included in the structure.

**INCLUDE_DEFINITION_NODES** determines whether definition nodes are added to the node set. If set to TRUE, the included nodes are the NKD of each included node, the RD of each relation of an included NKD, the generalizations of each included NKD and RD, the AD of each node attribute of each included NKD and of each relationship attribute of each included RD, the AKDs referenced by these ADs and the additional AKDs which define included AKDs. This rule is applied recursively; i.e., for each node so included, the definition nodes for that node are also included.

**EXCLUSIVE_READ_INTENT** determines the type of lock acquired by the node set on the included nodes. When TRUE, included nodes are opened with the intent EXCLUSIVE_-READ, otherwise they are opened with the intent READ.

**TIME_LIMIT** is the maximum time to wait for the unlocking of the nodes to be included in accordance with the specified intent.

**STATUS** is an established status indicator.



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Exceptions:

**STATUS_ERROR** is raised if NODE_SET is not an open node set handle [13], if NODE is not an open node handle [2], if any node handle in the array RELATIONS is not open [2], or if STATUS is not an established status indicator [3]. 9

**NAME_ERROR** is raised if any node to be included is unobtainable or inaccessible [1]. 10

**TRANSACTION_ERROR** is raised if the operation is included in more than one transaction due to the inclusion of nodes in the node set in different transactions [1]. 11

**LOCK_ERROR** is raised if access, with the specified intent (READ or EXCLUSIVE_READ), to all nodes to be included cannot be obtained within the specified TIME_LIMIT due to existing locks. 12

**INTENT_VIOLATION** is raised if NODE was not opened with an intent including READ [1]. 13

**ACCESS_VIOLATION** is raised if the discretionary access control rights of the current process are insufficient to allow access to any node to be included with the intent READ [1]. ACCESS_VIOLATION is only raised if the conditions for NAME_ERROR are not present. 14

**SECURITY_VIOLATION** is raised if an attempt by the current process to obtain access to an included node with READ intent represents a violation of mandatory access controls. SECURITY_VIOLATION is raised only if the conditions for other exceptions are not present. 15

Notes:

A node cannot be deleted while included within an open node set since included nodes are opened with at least READ intent. 16

### 7.5.8 Transmitting a node set in common external form

```
procedure TRANSMIT
   (FILE       : in    FILE_TYPE;
    NODE_SET   : in    NODE_SET_TYPE;
    STATUS     : in    STATUS_INDICATOR  : = NO_INDICATION);
```

Purpose:

This procedure writes out the node set identified by NODE_SET in the common external form using the CAIS_TEXT_IO file handle FILE. FILE must be open with mode OUT_FILE. 1

The line length and page length of FILE must be unbounded. However, an implementation is allowed to generate line terminators between lexical elements in the header section and in the content descriptors. There is no required ordering of nodes transmitted in the header section or of the node contents transmitted in the content section. 2



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Parameters:

**FILE** is an open CAIS_TEXT_IO file handle to be used for transmission of the node set.

**NODE_SET** is an open node set handle identifying the node set to be transmitted.

**STATUS** is an established status indicator.

Exceptions:

**STATUS_ERROR** is raised if FILE is not an open file handle [8], if NODE_SET is not an open node set handle [13], or if STATUS is not an established status indicator [3].

**MODE_ERROR** is raised if the mode of FILE is not OUT_FILE [1].

**LAYOUT_ERROR** is raised if the line length or page length for the file handle FILE is bounded [4].

**TRANSACTION_ERROR** is raised if the operation is included in more than one transaction (due to the inclusion of nodes in the node set and the opposite client node in different transactions) [1], if the operation is included in a transaction and the opposite client node is not included in the same transaction [2], or if the operation is included in a transaction, the opposite client is a SECONDARY_STORAGE node and the file handle FILE was not opened inside the transaction [3].

**SECURITY_VIOLATION** is raised if the operation represents a violation of mandatory access controls. SECURITY_VIOLATION is raised only if the conditions for other exceptions are not present.

Notes:

The information transmitted for each node depends on the state of the node at the time of transmission, not at the time of the INCLUDE operation. Also, the TRANSMIT operation is not atomic in that nodes included in a node set can be changed by other processes during transmission. Nodes can be locked with the intent EXCLUSIVE_READ when included in a node set to limit changes between the time of inclusion and the time of transmission.

### 7.5.9 Closing a node set

```
procedure CLOSE_NODE_SET
    (NODE_SET : in out NODE_SET_TYPE;
     STATUS   : in     STATUS_INDICATOR := NO_INDICATION);
```

Purpose:

This procedure closes the node set handle NODE_SET. All locks on included nodes held by the node set identified are released. Closing a node set handle that is already closed has no effect.



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Parameters:

**NODE_SET** is a node set handle identifying the node set to be closed.

**STATUS** is an established status indicator.

Exceptions:

**STATUS_ERROR** is raised if STATUS is not an established status indicator [3].

Notes:

A node cannot be deleted while included within a node set. Closing a node set may allow other operations on included nodes to complete.

If a process terminates without closing its node set handles explicitly, they are implicitly closed.

### 7.5.10 Introducing nodes in common external form

```
procedure INTRODUCE_NODES
    (FILE                  : in  FILE_TYPE;
     BASE                  : in  NODE_TYPE;
     KEY                   : in  RELATIONSHIP_KEY          := EMPTY_KEY_LIST;
     RELATION              : in  RD_TYPE                   := INTRODUCED_NODE_RD;
     UNIQUE_IDENTIFIERS    : in  UNIQUE_IDENTIFIER_LIST    := EMPTY_LIST;
     INCLUDE_CONTENTS      : in  BOOLEAN                   := TRUE;
     ATTRIBUTES            : in  ATTRIBUTE_LIST            := EMPTY_LIST;
     DISCRETIONARY_ACCESS  : in  DISCRETIONARY_ACCESS_LIST := ALL_RIGHTS;
     MANDATORY_ACCESS      : in  MANDATORY_ACCESS_LIST     := EMPTY_LIST;
     KIND                  : in  NODE_KIND_NODE_TYPE       := INTRODUCED_NODE_NKD;
     STATUS                : in  STATUS_INDICATOR          := NO_INDICATION;
     MEDIUM_RESOURCE       : in  MEDIUM_NODE_TYPE          := DEFAUL_MEDIUM);
```

Purpose:

This procedure reads a CAIS_TEXT_IO stream, identified by the file handle FILE, that contains an external node set (a CEF stream) and creates a set of introduced nodes representing nodes described in the external node set. FILE must have the mode IN_FILE.

The primary relationships to the newly created introduced nodes are instances of the RD identified by RELATION and emanate from the node identified by BASE. The relationships are created using the relationship key list specified by KEY. The nodes are created on the medium resource identified by MEDIUM_RESOURCE.

If the parameter UNIQUE_IDENTIFIERS is the empty list, all nodes in the CEF stream are introduced. Otherwise, UNIQUE_IDENTIFIERS must be a list of string items in which each string item is the unique identifier of a node in the CEF stream. In this case, only the identified nodes are introduced.

If the Boolean parameter INCLUDE_CONTENTS is TRUE, the contents of introduced nodes



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

representing SECONDARY_STORAGE nodes are also introduced. Otherwise, the introduced nodes have no contents.

The DISCRETIONARY_ACCESS parameter specifies the initial access control information to be established between the created nodes and the default GROUP node of the current user (see Section 5.4 on page 255).

The MANDATORY_ACCESS parameter specifies the object classification labels with which the nodes are to be created. If its value is the empty list, the nodes inherit the subject classification of the creating process as its object classification. Otherwise, it must be an unnamed list consisting of an identifier item and, optionally, an unnamed list of identifier items (see Table IX, page 256).

KIND is an open node handle identifying the node kind for the new nodes. It must identify an INTRODUCED_NODE node kind. It may specify either an NKD or an NKI. If KIND identifies an NKI, the NKI is made the current adopted interpretation for the interpretation of names in the parameter ATTRIBUTES.

FILE must be positioned at the first character of the external node set (the header section), otherwise SYNTAX_ERROR is raised. If INCLUDE_CONTENTS is FALSE, reading of the CEF stream stops before reading any content section. In this case, the procedure INTRODUCE_-CONTENTS may be subsequently called to selectively introduce the contents of nodes introduced by the call to INTRODUCE_NODES, without performing any other operations on FILE. If INCLUDE_CONTENTS is TRUE or an exception is raised, the position of FILE in the CEF stream is implementation-defined after the call.

Parameters:

**FILE** is an open file handle to the CEF stream (opened by the CAIS_TEXT_IO package).

**BASE** is an open node handle to the parent node from which the primary relationships to the newly created nodes are to emanate.

**KEY** is the relationship key list of the primary relationships to be created.

**RELATION** is an open node handle to the relation definition of the primary relationships to be created.

**UNIQUE_IDENTIFIERS** determines which nodes in the CEF stream are to be introduced. All are introduced if the value is the empty list. Otherwise, it is a list of the unique identifiers of the nodes to be introduced.

**INCLUDE_CONTENTS** determines whether the contents of the introduced nodes are introduced.

**ATTRIBUTES** is an empty or named list (see Section 9.1 on page 751) whose elements are used to establish initial values for attributes of the newly created nodes; each named item of the list specifies an attribute name and the value to be given to that attribute.

**DISCRETIONARY_ACCESS** is the initial access control information associated with the created nodes; it is the value of the GRANT attribute of the ACCESS relationships to the user's default GROUP node (see Section 5.4.4 on page 259).

**MANDATORY_ACCESS** is a list defining the classification label for the created nodes (see Table IX, page 256).

**KIND** is an open node handle identifying the node kind for the introduced nodes. It may identify either an NKI or an NKD.


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

**STATUS** is an established status indicator.

**MEDIUM_RESOURCE** is an open node handle identifying a MEDIUM node.

Exceptions:

**NAME_ERROR** is raised if there is no node included in the CEF stream identified by FILE with one of the unique identifiers specified in the list UNIQUE_IDENTIFIERS [4] or if the parent of the NKI identified by KIND is inaccessible [1].

**STATUS_ERROR** is raised if BASE, RELATION, KIND or MEDIUM_RESOURCE is not an open node handle [2], if FILE is not an open file handle [8] or if STATUS is not an established status indicator [3].

**PATHNAME_SYNTAX_ERROR** is raised if the node identification given by KEY is syntactically illegal (see Table I, page 52) [2].

**ITEM_KIND_ERROR** is raised if the list parameter UNIQUE_IDENTIFIERS contains any item which is not a string [1].

**SYNTAX_ERROR** is raised if the stream identified by FILE does not conform to the syntax of an external node set [15] (see Table II, page 73), or if a unique identifier in the list parameter UNIQUE_IDENTIFIERS (see Section 4.3.10 on page 54) [1], the ATTRIBUTES parameter (see description above) [8], the DISCRETIONARY_ACCESS parameter (see Section 5.4.4 on page 259) [10] or the MANDATORY_ACCESS parameter (see Table IX, page 256) [9] is syntactically illegal.

**EXISTING_NODE_ERROR** is raised if a node already exists with the identification derived from the parameters BASE, KEY and RELATION [1].

**NOT_IN_VIEW_ERROR** is raised if a key name specified in KEY or an attribute name specified in ATTRIBUTES cannot be translated [1] or if KEY consists of a single relationship key designator and no key of the relation identified by RELATION is visible [5].

**AMBIGUOUS_VIEW_ERROR** is raised if translation of a key name specified in KEY or an attribute name specified in ATTRIBUTES is ambiguous [1] or if KEY consists of a single relationship key designator and more than one key of the relation identified by RELATION is visible [4].

**PREDEFINED_RELATION_ERROR** is raised if RELATION identifies a relation that is CAIS-controlled [1].

**RELATIONSHIP_ERROR** is raised if KEY specifies a value for the same key more than once [5] or if RELATION or its corresponding reverse relation has no unique key set [6].

**PREDEFINED_ATTRIBUTE_ERROR** is raised if an attribute name specified within the ATTRIBUTES parameter identifies an attribute that is CAIS-controlled [1].

**ATTRIBUTE_ERROR** is raised if ATTRIBUTES specifies a value for the same attribute more than once [6].

**USE_ERROR** is raised if the value for the MANDATORY_ACCESS parameter [10] is semantically illegal.

**MODE_ERROR** is raised if the mode of FILE is not IN_FILE [1].

**LAYOUT_ERROR** is raised if the line or page length for the file handle FILE is bounded [4].



Source: https://assist.dla.mil -- Downloaded: 202610-26T07:54Z
Check the source to verify that this is the current version before use.

**NODE_KIND_ERROR** is raised if the open node handle RELATION does not identify an RD [1], if the open node handle KIND does not identify the predefine INTRODUCED_NODE node kind or a specialization of that node kind [2] or if the open node handle MEDIUM_-RESOURCE does not identify a MEDIUM node [1]. 36

**CONFORMANCE_ERROR** is raised if the operation would result in an attribute, node or relationship not conforming to its definition or the constraints in force on that definition [1, 2, 3, 4, 7, 8, 9, 10, 11]. 37

**RESOURCE_ALLOCATION_ERROR** is raised if the medium resource identified by MEDIUM_RESOURCE is not allocatable [I]. 38

**RESOURCE_AVAILABILITY_ERROR** is raised if the medium resource identified by MEDIUM_RESOURCE is not available [1]. 39

**TRANSACTION_ERROR** is raised if the operation is included in more than one transaction [1] or if the operation is included in a transaction and the operation modifies a node which is not included in the same transaction [2], or if the operation is included in a transaction, the opposite client is a SECONDARY_STORAGE node and the file handle FILE was not opened inside the transaction [3]. 40

**LOCK_ERROR** is raised if a node handle to the parent of the NKI specified by KIND cannot be opened with intent READ_RELATIONSHIPS due to outstanding locks in conflict with that intent [1]. 41

**INTENT_VIOLATION** is raised if BASE was not opened establishing the right to create relationships. 42

**ACCESS_VIOLATION** is raised if the discretionary access control rights of the current process are insufficient to gain access with CREATE_INSTANCES intent to the NKD identified through KIND or with READ_RELATIONSHIPS intent to the parent of the NKI specified by KIND or with APPEND_CONTENTS intent to the MEDIUM node identified by MEDIUM_RESOURCE [1]. 43

**SECURITY_VIOLATION** is raised if the operation represents a violation of mandatory access controls. SECURITY_VIOLATION is raised only if the conditions for other exceptions are not present. 44

Additional Interfaces:

```
procedure INTRODUCE_NODES
    (FILE                  : in  FILE_TYPE;
     BASE                  : in  NODE_TYPE;
     KEY                   : in  RELATIONSHIP_KEY           : = EMPTY_KEY_LIST;
     RELATION              : in  RELATION_NAME;
     INIQUE_IDENTIFIERS    : in  UNIQUE_IDENTIFIER_LIST : = EMPTY_LIST;
     INCLUDE_CONTENTS      : in  BOOLEAN                    : = TRUE;
     ATTRIBUTES            : in  ATTRIBUTE_LIST             : = EMPTY_LIST;
     DISCRETIONARY_ACCESS : in  DISCRETIONARY_ACCESS_LIST : = ALL_RIGHTS;
     MANDATORY_ACCESS      : in  MANDATORY_ACCESS_LIST : = EMPTY_LIST;
     KIND                  : in  NODE_KIND_NAME             : = “INTRODUCED_NODE”;
     STATUS                : in  STATUS_INDICATOR           : = NO_ INDICATION;
     MEDIUM_RESOURCE       : in  MEDIUM_NODE_TYPE           : = DEFAULT_MEDIUM)
i s
```



Source: https://assist.dla.mil -- Downloaded: 202610-26T07:54Z
Check the source to verify that this is the current version before use.

```
        RELATION_DEFINITION : RD_TYPE;
        NODE_KIND           : NKI_TYPE;
    begin
        DEFINITION_OF_RELATION (RELATION_DEFINITION, BASE, RELATION, STATUS);
        OPEN_NODE_KIND (NODE_KIND, KIND, STATUS);
        INTRODUCE_NODES
            (FILE, BASE, KEY, RELATION_DEFINITION, UNIQUE_IDENTIFIERS,
             INCLUDE_CONTENTS, ATTRIBUTES, DISCRETIONARY_ACCESS, MANDATORY_ACCESS,
             NODE_KIND, STATUS, MEDIUM_RESOURCE);
    exception
        when others =>
            CLOSE (RELATION_DEFINITION);
            CLOSE (NODE_KIND);
            raise;
    end INTRODUCE_NODES;
```

Notes:

When more than one node is to reintroduced the latest key suffix must be specified for some key(s) to ensure the generation of unique key values for the primary relationships to the introduced nodes.

### 7.5.11 Introducing contents of introduced nodes

```
procedure INTRODUCE_CONTENTS
    (FILE   : in  FILE_TYPE;
     NODES  : in  NODE_HANDLE_ARRAY;
     STATUS : in  STATUS_INDICATOR := NO_INDICATION);
```

Purpose:

This procedure reads a CAIS_TEXT_IO stream, identified by the open file handle FILE, that contains an external node set (a CEF stream) and introduces the contents of a set of previously introduced nodes identified by open node handles in the array NODES. FILE must have the mode IN_FILE. The existing contents of the introduced nodes (if any) are discarded.

This procedure may be called after calling INTRODUCE_NODES with the parameter INCLUDE_CONTENTS set to FALSE without performing any other operations using the file handle FILE. At the time of the call, the position of FILE in the CEF stream must be at the beginning of the header section, at the beginning of the contents section or at the position resulting from a successful prior call to INTRODUCE_NODES with INCLUDE_CONTENTS set to FALSE; otherwise SYNTAX_ERROR may be raised. The position of the stream identified by FILE is implementation_defied after the call, even if an exception is raised.

Parameters:

**FILE** is an open file handle to the CEF stream (opened via the CAIS_TEXT_IO package).


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

**NODES** is an array of open node handles which identify existing introduced nodes whose contents are included in the CEF stream identified by FILE.

**STATUS** is an established status indicator.


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Exceptions:

**NAME_ERROR** is raised if there are no contents included in the CEF stream identified by FILE for a node with the unique identifier given by the IMPORTED_UID attribute of one of the introduced nodes identified by an open node handle in the array NODES [4]. 6

**STATUS_ERROR** is raised if any element of the array NODES is not an open node handle [2], if FILE is not an open file handle [8] or if STATUS is not an established status indicator [3]. 7

**SYNTAX_ERROR** is raised if the stream identified by FILE does not conform to the syntax for an external node set [15] (see Table II, page 73). 8

**MODE_ERROR** is raised if the mode of FILE is not IN_FILE [1]. 9

**LAYOUT_ERROR** is raised if the line or page length for the file handle FILE is bounded [4]. 10

**NODE_KIND_ERROR** is raised if any of the open node handle in NODES does not identify a node of the kind INTRODUCED_NODE [1]. 11

**TRANSACTION_ERROR** is raised if the operation is included in more than one transaction [1] or if the operation is included in a transaction and the operation modifies a node which is not included in the same transaction [2], or if the operation is included in a transaction, the opposite client is a SECONDARY_STORAGE node and the file handle FILE was not opened inside the transaction [3]. 12

**INTENT_VIOLATION** is raised if any node handle in the array NODES was not opened with intents establishing the rights to read attributes and write contents. 13

**SECURITY_VIOLATION** is raised if the operation represents a violation of mandatory access controls. SECURITY_VIOLATION is raised only if the conditions for other exceptions are not present. 14

### 7.5.12 Creating VIEW nodes with specific UIDs

```
procedure CREATE_VIEW_STATING_UID
    (VIEW                    : in out VIEW_NODE_TYPE;
     INTRODUCED_NODE         : in     INTRODUCED_NODE_TYPE;
     BASE                    : in     NODE_TYPE;
     KEY                     : in     RELATIONSHIP_KEY            := EMPTY_KEY_LIST;
     ATTRIBUTES              : in     ATTRIBUTE_LIST              := EMPTY_LIST;
     DISCRETIONARY_ACCESS    : in     DISCRETIONARY_ACCESS_LIST   := ALL_RIGHTS;
     MANDATORY_ACCESS        : in     MANDATORY_ACCESS_LIST       := EMPTY_LIST;
     KIND                    : in     NODE_KIND_NODE_TYPE         := VIEW_NKD;
     STATUS                  : in     STATUS_INDICATOR            := NO_INDICATION;
     MEDIUM_RESOURCE         : in     MEDIUM_NODE_TYPE            := DEFAULT_MEDIUM) ;
```

Purpose:

This procedure creates a node with a specific UID representing an empty view. 1

The value of the IMPORTED_UID attribute on the node identified by INTRODUCED_NODE is used as the UID of the newly created node. Otherwise the semantics are identical to those of CREATE_VIEW (see Section 5.7.13 on page 363). 2



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Additional Parameters:

**INTRODUCED_NODE** is an open node handle to an introduced node. The value of the IMPORTED_UID attribute on this introduced node is used as the UID of the newly created node.

Additional Exceptions:

**UID_ERROR** is raised if the unique identifier identified by the value of the IMPORTED_UID attribute on the INTRODUCED_NODE node is not unique within the CAIS instance [1].

**NODE_KIND_ERROR** is raised if the open node handle INTRODUCED_NODE does not identify an introduced node [1].

**STATUS_ERROR** is raised if INTRODUCED_NODE is not an open node handle [z].

**INTENT_VIOLATION** is raised if INTRODUCED_NODE was not opened with intents establishing the right to read attributes [1].

### 7.5.13 Creating NKDs with specific UIDs

```
procedure CREATE_NODE_KIND_STATING_UID
    (NKI                          : in out NKI_TYPE;
     NKD                          : in out NKD_TYPE;
     NKI_INTRODUCED_NODE          : in     INTRODUCED_NODE_TYPE;
     NKD_INTRODUCED_NODE          : in     INTRODUCED_NODE_TYPE;
     VIEW                         : in     VIEW_NODE_TYPE;
     SPECIALIZATION_OF            : in     NKD_TYPE;
     BASE                         : in     NODE_TYPE;
     LOCAL_NAME                   : in     RELATIONSHIP_KEY        := EMPTY_KEY_LIST;
     KEY                          : in     RELATIONSHIP_KEY        := EMPTY_KEY_LIST;
     SPECIALIZATION_OF_KEY        : in     RELATIONSHIP_KEY        := EMPTY_KEY_LIST;
     GENERALIZATION_OF_KEY        : in     RELATIONSHIP_KEY        := EMPTY_KEY_LIST;
     INTERPRETATION_ATTRIBUTES : in     ATTRIBUTE_LIST          := EMPTY_LIST;
     DEFINITION_ATTRIBUTES        : in     ATTRIBUTE_LIST          := EMPTY_LIST;
     DISCRETIONARY_ACCESS         : in     DISCRETIONARY_ACCESS_LIST := ALL_RIGHTS;
     MANDATORY_ACCESS             : in     MANDATORY_ACCESS_LIST := EMPTY_LIST;
     INTERPRETATION_KIND          : in     NODE_KIND_NODE_TYPE :=
                                             NODE_KIND_INTERPRETATION_NKD;
     DEFINITION_KIND              : in     NODE_KIND_NODE_TYPE :=
                                             NODE_KIND_DEFINITION_NKD;
     STATUS                       : in     STATUS_INDICATOR        := NO_INDICATION;
     INTERPRETATION_MEDIUM        : in     MEDIUM_NODE_TYPE        := DEFAULT_MEDIUM;
     DEFINITION_MEDIUM            : in     MEDIUM_NODE_TYPE        := DEFAULT_MEDIUM);
```

Purpose:

This procedure creates anew node kind with a specific UID.

The value of the IMPORTED_UID attribute on the node identified by NKI_INTRODUCED_-



Source: https://assist.dla.mil -- Downloaded: 202610-26T07:54Z
Check the source to verify that this is the current version before use.

NODE is used as the UID of the newly created NKI. The value of the IMPORTED_UID attribute on the node identified by NKD_INTRODUCED_NODE is used as the UID of the newly created NKD. Otherwise the semantics are identical to those of CREATE_NODE_KIND (see Section 5.7.15 on page 367).

Additional Parameters:

**NKI_INTRODUCED_NODE** is an open node handle to an introduced node. The value of the IMPORTED_UID attribute on this introduced node is used as the UID of the newly created NKI.

**NKD_INTRODUCED_NODE** is an open node handle to an introduced node. The value of the IMPORTED_UID attribute on this introduced node is used as the UID of the newly created NKD.

Additional Exceptions:

**UID_ERROR** is raised if the unique identifier identified by the value of the IMPORTED_UID attribute on the NKD_INTRODUCED_NODE or NKI_INTRODUCED_NODE node is not unique within the CAIS instance [1].

**NODE_KIND_ERROR** is raised if the open node handle NKD_INTRODUCED_NODE or NKI_INTRODUCED_NODE does not identify an introduced node [1].

**STATUS_ERROR** is raised if NKD_INTRODUCED_NODE or NKI_INTRODUCED_NODE is not an open node handle [z].

**INTENT_VIOLATION** is raised if NKD_INTRODUCED_NODE or NKI_INTRODUCED_NODE was not opened with intents establishing the right to read attributes [1].

### 7.5.14 Creating NKIs with specific UIDS

```
procedure IMPORT_NODE_KIND_STATING_UID
   (NKI                    : in out NKI_TYPE;
    INTRODUCED_NODE        : in     INTRODUCED_NODE_TYPE;
    VIEW                   : in     VIEW_NODE_TYPE;
    NKD                    : in     NKD_TYPE;
    LOCAL_NAME             : in     RELATIONSHIP_KEY          := EMPTY_KEY_LIST;
    ATTRIBUTES             : in     ATTRIBUTE_LIST            := EMPTY_LIST;
    DISCRETIONARY_ACCESS   : in     DISCRETIONARY_ACCESS_LIST := ALL_RIGHTS;
    MANDATORY_ACCESS       : in     MANDATORY_ACCESS_LIST     := EMPTY_LIST;
    INTERPRETATION_KIND    : in     NODE_KIND_NODE_TYPE :=
                                      NODE_KIND_INTERPRETATION_NKD;
    STATUS                 : in     STATUS_INDICATOR          := NO_INDICATION;
    MEDIUM_RESOURCE        : in     MEDIUM_NODE_TYPE          := DEFAULT_MEDIUM);
```

Purpose:

This procedure creates a new node kind interpretation with a specific UID.

The value of the IMPORTED_UID attribute on the node identified by INTRODUCED_NODE is used as the UID of the newly created node. Otherwise the semantics are identical to those of IMPORT_NODE_KIND (see Section 5.7.17 on page 373).



Source: https://assist.dla.mil -- Downloaded: 202610-26T07:54Z
Check the source to verify that this is the current version before use.

Additional Parameters:

**INTRODUCED_NODE** is an open node handle to an introduced node. The value of the IMPORTED_UID attribute on the INTRODUCED_NODE is used as the UID of the newly created NKI.

Additional Exceptions:

**UID_ERROR** is raised if the unique identifier identified by the value of the IMPORTED_UID attribute on the INTRODUCED_NODE node is not unique within the CAIS instance [1].

**NODE_KIND_ERROR** is raised if the open node handle INTRODUCED_NODE does not identify an introduced node [1].

STATUS_ERROR is raised if INTRODUCED_NODE is not an open node handle [2].

INTENT_VIOLATION is raised if INTRODUCED_NODE was not opened with intents establishing the right to read attributes [1].

### 7.5.15 Creating RDs with specific UIDS

```
procedure CREATE_RELATION_STATING_UID
    (RI                          : in out RI_TYPE;
     RD                          : in out RD_TYPE;
     RI_INTRODUCED_NODE          : in     INTRODUCED_NODE_TYPE;
     RD_INTRODUCED_NODE          : in     INTRODUCED_NODE_TYPE;
     VIEW                        : in     VIEW_NODE_TYPE;
     SPECIALIZATION_OF           : in     RD_TYPE;
     BASE                        : in     NODE_TYPE;
     LOCAL_NAME                  : in     RELATIONSHIP_KEY       : = EMPTY_KEY_LIST;
     KEY                         : in     RELATIONSHIP_KEY       : = EMPTY_KEY_LIST;
     SPECIALIZATION_OF_KEY       : in     RELATIONSHIP_KEY       : = EMPTY_KEY_LIST;
     GENERALIZATION_OF_KEY       : in     RELATIONSHIP_KEY       := EMPTY_KEY_LIST;
     INTERPRETATION_ATTRIBUTES : in     ATTRIBUTE_LIST         := EMPTY_LIST;
     DEFINITION_ATTRIBUTES       : in     ATTRIBUTE_LIST         := EMPTY_LIST;
     DISCRETIONARY_ACCESS        : in     DISCRETIONARY_ACCESS_LIST := ALL_RIGHTS;
     MANDATORY_ACCESS            : in     MANDATORY_ACCESS_LIST  := EMPTY_LIST;
     INTERPRETATION_KIND         : in     NODE_KIND_NODE_TYPE :=
                                              RELATION_INTERPRETATION_NKD;
     DEFINITION_KIND             : in     NODE_KIND_NODE_TYPE :=
                                              RELATION_DEFINITION_NKD;
     STATUS                      : in     STATUS_INDICATOR       := NO_INDICATION;
     INTERPRETATION_MEDIUM       : in     MEDIUM_NODE_TYPE       := DEFAULT_MEDIUM;
     DEFINITION_MEDIUM           : in     MEDIUM_NODE_TYPE       := DEFAULT_MEDIUM);
```

Purpose:

This procedure creates anew relation with a specific UID.

The value of the IMPORTED_UID attribute on the node identified by RI_INTRODUCED_-


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

NODE is used as the UID of the newly created RI. The value of the IMPORTED_UID attribute on the node identified by RD_INTRODUCED_NODE is used as the UID of the newly created RD. Otherwise the semantics are identical to those of CREATE_RELATION (see Section 5.7.22 on page 384).

Additional Parameters:

**RI_INTRODUCED_NODE** is an open node handle to an introduced node. The value of the IMPORTED_UID attribute on this introduced node is used as the UID of the newly created RI.

**RD_INTRODUCED_NODE** is an open node handle to an introduced node. The value of the IMPORTED_UID attribute on this introduced node is used as the UID of the newly created RD.

Additional Exceptions:

**UID_ERROR** is raised if the unique identifier identified by the value of the IMPORTED_UID attribute on the RD_INTRODUCED_NODE or RI_INTRODUCED_NODE node is not unique within the CAIS instance [1].

**NODE_KIND_ERROR** is raised if the open node handle RD_INTRODUCED_NODE or RI_INTRODUCED_NODE does not identify an introduced node [1].

**STATUS_ERROR** is raised if RD_INTRODUCED_NODE or RI_INTRODUCED_NODE is not an open node handle [2].

**INTENT_VIOLATION** is raised if RD_INTRODUCED_NODE or RI_INTRODUCED_NODE was not opened with intents establishing the right to read attributes [1].

### 7.5.16 Creating RIs with specific UIDS

```
procedure IMPORT_RELATION_STATING_UID
    (RI                    : in out RI_TYPE;
     INTRODUCED_NODE       : in     INTRODUCED_NODE_TYPE;
     VIEW                  : in     VIEW_NODE_TYPE;
     RD                    : in     RD_TYPE;
     LOCAL_NAME            : in     RELATIONSHIP_KEY          := EMPTY_KEY_LIST;
     ATTRIBUTES            : in     ATTRIBUTE_LIST            := EMPTY_LIST;
     DISCRETIONARY_ACCESS  : in     DISCRETIONARY_ACCESS_LIST := ALL_RIGHTS;
     MANDATORY_ACCESS      : in     MANDATORY_ACCESS_LIST     := EMPTY_LIST;
     INTERPRETATION_KIND   : in     NODE_KIND_NODE_TYPE       :=
                                        RELATION_INTERPRETATION_NKD;
     STATUS                : in     STATUS_INDICATOR          := NO_INDICATION;
     MEDIUM_RESOURCE       : in     MEDIUM_NODE_TYPE          := DEFAULT_MEDIUM);
```

Purpose:

This procedure creates a new relation interpretation with a specific UID.

The value of the IMPORTED_UID attribute on the node identified by INTRODUCED_NODE is used as the UID of the newly created RI. Otherwise the semantics are identical to those of IMPORT_RELATION (see Section 5.7.24 on page 391).



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Additional Parameters:

**INTRODUCED_NODE** is an open node handle to an introduced node. The value of the IMPORTED_UID attribute on this introduced node is used as the UID of the newly created RI. 3

Additional Exceptions:

**UID_ERROR** is raised if the unique identifier identified by the value of the IMPORTED_UID attribute on the INTRODUCED_NODE node is not unique within the CAIS instance [1]. 4

**NODE_KIND_ERROR** is raised if the open node handle INTRODUCED_NODE does not identify an introduced node [1]. 5

**STATUS_ERROR** is raised if INTRODUCED_NODE is not an open node handle [2]. 6

**INTENT_VIOLATION** is raised if INTRODUCED_NODE was not opened with intents establishing the right to read attributes [1]. 7

### 7.5.17 Creating bidirectional RDs with specific UIDs

```
procedure CREATE. BIDIRECTIONAL_RELATIOH_STATING_UID
    (RI1                              : in out RI_TYPE;
     RI2                              : in out RI_TYPE;
     RD1                              : in out RD_TYPE;
     RD2                              : in out RD_TYPE;
     RI1_INTRODUCED_NODE              : in     INTRODUCED_NODE_TYPE;
     R12_INTRODUCED_NODE              : in     INTRODUCED_NODE_TYPE;
     RD1_lNTRODUCED_NODE              : in     INTRODUCED_NODE_TYPE;
     RD2_INTRODUCED_NODE              : in     INTRODUCED_NODE_TYPE;
     VIEW                             : in     VIEW_NODE_TYPE;
     SPECIALIZATION_OF1               : in     RD_TYPE;
     SPECIALIZATION_OF2               : in     RD_TYPE;
     BASE1                            : in     NODE_TYPE;
     BASE2                            : in     NODE_TYPE;
     LOCAL_NAME1                      : in     RELATIONSHIP_KEY := EMPTY_KEY_LIST;
     LOCAL_NAME2                      : in     RELATIONSHIP_KEY := EMPTY_KEY_LIST;
     KEY1                             : in     RELATIONSHIP_KEY := EMPTY_KEY_LIST;
     KEY2                             : in     RELATIONSHIP_KEY := EMPTY_KEY_LIST;
     SPECIALIZATION_OF1_KEY           : in     RELATIONSHIP_KEY := EMPTY_KEY_LIST;
     SPECIALIZATION_OF2_KEY           : in     RELATIONSHIP_KEY := EMPTY_KEY_LIST;
     GENERALIZATION_OF1_KEY           : in     RELATIONSHIP_KEY := EMPTY_KEY_LIST;
     GENERALIZATION_OF2_KEY           : in     RELATIONSHIP_KEY := EMPTY_KEY_LIST;
     INTERPRETATION_ATTRIBUTES1 : in     ATTRIBUTE_LIST   := EMPTY_LIST;
     INTERPRETATION_ATTRIBUTES2 : in     ATTRIBUTE_LIST   := EMPTY_LIST;
     DEFINITION_ATTRIBUTES1           : in     ATTRIBUTE_LIST   := EMPTY_LIST;
     DEFINITION_ATTRIBUTES2           : in     ATTRIBUTE_LIST   := EMPTY_LIST;
     DISCRETIONARY_ACCESS             : in     DISCRETIONARY_ACCESS_LIST := ALL_RIGHTS;
     MANDATORY_ACCESS                 : in     MANDATORY_ACCESS_LIST     := EMPTY_LIST;
     INTERPRETATION_KIND1             : in     NODE_KIND_NODE_TYPE :=
```


Source: https://assist.dla.mil -- Downloaded: 2026l0-26T07:54Z
Check the source to verify that this is the currentversion before use.

```
                                                    RELATION_INTERPRETATION_NKD;
    INTERPRETATION_KIND2        : in    NODE_KIND_NODE_TYPE         :=
                                                    RELATION_INTERPRETATION_NKD;
    DEFINITION_KIND1            : in    NODE_KIND_NODE_TYPE         :=
                                                    RELATION_DEFINITION_NKD;
    DEFINITION_KIND2            : in    NODE_KIND_NODE_TYPE         :=
                                                    RELATION_DEFINITION_NKD;
    STATUS                      : in    STATUS_INDICATOR := NO_INDICATION;
    INTERPRETATION_MEDIUM       : in    MEDIUM_NODE_TYPE := DEFAULT_MEDIUM;
    DEFINITION_MEDIUM           : in    MEDIUM_NODE_TYPE    := DEFAULT_MEDIUM);

procedure CREATE_BIDIRECTIONAL_RELATION_STATING_UID
   (RI                          : in out RI_TYPE;
    RD                          : in out RD_TYPE;
    RI_INTRODUCED_NODE          : in    INTRODUCED_NODE_TYPE;
    RD_INTRODUCED_EODE          : in    INTRODUCED_NODE_TYPE;
    VIEW                        : in    VIEW_NODE_TYPE;
    SPECIALIZATION_OF           : in    RD_TYPE;
    BASE                        : in    NODE_TYPE;
    LOCAL_NAME                  : in    RELATIONSHIP_KEY    := EMPTY_KEY_LIST;
    KEY                         : in    RELATIONSHIP_KEY := EMPTY_KEY_LIST;
    SPECIALIZATION_OF_KEY       : in    RELATIONSHIP_KEY    := EMPTY_KEY_LIST;
    GENERALIZATION_OF_KEY       : in    RELATIONSHIP_KEY    := EMPTY_KEY_LIST;
    INTERPRETATION_ATTRIBUTES   : in    ATTRIBUTE_LIST      := EMPTY_LIST;
    DEFINITION_ATTRIBUTES       : in    ATTRIBUTE_LIST      := EMPTY_LIST;
    DISCRETIONARY_ACCESS        : in    DISCRETIONARY_ACCESS_LIST := ALL_RIGHTS;
    MANDATORY_ACCESS            : in    MANDATORY_ACCESS_LIST     := EMPTY_LIST;
    INTERPRETATION_KIND         : in    NODE_KIND_NODE_TYPE         :=
                                                    RELATION_INTERPRETATION_NKD;
    DEFINITION_KIND             : in    NODE_KIND_NODE_TYPE         :=
                                                    RELATION_DEFINITION_NKD;
    STATUS                      : in    STATUS_INDICATOR := NO_INDICATION;
    INTERPRETATION_MEDIUM       : in    MEDIUM_NODE_TYPE := DEFAULT_MEDIUM;
    DEFINITION_MEDIUM           : in    MEDIUM_NODE_TYPE    := DEFAULT_MEDIUM);
```

Purpose:

This procedure creates new relations with specific UIDs.

The values of the IMPORTED_UID attributes on the nodes identified by RI_INTRODUCED_-NODE parameters are used as the UIDs of the newly created RIs. The values of the IMPORTED_-UID attributes on the nodes identified by the RD_INTRODUCED_NODE parameters are used as the UIDs of the newly created RDs. Otherwise the semantics are identical to those of CREATE_-BIDIRECTIONAL_RELATION (see Section 5.7.29 on page 402).

Additional Parameters:

**RI_INTRODUCED_NODE** parameters are an open node handles to introduced nodes. The values of the IMPORTED_UID attributes on these introduced nodes are used as the UIDs of the newly created RIs.



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

**RD_INTRODUCED_NODE** parameters are an open node handles to introduced nodes. The values of the IMPORTED_UID attributes on these introduced nodes are used as the UIDs of the newly created RDs.

Additional Exceptions:

**UID_ERROR** is raised if a unique identifier identified by the value of an IMPORTED_UID attribute on an RD_INTRODUCED_NODE or RI_INTRODUCED_NODE node is not unique within the CAIS instance [1].

**NODE_KIND_ERROR** is raised if the open node handle provided through an RD_INTRODUCED_NODE or RI_INTRODUCED_NODE parameter does not identify an introduced node [1].

**STATUS_ERROR** is raised if an RD_INTRODUCED_NODE or RI_INTRODUCED_NODE parameter is not an open node handle [2].

**INTENT_VIOLATION** is raised if an RD_INTRODUCED_NODE or RI_INTRODUCED_NODE parameter was not opened with intents establishing the right to read attributes [1].

### 7.5.18 Creating ADs with specific UIDs

```
procedure CREATE_ATTRIBUTE_DEFINITION_STATING_UID
    (AD                    : in out AD_TYPE;
     INTRODUCED_NODE       : in     INTRODUCED_NODE_TYPE;
     VIEW                  : in     VIEW_NODE_TYPE;
     BASE                  : in     NODE_TYPE;
     ATTRIBUTE_KIND        : in     AKD_TYPE;
     LOCAL_NAME            : in     RELATIONSHIP_KEY          := EMPTY_KEY_LIST;
     KEY                   : in     RELATIONSHIP_KEY          := EMPTY_KEY_LIST;
     ATTRIBUTE_KIND_KEY    : in     RELATIONSHIP_KEY          := EMPTY_KEY_LIST;
     ATTRIBUTES            : in     ATTRIBUTE_LIST            := EMPTY_LIST;
     DISCRETIONARY_ACCESS  : in     DISCRETIONARY_ACCESS_LIST := ALL_RIGHTS;
     MANDATORY_ACCESS      : in     MANDATORY_ACCESS_LIST     := EMPTY_LIST;
     KIND                  : in     NODE_KIND_NODE_TYPE :=
                                        ATTRIBUTE_DEFINITION_NKD;
     STATUS                : in     STATUS_INDICATOR          := NO_INDICATION;
     MEDIUM_RESOURCE       : in     MEDIUM_NODE_TYPE          := DEFAULT_MEDIUM);
```

Purpose:

This procedure creates anew attribute definition with a specific UID.

The value of the IMPORTED_UID attribute on the node identified by INTRODUCED_NODE is used as the UID of the newly created AD. Otherwise the semantics are identical to those of CREATE_ATTRIBUTE_DEFINITION (see Section 5.7.31 on page 412).


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Additional Parameters:

**INTRODUCED_NODE** is an open node handle to an introduced node. The value of the IMPORTED_UID attribute on this introduced node is used as the UID of the newly created AD.

Additional Exceptions:

**UID_ERROR** is raised if the unique identifier identified by the value of the IMPORTED_UID attribute on the INTRODUCED_NODE node is not unique within the CAIS instance [1].

**NODE_KIND_ERROR** is raised if the open node handle INTRODUCED_NODE does not identify an introduced node [1].

**STATUS_ERROR** is raised if INTRODUCED_NODE is not an open node handle [2].

**INTENT_VIOLATION** is raised if INTRODUCED_NODE was not opened with intents establishing the right to read attributes [1].

### 7.5.19 Creating AKDs with specific UIDs

```
procedure CREATE_ATTRIBUTE_KIND_STATING_UID
    (AKD                    : in out AKD_TYPE;
     INTRODUCED_NODE        : in     INTRODUCED_NODE_TYPE;
     VIEW                   : in     VIEW_NODE_TYPE;
     BASE                   : in     NODE_TYPE;
     LOCAL_NAME             : in     RELATIONSHIP_KEY          : = EMPTY_KEY_LIST;
     KEY                    : in     RELATIONSHIP_KEY          := EMPTY_KEY_LIST;
     ATTRIBUTES             : in     ATTRIBUTE_LIST            : = EMPTY_LIST;
     DISCRETIONARY_ACCESS : in     DISCRETIONARY_ACCESS_LIST : = ALL_RIGHTS;
     MANDATORY_ACCESS       : in     MANDATORY_ACCESS_LIST : = EMPTY_LIST;
     KIND                   : in     NODE_KIND_NODE_TYPE :=
                                         NAMED_COMPOSITE_LIST_AKD_NKD;
     STATUS                 : in     STATUS_INDICATOR          := NO_INDICATION;
     MEDIUM_RESOURCE        : in     MEDIUM_NODE_TYPE          := DEFAULT_MEDIUM);
```

Purpose:

This procedure creates anew attribute kind with a specific UID.

The value of the IMPORTED_UID attribute on the node identified by INTRODUCED_NODE is used as the UID of the newly created node. Otherwise the semantics are identical to those of CREATE_ATTRIBUTE_KIND (see Section 5.7.36 on page 421).

Additional Parameters:

**INTRODUCED_NODE** is an open node handle to an introduced node. The value of the IMPORTED_UID attribute on this introduced node is used as the UID of the newly created AKD.


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Additional Exceptions:

**UID_ERROR** is raised if the unique identifier identified by the value of the IMPORTED_UID attribute on the INTRODUCED_NODE node is not unique within the CAIS instance [1].

**NODE_KIND_ERROR** is raised if the open node handle INTRODUCED_NODE does not identify an introduced node [1].

**STATUS_ERROR** is raised if INTRODUCED_NODE is not an open node handle [2].

**INTENT_VIOLATION** is raised if INTRODUCED_NODE was not opened with intents establishing the right to read attributes [1].

### 7.5.20 Creating structural nodes with specific UIDs

```
procedure CREATE_NODE_STATING_UID
    (NODE                  : in out STRUCTURAL_NODE_TYPE;
     INTRODUCED_NODE       : in     INTRODUCED_NODE_TYPE;
     BASE                  : in     NODE_TYPE;
     KEY                   : in     RELATIONSHIP_KEY          := EMPTY_KEY_LIST;
     RELATION              : in     RD_TYPE;
     INTENT                : in     INTENT_ARRAY              := (l => WRITE);
     ATTRIBUTES            : in     ATTRIBUTE_LIST            : = EMPTY_LIST;
     DISCRETIONARY_ACCESS : in     DISCRETIONARY_ACCESS_LIST : = ALL_RIGHTS;
     MANDATORY_ACCESS      : in     MANDATORY_ACCESS_LIST  := EMPTY_LIST;
     KIND                  : in     NODE_KIND_NODE_TYPE       := STRUCTURAL_NKD;
     STATUS                : in     STATUS_INDICATOR          := NO_INDICATION;
     MEDIUM_RESOURCE       : in     MEDIUM_NODE_TYPE          := DEFAULT_MEDIUM);
```

Purpose:

This procedure creates anew STRUCTURAL node with a specific UID.

The value of the IMPORTED_UID attribute on the node identified by INTRODUCED_NODE is used as the UID of the newly created node. Otherwise the semantics are identical to those of CREATE_NODE (see Section 5.5.2 on page 280).

Additional Parameters:

**INTRODUCED_NODE** is an open node handle to an introduced node. The value of the IMPORTED_UID attribute on this introduced node is used as the UID of the newly created node.

Additional Exceptions:

**UID_ERROR** is raised if the unique identifier identified by the value of the IMPORTED_UID attribute on the INTRODUCED_NODE node is not unique within the CAIS instance [1].

**NODE_KIND_ERROR** is raised if the open node handle INTRODUCED_NODE does not identify an introduced node [1].

**STATUS_ERROR** is raised if the node handle INTRODUCED_NODE is not open [2].

**INTENT_VIOLATION** is raised if INTRODUCED_NODE was not opened with intents establishing the right to read attributes [1].


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

### 7.5.21 Creating a secondary storage node with a specific UID

```
procedure CREATE_SECONDARY_STORAGE_MODE_STATING_UID
    (MODE                    : in out SECONDARY_STORAGE_MODE_TYPE;
     INTRODUCED_MODE         : in     INTRODUCED_MODE_TYPE;
     BASE                    : in     MODE_TYPE;
     KEY                     : in     RELATIONSHIP_KEY          := EMPTY_KEY_LIST;
     RELATION                : in     RD_TYPE ;
     INTENT                  : in     INTENT_ARRAY              := (1=>WRITE);
     ATTRIBUTES              : in     ATTRIBUTE-LIST            := EMPTY-LIST;
     DISCRETIONARY.ACCESS : in     DISCRETIONARY-ACCESS-LIST := ALL-RIGHTS;
     MANDATORY-ACCESS        : in     MANDATORY_ACCESS_LIST  := EMPTY_LIST;
     KIND                    : in     MODE_KIND_MODE_TYPE       := SECONDARY_STORAGE_NKD;
     STATUS                  : in     STATUS_INDICATOR          := NO_INDICATION;
     MEDIUM-RESOURCE         : in     MEDIUM_NODE_TYPE          := DEFAULT_MEDIUM;
```

Purpose:

This procedure creates anew SECONDARY_STORAGE node with a specific UID. 1

The value of the IMPORTED_UID attribute on the node identified by INTRODUCED_NODE is used as the UID of the newly created node. Otherwise the semantics are identical to those of CREATE_SECONDARY_STORAGE_NODE (see Section 7.2.2 on page 571). 2

Additional Parameters:

**INTRODUCED-NODE** is an open node handle to an introduced node. The value of the IMPORTED_UID attribution this introduced node is used as the UID of the newly created SECONDARY-STORAGE node. 3

Additional Exceptions:

**UID_ERROR** is raised if the unique identifier identified by the value of the IMPORTED_UID attribute on the INTRODUCED_NODE node is not unique within the CAIS instance [1]. 4

**NODE-KIND-ERROR** is raised if the open node handle INTRODUCED-NODE does not identify an introduced node [1]. 5

**STATUS-ERROR** is raised if the node handle INTRODUCED-NODE is not open [2]. 6

**INTENT-VIOLATION** is raised if the open node handle INTRODUCED_NODE was not opened with intents establishing the right to read attributes [1]. 7



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

# 8. DATA TRANSFER PROTOCOLS DETAILED REQUIREMENTS

## 8.1 Input and Output Detailed Requirements

This section describes the protocol packages that are a mandatory part of the CAIS standard. The protocols defined in this section are:

a. CAIS_FRAME_IO defines a rudimentary data transfer protocol which can be applied to interprocess communication as well as other areas;

b. CAIS_MAGNETIC_TAPE_IO defines a data transfer protocol for magnetic tape drives;

c. CAIS_DIRECT_IO defines a data transfer protocol corresponding to the DIRECT_IO package given in 14.2.4 of [1815A];

d. CAIS_SEQUENTIAL_IO defines a data transfer protocol corresponding to the SEQUENTIAL_IO package given in 14.2.2 of [1815A];

e. CAIS_TEXT_IO defines a data transfer protocol corresponding to the TEXT_IO package given in 14.3 of [1815A];

f. CAIS_TERMINAL_IO defines a data transfer protocol for controlling character imaging display terminals and high resolution display terminals simulating these.

One other protocol package, CAIS_FORM_TERMINAL_IO, is given in Appendix C.


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

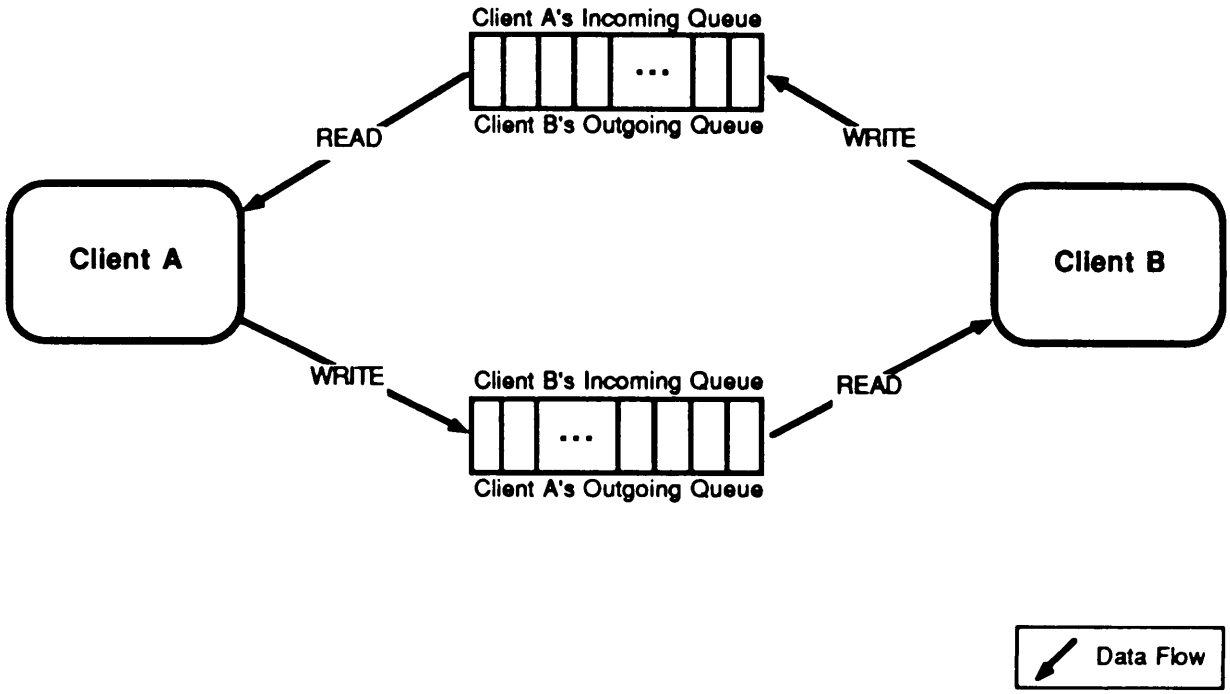


**FIGURE 43. Frame IO message queues**

## 8.2 Package CAIS_FRAME_IO

This package defines a low-level data transfer protocol intended primarily for inter-process communication. The operations provided allow one to OPEN and CLOSE a channel for use with CAIS_FRAME_IO and to READ and WRITE across this channel. 1

In the CAIS_FRAME_IO model, a data *stream* is an abstraction representing a sequence of storage units. A storage unit is the lowest level machine representation, typically an 8-bit byte. The data stream must be divided into segments to be transmitted across a channel. Each segment is termed a *frame.* A message is a frame with an associated priority. 2

The CAIS_FRAME_IO interfaces operate at the frame or message level; the do not deal directly with streams or with storage units. The process issuing WRITES is responsible for dividing the stream into frames and the process issuing READs is responsible for reassembling the frames into a stream. Frame boundaries are not usually significant with respect to the semantics of the stream. 3

As depicted in Figure 43, the CAIS_FRAME_IO model incorporates two queues, one for each direction, for buffering messages. To a given client node, an *outgoing queue* is the queue to which the client node writes frames; an *incoming queue* is the queue from which the client node reads frames. For two client nodes A and B, one queue is the outgoing queue to A and the incoming queue to B. The other queue is the outgoing queue to B and the incoming queue to A. 4

Priority levels are maintained by the queuing mechanism. Within a priority level, frames are kept in fist-in-first-out (FIFO) order. Higher priority messages are queued in front of lower priority messages. Since the queue mechanism also provides buffering, it acts as a foundation for flow control as well as priority message ordering. 5

When a SECONDARY_ STORAGE node is one of the client nodes to a channel established for CAIS_-FRAME_IO, priority levels are effectively ignored. In essence, a file consumes messages as quickly as they are produced and produces messages as quickly as they are requested. 6


Source: https://assist.dla.mil - Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Time limits are associated with both read and write requests to prevent a write from blocking when buffers are filled or a read from blocking when no data exists in the buffer. This mechanism is also useful for detecting channel disconnects.

## 8.2.1 Protocol parameters

The IO connection model allows protocol-specific parameters to be determined during the establishment of a channel (see Section 7.3.1.3 on page 576). For CAIS_FRAME_IO, all protocol parameters are represented by attributes on the PORT relationships to a CHANNEL node. None of the CAIS_-FRAME_IO protocol parameter attributes are copied to the CHANNEL node by the CAIS_FRAME_IO protocol open operation because each one describes a characteristic of one half of the channel; consequently a given protocol parameter attribute may not appear on both PORT relationships of the channel or may have a different value on the two PORT relationships of the channel.

Each PORT relationship of a channel being established for CAIS_FRAME_IO must express values for the following attributes:

**QUEUE_LENGTH** is an integer value reflecting the maximum number of messages that can be queued in the outgoing queue associated with the port. The implementation must guarantee that at least this many messages of the maximum frame length can be accommodated and that no more than this number of messages will be accommodated.

**PRIORITY_LEVEL** is an integer value reflecting the maximum priority level used for roessages written on this port.

**OUTGOING_** FRAME_ LENGTH_ RANGE is a CAIS list containing a pair of integer values reflecting the lower and upper bounds on the length of frames (expressed in memory storage units) written on the port.

**INCOMING_FRAME_LENGTH** is an integer value representing the maximum length of a frame (expressed in memory storage units) that can be read from the port.

These parameters control the way the protocol is used in specific situations. The absolute bounds that values of these parameters may assume is dependent upon the constants MAXIMUM-FRAME_ -LENGTH and MAXIMUM_ FRAME_ PRIORITY defined in the implementation-dependent package CAIS_PRAGMATICS (see Section 12 on page 815).

The values of these protocol parameters can be used to derive a number of differing implementations for the CAIS_FRAME_IO protocol. If a queue length of O is specified, CAIS_FRAME_IO must be implemented with a synchronous mechanism. In other words, a WRITE operation blocks until the opposite client node consumes the written frame via a READ operation. With a queue length of 1, a WRITE may complete before a READ begins, but a second WRITE cannot complete until the first READ has completed. With queue lengths of O or 1, message priority levels have no effect.

If the maximum priority level is set to 1, explicit priority information can be omitted from messages in the underlying implementation. Moreover, the queue discipline degenerates to a simple FIFO algorithm.

If the minimum and maximum frame lengths, as determined by the OUTGOING_FRAME_LENGTH_-RANGE attribute, are the same, a fixed-length mechanism can be used in the underlying implementation.


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

## 8.2.2 Types, subtypes, and exceptions

```
type FILE_TYPE is limited private;

type FILE_M0DE is (IN_FILE, INOUT_FILE, OUT_FILE, APPEND_FILE);
```

FILE_TYPE describes the type for file handles for all CAIS_FRAME_IO operations. 1

FILE_MODE indicates whether input operations, output operations, or both can be performed with the file handle. IN_ FILE corresponds to the case where only reading is to be performed. INOUT_FILE corresponds to the case where both reading and writing are to be performed. OUT-FILE corresponds to the case where only writing is to be performed. APPEND_FILE corresponds to the case where only writing (beginning at the end of the file) is to be performed; the file terminator is deleted at the time the file is opened. 2

```
subtype FRAME_ADDRESS is SYSTEM.ADDRESS;

subtype FRAME_LEHGTH is
    CAIS_IHTEGER range 1..CAIS_PRAGMATICS.MAXIMUM_FRAME_LENGTH;
```

A frame is actually an area of storage addressed by a value of type FRAME-ADDRESS. The length of a frame is denoted by a value of type FRAME_ LENGTH and is measured in memory storage units. The range of frame length is constrained initially by the value of CAIS_PRAGMATICS.MAXIMUM_-FRAME_LENGTH and further by the values of the minimum and maximum frame length protocol parameters. 3

```
MINIMUM_FRAME_ PRIORITY : constant : = 1;

subtype PRIORITY-TYPE is CAIS_INTEGER
  range MINIMUM-FRAME_PRIORITY.. CAIS_PRAGMATICS.MAXIMUM_FRAME_PRIORITY;
```

A value of type PRIORITY_TYPE specifies the priority of a message. Higher priority messages have larger PRIORITY-TYPE values. 4

```
QUEUE_EMPTY : exception;
QUEUE-FULL  : exception;
```

QUEUE_EMPTY is raised during a READ if there is no message in the queue to read. 5

QUEUE_FULL is raised during a WRITE if there is insufficient room in the queue for the message being written. 6

Other exceptions raised by the subprograms in this package are defined in the packages CAIS_-DEFINITIONS, CAIS_IO_DEFINITIONS and CAIS_PRAGMATICS. 7



Source: https://assist.dla.mil -- Downloaded: 202610-26T07:54Z
Check the source to verify that this is the current version before use.

## 8.2.3 Opening a frame file handle

```
procedure OPEN
    (FILE                : in out FILE-TYPE;
     CHANNEL             : in     CHANNEL_NODE_TYPE;
     OPPOSITE-CLIENT     : in     CLIENT_NODE_TYPE;
     MODE                : in     FILE-MODE;
     TIME-LIMIT          : in     CAIS_IO_DURATION := LONG_IO_DELAY;
     STATUS              : in     STATUS-INDICATOR  := N0_INDICATION;
```

Purpose:

This procedure opens a file handle for communicating over the channel identified by CHANNEL with the node identified by OPPOSITE_CLIENT.CAIS_FRAME_IO must be an allowed protocol of the CHANNEL node identified by CHANNEL. The mode of the file handle is set to MODE. 1

If the operation concludes successfully, the CONTROLLING-PROTOCOL attribute on the CHANNEL node is set to CAIS_FRAME_IO and the channel is in an ESTABLISHED state. 2

Parameters:

FILE is a file handle, initially closed, to be opened. 3

CHANNEL is an open node handle to a CHANNEL node to which both the current process and OPPOSITE_ CLIENT are client nodes. 4

OPPOSITE-CLIENT is an open node handle to the node to be accessed. 5

MODE indicates the mode under which the file handle is to be opened. 6

TIME_ LIMIT is the time limit for waiting for the OPEN to complete. 7

STATUS is an established status indicator. 8

Exceptions:

STATUS-ERROR is raised if FILE is an open file handle [7], if OPPOSITE_ CLIENT is not an open node handle [2], if CHANNEL is not an open node handle [2], if the channel denoted by CHANNEL is not in the CONNECTED state [9], or if STATUS is not an established status indicator [3]. 9

NODE-KIND_ ERROR is raised if the node identified by CHANNEL is not a CHANNEL node [1]. 10

CLIENT-ERROR is raised if the current PROCESS node [1] or the node identified by OPPOSITE_ CLIENT [5] is not a client node of the channel identified by the CHANNEL parameter [1]. 11

PORT. ERROR is raised if the DIRECTION attributes of the PORT relationships are missing or have incompatible values [4]. 12

PROTOCOL_ ERROR is raised if CAIS_FRAME_IO is not an allowed protocol of both PORT relationships of the channel [1], if the CONTROLLING-PROTOCOL attribute has a value other than NO-PROTOCOL or CAIS_FRAME_IO [2], if a required protocol parameter 13



Source: https://assist.dla.mil -- Downloaded: 202610-26T07:54Z
Check the source to verify that this is the current version before use.

attribute is not present on one or both PORT relationships at the time of the call [5], if the value of the INCOMING-FRAME-LENGTH attribute of the local PORT relationship is smaller than the upper bound of the OUTGOING. FRAME. LENGTH-RANGE attribute on the opposite PORT relationship [3] or if both clients of the channel are the current process and the MODE specified is INOUT_FILE [6].

**MODE-ERROR** is raised if MODE is incompatible with the DIRECTION attribute of the local PORT relationship to the channel identified by CHANNEL as determined by Table XXXIV on page 581 [2] or if both clients are processes and specify incompatible modes as determined by Table XXXIV on page 581 [3]. 14

**USE_ERROR** is raised if the node identified by OPPOSITE_ CLIENT is a SECONDARY_ - STORAGE node and the mode INOUT_FILE has been specified in the MODE parameter [44] or if another open file handle identifies the same external file and the CAIS implementation does not support the existence of multiple file handles identifying the same external file [41]. Any such restriction must be documented in Appendix F. An implementation is allowed to raise this exception only if it is based on operating system support that does not provide this capability. 15

**TRANSACTION-ERROR** is raised if the operation is included in more than one transaction [1] or if the operation is included in a transaction and the operation modifies a node which is not included in the same transaction [2]. 16

**INTENT-VIOLATION** is raised if the node handle OPPOSITE_CLIENT was not opened with an intent specification including at least the intents required for the MODE as determined by Table XXXIV on page 581 [1]. 17

**OPEN_DENIED** is raised if a remote client process or the opposite client, if it is a process, deleted the channel, signaling the denial of the connection attempt [1] or if a protocol open issued by the opposite client process fails for protocol compatibility reasons which are not determinable by the current process [2]. 18

**TIME_EXPIRED** is raised if the OPEN operation is delayed beyond the specified time limit for reasons other than a lock conflict [1]. 19

**SECURITY_VIOLATION** is raised if the operation represents a violation of mandatory access controls. SECURITY_VIOLATION is raised only if the conditions for other exceptions are not present. 20

Notes:

Closing the node handle OPPOSITE-CLIENT associated with the file handle FILE closes the file handle. 21

## 8.2.4 Closing a frame file handle

```
procedure CLOSE
    (FILE    : in out FILE_TYPE;
     STATUS : in      STATUS_INDICATOR  : = NO_INDICATION);
```



Source: https://assist.dla.mil -- Downloaded: 202610-26T07:54Z
Check the source to verify that this is the current version before use.

Purpose:

This procedure severs the association between the internal file identified by the file handle FILE and its associated channel. The file handle is closed. Closing a closed file handle has no effect. 1

The state of the channel associated with FILE becomes CLOSED if presently ESTABLISHED and becomes CONNECTED if already CLOSED. If the opposite client node is a file, the channel becomes immediately CONNECTED. If the charnel becomes CONNECTED, the CONTROL-LING_PROTOCOL attribute of the CHANNEL node is set to NO. PROTOCOL. 2

Parameters:

**FILE** is a file handle to be closed. 3

**STATUS** is an established status indicator. 4

Exceptions:

**TRANSACTION_ERROR** is raised if the operation is included in a transaction and the file handle FILE was not opened inside the transaction [3]. 5

**STATUS_ERROR** is raised if STATUS is not an established status indicator [3]. 6

### 8.2.5 Determining whether a frame file handle is open

```
function IS_OPEN
    (FILE   : in FILE. TYPE;
     STATUS : in STATUS_INDICATOR := NO_INDICATION)
    return BOOLEAN;
```

Purpose:

This function returns TRUE if the file handle FILE is open; otherwise, it returns FALSE. 1

Parameters:

**FILE** is a file handle. 2

**STATUS** is an established status indicator. 3

Exceptions:

**STATUS_ERROR** is raised if STATUS is not an established status indicator [3]. 4

### 8.2.6 Determining the mode of a frame file handle

```
function MODE
    (FILE   : in FILE_ TYPE;
     STATUS : in STATUS_ INDICATOR := NO_INDICATION)
    return FILE_ MODE;
```


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Purpose:

The function returns the mode under which the file handle FILE is open. 1

Parameters:

**FILE** is a file handle. 2

**STATUS** is an established status indicator. 3

Exceptions:

**STATUS_ERROR** is raised if FILE is not an open file handle [8] or if STATUS is not an established status indicator [3]. 4

## 8.2.7 Transmitting a frame

```
procedure WRITE
    (FILE       : in FILE-TYPE;
     FRAME      : in FRAME_ADDRESS;
     LENGTH     : in FRAME_LENGTH;
     PRIORITY   : in PRIORITY-TYPE       := MINIMUM_FRAME_ PRIORITY;
     BLOCKING   : in BOOLEAN             := FALSE;
     TIME_LIMIT : in CAIS_IO_DURATION := LONG_IO_DELAY;
     STATUS     : in STATUS_INDICATOR    := NO_INDICATION);
```

Purpose:

This procedure writes a frame to the channel specified by the FILE parameter. The frame to be written is stored beginning at address FRAME and is LENGTH memory storage units long. 1

Frames written at the same priority are queued in FIFO order. Higher priority frames are queued in front of lower priority frames. 2

If BLOCKING is TRUE and the queue is full, the operation is blocked until space is available or it times out. If BLOCKING is FALSE and the queue is full, the QUEUE_FULL exception is raised immediately. 3

Parameters:

**FILE** is an open file handle identifying the channel to which the frame is to be written. 4

**FRAME** is the beginning address of the frame to be sent. 5

**LENGTH** is the length of the frame in memory storage units. 6

**PRIORITY** is the the priority level for the written frame. 7

**BLOCKING** is a Boolean value indicating whether blocking is turned on. 8

**TIME_LIMIT** is the time limit for waiting on available space of a full queue. 9

**STATUS** is an established status indicator. 10


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Exceptions:

**STATUS_ERROR** is raised if FILE is not an open file handle at the time of the call [8], if the channel associated with FILE is not in the ESTABLISHED state [9], or if STATUS is not an established status indicator [3]. 11

**QUEUE_FULL** is raised when the BLOCKING parameter is FALSE and the queue is full [1]. 12

**PROTOCOL_ERROR** is raised if LENGTH lies outside the bounds specified by the OUTGOING_FRAME_LENGTH_RANGE attribute on the local PORT relationship of the channel denoted by FILE or if the frame priority specified in PRIORITY is greater than the maximum specified in the PRIORITY_LEVEL attribute on the local PORT relationship [4]. 13

**MODE_ERROR** is raised if the file handle FILE is of mode IN_FILE [1]. 14

**TIME_EXPIRED** is raised if TIME_LIMIT expires [1]. 15

**TRANSACTION_ERROR** is raised if the operation is included in a transaction, the opposite client is a SECONDARY_STORAGE node and the file handle FILE was not opened inside the transaction [3]. 16

**SECURITY_VIOLATION** is raised if the operation represents a violation of mandatory access controls. SECURITY_VIOLATION is raised only if the conditions for other exceptions are not present. 17

## 8.2.8 Receiving a frame

```
procedure READ
    (FILE          : in      FILE_TYPE;
     FRAME         : in      FRAME_ADDRESS ;
     LENGTH        :      out FRAME_LENGTH ;
     BLOCKING      : in      BOOLEAN              := FALSE;
     TIME_LIMIT    : in      CAIS_IO_DURATION : = LONG_IO_DELAY ;
     STATUS        : in      STATUS_INDICATOR  := NO_INDICATION);
```

Purpose:

This procedure reads one frame from the specified file. The frame is read from the front of the incoming queue, relative to the current process. 1

A frame is read into a block of storage units starting at address FRAME. The LENGTH returned depicts the number of significant memory storage units in the frame read. The buffer area addressed by FRAME should allow at least enough space for the maximum frame length as denoted by the INCOMING_FRAME_LENGTH attribute on the local PORT relationship to the channel denoted by FILE. 2

If the BLOCKING parameter is TRUE and the queue is empty, then the operation is blocked until the arrival of the next frame or until the specified TIME_LIMIT has elapsed. If BLOCKING is FALSE and the queue is empty, the QUEUE_EMPTY exception is raised. 3



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Parameters:

**FILE** is an open file handle identifying the channel from which the data is to be read.

**FRAME** is the address of the block of storage units into which the frame is to be read.

**LENGTH** is the number of memory storage units in the frame read.

**BLOCKING** is a Boolean value indicating whether blocking is turned on.

**TIME_LIMIT** is the time limit for waiting on the arrival of the next frame.

**STATUS** is an established status indicator.

Exceptions:

**STATUS_** ERROR is raised if FILE is not an open file handle at the time of the call [8], if the channel associated with FILE is not in the ESTABLISHED or CLOSED state [9], or if STATUS is not an established status indicator [3].

**QUEUE_EMPTY** is raised if the BLOCKING parameter is FALSE and the queue is empty [1].

**END_ERROR** is raised if the opposite client is a FILE node and the attempt to read another frame constitutes a read past the end of the file [1] or if the opposite client process has closed the protocol and the incoming queue of the current process is empty [2].

**MODE_ERROR** is raised if the file handle FILE is not of mode IN_FILE or INOUT_FILE [1].

**TIME_EXPIRED** is raised if TIME_LIMIT expires [1].

**TRANSACTION_EREOR** is raised if the opposite client is a SECONDARY_STORAGE node included in a transaction with write intent and the file handle FILE was not opened inside the transaction [3].

**SECURITY_VIOLATION** is raised if the operation represents a violation of mandatory access controls. SECURITY_VIOLATION is raised only if the conditions for other exceptions are not present.

## 8.2.9 Resetting a frame file handle

```
procedure RESET
   (FILE       : in out FILE_TYPE;
    MODE       : in     FILE_MODE;
    TIME_LIMIT : in     CAIS_IO_DURATION   := LONG_IO_DELAY;
    STATUS     : in     STATUS_INDICATOR   : = NO_INDICATIOE) ;
```

Purpose:

This procedure sets the current mode of the file handle FILE to the mode given by the MODE parameter. In effect, the CLOSE, AWAIT_CHANNEL (if the opposite client is process) and OPEN operations are performed as an atomic operation.

If the opposite client is also a process, it may continue to READ any frames remaining in its incoming queue related to the file handle being closed. However, it must RESET (or CLOSE and OPEN) its file handle in order to communicate anew with the current process through the channel associated with the file handle being reset.



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Parameters:

**FILE** is an open file handle identifying the internal file to be reset. 3

**MODE** indicates the new mode under which the file handle is to be reset. 4

**TIME_LIMIT** is the time limit for waiting for the implicit AWAIT_CHANNEL and OPEN to complete. 5

**STATUS** is an established status indicator. 6

Exceptions:

**STATUS_ERROR** is raised if FILE is not an open file handle [8] or if STATUS is not an established status indicator [3]. 7

**MODE_ERROR** is raised if MODE is incompatible with the DIRECTION attribute of the local PORT relationship as determined by Table XXXIV on page 581 [2] or if both clients are processes and specify incompatible modes as determined by Table XXXIV on page 581 [3]. 8

**TRANSACTION_ERROR** is raised if the operation is included in more than one transaction [1] or if the operation is included in a transaction and the operation modifies a node which is not included in the same transaction [2]. 9

**INTENT_VIOLATION** is raised if the node handle associated with the file handle FILE was not opened with an intent specification including at least the intents required for MODE as determined by Table XXXIV on page 581 [1]. 10

**OPEN_DENIED** is raised if a remote client process or the opposite client, if it is a process, deleted the channel, signaling the denial of the connection attempt [1] or if a protocol open issued by the opposite client process fails for protocol compatibility reasons which are not determinable by the current process [2]. 11

**TIME_EXPIRED** is raised if the operation is delayed beyond the specified time limit for reasons other than a lock conflict [1]. 12

**SECURITY_VIOLATION** is raised if the operation represents a violation of mandatory access controls. SECURITY_VIOLATION is raised only if the conditions for other exceptions are not present. 13

## 8.2.10 Determining if all messages have been read

```
function END_OF_FILE
    (FILE   : in  FILE_TYPE;
     STATUS : in  STATUS_INDICATOR := NO_INDICATION)
   return BOOLEAN;
```

Purpose:

This function returns TRUE if no more frames can be read from the incoming queue (relative to the current process) of the channel identified by the open file handle. Otherwise, the function returns FALSE. 1

No additional frames can be read from the incoming queue if the queue is empty and the opposite client is a FILE node and an attempt to read another frame would constitute a read past the end of the file, or if the queue is empty and the opposite client process has closed the protocol. 2



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Parameters:

**FILE** is an open file handle identifying the channel. 3

**STATUS** is an established status indicator. 4

Exceptions:

**STATUS_ERROR** is raised if FILE is not an open file handle [8], if the channel identified by FILE is not in the ESTABLISHED or CLOSED state [9], or if STATUS is not an established status indicator [3]. 5

**MODE_ERROR** is raised if the file handle FILE is not of mode IN_FILE or INOUT_FILE [1]. 6

**SECURITY_VIOLATION** is raised if the operation represents a violation of mandatory access controls. SECURITY_VIOLATION is raised only if the conditions for other exceptions are not present. 7

## 8.2.11 Determining if incoming queue is empty

```
function INCOMING_QUEUE_EMPTY
   (FILE   : in   FILE_TYPE;
    STATUS : in   STATUS_INDICATOR := NO_INDICATION)
   return BOOLEAN;
```

Purpose:

This function returns TRUE if no messages are currently queued on the incoming queue (relative to the current process) of the channel identified by the open file handle FILE. Otherwise, the function returns FALSE. 1

Parameters:

**FILE** is an open file handle identifying the channel. 2

**STATUS** is an established status indicator. 3

Exceptions:

**STATUS_ERROR** is raised if FILE is not an open file handle [8], if the channel identified by FILE is not in the ESTABLISHED or CLOSED state [9], or if STATUS is not an established status indicator [31. 4

**MODE_ERROR** is raised if the file handle FILE is not of mode IN_FILE or INOUT_FILE [1]. 5

**SECURITY_VIOLATION** is raised if the operation represents a violation of mandatory access controls. SECURITY_VIOLATION is raised only if the conditions for other exceptions are not present. 6



Source: https://assist.dla.mil -- Downloaded: 2026l0-26T07:54Z
Check the source to verify that this is the currentversion before use.

## 8.2.12 Determining if outgoing queue is full

```
function OUTGOING_QUEUE_FULL
    (FILE   : in    FILE_TYPE;
     STATUS : in    STATUS_INDICATOR := NO_INDICATION)
    return BOOLEAN;
```

Purpose:

This function returns TRUE if the maximum allowed number of messages are currently queued on the outgoing queue (relative to the current process) of the channel identified by the open file handle FILE. Otherwise, the function returns FALSE. 1

Parameters:

**FILE** is an open file handle identifying the channel.

**STATUS** is an established status indicator.

Exceptions:

**STATUS_ERROR** is raised if FILE is not an open file handle [8], if the channel identified by FILE is not in the ESTABLISHED or CLOSED state [9], or if STATUS is not an established status indicator [3]. 4

**MODE_ERROR** is raised if the file handle FILE is not of mode OUT_FILE or INOUT_FILE [1]. 5

**SECURITY_VIOLATION** is raised if the operation represents a violation of mandatory access controls. SECURITY_VIOLATION is raised only if the conditions for other exceptions are not present. 6

## 8.2.13 Determining the channel node associated with a file

```
procedure GET_ASSOCIATED_CHANNEL
    (CHANNEL : in out CHANNEL_NODE_TYPE;
     FILE    : in     FILE_TYPE;
     STATUS  : in     STATUS_INDICATOR := NO_INDICATION);
```

Purpose:

This procedure returns a node handle, with intent NO_ACCESS, to the CHANNEL node associated with the file handle FILE. Note that the association was made when the file was initially opened. 1

The last relationship traversed in opening the node handle CHANNEL is the same as that of the CHANNEL node handle provided when the file handle FILE was opened. See subprograms PATH_KEY (Section 5.2.20 on page 119) and PATH_RELATION (Section 5.2.21 on page 120). 2



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Parameters:

**CHANNEL** is a node handle to be opened to the associated CHANNEL node. 3

**FILE** is an open file handle identifying the internal file whose associated CHANNEL node is required. 4

**STATUS** is an established status indicator. 5

Exceptions:

**STATUS_ERROR** is raised if FILE is not an open file handle [8], if CHANNEL is open at the time of the call [1] or if STATUS is not an established status indicator [3]. 6

**SECURITY_VIOLATION** is raised if the operation represents a violation of mandatory access controls. SECURITY_VIOLATION is raised only if the conditions for other exceptions are not present. 7



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

# 8.3 Package CAIS_MAGNETIC_TAPE_IO

This package provides interfaces for the support of input and output operations on magnetic tapes. For purposes of interoperability the interfaces defined in this section should be used in accordance with level II of [ANSI 78]. 1

To use a tape drive, a file handle to the FILE node representing the tape drive must be obtained through an OPEN operation (see Section 8.3.3 on page 646). 2

When information transfer is completed, the tape is unloaded and dismounted using the UNLOAD (see Section 8.3.9 on page 651) and REQUEST_DISMOUNT (see Section 8.3.10 on page 651) operations. 3

Once a tape is dismounted, another tape may be mounted. When the user is finished utilizing the drive, the file handle on the file representing the tape on the drive should be closed through a CLOSE operation (see Section 8.3.4 on page 647). 4

Magnetic tape drive file nodes can only be created by the implementation. Implementation-defined file characteristics must be supported by the implementation and will include the densities and block sizes supported by the tape drive, whether or not a tape is mounted on the drive. 5

Character data is transferred to and from magnetic tapes in fixed length records. Each logical record is also a physical block. 6

When transferring an Ada text file to or from a magnetic tape, the text file must be read or written as blocks of Ada characters. Table XXXVI identifies the mapping that is to be used between the contents of an Ada text file and a file on magnetic tape containing an Ada text file. 7

**TABLE XXXVI: Allowed Magnetic Tape Characters**

| Allowed Characters | Representation of Characters |
|---|---|
| All printable characters | `CHARACTER(' ') .. ASCII.TILDE` |
| Horizontal Tab | `ASCII.HT` |
| Vertical Tab | `ASCII.VT` |
| Carriage Return | `ASCII.CR` |
| Line Terminator | `ASCII.LF` |
| Page Terminator | `ASCII.FF` |
| Fill Character | `ASCII.NUL` |

Use of other characters is not defined. Each block of a file may be terminated by zero or more fill characters. 8

All tape read and write operations use odd parity. 9

For magnetic tape, the exception DEVICE_ERROR maybe raised for many implementation-dependent reasons, e.g., a write ring not on the tape when attempting to write to it. 10

## 8.3.1 Protocol parameters

The IO connection model allows protocol-specific parameters to be determined during the establishment of a channel (see Section 7.3.1.3 on page 576). There are no protocol-specific parameters for MAGNETIC_TAPE_IO. 1



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

## 8.3.2 Types, subtypes, and exceptions

```
type FILE_TYPE is limited private;

type FILE_MODE is (IN_FILES OUT_FILE);
```

**FILE_TYPE** defines the type for file handles, which are used for controlling operations on tape drives. FILE_MODE describes whether a file handle is to be used for input or output; it can never be used for both. 1

```
subtype TAPE_NAME is STRING;

subtype TAPE_BLOCK is STRING;
```

**TAPE_NAME** defines a subtype for the name of a tape to be mounted. TAPE_BLOCK defines a subtype for buffers for the reading and writing of blocks of characters. 2

```
type TAPE_DRIVE_STATUS_KIND is
        (OPENED,
         MOUNT_REQUESTED,
         MOUNTED,
         LOADED,
         CLOSED);

type TAPE_POSITION_KIND is
        (BEGINNING_OF_VOLUME,
         END_OF_VOLUME,
         END_OF_TAPE,
         AFTER_TAPE_MARK,
         OTHER);

type TAPE_RECORDING_METOD_KIND is
        (NON_RETURN_TO_ZERO_INVERTED,
         PHASE_ENCODED,
         GROUP_CODED_RECORDING);
```

**TAPE_DRIVE_STATUS_KIND** defines the states of an internal magnetic tape drive file. Figure 44 shows the state transitions that may occur. The success (MOUNTED) or failure (OPENED) state of a mount request is determinedly a particular CAIS implementation. The time for the actual change of state from OPENED to MOUNT_REQUESTED for the REQUEST_MOUNT interface and the time for the actual change from MOUNTED to OPENED for the REQUEST_DISMOUNT interface are implementation-defined. In all other situations the time of the transition from one state to another is the completion of the interface call. The function STATUS should be used to determine the state of an internal magnetic tape drive file. 3

**TAPE_POSITION_KIND** describes the position of the tape on the tape drive; a value of AFTER_-TAPE_MARK means that the tape is positioned just after a tape mark. That is, a read in this position will read the next block. 4



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

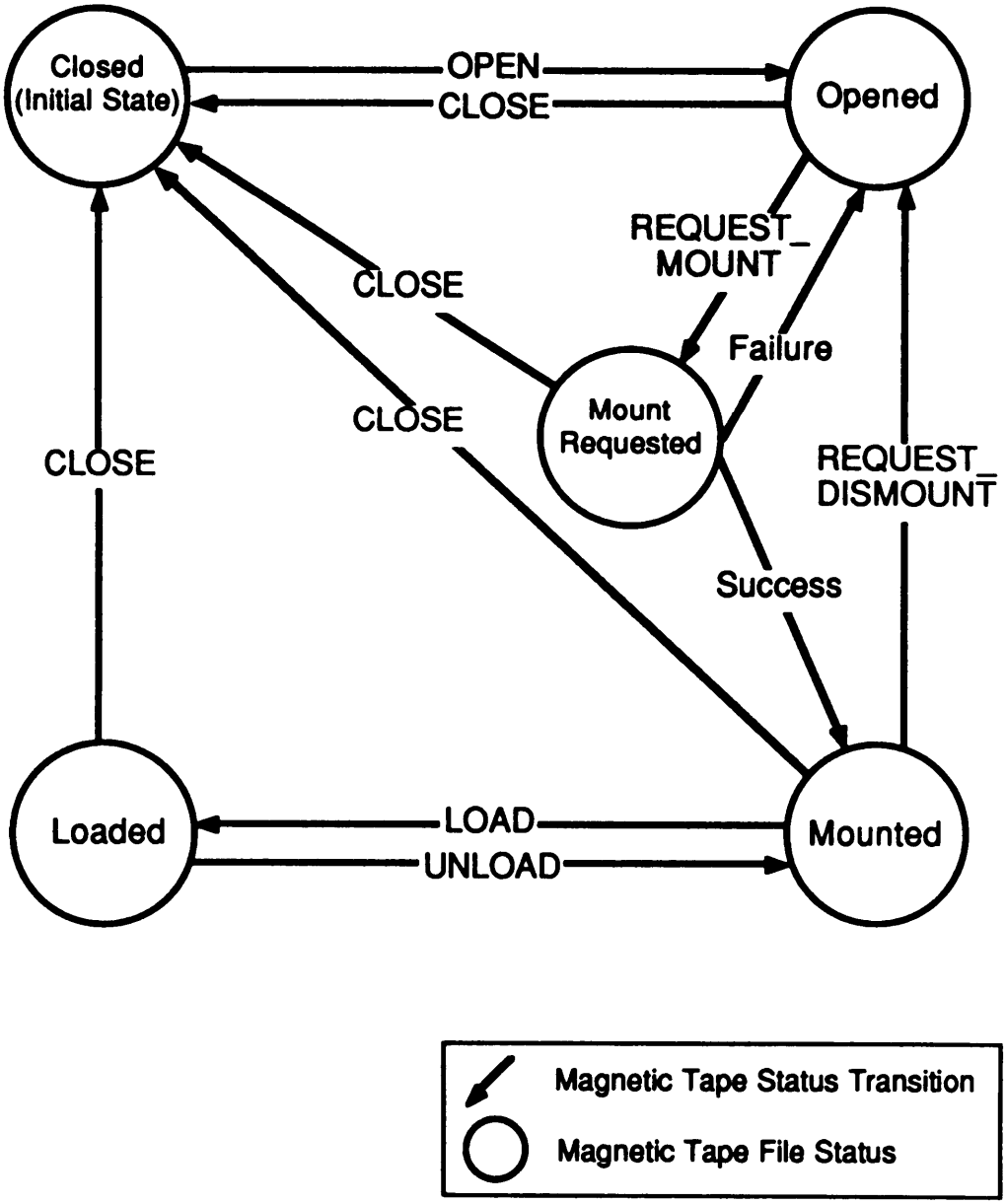


FIGURE 44. **Magnetic tape status transitions**



Source: https://assist.dla.mil - Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

**TAPE_RECORDING_METHOD_KIND** identifies the particular tape recording method and tape recording density being used. NON-RETURN_TO_ZERO_INVERTED indicates conformance to ANSI X3.22 [ANSI 73a] or ISO 1863 [ISO 76a]. PHASE_ENCODED indicates conformance to ANSI X3.39 [ANSI 73b] or ISO 3788 [ISO 76b]. GROUP_CODED-RECORDING indicates conformance to ANSI X3.54 [ANSI 76] or ISO 5652 [ISO 84]. 5

```
TAPE_STATUS_ERROR:  exception;
```

**TAPE_STATUS_ERROR** is raised if the mounted or loaded state of a tape drive is incorrect for the operation. 6

Other exceptions raised by the subprograms in CAIS_MAGNETIC_TAPE_IO are defined in the packages CAIS_DEFINITIONS, CAIS_IO_DEFINITIONS and CAIS_PRAGMATICS. 7

## 8.3.3 Opening a tape drive file handle

```
procedure OPEN
    (TAPE_DRIVE         : in out FILE_TYPE;
     CHANNEL            : in     CHANNEL_NODE_TYPE;
     OPPOSITE_CLIENT    : in     CLIENT_NODE_TYPE;
     MODE               : in     FILE_MODE;
     TIME_LIMIT         : in     CAIS_IO_DURATION := LONG_IO_DELAY;
     STATUS             : in     STATUS_INDICATOR  := NO_INDICATION) ;
```

Purpose:

This procedure opens a file handle for communicating over the channel identified by CHANNEL with the magnetic tape drive file identified by OPPOSITE_CLIENT. CAIS_MAGNETIC_TAPE_IO must be an allowed protocol of the CHANNEL node identified by CHANNEL. The mode of the file handle is set to MODE. 1

If the operation concludes successfully, the CONTROLLING_PROTOCOL attribute on the CHANNEL node is set to CAIS_MAGNETIC_TAPE_IO and the channel is in an ESTABLISHED state. 2

Parameters:

**TAPE_DRIVE** is a file handle, initially closed, to be opened. 3

**CHANNEL** is an open node handle to a CHANNEL node to which both the current process and OPPOSITE_CLIENT are client nodes. 4

**OPPOSITE_CLIENT** is an open node handle identifying a magnetic tape drive file. 6

**MODE** indicates the mode under which the file handle is to be opened. 6

**TIME_LIMIT** is the time limit for waiting for the OPEN to complete. 7

**STATUS** is an established status indicator. 8



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Exceptions:

**STATUS_ERROR** is raised if the file handle TAPE-DRIVE is an open file handle [7], if OPPOSITE_CLIENT is not an open node handle [2], if CHANNEL is not an open node handle [2], if the channel denoted by CHANNEL is not in the CONNECTED state [9], or if STATUS is not an established status indicator [3]. 9

**NODE_KIND_ERROR** is raised if the node identified by CHANNEL is not a CHANNEL node [1] or if the node identified by OPPOSITE_CLIENT is not MAGNETIC_TAPE node [1]. 10

**CLIENT_ERROR** is raised if the current process node [1] or the node identified by OPPOSITE_CLIENT [5] is not a client node of the channel identified by the CHANNEL parameter. 11

PORT_ERROR is raised if the DIRECTION attributes of the PORT relationships are missing or have incompatible values [4]. 12

**PROTOCOL_ERROR** is raised if CAIS_MAGNETIC_TAPE_IO is not an allowed protocol of both PORT relationships of the channel [1], or if the CONTROLLING_PROTOCOL attribute has a value other than NO_PROTOCOL or CAIS_MAGNETIC_TAPE_IO. 13

**MODE_ERROR** is raised if MODE is incompatible with the DIRECTION attribute of the local PORT relationship to the channel identified by CHANNEL as determined by Table XXXIV on page 581 [2]. 14

**USE_ERROR** is raised if the CAIS implementation does not support setting the file handle to the specified mode [43]. This exception is also raised if another open file handle identifies the same external file and the CAIS implementation does not support the existence of multiple file handles identifying the same external file [41]. Any such restriction must be documented in Appendix F. An implementation is allowed to raise this exception only if it is based on operating system support that does not provide this capability. 15

**TRANSACTION_ERROR** is raised if the operation is included in more than one transaction [1] or if the operation is included in a transaction and the operation modifies a node which is not included in the same transaction [2]. 16

**INTENT_VIOLATION** is raised if the node handle OPPOSITE_CLIENT was not opened with an intent specification including at least the intents required for MODE as determined by Table XXXIV on page 581 [1]. 17

**TIME_EXPIRED** is raised if the OPEN operation is delayed beyond the specified time limit for reasons other than a lock conflict [1]. 18

**SECURITY_VIOLATION** is raised if the operation represents a violation of mandatory access controls. SECURITY_VIOLATION is raised only if the conditions for other exceptions are not present. 19

Notes:

Closing the node handle OPPOSITE_CLIENT associated with the file handle TAPE_DRIVE closes the file handle. 20

## 8.3.4 Closing a tape drive file handle

```
procedure CLOSE
    (TAPE_DRIVE : in out FILE_TYPE;
     STATUS     : in     STATUS_INDICATOR := NO_INDICATION);
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Purpose:

This procedure severs any association between the internal file identified by the file handle TAPE_DRIVE and its associated channel. The file handle is closed. Closing an already closed file handle has no effect. 1

If the state of the file handle TAPE_DRIVE is LOADED, the tape represented by the file handle is unloaded and dismounted before closing. If the state of the file handle is MOUNTED or MOUNT_REQUESTED, the tape represented by the file handle is dismounted before closing. 2

The state of the channel associated with TERMINAL becomes CLOSED if presently ESTABLISHED and becomes CONNECTED if already CLOSED. If the opposite client node is a file, the charnel becomes immediately CONNECTED. If the channel becomes CONNECTED, the CONTROLLING_PROTOCOL attribute of the CHANNEL node is set to NO_PROTOCOL. 3

Parameters:

**TAPE_DRIVE** is a file handle to be closed. 4

**STATUS** is an established status indicator. 5

Exceptions:

**STATUS_ERROR** is raised if STATUS is not an established status indicator [3]. 6

**TRANSACTION_ERROR** is raised if the operation is included in a transaction and the file handle TAPE_DRIVE was not opened inside the transaction [3]. 7

## 8.3.5 Determining whether a file handle is open

```
function IS_OPEN
    (TAPE_DRIVE : in FILE_TYPE;
     STATUS     : in STATUS_INDICATON := NO_INDICATION)
    return BOOLEAN;
```

Purpose:

This function returns TRUE if the file handle is open; otherwise, it returns FALSE, 1

Parameters:

**TAPE_DRIVE** is a file handle. 2

**STATUS** is an established status indicator. 3

Exceptions:

**STATUS_ERROR** is raised if STATUS is not an established status indicator [3]. 4



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

## 8.3.6 Determining the mode of a magnetic tape drive

```
function MODE
    (TAPE_DRIVE : in FILE_TYPE;
     STATUS     : in STATUS_INDICATOR := NO_INDICATION)
    return FILE_MODE;
```

Purpose:

This function returns the mode under which TAPE_DRIVE is open.

Parameters:

**TAPE_DRIVE** is an open file handle identifying the internal file associated with the tape drive file. 2

**STATUS** is an established status indicator. 3

Exceptions:

**STATUS_ERROR** is raised if TAPE_DRIVE is not an open file handle [8] or if STATUS is not an established status indicator [3]. 4

## 8.3.7 Requesting the mounting of a tape

```
procedure REQUEST_MOUNT
    (TAPE_DRIVE          : in FILE-TYPE;
     NAME                : in TAPE_NAME;
     RECORDING_METHOD    : in TAPE_RECORDING_METHOD_KIND;
     INSTALL_WRITE_RING  : in BOOLEAN              := FALSE;
     STATUS              : in STATUS_INDICATOR := NO_INDICATION);
```

Purpose:

This procedure generates an implementation-defined request that the tape whose external name is NAME be mounted on the tape drive associated with the internal file identified by the file handle TAPE_DRIVE, the tape drive recording method be set to RECORDING_METHOD, and, if INSTALL_WRITE_RING is TRUE, that a write ring be installed on the tape before mounting. 1

Following completion of this procedure until the request is fulfilled or denied, STATUS(TAPE_-DRIVE) returns MOUNT_REQUESTED. If the request is fulfilled, the function STATUS(TAPE_-DRIVE) returns MOUNTED. If the request is denied, the function STATUS(TAPE_DRIVE) returns OPENED. 2


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Parameters:

**TAPE_DRIVE** is an open file handle identifying the internal file associated with the tape drive file.

**NAME** is an external name which identifies the tape to be mounted on the tape drive.

**RECORDING_METHOD** is the recording method to be used when writing to the tape.

**INSTALL_WRITE_RING** indicates whether or not a write ring is to be installed on the tape.

**STATUS** is an established status indicator.

Exceptions:

**STATUS_ERROR** is raised if TAPE_DRIVE is not an open file handle [8] or if STATUS is not an established status indicator [3].

## 8.3.8 Loading a tape

```
procedure LOAD
    (TAPE_DRIVE  : in FILE_TYPE;
     BLOCK_SIZE  : in CAIS_POSITIVE;
     STATUS      : in STATUS_INDICATOR : = NO_INDICATION);
```

Purpose:

This procedure loads the tape on the tape drive represented by the file associated with the internal file identified by TAPE_DRIVE. The tape is positioned after the beginning of tape mark. Following completion of this procedure, the function STATUS(TAPE_DRIVE) returns LOADED.

Parameters:

**TAPE_DRIVE** is an open file handle identifying the internal file associated with the tape drive file.

**BLOCK_SIZE** is the number of bytes to be read or written during input or output operations.

**STATUS** is an established status indicator.

Exceptions:

**STATUS_ERROR** is raised if TAPE_DRIVE is not an open file handle [8] or if STATUS is not an established status indicator [3].

**TAPE_STATUS_ERROR** is raised if STATUS(TAPE_DRIVE) is not MOUNTED [1].

**USE_ERROR** is raised if BLOCK_SIZE is less than CAIS_PRAGMATICS.MINIMUM_TAPE_BLOCK_LENGTH or greater than CAIS_PRAGMATICS.MAXIMUM_TAPE_BLOCK_LENGTH [42]. (See CAIS_PRAGMATICS, Section 12 on page 815.)


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

## 8.3.9 Unloading a tape

```
procedure INLOAD
    (TAPE-DRIVE : in FILE_TYPE;
     STATUS     : in STATUS_INDICATOR := NO_INDICATION);
```

Purpose:

This procedure unloads the tape on the tape drive file associated with the internal file identified by the file handle TAPE_DRIVE. Following completion of this procedure, the tape is positioned after the beginning of tape mark, the function STATUS(TAPE_DRIVE) will return MOUNTED, and the function POSITION(TAPE_DRIVE) will return BEGINNING_OF_VOLUME. Unloading a tape in which STATUS(TAPE_DRIVE) is MOUNTED has no effect. 1

Parameters:

**TAPE_DRIVE** is an open file handle identifying the internal file associated with the tape drive file. 2

**STATUS** is an established status indicator. 3

Exceptions:

**STATUS_ERROR** is raised if TAPE_DRIVE is not an open file handle [8] or if STATUS is not an established status indicator [3]. 4

**TAPE_STATUS_ERROR** is raised if STATUS(TAPE_DRIVE) is neither MOUNTED nor LOADED [1]. 5

## 8.3.10 Requesting the dismounting of a tape

```
procedure REQUEST_DISMOUNT
    (TAPE_DRIVE : in FILE_TYPE;
     STATUS     : in STATUS_INDICATOR := NO_INDICATION);
```

Purpose:

This procedure generates an implementation-defined request that the tape on the tape drive represented by the file associated with the internal file identified by file handle TAPE_DRIVE be removed from the drive. It makes the tape available for removal. Following the completion of this procedure, the function STATUS(TAPE_DRIVE) will return OPENED. 1

Parameters:

**TAPE_DRIVE** is an open file handle identifying the internal file associated with the tape drive file. 2

**STATUS** is an established status indicator. 3


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Exceptions:

**STATUS_ERROR** is raised if TAPE_DRIVE is not an open file handle [8] or if STATUS is not an established status indicator [3]. 4

**TAPE_STATUS_ERROR** is raised if STATUS(TAPE_DRIVE) is not MOUNTED [1]. 5

## 8.3.11 Determining the position of the tape

```
function POSITION
    (TAPE_DRIVE : in FILE_TYPE;
     STATUS     : in STATUS_INDICATOR := NO_INDICATION)
    return TAPE_POSITION_KIND;
```

Purpose:

This function returns the current position of the internal file identified by the file handle TAPE_DRIVE. 1

Parameters:

**TAPE_DRIVE** is an open file handle identifying the internal file associated with the tape drive file. 2

**STATUS** is an established status indicator. 3

Exceptions:

**STATUS_ERROR** is raised if TAPE_DRIVE is not an open file handle [8] or if STATUS is not an established status indicator [3]. 4

**TAPE_STATUS_ERROR** is raised if STATUS(TAPE_DRIVE) is not LOADED [2]. 5

## 8.3.12 Rewinding the tape

```
procedure REWIND_TAPE
    (TAPE_DRIVE : in FILE_TYPE;
     STATUS     : in STATUS_INDICATOR  : = NO_INDICATION) ;
```

Purpose:

This procedure positions the internal file identified by the file handle TAPE_DRIVE after the beginning of tape mark. Following completion of this procedure, the function POSITION(TAPE_DRIVE) returns BEGINNING_OF_VOLUME. 1


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Parameters:

**TAPE_DRIVE** is an open file handle identifying the internal file associated with the tape drive file. 2

**STATUS** is an established status indicator. 3

Exceptions:

**STATUS_ERROR** is raised if TAPE_DRIVE is not an open file handle [8] of if STATUS is not an established status indicator [3]. 4

**TAPE_STATUS_ERROR** is raised if STATUS(TAPE_DRIVE) is not LOADED [2]. 5

**SECURITY_VIOLATION** is raised if the operation represents a violation of mandatory access controls. SECURITY_VIOLATION is raised only if the conditions for other exceptions are not present. 6

## 8.3.13 Skipping tape marks

```
procedure SKIP_TAPE_MARK
    (TAPE_DRIVE : in FILE_TYPE;
     COUNT      : in CAIS_POSITIVE := 1;
     STATUS     : in STATUS_INDICATOR := NO_INDICATION);
```

Purpose:

This procedure skips COUNT tape marks on the internal file identified by the file handle TAPE_DRIVE. 1

Following a call to SKIP_TAPE_MARK, the tape is positioned immediately following the appropriate tape mark. If the end of tape mark is encountered during this operation, the function POSITION(TAPE_DRIVE) returns END_OF_TAPE; otherwise POSITION(TAPE_DRIVE) returns AFTER_TAPE_MARK. 2

If two consecutive tape marks are encountered during this operation, the tape is positioned between the two tape marks and the fiction POSITION(TAPE_DRIVE) returns END_OF_VOLUME. If at the time of the call the tape is positioned between two consecutive tape marks, the position of the tape is not changed and the function POSITION(TAPE_DRIVE) returns END_OF_VOLUME. 3

Parameters:

**TAPE_DRIVE** is an open file handle identifying the internal file associated with the tape drive file. 4

**COUNT** is the number of tape marks to skip. 5

**STATUS** is an established status indicator. 6



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Exceptions:

**STATUS_ERROR** is raised if TAPE_DRIVE is not an open file handle [8] or if STATUS is not an established status indicator [3].

**TAPE_STATUS_ERROR** is raised if STATUS(TAPE_DRIVE) is not LOADED [2].

**MODE_ERROR** is raised if the file handle TAPE_DRIVE is of mode OUT_FILE [1].

**SECURITY_VIOLATION** is raised if the operation represents a violation of mandatory access controls. SECURITY_VIOLATION is raised only if the conditions for other exceptions are not present.

## 8.3.14 Writing a tape mark

```
procedure WRITE_TAPE_MARK
    (TAPE_DRIVE : in FILE_TYPE;
     STATUS     : in STATUS_INDICATOR := NO_INDICATION);
```

Purpose:

This procedure writes a tape mark on the internal file identified by the file handle TAPE_DRIVE. If the end of tape mark is encountered during this operation, the function POSITION(TAPE_DRIVE) returns END_OF_TAPE; otherwise, POSITION(TAPE_DRIVE) returns AFTER_TAPE_MARK.

Parameters:

**TAPE_DRIVE** is an open file handle identifying the internal file associated with the tape drive file.

**STATUS** is an established status indicator.

Exceptions:

**STATUS-ERROR** is raised if TAPE_DRIVE is not an open file handle [8] or if STATUS is not an established status indicator [3].

**TAPE_STATUS_ERROR** is raised if STATUS(TAPE_DRIVE) is not LOADED [2].

**MODE_ERROR** is raised if the file handle TAPE_DRIVE is of mode IN_FILE [1].

**SECURITY_VIOLATION** is raised if the operation represents a violation of mandatory access controls. SECURITY_VIOLATION is raised only if the conditions for other exceptions are not present.

## 8.3.15 Determining the status of a magnetic tape drive

```
function  STATUS
    (TAPE_DRIVE : in FILE_TYPE;
     STATUS     : in STATUS_INDICATOR : = NO_INDICATION)
    return  TAPE_DRIVE_STATUS_KIND;
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Purpose:

This function returns the tape drive status of the file handle TAPE_DRIVE. 1

Parameters:

**TAPE_DRIVE** is an open file handle identifying the internal file associated with the tape drive file. 2

**STATUS** is an established status indicator. 3

Exceptions:

**STATUS_ERROR** is raised if TAPE_DRIVE is not an open file handle [8] or if STATUS is not an established status indicator [3]. 4

## 8.3.16 Determining the recording method of a magnetic tape

```
function RECORDING_METHOD
    (TAPE_DRIVE : in FILE_TYPE;
     STATUS     : in STATUS_INDICATOR : = NO_INDICATION)
    return TAPE_RECORDING_METHOD_KIND;
```

Purpose:

This function returns the value specified for RECORDING_METHOD during the most recent request to mount a tape on the tape drive associated with the internal file identified by the file handle TAPE_DRIVE. 1

Parameters:

**TAPE_DRIVE** is an open file handle identifying the internal file associated with the tape drive file. 2

**STATUS** is an established status indicator. 3

Exceptions:

**STATUS_ERROR** is raised if TAPE_DRIVE is not an open file handle [8] or if STATUS is not an established status indicator [3]. 4

**TAPE_STATUS_ERROR** is raised if STATUS(TAPE_DRIVE) is not MOUNTED [1]. 5

## 8.3.17 Determining whether a write ring is installed

```
function IS_WRITE_RING_INSTALLED
    (TAPE_DRIVE : in FILE_TYPE;
     STATUS     : in STATUS_INDICATOR : = NO_INDICATION)
    return BOOLEAN;
```


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Purpose:

This function returns the value specified for INSTALL_WRITE_RING during the most recent request to mount a tape on the tape drive associated with the internal file identified by the file handle TAPE_DRIVE. 1

Parameters:

**TAPE_DRIVE** is an open file handle identifying the internal file associated with the tape drive file. 2

**STATUS** is an established status indicator. 3

Exceptions:

**STATUS_ERROR** is raised if TAPE_DRIVE is not an open file handle [8] or if STATUS is not an established status indicator [3]. 4

**TAPE_STATUS_ERROR** is raised if STATUS(TAPE_DRIVE) is neither MOUNTED nor LOADED [1]. 5

## 8.3.18 Skipping blocks in a magnetic tape file

```
procedure SKIP_BLOCK
    (TAPE_DRIVE : in FILE_TYPE;
     COUNT      : in CAIS_POSITIVE := 1;
     STATUS     : in STATUS_INDICATON := NO_INDICATION);
```

Purpose:

This procedure skips COUNT blocks on the internal file identified by the file handle TAPE_DRIVE. 1

The tape is positioned COUNT blocks toward the end of the tape. If a tape mark is encountered during this operation, the tape is positioned after the tape mark and POSITION(TAPE_DRIVE) returns AFTER_TAPE_MARK. If during or after this operation the tape is positioned after the end of tape mark, POSITION(TAPE_DRIVE) returns END_OF_TAPE even if a tape mark was encountered. 2

Parameters:

**TAPE_DRIVE** is an open file handle identifying the internal file associated with the tape drive file. 3

**COUNT** is the number of blocks to skip. 4

**STATUS** is an established status indicator. 5



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Exceptions:

**STATUS_ERROR** is raised if TAPE_DRIVE is not an open file handle [8] or if STATUS is not an established status indicator [3]. 6

**TAPE_STATUS_ERROR** is raised if STATUS(TAPE_DRIVE) is not LOADED [2]. 7

**MODE_ERROR** is raised if the file handle TAPE_DRIVE is of mode OUT_FILE [1]. 8

**SECURITY_VIOLATION** is raised if the operation represents a violation of mandatory access controls. SECURITY_VIOLATION is raised only if the conditions for other exceptions are not present. 9

## 8.3.19 Reading a block from a magnetic tape file

```
procedure READ-BLOCK
    (TAPE_DRIVE     : in     FILE_TYPE;
     BLOCK          :    out TAPE_BLOCK;
     LAST           :    out CAIS_NATURAL;
     BLOCK_OVERFLOW :    out CAIS_NATURAL;
     STATUS         : in     STATUS_INDICATOR := NO_INDICATION);
```

Purpose:

This procedure reads a Mock of characters from the internal file identified by the file handle TAPE_DRIVE into the string BLOCK. The number LAST identifies the index into BLOCK of the last character read. Characters are read sequentially into BLOCK starting at BLOCK'FIRST. 1

If the block of characters in the internal file is greater than BLOCK'LENGTH, the characters in the block after BLOCK'LENGTH characters are lost. The number of characters that are lost is returned in BLOCK_OVERFLOW. 2

If during or after this operation the internal file is positioned after the end of tape mark, POSITION(TAPE_DRIVE) returns END_OF_TAPE; otherwise, POSITION(TAPE_DRIVE) returns OTHER. If during this operation a tape mark is read, POSITION(TAPE_DRIVE) returns AFTER_TAPE_MARK and LAST is equal to CAIS_NATURAL'PRED(BLOCK'FIRST). 3

Parameters:

**TAPE_DRIVE** is an open file handle identifying the internal file associated with the tape drive file. 4

**BLOCK** is a string to receive the characters that are read. 5

**LAST** is the index into BLOCK of the last character read. 6

**BLOCK_OVERFLOW** is the number of characters lost as a result of the block of characters on tape being greater than BLOCK'LENGTH. 7

**STATUS** is an established status indicator. 8


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Exceptions:

**STATUS_ERROR** is raised if TAPE_DRIVE is not an open file handle [8] or if STATUS is not an established status indicator [3]. 9

**TAPE_STATUS_ERROR** is raised if STATUS(TAPE_DRIVE) is not LOADED [2]. 10

**MODE_ERROR** is raised if the file handle TAPE_DRIVE is of mode OUT_FILE [1]. 11

**SECURITY_VIOLATION** is raised if the operation represents a violation of mandatory access controls. SECURITY_VIOLATION is raised only if the conditions for other exceptions are not present. 12

## 8.3.20 Writing a block to a magnetic tape file

```
procedure WRITE_BLOCK
    (TAPE_DRIVE : in FILE_TYPE;
     BLOCK      : in TAPE-BLOCK;
     STATUS     : in STATUS_INDICATOR : = NO_INDICATION);
```

Purpose:

This procedure writes the block of characters BLOCK to the internal file identified by the file handle TAPE_DRIVE. 1

If during or after this operation the tape is positioned after the end of tape mark, POSITION(TAPE_DRIVE) returns END_OF_TAPE; otherwise POSITION(TAPE_DRIVE) returns OTHER. 2

Parameters:

**TAPE_DRIVE** is an open file handle identifying the internal file associated with the tape drive file. 3

**BLOCK** is the string to be written. 4

**STATUS** is an established status indicator. 5

Exceptions:

**STATUS_ERROR** is raised if TAPE_DRIVE is not an open file handle [8] or if STATUS is not an established status indicator [3]. 6

**USE_ERROR** is raised if BLOCK'LENGTH is less than CAIS_PRAGMATICS. MINIMUM_TAPE_BLOCK_LENGTH or greater than CAIS_PRAGMATICS.MAXIMUM_TAPE_BLOCK-LENGTH or not equal to the block size specified when the tape was loaded [42]. (See CAIS_PRAGMATICS, Section 12 on page 815.) 7

**TAPE_STATUS_ERROR** is raised if STATUS(TAPE_DRIVE) is not LOADED [2]. 8

**MODE_ERROR** is raised if the file handle TAPE_DRIVE is of mode IN_FILE [1]. 9

**SECURITY_VIOLATION** is raised if the operation represents a violation of mandatory access controls. SECURITY_VIOLATION is raised only if the conditions for other exceptions are not present. 10


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

## 8.3.21 Resetting a magnetic tape file handle

```
procedure RESET
    (TIPE_DRIVE : in out FILE_TYPE;
     MODE       : in     FILE_MODE;
     STATUS     : in     STATUS_INDICATOR := NO_INDICATION);
```

Purpose:

This procedure sets the current mode of the file handle TAPE_DRIVE to the mode given by the MODE parameter. In effect, the file is closed and re-opened with the new mode. 1

This procedure also positions the tape after the beginning of tape mark. Following completion of this procedure, the function POSITION(TAPE_DRIVE) returns BEGINNING_OF_VOLUME. 2

Parameters:

**TAPE_DRIVE** is an open file handle identifying the tape drive file handle to be reset. 3

**MODE** indicates the new mode under which the file handle is to be reset. 4

**STATUS** is an established status indicator. 5

Exceptions:

**STATUS_ERROR** is raised if TAPE_DRIVE is not an open file handle [8] or if STATUS is not an established status indicator [3]. 6

**TAPE_STATUS_ERROR** is raised if STATUS(TAPE_DRIVE) is not LOADED [2]. 7

**MODE_ERROR** is raised if MODE is incompatible with the DIRECTION attribute of the local PORT relationship as determined by Table XXXIV on page 581 [2]. 8

**USE_ERROR** is raised if the CAIS implementation does not support resetting the file handle to the specified mode [43]. 9

**INTENT_VIOLATION** is raised if the node handle associated with the file handle TAPE_DRIVE was not opened with an intent specification including at least the intents required for MODE as determined by Table XXXIV on page 581 [1]. 10

**SECURITY_VIOLATION** is raised if the operation represents a violation of mandatory access controls. SECURITY_VIOLATION is raised only if the conditions for other exceptions are not present. 11

## 8.3.22 Determining the channel node associated with a file

```
procedure GET_ASSOCIATED_CHANNEL
    (CHANNEL     : in out CHANNEL_NODE_TYPE;
     TAPE_DRIVE : in     FILE_TYPE;
     STATUS     : in     STATUS_INDICATOR := NO_INDICATION);
```


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Purpose:

This procedure returns a node handle, with intent NO_ACCESS, to the CHANNEL node associated with the file handle TAPE_DRIVE. Note that the association was made when the file was initially opened. 1

The last relationship traversed in opening the node handle CHANNEL is the same as that of the CHANNEL node handle provided when the file handle TAPE_DRIVE was opened. See subprograms PATH_KEY (Section 5.2.20 on page 119) and PATH_RELATION (Section 5.2.21 on page 120). 2

Parameters:

**CHANNEL** is a node handle to be opened to the associated CHANNEL node. 3

**TAPE_DRIVE** is an open file handle identifying the internal file whose associated CHANNEL node is required. 4

**STATUS** is an established status indicator. 5

Exceptions:

**STATUS_ERROR** is raised if TAPE_DRIVE is not an open file handle [8], if CHANNEL is open at the time of the call [1] or if STATUS is not an established status indicator [3]. 6

**SECURITY_VIOLATION** is raised if the operation represents a violation of mandatory access controls. SECURITY_VIOLATION is raised only if the conditions for other exceptions are not present. 7


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

# 8.4 Package CAIS_DIRECT_IO

This package provides facilities for directly accessing data elements in CAIS files. [1815A] defines what constitutes an element. Files written with CAIS_DIRECT_IO are also readable by CAIS_-SEQUENTIAL_IO if the two generic packages are instantiated with the same data type. 1

The package specification and semantics of CAIS_DIRECT_IO are comparable to those of the [1815A] package DIRECT_IO. The subprograms of CAIS_DIRECT_IO correspond to the subprograms in [1815A] DIRECT_IO as follows: 2

**CREATE** replaces [1815A] DIRECT_IO_CREATE. 3

**OPEN** replaces [1815A] DIRECT_IO_OPEN. 4

**CLOSE** replaces [1815A] DIRECT_IO_CLOSE. 5

**DELETE** does not exist in the package CAIS_DIRECT_IO. 6

**RESET** replaces [1815A] DIRECT_IO_RESET. 7

**NAME** does not exist in the package CAM-DIRECT-IO. 8

**FORM** does not exist in the package CAIS_DIRECT_IO. 9

**SYNCHRONIZE** is an additional subprogram that does not exist in [1815A] DIRECT_IO. 10

**GET_ASSOCIATED_CHANNEL** is an additional subprogram that does not exist in [1815A] DIRECT_IO. 11

All other subprograms in [1815A] DIRECT_IO are also in CAIS_DIRECT_IO and have the same syntax and semantics with the following exceptions: 12

1. All types and subtypes are CAIS-defined types and subtypes. 13

2. Each subprogram has a final added STATUS parameter—an optional in parameter of type STATUS_INDICATOR defaulting to NO_INDICATION. The exception STATUS_ERROR is raised when the STATUS parameter is not an established status indicator. 14

The exceptions raised by the subprograms in CAIS_DIRECT_IO are defined in the packages CAIS_-DEFINITIONS, CAIS_IO_DEFINITIONS and CAIS_PRAGMATICS. 15

## 8.4.1 Protocol parameters

The IO connection model allows protocol-specific parameters to be determined during the establishment of a channel (see Section 7.3.1.3 on page 576). For CAIS_DIRECT_IO, the only protocol-specific parameter is ELEMENT_SIZE. The value of the ELEMENT_SIZE attribute on a PORT relationship reflects the number of memory storage units required for each element passed across the channel. The values of the ELEMENT_SIZE attributes on the two PORT relationships involved in a channel must be equal. 1


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

## 8.4.2 Definition of types

```
type FILE_TYPE is limited private;

typo FILE_MODE is (IN_FILE, INOUT_FILE, OUT_FILE);
```

**FILE_TYPE** describes the type for file handles for all direct input and output operations. 1

**FILE_MODE** indicates whether input operations, output operations, or both can be performed on the direct file handle. The values for FILE-MODE are the same as [1815A] and correspond to the following cases: 2

a. IN_FILE corresponds to the case where only reading is to be performed. 3

b. INOUT_FILE corresponds to the case where both reading and writing are to be performed. 4

c. OUT_FILE corresponds to the case where only writing is to be performed. 5

## 8.4.3 Creating a direct file

```
procedure CREATE
    (NODE                    : in out SECONDARY_STORAGE_NODE_TYPE;
     FILE                    : in out FILE_TYPE;
     BASE                    : in     MODE_TYPE;
     KEY                     : in     RELATIONSHIP_KEY        := EMPTY_KEY_LIST;
     RELATION                : in     RD_TYPE;
     INTENT                  : in     INTENT_ARRAY            :=  (l=>WRITE);
     MODE                    : in     FILE_MODE               := INOUT_FILE;
     ATTRIBUTES              : in     ATTRIBUTE_LIST          := EMPTY_LIST;
     MAXIMUM_FILE_SIZE       : in     CAIS_NATURAL            := UHBOUNDED_FILE_SIZE;
     DISCRETIONARY_ACCESS : in     DISCRETIONARY_ACCESS-LIST  :=  ALL-RIGHTS;
     MANDATORY_ACCESS        : in     MANDATORY_ACCESS_LIST   :=  EMPTY_LIST;
     KIND                    : in     NODE_KIND_NODE_TYPE     := SECONDARY_STORAGE_NKD;
     PROCESS_PORT_KEY        : in     RELATIONSHIP_KEY        := EMPTY_KEY_LIST;
     PROCESS_PORT_RELATION : in     PORT_RD_TYPE            :=  PORT_RD;
     FILE_PORT_KEY           : in     RELATIONSHIP_KEY        := EMPTY_KEY_LIST;
     FILE_PORT_RELATION      : in     PORT_RD_TYPE            :=  PORT_RD;
     CHANNEL_PARENNT         : in     PATHNAME                :=  CURRENT_PROCESS;
     STATUS                  : in     STATUS_INDICATOR        :=  NO_INDICATION;
     MEDIUM_RESOURCE         : in     MEDIUM_NODE_TYPE        :=  DEFAULT_MEDIUM);
```

Purpose:

This procedure creates a SECONDARY_STORAGE node and installs the primary relationship to it as well as the corresponding reverse relationship to the node identified by BASE. A SECONDARY_STORAGE node is a FILE node whose contents are stored on a secondary storage medium such as a disk. The primary relationship is of the relation RELATION with the key KEY, The MEDIUM_RESOURCE parameter specifies the medium resource on which the newly 1



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

created SECONDARY_STORAGE node will reside. An open node handle to the created SECONDARY_STORAGE node with intent as specified by the INTENT parameter is returned in NODE.

The ATTRIBUTES parameter provides initial values for attributes of the SECONDARY_STORAGE node to be created. The MAXIMUM_FILE_SIZE parameter provides the value for the attribute MAXIMUM-FILE-SIZE with a value of zero indicating unrestricted size. The DISCRETIONARY_ACCESS parameter specifies initial access control information to be established for the created node. 2

The MANDATORY_ACCESS parameter specifies the object classification label with which the FILE node is to be created. If its value is the empty list, the node inherits the subject classification of the creating process as its object classification. Otherwise, it must be an unnamed list consisting of an identifier item and, optionally, an unnamed list of identifier items (see Table IX, page 256). 3

KIND is an open node handle identifying the node kind for the new node. It must identify a SECONDARY_STORAGE node kind. It may specify either an NKD or an NKI. If KIND identifies an NKI, the NKI is made the current adopted interpretation for the node handle NODE. 4

The procedure creates a CHANNEL node and installs the primary CHANNEL relationship to it as well as the corresponding reverse CHANNEL-OF relationship to the node identified by CHANNEL_PARENT. The relation definition nodes PROCESS_PORT_RELATION and FILE_PORT_RELATION are used to determine the NKD for the CHANNEL node (see CREATE_CHANNEL). A secondary PROCESS_PORT_RELATION relationship is created emanating from the current PROCESS node to the created CHANNEL node with the key PROCESS_ PORT_KEY. A secondary FILE_PORT_RELATION relationship is created emanating from the created SECONDARY_ STORAGE node to the created CHANNEL node with the key FILE_PORT_KEY. The OBJECT_CLASSIFICATION attribute on the CHANNEL node is set to the SUBJECT_CLASSIFICATION attribute of the current PROCESS node. An ACCESS relationship with a GRANT attribute of ((ALL_RIGHTS)) is created which emanates from the newly created CHANNEL node and terminates at the default GROUP node of the current user. The newly created CHANNEL node resides on the default medium resource for the current processor. The ALLOWED_PROTOCOLS attribute on each of the PORT relationships is set to the value (CAIS_DIRECT_IO). 5

The DIRECTION attribute of the process PORT relationship is set to IN when MODE is IN_FILE, OUT when MODE is OUT_FILE, and IN_AND_OUT when MODE is INOUT_FILE. The DIRECTION attribute of the file PORT relationship is OUT when MODE is IN_FILE, IN when MODE is OUT_FILE, and IN_AND_OUT when MODE is INOUT_FILE. The CONTROLLING_PROTOCOL attribute on the CHANNEL node is set to CAIS_DIRECT_IO and the channel is in an ESTABLISHED state. 6

A direct file handle used for communicating over the created channel to the created SECONDARY_STORAGE node is opened and returned in FILE. Each element of the file is directly addressable by an index. The mode of the file handle is set to MODE. 7

Parameters:

**NODE** is a node handle, initially closed, to be opened to the newly created node. 8

**FILE** is a file handle, initially closed, to be opened. 9

**BASE** is an open node handle to the node which will be the source node of the primary relationship to the new node. 10



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

**KEY** is the relationship key designator of the primary relationship to be created.

**RELATION** is an open node handle to the RD of the primary relationship to be created.

INTENT is the intent of subsequent operations on the node; the actual parameter takes the form of an array aggregate.

**MODE** indicates the mode under which the file handle is to be opened.

**ATTRIBUTES** is an empty or named list whose elements are used to establish initial values for attributes of the newly created node; each named item of the list specifies an attribute name and the value to be given to that attribute.

**MAXIMUM_FILE_SIZE** defines the value for the predefine attribute MAXIMUM_FILE_-SIZE.

**DISCRETIONARY_ACCESS** is the initial access control information associated with the created node; it is the value of the GRANT attribute of the access relationship to the user's default group node (see Section 5.4 on page 255).

**MANDATORY_ACCESS** is a list defining the classification label for the created node (see Section 5.4, page 255).

**KIND** is an open node handle identifying the node kind for the new node.

**PROCESS_PORT_KEY** is the key value that the local port relationship is to have.

**PROCESS_PORT_RELATION** is an open node handle to the local port's RD.

**FILE_PORT_KEY** is the key value that the opposite port relationship is to have.

**FILE_PORT_RELATION** is an open node handle to the opposite port's RD.

**CHANNEL_PARENT** is a pathname identifying the node that will be the parent of the created CHANNEL node.

**STATUS** is an established status indicator.

**MEDIUM_RESOURCE** is an open node handle identifying a MEDIUM node.

Exceptions:

**PATHNAME_SYNTAX_ERROR** is raised if the relationship key list given in KEY, PRO-CESS_PORT_KEY, or FILE_PORT_KEY is syntactically illegal [2], or if the pathname given in CHANNEL_ PARENT is syntactically illegal [1] (see Table I, page 52).

**NAME_ERROR** is raised if the parent of the NKI identified by KIND is inaccessible or if any node traversed on the path specified by CHANNEL_PARENT is unobtainable or inaccessible [1].

**EXISTING_NODE_ERROR** is raised if a node already exists with the identification given by BASE, KEY and RELATION [1].

**SYNTAX_ERROR** is raised if the ATTRIBUTES parameter [8], the DISCRETIONARY_-ACCESS parameter [10], or the MANDATORY_ACCESS parameter [9] is syntactically il-legal.

**NOT_IN_VIEW_ERROR** is raised if a relation or key name specified in KEY, PROCESS_-PORT_KEY, FILE_PORT_KEY or CHANNEL_PARENT cannot be translated [1] or if KEY, PROCESS_PORT_KEY or FILE_PORT_KEY or a relationship key list specified in CHANNEL_PARENT consists of a single relationship key designator and no key of the re-lation identified by RELATION, PROCESS_PORT_RELATION, FILE_PORT_RELATION or the relation name given in the path element of CHANNEL_PARENT is visible, respec-tively [5].



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

**AMBIGUOUS_VIEW_ERROR** is raised if translation of a key name specified in KEY, PROCESS_PORT_KEY, FILE_PORT_KEY or CHANNEL_PARENT is ambiguous [1] or if KEY, PROCESS_PORT_KEY, FILE_PORT_KEY or a relationship key list specified in CHANNEL_PARENT consists of a single relationship key designator and more than one key of the relation identified by RELATION, PROCESS_PORT_RELATION, FILE_PORT_-RELATION or the relation name given in the path element of CHANNEL_PARENT is visible, respectively [4]. 32

**PREDEFINED_RELATION_ERROR** is raised if RELATION identifies a CAIS-controlled relation [1]. 33

**RELATIONSHIP_ERROR** is raised if any path element of CHANNEL_PARENT identifies more than one relationship [1]; if the relationship identified by any path element of CHANNEL_PARENT does not exist [2]; if a relationship with the identification given by PROCESS_PORT_RELATION and PROCESS_PORT_KEY already emanates from the current PROCESS node [3]; if KEY, PROCESS_PORT_KEY or FILE_PORT_KEY specifies a value for the same key more than once [5]; or if RELATION or its corresponding reverse relation has no unique key set [6]. RELATIONSHIP_ERROR is raised only if the conditions for NAME_ERROR are not present. 34

**PREDEFINED_ATTRIBUTE_ERROR** is raised if any attribute name given by the ATTRIBUTES parameter is the name of a CAIS-controlled attribute [1]. 35

**ATTRIBUTE_ERROR** is raised if ATTRIBUTES specifies a value for the same attribute more than once [6]. 36

**USE_ERROR** is raised if the value for the MANDATORY-ACCESS parameter is semantically illegal [10]. 37

**STATUS_ERROR** is raised if any of BASE, RELATION, KIND, PROCESS_PORT_RELATION, FILE_PORT_RELATION, or MEDIUM_RESOURCE is not an open node handle [2], if NODE is an open node handle [1], if FILE is an open file handle [7], or if STATUS is not an established status indicator [3]. 38

**NODE_KIND_ERROR** is raised if KIND does not identify an NKD or NKI [1], if the NKD identified through KIND is not an NKD for a SECONDARY_STORAGE node [2], if the NKD identified through KIND is an INTRODUCED_NODE NKD [4], if RELATION, PROCESS_PORT_RELATION, or FILE_PORT_RELATION does not identify an RD [1], if PROCESS_PORT_RELATION or FILE_PORT_RELATION does not identify an RD for a PORT relation [3], or if MEDIUM_RESOURCE does not identify a medium resource node [1]. 39

**CONFORMANCE_ERROR** is raised if the operation would result in an attribute, node, or relationship not conforming to its definition or the constraints in force on that definition [1, 2,4, 7-10, 11]. 40

**RESOURCE_ALLOCATION_ERROR** is raised if the medium resource identified through MEDIUM_RESOURCE is not allocatable [1]. 41

**RESOURCE_AVAILABILITY_ERROR** is raised if the medium resource identified through MEDIUM_RESOURCE is not available [1]. 42

**TRANSACTION_ERROR** is raised if the operation is included in more than one transaction [1] or if the operation is included in a transaction and the operation modifies a node which is not included in the same transaction [2]. 43

**LOCK_ERROR** is raised if a node handle to the parent of the NKI specified by KIND cannot 44



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

be opened with intent READ_RELATIONSHIPS due to outstanding locks in conflict with that intent, if any node traversed on the path specified by the pathname CHANNEL_ - PARENT cannot be opened with intent READ_RELATIONSHIPS due to outstanding locks in conflict with that intent, or if a node handle to the node identified by CHANNEL_ - PARENT cannot be opened with intent APPEND_RELATIONSHIPS due to outstanding locks in conflict with that intent [1].

**INTENT_VIOLATION** is raised if BASE was not opened with an intent establishing the right to append relationships [1]. 46

**PORT_ERROR** is raised if the DIRECTION attributes of the PORT relationships are missing or have incompatible values [2], or if an NKD for a CHANNEL node cannot be found that permits a node of that kind to be the target of relationships of both of the specified PORT relations [1]. 46

**PROTOCOL_ERROR** is raised if CAIS_DIRECT_IO is not an allowed protocol of both PORT relationships to CHANNEL [1], or if a required protocol parameter attribute is not present on one or both PORT relationships at the time of the call [5], if the value of the CONTROLLING_PROTOCOL parameter is not CAIS_DIRECT_IO [2], or if the values of the ELEMENT_SIZE attributes of both PORT relationships are not the same [3]. 47

**MODE_ERROR** is raised if the value of the MODE parameter is incompatible with the value of the DIRECTION attribute of the local PORT relationship as defined in Table XXXIV on page 581 [2]. 48

**ACCESS_VIOLATION** is raised if the discretionary access control rights of the current process are insufficient to gain access with CREATE_INSTANCES intents to the NKD identified through KIND [1] or with READ_RELATIONSHIPS intents to the parent of the NKI specified by KIND or with APPEND_CONTENTS intent to the MEDIUM node identified by MEDIUM_RESOURCE [1] or with READ_RELATIONSHIPS intent to the nodes on the path given in CHANNEL_PARENT [1] or with APPEND_RELATIONSHIPS intent to the node identified by CHANNEL_PARENT [1]. ACCESS_VIOLATION is raised only if the conditions for NAME_ERROR are not present. 49

**SECURITY_VIOLATION** is raised if the operation represents a violation of mandatory access controls. SECURITY_VIOLATION is raised only if the conditions for other exceptions are not present. 50

Additional Interfaces:

```
procedure CREATE
   (NODE                  : in out SECONDARY_STORAGE_NODE_TYPE;
    FILE                  : in out FILE_TYPE;
    BASE                  : in     NODE_TYPE;
    KEY                   : in     RELATIONSHIP_KEY          := EMPTY_KEY_LIST;
    RELATION              : in     RELATION_NAME             := DEFAULT_RELATION;
    INTENT                : in     INTENT_ARRAY              := (l=>WRITE);
    MODE                  : in     FILE_MODE                 := INOUT_FILE;
    ATTRIBUTES            : in     ATTRIBUTE_LIST            := EMPTY_LIST;
    MAXIMUM_FILE_SIZE     : in     CAIS_NATURAL              := UNBOUNDED_FILE_SIZE;
    DISCRETIONARY_ACCESS : in     DISCRETIONARY_ACCESS_LIST : = ALL_RIGHTS;
    MANDATORY_ACCESS      : in     MANDATORY_ACCESS_LIST : = EMPTY_LIST;
    STATUS                : in     STATUS_INDICATOR          := NO_INDICATION;
```



Source: https://assist.dla.mil -- Downloaded: 202610-26T07:54Z
Check the source to verify that this is the current version before use.

```
        MEDIUM_RESOURCE      : in    MEDIUM_MODE_TYPE      := DEFAULT_MEDIUM)
is
    RELATION_RD : RD_TYPE;
begin
    DEFINITION_OF_RELATION(RELATION_RD, BASE, RELATION, STATUS);

    CREATE (MODE, FILE, BASE, KEY, RELATION_RD, INTENT, MODE, ATTRIBUTES,
            MAXIMUM_FILE_SIZE, DISCRETIONARY_ACCESS, MANDATORY_ACCESS,
            STATUS => STATUS, MEDIUM_RESOURCE => MEDIUM_RESOURCE);
    CLOSE (RELATION_RD);
exception
    when others =>
        CLOSE (RELATION_RD);
        raise;
end CREATE;
```

## 8.4.4 Opening a direct file handle

```
procedure OPEN
    (FILE           : in out FILE_TYPE;
     CHANNEL        : in     CHANNEL_NODE_TYPE;
     OPPOSITE_CLIENT : in    CLIENT_NODE_TYPE;
     NODE           : in     FILE_NODE;
     TIME_LIMIT     : in     CAIS_IO_DURATION := LONG_IO_DELAY;
     STATUS         : in     STATUS_INDICATOR  := NO_INDICATION);
```

Purpose:

This procedure opens a direct file handle for communicating over the channel identified by CHANNEL with the node identified by OPPOSITE_CLIENT. CAIS_DIRECT_IO must be an allowed protocol of the CHANNEL node identified by CHANNEL. Each element of the file is directly addressable by an index. The mode of the file handle is set to MODE. 1

If the operation concludes successfully, the CONTROLLING_PROTOCOL attribute on the CHANNEL node is set to CAIS_DIRECT_IO and the channel is in an ESTABLISHED state. 2

Parameters:

**FILE** is a file handle, initially closed, to be opened. 3

**CHANNEL** is an open node handle to a CHANNEL node to which both the current process and OPPOSITE_CLIENT are client nodes. 4

**OPPOSITE_CLIENT** is an open node handle to the node to be accessed. 6

**MODE** indicates the mode under which the file handle is to reopened. 6

**TIME_LIMIT** is the time limit for waiting for the OPEN to complete. 7

**STATUS** is an established status indicator. 8



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Exceptions:

**STATUS_ERROR** is raised if FILE is an open file handle [7], if OPPOSITE_CLIENT is not an open node handle [2], if CHANNEL is not an open node handle [2], if the channel denoted by CHANNEL is not in the CONNECTED state [9] or if STATUS is not an established status indicator [3]. 9

**NODE_KIND_ERROR** is raised if the node identified by CHANNEL is not a CHANNEL node [1]. 10

**CLIENT_ERROR** is raised if the current process [1] or the node identified by OPPOSITE_ - CLIENT [5] is not a client node of the channel identified by the CHANNEL parameter. 11

**PORT_ERROR** is raised if the DIRECTION attributes of the PORT relationships are missing or have incompatible values [4]. 12

**PROTOCOL_ERROR** is raised if CAIS_DIRECT_IO is not an allowed protocol of both PORT relationships to CHANNEL [1], if the CONTROLLING_PROTOCOL attribute has a value other than NO_PROTOCOL or CAIS_DIRECT_IO [2], or if the values of the ELEMENT_SIZE attributes of both PORT relationships are not the same [3]. 13

**MODE_ERROR** is raised if MODE is incompatible with the DIRECTION attribute of the local PORT relationship to the channel identified by CHANNEL as determined by Table XXXIV on page 581 [2] or if both clients are processes and specify incompatible modes as determined by Table XXXIV on page 581 [3]. 14

**USE_ERROR** is raised if another open file handle identifies the same external file and the CAIS implementation does not support the existence of multiple file handles identifying the same external file [41]. Any such restriction must be documented in Appendix F. An implementation is allowed to raise this exception only if it is based on operating system support that does not provide this capability. 15

**TRANSACTION_ERROR** is raised if the operation is included in more than one transaction [1] or if the operation is included in a transaction and the operation modifies a node which is not included in the same transaction [2]. 16

**INTENT_VIOLATION** is raised if the node handle OPPOSITE_CLIENT was not opened with an intent specification including at least the intents required for MODE as determined by Table XXXIV on page 581 [1]. 17

**OPEN_DENIED** is raised if a remote client process or the opposite client, if it is a process, deleted the the channel, signaling the denial of the connection attempt [1] or if a protocol open issued by the opposite client process fails for protocol compatibility reasons which are not determinable by the current process [2]. 18

**TIME_EXPIRED** is raised if the OPEN operation is delayed beyond the specified time limit for reasons other than a lock conflict [1]. 19

**SECURITY_VIOLATION** is raised if the operation represents a violation of mandatory access controls. SECURITY_VIOLATION is raised only if the conditions for other exceptions are not present. 20

Notes:

Closing the node handle OPPOSITE_CLIENT associated with the file handle FILE closes the file handle. 21



Source: https://assist.dla.mil -- Downloaded: 202610-26T07:54Z
Check the source to verify that this is the current version before use.

## 8.4.5 Closing a direct file handle

```
procedure CLOSE
    (FILE   : in out FILE_TYPE;
     STATUS : in     STATUS_INDICATOR := NO_INDICATION);
```

Purpose:

This procedure severs the association between the internal file identified by the file handle FILE and its associated channel. An implicit SYNCHRONIZE operation is performed and the file handle is closed. Closing an already closed file handle has no effect. 1

If the initial state of the associated channel is CLOSED, it will enter the CONNECTED state. If the opposite client node is a DEVICE or SECONDARY_STORAGE node, the associated channel will enter the CONNECTED state. If the initial state of the associated channel is ESTABLISHED and the opposite client node is a GATEWAY or PROCESS node, the channel will enter the CLOSED state. If the channel enters the CONNECTED state, the CONTROLLING_-PROTOCOL attribute of the CHANNEL node is set to NO_PROTOCOL. 2

Parameters:

**FILE** is a file handle to be closed. 3

**STATUS** is an established status indicator. 4

Exceptions:

**STATUS_ERROR** is raised if STATUS is not an established status indicator [3]. 5

**TRANSACTION_ERROR** is raised if the operation is included in a transaction and the file handle FILE was not opened inside the transaction [3]. 6

## 8.4.6 Resetting a direct file handle

```
procedure RESET
    (FILE         : in out FILE-TYPE;
     MODE         : in     FILE_MODE;
     TIME_LIMIT   : in     CAIS_IO_DURATION := LONG_IO_DELAY;
     STATUS       : in     STATUS_INDICATOR := NO_INDICATION);
```

Purpose:

This procedure sets the current mode of the file handle FILE to the mode given by the MODE parameter. In effect, the CLOSE, AWAIT_CHANNEL (if the opposite client is a process), and OPEN operations are performed as an atomic operation. 1

The given internal file is positioned so that reading from or writing to its elements is restarted from the beginning of the internal file. This implies that the current index is set to one. 2



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Parameters:

**FILE** is an open file handle identifying the internal file to be reset.

**MODE** indicates the new mode under which the file handle is to be reset.

**TIME_LIMIT** is the time limit for waiting for the implicit AWAIT-CHANNEL and OPEN to complete.

**STATUS** is an established status indicator.

Exceptions:

**STATUS_ERROR** is raised if FILE is not an open file handle [8] or if STATUS is not an established status indicator [3].

**MODE_ERROR** is raised if MODE is incompatible with the DIRECTION attribute of the local PORT relationship as determined by Table XXXIV on page 581 [2] or if both clients are processes and specify incompatible modes as determined by Table XXXIV on page 581 [3].

**USE_ERROR** is raised if the CAIS implementation does not support resetting the file handle to the specified mode [43].

**TRANSACTION_ERROR** is raised if the operation is included in more than one transaction [1] or if the operation is included in a transaction and the operation modifies a node which is not included in the same transaction [z].

**INTENT_VIOLATION** is raised if the node handle associated with the file handle FILE was not opened with an intent specification including at least the intents required for MODE as determined by Table XXXIV on page 581 [1].

**OPEN_DENIED** is raised if a remote client process or the opposite client, if it is a process, deleted the channel, signaling the denial of the connection attempt [1] or if a protocol open issued by the opposite client process fails for protocol compatibility reasons which are not determinable by the current process [2].

**TIME_EXPIRED** is raised if the operation is delayed beyond the specified time limit for reasons other than a lock conflict [1].

**SECURITY_VIOLATION** is raised if the operation represents a violation of mandatory access controls. SECURITY_VIOLATION is raised only if the conditions for other exceptions are not present.

## 8.4.7 Synchronizing internal file with FILE node contents

```
procedure SYNCHRONIZE
   (FILE   : in     FILE_TYPE;
    STATUS : in     STATUS_INDICATOR := NO_INDICATION);
```

Purpose:

This procedure forces all data that has been written to the internal file identified by FILE to be transmitted to the contents of the external file with which it is associated.



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Parameters:

**FILE** is an open file handle identifying the internal file to be synchronized. 2

**STATUS** is an established status indicator. 3

Exceptions:

**STATUS_ERROR** is raised if FILE is not an open file handle [8] or if STATUS is not an established status indicator [3]. 4

**MODE_ERROR** is raised if the file handle FILE is of mode IN_FILE [1]. 5

Notes:

For all write operations in the CAIS, the conditions upon which data are transferred from an internal file to the contents of a FILE node are implementation-dependent. Data in the internal file of a process are inaccessible to other processes. This procedure ensures that the data in the internal file and the data in the contents of the FILE node coincide. 6

## 8.4.8 Determining the channel node associated with a file

```
procedure GET_ASSOCIATED_CHANNEL
    (CHANNEL : in out CHANNEL_NODE_TYPE;
     FILE    : in     FILE_TYPE;
     STATUS  : in     STATUS_INDICATOR := NO_INDICATION) ;
```

Purpose:

This procedure returns a node handle, with intent NO_ACCESS, to the CHANNEL node associated with the file handle FILE. Note that the association was made when the file handle was initially opened. 1

The last relationship traversed in opening the node handle CHANNEL is the same as that of the CHANNEL node handle provided when the file handle FILE was opened. See subprograms PATH-KEY (Section 5.2.20 on page 119) and PATH_RELATION (Section 5.2.21 on page 120). 2

Parameters:

**CHANNEL** is a node handle, initially closed, to be opened to the associated CHANNEL node. 3

**FILE** is an open file handle identifying the internal file whose associated CHANNEL node is required. 4

**STATUS** is an established status indicator. 5

Exceptions:

**STATUS_ERROR** is raised if FILE is not an open file handle [8], if CHANNEL is open at the time of the call [1] or if STATUS is not an established status indicator [3]. 6

**SECURITY_VIOLATION** is raised if the operation represents a violation of mandatory access controls. SECURITY_VIOLATION is raised only if the conditions for other exceptions are not present. 7



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

# 8.5 Package CAIS_SEQUENTIAL_IO

This package provides facilities for sequentially accessing data elements in CAIS files. [1815A] defines what constitutes an element. 1

The package specification and semantics of CAIS_SEQUENTIAL_IO are comparable to those of the [1815A] package SEQUENTIAL_IO. The subprograms of CAIS_SEQUENTIAL_IO correspond to the subprograms in [1815A] SEQUENTIAL_IO as follows: 2

**CREATE** replaces [1815A] SEQUENTIAL_IO.CREATE. 3

**OPEN** replaces [1815A] SEQUENTIAL_IO.OPEN. 4

**CLOSE** replaces [1815A] SEQUENTIAL_IO.CLOSE. 5

**DELETE** does not exist in the package CAIS_SEQUENTIAL_IO. 6

**RESET** replaces [1815A] SEQUENTIAL_IO.RESET. 7

**NAME** does not exist in the package CAIS_SEQUENTIAL_IO. 8

**FORM** does not exist in the package CAIS_SEQUENTIAL_IO. 9

**SYNCHRONIZE** is an additional subprogram that does not exist in [1815A] SEQUENTIAL_IO. 10

**GET_ASSOCIATED_CHANNEL** is an additional subprogram that does not exist in [1815A] SEQUENTIAL_IO. 11

All other subprograms in [1815A] SEQUENTIAL_IO are also in CAIS_SEQUENTIAL_IO and have the same syntax and semantics with the following exceptions: 12

1. All types and subtypes are CAIS-defined types and subtypes. 13
2. Each subprogram has a final added STATUS parameter—an optional in parameter of type STATUS_INDICATOR defaulting to NO_INDICATION. The exception STATUS_ERROR is raised when the STATUS parameter is not an established status indicator. 14
3. All operations on file handles of mode APPEND_FILE should be the same as those of mode OUT_FILE. 15

The exceptions raised by the subprograms in CAIS_SEQUENTIAL_IO are defied in the packages CAIS_DEFINITIONS, CAIS_IO_DEFINITIONS and CAIS_PRAGMATICS. 16

## 8.5.1 Protocol parameters

The IO connection model allows protocol-specific parameters to be determined during the establishment of a channel (see Section 7.3.1.3 on page 576). For CAIS_SEQUENTIAL_IO, the only protocol-specific parameter is ELEMENT_SIZE. The value of the ELEMENT_SIZE attribute on a PORT relationship reflects the number of memory storage units required for each element passed across the channel. The values of the ELEMENT_SIZE attributes on the two PORT relationships involved in a channel must be equal. 1



Source: https://assist.dla.mil -- Downloaded: 202610-26T07:54Z
Check the source to verify that this is the current version before use.

## 8.5.2 Definition of types

```
typo FILE_TYPE is limited private;

typo FILE.MODE is (IN_FILE, OUT_FILE, APPEND_FILE);
```

FILE_TYPE describes the type for file handles for all sequential input and output operations. 1

FILE_MODE indicates whether input operations or output operations can be performed on the sequential file handle. A mode of APPEND_FILE causes any elements that are written to the specified fle handle to be appended to the elements that are already in the file. The values for FILE_MODE, except for APPEND_FILE, are the same as [1815A] and correspond to the following cases: 2

a. IN_FILE corresponds to the case where only reading is to be performed. 3

b. OUT_FILE corresponds to the case where only writing is to be performed. 4

c. APPEND_FILE corresponds to the case where only writing (beginning at the end of the file) is to be performed. The file terminator is deleted at the time the file handle is opened. 5

## 8.5.3 Creating a sequential file

```
procedure CREATE
    (NODE                    : in out SECONDARY_STORAGE_NODE_TYPE;
     FILE                    : in out FILE_TYPE ;
     BASE                    : in     MODE_TYPE;
     KEY                     : in     RELATIONSHIP_KEY        : = EMPTY_KEY_LIST ;
     RELATION                : in     RD_TYPE;
     INTENT                  : in     INTENT_ARRAY            := (l=>WRITE);
     MODE                    : in     FILE_MODE               := OUT_FILE ;
     ATTRIBUTES              : in     ATTRIBUTE_LIST          : = EMPTY_LIST ;
     MAXIMUM_FILE_SIZE       : in     CAIS_NATURAL            := UNBOUNDED_FILE_SIZE;
     DISCRETIONARY_ACCESS : in     DISCRETIONARY_ACCESS_LIST := ALL_RIGHTS;
     MANDATORY_ACCESS        : in     MANDATORY_ACCESS_LIST := EMPTY_LIST;
     KIND                    : in     NODE_KIND_NODE_TYPE     := SECONDARY_STORAGE_KIND;
     PROCESS_PORT_KEY        : in     RELATIONSHIP_KEY        := EMPTY_KEY_LIST;
     PROCESS_PORT_RELATION : in     PORT_RD_TYPE            := PORT_RD;
     FILE_PORT_KEY           : in     RELATIONSHIP_KEY        := EMPTY_KEY_LIST;
     FILE_PORT_RELATION      : in     PORT_RD_TYPE            := PORT_RD;
     CHANNEL_PARENT          : in     PATHNAME                := CURRENT_PROCESS;
     STATUS                  : in     STATUS_INDICATOR        := NO_INDICATION;
     MEDIU_RESOURCE          : in     MEDIUM_MODE_TYPE        := DEFAULT_MEDIUM);
```

Purpose:

This procedure creates a SECONDARY_STORAGE node and installs the primary relationship to it as well as the corresponding reverse relationship to the node identified by BASE. ASECONDARY_STORAGE node is a FILE node whose contents are stored on a secondary storage 1



Source: https://assist.dla.mil -- Downloaded: 202610-26T07:54Z
Check the source to verify that this is the current version before use.

medium such as a disk. The primary relationship is of the relation RELATION with the key KEY. The MEDIUM_RESOURCE parameter specifies the medium resource on which the newly created SECONDARY_STORAGE node will reside. An open node handle to the created SECONDARY_STORAGE node with intent as specified by the INTENT parameter is returned in NODE.

The ATTRIBUTES parameter provides initial values for attributes of the SECONDARY_STORAGE node to be created. The MAXIMUM_FILE_SIZE parameter provides the value for the attribute MAXIMUM_FILE_SIZE with a value of zero indicating unrestricted size. The DISCRETIONARY_ACCESS parameter specifies initial access control information to be established for the created node. 2

The MANDATORY_ACCESS parameter specifies the object classification label with which the FILE node is to be created. If its value is the empty list, the node inherits the subject classification of the creating process as its object classification. Otherwise, it must be an unnamed list consisting of an identifier item and, optionally, an unnamed list of identifier items (see Table IX, page 256). 3

KIND is an open node handle identifying the node kind for the new node. It must identify a SECONDARY_STORAGE node kind. It may specify either an NKD or an NKI. If KIND identifies an NKI, the NKI is made the current adopted interpretation for the node handle NODE. 4

The procedure creates a CHANNEL node and installs the primary CHANNEL relationship to it as well as the corresponding reverse CHANNEL_OF relationship to the node identified by CHANNEL_PARENT. The relation definition nodes PROCESS_PORT_RELATION and FILE_PORT_RELATION are used to determine the NKD for the CHANNEL node (see CREATE_CHANNEL). A secondary PROCESS_PORT_RELATION relationship is created emanating from the current PROCESS node to the created CHANNEL node with the key PROCESS_PORT_KEY. A secondary FILE_PORT_RELATION relationship is created emanating from the created SECONDARY_STORAGE node to the created CHANNEL node with the key FILE_PORT_KEY. The OBJECT_CLASSIFICATION attribute on the CHANNEL node is set to the SUBJECT_CLASSIFICATION attribute of the current PROCESS node. An ACCESS relationship with a GRANT attribute of ((ALL_RIGHTS)) is created which emanates from the newly created CHANNEL node and terminates at the default GROUP node of the current user. The newly created CHANNEL node resides on the default medium resource for the current processor. The ALLOWED_PROTOCOLS attribute on each of the PORT relationships is set to the value (CAIS_SEQUENTIAL_IO). 5

The DIRECTION attribute of the process PORT relationship is set to IN when MODE is IN_FILE, OUT when MODE is OUT_FILE, and IN_AND_OUT when MODE is INOUT_FILE. The DIRECTION attribute of the file PORT relationship is OUT when MODE is IN_FILE, IN when MODE is OUT_FILE, and IN_AND_OUT when MODE is INOUT_FILE. The CONTROLLING_PROTOCOL attribute on the CHANNEL node is set to CAIS_SEQUENTIAL_IO and the charnel is in an ESTABLISHED state. 6

A sequential file handle used for communicating over the created channel to the created SECONDARY_STORAGE node is opened and returned in FILE. Each element of the file is sequentially accessible. The mode of the file handle is set to MODE. 7

Parameters:

**NODE** is a node handle, initially closed, to be opened to the newly created node. 8

**FILE** is a file handle, initially closed, to be opened. 9



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

**BASE** is an open node handle to the node which will be the source node of the primary relationship to the new node. 10

**KEY** is the relationship key designator of the primary relationship to be created. 11

**RELATION** is an open node handle to the RD of the primary relationship to be created. 12

**INTENT** is the intent of subsequent operations on the node; the actual parameter takes the form of an array aggregate. 13

**MODE** indicates the mode under which the file handle is to be opened. 14

**ATTRIBUTES** is an empty or named list whose elements are used to establish initial values for attributes of the newly created node; each named item of the list specifies an attribute name and the value to be given to that attribute. 15

**MAXIMUM_FILE_SIZE** defines the value for the predefine attribute MAXIMUM_FILE_-SIZE. 16

**DISCRETIONARY_ACCESS** is the initial access control information associated with the created node; it is the value of the GRANT attribute of the access relationship to the user's default group node (see Section 5.4 on page 255). 17

**MANDATORY_ACCESS** is a list defining the classification label for the created node (see Section 5.4, page 255). 18

**KIND** is an open node handle identifying the node kind for the new node. 19

**PROCESS_PORT_KEY** is the key value that the local port relationship is to have. 20

**PROCESS_PORT_RELATION** is an open node handle to the local port's RD. 21

**FILE_PORT_KEY** is the key value that the opposite port relationship is to have. 22

**FILE_PORT_RELATION** is an open node handle to the opposite port's RD. 23

**CHANNEL_PARENT** is a pathname identifying the node that will be the parent of the created CHANNEL node. 24

**STATUS** is an established status indicator. 25

**MEDIUM_RESOURCE** is an open node handle identifying a MEDIUM node. 26

Exceptions:

**PATHNAME_SYNTAX_ERROR** is raised if the relationship key list given in KEY, PRO-CESS_PORT_KEY, or FILE_PORT_KEY is syntactically illegal [2], or if the pathname given in CHANNEL_ PARENT is syntactically illegal [1] (see Table I, page 52). 27

**NAME_ERROR** is raised if the parent of the NKI identified by KIND is inaccessible or if any node traversed on the path specified by CHANNEL_PARENT is unobtainable or inaccessible [1]. 28

**EXISTING_NODE_ERROR** is raised if a node already exists with the identification given by BASE, KEY and RELATION [1]. 29

**SYNTAX_ERROR** is raised if the ATTRIBUTES parameter [8], the DISCRETIONARY_-ACCESS parameter [10], or the MANDATORY_ACCESS parameter [9] is syntactically illegal. 30



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

**NOT_IN_VIEW_ERROR** is raised if a relation or key name specified in KEY, PROCESS_PORT_KEY, FILE_PORT_KEY or CHANNEL_PARENT cannot be translated [1] or if KEY, PROCESS_PORT_KEY or FILE_PORT_KEY or a relationship key list specified in CHANNEL_PARENT consists of a single relationship key designator and no key of the relation identified by RELATION, PROCESS_PORT_RELATION, FILE_PORT_RELATION or the relation name given in the path element of CHANNEL_PARENT is visible, respectively [5]. 31

**AMBIGUOUS_VIEW_ERROR** is raised if translation of a key name specified in KEY, PROCESS_PORT_KEY, FILE_PORT_KEY or CHANNEL_PARENT is ambiguous [1] or if KEY, PROCESS_PORT_KEY, FILE_PORT_KEY or a relationship key list specified in CHANNEL_PARENT consists of a single relationship key designator and more than one key of the relation identified by RELATION, PROCESS_PORT_RELATION, FILE_PORT_RELATION or the relation name given in the path element of CHANNEL_PARENT is visible, respectively [4]. 32

**PREDEFINED_RELATION_ERROR** is raised if RELATION identifies a CAIS-controlled relation [1]. 33

**RELATIONSHIP_ERROR** is raised if any path element of CHANNEL_PARENT identifies more than one relationship [1]; if the relationship identified by any path element of CHANNEL_PARENT does not exist [2]; if a relationship with the identification given by PROCESS_PORT_RELATION and PROCESS_PORT_KEY already emanates from the current PROCESS node [3]; if KEY, PROCESS_PORT_KEY or FILE_PORT_KEY specifies a value for the same key more than once [5]; or if RELATION or its corresponding reverse relation has no unique key set [6]. RELATIONSHIP_ERROR is raised only if the conditions for NAME_ERROR are not present. 34

**PREDEFINED_ATTRIBUTE_ERROR** is raised if any attribute name given by the ATTRIBUTES parameter is the name of a CAIS-controlled attribute [1]. 35

**ATTRIBUTE_ERROR** is raised if ATTRIBUTES specifies a value for the same attribute more than once [6]. 36

**USE_ERROR** is raised if the value for the MANDATORY_ACCESS parameter is semantically illegal [10]. 37

**STATUS_ERROR** is raised if any of BASE, RELATION, KIND, PROCESS_PORT_RELATION, FILE_PORT_RELATION, or MEDIUM_RESOURCE is not an open node handle [2], if NODE is an open node handle [1], if FILE is an open file handle [7], or if STATUS is not an established status indicator [3]. 38

**NODE_KIND_ERROR** is raised if KIND does not identify an NKD or NKI [1], if the NKD identfied through KIND is not an NKD for a SECONDARY_STORAGE node [2], if the NKD identified through KIND is an INTRODUCED_NODE NKD [4], if RELATION, PROCESS_PORT_RELATION, or FILE_PORT_RELATION does not identify an RD [1], if PROCESS_PORT_RELATION or FILE_PORT_RELATION does not identify an RD for a PORT relation [3], or if MEDIUM_RESOURCE does not identify a medium resource node [1]. 39

**CONFORMANCE_ERROR** is raised if the operation would result in an attribute, node, or relationship not conforming to its definition or the constraints in force on that definition [1, 2, 4, 7-10, 11]. 40

**RESOURCE_ALLOCATION_ERROR** is raised if the medium resource identified through MEDIUM_RESOURCE is not allocatable [1]. 41



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

**RESOURCE_AVAILABILITY_ERROR** is raised if the medium resource identified through MEDIUM-RESOURCE is not available [1]. 42

**TRANSACTION_ERROR** is raised if the operation is included in more than one transaction [1] or if the operation is included in a transaction and the operation modifies a node which is not included in the same transaction [2]. 43

**LOCK_ERROR** is raised if a node handle to the parent of the NKI specified by KIND cannot be opened with intent READ-RELATIONSHIPS due to outstanding locks in conflict with that intent, if any node traversed on the path specified by the pathname CHANNEL_ - PARENT cannot be opened with intent READ-RELATIONSHIPS due to outstanding locks in conflict with that intent, or if a node handle to the node identified by CHANNEL-- PARENT cannot be opened with intent APPEND-RELATIONSHIPS due to outstanding locks in conflict with that intent [1]. 44

**INTENT_VIOLATION** is raised if BASE was not opened with an intent establishing the right to append relationships [1]. 45

**PORT_ERROR** is raised if the DIRECTION attributes of the PORT relationships are missing or have incompatible values [2], or if an NKD for a CHANNEL node cannot be found that permits a node of that kind to be the target of relationships of both of the specified PORT relations [1]. 46

**PROTOCOL_ERROR** is raised if CAIS_SEQUENTIAL_IO is not an allowed protocol of both PORT relationships to CHANNEL [1], or if a required protocol parameter attribute is not present on one or both PORT relationships at the time of the call [5], if the value of the CONTROLLING_PROTOCOL parameter is not CAIS_SEQUENTIAL_IO [2], or if the values of the ELEMENT_SIZE attributes of both PORT relationships are not the same [3] . 47

**MODE_ERROR** is raised if the value of the MODE parameter is incompatible with the value of the DIRECTION attribute of the local PORT relationship as defined in Table XXXIV on page 581 [2]. 48

**ACCESS_VIOLATION** is raised if the discretionary access control rights of the current process are insufficient to gain access with CREATE-INSTANCES intents to the NKD identified through KIND [1] or with READ_RELATIONSHIPS intents to the parent of the NKI specified by KIND or with APPEND_CONTENTS intent to the MEDIUM node identified by MEDIUM-RESOURCE [1] or with READ-RELATIONSHIPS intent to the nodes on the path given in CHANNEL-PARENT [1] or with APPEND_RELATIONSHIPS intent to the node identified by CHANNEL-PARENT [1]. ACCESS_VIOLATION is raised only if the conditions for NAME-ERROR are not present. 49

**SECURITY_VIOLATION** is raised if the operation represents a violation of mandatory access controls. SECURITY_VIOLATION is raised only if the conditions for other exceptions are not present. 60

Additional Interfaces:

```
procodure CREATE
   (MODE          : in out SECONDARY_STORAGE_NODE_TYPE;
    FILE          : in out FILE_TYPE;
    BASE          : in     MODE_TYPE;
    KEY           : in     RELATIONSHIP_KEY      := EMPTY_KEY_LIST ;
    RELATION      : in     RELATION_NAME         := DEFAULT_RELATION;
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
        INTENT                  :  in   INTENT-ARRAY              := (l=>WRITE);
        MODE                    :  in   FILE_MODE                 := OUT_FILE ;
        ATTRIBUTES              :  in   ATTRIBUTE_LIST            := EMPTY_LIST;
        MAXIMUM_FILE_SIZE       :  in   CAIS_NATURAL              := UNBOUNDED_FILE_SIZE;
        DISCRETIONARY_ACCESS    : in    DISCRETIONARY_ACCESS_LIST := ALL_RIGHTS;
        MANDATORY_ACCESS        :  in   MANDATORY_ACCESS_LIST     := EMPTY_LIST;
        STATUS                  :  in   STATUS_INDICATOR          := NO_INDICATION;
        MEDIUM_RESOURCE         :  in   MEDIUM_NODE_TYPE          := DEFAULT_MEDIUM)
    is
        RELATION_RD : RD_TYPE;
    begin
        DEFINITION-OF_RELATION(RELATION_RD, BASE, RELATION, STATUS);


        CREATE (NODE, FILE, BASE, KEY, RELATION_RD, INTENT, MODE, ATTRIBUTES,
                MAXIMUM_FILE_SIZE, DISCRETIONARY_ACCESS, MANDATORY_ACCESS,
                STATUS => STATUS, MEDIUM_RESOURCE => MEDIUM_RESOURCE);
        CLOSE (RELATION-RD);
    exception
        when others =>
            CLOSE (RELATION_RD);
            raise;
    and CREATE;
```

## 8.5.4 Opening a sequential file handle

```
procedure OPEN
    (FILE                   : in out FILE_TYPE;
     CHANNEL                :  in    CHANNEL_NODE_TYPE;
     OPPOSITE_CLIENT        : in     CLIENT_NODE_TYPE;
     MODE                   :  in    FILE_MODE;
     TIME_LIMIT             :  in    CAIS_IO_DURATION := LONG_IO_DELAY;
     STATUS                 :  in    STATUS_INDICATOR := NO_INDICATION);
```

Purpose:

This procedure opens a sequential file handle for communicating over the channel identified by CHANNEL with the node identified by OPPOSITE_CLIENT. CAIS_SEQUENTIAL_IO must be an allowed protocol of the CHANNEL node identified by CHANNEL. Each element of the file is sequentially accessible. The mode of the file handle is set to MODE. 1

If the operation concludes successfully, the CONTROLLING_PROTOCOL attribute on the CHANNEL node is set to CAIS_SEQUENTIAL_IO and the channel is in unestablished state. 2

Parameters:

**FILE** is a file handle, initially closed, to be opened. 3



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

**CHANNEL** is an open node handle to a CHANNEL node to which both the current process and OPPOSITE-CLIENT are client nodes. 4

**OPPOSITE_CLIENT** is an open node handle to the node to be accessed. 5

**MODE** indicates the mode under which the file handle is to be opened. 6

**TIME-LIMIT** is the time limit for waiting for the OPEN to complete. 7

**STATUS** is an established status indicator. 8

Exceptions:

**STATUS_ERROR** is raised if FILE is an open file handle [7], if OPPOSITE-CLIENT is not an open node handle [2], if CHANNEL is not an open node handle [2], if the channel denoted by CHANNEL is not in the CONNECTED state [9] or if STATUS is not an established status indicator [3]. 9

**NODE_KIND_ERROR** is raised if the node identfied by CHANNEL is not a CHANNEL node [1]. 10

**CLIENT_ERROR** is raised if the current process [1] or the node identified by OPPOSITE. - CLIENT [5] is not a client node of the channel identified by the CHANNEL parameter. 11

**PORT_ERROR** is raised if the DIRECTION attributes of the PORT relationships are missing or have incompatible values [4]. 12

**PROTOCOL_ERROR** is raised if CAIS-SEQUENTIAL_IO is not an allowed protocol of both PORT relationships to CHANNEL [1], if the CONTROLLING-PROTOCOL attribute has a value other than NO-PROTOCOL or CAIS_SEQUENTIAL_IO [2], or if the values of the ELEMENT-SIZE attributes of both PORT relationships are not the same [3] . 13

**MODE_ERROR** is raised if MODE is incompatible with the DIRECTION attribute of the local PORT relationship to the channel identified by CHANNEL as determined by Table XXXIV on page 581 [2] or if both clients are processes and specify incompatible modes as determined by Table XXXIV on page 581 [3]. 14

**USE_ERROR** is raised if another open file handle identifies the same external file and the CAIS implementation does not support the existence of multiple file handles identifying the same external file [41]. Any such restriction must be documented in Appendix F. An implementation is allowed to raise this exception only if it is based on operating system support that does not provide this capability. 15

**TRANSACTION_ERROR** is raised if the operation is included in more than one transaction [1] or if the operation is included in a transaction and the operation modifies a node which is not included in the same transaction [2]. 16

**INTENT_VIOLATION** is raised if the node handle OPPOSITE-CLIENT was not opened with an intent specification including at least the intents required for MODE as determined by Table XXXIV on page 581 [1]. 17

**OPEN_DENIED** is raised if a remote client process or the opposite client, if it is a process, deleted the the channel, signaling the denial of the connection attempt [1] or if a protocol open issued by the opposite client process fails for protocol compatibility reasons which are not determinable by the current process [2]. 18

**TIME_EXPIRED** is raised if the OPEN operation is delayed beyond the specified time limit for reasons other than a lock conflict [1]. 19



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

**SECURITY_VIOLATION** is raised if the operation represents a violation of mandatory access controls. SECURITY_ VIOLATION is raised only if the conditions for other exceptions are not present. 20

Notes:

Closing the node handle OPPOSITE-CLIENT associated with the file handle FILE closes the file handle. 21

## 8.5.5 Closing a sequential file handle

```
procedure CLOSE
   (FILE   : in out FILE-TYPE;
    STATUS : in     STATUS_INDICATOR := NO_INDICATION);
```

Purpose:

This procedure severs the association between the internal file identified by the file handle FILE and its associated channel. An implicit SYNCHRONIZE operation is performed and the file handle is closed. Closing an already closed file handle has no effect. 1

If the initial state of the associated channel is CLOSED, it will enter the CONNECTED state. If the opposite client node is a DEVICE or SECONDARY-STORAGE node, the associated channel will enter the CONNECTED state. If the initial state of the associated channel is ESTABLISHED and the opposite client node is a GATEWAY or PROCESS node, the channel will enter the CLOSED state. If the channel enters the CONNECTED state, the CONTROLLING_-PROTOCOL attribute of the CHANNEL node is set to NO-PROTOCOL. 2

Parameters:

FILE is a file handle to be closed. 3

STATUS is an established status indicator. 4

Exceptions:

**STATUS_ERROR** is raised if STATUS is not an established status indicator [3]. 5

**TRANSACTION_ERROR** is raised if the operation is included in a transaction and the file handle FILE was not opened inside the transaction [3]. 6

## 8.5.6 Resetting a sequential file handle

```
procedure RESET
   (FILE         : in out FILE_TYPE;
    MODE         : in     FILE_MODE;
    TIME_LIMIT   : in     CAIS_IO_DURATION := LONG_IO_DELAY;
    STATUS       : in     STATUS_INDICATOR := NO_IMDICATION);
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Purpose:

This procedure sets the current mode of the file handle FILE to the mode given by the MODE parameter. In effect, the CLOSE, AWAIT_CHANNEL (if the opposite client is a process), and OPEN operations are performed as an atomic operation. 1

If the new mode is IN_FILE or OUT_FILE, this procedure positions the given internal file so that reading from or writing to its elements can be restarted from the beginning of the internal file. If the new mode is APPEND_FILE, this procedure positions the given internal file so that writing to its elements can be restarted at the end of the internal file. 2

Parameters:

**FILE** is an open file handle identifying the internal file to be reset. 3

**MODE** indicates the new mode under which the file handle is to be reset. 4

**TIME_LIMIT** is the time limit for waiting for the implicit AWAIT-CHANNEL and OPEN to complete. 5

**STATUS** is an established status indicator. 6

Exceptions:

**STATUS_ERROR** is raised if FILE is not an open file handle [8] or if STATUS is not an established status indicator [3]. 7

**MODE_ERROR** is raised if MODE is incompatible with the DIRECTION attribute of the local PORT relationship as determined by Table XXXIV on page 581 [2] or if both clients are processes and specify incompatible modes as determined by Table XXXIV on page 581 [3]. 8

**USE_ERROR** is raised if the CAIS implementation does not support resetting the file handle to the specified mode [43]. 9

**TRANSACTION_ERROR** is raised if the operation is included in more than one transaction [1] or if the operation is included in a transaction and the operation modifies a node which is not included in the same transaction [2]. 10

**INTENT_VIOLATION** is raised if the node handle associated with the file handle FILE was not opened with an intent specification including at least the intents required for MODE as determined by Table XXXIV on page 581 [1]. 11

**OPEN_DENIED** is raised if a remote client process or the opposite client, if it is a process, deleted the channel, signaling the denial of the connection attempt [1] or if a protocol open issued by the opposite client process fails for protocol compatibility reasons which are not determinable by the current process [2]. 12

**TIME_EXPIRED** is raised if the operation is delayed beyond the specified time limit for reasons other than a lock conflict [1]. 13

**SECURITY_VIOLATION** is raised if the operation represents a violation of mandatory access controls. SECURITY_VIOLATION is raised only if the conditions for other exceptions are not present. 14


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

## 8.5.7 Synchronizing internal file with FILE node contents

```
procedure SYNCERONIZE
    (FILE   : in     FILE_TYPE;
     STATUS : in     STLTUS_IIDICATOR  := NO_INDICATION);
```

Purpose:

This procedure forces all data that has been written to the internal file identified by FILE to be transmitted to the contents of the external file with which it is associated. 1

Parameters:

**FILE** is an open file handle identifying the internal file to be synchronized.

**STATUS** is an established status indicator.

Exceptions:

**STATUS_ERROR** is raised if FILE is not an open file handle [8] or if STATUS is not an established status indicator [3]. 4

**MODE_ERROR** is raised if the file handle FILE is of mode IN-FILE [1]. 5

Notes:

For all write operations in the CAIS, the conditions upon which data are transferred from an internal file to the contents of a FILE node are implementation-dependent. Data in the internal file of a process are inaccessible to other processes. This procedure ensures that the data in the internal file and the data in the contents of the FILE node coincide. 6

## 8.5.8 Determining the channel node associated with a file

```
procedure  GET_ASSOCIATED_CHANNEL
    (CHANNEL  : in out CHANNEL_NODE_TYPE;
     FILE     :  in     FILE_TYPE;
     STATUS   : in      STATUS_INDICATOR := NO_INDICATION);
```

Purpose:

This procedure returns a node handle, with intent NO_ACCESS, to the CHANNEL node associated with the file handle FILE. Note that the association was made when the file handle was initially opened. 1

The last relationship traversed in opening the node handle CHANNEL is the same as that of the CHANNEL node handle provided when the file handle FILE was opened. See subprograms PATH_KEY (Section 5.2.20 on page 119) and PATH_RELATION (Section 5.2.21 on page 120). 2


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Parameters:

**CHANNEL** is a node handle, initially closed, to be opened to the associated CHANNEL node. 3

**FILE** is an open file handle identifying the internal file whose associated CHANNEL node is required. 4

**STATUS** is an established status indicator. 5

Exceptions:

**STATUS-ERROR** is raised if FILE is not an open file handle [8], if CHANNEL is open at the time of the call [1] or if STATUS is not an established status indicator [3]. 6

**SECURITY_VIOLATION** is raised if the operation represents a violation of mandatory access controls. SECURITY_VIOLATION is raised only if the conditions for other exceptions are not present. 7



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

# 8.6 Package CAIS_TEXT_IO

This package provides facilities for accessing textual data elements in CAIS files. [1815A] defines what constitutes an element. 1

The package specification and semantics of CAIS_TEXT_IO are comparable to those of the [1815A] package TEXT_IO. The subprograms of CAIS_TEXT_IO correspond to the subprograms in [1815A] TEXT_IO as follows: 2

**CREATE** replaces [1815A] TEXT_IO_CREATE. 3

**OPEN** replaces [1815A] TEXT_IO_OPEN. 4

**CLOSE** replaces [1815A] TEXT_IO_CLOSE. 5

**DELETE** does not exist in the package CAIS_TEXT_IO. 6

**RESET** replaces [1815A] TEXT_IO_RESET. 7

**NAME** does not exist in the package CAIS_TEXT_IO. 8

**FORM** does not exist in the package CAIS_TEXT_IO. 9

**STANDARD_INPUT** does not exist in the package CAIS_TEXT_IO. 10

**STANDARD_OUTPUT** does not exist in the package CAIS_TEXT_IO. 11

**SYNCHRONIZE** is an additional subprogram that does not exist in [1815A] TEXT_IO. 12

**GET_ASSOCIATED.** CHANNEL is an additional subprogram that does not exist in [1815A] TEXT_IO. 13

All other subprograms in [1815A] TEXT_IO are also in CAIS_TEXT_IO and have the same syntax and semantics with the following exceptions: 14

1. All types and subtypes are CAIS-defined types and subtypes. 15

2. Each subprogram has a final added STATUS parameter—an optional in parameter of type STATUS_INDICATOR defaulting to NO_INDICATION. The exception STATUS-ERROR is raised when the STATUS parameter is not an established status indicator. 16

3. All operations on file handles of mode APPEND_FILE should be the same as those of mode OUT_FILE. 17

4. Prior to the first call to the interface SET_INPUT no default input file is open and a closed file handle is returned by the function CURRENT_INPUT. 18

5. Prior to the fist call to the interface SET_OUTPUT no default output file is open and a closed file handle is returned by the function CURRENT_OUTPUT. 19

The exceptions raised by the subprograms in CAIS_TEXT_IO are defined in the packages CAIS_-DEFINITIONS, CAIS_IO_DEFINITIONS and CAIS_PRAGMATICS. 20


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

## 8.6.1 Protocol parameters

The IO connection model allows protocol-specific parameters to be determined during the establishment of a channel (see Section 7.3.1.3 on page 576). For CAIS_TEXT_IO, the protocol-specific parameters are MAXIMUM_LINE_LENGTH and MAXIMUM_PAGE_LENGTH. These integer-valued protocol-specific parameter attributes specify the maximum length of lines and pages within text file data that can be successfully received by the client from which the PORT relationship emanates. A zero value for one of these protocol parameters indicates an unbounded length as defined in [1815A] section 14.3(9). Specification of these protocol-specific parameter attributes on the PORT relationships is optional. If either of these protocol parameter attributes is not present on a PORT relationship, the protocol parameter compatibility checking performed during a CAIS_TEXT_IO protocol open proceeds as if the attribute existed with a value of zero. 1

When an attempt is made to open a protocol using the OPEN interface in package CAIS_TEXT_IO, the protocol-specific parameter compatibility checking and the adoption of protocol-specific parameter values occurs as follows for each of the protocol parameters MAXIMUM_PAGE_LENGTH and MAXIMUM_LINE_LENGTH. The parameter value of the receiving client (as determined by the MODE specified) must specify a maximum greater than or equal to the maximum indicated by the parameter value of the client sending the data. If these protocol-specific parameter compatibility checks succeed, the receiving client's protocol parameter is copied to the CHANNEL node. Otherwise, the exception PROTOCOL-ERROR is raised. 2

## 8.0.2 Definition of types

```
type FILE_TYPE is limited private;

type FILE_MODE is (IN_FILE, OUT_FILE, APPEND_FILE);
```

FILE_TYPE describes the type for file handles for all text input and output operations. 1

FILE_MODE indicates whether input operations or output operations can be performed on the text file handle. A mode of APPEND_FILE causes any text written to the specified file handle to be appended to the text that is already in the file. The values for FILE_MODE, except for APPEND_FILE, are the same as [1815A] and correspond to the following cases: 2

a. IN-FILE corresponds to the case where only reading is to be performed. 3

b. OUT-FILE corresponds to the case where only writing is to be performed. 4

c. APPEND-FILE corresponds to the case where only writing (beginning at the end of the file) is to be performed. The file terminator is deleted at the time the file handle is opened. 5

## 8.6.3 Creating a text file

```
procedure CREATE
    (MODE        : in out SECONDARY_STORAGE_NODE_TYPE;
     FILE        : in out FILE-TYPE;
     BASE        : in     MODE-TYPE;
```



Source: https://assist.dla.mil - Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
KEY                        : in  RELATIONSHIP_KEY          := EHPTY_KEY_LIST;
RELATION                   : in  RD_TYPE;
INTENT                     : in  INTENT_ARRAY              :=  (l=>WRITE);
MODE                       : in  FILE_MODE                 := OUT_FILE;
ATTRIBUTES                 : in  ATTRIBUTE_LIST            := EMPTY_LIST;
MAXIMUM_FILE_SIZE          : in  CAIS_NATURAL              := UNBOUNDED_FILE_SIZE;
DISCRETIONARY_ACCESS       : in  DISCRETIONARY_ACCESS_LIST := ALL_RIGHTS;
MANDATORY_ACCESS           : in  MANDATORY_ACCESS_LIST     := EMPTY_LIST;
KIND                       : in  NODE_KIND_NODE_TYPE       := SECONDARY_STORAGE_NKD;
PROCESS-PORT-KEY           : in  RELATIONSHIP_KEY          := EMPTY_KEY_LIST;
PROCESS_PORT_RELATION      : in  PORT_RD_TYPE              := PORT_RD;
FILE_PORT_KEY              : in  RELATIONSHIP_KEY          := EMPTY_KEY_LIST;
FILE_PORT_RELATION         : in  PORT_RD_TYPE              := PORT_RD;
CHANNEL_PARENT             : in  PATHNAME                  := CURREET_PROCESS;
STATUS                     : in  STATUS_INDICATOR          :=  NO_INDICATION;
MEDIUM_RESOURCE            : in  MEDIUM_NODE_TYPE          :=  DEFAULT_MEDIUM);
```

Purpose:

This procedure creates a SECONDARY_STORAGE node and installs the primary relationship to it as well as the corresponding reverse relationship to the node identified by BASE. A SECONDARY_STORAGE node is a FILE node whose contents are stored on a secondary storage medium such as a disk. The primary relationship is of the relation RELATION with the key KEY. The MEDIUM_RESOURCE parameter specifies the medium resource on which the newly created SECONDARY-STORAGE node will reside. An open node handle to the created SECONDARY_STORAGE node with intent as specified by the INTENT parameter is returned in NODE. 1

The ATTRIBUTES parameter provides initial values for attributes of the SECONDARY_STORAGE node to be created. The MAXIMUM_FILE_SIZE parameter provides the value for the attribute MAXIMUM_FILE_SIZE with a value of zero indicating unrestricted size. The DISCRETIONARY_ACCESS parameter specifies initial access control information to be established for the created node. 2

The MANDATORY_ACCESS parameter specifies the object classification label with which the FILE node is to be created. If its value is the empty list, the node inherits the subject classification of the creating process as its object classification. Otherwise, it must be an unnamed list consisting of an identfier item and, optionally, an unnamed list of identifier items (see Table IX, page 256). 3

KIND is an open node handle identifying the node kind for the new node. It must identify a SECONDARY-STORAGE node kind. It may specify either an NKD or an NKI. If KIND identifies an NKI, the NKI is made the current adopted interpretation for the node handle NODE. 4

The procedure creates a CHANNEL node and installs the primary CHANNEL relationship to it as well as the corresponding reverse CHANNEL_OF relationship to the node identified by CHANNEL_PARENT. The relation definition nodes PROCESS_PORT_RELATION and FILE_PORT_RELATION are used to determine the NKD for the CHANNEL node (see CREATE_CHANNEL). A secondary PROCESS_PORT_RELATION relationship is created emanating from the current PROCESS node to the created CHANNEL node with the key PROCESS_PORT_KEY. A secondary FILE_PORT_RELATION relationship is created emanating from the created 5


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

SECONDARY_STORAGE node to the created CHANNEL node with the key FILE. PORT_-KEY. The OBJECT_CLASSIFICATION attribute on the CHANNEL node is set to the SUB-JECT_CLASSIFICATION attribute of the current PROCESS node. An ACCESS relationship with a GRANT attribute of ((ALL_RIGHTS)) is created which emanates from the newly created CHANNEL node and terminates at the default GROUP node of the current user. The newly created CHANNEL node resides on the default medium resource for the current processor. The ALLOWED_PROTOCOLS attribute on each of the PORT relationships is set to the value (CAIS_TEXT_IO).

The DIRECTION attribute of the process PORT relationship is set to IN when MODE is IN_- 6
FILE, OUT when MODE is OUT_FILE, and IN_AND_OUT when MODE is INOUT_FILE. The DIRECTION attribute of the file PORT relationship is OUT when MODE is IN_FILE, IN when MODE is OUT_ FILE, and IN_AND_OUT when MODE is INOUT_FILE. The CONTROLLING. -PROTOCOL attribute on the CHANNEL node is set to CAIS_TEXT_IO and the channel is in an ESTABLISHED state.

A text file handle used for communicating over the created channel to the created SECONDARY_- 7
STORAGE node is opened and returned in FILE. The file is textual. The mode of the file handle is set to MODE.

Parameters:

**NODE** is a node handle, initially closed, to be opened to the newly created node. 8

**FILE** is a file handle, initially closed, to be opened. 9

**BASE** is an open node handle to the node which will be the source node of the primary relationship to the new node. 10

**KEY** is the relationship key designator of the primary relationship to be created. 11

**RELATION** is an open node handle to the RD of the primary relationship to be created. 12

**INTENT** is the intent of subsequent operations on the node; the actual parameter takes the form of an array aggregate. 13

**MODE** indicates the mode under which the file handle is to be opened. 14

**ATTRIBUTES** is an empty or named list whose elements are used to establish initial values for attributes of the newly created node; each named item of the list specifies an attribute name and the value to be given to that attribute. 16

**MAXIMUM_FILE_SIZE** defines the value for the predefined attribute MAXIMUM_FILE_- SIZE. 16

**DISCRETIONARY_ACCESS** is the initial access control information associated with the created node; it is the value of the GRANT attribute of the access relationship to the user's default group node (see Section 5.4 on page 255). 17

**MANDATORY_ACCESS** is a list defining the classification label for the created node (see Section 5.4, page 255). 18

**KIND** is an open node handle identifying the node kind for the new node. 19

**PROCESS_PORT_KEY** is the key value that the local port relationship is to have. 20

**PROCESS_PORT_RELATION** is an open node handle to the local port's RD. 21

**FILE_PORT_KEY** is the key value that the opposite port relationship is to have. 22



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

**FILE_PORT;** RELATION is an open node handle to the opposite port's RD.

**CHANNEL_PARENT** is a pathname identifying the node that will be the parent of the created CHANNEL node.

**STATUS** is an established status indicator.

**MEDIUM_RESOURCE** is an open node handle identifying a MEDIUM node.

Exceptions:

**PATHNAME_SYNTAX_ERROR** is raised if the relationship key list given in KEY, PROCESS_PORT_KEY, or FILE_PORT_KEY is syntactically illegal [2], or if the pathname given in CHANNEL_PARENT is syntactically illegal [1] (see Table I, page 52).

**NAME_ERROR** is raised if the parent of the NKI identified by KIND is inaccessible or if any node traversed on the path specified by CHANNEL-PARENT is unobtainable or inaccessible [1].

**EXISTING_NODE_ERROR** is raised if a node already exists with the identification given by BASE, KEY and RELATION [1].

**SYNTAX_ERROR** is raised if the ATTRIBUTES parameter [8], the DISCRETIONARY_ACCESS parameter [10], or the MANDATORY_ACCESS parameter [9] is syntactically illegal.

**NOT_IN_VIEW_ERROR** is raised if a relation or key name specified in KEY, PROCESS_PORT_KEY, FILE_PORT_KEY or CHANNEL_PARENT cannot be translated [1] or if KEY, PROCESS_PORT_KEY or FILE_PORT_KEY or a relationship key list specified in CHANNEL_PARENT consists of a single relationship key designator and no key of the relation identified by RELATION, PROCESS_PORT_RELATION, FILE_PORT_RELATION or the relation name given in the path element of CHANNEL_PARENT is visible, respectively [5].

**AMBIGUOUS_VIEW_ERROR** is raised if translation of a key name specified in KEY, PROCESS_PORT_KEY, FILE_PORT_KEY or CHANNEL_PARENT is ambiguous [1] or if KEY, PROCESS_PORT_KEY, FILE_PORT_KEY or a relationship key list specified in CHANNEL_PARENT consists of a single relationship key designator and more than one key of the relation identified by RELATION, PROCESS_PORT_RELATION, FILE_PORT_RELATION or the relation name given in the path element of CHANNEL_PARENT is visible, respectively [4].

**PREDEFINED_RELATION_ERROR** is raised if RELATION identifies a CAIS-controlled relation [1].

**RELATIONSHIP_ERROR** is raised if any path element of CHANNEL_PARENT identifies more than one relationship [1]; if the relationship identified by any path element of CHANNEL_PARENT does not exist [2]; if a relationship with the identification given by PROCESS_PORT_RELATION and PROCESS_PORT_KEY already emanates from the current PROCESS node [3]; if KEY, PROCESS_PORT_KEY or FILE_PORT_KEY specifies a value for the same key more than once [5]; or if RELATION or its corresponding reverse relation has no unique key set [6]. RELATIONSHIP_ERROR is raised only if the conditions for NAME_ERROR are not present.

**PREDEFINED_ATTRIBUTE_ERROR** is raised if any attribute name given by the ATTRIBUTES parameter is the name of a CAIS-controlled attribute [1].


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

**ATTRIBUTE_ERROR** is raised if ATTRIBUTES specifies a value for the same attribute more than once [6]. 36

**USE_ERROR** is raised if the value for the MANDATORY-ACCESS parameter is semantically illegal [10]. 37

**STATUS_ERROR** is raised if any of BASE, RELATION, KIND, PROCESS_PORT_RELATION, FILE_PORT_RELATION, or MEDIUM_RESOURCE is not an open node handle [2], if NODE is an open node handle [1], if FILE is an open file handle [7], or if STATUS is not an established status indicator [3]. 38

**NODE_KIND_ERROR** is raised if KIND does not identify an NKD or NKI [1], if the NKD identified through KIND is not an NKD for a SECONDARY_STORAGE node [2], if the NKD identified through KIND is an INTRODUCED_NODE NKD [4], if RELATION, PROCESS_PORT_RELATION, or FILE_PORT_RELATION does not identify an RD [1], if PROCESS_PORT_RELATION or FILE_PORT_RELATION does not identify an RD for a PORT relation [3], or if MEDIUM_RESOURCE does not identify a medium resource node [1]. 39

**CONFORMANCE-ERROR** is raised if the operation would result in an attribute, node, or relationship not conforming to its definition or the constraints in force on that definition [1, 2,4, 7-10, 11]. 40

**RESOURCE_ALLOCATION_ERROR** is raised if the medium resource identified through MEDIUM_RESOURCE is not allocatable [1]. 41

**RESOURCE_AVAILABILITY_ERROR** is raised if the medium resource identified through MEDIUM_RESOURCE is not available [1]. 42

**TRANSACTION_ERROR** is raised if the operation is included in more than one transaction [1] or if the operation is included in a transaction and the operation modifies a node which is not included in the same transaction [2]. 43

**LOCK_ERROR** is raised if a node handle to the parent of the NKI specified by KIND cannot be opened with intent READ_RELATIONSHIPS due to outstanding locks in conflict with that intent, if any node traversed on the path specified by the pathname CHANNEL_ - PARENT cannot be opened with intent READ_RELATIONSHIPS due to outstanding locks in conflict with that intent, or if a node handle to the node identified by CHANNEL_- PARENT cannot be opened with intent APPEND_RELATIONSHIPS due to outstanding locks in conflict with that intent [1]. 44

**INTENT_VIOLATION** is raised if BASE was not opened with an intent establishing the right to append relationships [1]. 45

**PORT_ERROR** is raised if the DIRECTION attributes of the PORT relationships are missing or have incompatible values [2], or if an NKD for a CHANNEL node cannot be found that permits a node of that kind to be the target of relationships of both of the specified PORT relations [1]. 46

**PROTOCOL_ERROR** is raised if CAIS_TEXT_IO is not an allowed protocol of both PORT relationships to CHANNEL [1], or if a required protocol parameter attribute is not present on one or both PORT relationships at the time of the call [5] or if the value of the CONTROLLING_PROTOCOL parameter is not CAIS_TEXT_IO [2]. 47

**MODE_ERROR** is raised if the value of the MODE parameter is incompatible with the value of the DIRECTION attribute of the local PORT relationship as defined in Table XXXIV on page 581 [2]. 48


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

**ACCESS_VIOLATION** is raised if the discretionary access control rights of the current process are insufficient to gain access with CREATE_INSTANCES intents to the NKD identified through KIND [1] or with READ_RELATIONSHIPS intents to the parent of the NKI specified by KIND or with APPEND_CONTENTS intent to the MEDIUM node identified by MEDIUM_RESOURCE [1] or with READ_RELATIONSHIPS intent to the nodes on the path given in CHANNEL_PARENT [1] or with APPEND_RELATIONSHIPS intent to the node identified by CHANNEL_PARENT [1]. ACCESS_VIOLATION is raised only if the conditions for NAME_ ERROR are not present. 49

**SECURITY_VIOLATION** is raised if the operation represents a violation of mandatory access controls. SECURITY_VIOLATION is raised only if the conditions for other exceptions are not present. 50

Additional Interfaces:

```
procedure CREATE
    (NODE                  : in out SECONDARY_STORAGE_NODE_TYPE;
     FILE                  : in out FILE_TYPE;
     BASE                  : in     NODE_TYPE ;
     KEY                   : in     RELATIONSHIP_KEY          : = EMPTY_KEY_LIST ;
     RELATION              : in     RELATION_NAME             : = DEFAULT_RELATION;
     INTENT                : in     INTENT_ARRAY              := (l=> WRITE) ;
     MODE                  : in     FILE_MODE                 : = OUT_FILE;
     ATTRIBUTES            : in     ATTRIBUTE_LIST            := EMPTY_LIST;
     MAXIMUM_FILE_SIZE     : in     CAIS_NATURAL              := UNBOUNDED_FILE_SIZE;
     DISCRETIONARY_ACCESS : in     DISCRETIONARY_ACCESS_LIST : = ALL_RIGHTS;
     MANDATORY_ACCESS      : in     MANDATORY_ACCESS_LIST  :=  EMPTY_LIST;
     STATUS                : in     STATUS_INDICATOR          :=  NO_INDICATION;
     MEDIUM_RESOURCE       : in     MEDIUM_NODE_TYPE          := DEFAULT_MEDIUM)
 is
    RELATION_RD : RD_TYPE ;
begin
    DEFINITION_OF_RELATION (RELATION-RD , BASE, RELATION, STATUS);

    CREATE (NODE, FILE, BASE, KEY, RELATION_RD, INTENT, MODE, ATTRIBUTES,
            MAXIMUM_FILE_SIZE, DISCRETIONARY_ACCESS, MANDATORY_ACCESS,
            STATUS => STATUS, MEDIUM_RESOURCE => MEDIUM_RESOURCE);
    CLOSE (RELATION_RD);
exception
    when others =>
        CLOSE (RELATION_RD);
        raise;
and CREATE;
```

## 8.0.4 Opening a text file handle

```
procedure OPEN
    (FILE                  : in out FILE_TYPE;
```


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
CHANNEL          : in  CHANNEL_MODE_TYPE;
OPPOSITE_CLIENT  : in  CLIENT_NODE_TYPE;
MODE             : in  FILE_MODE;
TIME_LIMIT       : in  CAIS_IO_DURATION := LONG_IO_DELAY;
STATUS           : in  STATUS_INDICATOR  := NO_INDICATION);
```

Purpose:

This procedure opens a text file handle for communicating over the channel identified by CHANNEL with the node identified by OPPOSITE_CLIENT. CAIS_TEXT_IO must be an allowed protocol of the CHANNEL node identified by CHANNEL. The mode of the file handle is set to MODE. 1

If the operation concludes successfully, the CONTROLLING_PROTOCOL attribute on the CHANNEL node is set to CAIS_TEXT_IO and the channel is in an ESTABLISHED state. 2

Parameters:

**FILE** is a file handle, initially closed, to be opened. 3

**CHANNEL** is an open node handle to a CHANNEL node to which both the current process and OPPOSITE_CLIENT are client nodes. 4

**OPPOSITE_CLIENT** is an open node handle to the node to be accessed. 5

**MODE** indicates the mode under which the file handle is to be opened. 6

**TIME_LIMIT** is the time limit for waiting for the OPEN to complete. 7

**STATUS** is an established status indicator. 8

Exceptions:

**STATUS_ERROR** is raised if FILE is an open file handle [7], if OPPOSITE_CLIENT is not an open node handle [2], if CHANNEL is not an open node handle [2], if the channel denoted by CHANNEL is not in the CONNECTED state [9] or if STATUS is not an established status indicator [3]. 9

**NODE_KIND_ERROR** is raised if the node identified by CHANNEL is not a CHANNEL node [1]. 10

**CLIENT_ERROR** is raised if the current process [1] or the node identified by OPPOSITE_CLIENT [5] is not a client node of the channel identified by the CHANNEL parameter. 11

**PORT-ERROR** is raised if the DIRECTION attributes of the PORT relationships are missing or have incompatible values [4]. 12

**PROTOCOL_ERROR** is raised if CAIS_TEXT_IO is not an allowed protocol of both PORT relationships to CHANNEL [1], if the CONTROLLING_PROTOCOL attribute has a value other than NO_PROTOCOL or CAIS_TEXT_IO [2], or if the MAXIMUM_LINE_LENGTH and MAXIMUM_PAGE_LENGTH protocol parameters specified are incompatible. 13

**MODE_ERROR** is raised if MODE is incompatible with the DIRECTION attribute of the local PORT relationship to the channel identified by CHANNEL as determined by Table XXXIV on page 581 [2] or if both clients are processes and specify incompatible modes as determined by Table XXXIV on page 581 [3]. 14



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

**USE_ERROR** is raised if another open file handle identifies the same external file and the CAIS implementation does not support the existence of multiple file handles identifying the same external file [41]. Any such restriction must be documented in Appendix F. An implementation is allowed to raise this exception only if it is based on operating system support that does not provide this capability. 15

**TRANSACTION_ERROR** is raised if the operation is included in more than one transaction [1] or if the operation is included in a transaction and the operation modifies a node which is not included in the same transaction [2]. 16

**INTENT_VIOLATION** is raised if the node handle OPPOSITE_CLIENT was not opened with an intent specification including at least the intents required for MODE as determined by Table XXXIV on page 581 [1]. 17

**OPEN_DENIED** is raised if a remote client process or the opposite client, if it is a process, deleted the the channel, signaling the denial of the connection attempt [1] or if a protocol open issued by the opposite client process fails for protocol compatibility reasons which are not determinable by the current process [2]. 18

**TIME_EXPIRED** is raised if the OPEN operation is delayed beyond the specified time limit for reasons other than a lock conflict [1]. 19

**SECURITY_VIOLATION** is raised if the operation represents a violation of mandatory access controls. SECURITY_VIOLATION is raised only if the conditions for other exceptions are not present. 20

Notes:

Closing the node handle OPPOSITE_CLIENT associated with the file handle FILE closes the file handle. 21

### 8.6.5 Closing a text file handle

```
procedure CLOSE
    (FILE   : in out FILE-TYPE;
     STATUS : in     STATUS_INDICATOR := NO_INDICATION);
```

Purpose:

This procedure severs the association between the internal file identified by the file handle FILE and its associated channel. An implicit SYNCHRONIZE operation is performed and the file handle is closed. Closing an already closed file handle has no effect. 1

If the initial state of the associated channel is CLOSED, it will enter the CONNECTED state. If the opposite client node is a DEVICE or SECONDARY_STORAGE node, the associated channel will enter the CONNECTED state. If the initial state of the associated channel is ESTABLISHED and the opposite client node is a GATEWAY or PROCESS node, the channel will enter the CLOSED state. If the channel enters the CONNECTED state, the CONTROLLING_-PROTOCOL attribute of the CHANNEL node is set to NO_PROTOCOL. 2


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Parameters:

**FILE** is a file handle to be closed. 3

**STATUS** is an established status indicator. 4

Exceptions:

**STATUS_ERROR** is raised if STATUS is not an established status indicator [3]. 5

**TRANSACTION_ERROR** is raised if the operation is included in a transaction and the file handle FILE was not opened inside the transaction [3]. 6

## 8.6.6 Resetting a text file handle

```
procedure RESET
   (FILE       : in out FILE_TYPE;
    MODE       : in     FILE_MODE;
    TIME_LIMIT : in     CAIS_IO_DURATION := LONG_IO_DELAY ;
    STATUS     : in     STATUS_INDICATOR  := NO_INDICATION);
```

Purpose:

This procedure sets the current mode of the file handle FILE to the mode given by the MODE parameter. In effect, the CLOSE, AWAIT_CHANNEL (if the opposite client is a process), and OPEN operations are performed as an atomic operation. 1

If the file handle FILE has the current mode OUT_FILE or APPEND_FILE, this procedure has the effect of calling NEW_PAGE, unless the current page is already terminated; it then outputs a file terminator. 2

If the new mode for the file handle is OUT_FILE or APPEND_FILE, the page and line lengths are unbounded. For all modes, the current column, line, and page numbers are set to one. 3

Parameters:

**FILE** is an open file handle identifying the internal file to be reset. 4

**MODE** indicates the new mode under which the file handle is to be reset. 5

**TIME_LIMIT** is the time limit for waiting for the implicit AWAIT_CHANNEL and OPEN to complete. 6

**STATUS** is an established status indicator. 7

Exceptions:

**STATUS_ERROR** is raised if FILE is not an open file handle [8] or if STATUS is not an established status indicator [3]. 8



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

**MODE_ERROR** is raised if MODE is incompatible with the DIRECTION attribute of the local PORT relationship as determined by Table XXXIV on page 581 [2] or if both clients are processes and specify incompatible modes as determined by Table XXXIV on page 581 [3]. 9

**USE_ERROR** is raised if the CAIS implementation does not support resetting the file handle to the specified mode [43]. 10

**TRANSACTION_ERROR** is raised if the operation is included in more than one transaction [1] or if the operation is included in a transaction and the operation modifies a node which is not included in the same transaction [2]. 11

**INTENT_VIOLATION** is raised if the node handle associated with the file handle FILE was not opened with an intent specification including at least the intents required for MODE as determined by Table XXXIV on page 581 [1]. 12

**OPEN_DENIED** is raised if a remote client process or the opposite client, if it is a process, deleted the channel, signaling the denial of the connection attempt [1] or if a protocol open issued by the opposite client process fails for protocol compatibility reasons which are not determinable by the current process [2]. 13

**TIME_EXPIRED** is raised if the operation is delayed beyond the specified time limit for reasons other than a lock conflict [1]. 14

**SECURITY_VIOLATION** is raised if the operation represents a violation of mandatory access controls. SECURITY_VIOLATION is raised only if the conditions for other exceptions are not present. 15

## 8.6.7 Synchronizing internal file with FILE node contents

```
procedure SYNCHRONIZE
    (FILE   : in     FILE. TYPE;
     STATUS : in     STATUS_INDICATOR := NO_INDICATION);
```

Purpose:

This procedure forces all data that has been written to the internal file identified by FILE to be transmitted to the contents of the external file with which it is associated. 1

Parameters:

**FILE** is an open file handle identifying the internal file to be synchronized. 2

**STATUS** is an established status indicator. 3

Exceptions:

**STATUS_ERROR** is raised if FILE is not an open file handle [8] or if STATUS is not an established status indicator [3]. 4

**MODE_ERROR** is raised if the file handle FILE is of mode IN_FILE [1]. 5



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Notes:

For all write operations in the CAIS, the conditions upon which data are transferred from an internal file to the contents of a FILE node are implementation-dependent. Data in t he internal file of a process are inaccessible to other processes. This procedure ensures that the data in the internal file and the data in the contents of the FILE node coincide. 6

## 8.6.8 Determining the channel node associated with a file

```
procedure GET_ASSOCIATED_CHANNEL_CHANNEL
    (CHANNEL : in out CHANNEL_NODE_TYPE;
     FILE    : in     FILE_TYPE;
     STATUS  : in     STATUS_INDICATOR := NO_INDICATION);
```

Purpose:

This procedure returns a node handle, with intent NO_ACCESS, to the CHANNEL node associated with the file handle FILE. Note that the association was made when the file handle was initially opened. 1

The last relationship traversed in opening the node handle CHANNEL is the same as that of the CHANNEL node handle provided when the file handle FILE was opened. See subprograms PATH_KEY (Section 5.2.20 on page 119) and PATH_RELATION (Section 5.2.21 on page 120). 2

Parameters:

**CHANNEL** is a node handle, initially closed, to be opened to the associated CHANNEL node. 3

**FILE** is an open file handle identifying the internal file whose associated CHANNEL node is required. 4

**STATUS** is an established status indicator. 5

Exceptions:

**STATUS_ERROR** is raised if FILE is not an open file handle [8], if CHANNEL is open at the time of the call [1] or if STATUS is not an established status indicator [3]. 6

**SECURITY_VIOLATION** is raised if the operation represents a violation of mandatory access controls. SECURITY_VIOLATION is raised only if the conditions for other exceptions are not present. 7

## 8.6.9 Setting the maximum line length of a text file handle

```
procedure SET_LINE_LENGTH
    (FILE    : in     FILE_TYPE;
     TO      : in     COUNT;
     STATUS  : in     STATUS_INDICATOR := NO_INDICATION);

procedure SET_LINE_LENGTH
    (TO      : in     COUNT;
     STATUS  : in     STATUS_INDICATOR := NO_INDICATION);
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Purpose:

This procedure sets the maximum line length (see [1815A] Section 14.3.3) of the file handle FILE (or the current default output file if the FILE parameter is not specified) to the number of characters specified by TO. The value zero for TO specifies an unbounded line length. 1

Parameters:

**FILE** is an open file handle identifying the internal file whose line length is to be modified. 2

**TO** contains the maximum line length for the file. 3

**STATUS** is an established status indicator. 4

Exceptions:

**STATUS_ERROR** is raised if FILE is not an open file handle [8] or if STATUS is not an established status indicator [3]. 5

**MODE_ERROR** is raised if the mode of the file is not OUT_FILE or APPEND_FILE [1]. 6

**USE_ERROR** is raised if the specified line length is inappropriate for the associated external file or if the specified line length exceeds the maximum indicated by the MAXIMUM_LINE_ - LENGTH protocol parameter attribute value chosen for the CHANNEL node associated with the open file handle FILE [64]. 7

**TRANSACTION_ERROR** is raised if the operation is included in a transaction and the file handle FILE was not opened inside the transaction. 8

## 8.6.10 Setting the maximum page length of a text file handle

```
procedure SET_PAGE_LENGTH
    (FILE   : in  FILE_TYPE ;
     TO     : in  COUNT;
     STATUS : in  STATUS_INDICATOR := NO_INDICATION);

procedure SET_PAGE_LENGTH
    (TO     : in  COUT;
     STATUS : in  STATUS_INDICATOR := NO_INDICATION);
```

Purpose:

This procedure sets the maximum page length (see [1815A] Section 14.3.3) of the file handle FILE (or the current default output file if the FILE parameter is not specified) to the number of lines specified by TO. The value zero for TO specifies an unbounded page length. 1

Parameters:

**FILE** is an open file handle identifying the internal file whose page length is to be modified. 2

**TO** contains the maximum page length for the file. 3

**STATUS** is an established status indicator. 4



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Exceptions:

**STATUS_ERROR** is raised if FILE is not an open file handle [8] or if STATUS is not an established status indicator [3]. 5

**MODE_ERROR** is raised if the mode of the file is not OUT_FILE or APPEND_FILE [1]. 6

**USE_ERROR** is raised if the specified page length is inappropriate for the associated external file or if the specified page length exceeds the maximum indicated by the MAXIMUM_PAGE_LENGTH protocol parameter attribute value chosen for the CHANNEL node associated with the open file handle FILE [65]. 7

**TRANSACTION_ERROR** is raised if the operation is included in a transaction and the file handle FILE was not opened inside the transaction. 8



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

# 8.7 Package CAIS_TERMINAL_IO

This package provides interfaces for communicating with an interactive video terminal. A typical terminal consists physically of a character mapped display, a keyboard, and a locator (e.g. mouse). Each terminal is represented as a DEVICE node within a CAIS instance. 1

The interfaces presented by the package CAIS_TERMINAL_IO are based upon a virtual terminal model. A *virtual terminal* is an abstraction representing an interactive video terminal and consists of a virtual display, a virtual keyboard, and a virtual locator. A *virtual display* is an abstraction that represents an idealized physical display. A *virtual keyboard* is an abstraction that represents a physical keyboard. A *virtual locator* is an abstraction that represents a physical locator. Input is accepted from the virtual keyboard and virtual locator. Output, rather than being written directly on the physical display, is written to screens bound to a virtual display. The virtual terminal model maintains a high degree of device independence, concealing from tools the details of handling terminal escape sequences. 2

A virtual display is a two-dimensional area on which tools place screens. This area is a relatively large coordinate system onto which a physical display is mapped; the physical display actually defines a "viewport" into the virtual display. This coordinate system provides a means to specify the relative orientation of one or more screens with respect to each other and to the physical display. (Various aspects of virtual displays and of screens are depicted in Figure 45. ) All output operations to the virtual display associated with a virtual terminal may be buffered, thus controlling the user visibility of operations to the virtual display. This permits a sequence of changes to be hidden from the terminal user until they are ready to be applied in a single update of the physical display. 3

A *screen* is a rectangular area composed of positions in which character cells are defined. The positions are arranged in horizontal rows and vertical columns. Each position is identifiable by the combination of positive row number and positive column number. A screen has a fixed number of columns and rows. The left-most column of a screen is termed its *minimum column;* the top-most row is termed its *minimum row;* the cell located in the minimum row and minimum column is termed its *minimum cell.* The right-most column of a screen is termed its *maximum column;* the bottom-most row is termed its *maximum row;* the cell located in the maximum row and maximum column is termed its *maximum cell.* A physical display contains an analogous minimum and maximum row, column, and cell. 4

A *line* is a single row of character cell positions. A *range* of cells is identified by two positions on a screen, a start position, SP, and an end position, EP. A given cell C is in the range identified by SP and EP if it is in the same row as SP and lies at or to the right of SP, if it is anywhere in a row below SP and above EP, or if it is in the same row as EP and lies at or to the left of EP. If EP is before or above SP, the range specified is a null range. 5

A *character cell* holds an ASCII character displayed using a certain *rendition*. The rendition of a cell includes the foreground and background colors used in displaying the character as well as other visual properties (e.g., bold, blinking, reverse video, underlined). The pixels that form the character are displayed in the foreground color. Though control characters are allowed, character cells normally hold printable ASCII characters. The displayed contents of a character cell containing a control character is implementation-dependent; it may be displayed as a blank. 6

Each screen contains a single cursor. The *cursor position* depicts the position at which the next operation will be performed if an explicit start position is not specified in the interface. The row containing the cursor is termed the *active row.* Along with the individual screen cursors, there is also a *locator cursor,* generally manipulated by a locator, associated with the physical display. 7

An *active rendition* is associated with each screen. The active rendition is applied to all output 8



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

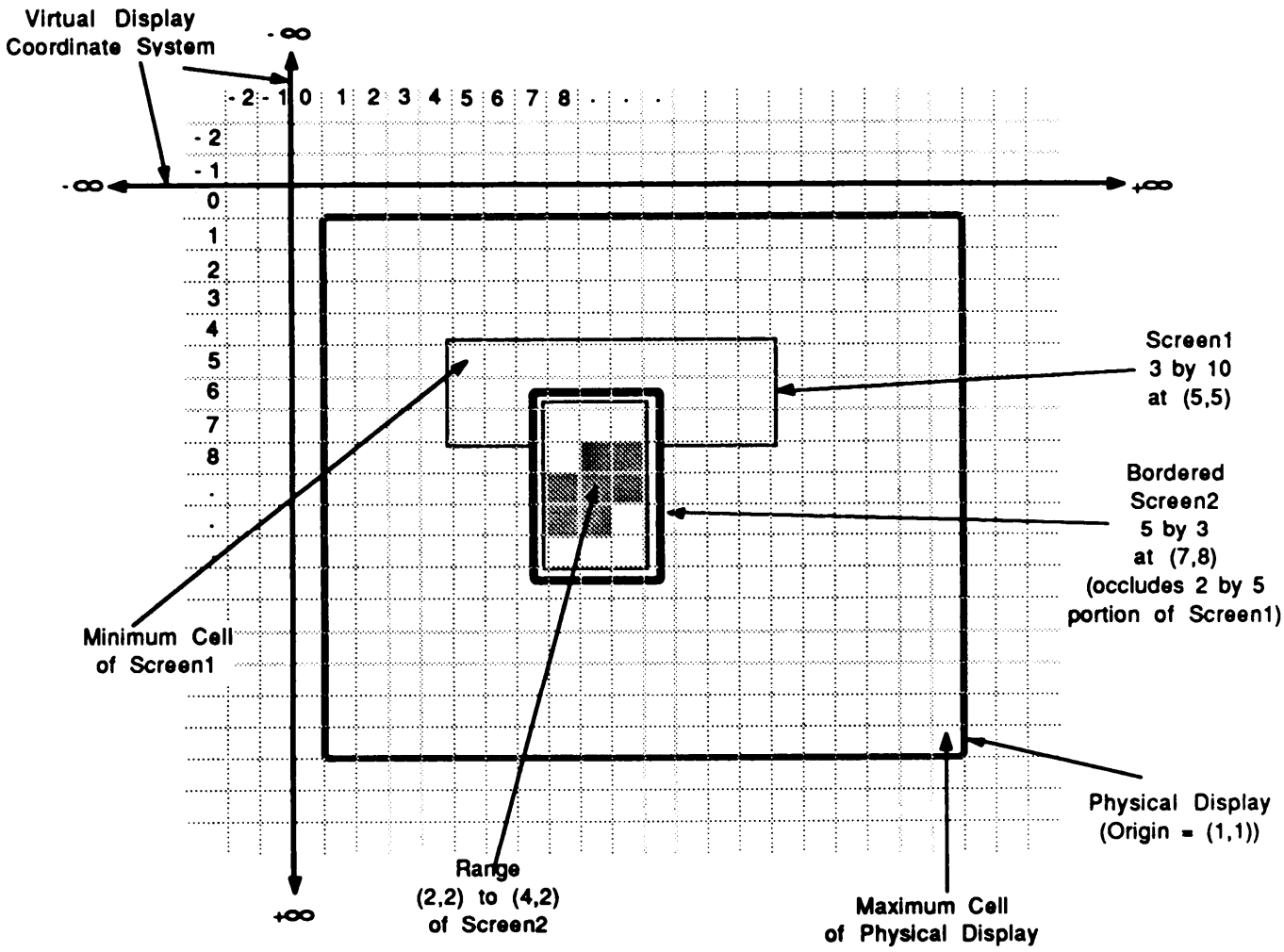


FIGURE 45. Virtual display with screens



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

operations to a given screen unless otherwise stated. In addition, a virtual terminal has a *default rendition* which is returned when a terminal file handle is opened. The default rendition is guaranteed to be supported by the particular terminal device; the active rendition for a screen can always be set to the default rendition of a terminal.

A screen is said to be *pasted* when it is placed on a virtual display and *unpasted* when it is removed from the virtual display or when it has never been pasted. A screen cannot be pasted to more than one virtual display at a time nor can it be simultaneously pasted onto multiple positions of a single virtual display. 9

A screen can only be visible when it is pasted to a region of a virtual display associated with a physical display. A screen may, then, be invisible, partly visible, or totally visible depending on its position on the virtual display. 10

Any number of screens may be pasted to a single virtual display. A screen can be pasted over other screens. The ordering of the screen pasting operations determines the *pasting order*. The latest screen pasted is at the front of the pasting order; screens pasted earlier are closer to the back. Some operations such as PASTE_SCREEN change the pasting order while operations such as MOVE_SCREEN preserve the order. A screen may occlude portions of screens behind but only to the extent that they are covered; portions not occluded remain visible. When unpasting a particular screen, portions of screens behind that were occluded by this screen become visible again. 11

A screen may be bordered. Though the manner in which a border is visually represented is implementation-dependent, at a maximum it can only affect character cells immediately adjacent to the actual screen character cells; i.e., the border can only be one character cell thick. The size of a screen is independent of whether or not the border is displayed. The amount of occluded text may increase when a screen is bordered. 12

Tools generally manipulate screens on a virtual display to create a desired physical display image. Interfaces are provided for pasting, unpasting, repasting, moving, and deleting screens. Witnin a screen, interfaces are provided for writing, deleting, erasing, inserting, and changing the character cells. All operations that affect a screen may be buffered, thus controlling the user visibility of operations to a particular screen. The effects of a buffered operation are invisible (or inaudible in the case of the interface SOUND_ BELL) to the terminal user. This permits a sequence of changes to a screen to be hidden from the user until they are ready to be applied in a single update. 13

Data is retrieved from a virtual keyboard either as Ada characters or function key identifiers. If function keys are enabled, each function keystroke is returned as a function key identification number. Otherwise, each returns an implementation-dependent sequence of Ada characters instead. When a terminal file handle is initially opened, it is implementation-defined whether function keys are enabled for subsequent operations using the file handle. 14

A locator directly manipulates the locator cursor associated with a terminal. Actions associated with the locator also generate data that may be read as input. Locator data is generated whenever a button on the locator is pressed or released and whenever the locator cursor moves, due to locator movement, across character cell boundaries. 15

An *input event* is an action performed on either the keyboard or the locator. There are five different types of input events: striking a normal keyboard key, striking a function key, moving a locator, pressing a locator button, and releasing a locator button after it has been pressed. Different information is captured for different event types. All input events are entered into an *input event queue* from which they may be extracted. Input operations, instead of accessing the keyboard or locator directly, retrieve events from this queue. The input operations provided by the package CAIS_TERMINAL_IO are 16



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

READ_INPUT_EVENT and READ_STRING. An operation also exists to flush the input event queue. Generation of locator button and locator movement events can be enabled and disabled using the ENABLE_LOCATOR interface which enables and disables the locator for a virtual terminal. The locator cursor is displayed only while the locator is enabled. When a terminal file handle is opened, the intial configuration of the virtual terminal locator (i.e. its position and whether it is enabled) is implementation- defied.

## 8.7.1 Protocol parameters

The IO connection model allows protocol-specific parameters to be determined during the establish- 1
ment of a channel (see Section 7.3.1.3 on page 576). There are no protocol-specific parameters for CAIS_TERMINAL_IO .

## 8.7.2 Types, Subtypes, Constants and Exceptions

```
typo FILE_TYPE is limited private;
```

FILE_TYPE describes the type for file handles for all terminal input and output operations. When a 1
file handle of this type is opened protocol compatibility checking is performed using the mode INOUT_-
FILE.

```
subtype PHYSICAL_DISPLAY_DIMENSION is
    CAIS_POSITIVE rang. 1.. CAIS_PRAGMATICS.MAXIMUM_PHYSICAL_DISPLAY_DIMENSION;

subtype VIRTUAL_DISPLAY_COORDINATE is CAIS_INTEGER;

type PHYSICAL_DISPLAY_POSITION is
    record
        ROW     : PHYSICAL_DISPLAY_DIMENSION;
        COLUMN  : PHYSICAL_DISPLAY_DIMENSION;
    and record;

type VIRTUAL_DISPLAY_POSITION is
    record
        ROW     : VIRTUAL-DISPLAY-COORDINATE;
        COLUMN  : VIRTUAL-DISPLAY-COORDINATE;
    and record;

VIRTUAL_DISPLAY_POSITION_ERROR : exception;
```

A value of type PHYSICAL_DISPLAY_POSITION represents the coordinates of a character cell on 2
a physical display. A value of type VIRTUAL_DISPLAY_POSITION represents the coordinates of a character cell on the virtual display. The exception VIRTUAL_DISPLAY_POSITION_ERROR is raised if an attempt is made to paste a screen such that the screen's maximum cell would be outside the boundaries of a virtual display, if the physical display is oriented so as to fall outside the boundaries of a virtual display, or if the locator cursor is set to a position on the virtual display to which the physical display is not mapped.



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
type SCREEN_TYPE is limited private;

subtype SCREEN_DIMENSION is
    CAIS_POSITIVE rang. 1 ..CAIS_PRAGMATICS.MAXIMUM_SCREEN_DIMENSION;

type SCREEN_POSITION is
    record
        ROW      : SCREEN_DIMENSION;
        COLUMN : SCREEN_DIMEMSION;
    and record;

SCREEN_POSITION_ERROR : exception;
```

A value of type SCREEN_TYPE is termed a *screen handle.* An open screen handle uniquely identifies a screen. A screen handle is opened by CREATE_SCREEN, COPY_SCREEN, and GET_SCREEN_-CONTAINING_LOCATOR_ CURSOR operations and closed by a DELETE_SCREEN operation. 3

Positions within screens are denoted by values of type SCREEN_ POSITION. The exception SCREEN_-POSITION-ERROR is raised if output is attempted to a screen outside its boundaries. 4

```
SCREEN_NOT_ON_DISPLAY : exception;
LOCATOR_STATUS_ERROR : exception;
```

The exception SCREEN_NOT_ON_DISPLAY is raised if an attempt is made to manipulate a screen on a virtual display to which it is not pasted. 5

The exception LOCATOR_STATUS_ERROR is raised if the locator is disabled and an operation requires it to be enabled, or if the location of the locator cursor is inappropriate for an operation. 6

```
type GRAPHIC_RENDITION_KIND is
    (PRIMARY_RENDITION, BOLD, FAINT, ITALIC, UNDERSCORE, SLOW_BLINK,
     RAPID_BLINK , REVERSE_IMAGE);

type BRAPHIC_RENDITION_SET is array (GRAPHIC_RENDITION_KIND) of BOOLEAN;

type FOREGROUND_COLOR_TYPE is private;
type BACKGROUND_COLOR_TYPE is private;

BLACK-BACKGROUND     : constant BACKGROUND_COLOR-TYPE;
WHITE-BACKGROUND     : constant BACKGROUND_COLOR-TYPE;
RED-BACKGROUND       : constant BACKGROUND_COLOR-TYPE;
YELLOW-BACKGROUND    : constant BACKGROUND_COLOR-TYPE;
BLUE-BACKGROUND      : constant BACKGROUND_COLOR-TYPE;
GREEN-BACKGROUND     : constant BACKGROUND_COLOR-TYPE;
MAGENTA-BACKGROUND   : constant BACKGROUND_COLOR-TYPE;
CYAN-BACKGROUND      : constant BACKGROUND_COLOR-TYPE;

BLACK-FOREGROUND     : constant FOREGROUND_COLOR.TYPE;
WHITE-FOREGROUND     : constant FOREGROUND_COLOR-TYPE;
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
RED_FOREGROUND       : constant FOREGROUND_COLOR_TYPE;
YELLOW_FOREGROUND    : constant FOREGROUND_COLOR_TYPE;
BLUE_FOREGROUND      : constant FOREGROUND_COLOR_TYPE;
GREEN_FOREGROUND     : constant FOREGROUND_COLOR_TYPE;
MAGENTA_FOREGROUND   : constant FOREGROUND_COLOR_TYPE;
CYAN_FOREGROUND      : constant FOREGROUND_COLOR_TYPE;

type RENDITION_TYPE is
   record
      KIND                : GRAPHIC_RENDITION_SET;
      FOREGROUND_COLOR    : FOREGROUND_COLOR_TYPE ;
      BACKGROUND_COLOR    : BACKGROUND_COLOR_TYPE;
   and record;
```

A value of type RENDITION_TYPE represents the displayable characteristics of a character cell or cells. The field KIND denotes non-color characteristics of the the cell, the field FOREGROUND_-COLOR denotes the color of the textual character, and the field BACKGROUND_COLOR denotes the color of the background. 7

The type GRAPHIC_RENDITION_KIND enumerates the possible displayable, non-color characteristics of a character cell or cells. These non-color cell characteristics are intended to affect the display of a character cell in the same manner as the corresponding graphic renditions defined in [ANSI 79] or the closest possible approximation thereof. The type GRAPHIC_RENDITION_SET is used to express a set oft he possible non-color cell characteristics where the Boolean-valued array component corresponding to each characteristic indicates whether that characteristic is included within (i.e. a member of) the set. A value of the type GRAPHIC_RENDITION_SET denotes the visual effect achieved through the combination of the non-color characteristics which are included within the set. 8

Values of the types FOREGROUND_COLOR_TYPE and BACKGROUND_COLOR_TYPE represent foreground and background colors respectively. The foreground and background colors defined by the CAIS are those denoted by the constants of these types indicated above. An implementation should map each of these CAIS-defined foreground and background colors into the color (if any) supported by the physical display which most closely approximates the color indicated in the name of the constant. No such mapping is necessary for implementations which support only monochrome displays. 9

An implementation is not required to produce the visual effect implied by a rendition if it is beyond the capabilities of the physical display being used. (A physical display may not support all of the non-color characteristics or all possible combinations thereof. A physical display may be capable of displaying only a subset of the CAIS-defined foreground and background colors.) However, an implementation is required to permit the specification of any possible rendition for a character cell or screen border even if the rendition specification has no effect or a degenerate effect on the appearance of the character cell or the screen border. 10

```
typE SCROLL_TYPE is (UP, DOW);
```

A value of type SCROLL_TYPE determines whether scrolling is to be done in an upward or downward direction. A setting of UP implies that text is moved towards the top of the screen. 11

```
type WRAP_TYPE is (ON, OFF);
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

A value of type WRAP_TYPE determines whether wrapping takes place during output operations to a screen. 12

```
typo ECHO_TYPE is (ON, OFF);
```

A value of type ECHO_TYPE determines whether or not keyboard input is echoed to a screen. 13

```
typo TERMINATOR_SET is array (CHARACTER) of BOOLEAN;
```

A value of type TERMINATOR-SET is a set of characters that are to act as terminators during a keyboard input operation. 14

```
subtype FUNCTION_KEY_NAME is STRING;

FUNCTION_KEY_STATUS_ERROR : exception;
```

A value of type FUNCTION_KEY_NAME is the implementation-dependent name used to identify a function key supported by a particular virtual keyboard. FUNCTION_KEY_STATUS_ERROR is raised if a non-existent function key is referenced. 15

```
subtype BUTTOH_NAME is STRING;

BUTTON_NAME_ERROR : exception;
```

A value of type BUTTON_NAME is the implementation-dependent name used to identify a button supported by a particular virtual locator. BUTTON_NAME_ERROR is raised if a non-existent button is referenced. 16

```
typo INPUT_EVENT-KIND is
    (NORMAL_KEY_STROKE, FUNCTION_KEY_STROKE, LOCATOR_MOVEMENT,
     BUTTON_DOWN, BUTTON_UP);

INPUT_EVENT_QUEUE_EMPTY :    exception;
```

A value of type INPUT_EVENT_KIND denotes whether an input event corresponds to striking a normal key (a non-function key), striking a function key, moving a locator, pressing a locator button, or releasing a locator button after it has been pressed. An input operation returns differing information depending upon the kind of the input event. 17

The exception INPUT_EVENT_QUEUE_EMPTY is raised if a non-blocking read request is made and the input event queue is empty. 18

## 8.7.8 Opening a terminal file handle.

```
procedure OPEN
    (TERMINAL          : in out FILE_TYPE;
     CHANNEL           : in CHANNEL_NODE_TYPE;
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
OPPOSITE_CLIENT   : in      CLIENT_NODE_TYPE;
SIZE              : out     PHYSICAL_DISPLAY_POSITION;
DEFAULT_RENDITION : out     RENDITION_TYPE;
ORIGIN            : in      VIRTUAL_DISPLAY_POSITION := (1,1);
TIME_LIMIT        : in      CAIS_IO_DURATION         := LONG_IO_DELAY;
STATUS            : in      STATUS_INDICATOR         := NO-INDICATION);
```

Purpose:

This procedure opens a terminal file handle for communicating over the channel identified by CHANNEL to the terminal identfied by OPPOSITE_CLIENT. CAIS_TERMINAL_IO must be an allowed protocol of the CHANNEL node identified by CHANNEL. 1

The minimum cell of the physical display is mapped onto position ORIGIN of the virtual display. The size of the physical display is returned in SIZE; the default rendition of the virtual terminal is returned in DEFAULT_RENDITION. Blanks are written, using the default rendition, to all character cells of the physical display. 2

If the operation concludes successfully, the CONTROLLING-PROTOCOL attribute on the CHANNEL node is set to CAIS_TERMINAL_IO and the channel is in the ESTABLISHED state. 3

Parameters:

**TERMINAL** is a terminal file handle, initially closed, to be opened. 4

**CHANNEL** is an open node handle to a CHANNEL node to which both the current process and OPPOSITE_CLIENT are client nodes. 5

**OPPOSITE_CLIENT** is an open node handle identifying a terminal. 6

**SIZE** is the returned position of the maximum cell of the physical display. 7

**DEFAULT_RENDITION** is the returned default rendition supported by the virtual terminal. 8

**ORIGIN** is the position on the virtual display to be associated with the minimum cell of the physical display. 9

**TIME_LIMIT** in the time limit for waiting for the OPEN to complete. 10

**STATUS** is an established status indicator. 11

Exceptions:

**STATUS_ERROR** is raised if TERMINAL is an open terminal file handle [7], if OPPOSITE_CLIENT is not an open node handle [2], if CHANNEL is not an open node handle [2], if the channel denoted by CHANNEL is not in the CONNECTED state [9] or if STATUS is not an established status indicator [3]. 12

**NODE_KIND_ERROR** is raised if the node identified by CHANNEL is not a CHANNEL node [1]. 13

**CLIENT_ERROR** is raised if the current process node [1] or the node identified by OPPOSITE_CLIENT [5] is not a client node of the channel node identified by CHANNEL. 14

**PORT_ERROR** is raised if the DIRECTION attributes of the PORT relationships are missing or have incompatible values [4]. 15



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

**PROTOCOL_ERROR** is raised if CAIS_TERMINAL_IO is not an ALLOWED_PROTOCOL of both PORT relationships of the channel [1] or if the value of the CONTROLLING. -PROTOCOL attribute is not CAIS_TERMINAL_IO or NO_PROTOCOL [2]. 16

**MODE_ERROR** is raised if the implied mode INOUT_FILE is incompatible with the DIRECTION attribute of the local PORT relationship as determined by Table XXXIV on page 581 [2]. 17

**USE_ERROR** is raised if the opposite client node of the channel is a SECONDARY_STORAGE or DEVICE node and the implementation can determine that the structure of the content is incompatible with the protocol according to implementation-defined criteria [54], or if the implementation does not support data transfer between the types of clients involved [55], or if another open file handle identifies the same physical terminal and the CAIS implementation does not support the existence of multiple file handles identifying the same terminal [41]. Any such restriction must be documented in Appendix F. An implementation is allowed to raise this exception" only if it is based on operating system support that does not provide this capability. 18

**TRANSACTION_ERROR** is raised if the operation is included in more than one transaction [1] or if the operation is included in a transaction and the operation modifies a node which is not included in the same transaction [2]. 19

**INTENT_VIOLATION** is raised if the node handle OPPOSITE-CLIENT was not opened with an intent specification including at least the intents required for the implied mode INOUT_FILE as determined by Table XXXIV on page 581 [1]. 20

**VIRTUAL_DISPLAY_POSITION_ERROR** is raised if any portion of the physical display would lie outside of the boundaries of the virtual display [1]. 21

**OPEN_DENIED** is raised if a remote client process or the opposite client, if it is a process, deleted the channel, signaling the denial of the connection attempt [1]. 22

**TIME_EXPIRED** is raised if the OPEN operation is delayed beyond the specified time limit for reasons other than a lock conflict [1]. 23

**SECURITY_VIOLATION** is raised if the operation represents a violation of mandatory access controls. SECURITY_VIOLATION is raised only if the conditions for other exceptions are not present. 24

Notes:

Closing the node handle OPPOSITE_CLIENT associated with the file handle TERMINAL closes the file handle. 25

When the file handle TERMINAL is opened, it is implementation-defined as to whether function keys and the virtual terminal locator are enabled. 26

## 8.7.4 Closing a terminal file handle

```
procedure CLOSE
    (TERMINAL : in out FILE_TYPE;
     STATUS   : in     STATUS- INDICATOR := NO_INDICATION);
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Purpose:

This procedure severs the association between the internal terminal file denoted by the terminal file handle TERMINAL and its associated channel. The terminal file handle becomes closed. All screens pasted to the virtual display using the terminal file handle are unpasted. Closing an already closed terminal file handle has no effect. 1

The state of the channel associated with TERMINAL becomes CLOSED if presently ESTABLISHED and becomes CONNECTED if already CLOSED. If the opposite client node is a file, the channel becomes immediately CONNECTED. If the channel becomes CONNECTED, the CONTROLLING_PROTOCOL attribute of the CHANNEL node is set to NO_PROTOCOL. 2

Parameters:

**TERMINAL** is a terminal file handle to be closed. 3

**STATUS** is an established status indicator. 4

Exceptions:

**STATUS_ERROR** is raised if STATUS is not an established status indicator [3]. 5

**TRANSACTION_ERROR** is raised if the operation is included in a transaction and the file handle TERMINAL was not opened inside the transaction. 6

## 8.7.5 Determining whether a terminal file handle is open

```
function IS_OPEN
    (TERMINAL : in FILE_TYPE;
     STATUS   : in STATUS_INDICATOR : = NO_INDICATION)
    return BOOLEAN;
```

Purpose:

This fiction returns TRUE if the terminal file handle is open; otherwise, it returns FALSE. 1

Parameters:

**TERMINAL** is a terminal file handle. 2

**STATUS** is an established status indicator. 3

Exceptions:

**STATUS_ERROR** is raised if STATUS is not an established status indicator [3]. 4

## 8.7.6 Determining the channel node associated with a terminal file handle

```
procedure GET_ASSOCIATED_CHAESEL
    (CHANNEL  : in out CHANNEL_NODE_TYPE;
     TERMINAL : in     FILE_TYPE;
     STATUS   : in     STATUS_INDICATOR := NO_INDICATION);
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Purpose:

This procedure returns a node handle, with intent NO_ACCESS, to the CHANNEL node associated with the terminal file handle TERMINAL. Note that the association was made when the file handle was initially opened. 1

The last relationship traversed in opening the node handle CHANNEL is the same as that of the CHANNEL node handle provided when the file handle TERMINAL was opened. See subprograms PATH_KEY (Section 5.2.20 on page 119) and PATH_RELATION (Section 5.2.21 on page 120)$_0$ 2

Parameters:

**CHANNEL** is a node handle, initially closed, to be opened to the associated CHANNEL node. 3

**TERMINAL** is an open terminal file handle identifying the internal file whose associated CHANNEL node is required. 4

**STATUS** is an established status indicator. 5

Exceptions:

**STATUS_ERROR** is raised if TERMINAL is not an open terminal file handle [8], if CHANNEL is open at the time of the call [1], or if STATUS is not an established status indicator [3]. 6

**SECURITY_VIOLATION** is raised if the operation represents a violation of mandatory access controls. SECURITY_ VIOLATION is raised only if the conditions for other exceptions are not present. 7

### 8.7.7 Determining the physical display origin on the virtual display

```
function PHYSICAL_DISPLAY_ORIGIS
    (TERMINAL :  in  FILE_TYPE;
     STATUS      : in STATUS_INDICATOR := NO-INDICATION)
    return  VIRTUAL_DISPLAY_POSITION;
```

Purpose:

This function returns the current orientation of the physical display (viewport) on the virtual display associated with the terminal file handle TERMINAL. 1

Parameters:

**TERMINAL** is an open terminal file handle.

**STATUS** is an established status indicator.



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Exceptions:

**STATUS_ERROR** is raised if TERMINAL is not an open terminal file handle [8] or if STATUS is not an established status indicator [3]. 4

## 8.7.8 Re-orienting the physical display on the virtual display

```
procedure  MOVE_PHYSICAL_DISPLAY_ORIGIN
    (TERMINAL  : in  FILE_TYPE;
     ORIGIN    : in  VIRTUAL_DISPLAY_POSITION;
     STATUS    : in STATUS_INDICATOR := NO_INDICATION);
```

Purpose:

This procedure changes the orientation of the physicd display (viewport) on the virtual display. The minimum cell of the physical display is aligned with the position ORIGIN of the virtual display. 1

The virtual display is redrawn by first erasing the physical display and then drawing all pasted screens that are visible based on the new origin of the physical display within the virtual display coordinate system. 2

Parameters:

**TERMINAL** is an open terminal file handle. 3

**ORIGIN** is the new position on the virtual display that corresponds to the minimum cell of the physical display. 4

**STATUS** is an established status indicator. 5

Exceptions:

**STATUS_ERROR** is raised if TERMINAL is not an open terminal file handle [8] or if STATUS is not an established status indicator [3]. 6

**VIRTUAL_DISPLAY-POSITION_ERROR** is raised if any portion of the physical display would lie outside of the boundaries of the virtual display [1]. 7

## 8.7.9 Determining if a terminal supports a set of graphic renditions

```
function  GRAPHIC_RENDITION_SET_SUPPORTED
    (TERMINAL :  in  FILE_TYPE;
     SET          : in CRAPHIC_RENDITION_SET;
     STATUS       : in STATUS_INDICATOR : = NO_INDICATION)
    return BOOLEAN;
```


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Purpose:

This function returns TRUE if the combination of non-color characteristics included in the GRAPHIC_RENDITION_SET value SET is supported by the physical display associated with TERMINAL; otherwise, it returns FALSE. 1

Parameters:

**TERMINAL** is an open terminal file handle. 2

**SET** is a combination of non-color characteristics of a rendition. 3

**STATUS** is an established status indicator. 4

Exceptions:

**STATUS_ERROR** is raised if TERMINAL is not an open terminal file handle [8] or if STATUS is not an established status indicator [3]. 5

## 8.7.10 Determining if a terminal supports a foreground color

```
function COLOR-SUPPORTED
    (TERMINAL :  in FILE_TYPE;
     COLOR     : in FOREGROUND_COLOR_TYPE;
     STATUS    : in STATUS_INDICATOR := NO-INDICATION)
   return BOOLEAN;
```

Purpose:

This function returns TRUE if the physical display associated with TERMINAL supports the foreground color identified by COLOR; otherwise, it returns FALSE. 1

Parameters:

**TERMINAL** is an open terminal file handle. 2

**COLOR** is a foreground color specification. 3

**STATUS** is an established status indicator. 4

Exceptions:

**STATUS_ERROR** is raised if TERMINAL is not an open terminal file handle [8] or if STATUS is not an established status indicator [3]. 5

## 8.7.11 Determining if a terminal supports a background color

```
function COLOR_SUPPORTED
    (TERMINAL :  in FILE_TYPE;
     COLOR     : in BACKGROUND_COLOR_TYPE;
     STATUS    : in STATUS_INDICATOR : = NO_INDICATION)
   return BOOLEAN;
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Purpose:

This function returns TRUE if the physical display associated with TERMINAL supports the background color identified by COLOR; otherwise, it returns FALSE. 1

Parameters:

**TERMINAL** is an open terminal file handle. 2

**COLOR** is a background color specification. 3

**STATUS** is an established status indicator. 4

Exceptions:

**STATUS_ERROR** is raised if TERMINAL is not an open terminal file handle [8] or if STATUS is not an established status indicator [3]. 5

## 8.7.12 Starting output buffering for a virtual display

```
procedure   BEGIN_VIRTUAL_DISPLAY_BUFFERING
    (TERMINAL :   in  FILE_TYPE;
     STATUS       : in STATUS_INDICATOR := NO_INDICATION);
```

Purpose:

This procedure causes subsequent output operations to the virtual display associated with TERMINAL to be buffered. This permits a sequence of changes to be hidden from the terminal user until they are ready to be applied in a single update. 1

A counter maintained with each virtual terminal is incremented by a BEGIN_VIRTUAL_DIS_PLAY_BUFFERING operation and decremented by an END_VIRTUAL_DISPLAY_BUFFERING operation. When the counter's value is zero, updates to the virtual display can be made visible. 2

Parameters:

**TERMINAL** is an open terminal file handle. 3

**STATUS** is an established status indicator. 4

Exceptions:

**STATUS_ERROR** is raised if TERMINAL is not an open terminal file handle [8] or if STATUS is not an established status indicator [3]. 5

## 8.7.13 Ending output buffering for a virtual display

```
procedure   END_VIRTUAL_DISPLAY_BUFFERING
    (TERMINAL :   in  FILE_TYPE;
     STATUS       : in STATUS_INDICATOR := NO_INDICATION);
```


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Purpose:

This procedure pairs with BEGIN_VIRTUAL_DISPLAY_BUFFERING to control the user visibility of operations to a particular virtual display. A counter maintained with each virtual terminal is incremented by a BEGIN_VIRTUAL_DISPLAY_BUFFERING operation and decremented by an END_VIRTUAL_DISPLAY_BUFFERING operation. When the counter's value is zero, updates to the virtual display can be made visible to the user. If the counter is already zero, this operation has no effect. 1

Parameters:

**TERMINAL** identifies an open terminal file handle. 2

**STATUS** is an established status indicator. 3

Exceptions:

**STATUS_ERROR** is raised if TERMINAL is not an open terminal file handle [8] or if STATUS is not an established status indicator [3]. 4

## 8.7.14 Refreshing a virtual display

```
procedure REFRESH_VIRTUAL_DISPLAY
    (TERMINAL : in FILE_TYPE;
     STATUS   : in STATUS_INDICATOR := NO_INDICATION);
```

Purpose:

This procedure refreshes the virtual display associated with TERMINAL by erasing the physical display and then redrawing all pasted screens. The pasting order is not affected by this procedure. 1

Parameters:

**TERMINAL** is an open terminal file handle. 2

**STATUS** is an established status indicator. 3

Exceptions:

**STATUS_ERROR** is raised if TERMINAL is not an open terminal file handle [8] or if STATUS is not an established status indicator [3]. 4

## 8.7.15 Creating a screen

```
procedure CREATE_SCEEN
    (SCREEN           : in out SCREEN_TYPE;
     SIZE             : in     SCREEN_POSITION;
     ACTIVE_RENDITION : in     RENDITION_TYPE;
     BORDER_ON        : in     BOOLEAN;
     BORDER_RENDITION : in     RENDITION_TYPE;
     STATUS           : in     STATUS_INDICATOR := NO_INDICATION);
```


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Purpose:

This procedure creates a new screen with a given size, active rendition, and bordering and returns its handle. After the active rendition is set to ACTIVE_RENDITION, blanks are written to the entire screen. The cursor is positioned in the minimum cell. 1

Parameters:

**SCREEN** is a screen handle, initially closed, to be opened to the created screen. 2

**SIZE** denotes the size of the screen to be created by stating the coordinates of the maximum cell. 3

**ACTIVE_RENDITION** is the active rendition for the created screen. 4

**BORDER_ON** determines whether or not a border is displayed. 5

**BORDER_RENDITION** is the rendition applied to the border if the screen is bordered. 6

**STATUS** is an established status indicator. 7

Exceptions:

**STATUS_ERROR** is raised if SCREEN is an open screen handle [10] or if STATUS is not an established status indicator [3]. 8

Notes:

The value of BORDER_ RENDITION may still be significant even if BORDER_ON is FALSE. This value is returned by the function BORDER_RENDITION even when a screen is not bordered. 9

## 8.7.16 Changing the active rendition for a screen

```
procedure CHANGE_ACTIVE_RENDITION
    (SCREEN           : in SCREEN_TYPE;
     ACTIVE_RENDITION : in RENDITION_TYPE;
     STATUS           : in STATUS_INDICATOR : = NO_INDICATION);
```

Purpose:

This procedure is used to change the active rendition of a screen. Subsequent output to the specified screen will use this new active rendition. 1

Parameters:

**SCREEN** is an open screen handle. 2

**ACTIVE_RENDITION** is the new active rendition for the specified screen. 3

**STATUS** is an established status indicator. 4


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Exceptions:

**STATUS_ERROR** is raised if SCREEN is not an open screen handle [11] or if STATUS is not an established status indicator [3]. 5

## 8.7.17 Changing screen size

```
procedure CHANGE_SCREEN_SIZE
    (SCREEN : in SCREEN_TYPE ;
     SIZE   : in SCREEN_POSITION;
     STATUS : in STATUS_INDICATOR := NO_INDICATION);
```

Purpose:

This procedure is used to change the size of a screen. If the new size of the screen is smaller, text is truncated. If the new size is larger, text is padded on the right with blanks using the active rendition of the screen. The cursor position is set to the minimum cell. If already pasted, the screen is unpasted and repasted with its new size thereby moving the screen to the front of the pasting order. 1

Parameters:

**SCREEN** is an open screen handle. 2

**SIZE** contains the coordinates of the maximum cell in SCREEN. 3

**STATUS** is an established status indicator. 4

Exceptions:

**STATUS_ERROR** is raised if SCREEN is not an open screen handle [11] or if STATUS is not an established status indicator [3]. 5

**VIRTUAL_DISPLAY_POSITION_ERROR** is raised if any portion of the screen (other than its border) would lie outside of the boundaries of the virtual display [2]. 6

## 8.7.18 Changing border characteristics

```
procedure CHANGE_BORDER
    (SCREEN           : in SCREEN_TYPE;
     BORDER_ON        : in BOOLEAN;
     BORDER_RENDITION : in RENDITION_TYPE;
     STATUS           : in STATUS_INDICATOR := NO_INDICATION);
```

Purpose:

This procedure is used to change the existence and rendition of a screen border. If BORDER_ON is TRUE, the screen is outlined by a border with an applied rendition of BORDER-RENDITION. If already pasted, the screen is unpasted and repasted with its new border characteristics thereby moving the screen to the front of the pasting order. 1


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Parameters:

**SCREEN** is an open screen handle. 2

**BORDER_ON** determines whether or not a border is displayed. 3

**BORDER_RENDITION** is the rendition that is applied to the border if the screen is bordered. 4

**STATUS** is an established status indicator. 5

Exceptions:

**STATUS_ERROR** is raised if SCREEN is not an open screen handle [11] or if STATUS is not an established status indicator [3]. 6

Notes:

The value of BORDER_RENDITION is still significant even if BORDER_ON is FALSE. This border rendition is returned by the function BORDER_RENDITION even when a screen is not bordered. 7

## 8.7.10 Creating a copy of an existing screen

```
procedure COPY_SCREEN
    (SCREEN     : in     SCREEN_TYPE;
     NEW_SCREEN : in out SCREEN_TYPE;
     STATUS     : in     STATUS_INDICATOR := NO_INDICATION);
```

Purpose:

This procedure creates a copy of an existing screen SCREEN and assigns it to the screen handle NEW_SCREEN. The newly created screen has the same size, active rendition, and border characteristics as the original. As well, the content and rendition of corresponding character cells is identical as is the cursor position. The new screen is not buffered nor is it pasted to any virtual display. 1

Parameters:

**SCREEN** is an open screen handle. 2

**NEW_SCREEN** is a screen handle to the newly created screen. 3

**STATUS** is an established status indicator. 4

Exceptions:

**STATUS_ERROR** is raised if SCREEN is not an open screen handle [11], if NEW_SCREEN is an open screen handle [10], or if STATUS is not an established status indicator [3]. 5


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

## 8.7.20 Deleting a screen

```
procedure DELETE_SCREEN
    (SCREEN : in out SCREEN_TYPE;
     STATUS : in     STATUS_INDICATOR := NO_INDICATION);
```

Purpose:

This procedure unpastes SCREEN if it is pasted and effectively deletes it by closing the screen handle. Deletion of a closed screen has no effect. 1

Parameters:

**SCREEN** is an open screen handle identifying the screen to be deleted. 2

**STATUS** is an established status indicator [3]. 3

Exceptions:

**STATUS_ERROR** is raised if STATUS is not an established status indicator [3]. 4

## 8.7.21 Determining the size of a screen

```
function SIZE
    (SCREEN : in SCREEN_TYPE;
     STATUS : in STATUS_INDICATOR : = NO_INDICATION)
    return SCREEN_POSITION;
```

Purpose:

This function returns the size of SCREEN by returning the coordinates of maximum cell in the screen. 1

Parameters:

**SCREEN** is an open screen handle. 2

**STATUS** is an established status indicator. 3

Exceptions:

**STATUS_ERROR** is raised if SCREEN is not an open screen handle [11] or if STATUS is not an established status indicator [3]. 4

## 8.7.22 Determining the active rendition of a screen

```
function ACTIVE_RENDITION
    (SCREEN : in SCREEN_TYPE;
     STATUS : in STATUS_INDICATOR := IO_INDICATION)
    return RENDITION_TYPE;
```


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Purpose:

This function returns the active rendition of SCREEN.

Parameters:

**SCREEN** is an open screen handle. 2

**STATUS** is an established status indicator. 3

Exceptions:

**STATUS_ERROR** is raised if SCREEN is not an open screen handle [11] or if STATUS is not an established status indicator [3]. 4

## 8.7.23 Determining whether a screen has a border

```
function BORDER-OH
    (SCREEN :  in SCREEN_TYPE;
     STATUS :  in STATUS_INDICATOR := IO_INDICATION)
    return BOOLEAN;
```

Purpose:

This function returns TRUE if SCREEN is bordered; otherwise, it returns FALSE. 1

Parameters:

**SCREEN** is an open screen handle. 2

**STATUS** is an established status indicator. 3

Exceptions:

**STATUS_ERROR** is raised if SCREEN is not an open screen handle [11] or if STATUS is not an established status indicator [3]. 4

## 8.7.24 Determining the border rendition of a screen

```
function BORDER_RENDITION
    (SCREEN :  in SCREEN_TYPE;
     STATUS  : in STATUS_INDICATOR := NO_INDICATION)
    return RENDITION_TYPE;
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Purpose:

This function returns the current rendition applied to the border of the screen identified by SCREEN. 1

Parameters:

**SCREEN** is an open screen handle.

**STATUS** is an established status indicator.

Exceptions:

**STATUS_ERROR** is raised if SCREEN is not an open screen handle [11] or if STATUS is not an established status indicator [3]. 4

Notes:

The value of BORDER_RENDITION is still significant even if BORDER_ON is FALSE. Its value is the value of the BORDER_RENDITION parameter passed to the CREATE_SCREEN or CHANGE_BORDER interfaces. 5

## 8.7.25 Querying sameness of screens

```
function IS_SAME
    (SCREEN1 : in SCREEN_TYPE;
     SCREEN2 : in SCREEN_TYPE;
     STATUS  : in STATUS-INDICATOR : = NO_INDICATION)
    return BOOLEAN;
```

Purpose:

This function returns TRUE if the screens identified by its arguments are the same screen; otherwise, it returns FALSE. 1

Parameters:

**SCREEN1** is an open screen handle to a screen. 2

**SCREEN2** is an open screen handle to a screen. 3

**STATUS** is an established status indicator. 4

Exceptions:

**STATUS_ERROR** is raised if one or both of the screen handles SCREEN1 or SCREEN2 is not open [11] or if STATUS is not an established status indicator [3]. 5


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

## 8.7.26 Pasting a screen onto the virtual display

```
procedure PASTE_SCREEN
    (TERMINAL : in FILE_TYPE;
     SCREEN   : in SCREW_TYPE;
     POSITION : in VIRTUAL_DISPLAY_POSITION;
     STATUS   : in STATUS_INDICATOR := IO_INDICATION);
```

Purpose:

This procedure pastes a screen on a virtual display by aligning the minimum cell of the screen with the position POSITION of the virtual display. If the screen is already pasted, it is first unpasted. The newly pasted screen becomes the front screen. 1

Parameters:

**TERMINAL** is an open terminal file handle. 2

**SCREEN** is an open screen handle identifying the screen to be pasted. 3

**POSITION** is the position on the virtual display at which the screen is to be pasted. 4

**STATUS** is an established status indicator. 5

Exceptions:

**STATUS_ERROR** is raised if SCREEN is not an open screen handle [11], if TERMINAL is not an open terminal file handle [8], or if STATUS is not an established status indicator [3]. 6

**VIRTUAL_DISPLAY_POSITION_ERROR** is raised if any portion of the screen (other than its border) would lie outside of the boundaries of the virtual display [2]. 7

Notes:

If the screen has already been pasted, this operation may force it to become unpasted from one virtual display and pasted onto another. 8

A screen can be pasted on the virtual display such that it is not totally visible on the physical display (viewport). 9

## 8.7.27 Removing a screen from a virtual display

```
procedure UNPASTE_SCREEN
    (SCREEN : in SCREEN_TYPE;
     STATUS : in STATUS_INDICATON := NO_INDICATION);
```

Purpose:

This procedure causes a screen to be unpasted from the virtual display to which it is presently pasted. The contents of the screen are not affected, but the association between the screen and the virtual display is severed and hence the visibility of the screen may be affected (invisible screens are unaffected). Regions of screens occluded by the screen being unpasted become visible again. 1



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Parameters:

**SCREEN** is an open screen handle identifying the screen to be unpasted.

**STATUS** is an established status indicator.

Exceptions:

**STATUS_ERROR** is raised if SCREEN is not an open screen handle [11], or if STATUS is not an established status indicator [3]. 4

**SCREEN_NOT_ON_DISPLAY** is raised if SCREEN is not currently pasted [1]. 5

## 8.7.28 Relocating a screen on a virtual display

```
procedure MOVE_SCREEN
    (SCREEN   : in SCREEN_TYPE;
     POSITION : in VIRTUAL_DISPLAY_POSITION;
     STATUS   : in STATUS_INDICATOR := NO_INDICATION);
```

Purpose:

This procedure moves a screen to a new position on the virtual display by aligning the minimum cell of the screen with the position POSITION on the virtual display to which it is pasted. This operation preserves the pasting order. 1

Parameters:

**SCREEN** is an open screen handle. 2

**POSITION** is the position on the virtual display to which the screen is to be moved. 3

**STATUS** is an established status indicator. 4

Exceptions:

**STATUS_ERROR** is raised if SCREEN is not an open screen handle [11], or if STATUS is not an established status indicator [3]. 5

**SCREEN_NOT_ON_DISPLAY** is raised if SCREEN is not currently pasted [1]. 6

**VIRTUAL-DISPLAY_POSITION_ERROR** is raised if any portion of the screen (other than its border) would lie outside of the boundaries of the virtual display [2]. 7

## 8.7.29 Move screen to front

```
procedure MOVE_SCREEN_TO_FRONT
    (SCREEN : in SCREEN_TYPE;
     STATUS : in STATUS_INDICATOR := NO_INDICATION);
```


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Purpose:

This procedure fist unpastes and then repastes a screen in the same position thereby moving the screen to the front of the pasting order. This operation may cause other screens to be occluded. 1

Parameters:

**SCREEN** is an open screen handle.

**STATUS** is an established status indicator.

Exceptions:

**STATUS_ERROR** is raised if SCREEN is not an open screen handle [11], or if STATUS is not an established status indicator [3]. 4

**SCREEN_NOT_ON_DISPLAY** is raised if SCREEN is not currently pasted [1]. 5

## 8.7.30 Move screen to back

```
procedueo MOVE_SCREEN_TO_BACK
    (SCREEN :  in  SCREEN_TYPE;
     STATUS :  in STATUS_INDICATOR : = NO_INDICATION);
```

Purpose:

This procedure repastes a screen from its current pasting order to the back of the paste order. This operation may cause the screen to be occluded by other screens forward in the pasting order. 1

Parameters:

**SCREEN** is an open screen handle.

**STATUS** is an established status indicator.

Exceptions:

**STATUS_ERROR** is raised if SCREEN is not an open screen handle [11], or if STATUS is not an established status indicator [3]. 4

**SCREEN_NOT_ON_DISPLAY** is raised if SCREEN is not currently pasted [1]. 5

## 8.7.31 Determining paste position of a screen

```
function   PASTE_POSITION
    (SCREEN :  in  SCREEN_TYPE;
     STATUS :  in STATUS_INDICATOR := NO_INDICATION)
   return   VIRTUAL_DISPLAY_POSITION;
```


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Purpose:

This procedure returns the paste position of SCREEN. The position returned contains the coordinates, relative to the virtual display, of the screen's minimum cell. 1

Parameters:

**SCREEN** is an open screen handle. 2

**STATUS** is an established status indicator. 3

Exceptions:

**STATUS_ERROR** is raised if SCREEN is not an open screen handle [11], or if STATUS is not an established status indicator [3]. 4

**SCREEN_NOT_ON_DISPLAY** is raised if SCREEN is not currently pasted [1]. 5

## 8.7.32 Determining whether a screen is occluded

```
function SCREEN_OCCLUDED
    (SCREEN :  in SCREEN_TYPE;
     STATUS :  in STATUS_INDICATOR := NO_INDICATION)
    return BOOLEAN;
```

Purpose:

This function returns TRUE if the specified screen is fully or partially occluded by another screen; otherwise, it returns FALSE. Whether or not the screen's border is occluded is not considered. 1

Parameters:

**SCREEN** is an open screen handle. 2

**STATUS** is an established status indicator. 3

Exceptions:

**STATUS_ERROR** is raised if SCREEN is not an open screen handle [11], or if STATUS is not an established status indicator [3]. 4

**SCREEN_NOT_ON_DISPLAY** is raised if SCREEN is not currently pasted [1]. 5

## 8.7.39 Moving the cursor position on a screen

```
procedure  SET_CURSOR_POSITION
    (SCREEN    : in SCREEN_TYPE;
     POSITION :  in SCREEN_POSITION;
     STATUS    : in STATUS_INDICATOR := NO_INDICATION);
```


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Purpose:

This procedure sets the cursor to the specified position in the screen. 1

Parameters:

**SCREEN** is an open screen handle identifying the screen whose cursor is to be moved. 2

**POSITION** indicates the screen position to which the cursor is to be moved. 3

**STATUS** is an established status indicator. 4

Exceptions:

**STATUS_ERROR** is raised if SCREEN is not an open screen handle [11] or if STATUS is not an established status indicator [3]. 5

**SCREEN_POSITION_ERROR** is raised if POSITION does not lie within the bounds of the screen [1]. 6

## 8.7.34 Determining the position of a cursor in a screen

```
function CURSOR_POSITION
    (SCREEN :  in SCREEN_TYPE;
     STATUS  : in STATUS_INDICATOR := NO_INDICATION)
    return  SCREEN_POSITION;
```

Purpose:

This function returns the cursor position for the specified screen. 1

Parameters:

**SCREEN** is an open screen handle identifying the screen whose cursor position is to be returned. 2

**STATUS** is an established status indicator. 3

Exceptions:

**STATUS_ERROR** is raised if SCREEN is not an open screen handle [11] or if STATUS is not an established status indicator [3]. 4

## 8.7.35 Writing a character to a screen

```
procedure  PUT
    (SCREEN  :  in  SCREEN_TYPE;
     ITEM     :  in  CHARACTER;
     WRAP     :   in  WRAP_TYPE            := OFF;
     STATUS  :  in  STATUS_INDICATOR  :=  NO_INDICATION);
```


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Purpose:

This procedure writes the character ITEM at the cursor position in the screen SCREEN. If the cursor is not in the maximum column, the cursor is advanced one column. If the cursor is in the maximum column and WRAP is OFF, the cursor remains in the same position. If WRAP is ON and the cursor is in the maximum column (but not in the maximum cell), the cursor is advanced to the first column of the next row. If WRAP is ON and the cursor is in the maximum cell, existing lines are scrolled up one line, a blank line is written at the bottom of the screen, and the cursor is set to the fist column of the maximum row. 1

Parameters:

**SCREEN** is an open screen handle identifying the screen to which the character will be written. 2

**ITEM** is the character to be written. 3

**WRAP** determines whether or not the cursor should wrap to the next line. 4

**STATUS** is an established status indicator. 5

Exceptions:

**STATUS_ERROR** is raised if SCREEN is not an open screen handle [11] or if STATUS is not an established status indicator [3]. 6

Additional Interfaces:

```
procedure PUT
    (SCREEN : in SCREEN_TYPE;
     ITEM   : in STRING;
     WRAP   : in WRAP_TYPE          := OFF;
     STATUS : in STATUS_INDICATOR   := NO_INDICATION)
is
begin
    for INDEX in ITEM'RANGE loop
        PUT(SCREEN, ITEM (INDEX), WRAP, STAUS);
    and loop;
and PUT;
```

## 8.7.36 Advancing the cursor to the next line

```
procedure NEW_LINE
    (SCREEn : in SCREEN_TYPE;
     STATUS : in STATUS_INDICATOR : = NO_INDICATION);
```

Purpose:

This procedure advances the cursor of SCREEN to the first column of the next row. If the cursor is in the maximum row, all lines are scrolled up in the screen causing the top line to be discarded, a blank line is written in the bottom row, and the cursor is placed in the first column of the maximum row. 1


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Parameters:

**SCREEN** is an open screen handle identifying the screen whose cursor is to be advanced. 2

**STATUS** is an established status indicator. 3

Exceptions:

**STATUS_ERROR** is raised if SCREEN is not an open screen handle [11] or if STATUS is not an established status indicator [3]. 4

## 8.7.37 Writing a line of text to a screen

```
procedure PUT_LINE
    (SCREEN : in SCREEN_TYPE;
     ITEM   : in STRING;
     WRAP   : in WRAP_TYPE           := OFF;
     STATUS : in STATUS_INDICATOR := NO_INDICATION);
```

Purpose:

This procedure writes the string ITEM at the cursor in the screen SCREEN and then advances the cursor to the minimum column of the next line. The effect is exactly that of calling PUT with the same set of parameters and then calling NEW-LINE with the same SCREEN and STATUS parameters. 1

Parameters:

**SCREEN** is an open screen handle identifying the screen to which the string will be written. 2

**ITEM** is the string to be written. 3

**WRAP** specifies whether or not overflow characters should wrap around to the next line. 4

**STATUS** is an established status indicator. 5

Exceptions:

**STATUS_ERROR** is raised if SCREEN is not an open screen handle [11] or if STATUS is not an established status indicator [3]. 6

## 8.7.38 Inserting a character into a line

```
procedure INSERT
    (SCREEN : in SCREEN_TYPE ;
     ITEM   : in CHARACTER;
     STATUS : in STATUS_INDICATOR := NO_INDICATION);
```


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Purpose:

This procedure inserts the character ITEM into the cell at the cursor in the screen SCREEN. Characters at and beyond the insertion position on the line containing the cursor are fist shifted one cell to the right. The character shifted off the end of the line is discarded. If the cursor is not in the maximum column, the cursor is advanced by one column; otherwise, the cursor remains in place, 1

Parameters:

**SCREEN** is an open screen handle identifying the screen into which the character will be inserted. 2

**ITEM** is the character to be inserted. 3

**STATUS** is an established status indicator. 4

Exceptions:

**STATUS_ERROR** is raised if SCREEN is not an open screen handle [11] or if STATUS is not an established status indicator [3]. 5

Additional Interfaces:

```
procedure INSERT
    (SCREEN : in SCREEN_TYPE;
     ITEM   : in STRING;
     STATUS : in STATUS- INDICATOR : = NO_INDICATION)
i s
begin
    for INDEX in ITEH'RANGE loop
        INSERT (SCREEN, ITEM (INDEX), STATUS);
    and loop;
and INSERT;
```

## 8.7.39 Deleting characters from a line

```
procedure DELETE_CHARACTERS
    (SCREEN : in SCREEN_TYPE;
     COUNT  : in CAIS_POSITIVE := 1;
     STATUS : in STATUS_INDICATOR := NO_INDICATION);
```

Purpose:

This procedure deletes COUNT characters starting at the cursor and advancing towards the maximum column. Only character cells on a single line are affected. Characters following the deleted characters on this line are shifted left to occupy the vacated positions. Blank characters are shifted in at the right hand side of the line. If COUNT is equal to or greater than the number of cells at and to the right of the cursor on the line, the effect is to blank the remaining portion of the line. The cursor position is not affected. 1


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Parameters:

**SCREEN** is an open screen handle identifying the screen from which the characters will be deleted. 2

**COUNT** is the number of characters to be deleted. 3

**STATUS** is an established status indicator. 4

Exceptions:

**STATUS_ERROR** is raised if SCREEN is not an open screen handle [11] or if STATUS is not an established status indicator [3]. 5

## 8.7.40 Inserting blank lines into a screen

```
procedure  INSERT_LINE
    (SCREEN :  in SCREEN_TYPE;
     COUNT   :  in  CAIS_POSITIVE  :  =  1;
     SCROLL :  in SCROLL-TYPE          := UP;
     STATUS :  in STATUS_INDICATOR : = NO_INDICATION);
```

Purpose:

This procedure inserts COUNT blank lines into SCREEN at the active row. Existing lines are scrolled in the direction specified by SCROLL to create space for the lines to be inserted. Lines scrolled off the screen are discarded. The cursor is set to the fist column of the active row. 1

Parameters:

**SCREEN** is an open screen handle identifying the screen to which a line will be inserted. 2

**COUNT** is the number of blank lines to be inserted. 3

**SCROLL** specifies whether the direction of scrolling should be up or down. 4

**STATUS** is an established status indicator. 5

Exceptions:

**STATUS_ERROR** is raised if SCREEN is not an open screen handle [11] or if STATUS is not an established status indicator [3]. 6

## 8.7.41 Deleting lines from a screen

```
procedure  DELETE_LINE
    (SCREEN :  in  SCREEN_TYPE;
     COUNT   :  in  CAIS_POSITIVE  :=  1;
     SCROLL :  in     SCROLL_TYPE     :=  DOW;
     STATUS :  in STATUS_INDICATOR :=  NO_INDICATION);
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Purpose:

This procedure deletes COUNT lines from SCREEN starting at the active row and advancing towards the maximum row. If the SCROLL direction is UP, the lines below the deleted lines are scrolled up into the empty space created by the deletion and blank lines appear at the bottom of the screen. If the SCROLL direction is DOWN, lines above the deleted lines are scrolled down and blank lines appear at the top of the screen. The cursor is set to the first column of the active row. 1

Parameters:

**SCREEN** is an open screen handle identifying the screen from which the line will be deleted. 2

**COUNT** is the number of lines to be deleted. 3

**SCROLL** specifies whether the direction of scrolling should be up or down. 4

**STATUS** is an established status indicator. 6

Exceptions:

**STATUS_ERROR** is raised if SCREEN is not an open screen handle [11] or if STATUS is not an established status indicator [3]. 7

## 8.7.42 Erasing a region of a screen

```
procedure ERASE_RANGE
    (SCREEN          : in SCREEN_TYPE;
     START_POSITION  : in SCREEN_POSITION;
     END_POSITION    : in SCREEN_POSITION;
     STATUS          : in STATUS_INDICATOR := NO_INDICATION);
```

Purpose:

This procedure erases characters in a screen by overwriting them with blank characters. The erase operation is performed within the specified range. START-POSITION and END-POSITION are the first and last positions blanked. The cursor position is set to START-POSITION. 1

Parameters:

**SCREEN** is an open screen handle identifying the screen to be erased.

**START-POSITION** is the start position of the region to be erased.

**END_POSITION** is the end position of the region to be erased.

**STATUS** is an established status indicator.



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Exceptions:

**STATUS_ERROR** is raised if SCREEN is not an open screen handle [11] or if STATUS is not an established status indicator [3].

**SCREEN_POSITION_ERROR** is raised if START_POSITION or END_POSITION exceeds the bounds of SCREEN [1].

## 8.7.43 Changing the rendition for a region of a screen

```
procedure CHANGE_RENDITION
    (SCREEN          : in SCREEN_TYPE;
     START_POSITION  : in SCREEN_POSITION;
     END_POSITION    : in SCREEN_POSITION;
     RENDITION       : in RENDITION_TYPE;
     STATUS          : in STATUS_INDICATOR := NO_INDICATION);
```

Purpose:

This procedure changes the rendition of a specified range of character cells within a screen. The active rendition of the screen is not changed.

Parameters:

**SCREEN** is an open screen handle.

**START_POSITION** is the position at which the rendition change is to start.

**END_POSITION** is the position at which the rendition change is to end.

**RENDITION** is the new rendition for the specified region.

**STATUS** is an established status indicator.

Exceptions:

**STATUS_ERROR** is raised if SCREEN is not an open screen handle [11] or if STATUS is not an established status indicator [3].

**SCREEN_POSITION_ERROR** is raised if START_POSITION or END_POSITION exceeds the bounds of SCREEN [1].

## 8.7.44 Determining a cell rendition

```
function CELL_RENDITION
    (SCREEN : in SCREEN_TYPE;
     STATUS : in STATUS-INDICATOR := NO_INDICATION)
    return RENDITION_TYPE;
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Purpose:

This function returns the rendition of the cell at the cursor of the screen SCREEN. 1

Parameters:

**SCREEN** is an open screen handle. 2

**STATUS** is an established status indicator. 3

Exceptions:

**STATUS_ERROR** is raised if SCREEN is not an open screen handle [11] or if STATUS is not an established status indicator [3]. 4

## 8.7.45 Reading characters from a screen

```
procedure READ_STRING
    (SCREEN : in     SCREEN_TYPE;
     ITEM   :    out STRING;
     LAST   :    out CAIS_NATURAL;
     STATUS : in     STATUS_INDICATOR := NO_INDICATION);
```

Purpose:

This procedure replaces successive characters of the string ITEM by successive characters contained in the cells starting at the cursor and advancing towards the maximum column. The replacement stops at the end of the line or when all characters in the string ITEM have been replaced. The returned value of LAST is such that ITEM (LAST) is the last character replaced. 1

The string returned will reflect all output operations to the screen up to the time of the call even if some of these output operations have been buffered. 2

Parameters:

**SCREEN** is an open screen handle. 3

**ITEM** is the character string into which the characters in the columns of the active row are placed. 4

**LAST** is the index value such that ITEM (LAST) is the last significant character in the string ITEM. 5

**STATUS** is an established status indicator. 6

Exceptions:

**STATUS_ERROR** is raised if SCREEN is not an open screen handle [11] or if STATUS is not an established status indicator [3]. 7



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

## 8.7.46 Sounding the terminal bell

```
procedure SOUND_BELL
    (SCREEN : in SCREEN_TYPE;
     STATUS : in STATUS_INDICATOR := NO_INDICATION);
```

Purpose:

If SCREEN is pasted to a virtual display, this procedure sounds the bell (or functional equivalent) at the terminal associated with the virtual display unless screen buffering is enabled. 1

Parameters:

**SCREEN** is an open screen handle. 2

**STATUS** is an established status indicator. 3

Exceptions:

**STATUS_ERROR** is raised if SCREEN is not an open screen handle [11] or if STATUS is not an established status indicator [3]. 4

## 8.7.47 Starting output buffering for a screen

```
procedure BEGIN_SCREEN_BUFFERING
    (SCREEN : in SCREEN_TYPE;
     STATUS : in STATUS_INDICATOR := NO_INDICATION) ;
```

Purpose:

This procedure causes subsequent operations to the specified screen to be buffered. The effects of a buffered operation are invisible (or inaudible in the case of SOUND_BELL) to the terminal user. This permits a sequence of changes to a screen to be hidden from the user until they are ready to be applied in a single update. 1

A counter maintained with each screen is incremented by a BEGIN-SCREEN-BUFFERING operation and decremented by an END_SCREEN_BUFFERING operation. When the counter's value is zero, updates to the screen can be made apparent to the user. 2

Parameters:

**SCREEN** is an open screen handle.

**STATUS** is an established status indicator.



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Exceptions:

**STATUS_ERROR** is raised if SCREEN is not an open screen handle [11] or if STATUS is not an established status indicator [3]. 5

## 8.7.48 Ending output buffering to a screen

```
procedure END_SCREEN_BUFFERING
    (SCREEN :  in SCREEN_TYPE;
     STATUS :  in STATUS_INDICATOR : = NO_INDICATION);
```

Purpose:

This procedure interacts with BEGIN_SCREEN_BUFFERING to control the user visibility of operations to a particular screen. A counter maintained with each screen is incremented by a BEGIN_SCREEN_BUFFERING operation and decremented by an END_SCREEN_BUFFERING operation. When the counter's value is zero, updates to the screen can be made visible on the virtual display. If the counter is already zero, this operation has no effect. 1

A terminal which has outstanding BEGIN_VIRTUAL_DISPLAY_BUFFERING operations will not be updated until the virtual display buffering is cleared for that terminal. 2

Parameters:

**SCREEN** is an open screen handle. 3

**STATUS** is an established status indicator. 4

Exceptions:

**STATUS_ERROR** is raised if SCREEN is not an open screen handle [11] or if STATUS is not an established status indicator [3]. 5

## 8.7.49 Enabling function key usage

```
procedure  ENABLE_FUNCTION_KEYS
    (TERMINAL : in  FILE_TYPE;
     ENABLE   :  in  BOOLEAN   := TRUE;
     STATUS   :  in  STATUS_INDICATOR := NO_INDICATION);
```

Purpose:

This procedure establishes whether function keys on the keyboard associated with TERMINAL are read as a sequence of characters or as a function key number. When the value of ENABLE is TRUE, function keys are read as function key numbers; otherwise, they are read as characters. 1



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Parameters:

**TERMINAL** is an open terminal file handle. 2

**ENABLE** indicates how function keys are to be captured. 3

**STATUS** is an established status indicator. 4

Exceptions:

**STATUS_ERROR** is raised if TERMINAL is not an open terminal file handle [8] or if STATUS is not an established status indicator [3]. 5

## 8.7.50 Determining function key usage

```
function FUNCTION_KEYS_ARE_ENABLE
    (TERMINAL  : in FILE_TYPE;
     STATUS    : in STATUS_INDICATOR := NO_INDICATION)
    return BOOLEAN;
```

Purpose:

This function returns TRUE if the function keys are enabled for the keyboard associated with TERMINAL; otherwise, it returns FALSE. 1

Parameters:

**TERMINAL** is an open terminal file handle.

**STATUS** is an established status indicator.

Exceptions:

**STATUS_ERROR** is raised if TERMINAL is not an open terminal file handle [8] or if STATUS is not an established status indicator [3]. 4

## 8.7.51 Determining the number of function keys

```
function NUMBER_OF_FUNCTION_KEYS
    (TERMINAL :  in FILE_TYPE;
     STATUS    : in STATUS_INDICATOR := NO_INDICATION)
    return CAIS_NATURAL;
```

Purpose:

This fuction returns the number of function keys defined for the terminal associated with the internal file identified by the file handle TERMINAL. 1



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Parameters:

**TERMINAL** is an open file handle identifying the internal file associated with the terminal file. 2

**STATUS** is an established status indicator. 3

Exceptions:

**STATUS_ERROR** is raised if TERMINAL is not an open terminal file handle [8] or if STATUS is not an established status indicator [3]. 4

## 8.7.52 Determining the identification of a function key

```
function FUNCTION_KEY_IDENTIFICATION
    (TERMINAL          : in FILE_TYPE;
     KEY_IDENTIFIER :  in CAIS_POSITIVE;
     STATUS            : in STATUS_INDICATOR : = NO_INDICATION)
    return FUNCTION_KEY_NAME;
```

Purpose:

This function returns the string identification of the function key designated by KEY_IDENTIFIER. 1

Parameters:

**TERMINAL** is an open file handle identifying the internal file associated with the terminal file. 2

**KEY_IDENTIFIER** is the identification number of a function key. 3

**STATUS** is an established status indicator. 4

Exceptions:

**STATUS_ERROR** is raised if TERMINAL is not an open file handle [8] or if STATUS is not an established status indicator [3]. 5

**FUNCTION_KEY_STATUS_ERROR** is raised if the value of KEY-IDENTIFIER is greater than NUMBER_OF_FUNCTION_KEYS (TERMINAL) [1]. 6

## 8.7.53 Enabling and disabling the locator

```
procedure ENABLE_LOCATOR
    (TERMINAL :  in FILE_TYPE;
     ENABLE    : in BOOLEAN              := TRUE;
     STATUS    : in STATUS_INDICATOR : = NO_INDICATION);
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Purpose:

This procedure enables and disables the locator associated with the virtual terminal identified by TERMINAL. If ENABLE is TRUE, the locator cursor is displayed and all locator events are captured on the input event queue. If ENABLE is FALSE, the locator cursor is not displayed and locator events are ignored. 1

Parameters:

**TERMINAL** is an open terminal file handle. 2

**ENABLE** denotes whether locator events are to be collected in the input event queue or whether they can be ignored. 3

**STATUS** is an established status indicator. 4

Exceptions:

**STATUS_ERROR** is raised if TERMINAL is not an open terminal file handle [8] or if STATUS is not an established status indicator [3]. 5

## 8.7.54 Determining locator usage

```
function LOCATOR_IS_ENABLED
    (TERMINAL :  in  FILE_TYPE;
     STATUS      : in STATUS_INDICATOR : = NO_INDICATION)
    return BOOLEAN;
```

Purpose:

This function returns TRUE if the locator associated with TERMINAL is enabled; otherwise, it returns FALSE. 1

Parameters:

**TERMINAL** is an open terminal file handle.

**STATUS** is an established status indicator.

Exceptions:

**STATUS_ERROR** is raised if TERMINAL is not an open terminal file handle [8] or if STATUS is not an established status indicator [3]. 4

## 8.7.55 Determining the locator cursor position

```
function LOCATOR. CURSOR_POSITION
    (TERMINAL : in       FILE_TYPE;
     STATUS    :  in     STATUS-INDICATOR  : = NO_INDICATION)
    return  VIRTUAL_DISPLAY_POSITION;
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Purpose:

This function returns the position of the locator cursor associated with the virtual terminal identified by the open terminal file handle TERMINAL. 1

Parameters:

**TERMINAL** is an open terminal file handle.

**STATUS** is an established status indicator.

Exceptions:

**STATUS_ERROR** is raised if TERMINAL is not an open terminal file handle [8], or if STATUS is not an established status indicator [3]. 4

**LOCATOR_STATUS_ERROR** is raised if the locator associated with the virtual terminal identified by TERMINAL is not enabled [1]. 5

## 8.7.56 Determining the screen containing the locator cursor

```
procedure GET_SCREEN_CONTAINING_LOCATOR_CURSOR
   (SCREEN   : in out SCREEN_TYPE;
    TERMINAL : in     FILE_TYPE;
    STATUS   : in     STATUS_INDICATOR := NO_INDICATION);
```

Purpose:

This procedure returns a handle to the screen which contains the locator cursor on the virtual display associated with TERMINAL. If the locator cursor is positioned over several screens, the screen forward-most in the pasting order is returned. For the purposes of this operation, a border is considered part of the screen it borders. 1

Parameters:

**SCREEN** is a screen handle, initially closed, to be opened to the screen which contains the current locator cursor. 2

**TERMINAL** is an open terminal file handle. 3

**STATUS** is an established status indicator. 4

Exceptions:

**STATUS_ERROR** is raised if SCREEN is an open screen handle [10], if TERMINAL is not an open terminal file handle [8], or if STATUS is not an established status indicator [3]. 5

**LOCATOR_STATUS_ERROR** is raised if the locator associated with the virtual terminal identified by TERMINAL is not enabled [1] or if no screen is located behind the locator cursor [2]. 6



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

## 8.7.57 Determining locator cursor position relative to screen border

```
function  LOCATOR_CURSOR_ON_BORDER
    (SCREEN    :  in      SCREEN_TYPE;
     STATUS    :  in      STATUS_INDICATOR  := NO_INDICATION);
    return BOOLEAN;
```

Purpose:

This function returns TRUE if the locator cursor for the virtual terminal to which the screen identified by SCREEN has been pasted is positioned over the border of the screen. Otherwise, the value FALSE is returned. 1

Parameters:

**SCREEN** is an open screen handle. 2

**STATUS** is an established status indicator. 3

Exceptions:

**STATUS_ERROR** is raised if SCREEN is an open screen handle [10], or if STATUS is not an established status indicator [3]. 4

**SCREEN_NOT_ON_DISPLAY** is raised if SCREEN is not currently pasted [1]. 5

**LOCATOR_STATUS_ERROR** is raised if the locator for the virtual terminal that SCREEN is associated with is not enabled [1] or if no screen is located behind the locator cursor [2] or if the locator cursor is not positioned over the screen identified by SCREEN [3]. 6

Notes:

If the screen does not have a border, FALSE is returned.

## 8.7.58 Moving the locator cursor

```
procedure  SET_LOCATOR_CURSOR_POSITION
    (TERMINAL :  in  FILE_TYPE;
     POSITION :  in  VIRTUAL_DISPLAY_POSITION;
     STATUS    : in STATUS-INDICATOR : = NO_INDICATION);
```

Purpose:

This procedure sets the locator cursor to a specified position on the virtual display associated with TERMINAL. 1


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Parameters:

**TERMINAL** is an open terminal file handle identifying the physical display on which the locator cursor is to be moved. 2

**POSITION** indicates the virtual display position to which the locator cursor is to be moved. 3

**STATUS** is an established status indicator. 4

Exceptions:

**STATUS_ERROR** is raised if TERMINAL is not an open terminal file handle [8] or if STATUS is not an established status indicator [3]. 5

**VIRTUAL_DISPLAY_POSITION_ERROR** is raised if POSITION does not lie within the portion of the virtual display upon which the physical display is mapped [3]. 6

**LOCATOR_STATUS_ERROR** is raised if the locator associated with the virtual terminal identified by TERMINAL is not enabled [1]. 7

## 8.7.59 Determining the number of locator buttons

```
function NUMBER_OF_BUTTONS
    (TERMINAL : in FILE_TYPE;
     STATUS   : in STATUS_INDICATOR := NO_INDICATION)
    return CAIS_NATURAL;
```

Purpose:

This function returns the number of buttons defined for the locator associated with TERMINAL. 1

Parameters:

**TERMINAL** is an open terminal file handle. 2

**STATUS** is an established status indicator. 3

Exceptions:

**STATUS_ERROR** is raised if TERMINAL is not an open terminal file handle [8] or if STATUS is not an established status indicator [3]. 4

## 8.7.60 Determining the identification of a locator button

```
function BUTTON-IDENTIFICATION
    (TERMINAL           : in FILE_TYPE;
     BUTTOH-IDENTIFIER  : in CAIS_POSITIVE;
     STATUS             : in STATUS_INDICATOR := NO_INDICATION)
    return BUTTON_NAME;
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Purpose:

This function returns the string identification of the button designated by BUTTON-IDENTIFIER for the locator associated with TERMINAL. 1

Parameters:

**TERMINAL** is an open terminal file handle. 2

**BUTTON_IDENTIFIER** is the identification number of a locator button. 3

**STATUS** is an established status indicator. 4

Exceptions:

**STATUS_ERROR** is raised if TERMINAL is not an open terminal file handle [8] or if STATUS is not an established status indicator [3]. 5

**BUTTON_NAME_ERROR** is raised if the value of BUTTON_IDENTIFIER is greater than NUMBER_OF_BUTTONS(TERMINAL) [1]. 6

## 8.7.61 Reading a single input event

```
procedure READ_INPUT_EVENT
    (TERMINAL          : in     FILE_TYPE;
     KIND              :    out INPUT_EVENT_KIND;
     NORMAL_KEY        :    out CHARACTER;
     FUNCTION_KEY      :    out CAIS_POSITIVE;
     LOCATOR_POSITION  :    out VIRTUAL_DISPLAY_POSITION;
     BUTTON_IDENTIFIER :    out CAIS_POSITIVE;
     BLOCKING          : in     BOOLEAN              := FALSE;
     TIME_LIMIT        : in     CAIS_IO_DURATION     := LONG_IO_DELAY;
     STATUS            : in     STATUS_INDICATOR     := NO_INDICATION);
```

Purpose:

This procedure retrieves a single input event from the virtual keyboard or virtual locator associated with TERMINAL. Upon return, only one of the parameters NORMAL_KEY, FUNCTION_ - KEY, LOCATOR_POSITION, and BUTTON_IDENTIFICATION is significant; the significant value is determined by the returned KIND parameter. 1

If BLOCKING is TRUE and the input event queue is empty, the procedure blocks until an event becomes available or until the specified time limit is exceeded. If BLOCKING is FALSE and the input event queue is empty, the INPUT_EVENT_QUEUE_EMPTY exception is raised instead. 2

Parameters:

**TERMINAL** is an open terminal file handle.



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

**KIND** is the kind of the event retrieved.

**NORMAL_KEY** is the character returned if KIND is NORMAL_KEY_STROKE.

**FUNCTION_KEY** is the function key identifier returned if KIND is FUNCTION. KEY_-STROKE.

**LOCATOR_POSITION** is the returned position of the locator if KIND is LOCATOR_-MOVEMENT.

**BUTTON_IDENTIFIER** is an identification of the button involved if KIND is BUTTON_UP or BUTTON_DOWN.

**BLOCKING** indicates whether blocking is to take place if the input event queue is empty.

**TIME_LIMIT** is the time limit for waiting for an input event to occur if the input event queue is empty.

**STATUS** is an established status indicator.

Exceptions:

**STATUS_ERROR** is raised if TERMINAL is not an open terminal file handle [8] or if STATUS is not an established status indicator [3].

**INPUT_EVENT_QUEUE_EMPTY** is raised if the BLOCKING parameter is FALSE and the input event queue associated with TERMINAL is empty [1].

**END_ERROR** is raised if the input event queue associated with TERMINAL has been exhausted and the opposite client is a FILE node from which retrieval of another input event would constitute a read past the end of file [1], or if the opposite client process has issued a CLOSE and the input event queue has been exhausted [2].

**TIME_EXPIRED** is raised if TIME_LIMIT expires [1].

Notes:

When function keys are not enabled, a function key can only be captured as a sequence of characters; FUNCTION_KEY will never be significant.

### 8.7.62 Reading a string from a virtual keyboard

```
procedure READ_STRING
    (TERMINAL                : in     FILE_TYPE;
     ITEM                    :    out STRING;
     LAST                    :    out CAIS_NATURAL;
     TERMINATOR_KIND         :    out INPUT_EVENT_KIND;
     RECEIVED_TERMINATOR     :    out CHARACTER;
     VALID_TERMINATORS       : in     TERMINATOR_SET;
     SCREEN                  : in     SCREEN_TYPE;
     ECHO                    : in     ECHO_TYPE          := ON;
     WRAP                    : in     WRAP_TYPE          := OFF;
     STATUS                  : in     STATUS_INDICATOR   := NO_INDICATION);
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Purpose:

This procedure dequeues successive NORMAL_KEY_STROKE events at the front of the input queue associated with TERMINAL and replaces successive characters of ITEM by the characters associated with the NORMAL_KEY_STROKE events. The replacement stops when a terminator is at the front of the queue or when ITEM has been completely replaced. Should the input queue become empty prior to the stopping of the replacement of characters within ITEM, the calling task is blocked and waits for the next input event. The returned value of LAST is such that ITEM (LAST) is the last character replaced, unless the first character is a terminator in which case LAST is set to the value ITEM'FIRST-1. 1

A terminator is either a non-NORMAL_KEY_STROKE event or a NORMAL_KEY_STROKE event corresponding to a character in the set VALID_TERMINATORS. The kind of terminator is returned in TERMINATOR_KIND. If the terminator is a NORMAL_KEY_STROKE then the character terminating the sequence is not present in the returned string but is rather returned in RECEIVED. TERMINATOR; the terminator is dequeued. If the terminator is not a NORMAL_KEY_STROKE, the returned value of RECEIVED_TERMINATOR is insignificant; the terminator must be dequeued explicitly by a READ_INPUT_EVENT operation. If the entire string ITEM is replaced, then, on return, LAST equals ITEM'LAST and the values of TERMINATOR_KIND and RECEIVED_TERMINATOR are insignificant. 2

SCREEN becomes the front screen. If the value of ECHO is ON, each character returned in ITEM is also written at the cursor position. If WRAP is ON, the echoed response may cause the screen to wrap. If wrapping occurs at the maximum cell, the scroll direction is always up. If WRAP is OFF, extra characters overwrite one another in the maximum column. 3

Parameters:

**TERMINAL** is an open terminal file handle. 4

**ITEM** is the string returned. 6

**LAST** is the position of the last significant character in ITEM. 6

**TERMINATOR_KIND** is the kind of the terminating event. 7

**RECEIVED_TERMINATOR** is the returned termination character. It is only significant if TERMINATOR_KIND is NORMAL_KEY_STROKE. 8

**VALID_TERMINATORS** is the set of valid termination characters. 9

**SCREEN** is the virtual screen to which input is to be echoed. 10

**ECHO** specifies whether echoing of the input characters is to be performed. 11

**WRAP** indicates whether wrapping is allowed to occur when echoing the input characters. 12

**STATUS** is an established status indicator. 13

Exceptions:

**STATUS_ERROR** is raised if TERMINAL is not an open terminal file handle [8], if SCREEN is not an open screen handle [11], or if STATUS is not an established status indicator [3]. 14

SCREEN_NOT_ON_DISPLAY is raised if SCREEN is not currently pasted [1]. 15

**END_ERROR** is raised if the input event queue associated with TERMINAL is exhausted and additional NORMAL_ KEY_STROKE events required to complete the operation cannot be obtained due to reaching the end of file when the opposite client is a FILE node [1] or due the opposite client process having closed the protocol [2]. 16



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

## 8.7.03 Flushing the input event queue

```
procedure  FLUSH_INPUT_EVENT_QUEUE
    (TERMINAL :  in  FILE_TYPE;
     STATUS      : in STATUS_INDICATOR := NO_INDICATION);
```

Purpose:

This procedure flushes the input event queue associated with the specified terminal. 1

Parameters:

**TERMINAL** is an open terminal file handle. 2

**STATUS** is an established status indcator. 3

Exceptions:

**STATUS_ERROR** is raised if TERMINAL is not open terminal file handle [8] or if STATUS is not an established status indicator [3]. 4

## 8.7.64 Determining if all input events have been read

```
function  END_OF_FILE
    (TERMINAL : in       FILE_TYPE;
     STATUS    :  in     STATUS_INDICATOR  :=  NO_INDICATION)
      return BOOLEAN;
```

Purpose:

This function returns TRUE if no more input events can be read from the input event queue associated with TERMINAL; otherwise it returns FALSE. 1

No additional frames can be read from the input event queue if the queue is empty and the opposite client is a FILE node and an attempt to read another event would constitute a read past the end of the file, or if the queue is empty and the opposite client process has closed the protocol. 2

Parameters:

**TERMINAL** is an open terminal file handle.

**STATUS** is an established status indicator.

Except ions:

**STATUS_ERROR** is raised if TERMINAL is not an open terminal file handle [8] or if STATUS is not an established status indicator [3]. 5



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

## 8.7.65 Determining if the input event queue is empty

```
function INPUT_QUEUE_EMPTY
    (TERMINAL : in     FILE_TYPE;
     STATUS   : in     STATUS_INDICATOR := NO_INDICATION)
      return BOOLEAN;
```

Purpose:

This function returns TRUE if at the time of the call the input event queue associated with 1
TERMINAL is empty; otherwise it returns FALSE.

Parameters:

**TERMINAL** is an open terminal file handle.

**STATUS** is an established status indicator.

Exceptions:

**STATUS_ERROR** is raised if TERMINAL is not an open terminal file handle [8] or if STATUS 4
is not an established status indicator [3].



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

# 9. CAIS LIST MANAGEMENT DETAILED REQUIREMENTS

This section describes the CAIS interfaces for the manipulation of lists. Lists are used as the values of node and relationship attributes, to return status information via status indicators and to define iteration predicates. The interfaces described in package CAIS_LIST_MANAGEMENT are intended for use by other CAIS interfaces. The exceptions raised by all subprograms in this package are defined in the packages CAIS_LIST_MANAGEMENT and CAIS_PRAGMATICS. 1

A *linear list* is a linearly ordered set of data elements called *list items.* Each list item has an *item value.* A list item may also have an *item name,* in which case it is called a *named item;* if a list item has no item name, it is called an *unnamed item.* 2

There are three kinds of linear lists: empty lists, named lists, and unnamed lists. An *empty list* is a linear list that contains no items. Such a list is not considered to be either named or unnamed. A *named list* is a non-empty linear list that contains only named items; the names of distinct items in a named list must be distinct. An *unnamed list* is a non-empty linear list that contains only unnamed items. The values of type LIST_KIND enumerate these three kinds of lists. 3

There are five basic kinds of list items, discriminated on the kind of values they can have: strings, integers, floating point numbers, identifiers and linear lists. The values of type ITEM_KIND enumerate these five kinds of list items. The actual, internal types of list item values are not specified, although certain properties of these types are specified in the package CAIS_PRAGMATICS (Section 12 on page 815). However, the means to manipulate these internal values are provided by explicit, external types (or generic type parameters) used by the subprograms in CAIS_LIST_MANAGEMENT. These external types have been chosen so as to maximize the likelihood that they will be related very efficiently to the internal types used by a given implementation of CAIS_LIST_MANAGEMENT; in some cases, the external types can even be the same as the internal types. 4

Although a CAIS implementation may choose a representation for internal values different from that of the external values at the interfaces, it must preserve certain properties. The process of inserting values into a list (for example by inserting list items or by replacing the values of existing list items) and then retrieving the inserted values must either preserve the values (or, in the case of floating point values, nearly preserve the values) or result in CAPACITY_ERROR upon insertion and CONSTRAINT_-ERROR upon retrieval, in case the constraints of the respective internal or external types are violated. 6

Because of the conversions between external, instantiated types and internal types, the storage and retrieval cycle for floating point numbers may not preserve the values precisely; the degree of error is completely contained within that provided by doing standard Ada type conversions. 6

Some interfaces, such as those which determine an item's position by value, require a value provided at the interface to be compared with an internal value in a list; these operations behave as if the value provided at the interface is converted to internal representation prior to comparison. 7

The principles of external values, internal values and comparison of values are realized in distinct ways for values of each of the five kinds of list items: 8

a. String values of list items are represented by the external type STRING at the interfaces. String equality is defined according to the predefine equality for the Ada type STRING. 9

b. Integer values of list items are represented at the interfaces by external integer types defined 10



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

in the programs using the interfaces; operations to manipulate these values are placed in generic packages that can be instantiated for user-defied types.

Internal integer values include exactly the values of the CAIS type CAIS_INTEGER (see Section 10 on page 805); integer equality is defined according to the predefine equality for the CAIS type CAIS_INTEGER. 11

c. Floating point values of list items are represented at the interfaces by external floating point types defined in the programs using the interfaces; operations to manipulate these values are placed in generic packages that can be instantiated for user-defined types. 12

Floating point internal values preserve at least CAIS_PRAGMATICS.LIST_MAXIMUM_-DIGITS accuracy. Floating point equality of two values is defined according to predefine equality for the two values when converted to a floating point type of CAIS_PRAGMA-TICS.LIST_MAXIMUM_DIGITS accuracy. 13

Upon insertion and subsequent extraction of floating point values, it is implementation-dependent whether or not changes to the physical representation of the value are made. If such changes are made, they must be consistent with the accuracy of the types used in the respective instantiation of the package CAIS_FLOAT_ITEM, but may affect the meaning of equality between the retrieved value and the inserted value. 14

Users should be aware of the accuracy issues for relational operations between floating point values explained in [1815A], Section 4.5.7, in order to avoid erroneous assumptions about the equality of float items and lists involving such items. 15

d. Identifier values of list items occur in three different forms which are represented by three different external types at the interfaces: a *token form* represented by a limited private type (TOKEN_TYPE), an *identifier text form* represented by the type IDENTIFIER_TEXT (a subtype of STRING restricted to the syntax of Ada identifiers) and an *enumeration form* represented by an Ada enumeration type. 16

The interfaces for manipulating identifier items using the token form and the identifier text form are defined in package CAIS_IDENTIFIER_ITEM (see Section 9.1.22 on page 771). The interfaces for manipulating the identifier item values in an enumeration form are defined by the generic package CAIS_ENUMERATION_ITEM (see Section 9.1.23 on page 780) which must be instantiated with the enumeration type. 17

Identifier items whose values are created or set using the enumeration form are called enumerated identifier items or enumeration items. Enumerated identifier items are also created to represent an attribute value or a component value of an attribute value which is defined by an enumerated identifier AKD (see Section 5.7.1.6.5 page 320). Identifier items whose values are created or set using the token form or identifier text form are called ordinary identifier items. 18

Enumerated identifier item values have an internal form that is slightly different from the internal form of ordinary identifier item values. The internal form of an enumerated identifier item value represents both the image (the identifier text form) of the enumeration value and the enumeration position of that value in the enumeration base type (the position returned by the Ada attribute operator 'POS). The internal form for an ordinary identifier item represents only the identifier text form. An enumerated identifier item may be interpreted as an ordinary identifier item and compared for equality with an ordinary identifier item. An identifier item cannot assume an enumeration value which is represented by an Ada character literal. 19



Source: https://assist.dla.mil -- Downloaded: 202640-26T07:54Z
Check the source to verify that this is the currentversion before use.

Identifier item value scan be manipulated directly using either the token form or the enumeration form; in addition, there are interfaces to transform the external representation of identifier values from the identifier text form to the token form and vice versa. Enumeration values may be converted to the identifier text form using the TEXT-FORM interface or the Ada attribute operator 'IMAGE; the identifier text form can be converted to an enumeration value using the Ada attribute operator 'VALUE. 20

Values of type TOKEN-TYPE are called *tokens*. Tokens provide a representation for identifier item values which can be manipulated more efficiently in CAIS implementations than strings. Essentially, a token may be implemented as a reference or pointer to an object containing a string rather than the string itself. Tokens associated with a given identifier may then be references to the same object. In this case, comparison of tokens may require only a comparison of the references rather than a character by character comparison of the associated strings. Depending on the Ada language implementation, use of tokens instead of strings may also substantially reduce string copying. 21

When a token is no longer needed, the DELETE-TOKEN interface may be called. This permits the implementation to release the storage for an identifier string when all tokens for the identifier have been deleted. When a token is passed to any interface as an IN OUT parameter, the token is automatically deleted and associated with a new identifier, unless an exception is raised. 12

The value of every token variable is initially a distinguished *undefined token;* the variable is given a valid token value by interfaces that copy tokens, transform identifier text forms into tokens, or otherwise produce tokens. There is no text form for an undefined token. No interface can produce the undefined token; no interface can use an undefined token as a legitimate input value. The undefined token can only be used as the value of an IN OUT parameter in an interface; in such a case, the purpose of the interface is to update the parameter value but not use it as input. All interfaces guarantee that attempts to use an undefined token in any other way result in raising the TOKEN-ERROR exception. 23

If an identifier text form is transformed to token form and then transformed back, the resulting identifier text will be the original identifier text (with all letters in their original case). When a value is converted from an enumeration form to the identifier text form, the result is always in upper case. 24

Identifier item values are equal if their identifier text forms are equal under identifier equality and their enumeration positions are equal (when both items are enumerated). Identifier equality follows the Ada rule when applied to the identifier text form: two identifier text forms which differ only in the case of alphabetic characters designate the same identifier and therefore satisfy identifier equality. A token obtained from an identifier text form designates the same identifier value as the identifier text form and is therefore identifier-equal to the identifier text form. Note that an ordinary identifier item value is equal to an enumerated identifier item value if the identifier texts forms are equal under identifier equality. 26

Identifiers are not only used as item values; they also serve as item names. Interfaces that refer to item names are provided with both the TOKEN-TYPE form and the identifier text form. The rule of identifier equality applies to item names; therefore, no two distinct items in a linear list may have names that are identifier-equal. 26

e. The value of a list item may be itself a linear list. Whether or not such a list item is named, its linear list value may be named, unnamed or empty. The linear list value is called a *nested sublist* of the linear list containing the list item. Any linear list containing a list item can 27



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

be viewed as a nested list structure consisting of the linear list with all of its nested sublists (and all of their nested sublists, recursively including all the nested sublists).

The only way to refer to linear list values at the interfaces is through the use of the limited private type LIST_TYPE. Every LIST_TYPE value includes a single, outermost linear list, which may constitute a nested list structure. By convention, the list kind of a LIST_TYPE value's outermost linear list is said to be the list kind of the LIST-TYPE value itself. 28

Many list manipulations provided by the CAIS_LIST_MANAGEMENT interfaces require designating a list item by its position within its linear list. To provide the ability to apply such list manipulations at all levels of nesting of a nested list structure, every LIST_TYPE value includes not only a list structure but also a designation of the *current linear list* within the list structure, to which the list manipulations implicitly refer. Exactly one designation of a current linear list is associated with each LIST_TYPE value; a nested sublist within that value does not have its own designated current Linear list. 29

The initial value of every Ada object of type LIST_TYPE represents the empty list; that is, the empty list is the Ada object's outermost linear list and is also designated as its current linear list. 30

CAIS_LIST_MANAGEMENT interfaces provide several operations upon the current linear list of LIST_TYPE values. These include such linear list manipulations as: 31

1. extracting values of items in a linear list, 32
2. extracting contiguous sequences of items from a linear list, 33
3. replacing or changing values of items in a linear list, and 34
4. inserting new items into a linear list. 35

They also include operations which change the designation of the current linear list of a LIST_TYPE value, such as: 36

1. making a nested sublist of the current linear list the (new) current linear list, and 37
z. making the linear list containing the current linear list the (new) current linear list. 38

These two kinds of operations (linear list manipulations and current linear list designations) are independent in the following sense: linear list manipulations do not change the designation of the current linear list of a LIST_TYPE value and current linear list designations applied to a LIST_TYPE value do not change the nested list structure being represented. 39

For convenience, a STRING-valued external representation (restricted to strings with the syntax given in Table XXXVII) is defined for linear lists. 40



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

**TABLE XXXVII: List External Representation BNF**

```
list            ::=  named_list
                     | unnamed_list
                     | empty_list
named_list      ::=  (named_item {, named_item } )
unnamed_list    ::=  (item_value {, item_value } )
empty_list      ::=  ()
named_item      ::=  item_name => item_value
item_value      ::=  list
                     | quoted_string
                     | integer_number
                     | float_number
                     | identifier
integer_number  ::=  [ - ] integer_literal
float_number    ::=  [ - ] decimal_literal
quoted_string   ::=  string_literal
```

**Notes:**

**1. See Appendix D for a description of the notation used.**

It is legal for blanks, format effecters and/or non-printing characters to occur between the syntactic constituents of LIST_TEXT representations of list values. 41

There are interfaces to transform the external representation of a linear list from list form to text form and vice versa. If a list representation is transformed to list text form the result is a canonical list text representation. The *canonical list text representation* of a list consists of the list text of its list items, composed according to the syntax of Table XXXVII, and in addition adheres to the following rules: 42

1. There are no blanks, format effecters or non-printing characters between the syntactic constituents. 43
2. For an integer value of a list item, the list text representation is the decimal representation of its numeric value without leading zeroes. Negative values have a leading minus sign; positive values are unsigned. 44
3. For a floating point value of a list item, the list text representation is the string image of its numeric value in decimal notation with a format as obtained under default settings of the FORE, AFT, and EXP parameters in PUT operations of Ada TEXT_-10 (see [1815A] 14.3.8), except that the value of 'DIGITS to be assumed for AFT is LIST_MAXIMUM_DIGITS (see CAIS_PRAGMATICS, Section 12 on page 815) and the FORE field does not contain any leading spaces. 46
4. For a string value of a list item, the list text representation is the string literal representing the string value (i.e., the string value enclosed by quotation characters and with inner quotation characters doubled). The replacement character "70" may be used uniformly instead of the quotation character as described in [1815A] 2.10. 46


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

5. For an identifier value of a list item or the name of a list item, the list text representation is the identifier string (with characters in their original case, without enclosing quotation characters). 47

6. For a linear list value of a list item, the list text representation is (recursively) the list text of the list value. 48

If a list text representation of a linear list is transformed to list form, the result is a LIST_TYPE value whose current (and outermost ) linear list is the linear list described in the list text representation. 49

List equality is defined according to the following rule: 50

Two linear lists are equal if and only if: 51

1. both lists are of the same kind (i.e., named, unnamed or empty), and 52
2. both lists contain the same number of list items, and 53
3. in the case of named lists, for each position in the list, the names of the list items at this position are equal under identifier-equality, and 54
4. for each position in the list, the values of the list items at this position are of the same kind and are equal according to the appropriate form of equality (as described above), i.e.: 55
   (a) for identifier items, identifier equality of the identifier text forms and, in the case of two enumerated identifier items, equality of enumeration positions as well; 56
   (b) for string items, string equality; 57
   (c) for integer items, integer equality; 58
   (d) for floating point items, floating point equality; 59
   (e) for list items (whose values are in turn lists), list equality as defined in this rule. 60

Equality of lists involving floating point items should be applied with considerable caution and awareness of all the issues documented in [1815A], Section 4.5.7, regarding the accuracy of relational operations with real operands and the discussion above defining floating point equality for floating point list items. 61


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

# 9.1 Package CAIS_LIST_MANAGEMENT

This package defines types, subtypes, constants, exceptions and general list manipulation interfaces. The latter are supplemented by generic subpackages for the manipulation of list items of numeric type. 1

## 9.1.1 Types, subtypes, constants and exceptions

```
type    LIST_TYPE        is limited private;

subtype LIST_TEXT        is STRING ;

type    LIST_SIZE        is range 0 . . CAIS_PRAGMATICS.LIST_LENGTH;
subtype POSITION_COUNT   is LIST_SIZE range 1 . . LIST_SIZE'LAST;
subtype INSERT_COUNT     is LIST_SIZE range 0 . . LIST_SIZE'LAST - 1;

type    LIST_KIND        is (UNNAMED, NAMED, EMPTY);

type    ITEM_KIND        is (LIST_ITEM_KIND,
                             STRING_ITEM_KIND,
                             INTEGER_ITEM_KIND,
                             FLOAT_ITEM_KIND,
                             IDENTIFIER_ITEM_KIND);
```

LIST_TYPE describes the values used for lists at the list manipulation interfaces. LIST_TEXT describes the values used for the text representation of lists. The interfaces enforce the syntax of Table XXXVII on page 749, for such text values. 1

LIST_SIZE describes the values that can be used to indicate the number of items in a linear list. POSITION_COUNT describes the values that can be used to indicate the position of an item in a non-empty linear list. INSERT_COUNT describes the values that can be used to indicate the position in a linear list after which items are to be inserted. LIST_KIND enumerates the kinds of lists. 2

ITEM_KIND enumerates the kinds of list item. The ITEM_KIND value for an enumerated identifier item is IDENTIFIER_ITEM_KIND. An enumerated identifier item can be distinguished from an ordinary identifier item using the IS_ENUMERATED function in package CAIS_IDENTIFIER_ITEM (see Section 9.1.22.10 on page 778). 3

```
typo    TOKEN_TYPE       is limited private;
subtype IDENTIFIER_TEXT  is STRING;
```

TOKEN_TYPE describes the token values used at the interfaces to designate identifiers. IDENTIFIER_TEXT describes the text values used at the interfaces to designate identifiers. The interfaces enforce the syntax of Ada identifiers for such text values. Blanks, format effecters and/or non-printing characters are permitted surrounding the identifier in input parameters of type IDENTIFIER-TEXT. 4

```
EMPTY_LIST : constant LIST_TYPE;
```

EMPTY_LIST is a deferred constant denoting the value of an empty list. The value of the function IS_EQUAL(EMPTY_LIST,X) is TRUE for any object X of type LIST_TYPE whose value is an empty list. 5



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
ITEM_KIND_ERROR       : exception;
LIST_KIND_ERROR       :   excection;
LIST_POSITION_ERROR   :   excection;
NAMED_LIST_ERROR      :   excection;
SEARCH_ERROR          :   excection;
SYNTAX_ERROR          :   excection;
TOKEN_ERROR           :   excection;
```

**ITEM_KIND_ERROR** is raised if the kind of item is incorrect for the operation being attempted. 6

**LIST_KIND_ERROR** is raised if the kind of list is incorrect for the operation being attempted. 7

**LIST_POSITION_ERROR** is raised if an attempt is made to specify a list item's position larger than the list's length, a start position larger than the list's length or an end position less than the specified start position. 8

**NAMED_LIST_ERROR** is raised if an attempt is made to specify an item by name in an unnamed list or to construct a linear list with more than one item with the same name. 9

**SEARCH_ERROR** is raised if a search for an item fails because the item is not present in a non-empty list. 10

**SYNTAX_ERROR** is raised if an attempt is made to use IDENTIFIER_TEXT values that do not satisfy the syntax of an Ada identifier or LIST_TEXT values that do not satisfy the syntax defined in Table XXXVII on page 749. 11

**TOKEN_ERROR** is raised if an undefined token value is used where a valid token is required. 12

Other exceptions raised by the subprograms in this package are defined in the packages CAIS_-DEFINITIONS and CAIS_PRAGMATICS. 13

## 9.1.2 Copying a list

```
procedure   COPY_LIST
    (FROM_LIST : in      LIST_TYPE;
     TO_LIST   : in out  LIST_TYPE;
     STATUS    : in      STATUS_INDICATOR : = NO_INDICATION);
```

Purpose:

This procedure returns in the parameter TO_LIST a copy of the current linear list value of the parameter FROM_LIST. In the newly copied TO_LIST, the outermost list is the current linear list. Subsequent modifications of either list do not affect the other list. 1

Parameters:

**FROM_LIST** is the list whose current linear list is to be copied. 2

**TO_LIST** is the list returned as a copy of the current linear list of FROM_LIST. 3

**STATUS** is an established status indicator. 4



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Exceptions:

STATUS_ERROR is raised if STATUS is not an established status indicator [3]. 5

## 9.1.3 Making a list empty

```
procedure SET_TO_EMPTY_LIST
    (LIST   : in out LIST_TYPE;
     STATUS : in     STATUS_INDICATOR := NO_INDICATION);
```

Purpose:

This procedure resets the parameter LIST so that its outermost linear list (which becomes the current linear list) is the empty list. 1

Parameters:

**LIST** is the list to be made empty.

**STATUS** is an established status indicator.

Exceptions:

**STATUS_ERROR** is raised if STATUS is not an established status indicator [3].

## 9.1.4 Converting from text to list form

```
procedure CONVERT_TEXT_TO_LIST
    (LIST_SRTING : in     LIST_TEXT;
     LIST        : in out LIST_TYPE;
     STATUS      : in     STATUS_INDICATOR := NO_INDICATION);
```

Purpose:

This procedure converts the text representation of a list into the private list representation. It establishes the current (and outermost) linear list of LIST to be a named, unnamed, or empty list. The individual list items are classified according to their text representation. For a numeric item value, the item is classified as an integer item if the numeric value can be interpreted as a (possibly negated) literal of universal-integer type; otherwise, the numeric item is classified as a floating point item. Blanks, format effectors and non-printing characters are allowed between syntactic elements in the value of the parameter LIST_STRING. 1

Parameters:

**LIST_STRING** is the text form to be interpreted as a linear list value. 2


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

**LIST** is the list whose current and outermost linear list is built and returned according to the contents of LIST_ STRING. 3

**STATUS** is an established status indicator. 4

Exceptions:

**STATUS_ERROR** is raised if STATUS is not an established status indicator [3]. 5

**SYNTAX_ERROR** is raised if the value of the parameter LIST_STRING does not conform to the syntax of Table XXXVII on page 749 [1]. 6

**CAPACITY_ERROR** is raised if the length of the LIST_STRING parameter exceeds the value of the constant CAIS_PRAGMATICS.LIST_TEXT_LENGTH or if its value contains any name or value which cannot be represented in the LIST result due to exceeding implementation limits for the particular CAIS implementation. [1] (See CAIS_PRAGMATICS, Section 12 on page 815.) 7

**NAMED_LIST_ERROR** is raised if the LIST_STRING parameter designates a named linear (sub)list with two or more items of the same name [1]. 8

## 9.1.5 Converting a list to its text representation

```
function TEXT_FORM
   (LIST   : in     LIST_TYPE;
    STATUS : in     STATUS_INDICATOR := NO_INDICATION)
  return LIST_TEXT;
```

Purpose:

This function returns the text representation of the value of the current linear list of LIST. The result is in the canonical list text representation defined in Section 9 on page 745. 1

Parameters:

**LIST** is the list whose current linear list is to be converted to its text representation.

**STATUS** is an established status indicator.

Exceptions:

**STATUS_ERROR** is raised if STATUS is not an established status indicator [3]. 4

**CAPACITY_ERROR** is raised if the length of the canonical list text representation to be returned exceeds the value of the constant CAIS_PRAGMATICS.LIST_TEXT_LENGTH. [1] (See CAIS_PRAGMATICS, Section 12 on page 815.) 5

Notes:

The conversion of an enumerated identifier item to text form results in the identifier text form which does not capture the enumeration position. Hence, if a list is converted to text form using TEXT_FORM and then back to list form using CONVERT_TEXT_TO_LIST any enumerated identifier items in the original list will become ordinary identifier items in the resulting list. 6


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

## 9.1.6 Determining the equality of two lists

```
function IS_EQUAL
    (LIST1  : in     LIST_TYPE;
     LIST2  : in     LIST_TYPE;
     STATUS : in     STATUS_INDICATOR := NO_INDICATION)
    return BOOLEAN;
```

Purpose:

This function returns TRUE if the values of the two current linear listsof LIST1 and LIST2 are equal according to list equality (see Section 9 on page 745; otherwise, it returns FALSE.

Parameters:

**LIST1, LIST2** are the lists containing the current linear lists whose equality is to be determined.

**STATUS** is an established status indicator.

Exceptions:

**STATUS_ERROR** is raised if STATUS is not an established status indicator [3].

Notes:

Equality of lists involving floating point items should be applied with considerable caution and awareness of all the issues documented in [1815A], Section 4.5.7, regarding the accuracy of relational operations with real operands, and the discussion in Section 9 on page 745, defining floating point equality for floating point list items.

## 9.1.7 Deleting an item from a linear list

```
procedure  DELETE
    (LIST          : in out  LIST_TYPE;
     ITEM_POSITION : in      POSITION_COUNT;
     STATUS        : in      STATUS_INDICATOR := NO_INDICATION);

procodure  DELETE
    (LIST      : in out  LIST_TYPE;
     lTEM_NAME : in      IDENTIFIER_TEXT;
     STATUS    : in      STATUS_INDICATOR : = NO_INDICATION);

procedure  DELETE
    (LIST      : in out  LIST_TYPE;
     ITEM_NAME : in      TOKEN_TYPE;
     STATUS    : in      STATUS_INDICATOR := NO_INDICATION);
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Purpose:

This procedure deletes the item specified by ITEM_POSITION or ITEM_NAME from the current linear list of LIST. If this was the only item in the linear list, the kind of the linear list changes to EMPTY. 1

Parameters:

**LIST** is the list containing the current linear list from which the item will be deleted. 2

**ITEM_POSITION** is the position within the current linear list of the item to be deleted. 3

**ITEM_NAME** is the name of the list item to be deleted. 4

**STATUS** is an established status indicator. 5

Exceptions:

**STATUS_ERROR** is raised if STATUS is not an established status indicator [3]. 6

**LIST_KIND_ERROR** is raised if the current linear list of LIST is empty [1]. 7

**LIST_POSITION_ERROR** is raised if ITEM_POSITION has a value larger than the current length of the current linear list of LIST [1]. 8

**SYNTAX_ERROR** is raised if the identifier-text value of the parameter ITEM_NAME of type IDENTIFIER_TEXT does not conform to the syntax of an Ada identifier [2]. 9

**TOKEN_ERROR** is raised if the ITEM_NAME of type TOKEN_TYPE is an undefined token [1]. 10

**NAMED_LIST_ERROR** is raised if the parameter ITEM_NAME is used and the current linear list of LIST is unnamed [2]. 11

**SEARCH_ERROR** is raised if there is no item in the current linear list of LIST with the name ITEM_NAME [1]. 12

## 9.1.8 Determining the kind of list

```
function   KIND_OF_LIST
    (LIST     : in      LIST_TYPE ;
     STATUS : in      STATUS_INDICATOR  : = NO_INDICATION)
    return  LIST_KIND;
```

Purpose:

This function returns the kind of the current linear list of LIST; the value returned is either UNNAMED, NAMED or EMPTY. 1

Parameters:

**LIST** is the list whose current linear list is of interest. 2

**STATUS** is an established status indicator. 3



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Exceptions:

**STATUS_ERROR** is raised if STATUS is not an established status indicator [3].

## 9.1.9 Determining the kind of list item

```
function KIND_OF_ITEM
    (LIST          : in  LIST_TYPE;
     ITEM_POSITION : in  POSITION_COUNT;
     STATUS        : in  STATUS_INDICATOR := NO_INDICATION)
    return ITEM_KIND;

function KIND_OF_ITEM
    (LIST          : in  LIST_TYPE;
     lTEM_NAME     : in  IDENTIFIER_TEXT;
     STATUS        : in  STATUS_INDICATOR := NO_INDICATION)
    return ITEM_KIND;

function KIND_OF_ITEM
    (LIST          : in  LIST_TYPE;
     ITEM_NAME     : in  TOKEN_TYPE;
     STATUS        : in  STATUS_INDICATOR := NO_INDICATION)
    return ITEM_KIND;
```

Purpose:

This function returns the kind of an item in the current linear fist of LIST. 1

Parameters:

**LIST** is the list of interest. 2

**ITEM_POSITION** is the position of the item of interest within the current linear list of LIST. 3

**ITEM_NAME** is the name of the list item of interest within the current linear list of LIST. 4

**STATUS** is an established status indicator. 5

Exceptions:

**STATUS_ERROR** is raised if STATUS is not unestablished status indicator [3]. 6

**LIST_KIND_ERROR** is raised if the current linear list of LIST is empty [l]. 7

**LIST_POSITION_ERROR** is raised if ITEM_POSITION has a value larger than the current length of the current linear list of LIST [1]. 8

**SYNTAX_ERROR** is raised if the identifier-text value of the parameter ITEM_NAME of type IDENTIFIER_TEXT does not conform to the syntax of an Ada identifier [2]. 9

**TOKEN_ERROR** is raised if the ITEM_NAME of type TOKEN_TYPE is an undefined token [1]. 10



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

**NAMED_LIST_ERROR** is raised if the parameter ITEM_NAME is used and the current linear list of LIST is unnamed [2]. 11

**SEARCH_ERROR** is raised if there is no item in the current linear list of LIST with the name ITEM_NAME [1]. 12

## 9.1.10 Inserting a sequence of items into a linear list

```
procedure SPLICE
   (LIST        : in out LIST_TYPE;
    POSITION    : in     INSERT_COUNT;
    SOURCE_LIST : in     LIST_TYPE;
    STATUS      : in     STATUS_INDICATOR := NO_INDICATION) ;
```

Purpose:

This procedure allows the items of the current linear list of one list to be inserted into the current linear list of another list. Copies of the items in the linear list to be inserted will become items in the resulting linear list. The copied items will appear, in order, immediately following the item position designated by POSITION. Subsequent modifications to the value of LIST or to the value of SOURCE_LIST do not affect the other list. 1

Parameters:

**LIST** is the list into whose current linear list the copied items are to be inserted. 2

**POSITION** is the position in LIST's current linear list after which the new items will be inserted. 3

**SOURCE_LIST** is the list whose current linear list supplies the items to be inserted. 4

**STATUS** is an established status indicator. 6

Exceptions:

**STATUS_ERROR** is raised if STATUS is not an established status indicator [3]. 6

**LIST_KIND_ERROR** is raised if the current linear lists of LIST and SOURCE-LIST are not of the same kind and neither of them is an empty list [3]. 7

**LIST_POSITION_ERROR** is raised if POSITION has a value larger than the current length of the current linear list of LIST [1]. 8

**CAPACITY_ERROR** is raised if the number of items in the resulting linear list would exceed the value of the constant CAIS_PRAGMATICS.LIST_LENGTH. [1] (See CAIS_PRAGMATICS, Section 12 on page 815.) 9

**NAMED_LIST_ERROR** is raised if the current linear lists of LIST and SOURCE_LIST are both named and contain an item of the same name [3]. 10


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

## 9.1.11 Concatenating two linear lists

```
procedure CONCATENATE_LISTS
    (FRONT  : in     LIST_TYPE;
     BACK   : in     LIST_TYPE;
     RESULT : in out LIST-TYPE;
     STATUS : in     STATUS_INDICATOR := NO_INDICATION);
```

Purpose:

This procedure returns in RESULT a list constructed by concatenating the current linear list of BACK to the end of the current linear list of FRONT. The current linear lists of FRONT and BACK must be of the same kind or one must be an empty fist. The values of FRONT and BACK are not affected. Subsequent modifications to the value of FRONT or of BACK or to the value of the returned RESULT list do not affect either of the other (unmodified) lists. 1

Parameters:

**FRONT** is the first list whose current linear list is to be concatenated. 2

**BACK** is the second list whose current linear list is to be concatenated. 3

**RESULT** is the list produced by the concatenation; its outermost linear list has as its initial items the items in the current linear list of FRONT and as the rest of its items the items in the current linear list of BACK. 4

**STATUS** is an established status indicator. 5

Exceptions:

**STATUS_ERROR** is raised if STATUS is not an established status indicator [3]. 6

**LIST_KIND_ERROR** is raised if the current linear lists of FRONT and BACK are not of the same kind and neither of them is an empty list [3]. 7

**CAPACITY-ERROR** is raised if the number of items in the resulting linear list would exceed the value of the constant CAIS_PRAGMATICS.LIST_LENGTH [1]. (See CAIS_PRAGMATICS, Section 12 on page 815.) 8

**NAMED_LIST_ERROR** is raised if the current linear lists of FRONT and BACK are both named and contain an item of the same name [3]. 9

## 9.1.12 Extracting a sequence of items from a linear list

```
procedure EXTRACT-LIST
    (LIST           : in     LIS_TYPE;
     START_POSITION : in     POSITION_COUNT;
     END_POSITION   : in     POSITION_COUNT;
     RESULT_LIST    : in out LIST\_TYPE;
     STATUS         : in     STATUS_INDICATOR : = NO_INDICATION);
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Purpose:

This procedure extracts a sequence of items from the current linear list of the list LIST, forming a new list from them. The items to be extracted are those in the positions from START_POSITION through END_POSITION inclusive. Copies of the extracted items form the current (and outermost) linear list of RESULT_LIST. The list LIST is not modified. 1

Parameters:

**LIST** is the list whose current linear list supplies the items to be extracted. 2

**START_POSITION** is the position (within LIST's current linear list) of the first in the sequence of items to be extracted. 3

**END_POSITION** is the position (within LIST's current linear list) of the last in the sequence of items to be extracted. 4

**RESULT_LIST** is the list constructed from copies of the extracted items. 5

**STATUS** is an established status indicator. 6

Exceptions:

**STATUS_ERROR** is raised if STATUS is not an established status indicator [3]. 7

**LIST_POSITION_ERROR** is raised if START_POSITION is greater than END_POSITION [2] or if either START_POSITION or END_POSITION is greater than the number of items in the current linear list of LIST [1]. 8

### 9.1.13 Determining the length of a linear list

```
function NUMBER_OF_ITEMS
   (LIST   : in     LIST_TYPE;
    STATUS : in     STATUS_INDICATOR := NO_INDICATION)
   return LIST_SIZE;
```

Purpose:

This function returns a count of the number of items in the current linear list of LIST. If the current linear list is empty, zero is returned. 1

Parameters:

**LIST** is the list whose current linear list is being measured.

**STATUS** is an established status indicator.

Exceptions:

**STATUS_ERROR** is raised if STATUS is not an established status indicator [3].



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

## 9.1.14 Determining the position of the current linear list

```
function   POSITION_OF_CURRENT_LIST
    (LIST     : in      LIST_TYPE;
     STATUS : in      STATUS_INDICATOR  := NO_INDICATION)
    return POSITION_COUNT;
```

Purpose:

This function returns the position (within the innermost linear list containing the current linear list) of the item whose value is the current linear list. 1

Parameters:

**LIST** is the list whose current linear list is the value of some list item. 2

**STATUS** is an established status indicator. 3

Exceptions:

**STATUS_ERROR** is raised if STATUS is not an established status indicator [3]. 4

**LIST_POSITION_ERROR** is raised if the current linear list of LIST is the outermost linear list of LIST [3]. 5

## 9.1.15 Determining whether the current linear list is outermost

```
function CURRENT_LIST_IS_OUTERMOST
    (LIST     : in      LIST_TYPE;
     STATUS : in      STATUS_INDICATOR  : = NO_INDICATION)
    return BOOLEAN;
```

Purpose:

This function returns TRUE if the current linear list of LIST is the outermost linear list of LIST; otherwise it returns FALSE. 1

Parameters:

**LIST** is the list whose current linear list may either be the outermost linear list or the value of some list item. 2

**STATUS** is an established status indicator. 3

Exceptions:

**STATUS_ERROR** is raised if STATUS is not an established status indicator [3]. 4

## 9.1.16 Making the next outer linear list current

```
procedure  MAKE_CONTAINING_LIST_CURRENT
    (IN_LIST  :  in  out  LIST_TYPE;
     STATUS  :  in      STATUS_INDICATOR  : = NO_INDICATION);
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Purpose:

This procedure causes the innermost list containing the current linear list to become the (new) current linear list.

Parameters:

**IN_LIST** is the list whose current linear list is of interest.

**STATUS** is an established status indicator.

Exceptions:

**STATUS_ERROR** is raised if STATUS is not an established status indicator [3].

**LIST_POSITION.** ERROR is raised if the current linear list of IN-LIST is the outermost linear list of IN-LIST [3].

## 9.1.17 Making a nested sublist the current linear list

```
procedure MAKE_THIS_ITEM_CURREHT
    (IN_LIST       : in out LIST_TYPE;
     ITEM_POSITION : in     POSITION_COUNT;
     STATUS        : in     STATUS_INDICATOR  : = NO_INDICATION);

procedure MAKE_THIS_ITEM_CURRElfT
    (IN_LIST    : in out LIST-TYPE;
     ITEM_NAME : in     IDENTIFIER_TEXT;
     STATUS     : in     STATUS_INDICATOR  := NO_INDICATION);

procedure MAKE_THIS_ITEM_CURRENT
    (IN_LIST    : in out LIST_TYPE;
     ITEN_NAME : in     TOKEN_TYPE;
     STATUS     : in     STATUS_INDICATOR  := NO_INDICATION);
```

Purpose:

This procedure causes the list value of an item in the current linear list of LIST to become the (new) current linear list.

Parameters:

**IN_LIST** is the list whose current linear list is of interest.

**ITEM_POSITION** is the position (in the current linear list) of the list-kind item whose value is to become the (new) current linear list.

**ITEM_NAME** is the name of the list-kind item (in the current linear list) whose value is to become the (new) current linear list.

**STATUS** is an established status indicator.



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Exceptions:

**STATUS_ERROR** is raised if STATUS is not an established status indicator [3]. 6

**LIST_KIND_ERROR** is raised if the current linear list of IN_LIST is empty [1]. 7

**LIST_POSITION_ERROR** is raised if ITEM_POSITION has a value larger than the current length of the current linear list of IN_LIST [1]. 8

**SYNTAX_ERROR** is raised if the value of the parameter ITEM_NAME of type IDENTIFIER_TEXT does not conform to the syntax of an Ada identifier [2]. 9

**TOKEN_ERROR** is raised if the ITEM_NAME of type TOKEN_TYPE is an undefined token [1]. 10

**NAMED_LIST_ERROR** is raised if the parameter ITEM_NAME is used and the current linear list of IN_LIST is unnamed [2]. 11

**ITEM_KIND_ERROR** is raised if ITEM_POSITION or ITEM_NAME specifies an item whose value is not a list [1]. 12

**SEARCH_ERROR** is raised if there is no item in the current linear list of IN_LIST with the name ITEM_NAME [1]. 13

## 9.1.18 Determining the length of the text form of a list or a list item

```
function TEXT_LENGTH
    (LIST     : in     LIST_TYPE ;
     STATUS   : in     STATUS_INDICATOR  : = NO_INDICATION)
    return  CAIS_POSITIVE;

function TEXT_LENGTH
    (LIST          : in     LIST_TYPE;
     ITEM_POSITION : in     POSITION_COUNT;
     STATUS        : in     STATUS_INDICATOR  : = NO_INDICATION)
    return  CAIS_POSITIVE;

function TEXT_LENGTH
    (LIST          : in     LIST_TYPE;
     ITEM_NAME     : in     IDENTIFIER_TEXT;
     STATUS        : in     STATUS_INDICATOR  := NO_INDICATION)
    return  CAIS_POSITIVE;

function TEXT_LENGTH
    (LIST          : in     LIST_TYPE;
     ITEM_NAME     : in     TOKEN_TYPE;
     STATUS        : in     STATUS_INDICATOR  := NO_INDICATION)
    return  CAIS_POSITIVE;
```

Purpose:

This function returns the length of the text form of the current linear list of LIST [first interface], or the length of the text form of the value of a list item (of the current linear list) identified by ITEM_POSITION or ITEM_NAME [last three interfaces]. 1



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Parameters:

**LIST** is the list whose current linear list is of interest.

**ITEM_POSITION** is the position within the current linear list that identifies the item of interest.

**ITEM_NAME** is the name of the list item of interest.

**STATUS** is an established status indicator.

Exceptions:

**STATUS_ERROR** is raised if STATUS is not an established status indicator [3].

**LIST_KIND_ERROR** is raised if the current linear list of LIST is empty (last three interfaces) [1].

**LIST_POSITION_ERROR** is raised if ITEM_POSITION has a value larger than the current length of the current linear list of LIST [1].

**SYNTAX_ERROR** is raised if the value of the parameter ITEM_NAME of type IDENTIFIER_TEXT does not conform to the syntax of an Ada identifier [2].

**TOKEN_ERROR** is raised if the ITEM_NAME of type TOKEN_TYPE is an undefined token [1].

**NAMED_LIST_ERROR** is raised if the parameter ITEM_NAME is used and the current linear list of LIST is unnamed [2].

**SEARCH_ERROR** is raised if there is no item in the current linear list of LIST with the name ITEM_NAME [1].

Notes:

The text form of every value is non-null, and therefore the result is always positive. This is even true of the empty list, which is represented by the string value designated by the Ada string literal "()" and therefore has length 2.

## 9.1.19 Determining the name of a named item

```
procedure GET_ITEM_NAME
   (LIST          : in     LIST_TYPE;
    ITEM_POSITION : in     POSITION_COUNT;
    NAME          : in out TOKEN_TYPE;
    STATUS        : in     STATUS_INDICATOR := NO_INDICATION);
```

Purpose:

This procedure returns, in NAME, the token form of the name of item that is in the position indicated by ITEM_POSITION in the (named) current linear list of the LIST:



Source: https://assist.dla.mil -- Downloaded: 2026t0-26T07:54Z
Check the source to verify that this is the currentversion before use.

Parameters:

**LIST** is the list whose current linear list is of interest.

**ITEM_POSITION** is the position within the current linear list that identifies the item.

**NAME** is the token representation of the name of the item in the named current linear list.

**STATUS** is an established status indicator.

Exceptions:

**STATUS_ERROR** is raised if STATUS is not an established status indicator [3].

**LIST_KIND_ERROR** is raised if the current linear list of LIST is not a named list [2].

**LIST_POSITION_ERROR** is raised if ITEM_POSITION has a value larger than the current length of the current linear list of LIST [1].

## 9.1.20 Determining the position of a named item

```
function POSITION_BY_NAME
    (LIST      : in  LIST_TYPE;
     ITEM_NAME : in  IDENTIFIER_TEXT;
     STATUS    : in  STATUS_INDICATOR := NO_INDICATION)
    return POSITION_COUNT;

function POSITION_BY_NAME
    (LIST      : in  LIST_TYPE;
     ITEM_NAME : in  TOKEN_TYPE;
     STATUS    : in  STATUS_INDICATOR := NO_INDICATION)
    return POSITION_COUNT;
```

Purpose:

This function returns the position at which an item with the given name ITEM_NAME is located in the current linear list of LIST.

Parameters:

**LIST** is the list in whose current linear list the named item is to be located.

**ITEM_NAME** is the name of the item to be located.

**STATUS** is an established status indicator.

Exceptions:

**STATUS_ERROR** is raised if STATUS is not an established status indicator [3].

**LIST_KIND_ERROR** is raised if the current linear list of LIST is empty [1]



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

**SYNTAX_ERROR** is raised if the value of the parameter ITEM_NAME of type IDENTIFIER_TEXT does not conform to the syntax of an Ada identifier [2]. 7

**TOKEN_ERROR** is raised if the ITEM_NAME of type TOKEN_TYPE is an undefined token [1]. 8

**NAMED_LIST_ERROR** is raised if the parameter ITEM_NAME is used and the current linear list of LIST is unnamed [2]. 9

**SEARCH_ERROR** is raised if there is no item in the current linear list of LIST with the name ITEM_NAME [1]. 10

## 9.1.21 Package CAIS_LIST_ITEM

This is a package for manipulating list items whose values are list items. The exceptions raised by all subprograms in this package are defined in the packages CAIS_LIST_MANAGEMENT and CAIS_PRAGMATICS. 1

### 9.1.21.1 Extracting a list value from a list item

```
procedure EXTRACT_VALUE
    (FROM_LIST     : in     LIST_TYPE;
     ITEM_POSITION : in     POSITION_COUNT;
     VALUE         : in out LIST_TYPE;
     STATUS        : in     STATUS_INDICATOR := NO_INDICATION);

procedure EXTRACT-VALUE
    (FROM_LIST : in     LIST_TYPE;
     ITEM_NAME : in     IDENTIFIER_TEXT;
     VALUE     : in out LIST_TYPE;
     STATUS    : in     STATUS_INDICATOR := NO_INDICATION) ;

procedure EXTRACT_VALUE
    (FROM_LIST : in     LIST_TYPE;
     ITEM_NAME : in     TOKEN_TYPE;
     VALUE     : in out LIST_TYPE;
     STATUS    : in     STATUS_INDICATOR := NO_INDICATION);
```

Purpose:

This procedure locates a list-valued item in the current linear list of FROM_LIST and returns a copy of the list value as the current (and outermost) linear list of VALUE. Subsequent modification to the value of FROM_LIST or to the value returned in VALUE does not affect the other value. The list FROM_LIST is not modified. 1

Parameters:

**FROM_LIST** is the list whose current linear list contains the item to be extracted. 2



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

**ITEM_POSITION** is the position within the current linear list that identifies the item whose value is to be extracted.

**ITEM_NAME** is the name of the item whose value is to be extracted.

**VALUE** is the list value extracted from the designated item.

**STATUS** is an established status indicator.

Exceptions:

**STATUS_ERROR** is raised if STATUS is not an established status indicator [3].

**LIST_KIND_ERROR** is raised if the current linear list of FROM-LIST is empty [1].

**LIST_POSITION_ERROR** is raised if ITEM_POSITION has a value larger than the current length of the current linear list of FROM_LIST [1].

**SYNTAX_ERROR** is raised if the value of the parameter ITEM_NAME of type IDENTIFIER_TEXT does not conform to the syntax of an Ada identifier [2].

**TOKEN_ERROR** is raised if the ITEM_NAME of type TOKEN_TYPE is an undefined token [1].

**NAMED_LIST_ERROR** is raised if the parameter ITEM_NAME is used and the current linear list of FROM_LIST is unnamed [2].

**ITEM_KIND_ERROR** is raised if ITEM_POSITION or ITEM_NAME specifies an item whose value is not a list [1].

**SEARCH_ERROR** is raised if there is no item in the current linear list of FROM_LIST with the name ITEM_NAME [1].

## 9.1.21.2 Replacing a list value in a list item

```
procedure REPLACE
    (IN_LIST       : in out LIST_TYPE;
     ITEM_POSITION : in     POSITION_COUNT;
     VALUE         : in     LIST_TYPE;
     STATUS        : in     STATUS_INDICATOR := NO_INDICATION);

procedure REPLACE
    (IN_LIST     : in out LIST_TYPE;
     ITEM_NAME : in     IDENTIFIER_TEXT;
     VALUE     : in     LIST_TYPE ;
     STATUS    : in     STATUS_INDICATOR : = NO_INDICATION) ;

procedure REPLACE
    (IN_LIST     : in out LIST_TYPE;
     ITEM_NAME : in     TOKEN_TYPE;
     VALUE     : in     LIST_TYPE;
     STATUS    : in     STATUS_INDICATOR : = NO_INDICATION) ;
```



Source: https://assist.dla.mil -- Downloaded: 202610-26T07:54Z
Check the source to verify that this is the current version before use.

Purpose:

This procedure replaces the value of a list-valued item in the current linear list of IN_LIST with the current linear list of VALUE. Subsequent modification to the value of IN_LIST or of VALUE does not affect the other value. 1

Parameters:

**IN_LIST** is the list whose current linear list contains the item whose value is to be replaced. 2

**ITEM_POSITION** is the position within the current linear list that identifies the item whose value is to be replaced. 3

**ITEM_NAME** is the name of the item whose value is to be replaced. 4

**VALUE** is the list value whose current linear list is to become the new value of the designated item. 5

**STATUS** is an established status indicator. 6

Exceptions:

**STATUS_ERROR** is raised if STATUS is not an established status indicator [3]. 7

**LIST_KIND_ERROR** is raised if the current linear list of IN_LIST is empty [1]. 8

**LIST_POSITION_ERROR** is raised if ITEM_POSITION has a value larger than the current length of the current linear list of IN_LIST [I]. 9

**SYNTAX_ERROR** is raised if the value of the parameter ITEM_NAME of type IDENTIFIER_TEXT does not conform to the syntax of an Ada identifier [2]. 10

**TOKEN_ERROR** is raised if the ITEM_NAME of type TOKEN_TYPE is an undefined token [1]. 11

**NAMED_LIST_ERROR** is raised if the parameter ITEM_NAME is used and the current linear list of IN_LIST is unnamed [2]. 12

**ITEM_KIND_ERROR** is raised if ITEM_POSITION or ITEM_NAME specifies an item whose value is not a list [1]. 13

**SEARCH_ERROR** is raised if there is no item in the current linear list of IN_LIST with the name ITEM_NAME [1]. 14

## 9.1.21.3 Inserting a list-valued item into a linear list

```
procedure INSERT
    (IN_LIST : in out LIS_-TYPE;
     POSITION : in     INSERT_COUNT;
     VALUE    : in     LIST_TYPE;
     STATUS   : in     STATUS_INDICATOR := NO_INDICATION);

procedure INSERT
    (IN_LIST : in out LIST_TYPE;
     POSITION : in     INSERT_COUNT;
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
        NAME      : in      IDENTIFIER_TEXT;
        VALUE     : in      LIST_TYPE;
        STATUS    : in      STATUS_INDICATOR  := NO_INDICATION);

    procedure  INSERT
        (IN_LIST  : in out  LIST_TYPE;
         POSITION : in      INSERT_COUNT;
         NAME     : in      TOKEN_TYPE;
         VALUE    : in      LIST_TYPE;
         STATUS   : in      STATUS_INDICATOR  := NO_INDICATION);
```

Purpose:

This procedure inserts a list-valued item into the current linear list of IN_LIST; the new list item will repositioned after the list item specified by POSITION. The list value of the item to be inserted is the current linear list of the VALUE parameter. A value of zero imposition specifies a position at the head of the current linear list. Subsequent modification to the value of IN_LIST or of VALUE does not affect the other value. If the current linear list of IN_LIST is empty, it will be a named list after the successful completion of the call, if the second or third interface is used, and an unnamed list otherwise. 1

Parameters:

**IN_LIST** is the list into whose current linear list the item will be inserted. 2

**POSITION** is the position in the current linear list after which the item is to be inserted. 3

**NAME** is the name of the new item to be inserted. 4

**VALUE** is the list value whose current linear list is the value of the new item to be inserted. 5

**STATUS** is an established status indicator. 6

Exceptions:

**STATUS_ERROR** is raised if STATUS is not an established status indicator [3]. 7

**LIST_KIND_ERROR** is raised if an attempt is made to insert an item by NAME into an unnamed list [4] or, conversely, if an attempt is made to insert an item without NAME into a named list [5]. 8

**LIST_POSITION_ERROR** is raised if ITEM_POSITION has a value larger than the current length of the current linear list of IN_LIST [1]. 9

**SYNTAX_ERROR** is raised if the value of the parameter NAME of type IDENTIFIER_TEXT does not conform to the syntax of an Ada identifier [2]. 10

**TOKEN_ERROR** is raised if the NAME of type TOKEN_TYPE is an undefined token [1]. 11

**CAPACITY_ERROR** is raised if the number of items in the resulting linear list would exceed the value of the constant CAIS_PRAGMATICS.LIST_LENGTH. [1] (See CAIS_PRAGMATICS, Section 12 on page 815.) 12

**NAMED_LIST_ERROR** is raised if the current linear list of IN_LIST is a named list that already contains an item with the name given by NAME [4]. 13



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

## 9.1.21.4 Locating a list-valued item by value within a linear list

```
function  POSITION_BY_VALUE
    (LIST           : in    LIST_TYPE;
     VALUE          : in    LIST_TYPE;
     START_POSITION : in    POSITION_COUNT      := POSITION_COUNT'FIRST;
     END_POSITION   : in    POSITION_COUNT      := POSITION_COUNT'LAST;
     STATUS         : in    STATUS_INDICATOR := NO_INDICATION)
    return POSITION_COUNT;
```

Purpose:

This function returns the position in the current linear list of LIST of the next list-valued item whose value equals that of the current linear list of the VALUE parameter under list equality (see Section 9 on page 745). The search begins at the START_POSITION and ends when either an item whose value equals the current linear list of VALUE is found, the last item of the list has been examined, or the item at the END_POSITION has been examined, whichever comes first. 1

Parameters:

**LIST** is the list in whose current liner list the item of interest is to be located. 2

**VALUE** is the list value whose current linear list is the value of interest. 3

**START_POSITION** is the position of the first item in the current linear list of LIST to be considered in the search. 4

**END_POSITION** is the position beyond which the search will not proceed; the search may terminate prior to reaching END_POSITION should the sought item be found or should the last element of the list be considered. 5

**STATUS** is an established status indicator. 6

Exceptions:

**STATUS_ERROR** is raised if STATUS is not an established status indicator [3]. 7

**LIST_KIND_ERROR** is raised if the current linear list of LIST is empty [1]. 8

**LIST_POSITION_ERROR** is raised if START_POSITION specifies a value larger than the current length of the current linear list of LIST [1], or if END_POSITION is less than START_POSITION [2]. 9

**SEARCH_ERROR** is raised if there is no item in the current linear list of LIST within the region specified by START_POSITION and END_POSITION that has the value specified by VALUE [2]. 10

Notes:

Determining the position by value of a list involving floating point items should be applied with considerable caution and awareness of all the issues documented in [1815A], Section 4.5.7, regarding the accuracy of relational operations with real operands and the discussion in Section 9 on page 745, defining floating point equality for floating point list items. 11



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

## 9.1.22 Package CAIS_IDENTIFIER_ITEM

This package provides interfaces for the manipulation of identifier values, including identifier items in lists, using the token form or the identifier text form. Manipulation of identifiers in the enumeration form is provided by the generic package CAIS_ENUMERATION_ITEM. The exceptions raised by all subprograms in this package are defined in the packages CAIS_LIST_MANAGEMENT and CAIS_-PRAGMATICS. 1

Identifier values are represented at the interfaces as values of the types TOKEN_TYPE or IDENTI-FIER_TEXT. 2

### 9.1.22.1 Copying a token

```
procedure COPY_TOXEN
    (PROM_TOKEN : in     TOKEN_TYPE;
     TO_TOKEN   : in out TOKEN_TYPE;
     STATUS     : in     STATUS_INDICATOR := NO_INDICATION);
```

Purpose:

This procedure returns in TO-TOKEN a copy of the token in FROM_ TOKEN.

Parameters:

**FROM_TOKEN** is the token to be copied. 2

**TO_TOKEN** is the copied token to be returned. 3

**STATUS** is an established status indicator. 4

Exceptions:

**STATUS_ERROR** is raised if STATUS is not an established status indicator [3]. 5

**TOKEN_ERROR** is raised if FROM_TOKEN is an undefined token [1]. 6

### 9.1.22.2 Converting an identifier from text to token form

```
procedure CONVERT_TEXT_TO_TOKEN
    (IDENTIFIER : in     IDENTIFIER_TEXT;
     TOKEN      : in out TOKEN_TYPE;
     STATUS     : in     STATUS_INDICATOR := NO_INDICATION);
```

Purpose:

This procedure converts the text representation of an identifier into the corresponding token representation. 1



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Parameters:

**IDENTIFIER** is the text to be converted to a token. 2

**TOKEN** is the token corresponding to the value of IDENTIFIER. 3

**STATUS** is an established status indicator. 4

Exceptions:

**STATUS_ERROR** is raised if STATUS is not an established status indicator [3]. 5

**SYNTAX_ERROR** is raised if the value of the parameter IDENTIFIER does not conform to the syntax of an Ada identifier [2]. 6

**CAPACITY_ERROR** is raised if the length of the IDENTIFIER parameter exceeds the value of the constant CAIS_PRAGMATICS.IDENTIFIER_ITEM_LENGTH (see CAIS_PRAGMATICS, Section 12 on page 815) [1]. 7

## 9.1.22.3 Converting an identifier from token to text form

```
function TEXT_FORM
    (TOKEN  : in     TOKEN_TYPE;
     STATUS : in     STATUS_INDICATOR  := NO_INDICATION)
    return   IDENTIFIER_TEXT;
```

Purpose:

This function returns the text representation of the token value of the TOKEN parameter. The result has the syntax of an Ada identifier. 1

Parameters:

**TOKEN** is the identifier expressed as a token.

**STATUS** is an established status indicator.

Exceptions:

**STATUS_ERROR** is raised if STATUS is not an established status indicator [3].

**TOKEN_ERROR** is raised if the TOKEN is an undefined token [1].

## 9.1.22.4 Deleting a token

```
procedure  DELETE_TOKEN
    (TOKEN   : in out TOKEN_TYPE;
     STATUS : in     STATUS_INDICATOR  := NO_INDICATION);
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Purpose:

This procedure sets the identifier token TOKEN to the undefined token value, so that TOKEN is no longer associated with an identifier. The procedure has no effect if TOKEN already has the undefined token value. 1

Parameters:

**TOKEN** is the token to be deleted. 2

**STATUS** is an established status indicator. 3

Exceptions:

**STATUS_ERROR** is raised if STATUS is not an established status indicator [3]. 4

## 9.1.22.6 Determining the equality of two identifier tokens

```
function IS_EQUAL
    (TOKEN1 : in     TOKEN_TYPE ;
     TOKEN2 : in     TOKEN_TYPE;
     STATUS : in     STATUS_INDICATOR : = NO_INDICATION)
    return BOOLEAN;
```

Purpose:

This function returns TRUE if the two identifier tokens TOKEN1 and TOKEN2 are equal under equality of identifiers; otherwise, it returns FALSE. 1

Parameters:

**TOKEN1, TOKEN2** are the identifier tokens whose equality is to be determined.

**STATUS** is an established status indicator.

Exceptions:

**STATUS_ERROR** is raised if STATUS is not an established status indicator [3].

**TOKEN_ERROR** is raised if either TOKEN1 or TOKEN2 is an undefined token [1].

## 9.1.22.6 Extracting an identifier value from a list item

```
procedure EXTRACT_VALUE
    (FROM_LIST     :  in      LIST_TYPE;
     ITEM_POSITION : in      POSITION_COUNT;
     VALUE         : in out  TOKEN_TYPE;
     STATUS        :  in      STATUS- INDICATOR  := NO_INDICATION);
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
procedure EXTRACT_VALUE
    (FRON_LIST : in      LIST_TYPE;
     lTEM_NAME : in      IDENTIFIER_TEXT;
     VALUE     : in out  TOKE1_TYPE;
     STATUS    : in      STATUS_INDICATOR := NO_INDICATION);

procedure EXTRACT_VALUE
    (FROM_LIST : in      LIST_TYPE;
     ITEN_NAME : in      TOKE1_TYPE;
     VALUE     : in out  TOKEN_TYPE;
     STATUS    : in      STATUS_INDICATOR := NO_INDICATION);
```

Purpose:

This procedure locates an identifier-valued item in the current linear list of FROM_LIST and returns a copy of the identifier value (in token form) in VALUE. The list FROM_LIST is not modified. 1

Parameters:

**FROM_LIST** is the list whose current linear list contains the item to be extracted. 2

**ITEM_POSITION** is the position within the current linear list that identifies the item whose value is to be extracted. 3

**ITEM_NAME** is the name of the item whose value is to be extracted. 4

**VALUE** is the identifier-value (in token form) extracted from the designated item. 5

**STATUS** is an established status indicator. 6

Exceptions:

**STATUS_ERROR** is raised if STATUS is not unestablished status indicator [3]. 7

**LIST_KIND_ERROR** is raised if the current linear list of FROM_LIST is empty [l]. 8

**LIST_POSITION_ERROR** is raised if ITEM_POSITION has a value larger than the current length of the current linear list of FROM_LIST [1]. 9

**SYNTAX_ERROR** is raised if the value of the parameter ITEM_NAME of type IDENTIFIER_TEXT does not conform to the syntax of an Ada identifier [2]. 10

**TOKEN_ERROR** is raised if the ITEM_NAME of type TOKEN_TYPE is an undefined token [1]. 11

**NAMED_LIST_ERROR** is raised if the parameter ITEM_NAME is used and the current linear list of FROM_LIST is unnamed [2]. 12

**ITEM_KIND_ERROR** is raised if ITEM_POSITION or ITEM_NAME specifies an item whose value is not an identifier [1]. 13

**SEARCH_ERROR** is raised if there is no item in the current linear list of FROM_LIST with the name ITEM_NAME [1]. 14



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

## 9.1.22.7 Replacing an identifier value in a list item

```
procedure REPLACE
    (IN_LIST       : in out LIST_TYPE;
     ITEM_POSITION : in     POSITION_COUNT
     VALUE         : in     TOKEN_TYPE;
     STATUS        : in     STATUS_INDICATOR := NO_INDICATION);

procedure REPLACE
    (IN_LIST     : in out LIST_TYPE;
     ITEM_NAME : in     IDENTIFIER_TEXT;
     VALUE     : in     TOKEN_TYPE;
     STATUS    : in     STATUS_INDICATOR := NO_INDICATION);

procedure REPLACE
    (IN_LIST     : in out LIST_TYPE;
     ITEM_NAME : in     TOKEN_TYPE;
     VALUE     : in     TOKEN_TYPE;
     STATUS    : in     STATUS_INDICATOR := NO_INDICATION);
```

Purpose:

This procedure replaces the value of an identifier-valued item in the current linear list of IN_LIST with VALUE. Subsequent modification to the value of IN_LIST or of VALUE does not affect the other value. 1

Parameters:

**IN_LIST** is the list whose current linear list contains the item whose value is to be replaced. 2

**ITEM_POSITION** is the position within the current linear list that identifies the item whose value is to be replaced. 3

**ITEM_NAME** is the name of the item whose value is to be replaced. 4

**VALUE** is the new identifier value (in token form) of the designated item. 5

**STATUS** is an established status indicator. 6

Exceptions:

**STATUS_ERROR** is raised if STATUS is not an established status indicator [3]. 7

**LIST_KIND_ERROR** is raised if the current linear list of IN_LIST is empty [1]. 8

**LIST_POSITION_ERROR** is raised if ITEM_POSITION has a value larger than the current length of the current linear list of IN_LIST [1]. 9

**SYNTAX_ERROR** is raised if the value of the parameter ITEM_NAME of type IDENTIFIER_TEXT does not conform to the syntax of an Ada identifier [2]. 10

**TOKEN_ERROR** is raised if the ITEM_NAME of type TOKEN_TYPE is an undefined token or if VALUE is an undefined token [1]. 11


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

**NAMED_LIST_ERROR** is raised if the parameter ITEM_NAME is used and the current linear list of IN-LIST is unnamed [2]. 12

**ITEM_KIND_ERROR** is raised if ITEM_POSITION or ITEM_NAME specifies an item whose value is not an identifier [1]. 13

**SEARCH_ERROR** is raised if there is no item in the current linear list of IN_LIST with the name ITEM_NAME [1]. 14

## 9.1.22.8 Inserting an identifler-valued item into a linear list

```
procedure INSERT
    (IN_LIST   : in out LIST_TYPE;
     POSITION  : in     IISERT_COUET;
     VALUE     : in     TOKEN_TYPE;
     STATUS    : in     STATUS_INDICATOR := NO_INDICATION);

procedure INSERT
    (IN_LIST   : in out LIST_TYPE;
     POSITION  : in     INSERT_COUIT;
     NAME      : in     IDENTIFIER_TEXT;
     VALUE     : in     TOKEN_TYPE;
     STATUS    : in     STATUS_INDICATOR := NO_INDICATION);

procedure INSERT
    (IN_LIST   : in out LIST_TYPE;
     POSITION  : in     INSERT_COUIT;
     NAME      : in     TOKE1_TYPE;
     VALUE     :  in    TOKE1_TYPE;
     STATUS    : in     STATUS_INDICATOR := NO_INDICATION);
```

Purpose:

This procedure inserts a identifier-valued item into the current linear list of IN_LIST; the new list item will be positioned after the list item specified by POSITION. A value of zero in POSITION specifies a position at the head of the current linear list. Subsequent modification to the value of IN_LIST or of VALUE does not affect the other value. If the current linear list of IN_LIST is empty, it will be a named list after the successful completion of the call, if the second or third interface is used, and an unnamed list otherwise. 1

Parameters:

**IN_LIST** is the list into whose current linear list the item will be inserted. 2

**POSITION** is the position in the current linear list after which the item is to be inserted. 3

**NAME** is the name of the new item to be inserted. 4

**VALUE** is the identifier value (in token form) of the new item to be inserted. 5

**STATUS** is an established status indicator. 6



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Exceptions:

**STATUS_ERROR** is raised if STATUS is not an established status indicator [3]. 7

**LIST_KIND_ERROR** is raised if an attempt is made to insert an item by NAME into an unnamed list [4] or, conversely, if an attempt is made to insert an item without NAME into a named list [5]. 8

**LIST_POSITION_ERROR** is raised if ITEM_POSITION has a value larger than the current length of the current linear list of IN_LIST [1]. 9

**SYNTAX_ERROR** is raised if the value of the parameter NAME of type IDENTIFIER_TEXT does not conform to the syntax of an Ada identifier [2]. 10

**TOKEN_ERROR** is raised if the NAME of type TOKEN_TYPE is an undefined token or if VALUE is an undefined token [1]. 11

**CAPACITY_ERROR** is raised if the number of items in the resulting linear list would exceed the value of the constant CAIS_PRAGMATICS.LIST_LENGTH. [1] (See CAIS_PRAGMATICS, Section 12 on page 815.) 12

**NAMED_LIST_ERROR** is raised if the current linear list of IN_LIST is a named list that already contains an item with the name given by NAME [4]. 13

## 9.1.22.9 Locating an identifier-valued item by value within a linear list

```
function POSITION_BY_VALUE
   (LIST           : in  LIST_TYPE;
    VALUE          : in  TOKEN_TYPE ;
    START_POSITION : in  POSITION_COUNT     : = POSITION_COUMT‘FIRST;
    END_POSITION   : in  POSITION_COUNT     : = POSITION_COUNT’LAST;
    STATUS         : in  STATUS- INDICATOR  : = NO_INDICATION)
   return POSITION_COUNT;
```

Purpose:

This function returns the position in the current linear list of LIST of the next identifier-valued item whose value equals that of the VALUE parameter under equality rule for identifier items (see Section 9 on page 745). The search begins at the START_POSITION and ends when either an item whose value equals VALUE is found, the last item of the list has been examined, or the item at the END-POSITION has been examined, whichever comes fist. 1

Parameters:

**LIST** is the list in whose current linear list the item of interest is to be located. 2

**VALUE** is the identifier value of interest. 3

**START_POSITION** is the position of the first item in the current linear list to be considered in the search. 4

**END_POSITION** is the position beyond which the search will not proceed; the search may terminate prior to reaching END_POSITION should the sought item be found or should the last element of the list be considered. 5

**STATUS** is an established status indicator. 6



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Exceptions:

**STATUS_ERROR** is raised if STATUS is not an established status indicator [3]. 7

**LIST_KIND_ERROR** is raised if the current linear list of LIST is empty [1]. 8

**LIST_POSITION_ERROR** is raised if START_POSITION specifies a value larger than the current length of the current linear list of LIST [1], or if END_POSITION is less than START_POSITION [2]. 9

**TOKEN_ERROR** is raised if VALUE is an undefined token [1]. 10

**SEARCH_ERROR** is raised if there is no item in the current linear list of LIST within the region specified by START_POSITION and END_POSITION that has the value specified by VALUE [2]. 11

## 9.1.22.10 Determining whether an identifier item is enumerated

```
function IS_ENUMERATED
    (FROM_LIST     : in    LIST_TYPE;
     ITEM_POSITION : in    POSITION_COUNT;
     STATUS        : in    STATUS_INDICATOR := NO_INDICATION)
    return BOOLEAN;

function IS_ENUMERATED
    (FROM_LIST : in    LIST_TYPE;
     ITEM_NAME : in    IDENTIFIER_TEXT;
     STATUS    : in    STATUS- INDICATOR : = NO_INDICATION)
    return BOOLEAN;

function IS_ENUMERATED
    (FROM_LIST : in    LIST_TYPE;
     ITEM_NAME : in    TOKEN1_TYPE;
     STATUS    : in    STATUS_INDICATOR := NO_INDICATION)
    return BOOLEAN;
```

Purpose:

This procedure locates an identifier-valued item in the current linear list of FROM_LIST and returns TRUE if the item is an enumerated identifier item and FALSE if the item is an ordinary identifier item. An identifier item is enumerated if its value was created or set using a value of an Ada enumeration type, or if the value resulted from reading an attribute or a component of an attribute which conforms to an enumerated identifier AKD. An enumerated identifier item has an associated relative position within an enumeration type. 1

Parameters:

**FROM_LIST** is the list whose current linear list contains the item to be tested. 2

**ITEM_POSITION** is the position within the current linear list of the item to be tested. 3

**ITEM_NAME** is the name within the current linear list of the item to be tested. 4

**STATUS** is an established status indicator. 5



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Exceptions:

**STATUS_ERROR** is raised if STATUS is not an established status indicator [3]. 6

**LIST_KIND_ERROR** is raised if the current linear list of FROM-LIST is empty [1]. 7

**LIST_POSITION_ERROR** is raised if ITEM_POSITION has a value larger than the current length of the current linear list of FROM_LIST [1]. 8

**SYNTAX_ERROR** is raised if the value of the parameter ITEM_NAME of type IDENTIFIER_TEXT does not conform to the syntax of an Ada identifier [2]. 9

**TOKEN_ERROR** is raised if the ITEM_NAME of type TOKEN_TYPE is an undefined token [1]. 10

**NAMED_LIST_ERROR** is raised if the parameter ITEM_NAME is used and the current linear list of FROM_LIST is unnamed [2]. 11

**ITEM_KIND_ERROR** is raised if ITEM_POSITION or ITEM_NAME specifies an item whose value is not an identifier [1]. 12

**SEARCH_ERROR** is raised if there is no item in the current linear list of FROM_LIST with the name ITEM_NAME [1]. 13

## 9.1.22.11 Determining the enumeration position of an enumerated identifier item

```
function ENUMERATIN_POSITION
    (FROM_LIST     : in    LIST_TYPE;
     ITEM_POSITION : in    POSITION_COUNT;
     STATUS        : in    STATUS_INDICATOR  := NO_INDICATION)
    return CAIS_NATURAL;

function ENUMERATION_POSITION
    (FROM_LIST : in    LIST_TYPE;
     ITEM_NAME : in    IDENTIFIER_TEXT;
     STATUS    : in    STATUS\_INDICATOR  := NO_INDICATION)
    return CAIS_NATURAL;

function ENUMERATION_POSITION
    (FROM_LIST : in    LIST_TYPE;
     ITEM_NAME : in    TOKEN_TYPE;
     STATUS    : in    STATUS_INDICATOR  := NO_INDICATION)
    return CAIS_NATURAL;
```

Purpose:

This procedure locates an enumerated identifier item in the current linear list of FROM_LIST and returns the enumeration position of the enumeration value (within its base enumeration type) that was used to create or set the value of the item. 1



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Parameters:

**FROM_LIST** is the list whose current linear list contains the enumerated identifier item. 2

**ITEM_POSITION** is the position of the item within the current linear list. 3

**ITEM_NAME** is the name of the item within the current linear list. 4

**STATUS** is an established status indicator. 5

Exceptions:

**STATUS_ERROR** is raised if STATUS is not an established status indicator [3]. 6

**LIST_KIND_ERROR** is raised if the current linear list of FROM_LIST is empty [1]. 7

**LIST_POSITION_ERROR** is raised if ITEM_POSITION has a value larger than the current length of the current linear list of FROM_LIST [1]. 8

**SYNTAX_ERROR** is raised if the value of the parameter ITEM_NAME of type IDENTIFIER_TEXT does not conform to the syntax of an Ada identifier [2]. 9

**TOKEN_ERROR** is raised if the ITEM_NAME of type TOKEN_TYPE is an undefined token [1]. 10

**NAMED_LIST_ERROR** is raised if the parameter ITEM_NAME is used and the current linear list of FROM_LIST is unnamed [2]. 11

**ITE_-KIND_ERROR** is raised if ITEM_POSITION or ITEM_NAME specifies an item which is not an enumerated identifier item [1]. 12

**SEARCH_ERROR** is raised if there is no item in the current linear list of FROM_LIST with the name ITEM_NAME [1]. 13

### 9.1.23 Generic package CAIS_ENUMERATION_ITEM

This is a generic package for manipulating identifier values and identifier items in lists in the external form of Ada enumeration values. This package must be instantiated for the enumeration type (indicated by ENUMERATION in the specification). The exceptions raised by all subprograms in this package are defined in the packages CAIS_LIST_MANAGEMENT and CAIS_PRAGMATICS. 1

Those interfaces defined by this package which manipulate list items may be applied to both ordinary identifier items and enumerated identifier items. Operations that retrieve an enumerated identifier item in enumeration form check that the enumeration position associated with the enumerated identifier item is the same as the position of the corresponding enumeration value (in the base type for ENUMERATION) which has the same image as the identifier text form of the item under identifier equality. 2

```
generic
    typo ENUMERATION is (<>);
package CAIS_ENUMERATION_ITEM is
    -- Specifications of subprograms for this generic package.
and CAIS_ENUMERATION_ITEM;
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

### 9.1.23.1 Converting an enumeration value to its canonical text represent at ion

```
function TEXT_FORM
    (ENUMERATION_VALUE : in    ENUMERATION;
     STATUS            : in    STATUS_INDICATOR := NO_INDICATION)
    return IDENTIFIER_TEXT;
```

Purpose:

This function returns the canonical text form representation of the value of the ENUMERATION_VALUE parameter. The canonical text form representation is the same as the string returned by the following Ada attribute operation 1

ENUMERATION_IMAGE (ENUMERATION_VALUE)

Parameters:

**ENUMERATION_VALUE** is the enumeration value whose identifier text form is to be returned. 2

**STATUS** is an established status indicator. 3

Exceptions:

**STATUS_ERROR** is raised if STATUS is not an established status indicator [3].

### 9.1.23.2 Extracting an enunmeration value from a list item

```
function EXTRACTED-VALUE
    (FROM_LIST     : in    LIST_TYPE;
     ITEM_POSITION : in    POSITION_COUNT;
     STATUS        : in    STATUS_INDICATOR := NO_INDICATION)
    return ENUMERATION;

function EXTRACTED_VALUE
    (FROM_LIST : in    LIST_TYPE;
     ITEM_NAME : in    IDENTIFIER_TEXT;
     STATUS    : in    STATUS_INDICATOR := NO_INDICATION)
    return ENUMERATION;

function EXTRACTED_VALUE
    (FROM_LIST : in    LIST_TYPE;
     ITEM_NAME : in    TOKEN_TYPE;
     STATUS    : in    STATUS_INDICATOR := NO_INDICATION)
    return ENUMERATION;
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Purpose:

This function locates an identifier item in the current linear list of FROM_LIST and returns a copy of its value in the form of an enumeration value of the type ENUMERATION. The value returned is the value of the Ada attribute ENUMERATION'VALUE applied to the identifier text form of the identifier item. The list FROM_LIST is not modified. 1

Parameters:

**FROM_LIST** is the list whose current linear list contains the item whose value is to be extracted. 2

**ITEM_POSITION** is the position within the current linear list of the item whose value is to be extracted. 3

**ITEM_NAME** is the name of the item whose value is to be extracted. 4

**STATUS** is an established status indicator. 5

Exceptions:

**STATUS_ERROR** is raised if STATUS is not an established status indicator [3]. 6

**LIST_KIND_ERROR** is raised if the current linear list of FROM_LIST is empty [1]. 7

**LIST_POSITION_ERROR** is raised if ITEM_POSITION has a value larger than the current length of the current linear list of FROM_LIST [1]. 8

**SYNTAX_ERROR** is raised if the value of the parameter ITEM_NAME of type IDENTIFIER_TEXT does not conform to the syntax of an Ada identifier [2]. 9

**TOKEN_ERROR** is raised if the ITEM_NAME of type TOKEN_TYPE is an undefined token [1]. 10

**NAMED_LIST_ERROR** is raised if the parameter ITEM_NAME is used and the current linear list of FROM_LIST is unnamed [2]. 11

**ITEM_KIND_ERROR** is raised if ITEM_POSITION or ITEM_NAME identifies an item which is not an identifier item [1], if there is no enumeration value of type ENUMERATION corresponding (by image) to the identifier item value [2] or if the item is an enumerated identifier item but the position of the corresponding enumeration value in the base type of ENUMERATION is not the same as the enumeration position of the item [3]. 12

**CONSTRAINT_ERROR** is raised if the value to be extracted violates the constraints of the type designated by NUMBER. 13

**SEARCH_ERROR** is raised if there is no item in the current linear list of FROM-LIST with the name ITEM_NAME [1]. 14

## 9.1.23.3 Replacing an identifier item in a list item with an enumeration value

```
procedure REPLACE
   (IN_LIST       : in out LIST_TYPE;
    ITEM_POSITION : in     POSITION_COUNT;
    VALUE         : in     ENUMERATION;
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
        STATUS          : in      STATUS_INDICATOR := NO_INDICATION);

    procedure REPLACE
        (IN_LIST    : in out LIST_TYPE;
         ITEM_NAME : in      IDENTIFIER_TEXT;
         VALUE      : in      ENUMERATION;
         STATUS     : in      STATUS_INDICATOR := NO_INDICATION);

    procedure REPLACE
        (IN_LIST    : in out LIST_TYPE;
         ITEM_NAME : in      TOKEN_TYPE;
         VALUE      : in      ENUMERATION;
         STATUS     : in      STATUS_INDICATOR := NO_INDICATION);
```

Purpose:

This procedure replaces the value of an identifier item in the current linear list of IN_LIST with an enumerated identifier value determined by VALUE. Subsequent modification to the value of IN_LIST or of VALUE does not affect the other value. 1

The enumerated identifier value placed in the list has the identifier text form given by the Ada attribute operation 2

```
ENUMERATION'IMAGE (VALUE)
```

and the enumeration position given by the Ada attribute operation

```
ENUMERATION'POS (VALUE)
```

Parameters:

**IN_LIST** is the list whose current linear list contains the item whose value is to be replaced. 3

**ITEM_POSITION** is the position within the current linear list that identifies the item whose value is to be replaced. 4

**ITEM_NAME** is the name within the current linear list of the item whose value is to be replaced. 5

**VALUE** is the new enumeration value of the designated item. 6

**STATUS** is an established status indicator. 7

Exceptions:

**STATUS_ERROR** is raised if STATUS is not an established status indicator [3]. 8

**LIST_KIND_ERROR** is raised if the current linear list of IN_LIST is empty [1]. 9

**LIST_POSITION_ERROR** is raised if ITEM_POSITION has a value larger than the current length of the current linear list of IN_LIST [1]. 10


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

**SYNTAX_ERROR** is raised if the value of the parameter ITEM_NAME of type IDENTIFIER_TEXT does not conform to the syntax of an Ada identifier [2] or if the string produced by applying the Ada attribute operator ENUMERATION'IMAGE to VALUE does not conform to the syntax of an Ada identifier [2]. 11

**TOKEN_ERROR** is raised if the ITEM_NAME of type TOKEN_TYPE is an undefined token [1]. 12

**NAMED_LIST_ERROR** is raised if the parameter ITEM_NAME is used and the current linear list of IN_LIST is unnamed [2]. 13

**ITEM_KIND_ERROR** is raised if ITEM_POSITION or ITEM_NAME specifies an item whose value is not an identifier [1]. 14

**SEARCH_ERROR** is raised if there is no item in the current linear list of IN_LIST with the name ITEM_NAME [1]. 15

## 9.1.23.4 Inserting an enumerated identifier item into a linear list

```
procedure INSERT
   (IN_LIST : in out LIST_TYPE;
    POSITION : in     INSERT_COUNT;
    VALUE    : in     ENUMERATION;
    STATUS   : in     STATUS_INDICATOR := NO_INDICATION);

procedure INSERT
   (IN_LIST : in out LIST_TYPE;
    POSITION : in     INSERT_COUNT;
    NAME     : in     IDENTIFIER_TEXT;
    VALUE    : in     ENUMERATION;
    STATUS   : in     STATUS_INDICATOR := NO_INDICATION);

procedure INSERT
   (IN_LIST : in out LIST_TYPE;
    POSITION : in     INSERT_COUNT ;
    NAME     : in     TOKEN_TYPE;
    VALUE    : in     ENUMERATION;
    STATUS   : in     STATUS_INDICATOR := NO_INDICATION);
```

Purpose:

This procedure inserts an enumerated identifier item into the current linear list of IN_LIST; the new list item will be positioned after the list item specified by POSITION. A value of zero in POSITION specifies a position at the head of the current linear list. Subsequent modification to the value of IN_LIST or of VALUE does not affect the other value. If the current linear list of IN_LIST is empty, it will be a named list after the successful completion of the call, if the second or third interface is used, and an unnamed list otherwise. 1

The enumerated identifier value inserted has the identifier text form given by the Ada attribute operation 2

```
ENUMERATION'IMAGE (VALUE)
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

and the enumeration position given by the Ada attribute operation

ENUMERATION'POS (VALUE).

Parameters:

**IN_LIST** is the list into whose current linear list the item will be inserted.

**POSITION** is the position in the current linear list after which the item is to be inserted.

**NAME** is the name of the new item to be inserted.

**VALUE** is the enumeration value of the new enumerated identifier item to be inserted.

**STATUS** is an established status indicator.

Exceptions:

**STATUS_ERROR** is raised if STATUS is not an established status indicator [3].

**LIST_KIND_ERROR** is raised if an attempt is made to insert an item by NAME into an unnamed list [4] or, conversely, if an attempt is made to insert an item without NAME into a named list [5].

**LIST_POSITION_ERROR** is raised if ITEM_POSITION has a value larger than the current length of the current linear list of IN_LIST [1].

**SYNTAX_ERROR** is raised if the value of the parameter NAME of type IDENTIFIER_ - TEXT does not conform to the syntax of an Ada identifier [2] or if the string produced by applying the Ada attribute operator ENUMERATION'IMAGE to VALUE does not conform to the syntax of an Ada identifier [2].

**TOKEN_ERROR** is raised if the NAME of type TOKEN_TYPE is an undefined token [1].

**CAPACITY_ERROR** is raised if the number of items in the resulting linear list would exceed the value of the constant CAIS_PRAGMATICS.LIST_LENGTH.

**NAMED_LIST_ERROR** is raised if the current linear list of IN_LIST is a named list that already contains an item with the name given by NAME [4].

## 9.1.23.5 Locating an identifier item by enumeration value within a linear list

```
function POSITIOS_BY_VALUE
   (LIST           : in  LIST_TYPE;
    VALUE          : in  ENUMERATION;
    START_POSITION : in  POSITION_COUNT     := POSITION_COUNT'FIRST;
    END_POSITION   : in  POSITION_COUNT     : = POSITION_COUNT'LAST;
    STATUS         : in  STATUS_INDICATOR   := NO_INDICATION)
   return POSITION_COUNT;
```

Purpose:

This function returns the position in the current linear list of LIST of the next identifier item which matches the value of the parameter VALUE (see Section 9 on page 745). For purposes of matching, VALUE is interpreted as an enumerated identifier value with the identifier text form given by the Ada attribute operation


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

ENUMERATION'IMAGE (VALUE)

and the enumeration position given by the Ada attribute operation

ENUMERATION'POS (VALUE).

An ordinary identifier item matches VALUE if the text form of the item is the same as the image of VALUE (under identifier equality). An enumerated identifier item matches VALUE if the text form of the item is the same as the image of VALUE (under identifier equality) and the enumeration position of the item is the same as that of VALUE. 2

The search begins at the START_POSITION and ends when either an item whose value matches VALUE is found, the last item of the list has been examined, or the item at the END_POSITION has been examined, whichever comes fist. 3

Parameters:

**LIST** is the list in whose current linear list the item of interest is to be located. 4

**VALUE** is the enumeration value of interest. 5

**START_POSITION** is the position of the first item in the current linear list to be considered in the search. 6

**END_POSITION** is the position beyond which the search will not proceed; the search may terminate prior to reaching END_ POSITION should the sought item be found or should the last element of the list be considered. 7

**STATUS** is an established status indicator. 8

Exceptions:

**STATUS_ERROR** is raised if STATUS is not an established status indicator [3]. 9

**LIST_KIND_ERROR** is raised if the current linear list of LIST is empty [1]. 10

**LIST_POSITION_ERROR** is raised if START_POSITION specifies a value larger than the current length of the current linear list of LIST [1], or if END_POSITION is less than START_POSITION [2]. 11

**SEARCH_ERROR** is raised if there is no item in the current linear list of LIST within the region specified by START_POSITION and END_POSITION that matches the value specified by VALUE [2]. 12

## 9.1.24 Package CAIS_BOOLEAN_ITEM

This is a library package for manipulating identifier item values which have the identifier text forms TRUE and FALSE using the values of the Ada enumeration type BOOLEAN as the external enumeration form. The exceptions raised by all subprograms in this package are defined in the packages CAIS_LIST_MANAGEMENT and CAIS_PRAGMATICS. 1

The specification and semantics of this package are the same as the following instantiation of package CAIS_ENUMERATION_ITEM (see Section 9.1.23 on page 780) but need not be implemented as such an instantiation: 2


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
package CAIS_BOOLEAN_ITEM is
    new CAIS_ENUMERATION_ITEM (ENUMERATION => BOOLEAN);
```

Note that the package CAIS_BOOLEAN_ITEM is instantiated as a library package rather than as a sub-package of CAIS_LIST_MANAGEMENT. 3

Identifier items created or set using this package are enumerated identifier items whose enumeration positions are the positions of corresponding values in the Ada enumeration type BOOLEAN. 4

## 9.1.25 Generic package CAIS_INTEGER_ITEM

This is a generic package for manipulating list items whose values are integers. This package must be instantiated for the appropriate integer type (indicated by NUMBER in the specification). The exceptions raised by all subprograms in this package are defined in the packages CAIS_LIST_MANAGEMENT and CAIS_PRAGMATICS. 1

```
generic
    type NUMBER is rang. <>;
package CAIS_INTEGER_ITEM is
    -- Specifications of subprograms for this generic package.
end CAIS_INTEGER_ITEM;
```

### 9.1.25.1 Converting an integer value to its canonical text representation

```
function TEXT_FORM
    (INTEGER_VALUE : in     NUMBER;
     STATUS        : in     STATUS_INDICATOR := NO_INDICATION)
    return STRING;
```

Purpose:

This function returns the canonical text form representation of the value of the INTEGER_VALUE parameter. The canonical text form representation is the string representation defined in Section 9 on page 745. 1

Parameters:

**INTEGER_VALUE** is the integer value whose external representation is to be returned. 2

**STATUS** is an established status indicator. 3

Exceptions:

**STATUS_ERROR** is raised if STATUS is not an established status indicator [3].



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

## 9.1.25.2 Extracting an integer value from a list item

```
function EXTRACTED_VALUE
    (FROM_LIST     : in  LIST_TYPE;
     ITEM_POSITION : in  POSITION_COUNT;
     STATUS        : in  STATUS_INDICATOR := NO_INDICATION)
    return NUMBER;

function EXTRACTED_VALUE
    (FROM_LIST : in  LIST_TYPE;
     ITEM_NAME : in  IDENTIFIER_TEKT ;
     STATUS    : in  STATUS_INDICATOR := NO_INDICATION)
    return NUMBER;

function EXTRACTED_VALUE
    (FROM_LIST : in  LIST_TYPE;
     ITEM-NAME : in  TOKEN_TYPE;
     STATUS    : in  STATUS_INDICATOR := NO_INDICATION)
    return NUMBER;
```

Purpose:

This function locates an integer-valued item in the current linear list of FROM_LIST and returns a copy of its numeric value. The list FROM_LIST is not modified. 1

Parameters:

**FROM_LIST** is the list whose current linear list contains the item whose value is to be extracted. 2

**ITEM_POSITION** is the position within the current linear list that identifies the item whose value is to be extracted. 3

**ITEM_NAME** is the name of the item whose value is to be extracted. 4

**STATUS** is an established status indicator. 6

Exceptions:

**STATUS_ERROR** is raised if STATUS is not unestablished status indicator [3]. 6

**LIST_KIND_ERROR** is raised if the current linear list of FROM_LIST is empty [l]. 7

**LIST_POSITION_ERROR** is raised if ITEM_POSITION has a value larger than the current length of the current linear list of FROM_LIST [1]. 8

**SYNTAX_ERROR** is raised if the value of the parameter ITEM_NAME of type IDENTIFIER_TEXT does not conform to the syntax of an Ada identifier [2]. 9

**TOKEN_ERROR** is raised if the ITEM_NAME of type TOKEN_TYPE is an undefined token [1], 10

**NAMED_LIST_ERROR** is raised if the parameter ITEM_NAME is used and the current linear list of FROM_LIST is unnamed [2]. 11



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

**ITEM_KIND_ERROR** is raised if ITEM_POSITION or ITEM_NAME specifies an item whose value is not an integer [1]. 12

**CONSTRAINT_ERROR** is raised if the value to be extracted violates the constraints of the type designated by NUMBER. 13

**SEARCH_ERROR** is raised if there is no item in the current linear list of FROM_LIST with the name ITEM_NAME [1]. 14

## 9.1.26.3 Replacing an integer value in a list item

```
procedure REPLACE
    (IN_LIST       : in out LIST_TYPE;
     ITEM_POSITION : in     POSITION_COUNT;
     VALUE         : in     NUMBER;
     STATUS        : in     STATUS_INDICATOR := NO_INDICATION);

procedure REPLACE
    (IN_LIST    : in out LIST_TYPE;
     ITEM_ITEM  : in     IDENTIFIER_TEXT;
     VALUE      : in     NUMBER;
     STATUS     : in     STATUS_INDICATOR := NO_INDICATION);

procedure REPLACE
    (IN_LIST    : in out LIS_-TYPE;
     ITEM_NAME  : in     TOKEN_TYPE;
     VALUE      : in     NUMBER;
     STATUS     : in     STATUS_INDICATOR := NO_INDICATION);
```

Purpose:

This procedure replaces the value of an integer-valued item in the current linear list of IN_LIST with VALUE. Subsequent modification to the value of IN_LIST or of VALUE does not affect the other value. 1

Parameters:

**IN_LIST** is the list whose current linear list contains the item whose value is to be replaced. 2

**ITEM_POSITION** is the position within the current linear list that identifies the item whose value is to be replaced. 3

**ITEM_NAME** is the name of the item whose value is to be replaced. 4

**VALUE** is the new integer value of the designated item. 5

**STATUS** is an established status indicator. 6

Exceptions:

**STATUS_ERROR** is raised if STATUS is not an established status indicator [3].



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

**LIST_KIND_ERROR** is raised if the current linear list of IN_LIST is empty [1]. 8

**LIST_POSITION_ERROR** is raised if ITEM_POSITION has a value larger than the current length of the current linear list of IN_LIST [I]. 9

**SYNTAX_ERROR** is raised if the value of the parameter ITEM_NAME of type IDENTIFIER_TEXT does not conform to the syntax of an Ada identifier [2]. 10

**TOKEN_ERROR** is raised if the ITEM_NAME of type TOKEN_TYPE is an undefined token [1]. 11

**CAPACITY_ERROR** is raised if the VALUE violates the constraints defined by the type CAIS_INTEGER (see Section 5.1 on page 88). 12

**NAMED_LIST_ERROR** is raised if the parameter ITEM_NAME is used and the current linear list of IN_LIST is unnamed [2]. 13

**ITEM_KIND_ERROR** is raised if ITEM_POSITION or ITEM_NAME specifies an item whose value is not an integer [1]. 14

**SEARCH_ERROR** is raised if there is no item in the current linear list of IN_LIST with the name ITEM_NAME [1]. 15

## 9.1.25.4 Inserting an integer-valued item into a linear list

```
procedure INSERT
   (IN_LIST : in out LIST_TYPE;
    POSITION : in     INSERT_COUNT;
    VALUE    : in     NUMBER;
    STATUS   : in     STATUS_INDICATOR := NO_INDICATION);

procedure INSERT
   (IN_LIST : in out LIST_TYPE;
    POSITION : in     INSERT_COUNT;
    NAME     : in     IDENTIFIER_TEXT;
    VALUE    : in     NUMBER;
    STATUS   : in     STATUS_INDICATOR : = NO_INDICATION);

procedure INSERT
   (IN_LIST : in out LIST_TYPE;
    POSITION : in     INSERT_COUNT;
    NAME     : in     TOKEN_TYPE;
    VALUE    : in     NUMBER;
    STATUS   : in     STATUS_INDICATOR := NO_INDICATION);
```

Purpose:

This procedure inserts an integer-valued item into the current linear list of IN_LIST; the new list item will be positioned after the list item specified by POSITION. A value of zero in POSITION specifies a position at the head of the current linear list. Subsequent modification to the value of IN_LIST or of VALUE does not affect the other value. If the current linear list of IN_LIST is empty, it will be a named list after the successful completion of the call if the second or third interface is used, and an unnamed list otherwise. 1


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Parameters:

**IN_LIST** is the list into whose current linear list the item will be inserted. 2

**POSITION** is the position in the current linear list after which the item is to be inserted. 3

**NAME** is the name of the new item to be inserted. 4

**VALUE** is the integer value of the new item to be inserted. 5

**STATUS** is an established status indicator. 6

Exceptions:

**STATUS_ERROR** is raised if STATUS is not an established status indicator [3]. 7

**LIST_KIND_ERROR** is raised if an attempt is made to insert an item by NAME into an unnamed list [4] or, conversely, if an attempt is made to insert an item without NAME into a named list [5]. 8

**LIST_POSITION_ERROR** is raised if ITEM_POSITION has a value larger than the current length of the current linear list of IN_LIST [1]. 9

**SYNTAX_ERROR** is raised if the value of the parameter NAME of type IDENTIFIER_TEXT does not conform to the syntax of an Ada identifier [2]. 10

**TOKEN_ERROR** is raised if the NAME of type TOKEN_TYPE is an undefined token [1]. 11

**CAPACITY_ERROR** is raised if the number of items in the resulting linear list would exceed the value of the constant CAIS_PRAGMATICS.LIST_LENGTH or if the VALUE to be inserted violates the constraints defined by the type CAIS_INTEGER (see Section 5.1 on page 88) [1]. 12

**NAMED_LIST_ERROR** is raised if the current linear list of IN_LIST is a named list that already contains an item with the name given by NAME [4]. 13

## 9.1.25.6 Locating an integer-valued item by value within a linear list

```
function POSITION_BY_VALUE
    (LIST           : in  LIST_TYPE;
     VALUE          : in  NUMBER;
     START_POSITION : in  POSITION_COUNT     := POSITION_COUNT'FIRST;
     END_POSITION   : in  POSITION_COUNT     : = POSITION_COUNT'LAST;
     STATUS         : in  STATUS_INDICATOR   : = NO_INDICATION)
   return POSITION_COUNT;
```

Purpose:

This function returns the position in the current linear list of LIST of the next integer-valued item whose value equals that of the VALUE parameter under integer equality (see Section 9 on page 745). The search begins at the START_POSITION and ends when either an item whose value equals VALUE is found, the last item of the list has been examined, or the item at the END_POSITION has been examined, whichever comes fist. 1



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Parameters:

**LIST** is the list in whose current linear list the item of interest is to be located.

**VALUE** is the integer value of interest.

**START_POSITION** is the position of the first item in the current linear list to be considered in the search.

**END_POSITION** is the position beyond which the search will not proceed; the search may terminate prior to reaching END_POSITION should the sought item be found or should the last element of the list be considered.

**STATUS** is an established status indicator.

Exceptions:

**STATUS_ERROR** is raised if STATUS is not an established status indicator [3].

**LIST_KIND_ERROR** is raised if the current linear list of LIST is empty [1].

**LIST_POSITION_ERROR** is raised if START_POSITION specifies a value larger than the current length of the current linear list of LIST [1], or if END_POSITION is less than START_POSITION [2].

**CAPACITY_ERROR** is raised if the VALUE violates the constraints defined by the type CAIS_INTEGER (see Section 5.1 on page 88) [1].

**SEARCH_ERROR** is raised if there is no item in the current linear list of LIST within the region specified by START_POSITION and END_POSITION that has the value specified by VALUE [2].

## 9.1.26 Generic package CAIS_FLOAT_ITEM

This is a generic package for manipulating list items whose values are floating point numbers. This package must be instantiated for the appropriate type (indicated by NUMBER in the specification). The exceptions raised by all subprograms in this package are defined in the packages CAIS_LIST_-MANAGEMENT and CAIS_PRAGMATICS.

```
generic
    type NUMBER is digits <>;
package CAIS_FLOAT_ITEM is
    -- Specifications of subprograms for this generic package.
and CAIS_FLOAT_ITEM;
```

Use of floating point values in lists has certain adverse consequences for the meaning of list equality as detailed in Section 9 on page 745. Users should be aware of the accuracy issues for relational operations between floating point values explained in [1815A], Section 4.5.7, in order to avoid erroneous assumptions about the equality of float items and lists involving such items.

See also the discussion of floating point values in Section 9 on page 745, for futher cautions.



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

## 9.1.26.1 converting a floating point value to its canonical text form

```
function TEXT_FORM
    (FLOAT_VALUE : in     NUMBER;
     STATUS      : in     STATUS_INDICATOR := NO_INDICATION)
    return STRING;
```

Purpose:

This function returns the canonical text form representation of the value of the FLOAT_VALUE parameter. The canonical text form representation is the string representation defied in Sectional on page 745. 1

Parameters:

**FLOAT_VALUE** is the floating point item whose external representation is to be returned. 2

**STATUS** is an established status indicator. 3

Exceptions:

**STATUS_ERROR** is raised if STATUS is not an established status indicator [3].

## 9.1.26.2 Extracting a floating point value from a list item

```
function EXTRACTED_VALUE
    (FROM_LIST     : in     LIST_TYPE;
     ITEM_POSITION : in     POSITION_COUNT;
     STATUS        : in     STATUS_INDICATOR := NO_INDICATION)
    return NUMBER;

function EXTRACTED_VALUE
    (FROM_LIST : in     LIST_TYPE;
     ITEM_NAME : in     IDENTIFIER_TEXT;
     STATUS    : in     STATUS_INDICATOR := NO_INDICATION)
    return NUMBER;

function EXTRACTED_VALUE
    (FROM_LIST : in     LIST_TYPE;
     ITEM_NAME : in     TOKEN_TYPE;
     STATUS    : in     STATUS_INDICATOR := NO_INDICATION)
    return NUMBER;
```

Purpose:

This function locates a floating point-valued item in the current linear list of FROM_LIST and returns a copy of its numeric value. The list FROM_LIST is not modified. 1



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Parameters:

**FROM_LIST** is the list whose current linear list contains the item whose value is to be extracted. 2

**ITEM_POSITION** is the position within the current linear list that identifies the item whose value is to be extracted. 3

**ITEM_NAME** is the name of the item whose value is to be extracted. 4

**STATUS** is an established status indicator. 6

Exceptions:

**STATUS_ERROR** is raised if STATUS is not an established status indicator [3]. 6

**LIST_KIND_ERROR** is raised if the current linear list of FROM_LIST is empty. 7

**LIST_POSITION_ERROR** is raised if ITEM_POSITION has a value larger than the current length of the current linear list of FROM_LIST [1]. 6

**SYNTAX_ERROR** is raised if the value of the parameter ITEM_NAME of type IDENTIFIER_TEXT does not conform to the syntax of an Ada identifier [2]. 9

**TOKEN_ERROR** is raised if the ITEM_NAME of type TOKEN_TYPE is an undefined token [1]. 10

**NAMED_LIST_ERROR** is raised if the parameter ITEM_NAME is used and the current linear list of FROM_LIST is unnamed [2]. 11

**ITEM_KIND_ERROR** is raised if ITEM_POSITION or ITEM_NAME specifies an item whose value is not a floating point number [1]. 12

**CONSTRAINT_ERROR** is raised if the value to be extracted violates the range constraints of the type designated by NUMBER. 13

**SEARCH_ERROR** is raised if there is no item in the current linear list of FROM_LIST with the name ITEM_NAME [1]. 14

## 9.1.26.3 Replacing a floating point value in a list item

```
procedure REPLACE
    (IN_LIST       : in out LIST_TYPE;
     ITEM_POSITION : in     POSITION_COUNT;
     VALUE         : in     NUMBER;
     STATUS        : in     STATUS-INDICATOR  : = NO_INDICATION);

procedure REPLACE
    (IN_LIST     : in out LIST_TYPE;
     ITEM_NAME : in     IDENTIFIER_TEXT;
     VALUE       : in     NUMBER;
     STATUS      : in     STATUS- INDICATOR  : = NO_INDICATIOH) ;

procedure REPLACE
    (IN_LIST     : in out LIST_TYPE;
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
    ITEM_NAME : in     TOKEN_TYPE;
    VALUE     : in     NUMBER;
    STATUS    : in     STATUS_INDICATOR := NO_INDICATION);
```

Purpose:

This procedure replaces the value of a floating point-valued item in the current linear list of IN_LIST with VALUE. Subsequent modification to the value of IN_LIST or of VALUE does not affect the other value.

Parameters:

**IN_LIST** is the list whose current linear list contains the item whose value is to be replaced.

**ITEM_POSITION** is the position within the current linear list that identifies the item whose value is to be replaced.

**ITEM_NAME** is the name of the item whose value is to be replaced.

**VALUE** is the new floating point value of the designated item.

**STATUS** is an established status indicator.

Exceptions:

**STATUS_ERROR** is raised if STATUS is not an established status indicator [3].

**LIST_KIND_ERROR** is raised if the current linear list of IN_LIST is empty [1].

**LIST_POSITION_ERROR** is raised if ITEM_POSITION has a value larger than the current length of the current Linear list of IN_LIST [1].

**SYNTAX_ERROR** is raised if the value of the parameter ITEM_NAME of type IDENTIFIER_TEXT does not conform to the syntax of an Ada identifier [2].

**TOKEN_ERROR** is raised if the ITEM_NAME of type TOKEN_TYPE is an undefined token [1].

**CAPACITY_ERROR** is raised if the VALUE (see note below) violates the range constraints of the CAIS implementation [1].

**NAMED_LIST_ERROR** is raised if the parameter ITEM_NAME is used and the current linear list of IN_LIST is unnamed [2].

**ITEM_KIND_ERROR** is raised if ITEM_POSITION or ITEM_NAME specifies an item whose value is not a floating point number [1].

**SEARCH_ERROR** is raised if there is no item in the current linear list of IN_LIST with the name ITEM_NAME [1].

Notes:

The most restrictive range constraints applicable to VALUE can be inferred from CAIS_PRAGMATICS.LIST_MAXIMUM_DIGITS according to rules defined in [1815A] 3.5.9, 4.5.7 and 4.6. (See CAIS_PRAGMATICS, Section 12 on page 815.)



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

## 9.1.26.4 Inserting a floating point-valued item into a linear list

```
procedure INSERT
    (IN_LIST   : in out LIST_TYPE;
     POSITION  : in     INSERT_COUNT;
     VALUE     : in     NUMBER;
     STATUS    : in     STATUS_INDICATOR := NO_INDICATION);

procedure INSERT
    (IN_LIST   : in out LIST_TYPE;
     POSITION  : in     INSERT_COUNT;
     NAME      : in     IDENTIFIER_TEXT;
     VALUE     : in     NUMBER;
     STATUS    : in     STATUS_INDICATOR := NO_INDICATION);

procedure INSERT
    (IN_LIST   : in out LIST_TYPE;
     POSITION  : in     INSERT_COUNT;
     NAME      : in     TOKEN_TYPE;
     VALUE     : in     NUMBER;
     STATUS    : in     STATUS_INDICATOR := NO_INDICATION);
```

Purpose:

This procedure inserts a floating point-valued item into the current linear list of IN_LIST; the new list item will be positioned after the list item specified by POSITION. A value of zero in POSITION specifies a position at the head of the current linear list. Subsequent modification to the value of IN_LIST or of VALUE does not affect the other value. If the current linear list of IN_LIST is empty, it will be a named list after the successful completion of the call, if the second or third interface is used, and an unnamed list otherwise. 1

Parameters:

**IN_LIST** is the list into whose current linear list the item will reinserted. 2

**POSITION** is the position in the current linear list after which the item is to be inserted. 3

**NAME** is the name of the new item to be inserted. 4

**VALUE** is the floating point value of the new item to reinserted. 5

**STATUS** is an established status indicator. 6

Exceptions:

**STATUS_ERROR** is raised if STATUS is not an established status indicator [3]. 7

**LIST_KIND_ERROR** is raised if an attempt is made to insert an item by NAME in to an unnamed list [4] or, conversely, if an attempt is made to insert an item without NAME into a named list [5]. 8



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

**LIST_POSITION_ERROR** is raised if ITEM_POSITION has a value larger than the current length of the current linear list of IN_LIST [1]. 9

**SYNTAX_ERROR** is raised if the value of the parameter NAME of type IDENTIFIER_TEXT does not conform to the syntax of an Ada identifier [2]. 10

**TOKEN_ERROR** is raised if the NAME of type TOKEN_TYPE is an undefined token [1]. 11

**CAPACITY_ERROR** is raised if the number of items in the resulting linear list would exceed the value of the constant CAIS_PRAGMATICS.LIST_LENGTH or if the VALUE to be inserted violates the range constraints of the CAIS implementation [1]. (See the note below.) 12

**NAMED_LIST_ERROR** is raised if the current linear list of IN_LIST is a named list that already contains an item with the name given by NAME [4]. 13

Notes:

The most restrictive range constraints applicable to VALUE can be inferred from the value of the constant CAIS_PRAGMATICS.LIST_MAXIMUM_DIGITS according to rules defined in [1815A] 3.5.9, 4.5.7 and 4.6. (See CAIS_PRAGMATICS, Section 12 on page 815.) 14

## 9.1.26.6 Locating a floating point-valued item by value within a linear list

```
function POSITION_BY_VALUE
    (LIST           : in  LIST_TYPE;
     VALUE          : in  NUMBER;
     START_POSITION : in  POSITION_COUNT     : = POSITION_COUNT'FIRST;
     END_POSITION   : in  POSITION_COUNT     := POSITION_COUNT'LAST;
     STATUS         : in  STATUS_INDICATOR   := NO_INDICATION)
    return POSITION_COUNT;
```

Purpose:

This function returns the position in the current linear list of LIST of the next floating point-valued item whose value equals that oft he VALUE parameter under floating point equality (see Section 9 on page 745). The search begins at the START_POSITION and ends when either an item whose value equals VALUE is found, the last item of the list has been examined, or the item at the END_POSITION has been examined, whichever comes fist. 1

Parameters:

**LIST** is the list in whose current linear list the item of interest is to be located. 2

**VALUE** is the floating point value of interest. 3

**START_POSITION** is the position of the fist item in the current linear list to be considered in the search. 4

**END_POSITION** is the position beyond which the search will not proceed; the search may terminate prior to reaching END_POSITION should the sought item be found or should the last element of the list be considered. 5

**STATUS** is an established status indicator. 6



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Exceptions:

**STATUS_ERROR** is raised if STATUS is not an established status indicator [3]. 7

**LIST_KIND_ERROR** is raised if the current linear list of LIST is empty [1]. 8

**LIST_POSITION_ERROR** is raised if START_POSITION specifies a value larger than the current length of the current linear list of LIST [1], or if END_POSITION is less than START_POSITION [2]. 9

**CAPACITY_ERROR** is raised if the VALUE violates the range constraints of the CAIS implementation [1]. (See the note below.) 10

**SEARCH_ERROR** is raised if there is no item in the current linear list of LIST within the region specified by START_POSITION and END_POSITION that has the value specified by VALUE [2]. 11

Notes:

Determining the position by value of floating point items should be applied with considerable caution and awareness of all the issues documented in [1815A], Section 4.5.7, regarding the accuracy of relational operations with real operands, and the discussion, in Section 9 on page 745, defining floating point equality for floating point list items. 12

The most restrictive range constraints applicable to VALUE can be inferred from the value of the constant CAIS_PRAGMATICS.LIST_MAXIMUM_DIGITS according to rules defined in [1815A] 3.5.9, 4.5.7 and 4.6. (See CAIS_PRAGMATICS, Section 12 on page 815.) 13

## 9.1.27 Package CAIS_STRING_ITEM

This is a package for manipulating list items whose values are strings. The exceptions raised by all subprograms in this package are defined in the packages CAIS_LIST_MANAGEMENT and CAIS_-PRAGMATICS. 1

## 9.1.27.1 Extracting a string value from a list item

```
function EXTRACTED_VALUE
    (FROM_LIST     : in    LIS_-TYPE;
     ITEM_POSITION : in    POSITION_COUNT;
     STATUS        : in    STATUS_INDICATOR := NO_INDICATION)
    return STRING;

function EXTRACTED_VALUE
    (FROM_LIST : in    LIST_TYPE;
     ITEM_NAME : in    IDENTIFIER_TEXT ;
     STATUS    : in    STATUS_INDICATOR : = NO_INDICATION)
    return STRING;

function EXTRACTED_VALUE
    (FROM_LIST : in    LIST_TYPE;
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
    ITEM_NAME : in     TOKEN_TYPE;
    STATUS    : in     STATUS_INDICATOR := NO_INDICATION)
  return STRING;
```

Purpose:

This function locates a string-valued item in the current linear list of FROM_LIST and returns a copy of its string value. The list FROM_LIST is not modified. 1

Parameters:

**FROM_LIST** is the list whose current linear list contains the item to be extracted. 2

**ITEM_POSITION** is the position within the current linear list that identifies the item whose value is to be extracted. 3

**ITEM_NAME** is the name of the item whose value is to be extracted. 4

**STATUS** is an established status indicator. 6

Exceptions:

**STATUS_ERROR** is raised if STATUS is not an established status indicator [3]. 6

**LIST_KIND_ERROR** is raised if the current linear list of FROM_LIST is empty [1]. 7

**LIST_POSITION_ERROR** is raised if ITEM_POSITION has a value larger than the current length of the current linear list of FROM_LIST [1]. 8

**SYNTAX_ERROR** is raised if the value of the parameter ITEM_NAME of type IDENTIFIER_TEXT does not conform to the syntax of an Ada identifier [2]. 9

**TOKEN_ERROR** is raised if the ITEM_NAME of type TOKEN_TYPE is an undefined token [1]. 10

**NAMED_LIST_ERROR** is raised if the parameter ITEM_NAME is used and the current linear list of FROM_LIST is unnamed [2]. 11

**SEARCH_ERROR** is raised if there is no item in the current linear list of FROM_LIST with the name ITEM_NAME [1]. 12

**ITEM_KIND_ERROR** is raised if ITEM_POSITION or ITEM_NAME specifies an item whose value is not a string [1]. 13

## 9.1.27.2 Replacing a string value in a list item

```
procedure REPLACE
    (IN_LIST       : in out LIST_TYPE;
     ITEM_POSITION : in     POSITION_COUNT;
     VALUE         : in     STRING;
     STATUS        : in     STATUS_INDICATOR := NO_INDICATION);

procedure REPLACE
    (IN_LIST    : in out LIST_TYPE;
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
        ITEM_NAME : in      IDENTIFIER_TEXT;
        VALUE     : in      STRING;
        STATUS    : in      STATUS_INDICATOR := NO_INDICATION);

    procedure REPLACE
       (IN_LIST   : in out LIST_TYPE;
        ITEM_NAME : in      TOKEN_TYPE;
        VALUE     : in      STRING;
        STATUS    : in      STATUS_INDICATOR := NO_INDICATION);
```

Purpose:

This procedure replaces the value of a string-valued item in the current linear list of IN_LIST with VALUE. Subsequent modification to the value of IN_LIST or of VALUE does not affect the other value. 1

Parameters:

**IN_LIST** is the list whose current linear list contains the item whose value is to be replaced. 2

**ITEM_POSITION** is the position within the current linear list that identifies the item whose value is to be replaced. 3

**ITEM_NAME** is the name of the item whose value is to be replaced. 4

**VALUE** is the new string value of the designated item. 5

**STATUS** is an established status indicator. 6

Exceptions:

**STATUS_ERROR** is raised if STATUS is not an established status indicator [3]. 7

**LIST_KIND_ERROR** is raised if the current linear list of IN_LIST is empty [1]. 8

**LIST_POSITION_ERROR** is raised if ITEM_POSITION has a value larger than the current length of the current linear list of IN_LIST [1]. 9

**SYNTAX_ERROR** is raised if the value of the parameter ITEM_NAME of type IDENTIFIER_TEXT does not conform to the syntax of an Ada identifier [2]. 10

**TOKEN_ERROR** is raised if the ITEM_NAME of type TOKEN_TYPE is an undefined token [1]. 11

**CAPACITY_ERROR** is raised if the string value of VALUE is longer than the value of the constant CAIS_PRAGMATICS.STRING_ITEM_LENGTH [1] (see Section 12 on page 815.) 12

**NAMED_LIST_ERROR** is raised if the parameter ITEM_NAME is used and the current linear list of IN_LIST is unnamed [2]. 13

**ITEM_KIND_ERROR** is raised if ITEM_POSITION or ITEM_NAME specifies an item whose value is not a string [1]. 14

**SEARCH_ERROR** is raised if there is no item in the current linear list of IN_LIST with the name ITEM_NAME [1]. 15



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

### 9.1.27.3 Inserting a string-valued item into a linear list

```
procedure INSERT
    (IN_LIST  : in out LIST_TYPE;
     POSITION : in     INSERT_COUNT;
     VALUE    : in     STRING;
     STATUS   : in     STATUS_INDICATOR := NO_INDICATION);

procedure INSERT
    (IN_LIST  : in out LIST_TYPE;
     POSITION : in     INSERT_COUNT;
     NAME     : in     IDENTIFIER_TEXT;
     VALUE    : in     STRING;
     STATUS   : in     STATUS_INDICATOR := NO_INDICATION);

procedure INSERT
    (IN_LIST  : in out LIST_TYPE;
     POSITION : in     INSERT_COUNT;
     NAME     : in     TOKEN_TYPE;
     VALUE    : in     STRING;
     STATUS   : in     STATUS_INDICATOR := NO_INDICATION);
```

Purpose:

This procedure inserts a string-valued item into the current linear list of IN_LIST; the new list item will be positioned after the list item specified by POSITION. A value of zero imposition specifies a position at the head of the current linear list. Subsequent modification to the value of IN_LIST or of VALUE does not affect the other value. If the current linear list of IN_LIST is empty, it will be a named list after the successful completion of the call, if the second or third interfaces used, and an unnamed list otherwise. 1

Parameters:

**IN_LIST** is the list into whose current linear list the item will be inserted. 2

**POSITION** is the position in the current linear list after which the item is to be inserted. 3

**NAME** is the name of the new item to be inserted. 4

**VALUE** is the string value of the new item to reinserted. 5

**STATUS** is an established status indicator. 6

Exceptions:

**STATUS_ERROR** is raised if STATUS is not an established status indicator [3]. 7

**LIST_KIND_ERROR** is raised if an attempt is made to insert an item by NAME into an unnamed list [4] or, conversely, if an attempt is made to insert an item without NAME into a named list [5]. 8


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

**LIST_POSITION_ERROR** is raised if ITEM_POSITION has a value larger than the current length of the current linear list of IN_LIST [1]. 9

**SYNTAX_ERROR** is raised if the value of the parameter NAME of type IDENTIFIER_TEXT does not conform to the syntax of an Ada identifier [2]. 10

**TOKEN_ERROR** is raised if the NAME of type TOKEN_TYPE is an undefined token [1]. 11

**CAPACITY_ERROR** is raised if the string value of VALUE to be inserted is longer than the value of the constant CAIS_PRAGMATICS.STRING_ITEM_LENGTH or if the number of items in the resulting linear list would exceed the value of the constant CAIS_PRAGMATICS.LIST_LENGTH (see Section 12 on page 815) [1]. 12

**NAMED_LIST_ERROR** is raised if the current linear list of IN_LIST is a named list that already contains an item with the name given by NAME [4]. 13

## 9.1.27.4 Locating a string-valued item by value within a linear list

```
function POSITIOM_BY_VALUE
   (LIST           : in  LIST_TYPE;
    VALUE          : in  STRING;
    START_POSITION : in  POSITION_COUNT    := POSITION_COUNT'FIRST;
    END_POSITION   : in  POSITION_COUNT    := POSITION_COUNT'LAST;
    STATUS         : in  STATUS_INDICATOR  := NO_INDICATION)
   return POSITION_COUNT;
```

Purpose:

This function returns the position in the current linear list of LIST of the next string-valued item whose value equals that of the VALUE parameter under string equality (see Section 9 on page 745). The search begins at the START-POSITION and ends when either an item whose value equals VALUE is found, the last item of the list has been examined, or the item at the END_POSITION has been examined, whichever comes first. 1

Parameters:

**LIST** is the list in whose current linear list the item of interest is to be located. 2

**VALUE** is the string value of interest. 3

**START_POSITION** is the position of the fist item in the current linear list to be considered in the search. 4

**END_POSITION** is the position beyond which the search will not proceed; the search may terminate prior to reaching END_POSITION should the sought item be found or should the last element of the list be considered. 5

**STATUS** is established status indicator. 6

Exceptions:

**STATUS_ERROR** is raised if STATUS is not an established status indicator [3].



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

**LIST_KIND_ERROR** is raised if the current linear list of LIST is empty [1]. 8

**LIST_POSITION_ERROR** is raised if START_POSITION specifies a value larger than the current length of the current linear list of LIST [1], or if END_POSITION is less than START_POSITION [2]. 9

**CAPACITY_ERROR** is raised if the string value of VALUE is longer than the value of the constant CAIS_PRAGMATICS.STRING_ITEM_LENGTH (see Section 12 on page 815) [1]. 10

**SEARCH_ERROR** is raised if there is no item in the current linear list of LIST within the region specified by START_POSITION and END_POSITION that has the value specified by VALUE [2]. 11



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Source: https://assist.dla.mil -- Downloaded: 202610-26T07:54Z
Check the source to verify that this is the current version before use.

# 10. CAIS STANDARD DETAILED REQUIREMENTS

This package contains certain scalar types predefined in the CAIS. The intent of providing this package is to make these types reasonably independent of any predefined types in the Ada language, whose characteristics may vary among compilers. 1

## 10.1 Package CAIS_STANDARD

```
type      CAIS_INTEGER is range
      CAIS_PRAGMATICS.MINIMUM_INTEGER . . CAIS_PRAGMATICS.MAXIMUM.INTEGER;
subtype CAIS_NATURUL is CAIS_INTEGER range 0. . CAIS_INTEGER'LAST;
subtype CAIS_POSITIVE is CAIS_INTEGER range 1 . . CAIS_INTEGER'LAST;
```

CAIS_INTEGER is a CAIS-defined type in analogy to the Ada type INTEGER. CAIS_NATURAL and CAIS_POSITIVE are CAIS-defined subtypes in analogy to the Ada subtypes NATURAL and POSITIVE, respectively. 1

```
type CAIS_DURATION is delta implementation-defined;
for CAIS_DURATION'SMALL use CAIS_PRAGMATICS.SMALL_FOR_CAIS_DURATION;
```

For the CAIS, an implementation of the type CAIS_DURATION must allow representation of durations (both positive and negative) up to at least 86400 seconds (one day). The smallest representable duration, CAIS_DURATION'SMALL must equal CAIS_PRAGMATICS. SMALL_FOR_CAIS_DURATION. 2



Source: https://assist.dla.mil -- Downloaded: 202610-26T07:54Z
Check the source to verify that this is the current version before use.

Source: https://assist.dla.mil -- Downloaded: 202610-26T07:54Z
Check the source to verify that this is the current version before use.

# 11. CAIS_CALENDAR DETAILED REQUIREMENTS

This package provides facilities for accessing a system clock and interpreting its values. It is semantically almost identical to the package CALENDAR in [1815A], Section 9.6. The differences relate to the use of the types CAIS_INTEGER and CAIS_DURATION in lieu of the corresponding Ada predefine types. 1

For the CAIS, an implementation of the type CAIS_DURATION must allow representation of durations (both positive and negative) up to at least 86400 seconds (one day). The smallest representable duration, CAIS_DURATION'SMALL, must equal CAIS_PRAGMATICS.SMALL_FOR_CAIS_DURATION (see Section 12 on page 815). 2

The meaning of the values of type CAIS_CALENDAR.TIME is implementation-dependent, as these values will usually be obtained from the underlying system clock. In particular, the values need not be synchronized to some standard time, such as Greenwich Mean Time. Prior to comparing two values of type CAIS_CALENDAR.TIME obtained by two different calls on CAIS_CALENDAR.CLOCK, the user should consult Appendix F of the respective CAIS implementations to determine and account for the implementation dependencies. 3

## 11.1 Package CAIS_CALENDAR

### 11.1.1 Types, subtypes, and exceptions

```
type TIME is private;
```

TIME is the type for the implementation-dependent time; values of type TIME must be able to be decomposed into values of the subtypes YEAR_NUMBER, MONTH_NUMBER, DAY_NUMBER and DAY_DURATION. 1

```
subtype YEAR_NUMBER is CAIS_INTEGER range 1901 . . 2099;
subtype MONTH_NUMBER is CAIS_INTEGER range 1 . . 12;
subtype DAY_NUMBER    is CAIS_INTEGER range 1 . . 31;
subtype DAY_DURATION is CAIS_DURATION range 0.0 . . 86-400.0;
```

YEAR_NUMBER, MONTH_NUMBER and DAY_NUMBER are subtypes for the year, month and day, respectively, of a time. DAY_DURATION is the type which identifies the second within the day. 2

A proper time is a time that is formed from these types, in particular, the year number must be in the range of the subtype YEAR_NUMBER. 3

```
TIME_ERROR: exception;
```

**TIME-ERROR** is raised if a proper time cannot be formed by the functions defined in this package or the operator "-" cannot return a result that is in the range of the type CAIS_DURATION. 4

Other exceptions raised by the subprograms in this package are defined in the packages CAIS_-DEFINITIONS and CAIS_PRAGMATICS. 5



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

## 11.1.2 Getting the current time

```
function CLOCK
    (STATUS : in     STATUS_INDICATOR := NO_INDICATION)
    return TIME;
```

Purpose:

This function returns a value of CAIS_CALENDAR.TIME, representing the implementation-dependent time at which the interface was called. 1

Parameters:

**STATUS** is an established status indicator.

Exceptions:

**STATUS_ERROR** is raised if STATUS is not an established status indicator [3].

## 11.1.3 Getting the year part of the time

```
function YEAR
    (DATE   : in     TIME;
     STATUS : in     STATUS_INDICATOR := NO_INDICATION)
    return YEAR_NUMBER;
```

Purpose:

This function returns the value of the year component within the time DATE for a given value of the type CAIS_CALENDAR.TIME. 1

Parameters:

**DATE** is the time from which to extract the value of the year.

**STATUS** is an established status indicator.

Exceptions:

**STATUS_ERROR** is raised if STATUS is not an established status indicator [3].

### 11.1.4 Getting the month part of the time

```
function MONTH
    (DATE   : in     TIME;
     STATUS : in     STATUS_INDICATOR := NO_INDICATION)
    return MONTH_NUMBER;
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Purpose:

This function returns the value of the month component within the time DATE for a given value of the type CAIS_CALENDAR.TIME.

Parameters:

**DATE** is the time from which to extract the value of the month.

**STATUS** is an established status indicator.

Exceptions:

**STATUS_ERROR** is raised if STATUS is not an established status indicator [3].

## 11.1.5 Getting the day part of the time

```
function DAY
    (DATE   : in     TIME;
     STATUS : in     STATUS_INDICATOR := NO_INDICATION)
    return DAY_NUMBER;
```

Purpose:

This function returns the value of the day component within the time DATE for a given value of the type CAIS_CALENDAR.TIME.

Parameters:

**DATE** is the time from which to extract the value of the day.

**STATUS** is an established status indicator.

Exceptions:

**STATUS_ERROR** is raised if STATUS is not an established status indicator [3].

## 11.1.6 Getting the seconds part of the time

```
function SECONDS
    (DATE   : in     TIME;
     STATUS : in     STATUS_INDICATOR := NO_INDICATION)
    return DAY_DURATION;
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Purpose:

This function returns the value of the seconds component within the time DATE for a given value of the type CAIS_CALENDAR.TIME. 1

Parameters:

**DATE** is the time from which to extract the value of the seconds.

**STATUS** is an established status indicator.

Exceptions:

**STATUS_ERROR** is raised if STATUS is not an established status indicator [3].

## 11.1.7 Splitting time into its components

```
procedure SPLIT
    (DATE    : in     TIME;
     YEAR    :    out YEAR_NUMBER;
     MONTH   :    out MONTH_NUMBER;
     DAY     :    out DAY_NUMBER;
     SECONDS :    out DAY_DURATION;
     STATUS  : in     STATUS_INDICATOR := NO_INDICATION);
```

Purpose:

This procedure returns all four component values (year, month, day and seconds) for the time DATE. 1

Parameters:

**DATE** is the value of type CAIS_CALENDAR.TIME that will be split into its components. 2

**YEAR** is the year component returned. 3

**MONTH** is the month component returned. 4

**DAY** is the day component returned. 5

**SECONDS** is the seconds component returned. 6

**STATUS** is an established status indicator. 7

Exceptions:

**STATUS_ERROR** is raised if STATUS is not an established status indicator [3].



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

## 11.1.8 Combining components of time

```
function TIME_OF
    (YEAR    : in    YEAR_NUMBER;
     MONTH   : in    MONTH_NUMBER;
     DAY     : in    DAY_NUMBER;
     SECONDS : in    DAY-DURATION;
     STATUS  : in    STATUS_INDICATOR := NO_INDICATION)
    return TIME;
```

Purpose:

This function combines a year number, a month number, a day number and a seconds number into a value of the type CAIS_CALENDAR.TIME.

Parameters:

**YEAR** is the value of the year component.

**MONTH** is the value of the month component.

**DAY** is the value of the day component.

**SECONDS** is the value of the seconds component.

**STATUS** is an established status indicator.

Exceptions:

**TIME_ERROR** is raised if the actual parameters do not form a proper time [1].

**STATUS_ERROR** is raised if STATUS is not an established status indicator [3].

## 11.1.9 Adding time and duration

```
function "+"
    (LEFT    : in    TIME;
     RIGHT   : in    CAIS_DURATION)
    return TIME;

function "+"
    (LEFT    : in    CAIS_DURATION;
     RIGHT   : in    TIME)
    return TIME;
```

Purpose:

This function performs the operation of addition of times and durations.



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Parameters:

**LEFT, RIGHT** are the values of type CAIS_CALENDAR.TIME and CAIS_DURATION to be added together. 2

Exceptions:

**TIME_ERROR** is raised if, for the given operands, the operator cannot return a time whose year number is in the range of the corresponding subtype [z]. 3

## 11.1.10 Subtracting time and duration

```
function "-"
    (LEFT     : in     TIME;
     RIGHT    : in     CAIS_DURATION)
    return TIME;

function "-"
    (LEFT     :  in     TIME;
     RIGHT    :  in     TIME)
    return CAIS_DURATION;
```

Purpose:

This function performs the operation of subtraction of times and durations. 1

Parameters:

**LEFT, RIGHT** are the values of type CAIS_CALENDAR.TIME and CAIS_DURATION to be subtracted from each other. 2

Exceptions:

**TIME_ERROR** is raised if, for the given operands, the operator cannot return a time whose year number is in the range of the corresponding subtype [2] or if the result is not in the range of the type CAIS_DURATION [3]. 3

## 11.1.11 Comparing two values of time

```
function "<"
    (LEFT     : in     TIME;
     RIGHT    : in     TIME)
    return BOOLEAN;

function "<="
    (LEFT     : in     TIME;
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
        RIGHT    : in   TIME)
    return BOOLEAN;

function ">"
    (LEFT    : in   TIME;
     RIGHT   : in   TIME)
    return BOOLEAN;

function ">="
    (LEFT    : in   TIME;
     RIGHT   : in   TIME)
    return BOOLEAN;
```

Purpose:

This function performs the operation of the relational operators for times and has the conventional mathematical meanings. 1

Parameters:

**LEFT, RIGHT** are the values of type CAIS_CALENDAR.TIME to be compared.

Exceptions:

None.


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

# 12. CAIS PRAGMATIC DETAILED REQUIREMENTS

Pragmatic are constraints imposed by an implementation that are not defined by the syntax or semantics of the CAIS. This section delineates the minimum capacities a conforming CAIS implementation must support. For most pragmatic limitations, two constants are defined in this package CAIS_PRAGMATICS. One is prefixed with “CAIS_”; it is the minimum value that any CAIS implementation must support. The other one without the prefix specifies an implementation-defined limit equal to or beyond the minimum required. Each CAIS implementation must supply this package with the actual values filled in. 1

A CAIS implementation is permitted to introduce implementation-defied exceptions to be raised in situations where the CAIS semantics cannot feasibly be obeyed due to restrictions or properties of the underlying system (hardware and operating system). All implemental ion-defined exceptions will be declared in package CAIS_PRAGMATICS. 2

## 12.1 Package CAIS_PRAGMATICS

```
CAPACITY_ERROR : exception;
RESOURCE_ERROR : exception;
```

CAPACITY_ERROR is raised if a call on a CAIS interface detects a violation of the implementation-dependent maxima for the pragmatic limitations specified in this package. RESOURCE_ ERROR is raised if a call on a CAIS interface violates a resource constraint imposed by the underlying implementation. This exception is raised only if the conditions for CAPACITY_ERROR are not present. 1

```
UNRESTRICTED : constant : = implementation-defined;
```

UNRESTRICTED is a very large universal_integer constant, usable only in universal expressions (see [1815A] 4.10). 2

### 12.1.1 Identifier and view pragmatic

```
CAIS_IDENTIFIER_LENGTH : constant : = 80;
IDENTIFIER_LEHGTH : constant : = implementation-defined;
```

CAIS_IDENTIFIER_LENGTH and IDENTIFIER_LENGTH are constants specifying, respectively, the smallest upper limit which can be imposed by any CAIS implementation on the number of characters in an identifier and the actual upper limit imposed by a particular implementation. 1

```
CAIS_PATHNAME_LEHGTH : constant : = 266;
PATHNAME_LEHGTH : constant : = implementation-defined;
```

CAIS_PATHNAME_LENGTH and PATHNAME_LENGTH are constants specifying, respectively, the smallest upper limit which can be imposed by any CAIS implementation on the number of characters in a pathname and the actual upper limit imposed by a particular implementation. 2



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
CAIS_ADOPTED_VIEWS_OF_PROCESS:  constant := 266;
ADOPTED_VIEW_OF_PROCESS : constant := implementation_defined;
```

CAIS_ADOPTED_VIEWS_OF_PROCESS and ADOPTED_VIEWS_OF_PROCESS are constants specifying, respectively, the smallest upper limit which can be imposed by any CAIS implementation on the number of views which a single process can have adopted at one time and the actual upper limit imposed by a particular implementation. 3

```
CAIS_UNIQUE_IDENTIFIER_PREFIX_LENGTH : constant := 80;
```

CAIS_UNIQUE_IDENTIFIER_PREFIX_LENGTH is a constant specifying the upper limit on the number of characters in a unique identifier prefix generated by any CAIS implementation, as well as the minimum number of characters in a unique identifier prefix that any CAIS implementation must support. 4

```
CAIS_UNIQUE_IDENTIFIER_SUFFIX_LENGTH : constant : = 80;
```

CAIS_UNIQUE_IDENTIFIER_SUFFIX_LENGTH is a constant specifying the upper limit on the number of characters in a unique identifier suffix generated by any CAIS implementation, as well as the minimum number of characters in a unique identifier suffix that any CAIS implementation must support. 5

```
CAIS_UNIQUE_IDENTIFIERS_PER_INSTANCE : constant : = 2 ** 63 - 1;
```

CAIS_UNIQUE_IDENTIFIERS_PER_INSTANCE is a constant specifying the minimum number of distinct unique identifier suffixes that any CAIS implementation must be able to produce. 6

## 12.1.2 Node model pragmatic

```
CAIS_NUMBER_OF_NODES : constant : = UNRESTRICTED;
```

CAIS_NUMBER_OF_NODES is a constant specifying the smallest upper limit which can be imposed by any CAIS implementation on the number of nodes that can be contained in a given CAIS implementation at one time. 1

```
CAIS_LENGTH_OF_PRIMARY_PATH : constant : = PATHNAME_LENGTH/2;
```

CAIS_LENGTH_OF_PRIMARY_PATH is a constant specifying the smallest upper limit which can be imposed by any CAIS implementation on the number of path elements in a primary path. 2

```
CAIS_EMANATING_PRIMARY_RELATIONSHIPS_PER_NODE : constant := 2 ** 10 - 1;
EMANATING_PRIMARY_RELATIONSHIPS_PER_NODE      : constant : =
    implementation-defined;
```

CAIS_EMANATING_PRIMARY_RELATIONSHIPS_PER_NODE and EMANATING_PRIMARY_RELATIONSHIPS_PER_NODE are constants specifying, respectively, the smallest upper limit which can be imposed by any CAIS implementation on the number of primary relationships that can emanate from a single node at one time and the actual upper limit imposed by a particular implementation. 3



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
CAIS_EMANATING_SECONDARY_RELATIONSHIPS_PER_NODE : constant := 2 ** 10 - 1;
EMANATING_SECONDARY_RELATIONSHIPS_PER_NODE : constant :=
    implementation-defined;
```

CAIS_EMANATING_SECONDARY_RELATIONSHIPS_PER_NODE and EMANATING-SECONDARY_RELATIONSHIPS_PER_NODE are constants specifying, respectively, the smallest upper limit which can be imposed by any CAIS implementation on the number of secondary relationships that can emanate from a single node at one time and the actual upper limit imposed by a particular implementation. 4

```
CAIS_ATTRIBUTES_PER_NODE : constant := 266;
ATTRIBUTES-PER-MODE : constant : = implementation-defined;
```

CAIS_ATTIUBUTES_PER_NODE and ATTRIBUTES_PER_NODE are constants specifying, respectively, the smallest upper limit which can be imposed by any CAIS implementation on the number of attributes that can be associated with a single node at one time and the actual upper limit imposed by a particular implementation. 5

```
CAIS_ATTRIBUTES_PER_RELATIOHSHIP : constant := 255;
ATTRIBUTES_PER_RELATIONSHIP : constant : = implementation-defined;
```

CAIS_ATTRIBUTES_PER_RELATIONSHIP and ATTRIBUTES_PER_RELATIONSHIP are constants specifying, respectively, the smallest upper limit which can be imposed by any CAIS implementation on the number of attributes that can be associated with a single relationship at one time and the actual upper limit imposed by a particular implementation. 6

```
CAIS_NODES_IN_COPY_TREE : constant : = 2 ** 15 - 1;
NODES_IN_COPY_TREE : constant : = implementation-defined;
```

CAIS_NODES_IN_COPY_TREE and NODES_IN_COPY_TREE are constants specifying, respectively, the smallest upper limit which can be imposed by any CAIS implementation on the number of nodes that can be copied with a single call of COPY-TREE and the actual upper limit imposed by a particular implementation. 7

```
CAIS_NODES_IN_DELETE_TREE : constant : = 2 ** 15 - 1;
NODES_IN_DELETE_TREE : constant : = implementation-defined;
```

CAIS_NODES_IN_DELETE_TREE and NODES_IN_DELETE_TREE are constants specifying, respectively, the smallest upper limit which can be imposed by any CAIS implementation on the number of nodes that can be deleted with a single call of DELETE_TREE and the actual upper limit imposed by a particular implementation. 8

## 12.1.3 Handle and iterator pragmatic

```
CAIS_NODE_HANDLES_PER_PROCESS : constant := 265;
NODE_HANDLES_PER_PROCESS : constant : = implementalion-definod;
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

CAIS_NODE_HANDLES_PER_PROCESS and NODE_HANDLES_PER_PROCESS are constants specifying, respectively, the smallest upper limit which can be imposed by any CAIS implementation on the number of open node handles a process can have at one time and the actual upper limit imposed by a particular implementation. Handles which are returned by CAIS functions and handles which are implicitly declared and opened by the CAIS implementation do not count against these limits. 1

```
CAIS_FILE_HANDLES_PER_PROCESS : constant := 16;
FILE_HANDLES_PER_PROCESS : constant := implementation-defined;
```

CAIS_FILE_HANDLES-PER_PROCESS and FILE_HANDLES_PER_PROCESS are constants specifying, respectively, the smallest upper limit which can be imposed by any CAIS implementation on the number of open file handles a process can have at one time and the actual upper limit imposed by a particular implementation. 2

```
CAIS_TRANSACTION_HANDLES_PER_PROCESS : constant := 266;
TRANSACTION_HANDLES_PER_PROCESS : constant := implementalion_defined;
```

CAIS_TRANSACTION_HANDLES_PER_PROCESS and TRANSACTION_HANDLES_PER_PROCESS are constants specifying, respectively, the smallest upper limit which can be imposed by any CAIS implementation on the number of open transaction handles a process can have at one time and the actual upper limit imposed by a particular implementation. Transaction handles which are implicitly declared and opened by the implementation, e.g. for the enclosing transaction of a process, do not count against these limits. 3

```
CAIS_ELEMENTS_OF_NODE_ITERATOR : constant := 2 ** 11 - 2;
ELEMENTS_OF_NODE_ITERATOR : constant := implementation_defined;
```

CAIS_ELEMENTS_OF_NODE_ITERATOR and ELEMENTS_OF_NODE_ITERATOR are constants specifying, respectively, the smallest upper limit which can be imposed by any CAIS implementation on the number of elements that can be contained in a node iterator at one time and the actual upper limit imposed by a particular implementation. The constant ELEMENTS_OF_NODE_ITERATOR must be at least as large as the sum of the two constants EMANATING_PRIMARY_RELATIONSHIPS_PER_NODE and EMANATING_SECONDARY_RELATIONSHIPS_PER_NODE. 4

```
CAIS_ELEMENTS_OF_NODE_KIND_INSTANCE_ITERATOR : constant := UNRESTRICTED;
```

CAIS_ELEMENTS_OF_NODE_KIND_INSTANCE_ITERATOR is a constant specifying the smallest upper limit which can be imposed by any CAIS implementation on the number of elements that can be contained in a node kind instance iterator at one time. 5

```
CAIS_ELEMENTS_OF_ATTRIBUTE_ITERATOR : constant := 256 ;
ELEMENTS_OF_ATTRIBUTE_ITERATOR : constant := implementation-defined;
```

CAIS_ELEMENTS_OF_ATTRIBUTE_ITERATOR and ELEMENTS_OF_ATTRIBUTE_ITERATOR are constants specifying, respectively, the smallest upper limit which can be imposed by any CAIS implementation on the number of elements that can be contained in an attribute iterator at one time and the actual upper limit imposed by a particular implementation. 6



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

## 12.1.4 Integer and real number pragmatic

```
CAIS_MINIMUM_INTEGER : constant : = - (2 ** 31 - 1);
MINIMUM_INTEGER      : constant := implementalion_defined;
```

CAIS_MINIMUM_INTEGER and MINIMUM_INTEGER are constants specifying, respectively, the largest lower limit which can be imposed by any CAIS implementation on the smallest (most negative) value of the type CAIS_INTEGER and the actual lower limit imposed by a particular implementation. CAIS_INTEGER is defined in Section 10 on page 805. 1

```
CAIS_MAXIMUM_INTEGER : constant := 2 ** 31 - 1;
MAXIMUM_INTEGER      : constant : = implementation_defined;
```

CAIS_MAXIMUM_INTEGER and MAXIMUM_INTEGER are constants specifying, respectively, the smallest upper limit which can be imposed by any CAIS implementation on the largest (most positive) value of the type CAIS_INTEGER and the actual upper limit imposed by a particular implementation. CAIS_INTEGER is defined in Section 10 on page 805. 2

```
CAIS_SMALL_FOR_CAIS_DURATION : constant : = 0.015626; -- 1/64
SMALL_FOR_CAIS_DURATION      : constant : = implementation_defined;
```

CAIS_SMALL_FOR_CAIS_DURATION and SMALL_FOR_CAIS_DURATION are constants specifying, respectively, the maximum for the smallest representable duration, i.e., for the value of CAIS_-DURATION'SMALL, and the actual smallest duration supported by a particular CAIS implementation. CAIS_DURATION is defined in Section 10 on page 805. 3

## 12.1.5 Access control pragmatic

```
CAIS_GROUP_NODES : constant : = 255;
GROUP_MODES      : constant : = implementation_defined;
```

CAIS_GROUP_NODES and GROUP_NODES are constants specifying, respectively, the smallest upper limit which can be imposed by any CAIS implementation on the number of group nodes that can be contained in a given CAIS implementation and the actual upper limit imposed by a particular implementation. 1

```
CAIS_ACCESS_RELATIONSHIPS_OF_OBJECT : constant := 255;
ACCESS_RELATIONSHIPS_OF_OBJECT      : constant : = implementation_defined;
```

CAIS_ACCESS_RELATIONSHIPS_OF_OBJECT and ACCESS_RELATIONSHIPS_OF_OBJECT are constants specifying, respectively, the smallest upper limit which can be imposed by any CAIS implementation on the number of relationships of the predefine relation ACCESS that can emanate from a node at one time and the actual upper limit imposed by a particular implementation. 2

```
CAIS_GRANT_ITEMS_ON_GRANT_ATTRIBUTE : constant := 15;
GRANT_ITEMS_ON_GRANT_ATTRIBUTE      : constant : = implementation-defined;
```


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

CAIS_GRANT_ITEMS_ON_GRANT_ATTRIBUTE and GRANT_ITEMS_ON_GRANT_ATTRIBUTE 3
are constants specifying, respectively, the smallest upper limit which can be imposed by any CAIS implementation on the number of grant items that can be contained in the value of a GRANT attribute at one time and the actual upper limit imposed by a particular implementation.

```
CAIS_ADOPTED_ROLES_OF_PROCESS : constant : = 7;
ADOPTED_ROLES_OF_PROCESS : constant : = implementation_defined;
```

CAIS_ADOPTED_ROLES_OF_PROCESS and ADOPTED_ROLES_OF_PROCESS are constants spec- 4
ifying, respectively, the smallest upper limit which can be imposed by any CAIS implementation on the number of roles which a single process can have adopted at one time and the actual upper limit imposed by a particular implementation.

## 12.1.6 File and device pragmatic

```
FILE_STORAGE_UNIT_SIZE : constant : = implementation_defined;
```

FILE_ STORAGE_UNIT_SIZE is a constant specifying the number of bits per file storage unit in se- 1
quential files and direct files for a particular CAIS implementation. Some storage devices utilized by the implementation may not conform to this unit size, but file size values returned by the implementation will be normalized to this unit.

```
MEMORY_STORAGE_UNIT_SIZE : constant : = implementalion_defined;
```

MEMORY_STORAGE_UNIT_SIZE is a constant specifying the number of bits per memory storage 2
unit for a particular CAIS implementation. Some memory devices utilized by the implementation may not conform to this unit size, but memory size values returned by the implementation will be normalized to this unit.

```
CAIS_DIRECT_IO_ELEMENT_SIZE : constant := 2 ** 16 - 1;
DIRECT_IO_ELEMENT_SIZE : constant : = implementation_defined;
```

CAIS_DIRECT_IO_ELEMENT_SIZE and DIRECT_IO_ELEMENT_SIZE are constants specifying, 3
respectively, the smallest upper limit which can be imposed by any CAIS implementation on the number of bits in a direct input or output element and the actual upper limit imposed by a particular implementation.

```
CAIS_DIRECT_IO_INDEX_RANGE_UPPER_BOUND : constant : = 2 ** 16 - 1;
DIRECT_IO_INDEX_RANGE_UPPER_BOUND : constant : = implementalion_defined;
```

CAIS_DIRECT_IO_INDEX_RANGE_UPPER_BOUND and DIRECT_IO_INDEX_RANGE_UPPER_- 4
BOUND are constants specifying, respectively, the smallest upper limit which can be imposed by any CAIS implementation on the type COUNT in the package CAIS_DIRECT_IO and the actual upper limit imposed by a particular implementation.

```
CAIS_SEQUENTIAL_IO_ELEMENT_SIZE : constant := 2 ** 15 - 1;
SEQUENTIAL_IO_ELEMENT_SIZE : constant : = implementation_defined;
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

CAIS_SEQUENTIAL_IO_ELEMENT_SIZE and SEQUENTIAL_IO_ELEMENT_SIZE are constants specifying, respectively, the smallest upper limit which can be imposed by any CAIS implementation on the number of bits in a sequential input or output element and the actual upper limit imposed by a particular implementation. The constant SEQUENTIAL_IO_ELEMENT_SIZE must be at least as large as the constant DIRECT_IO_ELEMENT_ SIZE. 5

```
CAIS.SEQUENTIAL_IO_FILE_SIZE : constant := 2 ** 15 - 1;
SEQUENTIAL_IO_FILE_SIZE      : constant : = implementation_defined;
```

CAIS_SEQUENTIAL_IO_FILE_SIZE and SEQUENTIAL_IO_FILE_SIZE are constants specifying, respectively, the smallest upper limit which can be imposed by any CAIS implementation on the number of CAIS_SEQUENTIAL_IO_WRITE operations that can be performed on a sequential file and the actual upper limit imposed by a particular implementation. SEQUENTIAL_IO_FILE_SIZE must be at least as large as DIRECT_IO_INDEX_RANGE_UPPER_BOUND. 6

```
CAIS_TEXT_IO_LINES_PER_FILE : constant := 2 ** 16 - 1;
TEXT_IO_LINES_PER_FILE      : constant : = implementation_defined;
```

CAIS_TEXT_IO_LINES_PER_FILE and TEXT_IO_LINES_PER_FILE are constants specifying, respectively, the smallest upper limit which can be imposed by any CAIS implementation on the number of lines in a text input or output file and the actual upper limit imposed by a particular implementation. 7

```
CAIS_TEXT_IO_LIEES_PER_PAGE : constant : = 2 ** 15 - 1;
TEXT_IO_LIEES_PER_PAGE      : constant : = implementation_defined;
```

CAIS_TEXT_IO_LINES_PER_PAGE and TEXT_IO_LINES_PER_PAGE are constants specifying, respectively, the smallest upper limit which can be imposed by any CAIS implementation on the number of lines per page in a text input or output file and the actual upper limit imposed by a particular implemental ion. 8

```
CAIS_TEXT_IO_COLUHMS_PER_LIME : constant := 266;
TEXT_IO_COLUMNS_PER_LIHE      : constant : = implementation_defined;
```

CAIS_TEXT_IO_COLUMNS_PER_LINE and TEXT_IO_COLUMNS_PER_LINE are constants specifying, respectively, the smallest upper limit which can be imposed by any CAIS implementation on the number of columns per line of a text input or output file and the actual upper limit imposed by a particular implementation. 9

```
CAISIMAXIMUM_FRAME_LENGTH : constant := 2 ** 15 - 1;
MAXIMUM_FRAME_LENGTH      : constant : = implementation_defined;
```

CAIS_MAXIMUM_FRAME_LENGTH and MAXIMUM_FRAME_LENGTH are constants specifying, respectively, the smallest upper limit which can be imposed by any CAIS implementation on the size of a frame allowed by the CAIS_FRAME_IO protocol and the actual upper limit imposed by a particular implementation. 10


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
CAIS_MAXIMUM_FRAME_PRIORITY : constant := 32;
MAXIMUM_FRAME_PRIORITY      : constant := implementation_defined;
```

CAIS_MAXIMUM_FRAME_PRIORITY and MAXIMUM_FRAME_PRIORITY are constants specifying, respectively, the smallest upper limit which can be imposed by any CAIS implementation on the number of message priority levels allowed by the CAIS_FRAME_IO protocol and the actual upper limit imposed by a particular implementation. 11

```
CAIS_MINIMUM_TAPE_BLOCK_LENGTH : constant : = 18;
MINIMUM_TAPE_BLOCK_LENGTH      : constant : = implementation_defined;
```

CAIS_MINIMUM_TAPE_BLOCK_LENGTH and MINIMUM_TAPE_BLOCK_LENGTH are constants, specifying, respectively, the largest lower limit which can be imposed by any CAIS implementation on the number of characters written to a magnetic tape in a single block and the actual lower limit imposed by a particular implementation. 12

```
CAIS_MAXIMUM_TAPE_BLOCK_LENGTH : constant := 2048;
MAXIMUM_TAPE_BLOCK_LENGTH      : constant := implementation_defined;
```

CAM_MAXIMUM_TAPE_BLOCK_LENGTH and MAXIMUM_TAPE_BLOCK_LENGTH are constants, specifying, respectively, the smallest upper limit which can be imposed by any CAIS implementation on the number of characters written to a magnetic tape in a single block and the actual upper limit imposed by a particular implementation. 13

```
CAIS_MAXIMUM_PHYSICAL_DISPLAY_DIMENSION : constant := 132;
MAXIMUM_PHYSICAL_DISPLAY_DIMENSION      : constant : = implementation_defined;
```

CAIS_MAXIMUM_PHYSICAL_DISPLAY_DIMENSION and MAXIMUM_PHYSICAL_DISPLAY_DIMENSION are constants, specifying, respectively, the smallest upper limit which can be imposed by any CAIS implementation on the number of character cells in any one row or column of a physical display and the actual upper limit imposed by a particular implementation. 14

```
CAIS_MAXIMUM_SCREEN_DIMENSION : constant := 132;
MAXIMUM_SCREEN_DIMENSION      : constant : = implementation_defined;
```

CAIS_MAXIMUM_SCREEN_DIMENSION and MAXIMUM_SCREEN_DIMENSION are constants, specifying, respectively, the smallest upper limit which can be imposed by any CAIS implementation on the number of character cells in any one row or column of a screen and the actual upper limit imposed by a particular implementation. 15

### 12.1.7 List pragmatic

```
CAIS_LIST_LENGTH : constant : = 266;
LIST_LENGTH      : constant : = implementation_defined;
```

CAIS_LIST_LENGTH and LIST_LENGTH are constants specifying, respectively, the smallest upper limit which can be imposed by any CAIS implementation on the number of items in a list and the actual upper limit imposed by a particular implementation. 1


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
CAIS_LIST_TEXT_LENGTH : constant := 2 ** 10;
LIST_TEXT_LENGTH      : constant := implementation_defined;
```

CAIS_LIST_TEXT_LENGTH and LIST_TEXT_LENGTH are constants specifying, respectively, the smallest upper limit which can reimposed by any CAIS implementation on the number of characters in the external representation of a list and the actual upper limit imposed by a particular implementation. 2

```
CAIS_LIST_MAXIMUM_DIGITS : constant := 6;
LIST_MAXIMUM_DIGITS      : constant : = implementation_defined;
```

CAIS_LIST_MAXIMUM_DIGITS and LIST_MAXIMUM_DIGITS are constants specifying, respectively, the smallest upper limit which can be imposed by any CAIS implementation on the number of significant decimal digits of the floating point type used to contain floating point item values and the actual upper limit imposed by a particular CAIS implementation. 3

```
CAIS_IDENTIFIER_ITEH_LEIGTH : constant : = CAIS_IDENTIFIER_LEHGTH;
IDENTIFIER_ITEH_LEIGTH      : constant : = implementation_defined;
```

CAIS_IDENTIFIER_ITEM_LENGTH and IDENTIFIER_ITEM_LENGTH are constants specifying, respectively, the smallest upper limit which can be imposed by any CAIS implementation on the number of characters in an identifier item or the value of a token and the actual upper limit imposed by a particular implementation. 4

```
CAIS_STRING_ITEM_LENGTH : constant := CAIS_PATHNAME_LENGTH;
STRING_ITEM_LENGTH      : constant := implementation_defined;
```

CAIS_STRING_ITEM_LENGTH and STRING_ITEM_LENGTH are constants specifying, respectively, the smallest upper limit which can be imposed by any CAIS implementation on the number of characters in a string item and the actual upper limit imposed by a particular implementation. STRING_ITEM_LENGTH must be at least as large as the value of PATHNAME_LENGTH. 5



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

# 13. CAIS STATUS MANAGEMENT DETAILED REQUIREMENTS

CAIS interfaces provide an optional STATUS parameter of type STATUS_INDICATOR. STATUS_INDICATOR is a limited private type declared within the package CAIS_STATUS_DEFINITIONS. Objects of type STATUS_INDICATOR are called status indicators. 1

An established STATUS parameter allows calling programs to obtain exception and status information beyond that provided through an exception alone. An established status indicator is one which has been initialized (generally by a call to CREATE_INDICATOR) to acquire exception and status information or one which has the implementation-defined value NO_INDICATION which specifies that exception and status information are not to be acquired. The value of an established status indicator upon the return from a call to a CAIS interface is independent of its value upon entry to that CAIS interface. Any exception and status information associated with a status indicator at the initiation of a CAIS operation is not associated with the status indicator upon completion of the operation. Exception and status information can be retrieved from an established status indicator at any time using the interfaces in the package CAIS_STATUS_MANAGEMENT. 2

The default value for STATUS parameters is the value of the deferred constant NO_INDICATION. When the value NO_INDICATION is passed as the STATUS parameter on a call to a CAIS interface, no exception or status information is recorded in the STATUS parameter. NO_INDICATION is distinct from the value of a status indicator before it is established. The exception indication and the status information list (see below) associated with the status indicator value NO_INDICATION are NO_ERROR_DETECTED and the empty list, respectively. 3

The exception information is provided as a value of the enumeration type EXCEPTION_INDICATION. This value is called the exception indication of the status indicator. The value NO_ERROR_DETECTED indicates the absence of any condition corresponding to a CAIS-defined exception. Other EXCEPTION_INDICATION values correspond to the CAIS-defined exceptions. When the exception indication has a value other than NO_ERROR_DETECTED, the condition corresponding to the indicated exception was detected. For a given call to a CAIS interface, the resultant exception indication of the status indicator indicates the particular exception raised, if any. 4

The status information acquired by a status indicator is provided as a named list (a value of the type LIST_TYPE) called a status information list. Regardless of the exception indication returned in a status indicator (including NO_ ERROR_DETECTED), the status information list and an appropriate natural language message may be obtained by passing a status indicator to the STATUS_INFORMATION interface and the MESSAGE interface, respectively, in package CAIS_STATUS_MANAGEMENT. 5

The syntax for status information lists is specified in Table XXXVIII, page 827. Table XXXIX, page 834 describes the corresponding semantics for each syntactic construct. Uppercase identifiers are used in the status information list syntax to denote literal item names and literal identifiers. The ordering of the named items in a status information list and its sublists is implementalion_defined. 6

The items named PACKAGE and INTERFACE are always included in the status information list. They are the Ada identifiers for the package and interface called. The remaining items are included only when an exception is raised (i.e., when the exception indication is not NO_ERROR_DETECTED). The CONDITION item is always included if an exception is raised. The value of CONDITION is an integer condition code which indicates the cause of the the exception. The predefine conditions and condition codes are described in Section 13.3 on page 862. Implementations may define additional conditions. In the “Exceptions:” paragraphs of the CAIS interface descriptions, the various conditions 7


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

which may cause an exception to be raised when the interface is called are denoted by integer condition code values in square brackets (e.g., [4]).



Source: https://assist.dla.mil -- Downloaded: 202610-26T07:54Z
Check the source to verify that this is the current version before use.

TABLE XXXVIII: Status Information List BNF

```
status_information_list ::= ( PACKAGE => identifier,
                              INTERFACE => identifier
                             [, CONDITION => condition_code]
                             [, SUBCONDITION => subcondition_code]
                             [, PARAMETERS => ( parameter{, parameter} )]
                             {, qualifier})
qualifier               ::=  AMBIGUOUS_ NAMES => ( identifier , identifier )
                           | ACCESS_RIGHT => string
                             ALLOWED_USAGE => relationship_kind
                             ANCESTOR_DEFINITION => NKD_unique_identifier
                             ANCESTOR_DEFINITION => RD_unique_identifier
                             ATTRIBUTE => AD_unique_identifier
                             ATTRIBUTE_KIND => AKD_unique_identifier
                             ATTRIBUTES_LIST => list
                           | ATTRIBUTE_NAME => string
                             BLOCK_SIZE => integer
                           | CHANNEL => CHANNEL_unique_identifier
                             CHARACTERISTICS => list
                             CLASSIFICATION_ ERROR => classification_error_type
                             CLASSIFICATION_ LABEL => label
                             CLIENT_NODE => unique_identifier
                             CHANNEL_STATE => channel_state_type
                             COMPONENT => component_specification
                             CONFLICT => constraints_conflict
                             CONFLICTING_INTENTS => intents_list
                             CONSTRAINT_VIOLATED => constraint_violated
                           | CONSTRAINT_ITEM => ( constraint_item )
                             CURRENT_LIST => current_linear_list_specification
                             CURRENT_LIST_1 => current_linear_list_specification
                             CURRENT_LIST_2 => current_linear_list_specification
                             DAY => integer
                             DEADLOCK => TRUE | FALSE
                           | DEFINITION => definition_unique_identifier
                             DEFINITION_NAME => identifier
                             DEFINITION_TYPE => definition_type
                           | DEVICE => DEVICE_unique_identifier
                             DIRECTION => direction_type
                           | END_POSITION => integer
                             ERROR_ITEM => integer
                             ERROR_SUBLIST => current_linear_list_specification
                             EXPECTED_ CHANNEL_ STATE => channel_state_type
                           | FACTOR => constraints_conflict_factor
```


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

TABLE XXXVIII: Status Information List BNF (cont.)

```
        FILE => FILE_unique_identifier
      | FILE_NAME => string
      | GATEWAY => GATEWAY_unique_identifier
        GRANT => list
        GROUP => GROUP_unique_identifier
        IDENTIFIER_TEXT => string
      | IMPLIED_USAGE => relationship_kind
        INPUT_PARAMETER => list
        INVALID_ITEM => string
      | INTENTS => intents_list
      | INTERPRETATION => interpretation_unique_identifier
      | INTERPRETATION => RI_unique_identifier
      | INTERPRETATION_1 => NKI_unique_identifier
        INTERPRETATION_2 => RI_unique_identifier
        INTERPRETATION_TYPE => interpretation_type
        ITEM_KIND => list_item_kind
        ITEM_NAME => identifier
        ITEM_POSITION => integer
        ITEM_VALUE => item_value
        KEY => AD_unique_identifier
        LIST => list
        LIST_1 => list
        LIST_2 => list
        LIST_KINDS => ( list_kind, list_kind )
        LIST_LENGTH => integer
      | LIST_TEXT => string
      | MAXIMUM_LINE_LENGTH => integer
      | MAXIMUM_PAGE_LENGTH => integer
      | MEDIUM => MEDIUM_unique_identifier
      | MODE => identifier
        MONITOR => MONITOR_unique_identifier
        MONTH => integer
        NAME => identifier
      | NAME_TYPE => name_type
      | NEW_INTENTS => intents
        NODE => unique_identifier
        NODE_1 => unique_identifier
        NODE_2 => unique_identifier
        OTHER_CLIENT_NODE => unique_identifier
        OTHER_DIRECTION => direction_type
        OTHER_PORT => RD_unique_identifier
        PATHNAME => string
        PATTERN => string
        PORT => RD_unique_identifier
```


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

TABLE XXXVIII: Status Information List BNF (cent.)

| | | |
|---|---|---|
| | | PRAGMATIC => identifier |
| | | PREDICATE => list |
| | | PRIORITY => integer |
| | | PROCESS => PROCESS_unique_identifier |
| | | PROCESSOR => PROCESSOR_unique_ identifier |
| | | PROPERTY => property_unique_identifier |
| | | PROPERTY_TYPE => property_type |
| | | PROTOCOL => identifier |
| | | RELATION => RD_unique_identifier |
| | | RELATIONSHIP => relationship_specification |
| | | RELATIONSHIP_ KEY => string |
| | | RELATIONSHIP_NAME => string |
| | | RELATION_NAME => string |
| | | REQUIRED_ITEM_KIND => list_item_kind |
| | | REQUIRED_NKD => NKD_unique_identifier |
| | | REQUIRED_RD => RD_unique_identifier |
| | | RESOURCE => resource_unique_identifier |
| | | RESOURCE_TYPE => resource_type |
| | | RESTRICTION => node_deletion_restriction |
| | | RESTRICTION => property_deletion_restriction |
| | | RIGHTS_NEEDED => rights_list |
| | | ROLE_TYPE => role_type |
| | | SECONDS => integer |
| | | SOURCE_NODE => unique_identifier |
| | | START_POSITION => integer |
| | | SYNTACTIC_ERROR => syntactic_error_type |
| | | TAPE_STATUS => tape_status_type |
| | | TARGET_NODE => unique_identifier |
| | | UID => unique_identifier |
| | | VALUE => list |
| | | VIEW => VIEW_unique_identifier |
| | | YEAR => integer |
| access_right | ::= | identifier |
| AD_unique_identifier | ::= | unique_identifier |
| base_type_restriction | ::= | constraints_conflict |
| | | NONCONFORMING_INSTANCES |
| channel_state_type | ::= | CONNECTED |
| | | ESTABLISHED |
| | | CLOSED |
| | | RETIRED |
| CHANNEL_unique_identifier | ::= | unique_identifier |
| classification_error_type | ::= | INVALID_HIERARCHICAL_ CLASSIFICATION |
| | | INVALID_NON_HIERARCHICAL_CATEGORY |



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

TABLE XXXVIII: Status Information List BNF (cent.)

```
component identification        ::= ( [ITEM_POSITION => integer]
                                    [, NAME=> identifier]
                                    [, POSITION => identifier] )
component -specification        ::= ( component_identification
                                    {, component_identification} )
condition_code                  ::= integer
constraint violated             ::= BOUNDS
                                  | ENUMERATION_POSITION
                                  | NAMING
                                  | ORDER
                                  | PATTERNS
                                  | SIZE
constraints_cordiict            ::= ATTRIBUTE_BOUNDS
                                  | ATTRIBUTE_DEFAULT
                                  | ATTRIBUTE_ DEFAULT_OUTSIDE_RANGE
                                  | ATTRIBUTE_DEFAULT_OUTSIDE_VALUE
                                  | ATTRIBUTE_NAMING
                                  | ATTRIBUTE_PATTERNS
                                  | ATTRIBUTE_SIZE
                                  | ATTRIBUTE_VALUE
                                  | ATTRIBUTE_VALUE_OUTSIDE_RANGE
                                  | RELATION_ALLOWED_USAGE
                                  | RELATION_BOUNDS
                                  | RELATION_TERMINATES
                                  | RELATION_UNIQUE_KEYS
constraints_conflict_factor     ::= HAS_LOCAL_CONSTRAINT
                                    HAS_OBTAINABLE_INSTANCES
constraint-item                 ::= value_constraint item
                                    range_constraint_item
                                    patterns_constraint_item
                                  | naming_constraint_item
                                  | uniform_list_size_constraint_item
current_linear_list_specification ::= () ( integer{, integer} )
definition_type                 ::= NKD | RD | AD | AKD
definition_unique_identifier    ::= NKD_unique_identifier
                                  | RD_unique_identifier
                                  | AD_unique_identifier
                                  | AKD_unique_identifier
DEVICE_unique_identifier        ::= unique-identifier
direction_type                  ::= IN | OUT | IN-AND-OUT | IN_OR_OUT
                                    NO_DIRECTION
FILE_unique_identifier          ::= unique_identifier
GATEWAY_unique_identifier       ::= unique_identifier
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

TABLE XXXVIII: Status Information List BNF (cont.)

```
GROUP_unique_identifier        ::=  unique_identifier
intents_list                   ::= ( intent_specification{, intent_specification} )
intent_specification           ::=  identifier
interpretation_type            ::=  NKI | RI
item_value                     ::= list | identifier | integer | string | float
key_list                       ::=  ( key_value_specification {, key_value_specification} )
key_value                      ::= identifier | ""
key_value_specification        ::= ( KEY => AD_unique_identifier,
                                     VALUE => key_value )
list_item_kind                 ::=  LIST | IDENTIFIER | INTEGER | STRING
                                  | FLOAT
list_kind                      ::= EMPTY | NAMED | UNNAMED
MEDIUM_unique_identifier       ::=  unique_identifier
MONITOR_unique_identifier      ::=  unique_identifier
name_type                      ::= RELATION | ATTRIBUTE | KEY
NKD_unique_identifier          ::=  unique_identifier
NKI_unique_identifier          ::=  unique_identifier
node_deletion_restriction      ::=  HAS_OBTAINABLE_ATTRIBUTE
                                    HAS_OBTAINABLE_INSTANCE
                                    HAS_OBTAINABLE_SPECIALIZATION
                                    IN_ADOPTED_VIEW
                                    IN_MULTIPLE_VIEWS
                                  | IS_AN_ALTERNATIVE
                                  | IS_AN_ATTRIBUTE_PR0PERTY
                                  | IS_A_COMPONENT
                                    IS_AN_EMANATES_PROPERTY
                                    IS_A_KEY_PROPERTY
                                    IS_A_TERMINATES_PROPERTY
                                    IS_PREDEFINED
parameter                      ::= parameter_name => parameter-value
parameter_name                 ::=  identfier
parameter_value                ::=  float | identifier | integer | intents-list
                                  | list | string | unique_identifier
PROCESS_unique_identifier      ::=  unique_identifier
PROCESSOR_unique_identifier    ::=  unique_identifier
property_deletion_restriction  ::=  IN_ADOPTED_VIEW
                                  | IN_MULTIPLE_VIEWS
                                  | IS_PREDEFINED
                                  | PROPERTY_INSTANCE
                                    RELATION_TERMINATES
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

TABLE XXXVIII: Status Information List BNF (cont.)

```
property_type               ::=   ALTERNATIVE
                                  ATTRIBUTE
                                  ATTRIBUTE_KIND
                                | COMPONENT
                                | EMANATES
                                | KEY
                                  TERMINATES
property_unique_identifier  ::=   RD_unique_identifier
                                | AD_unique_identifier
                                | AKD_unique_identifier
RD_unique_identifier        ::= unique_identifier
relationship_kind           ::= PRIMARY | SECONDARY
relationship_specification  ::= ( BASE => unique_identifier,
                                  RELATION => RD_unique_identifier
                                  [, KEY=> key_list] )
resource_type               ::= DEVICE | GATEWAY | MEDIUM | PROCESSOR
resource_unique_identifier  ::=   DEVICE_unique_identifier
                                  GATEWAY_unique_identifier
                                  MEDIUM_unique_identifier
                                  PROCESSOR_unique_identifier
rights_list                 ::= ( access_right {, access_right } )
RI_unique_identifier        ::= unique_identifier
role_type                   ::= IMAGE
                                | USER
subcondition_code           ::= integer
syntactic_error_type        ::=   COMMA_OR_RIGHT_PARENTHESIS_EXPECTED
                                  ARROW_EXPECTED
                                  ERROR_IN_DECIMAL-LITERAL
                                  ERROR_IN_IDENTIFIER
                                | ERROR_IN_INTEGER_LITERAL
                                | ERROR_IN_STRING_LITERAL
                                | LEFT_PARENTHESIS_EXPECTED
                                  IDENTIFIER_EXPECTED
                                  INTEGER_EXPECTED
                                | RIGHT_PARENTHESIS_EXPECTED
                                | TICK_EXPECTED
                                  UNEXPECTED_SYMBOL
tape_status_type            ::=   OPENED
                                  MOUNT_REQUESTED
                                  MOUNTED
                                | LOADED
                                | CLOSED
unique_identifier           ::= string
unique_identifier_list      ::= ( [ unique_identifier{, unique_identifier } ] )
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

TABLE XXXVIII: Status Information List BNF (cont.)

Notes:

1. See Appendix D for a description of the notation used.
2. The syntactic constructs identifier, string, integer, float and list are CAIS list items defined in Section 9, Package CAIS_LIST_MANAGEMENT.
3. The syntactic constructs comprising constraint_item are described in Section 5.7.1.7 on page 328 and in Table XVII, page 330.
4. The construct unique_identifier is a string conforming to the syntax of a unique identifier (see Section 6.1, page 88).

The SUB CONDITION item is an optional implementation_defined integer code which further qualifies the exceptional condition. Implementations may define subcondition codes for any condition. There are no predefine sub conditions. 8

The PARAMETERS item specifies the names and values of the parameter(s) passed to the interface. Most commonly occurring parameter types can be represented by list items. The correspondence between parameter types and list item kinds is specified in the “parameter_value” entry of Table XXXIX on page 842. Formal parameters with mode OUT are not described in the PARAMETERS qualifier. 9

Additional items called qualifiers specify the context in which the exception has occurred (e.g., a node, relationship, attribute, definition, intent etc.). The set of predefine qualifiers included depends on the exception raised and the condition which caused it to be raised but is independent of the interface called. Implementations may define additional qualifiers. The predefine exceptional conditions and the associated qualifiers are described in Section 13.3 on page 862. 10



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

| TABLE XXXIX: Status Information List Description | |
|---|---|
| Syntactic Construct | Description |
| status_information_list | A named list containing the status information. The names of the package and interface called are always returned in the items PACKAGE and INTERFACE. Other items are included only if an exception is raised. CONDITION identifies the condition causing the exception. Implementations may return a SUBCONDITION item to identify a more specific condition. The PARAMETERS item is a named list containing the parameter(s) passed to the interface. Depending on the value of CONDITION, additional qualifiers (named items) are returned. |
| qualifier | A named item providing additional information about the condition that caused the exception to be raised. The possible qualifiers and the information provided are: |
| ALLOWED_USAGE | The identifier PRIMARY or SECONDARY indicating the kind of relationships which a relation may be used to create. |
| AMBIGUOUS_NAMES | Two identifiers for the same property. |
| ANCESTOR_DEFINITION | An NKD or RD which is a generalization of another specified the NKD or RD. |
| ATTRIBUTE | An AD. |
| ATTRIBUTE_KIND | An AKD. |
| ATTRIBUTES_LIST | A named list of attribute values. |
| ATTRIBUTE_NAME | A string specifying the name of an attribute. |
| BLOCK_SIZE | An integer specifying the number of bytes to be read or written in an input and output operation, block size for magnetic tape for 10 operations. |
| CHANNEL | A CHANNEL node. |
| CHANNEL_STATE | The state of a channel. |
| CHARACTERISTICS | A list specifying implementation-dependent characteristics for a file to be imported or exported. |
| CLASSIFICATION_ERROR | An error in a classification of a mandatory access control label. |
| CLASSIFICATION_LABEL | A list representing a mandatory access control label. |
| CLIENT_NODE | A client node. |
| COMPONENT | A specification for a particular component of an attribute value and the corresponding AKD (in conjunction with a specified AD). |
| CONFLICT | A conflict of constraints which would result from modification of a definition. |
| CONFLICTING_INTENTS | A list of required intents which are in conflict with existing locks on a node. |
| CONSTRAINT_ITEM | A constraint item which specifies a constraint on an attribute value. |
| CONSTRAINT_VIOLATED | An identifier item which identifies a constraint on an attribute which is violated by a value specified for the attribute. |
| CURRENT_LIST | The current linear list of a list. |
| CURRENT_LIST_1 | The current linear list of LIST_1. |



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

TABLE XXXIX: Status Information List Description (cont).

| Syntactic Construct | Description |
|---|---|
| CURRENT_LIST_2 | The current linear list of LIST_2. |
| DAY | An integer representing a day of a month. |
| DEADLOCK | True if a deadlock has been detected in obtaining access to node. |
| DEFINITION | A definition node. |
| DEFINITION_NAME | An identifier which is a name of a definition node. |
| DEFINITION_TYPE | The kind of a definition node or name. |
| DEVICE | A DEVICE node. |
| DIRECTION | The value of a DIRECTION attribute. |
| END_POSITION | An integer which specifies the position of the last item in a series of items in a list. |
| ERROR_ITEM | An integer specifying the position of an item, in the current linear list of a list, which is syntactically or semantically invalid in the context in which the list is used (e.g., an iteration predicate). |
| ERROR_SUBLIST | A current linear list specification used to identify a sublist of a list which contains a syntactically or semantically invalid item in the context in which the list is used (e.g., an iteration predicate). |
| EXPECTED_CHANNEL-_STATE | The expected initial state of the channel. |
| FACTOR | A contributing factor without which a constraints conflict would not occur. |
| FILE | A FILE node. |
| FILE_NAME | A string which is the name of a file. |
| GATEWAY | A GATEWAY node. |
| GRANT | A list representing a GRANT attribute value. |
| GROUP | A GROUP node. |
| IDENTIFIER_TEXT | A string containing an identifier. |
| IMPLIED_USAGE | The identifier PRIMARY or SECONDARY indicating the implied kind of relationship to be created using a relation. |
| INPUT_PARAMETER | The list value of the INPUT_PARAMETER attribute of a PROCESS node. |
| INVALID_ITEM | A lexical item in an external node set which is syntactically incorrect. |
| INTENTS | A list of intents toward a node to be accessed. |
| INTERPRETATION | An interpretation node. |
| INTERPRETATION | An NKI. |
| INTERPRETATION_2 | An RI. |
| INTERPRETATION_TYPE | The kind of an interpretation node. |
| ITEM_KIND | The kind of an item in a list. |
| ITEM_NAME | The name of an item in a list. |
| ITEM_POSITION | The position of an item in a list. |
| ITEM_VALUE | The value of an item in a list. |
| KEY | An AD which defines a key. |
| LIST | A list. |



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

TABLE XXXIX: Status Information List Description (cont).

| Syntactic Construct | Description |
|---|---|
| LIST_1 | The fist list in an operation. |
| LIST_2 | The second list in an operation. |
| LIST_KINDS | An unnamed list specifying the list kinds of two lists. |
| LIST_LENGTH | The length of a list. |
| LIST_TEXT | A string containing the text form of a list. |
| MEDIUM | A MEDIUM node. |
| MODE | An identifier which is an enumeration literal of a FILE_MODE type defined by an IO protocol package. |
| MONITOR | A MONITOR node. |
| MONTH | An integer representing a month of the year. |
| NAME | An identifier which is the name of a relation, attribute or key. |
| NAME_TYPE | The type of object identified by a name. |
| NEW_INTENTS | A new list of intents for an open node handle. |
| NODE | The unique-identifier of a node. |
| NODE_1 | The unique identifier of the first node in an operation. |
| NODE_2 | The unique identifier of the second node in an operation. |
| OTHER_CLIENT_NODE | The unique identifier of the other client node. |
| OTHER_DIRECTION | The value of the other DIRECTION attribute. |
| OTHER_PORT | The unique identifier of the other PORT RD. |
| PATHNAME | A string representing a pathname. |
| PATTERN | A pattern to be matched. |
| PORT | The unique identifier of a PORT RD. |
| PRAGMATIC | An identifier specifying the name of an Ada constant defined in CAIS_PRAGMATICS (see Section 12). |
| PREDICATE | A list representing an iteration predicate. |
| PRIORITY | An integer representing a process priority. |
| PROCESS | A PROCESS node. |
| PROCESSOR | A PROCESSOR node. |
| PROPERTY | A definition node which represents a property of another specified definition node. |
| PROPERTY_TYPE | The kind of a property. |
| PROTOCOL | A protocol name. |
| RELATION | An RD. |
| RELATIONSHIP | A relationship. |
| RELATIONSHIP_KEY | A string containing a relationship key list or relationship key pattern. |
| RELATION_NAME | A string containing the name of a relation. |
| REQUIRED_ITEM_KIND | The kind of list item required in some context. |
| REQUIRED_NKD | An NKD which defines the node kind required by an operation. |
| REQUIRED_RD | An RD which defies the relation required by an operation. |
| RESOURCE | A resource node. |



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

TABLE XXXIX: Status Information List Description (cent).

| Syntactic Construct | Description |
|---|---|
| RESOURCE_TYPE | The kind of a resource node. |
| RESTRICTION | A restriction on the deletion of either a definition node or a relationship that defines a property. |
| RIGHTS_NEEDED | A list of access rights required. |
| ROLE_TYPE | An indication whether the default role is of the current user or of the current process's executable image. |
| SECONDS | An integer number of intervals of length CAIS_SMALL_FOR_-CAIS_DURATION that specifies a time of day. |
| SOURCE_NODE | The source node of a relationship. |
| START_POSITION | An integer specifying the position of the fist item in a series of items in a list. |
| SYNTACTIC_ERROR | A particular syntactic error. |
| TAPE_STATUS | The status of a tape drive. |
| TARGET_NODE | The target node of a relationship. |
| UID | A string containing a unique identifier. |
| VALUE | The value of an attribute or key represented as a list. |
| VIEW | A VIEW node. |
| YEAR | An integer representing a year. |
| access_right | An access right identifier (see Table X in Section 5.4.4 on page 260). |
| AD_unique_identifier | The unique identifier of an AD. |
| base_type_restriction | An identifier item that specifies a restriction that would be violated by changing the AKD for an AD. The restrictions are: |
| constraints_conflict | The change would result in a specified conflicting constraint. |
| NONCONFORMING_-INSTANCES | Instances of the attribute exist and the intended new AKD disallows some values allowed by the current AKD. |
| channel_state_type | The state of a channel. |
| CHANNEL_unique_identifier | The unique identifier for a CHANNEL node. |
| classification_error_type | An identifier indicating an error in a mandatory access control label: |
| INVALID_HIERARCHICAL-_CATEGORY | The hierarchical category is invalid. |
| INVALID_NON_HIER-ARCHICAL_CATEGORY | The non-hierarchical category is invalid. |
| component identification | A named list identifying a component of a composite list or uniform list and the corresponding AKD (in conjunction with an ATTRIBUTE qualifier). The ITEM_POSITION item is the position of the component in the list. For a composite list component, the NAME and POSITION items provide the values of the NAME and POSITION keys on the corresponding COMPONENT relationship emanating from the corresponding composite |


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

TABLE XXXIX: Status Information List Description (cent).

| Syntactic Construct | Deacription |
|---|---|
| | list AKD. For a uniform list component, only ITEM_POSITION is included since there is only one COMPONENT relationship emanating from a uniform list AKD. |
| component_specification | A uniform list of component identifications that identifies a component (or sub-component) of an attribute value and the corresponding AKD (in conjunction with an ATTRIBUTE qualifier). The first component identification identifies a component of the attribute; each successive component identification identifies a sub-component of the component identified by the preceding component identifications. |
| condition_code | An integer value indicating the condition which caused the exception to be raised (see Section 13.3 on page 862). |
| constraint_violated | An identifier item identifying a constraint which is not satisfied by a specified attribute or component value. The follow constraint violations are defined: |
| BOUNDS | An UPPER_BOUND or LOWER_BOUND constraint is violated. |
| ENUMERATION_POSITION | The enumeration position of an Ada enumeration value or an enumerated identifier item in a list is not the same as the position minus one of the corresponding pattern (in the PATTERNS attribute of the AKD) which has the same image. |
| NAMING | A list which must be named is unnamed or vice versa. |
| ORDER | The named items in a named and ordered composite list are not in the required order. |
| PATTERNS | A string, identifier, or enumeration value does not conform to any allowed pattern in a PATTERNS constraint. |
| SIZE | The size of a uniform list is longer than the MAXIMUM_SIZE allowed or shorter the MINIMUM_SIZE required. |
| constraints_conflict | An identifier item specifying a conflict in the constraints in force on the property of a definition. A constraints_conflict_-factor (HAS_LOCAL_CONSTRAINT or HAS_OBTAINABLE_-INSTANCES) may be used in conjunction with a constraints_-conflict. The following constraint conflicts are defined: |
| ATTRIBUTE_BOUNDS | A range constraint LOWER_BOUND or UPPER_BOUND on an AD or AKD would be strengthened (HAS-LOCAL_-CONSTRAINT or HAS_OBTAINABLE_INSTANCES) or comprises a null value range. |
| ATTRIBUTE_DEFAULT | Multiple, different default values for an AD or AKD would be applied. |
| ATTRIBUTE_DEFAULT-_OUTSIDE_RANGE | A default value for an AD or AKD would be outside the range constraints LOWER_BOUND or UPPER_BOUND in force. |
| ATTRIBUTE_DEFAULT-_OUTSIDE_VALUE | A default value for an AD or AKD would be outside the value constraint VALUE in force. |



Source: https://assist.dla.mil -- Downloaded: 202610-26T07:54Z
Check the source to verify that this is the current version before use.

TABLE XXXIX: Status Information List Description (cont).

| Syntactic Construct | Description |
|---|---|
| ATTRIBUTE_NAMING | A naming constraint NAMING on an AD or AKD would be applied (HAS_LOCAL_CONSTRAINT or HAS_OBTAINABLE_-INSTANCES) or two different NAMING constraints would be in force. |
| ATTRIBUTE_PATTERNS | A patterns constraint PATTERNS on an AD or AKD would be strengthened (HAS_LOCAL_CONSTRAINT or HAS_-OBTAINABLE_INSTANCES) or comprise a null value range. |
| ATTRIBUTE_SIZE | A uniform list size constraint MINIMUM_SIZE or MAXIMUM_-SIZE on an AD or AKD would be strengthened (HAS-LOCAL_ -CONSTRAINT or HAS_OBTAINABLE_INSTANCES) or comprise a null range. |
| ATTRIBUTE_VALUE | A value constraint VALUE on an AD or AKD would be applied (HAS_LOCAL_CONSTRAINT or HAS_OBTAINABLE_ -INSTANCES) or multiple value constraints would not be the same. |
| ATTRIBUTE_VALUE-_OUTSIDE_RANGE | A value constraint VALUE on an AD or AKD would be outside the range constraints LOWER_BOUND or UPPER_BOUND in force. |
| RELATION_ALLOWED-_USAGE | An ALLOWED_USAGE constraint on an RD would be applied. (HAS_LOCAL_CONSTRAINT or HAS_OBTAINABLE_INSTANCES) or two different ALLOWED_USAGE constraints would be in force. |
| RELATION_BOUNDS | A MINIMUM_INSTANCES or MAXIMUM_INSTANCES constraint on an emanating relation would be strengthened (HAS_ -LOCAL_CONSTRAINT or HAS_OBTAINABLE_INSTANCES) or comprise a null range. |
| RELATION_TERMINATES | Instances of an RD (the property) can terminate at instances of an NKD (the definition) and the RD would have a generalization (the constraint) for which instances could not terminate. |
| RELATION_UNIQUE_KEYS | A UNIQUE_KEYS constraint on an RD would be applied (HAS_-OBTAINABLE_INSTANCES). |
| constraints_conflict_factor | An identifier which provides additional information for the interpretation of a constraints_conflict item. It specifies the contributing factor (if any) without which the particular conflict would not have occurred. The possible identifier items and the factors they indicate are: |
| HAS_LOCAL_CONSTRAINT | NKD or RD would inherit a constraint which is stronger than one defined by the NKD or RD itself. |
| HAS_OBTAINABLE_-INSTANCES | A constraint would be tightened which applies to a property of which instances exist. |



Source: https://assist.dla.mil -- Downloaded: 202610-26T07:54Z
Check the source to verify that this is the current version before use.

| TABLE XXXIX: Status Information List Description (cont). | |
|---|---|
| Syntactic Construct | Description |
| constraint_item | A constraint item representing a constraint on an attribute value (see Section 5.7.1.7 and Table XVII, page 330). Only the following types of constraint items are used: |
| value_constraint_item | A VALUE constraint for any attribute kind. |
| range_constraint_item | An UPPER_BOUND or LOWER_BOUND constraint for an integer or float value. |
| patterns_constraint_item | A PATTERNS constraint for a string or identifier. |
| naming-constraint item | A NAMING (NAMED or UNNAMED) constraint for a uniform list. |
| uniform_list_size_constraint_-item | A MINIMUM_SIZE or MAXIMUM_SIZE constraint for a uniform list. |
| current_linear_list_specification | The position of the current linear list in a list represented as an unnamed list (possibly empty) of item positions that specify the position of each sublist which contains the current linear list beginning with the sublist (if any) of the outermost list and ending with the position of the current linear list in its innermost containing list (if any). |
| definition_type | An identifier representing the kind of definition node identified by a DEFINITION qualifier (NKD, RD, AD or AKD). |
| definition_unique_identifier | The unique identifier of a definition node. The possible definition nodes are: |
| NKD_unique_identifier | An NKD. |
| RD_unique_identifier | An RD. |
| AD_unique_identifier | An AD. |
| AKD_unique_identifier | An AKD. |
| DEVICE_unique_identifier | The unique identifier of a DEVICE node. |
| direction_type | An identifier indicating the value of a DIRECTION attribute or NO_DIRECTION if the attribute is missing. |
| FILE_unique_identifier | The unique identifier of a FILE node. |
| GATEWAY_unique_identifier | The unique identifier of a GATEWAY node. |
| GROUP_unique_identifier | The unique identifier of a GROUP node. |
| intents_list | An unnamed list of intent specifications. |
| intent_specification | An identifier specifying an intent (see Table IV, page 98). |
| interpret at ion | The unique identifier of an interpretation node: |
| NKI_unique_identifier | An NKI. |
| RI_unique_identifier | An RI. |
| interpretation_type | An identifier specifying a kind of interpretation node (NKI or RI). |
| item_value | Any type of list item value. |


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

| TABLE XXXIX: Status Information List Description (cont). | |
|---|---|
| Syntactic Construct | Description |
| key_list | A unnamed uniform list of key value specifications used to identify a relationship. |
| key_value | An identifier or the empty string representing the value of a key. |
| key_value_specification | A named composite list that specifies a KEY and a corresponding key VALUE. The KEY is identified by the unique identifier of the AD that defines the key. |
| list_item_kind | An identifier indicating the kind of a list item (LIST, IDENTIFIER, INTEGER, STRING, or FLOAT). |
| list_kind | An identifier indicating the kind of a list (EMPTY, NAMED, or UNNAMED). |
| MEDIUM_unique_identifier | The unique identifier of a MEDIUM node. |
| MONITOR_unique_identifier | The unique identifier of a MONITOR node. |
| name_type | An identifier indicating whether a name is a relation name, an attribute name or a key name (RELATION, ATTRIBUTE or KEY). |
| NKD_unique_identifier | The unique identifier of an NKD. |
| NKI_unique_identifier | The unique identifier of an NKI. |
| node_deletion_restriction | An identifier item specifying a restriction which would be violated by the deletion of a definition node. The possible identifier items and the restriction each indicates are: |
| HAS_OBTAINABLE_ATTRIBUTE | AKD is the attribute kind for an obtainable AD. |
| HAS_OBTAINABLE_INSTANCE | NKD has an obtainable instance. |
| HAS_OBTAINABLE_SPECIALIZATION | NKD or RD has an obtainable specialization. |
| IN_ADOPTED_VIEW | Definition node is in an adopted view. |
| IN_MULTIPLE_VIEWS | Definition node is in multiple views. |
| IS_AN_ALTERNATIVE | AKD is an alternative of another AKD. |
| IS_AN_ATTRIBUTE_PROPERTY | AD is the target of an ATTRIBUTE relationship emanating from an obtainable NKD or RD. |
| IS_A_COMPONENT | AKD is a component of another AKD. |
| IS_AN_EMANATES_PROPERTY | RD is the target of an EMANATES relationship from an obtainable NKD. |
| IS_A_KEY_PROPERTY | AD is the target of a KEY relationship emanating from an obtainable RD. |
| IS_A_TERMINATES_PROPERTY | RD is the target of a TERMINATES relationship from an obtainable NKD. |
| IS_PREDEFINED | Definition is predefine. |
| parameter | A named item representing a parameter passed to a CAIS interface and its value. The item name is the parameter name. The item's value is the parameter's value. |


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

TABLE XXXIX: Status Information List Description (cont).

| Syntactic Construct | Description |
|---|---|
| parameter_name | The Ada identifier for a parameter to the interface called. |
| parameter_value | A representation of a parameter value which depends on the type of the parameter as follows: |
| float | A parameter value which is representable as a floating point item in a CAIS list with CAIS_PRAGMATICS_LIST_MAXIMUM_-DIGITS accuracy. |
| identifier | A parameter value which is an Ada enumeration value (the identifier is the result of value'IMAGE) or a value of the type TOKEN-TYPE in CAIS_LIST_MANAGEMENT (which is not the undefined token). |
| integer | A parameter value which is an integer representable as an integer item in a CAIS list. |
| intents_list | A parameter value of type INTENT_ARRAY. |
| list | A parameter value which is a CAIS list. |
| string | A parameter value which is a STRING representable as a string item in a CAIS list. |
| unique_identifier | A parameter value which is an open node handle. |
| unique_identifier-list | A parameter value which is an array of open node handles. |
| "" (The empty string.) | Any other parameter value. |
| PROCESS_unique_identifier | The unique identifier of a PROCESS node. |
| PROCESSOR_unique_identifier | The unique identifier of a PROCESSOR node. |
| property_deletion_restriction | An identifier item specifying a restriction which would be violated by the deletion of a property definition relationship. The possible identifier items and the restriction each indicates are: |
| IN_ADOPTED_VIEW | The definition relationship has a representation (ATTRIBUTE_-NAME, KEY_NAME or RELATION_NAME) in an adopted view, or emanates from an AKD in an adopted view. |
| IN_MULTIPLE_VIEWS | The definition relationship has a representation (ATTRIBUTE_-NAME, KEY_NAME or RELATION_NAME) in multiple views. |
| IS_PREDEFINED | The definition relationship is predefine. |
| PROPERTY_INSTANCE | A node attribute, relationship or relationship attribute exists which would not be permitted were it not for the property definition relationship. |
| RELATION_TERMINATES | An NKD (the definition) has a TERMINATES relationship to an RD (the property) which is only allowed because the TERMINATES relationship to be deleted terminates at a generalization of the RD. |


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

| TABLE XXXIX: Status Information List Description (cent). | |
|---|---|
| Syntactic Construct | Description |
| property_type | The name of the predefine relation which defines a property. The property type identifiers and the properties they identify are: |
| ALTERNATIVE | An alternative of a variant AKD. |
| ATTRIBUTE | An attribute of a node kind or relation. |
| ATTRIBUTE_KIND | The attribute kind of an attribute. |
| COMPONENT | A component of a uniform list or composite list AKD. |
| EMANATES | An emanating relation of a node kind. |
| KEY | A key of a relation. |
| TERMINATES | A terminating relation of a node kind. |
| property_unique_identifier | The unique identifier of a property definition node: |
| RD_unique_identifier | An RD for an emanating relation. |
| AD_unique_identifier | An AD for an attribute. |
| AKD_unique_identifier | An AKD for the kind of an attribute. |
| RD_unique_identifier | The unique identifier of an RD. |
| relationship-kind | The type of a relationship (primary or secondary). |
| PRIMARY | Primary relationship. |
| SECONDARY | Secondary relationship. |
| RL_unique_identifier | The unique identifier of an RI. |
| relationship_specification | The identification of a relationship in the form of a named list having the following items: |
| BASE | The unique identifier for the base node of the relationship. |
| RELATION | The unique identifier of the corresponding RD. |
| KEY | A key-list specifying key names and values. The KEY item is omitted if RELATION has no keys. |
| resource_type | An identifier specifying type of resource represented by a resource node (DEVICE, GATEWAY, MEDIUM, PROCESSOR). |
| rights_list | An unnamed list of access rights. |
| role_type | The identifier item IMAGE or USER, indicating whether a default role is the default role of the executable image of the current process or of the current user, respectively. |
| subcondition_code | An implementation-defined integer value which may indicate more precisely the cause of an exceptional condition. |


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

| TABLE XXXIX: Status Information List Description (cont). | |
|---|---|
| Syntactic Construct | Description |
| syntactic_error_type | An identifier for a particular type of syntactic error. It must have one of the following values: |
| ARROW_EXPECTED | The symbol "=>" is required. |
| COMMA_OLRIGHT-_PARENTHESIS-_EXPECTED | Either a comma or a right parenthesis is required. |
| LEFT_PARENTHESIS-_EXPECTED | A left parenthesis is required. |
| ERROR_IN_DECIMAL-_LITERAL | Incorrect float syntax. |
| ERROR_IN_IDENTIFIER | Incorrect identifier syntax. |
| ERROR_INTEGER--LITERAL | Incorrect integer syntax. |
| ERROR-IN_ STRING-_LITERAL | Incorrect string syntax. |
| IDENTIFIER_EXPECTED | An identifier is required. |
| INTEGER_EXPECTED | An integer is required. |
| RIGHT_PARENTHESIS-_EXPECTED | A right parenthesis is required. |
| TICK_EXPECTED | A tick mark is required. |
| UNEXPECTED_SYMBOL | Any other violation of correct syntax. |
| tape_status_type | A tape drive status. |
| unique_identifier | A string containing the unique identifier of a node (see Section 5.1 on page 88). |


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

# 13.1 Package CAIS_STATUS_DEFINITIONS

This package defines the Ada type STATUS_INDICATOR. It also defines the enumeration type EXCEPTION_INDICATION_KIND, the deferred constant NO_INDICATION and condition code constants. 1

```
type STATUS_INDICATOR is limited private;

NO_INDICATION : constant STATUS_INDICATOR;
```

STATUS_INDICATOR describes the type for status indicators. 2

NO_INDICATION is an implementation-defied deferred constant denoting the value of an established status indicator which does not acquire status information. 3

```
typo EXCEPTION_INDICATION_KIND is
    (NO_ERROR_DETECTED,                -- No exception raised.

  -- For user-defined exceptions:

     USER_DEFINED_ERROR_DETECTED,

  -- For exceptions declared in CAIS_DEFINITIOIS:

     ACCESS_VIOLATION_DETECTED,
     AMBIGUOUS_VIEW_ERROR_DETECTED,
     ATTRIBUTE_ERROR_DETECTED,
     CONFORMANCE_ERROR_DETECTED,
     DEFINITION_ERROR_DETECTED,
     DEVICE_ERROR_DETECTED,
     EXISTING_NODE_ERROR_DETECTED,
     INTENT_VIOLATION_DETECTED,
     ITERATOR_ERROR_DETECTED,
     LOCK_ERROR_DETECTED,
     NAME_ERROR_DETECTED,
     NODE_KIND_ERROR_DETECTED,
     NOT_IN_VIEW_ERROR_DETECTED,
     PATHNAME_SYNTAX_ERROR_DETECTED,
     PREDEFINED_ATTRIBUTE_ERROR_DETECTED,
     PREDEFINED_RELATION_ERROR_DETECTED,
     RELATIONSHIP_ERROR_DETECTED,
     RESOURCE_ALLOCATION_ERROR_DETECTED,
     RESOURCE_AVAILABILITY_ERROR_DETECTED,
     SECURITY_VIOLATION_DETECTED,
     STATUS_ERROR_DETECTED,
     SYNTAX_ERROR_DETECTED,
     TRANSACTION_ERROR_DETECTED,
     USE_ERROR_DETECTED,
```


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
    -- For exceptions declared in CAIS_PROCESS_DEFINITIONS:

       EXECUTABL_IMAGE_ERROR_DETECTED,

    -- For exceptions declared in CAIS_IO_DEFINITIONS:

       CLIENT_ERROR_DETECTED,
       DATA_ERROR_DETECTED,
       END_ERROR_DETECTED,
       LAYOUT_ERROR_DETECTED,
       MODE_ERROR_DETECTED,
       OPEN_DENIED_DETECTED,
       PORT-ERROR-DETECTED,
       PROTOCOL_ERROR_DETECTED,
       TIME_EXPIRED_DETECTED,

    -- For exceptions declared in CAIS_FRAHE_IO:

       QUEUE_FULL_DETECTED,
       QUEUE_EMPTY_DETECTED,

    -- For exceptions declared in CAIS_MAGNETIC_TAPE_IO:

       TAPE_STATUS_ERROR_DETECTED,

    -- For exceptions declared in CAIS_TERMINAL_IO:

       BUTTON_NAME_ERROR_DETECTED,
       FUNCTION_KEY_STATUS_ERROR_DETECTED,
       INPUT_EVENT_QUEUE_EMPTY_DETECTED,
       LOCATOR_STATUS_ERROR_DETECTED,
       SCREEN_NOT_ON_DISPLAY_DETECTED,
       SCREEN_POSITION_ERROR_DETECTED,
       VIRTUAL_DISPLAY_POSITION_ERROR_DETECTED,

    -- For exceptions declared in CAIS_COMMON_IO:

       UID_ERROR_DETECTED,

    -- For exceptions declared in CAIS_LIST_MANAGEMENT:

       ITEM_KIND_ERROR_DETECTED,
       LIST_KIND_ERROR_DETECTED,
       LIST_POSITION_ERROR_DETECTED,
       LIST_SYNTAX_ERROR_DETECTED,
       NAMED_LIST_ERROR_DETECTED,
       SEARCH_ERROR_DETECTED,
       TOKEN_ERROR_DETECTED,
```


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
        -- For exceptions declared in CAIS_CALENDAR:

            TIME_ERROR_DETECTED,

        -- For exceptions declared in CAIS_PRAGMATICS:

            CAPACITY_ERROR_DETECTED,
            RESOURCE_ERROR_DETECTED,

        -- For exceptions declared in CAIS_FORM_TERMINAL_IO:

            FORM_STATUS_ERROR_DETECTED,
            FORM_FUNCTION_KEY_STATUS_ERROR_DETECTED,
            TERMINAL_POSITION_ERROR_DETECTED);
```

EXCEPTION_INDICATION_KIND is the type of a status indicator's exception indication. When a status indicator's exception indication has the EXCEPTION_INDICATION_KIND value NO_ERROR_DETECTED, it indicates the absence of any condition corresponding to an exception. When a status indicator's exception indication value is USER_DEFINED_ERROR_DETECTED it indicates that a condition corresponding to an exception which is not defined by the CAIS has been detected. The value USER_DEFINED_ERRROR_DETECTED is provided for user-defined extensions to the CAIS interface set which use status indicators to return status information (see procedure SET_INDICATOR in Section 13.2.2 on page 857). The remaining exception indication values indicate that a condition for a CAIS-defined exception has been detected. The enumeration values of these exception indications are formed by the name of the exception followed by "_DETECTED", except for LIST_SYNTAX_ERROR_DETECTED which corresponds to the exception SYNTAX_ERROR declared in CAIS_LIST_MANAGEMENT. The enumeration literals corresponding to implementation-defined exceptions in package CAIS_PRAGMATICS should be added by the implementer at the end of the enumeration. 4

```
    subtype MESSAGE_TEXT is STRING;
```

MESSAGE_TEXT is a subtype for implementation-defined natural language messages returned by the MESSAGE interface. 5

```
    subtype CONDITION_CODE is CAIS_INTEGER;
```

CONDITION_CODE is the subtype for conditions codes returned in the items named CONDITION and SUBCONDITION of a status information list. The following constants of type CONDITION_CODE identify the predefined exceptional conditions which may cause each CAIS exception to be raised. These identifiers for condition codes are provided for use in programs to avoid the use of numbers and thus make programs more readable. 6

The condition codes for ACCESS_VIOLATION: 7

```
    ACC_INSUFFICIENT_RIGHTS                 : constant CONDITION_CODE : = 1;
    ACC_ACCESS_RESTRICTION                  : constant CONDITION_CODE := 2;
    ACC_DENY_RESTRICTION                    : constant CONDITION_CODE := 3;
    ACC_INSUFFICIENT_RIGHTS_PREDICATE       : constant CONDITION_CODE : = 4;
    ACC_INSUFFICIENT_DEFAULT_RIGHTS         : constant CONDITION_CODE : = 5;
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

The condition codes for AMBIGUOUS_VIEW_ERROR: 8

```
AMB_NAME_TRANSLATION          : constant CONDITION_CODE : = 1;
AMB_NAME_FORMATION            : constant CONDITION_CODE := 2;
AMB_NAME_CREATION             : constant CONDITION_CODE := 3;
AMB_KEY_VALUE_ASSOCIATION     : constant CONDITION_CODE := 4;
```

The condition codes for ATTRIBUTE-ERROR: 9

```
ATT_NODE_ATTRIBUTE_EXISTS                : constant CONDITION_CODE := 1;
ATT_RELATIONSHIP_ATTRIBUTE_EXISTS        : constant CONDITION_CODE := 2;
ATT_NONEXISTENT_NODE_ATTRIBUTE           : constant CONDITION_CODE := 3;
ATT_NONEXISTENT_RELATIONSHIP_ATTRIBUTE : constant CONDITION_CODE := 4;
ATT_INVALID_ATTRIBUTE_REFERENCE          : constant CONDITION_CODE := 5;
ATT_ILLEGAL_ATTRIBUTES_LIST              : constant CONDITION_CODE := 6;
```

The condition codes for CONFORMANCE-ERROR: 10

```
CON_RELATIONSHIP_CANNOT_EMANATE      : constant CONDITION_CODE := 1;
CON_RELATIONSHIP_CANNOT_TERMINATE    : constant CONDITION_CODE := 2;
CON_EMANATES_BOUND_VIOLATION         : constant CONDITION_CODE := 3;
CON_UNDEFINED_ATTRIBUTE              : constant CONDITION_CODE := 4;
CON_ATTRIBUTE_NOT_MODIFIABLE         : constant CONDITION_CODE := 6;
CON_ATTRIBUTE_NOT_DELETABLE          : constant CONDITION_CODE := 6;
CON_UNDEFINED_COMPONENT              : constant CONDITION_CODE := 7;
CON_MISSING_COMPONENT                : constant CONDITION_CODE := 8;
CON_NO_ALTERNATIVE_MATCHED           : constant CONDITION_CODE := 9;
CON_CONSTRAINT_VIOLATION             : constant CONDITION_CODE := 10;
CON_RELATION_USAGE_VIOLATION         : constant CONDITION_CODE := 11;
```

The condition codes for DEFINITION_ERROR: 11

```
DEF_DEFINITION_NOT_DELETABLE         : constant CONDITION_CODE := 1;
DEF_PROPERTY_NOT_DELETABLE           : constant CONDITION_CODE := 2;
DEF_CONFLICTING_CONSTRAINTS          : constant CONDITION_CODE := 3;
DEF_ILLEGAL_BASE_TYPE                : constant CONDITION_CODE := 4;
DEF_UNNAMED_UNORDERED_AKD            : constant CONDITION_CODE := 5;
DEF_LAST_INTERPRETATION              : constant CONDITION_CODE := 6;
DEF_LAST_DEFINITION_NAMING           : constant CONDITION_CODE := 7;
DEF_LAST_PROPERTY_NAMING             : constant CONDITION_CODE := 8;
DEF_CAIS_CONTROLLED_CONSTRAINTS      : constant CONDITION_CODE := 9;
DEF_CAIS_CONTROLLED_SUBLATTICE       : constant CONDITION_CODE := 10;
```

The condition codes for DEVICE_ERROR are implementation defined. 12

The condition codes for EXISTING_NODE_ERROR: 13

```
EXI_NODE_ALREADY_EXISTS              : constant CONDITION_CODE := 1;
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

The condition codes for INTENT_VIOLATION: 14

```
INT_INSUFFICIENT_RIGHTS                : constant CONDITION_CODE := 1;
```

The condition codes for ITERATOR_ERROR: 15

```
ITE.ITERATOR_NOT_ESTABLISHED           : constant CONDITION_CODE := 1;
ITE_ITERATOR_ESTABLISHED               : constant CONDITION_CODE := 2;
ITE_ITERATOR_EXHAUSTED                 : constant CONDITI0N_CODE := 3;
```

The condition codes for LOCK_ERROR: 16

```
LOE_INTENT_CONFLICT                    : constant CONDITION_CODE := 1;
LOE_INTENT_CONFLICT_PREDICATE          : constant CONDITION_CODE := 2;
LOE_TRANSACTION_AFFECTS_DEFINITION     : constant CONDITION_CODE := 3;
```

The condition codes for NAME_ERROR: 17

```
NAM_INVALID_ACCESS                     : constant CONDITION_CODE := 1;
NAM_INVALID_UID_OPEN                   : constant CONDITION_CODE := 2;
NAM_INVALID_ACCESS_PREDICATE           : constant CONDITION_CODE := 3;
NAM_NO_NODE_WITH_UID                   : constant CONDITION_CODE := 4;
```

The condition codes for NODE_KIND_ERROR: 18

```
NKE_INAPPROPRIATE_INSTANCE             : constant CONDITION_CODE := 1;
NKE_INAPPROPRIATE_NKD                  : constant CONDITION_CODE := 2;
NKE_INAPPROPRIATE_RD                   : constant CONDITION_CODE := 3;
NKE_NKD_CAIS_CONTROLLED                : constant CONDITION_CODE := 4;
NKE_NODE_CAIS_CONTROLLED               : constant CONDITION_CODE := 5;
NKE_ILLEGAL_CONVERSION                 : constant CONDITION_CODE := 6;
```

The condition codes for NOT_IN_VIEW_ERROR: 19

```
NIV_PROPERTY_NAME_TRANSLATION          : constant CONDITION_CODE := 1;
NIV_DEFINITION_NAME_TRANSLATION        : constant CONDITION_CODE := 2;
NIV_NAME_FORMATION                     : constant CONDITION_CODE := 3;
NIV_VACUOUS_PATTERN                    : constant CONDITION_CODE := 4;
NIV_KEY_VALUE_ASSOCIATION              : constant CONDITION_CODE := 5;
```

The condition codes for PATHNAME_SYNTAX_ERROR: 20

```
PSY_PATHNAME                           : constant CONDITION_CODE := 1;
PSY_RELATIONSHIP_KEY                   : constant CONDITION_CODE := 2;
PSY_RELATION_NAME                      : constant CONDITION_CODE := 3;
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

The condition codes for PREDEFINED_ATTRIBUTE_ERROR: 21

```
PAE_CAIS_CONTROLLED_NODE_ATTRIBUTE          : constant CONDITION_CODE := 1;
PAE_CAIS_CONTROLLED_RELATIONSHIP_ATTRIBUTE
                                            : constant CONDITION_CODE := 2;
PAE_CAIS_CONTROLLED_AD                      : constant CONDITION_CODE := 3;
```

The condition codes for PREDEFINED_RELATION_ERROR: 22

```
PRE_RELATION_CAIS_CONTROLLED                : constant CONDITION_CODE := 1;
PRE_INHERITANCE_NOT_CHANGEABLE              : constant CONDITION_CODE := 2;
PRE_ILLEGAL_CONVERSION                      : constant CONDITION_CODE := 3;
PRE_NOT_JOB_RELATIONSHIP                    : constant CONDITION_CODE := 4;
PRE_RD_CAIS_CONTROLLED                      : constant CONDITION_CODE := 5;
```

The condition codes for RELATIONSHIP_ERROR: 23

```
RER_IDENTIFICATION_NOT_UNIQUE               : constant CONDITION_CODE := 1;
RER_NO_SUCH_RELATIONSHIP                    : constant CONDITION_CODE := 2;
RER_RELATIONSHIP_ALREADY_EXISTS             : constant CONDITION_CODE := 3;
RER_NODES_NOT_RELATED                       : constant CONDITION_CODE := 4;
RER_ILLEGAL_KEY_LIST                        : constant CONDITION_CODE := 5;
RER_N0_UNIQUE_KEY_SET                       : constant CONDITION_CODE := 6;
```

The condition codes for RESOURCE_ALLOCATION_ERROR: 24

```
RAL_RESOURCE_NOT_ALLOCATABLE                : constant CONDITION_CODE := 1;
RAL_INVALID_PROCESS_NODE_MEDIUM             : constant CONDITION_CODE := 2;
```

The condition codes for RESOURCE_AVAILABILITY_ERROR: 25

```
RAV_RESOURCE_NOT_AVAILABLE                  : constant CONDITION_CODE := 1;
```

The condition codes for SECURITY_VIOLATION are implementation-defined. 26

The condition codes for STATUS_ERROR: 27

```
STA_ALREADY_OPEN                            : constant CONDITION_CODE := 1;
STA_NOT_OPEN                                : constant CONDITION_CODE := 2;
STA_NOT_ESTABLISHED                         : constant CONDITION_CODE := 3;
STA_ALREADY_ESTABLISHED                     : constant CONDITION_CODE := 4;
STA_TRANSACTION_ALREADY_OPEN                : constant CONDITION_CODE := 5;
STA_TRANSACTION_NOT_OPEN                    : constant CONDITION_CODE := 6;
STA_FILE_ALREADY_OPEN                       : constant CONDITION_CODE := 7;
STA_FILE_NOT_OPEN                           : constant CONDITION_CODE := 8;
STA_UNEXPECTED_CHANNEL_STATE                : constant CONDITION_CODE := 9;
STA_SCREEN_ALREADY_OPEN                     : constant CONDITION_CODE := 10;
STA_SCREEN_NOT_OPEN                         : constant CONDITION_CODE := 11;
STA_NODE_SET_ALREADY_OPEN                   : constant CONDITION_CODE := 12;
STA_NODE_SET_NOT_OPEN                       : constant CONDITION_CODE := 13;
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

The condition codes for CAIS_DEFINITIONS.SYNTAX_ERROR: 28

```
SYN_ILLEGAL_UID                          : constant CONDITION_CODE := 1;
SYN_ILLEGAL_RELATION_NAME_PATTERN        : constant CONDITION_CODE := 2;
SYN_ILLEGAL_RELATIONSHIP_KEY_PATTERN     : constant CONDITION_CODE := 3;
SYN_ILLEGAL_PREDICATE_SYNTAX             : constant CONDITION_CODE := 4;
SYN_ILLEGAL_PREDICATE                    : constant CONDITION_CODE := 5;
SYN_ILLEGAL_ATTRIBUTE_NAME_PATTERN       : constant CONDITION_CODE := 6;
SYN_ILLEGAL_ACCESS_RIGHT_IDENTIFIER      : constant CONDITION_CODE := 7;
SYN_ILLEGAL_ATTRIBUTES_LIST              : constant CONDITION_CODE := 8;
SYN_ILLEGAL_MANDATORY_ACCESS             : constant CONDITION_CODE := 9;
SYN_ILLEGAL_DISCRETIONARY_ACCESS         : constant CONDITION_CODE := 10;
SYN_ILLEGAL_ATTRIBUTE_NAME               : constant CONDITION_CODE := 11;
SYN_ILLEGAL_RELATION_NAME                : constant CONDITION_CODE := 12;
SYN_ILLEGAL_PARAMETER_LIST               : constant CONDITION_CODE := 13;
SYN_ILLEGAL_DEFINITION_NAME              : constant CONDITION_CODE := 14;
SYN_ILLEGAL_EXTERNAL_NODE_SET            : constant CONDITION_CODE := 15;
```

The condition codes for TRANSACTION_ERROR: 29

```
TRN_AMBIGUOUS_INCLUSION                  : constant CONDITION_CODE := 1;
TRN_NODE_NOT_INCLUDED                    : constant CONDITION_CODE := 2;
TRN_FILE_NOT_INCLUDED                    : constant CONDITION_CODE := 3;
```

The condition codes for USE_ERROR: 30

```
USE_VACUOUS_INTENT                       : constant CONDITION_CODE := 1;
USE_ILLEGAL_CHANGE_INTENT                : constant CONDITION_CODE := 2;
USE_NO_PATH_ASSOCIATED_WITH_HANDLE       : constant CONDITION_CODE := 3;
USE_NO_POSSIBLE_BASE_PATH                : constant CONDITION_CODE := 4;
USE_ILLEGAL_PRIMARY                      : constant CONDITION_CODE := 5;
USE_CHANNEL_IN_USE                       : constant CONDITION_CODE := 6;
USE_HAS_EMANATING_PRIMARIES              : constant CONDITION_CODE := 7;
USE_NOT_SECONDARY_RELATIONSHIP           : constant CONDITION_CODE := 8;
USE_ILLEGAL_CONTENTS_REPLACEMENT         : constant CONDITION_CODE := 9;
USE_ILLEGAL_MANDATORY_ACCESS             : constant CONDITION_CODE := 10;
USE_RELATION_IS_NOT_BIDIRECTIONAL        : constant CONDITION_CODE := 11;
USE_RELATION_IS_BIDIRECTIONAL            : constant CONDITION_CODE := 12;
USE_INAPPROPRIATE_NKI                    : constant CONDITION_CODE := 13;
USE_NOT_ADOPTED_NKI                      : constant CONDITION_CODE := 14;
USE_HAS_NO_ADOPTED_NKI                   : constant CONDITION_CODE := 15;
USE_ALREADY_IN_VIEW                      : constant CONDITION_CODE := 16;
USE_ILLEGAL_SPECIALIZATION               : constant CONDITION_CODE := 17;
USE_ILLEGAL_GENERALIZATION               : constant CONDITION_CODE := 18;
USE_ANCESTOR_IS_BIDIRECTIONAL            : constant CONDITION_CODE := 19;
USE_DEFINITION_NOT_IN_VIEW               : constant CONDITION_CODE := 20;
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
USE_DISCONNECTED_SUBLATTICE                 : constant CONDITION_CODE := 21;
USE_PROPERTY_ALREADY_DEFIHED                : constant CONDITION_CODE := 22;
USE_PROPERTY_NOT_DEFIEED                    : constant CONDITION_CODE := 23;
USE_PREDEFINED_KEY                          : constant CONDITION_CODE := 24;
USE_NODE_SPECIFIED_TWICE                    : constant CONDITION_CODE := 25;
USE_NOT_DEFINITION_ATTRIBUTE                : constant CONDITION_CODE := 26;
USE_NOT_PROPERTY_ATTRIBUTE                  : constant CONDITION_CODE := 27;
USE_RELATIONSHIP_NOT_BIDIRECTIONAL          : constant CONDITION_CODE := 28;
USE_HANDLER_PROCESS_NOT_ACTIVE              : constant CONDITION_CODE := 29;
USE_PROCESS_ALREADY_HANDLER                 : constant CONDITION_CODE := 30;
USE_DUPLICATE_HANDLER                       : constant CONDITION_CODE := 31;
USE_DIFFERENT_USER                          : constant CONDITION_CODE := 32;
USE_NODE_ATTRIBUTE_ALREADY_MONITORED        : constant CONDITION_CODE := 33;
USE_NO_ATTRIBUTES_MONITORED                 : constant CONDITION_CODE := 34;

USE_CANNOT_BE_MONITORED                     : constant CONDITION_CODE := 36;
USE_CONTENTS_STRUCTURE_UNKNOWN              : constant CONDITION_CODE := 36;
USE_NOT_EVENT_HANDLER_PROCESS               : constant CONDITION_CODE := 37;
USE_ILLEGAL_PRIORITY                        : constant CONDITION_CODE := 38;
USE_PRIORITY_RANGE_NOT_DETERMINABLE         : constant CONDITION_CODE := 39;
USE_INVALID_PRIORITY_MODIFICATION           : constant CONDITION_CODE := 40;
USE_NO_MULTIPLE_FILE_HANDLES                : constant CONDITION_CODE := 41;
USE_BAD_BLOCK_SIZE                          : constant CONDITION_CODE := 42;
USE_ILLEGAL_FILE_HANDLE_RESET               : constant CONDITION_CODE := 43;
USE_INVALID_MODE                            : constant CONDITION_CODE := 44;
USE_NO_HOST_SYSTEM                          : constant CONDITION_CODE := 46;
USE_INVALID_HOST_FILE_NAME                  : constant CONDITION_CODE := 46;
USE_ILLEGAL_CONTENTS_CHARACTERISTICS        : constant CONDITION_CODE := 47;
USE_NO_POTENTIAL_MEMBER                     : constant CONDITION_CODE := 48;
USE_DEFAULT_ROLE                            : constant CONDITION_CODE := 49;

USE_PROCESS_STILL_ACTIVE                    : constant CONDITION_CODE := 50;
USE_KEY_ATTRIBUTE_NAMED                     : constant CONDITION_CODE := 51;
USE_RELATION_NOT_BIDIRECTIONAL              : constant CONDITION_CODE := 52;
USE_WRONG_DIRECTIONALITY                    : constant CONDITION_CODE := 53;
USE_INCOMPATIBLE_CONTENT_STRUCTURE          : constant CONDITION_CODE := 54;
USE_INCOMPATIBLE_CLIENT                     : constant CONDITION_CODE := 55;
USE_PROCESS_NOT_ACTIVATED                   : constant CONDITION_CODE := 56;
USE_PROCESS_CANNOT_BE_ACTIVATED             : constant CONDITION_CODE := 57;
USE_TRANSACTION_NOT_COMPLETED               : constant CONDITION_CODE := 58;
USE_NO_INTENT_TO_ACCESS                     : constant CONDITION_CODE := 59;
USE_PREDEFINED_NODE_ATTRIBUTE               : constant CONDITION_CODE := 60;
USE_PREDEFINED_PATH_ATTRIBUTE               : constant CONDITION_CODE := 61;
USE_PREDEFINED_NODE                         : constant CONDITION_CODE := 62;
USE_PREDEFINED_RELATIOHSHIP                 : constant CONDITION_CODE := 63;
USE_INAPPROPRIATE_LINE_LENGTH               : constant CONDITION_CODE := 64;
USE_INAPPROPRIATE_PAGE_LENGTH               : constant CONDITION_CODE := 66;
USE_INVALID_FORM_WRITE                      : constant CONDITION_CODE := 66;
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
    USE_INVALID_FORM_SIZE                : constant CONDITION_CODE := 67;
    USE_AREA_QUALIFIER-COEFLICT          : constant CONDITION_CODE := 68;
```

31

The condition codes for EXECUTABLE_IMAGE_ERROR:

```
    EXE_NOT_EXECUTABLE_IMAGE             : constant CONDITION_CODE := 1;
```

32

The condition codes for CLIENT_ERROR:

```
    CLI_CALLER_NOT_CLIENT                : constant CONDITION_CODE := 1;
    CLI_WRONG_CLIENT_KIND                : constant CONDITION_CODE := 2;
    CLI_PROCESS_OUT_OF_EXECUTION         : constant CONDITION_CODE := 3;
    CLI_NO_PROCESS                       : constant CONDITION_CODE := 4;
    CLI_NOT_CLIENT                       : constant CONDITION_CODE := 5;
```

33

The condition codes for DATA_ERROR:

```
    DAT_WRONG_TYPE                       : constant CONDITION_CODE := 1;
    DAT_OUT_OF_RANGE                     : constant CONDITION_CODE := 2;
    DAT_SYNTAX                           : constant CONDITION_CODE := 3;
```

34

The condition codes for END_ERROR:

```
    END_ATTEMPT_TO_PASS_END              : constant CONDITION_CODE := 1;
    END_BUFFER_EMPTY                     : constant CONDITION_CODE := 2;
```

35

The condition codes for LAYOUT_ERROR:

```
    LAY_OUT_OF_RANGE                     : constant CONDITION_CODE := 1;
    LAY_OUT_OF_BOUNDS                    : constant CONDITION_CODE := 2;
    LAY_STRING_TOO_LONG                  : constant CONDITION_CODE := 3;
    LAY_BOUNDED                          : constant CONDITION_CODE := 4;
```

36

The condition codes for MODE_ERROR:

```
    MOD_OPERATION_CONFLICT               : constant CONDITION_CODE := 1;
    MOD_DIRECTION_CONFLICT               : constant CONDITION_CODE := 2;
    MOD_MODE_CONFLICT                    : constant CONDITION_CODE := 3;
```

37

The condition codes for OPEN_DENIED_ERROR:

```
    OPD_CHANNEL_DELETED                  : constant CONDITION_CODE := 1;
    OPD_OPPOSITE_OPEN_FAILED             : constant CONDITION_CODE := 2;
```

38

The condition codes for PORT_ERROR:



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
POE_INCOMPATIBLE_RD                    : constant CONDITION_CODE := 1;
POE_INCOMPATIBLE_DIRECTIONS            : constant CONDITION_CODE := 2;
```

The condition codes for PROTOCOL_ERROR: 39

```
PRO_NOT_ALLOWED                        : constant CONDITION_CODE := 1;
PRO_DIFFERENT_PROTOCOLS                : constant CONDITION_CODE := 2;
PRO_RANGES_INCOMPATIBLE                : constant CONDITION_CODE := 3;
PRO_CONSTRAINT_VIOLATION               : constant CONDITION_CODE := 4;
PRO_MISSING_PARAMETER                  : constant CONDITION_CODE := 5;
PRO_AMBIGUOUS_PORT_SELECTION           : constant CONDITION_CODE := 6;
```

The condition codes for TIME_EXPIRED: 40

```
TEX_DELAYED_BEYOND_TIME_LIMIT          : constant CONDITION_CODE := 1;
```

The condition codes for QUEUE_FULL_ERROR: 41

```
QFU_QUEUE_FULL                         : constant CONDITION_CODE := 1;
```

The condition codes for QUEUE_EMPTY_ERROR: 42

```
QEM_QUEUE_EMPTY                        : constant CONDITION_CODE := 1;
```

The condition codes for TAPE_STATUS_ERROR: 43

```
TAP_NOT_MOUNTED                        : constant CONDITION_CODE := 1;
TAP_HOT_LOADED                         : constant CONDITION_CODE := 2;
```

The condition codes for BUTTON_NAME_ERROR: 44

```
BUT_NONEXISTENT_BUTTON                 : constant CONDITION_CODE := 1;
```

The condition codes for CAIS_TERMINAL_IO.FUNCTION_KEY_STATUS_ERROR: 45

```
FKY_NONEXISTENT_FUNCTION_KEY           : constant CONDITION_CODE := 1;
```

The condition codes for INPUT_EVENT_QUEUE_EMPTY: 46

```
IQE_QUEUE_EMPTY                        : constant CONDITION_CODE := 1;
```

The condition codes for LOCATOR_STATUS_ERROR: 47

```
LCS_LOCATOR_NOT_ENABLED                : constant CONDITION_CODE := 1;
LCS_CURSOR_NOT_ON_ANY_SCREEN           : constant CONDITION_CODE := 2;
LCS_CURSOR_NOT_ON_SCREEN               : constant CONDITION_CODE := 3;
```


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

The condition codes for SCREEN_NOT_ON_DISPLAY: 48

```
SCR_NOT_PASTED                    : constant CONDITION_CODE : = 1;
```

The condition codes for SCREEN_POSITION_ERROR: 49

```
SCP_OFF_SCREEN                    : constant CONDITION_CODE : = 1;
```

The condition codes for VIRTUAL_DISPLAY_POSITION_ERROR: 50

```
VDP_OFF_SCREEN                    : constant CONDITION_CODE := 1;
```

The condition codes for UID_ERROR: 51

```
UID_NOT_UNIQUE                    : constant CONDITION_CODE := 1;
```

The condition codes for ITEM_KIND_ERROR: 52

```
IKE-WRONG_ITEM_KIND               : constant CONDITION_CODE := 1;
IKE_NO_ENUMERATION_VALUE          : constant CONDITION_CODE := 2;
IKE_WRONG_ENUMERATION_POSITION    : constant CONDITION_CODE := 3;
```

The condition codes for LIST_KIND_ERROR: 53

```
LKE_EMPTY_LIST                    : constant CONDITION_CODE := 1;
LKE_NOT_NAMED                     : constant CONDITION_CODE := 2;
LKE_KIND_MISMATCH                 : constant CONDITION_CODE := 3;
LKE_CANNOT_BE_NAMED               : constant CONDITION_CODE := 4;
LKE_MUST_BE_NAMED                 : constant CONDITION_CODE := 5;
```

The condition codes for LIST_POSITION_ERROR: 54

```
LPE_POSITION_EXCEEDS_LENGTH       : constant CONDITION_CODE := 1;
LPE_START_BEYOND_END              : constant CONDITION_CODE := 2;
LPE_CURRENT_IS_OUTERMOST          : constant CONDITION_CODE := 3;
```

The condition codes for NAMED_LIST_ERROR: 55

```
NLE_DUPLICATE_ITEM_NAME           : constant CONDITION_CODE := 1;
NLE_NOT_NAMED_LIST                : constant CONDITION_CODE := 2;
NLE_NAME_CONFLICT                 : constant CONDITION_CODE := 3;
HLE_NAME_ALREADY_USED             : constant CONDITION_CODE := 4;
```

The condition codes for SEARCH_ERROR: 56



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
SEA_NO_SUCH_NAMED_ITEM          : constant CONDITION_CODE := 1;
SEA_NO_SUCH_VALUED_ITEM         : constant CONDITION_CODE := 2;
```

The condition codes for CAIS_LIST_MANAGEMENT.SYNTAX_ERROR: 57

```
LSY_ILLEGAL_LIST_TEXT           : constant CONDITION_CODE := 1;
LSY_ILLEGAL_IDENTIFIER_TEXT     : constant CONDITION_CODE := 2;
```

The condition codes for TOKEN_ ERROR: 58

```
TOK_UNDEFINED_TOKEN             : constant CONDITION_CODE := 1;
```

The condition codes for TIME_ERROR: 59

```
TIM_NOT_A_TIME                  : constant CONDITION_CODE := 1;
TIM_INVALID_YEAR                : constant CONDITION_CODE := 2;
TIM_INVALID_DURATION            : constant CONDITION_CODE := 3;
```

The condition codes for CAPACITY_ERROR: 60

```
CAP_PRAGMATIC_VIOLATION         : constant CONDITION_CODE := 1;
CAP_MAX_FILE_SIZE_EXCEEDED      : constant CONDITION_CODE := 2;
```

The condition codes for RESOURCE_ERROR: 61

```
RES_RESOURCE_NOT_REASSIGNABLE   : constant CONDITION_CODE := 1;
```

The condition codes for FORM_STATUS_ERROR: 62

```
FRM_NONEXISTENT_FORM            : constant CONDITION_CODE := 1;
FRM_FORM_EXISTS                 : constant CONDITION_CODE := 2;
```

The condition codes for CMS_FORMINAL_IO.FUNCTION_KEY_STATUS_ERROR: 63

```
FFK_NONEXISTENT_FUNCTION_KEY    : constant CONDITION_CODE := 1;
```

The condition codes for TERMINAL_POSITION_ERROR: 64

```
FTM_OFF_FORM                    : constant CONDITION_CODE := 1;
```


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

# 13.2 Package CAIS_STATUS_MANAGEMENT

This package defines the interfaces for the creation, inspection, and manipulation of status indicators. The exceptions raised by the subprograms in this package are defined in the packages CAIS_DEFINITIONS and CAIS_PRAGMATICS. 1

## 13.2.1 Creating a status indicator

```
procedure CREATE_INDICATOR
    (INDICATOR : in out STATUS_INDICATOR;
     STATUS    : in     STATUS_INDICATOR := NO_INDICATION);
```

Purpose:

This procedure establishes the status indicator INDICATOR, so that it can acquire status information. The newly established status indicator's exception indication is set to the value NO_ERROR_DETECTED and its status information is set to the value of the constant CAIS_LIST_MANAGEMENT.EMPTY_LIST. 1

Parameters:

**INDICATOR** is a status indicator to be established. 2

**STATUS** is an established status indicator. 3

Exceptions:

**STATUS_ERROR** is raised if INDICATOR is already established prior to the call on CREATE_INDICATOR [4] or if STATUS is not an established status indicator [3]. 4

## 13.2.2 Setting a status indicator

```
procedure SET_INDICATOR
    (INDICATOR             : in out STATUS_INDICATOR;
     EXCEPTION_INDICATION  : in     EXCEPTION_INDICATION_KIND;
     STATUS_INFORMATION    : in     LIST_TYPE           := EMPTY_LIST;
     MESSAGE               : in     MESSAGE_TEXT        := "";
     STATUS                : in     STATUS_INDICATOR    := NO_INDICATION);
```

Purpose:

This procedure sets the established status indicator INDICATOR to have the exception indication value specified in EXCEPTION_INDICATION. It sets the status information list for INDICATOR to value specified in STATUS_INFORMATION. This is the status information list returned when the function STATUS_INFORMATION is called. Finally, it associates the error message string specified in MESSAGE with INDICATOR. This is the message returned when the function MESSAGE is called. 1



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Parameters:

**INDICATOR** is unestablished status indicator to beset. 2

**EXCEPTION_INDICATION** is the enumeration value specifying the exception indication to be associated with INDICATOR. 3

**STATUS_INFORMATION** is the status information list to be associated with INDICATION. 4

**MESSAGE** is is a string containing the error message to be associated with INDICATOR. 5

**STATUS** is an established status indicator for the status of the operation performed when this - interface is called. 6

Exceptions:

**STATUS_ERROR** is raised if INDICATOR or STATUS is not an established status indicator [3]. 7

## 13.2.3 Copying a status indicator

```
procedure COPY_INDICATOR
    (FROM_INDICATOR : in     STATUS_INDICATOR;
     TO_INDICATOR   : in out STATUS_INDICATOR;
     STATUS         : in     STATUS_INDICATOR := NO_INDICATION);
```

Purpose:

This procedure returns in the parameter TO_INDICATOR a copy of the established status indicator FROM_INDICATOR. As a result, the current status information, the exception indication and the ability to acquire status information are identical for the two status indicators. Subsequent modifications of either status indicator do not affect the other. 1

Parameters:

**FROM_INDICATOR** is the established status indicator to be copied. 2

**TO_INDICATOR** is the status indicator returned as a copy of FROM-INDICATOR. 3

**STATUS** is an established status indicator. 4

Exceptions:

**STATUS_ERROR** is raised if FROM_INDICATOR or STATUS is not an established status indicator [3]. 5

## 13.2.4 Deleting a status indicator

```
procedure DELETE_INDICATOR
    (INDICATOR : in out STATUS_INDICATOR;
     STATUS    : in     STATUS_INDICATOR := NO_INDICATION);
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Purpose:

This procedure deletes the status indicator INDICATOR. When a status indicator is deleted, it ceases to be established. The value of the parameter INDICATOR after the call is as if it were never established by a call to CREATE_INDICATOR. Deleting a status indicator which is not established has no effect. 1

Parameters:

**INDICATOR** is a status indicator.

**STATUS** is an established status indicator.

Exceptions:

**STATUS_ERROR** is raised if STATUS is not an established status indicator [3].

## 13.2.5 Obtaining a status indicator's exception indication

```
function EXCEPTION_INDICATION
    (INDICATOR : in STATUS_INDICATOR ;
     STATUS    : in STATUS_INDICATOR := NO_INDICATION)
   return EXCEPTION_INDICATION_KIND;
```

Purpose:

This function returns the current exception indication of the status indicator INDICATOR. 1

Parameters:

**INDICATOR** is an established status indicator.

**STATUS** is an established status indicator.

Exceptions:

**STATUS_ERROR** is raised if INDICATOR or STATUS is not an established status indicator [3]. 4

## 13.2.6 Obtaining status information from a status indicator

```
function STATUS_INFORMATION
    (INDICATOR : in STATUS_INDICATOR;
     STATUS    : in STATUS_INDICATOR := NO_INDICATION)
   return LIST_TYPE;
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Purpose:

This function returns the current status information of the status indicator INDICATOR. 1

Parameters:

**INDICATOR** is an established status indicator. 2

**STATUS** is an established status indicator. 3

Exceptions:

**STATUS_EREOR** is if INDICATOR or STATUS is not an established status indicator [3]. 4

## 13.2.7 Obtaining a descriptive message from a status indicator

```
function MESSAGE
   (INDICATOR : in     STATUS_INDICATOR;
    STATUS    : in     STATUS_INDICATOR := NO_INDICATIOI)
   return MESSAGE_TEXT;
```

Purpose:

This function returns an implementation-defined natural language message describing the exceptional condition (if any) indicated by the CONDITION and implementation-defined SUBCONDITION components of the status information of the status indicator INDICATOR. 1

Parameters:

**INDICATOR** is an established status indicator containing the status information specifying the exceptional condition to be described by the message returned. 2

**STATUS** is an established status indicator. 3

Exceptions:

**STATUS_ERROR** is raised if INDICATOR or STATUS is not an established status indicator [3]. 4

## 13.2.8 Determining if a status indicator is established

```
function  IS-ESTABLISHED
   (INDICATOR : in     STATUS_INDICATOR;
    STATUS    : in     STATUS_INDICATOR  := NO_INDICATION)
   return BOOLEAN;
```



Source: https://assist.dla.mil -- Downloaded: 202610-26T07:54Z
Check the source to verify that this is the current version before use.

Purpose:

This function returns TRUE if the status indicator INDICATOR is established; otherwise it returns FALSE. 1

Parameters:

**INDICATOR** is a status indicator. 2

**STATUS** is an established status indicator. 3

Exceptions:

**STATUS_ERROR** is raised if STATUS is not an established status indicator [3]. 4

Notes:

This interface returns TRUE when the value NO_INDICATION is passed for the formal parameter INDICATOR. 5



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

## 13.3 Exception Conditions

This section describes the predefine exceptional conditions that cause each CAIS exception to be raised. The condition which causes an exception to be raised is identified by the item named CONDITION in the status information list. The integer values returned in CONDITION are condition codes. All predefine condition codes are declared as positive integer constants in package CAIS_STATUS_DEFINITIONS. 1

For a given exception, a distinct positive integer value is assigned to each condition. A CAIS implementation may define additional conditions and subconditions for an exception as long as the condition code values are negative (integer values less than 0). 2

In the CAIS interface descriptions, the “Exceptions:” paragraphs describe the specific conditions which cause an exception to be raised. Each condition described is followed by the associated condition code value enclosed in square brackets, ‘[’ and ‘]’. That value is returned in the status information item CONDITION when the condition occurs. The condition which causes an exception to be raised determines which additional qualifiers are returned in the status information list. 3

The following subsections describe the conditions for each exception. The description of each condition includes the condition code value, the corresponding constant name, a list of condition qualifiers (which are returned in the status information list when the condition occurs) and a description of the information provided by each qualifier. A qualifier preceded by ‘1‘ is an alternative for the preceding qualifier. A qualifier in square brackets, ‘[’ and ‘]’, is included in the status information list only if the additional information it provides is required to describe the” condition. 4

### 13.3.1 Conditions for CAIS_DEFINITIONS exceptions

#### 13.3.1.1 ACCESS_VIOLATION_DETECTED conditions

When the exception indication of an established status indicator is ACCESS_VIOLATION_DETECTED, one of the following condition codes is returned as the value of the CONDITION status information item. 1

**[1] ACC_INSUFFICIENT_RIGHTS** is returned if the discretionary access control rights of the current process are insufficient to obtain access to a node with either explicit or implied intents. 2

Additional qualifiers:

**NODE** => unique_identifier
**INTENTS** => intents-list

**NODE** identifies the node to which access was denied. 3

**INTENTS** indicates the intents with which the access to the node was attempted. 4

**[2] ACC_ACCESS_RESTRICTION** is returned if the current process, (subject) is not allowed to establish or alter an ACCESS relationship to the given GROUP node according to implementation-defined criteria. 5



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Additional qualifiers:

**NODE** => unique_identifier
**CROUP** => CROUP_unique_identifier

**NODE** identifies the node from which the ACCESS relationship was to emanate.

**GROUP** identifies the GROUP node that was the intended target of the ACCESS relationship.

**[3] ACC_DENY_RESTRICTION** is returned if the current process (subject) is not allowed to establish or alter a DENY relationship to a given GROUP node according to implementation-defined criteria.

Additional qualifiers:

**NODE** => Unique_identifier
**GROUP** => GROW_unique_identifier

**NODE** identifies the node from which the DENY relationship was to emanate.

**GROUP** identifies the GROUP node that was the intended target of the DENY relationship.

**[4] ACC_INSUFFICIENT_RIGHTS_PREDICATE** is returned if the discretionary access control rights of the current process are insufficient to obtain access to a node with implied intents for the purpose of evaluating an iteration predicate.

Additional qualifiers:

**NODE** => unique_identifier
**INTENTS** => intents_list

**NODE** identifies the node to which access was denied.

**INTENTS** indicates the intents with which the access to the node was attempted.

**[6] ACC_INSUFFICIENT_DEFAULT_RIGHTS** is returned if the discretionary access control rights of a process would be insufficient to obtain implicit access to a node as required to complete creation of that process.

Additional qualifiers:

**NODE** => Unique_identifier
**INTENTS** => intents-list

**NODE** identifies the node to which access was denied.

**INTENTS** indicates the intents with which the access to the node was attempted.

## 13.3.1.2 AMBIGUOUS_VIEW_ERROR-DETECTED conditions

When the exception indication of an established status indicator is AMBIGUOUS_VIEW_ERROR_-DETECTED, one of the following condition codes is returned as the value of the CONDITION status information item.

**[1] AMB_NAME_TRANSLATION** is returned if a name is ambiguous for translation in the composite view of the current process.



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Additional qualifiers:

```
DEFINITION          => EKD_identifier
| DEFINITION        => RD_identifier
DEFINITION_TYPE  => definition_type
NAME                => identifier
NAME_TYPE           => name_type
VIEW                =>  VIEW_unique_identifier
```

**DEFINITION** identifies the NKD or RD which is the context in which the translation was attempted. 3

**DEFINITION_TYPE** indicates the type of the definition. 4

**NAME** is the name that could not retranslated. 5

**NAME_TYPE** indicates the type of the name given in NAME. 6

**VIEW** identifies the VIEW node representing the view in which the ambiguity was detected. 7

**[2] AMB_NAME_FORMATION** is returned if formation of a name for a property is not possible due to ambiguity in an adopted view. 8

Additional qualifiers:

```
DEFINITION          =>  NKD_unique_identifier
| DEFINITION        =>  RD_unique_identifier
DEFINITION_TYPE  => definition_type
PROPERTY            =>  RD_unique_identifier
| PROPERTY         =>  AD_unique_identifier
PROPERTY_TYPE     =>  property_type
AMBIGUOUS_NAMES  => ( identifier , identifier )
NAME_TYPE           => name_type
VIEW                =>  VIEW_unique_identifier
```

**DEFINITION** identifies the NKD or RD which is the context in which the name formation was attempted. 9

**DEFINITION_TYPE** indicates the type of the definition. 10

**PROPERTY** identifies an RD or an AD. The RD or AD identified by this qualifier is the deifinition node for the relation or attribute property for which a name could not be formed. 11

**PROPERTY_TYPE** indicates the type of the property. 12

**AMBIGUOUS_NAMES** indicates the pair of names for the relation or attribute property which constituted the ambiguity. 13

**NAME_TYPE** indicates the type of the names given in AMBIGUOUS_NAMES. 14

**VIEW** identifies the VIEW node representing the view in which the ambiguity was detected. 15

**[3] AMB_NAME_CREATION** is returned if an attempt is made to define a second name for a relation or attribute property in the context of a single interpretation node. 16



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Additional qualifiers:

```
INTERPRETATION          => NKI_unique_identifier
| INTERPRETATION        => RI_unique_identifier
INTERPRETATION_TYPE     => interpretation_type
PROPERTY                => RD_uique_identifier
| PROPERTY              => AD_unique_identifier
PROPERTY_TYPE           => property_type
AMBIGUOUS_NAMES         => ( identifier , identifier )
SAME_TYPE               => name_type
VIEW                    => VIEW_unique_identifier
```

**INTERPRETATION** identifies the INTERPRETATION node.

**INTERPRETATION_TYPE** indicates the type of the INTERPRETATION node.

**PROPERTY** identifies an RD or an AD. The RD or AD identified by this qualifier is the definition node for the relation or attribute property which was being named.

**PROPERTY_TYPE** indicates the type of the property.

**AMBIGUOUS_NAMES** indicates the pair of names for the relation or attribute property which constituted the ambiguity.

**NAME_TYPE** indicates the type of the names given in AMBIGUOUS_NAMES.

**VIEW** identifies the VIEW node representing the view in which the ambiguity was detected.

**[4] AMB_KEY_VALUE_ASSOCIATION** is returned if a relationship key list consists of a single relationship key designator and association of that key value with a particular key is not possible because more than one key of the relation is visible.

```
\Additional qualifiers:

RELATION          => RD_unique_identifier
AMBIGUOUS_NAMES => ( identifier, identifier )
VIEWS             => ( VIEW_unique_identifier, VIEW_unique_identifier )
RELATIONSHIP_KEY => string
```

**RELATION** identifies the RD which is the context in which the key value association was attempted.

**AMBIGUOUS_NAMES** indicates the key names defined which can be used to identify the different keys constituting the ambiguity.

**VIEWS** identifies the VIEW nodes representing the views which define the key names.

**RELATIONSHIP_KEY** is the specified relationship key list.

## 13.3.1.3 ATTRIBUTE_ERROR_DETECTED conditions

When the exception indication of an established status indicator is ATTRIBUTE_ERROR_DETECTED, one of the following condition codes is returned as the value of the CONDITION status information item.



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

**[1] ATT_NODE_ATTRIBUTE_EXISTS** is returned if an attempt is made to create a node attribute that already exists on the node. 2

Additional qualifiers:

**NODE** => unique_identifier
**ATTRIBUTE** => AD_unique_identifier

**NODE** identifies the specified node. 3

**ATTRIBUTE** identifies the AD for the attribute. 4

**[2] ATT_RELATIONSHIP_ATTRIBUTE_EXISTS** is returned if an attempt is made to create , an attribute on a relationship and the specified attribute already exists on that relationship.

Additional qualifiers:

**RELATIONSHIP** => relationship_specification
**ATTRIBUTE** => AD_unique_identifier

**RELATIONSHIP** identifies the relationship. 6

**ATTRIBUTE** identifies the AD for the attribute. 7

**[3] ATT_NONEXISTENT_NODE_ATTRIBUTE** is returned if a reference is made to a non-existent node attribute. 8

Additional qualifiers:

**NODE** => unique_identifier
**ATTRIBUTE** => AD_unique_identifier

**NODE** identifies the specified node. 9

**ATTRIBUTE** identifies the AD for the attribute. 10

**[4] ATT_NONEXISTENT_RELATIONSHIP_ATTRIBUTE** is returned if a reference is made to a non-existent relationship attribute. 11

Additional qualifiers:

**RELATIONSHIP** => relationship_specification
**ATTRIBUTE** => AD_unique_identifier

**RELATIONSHIP** identifies the relationship. 12

**ATTRIBUTE** identifies the AD for the attribute. 13

**[5] ATT_INVALID_ATTRIBUTE_REFERENCE** is returned if a reference is made to a predefined, mandatory access control-related attribute that is illegal according to implementation-defined criteria. 14



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Additional qualifiers:

```
NODE      => unique_identifier
ATTRIBUTE => AD_unique_identifier
```

**NODE** identifies the specified node.

**ATTRIBUTE** identifies the AD for the attribute.

**[6] ATT_ILLEGAL_ATTRIBUTES_LIST** is returned if a value is specified for the same attribute more than once in a parameter of the type ATTRIBUTE_LIST.

Additional qualifiers:

```
ATTRIBUTES_LIST => list
ERROR_SUBLIST   => current  -linear-list-specification
ERROR_ITEM      => integer
DEFINITION      => NKD_unique_identifier
| DEFINITION    => RD_unique_identifier
ATTRIBUTE       => AD_unique_identifier
```

**ATTRIBUTES_LIST** the parameter of type ATTRIBUTE_LIST.

**ERROR_SUBLIST** identifies the linear list containing the syntactic error.

**ERROR_ITEM** identifies the location of the error within the linear list identified by ERROR_SUBLIST.

**DEFINITION** identifies the context (NKD or RD) in which the list names the same attribute more than once.

**ATTRIBUTE** identifies the AD for the attribute.

## 13.3.1.4 CONFORMANCE_ERROR_DETECTED conditions

When the exception indication of an established status indicator is CONFORMANCE_ERROR_DETECTED, one of the following condition codes is returned as the value of the CONDITION status information item.

**[1] CON_RELATIONSHIP_CANNOT_EMANATE** is returned if a relationship to be created cannot emanate from the base node.

Additional qualifiers:

```
NODE     => unique_identifier
RELATION => RD_unique_identifier
```

**NODE** identifies the base node for the relationship to be created.

**RELATION** identifies the relation for the relationship to be created.

**[2] CON_RELATIONSHIP_CANNOT_TERMINATE** is returned if a relationship to be created cannot terminate at the target node.



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Additional qualifiers:

```
NODE          => unique_identifier
RELATION      => RD_uique_identifier
TARGET_NODE => unique_identifier
```

**NODE** identifies the base node for the relationship to be created. 6

**RELATION** identifies the relation for the relationship to be created. 7

**TARGET_NODE** identifies the target node for the relationship to be created. 8

**[3] CON_EMANATES_BOUND_VIOLATION** is returned if creating or deleting a relationship would cause too many or too few relationships of a relation to emanate from a node. 9

Additional qualifiers:

```
NODE          => unique_identifier
RELATION      => RD_unique_identifier
```

**NODE** identifies the base node for the relationship to be created or deleted. 10

**RELATION** identifies the relation for the which the emanates bound would be violated. 11

**[4] CON_UNDEFINED_ATTRIBUTE** is returned if an attribute to be created is not an attribute defined for the node or relationship. 12

Additional qualifiers:

```
NODE              => unique_identifier
| RELATIONSHIP => relationship_specification
ATTRIBUTE         => AD_unique_identifier
VALUE             => list
```

**NODE** identifies the node to receive the attribute. The qualifier will consist of the empty string (“”) if the node was to be created by the operation. 13

**RELATIONSHIP** identifies the relationship to receive the attribute. 14

**ATTRIBUTE** identifies the AD for the attribute. 15

**VALUE** is the value to be assigned to the attribute in list form. 16

**[5] CON_ATTRIBUTE_NOT_MODIFIABLE** is returned if the MODIFIABLE attribute which applies to an attribute to be written is FALSE. 17

Additional qualifiers:

```
NODE              => unique_identifier
| RELATIONSHIP => relationship_specification
ATTRIBUTE         => AD_unique_identifier
VALUE             => list
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

**NODE** identifies the node to receive the attribute value. 18

**RELATIONSHIP** identifies the relationship to receive the attribute value. 19

**ATTRIBUTE** identifies the AD for the attribute. 20

**VALUE** is the value to be assigned to the attribute in list form. 21

**[6] CON_ATTRIBUTE_NOT_DELETABLE** is returned if the DELETABLE attribute which applies to an attribute to be deleted is FALSE. 22

Additional qualifiers:

```
NODE           => unique_identifier
| RELATIONSHIP => relationship_specification
ATTRIBUTE      => AD_unique_identifier
```

**NODE** identifies the node with the attribute to be deleted. 23

**RELATIONSHIP** identifies the relationship with the attribute to be deleted. 24

**ATTRIBUTE** identifies the AD for the attribute to be deleted. 25

**[7] CON_UNDEFINED_COMPONENT** is returned if an attribute value to be written contains a composite list and a component in that list does not correspond to any component defined by the corresponding composite list AKD. 26

Additional qualifiers:

```
NODE           => unique_identifier
| RELATIONSHIP => relationship_specification
ATTRIBUTE      => AD_unique_identifier
COMPONENT      => componont_specification
VALUE          => list
```

**NODE** identfies the node to receive the attribute value. The qualifier will consist of the empty string (“”) if the node was to be created by the operation. 27

**RELATIONSHIP** identifies the relationship to receive the attribute value. 28

**ATTRIBUTE** identifies the attribute to be written. 29

**COMPONENT** identifies the component of the composite list which is undefined (no NAME or **POSITION** item is included for the undefined component). 30

**VALUE** is the value to be assigned to the attribute in list form. 31

**[8] CON_MISSING_COMPONENT** is returned if a value to be assigned to an attribute contains a composite list and there is no component in that composite list corresponding to a required component defined by the corresponding composite list AKD. 32

Additional qualifiers:

```
NODE           => unique_identifier
| RELATIONSHIP => relationship_specification
ATTRIBUTE      => AD_unique_identifier
COMPONENT      => component_specification
VALUE          => list
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

**NODE** identifies the node to receive the attribute value. The qualifier will consist of the empty string (“”) if the node was to be created by the operation. 33

**RELATIONSHIP** identifies the relationship to receive the attribute value. 34

**ATTRIBUTE** identifies the attribute to be written. 35

**COMPONENT** identifies the the missing component (no ITEM_POSITION item is provided for the missing component). 36

**VALUE** is the value to be assigned to the attribute in list form. 37

**[9] CON_NO_ALTERNATIVE_MATCHED** is returned if a value to be assigned to an attribute contains a variant item which does not conform to any of the alternatives defined by the corresponding variant AKD. 38

Additional qualifiers:

```
NODE             => unique_identifier
| RELATIONSHIP   => relationship_specification
ATTRIBUTE        => AD_unique_identifier
[COMPONENT       => componont_specification]
VALUE            => list
```

**NODE** identifies the node to receive the attribute value. The qualifier will consist of the empty string (“”) if the node was to be created by the operation. 39

**RELATIONSHIP** identifies the relationship to receive the attribute value. 40

**ATTRIBUTE** identifies the attribute to be written. 41

**COMPONENT** identifies the component containing the variant, if it is a component of the attribute. 42

**VALUE** is the value to be assigned to the attribute in list form. 43

**[10] CON_CONSTRAINT_VIOLATION** is returned if a value to be assigned to an attribute does not conform (or contains a component that does not conform) to a constraint defined by an attribute of the corresponding AKD or by a CONSTRAINTS attribute that applies to the AKD. 44

Additional qualifiers:

```
NODE                => unique_identifier
| RELATIONSHIP      => relationship_specification
ATTRIBUTE           => AD_unique_identifier
[COMPONENT          => component_specification]
VALUE               => list
CONSTRAINT_VIOLATED => constraint_violated
```

**NODE** identifies the node to receive the attribute value. The qualifier will consist of the empty string (“” ) if the node was to be created by the operation. 45

**RELATIONSHIP** identifies the relationship to receive the attribute value. 46

**ATTRIBUTE** identifies the attribute to be written. 47

**COMPONENT** identifies the non-conforming component, if the non-conforming value is a corn- 48



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

ponent of the attribute.

**VALUE** is the value to be assigned to the attribute in list form.

**CONSTRAINT_VIOLATED** is the type of constraint violated.

**[11] CON_RELATION_USAGE_VIOLATION** is returned if an attempt is made to create an instance of a relation definition which would violate the ALLOWED_USAGE constraint on that definition.

Additional qualifiers:

**RELATIONSHIP** => relationship_specification
**IMPLIED_USAGE** => relationship_kind
**ALLOWED_USAGE** => relationship_kind

**RELATIONSHIP** is the given relationship identification.

**IMPLIED_USAGE** indicates the kind of relationship which was implied by the attempted operation (PRIMARY or SECONDARY).

**ALLOWED_USAGE** indicates the allowable usage of the identified relation (PRIMARY or SECONDARY).

### 13.3.1.6 DEFINITION_ERROR_DETECTED conditions

When the exception indication of an established status indicator is DEFINITION_ERROR_DETECTED, one of the following condition codes is returned as the value of the CONDITION status information item.

**[1] DEF_DEFINITION_NOT_DELETABLE** is returned if a restriction on the deletion of an NKD, RD, AD or AKD would be violated.

Additional qualifiers:

**DEFINITION** => definition_unique_identifier
**DEFINITION_TYPE** => definition_type
**RESTRICTION** => node_deletion_restriction

**DEFINITION** identifies the definition node whose deletion would violate a restriction.

**DEFINITION_TYPE** specifies whether the definition node is an NKD, RD, AD or AKD.

**RESTRICTION** specifies the restriction which would be violated if the definition node were deleted.

**[2] DEF_PROPERTY_NOT_DELETABLE** is returned if a restriction on the deletion of a property definition relationship would be violated.

Additional qualifiers:

DEFINITION => definition_unique_identifier
DEFINITION_TYPE => definition_type
PROPERTY => property_unique_identifier
PROPERTY_TYPE => property_type
RESTRICTION => property_deletion_restriction



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

**DEFINITION** identifies the definition node for which deletion of a property would violate a restriction. 7

**DEFINITION_TYPE** specifies whether the definition node is an NKD, RD, AD or AKD. 8

**PROPERTY** identifies the definition node constituting the property whose deletion would violate a restriction. 9

**PROPERTY_TYPE** specifies the type of the property to be deleted. 10

**RESTRICTION** specifies the restriction which would be violated if the property definition relationship were deleted. 11

**[3] DEF_CONFLICTING_CONSTRAINTS** is returned if an operation would result in a definition with a constraint or property which conflicts with another constraint or property or if the operation would result in the application of constraints which are prohibited when instances exist. 12

Additional qualifiers:

```
DEFINITION          => definition_unique_identifier
DEFINITION_TYPE     => definition_type
PROPERTY            => property_unique_identifier
PROPERTY_TYPE       => property_type
CONFLICT            => constraints_conflict
[FACTOR             => constraints_conflict_factor]
```

**DEFINITION** identifies the definition node for which a conflicting constraint would be in force. 13

**DEFINITION_TYPE** specifies whether the definition node is an NKD, RD, AD or AKD. 14

**PROPERTY** identifies the definition node constituting the property on which conflicting constraints would be in force. 15

**PROPERTY_TYPE** specifies the type of property constrained. 16

**CONFLICT** specifies the constraints conflict which would result from the operation. 17

**FACTOR** is an optional item which specifies the contributing factor (if any) without which the particular constraints conflict would not have occurred. 18

**[4] DEF_ILLEGAL_BASE_TYPE** is returned if a new attribute kind cannot be assigned to an attribute definition. 19

Additional qualifiers:

```
ATTRIBUTE          => AD-unique-identifier
ATTRIBUTE_KIND     => AKD_unique_identifier
RESTRICTION        => base_type_restriction
```

**ATTRIBUTE** identifies the AD whose AKD cannot be changed. 20

**ATTRIBUTE_KIND** identifies the AKD that is invalid for the AD. 21

**RESTRICTION** identifies the restriction that would be violated by changing the AD's AKD. 22

**[5] DEF_UNNAMED_UNORDERED_AKD** is returned if a composite list attribute kind would not be constrained to be named or ordered (or both). 23



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Additional qualifiers:

**ATTRIBUTE_KIND** => AKD_unique_identifier

**ATTRIBUTE_KIND** identifies the CLAKD that would allow unnamed, unordered values.

**[6] DEF_LAST_INTERPRETATION** is returned if an operation would result in deletion of the last interpretation node corresponding to a definition node without deletion of the definition node.

Additional qualifiers:

**DEFINITION** => definition_unique_identifier
**DEFINITION_TYPE** => definition_type
**INTERPRETATION** => interpretation_unique_identifier
**INTERPRETATION_TYPE** => interpretation_type

**DEFINITION** identifies the definition node for which there would be no interpretation.

**DEFINITION_TYPE** specifies whether the definition node is an NKD or RD.

**INTERPRETATION** identifies the last interpretation node for the definition.

**INTERPRETATION_TYPE** specifies whether the interpretation node is an NKI or RI.

**[7] DEF_LAST_DEFINITION_NAMING** is returned if an operation would result in deletion of the last secondary ATTRIBUTE_DEFINITION or ATTRIBUTE_KIND relationship from VIEW nodes to an AD or AKD without deletion of the definition node.

Additional qualifiers:

**DEFINITION** => definition_unique_identifier
**DEFINITION_TYPE** => definition_type
**VIEW** => VIEW_unique_identifier

**DEFINITION** identifies the definition node for which there would be no naming.

**DEFINITION_TYPE** specifies whether the definition node is an AD or AKD.

**VIEW** identifies the VIEW node for the last view having a naming for the definition.

**[8] DEF_LAST_PROPERTY_NAMING** is returned if an operation would result in deletion of the last ATTRIBUTE-NAME, KEY_NAME or RELATION_NAME naming relationship corresponding to an ATTRIBUTE, KEY or EMANATES property definition relationship without deletion of the definition relationship.

Additional qualifiers:

INTERPRETATION => interpretation_unique_identifier
INTERPRETATION_TYPE => interpretation_type
NAME => identifier
NAME_TYPE => name_type



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

**INTERPRETATION** identifies the source node of the naming relationship. 35

**INTERPRETATION_TYPE** specifies whether the source node is an NKI or RI. 36

**NAME** is the NAME key value on the naming relationship. 37

**NAME_TYPE** specifies whether the property is an attribute, key or emanates property. 38

**[9] DEF_CAIS_CONTROLLED_CONSTRAINTS** is returned if an operation would result in constraints being modified for a CAIS-controlled property in the context of a predefine definition. 39

Additional qualifiers:

```
DEFINITION      => definition_unique_identifier
DEFINITION_TYPE => definition_type
PROPERTY        => property_unique_identifier
PROPERTY_TYPE   => property_type
```

**DEFINITION** identifies the definition having the property on which constraints would be modified. 40

**DEFINITION_TYPE** specifies whether the definition node is an NKD, RD or AD. 41

**PROPERTY** identifies the definition constituting the property on which constraints would be modified. 42

**PROPERTY_TYPE** specifies the type of property constrained. 43

**[10] DEF_CAIS_CONTROLLED_SUBLATTICE** is returned if an operation would result in one predeifined, CAIS-controlled definition becoming a specialization of another when it is not already. 44

Additional qualifiers:

```
ANCESTOR_DEFINITION => definition-uniquo-idontifior
DEFINITION          => definition_unique_identifier
DEFINITION_TYPE     => definition_type
```

**ANCESTOR_DEFINITION** identifies the CAIS-controlled definition which would become a generalization of another. 45

**DEFINITION** identifies the CAIS-controlled definition which would become a specialization of another. 46

**DEFINITION_TYPE** specifies whether the definition nodes are NKDs or RDs. 47

## 13.3.1.6 DEVICE_ERROR_DETECTED conditions

When the exception indication of an established status indicator is DEVICE_ERROR_DETECTED, the condition codes returned as the value of the CONDITION status information item are implementation-defined. 1



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

## 13.3.1.7 EXISTING_NODE_ERROR_DETECTED conditions

When the exception indication of an established status indicator is EXISTING_NODE_ERROR_DETECTED, one of the following condition codes is returned as the value of the CONDITION status information item. 1

**[1] EXI_NODE_ALREADY_EXISTS** is returned if an attempt is made to specify values for some unique key set of a primary relationship which are the same as the corresponding key values of another relationship belonging to the relation that defines the unique key set. 2

Additional qualifiers:

**NODE** => unique.identifier
**RELATIONSHIP** => relationship-specification

**NODE** identifies the node which already exists with the identification given. 3

**RELATIONSHIP** identifies the existing relationship using the keys in the unique key set and the relation which defines the unique key set. 4

### 13.3.1.8 INTENT_VIOLATION_DETECTED conditions

When the exception indication of an established status indicator is INTENT_VIOLATION_DETECTED, one of the following condition codes is returned as the value of the CONDITION status information item. 1

**[1] INT_INSUFFICIENT_RIGHTS** is returned if a node handle used to access a node was not opened with intents establishing one or more predefine discretionary access rights required by the operation. 2

Additional qualifiers:

**NODE** => unique_identifier
**RIGHTS_NEEDED** => rights_list
**INTENTS** => intents_list

**NODE** identifies the node to be accessed. 3

**RIGHTS_NEEDED** indicates the predefine discretionary access rights required by the operation. 4

**INTENTS** indicates the intents with which the node handle was opened. 5

## 13.3.1.9 ITERATOR_ERROR_DETECTED conditions

When the exception indication of an established status indicator is ITERATOR_ERROR_DETECTED, one of the following condition codes is returned as the value of the CONDITION status information item. 1

**[1] ITE_ITERATOR_NOT_ESTABLISHED** is returned if an attempt is made to use an iterator that has not been created. 2



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

**[2] ITE_ITERATOR_ESTABLISHED** is returned if an attempt is made to create an iterator has been created but not deleted. 3

**[3] ITE_ITERATOR_EXHAUSTED** is returned if an attempt is made to advance an exhausted iterator. 4

## 13.3.1.10 LOCK_ERROR_DETECTED conditions

When the exception indication of an established status indicator is LOCK_ERROR_DETECTED, one of the following condition codes is returned as the value of the CONDITION status information item. 1

**[1] LOE_INTENT_CONFLICT** is returned if access to a node with a given intent could not be obtained because the node was locked against access with that intent. If a time limit applied to obtaining access to the node, either the time limit expired or the implementation determined that a deadlock situation existed. 2

Additional qualifiers:

```
NODE                  => unique_identifier
INTENTS               => intents_list
CONFLICTING_INTENTS   => intents_list
DEADLOCK              => TRUE | FALSE
```

**NODE** identifies the node to be accessed. 3

**INTENTS** indicates the intents toward the node implied by or explicitly specified in the operation attempted. 4

**CONFLICTING_INTENTS** indicates the intents (locks) that were in conflict with the intended access to the node. 5

**DEADLOCK** indicates whether or not the cause of the exceptional condition was the detection of a deadlock situation. If a deadlock situation was detected DEADLOCK has the value TRUE, otherwise DEADLOCK has the value FALSE. 6

**[2] LOE_INTENT_CONFLICT_PREDICATE** is returned if access to a node was required to evaluate an iteration predicate and that node was locked against the required access. 7

Additional qualifiers:

```
NODE                  => unique_identifier
INTENTS               => intents_list
CONFLICTING_INTENTS   => intents_list
```

**NODE** identifies the node to be accessed. 8

**INTENTS** indicates the intents toward the node implied by in the predicate evaluation attempted. 9

**CONFLICTING_INTENTS** indicates the intents (locks) that were in conflict with the intended access to the node. 10

**[3] LOE_TRANSACTION_AFFECTS_DEFINITION** is returned if an operation requires information about a definition (such as its properties, constraints or whether it has instances) and an open transaction not containing the operation contains operations which determine the information. 11



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Additional qualifiers:

DEFINITION => definition_unique_identified
DEFINITION_TYPE => definition_type

**DEFINITION** identifies the definition node which a transaction is affecting by containing operations on it or its instances. 12

**DEFINITION_TYPE** indicates the type of the definition node (AD, AKD, NKD or RD). 13

## l3.3.1.11 NAME_ERROR_DETECTED conditions

When the exception indication of an established status indicator is NAME_ERROR_DETECTED, one of the following condition codes is returned as the value of the CONDITION status information item. 1

**[1] NAM_INVALID_ACCESS** is returned if an attempt is made to access an unobtainable or inaccessible node. 2

Additional qualifiers:

NODE => unique_identifier
INTENTS => intents_list

**NODE** identifies the unobtainable or inaccessible node. 3

**INTENTS** indicates the intents with which the access to the node was attempted. 4

**[2] NAM_INVALID_UID_OPEN** is returned if an attempt is made to open a node handle to a non-existing, inaccessible or unobtainable node and the node to be opened was identified by its unique identifier. 5

Additional qualifiers:

**NODE** => Uni.quo-identifier

**NODE** identifies the non-existing, inaccessible or unobtainable node. 6

**[3] NAM-INVALID-ACCESS-PREDICATE** is returned if an attempt is made to access an unobtainable or inaccessible node in order to evaluate an iteration predicate. 7

Additional qualifiers:

**NODE** => unique_identifier
**INTENTS** => intents-list

**NODE** identifies the unobtainable or inaccessible node. 8

**INTENTS** indicates the intents with which the access to the node was attempted. 9

**[4] NAM_NO_NODE_WITH_UID** is returned if a node or its contents cannot be located in an external node set by its unique identifier. 10


Source: https://assist.dla.mil -- Downloaded: 2026l0-26T07:54Z
Check the source to verify that this is the current version before use.

Additional qualifiers:

**NODE** => unique_identifier

## 13.3.1.12 NODE_KIND_ERROR_DETECTED conditions

When the exception indication of an established status indicator is NODE_KIND_ERROR_DETECTED, one of the following condition codes is returned as the value of the CONDITION status information item.

**[1] NKE_INAPPROPRIATE_INSTANCE** is returned if the node identified in a call to a CAIS interface is not an instance of the required node kind definition or a specialization of that definition.

Additional qualifiers:

```
NODE          => unique_identifier
REQUIRED_NKD  => NKD_unique_identifier
```

**NODE** identifies the specified node.

**REQUIRED_NKD** identifies the NKD representing the node kind required by the interface called.

**[2] NKE_INAPPROPRIATE_NKD** is returned if the NKD identified in a call to a CAIS interface is not the NKD for the required node kind definition or a specialization of that definition.

Additional qualifiers:

```
DEFINITION    => NKD_unique_identifier
REQUIRED_NKD  => NKD_unique_identifier
```

**DEFINITION** identifies the specified NKD.

**REQUIRED_NKD** identifies the NKD representing the node kind required by the interface called.

**[3] NKE_INAPPROPRIATE_RD** is returned if the RD identified in a call to a CAIS interface is not the RD for the required relation definition or a specialization of that definition.

Additional qualifiers:

```
DEFINITION    => RD_unique_identifier
REQUIRED_RD   => RD_unique_identifier
```

**DEFINITION** identifies the specified RD.

**REQUIRED_RD** identifies the RD for the predefined relation required by the interface called.

**[4] NKE_NKD_CAIS_CONTROLLED** is returned if the NKD identified in a call to a general-purpose node creation interface is CAIS-controlled.



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Additional qualifiers:

**DEFINITION** => NKD_unique_identifier

**DEFINITION** identifies the specified NKD. 12

**[3] NKE_NODE_CAIS_CONTROLLED** is returned if an attempt is made to perform an operation on a CAIS-controlled node using an interface which may not be applied to such CAIS-controlled nodes. 13

Additional qualifiers:

**NODE** => unique_identifier

**NODE** identifies the node. 14

**[6] NKE_ILLEGAL_CONVERSION** is returned if an attempt is made to change the NKD of a node so that the set of predefine, CAIS-controlled node kinds to which the node belongs is changed. 15

Additional qualifiers:

**NODE** => uniquo-identifier
**DEFINITION** => NKD_unique_identifier

**NODE** identifies the node to be converted. 16

**DEFINITION** identifies the NKD for the new node kind to which the node is to be converted. 17

## 13.3.1.13 NOT_IN_VIEW_ERROR_DETECTED conditions

When the exception indication of an established status indicator is NOT_IN_VIEW_ERROR_DETECTED, one of the following condition codes is returned as the value of the CONDITION status information item. 1

**[1] NIV_PROPERTY_NAME_TRANSLATION** is returned if a property name cannot be translated using the composite view of the current process. 2

Additional qualifiers:

```
DEFINITION        => NKD_identifier
| DEFINITION      => RD_identifier
DEFINITION_TYPE => definition_type
NAME              => identifier
NAME_TYPE         => name_type
```

**DEFINITION** identifies the NKD or RD which is the context in which the translation was attempted. 3

**DEFINITION_TYPE** indicates the type of the definition. 4

**NAME** is the name that could not be translated. 5

**NAME_TYPE** indicates the type of the name given in NAME. 6

**[2] NIV_DEFINITION_NAME_TRANSLATION** is returned if a definition name cannot be translated using the composite view of the current process. 7



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Additional qualifiers:

```
DEFINITION_NAME => identifier
DEFINITION_TYPE => name_type
```

**DEFINITION_NAME** is the definition name that could not be translated. 8

**DEFINITION_TYPE** indicates the type of the definition name given in NAME. 9

**[3] NIV_NAME_FORMATION** is returned if a name for a relation, a key or an attribute cannot be formed from the composite view of the current process. 10

Additional qualifiers:

```
DEFINITION      => NKD_unique_identifier
| DEFINITION    => RD_unique_identifier
DEFINITION_TYPE => definition_type
PROPERTY        => RD_unique_identifier
|PROPERTY       => AD_unique_identifier
PROPERTY_TYPE   => property_type
NAME_TYPE       => name_type
```

**DEFINITION** identifies the NKD or RD which is the context in which the name formation was attempted. 11

**DEFINITION_TYPE** indicates the type of the definition. 12

**PROPERTY** identifies an RD or an AD. The RD or AD identified by this qualifier is the definition node for the relation or attribute property for which a name could not be formed. 13

**PROPERTY_TYPE** indicates the type of the property. 14

**NAME_TYPE** indicates the type of the name that could not be formed. 15

**[4] NIV_VACUOUS_PATTERN** is returned if a RELATION_NAME_PATTERN does not match any relation name in the composite view of the current process. 16

Additional qualifiers:

```
PATTERS => string
```

**PATTERN** is the relation name pattern. 17

**[5] NIV_KEY_VALUE_ASSOCIATION** is returned if a relationship key list consisting of a single relationship key designator is specified and association of that key value with a key is not possible because no key of the relation is visible. 18

Additional qualifiers:

```
RELATION         => RD_unique_identifier
RELATIONSHIP_KEY => string
```

**RELATION** identifies the RD which is the context in which the key value association was attempted. 19

**RELATIONSHIP_KEY** is the specified relationship key list. 20



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

## 13.3.1.14 PATHNAME_SYNTAX_ERROR_DETECTED conditions

When the exception indication of an established status indicator is PATHNAME_SYNTAX_ERROR_DETECTED, one of the following condition codes is returned as the value of the CONDITION status information item. 1

**[1] PSY_PATHNAME** is returned if a specified pathname is syntactically illegal. 2

Additional qualifiers:

```
PATHNAME        => string
SYNTACTIC_ERROR => syntactic_error_type
```

**PATHNAME** is the specified pathname. 3

**SYNTACTIC_ERROR** indicates the nature of the syntactic error. 4

**[2] PSY_RELATIONSHIP_KEY** is returned if a specified RELATIONSHIP_KEY value is syntactically illegal. 5

Additional qualifiers:

```
RELATIONSHIP_KEY => string
SYNTACTIC_ERROR => syntactic_error_type
```

**RELATIONSHIP_KEY** is the specified RELATIONSHIP_KEY value. 6

**SYNTACTIC_ERROR** indicates the nature of the syntactic error. 7

**[3] PSY_RELATION_NAME** is returned if a specified relation name is syntactically illegal. 8

Additional qualifiers:

```
RELATION_NAME => string
```

RELATION_NAME is the specified RELATION_NAME value. 9

## 13.3.1.16 PREDEFINED_ATTRIBUTE_ERROR_DETECTED conditions

When the exception indication of an established status indicator is PREDEFINED_ATTRIBUTE_ERROR_DETECTED, one of the following condition codes is returned as the value of the CONDITION status information item. 1

**[1] PAE_CAIS_CONTROLLED_NODE_ATTRIBUTE** is returned if an attempt is made to create, set or delete the value of a node attribute which is CAIS-controlled using an interface cannot be applied to that CAIS-controlled attribute. 2

Additional qualifiers:

```
NODE      => unique_identifier
ATTRIBUTE => AD_unique_identifier
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

**NODE** identifies the node having the attribute. The qualifier will consist of the empty string ("") if the node was to be created by the operation.

**ATTRIBUTE** identifies the CAIS-controlled AD representing the attribute.

**[2] PAE_CAIS_CONTROLLED_RELATIONSHIP_ATTRIBUTE** is returned if an attempt is made to create, set or delete the value of a relationship attribute that is CAIS-controlled using an interface which may not be applied to that attribute.

Additional qualifiers:

```
RELATIONSHIP => relationship_specification
ATTRIBUTE    => AD_unique_identifier
```

**RELATIONSHIP** identifies the relationship having the attribute.

**ATTRIBUTE** identifies the CAIS-controlled AD representing the attribute.

**[3] PAE_CAIS_CONTROLLED_AD** is returned if an attempt is made to use a CAIS-controlled AD to define a new attribute of a node kind or relation.

Additional qualifiers:

```
ATTRIBUTE       => AD_unique_identifier
DEFINITION      => definition_unique_identifier
DEFINITION_TYPE => definition_type
```

**ATTRIBUTE** identifies the CAIS-controlled AD.

**DEFINITION** identifies the NKD or RD for which the attribute would be a new property.

**DEFINITION_TYPE** indicates the type of the definition.

## 13.3.1.16 PREDEFINED_RELATION_ERROR_DETECTED conditions

When the exception indication of an established status indicator is PREDEFINED_RELATION_-ERROR_DETECTED, one of the following condition codes is returned as the value of the CONDITION status information item.

**[1] PRE_CAIS_CONTROLLED** is returned if an attempt is made to create a relationship which is CAIS-controlled or to change the source node or target node of a relationship of a relation which is CAIS-controlled, using an interface which may not be applied to that relationships of that relation.

Additional qualifiers:

```
RELATIONSHIP => relationship_specification
```

**RELATIONSHIP** identifies the relationship.

**[2] PRE_INHERITANCE_NOT_CHANGEABLE** is returned if an attempt is made to change the inheritance property of a relationship of a predefine relation and the inheritance property of relationships of that relation may not be altered by the user.



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Additional qualifiers:

RELATIONSHIP => relationship-specification

**RELATIONSHIP** identifies the relationship.

**[3] PRE_ILLEGAL_CONVERSION** is returned if an attempt is made to change the RD of a relationship so that the set of predefine, CAIS-controlled relations to which the relationship belongs is changed. This may occur when a relationship is renamed or converted.

Additional qualifiers:

RELATIONSHIP => relationship_specification
DEFINITION => RD_unique_identifier

**RELATIONSHIP** identifies the relationship being converted.

**DEFINITION** identifies the RD for the new relation to which the relationship is being converted.

**[4] PRE_NOT_JOB_RELATIONSHIP** is returned if the primary relationship to the specified PROCESS node is not a JOB relationship.

Additional qualifiers:

NODE => unique_identifier

**NODE** identifies the PROCESS node.

**[5] PRE_RD_CAIS_CONTROLLED** is returned if an attempt is made to use a CAIS-controlled RD to define a new emanates property for a node kind.

Additional qualifiers:

RELATION => RD_unique_identifier
NODE_KIND => NKD_unique_identifier

**RELATION** identifies the CAIS-controlled RD.

**NODE_KIND** identifies the NKD which does not currently have the emanates property.

### 13.3.1.17 RELATIONSHIP_ERROR_DETECTED conditions

When the exception indication of an established status indicator is RELATIONSHIP_ERROR_DETECTED, one of the following condition codes is returned as the value of the CONDITION status information item.

**[1] RER_IDENTIFICATION_NOT_UNIQUE** is returned if an attempt is made to identify a relationship and the relationship identification identifies more than one relationship.


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Additional qualifiers:

```
RELATIONSHIP => relationship_specification
```

**RELATIONSHIP** indicates the relationship identification that did not identify a single relationship. 3

**[2] RER_NO_SUCH_RELATIONSHIP** is returned if an attempt is made to identify an existing relationship and the relationship identified does not exist. 4

Additional qualifiers:

```
RELATIONSHIP => relationship.specification
```

**RELATIONSHIP** indicates the relationship identification that did not identify an existing relationship. 5

**[3] RER_RELATIONSHIP_ALREADY_EXISTS** is returned if an attempt is made to specify values for some unique key set of a relationship which are the same as the corresponding key values of another relationship belonging to the relation that defines the unique key set. 6

Additional qualifiers:

```
RELATIONSHIP => relationship-specification
```

**RELATIONSHIP** identifies the existing relationship using the keys in the unique key set and the relation which defines the unique key set. 7

**[4] RER_NODES_NOT_RELATED** is returned if an operation requires a relationship of a given relation having the specified source and target nodes and no such relationship exists. 8

Additional qualifiers:

```
SOURCE   => unique-identifier
TARGET   => unique_identifier
RELATION => RD_unique_identifier
```

**SOURCE** identifies the specified source node. 9

**TARGET** identifies the specified target node. 10

**RELATION** identifies the RD failing to have an instance connecting the specified nodes. 11

**[5] RER_ILLEGAL_KEY_LIST** is returned if a relationship key list specifies the value of the same key more than once. 12

Additional qualifiers:

```
RELATION         => RD_unique_identifier
RELATIONSHIP_KEY => string
KEY              => AD_unique_identifier
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

**RELATION** identifies the RD which is the context in which the relationship key list was interpreted. 13

**RELATIONSHIP_KEY** is the specified relationship key list. 14

**KEY** identifies the AD for the key whose value was specified more than once. 15

**[6] RER_NO_UNIQUE_KEY_SET** is returned if the relation of a relationship to be created has no unique key set. 16


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Additional qualifiers:

RELATION => RD_unique_identifier

**RELATION** identifies the RD for the relationship to be created.

## 13.3.1.18 RESOURCE_ALLOCATION_ERROR_DETECTED conditions

When the exception indication of an established status indicator is RESOURCE_ALLOCATION_-ERROR_DETECTED, one of the following condition codes is returned as the value of the CONDITION status information item.

**[1] RAL_RESOURCE_NOT_ALLOCATABLE** is returned if an attempt is made to allocate a resource that is not allocatable.

Additional qualifiers:

RESOURCE => resource_unique_identifier
RESOURCE-TYPE => resource_type

**RESOURCE** identifies the resource node representing the resource that could not be allocated.

**RESOURCE_TYPE** indicates the type of the resource.

**[2] RAL_INVALID_PROCESS_NODE_MEDIUM** is returned if an attempt is made to assign a PROCESS node to a medium resource that is not allocatable by the processor resource designated for the new process.

Additional qualifiers:

MEDIUM => MEDIUM_unique_identifier
PROCESSOR => PROCESSOR_unique_identifier

**MEDIUM** identifies the specified MEDIUM node.

**PROCESSOR** identifies the specified PROCESSOR node.

## 13.3.1.19 RESOURCE_AVAILABILITY_ERROR_DETECTED conditions

When the exception indication of an established status indicator is RESOURCE_AVAILABILITY_-ERROR_DETECTED, one of the following condition codes is returned as the value of the CONDITION status information item.

**[1] RAV_RESOURCE_NOT_AVAILABLE** is returned if an attempt is made to allocate a resource that is not available.

Additional qualifiers:

RESOURCE => resource_unique_identifier
RESOURCE_TYPE => resource_type

RESOURCE identifies the resource node representing the resource that could not be allocated.

RESOURCE_TYPE indicate the type of the resource.


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

### 13.3.1.20 SECURITY_VIOLATION_DETECTED conditions

When the exception indication of an established status indicator is SECURITY_VIOLATION_DETECTED, the condition codes returned as the value of the CONDITION status information item are implementation-defined.

### 13.3.1.21 STATUS_ERROR_DETECTED conditions

When the exception indication of an established status indicator is STATUS_ERROR_DETECTED, one of the following condition codes is returned as the value of the CONDITION status information item.

**[1] STA_ALREADY_OPEN** is returned if a node handle to be opened is already open.

Additional qualifiers:

NODE => unique_identifier

**NODE** identifies the node identified by the specified open node handle.

**[2] STA_NOT_OPEN** is returned if a closed node handle is passed as a parameter to a CAIS interface requiring an open node handle.

**[3] STA_NOT_ESTABLISHED** is returned if the CAIS interface called requires an established status indicator and the status indicator passed is not established. Only interfaces in CAIS_STATUS_MANAGEMENT can return this condition code, since to do so requires a second, established STATUS parameter. However, the condition can occur for most CAIS interfaces, since most interfaces have a status indicator parameter.

**[4] STA_ALREADY_ESTABLISHED** is returned if the status indicator to be established is already established.

**[6] STA_TRANSACTION_ALREADY_OPEN** is returned if a transaction handle to be opened is already open.

**[6] STA_TRANSACTION_NOT_OPEN** is returned if the CAIS interface called requires an open transaction handle and the transaction handle passed is not open.

**[7] STA_FILE_ALREADY_OPEN** is returned if a file handle to be opened is already open.

Additional qualifiers:

CLIENT_NODE => unique_identifier

**CLIENT_NODE** identifies the opposite client node to which the process already has an open file handle.

**[8] STA_FILE_NOT_OPEN** is returned if the CAIS interface called requires an open file handle and the file handle passed is not open.

**[9] STA_UNEXPECTED_CHANNEL_STATE** is returned if the channel is not in the state expected for the operation.


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Additional qualifiers:

```
CHANNEL                  => CHANNEL_unique_identifier
CHANNEL_STATE            => channel_state_type
EXPECTED_CHANNEL_STATE   => channel_state_type
```

**CHANNEL** identifies the CHANNEL node of the channel on which the operation was attempted. 13

**CHANNEL_STATE** gives the state the channel was in at the beginning of the attempted operation. 14

**EXPECTED_CHANNEL_STATE** gives the initial state which is expected for the operation. 15

**[10] STA_SCREEN_ALREADY_OPEN** is returned if a screen handle to be opened is already open. 16

**[11] STA_SCREEN_NOT_OPEN** is returned if the CAIS interface called requires an open screen handle and the screen handle passed is not open. 17

**[12] STA_NODE_SET_ALREADY_OPEN** is returned if a node set handle to be opened is already open. 18

**[13] STA_NODE_SET_NOT_OPEN** is returned if the CAIS interface called requires an open node set and the node set handle supplied is not open. 19

## 13.3.1.22 SYNTAX_ERROR_DETECTED conditions

When the exception indication of an established status indicator is SYNTAX_ERROR_DETECTED, one of the following condition codes is returned as the value of the CONDITION status information item. 1

**[1] SYN_ILLEGAL_UID** is returned if a unique identifier passed to a CAIS interface is syntactically illegal. 2

Additional qualifiers:

```
UID => unique_identifier
```

**UID** is the specified UNIQUE_ IDENTIFIER value. 3

**[2] SYN_ILLEGAL_RELATION_NAME_PATTERN** is returned if a relation name pattern specified is syntactically illegal. 4

Additional qualifiers:

```
RELATION_NAME => string
```

**RELATION_NAME** is the specified RELATION_NAME value. 5

**[3] SYN_ILLEGAL_RELATIONSHIP_KEY_PATTERN** is returned if a relationship key pattern specified is syntactically illegal. 6



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Additional qualifiers:

```
RELATIONSHIP_KEY => string
SYNTACTIC_ERROR => syntactic_error_type
```

**RELATIONSHIP_KEY** is the specified RELATIONSHIP_KEY_PATTERN value. 7

**SYNTACTIC_ERROR** indicates the nature of the syntactic error. 8

**[4] SYN_ILLEGAL_PREDICATE_SYNTAX** is returned if an iteration predicate specified is syntactically illegal. 9

Additional qualifiers:

```
PREDICATE      => list
ERROR_SUBLIST  => current_linear_list_specification
ERROR_ITEM     => integer
```

**PREDICATE** is the specified ITERATION_PREDICATE value. 10

**ERROR_SUBLIST** identifies the linear list containing the syntactic error. 11

**ERROR_ITEM** identifies the location of the error within the linear list identified by ERROR_SUBLIST. 12

**[6] SYN_ILLEGAL_PREDICATE** is returned if the iteration predicate applied to the creation of a node kind instance iterator uses the RELATIONSHIP_ATTRIBUTE function or the IS_PRIMARY function. 13

Additional qualifiers:

```
PREDICATE      => list
ERROR_SUBLIST  => current_linear_list_specification
ERROR_ITEM     => integer
```

**PREDICATE** is the specified ITERATION_PREDICATE value. 14

**ERROR_SUBLIST** identifies the linear list containing the syntactic error. 15

**ERROR_ITEM** identifies the location of the error within the linear list identified by ERROR_SUBLIST. 16

**[6] SYN_ILLEGAL_ATTRIBUTE_NAME_PATTERN** is returned if an attribute name pattern specified is syntactically illegal. 17

Additional qualifiers:

**ATTRIBUTE_NAME** => string

**ATTRIBUTE_NAME** is the ATTRIBUTE_NAME_PATTERN value specified. 18

**[7] SYN_ILLEGAL_ACCESS_RIGHT_IDENTIFIER** is returned if an ACCESS_RIGHTS value specified is not a legal Ada identifier. 19



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Additional qualifiers:

**ACCESS_RIGHT** => string

**ACCESS_RIGHT** is the ACCESS_RIGHTS value specified.

**[8] SYN_ILLEGAL_ATTRIBUTES_LIST** is returned if an ATTRIBUTES_LIST parameter value specified is syntactically illegal.

Additional qualifiers:

ATTRIBUTES_LIST => list

**ATTRIBUTES_LIST** is the ATTRIBUTES_LIST value specified.

**[9] SYN_ILLEGAL_MANDATORY_ACCESS** is returned if a MANDATORY_ACCESS_LIST parameter value specified is syntactically illegal.

Additional qualifiers:

CLASSIFICATION_LABEL => list

**CLASSIFICATION_LABEL** is the MANDATORY_ACCESS_LIST parameter specified.

**[10] SYN_ILLEGAL_DISCRETIONARY_ACCESS** is returned if a DISCRETIONARY_ACCESS_LIST parameter value specified (containing the value for a GRANT attribute on an ACCESS relationship) is syntactically illegal.

Additional qualifiers:

GRANT => list

**GRANT** is the DISCRETIONARY-ACCESS-LIST value specified.

**[11] SYN_ILLEGAL_ATTRIBUTE_NAME** is returned if an ATTRIBUTE_NAME parameter value specified is syntactically illegal.

Additional qualifiers:

ATTRIBUTE_NAME => string

**ATTRIBUTE_NAME** is the ATTRIBUTE_NAME value specified.

**[12] SYN_ILLEGAL_RELATION_NAME** is returned if a RELATION_NAME parameter value specified is syntactically illegal.

Additional qualifiers:

RELATION_NAME => string

**RELATION_NAME** is the RELATION_NAME value specified.

**[13] SYN_ILLEGAL_PARAMETER_LIST** is returned if a PARAMETER_LIST parameter value specified is syntactically illegal.


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Additional qualifiers:

INPUT_PARAMETERS => list

**INPUT_PARAMETERS** is the PARAMETER_LIST value specified.

**[14] SYN_ILLEGAL_DEFINITION_NAME** is returned if a NODE_KIND_NAME, RELATION_DEFINITION_NAME, ATTRIBUTE_NAME or ATTRIBUTE_KIND_NAME type parameter value does not contain a syntactically legal definition name.

Additional qualifiers:

DEFINITION_NAME => string
DEFINITION_TYPE => definition_type

**DEFINITION_NAME** is the syntactically illegal parameter value.

**DEFINITION_TYPE** indicates the type of definition name required.

**[15] SYN_ILLEGAL_EXTERNAL_NODE_SET** is returned by an operation that must read a stream containing a set of nodes in common external form and the contents of the stream do not conform to the syntax of an external node set. This condition can occur if the file handle specified is incorrectly positioned within a syntactically correct CEF stream.

Additional qualifiers:

CLIENT_NODE => unique_identifier
INVALID_ITEM => string

**CLIENT_NODE** identifies the opposite client node corresponding to the file handle specified in the call.

**INVALID_ITEM** is the lexical item which did not conform to the external node set syntax. The **CAIS_TEXT_IO** file handle used to read the stream remains positioned after the last character read when the problem was detected. Thus, the CAIS_TEXT_IO functions COL, LINE, and PAGE can be used to locate the problem in the stream.

## 13.3.1.23 TRANSACTION_ERROR_DETECTED conditions

When the exception indication of an established status indicator is TRANSACTION_ERROR_DETECTED, one of the following condition codes is returned as the value of the CONDITION status information item.

**[1] TRN_AMBIGUOUS_INCLUSION** is returned if the operation would be included in more than one transaction due to the inclusion of two of the nodes operated upon in distinct transactions neither of which is a descendant of the other.

Additional qualifiers:

NODE_1 => unique_identifier
NODE_2 => unique_identifier



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

**NODE_1** identifies a node to be operated upon which is included in one of the transactions.

**NODE_2** identifies a node to be operated upon which is included in the other transaction.

**[2] TRN_NODE_NOT_INCLUDED** is returned if the operation would be included in a transaction and the operation would modify a node which is not included in the same transaction.



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Additional qualifiers:

NODE => unique_identifier

**NODE** identifies a node to be modified which is not in the same transaction.

**[3] TRN_FILE_NOT_INCLUDED** is returned if the operation would be included in a transaction and would access a SECONDARY_STORAGE node or CHANNEL node using a file handle that was not opened inside the transaction.

Additional qualifiers:

NODE => unique_identifier

**NODE** identifies a node to be modified using the file handle.

## 13.3.1.24 USE_ERROR_DETECTED conditions

When the exception indication of an established status indicator is USE_ERROR_DETECTED, one of the following condition codes is returned as the value of the CONDITION status information item.

**[1] USE_VACUOUS_INTENT** is returned if an intent specification is an empty array.

**[2] USE_ILLEGAL_CHANGE_INTENT** is returned if an attempt is made to change the intention regarding the use of a node handle and there are open file handles associated with the node identified by the node handle and the new intent differs from the existing intent regarding the contents of the node identified.

Additional qualifiers:

```
NODE        => unique_identifier
INTENTS     => intents_list
NEW_INTENTS => intents_list
```

**NODE** identifies the node identified by the specified node handle.

**INTENTS** indicates the intents with which the node handle was opened.

**NEW_INTENTS** indicates the new set of intents requested regarding the use of the specified node handle.

**[3] USE_NO_PATH_ASSOCIATED_WITH_HANDLE** is returned if an attempt is made to identify the relationship traversed when a node handle was opened and the specified node handle was opened using the pathname ":" or other means whereby no path is associated with the handle.

Additional qualifiers:

NODE => unique_identifier

**NODE** identifies the node identified by the specified node handle.

**[4] USE_NO_POSSIBLE_BASE_PATH** is returned if an attempt is made to obtain the base path indicated in a pathname and the pathname specified is ":", for which no base path can be returned.



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Additional qualifiers:

```
PATHNAME => string
```

**PATHNAME** is the specified PATHNAME value. 10

**[6] USE_ILLEGAL_PRIMARY** is returned if an attempted modification of a primary relationship would cause more than one primary relationship to terminate at the same node or would create a circular path consisting of primary relationships. 11

Additional qualifiers:

```
RELATIONSHIP => relationship_specification
SOURCE_NODE => unique_identifier
TARGET_NODE => unique_identifier
```

**RELATIONSHIP** indicates the primary relationship to be modified. 12

**SOURCE_NODE** identifies the new source node. 13

**TARGET_NODE** identifies the new target node. 14

**[6] USE_CHANNEL_IN_USE** is returned if an attempt is made to set the values of PORT relationship attributes when protocol compatibility checking has been initiated by one or both clients of the channel and is still in progress. 15

**[7] USE_HAS_EMANATING_PRIMARIES** is returned if an attempt is made to delete the primary relationship to a node which has emanating primary relationships. 16

Additional qualifiers:

```
NODE => unique_identifier
```

**NODE** identifies the node to be deleted. 17

**[8] USE_NOT_SECONDARY_RELATIONSHIP** is returned if an attempt is made to delete a relationship and the identified relationship is a primary relationship which cannot be deleted by the CAIS interface called. 18

Additional qualifiers:

```
RELATIONSHIP => relationship_specification
```

**RELATIONSHIP** identifies the relationship to be deleted. 19

**[9] USE_ILLEGAL_CONTENTS_REPLACEMENT** is returned if an attempt is made to replace the contents of a node with a host file and there are existing open file handles to the node's current content. 20

**[10] USE_ILLEGAL_MANDATORY_ACCESS** is returned if the specified value for a MANDATORY_ACCESS_LIST parameter is semantically illegal. 21

Additional qualifiers:



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
CLASSIFICATION_LABEL => list
ERROR_SUBLIST        => current_linear_list_specification
ERROR_ITEM           => integer
CLASSIFICATION_ERROR => classification_error_type
```

**CLASSIFICATION_LABEL** is the MANDATORY_ACCESS_LIST parameter specified.

**ERROR_SUBLIST** identifies the linear list containing the semantic error.

**ERROR_ITEM** identifies the location of the error within the linear list identified by ERROR_SUBLIST.

**CLASSIFICATION_ERROR** indicates the nature of the semantic error.

**[11] USE_RELATION_IS_NOT_BIDIRECTIONAL** is returned if an operation requires an identification of a bidirectional relation and a unidirectional relation is identified.

Additional qualifiers:

```
RELATION => RD_unique_identifier
```

**RELATION** is the unidirectional relation identified.

**[12] USE_RELATION_IS_BIDIRECTIONAL** is returned if an operation requires an identification of a unidirectional relation and a bidirectional relation is identified.

Additional qualifiers:

```
RELATION => RD_unique_identifier
```

**RELATION** is the bidirectional relation identified.

**[13] USE_INAPPROPRIATE_NKI** is returned if an attempt is made to adopt a node kind interpretation for a node handle that does not apply to the node identified by the node handle.

Additional qualifiers:

```
NODE           => uniquo_identifier
NAME           => identifier
INTERPRETATION => NKI_unique_identifier
```

**NODE** identifies the node identified by the node handle.

**NAME** is the NODE_KIND_NAME value specified.

**INTERPRETATION** identifies the NKI identified by NAME.

**[14] USE_NOT_ADOPTED_NKI** is returned if an NKI identified as the current adopted interpretation for a node handle is not.

Additional qualifiers:

```
NODE           => unique_identifier
INTERPRETATION => interpretation_unique_identifier
```


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

**NODE** identifies the node identified by the node handle. 35

**INTERPRETATION** identifies the NKI specified. 36

**[15] USE_HAS_NO_ADOPTED_NKI** is returned if an attempt is made to cancel the adoption of a node kind interpretation for a node handle and the specified node handle does not have an adopted node kind interpretation. 37



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Additional qualifiers:

```
NODE => unique_identifier
NAME => identifier
```

**NODE** identifies the node identified by the node handle. 38

**NAME** is the NODE_KIND_NAME value specified. 39

**[16] USE_ALREADY_IN_VIEW** is returned if an attempt is made to create an new interpretation node in a view and the specified view already contains an interpretation node for the NKD or RD identified. This condition code is also returned if an attempt is made to import an AD or AKD into a view and the specified definition node is already named by that view. 40

Additional qualifiers:

```
VIEW            => VIEW_unique_identifier
DEFINITION      => definition_unique_identifier
DEFINITION_TYPE => definition_type
```

**VIEW** identifies the VIEW node to be modified. 41

**DEFINITION** identifies the definition node being imported into the view. 42

**DEFINITION_TYPE** indicates the type of the definition node. 43

**[17] USE_ILLEGAL_SPECIALIZATION** is returned if an attempted operation would result in a cyclic path composed of SPECIALIZATION_OF relationships. 44

Additional qualifiers:

```
ANCESTOR_DEFINITION    => NKD_unique_identifier
| ANCESTOR_DEFINITION => RD_unique_identifier
DEFINITION             => NKD_unique_identifier
| DEFINITION           => RD_unique_identifier
DEFINITION_TYPE        => definition_type
```

**ANCESTOR_DEFINITION** identifies the NKD or RD which is to become a generalization of the NKD or RD identified by DEFINITION. 45

**DEFINITION** identifies the NKD or RD which is to become a specialization of the NKD or RD identified by ANCESTOR_DEFINITION. 46

**DEFINITION_TYPE** indicates the type of the definition nodes. 47

**[18] USE-ILLEGAL_GENERALIZATION** is returned if an attempt is made to perform a generalization and one the view specified does not contain an interpretation for the intended generalization. 48

Additional qualifiers:

```
VIEW            => VIEW_unique_identifier
DEFINITION      => definition_unique_identifier
DIFINITION_TYPE => definition_type
```


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

**VIEW** identifies the specified VIEW node. 49

**DEFINITION** identifies the definition node that is not in the view. 50

**DEFINITION_TYPE** indicates the type of the definition node identified by definition. 51

**[19] USE-ANCESTOR_IS_BIDIRECTIONAL** is returned if an attempt is made to create a specialization or generalization and the RD identified is not a bidirectional relation definition, yet the specified ancestor RD is a bidirectional relation definition. 52

Additional qualifiers:

```
ANCESTOR_DEFINITION  =>  RD_udque_identifier
DEFINITION           =>  RD_unique_identifier
```

**ANCESTOR_DEFINITION** identifies the RD which is to become a generalization of the RD identified by DEFINITION. 53

**DEFINITION** identifies the RD which is to become a specialization of the RD identified by ANCESTOR_DEFINITION. 54

**[20] USE_DEFINITION_NOT_IN_VIEW** is returned if an attempt is made to define and name a property for a definition and the view specified does not contain an interpretation for the definition. 55

Additional qualifiers:

```
VIEW             =>  VIEW_unique_identifier
DEFINITION       =>  definition_unique_identifier
DEFINITION_TYPE  =>  definition_type
```

**VIEW** identifies the specified VIEW node. 56

**DEFINITION** identifies the definition node that is not in the view. 57

**DEFINITION_TYPE** indicates the type of the definition node identified by definition. 58

**[21] USE_DISCONNECTED_SUBLATTICE** is returned if a SPECIALIZATION_OF relationship to be deleted is the last one emanating from the specified NKD or RD. 59

Additional qualifiers:

```
DEFINITION       =>  NKD_unique_identifier
| DEFINITION     =>  RD_uique_identifier
DEFINTION_TYPE   =>  definition_type
```

**DEFINITION** identifies the NKD or RD from which exactly one SPECIALIZATION_OF relationship emanates. 60

**DEFINITION_TYPE** indicates the type of the definition. 61

**[22] USE_PROPERTY_ALREADY_DEFINED** is returned if an attempt is made to define a new property for a node kind or relation and the specified property is already a property of that definition. 62



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Additional qualifiers:

```
DEFINITION          => NKD_unique_identifier
| DEFINITION        => RD_unique_identifier
DEFINITION_TYPE     => definition_type
PROPERTY            => RD_unique_identifier
| PROPERTY          => AD_unique_identifier
PROPERTY_TYPE       => proporty_type
```

**DEFINITION** identifies the NKD or RD having the property. 63

**DEFINITION_TYPE** indicates the type of the definition having the property. 64

**PROPERTY** identifies the RD or AD which is the target of the existing property definition relationship. 65

**PROPERTY_TYPE** indicates the type of the already existing property. 66

**[23] USE_PROPERTY_NOT_DEFINED** is returned if an attempt is made to rename or delete a property for a node kind or relation and the specified property is not a defined property of that definition. 67

Additional qualifiers:

```
DEFINITION          => NKD_unique_identifier
| DEFINITION        => RD_unique_identifier
DEFINITION_TYPE     => definition_type
PROPERTY            => RD_unique_identifier
| PROPERTY          => AD_unique_identifier
PROPERTY_TYPE       => property_type
```

**DEFINITION** identifies the NKD or RD. 68

**DEFINITION_TYPE** indicates the type of the definition identified by DEFINITION. 69

**PROPERTY** identifies the specified RD or AD. 70

**PROPERTY_TYPE** indicates the type of the non-existent property. 71

**[24] USE_PREDEFINED_KEY** is returned if an attempt is made to define an attribute kind component and a key name specified in the KEY parameter identifies a predefine key that cannot be created by the user. 72

Additional qualifiers:

```
RELATIONSHIP    => relationship_specification
KEY             => AD-unique_identifier
```

**RELATIONSHIP** identifies the COMPONENT relationship to be created. 73

**KEY** identifies the AD for the predefine key that cannot be created by the user. 74

**[25] USE_NODE_SPECIFIED_TWICE** is returned if an operation requires a set of distinct nodes to be specified and a node is specified more than once. 75



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Additional qualifiers:

NODE => unique_identifier

**NODE** identifies the node specified repeatedly. 76

**[26] USE_NOT_DEFINITION_ATTRIBUTE** is returned if an attempt is made to modify a predefine attribute of a definition node and the identified attribute is not a predefine attribute for the definition node specified. 77

Additional qualifiers:

DEFINITION => definition_unique_identifier
DEFINITION_TYPE => definition_type
ATTRIBUTE => AD_unique_identifier

**DEFINITION** identifies the definition node to be modified. 78

**DEFINITION_TYPE** indicates the type of the definition node identified by DEFINITION. 79

**ATTRIBUTE** identifies the AD for the specified attribute. 80

**[27] USE_NOT_PROPERTY_ATTRIBUTE** is returned if an attempt is made to modify a predefine attribute of a property definition relationship and the identified attribute is not a predefine attribute for the property definition relationship identified. 81

Additional qualifiers:

RELATIONSHIP => relationship_specification
ATTRIBUTE => AD_unique_identifier

**RELATIONSHIP** identifies the specified property definition relationship to be modified. 82

**ATTRIBUTE** identifies the AD for the specified attribute. 83

**[28] USE_RELATIONSHIP_NOT_BIDIRECTIONAL** is returned if an attempt is made to determine the corresponding reverse relationship of a relationship that is not a bidirectional relationship. 84

Additional qualifiers:

RELATIONSHIP => relationship_specification

**RELATIONSHIP** identifies the relationship specified. 85

**[29] USE_HANDLER_PROCESS_NOT_ACTIVE** is returned if an attempt is made to associate an event handler process with a MONITOR node and the STATUS attribute of the specified PROCESS node has the value TERMINATED or ABORTED. 86

Additional qualifiers:

MONITOR => MONITOR_uique_identifier
PROCESS => PROCESS_unique_identifier



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

**MONITOR** identifies the specified MONITOR node. 87

**PROCESS** identifies the specified PROCESS node. 88

**[30] USE_PROCESS_ALREADY_HANDLER** is returned if an attempt is made to associate an event handler process with a MONITOR node and the specified PROCESS node represents the execution of a process that is already an event handler process for the specified MONITOR node. 89

Additional qualifiers:

```
MONITOR => MONITOR_unique_identifier
PROCESS => PROCESS_unique_identifier
```

**MONITOR** identifies the specified MONITOR node. 90

**PROCESS** identifies the specified PROCESS node. 91

**[31] USE_DUPLICATE_HANDLER** is returned if an attempt is made to associate an event handler process with a MONITOR node and the MONITOR node already has an associated handler process with the same mandatory access control subject classification as the process whose execution is represented by the specified PROCESS node. 92

Additional qualifiers:

```
MONITOR => MONITOR_unique_identifier
PROCESS => PROCESS_unique_identifier
```

**MONITOR** identifies the specified MONITOR node. 93

**PROCESS** identifies the specified PROCESS node. 94

**[32] USE_DIFFERENT_USER** is returned if an attempt is made to associate an event handler process with a MONITOR node and the USER node which is the parent of the MONITOR node is different from the USER node that is the target of the CURRENT_USER( ) relationship emanating from the PROCESS node specified. 95

Additional qualifiers:

```
MONITOR => MONITOR_unique_identifier
PROCESS => PROCESS_unique_identifier
```

**MONITOR** identifies the specified MONITOR node. 96

**PROCESS** identifies the specified PROCESS node. 97

**[33] USE_NODE_ATTRIBUTE_ALREADY_MONITORED** is returned if an attempt is made to add a node attribute to the set of attributes monitored by a MONITOR node and the node attribute to be monitored is already being monitored by the specified MONITOR node. 98

Additional qualifiers:

```
MONITOR   => MONITOR_unique_identifier
NODE      => unique_identifier
ATTRIBUTE => AD_unique_identifier
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

**MONITOR** identifies the specified MONITOR node. 99

**NODE** identifies the node with the attribute that is already being monitored by the specified **MONITOR** node. 100

**ATTRIBUTE** identifies the AD for the attribute being monitored. 101

**[34] USE_NO_ATTRIBUTES_MONITORED** is returned if an attempt is made to cancel the monitoring of an attribute and no attributes of the specified node or its relationships are monitored by the specified MONITOR node. 102

Additional qualifiers:

**MONITOR** identifies the specified MONITOR node. 103

**NODE** identifies the specified node. 104

**[35] USE_CANNOT_BE_MONITORED** is returned if an attempt is made to monitor a predefine attribute which cannot be monitored. 105

Additional qualifiers:

```
MONITOR   => MONITOR_unique_identifier
NODE      => unique_identifier
ATTRIBUTE => AD_unique_identifier
```

**MONITOR** identifies the specified MONITOR node. 106

**NODE** identifies the node with the attribute that cannot be monitored. 107

**ATTRIBUTE** identifies the AD for the attribute that cannot be monitored. 108

**[36] USE_CONTENTS_STRUCTURE_UNKNOWN** is returned if an attempt is made to export the contents of a node and the implementation cannot determine the structure of the node's contents due to the manner in which the node was created. (Attempts to export the contents of SECONDARY_STORAGE nodes created by the CREATE_SECONDARY_STORAGE_NODE interface may fail with this error condition if the contents were never opened using a data-transfer protocol or replaced via the IMPORT_CONTENTS interface.) 109

**[37] USE_NOT_EVENT_HANDLER_PROCESS** is returned if an attempt is made to retrieve an attribute monitor event and the current process is not an attribute monitor event handler process that has been associated with a MONITOR node. 110

**[38] USE_ILLEGAL_PRIORITY** is returned if a PROCESS_PRIORITY value is semantically illegal. 111

Additional qualifiers:

```
PRIORITY => integer
```

**PRIORITY** indicates the PROCESS_PRIORITY value specified. 112

**[39] USE_PRIORITY_RANGE_NOT_DETERMINABLE** is returned if an attempt is made to determine either the lowest or highest priority at which the current process can initiate a new process on a specified processor resource and the current process cannot initiate processes on that processor. 113



Source: https://assist.dla.mil -- Downloaded: 202610-26T07:54Z
Check the source to verify that this is the current version before use.

Additional qualifiers:

PROCESSOR => PROCESSOR_unique_identifier

**PROCESSOR** identifies the PROCESSOR node. 114

**[40] USE_INVALID_PRIORITY_MODIFICATION** is returned if an attempt is made by the current process to alter its priority and the current process is not allowed to set its priority to the specified PROCESS_PRIORITY value according to implementation-defied criteria. 115

Additional qualifiers:

PRIORITY => integer

**PRIORITY** identifies the PROCESS_PRIORITY value specified. 116

**[41] USE_NO_MULTIPLE_FILE_HANDLES** is returned if an operation is attempted which violates an implementation-defied restriction on multiple file handles identifying the same file node contents. 117

Additional qualifiers:

CHANNEL => CHANNEL_unique_identifier

**CHANNEL** identifies the CHANNEL node specified. 118

**[42] USE_BAD_BLOCK_SIZE** is returned if an attempt is made to load a tape on a tape drive and the specified BLOCK_SIZE is less than MINIMUM_TAPE_BLOCK_LENGTH or greater than MAXIMUM_TAPE_BLOCK_LENGTH or if an attempt is made to write a block which is not of the same size as the block size specified when the tape was loaded. 119

Additional qualifiers:

BLOCK_SIZE => integer

**BLOCK_SIZE** indicates the specified BLOCK_SIZE value. 120

**[43] USE_ILLEGAL_FILE_HANDLE_RESET** is returned if an attempt is made to reset a file handle and the CAIS implementation does not support the resetting of the file handle to the specified mode. 121

Additional qualifiers:

FILE => FILE_unique_identifier
MODE => identifier

**FILE** identifies the FILE node denoted by the specified file handle. 122

**MODE** indicates the specified new mode for the file handle. 123

**[44] USE_INVALID_MODE** is returned if an attempt is made to open a file handle to an external file and the external file is not allowed for the specified mode. 124



Source: https://assist.dla.mil -- Downloaded: 202610-26T07:54Z
Check the source to verify that this is the current version before use.

Additional qualifiers:

```
FILE => FILE_unique_identifier
MODE => identifier
```

**FILE** identifies the FILE node denoted by the specified file handle. 125

**MODE** indicates the specified new mode for the file handle. 126

**[45] USE_NO_HOST_SYSTEM** is returned if an attempt is made to copy the contents of a node to a host file and there is no underlying host system. 127

**[46] USE_INVALID_HOST_FILE_NAME** is returned if an attempt is made to copy the contents of a node to or from a host file and the implementation-dependent host file name specified is invalid. 128

Additional qualifiers:

```
NODE      => unique_identifier
FILE_NAME => string
```

**NODE** identifies the node whose contents are to be copied. 129

**FILE_NAME** indicates the host file name specified. 130

**[47] USE_ILLEGAL_CONTENTS_CHARACTERISTICS** is returned if an attempt is made to cop y the contents of a node to or from a host file and the implementation-dependent characteristic information specified does not permit the copying of the node's contents. 131

Additional qualifiers:

```
NODE            => unique_identifier
FILE_NAME       => string
CHARACTERISTICS => list
```

**NODE** identifies the node whose contents are to be copied. 132

**FILE_NAME** indicates the host file name specified. 133

**CHARACTERISTICS** indicates the implementation-dependent characteristic information specified. 134

**[48] USE_NO_POTENTIAL_MEMBER** is returned if an attempt is made to adopt a role and no adopted role oft he current process is a potential member of the associated group. 135

Additional qualifiers:

```
CROUP => GROUP_unique_identifier
```

**GROUP** identifies the GROUP node specified. 136

**[49] USE_DEFAULT_ROLE** is returned if an attempt is made to unadopt a role which is a default role of the current user or of the current process's executable image. 137



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Additional qualifiers:

```
GROUP =>      GROUP_unique_identifier
ROLE_TYPE  =>  role_type
```

**GROUP** identifies the GROUP node specified. 138

**ROLE_TYPE** indicates whether the group is the default role of the current user or the default role of the executable image. 139

**[50] USE_PROCESS_STILL_ACTIVE** is returned if an attempt is made to determine the time of the termination or abortion of a process and the process has not yet terminated or aborted. 140

Additional qualifiers:

```
MODE  =>  unique_identifier
```

NODE identifies the PROCESS node specified. 141

**[51] USE_KEY_ATTRIBUTE_NAMED** is returned if a key attribute is named in an ATTRIBUTES parameter. 142

Additional qualifiers:

```
ATTRIBUTE_NAME  =>  string
RELATION        =>  RD_unique_identifier
```

**ATTRIBUTE_NAME** is the name of the attribute which is a key. 143

**RELATION** identifies the RD in which the named attribute is a key. 144

**[52] USE_RELATION_NOT_BIDIRECTIONAL** is returned if an operation requires a bidirection relation definition, but a unidirectional relation definition is specified. 145

Additional qualifiers:

```
RELATION  =>  RD_unique_identifier
```

**RELATION** identifies the RD in which is not bidirectional. 146

**[53] USE_WRONG_DIRECTIONALITY** is returned if an operation would change a unidirectional relationship into a bidirectional relationship or vice versa. 147

Additional qualifiers:

```
RELATIONSHIP  =>  relationship_specification
RELATION      =>  RD_unique_identifier
```

**RELATIONSHIP** identifies the relationship to be modified. 148

**RELATION** identifies the new RD for the relationship. 149

**[54] USE_INCOMPATIBLE_CONTENT_STRUCTURE** is returned if an attempt is made to open a file handle to a FILE node and the structure of the node's content is incompatible with the protocol used. 150



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Additional qualifiers:

FILE => FILE_unique_identifier

**FILE** identifies the FILE node to reopened. 151

**[55] USE_INCOMPATIBLE_CLIENT** is returned if an attempt is made to open a protocol over a channel and the implementation of the protocol does not support data transfer between the types of client nodes involved. 152

Additional qualifiers:

CLIENT => unique_identifier

**CLIENT** identifies the client which is incompatible with the protocol. 153

**[56] USE_PROCESS_NOT_ACTIVATED** is returned if an attempt is made to perform an operation on a process that is inappropriate because the process has never been activated. 154

Additional qualifiers:

PROCESS => PROCESS_unique_identifier

**PROCESS** identifies the PROCESS node for the process which has never been activated. 156

**[57] USE_PROCESS_CANNOT_BE_ACTIVATED** is returned if an attempt is made to activate a process which does not have the CREATED status. The process has already been activated or has been aborted. 156

Additional qualifiers:

PROCESS => PROCESS_unique_identifier

**PROCESS** identifies the PROCESS node for the process which cannot be activated. 157

**[58] USE_TRANSACTION_NOT_COMPLETED** is returned if an attempt is made to commit a transaction which has an open subtransaction or encloses a process which has not terminated. 158

Additional qualifiers:

PROCESS => PROCESS_unique_identifier

**PROCESS** identifies a PROCESS node for a process which has not terminated. If all enclosed processes have terminated but a subtransaction has not been closed, the empty string (“”) is returned. 159

**[59] USE_NO_INTENT_TO_ACCESS** is returned if an attempt is made to include a node in a transaction using a node handle with no access intent. 160



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Additional qualifiers:

MODE => unique_identifier

**NODE** identifies the node to be included in the transaction. 161

**[60] USE_PREDEFINED_NODE_ATTRIBUTE** is returned if an attempt is made to modify a predefine, CAIS-controlled node attribute. 162

Additional qualifiers:

NODE => unique_identifier
ATTRIBUTE => AD_unique_identifier

**NODE** identifies the predefine node. 163

**ATTRIBUTE** identifies the predefine, CAIS-controlled attribute. 164

**[61] USE_PREDEFINED_PATH_ATTRIBUTE** is returned if an attempt is made to modify a predefined, CAIS-controlled relationship attribute. 165

Additional qualifiers:

RELATIONSHIP => relationship_specification
ATTRIBUTE => AD_unique_identifier

**RELATIONSHIP** identifies the predefine relationship. 166

**ATTRIBUTE** identifies the predefine, CAIS-controlled attribute. 167

**[62] USE_PREDEFINED_NODE** is returned if an attempt is made to delete a predefine, CAIS-controlled node. 168

Additional qualifiers:

NODE => unique_identifier

**NODE** identifies the predefine node. 169

**[63] USE_PREDEFINED_RELATIONSHIP** is returned if an attempt is made to delete or change the source or target of a predefined, CAIS-controlled relationship. 170

Additional qualifiers:

RELATIONSHIP => relationship-specification

**RELATIONSHIP** identifies the predefine relationship. 171

**[64] USE_INAPPROPRIATE_LINE_LENGTH** is returned if an attempt is made to set the maximum line length of a CAIS_TEXT_IO file handle to an inappropriate value. 172


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Additional qualifiers:

MAXIMUM_LINE_LENGTH => integer

**MAXIMUM_LINE_LENGTH** indicates the value of the MAXIMUM_LINE_LENGTH protocol parameter attribute of the CHANNEL node. 173

**[66] USE_INAPPROPRIATE_PAGE_LENGTH** is returned if an attempt is made to set the maximum page length of a CAIS_TEXT_IO file handle to an inappropriate value. 174

MAXIMUM_PAGE_LENGTH => integer

**MAXIMUM_PAGE_LENGTH** indicates the value of the MAXIMUM_PAGE_LENGTH protocol parameter attribute of the CHANNEL node. 175

**[66] USE_INVALID_FORM_WRITE** is returned if an attempt is made to write to a form whose area qualifier requires space on the form and the active position contains an area qualifier. 176

**[67] USE_INVALID_FORM_SIZE** is returned if an attempt is made to activate a form on a terminal and the size of the form does not match the size of the terminal. 177

**[68] USE_AREA_QUALIFIER_CONFLICT** is returned if an attempt is made to activate a form on a terminal and the space requirements of the area qualifier on the form do not match the space requirements of the area qualifier on the terminal. 178

## 13.3.2 Conditions for CAIS_PROCESS_DEFINITIONS exceptions

### 13.3.2.1 EXECUTABLE_IMAGE_ERROR_DETECTED conditions

When the exception indication of an established status indicator is EXECUTABLE_IMAGE_ERROR_- DETECTED, one of the following condition codes is returned as the value of the CONDITION status information item. 1

**[1] EXE_NOT_EXECUTABLE_IMAGE** is returned if the contents of the EXECUTABLE_- , IMAGE node specified cannot be executed.

Additional qualifiers:

NODE => unique_identifier

**NODE** identifies the specified node.

## 13.3.3 Conditions for CAIS_IO_DEFINITIONS exceptions

### 13.3.3.1 CLIENT_ERROR_DETECTED conditions

When the exception indication of an established status indicator is CLIENT_ERROR_DETECTED, one of the following condition codes is returned as the value of the CONDITION status information item. 1

**[1] CLI_CALLER_NOT_CLIENT** is returned if the calling process is not a client process of the channel to which the operation applies. 2



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Additional qualifiers:

CHANNEL => CHANNEL_unique_identifier

**CHANNEL** identifies the CHANNEL node of the channel specified in the call.

**[2] CLI_WRONG_CLIENT_KIND** is returned if a specified client node is not a FILE, DEVICE, PROCESS, or GATEWAY node.

Additional qualifiers:

CLIENT_NODE => unique_identifier

**CLIENT_NODE** identifies the client node specified in the call.

**[3] CLI_PROCESS_OUT_OF_EXECUTION** is returned if the specified client node is a PROCESS node whose CURRENT_STATUS attribute is not equal to READY or SUSPENDED.

Additional qualifiers:

CLIENT_NODE => unique_identifier

**CLIENT_NODE** identifies the client process node specified in the call.

**[4] CLI_NO_PROCESS** is returned if at least one of the client nodes for a prospective channel is not a PROCESS node.

Additional qualifiers:

CLIENT_NODE => unique_identifier
OTHER_CLIENT_NODE => unique_identifier

**CLIENT_NODE** identifies one of the client nodes specified in the call.

**OTHER_CLIENT_NODE** identifies the other client node specified in the call.

**[5] CLI_NOT_CLIENT** is returned if an attempt is made to open a protocol on a channel and the opposite client identified is not a client of the specified channel.

Additional qualifiers:

CHANNEL => CHANNEL_unique_identifier
OPPOSITE_CLIENT => unique_identifier

**CHANNEL** identifies the CHANNEL node of the channel specified in the call.

**OPPOSITE_CLIENT** identifies the opposite client node specified in the call which is not a client of the specified channel.

### 13.3.3.2 DATA_ERROR_DETECTED conditions

When the exception indication of an established status indicator is DATA_ERROR_DETECTED, one of the following condition codes is returned as the value of the CONDITION status information item.

**[1] DAT_WRONG_TYPE** is returned if the element read cannot be interpreted as a value of the required type.



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Additional qualifiers:

CLIENT_NODE => unique_identifier

**CLIENT_NODE** identifies the opposite client node corresponding to the file handle specified in the call. (This qualifier is omitted from the status information list if a channel is not used in the transfer of the data such as in reading from a string object using interfaces in the package CAIS_TEXT_IO.) 3

**[2] DAT_OUT_OF_RANGE** is returned if the value input does not belong to the range of the required type or subtype. 4

Additional qualifiers:

CLIENT_NODE => unique_identifier

**CLIENT_NODE** identifies the opposite client node corresponding to the file handle specified in the call. (This qualifier is omitted from the status information list if a channel is not used in the transfer of the data such as in reading from a string object using interfaces in the package CAIS_TEXT_IO.) 5

**[3] DAT_SYNTAX** is returned if the input character sequence fails to satisfy the required syntax. 6

Additional qualifiers:

CLIENT_NODE => unique_identifier

**CLIENT_NODE** identifies the opposite client node corresponding to the file handle specified in the call. (This qualifier is omitted from the status information list if a channel is not used in the transfer of the data such as in reading from a string object using interfaces in the package CAIS_TEXT_IO.) 7

## 13.3.3.3 END_ERROR_DETECTED conditions

When the exception indication of an established status indicator is END_ERROR_DETECTED, one of the following condition codes is returned as the value of the CONDITION status information item. 1

**[1] END_ATTEMPT_TO_PASS_END** is returned if an attempt is made to read or skip past the end of a file. 2

Additional qualifiers:

CLIENT_NODE => unique_identifier

**CLIENT_NODE** identifies the opposite client node corresponding to the file handle specified in the call. 3

**[2] END_BUFFER_EMPTY** is returned if an attempt is made to read from an empty buffer and the opposite client process has closed the protocol. 4



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Additional qualifiers:

CLIENT_NODE => unique_identifier

**CLIENT_NODE** identifies the opposite client node corresponding to the file handle specified in the call.

### 13.3.3.4 LAYOUT_ERROR_DETECTED conditions

When the exception indication of an established status indicator is LAYOUT_ERROR_DETECTED, one of the following condition codes is returned as the value of the CONDITION status information item.

**[1] LAY_OUT_OF_RANGE** is returned if a value returned exceeds COUNT'LAST.

Additional qualifiers:

CLIENT_NODE => unique_identifier

**CLIENT_NODE** identifies the opposite client node corresponding to the file handle specified in the call.

**[2] LAY_OUT_OF_BOUNDS** is returned if an attempt was made to set a column or line number beyond a specified maximum line or page length.

Additional qualifiers:

CLIENT_NODE => unique_identifier

**CLIENT_NODE** identifies the opposite client node corresponding to the file handle specified in the call.

**[3] LAY_STRING_TOO_LONG** is returned if an attempt was made to PUT too many characters in a string.

Additional qualifiers:

CLIENT_NODE => unique_identifier

**CLIENT_NODE** identifies the opposite client node corresponding to the file handle specified in the call.

**[4] LAY_BOUNDED** is returned if an attempt was made to perform an operation that requires an unbounded line length or page size using a file handle has such a bound.

Additional qualifiers:

CLIENT_NODE => unique_identifier

**CLIENT_NODE** identifies the opposite client node corresponding to the file handle specified in the call.


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

### 13.3.3.6 MODE_ERROR_DETECTED conditions

When the exception indication of an established status indicator is MODE_ERROR_DETECTED, one of the following condition codes is returned as the value of the CONDITION status information item. 1

**[1] MOD_OPERATION_CONFLICT** is returned if the attempted operation conflicts with the mode with which the protocol was opened. 2

Additional qualifiers:

```
MODE => identifier
```

**MODE** gives the mode in effect for the protocol. 3

**[2] MOD_DIRECTION_CONFLICT** is returned if an attempt is made to open a protocol with a mode that conflicts with the DIRECTION attribute of the local PORT relationship. 4

Additional qualifiers:

```
MODE      => identifier
DIRECTION => direction_type
```

**MODE** gives the mode in effect for the protocol. 5

**DIRECTION** gives the value of the DIRECTION attribute on the local PORT relationship. 6

**[3] MOD_MODE_CONFLICT** is returned if an attempt is made to open a protocol with a mode that conflicts with the mode specified by the opposite process client. 7

Additional qualifiers:

```
MODE          => identifier
OPPOSITE_MODE => identifier
```

**MODE** gives the mode specified by the caller. 8

**OPPOSITE_MODE** gives the mode specified by the opposite client. 9

### 13.3.3.6 OPEN_ DENIED_ DETECTED conditions

When the exception indication of an established status indicator is OPEN_DENIED_DETECTED, one of the following condition codes is returned as the value of the CONDITION status information item. 1

**[1] OPD_CHANNEL_DELETED** is returned if the opposite client processor remote client process caused the channel to be deleted. 2

**[2] OPD_OPPOSITE_OPEN_FAILED** is returned when a protocol open issued by the opposite client process fails for protocol compatibility reasons which are not determinable by the current process. 3



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

## 13.3.3.7 PORT_ERROR_DETECTED conditions

When the exception indication of an established status indicator is PORT_ERROR_DETECTED, one of the following condition codes is returned as the value of the CONDITION status information item. 1

**[1] POE_INCOMPATIBLE_RD** is returned if there does not exist a CHANNEL NKD such that , relationships of the specified PORT relations cannot both terminate at some CHANNEL node.

Additional qualifiers:

```
PORT       => RD_unique_identifier
OTHER-PORT => RD_unique_identifier
```

**PORT** gives the UID of one PORT RD. 3

**OTHER_PORT** gives the UID of the other PORT RD. 4

**[2] POE_INCOMPATIBLE_DIRECTIONS** is returned if the DIRECTION attributes of the channel are incompatible or missing. 5

Additional qualifiers:

```
DIRECTION       => direction_type
OTHER_DIRECTION => direction_type
```

**DIRECTION** gives the value of one DIRECTION attribute. 6

**OTHER_DIRECTION** gives the value of the other DIRECTION attribute. 7

## 13.3.3.8 PROTOCOL-ERROR-DETECTED conditions

When the exception indication of an established status indicator is PROTOCOL_ERROR_DETECTED, one of the following condition codes is returned as the value of the CONDITION status information item. 1

**[1] PRO_NOT_ALLOWED** is returned if the name of the protocol which is to be opened is not in the values of both ALLOWED_PROTOCOLS attributes. 2

**[2] PRO_DIFFERENT_PROTOCOLS** is returned if the name of the protocol which is to be opened is not the same as the value of the CONTROLLING_PROTOCOL attribute of the CHANNEL node, if the value of that attribute is other than NO_PROTOCOL. 3

Additional qualifiers:

```
PROTOCOL => identifier
```

**PROTOCOL** gives the value of the CONTROLLING_PROTOCOL attribute. 4

**[3] PRO_RANGES_INCOMPATIBLE** is returned if the values of the protocol range parameter attributes of the PORT relationships are incompatible. 5


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

**[4] PRO_CONSTRAINT_VIOLATION** is returned if an attempted data transfer operation violates a constraint specified by a protocol parameter. 6

**[5] PRO_MISSING_PARAMETER** is returned if an a required protocol parameter attribute is not present on one or both PORT relationships of a channel at the time of an attempted protocol open. 7

**[6] PRO_AMBIGUOUS_PORT_SELECTION** is returned if a protocol open operation is attempted and the calling process is both clients of the channel and a selection between the two PORT relationships emanating from the current process node (for the purposes of protocol parameter compatibility checking) cannot be made given the mode specified on the open operation. 8

### 13.3.3.9 TIME_EXPIRED_DETECTED conditions

When the exception indication of an established status indicator is TIME_EXPIRED_DETECTED, one of the following condition codes is returned as the value of the CONDITION status information item. 1

**[1] TEX_DELAYED_BEYOND_TIME_LIMIT** is returned if the operation is blocked for a time exceeding the specified limit. 2

## 13.3.4 Conditions for CAIS_FRAME_IO exceptions

### 13.3.4.1 QUEUE_ FULL_ DETECTED conditions

When the exception indication of an established status indicator is QUEUE_FULL_DETECTED, one of the following condition codes is returned as the value of the CONDITION status information item. 1

**[1] QFU_QUEUE_FULL** is returned if a non-blocking WRITE operation cannot complete because the outgoing buffer is full. 2

Additional qualifiers:

CLIENT_NODE => unique_identifier

**CLIENT_NODE** identifies the opposite client node corresponding to the file handle specified in the call. 3

### 13.3.4.2 QUEUE_EMPTY_DETECTED conditions

When the exception indication of an established status indicator is QUEUE_EMPTY_DETECTED, one of the following condition codes is returned as the value of the CONDITION status information item. 1

**[1] QEM_QUEUE_EMPTY** is returned if a non-blocking READ operation cannot complete because the incoming buffer is full. 2


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Additional qualifiers:

CLIENT_NODE => unique_identified

**CLIENT_NODE** identifies the opposite client node corresponding to the file handle specified in the call. 3

## 13.3.5 Conditions for CAIS_MAGNETIC_TAPE_IO exceptions

### 13.3.6.1 TAPE_STATUS_ERROR_DETECTED conditions

When the exception indication of an established status indicator is TAPE_STATUS_ERROR_DETECTED, one of the following condition codes is returned as the value of the CONDITION status information item. 1

**[1] TAP_NOT_MOUNTED** is returned if the initial TAPE_DRIVE_STATUS is expected to be MOUNTED, but was not. This condition is also used if the initial TAPE_DRIVE_STATUS is expected to be either MOUNTED or LOADED. 2

Additional qualifiers:

TAPE_STATUS => tape-status-type

**TAPE_STATUS** gives the actual status of the tape drive at the time the attempted operation began. 3

**[2] TAP_NOT_LOADED** is returned if the initial TAPE_DRIVE_STATUS is expected to be LOADED, but was not. 4

Additional qualifiers:

TAPE_STATUS => tape_status_type

**TAPE_STATUS** gives the actual status of the tape drive at the time the attempted operation began. 5

## 13.3.6 Conditions for CAIS_TERMINAL_IO exceptions

### 13.3.6.1 BUTTON_NAME_ERROR_DETECTED conditions

When the exception indication of an established status indicator is BUTTON_NAME_ERROR_DETECTED, the following condition code is returned as the value of the CONDITION status information item. 1

**[1] BUT_NONEXISTENT_BUTTON** is returned if a non-existent button is referenced. 2



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Additional qualifiers:

CLIENT_NODE => unique_identifier

**CLIENT_NODE** identifies the opposite client node corresponding to the terminal handle specified in the call. 3

## 13.3.6.2 FUNCTION_KEY_STATUS_ERROR_DETECTED conditions

When the exception indication of an established status indicator is FUNCTION_KEY_STATUS_-ERROR_DETECTED, the following condition code is returned as the value of the CONDITION status information item. 1

**[1] FKY_NONEXISTENT_FUNCTION_KEY** is returned if a non-existent function key is referenced. 2

Additional qualifiers:

CLIENT_NODE => unique_identifier

**CLIENT_NODE** identifies the opposite client node corresponding to the terminal handle specified in the call. 3

## 13.3.6.3 INPUT_EVENT_QUEUE_EMPTY_DETECTED conditions

When the exception indication of an established status indicator is INPUT_EVENT_QUEUE_EMPTY_-DETECTED, the following condition code is returned as the value of the CONDITION status information item. 1

**[1] IQE_QUEUE_EMPTY** is returned if a non-blocking READ operation cannot complete because the input event queue is empty. 2

Additional qualifiers:

CLIENT_NODE => unique_identifier

**CLIENT_NODE** identifies the opposite client node corresponding to the terminal handle specified in the call. 3

## 13.3.6.4 LOCATOR_STATUS_ERROR_DETECTED conditions

When the exception indication of an established status indicator is LOCATOR_STATUS_ERROR_DE-TECTED, the following condition code is returned as the value of the CONDITION status information item. 1

**[1] LCS_LOCATOR_NOT_ENABLED** is returned if an attempt is made to determine the position of the locator cursor and the locator for the virtual terminal is not enabled. 2



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Additional qualifiers:

CLIENT_MODE => unique_identifier

**CLIENT_NODE** identifies the opposite client node corresponding to the terminal handle specified in the call. 3

**[2] LCS_CURSOR_NOT_ON_ANY_SCREEN** is returned if an attempt is made to determine which screen most forward in the pasting order contains the locator cursor and the locator cursor position does not coincide with a character position of any pasted screen. 4

Additional qualifiers:

CLIENT_NODE => unique_identifier

**CLIENT_NODE** identifies the opposite client node corresponding to the terminal handle specified in the call. 5

**[3] LCS_CURSOR_NOT_ON_SCREEN** is returned if an attempt is made to determine if the locator cursor is positioned over the border of a screen and the locator cursor is not positioned over the identified screen. 6

Additional qualifiers:

CLIENT_NODE => unique_identifier

**CLIENT_NODE** identifies the opposite client node corresponding to the terminal handle specified in the call. 7

## 13.3.6.6 SCREEN_NOT_ON_ISPLAY_DETECTED conditions

When the exception indication of an established status indicator is SCREEN_NOT_ON_DISPLAY_-DETECTED, one of the following condition codes is returned as the value of the CONDITION status information item. 1

**[1] SCR_NOT_PASTED** is returned if an attempt is made to manipulate a screen relative to a virtual display to which the screen is not pasted. 2

## 13.3.6.6 SCREEN_POSITION_ERROR_DETECTED conditions

When the exception indication of an established status indicator is SCREEN_POSITION_ERROR_DE-TECTED, the following condition code is returned as the value of the CONDITION status information item. 1

**[1] SCP_OFF_SCREEN** is returned if an output operation beyond the boundaries of a screen is attempted. 2



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

### 13.3.6.7 VIRTUAL_DISPLAY_POSITION_ERROR_DETECTED conditions

When the exception indication of an established status indicator is VIRTUAL_DISPLAY_POSITION_-ERROR_DETECTED, the following condition code is returned as the value of the CONDITION status information item. 1

**[1] VDP_OFF_SCREEN** is returned if an output operation beyond the boundaries of the virtual display is attempted. 2

## 13.3.7 Conditions for CAIS_COMMON_IO exceptions

### 13.3.7.1 UID_ERROR_DETECTED conditions

When the exception indication of an established status indicator is UID_ERROR_DETECTED, one of the following condition codes is returned as the value of the CONDITION status information item. 1

**[1] UID_NOT_UNIQUE** is returned if when an attempt is made to create a node with the UID indicated by an introduced node and the UID is not unique within the CAIS instance. 2

Additional qualifiers:

```
NODE => unique_identifier
UID  => unique_identifier
```

**NODE** identifies the introduced node. 3

**UID** indicates the UID which is not unique within the CAIS instance. 4

## 13.3.8 Conditions for CAIS_LIST_MANAGEMENT exceptions

### 13.3.8.1 ITEM_KIND_ERROR_DETECTED conditions

When the exception indication of an established status indicator is ITEM_KIND_ERROR_DETECT-ED, one of the following condition codes is returned as the value of the CONDITION status information item. 1

**[1] IKE_WRONG_ITEM_KIND** is returned if the kind of an item in a list is inappropriate for the attempted operation. 2

Additional qualifiers:

```
LIST               => list
CURRENT_LIST       => current_linear_list_specification
ITEM_POSITION      => integer
REQUIRED_ITEM_KIND => list_item_kind
```

**LIST** is the list identified by the LIST_TYPE value specified or, in the case of interfaces which retrieve simple attribute values directly, the list form of the value of an attribute being retrieved. 3

**CURRENT_LIST** identifies the current linear list of LIST. 4

**ITEM_POSITION** is the position of the item in the current linear list. 5

**REQUIRED_ITEM_KIND** indicates the kind of list item required by the operation attempted. 6


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

**[2] IKE_NO_ENUMERATION_VALUE** is returned if the value of an identifier item is to be retrieved in enumeration form but there is no value of the enumeration type whose image (as defined by the Ada attribute operator 'IMAGE) is equal to the identifier text form of the identifier item under identifier equality. 7

Additional qualifiers:

```
LIST          => list
CURRENT_LIST  =>  current_linear_list_specification
ITEM_POSITION => integer
```

**LIST** is the list identified by the LIST_TYPE value specified or, in the case of interfaces which retrieve simple attribute values directly, the list form of the value of an attribute being retrieved. 8

**CURRENT_LIST** identifies the current linear list of LIST. 9

**ITEM_POSITION** is the position of the item in the current linear list. 10

**[3] IKE_WRONG_ENUMERATION_POSITION** is returned if the value of an enumerated identifier item is to be retrieved in enumeration form but the position of the corresponding enumeration value in the base type of the enumeration type does not match the enumeration position of the item. The corresponding emuneration value is the value of the enumeration type whose image (as defined by the Ada attribute operator 'IMAGE) is equal to the identifier text form of the enumerated identifier item under identifier equality. 11

Additional qualifiers:

```
LIST          => list
CURRENT_LIST  =>  current_linear_list_specification
ITEM_POSITION => integer
```

**LIST** is the list identified by the LIST_TYPE value specified or, in the case of interfaces which retrieve simple attribute values directly, the list form of the value of an attribute being retrieved. 12

**CURRENT_LIST** identifies the current linear list of LIST. 13

**ITEM_POSITION** is the position of the item in the current linear list. 14

## 13.3.8.2 LIST_KIND_ERROR_DETECTED conditions

When the exception indication of an established status indicator is LIST_KIND_ERROR_DETECTED, one of the following condition codes is returned as the value of the CONDITION status information item. 1

**[1] LKE_EMPTY_LIST** is returned if the current linear list is empty and the operation attempted requires either a named or unnamed list. 2

Additional qualifiers:

```
LIST         => list
CURRENT_LIST =>  current-linear.list-specification
```

**LIST** is the list identified by the LIST_TYPE value specified. 3

**CURRENT_LIST** identifies the current linear list of LIST. 4

**[2] LKE_NOT_NAMED** is returned if the current linear list is not a named list. 5



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Additional qualifiers:

```
LIST         => list
CURRENT_LIST => current_linear_list_spcification
```

**LIST** is the list identified by the LIST_TYPE value specified. 6

**CURRENT_LIST** identifies the current linear list of LIST. 7

**[3] LKE_KIND_MISMATCH** is returned if the current linear lists specified are not of the same kind and neither of them is the empty list. 8

Additional qualifiers:

```
LIST_1 => list
LIST_2 => list
CURRENT_LIST_1 => current_linear_list_specification
CURRENT_LIST_2 => current_linear_list_specification
```

**LIST_1** and **LIST_2** are the lists identified by the LIST_TYPE values specified. 9

**CURRENT_LIST_1** and **CURRENT_LIST_2** identify the current linear lists of LIST_1 and LIST_2 respectively. 10

**[4] LKE_CANNOT_BE_NAMED** is returned if an attempt is made to insert an item by name into an unnamed list. 11

Additional qualifiers:

```
LIST         => list
CURRENT_LIST => current_linear_list_specification
NAME         => identifier
```

**LIST** is the list identified by the LIST_TYPE value specified. 12

**CURRENT_LIST** identifies the current linear list of LIST. 13

**NAME** is the list item name specified. 14

**[5] LKE_MUST_BE_NAMED** is returned if an attempt is made to insert an item into a named list without specifying a name. 15

Additional qualifiers:

```
LIST          => list
CURRENT_LIST  => current_linear_list_specification
ITEM_POSITION => integer
```

**LIST** is the list identified by the LIST_TYPE value specified; 16

**CURRENT_LIST** identifies the current linear list of LIST. 17

**ITEM_POSITION** indicates the position within the current linear list where the insertion was attempted. 18


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

## 13.3.8.3 LIST_POSITION-ERROR_DETECTED conditions

When the exception indication of an established status indicator is LIST_POSITION_ERROR_DETECTED, one of the following condition codes is returned as the value of the CONDITION status information item. 1

**[1] LPE_POSITION_EXCEEDS_LENGTH** is returned if the specified list item position is larger than the current length of the current linear list. 2

Additional qualifiers:

```
LIST          => list
CURRENT_LIST  => current_linear_list_specification
ITEM_POSITION => integer
```

**LIST** is the list identified by the LIST_TYPE value specified. 3

**CURRENT_LIST** identifies the current linear list of LIST. 4

**ITEM_POSITION** is the list item position specified within the current linear list. 5

**[2] LPE_START_BEYOND_END** is returned if the specified start list item position is greater than the specified end list item position. 6

Additional qualifiers:

```
LIST          => list
CURRENT_LIST  => current_linear_list_specification
START_POSITION => integer
END_POSITION  => integer
```

**LIST** is the list identified by the LIST_TYPE value specified. 7

**CURRENT_LIST** identifies the current linear list of LIST. 8

**START_POSITION** is the specified starting list item position. 9

**END_POSITION** is the specified ending list item position. 10

**[3] LPE_CURRENT_IS_OUTERMOST** is returned if the current linear list of the specified list is the outermost linear list and an attempt is made to establish the containing list of the current linear list as the current linear list. 11

Additional qualifiers:

```
LIST => list
```

**LIST** is the list identified by the LIST-TYPE value specified.

## 13.3.8.4 NAMED_LIST_ERROR_DETECTED conditions

When the exception indication of an established status indicator is NAMED_LIST_ERROR_DETECTED, one of the following condition codes is returned as the value of the CONDITION status information item. 1

**[1] NLE_DUPLICATE_ITEM_NAME** is returned if the specified list text designates a list with two or more items of the same name. 2



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Additional qualifiers:

```
LIST_TEXT => string
NAME      => identifier
```

**LIST_TEXT** is the LIST_TEXT value specified. 3

**NAME** indicates the duplicate item name. 4

**[2] NLE_NOT_NAMED_LIST** is returned if an attempt is made to reference a list item by name within an unnamed list. 5

Additional qualifiers:

```
LIST         => list
CURRENT_LIST => current_linear_list_specification
NAME         => identifier
```

**LIST** is the list identified by the LIST_TYPE value specified. 6

**CURRENT_LIST** identifies the current linear list of LIST. 7

**NAME** is the specified item name. 8

**[3] NLE_NAME_CONFLICT** is returned if an attempt is made to insert one list into another list or concatenate one list with another list and the current linear lists of the specified lists are both named and contain an item of the same name. 9

Additional qualifiers:

```
LIST_1         => list
LIST_2         => list
CURRENT_LIST_1 => current_linear_list_specification
CURRENT_LIST_1 => current_linear_list_specification
NAME           => identifier
```

**LIST_1** and **LIST_2** are the lists identified by the LIST_TYPE values specified. 10

**CURRENT_LIST_1** and **CURRENT_LIST_2** identify the current linear lists of LIST_1 and LIST_2 respectively. 11

**NAME** indicates the duplicate list item name. 12

**[4] NLE_NAME_ALREADY_USED** is returned if an attempt is made to insert an item into the current linear list of the specified list and the current linear list already contains an item with the specified name. 13

Additional qualifiers:

```
LIST         => list
CURRENT_LIST => current_linear_list_specification
NAME         => identifier
```

**LIST** is the list identified by the LIST_TYPE value specified. 14

**CURRENT_LIST** identifies the current linear list of LIST. 15

**NAME** is the specified item name. 16



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

### 13.3.8.5 SEARCH_ERROR_DETECTED conditions

When the exception indication of an established status indicator is SEARCH_ERROR_DETECTED, one of the following condition codes is returned as the value of the CONDITION status information item. 1

**[1] SEA_NO_SUCH_NAMED_ITEM** is returned if there is no item with the specified name in the current linear list of the specified list. 2

Additional qualifiers:

```
LIST          => list
CURRENT_LIST  => current_linear_list_specification
NAME          => identifier
```

**LIST** is the list identified by the LIST_TYPE value specified. 3

**CURRENT_LIST** identifies the current linear list of LIST. 4

**NAME** is the specified item name. 5

**[2] SEA_NO_SUCH_VALUED_ITEM** is returned if an attempt is made to determine the position of an item in the current linear list by its value and no item with the specified value in the searched region of the current linear list of the specified list. 6

Additional qualifiers:

```
LIST           => list
CURRENT_LIST   => current_linear_list_specification
START_POSITION => integer
END_POSITION   => integer
ITEM_VALUE     => item_value
```

**LIST** is the list identified by the LIST_TYPE value specified. 7

**START_POSITION** and **END-POSITION** indicate the list item positions corresponding to the start and end of the search within the current linear list. 8

**ITEM_VALUE** indicates the list item value sought. 9

**CURRENT-LIST** identifies the current linear list of LIST. 10

### 13.3.8.6 LIST_SYNTAX_ERROR_DETECTED conditions

When the exception indication of an established status indicator is LIST_SYNTAX_ERROR_DETECTED, one of the following condition codes is returned as the value of the CONDITION status information item. 1

**[1] LSY_ILLEGAL_LIST_TEXT** is returned if the specified LIST_TEXT value is syntactically illegal. 2



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Additional qualifier

```
LIST_TEXT        => string
SYNTACTIC_ERROR => syntactic_error_type
```

**LIST_TEXT** is the LIST_TEXT value specified. 3

**SYNTACTIC_ERROR** indicates the nature of the syntactic error. 4

**[2] LSY_ILLEGAL_IDENTIFIED_TEXT** is returned if a specified IDENTIFIER_TEXT value does not conform to the syntax of an Ada identifier. 5

Additional qualifiers:

```
IDENTIFIER_TEXT => string
```

**IDENTIFIER_TEXT** is the IDENTIFIER-TEXT value specified.

### 13.3.8.7 TOKEN_ERROR_DETECTED conditions

When the exception indication of an established status indicator is TOKEN_ERROR_DETECTED, one of the following condition codes is returned as the value of the CONDITION status information item. 1

**[1] TOK_UNDEFINED_TOKEN** is returned if a specified token is undefined.

## 13.3.9 Conditions for CAIS_CALENDAR exceptions

### 13.3.9.1 TIME_ERROR_DETECTED conditions

When the exception indication of an established status indicator is TIME_ERROR_DETECTED, one of the following condition codes is returned as the value of the CONDITION status information item. 1

**[1] TIM_NOT_A_TIME** is returned if the specified parameters do not forma proper time value. 2

Additional qualifiers:

```
YEAR    => integer
MONTH   => integer
DAY     => integer
SECONDS => integer
```

**YEAR** is the year component specified. 3

**MONTH** is the month component specified. 4

**DAY** is the day component specified. 5

**SECONDS** is the DAY_DURATION value specified expressed as a multiple of CAIS_PRAGMATICS.SMALL_FOR_CMS_DURATION. 6

**[2] TIM_INVALID_YEAR** is returned if the year number calculated is not within the proper range. 7



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Additional qualifiers:

YEAR => integer

**YEAR** is the year component calculated that is not within the proper range. 8

**[3] TIM_INVALID_DURATION** is returned if the result of an interval calculation is not in the range of the type CAIS_DURATION. 9

## 13.3.10 Conditions for CAIS_PRAGMATICS exceptions

### 18.3.10.1 CAPACITY_ERROR_DETECTED conditions

When the exception indication of an established status indicator is CAPACITY_ERROR_DETECTED, one of the following condition codes is returned as the value of the CONDITION status information item. 1

**[1] CAP_PRAGMATIC_VIOLATION** is returned when an attempt is made to violate one of the pragmatic limitations defined in package CAIS_PRAGMATICS. 2

Additional qualifiers:

PRAGMATIC => identifier

**PRAGMATIC** is an identifier item indicating the pragmatic limitation which was violated. The identifier returned in this qualifier is the constant identifier defined in package CAIS_PRAGMATICS for the pragmatic limitation violated. For those pragmatic limitations with a constant declared for an implementation-defined limit, the identifier for the implementation-defined limit is returned. Otherwise, the identifier denoting the CAIS-defined limit is returned. 3

**[2] CAP_MAX_FILE_SIZE_EXCEEDED** is returned if an attempt is made to import a host file into a SECONDARY_STORAGE node when the size of the external file is larger than the maximum allowed size for the node's contents as specified by the MAXIMUM_FILE_SIZE attribute on the node. 4

### 13.3.10.2 RESOURCE_ERROR_DETECTED conditions

When the exception indication of an established status indicator is RESOURCE_ERROR_DETECTED, one of the following condition codes is returned as the value of the CONDITION status information item. 1

**[1] RES_RESOURCE_NOT_REASSIGNABLE** is returned if the implementation does not support the reassignment of the specified node to the specified medium. 2

Additional qualifiers:

NODE => unique_identifier
MEDIUM => MEDIUM_unique_identifier

**NODE** identifies the node for which reassignment is desired. 3

**MEDIUM** identifies the MEDIUM node to which NODE was to have been assigned. 4



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

## 13.3.11 Conditions for CAIS_FORM_TERMINAL_IO exceptions

### 13.3.11.1 FORM_STATUS_ERROR_DETECTED conditions

When the exception indication of an established status indicator is FORM_STATUS-ERROR_DETECTED, the following condition code is returned as the value of the CONDITION status information item. 1

**[1] FRM_NONEXISTENT_FORM** is returned if a form does not exist when it should exist. 2

**[2] FRM_FORM_EXISTS** is returned if a form exists when it should not exist. 3

### 13.3.11.2 FORM-FUNCTION_ KEY-STATUS-ERROR_ DETECTED conditions

When the exception indication of an established status indicator is FORM_FUNCTION_KEY_STATUS_ERROR_DETECTED, the following condition code is returned as the value of the CONDITION status information item. 1

**[1] FFK_NONEXISTENT_FUNCTION_KEY** is returned if a non-existent function key is referenced. 2

Additional qualifiers:

CLIENT_NODE => unique_identifier

**CLIENT_NODE** identifies the opposite client node corresponding to the terminal handle specified in the call. 3

### 13.3.11.3 TERMINAL_POSITION_ERROR_DETECTED conditions

When the exception indication of an established status indicator is TERMINAL_POSITION_ ERROR_DETECTED, the following condition code is returned as the value of the CONDITION status information item. 1

**[1] FTM_OFF_FORM** is returned if an operation beyond the boundaries of the form is attempted. 2



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

# 14. GUIDANCE INFORMATION

## 14.1 Keywords

The following list represents the keywords applicable to this standard. These keywords may be used to categorize the concepts presented within this standard and assist in automatic retrieval of appropriate data used in automated document retrieval systems.

Ada
APSE
CAIS
Common APSE Interface Set
computer file system
database management system
KAPSE
high level languages
interfaces
interoperability
operating system
portability
portable tool interfaces
programming support environment
software engineering environment
transportability
virtual operating system


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.
* U.S. GOVERNMENT PRINTING OFFICE: 1990 704-034/30413

METRIC
MIL-STD-1838A
6 April 1989

Superseding
DOD-STD-1838
9 October 1986

# MILITARY STANDARD

## COMMON ADA PROGRAMMING SUPPORT ENVIRONMENT (APSE) INTERFACE SET (CAIS)

(Revision A)

Volume IV

NO DELIVERABLE DATA
REQUIRED BY THIS DOCUMENT

AREA MCCR/IPSC

Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

# CONTENTS


1 SCOPE 1
1.1 Purpose . . . 1
1.2 Application Guidance . . . 2

2 REFERENCED DOCUMENTS 3
2.1 Government Documents . . . 3
2.2 Other Publications . . . 3
2.3 Order of Precedence . . . 4
2.4 Source of Documents . . . 5

3 DEFINITIONS 7

4 GENERAL REQUIREMENTS 33
4.1 Introduction . . . 33
4.2 Method of Description . . . 37
4.2.1 Allowable differences . . . 37
4.2.2 Typographical conventions . . . 37
4.2.3 Semantic descriptions . . . 38
4.3 CAIS Node Model . . . 40
4.3.1 Nodes . . . 40
4.3.2 Relationships . . . 41
4.3.3 Attributes . . . 43
4.3.4 Processes . . . 43
4.3.5 Transactions . . . 44
4.3.6 Basic predefine definitions . . . 44
4.3.7 Names and identification . . . 50
4.3.8 Relationship Identification . . . 51
4.3.9 Pathnames . . . 53
4.3.10 Unique identifiers . . . 54
4.3.11 CAIS definitions . . . 55
4.3.12 Views . . . 59
4.3.13 Attribute monitors . . . 61
4.4 Access Control . . . 62
4.5 Input and Output . . . 64
4.5.1 Protocols . . . 64
4.5.2 Channels, client nodes, and PORT relationships . . . 65
4.5.3 Creating and using channels . . . 67
4.5.4 Compatibility checks . . . 69
4.5.5 Common IO . . . 69
4.6 Resource Control . . . 76
4.6.1 Resource model elements . . . 77
4.6.2 Resource allocatability . . . 80
4.6.3 Resource visibility . . . 80
4.6.4 Transparent resources . . . 82




Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

4.6.5 Resource access control. . . . . 82
4.6.6 Assigning resources . . . . . 82
4.7 Distribution . . . . . 85
4.7. I Remote interprocess communications . . . . . 85

**5 CAIS NODE MANAGEMENT DETAILED REQUIREMENTS** 87

5.1 Package CAIS_DEFINITIONS. . . . . 88
5.2 Package CAIS_NODE_MANAGEMENT . . . . . 95
5.2.1 Open operations . . . . . 95
5.2.2 Accesses without open operations . . . . . 96
5.2.3 Access delay s . . . . . 97
5.2.4 Opening a node handle . . . . . 103
5.2.5 Opening a node handle using a unique identifier . . . . . 106
5.2.6 Closing a node handle . . . . . 107
5.2.7 Changing the intent regarding node handle usage . . . . . 108
5.2.8 Examining the open status of a node handle . . . . . 109
5.2.9 Querying the intention of a node handle . . . . . 109
5.2.10 Querying the kind of a node . . . . . 110
5.2.11 Determining if one node kind is a specialization of another . . . . . 110
5.2.12 Determining if one relation is a specialization of another . . . . . 112
5.2.13 Determining if a node belongs to a node kind . . . . . 113
5.2.14 Determining if a relationship belongs to a relation . . . . . 114
5.2.15 Querying the number of open file handles on a CLIENT node . . . . . 116
5.2.16 Obtaining the unique identifier for a node . . . . . 116
5.2.17 Obtaining a pathname for the unique primary path . . . . . 117
5.2.18 Obtaining the relationship key list of a primary relationship . . . . . 118
5.2.19 Obtaining the relation of a primary relationship . . . . . 118
5.2.20 Obtaining the relationship key list of the last relationship traversed . . . 119
5.2.21 Obtaining the relation of the last relationship traversed . . . . . 120
5.2.22 Obtaining a partial pathname. . . . . 121
5.2.23 Obtaining the name of the last relation in a pathname . . . . . 122
5.2.24 Obtaining the key list of the last relationship in a pathname . . . . . 122
5.2.25 Querying the existence of a node. . . . . 123
5.2.26 Querying sameness . . . . . 124
5.2.27 Obtaining an index for a node handle . . . . . 124
5.2.28 Opening anode handle to the parent . . . . . 125
5.2.29 Copying a node . . . . . 126
5.2.30 Copying trees . . . . . 129
5.2.31 Renaming a relationship . . . . . 134
5.2.32 Deleting the primary relationship to a node . . . . . 138
5.2.33 Deleting the primary relationships of a tree . . . . . 139
5.2.34 Creating secondary relationships . . . . . 141
5.2.35 Creating bidirectional secondary relationships . . . . . 143
5.2.36 Deleting secondary relationships . . . . . 146
5.2.37 Obtaining a corresponding reverse relationship’s relation . . . . . 148
5.2.38 Obtaining a corresponding reverse relationship’s key . . . . . 150
5.2.39 Setting the inheritance property of a relationship . . . . . 151
5.2.40 Determining if a secondary relationship is inheritable . . . . . 153



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

5.2.41 Setting the current node relationship . . . 154
5.2.42 Opening a node handle to the current node . . . 155
5.2.43 Determining the creation time of a node . . . 155
5.2.44 Determining the last time a relationship was modified . . . 156
5.2.45 Determining the last time that node contents were written . . . 157
5.2.46 Determining the last time an attribute was modified . . . 157
5.2.47 Iteration predicate subtype . . . 158
5.2.48 Obtaining the default iteration predicate . . . 168
5.2.49 Node iteration types and subtypes . . . 169
5.2.50 Creating an iterator over nodes.. . . . 169
5.2.51 Determining node iteration status . . . 173
5.2.52 Determining the approximate size of the node iterator . . . 174
5.2.53 Getting the next node in a node iteration . . . 175
5.2.54 Skipping the next node in a node iteration . . . 176
5.2.55 Obtaining the path element for the next node in a node iteration . . . 177
5.2.56 Deleting a node iterator . . . 178
5.2.57 Node kind instance iteration type . . . 178
5.2.58 Creating an iterator over instances of a node kind definition . . . 178
5.2.59 Determining node kind instance iteration status . . . 180
5.2.60 Determining the approximate size of the node kind instance iterator . . . 181
5.2.61 Getting the next node in a node kind instance iteration . . . 181
5.2.62 Skipping the next node in a node kind instance iteration . . . 183
5.2.63 Deleting a node kind instance iterator . . . 184
5.3 Package CAIS_ATTRIBUTE_MANAGEMENT . . . 185
5.3.1 Creating node attributes . . . 186
5.3.2 Creating relationships attributes . . . 187
5.3.3 Deleting node attributes . . . 189
5.3.4 Deleting relationship attributes . . . 190
5.3.5 Setting node attributes . . . 192
5.3.6 Setting relationships attributes . . . 193
5.3.7 Getting node attributes . . . 195
5.3.8 Getting relationship attributes . . . 196
5.3.9 Generic package CAIS_INTEGER_ATTRIBUTE . . . 198
5.3.10 Generic package CAIS_FLOAT_ATTRIBUTE . . . 207
5.3.11 Package CAIS_STRING_ATTRIBUTE . . . 217
5.3.12 Package CAIS_IDENTIFIER_ATTRIBUTE . . . 226
5.3.13 Generic package CAIS_ENUMERATION_ATTRIBUTE . . . 235
5.3.14 Package CAIS_BOOLEAN_ATTRIBUTE . . . 245
5.3.15 Attribute iteration types and subtypes . . . 245
5.3.16 Creating an iterator over node attributes . . . 246
5.3.17 Creating an iterator over relationship attributes . . . 247
5.3.18 Determining iteration status . . . 250
5.3.19 Determining the approximate size of the iterator . . . 250
5.3.20 Getting an identification for the next attribute . . . 251
5.3.21 Getting the next attribute value . . . 252
5.3.22 Skipping the next attribute in an iteration . . . 253
5.3.23 Deleting an attribute iterator . . . 253
5.4 Package CAIS_ACCESS_CONTROL_MANAGEMENT . . . 255



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

5.4.1 Node access . . . 255
5.4.2 Security labeling of CAIS nodes . . . 255
5.4.3 Groups and roles . . . 257
5.4.4 Granting access rights . . . 259
5.4.5 Denying access . . . 262
5.4.6 Access checking . . . 262
5.4.7 An example of CAIS access control . . . 263
5.4,8 Subtypes . . . 271
5.4.9 Value of all access rights . . . 271
5.4.10 Setting access control . . . 271
5.4.11 Deleting ACCESS relationships . . . 273
5.4.12 Obtaining the value of a GRANT attribute . . . 274
5.4.13 Denying access . . . 275
5.4.14 Deleting DENY relationships . . . 275
5.4.15 Examining access rights. . . . 276
5.4.16 Adopting a role . . . 277
5.4.17 Unlinking an adopted rule. . . . 278
5.5 Package CAIS_STRUCTURAL_NODE_MANAGEMENT . . . 280
5.5.1 Opening the predefined STRUCTURAL NKD . . . 280
5.5.2 Creating structural nodes . . . 280
5.6 Package CAIS_TRANSLATION_MANAGEMENT . . . 284
5.6.1 Subtypes . . . 284
5.6.2 Adopting a view . . . 284
5.6.3 Cancelling a view adoption . . . 285
5.6.4 Adopting an SKI . . . 286
5.6.5 Cancelling an NKI adoption . . . 288
5.6.6 Opening a handle to an NKD by local name . . . 289
5.6.7 Opening a handle to an NKI by local name . . . 290
5.6.8 Opening a handle to all RD by local name . . . 291
5.6.9 Opening a handle to an RI by local name . . . 291
5.6.10 Opening a handle to an AD by local name . . . 292
5.6.11 Opening a handle to an AKD by local name . . . 293
5.6.12 Determining the name for a definition . . . 294
5.6.13 Determining the NKD for a node . . . 295
5.6.14 Determining the name for are relation . . . 296
5.6.15 Determining the RD for a relation . . . 297
5.6.16 Determining the name for a node attribute . . . 299
5.6.17 Determining the AD for a node attribute . . . 300
5.6.18 Determining the name for a relationship attribute . . . 301
5.6.19 Determining the AD for a relationship attribute . . . 302
5.7 Package CAIS_DEFINITION_MANAGEMENT . . . 304
5.7.1 Definition nodes. . . . 304
5.7.2 Predefined entities . . . 342
5.7.3 Views . . . 347
5.7.4 Types and subtypes.. . . . 358
5.7.5 Opening the predefine VIEW NKD . . . 359
5.7.6 Opening the predefined NKI NKD. . . . 359
5.7.7 Opening the predefined NKD NKD . . . 360



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

5.7.8 Opening the predefine RI ND . . . . . 360
5.7.9 Opening the predefine RD NKD . . . . . 361
5.7.10 Opening the predefine AD NOD . . . . . 361
5.7.11 Opening the predefine named composite list AKD NKD . . . . . 362
5.7.12 Opening the predefine identifier AKD NKD . . . . . 362
5.7.13 Creating VIEW nodes . . . . . 363
5.7.14 Deleting the primary relationships of a tree rooted at a VIEW node . . . 366
5.7.15 Creating Nods . . . . . 367
5.7.16 Deleting NKIs and Nods . . . . . 372
5.7.17 Creating Skis . . . . . 373
5.7.18 Renaming NODE-KIND relationships in a view . . . . . 377
5.7.19 Specializing a node kind . . . . . 378
5.7.20 Generalizing node kinds . . . . . 380
5.7.21 Deleting SPECIALIZATION-OF relationships from NKDs . . . . . 382
5.7.22 Creating RDs . . . . . 384
5.7.23 Deleting RIs and RDs . . . . . 389
5.7.24 Creating Rib . . . . . 391
5.7.25 Renaming RELATION relationships in a view . . . . . 394
5.7.26 Specializing a relation . . . . . 395
5.7.27 Generalizing relations . . . . . 397
5.7.28 Deleting SPECIALIZATION-OF relationships from RDs . . . . . 400
5.7.29 Creating bidirectional RDs . . . . . 402
5.7.30 Deleting bidirectional RDs . . . . . 410
5.7.31 Creating Ads . . . . . 412
5.7.32 Deleting Ads . . . . . 415
5.7.33 Importing Ads . . . . . 417
5.7.34 Renaming ATTRIBUTE relationships in a view . . . . . 418
5.7.35 Redefining the AKD for an AD. . . . . 419
5.7.36 Creating Adds . . . . . 421
5.7.37 Creating identifier Adds . . . . . 425
5.7.38 Deleting Adds . . . . . 428
5.7.39 Importing Adds . . . . . 430
5.7.40 Renaming ATTRIBUTE-KIND relationships in a view . . . . . 431
5.7.41 Deleting the primary relationships of a tree rooted at a definition node . 432
5.7.42 Specifying source node kinds for a relation . . . . . 434
5.7.43 Deleting source node kinds for a relation . . . . . 436
5.7.44 Respecifying source node kinds for a relation . . . . . 438
5.7.45 Specifying key attributes for a relation . . . . . 439
5.7.46 Deleting key attributes for a relation . . . . . 442
5.7.47 Respecifying key attributes for a relation . . . . . 443
5.7.48 Specifying target node kinds for a relation . . . . . 445
5.7.49 Deleting target node kinds for a relation . . . . . 447
5.7.50 Specifying attributes for node kinds and relations . . . . . 448
5.7.51 Deleting attributes for node kinds and relations . . . . . 450
5.7.52 Respecifying attributes for node kinds and relations . . . . . 452
5.7.53 Specifying an attribute kind component . . . . . 453
5.7.54 Deleting an attribute kind component . . . . . 455
5.7.55 Specifying a variant attribute alternative . . . . . 457



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

5.7.56 Deleting a variant attribute alternative . . . 458
5.7.57 Setting definition node attributes . . . 460
5.7.58 Deleting definition node attributes . . . 462
5.7.59 Setting definition relationship attributes . . . 464
5.7.60 Delete definition relationship attributes . . . 467
5.7.61 Changing the NKD for a node . . . 469
5.7.62 Checking conformance of a node to an NKD . . . 471
5.7.63 Checking conformance of a relationship to an RD . . . 472
5.7.64 Checking conformance of a list to an AD or AKD . . . 475
5.8 Package CAIS_ATTRIBUTE_MONITOR_MANAGEMENT . . . 477
5.8.1 Types . . . 481
5.8.2 Opening the predefine MONITOR NKD . . . 481
5.8.3 Creating MONITOR nodes . . . 482
5.8.4 Deleting the primary relationship to a MONITOR node . . . 489
5.8.5 Associating an event handler process with a MONITOR node . . . 490
5.8.6 Specifying that a node attribute is to be monitored . . . 491
5.8.7 Specifying that a path attribute is to be monitored . . . 494
5.8.8 Canceling monitoring of designated attributes . . . 496
5.8.9 Determining a monitored attribute modification . . . 497
5.8.10 Awaiting a monitored attribute modification . . . 499
5.9 Package CAIS_RESOURCE_CONTROL . . . 503
5.9.1 Reassigning a node to a new medium resource . . . 503
5.9.2 Determining the medium resource to which a node is assigned . . . 504
5.9.3 Determining the processor resource to which a process is assigned . . . 505
5.9.4 Determining the current default medium resource . . . 506
5.9.5 Determining the current default processor resource . . . 506

**6 CAIS PROCESS MANAGEMENT DETAILED REQUIREMENTS** 509
6.1 Transactions . . . 514
6.1.1 Subtransactions and enclosing transactions . . . 514
6.1.2 Opening transactions. . . . 514
6.1.3 Inclusion of nodes. . . . 515
6.1.4 Inclusion of operations. . . . 515
6.1.5 Visibility of effects . . . 516
6.1.6 Closing Transactions . . . 517
6.1.7 Enclosing processes in transactions . . . 518
6.1.8 Eligible effects of operations . . . 518
6.2 Package CAIS_PROCESS_DEFINITIONS . . . 520
6.3 Package CAIS_PROCESS_MANAGEMENT . . . 521
6.3.1 Determining the default priority . . . 524
6.3.2 Creating a dependent PROCESS node . . . 524
6.3.3 Creating a root PROCESS node. . . . 530
6.3.4 Activating a process . . . 535
6.3.5 Awaiting termination of another process . . . 536
6.3.6 Activating a process and awaiting its termination . . . 537
6.3.7 Spawning a new process . . . 538
6.3.8 Invoking a new process . . . 540
6.3.9 Creating and activating a new job . . . 543



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

6.3.10 Deleting a job. . . . . 545
6.3.11 Appending results to. . . . . 546
6.3.12 Overwriting resets . . . . . 547
6.3.13 Getting results from a process . . . . . 547
6.3.14 Determining the status of a process . . . . . 548
6.3.15 Getting the parameter list. . . . . 549
6.3.16 Aborting a process. . . . . 550
6.3.17 Suspending a process. . . . . 551
6.3.18 Resurging a process . . . . . 552
6.3.19 Determining the lowest specifiable priority . . . . . 553
6.3.20 Determining the highest specifiable priority . . . . . 553
6.3.21 Determining if the current priority is modifiable . . . . . 554
6.3.22 Modifying the current priority . . . . . 554
6.3.23 Determining the number of open node handles . . . . . 555
6.3.24 Determining the number of input and output units used . . . . . 556
6.3.25 Determining the time of activation . . . . . 556
6.3.26 Determining the time of termination . . . . . 557
6.3.27 Determining the time a process has been active . . . . . 558
6.3.28 Determining the size of a process . . . . . 558
6.4 Package CAIS_TRANSACTION_MANAGEMENT . . . . . 560
6.4.1 Opening a transaction.. . . . . 560
6.4.2 Including anode in a transaction . . . . . 561
6.4.3 Including a tree in a transaction . . . . . 562
6.4.4 Closing a transaction. . . . . 563
6.4.5 Examining the open status of a transaction . . . . . 564
6.4.6 Querying sameness for two transactions . . . . . 565
6.4.7 Determining whether a node is included in a transaction . . . . . 565

**7 INPUT AND OUTPUT DETAILED REQUIREMENTS** 567
7.1 Package CAIS_IO_DEFINITIONS . . . . . 569
7.2 Package CAIS_FILE_NODE_MANAGEMENT . . . . . 571
7.2.1 Opening the predefine SECONDARY-STORAGE NKD . . . . . 571
7.2.2 Creating secondary storage nodes . . . . . 571
7.3 Package CAIS_IO_CONNECTION. . . . . 575
7.3.1 Channel representation details . . . . . 575
7.3.2 Creating and deleting channels. . . . . 578
7.3.3 Opening protocols . . . . . 578
7.3.4 Compatibility rules . . . . . 578
7.3.5 Detailed protocol opening scenarios . . . . . 582
7.3.6 Discontinuing communication. . . . . 587
7.3.7 Opening the predefined PORT RD.. . . . . 589
7.3.8 Creating a channel . . . . . 589
7.3.9 Releasing a channel . . . . . 593
7.3.10 Deleting a CHANNEL node..... . . . . 594
7.3.11 Waiting for notification of attempted opening . . . . . 595
7.3.12 Opening the opposite client node. . . . . 597
7.3.13 Checking PORT relationship kind compatibility . . . . . 598
7.3.14 Setting PORT relationship attributes . . . . . 599



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

7.4 Package CAIS_IMPORT_EXPORT. . . . . 602
7.4.1 Importing a file from the host file system . . . . . 602
7.4.2 Exporting a file to the host file system . . . . . 603
7.5 Package CAIS_COMMON_IO . . . . . 604
7.5.1 Types and exceptions . . . . . 604
7.5.2 Opening the predefine INTRODUCED_NODE NKD . . . . . 604
7.5.3 Opening the predefine INTRODUCED_NODE_RD . . . . . 605
7.5.4 Opening anode set handle . . . . . 605
7.5.5 Including anode in a node set.. . . . . 606
7.5.6 Including a tree in a node set. . . . . 607
7.5.7 Including a structure of nodes in a node set . . . . . 609
7.5.8 Transmitting a node set in common external form . . . . . 610
7.5.9 Closing a node set . . . . . 611
7.5.10 Introducing nodes in common external form . . . . . 612
7.5.11 Introducing contents of introduced nodes . . . . . 616
7.5.12 Creating VIEW nodes with specific UIDs. . . . . 618
7.5.13 Creating NKDs with specific Aids . . . . . 619
7.5.14 Creating NKIs with specific Aids . . . . . 620
7.5.15 Creating RDs with specific Aids . . . . . 621
7.5.16 Creating RIs with specific Aids . . . . . 622
7.5.17 Creating bidirectional RDs with specific UIDS . . . . . 623
7.5.18 Creating ADs with specific Aids . . . . . 625
7.5.19 Creating AKDs with specific Aids . . . . . 626
7.5.20 Creating structural nodes with specific UIDS . . . . . 627
7.5.21 Creating a secondary storage node with a specific UID . . . . . 628

**8 DATA TRANSFER PROTOCOLS DETAILED REQUIREMENTS** 629
8.1 Input and Output Detailed Requirements . . . . . 629
8.2 Package CAIS_FRAME_IO . . . . . 630
8.2.1 Protocol parameters . . . . . 631
8.2.2 Types. subtypes. and exceptions . . . . . 632
8.2.3 Opening a frame file handle . . . . . 633
8.2.4 Closing a frame file handle . . . . . 634
8.2.5 Determining whether a frame file handle is open . . . . . 635
8.2.6 Determining the mode of a frame file handle . . . . . 635
8.2.7 Transmitting a flame . . . . . 636
8.2.8 Receiving a frame . . . . . 637
8.2.9 Resetting a frame file handle . . . . . 638
8.2.10 Determining if all messages have been read . . . . . 639
8.2.11 Determining if incoming queue is empty . . . . . 640
8.2.12 Determining if outgoing queue is full . . . . . 641
8.2.13 Determining the channel node associated with a file . . . . . 641
8.3 Package CAIS_MAGNETIC_TAPE_IO . . . . . 643
8.3.1 Protocol parameters . . . . . 643
8.3.2 Types, subtypes, and exceptions . . . . . 644
8.3.3 Opening a tape drive file handle.. . . . . 646
8.3.4 Closing a tape drive file handle. . . . . 647
8.3.5 Determining whether a file handle is open . . . . . 648



Source: https://assist.dla.mil -- Downloaded: 202610-26T07:54Z
Check the source to verify that this is the current version before use.

8.3.6 Determining the mode of a magnetic tape drive . . . 649
8.3.7 Requesting the mounting of a tape . . . 649
8.3.8 Loading a gape . . . 650
8.3.9 Unloading a gape . . . 651
8.3.10 Requesting the dismounting of a tape . . . 651
8.3.11 Determining the position of the tape . . . 652
8.3.12 Rewinding the tape.... . . . 652
8.3.13 Skipping tape marks... . . . 653
8.3.14 Writing a tape mark... . . . 654
8.3.15 Determining the status of a magnetic tape drive . . . 654
8.3.16 Determining the recording method of a magnetic tape . . . 655
8.3.17 Determining whether a write ring is installed . . . 655
8.3.18 Skipping blocks in a magnetic tape file . . . 656
8.3.19 Reading a block from a magnetic tape fle . . . 657
8.3.20 Writing a block to a magnetic tape file . . . 658
8.3.21 Resetting a magnetic tape file handle . . . 659
8.3.22 Determining the channel node associated with a file . . . 659
8.4 Package CAIS_DIRECT_IO. . . . 661
8.4.1 Protocol parameters . . . 661
8.4.2 Definition of types . . . 662
8.4.3 Creating a direct file. . . . 662
8.4.4 Opening a direct file handle. . . . 667
8.4.5 Closing a direct file handle . . . 669
8.4.6 Resetting a direct file handle. . . . 669
8.4.7 Synchronizing internal file with FILE node contents . . . 670
8.4.8 Determining the channel node associated with a file . . . 671
8.5 Package CAIS_SEQUENTIAL_IO . . . 672
8.5.1 Protocol parameters.. . . . 672
8.5.2 Definition of types . . . 673
8.5.3 Creating a sequential file. . . . 673
8.5.4 Opening a sequential file handle . . . 678
8.5.5 Closing a sequential file handle. . . . 680
8.5.6 Resetting a sequential file hande. . . . 680
8.5.7 Synchronizing internal file with FILE node contents . . . 682
8.5.8 Determining the channel node associated with a file . . . 682
8.6 Package CAIS_TEXT_IO . . . 684
8.6.1 Protocol parameters . . . 685
8.6.2 Definition of types . . . 685
8.6.3 Creating a text file. . . . 685
8.6.4 Opening a text file handle . . . 690
8.6.5 Closing a text file handle. . . . 692
8.6.6 Resetting a text file handle . . . 693
8.6.7 Synchronizing internal file with FILE node contents . . . 694
8.6.8 Determining the channel node associated with a file . . . 695
8.6.9 Setting the maximum line length of a text file handle . . . 695
8.6.10 Setting the maximum page length of a text file handle . . . 696
8.7 Package CAIS_TERMINAL_IO. . . . 698
8.7.1 Protocol parameters . . . 701



Source: https://assist.dla.mil -- Downloaded: 202610-26T07:54Z
Check the source to verify that this is the current version before use.

8.7.2 Types, Subtypes, Constants and Exceptions . . . . . 701
8.7.3 Opening a terminal file hande . . . . . 704
8.7.4 Closing a terminal file handle. . . . . 706
8.7.5 Determining whether a terminal file handle is open . . . . . 707
8.7.6 Determining the channel node associated with a terminal file handle . . . 707
8.7.7 Determining the physical display origin on the virtual display . . . . . 708
8.7.8 Re-orienting the physical display on the virtual display . . . . . 709
8.7.9 Determining if a terminal supports a set of graphic renditions . . . . . 709
8.7.10 Determining if a terminal supports a foreground color . . . . . 710
8.7.11 Determining if a terminal supports a background color . . . . . 710
8.7.12 Starting output buffering for a virtual display . . . . . 711
8.7.13 Ending output buffering for a virtual display . . . . . 711
8.7.14 Refreshing a virtual display . . . . . 712
8.7.15 Creating a screen . . . . . 712
8.7.16 Changing the active rendition for a screen . . . . . 713
8.7.17 Changing screen size . . . . . 714
8.7.18 Changing border characteristics . . . . . 714
8.7.19 Creating a copy of an existing screen . . . . . 715
8.7.20 Deleting a screen . . . . . 716
8.7.21 Determining the size of a screen . . . . . 716
8.7.22 Determining the active rendition of a screen . . . . . 716
8.7.23 Determining whether a screen has a border . . . . . 717
8.7.24 Determining the border rendition of a screen . . . . . 717
8.7.25 Querying sameness of screens . . . . . 718
8.7.26 Pasting a screen onto the virtual display . . . . . 719
8.7.27 Removing a screen from a virtual display . . . . . 719
8.7.28 Relocating a screen on a virtual display . . . . . 720
8.7.29 Move screen to front . . . . . ...0. 720
8.7.30 Move screen to back . . . . . 721
8.7.31 Determining paste position of a screen . . . . . 721
8.7.32 Determining whether a screen is occluded . . . . . 722
8.7.33 Moving the cursor position on-screen . . . . . 722
8.7.34 Determining the position of a cursor in a screen . . . . . 723
8.7.35 Writing a character to a screen. . . . . 723
8.7.36 Advancing the cursor to the next line . . . . . 724
8.7.37 Writing a line of text to a screen . . . . . 725
8.7.38 Inserting a character into a line. . . . . 725
8.7.39 Deleting characters from a line. . . . . 726
8.7.40 Inserting blank lines into a screen . . . . . 727
8.7.41 Deleting lines from a screen . . . . . 727
8.7.42 Erasing a region of a screen . . . . . 728
8.7.43 Changing the rendition for a region of a screen . . . . . 729
8.7.44 Determining a cell rendition . . . . . 729
8.7.45 Reading characters from a screen. . . . . 730
8.7.46 Sounding the terminal bell. . . . . 731
8.7.47 Starting output buffering for a screen . . . . . 731
8.7.48 Ending output buffering to a screen . . . . . 732
8.7.49 Enabling function key usage... . . . . 732



Source: https://assist.dla.mil -- Downloaded: 202610-26T07:54Z
Check the source to verify that this is the current version before use.

8.7.50 Determining function key usage . . . . . 733
8.7.51 Determining the number of function keys . . . . . 733
8.7.52 Determining the identification of a function key . . . . . 734
8.7.53 Enabling and disabling the locator . . . . . 734
8.7.54 Determining locator usage . . . . . 735
8.7.55 Determining the locator cursor position . . . . . 735
8.7.56 Determining the screen containing the locator cursor . . . . . 736
8.7.57 Determining locator cursor position relative to screen border . . . . . 737
8.7.58 Moving the locator cursor . . . . . 737
8.7.59 Determining the number of locator buttons . . . . . 738
8.7.60 Determining the identification of a locator button . . . . . 738
8.7.61 Reading a single input event... . . . . 739
8.7.62 Reading a string from a virtual keyboard . . . . . 740
8.7.63 Flushing the inpute vent queue. . . . . 742
8.7.64 Determining if all input events have been read . . . . . 742
8.7.65 Determining if the input event queue is empty . . . . . 743

9 CAIS LIST MANAGEMENT DETAILED REQUIREMENTS 745
9.1 Package CAIS_LIST_MANAGEMENT . . . . . 751
9.1.1 Types, subtypes, constants and exceptions . . . . . 751
9.1.2 Copying a list . . . . . 752
9.1.3 Making a list empty . . . . . 753
9.1.4 Converting from text to list form . . . . . 753
9.1.5 Converting a list to its text representation . . . . . 754
9.1.6 Determining the equality of two lists. . . . . 755
9.1.7 Deleting an item from alinear list . . . . . 755
9.1.8 Determining the kind of list. . . . . 756
9.1.9 Determining the kind of fist item. . . . . 757
9.1.10 Inserting a sequence of items into a linear list . . . . . 758
9.1.11 Concatenating two linear lists . . . . . 759
9.1.12 Extracting a sequence of items from a linear list . . . . . 759
9.1.13 Determining the length of a linear list . . . . . 760
9.1.14 Determining the position of the current linear list . . . . . 761
9.1.15 Determining whether the current linear List is outermost . . . . . 761
9.1.16 Making the next outer linear list current . . . . . 761
9.1.17 Making a nested sublist the current linear list . . . . . 762
9.1.18 Determining the length of the text form of a list or a list item . . . . . 763
9.1.19 Determining the name of a named item . . . . . 764
9.1.20 Determining the position of a named item . . . . . 765
9.1.21 Package CAIS_LIST_ITEM . . . . . 766
9.1.22 Package CAIS_IDENTIFIER_ITEM . . . . . 771
9.1.23 Generic package CAIS_ENUMERATION_ITEM . . . . . 780
9.1.24 Package CAIS_BOOLEAN_ITEM . . . . . 786
9.1.25 Generic package CAIS_INTEGER_ITEM . . . . . 787
9.1.26 Generic package CAIS_FLOAT_ITEM . . . . . 792
9.1.27 Package CAIS_STRING_ITEM . . . . . 798



Source: https://assist.dla.mil -- Downloaded: 202610-26T07:54Z
Check the source to verify that this is the current version before use.

**10 CAIS STANDARD DETAILED REQUIREMENTS** 805
10.l Package CAIS_STANDARD. . . . . 805

**11 CAIS CALENDAR DETAILED REQUIREMENTS** 807
ll.l Package CAIS_CALENDAR. . . . . 807
11.1.1 Types, subtypes, and exceptions. . . . . 807
11.1.2 Getting the current time . . . . . 808
11.1.3 Getting the year part of the time . . . . . 808
11.1.4 Getting the month part of the time . . . . . 808
11.1.5 Getting the day part of the time. . . . . 809
11.1.6 Getting the seconds part of the time. . . . . 809
11.1.7 Splitting time into its components . . . . . 810
11.1.8 Combining components of time. . . . . 811
11.1.9 Adding time and duration . . . . . 811
ll.l.l0 Subtracting time and duration . . . . . 812
ll.l.ll Comparing two values of time . . . . . 812

**12 CAIS PRAGMATIC DETAILED REQUIREMENTS** 815
12.1 Package CAIS_PRAGMATUCS . . . . . 815
12.1.1 Identifier and view pragmatics. . . . . 815
12.1.2 Node model pragmatics. . . . . 816
12.1.3 Handle and iterator pragmatics. . . . . 817
12.1.4 Integer and real number pragmatic . . . . . 819
12.1.5 Access control pragmatics . . . . . 819
12.1.6 File and device pragmatics . . . . . 820
12.1.7 List pragmatics . . . . . 822

**13 CAIS STATUS MANAGEMENT DETAILED REQUIREMENTS** 825
13.1 Package CAIS_STATUS_DEFINITIONS . . . . . 845
13.2 Package CAIS_STATUS_MANAGEMENT . . . . . 857
13.2.1 Creating a status indicator . . . . . 857
13.2.2 Setting a status indicator . . . . . 857
13.2.3 Copying a status indicator . . . . . 858
13.2.4 Deleting a status indicator . . . . . 858
13.2.5 Obtaining a status indicator's exception indication . . . . . 859
13.2.6 Obtaining status information from a status indicator . . . . . 859
13.2.7 Obtaining a descriptive message from a status indicator . . . . . 860
13.2.8 Determining if a status indicator is established . . . . . 860
13.3 Exception Conditions . . . . . 862
13.3.1 Conditions for CAIS_DEFINITIONS exceptions . . . . . 862
13.3.2 Conditions for CAIS_PROCESS.DEFINITIONS exceptions . . . . . 908
13.3.3 Conditions for CAIS_IO_DEFINITIONS exceptions . . . . . 908
13.3.4 Conditions for CAIS_FRAME_IO exceptions . . . . . 914
13.3.5 Conditions for CAIS_MAGNETIC_TAPE_IO exceptions . . . . . 915
13.3.6 Conditions for CAIS_TERMINAL_IO exceptions . . . . . 915
13.3.7 Conditions for CAIS_COMMON_IO exceptions . . . . . 918
13.3.8 Conditions for CAIS_LIST_MANAGEMENT exceptions . . . . . 918
13.3.9 Conditions for CAIS_CALENDAR exceptions . . . . . 924
13.3.10 Conditions for CAIS_PRAGMATICS exceptions . . . . . 925




Source: https://assist.dla.mil -- Downloaded: 202610-26T07:54Z
Check the source to verify that this is the current version before use.

13.3.11 Conditions for CAIS_FORM_TERMINAL_IO exceptions . . . . . 926

**14 GUIDANCE INFORMATION** 927

14.1 Keywords . . . . . 927



Source: https://assist.dla.mil -- Downloaded: 202610-26T07:54Z
Check the source to verify that this is the current version before use.

Source: https://assist.dla.mil -- Downloaded: 202610-26T07:54Z
Check the source to verify that this is the current version before use.

# APPENDIXES


## Appendix A Predefined Entities 929

10 Predefined node description index . . . . . 929
20 Predefied Node Descriptions . . . . . 940
20.1 Top-level nodes . . . . . 943
20.2 Attribute definition nodes (Ads).... . . . . 960
20.3 Attribute kind definition nodes (AKDs) . . . . . 1000
20.4 Node kind definition nodes (NKDs) . . . . . 1027
20.5 Relation definition nodes (RDs) . . . . . 1050
20.6 Node kind interpretation nodes (NKIs) . . . . . 1114
20.7 Relation interpretation nodes (RIs) . . . . . 1130

## B CAIS Specification 1167

## C Compatibility with Pre-existing Standards 1321

10 Package CAIS_NODE_MANAGEMENT . . . . . 1321
10.1 Opening a node handle . . . . . 1321
10.2 Changing the intent regarding node handle usage . . . . . 1322
10.3 Querying the existence of a node. . . . . 1322
10.4 Querying sameness . . . . . 1322
10.5 Obtaining an open node handle to the parent . . . . . 1323
10.6 Copying a node. . . . . 1323
10.7 Copying trees . . . . . 1324
10.8 Renaming the primary relationship of a node . . . . . 1324
10.9 Deleting the primary relationship to a node . . . . . 1324
10.10 Deleting the primary relationships of a tree . . . . . 1325
10.11 Creating secondary relationships . . . . . 1325
10.12 Deleting secondary relationships . . . . . 1326
10.13 Setting inheritance property of a relationship . . . . . 1326
10.14 Determining if a secondary relationship is in-heritable . . . . . 1326
10.15 Setting the current node relationship . . . . . 1327
10.16 Opening a node handle to the current node . . . . . 1327
10.17 Determining the creation time of anode. . . . . 1327
10.18 Determining the last time a relationship was modified . . . . . 1328
10.19 Determining the last time that node contents were written . . . . . 1328
10.20 Determining the last time an attribute was modified . . . . . 1328
10.21 Creating an iterator over nodes.. . . . . 1329
10.22 Getting the next node in a node iteration . . . . . 1329
20 Package CAIS_ATTRIBUTE_MANAGEMENT . . . . . 1330
20.1 Creating node attributes . . . . . 1330
20.2 Creating path attributes . . . . . 1330
20.3 Deleting node attributes . . . . . 1330
20.4 Deleting path attributes . . . . . 1331
20.5 Setting node attributes . . . . . 1331
20.6 Setting path attributes . . . . . 1331
20.7 Getting node attributes . . . . . 1332




Source: https://assist.dla.mil -- Downloaded: 202610-26T07:54Z
Check the source to verify that this is the current version before use.

20.8 Getting path attributes . . . . . 1332
20.9 Creating an iterator over node attributes . . . . . 1333
20.10 Creating an iterator over relationship attributes . . . . . 1333
30 Package CAIS_ACCESS_CONTROL_MANAGEMENT . . . . . 1334
30.1 Subtypes . . . . . 1334
30.2 Setting access control . . . . . 1334
30.3 Deleting access relationships . . . . . 1334
30.4 Obtaining the value of a GRANT attribute . . . . . 1335
30.5 Examining access rights . . . . . 1335
30.6 Adopting a rose . . . . . 1335
40 Package CAIS_STRUCTURAL_NODE_MANAGEMENT . . . . . 1336
40.1 Creating structural nodes . . . . . 1336
50 Package CAIS_PROCESS_MANAGEMENT . . . . . 1337
50.1 Spawning anew process . . . . . 1337
50.2 Invoking a new process . . . . . 1338
50.3 Awaiting termination of another process . . . . . 1340
50.4 Deleting a job . . . . . 1340
50.5 Getting results from a process . . . . . 1340
50.6 Determining the status of a process . . . . . 1341
50.7 Aborting a process . . . . . 1341
50.8 Suspending a process. . . . . 1342
50.9 Resuming a process . . . . . 1342
50.10 Determining the number of open node handles . . . . . 1343
50.11 Determining the number of input and output units used . . . . . 1343
50.12 Determining the time of activation . . . . . 1344
50.13 Determining the time of termination . . . . . 1344
50.14 Determining the time a process has been active . . . . . 1344
50.15 Determining the size of a process . . . . . 1345
60 Package CAIS_IMPORT_EXPORT. . . . . 1345
60.1 Importing a file from the host file system . . . . . 1345
60.2 Exporting a file to the host file systern . . . . . 1345
70 Package CAIS_FORM_TERMINAL_IO . . . . . 1346
70.1 Protocol parameters . . . . . 1347
70.2 Types, subtypes, and exceptions . . . . . 1347
70.3 Opening a form terminal file handle . . . . . 1348
70.4 Closing a form terminal file handle . . . . . 1350
70.5 Determining whether a file handle is open . . . . . 1350
70.6 Determining the channel node associated with a file . . . . . 1351
70.7 Determining the number of function keys . . . . . 1351
70.8 Determining intercepted input characters . . . . . 1352
70.9 Determining intercepted output characters . . . . . 1352
70.10 Creating a form . . . . . 1353
70.11 Deleting a form . . . . . 1354
70.12 Copying a form . . . . . 1354
70.13 Defining a qualified area. . . . . 1355
70.14 Removing an area qualifier . . . . . 1355
70.15 Changing the active position . . . . . 1356
70.16 Querying the active position . . . . . 1357



Source: https://assist.dla.mil -- Downloaded: 202610-26T07:54Z
Check the source to verify that this is the current version before use.

70.17 Advancing forward to qualified area . . . 1357
70.18 Writing to a form . . . 1358
70.19 Erasing a qualified area . . . 1359
70.20 Erasing a form . . . 1359
70.21 Activating a form on a terminal . . . 1360
70.22 Reading from a form . . . 1360
70.23 Determining changes to a form. . . . 1361
70.24 Determining the identification of a function key . . . 1362
70.25 Determining the termination key. . . . 1362
70.26 Determining the size of a forrn. . . . 1363
70.27 Determining the size of the terminal . . . 1363
70.28 Determining if the area qualifier requires space in the form . . . 1364
70.29 Determining if the area qualifier requires space on a terminal . . . 1364
80 Package CAIS_PRAGMATICS . . . 1365

**D Syntax Summary** 1367
l0 Pathname Syntax . . . ...1368
20 GRANT Attribute Value Syntax . . . 1368
30 Classification Attribute Value Syntax . . . 1368
40 List External Representation Syntax . . . 1369
50 Iteration Predicate Syntax . . . 1370
60 External Node Set Syntax. . . . ...1371
70 ATTRIBUTES_MONITORED Attribute Value Syntax . . . 1372
80 Constraints Attribute Value Syntax . . . 1373
90 Status Information List Syntax.. . . . ...1374

**E Discretionary Access Control Conformance Guidelines** 1381
10 Class C1 . . . 1381
10.1 Capability a: user control of Objects . . . 1381
10.2 Capability b: user ability to specify access modes . . . 1381
10.3 Capability c: user ability to specify individuals or groups . . . 1382
20 Class C2 . . . 1382
20.1 Capability a: user ability to control objects . . . 1383
20.2 Capability b: user ability to specify and control access modes . . . 1383
20.3 Capability c: user ability to specify individual users or groups of individual users . . . 1383
20.4 Capability d: user ability to include or exclude object access to the granularity of an individual . . . 1383
20.5 Capability e: assignment of access control by an authorized user . . . 1384
30 Class B3 . . . 1384
30.1 Capability a: user ability to control objects . . . 1385
30.2 Capability b: user ability to specify and control access modes . . . 1385
30.3 Capability c: access control list . . . 1385
30.4 Capability d: user ability to include or exclude object access to the granularity of an individual . . . 1385
30.5 Capability e: access permission to an object must be assigned by an authorized user . . . 1385
30.6 Capability f: ability to deny access to individuals or groups . . . 1385



Source: https://assist.dla.mil -- Downloaded: 202610-26T07:54Z
Check the source to verify that this is the current version before use.

**F Implementation Dependencies** 1387

**G List Of Interfaces** 1389

**Index** 1407



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

# FIGURES


Figure 1 CAIS system concept . . . . . 45
2 Tree formed by primary relationships . . . . . 47
3 Some predefined relations . . . . . 49
4 Example node kind definition and attribute definition . . . . . 56
5 primary relationships to definition nodes . . . . . 57
6 VIEW node and emanating relationships . . . . . 60
7 Channels . . . . . 65
8 Multiple channels . . . . . 66
9 Channel state transition . . . . . 68
10 External node set . . . . . 72
11 Resource nodes example . . . . . 78
12 Resource node groups example . . . . . 79
13 Allocatabitity relationship example . . . . . 81
14 Resource visibility . . . . . 83
15 Remote interprocess communication . . . . . 86
16 COPY_TREE example . . . . . 131
17 Discretionary access control . . . . . 264
18 Discretionary access control after process creation . . . . . 265
19 ACCESS relationships, Example l . . . . . 267
20 ACCESS relationships, Example 2 . . . . . 269
21 ACCESS relationships, Example 3 . . . . . 270
22 Inheritance of properties and constraints . . . . . 305
23 Example of an NKD lattice . . . . . 307
24 Node kind definition node (NKD). . . . . 308
25 NKD and RD property representation . . . . . 309
26 Relation definition node (RD) . . . . . 311
27 Bidirectional RD . . . . . 312
28 Basic NKD and RD . . . . . 314
29 Attribute definition node (AD) and attribute kind definition node (AKD) . . . . . 315
30 Simple and variant Adds . . . . . 321
31 Example of a variant AKD . . . . . 323
32 List, uniform list and composite list AKDs . . . . . 325
33 GRANT attribute kind representation . . . . . 326
34 TIME attribute kind representation . . . . . 327
35 Node kind interpretation node (NKI) . . . . . 347
36 Local names defined by an NKI . . . . . 348
37 Relation interpretation node (RI) . . . . . 349
38 Local names defined by an RI.... . . . . 350
39 Adoption of views by a root process . . . . . 352
40 Adoption of views by a dependent process . . . . . 353
41 Allowable process status transitions . . . . . 511
42 Full channel representation . . . . . 575
43 Frame IO message queues . . . . . 630
44 Magnetic tape status transit ions . . . . . 645




Source: https://assist.dla.mil -- Downloaded: 202610-26T07:54Z
Check the source to verify that this is the current version before use.

45 Virtual display with screens . . . . . 699



Source: https://assist.dla.mil -- Downloaded: 202610-26T07:54Z
Check the source to verify that this is the current version before use.

# TABLES


| | | |
|---|---|---|
| **Table** I | Pathname BNF | 52 |
| II | External Node Set BNF | 73 |
| III | Resource Model Elements.. | 77 |
| IV | Intents. | 98 |
| V | Matrix of Access Synchronization Constraints | 102 |
| VI | Iteration Predicate BNF | 160 |
| VII | Logical Operators in Iteration Predicates | 165 |
| VIII | Applications of Attribute Functions | 165 |
| IX | Classification Attribute Value BNF | 256 |
| X | Predefined access rights | 260 |
| XI | GRANT Attribute Value BNF. | 262 |
| XII | Predefined AKD Attributes. | 319 |
| XIII | Predefined Emanating Relations for AKDs | 320 |
| XIV | Predefined Attributes of Relations Emanating from AKDs. | 320 |
| XV | Matrix of Composite List AKDs | 324 |
| XVI | Predefine Relations that allow the CONSTRAINTS Attribute | 329 |
| XVII | Constraints Attribute Value BNF. | 330 |
| XVIII | Applicable Constraint Items. | 331 |
| XIX | Inherited Properties and Constraints | 336 |
| XX | Predefined Definition Node Attributes | 340 |
| XXI | Predefine Definition Relationship Attributes | 341 |
| XXII | ATTRIBUTES_MONITORED Attribute Value BNF | 478 |
| XXIII | Relationships Created as a Result of CREATE--MONITOR-NODE | 480 |
| XXIV | MONITOR Node Attributes Created by CREATE-MONITOR. NODE | 481 |
| XXV | Events Causing Process Status Transition | 512 |
| XXVI | Relationships Created for Root PROCESS Nodes | 522 |
| XXVII | Relationships Inherited or Created for Dependent PROCESS Nodes | 523 |
| XXVIII | DIRECTION Attribute Values.. | 576 |
| XXIX | Channel State Transition | 577 |
| XXX | DIRECTION Compatibility | 580 |
| XXXI | Intent-DIRECTION Compatibility | 580 |
| XXXII | Mode-DIRECTION Compatibility. | 581 |
| XXXIII | Intents for Protocol Opening | 581 |
| XXXIV | Mode Compatibility | 581 |
| XXXV | Detailed Protocol Opening Scenario | 583 |
| XXXVI | Allowed Magnetic Tape Characters | 643 |
| XXXVII | List External Representation BNF. | 749 |
| XXXVIII | Status Information List BNF | 827 |
| XXXIX | Status information List Description | 834 |
| XL | Index To Predefined Node Descriptions | 929 |
| XLI | Predefined Node Description Fields | 941 |




Source: https://assist.dla.mil -- Downloaded: 202610-26T07:54Z
Check the source to verify that this is the current version before use.

Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

# A. Predefine Entities

The material contained in this appendix is a mandatory part of the standard.

This appendix describes the predefined nodes, relationships and attributes which form the basis of a CAIS instance.

## 10 Predefine node description index

Table XL contains an index to the descriptions of the predefined nodes in Section 20. Each entry in this table gives the suffix of the UID of the predefined node, a pathname for the unique primary path to the node (formed using relation names and key names from the predefine view) and the number of the page on which the description of the node begins.

TABLE XL. Index To Predefined Node Descriptions

| UID | Path From System-Level Node | Page |
|---|---|---|
| N_1 | *system-level node* | 943 |
| N_2 | 'DEFINITIONS( PREDEFINED ) | 943 |
| N_3 | 'MEDIUM( TRANSPARENT_MEDIUM_RESOURCE ) | 951 |
| N_4 | 'PROCESSOR( TRANSPARENT_PROCESSOR_RESOURCE ) | 951 |
| N_5 | 'RESOURCE_GROUP( TRANSPARENT_RESOURCES ) | 951 |
| N_6 | 'VIEW( PREDEFINED ) | 951 |
| **UID** | **Path From 'DEFINITIONS( PREDEFINED )** | **Page** |
| AD_1 | 'ATTRIBUTE_DEFINITION( ALLOWED_PROTOCOLS ) | 960 |
| AD_2 | 'ATTRIBUTE_DEFINITION( ALLOWED_USAGE ) | 960 |
| AD_3 | 'ATTRIBUTE_DEFINITION( ANNOTATION ) | 961 |
| AD_4 | 'ATTRIBUTE_DEFINITION( ATTRIBUTE_VALUE ) | 961 |
| AD_5 | 'ATTRIBUTE_DEFINITION( ATTRIBUTES_MONITORED ) | 961 |
| AD_6 | 'ATTRIBUTE_DEFINITION( AVAILABLE ) | 962 |
| AD_7 | 'ATTRIBUTE_DEFINITION( CAIS_CONTROLLED ) | 962 |
| AD_8 | 'ATTRIBUTE_DEFINITION( CAIS_DETERMINED ) | 962 |
| AD_9 | 'ATTRIBUTE_DEFINITION( CAIS_KEY ) | 963 |
| AD_10 | 'ATTRIBUTE_DEFINITION( CONSTRAINTS ) | 963 |
| AD_11 | 'ATTRIBUTE_DEFINITION( CONTROLLING_PROTOCOL ) | 964 |
| AD_12 | 'ATTRIBUTE_DEFINITION( CURRENT_FILE_SIZE ) | 964 |
| AD_13 | 'ATTRIBUTE_DEFINITION( CURRENT_STATUS ) | 964 |
| AD_14 | 'ATTRIBUTE_DEFINITION( DEFAULT_VALUE ) | 965 |
| AD_15 | 'ATTRIBUTE_DEFINITION( DEFINITION_KIND ) | 965 |
| AD_16 | 'ATTRIBUTE_DEFINITION( DELETABLE ) | 966 |
| AD_17 | 'ATTRIBUTE_DEFINITION( DEVICE_KIND ) | 966 |
| AD_18 | 'ATTRIBUTE_DEFINITION( DIGITS ) | 967 |
| AD_19 | 'ATTRIBUTE_DEFINITION( DIRECTION ) | 967 |
| AD_20 | 'ATTRIBUTE_DEFINITION( ELEMENT_SIZE ) | 967 |
| AD_21 | 'ATTRIBUTE_DEFINITION( ENUMERATED ) | 968 |
| AD_22 | 'ATTRIBUTE_DEFINITION( FILE_KIND ) | 968 |
| AD_23 | 'ATTRIBUTE_DEFINITION( GENERIC_KEY ) | 969 |



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

| UID | Path From 'DEFINITIONS( PREDEFINED ) | Page |
|---|---|---|
| AD_24 | 'ATTRIBUTE_DEFINITION( GRANT ) | 970 |
| AD_25 | 'ATTRIBUTE_DEFINITION( HANDLER_AS_A_TRANSACTION ) | 970 |
| AD_26 | 'ATTRIBUTE_DEFINITION( HANDLER_DELETE_WHEN_TERMINATED ) | 971 |
| AD_27 | 'ATTRIBUTE_DEFINITION( HANDLER_DISCRETIONARY_ACCESS ) | 971 |
| AD_28 | 'ATTRIBUTE_DEFINITION( HANDLER_INPUT_PARAMETERS ) | 971 |
| AD_29 | 'ATTRIBUTE_DEFINITION( HANDLER_KEY ) | 972 |
| AD_30 | 'ATTRIBUTE_DEFINITION( HANDLER_MANDATORY_ACCESS ) | 972 |
| AD_31 | 'ATTRIBUTE_DEFINITION( HANDLER_PRIORITY ) | 972 |
| AD_32 | 'ATTRIBUTE_DEFINITION( HIGHEST_CLASSIFICATION ) | 973 |
| AD_33 | 'ATTRIBUTE_DEFINITION( IMPORTED_DEFINITION_UID ) | 973 |
| AD_34 | 'ATTRIBUTE_DEFINITION( IMPORTED_NAME ) | 974 |
| AD_35 | 'ATTRIBUTE_DEFINITION( IMPORTED_RELATIONSHIP_KIND ) | 974 |
| AD_36 | 'ATTRIBUTE_DEFINITION( IMPORTED_TARGET_UID ) | 974 |
| AD_37 | 'ATTRIBUTE_DEFINITION( IMPORTED_UID ) | 975 |
| AD_38 | 'ATTRIBUTE_DEFINITION( INCOMING_FRAME_LENGTH ) | 975 |
| AD_39 | 'ATTRIBUTE_DEFINITION( INHERITABLE ) | 975 |
| AD_40 | 'ATTRIBUTE_DEFINITION( INTERPRETATION_KIND ) | 978 |
| AD_41 | 'ATTRIBUTE_DEFINITION( INTRODUCED_KEY_ATTRIBUTES ) | 978 |
| AD_42 | 'ATTRIBUTE_DEFINITION( INTRODUCED_NODE_ATTRIBUTES ) | 979 |
| AD_43 | 'ATTRIBUTE_DEFINITION( INTRODUCED_RELATIONSHIP_ATTRIBUTES ) | 979 |
| AD_44 | 'ATTRIBUTE_DEFINITION( IO_UNIT_COUNT ) | 979 |
| AD_45 | 'ATTRIBUTE_DEFINITION( KIND ) | 980 |
| AD_46 | 'ATTRIBUTE_DEFINITION( LOWER_BOUND_FLOAT ) | 980 |
| AD_47 | 'ATTRIBUTE_DEFINITION( LOWER_BOUND_INTEGER ) | 980 |
| AD_48 | 'ATTRIBUTE_DEFINITION( LOWEST_CLASSIFICATION ) | 981 |
| AD_49 | 'ATTRIBUTE_DEFINITION( MACHINE_TIME ) | 981 |
| AD_50 | 'ATTRIBUTE_DEFINITION( MAXIMUM_FILE_SIZE ) | 982 |
| AD_51 | 'ATTRIBUTE_DEFINITION( MAXIMUM_INSTANCES ) | 982 |
| AD_52 | 'ATTRIBUTE_DEFINITION( MAXIMUM_LINE_LENGTH ) | 982 |
| AD_53 | 'ATTRIBUTE_DEFINITION( MAXIMUM_PAGE_LENGTH ) | 983 |
| AD_54 | 'ATTRIBUTE_DEFINITION( MAXIMUM_SIZE ) | 983 |
| AD_55 | 'ATTRIBUTE_DEFINITION( MINIMUM_INSTANCES ) | 984 |
| AD_56 | 'ATTRIBUTE_DEFINITION( MINIMUM_SIZE ) | 984 |
| AD_57 | 'ATTRIBUTE_DEFINITION( MODIFIABLE ) | 984 |
| AD_58 | 'ATTRIBUTE_DEFINITION( NAME_KEY ) | 985 |
| AD_59 | 'ATTRIBUTE_DEFINITION( NAMING ) | 985 |
| AD_60 | 'ATTRIBUTE_DEFINITION( NODE_KIND ) | 986 |
| AD_61 | 'ATTRIBUTE_DEFINITION( OBJECT_CLASSIFICATION ) | 988 |
| AD_62 | 'ATTRIBUTE_DEFINITION( OPEN_NODE_HANDLE_COUNT ) | 989 |
| AD_63 | 'ATTRIBUTE_DEFINITION( OPTIONAL ) | 989 |
| AD_64 | 'ATTRIBUTE_DEFINITION( ORDERED ) | 990 |
| AD_65 | 'ATTRIBUTE_DEFINITION( OUTGOING_FRAME_LENGTH_RANGE ) | 990 |
| AD_66 | 'ATTRIBUTE_DEFINITION( PARAMETERS ) | 990 |
| AD_67 | 'ATTRIBUTE_DEFINITION( PATTERNS ) | 991 |
| AD_68 | 'ATTRIBUTE_DEFINITION( POSITION_KEY ) | 991 |
| AD_69 | 'ATTRIBUTE_DEFINITION( PRIORITY ) | 992 |
| AD_70 | 'ATTRIBUTE_DEFINITION( PRIORITY_LEVEL ) | 992 |
| AD_71 | 'ATTRIBUTE_DEFINITION( PROCESS_SIZE ) | 992 |
| AD_72 | 'ATTRIBUTE_DEFINITION( QUEUE_LENGTH ) | 993 |
| AD_73 | 'ATTRIBUTE_DEFINITION( RESULTS ) | 993 |




Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

| UID | Path From 'DEFINITIONS( PREDEFINED ) | Page |
|---|---|---|
| AD_74 | 'ATTRIBUTE_DEFINITION( SUBJECT_CLASSIFICATION ) | 993 |
| AD_75 | 'ATTRIBUTE_DEFINITION( STATE ) | 994 |
| AD_76 | 'ATTRIBUTE_DEFINITION( STRUCTURAL_KIND ) | 994 |
| AD_77 | 'ATTRIBUTE_DEFINITION( TIME_ATTRIBUTE_WRITTEN ) | 995 |
| AD_78 | 'ATTRIBUTE_DEFINITION( TIME_CONTENTS_WRITTEN ) | 995 |
| AD_79 | 'ATTRIBUTE_DEFINITION( TIME_CREATED ) | 996 |
| AD_80 | 'ATTRIBUTE_DEFINITION( TIME_FINISHED ) | 996 |
| AD_81 | 'ATTRIBUTE_DEFINITION( TIME_RELATIONSHIP_WRITTEN ) | 996 |
| AD_82 | 'ATTRIBUTE_DEFINITION( TIME_STARTED ) | 997 |
| AD_83 | 'ATTRIBUTE_DEFINITION( UNIQUE_IDENTIFIER ) | 997 |
| AD_84 | 'ATTRIBUTE_DEFINITION( UNIQUE_KEYS ) | 997 |
| AD_85 | 'ATTRIBUTE_DEFINITION( UPPER_BOUND_FLOAT ) | 998 |
| AD_86 | 'ATTRIBUTE_DEFINITION( UPPER_BOUND_INTEGER ) | 998 |
| AD_87 | 'ATTRIBUTE_DEFINITION( VALUE_TYPE ) | 999 |
| AKD_1 | 'ATTRIBUTE_KIND_DEFINITION( ALTERNATIVE_CONSTRAINT ) | 1000 |
| AKD_2 | 'ATTRIBUTE_KIND_DEFINITION( ANNOTATION_KINDS ) | 1000 |
| AKD_3 | 'ATTRIBUTE_KIND_DEFINITION( ATTRIBUTE_DESIGNATOR ) | 1001 |
| AKD_4 | 'ATTRIBUTE_KIND_DEFINITION( ATTRIBUTES_MONITORED ) | 1001 |
| AKD_5 | 'ATTRIBUTE_KIND_DEFINITION( BOOLEAN ) | 1001 |
| AKD_6 | 'ATTRIBUTE_KIND_DEFINITION( CAIS_FLOAT ) | 1002 |
| AKD_7 | 'ATTRIBUTE_KIND_DEFINITION( CAIS_INTEGER ) | 1002 |
| AKD_8 | 'ATTRIBUTE_KIND_DEFINITION( CAIS_NATURAL ) | 1003 |
| AKD_9 | 'ATTRIBUTE_KIND_DEFINITION( CAIS_POSITIVE ) | 1003 |
| AKD_10 | 'ATTRIBUTE_KIND_DEFINITION( CHANNEL_STATE ) | 1003 |
| AKD_11 | 'ATTRIBUTE_KIND_DEFINITION( CLASSIFICATION ) | 1004 |
| AKD_12 | 'ATTRIBUTE_KIND_DEFINITION( COMPONENT_CONSTRAINT ) | 1004 |
| AKD_13 | 'ATTRIBUTE_KIND_DEFINITION( COMPOSITE_LIST_COMPONENT_-CONSTRAINT ) | 1005 |
| AKD_14 | 'ATTRIBUTE_KIND_DEFINITION( CONSTRAINTS_LIST ) | 1005 |
| AKD_15 | 'ATTRIBUTE_KIND_DEFINITION( DAY ) | 1006 |
| AKD_16 | 'ATTRIBUTE_KIND_DEFINITION( DEFINITION_KIND ) | 1006 |
| AKD_17 | 'ATTRIBUTE_KIND_DEFINITION( DEVICE_KIND ) | 1006 |
| AKD_18 | 'ATTRIBUTE_KIND_DEFINITION( DIGITS ) | 1007 |
| AKD_19 | 'ATTRIBUTE_KIND_DEFINITION( DIRECTION ) | 1007 |
| AKD_20 | 'ATTRIBUTE_KIND_DEFINITION( ELEMENT_SIZE ) | 1007 |
| AKD_21 | 'ATTRIBUTE_KIND_DEFINITION( EMANATES_BOUND ) | 1008 |
| AKD_22 | 'ATTRIBUTE_KIND_DEFINITION( FILE_KIND ) | 1008 |
| AKD_23 | 'ATTRIBUTE_KIND_DEFINITION( FILE_SIZE ) | 1009 |
| AKD_24 | 'ATTRIBUTE_KIND_DEFINITION( FRAME_IO_PRIORITY_LEVEL ) | 1009 |
| AKD_25 | 'ATTRIBUTE_KIND_DEFINITION( FRAME_LENGTH ) | 1009 |
| AKD_26 | 'ATTRIBUTE_KIND_DEFINITION( GRANT ) | 1010 |
| AKD_27 | 'ATTRIBUTE_KIND_DEFINITION( GRANT_ITEM ) | 1010 |
| AKD_28 | 'ATTRIBUTE_KIND_DEFINITION( HIERARCHICAL_CLASSIFICATION ) | 1010 |
| AKD_29 | 'ATTRIBUTE_KIND_DEFINITION( IDENTIFIER_LIST ) | 1011 |
| AKD_30 | 'ATTRIBUTE_KIND_DEFINITION( INTERPRETATION_KIND ) | 1011 |
| AKD_31 | 'ATTRIBUTE_KIND_DEFINITION( INTRODUCED_ATTRIBUTE ) | 1011 |
| AKD_32 | 'ATTRIBUTE_KIND_DEFINITION( INTRODUCED_ATTRIBUTE_-DESCRIPTION ) | 1012 |
| AKD_33 | 'ATTRIBUTE_KIND_DEFINITION( ITEM_VALUE ) | 1012 |
| AKD_34 | 'ATTRIBUTE_KIND_DEFINITION( KEY_VALUE_DESIGNATOR ) | 1013 |



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

| UID | Path From 'DEFINITIONS( PREDEFINED ) | Page |
|---|---|---|
| AKD_35 | 'ATTRIBUTE_KIND_DEFINITION( KEY_VALUE_LIST ) | 1013 |
| AKD_36 | 'ATTRIBUTE_KIND_DEFINITION( KIND ) | 1013 |
| AKD_37 | 'ATTRIBUTE_KIND_DEFINITION( LIST_SIZE ) | 1014 |
| AKD_38 | 'ATTRIBUTE_KIND_DEFINITION( MAXIMUM_LINE_LENGTH ) | 1014 |
| AKD_39 | 'ATTRIBUTE_KIND_DEFINITION( MAXIMUM_PAGE_LENGTH ) | 1015 |
| AKD_40 | 'ATTRIBUTE_KIND_DEFINITION( MONTH ) | 1015 |
| AKD_41 | 'ATTRIBUTE_KIND_DEFINITION( NAMING ) | 1015 |
| AKD_42 | 'ATTRIBUTE_KIND_DEFINITION( NON_HIERARCHICAL_CATEGORY ) | 1016 |
| AKD_43 | 'ATTRIBUTE_KIND_DEFINITION( NON_HIERARCHICAL_CATEGORY_LIST ) | 1016 |
| AKD_44 | 'ATTRIBUTE_KIND_DEFINITION( OPEN_NODE_HANDLE_COUNT ) | 1016 |
| AKD_45 | 'ATTRIBUTE_KIND_DEFINITION( OUTGOING_FRAME_LENGTH_RANGE ) | 1017 |
| AKD_46 | 'ATTRIBUTE_KIND_DEFINITION( PARAMETERS_KINDS ) | 1017 |
| AKD_47 | 'ATTRIBUTE_KIND_DEFINITION( PATTERN_LIST ) | 1018 |
| AKD_48 | 'ATTRIBUTE_KIND_DEFINITION( PROCESS_PRIORITY ) | 1018 |
| AKD_49 | 'ATTRIBUTE_KIND_DEFINITION( PROCESS_STATUS_KIND ) | 1018 |
| AKD_50 | 'ATTRIBUTE_KIND_DEFINITION( PROTOCOL ) | 1019 |
| AKD_51 | 'ATTRIBUTE_KIND_DEFINITION( PROTOCOL_LIST ) | 1019 |
| AKD_52 | 'ATTRIBUTE_KIND_DEFINITION( RANGE_CONSTRAINT ) | 1019 |
| AKD_53 | 'ATTRIBUTE_KIND_DEFINITION( RELATIONSHIP_ATTRIBUTE_-DESIGNATOR ) | 1020 |
| AKD_54 | 'ATTRIBUTE_KIND_DEFINITION( RELATIONSHIP_KEY_LIST ) | 1020 |
| AKD_55 | 'ATTRIBUTE_KIND_DEFINITION( RELATIONSHIP_KIND ) | 1021 |
| AKD_56 | 'ATTRIBUTE_KIND_DEFINITION( RESULTING_RIGHTS_LIST ) | 1021 |
| AKD_57 | 'ATTRIBUTE_KIND_DEFINITION( RESULTS ) | 1021 |
| AKD_58 | 'ATTRIBUTE_KIND_DEFINITION( SMALL_DURATION_MULTIPLE ) | 1022 |
| AKD_59 | 'ATTRIBUTE_KIND_DEFINITION( STRUCTURAL_KIND ) | 1022 |
| AKD_60 | 'ATTRIBUTE_KIND_DEFINITION( TIME ) | 1022 |
| AKD_61 | 'ATTRIBUTE_KIND_DEFINITION( UNCONSTRAINED_IDENTIFIER ) | 1023 |
| AKD_62 | 'ATTRIBUTE_KIND_DEFINITION( UNCONSTRAINED_LIST ) | 1024 |
| AKD_63 | 'ATTRIBUTE_KIND_DEFINITION( UNCONSTRAINED_STRING ) | 1024 |
| AKD_64 | 'ATTRIBUTE_KIND_DEFINITION( UNIQUE_IDENTIFIER ) | 1025 |
| AKD_65 | 'ATTRIBUTE_KIND_DEFINITION( VALUE_TYPE ) | 1025 |
| AKD_66 | 'ATTRIBUTE_KIND_DEFINITION( YEAR ) | 1025 |
| NKD_1 | 'NODE_KIND_DEFINITION( ATTRIBUTE_DEFINITION ) | 1027 |
| NKD_2 | 'NODE_KIND_DEFINITION( ATTRIBUTE_KIND_DEFINITION ) | 1028 |
| NKD_3 | 'NODE_KIND_DEFINITION( BASIC ) | 1029 |
| NKD_4 | 'NODE_KIND_DEFINITION( CHANNEL ) | 1029 |
| NKD_5 | 'NODE_KIND_DEFINITION( CLIENT ) | 1030 |
| NKD_6 | 'NODE_KIND_DEFINITION( COMPOSITE_LIST_ATTRIBUTE_KIND_-DEFINITION ) | 1030 |
| NKD_7 | 'NODE_KIND_DEFINITION( DEFINITION ) | 1031 |
| NKD_8 | 'NODE_KIND_DEFINITION( DEFINITIONS ) | 1031 |
| NKD_9 | 'NODE_KIND_DEFINITION( DEVICE ) | 1032 |
| NKD_10 | 'NODE_KIND_DEFINITION( EXECUTABLE_IMAGE ) | 1033 |
| NKD_11 | 'NODE_KIND_DEFINITION( FILE ) | 1033 |
| NKD_12 | 'NODE_KIND_DEFINITION( FLOAT_ATTRIBUTE_KIND_DEFINITION ) | 1034 |
| NKD_13 | 'NODE_KIND_DEFINITION( GATEWAY ) | 1034 |
| NKD_14 | 'NODE_KIND_DEFINITION( GROUP ) | 1034 |
| NKD_15 | 'NODE_KIND_DEFINITION( IDENTIFIER_ATTRIBUTE_KIND_DEFINITION ) | 1035 |
| NKD_16 | 'NODE_KIND_DEFINITION( INTEGER_ATTRIBUTE_KIND_DEFINITION ) | 1036 |




Source: https://assist.dla.mil -- Downloaded: 2026l0-26T07:54Z
Check the source to verify that this is the currentversion before use.

| UID | Path From 'DEFINITIONS( PREDEFINED ) | Page |
|---|---|---|
| NKD_17 | 'NODE_KIND_DEFINITION( INTERPRETATION ) | 1036 |
| NKD_18 | 'NODE_KIND_DEFINITION( INTRODUCED_NODE ) | 1036 |
| NKD_19 | 'NODE_KIND_DEFINITION( LIST_ATTRIBUTE_KIND_DEFINITION ) | 1037 |
| NKD_20 | 'NODE_KIND_DEFINITION( MAGNETIC_TAPE ) | 1037 |
| NKD_21 | 'NODE_KIND_DEFINITION( MEDIUM ) | 1038 |
| NKD_22 | 'NODE_KIND_DEFINITION( MONITOR ) | 1038 |
| NKD_23 | 'NODE_KIND_DEFINITION( NAMED_COMPOSITE_LIST_AKD ) | 1039 |
| NKD_24 | 'NODE_KIND_DEFINITION( NAMED_ORDERED_COMPOSITE_LIST_AKD ) | 1039 |
| NKD_25 | 'NODE_KIND_DEFINITION( NODE_KIND_DEFINITION ) | 1040 |
| NKD_26 | 'NODE_KIND_DEFINITION( NODE_KIND_INTERPRETATION ) | 1040 |
| NKD_27 | 'NODE_KIND_DEFINITION( ORDERED_COMPOSITE_LIST_AKD ) | 1041 |
| NKD_28 | 'NODE_KIND_DEFINITION( PROCESS ) | 1041 |
| NKD_29 | 'NODE_KIND_DEFINITION( PROCESSOR ) | 1042 |
| NKD_30 | 'NODE_KIND_DEFINITION( RELATION_DEFINITION ) | 1043 |
| NKD_31 | 'NODE_KIND_DEFINITION( RELATION_INTERPRETATION ) | 1044 |
| NKD_32 | 'NODE_KIND_DEFINITION( RESOURCE ) | 1044 |
| NKD_33 | 'NODE_KIND_DEFINITION( RESOURCE_GROUP ) | 1045 |
| NKD_34 | 'NODE_KIND_DEFINITION( SECONDARY_STORAGE ) | 1045 |
| NKD_35 | 'NODE_KIND_DEFINITION( STRING_ATTRIBUTE_KIND_DEFINITION ) | 1045 |
| NKD_36 | 'NODE_KIND_DEFINITION( STRUCTURAL ) | 1046 |
| NKD_37 | 'NODE_KIND_DEFINITION( SYSTEM_LEVEL ) | 1046 |
| NKD_38 | 'NODE_KIND_DEFINITION( TERMINAL ) | 1047 |
| NKD_39 | 'NODE_KIND_DEFINITION( UNIFORM_LIST_ATTRIBUTE_KIND_-DEFINITION ) | 1047 |
| NKD_40 | 'NODE_KIND_DEFINITION( USER ) | 1048 |
| NKD_41 | 'NODE_KIND_DEFINITION( VARIANT_ATTRIBUTE_KIND_DEFINITION ) | 1048 |
| NKD_42 | 'NODE_KIND_DEFINITION( VIEW ) | 1049 |
| RD_1 | 'RELATION_DEFINITION( ACCESS ) | 1050 |
| RD_2 | 'RELATION_DEFINITION( ADOPTED_ROLE ) | 1050 |
| RD_3 | 'RELATION_DEFINITION( ADOPTED_VIEW ) | 1051 |
| RD_4 | 'RELATION_DEFINITION( ALLOCATABLE_DEVICE ) | 1051 |
| RD_5 | 'RELATION_DEFINITION( ALLOCATABLE_GATEWAY ) | 1052 |
| RD_6 | 'RELATION_DEFINITION( ALLOCATABLE_MEDIUM ) | 1052 |
| RD_7 | 'RELATION_DEFINITION( ALLOCATABLE_PROCESSOR ) | 1053 |
| RD_8 | 'RELATION_DEFINITION( ALTERNATIVE ) | 1053 |
| RD_9 | 'RELATION_DEFINITION( ALTERNATIVE_OF ) | 1054 |
| RD_10 | 'RELATION_DEFINITION( ATTRIBUTE ) | 1054 |
| RD_11 | 'RELATION_DEFINITION( ATTRIBUTE_DEFINITION ) | 1055 |
| RD_12 | 'RELATION_DEFINITION( ATTRIBUTE_DEFINITION_NAME ) | 1056 |
| RD_13 | 'RELATION_DEFINITION( ATTRIBUTE_DEFINITION_PARENT ) | 1056 |
| RD_14 | 'RELATION_DEFINITION( ATTRIBUTE_KIND ) | 1056 |
| RD_15 | 'RELATION_DEFINITION( ATTRIBUTE_KIND_DEFINITION ) | 1057 |
| RD_16 | 'RELATION_DEFINITION( ATTRIBUTE_KIND_DEFINITION_PARENT ) | 1058 |
| RD_17 | 'RELATION_DEFINITION( ATTRIBUTE_KIND_NAME ) | 1058 |
| RD_18 | 'RELATION_DEFINITION( ATTRIBUTE_KIND_OF ) | 1058 |
| RD_19 | 'RELATION_DEFINITION( ATTRIBUTE_MONITOR ) | 1059 |
| RD_20 | 'RELATION_DEFINITION( ATTRIBUTE_MONITOR_PARENT ) | 1060 |
| RD_21 | 'RELATION_DEFINITION( ATTRIBUTE_NAME ) | 1060 |
| RD_22 | 'RELATION_DEFINITION( ATTRIBUTE_OF ) | 1061 |
| RD_23 | 'RELATION_DEFINITION( BASIC ) | 1061 |




Source: https://assist.dla.mil -- Downloaded: 202610-26T07:54Z
Check the source to verify that this is the current version before use.

| UID | Path From 'DEFINITIONS( PREDEFINED ) | Page |
|---|---|---|
| RD_24 | 'RELATION_DEFINITION( CHANNEL ) | 1063 |
| RD_25 | 'RELATION_DEFINITION( CHANNEL_PARENT ) | 1064 |
| RD_26 | 'RELATION_DEFINITION( CLIENT ) | 1064 |
| RD_27 | 'RELATION_DEFINITION( COMPONENT ) | 1065 |
| RD_28 | 'RELATION_DEFINITION( COMPONENT_NAMED ) | 1066 |
| RD_29 | 'RELATION_DEFINITION( COMPONENT_NAMED_ORDERED ) | 1066 |
| RD_30 | 'RELATION_DEFINITION( COMPONENT_OF ) | 1067 |
| RD_31 | 'RELATION_DEFINITION( COMPONENT_OF_NAMED ) | 1067 |
| RD_32 | 'RELATION_DEFINITION( COMPONENT_OF_NAMED_ORDERED) | 1068 |
| RD_33 | 'RELATION_DEFINITION( COMPONENT_OF_ORDERED ) | 1068 |
| RD_34 | 'RELATION_DEFINITION( COMPONENT_OF_UNIFORM_LIST ) | 1069 |
| RD_35 | 'RELATION_DEFINITION( COMPONENT_ORDERED ) | 1069 |
| RD_36 | 'RELATION_DEFINITION( COMPONENT_UNIFORM_LIST ) | 1069 |
| RD_37 | 'RELATION_DEFINITION( COPY_OF ) | 1070 |
| RD_38 | 'RELATION_DEFINITION( CURRENT_JOB ) | 1070 |
| RD_39 | 'RELATION_DEFINITION( CURRENT_NODE ) | 1071 |
| RD_40 | 'RELATION_DEFINITION( CURRENT_PROCESSOR ) | 1071 |
| RD_41 | 'RELATION_DEFINITION( CURRENT_USER ) | 1072 |
| RD_42 | 'RELATION_DEFINITION( DEFAULT_MEDIUM ) | 1072 |
| RD_43 | 'RELATION_DEFINITION( DEFAULT_PROCESSOR ) | 1073 |
| RD_44 | 'RELATION_DEFINITION( DEFAULT_PROCESS_NODE_KIND ) | 1073 |
| RD_45 | 'RELATION_DEFINITION( DEFAULT_ROLE ) | 1074 |
| RD_46 | 'RELATION_DEFINITION( DEFINITION ) | 1074 |
| RD_47 | 'RELATION_DEFINITION( DEFINITIONS_PARENT ) | 1075 |
| RD_48 | 'RELATION_DEFINITION( DEFINITIONS_PRIMARY ) | 1075 |
| RD_49 | 'RELATION_DEFINITION( DEFINITIONS_SECONDARY ) | 1076 |
| RD_50 | 'RELATION_DEFINITION( DENY ) | 1076 |
| RD_51 | 'RELATION_DEFINITION( DEVICE_PARENT) | 1077 |
| RD_52 | 'RELATION_DEFINITION( DEVICE_PRIMARY ) | 1077 |
| RD_53 | 'RELATION_DEFINITION( DEVICE_SECONDARY ) | 1078 |
| RD_54 | 'RELATION_DEFINITION( EMANATES ) | 1078 |
| RD_55 | 'RELATION_DEFINITION( EMANATES_FROM ) | 1079 |
| RD_56 | 'RELATION_DEFINITION( EXECUTABLE_IMAGE ) | 1079 |
| RD_57 | 'RELATION_DEFINITION( GATEWAY_PARENT ) | 1080 |
| RD_58 | 'RELATION_DEFINITION( GATEWAY_PRIMARY ) | 1080 |
| RD_59 | 'RELATION_DEFINITION( GATEWAY_SECONDARY ) | 1081 |
| RD_60 | 'RELATION_DEFINITION( GENERALIZATION_OF_NKD ) | 1081 |
| RD_61 | 'RELATION_DEFINITION( GENERALIZATION_OF_RD ) | 1082 |
| RD_62 | 'RELATION_DEFINITION( GROUP_PARENT ) | 1082 |
| RD_63 | 'RELATION_DEFINITION( GROUP_PRIMARY ) | 1083 |
| RD_64 | 'RELATION_DEFINITION( GROUP_SECONDARY ) | 1083 |
| RD_65 | 'RELATION_DEFINITION( HANDLER_ATTRIBUTE ) | 1084 |
| RD_66 | 'RELATION_DEFINITION( HANDLER_ENVIRONMENT_NODE ) | 1084 |
| RD_67 | 'RELATION_DEFINITION( HANDLER_ERROR_FILE ) | 1085 |
| RD_68 | 'RELATION_DEFINITION( HANDLER_IMAGE ) | 1085 |
| RD_69 | 'RELATION_DEFINITION( HANDLER_INPUT_FILE ) | 1086 |
| RD_70 | 'RELATION_DEFINITION( HANDLER_MEDIUM_RESOURCE ) | 1086 |
| RD_71 | 'RELATION_DEFINITION( HANDLER_OUTPUT_FILE ) | 1086 |
| RD_72 | 'RELATION_DEFINITION( HANDLER_PROCESS ) | 1087 |
| RD_73 | 'RELATION_DEFINITION( HANDLER_PROCESS_NODE_KIND ) | 1087 |




Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

| UID | Path From 'DEFINITIONS( PREDEFINED ) | Page |
|---|---|---|
| RD_74 | 'RELATION_DEFINITION( HANDLER_PROCESS_OF ) | 1088 |
| RD_75 | 'RELATION_DEFINITION( HANDLER_PROCESSOR_RESOURCE ) | 1088 |
| RD_76 | 'RELATION_DEFINITION( INTRODUCED_NODE ) | 1089 |
| RD_77 | 'RELATION_DEFINITION( INTRODUCED_NODE_PARENT ) | 1089 |
| RD_78 | 'RELATION_DEFINITION( INTRODUCED_RELATION ) | 1090 |
| RD_79 | 'RELATION_DEFINITION( JOB ) | 1090 |
| RD_80 | 'RELATION_DEFINITION( JOB_PARENT ) | 1091 |
| RD_81 | 'RELATION_DEFINITION( KEY ) | 1091 |
| RD_82 | 'RELATION_DEFINITION( KEY_NAME ) | 1092 |
| RD_83 | 'RELATION_DEFINITION( KEY_OF ) | 1092 |
| RD_84 | 'RELATION_DEFINITION( MEDIUM_PARENT ) | 1093 |
| RD_85 | 'RELATION_DEFINITION( MEDIUM_PRIMARY ) | 1093 |
| RD_86 | 'RELATION_DEFINITION( MEDIUM_SECONDARY ) | 1094 |
| RD_87 | 'RELATION_DEFINITION( MONITORED_BY ) | 1094 |
| RD_88 | 'RELATION_DEFINITION( MONITORS ) | 1095 |
| RD_89 | 'RELATION_DEFINITION( NODE_KIND ) | 1095 |
| RD_90 | 'RELATION_DEFINITION( NODE_KIND_DEFINITION ) | 1096 |
| RD_91 | 'RELATION_DEFINITION( NODE_KIND_DEFINITION_PARENT ) | 1096 |
| RD_92 | 'RELATION_DEFINITION( NODE_KIND_PARENT ) | 1097 |
| RD_93 | 'RELATION_DEFINITION( PARENT ) | 1097 |
| RD_94 | 'RELATION_DEFINITION( PERMANENT_MEMBER ) | 1098 |
| RD_95 | 'RELATION_DEFINITION( PERMANENT_MEMBER_PARENT ) | 1098 |
| RD_96 | 'RELATION_DEFINITION( PORT ) | 1099 |
| RD_97 | 'RELATION_DEFINITION( POTENTIAL_MEMBER ) | 1100 |
| RD_98 | 'RELATION_DEFINITION( PROCESSOR_PARENT) | 1100 |
| RD_99 | 'RELATION_DEFINITION( PROCESSOR_PRIMARY ) | 1100 |
| RD_100 | 'RELATION_DEFINITION( PROCESSOR_SECONDARY ) | 1101 |
| RD_101 | 'RELATION_DEFINITION( RELATION ) | 1101 |
| RD_102 | 'RELATION_DEFINITION( RELATION_DEFINITION ) | 1102 |
| RD_103 | 'RELATION_DEFINITION( RELATION_DEFINITION_PARENT ) | 1103 |
| RD_104 | 'RELATION_DEFINITION( RELATION_NAME ) | 1103 |
| RD_105 | 'RELATION_DEFINITION( RELATION_PARENT ) | 1103 |
| RD_106 | 'RELATION_DEFINITION( RESOURCE_GROUP_PARENT ) | 1104 |
| RD_107 | 'RELATION_DEFINITION( RESOURCE_GROUP_PRIMARY ) | 1104 |
| RD_108 | 'RELATION_DEFINITION( RESOURCE_GROUP_SECONDARY ) | 1105 |
| RD_109 | 'RELATION_DEFINITION( REVERSE_LINK ) | 1105 |
| RD_110 | 'RELATION_DEFINITION( REVERSE_LINK_BIDIRECTIONAL ) | 1106 |
| RD_111 | 'RELATION_DEFINITION( SPECIALIZATION_OF_NKD ) | 1106 |
| RD_112 | 'RELATION_DEFINITION( SPECIALIZATION_OF_RD ) | 1107 |
| RD_113 | 'RELATION_DEFINITION( STANDARD_ERROR ) | 1107 |
| RD_114 | 'RELATION_DEFINITION( STANDARD_INPUT ) | 1108 |
| RD_115 | 'RELATION_DEFINITION( STANDARD_OUTPUT ) | 1108 |
| RD_116 | 'RELATION_DEFINITION( TERMINATES ) | 1109 |
| RD_117 | 'RELATION_DEFINITION( TERMINATES_AT ) | 1109 |
| RD_118 | 'RELATION_DEFINITION( USER_PARENT) | 1110 |
| RD_119 | 'RELATION_DEFINITION( USER_PRIMARY ) | 1110 |
| RD_120 | 'RELATION_DEFINITION( USER_SECONDARY ) | 1111 |
| RD_121 | 'RELATION_DEFINITION( VIEW_PARENT ) | 1111 |
| RD_122 | 'RELATION_DEFINITION( VIEW_PRIMARY ) | 1112 |
| RD_123 | 'RELATION_DEFINITION( VIEW_SECONDARY ) | 1112 |



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

| UID | Path From 'VIEW( PREDEFINED ) | Page |
|---|---|---|
| NKI_1 | 'NODE_KIND( ATTRIBUTE_DEFINITION ) | 1114 |
| NKI_2 | 'NODE_KIND( ATTRIBUTE_KIND_DEFINITION ) | 1114 |
| NKI_3 | 'NODE_KIND( BASIC ) | 1115 |
| NKI_4 | 'NODE_KIND( CHANNEL ) | 1115 |
| NKI_5 | 'NODE_KIND( CLIENT ) | 1116 |
| NKI_6 | 'NODE_KIND( COMPOSITE_LIST_ATTRIBUTE_KIND_DEFINITION ) | 1116 |
| NKI_7 | 'NODE_KIND( DEFINITION ) | 1116 |
| NKI_8 | 'NODE_KIND( DEFINITIONS ) | 1117 |
| NKI_9 | 'NODE_KIND( DEVICE ) | 1117 |
| NKI_10 | 'NODE_KIND( EXECUTABLE_IMAGE ) | 1117 |
| NKI_11 | 'NODE_KIND( FILE ) | 1118 |
| NKI_12 | 'NODE_KIND( FLOAT_ATTRIBUTE_KIND_DEFINITION ) | 1118 |
| NKI_13 | 'NODE_KIND( GATEWAY ) | 1118 |
| NKI_14 | 'NODE_KIND( GROUP ) | 1119 |
| NKI_15 | 'NODE_KIND( IDENTIFIER_ATTRIBUTE_KIND_DEFINITION ) | 1119 |
| NKI_16 | 'NODE_KIND( INTEGER_ATTRIBUTE_KIND_DEFINITION ) | 1119 |
| NKI_17 | 'NODE_KIND( INTERPRETATION ) | 1120 |
| NKI_18 | 'NODE_KIND( INTRODUCED_NODE ) | 1120 |
| NKI_19 | 'NODE_KIND( LIST_ATTRIBUTE_KIND_DEFINITION ) | 1120 |
| NKI_20 | 'NODE_KIND( MAGNETIC_TAPE ) | 1121 |
| NKI_21 | 'NODE_KIND( MEDIUM ) | 1121 |
| NKI_22 | 'NODE_KIND( MONITOR ) | 1121 |
| NKI_23 | 'NODE_KIND( NAMED_COMPOSITE_LIST_AKD ) | 1122 |
| NKI_24 | 'NODE_KIND( NAMED_ORDERED_COMPOSITE_LIST_AKD ) | 1122 |
| NKI_25 | 'NODE_KIND( NODE_KIND_DEFINITION ) | 1122 |
| NKI_26 | 'NODE_KIND( NODE_KIND_INTERPRETATION ) | 1123 |
| NKI_27 | 'NODE_KIND( ORDERED_COMPOSITE_LIST_AKD ) | 1123 |
| NKI_28 | 'NODE_KIND( PROCESS ) | 1123 |
| NKI_29 | 'NODE_KIND( PROCESSOR ) | 1124 |
| NKI_30 | 'NODE_KIND( RELATION_DEFINITION ) | 1125 |
| NKI_31 | 'NODE_KIND( RELATION_INTERPRETATION ) | 1125 |
| NKI_32 | 'NODE_KIND( RESOURCE ) | 1125 |
| NKI_33 | 'NODE_KIND( RESOURCE_GROUP ) | 1126 |
| NKI_34 | 'NODE_KIND( SECONDARY_STORAGE ) | 1126 |
| NKI_35 | 'NODE_KIND( STRING_ATTRIBUTE_KIND_DEFINITION ) | 1126 |
| NKI_36 | 'NODE_KIND( STRUCTURAL ) | 1127 |
| NKI_37 | 'NODE_KIND( SYSTEM_LEVEL ) | 1127 |
| NKI_38 | 'NODE_KIND( TERMINAL ) | 1127 |
| NKI_39 | 'NODE_KIND( UNIFORM_LIST_ATTRIBUTE_KIND_DEFINITION ) | 1128 |
| NKI_40 | 'NODE_KIND( USER ) | 1128 |
| NKI_41 | 'NODE_KIND( VARIANT_ATTRIBUTE_KIND_DEFINITION ) | 1128 |
| NKI_42 | 'NODE_KIND( VIEW ) | 1129 |
| RI_1 | 'RELATION( ACCESS ) | 1130 |
| RI_2 | 'RELATION( ADOPTED_ROLE ) | 1130 |
| RI_3 | 'RELATION( ADOPTED_VIEW ) | 1130 |
| RI_4 | 'RELATION( ALLOCATABLE_DEVICE ) | 1131 |
| RI_5 | 'RELATION( ALLOCATABLE_GATEWAY ) | 1131 |
| RI_6 | 'RELATION( ALLOCATABLE_MEDIUM ) | 1131 |
| RI_7 | 'RELATION( ALLOCATABLE_PROCESSOR ) | 1132 |
| RI_8 | 'RELATION( ALTERNATIVE ) | 1132 |




Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

| UID | Path From 'VIEW( PREDEFINED ) | Page |
|---|---|---|
| RI_9 | 'RELATION( ALTERNATIVE_OF ) | 1132 |
| RI_10 | 'RELATION( ATTRIBUTE ) | 1132 |
| RI_11 | 'RELATION( ATTRIBUTE_DEFINITION ) | 1133 |
| RI_12 | 'RELATION( ATTRIBUTE_DEFINITION_NAME ) | 1133 |
| RI_13 | 'RELATION( ATTRIBUTE_DEFINITION_PARENT ) | 1133 |
| RI_14 | 'RELATION( ATTRIBUTE_KIND ) | 1134 |
| RI_15 | 'RELATION( ATTRIBUTE_KIND_DEFINITION ) | 1134 |
| RI_16 | 'RELATION( ATTRIBUTE_KIND_DEFINITION_PARENT ) | 1134 |
| RI_17 | 'RELATION( ATTRIBUTE_KIND_NAME ) | 1135 |
| RI_18 | 'RELATION( ATTRIBUTE_KIND_OF ) | 1135 |
| RI_19 | 'RELATION( ATTRIBUTE_MONITOR ) | 1135 |
| RI_20 | 'RELATION( ATTRIBUTE_MONITOR_PARENT ) | 1135 |
| RI_21 | 'RELATION( ATTRIBUTE_NAME ) | 1136 |
| RI_22 | 'RELATION( ATTRIBUTE_OF ) | 1136 |
| RI_23 | 'RELATION( BASIC ) | 1136 |
| RI_24 | 'RELATION( CHANNEL ) | 1137 |
| RI_25 | 'RELATION( CHANNEL_PARENT ) | 1137 |
| RI_26 | 'RELATION( CLIENT ) | 1137 |
| RI_27 | 'RELATION( COMPONENT ) | 1137 |
| RI_28 | 'RELATION( COMPONENT_NAMED ) | 1138 |
| RI_29 | 'RELATION( COMPONENT_NAMED_ORDERED ) | 1138 |
| RI_30 | 'RELATION( COMPONENT_OF ) | 1138 |
| RI_31 | 'RELATION( COMPONENT_OF_NAMED ) | 1139 |
| RI_32 | 'RELATION( COMPONENT_OF_NAMED_ORDERED ) | 1139 |
| RI_33 | 'RELATION( COMPONENT_OF_ORDERED ) | 1139 |
| RI_34 | 'RELATION( COMPONENT_OF_UNIFORM_LIST ) | 1140 |
| RI_35 | 'RELATION( COMPONENT_ORDERED ) | 1140 |
| RI_36 | 'RELATION( COMPONENT_UNIFORM_LIST ) | 1140 |
| RI_37 | 'RELATION( COPY_OF ) | 1140 |
| RI_38 | 'RELATION( CURRENT_JOB ) | 1141 |
| RI_39 | 'RELATION( CURRENT_NODE ) | 1141 |
| RI_40 | 'RELATION( CURRENT_PROCESSOR ) | 1141 |
| RI_41 | 'RELATION( CURRENT_USER ) | 1142 |
| RI_42 | 'RELATION( DEFAULT_MEDIUM ) | 1142 |
| RI_43 | 'RELATION( DEFAULT_PROCESSOR ) | 1142 |
| RI_44 | 'RELATION( DEFAULT_PROCESS_NODE_KIND ) | 1142 |
| RI_45 | 'RELATION( DEFAULT_ROLE ) | 1143 |
| RI_46 | 'RELATION( DEFINITION ) | 1143 |
| RI_47 | 'RELATION( DEFINITIONS_PARENT ) | 1143 |
| RI_48 | 'RELATION( DEFINITIONS_PRIMARY ) | 1144 |
| RI_49 | 'RELATION( DEFINITIONS_SECONDARY ) | 1144 |
| RI_50 | 'RELATION( DENY ) | 1144 |
| RI_51 | 'RELATION( DEVICE_PARENT ) | 1145 |
| RI_52 | 'RELATION( DEVICE_PRIMARY ) | 1145 |
| RI_53 | 'RELATION( DEVICE_SECONDARY ) | 1145 |
| RI_54 | 'RELATION( EMANATES ) | 1145 |
| RI_55 | 'RELATION( EMANATES_FROM ) | 1146 |
| RI_56 | 'RELATION( EXECUTABLE_IMAGE ) | 1146 |
| RI_57 | 'RELATION( GATEWAY_PARENT ) | 1146 |
| RI_58 | 'RELATION( GATEWAY_PRIMARY ) | 1147 |



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

| UID | Path From 'VIEW( PREDEFINED ) | Page |
|---|---|---|
| RI_59 | 'RELATION( GATEWAY_SECONDARY ) | 1147 |
| RI_60 | 'RELATION( GENERALIZATION_OF_NKD ) | 1147 |
| RI_61 | 'RELATION( GENERALIZATION_OF_RD ) | 1148 |
| RI_62 | 'RELATION( GROUP_PARENT ) | 1148 |
| RI_63 | 'RELATION( GROUP_PRIMARY ) | 1148 |
| RI_64 | 'RELATION( GROUP_SECONDARY ) | 1148 |
| RI_65 | 'RELATION( HANDLER_ATTRIBUTE ) | 1149 |
| RI_66 | 'RELATION( HANDLER_ENVIRONMENT_NODE ) | 1149 |
| RI_67 | 'RELATION( HANDLER_ERROR_FILE ) | 1149 |
| RI_68 | 'RELATION( HANDLER_IMAGE ) | 1150 |
| RI_69 | 'RELATION( HANDLER_INPUT_FILE ) | 1150 |
| RI_70 | 'RELATION( HANDLER_MEDIUM_RESOURCE ) | 1150 |
| RI_71 | 'RELATION( HANDLER_OUTPUT_FILE ) | 1151 |
| RI_72 | 'RELATION( HANDLER_PROCESS ) | 1151 |
| RI_73 | 'RELATION( HANDLER_PROCESS_NODE_KIND ) | 1151 |
| RI_74 | 'RELATION( HANDLER_PROCESS_OF ) | 1151 |
| RI_75 | 'RELATION( HANDLER_PROCESSOR_RESOURCE ) | 1152 |
| RI_76 | 'RELATION( INTRODUCED_NODE ) | 1152 |
| RI_77 | 'RELATION( INTRODUCED_NODE_PARENT ) | 1152 |
| RI_78 | 'RELATION( INTRODUCED_RELATION ) | 1153 |
| RI_79 | 'RELATION( JOB ) | 1153 |
| RI_80 | 'RELATION( JOB_PARENT ) | 1153 |
| RI_81 | 'RELATION( KEY ) | 1154 |
| RI_82 | 'RELATION( KEY_NAME ) | 1154 |
| RI_83 | 'RELATION( KEY_OF ) | 1154 |
| RI_84 | 'RELATION( MEDIUM_PARENT ) | 1154 |
| RI_85 | 'RELATION( MEDIUM_PRIMARY ) | 1155 |
| RI_86 | 'RELATION( MEDIUM_SECONDARY ) | 1155 |
| RI_87 | 'RELATION( MONITORED_BY ) | 1155 |
| RI_88 | 'RELATION( MONITORS ) | 1156 |
| RI_89 | 'RELATION( NODE_KIND ) | 1156 |
| RI_90 | 'RELATION( NODE_KIND_DEFINITION ) | 1156 |
| RI_91 | 'RELATION( NODE_KIND_DEFINITION_PARENT ) | 1157 |
| RI_92 | 'RELATION( NODE_KIND_PARENT ) | 1157 |
| RI_93 | 'RELATION( PARENT ) | 1157 |
| RI_94 | 'RELATION( PERMANENT_MEMBER ) | 1157 |
| RI_95 | 'RELATION( PERMANENT_MEMBER_PARENT ) | 1158 |
| RI_96 | 'RELATION( PORT ) | 1158 |
| RI_97 | 'RELATION( POTENTIAL_MEMBER ) | 1158 |
| RI_98 | 'RELATION( PROCESSOR_PARENT ) | 1159 |
| RI_99 | 'RELATION( PROCESSOR_PRIMARY ) | 1159 |
| RI_100 | 'RELATION( PROCESSOR_SECONDARY ) | 1159 |
| RI_101 | 'RELATION( RELATION ) | 1160 |
| RI_102 | 'RELATION( RELATION_DEFINITION ) | 1160 |
| RI_103 | 'RELATION( RELATION_DEFINITION_PARENT ) | 1160 |
| RI_104 | 'RELATION( RELATION_NAME ) | 1160 |
| RI_105 | 'RELATION( RELATION_PARENT ) | 1161 |
| RI_106 | 'RELATION( RESOURCE_GROUP_PARENT ) | 1161 |
| RI_107 | 'RELATION( RESOURCE_GROUP_PRIMARY ) | 1161 |
| RI_108 | 'RELATION( RESOURCE_GROUP_SECONDARY ) | 1162 |




Source: https://assist.dla.mil -- Downloaded: 202610-26T07:54Z
Check the source to verify that this is the current version before use.

| UID | Path From 'VIEW( PREDEFINED ) | Page |
|---|---|---|
| RI_109 | 'RELATION( REVERSE_LINK ) | 1162 |
| RI_110 | 'RELATION( REVERSE_LINK_BIDIRECTIONAL ) | 1162 |
| RI_111 | 'RELATION( SPECIALIZATION_OF_NKD ) | 1162 |
| RI_112 | 'RELATION( SPECIALIZATION_OF_RD ) | 1163 |
| RI_113 | 'RELATION( STANDARD_ERROR ) | 1163 |
| RI_114 | 'RELATION( STANDARD_INPUT ) | 1163 |
| RI_115 | 'RELATION( STANDARD_OUTPUT ) | 1164 |
| RI_116 | 'RELATION( TERMINATES ) | 1164 |
| RI_117 | 'RELATION( TERMINATES_AT ) | 1164 |
| RI_118 | 'RELATION( USER_PARENT ) | 1165 |
| RI_119 | 'RELATION( USER_PRIMARY ) | 1165 |
| RI_120 | 'RELATION( USER_SECONDARY ) | 1165 |
| RI_121 | 'RELATION( VIEW_PARENT ) | 1165 |
| RI_122 | 'RELATION( VIEW_PRIMARY ) | 1166 |
| RI_123 | 'RELATION( VIEW_SECONDARY ) | 1166 |




Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

# 20 Predefine Node Descriptions

Each predefine node is described in a table as outlined in Table XLI. Each predefined node description begins with a subtable which identifies the node being described and the NKD of which it is an instance. A second subtable lists the attributes of the node and their values. A third subtable lists the relationships which emanate from the node being described. Each relationship description describes the values of attributes and keys of a relationship and indicates the target node of the relationship. If the relationship is one of a pair of unidirectional relationships forming a bidirectional relationship, the corresponding reverse relationship emanates from the indicated target node and is identified by a key value equal to the indicated node identifier for its target node. There are two exceptions to this convention. (1) If the relationship is a primary relationship its corresponding reverse relationship has no key. (2) Whenever two relationships of a given bidirectional relation have the same source and target nodes the corresponding reverse relationships are explicitly indicated. 1

The relation, attribute and key names shown in the predefine node descriptions are those defined by the predefine interpretation nodes that constitute the predefine view. 2

A CAIS implementation must assign a standard unique identifier to each of the predefined nodes. The unique identifier which must be assigned to a given predefine node must be of the form “PREDEFINED.suffix” where “PREDEFINED” is a reserved unique identifier prefix and “suffix” denotes the unique identifier suffix indicated in the corresponding predefine node description. Each node has the predefine string-valued attribute UNIQUE_IDENTIFIER which contains the unique identifier for the node. These attributes are not listed in the node descriptions since they can be derived from the suffix value which is specified in each description. 3

Although they are not listed in the predefine node descriptions, the predefine attributes TIME_CREATED, TIME_ATTRIBUTE_WRITTEN and TIME_RELATIONSHIP_WRITTEN must be created by a CAIS implementation on each predefined node. The YEAR, MONTH and DAY components of the TIME-CREATED attribute value on each predefined node must correspond to the date of this standard. The SECONDS component of the TIME_CREATED attribute value on each predefined node must have the value zero. The values of the predefine attributes TIME_ATTRIBUTE_WRITTEN and TIME_RELATIONSHIP_WRITTEN on the predefined nodes are implementation-dependent, however, for a given predefined node, those attributes must have values which are greater than or equal to the value of the TIME_CREATED attribute. 4

The value of the ANNOTATION attribute on each predefined node is implementation-defined. Within the descriptions of these predefined nodes, ANNOTATION attribute values are sometimes specified which summarize the usage of a definition. However, a CAIS implementation is not required to create any of these indicated ANNOTATION attribute values. 5

The predefine relationship attribute NODE_KIND is not explicitly indicated but must be created by a CAIS implementation on each relationship specified by the tables and must have the same value as the KIND attribute of the target node of the relationship. 6

The INHERITABLE attribute must be created on all relationships. The INHERITABLE attribute is not indicated in the predefine node descriptions for unidirectional relationships when its value is TRUE. The INHERITABLE attribute is never indicated for bidirectional relationships since the value is always FALSE. 7

The value of the predefine attribute OBJECT_CLASSIFICATION on each of the predefined nodes is implementation-defied and not indicated in the predefine node descriptions. 8

Because all GROUP nodes are implementation-defined, the granting of discretionary access control 9



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

**TABLE XLI. Predefined Node Description Fields**

| Column | Contents |
|---|---|
| Node | Suffix of the standard unique identifier (UID) for the predefined node. |
| Node Identification | Relationship key of the primary relationship to the predefine node. |
| NKD | Suffix of the standard unique identifier (UID) for the NKD of which the predefine node is an instance. |
| NKD Identification | Relationship key of the primary relationship to the NKD. |
| Attribute Name | Name of an attribute of the predefined node. |
| AD | Suffix of the unique identifier (UID) for the AD identified by the indicated attribute name. |
| Attribute Value | Value of the indicated attribute expressed as a CAIS_LIST. MANAGEMENT list. |
| c | Contains a code letter designating the content of each line of a relationship description entry. On the initial line of a relationship description entry, this column contains "P" if the relationship is a primary relationship or "S" if the relationship is a secondary relationship. Additional lines within a relationship description entry are used to specify relationship attributes, relationship key values for relations with more than one key and corresponding reverse relationships. Such entries are indicated by the codes "A", "K" and "R" respectively. (Corresponding reverse relationships are specified whenever the identification of such relationships cannot be unambiguously determined from the context.) |
| RD/AD | On the initial line of a relationship description entry, this column contains the suffix of the unique identifier (UID) for the RD of which the relationship is an instance. For lines describing attribute or key values, this column contains the suffix of the UID of the corresponding AD. |
| Relation/Attribute Name | Contains either a relation, attribute or key name depending on the code indicated for the same line of the relationship description entry. |
| Key/Attribute Value | Relationship key or attribute value expressed as a CAIS_LIST_MANAGEMENT list. |
| Target | Suffix of the standard unique identifier for the target node of the indicated relationship. |


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

access rights to the predefine nodes is implementation-defined. For this reason, the descriptions of the predefine nodes do not indicate the existence of any relationships of the predefine relations ACCESS and DENY.



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

## 20.1 Top-level nodes

The following table describes the node attributes which have the same value on all of the predefined top-level nodes:

| Attribute Name | AD | Attribute Value |
|---|---|---|
| KIND | AD_45 | (STRUCTURAL) |

### 20.1.1 System-level node

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| N_1 | System-level node | NKD_37 | SYSTEM_LEVEL |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| STRUCTURAL_KIND | AD_76 | (SYSTEM_LEVEL) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| P | RD_48 | DEFINITIONS | (PREDEFINED) | N_2 |
| P | RD_85 | MEDIUM | (TRANSPARENT_MEDIUM_-RESOURCE) | N_3 |
| P | RD_99 | PROCESSOR | (TRANSPARENT_PROCESSOR_-RESOURCE) | N_4 |
| P | RD_107 | RESOURCE_GROUP | (TRANSPARENT_RESOURCES) | N_5 |
| P | RD_122 | VIEW | (PREDEFINED) | N_6 |

### 20.1.2 'DEFINITIONS( PREDEFINED )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| N_2 | Predefined Definitions Node | NKD_8 | DEFINITIONS |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("Node which is the parent of all the predefined definition nodes.") |
| STRUCTURAL_KIND | AD_76 | (DEFINITIONS) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| P | RD_11 | ATTRIBUTE_DEFINITION | (ALLOWED_PROTOCOLS) | AD_1 |
| P | RD_11 | ATTRIBUTE_DEFINITION | (ALLOWED_USAGE) | AD_2 |
| P | RD_11 | ATTRIBUTE_DEFINITION | (ANNOTATION) | AD_3 |
| P | RD_11 | ATTRIBUTE_DEFINITION | (ATTRIBUTE_VALUE) | AD_4 |
| P | RD_11 | ATTRIBUTE_DEFINITION | (ATTRIBUTES_MONITORED) | AD_5 |
| P | RD_11 | ATTRIBUTE_DEFINITION | (AVAILABLE) | AD_6 |
| P | RD_11 | ATTRIBUTE_DEFINITION | (CAIS_CONTROLLED) | AD_7 |
| P | RD_11 | ATTRIBUTE_DEFINITION | (CAIS_DETERMINED) | AD_8 |
| P | RD_11 | ATTRIBUTE_DEFINITION | (CAIS_KEY) | AD_9 |
| P | RD_11 | ATTRIBUTE_DEFINITION | (CONSTRAINTS) | AD_10 |
| P | RD_11 | ATTRIBUTE_DEFINITION | (CONTROLLING_PROTOCOL) | AD_11 |
| P | RD_11 | ATTRIBUTE_DEFINITION | (CURRENT_FILE_SIZE) | AD_12 |
| P | RD_11 | ATTRIBUTE_DEFINITION | (CURRENT_STATUS) | AD_13 |
| P | RD_11 | ATTRIBUTE_DEFINITION | (DEFAULT_VALUE) | AD_14 |
| P | RD_11 | ATTRIBUTE_DEFINITION | (DEFINITION_KIND) | AD_15 |


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| P | RD_11 | ATTRIBUTE_DEFINITION | (DELETABLE) | AD_16 |
| P | RD_11 | ATTRIBUTE_DEFINITION | (DEVICE_KIND) | AD_17 |
| P | RD_11 | ATTRIBUTE_DEFINITION | (DIGITS) | AD_18 |
| P | RD_11 | ATTRIBUTE_DEFINITION | (DIRECTION) | AD_19 |
| P | RD_11 | ATTRIBUTE_DEFINITION | (ELEMENT_SIZE) | AD_20 |
| P | RD_11 | ATTRIBUTE_DEFINITION | (ENUMERATED) | AD_21 |
| P | RD_11 | ATTRIBUTE_DEFINITION | (FILE_KIND) | AD_22 |
| P | RD_11 | ATTRIBUTE_DEFINITION | (GENERIC_KEY) | AD_23 |
| P | RD_11 | ATTRIBUTE_DEFINITION | (GRANT) | AD_24 |
| P | RD_11 | ATTRIBUTE_DEFINITION | (HANDLER_AS_A_-TRANSACTION) | AD_25 |
| P | RD_11 | ATTRIBUTE_DEFINITION | (HANDLER_DELETE_WHEN_-TERMINATED) | AD_26 |
| P | RD_11 | ATTRIBUTE_DEFINITION | (HANDLER_DISCRETIONARY_-ACCESS) | AD_27 |
| P | RD_11 | ATTRIBUTE_DEFINITION | (HANDLER_INPUT_-PARAMETERS) | AD_28 |
| P | RD_11 | ATTRIBUTE_DEFINITION | (HANDLER_KEY) | AD_29 |
| P | RD_11 | ATTRIBUTE_DEFINITION | (HANDLER_MANDATORY_-ACCESS) | AD_30 |
| P | RD_11 | ATTRIBUTE_DEFINITION | (HANDLER_PRIORITY) | AD_31 |
| P | RD_11 | ATTRIBUTE_DEFINITION | (HIGHEST_CLASSIFICATION) | AD_32 |
| P | RD_11 | ATTRIBUTE_DEFINITION | (IMPORTED_DEFINITION_UID) | AD_33 |
| P | RD_11 | ATTRIBUTE_DEFINITION | (IMPORTED_NAME) | AD_34 |
| P | RD_11 | ATTRIBUTE_DEFINITION | (IMPORTED_RELATIONSHIP_-KIND) | AD_35 |
| P | RD_11 | ATTRIBUTE_DEFINITION | (IMPORTED_TARGET_UID) | AD_36 |
| P | RD_11 | ATTRIBUTE_DEFINITION | (IMPORTED_UID) | AD_37 |
| P | RD_11 | ATTRIBUTE_DEFINITION | (INCOMING_FRAME_LENGTH) | AD_38 |
| P | RD_11 | ATTRIBUTE_DEFINITION | (INHERITABLE) | AD_39 |
| P | RD_11 | ATTRIBUTE_DEFINITION | (INTERPRETATION_KIND) | AD_40 |
| P | RD_11 | ATTRIBUTE_DEFINITION | (INTRODUCED_KEY_-ATTRIBUTES) | AD_41 |
| P | RD_11 | ATTRIBUTE_DEFINITION | (INTRODUCED_NODE_-ATTRIBUTES) | AD_42 |
| P | RD_11 | ATTRIBUTE_DEFINITION | (INTRODUCED_-RELATIONSHIP_ATTRIBUTES) | AD_43 |
| P | RD_11 | ATTRIBUTE_DEFINITION | (IO_UNIT_COUNT) | AD_44 |
| P | RD_11 | ATTRIBUTE_DEFINITION | (KIND) | AD_45 |
| P | RD_11 | ATTRIBUTE_DEFINITION | (LOWER_BOUND_FLOAT) | AD_46 |
| P | RD_11 | ATTRIBUTE_DEFINITION | (LOWER_BOUND_INTEGER) | AD_47 |
| P | RD_11 | ATTRIBUTE_DEFINITION | (LOWEST_CLASSIFICATION) | AD_48 |
| P | RD_11 | ATTRIBUTE_DEFINITION | (MACHINE_TIME) | AD_49 |
| P | RD_11 | ATTRIBUTE_DEFINITION | (MAXIMUM_FILE_SIZE) | AD_50 |
| P | RD_11 | ATTRIBUTE_DEFINITION | (MAXIMUM_INSTANCES) | AD_51 |
| P | RD_11 | ATTRIBUTE_DEFINITION | (MAXIMUM_LINE_LENGTH) | AD_52 |
| P | RD_11 | ATTRIBUTE_DEFINITION | (MAXIMUM_PAGE_LENGTH) | AD_53 |
| P | RD_11 | ATTRIBUTE_DEFINITION | (MAXIMUM_SIZE) | AD_54 |
| P | RD_11 | ATTRIBUTE_DEFINITION | (MINIMUM_INSTANCES) | AD_55 |
| P | RD_11 | ATTRIBUTE_DEFINITION | (MINIMUM_SIZE) | AD_56 |
| P | RD_11 | ATTRIBUTE_DEFINITION | (MODIFIABLE) | AD_57 |



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| P | RD_11 | ATTRIBUTE_DEFINITION | (NAME_KEY) | AD_58 |
| P | RD_11 | ATTRIBUTE_DEFINITION | (NAMING) | AD_59 |
| P | RD_11 | ATTRIBUTE_DEFINITION | (NODE_KIND) | AD_60 |
| P | RD_11 | ATTRIBUTE_DEFINITION | (OBJECT_CLASSIFICATION) | AD_61 |
| P | RD_11 | ATTRIBUTE_DEFINITION | (OPEN_NODE_HANDLE_COUNT) | AD_62 |
| P | RD_11 | ATTRIBUTE_DEFINITION | (OPTIONAL) | AD_63 |
| P | RD_11 | ATTRIBUTE_DEFINITION | (ORDERED) | AD_64 |
| P | RD_11 | ATTRIBUTE_DEFINITION | (OUTGOING_FRAME_LENGTH_-RANGE) | AD_65 |
| P | RD_11 | ATTRIBUTE_DEFINITION | (PARAMETERS) | AD_66 |
| P | RD_11 | ATTRIBUTE_DEFINITION | (PATTERNS) | AD_67 |
| P | RD_11 | ATTRIBUTE_DEFINITION | (POSITION_KEY) | AD_68 |
| P | RD_11 | ATTRIBUTE_DEFINITION | (PRIORITY) | AD_69 |
| P | RD_11 | ATTRIBUTE_DEFINITION | (PRIORITY_LEVEL) | AD_70 |
| P | RD_11 | ATTRIBUTE_DEFINITION | (PROCESS_SIZE) | AD_71 |
| P | RD_11 | ATTRIBUTE_DEFINITION | (QUEUE_LENGTH) | AD_72 |
| P | RD_11 | ATTRIBUTE_DEFINITION | (RESULTS) | AD_73 |
| P | RD_11 | ATTRIBUTE_DEFINITION | (SUBJECT_CLASSIFICATION) | AD_74 |
| P | RD_11 | ATTRIBUTE_DEFINITION | (STATE) | AD_75 |
| P | RD_11 | ATTRIBUTE_DEFINITION | (STRUCTURAL_KIND) | AD_76 |
| P | RD_11 | ATTRIBUTE_DEFINITION | (TIME_ATTRIBUTE_WRITTEN) | AD_77 |
| P | RD_11 | ATTRIBUTE_DEFINITION | (TIME_CONTENTS_WRITTEN) | AD_78 |
| P | RD_11 | ATTRIBUTE_DEFINITION | (TIME_CREATED) | AD_79 |
| P | RD_11 | ATTRIBUTE_DEFINITION | (TIME_FINISHED) | AD_80 |
| P | RD_11 | ATTRIBUTE_DEFINITION | (TIME_RELATIONSHIP_-WRITTEN) | AD_81 |
| P | RD_11 | ATTRIBUTE_DEFINITION | (TIME_STARTED) | AD_82 |
| P | RD_11 | ATTRIBUTE_DEFINITION | (UNIQUE_IDENTIFIER) | AD_83 |
| P | RD_11 | ATTRIBUTE_DEFINITION | (UNIQUE_KEYS) | AD_84 |
| P | RD_11 | ATTRIBUTE_DEFINITION | (UPPER_BOUND_FLOAT) | AD_85 |
| P | RD_11 | ATTRIBUTE_DEFINITION | (UPPER_BOUND_INTEGER) | AD_86 |
| P | RD_11 | ATTRIBUTE_DEFINITION | (VALUE_TYPE) | AD_87 |
| P | RD_15 | ATTRIBUTE_KIND_DEFINITION | (ALTERNATIVE_CONSTRAINT) | AKD_1 |
| P | RD_15 | ATTRIBUTE_KIND_DEFINITION | (ANNOTATION_KINDS) | AKD_2 |
| P | RD_15 | ATTRIBUTE_KIND_DEFINITION | (ATTRIBUTE_DESIGNATOR) | AKD_3 |
| P | RD_15 | ATTRIBUTE_KIND_DEFINITION | (ATTRIBUTES_MONITORED) | AKD_4 |
| P | RD_15 | ATTRIBUTE_KIND_DEFINITION | (BOOLEAN) | AKD_5 |
| P | RD_15 | ATTRIBUTE_KIND_DEFINITION | (CAIS_FLOAT) | AKD_6 |
| P | RD_15 | ATTRIBUTE_KIND_DEFINITION | (CAIS_INTEGER) | AKD_7 |
| P | RD_15 | ATTRIBUTE_KIND_DEFINITION | (CAIS_NATURAL) | AKD_8 |
| P | RD_15 | ATTRIBUTE_KIND_DEFINITION | (CAIS_POSITIVE) | AKD_9 |
| P | RD_15 | ATTRIBUTE_KIND_DEFINITION | (CHANNEL_STATE) | AKD_10 |
| P | RD_15 | ATTRIBUTE_KIND_DEFINITION | (CLASSIFICATION) | AKD_11 |
| P | RD_15 | ATTRIBUTE_KIND_DEFINITION | (COMPONENT_CONSTRAINT) | AKD_12 |
| P | RD_15 | ATTRIBUTE_KIND_DEFINITION | (COMPOSITE_LIST_-COMPONENT_CONSTRAINT) | AKD_13 |
| P | RD_15 | ATTRIBUTE_KIND_DEFINITION | (CONSTRAINTS_LIST) | AKD_14 |
| P | RD_15 | ATTRIBUTE_KIND_DEFINITION | (DAY) | AKD_15 |
| P | RD_15 | ATTRIBUTE_KIND_DEFINITION | (DEFINITION_KIND) | AKD_16 |
| P | RD_15 | ATTRIBUTE_KIND_DEFINITION | (DEVICE_KIND) | AKD_17 |
| P | RD_15 | ATTRIBUTE_KIND_DEFINITION | (DIGITS) | AKD_18 |


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| P | RD_15 | ATTRIBUTE_KIND_DEFINITION | (DIRECTION) | AKD_19 |
| P | RD_15 | ATTRIBUTE_KIND_DEFINITION | (ELEMENT_SIZE) | AKD_20 |
| P | RD_15 | ATTRIBUTE_KIND_DEFINITION | (EMANATES_BOUND) | AKD_21 |
| P | RD_15 | ATTRIBUTE_KIND_DEFINITION | (FILE_KIND) | AKD_22 |
| P | RD_15 | ATTRIBUTE_KIND_DEFINITION | (FILE_SIZE) | AKD_23 |
| P | RD_15 | ATTRIBUTE_KIND_DEFINITION | (FRAME_IO_PRIORITY_LEVEL) | AKD_24 |
| P | RD_15 | ATTRIBUTE_KIND_DEFINITION | (FRAME_LENGTH) | AKD_25 |
| P | RD_15 | ATTRIBUTE_KIND_DEFINITION | (GRANT) | AKD_26 |
| P | RD_15 | ATTRIBUTE_KIND_DEFINITION | (GRANT_ITEM) | AKD_27 |
| P | RD_15 | ATTRIBUTE_KIND_DEFINITION | (HIERARCHICAL_-CLASSIFICATION) | AKD_28 |
| P | RD_15 | ATTRIBUTE_KIND_DEFINITION | (IDENTIFIER_LIST) | AKD_29 |
| P | RD_15 | ATTRIBUTE_KIND_DEFINITION | (INTERPRETATION_KIND) | AKD_30 |
| P | RD_15 | ATTRIBUTE_KIND_DEFINITION | (INTRODUCED_ATTRIBUTE) | AKD_31 |
| P | RD_15 | ATTRIBUTE_KIND_DEFINITION | (INTRODUCED_ATTRIBUTE_-DESCRIPTION) | AKD_32 |
| P | RD_15 | ATTRIBUTE_KIND_DEFINITION | (ITEM_VALUE) | AKD_33 |
| P | RD_15 | ATTRIBUTE_KIND_DEFINITION | (KEY_VALUE_DESIGNATOR) | AKD_34 |
| P | RD_15 | ATTRIBUTE_KIND_DEFINITION | (KEY_VALUE_LIST) | AKD_35 |
| P | RD_15 | ATTRIBUTE_KIND_DEFINITION | (KIND) | AKD_36 |
| P | RD_15 | ATTRIBUTE_KIND_DEFINITION | (LIST_SIZE) | AKD_37 |
| P | RD_15 | ATTRIBUTE_KIND_DEFINITION | (MAXIMUM_LINE_LENGTH) | AKD_38 |
| P | RD_15 | ATTRIBUTE_KIND_DEFINITION | (MAXIMUM_PAGE_LENGTH) | AKD_39 |
| P | RD_15 | ATTRIBUTE_KIND_DEFINITION | (MONTH) | AKD_40 |
| P | RD_15 | ATTRIBUTE_KIND_DEFINITION | (NAMING) | AKD_41 |
| P | RD_15 | ATTRIBUTE_KIND_DEFINITION | (NON_HIERARCHICAL_-CATEGORY) | AKD_42 |
| P | RD_15 | ATTRIBUTE_KIND_DEFINITION | (NON_HIERARCHICAL_-CATEGORY_LIST) | AKD_43 |
| P | RD_15 | ATTRIBUTE_KIND_DEFINITION | (OPEN_NODE_HANDLE_COUNT) | AKD_44 |
| P | RD_15 | ATTRIBUTE_KIND_DEFINITION | (OUTGOING_FRAME_LENGTH_-RANGE) | AKD_45 |
| P | RD_15 | ATTRIBUTE_KIND_DEFINITION | (PARAMETERS_KINDS) | AKD_46 |
| P | RD_15 | ATTRIBUTE_KIND_DEFINITION | (PATTERN_LIST) | AKD_47 |
| P | RD_15 | ATTRIBUTE_KIND_DEFINITION | (PROCESS_PRIORITY) | AKD_48 |
| P | RD_15 | ATTRIBUTE_KIND_DEFINITION | (PROCESS_STATUS_KIND) | AKD_49 |
| P | RD_15 | ATTRIBUTE_KIND_DEFINITION | (PROTOCOL) | AKD_50 |
| P | RD_15 | ATTRIBUTE_KIND_DEFINITION | (PROTOCOL_LIST) | AKD_51 |
| P | RD_15 | ATTRIBUTE_KIND_DEFINITION | (RANGE_CONSTRAINT) | AKD_52 |
| P | RD_15 | ATTRIBUTE_KIND_DEFINITION | (RELATIONSHIP_ATTRIBUTE_-DESIGNATOR) | AKD_53 |
| P | RD_15 | ATTRIBUTE_KIND_DEFINITION | (RELATIONSHIP_KEY_LIST) | AKD_54 |
| P | RD_15 | ATTRIBUTE_KIND_DEFINITION | (RELATIONSHIP_KIND) | AKD_55 |
| P | RD_15 | ATTRIBUTE_KIND_DEFINITION | (RESULTING_RIGHTS_LIST) | AKD_56 |
| P | RD_15 | ATTRIBUTE_KIND_DEFINITION | (RESULTS) | AKD_57 |
| P | RD_15 | ATTRIBUTE_KIND_DEFINITION | (SMALL_DURATION_MULTIPLE) | AKD_58 |
| P | RD_15 | ATTRIBUTE_KIND_DEFINITION | (STRUCTURAL_KIND) | AKD_59 |
| P | RD_15 | ATTRIBUTE_KIND_DEFINITION | (TIME) | AKD_60 |
| P | RD_15 | ATTRIBUTE_KIND_DEFINITION | (UNCONSTRAINED_-IDENTIFIER) | AKD_61 |
| P | RD_15 | ATTRIBUTE_KIND_DEFINITION | (UNCONSTRAINED_LIST) | AKD_62 |


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| P | RD_15 | ATTRIBUTE_KIND_DEFINITION | (UNCONSTRAINED_STRING) | AKD_63 |
| P | RD_15 | ATTRIBUTE_KIND_DEFINITION | (UNIQUE_IDENTIFIER) | AKD_64 |
| P | RD_15 | ATTRIBUTE_KIND_DEFINITION | (VALUE_TYPE) | AKD_65 |
| P | RD_15 | ATTRIBUTE_KIND_DEFINITION | (YEAR) | AKD_66 |
| P | RD_90 | NODE_KIND_DEFINITION | (ATTRIBUTE_DEFINITION) | NKD_1 |
| P | RD_90 | NODE_KIND_DEFINITION | (ATTRIBUTE_KIND_-DEFINITION) | NKD_2 |
| P | RD_90 | NODE_KIND_DEFINITION | (BASIC) | NKD_3 |
| P | RD_90 | NODE_KIND_DEFINITION | (CHANNEL) | NKD_4 |
| P | RD_90 | NODE_KIND_DEFINITION | (CLIENT) | NKD_5 |
| P | RD_90 | NODE_KIND_DEFINITION | (COMPOSITE_LIST_-ATTRIBUTE_KIND_DEFINITION) | NKD_6 |
| P | RD_90 | NODE_KIND_DEFINITION | (DEFINITION) | NKD_7 |
| P | RD_90 | NODE_KIND_DEFINITION | (DEFINITIONS) | NKD_8 |
| P | RD_90 | NODE_KIND_DEFINITION | (DEVICE) | NKD_9 |
| P | RD_90 | NODE_KIND_DEFINITION | (EXECUTABLE_IMAGE) | NKD_10 |
| P | RD_90 | NODE_KIND_DEFINITION | (FILE) | NKD_11 |
| P | RD_90 | NODE_KIND_DEFINITION | (FLOAT_ATTRIBUTE_KIND_-DEFINITION) | NKD_12 |
| P | RD_90 | NODE_KIND_DEFINITION | (GATEWAY) | NKD_13 |
| P | RD_90 | NODE_KIND_DEFINITION | (GROUP) | NKD_14 |
| P | RD_90 | NODE_KIND_DEFINITION | (IDENTIFIER_ATTRIBUTE_-KIND_DEFINITION) | NKD_15 |
| P | RD_90 | NODE_KIND_DEFINITION | (INTEGER_ATTRIBUTE_KIND_-DEFINITION) | NKD_16 |
| P | RD_90 | NODE_KIND_DEFINITION | (INTERPRETATION) | NKD_17 |
| P | RD_90 | NODE_KIND_DEFINITION | (INTRODUCED_NODE) | NKD_18 |
| P | RD_90 | NODE_KIND_DEFINITION | (LIST_ATTRIBUTE_KIND_-DEFINITION) | NKD_19 |
| P | RD_90 | NODE_KIND_DEFINITION | (MAGNETIC_TAPE) | NKD_20 |
| P | RD_90 | NODE_KIND_DEFINITION | (MEDIUM) | NKD_21 |
| P | RD_90 | NODE_KIND_DEFINITION | (MONITOR) | NKD_22 |
| P | RD_90 | NODE_KIND_DEFINITION | (NAMED_COMPOSITE_LIST_-AKD) | NKD_23 |
| P | RD_90 | NODE_KIND_DEFINITION | (NAMED_ORDERED_-COMPOSITE_LIST_AKD) | NKD_24 |
| P | RD_90 | NODE_KIND_DEFINITION | (NODE_KIND_DEFINITION) | NKD_25 |
| P | RD_90 | NODE_KIND_DEFINITION | (NODE_KIND_-INTERPRETATION) | NKD_26 |
| P | RD_90 | NODE_KIND_DEFINITION | (ORDERED_COMPOSITE_LIST_-AKD) | NKD_27 |
| P | RD_90 | NODE_KIND_DEFINITION | (PROCESS) | NKD_28 |
| P | RD_90 | NODE_KIND_DEFINITION | (PROCESSOR) | NKD_29 |
| P | RD_90 | NODE_KIND_DEFINITION | (RELATION_DEFINITION) | NKD_30 |
| P | RD_90 | NODE_KIND_DEFINITION | (RELATION_INTERPRETATION) | NKD_31 |
| P | RD_90 | NODE_KIND_DEFINITION | (RESOURCE) | NKD_32 |
| P | RD_90 | NODE_KIND_DEFINITION | (RESOURCE_GROUP) | NKD_33 |
| P | RD_90 | NODE_KIND_DEFINITION | (SECONDARY_STORAGE) | NKD_34 |
| P | RD_90 | NODE_KIND_DEFINITION | (STRING_ATTRIBUTE_KIND_-DEFINITION) | NKD_35 |
| P | RD_90 | NODE_KIND_DEFINITION | (STRUCTURAL) | NKD_36 |
| P | RD_90 | NODE_KIND_DEFINITION | (SYSTEM_LEVEL) | NKD_37 |



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| P | RD_90 | NODE_KIND_DEFINITION | (TERMINAL) | NKD_38 |
| P | RD_90 | NODE_KIND_DEFINITION | (UNIFORM_LIST_ATTRIBUTE_-KIND_DEFINITION) | NKD_39 |
| P | RD_90 | NODE_KIND_DEFINITION | (USER) | NKD_40 |
| P | RD_90 | NODE_KIND_DEFINITION | (VARIANT_ATTRIBUTE_KIND_-DEFINITION) | NKD_41 |
| P | RD_90 | NODE_KIND_DEFINITION | (VIEW) | NKD_42 |
| S | RD_93 | PARENT | () | N_1 |
| P | RD_102 | RELATION_DEFINITION | (ACCESS) | RD_1 |
| P | RD_102 | RELATION_DEFINITION | (ADOPTED_ROLE) | RD_2 |
| P | RD_102 | RELATION_DEFINITION | (ADOPTED_VIEW) | RD_3 |
| P | RD_102 | RELATION_DEFINITION | (ALLOCATABLE_DEVICE) | RD_4 |
| P | RD_102 | RELATION_DEFINITION | (ALLOCATABLE_GATEWAY) | RD_5 |
| P | RD_102 | RELATION_DEFINITION | (ALLOCATABLE_MEDIUM) | RD_6 |
| P | RD_102 | RELATION_DEFINITION | (ALLOCATABLE_PROCESSOR) | RD_7 |
| P | RD_102 | RELATION_DEFINITION | (ALTERNATIVE) | RD_8 |
| P | RD_102 | RELATION_DEFINITION | (ALTERNATIVE_OF) | RD_9 |
| P | RD_102 | RELATION_DEFINITION | (ATTRIBUTE) | RD_10 |
| P | RD_102 | RELATION_DEFINITION | (ATTRIBUTE_DEFINITION) | RD_11 |
| P | RD_102 | RELATION_DEFINITION | (ATTRIBUTE_DEFINITION_-NAME) | RD_12 |
| P | RD_102 | RELATION_DEFINITION | (ATTRIBUTE_DEFINITION_-PARENT) | RD_13 |
| P | RD_102 | RELATION_DEFINITION | (ATTRIBUTE_KIND) | RD_14 |
| P | RD_102 | RELATION_DEFINITION | (ATTRIBUTE_KIND_-DEFINITION) | RD_15 |
| P | RD_102 | RELATION_DEFINITION | (ATTRIBUTE_KIND_-DEFINITION_PARENT) | RD_16 |
| P | RD_102 | RELATION_DEFINITION | (ATTRIBUTE_KIND_NAME) | RD_17 |
| P | RD_102 | RELATION_DEFINITION | (ATTRIBUTE_KIND_OF) | RD_18 |
| P | RD_102 | RELATION_DEFINITION | (ATTRIBUTE_MONITOR) | RD_19 |
| P | RD_102 | RELATION_DEFINITION | (ATTRIBUTE_MONITOR_-PARENT) | RD_20 |
| P | RD_102 | RELATION_DEFINITION | (ATTRIBUTE_NAME) | RD_21 |
| P | RD_102 | RELATION_DEFINITION | (ATTRIBUTE_OF) | RD_22 |
| P | RD_102 | RELATION_DEFINITION | (BASIC) | RD_23 |
| P | RD_102 | RELATION_DEFINITION | (CHANNEL) | RD_24 |
| P | RD_102 | RELATION_DEFINITION | (CHANNEL_PARENT) | RD_25 |
| P | RD_102 | RELATION_DEFINITION | (CLIENT) | RD_26 |
| P | RD_102 | RELATION_DEFINITION | (COMPONENT) | RD_27 |
| P | RD_102 | RELATION_DEFINITION | (COMPONENT_NAMED) | RD_28 |
| P | RD_102 | RELATION_DEFINITION | (COMPONENT_NAMED_-ORDERED) | RD_29 |
| P | RD_102 | RELATION_DEFINITION | (COMPONENT_OF) | RD_30 |
| P | RD_102 | RELATION_DEFINITION | (COMPONENT_OF_NAMED) | RD_31 |
| P | RD_102 | RELATION_DEFINITION | (COMPONENT_OF_NAMED_-ORDERED) | RD_32 |
| P | RD_102 | RELATION_DEFINITION | (COMPONENT_OF_ORDERED) | RD_33 |
| P | RD_102 | RELATION_DEFINITION | (COMPONENT_OF_UNIFORM_-LIST) | RD_34 |
| P | RD_102 | RELATION_DEFINITION | (COMPONENT_ORDERED) | RD_35 |


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| P | RD_102 | RELATION_DEFINITION | (COMPONENT_UNIFORM_LIST) | RD_36 |
| P | RD_102 | RELATION_DEFINITION | (COPY_OF) | RD_37 |
| P | RD_102 | RELATION_DEFINITION | (CURRENT_JOB) | RD_38 |
| P | RD_102 | RELATION_DEFINITION | (CURRENT_NODE) | RD_39 |
| P | RD_102 | RELATION_DEFINITION | (CURRENT_PROCESSOR) | RD_40 |
| P | RD_102 | RELATION_DEFINITION | (CURRENT_USER) | RD_41 |
| P | RD_102 | RELATION_DEFINITION | (DEFAULT_MEDIUM) | RD_42 |
| P | RD_102 | RELATION_DEFINITION | (DEFAULT_PROCESSOR) | RD_43 |
| P | RD_102 | RELATION_DEFINITION | (DEFAULT_PROCESS_NODE_-KIND) | RD_44 |
| P | RD_102 | RELATION_DEFINITION | (DEFAULT_ROLE) | RD_45 |
| P | RD_102 | RELATION_DEFINITION | (DEFINITION) | RD_46 |
| P | RD_102 | RELATION_DEFINITION | (DEFINITIONS_PARENT) | RD_47 |
| P | RD_102 | RELATION_DEFINITION | (DEFINITIONS_PRIMARY) | RD_48 |
| P | RD_102 | RELATION_DEFINITION | (DEFINITIONS_SECONDARY) | RD_49 |
| P | RD_102 | RELATION_DEFINITION | (DENY) | RD_50 |
| P | RD_102 | RELATION_DEFINITION | (DEVICE_PARENT) | RD_51 |
| P | RD_102 | RELATION_DEFINITION | (DEVICE_PRIMARY) | RD_52 |
| P | RD_102 | RELATION_DEFINITION | (DEVICE_SECONDARY) | RD_53 |
| P | RD_102 | RELATION_DEFINITION | (EMANATES) | RD_54 |
| P | RD_102 | RELATION_DEFINITION | (EMANATES_FROM) | RD_55 |
| P | RD_102 | RELATION_DEFINITION | (EXECUTABLE_IMAGE) | RD_56 |
| P | RD_102 | RELATION_DEFINITION | (GATEWAY_PARENT) | RD_57 |
| P | RD_102 | RELATION_DEFINITION | (GATEWAY_PRIMARY) | RD_58 |
| P | RD_102 | RELATION_DEFINITION | (GATEWAY_SECONDARY) | RD_59 |
| P | RD_102 | RELATION_DEFINITION | (GENERALIZATION_OF_NKD) | RD_60 |
| P | RD_102 | RELATION_DEFINITION | (GENERALIZATION_OF_RD) | RD_61 |
| P | RD_102 | RELATION_DEFINITION | (GROUP_PARENT) | RD_62 |
| P | RD_102 | RELATION_DEFINITION | (GROUP_PRIMARY) | RD_63 |
| P | RD_102 | RELATION_DEFINITION | (GROUP_SECONDARY) | RD_64 |
| P | RD_102 | RELATION_DEFINITION | (HANDLER_ATTRIBUTE) | RD_65 |
| P | RD_102 | RELATION_DEFINITION | (HANDLER_ENVIRONMENT_-NODE) | RD_66 |
| P | RD_102 | RELATION_DEFINITION | (HANDLER_ERROR_FILE) | RD_67 |
| P | RD_102 | RELATION_DEFINITION | (HANDLER_IMAGE) | RD_68 |
| P | RD_102 | RELATION_DEFINITION | (HANDLER_INPUT_FILE) | RD_69 |
| P | RD_102 | RELATION_DEFINITION | (HANDLER_MEDIUM_-RESOURCE) | RD_70 |
| P | RD_102 | RELATION_DEFINITION | (HANDLER_OUTPUT_FILE) | RD_71 |
| P | RD_102 | RELATION_DEFINITION | (HANDLER_PROCESS) | RD_72 |
| P | RD_102 | RELATION_DEFINITION | (HANDLER_PROCESS_NODE_-KIND) | RD_73 |
| P | RD_102 | RELATION_DEFINITION | (HANDLER_PROCESS_OF) | RD_74 |
| P | RD_102 | RELATION_DEFINITION | (HANDLER_PROCESSOR_-RESOURCE) | RD_75 |
| P | RD_102 | RELATION_DEFINITION | (INTRODUCED_NODE) | RD_76 |
| P | RD_102 | RELATION_DEFINITION | (INTRODUCED_NODE_PARENT) | RD_77 |
| P | RD_102 | RELATION_DEFINITION | (INTRODUCED_RELATION) | RD_78 |
| P | RD_102 | RELATION_DEFINITION | (JOB) | RD_79 |
| P | RD_102 | RELATION_DEFINITION | (JOB_PARENT) | RD_80 |
| P | RD_102 | RELATION_DEFINITION | (KEY) | RD_81 |



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| P | RD_102 | RELATION_DEFINITION | (KEY_NAME) | RD_82 |
| P | RD_102 | RELATION_DEFINITION | (KEY_OF) | RD_83 |
| P | RD_102 | RELATION_DEFINITION | (MEDIUM_PARENT) | RD_84 |
| P | RD_102 | RELATION_DEFINITION | (MEDIUM_PRIMARY) | RD_85 |
| P | RD_102 | RELATION_DEFINITION | (MEDIUM_SECONDARY) | RD_86 |
| P | RD_102 | RELATION_DEFINITION | (MONITORED_BY) | RD_87 |
| P | RD_102 | RELATION_DEFINITION | (MONITORS) | RD_88 |
| P | RD_102 | RELATION_DEFINITION | (NODE_KIND) | RD_89 |
| P | RD_102 | RELATION_DEFINITION | (NODE_KIND_DEFINITION) | RD_90 |
| P | RD_102 | RELATION_DEFINITION | (NODE_KIND_DEFINITION_-PARENT) | RD_91 |
| P | RD_102 | RELATION_DEFINITION | (NODE_KIND_PARENT) | RD_92 |
| P | RD_102 | RELATION_DEFINITION | (PARENT) | RD_93 |
| P | RD_102 | RELATION_DEFINITION | (PERMANENT_MEMBER) | RD_94 |
| P | RD_102 | RELATION_DEFINITION | (PERMANENT_MEMBER_-PARENT) | RD_95 |
| P | RD_102 | RELATION_DEFINITION | (PORT) | RD_96 |
| P | RD_102 | RELATION_DEFINITION | (POTENTIAL_MEMBER) | RD_97 |
| P | RD_102 | RELATION_DEFINITION | (PROCESSOR_PARENT) | RD_98 |
| P | RD_102 | RELATION_DEFINITION | (PROCESSOR_PRIMARY) | RD_99 |
| P | RD_102 | RELATION_DEFINITION | (PROCESSOR_SECONDARY) | RD_100 |
| P | RD_102 | RELATION_DEFINITION | (RELATION) | RD_101 |
| P | RD_102 | RELATION_DEFINITION | (RELATION_DEFINITION) | RD_102 |
| P | RD_102 | RELATION_DEFINITION | (RELATION_DEFINITION_-PARENT) | RD_103 |
| P | RD_102 | RELATION_DEFINITION | (RELATION_NAME) | RD_104 |
| P | RD_102 | RELATION_DEFINITION | (RELATION_PARENT) | RD_105 |
| P | RD_102 | RELATION_DEFINITION | (RESOURCE_GROUP_PARENT) | RD_106 |
| P | RD_102 | RELATION_DEFINITION | (RESOURCE_GROUP_PRIMARY) | RD_107 |
| P | RD_102 | RELATION_DEFINITION | (RESOURCE_GROUP_-SECONDARY) | RD_108 |
| P | RD_102 | RELATION_DEFINITION | (REVERSE_LINK) | RD_109 |
| P | RD_102 | RELATION_DEFINITION | (REVERSE_LINK_-BIDIRECTIONAL) | RD_110 |
| P | RD_102 | RELATION_DEFINITION | (SPECIALIZATION_OF_NKD) | RD_111 |
| P | RD_102 | RELATION_DEFINITION | (SPECIALIZATION_OF_RD) | RD_112 |
| P | RD_102 | RELATION_DEFINITION | (STANDARD_ERROR) | RD_113 |
| P | RD_102 | RELATION_DEFINITION | (STANDARD_INPUT) | RD_114 |
| P | RD_102 | RELATION_DEFINITION | (STANDARD_OUTPUT) | RD_115 |
| P | RD_102 | RELATION_DEFINITION | (TERMINATES) | RD_116 |
| P | RD_102 | RELATION_DEFINITION | (TERMINATES_AT) | RD_117 |
| P | RD_102 | RELATION_DEFINITION | (USER_PARENT) | RD_118 |
| P | RD_102 | RELATION_DEFINITION | (USER_PRIMARY) | RD_119 |
| P | RD_102 | RELATION_DEFINITION | (USER_SECONDARY) | RD_120 |
| P | RD_102 | RELATION_DEFINITION | (VIEW_PARENT) | RD_121 |
| P | RD_102 | RELATION_DEFINITION | (VIEW_PRIMARY) | RD_122 |
| P | RD_102 | RELATION_DEFINITION | (VIEW_SECONDARY) | RD_123 |



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

### 20.1.3 'MEDIUM( TRANSPARENT_MEDIUM_RESOURCE )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| N_3 | TRANSPARENT_MEDIUM_-RESOURCE | NKD_21 | MEDIUM |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("Represents all media which cannot be explicitly assigned by processes.") |
| AVAILABLE | AD_6 | (TRUE) |
| STRUCTURAL_KIND | AD_76 | (MEDIUM) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_84 | PARENT | () | N_1 |

### 20.1.4 'PROCESSOR( TRANSPARENT_PROCESSOR_RESOURCE )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| N_4 | TRANSPARENT_PROCESSOR_-RESOURCE | NKD_29 | PROCESSOR |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("Represents all processors which cannot be explicitly assigned by processes.") |
| AVAILABLE | AD_6 | (TRUE) |
| STRUCTURAL_KIND | AD_76 | (PROCESSOR) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_98 | PARENT | () | N_1 |

### 20.1.5 'RESOURCE_GROUP( TRANSPARENT_RESOURCES )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| N_5 | TRANSPARENT_RESOURCES | NKD_33 | RESOURCE_GROUP |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("Represents the group of resources which cannot be explicitly assigned by processes.") |
| STRUCTURAL_KIND | AD_76 | (RESOURCE_GROUP) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| P | RD_86 | MEDIUM | (TRANSPARENT_MEDIUM_-RESOURCE) | N_3 |
| S | RD_106 | PARENT | () | N_1 |
| P | RD_100 | PROCESSOR | (TRANSPARENT_PROCESSOR_-RESOURCE) | N_4 |

### 20.1.6 'VIEW( PREDEFINED )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| N_6 | Predefined VIEW node | NKD_42 | VIEW |



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("Node which is the parent of all the predefined interpretation nodes.") |
| STRUCTURAL_KIND | AD_76 | (VIEW) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_12 | ATTRIBUTE | (ALLOWED_PROTOCOLS) | AD_1 |
| S | RD_12 | ATTRIBUTE | (ALLOWED_USAGE) | AD_2 |
| S | RD_12 | ATTRIBUTE | (ANNOTATION) | AD_3 |
| S | RD_12 | ATTRIBUTE | (ATTRIBUTE_VALUE) | AD_4 |
| S | RD_12 | ATTRIBUTE | (ATTRIBUTES_MONITORED) | AD_5 |
| S | RD_12 | ATTRIBUTE | (AVAILABLE) | AD_6 |
| S | RD_12 | ATTRIBUTE | (CAIS_CONTROLLED) | AD_7 |
| S | RD_12 | ATTRIBUTE | (CAIS_DETERMINED) | AD_8 |
| S | RD_12 | ATTRIBUTE | (CAIS_KEY) | AD_9 |
| S | RD_12 | ATTRIBUTE | (CONSTRAINTS) | AD_10 |
| S | RD_12 | ATTRIBUTE | (CONTROLLING_PROTOCOL) | AD_11 |
| S | RD_12 | ATTRIBUTE | (CURRENT_FILE_SIZE) | AD_12 |
| S | RD_12 | ATTRIBUTE | (CURRENT_STATUS) | AD_13 |
| S | RD_12 | ATTRIBUTE | (DEFAULT_VALUE) | AD_14 |
| S | RD_12 | ATTRIBUTE | (DEFINITION_KIND) | AD_15 |
| S | RD_12 | ATTRIBUTE | (DELETABLE) | AD_16 |
| S | RD_12 | ATTRIBUTE | (DEVICE_KIND) | AD_17 |
| S | RD_12 | ATTRIBUTE | (DIGITS) | AD_18 |
| S | RD_12 | ATTRIBUTE | (DIRECTION) | AD_19 |
| S | RD_12 | ATTRIBUTE | (ELEMENT_SIZE) | AD_20 |
| S | RD_12 | ATTRIBUTE | (ENUMERATED) | AD_21 |
| S | RD_12 | ATTRIBUTE | (FILE_KIND) | AD_22 |
| S | RD_12 | ATTRIBUTE | (GENERIC_KEY) | AD_23 |
| S | RD_12 | ATTRIBUTE | (GRANT) | AD_24 |
| S | RD_12 | ATTRIBUTE | (HANDLER_AS_A_-TRANSACTION) | AD_25 |
| S | RD_12 | ATTRIBUTE | (HANDLER_DELETE_WHEN_-TERMINATED) | AD_26 |
| S | RD_12 | ATTRIBUTE | (HANDLER_DISCRETIONARY_-ACCESS) | AD_27 |
| S | RD_12 | ATTRIBUTE | (HANDLER_INPUT_-PARAMETERS) | AD_28 |
| S | RD_12 | ATTRIBUTE | (HANDLER_KEY) | AD_29 |
| S | RD_12 | ATTRIBUTE | (HANDLER_MANDATORY_-ACCESS) | AD_30 |
| S | RD_12 | ATTRIBUTE | (HANDLER_PRIORITY) | AD_31 |
| S | RD_12 | ATTRIBUTE | (HIGHEST_CLASSIFICATION) | AD_32 |
| S | RD_12 | ATTRIBUTE | (IMPORTED_DEFINITION_UID) | AD_33 |
| S | RD_12 | ATTRIBUTE | (IMPORTED_NAME) | AD_34 |
| S | RD_12 | ATTRIBUTE | (IMPORTED_RELATIONSHIP_-KIND) | AD_35 |
| S | RD_12 | ATTRIBUTE | (IMPORTED_TARGET_UID) | AD_36 |
| S | RD_12 | ATTRIBUTE | (IMPORTED_UID) | AD_37 |
| S | RD_12 | ATTRIBUTE | (INCOMING_FRAME_LENGTH) | AD_38 |
| S | RD_12 | ATTRIBUTE | (INHERITABLE) | AD_39 |
| S | RD_12 | ATTRIBUTE | (INTERPRETATION_KIND) | AD_40 |



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_12 | ATTRIBUTE | (INTRODUCED_KEY_-ATTRIBUTES) | AD_41 |
| S | RD_12 | ATTRIBUTE | (INTRODUCED_NODE_-ATTRIBUTES) | AD_42 |
| S | RD_12 | ATTRIBUTE | (INTRODUCED_-RELATIONSHIP_ATTRIBUTES) | AD_43 |
| S | RD_12 | ATTRIBUTE | (IO_UNIT_COUNT) | AD_44 |
| S | RD_12 | ATTRIBUTE | (KIND) | AD_45 |
| S | RD_12 | ATTRIBUTE | (LOWER_BOUND_FLOAT) | AD_46 |
| S | RD_12 | ATTRIBUTE | (LOWER_BOUND_INTEGER) | AD_47 |
| S | RD_12 | ATTRIBUTE | (LOWEST_CLASSIFICATION) | AD_48 |
| S | RD_12 | ATTRIBUTE | (MACHINE_TIME) | AD_49 |
| S | RD_12 | ATTRIBUTE | (MAXIMUM_FILE_SIZE) | AD_50 |
| S | RD_12 | ATTRIBUTE | (MAXIMUM_INSTANCES) | AD_51 |
| S | RD_12 | ATTRIBUTE | (MAXIMUM_LINE_LENGTH) | AD_52 |
| S | RD_12 | ATTRIBUTE | (MAXIMUM_PAGE_LENGTH) | AD_53 |
| S | RD_12 | ATTRIBUTE | (MAXIMUM_SIZE) | AD_54 |
| S | RD_12 | ATTRIBUTE | (MINIMUM_INSTANCES) | AD_55 |
| S | RD_12 | ATTRIBUTE | (MINIMUM_SIZE) | AD_56 |
| S | RD_12 | ATTRIBUTE | (MODIFIABLE) | AD_57 |
| S | RD_12 | ATTRIBUTE | (NAMING) | AD_59 |
| S | RD_12 | ATTRIBUTE | (NAME_KEY) | AD_58 |
| S | RD_12 | ATTRIBUTE | (NODE_KIND) | AD_60 |
| S | RD_12 | ATTRIBUTE | (OBJECT_CLASSIFICATION) | AD_61 |
| S | RD_12 | ATTRIBUTE | (OPEN_NODE_HANDLE_COUNT) | AD_62 |
| S | RD_12 | ATTRIBUTE | (OPTIONAL) | AD_63 |
| S | RD_12 | ATTRIBUTE | (ORDERED) | AD_64 |
| S | RD_12 | ATTRIBUTE | (OUTGOING_FRAME_LENGTH_-RANGE) | AD_65 |
| S | RD_12 | ATTRIBUTE | (PARAMETERS) | AD_66 |
| S | RD_12 | ATTRIBUTE | (PATTERNS) | AD_67 |
| S | RD_12 | ATTRIBUTE | (POSITION_KEY) | AD_68 |
| S | RD_12 | ATTRIBUTE | (PRIORITY) | AD_69 |
| S | RD_12 | ATTRIBUTE | (PRIORITY_LEVEL) | AD_70 |
| S | RD_12 | ATTRIBUTE | (PROCESS_SIZE) | AD_71 |
| S | RD_12 | ATTRIBUTE | (QUEUE_LENGTH) | AD_72 |
| S | RD_12 | ATTRIBUTE | (RESULTS) | AD_73 |
| S | RD_12 | ATTRIBUTE | (SUBJECT_CLASSIFICATION) | AD_74 |
| S | RD_12 | ATTRIBUTE | (STATE) | AD_75 |
| S | RD_12 | ATTRIBUTE | (STRUCTURAL_KIND) | AD_76 |
| S | RD_12 | ATTRIBUTE | (TIME_ATTRIBUTE_WRITTEN) | AD_77 |
| S | RD_12 | ATTRIBUTE | (TIME_CONTENTS_WRITTEN) | AD_78 |
| S | RD_12 | ATTRIBUTE | (TIME_CREATED) | AD_79 |
| S | RD_12 | ATTRIBUTE | (TIME_FINISHED) | AD_80 |
| S | RD_12 | ATTRIBUTE | (TIME_RELATIONSHIP_-WRITTEN) | AD_81 |
| S | RD_12 | ATTRIBUTE | (TIME_STARTED) | AD_82 |
| S | RD_12 | ATTRIBUTE | (UNIQUE_IDENTIFIER) | AD_83 |
| S | RD_12 | ATTRIBUTE | (UNIQUE_KEYS) | AD_84 |
| S | RD_12 | ATTRIBUTE | (UPPER_BOUND_FLOAT) | AD_85 |
| S | RD_12 | ATTRIBUTE | (UPPER_BOUND_INTEGER) | AD_86 |


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_12 | ATTRIBUTE | (VALUE_TYPE) | AD_87 |
| S | RD_17 | ATTRIBUTE_KIND | (ALTERNATIVE_CONSTRAINT) | AKD_1 |
| S | RD_17 | ATTRIBUTE_KIND | (ANNOTATION_KINDS) | AKD_2 |
| S | RD_17 | ATTRIBUTE_KIND | (ATTRIBUTE_DESIGNATOR) | AKD_3 |
| S | RD_17 | ATTRIBUTE_KIND | (ATTRIBUTES_MONITORED) | AKD_4 |
| S | RD_17 | ATTRIBUTE_KIND | (BOOLEAN) | AKD_5 |
| S | RD_17 | ATTRIBUTE_KIND | (CAIS_FLOAT) | AKD_6 |
| S | RD_17 | ATTRIBUTE_KIND | (CAIS_INTEGER) | AKD_7 |
| S | RD_17 | ATTRIBUTE_KIND | (CAIS_NATURAL) | AKD_8 |
| S | RD_17 | ATTRIBUTE_KIND | (CAIS_POSITIVE) | AKD_9 |
| S | RD_17 | ATTRIBUTE_KIND | (CHANNEL_STATE) | AKD_10 |
| S | RD_17 | ATTRIBUTE_KIND | (CLASSIFICATION) | AKD_11 |
| S | RD_17 | ATTRIBUTE_KIND | (COMPONENT_CONSTRAINT) | AKD_12 |
| S | RD_17 | ATTRIBUTE_KIND | (COMPOSITE_LIST_-COMPONENT_CONSTRAINT) | AKD_13 |
| S | RD_17 | ATTRIBUTE_KIND | (CONSTRAINTS_LIST) | AKD_14 |
| S | RD_17 | ATTRIBUTE_KIND | (DAY) | AKD_15 |
| S | RD_17 | ATTRIBUTE_KIND | (DEFINITION_KIND) | AKD_16 |
| S | RD_17 | ATTRIBUTE_KIND | (DEVICE_KIND) | AKD_17 |
| S | RD_17 | ATTRIBUTE_KIND | (DIGITS) | AKD_18 |
| S | RD_17 | ATTRIBUTE_KIND | (DIRECTION) | AKD_19 |
| S | RD_17 | ATTRIBUTE_KIND | (ELEMENT_SIZE) | AKD_20 |
| S | RD_17 | ATTRIBUTE_KIND | (EMANATES_BOUND) | AKD_21 |
| S | RD_17 | ATTRIBUTE_KIND | (FILE_KIND) | AKD_22 |
| S | RD_17 | ATTRIBUTE_KIND | (FILE_SIZE) | AKD_23 |
| S | RD_17 | ATTRIBUTE_KIND | (FRAME_IO_PRIORITY_LEVEL) | AKD_24 |
| S | RD_17 | ATTRIBUTE_KIND | (FRAME_LENGTH) | AKD_25 |
| S | RD_17 | ATTRIBUTE_KIND | (GRANT) | AKD_26 |
| S | RD_17 | ATTRIBUTE_KIND | (GRANT_ITEM) | AKD_27 |
| S | RD_17 | ATTRIBUTE_KIND | (HIERARCHICAL_-CLASSIFICATION) | AKD_28 |
| S | RD_17 | ATTRIBUTE_KIND | (IDENTIFIER_LIST) | AKD_29 |
| S | RD_17 | ATTRIBUTE_KIND | (INTERPRETATION_KIND) | AKD_30 |
| S | RD_17 | ATTRIBUTE_KIND | (INTRODUCED_ATTRIBUTE) | AKD_31 |
| S | RD_17 | ATTRIBUTE_KIND | (INTRODUCED_ATTRIBUTE_-DESCRIPTION) | AKD_32 |
| S | RD_17 | ATTRIBUTE_KIND | (KEY_VALUE_DESIGNATOR) | AKD_34 |
| S | RD_17 | ATTRIBUTE_KIND | (KEY_VALUE_LIST) | AKD_35 |
| S | RD_17 | ATTRIBUTE_KIND | (KIND) | AKD_36 |
| S | RD_17 | ATTRIBUTE_KIND | (LIST_SIZE) | AKD_37 |
| S | RD_17 | ATTRIBUTE_KIND | (MAXIMUM_LINE_LENGTH) | AKD_38 |
| S | RD_17 | ATTRIBUTE_KIND | (MAXIMUM_PAGE_LENGTH) | AKD_39 |
| S | RD_17 | ATTRIBUTE_KIND | (MONTH) | AKD_40 |
| S | RD_17 | ATTRIBUTE_KIND | (NAMING) | AKD_41 |
| S | RD_17 | ATTRIBUTE_KIND | (NON_HIERARCHICAL_-CATEGORY) | AKD_42 |
| S | RD_17 | ATTRIBUTE_KIND | (NON_HIERARCHICAL_-CATEGORY_LIST) | AKD_43 |
| S | RD_17 | ATTRIBUTE_KIND | (OPEN_NODE_HANDLE_COUNT) | AKD_44 |
| S | RD_17 | ATTRIBUTE_KIND | (OUTGOING_FRAME_LENGTH_-RANGE) | AKD_45 |


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_17 | ATTRIBUTE_KIND | (PARAMETERS_KINDS) | AKD_46 |
| S | RD_17 | ATTRIBUTE_KIND | (PATTERN_LIST) | AKD_47 |
| S | RD_17 | ATTRIBUTE_KIND | (PROCESS_PRIORITY) | AKD_48 |
| S | RD_17 | ATTRIBUTE_KIND | (PROCESS_STATUS_KIND) | AKD_49 |
| S | RD_17 | ATTRIBUTE_KIND | (PROTOCOL) | AKD_50 |
| S | RD_17 | ATTRIBUTE_KIND | (PROTOCOL_LIST) | AKD_51 |
| S | RD_17 | ATTRIBUTE_KIND | (RANGE_CONSTRAINT) | AKD_52 |
| S | RD_17 | ATTRIBUTE_KIND | (RELATIONSHIP_ATTRIBUTE_-DESIGNATOR) | AKD_53 |
| S | RD_17 | ATTRIBUTE_KIND | (RELATIONSHIP_KEY_LIST) | AKD_54 |
| S | RD_17 | ATTRIBUTE_KIND | (RELATIONSHIP_KIND) | AKD_55 |
| S | RD_17 | ATTRIBUTE_KIND | (RESULTING_RIGHTS_LIST) | AKD_56 |
| S | RD_17 | ATTRIBUTE_KIND | (RESULTS) | AKD_57 |
| S | RD_17 | ATTRIBUTE_KIND | (SMALL_DURATION_MULTIPLE) | AKD_58 |
| S | RD_17 | ATTRIBUTE_KIND | (STRUCTURAL_KIND) | AKD_59 |
| S | RD_17 | ATTRIBUTE_KIND | (TIME) | AKD_60 |
| S | RD_17 | ATTRIBUTE_KIND | (UNCONSTRAINED_-IDENTIFIER) | AKD_61 |
| S | RD_17 | ATTRIBUTE_KIND | (UNCONSTRAINED_LIST) | AKD_62 |
| S | RD_17 | ATTRIBUTE_KIND | (UNCONSTRAINED_STRING) | AKD_63 |
| S | RD_17 | ATTRIBUTE_KIND | (UNIQUE_IDENTIFIER) | AKD_64 |
| S | RD_17 | ATTRIBUTE_KIND | (VALUE_TYPE) | AKD_65 |
| S | RD_17 | ATTRIBUTE_KIND | (YEAR) | AKD_66 |
| P | RD_89 | NODE_KIND | (ATTRIBUTE_DEFINITION) | NKI_1 |
| P | RD_89 | NODE_KIND | (ATTRIBUTE_KIND_-DEFINITION) | NKI_2 |
| P | RD_89 | NODE_KIND | (BASIC) | NKI_3 |
| P | RD_89 | NODE_KIND | (CHANNEL) | NKI_4 |
| P | RD_89 | NODE_KIND | (CLIENT) | NKI_5 |
| P | RD_89 | NODE_KIND | (COMPOSITE_LIST_-ATTRIBUTE_KIND_DEFINITION) | NKI_6 |
| P | RD_89 | NODE_KIND | (DEFINITION) | NKI_7 |
| P | RD_89 | NODE_KIND | (DEFINITIONS) | NKI_8 |
| P | RD_89 | NODE_KIND | (DEVICE) | NKI_9 |
| P | RD_89 | NODE_KIND | (EXECUTABLE_IMAGE) | NKI_10 |
| P | RD_89 | NODE_KIND | (FILE) | NKI_11 |
| P | RD_89 | NODE_KIND | (FLOAT_ATTRIBUTE_KIND_-DEFINITION) | NKI_12 |
| P | RD_89 | NODE_KIND | (GATEWAY) | NKI_13 |
| P | RD_89 | NODE_KIND | (GROUP) | NKI_14 |
| P | RD_89 | NODE_KIND | (IDENTIFIER_ATTRIBUTE_-KIND_DEFINITION) | NKI_15 |
| P | RD_89 | NODE_KIND | (INTEGER_ATTRIBUTE_KIND_-DEFINITION) | NKI_16 |
| P | RD_89 | NODE_KIND | (INTERPRETATION) | NKI_17 |
| P | RD_89 | NODE_KIND | (INTRODUCED_NODE) | NKI_18 |
| P | RD_89 | NODE_KIND | (LIST_ATTRIBUTE_KIND_-DEFINITION) | NKI_19 |
| P | RD_89 | NODE_KIND | (MAGNETIC_TAPE) | NKI_20 |
| P | RD_89 | NODE_KIND | (MEDIUM) | NKI_21 |
| P | RD_89 | NODE_KIND | (MONITOR) | NKI_22 |



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| P | RD_89 | NODE_KIND | (NAME_COMPOSITE_LIST_AKD) | NKI_23 |
| P | RD_89 | NODE_KIND | (NAMED_ORDERED_-COMPOSITE_LIST_AKD) | NKI_24 |
| P | RD_89 | NODE_KIND | (NODE_KIND_DEFINITION) | NKI_25 |
| P | RD_89 | NODE_KIND | (NODE_KIND_-INTERPRETATION) | NKI_26 |
| P | RD_89 | NODE_KIND | (ORDERED_COMPOSITE_LIST_-AKD) | NKI_27 |
| P | RD_89 | NODE_KIND | (PROCESS) | NKI_28 |
| P | RD_89 | NODE_KIND | (PROCESSOR) | NKI_29 |
| P | RD_89 | NODE_KIND | (RELATION_DEFINITION) | NKI_30 |
| P | RD_89 | NODE_KIND | (RELATION_INTERPRETATION) | NKI_31 |
| P | RD_89 | NODE_KIND | (RESOURCE) | NKI_32 |
| P | RD_89 | NODE_KIND | (RESOURCE_GROUP) | NKI_33 |
| P | RD_89 | NODE_KIND | (SECONDARY_STORAGE) | NKI_34 |
| P | RD_89 | NODE_KIND | (STRING_ATTRIBUTE_KIND_-DEFINITION) | NKI_35 |
| P | RD_89 | NODE_KIND | (STRUCTURAL) | NKI_36 |
| P | RD_89 | NODE_KIND | (SYSTEM_LEVEL) | NKI_37 |
| P | RD_89 | NODE_KIND | (TERMINAL) | NKI_38 |
| P | RD_89 | NODE_KIND | (UNIFORM_LIST_ATTRIBUTE_-KIND_DEFINITION) | NKI_39 |
| P | RD_89 | NODE_KIND | (USER) | NKI_40 |
| P | RD_89 | NODE_KIND | (VARIANT_ATTRIBUTE_KIND_-DEFINITION) | NKI_41 |
| P | RD_89 | NODE_KIND | (VIEW) | NKI_42 |
| S | RD_93 | PARENT | () | N_1 |
| P | RD_101 | RELATION | (ACCESS) | RI_1 |
| P | RD_101 | RELATION | (ADOPTED_ROLE) | RI_2 |
| P | RD_101 | RELATION | (ADOPTED_VIEW) | RI_3 |
| P | RD_101 | RELATION | (ALLOCATABLE_DEVICE) | RI_4 |
| P | RD_101 | RELATION | (ALLOCATABLE_GATEWAY) | RI_5 |
| P | RD_101 | RELATION | (ALLOCATABLE_MEDIUM) | RI_6 |
| P | RD_101 | RELATION | (ALLOCATABLE_PROCESSOR) | RI_7 |
| P | RD_101 | RELATION | (ALTERNATIVE) | RI_8 |
| P | RD_101 | RELATION | (ALTERNATIVE_OF) | RI_9 |
| P | RD_101 | RELATION | (ATTRIBUTE) | RI_10 |
| P | RD_101 | RELATION | (ATTRIBUTE_DEFINITION) | RI_11 |
| P | RD_101 | RELATION | (ATTRIBUTE_DEFINITION_-NAME) | RI_12 |
| P | RD_101 | RELATION | (ATTRIBUTE_DEFINITION_-PARENT) | RI_13 |
| P | RD_101 | RELATION | (ATTRIBUTE_KIND) | RI_14 |
| P | RD_101 | RELATION | (ATTRIBUTE_KIND_-DEFINITION) | RI_15 |
| P | RD_101 | RELATION | (ATTRIBUTE_KIND_-DEFINITION_PARENT) | RI_16 |
| P | RD_101 | RELATION | (ATTRIBUTE_KIND_NAME) | RI_17 |
| P | RD_101 | RELATION | (ATTRIBUTE_KIND_OF) | RI_18 |
| P | RD_101 | RELATION | (ATTRIBUTE_MONITOR) | RI_19 |
| P | RD_101 | RELATION | (ATTRIBUTE_MONITOR_-PARENT) | RI_20 |


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| P | RD_101 | RELATION | (ATTRIBUTE_NAME) | RI_21 |
| P | RD_101 | RELATION | (ATTRIBUTE_OF) | RI_22 |
| P | RD_101 | RELATION | (BASIC) | RI_23 |
| P | RD_101 | RELATION | (CHANNEL) | RI_24 |
| P | RD_101 | RELATION | (CHANNEL_PARENT) | RI_25 |
| P | RD_101 | RELATION | (CLIENT) | RI_26 |
| P | RD_101 | RELATION | (COMPONENT) | RI_27 |
| P | RD_101 | RELATION | (COMPONENT_NAMED) | RI_28 |
| P | RD_101 | RELATION | (COMPONENT_NAMED_-ORDERED) | RI_29 |
| P | RD_101 | RELATION | (COMPONENT_OF) | RI_30 |
| P | RD_101 | RELATION | (COMPONENT_OF_NAMED) | RI_31 |
| P | RD_101 | RELATION | (COMPONENT_OF_NAMED_-ORDERED) | RI_32 |
| P | RD_101 | RELATION | (COMPONENT_OF_ORDERED) | RI_33 |
| P | RD_101 | RELATION | (COMPONENT_OF_UNIFORM_-LIST) | RI_34 |
| P | RD_101 | RELATION | (COMPONENT_ORDERED) | RI_35 |
| P | RD_101 | RELATION | (COMPONENT_UNIFORM_LIST) | RI_36 |
| P | RD_101 | RELATION | (COPY_OF) | RI_37 |
| P | RD_101 | RELATION | (CURRENT_JOB) | RI_38 |
| P | RD_101 | RELATION | (CURRENT_NODE) | RI_39 |
| P | RD_101 | RELATION | (CURRENT_PROCESSOR) | RI_40 |
| P | RD_101 | RELATION | (CURRENT_USER) | RI_41 |
| P | RD_101 | RELATION | (DEFAULT_MEDIUM) | RI_42 |
| P | RD_101 | RELATION | (DEFAULT_PROCESSOR) | RI_43 |
| P | RD_101 | RELATION | (DEFAULT_PROCESS_NODE_-KIND) | RI_44 |
| P | RD_101 | RELATION | (DEFAULT_ROLE) | RI_45 |
| P | RD_101 | RELATION | (DEFINITION) | RI_46 |
| P | RD_101 | RELATION | (DEFINITIONS_PARENT) | RI_47 |
| P | RD_101 | RELATION | (DEFINITIONS_PRIMARY) | RI_48 |
| P | RD_101 | RELATION | (DEFINITIONS_SECONDARY) | RI_49 |
| P | RD_101 | RELATION | (DENY) | RI_50 |
| P | RD_101 | RELATION | (DEVICE_PARENT) | RI_51 |
| P | RD_101 | RELATION | (DEVICE_PRIMARY) | RI_52 |
| P | RD_101 | RELATION | (DEVICE_SECONDARY) | RI_53 |
| P | RD_101 | RELATION | (EMANATES) | RI_54 |
| P | RD_101 | RELATION | (EMANATES_FROM) | RI_55 |
| P | RD_101 | RELATION | (EXECUTABLE_IMAGE) | RI_56 |
| P | RD_101 | RELATION | (GATEWAY_PARENT) | RI_57 |
| P | RD_101 | RELATION | (GATEWAY_PRIMARY) | RI_58 |
| P | RD_101 | RELATION | (GATEWAY_SECONDARY) | RI_59 |
| P | RD_101 | RELATION | (GENERALIZATION_OF_NKD) | RI_60 |
| P | RD_101 | RELATION | (GENERALIZATION_OF_RD) | RI_61 |
| P | RD_101 | RELATION | (GROUP_PARENT) | RI_62 |
| P | RD_101 | RELATION | (GROUP_PRIMARY) | RI_63 |
| P | RD_101 | RELATION | (GROUP_SECONDARY) | RI_64 |
| P | RD_101 | RELATION | (HANDLER_ATTRIBUTE) | RI_65 |
| P | RD_101 | RELATION | (HANDLER_ENVIRONMENT_-NODE) | RI_66 |



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| P | RD_101 | RELATION | (HANDLER_ERROR_FILE) | RI_67 |
| P | RD_101 | RELATION | (HANDLER_IMAGE) | RI_68 |
| P | RD_101 | RELATION | (HANDLER_INPUT_FILE) | RI_69 |
| P | RD_101 | RELATION | (HANDLER_MEDIUM_-RESOURCE) | RI_70 |
| P | RD_101 | RELATION | (HANDLER_OUTPUT_FILE) | RI_71 |
| P | RD_101 | RELATION | (HANDLER_PROCESS) | RI_72 |
| P | RD_101 | RELATION | (HANDLER_PROCESS_NODE_-KIND) | RI_73 |
| P | RD_101 | RELATION | (HANDLER_PROCESS_OF) | RI_74 |
| P | RD_101 | RELATION | (HANDLER_PROCESSOR_-RESOURCE) | RI_75 |
| P | RD_101 | RELATION | (INTRODUCED_NODE) | RI_76 |
| P | RD_101 | RELATION | (INTRODUCED_NODE_PARENT) | RI_77 |
| P | RD_101 | RELATION | (INTRODUCED_RELATION) | RI_78 |
| P | RD_101 | RELATION | (JOB) | RI_79 |
| P | RD_101 | RELATION | (JOB_PARENT) | RI_80 |
| P | RD_101 | RELATION | (KEY) | RI_81 |
| P | RD_101 | RELATION | (KEY_NAME) | RI_82 |
| P | RD_101 | RELATION | (KEY_OF) | RI_83 |
| P | RD_101 | RELATION | (MEDIUM_PARENT) | RI_84 |
| P | RD_101 | RELATION | (MEDIUM_PRIMARY) | RI_85 |
| P | RD_101 | RELATION | (MEDIUM_SECONDARY) | RI_86 |
| P | RD_101 | RELATION | (MONITORED_BY) | RI_87 |
| P | RD_101 | RELATION | (MONITORS) | RI_88 |
| P | RD_101 | RELATION | (NODE_KIND) | RI_89 |
| P | RD_101 | RELATION | (NODE_KIND_DEFINITION) | RI_90 |
| P | RD_101 | RELATION | (NODE_KIND_DEFINITION_-PARENT) | RI_91 |
| P | RD_101 | RELATION | (NODE_KIND_PARENT) | RI_92 |
| P | RD_101 | RELATION | (PARENT) | RI_93 |
| P | RD_101 | RELATION | (PERMANENT_MEMBER) | RI_94 |
| P | RD_101 | RELATION | (PERMANENT_MEMBER_-PARENT) | RI_95 |
| P | RD_101 | RELATION | (PORT) | RI_96 |
| P | RD_101 | RELATION | (POTENTIAL_MEMBER) | RI_97 |
| P | RD_101 | RELATION | (PROCESSOR_PARENT) | RI_98 |
| P | RD_101 | RELATION | (PROCESSOR_PRIMARY) | RI_99 |
| P | RD_101 | RELATION | (PROCESSOR_SECONDARY) | RI_100 |
| P | RD_101 | RELATION | (RELATION) | RI_101 |
| P | RD_101 | RELATION | (RELATION_DEFINITION) | RI_102 |
| P | RD_101 | RELATION | (RELATION_DEFINITION_-PARENT) | RI_103 |
| P | RD_101 | RELATION | (RELATION_NAME) | RI_104 |
| P | RD_101 | RELATION | (RELATION_PARENT) | RI_105 |
| P | RD_101 | RELATION | (RESOURCE_GROUP_PARENT) | RI_106 |
| P | RD_101 | RELATION | (RESOURCE_GROUP_PRIMARY) | RI_107 |
| P | RD_101 | RELATION | (RESOURCE_GROUP_-SECONDARY) | RI_108 |
| P | RD_101 | RELATION | (REVERSE_LINK) | RI_109 |


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| P | RD_101 | RELATION | (REVERSE_LINK_-BIDIRECTIONAL) | RI_110 |
| P | RD_101 | RELATION | (SPECIALIZATION_OF_NKD) | RI_111 |
| P | RD_101 | RELATION | (SPECIALIZATION_OF_RD) | RI_112 |
| P | RD_101 | RELATION | (STANDARD_ERROR) | RI_113 |
| P | RD_101 | RELATION | (STANDARD_INPUT) | RI_114 |
| P | RD_101 | RELATION | (STANDARD_OUTPUT) | RI_115 |
| P | RD_101 | RELATION | (TERMINATES) | RI_116 |
| P | RD_101 | RELATION | (TERMINATES_AT) | RI_117 |
| P | RD_101 | RELATION | (USER_PARENT) | RI_118 |
| P | RD_101 | RELATION | (USER_PRIMARY) | RI_119 |
| P | RD_101 | RELATION | (USER_SECONDARY) | RI_120 |
| P | RD_101 | RELATION | (VIEW_PARENT) | RI_121 |
| P | RD_101 | RELATION | (VIEW_PRIMARY) | RI_122 |
| P | RD_101 | RELATION | (VIEW_SECONDARY) | RI_123 |


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

## 20.2 Attribute definition nodes (ADs)

The following table describes the node attributes which have the same value on all ADs: 1

| Attribute Name | AD | Attribute Value |
|---|---|---|
| KIND | AD_45 | (STRUCTURAL) |
| STRUCTURAL_KIND | AD_76 | (DEFINITION) |
| DEFINITION_KIND | AD_15 | (ATTRIBUTE_DEFINITION) |

The following subsections describe each predefined AD. Each section name is the primary path element 2
from 'DEFINITIONS( PREDEFINED ) to the AD described.

### 20.2.1 'ATTRIBUTE_DEFINITION( ALLOWED_PROTOCOLS )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| AD_1 | ALLOWED_PROTOCOLS | NKD_1 | ATTRIBUTE_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("PORT relationship attribute identifying protocols allowed for the port.") |
| CAIS_CONTROLLED | AD_7 | (TRUE) |
| CAIS_DETERMINED | AD_8 | (FALSE) |
| DELETABLE | AD_16 | (FALSE) |
| MODIFIABLE | AD_57 | (TRUE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_14 | ATTRIBUTE_KIND | () | AKD_51 |
| S | RD_22 | ATTRIBUTE_OF | (PORT) | RD_96 |
| S | RD_13 | PARENT | () | N_2 |

### 20.2.2 'ATTRIBUTE_DEFINITION( ALLOWED_USAGE )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| AD_2 | ALLOWED_USAGE | NKD_1 | ATTRIBUTE_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("Bidirectional RD attribute controlling whether instances are primary, secondary or both.") |
| CAIS_CONTROLLED | AD_7 | (TRUE) |
| CAIS_DETERMINED | AD_8 | (FALSE) |
| DELETABLE | AD_16 | (TRUE) |
| MODIFIABLE | AD_57 | (TRUE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_14 | ATTRIBUTE_KIND | () | AKD_55 |
| S | RD_22 | ATTRIBUTE_OF | (RELATION_DEFINITION) | NKD_30 |
| S | RD_13 | PARENT | () | N_2 |


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

### 20.2.3 'ATTRIBUTE_DEFINITION( ANNOTATION )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| AD_3 | ANNOTATION | NKD_1 | ATTRIBUTE_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("DEFINITION or VIEW node attribute intended for documentation.") |
| CAIS_CONTROLLED | AD_7 | (FALSE) |
| CAIS_DETERMINED | AD_8 | (FALSE) |
| DELETABLE | AD_16 | (TRUE) |
| MODIFIABLE | AD_57 | (TRUE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_14 | ATTRIBUTE_KIND | () | AKD_2 |
| S | RD_22 | ATTRIBUTE_OF | (DEFINITION) | NKD_7 |
| S | RD_22 | ATTRIBUTE_OF | (DEFINITIONS) | NKD_8 |
| S | RD_22 | ATTRIBUTE_OF | (RESOURCE) | NKD_32 |
| S | RD_22 | ATTRIBUTE_OF | (RESOURCE_GROUP) | NKD_33 |
| S | RD_22 | ATTRIBUTE_OF | (VIEW) | NKD_42 |
| S | RD_13 | PARENT | () | N_2 |

### 20.2.4 'ATTRIBUTE_DEFINITION( ATTRIBUTE_VALUE )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| AD_4 | ATTRIBUTE_VALUE | NKD_1 | ATTRIBUTE_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("HANDLE_ATTRIBUTE relationship attribute specifying the value of a monitor PROCESS node attribute.") |
| CAIS_CONTROLLED | AD_7 | (TRUE) |
| CAIS_DETERMINED | AD_8 | (FALSE) |
| DELETABLE | AD_16 | (FALSE) |
| MODIFIABLE | AD_57 | (FALSE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_14 | ATTRIBUTE_KIND | () | AKD_62 |
| S | RD_22 | ATTRIBUTE_OF | (HANDLER_ATTRIBUTE) | RD_65 |
| S | RD_13 | PARENT | () | N_2 |

### 20.2.5 'ATTRIBUTE_DEFINITION( ATTRIBUTES_MONITORED )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| AD_5 | ATTRIBUTES_MONITORED | NKD_1 | ATTRIBUTE_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("MONITORS relationship attribute identifying attributes being monitored.") |
| CAIS_CONTROLLED | AD_7 | (TRUE) |
| CAIS_DETERMINED | AD_8 | (FALSE) |
| DELETABLE | AD_16 | (TRUE) |


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

| Attribute Name | AD | Attribute Value |
|---|---|---|
| MODIFIABLE | AD_57 | (TRUE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_14 | ATTRIBUTE_KIND | () | AKD_4 |
| S | RD_22 | ATTRIBUTE_OF | (MONITORS) | RD_88 |
| S | RD_13 | PARENT | () | N_2 |

### 20.2.6 'ATTRIBUTE_DEFINITION( AVAILABLE )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| AD_6 | AVAILABLE | NKD_1 | ATTRIBUTE_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("RESOURCE node attribute indicating whether the resource is available for assignment.") |
| CAIS_CONTROLLED | AD_7 | (TRUE) |
| CAIS_DETERMINED | AD_8 | (TRUE) |
| DELETABLE | AD_16 | (FALSE) |
| MODIFIABLE | AD_57 | (TRUE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_14 | ATTRIBUTE_KIND | () | AKD_5 |
| S | RD_22 | ATTRIBUTE_OF | (RESOURCE) | NKD_32 |
| S | RD_13 | PARENT | () | N_2 |

### 20.2.7 'ATTRIBUTE_DEFINITION( CAIS_CONTROLLED )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| AD_7 | CAIS_CONTROLLED | NKD_1 | ATTRIBUTE_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("DEFINITION node attribute indicating whether instances must be manipulated by special purpose interfaces.") |
| CAIS_CONTROLLED | AD_7 | (TRUE) |
| CAIS_DETERMINED | AD_8 | (TRUE) |
| DELETABLE | AD_16 | (FALSE) |
| MODIFIABLE | AD_57 | (FALSE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_14 | ATTRIBUTE_KIND | () | AKD_5 |
| S | RD_22 | ATTRIBUTE_OF | (DEFINITION) | NKD_7 |
| S | RD_13 | PARENT | () | N_2 |

### 20.2.8 'ATTRIBUTE_DEFINITION( CAIS_DETERMINED )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| AD_8 | CAIS_DETERMINED | NKD_1 | ATTRIBUTE_DEFINITION |


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("DEFINITION node attribute indicating direct modifications can be performed on instances.") |
| CAIS_CONTROLLED | AD_7 | (TRUE) |
| CAIS_DETERMINED | AD_8 | (TRUE) |
| DELETABLE | AD_16 | (FALSE) |
| MODIFIABLE | AD_57 | (FALSE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_14 | ATTRIBUTE_KIND | () | AKD_5 |
| S | RD_22 | ATTRIBUTE_OF | (DEFINITION) | NKD_7 |
| S | RD_13 | PARENT | () | N_2 |

### 20.2.9 'ATTRIBUTE_DEFINITION( CAIS_KEY )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| AD_9 | CAIS_KEY | NKD_1 | ATTRIBUTE_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("General purpose CAIS-determined key.") |
| CAIS_CONTROLLED | AD_7 | (TRUE) |
| CAIS_DETERMINED | AD_8 | (TRUE) |
| DELETABLE | AD_16 | (FALSE) |
| MODIFIABLE | AD_57 | (FALSE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_14 | ATTRIBUTE_KIND | () | AKD_61 |
| A | AD_14 | DEFAULT_VALUE | ("#") | |
| S | RD_83 | KEY_OF | (ACCESS) | RD_1 |
| S | RD_83 | KEY_OF | (CLIENT) | RD_26 |
| S | RD_83 | KEY_OF | (DENY) | RD_50 |
| S | RD_83 | KEY_OF | (HANDLER_ATTRIBUTE) | RD_65 |
| S | RD_83 | KEY_OF | (HANDLER_PROCESS) | RD_72 |
| S | RD_83 | KEY_OF | (INTRODUCED_RELATION) | RD_78 |
| S | RD_83 | KEY_OF | (MONITORED_BY) | RD_87 |
| S | RD_83 | KEY_OF | (MONITORS) | RD_88 |
| S | RD_13 | PARENT | () | N_2 |

### 20.2.10 'ATTRIBUTE_DEFINITION( CONSTRAINTS )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| AD_10 | CONSTRAINTS | NKD_1 | ATTRIBUTE_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("ATTRIBUTE, ATTRIBUTE_KIND, ALTERNATIVE, or COMPONENT relationship attribute constraining value ranges.") |
| CAIS_CONTROLLED | AD_7 | (TRUE) |
| CAIS_DETERMINED | AD_8 | (FALSE) |
| DELETABLE | AD_16 | (TRUE) |
| MODIFIABLE | AD_57 | (TRUE) |



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_14 | ATTRIBUTE_KIND | () | AKD_14 |
| S | RD_22 | ATTRIBUTE_OF | (ALTERNATIVE) | RD_8 |
| S | RD_22 | ATTRIBUTE_OF | (ATTRIBUTE) | RD_10 |
| S | RD_22 | ATTRIBUTE_OF | (ATTRIBUTE_KIND) | RD_14 |
| S | RD_22 | ATTRIBUTE_OF | (COMPONENT) | RD_27 |
| S | RD_13 | PARENT | () | N_2 |

### 20.2.11 'ATTRIBUTE_DEFINITION( CONTROLLING_PROTOCOL )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| AD_11 | CONTROLLING_PROTOCOL | NKD_1 | ATTRIBUTE_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("CHANNEL node attribute indicating the protocol in use for a channel.") |
| CAIS_CONTROLLED | AD_7 | (TRUE) |
| CAIS_DETERMINED | AD_8 | (TRUE) |
| DELETABLE | AD_16 | (FALSE) |
| MODIFIABLE | AD_57 | (TRUE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_14 | ATTRIBUTE_KIND | () | AKD_50 |
| S | RD_22 | ATTRIBUTE_OF | (CHANNEL) | NKD_4 |
| S | RD_13 | PARENT | () | N_2 |

### 20.2.12 'ATTRIBUTE_DEFINITION( CURRENT_FILE_SIZE )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| AD_12 | CURRENT_FILE_SIZE | NKD_1 | ATTRIBUTE_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("SECONDARY_STORAGE node attribute indicating the current content size.") |
| CAIS_CONTROLLED | AD_7 | (TRUE) |
| CAIS_DETERMINED | AD_8 | (TRUE) |
| DELETABLE | AD_16 | (FALSE) |
| MODIFIABLE | AD_57 | (TRUE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_14 | ATTRIBUTE_KIND | () | AKD_23 |
| A | AD_14 | DEFAULT_VALUE | (0) | |
| S | RD_22 | ATTRIBUTE_OF | (SECONDARY_STORAGE) | NKD_34 |
| S | RD_13 | PARENT | () | N_2 |

### 20.2.13 'ATTRIBUTE_DEFINITION( CURRENT_STATUS )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| AD_13 | CURRENT_STATUS | NKD_1 | ATTRIBUTE_DEFINITION |



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("PROCESS node attribute indicating the process' current execution state.") |
| CAIS_CONTROLLED | AD_7 | (TRUE) |
| CAIS_DETERMINED | AD_8 | (TRUE) |
| DELETABLE | AD_16 | (FALSE) |
| MODIFIABLE | AD_57 | (TRUE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_14 | ATTRIBUTE_KIND | () | AKD_49 |
| S | RD_22 | ATTRIBUTE_OF | (PROCESS) | NKD_28 |
| S | RD_13 | PARENT | () | N_2 |

### 20.2.14 'ATTRIBUTE_DEFINITION( DEFAULT_VALUE )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| AD_14 | DEFAULT_VALUE | NKD_1 | ATTRIBUTE_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("ATTRIBUTE or ATTRIBUTE_KIND relationship attribute giving an attribute value to use at creation of nodes and relationships when none is otherwise provided.") |
| CAIS_CONTROLLED | AD_7 | (TRUE) |
| CAIS_DETERMINED | AD_8 | (FALSE) |
| DELETABLE | AD_16 | (TRUE) |
| MODIFIABLE | AD_57 | (TRUE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_14 | ATTRIBUTE_KIND | () | AKD_62 |
| S | RD_22 | ATTRIBUTE_OF | (ATTRIBUTE) | RD_10 |
| S | RD_22 | ATTRIBUTE_OF | (ATTRIBUTE_KIND) | RD_14 |
| S | RD_13 | PARENT | () | N_2 |

### 20.2.15 'ATTRIBUTE_DEFINITION( DEFINITION_KIND )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| AD_15 | DEFINITION_KIND | NKD_1 | ATTRIBUTE_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("DEFINITION node attribute partitioning NKDs for ADs, AKDs, NKDs and RDs.") |
| CAIS_CONTROLLED | AD_7 | (TRUE) |
| CAIS_DETERMINED | AD_8 | (FALSE) |
| DELETABLE | AD_16 | (FALSE) |
| MODIFIABLE | AD_57 | (FALSE) |



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_14 | ATTRIBUTE_KIND | () | AKD_16 |
| S | RD_22 | ATTRIBUTE_OF | (ATTRIBUTE_DEFINITION) | NKD_1 |
| S | RD_22 | ATTRIBUTE_OF | (ATTRIBUTE_KIND_-DEFINITION) | NKD_2 |
| S | RD_22 | ATTRIBUTE_OF | (DEFINITION) | NKD_7 |
| S | RD_22 | ATTRIBUTE_OF | (NODE_KIND_DEFINITION) | NKD_25 |
| S | RD_22 | ATTRIBUTE_OF | (RELATION_DEFINITION) | NKD_30 |
| S | RD_13 | PARENT | () | N_2 |

### 20.2.16 'ATTRIBUTE_DEFINITION( DELETABLE )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| AD_16 | DELETABLE | NKD_1 | ATTRIBUTE_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("AD attribute indicating whether instances can be deleted.") |
| CAIS_CONTROLLED | AD_7 | (TRUE) |
| CAIS_DETERMINED | AD_8 | (FALSE) |
| DELETABLE | AD_16 | (FALSE) |
| MODIFIABLE | AD_57 | (TRUE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_14 | ATTRIBUTE_KIND | () | AKD_5 |
| S | RD_22 | ATTRIBUTE_OF | (ATTRIBUTE_DEFINITION) | NKD_1 |
| S | RD_13 | PARENT | () | N_2 |

### 20.2.17 'ATTRIBUTE_DEFINITION( DEVICE_KIND )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| AD_17 | DEVICE_KIND | NKD_1 | ATTRIBUTE_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("DEVICE node attribute partitioning DEVICE NKD specializations.") |
| CAIS_CONTROLLED | AD_7 | (TRUE) |
| CAIS_DETERMINED | AD_8 | (FALSE) |
| DELETABLE | AD_16 | (FALSE) |
| MODIFIABLE | AD_57 | (FALSE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_14 | ATTRIBUTE_KIND | () | AKD_17 |
| S | RD_22 | ATTRIBUTE_OF | (TERMINAL) | NKD_38 |
| S | RD_22 | ATTRIBUTE_OF | (DEVICE) | NKD_9 |
| S | RD_22 | ATTRIBUTE_OF | (MAGNETIC_TAPE) | NKD_20 |
| S | RD_13 | PARENT | () | N_2 |



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

### 20.2.18 'ATTRIBUTE_DEFINITION( DIGITS )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| AD_18 | DIGITS | NKD_1 | ATTRIBUTE_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("Float AKD attribute indicating the minimum precision of instances.") |
| CAIS_CONTROLLED | AD_7 | (TRUE) |
| CAIS_DETERMINED | AD_8 | (FALSE) |
| DELETABLE | AD_16 | (FALSE) |
| MODIFIABLE | AD_57 | (TRUE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_14 | ATTRIBUTE_KIND | () | AKD_18 |
| S | RD_22 | ATTRIBUTE_OF | (FLOAT_ATTRIBUTE_KIND_-DEFINITION) | NKD_12 |
| S | RD_13 | PARENT | () | N_2 |

### 20.2.19 'ATTRIBUTE_DEFINITION( DIRECTION )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| AD_19 | DIRECTION | NKD_1 | ATTRIBUTE_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("PORT relationship attribute indicating the direction of data flow.") |
| CAIS_CONTROLLED | AD_7 | (TRUE) |
| CAIS_DETERMINED | AD_8 | (FALSE) |
| DELETABLE | AD_16 | (FALSE) |
| MODIFIABLE | AD_57 | (TRUE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_14 | ATTRIBUTE_KIND | () | AKD_19 |
| S | RD_22 | ATTRIBUTE_OF | (PORT) | RD_96 |
| S | RD_13 | PARENT | () | N_2 |

### 20.2.20 'ATTRIBUTE_DEFINITION( ELEMENT_SIZE )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| AD_20 | ELEMENT_SIZE | NKD_1 | ATTRIBUTE_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("PORT relationship attribute indicating the quantum for data flow.") |
| CAIS_CONTROLLED | AD_7 | (TRUE) |
| CAIS_DETERMINED | AD_8 | (FALSE) |
| DELETABLE | AD_16 | (FALSE) |
| MODIFIABLE | AD_57 | (TRUE) |


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_14 | ATTRIBUTE_KIND | () | AKD_20 |
| S | RD_22 | ATTRIBUTE_OF | (CHANNEL) | NKD_4 |
| S | RD_22 | ATTRIBUTE_OF | (PORT) | RD_96 |
| S | RD_13 | PARENT | () | N_2 |

### 20.2.21 'ATTRIBUTE_DEFINITION( ENUMERATED )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| AD_21 | ENUMERATED | NKD_1 | ATTRIBUTE_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("Identifier AKD attribute indicating whether values are ordered.") |
| CAIS_CONTROLLED | AD_7 | (TRUE) |
| CAIS_DETERMINED | AD_8 | (FALSE) |
| DELETABLE | AD_16 | (FALSE) |
| MODIFIABLE | AD_57 | (TRUE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_14 | ATTRIBUTE_KIND | () | AKD_5 |
| S | RD_22 | ATTRIBUTE_OF | (IDENTIFIER_ATTRIBUTE_-KIND_DEFINITION) | NKD_15 |
| S | RD_13 | PARENT | () | N_2 |

### 20.2.22 'ATTRIBUTE_DEFINITION( FILE_KIND )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| AD_22 | FILE_KIND | NKD_1 | ATTRIBUTE_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("FILE node attribute partitioning FILE NKD specializations.") |
| CAIS_CONTROLLED | AD_7 | (TRUE) |
| CAIS_DETERMINED | AD_8 | (FALSE) |
| DELETABLE | AD_16 | (FALSE) |
| MODIFIABLE | AD_57 | (FALSE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_14 | ATTRIBUTE_KIND | () | AKD_22 |
| S | RD_22 | ATTRIBUTE_OF | (DEVICE) | NKD_9 |
| S | RD_22 | ATTRIBUTE_OF | (FILE) | NKD_11 |
| S | RD_22 | ATTRIBUTE_OF | (GATEWAY) | NKD_13 |
| S | RD_22 | ATTRIBUTE_OF | (SECONDARY_STORAGE) | NKD_34 |
| S | RD_13 | PARENT | () | N_2 |



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| AD_23 | GENERIC_KEY | NKD_1 | ATTRIBUTE_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("General purpose key which is not CAIS-controlled.") |
| CAIS_CONTROLLED | AD_7 | (FALSE) |
| CAIS_DETERMINED | AD_8 | (FALSE) |
| DELETABLE | AD_16 | (FALSE) |
| MODIFIABLE | AD_57 | (TRUE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_14 | ATTRIBUTE_KIND | () | AKD_61 |
| A | AD_14 | DEFAULT_VALUE | ("#") | |
| S | RD_83 | KEY_OF | (ALLOCATABLE_DEVICE) | RD_4 |
| S | RD_83 | KEY_OF | (ALLOCATABLE_GATEWAY) | RD_5 |
| S | RD_83 | KEY_OF | (ALLOCATABLE_MEDIUM) | RD_6 |
| S | RD_83 | KEY_OF | (ALLOCATABLE_PROCESSOR) | RD_7 |
| S | RD_83 | KEY_OF | (ALTERNATIVE_OF) | RD_9 |
| S | RD_83 | KEY_OF | (ATTRIBUTE) | RD_10 |
| S | RD_83 | KEY_OF | (ATTRIBUTE_DEFINITION) | RD_11 |
| S | RD_83 | KEY_OF | (ATTRIBUTE_KIND_-DEFINITION) | RD_15 |
| S | RD_83 | KEY_OF | (ATTRIBUTE_KIND_OF) | RD_18 |
| S | RD_83 | KEY_OF | (ATTRIBUTE_MONITOR) | RD_19 |
| S | RD_83 | KEY_OF | (ATTRIBUTE_OF) | RD_22 |
| S | RD_83 | KEY_OF | (CHANNEL) | RD_24 |
| S | RD_83 | KEY_OF | (COMPONENT_OF) | RD_30 |
| S | RD_83 | KEY_OF | (DEFINITIONS_PRIMARY) | RD_48 |
| S | RD_83 | KEY_OF | (DEFINITIONS_SECONDARY) | RD_49 |
| S | RD_83 | KEY_OF | (DEVICE_PRIMARY) | RD_52 |
| S | RD_83 | KEY_OF | (DEVICE_SECONDARY) | RD_53 |
| S | RD_83 | KEY_OF | (EMANATES) | RD_54 |
| S | RD_83 | KEY_OF | (EMANATES_FROM) | RD_55 |
| S | RD_83 | KEY_OF | (GATEWAY_PRIMARY) | RD_58 |
| S | RD_83 | KEY_OF | (GATEWAY_SECONDARY) | RD_59 |
| S | RD_83 | KEY_OF | (GENERALIZATION_OF_NKD) | RD_60 |
| S | RD_83 | KEY_OF | (GENERALIZATION_OF_RD) | RD_61 |
| S | RD_83 | KEY_OF | (GROUP_PRIMARY) | RD_63 |
| S | RD_83 | KEY_OF | (GROUP_SECONDARY) | RD_64 |
| S | RD_83 | KEY_OF | (INTRODUCED_NODE) | RD_76 |
| S | RD_83 | KEY_OF | (JOB) | RD_79 |
| S | RD_83 | KEY_OF | (MEDIUM_PRIMARY) | RD_85 |
| S | RD_83 | KEY_OF | (MEDIUM_SECONDARY) | RD_86 |
| S | RD_83 | KEY_OF | (NODE_KIND_DEFINITION) | RD_90 |
| S | RD_83 | KEY_OF | (PERMANENT_MEMBER) | RD_94 |
| S | RD_83 | KEY_OF | (POTENTIAL_MEMBER) | RD_97 |
| S | RD_83 | KEY_OF | (PROCESSOR_PRIMARY) | RD_99 |
| S | RD_83 | KEY_OF | (PROCESSOR_SECONDARY) | RD_100 |
| S | RD_83 | KEY_OF | (RELATION_DEFINITION) | RD_102 |
| S | RD_83 | KEY_OF | (RESOURCE_GROUP_PRIMARY) | RD_107 |



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_83 | KEY_OF | (RESOURCE_GROUP_-SECONDARY) | RD_108 |
| S | RD_83 | KEY_OF | (SPECIALIZATION_OF_NKD) | RD_111 |
| S | RD_83 | KEY_OF | (SPECIALIZATION_OF_RD) | RD_112 |
| S | RD_83 | KEY_OF | (TERMINATES) | RD_116 |
| S | RD_83 | KEY_OF | (TERMINATES_AT) | RD_117 |
| S | RD_83 | KEY_OF | (USER_PRIMARY) | RD_119 |
| S | RD_83 | KEY_OF | (USER_SECONDARY) | RD_120 |
| S | RD_83 | KEY_OF | (VIEW_PRIMARY) | RD_122 |
| S | RD_83 | KEY_OF | (VIEW_SECONDARY) | RD_123 |
| S | RD_13 | PARENT | () | N_2 |

### 20.2.24 'ATTRIBUTE_DEFINITION( GRANT )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| AD_24 | GRANT | NKD_1 | ATTRIBUTE_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("ACCESS relationship attribute indicating the rights being granted to the target group.") |
| CAIS_CONTROLLED | AD_7 | (TRUE) |
| CAIS_DETERMINED | AD_8 | (FALSE) |
| DELETABLE | AD_16 | (FALSE) |
| MODIFIABLE | AD_57 | (TRUE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_14 | ATTRIBUTE_KIND | () | AKD_26 |
| S | RD_22 | ATTRIBUTE_OF | (ACCESS) | RD_1 |
| S | RD_13 | PARENT | () | N_2 |

### 20.2.25 'ATTRIBUTE_DEFINITION( HANDLER_AS_A_TRANSACTION )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| AD_25 | HANDLER_AS_A_TRANSACTION | NKD_1 | ATTRIBUTE_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("MONITOR node attribute indicating whether handler processes will execute as transactions.") |
| CAIS_CONTROLLED | AD_7 | (TRUE) |
| CAIS_DETERMINED | AD_8 | (FALSE) |
| DELETABLE | AD_16 | (FALSE) |
| MODIFIABLE | AD_57 | (FALSE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_14 | ATTRIBUTE_KIND | () | AKD_5 |
| S | RD_22 | ATTRIBUTE_OF | (MONITOR) | NKD_22 |
| S | RD_13 | PARENT | () | N_2 |



Source: https://assist.dla.mil -- Downloaded: 2026l0-26T07:54Z
Check the source to verify that this is the currentversion before use.

### 20.2.26 'ATTRIBUTE_DEFINITION( HANDLER_DELETE_WHEN_TERMINATED )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| AD_26 | HANDLER_DELETE_WHEN_-TERMINATED | NKD_1 | ATTRIBUTE_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("MONITOR node attribute indicating whether handler processes' nodes will be deleted at termination.") |
| CAIS_CONTROLLED | AD_7 | (TRUE) |
| CAIS_DETERMINED | AD_8 | (FALSE) |
| DELETABLE | AD_16 | (FALSE) |
| MODIFIABLE | AD_57 | (FALSE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_14 | ATTRIBUTE_KIND | () | AKD_5 |
| S | RD_22 | ATTRIBUTE_OF | (MONITOR) | NKD_22 |
| S | RD_13 | PARENT | () | N_2 |

### 20.2.27 'ATTRIBUTE_DEFINITION( HANDLER_DISCRETIONARY_ACCESS )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| AD_27 | HANDLER_DISCRETIONARY_-ACCESS | NKD_1 | ATTRIBUTE_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("MONITOR node attribute indicating initial access rights of the current user to handler processes' nodes.") |
| CAIS_CONTROLLED | AD_7 | (TRUE) |
| CAIS_DETERMINED | AD_8 | (FALSE) |
| DELETABLE | AD_16 | (FALSE) |
| MODIFIABLE | AD_57 | (FALSE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_14 | ATTRIBUTE_KIND | () | AKD_26 |
| S | RD_22 | ATTRIBUTE_OF | (MONITOR) | NKD_22 |
| S | RD_13 | PARENT | () | N_2 |

### 20.2.28 'ATTRIBUTE_DEFINITION( HANDLER_INPUT_PARAMETERS )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| AD_28 | HANDLER_INPUT_PARAMETERS | NKD_1 | ATTRIBUTE_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("MONITOR node attribute providing input parameters to handler processes.") |
| CAIS_CONTROLLED | AD_7 | (TRUE) |
| CAIS_DETERMINED | AD_8 | (FALSE) |
| DELETABLE | AD_16 | (FALSE) |
| MODIFIABLE | AD_57 | (FALSE) |



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_14 | ATTRIBUTE_KIND | () | AKD_46 |
| S | RD_22 | ATTRIBUTE_OF | (MONITOR) | NKD_22 |
| S | RD_13 | PARENT | () | N_2 |

### 20.2.29 'ATTRIBUTE_DEFINITION( HANDLER_KEY )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| AD_29 | HANDLER_KEY | NKD_1 | ATTRIBUTE_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("MONITOR node attribute providing the primary relationship key list for handler processes.") |
| CAIS_CONTROLLED | AD_7 | (TRUE) |
| CAIS_DETERMINED | AD_8 | (FALSE) |
| DELETABLE | AD_16 | (FALSE) |
| MODIFIABLE | AD_57 | (TRUE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_14 | ATTRIBUTE_KIND | () | AKD_54 |
| S | RD_22 | ATTRIBUTE_OF | (MONITOR) | NKD_22 |
| S | RD_13 | PARENT | () | N_2 |

### 20.2.30 'ATTRIBUTE_DEFINITION( HANDLER_MANDATORY_ACCESS )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| AD_30 | HANDLER_MANDATORY_-ACCESS | NKD_1 | ATTRIBUTE_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("MONITOR node attribute providing the minimum mandatory access classification for handler processes.") |
| CAIS_CONTROLLED | AD_7 | (TRUE) |
| CAIS_DETERMINED | AD_8 | (FALSE) |
| DELETABLE | AD_16 | (FALSE) |
| MODIFIABLE | AD_57 | (FALSE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_14 | ATTRIBUTE_KIND | () | AKD_11 |
| S | RD_22 | ATTRIBUTE_OF | (MONITOR) | NKD_22 |
| S | RD_13 | PARENT | () | N_2 |

### 20.2.31 'ATTRIBUTE_DEFINITION( HANDLER_PRIORITY )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| AD_31 | HANDLER_PRIORITY | NKD_1 | ATTRIBUTE_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("MONITOR node attribute providing the priority level at which handler processes are created.") |



Source: https://assist.dla.mil -- Downloaded: 2026l0-26T07:54Z
Check the source to verify that this is the currentversion before use.

| Attribute Name | AD | Attribute Value |
|---|---|---|
| CAIS_CONTROLLED | AD_7 | (TRUE) |
| CAIS_DETERMINED | AD_8 | (FALSE) |
| DELETABLE | AD_16 | (FALSE) |
| MODIFIABLE | AD_57 | (FALSE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_14 | ATTRIBUTE_KIND | () | AKD_48 |
| S | RD_22 | ATTRIBUTE_OF | (MONITOR) | NKD_22 |
| S | RD_13 | PARENT | () | N_2 |

### 20.2.32 'ATTRIBUTE_DEFINITION( HIGHEST_CLASSIFICATION )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| AD_32 | HIGHEST_CLASSIFICATION | NKD_1 | ATTRIBUTE_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("DEVICE node attribute indicating the maximum allowable mandatory access classification for a device.") |
| CAIS_CONTROLLED | AD_7 | (TRUE) |
| CAIS_DETERMINED | AD_8 | (FALSE) |
| DELETABLE | AD_16 | (FALSE) |
| MODIFIABLE | AD_57 | (FALSE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_14 | ATTRIBUTE_KIND | () | AKD_11 |
| S | RD_22 | ATTRIBUTE_OF | (DEVICE) | NKD_9 |
| S | RD_13 | PARENT | () | N_2 |

### 20.2.33 'ATTRIBUTE_DEFINITION( IMPORTED_DEFINITION_UID )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| AD_33 | IMPORTED_DEFINITION_UID | NKD_1 | ATTRIBUTE_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("INTRODUCED_NODE node attribute giving the UID of the NKD for the node which was introduced.") |
| CAIS_CONTROLLED | AD_7 | (TRUE) |
| CAIS_DETERMINED | AD_8 | (TRUE) |
| DELETABLE | AD_16 | (FALSE) |
| MODIFIABLE | AD_57 | (FALSE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_14 | ATTRIBUTE_KIND | () | AKD_64 |
| S | RD_22 | ATTRIBUTE_OF | (INTRODUCED_NODE) | NKD_18 |
| S | RD_13 | PARENT | () | N_2 |


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

### 20.2.34 'ATTRIBUTE_DEFINITION( IMPORTED_NAME )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| AD_34 | IMPORTED_NAME | NKD_1 | ATTRIBUTE_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("INTRODUCED_RELATION relationship attribute giving the relation name for the relationship which was introduced.") |
| CAIS_CONTROLLED | AD_7 | (TRUE) |
| CAIS_DETERMINED | AD_8 | (TRUE) |
| DELETABLE | AD_16 | (FALSE) |
| MODIFIABLE | AD_57 | (FALSE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_14 | ATTRIBUTE_KIND | () | AKD_61 |
| S | RD_22 | ATTRIBUTE_OF | (INTRODUCED_RELATION) | RD_78 |
| S | RD_13 | PARENT | () | N_2 |

### 20.2.35 'ATTRIBUTE_DEFINITION( IMPORTED_RELATIONSHIP_KIND )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| AD_35 | IMPORTED_RELATIONSHIP_-KIND | NKD_1 | ATTRIBUTE_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("INTRODUCED_RELATION relationship attribute giving the UID of the RD for the relationship which was introduced.") |
| CAIS_CONTROLLED | AD_7 | (TRUE) |
| CAIS_DETERMINED | AD_8 | (TRUE) |
| DELETABLE | AD_16 | (FALSE) |
| MODIFIABLE | AD_57 | (FALSE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_14 | ATTRIBUTE_KIND | () | AKD_55 |
| S | RD_22 | ATTRIBUTE_OF | (INTRODUCED_RELATION) | RD_78 |
| S | RD_13 | PARENT | () | N_2 |

### 20.2.36 'ATTRIBUTE_DEFINITION( IMPORTED_TARGET_UID )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| AD_36 | IMPORTED_TARGET_UID | NKD_1 | ATTRIBUTE_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("INTRODUCED_RELATION relationship attribute giving the UID of the target node for the relationship which was introduced.") |
| CAIS_CONTROLLED | AD_7 | (TRUE) |
| CAIS_DETERMINED | AD_8 | (TRUE) |
| DELETABLE | AD_16 | (FALSE) |
| MODIFIABLE | AD_57 | (FALSE) |



Source: https://assist.dla.mil -- Downloaded: 2026l0-26T07:54Z
Check the source to verify that this is the currentversion before use.

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_14 | ATTRIBUTE_KIND | () | AKD_64 |
| S | RD_22 | ATTRIBUTE_OF | (INTRODUCED_RELATION) | RD_78 |
| S | RD_13 | PARENT | () | N_2 |

### 20.2.37 'ATTRIBUTE_DEFINITION( IMPORTED_UID )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| AD_37 | IMPORTED_UID | NKD_1 | ATTRIBUTE_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("INTRODUCED_NODE node attribute giving the UID of the node which was introduced.") |
| CAIS_CONTROLLED | AD_7 | (TRUE) |
| CAIS_DETERMINED | AD_8 | (TRUE) |
| DELETABLE | AD_16 | (FALSE) |
| MODIFIABLE | AD_57 | (FALSE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_14 | ATTRIBUTE_KIND | () | AKD_64 |
| S | RD_22 | ATTRIBUTE_OF | (INTRODUCED_NODE) | NKD_18 |
| S | RD_13 | PARENT | () | N_2 |

### 20.2.38 'ATTRIBUTE_DEFINITION( INCOMING_FRAME_LENGTH )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| AD_38 | INCOMING_FRAME_LENGTH | NKD_1 | ATTRIBUTE_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("PORT relationship attribute indicating the quantum for data flow.") |
| CAIS_CONTROLLED | AD_7 | (TRUE) |
| CAIS_DETERMINED | AD_8 | (FALSE) |
| DELETABLE | AD_16 | (FALSE) |
| MODIFIABLE | AD_57 | (TRUE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_14 | ATTRIBUTE_KIND | () | AKD_25 |
| S | RD_22 | ATTRIBUTE_OF | (PORT) | RD_96 |
| S | RD_13 | PARENT | () | N_2 |

### 20.2.39 'ATTRIBUTE_DEFINITION( INHERITABLE )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| AD_39 | INHERITABLE | NKD_1 | ATTRIBUTE_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("BASIC relationship attribute indicating whether instances are copied at node copy and subprocess creation.") |
| CAIS_CONTROLLED | AD_7 | (TRUE) |



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

| Attribute Name | AD | Attribute Value |
|---|---|---|
| CAIS_DETERMINED | AD_8 | (FALSE) |
| DELETABLE | AD_16 | (FALSE) |
| MODIFIABLE | AD_57 | (TRUE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_14 | ATTRIBUTE_KIND | () | AKD_5 |
| S | RD_22 | ATTRIBUTE_OF | (ACCESS) | RD_1 |
| S | RD_22 | ATTRIBUTE_OF | (ADOPTED_ROLE) | RD_2 |
| S | RD_22 | ATTRIBUTE_OF | (ADOPTED_VIEW) | RD_3 |
| S | RD_22 | ATTRIBUTE_OF | (ALLOCATABLE_DEVICE) | RD_4 |
| S | RD_22 | ATTRIBUTE_OF | (ALLOCATABLE_GATEWAY) | RD_5 |
| S | RD_22 | ATTRIBUTE_OF | (ALLOCATABLE_MEDIUM) | RD_6 |
| S | RD_22 | ATTRIBUTE_OF | (ALLOCATABLE_PROCESSOR) | RD_7 |
| S | RD_22 | ATTRIBUTE_OF | (ALTERNATIVE) | RD_8 |
| S | RD_22 | ATTRIBUTE_OF | (ALTERNATIVE_OF) | RD_9 |
| S | RD_22 | ATTRIBUTE_OF | (ATTRIBUTE) | RD_10 |
| S | RD_22 | ATTRIBUTE_OF | (ATTRIBUTE_DEFINITION) | RD_11 |
| S | RD_22 | ATTRIBUTE_OF | (ATTRIBUTE_DEFINITION_-NAME) | RD_12 |
| S | RD_22 | ATTRIBUTE_OF | (ATTRIBUTE_KIND) | RD_14 |
| S | RD_22 | ATTRIBUTE_OF | (ATTRIBUTE_KIND_-DEFINITION) | RD_15 |
| S | RD_22 | ATTRIBUTE_OF | (ATTRIBUTE_KIND_NAME) | RD_17 |
| S | RD_22 | ATTRIBUTE_OF | (ATTRIBUTE_KIND_OF) | RD_18 |
| S | RD_22 | ATTRIBUTE_OF | (ATTRIBUTE_MONITOR) | RD_19 |
| S | RD_22 | ATTRIBUTE_OF | (ATTRIBUTE_NAME) | RD_21 |
| S | RD_22 | ATTRIBUTE_OF | (ATTRIBUTE_OF) | RD_22 |
| S | RD_22 | ATTRIBUTE_OF | (BASIC) | RD_23 |
| S | RD_22 | ATTRIBUTE_OF | (CHANNEL) | RD_24 |
| S | RD_22 | ATTRIBUTE_OF | (CLIENT) | RD_26 |
| S | RD_22 | ATTRIBUTE_OF | (COMPONENT) | RD_27 |
| S | RD_22 | ATTRIBUTE_OF | (COMPONENT_OF) | RD_30 |
| S | RD_22 | ATTRIBUTE_OF | (COPY_OF) | RD_37 |
| S | RD_22 | ATTRIBUTE_OF | (CURRENT_JOB) | RD_38 |
| S | RD_22 | ATTRIBUTE_OF | (CURRENT_NODE) | RD_39 |
| S | RD_22 | ATTRIBUTE_OF | (CURRENT_PROCESSOR) | RD_40 |
| S | RD_22 | ATTRIBUTE_OF | (CURRENT_USER) | RD_41 |
| S | RD_22 | ATTRIBUTE_OF | (DEFAULT_MEDIUM) | RD_42 |
| S | RD_22 | ATTRIBUTE_OF | (DEFAULT_PROCESSOR) | RD_43 |
| S | RD_22 | ATTRIBUTE_OF | (DEFAULT_PROCESS_NODE_-KIND) | RD_44 |
| S | RD_22 | ATTRIBUTE_OF | (DEFAULT_ROLE) | RD_45 |
| S | RD_22 | ATTRIBUTE_OF | (DEFINITIONS_PRIMARY) | RD_48 |
| S | RD_22 | ATTRIBUTE_OF | (DEFINITIONS_SECONDARY) | RD_49 |
| S | RD_22 | ATTRIBUTE_OF | (DENY) | RD_50 |
| S | RD_22 | ATTRIBUTE_OF | (DEVICE_PRIMARY) | RD_52 |
| S | RD_22 | ATTRIBUTE_OF | (DEVICE_SECONDARY) | RD_53 |
| S | RD_22 | ATTRIBUTE_OF | (EMANATES) | RD_54 |
| S | RD_22 | ATTRIBUTE_OF | (EMANATES_FROM) | RD_55 |
| S | RD_22 | ATTRIBUTE_OF | (EXECUTABLE_IMAGE) | RD_56 |
| S | RD_22 | ATTRIBUTE_OF | (GATEWAY_PRIMARY) | RD_58 |


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_22 | ATTRIBUTE_OF | (GATEWAY_SECONDARY) | RD_59 |
| S | RD_22 | ATTRIBUTE_OF | (GENERALIZATION_OF_NKD) | RD_60 |
| S | RD_22 | ATTRIBUTE_OF | (GENERALIZATION_OF_RD) | RD_61 |
| S | RD_22 | ATTRIBUTE_OF | (GROUP_PRIMARY) | RD_63 |
| S | RD_22 | ATTRIBUTE_OF | (GROUP_SECONDARY) | RD_64 |
| S | RD_22 | ATTRIBUTE_OF | (HANDLER_ATTRIBUTE) | RD_65 |
| S | RD_22 | ATTRIBUTE_OF | (HANDLER_ENVIRONMENT_-NODE) | RD_66 |
| S | RD_22 | ATTRIBUTE_OF | (HANDLER_ERROR_FILE) | RD_67 |
| S | RD_22 | ATTRIBUTE_OF | (HANDLER_IMAGE) | RD_68 |
| S | RD_22 | ATTRIBUTE_OF | (HANDLER_INPUT_FILE) | RD_69 |
| S | RD_22 | ATTRIBUTE_OF | (HANDLER_MEDIUM_-RESOURCE) | RD_70 |
| S | RD_22 | ATTRIBUTE_OF | (HANDLER_OUTPUT_FILE) | RD_71 |
| S | RD_22 | ATTRIBUTE_OF | (HANDLER_PROCESS) | RD_72 |
| S | RD_22 | ATTRIBUTE_OF | (HANDLER_PROCESS_NODE_-KIND) | RD_73 |
| S | RD_22 | ATTRIBUTE_OF | (HANDLER_PROCESS_OF) | RD_74 |
| S | RD_22 | ATTRIBUTE_OF | (HANDLER_PROCESSOR_-RESOURCE) | RD_75 |
| S | RD_22 | ATTRIBUTE_OF | (INTRODUCED_NODE) | RD_76 |
| S | RD_22 | ATTRIBUTE_OF | (INTRODUCED_RELATION) | RD_78 |
| S | RD_22 | ATTRIBUTE_OF | (JOB) | RD_79 |
| S | RD_22 | ATTRIBUTE_OF | (MEDIUM_PRIMARY) | RD_85 |
| S | RD_22 | ATTRIBUTE_OF | (MEDIUM_SECONDARY) | RD_86 |
| S | RD_22 | ATTRIBUTE_OF | (MONITORED_BY) | RD_87 |
| S | RD_22 | ATTRIBUTE_OF | (MONITORS) | RD_88 |
| S | RD_22 | ATTRIBUTE_OF | (NODE_KIND_DEFINITION) | RD_90 |
| S | RD_22 | ATTRIBUTE_OF | (NODE_KIND) | RD_89 |
| S | RD_22 | ATTRIBUTE_OF | (PARENT) | RD_93 |
| S | RD_22 | ATTRIBUTE_OF | (PERMANENT_MEMBER) | RD_94 |
| S | RD_22 | ATTRIBUTE_OF | (PORT) | RD_96 |
| S | RD_22 | ATTRIBUTE_OF | (POTENTIAL_MEMBER) | RD_97 |
| S | RD_22 | ATTRIBUTE_OF | (PROCESSOR_PRIMARY) | RD_99 |
| S | RD_22 | ATTRIBUTE_OF | (PROCESSOR_SECONDARY) | RD_100 |
| S | RD_22 | ATTRIBUTE_OF | (RELATION) | RD_101 |
| S | RD_22 | ATTRIBUTE_OF | (RELATION_DEFINITION) | RD_102 |
| S | RD_22 | ATTRIBUTE_OF | (RELATION_NAME) | RD_104 |
| S | RD_22 | ATTRIBUTE_OF | (RESOURCE_GROUP_PRIMARY) | RD_107 |
| S | RD_22 | ATTRIBUTE_OF | (RESOURCE_GROUP_-SECONDARY) | RD_108 |
| S | RD_22 | ATTRIBUTE_OF | (REVERSE_LINK) | RD_109 |
| S | RD_22 | ATTRIBUTE_OF | (REVERSE_LINK_-BIDIRECTIONAL) | RD_110 |
| S | RD_22 | ATTRIBUTE_OF | (SPECIALIZATION_OF_NKD) | RD_111 |
| S | RD_22 | ATTRIBUTE_OF | (SPECIALIZATION_OF_RD) | RD_112 |
| S | RD_22 | ATTRIBUTE_OF | (STANDARD_ERROR) | RD_113 |
| S | RD_22 | ATTRIBUTE_OF | (STANDARD_INPUT) | RD_114 |
| S | RD_22 | ATTRIBUTE_OF | (STANDARD_OUTPUT) | RD_115 |
| S | RD_22 | ATTRIBUTE_OF | (TERMINATES) | RD_116 |
| S | RD_22 | ATTRIBUTE_OF | (TERMINATES_AT) | RD_117 |


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_22 | ATTRIBUTE_OF | (USER_PRIMARY) | RD_119 |
| S | RD_22 | ATTRIBUTE_OF | (USER_SECONDARY) | RD_120 |
| S | RD_22 | ATTRIBUTE_OF | (VIEW_PRIMARY) | RD_122 |
| S | RD_22 | ATTRIBUTE_OF | (VIEW_SECONDARY) | RD_123 |
| S | RD_13 | PARENT | () | N_2 |

### 20.2.40 'ATTRIBUTE_DEFINITION( INTERPRETATION_KIND )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| AD_40 | INTERPRETATION_KIND | NKD_1 | ATTRIBUTE_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("INTERPRETATION node attribute partitioning NKDs for NKIs and RIs.") |
| CAIS_CONTROLLED | AD_7 | (TRUE) |
| CAIS_DETERMINED | AD_8 | (FALSE) |
| DELETABLE | AD_16 | (FALSE) |
| MODIFIABLE | AD_57 | (FALSE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_14 | ATTRIBUTE_KIND | () | AKD_30 |
| S | RD_22 | ATTRIBUTE_OF | (INTERPRETATION) | NKD_17 |
| S | RD_22 | ATTRIBUTE_OF | (NODE_KIND_-INTERPRETATION) | NKD_26 |
| S | RD_22 | ATTRIBUTE_OF | (RELATION_INTERPRETATION) | NKD_31 |
| S | RD_13 | PARENT | () | N_2 |

### 20.2.41 'ATTRIBUTE_DEFINITION( INTRODUCED_KEY_ATTRIBUTES )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| AD_41 | INTRODUCED_KEY_-ATTRIBUTES | NKD_1 | ATTRIBUTE_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("INTRODUCED_RELATION relationship attribute providing the key list of the introduced relationship.") |
| CAIS_CONTROLLED | AD_7 | (TRUE) |
| CAIS_DETERMINED | AD_8 | (TRUE) |
| DELETABLE | AD_16 | (FALSE) |
| MODIFIABLE | AD_57 | (FALSE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_14 | ATTRIBUTE_KIND | () | AKD_31 |
| S | RD_22 | ATTRIBUTE_OF | (INTRODUCED_RELATION) | RD_78 |
| S | RD_13 | PARENT | () | N_2 |


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| AD_42 | INTRODUCED_NODE_-ATTRIBUTES | NKD_1 | ATTRIBUTE_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("INTRODUCED_NODE node attribute providing the attributes of the introduced node.") |
| CAIS_CONTROLLED | AD_7 | (TRUE) |
| CAIS_DETERMINED | AD_8 | (TRUE) |
| DELETABLE | AD_16 | (FALSE) |
| MODIFIABLE | AD_57 | (FALSE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_14 | ATTRIBUTE_KIND | () | AKD_31 |
| S | RD_22 | ATTRIBUTE_OF | (INTRODUCED_NODE) | AD_42 |
| S | RD_13 | PARENT | () | N_2 |

### 20.2.43 'ATTRIBUTE_DEFINITION( INTRODUCED_RELATIONSHIP-_ATTRIBUTES )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| AD_43 | INTRODUCED_RELATIONSHIP_-ATTRIBUTES | NKD_1 | ATTRIBUTE_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("INTRODUCED_RELATION relationship attribute providing the non-key attributes of the introduced relationship.") |
| CAIS_CONTROLLED | AD_7 | (TRUE) |
| CAIS_DETERMINED | AD_8 | (TRUE) |
| DELETABLE | AD_16 | (FALSE) |
| MODIFIABLE | AD_57 | (FALSE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_14 | ATTRIBUTE_KIND | () | AKD_31 |
| S | RD_22 | ATTRIBUTE_OF | (INTRODUCED_RELATION) | RD_78 |
| S | RD_13 | PARENT | () | N_2 |

### 20.2.44 'ATTRIBUTE_DEFINITION( IO_UNIT_COUNT )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| AD_44 | IO_UNIT_COUNT | NKD_1 | ATTRIBUTE_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("PROCESS node attribute indicating the number of IO operations completed by the process.") |
| CAIS_CONTROLLED | AD_7 | (TRUE) |
| CAIS_DETERMINED | AD_8 | (TRUE) |
| DELETABLE | AD_16 | (FALSE) |
| MODIFIABLE | AD_57 | (TRUE) |



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_14 | ATTRIBUTE_KIND | () | AKD_8 |
| S | RD_22 | ATTRIBUTE_OF | (PROCESS) | NKD_28 |
| S | RD_13 | PARENT | () | N_2 |

### 20.2.45 'ATTRIBUTE_DEFINITION( KIND )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| AD_45 | KIND | NKD_1 | ATTRIBUTE_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("BASIC node attribute classifying instances as FILE, PROCESS or STRUCTURAL.") |
| CAIS_CONTROLLED | AD_7 | (TRUE) |
| CAIS_DETERMINED | AD_8 | (FALSE) |
| DELETABLE | AD_16 | (FALSE) |
| MODIFIABLE | AD_57 | (FALSE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_14 | ATTRIBUTE_KIND | () | AKD_36 |
| S | RD_22 | ATTRIBUTE_OF | (BASIC) | NKD_3 |
| S | RD_22 | ATTRIBUTE_OF | (FILE) | NKD_11 |
| S | RD_22 | ATTRIBUTE_OF | (PROCESS) | NKD_28 |
| S | RD_22 | ATTRIBUTE_OF | (STRUCTURAL) | NKD_36 |
| S | RD_13 | PARENT | () | N_2 |

### 20.2.46 'ATTRIBUTE_DEFINITION( LOWER_BOUND_FLOAT )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| AD_46 | LOWER_BOUND_FLOAT | NKD_1 | ATTRIBUTE_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("Float AKD attribute providing a value range lower bound.") |
| CAIS_CONTROLLED | AD_7 | (TRUE) |
| CAIS_DETERMINED | AD_8 | (FALSE) |
| DELETABLE | AD_16 | (TRUE) |
| MODIFIABLE | AD_57 | (TRUE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_14 | ATTRIBUTE_KIND | () | AKD_6 |
| S | RD_22 | ATTRIBUTE_OF | (FLOAT_ATTRIBUTE_KIND_-DEFINITION) | NKD_12 |
| S | RD_13 | PARENT | () | N_2 |

### 20.2.47 'ATTRIBUTE_DEFINITION( LOWER_BOUND_INTEGER )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| AD_47 | LOWER_BOUND_INTEGER | NKD_1 | ATTRIBUTE_DEFINITION |



Source: https://assist.dla.mil -- Downloaded: 202610-26T07:54Z
Check the source to verify that this is the current version before use.

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("Integer AKD attribute providing a value range lower bound.") |
| CAIS_CONTROLLED | AD_7 | (TRUE) |
| CAIS_DETERMINED | AD_8 | (FALSE) |
| DELETABLE | AD_16 | (TRUE) |
| MODIFIABLE | AD_57 | (TRUE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_14 | ATTRIBUTE_KIND | () | AKD_7 |
| S | RD_22 | ATTRIBUTE_OF | (INTEGER_ATTRIBUTE_KIND_-DEFINITION) | NKD_16 |
| S | RD_13 | PARENT | () | N_2 |

### 20.2.48 'ATTRIBUTE_DEFINITION( LOWEST_CLASSIFICATION )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| AD_48 | LOWEST_CLASSIFICATION | NKD_1 | ATTRIBUTE_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("DEVICE node attribute indicating the minimum allowable mandatory access classification for a device.") |
| CAIS_CONTROLLED | AD_7 | (TRUE) |
| CAIS_DETERMINED | AD_8 | (FALSE) |
| DELETABLE | AD_16 | (FALSE) |
| MODIFIABLE | AD_57 | (FALSE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_14 | ATTRIBUTE_KIND | () | AKD_11 |
| S | RD_22 | ATTRIBUTE_OF | (DEVICE) | NKD_9 |
| S | RD_13 | PARENT | () | N_2 |

### 20.2.49 'ATTRIBUTE_DEFINITION( MACHINE_TIME )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| AD_49 | MACHINE_TIME | NKD_1 | ATTRIBUTE_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("PROCESS node attribute indicating the CPU time utilized by the process in multiples of CAIS_PRAGMATICS_.SMALL_FOR_CAIS_DURATION.") |
| CAIS_CONTROLLED | AD_7 | (TRUE) |
| CAIS_DETERMINED | AD_8 | (TRUE) |
| DELETABLE | AD_16 | (FALSE) |
| MODIFIABLE | AD_57 | (TRUE) |



Source: https://assist.dla.mil -- Downloaded: 202610-26T07:54Z
Check the source to verify that this is the current version before use.

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_14 | ATTRIBUTE_KIND | () | AKD_58 |
| S | RD_22 | ATTRIBUTE_OF | (PROCESS) | NKD_28 |
| S | RD_13 | PARENT | () | N_2 |

### 20.2.50 'ATTRIBUTE_DEFINITION( MAXIMUM_FILE_SIZE )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| AD_50 | MAXIMUM_FILE_SIZE | NKD_1 | ATTRIBUTE_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("SECONDARY_STORAGE node attribute providing a limit on content size.") |
| CAIS_CONTROLLED | AD_7 | (TRUE) |
| CAIS_DETERMINED | AD_8 | (FALSE) |
| DELETABLE | AD_16 | (FALSE) |
| MODIFIABLE | AD_57 | (TRUE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_14 | ATTRIBUTE_KIND | () | AKD_23 |
| A | AD_14 | DEFAULT_VALUE | (0) | |
| S | RD_22 | ATTRIBUTE_OF | (SECONDARY_STORAGE) | NKD_34 |
| S | RD_13 | PARENT | () | N_2 |

### 20.2.51 'ATTRIBUTE_DEFINITION( MAXIMUM_INSTANCES )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| AD_51 | MAXIMUM_INSTANCES | NKD_1 | ATTRIBUTE_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("EMANATES relationship attribute providing a limit on the number of instances from any one base node.") |
| CAIS_CONTROLLED | AD_7 | (TRUE) |
| CAIS_DETERMINED | AD_8 | (FALSE) |
| DELETABLE | AD_16 | (TRUE) |
| MODIFIABLE | AD_57 | (TRUE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_14 | ATTRIBUTE_KIND | () | AKD_21 |
| S | RD_22 | ATTRIBUTE_OF | (EMANATES) | RD_54 |
| S | RD_13 | PARENT | () | N_2 |

### 20.2.52 'ATTRIBUTE_DEFINITION( MAXIMUM_LINE_LENGTH )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| AD_52 | MAXIMUM_LINE_LENGTH | NKD_1 | ATTRIBUTE_DEFINITION |



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("PORT relationship attribute providing a limit on the length of any line transferred.") |
| CAIS_CONTROLLED | AD_7 | (TRUE) |
| CAIS_DETERMINED | AD_8 | (FALSE) |
| DELETABLE | AD_16 | (FALSE) |
| MODIFIABLE | AD_57 | (TRUE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_14 | ATTRIBUTE_KIND | () | AKD_38 |
| S | RD_22 | ATTRIBUTE_OF | (CHANNEL) | NKD_4 |
| S | RD_22 | ATTRIBUTE_OF | (PORT) | RD_96 |
| S | RD_13 | PARENT | () | N_2 |

### 20.2.53 'ATTRIBUTE_DEFINITION( MAXIMUM_PAGE_LENGTH )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| AD_53 | MAXIMUM_PAGE_LENGTH | NKD_1 | ATTRIBUTE_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("PORT relationship attribute providing a limit on the length of any page transferred.") |
| CAIS_CONTROLLED | AD_7 | (TRUE) |
| CAIS_DETERMINED | AD_8 | (FALSE) |
| DELETABLE | AD_16 | (FALSE) |
| MODIFIABLE | AD_57 | (TRUE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_14 | ATTRIBUTE_KIND | () | AKD_39 |
| S | RD_22 | ATTRIBUTE_OF | (CHANNEL) | NKD_4 |
| S | RD_22 | ATTRIBUTE_OF | (PORT) | RD_96 |
| S | RD_13 | PARENT | () | N_2 |

### 20.2.54 'ATTRIBUTE_DEFINITION( MAXIMUM_SIZE )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| AD_54 | MAXIMUM_SIZE | NKD_1 | ATTRIBUTE_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("Uniform List AKD attribute providing a limit on the number of items in an instance.") |
| CAIS_CONTROLLED | AD_7 | (TRUE) |
| CAIS_DETERMINED | AD_8 | (FALSE) |
| DELETABLE | AD_16 | (TRUE) |
| MODIFIABLE | AD_57 | (TRUE) |



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_14 | ATTRIBUTE_KIND | () | AKD_37 |
| S | RD_22 | ATTRIBUTE_OF | (UNIFORM_LIST_ATTRIBUTE_-KIND_DEFINITION) | NKD_39 |
| S | RD_13 | PARENT | () | N_2 |

### 20.2.55 'ATTRIBUTE_DEFINITION( MINIMUM_INSTANCES )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| AD_55 | MINIMUM_INSTANCES | NKD_1 | ATTRIBUTE_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("EMANATES relationship attribute providing a lower limit (applied at deletion) on the number of instances from any one base node.") |
| CAIS_CONTROLLED | AD_7 | (TRUE) |
| CAIS_DETERMINED | AD_8 | (FALSE) |
| DELETABLE | AD_16 | (TRUE) |
| MODIFIABLE | AD_57 | (TRUE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_14 | ATTRIBUTE_KIND | () | AKD_21 |
| S | RD_22 | ATTRIBUTE_OF | (EMANATES) | RD_54 |
| S | RD_13 | PARENT | () | N_2 |

### 20.2.56 'ATTRIBUTE_DEFINITION( MINIMUM_SIZE )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| AD_56 | MINIMUM_SIZE | NKD_1 | ATTRIBUTE_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("Uniform List AKD attribute providing a lower limit on the number of items in an instance.") |
| CAIS_CONTROLLED | AD_7 | (TRUE) |
| CAIS_DETERMINED | AD_8 | (FALSE) |
| DELETABLE | AD_16 | (TRUE) |
| MODIFIABLE | AD_57 | (TRUE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_14 | ATTRIBUTE_KIND | () | AKD_37 |
| S | RD_22 | ATTRIBUTE_OF | (UNIFORM_LIST_ATTRIBUTE_-KIND_DEFINITION) | NKD_39 |
| S | RD_13 | PARENT | () | N_2 |

### 20.2.57 'ATTRIBUTE_DEFINITION( MODIFIABLE )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| AD_57 | MODIFIABLE | NKD_1 | ATTRIBUTE_DEFINITION |



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("AD attribute indicating whether instances can be modified.") |
| CAIS_CONTROLLED | AD_7 | (TRUE) |
| CAIS_DETERMINED | AD_8 | (FALSE) |
| DELETABLE | AD_16 | (FALSE) |
| MODIFIABLE | AD_57 | (TRUE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_14 | ATTRIBUTE_KIND | () | AKD_5 |
| S | RD_22 | ATTRIBUTE_OF | (ATTRIBUTE_DEFINITION) | NKD_1 |
| S | RD_13 | PARENT | () | N_2 |

### 20.2.58 'ATTRIBUTE_DEFINITION( NAME_KEY )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| AD_58 | NAME_KEY | NKD_1 | ATTRIBUTE_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("CAIS-controlled key which defines a name.") |
| CAIS_CONTROLLED | AD_7 | (TRUE) |
| CAIS_DETERMINED | AD_8 | (FALSE) |
| DELETABLE | AD_16 | (FALSE) |
| MODIFIABLE | AD_57 | (TRUE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_14 | ATTRIBUTE_KIND | () | AKD_61 |
| A | AD_14 | DEFAULT_VALUE | ("#") | |
| S | RD_83 | KEY_OF | (ADOPTED_ROLE) | RD_2 |
| S | RD_83 | KEY_OF | (ALTERNATIVE) | RD_8 |
| S | RD_83 | KEY_OF | (ATTRIBUTE_DEFINITION_-NAME) | RD_12 |
| S | RD_83 | KEY_OF | (ATTRIBUTE_KIND_NAME) | RD_17 |
| S | RD_83 | KEY_OF | (ATTRIBUTE_NAME) | RD_21 |
| S | RD_83 | KEY_OF | (COMPONENT_NAMED) | RD_28 |
| S | RD_83 | KEY_OF | (NODE_KIND) | RD_89 |
| S | RD_83 | KEY_OF | (PORT) | RD_96 |
| S | RD_83 | KEY_OF | (RELATION) | RD_101 |
| S | RD_83 | KEY_OF | (RELATION_NAME) | RD_104 |
| S | RD_13 | PARENT | () | N_2 |

### 20.2.59 'ATTRIBUTE_DEFINITION( NAMING )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| AD_59 | NAMING | NKD_1 | ATTRIBUTE_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("LAKD, ULAKD and CLAKD attribute indicating whether instances must contain named items.") |
| CAIS_CONTROLLED | AD_7 | (TRUE) |
| CAIS_DETERMINED | AD_8 | (FALSE) |



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

| Attribute Name | AD | Attribute Value |
|---|---|---|
| DELETABLE | AD_16 | (TRUE) |
| MODIFIABLE | AD_57 | (TRUE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_14 | ATTRIBUTE_KIND | () | AKD_41 |
| S | RD_22 | ATTRIBUTE_OF | (COMPOSITE_LIST_-ATTRIBUTE_KIND_DEFINITION) | NKD_6 |
| S | RD_22 | ATTRIBUTE_OF | (LIST_ATTRIBUTE_KIND_-DEFINITION) | NKD_19 |
| S | RD_22 | ATTRIBUTE_OF | (NAMED_COMPOSITE_LIST_-AKD) | NKD_23 |
| S | RD_22 | ATTRIBUTE_OF | (UNIFORM_LIST_ATTRIBUTE_-KIND_DEFINITION) | NKD_39 |
| S | RD_13 | PARENT | () | N_2 |

### 20.2.60 'ATTRIBUTE_DEFINITION( NODE_KIND )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| AD_60 | NODE_KIND | NKD_1 | ATTRIBUTE_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("BASIC relationship attribute classifying target nodes as FILE, PROCESS or STRUCTURAL.") |
| CAIS_CONTROLLED | AD_7 | (TRUE) |
| CAIS_DETERMINED | AD_8 | (TRUE) |
| DELETABLE | AD_16 | (FALSE) |
| MODIFIABLE | AD_57 | (FALSE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_14 | ATTRIBUTE_KIND | () | AKD_36 |
| S | RD_22 | ATTRIBUTE_OF | (ACCESS) | RD_1 |
| S | RD_22 | ATTRIBUTE_OF | (ADOPTED_ROLE) | RD_2 |
| S | RD_22 | ATTRIBUTE_OF | (ADOPTED_VIEW) | RD_3 |
| S | RD_22 | ATTRIBUTE_OF | (ALLOCATABLE_DEVICE) | RD_4 |
| S | RD_22 | ATTRIBUTE_OF | (ALLOCATABLE_GATEWAY) | RD_5 |
| S | RD_22 | ATTRIBUTE_OF | (ALLOCATABLE_MEDIUM) | RD_6 |
| S | RD_22 | ATTRIBUTE_OF | (ALLOCATABLE_PROCESSOR) | RD_7 |
| S | RD_22 | ATTRIBUTE_OF | (ALTERNATIVE) | RD_8 |
| S | RD_22 | ATTRIBUTE_OF | (ALTERNATIVE_OF) | RD_9 |
| S | RD_22 | ATTRIBUTE_OF | (ATTRIBUTE) | RD_10 |
| S | RD_22 | ATTRIBUTE_OF | (ATTRIBUTE_DEFINITION) | RD_11 |
| S | RD_22 | ATTRIBUTE_OF | (ATTRIBUTE_DEFINITION_-NAME) | RD_12 |
| S | RD_22 | ATTRIBUTE_OF | (ATTRIBUTE_KIND) | RD_14 |
| S | RD_22 | ATTRIBUTE_OF | (ATTRIBUTE_KIND_-DEFINITION) | RD_15 |
| S | RD_22 | ATTRIBUTE_OF | (ATTRIBUTE_KIND_NAME) | RD_17 |
| S | RD_22 | ATTRIBUTE_OF | (ATTRIBUTE_KIND_OF) | RD_18 |
| S | RD_22 | ATTRIBUTE_OF | (ATTRIBUTE_MONITOR) | RD_19 |
| S | RD_22 | ATTRIBUTE_OF | (ATTRIBUTE_MONITOR_-PARENT) | RD_20 |



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_22 | ATTRIBUTE_OF | (ATTRIBUTE_NAME) | RD_21 |
| S | RD_22 | ATTRIBUTE_OF | (ATTRIBUTE_OF) | RD_22 |
| S | RD_22 | ATTRIBUTE_OF | (BASIC) | RD_23 |
| S | RD_22 | ATTRIBUTE_OF | (CHANNEL) | RD_24 |
| S | RD_22 | ATTRIBUTE_OF | (CLIENT) | RD_26 |
| S | RD_22 | ATTRIBUTE_OF | (COMPONENT) | RD_27 |
| S | RD_22 | ATTRIBUTE_OF | (COMPONENT_OF) | RD_30 |
| S | RD_22 | ATTRIBUTE_OF | (CURRENT_JOB) | RD_38 |
| S | RD_22 | ATTRIBUTE_OF | (CURRENT_PROCESSOR) | RD_40 |
| S | RD_22 | ATTRIBUTE_OF | (CURRENT_USER) | RD_41 |
| S | RD_22 | ATTRIBUTE_OF | (DEFAULT_MEDIUM) | RD_42 |
| S | RD_22 | ATTRIBUTE_OF | (DEFAULT_PROCESSOR) | RD_43 |
| S | RD_22 | ATTRIBUTE_OF | (DEFAULT_PROCESS_NODE_-KIND) | RD_44 |
| S | RD_22 | ATTRIBUTE_OF | (DEFAULT_ROLE) | RD_45 |
| S | RD_22 | ATTRIBUTE_OF | (DEFINITIONS) | RD_46 |
| S | RD_22 | ATTRIBUTE_OF | (DEFINITIONS_PARENT) | RD_47 |
| S | RD_22 | ATTRIBUTE_OF | (DEFINITIONS_PRIMARY) | RD_48 |
| S | RD_22 | ATTRIBUTE_OF | (DEFINITIONS_SECONDARY) | RD_49 |
| S | RD_22 | ATTRIBUTE_OF | (DENY) | RD_50 |
| S | RD_22 | ATTRIBUTE_OF | (DEVICE_PARENT) | RD_51 |
| S | RD_22 | ATTRIBUTE_OF | (DEVICE_PRIMARY) | RD_52 |
| S | RD_22 | ATTRIBUTE_OF | (DEVICE_SECONDARY) | RD_53 |
| S | RD_22 | ATTRIBUTE_OF | (EMANATES) | RD_54 |
| S | RD_22 | ATTRIBUTE_OF | (EMANATES_FROM) | RD_55 |
| S | RD_22 | ATTRIBUTE_OF | (EXECUTABLE_IMAGE) | RD_56 |
| S | RD_22 | ATTRIBUTE_OF | (GATEWAY_PARENT) | RD_57 |
| S | RD_22 | ATTRIBUTE_OF | (GATEWAY_PRIMARY) | RD_58 |
| S | RD_22 | ATTRIBUTE_OF | (GATEWAY_SECONDARY) | RD_59 |
| S | RD_22 | ATTRIBUTE_OF | (GENERALIZATION_OF_NKD) | RD_60 |
| S | RD_22 | ATTRIBUTE_OF | (GENERALIZATION_OF_RD) | RD_61 |
| S | RD_22 | ATTRIBUTE_OF | (GROUP_PARENT) | RD_62 |
| S | RD_22 | ATTRIBUTE_OF | (GROUP_PRIMARY) | RD_63 |
| S | RD_22 | ATTRIBUTE_OF | (GROUP_SECONDARY) | RD_64 |
| S | RD_22 | ATTRIBUTE_OF | (HANDLER_ATTRIBUTE) | RD_65 |
| S | RD_22 | ATTRIBUTE_OF | (HANDLER_ERROR_FILE) | RD_67 |
| S | RD_22 | ATTRIBUTE_OF | (HANDLER_IMAGE) | RD_68 |
| S | RD_22 | ATTRIBUTE_OF | (HANDLER_INPUT_FILE) | RD_69 |
| S | RD_22 | ATTRIBUTE_OF | (HANDLER_MEDIUM_-RESOURCE) | RD_70 |
| S | RD_22 | ATTRIBUTE_OF | (HANDLER_OUTPUT_FILE) | RD_71 |
| S | RD_22 | ATTRIBUTE_OF | (HANDLER_PROCESS) | RD_72 |
| S | RD_22 | ATTRIBUTE_OF | (HANDLER_PROCESS_NODE_-KIND) | RD_73 |
| S | RD_22 | ATTRIBUTE_OF | (HANDLER_PROCESS_OF) | RD_74 |
| S | RD_22 | ATTRIBUTE_OF | (HANDLER_PROCESSOR_-RESOURCE) | RD_75 |
| S | RD_22 | ATTRIBUTE_OF | (INTRODUCED_NODE) | RD_76 |
| S | RD_22 | ATTRIBUTE_OF | (INTRODUCED_RELATION) | RD_78 |
| S | RD_22 | ATTRIBUTE_OF | (JOB) | RD_79 |
| S | RD_22 | ATTRIBUTE_OF | (JOB_PARENT) | RD_80 |



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_22 | ATTRIBUTE_OF | (MEDIUM_PARENT) | RD_84 |
| S | RD_22 | ATTRIBUTE_OF | (MEDIUM_PRIMARY) | RD_85 |
| S | RD_22 | ATTRIBUTE_OF | (MEDIUM_SECONDARY) | RD_86 |
| S | RD_22 | ATTRIBUTE_OF | (MONITORED_BY) | RD_87 |
| S | RD_22 | ATTRIBUTE_OF | (MONITORS) | RD_88 |
| S | RD_22 | ATTRIBUTE_OF | (NODE_KIND_DEFINITION) | RD_90 |
| S | RD_22 | ATTRIBUTE_OF | (NODE_KIND) | RD_89 |
| S | RD_22 | ATTRIBUTE_OF | (NODE_KIND_PARENT) | RD_92 |
| S | RD_22 | ATTRIBUTE_OF | (PERMANENT_MEMBER) | RD_94 |
| S | RD_22 | ATTRIBUTE_OF | (PERMANENT_MEMBER_-PARENT) | RD_95 |
| S | RD_22 | ATTRIBUTE_OF | (PORT) | RD_96 |
| S | RD_22 | ATTRIBUTE_OF | (POTENTIAL_MEMBER) | RD_97 |
| S | RD_22 | ATTRIBUTE_OF | (PROCESSOR_PARENT) | RD_98 |
| S | RD_22 | ATTRIBUTE_OF | (PROCESSOR_PRIMARY) | RD_99 |
| S | RD_22 | ATTRIBUTE_OF | (PROCESSOR_SECONDARY) | RD_100 |
| S | RD_22 | ATTRIBUTE_OF | (RELATION) | RD_101 |
| S | RD_22 | ATTRIBUTE_OF | (RELATION_DEFINITION) | RD_102 |
| S | RD_22 | ATTRIBUTE_OF | (RELATION_NAME) | RD_104 |
| S | RD_22 | ATTRIBUTE_OF | (RELATION_PARENT) | RD_105 |
| S | RD_22 | ATTRIBUTE_OF | (RESOURCE_GROUP_PARENT) | RD_106 |
| S | RD_22 | ATTRIBUTE_OF | (RESOURCE_GROUP_PRIMARY) | RD_107 |
| S | RD_22 | ATTRIBUTE_OF | (RESOURCE_GROUP_-SECONDARY) | RD_108 |
| S | RD_22 | ATTRIBUTE_OF | (REVERSE_LINK) | RD_109 |
| S | RD_22 | ATTRIBUTE_OF | (REVERSE_LINK_-BIDIRECTIONAL) | RD_110 |
| S | RD_22 | ATTRIBUTE_OF | (SPECIALIZATION_OF_NKD) | RD_111 |
| S | RD_22 | ATTRIBUTE_OF | (SPECIALIZATION_OF_RD) | RD_112 |
| S | RD_22 | ATTRIBUTE_OF | (STANDARD_ERROR) | RD_113 |
| S | RD_22 | ATTRIBUTE_OF | (STANDARD_INPUT) | RD_114 |
| S | RD_22 | ATTRIBUTE_OF | (STANDARD_OUTPUT) | RD_115 |
| S | RD_22 | ATTRIBUTE_OF | (TERMINATES) | RD_116 |
| S | RD_22 | ATTRIBUTE_OF | (TERMINATES_AT) | RD_117 |
| S | RD_22 | ATTRIBUTE_OF | (USER_PARENT) | RD_118 |
| S | RD_22 | ATTRIBUTE_OF | (USER_PRIMARY) | RD_119 |
| S | RD_22 | ATTRIBUTE_OF | (USER_SECONDARY) | RD_120 |
| S | RD_22 | ATTRIBUTE_OF | (VIEW_PRIMARY) | RD_122 |
| S | RD_22 | ATTRIBUTE_OF | (VIEW_SECONDARY) | RD_123 |
| S | RD_13 | PARENT | () | N_2 |

### 20.2.61 'ATTRIBUTE_DEFINITION( OBJECT_CLASSIFICATION )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| AD_61 | OBJECT_CLASSIFICATION | NKD_1 | ATTRIBUTE_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("BASIC node attribute providing the mandatory access classification of instances.") |
| CAIS_CONTROLLED | AD_7 | (TRUE) |



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

| Attribute Name | AD | Attribute Value |
|---|---|---|
| CAIS_DETERMINED | AD_8 | (FALSE) |
| DELETABLE | AD_16 | (FALSE) |
| MODIFIABLE | AD_57 | (FALSE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_14 | ATTRIBUTE_KIND | () | AKD_11 |
| S | RD_22 | ATTRIBUTE_OF | (BASIC) | NKD_3 |
| S | RD_13 | PARENT | () | N_2 |

### 20.2.62 'ATTRIBUTE_DEFINITION( OPEN_NODE_HANDLE_COUNT )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| AD_62 | OPEN_NODE_HANDLE_COUNT | NKD_1 | ATTRIBUTE_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("PROCESS node attribute indicating the number of open node handles held by the process.") |
| CAIS_CONTROLLED | AD_7 | (TRUE) |
| CAIS_DETERMINED | AD_8 | (TRUE) |
| DELETABLE | AD_16 | (FALSE) |
| MODIFIABLE | AD_57 | (TRUE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_14 | ATTRIBUTE_KIND | () | AKD_44 |
| S | RD_22 | ATTRIBUTE_OF | (PROCESS) | NKD_28 |
| S | RD_13 | PARENT | () | N_2 |

### 20.2.63 'ATTRIBUTE_DEFINITION( OPTIONAL )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| AD_63 | OPTIONAL | NKD_1 | ATTRIBUTE_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("COMPONENT relationship attribute indicating whether instances of the target AKD are required in instances of the source AKD.") |
| CAIS_CONTROLLED | AD_7 | (TRUE) |
| CAIS_DETERMINED | AD_8 | (FALSE) |
| DELETABLE | AD_16 | (FALSE) |
| MODIFIABLE | AD_57 | (TRUE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_14 | ATTRIBUTE_KIND | () | AKD_5 |
| S | RD_22 | ATTRIBUTE_OF | (COMPONENT_NAMED) | RD_28 |
| S | RD_22 | ATTRIBUTE_OF | (COMPONENT_ORDERED) | RD_35 |
| S | RD_13 | PARENT | () | N_2 |



Source: https://assist.dla.mil -- Downloaded: 202610-26T07:54Z
Check the source to verify that this is the current version before use.

### 20.2.64 'ATTRIBUTE_DEFINITION( ORDERED )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| AD_64 | ORDERED | NKD_1 | ATTRIBUTE_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("Composite List AKD attribute indicating whether instances must have their components in order.") |
| CAIS_CONTROLLED | AD_7 | (TRUE) |
| CAIS_DETERMINED | AD_8 | (FALSE) |
| DELETABLE | AD_16 | (FALSE) |
| MODIFIABLE | AD_57 | (TRUE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_14 | ATTRIBUTE_KIND | () | AKD_5 |
| S | RD_22 | ATTRIBUTE_OF | (COMPOSITE_LIST_-ATTRIBUTE_KIND_DEFINITION) | NKD_6 |
| S | RD_22 | ATTRIBUTE_OF | (ORDERED_COMPOSITE_LIST_-AKD) | NKD_27 |
| S | RD_13 | PARENT | () | N_2 |

### 20.2.65 'ATTRIBUTE_DEFINITION( OUTGOING_FRAME_LENGTH_RANGE )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| AD_65 | OUTGOING_FRAME_LENGTH_-RANGE | NKD_1 | ATTRIBUTE_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("PORT relationship attribute indicating the allowed range for data flow quanta.") |
| CAIS_CONTROLLED | AD_7 | (TRUE) |
| CAIS_DETERMINED | AD_8 | (FALSE) |
| DELETABLE | AD_16 | (FALSE) |
| MODIFIABLE | AD_57 | (TRUE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_14 | ATTRIBUTE_KIND | () | AKD_45 |
| S | RD_22 | ATTRIBUTE_OF | (PORT) | RD_96 |
| S | RD_13 | PARENT | () | N_2 |

### 20.2.66 'ATTRIBUTE_DEFINITION( PARAMETERS )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| AD_66 | PARAMETERS | NKD_1 | ATTRIBUTE_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("PROCESS node attribute providing the process input parameters.") |
| CAIS_CONTROLLED | AD_7 | (FALSE) |
| CAIS_DETERMINED | AD_8 | (FALSE) |
| DELETABLE | AD_16 | (FALSE) |
| MODIFIABLE | AD_57 | (TRUE) |



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_14 | ATTRIBUTE_KIND | () | AKD_46 |
| S | RD_22 | ATTRIBUTE_OF | (PROCESS) | NKD_28 |
| S | RD_13 | PARENT | () | N_2 |

### 20.2.67 'ATTRIBUTE_DEFINITION( PATTERNS )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| AD_67 | PATTERNS | NKD_1 | ATTRIBUTE_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("String and Identifier AKD attribute providing the value range allowed for instances.") |
| CAIS_CONTROLLED | AD_7 | (TRUE) |
| CAIS_DETERMINED | AD_8 | (FALSE) |
| DELETABLE | AD_16 | (FALSE) |
| MODIFIABLE | AD_57 | (TRUE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_14 | ATTRIBUTE_KIND | () | AKD_47 |
| S | RD_22 | ATTRIBUTE_OF | (IDENTIFIER_ATTRIBUTE_-KIND_DEFINITION) | NKD_15 |
| S | RD_22 | ATTRIBUTE_OF | (STRING_ATTRIBUTE_KIND_-DEFINITION) | NKD_35 |
| S | RD_13 | PARENT | () | N_2 |

### 20.2.68 'ATTRIBUTE_DEFINITION( POSITION_KEY )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| AD_68 | POSITION_KEY | NKD_1 | ATTRIBUTE_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("CAIS-controlled key used to define a positional ordering of components of a composite list or a composite view.") |
| CAIS_CONTROLLED | AD_7 | (TRUE) |
| CAIS_DETERMINED | AD_8 | (FALSE) |
| DELETABLE | AD_16 | (FALSE) |
| MODIFIABLE | AD_57 | (TRUE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_14 | ATTRIBUTE_KIND | () | AKD_61 |
| A | AD_14 | DEFAULT_VALUE | ("#") | |
| S | RD_83 | KEY_OF | (ADOPTED_VIEW) | RD_3 |
| S | RD_83 | KEY_OF | (COMPONENT_ORDERED) | RD_35 |
| S | RD_13 | PARENT | () | N_2 |


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

### 20.2.69 'ATTRIBUTE_DEFINITION( PRIORITY )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| AD_69 | PRIORITY | NKD_1 | ATTRIBUTE_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("PROCESS node attribute indicating the execution priority of the process.") |
| CAIS_CONTROLLED | AD_7 | (TRUE) |
| CAIS_DETERMINED | AD_8 | (FALSE) |
| DELETABLE | AD_16 | (FALSE) |
| MODIFIABLE | AD_57 | (TRUE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_14 | ATTRIBUTE_KIND | () | AKD_48 |
| S | RD_22 | ATTRIBUTE_OF | (PROCESS) | NKD_28 |
| S | RD_13 | PARENT | () | N_2 |

### 20.2.70 'ATTRIBUTE_DEFINITION( PRIORITY_LEVEL )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| AD_70 | PRIORITY_LEVEL | NKD_1 | ATTRIBUTE_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("PORT relationship attribute indicating the maximum priority assignable to a data packet.") |
| CAIS_CONTROLLED | AD_7 | (TRUE) |
| CAIS_DETERMINED | AD_8 | (FALSE) |
| DELETABLE | AD_16 | (FALSE) |
| MODIFIABLE | AD_57 | (TRUE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_14 | ATTRIBUTE_KIND | () | AKD_24 |
| S | RD_22 | ATTRIBUTE_OF | (PORT) | RD_96 |
| S | RD_13 | PARENT | () | N_2 |

### 20.2.71 'ATTRIBUTE_DEFINITION( PROCESS_SIZE )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| AD_71 | PROCESS_SIZE | NKD_1 | ATTRIBUTE_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("PROCESS node attribute indicating the memory space occupied by the process.") |
| CAIS_CONTROLLED | AD_7 | (TRUE) |
| CAIS_DETERMINED | AD_8 | (TRUE) |
| DELETABLE | AD_16 | (FALSE) |
| MODIFIABLE | AD_57 | (TRUE) |



Source: https://assist.dla.mil -- Downloaded: 202640-26T07:54Z
Check the source to verify that this is the current version before use.

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_14 | ATTRIBUTE_KIND | () | AKD_8 |
| S | RD_22 | ATTRIBUTE_OF | (PROCESS) | NKD_28 |
| S | RD_13 | PARENT | () | N_2 |

### 20.2.72 'ATTRIBUTE_DEFINITION( QUEUE_LENGTH )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| AD_72 | QUEUE_LENGTH | NKD_1 | ATTRIBUTE_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("PORT relationship attribute limiting the number of packets which can be queued at one time.") |
| CAIS_CONTROLLED | AD_7 | (TRUE) |
| CAIS_DETERMINED | AD_8 | (FALSE) |
| DELETABLE | AD_16 | (FALSE) |
| MODIFIABLE | AD_57 | (TRUE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_14 | ATTRIBUTE_KIND | () | AKD_8 |
| S | RD_22 | ATTRIBUTE_OF | (PORT) | RD_96 |
| S | RD_13 | PARENT | () | N_2 |

### 20.2.73 'ATTRIBUTE_DEFINITION( RESULTS )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| AD_73 | RESULTS | NKD_1 | ATTRIBUTE_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("PROCESS node attribute noting the results of the process.") |
| CAIS_CONTROLLED | AD_7 | (TRUE) |
| CAIS_DETERMINED | AD_8 | (FALSE) |
| DELETABLE | AD_16 | (FALSE) |
| MODIFIABLE | AD_57 | (TRUE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_14 | ATTRIBUTE_KIND | () | AKD_57 |
| S | RD_22 | ATTRIBUTE_OF | (PROCESS) | NKD_28 |
| S | RD_13 | PARENT | () | N_2 |

### 20.2.74 'ATTRIBUTE_DEFINITION( SUBJECT_CLASSIFICATION )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| AD_74 | SUBJECT_CLASSIFICATION | NKD_1 | ATTRIBUTE_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("PROCESS node attribute indicating the mandatory access subject classification of the process.") |
| CAIS_CONTROLLED | AD_7 | (TRUE) |



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

| Attribute Name | AD | Attribute Value |
|---|---|---|
| CAIS_DETERMINED | AD_8 | (FALSE) |
| DELETABLE | AD_16 | (FALSE) |
| MODIFIABLE | AD_57 | (FALSE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_14 | ATTRIBUTE_KIND | () | AKD_11 |
| S | RD_22 | ATTRIBUTE_OF | (PROCESS) | NKD_28 |
| S | RD_13 | PARENT | () | N_2 |

### 20.2.75 'ATTRIBUTE_DEFINITION( STATE )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| AD_75 | STATE | NKD_1 | ATTRIBUTE_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("CHANNEL node attribute indicating the state of the data transfer connection.") |
| CAIS_CONTROLLED | AD_7 | (TRUE) |
| CAIS_DETERMINED | AD_8 | (TRUE) |
| DELETABLE | AD_16 | (FALSE) |
| MODIFIABLE | AD_57 | (TRUE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_14 | ATTRIBUTE_KIND | () | AKD_10 |
| S | RD_22 | ATTRIBUTE_OF | (CHANNEL) | NKD_4 |
| S | RD_13 | PARENT | () | N_2 |

### 20.2.76 'ATTRIBUTE_DEFINITION( STRUCTURAL_KIND )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| AD_76 | STRUCTURAL_KIND | NKD_1 | ATTRIBUTE_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("STRUCTURAL node attribute partitioning STRUCTURAL NKD specializations.") |
| CAIS_CONTROLLED | AD_7 | (TRUE) |
| CAIS_DETERMINED | AD_8 | (FALSE) |
| DELETABLE | AD_16 | (FALSE) |
| MODIFIABLE | AD_57 | (FALSE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_14 | ATTRIBUTE_KIND | () | AKD_59 |
| S | RD_22 | ATTRIBUTE_OF | (CHANNEL) | NKD_4 |
| S | RD_22 | ATTRIBUTE_OF | (DEFINITION) | NKD_7 |
| S | RD_22 | ATTRIBUTE_OF | (DEFINITIONS) | NKD_8 |
| S | RD_22 | ATTRIBUTE_OF | (GROUP) | NKD_14 |
| S | RD_22 | ATTRIBUTE_OF | (INTERPRETATION) | NKD_17 |
| S | RD_22 | ATTRIBUTE_OF | (MEDIUM) | NKD_21 |



Source: https://assist.dla.mil -- Downloaded: 202610-26T07:54Z
Check the source to verify that this is the current version before use.

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_22 | ATTRIBUTE_OF | (MONITOR) | NKD_22 |
| S | RD_22 | ATTRIBUTE_OF | (PROCESSOR) | NKD_29 |
| S | RD_22 | ATTRIBUTE_OF | (RESOURCE_GROUP) | NKD_33 |
| S | RD_22 | ATTRIBUTE_OF | (STRUCTURAL) | NKD_36 |
| S | RD_22 | ATTRIBUTE_OF | (SYSTEM_LEVEL) | NKD_37 |
| S | RD_22 | ATTRIBUTE_OF | (USER) | NKD_40 |
| S | RD_22 | ATTRIBUTE_OF | (VIEW) | NKD_42 |
| S | RD_13 | PARENT | () | N_2 |

### 20.2.77 'ATTRIBUTE_DEFINITION( TIME_ATTRIBUTE_WRITTEN )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| AD_77 | TIME_ATTRIBUTE_WRITTEN | NKD_1 | ATTRIBUTE_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("BASIC node attribute indicating the last time an attribute on the instance was modified.") |
| CAIS_CONTROLLED | AD_7 | (TRUE) |
| CAIS_DETERMINED | AD_8 | (TRUE) |
| DELETABLE | AD_16 | (FALSE) |
| MODIFIABLE | AD_57 | (TRUE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_14 | ATTRIBUTE_KIND | () | AKD_60 |
| S | RD_22 | ATTRIBUTE_OF | (BASIC) | NKD_3 |
| S | RD_13 | PARENT | () | N_2 |

### 20.2.78 'ATTRIBUTE_DEFINITION( TIME_CONTENTS_WRITTEN )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| AD_78 | TIME_CONTENTS_WRITTEN | NKD_1 | ATTRIBUTE_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("FILE node attribute indicating the last time the content of the instance was modified.") |
| CAIS_CONTROLLED | AD_7 | (TRUE) |
| CAIS_DETERMINED | AD_8 | (TRUE) |
| DELETABLE | AD_16 | (FALSE) |
| MODIFIABLE | AD_57 | (TRUE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_14 | ATTRIBUTE_KIND | () | AKD_60 |
| S | RD_22 | ATTRIBUTE_OF | (FILE) | NKD_11 |
| S | RD_13 | PARENT | () | N_2 |


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

### 20.2.79 'ATTRIBUTE_DEFINITION( TIME_CREATED )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| AD_79 | TIME_CREATED | NKD_1 | ATTRIBUTE_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("BASIC node attribute indicating the time the instance was created.") |
| CAIS_CONTROLLED | AD_7 | (TRUE) |
| CAIS_DETERMINED | AD_8 | (TRUE) |
| DELETABLE | AD_16 | (FALSE) |
| MODIFIABLE | AD_57 | (FALSE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_14 | ATTRIBUTE_KIND | () | AKD_60 |
| S | RD_22 | ATTRIBUTE_OF | (BASIC) | NKD_3 |
| S | RD_13 | PARENT | () | N_2 |

### 20.2.80 'ATTRIBUTE_DEFINITION( TIME_FINISHED )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| AD_80 | TIME_FINISHED | NKD_1 | ATTRIBUTE_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("PROCESS node attribute indicating the time the process terminated.") |
| CAIS_CONTROLLED | AD_7 | (TRUE) |
| CAIS_DETERMINED | AD_8 | (TRUE) |
| DELETABLE | AD_16 | (FALSE) |
| MODIFIABLE | AD_57 | (FALSE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_14 | ATTRIBUTE_KIND | () | AKD_60 |
| S | RD_22 | ATTRIBUTE_OF | (PROCESS) | NKD_28 |
| S | RD_13 | PARENT | () | N_2 |

### 20.2.81 'ATTRIBUTE_DEFINITION( TIME_RELATIONSHIP_WRITTEN )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| AD_81 | TIME_RELATIONSHIP_WRITTEN | NKD_1 | ATTRIBUTE_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("BASIC node attribute indicating the time a relationship on the instance was modified.") |
| CAIS_CONTROLLED | AD_7 | (TRUE) |
| CAIS_DETERMINED | AD_8 | (TRUE) |
| DELETABLE | AD_16 | (FALSE) |
| MODIFIABLE | AD_57 | (TRUE) |



Source: https://assist.dla.mil -- Downloaded: 202610-26T07:54Z
Check the source to verify that this is the current version before use.

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_14 | ATTRIBUTE_KIND | () | AKD_60 |
| S | RD_22 | ATTRIBUTE_OF | (BASIC) | NKD_3 |
| S | RD_13 | PARENT | () | N_2 |

### 20.2.82 'ATTRIBUTE_DEFINITION( TIME_STARTED )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| AD_82 | TIME_STARTED | NKD_1 | ATTRIBUTE_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("PROCESS node attribute indicating the time the process was activated.") |
| CAIS_CONTROLLED | AD_7 | (TRUE) |
| CAIS_DETERMINED | AD_8 | (TRUE) |
| DELETABLE | AD_16 | (FALSE) |
| MODIFIABLE | AD_57 | (FALSE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_14 | ATTRIBUTE_KIND | () | AKD_60 |
| S | RD_22 | ATTRIBUTE_OF | (PROCESS) | NKD_28 |
| S | RD_13 | PARENT | () | N_2 |

### 20.2.83 'ATTRIBUTE_DEFINITION( UNIQUE_IDENTIFIER )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| AD_83 | UNIQUE_IDENTIFIER | NKD_1 | ATTRIBUTE_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("BASIC node attribute providing the UID of the node.") |
| CAIS_CONTROLLED | AD_7 | (TRUE) |
| CAIS_DETERMINED | AD_8 | (TRUE) |
| DELETABLE | AD_16 | (FALSE) |
| MODIFIABLE | AD_57 | (FALSE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_14 | ATTRIBUTE_KIND | () | AKD_64 |
| S | RD_22 | ATTRIBUTE_OF | (BASIC) | NKD_3 |
| S | RD_13 | PARENT | () | N_2 |

### 20.2.84 'ATTRIBUTE_DEFINITION( UNIQUE_KEYS )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| AD_84 | UNIQUE_KEYS | NKD_1 | ATTRIBUTE_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("RD attribute indicating whether the keys defined for the relation are sufficient to uniquely identify instances operated on through the relation.") |
| CAIS_CONTROLLED | AD_7 | (TRUE) |



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

| Attribute Name | AD | Attribute Value |
|---|---|---|
| CAIS_DETERMINED | AD_8 | (FALSE) |
| DELETABLE | AD_16 | (TRUE) |
| MODIFIABLE | AD_57 | (TRUE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_14 | ATTRIBUTE_KIND | () | AKD_5 |
| A | AD_10 | CONSTRAINTS | (VALUE=>TRUE) | |
| S | RD_22 | ATTRIBUTE_OF | (RELATION_DEFINITION) | NKD_30 |
| S | RD_13 | PARENT | () | N_2 |

### 20.2.85 'ATTRIBUTE_DEFINITION( UPPER_BOUND_FLOAT )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| AD_85 | UPPER_BOUND_FLOAT | NKD_1 | ATTRIBUTE_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("Float AKD attribute providing a value range upper bound.") |
| CAIS_CONTROLLED | AD_7 | (TRUE) |
| CAIS_DETERMINED | AD_8 | (FALSE) |
| DELETABLE | AD_16 | (TRUE) |
| MODIFIABLE | AD_57 | (TRUE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_14 | ATTRIBUTE_KIND | () | AKD_6 |
| S | RD_22 | ATTRIBUTE_OF | (FLOAT_ATTRIBUTE_KIND_-DEFINITION) | NKD_12 |
| S | RD_13 | PARENT | () | N_2 |

### 20.2.86 'ATTRIBUTE_DEFINITION( UPPER_BOUND_INTEGER )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| AD_86 | UPPER_BOUND_INTEGER | NKD_1 | ATTRIBUTE_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("Integer AKD attribute providing a value range lower bound.") |
| CAIS_CONTROLLED | AD_7 | (TRUE) |
| CAIS_DETERMINED | AD_8 | (FALSE) |
| DELETABLE | AD_16 | (TRUE) |
| MODIFIABLE | AD_57 | (TRUE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_14 | ATTRIBUTE_KIND | () | AKD_7 |
| S | RD_22 | ATTRIBUTE_OF | (INTEGER_ATTRIBUTE_KIND_-DEFINITION) | NKD_16 |
| S | RD_13 | PARENT | () | N_2 |



Source: https://assist.dla.mil -- Downloaded: 202640-26T07:54Z
Check the source to verify that this is the current version before use.

### 20.2.87 'ATTRIBUTE_DEFINITION( VALUE_TYPE )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| AD_87 | VALUE_TYPE | NKD_1 | ATTRIBUTE_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("AKD attribute partitioning specializations of the AKD NKD and indicating the value type defined by the AKD.") |
| CAIS_CONTROLLED | AD_7 | (TRUE) |
| CAIS_DETERMINED | AD_8 | (FALSE) |
| DELETABLE | AD_16 | (FALSE) |
| MODIFIABLE | AD_57 | (TRUE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_14 | ATTRIBUTE_KIND | () | AKD_65 |
| S | RD_22 | ATTRIBUTE_OF | (ATTRIBUTE_KIND_DEFINITON) | NKD_2 |
| S | RD_22 | ATTRIBUTE_OF | (COMPOSITE_LIST_-ATTRIBUTE_KIND_DEFINITION) | NKD_6 |
| S | RD_22 | ATTRIBUTE_OF | (FLOAT_ATTRIBUTE_KIND_-DEFINITION) | NKD_12 |
| S | RD_22 | ATTRIBUTE_OF | (IDENTIFIER_ATTRIBUTE_-KIND_DEFINITION) | NKD_15 |
| S | RD_22 | ATTRIBUTE_OF | (INTEGER_ATTRIBUTE_KIND_-DEFINITION) | NKD_16 |
| S | RD_22 | ATTRIBUTE_OF | (LIST_ATTRIBUTE_KIND_-DEFINITION) | NKD_19 |
| S | RD_22 | ATTRIBUTE_OF | (STRING_ATTRIBUTE_KIND_-DEFINITION) | NKD_35 |
| S | RD_22 | ATTRIBUTE_OF | (UNIFORM_LIST_ATTRIBUTE_-KIND_DEFINITION) | NKD_39 |
| S | RD_22 | ATTRIBUTE_OF | (VARIANT_ATTRIBUTE_KIND_-DEFINITION) | NKD_41 |
| S | RD_13 | PARENT | () | N_2 |



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

## 20.3 Attribute kind definition nodes (AKDs)

The following table describes the node attributes which have the same value on all AKDs:

| Attribute Name | AD | Attribute Value |
|---|---|---|
| KIND | AD_45 | (STRUCTURAL) |
| STRUCTURAL_KIND | AD_76 | (DEFINITION) |
| DEFINITION_KIND | AD_15 | (ATTRIBUTE_KIND) |
| CAIS_CONTROLLED | AD_7 | (FALSE) |
| CAIS_DETERMINED | AD_8 | (FALSE) |

The following subsections describe each predefined AKD. Each section name is the primary path element from 'DEFINITIONS( PREDEFINED ) to the AKD described.

### 20.3.1 'ATTRIBUTE_KIND_DEFINITION( ALTERNATIVE_CONSTRAINT )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| AKD_1 | ALTERNATIVE_CONSTRAINT | NKD_39 | UNIFORM_LIST_ATTRIBUTE_-KIND_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("Instances specify constraints on alternatives of variant AKDs.") |
| MINIMUM_SIZE | AD_56 | (1) |
| NAMING | AD_59 | (NAMED) |
| VALUE_TYPE | AD_87 | (UNIFORM_LIST) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_27 | COMPONENT | () | AKD_14 |
| S | RD_31 | COMPONENT_OF | (CONSTRAINTS_LIST) | AKD_14 |
| S | RD_16 | PARENT | () | N_2 |

### 20.3.2 'ATTRIBUTE_KIND_DEFINITION( ANNOTATION_KINDS )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| AKD_2 | ANNOTATION_KINDS | NKD_41 | VARIANT_ATTRIBUTE_KIND_-DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("Instances are annotations.") |
| VALUE_TYPE | AD_87 | (VARIANT) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_8 | ALTERNATIVE | (UNCONSTRAINED_LIST) | AKD_62 |
| S | RD_18 | ATTRIBUTE_KIND_OF | (ANNOTATION) | AD_3 |
| S | RD_16 | PARENT | () | N_2 |



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

### 20.3.3 'ATTRIBUTE_KIND_DEFINITION( ATTRIBUTE_DESIGNATOR )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| AKD_3 | ATTRIBUTE_DESIGNATOR | NKD_41 | VARIANT_ATTRIBUTE_KIND_-DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("Instances designate a node or relationship attribute for an attribute monitor.") |
| VALUE_TYPE | AD_87 | (VARIANT) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_8 | ALTERNATIVE | (UNIQUE_IDENTIFIER) | AKD_64 |
| S | RD_8 | ALTERNATIVE | (RELATIONSHIP_ATTRIBUTE_-DESIGNATOR) | AKD_53 |
| S | RD_30 | COMPONENT_OF | (ATTRIBUTES_MONITORED) | AKD_4 |
| S | RD_16 | PARENT | () | N_2 |

### 20.3.4 'ATTRIBUTE_KIND_DEFINITION( ATTRIBUTES_MONITORED )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| AKD_4 | ATTRIBUTES_MONITORED | NKD_39 | UNIFORM_LIST_ATTRIBUTE_-KIND_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("Instances are named lists designating attributes to be monitored.") |
| NAMING | AD_59 | (NAMED) |
| VALUE_TYPE | AD_87 | (UNIFORM_LIST) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_18 | ATTRIBUTE_KIND_OF | (ATTRIBUTES_MONITORED) | AD_5 |
| S | RD_27 | COMPONENT | () | AKD_3 |
| S | RD_16 | PARENT | () | N_2 |

### 20.3.5 'ATTRIBUTE_KIND_DEFINITION( BOOLEAN )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| AKD_5 | BOOLEAN | NKD_15 | IDENTIFIER_ATTRIBUTE_KIND_-DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("Instances represent boolean values.") |
| ENUMERATED | AD_21 | (TRUE) |
| PATTERNS | AD_67 | ("FALSE", "TRUE") |
| VALUE_TYPE | AD_87 | (IDENTIFIER) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_18 | ATTRIBUTE_KIND_OF | (AVAILABLE) | AD_6 |
| S | RD_18 | ATTRIBUTE_KIND_OF | (CAIS_CONTROLLED) | AD_7 |
| S | RD_18 | ATTRIBUTE_KIND_OF | (CAIS_DETERMINED) | AD_8 |
| S | RD_18 | ATTRIBUTE_KIND_OF | (DELETABLE) | AD_16 |



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_18 | ATTRIBUTE_KIND_OF | (ENUMERATED) | AD_21 |
| S | RD_18 | ATTRIBUTE_KIND_OF | (HANDLER_AS_A_-TRANSACTION) | AD_25 |
| S | RD_18 | ATTRIBUTE_KIND_OF | (HANDLER_DELETE_WHEN_-TERMINATED) | AD_26 |
| S | RD_18 | ATTRIBUTE_KIND_OF | (INHERITABLE) | AD_39 |
| S | RD_18 | ATTRIBUTE_KIND_OF | (MODIFIABLE) | AD_57 |
| S | RD_18 | ATTRIBUTE_KIND_OF | (OPTIONAL) | AD_63 |
| S | RD_18 | ATTRIBUTE_KIND_OF | (ORDERED) | AD_64 |
| S | RD_18 | ATTRIBUTE_KIND_OF | (UNIQUE_KEYS) | AD_84 |
| S | RD_16 | PARENT | () | N_2 |

### 20.3.6 'ATTRIBUTE_KIND_DEFINITION( CAIS_FLOAT )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| AKD_6 | CAIS_FLOAT | NKD_12 | FLOAT_ATTRIBUTE_KIND_-DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("Instances are CAIS list floating point items with CAIS maximum digits accuracy.") |
| DIGITS | AD_18 | (CAIS_PRAGMATICS.CAIS_LIST_MAXIMUM_DIGITS) |
| VALUE_TYPE | AD_87 | (FLOAT) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_9 | ALTERNATIVE_OF | (ITEM_VALUE) | AKD_33 |
| S | RD_9 | ALTERNATIVE_OF | (RANGE_CONSTRAINT) | AKD_52 |
| S | RD_18 | ATTRIBUTE_KIND_OF | (LOWER_BOUND_FLOAT) | AD_46 |
| S | RD_18 | ATTRIBUTE_KIND_OF | (UPPER_BOUND_FLOAT) | AD_85 |
| S | RD_16 | PARENT | () | N_2 |

### 20.3.7 'ATTRIBUTE_KIND_DEFINITION( CAIS_INTEGER )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| AKD_7 | CAIS_INTEGER | NKD_16 | INTEGER_ATTRIBUTE_KIND_-DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("Instances are integers of type CAIS_STANDARD.CAIS_-INTEGER") |
| LOWER_BOUND | AD_47 | (CAIS_PRAGMATICS.MINIMUM_INTEGER) |
| UPPER_BOUND | AD_86 | (CAIS_PRAGMATICS.MAXIMUM_INTEGER) |
| VALUE_TYPE | AD_87 | (INTEGER) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_9 | ALTERNATIVE_OF | (ITEM_VALUE) | AKD_33 |
| S | RD_9 | ALTERNATIVE_OF | (RANGE_CONSTRAINT) | AKD_52 |
| S | RD_18 | ATTRIBUTE_KIND_OF | (LOWER_BOUND_INTEGER) | AD_47 |
| S | RD_18 | ATTRIBUTE_KIND_OF | (UPPER_BOUND_INTEGER) | AD_86 |
| S | RD_16 | PARENT | () | N_2 |



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

### 20.3.8 'ATTRIBUTE_KIND_DEFINITION( CAIS_NATURAL )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| AKD_8 | CAIS_NATURAL | NKD_16 | INTEGER_ATTRIBUTE_KIND_-DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("Instances are integers of type CAIS_STANDARD.CAIS_-NATURAL") |
| LOWER_BOUND | AD_47 | (0) |
| UPPER_BOUND | AD_86 | (CAIS_PRAGMATICS.MAXIMUM_INTEGER) |
| VALUE_TYPE | AD_87 | (INTEGER) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_18 | ATTRIBUTE_KIND_OF | (IO_UNIT_COUNT) | AD_44 |
| S | RD_18 | ATTRIBUTE_KIND_OF | (PROCESS_SIZE) | AD_71 |
| S | RD_18 | ATTRIBUTE_KIND_OF | (QUEUE_LENGTH) | AD_72 |
| S | RD_16 | PARENT | () | N_2 |

### 20.3.9 'ATTRIBUTE_KIND_DEFINITION( CAIS_POSITIVE )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| AKD_9 | CAIS_POSITIVE | NKD_16 | INTEGER_ATTRIBUTE_KIND_-DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("Instances are integers of type CAIS_STANDARD.CAIS_-POSITIVE") |
| LOWER_BOUND | AD_47 | (1) |
| UPPER_BOUND | AD_86 | (CAIS_PRAGMATICS.MAXIMUM_INTEGER) |
| VALUE_TYPE | AD_87 | (INTEGER) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_16 | PARENT | () | N_2 |

### 20.3.10 'ATTRIBUTE_KIND_DEFINITION( CHANNEL_STATE )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| AKD_10 | CHANNEL_STATE | NKD_15 | IDENTIFIER_ATTRIBUTE_KIND_-DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("Instances designate IO states of channel nodes. ") |
| ENUMERATED | AD_21 | (TRUE) |
| PATTERNS | AD_67 | ("CONNECTED", "ESTABLISHED", "CLOSED", "RETIRED") |
| VALUE_TYPE | AD_87 | (IDENTIFIER) |


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_18 | ATTRIBUTE_KIND_OF | (STATE) | AD_75 |
| S | RD_16 | PARENT | () | N_2 |

### 20.3.11 'ATTRIBUTE_KIND_DEFINITION( CLASSIFICATION )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| AKD_11 | CLASSIFICATION | NKD_27 | ORDERED_COMPOSITE_LIST_-AKD |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("Instances designate mandatory access control classifications.") |
| NAMING | AD_59 | (UNNAMED) |
| ORDERED | AD_64 | (TRUE) |
| VALUE_TYPE | AD_87 | (COMPOSITE_LIST) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_18 | ATTRIBUTE_KIND_OF | (HANDLER_MANDATORY_-ACCESS) | AD_30 |
| S | RD_18 | ATTRIBUTE_KIND_OF | (HIGHEST_CLASSIFICATION) | AD_32 |
| S | RD_18 | ATTRIBUTE_KIND_OF | (LOWEST_CLASSIFICATION) | AD_48 |
| S | RD_18 | ATTRIBUTE_KIND_OF | (OBJECT_CLASSIFICATION) | AD_61 |
| S | RD_18 | ATTRIBUTE_KIND_OF | (SUBJECT_CLASSIFICATION) | AD_74 |
| S<br>A | RD_35<br>AD_63 | COMPONENT<br>OPTIONAL | (A)<br>(FALSE) | AKD_28 |
| S<br>A | RD_35<br>AD_63 | COMPONENT<br>OPTIONAL | (B)<br>(TRUE) | AKD_43 |
| S | RD_16 | PARENT | () | N_2 |

### 20.3.12 'ATTRIBUTE_KIND_DEFINITION( COMPONENT_CONSTRAINT )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| AKD_12 | COMPONENT_CONSTRAINT | NKD_41 | VARIANT_ATTRIBUTE_KIND_-DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("Instances specify constraints on components of uniform or composite lists.") |
| VALUE_TYPE | AD_87 | (VARIANT) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_8 | ALTERNATIVE | (COMPOSITE_LIST_-COMPONENT_CONSTRAINT) | AKD_13 |
| S | RD_8 | ALTERNATIVE | (CONSTRAINTS_LIST) | AKD_14 |
| S | RD_31 | COMPONENT_OF | (CONSTRAINTS_LIST) | AKD_14 |
| S | RD_16 | PARENT | () | N_2 |



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

### 20.3.13 'ATTRIBUTE_KIND_DEFINITION( COMPOSITE_LIST_COMPONENT_-CONSTRAINT )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| AKD_13 | COMPOSITE_LIST_-COMPONENT_CONSTRAINT | NKD_39 | UNIFORM_LIST_ATTRIBUTE_-KIND_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("Instances are lists of constraints lists which apply to components of a composite list.") |
| VALUE_TYPE | AD_87 | (UNIFORM_LIST) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_9 | ALTERNATIVE_OF | (COMPONENT_CONSTRAINT) | AKD_12 |
| S | RD_27 | COMPONENT | () | AKD_14 |
| S | RD_16 | PARENT | () | N_2 |

### 20.3.14 'ATTRIBUTE_KIND_DEFINITION( CONSTRAINTS_LIST )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| AKD_14 | CONSTRAINTS_LIST | NKD_23 | NAMED_COMPOSITE_LIST_AKD |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("Instances are lists of constraint items.") |
| NAMING | AD_59 | (NAMED) |
| ORDERED | AD_64 | (FALSE) |
| VALUE_TYPE | AD_87 | (COMPOSITE_LIST) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_9 | ALTERNATIVE_OF | (COMPONENT_CONSTRAINT) | AKD_12 |
| S | RD_18 | ATTRIBUTE_KIND_OF | (CONSTRAINTS) | AD_10 |
| S<br>A | RD_28<br>AD_63 | COMPONENT<br>OPTIONAL | (ALTERNATIVE)<br>(TRUE) | AKD_1 |
| S<br>A | RD_28<br>AD_63 | COMPONENT<br>OPTIONAL | (COMPONENT)<br>(TRUE) | AKD_12 |
| S<br>A<br>R | RD_28<br>AD_63<br>RD_31 | COMPONENT<br>OPTIONAL<br>COMPONENT_OF | (LOWER_BOUND)<br>(TRUE)<br>(CONSTRAINTS_LIST_LOWER_-BOUND) | AKD_52<br><br>AKD_14 |
| S<br>A<br>R | RD_28<br>AD_63<br>RD_31 | COMPONENT<br>OPTIONAL<br>COMPONENT_OF | (MAXIMUM_SIZE)<br>(TRUE)<br>(CONSTRAINTS_LIST_-MAXIMUM_SIZE) | AKD_37<br><br>AKD_14 |
| S<br>A<br>R | RD_28<br>AD_63<br>RD_31 | COMPONENT<br>OPTIONAL<br>COMPONENT_OF | (MINIMUM_SIZE)<br>(TRUE)<br>(CONSTRAINTS_LIST_-MINIMUM_SIZE) | AKD_37<br><br>AKD_14 |
| S<br>A | RD_28<br>AD_63 | COMPONENT<br>OPTIONAL | (PATTERNS)<br>(TRUE) | AKD_47 |



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_28 | COMPONENT | (UPPER_BOUND) | AKD_52 |
| A | AD_63 | OPTIONAL | (TRUE) | |
| R | RD_31 | COMPONENT_OF | (CONSTRAINTS_LIST_UPPER_-BOUND) | AKD_14 |
| S | RD_28 | COMPONENT | (VALUE) | AKD_33 |
| A | AD_63 | OPTIONAL | (TRUE) | |
| S | RD_30 | COMPONENT_OF | (COMPOSITE_LIST_-COMPONENT_CONSTRAINT) | AKD_13 |
| S | RD_30 | COMPONENT_OF | (ALTERNATIVE_CONSTRAINT) | AKD_1 |
| S | RD_16 | PARENT | () | N_2 |

### 20.3.15 'ATTRIBUTE_KIND_DEFINITION( DAY )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| AKD_15 | DAY | NKD_16 | INTEGER_ATTRIBUTE_KIND_-DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("Instances designate a day of the month as an integer. ") |
| LOWER_BOUND | AD_47 | (1) |
| UPPER_BOUND | AD_86 | (31) |
| VALUE_TYPE | AD_87 | (INTEGER) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_32 | COMPONENT_OF | (TIME) | AKD_60 |
| S | RD_16 | PARENT | () | N_2 |

### 20.3.16 'ATTRIBUTE_KIND_DEFINITION( DEFINITION_KIND )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| AKD_16 | DEFINITION_KIND | NKD_15 | IDENTIFIER_ATTRIBUTE_KIND_-DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("Instances designate a kind of a definition. ") |
| ENUMERATED | AD_21 | (FALSE) |
| PATTERNS | AD_67 | ("NODE_KIND", "RELATION", "ATTRIBUTE", "ATTRIBUTE_KIND") |
| VALUE_TYPE | AD_87 | (IDENTIFIER) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_18 | ATTRIBUTE_KIND_OF | (DEFINITION_KIND) | AD_15 |
| S | RD_16 | PARENT | () | N_2 |

### 20.3.17 'ATTRIBUTE_KIND_DEFINITION( DEVICE_KIND )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| AKD_17 | DEVICE_KIND | NKD_15 | IDENTIFIER_ATTRIBUTE_KIND_-DEFINITION |



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("Instances designate a kind of device.") |
| ENUMERATED | AD_21 | (FALSE) |
| PATTERNS | AD_67 | ("MAGNETIC_TAPE", "TERMINAL") |
| VALUE_TYPE | AD_87 | (IDENTIFIER) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_18 | ATTRIBUTE_KIND_OF | (DEVICE_KIND) | AD_17 |
| S | RD_16 | PARENT | () | N_2 |

### 20.3.18 'ATTRIBUTE_KIND_DEFINITION( DIGITS )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| AKD_18 | DIGITS | NKD_16 | INTEGER_ATTRIBUTE_KIND_-DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("Instances specify the number of digits of precision of float values.") |
| LOWER_BOUND | AD_47 | (1) |
| UPPER_BOUND | AD_86 | (CAIS_PRAGMATICS.LIST_MAXIMUM_DIGITS) |
| VALUE_TYPE | AD_87 | (INTEGER) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_18 | ATTRIBUTE_KIND_OF | (DIGITS) | AD_18 |
| S | RD_16 | PARENT | () | N_2 |

### 20.3.19 'ATTRIBUTE_KIND_DEFINITION( DIRECTION )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| AKD_19 | DIRECTION | NKD_15 | IDENTIFIER_ATTRIBUTE_KIND_-DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("Instances specify the direction of IO operations relative to a client.") |
| ENUMERATED | AD_21 | (FALSE) |
| PATTERNS | AD_67 | ("IN", "OUT", "IN_AND_OUT", "IN_OR_OUT") |
| VALUE_TYPE | AD_87 | (IDENTIFIER) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_18 | ATTRIBUTE_KIND_OF | (DIRECTION) | AD_19 |
| S | RD_16 | PARENT | () | N_2 |

### 20.3.20 'ATTRIBUTE_KIND_DEFINITION( ELEMENT_SIZE )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| AKD_20 | ELEMENT_SIZE | NKD_16 | INTEGER_ATTRIBUTE_KIND_-DEFINITION |



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("Instances specify the size of an element for the CAIS_SEQUENTIAL_IO and CAIS_DIRECT_IO protocols.") |
| LOWER_BOUND | AD_47 | (0) |
| UPPER_BOUND | AD_86 | (CAIS_PRAGMATICS.SEQUENTIAL_IO_RECORD_SIZE) |
| VALUE_TYPE | AD_87 | (INTEGER) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_18 | ATTRIBUTE_KIND_OF | (ELEMENT_SIZE) | AD_20 |
| S | RD_16 | PARENT | () | N_2 |

### 20.3.21 'ATTRIBUTE_KIND_DEFINITION( EMANATES_BOUND )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| AKD_21 | EMANATES_BOUND | NKD_16 | INTEGER_ATTRIBUTE_KIND_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("Instances limit the number of relationships of a given relation which emanate from a given node. ") |
| LOWER_BOUND | AD_47 | (0) |
| UPPER_BOUND | AD_86 | (CAIS_PRAGMATICS.EMANATING_SECONDARY_RELATIONSHIPS_PER_NODE) |
| VALUE_TYPE | AD_87 | (INTEGER) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_18 | ATTRIBUTE_KIND_OF | (MINIMUM_INSTANCES) | AD_55 |
| S | RD_18 | ATTRIBUTE_KIND_OF | (MAXIMUM_INSTANCES) | AD_51 |
| S | RD_16 | PARENT | () | N_2 |

### 20.3.22 'ATTRIBUTE_KIND_DEFINITION( FILE_KIND )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| AKD_22 | FILE_KIND | NKD_15 | IDENTIFIER_ATTRIBUTE_KIND_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("Instances designate a kind of file.") |
| ENUMERATED | AD_21 | (FALSE) |
| PATTERNS | AD_67 | ("SECONDARY_STORAGE", "DEVICE", "GATEWAY") |
| VALUE_TYPE | AD_87 | (IDENTIFIER) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_18 | ATTRIBUTE_KIND_OF | (FILE_KIND) | AD_22 |
| S | RD_16 | PARENT | () | N_2 |



Source: https://assist.dla.mil -- Downloaded: 202640-26T07:54Z
Check the source to verify that this is the current version before use.

### 20.3.23 'ATTRIBUTE_KIND_DEFINITION( FILE_SIZE )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| AKD_23 | FILE_SIZE | NKD_16 | INTEGER_ATTRIBUTE_KIND_-DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("Instances specify a file size in multiples of CAIS_PRAGMATICS.FILE_STORAGE_UNIT_SIZE.") |
| LOWER_BOUND | AD_47 | (0) |
| VALUE_TYPE | AD_87 | (INTEGER) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_18 | ATTRIBUTE_KIND_OF | (CURRENT_FILE_SIZE) | AD_12 |
| S | RD_18 | ATTRIBUTE_KIND_OF | (MAXIMUM_FILE_SIZE) | AD_50 |
| S | RD_16 | PARENT | () | N_2 |

### 20.3.24 'ATTRIBUTE_KIND_DEFINITION( FRAME_IO_PRIORITY_LEVEL )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| AKD_24 | FRAME_IO_PRIORITY_LEVEL | NKD_16 | INTEGER_ATTRIBUTE_KIND_-DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("Instances specify a priority level for CAIS_FRAME_IO protocol operations.") |
| LOWER_BOUND | AD_47 | (0) |
| UPPER_BOUND | AD_86 | (CAIS_PRAGMATICS.MAXIMUM_FRAME_PRIORITY) |
| VALUE_TYPE | AD_87 | (INTEGER) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_18 | ATTRIBUTE_KIND_OF | (PRIORITY_LEVEL) | AD_70 |
| S | RD_16 | PARENT | () | N_2 |

### 20.3.25 'ATTRIBUTE_KIND_DEFINITION( FRAME_LENGTH )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| AKD_25 | FRAME_LENGTH | NKD_16 | INTEGER_ATTRIBUTE_KIND_-DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("Instances specify the size of frames for the CAIS_FRAME_IO protocol operations.") |
| LOWER_BOUND | AD_47 | (0) |
| UPPER_BOUND | AD_86 | (CAIS_PRAGMATICS.MAXIMUM_FRAME_LENGTH) |
| VALUE_TYPE | AD_87 | (INTEGER) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_18 | ATTRIBUTE_KIND_OF | (INCOMING_FRAME_LENGTH) | AD_38 |
| S | RD_31 | COMPONENT_OF | (OUTGOING_FRAME_LENGTH_-LOWER) | AKD_45 |
| R | RD_28 | COMPONENT | (LOWER) | AKD_25 |



Source: https://assist.dla.mil -- Downloaded: 202610-26T07:54Z
Check the source to verify that this is the current version before use.

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_31 | COMPONENT_OF | (OUTGOING_FRAME_LENGTH_-UPPER) | AKD_45 |
| R | RD_28 | COMPONENT | (UPPER) | AKD_25 |
| S | RD_16 | PARENT | () | N_2 |

### 20.3.26 'ATTRIBUTE_KIND_DEFINITION( GRANT )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| AKD_26 | GRANT | NKD_39 | UNIFORM_LIST_ATTRIBUTE_-KIND_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("Instances are GRANT attribute values.") |
| NAMING | AD_59 | (UNNAMED) |
| VALUE_TYPE | AD_87 | (UNIFORM_LIST) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_18 | ATTRIBUTE_KIND_OF | (GRANT) | AD_24 |
| S | RD_18 | ATTRIBUTE_KIND_OF | (HANDLER_DISCRETIONARY_-ACCESS) | AD_27 |
| S | RD_27 | COMPONENT | () | AKD_27 |
| S | RD_16 | PARENT | () | N_2 |

### 20.3.27 'ATTRIBUTE_KIND_DEFINITION( GRANT_ITEM )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| AKD_27 | GRANT_ITEM | NKD_39 | UNIFORM_LIST_ATTRIBUTE_-KIND_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("Instances are grant items in a GRANT attribute value.") |
| MAXIMUM_SIZE | AD_54 | (1) |
| MINIMUM_SIZE | AD_56 | (1) |
| VALUE_TYPE | AD_87 | (UNIFORM_LIST) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_27 | COMPONENT | () | AKD_56 |
| S | RD_30 | COMPONENT_OF | (GRANT) | AKD_26 |
| S | RD_16 | PARENT | () | N_2 |

### 20.3.28 'ATTRIBUTE_KIND_DEFINITION( HIERARCHICAL_CLASSIFICATION )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| AKD_28 | HIERARCHICAL_-CLASSIFICATION | NKD_15 | IDENTIFIER_ATTRIBUTE_KIND_-DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("Instances specify the hierarchical component of mandatory access control labels.") |
| ENUMERATED | AD_21 | (TRUE) |


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

| Attribute Name | AD | Attribute Value |
|---|---|---|
| PATTERNS | AD_67 | (implementation defined) |
| VALUE_TYPE | AD_87 | (IDENTIFIER) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_33 | COMPONENT_OF | (CLASSIFICATION) | AKD_11 |
| S | RD_16 | PARENT | () | N_2 |

### 20.3.29 'ATTRIBUTE_KIND_DEFINITION( IDENTIFIER_LIST )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| AKD_29 | IDENTIFIER_LIST | NKD_39 | UNIFORM_LIST_ATTRIBUTE_-KIND_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("Instances are lists containing one or more identifiers.") |
| MINIMUM_SIZE | AD_56 | (1) |
| NAMING | AD_59 | (UNNAMED) |
| VALUE_TYPE | AD_87 | (UNIFORM_LIST) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_9 | ALTERNATIVE_OF | (RESULTING_RIGHTS_LIST) | AKD_56 |
| S | RD_27 | COMPONENT | () | AKD_61 |
| S | RD_16 | PARENT | () | N_2 |

### 20.3.30 'ATTRIBUTE_KIND_DEFINITION( INTERPRETATION_KIND )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| AKD_30 | INTERPRETATION_KIND | NKD_15 | IDENTIFIER_ATTRIBUTE_KIND_-DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("Instances designate a kind of interpretation.") |
| ENUMERATED | AD_21 | (FALSE) |
| PATTERNS | AD_67 | ("NODE_KIND", "RELATION") |
| VALUE_TYPE | AD_87 | (IDENTIFIER) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_18 | ATTRIBUTE_KIND_OF | (INTERPRETATION_KIND) | AD_40 |
| S | RD_16 | PARENT | () | N_2 |

### 20.3.31 'ATTRIBUTE_KIND_DEFINITION( INTRODUCED_ATTRIBUTE )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| AKD_31 | INTRODUCED_ATTRIBUTE | NKD_39 | UNIFORM_LIST_ATTRIBUTE_-KIND_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("Instances are lists containing the attributes of an introduced node or relationship.") |
| NAMING | AD_59 | (UNNAMED) |



Source: https://assist.dla.mil -- Downloaded: 202640-26T07:54Z
Check the source to verify that this is the current version before use.

| Attribute Name | AD | Attribute Value |
|---|---|---|
| VALUE_TYPE | AD_87 | (UNIFORM_LIST) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_18 | ATTRIBUTE_KIND_OF | (INTRODUCED_KEY_-ATTRIBUTES) | AD_41 |
| S | RD_18 | ATTRIBUTE_KIND_OF | (INTRODUCED_NODE_-ATTRIBUTES) | AD_42 |
| S | RD_18 | ATTRIBUTE_KIND_OF | (INTRODUCED_-RELATIONSHIP_ATTRIBUTES) | AD_43 |
| S | RD_27 | COMPONENT | () | AKD_32 |
| S | RD_16 | PARENT | () | N_2 |

### 20.3.32 'ATTRIBUTE_KIND_DEFINITION( INTRODUCED_ATTRIBUTE_-DESCRIPTION )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| AKD_32 | INTRODUCED_ATTRIBUTE_-DESCRIPTION | NKD_23 | NAMED_COMPOSITE_LIST_AKD |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("Instances identify and specify the value of an introduced attribute.") |
| NAMING | AD_59 | (NAMED) |
| ORDERED | AD_64 | (FALSE) |
| VALUE_TYPE | AD_87 | (COMPOSITE_LIST) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_28 | COMPONENT | (AD) | AKD_64 |
| A | AD_63 | OPTIONAL | (FALSE) | |
| S | RD_28 | COMPONENT | (VALUE) | AKD_62 |
| A | AD_63 | OPTIONAL | (FALSE) | |
| S | RD_30 | COMPONENT_OF | (INTRODUCED_ATTRIBUTE) | AKD_31 |
| S | RD_16 | PARENT | () | N_2 |

### 20.3.33 'ATTRIBUTE_KIND_DEFINITION( ITEM_VALUE )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| AKD_33 | ITEM_VALUE | NKD_41 | VARIANT_ATTRIBUTE_KIND_-DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("Instances are VALUE constraints in a CONSTRAINTS attribute.") |
| VALUE_TYPE | AD_87 | (VARIANT) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_8 | ALTERNATIVE | (FLOAT) | AKD_6 |
| S | RD_8 | ALTERNATIVE | (IDENTIFIER) | AKD_61 |
| S | RD_8 | ALTERNATIVE | (INTEGER) | AKD_7 |
| S | RD_8 | ALTERNATIVE | (LIST) | AKD_62 |


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_8 | ALTERNATIVE | (STRING) | AKD_63 |
| S | RD_31 | COMPONENT_OF | (CONSTRAINTS_LIST) | AKD_14 |
| S | RD_16 | PARENT | () | N_2 |

### 20.3.34 'ATTRIBUTE_KIND_DEFINITION( KEY_VALUE_DESIGNATOR )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| AKD_34 | KEY_VALUE_DESIGNATOR | NKD_23 | NAMED_COMPOSITE_LIST_AKD |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("Instances are named composite lists designating key values.") |
| NAMING | AD_59 | (NAMED) |
| VALUE_TYPE | AD_87 | (COMPOSITE_LIST) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_28 | COMPONENT | (AD) | AKD_64 |
| A | AD_63 | OPTIONAL | (FALSE) | |
| S | RD_28 | COMPONENT | (VALUE) | AKD_61 |
| A | AD_63 | OPTIONAL | (TRUE) | |
| S | RD_30 | COMPONENT_OF | (KEY_VALUE_LIST) | AKD_35 |
| S | RD_16 | PARENT | () | N_2 |

### 20.3.35 'ATTRIBUTE_KIND_DEFINITION( KEY_VALUE_LIST )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| AKD_35 | KEY_VALUE_LIST | NKD_39 | UNIFORM_LIST_ATTRIBUTE_KIND_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("Instances are unnamed lists of key value designators (possibly empty).") |
| NAMING | AD_59 | (UNNAMED) |
| VALUE_TYPE | AD_87 | (UNIFORM_LIST) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_27 | COMPONENT | () | AKD_34 |
| S | RD_31 | COMPONENT_OF | (RELATIONSHIP_ATTRIBUTE_DESIGNATOR) | AKD_53 |
| S | RD_16 | PARENT | () | N_2 |

### 20.3.36 'ATTRIBUTE_KIND_DEFINITION( KIND )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| AKD_36 | KIND | NKD_15 | IDENTIFIER_ATTRIBUTE_KIND_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("Instances designate a kind of node. ") |



Source: https://assist.dla.mil -- Downloaded: 202610-26T07:54Z
Check the source to verify that this is the currentversion before use.

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ENUMERATED | AD_21 | (FALSE) |
| PATTERNS | AD_67 | ("FILE", "STRUCTURAL", "PROCESS") |
| VALUE_TYPE | AD_87 | (IDENTIFIER) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_18 | ATTRIBUTE_KIND_OF | (KIND) | AD_45 |
| S | RD_18 | ATTRIBUTE_KIND_OF | (NODE_KIND) | AD_60 |
| S | RD_16 | PARENT | () | N_2 |

### 20.3.37 'ATTRIBUTE_KIND_DEFINITION( LIST_SIZE )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| AKD_37 | LIST_SIZE | NKD_16 | INTEGER_ATTRIBUTE_KIND_-DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("Instances specify the number of items in a list.") |
| LOWER_BOUND | AD_47 | (0) |
| UPPER_BOUND | AD_86 | (CAIS_PRAGMATICS.LIST_LENGTH) |
| VALUE_TYPE | AD_87 | (INTEGER) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_31 | COMPONENT_OF | (CONSTRAINTS_LIST_-MAXIMUM_SIZE) | AKD_14 |
| R | RD_28 | COMPONENT | (MAXIMUM_SIZE) | AKD_37 |
| S | RD_31 | COMPONENT_OF | (CONSTRAINTS_LIST_-MINIMUM_SIZE) | AKD_14 |
| R | RD_28 | COMPONENT | (MINIMUM_SIZE) | AKD_37 |
| S | RD_18 | ATTRIBUTE_KIND_OF | (MAXIMUM_SIZE) | AD_54 |
| S | RD_18 | ATTRIBUTE_KIND_OF | (MINIMUM_SIZE) | AD_56 |
| S | RD_16 | PARENT | () | N_2 |

### 20.3.38 'ATTRIBUTE_KIND_DEFINITION( MAXIMUM_LINE_LENGTH )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| AKD_38 | MAXIMUM_LINE_LENGTH | NKD_16 | INTEGER_ATTRIBUTE_KIND_-DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("Instances specify the maximum length of a line for the CAIS_TEXT_IO protocol.") |
| LOWER_BOUND | AD_47 | (0) |
| UPPER_BOUND | AD_86 | (CAIS_PRAGMATICS.TEXT_IO_COLUMNS_PER_LINE) |
| VALUE_TYPE | AD_87 | (INTEGER) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_18 | ATTRIBUTE_KIND_OF | (MAXIMUM_LINE_LENGTH) | AD_52 |
| S | RD_16 | PARENT | () | N_2 |



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

### 20.3.39 'ATTRIBUTE_KIND_DEFINITION( MAXIMUM_PAGE_LENGTH )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| AKD_39 | MAXIMUM_PAGE_LENGTH | NKD_16 | INTEGER_ATTRIBUTE_KIND_-DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("Instances specify the length of a page in characters for the CAIS_TEXT_IO protocol.") |
| LOWER_BOUND | AD_47 | (0) |
| UPPER_BOUND | AD_86 | (CAIS_PRAGMATICS.TEXT_IO_LINES_PER_PAGE) |
| VALUE_TYPE | AD_87 | (INTEGER) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_18 | ATTRIBUTE_KIND_OF | (MAXIMUM_PAGE_LENGTH) | AD_53 |
| S | RD_16 | PARENT | () | N_2 |

### 20.3.40 'ATTRIBUTE_KIND_DEFINITION( MONTH )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| AKD_40 | MONTH | NKD_16 | INTEGER_ATTRIBUTE_KIND_-DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("Instances specify a month of the year as an integer.") |
| LOWER_BOUND | AD_47 | (1) |
| UPPER_BOUND | AD_86 | (12) |
| VALUE_TYPE | AD_87 | (INTEGER) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_32 | COMPONENT_OF | (TIME) | AKD_60 |
| S | RD_16 | PARENT | () | N_2 |

### 20.3.41 'ATTRIBUTE_KIND_DEFINITION( NAMING )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| AKD_41 | NAMING | NKD_15 | IDENTIFIER_ATTRIBUTE_KIND_-DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("Instances indicate whether a list is constrained to be named or unnamed.") |
| ENUMERATED | AD_21 | (FALSE) |
| PATTERNS | AD_67 | ("UNNAMED", "NAMED") |
| VALUE_TYPE | AD_87 | (IDENTIFIER) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_18 | ATTRIBUTE_KIND_OF | (NAMING) | AD_59 |
| S | RD_16 | PARENT | () | N_2 |



Source: https://assist.dla.mil -- Downloaded: 202610-26T07:54Z
Check the source to verify that this is the current version before use.

### 20.3.42 'ATTRIBUTE_KIND_DEFINITION( NON_HIERARCHICAL_CATEGORY )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| AKD_42 | NON_HIERARCHICAL_-CATEGORY | NKD_15 | IDENTIFIER_ATTRIBUTE_KIND_-DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("Instances are names of mandatory access control non-hierarchical categories.") |
| ENUMERATED | AD_21 | (FALSE) |
| PATTERNS | AD_67 | (implementation defined) |
| VALUE_TYPE | AD_87 | (IDENTIFIER) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_30 | COMPONENT_OF | (NON_HIERARCHICAL_-CATEGORY_LIST) | AKD_43 |
| S | RD_16 | PARENT | () | N_2 |

### 20.3.43 'ATTRIBUTE_KIND_DEFINITION( NON_HIERARCHICAL_CATEGORY_-LIST )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| AKD_43 | NON_HIERARCHICAL_-CATEGORY_LIST | NKD_39 | UNIFORM_LIST_ATTRIBUTE_-KIND_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("Instances are lists of mandatory access control non-hierarchical categories.") |
| MINIMUM_SIZE | AD_56 | (1) |
| NAMING | AD_59 | (UNNAMED) |
| VALUE_TYPE | AD_87 | (UNIFORM_LIST) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_27 | COMPONENT | () | AKD_42 |
| S | RD_33 | COMPONENT_OF | (CLASSIFICATION) | AKD_11 |
| S | RD_16 | PARENT | () | N_2 |

### 20.3.44 'ATTRIBUTE_KIND_DEFINITION( OPEN_NODE_HANDLE_COUNT )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| AKD_44 | OPEN_NODE_HANDLE_COUNT | NKD_16 | INTEGER_ATTRIBUTE_KIND_-DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("Instances specify the number of open node handles for a process.") |
| LOWER_BOUND | AD_47 | (0) |
| UPPER_BOUND | AD_86 | (CAIS_PRAGMATICS.NODE_HANDLES_PER_-PROCESS) |
| VALUE_TYPE | AD_87 | (INTEGER) |



Source: https://assist.dla.mil -- Downloaded: 202610-26T07:54Z
Check the source to verify that this is the current version before use.

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_18 | ATTRIBUTE_KIND_OF | (OPEN_NODE_HANDLE_COUNT) | AD_62 |
| S | RD_16 | PARENT | () | N_2 |

### 20.3.45 'ATTRIBUTE_KIND_DEFINITION( OUTGOING_FRAME_LENGTH_RANGE )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| AKD_45 | OUTGOING_FRAME_LENGTH_-RANGE | NKD_23 | NAMED_COMPOSITE_LIST_AKD |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("Instances specify the range of sizes for frames output by a client using the CAIS_FRAME_IO protocol.") |
| NAMING | AD_59 | (NAMED) |
| ORDERED | AD_64 | (FALSE) |
| VALUE_TYPE | AD_87 | (COMPOSITE_LIST) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_18 | ATTRIBUTE_KIND_OF | (OUTGOING_FRAME_LENGTH_-RANGE) | AD_65 |
| S | RD_28 | COMPONENT | (LOWER) | AKD_25 |
| A | AD_63 | OPTIONAL | (FALSE) | |
| R | RD_31 | COMPONENT_OF | (OUTGOING_FRAME_LENGTH_-LOWER) | AKD_45 |
| S | RD_28 | COMPONENT | (UPPER) | AKD_25 |
| A | AD_63 | OPTIONAL | (FALSE) | |
| R | RD_31 | COMPONENT_OF | (OUTGOING_FRAME_LENGHT_-UPPER) | AKD_45 |
| S | RD_16 | PARENT | () | N_2 |

### 20.3.46 'ATTRIBUTE_KIND_DEFINITION( PARAMETERS_KINDS )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| AKD_46 | PARAMETERS_KINDS | NKD_41 | VARIANT_ATTRIBUTE_KIND_-DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("Instances are parameter lists of processes.") |
| VALUE_TYPE | AD_87 | (VARIANT) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_8 | ALTERNATIVE | (UNCONSTRAINED_LIST) | AKD_62 |
| S | RD_18 | ATTRIBUTE_KIND_OF | (HANDLER_INPUT_-PARAMETERS) | AD_28 |
| S | RD_18 | ATTRIBUTE_KIND_OF | (PARAMETERS) | AD_66 |
| S | RD_16 | PARENT | () | N_2 |



Source: https://assist.dla.mil -- Downloaded: 202610-26T07:54Z
Check the source to verify that this is the current version before use.

### 20.3.47 'ATTRIBUTE_KIND_DEFINITION( PATTERN_LIST )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| AKD_47 | PATTERN_LIST | NKD_39 | UNIFORM_LIST_ATTRIBUTE_-KIND_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("Instances are lists of patterns which a string or identifier attribute value must match.") |
| NAMING | AD_59 | (UNNAMED) |
| VALUE_TYPE | AD_87 | (UNIFORM_LIST) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_18 | ATTRIBUTE_KIND_OF | (PATTERNS) | AD_67 · |
| S | RD_27 | COMPONENT | () | AKD_63 |
| S | RD_31 | COMPONENT_OF | (CONSTRAINTS_LIST) | AKD_14 |
| S | RD_16 | PARENT | () | N_2 |

### 20.3.48 'ATTRIBUTE_KIND_DEFINITION( PROCESS_PRIORITY )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| AKD_48 | PROCESS_PRIORITY | NKD_16 | INTEGER_ATTRIBUTE_KIND_-DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("Instances specify a process priority.") |
| LOWER_BOUND | AD_47 | (CAIS_PRAGMATICS.LOWEST_PRIORITY) |
| UPPER_BOUND | AD_86 | (CAIS_PRAGMATICS.HIGHEST_PRIORITY) |
| VALUE_TYPE | AD_87 | (INTEGER) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_18 | ATTRIBUTE_KIND_OF | (HANDLER_PRIORITY) | AD_31 |
| S | RD_18 | ATTRIBUTE_KIND_OF | (PRIORITY) | AD_69 |
| S | RD_16 | PARENT | () | N_2 |

### 20.3.49 'ATTRIBUTE_KIND_DEFINITION( PROCESS_STATUS_KIND )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| AKD_49 | CURRENT_STATUS | NKD_15 | IDENTIFIER_ATTRIBUTE_KIND_-DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("Instances indicate the current status of a process.") |
| ENUMERATED | AD_21 | (TRUE) |
| PATTERNS | AD_67 | ("CREATED", "READY", "SUSPENDED", "COMPLETED_ABNORMALLY", "COMPLETED_NORMALLY", "ABORTED", "TERMINATED") |
| VALUE_TYPE | AD_87 | (IDENTIFIER) |



Source: https://assist.dla.mil -- Downloaded: 202610-26T07:54Z
Check the source to verify that this is the current version before use.

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_18 | ATTRIBUTE_KIND_OF | (CURRENT_STATUS) | AD_13 |
| S | RD_16 | PARENT | () | N_2 |

### 20.3.50 'ATTRIBUTE_KIND_DEFINITION( PROTOCOL )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| AKD_50 | PROTOCOL | NKD_15 | IDENTIFIER_ATTRIBUTE_KIND_-DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("Instances designate an IO protocol.") |
| ENUMERATED | AD_21 | (FALSE) |
| PATTERNS | AD_67 | ("NO_PROTOCOL", "CAIS_FRAME_IO", "CAIS_MAGNETIC_TAPE_IO", "CAIS_DIRECT_IO", "CAIS_SEQUENTIAL_IO", "CAIS_TEXT_IO", "CAIS_TERMINAL_IO", "CAIS_FORM_TERMINAL_IO") |
| VALUE_TYPE | AD_87 | (IDENTIFIER) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_18 | ATTRIBUTE_KIND_OF | (CONTROLLING_PROTOCOL) | AD_11 |
| S | RD_30 | COMPONENT_OF | (PROTOCOL_LIST) | AKD_51 |
| S | RD_16 | PARENT | () | N_2 |

### 20.3.51 'ATTRIBUTE_KIND_DEFINITION( PROTOCOL_LIST )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| AKD_51 | PROTOCOL_LIST | NKD_39 | UNIFORM_LIST_ATTRIBUTE_-KIND_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("Instances are lists of IO protocols.") |
| MINIMUM_SIZE | AD_56 | (1) |
| NAMING | AD_59 | (UNNAMED) |
| VALUE_TYPE | AD_87 | (UNIFORM_LIST) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_18 | ATTRIBUTE_KIND_OF | (ALLOWED_PROTOCOLS) | AD_1 |
| S | RD_27 | COMPONENT | () | AKD_50 |
| S | RD_16 | PARENT | () | N_2 |

### 20.3.52 'ATTRIBUTE_KIND_DEFINITION( RANGE_CONSTRAINT )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| AKD_52 | RANGE_CONSTRAINT | NKD_41 | VARIANT_ATTRIBUTE_KIND_-DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("Instances specify bounds for an integer or floating point value.") |



Source: https://assist.dla.mil -- Downloaded: 202610-26T07:54Z
Check the source to verify that this is the current version before use.

| Attribute Name | AD | Attribute Value |
|---|---|---|
| VALUE_TYPE | AD_87 | (VARIANT) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_8 | ALTERNATIVE | (FLOAT_BOUND) | AKD_6 |
| S | RD_8 | ALTERNATIVE | (INTEGER_BOUND) | AKD_7 |
| S | RD_31 | COMPONENT_OF | (CONSTRAINTS_LIST_LOWER_-BOUND) | AKD_14 |
| R | RD_28 | COMPONENT | (LOWER_BOUND) | AKD_52 |
| S | RD_31 | COMPONENT_OF | (CONSTRAINTS_LIST_UPPER_-BOUND) | AKD_14 |
| R | RD_28 | COMPONENT | (UPPER_BOUND) | AKD_52 |
| S | RD_16 | PARENT | () | N_2 |

### 20.3.53 'ATTRIBUTE_KIND_DEFINITION( RELATIONSHIP_ATTRIBUTE_-DESIGNATOR )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| AKD_53 | RELATIONSHIP_ATTRIBUTE_-DESIGNATOR | NKD_23 | NAMED_COMPOSITE_LIST_AKD |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("Instances designate a relationship attribute which is being monitored by an attribute monitor.") |
| NAMING | AD_59 | (NAMED) |
| ORDERED | AD_64 | (FALSE) |
| VALUE_TYPE | AD_87 | (COMPOSITE_LIST) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_9 | ALTERNATIVE_OF | (ATTRIBUTE_DESIGNATOR) | AKD_3 |
| S | RD_28 | COMPONENT | (AD) | AKD_64 |
| A | AD_63 | OPTIONAL | (FALSE) | |
| R | RD_31 | COMPONENT_OF | (RELATIONSHIP_ATTRIBUTE_-DESIGNATOR_AD) | AKD_53 |
| S | RD_28 | COMPONENT | (KEYS) | AKD_35 |
| A | AD_63 | OPTIONAL | (FALSE) | |
| S | RD_28 | COMPONENT | (RD) | AKD_64 |
| A | AD_63 | OPTIONAL | (FALSE) | |
| R | RD_31 | COMPONENT_OF | (RELATIONSHIP_ATTRIBUTE_-DESIGNATOR_RD) | AKD_53 |
| S | RD_16 | PARENT | () | N_2 |

### 20.3.54 'ATTRIBUTE_KIND_DEFINITION( RELATIONSHIP_KEY_LIST )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| AKD_54 | RELATIONSHIP_KEY_LIST | NKD_35 | STRING_ATTRIBUTE_KIND_-DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("Instances are strings having the syntax of relationship key lists.") |



Source: https://assist.dla.mil -- Downloaded: 202610-26T07:54Z
Check the source to verify that this is the current version before use.

| Attribute Name | AD | Attribute Value |
|---|---|---|
| PATTERNS | AD_67 | ("*") |
| VALUE_TYPE | AD_87 | (STRING) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_18 | ATTRIBUTE_KIND_OF | (HANDLER_KEY) | AD_29 |
| S | RD_16 | PARENT | () | N_2 |

### 20.3.55 'ATTRIBUTE_KIND_DEFINITION( RELATIONSHIP_KIND )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| AKD_55 | RELATIONSHIP_KIND | NKD_15 | IDENTIFIER_ATTRIBUTE_KIND_-DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("Instances designate a kind of relationship.") |
| ENUMERATED | AD_21 | (FALSE) |
| PATTERNS | AD_67 | ("PRIMARY", "SECONDARY") |
| VALUE_TYPE | AD_87 | (IDENTIFIER) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_18 | ATTRIBUTE_KIND_OF | (ALLOWED_USAGE) | AD_2 |
| S | RD_18 | ATTRIBUTE_KIND_OF | (IMPORTED_RELATIONSHIP_-KIND) | AD_35 |
| S | RD_16 | PARENT | () | N_2 |

### 20.3.56 'ATTRIBUTE_KIND_DEFINITION( RESULTING_RIGHTS_LIST )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| AKD_56 | RESULTING_RIGHTS_LIST | NKD_41 | VARIANT_ATTRIBUTE_KIND_-DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("Instances specify rights granted by a GRANT attribute.") |
| VALUE_TYPE | AD_87 | (VARIANT) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_8 | ALTERNATIVE | (IDENTIFIER) | AKD_61 |
| S | RD_8 | ALTERNATIVE | (IDENTIFIER_LIST) | AKD_29 |
| S | RD_30 | COMPONENT_OF | (GRANT_ITEM) | AKD_27 |
| S | RD_16 | PARENT | () | N_2 |

### 20.3.57 'ATTRIBUTE_KIND_DEFINITION( RESULTS )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| AKD_57 | RESULTS | NKD_39 | UNIFORM_LIST_ATTRIBUTE_-KIND_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("Instances are lists of strings containing process results.") |
| VALUE_TYPE | AD_87 | (UNIFORM_LIST) |



Source: https://assist.dla.mil -- Downloaded: 202610-26T07:54Z
Check the source to verify that this is the current version before use.

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_18 | ATTRIBUTE_KIND_OF | (RESULTS) | AD_73 |
| S | RD_27 | COMPONENT | () | AKD_63 |
| S | RD_16 | PARENT | () | N_2 |

### 20.3.58 'ATTRIBUTE_KIND_DEFINITION( SMALL_DURATION_MULTIPLE )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| AKD_58 | SMALL_DURATION_MULTIPLE | NKD_16 | INTEGER_ATTRIBUTE_KIND_-DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("Instances represent elapsed time in integer multiples of CAIS_PRAGMATICS.SMALL_FOR_CAIS_DURATION") |
| LOWER_BOUND | AD_47 | (0) |
| UPPER_BOUND | AD_86 | (CAIS_PRAGMATICS.MAXIMUM_INTEGER) |
| VALUE_TYPE | AD_87 | (INTEGER) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_18 | ATTRIBUTE_KIND_OF | (MACHINE_TIME) | AD_49 |
| S | RD_32 | COMPONENT_OF | (TIME) | AKD_60 |
| S | RD_16 | PARENT | () | N_2 |

### 20.3.59 'ATTRIBUTE_KIND_DEFINITION( STRUCTURAL_KIND )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| AKD_59 | STRUCTURAL_KIND | NKD_15 | IDENTIFIER_ATTRIBUTE_KIND_-DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("Instances designate a kind of structural node.") |
| ENUMERATED | AD_21 | (FALSE) |
| PATTERNS | AD_67 | ("CHANNEL", "DEFINITION", "DEFINITIONS", "GROUP", "INTERPRETATION", "MEDIUM", "MONITOR", "PROCESSOR", "RESOURCE_GROUP", "SYSTEM_LEVEL", "USER", "USER_DEFINED", "VIEW") |
| VALUE_TYPE | AD_87 | (IDENTIFIER) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_18 | ATTRIBUTE_KIND_OF | (STRUCTURAL_KIND) | AD_76 |
| S | RD_16 | PARENT | () | N_2 |

### 20.3.60 'ATTRIBUTE_KIND_DEFINITION( TIME )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| AKD_60 | TIME | NKD_24 | NAMED_ORDERED_-COMPOSITE_LIST_AKD |


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("Instances represent a date and time.") |
| NAMING | AD_59 | (NAMED) |
| ORDERED | AD_64 | (TRUE) |
| VALUE_TYPE | AD_87 | (COMPOSITE_LIST) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_29 | COMPONENT | | AKD_66 |
| K | AD_58 | NAME | (YEAR) | |
| K | AD_68 | POSITION | (A) | |
| A | AD_63 | OPTIONAL | (FALSE) | |
| S | RD_29 | COMPONENT | | AKD_40 |
| K | AD_58 | NAME | (MONTH) | |
| K | AD_68 | POSITION | (B) | |
| A | AD_63 | OPTIONAL | (FALSE) | |
| S | RD_29 | COMPONENT | | AKD_15 |
| K | AD_58 | NAME | (DAY) | |
| K | AD_68 | POSITION | (C) | |
| A | AD_63 | OPTIONAL | (FALSE) | |
| S | RD_29 | COMPONENT | | AKD_58 |
| K | AD_58 | NAME | (SECONDS) | |
| K | AD_68 | POSITION | (D) | |
| A | AD_63 | OPTIONAL | (FALSE) | |
| S | RD_18 | ATTRIBUTE_KIND_OF | (TIME_ATTRIBUTE_WRITTEN) | AD_77 |
| S | RD_18 | ATTRIBUTE_KIND_OF | (TIME_CONTENTS_WRITTEN) | AD_78 |
| S | RD_18 | ATTRIBUTE_KIND_OF | (TIME_CREATED) | AD_79 |
| S | RD_18 | ATTRIBUTE_KIND_OF | (TIME_FINISHED) | AD_80 |
| S | RD_18 | ATTRIBUTE_KIND_OF | (TIME_RELATIONSHIP_-WRITTEN) | AD_81 |
| S | RD_18 | ATTRIBUTE_KIND_OF | (TIME_STARTED) | AD_82 |
| S | RD_16 | PARENT | () | N_2 |

### 20.3.61 'ATTRIBUTE_KIND_DEFINITION( UNCONSTRAINED_IDENTIFIER )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| AKD_61 | UNCONSTRAINED_IDENTIFIER | NKD_15 | IDENTIFIER_ATTRIBUTE_KIND_-DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("Instances are Ada identifiers. All key values conform to this AKD.") |
| ENUMERATED | AD_21 | (FALSE) |
| PATTERNS | AD_67 | ("?*") |
| VALUE_TYPE | AD_87 | (IDENTIFIER) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_9 | ALTERNATIVE_OF | (ITEM_VALUE) | AKD_33 |
| S | RD_9 | ALTERNATIVE_OF | (RESULTING_RIGHTS_LIST) | AKD_56 |
| S | RD_18 | ATTRIBUTE_KIND_OF | (CAIS_KEY) | AD_9 |
| S | RD_18 | ATTRIBUTE_KIND_OF | (GENERIC_KEY) | AD_23 |


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_18 | ATTRIBUTE_KIND_OF | (IMPORTED_NAME) | AD_34 |
| S | RD_18 | ATTRIBUTE_KIND_OF | (NAME_KEY) | AD_58 |
| S | RD_18 | ATTRIBUTE_KIND_OF | (POSITION_KEY) | AD_68 |
| S | RD_31 | COMPONENT_OF | (KEY_VALUE_DESIGNATOR) | AKD_34 |
| S | RD_30 | COMPONENT_OF | (IDENTIFIER_LIST) | AKD_29 |
| S | RD_16 | PARENT | () | N_2 |

### 20.3.62 'ATTRIBUTE_KIND_DEFINITION( UNCONSTRAINED_LIST )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| AKD_62 | UNCONSTRAINED_LIST | NKD_19 | LIST_ATTRIBUTE_KIND_-DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("Instances are unconstrained CAIS lists. Every attribute value conforms to this AKD.") |
| VALUE_TYPE | AD_87 | (LIST) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_9 | ALTERNATIVE_OF | (ANNOTATION_KINDS) | AKD_2 |
| S | RD_9 | ALTERNATIVE_OF | (ITEM_VALUE) | AKD_33 |
| S | RD_9 | ALTERNATIVE_OF | (PARAMETERS_KINDS) | AKD_46 |
| S | RD_18 | ATTRIBUTE_KIND_OF | (ATTRIBUTE_VALUE) | AD_4 |
| S | RD_18 | ATTRIBUTE_KIND_OF | (DEFAULT_VALUE) | AD_14 |
| S | RD_31 | COMPONENT_OF | (CONSTRAINTS_LIST) | AKD_14 |
| S | RD_31 | COMPONENT_OF | (INTRODUCED_ATTRIBUTE_-DESCRIPTION) | AKD_32 |
| S | RD_16 | PARENT | () | N_2 |

### 20.3.63 'ATTRIBUTE_KIND_DEFINITION( UNCONSTRAINED_STRING )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| AKD_63 | UNCONSTRAINED_STRING | NKD_35 | STRING_ATTRIBUTE_KIND_-DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("Instances are unconstrained string items. Every string attribute value conforms to this AKD.") |
| PATTERNS | AD_67 | ("*") |
| VALUE_TYPE | AD_87 | (STRING) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_9 | ALTERNATIVE_OF | (ITEM_VALUE) | AKD_33 |
| S | RD_30 | COMPONENT_OF | (PATTERN_LIST) | AKD_47 |
| S | RD_30 | COMPONENT_OF | (RESULTS) | AKD_57 |
| S | RD_16 | PARENT | () | N_2 |


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

### 20.3.64 'ATTRIBUTE_KIND_DEFINITION( UNIQUE_IDENTIFIER )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| AKD_64 | UNIQUE_IDENTIFIER | NKD_35 | STRING_ATTRIBUTE_KIND_-DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("Instances uniquely identify a node.") |
| PATTERNS | AD_67 | ("?*.?*") |
| VALUE_TYPE | AD_87 | (STRING) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_9 | ALTERNATIVE_OF | (ATTRIBUTE_DESIGNATOR) | AKD_3 |
| S | RD_18 | ATTRIBUTE_KIND_OF | (IMPORTED_DEFINITION_UID) | AD_33 |
| S | RD_18 | ATTRIBUTE_KIND_OF | (IMPORTED_TARGET_UID) | AD_36 |
| S | RD_18 | ATTRIBUTE_KIND_OF | (IMPORTED_UID) | AD_37 |
| S | RD_18 | ATTRIBUTE_KIND_OF | (UNIQUE_IDENTIFIER) | AD_83 |
| S | RD_31 | COMPONENT_OF | (INTRODUCED_ATTRIBUTE_-DESCRIPTION) | AKD_32 |
| S | RD_31 | COMPONENT_OF | (KEY_VALUE_DESIGNATOR) | AKD_34 |
| S | RD_31 | COMPONENT_OF | (RELATIONSHIP_ATTRIBUTE_-DESIGNATOR_AD) | AKD_53 |
| R | RD_28 | COMPONENT | (AD) | AKD_64 |
| S | RD_31 | COMPONENT_OF | (RELATIONSHIP_ATTRIBUTE_-DESIGNATOR_RD) | AKD_53 |
| R | RD_28 | COMPONENT | (RD) | AKD_64 |
| S | RD_16 | PARENT | () | N_2 |

### 20.3.65 'ATTRIBUTE_KIND_DEFINITION( VALUE_TYPE )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| AKD_65 | VALUE_TYPE | NKD_15 | IDENTIFIER_ATTRIBUTE_KIND_-DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("Instances designate a type of attribute value.") |
| ENUMERATED | AD_21 | (FALSE) |
| PATTERNS | AD_67 | ("INTEGER", "FLOAT", "STRING", "IDENTIFIER", "VARIANT", "LIST", "UNIFORM_LIST", "COMPOSITE_LIST") |
| VALUE_TYPE | AD_87 | (IDENTIFIER) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_18 | ATTRIBUTE_KIND_OF | (VALUE_TYPE) | AD_87 |
| S | RD_16 | PARENT | () | N_2 |

### 20.3.66 'ATTRIBUTE_KIND_DEFINITION( YEAR )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| AKD_66 | YEAR | NKD_16 | INTEGER_ATTRIBUTE_KIND_-DEFINITION |



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("Instances represent a year A.D.") |
| LOWER_BOUND | AD_47 | (1901) |
| UPPER_BOUND | AD_86 | (2099) |
| VALUE_TYPE | AD_87 | (INTEGER) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_32 | COMPONENT_OF | (TIME) | AKD_60 |
| S | RD_16 | PARENT | () | N_2 |



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

## 20.4 Node kind definition nodes (NKDs)

The following table describes the node attributes which have the same value on all NKDs:

| Attribute Name | AD | Attribute Value |
|---|---|---|
| KIND | AD_45 | (STRUCTURAL) |
| STRUCTURAL_KIND | AD_76 | (DEFINITION) |
| DEFINITION_KIND | AD_15 | (NODE_KIND) |
| CAIS_DETERMINED | AD_8 | (FALSE) |

The following subsections describe each predefined NKD. Each section name is the primary path element from 'DEFINITIONS( PREDEFINED ) to the NKD described.

### 20.4.1 'NODE_KIND_DEFINITION( ATTRIBUTE_DEFINITION )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| NKD_1 | ATTRIBUTE_DEFINITION | NKD_25 | NODE_KIND_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("NKD for ADs.") |
| CAIS_CONTROLLED | AD_7 | (TRUE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_10 | ATTRIBUTE | (DEFINITION_KIND) | AD_15 |
| A | AD_10 | CONSTRAINTS | (VALUE=>ATTRIBUTE) | |
| A | AD_14 | DEFAULT_VALUE | (ATTRIBUTE) | |
| S | RD_10 | ATTRIBUTE | (DELETABLE) | AD_16 |
| A | AD_14 | DEFAULT_VALUE | (TRUE) | |
| S | RD_10 | ATTRIBUTE | (MODIFIABLE) | AD_57 |
| A | AD_14 | DEFAULT_VALUE | (TRUE) | |
| S | RD_54 | EMANATES | (ATTRIBUTE_DEFINITION_-PARENT) | RD_13 |
| S | RD_54 | EMANATES | (ATTRIBUTE_KIND) | RD_14 |
| A | AD_51 | MAXIMUM_INSTANCES | (1) | |
| A | AD_55 | MINIMUM_INSTANCES | (1) | |
| S | RD_54 | EMANATES | (ATTRIBUTE_OF) | RD_22 |
| S | RD_54 | EMANATES | (KEY_OF) | RD_83 |
| S | RD_91 | PARENT | () | N_2 |
| S | RD_111 | SPECIALIZATION_OF | (DEFINITION) | NKD_7 |
| S | RD_116 | TERMINATES | (ATTRIBUTE) | RD_10 |
| S | RD_116 | TERMINATES | (ATTRIBUTE_DEFINITION) | RD_11 |
| S | RD_116 | TERMINATES | (ATTRIBUTE_DEFINITION_-NAME) | RD_12 |
| S | RD_116 | TERMINATES | (ATTRIBUTE_KIND_OF) | RD_18 |
| S | RD_116 | TERMINATES | (ATTRIBUTE_NAME) | RD_21 |
| S | RD_116 | TERMINATES | (HANDLER_ATTRIBUTE) | RD_65 |
| S | RD_116 | TERMINATES | (KEY) | RD_81 |
| S | RD_116 | TERMINATES | (KEY_NAME) | RD_82 |



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

### 20.4.2 'NODE_KIND_DEFINITION( ATTRIBUTE_KIND_DEFINITION )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| NKD_2 | ATTRIBUTE_KIND_DEFINITION | NKD_25 | NODE_KIND_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("This NKD is a generalization which defines properties common to the various kinds of AKDs.") |
| CAIS_CONTROLLED | AD_7 | (TRUE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_10 | ATTRIBUTE | (DEFINITION_KIND) | AD_15 |
| A | AD_10 | CONSTRAINTS | (VALUE=>ATTRIBUTE_KIND) | |
| A | AD_14 | DEFAULT_VALUE | (ATTRIBUTE_KIND) | |
| S | RD_10 | ATTRIBUTE | (VALUE_TYPE) | AD_87 |
| S | RD_54 | EMANATES | (ALTERNATIVE_OF) | RD_9 |
| S | RD_54 | EMANATES | (ATTRIBUTE_KIND_-DEFINITION_PARENT) | RD_16 |
| S | RD_54 | EMANATES | (ATTRIBUTE_KIND_OF) | RD_18 |
| S | RD_54 | EMANATES | (COMPONENT_OF) | RD_30 |
| S | RD_54 | EMANATES | (COMPONENT_OF_NAMED) | RD_31 |
| S | RD_54 | EMANATES | (COMPONENT_OF_NAMED_-ORDERED) | RD_32 |
| S | RD_54 | EMANATES | (COMPONENT_OF_ORDERED) | RD_33 |
| S | RD_54 | EMANATES | (COMPONENT_OF_UNIFORM_-LIST) | RD_34 |
| S | RD_60 | GENERALIZATION_OF | (COMPOSITE_LIST_-ATTRIBUTE_KIND_DEFINITION) | NKD_6 |
| S | RD_60 | GENERALIZATION_OF | (FLOAT_ATTRIBUTE_KIND_-DEFINITION) | NKD_12 |
| S | RD_60 | GENERALIZATION_OF | (IDENTIFIER_ATTRIBUTE_-KIND_DEFINITION) | NKD_15 |
| S | RD_60 | GENERALIZATION_OF | (INTEGER_ATTRIBUTE_KIND_-DEFINITION) | NKD_16 |
| S | RD_60 | GENERALIZATION_OF | (LIST_ATTRIBUTE_KIND_-DEFINITION) | NKD_19 |
| S | RD_60 | GENERALIZATION_OF | (STRING_ATTRIBUTE_KIND_-DEFINITION) | NKD_35 |
| S | RD_60 | GENERALIZATION_OF | (UNIFORM_LIST_ATTRIBUTE_-KIND_DEFINITION) | NKD_39 |
| S | RD_60 | GENERALIZATION_OF | (VARIANT_ATTRIBUTE_KIND_-DEFINITION) | NKD_41 |
| S | RD_91 | PARENT | () | N_2 |
| S | RD_111 | SPECIALIZATION_OF | (DEFINITION) | NKD_7 |
| S | RD_116 | TERMINATES | (ALTERNATIVE) | RD_8 |
| S | RD_116 | TERMINATES | (ATTRIBUTE_KIND) | RD_14 |
| S | RD_116 | TERMINATES | (ATTRIBUTE_KIND_-DEFINITION) | RD_15 |
| S | RD_116 | TERMINATES | (ATTRIBUTE_KIND_NAME) | RD_17 |
| S | RD_116 | TERMINATES | (COMPONENT) | RD_27 |
| S | RD_116 | TERMINATES | (COMPONENT_NAMED) | RD_28 |
| S | RD_116 | TERMINATES | (COMPONENT_NAMED_-ORDERED) | RD_29 |
| S | RD_116 | TERMINATES | (COMPONENT_ORDERED) | RD_35 |



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

### 20.4.3 'NODE_KIND_DEFINITION( BASIC )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| NKD_3 | BASIC | NKD_25 | NODE_KIND_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("Most general NKD; all other NKDs are specializations of this NKD") |
| CAIS_CONTROLLED | AD_7 | (FALSE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_10 | ATTRIBUTE | (KIND) | AD_45 |
| S | RD_10 | ATTRIBUTE | (OBJECT_CLASSIFICATION) | AD_61 |
| S | RD_10 | ATTRIBUTE | (TIME_ATTRIBUTE_WRITTEN) | AD_77 |
| S | RD_10 | ATTRIBUTE | (TIME_CREATED) | AD_79 |
| S | RD_10 | ATTRIBUTE | (TIME_RELATIONSHIP_-WRITTEN) | AD_81 |
| S | RD_10 | ATTRIBUTE | (UNIQUE_IDENTIFIER) | AD_83 |
| S | RD_54 | EMANATES | (ACCESS) | RD_1 |
| S | RD_54 | EMANATES | (BASIC) | RD_23 |
| S | RD_54 | EMANATES | (CHANNEL) | RD_24 |
| S | RD_54 | EMANATES | (COPY_OF) | RD_37 |
| S | RD_54 | EMANATES | (DEFINITIONS_PRIMARY) | RD_48 |
| S | RD_54 | EMANATES | (DENY) | RD_50 |
| S | RD_54 | EMANATES | (INTRODUCED_NODE) | RD_76 |
| S | RD_54 | EMANATES | (MONITORED_BY) | RD_87 |
| S | RD_54 | EMANATES | (PARENT) | RD_93 |
| S | RD_54 | EMANATES | (VIEW_PRIMARY) | RD_122 |
| S | RD_60 | GENERALIZATION_OF | (CLIENT) | NKD_5 |
| S | RD_60 | GENERALIZATION_OF | (FILE) | NKD_11 |
| S | RD_60 | GENERALIZATION_OF | (PROCESS) | NKD_28 |
| S | RD_60 | GENERALIZATION_OF | (RESOURCE) | NKD_32 |
| S | RD_60 | GENERALIZATION_OF | (STRUCTURAL) | NKD_36 |
| S | RD_91 | PARENT | () | N_2 |
| S | RD_116 | TERMINATES | (BASIC) | RD_23 |
| S | RD_116 | TERMINATES | (CHANNEL_PARENT) | RD_25 |
| S | RD_116 | TERMINATES | (COPY_OF) | RD_37 |
| S | RD_116 | TERMINATES | (CURRENT_NODE) | RD_39 |
| S | RD_116 | TERMINATES | (DEFINITIONS_PARENT) | RD_47 |
| S | RD_116 | TERMINATES | (HANDLER_ENVIRONMENT_-NODE) | RD_66 |
| S | RD_116 | TERMINATES | (INTRODUCED_NODE_PARENT) | RD_77 |
| S | RD_116 | TERMINATES | (MONITORS) | RD_88 |
| S | RD_116 | TERMINATES | (PARENT) | RD_93 |
| S | RD_116 | TERMINATES | (VIEW_PARENT) | RD_121 |

### 20.4.4 'NODE_KIND_DEFINITION( CHANNEL )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| NKD_4 | CHANNEL | NKD_25 | NODE_KIND_DEFINITION |



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("NKD for CHANNEL nodes.") |
| CAIS_CONTROLLED | AD_7 | (TRUE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_10 | ATTRIBUTE | (CONTROLLING_PROTOCOL) | AD_11 |
| S | RD_10 | ATTRIBUTE | (ELEMENT_SIZE) | AD_20 |
| S | RD_10 | ATTRIBUTE | (MAXIMUM_LINE_LENGTH) | AD_52 |
| S | RD_10 | ATTRIBUTE | (MAXIMUM_PAGE_LENGTH) | AD_53 |
| S | RD_10 | ATTRIBUTE | (STATE) | AD_75 |
| S | RD_10 | ATTRIBUTE | (STRUCTURAL_KIND) | AD_76 |
| A | AD_10 | CONSTRAINTS ' | (VALUE=>CHANNEL) | |
| A | AD_14 | DEFAULT_VALUE | (CHANNEL) | |
| S | RD_54 | EMANATES | (CHANNEL_PARENT) | RD_25 |
| S | RD_54 | EMANATES | (CLIENT) | RD_26 |
| S | RD_91 | PARENT | () | N_2 |
| S | RD_111 | SPECIALIZATION_OF | (STRUCTURAL) | NKD_36 |
| S | RD_116 | TERMINATES | (CHANNEL) | RD_24 |
| S | RD_116 | TERMINATES | (PORT) | RD_96 |

### 20.4.5 'NODE_KIND_DEFINITION( CLIENT )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| NKD_5 | CLIENT | NKD_25 | NODE_KIND_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("NKD for CLIENT nodes. Defines properties common to IO clients.") |
| CAIS_CONTROLLED | AD_7 | (FALSE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_54 | EMANATES | (PORT) | RD_96 |
| S | RD_60 | GENERALIZATION_OF | (FILE) | NKD_11 |
| S | RD_60 | GENERALIZATION_OF | (PROCESS) | NKD_28 |
| S | RD_91 | PARENT | () | N_2 |
| S | RD_111 | SPECIALIZATION_OF | (BASIC) | NKD_3 |
| S | RD_116 | TERMINATES | (CLIENT) | RD_26 |

### 20.4.6 'NODE_KIND_DEFINITION( COMPOSITE_LIST_ATTRIBUTE_KIND_DEFINITION )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| NKD_6 | COMPOSITE_LIST_ATTRIBUTE_KIND_DEFINITION | NKD_25 | NODE_KIND_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("NKD for Composite List AKDs.") |
| CAIS_CONTROLLED | AD_7 | (TRUE) |



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_10 | ATTRIBUTE | (NAMING) | AD_59 |
| S | RD_10 | ATTRIBUTE | (ORDERED) | AD_64 |
| S<br>A<br>A | RD_10<br>AD_10<br>AD_14 | ATTRIBUTE<br>CONSTRAINTS<br>DEFAULT_VALUE | (VALUE_TYPE)<br>(VALUE=>COMPOSITE_LIST)<br>(COMPOSITE_LIST) | AD_87 |
| S | RD_54 | EMANATES | (COMPONENT) | RD_27 |
| S | RD_60 | GENERALIZATION_OF | (NAMED_COMPOSITE_LIST_-AKD) | NKD_23 |
| S | RD_60 | GENERALIZATION_OF | (ORDERED_COMPOSITE_LIST_-AKD) | NKD_27 |
| S | RD_91 | PARENT | () | N_2 |
| S | RD_111 | SPECIALIZATION_OF | (ATTRIBUTE_KIND_-DEFINITION) | NKD_2 |
| S | RD_116 | TERMINATES | (COMPONENT_OF) | RD_30 |

### 20.4.7 'NODE_KIND_DEFINITION( DEFINITION )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| NKD_7 | DEFINITION | NKD_25 | NODE_KIND_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("This NKD is a generalization which defines properties of definition nodes.") |
| CAIS_CONTROLLED | AD_7 | (TRUE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_10 | ATTRIBUTE | (ANNOTATION) | AD_3 |
| S | RD_10 | ATTRIBUTE | (CAIS_CONTROLLED) | AD_7 |
| S | RD_10 | ATTRIBUTE | (CAIS_DETERMINED) | AD_8 |
| S | RD_10 | ATTRIBUTE | (DEFINITION_KIND) | AD_15 |
| S<br>A<br>A | RD_10<br>AD_10<br>AD_14 | ATTRIBUTE<br>CONSTRAINTS<br>DEFAULT_VALUE | (STRUCTURAL_KIND)<br>(VALUE=>DEFINITION)<br>(DEFINITION) | AD_76 |
| S | RD_60 | GENERALIZATION_OF | (ATTRIBUTE_DEFINITION) | NKD_1 |
| S | RD_60 | GENERALIZATION_OF | (ATTRIBUTE_KIND_-DEFINITION) | NKD_2 |
| S | RD_60 | GENERALIZATION_OF | (NODE_KIND_DEFINITION) | NKD_25 |
| S | RD_60 | GENERALIZATION_OF | (RELATION_DEFINITION) | NKD_30 |
| S | RD_91 | PARENT | () | N_2 |
| S | RD_111 | SPECIALIZATION_OF | (STRUCTURAL) | NKD_36 |

### 20.4.8 'NODE_KIND_DEFINITION( DEFINITIONS )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| NKD_8 | DEFINITIONS | NKD_25 | NODE_KIND_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("NKD for a STUCTURAL node which groups DEFINITION nodes.") |
| CAIS_CONTROLLED | AD_7 | (FALSE) |



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_10 | ATTRIBUTE | (ANNOTATION) | AD_3 |
| S<br>A | RD_10<br>AD_14 | ATTRIBUTE<br>DEFAULT_VALUE | (STRUCTURAL_KIND)<br>(DEFINITIONS) | AD_76 |
| S | RD_54 | EMANATES | (ATTRIBUTE_DEFINITION) | RD_11 |
| S | RD_54 | EMANATES | (ATTRIBUTE_KIND_-<br>DEFINITION) | RD_15 |
| S | RD_54 | EMANATES | (DEFINITIONS_PARENT) | RD_47 |
| S | RD_54 | EMANATES | (NODE_KIND_DEFINITION) | RD_90 |
| S | RD_54 | EMANATES | (RELATION_DEFINITION) | RD_102 |
| S | RD_91 | PARENT | () | N_2 |
| S | RD_111 | SPECIALIZATION_OF | (STRUCTURAL) | NKD_36 |
| S | RD_116 | TERMINATES | (ATTRIBUTE_DEFINITION_-<br>PARENT) | RD_13 |
| S | RD_116 | TERMINATES | (ATTRIBUTE_KIND_-<br>DEFINITION_PARENT) | RD_16 |
| S | RD_116 | TERMINATES | (DEFINITIONS_PRIMARY) | RD_48 |
| S | RD_116 | TERMINATES | (DEFINITIONS_SECONDARY) | RD_49 |
| S | RD_116 | TERMINATES | (NODE_KIND_DEFINITION_-<br>PARENT) | RD_91 |
| S | RD_116 | TERMINATES | (RELATION_DEFINITION_-<br>PARENT) | RD_103 |

### 20.4.9 'NODE_KIND_DEFINITION( DEVICE )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| NKD_9 | DEVICE | NKD_25 | NODE_KIND_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("NKD for DEVICE nodes.") |
| CAIS_CONTROLLED | AD_7 | (TRUE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_10 | ATTRIBUTE | (DEVICE_KIND) | AD_17 |
| S<br>A<br>A | RD_10<br>AD_10<br>AD_14 | ATTRIBUTE<br>CONSTRAINTS<br>DEFAULT_VALUE | (FILE_KIND)<br>(VALUE=>DEVICE)<br>(DEVICE) | AD_22 |
| S | RD_10 | ATTRIBUTE | (HIGHEST_CLASSIFICATION) | AD_32 |
| S | RD_10 | ATTRIBUTE | (LOWEST_CLASSIFICATION) | AD_48 |
| S | RD_54 | EMANATES | (DEVICE_PARENT) | RD_51 |
| S | RD_60 | GENERALIZATION_OF | (MAGNETIC_TAPE) | NKD_20 |
| S | RD_60 | GENERALIZATION_OF | (TERMINAL) | NKD_38 |
| S | RD_91 | PARENT | () | N_2 |
| S | RD_111 | SPECIALIZATION_OF | (FILE) | NKD_11 |
| S | RD_111 | SPECIALIZATION_OF | (RESOURCE) | NKD_32 |
| S | RD_116 | TERMINATES | (ALLOCATABLE_DEVICE) | RD_4 |
| S | RD_116 | TERMINATES | (DEVICE_PRIMARY) | RD_52 |
| S | RD_116 | TERMINATES | (DEVICE_SECONDARY) | RD_53 |



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

### 20.4.10 'NODE_KIND_DEFINITION( EXECUTABLE_IMAGE )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| NKD_10 | EXECUTABLE_IMAGE | NKD_25 | NODE_KIND_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("NKD for EXECUTABLE_IMAGE nodes.") |
| CAIS_CONTROLLED | AD_7 | (FALSE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_54 | EMANATES | (DEFAULT_PROCESS_NODE_-KIND) | RD_44 |
| S | RD_54 | EMANATES | (DEFAULT_ROLE) | RD_45 |
| S | RD_54 | EMANATES | (VIEW_SECONDARY) | RD_123 |
| S | RD_91 | PARENT | () | N_2 |
| S | RD_111 | SPECIALIZATION_OF | (SECONDARY_STORAGE) | NKD_34 |
| S | RD_116 | TERMINATES | (EXECUTABLE_IMAGE) | RD_56 |
| S | RD_116 | TERMINATES | (HANDLER_IMAGE) | RD_68 |

### 20.4.11 'NODE_KIND_DEFINITION( FILE )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| NKD_11 | FILE | NKD_25 | NODE_KIND_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("NKD for FILE nodes.") |
| CAIS_CONTROLLED | AD_7 | (FALSE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_10 | ATTRIBUTE | (FILE_KIND) | AD_22 |
| S | RD_10 | ATTRIBUTE | (KIND) | AD_45 |
| A | AD_10 | CONSTRAINTS | (VALUE=>FILE) | |
| A | AD_14 | DEFAULT_VALUE | (FILE) | |
| S | RD_10 | ATTRIBUTE | (TIME_CONTENTS_WRITTEN) | AD_78 |
| S | RD_60 | GENERALIZATION_OF | (DEVICE) | NKD_9 |
| S | RD_60 | GENERALIZATION_OF | (GATEWAY) | NKD_13 |
| S | RD_60 | GENERALIZATION_OF | (SECONDARY_STORAGE) | NKD_34 |
| S | RD_91 | PARENT | () | N_2 |
| S | RD_111 | SPECIALIZATION_OF | (BASIC) | NKD_3 |
| S | RD_111 | SPECIALIZATION_OF | (CLIENT) | NKD_5 |
| S | RD_116 | TERMINATES | (HANDLER_ERROR_FILE) | RD_67 |
| S | RD_116 | TERMINATES | (HANDLER_INPUT_FILE) | RD_69 |
| S | RD_116 | TERMINATES | (HANDLER_OUTPUT_FILE) | RD_71 |
| S | RD_116 | TERMINATES | (STANDARD_ERROR) | RD_113 |
| S | RD_116 | TERMINATES | (STANDARD_INPUT) | RD_114 |
| S | RD_116 | TERMINATES | (STANDARD_OUTPUT) | RD_115 |



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

### 20.4.12 'NODE_KIND_DEFINITION( FLOAT_ATTRIBUTE_KIND_DEFINITION )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| NKD_12 | FLOAT_ATTRIBUTE_KIND_-DEFINITION | NKD_25 | NODE_KIND_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("NKD for Float AKDs.") |
| CAIS_CONTROLLED | AD_7 | (TRUE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_10 | ATTRIBUTE | (DIGITS) | AD_18 |
| A | AD_14 | DEFAULT_VALUE | (CAIS_PRAGMATICS.LIST_-MAXIMUM_DIGITS) | |
| S | RD_10 | ATTRIBUTE | (LOWER_BOUND) | AD_46 |
| S | RD_10 | ATTRIBUTE | (UPPER_BOUND) | AD_85 |
| S | RD_10 | ATTRIBUTE | (VALUE_TYPE) | AD_87 |
| A | AD_10 | CONSTRAINTS | (VALUE=>FLOAT) | |
| A | AD_14 | DEFAULT_VALUE | (FLOAT) | |
| S | RD_91 | PARENT | () | N_2 |
| S | RD_111 | SPECIALIZATION_OF | (ATTRIBUTE_KIND_-DEFINITION) | NKD_2 |

### 20.4.13 'NODE_KIND_DEFINITION( GATEWAY )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| NKD_13 | GATEWAY | NKD_25 | NODE_KIND_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("NKD for GATEWAY nodes.") |
| CAIS_CONTROLLED | AD_7 | (TRUE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_10 | ATTRIBUTE | (FILE_KIND) | AD_22 |
| A | AD_10 | CONSTRAINTS | (VALUE=>GATEWAY) | |
| A | AD_14 | DEFAULT_VALUE | (GATEWAY) | |
| S | RD_54 | EMANATES | (GATEWAY_PARENT) | RD_57 |
| S | RD_91 | PARENT | () | N_2 |
| S | RD_111 | SPECIALIZATION_OF | (FILE) | NKD_11 |
| S | RD_111 | SPECIALIZATION_OF | (RESOURCE) | NKD_32 |
| S | RD_116 | TERMINATES | (ALLOCATABLE_GATEWAY) | RD_5 |
| S | RD_116 | TERMINATES | (GATEWAY_PRIMARY) | RD_58 |
| S | RD_116 | TERMINATES | (GATEWAY_SECONDARY) | RD_59 |

### 20.4.14 'NODE_KIND_DEFINITION( GROUP )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| NKD_14 | GROUP | NKD_25 | NODE_KIND_DEFINITION |



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("NKD for GROUP nodes.") |
| CAIS_CONTROLLED | AD_7 | (TRUE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S<br>A<br>A | RD_10<br>AD_10<br>AD_14 | ATTRIBUTE<br>CONSTRAINTS<br>DEFAULT_VALUE | (STRUCTURAL_KIND)<br>(VALUE=>GROUP)<br>(GROUP) | AD_76 |
| S | RD_54 | EMANATES | (GROUP_PARENT) | RD_62 |
| S | RD_54 | EMANATES | (PERMANENT_MEMBER) | RD_94 |
| S | RD_54 | EMANATES | (PERMANENT_MEMBER_-PARENT) | RD_95 |
| S | RD_54 | EMANATES | (POTENTIAL_MEMBER) | RD_97 |
| S | RD_91 | PARENT | () | N_2 |
| S | RD_111 | SPECIALIZATION_OF | (STRUCTURAL) | NKD_36 |
| S | RD_116 | TERMINATES | (ACCESS) | RD_1 |
| S | RD_116 | TERMINATES | (ADOPTED_ROLE) | RD_2 |
| S | RD_116 | TERMINATES | (DEFAULT_ROLE) | RD_45 |
| S | RD_116 | TERMINATES | (DENY) | RD_50 |
| S | RD_116 | TERMINATES | (GROUP_PRIMARY) | RD_63 |
| S | RD_116 | TERMINATES | (GROUP_SECONDARY) | RD_64 |
| S | RD_116 | TERMINATES | (PERMANENT_MEMBER) | RD_94 |
| S | RD_116 | TERMINATES | (PERMANENT_MEMBER_-PARENT) | RD_95 |
| S | RD_116 | TERMINATES | (POTENTIAL_MEMBER) | RD_97 |

### 20.4.15 'NODE_KIND_DEFINITION( IDENTIFIER_ATTRIBUTE_KIND_DEFINITION

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| NKD_15 | IDENTIFIER_ATTRIBUTE_KIND_-DEFINITION | NKD_25 | NODE_KIND_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("NKD for Identifier AKDs.") |
| CAIS_CONTROLLED | AD_7 | (TRUE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S<br>A | RD_10<br>AD_14 | ATTRIBUTE<br>DEFAULT_VALUE | (ENUMERATED)<br>(FALSE) | AD_21 |
| S<br>A | RD_10<br>AD_14 | ATTRIBUTE<br>DEFAULT_VALUE | (PATTERNS)<br>("*") | AD_67 |
| S<br>A<br>A | RD_10<br>AD_10<br>AD_14 | ATTRIBUTE<br>CONSTRAINTS<br>DEFAULT_VALUE | (VALUE_TYPE)<br>(VALUE=>IDENTIFIER)<br>(IDENTIFIER) | AD_87 |
| S | RD_91 | PARENT | () | N_2 |
| S | RD_111 | SPECIALIZATION_OF | (ATTRIBUTE_KIND_-DEFINITION) | NKD_2 |



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

### 20.4.16 'NODE_KIND_DEFINITION( INTEGER_ATTRIBUTE_KIND_DEFINITION )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| NKD_16 | INTEGER_ATTRIBUTE_KIND_-DEFINITION | NKD_25 | NODE_KIND_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("NKD for Integer AKDs.") |
| CAIS_CONTROLLED | AD_7 | (TRUE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_10 | ATTRIBUTE | (LOWER_BOUND) | AD_47 |
| S | RD_10 | ATTRIBUTE | (UPPER_BOUND) | AD_86 |
| S | RD_10 | ATTRIBUTE | (VALUE_TYPE) | AD_87 |
| A | AD_10 | CONSTRAINTS | (VALUE=>INTEGER) | |
| A | AD_14 | DEFAULT_VALUE | (INTEGER) | |
| S | RD_91 | PARENT | () | N_2 |
| S | RD_111 | SPECIALIZATION_OF | (ATTRIBUTE_KIND_-DEFINITION) | NKD_2 |

### 20.4.17 'NODE_KIND_DEFINITION( INTERPRETATION )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| NKD_17 | INTERPRETATION | NKD_25 | NODE_KIND_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("This NKD is a generalization which defines the properties common to interpretation nodes (NKIs and RIs).") |
| CAIS_CONTROLLED | AD_7 | (TRUE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_10 | ATTRIBUTE | (ANNOTATION) | AD_3 |
| S | RD_10 | ATTRIBUTE | (INTERPRETATION_KIND) | AD_40 |
| S | RD_10 | ATTRIBUTE | (STRUCTURAL_KIND) | AD_76 |
| A | AD_10 | CONSTRAINTS | (VALUE=>INTERPRETATION) | |
| A | AD_14 | DEFAULT_VALUE | (INTERPRETATION) | |
| S | RD_54 | EMANATES | (ATTRIBUTE_NAME) | RD_21 |
| S | RD_54 | EMANATES | (DEFINITION) | RD_46 |
| S | RD_60 | GENERALIZATION_OF | (NODE_KIND_-INTERPRETATION) | NKD_26 |
| S | RD_60 | GENERALIZATION_OF | (RELATION_INTERPRETATION) | NKD_31 |
| S | RD_91 | PARENT | () | N_2 |
| S | RD_111 | SPECIALIZATION_OF | (STRUCTURAL) | NKD_36 |

### 20.4.18 'NODE_KIND_DEFINITION( INTRODUCED_NODE )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| NKD_18 | INTRODUCED_NODE | NKD_25 | NODE_KIND_DEFINITION |



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("NKD for INTRODUCED_NODE nodes.") |
| CAIS_CONTROLLED | AD_7 | (TRUE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_10 | ATTRIBUTE | (IMPORTED_DEFINITION_UID) | AD_33 |
| S | RD_10 | ATTRIBUTE | (IMPORTED_UID) | AD_37 |
| S | RD_10 | ATTRIBUTE | (INTRODUCED_NODE_-ATTRIBUTES) | AD_42 |
| S | RD_54 | EMANATES | (INTRODUCED_NODE_PARENT) | RD_77 |
| S | RD_54 | EMANATES | (INTRODUCED_RELATION) | RD_78 |
| S | RD_91 | PARENT | () | N_2 |
| S | RD_111 | SPECIALIZATION_OF | (SECONDARY_STORAGE) | NKD_34 |
| S | RD_116 | TERMINATES | (INTRODUCED_NODE) | RD_76 |
| S | RD_116 | TERMINATES | (INTRODUCED_RELATION) | RD_78 |

### 20.4.19 'NODE_KIND_DEFINITION( LIST_ATTRIBUTE_KIND_DEFINITION )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| NKD_19 | LIST_ATTRIBUTE_KIND_-DEFINITION | NKD_25 | NODE_KIND_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("NKD for List AKDs.") |
| CAIS_CONTROLLED | AD_7 | (TRUE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_10 | ATTRIBUTE | (NAMING) | AD_59 |
| S<br>A<br>A | RD_10<br>AD_10<br>AD_14 | ATTRIBUTE<br>CONSTRAINTS<br>DEFAULT_VALUE | (VALUE_TYPE)<br>(VALUE=>LIST)<br>(LIST) | AD_87 |
| S | RD_91 | PARENT | () | N_2 |
| S | RD_111 | SPECIALIZATION_OF | (ATTRIBUTE_KIND_-DEFINITION) | NKD_2 |

### 20.4.20 'NODE_KIND_DEFINITION( MAGNETIC_TAPE )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| NKD_20 | MAGNETIC_TAPE | NKD_25 | NODE_KIND_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("NKD for MAGNETIC_TAPE device nodes.") |
| CAIS_CONTROLLED | AD_7 | (TRUE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S<br>A<br>A | RD_10<br>AD_10<br>AD_14 | ATTRIBUTE<br>CONSTRAINTS<br>DEFAULT_VALUE | (DEVICE_KIND)<br>(VALUE=>MAGNETIC_TAPE)<br>(MAGNETIC_TAPE) | AD_17 |
| S | RD_91 | PARENT | () | N_2 |
| S | RD_111 | SPECIALIZATION_OF | (DEVICE) | NKD_9 |



Source: https://assist.dla.mil -- Downloaded: 2026l0-26T07:54Z
Check the source to verify that this is the currentversion before use.

### 20.4.21 'NODE_KIND_DEFINITION( MEDIUM )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| NKD_21 | MEDIUM | NKD_25 | NODE_KIND_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("NKD for MEDIUM nodes.") |
| CAIS_CONTROLLED | AD_7 | (TRUE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_10 | ATTRIBUTE | (STRUCTURAL_KIND) | AD_76 |
| A | AD_10 | CONSTRAINTS | (VALUE=>MEDIUM) | |
| A | AD_14 | DEFAULT_VALUE | (MEDIUM) | |
| S | RD_54 | EMANATES | (MEDIUM_PARENT) | RD_84 |
| S | RD_91 | PARENT | () | N_2 |
| S | RD_111 | SPECIALIZATION_OF | (RESOURCE) | NKD_32 |
| S | RD_111 | SPECIALIZATION_OF | (STRUCTURAL) | NKD_36 |
| S | RD_116 | TERMINATES | (ALLOCATABLE_MEDIUM) | RD_6 |
| S | RD_116 | TERMINATES | (DEFAULT_MEDIUM) | RD_42 |
| S | RD_116 | TERMINATES | (HANDLER_MEDIUM_-RESOURCE) | RD_70 |
| S | RD_116 | TERMINATES | (MEDIUM_PRIMARY) | RD_85 |
| S | RD_116 | TERMINATES | (MEDIUM_SECONDARY) | RD_86 |

### 20.4.22 'NODE_KIND_DEFINITION( MONITOR )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| NKD_22 | MONITOR | NKD_25 | NODE_KIND_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("NKD for MONITOR nodes.") |
| CAIS_CONTROLLED | AD_7 | (TRUE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_10 | ATTRIBUTE | (STRUCTURAL_KIND) | AD_76 |
| A | AD_10 | CONSTRAINTS | (VALUE=>MONITOR) | |
| A | AD_14 | DEFAULT_VALUE | (MONITOR) | |
| S | RD_10 | ATTRIBUTE | (HANDLER_AS_A_-TRANSACTION) | AD_25 |
| S | RD_10 | ATTRIBUTE | (HANDLER_DELETE_WHEN_-TERMINATED) | AD_26 |
| S | RD_10 | ATTRIBUTE | (HANDLER_DISCRETIONARY_-ACCESS) | AD_27 |
| S | RD_10 | ATTRIBUTE | (HANDLER_INPUT_-PARAMETERS) | AD_28 |
| S | RD_10 | ATTRIBUTE | (HANDLER_KEY) | AD_29 |
| S | RD_10 | ATTRIBUTE | (HANDLER_MANDATORY_-ACCESS) | AD_30 |
| S | RD_10 | ATTRIBUTE | (HANDLER_PRIORITY) | AD_31 |
| S | RD_54 | EMANATES | (ATTRIBUTE_MONITOR_-PARENT) | RD_20 |
| S | RD_54 | EMANATES | (HANDLER_ATTRIBUTE) | RD_65 |



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_54 | EMANATES | (HANDLER_ENVIRONMENT_-NODE) | RD_66 |
| S | RD_54 | EMANATES | (HANDLER_ERROR_FILE) | RD_67 |
| S | RD_54 | EMANATES | (HANDLER_IMAGE) | RD_68 |
| S | RD_54 | EMANATES | (HANDLER_INPUT_FILE) | RD_69 |
| S | RD_54 | EMANATES | (HANDLER_MEDIUM_-RESOURCE) | RD_70 |
| S | RD_54 | EMANATES | (HANDLER_OUTPUT_FILE) | RD_71 |
| S | RD_54 | EMANATES | (HANDLER_PROCESS) | RD_72 |
| S | RD_54 | EMANATES | (HANDLER_PROCESS_NODE_-KIND) | RD_73 |
| S | RD_54 | EMANATES | (HANDLER_PROCESSOR_-RESOURCE) | RD_75 |
| S | RD_54 | EMANATES | (MONITORS) | RD_88 |
| S | RD_91 | PARENT | () | N_2 |
| S | RD_111 | SPECIALIZATION_OF | (STRUCTURAL) | NKD_36 |
| S | RD_116 | TERMINATES | (ATTRIBUTE_MONITOR) | RD_19 |
| S | RD_116 | TERMINATES | (HANDLER_PROCESS_OF) | RD_74 |
| S | RD_116 | TERMINATES | (MONITORED_BY) | RD_87 |

### 20.4.23 'NODE_KIND_DEFINITION( NAMED_COMPOSITE_LIST_AKD )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| NKD_23 | NAMED_COMPOSITE_LIST_AKD | NKD_25 | NODE_KIND_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("NKD for Named Composite List AKDs.") |
| CAIS_CONTROLLED | AD_7 | (TRUE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_10 | ATTRIBUTE | (NAMING) | AD_59 |
| A | AD_10 | CONSTRAINTS | (VALUE=>NAMED) | |
| A | AD_14 | DEFAULT_VALUE | (NAMED) | |
| S | RD_54 | EMANATES | (COMPONENT_NAMED) | RD_28 |
| S | RD_60 | GENERALIZATION_OF | (NAMED_ORDERED_-COMPOSITE_LIST_AKD) | NKD_24 |
| S | RD_91 | PARENT | () | N_2 |
| S | RD_111 | SPECIALIZATION_OF | (COMPOSITE_LIST_-ATTRIBUTE_KIND_DEFINITION) | NKD_6 |
| S | RD_116 | TERMINATES | (COMPONENT_OF_NAMED) | RD_31 |

### 20.4.24 'NODE_KIND_DEFINITION( NAMED_ORDERED_COMPOSITE_LIST_AKD )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| NKD_24 | NAMED_ORDERED_-COMPOSITE_LIST_AKD | NKD_25 | NODE_KIND_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("NKD for Named and Ordered Composite List AKDs.") |
| CAIS_CONTROLLED | AD_7 | (TRUE) |



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_54 | EMANATES | (COMPONENT_NAMED_-ORDERED) | RD_29 |
| S | RD_91 | PARENT | () | N_2 |
| S | RD_111 | SPECIALIZATION_OF | (NAMED_COMPOSITE_LIST_-AKD) | NKD_23 |
| S | RD_111 | SPECIALIZATION_OF | (ORDERED_COMPOSITE_LIST_-AKD) | NKD_27 |
| S | RD_116 | TERMINATES | (COMPONENT_OF_NAMED_-ORDERED) | RD_32 |

### 20.4.25 'NODE_KIND_DEFINITION( NODE_KIND_DEFINITION )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| NKD_25 | NODE_KIND_DEFINITION | NKD_25 | NODE_KIND_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("NKD for NKDs.") |
| CAIS_CONTROLLED | AD_7 | (TRUE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_10 | ATTRIBUTE | (DEFINITION_KIND) | AD_15 |
| A | AD_10 | CONSTRAINTS | (VALUE=>NODE_KIND) | |
| A | AD_14 | DEFAULT_VALUE | (NODE_KIND) | |
| S | RD_54 | EMANATES | (ATTRIBUTE) | RD_10 |
| S | RD_54 | EMANATES | (EMANATES) | RD_54 |
| S | RD_54 | EMANATES | (GENERALIZATION_OF_NKD) | RD_60 |
| S | RD_54 | EMANATES | (NODE_KIND_DEFINITION_-PARENT) | RD_91 |
| S | RD_54 | EMANATES | (SPECIALIZATION_OF_NKD) | RD_111 |
| S | RD_54 | EMANATES | (TERMINATES) | RD_116 |
| S | RD_91 | PARENT | () | N_2 |
| S | RD_111 | SPECIALIZATION_OF | (DEFINITION) | NKD_7 |
| S | RD_116 | TERMINATES | (ATTRIBUTE_OF) | RD_22 |
| S | RD_116 | TERMINATES | (DEFAULT_PROCESS_NODE_-KIND) | RD_44 |
| S | RD_116 | TERMINATES | (DEFINITION) | RD_46 |
| S | RD_116 | TERMINATES | (EMANATES_FROM) | RD_55 |
| S | RD_116 | TERMINATES | (GENERALIZATION_OF_NKD) | RD_60 |
| S | RD_116 | TERMINATES | (HANDLER_PROCESS_NODE_-KIND) | RD_73 |
| S | RD_116 | TERMINATES | (NODE_KIND_DEFINITION) | RD_90 |
| S | RD_116 | TERMINATES | (SPECIALIZATION_OF_NKD) | RD_111 |
| S | RD_116 | TERMINATES | (TERMINATES_AT) | RD_117 |

### 20.4.26 'NODE_KIND_DEFINITION( NODE_KIND_INTERPRETATION )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| NKD_26 | NODE_KIND_INTERPRETATION | NKD_25 | NODE_KIND_DEFINITION |



Source: https://assist.dla.mil -- Downloaded: 202610-26T07:54Z
Check the source to verify that this is the current version before use.

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("NKD for NKIs.") |
| CAIS_CONTROLLED | AD_7 | (TRUE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_10 | ATTRIBUTE | (INTERPRETATION_KIND) | AD_40 |
| A | AD_10 | CONSTRAINTS | (VALUE=>NODE_KIND) | |
| A | AD_14 | DEFAULT_VALUE | (NODE_KIND) | |
| S | RD_54 | EMANATES | (NODE_KIND_PARENT) | RD_92 |
| S | RD_54 | EMANATES | (RELATION_NAME) | RD_104 |
| S | RD_91 | PARENT | () | N_2 |
| S | RD_111 | SPECIALIZATION_OF | (INTERPRETATION) | NKD_17 |
| S | RD_116 | TERMINATES | (NODE_KIND) | RD_89 |

### 20.4.27 'NODE_KIND_DEFINITION( ORDERED_COMPOSITE_LIST_AKD )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| NKD_27 | ORDERED_COMPOSITE_LIST_-AKD | NKD_25 | NODE_KIND_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("NKD for Ordered Composite List AKDs.") |
| CAIS_CONTROLLED | AD_7 | (TRUE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_10 | ATTRIBUTE | (ORDERED) | AD_64 |
| A | AD_10 | CONSTRAINTS | (VALUE=>TRUE) | |
| A | AD_14 | DEFAULT_VALUE | (TRUE) | |
| S | RD_54 | EMANATES | (COMPONENT_ORDERED) | RD_35 |
| S | RD_60 | GENERALIZATION_OF | (NAMED_ORDERED_-COMPOSITE_LIST_AKD) | NKD_24 |
| S | RD_91 | PARENT | () | N_2 |
| S | RD_111 | SPECIALIZATION_OF | (COMPOSITE_LIST_-ATTRIBUTE_KIND_DEFINITION) | NKD_6 |
| S | RD_116 | TERMINATES | (COMPONENT_OF_ORDERED) | RD_33 |

### 20.4.28 'NODE_KIND_DEFINITION( PROCESS )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| NKD_28 | PROCESS | NKD_25 | NODE_KIND_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("NKD for PROCESS nodes.") |
| CAIS_CONTROLLED | AD_7 | (TRUE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_10 | ATTRIBUTE | (CURRENT_STATUS) | AD_13 |
| S | RD_10 | ATTRIBUTE | (KIND) | AD_45 |
| A | AD_10 | CONSTRAINTS | (VALUE=>PROCESS) | |
| A | AD_14 | DEFAULT_VALUE | (PROCESS) | |
| S | RD_10 | ATTRIBUTE | (IO_UNIT_COUNT) | AD_44 |



Source: https://assist.dla.mil -- Downloaded: 202610-26T07:54Z
Check the source to verify that this is the currentversion before use.

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_10 | ATTRIBUTE | (MACHINE_TIME) | AD_49 |
| S | RD_10 | ATTRIBUTE | (OPEN_NODE_HANDLE_COUNT) | AD_62 |
| S | RD_10 | ATTRIBUTE | (PARAMETERS) | AD_66 |
| S | RD_10 | ATTRIBUTE | (PRIORITY) | AD_69 |
| S | RD_10 | ATTRIBUTE | (PROCESS_SIZE) | AD_71 |
| S | RD_10 | ATTRIBUTE | (RESULTS) | AD_73 |
| A | AD_14 | DEFAULT_VALUE | () | |
| S | RD_10 | ATTRIBUTE | (SUBJECT_CLASSIFICATION) | AD_74 |
| S | RD_10 | ATTRIBUTE | (TIME_FINISHED) | AD_80 |
| S | RD_10 | ATTRIBUTE | (TIME_STARTED) | AD_82 |
| S | RD_54 | EMANATES | (ADOPTED_ROLE) | RD_2 |
| S | RD_54 | EMANATES | (ADOPTED_VIEW) | RD_3 |
| S | RD_54 | EMANATES | (CURRENT_JOB) | RD_38 |
| S | RD_54 | EMANATES | (CURRENT_NODE) | RD_39 |
| S | RD_54 | EMANATES | (CURRENT_PROCESSOR) | RD_40 |
| S | RD_54 | EMANATES | (CURRENT_USER) | RD_41 |
| S | RD_54 | EMANATES | (DEFINITIONS_SECONDARY) | RD_49 |
| S | RD_54 | EMANATES | (DEVICE_SECONDARY) | RD_53 |
| S | RD_54 | EMANATES | (EXECUTABLE_IMAGE) | RD_56 |
| S | RD_54 | EMANATES | (GROUP_SECONDARY) | RD_64 |
| S | RD_54 | EMANATES | (HANDLER_PROCESS_OF) | RD_74 |
| S | RD_54 | EMANATES | (JOB_PARENT) | RD_80 |
| S | RD_54 | EMANATES | (RESOURCE_GROUP_-SECONDARY) | RD_108 |
| S | RD_54 | EMANATES | (STANDARD_ERROR) | RD_113 |
| S | RD_54 | EMANATES | (STANDARD_INPUT) | RD_114 |
| S | RD_54 | EMANATES | (STANDARD_OUTPUT) | RD_115 |
| S | RD_54 | EMANATES | (USER_SECONDARY) | RD_120 |
| S | RD_54 | EMANATES | (VIEW_SECONDARY) | RD_123 |
| S | RD_91 | PARENT | () | N_2 |
| S | RD_111 | SPECIALIZATION_OF | (BASIC) | NKD_3 |
| S | RD_111 | SPECIALIZATION_OF | (CLIENT) | NKD_5 |
| S | RD_116 | TERMINATES | (CURRENT_JOB) | RD_38 |
| S | RD_116 | TERMINATES | (HANDLER_PROCESS) | RD_72 |
| S | RD_116 | TERMINATES | (JOB) | RD_79 |

### 20.4.29 'NODE_KIND_DEFINITION( PROCESSOR )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| NKD_29 | PROCESSOR | NKD_25 | NODE_KIND_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("NKD for PROCESSOR nodes.") |
| CAIS_CONTROLLED | AD_7 | (TRUE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_10 | ATTRIBUTE | (STRUCTURAL_KIND) | AD_76 |
| A | AD_10 | CONSTRAINTS | (VALUE=>PROCESSOR) | |
| A | AD_14 | DEFAULT_VALUE | (PROCESSOR) | |
| S | RD_54 | EMANATES | (ALLOCATABLE_DEVICE) | RD_4 |


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_54 | EMANATES | (ALLOCATABLE_GATEWAY) | RD_5 |
| S | RD_54 | EMANATES | (ALLOCATABLE_MEDIUM) | RD_6 |
| S | RD_54 | EMANATES | (ALLOCATABLE_PROCESSOR) | RD_7 |
| S | RD_54 | EMANATES | (DEFAULT_MEDIUM) | RD_42 |
| S | RD_54 | EMANATES | (DEFAULT_PROCESSOR) | RD_43 |
| S | RD_54 | EMANATES | (PROCESSOR_PARENT) | RD_98 |
| S | RD_91 | PARENT | () | N_2 |
| S | RD_111 | SPECIALIZATION_OF | (RESOURCE) | NKD_32 |
| S | RD_111 | SPECIALIZATION_OF | (STRUCTURAL) | NKD_36 |
| S | RD_116 | TERMINATES | (ALLOCATABLE_PROCESSOR) | RD_7 |
| S | RD_116 | TERMINATES | (CURRENT_PROCESSOR) | RD_40 |
| S | RD_116 | TERMINATES | (DEFAULT_PROCESSOR) | RD_43 |
| S | RD_116 | TERMINATES | (HANDLER_PROCESSOR_-RESOURCE) | RD_75 |
| S | RD_116 | TERMINATES | (PROCESSOR_PRIMARY) | RD_99 |
| S | RD_116 | TERMINATES | (PROCESSOR_SECONDARY) | RD_100 |

### 20.4.30 'NODE_KIND_DEFINITION( RELATION_DEFINITION )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| NKD_30 | RELATION_DEFINITION | NKD_25 | NODE_KIND_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("NKD for RDs.") |
| CAIS_CONTROLLED | AD_7 | (TRUE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_10 | ATTRIBUTE | (ALLOWED_USAGE) | AD_2 |
| S<br>A<br>A | RD_10<br>AD_10<br>AD_14 | ATTRIBUTE<br>CONSTRAINTS<br>DEFAULT_VALUE | (DEFINITION_KIND)<br>(VALUE=>RELATION)<br>(RELATION) | AD_15 |
| S<br>A | RD_10<br>AD_14 | ATTRIBUTE<br>DEFAULT_VALUE | (UNIQUE_KEYS)<br>(TRUE) | AD_84 |
| S | RD_54 | EMANATES | (ATTRIBUTE) | RD_10 |
| S | RD_54 | EMANATES | (EMANATES_FROM) | RD_55 |
| S | RD_54 | EMANATES | (GENERALIZATION_OF_RD) | RD_61 |
| S | RD_54 | EMANATES | (KEY) | RD_81 |
| S | RD_54 | EMANATES | (RELATION_DEFINITION_-PARENT) | RD_103 |
| S | RD_54 | EMANATES | (REVERSE_LINK) | RD_109 |
| S | RD_54 | EMANATES | (REVERSE_LINK_-BIDIRECTIONAL) | RD_110 |
| S | RD_54 | EMANATES | (SPECIALIZATION_OF_RD) | RD_112 |
| S | RD_54 | EMANATES | (TERMINATES_AT) | RD_117 |
| S | RD_91 | PARENT | () | N_2 |
| S | RD_111 | SPECIALIZATION_OF | (DEFINITION) | NKD_7 |
| S | RD_116 | TERMINATES | (ATTRIBUTE_OF) | RD_22 |
| S | RD_116 | TERMINATES | (DEFINITION) | RD_46 |
| S | RD_116 | TERMINATES | (EMANATES) | RD_54 |
| S | RD_116 | TERMINATES | (GENERALIZATION_OF_RD) | RD_61 |



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_116 | TERMINATES | (KEY_OF) | RD_83 |
| S | RD_116 | TERMINATES | (RELATION_DEFINITION) | RD_102 |
| S | RD_116 | TERMINATES | (REVERSE_LINK) | RD_109 |
| S | RD_116 | TERMINATES | (REVERSE_LINK_-BIDIRECTIONAL) | RD_110 |
| S | RD_116 | TERMINATES | (SPECIALIZATION_OF_RD) | RD_112 |
| S | RD_116 | TERMINATES | (TERMINATES) | RD_116 |

### 20.4.31 'NODE_KIND_DEFINITION( RELATION_INTERPRETATION )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| NKD_31 | RELATION_INTERPRETATION | NKD_25 | NODE_KIND_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("NKD for RIs.") |
| CAIS_CONTROLLED | AD_7 | (TRUE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_10 | ATTRIBUTE | (INTERPRETATION_KIND) | AD_40 |
| A | AD_10 | CONSTRAINTS | (VALUE=>RELATION) | |
| A | AD_14 | DEFAULT_VALUE | (RELATION) | |
| S | RD_54 | EMANATES | (KEY_NAME) | RD_82 |
| S | RD_54 | EMANATES | (RELATION_PARENT) | RD_105 |
| S | RD_91 | PARENT | () | N_2 |
| S | RD_111 | SPECIALIZATION_OF | (INTERPRETATION) | NKD_17 |
| S | RD_116 | TERMINATES | (RELATION) | RD_101 |
| S | RD_116 | TERMINATES | (RELATION_NAME) | RD_104 |

### 20.4.32 'NODE_KIND_DEFINITION( RESOURCE )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| NKD_32 | RESOURCE | NKD_25 | NODE_KIND_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("This NKD is a generalization which defines properties common to the various kinds of nodes representing resources.") |
| CAIS_CONTROLLED | AD_7 | (TRUE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_10 | ATTRIBUTE | (ANNOTATION) | AD_3 |
| S | RD_10 | ATTRIBUTE | (AVAILABLE) | AD_6 |
| S | RD_60 | GENERALIZATION_OF | (DEVICE) | NKD_9 |
| S | RD_60 | GENERALIZATION_OF | (GATEWAY) | NKD_13 |
| S | RD_60 | GENERALIZATION_OF | (MEDIUM) | NKD_21 |
| S | RD_60 | GENERALIZATION_OF | (PROCESSOR) | NKD_29 |
| S | RD_91 | PARENT | () | N_2 |
| S | RD_111 | SPECIALIZATION_OF | (BASIC) | NKD_3 |



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

### 20.4.33 'NODE_KIND_DEFINITION( RESOURCE_GROUP )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| NKD_33 | RESOURCE_GROUP | NKD_25 | NODE_KIND_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("NKD for STRUCTURAL nodes used to form groups of resource nodes.") |
| CAIS_CONTROLLED | AD_7 | (TRUE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_10 | ATTRIBUTE | (ANNOTATION) | AD_3 |
| S<br>A<br>A | RD_10<br>AD_10<br>AD_14 | ATTRIBUTE<br>CONSTRAINTS<br>DEFAULT_VALUE | (STRUCTURAL_KIND)<br>(VALUE=>RESOURCE_GROUP)<br>(RESOURCE_GROUP) | AD_76 |
| S | RD_54 | EMANATES | (DEVICE_SECONDARY) | RD_53 |
| S | RD_54 | EMANATES | (GATEWAY_SECONDARY) | RD_59 |
| S | RD_54 | EMANATES | (MEDIUM_SECONDARY) | RD_86 |
| S | RD_54 | EMANATES | (PROCESSOR_SECONDARY) | RD_100 |
| S | RD_54 | EMANATES | (RESOURCE_GROUP_PARENT) | RD_106 |
| S | RD_91 | PARENT | () | N_2 |
| S | RD_111 | SPECIALIZATION_OF | (STRUCTURAL) | NKD_36 |
| S | RD_116 | TERMINATES | (RESOURCE_GROUP_PRIMARY) | RD_107 |
| S | RD_116 | TERMINATES | (RESOURCE_GROUP_-SECONDARY) | RD_108 |

### 20.4.34 'NODE_KIND_DEFINITION( SECONDARY_STORAGE )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| NKD_34 | SECONDARY_STORAGE | NKD_25 | NODE_KIND_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("NKD for SECONDARY_STORAGE nodes.") |
| CAIS_CONTROLLED | AD_7 | (FALSE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_10 | ATTRIBUTE | (CURRENT_FILE_SIZE) | AD_12 |
| S<br>A<br><br>A | RD_10<br>AD_10<br><br>AD_14 | ATTRIBUTE<br>CONSTRAINTS<br><br>DEFAULT_VALUE | (FILE_KIND)<br>(VALUE=>SECONDARY_-STORAGE)<br>(SECONDARY_STORAGE) | AD_22 |
| S | RD_10 | ATTRIBUTE | (MAXIMUM_FILE_SIZE) | AD_50 |
| S | RD_60 | GENERALIZATION_OF | (EXECUTABLE_IMAGE) | NKD_10 |
| S | RD_60 | GENERALIZATION_OF | (INTRODUCED_NODE) | NKD_18 |
| S | RD_91 | PARENT | () | N_2 |
| S | RD_111 | SPECIALIZATION_OF | (FILE) | NKD_11 |

### 20.4.35 'NODE_KIND_DEFINITION( STRING_ATTRIBUTE_KIND_DEFINITION )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| NKD_35 | STRING_ATTRIBUTE_KIND_-DEFINITION | NKD_25 | NODE_KIND_DEFINITION |



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("NKD for String AKDs.") |
| CAIS_CONTROLLED | AD_7 | (TRUE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_10 | ATTRIBUTE | (PATTERNS) | AD_67 |
| A | AD_14 | DEFAULT_VALUE | ("*") | |
| S | RD_10 | ATTRIBUTE | (VALUE_TYPE) | AD_87 |
| A | AD_10 | CONSTRAINTS | (VALUE=>STRING) | |
| A | AD_14 | DEFAULT_VALUE | (STRING) | |
| S | RD_91 | PARENT | () | N_2 |
| S | RD_111 | SPECIALIZATION_OF | (ATTRIBUTE_KIND_-DEFINITION) | NKD_2 |

### 20.4.36 'NODE_KIND_DEFINITION( STRUCTURAL )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| NKD_36 | STRUCTURAL | NKD_25 | NODE_KIND_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("NKD for most general STRUCTURAL node kind.") |
| CAIS_CONTROLLED | AD_7 | (FALSE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_10 | ATTRIBUTE | (KIND) | AD_45 |
| A | AD_10 | CONSTRAINTS | (VALUE=>STRUCTURAL) | |
| A | AD_14 | DEFAULT_VALUE | (STRUCTURAL) | |
| S | RD_10 | ATTRIBUTE | (STRUCTURAL_KIND) | AD_76 |
| S | RD_60 | GENERALIZATION_OF | (CHANNEL) | NKD_4 |
| S | RD_60 | GENERALIZATION_OF | (DEFINITION) | NKD_7 |
| S | RD_60 | GENERALIZATION_OF | (DEFINITIONS) | NKD_8 |
| S | RD_60 | GENERALIZATION_OF | (GROUP) | NKD_14 |
| S | RD_60 | GENERALIZATION_OF | (INTERPRETATION) | NKD_17 |
| S | RD_60 | GENERALIZATION_OF | (MEDIUM) | NKD_21 |
| S | RD_60 | GENERALIZATION_OF | (MONITOR) | NKD_22 |
| S | RD_60 | GENERALIZATION_OF | (PROCESSOR) | NKD_29 |
| S | RD_60 | GENERALIZATION_OF | (RESOURCE_GROUP) | NKD_33 |
| S | RD_60 | GENERALIZATION_OF | (SYSTEM_LEVEL) | NKD_37 |
| S | RD_60 | GENERALIZATION_OF | (USER) | NKD_40 |
| S | RD_60 | GENERALIZATION_OF | (VIEW) | NKD_42 |
| S | RD_91 | PARENT | () | N_2 |
| S | RD_111 | SPECIALIZATION_OF | (BASIC) | NKD_3 |

### 20.4.37 'NODE_KIND_DEFINITION( SYSTEM_LEVEL )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| NKD_37 | SYSTEM_LEVEL | NKD_25 | NODE_KIND_DEFINITION |


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("NKD for the System Level node.") |
| CAIS_CONTROLLED | AD_7 | (TRUE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_10 | ATTRIBUTE | (STRUCTURAL_KIND) | AD_76 |
| A | AD_10 | CONSTRAINTS | (VALUE=>SYSTEM_LEVEL) | |
| A | AD_14 | DEFAULT_VALUE | (SYSTEM_LEVEL) | |
| S | RD_54 | EMANATES | (DEVICE_PRIMARY) | RD_52 |
| S | RD_54 | EMANATES | (GATEWAY_PRIMARY) | RD_58 |
| S | RD_54 | EMANATES | (GROUP_PRIMARY) | RD_63 |
| S | RD_54 | EMANATES | (MEDIUM_PRIMARY) | RD_85 |
| S | RD_54 | EMANATES | (PROCESSOR_PRIMARY) | RD_99 |
| S | RD_54 | EMANATES | (RESOURCE_GROUP_PRIMARY) | RD_107 |
| S | RD_54 | EMANATES | (USER_PRIMARY) | RD_119 |
| S | RD_91 | PARENT | () | N_2 |
| S | RD_111 | SPECIALIZATION_OF | (STRUCTURAL) | NKD_36 |
| S | RD_116 | TERMINATES | (DEVICE_PARENT) | RD_51 |
| S | RD_116 | TERMINATES | (GATEWAY_PARENT) | RD_57 |
| S | RD_116 | TERMINATES | (GROUP_PARENT) | RD_62 |
| S | RD_116 | TERMINATES | (MEDIUM_PARENT) | RD_84 |
| S | RD_116 | TERMINATES | (PROCESSOR_PARENT) | RD_98 |
| S | RD_116 | TERMINATES | (RESOURCE_GROUP_PARENT) | RD_106 |
| S | RD_116 | TERMINATES | (USER_PARENT) | RD_118 |

### 20.4.38 'NODE_KIND_DEFINITION( TERMINAL )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| NKD_38 | TERMINAL | NKD_25 | NODE_KIND_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("NKD for TERMINAL device nodes.") |
| CAIS_CONTROLLED | AD_7 | (TRUE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_10 | ATTRIBUTE | (DEVICE_KIND) | AD_17 |
| A | AD_10 | CONSTRAINTS | (VALUE=>TERMINAL) | |
| A | AD_14 | DEFAULT_VALUE | (TERMINAL) | |
| S | RD_91 | PARENT | () | N_2 |
| S | RD_111 | SPECIALIZATION_OF | (DEVICE) | NKD_9 |

### 20.4.39 'NODE_KIND_DEFINITION( UNIFORM_LIST_ATTRIBUTE_KIND_-DEFINITION )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| NKD_39 | UNIFORM_LIST_ATTRIBUTE_-KIND_DEFINITION | NKD_25 | NODE_KIND_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("NKD for Uniform List AKDs.") |
| CAIS_CONTROLLED | AD_7 | (TRUE) |



Source: https://assist.dla.mil -- Downloaded: 202610-26T07:54Z
Check the source to verify that this is the current version before use.

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_10 | ATTRIBUTE | (MAXIMUM_SIZE) | AD_54 |
| S | RD_10 | ATTRIBUTE | (MINIMUM_SIZE) | AD_56 |
| S | RD_10 | ATTRIBUTE | (NAMING) | AD_59 |
| S<br>A<br>A | RD_10<br>AD_10<br>AD_14 | ATTRIBUTE<br>CONSTRAINTS<br>DEFAULT_VALUE | (VALUE_TYPE)<br>(VALUE=>UNIFORM_LIST)<br>(UNIFORM_LIST) | AD_87 |
| S | RD_54 | EMANATES | (COMPONENT) | RD_27 |
| S<br>A<br>A | RD_54<br>AD_51<br>AD_55 | EMANATES<br>MAXIMUM_INSTANCES<br>MINIMUM_INSTANCES | (COMPONENT_UNIFORM_LIST)<br>(1)<br>(1) | RD_36 |
| S | RD_91 | PARENT | () | N_2 |
| S | RD_111 | SPECIALIZATION_OF | (ATTRIBUTE_KIND_-DEFINITION) | NKD_2 |
| S | RD_116 | TERMINATES | (COMPONENT_OF) | RD_30 |
| S | RD_116 | TERMINATES | (COMPONENT_OF_UNIFORM_-LIST) | RD_34 |

### 20.4.40 'NODE_KIND_DEFINITION( USER )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| NKD_40 | USER | NKD_25 | NODE_KIND_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("NKD for USER nodes.") |
| CAIS_CONTROLLED | AD_7 | (TRUE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S<br>A<br>A | RD_10<br>AD_10<br>AD_14 | ATTRIBUTE<br>CONSTRAINTS<br>DEFAULT_VALUE | (STRUCTURAL_KIND)<br>(VALUE=>USER)<br>(USER) | AD_76 |
| S | RD_54 | EMANATES | (ATTRIBUTE_MONITOR) | RD_19 |
| S | RD_54 | EMANATES | (DEFAULT_ROLE) | RD_45 |
| S | RD_54 | EMANATES | (JOB) | RD_79 |
| S | RD_54 | EMANATES | (USER_PARENT) | RD_118 |
| S | RD_91 | PARENT | () | N_2 |
| S | RD_111 | SPECIALIZATION_OF | (STRUCTURAL) | NKD_36 |
| S | RD_116 | TERMINATES | (ATTRIBUTE_MONITOR_-PARENT) | RD_20 |
| S | RD_116 | TERMINATES | (CURRENT_USER) | RD_41 |
| S | RD_116 | TERMINATES | (JOB_PARENT) | RD_80 |
| S | RD_116 | TERMINATES | (USER_PRIMARY) | RD_119 |
| S | RD_116 | TERMINATES | (USER_SECONDARY) | RD_120 |

### 20.4.41 'NODE_KIND_DEFINITION( VARIANT_ATTRIBUTE_KIND_DEFINITION )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| NKD_41 | VARIANT_ATTRIBUTE_KIND_-DEFINITION | NKD_25 | NODE_KIND_DEFINITION |


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("NKD for Variant AKDs.") |
| CAIS_CONTROLLED | AD_7 | (TRUE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_10 | ATTRIBUTE | (VALUE_TYPE) | AD_87 |
| A | AD_10 | CONSTRAINTS | (VALUE=>VARIANT) | |
| A | AD_14 | DEFAULT_VALUE | (VARIANT) | |
| S | RD_54 | EMANATES | (ALTERNATIVE) | RD_8 |
| A | AD_55 | MINIMUM_INSTANCES | (1) | |
| S | RD_91 | PARENT | () | N_2 |
| S | RD_111 | SPECIALIZATION_OF | (ATTRIBUTE_KIND_-DEFINITION) | NKD_2 |
| S | RD_116 | TERMINATES | (ALTERNATIVE_OF) | RD_9 |

### 20.4.42 'NODE_KIND_DEFINITION( VIEW )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| NKD_42 | VIEW | NKD_25 | NODE_KIND_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("NKD for VIEW nodes.") |
| CAIS_CONTROLLED | AD_7 | (TRUE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_10 | ATTRIBUTE | (ANNOTATION) | AD_3 |
| S | RD_10 | ATTRIBUTE | (STRUCTURAL_KIND) | AD_76 |
| A | AD_10 | CONSTRAINTS | (VALUE=>VIEW) | |
| A | AD_14 | DEFAULT_VALUE | (VIEW) | |
| S | RD_54 | EMANATES | (ATTRIBUTE_DEFINITION_-NAME) | RD_12 |
| S | RD_54 | EMANATES | (ATTRIBUTE_KIND_NAME) | RD_17 |
| S | RD_54 | EMANATES | (NODE_KIND) | RD_89 |
| S | RD_54 | EMANATES | (RELATION) | RD_101 |
| S | RD_54 | EMANATES | (VIEW_PARENT) | RD_121 |
| S | RD_91 | PARENT | () | N_2 |
| S | RD_111 | SPECIALIZATION_OF | (STRUCTURAL) | NKD_36 |
| S | RD_116 | TERMINATES | (ADOPTED_VIEW) | RD_3 |
| S | RD_116 | TERMINATES | (NODE_KIND_PARENT) | RD_92 |
| S | RD_116 | TERMINATES | (RELATION_PARENT) | RD_105 |
| S | RD_116 | TERMINATES | (VIEW_PRIMARY) | RD_122 |
| S | RD_116 | TERMINATES | (VIEW_SECONDARY) | RD_123 |



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

## 20.5 Relation definition nodes (RDs)

The following table describes the node attributes which have the same value on all RDs: 1

| Attribute Name | AD | Attribute Value |
|---|---|---|
| KIND | AD_45 | (STRUCTURAL) |
| STRUCTURAL_KIND | AD_76 | (DEFINITION) |
| DEFINITION_KIND | AD_15 | (RELATION) |

The following subsections describe each predefined RD. Each section name is the primary path element from 'DEFINITIONS( PREDEFINED ) to the RD described. 2

### 20.5.1 'RELATION_DEFINITION( ACCESS )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RD_1 | ACCESS | NKD_30 | RELATION_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("Instances identify accesses permitted on (source) objects by (target) groups.") |
| CAIS_CONTROLLED | AD_7 | (TRUE) |
| CAIS_DETERMINED | AD_8 | (FALSE) |
| UNIQUE_KEYS | AD_84 | (TRUE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_10 | ATTRIBUTE | (GRANT) | AD_24 |
| S | RD_10 | ATTRIBUTE | (INHERITABLE) | AD_39 |
| A | AD_14 | DEFAULT_VALUE | (TRUE) | |
| S | RD_10 | ATTRIBUTE | (NODE_KIND) | AD_60 |
| A | AD_10 | CONSTRAINTS | (VALUE => STRUCTURAL) | |
| A | AD_14 | DEFAULT_VALUE | (STRUCTURAL) | |
| S | RD_55 | EMANATES_FROM | (BASIC) | NKD_3 |
| S | RD_81 | KEY | (CAIS_KEY) | AD_9 |
| S | RD_103 | PARENT | () | N_2 |
| S | RD_112 | SPECIALIZATION_OF | (BASIC) | RD_23 |
| S | RD_117 | TERMINATES_AT | (GROUP) | NKD_14 |

### 20.5.2 'RELATION_DEFINITION( ADOPTED_ROLE )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RD_2 | ADOPTED_ROLE | NKD_30 | RELATION_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("Instances identify the roles under which the (source) process executes.") |
| CAIS_CONTROLLED | AD_7 | (TRUE) |
| CAIS_DETERMINED | AD_8 | (FALSE) |
| UNIQUE_KEYS | AD_84 | (TRUE) |


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_10 | ATTRIBUTE | (INHERITABLE) | AD_39 |
| A | AD_14 | DEFAULT_VALUE | (TRUE) | |
| S | RD_10 | ATTRIBUTE | (NODE_KIND) | AD_60 |
| A | AD_10 | CONSTRAINTS | (VALUE => STRUCTURAL) | |
| A | AD_14 | DEFAULT_VALUE | (STRUCTURAL) | |
| S | RD_55 | EMANATES_FROM | (PROCESS) | NKD_28 |
| S | RD_81 | KEY | (NAME_KEY) | AD_58 |
| S | RD_103 | PARENT | () | N_2 |
| S | RD_112 | SPECIALIZATION_OF | (BASIC) | RD_23 |
| S | RD_117 | TERMINATES_AT | (GROUP) | NKD_14 |

### 20.5.3 'RELATION_DEFINITION( ADOPTED_VIEW )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RD_3 | ADOPTED_VIEW | NKD_30 | RELATION_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("Instances identify the views adopted by the (source) process.") |
| CAIS_CONTROLLED | AD_7 | (TRUE) |
| CAIS_DETERMINED | AD_8 | (FALSE) |
| UNIQUE_KEYS | AD_84 | (TRUE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_10 | ATTRIBUTE | (INHERITABLE) | AD_39 |
| A | AD_14 | DEFAULT_VALUE | (FALSE) | |
| S | RD_10 | ATTRIBUTE | (NODE_KIND) | AD_60 |
| A | AD_10 | CONSTRAINTS | (VALUE => STRUCTURAL) | |
| A | AD_14 | DEFAULT_VALUE | (STRUCTURAL) | |
| S | RD_55 | EMANATES_FROM | (PROCESS) | NKD_28 |
| S | RD_81 | KEY | (POSITION_KEY) | AD_68 |
| S | RD_103 | PARENT | () | N_2 |
| S | RD_112 | SPECIALIZATION_OF | (BASIC) | RD_23 |
| S | RD_117 | TERMINATES_AT | (VIEW) | NKD_42 |

### 20.5.4 'RELATION_DEFINITION( ALLOCATABLE_DEVICE )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RD_4 | ALLOCATABLE_DEVICE | NKD_30 | RELATION_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("Instances identify the devices allocatable from the (source) processor.") |
| CAIS_CONTROLLED | AD_7 | (TRUE) |
| CAIS_DETERMINED | AD_8 | (FALSE) |
| UNIQUE_KEYS | AD_84 | (TRUE) |



Source: https://assist.dla.mil -- Downloaded: 202610-26T07:54Z
Check the source to verify that this is the currentversion before use.

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_10 | ATTRIBUTE | (INHERITABLE) | AD_39 |
| A | AD_14 | DEFAULT_VALUE | (TRUE) | |
| S | RD_10 | ATTRIBUTE | (NODE_KIND) | AD_60 |
| A | AD_10 | CONSTRAINTS | (VALUE => FILE) | |
| A | AD_14 | DEFAULT_VALUE | (FILE) | |
| S | RD_55 | EMANATES_FROM | (PROCESSOR) | NKD_29 |
| S | RD_81 | KEY | (GENERIC_KEY) | AD_23 |
| S | RD_103 | PARENT | () | N_2 |
| S | RD_112 | SPECIALIZATION_OF | (BASIC) | RD_23 |
| S | RD_117 | TERMINATES_AT | (DEVICE) | NKD_9 |

### 20.5.5 'RELATION_DEFINITION( ALLOCATABLE_GATEWAY )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RD_5 | ALLOCATABLE_GATEWAY | NKD_30 | RELATION_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("Instances identify the gateways allocatable from the (source) processor.") |
| CAIS_CONTROLLED | AD_7 | (TRUE) |
| CAIS_DETERMINED | AD_8 | (FALSE) |
| UNIQUE_KEYS | AD_84 | (TRUE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_10 | ATTRIBUTE | (INHERITABLE) | AD_39 |
| A | AD_14 | DEFAULT_VALUE | (TRUE) | |
| S | RD_10 | ATTRIBUTE | (NODE_KIND) | AD_60 |
| A | AD_10 | CONSTRAINTS | (VALUE => FILE) | |
| A | AD_14 | DEFAULT_VALUE | (FILE) | |
| S | RD_55 | EMANATES_FROM | (PROCESSOR) | NKD_29 |
| S | RD_81 | KEY | (GENERIC_KEY) | AD_23 |
| S | RD_103 | PARENT | () | N_2 |
| S | RD_112 | SPECIALIZATION_OF | (BASIC) | RD_23 |
| S | RD_117 | TERMINATES_AT | (GATEWAY) | NKD_13 |

### 20.5.6 'RELATION_DEFINITION( ALLOCATABLE_MEDIUM )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RD_6 | ALLOCATABLE_MEDIUM | NKD_30 | RELATION_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("Instances identify the media allocatable from the (source) processor.") |
| CAIS_CONTROLLED | AD_7 | (TRUE) |
| CAIS_DETERMINED | AD_8 | (FALSE) |
| UNIQUE_KEYS | AD_84 | (TRUE) |


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_10 | ATTRIBUTE | (INHERITABLE) | AD_39 |
| A | AD_14 | DEFAULT_VALUE | (TRUE) | |
| S | RD_10 | ATTRIBUTE | (NODE_KIND) | AD_60 |
| A | AD_10 | CONSTRAINTS | (VALUE => STRUCTURAL) | |
| A | AD_14 | DEFAULT_VALUE | (STRUCTURAL) | |
| S | RD_55 | EMANATES_FROM | (PROCESSOR) | NKD_29 |
| S | RD_81 | KEY | (GENERIC_KEY) | AD_23 |
| S | RD_103 | PARENT | () | N_2 |
| S | RD_112 | SPECIALIZATION_OF | (BASIC) | RD_23 |
| S | RD_117 | TERMINATES_AT | (MEDIUM) | NKD_21 |

### 20.5.7 'RELATION_DEFINITION( ALLOCATABLE_PROCESSOR )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RD_7 | ALLOCATABLE_PROCESSOR | NKD_30 | RELATION_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("Instances identify the processors allocatable from the (source) processor.") |
| CAIS_CONTROLLED | AD_7 | (TRUE) |
| CAIS_DETERMINED | AD_8 | (FALSE) |
| UNIQUE_KEYS | AD_84 | (TRUE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_10 | ATTRIBUTE | (INHERITABLE) | AD_39 |
| A | AD_14 | DEFAULT_VALUE | (TRUE) | |
| S | RD_10 | ATTRIBUTE | (NODE_KIND) | AD_60 |
| A | AD_10 | CONSTRAINTS | (VALUE => STRUCTURAL) | |
| A | AD_14 | DEFAULT_VALUE | (STRUCTURAL) | |
| S | RD_55 | EMANATES_FROM | (PROCESSOR) | NKD_29 |
| S | RD_81 | KEY | (GENERIC_KEY) | AD_23 |
| S | RD_103 | PARENT | () | N_2 |
| S | RD_112 | SPECIALIZATION_OF | (BASIC) | RD_23 |
| S | RD_117 | TERMINATES_AT | (PROCESSOR) | NKD_29 |

### 20.5.8 'RELATION_DEFINITION( ALTERNATIVE )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RD_8 | ALTERNATIVE | NKD_30 | RELATION_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ALLOWED_USAGE | AD_2 | (SECONDARY) |
| ANNOTATION | AD_3 | ("Instances identify the AKDs which are alternatives of the (source) variant AKD.") |
| CAIS_CONTROLLED | AD_7 | (TRUE) |
| CAIS_DETERMINED | AD_8 | (FALSE) |
| UNIQUE_KEYS | AD_84 | (TRUE) |



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_10 | ATTRIBUTE | (CONSTRAINTS) | AD_10 |
| S | RD_10 | ATTRIBUTE | (INHERITABLE) | AD_39 |
| A | AD_10 | CONSTRAINTS | (VALUE => FALSE) | |
| A | AD_14 | DEFAULT_VALUE | (FALSE) | |
| S | RD_10 | ATTRIBUTE | (NODE_KIND) | AD_60 |
| A | AD_10 | CONSTRAINTS | (VALUE => STRUCTURAL) | |
| A | AD_14 | DEFAULT_VALUE | (STRUCTURAL) | |
| S | RD_55 | EMANATES_FROM | (VARIANT_ATTRIBUTE_KIND_-DEFINITION) | NKD_41 |
| S | RD_81 | KEY | (NAME_KEY) | AD_58 |
| S | RD_103 | PARENT | () | N_2 |
| S | RD_110 | REVERSE_LINK | () | RD_9 |
| S | RD_112 | SPECIALIZATION_OF | (BASIC) | RD_23 |
| S | RD_117 | TERMINATES_AT | (ATTRIBUTE_KIND_-DEFINITION) | NKD_2 |

### 20.5.9 'RELATION_DEFINITION( ALTERNATIVE_OF )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RD_9 | ALTERNATIVE | NKD_30 | RELATION_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ALLOWED_USAGE | AD_2 | (SECONDARY) |
| ANNOTATION | AD_3 | ("Instances identify the variant AKDs of which the (source) AKD is an alternative.") |
| CAIS_CONTROLLED | AD_7 | (TRUE) |
| CAIS_DETERMINED | AD_8 | (FALSE) |
| UNIQUE_KEYS | AD_84 | (TRUE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_10 | ATTRIBUTE | (INHERITABLE) | AD_39 |
| A | AD_10 | CONSTRAINTS | (VALUE => FALSE) | |
| A | AD_14 | DEFAULT_VALUE | (FALSE) | |
| S | RD_10 | ATTRIBUTE | (NODE_KIND) | AD_60 |
| A | AD_10 | CONSTRAINTS | (VALUE => STRUCTURAL) | |
| A | AD_14 | DEFAULT_VALUE | (STRUCTURAL) | |
| S | RD_55 | EMANATES_FROM | (ATTRIBUTE_KIND_-DEFINITION) | NKD_2 |
| S | RD_81 | KEY | (GENERIC_KEY) | AD_23 |
| S | RD_103 | PARENT | () | N_2 |
| S | RD_110 | REVERSE_LINK | () | RD_8 |
| S | RD_112 | SPECIALIZATION_OF | (BASIC) | RD_23 |
| S | RD_117 | TERMINATES_AT | (VARIANT_ATTRIBUTE_KIND_-DEFINITION) | NKD_41 |

### 20.5.10 'RELATION_DEFINITION( ATTRIBUTE )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RD_10 | ATTRIBUTE | NKD_30 | RELATION_DEFINITION |


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ALLOWED_USAGE | AD_2 | (SECONDARY) |
| ANNOTATION | AD_3 | ("Instances identify the attributes for (source) node kinds and relations.") |
| CAIS_CONTROLLED | AD_7 | (TRUE) |
| CAIS_DETERMINED | AD_8 | (FALSE) |
| UNIQUE_KEYS | AD_84 | (TRUE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_10 | ATTRIBUTE | (CONSTRAINTS) | AD_10 |
| S | RD_10 | ATTRIBUTE | (DEFAULT_VALUE) | AD_14 |
| S | RD_10 | ATTRIBUTE | (INHERITABLE) | AD_39 |
| A | AD_10 | CONSTRAINTS | (VALUE => FALSE) | |
| A | AD_14 | DEFAULT_VALUE | (FALSE) | |
| S | RD_10 | ATTRIBUTE | (NODE_KIND) | AD_60 |
| A | AD_10 | CONSTRAINTS | (VALUE => STRUCTURAL) | |
| A | AD_14 | DEFAULT_VALUE | (STRUCTURAL) | |
| S | RD_55 | EMANATES_FROM | (NODE_KIND_DEFINITION) | NKD_25 |
| S | RD_55 | EMANATES_FROM | (RELATION_DEFINITION) | NKD_30 |
| S | RD_61 | GENERALIZATION_OF | (KEY) | RD_81 |
| S | RD_81 | KEY | (GENERIC_KEY) | AD_23 |
| S | RD_103 | PARENT | () | N_2 |
| S | RD_110 | REVERSE_LINK | () | RD_22 |
| S | RD_112 | SPECIALIZATION_OF | (BASIC) | RD_23 |
| S | RD_117 | TERMINATES_AT | (ATTRIBUTE_DEFINITION) | NKD_1 |

### 20.5.11 'RELATION_DEFINITION( ATTRIBUTE_DEFINITION )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RD_11 | ATTRIBUTE_DEFINITION | NKD_30 | RELATION_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ALLOWED_USAGE | AD_2 | (PRIMARY) |
| ANNOTATION | AD_3 | ("Instances are primary relationships to ADs.") |
| CAIS_CONTROLLED | AD_7 | (FALSE) |
| CAIS_DETERMINED | AD_8 | (FALSE) |
| UNIQUE_KEYS | AD_84 | (TRUE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_10 | ATTRIBUTE | (INHERITABLE) | AD_39 |
| A | AD_10 | CONSTRAINTS | (VALUE => FALSE) | |
| A | AD_14 | DEFAULT_VALUE | (FALSE) | |
| S | RD_10 | ATTRIBUTE | (NODE_KIND) | AD_60 |
| A | AD_10 | CONSTRAINTS | (VALUE => STRUCTURAL) | |
| A | AD_14 | DEFAULT_VALUE | (STRUCTURAL) | |
| S | RD_55 | EMANATES_FROM | (DEFINITIONS) | NKD_8 |
| S | RD_81 | KEY | (GENERIC_KEY) | AD_23 |
| S | RD_103 | PARENT | () | N_2 |
| S | RD_110 | REVERSE_LINK | () | RD_13 |
| S | RD_112 | SPECIALIZATION_OF | (BASIC) | RD_23 |
| S | RD_117 | TERMINATES_AT | (ATTRIBUTE_DEFINITION) | NKD_1 |



Source: https://assist.dla.mil -- Downloaded: 202610-26T07:54Z
Check the source to verify that this is the current version before use.

### 20.5.12 'RELATION_DEFINITION( ATTRIBUTE_DEFINITION_NAME )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RD_12 | ATTRIBUTE_DEFINITION_NAME | NKD_30 | RELATION_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("Instances specify a (source) view-relative name for the (target) AD.") |
| CAIS_CONTROLLED | AD_7 | (TRUE) |
| CAIS_DETERMINED | AD_8 | (FALSE) |
| UNIQUE_KEYS | AD_84 | (TRUE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_10 | ATTRIBUTE | (INHERITABLE) | AD_39 |
| A | AD_14 | DEFAULT_VALUE | (TRUE) | |
| S | RD_10 | ATTRIBUTE | (NODE_KIND) | AD_60 |
| A | AD_10 | CONSTRAINTS | (VALUE => STRUCTURAL) | |
| A | AD_14 | DEFAULT_VALUE | (STRUCTURAL) | |
| S | RD_55 | EMANATES_FROM | (VIEW) | NKD_42 |
| S | RD_81 | KEY | (NAME_KEY) | AD_58 |
| S | RD_103 | PARENT | () | N_2 |
| S | RD_112 | SPECIALIZATION_OF | (BASIC) | RD_23 |
| S | RD_117 | TERMINATES_AT | (ATTRIBUTE_DEFINITION) | NKD_1 |

### 20.5.13 'RELATION_DEFINITION( ATTRIBUTE_DEFINITION_PARENT )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RD_13 | ATTRIBUTE_DEFINITION_-PARENT | NKD_30 | RELATION_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ALLOWED_USAGE | AD_2 | (SECONDARY) |
| ANNOTATION | AD_3 | ("Instances identify the parent of the (source) AD.") |
| CAIS_CONTROLLED | AD_7 | (FALSE) |
| CAIS_DETERMINED | AD_8 | (FALSE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_55 | EMANATES_FROM | (ATTRIBUTE_DEFINITION) | NKD_1 |
| S | RD_103 | PARENT | () | N_2 |
| S | RD_110 | REVERSE_LINK | () | RD_11 |
| S | RD_112 | SPECIALIZATION_OF | (PARENT) | RD_93 |
| S | RD_117 | TERMINATES_AT | (DEFINITIONS) | NKD_8 |

### 20.5.14 'RELATION_DEFINITION( ATTRIBUTE_KIND )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RD_14 | ATTRIBUTE_KIND | NKD_30 | RELATION_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ALLOWED_USAGE | AD_2 | (SECONDARY) |
| ANNOTATION | AD_3 | ("Instances identify the AKD for the (source) AD.") |



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

| Attribute Name | AD | Attribute Value |
|---|---|---|
| CAIS_CONTROLLED | AD_7 | (TRUE) |
| CAIS_DETERMINED | AD_8 | (FALSE) |
| UNIQUE_KEYS | AD_84 | (TRUE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_10 | ATTRIBUTE | (CONSTRAINTS) | AD_10 |
| S | RD_10 | ATTRIBUTE | (DEFAULT_VALUE) | AD_14 |
| S<br>A<br>A | RD_10<br>AD_10<br>AD_14 | ATTRIBUTE<br>CONSTRAINTS<br>DEFAULT_VALUE | (INHERITABLE)<br>(VALUE => FALSE)<br>(FALSE) | AD_39 |
| S<br>A<br>A | RD_10<br>AD_10<br>AD_14 | ATTRIBUTE<br>CONSTRAINTS<br>DEFAULT_VALUE | (NODE_KIND)<br>(VALUE => STRUCTURAL)<br>(STRUCTURAL) | AD_60 |
| S | RD_55 | EMANATES_FROM | (ATTRIBUTE_DEFINITION) | NKD_1 |
| S | RD_103 | PARENT | () | N_2 |
| S | RD_110 | REVERSE_LINK | () | RD_18 |
| S | RD_112 | SPECIALIZATION_OF | (BASIC) | RD_23 |
| S | RD_117 | TERMINATES_AT | (ATTRIBUTE_KIND_-<br>DEFINITION) | NKD_2 |

### 20.5.15 'RELATION_DEFINITION( ATTRIBUTE_KIND_DEFINITION )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RD_15 | ATTRIBUTE_KIND_DEFINITION | NKD_30 | RELATION_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ALLOWED_USAGE | AD_2 | (PRIMARY) |
| ANNOTATION | AD_3 | ("Instances are primary relationships to AKDs.") |
| CAIS_CONTROLLED | AD_7 | (FALSE) |
| CAIS_DETERMINED | AD_8 | (FALSE) |
| UNIQUE_KEYS | AD_84 | (TRUE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S<br>A<br>A | RD_10<br>AD_10<br>AD_14 | ATTRIBUTE<br>CONSTRAINTS<br>DEFAULT_VALUE | (INHERITABLE)<br>(VALUE => FALSE)<br>(FALSE) | AD_39 |
| S<br>A<br>A | RD_10<br>AD_10<br>AD_14 | ATTRIBUTE<br>CONSTRAINTS<br>DEFAULT_VALUE | (NODE_KIND)<br>(VALUE => STRUCTURAL)<br>(STRUCTURAL) | AD_60 |
| S | RD_55 | EMANATES_FROM | (DEFINITIONS) | NKD_8 |
| S | RD_81 | KEY | (GENERIC_KEY) | AD_23 |
| S | RD_103 | PARENT | () | N_2 |
| S | RD_110 | REVERSE_LINK | () | RD_16 |
| S | RD_112 | SPECIALIZATION_OF | (BASIC) | RD_23 |
| S | RD_117 | TERMINATES_AT | (ATTRIBUTE_KIND_-<br>DEFINITION) | NKD_2 |



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

### 20.5.16 'RELATION_DEFINITION( ATTRIBUTE_KIND_DEFINITION_PARENT )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RD_16 | ATTRIBUTE_KIND_DEFINITION_-PARENT | NKD_30 | RELATION_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ALLOWED_USAGE | AD_2 | (SECONDARY) |
| ANNOTATION | AD_3 | ("Instances identify the parent of the (source) AKD.") |
| CAIS_CONTROLLED | AD_7 | (FALSE) |
| CAIS_DETERMINED | AD_8 | (FALSE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_55 | EMANATES_FROM | (ATTRIBUTE_KIND_-DEFINITION) | NKD_2 |
| S | RD_103 | PARENT | () | N_2 |
| S | RD_110 | REVERSE_LINK | () | RD_15 |
| S | RD_112 | SPECIALIZATION_OF | (PARENT) | RD_93 |
| S | RD_117 | TERMINATES_AT | (DEFINITIONS) | NKD_8 |

### 20.5.17 'RELATION_DEFINITION( ATTRIBUTE_KIND_NAME )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RD_17 | ATTRIBUTE_KIND_NAME | NKD_30 | RELATION_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("Instances specify a (source) view-relative name for the AKD.") |
| CAIS_CONTROLLED | AD_7 | (TRUE) |
| CAIS_DETERMINED | AD_8 | (FALSE) |
| UNIQUE_KEYS | AD_84 | (TRUE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_10 | ATTRIBUTE | (INHERITABLE) | AD_39 |
| A | AD_14 | DEFAULT_VALUE | (TRUE) | |
| S | RD_10 | ATTRIBUTE | (NODE_KIND) | AD_60 |
| A | AD_10 | CONSTRAINTS | (VALUE => STRUCTURAL) | |
| A | AD_14 | DEFAULT_VALUE | (STRUCTURAL) | |
| S | RD_55 | EMANATES_FROM | (VIEW) | NKD_42 |
| S | RD_81 | KEY | (NAME_KEY) | AD_58 |
| S | RD_103 | PARENT | () | N_2 |
| S | RD_112 | SPECIALIZATION_OF | (BASIC) | RD_23 |
| S | RD_117 | TERMINATES_AT | (ATTRIBUTE_KIND_-DEFINITION) | NKD_2 |

### 20.5.18 'RELATION_DEFINITION( ATTRIBUTE_KIND_OF )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RD_18 | ATTRIBUTE_KIND_OF | NKD_30 | RELATION_DEFINITION |



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ALLOWED_USAGE | AD_2 | (SECONDARY) |
| ANNOTATION | AD_3 | ("Instances identify the ADs for the (source) AKD.") |
| CAIS_CONTROLLED | AD_7 | (TRUE) |
| CAIS_DETERMINED | AD_8 | (FALSE) |
| UNIQUE_KEYS | AD_84 | (TRUE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_10 | ATTRIBUTE | (INHERITABLE) | AD_39 |
| A | AD_10 | CONSTRAINTS | (VALUE => FALSE) | |
| A | AD_14 | DEFAULT_VALUE | (FALSE) | |
| S | RD_10 | ATTRIBUTE | (NODE_KIND) | AD_60 |
| A | AD_10 | CONSTRAINTS | (VALUE => STRUCTURAL) | |
| A | AD_14 | DEFAULT_VALUE | (STRUCTURAL) | |
| S | RD_55 | EMANATES_FROM | (ATTRIBUTE_KIND_-DEFINITION) | NKD_2 |
| S | RD_81 | KEY | (GENERIC_KEY) | AD_23 |
| S | RD_103 | PARENT | () | N_2 |
| S | RD_110 | REVERSE_LINK | () | RD_14 |
| S | RD_112 | SPECIALIZATION_OF | (BASIC) | RD_23 |
| S | RD_117 | TERMINATES_AT | (ATTRIBUTE_DEFINITION) | NKD_1 |

### 20.5.19 'RELATION_DEFINITION( ATTRIBUTE_MONITOR )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RD_19 | ATTRIBUTE_MONITOR | NKD_30 | RELATION_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ALLOWED_USAGE | AD_2 | (PRIMARY) |
| ANNOTATION | AD_3 | ("Instances identify the attribute monitors of the (source) user.") |
| CAIS_CONTROLLED | AD_7 | (TRUE) |
| CAIS_DETERMINED | AD_8 | (FALSE) |
| UNIQUE_KEYS | AD_84 | (TRUE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_10 | ATTRIBUTE | (INHERITABLE) | AD_39 |
| A | AD_10 | CONSTRAINTS | (VALUE => FALSE) | |
| A | AD_14 | DEFAULT_VALUE | (FALSE) | |
| S | RD_10 | ATTRIBUTE | (NODE_KIND) | AD_60 |
| A | AD_10 | CONSTRAINTS | (VALUE => STRUCTURAL) | |
| A | AD_14 | DEFAULT_VALUE | (STRUCTURAL) | |
| S | RD_55 | EMANATES_FROM | (USER) | NKD_40 |
| S | RD_81 | KEY | (GENERIC_KEY) | AD_23 |
| S | RD_103 | PARENT | () | N_2 |
| S | RD_110 | REVERSE_LINK | () | RD_20 |
| S | RD_112 | SPECIALIZATION_OF | (BASIC) | RD_23 |
| S | RD_117 | TERMINATES_AT | (MONITOR) | NKD_22 |



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

### 20.5.20 'RELATION_DEFINITION( ATTRIBUTE_MONITOR_PARENT )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RD_20 | ATTRIBUTE_MONITOR_PARENT | NKD_30 | RELATION_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ALLOWED_USAGE | AD_2 | (SECONDARY) |
| ANNOTATION | AD_3 | ("Instances identify the parent and user of the (source) attribute monitor.") |
| CAIS_CONTROLLED | AD_7 | (TRUE) |
| CAIS_DETERMINED | AD_8 | (FALSE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_10 | ATTRIBUTE | (NODE_KIND) | AD_60 |
| A | AD_10 | CONSTRAINTS | (VALUE => STRUCTURAL) | |
| A | AD_14 | DEFAULT_VALUE | (STRUCTURAL) | |
| S | RD_55 | EMANATES_FROM | (MONITOR) | NKD_22 |
| S | RD_103 | PARENT | () | N_2 |
| S | RD_110 | REVERSE_LINK | () | RD_19 |
| S | RD_112 | SPECIALIZATION_OF | (PARENT) | RD_93 |
| S | RD_117 | TERMINATES_AT | (USER) | NKD_40 |

### 20.5.21 'RELATION_DEFINITION( ATTRIBUTE_NAME )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RD_21 | ATTRIBUTE_NAME | NKD_30 | RELATION_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("Instances specify a (source) interpretation-relative name for the (target) attribute.") |
| CAIS_CONTROLLED | AD_7 | (TRUE) |
| CAIS_DETERMINED | AD_8 | (FALSE) |
| UNIQUE_KEYS | AD_84 | (TRUE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_10 | ATTRIBUTE | (INHERITABLE) | AD_39 |
| A | AD_14 | DEFAULT_VALUE | (TRUE) | |
| S | RD_10 | ATTRIBUTE | (NODE_KIND) | AD_60 |
| A | AD_10 | CONSTRAINTS | (VALUE => STRUCTURAL) | |
| A | AD_14 | DEFAULT_VALUE | (STRUCTURAL) | |
| S | RD_55 | EMANATES_FROM | (INTERPRETATION) | NKD_17 |
| S | RD_61 | GENERALIZATION_OF | (KEY_NAME) | RD_82 |
| S | RD_81 | KEY | (NAME_KEY) | AD_58 |
| S | RD_103 | PARENT | () | N_2 |
| S | RD_112 | SPECIALIZATION_OF | (BASIC) | RD_23 |
| S | RD_117 | TERMINATES_AT | (ATTRIBUTE_DEFINITION) | NKD_1 |



Source: https://assist.dla.mil -- Downloaded: 202610-26T07:54Z
Check the source to verify that this is the current version before use.

### 20.5.22 'RELATION_DEFINITION( ATTRIBUTE_OF )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RD_22 | ATTRIBUTE_OF | NKD_30 | RELATION_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ALLOWED_USAGE | AD_2 | (SECONDARY) |
| ANNOTATION | AD_3 | ("Instances identify the node kinds and relations utilizing the (source) attribute.") |
| CAIS_CONTROLLED | AD_7 | (TRUE) |
| CAIS_DETERMINED | AD_8 | (FALSE) |
| UNIQUE_KEYS | AD_84 | (TRUE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_10 | ATTRIBUTE | (INHERITABLE) | AD_39 |
| A | AD_10 | CONSTRAINTS | (VALUE => FALSE) | |
| A | AD_14 | DEFAULT_VALUE | (FALSE) | |
| S | RD_10 | ATTRIBUTE | (NODE_KIND) | AD_60 |
| A | AD_10 | CONSTRAINTS | (VALUE => STRUCTURAL) | |
| A | AD_14 | DEFAULT_VALUE | (STRUCTURAL) | |
| S | RD_55 | EMANATES_FROM | (ATTRIBUTE_DEFINITION) | NKD_1 |
| S | RD_61 | GENERALIZATION_OF | (KEY_OF) | RD_83 |
| S | RD_81 | KEY | (GENERIC_KEY) | AD_23 |
| S | RD_103 | PARENT | () | N_2 |
| S | RD_110 | REVERSE_LINK | () | RD_10 |
| S | RD_112 | SPECIALIZATION_OF | (BASIC) | RD_23 |
| S | RD_117 | TERMINATES_AT | (NODE_KIND_DEFINITION) | NKD_25 |
| S | RD_117 | TERMINATES_AT | (RELATION_DEFINITION) | NKD_30 |

### 20.5.23 'RELATION_DEFINITION( BASIC )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RD_23 | BASIC | NKD_30 | RELATION_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("Most general RD; all other RDs are specializations of this one.") |
| CAIS_CONTROLLED | AD_7 | (FALSE) |
| CAIS_DETERMINED | AD_8 | (FALSE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_10 | ATTRIBUTE | (INHERITABLE) | AD_39 |
| S | RD_10 | ATTRIBUTE | (NODE_KIND) | AD_60 |
| S | RD_55 | EMANATES_FROM | (BASIC) | NKD_3 |
| S | RD_61 | GENERALIZATION_OF | (ACCESS) | RD_1 |
| S | RD_61 | GENERALIZATION_OF | (ADOPTED_ROLE) | RD_2 |
| S | RD_61 | GENERALIZATION_OF | (ADOPTED_VIEW) | RD_3 |
| S | RD_61 | GENERALIZATION_OF | (ALLOCATABLE_DEVICE) | RD_4 |
| S | RD_61 | GENERALIZATION_OF | (ALLOCATABLE_GATEWAY) | RD_5 |
| S | RD_61 | GENERALIZATION_OF | (ALLOCATABLE_MEDIUM) | RD_6 |
| S | RD_61 | GENERALIZATION_OF | (ALLOCATABLE_PROCESSOR) | RD_7 |
| S | RD_61 | GENERALIZATION_OF | (ALTERNATIVE) | RD_8 |



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_61 | GENERALIZATION_OF | (ALTERNATIVE_OF) | RD_9 |
| S | RD_61 | GENERALIZATION_OF | (ATTRIBUTE) | RD_10 |
| S | RD_61 | GENERALIZATION_OF | (ATTRIBUTE_DEFINITION) | RD_11 |
| S | RD_61 | GENERALIZATION_OF | (ATTRIBUTE_DEFINITION_-NAME) | RD_12 |
| S | RD_61 | GENERALIZATION_OF | (ATTRIBUTE_KIND) | RD_14 |
| S | RD_61 | GENERALIZATION_OF | (ATTRIBUTE_KIND_-DEFINITION) | RD_15 |
| S | RD_61 | GENERALIZATION_OF | (ATTRIBUTE_KIND_NAME) | RD_17 |
| S | RD_61 | GENERALIZATION_OF | (ATTRIBUTE_KIND_OF) | RD_18 |
| S | RD_61 | GENERALIZATION_OF | (ATTRIBUTE_MONITOR) | RD_19 |
| S | RD_61 | GENERALIZATION_OF | (ATTRIBUTE_NAME) | RD_21 |
| S | RD_61 | GENERALIZATION_OF | (ATTRIBUTE_OF) | RD_22 |
| S | RD_61 | GENERALIZATION_OF | (CHANNEL) | RD_24 |
| S | RD_61 | GENERALIZATION_OF | (CLIENT) | RD_26 |
| S | RD_61 | GENERALIZATION_OF | (COMPONENT) | RD_27 |
| S | RD_61 | GENERALIZATION_OF | (COMPONENT_OF) | RD_30 |
| S | RD_61 | GENERALIZATION_OF | (COPY_OF) | RD_37 |
| S | RD_61 | GENERALIZATION_OF | (CURRENT_JOB) | RD_38 |
| S | RD_61 | GENERALIZATION_OF | (CURRENT_NODE) | RD_39 |
| S | RD_61 | GENERALIZATION_OF | (CURRENT_PROCESSOR) | RD_40 |
| S | RD_61 | GENERALIZATION_OF | (CURRENT_USER) | RD_41 |
| S | RD_61 | GENERALIZATION_OF | (DEFAULT_MEDIUM) | RD_42 |
| S | RD_61 | GENERALIZATION_OF | (DEFAULT_PROCESSOR) | RD_43 |
| S | RD_61 | GENERALIZATION_OF | (DEFAULT_PROCESS_NODE_-KIND) | RD_44 |
| S | RD_61 | GENERALIZATION_OF | (DEFAULT_ROLE) | RD_45 |
| S | RD_61 | GENERALIZATION_OF | (DEFINITION) | RD_46 |
| S | RD_61 | GENERALIZATION_OF | (DEFINITIONS_PRIMARY) | RD_48 |
| S | RD_61 | GENERALIZATION_OF | (DEFINITIONS_SECONDARY) | RD_49 |
| S | RD_61 | GENERALIZATION_OF | (DENY) | RD_50 |
| S | RD_61 | GENERALIZATION_OF | (DEVICE_PRIMARY) | RD_52 |
| S | RD_61 | GENERALIZATION_OF | (DEVICE_SECONDARY) | RD_53 |
| S | RD_61 | GENERALIZATION_OF | (EMANATES) | RD_54 |
| S | RD_61 | GENERALIZATION_OF | (EMANATES_FROM) | RD_55 |
| S | RD_61 | GENERALIZATION_OF | (EXECUTABLE_IMAGE) | RD_56 |
| S | RD_61 | GENERALIZATION_OF | (GATEWAY_PRIMARY) | RD_58 |
| S | RD_61 | GENERALIZATION_OF | (GATEWAY_SECONDARY) | RD_59 |
| S | RD_61 | GENERALIZATION_OF | (GENERALIZATION_OF_NKD) | RD_60 |
| S | RD_61 | GENERALIZATION_OF | (GENERALIZATION_OF_RD) | RD_61 |
| S | RD_61 | GENERALIZATION_OF | (GROUP_PRIMARY) | RD_63 |
| S | RD_61 | GENERALIZATION_OF | (GROUP_SECONDARY) | RD_64 |
| S | RD_61 | GENERALIZATION_OF | (HANDLER_ATTRIBUTE) | RD_65 |
| S | RD_61 | GENERALIZATION_OF | (HANDLER_ENVIRONMENT_-NODE) | RD_66 |
| S | RD_61 | GENERALIZATION_OF | (HANDLER_ERROR_FILE) | RD_67 |
| S | RD_61 | GENERALIZATION_OF | (HANDLER_IMAGE) | RD_68 |
| S | RD_61 | GENERALIZATION_OF | (HANDLER_INPUT_FILE) | RD_69 |
| S | RD_61 | GENERALIZATION_OF | (HANDLER_MEDIUM_-RESOURCE) | RD_70 |
| S | RD_61 | GENERALIZATION_OF | (HANDLER_OUTPUT_FILE) | RD_71 |



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_61 | GENERALIZATION_OF | (HANDLER_PROCESS) | RD_72 |
| S | RD_61 | GENERALIZATION_OF | (HANDLER_PROCESS_NODE_-KIND) | RD_73 |
| S | RD_61 | GENERALIZATION_OF | (HANDLER_PROCESS_OF) | RD_74 |
| S | RD_61 | GENERALIZATION_OF | (HANDLER_PROCESSOR_-RESOURCE) | RD_75 |
| S | RD_61 | GENERALIZATION_OF | (INTRODUCED_NODE) | RD_76 |
| S | RD_61 | GENERALIZATION_OF | (INTRODUCED_RELATION) | RD_78 |
| S | RD_61 | GENERALIZATION_OF | (JOB) | RD_79 |
| S | RD_61 | GENERALIZATION_OF | (MEDIUM_PRIMARY) | RD_85 |
| S | RD_61 | GENERALIZATION_OF | (MEDIUM_SECONDARY) | RD_86 |
| S | RD_61 | GENERALIZATION_OF | (MONITORED_BY) | RD_87 |
| S | RD_61 | GENERALIZATION_OF | (MONITORS) | RD_88 |
| S | RD_61 | GENERALIZATION_OF | (NODE_KIND) | RD_89 |
| S | RD_61 | GENERALIZATION_OF | (NODE_KIND_DEFINITION) | RD_90 |
| S | RD_61 | GENERALIZATION_OF | (PARENT) | RD_93 |
| S | RD_61 | GENERALIZATION_OF | (PERMANENT_MEMBER) | RD_94 |
| S | RD_61 | GENERALIZATION_OF | (PORT) | RD_96 |
| S | RD_61 | GENERALIZATION_OF | (POTENTIAL_MEMBER) | RD_97 |
| S | RD_61 | GENERALIZATION_OF | (PROCESSOR_PRIMARY) | RD_99 |
| S | RD_61 | GENERALIZATION_OF | (PROCESSOR_SECONDARY) | RD_100 |
| S | RD_61 | GENERALIZATION_OF | (RELATION) | RD_101 |
| S | RD_61 | GENERALIZATION_OF | (RELATION_DEFINITION) | RD_102 |
| S | RD_61 | GENERALIZATION_OF | (RELATION_NAME) | RD_104 |
| S | RD_61 | GENERALIZATION_OF | (RESOURCE_GROUP_PRIMARY) | RD_107 |
| S | RD_61 | GENERALIZATION_OF | (RESOURCE_GROUP_-SECONDARY) | RD_108 |
| S | RD_61 | GENERALIZATION_OF | (REVERSE_LINK) | RD_109 |
| S | RD_61 | GENERALIZATION_OF | (SPECIALIZATION_OF_NKD) | RD_111 |
| S | RD_61 | GENERALIZATION_OF | (SPECIALIZATION_OF_RD) | RD_112 |
| S | RD_61 | GENERALIZATION_OF | (STANDARD_ERROR) | RD_113 |
| S | RD_61 | GENERALIZATION_OF | (STANDARD_INPUT) | RD_114 |
| S | RD_61 | GENERALIZATION_OF | (STANDARD_OUTPUT) | RD_115 |
| S | RD_61 | GENERALIZATION_OF | (TERMINATES) | RD_116 |
| S | RD_61 | GENERALIZATION_OF | (TERMINATES_AT) | RD_117 |
| S | RD_61 | GENERALIZATION_OF | (USER_PRIMARY) | RD_119 |
| S | RD_61 | GENERALIZATION_OF | (USER_SECONDARY) | RD_120 |
| S | RD_61 | GENERALIZATION_OF | (VIEW_PRIMARY) | RD_122 |
| S | RD_61 | GENERALIZATION_OF | (VIEW_SECONDARY) | RD_123 |
| S | RD_103 | PARENT | () | N_2 |
| S | RD_117 | TERMINATES_AT | (BASIC) | NKD_3 |

### 20.5.24 'RELATION_DEFINITION( CHANNEL )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RD_24 | CHANNEL | NKD_30 | RELATION_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ALLOWED_USAGE | AD_2 | (PRIMARY) |
| ANNOTATION | AD_3 | ("Instances are primary relationships to CHANNEL nodes.") |



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

| Attribute Name | AD | Attribute Value |
|---|---|---|
| CAIS_CONTROLLED | AD_7 | (FALSE) |
| CAIS_DETERMINED | AD_8 | (FALSE) |
| UNIQUE_KEYS | AD_84 | (TRUE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_10 | ATTRIBUTE | (INHERITABLE) | AD_39 |
| A | AD_10 | CONSTRAINTS | (VALUE => FALSE) | |
| A | AD_14 | DEFAULT_VALUE | (FALSE) | |
| S | RD_10 | ATTRIBUTE | (NODE_KIND) | AD_60 |
| A | AD_10 | CONSTRAINTS | (VALUE => STRUCTURAL) | |
| A | AD_14 | DEFAULT_VALUE | (STRUCTURAL) | |
| S | RD_55 | EMANATES_FROM | (BASIC) | NKD_3 |
| S | RD_81 | KEY | (GENERIC_KEY) | AD_23 |
| S | RD_103 | PARENT | () | N_2 |
| S | RD_110 | REVERSE_LINK | () | RD_25 |
| S | RD_112 | SPECIALIZATION_OF | (BASIC) | RD_23 |
| S | RD_117 | TERMINATES_AT | (CHANNEL) | NKD_4 |

### 20.5.25 'RELATION_DEFINITION( CHANNEL_PARENT )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RD_25 | CHANNEL_PARENT | NKD_30 | RELATION_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ALLOWED_USAGE | AD_2 | (SECONDARY) |
| ANNOTATION | AD_3 | ("Instances identify the parent of (source) CHANNEL node.") |
| CAIS_CONTROLLED | AD_7 | (FALSE) |
| CAIS_DETERMINED | AD_8 | (FALSE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_55 | EMANATES_FROM | (CHANNEL) | NKD_4 |
| S | RD_103 | PARENT | () | N_2 |
| S | RD_110 | REVERSE_LINK | () | RD_24 |
| S | RD_112 | SPECIALIZATION_OF | (PARENT) | RD_93 |
| S | RD_117 | TERMINATES_AT | (BASIC) | NKD_3 |

### 20.5.26 'RELATION_DEFINITION( CLIENT )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RD_26 | CLIENT | NKD_30 | RELATION_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ALLOWED_USAGE | AD_2 | (SECONDARY) |
| ANNOTATION | AD_3 | ("Instances identify the clients of the (source) CHANNEL node.") |
| CAIS_CONTROLLED | AD_7 | (TRUE) |
| CAIS_DETERMINED | AD_8 | (FALSE) |
| UNIQUE_KEYS | AD_84 | (TRUE) |



Source: https://assist.dla.mil -- Downloaded: 202640-26T07:54Z
Check the source to verify that this is the current version before use.

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_10 | ATTRIBUTE | (INHERITABLE) | AD_39 |
| A | AD_10 | CONSTRAINTS | (VALUE => FALSE) | |
| A | AD_14 | DEFAULT_VALUE | (FALSE) | |
| S | RD_10 | ATTRIBUTE | (NODE_KIND) | AD_60 |
| A | AD_10 | CONSTRAINTS | (PATTERNS => ("FILE", "PROCESS")) | |
| S | RD_55 | EMANATES_FROM | (CHANNEL) | NKD_4 |
| S | RD_81 | KEY | (CAIS_KEY) | AD_9 |
| S | RD_103 | PARENT | () | N_2 |
| S | RD_110 | REVERSE_LINK | () | RD_96 |
| S | RD_112 | SPECIALIZATION_OF | (BASIC) | RD_23 |
| S | RD_117 | TERMINATES_AT | (CLIENT) | NKD_5 |

### 20.5.27 'RELATION_DEFINITION( COMPONENT )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RD_27 | COMPONENT | NKD_30 | RELATION_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ALLOWED_USAGE | AD_2 | (SECONDARY) |
| ANNOTATION | AD_3 | ("A generalization of relations whose instances identify AKDs which are components of the (source) composite or uniform list AKD.") |
| CAIS_CONTROLLED | AD_7 | (TRUE) |
| CAIS_DETERMINED | AD_8 | (FALSE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_10 | ATTRIBUTE | (CONSTRAINTS) | AD_10 |
| S | RD_10 | ATTRIBUTE | (INHERITABLE) | AD_39 |
| A | AD_10 | CONSTRAINTS | (VALUE => FALSE) | |
| A | AD_14 | DEFAULT_VALUE | (FALSE) | |
| S | RD_10 | ATTRIBUTE | (NODE_KIND) | AD_60 |
| A | AD_10 | CONSTRAINTS | (VALUE => STRUCTURAL) | |
| A | AD_14 | DEFAULT_VALUE | (STRUCTURAL) | |
| S | RD_55 | EMANATES_FROM | (COMPOSITE_LIST_ATTRIBUTE_KIND_DEFINITION) | NKD_6 |
| S | RD_55 | EMANATES_FROM | (UNIFORM_LIST_ATTRIBUTE_KIND_DEFINITION) | NKD_39 |
| S | RD_61 | GENERALIZATION_OF | (COMPONENT_NAMED) | RD_28 |
| S | RD_61 | GENERALIZATION_OF | (COMPONENT_ORDERED) | RD_35 |
| S | RD_61 | GENERALIZATION_OF | (COMPONENT_UNIFORM_LIST) | RD_36 |
| S | RD_103 | PARENT | () | N_2 |
| S | RD_110 | REVERSE_LINK | () | RD_30 |
| S | RD_112 | SPECIALIZATION_OF | (BASIC) | RD_23 |
| S | RD_117 | TERMINATES_AT | (ATTRIBUTE_KIND_DEFINITION) | NKD_2 |



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

### 20.5.28 'RELATION_DEFINITION( COMPONENT_NAMED )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RD_28 | COMPONENT_NAMED | NKD_30 | RELATION_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ALLOWED_USAGE | AD_2 | (SECONDARY) |
| ANNOTATION | AD_3 | ("Instances identify the AKDs which are named components of the (source) composite list AKD.") |
| CAIS_CONTROLLED | AD_7 | (TRUE) |
| CAIS_DETERMINED | AD_8 | (FALSE) |
| UNIQUE_KEYS | AD_84 | (TRUE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_10 | ATTRIBUTE | (OPTIONAL) | AD_63 |
| A | AD_14 | DEFAULT_VALUE | (FALSE) | |
| S | RD_55 | EMANATES_FROM | (NAMED_COMPOSITE_LIST_AKD) | NKD_23 |
| S | RD_61 | GENERALIZATION_OF | (COMPONENT_NAMED_ORDERED) | RD_29 |
| S | RD_81 | KEY | (NAME_KEY) | AD_58 |
| S | RD_103 | PARENT | () | N_2 |
| S | RD_110 | REVERSE_LINK | () | RD_31 |
| S | RD_112 | SPECIALIZATION_OF | (COMPONENT) | RD_27 |
| S | RD_117 | TERMINATES_AT | (ATTRIBUTE_KIND_DEFINITION) | NKD_2 |

### 20.5.29 'RELATION_DEFINITION( COMPONENT_NAMED_ORDERED )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RD_29 | COMPONENT_NAMED_ORDERED | NKD_30 | RELATION_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("Instances identify the AKDs which are named, ordered components of the (source) composite list AKD.") |
| CAIS_CONTROLLED | AD_7 | (TRUE) |
| CAIS_DETERMINED | AD_8 | (FALSE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_55 | EMANATES_FROM | (NAMED_ORDERED_COMPOSITE_LIST_AKD) | NKD_24 |
| S | RD_103 | PARENT | () | N_2 |
| S | RD_110 | REVERSE_LINK | () | RD_32 |
| S | RD_112 | SPECIALIZATION_OF | (COMPONENT_NAMED) | RD_28 |
| S | RD_112 | SPECIALIZATION_OF | (COMPONENT_ORDERED) | RD_35 |
| S | RD_117 | TERMINATES_AT | (ATTRIBUTE_KIND_DEFINITION) | NKD_2 |



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

### 20.5.30 'RELATION_DEFINITION( COMPONENT_OF )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RD_30 | COMPONENT_OF | NKD_30 | RELATION_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ALLOWED_USAGE | AD_2 | (SECONDARY) |
| ANNOTATION | AD_3 | ("A generalisation of relations whose instances identify the composite or uniform list AKDs of which the (source) AKD is a component.") |
| CAIS_CONTROLLED | AD_7 | (TRUE) |
| CAIS_DETERMINED | AD_8 | (FALSE) |
| UNIQUE_KEYS | AD_84 | (TRUE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_10 | ATTRIBUTE | (INHERITABLE) | AD_39 |
| A | AD_10 | CONSTRAINTS | (VALUE => FALSE) | |
| A | AD_14 | DEFAULT_VALUE | (FALSE) | |
| S | RD_10 | ATTRIBUTE | (NODE_KIND) | AD_60 |
| A | AD_10 | CONSTRAINTS | (VALUE => STRUCTURAL) | |
| A | AD_14 | DEFAULT_VALUE | (STRUCTURAL) | |
| S | RD_55 | EMANATES_FROM | (ATTRIBUTE_KIND_-DEFINITION) | NKD_2 |
| S | RD_61 | GENERALIZATION_OF | (COMPONENT_OF_NAMED) | RD_31 |
| S | RD_61 | GENERALIZATION_OF | (COMPONENT_OF_ORDERED) | RD_33 |
| S | RD_61 | GENERALIZATION_OF | (COMPONENT_OF_UNIFORM_-LIST) | RD_34 |
| S | RD_81 | KEY | (GENERIC_KEY) | AD_23 |
| S | RD_103 | PARENT | () | N_2 |
| S | RD_110 | REVERSE_LINK | () | RD_27 |
| S | RD_112 | SPECIALIZATION_OF | (BASIC) | RD_23 |
| S | RD_117 | TERMINATES_AT | (COMPOSITE_LIST_-ATTRIBUTE_KIND_DEFINITION) | NKD_6 |
| S | RD_117 | TERMINATES_AT | (UNIFORM_LIST_ATTRIBUTE_-KIND_DEFINITION) | NKD_39 |

### 20.5.31 'RELATION_DEFINITION( COMPONENT_OF_NAMED )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RD_31 | COMPONENT_OF_NAMED | NKD_30 | RELATION_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("Instances identify the composite list AKDs of which the (source) AKD is a named component.") |
| CAIS_CONTROLLED | AD_7 | (TRUE) |
| CAIS_DETERMINED | AD_8 | (FALSE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_55 | EMANATES_FROM | (ATTRIBUTE_KIND_-DEFINITION) | NKD_2 |
| S | RD_61 | GENERALIZATION_OF | (COMPONENT_OF_NAMED_-ORDERED) | RD_32 |
| S | RD_103 | PARENT | () | N_2 |


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_110 | REVERSE_LINK | () | RD_28 |
| S | RD_112 | SPECIALIZATION_OF | (COMPONENT_OF) | RD_30 |
| S | RD_117 | TERMINATES_AT | (NAMED_COMPOSITE_LIST_-AKD) | NKD_23 |

### 20.5.32 'RELATION_DEFINITION( COMPONENT_OF_NAMED_ORDERED )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RD_32 | COMPONENT_OF_NAMED_-ORDERED | NKD_30 | RELATION_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("Instances identify the composite list AKDs of which the (source) AKD is a named, ordered component.") |
| CAIS_CONTROLLED | AD_7 | (TRUE) |
| CAIS_DETERMINED | AD_8 | (FALSE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_55 | EMANATES_FROM | (ATTRIBUTE_KIND_-DEFINITION) | NKD_2 |
| S | RD_103 | PARENT | () | N_2 |
| S | RD_110 | REVERSE_LINK | () | RD_29 |
| S | RD_112 | SPECIALIZATION_OF | (COMPONENT_OF_NAMED) | RD_31 |
| S | RD_112 | SPECIALIZATION_OF | (COMPONENT_OF_ORDERED) | RD_33 |
| S | RD_117 | TERMINATES_AT | (NAMED_ORDERED_-COMPOSITE_LIST_AKD) | NKD_24 |

### 20.5.33 'RELATION_DEFINITION( COMPONENT_OF_ORDERED )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RD_33 | COMPONENT_OF_ORDERED | NKD_30 | RELATION_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("Instances identify the composite list AKDs of which the (source) AKD is an ordered component.") |
| CAIS_CONTROLLED | AD_7 | (TRUE) |
| CAIS_DETERMINED | AD_8 | (FALSE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_55 | EMANATES_FROM | (ATTRIBUTE_KIND_-DEFINITION) | NKD_2 |
| S | RD_61 | GENERALIZATION_OF | (COMPONENT_OF_NAMED_-ORDERED) | RD_32 |
| S | RD_103 | PARENT | () | N_2 |
| S | RD_110 | REVERSE_LINK | () | RD_35 |
| S | RD_112 | SPECIALIZATION_OF | (COMPONENT_OF) | RD_30 |
| S | RD_117 | TERMINATES_AT | (ORDERED_COMPOSITE_LIST_-AKD) | NKD_27 |



Source: https://assist.dla.mil -- Downloaded: 202610-26T07:54Z
Check the source to verify that this is the current version before use.

### 20.5.34 'RELATION_DEFINITION( COMPONENT_OF_UNIFORM_LIST )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RD_34 | COMPONENT_OF_UNIFORM_-LIST | NKD_30 | RELATION_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("Instances identify the uniform list AKDs of which the (source) AKD is the component.") |
| CAIS_CONTROLLED | AD_7 | (TRUE) |
| CAIS_DETERMINED | AD_8 | (FALSE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_55 | EMANATES_FROM | (ATTRIBUTE_KIND_-DEFINITION) | NKD_2 |
| S | RD_103 | PARENT | () | N_2 |
| S | RD_110 | REVERSE_LINK | () | RD_36 |
| S | RD_112 | SPECIALIZATION_OF | (COMPONENT_OF) | RD_30 |
| S | RD_117 | TERMINATES_AT | (UNIFORM_LIST_ATTRIBUTE_-KIND_DEFINITION) | NKD_39 |

### 20.5.35 'RELATION_DEFINITION( COMPONENT_ORDERED )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RD_35 | COMPONENT_ORDERED | NKD_30 | RELATION_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("Instances identify the AKDs which are ordered components of the (source) composite list AKD.") |
| CAIS_CONTROLLED | AD_7 | (TRUE) |
| CAIS_DETERMINED | AD_8 | (FALSE) |
| UNIQUE_KEYS | AD_84 | (TRUE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_10 | ATTRIBUTE | (OPTIONAL) | AD_63 |
| A | AD_14 | DEFAULT_VALUE | (FALSE) | |
| S | RD_55 | EMANATES_FROM | (ORDERED_COMPOSITE_LIST_-AKD) | NKD_27 |
| S | RD_61 | GENERALIZATION_OF | (COMPONENT_NAMED_-ORDERED) | RD_29 |
| S | RD_81 | KEY | (POSITION_KEY) | AD_68 |
| S | RD_103 | PARENT | () | N_2 |
| S | RD_110 | REVERSE_LINK | () | RD_33 |
| S | RD_112 | SPECIALIZATION_OF | (COMPONENT) | RD_27 |
| S | RD_117 | TERMINATES_AT | (ATTRIBUTE_KIND_-DEFINITION) | NKD_2 |

### 20.5.36 'RELATION_DEFINITION( COMPONENT_UNIFORM_LIST )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RD_36 | COMPONENT_UNIFORM_LIST | NKD_30 | RELATION_DEFINITION |



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("Instances identify the AKD which is the component of the (source) uniform list AKD.") |
| CAIS_CONTROLLED | AD_7 | (TRUE) |
| CAIS_DETERMINED | AD_8 | (FALSE) |
| UNIQUE_KEYS | AD_84 | (TRUE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_55 | EMANATES_FROM | (UNIFORM_LIST_ATTRIBUTE_-KIND_DEFINITION) | NKD_39 |
| S | RD_103 | PARENT | () | N_2 |
| S | RD_110 | REVERSE_LINK | () | RD_34 |
| S | RD_112 | SPECIALIZATION_OF | (COMPONENT) | RD_27 |
| S | RD_117 | TERMINATES_AT | (ATTRIBUTE_KIND_-DEFINITION) | NKD_2 |

### 20.5.37 'RELATION_DEFINITION( COPY_OF )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RD_37 | BASIC | NKD_30 | RELATION_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ALLOWED_USAGE | AD_2 | (SECONDARY) |
| ANNOTATION | AD_3 | ("Instances identify the original node for a node created by a copy operation.") |
| CAIS_CONTROLLED | AD_7 | (FALSE) |
| CAIS_DETERMINED | AD_8 | (FALSE) |
| UNIQUE_KEYS | AD_84 | (TRUE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_10 | ATTRIBUTE | (INHERITABLE) | AD_39 |
| A | AD_10 | CONSTRAINTS | (VALUE => FALSE) | |
| A | AD_14 | DEFAULT_VALUE | (FALSE) | |
| S | RD_55 | EMANATES_FROM | (BASIC) | NKD_3 |
| S | RD_103 | PARENT | () | N_2 |
| S | RD_112 | SPECIALIZATION_OF | (BASIC) | RD_23 |
| S | RD_117 | TERMINATES_AT | (BASIC) | NKD_3 |

### 20.5.38 'RELATION_DEFINITION( CURRENT_JOB )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RD_38 | CURRENT_JOB | NKD_30 | RELATION_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("Instances identify the process which is root of the tree containing the source process.") |
| CAIS_CONTROLLED | AD_7 | (TRUE) |
| CAIS_DETERMINED | AD_8 | (TRUE) |
| UNIQUE_KEYS | AD_84 | (TRUE) |



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_10 | ATTRIBUTE | (INHERITABLE) | AD_39 |
| A | AD_10 | CONSTRAINTS | (VALUE => TRUE) | |
| A | AD_14 | DEFAULT_VALUE | (TRUE) | |
| S | RD_10 | ATTRIBUTE | (NODE_KIND) | AD_60 |
| A | AD_10 | CONSTRAINTS | (VALUE => PROCESS) | |
| A | AD_14 | DEFAULT_VALUE | (PROCESS) | |
| S | RD_55 | EMANATES_FROM | (PROCESS) | NKD_28 |
| S | RD_103 | PARENT | () | N_2 |
| S | RD_112 | SPECIALIZATION_OF | (BASIC) | RD_23 |
| S | RD_117 | TERMINATES_AT | (PROCESS) | NKD_28 |

### 20.5.39 'RELATION_DEFINITION( CURRENT_NODE )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RD_39 | CURRENT_NODE | NKD_30 | RELATION_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("Instances identify the node to which pathnames starting with a key are relative for the source process.") |
| CAIS_CONTROLLED | AD_7 | (TRUE) |
| CAIS_DETERMINED | AD_8 | (FALSE) |
| UNIQUE_KEYS | AD_84 | (TRUE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_10 | ATTRIBUTE | (INHERITABLE) | AD_39 |
| A | AD_10 | CONSTRAINTS | (VALUE => FALSE) | |
| A | AD_14 | DEFAULT_VALUE | (FALSE) | |
| S | RD_55 | EMANATES_FROM | (PROCESS) | NKD_28 |
| S | RD_103 | PARENT | () | N_2 |
| S | RD_112 | SPECIALIZATION_OF | (BASIC) | RD_23 |
| S | RD_117 | TERMINATES_AT | (BASIC) | NKD_3 |

### 20.5.40 'RELATION_DEFINITION( CURRENT_PROCESSOR )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RD_40 | CURRENT_PROCESSOR | NKD_30 | RELATION_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("Instances identify the processor on which the (source) process executes.") |
| CAIS_CONTROLLED | AD_7 | (TRUE) |
| CAIS_DETERMINED | AD_8 | (TRUE) |
| UNIQUE_KEYS | AD_84 | (TRUE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_10 | ATTRIBUTE | (INHERITABLE) | AD_39 |
| A | AD_10 | CONSTRAINTS | (VALUE => FALSE) | |
| A | AD_14 | DEFAULT_VALUE | (FALSE) | |



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_10 | ATTRIBUTE | (NODE_KIND) | AD_60 |
| A | AD_10 | CONSTRAINTS | (VALUE => STRUCTURAL) | |
| A | AD_14 | DEFAULT_VALUE | (STRUCTURAL) | |
| S | RD_55 | EMANATES_FROM | (PROCESS) | NKD_28 |
| S | RD_103 | PARENT | () | N_2 |
| S | RD_112 | SPECIALIZATION_OF | (BASIC) | RD_23 |
| S | RD_117 | TERMINATES_AT | (PROCESSOR) | NKD_29 |

### 20.5.41 'RELATION_DEFINITION( CURRENT_USER )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RD_41 | CURRENT_USER | NKD_30 | RELATION_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("Instances identify the user node under which the (source) process executes.") |
| CAIS_CONTROLLED | AD_7 | (TRUE) |
| CAIS_DETERMINED | AD_8 | (TRUE) |
| UNIQUE_KEYS | AD_84 | (TRUE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_10 | ATTRIBUTE | (INHERITABLE) | AD_39 |
| A | AD_10 | CONSTRAINTS | (VALUE => TRUE) | |
| A | AD_14 | DEFAULT_VALUE | (TRUE) | |
| S | RD_10 | ATTRIBUTE | (NODE_KIND) | AD_60 |
| A | AD_10 | CONSTRAINTS | (VALUE => STRUCTURAL) | |
| A | AD_14 | DEFAULT_VALUE | (STRUCTURAL) | |
| S | RD_55 | EMANATES_FROM | (PROCESS) | NKD_28 |
| S | RD_103 | PARENT | () | N_2 |
| S | RD_112 | SPECIALIZATION_OF | (BASIC) | RD_23 |
| S | RD_117 | TERMINATES_AT | (USER) | NKD_40 |

### 20.5.42 'RELATION_DEFINITION( DEFAULT_MEDIUM )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RD_42 | DEFAULT_MEDIUM | NKD_30 | RELATION_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("Instances identify the medium which is allocated by default for processes on the (source) processor.") |
| CAIS_CONTROLLED | AD_7 | (TRUE) |
| CAIS_DETERMINED | AD_8 | (FALSE) |
| UNIQUE_KEYS | AD_84 | (TRUE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_10 | ATTRIBUTE | (INHERITABLE) | AD_39 |
| A | AD_14 | DEFAULT_VALUE | (TRUE) | |
| S | RD_10 | ATTRIBUTE | (NODE_KIND) | AD_60 |
| A | AD_10 | CONSTRAINTS | (VALUE => STRUCTURAL) | |
| A | AD_14 | DEFAULT_VALUE | (STRUCTURAL) | |



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_55 | EMANATES_FROM | (PROCESSOR) | NKD_29 |
| S | RD_103 | PARENT | () | N_2 |
| S | RD_112 | SPECIALIZATION_OF | (BASIC) | RD_23 |
| S | RD_117 | TERMINATES_AT | (MEDIUM) | NKD_21 |

### 20.5.43 'RELATION_DEFINITION( DEFAULT_PROCESSOR )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RD_43 | DEFAULT_PROCESSOR | NKD_30 | RELATION_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("Instances identify the processor which is allocated by default for processes on the (source) processor.") |
| CAIS_CONTROLLED | AD_7 | (TRUE) |
| CAIS_DETERMINED | AD_8 | (FALSE) |
| UNIQUE_KEYS | AD_84 | (TRUE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_10 | ATTRIBUTE | (INHERITABLE) | AD_39 |
| A | AD_14 | DEFAULT_VALUE | (TRUE) | |
| S | RD_10 | ATTRIBUTE | (NODE_KIND) | AD_60 |
| A | AD_10 | CONSTRAINTS | (VALUE => STRUCTURAL) | |
| A | AD_14 | DEFAULT_VALUE | (STRUCTURAL) | |
| S | RD_55 | EMANATES_FROM | (PROCESSOR) | NKD_29 |
| S | RD_103 | PARENT | () | N_2 |
| S | RD_112 | SPECIALIZATION_OF | (BASIC) | RD_23 |
| S | RD_117 | TERMINATES_AT | (PROCESSOR) | NKD_29 |

### 20.5.44 'RELATION_DEFINITION( DEFAULT_PROCESS_NODE_KIND )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RD_44 | DEFAULT_PROCESS_NODE_KIND | NKD_30 | RELATION_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("Instances identify the process NKD which is instantiated by default for execution of the (source) executable image.") |
| CAIS_CONTROLLED | AD_7 | (FALSE) |
| CAIS_DETERMINED | AD_8 | (FALSE) |
| UNIQUE_KEYS | AD_84 | (TRUE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_10 | ATTRIBUTE | (INHERITABLE) | AD_39 |
| A | AD_14 | DEFAULT_VALUE | (TRUE) | |
| S | RD_10 | ATTRIBUTE | (NODE_KIND) | AD_60 |
| A | AD_10 | CONSTRAINTS | (VALUE => STRUCTURAL) | |
| A | AD_14 | DEFAULT_VALUE | (STRUCTURAL) | |
| S | RD_55 | EMANATES_FROM | (EXECUTABLE_IMAGE) | *NKD_10* |
| S | RD_103 | PARENT | () | N_2 |
| S | RD_112 | SPECIALIZATION_OF | (BASIC) | RD_23 |
| S | RD_117 | TERMINATES_AT | (NODE_KIND_DEFINITION) | NKD_25 |



Source: https://assist.dla.mil -- Downloaded: 202610-26T07:54Z
Check the source to verify that this is the current version before use.

### 20.5.45 'RELATION_DEFINITION( DEFAULT_ROLE )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RD_45 | DEFAULT_ROLE | NKD_30 | RELATION_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("Instances identify the roles which are always adopted for processes of the (source) user or executable image.") |
| CAIS_CONTROLLED | AD_7 | (TRUE) |
| CAIS_DETERMINED | AD_8 | (FALSE) |
| UNIQUE_KEYS | AD_84 | (TRUE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_10 | ATTRIBUTE | (INHERITABLE) | AD_39 |
| A | AD_10 | CONSTRAINTS | (VALUE => FALSE) | |
| A | AD_14 | DEFAULT_VALUE | (FALSE) | |
| S | RD_10 | ATTRIBUTE | (NODE_KIND) | AD_60 |
| A | AD_10 | CONSTRAINTS | (VALUE => STRUCTURAL) | |
| A | AD_14 | DEFAULT_VALUE | (STRUCTURAL) | |
| S | RD_55 | EMANATES_FROM | (EXECUTABLE_IMAGE) | NKD_10 |
| S | RD_55 | EMANATES_FROM | (USER) | NKD_40 |
| S | RD_103 | PARENT | () | N_2 |
| S | RD_112 | SPECIALIZATION_OF | (BASIC) | RD_23 |
| S | RD_117 | TERMINATES_AT | (GROUP) | NKD_14 |

### 20.5.46 'RELATION_DEFINITION( DEFINITION )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RD_46 | DEFINITION | NKD_30 | RELATION_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("Instances identify the DEFINITION node for which the source node is an interpretation.") |
| CAIS_CONTROLLED | AD_7 | (TRUE) |
| CAIS_DETERMINED | AD_8 | (FALSE) |
| UNIQUE_KEYS | AD_84 | (TRUE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_10 | ATTRIBUTE | (INHERITABLE) | AD_39 |
| A | AD_14 | DEFAULT_VALUE | (TRUE) | |
| S | RD_10 | ATTRIBUTE | (NODE_KIND) | AD_60 |
| A | AD_10 | CONSTRAINTS | (VALUE => STRUCTURAL) | |
| A | AD_14 | DEFAULT_VALUE | (STRUCTURAL) | |
| S | RD_55 | EMANATES_FROM | (INTERPRETATION) | NKD_17 |
| S | RD_103 | PARENT | () | N_2 |
| S | RD_112 | SPECIALIZATION_OF | (BASIC) | RD_23 |
| S | RD_117 | TERMINATES_AT | (NODE_KIND_DEFINITION) | NKD_25 |
| S | RD_117 | TERMINATES_AT | (RELATION_DEFINITION) | NKD_30 |


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

### 20.5.47 'RELATION_DEFINITION( DEFINITIONS_PARENT )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RD_47 | DEFINITIONS_PARENT | NKD_30 | RELATION_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("Instances identify the parent of the (source) DEFINITIONS node.") |
| CAIS_CONTROLLED | AD_7 | (FALSE) |
| CAIS_DETERMINED | AD_8 | (FALSE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_10 | ATTRIBUTE | (NODE_KIND) | AD_60 |
| A | AD_10 | CONSTRAINTS | (VALUE => STRUCTURAL) | |
| A | AD_14 | DEFAULT_VALUE | (STRUCTURAL) | |
| S | RD_55 | EMANATES_FROM | (DEFINITIONS) | NKD_8 |
| S | RD_103 | PARENT | () | N_2 |
| S | RD_110 | REVERSE_LINK | () | RD_48 |
| S | RD_112 | SPECIALIZATION_OF | (PARENT) | RD_93 |
| S | RD_117 | TERMINATES_AT | (BASIC) | NKD_3 |

### 20.5.48 'RELATION_DEFINITION( DEFINITIONS_PRIMARY )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RD_48 | DEFINITIONS_PRIMARY | NKD_30 | RELATION_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ALLOWED_USAGE | AD_2 | (PRIMARY) |
| ANNOTATION | AD_3 | ("Instances identify DEFINITIONS nodes which group definition nodes, including the predefined definition nodes.") |
| CAIS_CONTROLLED | AD_7 | (FALSE) |
| CAIS_DETERMINED | AD_8 | (FALSE) |
| UNIQUE_KEYS | AD_84 | (TRUE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_10 | ATTRIBUTE | (INHERITABLE) | AD_39 |
| A | AD_10 | CONSTRAINTS | (VALUE => FALSE) | |
| A | AD_14 | DEFAULT_VALUE | (FALSE) | |
| S | RD_10 | ATTRIBUTE | (NODE_KIND) | AD_60 |
| A | AD_10 | CONSTRAINTS | (VALUE => STRUCTURAL) | |
| A | AD_14 | DEFAULT_VALUE | (STRUCTURAL) | |
| S | RD_55 | EMANATES_FROM | (BASIC) | NKD_3 |
| S | RD_81 | KEY | (GENERIC_KEY) | AD_23 |
| S | RD_103 | PARENT | () | N_2 |
| S | RD_110 | REVERSE_LINK | () | RD_47 |
| S | RD_112 | SPECIALIZATION_OF | (BASIC) | RD_23 |
| S | RD_117 | TERMINATES_AT | (DEFINITIONS) | NKD_8 |



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

### 20.5.49 'RELATION_DEFINITION( DEFINITIONS_SECONDARY )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RD_49 | DEFINITIONS_SECONDARY | NKD_30 | RELATION_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("Instances provide identification of DEFINITIONS nodes (e.g., the top-level, predefined DEFINITIONS node) from the (source) current process node.") |
| CAIS_CONTROLLED | AD_7 | (FALSE) |
| CAIS_DETERMINED | AD_8 | (FALSE) |
| UNIQUE_KEYS | AD_84 | (TRUE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_10 | ATTRIBUTE | (INHERITABLE) | AD_39 |
| A | AD_14 | DEFAULT_VALUE | (TRUE) | |
| S | RD_10 | ATTRIBUTE | (NODE_KIND) | AD_60 |
| A | AD_10 | CONSTRAINTS | (VALUE => STRUCTURAL) | |
| A | AD_14 | DEFAULT_VALUE | (STRUCTURAL) | |
| S | RD_55 | EMANATES_FROM | (PROCESS) | NKD_28 |
| S | RD_81 | KEY | (GENERIC_KEY) | AD_23 |
| S | RD_103 | PARENT | () | N_2 |
| S | RD_112 | SPECIALIZATION_OF | (BASIC) | RD_23 |
| S | RD_117 | TERMINATES_AT | (DEFINITIONS) | NKD_8 |

### 20.5.50 'RELATION_DEFINITION( DENY )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RD_50 | DENY | NKD_30 | RELATION_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("Instances identify the groups which can have no access to the (source) object.") |
| CAIS_CONTROLLED | AD_7 | (TRUE) |
| CAIS_DETERMINED | AD_8 | (FALSE) |
| UNIQUE_KEYS | AD_84 | (TRUE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_10 | ATTRIBUTE | (INHERITABLE) | AD_39 |
| A | AD_14 | DEFAULT_VALUE | (TRUE) | |
| S | RD_10 | ATTRIBUTE | (NODE_KIND) | AD_60 |
| A | AD_10 | CONSTRAINTS | (VALUE => STRUCTURAL) | |
| A | AD_14 | DEFAULT_VALUE | (STRUCTURAL) | |
| S | RD_55 | EMANATES_FROM | (BASIC) | NKD_3 |
| S | RD_81 | KEY | (CAIS_KEY) | AD_9 |
| S | RD_103 | PARENT | () | N_2 |
| S | RD_112 | SPECIALIZATION_OF | (BASIC) | RD_23 |
| S | RD_117 | TERMINATES_AT | (GROUP) | NKD_14 |


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

### 20.5.51 'RELATION_DEFINITION( DEVICE_PARENT )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RD_51 | DEVICE_PARENT | NKD_30 | RELATION_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("Instances identify the parent (system node) of the (source) top-level DEVICE node.") |
| CAIS_CONTROLLED | AD_7 | (TRUE) |
| CAIS_DETERMINED | AD_8 | (FALSE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_10 | ATTRIBUTE | (NODE_KIND) | AD_60 |
| A | AD_10 | CONSTRAINTS | (VALUE => STRUCTURAL) | |
| A | AD_14 | DEFAULT_VALUE | (STRUCTURAL) | |
| S | RD_55 | EMANATES_FROM | (DEVICE) | NKD_9 |
| S | RD_103 | PARENT | () | N_2 |
| S | RD_110 | REVERSE_LINK | () | RD_52 |
| S | RD_112 | SPECIALIZATION_OF | (PARENT) | RD_93 |
| S | RD_117 | TERMINATES_AT | (SYSTEM_LEVEL) | NKD_37 |

### 20.5.52 'RELATION_DEFINITION( DEVICE_PRIMARY )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RD_52 | DEVICE_PRIMARY | NKD_30 | RELATION_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ALLOWED_USAGE | AD_2 | (PRIMARY) |
| ANNOTATION | AD_3 | ("Instances are primary relationships to the top-level DEVICE nodes.") |
| CAIS_CONTROLLED | AD_7 | (TRUE) |
| CAIS_DETERMINED | AD_8 | (FALSE) |
| UNIQUE_KEYS | AD_84 | (TRUE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_10 | ATTRIBUTE | (INHERITABLE) | AD_39 |
| A | AD_10 | CONSTRAINTS | (VALUE => FALSE) | |
| A | AD_14 | DEFAULT_VALUE | (FALSE) | |
| S | RD_10 | ATTRIBUTE | (NODE_KIND) | AD_60 |
| A | AD_10 | CONSTRAINTS | (VALUE => FILE) | |
| A | AD_14 | DEFAULT_VALUE | (FILE) | |
| S | RD_55 | EMANATES_FROM | (SYSTEM_LEVEL) | NKD_37 |
| S | RD_81 | KEY | (GENERIC_KEY) | AD_23 |
| S | RD_103 | PARENT | () | N_2 |
| S | RD_110 | REVERSE_LINK | () | RD_51 |
| S | RD_112 | SPECIALIZATION_OF | (BASIC) | RD_23 |
| S | RD_117 | TERMINATES_AT | (DEVICE) | NKD_9 |



Source: https://assist.dla.mil -- Downloaded: 2026l0-26T07:54Z
Check the source to verify that this is the currentversion before use.

### 20.5.53 'RELATION_DEFINITION( DEVICE_SECONDARY )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RD_53 | DEVICE_SECONDARY | NKD_30 | RELATION_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("Instances provide identification of top-level DEVICE nodes from the (source) current process or RESOURCE_GROUP node.") |
| CAIS_CONTROLLED | AD_7 | (TRUE) |
| CAIS_DETERMINED | AD_8 | (FALSE) |
| UNIQUE_KEYS | AD_84 | (TRUE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_10 | ATTRIBUTE | (INHERITABLE) | AD_39 |
| A | AD_14 | DEFAULT_VALUE | (TRUE) | |
| S | RD_10 | ATTRIBUTE | (NODE_KIND) | AD_60 |
| A | AD_10 | CONSTRAINTS | (VALUE => FILE) | |
| A | AD_14 | DEFAULT_VALUE | (FILE) | |
| S | RD_55 | EMANATES_FROM | (PROCESS) | NKD_28 |
| S | RD_55 | EMANATES_FROM | (RESOURCE_GROUP) | NKD_33 |
| S | RD_81 | KEY | (GENERIC_KEY) | AD_23 |
| S | RD_103 | PARENT | () | N_2 |
| S | RD_112 | SPECIALIZATION_OF | (BASIC) | RD_23 |
| S | RD_117 | TERMINATES_AT | (DEVICE) | NKD_9 |

### 20.5.54 'RELATION_DEFINITION( EMANATES )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RD_54 | EMANATES | NKD_30 | RELATION_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ALLOWED_USAGE | AD_2 | (SECONDARY) |
| ANNOTATION | AD_3 | ("Instances identify the emanating relations for the (source) node kind.") |
| CAIS_CONTROLLED | AD_7 | (TRUE) |
| CAIS_DETERMINED | AD_8 | (FALSE) |
| UNIQUE_KEYS | AD_84 | (TRUE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_10 | ATTRIBUTE | (INHERITABLE) | AD_39 |
| A | AD_10 | CONSTRAINTS | (VALUE => FALSE) | |
| A | AD_14 | DEFAULT_VALUE | (FALSE) | |
| S | RD_10 | ATTRIBUTE | (MAXIMUM_INSTANCES) | AD_51 |
| S | RD_10 | ATTRIBUTE | (MINIMUM_INSTANCES) | AD_55 |
| S | RD_10 | ATTRIBUTE | (NODE_KIND) | AD_60 |
| A | AD_10 | CONSTRAINTS | (VALUE => STRUCTURAL) | |
| A | AD_14 | DEFAULT_VALUE | (STRUCTURAL) | |
| S | RD_55 | EMANATES_FROM | (NODE_KIND_DEFINITION) | NKD_25 |
| S | RD_81 | KEY | (GENERIC_KEY) | AD_23 |
| S | RD_103 | PARENT | () | N_2 |
| S | RD_110 | REVERSE_LINK | () | RD_55 |


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_112 | SPECIALIZATION_OF | (BASIC) | RD_23 |
| S | RD_117 | TERMINATES_AT | (RELATION_DEFINITION) | NKD_30 |

### 20.5.55 'RELATION_DEFINITION( EMANATES_FROM )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RD_55 | EMANATES_FROM | NKD_30 | RELATION_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ALLOWED_USAGE | AD_2 | (SECONDARY) |
| ANNOTATION | AD_3 | ("Instances identify the node kinds for the (source) emanating relation.") |
| CAIS_CONTROLLED | AD_7 | (TRUE) |
| CAIS_DETERMINED | AD_8 | (FALSE) |
| UNIQUE_KEYS | AD_84 | (TRUE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_10 | ATTRIBUTE | (INHERITABLE) | AD_39 |
| A | AD_10 | CONSTRAINTS | (VALUE => FALSE) | |
| A | AD_14 | DEFAULT_VALUE | (FALSE) | |
| S | RD_10 | ATTRIBUTE | (NODE_KIND) | AD_60 |
| A | AD_10 | CONSTRAINTS | (VALUE => STRUCTURAL) | |
| A | AD_14 | DEFAULT_VALUE | (STRUCTURAL) | |
| S | RD_55 | EMANATES_FROM | (RELATION_DEFINITION) | NKD_30 |
| S | RD_81 | KEY | (GENERIC_KEY) | AD_23 |
| S | RD_103 | PARENT | () | N_2 |
| S | RD_110 | REVERSE_LINK | () | RD_54 |
| S | RD_112 | SPECIALIZATION_OF | (BASIC) | RD_23 |
| S | RD_117 | TERMINATES_AT | (NODE_KIND_DEFINITION) | NKD_25 |

### 20.5.56 'RELATION_DEFINITION( EXECUTABLE_IMAGE )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RD_56 | EXECUTABLE_IMAGE | NKD_30 | RELATION_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("Instances identify the EXECUTABLE_IMAGE node containing the (source) current process's executable image.") |
| CAIS_CONTROLLED | AD_7 | (TRUE) |
| CAIS_DETERMINED | AD_8 | (TRUE) |
| UNIQUE_KEYS | AD_84 | (TRUE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_10 | ATTRIBUTE | (INHERITABLE) | AD_39 |
| A | AD_10 | CONSTRAINTS | (VALUE => FALSE) | |
| A | AD_14 | DEFAULT_VALUE | (FALSE) | |
| S | RD_10 | ATTRIBUTE | (NODE_KIND) | AD_60 |
| A | AD_10 | CONSTRAINTS | (VALUE => FILE) | |
| A | AD_14 | DEFAULT_VALUE | (FILE) | |
| S | RD_55 | EMANATES_FROM | (PROCESS) | NKD_28 |



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_103 | PARENT | () | N_2 |
| S | RD_112 | SPECIALIZATION_OF | (BASIC) | RD_23 |
| S | RD_117 | TERMINATES_AT | (EXECUTABLE_IMAGE) | NKD_10 |

### 20.5.57 'RELATION_DEFINITION( GATEWAY_PARENT )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RD_57 | GATEWAY_PARENT | NKD_30 | RELATION_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("Instances identify the parent (system node) of the (source) top-level GATEWAY node.") |
| CAIS_CONTROLLED | AD_7 | (TRUE) |
| CAIS_DETERMINED | AD_8 | (FALSE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_10 | ATTRIBUTE | (NODE_KIND) | AD_60 |
| A | AD_10 | CONSTRAINTS | (VALUE => STRUCTURAL) | |
| A | AD_14 | DEFAULT_VALUE | (STRUCTURAL) | |
| S | RD_55 | EMANATES_FROM | (GATEWAY) | NKD_13 |
| S | RD_103 | PARENT | () | N_2 |
| S | RD_110 | REVERSE_LINK | () | RD_58 |
| S | RD_112 | SPECIALIZATION_OF | (PARENT) | RD_93 |
| S | RD_117 | TERMINATES_AT | (SYSTEM_LEVEL) | NKD_37 |

### 20.5.58 'RELATION_DEFINITION( GATEWAY_PRIMARY )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RD_58 | GATEWAY_PRIMARY | NKD_30 | RELATION_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ALLOWED_USAGE | AD_2 | (PRIMARY) |
| ANNOTATION | AD_3 | ("Instances are primary relationships to the top-level GATEWAY nodes.") |
| CAIS_CONTROLLED | AD_7 | (TRUE) |
| CAIS_DETERMINED | AD_8 | (FALSE) |
| UNIQUE_KEYS | AD_84 | (TRUE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_10 | ATTRIBUTE | (INHERITABLE) | AD_39 |
| A | AD_10 | CONSTRAINTS | (VALUE => FALSE) | |
| A | AD_14 | DEFAULT_VALUE | (FALSE) | |
| S | RD_10 | ATTRIBUTE | (NODE_KIND) | AD_60 |
| A | AD_10 | CONSTRAINTS | (VALUE => FILE) | |
| A | AD_14 | DEFAULT_VALUE | (FILE) | |
| S | RD_55 | EMANATES_FROM | (SYSTEM_LEVEL) | NKD_37 |
| S | RD_81 | KEY | (GENERIC_KEY) | AD_23 |
| S | RD_103 | PARENT | () | N_2 |
| S | RD_110 | REVERSE_LINK | () | RD_57 |
| S | RD_112 | SPECIALIZATION_OF | (BASIC) | RD_23 |


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_117 | TERMINATES_AT | (GATEWAY) | NKD_13 |

### 20.5.59 'RELATION_DEFINITION( GATEWAY_SECONDARY )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RD_59 | GATEWAY_SECONDARY | NKD_30 | RELATION_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("Instances provide identification of top-level GATEWAY nodes from the (source) RESOURCE_GROUP node.") |
| CAIS_CONTROLLED | AD_7 | (TRUE) |
| CAIS_DETERMINED | AD_8 | (FALSE) |
| UNIQUE_KEYS | AD_84 | (TRUE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_10 | ATTRIBUTE | (INHERITABLE) | AD_39 |
| A | AD_14 | DEFAULT_VALUE | (TRUE) | |
| S | RD_10 | ATTRIBUTE | (NODE_KIND) | AD_60 |
| A | AD_10 | CONSTRAINTS | (VALUE => FILE) | |
| A | AD_14 | DEFAULT_VALUE | (FILE) | |
| S | RD_55 | EMANATES_FROM | (RESOURCE_GROUP) | NKD_33 |
| S | RD_81 | KEY | (GENERIC_KEY) | AD_23 |
| S | RD_103 | PARENT | () | N_2 |
| S | RD_112 | SPECIALIZATION_OF | (BASIC) | RD_23 |
| S | RD_117 | TERMINATES_AT | (GATEWAY) | NKD_13 |

### 20.5.60 'RELATION_DEFINITION( GENERALIZATION_OF_NKD )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RD_60 | GENERALIZATION_OF_NKD | NKD_30 | RELATION_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ALLOWED_USAGE | AD_2 | (SECONDARY) |
| ANNOTATION | AD_3 | ("Instances identify the generalization NKDs of the (source) NKD.") |
| CAIS_CONTROLLED | AD_7 | (TRUE) |
| CAIS_DETERMINED | AD_8 | (FALSE) |
| UNIQUE_KEYS | AD_84 | (TRUE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_10 | ATTRIBUTE | (INHERITABLE) | AD_39 |
| A | AD_10 | CONSTRAINTS | (VALUE => FALSE) | |
| A | AD_14 | DEFAULT_VALUE | (FALSE) | |
| S | RD_10 | ATTRIBUTE | (NODE_KIND) | AD_60 |
| A | AD_10 | CONSTRAINTS | (VALUE => STRUCTURAL) | |
| A | AD_14 | DEFAULT_VALUE | (STRUCTURAL) | |
| S | RD_55 | EMANATES_FROM | (NODE_KIND_DEFINITION) | NKD_25 |
| S | RD_81 | KEY | (GENERIC_KEY) | AD_23 |
| S | RD_103 | PARENT | () | N_2 |
| S | RD_110 | REVERSE_LINK | () | RD_111 |



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_112 | SPECIALIZATION_OF | (BASIC) | RD_23 |
| S | RD_117 | TERMINATES_AT | (NODE_KIND_DEFINITION) | NKD_25 |

### 20.5.61 'RELATION_DEFINITION( GENERALIZATION_OF_RD )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RD_61 | GENERALIZATION_OF_RD | NKD_30 | RELATION_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ALLOWED_USAGE | AD_2 | (SECONDARY) |
| ANNOTATION | AD_3 | ("Instances identify the generalization RDs of the (source) RD.") |
| CAIS_CONTROLLED | AD_7 | (TRUE) |
| CAIS_DETERMINED | AD_8 | (FALSE) |
| UNIQUE_KEYS | AD_84 | (TRUE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_10 | ATTRIBUTE | (INHERITABLE) | AD_39 |
| A | AD_10 | CONSTRAINTS | (VALUE => FALSE) | |
| A | AD_14 | DEFAULT_VALUE | (FALSE) | |
| S | RD_10 | ATTRIBUTE | (NODE_KIND) | AD_60 |
| A | AD_10 | CONSTRAINTS | (VALUE => STRUCTURAL) | |
| A | AD_14 | DEFAULT_VALUE | (STRUCTURAL) | |
| S | RD_55 | EMANATES_FROM | (RELATION_DEFINITION) | NKD_30 |
| S | RD_81 | KEY | (GENERIC_KEY) | AD_23 |
| S | RD_103 | PARENT | () | N_2 |
| S | RD_110 | REVERSE_LINK | () | RD_112 |
| S | RD_112 | SPECIALIZATION_OF | (BASIC) | RD_23 |
| S | RD_117 | TERMINATES_AT | (RELATION_DEFINITION) | NKD_30 |

### 20.5.62 'RELATION_DEFINITION( GROUP_PARENT )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RD_62 | GROUP_PARENT | NKD_30 | RELATION_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("Instances identify the parent (system node) of the (source) top-level GROUP node.") |
| CAIS_CONTROLLED | AD_7 | (TRUE) |
| CAIS_DETERMINED | AD_8 | (FALSE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_10 | ATTRIBUTE | (NODE_KIND) | AD_60 |
| A | AD_10 | CONSTRAINTS | (VALUE => STRUCTURAL) | |
| A | AD_14 | DEFAULT_VALUE | (STRUCTURAL) | |
| S | RD_55 | EMANATES_FROM | (GROUP) | NKD_14 |
| S | RD_103 | PARENT | () | N_2 |
| S | RD_110 | REVERSE_LINK | () | RD_63 |
| S | RD_112 | SPECIALIZATION_OF | (PARENT) | RD_93 |
| S | RD_117 | TERMINATES_AT | (SYSTEM_LEVEL) | NKD_37 |


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

### 20.5.63 'RELATION_DEFINITION( GROUP_PRIMARY )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RD_63 | GROUP_PRIMARY | NKD_30 | RELATION_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ALLOWED_USAGE | AD_2 | (PRIMARY) |
| ANNOTATION | AD_3 | ("Instances are primary relationships to the top-level GROUP nodes.") |
| CAIS_CONTROLLED | AD_7 | (TRUE) |
| CAIS_DETERMINED | AD_8 | (FALSE) |
| UNIQUE_KEYS | AD_84 | (TRUE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_10 | ATTRIBUTE | (INHERITABLE) | AD_39 |
| A | AD_10 | CONSTRAINTS | (VALUE => FALSE) | |
| A | AD_14 | DEFAULT_VALUE | (FALSE) | |
| S | RD_10 | ATTRIBUTE | (NODE_KIND) | AD_60 |
| A | AD_10 | CONSTRAINTS | (VALUE => STRUCTURAL) | |
| A | AD_14 | DEFAULT_VALUE | (STRUCTURAL) | |
| S | RD_55 | EMANATES_FROM | (SYSTEM_LEVEL) | NKD_37 |
| S | RD_81 | KEY | (GENERIC_KEY) | AD_23 |
| S | RD_103 | PARENT | () | N_2 |
| S | RD_110 | REVERSE_LINK | () | RD_62 |
| S | RD_112 | SPECIALIZATION_OF | (BASIC) | RD_23 |
| S | RD_117 | TERMINATES_AT | (GROUP) | NKD_14 |

### 20.5.64 'RELATION_DEFINITION( GROUP_SECONDARY )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RD_64 | GROUP_SECONDARY | NKD_30 | RELATION_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("Instances provide identification of top-level GROUP nodes from the (source) current process node.") |
| CAIS_CONTROLLED | AD_7 | (TRUE) |
| CAIS_DETERMINED | AD_8 | (FALSE) |
| UNIQUE_KEYS | AD_84 | (TRUE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_10 | ATTRIBUTE | (INHERITABLE) | AD_39 |
| A | AD_14 | DEFAULT_VALUE | (TRUE) | |
| S | RD_10 | ATTRIBUTE | (NODE_KIND) | AD_60 |
| A | AD_10 | CONSTRAINTS | (VALUE => STRUCTURAL) | |
| A | AD_14 | DEFAULT_VALUE | (STRUCTURAL) | |
| S | RD_55 | EMANATES_FROM | (PROCESS) | NKD_28 |
| S | RD_81 | KEY | (GENERIC_KEY) | AD_23 |
| S | RD_103 | PARENT | () | N_2 |
| S | RD_112 | SPECIALIZATION_OF | (BASIC) | RD_23 |
| S | RD_117 | TERMINATES_AT | (GROUP) | NKD_14 |



Source: https://assist.dla.mil -- Downloaded: 202640-26T07:54Z
Check the source to verify that this is the current version before use.

### 20.5.65 'RELATION_DEFINITION( HANDLER_ATTRIBUTE )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RD_65 | HANDLER_ATTRIBUTE | NKD_30 | RELATION_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("Instances identify the PROCESS node attributes to be created when an event handler process is initiated for the (source) attribute monitor.") |
| CAIS_CONTROLLED | AD_7 | (TRUE) |
| CAIS_DETERMINED | AD_8 | (FALSE) |
| UNIQUE_KEYS | AD_84 | (TRUE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_10 | ATTRIBUTE | (INHERITABLE) | AD_39 |
| A | AD_14 | DEFAULT_VALUE | (TRUE) | |
| S | RD_10 | ATTRIBUTE | (NODE_KIND) | AD_60 |
| A | AD_10 | CONSTRAINTS | (VALUE => STRUCTURAL) | |
| A | AD_14 | DEFAULT_VALUE | (STRUCTURAL) | |
| S | RD_10 | ATTRIBUTE | (ATTRIBUTE_VALUE) | AD_4 |
| S | RD_55 | EMANATES_FROM | (MONITOR) | NKD_22 |
| S | RD_81 | KEY | (CAIS_KEY) | AD_9 |
| S | RD_103 | PARENT | () | N_2 |
| S | RD_112 | SPECIALIZATION_OF | (BASIC) | RD_23 |
| S | RD_117 | TERMINATES_AT | (ATTRIBUTE_DEFINITION) | NKD_1 |

### 20.5.66 'RELATION_DEFINITION( HANDLER_ENVIRONMENT_NODE )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RD_66 | HANDLER_ENVIRONMENT_-NODE | NKD_30 | RELATION_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("Instances identify the node at which the CURRENT_-NODE relationship is to terminate when an event handler process is initiated for the (source) attribute monitor.") |
| CAIS_CONTROLLED | AD_7 | (TRUE) |
| CAIS_DETERMINED | AD_8 | (FALSE) |
| UNIQUE_KEYS | AD_84 | (TRUE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_10 | ATTRIBUTE | (INHERITABLE) | AD_39 |
| A | AD_14 | DEFAULT_VALUE | (TRUE) | |
| S | RD_55 | EMANATES_FROM | (MONITOR) | NKD_22 |
| S | RD_103 | PARENT | () | N_2 |
| S | RD_112 | SPECIALIZATION_OF | (BASIC) | RD_23 |
| S | RD_117 | TERMINATES_AT | (BASIC) | NKD_3 |


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

### 20.5.67 'RELATION_DEFINITION( HANDLER_ERROR_FILE )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RD_67 | HANDLER_ERROR_FILE | NKD_30 | RELATION_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("Instances identify the file at which the STANDARD_-ERROR relationship is to terminate when an event handler process is initiated for the (source) attribute monitor.") |
| CAIS_CONTROLLED | AD_7 | (TRUE) |
| CAIS_DETERMINED | AD_8 | (FALSE) |
| UNIQUE_KEYS | AD_84 | (TRUE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_10 | ATTRIBUTE | (INHERITABLE) | AD_39 |
| A | AD_14 | DEFAULT_VALUE | (TRUE) | |
| S | RD_10 | ATTRIBUTE | (NODE_KIND) | AD_60 |
| A | AD_10 | CONSTRAINTS | (VALUE => FILE) | |
| A | AD_14 | DEFAULT_VALUE | (FILE) | |
| S | RD_55 | EMANATES_FROM | (MONITOR) | NKD_22 |
| S | RD_103 | PARENT | () | N_2 |
| S | RD_112 | SPECIALIZATION_OF | (BASIC) | RD_23 |
| S | RD_117 | TERMINATES_AT | (FILE) | NKD_11 |

### 20.5.68 'RELATION_DEFINITION( HANDLER_IMAGE )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RD_68 | HANDLER_IMAGE | NKD_30 | RELATION_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("Instances identify the node at which the EXECUTABLE_-IMAGE relationship is to terminate when an event handler process is initiated for the (source) attribute monitor.") |
| CAIS_CONTROLLED | AD_7 | (TRUE) |
| CAIS_DETERMINED | AD_8 | (FALSE) |
| UNIQUE_KEYS | AD_84 | (TRUE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_10 | ATTRIBUTE | (INHERITABLE) | AD_39 |
| A | AD_14 | DEFAULT_VALUE | (TRUE) | |
| S | RD_10 | ATTRIBUTE | (NODE_KIND) | AD_60 |
| A | AD_10 | CONSTRAINTS | (VALUE => FILE) | |
| A | AD_14 | DEFAULT_VALUE | (FILE) | |
| S | RD_55 | EMANATES_FROM | (MONITOR) | NKD_22 |
| S | RD_103 | PARENT | () | N_2 |
| S | RD_112 | SPECIALIZATION_OF | (BASIC) | RD_23 |
| S | RD_117 | TERMINATES_AT | (EXECUTABLE_IMAGE) | NKD_10 |



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

### 20.5.69 'RELATION_DEFINITION( HANDLER_INPUT_FILE )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RD_69 | HANDLER_INPUT_FILE | NKD_30 | RELATION_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("Instances identify the node at which the STANDARD_-INPUT relationship is to terminate when an event handler process is initiated for the (source) attribute monitor.") |
| CAIS_CONTROLLED | AD_7 | (TRUE) |
| CAIS_DETERMINED | AD_8 | (FALSE) |
| UNIQUE_KEYS | AD_84 | (TRUE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_10 | ATTRIBUTE | (INHERITABLE) | AD_39 |
| A | AD_14 | DEFAULT_VALUE | (TRUE) | |
| S | RD_10 | ATTRIBUTE | (NODE_KIND) | AD_60 |
| S | RD_55 | EMANATES_FROM | (MONITOR) | NKD_22 |
| S | RD_103 | PARENT | () | N_2 |
| S | RD_112 | SPECIALIZATION_OF | (BASIC) | RD_23 |
| S | RD_117 | TERMINATES_AT | (FILE) | NKD_11 |

### 20.5.70 'RELATION_DEFINITION( HANDLER_MEDIUM_RESOURCE )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RD_70 | HANDLER_MEDIUM_RESOURCE | NKD_30 | RELATION_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("Instances identify the medium resource on which the process node is to be allocated when an event handler process is initiated for the (source) attribute monitor.") |
| CAIS_CONTROLLED | AD_7 | (TRUE) |
| CAIS_DETERMINED | AD_8 | (FALSE) |
| UNIQUE_KEYS | AD_84 | (TRUE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_10 | ATTRIBUTE | (INHERITABLE) | AD_39 |
| A | AD_14 | DEFAULT_VALUE | (TRUE) | |
| S | RD_10 | ATTRIBUTE | (NODE_KIND) | AD_60 |
| A | AD_10 | CONSTRAINTS | (VALUE => STRUCTURAL) | |
| A | AD_14 | DEFAULT_VALUE | (STRUCTURAL) | |
| S | RD_55 | EMANATES_FROM | (MONITOR) | NKD_22 |
| S | RD_103 | PARENT | () | N_2 |
| S | RD_112 | SPECIALIZATION_OF | (BASIC) | RD_23 |
| S | RD_117 | TERMINATES_AT | (MEDIUM) | NKD_21 |

### 20.5.71 'RELATION_DEFINITION( HANDLER_OUTPUT_FILE )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RD_71 | HANDLER_OUTPUT_FILE | NKD_30 | RELATION_DEFINITION |


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("Instances identify the node at which the STANDARD_-OUTPUT relationship is to terminate when an event handler process is initiated for the (source) attribute monitor.") |
| CAIS_CONTROLLED | AD_7 | (TRUE) |
| CAIS_DETERMINED | AD_8 | (FALSE) |
| UNIQUE_KEYS | AD_84 | (TRUE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_10 | ATTRIBUTE | (INHERITABLE) | AD_39 |
| A | AD_14 | DEFAULT_VALUE | (TRUE) | |
| S | RD_10 | ATTRIBUTE | (NODE_KIND) | AD_60 |
| S | RD_55 | EMANATES_FROM | (MONITOR) | NKD_22 |
| S | RD_103 | PARENT | () | N_2 |
| S | RD_112 | SPECIALIZATION_OF | (BASIC) | RD_23 |
| S | RD_117 | TERMINATES_AT | (FILE) | NKD_11 |

### 20.5.72 'RELATION_DEFINITION( HANDLER_PROCESS )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RD_72 | HANDLER_PROCESS | NKD_30 | RELATION_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("Instances identify the event handler processes for the (source) attribute monitor.") |
| CAIS_CONTROLLED | AD_7 | (TRUE) |
| CAIS_DETERMINED | AD_8 | (TRUE) |
| UNIQUE_KEYS | AD_84 | (TRUE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_10 | ATTRIBUTE | (INHERITABLE) | AD_39 |
| A | AD_10 | CONSTRAINTS | (VALUE => FALSE) | |
| A | AD_14 | DEFAULT_VALUE | (FALSE) | |
| S | RD_10 | ATTRIBUTE | (NODE_KIND) | AD_60 |
| A | AD_10 | CONSTRAINTS | (VALUE => PROCESS) | |
| A | AD_14 | DEFAULT_VALUE | (PROCESS) | |
| S | RD_55 | EMANATES_FROM | (MONITOR) | NKD_22 |
| S | RD_81 | KEY | (CAIS_KEY) | AD_9 |
| S | RD_103 | PARENT | () | N_2 |
| S | RD_110 | REVERSE_LINK | () | RD_74 |
| S | RD_112 | SPECIALIZATION_OF | (BASIC) | RD_23 |
| S | RD_117 | TERMINATES_AT | (PROCESS) | NKD_28 |

### 20.5.73 'RELATION_DEFINITION( HANDLER_PROCESS_NODE_KIND )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RD_73 | HANDLER_PROCESS_NODE_-KIND | NKD_30 | RELATION_DEFINITION |



Source: https://assist.dla.mil -- Downloaded: 202610-26T07:54Z
Check the source to verify that this is the current version before use.

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("Instances identify the kind of PROCESS node to be created when an event handler process is initiated for the (source) attribute monitor.") |
| CAIS_CONTROLLED | AD_7 | (TRUE) |
| CAIS_DETERMINED | AD_8 | (FALSE) |
| UNIQUE_KEYS | AD_84 | (TRUE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_10 | ATTRIBUTE | (INHERITABLE) | AD_39 |
| A | AD_14 | DEFAULT_VALUE | (TRUE) | |
| S | RD_10 | ATTRIBUTE | (NODE_KIND) | AD_60 |
| A | AD_10 | CONSTRAINTS | (VALUE => STRUCTURAL) | |
| A | AD_14 | DEFAULT_VALUE | (STRUCTURAL) | |
| S | RD_55 | EMANATES_FROM | (MONITOR) | NKD_22 |
| S | RD_103 | PARENT | () | N_2 |
| S | RD_112 | SPECIALIZATION_OF | (BASIC) | RD_23 |
| S | RD_117 | TERMINATES_AT | (NODE_KIND_DEFINITION) | NKD_25 |

### 20.5.74 'RELATION_DEFINITION( HANDLER_PROCESS_OF )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RD_74 | HANDLER_PROCESS_OF | NKD_30 | RELATION_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("Instances identify the MONITOR nodes with which the (source) PROCESS nodes are associated.") |
| CAIS_CONTROLLED | AD_7 | (TRUE) |
| CAIS_DETERMINED | AD_8 | (FALSE) |
| UNIQUE_KEYS | AD_84 | (TRUE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_10 | ATTRIBUTE | (INHERITABLE) | AD_39 |
| A | AD_10 | CONSTRAINTS | (VALUE => FALSE) | |
| A | AD_14 | DEFAULT_VALUE | (FALSE) | |
| S | RD_10 | ATTRIBUTE | (NODE_KIND) | AD_60 |
| A | AD_10 | CONSTRAINTS | (VALUE => STRUCTURAL) | |
| A | AD_14 | DEFAULT_VALUE | (STRUCTURAL) | |
| S | RD_55 | EMANATES_FROM | (PROCESS) | NKD_28 |
| S | RD_103 | PARENT | () | N_2 |
| S | RD_110 | REVERSE_LINK | () | RD_72 |
| S | RD_112 | SPECIALIZATION_OF | (BASIC) | RD_23 |
| S | RD_117 | TERMINATES_AT | (MONITOR) | NKD_22 |

### 20.5.75 'RELATION_DEFINITION( HANDLER_PROCESSOR_RESOURCE )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RD_75 | HANDLER_PROCESSOR_-RESOURCE | NKD_30 | RELATION_DEFINITION |


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("Instances identify the processor resource on which the process is to be allocated when an event handler process is initiated for the (source) attribute monitor.") |
| CAIS_CONTROLLED | AD_7 | (TRUE) |
| CAIS_DETERMINED | AD_8 | (FALSE) |
| UNIQUE_KEYS | AD_84 | (TRUE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_10 | ATTRIBUTE | (INHERITABLE) | AD_39 |
| A | AD_14 | DEFAULT_VALUE | (TRUE) | |
| S | RD_10 | ATTRIBUTE | (NODE_KIND) | AD_60 |
| A | AD_10 | CONSTRAINTS | (VALUE => STRUCTURAL) | |
| A | AD_14 | DEFAULT_VALUE | (STRUCTURAL) | |
| S | RD_55 | EMANATES_FROM | (MONITOR) | NKD_22 |
| S | RD_103 | PARENT | () | N_2 |
| S | RD_112 | SPECIALIZATION_OF | (BASIC) | RD_23 |
| S | RD_117 | TERMINATES_AT | (PROCESSOR) | NKD_29 |

### 20.5.76 'RELATION_DEFINITION( INTRODUCED_NODE )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RD_76 | INTRODUCED_NODE | NKD_30 | RELATION_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ALLOWED_USAGE | AD_2 | (PRIMARY) |
| ANNOTATION | AD_3 | ("Instances are primary relationships to introduced nodes.") |
| CAIS_CONTROLLED | AD_7 | (FALSE) |
| CAIS_DETERMINED | AD_8 | (FALSE) |
| UNIQUE_KEYS | AD_84 | (TRUE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_10 | ATTRIBUTE | (INHERITABLE) | AD_39 |
| A | AD_10 | CONSTRAINTS | (VALUE => FALSE) | |
| A | AD_14 | DEFAULT_VALUE | (FALSE) | |
| S | RD_10 | ATTRIBUTE | (NODE_KIND) | AD_60 |
| A | AD_10 | CONSTRAINTS | (VALUE => FILE) | |
| A | AD_14 | DEFAULT_VALUE | (FILE) | |
| S | RD_55 | EMANATES_FROM | (BASIC) | NKD_3 |
| S | RD_81 | KEY | (GENERIC_KEY) | AD_23 |
| S | RD_103 | PARENT | () | N_2 |
| S | RD_110 | REVERSE_LINK | () | RD_77 |
| S | RD_112 | SPECIALIZATION_OF | (BASIC) | RD_23 |
| S | RD_117 | TERMINATES_AT | (INTRODUCED_NODE) | NKD_18 |

### 20.5.77 'RELATION_DEFINITION( INTRODUCED_NODE_PARENT )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RD_77 | INTRODUCED_NODE_PARENT | NKD_30 | RELATION_DEFINITION |



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("Instances identify the parent of the (source) INTRODUCED_NODE.") |
| CAIS_CONTROLLED | AD_7 | (FALSE) |
| CAIS_DETERMINED | AD_8 | (FALSE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_55 | EMANATES_FROM | (INTRODUCED_NODE) | NKD_18 |
| S | RD_103 | PARENT | () | N_2 |
| S | RD_110 | REVERSE_LINK | () | RD_76 |
| S | RD_112 | SPECIALIZATION_OF | (PARENT) | RD_93 |
| S | RD_117 | TERMINATES_AT | (BASIC) | NKD_3 |

### 20.5.78 'RELATION_DEFINITION( INTRODUCED_RELATION )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RD_78 | INTRODUCED_RELATION | NKD_30 | RELATION_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("Instances are the introduced representation of relationships between imported nodes.") |
| CAIS_CONTROLLED | AD_7 | (TRUE) |
| CAIS_DETERMINED | AD_8 | (TRUE) |
| UNIQUE_KEYS | AD_84 | (TRUE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_10 | ATTRIBUTE | (IMPORTED_NAME) | AD_34 |
| S | RD_10 | ATTRIBUTE | (IMPORTED_RELATIONSHIP_KIND) | AD_35 |
| S | RD_10 | ATTRIBUTE | (IMPORTED_TARGET_UID) | AD_36 |
| S<br>A | RD_10<br>AD_14 | ATTRIBUTE<br>DEFAULT_VALUE | (INHERITABLE)<br>(TRUE) | AD_39 |
| S | RD_10 | ATTRIBUTE | (INTRODUCED_KEY_ATTRIBUTES) | AD_41 |
| S | RD_10 | ATTRIBUTE | (INTRODUCED_RELATIONSHIP_ATTRIBUTES) | AD_43 |
| S<br>A<br>A | RD_10<br>AD_10<br>AD_14 | ATTRIBUTE<br>CONSTRAINTS<br>DEFAULT_VALUE | (NODE_KIND)<br>(VALUE => FILE)<br>(FILE) | AD_60 |
| S | RD_55 | EMANATES_FROM | (INTRODUCED_NODE) | NKD_18 |
| S | RD_81 | KEY | (CAIS_KEY) | AD_9 |
| S | RD_103 | PARENT | () | N_2 |
| S | RD_112 | SPECIALIZATION_OF | (BASIC) | RD_23 |
| S | RD_117 | TERMINATES_AT | (INTRODUCED_NODE) | NKD_18 |

### 20.5.79 'RELATION_DEFINITION( JOB )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RD_79 | JOB | NKD_30 | RELATION_DEFINITION |



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ALLOWED_USAGE | AD_2 | (PRIMARY) |
| ANNOTATION | AD_3 | ("Instances identify the top level processes of the (source) current user.") |
| CAIS_CONTROLLED | AD_7 | (TRUE) |
| CAIS_DETERMINED | AD_8 | (FALSE) |
| UNIQUE_KEYS | AD_84 | (TRUE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_10 | ATTRIBUTE | (INHERITABLE) | AD_39 |
| A | AD_10 | CONSTRAINTS | (VALUE => FALSE) | |
| A | AD_14 | DEFAULT_VALUE | (FALSE) | |
| S | RD_10 | ATTRIBUTE | (NODE_KIND) | AD_60 |
| A | AD_10 | CONSTRAINTS | (VALUE => PROCESS) | |
| A | AD_14 | DEFAULT_VALUE | (PROCESS) | |
| S | RD_55 | EMANATES_FROM | (USER) | NKD_40 |
| S | RD_81 | KEY | (GENERIC_KEY) | AD_23 |
| S | RD_103 | PARENT | () | N_2 |
| S | RD_110 | REVERSE_LINK | () | RD_80 |
| S | RD_112 | SPECIALIZATION_OF | (BASIC) | RD_23 |
| S | RD_117 | TERMINATES_AT | (PROCESS) | NKD_28 |

### 20.5.80 'RELATION_DEFINITION( JOB_PARENT )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RD_80 | JOB_PARENT | NKD_30 | RELATION_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("Instances identify the top level PROCESS node's parent, the (target) current USER node.") |
| CAIS_CONTROLLED | AD_7 | (TRUE) |
| CAIS_DETERMINED | AD_8 | (FALSE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_10 | ATTRIBUTE | (NODE_KIND) | AD_60 |
| A | AD_10 | CONSTRAINTS | (VALUE => STRUCTURAL) | |
| A | AD_14 | DEFAULT_VALUE | (STRUCTURAL) | |
| S | RD_55 | EMANATES_FROM | (PROCESS) | NKD_28 |
| S | RD_103 | PARENT | () | N_2 |
| S | RD_110 | REVERSE_LINK | () | RD_79 |
| S | RD_112 | SPECIALIZATION_OF | (PARENT) | RD_93 |
| S | RD_117 | TERMINATES_AT | (USER) | NKD_40 |

### 20.5.81 'RELATION_DEFINITION( KEY )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RD_81 | KEY | NKD_30 | RELATION_DEFINITION |


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("Instances identify the keys for the (source) relation.") |
| CAIS_CONTROLLED | AD_7 | (TRUE) |
| CAIS_DETERMINED | AD_8 | (FALSE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_55 | EMANATES_FROM | (RELATION_DEFINITION) | NKD_30 |
| S | RD_103 | PARENT | () | N_2 |
| S | RD_110 | REVERSE_LINK | () | RD_83 |
| S | RD_112 | SPECIALIZATION_OF | (ATTRIBUTE) | RD_10 |
| S | RD_117 | TERMINATES_AT | (ATTRIBUTE_DEFINITION) | NKD_1 |

### 20.5.82 'RELATION_DEFINITION( KEY_NAME )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RD_82 | KEY_NAME | NKD_30 | RELATION_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("Instances specify a relation interpretation-relative name for the (target) key.") |
| CAIS_CONTROLLED | AD_7 | (TRUE) |
| CAIS_DETERMINED | AD_8 | (FALSE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_55 | EMANATES_FROM | (RELATION_INTERPRETATION) | NKD_31 |
| S | RD_103 | PARENT | () | N_2 |
| S | RD_112 | SPECIALIZATION_OF | (ATTRIBUTE_NAME) | RD_21 |
| S | RD_117 | TERMINATES_AT | (ATTRIBUTE_DEFINITION) | NKD_1 |

### 20.5.83 'RELATION_DEFINITION( KEY_OF )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RD_83 | KEY_OF | NKD_30 | RELATION_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("Instances identify the relations utilizing the (source) key.") |
| CAIS_CONTROLLED | AD_7 | (TRUE) |
| CAIS_DETERMINED | AD_8 | (FALSE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_55 | EMANATES_FROM | (ATTRIBUTE_DEFINITION) | NKD_1 |
| S | RD_103 | PARENT | () | N_2 |
| S | RD_110 | REVERSE_LINK | () | RD_81 |
| S | RD_112 | SPECIALIZATION_OF | (ATTRIBUTE_OF) | RD_81 |
| S | RD_117 | TERMINATES_AT | (RELATION_DEFINITION) | NKD_30 |



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

### 20.5.84 'RELATION_DEFINITION( MEDIUM_PARENT )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RD_84 | MEDIUM_PARENT | NKD_30 | RELATION_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("Instances identify the parent (system node) of the (source) top-level MEDIUM node.") |
| CAIS_CONTROLLED | AD_7 | (TRUE) |
| CAIS_DETERMINED | AD_8 | (FALSE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_10 | ATTRIBUTE | (NODE_KIND) | AD_60 |
| A | AD_10 | CONSTRAINTS | (VALUE => STRUCTURAL) | |
| A | AD_14 | DEFAULT_VALUE | (STRUCTURAL) | |
| S | RD_55 | EMANATES_FROM | (MEDIUM) | NKD_21 |
| S | RD_103 | PARENT | () | N_2 |
| S | RD_110 | REVERSE_LINK | () | RD_85 |
| S | RD_112 | SPECIALIZATION_OF | (PARENT) | RD_93 |
| S | RD_117 | TERMINATES_AT | (SYSTEM_LEVEL) | NKD_37 |

### 20.5.85 'RELATION_DEFINITION( MEDIUM_PRIMARY )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RD_85 | MEDIUM_PRIMARY | NKD_30 | RELATION_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ALLOWED_USAGE | AD_2 | (PRIMARY) |
| ANNOTATION | AD_3 | ("Instances are primary relationships to the top-level MEDIUM nodes.") |
| CAIS_CONTROLLED | AD_7 | (TRUE) |
| CAIS_DETERMINED | AD_8 | (FALSE) |
| UNIQUE_KEYS | AD_84 | (TRUE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_10 | ATTRIBUTE | (INHERITABLE) | AD_39 |
| A | AD_10 | CONSTRAINTS | (VALUE => FALSE) | |
| A | AD_14 | DEFAULT_VALUE | (FALSE) | |
| S | RD_10 | ATTRIBUTE | (NODE_KIND) | AD_60 |
| A | AD_10 | CONSTRAINTS | (VALUE => STRUCTURAL) | |
| A | AD_14 | DEFAULT_VALUE | (STRUCTURAL) | |
| S | RD_55 | EMANATES_FROM | (SYSTEM_LEVEL) | NKD_37 |
| S | RD_81 | KEY | (GENERIC_KEY) | AD_23 |
| S | RD_103 | PARENT | () | N_2 |
| S | RD_110 | REVERSE_LINK | () | RD_84 |
| S | RD_112 | SPECIALIZATION_OF | (BASIC) | RD_23 |
| S | RD_117 | TERMINATES_AT | (MEDIUM) | NKD_21 |



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

### 20.5.86 'RELATION_DEFINITION( MEDIUM_SECONDARY )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RD_86 | MEDIUM_SECONDARY | NKD_30 | RELATION_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("Instances provide identification of top-level MEDIUM nodes from the (source) RESOURCE_GROUP node.") |
| CAIS_CONTROLLED | AD_7 | (TRUE) |
| CAIS_DETERMINED | AD_8 | (FALSE) |
| UNIQUE_KEYS | AD_84 | (TRUE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_10 | ATTRIBUTE | (INHERITABLE) | AD_39 |
| A | AD_14 | DEFAULT_VALUE | (TRUE) | |
| S | RD_10 | ATTRIBUTE | (NODE_KIND) | AD_60 |
| A | AD_10 | CONSTRAINTS | (VALUE => STRUCTURAL) | |
| A | AD_14 | DEFAULT_VALUE | (STRUCTURAL) | |
| S | RD_55 | EMANATES_FROM | (RESOURCE_GROUP) | NKD_33 |
| S | RD_81 | KEY | (GENERIC_KEY) | AD_23 |
| S | RD_103 | PARENT | () | N_2 |
| S | RD_112 | SPECIALIZATION_OF | (BASIC) | RD_23 |
| S | RD_117 | TERMINATES_AT | (MEDIUM) | NKD_21 |

### 20.5.87 'RELATION_DEFINITION( MONITORED_BY )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RD_87 | MONITORED_BY | NKD_30 | RELATION_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ALLOWED_USAGE | AD_2 | (SECONDARY) |
| ANNOTATION | AD_3 | ("Instances identify the attribute monitors for the (source) node.") |
| CAIS_CONTROLLED | AD_7 | (TRUE) |
| CAIS_DETERMINED | AD_8 | (FALSE) |
| UNIQUE_KEYS | AD_84 | (TRUE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_10 | ATTRIBUTE | (INHERITABLE) | AD_39 |
| A | AD_10 | CONSTRAINTS | (VALUE => FALSE) | |
| A | AD_14 | DEFAULT_VALUE | (FALSE) | |
| S | RD_10 | ATTRIBUTE | (NODE_KIND) | AD_60 |
| A | AD_10 | CONSTRAINTS | (VALUE => STRUCTURAL) | |
| A | AD_14 | DEFAULT_VALUE | (STRUCTURAL) | |
| S | RD_55 | EMANATES_FROM | (BASIC) | NKD_3 |
| S | RD_81 | KEY | (CAIS_KEY) | AD_9 |
| S | RD_103 | PARENT | () | N_2 |
| S | RD_110 | REVERSE_LINK | () | RD_88 |
| S | RD_112 | SPECIALIZATION_OF | (BASIC) | RD_23 |
| S | RD_117 | TERMINATES_AT | (MONITOR) | NKD_22 |



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

### 20.5.88 'RELATION_DEFINITION( MONITORS )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RD_88 | MONITORS | NKD_30 | RELATION_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ALLOWED_USAGE | AD_2 | (SECONDARY) |
| ANNOTATION | AD_3 | ("Instances identify the nodes being monitored by the (source) attribute monitor.") |
| CAIS_CONTROLLED | AD_7 | (TRUE) |
| CAIS_DETERMINED | AD_8 | (FALSE) |
| UNIQUE_KEYS | AD_84 | (TRUE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_10 | ATTRIBUTE | (ATTRIBUTES_MONITORED) | AD_5 |
| S | RD_10 | ATTRIBUTE | (INHERITABLE) | AD_39 |
| A | AD_10 | CONSTRAINTS | (VALUE => FALSE) | |
| A | AD_14 | DEFAULT_VALUE | (FALSE) | |
| S | RD_10 | ATTRIBUTE | (NODE_KIND) | AD_60 |
| S | RD_55 | EMANATES_FROM | (MONITOR) | NKD_22 |
| S | RD_81 | KEY | (CAIS_KEY) | AD_9 |
| S | RD_103 | PARENT | () | N_2 |
| S | RD_110 | REVERSE_LINK | () | RD_87 |
| S | RD_112 | SPECIALIZATION_OF | (BASIC) | RD_23 |
| S | RD_117 | TERMINATES_AT | (BASIC) | NKD_3 |

### 20.5.89 'RELATION_DEFINITION( NODE_KIND )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RD_89 | NODE_KIND | NKD_30 | RELATION_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ALLOWED_USAGE | AD_2 | (PRIMARY) |
| ANNOTATION | AD_3 | ("Instances are primary relationships providing view-relative names for the (target) NKI.") |
| CAIS_CONTROLLED | AD_7 | (TRUE) |
| CAIS_DETERMINED | AD_8 | (FALSE) |
| UNIQUE_KEYS | AD_84 | (TRUE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_10 | ATTRIBUTE | (INHERITABLE) | AD_39 |
| A | AD_10 | CONSTRAINTS | (VALUE => FALSE) | |
| A | AD_14 | DEFAULT_VALUE | (FALSE) | |
| S | RD_10 | ATTRIBUTE | (NODE_KIND) | AD_60 |
| A | AD_10 | CONSTRAINTS | (VALUE => STRUCTURAL) | |
| A | AD_14 | DEFAULT_VALUE | (STRUCTURAL) | |
| S | RD_55 | EMANATES_FROM | (VIEW) | NKD_42 |
| S | RD_81 | KEY | (NAME_KEY) | AD_58 |
| S | RD_103 | PARENT | () | N_2 |
| S | RD_110 | REVERSE_LINK | () | RD_92 |
| S | RD_112 | SPECIALIZATION_OF | (BASIC) | RD_23 |



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_117 | TERMINATES_AT | (NODE_KIND_-INTERPRETATION) | NKD_26 |

### 20.5.90 'RELATION_DEFINITION( NODE_KIND_DEFINITION )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RD_90 | NODE_KIND_DEFINITION | NKD_30 | RELATION_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ALLOWED_USAGE | AD_2 | (PRIMARY) |
| ANNOTATION | AD_3 | ("Instances are primary relationships to NKDs.") |
| CAIS_CONTROLLED | AD_7 | (FALSE) |
| CAIS_DETERMINED | AD_8 | (FALSE) |
| UNIQUE_KEYS | AD_84 | (TRUE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_10 | ATTRIBUTE | (INHERITABLE) | AD_39 |
| A | AD_10 | CONSTRAINTS | (VALUE => FALSE) | |
| A | AD_14 | DEFAULT_VALUE | (FALSE) | |
| S | RD_10 | ATTRIBUTE | (NODE_KIND) | AD_60 |
| A | AD_10 | CONSTRAINTS | (VALUE => STRUCTURAL) | |
| A | AD_14 | DEFAULT_VALUE | (STRUCTURAL) | |
| S | RD_55 | EMANATES_FROM | (DEFINITIONS) | NKD_8 |
| S | RD_81 | KEY | (GENERIC_KEY) | AD_23 |
| S | RD_103 | PARENT | () | N_2 |
| S | RD_110 | REVERSE_LINK | () | RD_91 |
| S | RD_112 | SPECIALIZATION_OF | (BASIC) | RD_23 |
| S | RD_117 | TERMINATES_AT | (NODE_KIND_DEFINITION) | NKD_25 |

### 20.5.91 'RELATION_DEFINITION( NODE_KIND_DEFINITION_PARENT )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RD_91 | NODE_KIND_DEFINITION_-PARENT | NKD_30 | RELATION_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("Instances identify the parent of the (source) NKD.") |
| CAIS_CONTROLLED | AD_7 | (FALSE) |
| CAIS_DETERMINED | AD_8 | (FALSE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_55 | EMANATES_FROM | (NODE_KIND_DEFINITION) | NKD_25 |
| S | RD_103 | PARENT | () | N_2 |
| S | RD_110 | REVERSE_LINK | () | RD_90 |
| S | RD_112 | SPECIALIZATION_OF | (PARENT) | RD_93 |
| S | RD_117 | TERMINATES_AT | (DEFINITIONS) | NKD_8 |



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

### 20.5.92 'RELATION_DEFINITION( NODE_KIND_PARENT )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RD_92 | NODE_KIND_PARENT | NKD_30 | RELATION_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("Instances identify the parent VIEW node of the (source) NKI.") |
| CAIS_CONTROLLED | AD_7 | (TRUE) |
| CAIS_DETERMINED | AD_8 | (FALSE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_10 | ATTRIBUTE | (NODE_KIND) | AD_60 |
| A | AD_10 | CONSTRAINTS | (VALUE => STRUCTURAL) | |
| A | AD_14 | DEFAULT_VALUE | (STRUCTURAL) | |
| S | RD_55 | EMANATES_FROM | (NODE_KIND_-INTERPRETATION) | NKD_26 |
| S | RD_103 | PARENT | () | N_2 |
| S | RD_110 | REVERSE_LINK | () | RD_89 |
| S | RD_112 | SPECIALIZATION_OF | (PARENT) | RD_93 |
| S | RD_117 | TERMINATES_AT | (VIEW) | NKD_42 |

### 20.5.93 'RELATION_DEFINITION( PARENT )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RD_93 | PARENT | NKD_30 | RELATION_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ALLOWED_USAGE | AD_2 | (SECONDARY) |
| ANNOTATION | AD_3 | ("A generalization of relations whose instances identify the parent of the (source) node.") |
| CAIS_CONTROLLED | AD_7 | (FALSE) |
| CAIS_DETERMINED | AD_8 | (FALSE) |
| UNIQUE_KEYS | AD_84 | (TRUE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_10 | ATTRIBUTE | (INHERITABLE) | AD_39 |
| A | AD_10 | CONSTRAINTS | (VALUE => FALSE) | |
| A | AD_14 | DEFAULT_VALUE | (FALSE) | |
| S | RD_55 | EMANATES_FROM | (BASIC) | NKD_3 |
| S | RD_61 | GENERALIZATION_OF | (ATTRIBUTE_DEFINITION_-PARENT) | RD_13 |
| S | RD_61 | GENERALIZATION_OF | (ATTRIBUTE_KIND_-DEFINITION_PARENT) | RD_16 |
| S | RD_61 | GENERALIZATION_OF | (ATTRIBUTE_MONITOR_-PARENT) | RD_20 |
| S | RD_61 | GENERALIZATION_OF | (CHANNEL_PARENT) | RD_25 |
| S | RD_61 | GENERALIZATION_OF | (DEFINITIONS_PARENT) | RD_47 |
| S | RD_61 | GENERALIZATION_OF | (DEVICE_PARENT) | RD_51 |
| S | RD_61 | GENERALIZATION_OF | (GATEWAY_PARENT) | RD_57 |
| S | RD_61 | GENERALIZATION_OF | (GROUP_PARENT) | RD_62 |
| S | RD_61 | GENERALIZATION_OF | (INTRODUCED_NODE_PARENT) | RD_77 |
| S | RD_61 | GENERALIZATION_OF | (JOB_PARENT) | RD_80 |



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_61 | GENERALIZATION_OF | (MEDIUM_PARENT) | RD_84 |
| S | RD_61 | GENERALIZATION_OF | (NODE_KIND_DEFINITION_-PARENT) | RD_91 |
| S | RD_61 | GENERALIZATION_OF | (NODE_KIND_PARENT) | RD_92 |
| S | RD_61 | GENERALIZATION_OF | (PERMANENT_MEMBER_-PARENT) | RD_95 |
| S | RD_61 | GENERALIZATION_OF | (PROCESSOR_PARENT) | RD_98 |
| S | RD_61 | GENERALIZATION_OF | (RELATION_DEFINITION_-PARENT) | RD_103 |
| S | RD_61 | GENERALIZATION_OF | (RELATION_PARENT) | RD_105 |
| S | RD_61 | GENERALIZATION_OF | (RESOURCE_GROUP_PARENT) | RD_106 |
| S | RD_61 | GENERALIZATION_OF | (USER_PARENT) | RD_118 |
| S | RD_61 | GENERALIZATION_OF | (VIEW_PARENT) | RD_121 |
| S | RD_103 | PARENT | () | N_2 |
| S | RD_112 | SPECIALIZATION_OF | (BASIC) | RD_23 |
| S | RD_117 | TERMINATES_AT | (BASIC) | NKD_3 |

### 20.5.94 'RELATION_DEFINITION( PERMANENT_MEMBER )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RD_94 | PERMANENT_MEMBER | NKD_30 | RELATION_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ALLOWED_USAGE | AD_2 | (PRIMARY) |
| ANNOTATION | AD_3 | ("Instances are primary relationships between GROUP nodes.") |
| CAIS_CONTROLLED | AD_7 | (TRUE) |
| CAIS_DETERMINED | AD_8 | (FALSE) |
| UNIQUE_KEYS | AD_84 | (TRUE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_10 | ATTRIBUTE | (INHERITABLE) | AD_39 |
| A | AD_10 | CONSTRAINTS | (VALUE => FALSE) | |
| A | AD_14 | DEFAULT_VALUE | (FALSE) | |
| S | RD_10 | ATTRIBUTE | (NODE_KIND) | AD_60 |
| A | AD_10 | CONSTRAINTS | (VALUE => STRUCTURAL) | |
| A | AD_14 | DEFAULT_VALUE | (STRUCTURAL) | |
| S | RD_55 | EMANATES_FROM | (GROUP) | NKD_14 |
| S | RD_81 | KEY | (GENERIC_KEY) | AD_23 |
| S | RD_103 | PARENT | () | N_2 |
| S | RD_110 | REVERSE_LINK | () | RD_95 |
| S | RD_112 | SPECIALIZATION_OF | (BASIC) | RD_23 |
| S | RD_117 | TERMINATES_AT | (GROUP) | NKD_14 |

### 20.5.95 'RELATION_DEFINITION( PERMANENT_MEMBER_PARENT )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RD_95 | PERMANENT_MEMBER_PARENT | NKD_30 | RELATION_DEFINITION |



Source: https://assist.dla.mil -- Downloaded: 202610-26T07:54Z
Check the source to verify that this is the current version before use.

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("Instances are PARENT relationships between GROUP nodes.") |
| CAIS_CONTROLLED | AD_7 | (TRUE) |
| CAIS_DETERMINED | AD_8 | (FALSE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_10 | ATTRIBUTE | (NODE_KIND) | AD_60 |
| A | AD_10 | CONSTRAINTS | (VALUE => STRUCTURAL) | |
| A | AD_14 | DEFAULT_VALUE | (STRUCTURAL) | |
| S | RD_55 | EMANATES_FROM | (GROUP) | NKD_14 |
| S | RD_103 | PARENT | () | N_2 |
| S | RD_110 | REVERSE_LINK | () | RD_94 |
| S | RD_112 | SPECIALIZATION_OF | (PARENT) | RD_93 |
| S | RD_117 | TERMINATES_AT | (GROUP) | NKD_14 |

### 20.5.96 'RELATION_DEFINITION( PORT )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RD_96 | PORT | NKD_30 | RELATION_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ALLOWED_USAGE | AD_2 | (SECONDARY) |
| ANNOTATION | AD_3 | ("Instances identify the channels of the (source) file or process.") |
| CAIS_CONTROLLED | AD_7 | (TRUE) |
| CAIS_DETERMINED | AD_8 | (FALSE) |
| UNIQUE_KEYS | AD_84 | (TRUE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_10 | ATTRIBUTE | (ALLOWED_PROTOCOLS) | AD_1 |
| S | RD_10 | ATTRIBUTE | (DIRECTION) | AD_19 |
| S | RD_10 | ATTRIBUTE | (ELEMENT_SIZE) | AD_20 |
| S | RD_10 | ATTRIBUTE | (INCOMING_FRAME_LENGTH) | AD_38 |
| S | RD_10 | ATTRIBUTE | (INHERITABLE) | AD_39 |
| A | AD_10 | CONSTRAINTS | (VALUE => FALSE) | |
| A | AD_14 | DEFAULT_VALUE | (FALSE) | |
| S | RD_10 | ATTRIBUTE | (MAXIMUM_LINE_LENGTH) | AD_52 |
| S | RD_10 | ATTRIBUTE | (MAXIMUM_PAGE_LENGTH) | AD_53 |
| S | RD_10 | ATTRIBUTE | (OUTGOING_FRAME_LENGTH_-RANGE) | AD_65 |
| S | RD_10 | ATTRIBUTE | (PRIORITY_LEVEL) | AD_70 |
| S | RD_10 | ATTRIBUTE | (QUEUE_LENGTH) | AD_72 |
| S | RD_10 | ATTRIBUTE | (NODE_KIND) | AD_60 |
| A | AD_10 | CONSTRAINTS | (VALUE => STRUCTURAL) | |
| A | AD_14 | DEFAULT_VALUE | (STRUCTURAL) | |
| S | RD_55 | EMANATES_FROM | (CLIENT) | NKD_5 |
| S | RD_81 | KEY | (NAME_KEY) | AD_58 |
| S | RD_103 | PARENT | () | N_2 |
| S | RD_110 | REVERSE_LINK | () | RD_26 |
| S | RD_112 | SPECIALIZATION_OF | (BASIC) | RD_23 |
| S | RD_117 | TERMINATES_AT | (CHANNEL) | NKD_4 |



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

### 20.5.97 'RELATION_DEFINITION( POTENTIAL_MEMBER )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RD_97 | POTENTIAL_MEMBER | NKD_30 | RELATION_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("Instances identify groups which members of the (source) group may adopt.") |
| CAIS_CONTROLLED | AD_7 | (TRUE) |
| CAIS_DETERMINED | AD_8 | (FALSE) |
| UNIQUE_KEYS | AD_84 | (TRUE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_10 | ATTRIBUTE | (INHERITABLE) | AD_39 |
| A | AD_14 | DEFAULT_VALUE | (TRUE) | |
| S | RD_10 | ATTRIBUTE | (NODE_KIND) | AD_60 |
| A | AD_10 | CONSTRAINTS | (VALUE => STRUCTURAL) | |
| A | AD_14 | DEFAULT_VALUE | (STRUCTURAL) | |
| S | RD_55 | EMANATES_FROM | (GROUP) | NKD_14 |
| S | RD_81 | KEY | (GENERIC_KEY) | AD_23 |
| S | RD_103 | PARENT | () | N_2 |
| S | RD_112 | SPECIALIZATION_OF | (BASIC) | RD_23 |
| S | RD_117 | TERMINATES_AT | (GROUP) | NKD_14 |

### 20.5.98 'RELATION_DEFINITION( PROCESSOR_PARENT )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RD_98 | PROCESSOR_PARENT | NKD_30 | RELATION_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("Instances identify the parent (system node) of the (source) top-level PROCESSOR node.") |
| CAIS_CONTROLLED | AD_7 | (TRUE) |
| CAIS_DETERMINED | AD_8 | (FALSE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_10 | ATTRIBUTE | (NODE_KIND) | AD_60 |
| A | AD_10 | CONSTRAINTS | (VALUE => STRUCTURAL) | |
| A | AD_14 | DEFAULT_VALUE | (STRUCTURAL) | |
| S | RD_55 | EMANATES_FROM | (PROCESSOR) | NKD_29 |
| S | RD_103 | PARENT | () | N_2 |
| S | RD_110 | REVERSE_LINK | () | RD_99 |
| S | RD_112 | SPECIALIZATION_OF | (PARENT) | RD_93 |
| S | RD_117 | TERMINATES_AT | (SYSTEM_LEVEL) | NKD_37 |

### 20.5.99 'RELATION_DEFINITION( PROCESSOR_PRIMARY )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RD_99 | PROCESSOR_PRIMARY | NKD_30 | RELATION_DEFINITION |



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ALLOWED_USAGE | AD_2 | (PRIMARY) |
| ANNOTATION | AD_3 | ("Instances are primary relationships to the top-level PROCESSOR nodes.") |
| CAIS_CONTROLLED | AD_7 | (TRUE) |
| CAIS_DETERMINED | AD_8 | (FALSE) |
| UNIQUE_KEYS | AD_84 | (TRUE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_10 | ATTRIBUTE | (INHERITABLE) | AD_39 |
| A | AD_10 | CONSTRAINTS | (VALUE => FALSE) | |
| A | AD_14 | DEFAULT_VALUE | (FALSE) | |
| S | RD_10 | ATTRIBUTE | (NODE_KIND) | AD_60 |
| A | AD_10 | CONSTRAINTS | (VALUE => STRUCTURAL) | |
| A | AD_14 | DEFAULT_VALUE | (STRUCTURAL) | |
| S | RD_55 | EMANATES_FROM | (SYSTEM_LEVEL) | NKD_37 |
| S | RD_81 | KEY | (GENERIC_KEY) | AD_23 |
| S | RD_103 | PARENT | () | N_2 |
| S | RD_110 | REVERSE_LINK | () | RD_98 |
| S | RD_112 | SPECIALIZATION_OF | (BASIC) | RD_23 |
| S | RD_117 | TERMINATES_AT | (PROCESSOR) | NKD_29 |

### 20.5.100 'RELATION_DEFINITION( PROCESSOR_SECONDARY )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RD_100 | PROCESSOR_SECONDARY | NKD_30 | RELATION_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("Instances provide identification of top-level PROCESSOR nodes from the (source) RESOURCE_GROUP node.") |
| CAIS_CONTROLLED | AD_7 | (TRUE) |
| CAIS_DETERMINED | AD_8 | (FALSE) |
| UNIQUE_KEYS | AD_84 | (TRUE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_10 | ATTRIBUTE | (INHERITABLE) | AD_39 |
| A | AD_14 | DEFAULT_VALUE | (TRUE) | |
| S | RD_10 | ATTRIBUTE | (NODE_KIND) | AD_60 |
| A | AD_10 | CONSTRAINTS | (VALUE => STRUCTURAL) | |
| A | AD_14 | DEFAULT_VALUE | (STRUCTURAL) | |
| S | RD_55 | EMANATES_FROM | (RESOURCE_GROUP) | NKD_33 |
| S | RD_81 | KEY | (GENERIC_KEY) | AD_23 |
| S | RD_103 | PARENT | () | N_2 |
| S | RD_112 | SPECIALIZATION_OF | (BASIC) | RD_23 |
| S | RD_117 | TERMINATES_AT | (PROCESSOR) | NKD_29 |

### 20.5.101 'RELATION_DEFINITION( RELATION )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RD_101 | RELATION | NKD_30 | RELATION_DEFINITION |



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ALLOWED_USAGE | AD_2 | (PRIMARY) |
| ANNOTATION | AD_3 | ("Instances are primary relationships providing view-relative names for the (target) RI.") |
| CAIS_CONTROLLED | AD_7 | (TRUE) |
| CAIS_DETERMINED | AD_8 | (FALSE) |
| UNIQUE_KEYS | AD_84 | (TRUE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_10 | ATTRIBUTE | (INHERITABLE) | AD_39 |
| A | AD_10 | CONSTRAINTS | (VALUE => FALSE) | |
| A | AD_14 | DEFAULT_VALUE | (FALSE) | |
| S | RD_10 | ATTRIBUTE | (NODE_KIND) | AD_60 |
| A | AD_10 | CONSTRAINTS | (VALUE => STRUCTURAL) | |
| A | AD_14 | DEFAULT_VALUE | (STRUCTURAL) | |
| S | RD_55 | EMANATES_FROM | (VIEW) | NKD_42 |
| S | RD_81 | KEY | (NAME_KEY) | AD_58 |
| S | RD_103 | PARENT | () | N_2 |
| S | RD_110 | REVERSE_LINK | () | RD_105 |
| S | RD_112 | SPECIALIZATION_OF | (BASIC) | RD_23 |
| S | RD_117 | TERMINATES_AT | (RELATION_INTERPRETATION) | NKD_31 |

### 20.5.102 'RELATION_DEFINITION( RELATION_DEFINITION )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RD_102 | RELATION_DEFINITION | NKD_30 | RELATION_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ALLOWED_USAGE | AD_2 | (PRIMARY) |
| ANNOTATION | AD_3 | ("Instances are primary relationships to RDs.") |
| CAIS_CONTROLLED | AD_7 | (FALSE) |
| CAIS_DETERMINED | AD_8 | (FALSE) |
| UNIQUE_KEYS | AD_84 | (TRUE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_10 | ATTRIBUTE | (INHERITABLE) | AD_39 |
| A | AD_10 | CONSTRAINTS | (VALUE => FALSE) | |
| A | AD_14 | DEFAULT_VALUE | (FALSE) | |
| S | RD_10 | ATTRIBUTE | (NODE_KIND) | AD_60 |
| A | AD_10 | CONSTRAINTS | (VALUE => STRUCTURAL) | |
| A | AD_14 | DEFAULT_VALUE | (STRUCTURAL) | |
| S | RD_55 | EMANATES_FROM | (DEFINITIONS) | NKD_8 |
| S | RD_81 | KEY | (GENERIC_KEY) | AD_23 |
| S | RD_103 | PARENT | () | N_2 |
| S | RD_110 | REVERSE_LINK | () | RD_103 |
| S | RD_112 | SPECIALIZATION_OF | (BASIC) | RD_23 |
| S | RD_117 | TERMINATES_AT | (RELATION_DEFINITION) | NKD_30 |



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

### 20.5.103 'RELATION_DEFINITION( RELATION_DEFINITION_PARENT )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RD_103 | RELATION_DEFINITION_-PARENT | NKD_30 | RELATION_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ALLOWED_USAGE | AD_2 | (SECONDARY) |
| ANNOTATION | AD_3 | ("Instances identify the parent of the (source) RD.") |
| CAIS_CONTROLLED | AD_7 | (FALSE) |
| CAIS_DETERMINED | AD_8 | (FALSE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_55 | EMANATES_FROM | (RELATION_DEFINITION) | NKD_30 |
| S | RD_103 | PARENT | () | N_2 |
| S | RD_110 | REVERSE_LINK | () | RD_102 |
| S | RD_112 | SPECIALIZATION_OF | (PARENT) | RD_93 |
| S | RD_117 | TERMINATES_AT | (DEFINITIONS) | NKD_8 |

### 20.5.104 'RELATION_DEFINITION( RELATION_NAME )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RD_104 | RELATION_NAME | NKD_30 | RELATION_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("Instances specify a (source) NKI-relative name for the relation.") |
| CAIS_CONTROLLED | AD_7 | (TRUE) |
| CAIS_DETERMINED | AD_8 | (FALSE) |
| UNIQUE_KEYS | AD_84 | (TRUE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_10 | ATTRIBUTE | (INHERITABLE) | AD_39 |
| A | AD_14 | DEFAULT_VALUE | (TRUE) | |
| S | RD_10 | ATTRIBUTE | (NODE_KIND) | AD_60 |
| A | AD_10 | CONSTRAINTS | (VALUE => STRUCTURAL) | |
| A | AD_14 | DEFAULT_VALUE | (STRUCTURAL) | |
| S | RD_55 | EMANATES_FROM | (NODE_KIND_-INTERPRETATION) | NKD_26 |
| S | RD_81 | KEY | (NAME_KEY) | AD_58 |
| S | RD_103 | PARENT | () | N_2 |
| S | RD_112 | SPECIALIZATION_OF | (BASIC) | RD_23 |
| S | RD_117 | TERMINATES_AT | (RELATION_INTERPRETATION) | NKD_31 |

### 20.5.105 'RELATION_DEFINITION( RELATION_PARENT )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RD_105 | RELATION_PARENT | NKD_30 | RELATION_DEFINITION |



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ALLOWED_USAGE | AD_2 | (SECONDARY) |
| ANNOTATION | AD_3 | ("Instances identify the parent VIEW node of the (source) RI.") |
| CAIS_CONTROLLED | AD_7 | (TRUE) |
| CAIS_DETERMINED | AD_8 | (FALSE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_10 | ATTRIBUTE | (NODE_KIND) | AD_60 |
| A | AD_10 | CONSTRAINTS | (VALUE => STRUCTURAL) | |
| A | AD_14 | DEFAULT_VALUE | (STRUCTURAL) | |
| S | RD_55 | EMANATES_FROM | (RELATION_INTERPRETATION) | NKD_31 |
| S | RD_103 | PARENT | () | N_2 |
| S | RD_110 | REVERSE_LINK | () | RD_101 |
| S | RD_112 | SPECIALIZATION_OF | (PARENT) | RD_93 |
| S | RD_117 | TERMINATES_AT | (VIEW) | NKD_42 |

### 20.5.106 'RELATION_DEFINITION( RESOURCE_GROUP_PARENT )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RD_106 | RESOURCE_GROUP_PARENT | NKD_30 | RELATION_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("Instances identify the parent (system node) of the (source) top-level RESOURCE_GROUP node.") |
| CAIS_CONTROLLED | AD_7 | (TRUE) |
| CAIS_DETERMINED | AD_8 | (FALSE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_10 | ATTRIBUTE | (NODE_KIND) | AD_60 |
| A | AD_10 | CONSTRAINTS | (VALUE => STRUCTURAL) | |
| A | AD_14 | DEFAULT_VALUE | (STRUCTURAL) | |
| S | RD_55 | EMANATES_FROM | (RESOURCE_GROUP) | NKD_33 |
| S | RD_103 | PARENT | () | N_2 |
| S | RD_110 | REVERSE_LINK | () | RD_107 |
| S | RD_112 | SPECIALIZATION_OF | (PARENT) | RD_93 |
| S | RD_117 | TERMINATES_AT | (SYSTEM_LEVEL) | NKD_37 |

### 20.5.107 'RELATION_DEFINITION( RESOURCE_GROUP_PRIMARY )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RD_107 | RESOURCE_GROUP_PRIMARY | NKD_30 | RELATION_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ALLOWED_USAGE | AD_2 | (PRIMARY) |
| ANNOTATION | AD_3 | ("Instances are primary relationships to the top-level RESOURCE_GROUP nodes.") |
| CAIS_CONTROLLED | AD_7 | (TRUE) |
| CAIS_DETERMINED | AD_8 | (FALSE) |
| UNIQUE_KEYS | AD_84 | (TRUE) |



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_10 | ATTRIBUTE | (INHERITABLE) | AD_39 |
| A | AD_10 | CONSTRAINTS | (VALUE => FALSE) | |
| A | AD_14 | DEFAULT_VALUE | (FALSE) | |
| S | RD_10 | ATTRIBUTE | (NODE_KIND) | AD_60 |
| A | AD_10 | CONSTRAINTS | (VALUE => STRUCTURAL) | |
| A | AD_14 | DEFAULT_VALUE | (STRUCTURAL) | |
| S | RD_55 | EMANATES_FROM | (SYSTEM_LEVEL) | NKD_37 |
| S | RD_81 | KEY | (GENERIC_KEY) | AD_23 |
| S | RD_103 | PARENT | () | N_2 |
| S | RD_110 | REVERSE_LINK | () | RD_106 |
| S | RD_112 | SPECIALIZATION_OF | (BASIC) | RD_23 |
| S | RD_117 | TERMINATES_AT | (RESOURCE_GROUP) | NKD_33 |

### 20.5.108 'RELATION_DEFINITION( RESOURCE_GROUP_SECONDARY )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RD_108 | RESOURCE_GROUP_-SECONDARY | NKD_30 | RELATION_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("Instances provide identification of top-level RESOURCE_-GROUP nodes from the (source) current process node.") |
| CAIS_CONTROLLED | AD_7 | (TRUE) |
| CAIS_DETERMINED | AD_8 | (FALSE) |
| UNIQUE_KEYS | AD_84 | (TRUE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_10 | ATTRIBUTE | (INHERITABLE) | AD_39 |
| A | AD_14 | DEFAULT_VALUE | (TRUE) | |
| S | RD_10 | ATTRIBUTE | (NODE_KIND) | AD_60 |
| A | AD_10 | CONSTRAINTS | (VALUE => STRUCTURAL) | |
| A | AD_14 | DEFAULT_VALUE | (STRUCTURAL) | |
| S | RD_55 | EMANATES_FROM | (PROCESS) | NKD_28 |
| S | RD_81 | KEY | (GENERIC_KEY) | AD_23 |
| S | RD_103 | PARENT | () | N_2 |
| S | RD_112 | SPECIALIZATION_OF | (BASIC) | RD_23 |
| S | RD_117 | TERMINATES_AT | (RESOURCE_GROUP) | NKD_33 |

### 20.5.109 'RELATION_DEFINITION( REVERSE_LINK )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RD_109 | REVERSE_LINK | NKD_30 | RELATION_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("Instances identify the reverse RD for the (source and target) RD when a bidirectional RD is comprised of a single relation.") |
| CAIS_CONTROLLED | AD_7 | (TRUE) |
| CAIS_DETERMINED | AD_8 | (FALSE) |
| UNIQUE_KEYS | AD_84 | (TRUE) |



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_10 | ATTRIBUTE | (INHERITABLE) | AD_39 |
| A | AD_14 | DEFAULT_VALUE | (TRUE) | |
| S | RD_10 | ATTRIBUTE | (NODE_KIND) | AD_60 |
| A | AD_10 | CONSTRAINTS | (VALUE => STRUCTURAL) | |
| A | AD_14 | DEFAULT_VALUE | (STRUCTURAL) | |
| S | RD_55 | EMANATES_FROM | (RELATION_DEFINITION) | NKD_30 |
| S | RD_61 | GENERALIZATION_OF | (REVERSE_LINK_-BIDIRECTIONAL) | RD_110 |
| S | RD_103 | PARENT | () | N_2 |
| S | RD_112 | SPECIALIZATION_OF | (BASIC) | RD_23 |
| S | RD_117 | TERMINATES_AT | (RELATION_DEFINITION) | NKD_30 |

### 20.5.110 'RELATION_DEFINITION( REVERSE_LINK_BIDIRECTIONAL )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RD_110 | REVERSE_LINK_BIDIRECTIONAL | NKD_30 | RELATION_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ALLOWED_USAGE | AD_2 | (SECONDARY) |
| ANNOTATION | AD_3 | ("Instances identify the reverse RD for the (source) RD when a bidirectional RD is comprised of two distinct relations.") |
| CAIS_CONTROLLED | AD_7 | (TRUE) |
| CAIS_DETERMINED | AD_8 | (FALSE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_10 | ATTRIBUTE | (INHERITABLE) | AD_39 |
| A | AD_10 | CONSTRAINTS | (VALUE => FALSE) | |
| A | AD_14 | DEFAULT_VALUE | (FALSE) | |
| S | RD_10 | ATTRIBUTE | (NODE_KIND) | AD_60 |
| A | AD_10 | CONSTRAINTS | (VALUE => STRUCTURAL) | |
| A | AD_14 | DEFAULT_VALUE | (STRUCTURAL) | |
| S | RD_55 | EMANATES_FROM | (RELATION_DEFINITION) | NKD_30 |
| S | RD_103 | PARENT | () | N_2 |
| S | RD_109 | REVERSE_LINK | () | RD_110 |
| S | RD_112 | SPECIALIZATION_OF | (REVERSE_LINK) | RD_109 |
| S | RD_117 | TERMINATES_AT | (RELATION_DEFINITION) | NKD_30 |

### 20.5.111 'RELATION_DEFINITION( SPECIALIZATION_OF_NKD )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RD_111 | SPECIALIZATION_OF_NKD | NKD_30 | RELATION_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ALLOWED_USAGE | AD_2 | (SECONDARY) |
| ANNOTATION | AD_3 | ("Instances identify the specialization NKDs of the (source) NKD.") |
| CAIS_CONTROLLED | AD_7 | (TRUE) |
| CAIS_DETERMINED | AD_8 | (FALSE) |
| UNIQUE_KEYS | AD_84 | (TRUE) |


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_10 | ATTRIBUTE | (INHERITABLE) | AD_39 |
| A | AD_10 | CONSTRAINTS | (VALUE => FALSE) | |
| A | AD_14 | DEFAULT_VALUE | (FALSE) | |
| S | RD_10 | ATTRIBUTE | (NODE_KIND) | AD_60 |
| A | AD_10 | CONSTRAINTS | (VALUE => STRUCTURAL) | |
| A | AD_14 | DEFAULT_VALUE | (STRUCTURAL) | |
| S | RD_55 | EMANATES_FROM | (NODE_KIND_DEFINITION) | NKD_25 |
| S | RD_81 | KEY | (GENERIC_KEY) | AD_23 |
| S | RD_103 | PARENT | () | N_2 |
| S | RD_110 | REVERSE_LINK | () | RD_60 |
| S | RD_112 | SPECIALIZATION_OF | (BASIC) | RD_23 |
| S | RD_117 | TERMINATES_AT | (NODE_KIND_DEFINITION) | NKD_25 |

### 20.5.112 'RELATION_DEFINITION( SPECIALIZATION_OF_RD )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RD_112 | SPECIALIZATION_OF_RD | NKD_30 | RELATION_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ALLOWED_USAGE | AD_2 | (SECONDARY) |
| ANNOTATION | AD_3 | ("Instances identify the specialization RDs of the (source) RD.") |
| CAIS_CONTROLLED | AD_7 | (TRUE) |
| CAIS_DETERMINED | AD_8 | (FALSE) |
| UNIQUE_KEYS | AD_84 | (TRUE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_10 | ATTRIBUTE | (INHERITABLE) | AD_39 |
| A | AD_10 | CONSTRAINTS | (VALUE => FALSE) | |
| A | AD_14 | DEFAULT_VALUE | (FALSE) | |
| S | RD_10 | ATTRIBUTE | (NODE_KIND) | AD_60 |
| A | AD_10 | CONSTRAINTS | (VALUE => STRUCTURAL) | |
| A | AD_14 | DEFAULT_VALUE | (STRUCTURAL) | |
| S | RD_55 | EMANATES_FROM | (RELATION_DEFINITION) | NKD_30 |
| S | RD_81 | KEY | (GENERIC_KEY) | AD_23 |
| S | RD_103 | PARENT | () | N_2 |
| S | RD_110 | REVERSE_LINK | () | RD_61 |
| S | RD_112 | SPECIALIZATION_OF | (BASIC) | RD_23 |
| S | RD_117 | TERMINATES_AT | (RELATION_DEFINITION) | NKD_30 |

### 20.5.113 'RELATION_DEFINITION( STANDARD_ERROR )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RD_113 | STANDARD_ERROR | NKD_30 | RELATION_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("Instances identify the intended standard error file for the (source) process.") |
| CAIS_CONTROLLED | AD_7 | (TRUE) |
| CAIS_DETERMINED | AD_8 | (FALSE) |



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

| Attribute Name | AD | Attribute Value |
|---|---|---|
| UNIQUE_KEYS | AD_84 | (TRUE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_10 | ATTRIBUTE | (INHERITABLE) | AD_39 |
| A | AD_10 | CONSTRAINTS | (VALUE => FALSE) | |
| A | AD_14 | DEFAULT_VALUE | (FALSE) | |
| S | RD_10 | ATTRIBUTE | (NODE_KIND) | AD_60 |
| A | AD_10 | CONSTRAINTS | (VALUE => FILE) | |
| A | AD_14 | DEFAULT_VALUE | (FILE) | |
| S | RD_55 | EMANATES_FROM | (PROCESS) | NKD_28 |
| S | RD_103 | PARENT | () | N_2 |
| S | RD_112 | SPECIALIZATION_OF | (BASIC) | RD_23 |
| S | RD_117 | TERMINATES_AT | (FILE) | NKD_11 |

### 20.5.114 'RELATION_DEFINITION( STANDARD_INPUT )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RD_114 | STANDARD_INPUT | NKD_30 | RELATION_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("Instances identify the intended standard input file for the (source) process.") |
| CAIS_CONTROLLED | AD_7 | (TRUE) |
| CAIS_DETERMINED | AD_8 | (FALSE) |
| UNIQUE_KEYS | AD_84 | (TRUE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_10 | ATTRIBUTE | (INHERITABLE) | AD_39 |
| A | AD_10 | CONSTRAINTS | (VALUE => FALSE) | |
| A | AD_14 | DEFAULT_VALUE | (FALSE) | |
| S | RD_10 | ATTRIBUTE | (NODE_KIND) | AD_60 |
| A | AD_10 | CONSTRAINTS | (VALUE => FILE) | |
| A | AD_14 | DEFAULT_VALUE | (FILE) | |
| S | RD_55 | EMANATES_FROM | (PROCESS) | NKD_28 |
| S | RD_103 | PARENT | () | N_2 |
| S | RD_112 | SPECIALIZATION_OF | (BASIC) | RD_23 |
| S | RD_117 | TERMINATES_AT | (FILE) | NKD_11 |

### 20.5.115 'RELATION_DEFINITION( STANDARD_OUTPUT )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RD_115 | STANDARD_OUTPUT | NKD_30 | RELATION_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("Instances identify the intended standard output file for the (source) process.") |
| CAIS_CONTROLLED | AD_7 | (TRUE) |
| CAIS_DETERMINED | AD_8 | (FALSE) |
| UNIQUE_KEYS | AD_84 | (TRUE) |


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_10 | ATTRIBUTE | (INHERITABLE) | AD_39 |
| A | AD_10 | CONSTRAINTS | (VALUE => FALSE) | |
| A | AD_14 | DEFAULT_VALUE | (FALSE) | |
| S | RD_10 | ATTRIBUTE | (NODE_KIND) | AD_60 |
| A | AD_10 | CONSTRAINTS | (VALUE => FILE) | |
| A | AD_14 | DEFAULT_VALUE | (FILE) | |
| S | RD_55 | EMANATES_FROM | (PROCESS) | NKD_28 |
| S | RD_103 | PARENT | () | N_2 |
| S | RD_112 | SPECIALIZATION_OF | (BASIC) | RD_23 |
| S | RD_117 | TERMINATES_AT | (FILE) | NKD_11 |

### 20.5.116 'RELATION_DEFINITION( TERMINATES )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RD_116 | TERMINATES | NKD_30 | RELATION_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ALLOWED_USAGE | AD_2 | (SECONDARY) |
| ANNOTATION | AD_3 | ("Instances identify the terminating relations for the (source) node kind.") |
| CAIS_CONTROLLED | AD_7 | (TRUE) |
| CAIS_DETERMINED | AD_8 | (FALSE) |
| UNIQUE_KEYS | AD_84 | (TRUE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_10 | ATTRIBUTE | (INHERITABLE) | AD_39 |
| A | AD_10 | CONSTRAINTS | (VALUE => FALSE) | |
| A | AD_14 | DEFAULT_VALUE | (FALSE) | |
| S | RD_10 | ATTRIBUTE | (NODE_KIND) | AD_60 |
| A | AD_10 | CONSTRAINTS | (VALUE => STRUCTURAL) | |
| A | AD_14 | DEFAULT_VALUE | (STRUCTURAL) | |
| S | RD_55 | EMANATES_FROM | (NODE_KIND_DEFINITION) | NKD_25 |
| S | RD_81 | KEY | (GENERIC_KEY) | AD_23 |
| S | RD_103 | PARENT | () | N_2 |
| S | RD_110 | REVERSE_LINK | () | RD_117 |
| S | RD_112 | SPECIALIZATION_OF | (BASIC) | RD_23 |
| S | RD_117 | TERMINATES_AT | (RELATION_DEFINITION) | NKD_30 |

### 20.5.117 'RELATION_DEFINITION( TERMINATES_AT )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RD_117 | TERMINATES_AT | NKD_30 | RELATION_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ALLOWED_USAGE | AD_2 | (SECONDARY) |
| ANNOTATION | AD_3 | ("Instances identify the node kinds for the (source) terminating relation.") |
| CAIS_CONTROLLED | AD_7 | (TRUE) |
| CAIS_DETERMINED | AD_8 | (FALSE) |
| UNIQUE_KEYS | AD_84 | (TRUE) |



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_10 | ATTRIBUTE | (INHERITABLE) | AD_39 |
| A | AD_10 | CONSTRAINTS | (VALUE => FALSE) | |
| A | AD_14 | DEFAULT_VALUE | (FALSE) | |
| S | RD_10 | ATTRIBUTE | (NODE_KIND) | AD_60 |
| A | AD_10 | CONSTRAINTS | (VALUE => STRUCTURAL) | |
| A | AD_14 | DEFAULT_VALUE | (STRUCTURAL) | |
| S | RD_55 | EMANATES_FROM | (RELATION_DEFINITION) | NKD_30 |
| S | RD_81 | KEY | (GENERIC_KEY) | AD_23 |
| S | RD_103 | PARENT | () | N_2 |
| S | RD_110 | REVERSE_LINK | () | RD_116 |
| S | RD_112 | SPECIALIZATION_OF | (BASIC) | RD_23 |
| S | RD_117 | TERMINATES_AT | (NODE_KIND_DEFINITION) | NKD_25 |

### 20.5.118 'RELATION_DEFINITION( USER_PARENT )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RD_118 | USER_PARENT | NKD_30 | RELATION_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("Instances identify the parent (system node) of the (source) top-level USER node.") |
| CAIS_CONTROLLED | AD_7 | (TRUE) |
| CAIS_DETERMINED | AD_8 | (FALSE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_10 | ATTRIBUTE | (NODE_KIND) | AD_60 |
| A | AD_10 | CONSTRAINTS | (VALUE => STRUCTURAL) | |
| A | AD_14 | DEFAULT_VALUE | (STRUCTURAL) | |
| S | RD_55 | EMANATES_FROM | (USER) | NKD_40 |
| S | RD_103 | PARENT | () | N_2 |
| S | RD_110 | REVERSE_LINK | () | RD_119 |
| S | RD_112 | SPECIALIZATION_OF | (PARENT) | RD_93 |
| S | RD_117 | TERMINATES_AT | (SYSTEM_LEVEL) | NKD_37 |

### 20.5.119 'RELATION_DEFINITION( USER_PRIMARY )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RD_119 | USER_PRIMARY | NKD_30 | RELATION_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ALLOWED_USAGE | AD_2 | (PRIMARY) |
| ANNOTATION | AD_3 | ("Instances are primary relationships to the top-level USER nodes.") |
| CAIS_CONTROLLED | AD_7 | (TRUE) |
| CAIS_DETERMINED | AD_8 | (FALSE) |
| UNIQUE_KEYS | AD_84 | (TRUE) |


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_10 | ATTRIBUTE | (INHERITABLE) | AD_39 |
| A | AD_10 | CONSTRAINTS | (VALUE => FALSE) | |
| A | AD_14 | DEFAULT_VALUE | (FALSE) | |
| S | RD_10 | ATTRIBUTE | (NODE_KIND) | AD_60 |
| A | AD_10 | CONSTRAINTS | (VALUE => STRUCTURAL) | |
| A | AD_14 | DEFAULT_VALUE | (STRUCTURAL) | |
| S | RD_55 | EMANATES_FROM | (SYSTEM_LEVEL) | NKD_37 |
| S | RD_81 | KEY | (GENERIC_KEY) | AD_23 |
| S | RD_103 | PARENT | () | N_2 |
| S | RD_110 | REVERSE_LINK | () | RD_118 |
| S | RD_112 | SPECIALIZATION_OF | (BASIC) | RD_23 |
| S | RD_117 | TERMINATES_AT | (USER) | NKD_40 |

### 20.5.120 'RELATION_DEFINITION( USER_SECONDARY )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RD_120 | USER_SECONDARY | NKD_30 | RELATION_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("Instances provide identification of top-level USER nodes from the (source) current process node.") |
| CAIS_CONTROLLED | AD_7 | (TRUE) |
| CAIS_DETERMINED | AD_8 | (FALSE) |
| UNIQUE_KEYS | AD_84 | (TRUE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_10 | ATTRIBUTE | (INHERITABLE) | AD_39 |
| A | AD_14 | DEFAULT_VALUE | (TRUE) | |
| S | RD_10 | ATTRIBUTE | (NODE_KIND) | AD_60 |
| A | AD_10 | CONSTRAINTS | (VALUE => STRUCTURAL) | |
| A | AD_14 | DEFAULT_VALUE | (STRUCTURAL) | |
| S | RD_55 | EMANATES_FROM | (PROCESS) | NKD_28 |
| S | RD_81 | KEY | (GENERIC_KEY) | AD_23 |
| S | RD_103 | PARENT | () | N_2 |
| S | RD_112 | SPECIALIZATION_OF | (BASIC) | RD_23 |
| S | RD_117 | TERMINATES_AT | (USER) | NKD_40 |

### 20.5.121 'RELATION_DEFINITION( VIEW_PARENT )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RD_121 | VIEW_PARENT | NKD_30 | RELATION_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("Instances identify the parent of the (source) VIEW node.") |
| CAIS_CONTROLLED | AD_7 | (FALSE) |
| CAIS_DETERMINED | AD_8 | (FALSE) |



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_10 | ATTRIBUTE | (NODE_KIND) | AD_60 |
| A | AD_14 | DEFAULT_VALUE | (STRUCTURAL) | |
| S | RD_55 | EMANATES_FROM | (VIEW) | NKD_42 |
| S | RD_103 | PARENT | () | N_2 |
| S | RD_110 | REVERSE_LINK | () | RD_122 |
| S | RD_112 | SPECIALIZATION_OF | (PARENT) | RD_93 |
| S | RD_117 | TERMINATES_AT | (BASIC) | NKD_3 |

### 20.5.122 'RELATION_DEFINITION( VIEW_PRIMARY )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RD_122 | VIEW_PRIMARY | NKD_30 | RELATION_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ALLOWED_USAGE | AD_2 | (PRIMARY) |
| ANNOTATION | AD_3 | ("Instances are primary relationships to VIEW nodes.") |
| CAIS_CONTROLLED | AD_7 | (FALSE) |
| CAIS_DETERMINED | AD_8 | (FALSE) |
| UNIQUE_KEYS | AD_84 | (TRUE) |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_10 | ATTRIBUTE | (INHERITABLE) | AD_39 |
| A | AD_10 | CONSTRAINTS | (VALUE => FALSE) | |
| A | AD_14 | DEFAULT_VALUE | (FALSE) | |
| S | RD_10 | ATTRIBUTE | (NODE_KIND) | AD_60 |
| A | AD_10 | CONSTRAINTS | (VALUE => STRUCTURAL) | |
| A | AD_14 | DEFAULT_VALUE | (STRUCTURAL) | |
| S | RD_55 | EMANATES_FROM | (BASIC) | NKD_3 |
| S | RD_81 | KEY | (GENERIC_KEY) | AD_23 |
| S | RD_103 | PARENT | () | N_2 |
| S | RD_110 | REVERSE_LINK | () | RD_121 |
| S | RD_112 | SPECIALIZATION_OF | (BASIC) | RD_23 |
| S | RD_117 | TERMINATES_AT | (VIEW) | NKD_42 |

### 20.5.123 'RELATION_DEFINITION( VIEW_SECONDARY )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RD_123 | VIEW_SECONDARY | NKD_30 | RELATION_DEFINITION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("Instances identify views.") |
| CAIS_CONTROLLED | AD_7 | (TRUE) |
| CAIS_DETERMINED | AD_8 | (FALSE) |
| UNIQUE_KEYS | AD_84 | (TRUE) |


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_10 | ATTRIBUTE | (INHERITABLE) | AD_39 |
| A | AD_14 | DEFAULT_VALUE | (TRUE) | |
| S | RD_10 | ATTRIBUTE | (NODE_KIND) | AD_60 |
| A | AD_10 | CONSTRAINTS | (VALUE => STRUCTURAL) | |
| A | AD_14 | DEFAULT_VALUE | (STRUCTURAL) | |
| S | RD_55 | EMANATES_FROM | (EXECUTABLE_IMAGE) | NKD_10 |
| S | RD_55 | EMANATES_FROM | (PROCESS) | NKD_28 |
| S | RD_81 | KEY | (GENERIC_KEY) | AD_23 |
| S | RD_103 | PARENT | () | N_2 |
| S | RD_112 | SPECIALIZATION_OF | (BASIC) | RD_23 |
| S | RD_117 | TERMINATES_AT | (VIEW) | NKD_42 |



Source: https://assist.dla.mil -- Downloaded: 202610-26T07:54Z
Check the source to verify that this is the current version before use.

## 20.6 Node kind interpretation nodes (NKIs)

The following table describes the node attributes which have the same value on all NKIs:

| Attribute Name | AD | Attribute Value |
|---|---|---|
| KIND | AD_45 | (STRUCTURAL) |
| STRUCTURAL_KIND | AD_76 | (INTERPRETATION) |
| INTERPRETATION_KIND | AD_40 | (NODE_KIND) |

The following subsections describe each predefined NKI. Each section name is the primary path element from 'VIEW( PREDEFINED ) to the NKI described.

### 20.6.1 'NODE_KIND( ATTRIBUTE_DEFINITION )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| NKI_1 | ATTRIBUTE_DEFINITION | NKD_26 | NODE_KIND_INTERPRETATION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("NKI corresponding to the NKD for ADs.") |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_21 | ATTRIBUTE_NAME | (DELETABLE) | AD_16 |
| S | RD_21 | ATTRIBUTE_NAME | (MODIFIABLE) | AD_57 |
| S | RD_46 | DEFINITION | () | NKD_1 |
| S | RD_92 | PARENT | () | N_6 |
| S | RD_104 | RELATION_NAME | (ATTRIBUTE_KIND) | RI_14 |
| S | RD_104 | RELATION_NAME | (ATTRIBUTE_OF) | RI_22 |
| S | RD_104 | RELATION_NAME | (KEY_OF) | RI_83 |
| S | RD_104 | RELATION_NAME | (PARENT) | RI_13 |

### 20.6.2 'NODE_KIND( ATTRIBUTE_KIND_DEFINITION )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| NKI_2 | ATTRIBUTE_KIND_DEFINITION | NKD_26 | NODE_KIND_INTERPRETATION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("NKI corresponding to the NKD which is a generalization defining the properties common to the various kinds of AKDs.") |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_21 | ATTRIBUTE_NAME | (VALUE_TYPE) | AD_87 |
| S | RD_46 | DEFINITION | () | NKD_2 |
| S | RD_92 | PARENT | () | N_6 |
| S | RD_104 | RELATION_NAME | (ALTERNATIVE_OF) | RI_9 |
| S | RD_104 | RELATION_NAME | (ATTRIBUTE_KIND_OF) | RI_18 |
| S | RD_104 | RELATION_NAME | (COMPONENT_OF) | RI_30 |
| S | RD_104 | RELATION_NAME | (COMPONENT_OF_NAMED) | RI_31 |
| S | RD_104 | RELATION_NAME | (COMPONENT_OF_NAMED_-ORDERED) | RI_32 |
| S | RD_104 | RELATION_NAME | (COMPONENT_OF_ORDERED) | RI_33 |


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_104 | RELATION_NAME | (COMPONENT_OF_UNIFORM_-LIST) | RI_34 |
| S | RD_104 | RELATION_NAME | (PARENT) | RI_16 |

### 20.6.3 'NODE_KIND( BASIC )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| NKI_3 | BASIC | NKD_26 | NODE_KIND_INTERPRETATION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("NKI corresponding to the most general NKD.") |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_21 | ATTRIBUTE_NAME | (KIND) | AD_45 |
| S | RD_21 | ATTRIBUTE_NAME | (OBJECT_CLASSIFICATION) | AD_61 |
| S | RD_21 | ATTRIBUTE_NAME | (TIME_ATTRIBUTE_WRITTEN) | AD_77 |
| S | RD_21 | ATTRIBUTE_NAME | (TIME_CREATED) | AD_79 |
| S | RD_21 | ATTRIBUTE_NAME | (TIME_RELATIONSHIP_-WRITTEN) | AD_81 |
| S | RD_21 | ATTRIBUTE_NAME | (UNIQUE_IDENTIFIER) | AD_83 |
| S | RD_46 | DEFINITION | () | NKD_3 |
| S | RD_92 | PARENT | () | N_6 |
| S | RD_104 | RELATION_NAME | (ACCESS) | RI_1 |
| S | RD_104 | RELATION_NAME | (BASIC) | RI_23 |
| S | RD_104 | RELATION_NAME | (CHANNEL) | RI_24 |
| S | RD_104 | RELATION_NAME | (COPY_OF) | RI_37 |
| S | RD_104 | RELATION_NAME | (DEFINITIONS) | RI_48 |
| S | RD_104 | RELATION_NAME | (DENY) | RI_50 |
| S | RD_104 | RELATION_NAME | (INTRODUCED_NODE) | RI_76 |
| S | RD_104 | RELATION_NAME | (MONITORED_BY) | RI_87 |
| S | RD_104 | RELATION_NAME | (PARENT) | RI_93 |
| S | RD_104 | RELATION_NAME | (VIEW) | RI_122 |

### 20.6.4 'NODE_KIND( CHANNEL )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| NKI_4 | CHANNEL | NKD_26 | NODE_KIND_INTERPRETATION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("NKI corresponding to the NKD for CHANNEL nodes.") |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_21 | ATTRIBUTE_NAME | (CONTROLLING_PROTOCOL) | AD_11 |
| S | RD_21 | ATTRIBUTE_NAME | (ELEMENT_SIZE) | AD_20 |
| S | RD_21 | ATTRIBUTE_NAME | (MAXIMUM_LINE_LENGTH) | AD_52 |
| S | RD_21 | ATTRIBUTE_NAME | (MAXIMUM_PAGE_LENGTH) | AD_53 |
| S | RD_21 | ATTRIBUTE_NAME | (STATE) | AD_75 |
| S | RD_46 | DEFINITION | () | NKD_4 |
| S | RD_92 | PARENT | () | N_6 |



Source: https://assist.dla.mil -- Downloaded: 202610-26T07:54Z
Check the source to verify that this is the current version before use.

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_104 | RELATION_NAME | (CLIENT) | RI_26 |
| S | RD_104 | RELATION_NAME | (PARENT) | RI_25 |

### 20.6.5 'NODE_KIND( CLIENT )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| NKI_5 | CLIENT | NKD_26 | NODE_KIND_INTERPRETATION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("NKI corresponding to the NKD for CLIENT nodes.") |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_46 | DEFINITION | () | NKD_5 |
| S | RD_92 | PARENT | () | N_6 |
| S | RD_104 | RELATION_NAME | (PORT) | RI_96 |

### 20.6.6 'NODE_KIND( COMPOSITE_LIST_ATTRIBUTE_KIND_DEFINITION )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| NKI_6 | COMPOSITE_LIST_ATTRIBUTE_-KIND_DEFINITION | NKD_26 | NODE_KIND_INTERPRETATION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("NKI corresponding to the NKD for Composite List AKDs.") |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_21 | ATTRIBUTE_NAME | (NAMING) | AD_59 |
| S | RD_21 | ATTRIBUTE_NAME | (ORDERED) | AD_64 |
| S | RD_46 | DEFINITION | () | NKD_6 |
| S | RD_92 | PARENT | () | N_6 |
| S | RD_104 | RELATION_NAME | (COMPONENT) | RI_27 |

### 20.6.7 'NODE_KIND( DEFINITION )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| NKI_7 | DEFINITION | NKD_26 | NODE_KIND_INTERPRETATION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("NKI corresponding to the NKD which is a generalization defining the properties of definition nodes.") |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_21 | ATTRIBUTE_NAME | (ANNOTATION) | AD_3 |
| S | RD_21 | ATTRIBUTE_NAME | (CAIS_CONTROLLED) | AD_7 |
| S | RD_21 | ATTRIBUTE_NAME | (CAIS_DETERMINED) | AD_8 |
| S | RD_21 | ATTRIBUTE_NAME | (DEFINITION_KIND) | AD_15 |
| S | RD_46 | DEFINITION | () | NKD_7 |
| S | RD_92 | PARENT | () | N_6 |


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

### 20.6.8 'NODE_KIND( DEFINITIONS )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| NKI_8 | DEFINITIONS | NKD_26 | NODE_KIND_INTERPRETATION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("NKI corresponding to the NKD for DEFINITIONS nodes.") |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_21 | ATTRIBUTE_NAME | (ANNOTATION) | AD_3 |
| S | RD_46 | DEFINITION | () | NKD_8 |
| S | RD_92 | PARENT | () | N_6 |
| S | RD_104 | RELATION_NAME | (ATTRIBUTE_DEFINITION) | RI_11 |
| S | RD_104 | RELATION_NAME | (ATTRIBUTE_KIND_-DEFINITION) | RI_15 |
| S | RD_104 | RELATION_NAME | (NODE_KIND_DEFINITION) | RI_90 |
| S | RD_104 | RELATION_NAME | (PARENT) | RI_47 |
| S | RD_104 | RELATION_NAME | (RELATION_DEFINITION) | RI_102 |

### 20.6.9 'NODE_KIND( DEVICE )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| NKI_9 | DEVICE | NKD_26 | NODE_KIND_INTERPRETATION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("NKI corresponding to the NKD for DEVICE nodes.") |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_21 | ATTRIBUTE_NAME | (DEVICE_KIND) | AD_17 |
| S | RD_21 | ATTRIBUTE_NAME | (HIGHEST_CLASSIFICATION) | AD_32 |
| S | RD_21 | ATTRIBUTE_NAME | (LOWEST_CLASSIFICATION) | AD_48 |
| S | RD_46 | DEFINITION | () | NKD_9 |
| S | RD_92 | PARENT | () | N_6 |
| S | RD_104 | RELATION_NAME | (PARENT) | RI_51 |

### 20.6.10 'NODE_KIND( EXECUTABLE_IMAGE )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| NKI_10 | EXECUTABLE_IMAGE | NKD_26 | NODE_KIND_INTERPRETATION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("NKI corresponding to the NKD for EXECUTABLE_-IMAGE nodes.") |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_46 | DEFINITION | () | NKD_10 |
| S | RD_92 | PARENT | () | N_6 |
| S | RD_104 | RELATION_NAME | (DEFAULT_PROCESS_NODE_-KIND) | RI_44 |
| S | RD_104 | RELATION_NAME | (DEFAULT_ROLE) | RI_45 |



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_104 | RELATION_NAME | (VIEW) | RI_123 |

### 20.6.11 'NODE_KIND( FILE )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| NKI_11 | FILE | NKD_26 | NODE_KIND_INTERPRETATION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("NKI corresponding to the NKD for FILE nodes.") |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_21 | ATTRIBUTE_NAME | (FILE_KIND) | AD_22 |
| S | RD_21 | ATTRIBUTE_NAME | (TIME_CONTENTS_WRITTEN) | AD_78 |
| S | RD_46 | DEFINITION | () | NKD_11 |
| S | RD_92 | PARENT | () | N_6 |

### 20.6.12 'NODE_KIND( FLOAT_ATTRIBUTE_KIND_DEFINITION )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| NKI_12 | FLOAT_ATTRIBUTE_KIND_-DEFINITION | NKD_26 | NODE_KIND_INTERPRETATION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("NKI corresponding to the NKD for Float AKDs.") |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_21 | ATTRIBUTE_NAME | (DIGITS) | AD_18 |
| S | RD_21 | ATTRIBUTE_NAME | (LOWER_BOUND) | AD_46 |
| S | RD_21 | ATTRIBUTE_NAME | (UPPER_BOUND) | AD_85 |
| S | RD_46 | DEFINITION | () | NKD_12 |
| S | RD_92 | PARENT | () | N_6 |

### 20.6.13 'NODE_KIND( GATEWAY )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| NKI_13 | GATEWAY | NKD_26 | NODE_KIND_INTERPRETATION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("NKI corresponding to the NKD for GATEWAY nodes.") |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_46 | DEFINITION | () | NKD_13 |
| S | RD_92 | PARENT | () | N_6 |
| S | RD_104 | RELATION_NAME | (PARENT) | RI_57 |


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

### 20.6.14 'NODE_KIND( GROUP )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| NKI_14 | GROUP | NKD_26 | NODE_KIND_INTERPRETATION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("NKI corresponding to the NKD for GROUP nodes.") |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_46 | DEFINITION | () | NKD_14 |
| S | RD_92 | PARENT | () | N_6 |
| S | RD_104 | RELATION_NAME | (GROUP_PARENT) | RI_62 |
| S | RD_104 | RELATION_NAME | (PARENT) | RI_93 |
| S | RD_104 | RELATION_NAME | (PERMANENT_MEMBER) | RI_94 |
| S | RD_104 | RELATION_NAME | (PERMANENT_MEMBER_-PARENT) | RI_95 |
| S | RD_104 | RELATION_NAME | (POTENTIAL_MEMBER) | RI_97 |

### 20.6.15 'NODE_KIND( IDENTIFIER_ATTRIBUTE_KIND_DEFINITION )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| NKI_15 | IDENTIFIER_ATTRIBUTE_KIND_-DEFINITION | NKD_26 | NODE_KIND_INTERPRETATION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("NKI corresponding to the NKD for Identifier AKDs.") |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_21 | ATTRIBUTE_NAME | (ENUMERATED) | AD_21 |
| S | RD_21 | ATTRIBUTE_NAME | (PATTERNS) | AD_67 |
| S | RD_46 | DEFINITION | () | NKD_15 |
| S | RD_92 | PARENT | () | N_6 |

### 20.6.16 'NODE_KIND( INTEGER_ATTRIBUTE_KIND_DEFINITION )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| NKI_16 | INTEGER_ATTRIBUTE_KIND_-DEFINITION | NKD_26 | NODE_KIND_INTERPRETATION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("NKI corresponding to the NKD for Integer AKDs.") |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_21 | ATTRIBUTE_NAME | (LOWER_BOUND) | AD_47 |
| S | RD_21 | ATTRIBUTE_NAME | (UPPER_BOUND) | AD_86 |
| S | RD_46 | DEFINITION | () | NKD_16 |
| S | RD_92 | PARENT | () | N_6 |



Source: https://assist.dla.mil -- Downloaded: 202640-26T07:54Z
Check the source to verify that this is the current version before use.

### 20.6.17 'NODE_KIND( INTERPRETATION )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| NKI_17 | INTERPRETATION | NKD_26 | NODE_KIND_INTERPRETATION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("NKI corresponding to the NKD which is a generalization defining the properties common to interpretation nodes (NKIs and RIs).") |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_21 | ATTRIBUTE_NAME | (ANNOTATION) | AD_3 |
| S | RD_21 | ATTRIBUTE_NAME | (INTERPRETATION_KIND) | AD_40 |
| S | RD_46 | DEFINITION | () | NKD_17 |
| S | RD_92 | PARENT | () | N_6 |
| S | RD_104 | RELATION_NAME | (ATTRIBUTE_NAME) | RI_21 |
| S | RD_104 | RELATION_NAME | (DEFINITION) | RI_46 |

### 20.6.18 'NODE_KIND( INTRODUCED_NODE )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| NKI_18 | INTRODUCED_NODE | NKD_26 | NODE_KIND_INTERPRETATION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("NKI corresponding to the NKD for INTRODUCED_NODE nodes.") |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_21 | ATTRIBUTE_NAME | (IMPORTED_DEFINITION_UID) | AD_33 |
| S | RD_21 | ATTRIBUTE_NAME | (IMPORTED_UID) | AD_37 |
| S | RD_21 | ATTRIBUTE_NAME | (INTRODUCED_NODE_ATTRIBUTES) | AD_42 |
| S | RD_46 | DEFINITION | () | NKD_18 |
| S | RD_92 | PARENT | () | N_6 |
| S | RD_104 | RELATION_NAME | (INTRODUCED_RELATION) | RI_78 |
| S | RD_104 | RELATION_NAME | (PARENT) | RI_77 |

### 20.6.19 'NODE_KIND( LIST_ATTRIBUTE_KIND_DEFINITION )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| NKI_19 | LIST_ATTRIBUTE_KIND_DEFINITION | NKD_26 | NODE_KIND_INTERPRETATION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("NKI corresponding to the NKD for List AKDs.") |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_21 | ATTRIBUTE_NAME | (NAMING) | AD_59 |
| S | RD_46 | DEFINITION | () | NKD_19 |
| S | RD_92 | PARENT | () | N_6 |


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

### 20.6.20 'NODE_KIND( MAGNETIC_TAPE )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| NKI_20 | MAGNETIC_TAPE | NKD_26 | NODE_KIND_INTERPRETATION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("NKI corresponding to the NKD for MAGNETIC_TAPE device nodes.") |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_46 | DEFINITION | () | NKD_20 |
| S | RD_92 | PARENT | () | N_6 |

### 20.6.21 'NODE_KIND( MEDIUM )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| NKI_21 | MEDIUM | NKD_26 | NODE_KIND_INTERPRETATION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("NKI corresponding to the NKD for MEDIUM nodes.") |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_46 | DEFINITION | () | NKD_21 |
| S | RD_92 | PARENT | () | N_6 |
| S | RD_104 | RELATION_NAME | (PARENT) | RI_84 |

### 20.6.22 'NODE_KIND( MONITOR )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| NKI_22 | MONITOR | NKD_26 | NODE_KIND_INTERPRETATION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("NKI corresponding to the NKD for MONITOR nodes.") |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_21 | ATTRIBUTE_NAME | (HANDLER_AS_A_-TRANSACTION) | AD_25 |
| S | RD_21 | ATTRIBUTE_NAME | (HANDLER_DELETE_WHEN_-TERMINATED) | AD_26 |
| S | RD_21 | ATTRIBUTE_NAME | (HANDLER_DISCRETIONARY_-ACCESS) | AD_27 |
| S | RD_21 | ATTRIBUTE_NAME | (HANDLER_INPUT_-PARAMETERS) | AD_28 |
| S | RD_21 | ATTRIBUTE_NAME | (HANDLER_KEY) | AD_29 |
| S | RD_21 | ATTRIBUTE_NAME | (HANDLER_MANDATORY_-ACCESS) | AD_30 |
| S | RD_21 | ATTRIBUTE_NAME | (HANDLER_PRIORITY) | AD_31 |
| S | RD_46 | DEFINITION | () | NKD_22 |
| S | RD_92 | PARENT | () | N_6 |
| S | RD_104 | RELATION_NAME | (HANDLER_ATTRIBUTE) | RI_65 |
| S | RD_104 | RELATION_NAME | (HANDLER_ENVIRONMENT_-NODE) | RI_66 |



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_104 | RELATION_NAME | (HANDLER_ERROR_FILE) | RI_67 |
| S | RD_104 | RELATION_NAME | (HANDLER_IMAGE) | RI_68 |
| S | RD_104 | RELATION_NAME | (HANDLER_INPUT_FILE) | RI_69 |
| S | RD_104 | RELATION_NAME | (HANDLER_MEDIUM_-RESOURCE) | RI_70 |
| S | RD_104 | RELATION_NAME | (HANDLER_OUTPUT_FILE) | RI_71 |
| S | RD_104 | RELATION_NAME | (HANDLER_PROCESS) | RI_72 |
| S | RD_104 | RELATION_NAME | (HANDLER_PROCESS_NODE_-KIND) | RI_73 |
| S | RD_104 | RELATION_NAME | (HANDLER_PROCESSOR_-RESOURCE) | RI_75 |
| S | RD_104 | RELATION_NAME | (MONITORS) | RI_88 |
| S | RD_104 | RELATION_NAME | (PARENT) | RI_20 |

### 20.6.23 'NODE_KIND( NAMED_COMPOSITE_LIST_AKD )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| NKI_23 | NAMED_COMPOSITE_LIST_AKD | NKD_26 | NODE_KIND_INTERPRETATION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("NKI corresponding to the NKD for Named Composite List AKDs.") |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_46 | DEFINITION | () | NKD_23 |
| S | RD_92 | PARENT | () | N_6 |
| S | RD_104 | RELATION_NAME | (COMPONENT) | RI_28 |

### 20.6.24 'NODE_KIND( NAMED_ORDERED_COMPOSITE_LIST_AKD )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| NKI_24 | NAMED_ORDERED_-COMPOSITE_LIST_AKD | NKD_26 | NODE_KIND_INTERPRETATION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("NKI corresponding to the NKD for Named and Ordered Composite List AKDs.") |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_46 | DEFINITION | () | NKD_24 |
| S | RD_92 | PARENT | () | N_6 |
| S | RD_104 | RELATION_NAME | (COMPONENT) | RI_29 |

### 20.6.25 'NODE_KIND( NODE_KIND_DEFINITION )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| NKI_25 | NODE_KIND_DEFINITION | NKD_26 | NODE_KIND_INTERPRETATION |



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("NKI corresponding to the NKD for NKDs.") |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_46 | DEFINITION | () | NKD_25 |
| S | RD_92 | PARENT | () | N_6 |
| S | RD_104 | RELATION_NAME | (ATTRIBUTE) | RI_10 |
| S | RD_104 | RELATION_NAME | (EMANATES) | RI_54 |
| S | RD_104 | RELATION_NAME | (GENERALIZATION_OF) | RI_60 |
| S | RD_104 | RELATION_NAME | (PARENT) | RI_91 |
| S | RD_104 | RELATION_NAME | (SPECIALIZATION_OF) | RI_111 |
| S | RD_104 | RELATION_NAME | (TERMINATES) | RI_116 |

### 20.6.26 'NODE_KIND( NODE_KIND_INTERPRETATION )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| NKI_26 | NODE_KIND_INTERPRETATION | NKD_26 | NODE_KIND_INTERPRETATION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("NKI corresponding to the NKD for NKIs.") |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_46 | DEFINITION | () | NKD_26 |
| S | RD_92 | PARENT | () | N_6 |
| S | RD_104 | RELATION_NAME | (PARENT) | RI_92 |
| S | RD_104 | RELATION_NAME | (RELATION_NAME) | RI_104 |

### 20.6.27 'NODE_KIND( ORDERED_COMPOSITE_LIST_AKD )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| NKI_27 | ORDERED_COMPOSITE_LIST_-AKD | NKD_26 | NODE_KIND_INTERPRETATION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("NKI corresponding to the NKD for Ordered Composite List AKDs.") |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_46 | DEFINITION | () | NKD_27 |
| S | RD_92 | PARENT | () | N_6 |
| S | RD_104 | RELATION_NAME | (COMPONENT) | RI_35 |

### 20.6.28 'NODE_KIND( PROCESS )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| NKI_28 | PROCESS | NKD_26 | NODE_KIND_INTERPRETATION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("NKI corresponding to the NKD for PROCESS nodes.") |



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_21 | ATTRIBUTE_NAME | (CURRENT_STATUS) | AD_13 |
| S | RD_21 | ATTRIBUTE_NAME | (IO_UNIT_COUNT) | AD_44 |
| S | RD_21 | ATTRIBUTE_NAME | (MACHINE_TIME) | AD_49 |
| S | RD_21 | ATTRIBUTE_NAME | (OPEN_NODE_HANDLE_COUNT) | AD_62 |
| S | RD_21 | ATTRIBUTE_NAME | (PARAMETERS) | AD_66 |
| S | RD_21 | ATTRIBUTE_NAME | (PRIORITY) | AD_69 |
| S | RD_21 | ATTRIBUTE_NAME | (PROCESS_SIZE) | AD_71 |
| S | RD_21 | ATTRIBUTE_NAME | (RESULTS) | AD_73 |
| S | RD_21 | ATTRIBUTE_NAME | (SUBJECT_CLASSIFICATION) | AD_74 |
| S | RD_21 | ATTRIBUTE_NAME | (TIME_FINISHED) | AD_80 |
| S | RD_21 | ATTRIBUTE_NAME | (TIME_STARTED) | AD_82 |
| S | RD_46 | DEFINITION | () | NKD_28 |
| S | RD_92 | PARENT | () | N_6 |
| S | RD_104 | RELATION_NAME | (ADOPTED_ROLE) | RI_2 |
| S | RD_104 | RELATION_NAME | (ADOPTED_VIEW) | RI_3 |
| S | RD_104 | RELATION_NAME | (CURRENT_JOB) | RI_38 |
| S | RD_104 | RELATION_NAME | (CURRENT_NODE) | RI_39 |
| S | RD_104 | RELATION_NAME | (CURRENT_PROCESSOR) | RI_40 |
| S | RD_104 | RELATION_NAME | (CURRENT_USER) | RI_41 |
| S | RD_104 | RELATION_NAME | (DEFINITIONS) | RI_49 |
| S | RD_104 | RELATION_NAME | (DEVICE) | RI_53 |
| S | RD_104 | RELATION_NAME | (EXECUTABLE_IMAGE) | RI_56 |
| S | RD_104 | RELATION_NAME | (GROUP) | RI_64 |
| S | RD_104 | RELATION_NAME | (HANDLER_PROCESS_OF) | RI_74 |
| S | RD_104 | RELATION_NAME | (PARENT) | RI_80 |
| S | RD_104 | RELATION_NAME | (RESOURCE_GROUP) | RI_108 |
| S | RD_104 | RELATION_NAME | (STANDARD_ERROR) | RI_113 |
| S | RD_104 | RELATION_NAME | (STANDARD_INPUT) | RI_114 |
| S | RD_104 | RELATION_NAME | (STANDARD_OUTPUT) | RI_115 |
| S | RD_104 | RELATION_NAME | (USER) | RI_120 |
| S | RD_104 | RELATION_NAME | (VIEW) | RI_123 |

### 20.6.29 'NODE_KIND( PROCESSOR )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| NKI_29 | PROCESSOR | NKD_26 | NODE_KIND_INTERPRETATION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("NKI corresponding to the NKD for PROCESSOR nodes.") |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_46 | DEFINITION | () | NKD_29 |
| S | RD_92 | PARENT | () | N_6 |
| S | RD_104 | RELATION_NAME | (ALLOCATABLE_DEVICE) | RI_4 |
| S | RD_104 | RELATION_NAME | (ALLOCATABLE_GATEWAY) | RI_5 |
| S | RD_104 | RELATION_NAME | (ALLOCATABLE_MEDIUM) | RI_6 |
| S | RD_104 | RELATION_NAME | (ALLOCATABLE_PROCESSOR) | RI_7 |
| S | RD_104 | RELATION_NAME | (DEFAULT_MEDIUM) | RI_42 |
| S | RD_104 | RELATION_NAME | (DEFAULT_PROCESSOR) | RI_43 |
| S | RD_104 | RELATION_NAME | (PARENT) | RI_98 |



Source: https://assist.dla.mil -- Downloaded: 202610-26T07:54Z
Check the source to verify that this is the current version before use.

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| NKI_30 | RELATION_DEFINITION | NKD_26 | NODE_KIND_INTERPRETATION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("NKI corresponding to the NKD for RDs.") |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_21 | ATTRIBUTE_NAME | (ALLOWED_USAGE) | AD_2 |
| S | RD_21 | ATTRIBUTE_NAME | (UNIQUE_KEYS) | AD_84 |
| S | RD_46 | DEFINITION | () | NKD_30 |
| S | RD_92 | PARENT | () | N_6 |
| S | RD_104 | RELATION_NAME | (ATTRIBUTE) | RI_10 |
| S | RD_104 | RELATION_NAME | (EMANATES_FROM) | RI_55 |
| S | RD_104 | RELATION_NAME | (GENERALIZATION_OF) | RI_61 |
| S | RD_104 | RELATION_NAME | (KEY) | RI_81 |
| S | RD_104 | RELATION_NAME | (PARENT) | RI_103 |
| S | RD_104 | RELATION_NAME | (REVERSE_LINK) | RI_109 |
| S | RD_104 | RELATION_NAME | (REVERSE_LINK_-BIDIRECTIONAL) | RI_110 |
| S | RD_104 | RELATION_NAME | (SPECIALIZATION_OF) | RI_112 |
| S | RD_104 | RELATION_NAME | (TERMINATES_AT) | RI_117 |

### 20.6.31 'NODE_KIND( RELATION_INTERPRETATION )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| NKI_31 | RELATION_INTERPRETATION | NKD_26 | NODE_KIND_INTERPRETATION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("NKI corresponding to the NKD for RIs.") |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_46 | DEFINITION | () | NKD_31 |
| S | RD_92 | PARENT | () | N_6 |
| S | RD_104 | RELATION_NAME | (KEY_NAME) | RI_82 |
| S | RD_104 | RELATION_NAME | (PARENT) | RI_105 |

### 20.6.32 'NODE_KIND( RESOURCE )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| NKI_32 | RESOURCE | NKD_26 | NODE_KIND_INTERPRETATION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("NKI corresponding to the NKD which is a generalisation defining the properties common to the various kinds of nodes representing resources.") |



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_21 | ATTRIBUTE_NAME | (ANNOTATION) | AD_3 |
| S | RD_21 | ATTRIBUTE_NAME | (AVAILABLE) | AD_6 |
| S | RD_46 | DEFINITION | () | NKD_32 |
| S | RD_92 | PARENT | () | N_6 |

### 20.6.33 'NODE_KIND( RESOURCE_GROUP )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| NKI_33 | RESOURCE_GROUP | NKD_26 | NODE_KIND_INTERPRETATION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("NKI corresponding to the NKD for RESOURCE_GROUP nodes.") |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_21 | ATTRIBUTE_NAME | (ANNOTATION) | AD_3 |
| S | RD_46 | DEFINITION | () | NKD_33 |
| S | RD_92 | PARENT | () | N_6 |
| S | RD_104 | RELATION_NAME | (DEVICE) | RI_53 |
| S | RD_104 | RELATION_NAME | (GATEWAY) | RI_59 |
| S | RD_104 | RELATION_NAME | (MEDIUM) | RI_86 |
| S | RD_104 | RELATION_NAME | (PROCESSOR) | RI_100 |
| S | RD_104 | RELATION_NAME | (PARENT) | RI_106 |

### 20.6.34 'NODE_KIND( SECONDARY_STORAGE )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| NKI_34 | SECONDARY_STORAGE | NKD_26 | NODE_KIND_INTERPRETATION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("NKI corresponding to the NKD for SECONDARY_-STORAGE nodes.") |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_21 | ATTRIBUTE_NAME | (CURRENT_FILE_SIZE) | AD_12 |
| S | RD_21 | ATTRIBUTE_NAME | (MAXIMUM_FILE_SIZE) | AD_50 |
| S | RD_46 | DEFINITION | () | NKD_34 |
| S | RD_92 | PARENT | () | N_6 |

### 20.6.35 'NODE_KIND( STRING_ATTRIBUTE_KIND_DEFINITION )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| NKI_35 | STRING_ATTRIBUTE_KIND_-DEFINITION | NKD_26 | NODE_KIND_INTERPRETATION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("NKI corresponding to the NKD for String AKDs.") |



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_21 | ATTRIBUTE_NAME | (PATTERNS) | AD_67 |
| S | RD_46 | DEFINITION | () | NKD_35 |
| S | RD_92 | PARENT | () | N_6 |

### 20.6.36 'NODE_KIND( STRUCTURAL )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| NKI_36 | STRUCTURAL | NKD_26 | NODE_KIND_INTERPRETATION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("NKI corresponding to the NKD for most general STRUCTURAL node kind.") |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_21 | ATTRIBUTE_NAME | (STRUCTURAL_KIND) | AD_76 |
| S | RD_46 | DEFINITION | () | NKD_36 |
| S | RD_92 | PARENT | () | N_6 |

### 20.6.37 'NODE_KIND( SYSTEM_LEVEL )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| NKI_37 | SYSTEM_LEVEL | NKD_26 | NODE_KIND_INTERPRETATION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("NKI corresponding to the NKD for the System Level node.") |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_46 | DEFINITION | () | NKD_37 |
| S | RD_92 | PARENT | () | N_6 |
| S | RD_104 | RELATION_NAME | (DEVICE) | RI_52 |
| S | RD_104 | RELATION_NAME | (GATEWAY) | RI_58 |
| S | RD_104 | RELATION_NAME | (GROUP) | RI_63 |
| S | RD_104 | RELATION_NAME | (MEDIUM) | RI_85 |
| S | RD_104 | RELATION_NAME | (PROCESSOR) | RI_99 |
| S | RD_104 | RELATION_NAME | (RESOURCE_GROUP) | RI_107 |
| S | RD_104 | RELATION_NAME | (USER) | RI_119 |

### 20.6.38 'NODE_KIND( TERMINAL )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| NKI_38 | TERMINAL | NKD_26 | NODE_KIND_INTERPRETATION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("NKI corresponding to the NKD for TERMINAL device nodes.") |



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_46 | DEFINITION | () | NKD_38 |
| S | RD_92 | PARENT | () | N_6 |

### 20.6.39 'NODE_KIND( UNIFORM_LIST_ATTRIBUTE_KIND_DEFINITION )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| NKI_39 | UNIFORM_LIST_ATTRIBUTE_-KIND_DEFINITION | NKD_26 | NODE_KIND_INTERPRETATION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("NKI corresponding to the NKD for Uniform List AKDs.") |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_21 | ATTRIBUTE_NAME | (MAXIMUM_SIZE) | AD_54 |
| S | RD_21 | ATTRIBUTE_NAME | (MINIMUM_SIZE) | AD_56 |
| S | RD_21 | ATTRIBUTE_NAME | (NAMING) | AD_59 |
| S | RD_46 | DEFINITION | () | NKD_39 |
| S | RD_92 | PARENT | () | N_6 |
| S | RD_104 | RELATION_NAME | (COMPONENT) | RI_27 |
| S | RD_104 | RELATION_NAME | (COMPONENT_UNIFORM_LIST) | RI_36 |

### 20.6.40 'NODE_KIND( USER )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| NKI_40 | USER | NKD_26 | NODE_KIND_INTERPRETATION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("NKI corresponding to the NKD for USER nodes.") |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_46 | DEFINITION | () | NKD_40 |
| S | RD_92 | PARENT | () | N_6 |
| S | RD_104 | RELATION_NAME | (ATTRIBUTE_MONITOR) | RI_19 |
| S | RD_104 | RELATION_NAME | (DEFAULT_ROLE) | RI_45 |
| S | RD_104 | RELATION_NAME | (JOB) | RI_79 |
| S | RD_104 | RELATION_NAME | (PARENT) | RI_118 |

### 20.6.41 'NODE_KIND( VARIANT_ATTRIBUTE_KIND_DEFINITION )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| NKI_41 | VARIANT_ATTRIBUTE_KIND_-DEFINITION | NKD_26 | NODE_KIND_INTERPRETATION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("NKI corresponding to the NKD for Variant AKDs.") |



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_46 | DEFINITION | () | NKD_41 |
| S | RD_92 | PARENT | () | N_6 |
| S | RD_104 | RELATION_NAME | (ALTERNATIVE) | RI_8 |

### 20.6.42 'NODE_KIND( VIEW )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| NKI_42 | VIEW | NKD_26 | NODE_KIND_INTERPRETATION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("NKI corresponding to the NKD for VIEW nodes.") |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_21 | ATTRIBUTE_NAME | (ANNOTATION) | AD_3 |
| S | RD_46 | DEFINITION | () | NKD_42 |
| S | RD_92 | PARENT | () | N_6 |
| S | RD_104 | RELATION_NAME | (ATTRIBUTE) | RI_12 |
| S | RD_104 | RELATION_NAME | (ATTRIBUTE_KIND) | RI_17 |
| S | RD_104 | RELATION_NAME | (NODE_KIND) | RI_89 |
| S | RD_104 | RELATION_NAME | (RELATION) | RI_101 |
| S | RD_104 | RELATION_NAME | (PARENT) | RI_121 |


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

## 20.7 Relation interpretation nodes (RIs)

The following table describes the node attributes which have the same value on all RIs: 1

| Attribute Name | AD | Attribute Value |
|---|---|---|
| KIND | AD_45 | (STRUCTURAL) |
| STRUCTURAL_KIND | AD_76 | (INTERPRETATION) |
| INTERPRETATION_KIND | AD_40 | (RELATION) |

The following subsections describe each predefined RI. Each section name is the primary path element from 'VIEW( PREDEFINED ) to the RI described. 2

### 20.7.1 'RELATION( ACCESS )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RI_1 | ACCESS | NKD_31 | RELATION_INTERPRETATION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("RI corresponding to the RD whose instances identify accesses permitted on (source) objects by (target) groups.") |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_21 | ATTRIBUTE_NAME | (GRANT) | AD_24 |
| S | RD_46 | DEFINITION | () | RD_1 |
| S | RD_82 | KEY_NAME | (KEY) | AD_9 |
| S | RD_105 | PARENT | () | N_6 |

### 20.7.2 'RELATION( ADOPTED_ROLE )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RI_2 | ADOPTED_ROLE | NKD_31 | RELATION_INTERPRETATION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("RI corresponding to the RD whose instances identify the roles under which the (source) process executes.") |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_46 | DEFINITION | () | RD_2 |
| S | RD_82 | KEY_NAME | (NAME) | AD_58 |
| S | RD_105 | PARENT | () | N_6 |

### 20.7.3 'RELATION( ADOPTED_VIEW )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RI_3 | ADOPTED_VIEW | NKD_31 | RELATION_INTERPRETATION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("RI corresponding to the RD whose instances identify the views adopted by the (source) process.") |



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_46 | DEFINITION | () | RD_3 |
| S | RD_82 | KEY_NAME | (POSITION) | AD_68 |
| S | RD_105 | PARENT | () | N_6 |

### 20.7.4 'RELATION( ALLOCATABLE_DEVICE )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RI_4 | ALLOCATABLE_DEVICE | NKD_31 | RELATION_INTERPRETATION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("RI corresponding to the RD whose instances identify the devices allocatable from the (source) processor.") |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_46 | DEFINITION | () | RD_4 |
| S | RD_82 | KEY_NAME | (KEY) | AD_23 |
| S | RD_105 | PARENT | () | N_6 |

### 20.7.5 'RELATION( ALLOCATABLE_GATEWAY )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RI_5 | ALLOCATABLE_GATEWAY | NKD_31 | RELATION_INTERPRETATION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("RI corresponding to the RD whose instances identify the gateways allocatable from the (source) processor.") |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_46 | DEFINITION | () | RD_5 |
| S | RD_82 | KEY_NAME | (KEY) | AD_23 |
| S | RD_105 | PARENT | () | N_6 |

### 20.7.6 'RELATION( ALLOCATABLE_MEDIUM )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RI_6 | ALLOCATABLE_MEDIUM | NKD_31 | RELATION_INTERPRETATION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("RI corresponding to the RD whose instances identify the media allocatable from the (source) processor.") |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_46 | DEFINITION | () | RD_6 |
| S | RD_82 | KEY_NAME | (KEY) | AD_23 |
| S | RD_105 | PARENT | () | N_6 |



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

### 20.7.7 'RELATION( ALLOCATABLE_PROCESSOR )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RI_7 | ALLOCATABLE_PROCESSOR | NKD_31 | RELATION_INTERPRETATION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("RI corresponding to the RD whose instances identify the processors allocatable from the (source) processor.") |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_46 | DEFINITION | () | RD_7 |
| S | RD_82 | KEY_NAME | (KEY) | AD_23 |
| S | RD_105 | PARENT | () | N_6 |

### 20.7.8 'RELATION( ALTERNATIVE )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RI_8 | ALTERNATIVE | NKD_31 | RELATION_INTERPRETATION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("RI corresponding to the RD whose instances identify the AKDs which are alternatives of the (source) variant AKD.") |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_21 | ATTRIBUTE_NAME | (CONSTRAINTS) | AD_10 |
| S | RD_46 | DEFINITION | () | RD_8 |
| S | RD_82 | KEY_NAME | (NAME) | AD_58 |
| S | RD_105 | PARENT | () | N_6 |

### 20.7.9 'RELATION( ALTERNATIVE_OF )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RI_9 | ALTERNATIVE | NKD_31 | RELATION_INTERPRETATION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("RI corresponding to the RD whose instances identify the variant AKDs of which the (source) AKD is an alternative.") |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_46 | DEFINITION | () | RD_9 |
| S | RD_82 | KEY_NAME | (KEY) | AD_23 |
| S | RD_105 | PARENT | () | N_6 |

### 20.7.10 'RELATION( ATTRIBUTE )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RI_10 | ATTRIBUTE_NAME | NKD_31 | RELATION_INTERPRETATION |



Source: https://assist.dla.mil -- Downloaded: 202610-26T07:54Z
Check the source to verify that this is the current version before use.

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("RI corresponding to the RD whose instances identify the attributes for (source) node kinds and relations.") |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_21 | ATTRIBUTE_NAME | (CONSTRAINTS) | AD_10 |
| S | RD_21 | ATTRIBUTE_NAME | (DEFAULT_VALUE) | AD_14 |
| S | RD_46 | DEFINITION | () | RD_10 |
| S | RD_82 | KEY_NAME | (KEY) | AD_23 |
| S | RD_105 | PARENT | () | N_6 |

### 20.7.11 'RELATION( ATTRIBUTE_DEFINITION )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RI_11 | ATTRIBUTE_DEFINITION | NKD_31 | RELATION_INTERPRETATION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("RI corresponding to the RD whose instances are primary relationships to ADs.") |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_46 | DEFINITION | () | RD_11 |
| S | RD_82 | KEY_NAME | (KEY) | AD_23 |
| S | RD_105 | PARENT | () | N_6 |

### 20.7.12 'RELATION( ATTRIBUTE_DEFINITION_NAME )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RI_12 | ATTRIBUTE_DEFINITION_NAME | NKD_31 | RELATION_INTERPRETATION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("RI corresponding to the RD whose instances specify a (source) view-relative name for the (target) AD.") |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_46 | DEFINITION | () | RD_12 |
| S | RD_82 | KEY_NAME | (NAME) | AD_58 |
| S | RD_105 | PARENT | () | N_6 |

### 20.7.13 'RELATION( ATTRIBUTE_DEFINITION_PARENT )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RI_13 | ATTRIBUTE_DEFINITION_-PARENT | NKD_31 | RELATION_INTERPRETATION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("RI corresponding to the RD whose instances identify the parent of the (source) AD.") |



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_46 | DEFINITION | () | RD_13 |
| S | RD_105 | PARENT | () | N_6 |

### 20.7.14 'RELATION( ATTRIBUTE_KIND )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RI_14 | ATTRIBUTE_KIND | NKD_31 | RELATION_INTERPRETATION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("RI corresponding to the RD whose instances identify the AKD for the (source) AD.") |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_21 | ATTRIBUTE_NAME | (CONSTRAINTS) | AD_10 |
| S | RD_21 | ATTRIBUTE_NAME | (DEFAULT_VALUE) | AD_14 |
| S | RD_46 | DEFINITION | () | RD_14 |
| S | RD_105 | PARENT | () | N_6 |

### 20.7.15 'RELATION( ATTRIBUTE_KIND_DEFINITION )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RI_15 | ATTRIBUTE_KIND_DEFINITION | NKD_31 | RELATION_INTERPRETATION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("RI corresponding to the RD whose instances are primary relationships to AKDs.") |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_46 | DEFINITION | () | RD_15 |
| S | RD_82 | KEY_NAME | (KEY) | AD_23 |
| S | RD_105 | PARENT | () | N_6 |

### 20.7.16 'RELATION( ATTRIBUTE_KIND_DEFINITION_PARENT )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RI_16 | ATTRIBUTE_KIND_DEFINITION_-PARENT | NKD_31 | RELATION_INTERPRETATION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("RI corresponding to the RD whose instances identify the parent of the (source) AKD.") |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_46 | DEFINITION | () | RD_16 |
| S | RD_105 | PARENT | () | N_6 |



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

### 20.7.17 'RELATION( ATTRIBUTE_KIND_NAME )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RI_17 | ATTRIBUTE_KIND_NAME | NKD_31 | RELATION_INTERPRETATION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("RI corresponding to the RD whose instances specify a (source) view-relative name for the AKD.") |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_46 | DEFINITION | () | RD_17 |
| S | RD_82 | KEY_NAME | (NAME) | AD_58 |
| S | RD_105 | PARENT | () | N_6 |

### 20.7.18 'RELATION( ATTRIBUTE_KIND_OF )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RI_18 | ATTRIBUTE_KIND_OF | NKD_31 | RELATION_INTERPRETATION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("RI corresponding to the RD whose instances identify the ADs for the (source) AKD.") |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_46 | DEFINITION | () | RD_18 |
| S | RD_82 | KEY_NAME | (KEY) | AD_23 |
| S | RD_105 | PARENT | () | N_6 |

### 20.7.19 'RELATION( ATTRIBUTE_MONITOR )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RI_19 | ATTRIBUTE_MONITOR | NKD_31 | RELATION_INTERPRETATION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("RI corresponding to the RD whose instances identify the attribute monitors of the (source) user.") |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_46 | DEFINITION | () | RD_19 |
| S | RD_82 | KEY_NAME | (KEY) | AD_23 |
| S | RD_105 | PARENT | () | N_6 |

### 20.7.20 'RELATION( ATTRIBUTE_MONITOR_PARENT )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RI_20 | ATTRIBUTE_MONITOR_PARENT | NKD_31 | RELATION_INTERPRETATION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("RI corresponding to the RD whose instances identify the parent and user of the (source) attribute monitor.") |



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_46 | DEFINITION | () | RD_20 |
| S | RD_105 | PARENT | () | N_6 |

### 20.7.21 'RELATION( ATTRIBUTE_NAME )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RI_21 | ATTRIBUTE_NAME | NKD_31 | RELATION_INTERPRETATION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("RI corresponding to the RD whose instances specify a (source) interpretation-relative name for the (target) attribute.") |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_46 | DEFINITION | () | RD_21 |
| S | RD_82 | KEY_NAME | (NAME) | AD_58 |
| S | RD_105 | PARENT | () | N_6 |

### 20.7.22 'RELATION( ATTRIBUTE_OF )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RI_22 | ATTRIBUTE_NAME_OF | NKD_31 | RELATION_INTERPRETATION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("RI corresponding to the RD whose instances identify the node kinds and relations utilising the (source) attribute.") |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_46 | DEFINITION | () | RD_22 |
| S | RD_82 | KEY_NAME | (KEY) | AD_23 |
| S | RD_105 | PARENT | () | N_6 |

### 20.7.23 'RELATION( BASIC )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RI_23 | BASIC | NKD_31 | RELATION_INTERPRETATION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("RI corresponding to the most general RD. ") |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_21 | ATTRIBUTE_NAME | (INHERITABLE) | AD_39 |
| S | RD_21 | ATTRIBUTE_NAME | (NODE_KIND) | AD_60 |
| S | RD_46 | DEFINITION | () | RD_23 |
| S | RD_105 | PARENT | () | N_6 |



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

### 20.7.24 'RELATION( CHANNEL )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RI_24 | CHANNEL | NKD_31 | RELATION_INTERPRETATION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("RI corresponding to the RD whose instances are primary relationships to CHANNEL nodes.") |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_46 | DEFINITION | () | RD_24 |
| S | RD_82 | KEY_NAME | (KEY) | AD_23 |
| S | RD_105 | PARENT | () | N_6 |

### 20.7.25 'RELATION( CHANNEL_PARENT )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RI_25 | CHANNEL_PARENT | NKD_31 | RELATION_INTERPRETATION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("RI corresponding to the RD whose instances identify the parent of (source) CHANNEL node.") |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_46 | DEFINITION | () | RD_25 |
| S | RD_105 | PARENT | () | N_6 |

### 20.7.26 'RELATION( CLIENT )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RI_26 | CLIENT | NKD_31 | RELATION_INTERPRETATION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("RI corresponding to the RD whose instances identify the clients of the (source) CHANNEL node.") |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_46 | DEFINITION | () | RD_26 |
| S | RD_82 | KEY_NAME | (KEY) | AD_9 |
| S | RD_105 | PARENT | () | N_6 |

### 20.7.27 'RELATION( COMPONENT )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RI_27 | COMPONENT | NKD_31 | RELATION_INTERPRETATION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("RI corresponding to the RD which is a generalization of relations whose instances identify AKDs which are components of the (source) composite or uniform list AKD.") |



Source: https://assist.dla.mil -- Downloaded: 202610-26T07:54Z
Check the source to verify that this is the current version before use.

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_21 | ATTRIBUTE_NAME | (CONSTRAINTS) | AD_10 |
| S | RD_46 | DEFINITION | () | RD_27 |
| S | RD_105 | PARENT | () | N_6 |

### 20.7.28 'RELATION( COMPONENT_NAMED )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RI_28 | COMPONENT_NAMED | NKD_31 | RELATION_INTERPRETATION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("RI corresponding to the RD whose instances identify the AKDs which are named components of the (source) composite list AKD.") |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_21 | ATTRIBUTE_NAME | (OPTIONAL) | AD_63 |
| S | RD_46 | DEFINITION | () | RD_28 |
| S | RD_82 | KEY_NAME | (NAME) | AD_58 |
| S | RD_105 | PARENT | () | N_6 |

### 20.7.29 'RELATION( COMPONENT_NAMED_ORDERED )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RI_29 | COMPONENT_NAMED_-ORDERED | NKD_31 | RELATION_INTERPRETATION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("RI corresponding to the RD whose instances identify the AKDs which are named, ordered components of the (source) composite list AKD.") |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_46 | DEFINITION | () | RD_29 |
| S | RD_105 | PARENT | () | N_6 |

### 20.7.30 'RELATION( COMPONENT_OF )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RI_30 | COMPONENT_OF | NKD_31 | RELATION_INTERPRETATION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("RI corresponding to the RD which is a generalization of relations whose instances identify the composite or uniform list AKDs of which the (source) AKD is a component.") |


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_46 | DEFINITION | () | RD_30 |
| S | RD_82 | KEY_NAME | (KEY) | AD_23 |
| S | RD_105 | PARENT | () | N_6 |

### 20.7.31 'RELATION( COMPONENT_OF_NAMED )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RI_31 | COMPONENT_OF_NAMED | NKD_31 | RELATION_INTERPRETATION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("RI corresponding to the RD whose instances identify the composite list AKDs of which the (source) AKD is a named component.") |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_46 | DEFINITION | () | RD_31 |
| S | RD_105 | PARENT | () | N_6 |

### 20.7.32 'RELATION( COMPONENT_OF_NAMED_ORDERED )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RI_32 | COMPONENT_OF_NAMED_-ORDERED | NKD_31 | RELATION_INTERPRETATION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("RI corresponding to the RD whose instances identify the composite list AKDs of which the (source) AKD is a named, ordered component.") |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_46 | DEFINITION | () | RD_32 |
| S | RD_105 | PARENT | () | N_6 |

### 20.7.33 'RELATION( COMPONENT_OF_ORDERED )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RI_33 | COMPONENT_OF_ORDERED | NKD_31 | RELATION_INTERPRETATION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("RI corresponding to the RD whose instances identify the composite list AKDs of which the (source) AKD is an ordered component.") |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_46 | DEFINITION | () | RD_33 |
| S | RD_105 | PARENT | () | N_6 |



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

### 20.7.34 'RELATION( COMPONENT_OF_UNIFORM_LIST )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RI_34 | COMPONENT_OF_UNIFORM_-LIST | NKD_31 | RELATION_INTERPRETATION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("RI corresponding to the RD whose instances identify the unifrom list AKDs of which the (source) AKD is the component.") |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_46 | DEFINITION | () | RD_33 |
| S | RD_105 | PARENT | () | N_6 |

### 20.7.35 'RELATION( COMPONENT_ORDERED )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RI_35 | COMPONENT_ORDERED | NKD_31 | RELATION_INTERPRETATION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("RI corresponding to the RD whose instances identify the AKDs which are ordered components of the (source) composite list AKD.") |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_21 | ATTRIBUTE_NAME | (OPTIONAL) | AD_63 |
| S | RD_46 | DEFINITION | () | RD_35 |
| S | RD_82 | KEY_NAME | (POSITION) | AD_68 |
| S | RD_105 | PARENT | () | N_6 |

### 20.7.36 'RELATION( COMPONENT_UNIFORM_LIST )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RI_36 | COMPONENT_UNIFORM_LIST | NKD_31 | RELATION_INTERPRETATION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("RI corresponding to the RD whose instances identify the AKD which is the component of the (source) uniform list AKD.") |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_46 | DEFINITION | () | RD_35 |
| S | RD_105 | PARENT | () | N_6 |

### 20.7.37 'RELATION( COPY_OF )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RI_37 | COPY_OF | NKD_31 | RELATION_INTERPRETATION |


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("RI corresponding to the RD whose instances identify the origin node corresponding to a node created by a copy operation.") |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_46 | DEFINITION | () | RD_37 |
| S | RD_105 | PARENT | () | N_6 |

### 20.7.38 'RELATION( CURRENT_JOB )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RI_38 | CURRENT_JOB | NKD_31 | RELATION_INTERPRETATION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("RI corresponding to the RD whose instances identify the process which is root of the tree containing the source process.") |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_46 | DEFINITION | () | RD_38 |
| S | RD_105 | PARENT | () | N_6 |

### 20.7.39 'RELATION( CURRENT_NODE )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RI_39 | CURRENT_NODE | NKD_31 | RELATION_INTERPRETATION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("RI corresponding to the RD whose instances identify the node to which pathnames starting with a key are relative for the source process.") |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_46 | DEFINITION | () | RD_39 |
| S | RD_105 | PARENT | () | N_6 |

### 20.7.40 'RELATION( CURRENT_PROCESSOR )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RI_40 | CURRENT_PROCESSOR | NKD_31 | RELATION_INTERPRETATION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("RI corresponding to the RD whose instances identify the processor on which the (source) process executes.") |



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_46 | DEFINITION | () | RD_40 |
| S | RD_105 | PARENT | () | N_6 |

### 20.7.41 'RELATION( CURRENT_USER )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RI_41 | CURRENT_USER | NKD_31 | RELATION_INTERPRETATION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("RI corresponding to the RD whose instances identify the user node under which the (source) process executes.") |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_46 | DEFINITION | () | RD_41 |
| S | RD_105 | PARENT | () | N_6 |

### 20.7.42 'RELATION( DEFAULT_MEDIUM )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RI_42 | DEFAULT_MEDIUM | NKD_31 | RELATION_INTERPRETATION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("RI corresponding to the RD whose instances identify the medium which is allocated by default for processes on the (source) processor.") |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_46 | DEFINITION | () | RD_42 |
| S | RD_105 | PARENT | () | N_6 |

### 20.7.43 'RELATION( DEFAULT_PROCESSOR )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RI_43 | DEFAULT_PROCESSOR | NKD_31 | RELATION_INTERPRETATION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("RI corresponding to the RD whose instances identify the processor which is allocated by default for processes on the (source) processor.") |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_46 | DEFINITION | () | RD_43 |
| S | RD_105 | PARENT | () | N_6 |

### 20.7.44 'RELATION( DEFAULT_PROCESS_NODE_KIND )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RI_44 | DEFAULT_PROCESS_NODE_KIND | NKD_31 | RELATION_INTERPRETATION |



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("RI corresponding to the RD whose instances identify the process NKD which is instantiated by default for execution of the (source) executable image.") |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_46 | DEFINITION | () | RD_44 |
| S | RD_105 | PARENT | () | N_6 |

### 20.7.45 'RELATION( DEFAULT_ROLE )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RI_45 | DEFAULT_ROLE | NKD_31 | RELATION_INTERPRETATION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("RI corresponding to the RD whose instances identify the roles which are always adopted for processes of the (source) user or executable image.") |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_46 | DEFINITION | () | RD_45 |
| S | RD_105 | PARENT | () | N_6 |

### 20.7.46 'RELATION( DEFINITION )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RI_46 | DEFINITION | NKD_31 | RELATION_INTERPRETATION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("RI corresponding to the RD whose instances identify the DEFINITION node for which the source node is an interpretation.") |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_46 | DEFINITION | () | RD_46 |
| S | RD_105 | PARENT | () | N_6 |

### 20.7.47 'RELATION( DEFINITIONS_PARENT )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RI_47 | DEFINITIONS_PARENT | NKD_31 | RELATION_INTERPRETATION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("RI corresponding to the RD whose instances identify the parent of the (source) DEFINITIONS node.") |



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_46 | DEFINITION | () | RD_47 |
| S | RD_105 | PARENT | () | N_6 |

### 20.7.48 'RELATION( DEFINITIONS_PRIMARY )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RI_48 | DEFINITIONS_PRIMARY | NKD_31 | RELATION_INTERPRETATION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("RI corresponding to the RD whose instances identify DEFINITIONS nodes.") |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_46 | DEFINITION | () | RD_48 |
| S | RD_82 | KEY_NAME | (KEY) | AD_23 |
| S | RD_105 | PARENT | () | N_6 |

### 20.7.49 'RELATION( DEFINITIONS_SECONDARY )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RI_49 | DEFINITIONS_SECONDARY | NKD_31 | RELATION_INTERPRETATION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("RI corresponding to the RD whose instances identify the DEFINITIONS nodes from the (source) current process node.") |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_46 | DEFINITION | () | RD_49 |
| S | RD_82 | KEY_NAME | (KEY) | AD_23 |
| S | RD_105 | PARENT | () | N_6 |

### 20.7.50 'RELATION( DENY )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RI_50 | DENY | NKD_31 | RELATION_INTERPRETATION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("RI corresponding to the RD whose instances identify the groups which can have no access to the (source) object.") |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_46 | DEFINITION | () | RD_50 |
| S | RD_82 | KEY_NAME | (KEY) | AD_9 |
| S | RD_105 | PARENT | () | N_6 |


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

### 20.7.51 'RELATION( DEVICE_PARENT )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RI_51 | DEVICE_PARENT | NKD_31 | RELATION_INTERPRETATION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("RI corresponding to the RD whose instances identify the parent (system node) of the (source) top-level DEVICE node.") |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_46 | DEFINITION | () | RD_51 |
| S | RD_105 | PARENT | () | N_6 |

### 20.7.52 'RELATION( DEVICE_PRIMARY )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RI_52 | DEVICE_PRIMARY | NKD_31 | RELATION_INTERPRETATION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("RI corresponding to the RD whose instances are primary relationships to the top-level DEVICE nodes.") |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_46 | DEFINITION | () | RD_52 |
| S | RD_82 | KEY_NAME | (KEY) | AD_23 |
| S | RD_105 | PARENT | () | N_6 |

### 20.7.53 'RELATION( DEVICE_SECONDARY )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RI_53 | DEVICE_SECONDARY | NKD_31 | RELATION_INTERPRETATION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("RI corresponding to the RD whose instances provide identification of top-level DEVICE nodes from the (source) current process or RESOURCE_GROUP node.") |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_46 | DEFINITION | () | RD_53 |
| S | RD_82 | KEY_NAME | (KEY) | AD_23 |
| S | RD_105 | PARENT | () | N_6 |

### 20.7.54 'RELATION( EMANATES )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RI_54 | EMANATES | NKD_31 | RELATION_INTERPRETATION |



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("RI corresponding to the RD whose instances identify the emanating relations for the (source) node kind.") |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_21 | ATTRIBUTE_NAME | (MAXIMUM_INSTANCES) | AD_51 |
| S | RD_21 | ATTRIBUTE_NAME | (MINIMUM_INSTANCES) | AD_55 |
| S | RD_46 | DEFINITION | () | RD_54 |
| S | RD_82 | KEY_NAME | (KEY) | AD_23 |
| S | RD_105 | PARENT | () | N_6 |

### 20.7.55 'RELATION( EMANATES_FROM )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RI_55 | EMANATES_FROM | NKD_31 | RELATION_INTERPRETATION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("RI corresponding to the RD whose instances identify the node kinds for the (source) emanating relation.") |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_46 | DEFINITION | () | RD_55 |
| S | RD_82 | KEY_NAME | (KEY) | AD_23 |
| S | RD_105 | PARENT | () | N_6 |

### 20.7.56 'RELATION( EXECUTABLE_IMAGE )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RI_56 | EXECUTABLE_IMAGE | NKD_31 | RELATION_INTERPRETATION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("RI corresponding to the RD whose instances identify the FILE node containing the (source) current process's executable image.") |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_46 | DEFINITION | () | RD_56 |
| S | RD_105 | PARENT | () | N_6 |

### 20.7.57 'RELATION( GATEWAY_PARENT )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RI_57 | GATEWAY_PARENT | NKD_31 | RELATION_INTERPRETATION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("RI corresponding to the RD for relationships which identify the parent (system node) of the (source) top-level GATEWAY node.") |


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_46 | DEFINITION | () | RD_57 |
| S | RD_105 | PARENT | () | N_6 |

### 20.7.58 'RELATION( GATEWAY_PRIMARY )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RI_58 | GATEWAY_PRIMARY | NKD_31 | RELATION_INTERPRETATION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("RI corresponding to the RD whose instances are primary relationships to the top-level GATEWAY nodes.") |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_46 | DEFINITION | () | RD_58 |
| S | RD_82 | KEY_NAME | (KEY) | AD_23 |
| S | RD_105 | PARENT | () | N_6 |

### 20.7.59 'RELATION( GATEWAY_SECONDARY )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RI_59 | GATEWAY_SECONDARY | NKD_31 | RELATION_INTERPRETATION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("RI corresponding to the RD whose instances provide identification of top-level GATEWAY nodes from the (source) RESOURCE_GROUP node.") |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_46 | DEFINITION | () | RD_59 |
| S | RD_82 | KEY_NAME | (KEY) | AD_23 |
| S | RD_105 | PARENT | () | N_6 |

### 20.7.60 'RELATION( GENERALIZATION_OF_NKD )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RI_60 | GENERALIZATION_OF_NKD | NKD_31 | RELATION_INTERPRETATION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("RI corresponding to the RD whose instances identify the generalization NKDs of the (source) NKD.") |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_46 | DEFINITION | () | RD_60 |
| S | RD_82 | KEY_NAME | (KEY) | AD_23 |
| S | RD_105 | PARENT | () | N_6 |



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

### 20.7.61 'RELATION( GENERALIZATION_OF_RD )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RI_61 | GENERALIZATION_OF_RD | NKD_31 | RELATION_INTERPRETATION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("RI corresponding to the RD whose instances identify the generalization RDs of the (source) RD.") |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_46 | DEFINITION | () | RD_61 |
| S | RD_82 | KEY_NAME | (KEY) | AD_23 |
| S | RD_105 | PARENT | () | N_6 |

### 20.7.62 'RELATION( GROUP_PARENT )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RI_62 | GROUP_PARENT | NKD_31 | RELATION_INTERPRETATION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("RI corresponding to the RD whose instances identify the parent (system node) of the (source) top-level GROUP node.") |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_46 | DEFINITION | () | RD_62 |
| S | RD_105 | PARENT | () | N_6 |

### 20.7.63 'RELATION( GROUP_PRIMARY )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RI_63 | GROUP_PRIMARY | NKD_31 | RELATION_INTERPRETATION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("RI corresponding to the RD whose instances are primary relationships to the top-level GROUP nodes.") |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_46 | DEFINITION | () | RD_63 |
| S | RD_82 | KEY_NAME | (KEY) | AD_23 |
| S | RD_105 | PARENT | () | N_6 |

### 20.7.64 'RELATION( GROUP_SECONDARY )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RI_64 | GROUP_SECONDARY | NKD_31 | RELATION_INTERPRETATION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("RI corresponding to the RD whose instances provide identification of top-level GROUP nodes from the (source) current process node.") |


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_46 | DEFINITION | () | RD_64 |
| S | RD_82 | KEY_NAME | (KEY) | AD_23 |
| S | RD_105 | PARENT | () | N_6 |

### 20.7.65 'RELATION( HANDLER_ATTRIBUTE )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RI_65 | HANDLER_ATTRIBUTE | NKD_31 | RELATION_INTERPRETATION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("RI corresponding to the RD whose instances identify the process node attributes to be created when an event handler process is initiated for the (source) attribute monitor.") |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_46 | DEFINITION | () | RD_65 |
| S | RD_21 | ATTRIBUTE_NAME | (ATTRIBUTE_VALUE) | AD_4 |
| S | RD_82 | KEY_NAME | (KEY) | AD_9 |
| S | RD_105 | PARENT | () | N_6 |

### 20.7.66 'RELATION( HANDLER_ENVIRONMENT_NODE )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RI_66 | HANDLER_ENVIRONMENT_NODE | NKD_31 | RELATION_INTERPRETATION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("RI corresponding to the RD whose instances identify the node at which the CURRENT_NODE relationship is to terminate when an event handler process is initiated for the (source) attribute monitor.") |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_46 | DEFINITION | () | RD_66 |
| S | RD_105 | PARENT | () | N_6 |

### 20.7.67 'RELATION( HANDLER_ERROR_FILE )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RI_67 | HANDLER_ERROR_FILE | NKD_31 | RELATION_INTERPRETATION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("RI corresponding to the RD whose instances identify the file at which the STANDARD_ERROR relationship is to terminate when an event handler process is initiated for the (source) attribute monitor.") |



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_46 | DEFINITION | () | RD_67 |
| S | RD_105 | PARENT | () | N_6 |

### 20.7.68 'RELATION( HANDLER_IMAGE )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RI_68 | HANDLER_IMAGE | NKD_31 | RELATION_INTERPRETATION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("RI corresponding to the RD whose instances identify the node at which the EXECUTABLE_IMAGE relationship is to terminate when an event handler process is initiated for the (source) attribute monitor.") |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_46 | DEFINITION | () | RD_68 |
| S | RD_105 | PARENT | () | N_6 |

### 20.7.69 'RELATION( HANDLER_INPUT_FILE )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RI_69 | HANDLER_INPUT_FILE | NKD_31 | RELATION_INTERPRETATION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("RI corresponding to the RD whose instances identify the node at which the STANDARD_INPUT relationship is to terminate when an event handler process is initiated for the (source) attribute monitor.") |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_46 | DEFINITION | () | RD_69 |
| S | RD_105 | PARENT | () | N_6 |

### 20.7.70 'RELATION( HANDLER_MEDIUM_RESOURCE )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RI_70 | HANDLER_MEDIUM_RESOURCE | NKD_31 | RELATION_INTERPRETATION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("RI corresponding to the RD whose instances identify the medium resource on which the process node is to be allocated when an event handler process is initiated for the (source) attribute monitor.") |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_46 | DEFINITION | () | RD_70 |
| S | RD_105 | PARENT | () | N_6 |



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

### 20.7.71 'RELATION( HANDLER_OUTPUT_FILE )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RI_71 | HANDLER_OUTPUT_FILE | NKD_31 | RELATION_INTERPRETATION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("RI corresponding to the RD whose instances identify the node at which the STANDARD_OUTPUT relationship is to terminate when an event handler process is initiated for the (source) attribute monitor.") |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_46 | DEFINITION | () | RD_71 |
| S | RD_105 | PARENT | () | N_6 |

### 20.7.72 'RELATION( HANDLER_PROCESS )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RI_72 | HANDLER_PROCESS | NKD_31 | RELATION_INTERPRETATION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("RI corresponding to the RD whose instances identify the event handler processes for the (source) attribute monitor.") |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_46 | DEFINITION | () | RD_72 |
| S | RD_82 | KEY_NAME | (KEY) | AD_9 |
| S | RD_105 | PARENT | () | N_6 |

### 20.7.73 'RELATION( HANDLER_PROCESS_NODE_KIND )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RI_73 | HANDLER_PROCESS_NODE_-KIND | NKD_31 | RELATION_INTERPRETATION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("RI corresponding to the RD whose instances identify the kind of PROCESS node to be created when an event handler process is initiated for the (source) attribute monitor.") |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_46 | DEFINITION | () | RD_73 |
| S | RD_105 | PARENT | () | N_6 |

### 20.7.74 'RELATION( HANDLER_PROCESS_OF )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RI_74 | HANDLER_PROCESS_OF | NKD_31 | RELATION_INTERPRETATION |



Source: https://assist.dla.mil -- Downloaded: 202610-26T07:54Z
Check the source to verify that this is the current version before use.

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("RI corresponding to the RD whose instances identify the MONITOR nodes with which the (source) PROCESS nodes are associated.") |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_46 | DEFINITION | () | RD_74 |
| S | RD_105 | PARENT | () | N_6 |

### 20.7.75 'RELATION( HANDLER_PROCESSOR_RESOURCE )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RI_75 | HANDLER_PROCESSOR_-RESOURCE | NKD_31 | RELATION_INTERPRETATION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("RI corresponding to the RD whose instances identify the processor resource on which the process is to be allocated when an event handler process is initiated for the (source) attribute monitor.") |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_46 | DEFINITION | () | RD_75 |
| S | RD_105 | PARENT | () | N_6 |

### 20.7.76 'RELATION( INTRODUCED_NODE )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RI_76 | INTRODUCED_NODE | NKD_31 | RELATION_INTERPRETATION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("RI corresponding to the RD whose instances are primary relationships to introduced nodes.") |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_46 | DEFINITION | () | RD_76 |
| S | RD_82 | KEY_NAME | (KEY) | AD_23 |
| S | RD_105 | PARENT | () | N_6 |

### 20.7.77 'RELATION( INTRODUCED_NODE_PARENT )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RI_77 | INTRODUCED_NODE_PARENT | NKD_31 | RELATION_INTERPRETATION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("RI corresponding to the RD whose instances identify the parent of the INTRODUCED_NODE.") |


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_46 | DEFINITION | () | RD_77 |
| S | RD_105 | PARENT | () | N_6 |

### 20.7.78 'RELATION( INTRODUCED_RELATION )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RI_78 | INTRODUCED_RELATION | NKD_31 | RELATION_INTERPRETATION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("RI corresponding to the RD whose instances are the introduced representation of relationships between imported nodes.") |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_21 | ATTRIBUTE_NAME | (IMPORTED_NAME) | AD_34 |
| S | RD_21 | ATTRIBUTE_NAME | (IMPORTED_RELATIONSHIP_KIND) | AD_35 |
| S | RD_21 | ATTRIBUTE_NAME | (IMPORTED_TARGET_UID) | AD_36 |
| S | RD_21 | ATTRIBUTE_NAME | (INTRODUCED_KEY_ATTRIBUTES) | AD_41 |
| S | RD_21 | ATTRIBUTE_NAME | (INTRODUCED_RELATIONSHIP_ATTRIBUTES) | AD_43 |
| S | RD_46 | DEFINITION | () | RD_78 |
| S | RD_82 | KEY_NAME | (KEY) | AD_9 |
| S | RD_105 | PARENT | () | N_6 |

### 20.7.79 'RELATION( JOB )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RI_79 | JOB | NKD_31 | RELATION_INTERPRETATION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("RI corresponding to the RD whose instances identify the top level processes of the (source) current user.") |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_46 | DEFINITION | () | RD_79 |
| S | RD_82 | KEY_NAME | (KEY) | AD_23 |
| S | RD_105 | PARENT | () | N_6 |

### 20.7.80 'RELATION( JOB_PARENT )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RI_80 | JOB_PARENT | NKD_31 | RELATION_INTERPRETATION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("RI corresponding to the RD whose instances identify the top level PROCESS node's parent, the (target) current USER node.") |


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_46 | DEFINITION | () | RD_80 |
| S | RD_105 | PARENT | () | N_6 |

### 20.7.81 'RELATION( KEY )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RI_81 | KEY_NAME | NKD_31 | RELATION_INTERPRETATION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("RI corresponding to the RD whose instances identify the keys for the (source) relation.") |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_46 | DEFINITION | () | RD_81 |
| S | RD_105 | PARENT | () | N_6 |

### 20.7.82 'RELATION( KEY_NAME )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RI_82 | KEY_NAME | NKD_31 | RELATION_INTERPRETATION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("RI corresponding to the RD whose instances specify a relation interpretation-relative name for the (target) key.") |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_46 | DEFINITION | () | RD_82 |
| S | RD_105 | PARENT | () | N_6 |

### 20.7.83 'RELATION( KEY_OF )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RI_83 | KEY_OF | NKD_31 | RELATION_INTERPRETATION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("RI corresponding to the RD whose instances identify the relations utilizing the (source) key.") |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_46 | DEFINITION | () | RD_83 |
| S | RD_105 | PARENT | () | N_6 |

### 20.7.84 'RELATION( MEDIUM_PARENT )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RI_84 | MEDIUM_PARENT | NKD_31 | RELATION_INTERPRETATION |



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("RI corresponding to the RD whose instances identify the parent (system node) of the (source) top-level MEDIUM node.") |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_46 | DEFINITION | () | RD_84 |
| S | RD_105 | PARENT | () | N_6 |

### 20.7.85 'RELATION( MEDIUM_PRIMARY )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RI_85 | MEDIUM_PRIMARY | NKD_31 | RELATION_INTERPRETATION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("RI corresponding to the RD whose instances are primary relationships to the top-level MEDIUM nodes.") |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_46 | DEFINITION | () | RD_85 |
| S | RD_82 | KEY_NAME | (KEY) | AD_23 |
| S | RD_105 | PARENT | () | N_6 |

### 20.7.86 'RELATION( MEDIUM_SECONDARY )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RI_86 | MEDIUM_SECONDARY | NKD_31 | RELATION_INTERPRETATION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("RI corresponding to the RD whose instances provide identification of top-level MEDIUM nodes from the (source) RESOURCE_GROUP node.") |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_46 | DEFINITION | () | RD_86 |
| S | RD_82 | KEY_NAME | (KEY) | AD_23 |
| S | RD_105 | PARENT | () | N_6 |

### 20.7.87 'RELATION( MONITORED_BY )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RI_87 | MONITORED_BY | NKD_31 | RELATION_INTERPRETATION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("RI corresponding to the RD whose instances identify the attribute monitors for the (source) node.") |



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_46 | DEFINITION | () | RD_87 |
| S | RD_82 | KEY_NAME | (KEY) | AD_9 |
| S | RD_105 | PARENT | () | N_6 |

### 20.7.88 'RELATION( MONITORS )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RI_88 | MONITORS | NKD_31 | RELATION_INTERPRETATION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("RI corresponding to the RD whose instances identify the nodes being monitored by the (source) attribute monitor.") |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_21 | ATTRIBUTE_NAME | (ATTRIBUTES_MONITORED) | AD_5 |
| S | RD_46 | DEFINITION | () | RD_88 |
| S | RD_82 | KEY_NAME | (KEY) | AD_9 |
| S | RD_105 | PARENT | () | N_6 |

### 20.7.89 'RELATION( NODE_KIND )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RI_89 | NODE_KIND | NKD_31 | RELATION_INTERPRETATION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("RI corresponding to the RD whose instances are primary relationships providing view-relative names for the (target) NKI.") |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_46 | DEFINITION | () | RD_89 |
| S | RD_82 | KEY_NAME | (NAME) | AD_58 |
| S | RD_105 | PARENT | () | N_6 |

### 20.7.90 'RELATION( NODE_KIND_DEFINITION )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RI_90 | NODE_KIND_DEFINITION | NKD_31 | RELATION_INTERPRETATION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("RI corresponding to the RD whose instances are primary relationships to NKDs.") |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_46 | DEFINITION | () | RD_90 |
| S | RD_82 | KEY_NAME | (KEY) | AD_23 |
| S | RD_105 | PARENT | () | N_6 |



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

### 20.7.91 'RELATION( NODE_KIND_DEFINITION_PARENT )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RI_91 | NODE_KIND_DEFINITION_-PARENT | NKD_31 | RELATION_INTERPRETATION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("RI corresponding to the RD whose instances identify the parent of the (source) NKD.") |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_46 | DEFINITION | () | RD_91 |
| S | RD_105 | PARENT | () | N_6 |

### 20.7.92 'RELATION( NODE_KIND_PARENT )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RI_92 | NODE_KIND_PARENT | NKD_31 | RELATION_INTERPRETATION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("RI corresponding to the RD whose instances identify the parent VIEW node of the (source) NKI.") |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_46 | DEFINITION | () | RD_92 |
| S | RD_105 | PARENT | () | N_6 |

### 20.7.93 'RELATION( PARENT )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RI_93 | PARENT | NKD_31 | RELATION_INTERPRETATION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("RI corresponding to the generalization of relations whose instances identify the parent of the (source) node.") |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_46 | DEFINITION | () | RD_93 |
| S | RD_105 | PARENT | () | N_6 |

### 20.7.94 'RELATION( PERMANENT_MEMBER )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RI_94 | PERMANENT_MEMBER | NKD_31 | RELATION_INTERPRETATION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("RI corresponding to the RD whose instances are primary relationships between GROUP nodes.") |



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_46 | DEFINITION | () | RD_94 |
| S | RD_82 | KEY_NAME | (KEY) | AD_23 |
| S | RD_105 | PARENT | () | N_6 |

### 20.7.95 'RELATION( PERMANENT_MEMBER_PARENT )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RI_95 | PERMANENT_MEMBER_PARENT | NKD_31 | RELATION_INTERPRETATION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("RI corresponding to the RD whose instances are PARENT relationships between GROUP nodes.") |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_46 | DEFINITION | () | RD_95 |
| S | RD_105 | PARENT | () | N_6 |

### 20.7.96 'RELATION( PORT )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RI_96 | PORT | NKD_31 | RELATION_INTERPRETATION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("RI corresponding to the RD whose instances identify the channels of the (source) file or process.") |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_21 | ATTRIBUTE_NAME | (ALLOWED_PROTOCOLS) | AD_1 |
| S | RD_21 | ATTRIBUTE_NAME | (DIRECTION) | AD_19 |
| S | RD_21 | ATTRIBUTE_NAME | (ELEMENT_SIZE) | AD_20 |
| S | RD_21 | ATTRIBUTE_NAME | (INCOMING_FRAME_LENGTH) | AD_38 |
| S | RD_21 | ATTRIBUTE_NAME | (MAXIMUM_LINE_LENGTH) | AD_52 |
| S | RD_21 | ATTRIBUTE_NAME | (MAXIMUM_PAGE_LENGTH) | AD_53 |
| S | RD_21 | ATTRIBUTE_NAME | (OUTGOING_FRAME_LENGTH_RANGE) | AD_65 |
| S | RD_21 | ATTRIBUTE_NAME | (PRIORITY_LEVEL) | AD_70 |
| S | RD_21 | ATTRIBUTE_NAME | (QUEUE_LENGTH) | AD_72 |
| S | RD_46 | DEFINITION | () | RD_96 |
| S | RD_82 | KEY_NAME | (NAME) | AD_58 |
| S | RD_105 | PARENT | () | N_6 |

### 20.7.97 'RELATION( POTENTIAL_MEMBER )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RI_97 | POTENTIAL_MEMBER | NKD_31 | RELATION_INTERPRETATION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("RI corresponding to the RD whose instances identify groups which members of the (source) group may adopt.") |



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_46 | DEFINITION | () | RD_97 |
| S | RD_82 | KEY_NAME | (KEY) | AD_23 |
| S | RD_105 | PARENT | () | N_6 |

### 20.7.98 'RELATION( PROCESSOR_PARENT )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RI_98 | PROCESSOR_PARENT | NKD_31 | RELATION_INTERPRETATION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("RI corresponding to the RD whose instances identify the parent (system node) of the (source) top-level PROCESSOR node.") |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_46 | DEFINITION | () | RD_98 |
| S | RD_105 | PARENT | () | N_6 |

### 20.7.99 'RELATION( PROCESSOR_PRIMARY )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RI_99 | PROCESSOR_PRIMARY | NKD_31 | RELATION_INTERPRETATION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("RI corresponding to the RD whose instances are primary relationships to the top-level PROCESSOR nodes.") |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_46 | DEFINITION | () | RD_99 |
| S | RD_82 | KEY_NAME | (KEY) | AD_23 |
| S | RD_105 | PARENT | () | N_6 |

### 20.7.100 'RELATION( PROCESSOR_SECONDARY )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RI_100 | PROCESSOR_SECONDARY | NKD_31 | RELATION_INTERPRETATION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("RI corresponding to the RD whose instances provide identification of top-level PROCESSOR nodes from the (source) RESOURCE_GROUP node.") |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_46 | DEFINITION | () | RD_100 |
| S | RD_82 | KEY_NAME | (KEY) | AD_23 |
| S | RD_105 | PARENT | () | N_6 |



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

### 20.7.101 'RELATION( RELATION )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RI_101 | RELATION | NKD_31 | RELATION_INTERPRETATION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("RI corresponding to the RD whose instances are primary relationships providing view-relative names for the (target) RI.") |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_46 | DEFINITION | () | RD_101 |
| S | RD_82 | KEY_NAME | (NAME) | AD_58 |
| S | RD_105 | PARENT | () | N_6 |

### 20.7.102 'RELATION( RELATION_DEFINITION )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RI_102 | RELATION_DEFINITION | NKD_31 | RELATION_INTERPRETATION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("RI corresponding to the RD whose instances are primary relationships to RDs.") |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_46 | DEFINITION | () | RD_102 |
| S | RD_82 | KEY_NAME | (KEY) | AD_23 |
| S | RD_105 | PARENT | () | N_6 |

### 20.7.103 'RELATION( RELATION_DEFINITION_PARENT )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RI_103 | RELATION_INTERPRETATION_-PARENT | NKD_31 | RELATION_INTERPRETATION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("RI corresponding to the RD whose instances identify the parent of the (source) RD.") |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_46 | DEFINITION | () | RD_103 |
| S | RD_105 | PARENT | () | N_6 |

### 20.7.104 'RELATION( RELATION_NAME )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RI_104 | RELATION_NAME | NKD_31 | RELATION_INTERPRETATION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("RI corresponding to the RD whose instances specify a (source) NKI-relative name for the relation.") |



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_46 | DEFINITION | () | RD_104 |
| S | RD_82 | KEY_NAME | (NAME) | AD_58 |
| S | RD_105 | PARENT | () | N_6 |

### 20.7.105 'RELATION( RELATION_PARENT )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RI_105 | RELATION_PARENT | NKD_31 | RELATION_INTERPRETATION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("RI corresponding to the RD whose instances identify the parent VIEW node of the (source) RI.") |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_46 | DEFINITION | () | RD_105 |
| S | RD_105 | PARENT | () | N_6 |

### 20.7.106 'RELATION( RESOURCE_GROUP_PARENT )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RI_106 | RESOURCE_GROUP_PARENT | NKD_31 | RELATION_INTERPRETATION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("RI corresponding to the RD whose instances identify the parent (system node) of the (source) top-level RESOURCE_GROUP node.") |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_46 | DEFINITION | () | RD_106 |
| S | RD_105 | PARENT | () | N_6 |

### 20.7.107 'RELATION( RESOURCE_GROUP_PRIMARY )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RI_107 | RESOURCE_GROUP_PRIMARY | NKD_31 | RELATION_INTERPRETATION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("RI corresponding to the RD whose instances are primary relationships to the top-level RESOURCE_GROUP nodes.") |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_46 | DEFINITION | () | RD_107 |
| S | RD_82 | KEY_NAME | (KEY) | AD_23 |
| S | RD_105 | PARENT | () | N_6 |


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

### 20.7.108 'RELATION( RESOURCE_GROUP_SECONDARY )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RI_108 | RESOURCE_GROUP_-SECONDARY | NKD_31 | RELATION_INTERPRETATION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("RI corresponding to the RD whose instances provide identification of top-level RESOURCE_GROUP nodes from the (source) current process node.") |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_46 | DEFINITION | () | RD_108 |
| S | RD_82 | KEY_NAME | (KEY) | AD_23 |
| S | RD_105 | PARENT | () | N_6 |

### 20.7.109 'RELATION( REVERSE_LINK )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RI_109 | REVERSE_LINK | NKD_31 | RELATION_INTERPRETATION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("RI corresponding to the RD whose instances identify the reverse RD for the (source and target) RD when a bidirectional RD is comprised of a single relation.") |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_46 | DEFINITION | () | RD_109 |
| S | RD_105 | PARENT | () | N_6 |

### 20.7.110 'RELATION( REVERSE_LINK_BIDIRECTIONAL )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RI_110 | REVERSE_LINK_BIDIRECTIONAL | NKD_31 | RELATION_INTERPRETATION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("RI corresponding to the RD whose instances identify the reverse RD for the (source) RD when a bidirectional RD is comprised of two distinct relations.") |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_46 | DEFINITION | () | RD_110 |
| S | RD_105 | PARENT | () | N_6 |

### 20.7.111 'RELATION( SPECIALIZATION_OF_NKD )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RI_111 | SPECIALIZATION_OF_NKD | NKD_31 | RELATION_INTERPRETATION |



Source: https://assist.dla.mil -- Downloaded: 202610-26T07:54Z
Check the source to verify that this is the currentversion before use.

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("RI corresponding to the RD whose instances identify the specialisation NKDs of the (source) NKD.") |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_46 | DEFINITION | () | RD_111 |
| S | RD_82 | KEY_NAME | (KEY) | AD_23 |
| S | RD_105 | PARENT | () | N_6 |

### 20.7.112 'RELATION( SPECIALIZATION_OF_RD )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RI_112 | SPECIALIZATION_OF_RD | NKD_31 | RELATION_INTERPRETATION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("RI corresponding to the RD whose instances identify the specialisation RDs of the (source) RD.") |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_46 | DEFINITION | () | RD_112 |
| S | RD_82 | KEY_NAME | (KEY) | AD_23 |
| S | RD_105 | PARENT | () | N_6 |

### 20.7.113 'RELATION( STANDARD_ERROR )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RI_113 | STANDARD_ERROR | NKD_31 | RELATION_INTERPRETATION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("RI corresponding to the RD whose instances identify the intended standard error file for the (source) process.") |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_46 | DEFINITION | () | RD_113 |
| S | RD_105 | PARENT | () | N_6 |

### 20.7.114 'RELATION( STANDARD_INPUT )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RI_114 | STANDARD_INPUT | NKD_31 | RELATION_INTERPRETATION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("RI corresponding to the RD whose instances identify the intended standard input file for the (source) process.") |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_46 | DEFINITION | () | RD_114 |
| S | RD_105 | PARENT | () | N_6 |


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

### 20.7.115 'RELATION( STANDARD_OUTPUT )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RI_115 | STANDARD_OUTPUT | NKD_31 | RELATION_INTERPRETATION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("RI corresponding to the RD whose instances identify the intended standard output file for the (source) process.") |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_46 | DEFINITION | () | RD_115 |
| S | RD_105 | PARENT | () | N_6 |

### 20.7.116 'RELATION( TERMINATES )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RI_116 | TERMINATES | NKD_31 | RELATION_INTERPRETATION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("RI corresponding to the RD whose instances identify the terminating relations for the (source) node kind.") |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_46 | DEFINITION | () | RD_116 |
| S | RD_82 | KEY_NAME | (KEY) | AD_23 |
| S | RD_105 | PARENT | () | N_6 |

### 20.7.117 'RELATION( TERMINATES_AT )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RI_117 | TERMINATES_AT | NKD_31 | RELATION_INTERPRETATION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("RI corresponding to the RD whose instances identify the node kinds for the (source) terminating relation.") |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_46 | DEFINITION | () | RD_117 |
| S | RD_82 | KEY_NAME | (KEY) | AD_23 |
| S | RD_105 | PARENT | () | N_6 |

### 20.7.118 'RELATION( USER_PARENT )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RI_118 | USER_PARENT | NKD_31 | RELATION_INTERPRETATION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("RI corresponding to the RD whose instances identify the parent (system node) of the (source) top-level USER node.") |



Source: https://assist.dla.mil -- Downloaded: 202610-26T07:54Z
Check the source to verify that this is the currentversion before use.

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_46 | DEFINITION | () | RD_118 |
| S | RD_105 | PARENT | () | N_6 |

### 20.7.119 'RELATION( USER_PRIMARY )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RI_119 | USER_PRIMARY | NKD_31 | RELATION_INTERPRETATION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("RI corresponding to the RD whose instances are primary relationships to the top-level USER nodes.") |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_46 | DEFINITION | () | RD_119 |
| S | RD_82 | KEY_NAME | (KEY) | AD_23 |
| S | RD_105 | PARENT | () | N_6 |

### 20.7.120 'RELATION( USER_SECONDARY )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RI_120 | USER_SECONDARY | NKD_31 | RELATION_INTERPRETATION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("RI corresponding to the RD whose instances provide identification of top-level USER nodes from the (source) current process node.") |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_46 | DEFINITION | () | RD_120 |
| S | RD_82 | KEY_NAME | (KEY) | AD_23 |
| S | RD_105 | PARENT | () | N_6 |

### 20.7.121 'RELATION( VIEW_PARENT )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RI_121 | VIEW_PARENT | NKD_31 | RELATION_INTERPRETATION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("RI corresponding to the RD whose instances identify the parent of the (source) VIEW node.") |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_46 | DEFINITION | () | RD_121 |
| S | RD_105 | PARENT | () | N_6 |



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

### 20.7.122 'RELATION( VIEW_PRIMARY )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RI_122 | VIEW_PRIMARY | NKD_31 | RELATION_INTERPRETATION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("RI corresponding to the RD whose instances are primary relationships to VIEW nodes.") |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_46 | DEFINITION | () | RD_122 |
| S | RD_82 | KEY_NAME | (KEY) | AD_23 |
| S | RD_105 | PARENT | () | N_6 |

### 20.7.123 'RELATION( VIEW_SECONDARY )

| Node | Node Identification | NKD | NKD Identification |
|---|---|---|---|
| RI_123 | VIEW_SECONDARY | NKD_31 | RELATION_INTERPRETATION |

| Attribute Name | AD | Attribute Value |
|---|---|---|
| ANNOTATION | AD_3 | ("RI corresponding to the RD whose instances identify views.") |

| C | RD/AD | Relation/Attribute Name | Key/Attribute Value | Target |
|---|---|---|---|---|
| S | RD_46 | DEFINITION | () | RD_123 |
| S | RD_82 | KEY_NAME | (KEY) | AD_23 |
| S | RD_105 | PARENT | () | N_6 |



Source: https://assist.dla.mil -- Downloaded: 202610-26T07:54Z
Check the source to verify that this is the current version before use.

# B. CAIS Specification

The material contained in this appendix is a mandatory part of the standard. 1

This appendix contains a set of Ada package specifications of the CAIS interfaces in their canonical form (see Section 4.2 on page 37). Although the interfaces are not necessarily shown here in the order in which they are discussed in the text, this appendix provides a reference listing of the CAIS. 2

The following two library packages must be compiled after the bodies of packages CAIS_ENUMERATION_ITEM and CAIS_ENUMERATION_ATTRIBUTE have been compiled, respectively: 3

```
with CAIS_LIST_MANAGEMENT;
use  CAIS_LIST_MANAGEMENT;

package  CAIS_BOOLEAN_ITEM  is
     new CAIS_ENUMERATION_ITEM (ENUMERATION => BOOLEAN);


with   CAIS_ATTRIBUTE_MANAGEMENT;
use CAIS_ATTRIBUTE_MANAGEMENT;

package  CAIS_BOOLEAN_ATTRIBUTE  is
     now CAIS_ENUMERATION_ATTRIBUTE (ENUMERATION => BOOLEAN);
```

The following are the remaining package specifications in compilation order:

```
package CAIS_PRAGMATICS is

        CAPACITY_ERROR : exception;
        RESOURCE_ERROR : exception;

        UNRESTRICTED : constant : =  implementation_defined;

        CAIS_IDENTIFIER_LENGTH : constant := 80;
        IDENTIFIER_LENGTH        : constant  :=  implementation_defined;

        CAIS_PATHNAME_LENGTH : constant  := 266;
        PATHNAME_LENGTH          : constant  :=  implementation_defined;

        CAIS_ADOPTED_VIEWS_OF_PROCESS  :  constant  :=  266;
        ADOPTED_VIEWS_OF_PROCESS :  constant  :=  implementation_defined;

        CAIS_UNIQUE_IDENTIFIER_PREFIX_LENGTH : constant := 80;
        CAIS_UNIQUE_IDENTIFIER_SUFFIX_LENGTH : constant := 80;

        CAIS_UNIQUE_IDENTIFIERS_PER_INSTANCE : constant := 2 ** 63 - 1;

        CAIS_NUMBER_OF_NODES : constant := UNRESTRICTED;

        CAIS_LENGTH_OF_PRIMARY_PATH : constant := PATHNAME_LENGTH/2;
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
CAIS_EMANATING_PRIMARY_RELATIONSHIPS_PER_NODE : constant := 2 ** 10 - 1;
ENAMATING_PRIMARY_RELATIONSHIPS_PER_NODE : constant :=
    implementation_defined;

CAIS_EMANATING_SECONDARY_RELATIONSHIPS_PER_NODE : constant := 2 ** 10 - 1;
EMANATING_SECONDARY_RELATIONSHIPS_PER_NODE : constant :=
    implementation_defined;

CAIS_ATTRIBUTES_PER_NODE : constant := 266;
ATTRIBUTES_PER_NODE : constant := implementation_defined;

CAIS_ATTRIBUTES_PER_RELATIONSHIP : constant := 266;
ATTRIBUTES_PER_RELATIONSHIP : constant := implementation_defined;

CAIS_NODES_IN_COPY_TREE : constant := 2 ** 16 - 1;
NODES_IN_COPY_TREE : constant := implementation_defined;

CAIS_NODES_IN_DELETE_TREE : constant := 2 ** 16 - 1;
NODES_IN_DELETE_TREE : constant := implementation_defined;

CAIS_NODE_HANDLES_PER_PROCESS : constant := 266;
NODE_HANDLES_PER_PROCESS : constant := implementation_defined;

CAIS_FILE_HANDLES_PER_PROCESS : constant := 16;
FILE_HANDLES_PER_PROCESS : constant := implementation_defined;

CAIS_TRANSACTION_HANDLES_PER_PROCESS : constant := 266;
TRANSACTION_HANDLES_PER_PROCESS : constant := implementation_defined

CAIS_ELEMENTS_OF_NODE_ITERATOR : constant := 2 ** 11 - 2;
ELEMENTS_OF_NODE_ITERATOR : constant := implementation_defined;

CAIS_ELEMENTS_OF_NODE_KIND_INSTANCE_ITERATOR : constant := UNRESTRICTED;
ELEMENTS_OF_NODE_KIND_INSTANCE_ITERATOR : constant := UNRESTRICTED;

CAIS_ELEMENTS_OF_ATTRIBUTE_ITERATOR : constant := 266;
ELEMENTS_OF_ATTRIBUTE_ITERATOR : constant := implementation_defined

CAIS_MINIMUM_INTEGER : constant := - (2 ** 31 - 1);
MINIMUM_INTEGER : constant := implementation_defined;

CAIS_MAXIMUM_INTEGER : constant := 2 ** 31 - 1;
MAXIMUM_INTEGER : constant := implementation_defined;

CAIS_SMALL_FOR_CAIS_DURATION : constant := 0.015625; -- 1/64
SMALL_FOR_CAIS_DURATION : constant := implementation_defined;

CAIS_GROUP_NODES : constant := 266;
GROUP_NODES : constant := implementation_defined;

CAIS_ACCESS_RELATIONSHIPS_OF_OBJECT : constant := 266;
ACCESS_RELATIONSHIPS_OF_OBJECT : constant := implementation_defined;
```


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
CAIS_GRANT_ITEMS_ON_GRANT_ATTRIBUTE : constant := 15;
GRANT_ITEMS_ON_GRANT_ATTRIBUTE          : constant := implementation_defined;

CAIS_ADOPTED_ROLES_OF_PROCESS : constant := 7;
ADOPTED_ROLES_OF_PROCESS        : constant := implementation_defined;

FILE_STORAGE_UNIT_SIZE : constant := implementation_defined;

MEMORY_STORAGE_UNIT_SIZE : constant := implementation_defined;

CAIS_DIRECT_IO_ELEMENT_SIZE : constant := 2 ** 15 - 1;
DIRECT_IO_ELEMENT_SIZE        : constant := implementation_defined;

CAIS_DIRECT_IO_INDEX_RANGE_UPPER_BOUND : constant := 2 ** 15 - 1;
DIRECT_IO_INDEX_RANGE_UPPER_BOUND         : constant := implementation_defined;

CAIS_SEQUENTIAL_IO_ELEMENT_SIZE : constant := 2 ** 15 - 1;
SEQUENTIAL_IO_ELEMENT_SIZE         : constant := implementation_defined;

CAIS_DIRECT_IO_RECORD_SIZE       : constant := CAIS_DIRECT_IO_ELEMENT_SIZE;
DIRECT_IO_RECORD_SIZE            : constant := DIRECT_IO_ELEMENT_SIZE;

CAIS_SEQUENTIAL_IO_RECORD_SIZE : constant := CAIS_SEQUENTIAL_IO_ELEMENT_SIZE;
SEQUENTIAL_IO_RECORD_SIZE         : constant := SEQUENTIAL_IO_ELEMENT_SIZE;

CAIS_SEQUENTIAL_IO_FILE_SIZE : constant := 2 ** 15 - 1;
SEQUENTIAL_IO_FILE_SIZE         : constant := implementation_defined;

CAIS_TEXT_IO_LINES_PER_FILE : constant := 2 ** 15 - 1;
TEXT_IO_LINES_PER_FILE        : constant := implementation_defined;

CAIS_TEXT_IO_LINES_PER_PAGE : constant := 2 ** 15 - 1;
TEXT_IO_LINES_PER_PAGE        : constant := implementation_defined;

CAIS_TEXT_IO_COLUMNS_PER_LINE : constant := 266;
TEXT_IO_COLUMNS_PER_LINE        : constant := implementation_defined;

CAIS_MAXIMUM_FRAME_LENGTH : constant := 2 ** 15 - 1;
MAXIMUM_FRAME_LENGTH : constant := implementation_defined;

CAIS_MAXIMUM_FRAME_PRIORITY   : constant := 32;
MAXIMUM_FRAME_PRIORITY        : constant := implementation_defined;

CAIS_MINIMUM_TAPE_BLOCK_LENGTH : constant := 18;
MINIMUM_TAPE_BLOCK_LENGTH       : constant := implementation_defined;

CAIS_MAXIMUM_TAPE_BLOCK_LENGTH : constant := 2048;
MAXIMUM_TAPE_BLOCK_LENGTH       : constant := implementation_defined;

CAIS_MAXIMUM_PHYSICAL_DISPLAY_DIMENSION : constant := 132;
MAXIMUM_PHYSICAL_DISPLAYED_DIMENSION          : constant := implementation_defined;

CAIS_MAXIMUM_SCREEN_DIMENSION : constant := 132;
MAXIMUM_SCREEN_DIMENSION        : constant := implementation_defined;
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
        CAIS_LIST_LENGTH : constant := 266;
        LIST_LENGTH          : constant  :=  implementation_defined;

        CAIS_LIST_TEXT_LENGTH : constant := 2 ** 10;
        LIST_TEXT_LENGTH          : constant  :=  implementation_defined;

        CAIS_LIST_MAXIMUM_DIGITS : constant := 6;
        LIST_MAXIMUM_DIGITS          : constant  :=  implementation_defined;


        CAIS_IDENTIFIER_ITEM_LENGTH : constant := CAIS_IDENTIFIER_LEMGTH;
        IDENTIFIER_ITEM_LENGTH          : constant  :=  implementation_defined;

        CAIS_STRING_ITEM_LENGTH : constant := CAIS_PATHNAME_LENGTH;
        STRING_ITEM_LENGTH          : constant  :=  implementation_defined;

end CAIS_PRAGMATICS;
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
with CAIS_PRAGMATICS;

package CAIS_STANDARD is

    type CAIS_INTEGER is range
        CAIS_PRAGMATICS_MINIMUM_INTEGER . . CAIS_PRAGMATICS_MAXIMUM.INTEGER;
    subtype CAIS_NATURAL  is CAIS_INTEGER range O..CAIS_INTEGER'LAST;
    subtype CAIS_POSITIVE is CAIS_INTEGER range l..CAIS_INTEGER'LAST;

    type CAIS_DURATION is delta impplementation_defined;
    for CAIS_DURATION'SMALL use CAIS_PRAGMATICS.SMALL_FOR_CAIS_DURATION;

end CAIS_STANDARD;
```


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
with CAIS_STANDARD;

package CAIS_STATUS_DEFINITION is

    use CAIS_STANDARD;

        type STATUS_INDICATOR is limited private;

        NO_INDICATION : constant STATUS_INDICATOR;

        type EXCEPTION_INDICATION_KIND is
            (

                -- No exception raised.

            NO_ERROR_DETECTED,

                -- For user_defined exceptions:

            USER_DEFINED_ERROR_DETECTED,

                -- For exceptions declared in CAIS_DEFINITIONS:

            ACCESS_VIOLATION_DETECTED,
            AMBIGUOUS_VIEW_ERROR_DETECTED,
            ATTRIBUTE_ERROR_DETECTED,
            CONFORMANCE_ERROR_DETECTED,
            DEFINITION_ERROR_DETECTED,
            DEVICE_ERROR_DETECTED,
            EXISTING_NODE_ERROR_DETECTED,
            INTENT_VIOLATION_DETECTED,
            ITERATOR_ERROR_DETECTED,
            LOCK_ERROR_DETECTED,
            NAME_ERROR_DETECTED,
            NODE_KIND_ERROR_DETECTED,
            NOT_IN_VIEW_ERROR_DETECTED,
            PATHNAME_SYNTAX_ERROR_DETECTED,
            PREDEFINED_ATTRIBUTE_ERROR_DETECTED,
            PREDEFINED_RELATION_ERROR_DETECTED,
            RELATIONSHIP_ERROR_DETECTED,
            RESOURCE_ALLOCATION_ERROR_DETECTED,
            RESOURCE_AVAILABILITY_ERROR_DETECTED,
            SECURITY_VIOLATION_DETECTED,
            STATUS_ERROR_DETECTED,
            SYNTAX_ERROR-DETECTED,
            TRANSACTION-ERROR-DETECTED,
            USE-ERROR-DETECTED,

                -- For exceptions declared in CAIS_PROCESS_DEFINITIONS:

            EXECUTABLE-IMAGE-ERROR-DETECTED,

                -- For exceptions declared in CAIS_IO_DEFINITIONS:
```


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
    CLIENT_ERROR_DETECTED,
    DATA_ERROR_DETECTED,
    END_ERROR_DETECTED,
    LAYOUT_ERROR_DETECTED,
    MODE_ERROR_DETECTED,
    OPEN_DENIED_DETECTED,
    PORT_ERROR_DETECTED,
    PROTOCOL_ERROR_DETECTED,
    TIME_EXPIRED_DETECTED,

        -- For exceptions declared in CAIS_FRAME_IO:

    QUEUE_FULL_DETECTED,
    QUEUE_EMPTY_DETECTED,

        -- For exceptions declared in CAIS_MAGNETIC_TAPE_IO:

    TAPE_STATUS_ERROR_DETECTED,

        -- For exceptions declared in CAIS_TERMINAL_IO:

    BUTTON_NAME_ERROR_DETECTED,
    FUNCTION_KEY_STATUS_ERROR_DETECTED,
    INPUT_EVENT_QUEUE_EMPTY_DETECTED,
    LOCATOR_STATUS_ERROR_DETECTED,
    SCREEN_NOT_ON_DISPLAY_DETECTED,
    SCREEN_POSITION_ERROR_DETECTED,
    VIRTUAL_DISPLAY_POSITION_ERROR_DETECTED,

        -- For exceptions declared in CAIS_COMMON_IO:

    UID_ERROR_DETECTED,

        -- For exceptions declared in CAIS_LIST_MANAGEMENT:

    ITEM_KIND_ERROR_DETECTED,
    LIST_KIND_ERROR_DETECTED,
    LIST_POSITION_ERR_DETECTED,
    LIST_SYNTAX_ERROR_DETECTED,
    NAMED_LIST_ERROR_DETECTED,
    SEARCH_ERROR_DETECTED,
    TOXEN_ERROR_DETECTED,

        -- For exceptions declared in CAIS_CALENDAR:

    TIME_ERROR_DETECTED,

        -- For exceptions declared in CAIS_PRAGMATICS:

    CAPACITY_ERROR_DETECTED,
    RESOURCE_ERROR_DETECTED,
```


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
        -- For exceptions declared in CAIS_FORM_TERMINAL_IO:

        FORM_STATUS_ERROR_DETECTED,
        FORM_FUNCTION_KEY_STATUS_ERROR_DETECTED,
        TERMINAL_POSITION_ERROR_DETCCTED);

    subtype MESSAGE_TEXT is STRING;

    subtype CONDITION_CODE is CAIS_INTEGER;

    -- Condition codes for ACCESS_VIOLATION:

    ACC_INSUFFICIENT_RIGHTS                 : constant CONDITION_CODE := 1;
    ACC_ACCESS_RESTRICTION                  : constant CONDITION_CODE := 2;
    ACC_DENY_RESTRICTION                    : constant CONDITION_CODE := 3;
    ACC_INSUFFICIENT_RIGHTS_PREDICATE       : constant CONDITION_CODE := 4;
    ACC_INSUFFICIENT_DEFAULT_RIGHTS         : constant CONDITION_CODE := 6;

    -- Condition codes for AMBIGUOUS_VIEW_ERROR:

    AMB_NAME_TRANSLATION                    : constant CONDITION_CODE := 1;
    AMB_NAME_FORMATION                      : constant CONDITION_CODE := 2;
    AMB_NAME_CREATION                       : constant CONDITION_CODE := 3;
    AMB_KEY_VALUE_ASSOCIATION               : constant CONDITION_CODE := 4;

    -- Condition codes for ATTRIBUTE_ERROR:

    ATT_NODE_ATTRIBUTE_EXISTS               : constant CONDITION_CODE := 1;
    ATT_RELATIONSHIP_ATTRIBUTE_EXISTS       : constant CONDITION_CODE := 2;
    ATT_NONEXISTENT_NODE_ATTRIBUTE          : constant CONDITION_CODE := 3;
    ATT_NONEXISTENT_RELATIONSHIP_ATTRIBUTE  : constant CONDITION_CODE := 4;
    ATT_INVALID_ATTRIBUTE_REFERENCE         : constant CONDITION_CODE := 5;
    ATT_ILLEGAL_ATTRIBUTES_LIST             : constant CONDITION_CODE := 6;

    -- Condition codes for COMFORMANCE_ERROR:

    CON_RELATIONSHIP_CANNOT_EMANATE         : constant CONDITION_CODE := 1;
    CON_RELATIONSHIP_CANNOT_TERMINATE       : constant CONDITION_CODE := 2;
    CON_EMANATES_BOUND_VIOLATION            : constant CONDITION_CODE := 3;
    CON_UNDEFINED_ATTRIBUTE                 : constant CONDITION_CODB := 4;
    CON_ATTRIBUTE_NOT_MODIFIABLE            : constant CONDITION_CODE := 5;
    CON_ATTRIBUTE_NOT_DELETABLE             : constant CONDITION_CODE := 6;
    CON_UNDEFINED_COMPONENT                 : constant CONDITION_CODE := 7;
    CON_MISSING_COMPOMENT                   : constant CONDITION_CODE := 8;
    CON_NO_ALTERNATIVE_MATCHED              : constant CONDITION_CODE := 9;
    CON_CONSTRAINT_VIOLATION                : constant CONDITION_CODE := 10;
    CON_RELATION_USAGE_VIOLATION            : constant CONDITION_CODE := 11;

    -- Condition codes for DEFINITION_ERROR:

    DEF_DEFINITION_NOT_DELETABLE            : constant CONDITION_CODE := 1;
    DEF_PROPERTY_NOT_DELETABLE              : constant CONDITION_CODE := 2;
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
DEF_CONFLICTING_CONSTRAINTS          : constant CONDITION_CODE := 3;
DEF_ILLEGAL_BASE_TYPE                : constant CONDITION_CODE := 4;
DEF_UNNAMED_UNORDERED_AKD            : constant CONDITION_CODE := 5;
DEF_LAST_INTERPRETATION              : constant CONDITION_CODE := 6;
DEF_LAST_DEFINITION_NAMING           : constant CONDITION_CODE := 7;
DEF_LAST_PROPERTY_NAMING             : constant CONDITION_CODE := 8;
DEF_CAIS_CONTROLLED_COMSTRAIHTS      : constant CONDITION_CODE := 9;
DEF_CAIS_CONTROLLED_SUBLATTICE       : constant CONDITION_CODE := 10;

-- Condition codes for DEVICE_ERROR are implementation defined.

-- Condition cod. for EXISTING_NODE_ERROR:

EXI_NODE_ALREADY_EXISTS              : constant CONDITION_CODE := 1;

-- Condition code for INTENT_VIOLATION:

INT_INSUFFICIENT_RIGHTS              : constant CONDITION_CODE := 1;

-- Condition codes for ITERATOR_ERROR:

ITE_ITERATOR_NOT_ESTABLISHED         : constant CONDITION_CODE := 1;
ITE_ITERATOR_ESTABLISHED             : constant CONDITION_CODE := 2;
ITE_ITERATOR_EXHAUSTED               : constant CONDITION_CODE := 3;

-- Condition codes for LOCK_ERROR:

LOE_INTENT_CONFLICT                  : constant CONDITION_CODE := 1;
LOE_INTENT_CONFLICT_PREDICATE        : constant CONDITION_CODE := 2;
LOE_TRANSACTION_AFFECTS_DEFINITION   : constant CONDITION_CODE := 3;

-- Condition codes for NAME_ERROR:

NAM_INVALID_ACCESS                   : constant CONDITION_CODE := 1;
NAM_INVALID_UID_OPEN                 : constant CONDITION_CODE := 2;
NAM_INVALID_ACCESS_PREDICATE         : constant CONDITION_CODE := 3;
NAM_NO_NODE_WITH_UID                 : constant CONDITION_CODE := 4;

-- Condition codes for NODE_KIND_ERROR:

NKE_INAPPROPRIATE_INSTANCE           : constant CONDITION_CODE := 1;
NKE_INAPPROPRIATE_NKD                : constant CONDITION_CODE := 2;
NKE_INAPPROPRIATE_RD                 : constant CONDITION_CODE := 3;
NKE_NKD_CAIS_CONTROLLED              : constant CONDITION_CODE := 4;
NKE_NODE_CAIS_CONTROLLED             : constant CONDITION_CODE := 5;
NKE_ILLEGAL_CONVERSION               : constant CONDITIOIN_CODE := 6;

-- Condition codes for NOT_IN_VIEW_ERROR:

NIV_PROPERTY_NAME_TRANSLATION        : constant CONDITION_CODE := 1;
NIV_DEFINITION_NAME_TRANSLATION      : constant CONDITION_CODE := 2;
NIV_NAME_FORMATION                   : constant CONDITION_CODE := 3;
NIV_VACUOUS_PATTERN                  : constant CONDITION_CODE := 4;
NIV_KEY_VALUE_ASSOCIATION            : constant CONDITION_CODE := 5;
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
-- Condition codes for PATHNAME_SYNTAX_ERROR:

PSY_PATHNAME                          : constant CONDITION_CODE := i;
PSY_RELATIONSHIP_KEY                  : constant CONDITION_CODE := 2;
PSY_RELATION_NAME                     : constant CONDITION_CODE := 3;

-- Condition codes for PREDEFINED_ATTRIBUTE_ERROR:

PAE_CAIS_CONTROLLED_NODE_ATTRIBUTE    : constant CONDITION_CODE := 1;
PAE_CAIS_CONTROLLED_RELATIOISEIP_ATTRIBUTE
                                      : constant CONDITION_CODE := 2;
PAE_CAIS_CONTROLLED_AD                : constant CONDITION_CODE := 3;

-- Condition codes for PREDEFINED_RELATION_ERROR:

PRE_RELATIOM_CAIS_CONTROLLED          : constant CONDITION_CODE := 1;
PRE_INHERITANCE_NOT_CHANGEABLE        : constant CONDITION_CODE := 2;
PRE_ILLEGAL_CONVERSION                : constant CONDITION_CODE := 3;
PRE_NOT_JOB_RELATIONSHIP              : constant CONDITION_CODE := 4;
PRE_RD_CAIS_CONTROLLED                : constant CONDITION_CODE := 6;

-- Condition codes for RELATIONSHIP_ERROR:

RER_IDENTIFICATION_NOT_UNIQUE         : constant CONDITION_CODE := 1;
RER_NO_SUCH_RELATIONSHIP              : constant CONDITION_CODE := 2;
RER_RELATIONSHIP_ALREADY_EXISTS       : constant CONDITION_CODE := 3;
RER_NODES_NOT_RELATED                 : constant CONDITION_CODE := 4;
RER_ILLEGAL_KEY_LIST                  : constant CONDITION_CODE := 5;
RER_NO_UNIQUE_KEY_SET                 : constant CONDITION_CODE := 6;

-- Condition codes for RESOURCE_ALLOCATION_ERROR:

RAL_RESOURCE_NOT_ALLOCATABLE          : constant CONDITION_CODE := 1;
RAL_INVALID_PROCESS_NODE_MEDIUM       : constant CONDITION_CODE := 2;

-- Condition cod. for RESOURCE_AVAILABILITY_ERROR:

RAV_RESOURCE_NOT_AVAILABLE            : constant CONDITION_CODE := 1;

-- Condition codes for STATUS_ERROR:

STA_ALREADY_OPEN                      : constant CONDITION_CODE := 1;
STA_NOT_OPEN                          : constant CONDITION_CODB := 2;
STA_NOT_ESTABLISHED                   : constant CONDITION_CODE := 3;
STA_ALREADY_ESTABLISHED               : constant CONDITION_CODE := 4;
STA_TRANSACTION_ALREADY_OPEN          : constant CONDITION_CODE := 5;
STA_TRANSACTION_NOT_OPEN              : constant CONDITION_CODE := 6;
STA_FILE_ALREADY_OPEN                 : constant CONDITION_CODB := 7;
STA_FILE_NOT_OPEN                     : constant CONDITION_CODE := 8;
STA_UNEXPECTED_CHANNEL_STATE          : constant CONDITION_CODE := 9;
STA_SCREEN_ALREADY_OPEN               : constant CONDITION_CODE := 10;
STA_SCREEN_NOT_OPEN                   : constant CONDITION_CODE := 11;
STA_NODE_SET_ALREADY_OPEN             : constant CONDITION_CODE := 12;
STA_NODE_SET_NOT_OPEN                 : constant CONDITION_CODE := 13;
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
-- Condition codes for SYNTAX_ERROR:

SYN_ILLEGAL_UID                        : constant CONDITION_CODE := 1;
SYN_ILLEGAL_RELATION_NAME_PATTERN      : constant CONDITION_CODE := 2;
STN_ILLEGAL_RELATIONSHIP_KET_PATTERN   : constant CONDITION_CODE := 3;
SYN_ILLEGAL_PREDICATE_SYNTAX           : constant CONDITION_CODE := 4;
SYN_ILLEGAL_PREDICATICATE              : constant CONDITION_CODE := 5;
SYN_ILLEGAL_ATTRIBUTE_NAME_PATTERN     : constant CONDITION_CODE := 6;
SYN_ILLEGAL_ACCESS_RIGHT_IDENTIFIER    : constant CONDITION_CODE := 7;
SYN_ILLEGAL_ATTRIBUTES_LIST            : constant CONDITION_CODE := 8;
SYN_ILLEGAL_MANDATORY_ACCESS           : constant CONDITION_CODE := 9;
SYN_ILLEGAL_DISCRETIONARY_ACCESS       : constant CONDITION_CODE := 10;
SYN_ILLEGAL_ATTRIBUTE_NAME             : constant CONTDITION_CODE := 11;
SYN_ILLEGAL_RELATION_NAME              : constant CONDITION_CODE := 12;
SYN_ILLEGAL_PARAMETER_LIST             : constant CONDITION_CODE := 13;
SYN_ILLEGAL_DEFINITION_NAME            : constant CONDITION_CODE := 14;
SYN_ILLEGAL_EXTERNAL_NODE_SET          : constant CONDITION_CODE := 15;

-- Condition codes for TRANSACTION_ERROR:

TRN_AMBIGUOUS_INCLUSION                : constant CONDITION_CODE := 1;
TRN_NODE_NOT_INCLUDED                  : constant CONDITION_CODE := 2;
TRN_FILE_NOT_INCLUDED                  : constant CONDITION_CODE := 3;

-- Condition codes for USE_ERROR:

USE_VACUOUS_INTENT                     : constant CONDITION_CODE := 1;
USE_ILLEGAL_CHANGE_INTENT              : constant CONDITION_CODE := 2;
USE_NO_PATH_ASSOCIATED_WITH_HANDLE     : constant CONDITION_CODE := 3;
USE_NO_POSSIBLE_BASE_PATH              : constant CONDITION_CODE := 4;
USE_ILLEGAL_PRIMARY                    : constant CONDITION_CODE := 5;
USE_ILLEGAL_TRANSACTION                : constant CONDITION_CODE := 6;
USE_HAS_EMANATING_PRIMARIES            : constant CONDITION_CODE := 7;
USE_NOT_SECONDARY_RELATIONSHIP         : constant CONDITION_CODE := 8;
USE_ILLEGAL_CONTENTS_REPLACEMENT       : constant CONDITION_CODE := 9;
USE_ILLEGAL_MANDATORY_ACCESS           : constant CONDITIN_CODE := 10;
USE_RELATION_IS_NOT_BIDIRECTIONAL      : constant CONDITION_CODE := 11;
USE_RELATION_IS_BIDIRECTIONAL          : constant CONDITION_CODE := 12;
USE_INAPPROPRIATE_NKI                  : constant CONDITION_CODE := 13;
USE_NOT_ADOPTED_NKI                    : constant CONDITION_CODE := 14;
USE_HAS_NO_ADOPTED_NKI                 : constant CONDITION_CODE := 15;
USE_ALREADY_IN_VIEW                    : constant CONDITION_CODE := 16;
USE_ILLEGAL_SPECIALIZATION             : constant CONDITION_CODE := 17;
USE_ILLEGAL_GENERALIZATION             : constant CONDITION_CODE := 18;
USE_ANCESTOR_IS_BIDIRECTIONAL          : constant CONDITION_CODE := 19;
USE_DEFINITION_NOT_IN_VIEW             : constant CONDITION_CODE := 20;

USE_DISCONNECTED_SUBLATTICE            : constant CONDITION_CODE := 21;
USE_PROPERTY_ALREADY_DEFINED           : constant CONDITION_CODE := 22;
USE_PROPERTY_NOT_DEFINED               : constant CONDITION_CODE := 23;
USE_PREDEFINED_KEY                     : constant CONDITION_CODE := 24;
USE_NODE_SPECIFIED_TWICE               : constant CONDITION_CODE := 25;
USE_NOT_DEFINITION_ATTRIBUTE           : constant CONDITION_CODE := 26;
USE_NOT_PROPERTY_ATTRIBUTE             : constant CONDITION_CODE := 27;
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
USE_RELATIONSHIP_NOT_BIDERECTIONAL      : constant CONDITION_CODE := 28;
USE_HANDLER_PROCESS_NOT_ACTIVE          : constant CONDITION_CODE := 29;
USE_PROCESS_ALREADY_HANDLER             : constant CONDITION_CODE := 30;
USE_DUPLICATE_HANDLER                   : constant CONDITION_CODE := 31;
USE_DIFFERENT_USER                      : constant CONDITION_CODE := 32;
USE_NODE_ATTRIBUTE_ALREADY_MONITORED    : constant CONDITION_CODE := 33;
USE_NO_ATTRIBUTES_MONITORED             : constant CONDITION_CODE := 34;

USE_CANNOT_BE_MONITORED                 : constant CONDITION_CODE := 35;
USE_CONTENTS_STRUCTURE_UNKNOWN          : constant CONDITION_CODE := 36;
USE_NOT_EVENT_HANDLER_PROCESS           : constant CONDITION_CODE := 37;
USE_ILLEGAL_PRIORITY                    : constant CONDITION_CODE := 38;
USE_PRIORTY_RANGE_NOT_DETERMINABLE      : constant CONDITION_CODE := 39;
USE_INVALID_PRIORITY_MODIFICATION       : constant CONDITION_CODE := 40;
USE_NO_MULTIPLE_FILE_HANDLES            : constant CONDITION_CODE := 41;
USE_BAD_BLOCK_SIZE                      : constant CONDITION_CODE := 42;
USE_ILLEGAL_FILE_HANDLE_RESET           : constant CONDITION_CODE := 43;
USE_INVALID_MODE                        : constant CONDITION_CODE := 44;
USE_NO_HOST_SYSTEM                      : constant CONDITION_CODE := 45;
USE_INVALID_HOST_FILE_NAME              : constant CONDITION_CODE := 46;
USE_ILLEGAL_CONTENTS_CHARACTERISTICS    : constant CONDITION_CODE := 47;
USE_NO_POTENTIAL_MEMBER                 : constant CONDITION_CODE := 48;
USE_DEFAULT_ROLE                        : constant CONDITION_CODE := 49;

USE_PROCESS_STILL_ACTIVE                : constant CONDITION_CODE := 50;
USE_KEY_ATTRIBUTE_NAMED                 : constant CONDITION_CODE := 51;
USE_RELATION_NOT_BIDIRECTIONAL          : constant CONDITION_CODE := 52;
USE_WRONG_DIRECTIONALITY                : constant CONDITION_CODE := 53;
USE_INCOMPATBILE_CONTENT_STRUCTURE      : constant CONDITION_CODE := 54;
USE_INCOMPATIBLE_CLIENT                 : constant CONDITION_CODE := 55;
USE_PROCESS_NOT_ACTIVATED               : constant CONDITION_CODE := 56;
USE_PROCESS_CANNOT_BE_ACTIVATED         : constant CONDITION_CODE := 57;
USE_TRANSACTION_NOT_COMPLETED           : constant CONDITION_CODE := 58;
USE_NO_INTENT_TO_ACCESS                 : constant CONDITION_CODE := 59;
USE_PREDEFINED_NODE_ATTRIBUTE           : constant CONDITION_CODE := 60;
USE_PREDEFINED_PATH_ATTRIBUTE           : constant CONDITION_CODE := 61;
USE_PREDEFINED_NODE                     : constant CONDITION_CODE := 62;
USE_PREDEFINED_RELATIONSHIP             : constant CONDITION_CODE := 63;
USE_INAPPROPRIATE_LINE_LENGTH           : constant CONDITION_CODE := 64;
USE_INAPPROPRIATE_PAGE_LENGTH           : constant CONDITION_CODE := 65;
USE_INVALID_FORM_WRITE                  : constant CONDITION_CODE := 66;
USE_INVALID_FORM_SIZE                   : constant CONDITION_CODE := 67;
USE_AREA_QUALIFIER_CONFLICT             : constant CONDITION_CODE := 68;
USE_CHANNEL_IN_USE                      : constant CONDITION_CODE := 69;

-- Condition cod. for EXECUTABLE_IMAGE_ERROR:

EXE_NOT_EXECUTABLE_IMAGE                : constant CONDITION_CODE := 1;

-- Condition codes for CLIENT_ERROR:

CLI_CALLER_NOT_CLIENT                   : constant CONDITION_CODE := 1;
CLI_WRONG_CLIENT_KIND                   : constant CONDITION_CODE := 2;
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
CLI_PROCESS_OUT_OF_EXECUTION        : constant CONDITION_CODE := 3;
CLI_NO_PROCESS                      : constant CONDITION_CODE := 4;
CLI_NOT_CLIENT                      : constant CONDITION_CODE := 5;

-- Condition codes for DATA_ERROR:

DAT_WRONG_TYPE                      : constant CONDITION_CODE := 1;
DAT_OUT_OF_RANGE                    : constant CONDITION_CODE := 2;
DAT_SYNTAX                          : constant CONDITION_CODE := 3;

-- Condition codes for END_ERROR:

END_ATTEMPT_TO_PASS_END             : constant CONDITION_CODE := 1;
END_BUFFER_EMPTY                    : constant CONDITION_CODE := 2;

-- Condition codes for LAYOUT_ERROR:

LAY_OUT_OF_RANGE                    : constant CONDITION_CODE := 1;
LAY_OUT_OF_BOUNDS                   : constant CONDITION_CODE := 2;
LAY_STRING_TOO_LONG                 : constant CONDITION_CODE := 3;
LAY_BOUNDED                         : constant CONDITION_CODE := 4;

-- Condition codes for MODE_ERROR:

MOD_OPERATION_CONFLICT              : constant CONDITION_CODE := 1;
MOD_DIRECTION_CONFLICT              : constant CONDITION_CODE := 2;
MOD_MODE_CONFLICT                   : constant CONDITION_CODE := 3;

-- Condition codes for OPEN_DENIED_ERROR:

OPD_CHANNEL_DELETED                 : constant CONDITION_CODE := 1;
OPD_OPPOSITE_OPEN_FAILED            : constant CONDITION_CODE := 2;

-- Condition cods for PORT_ERROR:

POE_INCOMPATIBLE_RD                 : constant CONDITION_CODE := 1;
POE_INCOMPATIBLE_DIRECTIONS         : constant CONDITION_CODE := 2;

-- Condition codes for PROTOCOL_ERROR:

PRO_NOT_ALLOWED                     : constant CONDITION_CODE := 1;
PRO_DIFFERENT_PROTOCOLS             : constant CONDITION_CODE := 2;
PRO_RANGES_INCOMPATIBLE             : constant CONDITION_CODE := 3;
PRO_CONSTRAINT_VIOLATION            : constant CONDITION_CODE := 4;
PRO_MISSING_PARAMETER               : constant CONDITION_CODE := 5;
PRO_AMBIGUOUS_PORT_SELECTION        : constant CONDITION_CODE := 6;

-- Condition code for TIME_EXPIRED:

TEX_DELATED_BEYOND_TIME_LIMIT       : constant CONDITION_CODE := 1;

-- Condition code for QUEUE_FULL_ERROR:

QFU_QUEUE_FULL                      : constant CONDITION_CODE := 1;
```


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
-- Condition code for QUEUE_EMPTY_ERROR:

QEM_QUEUE_EMPTY                          : constant CONDITION_CODE := 1;

-- Condition codes for TAPE_STATUS_ERROR:

TAP_NOT_MOUNTED                          : constant CONDITION_CODE := 1;
TAP_NOT_LOADED                           : constant CONDITION_CODE := 2;

-- Condition code for BUTTON_STATUS_ERROR:

BUT_NONEXISTENT_BUTTON                   : constant CONDITION_CODE := 1;

-- Condition code for CAIS_TERMINAL_IO_FUNCTION_KEY_STATUS_ERROR:

FKY_NONEXISTENT_FUNCTION_KEY             : constant CONDITION_CODE := 1;

-- Condition code for INPUT_EVENT-QUEUE-EMPTY:

IQE-QUEUE-EMPTY                          : constant CONDITION-CODE := 1;

-- Condition codes for LOCATOR_CURSOR_NOT_ON_SCREEN:

LCS_LOCATOR_NOT_ENABLED                  : constant CONDITION_CODE := 1;
LCS_CURSOR_NOT_ON_ANY_SCREEN             : constant CONDITION_CODE := 2;
LCS_CURSOR_NOT_ON_SCREEN                 : constant CONDITION_CODE := 3;

-- Condition code for SCREEN_NOT_ON_DISPLAY:

SCR_NOT_PASTED                           : constant CONDITION_CODE := 1;

-- Condition code for SCREEN_POSITION_ERROR:

SCP_OFF_SCREEN                           : constant CONDITION_CODE := 1;

-- Condition code for VIRTUAL_DISPLAY_POSITION_ERROR:

VDP_OFF_SCREEN                           : constant CONDITION_CODE := 1;

-- Condition code for UID_ERROR:

UID_NOT_UNIQUE                           : constant CONDITION_CODE := 1;

-- Condition codes for ITEM_KIND_ERROR:

IKE_WRONG_ITEM_KIND                      : constant CONDITION_CODE := 1;
IKE_NO_ENUMERATION_VALUE                 : constant CONDITION_CODE := 2;
IKE_WRONG_ENUMERATION_POSITION           : constant CONDITION_CODE := 3;
```


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
-- Condition codes for LIST_KIND_ERROR:

LKE_EMPTY_LIST                  : constant CONDITION_CODE := 1;
LKE_NOT_NAMED                   : constant CONDITION_CODE := 2;
LKE_KIND_MISMATCH               : constant CONDITION_CODE := 3;
LKE_CANNOT_BE_NAMED             : constant CONDITION_CODE := 4;
LKE_MUST_BE_NAMED               : constant CONDITION_CODE := 5;

-- Condition codes for LIST_POSITION_ERROR:

LPE_POSITION_EXCEEDS_LENGTH     : constant CONDITION_CODE := 1;
LPE_START_BEYOND_END            : constant CONDITION_CODE := 2;
LPE_CURRENT_IS_OUTERMOST        : constant CONDITION_CODE := 3;

-- Condition codes for NAMED_LIST_ERROR:

NLE_DUPLICATE_ITEM_NAMED        : constant CONDITION_CODE := 1;
NLE_NOT_NAMED_LIST              : constant CONDITION_CODE := 2;
NLE_NAME_CONFLICT               : constant CONDITION_CODB := 3;
NLE_NAME_ALREADY_USED           : constant CONDITION_CODE := 4;

-- Condition codes for SEARCH_ERROR:

SEA_NO_SUCH_NAMED_ITEM          : constant CONDITION_CODE := 1;
SEA_NO_SUCH_VALUED_ITEM         : constant CONDITION_CODE := 2;

-- Condition codes for CAIS_LIST_MANAGEMENT.SYNTAX_ERROR:

LSY_ILLEGAL_LIST_TEXT           : constant CONDITION_CODE := 1;
LSY_ILLEGAL_IDENTIFIER_TEXT     : constant CONDITION_CODE := 2;

-- Condition code for TOKEN_ERROR:

TOK_UNDEFINED_TOKEN             : constant CONDITION_CODE := 1;

-- Condition codes for TIME_ERROR:

TIM_NOT_A_TIME                  : constant CONDITION_CODE := 1;
TIM_INVALID_YEAR                : constant CONDITION_CODE := 2;
TIM_INVALID_DURATION            : constant CONDITION_CODB := 3;

-- Condition codes for CAPACITY_ERROR:

CAP_PRAGMATIC_VIOLATION         : constant CONDITION_CODE := 1;
CAP_MAX_FILE_SIZE_EXCEEDED      : constant CONDITION_CODE := 2;

-- Condition code for RESOURCE_ERROR:

RES_RESOURCE_NOT_REASSIGNABLE   : constant CONDITION_CODE := 1;

-- Condition codes for FORM_STATUS_ERROR:

FRH_NONEXISTENT_FORM            : constant CONDITION_CODE := 1;
FRM_FORM_EXISTS                 : constant CONDITION_CODE := 2;
```


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
        -- Condition cod. for CAIS_FORM_TERMINAL_IO.FUNCTION_KEY_STATUS_ERROR:

        FFK_NONEXISTENT_FUNCTION_KEY          : constant CONDITION_CODE := 1;

        -- Condition code for TERMINAL_POSITION_ERROR:

        FTM_OFF_FORM                          : constant CONDITION_CODE := 1;

        private

            type STATUS_INDICATOR is (IMPLEMENTATION_DEFINED);
            -- This type should be defined by the implementer.

            NO_INDICATION : constant STATUS_INDICATOR := IMPLEMENTATION_DEFINED;
            -- This constant should be defined by the implementer.


end CAIS-STATUS_DBFINITIONS;
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
with CAIS_STANDARD;
with CAIS_PRAGMATICS;
with CAIS_STATUS_DEFINITIONS;

package CAIS_LIST_MANAGEMENT is


    use CAIS_STANDARD;
    use CAIS_STATUS_DEFINITIONS;

    type    LIST_TYPE       is limited private;

    subtype LIST_TEXT       is STRING;

    type    LIST_SIZE       is range 0 .. CAIS_PRAGMATICS.LIST_LENGTH;
    subtype POSITION_COUNT  is LIST_SIZE range 1 . . LIST_SIZE'LAST;
    subtype INSERT_COUNT    is LIST_SIZE range 0 . . LIST_SIZE'LAST - 1;

    type    LIST_KIND       is (UNNAMED, NAMED, EMPTY);

    type    ITEM_KIND       is (LIST_ITEM_KIND,
                                STRING_ITEM_KIND,
                                INTEGER_ITEM_KIND,
                                FLOAT_ITEM_KIND,
                                IDENTIFIER_ITEM_KIND);

    type    TOKEN_TYPE      is limited private;
    subtype IDENTIFIER_TEXT is STRING;


    EMPTY_LIST : constant LIST_TYPE;

    ITEM_KIND_ERROR      : exception;
    LIST_KIND_ERROR      : exception;
    LIST_POSITION_ERROR  : exception;
    NAMED_LIST_ERROR     : exception;
    SEARCH_ERROR         : exception;
    SYNTAX_ERROR         : exception;
    TOKEN_ERROR          : exception;


    procodure COPY_LIST
        (FROM_LIST : in     LIST_TYPE;
         TO_LIST   : in out LIST_TYPE;
         STATUS    : in     STATUS_INDICATOR := NO_INDICATION);


    procodure SET_TO_EMPTY_LIST
        (LIST   : in out LIST_TYPE;
         STATUS : in     STATUS_INDICATOR := NO_INDICATION);


    procedure CONVERT_TEXT_TO_LIST
        (LIST_STRING : in     LIST_TEXT;
         LIST        : in out LIST_TYPE;
         STATUS      : in     STATUS_INDICATOR := NO_INDICATION);
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
function TEXT_FORM
    (LIST     : in      LIST_TYPE;
     STATUS : in      STATUS_INDICATOR  :=  NO_INDICATION)
    return LIST_TEXT;

function  IS_EQUAL
    (LIST1  :  in      LIST_TYPE;
     LIST2  :  in      LIST_TYPE;
     STATUS : in      STATUS_INDICATOR  := NO_INDICATION)
    return BOOLEAN;

procedure  DELETE
    (LIST              : in out LIST_TYPE;
     ITEM_POSITION : in      POSITION_COUNT;
     STATUS            :  in      STATUS_INDICATOR  := NO_INDICATION);

procedure  DELETE
    (LIST       : in out LIST_TYPE;
     ITEM_NAME : in      IDENTIFIER_TEXT;
     STATUS      :  in      STATUS_INDICATOR  := NO_INDICATION);

procedure  DELETE
    (LIST       : in out LIST_TYPE;
     ITEM_NAME : in      TOKEN_TYPE;
     STATUS      :  in      STATUS_INDICATOR  := NO_INDICATION);

function  KIND_OF_LIST
    (LIST    :  in      LIST_TYPE;
     STATUS : in      STATUS_INDICATOR  := NO_INDICATION)
    return LIST_KIND;

function  KIND_OF_ITEM
    (LIST              :  in      LIST_TYPE;
     ITEM_POSITION : in      POSITION_COUNT;
     STATUS            :  in      STATUS_INDICATOR  :=  NO_INDICATION)
    return  ITEM_KIND;

function  KIND_OF_ITEM
    (LIST       :  in      LIST_TYPE;
     ITEM_NAME : in      IDENTIFIER_TEXT ;
     STATUS      :  in      STATUS_INDICATOR  := NO_INDICATION)
    return ITEM_KIND;

function  KIND_OF_ITEM
    (LIST       :  in      LIST_TYPE;
     ITEM_NAME : in      TOKEN_TYPE;
     STATUS      :  in      STATUS_INDICATOR  := NO_INDICATION)
    return ITEM_KIND;

procedure  SPLICE
    (LIST            : in out LIST_TYPE;
     POSITION       :  in      INSERT_COUNT;
     SOURCE_LIST : in      LIST_TYPE;
     STATUS          :  in      STATUS_INDICATOR  := NO_INDICATION);
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
procedure CONCATENATE_LISTS
    (FRONT  : in     LIST_TYPE;
     BACK   : in     LIST_TYPE;
     RESULT : in out LIST_TYPE;
     STATUS : in     STATUS_INDICATOR := NO_INDICATION);

procedure EXTRACT_LIST
    (LIST           : in     LIST_TYPE;
     START_POSITION : in     POSITION_COUNT;
     END_POSITION   : in     POSITION_COUNT;
     RESULT_LIST    : in out LIST_TYPE;
     STATUS         : in     STATUS_INDICATOR := NO_INDICATION);

function NUMBER_OF_ITEMS
    (LIST   : in     LIST_TYPE;
     STATUS : in     STATUS_INDICATOR := NO_INDICATION)
    return LIST_SIZE;

function POSITION_OF_CURRENT_LIST
    (LIST   : in     LIST_TYPE;
     STATUS : in     STATUS_INDICATOR := NO_INDICATION)
    return POSITION_COUNT;

function CURRENT_LIST_IS_OUTERMOST
    (LIST   : in     LIST_TYPE;
     STATUS : in     STATUS_INDICATOR := NO_INDICATION)
    return BOOLEAN;

procedure MAKE_CONTAINING_LIST_CURRENT
    (IN_LIST : in out LIST_TYPE;
     STATUS  : in     STATUS_INDICATOR := NO_INDICATION);

procedure MAKE_THIS_ITEM_CURRENT
    (IN_LIST       : in out LIST_TYPE;
     ITEM_POSITION : in     POSITION_COUNT;
     STATUS        : in     STATUS_INDICATOR := NO_INDICATION);

procedure MAKE_THIS_ITEM_CURRENT
    (IN_LIST   : in out LIST_TYPE;
     ITEM_NAME : in     IDENTIFIER_TEXT;
     STATUS    : in     STATUS_INDICATOR := NO_INDICATION);

procedure MAKE_THIS_ITEM_CURRENT
    (IN_LIST   : in out LIST_TYPE;
     ITEM_NAME : in     TOKEN_TYPE;
     STATUS    : in     STATUS_INDICATOR := NO_INDICATION);

function TEXT_LENGTH
    (LIST   : in     LIST_TYPE;
     STATUS : in     STATUS_INDICATOR := NO_INDICATION)
    return CAIS_POSITIVE;

function TEXT_LENGTH
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
    (LIST          : in     LIST_TYPE;
     ITEM_POSITION : in     POSITION_COUNT;
     STATUS        : in     STATUS_INDICATOR := NO_INDICATION)
    return CAIS_POSITIVE;

function TEXT_LENGTH
    (LIST          : in     LIST_TYPE;
     ITEM_NAME     : in     IDENTIFIER_TEXT;
     STATUS        : in     STATUS_INDICATOR := NO_INDICATION)
    return CAIS_POSITIVE;

function TEXT_LENGTH
    (LIST          : in     LIST_TYPE;
     ITEM_NAME     : in     TOKEN_TYPE;
     STATUS        : in     STATUS_INDICATOR := NO_INDICATION)
    return CAIS_POSITIVE;


procedure GET_ITEM_NAME
    (LIST          : in     LIST_TYPE;
     ITEM_POSITION : in     POSITION_COUNT;
     NAME          : in out TOKEN_TYPE;
     STATUS        : in     STATUS_INDICATOR := NO_INDICATION);


function POSITION_BY_NAME
    (LIST          : in     LIST_TYPE;
     ITEM_NAME     : in     IDENTIFIER_TEXT;
     STATUS        : in     STATUS_INDICATOR := NO_INDICATION)
    return POSITION_COUNT;

function POSITION_BY_NAME
    (LIST          : in     LIST_TYPE;
     ITEM_NAME     : in     TOKEN_TYPE;
     STATUS        : in     STATUS_INDICATOR := NO_INDICATION)
    return POSITION_COUNT;
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
package CAIS_LIST_ITEM is

    procedure EXTRACT_VALUE
        (FROM_LIST      : in      LIST_TYPE;
         ITEM_POSITION : in      POSITION_COUNT;
         VALUE          : in out LIST_TYPE;
         STATUS         : in      STATUS_INDICATOR  := NO_INDICATION);

    procedure EXTRACT_VALUE
        (FROM_LIST : in      LIST_TYPE;
         ITEM_NAME : in      IDENTIFIER_TEXT;
         VALUE     : in out LIST_TYPE;
         STATUS    : in      STATUS_INDICATOR  := NO_INDICATION);

    procedure EXTRACT_VALUE
        (FROM_LIST : in      LIST_TYPE;
         ITEM_NAME : in      TOKEN_TYPE;
         VALUE     : in out  LIST_TYPE;
         STATUS    : in      STATUS_INDICATOR  := NO_INDICATION);


    procedure  REPLACE
        (IN_LIST        : in out  LIST_TYPE;
         ITEM_POSITION : in      POSITION_COUNT;
         VALUE          : in      LIST_TYPE;
         STATUS         : in      STATUS_INDICATOR  := NO_INDICATION);

    procedure REPLACE
        (IN_LIST    : in out  LIST_TYPE;
         ITEM_NAME : in      IDENTIFIER_TEXT;
         VALUE     : in      LIST_TYPE;
         STATUS    : in      STATUS_INDICATOR  := NO_INDICATION);

    procedure REPLACE
        (IN_LIST    : in out  LIST_TYPE;
         ITEM_NAME : in      TOKEN_TYPE;
         VALUE     : in      LIST_TYPE;
         STATUS    : in      STATUS_INDICATOR  := NO_INDICATION);


    procedure INSERT
        (IN_LIST  : in out  LIST_TYPE;
         POSITION : in      INSERT_COUNT;
         VALUE    : in      LIST_TYPE;
         STATUS   : in      STATUS_INDICATOR  := NO_INDICATION);

    procedure INSERT
        (IN_LIST  : in out  LIST_TYPE;
         POSITION : in      INSERT_COUNT;
         NAME     : in      IDENTIFIER_TEXT;
         VALUE    : in      LIST_TYPE;
         STATUS   : in      STATUS_INDICATOR  := NO_INDICATION);

    procedure INSERT
        (IN_LIST  : in out  LIST_TYPE;
         POSITION : in      INSERT_COUNT;
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
        NAME      : in      TOKEN_TYPE;
        VALUE     : in      LIST_TYPE;
        STATUS    : in      STATUS_INDICATON  := NO_INDICATION);

    function  POSITION_BY_VALUE
        (LIST             : in      LIST_TYPE;
        VALUE             : in      LIST_TYPE;
        START_POSITION    : in      POSITION_COUNT     := POSITION_COUNT'FIRST;
        END_POSITION      : in      POSITION_COUNT     := POSITIOM_COUNT'LAST;
        STATUS            : in      STATUS_INDICATON   := NO_INDICATION)
        return POSITION_COUNT;

end CAIS_LIST_ITEM;
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
package CAIS_IDENTIFIER_ITEM is

    procedure  COPY_TOKEN
        (FROM_TOKEN : in      TOKEN_TYPE;
         TO_TOKEN   : in out TOKEN_TYPE;
         STATUS     :  in    STATUS_INDICATOR  := NO_INDICATION);

    procedure  CONVERT_TEXT_TO_TOKEN
        (IDENTIFIER : in      IDENTIFIER_TEXT;
         TOKEN      : in out TOKEN_TYPE;
         STATUS     :  in    STATUS_INDICATOR  := NO_INDICATION);

    function TEXT_FORM
        (TOKEN  : in      TOKEN_TYPE;
         STATUS : in      STATUS_INDICATOR  := NO_INDICATION)
        return IDENTIFIER_TEXT;

    procedure  DELETE_TOKEN
        (TOKEN  : in out TOKEN_TYPE;
         STATUS : in     STATUS_INDICATOR  := NO_INDICATION);

    function  IS_EQUAL
        (TOKEN1 : in      TOKEN_TYPE;
         TOKEN2 : in      TOKEN_TYPE;
         STATUS : in      STATUS_INDICATOR  := NO_INDICATION)
        return BOOLEAN;

    procedure EXTRACT_VALUE
        (FROM_LIST     :  in     LIST_TYPE;
         ITEM_POSITION : in      POSITION_COUNT;
         VALUE         : in out TOKEN_TYPE;
         STATUS        :  in    STATUS_INDICATOR  := NO_INDICATION);

    procedure EXTRACT_VALUE
        (FROM_LIST : in      LIST_TYPE;
         ITEM_NAME : in      IDENTIFIER_TEXT;
         VALUE     : in out TOKEN_TYPE;
         STATUS    :  in    STATUS_INDICATOR  := NO_INDICATION);

    procedure EXTRACT_VALUE
        (FROM_LIST : in      LIST_TYPE;
         ITEM_NAME : in      TOKEN_TYPE;
         VALUE     : in out TOKEN_TYPE;
         STATUS    :  in    STATUS_INDICATOR  := NO_INDICATION);

    procedure  REPLACE
        (IN_LIST       : in out LIST_TYPE;
         ITEM_POSITION : in     POSITION_COUNT;
         VALUE         :  in    TOKEN_TYPE;
         STATUS        :  in    STATUS_INDICATOR  := NO_INDICATION);

    procedure  REPLACE
        (IN_LIST  : in out LIST_TYPE;
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
        ITEM_NAME : in      IDENTIFIER_TEXT;
        VALUE     : in      TOKEN_TYPE;
        STATUS    : in      STATUS_INDICATOR  := NO_INDICATION);

    procedure REPLACE
        (IN_LIST  : in out LIST_TYPE;
        ITEM_NAME : in      TOKEN_TYPE;
        VALUE     : in      TOKEN_TYPE;
        STATUS    : in      STATUS_INDICATOR  := NO_INDICATION);

    procedure INSERT
        (IN_LIST  : in out LIST_TYPE;
        POSITION  : in      INSERT_COUNT;
        VALUE     : in      TOKEN_TYPE;
        STATUS    : in      STATUS_INDICATOR  := NO_INDICATION);

    procedure INSERT
        (IN_LIST  : in out LIST_TYPE;
        POSITION  : in      INSERT_COUNT;
        NAME      : in      IDENTIFIER_TEXT;
        VALUE     : in      TOKEN_TYPE;
        STATUS    : in      STATUS_INDICATOR  := NO_INDICATION);

    procedure INSERT
        (IN_LIST  : in out LIST_TYPE;
        POSITION  : in      INSERT_COUNT;
        NAME      : in      TOKEN_TYPE;
        VALUE     : in      TOKEN_TYPE;
        STATUS    : in      STATUS_INDICATOR  := NO_INDICATION);

    function  POSITION_BY_VALUE
        (LIST           : in    LIST_TYPE;
        VALUE           : in    TOKEN_TYPE;
        START_POSITION  : in    POSITION_COUNT    := POSITION_COUNT'FIRST;
        END_POSITION    : in    POSITION_COUNT    := POSITION_COUNT'LAST;
        STATUS          : in    STATUS_INDICATOR  := NO_INDICATION)
        return POSITION_COUNT;

    function  IS_ENUMERATED
        (FROM_LIST      : in    LIST_TYPE;
        ITEM_POSITION   : in    POSITION_COUNT;
        STATUS          : in    STATUS_INDICATOR  := NO_INDICATION)
        return BOOLEAN;

    function  IS_ENUMERATED
        (FROM_LIST : in     LIST_TYPE;
        ITEM_NAME  : in     IDENTIFIER_TEXT;
        STATUS     : in     STATUS_INDICATOR  := NO_INDICATION)
        return BOOLEAN;

    function  IS_ENUMERATED
        (FROM_LIST : in     LIST_TYPE;
        ITEM_NAME  : in     TOKEN_TYPE;
        STATUS     : in     STATUS_INDICATOR  := NO_INDICATION)
        return BOOLEAN;
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
        function ENUMERATION_POSITIOH
            (FROM_LIST     : in    LIST_TYPE;
             ITEM_POSITION : in    POSITION_COUNT;
             STATUS        : in    STATUS_INDICATOR  := NO_INDICATION)
            return CAIS_NATURAL;

        function ENUMERATION_POSITION
            (FROM_LIST : in    LIST_TYPE;
             ITEM_NAME : in    IDENTIFIER_TEXT;
             STATUS    : in    STATUS_INDICATOR  := NO_INDICATION)
            return CAIS_NATURAL;

        function ENUMERATION_POSITION
            (FROM_LIST : in    LIST_TYPE;
             ITEM_NAME : in    TOKEN_TYPE;
             STATUS    : in    STATUS_INDICATOR  := NO_INDICATION)
            return CAIS_NATURAL;

    l nd CAIS_IDENTIFIER_ITEM;
```



Source: https://assist.dla.mil -- Downloaded: 202610-26T07:54Z
Check the source to verify that this is the currentversion before use.

```
generic
      typo ENUMERATION is (<>);
package CAIS_ENUMERATION_ITEM is

    function TEXT_FORM
        (ENUMERATION_VALUE : in      ENUMERATION;
         STATUS            : in      STATUS_INDICATOR  := NO_INDICATION)
        return IDENTIFIER_TEXT;

    function EXTRACTED_VALUE
        (FROM_LIST        : in      LIST_TYPE;
         ITEM_POSITION    : in      POSITION_COUNT;
         STATUS           : in      STATUS_INDICATOR  := NO_INDICATION)
        return ENUMERATION;

    function EXTRACTED_VALUE
        (FROM_LIST : in      LIST_TYPE;
         ITEM_NAME : in      IDENTIFIER_TEXT;
         STATUS    : in      STATUS_INDICATOR  := NO_INDICATION)
        return ENUMERATION;

    function EXTRACTED_VALUE
        (FROM_LIST : in      LIST_TYPE;
         ITEM_NAME : in      TOKEN_TYPE;
         STATUS    : in      STATUS_INDICATOR  := NO_INDICATION)
        return ENUMERATION;

    procedure REPLACE
        (IN_LIST          : in out LIST_TYPE;
         ITEM_POSITION    : in     POSITION_COUNT;
         VALUE            : in     ENUMERATION;
         STATUS           : in     STATUS_INDICATOR  := NO_INDICATION);

    procedure REPLACE
        (IN_LIST     : in out LIST_TYPE;
         ITEM_NAME   : in     IDENTIFIER_TEXT;
         VALUE       : in     ENUMERATION;
         STATUS      : in     STATUS_INDICATOR  := NO_INDICATION);

    procedure  REPLACE
        (IN_LIST     : in out LIST_TYPE;
         ITEM_NAME   : in     TOKN_TYPE;
         VALUE       : in     ENUMERATION;
         STATUS      : in     STATUS_INDICATOR  := NO_INDICATION);

    procedure INSERT
        (IN_LIST  : in out LIST_TYPE;
         POSITION : in     INSERT_COUNT;
         VALUE    : in     ENUMERATION;
         STATUS   : in     STATUS_INDICATOR  := NO_INDICATION);

    procedure INSERT
        (IN_LIST  : in out LIST_TYPE;
         POSITION : in     INSERT_COUNT;
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
            NAME        : in        IDENTIFIER_TEXT;
            VALUE       : in        ENUMERATION;
            STATUS      : in        STATUS_INDICATOR  := NO_INDICATION);

        procedure INSERT
            (IN_LIST  : in  out  LIST_TYPE;
             POSITION : in       INSERT_COUNT;
             NAME     : in       TOKEN_TYPE;
             VALUE    : in       ENUMERATION;
             STATUS   : in       STATUS_INDICATOR  := NO_INDICATION);

        function  POSITION_BY_VALUE
            (LIST           : in   LIST_TYPE;
             VALUE          : in   ENUMERATION;
             START_POSITION : in   POSITION_COUNT     := POSITION_COUNT'FIRST;
             END_POSITION   : in   POSITION_COUNT     := POSITION_COUNT'LAST;
             STATUS         : in   STATUS_INDICATOR   := NO_INDICATION)
            return POSITION_COUNT;

    end CAIS_ENUMERATION_ITEM;
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
generic
      type NUMBER is range <>;
package CAIS_INTEGER_ITEM is

    function TEXT_FORM
        (INTEGER_VALUE : in      NUMBER;
         STATUS        : in      STATUS_INDICATOR  := NO_INDICATION)
        return STRING;

    function EXTRACTED_VALUE
        (FROM_LIST     : in      LIST_TYPE;
         ITEM_POSITION : in      POSITION_COUNT;
         STATUS        : in      STATUS_INDICATOR  := NO_INDICATION)
        return NUMBER;

    function EXTRACTED_VALUE
        (FROM_LIST : in      LIST_TYPE;
         ITEM_NAME : in      IDENTIFIER_TEXT;
         STATUS    : in      STATUS_INDICATION  :=  NO_INDICATION)
        return NUMBER;

    function EXTRACTED_VALUE
        (FROM_LIST : in      LIST_TYPE;
         ITEM_NAME : in      TOKEN_TYPE;
         STATUS    : in      STATUS_INDICATOR  :=  NO_INDICATION)
        return NUMBER;

    procedure  REPLACE
        (IN_LIST       : in  out  LIST_TYPE;
         ITEM_POSITION : in       POSITION_COUNT;
         VALUE         : in       NUMBER;
         STATUS        : in       STATUS_INDICATOR  := NO_INDICATION);

    procedure REPLACE
        (IN_LIST   : in  out  LIST_TYPE;
         ITEM_NAME : in       IDENTIFIER_TEXT;
         VALUE     : in       NUMBER;
         STATUS    : in       STATUS_INDICATOR  := NO_INDICATION);

    procedure  REPLACE
        (IN_LIST   : in  out  LIST_TYPE;
         ITEM_NAME : in       TOKEN_TYPE;
         VALUE     : in       NUMBER;
         STATUS    : in       STATUS_INDICATOR  := NO_INDICATION);

    procedure INSERT
        (IN_LIST : in  out  LIST_TYPE;
         POSITION : in      INSERT_COUNT;
         VALUE    : in      NUMBER;
         STATUS   : in      STATUS_INDICATOR  := NO_INDICATION);

    procedure INSERT
        (IN_LIST : in  out  LIST_TYPE;
         POSITION : in      INSERT_COUNT;
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
        NAME       : in      IDENTIFIER_TEXT;
        VALUE      : in      NUMBER;
        STATUS     : in      STATUS_INDICATOR  := NO_INDICATION);

    procedure INSERT
        (IN_LIST   : in out LIST_TYPE;
         POSITION  : in    INSERT_COUNT;
         NAME      : in    TOKEN_TYPE;
         VALUE     : in    NUMBER;
         STATUS    : in    STATUS_INDICATOR  := NO_INDICATION);

    function  POSITION_BY_VALUE
        (LIST           : in    LIST_TYPE;
         VALUE          : in    NUMBER;
         START_POSITION : in    POSITION_COUNT     := POSITION_COUNT'FIRST;
         END_POSITION   : in    POSITION_COUNT     :=  POSITION_COUNT'LAST;
         STATUS         : in    STATUS_INDICATOR   := NO_INDICATION)
        return POSITION_COUNT;

end CAIS_INTEGER_ITEM;
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
generic
    type NUMBER is digits <>;
package CAIS_FLOAT_ITEM is

    function TEXT_FORM
        (FLOAT_VALUE : in     NUMBER;
         STATUS      : in     STATUS_INDICATOR := NO_INDICATION)
        return STRING;

    function EXTRACTED_VALUE
        (FROM_LIST     : in     LIST_TYPE;
         ITEM_POSITION : in     POSITION_COUNT;
         STATUS        : in     STATUS_INDICATOR := NO_INDICATION)
        return NUMBER;

    function EXTRACTED_VALUE
        (FROM_LIST : in     LIST_TYPE;
         ITEM_NAME : in     IDENTIFIER_TEXT;
         STATUS    : in     STATUS_INDICATOR := NO_INDICATION)
        return NUMBER;

    function EXTRACTED_VALUE
        (FROM_LIST : in     LIST_TYPE;
         ITEM_NAME : in     TOKEN_TYPE;
         STATUS    : in     STATUS_INDICATOR := NO_INDICATION)
        return NUMBER;

    procedure REPLACE
        (IN_LIST      : in out LIST_TYPE;
         ITEM_POSITION : in     POSITION_COUNT;
         VALUE        : in     NUMBER;
         STATUS       : in     STATUS_INDICATOR := NO_INDICATION);

    procedure REPLACE
        (IN_LIST   : in out LIST_TYPE;
         ITEM_NAME : in     IDENTIFIER_TEXT;
         VALUE     : in     NUMBER;
         STATUS    : in     STATUS_INDICATOR := NO_INDICATION);

    procedure REPLACE
        (IN_LIST   : in out LIST_TYPE;
         ITEM_NAME : in     TOKEN_TYPE;
         VALUE     : in     NUMBER;
         STATUS    : in     STATUS_INDICATOR := NO_INDICATION);

    procedure INSERT
        (IN_LIST  : in out LIST_TYPE;
         POSITION : in     INSERT_COUNT;
         VALUE    : in     NUMBER;
         STATUS   : in     STATUS_INDICATOR := NO_INDICATION);

    procedure INSERT
        (IN_LIST  : in out LIST_TYPE;
         POSITION : in     INSERT_COUNT;
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
        NAME      : in     IDENTIFIER_TEXT;
        VALUE     : in     NUMBER;
        STATUS    : in     STATUS_INDICATOR  := NO_INDICATION);

    procedure INSERT
        (IN_LIST  : in Out LIST_TYPE;
         POSITION : in     INSERT_COUNT;
         NAME     : in     TOKEN_TYPE;
         VALUE    : in     NUMBER;
         STATUS   : in     STATUS_INDICATON  := NO_INDICATION);

    function POSITION_BY_VALUE
        (LIST             : in   LIST_TYPE;
         VALUE            : in   NUMBER;
         START_POSITION   : in   POSITION_COUNT := POSITION_COUNT'FIRST;
         END_POSITION     : in   POSITION_COUNT := POSITION_COUNT'LAST;
         STATUS           : in   STATUS_INDICATOR := NO_INDICATION)
        return POSITION_COUNT;

l nd CAIS_FLOAT_ITEM;
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
package CAIS_STRING_ITEM is

    function EXTRACTED_ VALUE
        (FROM_LIST      : in      LIST_TYPE;
         ITEM_POSITION : in      POSITION_COUNT;
         STATUS         : in      STATUS_INDICATOR  :=  NO_INDICATION)
        return STRING;

    function EXTRACTED_VALUE
        (FROM_LIST : in      LIST_TYPE;
         ITEM_NAME : in      IDENTIFIER_TEXT;
         STATUS     : in      STATUS_INDICATOR  :=  NO_INDICATION)
        return STRING;

    function EXTRACTED_VALUE
        (FROM_LIST : in      LIST_TYPE;
         ITEM_NAME : in      TOKEN_TYPE;
         STATUS     : in      STATUS_INDICATOR  :=  NO_INDICATION)
        return STRING;


    procedure  REPLACE
        (IN_LIST        : in out LIST_TYPE;
         ITEM_POSITION : in      POSITION_COUNT;
         VALUE          : in      STRING ;
         STATUS         : in      STATUS_INDICATOR  := NO_INDICATION);

    procedure  REPLACE
        (IN_LIST    : in out LIST_TYPE;
         ITEM_NAME : in      IDENTIFIER_TEXT;
         VALUE      : in      STRING;
         STATUS     : in      STATUS_INDICATOR  := NO_INDICATION);

    procedure  REPLACE
        (IN_LIST    : in out LIST_TYPE;
         ITEM_NAME : in      TOKEN_TYPE;
         VALUE      : in      STRING;
         STATUS     : in      STATUS_INDICATOR  := NO_INDICATION);


    procedure INSERT
        (IN_LIST   : in out LIST_TYPE;
         POSITION : in      INSERT_COUNT;
         VALUE      : in      STRING ;
         STATUS     : in      STATUS_INDICATOR  := NO_INDICATION);

    procedure INSERT
        (IN_LIST   : in out LIST_TYPE;
         POSITION : in      INSERT_COUNT;
         NAME       : in      IDENTIFIER_TEXT;
         VALUE      : in      STRING;
         STATUS     : in      STATUS_INDICATOR  := NO_INDICATION);

    procedure INSERT
        (IN_LIST   : in out LIST_TYPE;
         POSITION : in      INSERT_COUNT;
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
            NAME      : in     TOKEN_TYPE;
            VALUE     : in     STRING;
            STATUS    : in     STATUS_INDICATOR  := NO_INDICATION);

        function  POSITION_BY_VALUE
            (LIST             : in     LIST_TYPE;
             VALUE            : in     STRING;
             START_POSITION   : in     POSITION_COUNT := POSITION_COUNT'FIRST;
             END_POSITION     : in     POSITION_COUNT := POSITION_COUNT'LAST;
             STATUS           : in     STATUS_INDICATOR := NO_INDICATION)
            return POSITION_COUNT;

    end CAIS_STRING_ITEM;

private

    type LIST_TYPE is (IMPLEMENTATION_DEFINED);
    -- This type should be defined by the implementer.

    EMPTY_LIST :  constant LIST_TYPE  := IMPLEMENTATION_DEFINED;
    -- This constant should b. defined by the implementer.

    type TOKEN_TYPE is (IMPLEMENTATION_DEFINED);
    -- This type should be dofined by the implementer.

l nd CAIS_LIST_MANAGEMENT;
```



Source: https://assist.dla.mil -- Downloaded: 2026l0-26T07:54Z
Check the source to verify that this is the currentversion before use.

```
with CAIS_STATUS_DEFINIITIONS;
with CAIS_LIST_MANAGEMEMT;

package CAIS_STATUS_MANAGEMENT is

        use CAIS_STATUS_DEFINITIONS;
        use CAIS_LIST_MANAGEMENT;

        procedure CREATE_INDICATOR
            (INDICATOR : in out STATUS_INDICATOR;
             STATUS    : in     STATUS_INDICATOR := NO_INDICATION);

        procedure SET_INDICATOR
            (INDICATOR           : in out STATUS_INDICATOR;
             EXCEPTION_INDICATION : in     EXCEPTION_INDICATION_KIND;
             STATUS_INFORMATION  : in     LIST_TYPE          := EMPTY_LIST;
             MESSAGE             : in     MESSAGE_TEXT       := "";
             STATUS              : in     STATUS_INDICATOR   := NO_INDICATION);

        procedure COPY_INDICATOR
            (FROM_INDICATOR : in     STATUS_INDICATOR;
             TO_INDICATOR   : in out STATUS_INDICATOR;
             STATUS         : in     STATUS_INDICATOR := NO_INDICATION);

        procedure DELETE_INDICATOR
            (INDICATOR : in out STATUS_INDICATOR;
             STATUS    : in     STATUS_INDICATOR := NO_INDICATION);

        function EXCEPTION_INDICATION
            (INDICATOR : in STATUS_INDICATOR;
             STATUS    : in STATUS_INDICATOR := NO_INDICATION)
            return EXCEPTION_INDICATION_KIND;

        function STATUS_INFORMATION
            (INDICATOR : in STATUS_INDICATOR;
             STATUS    : in STATUS_INDICATOR := NO_INDICATION)
            return LIST_TYPE;

        function MESSAGE
            (INDICATOR : in STATUS_INDICATOR;
             STATUS    : in STATUS_INDICATOR := NO_INDICATION)
            return MESSAGE_TEXT;

        function IS_ESTABLISHED
            (INDICATOR : in STATUS_INDICATOR;
             STATUS    : in STATUS_INDICATOR := NO_INDICATION)
            return BOOLEAN;

end CAIS_STATUS_MANAGEMENT;
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
with CAIS_PRAGMATICS;
with  CAIS_STANDARD;
with CAIS_LIST_MANAGEMENT;

package CAIS_DEFINITIONS is

    use  CAIS_STANDARD;

    type  NODE_TYPE  is  limited  private;

    type NODE_HANDLE_ARRAY is array (CAIS_POSITIVE range <>) of NODE_TYPE;

    type NODE_KIND is (FILE, STRUCTURAL, PROCESS);

    type  INTENT_SPECIFICATION  is
        (NO_ACCESS, READ, WRITE, APPEND,
         READ_ATTRIBUTES, WRITE_ATTRIBUTES, APPEND_ATTRIBUTES,
         READ_RELATIONSHIPS,  WRITE_RELATIONSHIPS,  APPEND_RELATIONSHIPS,
         READ_CONTENTS,  WRITE_CONTENTS,  APPEND_CONTENTS,
         CONTROL, EXECUTE, CREATE_INSTANCES,
         EXCLUSIVE_READ, EXCLUSIVE_WRITE, EXCLUSIVE_APPEND,
         EXCLUSIVE_READ_ATTRIBUTES,     EXCLUSIVE_WRITE_ATTRIBUTES,
         EXCLUSIVE_APPEND_ATTRIBUTES,     EXCLUSIVE_READ_RELATIONSHIPS,
         EXCLUSIVE_WRITE_RELATIONSHIPS, EXCLUSIVE_APPEND_RELATIONSHIPS,
         EXCLUSIVE_READ_CONTENTS,   EXCLUSIVE_WRITE_CONTENTS,
         EXCLUSIVE_APPEND_CONTENTS,    EXCLUSIVE_CONTROL);

    type INTENT_ARRAY is array (CAIS_POSITIVE range <>) of INTENT_SPECIFICATION;

    subtype CLIENT_NODE_TYPE                    is NODE_TYPE;
        subtype  FILE_NODE_TYPE                 is CLIENT_NODE_TYPE;
            subtype DEVICE_NODE_TYPE            is  FILE_NODE_TYPE;
                subtype MAGNETIC_TAPE_NODE_TYPE is  DEVICE_NODE_TYPE;
                subtype  TERMINAL_NODE_TYPE     is  DEVICE_NODE_TYPE;
            subtype GATEWAY_NODE_TYPE           is  FILE_NODE_TYPE;
            subtype   SECONDARY_STORAGE_NODE_TYPE is  FILE_NODE_TYPE;
                subtype EXECUTABLE_IMAGE_NODE_TYPE is  SECONDARY_STORAGE_NODE_TYPE;
                subtype INTRODUCED_NODE_TYPE    is  SECONDARY_STORAGE_NODE_TYPE;

        subtype  PROCESS_NODE_TYPE              is CLIENT_NODE_TYPE;

    subtype RESOURCE_NODE_TYPE                  is NODE_TYPE;

    subtype STRUCTURAL_NODE_TYPE                is NODE_TYPE;

        subtype CHANNEL_NODE_TYPE               is  STRUCTURAL_NODE_TYPE;
        subtype  DEFINITIONS_NODE_TYPE          is  STRUCTURAL_NODE_TYPE;

        subtype  DEFINITION_NODE_TYPE           is  STRUCTURAL_NODE_TYPE;
            subtype  ATTRIBUTE_NODE_TYPE        is DEFINITION_NODE_TYPE;
            subtype AD_TYPE                     is DEFINITION_NODE_TYPE;
                subtype FLOAT_AD_TYPE           is AD_TYPE;
                subtype IDENTIFIER_AD_TYPE      is AD_TYPE;
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
        subtype ENUMERATION_AD_TYPE          is IDENTIFIER_AD_TYPE;
      subtype INTEGER_AD_TYPE                is AD_TYPE;
      subtype STRING_AD_TYPE                 is AD_TYPE;
    subtype AKD_TYPE                         is DEFINITION_NODE_TYPE;
      subtype CLAKD_TYPE                     is AKD_TYPE;
        subtype NAMED_CLAKD_TYPE             is CLAKD_TYPE;
          subtype NAMED_ORDERED_CLAKD_TYPE   is NAMED_CLAKD_TYPE;
        subtype ORDERED_CLAKD_TYPE           is CLAKD_TYPE;
      subtype FAKD_TYPE                      is AKD_TYPE;
      subtype IDAKD_TYPE                     is AKD_TYPE;
      subtype IAKD_TYPE                      is AKD_TYPE;
      subtype LAKD_TYPE                      is AKD_TYPE;
      subtype SAKD_TYPE                      is AKD_TYPE;
      subtype ULAKD_TYPE                     is AKD_TYPE;
      subtype VAKD_TYPE                      is AKD_TYPE;
    subtype NKD_TYPE                         is DEFINITION_NODE_TYPE;
    subtype RD_TYPE                          is DEFINITION_NODE_TYPE;
      subtype PORT_RD_TYPE                   is RD_TYPE;

  subtype GROUP_NODE_TYPE                    is STRUCTURAL_NODE_TYPE;
  subtype INTERPRETATION_NODE_TYPE           is STRUCTURAL_NODE_TYPE;
    subtype NKI_TYPE                         is INTERPRETATION_NODE_TYPE;
    subtype RI_TYPE                          is INTERPRETATION_NODE_TYPE;
  subtype MEDIUM_NODE_TYPE                   is STRUCTURAL_NODE_TYPE;
  subtype MONITOR_NODE_TYPE                  is STRUCTURAL_NODE_TYPE;
  subtype NODE_KIND_NODE_TYPE                is STRUCTURAL_NODE_TYPE;
  subtype PROCESSOR_NODE_TYPE                is STRUCTURAL_NODE_TYPE;
  subtype SYSTEM_LEVEL_NODE_TYPE             is STRUCTURAL_NODE_TYPE;
  subtype USER_NODE_TYPE                     is STRUCTURAL_NODE_TYPE;
  subtype VIEW_NODE_TYPE                     is STRUCTURAL_NODE_TYPE;

subtype ATTRIBUTE_NAME          is STRING;
subtype ATTRIBUTE_KIND_NAME is STRING;
subtype NODE_KIND_NAME          is STRING;
subtype RELATION_NAME           is STRING;
subtype PATHNAME                is STRING;
subtype RELATIONSHIP_KEY        is STRING;
subtype UNIQUE_IDENTIFIER       is STRING;

subtype ATTRIBUTE_LIST              is CAIS_LIST_MANAGEMENT.LIST_TYPE;
subtype DISCRETIONARY_ACCESS_LIST is CAIS_LIST_MANAGEMENT.LIST_TYPE;
subtype MANDATORY_ACCESS_LIST       is CAIS_LIST_MANAGEMENT.LIST_TYPE;
subtype UNIQUE_IDENTIFIER_LIST      is CAIS_LIST_MANAGEMENT.LIST_TYPE;

NULL_NODE : constant NODE_TYPE;

CURRENT_USER : constant PATHNAME          := "'CURRENT_USER";
CURRENT_NODE    : constant PATHNAME       := "'CURRENT_NODE";
CURRENT_PROCESS : constant PATHNAME       := ":";
EMPTY_KEY_LIST  : constant RELATIONSHIP_KEY := "";
LATEST_KEY      : constant RELATIONSHIP_KEY := "#";
DEFAULT_RELATION : constant RELATION_NAME := "DOT";

LONG_DELAY : constant CAIS_DURATION := CAIS_DURATION'LAST;
```


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
    ACCESS_VIOLATION              : exception;
    AMBIGUOUS_VIEW_ERROR          : exception;
    ATTRIBUTE_ERROR               : exception;
    CONFORMANCE_ERROR             : exception;
    DEFINITION_ERROR              : exception;
    DEVICE_ERROR                  : exception;
    EXISTING_NODE_ERROR           : exception;
    INTENT_VIOLATION              : exception;
    ITERATOR_ERROR                : exception;
    LOCK_ERROR                    : exception;
    NAME_ERROR                    : exception;
    NODE_KIND_ERROR               : exception;
    NOT_IN_VIEW_ERROR             : exception;
    PATHNAME_SYNTAX_ERROR         : exception;
    PREDEFINED_ATTRIBUTE_ERROR    : exception;
    PREDEFINED_RELATION_ERROR     : exception;
    RELATIONSHIP_ERROR            : exception;
    RESOURCE_ALLOCATION_ERROR     : exception;
    RESOURCE_AVAILABILITY_ERROR   : exception;
    SECURITY_VIOLATION            : exception;
    STATUS_ERROR                  : exception;
    SYNTAX_ERROR                  : exception;
    TRANSACTION_ERROR             : exception;
    USE_ERROR                     : exception;

private

    type NODE_TYPE is (IMPLEMENTATION_DEFINED);
    -- This type should be defined by the implementer.

    NULL_NODE : constant NODE_TYPE := IMPLEMENTATION_DEFINED;
    -- This constant should be defined by the implementer.

end CAIS_DEFINITIONS;
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
with CAIS_STANDARD;
with CAIS_STATUS_DEFINITIONS;

package CAIS_CALENDAR is

    use CAIS-STANDARD;
    use CAIS_STATUS_DEFINITIONS;

    type TIME is private;

    subtype YEAR_NUMBER in CAIS_INTEGER range 1901 . . 2099;
    subtype MONTH_NUMBER is CAIS_INTEGER range 1 . . 12;
    subtype DAY_NUMBER is CAIS_INTEGER range 1 . . 31;
    subtype DAY_DURATION is CAIS_DURATION range 0.0 . . 86_400.0;

    TIME_ERROR: exception;

    function CLOCK
        (STATUS : in     STATUS_INDICATOR := NO_INDICATION)
        return TIME;

    function YEAR
        (DATE   : in     TIME;
         STATUS : in     STATUS_INDICATOR := NO_INDICATION)
        return YEAR_NUMBER;

    function MONTH
        (DATE   : in     TIME;
         STATUS : in     STATUS_INDICATOR := NO_INDICATION)
        return MONTH_NUMBER;

    function DAY
        (DATE   : in     TIME;
         STATUS : in     STATUS_INDICATOR := NO_INDICATION)
        return DAY_NUMBER;

    function SECONDS
        (DATE   : in     TIME;
         STATUS : in     STATUS_INDICATOR := NO_INDICATION)
        return DAY_DURATION;

    procedure SPLIT
        (DATE   : in     TIME ;
         YEAR   :    out YEAR_NUMBER;
         MONTH  :    out MONTH_NUMBER;
         DAY    :    out DAY_NUMBER;
         SECONDS :   out DAY_DURATION;
         STATUS : in     STATUS_INDICATOR := NO_INDICATION);
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
        function TIME_OF
            (YEAR       : in      YEAR_NUMBER;
             MONTH      : in      MONTH_NUMBER;
             DAY        : in      DAY_NUMBER;
             SECONDS    : in      DAY_DURATION;
             STATUS     : in      STATUS_INDICATOR  := NO_INDICATION)
            return TIME;

        function "+"
            (LEFT       : in      TIME ;
             RIGHT      : in      CAIS_DURATION)
            return TIME;

        function "+"
            (LEFT       : in      CAIS_DURATION;
             RIGHT      : in      TIME)
            return TIME;

        function "-"
            (LEFT       : in      TIME ;
             RIGHT      : in      CAIS_DURATION)
            return TIME;

        function "-"
            (LEFT       : in      TIME;
             RIGHT      : in      TIME)
            return CAIS_DURATION;

        function "<"
            (LEFT       : in      TIME;
             RIGHT      : in      TIME)
            return BOOLEAN;

        function "<="
            (LEFT       : in      TIME;
             RIGHT      : in      TIME)
            return BOOLEAN;

        function ">"
            (LEFT       : in      TIME;
             RIGHT      : in      TIME)
            return BOOLEAN;

        function ">="
            (LEFT       : in      TIME ;
             RIGHT      : in      TIME)
            return BOOLEAN;

    private

        type TIME is (IMPLEMEMTATION_DEFINED);
        --Thin type should be defined by the implementer.

    and CAIS_CALENDAR;
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
with CAIS_DEFINITIONS;
with  CAIS_STATUS_DEFINITIONS;

package CAIS_RESOURCE_CONTROL is

        use CAIS_DEFINITIONS;
        use  CAIS_STATUS_DEFINITIONS;

        procodure  REASSIGN_NODE
             (NODE                 :  in       NODE_TYPE;
              MEDIUM_RESOURCE : in       MEDIUM_NODE_TYPE;
              STATUS               :  in       STATUS_INDICATOR  :=  NO_INDICATION);

        procedure  GET_MEDIUM_RESOURCE
             (MEDIUM_RESOURCE :  in  out  MEDIUM_NODE_TYPE;
              NODE                 :  in       NODE_TYPE;
              STATUS               :  in       STATUS_INDICATOR   :=  NO_INDICATIOR);

        procedure   GET_PROCESSOR_RESOURCE
             (PROCESSOR_RESOURCE  :  in out PROCESSOR_NODE_TYPE;
              PROCESS_NODE         :  in       PROCESS_NODE_TYPE;
              STATUS                  :  in       STATUS_INDICATOR   := NOP_INDICATION);

        function DEFAULT_MEDIUM
             (STATUS :   in STATUS_INDICATOR := NO_INDICATION)
             return   MEDIUM_NODE_TYPE;

        function  DEFAULT_PROCESSOR
             (STATUS :   in STATUS_INDICATOR := NO_INDICATION)
             return PROCESSOR_NODE_TYPE;

end   CAIS_RESOURCE_CONTROL;
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
with CAIS_STATUS_DEFINITIONS;
with CAIS_DEFINITIONS;

package CAIS_ACCESS_CONTROL_MANAGEMENT is

    use CAIS_DEFINITIONS;
    use CAIS_STATUS_DEFINITIONS;

    subtype GRANT_VALUE     is DISCRETIONARY_ACCESS_LIST;
    subtype ACCESS_RIGHTS is STRING;

    function ALL_RIGETS
        (STATUS : in        STATUS_INDICATOR  := NO_INDICATION)
        return DISCRETIONARY_ACCESS_LIST;

    procedure SET_GRANTED_RIGHTS
        (MODE          : in      MODE_TYPE;
         GROUP_MODE : in      GROUP_NODE_TYPE;
         GRANT         : in      DISCRETIONARY_ACCESS_LIST;
         STATUS        : in      STATUS_INDICATOR  := NO_INDICATION);

    procedure SET_GRANTED_RIGHTS
        (NAME          : in      PATHNAME;
         GROUP_NAME : in      PATHNAME;
         GRAMT         : in      DISCRETIONARY_ACCESS_LIST;
         STATUS        : in      STATUS_INDICATOR  := NO_INDICATION);

    procedure DELETE_GRANTED_RIGHTS
        (NODE          : in      NODE_TYPE;
         GROUP_NODE : in      GROUP_NODE_TYPE ;
         STATUS        : in      STATUS_INDICATOR  := NO_INDICATION);

    procedure DELETE_GRANTED_RIGHTS
        (NAME     :     in      PATHNAME;
         GROUP_NAME : in      PATHNAME;
         STATUS        : in      STATUS_INDICATOR  := NO_INDICATION);

    procedure GET_GRANTED_RIGHTS
        (NODE          : in      NODE_TYPE;
         GROUP_MODE : in      GROUP_NODE_TYPE;
         GRANT         : in out DISCRETIONARY_ACCESS_LIST;
         STATUS        : in      STATUS_INDICATOR  := NO_INDICATION);

    procedure GET_GRANTED_RIGHTS
        (NAME          : in      PATHNAME;
         GROUP_NAME : in      PATHNAME;
         GRANT         : in  out  DISCRETIONARY_ACCESS_LIST;
         STATUS        : in      STATUS_INDICATOR  := NO_INDICATION);

    procedure  DENY_ACCESS
        (MODE          : in      NODE_TYPE;
         GROUP_MODE : in      GROUP_MODE_TYPE;
         STATUS        : in      STATUS_INDICATOR  := NO_INDICATION);
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
        procedure DELETE_DENY_RELATIONSHIP
            (NODE       : in    MODE_TYPE;
             GROUP_MODE : in    GROUP_NODE_TYPE ;
             STATUS     : in    STATUS_INDICATOR  := NO_INDICATION);

        function  IS_APPROVED
            (OBJECT_NODE : in    MODE_TYPE;
             ACCESS_RIGET : in   ACCESS_RIGHTS;
             STATUS      : in    STATUS_INDICATOR  := NO_INDICATION)
            return BOOLEAN;

        function  IS_APPROVED
            (OBJECT_NAME : in    PATHNAME;
             ACCESS_RIGHT : in   ACCESS_RIGHTS;
             STATUS      : in    STATUS_INDICATOR  := NO_INDICATION)
              return BOOLEAN;

        procedure ADOPT_ROLE
            (GROUP_MODE : in    GROUP_MODE_TYPE;
             KEY        : in    RELATIONSHIP_KEY  := EMPTY_KEY_LIST;
             INHERITABLE : in   BOOLEAN           := TRUE;
             STATUS     : in    STATUS_INDICATOR  := NO_INDICATION);

        procedure ADOPT_ROLE
            (GROUP_NAME : in    PATHNAME;
             KEY        : in    RELATIONSHIP_KEY := LATEST_KEY;
             INHERITABLE : in   BOOLEAN           := TRUE;
             STATUS     : in    STATUS_INDICATOR  := NO_INDICATION);

          procedure  UNADOPT_ROLE
            (KEY      : in    RELATIONSHIP_KEY;
             STATUS : in      STATUS_INDICATOR  := NO_INDICATION);

end CAIS_ACCESS_CONTROL_MANAGEMENT;
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
with CAIS_DEFINITIONS;
with CAIS_STANDARD;
with CAIS_STATUS_DEFINITIONS;

package CAIS_TRANSLATION_MANAGEMENT is

    use CAIS_DEFINITIONS;
    use CAIS_STANDARD;
    use CAIS_STATUS_DEFINITIONS;

    subtype DEFINITION_NAME is STRING;

    procedure ADOPT_VIEW
        (NODE        : in  VIEW_NODE_TYPE;
         INHERITABLE : in  BOOLEAN           := FALSE;
         STATUS      : in  STATUS_INDICATOR  := NO_INDICATION);

    procedure UHADOPT_VIEW
        (NODE   : in  VIEW_NODE_TYPE ;
         STATUS : in  STATUS_INDICATON  := NO_INDICATION);

    procedure ADOPT_INTERPRETATION
        (MODE           : in out NODE_TYPE;
         INTERPRETATION : in  NKI_TYPE;
         STATUS         : in  STATUS_INDICATOR  := NO_INDICATION);

    procedure ADOPT_INTERPRETATION
        (NODE           : in out NODE_TYPE;
         INTERPRETATION : in  NODE_KIND_NAME;
         STATUS         : in  STATUS_INDICATOR  := NO_INDICATION);

     procedure UNADOPT_INTERPRETATION
         (NODE           : in out NODE_TYPE;
          INTERPRETATION : in  NKI_TYPE          := NULL_NODE;
          STATUS         : in  STATUS_INDICATOR  := NO_INDICATION);

    procedure UNADOPT_INTERPRETATION
        (NODE           : in out NODE_TYPE;
         INTERPRETATION : in  NODE_KIND_NAME;
         STATUS         : in  STATUS_INDICATOR  := NO_INDICATION);

    procedure 0PEN_NODE_KIND_DEFINITION
        (NODE   : in out NKD_TYPE;
         KIND   : in  NODE_KIND_NAME ;
         STATUS : in  STATUS_INDICATOR  := NO_INDICATION);

    procedure OPEN_NODE_KIND
        (NODE   : in out NKI_TYPE;
         KIND   : in  NODE_KIND_NAME  ;
         STATUS : in  STATUS_INDICATOR  := NO_INDICATION);
```


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
procedure OPEN_RELATION_DEFINITION
    (NODE   : in out RD_TYPE;
     KIND   : in     RELATION_ NAME;
     STATUS : in     STATUS_INDICATOR := NO_INDICATION);

procedure OPEN_RELATION
    (NODE   : in out RI_TYPE;
     KIND   : in     RELATION_NAME ;
     STATUS : in     STATUS_INDICATOR := NO_INDICATION);

procedure OPEN_ATTRIBUTE_DEFINITION
    (NODE   : in out AD_TYPE;
     NAME   : in     ATTRIBUTE_NAME ;
     STATUS : in     STATUS_INDICATOR := NO_INDICATION);

procedure OPEN_ATTRIBUTE_KIND_DEFINITION
    (NODE   : in out AKD_TYPE;
     KIND   : in     ATTRIBUTE_KIND_NAME ;
     STATUS : in     STATUS_INDICATOR := NO_INDICATION);

function NAME_OF_DEFINITION
    (DEFINITION : in     DEFINITION_NODE_TYPE ;
     STATUS     : in     STATUS_INDICATOR := NO_INDICATION)
    return DEFINITION_NAME;

procedure DEFINITION_OF_NODE
    (DEFINITION : in out NKD_TYPE;
     NODE       : in     NODE_TYPE;
     STATUS     : in     STATUS_INDICATOR := NO_INDICATION);

function NAME_OF_RELATION
    (BASE     : in     NODE_TYPE;
     RELATION : in     RD_TYPE ;
     STATUS   : in     STATUS_INDICATOR := NO_INDICATION)
    return RELATION_NAME;

function NAME_OF_RELATION
    (BASE     : in     NODE_TYPE;
     KEY      : in     RELATIONSHIP_KEY ;
     RELATION : in     RD_TYPE ;
     STATUS   : in     STATUS_INDICATOR := NO_INDICATION)
    return RELATION_NANE;

procedure DEFINITION_OF_RELATION
    (DEFINITION : in out RD_TYPE;
     BASE       : in     NODE_TYPE;
     RELATION   : in     RELATION_NAME ;
     STATUS     : in     STATUS_INDICATOR := NO_INDICATION);

procedure DEFINITION_OF_RELATION
    (DEFINITION : in out RD_TYPE;
     BASE       : in     NODE_TYPE;
     KEY        : in     RELATIONSHIP_KEY ;
     RELATION   : in     RD_TYPE ;
     STATUS     : in     STATUS_INDICATOR := NO_INDICATION);
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
        procedure DEFINITION_ OF_ RELATION
            (DEFINITION :  in out RD_TYPE;
             BASE          :  in       NODE_TYPE;
             KEY           :  in       RELATIONSHIP_KEY ;
             RELATION      :  in       RELATION_NAME ;
             STATUS        :  in       STATUS_INDICATOR  :=  NO_INDICATION);

        function  NAME_OF_NODE_ATTRIBUTE
            (NoDE          :  in       NODE_TYPE;
             ATTRIBUTE :  in       AD_TYPE;
             STATUS        :  in       STATUS_INDICATOR  := NO_INDICATION)
            return  ATTRIBUTE_NAME;

        procedure  DEFINITION_OF_NODE_ATTRIBUTE
            (DEFINITION :  in out AD_TYPE;
             NODE          :  in       NODE_TYPE;
             ATTRIBUTE :  in       ATTRIBUTE_NAME ;
             STATUS        :  in       STATUS_INDICATOR  :=  NO_INDICATION);

        function   NAME_OF_PATH_ATTRIBUTE
            (RELATION      :  in       RD_TYPE ;
             ATTRIBUTE :  in       AD_TYPE;
             STATUS        :  in       STATUS_INDICATOR  := NO_INDICATION)
            return  ATTRIBUTE_NAME;

        procedure   DEFINITION_OF_PATH_ATTRIBUTE
            (DEFINITION :  in out AD_TYPE;
             RELATION      :  in       RD_TYPE ;
             ATTRIBUTE :  in       ATTRIBUTE_NAME ;
             STATUS        :  in       STATUS_INDICATOR  :=  NO_INDICATION);

        procedure   DEFINITION_OF_PATH_ATTRIBUTE
            (DEFINITION :  in out AD_TYPE;
             BASE          :  in       NODE_TYPE;
             RELATION      :  in       RELATION_NAME ;
             ATTRIBUTE     :  in       ATTRIBUTE_NAME ;
             STATUS        :  in       STATUS_INDICATOR  :=  NO_INDICATION);

    end CAIS_TRANSLATION_MANAGEMENT;
```


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
with CAIS_STANDARD;
with CAIS_LIST_MANAGEMENT;
with CAIS_DEFINITIONS;
with CAIS_ACCESS_CONTROL_MANAGEMENT;
with CAIS_STATUS_DEFINITIONS;
with CAIS_RESOURCE_CONTROL;

package CAIS_DEFINITION_MANAGEMENT is

    use CAIS_STANDARD;
    use CAIS_LIST_MANAGEMENT;
    use CAIS_DEFINITIONS;
    use CAIS_STATUS_DEFINITION;
    use CAIS_RESOURCE_CONTROL;
    use CAIS_ACCESS_CONTROL_MANAGEMENT;

    type RELATIONSHIP_KIND is (PRIMARY, SECONDARY);

    type VALUE_TYPE is (INTEGER,
                        FLOAT,
                        STRING,
                        IDENTIFIER,
                        VARIANT,
                        LIST,
                        UNIFORM_LIST,
                        COMPOSITE_LIST);

    function VIEW_NKD
        (STATUS : in      STATUS_INDICATOR  := NO_INDICATION)
        return NXD_TYPE;

    function  NODE_KIND_INTERPRETATION_NKD
        (STATUS : in      STATUS_INDICATON  := NO_INDICATION)
        return NKD_TYPE;

    function NODE_KIND_DEFINITION_NKD
        (STATUS : in      STATUS_INDICATOR  := NO_INDICATION)
        return NKD_TYPE;

    function  RELATION_INTERPRETATION_NKD
        (STATUS : in      STATUS_INDICATOR  := NO_INDICATION)
        return NKD_TYPE;

    function  RELATION_DEFINITION_NKD
        (STATUS : in      STATUS_INDICATOR  := NO_INDICATION)
        return NKD_TYPE;

    function  ATTRIBUTE_DEFINITION_NKD
        (STATUS : in      STATUS_INDICATOR  := NO_INDICATION)
        return NKD_TYPE;
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
function NAMED_COMPOSITE_LIST_AKD_NKD
    (STATUS : in       STATUS_INDICATOR  := NO_INDICATION)
    return  NKD_TYPE;

function  IDENTIFIER_ATTRIBUTE_KIND_DEFINITION_NKD
    (STATUS : in       STATUS_INDICATOR  := NO_INDICATION)
    return  NKD_TYPE;

procedure  CREATE_VIEW
    (VIEW                    : in out  VIEW_NODE_TYPE;
     BASE                    : in      NODE_TYPE;
     KEY                     : in      RELATIONSHIP_KEY           :=  EMPTY_KEY_LIST;
     ATTRIBUTES              : in      ATTRIBUTE_LIST             :=  EMPTY_LIST;
     DISCRETIONARY_ACCESS : in         DISCRETIONARY_ACCESS_LIST  :=  ALL_RIGHTS;
     MANDATORY_ACCESS        : in      MANDATORY_ACCESS_LIST      :=  EMPTY_LIST;
     KIND                    : in      NODE_KIND_NODE_TYPE        :=  VIEW_NKD;
     STATUS                  : in      STATUS_INDICATOR           :=  NO_INDICATION;
     MEDIUM_RESOURCE         : in      MEDIUM_NODE_TYPE           :=  DEFAULT_MEDIUM);

procedure  CREATE_VIEW
    (VIEW                    : in out  VIEW_NODE_TYPE ;
     BASE                    : in      NODE_TYPE;
     KEY                     : in      RELATIONSHIP_KEY           :=  EMPTY_KEY_LIST;
     ATTRIBUTES              : in      ATTRIBUTE_LIST             :=  EMPTY_LIST;
     DISCRETIONARY_ACCESS : in         DISCRETIONARY_ACCESS_LIST  :=  ALL_RIGHTS;
     MANDATORY_ACCESS        : in      MANDATORY_ACCESS_LIST      :=  EMPTY_LIST;
     KIND                    : in      NODE_KIND_NAME;
     STATUS                  : in      STATUS_INDICATOR           :=  NO_INDICATION;
     MEDIUM_RESOURCE         : in      MEDIUM_NODE_TYPE           :=  DEFAULT_MEDIUM);

procedure DELETE_VIEW
    (NODE     : in out VIEW_NODE_TYPE;
     STATUS : in       STATUS_INDICATOR  := NO_INDICATION);

procedure CREATE_NODE_KIND
    (NKI                          : in  out  NKI_TYPE;
     NKD                          : in  out  NKD_TYPE;
     VIEW                         : in       VIEW_NODE_TYPE ;
     SPECIALIZATION_OF            : in       NKD_TYPE;
     BASE                         : in       NODE_TYPE;
     LOCAL_NAME                   : in       RELATIONSHIP_KEY ;
     KEY                          : in       RELATIONSHIP_KEY          := EMPTY_KEY_LIST;
     SPECIALIZATION_OF_KEY        : in       RELATIONSHIP_KEY          := EMPTY_KEY_LIST;
     GENERALIZATION_OF_KEY        : in       RELATIONSHIP_KEY          := EMPTY_KEY_LIST;
     INTERPRETATION_ATTRIBUTES : in          ATTRIBUTE_LIST            := EMPTY_LIST;
     DEFINITION_ATTRIBUTES        : in       ATTRIBUTE_LIST            := EMPTY_LIST;
     DISCRETIONARY_ACCESS         : in       DISCRETIONARY_ACCESS_LIST := ALL_RIGHTS;
     MANDATORY_ACCESS             : in       MANDATORY_ACCESS_LIST := EMPTY_LIST;
     INTERPRETATION_KIND          : in       NODE_KIND_NODE_TYPE :=
                                                NODE_KIND_INTERPRETATION_NKD;
     DEFINITION_KIND              : in       NODE_KIND_NODE_TYPE :=
                                                NODE_KIND_DEFINITION_NKD;
     STATUS                       : in       STATUS_INDICATOR          := NO_INDICATION;
     INTERPRETATION_MEDIUM        : in       MEDIUM_NODE_TYPE          := DEFAULT_MEDIUM;
     DEFINITION_MEDIUM            : in       MEDIUM_NODE_TYPE          := DEFAULT_MEDIUM);
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
procedure CREATE_NODE_KIND
    (NKI                          : in out NKI_TYPE;
     NKD                          : in out NKD_TYPE;
     VIEW                         : in     VIEW_NODE_TYPE;
     SPECIALIZATION_OF            : in     NKD_TYPE;
     BASE                         : in     NODE_TYPE;
     LOCAL_NAME                   : in     RELATIONSHIP_KEY;
     KEY                          : in     RELATIONSHIP_KEY          := EMPTY_KEY_LIST;
     SPECIALIZATION_OF_KEY        : in     RELATIONSHIP_KEY          := EMPTY_KEY_LIST;
     GENERALIZATION_OF_KEY        : in     RELATIONSHIP_KEY          := EMPTY_KEY_LIST;
     INTERPRETATION_ATTRIBUTES    : in     ATTRIBUTE_LIST            := EMPTY_LIST;
     DEFINITION_ATTRIBUTES        : in     ATTRIBUTE_LIST            := EMPTY_LIST;
     DISCRETIONARY_ACCESS         : in     DISCRETIONARY_ACCESS_LIST := ALL_RIGHTS;
     MANDATORY_ACCESS             : in     MANDATORY_ACCESS_LIST := EMPTY_LIST;
     INTERPRETATION_KIND          : in     NODE_KIND_NAME;
     DEFINITION_KIND              : in     NODE_KIND_NAME            :=
                                               "NODE_KIND_DEFINITION";
     STATUS                       : in     STATUS_INDICATOR          := NO_INDICATION;
     INTERPRETATION_MEDIUM        : in     MEDIUM_NODE_TYPE          := DEFAULT_MEDIUM;
     DEFINITION_MEDIUM            : in     MEDIUM_NODE_TYPE          := DEFAULT_MEDIUM);

procedure DELETE_NODE_KIND
    (NKD                : in out NKD_TYPE;
     VIEW               : in     VIEW_NODE_TYPE;
     DELETE_DEFINITION  : in     BOOLEAN          := FALSE;
     TIME_LIMIT         : in     CAIS_DURATION    := LONG_DELAY;
     STATUS             : in     STATUS_INDICATOR := NO_INDICATION);

procedure IMPORT_NODE_KIND
    (NKI                : in out NKI_TYPE;
     VIEW               : in     VIEW_NODE_TYPE;
     NKD                : in     NKD_TYPE;
     LOCAL_NAME         : in     RELATIONSHIP_KEY;
     ATTRIBUTES         : in     ATTRIBUTE_LIST            := EMPTY_LIST;
     DISCRETIONARY_ACCESS : in   DISCRETIONARY_ACCESS_LIST := ALL_RIGHTS;
     MANDATORY_ACCESS   : in     MANDATORY_ACCESS_LIST := EMPTY_LIST;
     INTERPRETATION_KIND : in    NODE_KIND_NODE_TYPE :=
                                     NODE_KIND_INTERPRETATION_NKD;
     STATUS             : in     STATUS_INDICATOR          := NO_INDICATION;
     MEDIUM_RESOURCE    : in     MEDIUM_NODE_TYPE          := DEFAULT_MEDIUM);

procedure IMPORT_NODE_KIND
    (NKI                : in out NKI_TYPE;
     VIEW               : in     VIEW_NODE_TYPE;
     NKD                : in     NKD_TYPE;
     LOCAL_NAME         : in     RELATIONSHIP_KEY;
     ATTRIBUTES         : in     ATTRIBUTE_LIST            := EMPTY_LIST;
     DISCRETIONARY_ACCESS : in   DISCRETIONARY_ACCESS_LIST := ALL_RIGHTS;
     MANDATORY_ACCESS   : in     MANDATORY_ACCESS_LIST := EMPTY_LIST;
     INTERPRETATION_KIND : in    NODE_KIND_NAME;
     STATUS             : in     STATUS_INDICATOR          := NO_INDICATION;
     MEDIUM_RESOURCE    : in     MEDIUM_NODE_TYPE          := DEFAULT_MEDIUM);

procedure RENAME_NODE_KIND
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
    (VIEW       : in    VIEW_MODE_TYPE ;
     OLD_NAME   : in    RELATIONSHIP_KEY;
     NEW_NAME   : in    RELATIONSHIP_KEY;
     STATUS     : in    STATUS_INDICATOR  := NO_INDICATION);

procedure  SPECIALIZE_NODE_KIND
    (NKD            : in    NKD_TYPE;
     ANCESTOR       : in    NKD_TYPE;
     KEY            : in    RELATIONSHIP_KEY  := EMPTY_KEY_LIST;
     REVERSE_KEY    : in    RELATIONSHIP_KEY  := EMPTY_KEY_LIST;
     STATUS         : in    STATUS_INDICATOR  := NO_INDICATION);

procedure GENERALIZE_NODE_KIND
    (NKD                        : in    NKD_TYPE;
     DESCENDANTS                : in    NODE_HANDLE_ARRAY;
     KEY_GENERATOR              : in    RELATIONSHIP_KEY := EMPTY_KEY_LIST;
     REVERSE_KEY_GENERATOR      : in    RELATIONSHIP_KEY := EMPTY_KEY_LIST;
     VIEW                       : in    VIEW_NODE_TYPE := NULL_NODE;
     STATUS                     : in    STATUS_INDICATOR := NO_INDICATION) ;

procedure DELETE_NODE_KIND_SPECIALIZATION
    (DESCENDANT : in    NKD_TYPE;
     ANCESTOR   : in    NKD_TYPE;
     STATUS     : in    STATUS_INDICATOR  := NO_INDICATION);

procedure  CREATE_RELATION
    (RI                         : in out RI_TYPE;
     RD                         : in out RD_TYPE;
     VIEW                       : in    VIEW_NODE_TYPE;
     SPECIALIZATION_OF          : in    RD_TYPE;
     BASE                       : in    NODE_TYPE;
     LOCAL_NAME                 : in    RELATIONSHIP_KEY ;
     KEY                        : in    RELATIONSHIP_KEY  := EMPTY_KEY_LIST;
     SPECIALIZATION_OF_KEY      : in    RELATIONSHIP_KEY  := EMPTY_KEY_LIST;
     GENERALIZATION_OF_KEY      : in    RELATIONSHIP_KEY  := EMPTY_KEY_LIST;
     INTERPRETATION_ATTRIBUTES  : in    ATTRIBUTE_LIST    := EMPTY_LIST;
     DEFINITION_ATTRIBUTES      : in    ATTRIBUTE_LIST    := EMPTY_LIST;
     DISCRETIONARY_ACCESS       : in    DISCRETIONARY_ACCESS_LIST  := ALL_RIGHTS;
     MANDATORY_ACCESS           : in    MANDATORY_ACCESS_LIST := EMPTY_LIST;
     INTERPRETATION_KIND        : in    NODE_KIND_NODE_TYPE :=
                                          RELATION_INTERPRETATION_NKD;
     DEFINITION_KIND            : in    NODE_KIND_NODE_TYPE :=
                                          RELATION_DEFINITION_NKD;
     STATUS                     : in    STATUS_INDICATOR  := NO_INDICATION;
     INTERPRETATION_MEDIUM      : in    MEDIUM_NODE_TYPE  := DEFAULT_MEDIUM;
     DEFINITION_MEDIUM          : in    MEDIUM_NODE_TYPE  := DEFAULT_MEDIUM);

procedure  CREATE_RELATION
    (RI                         : in out RI_TYPE;
     RD                         : in out RD_TYPE;
     VIEW                       : in    VIEW_NODE_TYPE;
     SPECIALIZATION_OF          : in    RD_TYPE;
     BASE                       : in    NODE_TYPE;
```


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
        LOCAL_NAME                  : in    RELATIONSHIP_KEY;
        KEY                         : in    RELATIONSHIP_KEY   := EMPTY_KEY_LIST;
        SPECIALIZATION_OF_KEY       : in    RELATIONSHIP_KEY   := EMPTY_KEY_LIST;
        GENERALIZATION_OF_KEY       : in    RELATIONSHIP_KEY   := EMPTY_KEY_LIST;
        INTERPRETATION_ATTRIBUTES : in      ATTRIBUTE_LIST     := EMPTY_LIST;
        DEFINITION_ATTRIBUTES       : in    ATTRIBUTE_LIST     := EMPTY_LIST;
        DISCRETIONARY_ACCESS        : in    DISCRETIONARY_ACCESS_LIST := ALL_RIGHTS;
        MANDATORY_ACCESS            : in    MANDATORY_ACCESS_LIST := EMPTY_LIST;
        INTERPRETATION_KIND         : in    NODE_KIND_NAME;
        DEFINITION_KIND             : in    NODE_KIND_NAME :=
                                              "RELATION_DEFINITION";
        STATUS                      : in    STATUS_INDICATOR := NO_INDICATION;
        INTERPRETATION_MEDIUM       : in    MEDIUM_NODE_TYPE := DEFAULT_MEDIUM;
        DEFINITION_MEDIUM           : in    MEDIUM_NODE_TYPE := DEFAULT_MEDIUM) ;


    procedure DELETE_RELATION
        (RD                         : in out RD_TYPE;
        VIEW                        : in    VIEW_NODE_TYPE;
        DELETE_DEFINITION : in      BOOLEAN          := FALSE ;
        TIME_LIMIT                  : in    CAIS_DURATION := LONG_DELAY;
        STATUS                      : in    STATUS_INDICATOR := NO_INDICATION);


    procedure IMPORT_RELATION
        (RI                         : in out RI_TYPE;
        VIEW                        : in    VIEW_NODE_TYPE;
        RD                          : in    RD_TYPE;
        LOCAL_NAME                  : in    RELATIONSHIP_KEY;
        ATTRIBUTES                  : in    ATTRIBUTE_LIST          := EMPTY_LIST;
        DISCRETIONARY_ACCESS : in   DISCRETIONARY_ACCESS_LIST := ALL_RIGHTS;
        MANDATORY_ACCESS            : in    MANDATORY_ACCESS_LIST := EMPTY_LIST;
        INTERPRETATION_KIND         : in    NODE_KIND_NODE_TYPE :=
                                              RELATION_INTERPRETATION_NKD;
        STATUS                      : in    STATUS_INDICATOR        := NO_INDICATION;
        MEDIUM_RESOURCE             : in    MEDIUM_NODE_TYPE        := DEFAULT_MEDIUM);


    procedure IMPORT_RELATION
        (RI                         : in out RI_TYPE;
        VIEW                        : in    VIEW_NODE_TYPE;
        RD                          : in    RD_TYPE;
        LOCAL_NAME                  : in    RELATIONSHIP_KEY;
        ATTRIBUTES                  : in    ATTRIBUTE_LIST          := EMPTY_LIST;
        DISCRETIONARY_ACCESS : in   DISCRETIONARY_ACCESS_LIST := ALL_RIGHTS;
        MANDATORY_ACCESS            : in    MANDATORY_ACCESS_LIST := EMPTY_LIST;
        INTERPRETATION_KIND : in    NODE_KIND_NAME;
        STATUS                      : in    STATUS_INDICATOR := NO_INDICATION;
        MEDIUM_RESOURCE             : in    MEDIUM_NODE_TYPE := DEFAULT_MEDIUM);


    procedure RENAME_RELATION
        (VIEW       : in    VIEW_NODE_TYPE;
        OLD_NAME : in       RELATIONSHIP_KEY;
        NEW_NAME : in       RELATIONSHIP_KEY;
        STATUS      : in    STATUS_INDICATOR := NO_INDICATION);
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
procedure SPECIALIZE_RELATION
    (RD              : in     RD_TYPE;
     ANCESTOR        : in     RD_TYPE;
     KEY             : in     RELATIONSHIP_KEY  :=  EMPTY_KEY_LIST;
     REVERSE_KEY : in     RELATIONSHIP_KEY  :=  EMPTY_KEY_LIST;
     STATUS          : in     STATUS_INDICATOR :=  NO_INDICATION);

procedure  GENERALIZE_RELATION
    (RD                          : in     RD_TYPE ;
     DESCENDANTS                 : in     NODE_HAHDLE_ARRAY;
     KEY_GENERATOR               : in     RELATIONSHIP.KEY :=  EMPTY_KEY_LIST;
     REVERSE_KEY_GENERATOR : in     RELATIONSHIP_KEY  :=  EMPTY_KEY_LIST;
     VIEW                        : in     VIEW_NODE_TYPE :=  NULL_NODE;
     STATUS                      : in     STATUS_INDICATOR :=  NO_INDICATIO);

procedure  DELETE_RELATION_SPECIALIZATION
    (DESCENDANT : in     RD_TYPE;
     ANCESTOR        : in     RD_TYPE;
     STATUS          : in     STATUS_INDICATOR   := NO_INDICATION);

procedure CREATE_BIDIRECTIONAL_RELATION
    (RI1                              : in  out  RI_TYPE;
     RI2                              : in  out  RI_TYPE;
     RD1                              : in  out  RD_TYPE;
     RD2                              : in  out  RD_TYPE;
     VIEW                             : in       VIEW_NODE_TYPE;
     SPECIALIZATION_OF1               : in       RD_TYPE;
     SPECIALIZATION_OF2               : in       RD_TYPE;
     BASE1                            : in       NODE_TYPE;
     BASE2                            : in       NODE_TYPE;
     LOCAL_NAMEl                      : in       RELATIONSHIP_KEY;
     LOCAL_NAME2                      : in       RELATIONSHIP_KEY;
     KEY1                             : in       RELATIONSHIP_KEY  :=  EMPTY_KEY_LIST;
     KEY2                             : in       RELATIONSHIP_KEY  :=  EMPTY_KEY_LIST;
     SPECIALIZATION_OF1_KEY           : in       RELATIONSEIP_KEY  :=  EMPTY_KEY_LIST;
     SPECIALIZATION_OF2_KEY           : in       RELATIONSHIP_KEY  :=  EMPTY_KEY_LIST;
     GENERALIZATION_OFl_KEY           : in       RELATIONSHIP_KEY  :=  EMPTY_KEY_LIST;
     GENERALIZATION_OF2_KEY           : in       RELATIONSHIP_KEY  :=  EMPTY_KEY_LIST;
     INTERPRETATION_ATTRIBUTES1 : in       ATTRIBUTE_LIST :=  EMPTY_LIST;
     INTERPRETATION_ATTRIBUTES2 : in       ATTRIBUTE_LIST :=  EMPTY_LIST;
     DEFINITION_ATTRIBUTES1           : in       ATTRIBUTE_LIST :=  EMPTY_LIST;
     DEFINITION_ATTRIBUTES2           : in       ATTRIBUTE_LIST :=  EMPTY_LIST;
     DISCRETIONARY_ACCESS             : in       DISCRETIONARY_ACCESS_LIST  :=  ALL_RIGHTS;
     MANDATORY_ACCESS                 : in       MANDATORY_ACCESS_LIST          := EMPTY_LIST;
     INTERPRETATION_KINDl             : in       NODE_KIND_MODE_TYPE            :=
                                                     RELATION_INTERPRETATION_NKD;
     INTERPRETATION_KIND2             : in       NODE_KIND_NODE_TYPE            :=
                                                     RELATION_INTERPRETATION_NKD;
     DEFINITION_KINDl                 : in       NODE_KIND_NODE_TYPE            :=
                                                     RELATION_DEFINITION_NKD;
     DEFINITION_KIND2                 : in       NODE_KIND_NODE_TYPE   :=
                                                     RELATION_DEFINITION_NKD;
     STATUS                           : in       STATUS_INDICATOR  :=  NO_INDICATION;
     INTERPRETATION_MEDIUM            : in       MEDIUM_NODE_TYPE   := DEFAULT_MEDIUM;
     DEFINITION_MEDIUM                : in       MEDIUM_NODE_TYPE   :=  DEFAULT_MEDIUM);
```



Source: https://assist.dla.mil -- Downloaded: 2026l0-26T07:54Z
Check the source to verify that this is the currentversion before use.

```
procedure CREATE_BIDIRECTIONAL_RELATION
    (RI                          : in out RI_TYPE;
     RD                          : in out RD_TYPE;
     VIEW                        : in     VIEW_NODE_TYPE;
     SPECIALIZATION_OF           : in     RD_TYPE;
     BASE                        : in     NODE_TYPE;
     LOCAL_NAME                  : in     RELATIONSHIP_KEY;
     KEY                         : in     RELATIONSHIP_KEY := EMPTY_KEY_LIST;
     SPECIALIZATION_OF_KEY       : in     RELATIONSHIP_KEY := EMPTY_KEY_LIST;
     GENERALIZATION_OF_KEY       : in     RELATIOHSHIP_KEY := EMPTY_KEY_LIST;
     INTERPRETATION_ATTRIBUTES   : in     ATTRIBUTE_LIST := EMPTY_LIST;
     DEFINITION_ATTRIBUTES       : in     ATTRIBUTE_LIST := EMPTY_LIST;
     DISCRETIONARY_ACCESS        : in     DISCRETIONARY_ACCESS_LIST := ALL_RIGHTS;
     MANDATORY_ACCESS            : in     MANDATORY_ACCESS_LIST := EMPTY_LIST;
     INTERPRETATION_KIND         : in     NODE_KIND_NODE_TYPE :=
                                            RELATION_INTERPRETATION_NKD;
     DEFINITION_KIND             : in     NODE_KIND_NODE_TYPE :=
                                            RELATION_DEFINITION_NKD;
     STATUS                      : in     STATUS_INDICATOR := NO_INDICATION;
     INTERPRETATION_MEDIUM       : in     MEDIUM_NODE_TYPE := DEFAULT_MEDIUM;
     DEFINITION_MEDIUM           : in     MEDIUM_NODE_TYPE := DEFAULT_MEDIUM);

procedure CREATE_BIDIRECTIONAL_RELATION
    (RI1                         : in out RI_TYPE;
     RI2                         : in out RI_TYPE;
     RD1                         : in out RD_TYPE;
     RD2                         : in out RD_TYPE ;
     VIEW                        : in     VIEW_NODE_TYPE;
     SPECIALIZATION_OF1          : in     RD_TYPE;
     SPECIALIZATION_OF2          : in     RD_TYPE;
     BASE1                       : in     NODE_TYPE;
     BASE2                       : in     MODE_TYPE;
     LOCAL_NAMEl                 : in     RELATIONSHIP_KEY;
     LOCAL_NAME2                 : in     RELATIONSHIP_KEY;
     KEY1                        : in     RELATIONSHIP_KEY := EMPTY_KEY_LIST ;
     KEY2                        : in     RELATIONSHIP_KEY := EMPTY_KEY_LIST ;
     SPECIALIZATION_OFl_KEY      : in     RELATIONSHIP_KEY := EMPTY_KEY_LIST ;
     SPECIALIZATION_OF2_KEY      : in     RELATIONSHIP_KEY := EMPTY_KEY_LIST ;
     GENERALIZATION_OFl_KEY      : in     RELATIONSHIP_KEY := EMPTY_KEY_LIST ;
     GENERALIZATION_OF2_KEY      : in     RELATIONSHIP_KEY := EMPTY_KEY_LIST ;
     INTERPRETATION_ATTRIBUTESl  : in     ATTRIBUTE_LIST := EMPTY_LIST;
     INTERPRETATION_ATTRIBUTES2  : in     ATTRIBUTE_LIST := EMPTY_LIST;
     DEFINITION_ATTRIBUTESl      : in     ATTRIBUTE_LIST := EMPTY_LIST;
     DEFINITION_ATTRIBUTES2      : in     ATTRIBUTE_LIST := EMPTY_LIST;
     DISCRETIONARY_ACCESS        : in     DISCRETIONARY_ACCESS_LIST := ALL_RIGHTS;
     MANDATORY_ACCESS            : in     MANDATORY_ACCESS_LIST := EMPTY_LIST;
     INTERPRETATION_KINDl        : in     NODE_KIND_NAME;
     INTERPRETATION_KIND2        : in     NODE_KIND_NAME :=
                                            "RELATION_INTERPRETANION";
     DEFINITION_KINDl            : in     NODE_KIND_NAME :=
                                            "RELATION DEFINITION";
     DEFINITION_KIND2            : in     NODE_KIND_NAME :=
                                            "RELATION_DEFINITION";
     STATUS                      : in     STATUS_INDICATOR := NO_INDICATION;
     INTERPRETATION_MEDIUM       : in     MEDIUM_NODE_TYPE := DEFAULT_MEDIUM;
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
        DEFINITION_MEDIUM              : in    MEDIUM_NODE_TYPE := DEFAULT_MEDIUM);

    procedure  CREATE_BIDIRECTIONAL_RELATIOIN
        (RI                            : in out RI_TYPE;
         RD                            : in out RD_TYPE ;
         VIEW                          : in    VIEW_NODE_TYPE;
         SPECIALIZATION_OF             : in    RD_TYPE;
         BASE                          : in    NODE_TYPE;
         LOCAL_NAME                    : in    RELATIONSHIP_KEY;
         KEY                           : in    RELATIONSHIP_KEY  := EMPTY_KEY_LIST;
         SPECIALIZATION_OF_KEY         : in    RELATIONSHIP_KEY  := EMPTY_KEY_LIST;
         GENERALIZATION_OF_KEY         : in    RELATIONSHIP_KEY  := EMPTY_KEY_LIST;
         INTERPRETATION_ATTRIBUTES : in    ATTRIBUTE_LIST    := EMPTY_LIST;
         DEFINITION_ATTRIBUTES         : in    ATTRIBUTE_LIST    := EMPTY_LIST;
         DISCRETIONARY_ACCESS          : in    DISCRETIONARY_ACCESS_LIST := ALL_RIGHTS;
         MANDATORY_ACCESS              : in    MANDATORY_ACCESS_LIST := EMPTY_LIST;
         INTERPRETATION_KIND           : in    NODE_KIND_NAME;
         DEFINITION_KIND               : in    NODE_KIND_NAME    :=
                                                  "RELATION_DEFINITION";
         STATUS                        : in    STATUS_INDICATOR  := NO_INDICATION;
         INTERPRETATION_MEDIUM         : in    MEDIUM_NODE_TYPE  := DEFAULT_MEDIUM;
         DEFINITION_MEDIUM             : in    MEDIUM_NODE_TYPE  := DEFAULT_MEDIUM);

    procedure DELETE_BIDIRECTIONAL_RELATION
        (RD                    : in out RD_TYPE;
         VIEW                  : in    VIEW_NODE_TYPE;
         DELETE_DEFINITION : in    BOOLEAN          := FALSE;
         TIME_LIMIT            : in    CAIS_DURATION    := LONG_DELAY;
         STATUS                : in    STATUS_INDICATOR := NO_INDICATION);

    procedure  CREATE_ATTRIBUTE_DEFINITION
        (AD                    : in out AD_TYPE;
         VIEW                  : in    VIEW_NODE_TYPE;
         BASE                  : in    NODE_TYPE;
         ATTRIBUTE_KIND        : in    AKD_TYPE;
         LOCAL_NAME            : in    RELATIONSHIP_KEY;
         KEY                   : in    RELATIONSHIP_KEY  := EMPTY_KEY_LIST;
         ATTRIBUTE_KIND_KEY    : in    RELATIONSHIP_KEY  := EMPTY_KEY_LIST;
         ATTRIBUTES            : in    ATTRIBUTE_LIST    := EMPTY_LIST;
         DISCRETIONARY_ACCESS : in    DISCRETIONARY_ACCESS_LIST := ALL_RIGHTS;
         MANDATORY_ACCESS      : in    MANDATORY_ACCESS_LIST := EMPTY_LIST;
         KIND                  : in    NODE_KIND_NODE_TYPE :=
                                          ATTRIBUTE_DEFINITION_NKD ;
         STATUS                : in    STATUS_INDICATOR  := NO_INDICATION ;
         MEDIUM_RESOURCE       : in    MEDIUM_NODE_TYPE  := DEFAULT_MEDIUM) ;

    procedure  CREATE_ATTRIBUTE_DEFINITION
        (AD                    : in out AD_TYPE;
         VIEW                  : in    VIEW_NODE_TYPE;
         BASE                  : in    NODE_TYPE;
         ATTRIBUTE_KIND        : in    ATTRIBUTE_KIND_NAME;
         LOCAL_NAME            : in    RELATIONSHIP_KEY;
         KEY                   : in    RELATIONSHIP_KEY  := EMPTY_KEY_LIST;
         ATTRIBUTE_KIND_KEY    : in    RELATIONSHIP_KEY  := EMPTY_KEY_LIST;
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
        ATTRIBUTES              : in    ATTRIBUTE_LIST           := EMPTY_LIST;
        DISCRETIONARY_ACCESS    : in    DISCRETIONARY_ACCESS_LIST := ALL_RIGHTS;
        MANDATORY_ACCESS        : in    MANDATORY_ACCESS_LIST    := EMPTY_LIST;
        KIND                    : in    NODE_KIND_NAME           := "ATTRIBUTE_DEFINITION";
        STATUS                  : in    STATUS_INDICATOR         := NO_INDICATION;
        MEDIUM_RESOURCE         : in    MEDIUM_NODE_TYPE         := DEFAULT_MEDIUM);

    procedure DELETE_ATTRIBUTE_DEFINITION
        (AD                     : in out AD_TYPE;
        VIEW                    : in    VIEW_NODE_TYPE;
        DELETE_DEFINITION       : in    BOOLEAN          := FALSE ;
        TIME_LIMIT              : in    CAIS_DURATION    := LONG_DELAY;
        STATUS                  : in    STATUS_INDICATOR := NO_INDICATION);

    procedure IMPORT_ATTRIBUTE_DEFINITION
        (VIEW           : in    VIEW_NODE_TYPE;
        AD              : in    AD_TYPE;
        LOCAL_NAME      : in    RELATIONSHIP_KEY;
        STATUS          : in    STATUS_INDICATOR  := NO_INDICATION);

    procedure RENAME_ATTRIBUTE_DEFINITION
        (VIEW           : in    VIEW_NODE_TYPE;
        OLD_NAME        : in    RELATIONSHIP_KEY;
        NEW_NAME        : in    RELATIONSHIP_KEY;
        STATUS          : in    STATUS_INDICATOR  := NO_INDICATION);

    procedure REDEFINE_ATTRIBUTE_KIND
        (AD                     : in    AD_TYPE;
        AKD                     : in    AKD_TYPE;
        REVERSE_KEY             : in    RELATIONSHIP_KEY  := EMPTY_KEY_LIST;
        TIME_LIMIT              : in    CAIS_DURATION     := LONG_DELAY;
        STATUS                  : in    STATUS_INDICATOR  := NO_INDICATION);

    procedure CREATE_ATTRIBUTE_KIND
        (AKD                    : in out AKD_TYPE;
        VIEW                    : in    VIEW_NODE_TYPE;
        BASE                    : in    NODE_TYPE;
        LOCAL_NAME              : in    RELATIONSHIP_KEY;
        KEY                     : in    RELATIONSHIP_KEY          := EMPTY_KEY_LIST;
        ATTRIBUTES              : in    ATTRIBUTE_LIST            := EMPTY_LIST;
        DISCRETIONARY_ACCESS    : in    DISCRETIONARY_ACCESS_LIST := ALL_RIGHTS;
        MANDATORY_ACCESS        : in    MANDATORY_ACCESS_LIST     := EMPTY_LIST;
        KIND                    : in    NODE_KIND_NODE_TYPE :=
                                            NAMED_COMPOSITE_LIST_AKD_NKD;
        STATUS                  : in    STATUS_INDICATOR          := NO_INDICATION;
        MEDIUM_RESOURCE         : in    MEDIUM_NODE_TYPE          := DEFAULT_MEDIUM);

    procedure CREATE_ATTRIBUTE_KIND
        (AKD                    : in out AKD_TYPE;
        VIEW                    : in    VIEW_NODE_TYPE;
        BASE                    : in    NODE_TYPE;
        LOCAL_NAME              : in    RELATIONSHIP_KEY;
        KEY                     : in    RELATIONSHIP_KEY          := EMPTY_KEY_LIST;
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
        ATTRIBUTES                : in    ATTRIBUTE_LIST            := EMPTY_LIST;
        DISCRETIONARY_ACCESS : in    DISCRETIONARY_ACCESS_LIST  :=  ALL_RIGHTS;
        MANDATORY_ACCESS          : in    MANDATORY_ACCESS_LIST := EMPTY_LIST;
        KIND                      : in    NODE_KIND_NAME;
        STATUS                    : in    STATUS_INDICATOR          := NO_INDICATION;
        MEDIUM_RESOURCE           : in    MEDIUM_NODE_TYPE          :=  DEFAULT_MEDIUM);

    generic
        type ENUMERATION is (<>);
    procedure  CREATE_IDENTIFIER_ATTRIBUTE_KIND
        (AKD                      : in out AKD_TYPE;
        VIEW                      : in    VIEW_NODE_TYPE;
        BASE                      : in    NODE_TYPE;
        LOCAL_NAME                : in    RELATIONSHIP_KEY;
        KEY                       : in    RELATIONSHIP_KEY          := EMPTY_KEY_LIST;
        ATTRIBUTES                : in    ATTRIBUTE_LIST            := EMPTY_LIST;
        DISCRETIONARY_ACCESS : in    DISCRETIONARY_ACCESS_LIST  :=  ALL_RIGHTS;
        MANDATORY_ACCESS          : in    MAMDATORY_ACCESS_LIST  :=  EMPTY_LIST;
        KIND                      : in    NODE_KIND_NODE_TYPE       : =
                                              IDENTIFIER_ATTRIBUTE_KIND_DEFINITION_NKD;
        STATUS                    : in    STATUS_INDICATOR          := NO_INDICATION;
        MEDIUM_RESOURCE           : in    MEDIUM_NODE_TYPE          :=  DEFAULT_MEDIUM);

    generic
        typo ENUMERATION is (<>);
    procedure   CREATE_IDENTIFIER_ATTRIBUTE_KIND_BY_NAME
        (AKD                      : in out AKD_TYPE;
        VIEW                      : in    VIEW_MODE_TYPE;
        BASE                      : in    MODE_TYPE;
        LOCAL_NAME                : in    RELATIONSHIP_KEY;
        KEY                       : in    RELATIONSHIP_KEY          := EMPTY_KEY_LIST;
        ATTRIBUTES                : in    ATTRIBUTE_LIST            := EMPTY_LIST;
        DISCRETIONARY_ACCESS      : in    DISCRETIONARY_ACCESS_LIST  :=  ALL_RIGHTS;
        MANDATORY_ACCESS          : in    MANDATORY_ACCESS_LIST  :=  EMPTY_LIST;
        KIND                      : in    NODE_KIND_NAME;
        STATUS                    : in    STATUS_INDICATOR          :=  NO_INDICATION;
        MEDIUM_RESOURCE           : in    MEDIUM_NODE_TYPE          :=  DEFAULT_MEDIUM);

    procedure  DELETE_ATTRIBUTE_KIND
        (AKD                      : in out AKD_TYPE;
        VIEW                      : in    VIEW_NODE_TYPE;
        DELETE_DEFINITION : in    BOOLEAN                 : =  FALSE ;
        TIME_LIMIT                : in    CAIS_DURATION  :=  LONG_DELAY;
        STATUS                    : in    STATUS_INDICATOR :=  NO_INDICATION);

    procedure  IMPORT_ATTRIBUTE_KIND
        (VIEW                     : in    VIEW_NODE_TYPE;
        AKD                       : in    AKD_TYPE;
        LOCAL_NAME : in    RELATIONSHIP_KEY;
        STATUS                    : in    STATUS_INDICATOR   :=  NO_INDICATION);

    procedure   RENAME_ATTRIBUTE_KIND
        (VIEW                     : in    VIEW_NODE_TYPE;
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
        OLD_NAME : in      RELATIONSHIP_KEY;
        NEW_NAME : in      RELATIONSHIP_KEY ;
        STATUS   : in      STATUS_INDICATOR  := NO_INDICATION);

    procedure  DELETE_DEFINITION_TREE
        (NODE    : in out DEFINITION_NODE_TYPE;
        STATUS : in       STATUS_INDICATOR  := NO_INDICATION);

    procedure DEFINE_SOURCE
        (VIEW          : in      VIEW_NODE_TYPE;
        NKD            : in      NKD_TYPE;
        RD             : in      RD_TYPE ;
        LOCAL_NAME : in      RELATIONSHIP_KEY;
        KEY            : in      RELATIONSHIP_KEY  :=  EMPTY_KEY_LIST;
        REVERSE_KEY : in      RELATIONSHIP_KEY  :=  EMPTY_KEY_LIST;
        ATTRIBUTES     : in      ATTRIBUTE_LIST  :=  EMPTY_LIST;
        STATUS         : in      STATUS_INDICATOR :=  NO_INDICATION);

    procedure  DELETE_SOURCE
        (VIEW                  : in      VIEW_NODE_TYPE;
        NKD                    : in      NKD_TYPE;
        RD                     : in      RD_TYPE;
        DELETE_DEFINITION : in      BOOLEAN              :=  FALSE ;
        TIME_LIMIT             : in      CAIS_DURATION  :=  LONG_DELAY;
        STATUS                 : in      STATUS_INDICATOR :=  NO_INDICATION);

    procedure  NAME_SOURCE
        (VIEW          : in      VIEW_NODE_TYPE;
        NKD            : in      NKD_TYPE;
        RD             : in      RD_TYPE ;
        LOCAL_NAME     : in      RELATIONSHIP_KEY;
        STATUS         : in      STATUS_INDICATOR  := NO_INDICATION);

    procedure  DEFINE_KEY
        (VIEW          : in      VIEW_NODE_TYPE;
        RD             : in      RD_TYPE;
        AD             : in      IDENTIFIER_AD_TYPE;
        LOCAL_NAME     : in      RELATIONSHIP_KEY;
        KEY            : in      RELATIONSHIP_KEY   := EMPTY_KEY_LIST;
        REVERSE_KEY    : in      RELATIONSHIP_KEY   := EMPTY_KEY_LIST;
        ATTRIBUTES     : in      ATTRIBUTE_LIST     := EMPTY_LIST;
        STATUS         : in      STATUS_INDICATOR   := NO_INDICATION);

    procedure DELETE_KEY
        (VIEW                  : in      VIEW_NODE_TYPE;
        RD                     : in      RD_TYPE ;
        AD                     : in      IDENTIFIER_AD_TYPE;
        DELETE_DEFINITION : in      BOOLEAN              := FALSE;
        TIME_LIMIT             : in      CAIS_DURATION        := LONG_DELAY;
        STATUS                 : in      STATUS_INDICATOR   := NO_INDICATION);

    procedure  NAME_KEY
        (VIEW          : in      VIEW_NODE_TYPE;
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
        RD              : in    RD_TYPE;
        AD              : in    IDENTIFIER_AD_TYPE;
        LOCAL_NAME      : in    RELATIOINSHIP_KEY;
        STATUS          : in    STATUS_INDICATOR  := NO_INDICATION);

    procedure DEFINE_TARGET
        (NKD            : in    NKD_TYPE;
        RD              : in    RD_TYPE;
        KEY             : in    RELATIONSHIP_KEY  :=  EMPTY_KEY_LIST;
        REVERSE_KEY     : in    RELATIONSHIP_KEY  :=  EMPTY_KEY_LIST;
        ATTRIBUTES      : in    ATTRIBUTE_LIST    :=  EMPTY_LIST;
        STATUS          : in    STATUS_INDICATOR  :=  NO_INICATION);

    procedure  DELETE_TARGET
        (NKD            : in    NKD_TYPE;
        RD              : in    RD_TYPE;
        TIME_LIMIT      : in    CAIS_DURATION     := LONG_DELAY;
        STATUS          : in    STATUS_INDICATOR  := NO_INDICATION);

    procedure DEFINE_ATTRIBUTE
        (VIEW           : in    VIEW_NODE_TYPE;
        MODE            : in    DEFINITION_NODE_TYPE;
        AD              : in    AD_TYPE;
        LOCAL_NAME      : in    RELATIONSHIP_KEY;
        KEY             : in    RELATIONSHIP_KEY  :=  EMPTY_KEY_LIST;
        REVERSE_KEY     : in    RELATIONSHIP_KEY  :=  EMPTY_KEY_LIST ;
        ATTRIBUTES      : in    ATTRIBUTE_LIST    :=  EMPTY_LIST;
        STATUS          : in    STATUS_INDICATOR  :=  NO_INDICATION);

    procedure  DELETE_ATTRIBUTE
        (VIEW           : in    VIEW_NODE_TYPE;
        NODE            : in    DEFINITION_NODE_TYPE;
        AD              : in    AD_TYPE ;
        DELETE_DEFINITION : in  BOOLEAN           := FALSE;
        TIME_LIMIT      : in    CAIS_DURATION     := LONG_DELAY;
        STATUS          : in    STATUS_INDICATOR  := NO_INDICATION);

    procedure  NAME_ATTRIBUTE
        (VIEW           : in    VIEW_NODE_TYPE;
        DEFINITION      : in    DEFINITION_NODE_TYPE;
        AD              : in    AD_TYPE ;
        LOCAL_NAME      : in    RELATIONSHIP_KEY;
        STATUS          : in    STATUS_INDICATOR  := NO_INDICATION);

    procedure DEFINE_COMPONENT
        (STRUCTURED_AKD : in    AKD_TYPE;
        COMPONENT_AKD   : in    AKD_TYPE;
        KEY             : in    RELATIONSHIP_KEY  := EMPTY_KEY_LIST;
        REVERSE_KEY     : in    RELATIONSHIP_KEY  := EMPTY_KEY_LIST;
        ATTRIBUTES      : in    ATTRIBUTE_LIST    := EMPTY_LIST;
        STATUS          : in    STATUS_INDICATOR  := NO_INDICATION) ;
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
procedure DELETE_COMPONENT
    (STRUCTURED_AKD : in  AKD_TYPE;
     COMPONENT_AKD  : in  AKD_TYPE;
     TIME_LIMIT     : in  CAIS_DURATION := LONG_DELAY;
     STATUS         : in  STATUS_INDICATOR := NO_INDICATION);

procedure DEFIIVE_ALTERNATIVE
    (VAKD           : in  VAKD_TYPE;
     ALTERNATIVE_AKD : in  AKD_TYPE;
     KEY            : in  RELATIONSHIP_KEY := EMPTY_KEY_LIST;
     REVERSE_KEY    : in  RELATIONSHIP_KEY := EMPTY_KEY_LIST;
     ATTRIBUTES     : in  ATTRIBUTE_LIST := EMPTY_LIST;
     STATUS         : in  STATUS_INDICATOR := NO_INDICATION);

procedure DELETE_ALTERNATIVE
    (VAKD           : in  VAKD_TYPE;
     ALTERNATIVE_AKD : in  AKD_TYPE ;
     TIME_LIMIT     : in  CAIS_DURATION := LONG_DELAY;
     STATUS         : in  STATUS_INDICATOR := NO_INDICATION);

procedure SET_DEFINITION_ATTRIBUTE
    (NODE           : in  DEFINITION_NODE_TYPE;
     ATTRIBUTE      : in  AD_TYPE;
     VALUE          : in  LIST_TYPE;
     STATUS         : in  STATUS_INDICATOR := NO_INDICATION);

procedure SET_DEFINITION_ATTRIBUTE
    (NODE           : in  DEFINITION_NODE_TYPE;
     ATTRIBUTE      : in  ATTRIBUTE_NAME;
     VALUE          : in  LIST_TYPE;
     STATUS         : in  STATUS_INDICATOR := NO_INDICATION);

procedure DELETE_DEFINITION_ATTRIBUTE
    (NODE           : in  DEFINITION_NODE_TYPE;
     ATTRIBUTE      : in  AD_TYPE;
     STATUS         : in  STATUS_INDICATOR := NO_INDICATION);

procedure DELETE_DEFINITION_ATTRIBUTE
    (NODE           : in  DEFINIITION_NODE_TYPE;
     ATTRIBUTE      : in  ATTRIBUTE_NAME;
     STATUS         : in  STATUS_INDICATOR := NO_INDICATION);

procedure SET_PROPERTY_ATTRIBUTE
    (BASE           : in  DEFINITION_NODE_TYPE;
     KEY            : in  RELATIONSHIP_KEY;
     RELATION       : in  RD_TYPE;
     ATTRIBUTE      : in  AD_TYPE ;
     VALUE          : in  LIST_TYPE;
     STATUS         : in  STATUS_INDICATOR := NO_INDICATION);

procedure SET_PROPERTY_ATTRIBUTE
    (BASE           : in  DEFINITION_NODE_TYPE;
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
        KEY          : in      RELATIONSHIP_KEY;
        RELATION     : in      RELATION_NAME;
        ATTRIBUTE    : in      ATTRIBUTE_NAME;
        VALUE        : in      LIST_TYPE ;
        STATUS       : in      STATUS_INDICATOR  := NO_INDICATION);

    procedure DELETE_PROPERTY_ATTRIBUTE
        (BASE        : in      DEFINITIOM_NODE_TYPE;
         KEY         : in      RELATIONSHIP_KEY;
         RELATION    : in      RD_TYPE;
         ATTRIBUTE   : in      AD_TYPE;
         STATUS      : in      STATUS_INDICATOR  := NO_INDICATION);

    procedure  DELETE_PROPERTY_ATTRIBUTE
        (BASE        : in      DEFINITION_NODE_TYPE;
         KEY         : in      RELATIONSHIP_KEY;
         RELATION    : in      RELATION_NAME;
         ATTRIBUTE   : in      ATTRIBUTE_NAME;
         STATUS      : in      STATUS_INDICATOR  := NO_INDICATION);

    procedure CONVERT_NODE
        (NODE                : in      NODE_TYPE;
         KIND                : in      NKD_TYPE;
         NEW_ATTRIBUTES      : in      ATTRIBUTE_LIST  := EMPTY_LIST;
         STATUS              : in      STATUS_INDICATOR := NO_INDICATION);

    procedure CONVERT_NODE
        (NODE                : in      NODE_TYPE;
         KIND                : in      NODE_KIND_NAME;
         NEW_ATTRIBUTES      : in      ATTRIBUTE_LIST  := EMPTY_LIST;
         STATUS              : in      STATUS_INDICATOR := NO_INDICATION);

    function  CONFORMS
        (NODE                          : in      MODE_TYPE;
         NKD                           : in      NKD_TYPB;
         CONFORMANCE_ERROR_INFO        : in      STATUS_INDICATOR := NO_INDICATION;
         STATUS                        : in      STATUS_INDICATOR := NO_INDICATION)
        return BOOLEAN;

    function  CONFORMS
        (NODE                          : in      NODE_TYPE;
         NKD                           : in      NODE_KIND_NAME;
         CONFORMANCE_ERROR_INFO        : in      STATUS_INDICATOR := NO_INDICATION;
         STATUS                        : in      STATUS_INDICATOR := NO_INDICATION)
        return BOOLEAN;

    function  CONFORMS
        (BASE                          : in      NODE_TYPE;
         KEY                           : in      RELATIONSHIP_KEY := EMPTY_KEY_LIST;
         RELATION                      : in      RD_TYPE;
         RD                            : in      RD_TYPE;
         COMFORMANCE_ERROR_INF0        : in      STATUS_INDICATOR := NO_INDICATION;
         STATUS                        : in      STATUS_INDICATOR := NO_INDICATION)
        return BOOLEAN;
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
    function CONFORMS
        (BASE                       : in  NODE_TYPE;
         KEY                        : in  RELATIONSHIP_KEY   := EMPTY_KEY_LIST;
         RELATION                   : in  RELATION_NAME;
         RD                         : in  RELATION_NAME;
         CONFORMANCE_ERROR_INFO : in  STATUS_INDICATOR  := NO_INDICATION;
         STATUS                     : in  STATUS_INDICATOR   := NO_INDICATION)
        return BOOLEAN;

    function CONFORMS
        (LIST                       : in  LIST_TYPE;
         DEFINITION                 : in  ATTRIBUTE_MODE_TYPE;
         CONFORMANCE_ERROR_INFO : in  STATUS_INDICATOR  := NO_INDICATION;
         STATUS                     : in  STATUS_INDICATOR   := NO_INDICATION)
        return BOOLEAN;

    function CONFORMS
        (LIST                       : in  LIST_TYPE;
         DEFINITION                 : in  ATTRIBUTE_NAME;
         CONFORMANCE_ERROR-INFO : in  STATUS_INDICATOR  := NO_INDICATION;
         STATUS                     : in  STATUS_INDICATOR   := NO_INDICATION)
        return BOOLEAN;

end CAIS_DEFINITION_MANAGEMENT;
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
with CAIS_STANDARD;
with CAIS_DEFINITIONS;
with CAIS_CALENDAR;
with CAIS_LIST_MANAGEMENT;
with CAIS_STATUS_DEFINITIONS;
with CAIS_RESOURCE_CONTROL;
with CAIS_DEFINITION_MANAGEMENT;
with CAIS_ACCESS_CONTROL_MANAGEMENT;

package CAIS_NODE_MANAGEMENT is

    use CAIS_STANDARD;
    use CAIS_DEFINITIONS;
    use CAIS_CALENDAR;
    use CAIS_LIST_MANAGEMENT;
    use CAIS_STATUS_DEFINITIONS ;
    use CAIS_RESOURCE_CONTROL;
    use CAIS_DEFINITION_MANAGEMENT;
    use CAIS_ACCESS_CONTROL_MANAGEMENT;

    procedure OPEN
        (NODE       : in out MODE_TYPE;
         BASE       : in     NODE_TYPE;
         KEY        : in     RELATIONSHIP_KEY := EMPTY_KEY_LIST;
         RELATION   : in     RD_TYPE;
         INTENT     : in     INTENT_ARRAY  := (1 => READ);
         TIME_LIMIT : in     CAIS_DURATION := LONG_DELAY;
         STATUS     : in     STATUS_INDICATOR := NO_INDICATION);

    procedure OPEN
        (NODE       : in out NODE_TYPE;
         NAME       : in     PATHNAME;
         INTENT     : in     INTENT_ARRAY  := (1 => READ);
         TIME_LIMIT : in     CAIS_DURATION := LONG_DELAY;
         STATUS     : in     STATUS_INDICATOR := NO_INDICATION) ;

    procedure OPEN
        (NODE       : in out MODE_TYPE;
         HANDLE     : in     NODE_TYPE;
         INTENT     : in     INTENT_ARRAY  := (1 => READ);
         TIME_LIMIT : in     CAIS_DURATION := LONG_DELAY;
         STATUS     : in     STATUS_INDICATOR := NO_INDICATION) ;

    procedure OPEN
        (NOSE       : in out NODE_TYPE;
         BASE       : in     NODE_TYPE;
         KEY        : in     RELATIONSHIP_KEY := EMPTY_KEY_LIST;
         RELATION   : in     RD_TYPE ;
         INTENT     : in     INTENT_ARRAY  := (1 => READ);
         KIND       : in     NKI_TYPE;
         TIME_LIMIT : in     CAIS_DURATION := LONG_DELAY;
         STATUS     : in     STATUS_INDICATOR := NO_INDICATION) ;
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
procedure OPEN
    (NODE      : in out NODE_TYPE;
     BASE      : in     NODE_TYPE;
     KEY       : in     RELATIONSHIP_KEY := EMPTY_KEY_LIST;
     RELATION  : in     RELATION_NAME := DEFAULT_RELATION;
     INTENT    : in     INTENT_ARRAY  := (1 => READ);
     TIME-LIMIT : in    CAIS_DURATION := LONG_DELAY;
     STATUS    : in     STATUS_INDICATON := NO_INDICATION);


procedure OPEN
    (NODE      : in out MODE_TYPE;
     NAME      : in     PATHNAME;
     INTENT    : in     INTENT_SPECIFICATION;
     TIME-LIMIT : in    CAIS_DURATION := LONG_DELAY;
     STATUS    : in     STATUS_INDICATOR := NO_INDICATION);


procedure OPEN
    (NODE      : in out NODE_TYPE;
     BASE      : in     NODE_TYPE;
     KEY       : in     RELATIONSHIP_KEY;
     RELATION  : in     RELATION_NAME := DEFAULT_RELATION;
     INTENT    : in     INTENT_SPECIFICATION ;
     TIME-LIMIT : in    CAIS_DURATION := LONG_DELAY;
     STATUS    : in     STATUS_INDICATOR := NO_INDICATION);


procedure OPEN_VIA_UNIQUE_IDENTIFIER
    (NODE      : in out NODE_TYPE;
     IDENTIFIER : in    UNIQUE_IDENTIFIER;
     INTENT    : in     INTENT_ARRAY := (1 => READ);
     TIME_LIMIT : in    CAIS_DURATION := LONG_DELAY;
     STATUS    : in     STATUS_INDICATOR := NO_INDICATION);


procedure CLOSE
    (NODE   : in out NODE_TYPE;
     STATUS : in     STATUS_INDICATOR := NO_INDICATION);


proceduro CHANGE_INTENT
    (NODE      : in out NODE_TYPE;
     INTENT    : in     INTENT_ARRAY;
     TIME_LIMIT : in    CAIS_DURATION := LONG_DELAY;
     STATUS    : in     STATUS_INDICATOR := NO_INDICATION);


procedure CHANGE_INTENT
    (NODE      : in out MODE_TYPE;
     INTENT    : in     INTENT_SPECIFICATION ;
     TIME_LIMIT : in    CAIS_DURATION := LONG_DELAY;
     STATUS    : in     STATUS_INDICATOR := NO_INDICATION);


function IS_OPEN
    (NODE   : in  NODE_TYPE;
     STATUS : in  STATUS_INDICATOR := NO_INDICATION)
    return BOOLEAN;
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
function INTENT
    (MODE   : in     NODE_TYPE;
     STATUS : in     STATUS_INDICATOR  := NO_INDICATION)
    return INTENT_ARRAY;

function KIND_OF_NODE
    (NODE   : in     NODE_TYPE;
     STATUS : in     STATUS_INDICATOR  := NO_INDICATION)
    return NODE_KIND;

function IS_SUB_KIND
    (KIND1  : in     NKD_TYPE;
     KIND2  : in     NKD_TYPE;
     STATUS : in     STATUS_INDICATOR  := NO_INDICATION)
    return BOOLEAN;

function IS_SUB_KIND
    (KIND1  : in     NODE_KIND_NAME;
     KIND2  : in     NODE_KIND_NAME;
     STATUS : in     STATUS_INDICATOR  := NO_INDICATION)
    return BOOLEAN;

function IS_SUB_RELATION
    (RELATION1 : in     RD_TYPE;
     RELATION2 : in     RD_TYPE;
     STATUS    : in     STATUS_INDICATOR  := NO_INDICATION)
    return BOOLEAN;

function IS_SUB_RELATION
    (RELATION1 : in     RELATION_NAME;
     RELATION2 : in     RELATION_NAME;
     STATUS    : in     STATUS_INDICATOR  := NO_INDICATION)
    return BOOLEAN;

function IS_OF_KIND
    (NODE   : in     MODE_TYPE;
     KIND   : in     NKD_TYPE;
     STATUS : in     STATUS_INDICATOR  := NO_INDICATION)
    return BOOLEAN;

function IS_OF_KIND
    (NODE   : in     NODE_TYPE;
     KIND   : in     NODE_KIND_NAME;
     STATUS : in     STATUS_INDICATOR  := NO_INDICATION)
    return BOOLEAN;

function IS_OF_RELATION
    (BASE        : in     NODE_TYPE;
     KEY         : in     RELATIONSHIP_KEY  := EMPTY_KEY_LIST;
     RELATION    : in     RD_TYPE;
     OF_RELATION : in     RD_TYPE;
     STATUS      : in     STATUS_INDICATOR  := NO_INDICATION)
    return BOOLEAN;
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
function  IS_OF_RELATION
     (BASE            : in      NODE_TYPE ;
      KEY             : in      RELATIONSHIP_KEY  := EMPTY_KEY_LIST;
      RELATION        : in      RELATION_NAME;
      OF_RELATION : in      RELATION_NAME;
      STATUS          : in      STATUS_INDICATOR  :=  NO_INDICATION)
     return BOOLEAN;

function OPEN_FILE_HANDLE_COUNT
     (NODE     : in      CLIENT_NODE_TYPE;
      STATUS : in      STATUS_INDICATOR  :=  NO_INDICATION)
     return CAIS_NATURAL;

function UNIQUE_IDENTIFICATION
     (NODE           : in      NODE_TYPE;
      STATUS         : in      STATUS_INDICATOR  :=  NO_INDICATION)
     return  UNIQUE_IDENTIFIER;

function PRIMARY_NAME
     (NODE     : in      NODE_TYPE;
      STATUS : in      STATUS_INDICATOR  :=  NO_INDICATION)
     return  PATHNAME;

function  PRIMARY_KEY
     (NODE     : in      NODE_TYPE;
      STATUS : in      STATUS_INDICATOR  := NO_INDICATION)
     return RELATIONSHIP_KEY;

procedure  PRIMARY_RELATION
     (RELATION :  in out RD_TYPE;
      NODE       : in      NODE_TYPE;
      STATUS     : in      STATUS_INDICATOR  := NO_INDICATION);

function  PRIMARY_RELATION
     (NODE       : in      NODE_TYPE;
      STATUS     : in      STATUS_INDICATOR  :=  NO_INDICATION)
     return  RELATION_NAME;

function  PATH_KEY
     (NODE     : in      NODE_TYPE;
      STATUS : in      STATUS_INDICATOR  := NO_INDICATION)
     return RELATIONSHIP_KEY;

procedure  PATH_RELATION
     (RELATION  :  in out RD_TYPE;
      NODE       : in      NODE_TYPE;
      STATUS     : in      STATUS_INDICATOR  := NO_INDICATION);

function  PATH_RELATION
     (NODE       : in      NODE_TYPE;
      STATUS     : in      STATUS_INDICATOR  :=  NO_INDICATION)
     return  RELATION_NAME;
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
function BASE_PATH
    (NAME   : in  PATHNAME;
     STATUS : in  STATUS_INDICATOR := NO_INDICATION)
    return PATHNAME;

function LAST_RELATION
    (NAME   : in  PATHNAME;
     STATUS : in  STATUS_INDICATOR := NO_INDICATION)
    return RELATION_NAME;

function LAST_KEY
    (NAME   : in  PATHNAME;
     STATUS : in  STATUS_INDICATOR := NO_INDICATION)
    return RELATIONSHIP_KEY;

function IS_OBTAINABLE
    (NODE   : in  NODE_TYPE;
     STATUS : in  STATUS_INDICATOR := NO_INDICATION)
    return BOOLEAN;

function IS_OBTAINABLE
    (NAME   : in  PATHNAME;
     STATUS : in  STATUS_INDICATOR := NO_INDICATION)
    return BOOLEAN;

function IS_OBTAINABLE
    (BASE     : in  NODE_TYPE;
     KEY      : in  RELATIONSHIP_KEY;
     RELATION : in  RELATION_NAME   := DEFAULT_RELATION;
     STATUS   : in  STATUS_INDICATOR := NO_INDICATION)
    return BOOLEAN;

function IS_SAME
    (NODE1  : in  NODE_TYPE;
     NODE2  : in  NODE_TYPE;
     STATUS : in  STATUS_INDICATOR := NO_INDICATION)
    return BOOLEAN;

function IS_SAME
    (NAME1  : in  PATHNAME;
     NAME2  : in  PATHNAME;
     STATUS : in  STATUS_INDICATOR := NO_INDICATION)
    return BOOLEAN;

function INDEX
    (NODE   : in  NODE_TYPE;
     MODUL0 : in  CAIS_POSITIVE;
     STATUS : in  STATUS_INDICATOR := NO_INDICATION)
    return CAIS_NATURAL;
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
procedure OPEN_PARENT
    (PARENT      : in out NODE_TYPE;
     NODE        : in     NODE_TYPE;
     INTENT      : in     INTENT_ARRAY      := (1 => READ);
     TIME_ LIMIT : in     CAIS_DURATION     := LONG_DELAY;
     STATUS      : in     STATUS_INDICATOR  := NO_INDICATION);


procedure OPEN_PARENT
    (PARENT      : in out NODE_TYPE;
     NODE        : in     NODE_TYPE;
     INTENT      : in     INTENT_SPECIFICATION;
     TIME_LIMIT  : in     CAIS_DURATION     := LONG_DELAY;
     STATUS      : in     STATUS_INDICATOR  := NO_INDICATION);


procedure COPY_NODE
    (FROM                    : in  NODE_TYPE;
     TO_BASE                 : in  NODE_TYPE;
     TO_KEY                  : in  RELATIONSHIP_KEY      := EMPTY_KEY_LIST;
     TO_RELATION             : in  RD_TYPE;
     DISCRETIONARY_ACCESS    : in  DISCRETIONARY_ACCESS_LIST := ALL_RIGHTS;
     MANDATORY_ACCESS        : in  MANDATORY_ACCESS_LIST := EMPTY_LIST;
     STATUS                  : in  STATUS_INDICATOR := NO_INDICATION;
     MEDIUM_RESOURCE         : in  MEDIUM_NODE_TYPE := DEFAULT_MEDIUM);


procedure COPY_NODE
    (FROM                    : in  NODE_TYPE;
     TO_BASE                 : in  NODE_TYPE;
     TO_KEY                  : in  RELATIONSHIP_KEY := EMPTY_KEY_LIST;
     TO_RELATION             : in  RELATION_NAME    := DEFAULT_RELATION;
     DISCRETIONARY_ACCESS    : in  DISCRETIONARY_ACCESS_LIST := ALL_RIGHTS;
     MANDATORY_ACCESS        : in  MANDATORY_ACCESS_LIST := EMPTY_LIST;
     STATUS                  : in  STATUS_INDICATOR      := NO_INDICATION;
     MEDIUM_RESOURCE         : in  MEDIUM_NODE_TYPE      := DEFAULT_MEDIUM);


procedure COPY_NODE
    (FROM                    : in  NODE_TYPE;
     TO                      : in  PATHNAME;
     DISCRETIONARY_ACCESS    : in  DISCRETIONARY_ACCESS_LIST := ALL_RIGHTS;
     MANDATORY_ACCESS        : in  MANDATORY_ACCESS_LIST := EMPTY_LIST;
     STATUS                  : in  STATUS_INDICATOR      := NO_INDICATION;
     MEDIUM_RESOURCE         : in  MEDIUM_NODE_TYPE      := DEFAULT_MEDIUM);


procedue COPY_TREE
    (FROM                    : in  NODE_TYPE;
     TO_BASE                 : in  NODE_TYPE;
     TO_KEY                  : in  RELATIONSHIP_KEY      := EMPTY_KEY_LIST;
     TO_RELATION             : in  RD_TYPE ;
     DISCRETIONARY_ACCESS    : in  DISCRETIONARY_ACCESS_LIST := ALL_RIGHTS;
     MANDATORY_ACCESS        : in  MANDATORY_ACCESS_LIST := EMPTY_LIST;
     STATUS                  : in  STATUS_INDICATOR      := NO_INDICATION;
     MEDIUM_RESOURCE         : in  MEDIUM_NODE_TYPE      := DEFAULT_MEDIUM);
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
procedure COPY_TREE
    (FROM                 : in  NODE_TYPE;
     TO_BASE              : in  NODE_TYPE;
     TO_KEY               : in  RELATIONSHIP_KEY          := EMPTY_KEY_LIST;
     TO_RELATION          : in  RELATION_NAME             := DEFAULT_RELATION;
     DISCRETIONARY_ACCESS : in  DISCRETIONARY_ACCESS_LIST := ALL_RIGHTS;
     MANDATORY_ACCESS     : in  MANDATORY_ACCESS_LIST     := EMPTY_LIST;
     STATUS               : in  STATUS_INDICATOR          := NO_INDICATION;
     MEDIUM_RESOURCE      : in  MEDIUM_NODE_TYPE          := DEFAULT_MEDIUM);


procedure COPY_TREE
    (FROM                 : in  NODE_TYPE;
     TO                   : in  PATHNAME;
     DISCRETIONARY_ACCESS : in  DISCRETIONARY_ACCESS_LIST := ALL_RIGHTS;
     MANDATORY_ACCESS     : in  MANDATORY_ACCESS_LIST     := EMPTY_LIST;
     STATUS               : in  STATUS_INDICATOR          := NO_INDICATION;
     MEDIUM_RESOURCE      : in  MEDIUM_MODE_TYPE          := DEFAULT_MEDIUM);


procedure RENAME
    (BASE                 : in  MODE_TYPE;
     KEY                  : in  RELATIONSHIP_KEY := EMPTY_KEY_LIST;
     RELATION             : in  RD_TYPE;

     NEW_BASE             : in  NODE_TYPE        := MULL_MODE;
     NEW_KEY              : in  RELATIONSHIP_KEY := EMPTY_KEY_LIST;
     NEW_RELATION         : in  RD_TYPE          := NULL_NODE;
     NEW_ATTRIBUTES       : in  ATTRIBUTE_LIST   := EMPTY_LIST;

     NEW_TARGET           : in  NODE_TYPE        := NULL_NODE;
     NEW_REVERSE_KEY      : in  RELATIONSHIP_KEY := EMPTY_KEY_LIST;
     NEW_REVERSE_ATTRIBUTES : in ATTRIBUTE_LIST  := EMPTY_LIST;
     STATUS               : in  STATUS_INDICATOR := NO_INDICATION);


procedure RENAME
    (NODE                 : in  MODE_TYPE;
     NEW_BASE             : in  NODE_TYPE        := NULL_NODE;
     NEW_KEY              : in  RELATIONSHIP_KEY := EMPTY_KEY_LIST;
     NEW_RELATION         : in  RD_TYPE          := NULL_MODE;
     STATUS               : in  STATUS_INDICATOR := NO_INDICATION);


procedure RENAME
    (NODE                 : in  NODE_TYPE;
     NEW_BASE             : in  NODE_TYPE;
     NEW_KEY              : in  RELATIONSHIP_KEY := EMPTY_KEY_LIST;
     NEW_RELATION         : in  RELATION_NAME;
     STATUS               : in  STATUS_INDICATOR := NO_INDICATION);


procedure RENAME
    (MODE                 : in  MODE_TYPE;
     NEW_NAME             : in  PATHNAME;
     STATUS               : in  STATUS_INDICATOR := NO_INDICATION);
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
procedure DELETE_NODE
    (NODE          : in out NODE_TYPE;
     TIME. LIMIT : in     CAIS_DURATION     := LONG_DELAY;
     STATUS        : in     STATUS_INDICATOR  := NO_INDICATION);


procedure DELETE_NODE
    (NAME     : in     PATHNAME;
     STATUS : in     STATUS_INDICATOR  := NO_INDICATION);


procedure DELETE_TREE
    (NODE     : in out NODE_TYPE;
     STATUS : in     STATUS_INDICATOR  := NO_INDICATION);


procedure DELETE_TREE
    (NAME     : in     PATHNAME;
     STATUS : in     STATUS_INDICATOR  := NO_INDICATION);


procedure CREATE_SECONDARY_RELATIONSHIP
    (TARGET_NODE     : in     NODE_TYPE;
     SOURCE_BASE     : in     NODE_TYPE;
     NEW_KEY         : in     RELATIONSHIP_KEY := EMPTY_KEY_LIST;
     NEW_RELATION    : in     RD_TYPE;
     INHERITABLE     : in     BOOLEAN          := FALSE ;
     NEW_ATTRIBUTES  : in     ATTRIBUTE_LIST   := EMPTY_LIST;
     STATUS          : in     STATUS_INDICATOR := NO_INDICATION);


procedure CREATE_SECONDARY_RELATIONSHIP
    (TARGET_NODE     : in     NODE_TYPE;
     SOURCE_BASE     : in     NODE_TYPE;
     NEW_KEY         : in     RELATIONSHIP_KEY := EMPTY_KEY_LIST;
     MEW_RELATION    : in     RELATION_NAME    := DEFAULT_RELATION;
     INHERITABLE     : in     BOOLEAN          := FALSE;
     NEW_ATTRIBUTES : in     ATTRIBUTE_LIST   := EMPTY_LIST;
     STATUS          : in     STATUS_INDICATOR := NO_INDICATION);


procedure CREATE_SECONDARY_RELATIONSHIP
    (TARGET_NODE : in     NODE_TYPE;
     NEW_NAME       : in     PATHNAME;
     INHERITABLE : in     BOOLEAN          := FALSE;
     STATUS         : in     STATUS_INDICATOR := NO_INDICATION);


procedure CREATE_BIDIRECTIONAL_SECONDARY_RELATIONSHIP
    (TARGET_NODE              : in     NODE_TYPE;
     SOURCE_BASE              : in     NODE_TYPE;
     NEW_KEY                  : in     RELATIONSHIP_KEY := EMPTY_KEY_LIST;
     NEW_RELATION             : in     RD_TYPE;
     NEW_ATTRIBUTES           : in     ATTRIBUTE_LIST := EMPTY_LIST;
     NEW_REVERSE_KEY          : in     RELATIONSHIP_KEY := EMPTY_KEY_LIST;
     NEW_REVERSE_ATTRIBUTES : in     ATTRIBUTE_LIST := EMPTY_LIST;
     STATUS                   : in     STATUS_INDICATOR := NO_INDICATION);
```


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
procedure CREATE_BIDIRECTIONAL_SECONDARY_RELATIONSHIP
    (TARGET_NODE                : in  NODE_TYPE;
     SOURCE_BASE                : in  NODE_TYPE;
     NEW_KEY                    : in  RELATIONSHIP_KEY  := EMPTY_KEY_LIST;
     NEW_RELATION               : in  RELATION_NAME     := DEFAULT_RELATION;
     NEW_ATTRIBUTES             : in  ATTRIBUTE_LIST    := EMPTY_LIST;
     NEW_REVERSE_KEY            : in  RELATIONSHIP_KEY  := EMPTY_KEY_LIST;
     NEW_REVERSE_ATTRIBUTES : in  ATTRIBUTE_LIST    := EMPTY_LIST;
     STATUS                     : in  STATUS_INDICATOR  := NO_INDICATION);

procedure DELETE_SECONDARY_RELATIONSHIP
    (BASE     : in  NODE_TYPE;
     KEY      : in  RELATIONSHIP_KEY := EMPTY_KEY_LIST;
     RELATION : in  RD_TYPE ;
     STATUS   : in  STATUS_INDICATOR := NO_INDICATION);

procedure DELETE_SECONDARY_RELATIONSHIP
    (BASE     : in  NODE_TYPE;
     KEY      : in  RELATIONSHIP_KEY := EMPTY_KEY_LIST;
     RELATION : in  RELATION_NAME := DEFAULT_RELATION;
     STATUS   : in  STATUS_INDICATOR := NO_INDICATION);

procedure DELETE_SECONDARY_RELATIONSHIP
    (NAME   : in  PATHNAME;
     STATUS : in  STATUS_INDICATOR  := NO_INDICATION);

procedure GET_REVERSE_RELATION
    (REVERSE_RELATION : in out RD_TYPE;
     BASE             : in  NODE_TYPE;
     KEY              : in  RELATIONSHIP_KEY  := EMPTY_KEY_LIST;
     RELATION         : in  RD_TYPE;
     STATUS           : in  STATUS_INDICATOR  := NO_INDICATION);

function GET_REVERSE_RELATION
    (BASE             : in  NODE_TYPE;
     KEY              : in  RELATIONSHIP_KEY  := EMPTY_KEY_LIST;
     RELATION         : in  RD_TYPE;
     STATUS           : in  STATUS_INDICATOR  := NO_INDICATION)
    return RELATION_NAME;

function GET_REVERSE_KEY
    (BASE     : in  NODE_TYPE;
     KEY      : in  RELATIONSHIP_KEY := EMPTY_KEY_LIST;
     RELATION : in  RD_TYPE;
     STATUS   : in  STATUS_INDICATOR := MO_INDICATION)
    return RELATIONSHIP_KEY;

procedure SET_INHERITANCE
    (BASE        : in  NODE_TYPE;
     KEY         : in  RELATIONSHIP_KEY  := EMPTY_KEY_LIST;
     RELATION    : in  RD_TYPE;
     INHERITABLE : in  BOOLEAN;
     STATUS      : in  STATUS_INDICATOR  := NO_INDICATION) ;
```


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
procedure SET_INHERITANCE
    (BASE        : in  NODE_TYPE;
     KEY         : in  RELATIONSHIP_KEY  := EMPTY_KEY_LIST;
     RELATION    : in  RELATION_NAME     := DEFAULT_RELATION;
     INHERITABLE : in  BOOLEAN;
     STATUS      : in  STATUS_INDICATOR  := NO_INDICATION);

procedure SET_INHERITANCE
    (NAME        : in  PATHNAME;
     INHERITABLE : in  BOOLEAN;
     STATUS      : in  STATUS_INDICATOR  := NO_INDICATION);

function IS_INHERITABLE
    (BASE        : in  NODE_TYPE;
     KEY         : in  RELATIONSHIP_KEY  := EMPTY_KEY_LIST;
     RELATION    : in  RD_TYPE;
     STATUS      : in  STATUS_INDICATOR  := NO_INDICATION)
    return BOOLEAN;

function IS_INHERITABLE
    (BASE        : in  MODE_TYPE;
     KEY         : in  RELATIONSHIP_KEY  := EMPTY_KEY_LIST;
     RELATION    : in  RELATION_NAME     := DEFAULT_RELATION;
     STATUS      : in  STATUS_INDICATOR  := NO_INDICATION)
    return BOOLEAN;

function IS_INHERITABLE
    (NAME        : in  PATHNAME;
     STATUS      : in  STATUS_INDICATOR  := NO_INDICATION)
    return BOOLEAN;

procedure SET_CURRENT_NODE
    (NODE        : in  NODE_TYPE;
     TIME_LIMIT  : in  CAIS_DURATION     := LONG_DELAY;
     STATUS      : in  STATUS_INDICATOR  := NO_INDICATION);

procedure SET_CURRENT_NODE
    (NAME        : in  PATHNAME;
     TIME_LIMIT  : in  CAIS_DURATION     := LONG_DELAY;
     STATUS      : in  STATUS_INDICATOR  := NO_INDICATION);

procedure  GET_CURRENT_NODE
    (NODE        : in out NODE_TYPE;
     INTENT      : in  INTENT_ARRAY      := (1 => NO_ACCESS);
     TIME_LIMIT  : in  CAIS_DURATION     := LONG_DELAY;
     STATUS      : in  STATUS_INDICATOR  := NO_INDICATION);

procedure GET_CURRENT_NODE
    (NODE        : in out NODE_TYPE;
     INTENT      : in  IMTENT_SPECIFICATION ;
     TIME_LIMIT  : in  CAIS_DURATION         := LONG_DELAY;
     STATUS      : in  STATUS_INDICATOR      := NO_INDICATION);
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
function TIME_CREATED
    (NODE   : in    NODE_TYPE ;
     STATUS : in    STATUS_ INDICATOR  := NO_INDICATION)
    return CAIS_CALENDAR_TIME;

function TIME_CREATED
    (NAME   : in    PATHNAME ;
     STATUS : in    STATUS_INDICATOR  := NO_INDICATION)
    return CAIS_CALENDAR_TIME;

function TIME_RELATIONSHIP_WRITTEN
    (NODE   : in    NODE_TYPE;
     STATUS : in    STATUS_INDICATOR  := NO_INDICATION)
    return CAIS_CALENDAR_TIME;

function TIME_RELATIONSHIP_WRITTEN
    (NAME   : in    PATHNAME;
     STATUS : in    STATUS_INDICATOR  := NO_INDICATION)
    return CAIS_CALENDAR_TIME;

function TIME_CONTENTS_WRITTEN
    (NODE   : in    FILE_NODE_TYPE;
     STATUS : in    STATUS_INDICATOR  := NO_INDICATION)
    return CAIS_CALENDAR_TIME;

function TIME_CONTENTS_WRITTEN
    (NAME   : in    PATHNAME;
     STATUS : in    STATUS_INDICATOR  := NO_INDICATION)
    return CAIS_CALENDAR_TIME;

function TIME_ATTRIBUTE_WRITTEN
    (NODE   : in    MODE_TYPE;
     STATUS : in    STATUS_INDICATOR  := NO_INDICATION)
    return CAIS_CALENDAR_TIME;

function TIME_ATTRIBUTE_WRITTEN
    (NAME   : in    PATHNAME ;
     STATUS : in    STATUS_INDICATOR  := NO_INDICATION)
    return  CAIS_CALENDAR.TIME;

subtype ITERATION_PREDICATE is LIST_TYPE;

function DEFAULT_PREDICATE
    (STATUS :  in STATUS_INDICATOR  := NO_INDICATION)
    return ITERATION_PREDICATE;

typo NODE_ITERATOR is limited private;

subtype RELATION_NAME_PATTERN  is RELATION_NAME;

subtype RELATIONSHIP_KEY_PATTERM is RELATIONSHIP_KEY;
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
type RELATIONSHIP_KIND is (PRIMARY, SECONDARY, BOTH);
type NODE_KIND_ARRAY   is array (CAIS_NATURAL range <>) of NODE_KIND;

procedure CREATE_ITERATOR
    ( ITERATOR              : in out NODE_ITERATOR;
      NODE                  : in     NODE_TYPE;
      RELATION              : in     RD_TYPE ;
      PREDICATE             : in     ITERATION_PREDICATE   := DEFAULT_PREDICATE;
      INCLUDE_SPECIALIZATIONS
                            : in     BOOLEAN               := TRUE;
      TIME_LIMIT            : in     CAIS_DURATION         := LONG_DELAY;
      STATUS                : in     STATUS_INDICATOR      := NO_INDICATION);

procedure CREATE_ITERATOR
    (ITERATOR               : in out NODE_ITERATOR;
      MODE                  : in     NODE_TYPE;
      KIND                  : in     NODE_KIND_ARRAY             := (FILE, STRUCTURAL);
      KEY                   : in     RELATIONSHIP_KEY_PATTERN := EMPTY_KEY_LIST ;
      RELATION              : in     RD_TYPE;
      KIND_OF_RELATION      : in     RELATIONSHIP_KIND           := PRIMARY ;
      STATUS                : in     STATUS_INDICATOR            := NO_INDICATION) ;

procedure CREATE_ITERATOR
    (ITERATOR               : in out NODE_ITERATOR;
      NODE                  : in     NODE_TYPE;
      KIND                  : in     NODE_KIND_ARRAY             := (FILE, STRUCTURAL);
      KEY                   : in     RELATIONSHIP_KEY_PATTERN := EMPTY_KEY_LIST ;
      RELATION              : in     RELATION_NAME_PATTERN    := DEFAULT_RELATION;
      KIND_OF_RELATION      : in     RELATIONSHIP_KIND        := PRIMARY;
      STATUS                : in     STATUS_INDICATOR         := NO_INDICATION);

procedure CREATE_ITERATOR
    (ITERATOR               : in out NODE_ITERATOR;
      NAME                  : in     PATHNAME;
      KIND                  : in     NODE_KIND_ARRAY             := (FILE, STRUCTURAL);
      KEY                   : in     RELATIONSHIP_KEY_PATTERN := "*".
      RELATION              : in     RELATION_NAME_PATTERN    := DEFAULT_RELATION;
      KIND_OF_RELATION      : in     RELATIONSHIP_KIND        := PRIMARY ;
      STATUS                : in     STATUS_INDICATOR         := NO_INDICATION);

function MORE
    (ITERATOR : in     NODE_ITERATOR;
     STATUS   : in     STATUS_INDICATOR := NO_INDICATION)
    return BOOLEAN;

function APPROXIMATE_SIZE
    (ITERATOR : in     NODE_ITERATOR;
     STATUS   : in     STATUS_INDICATOR := NO_INDICATION)
    return CAIS_NATURAL;

procedure GET_NEXT
    (ITERATOR      : in out NODE_ITERATOR;
     NEXT_NODE     : in out NODE_TYPE;
     INTENT        : in     INTENT_ARRAY   := (1 => NO_ACCESS);
     TIME_LIMIT    : in     CAIS_DURATION  := LONG_DELAY;
     STATUS        : in     STATUS_INDICATOR := NO_INDICATION);
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
procedure GET_NEXT
    (ITERATOR     : in out NODE_ITERATOR;
     NEXT-NODE : in out NODE_TYPE;
     INTENT       : in     INTENT_SPECIFICATION ;
     TIME_LIMIT : in     CAIS_DURATION          := LONG_DELAY;
     STATUS       : in     STATUS_INDICATOR       := NO_INDICATION);

procedure SKIP_NEXT
    (ITERATOR :  in out NODE_ITERATOR;
     STATUS     : in     STATUS_INDICATOR  := NO_INDICATION);

function NEXT_NAME
    (ITERATOR : in     NODE_ITERATOR;
     STATUS     : in     STATUS_INDICATOR  := NO_INDICATION)
    return PATHNAME;

procedure DELETE_ITERATOR
    (ITERATOR :  in out NODE_ITERATOR;
     STATUS     : in     STATUS_INDICATOR  := NO_INDICATION);

type NODE_KIND_INSTANCE_ITERATOR is limited private;

procedure CREATE_ITERATOR
    (INTERATOR                  : in out NODE_KIND_INSTANCE_ITERATOR;
     NODE_KIND                  : in     NKD_TYPE;
     PREDICATE                  : in     ITERATION_PREDICATE := DEFAULT_PREDICATE;
     INCLUDE_SPECIALIZATIONS : in     BOOLEAN             := TRUE;
     TIME_LIMIT                 : in     CAIS_DURATION       := LONG_DELAY;
     STATUS                     : in     STATUS_INDICATOR    := NO_INDICATION);

function MORE
    (ITERATOR : in     NODE_KIND_INSTANCE_ITERATOR;
     STATUS     : in     STATUS_INDICATOR  := NO_INDICATION)
    return BOOLEAN;

function APPROXIMATE_SIZE
    (ITERATOR : in     NODE_KIND_INSTANCE_ITERATOR;
     STATUS     : in     STATUS_INDICATOR  := NO_INDICATION)
    return CAIS_NATURAL;

procedure GET_NEXT
    (ITERATOR     : in out NODE_KIND_INSTANCE_ITERATOR;
     NEXT_NODE : in out NODE_TYPE;
     INTENT       : in     INTENT_ARRAY       := (1 => NO_ACCESS);
     TIME_LIMIT : in     CAIS_DURATION      := LONG_DELAY;
     STATUS       : in     STATUS_INDICATOR   := NO_INDICATION);

procedure SKIP_NEXT
    (ITERATOR : in out NODE_KIND_INSTANCE_ITERATOR;
     STATUS     : in     STATUS_INDICATOR  := NO_INDICATION);
```


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
        procedure DELETE_ITERATOR
            (ITERATOR : in out NODE_KIND_INSTANCE_ITERATOR;
             STATUS   : in      STATUS_INDICATOR := NO_INDICATION);

private

        typo NODE_ITERATOR is (IMPLEMENTATION_DEFINED);
        -- This type should be defined by the implementer.

        type NODE_KIND_INSTANCE_ITERATOR is (IMPLEMENTATION_DEFINED);
        -- This type should be defined by the implementer.

        type RELATION_INSTANCE_ITERATOR is (IMPLEMENTATION_DEFINED);
        -- This type should be defined by the implementor.

end CAIS_NODE_MANAGEMENT;
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
with CAIS_STANDARD;
with CAIS_DEFINITIONS;
with CAIS_LIST_MANAGEMENT;
with CAIS_STATUS_DEFINITIONS;

package CAIS_ATTRIBUTE_MANAGEMENT is

    use CAIS_STANDARD;
    use CAIS_DEFINITIONS;
    use CAIS_LIST_MANAGEMENT;
    use CAIS_STATUS_DEFINITIONS;

    procedure CREATE_NODE_ATTRIBUTE
        (NODE      : in  NODE_TYPE;
         ATTRIBUTE : in  AD_TYPE;
         VALUE     : in  LIST_TYPE;
         STATUS    : in  STATUS_INDICATOR := NO_INDICATION);

    procedure CREATE_NODE_ATTRIBUTE
        (NODE      : in  NODE_TYPE;
         ATTRIBUTE : in  ATTRIBUTE_NAME;
         VALUE     : in  LIST_TYPE;
         STATUS    : in  STATUS_INDICATOR := NO_INDICATION);

    procedure CREATE_NODE_ATTRIBUTE
        (NAME      : in  PATHNAME;
         ATTRIBUTE : in  ATTRIBUTE_NAME;
         VALUE     : in  LIST_TYPE;
         STATUS    : in  STATUS_INDICATOR := NO_INDICATION);

    procedure CREATE_PATH_ATTRIBUTE
        (BASE      : in  NODE_TYPE;
         KEY       : in  RELATIOHSHIP_KEY := EMPTY_KEY_LIST;
         RELATION  : in  RD_TYPE;
         ATTRIBUTE : in  AD_TYPE;
         VALUE     : in  LIST_TYPE;
         STATUS    : in  STATUS_INDICATOR := NO_INDICATION);

    procedure CREATE_PATH_ATTRIBUTE
        (BASE      : in  NODE_TYPE;
         KEY       : in  RELATIONSHIP_KEY := EMPTY_KEY_LIST;
         RELATION  : in  RELATION_NAME    := DEFAULT_RELATION;
         ATTRIBUTE : in  ATTRIBUTE_NAME;
         VALUE     : in  LIST_TYPE;
         STATUS    : in  STATUS_INDICATOR := NO_INDICATION);

    procedure CREATE_PATH_ATTRIBUTE
        (NAME      : in  PATHNAME;
         ATTRIBUTE : in  ATTRIBUTE_NAME;
         VALUE     : in  LIST_TYPE;
         STATUS    : in  STATUS_INDICATOR := NO_INDICATION);
```


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
procedure DELETE_ MODE_ ATTRIBUTE
     (NODE      : in    NODE_TYPE ;
      ATTRIBUTE : in    AD_ TYPE;
      STATUS    : in    STATUS_ INDICATOR  := NO_INDICATION);

procedure DELETE_NODE_ATTRIBUTE
     (NODE      : in    NODE_TYPE;
      ATTRIBUTE : in    ATTRIBUTE_NAME ;
      STATUS    : in    STATUS_INDICATOR :=  NO_INDICATION);

procedure DELETE_NODE_ATTRIBUTE
     (NAME      : in    PATHNAME;
      ATTRIBUTE : in    ATTRIBUTE_NAME;
      STATUS    : in    STATUS_INDICATOR :=  NO_INDICATION);

procedure DELETE_PATH_ATTRIBUTE
     (BASE      : in    NODE_TYPE;
      KEY       : in    RELATIONSHIP_KEY :=  EMPTY_KEY_LIST;
      RELATION  : in    RD_TYPE;
      ATTRIBUTE : in    AD_TTPE;
      STATUS    : in    STATUS_INDICATOR :=  NO_INDICATION);

procedure DELETE_PATH_ATTRIBUTE
     (BASE      : in    NODE_TYPE;
      KEY       : in    RELATIONSHIP_KEY :=  EMPTY_KEY_LIST;
      RELATION  : in    RELATION_NAME :=  DEFAULT_RELATION;
      ATTRIBUTE : in    ATTRIBUTE_NAME ;
      STATUS    : in    STATUS_INDICATOR :=  NO_INDICATION);

procedure DELETE_PATH_ATTRIBUTE
     (NAME      : in    PATHNAME;
      ATTRIBUTE : in    ATTRIBUTE_NAME;
      STATUS    : in    STATUS_INDICATOR :=  NO_INDICATION);

procedure  SET_NODE_ATTRIBUTE
     (NODE      : in    NODE_TYPE;
      ATTRIBUTE : in    AD_TYPE;
      VALUE     : in    LIST_TYPE;
      STATUS    : in    STATUS_INDICATOR :=  NO_INDICATION);

procedure  SET_NODE_ATTRIBUTE
     (NODE      : in    NODE_TYPE;
      ATTRIBUTE : in    ATTRIBUTE_NAME;
      VALUE     : in    LIST_TYPE;
      STATUS    : in    STATUS_INDICATOR :=  NO_INDICATION);

procedure  SET_NODE_ATTRIBUTE
     (NAME      : in    PATHNAME;
      ATTRIBUTE : in    ATTRIBUTE_NAME;
      VALUE     : in    LIST_TYPE;
      STATUS    : in    STATUS_INDICATOR :=  NO_INDICATION);
```


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
procedure SET_PATH_ATTRIBUTE
    (BASE      : in     NODE_TYPE;
     KEY       : in     RELATIONSHIP_KEY := EMPTY_KEY_LIST;
     RELATION  : in     RD_TYPE;
     ATTRIBUTE : in     AD_TYPE;
     VALUE     : in     LIST_TYPE;
     STATUS    : in     STATUS_INDICATOR := NO_INDICATION);

procedure SET_PATH_ATTRIBUTE
    (BASE      : in     NODE_TYPE;
     KEY       : in     RELATIONSHIP_KEY := EMPTY_KEY_LIST;
     RELATION  : in     RELATION_NAME := DEFAULT_RELATION;
     ATTRIBUTE : in     ATTRIBUTE_NAME;
     VALUE     : in     LIST_TYPE;
     STATUS    : in     STATUS_INDICATOR := NO_INDICATION);

procedure SET_PATH_ATTRIBUTE
    (NAME      : in     PATHNAME;
     ATTRIBUTE : in     ATTRIBUTE_NAME;
     VALUE     : in     LIST_TYPE;
     STATUS    : in     STATUS_INDICATOR := NO_INDICATION);

procedure GET_NODE_ATTRIBUTE
    (NODE      : in     NODE_TYPE;
     ATTRIBUTE : in     AD_TYPE;
     VALUE     : in out LIST_TYPE;
     STATUS    : in     STATUS_INDICATOR := NO_INDICATION);

procedure GET_NODE_ATTRIBUTE
    (NODE      : in     NODE_TYPE;
     ATTRIBUTE : in     ATTRIBUTE_NAME;
     VALUE     : in out LIST_TYPE;
     STATUS    : in     STATUS_INDICATOR := NO_INDICATION);

procedure GET_NODE_ATTRIBUTE
    (NAME      : in     PATHNAHE;
     ATTRIBUTE : in     ATTRIBUTE_NAME;
     VALUE     : in out LIST_TYPE;
     STATUS    : in     STATUS_INDICATOR := NO_INDICATION);

procedure GET_PATH_ATTRIBUTE
    (BASE      : in     NODE_TYPE;
     KEY       : in     RELATIONSHIP_KEY := EMPTY_KEY_LIST;
     RELATION  : in     RD_TYPE;
     ATTRIBUTE : in     AD_TYPE;
     VALUE     : in out LIST_TYPE;
     STATUS    : in     STATUS_INDICATOR := NO_INDICATION);

procedure GET_PATH_ATTRIBUTE
    (BASE      : in     NODE_TYPE;
     KEY       : in     RELATIONSHIP_KEY := EMPTY_KEY_LIST;
     RELATION  : in     RELATION_NAME := DEFAULT_RELATION;
     ATTRIBUTE : in     ATTRIBUTE_NAME;
     VALUE     : in out LIST_TYPE;
     STATUS    : in     STATUS_INDICATOR := NO_INDICATION);
```


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
procedure GET_ PATE_ ATTRIBUTE
    (NAME      : in     PATHNAME;
     ATTRIBUTE : in     ATTRIBUTE_NAME;
     VALUE     : in out LIST_ TYPE;
     STATUS    : in     STATUS_INDICATOR := NO_INDICATION);
```


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
generic
    type NUMBER is range <>;
package CAIS_INTEGER_ATTRIBUTE is

    procedure  CREATE_NODE_ATTRIBUTE
        (NODE      : in    NODE_TYPE;
         ATTRIBUTE : in    INTEGER_AD_TYPE;
         VALUE     : in    NUMBER;
         STATUS    : in    STATUS_INDICATOR := NO_INDICATION);

    procedure  CREATE_NODE_ATTRIBUTE
        (NODE      : in    NODE_TYPE;
         ATTRIBUTE : in    ATTRIBUTE_NAME;
         VALUE     : in    NUMBER;
         STATUS    : in    STATUS_INDICATOR := NO_INDICATION);

    procedure CREATE_PATE_ATTRIBUTE
        (BASE      : in    NODE_TYPE;
         KEY       : in    RELATIONSHIP_KEY := EMPTY_KEY_LIST;
         RELATION  : in    RD_TYPE;
         ATTRIBUTE : in    INTEGER_AD_TYPE;
         VALUE     : in    NUMBER;
         STATUS    : in    STATUS_INDICATOR := NO_INDICATION);

    procedure CREATE_PATH_ATTRIBUTE
        (BASE      : in    NODE_TYPE;
         KEY       : in    RELATIONSHIP_KEY := EMPTY_KEY_LIST;
         RELATION  : in    RELATION_NAME := DEFAULT_RELATION;
         ATTRIBUTE : in    ATTRIBUTE_NAME;
         VALUE     : in    NUMBER;
         STATUS    : in    STATUS_INDICATOR := NO_INDICATION);

    procedure  SET_NODE_ATTRIBUTE
        (NODE      : in    NODE_TYPE;
         ATTRIBUTE : in    INTEGER_AD_TYPE;
         VALUE     : in    NUMBER;
         STATUS    : in    STATUS_INDICATOR := NO_INDICATION);

    procedure  SET_NODE_ATTRIBUTE
        (NODE      : in    NODE_TYPE;
         ATTRIBUTE : in    ATTRIBUTE_NAME;
         VALUE     : in    NUMBER;
         STATUS    : in    STATUS_INDICATOR := NO_INDICATION);

    procedure  SET_PATH_ATTRIBUTE
        (BASE      : in    NODE_TYPE;
         KEY       : in    RELATIONSHIP_KEY := EMPTY_KEY_LIST;
         RELATION  : in    RD_TYPE;
         ATTRIBUTE : in    INTEGER_AD_TYPE;
         VALUE     : in    NUMBER;
         STATUS    : in    STATUS_INDICATOR := NO_INDICATION);
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
    procedure SET_PATH_ATTRIBUTE
        (BASE      : in  NODE_TYPE;
         KEY       : in  RELATIOHSHIP_KEY := EMPTY_KEY_LIST;
         RELATION  : in  RELATION_NAME := DEFAULT_RELATION;
         ATTRIBUTE : in  ATTRIBUTE_NAME;
         VALUE     : in  NUMBER;
         STATUS    : in  STATUS_INDICATOR := NO_INDICATION);

    function NODE_ATTRIBUTE
        (NODE      : in  NODE_TYPE;
         ATTRIBUTE : in  INTEGER_AD_TYPE;
         STATUS    : in  STATUS_INDICATOR := NO_INDICATION)
        return NUMBER;

    function NODE_ATTRIBUTE
        (NODE      : in  NODE_TYPE;
         ATTRIBUTE : in  ATTRIBUTE_NAME;
         STATUS    : in  STATUS_INDICATOR := NO_INDICATION)
        return NUMBER;

    function PATH_ATTRIBUTE
        (BASE      : in  NODE_TYPE;
         KEY       : in  RELATIONSHIP_KEY := EMPTY_KEY_LIST;
         RELATION  : in  RD_TYPE ;
         ATTRIBUTE : in  INTEGER_AD_TYPE;
         STATUS    : in  STATUS_INDICATOR := NO_INDICATION)
        return NUMBER;

    function PATH_ATTRIBUTE
        (BASE      : in  NODE_TYPE;
         KEY       : in  RELATIONSHIP_KEY := EMPTY_KEY_LIST;
         RELATION  : in  RELATION_NAME := DEFAULT_RELATION;
         ATTRIBUTE : in  ATTRIBUTE_NAME;
         STATUS    : in  STATUS_INDICATOR := NO_INDICATION)
        return NUMBER;

end CAIS_INTEGER_ATTRIBUTE;
```


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
generic
    type NUMBER is digits <>;
package CAIS_FLOAT_ATTRIBUTE is

    procedure CREATE_NODE_ATTRIBUTE
        (NODE      : in  NODE_TYPE;
         ATTRIBUTE : in  FLOAT_AD_TYPE;
         VALUE     : in  NUMBER ;
         STATUS    : in  STATUS_INDICATOR := NO_INDICATION);

    procedure CREATE_NODE_ATTRIBUTE
        (NODE      : in  NODE_TYPE;
         ATTRIBUTE : in  ATTRIBUTE_NAME;
         VALUE     : in  NUMBER;
         STATUS    : in  STATUS_INDICATOR := NO_INDICATION);

    procedure CREATE_PATE_ATTRIBUTE
        (BASE      : in  NODE_TYPE;
         KEY       : in  RELATIONSHIP_KEY := EMPTY_KEY_LIST ;
         RELATION  : in  RD_TYPE;
         ATTRIBUTE : in  FLOAT_AD_TYPE;
         VALUE     : in  NUMBER;
         STATUS    : in  STATUS_INDICATOR := NO_INDICATION);

    procedure CREATE_PATE_ATTRIBUTE
        (BASE      : in  NODE_TYPE;
         KEY       : in  RELATIONSHIP_KEY := EMPTY_KEY_LIST;
         RELATION  : in  RELATION_NAME := DEFAULT_RELATION;
         ATTRIBUTE : in  ATTRIBUTE_NAME;
         VALUE     : in  NUMBER;
         STATUS    : in  STATUS_INDICATOR := NO_INDICATION);

    procedure SET_NODE_ATTRIBUTE
        (NODE      : in  NODE_TYPE;
         ATTRIBUTE : in  FLOAT_AD_TYPE;
         VALUE     : in  NUMBER;
         STATUS    : in  STATUS_INDICATOR := NO_INDICATION);

    procedure SET_NODE_ATTRIBUTE
        (NODE      : in  NODE_TYPE;
         ATTRIBUTE : in  ATTRIBUTE_NAME;
         VALUE     : in  NUMBER;
         STATUS    : in  STATUS_INDICATOR := NO_INDICATION);

    procedure SET_PATH_ATTRIBUTE
        (BASE      : in  NODE_TYPE;
         KEY       : in  RELATIONSHIP_KEY := EMPTY_KEY_LIST;
         RELATION  : in  RD_TYPE;
         ATTRIBUTE : in  FLOAT_AD_TYPE;
         VALUE     : in  NUMBER;
         STATUS    : in  STATUS_INDICATOR := NO_INDICATION);
```


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
    procedure SET_PATH_ATTRIBUTE
        (BASE      : in  NODE_TYPE;
         KEY       : in  RELATIONSHIP_KEY  := EMPTY_KEY_LIST;
         RELATION  : in  RELATION_NAME     := DEFAULT_RELATION;
         ATTRIBUTE : in  ATTRIBUTE_NAME;
         VALUE     : in  NUMBER;
         STATUS    : in  STATUS_INDICATOR  := NO_INDICATION);

    function NODE_ATTRIBUTE
        (NODE      : in  NODE_TYPE;
         ATTRIBUTE : in  FLOAT_AD_TYPE;
         STATUS    : in  STATUS_INDICATOR  := NO_INDICATION)
        return NUMBER;

    function NODE_ATTRIBUTE
        (NODE      : in  NODE_TYPE;
         ATTRIBUTE : in  ATTRIBUTE_NAME;
         STATUS    : in  STATUS_INDICATOR  := NO_INDICATION)
        return NUMBER;

    function PATH_ATTRIBUTE
        (BASE      : in  NODE_TYPE;
         KEY       : in  RELATIONSHIP_KEY  := EMPTY_KEY_LIST;
         RELATION  : in  RD_TYPE;
         ATTRIBUTE : in  FLOAT_AD_TYPE;
         STATUS    : in  STATUS_INDICATOR  := NO_INDICATION)
        return NUMBER;

    function PATH_ATTRIBUTE
        (BASE      : in  NODE_TYPE;
         KEY       : in  RELATIONSHIP_KEY  := EMPTY_KEY_LIST;
         RELATION  : in  RELATION_NAME     := DEFAULT_RELATION;
         ATTRIBUTE : in  ATTRIBUTE_NAME;
         STATUS    : in  STATUS_INDICATOR  := NO_INDICATION)
        return NUMBER;

end CAIS_FLOAT_ATTRIBUTE;
```


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
package CAIS_STRING_ATTRIBUTE is

    procedure  CREATE_MODE_ATTRIBUTE
        (NODE      : in    NODE_TYPE;
         ATTRIBUTE : in    STRING_AD_TYPE;
         VALUE     : in    STRING ;
         STATUS    : in    STATUS_INDICATON := NO_INDICATION);

    procedure  CREATE_NODE_ATTRIBUTE
        (NODE      : in    NODE_TYPE;
         ATTRIBUTE : in    ATTRIBUTE_NAME;
         VALUE     : in    STRING;
         STATUS    : in    STATUS_INDICATOR :=  NO_INDICATION);


    procedure CREATE_PATH_ATTRIBUTE
        (BASE      : in    NODE_TYPE;
         KEY       : in    RELATIONSHIP_KEY :=  EMPTY_KEY_LIST;
         RELATION  : in    RD_TYPE;
         ATTRIBUTE : in    STRING_AD_TYPE;
         VALUE     : in    STRING ;
         STATUS    : in    STATUS_INDICATOR :=  NO_INDICATION);

    procedure CREATE_PATH_ATTRIBUTE
        (BASE      : in    NODE_TYPE;
         KEY       : in    RELATIONSHIP_KEY :=  EMPTY_KEY_LIST;
         RELATION  : in    RELATION_NAME :=  DEFAULT_RELATION;
         ATTRIBUTE : in    ATTRIBUTE_NAME;
         VALUE     : in    STRING;
         STATUS    : in    STATUS_INDICATOR :=  NO_INDICATION);

    procedure  SET_NODE_ATTRIBUTE
        (NODE      : in    NODE_TYPE;
         ATTRIBUTE : in    STRING_AD_TYPE;
         VALUE     : in    STRING;
         STATUS    : in    STATUS_INDICATOR :=  NO_INDICATION);

    procedure  SET_NODE_ATTRIBUTE
        (NODE      : in    NODE_TYPE;
         ATTRIBUTE : in    ATTRIBUTE_NAME;
         VALUE     : in    STRING;
         STATUS    : in    STATUS_INDICATOR :=  NO_INDICATION);

    procedure  SET_PATH_ATTRIBUTE
        (BASE      : in    NODE_TYPE;
         KEY       : in    RELATIONSHIP_KEY :=  EMPTY_KEY_LIST;
         RELATION  : in    RD_TYPE;
         ATTRIBUTE : in    STRING_AD_TYPE;
         VALUE     : in    STRING;
         STATUS    : in    STATUS_INDICATOR :=  NO_INDICATION) ;
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
    procedure SET_PATH_ATTRIBUTE
        (BASE      : in  MODE_TYPE;
         KEY       : in  RELATIONSHIP_KEY  := EMPTY_KEY_LIST;
         RELATION  : in  RELATION_NAME     := DEFAULT_RELATION;
         ATTRIBUTE : in  ATTRIBUTE_NAME;
         VALUE     : in  STRING;
         STATUS    : in  STATUS_INDICATOR  := NO_INDICATION);

    function NODE_ATTRIBUTE
        (NODE      : in  NODE_TYPE;
         ATTRIBUTE : in  STRING_AD_TYPE;
         STATUS    : in  STATUS_INDICATOR  := NO_INDICATION)
        return STRING;

    function NODE_ATTRIBUTE
        (NODE      : in  NODE_TYPE;
         ATTRIBUTE : in  ATTRIBUTE_NAME;
         STATUS    : in  STATUS_INDICATOR  := NO_INDICATION)
        return STRING;

    function PATH_ATTRIBUTE
        (BASE      : in  NODE_TYPE;
         KEY       : in  RELATIONSHIP_KEY  := EMPTY_KEY_LIST;
         RELATION  : in  RD_TYPE;
         ATTRIBUTE : in  STRING_AD_TYPE;
         STATUS    : in  STATUS_INDICATOR  := NO_INDICATION)
        return STRING;

    function PATH_ATTRIBUTE
        (BASE      : in  NODE_TYPE;
         KEY       : in  RELATIONSHIP_KEY  := EMPTY_KEY_LIST;
         RELATION  : in  RELATION_NAME     := DEFAULT_RELATION;
         ATTRIBUTE : in  ATTRIBUTE_NAME;
         STATUS    : in  STATUS_INDICATOR  := NO_INDICATION)
        return STRING;

end CAIS_STRING_ATTRIBUTE;
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
package CAIS_IDENTIFIER_ATTRIBUTE is

    procedure CREATE_NODE_ATTRIBUTE
        (NODE      : in  NODE_TYPE;
         ATTRIBUTE : in  IDENTIFIER_AD_TYPE ;
         VALUE     : in  TOKEN_TYPE;
         STATUS    : in  STATUS_INDICATOR := NO_INDICATION);

    procedure CREATE_NODE_ATTRIBUTE
        (NODE      : in  NODE_TYPE;
         ATTRIBUTE : in  ATTRIBUTE_NAME;
         VALUE     : in  TOKEN_TYPE;
         STATUS    : in  STATUS_INDICATOR := NO_INDICATION);

    procedure CREATE_PATH_ATTRIBUTE
        (BASE      : in  NODE_TYPE;
         KEY       : in  RELATIONSHIP_KEY := EMPTY_KEY_LIST;
         RELATION  : in  RD_TYPE ;
         ATTRIBUTE : in  IDENTIFIER_AD_TYPE ;
         VALUE     : in  TOKEN_TYPE;
         STATUS    : in  STATUS_INDICATOR := NO_INDICATION);

    procedure CREATE_PATH_ATTRIBUTE
        (BASE      : in  NODE_TYPE;
         KEY       : in  RELATIONSHIP_KEY := EMPTY_KEY_LIST;
         RELATION  : in  RELATION_NAME := DEFAULT_RELATION;
         ATTRIBUTE : in  ATTRIBUTE_NAME;
         VALUE     : in  TOKEN_TYPE;
         STATUS    : in  STATUS_INDICATOR := NO_INDICATION) ;

    procedure SET_NODE_ATTRIBUTE
        (NODE      : in  NODE_TYPE;
         ATTRIBUTE : in  IDENTIFIER_AD_TYPE ;
         VALUE     : in  TOKEN_TYPE;
         STATUS    : in  STATUS_INDICATOR := NO_INDICATION);

    procedure SET_NODE_ATTRIBUTE
        (NODE      : in  NODE_TYPE;
         ATTRIBUTE : in  ATTRIBUTE_NAME;
         VALUE     : in  TOKEN_TYPE;
         STATUS    : in  STATUS_INDICATOR := NO_INDICATION);

    procedure SET_PATH_ATTRIBUTE
        (BASE      : in  NODE_TYPE;
         KEY       : in  RELATIONSHIP_KEY := EMPTY_KEY_LIST;
         RELATION  : in  RD_TYPE;
         ATTRIBUTE : in  IDENTIFIER_AD_TYPE;
         VALUE     : in  TOKEN_TYPE;
         STATUS    : in  STATUS_INDICATOR := NO_INDICATION);

    procedure SET_PATH_ATTRIBUTE
        (BASE      : in  NODE_TYPE;
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
            KEY          : in      RELATIOINSHIP_KEY := EMPTY_KEY_LIST;
            RELATION     : in      RELATION_NAME := DEFAULT_RELATION;
            ATTRIBUTE    : in      ATTRIBUTE_NAME;
            VALUE        : in      TOKEN_TYPE;
            STATUS       : in      STATUS_INDICATOR := NO_INDICATION);

        procedure GET_NODE_ATTRIBUTE
            (NODE        : in      NODE_TYPE;
             ATTRIBUTE   : in      IDENTIFIER_AD_TYPE;
             VALUE       : in out  TOKEN_TYPE;
             STATUS      : in      STATUS_INDICATOR := NO_INDICATION);

        procedure GET_NODE_ATTRIBUTE
            (NODE        : in      NODE_TYPE;
             ATTRIBUTE   : in      ATTRIBUTE_NAME;
             VALUE       : in out  TOKEN_TYPE;
             STATUS      : in      STATUS_INDICATOR := NO_INDICATION);

        procedure GET_PATH_ATTRIBUTE
            (BASE        : in      NODE_TYPE;
             KEY         : in      RELATIONSHIP_KEY := EMPTY_KEY_LIST;
             RELATION    : in      RD_TYPE;
             ATTRIBUTE   : in      IDENTIFIER_AD_TYPE ;
             VALUE       : in  out TOKEN_TYPE;
             STATUS      : in      STATUS_INDICATOR := NO_INDICATION);

        procedure GET_PATH_ATTRIBUTE
            (BASE        : in      NODE_TYPE;
             KEY         : in      RELATIONSHIP_KEY := EMPTY_KEY_LIST;
             RELATION    : in      RELATION_NAME := DEFAULT_RELATION;
             ATTRIBUTE   : in      ATTRIBUTE_NAME;
             VALUE       : in out  TOKEN_TYPE;
             STATUS      : in      STATUS_INDICATOR := NO_INDICATION);

    end CAIS_IDENTIFIER_ATTRIBUTE;
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
generic
    type ENUMERATION is (<>);
package CAIS_ENUMERATION_ATTRIBUTE is

    procedure CREATE_NODE_ATTRIBUTE
        (NODE      : in  NODE_TYPE;
         ATTRIBUTE : in  IDENTIFIER_AD_TYPE;
         VALUE     : in  ENUMERATION;
         STATUS    : in  STATUS_INDICATOR := NO_INDICATION);

    procedure  CREATE_NODE_ATTRIUTE
        (NODE      : in  MODE_TYPE;
         ATTRIBUTE : in  ATTRIBUTE_NAME ;
         VALUE     : in  ENUMERATION;
         STATUS    : in  STATUS_IlfDICATOR  :=  NO_INDICATION);

    procedure CREATE_PATH_ATTRIBUTE
        (BASE      : in  NODE_TYPE;
         KEY       : in  RELATIONSHIP_KEY:=  EMPTY_KEY_LIST ;
         RELATION  : in  RD_TYPE;
         ATTRIBUTE : in  IDENTIFIER_AD_TYPE;
         VALUE     : in  ENUMERATION;
         STATUS    : in  STATUS_INDICATOR :=  NO_INDICATION);

    procedure CREATE_PATH_ATTRIBUTE
        (BASE      : in  NODE_TYPE;
         KEY       : in  RELATIOISHIP_KEY :=  EMPTY_KEY_LIST;
         RELATION  : in  RELATION_NAME :=  DEFAULT_RELATION;
         ATTRIBUTE : in  ATTRIBUTE_NAME ;
         VALUE     : in  ENUMERATION;
         STATUS    : in  STATUS_INDICATOR :=  NO_INDICATION);

    procedure  SET_HODE_ATTRIBUTE
        (NODE      : in  BODE_TYPE;
         ATTRIBUTE : in  IDENTIFIER_AD_TYPE;
         VALUE     : in  ENUMERATION;
         STATUS    : in  STATUS_INDICATOR :=  NO_INDICATION);

    procedure  SET_NODE_ATTRIBUTE
        (NODE      : in  NODE_TYPE;
         ATTRIBUTE : in  ATTRIBUTE_NAME;
         VALUE     : in  ENUMERATION;
         STATUS    : in  STATUS_INDICATOR :=  NO_INDICATION);

    procedure  SET_PATH_ATTRIBUTE
        (BASE      : in  NODE_TYPE;
         KEY       : in  RELATIONSHIP_KEY :=  EMPTY_KEY_LIST;
         RELATION  : in  RD_TYPE;
         ATTRIBUTE : in  IDENTIFIER_AD_TYPE;
         VALUE     : in  EMUMERATION;
         STATUS    : in  STATUS_INDICATOR:=   NO_INDICATION);
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
        proceduro SET_ PATH_ ATTRIBUTE
            (BASE      : in    MODE_TYPE;
             KEY       : in    RELATIONSHIP_KEY   := EMPTY_KEY_LIST;
             RELATION  : in    RELATION_NAME      := DEFAULT_RELATION;
             ATTRIBUTE : in    ATTRIBUTE_NAME ;
             VALUE     : in    EMUMERATION;
             STATUS    : in    STATUS_INDICATOR   := NO_INDICATION);

        function NODE_ATTRIBUTE
            (NODE      : in    NODE_TYPE;
             ATTRIBUTE : in    IDENTIFIER_AD_TYPE;
             STATUS    : in    STATUS_INDICATOR   := NO_INDICATION)
           return ENUMERATION; ,

        function NODE_ATTRIBUTE
            (NODE      : in    NODE_TYPE;
             ATTRIBUTE : in    ATTRIBUTE_NAME ;
             STATUS    : in    STATUS_INDICATOR   := NO_INDICATION)
           return ENUMERATION;

        function PATH_ATTRIBUTE
            (BASE      : in    NODE_TYPE;
             KEY       : in    RELATIONSHIP_KEY   := EMPTY_KEY_LIST;
             RELATION  : in    RD_TYPE ;
             ATTRIBUTE : in    IDENTIFIER_AD_TYPE ;
             STATUS    : in    STATUS_INDICATOR   := NO_INDICATION)
           return ENUMERATION;

        function PATH_ATTRIBUTE
            (BASE      : in    NODE_TYPE;
             KEY       : in    RELATIONSHIP_KEY   := EHPTY_KEY_LIST;
             RELATION  : in    RELATION_NAME      := DEFAULT_RELATION;
             ATTRIBUTE : in    ATTRIBUTE_NAME ;
             STATUS    : in    STATUS_INDICATOR   := NO_INDICATION)
           return ENUMERATION;

    end  CAIS_ENUMERATION_ATTRIBUTE;
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
type. ATTRIBUTE. ITERATOR is limited private;

subtype ATTRIBUTE_NAME_PATTERN is STRING;

typo ATTRIBUTE_ITERATOR_SCOPE is (VISIBLE_ATTRIBUTES, ALL_ATTRIBUTES);

procodure  CREATE_NODE_ATTRIBUTE_ITERATOR
    (ITERATOR :  in out ATTRIBUTE_ITERATOR;
     NODE       : in      NODE_TYPE;
     PATTERN : in      ATTRIBUTE_PATTERN  :=  :="*".
     SCOPE      : in      ATTRIBUTE_ITERATOR_SCOPE :=  VISIBLE_ATTRIBUTES ;
     STATUS     : in      STATUS_INDICATOR         :=  NO_INDICATION);

procoduro CREATE_NODE_ATTRIBUTE_ITEERATOR
    (ITERATOR : in out  ATTRIBUTE_ITERATOR;
     NAME       : in      PATHNAME;
     PATTERN : in      ATTRIBUTE_NAME_PATTERN :=  "*":
     SCOPE      : in      ATTRIBUTE_ITERATOR_SCOPE :=  VISIBLE_ATTRIBUTES ;
     STATUS     : in      STATUS_INDICATOR         :=  NO_INDICATION);

procodure  CREATE_PATH_ATTRIBUTE_ITERATOR
    (ITERATOR :  in out ATTRIBUTE_ITERATOR;
     BASE       : in      NODE_TYPE;
     KEY        : in      RELATIONSHIP_KEY          EMPTY_KEY_LIST;
     RELATION : in      RD_TYPE ;
     PATTERN : in      ATTRIBUTE_NAME_PATTERN :=  "*".
     SCOPE      : in      ATTRIBUTE_ITERATOR_SCOPE :=  VISIBLE_ATTRIBUTES ;
     STATUS     : in      STATUS_INDICATOR         :=  NO_INDICATION) ;

procodure  CREATE_PATH_ATTRIBUTE_ITERATOR
    (ITERATOR :  in out ATTRIBUTE_ITERATOR;
     BASE       : in      NODE_TYPE;
     KEY        : in      RELATIONSHIP_KEY          :=  EMPTY_KEY_LIST;
     RELATION : in      RELATION_NAME             DEFAULT_RELATION  ;
     PATTERN : in      ATTRIBUTE_NAME_PATTERN :="*"  "*":
     SCOPE      : in      ATTRIBUTE_ITERATOR_SCOPE :=  VISIBLE_ATTRIBUTES ;
     STATUS     : in      STATUS_INDICATOR         :=  NO_INDICATION) ;

procedure  CREATE_PATH_ATTRIBUTE_ITERATOR
    (ITERATOR :  in out ATTRIBUTE_ITERATOR;
     NAME       : in      PATHNAME;
     PATTERN : in      ATTRIBUTE_NAME_PATTERH :=  "*":
     SCOPE      : in      ATTRIBUTE_ITERATOR_SCOPE :=  VISIBLE_ATTRIBUTES ;
     STATUS     : in      STATUS_INDICATOR         :=  NO_INDICATION) ;

function MORE
    (ITERATOR : in      ATTRIBUTE_ITERATOR ;
     STATUS     : in      STATUS_INDICATOR   := NO_INDICATION)
    return  BOOLEAM;

function  APPROXIMATE_SIZE
    (ITERATOR : in      ATTRIBUTE_ITERATOR ;
     STATUS     : in      STATUS_INDICATOR   := NO_INDICATION)
    return  CAIS_NATURAL;
```



Source: https://assist.dla.mil -- Downloaded: 2026l0-26T07:54Z
Check the source to verify that this is the currentversion before use.

```
    procedure NEXT_NAME
        (ATTRIBUTE : in out AD_ TYPE;
         ITERATOR : in      ATTRIBUTE_ITERATOR;
         STATUS   : in      STATUS_INDICATOR := NO_INDICATION);

    function NEXT_NAME
        (ITERATOR : in      ATTRIBUTE_ITERATOR ;
         STATUS   : in      STATUS_INDICATOR := NO_INDICATION)
        return ATTRIBUTE_NAME;

     procodure GET_NEXT_VALUE
         (ITERATOR : in out ATTRIBUTE_ITERATOR;
          VALUE    : in out LIST_TYPE;
          STATUS   : in      STATUS_INDICATOR := NO_INDICATION);

    procedure SKIP_NEXT
        (ITERATOR : in out ATTRIBUTE_ITERATOR;
         STATUS   : in      STATUS_INDICATOR := NO_INDICATION);

    procdure DELETE_ITERATOR
        (ITERATOR: in out ATTRIBUTE_ITERATOR;
         STATUS   : in      STATUS_INDICATOR := NO_INDICATION);

private

    type ATTRIBUTE_ITERATOR is (IMPLEMENTATION_DEFIliED);
    -- This typo should ber defined by the implementer.

end CAIS_ATTRIBUTE_MANAGEMENT;
```



Source: https://assist.dla.mil -- Downloaded: 2026l0-26T07:54Z
Check the source to verify that this is the currentversion before use.

```
with CAIS_DEFINITION;
with CAIS_ACCESS_CONTROL_MANAGEMENT;
with CAIS_LIST_MANAGEMENT;
with CAIS_STATUS_DEFINITIONS;
with CAIS_RESOURCE_CONTROL;
with CAIS_DEFINITION_MANAGEMENT;

package CAIS_STRUCTURAL_NODE_MANAGEMENT is

    use CAIS_DEFINITIONS;
    use CAIS_LIST_MANAGEMENT;
    use CAIS_STATUS_DEFINITIONS;
    use CAIS_RESOURCE_CONTROL;
    use CAIS_DEFINITION_MANAGEMENT;
    use CAIS_ACCESS_CONTROL_MANAGEMENT;

    function STRUCTURAL_NKD
        (STATUS : in      STATUS_INDICATOR  := NO_INDICATION)
        return NKD_TYPE;

    procedure CREATE_NODE
        (NODE                 : in out STRUCTURAL_NODE_TYPE;
         BASE                 : in     NODE_TYPE;
         KEY                  : in     RELATIONSHIP_KEY  := EMPTY_KEY_LIST;
         RELATION             : in     RD_TYPE;
         INTENT               : in     INTENT_ARRAY      := (1 => WRITE);
         ATTRIBUTES           : in     ATTRIBUTE_LIST    := EMPTY_LIST;
         DISCRETIONARY_ACCESS : in     DISCRETIONARY_ACCESS_LIST := ALL_RIGHTS;
         MANDATORY_ACCESS     : in     MANDATORY_ACCESS_LIST := EMPTY_LIST;
         KIND                 : in     NODE_KIND_NODE_TYPE := STRUCTURAL_NKD;
         STATUS               : in     STATUS_INDICATOR  := NO_INDICATION;
         MEDIUM_RESOURCE      : in     MEDIUM_NODE_TYPE  := DEFAULT_MEDIUM);

    procedure CREATE_NODE
        (NODE                 : in out STRUCTURAL_NODE_TYPE;
         BASE                 : in     NODE_TYPE;
         KEY                  : in     RELATIONSHIP_KEY  := EMPTY_KEY_LIST;
         RELATION             : in     RELATION_NAME     := DEFAULT_RELATION;
         INTENT               : in     INTENT_ARRAY      := (1 => WRITE);
         ATTRIBUTES           : in     ATTRIBUTE_LIST    := EMPTY_LIST;
         DISCRETIONARY_ACCESS : in     DISCRETIONARY_ACCESS_LIST := ALL_RIGHTS;
         MANDATORY_ACCESS     : in     MANDATORY_ACCESS_LIST := EMPTY_LIST;
         KIND                 : in     NODE_KIND_NAME    := "STRUCTURAL";
         STATUS               : in     STATUS_INDICATOR  := NO_INDICATION;
         MEDIUM_RESOURCE      : in     MEDIUM_NODE_TYPE  := DEFAULT_MEDIUM);

    procedure CREATE_NODE
        (NODE                 : in out STRUCTURAL_NODE_TYPE;
         NAME                 : in     PATHNAME;
         INTENT               : in     INTENT_ARRAY      := (1 => WRITE);
         ATTRIBUTES           : in     ATTRIBUTE_LIST    := EMPTY_LIST;
         DISCRETIONARY_ACCESS : in     DISCRETIONARY_ACCESS_LIST := ALL_RIGHTS;
         MANDATORY_ACCESS     : in     MANDATORY_ACCESS_LIST := EMPTY_LIST;
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
            KIND                    : in    NODE_KIND_NAME          := "STRUCTURAL";
            STATUS                  : in    STATUS_INDICATOR        :=  NO_INDICATION;
            MEDIUM_RESOURCE         : in    MEDIUM_NODE_TYPE        :=  DEFAULT_MEDIUM);

        procedure  CREATE_NODE
            (BASE                   : in    NODE_TYPE;
             KEY                    : in    RELATIONSHIP_KEY        := LATEST_KEY;
             RELATION               : in    RELATION_NAME           := DEFAULT_RELATION;
             INTENT                 : in    INTENT_ARRAY            := (l => WRITE);
             ATTRIBUTES             : in    ATTRIBUTE_LIST          := EMPTY_LIST;
             DISCRETIONARY_ACCESS : in    DISCRETIONARY_ACCESS_LIST  :=  ALL_RIGHTS;
             MANDATORY_ACCESS       : in    MANDATORY_ACCESS_LIST := EMPTY_LIST;
             KIND                   : in    NODE_KIND_NAME          := "STRUCTURAL";
             STATUS                 : in    STATUS_INDICATOR        :=  NO_INDICATION;
             MEDIUM_RESOURCE        : in    MEDIUM_NODE_TYPE        :=  DEFAULT_MEDIUM);

        procedure  CREATE_NODE
            (NAME                   : in    PATHNAME;
             INTENT                 : in    INTENT_ARRAY            := (1 => WRITE);
             ATTRIBUTES             : in    ATTRIBUTE_LIST          := EMPTY_LIST;
             DISCRETIONARY_ACCESS : in    DISCRETIONARY_ACCESS_LIST  :=  ALL_RIGHTS;
             MANDATORY_ACCESS       : in    MANDATORY_ACCESS_LIST := EMPTY_LIST;
             KIND                   : in    NODE_KIND_NAME          := "STRUCTURAL";
             STATUS                 : in    STATUS_INDICATOR        :=  NO_INDICATION;
             MEDIUM_RESOURCE        : in    NODE_TYPE               :=  DEFAULT_MEDIUM);

end    CAIS_STRUCTURAL_NODE_MANAGEMENT;
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
with CAIS_DEFINITIONS;
with CAIS_STANDARD;
with CAIS_PRAGMATICS;
with CAIS_LIST_MANAGEMENT;

package CAIS_PROCESS_DEFINITIONS is

        use CAIS_DEFINITIONS;
        use CAIS_STANDARD;

        type PROCESS_STATUS_KIND is (CREATED, READY, SUSPENDED,
                                     COMPLETED_ABNORMALLY, COMPLETED_NORMALLY,
                                          ABORTED, TERMINATED);

        subtype PROCESS_PRIORITY is CAIS_INTEGER;

        subtype RESULTS_LIST     is  CAIS_LIST_MANAGEMENT_LIST_TYPE;
        subtype RESULTS_STRING   is  STRING;
        subtype PARAMETER_LIST is CAIS_LIST_MANAGEMENT.LIST_TYPE;

        ROOT_PROCESS       : constant PATHNAME := "'CURRENT_JOB";
        STANDARD_INPUT     : constant PATHNAME  := "'STANDARD_INPUT";
        STANDARD_OUTPUT    : constant PATHNAME := "'STANDARD_OUTPUT";
        STANDARD_ERROR     : constant PATHNAME := "'STANDARD_ERROR";

        EXECUTABLE_IMAGE_ERROR :     exception;

end   CAIS_PROCESS_DEFINITIONS;
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
with CAIS_DEFINITIONS;
with CAIS_STANDARD;
with CAIS_STATUS_DEFINITIONS;

package CAIS_TRANSACTION_MANAGEMENT is

    use CAIS_DEFINITIONS;
    use CAIS_STANDARD;
    use CAIS_STATUS_DEFINITIONS;

    type TRANSACTION_TYPE is limited private;

    NULL_TRANSACTION : constant TRANSACTION_TYPE;

    procedure OPEN_TRANSACTION
        (TRANSACTION : in out TRANSACTION_TYPE;
         PARENT      : in     TRANSACTION_TYPE := NULL_TRANSACTION;
         STATUS      : in     STATUS_INDICATOR := NO_INDICATION);

    procedure INCLUDE
        (TRANSACTION : in     TRANSACTION_TYPE;
         NODE        : in     NODE_TYPE;
         TIME_LIMIT  : in     CAIS_DURATION := LONG_DELAY;
         STATUS      : in     STATUS_INDICATOR := NO_INDICATION);

    procedure INCLUDE_TREE
        (TRANSACTION : in     TRANSACTION_TYPE;
         NODE        : in     NODE_TYPE;
         TIME_LIMIT  : in     CAIS_DURATION := LONG_DELAY;
         STATUS      : in     STATUS_INDICATOR := NO_INDICATION);

    procedure CLOSE_TRANSACTION
        (TRANSACTION          : in out TRANSACTION_TYPE;
         COMMIT_MODIFICATION  : in     BOOLEAN           := TRUE;
         STATUS               : in     STATUS_INDICATOR  := NO_INDICATION);

    function IS_OPEN
        (TRANSACTION : in     TRANSACTION_TYPE;
         STATUS      : in     STATUS_INDICATOR  := NO_INDICATION)
        return BOOLEAN;

    function IS_SAME
        (TRANSACTION1 : in     TRANSACTION_TYPE;
         TRANSACTION2 : in     TRANSACTION_TYPE;
         STATUS       : in     STATUS_INDICATOR  := NO_INDICATION)
        return BOOLEAN;

    function IS_INCLUDED
        (NODE        : in     NODE_TYPE;
         TRANSACTION : in     TRANSACTION_TYPE := NULL_TRANSACTION;
         STATUS      : in     STATUS_INDICATOR := NO_INDICATION)
        return BOOLEAN;
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
private

    type TRANSACTION_TYPE is (IMPLEMENTATION_DEFINED);
    -- This type should be defined by the implementer.

    NULL_TRANSACTION : constant TRANSACTION_TYPE := IMPLEMEMTATION_DEFINED;
    -- This constant should be defined by the implementer.

end CAIS_TRANSACTION_MANAGEMENT;
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
with CAIS_STANDARD;
with CAIS_STATUS_DEFINITIONS;
with CAIS_DEFINITIONS;
with CAIS_LIST_MANAGEMENT;
with CAIS_CALENDAR;
with CAIS_RESOURCE_CONTROL;
with CAIS_PROCESS_DEFINITIONS;
with CAIS_ACCESS_CONTROL_MANAGEMENT;
with CAIS_TRANSACTION_MANAGEMENT;

package CAIS_PROCESS_MANAGEMENT is

    use CAIS_STANDARD;
    use CAIS_STATUS_DEFINITIONS;
    use CAIS_DEFINITIONS;
    use CAIS_LIST_MANAGEMENT;
    use CAIS_RESOURCE_CONTROL;
    use CAIS_PROCESS_DEFINITIONS;
    use CAIS_ACCESS_CONTROL_MANAGEMENT;
    use CAIS_TRANSACTION_MANAGEMENT;

    function DEFAULT_PRIORITY
        (PROCESSOR : in    PROCESSOR_NODE_TYPE  := DEFAULT_PROCESSOR;
         STATUS    : in    STATUS_INDICATOR     := NO_INDICATION)
        return PROCESS_PRIORITY;

    procedure CREATE_PROCESS_NODE
        (NODE                    : in out PROCESS_NODE_TYPE;
         FILE_NODE,              : in     EXECUTABLE_IMAGE_NODE_TYPE;
         INTENT                  : in     INTENT_ARRAY          := (READ, WRITE);
         INPUT_PARAMETERS        : in     PARAMETER_LIST        := EMPTY_LIST;
         KEY                     : in     RELATIONSHIP_KEY      := EMPTY_KEY_LIST;
         RELATION                : in     RD_TYPE;
         DISCRETIONARY_ACCESS    : in     DISCRETIONARY_ACCESS_LIST :=ALL_RIGHTS;
         ATTRIBUTES              : in     ATTRIBUTE_LIST        := EMPTY_LIST;
         INPUT_FILE              : in     PATHNAME              :=  "";
         OUTPUT_FILE             : in     PATHNAME              :=  "";
         ERROR_FILE              : in     PATHNAME              :=  "";
         ENVIRONMENT_NODE        : in     PATHNAME              := CURRENT_NODE;
         PROCESS_NODE_KIND       : in     NODE_KIND_NODE_TYPE := NULL_N0DE;
         ENCLOSING_TRANSACTION   : in     TRANSACTION_TYPE      := NULL_TRANSACTION;
         STATUS                  : in     STATUS_INDICATOR      := NO_INDICATION;
         PRIORITY                : in     PROCESS_PRIORITY      := DEFAULT_PRIORITY;
         PROCESSOR_RESOURCE      : in     PROCESSOR_NODE_TYPE   := DEFAULT_PROCESSOR;
         MEDIUM_RESOURCE         : in     MEDIUM_NODE_TYPE      := DEFAULT_MEDIUM);

    procedure CREATE_PROCESS_NODE
        (NODE                    : in out PROCESS_NODE_TYPE;
         FILE_MODE               : in     EXECUTABLE_IMAGE_NODE_TYPE;
         INTENT                  : in     INTENT_ARRAY          := (READ, WRITE);
         INPUT_PARAMETERS        : in     PARAMETER_LIST        := EMPTY_LIST;
         KEY                     : in     RELATIONSHIP_KEY      := EMPTY_KEY_LIST;
         RELATION                : in     RELATION_NAME         := DEFAULT_RELATION;
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
        DISCRETIONARY_ACCESS    : in    DISCRETIONARY_ACCESS_LIST := ALL_RIGHTS;
        ATTRIBUTES              : in    ATTRIBUTE_LIST  := EMPTY_LIST;
        INPUT_FILE              : in    PATHNAME        := "";
        OUTPUT_FILE             : in    PATHNAME        := "";
        ERROR_FILE              : in    PATHNAME        := "";
        ENVIRONMENT_NODE        : in    PATHNAME        := CURRENT_NODE;
        PROCESS_NODE_KIND       : in    NODE_KIND_NAME  := "";
        ENCLOSING_TRANSACTION   : in    TRANSACTION_TYPE := NULL_TRANSACTION;
        STATUS                  : in    STATUS_INDICATOR := NO_INDICATION;
        PRIORITY                : in    PROCESS_PRIORITY := DEFAULT_PRIORITY;
        PROCESSOR_RESOURCE      : in    PROCESSOR_NODE_TYPE := DEFAULT_PROCESSOR;
        MEDIUM_RESOURCE         : in    MEDIUM_NODE_TYPE := DEFAULT_MEDIUM);

    procedure CREATE_JOB_NODE
        (NODE                   : in out PROCESS_NODE_TYPE;
        FILE_NODE               : in    EXECUTABLE_IMAGE_NODE_TYPE;
        INTENT                  : in    INTENT_ARRAY            := (READ, WRITE);
        INPUT_PARAMETERS        : in    PARAMETER_LIST          := EMPTY_LIST;
        KEY                     : in    RELATIONSHIP_KEY        := EMPTY_KEY_LIST;
        DISCRETIONARY_ACCESS    : in    DISCRETIONARY_ACCESS_LIST :=ALL_RIGHTS;
        MANDATORY_ACCESS        : in    MANDATORY_ACCESS_LIST := EMPTY_LIST;
        ATTRIBUTES              : in    ATTRIBUTE_LIST          := EMPTY_LIST;
        INPUT_FILE              : in    PATHNAME                := "";
        OUTPUT_FILE             : in    PATHNAME                := "";
        ERROR_FILE              : in    PATHNAME                := "";
        ENVIRONMENT_NODE        : in    PATHNAME                := CURRENT_USER;
        DELETE_WHEN_TERMINATED  : in    BOOLEAN                 := TRUE;
        PROCESS_NODE_KIND       : in    NODE_KIND_NODE_TYPE     := NULL_NODE;
        AS_A_TRANSACTION        : in    BOOLEAN                 := FALSE;
        STATUS                  : in    STATUS_INDICATOR        := NO_INDICATION;
        PRIORITY                : in    PROCESS_PRIORITY        := DEFAULT_PRIORITY;
        PROCESSOR_RESOURCE      : in    PROCESSOR_NODE_TYPE     := DEFAULT_PROCESSOR;
        MEDIUM_RESOURCE         : in    MEDIUM_NODE_TYPE        := DEFAULT_MEDIUM);

    procedure CREATE_JOB_NODE
        (NODE                   : in out PROCESS_NODE_TYPE;
        FILE_NODE               : in    EXECUTABLE_IMAGE_NODE_TYPE;
        INTENT                  : in    INTENT_ARRAY            := (READ, WRITE);
        INPUT_PARAMETERS        : in    PARAMETER_LIST          := EMPTY_LIST;
        KEY                     : in    RELATIONSHIP_KEY        := EMPTY_KEY_LIST;
        DISCRETIONARY_ACCESS    : in    DISCRETIONARY_ACCESS_LIST := ALL_RIGHTS;
        MANDATORY_ACCESS        : in    MANDATORY_ACCESS_LIST   := EMPTY_LIST;
        ATTRIBUTES              : in    ATTRIBUTE_LIST          := EMPTY_LIST;
        INPUT_FILE              : in    PATHNAME                := "";
        OUTPUT_FILE             : in    PATHNAME                := "";
        ERROR_FILE              : in    PATHNAME                := "";
        ENVIRONMENT_NODE        : in    PATHNAME                := CURRENT_NODE;
        DELETE_WHEN_TERMINATED  : in    BOOLEAN                 := TRUE ;
        PROCESS_NODE_KIND       : in    NODE_KIND_NAME;
        AS_A_TRANSACTION        : in    BOOLEAN                 := FALSE ;
        STATUS                  : in    STATUS_INDICATOR        := NO_INDICATION;
        PRIORITY                : in    PROCESS_PRIORITY        := DEFAULT_PRIORITY ;
        PROCESSOR_RESOURCE      : in    PROCESSOR_NODE_TYPE     := DEFAULT_PROCESSOR;
        MEDIUM_RESOURCE         : in    MEDIUM_NODE_TYPE        := DEFAULT_MEDIUM);
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
procedure ACTIVATE_PROCESS
    (NODE                : in     PROCESS_NODE_TYPE;
     STATUS              : in     STATUS_ INDICATOR := NO_INDICATION);

procedure AWAIT_PROCESS_TERMINATION
    (NODE          : in     PROCESS_NODE_TYPE;
     TIME_LIMIT : in     CAIS_DURATION     := LONG_DELAY;
     STATUS        : in     STATUS_INDICATOR := NO_INDICATION);

procedure AWAIT_PROCESS_TERMINATION
    (NODE                : in     PROCESS_NODE_TYPE;
     RESULTS_RETURNED : in out RESULTS_LIST;
     STATUS              :    out PROCESS_STATUS_KIND;
     TIME_LIMIT          : in     CAIS_DURATION     := LONG_DELAY;
     AWAIT_STATUS        : in     STATUS_INDICATOR := NO_INDICATION);

procedure AWAIT_PROCESS_COMPLETION
    (NODE          : in     PROCESS_NODE_TYPE;
     TIME_LIMIT : in     CAIS_DURATION     := LONG_DELAY;
     STATUS        : in     STATUS_INDICATOR := NO_INDICATION)
    renames AWAIT_PROCESS_TERMINATION;

procedure AWAIT_PROCESS_COMPLETION
    (NODE                : in     PROCESS_NODE_TYPE;
     RESULTS_RETURNED : in out RESULTS_LIST;
     STATUS              :    out PROCESS_STATUS_KIND;
     TIME_LIMIT          : in     CAIS_DURATION := LONG_DELAY;
     AWAIT_STATUS        : in     STATUS_INDICATOR := N0_INDICATION)
    renames AWAIT_PROCESS_TERMINATION;

procedure ACTIVATE_AND_AWAIT_TERMINATION
    (NODE                : in     PROCESS_NODE_TYPE;
     TIME_LIM1T          : in     CAIS_DURATION := LONG_DELAY;
     STATUS              : in     STATUS_INDICATOR := NO_INDICATION);

procedure ACTIVATE_AND_AWAIT_TERMINATION
    (NODE                : in     PROCESS_NODE_TYPE;
     RESULTS_RETURNED : in out RESULTS_LIST;
     STATUS              :    out PROCESS_STATUS_KIND;
     TIME_LIMIT          : in     CAIS_DURATION     := LONG_DELAY;
     AWAIT_STATUS        : in     STATUS_INDICATOR := NO_INDICATION);

procedure SPAWN_PROCESS
    (NODE                    : in out PROCESS_NODE_TYPE;
     FILE_NODE               : in     EXECUTABLE_IMAGE_NODE_TYPE;
     INTENT                  : in     INTENT_ARRAY          := (1 => READ);
     INPUT_PARAMETERS        : in     PARAMETER_LIST        := EMPTY_LIST;
     KEY                     : in     RELATIONSHIP_KEY      := EMPTY_KEY_LIST;
     RELATION                : in     RD_TYPE;
     DISCRETIONARY_ACCESS : in     DISCRETIONARY_ACCESS_LIST :=ALL_RIGHTS;
     ATTRIBUTES              : in     ATTRIBUTE_LIST        := EMPTY_LIST;
     INPUT_FILE              : in     PATHNAME              := STANDARD_INPUT;
     OUTPUT_FILE             : in     PATHNAME              := STANDARD_OUTPUT;
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
        ERROR_FILE                : in    PATHNAME                := STANDARD_ERROR ;
        ENVIRONMENT_NODE          : in    PATHNAME                := CURRENT_NODE;
        PROCESS_NODE_KIND         : in    NODE_KIND_NODE_TYPE     := NULL_NODE;
        ENCLOSING-TRANSACTION     : in    TRANSACTION_TYPE        := NULL_TRANSACTION;
        STATUS                    : in    STATUS_INDICATOR        := NO_INDICATION;
        PRIORITY                  : in    PROCESS_PRIORITY        := DEFAULT_PRIORITY;
        PROCESSOR-RESOURCE        : in    PROCESSOR_NODE_TYPE     := DEFAULT_PROCESSOR;
        MEDIUM_RESOURCE           : in    MEDIUM_NODE_TYPE        := DEFAULT_MEDIUM);

    procedure SPAWN_PROCESS
        (N0DE                     : in out PROCESS_NODE_TYPE;
        FILE_NODE                 : in    EXECUTABLE_IMAGE_NODE_TYPE ;
        INTENT                    : in    INTENT_ARRAY            := (1 => READ);
        INPUT_PARAMETERS          : in    PARAMETER_LIST          := EMPTY_LIST;
        KEY                       : in    RELATIONSHIP_KEY        := EMPTY_KEY_LIST;
        RELATION                  : in    RELATION_NAME           := DEFAULT_RELATION;
        DISCRETIONARY_ACCESS      : in    DISCRETIONARY_ACCESS_LIST :=ALL_RIGHTS;
        ATTRIBUTES                : in    ATTRIBUTE_LIST          := EMPTY_LIST;
        INPUT_FILE                : in    PATHNAME                := STANDARD_INPUT;
        OUTPUT_FILE               : in    PATHNAME                := STANDARD_OUTPUT;
        ERROR-FILE                : in    PATHNAME                := STANDARD_ERROR;
        ENVIRONMENT-NODE          : in    PATHNAME                := CURRE19T_MODE;
        PROCESS-NODE-KIND         : in    NODE_KIND_NAME          := "";
        ENCLOSING_TRANSACTION     : in    TRANSACTION_TYPE        := NULL_TRANSACTION;
        STATUS                    : in    STATUS_INDICATOR        := NO_INDICATION;
        PRIORITY                  : in    PROCESS_PRIORITY        := DEFAULT_PRIORITY;
        PROCESSOR-RESOURCE        : in    PROCESSOR_MODE_TYPE     := DEFAULT_PROCESSOR;
        MEDIUM_RESOURCE           : in    MEDIUM_NODE_TYPE        := DEFAULT_MEDIUM);

    procedure SPAWN_PROCESS
        (NODE                     : in out PROCESS_NODE_TYPE;
        FILE_NODE                 : in    EXECUTABLE_IMAGE_NODE_TYPE;
        INTENT                    : in    INTENT_SPECIFICATION    := READ_ATTRIBUTES;
        INPUT_PARAMETERS          : in    PARAMETER_LIST          := EMPTY_LIST;
        KEY                       : in    RELATIONSHIP_KEY        := EMPTY_KEY_LIST;
        RELATION                  : in    RELATION_NAME           := DEFAULT_RELATION;
        DISCRETIONARY_ACCESS      : in    DISCRETIONARY_ACCESS_LIST :=ALL_RIGHTS;
        ATTRIBUTES                : in    ATTRIBUTE_LIST          := EMPTY_LIST;
        INPUT_FILE                : in    PATHNAME                := STANDARD_INPUT;
        OUTPUT_FILE               : in    PATHNAME                := STANDARD_OUTPUT;
        ERROR_FILE                : in    PATHNAME                := STANDARD_ERROR;
        ENVIRONMENT_NODE          : in    PATHNAME                := CURRENT_NODE;
        PROCESS_NODE_KIND         : in    MODE_KIND_NAME          := "";
        ENCLOSING_TRANSACTION     : in    TRANSACTION_TYPE        := NULL_TRANSACTION;
        STATUS                    : in    STATUS_INDICATOR        := NO_INDICATION;
        PRIORITY                  : in    PROCESS_PRIORITY        := DEFAULT_PRIORITY;
        PROCESSOR_RESOURCE        : in    PROCESSOR_NODE_TYPE     := DEFAULT_PROCESSOR;
        MEDIUM_RESOURCE           : in    MEDIUM_NODE_TYPE        := DEFAULT_MEDIUM);

    procedure INVOKE_PROCESS
        (NODE                     : in out PROCESS_NODE_TYPE;
        FILE_NODE                 : in    EXECUTABLE_IMAGE_NODE_TYPE;
        INTENT                    : in    IHTENT_ARRAY            := (1 => READ);
        RESULTS_RETURNED          : in out RESULTS_LIST;
```



Source: https://assist.dla.mil -- Downloaded: 2026l0-26T07:54Z
Check the source to verify that this is the currentversion before use.

```
        STATUS                  :       out  PROCESS_STATUS_KIND;
        INPUT_PARAMETERS        :  in        PARAMETER_LIST          := EMPTY_LIST;
        KEY                     :  in        RELATIONSHIP_KEY        :=  EMPTY_KEY_LIST;
        RELATION                :  in        RD_TYPE;
        DISCRETIONARY_  ACCESS  :  in        DISCRETIONARY_ACCESS_LIST  :=ALL_RIGHTS;
        ATTRIBUTES              :  in        ATTRIBUTE_LIST          := EMPTY_LIST;
        INPUT_FILE              :  in        PATHNAME                := STANDARD_INPUT;
        OUTPUT_FILE             :  in        PATHNAME                :=  STANDARD_OUTPUT;
        ERROR_FILE              :  in        PATHNAME                :=  STANDARD_ERROR;
        ENVIRONMENT_NODE        :  in        PATHNAME                :=  CURRENT_NODE;
        TIME_LIMIT              :  in        CAIS_DURATION           :=  LONG_DELAY;
        PROCESS_NODE_KIND       :  in        NODE_KIND_NODE_TYPE     :=  NULL_NODE;
        ENCLOSING_TRANSACTION   :  in        TRANSACTION_TYPE        :=  NULL_TRANSACTION;
        INVOKE_PROCESS_STATUS   :  in        STATUS_INDICATOR        :=  NO_INDICATION;
        PRIORITY                :  in        PROCESS_PRIORITY        :=  DEFAULT_PRIORITY;
        PROCESSOR_RESOURCE      :  in        PROCESSOR_NODE_TYPE     :=  DEFAULT_PROCESSOR;
        MEDIUM_RESOURCE         :  in        MEDIUM_NODE_TYPE        :=  DEFAULT_MEDIUM);

    procedure   INVOKE_PROCESS
        (NODE                   : in out PROCESS_NODE_TYPE;
        FILE_NODE               :  in        EXECUTABLE_IMAGE_NODE_TYPE;
        INTENT                  :  in        INTENT_ARRAY            := (1 => READ);
        RESULTS_RETURNED        : in out RESULTS_LIST;
        STATUS                  :       out  PROCESS_STATUS_KIND;
        INPUT_PARAMETERS        :  in        PARAMETER_LIST          := EMPTY_LIST;
        KEY                     :  in        RELATIONSHIP_KEY        :=  EMPTY_KEY_LIST;
        RELATION                :  in        RELATION_NAME           := DEFAULT_RELATION;
        DISCRETIONARY_ACCESS    :  in        DISCRETIONARY_ACCESS_LIST  :=ALL_RIGHTS;
        ATTRIBUTES              :  in        ATTRIBUTE_LIST          := EMPTY_LIST;
        INPUT_FILE              :  in        PATHNAME                := STANDARD_INPUT;
        OUTPUT_FILE             :  in        PATHNAME                :=  STANDARD_OUTPUT;
        ERROR_FILE              :  in        PATHNAHE                :=  STANDARD_ERROR;
        ENVIRONMENT_NODE        :  in        PATHNAME                :=  CURRENT_NODE;
        TIME_LIMIT              :  in        CAIS_DURATION           :=  LONG_DELAY;
        PROCESS_NODE_KIND       :  in        NODE_KIND_NAME          :=  "";
        ENCLOSING_TRANSACTION   :  in        TRANSACTION_TYPE        :=  NULL_TRANSACTION;
        INVOKE_PROCESS_STATUS   :  in        STATUS_INDICATOR        := NO_INDICATION;
        PRIORITY                :  in        PROCESS_PRIORITY        :=  DEFAULT_PRIORITY;
        PROCESSOR_RESOURCE      :  in        PROCESSOR_NODE_TYPE     :=  DEFAULT_PROCESSOR;
        MEDIUM_RESOURCE         :  in        MEDIUM_NODE_TYPE        :=  DEFAULT_MEDIUM);

    procedure   INVOKE_PROCESS
        (NODE                   : in out PROCESS_NODE_TYPE;
        FILE_NODE               :  in        EXECUTABLE_IMAGE_NODE_TYPE;
        INTENT                  :  in        INTENT_SPECIFICATION;
        RESULTS_RETURNED        : in out RESULTS_LIST;
        STATUS                  :       out  PROCESS_STATUS_KIND;
        INPUT_PARAMETERS        :  in        PARAMETER_LIST          := EMPTY_LIST;
        KEY                     :  in        RELATIONSHIP_KEY        :=  EMPTY_KEY_LIST;
        RELATION                :  in        RELATION_NAME           := DEFAULT_RELATION;
        DISCRETIONARY_ACCESS    :  in        DISCRETIONARY_ACCESS_LIST  :=ALL_RIGHTS;
        ATTRIBUTES              :  in        ATTRIBUTE_LIST          := EMPTY_LIST;
        INPUT_FILE              :  in        PATHNAME                := STANDARD_INPUT;
        OUTPUT_FILE             :  in        PATHNAHE                := STANDARD_OUTPUT;
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
        ERROR_FILE                : in  PATHNAME                := STANDARD_ERROR;
        ENVIRONMENT_NODE          : in  PATHNAME                := CURRENT_NODE;
        TIME_LIMIT                : in  CAIS_DURATION           := LONG_DELAY;
        PROCESS_NODE_KIND         : in  NODE_KIND_NAME          := "";
        ENCLOSING_TRANSACTION     : in  TRANSACTION_TYPE        := NULL_TRANSACTION;
        INVOKE_PROCESS_STATUS     : in  STATUS_INDICATOR        := NO_INDICATION;
        PRIORITY                  : in  PROCESS_PRIORITY        := DEFAULT_PRIORITY;
        PROCESSOR_RESOURCE        : in  PROCESSOR_NODE_TYPE     := DEFAULT_PROCESSOR;
        MEDIUM_RESOURCE           : in  MEDIUM_NODE_TYPE        := DEFAULT_MEDIUM);

    procedure CREATE_JOB
        (FILE_NODE                : in  EXECUTABLE_IMAGE_NODE_TYPE;
         INPUT_PARAMETERS         : in  PARAMETER_LIST          := EMPTY_LIST;
         KEY                      : in  RELATIONSHIP_KEY        := EMPTY_KEY_LIST;
         DISCRETIONARY_ACCESS     : in  DISCRETIONARY_ACCESS_LIST :=ALL_RIGHTS;
         MANDATORY_ACCESS         : in  MANDATORY_ACCESS_LIST   := EMPTY_LIST;
         ATTRIBUTES               : in  ATTRIBUTE_LIST          := EMPTY_LIST;
         INPUT_FILE               : in  PATHNAME                := STANDARD_INPUT;
         OUTPUT_FILE              : in  PATHNAME                := STANDARD_OUTPUT;
         ERROR_FILE               : in  PATHNAME                := STANDARD_ERROR;
         ENVIRONMENT_NODE         : in  PATHNAME                := CURRENT_USER;
         DELETE_WHEN_TERMINATED   : in  BOOLEAN                 := TRUE;
         PROCESS_NODE_KIND        : in  NODE_KIND_NODE_TYPE     := NULL_NODE;
         AS_A_TRANSACTION         : in  BOOLEAN                 := FALSE;
         STATUS                   : in  STATUS_INDICATOR        := NO_INDICATION;
         PRIORITY                 : in  PROCESS_PRIORITY        := DEFAULT_PRIORITY;
         PROCESSOR_RESOURCE       : in  PROCESSOR_NODE_TYPE     := DEFAULT_PROCESSOR;
         MEDIUM_RESOURCE          : in  MEDIUM_NODE_TYPE        := DEFAULT_MEDIUM);

    procedure CREATE_JOB
        (FILE_NODE                : in  EXECUTABLE_IMAGE_NODE_TYPE;
         INPUT_PARAMETERS         : in  PARAMETER_LIST          := EMPTY_LIST;
         KEY                      : in  RELATIONSHIP_KEY        := EMPTY_KEY_LIST;
         DISCRETIONARY_ACCESS     : in  DISCRETIONARY_ACCESS_LIST := ALL_RIGHTS;
         MANDATORY_ACCESS         : in  MANDATORY_ACCESS_LIST   := EMPTY_LIST;
         ATTRIBUTES               : in  ATTRIBUTE_LIST          := EMPTY_LIST;
         INPUT_FILE               : in  PATHNAME                := STANDARD_INPUT;
         OUTPUT_FILE              : in  PATHNAME                := STANDARD_OUTPUT;
         ERROR_FILE               : in  PATHNAME                := STANDARD_ERROR;
         ENVIRONMENT_NODE         : in  PATHNAME                := CURRENT_NODE;
         DELETE_WHEN_TERMINATED   : in  BOOLEAN                 := TRUE;
         PROCESS_NODE_KIND        : in  NODE_KIND_NAME;
         AS_A_TRANSACTION         : in  BOOLEAN                 := FALSE;
         STATUS                   : in  STATUS_INDICATOR        := NO_INDICATION;
         PRIORITY                 : in  PROCESS_PRIORITY        := DEFAULT_PRIORITY;
         PROCESSOR_RESOURCE       : in  PROCESSOR_NODE_TYPE     := DEFAULT_PROCESSOR;
         MEDIUM_RESOURCE          : in  MEDIUM_NODE_TYPE        := DEFAULT_MEDIUM);

    procedure DELETE_JOB
        (NODE     : in out PROCESS_NODE_TYPE;
         STATUS : in       STATUS_INDICATOR := NO_INDICATION);

    procedure DELETE_JOB
        (NAME     : in      PATHNAME;
         STATUS : in       STATUS_INDICATOR  := NO_INDICATION);
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
procedure APPEND_RESULTS
    (RESULTS : in     RESULTS_STRING;
     STATUS  : in     STATUS_INDICATON := NO_INDICATION);

procedure WRITE_RESULTS
    (RESULTS : in     RESULTS_STRING;
     STATUS  : in     STATUS_INDICATOR := NO_INDICATION);

procedure GET_RESULTS
    (NODE    : in     PROCESS_NODE_TYPE;
     RESULTS : in out RESULTS_LIST;
     STATUS  : in     STATUS_INDICATOR := NO_INDICATION);

procedure GET_RESULTS
    (NODE                : in     PROCESS_NODE_TYPE;
     RESULTS             : in out RESULTS_LIST;
     STATUS              :    out PROCESS_STATUS_KIND;
     GET_RESULTS_STATUS : in     STATUS_INDICATOR := NO_INDICATION);

procedure GET_RESULTS
    (NAME                : in     PATHNAME;
     RESULTS             : in out RESULTS_LIST;
     STATUS              :    out PROCESS_STATUS_KIND;
     GET_RESULTS_STATUS : in     STATUS_INDICATOR := NO_INDICATION);

procedure GET_RESULTS
    (NAME                : in     PATHNAME;
     RESULTS             : in out RESULTS_LIST;
     GET_RESULTS_STATUS : in     STATUS_INDICATOR := NO_INDICATION);

function CURRENT_STATUS
    (NODE   : in     PROCESS_NODE_TYPE;
     STATUS : in     STATUS_INDICATOR := N0_INDICATION)
    return PROCESS_STATUS_KIND;

function CURRENT_STATUS
    (NAME   : in     PATHNAME;
     STATUS : in     STATUS_INDICATOR := NO_INDICATION)
    return PROCESS_STATUS_KIND;

procedure GET_PARAMETERS
    (PARAMETERS : in out PARAMETER_LIST;
     STATUS     : in     STATUS_INDICATOR := NO_INDICATION);

procedure ABORT_PROCESS
    (NODE    : in     PROCESS_NODE_TYPE;
     RESULTS : in     RESULTS_STRING;
     STATUS  : in     STATUS_INDICATOR := NO_INDICATION);

procedure ABORT_PROCESS
    (NAME    : in     PATHNAME;
     RESULTS : in     RESULTS_STRING;
     STATUS  : in     STATUS_INDICATOR := NO_INDICATION);
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
procedure ABORT_PROCESS
    (NODE   : in  PROCESS_NODE_TYPE;
     STATUS : in  STATUS_ INDICATOR := NO_INDICATION);

procedure ABORT_PROCESS
    (NAME   : in  PATHNAME;
     STATUS : in  STATUS_INDICATOR := NO_INDICATION);

procedure SUSPEND_PROCESS
    (NODE   : in  PROCESS_NODE_TYPE;
     STATUS : in  STATUS_INDICATOR := NO_INDICATION);

procedure SUSPEND_PROCESS
    (NAME   : in  PATHNAME;
     STATUS : in  STATUS_INDICATOR := NO_INDICATION);

procedure RESUME_PROCESS
    (NODE   : in  PROCESS_NODE_TYPE;
     STATUS : in  STATUS_INDICATOR := NO_INDICATION);

procedure RESUME_PROCESS
    (NAME   : in  PATHNAME;
     STATUS : in  STATUS_INDICATOR := NO_INDICATION);

function LOWEST_USER_PRIORITY
    (PROCESSOR : in  PROCESSOR_NODE_TYPE := DEFAULT_PROCESSOR ;
     STATUS    : in  STATUS_INDICATOR    := NO_INDICATION)
    return PROCESS_PRIORITY;

function HIGHEST_USER_PRIORITY
    (PROCESSOR : in  PROCESSOR_NODE_TYPE := DEFAULT_PROCESSOR ;
     STATUS    : in  STATUS_INDICATOR    := NO_INDICATION)
    return PROCESS_PRIORITY;

function PRIORITIES_ARE_MODIFIABLE
    (STATUS : in  STATUS_INDICATOR := NO_INDICATION)
    return BOOLEAN;

procedure SET_PRIORITY
    (PRIORITY : in  PROCESS_PRIORITY;
     STATUS   : in  STATUS_INDICATOR := NO_INDICATION);

function OPEN_HANDLE_COUNT
    (NODE   : in  PROCESS_NODE_TYPE;
     STATUS : in  STATUS_INDICATOR := NO_INDICATION)
    return CAIS_NATURAL;

function OPEN_HANDLE_COUNT
    (NAME   : in  PATHNAME;
     STATUS : in  STATUS_INDICATOR := NO_INDICATION)
    return CAIS_NATURAL;
```


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
        function IO_UNIT_COUNT
            (NODE   : in    PROCESS_NODE_TYPE;
             STATUS : in    STATUS_ INDICATOR  := NO_INDICATION)
            return CAIS_NATURAL;

        function IO_UNIT_COUNT
            (NAME   : in    PATHNAME;
             STATUS : in    STATUS_INDICATOR   := NO_INDICATION)
            return CAIS_NATURAL;

        function TIME_STARTED
            (NODE   : in    PROCESS_NODE_TYPE;
             STATUS : in    STATUS_INDICATOR   := NO_INDICATION)
            return  CAIS_CALENDAR.TIME;

        function TIME_STARTED
            (NAME   : in    PATHNAME;
             STATUS : in    STATUS_INDICATOR   := NO_INDICATION)
            return  CAIS_CALENDAR.TIME;

        function TIME_FINISHED
            (NODE   : in    PROCESS_NODE_TYPE;
             STATUS : in    STATUS_INDICATOR   := NO_INDICATIOR)
            return  CAIS_CALENDAR.TIME;

        function TIME_FINISHED
            (NAME   : in    PATHNAME;
             STATUS : in    STATUS_INDICATOR   := NO_INDICATION)
            return  CAIS_CALENDAR.TIME;

        function  MACHIHE_TIME
            (NODE   : in    PROCESS_NODE_TYPE;
             STATUS : in    STATUS_INDICATOR   := NO_INDICATION)
            return CAIS_DURATION;

        function  MACHINE_TIME
            (NAME   : in    PATHNAME;
             STATUS : in    STATUS_INDICATOR   := NO_INDICATION)
            return CAIS_DURATION;

        function PROCESS_SIZE
            (NODE   : in    PROCESS_NODE_TYPE;
             STATUS : in    STATUS_INDICATOR   := NO_INDICATION)
            return CAIS_NATURAL;

        function PROCESS_SIZE
            (NAME   : in    PATHNAME;
             STATUS : in    STATUS_INDICATOR   := NO_INDICATION)
            return CAIS_NATURAL;

end CAIS_PROCESS_MANAGEMENT;
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
with CAIS_STANDARD;
with CAIS_DEFINITIONS;
with CAIS_LIST_MANAGEMENT;
with CAIS_STATUS_DEFINITIONS;
with CAIS_RESOURCE_CONTROL;
with CAIS_ACCESS_CONTROL_MANAGEMENT;
with CAIS_DEFINITION_MANAGEMENT;
with CAIS_PROCESS_DEFINITIONS;
with CAIS_PROCESS_MANAGEMENT;

package CAIS_ATTRIBUTE_MONITOR_MANAGEMENT is

    use CAIS_STANDARD;
    use CAIS_DEFINITIONS;
    use CAIS_LIST_MANAGEMENT;
    use CAIS_STATUS_DEFINITIONS;
    use CAIS_RESOURCE_CONTROL;
    use CAIS_DEFINITION_MANAGEMENT;
    use CAIS_PROCESS_DEFINITIONS;
    use CAIS_PROCESS_MANAGEMENT;
    use CAIS_ACCESS_CONTROL_MANAGEMENT;

    typo MONITOR_EVENT_KIND is
        (NO_EVENT, NODE_ATTRIBUTE_EVENT, PATH_ATTRIBUTE_EVENT);

    function MONITOR_NKD
        (STATUS : in      STATUS_INDICATOR  := NO_INDICATION)
        return NKD_TYPE;

    procedure CREATE_MONITOR_NODE
        (NODE                              : in out MONITOR_MODE_TYPE;
         HANDLER_FILE_NODE                 : in     EXECUTABLE_IMAGE_NODE_TYPE ;
         HANDLER_PROCESS_NODE_KIND         : in     NODE_KIND_N0DE_TYPE    := NULL_NODE;
         HANDLER_INPUT_PARAMETERS          : in     PARAMETER_LIST         := EMPTY_LIST;
         HANDLER_KEY                       : in     RELATIONSHIP_KEY       := EMPTY_KEY_LIST;
         HANDLER_DISCRETIONARY_ACCESS      : in     DISCRETIONARY_ACCESS_LIST := ALL_RIGHTS;
         HANDLER_MANDATORY_ACCESS          : in     MANDATORY_ACCESS_LIST  := EMPTY_LIST;
         HANDLER_ATTRIBUTES                : in     ATTRIBUTE_LIST         := EMPTY_LIST;
         HANDLER_INPUT_FILE                : in     PATHNAME               := STANDARD_INPUT;
         HANDLER_OUTPUT_FILE               : in     PATHNAME               := STANDARD_OUTPUT;
         HANDLER_ERROR_FILE                : in     PATHMAME               := STANDARD_ERROR;
         HANDLER_ENVIRONMENT_NODE          : in     PATHNAME               := CURRENT_USER;
         HANDLER_DELETE_WHEN_TERMINATED    : in     BOOLEAN                := TRUE;
         HANDLER_AS_A_TRANSACTION          : in     BOOLEAN                := FALSE;
         HANDLER_PROCESSOR_RESOURCE        : in     PROCESSOR_NODE_TYPE    := DEFAULT_PROCESSOR;
         HANDLER_PRIORITY                  : in     PROCESS_PRIORITY       := DEFAULT_PRIORITY;
         HANDLER_MEDIUM_RESOURCE           : in     MEDIUM_NODE_TYPE       := DEFAULT_MEDIUM;
         KEY                               : in     RELATIONSHIP_KEY       := EMPTY_KEY_LIST;
         ATTRIBUTES                        : in     ATTRIBUTE_LIST         := EMPTY_LIST;
         DISCRETIONARY_ACCESS              : in     DISCRETIONARY_ACCESS_LIST := ALL_RIGHTS;
         MANDATORY_ACCESS                  : in     MANDATORY_ACCESS_LIST  := EMPTY_LIST;
         KIND                              : in     NODE_KIND_NODE_TYPE    := MONITOR_NKD;
         STATUS                            : in     STATUS_INDICATOR       := NO_INDICATION;
         MEDIUM_RESOURCE                   : in     MEDIUM_NODE_TYPE       := DEFAULT_MEDIUM);
```


Source: https://assist.dla.mil -- Downloaded: 2026l0-26T07:54Z
Check the source to verify that this is the currentversion before use.

```
procedure CREATE_MONITOR_NODE
    (NODE                              : in out MONITOR_NODE_TYPE;
     HANDLER_FILE_NODE                 : in     EXECUTABLE_IMAGE_NODE_TYPE;
     HANDLER_PROCESS_NODE_KIND         : in     NODE_KIND_NAME;
     HANDLER_INPUT_PARAMETERS          : in     PARAMETER_LIST          := EMPTY_LIST;
     HANDLER_KEY                       : in     RELATIONSHIP_KEY        := EMPTY_KEY_LIST;
     HANDLER_DISCRETIONARY_ACCESS      : in     DISCRETIONARY_ACCESS_LIST := ALL_RIGHTS;
     HANDLER_MANDATORY_ACCESS          : in     MANDATORY_ACCESS_LIST := EMPTY_LIST;
     HANDLER_ATTRIBUTES                : in     ATTRIBUTE_LIST          := EMPTY_LIST;
     HANDLER_INPUT_FILE                : in     PATHNAME                := STANDARD_INPUT;
     HANDLER_OUTPUT_FILE               : in     PATHNAME                := STANDARD_OUTPUT;
     HANDLER_ERROR_FILE                : in     PATHNAME                := STANDARD_ERROR;
     HANDLER_ENVIRONMENT_NODE          : in     PATHNAME                := CURRENT_USER;
     HANDLER_DELETE_WHENl_TERMINATED   : in     BOOLEAN                 := TRUE ;
     HANDLER_AS_A_TRANSACTION          : in     BOOLEAN                 := FALSE;
     HANDLER_PROCESSOR_RESOURCE        : in     PROCESSOR_NODE_TYPE := DEFAULT_PROCESSOR;
     HANDLER_PRIORITY                  : in     PROCESS_PRIORITY    := DEFAULT_PRIORITY;
     HANDLER_MEDIUM_RESOURCE           : in     MEDIUM_NODE_TYPE    := DEFAULT_MEDIUM;
     KEY                               : in     RELATIONSHIP_KEY    := EMPTY_KEY_LIST;
     ATTRIBUTES                        : in     ATTRIBUTE_LIST          := EMPTY_LIST;
     DISCRETIONARY_ACCESS              : in     DISCRETIONARY_ACCESS_LIST := ALL_RIGHTS;
     MANDATORY_ACCESS                  : in     MANDATORY_ACCESS_LIST := EMPTY_LIST;
     KIND                              : in     NODE_KIND_NAME          := "MONITOR";
     STATUS                            : in     STATUS_INDICATOR    := NO_INDICATION;
     MEDIUM_RESOURCE                   : in     MEDIUM_NODE_TYPE := DEFAULT_MEDIUM);

procedure DELETE_MONITOR_NODE
    (NODE       : in     MONITOR_NODE_TYPE;
     TIME_LIMIT : in     CAIS_DURATION := LONG_DELAY;
     STATUS     : in     STATUS_INDICATOR := NO_INDICATION);

procedure ASSOCIATE_EVENT_HANDLER_PROCESS
    (MONITOR : in     MONITOR_NODE_TYPE;
     NODE    : in     PROCESS_NODE_TYPE  ;
     KEY     : in     RELATIONSHIP_KEY   := EMPTY_KEY_LIST;
     STATUS  : in     STATUS_INDICATOR   := NO_INDICATION);

procedure MONITOR_NODE_ATTRIBUTE
    (MONITOR   : in     MONITOR_NODE_TYPE;
     NODE      : in     NODE_TYPE;
     ATTRIBUTE : in     AD_TYPE ;
     NAME      : in     IDENTIFIER_TEXT ;
     STATUS    : in     STATUS_INDICATOR   := NO_INDICATION);

procedure MONITOR_NODE_ATTRIBUTE
    (MOINITOR  : in     MONITOR_NODE_TYPE;
     NODE      : in     NODE_TYPE;
     ATTRIBUTE : in     ATTRIBUTE_NAME;
     NAME      : in     IDENTIFIER_TEXT := "";
     STATUS    : in     STATUS_INDICATOR   := NO_INDICATION);

procedure MONITOR_PATH_ATTRIBUTE
    (MONITOR   : in     MONITOR_NODE_TYPE;
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
        NODE      : in    NODE_TYPE;
        KEY       : in    RELATIONSHIP_KEY := EMPTY_KEY_LIST;
        RELATION  : in    RD_TYPE;
        ATTRIBUTE : in    AD_TYPE;
        NAME      : in    IDENTIFIER_TEXT;
        STATUS    : in    STATUS_INDICATOR := NO_INDICATION);

    procedure MONITOR_PATH_ATTRIBUTE
        (MONITOR  : in    MONITOR_NODE_TYPE;
        NODE      : in    NODE_TYPE;
        KEY       : in    RELATIONSHIP_KEY := EMPTY_KEY_LIST;
        RELATION  : in    RELATION_NAME;
        ATTRIBUTE : in    ATTRIBUTE_NAME;
        NAME      : in    IDENTIFIER_TEXT := "".
        STATUS    : in    STATUS_INDICATOR := NO_INDICATION);

    procedure CANCEL_ATTRIBUTE_MONITORING
        (MONITOR  : in    MONITOR_NODE_TYPE;
        NODE      : in    NODE_TYPE;
        NAME      : in    IDENTIFIER_TEXT;
        STATUS    : in    STATUS_INDICATOR := NO_INDICATION);

    procedure GET_MONITOR_EVENT
        (NODE           : in out NODE_TYPE;
        ATTRIBUTE       : in out AD_TYPE;
        RELATION        : in out RD_TYPE;
        KEY             :    out RELATIONSHIP_KEY;
        LAST            :    out CAIS_NATURAL;
        EVENT_KIND      :    out MONITOR_EVENT_KIND;
        TIME_LIMIT      : in     CAIS_DURATION      := LONG_DELAY;
        STATUS          : in     STATUS_INDICATOR   := NO_INDICATION);

    procedure AWAIT_M0NITOR_EVENT
        (NODE                   : in out NODE_TYPE;
        ATTRIBUTE               : in out AD_TYPE;
        RELATION                : in out RD_TYPE
        KEY                     :    out RELATIONSHIP_KEY;
        LAST                    :    out CAIS_NATURAL;
        EVENT_KIND              :    out MONITOR_EVENT_KIND ;
        ARRIVAL_TIME_LIMIT : in      CAIS_DURATION := LONG_DELAY;
        OPEN_TIME_LIMIT    : in      CAIS_DURATION := LONG_DELAY;
        STATUS             : in      STATUS_INDICATOR := NO_INDICATION);

end CAIS_ATTRIBUTE_MONITOR_MANAGEMENT;
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
with CAIS_STANDARD;
with CAIS_DEFINITIONS;
with CAIS_PRAGMATICS;

package CAIS_IO_DEFINITIONS is

    use CAIS_STANDARD;
    use CAIS_DEFINITIONS;

    type CHANNEL_STATE is (CONNECTED, ESTABLISHED, CLOSED, RETIRED);

    type CAIS_IO_DURATION is new CAIS_DURATION;

    INFINITE_IO_DELAY : constant CAIS_IO_DURATION := CAIS_IO_DURATION'LAST;
    LONG_IO_DELAY     : constant CAIS_IO_DURATION := CAIS_IO_DURATION'LAST -
                                  CAIS_PRAGMATICS.SMALL_FOR_CAIS_DURATION;

    IN_INTENT      : constant INTENT_ARRAY := (1 => READ_CONTENTS);
    INOUT_INTENT   : constant INTENT_ARRAY := (READ_CONTENTS, WRITE_CONTENTS);
    OUT_INTENT     : constant INTENT_ARRAY := (1 => WRITE_CONTENTS);
    APPEND_INTENT  : constant INTENT_ARRAY := (1 => APPEND_CONTENTS);

    UNBOUNDED_FILE_SIZE : constant CAIS_NATURAL := O;

    CLIENT_ERROR   : exception;
    DATA_ERROR     : exception;
    END_ERROR      : exception;
    LAYOUT_ERROR   : exception;
    NODE_ERROR     : exception;
    OPEN_DENIED    : exception;
    PORT_ERROR     : exception;
    PROTOCOL_ERROR : exception;
    TIME_EXPIRED   : exception;

end CAIS_IO_DEFINITIONS;
```


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
with CAIS_STATUS_DEFINITIONS;
with CAIS_DEFINITIONS;
with CAIS_LIST_MANAGEMENT;
with CAIS_RESOURCE_CONTROL;
with CAIS_DEFINITION_MANAGEMENT;
with CAIS_ACCESS_CONTROL_MANAGEMENT;

package CAIS_FILE_NODE_MANAGEMENT is

    use CAIS_STATUS_DEFINITIONS;
    use CAIS_DEFINITIONS;
    use CAIS_LIST_MANAGEMENT;
    use CAIS_RESOURCE_CONTROL;
    use CAIS_DEFINITION_MANAGEMENT;
    use CAIS_ACCESS_CONTROL_MANAGEMENT;

    function SECONDARY_STORAGE_NKD
        (STATUS : in      STATUS_INDICATOR   := NO_INDICATION)
        return NKD_TYPE;

    procedure CREATE_SECONDARY_STORAGE_NODE
        (NODE                 : in out SECONDARY_STORAGE_NODE_TYPE;
         BASE                 :  in    NODE_TYPE;
         KEY                  :  in    RELATIONSHIP_KEY        := EMPTY_KEY_LIST;
         RELATION             :  in    RD_TYPE;
         INTENT               :  in    INTENT_ARRAY            := (1=>WRITE);
         ATTRIBUTES           :  in    ATTRIBUTE_LIST          := EMPTY_LIST;
         DISCRETIONARY_ACCESS : in     DISCRETIONARY_ACCESS_LIST := ALL_RIGHTS;
         MANDATORY_ACCESS     :  in    MANDATORY_ACCESS_LIST := EMPTY_LIST;
         KIND                 :  in    NODE_KIND_NODE_TYPE     := SECONDARY_STORAGE_NKD;
         STATUS               :  in    STATUS_INDICATOR        := NO_INDICATION;
         MEDIUM_RESOURCE      :  in    MEDIUM_NODE_TYPE        := DEFAULT_MEDIUM);

    procedure CREATE_SECONDARY_STORAGE_NODE
        (NODE                 : in out SECONDARY_STORAGE_NODE_TYPE;
         BASE                 :  in    MODE_TYPE;
         KEY                  :  in    RELATIONSHIP_KEY        := EMPTY_KEY_LIST;
         RELATION             :  in    RELATION_NAME           := DEFAULT_RELATION;
         INTENT               :  in    INTENT_ARRAY            := (l=>WRITE);
         ATTRIBUTES           :  in    ATTRIBUTE_LIST          := EMPTY_LIST;
         DISCRETIONARY_ACCESS : in     DISCRETIONARY_ACCESS_LIST := ALL_RIGHTS;
         MANDATORY_ACCESS     :  in    MANDATORY_ACCESS_LIST := EMPTY_LIST;
         KIND                 :  in    NODE_KIND_NAME          := "SECONDARY_STORAGE";
         STATUS               :  in    STATUS_INDICATOR        := NO_INDICATION;
         MEDIUM_RESOURCE      :  in    MEDIUM_NODE_TYPE        := DEFAULT_MEDIUM);

end CAIS_FILE_NODE_MANAGEMENT;
```


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
with CAIS_STANDARD;
with CAIS_DEFINITIONS;
with CAIS_IO_DEFINITIONS;
with CAIS_STATUS_DEFINITIONS;
with CAIS_NODE_MANAGEMENT;
with CAIS_LIST_MANAGEMENT;
with CAIS_ACCESS_CONTROL_MANAGEMENT;
with CAIS_RESOURCE_CONTROL;

package CAIS_IO_CONNECTION is

    use CAIS_STANDARD;
    use CAIS_DEFINITIONS;
    use CAIS_IO_DEFINITIONS;
    use CAIS_STATUS_DEFINITIONS;
    use CAIS_NODE_MANAGEMENT;
    use CAIS_LIST_MANAGEMENT;
    use CAIS_ACCESS_CONTROL_MANAGEMENT;
    use CAIS_RESOURCE_CONTROL;

    function PORT_RD
        (STATUS :   in STATUS_INDICATOR := NO_INDICATION)
        return PORT_RD_TYPE;

    procedure CREATE_CHANNEL
        (CHANNEL                : in out CHANNEL_NODE_TYPE;
         CLIENT_A               :   in   CLIENT_NODE_TYPE;
         PORT_KEY_A             :   in   RELATIONSHIP_KEY   := EMPTY_KEY_LIST;
         PORT_RELATION_A        :   in   PORT_RD_TYPE       := PORT_RD;
         CLIENT_B               :   in   CLIENT_NODE_TYPE ;
         PORT_KEY_B             :   in   RELATIONSHIP_KEY   := EMPTY_KEY_LIST;
         PORT_RELATION_B        :   in   PORT_RD_TYPE       := PORT_RD;
         ATTRIBUTES_A           :   in   ATTRIBUTE_LIST     := EMPTY_LIST;
         ATTRIBUTES_B           :   in   ATTRIBUTE_LIST     := EMPTY_LIST;
         CHANNEL_PARENT         :   in   PATHNAME           :=  CURRENT_PROCESS;
         DISCRETIONARY_ACCESS : in   DISCRETIONARY_ACCESS_LIST  :=  ALL_RIGHTS;
         MANDATORY_ACCESS       :   in   MANDATORY_ACCESS_LIST := EMPTY_LIST;
         TIME_LIMIT             :   in   CAIS_DURATION      := LONG_DELAY;
         STATUS                 :   in   STATUS_INDICATOR  :=  NO_INDICATION;
         MEDIUM_RESOURCE        :   in   MEDIUM_NODE_TYPE   :=  DEFAULT_MEDIUM);

    procedure RELEASE_CHANNEL
        (CHANNEL            : in CHANNEL_NODE_TYPE;
         STATUS             : in STATUS_INDICATOR := NO_INDICATION);

    procedure  DELETE_CHANNEL_NODE
        (CHANNEL            : in out CHANNEL_NODE_TYPE;
         TIME_LIMIT         :   in   CAIS_DURATION      := LONG_DELAY;
         STATUS             :   in   STATUS_INDICATOR   := NO_INDICATION);

    procedure AWAIT_CHANNEL
        (CHANNEL            : in out CHANNEL_NODE_TYPE;
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
        PORT_KEY          : in    RELATIONSHIP_KEY_PATTERN := EMPTY_KEY_LIST;
        PORT_RELATION     : in    PORT_RD_TYPE;
        TIME_LIMIT        : in    CAIS_IO_DURATION         := LONG_IO_DELAY;
        STATUS            : in    STATUS_INDICATOR         := NO_INDICATION);

    procedure AWAIT_CHANNEL
        (CHANNEL          : in out CHANNEL_NODE_TYPE;
        PORT_KEY          : in     RELATIONSHIP_KEY_PATTERN := EMPTY_KEY_LIST;
        PORT_RELATION_NAME : in    RELATION_NAME_PATTERN    := "PORT";
        TIME_LIMIT        : in     CAIS_IO_DURATION         := LONG_IO_DELAY;
        STATUS            : in     STATUS_INDICATOR         := NO_INDICATION);

    procedure  OPEN_OPPOSITE_CLIENT
        (OPPOSITE_CLIENT : in out CLIENT_NODE_TYPE;
        CHANNEL           : in    CHANNEL_NODE_TYPE;
        INTENT            : in    INTENT_ARRAY     := (1 => NO_ACCESS);
        TIME_LIMIT        : in    CAIS_DURATION    := LONG_DELAY;
        STATUS            : in    STATUS_INDICATOR := NO_INDICATION);

    function COMPATIBLE_PORT_KINDS
        (PORT_RELATION_A : in    PORT_RD_TYPE;
        PORT_RELATION_B : in     PORT_RD_TYPE;
        STATUS          : in     STATUS_INDICATOR := NO_INDICATION)
        return BOOLEAN;

    procedure  SET_PORT_ATTRIBUTE
        (CLIENT           : in    CLIENT_NODE_TYPE;
        KEY               : in    RELATIONSHIP_KEY;
        PORT_RELATION     : in    PORT_RD_TYPE     := PORT_RD;
        ATTRIBUTE         : in    AD_TYPE;
        VALUE             : in    LIST_TYPE;
        STATUS            : in    STATUS_INDICATOR := NO_INDICATION);

    procedure  SET_PORT_ATTRIBUTE
        (CLIENT           : in    CLIENT_NODE_TYPE;
        KEY               : in    RELATIONSHIP_KEY;
        PORT_RELATION     : in    RELATION_NAME    := "PORT";
        ATTRIBUTE         : in    ATTRIBUTE_NAME;
        VALUE             : in    LIST_TYPE;
        STATUS            : in    STATUS_INDICATOR := NO_INDICATION);

end CAIS_IO_CONNECTION;
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
with CAIS_DEFINITIONS;
with CAIS_LIST_MANAGEMENT;
with  CAIS_STATUS_DEFINITIONS;

package CAIS_INPORT_EXPORT is

        use CAIS_DEFINITIONS;
        use CAIS_LIST_MANAGEMENT;
        use  CAIS_STATUS_DEFINITIONS;

        procedure  IMPORT_CONTENTS
            (FROM                : in    STRING;
             TO                  : in    SECONDARY_STORAGE_NODE_TYPE;
             CHARACTERISTICS : in    LIST_TYPE            := EMPTY_LIST;
             STATUS              : in    STATUS_INDICATOR  :=  NO_INDICATION);

        procedure  IMPORT_CONTENTS
            (FROM                : in    STRING;
             TO                  : in    PATHNAME;
             CHARACTERISTICS : in    LIST_TYPE            := EMPTY_LIST;
             STATUS              : in    STATUS_INDICATOR  :=  NO_INDICATION);

        procedure EXPORT_CONTENTS
            (FROM                : in    SECONDARY_STORAGE_NODE_TYPE;
             TO                  : in    STRING;
             CHARACTERISTICS  : in    LIST_TYPE            := EMPTY_LIST;
             STATUS              : in    STATUS_INDICATOR  :=  NO_INDICATION);

        procedure EXPORT_CONTENTS
            (FROM                : in    PATHNAME;
             TO                  : in    STRING;
             CHARACTERISTICS : in    LIST_TYPE            := EMPTY_LIST;
             STATUS              : in    STATUS_INDICATOR  :=  NO_INDICATION);

end CAIS_IMPORT_EXPORT;
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
with SYSTEM;
with CAIS_STANDARD;
with CAIS_PRAGMATICS;
with CAIS_DEFINITIONS;
with CAIS_STATUS_DEFIMITIONS;
with CAIS_IO_DEFINITIONS;

package CAIS_FRAME_IO is

    use CAIS_STANDARD;
    use CAIS_PRAGMATICS;
    use CAIS_DEFINITIONS;
    use CAIS_STATUS_DEFINITIONS;
    use CAIS_IO_DEFINITIONS;

    type FILE_TYPE is limited private;

    type FILE_NODE is (IN_FILE, INOUT_FILE, OUT_FILE, APPEND_FILE);

    subtype FRAME_ADDRESS is SYSTEM_ADDRESS;

    subtype FRAME_LENGTH is
        CAIS_INTEGER range 1.. CAIS_PRAGMATICS.MAXIMUM_FRAME_LENGTH;

    MINIMUM_FRAME_PRIORITY : constant := 1;

    subtype PRIORITY_TYPE is CAIS_INTEGER
        range MINIMUM_FRAME_PRIORITY . . CAIS_PRAGMATICS.MAXIMUM_FRAME_PRIORITY;

    QUEUE_EMPTY : exception;
    QUEUE_FULL  : exception;

    procedure OPEN
        (FILE           : in out FILE_TYPE;
         CHANNEL        : in     CHANNEL_NODE_TYPE;
         OPPOSITE_CLIENT : in    CLIENT_NODE_TYPE;
         NODE           : in     FILE_MODE;
         TIME_LIMIT     : in     CAIS_IO_DURATION := LONG_IO_DELAY;
         STATUS         : in     STATUS_INDICATOR := NO_INDICATION);

    procedure CLOSE
        (FILE   : in out FILE_TYPE;
         STATUS : in     STATUS_INDICATOR := NO_INDICATION);

    function IS_OPEN
        (FILE   : in FILE_TYPE;
         STATUS : in STATUS_INDICATOR := NO_INDICATION)
        return BOOLEAN;

    function MODE
        (FILE   : in FILE_TYPE;
         STATUS : in STATUS_INDICATOR := NO_INDICATION)
        return FILE_MODE;
```


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
        procedure WRITE
            (FILE      : in FILE_TYPE;
             FRAME     : in FRAME_ADDRESS;
             LENGTH    : in FRAME_LENGTE;
             PRIORITY  : in PRIORITY_TYPE    := MINIMUM_FRAME_PRIORITY;
             BLOCKING  : in BOOLEAN          := FALSE ;
             TIME_LIMIT : in CAIS_IO_DURATION := LONG_IO_DELAY;
             STATUS    : in STATUS_INDICATOR := NO_INDICATION);

        procedure READ
            (FILE      : in     FILE_TYPE;
             FRAME     : in     FRAME_ADDRESS;
             LENGTH    :        out FRAME_LEMGTH;
             BLOCKING  : in     BOOLEAN          := FALSE;
             TIME_LIMIT : in    CAIS_IO_DURATION := LONG_IO_DELAY;
             STATUS    : in     STATUS_INDICATOR := NO_INDICATION);

        procedure RESET
            (FILE   : in out FILE_TYPE;
             MODE   : in     FILE_MODE;
             STATUS : in     STATUS_INDICATOR := NO_INDICATION);

        function END_OF_FILE
            (FILE   : in FILE_TYPE;
             STATUS : in STATUS_INDICATOR := NO_INDICATION)
            return BOOLEAN;

        function INCOMING_QUEUE_EMPTY
            (FILE   : in FILE_TYPE;
             STATUS : in STATUS_INDICATOR := NO_INDICATION)
            return BOOLEAN;

        function OUTGOING_QUEUE_FULL
            (FILE   : in FILE_TYPE;
             STATUS : in STATUS_INDICATOR := NO_INDICATION)
            return BOOLEAN;

        procedure GET_ASSOCIATED_CHANNEL
            (CHANNEL : in out CHANNEL_NODE_TYPE;
             FILE    : in FILE_TYPE;
             STATUS  : in STATUS_INDICATOR := NO_INDICATION);

    private

        type FILE_TYPE is (IMPLEMENTATION_DEFINED);
        -- This type should b. defined by the implementer.

    end CAIS_FRAME_IO;
```


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
with CAIS_DEFINITIONS;
with CAIS_STATUS_DEFINITIONS;
with CAIS_STANDARD;
with CAIS_IO_DEFINITIONS;

package CAIS_MAGNETIC_TAPE_IO is

    use CAIS_DEFINITIONS;
    use CAIS_STATUS_DEFINITIONS;
    use CAIS_STANDARD;
    use CAIS_IO_DEFINITIONS;

    type FILE_TYPE is limited private;

    type FILE_MODE is (IN_FILE, OUT_FILE);

    subtype TAPE_NAME is STRING;

    subtype TAPE_BLOCK is STRING;

    type TAPE_DRIVE_STATUS_KIND is
            (OPENED,
             MOUNT_REQUESTED,
             MOUNTED,
             LOADED,
             CLOSED);

    type TAPE_POSITION_KIND is
            (BEGINNING_OF_VOLUME,
             END_OF_VOLUME,
             END_OF_TAPE,
             AFTER_TAPE_MARK,
             OTHER);

    type TAPE_RECORDING_METHOD_KIND is
            (NON_RETURN_TO_ZERO_INVERTED,
             PHASE_ENCODED,
             GROUP_CODED_RECORDING);

    TAPE_STATUS_ERROR: exception;

    procedure OPEN
        (TAPE_DRIVE     : in out FILE_TYPE;
         CHANNEL        : in     CHANNEL_NODE_TYPE;
         OPPOSITE_CLIENT : in    CLIENT_NODE_TYPE;
         MODE           : in     FILE_MODE;
         TIME_LIMIT     : in     CAIS_IO_DURATION := LONG_IO_DELAY;
         STATUS         : in     STATUS_INDICATOR := NO_INDICATION);

    procedure CLOSE
        (TAPE_DRIVE : in out FILE_TYPE;
         STATUS     : in     STATUS_INDICATOR := NO_INDICATION);
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
function IS_OPEN
    (TAPE_DRIVE : in FILE_TYPE;
     STATUS     : in STATUS_INDICATOR := NO_INDICATION)
    return BOOLEAN;

function MODE
    (TAPE_DRIVE : in FILE_TYPE;
     STATUS     : in STATUS_INDICATOR := NO_INDICATION)
    return FILE_MODE;

procedure REQUEST_MOUNT
    (TAPE_DRIVE       : in FILE_TYPE;
     NAME             : in TAPE_NAME;
     RECORDING_METHOD : in TAPE_RECORDING_METEOD_KIND;
     INSTALL_WRITE_RING : in BOOLEAN := FALSE;
     STATUS           : in STATUS_INDICATOR := NO_INDICATION);

procedure LOAD
    (TAPE_DRIVE : in FILE_TYPE;
     BLOCK_SIZE : in CAIS_POSITIVE;
     STATUS     : in STATUS_INDICATOR := NO_INDICATION);

procedure UNLOAD
    (TAPE_DRIVE : in FILE_TYPE;
     STATUS     : in STATUS_INDICATOR := NO_INDICATION);

procedure REQUEST_DISMOUNT
    (TAPE_DRIVE : in FILE_TYPE;
     STATUS     : in STATUS_INDICATOR := NO_INDICATION);

function POSITION
    (TAPE_DRIVE : in FILE_TYPE;
     STATUS     : in STATUS_INDICATOR := NO_INDICATION)
    return TAPE_POSITION_KIND;

procedure REWIND_TAPE
    (TAPE_DRIVE : in FILE_TYPE;
     STATUS     : in STATUS_INDICATOR := NO_INDICATION);

procedure SKIP_TAPE_MARK
    (TAPE_DRIVE : in FILE_TYPE;
     COUNT      : in CAIS_POSITIVE := 1;
     STATUS     : in STATUS_INDICATOR := NO_INDICATION);

procedure WRITE_TAPE_MARK
    (TAPE_DRIVE : in FILE_TYPE;
     STATUS     : in STATUS_INDICATOR := NO_INDICATION);

function STATUS
    (TAPE_DRIVE : in FILE_TYPE;
     STATUS     : in STATUS_INDICATOR := NO_INDICATION)
    return TAPE_DRIVE_STATUS_KIND;
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
    function RECORDING_METHOD
        (TAPE_DRIVE : in FILE_TYPE;
         STATUS     : in STATUS_INDICATOR := NO_INDICATION)
        return TAPE_RECORDING_METHOD_KIND;

    function IS_WRITE_RING_INSTALLED
        (TAPE_DRIVE : in FILE_TYPE;
         STATUS     : in STATUS_INDICATOR := NO_INDICATION)
        return BOOLEAN;

    procedure SKIP_BLOCK
        (TAPE_DRIVE : in FILE_TYPE;
         COUNT      : in CAIS_POSITIVE := 1;
         STATUS     : in STATUS_INDICATOR := NO_INDICATION);

    procedure READ_BLOCK
        (TAPE_DRIVE      : in  FILE_TYPE;
         BLOCK           : out TAPE_BLOCK;
         LAST            : out CAIS_NATURAL;
         BLOCK_OVERFLOW  : out CAIS_NATURAL;
         STATUS          : in  STATUS_INDICATOR := NO_INDICATION);

    procedure WRITE_BLOCK
        (TAPE_DRIVE : in FILE_TYPE;
         BLOCK      : in TAPE_BLOCK;
         STATUS     : in STATUS_INDICATOR := NO_INDICATION);

    procedure RESET
        (TAPE_DRIVE : in out FILE_TYPE;
         MODE       : in  FILE_MODE;
         STATUS     : in  STATUS_INDICATOR := NO_INDICATION);

    procedure GET_ASSOCIATED_CHANNEL
        (CHANNEL    : in out CHANNEL_NODE_TYPE;
         TAPE_DRIVE : in  FILE_TYPE;
         STATUS     : in  STATUS_INDICATOR := NO_INDICATION);

private

    type FILE_TYPE is (IMPLEMENTATION_DEFINED);
    -- This type should be defined by the implementer.

end CAIS_MAGNETIC_TAPE_IO;
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
with CAIS_DEFINITIONS;
with CAIS_IO_DEFINITIONS;
with CAIS_IO_CONNECTION;
with CAIS_STATUS_DEFINITIONS;
with CAIS_STANDARD;
with CAIS_LIST_MANAGEMENT;
with CAIS_ACCESS_CONTROL_MANAGEMENT;
with CAIS_RESOURCE_CONTROL;
with CAIS_FILE_NODE_MANAGEMENT;


generic
    type ELEMENT_TYPE in private;
package CAIS_DIRECT_IO is


    use CAIS_DEFINITIONS;
    use CAIS_IO_DEFINITIONS;
    use CAIS_IO_CONNECTION;
    use CAIS_STATUS_DEFINITIONS;
    use CAIS_STANDARD;
    use CAIS_LIST_MANAGEMENT;
    use CAIS_ACCESS_CONTROL_MANAGEMENT;
    use CAIS_RESOURCE_CONTROL;
    use CAIS_FILE_NODE_MANAGEMENT;


    type FILE_TYPE          is limited private;

    type FILE_MODE          is (IN_FILE, INOUT_FILE, OUT_FILE);

    type COUNT              is range 0 . . implementation_defined;

    subtype POSITIVE_COUNT is COUNT range 1 . . COUNT'LAST;


    procedure CREATE
        (NODE                   : in out SECONDARY_STORAGE_NODE_TYPE;
         FILE                   : in out FILE_TYPE;
         BASE                   : in     NODE_TYPE;
         KEY                    : in     RELATIONSHIP_KEY   := EMPTY_KEY_LIST;
         RELATION               : in     RD_TYPE ;
         INTENT                 : in     INTENT_ARRAY       := (l=>WRITE);
         MODE                   : in     FILE_MODE          := INOUT_FILE;
         ATTRIBUTES             : in     ATTRIBUTE_LIST     := EMPTY_LIST;
         MAXIMUM_FILE_SIZE      : in     CAIS_NATURAL       := UNBOUNDED_FILE_SIZE;
         DISCRETIONARY_ACCESS   : in     DISCRETIONARY_ACCESS_LIST := ALL_RIGHTS;
         MANDATORY_ACCESS       : in     MANDATORY_ACCESS_LIST :=EMPTY_LIST;
         KIND                   : in     NODE_KIND_NODE_TYPE := SECONDARY_STORAGE_NKD;
         PROCESS_PORT_KEY       : in     RELATIONSHIP_KEY   := EMPTY_KEY_LIST;
         PROCESS_PORT_RELATION : in      PORT_RD_TYPE       := PORT_RD;
         FILE_PORT_KEY          : in     RELATIONSHIP_KEY   := EMPTY_KEY_LIST;
         FILE_PORT_RELATION     : in     PORT_RD_TYPE       := PORT_RD;
         CHANNEL_PARENT         : in     PATHNAME           := CURRENT_PROCESS;
         STATUS                 : in     STATUS_INDICATOR   := NO_INDICATION;
         MEDIUM_RESOURCE        : in     MEDIUM_NODE_TYPE   := DEFAULT_MEDIUM);
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
procedure CREATE
    (NODE                    : in out SECONDARY_STORAGE_NODE_TYPE;
     FILE                    : in out FILE_TYPE;
     BASE                    : in     NODE_TYPE;
     KEY                     : in     RELATIONSHIP_KEY      := EMPTY_KEY_LIST;
     RELATION                : in     RELATION_NAME         := DEFAULT_RELATION;
     INTENT                  : in     INTENT_ARRAY          := (1=>WRITE);
     MODE                    : in     FILE_MODE             := INOUT_FILE;
     ATTRIBUTES              : in     ATTRIBUTE_LIST        := EMPTY_LIST;
     MAXIMUM_FILE_SIZE       : in     CAIS_NATURAL          := UNBOUNDED_FILE_SIZE;
     DISCRETIONARY_ACCESS : in     DISCRETIONARY_ACCESS_LIST := ALL_RIGHTS;
     MANDATORY_ACCESS        : in     MANDATORY_ACCESS_LIST := EMPTY_LIST;
     STATUS                  : in     STATUS_INDICATOR      := NO_INDICATION;
     MEDIUM_RESOURCE         : in     MEDIUM_NODE_TYPE      := DEFAULT_MEDIUM);


procedure OPEN
    (FILE                : in out FILE_TYPE ;
     CHANNEL             : in     CHANNEL_NODE_TYPE;
     OPPOSITE_CLIENT     : in     CLIENT_NODE_TYPE;
     MODE                : in     FILE_MODE;
     TIME_LIMIT          : in     CAIS_IO_DURATION := LONG_IO_DELAY;
     STATUS              : in     STATUS_INDICATOR  := NO_INDICATION);


procedure CLOSE
    (FILE     : in out FILE_TYPE;
     STATUS : in     STATUS_INDICATOR  := NO_INDICATION);


procedure RESET
    (FILE          : in out FILE_TYPE;
     MODE          : in     FILE_MODE;
     STATUS        : in     STATUS_INDICATOR := NO_INDICATION);


procedure SYNCHRONIZE
    (FILE     : in     FILE_TYPE;
     STATUS : in     STATUS_INDICATOR  := NO_INDICATION);


proceduee GET_ASSOCIATED_CHANNEL
    (CHANNEL : in out CHANNEL_NODE_TYPE;
     FILE       : in     FILE_TYPE;
     STATUS     : in     STATUS_INDICATOR  := NO_INDICATION);


function MODE
    (FILE     : in     FILE_TYPE;
     STATUS : in     STATUS_INDICATOR  := NO_INDICATION)
    return FILE_MODE;


function IS_OPEN
    (FILE     : in     FILE_TYPE;
     STATUS : in     STATUS_INDICATOR  := NO_INDICATION)
    return BOOLEAN;
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
        procedure READ
            (FILE     : in      FILE_TYPE;
             ITEM     :    out  ELEMENT_TYPE;
             FROM     : in      POSITIVE_COUNT;
             STATUS : in        STATUS_INDICATOR  := NO_INDICATION);

        procedure READ
            (FILE     : in      FILE_TYPE;
             ITEM     :    out  ELEMENT_TYPE;
             STATUS : in        STATUS_INDICATOR  := NO_INDICATION);

        procedure WRITE
            (FILE     : in      FILE_TYPE;
             ITEM     : in      ELEMENT_TYPE;
             TO       : in      POSITIVE_COUNT;
             STATUS : in        STATUS_INDICATOR  := NO_INDICATION);

        proceduee WRITE
            (FILE     : in      FILE_TYPE;
             ITEM     : in      ELEMENT_TYPE;
             STATUS : in        STATUS_INDICATOR  := NO_INDICATION);

        procedure SET. INDEX
            (FILE     : in      FILE_TYPE;
             TO       : in      POSITIVE_COUNT;
             STATUS : in        STATUS_INDICATOR  := NO_INDICATION);

        function INDEX
            (FILE     : in      FILE_TYPE;
             STATUS : in        STATUS_INDICATOR  := NO_INDICATION)
            return POSITIVE_COUNT;

        function SIZE
            (FILE     : in      FILE_TYPE;
             STATUS : in        STATUS_INDICATOR  := NO_INDICATION)
            return COUNT;

        function END_OF_FILE
            (FILE     : in      FILE_TYPE;
             STATUS : in        STATUS_INDICATOR  := NO_INDICATION)
            return BOOLEAN;

    private

        type FILE_TYPE is (IMPLEMENTATION_DEFINED);
        -- This type should be defined by the implementer.

    end CAIS_DIRECT_IO;
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
with CAIS_DEFINITIONS;
with CAIS_IO_DEFINITIONS;
with CAIS_IO_CONNECTION;
with  CAIS_STATUS_DEFINITIONS;
with  CAIS_STANDARD;
with CAIS_LIST_MANAGEMENT;
with  CAIS_ACCESS_CONTROL_MANAGEMENT;
with CAIS_RESOURCE_CONTROL;
with   CAIS_FILE_NODE_MANAGEMENT;

generic
    type ELEMENT_TYPE is private;
package CAIS_SEQUENTIAL_IO is

        use CAIS_DEFINITIONS;
        use CAIS_IO_DEFINITIONS;
        use CAIS_IO_CONNECTION;
        use  CAIS_STATUS_DEFINITIONS;
        use  CAIS_STANDARD;
        use CAIS_LIST_MANAGEMENT;
        use CAIS_RESOURCE_CONTROL;
        use   CAIS_FILE_NODE_MANAGEMENT;
        use CAIS_ACCESS_CONTROL_MANAGEMENT;

        type  FILE_TYPE  is  limited  private;

        type FILE_MODE is (IN_FILE, OUT_FILE, APPEND_FILE);

        procedure CREATE
            (NODE                    : in out NODE_TYPE;
             FILE                    : in out FILE_TYPE;
             BASE                    :  in    NODE_TYPE;
             KEY                     :  in    RELATIONSHIP_KEY      := LATEST_KEY;
             RELATION                :  in    NODE_TYPE;
             INTENT                  :  in    INTENT_ARRAY          := (1=>WRITE);
             MODE                    :  in    FILE_MODE             := OUT_FILE;
             ATTRIBUTES              :  in    ATTRIBUTE_LIST        := EMPTY_LIST;
             MAXIMUM_FILE_SIZE       :  in    CAIS_NATURAL          :=
                                                 UNBOUNDED_FILE_SIZE;
             DISCRETIONARY_ACCESS : in    DISCRETIONARY_ACCESS_LIST  :=  ALL_RIGHTS;
             MANDATORY_ACCESS        :  in    MANDATORY_ACCESS_LIST := EMPTY_LIST;
             KIND                    :  in    NODE_TYPE             :=
                                                 SECONDARY_STORAGE_NKE;
             PROCESS_PORT_KEY        :  in    RELATIONSHIP_KEY      := LATEST_KEY;
             PROCESS_PORT_RELATION : in    NODE_TYPE             := PORT_RD;
             FILE_PORT_KEY           :  in    RELATIONSHIP_KEY      := LATEST_KEY;
             FILE_PORT_RELATION      :  in    NODE_TYPE             := PORT_RD;
             CHANNEL_PARENT          :  in    PATHNAME              := CURRENT_PROCESS;
             STATUS                  :  in    STATUS_INDICATOR      := NO_INDICATION;
             MEDIUM_RESOURCE         :  in    NODE_TYPE             :=  DEFAULT_MEDIUM);

        procedure CREATE
            (NODE                    : in out NODE_TYPE;
             FILE                    : in out FILE_TYPE;
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
        BASE                    : in    MODE_TYPE;
        KEY                     : in    RELATIONSHIP_KEY        := LATEST_KEY;
        RELATION                : in    RELATION_NAME           := DEFAULT_RELATION;
        INTENT                  : in    INTENT_ARRAY            := (1=>WRITE);
        MODE                    : in    FILE_MODE               := OUT_FILE;
        ATTRIBUTES              : in    ATTRIBUTE_LIST          := EMPTY_LIST;
        MAXIMUM_FILE_SIZE       : in    CAIS_NATURAL            :=
                                            UNBOUNDED_FILE_SIZE;
        DISCRETIONARY_ACCESS : in    DISCRETIONARY_ACCESS_LIST  := ALL_RIGHTS;
        MANDATORY_ACCESS        : in    MANDATORY_ACCESS_LIST := EMPTY_LIST;
        STATUS                  : in    STATUS_INDICATOR        := NO_INDICATION;
        MEDIUM_RESOURCE         : in    NODE_TYPE               :=  DEFAULT_MEDIUM);

    procedure  OPEN
        (FILE                : in  out  FILE_TYPE;
         CHANNEL             : in       NODE_TYPE;
         OPPOSITE_CLIENT : in       NODE_TYPE;
         MODE                : in       FILE_MODE;
         TIME_LIMIT          : in       CAIS_IO_DURATION := LONG_IO_DELAY;
         STATUS              : in       STATUS_INDICATOR  := NO_INDICATION);

    procedure  CLOSE
        (FILE     : in out FILE_TYPE;
         STATUS : in       STATUS_INDICATOR  := NO_INDICATION);

    procedure RESET
        (FILE          : in out FILE_TYPE;
         MODE          : in       FILE_MODE;
         STATUS        : in       STATUS_INDICATOR  := NO_INDICATION);

    procedure  SYNCHRONIZE
        (FILE     : in       FILE_TYPE;
         STATUS : in       STATUS_INDICATOR  := NO_INDICATION);

    procedure  GET_ASSOCIATED_CHANNEL
        (FILE       : in       NODE_TYPE;
         CHANNEL :  in out NODE_TYPE;
         STATUS  : in       STATUS_INDICATOR  := NO_INDICATION);

    function  MODE
        (FILE     : in       FILE_TYPE;
         STATUS : in       STATUS_INDICATOR  := NO_INDICATION)
        return FILE_MODE;

    function IS_OPEN
        (FILE     : in       FILE_TYPE;
         STATUS : in       STATUS_INDICATOR  := NO_INDICATION)
        return BOOLEAN;

    procedure READ
        (FILE     : in       FILE_TYPE;
         ITEM   :      out  ELEMENT_TYPE;
         STATUS : in       STATUS_INDICATOR  := NO_INDICATION);
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
        procedure WRITE
            (FILE   : in    FILE_TYPE ;
             ITEM   : in    ELEMENT_TYPE ;
             STATUS : in    STATUS_INDICATOR  := NO_INDICATION);

        function END_OF_FILE
            (FILE   : in    FILE_TYPE;
             STATUS : in    STATUS_INDICATOR  := NO_INDICATIOM)
            return BOOLEAN;

private

    type FILE_TYPE is (IMPLEMENTATION_DEFINED);
    -- This type should b. defined by the implementor.

end CAIS_SEQUENTIAL_IO;
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
with CAIS_DEFINITIONS;
with CAIS_IO_DEFINITIONS;
with CAIS_IO_CONNECTION;
with  CAIS_STATUS_DEFINITIONS;
with  CAIS_STANDARD;
with CAIS_LIST_MANAGEMENT;
with  CAIS_ACCESS_CONTROL_MANAGEMENT;
with CAIS_RESOURCE_CONTROL;
with   CAIS_FILE_NODE_MANAGEMENT;

package CAIS_TEXT_IO is

      use  CAIS_DEFINITIONS;
      use  CAIS_IO_DEFINITIONS;
      use  CAIS_IO_CONNECTION;
      use  CAIS_STATUS_DEFINITIONS;
      use  CAIS_STANDARD;
      use  CAIS_LIST_MANAGEMENT;
      use  CAIS_ACCESS_CONTROL_MANAGEMENT;
      use  CAIS_RESOURCE_CONTROL;
      use  CAIS_FILE_NODE_MANAGEMENT;

      type  FILE_TYPE          is  limited  private;

      type FILE_M0DE           is (IN_FILE, OUT_FILE, APPEND_FILE);

      type COUNT               is  range  0  . .  implementation_defined;

      subtype POSITIVE_COUNT is COUNT range 1 . . COUNT'LAST;

      subtype  FIELD           is  CAIS_INTEGER  range  0  . .  implementation_defined;

      subtype  NUMBER_BASE     is CAIS_INTEGER rang. 2 . . 16;

      typo TYPE_SET            is  (LOWERCASE,  UPPERCASE);

      procedure  CREATE
          (NODE                    : in out NODE_TYPE;
           FILE                    : in out FILE_TYPE;
           BASE                    :  in      NODE_TYPE;
           KEY                     :  in      RELATIONSHIP_KEY        := LATEST_KEY;
           RELATION                :  in      NODE_TYPE;
           INTENT                  :  in      INTENT_ARRAY            := (l=>WRITE);
           MODE                    :  in      FILE_MODE               := OUT_FILE;
           ATTRIBUTES              :  in      ATTRIBUTE_LIST          := EMPTY_LIST;
           MAXIMUM_FILE_SIZE       :  in      CAIS_NATURAL            :=
                                                  UNBOUNDED_FILE_SIZE;
           DISCRETIONARY_ACCESS    :  in      DISCRETIONARY_ACCESS_LIST  :=  ALL_RIGHTS;
           MANDATORY_ACCESS        :  in      MANDATORY_ACCESS_LIST := EMPTY_LIST;
           KIND                    :  in      NODE_TYPE               :=
                                                  SECONDARY_STORAGE_NKD  ;
           PROCESS_PORT_KEY        :  in      RELATIONSHIP_KEY        := LATEST_KEY;
           PROCESS_PORT_RELATION : in      NODE_TYPE               :=  PORT_RD;
```



Source: https://assist.dla.mil -- Downloaded: 2026l0-26T07:54Z
Check the source to verify that this is the currentversion before use.

```
        FILE_PORT_KEY          : in    RELATIONSHIP_KEY  := LATEST_KEY;
        FILE_PORT_RELATIOH     : in    NODE_TYPE         := PORT_RD;
        CHANNEL_PARENT         : in    PATHNAME          := CURRENT_PROCESS;
        STATUS                 : in    STATUS_INDICATOR  := NO_INDICATION;
        MEDIUM_RESOURCE        : in    NODE_TYPE         := DEFAULT_MEDIUM);

    procedure CREATE
        (NODE                  : in out NODE_TYPE;
        FILE                   : in out FILE_TYPE;
        BASE                   : in    NODE_TYPE;
        KEY                    : in    RELATIONSHIP_KEY  := LATEST_KEY;
        RELATION               : in    RELATION_NAME     := DEFAULT_RELATION;
        INTENT                 : in    INTENT_ARRAY      := (1=>WRITE);
        MODE                   : in    FILE_MODE         := OUT_FILE;
        ATTRIBUTES             : in    ATTRIBUTE_LIST    := EMPTY_LIST;
        MAXIMUM_FILE_SIZE      : in    CAIS_NATURAL      :=
                                           UNBOUNDED_FILE_SIZE;
        DISCRETIONARY_ACCESS : in      DISCRETIONARY_ACCESS_LIST := ALL_RIGHTS;
        MANDATORY_ACCESS       : in    MANDATORY_ACCESS_LIST := EMPTY_LIST;
        STATUS                 : in    STATUS_INDICATOR  := NO_INDICATION;
        MEDIUM_RESOURCE        : in    NODE_TYPE         := DEFAULT_MEDIUM);

    procedure OPEN
        (FILE                  : in out FILE_TYPE;
        CHANNEL                : in    NODE_TYPE;
        OPPOSITE_CLIENT        : in    NODE_TYPE;
        MODE                   : in    FILE_MODE;
        TIME_LIMIT             : in    CAIS_IO_DURATION := LONG_IO_DELAY;
        STATUS                 : in    STATUS_INDICATOR  := NO_INDICATION);

    procedure CLOSE
        (FILE     : in out FILE_TYPE;
        STATUS : in       STATUS_INDICATOR  := NO_INDICATION);

    procedure RESET
        (FILE          : in out FILE_TYPE;
        MODE           : in    FILE_MODE;
        STATUS         : in    STATUS_INDICATOR := NO_INDICATION);

    procedure SYNCHRONIZE
        (FILE     : in    FILE_TYPE;
        STATUS : in       STATUS_INDICATOR  := NO_INDICATION);

    procedure GET_ASSOCIATED_CHANNEL
        (FILE     : in    FILE_TYPE;
        CHANNEL : in out NODE_TYPE;
        STATUS    : in    STATUS_INDICATOR  := NO_INDICATION);

    function MODE
        (FILE     : in    FILE_TYPE;
        STATUS : in       STATUS_INDICATOR  := NO_INDICATION)
        return FILE_MODE;
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
function IS_OPEM
    (FILE   : in    FILE_TYPE;
     STATUS : in    STATUS_INDICATOR  := NO_INDICATION)
    return BOOLEAN;

procedure SET_INPUT
    (FILE   : in    FILE_TYPE;
     STATUS : in    STATUS_INDICATOR  := NO_INDICATION);

procedure SET_OUTPUT
    (FILE   : in    FILE_TYPE;
     STATUS : in    STATUS_INDICATOR  := NO_INDICATION);

function CURRENT_INPUT
    (STATUS : in    STATUS_INDICATOR  := NO_INDICATION)
    return FILE_TYPE;

function CURRENT_OUTPUT
    (STATUS : in    STATUS_INDICATOR  := NO_INDICATION)
    return FILE_TYPE;

procedure SET_LINE_LENGTH
    (FILE   : in    FILE_TYPE;
     TO     : in    COUNT ;
     STATUS : in    STATUS_INDICATOR  := NO_INDICATION);

procedure SET_LINE_LENGTH
    (TO     : in    COUNT;
     STATUS : in    STATUS_INDICATOR  := NO_INDICATION);

procedure SET_PAGE_LENGTH
    (FILE   : in    FILE_TYPE;
     TO     : in    COUNT;
     STATUS : in    STATUS_INDICATOR  := NO_INDICATION);

procedure SET_PAGE_LENGTH
    (TO     : in    COUNT;
     STATUS : in    STATUS_INDICATOR  := NO_INDICATION);

function LINE_LENGTH
    (FILE   : in    FILE_TYPE;
     STATUS : in    STATUS_INDICATOR  := NO_INDICATION)
    return COUNT;

function LINE_LENGTH
    (STATUS : in    STATUS_INDICATOR  := NO_INDICATION)
    return COUNT;

function PAGE_LENGTH
    (FILE   : in    FILE_TYPE;
     STATUS : in    STATUS_INDICATOR  := NO_INDICATION)
    return COUNT;
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
function  PAGE_LENGTH
     (STATUS : in       STATUS_INDICATOR  := NO_INDICATION)
     return COUNT;

procedure  NEW_LINE
     (FILE      :  in      FILE_TYPE;
      SPACING : in       POSITIVE_COUNT := 1;
      STATUS  :  in      STATUS_INDICATOR  :=  NO_INDICATION);

procedure  NEW_LINE
     (SPACING : in       POSITIVE_COUNT := 1;
      STATUS  :  in      STATUS_INDICATOR  :=  NO_INDICATION);

procedure SKIP_LINE
     (FILE      :  in      FILE_TYPE;
      SPACING : in       POSITIVE_COUNT := 1;
      STATUS  :  in      STATUS_INDICATOR  :=  NO_INDICATION);

procedure SKIP_LINE
     (SPACING : in       POSITIVE_COUNT := 1;
      STATUS    :  in      STATUS_INDICATOR  :=  NO_INDICATION);

function  END_OF_LINE
     (FILE      :  in      FILE_TYPE;
      STATUS : in       STATUS_INDICATOR  := NO_INDICATION)
     return BOOLEAN;

function  END_OF_LINE
     (STATUS : in       STATUS_INDICATOR  := NO_INDICATION)
     return BOOLEAN;

procedure  NEW_PAGE
     (FILE      :  in      FILE_TYPE;
      STATUS : in       STATUS_INDICATOR  :=  NO_INDICATION);

procedure NEW_PAGE
     (STATUS : in       STATUS_INDICATOR  :=  NO_INDICATION);

procedure  SKIP_PAGE
     (FILE      :  in      FILE_TYPE;
      STATUS : in       STATUS_INDICATOR  :=  NO_INDICATION);

procedure  SKIP_PAGE
     (STATUS : in       STATUS_INDICATOR  :=  NO_INDICATION);

function END_OF_PAGE
     (FILE      :  in      FILE_TYPE;
      STATUS : in       STATUS_INDICATOR  := NO_INDICATION)
     return BOOLEAN;

function END_OF_PAGE
     (STATUS : in       STATUS_INDICATOR  := NO_INDICATION)
     return BOOLEAN;
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
function END_OF_FILE
    (FILE   : in    FILE_TYPE ;
     STATUS : in    STATUS_ INDICATOR  := NO_INDICATION)
    return BOOLEAN;

function END_OF_FILE
    (STATUS : in    STATUS_INDICATOR   := NO_INDICATION)
    return BOOLEAN;

procedure SET_COL
    (FILE   : in    FILE_TYPE;
     TO     : in    POSITIVE_COUNT;
     STATUS : in    STATUS_INDICATOR   := NO_INDICATION);

procedure SET_COL
    (TO     : in    POSITIVE_COUNT;
     STATUS : in    STATUS_INDICATOR   := NO_INDICATION);

procedure SET_LINE
    (FILE   : in    FILE_TYPE;
     TO     : in    POSITIVE_COUNT;
     STATUS : in    STATUS_INDICATOR   := NO_INDICATION);

procedure SET_LINE
    (TO     : in    POSITIVE_COUNT;
     STATUS : in    STATUS_INDICATOR   := NO_INDICATION);

function COL
    (FILE   : in    FILE_TYPE;
     STATUS : in    STATUS_INDICATOR   := NO_INDICATION)
    return POSITIVE_COUNT;

function COL
    (STATUS : in    STATUS_INDICATOR   := NO_INDICATION)
    return POSITIVE_COUNT;

function LINE
    (FILE   : in    FILE_TYPE;
     STATUS : in    STATUS_INDICATOR   := NO_INDICATION)
    return POSITIVE_COUNT;

function LINE
    (STATUS : in    STATUS_INDICATOR   := NO_INDICATION)
    return POSITIVE_COUNT;

function PAGE
    (FILE   : in    FILE_TYPE;
     STATUS : in    STATUS_INDICATOR   := NO_INDICATION)
    return POSITIVE_COUNT;

function PAGE
    (STATUS : in    STATUS_INDICATOR   := NO_INDICATION)
    return POSITIVE_COUNT;
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
procedure GET
    (FILE   : in     FILE_TYPE;
     ITEM   :    out CHARACTER;
     STATUS : in     STATUS_INDICATOR  := NO_INDICATION);

procedure GET
    (ITEM   :    out CHARACTER;
     STATUS : in     STATUS_INDICATOR  := NO_INDICATION);

procedure PUT
    (FILE   : in     FILE_TYPE;
     ITEM   : in     CHARACTER ;
     STATUS : in     STATUS_INDICATOR  := NO_INDICATION);

procedure PUT
    ( ITEM  : in     CHARACTER;
     STATUS : in     STATUS_INDICATOR  := NO_INDICATION);

procedure GET
    (FILE   : in     FILE_TYPE;
     ITEM   :    out STRING;
     STATUS : in     STATUS_INDICATOR  := NO_INDICATION);

procedure GET
    (ITEM   :    out STRING;
     STATUS : in     STATUS_INDICATOR  := NO_INDICATION);

procedure PUT
    (FILE   : in     FILE_TYPE;
     ITEM   : in     STRING;
     STATUS : in     STATUS_INDICATOR  := NO_INDICATION);

procedure PUT
    ( ITEM  : in     STRING;
     STATUS : in     STATUS_INDICATOR  := NO_INDICATION);

procedure GET_LINE
    (FILE   : in     FILE_TYPE;
     ITEM   :    out STRING;
     LAST   :    out CAIS_NATURAL;
     STATUS : in     STATUS_INDICATOR  := NO_INDICATION);

procedure GET_LINE
    (ITEM   :    out STRING;
     LAST   :    out CAIS_NATURAL;
     STATUS : in     STATUS_INDICATOR  := NO_INDICATION);

procedure PUT_LINE
    (FILE   : in     FILE_TYPE;
     ITEM   : in     STRING;
     STATUS : in     STATUS_INDICATOR  := NO_INDICATION);

procedure PUT_LINE
    ( ITEM  : in     STRING;
     STATUS : in     STATUS_INDICATOR  := NO_INDICATION);
```


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
generic
    typo NUM is range <>;
package CAIS_INTEGER_IO is

    DEFAULT_WIDTH : FIELD          := NUM'WIDTH;
    DEFAULT_BASE  : NUMBER_BASE := 10;

    procedure GET
        (FILE   : in     FILE_TYPE;
         ITEM   :    out NUM;
         WIDTH  : in     FIELD             := 0;
         STATUS : in     STATUS_INDICATOR  := NO_INDICATION);

    procedure GET
        (ITEM   :    out NUM;
         WIDTH  : in     FIELD             := 0;
         STATUS : in     STATUS_INDICATOR  := NO_INDICATION);

    procedure PUT
        (FILE   : in     FILE_TYPE;
         ITEM   : in     NUM;
         WIDTH  : in     FIELD             := DEFAULT_WIDTH;
         BASE   : in     NUHBER_BASE       := DEFAULT_BASE;
         STATUS : in     STATUS_INDICATOR  := NO_INDICATION);

    procedure PUT
        ( ITEM  : in     NUM;
         WIDTH  : in     FIELD             := DEFAULT_WIDTH;
         BASE   : in     NUMBER_BASE       := DEFAULT_BASE;
         STATUS : in     STATUS_INDICATOR  := NO_INDICATION);

    procedure GET
        (FROM   : in     STRING;
         ITEM   :    out NUM;
         LAST   :    out  CAIS_POSITIVE;
         STATUS : in     STATUS_INDICATOR  := NO_INDICATION);

    procedure PUT
        (TO     :    out  STRING;
         ITEM   : in     NUM;
         BASE   : in     NUMBER_BASE       := DEFAULT_BASE;
         STATUS : in     STATUS_INDICATOR  := NO_INDICATION);

end  CAIS_INTEGER_IO;
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
generic
    type NUM is digits <>;
package CAIS_FLOAT_IO is

    DEFAULT_FORE : FIELD := 2;
    DEFAULT_AFT : FIELD := HUM'DIGITS - 1;
    DEFAULT_EXP : FIELD := 3;

    procedure GET
        (FILE   : in     FILE_TYPE;
         ITEM   :    out NUM;
         WIDTH  : in     FIELD             := 0;
         STATUS : in     STATUS_INDICATOR  := NO_IMDICATIOH) ;

    procedure GET
        (ITEM   :    out NUM;
         WIDTH  : in     FIELD             := 0;
         STATUS : in     STATUS_INDICATOR  := NO_INDICATION);

    procedure PUT
        (FILE   : in     FILE_TYPE;
         ITEM   : in     NUM;
         FORE   : in     FIELD             := DEFAULT_FORE;
         AFT    : in     FIELD             := DEFAULT_AFT;
         EXP    : in     FIELD             := DEFAULT_EXP;
         STATUS : in     STATUS_INDICATOR  := NO_INDICATION);

    procedure PUT
        (ITEM   : in     NUM;
         FORE   : in     FIELD             := DEFAULT_FORE;
         AFT    : in     FIELD             := DEFAULT_AFT;
         EXP    : in     FIELD             := DEFAULT_EXP;
         STATUS : in     STATUS_INDICATOR  := NO_INDICATION);

    procedure GET
        (FROM   : in     STRING;
         ITEM   :    out NUM;
         LAST   :    out CAIS_POSITIVE;
         STATUS : in     STATUS_INDICATOR  := NO_INDICATION);

    procedure PUT
        (TO     :    out STRING;
         ITEM   : in     NUM;
         AFT    : in     FIELD             := DEFAULT_AFT;
         EXP    : in     FIELD             := DEFAULT_EXP;
         STATUS : in     STATUS_INDICATOR  := NO_INDICATION);

end CAIS_FLOAT_IO;
```


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
generic
    type NUM is delta <>;
package CAIS_FIXED_IO is

    DEFAULT_FORE : FIELD := NUM'FORE;
    DEFAULT_AFT : FIELD := NUM'AFT;
    DEFAULT_EXP    : FIELD := 0;

    procedure GET
        (FILE     : in     FILE_TYPE;
         ITEM     :    out NUM;
         WIDTH    : in     FIELD              := 0;
         STATUS : in     STATUS_INDICATOR := NO_INDICATION);

    procedure GET
        (ITEM     :    out NUM;
         WIDTH  : in     FIELD              := 0;
         STATUS : in     STATUS_INDICATOR := NO_INDICATION);

    procedure PUT
        (FILE     : in     FILE_TYPE;
         ITEM     : in     NUM;
         FORE     : in     FIELD              := DEFAULT_FORE;
         AFT      : in     FIELD              := DEFAULT_AFT;
         EXP      : in     FIELD              := DEFAULT_EXP;
         STATUS : in     STATUS_INDICATOR := NO_INDICATION);

    procedure PUT
        (ITEM     : in     NUM;
         FORE     : in     FIELD              := DEFAULT_FORE;
         AFT      : in     FIELD              := DEFAULT_AFT;
         EXP      : in     FIELD              := DEFAULT_EXP;
         STATUS : in     STATUS_INDICATOR := NO_INDICATION);

    procedure GET
        (FROM     : in     STRING;
         ITEM     :    out NUM;
         LAST     :    out CAIS_POSITIVE;
         STATUS : in     STATUS_INDICATOR := NO_INDICATION);

    procedure PUT
        (TO       :    out STRING;
         ITEM     : in     NUM;
         AFT      : in     FIELD              := DEFAULT_AFT;
         EXP      : in     FIELD              := DEFAULT_EXP;
         STATUS : in     STATUS_INDICATOR := NO_INDICATION);

end CAIS_FIXED_IO;
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
    generic
        type ENUM is (<>);
    package CAIS_ENUMERATION_IO is

        DEFAULT_WIDTH   : FIELD    := 0;
        DEFAULT_SETTING : TYPE_SET := UPPERCASE;

        procedure GET
            (FILE   : in  FILE_TYPE;
             ITEM   : out ENUM;
             STATUS : in  STATUS_INDICATOR := NO_INDICATION);

        procedure GET
            (ITEM   : out ENUM;
             STATUS : in  STATUS_INDICATOR := NO_INDICATION);

        procedure PUT
            (FILE   : in  FILE_TYPE;
             ITEM   : in  ENUM;
             WIDTH  : in  FIELD          := DEFAULT_WIDTH;
             SET    : in  TYPE_SET       := DEFAULT_SETTING;
             STATUS : in  STATUS_INDICATOR := NO_INDICATION);

        procedure PUT
            ( ITEM  : in  ENUM;
             WIDTH  : in  FIELD          := DEFAULT_WIDTH;
             SET    : in  TYPE_SET       := DEFAULT_SETTING;
             STATUS : in  STATUS_INDICATOR := NO_INDICATION);

        procedure GET
            (FROM   : in  STRING;
             ITEM   : out ENUM;
             LAST   : out CAIS_POSITIVE;
             STATUS : in  STATUS_INDICATOR := NO_INDICATION);

        procedure PUT
            (TO     : out STRING;
             ITEM   : in  ENUM;
             SET    : in  TYPE_SET       := DEFAULT_SETTING;
             STATUS : in  STATUS_INDICATOR := NO_INDICATION);

    end CAIS_ENUMERATION_IO;

private

    type FILE_TYPE is (IMPLEMENTATION_DEFINED);
    -- This type should be defined by the implementer.

end CAIS_TEXT_IO;
```


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
with CAIS_STANDARD;
with CAIS_DEFINITIONS;
with CAIS_STATUS_DEFINITIONS;
with CAIS_LIST_MANAGMENT;
with CAIS_DEFINITION_MANAGEMENT;
with CAIS_ACCESS_CONTROL_MANAGEMENT;
with CAIS_RESOURCE_CONTROL;
with CAIS_FILE_NODE_MANAGEMENT;
with CAIS_STRUCTURAL_NODE_MANAGEMENT;
with CAIS_TEXT_IO;

package CAIS_COMMOM_IO is

    use CAIS_STANDARD;
    use CAIS_DEFINITIONS;
    use CAIS_STATUS_DEFINITIONS;
    use CAIS_LIST_MANAGEMENT;
    use CAIS_DEFINITION_MANAGEMENT;
    use CAIS_ACCESS_CONTROL_MANAGEMENT;
    use CAIS_RESOURCE_CONTROL;
    use CAIS_FILE_NODE_MANAGEMENT;
    use CAIS_STRUCTURAL_NODE_MANAGEMENT;
    use CAIS_TEXT_IO;

    type NODE_SET_TYPE is limited private;

    UID_ERROR : exception;

    function INTRODUCED_NODE_NKD
        (STATUS : in      STATUS_INDICATOR  := NO_INDICATION)
        return NKD_TYPE;

    function INTRODUCED_NODE_RD
        (STATUS : in      STATUS_INDICATOR  := NO_INDICATION)
        return RD_TYPE;

    procedure OPEN_NODE_SET
        (NODE_SET :  in out NODE_SET_TYPE;
         STATUS   :  in     STATUS_INDICATOR  := NO_INDICATION);

    procedure INCLUDE_NODE
        (NODE_SET                 : in out NODE_SET_TYPE;
         NODE                     : in     NODE_TYPE;
         INCLUDE_DEFINITION_NODES : in     BOOLEAN          := TRUE;
         EXCLUSIVE_READ_INTENT    : in     BOOLEAN          := TRUE;
         TIME_LIMIT               : in     CAIS_DURATION    := LONG_DELAY;
         STATUS                   : in     STATUS_INDICATOR := NO_INDICATION);

    procedure INCLUDE_TREE
        (NODE_SET                 : in out NODE_SET_TYPE;
         NODE                     : in     NODE_TYPE;
         INCLUDE_DEFINITION_NODES : in     BOOLEAN          := TRUE;
         EXCLUSIVE_READ_INTENT    : in     BOOLEAN          := TRUE;
         TIME_LIMIT               : in     CAIS_DURATION    := LONG_DELAY;
         STATUS                   : in     STATUS_INDICATOR := NO_INDICATION);
```


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
procedure INCLUDE_STRUCTURE
    (NODE_SET                    : in out NODE_SET_TYPE;
     NODE                        : in     NODE_TYPE;
     RELATIONS                   : in     NODE_HANDLE_ARRAY ;
     INCLUDE_DEFINITION_NODES    : in     BOOLEAN             := TRUE;
     EXCLUSIVE_READ_INTENT       : in     BOOLEAN             := TRUE;
     TIME_LIMIT                  : in     CAIS_DURATION       := LONG_DELAY;
     STATUS                      : in     STATUS_INDICATOR    := NO_INDICATION);

procedure TRANSMIT
    (FILE          : in     FILE_TYPE;
     NODE_SET      : in     NODE_SET_TYPE;
     STATUS        : in     STATUS_INDICATOR   := NO_INDICATION);

procedure CLOSE_NODE_SET
    (MODE_SET : in out NODE_SET-TYPE;
     STATUS    : in     STATUS_INDICATOR   := NO_INDICATION);

procedure INTRODUCE-NODES
    (FILE                        : in     FILE-TYPE;
     BASE                        : in     NODE_TYPE;
     KEY                         : in     RELATIONSHIP_KEY          := EMPTY-KEY_LIST;
     RELATION                    : in     RD_TYPE                   := INTRODUCED-NODE-RD;
     UNIQUE-IDENTIFIERS          : in     UNIQUE-IDENTIFIER-LIST    := EMPTY-LIST;
     INCLUDE_CONTENTS            : in     BOOLEAN                   := TRUE;
     ATTRIBUTES                  : in     ATTRIBUTE-LIST            := EMPTY_LIST;
     DISCRETIONARY_ACCESS        : in     DISCRETIONARY-ACCESS_LIS; := ALL-RIGHTS;
     MANDATORY_ACCESS            : in     MANDATORY-ACCESS-LIST     := EMPTY_LIST;
     KIND                        : in     NODE-KIND_NODE_TYPE       := INTRODUCED-NODE_NKD;
     STATUS                      : in     STATUS_INDICATOR          := NO-INDICATION;
     MEDIUM_RESOURCE             : in     MEDIUM-NODE-TYPE          := DEFAULT_MEDIUM);

procedure INTRODUCE_NODES
    (FILE                        : in     FILE_TYPE;
     BASE                        : in     NODE-TYPE;
     KEY                         : in     RELATIONSHIP_KEY          := EMPTY-KEY-LIST;
     RELATION                    : in     RELATION-NAME;
     UNIQUE_IDENTIFIERS          : in     UBTIQUE-IDENTIFIER-LIST   := EMPTY-LIST;
     INCLUDE_CONTENTS            : in     BOOLEAN                   := TRUE;
     ATTRIBUTES                  : in     ATTRIBUTE-LIST            := EMPTY_LIST;
     DISCRETIONARY-ACCESS        : in     DISCRETIONARY-ACCESS-LIST; := ALL-RIGHTS;
     MANDATORY-ACCESS            : in     MANDATORY-ACCESS-LIST     := EMPTY-LIST;
     KIND                        : in     NODE-KIND-NAME            := "INTRODUCED_NODE";
     STATUS                      : in     STATUS-INDICATOR          := NO_INDICATION;
     MEDIUM_RESOURCE             : in     MEDIUM-NODE-TYPE          := DEFAULT-MEDIUM);

procedure INTRODUCE-CONTENTS
    (FILE       : in     FILE-TYPE;
     NODES      : in     NODE_HANDLE_ARRAY;
     STATUS     : in     STATUS-INDICATOR          := NO-INDICATION);

procedure CREATE-VIEW_STATING-UID
    (VIEW                        : in out VIEW-NODE-TYPE;
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
        INTRODUCED_NODE          : in      INTRODUCED_NODE_TYPE;
        BASE                     : in      NODE_TYPE;
        KEY                      : in      RELATIONSHIP_KEY          := EMPTY_KEY_LIST;
        ATTRIBUTES               : in      ATTRIBUTE_LIST            := EMPTY_LIST;
        DISCRETIONARY_ACCESS : in      DISCRETIONARY_ACCESS_LIST  :=  ALL_RIGHTS;
        MANDATORY_ACCESS         : in      MANDATORY_ACCESS_LIST     := EMPTY_LIST;
        KIND                     : in      NODE_KIND_NODE_TYPE       := VIEW_NKD;
        STATUS                   : in      STATUS_INDICATOR          := NO_INDICATION;
        MEDIUM_RESOURCE          : in      MEDIUM_NODE_TYPE          :=  DEFAULT_MEDIUM);

    procedure   CREATE_NODE_KIND_STATING_UID
        (NKI                             : in out NKI_TYPE;
         NKD                             : in out NKD_TYPE;
         NKI_INTRODUCED_NODE             : in     INTRODUCED_NODE_TYPE;
         NKD_INTRODUCED_NODE             : in     INTRODUCED_NODE_TYPE;
         VIEW                            : in     VIEW_NODE_TYPE;
         SPECIALIZATION_OF               : in     NKD_TYPE;
         BASE                            : in     NODE_TYPE;
         LOCAL_NAME                      : in     RELATIONSHIP_KEY          := EMPTY_KEY_LIST;
         KEY                             : in     RELATIONSHIP_KEY          := EMPTY_KEY_LIST;
         SPECIALIZATION_OF_KEY           : in     RELATIONSHIP_KEY          := EMPTY_KEY_LIST;
         GENERALIZATION_OF_KEY           : in     RELATIONSHIP_KEY          := EMPTY_KEY_LIST;
         INTERPRETATION_ATTRIBUTES : in     ATTRIBUTE_LIST            := EMPTY_LIST;
         DEFINITION_ATTRIBUTES           : in     ATTRIBUTE_LIST            := EMPTY_LIST;
         DISCRETIONARY_ACCESS            : in     DISCRETIONARY_ACCESS_LIST  :=  ALL_RIGHTS;
         MANDATORY_ACCESS                : in     MANDATORY_ACCESS_LIST := EMPTY_LIST;
         INTERPRETATION_KIND             : in     NODE_KIND_NODE_TYPE  :=
                                                      NODE_KIND_INTERPRETATION_NKD;
         DEFINITION_KIND                 : in     NODE_KIND_NODE_TYPE  :=
                                                      NODE_KIND_DEFINITION_NKD;
         STATUS                          : in     STATUS_INDICATOR          :=  NO_INDICATION;
         INTERPRETATION_MEDIUM           : in     MEDIUM_NODE_TYPE          := DEFAULT_MEDIUM;
         DEFINITION_MEDIUM               : in     MEDIUM_NODE_TYPE          :=  DEFAULT_MEDIUM);

procedure INPORT_NODE_KIND_STATING_UID
    (NKI                             : in out NKI_TYPE;
     INTRODUCED_NODE                 : in     INTRODUCED_NODE_TYPE;
     VIEW                            : in     VIEW_NODE_TYPE;
     NKD                             : in     NKD_TYPE;
     LOCAL_NAME                      : in     RELATIONSHIP_KEY          := EMPTY_KEY_LIST;
     ATTRIBUTES                      : in     ATTRIBUTE_LIST            := EMPTY_LIST;
     DISCRETIONARY_ACCESS : in     DISCRETIONARY_ACCESS_LIST  :=  ALL_RIGHTS;
     MANDATORY_ACCESS                : in     MANDATORY_ACCESS_LIST := EMPTY_LIST;
     INTERPRETATION_KIND             : in     NODE_KIND_NODE_TYPE       :=
                                                  NODE_KIND_INTERPRETATION_NKD;
     STATUS                          : in     STATUS_INDICATOR          :=  NO_INDICATION;
     MEDIUM_RESOURCE                 : in     MEDIUM_NODE_TYPE          :=  DEFAULT_MEDIUM);

procedure CREATE_RELATION_STATING_UID
    (RI                              : in out RI_TYPE;
     RD                              : in out RD_TYPE;
     RI_INTRODUCED_NODE              : in     INTRODUCED_NODE_TYPE;
     RD_INTRODUCED_NODE              : in     INTRODUCED_NODE_TYPE;
     VIEW                            : in     VIEW_NODE_TYPE;
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
        SPECIALIZATION_OF              : in    RD_TYPE;
        BASE                           : in    MODE_TYPE;
        LOCAL_NAME                     : in    RELATIONSHIP_KEY  := EMPTY_KEY_LIST;
        KEY                            : in    RELATIONSHIP_KEY  := EMPTY_KEY_LIST;
        SPECIALIZATION_OF_KEY          : in    RELATIONSHIP_KEY  := EMPTY_KEY_LIST;
        GENERALIZATION_OF_KEY          : in    RELATIONSHIP_KEY  := EMPTY_KEY_LIST;
        INTERPRETATION_ATTRIBUTES : in ATTRIBUTE_LIST            := EMPTY_LIST;
        DEFINITION_ATTRIBUTES          : in    ATTRIBUTE_LIST    := EMPTY_LIST;
        DISCRETIONARY_ACCESS           : in    DISCRETIONARY_ACCESS_LIST := ALL_RIGHTS;
        MANDATORY_ACCESS               : in    MANDATORY_ACCESS_LIST := EMPTY_LIST;
        INTERPRETATION_KIND            : in    NODE_KIND_NODE_TYPE :=
                                                   RELATION_INTERPRETATION_NKD;
        DEFINITION_KIND                : in    NODE_KIND_NODE_TYPE :=
                                                   RELATION_DEFINITION_NKD ;
        STATUS                         : in    STATUS_INDICATOR  := NO_INDICATION;
        INTERPRETATION_MEDIUM          : in    MEDIUM_NODE_TYPE  := DEFAULT_MEDIUM;
        DEFINITION_MEDIUM              : in    MEDIUM_NODE_TYPE  := DEFAULT_MEDIUM);

    procedure INPORT_RELATION_STATING_UID
        (RI                            : in out RI_TYPE;
        INTRODUCED_NODE                : in    INTRODUCED_NODE_TYPE;
        VIEW                           : in    VIEW_NODE_TYPE;
        RD                             : in    RD_TYPE;
        LOCAL_NAME                     : in    RELATIONSHIP_KEY  := EMPTY_KEY_LIST;
        ATTRIBUTES                     : in    ATTRIBUTE_LIST    := EMPTY_LIST;
        DISCRETIONARY_ACCESS           : in    DISCRETIONARY_ACCESS_LIST := ALL_RIGHTS;
        MANDATORY_ACCESS               : in    MANDATORY_ACCESS_LIST := EMPTY_LIST;
        INTERPRETATION_KIND            : in    NODE_KIND_NODE_TYPE :=
                                                   RELATION_INTERPRETATION_NKD;
        STATUS                         : in    STATUS_INDICATOR  := NO_INDICATION;
        MEDIUM_RESOURCE                : in    MEDIUM_NODE_TYPE  := DEFAULT_MEDIUM);

    procedure  CREATE_BIDIRECTIONAL_RELATION_STATING_UID
        (RI1                                 : in out RI_TYPE;
        RI2                                  : in out RI_TYPE;
        RD1                                  : in out RD_TYPE ;
        RD2                                  : in out RD_TYPE;
        RI1_INTRODUCED_NODE                  : in    INTRODUCED_NODE_TYPE;
        RI2_INTRODUCED_NODE                  : in    INTRODUCED_NODE_TYPE;
        RD1_INTRODUCED_NODE                  : in    INTRODUCED_NODE_TYPE;
        RD2_INTRODUCED_NODE                  : in    INTRODUCED_NODE_TYPE;
        VIEW                                 : in    VIEW_NODE_TYPE;
        SPECIALIZATION_OF1                   : in    RD_TYPE;
        SPECIALIZATION_OF2                   : in    RD_TYPE;
        BASE1                                : in    NODE_TYPE;
        BASE2                                : in    NODE_TYPE;
        LOCAL_NAME1                          : in    RELATIONSHIP_KEY  := EMPTY_KEY_LIST;
        LOCAL_NAME2                          : in    RELATIONSHIP_KEY  := EMPTY_KEY_LIST;
        KEY1                                 : in    RELATIONSHIP_KEY  := EMPTY_KEY_LIST;
        KEY2                                 : in    RELATIONSHIP_KEY  := EMPTY_KEY_LIST;
        SPECIALIZATION_OF1_KEY               : in    RELATIONSHIP_KEY  := EMPTY_KEY_LIST;
        SPECIALIZATION_OF2_KEY               : in    RELATIONSHIP_KEY  := EMPTY_KEY_LIST;
        GENERALIZATION_OF1_KEY               : in    RELATIONSHIP_KEY  := EMPTY_KEY_LIST;
        GENERALIZATION_OF2_KEY               : in    RELATIONSHIP_KEY  := EMPTY_KEY_LIST;
```


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
        INTERPRETATION_ATTRIBUTES1 : in     ATTRIBUTE_LIST      := EMPTY_LIST;
        IIJTERPRETATION_ATTRIBUTES2 : in    ATTRIBUTE_LIST      := EMPTY_LIST;
        DEFINITION_ATTRIBUTESl      :  in   ATTRIBUTE_LIST      := EMPTY_LIST;
        DEFINITION_ATTRIBUTES2      :  in   ATTRIBUTE_LIST      := EMPTY_LIST;
        DISCRETIONARY_ACCESS        :  in   DISCRETIONARY_ACCESS_LIST  :=  ALL_RIGHTS;
        MANDATORY_ACCESS            :  in   MANDATORY_ACCESS_LIST      := EMPTY_LIST;
        INTERPRETATION_KIND1        :  in   NODE_KIND_NODE_TYPE   :=
                                              RELATION_INTERPRETATION_NKD;
        INTERPRETATION_KIND2        :  in   NODE_KIND_NODE_TYPE   :=
                                              RELATION_INTERPRETATION_NKD;
        DEFINITION_KIND1            :  in   NODE_KIND_NODE_TYPE   :=
                                              RELATION_DEFINITION_NKD;
        DEFINITION_KIND2            :  in   NODE_KIND_NODE_TYPE   :=
                                              RELATION_DEFINITION_NKD;
        STATUS                      :  in   STATUS_INDICATOR  :=  NO_INDICATION;
        INTERPRETATION_MEDIUM       :  in   MEDIUM_NODE_TYPE  :=  DEFAULT_MEDIUM;
        DEFINITION_MEDIUM           :  in   MEDIUM_NODE_TYPE  :=  DEFAULT_MEDIUM);

    procedure   CREATE_BIDIRECTIONAL_RELATION_STATING_UID
        (RI                                 : in out RI_TYPE;
         RD                                 : in out RD_TYPE;
         RI_INTRODUCED_NODE                 :  in    INTRODUCED_NODE_TYPE;
         RD_INTRODUCED_N0DE                 :  in    INTRODUCED_NODE_TYPE;
         VIEW                               :  in    VIEW_NODE_TYPE;
         SPECIALIZATION_OF                  :  in    RD_TYPE;
         BASE                               :  in    NODE_TYPE;
         LOCAL_NAME                         :  in    RELATIONSHIP_KEY   := EMPTY_KEY_LIST;
         KEY                                :  in    RELATIONSHIP_KEY   := EMPTY_KEY_LIST;
         SPECIALIZATION_OF_KEY              :  in    RELATIONSHIP_KEY   := EMPTY_KEY_LIST;
         GENERALIZATION_OF_KEY              :  in    RELATIONSHIP_KEY   := EMPTY_KEY_LIST;
         INTERPRETATION_ATTRIBUTES          :  in    ATTRIBUTE_LIST     := EMPTY_LIST;
         DEFINITION_ATTRIBUTES              :  in    ATTRIBUTE_LIST     := EMPTY_LIST;
         DISCRETIONARY_ACCESS               :  in    DISCRETIONARY_ACCESS_LIST  :=  ALL_RIGHTS;
         MANDATORY_ACCESS                   :  in    MANDATORY_ACCESS_LIST      := EMPTY_LIST;
         INTERPRETATION_KIND                :  in    NODE_KIND_NODE_TYPE   :=
                                                       RELATION_INTERPRETATION_NKD;
         DEFINITION_KIND                    :  in    NODE_KIND_NODE_TYPE   :=
                                                       RELATION_DEFINITION_NKD;
         STATUS                             :  in    STATUS_INDICATOR  :=  NO_INDICATION;
         INTERPRETATION_MEDIUM              :  in    MEDIUM_NODE_TYPE  :=  DEFAULT_MEDIUM;
         DEFINITION_MEDIUM                  :  in    MEDIUM_NODE_TYPE  :=  DEFAULT_MEDIUM);

    procedure CREATE_ATTRIBUTE_DEFINITION_STATING_UID
        (AD                                 : in out AD_TYPE;
         INTRODUCED_NODE                    :  in    INTRODUCED_NODE_TYPE;
         VIEW                               :  in    VIEW_NODE_TYPE;
         BASE                               :  in    NODE_TYPE;
         ATTRIBUTE_KIND                     :  in    AKD_TYPE;
         LOCAL_NAME                         :  in    RELATIONSHIP_KEY   :=  EMPTY_KEY_LIST;
         KEY                                :  in    RELATIONSHIP_KEY   :=  EMPTY_KEY_LIST;
         ATTRIBUTE_KIND_KEY                 :  in    RELATIONSHIP_KEY   :=  EMPTY_KEY_LIST;
         ATTRIBUTES                         :  in    ATTRIBUTE_LIST     :=  EMPTY_LIST;
         DISCRETIONARY_ACCESS               : in     DISCRETIONARY_ACCESS_LIST   := ALL_RIGHTS;
         MANDATORY_ACCESS                   :  in    MANDATORY_ACCESS_LIST := EMPTY_LIST;
```



Source: https://assist.dla.mil -- Downloaded: 2026l0-26T07:54Z
Check the source to verify that this is the currentversion before use.

```
        KIND                    : in    NODE_KIND_NODE_TYPE :=
                                            ATTRIBUTE_DEFINITION_NKD;
        STATUS                  : in    STATUS_INDICATOR        := NO_INDICATION;
        MEDIUM_RESOURCE         : in    MEDIUM_NODE_TYPE        := DEFAULT_MEDIUM);

    procedure   CREATE_ATTRIBUTE_KIND_STATING_UID
        (AKD                    : in out AKD_TYPE;
        INTRODUCED_NODE         : in    INTRODUCED_NODE_TYPE;
        VIEW                    : in    VIEW_NODE_TYPE;
        BASE                    : in    NODE_TYPE;
        LOCAL_NAME              : in    RELATIONSHIP_KEY        := EMPTY_KEY_LIST;
        KEY                     : in    RELATIONSHIP_KEY        := EMPTY_KEY_LIST;
        ATTRIBUTES              : in    ATTRIBUTE_LIST          := EMPTY_LIST;
        DISCRETIONARY_ACCESS    : in    DISCRETIONARY_ACCESS_LIST := ALL_RIGHTS;
        MANDATORY_ACCESS        : in    MANDATORY_ACCESS_LIST := EMPTY_LIST;
        KIND                    : in    NODE_KIND_NODE_TYPE :=
                                            NAMED_COMPOSITE_LIST_AKD_NKD;
        STATUS                  : in    STATUS_INDICATOR        := NO_INDICATION;
        MEDIUM_RESOURCE         : in    MEDIUM_N0DE_TYPE        := DEFAULT_MEDIUM);

    procedure CREATE_NODE_STATING_UID
        (NODE                   : in out STRUCTURAL_NODE_TYPE;
        INTRODUCED_NODE         : in    INTRODUCED_NODE_TYPE;
        BASE                    : in    NODE_TYPE;
        KEY                     : in    RELATIONSHIP_KEY        := EMPTY_KEY_LIST;
        RELATION                : in    RD_TYPE;
        INTENT                  : in    INTENT_ARRAY            := (1=>WRITE);
        ATTRIBUTES              : in    ATTRIBUTE_LIST          := EMPTY_LIST;
        DISCRETIONARY_ACCESS    : in    DISCRETIONARY_ACCESS_LIST := ALL_RIGHTS;
        MANDATORY_ACCESS        : in    MANDATORY_ACCESS_LIST := EMPTY_LIST;
        KIND                    : in    NODE_KIND_NODE_TYPE     := STRUCTURAL_NKD;
        STATUS                  : in    STATUS_INDICATOR        := NO_INDICATION;
        MEDIUM_RESOURCE         : in    MEDIUM_NODE_TYPE        := DEFAULT_MEDIUM);

    procedure  CREATE_SECONDARY_STORAGE_NODE_STATING_UID
        (NODE                   : in out SECONDARY_STORAGE_NODE_TYPE;
        INTRODUCED_NODE         : in    INTRODUCED_NODE_TYPE;
        BASE                    : in    NODE_TYPE;
        KEY                     : in    RELATIONSHIP_KEY        := EMPTY_KEY_LIST;
        RELATION                : in    RD_TYPE;
        INTENT                  : in    INTENT_ARRAY            := (1=>WRITE);
        ATTRIBUTES              : in    ATTRIBUTE_LIST          := EMPTY_LIST;
        DISCRETIONARY_ACCESS    : in    DISCRETIONARY_ACCESS_LIST := ALL_RIGHTS;
        MANDATORY_ACCESS        : in    MANDATORY_ACCESS_LIST := EMPTY_LIST;
        KIND                    : in    NODE_KIND_NODE_TYPE := SECONDARY_STORAGE_NKD;
        STATUS                  : in    STATUS_INDICATOR := NO_INDICATION;
        MEDIUM_RESOURCE         : in    MEDIUM_NODE_TYPE := DEFAULT_MEDIUM);

private

    type NODE_SET_TYPE is (IMPLEMENTATION_DEFINED);
    -- This type should be defined by the implementer.

end CAIS_COMMON_IO;
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
with CAIS_STANDARD;
with CAIS_DEFIN1TIONS;
with CAIS_IO_DEFINITIONS;
with CAIS_STATUS_DEFINITIONS;

package CAIS_FORM_TERMINAL_IO is

    use CAIS_STANDARD;
    use CAIS_DEFINITIONS;
    use CAIS_IO_DEFINITIONS;
    use CAIS_STATUS_DEFINITIONS;

    type FILE_TYPE is limited private;

    type FORM_TYPE is limited private;

    FORM_STATUS_ERROR : exception;

    type CHARACTER_ARRAY is array (CHARACTER) of BOOLEAN;

    subtype FUNCTION_KEY_NAME is STRING;

    FUNCTION_KEY_STATUS_ERROR : exception;

    type TERMINAL_POSITION_TYPE is
        record
            ROW    : CAIS_POSITIVE;
            COLUMN : CAIS_POSITIVE;
        end record;

    TERMINAL_POSITION_ERROR : exception;

    type AREA_INTENSITY_KIND is
            (NONE,
             NORMAL,
             HIGH) ;

    type AREA_PROTECTION_KIND is
            (UNPROTECTED,
             PROTECTED);

    type AREA_INPUT_KIND is
            (GRAPHIC_CHARACTERS,
             NUMERICS,
             ALPEABETICS);

    typo AREA_VALUE_KIND is
            (NO_FILL,
             FILL_WITH_ZEROES,
             FILL_WITH_SPACES );
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
subtype PRINTABLE_ CHARACTER is CHARACTER range " .. ASCII.TILDE;

procedure OPEN
    (TERMINAL            : in out FILE_TYPE;
     CHANNEL             : in     CHANNEL_NODE_TYPE;
     OPPOSITE_CLIENT     : in     CLIENT_NODE_TYPE;
     TIME_LIMIT          : in     CAIS_IO_DURATION := LONG_IO_DELAY;
     STATUS              : in     STATUS_INDICATOR := NO_INDICATION);

procedure CLOSE
    (TERMINAL : in out FILE_TYPE;
     STATUS    : in     STATUS_INDICATOR := NO_INDICATION);

function IS_OPEN
    (TERMINAL : in FILE_TYPE;
     STATUS    : in STATUS_INDICATOR := NO_INDICATION)
    return BOOLEAN;

procedure GET_ASSOCIATED_CHANNEL
    (CHANNEL : in out CHANNEL_NODE_TYPE;
     FILE    : in     FILE_TYPE;
     STATUS  : in     STATUS_INDICATOR := NO_INDICATION);

function NUMBER_OF_FUNCTION_KEYS
    (TERMINAL : in FILE_TYPE;
     STATUS    : in STATUS_INDICATOR := NO_INDICATION)
    return CAIS_NATURAL;

function INTERCEPTED_INPUT_CHARACTERS
    (TERMINAL : in FILE_TYPE;
     STATUS    : in STATUS_INDICATOR := NO_INDICATION)
    return CHARACTER_ARRAY;

function INTERCEPTED_OUTPUT_CHARACTERS
    (TERMINAL : in FILE_TYPE;
     STATUS    : in STATUS_INDICATOR := NO_INDICATION)
    return CHARACTER_ARRAY;

procedure CREATE_FORK
    (FORM                              : in out FORK_TYPE;
     ROWS                              : in     CAIS_POSITIVE;
     COLUMNS                           : in     CAIS_POSITIVE;
     AREA_QUALIFIER_REQUIRES_SPACE : in     BOOLEAN;
     STATUS                            : in     STATUS_INDICATOR := NO_INDICATION);

procedure DELETE_FORM
    (FORM   : in out FORM_TYPE;
     STATUS : in     STATUS_INDICATOR := NO_INDICATION);

procedure COPY_FORM
    (FROM   : in     FORM_TYPE;
     TO     : in out FORM_TYPE;
     STATUS : in     STATUS_INDICATOR := NO_INDICATION);
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
procedure DEFINE_QUALIFIED_AREA
    (FORM       : in out FORM_ TYPE;
     INTENSITY  : in     AREA_INTENSITY_KIND := NORMAL;
     PROTECTION : in     AREA_PROTECTION_KIND := PROTECTED;
     INPUT      : in     AREA_INPUT_KIND      := GRAPHIC_CHARACTERS;
     VALUE      : in     AREA_VALUE_KIND      := lNTO_FILL;
     STATUS     : in     STATUS_INDICATOR     := NO_INDICATION);

procedure REMOVE_AREA_QUALIFIER
    (FORM   : in out FORM_TYPE;
     STATUS : in     STATUS_INDICATOR := NO_INDICATION);

procedure SET_ACTIVE_POSITIOM
    (FORM     : in out FORM_TYPE;
     POSITION : in     TERMINAL_POSITION_TYPE;
     STATUS   : in     STATUS_INDICATOR := NO_INDICATION);

function ACTIVE_POSITION
    (FORM   : in FORM_TYPE;
     STATUS : in STATUS_INDICATOR := NO_INDICATION)
    return TERMINAL_POSITION_TYPE;

procedure ADVANCE_TO_QUALIFIED_AREA
    (FORM   : in out FORM_TYPE;
     COUNT  : in     CAIS_POSITIVE := 1;
     STATUS : in     STATUS_INDICATOR := NO_INDICATION);

procedure PUT
    (FORM   : in out FORM_TYPE;
     ITEM   : in     PRINTABLE_CHARACTER;
     STATUS : in     STATUS_INDICATOR := NO_INDICATION);

procedure PUT
    (FORM   : in out FORM_TYPE;
     ITEM   : in     STRING;
     STATUS : in     STATUS_INDICATOR := NO_INDICATION);

procedure ERASE_AREA
    (FORM   : in out FORM_TYPE;
     STATUS : in     STATUS_INDICATOR := NO_INDICATION);

procedure ERASE_FORM
    (FORM   : in out FORM_TYPE;
     STATUS : in     STATUS_INDICATOR := NO_INDICATION);

procedure ACTIVATE
    (TERMINAL : in     FILE_TYPE;
     FORM     : in out FORM_TYPE;
     STATUS   : in     STATUS_INDICATOR := NO_INDICATION);

procedure GET
    (FORM   : in out FORM_TYPE;
     ITEM   :    out PRINTABLE_CHARACTER;
     STATUS : in     STATUS_INDICATOR := NO_INDICATION);
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
    procedure GET
        (FORM   : in out FORM_TYPE ;
         ITEM   :    out STRING;
         STATUS : in     STATUS_INDICATOR := NO_INDICATION);

    function IS_FORM_UPDATED
        (FORM   : in FORM_TYPE;
         STATUS : in STATUS_INDICATOR := NO_INDICATION)
        return BOOLEAN;

    function FUNCTION_KEY_IDENTIFICATION
        (TERMINAL       : in FILE_TYPE;
         KEY_IDENTIFIER : in CAIS_POSITIVE;
         STATUS         : in STATUS_INDICATOR := NO_INDICATION)
        return FUNCTION_KEY_NAME;

    function TERMINATION_KEY
        (FORM   : in FORM_TYPE;
         STATUS : in STATUS_INDICATOR := NO_INDICATION)
        return CAIS_NATURAL;

    function FORM_SIZE
        (FORM   : in FORM_TYPE;
         STATUS : in STATUS_INDICATOR := NO_INDICATION)
        return TERMINAL_POSITION_TYPE;

    function TERMINAL_SIZE
        (TERMINAL : in FILE_TYPE;
         STATUS   : in STATUS_INDICATOR := NO_INDICATION)
        return TERMINAL_POSITION_TYPE;

    function AREA_QUALIFIER_REQUIRES_SPACE
        (FORK   : in FORM_TYPE;
         STATUS : in STATUS_INDICATOR := NO_INDICATION)
        return BOOLEAN;

    function AREA_QUALIFIER_REQUIRES_SPACE
        (TERMINAL : in FILE_TYPE;
         STATUS   : in STATUS_INDICATOR := NO_INDICATION)
        return BOOLEAN;

private

    type FILE_TYPE is (IMPLEMENTATION_DEFINED);
    -- This type should be defined by the implementer.

    type FORM_TYPE is (IMPLEMENTATION_DEFINED);
    -- This type should be defined by the implementor.

end CAIS_FORM_TERMINAL_IO;
```


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
with CAIS_PRAGMATICS;
with CAIS_STANDARD;
with CAIS_DEFINITION;
with CAIS_IO_DEFINITIONS;
with CAIS_STATUS_DEFINITIONS;

package CAIS_TERMINAL_IO is

    use CAIS_STANDARD;
    use CAIS_DEFINITIONS;
    use CAIS_IO_DEFINITIONS;
    use CAIS_STATUS_DEFINITIONS;

    type FILE_TYPE is limited private;

    subtype PHYSICAL_DISPLAY_DIMENSION is
        CAIS_POSITIVE range 1.. CAIS_PRAGMATICS.MAXIMUM_PHYSICAL_DISPLAY_DIMENSION;

    subtype VIRTUAL_DISPLAY_COORDINATE is CAIS_INTEGER;

    type PHYSICAL_DISPLAY_POSITION is
        record
            ROW    : PHYSICAL_DISPLAY_DIMENSION;
            COLUMN : PHYSICAL_DISPLAY_DIMENSION;
        end record;

    typo VIRTUAL_DISPLAY_POSITION is
        record
            ROW    : VIRTUAL_DISPLAY_COORDINATE;
            COLUMN : VIRTUAL_DISPLAY_COORDINATE;
        end record;

    VIRTUAL_DISPLAY_POSITION_ERROR : exception;

    type SCREEN_TYPE is limited private;

    subtype SCREEN_DIMENSION is
        CAIS_POSITIVE range 1.. CAIS_PRAGMATICS.MAXIMUM_SCREEN_DIMENSION;

    type SCREEN_POSITION is
        record
            ROW    : SCREEN_DIMENSION;
            COLUMN : SCREEN_DIMENSION;
        end record;

    SCREEN_POSITION_ERROR : exception;
    SCREEN_NOT_ON_DISPLAY : exception;
    LOCATOR_STATUS_ERROR : exception;

    type GRAPHIC_RENDITION_KIND is
        (PRIMARY_RENDITION, BOLD, FAINT, ITALIC, UNDERSCORE, SLOW_BLINK,
         RAPID_BLINK, REVERSE_IMAGE);
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
type GRAPHIC_RENDITION_SET is array (GRAPHIC_RENDITION_KIND) of BOOLEAN;

type FOREGROUND_COLOR_ TYPE is private;
type BACKGROUND_COLOR_TYPE is private;

BLACK_BACKGROUND     : constant BACKGROUND_COLOR_TYPE;
WHITE_BACKGROUND     : constant BACKGROUND_COLOR_TYPE;
RED_BACKGROUND       : constant BACKGROUND_COLOR_TYPE;
YELLOW_BACKGROUND    : constant BACKGROUND_COLOR_TYPE;
BLUE_BACKGROUND      : constant BACKGROUND_COLOR_TYPE;
GREEN_BACKGROUND     : constant BACKGROUND_COLOR_TYPE;
MAGENTA_BACKGROUND   : constant BACKGROUND_COLOR_TYPE;
CYAN_BACKGROUND      : constant BACKGROUND_COLOR_TYPE;

BLACK_FOREGROUND     : constant FOREGROUND_COLOR_TYPE;
WHITE_FOREGROUND     : constant FOREGROUND_COLOR_TYPE;
RED_FOREGROUND       : constant FOREGROUND_COLOR_TYPE;
YELLOW_FOREGROUND    : constant FOREGROUND_COLOR_TYPE;
BLUE_FOREGROUND      : constant FOREGROUND_COLOR_TYPE;
GREEN_FOREGROUND     : constant FOREGROUND_COLOR_TYPE;
MAGENTA_FOREGROUND   : constant FOREGROUND_COLOR_TYPE;
CYAN_FOREGROUND      : constant FOREGROUND_COLOR_TYPE;

typo RENDITION_TYPE is
    record
        KIND             : GRAPHIC_RENDITION_SET;
        FOREGROUND_COLOR : FOREGROUND_COLOR_TYPE;
        BACKGROUND_COLOR : BACKGROUND_COLOR_TYPE;
    end record;

type SCROLL_TYPE is (UP, DOWN);

type WRAP_TYPE is (ON, OFF);

type ECHO_TYPE is (ON, OFF);

type TERMINATOR_SET is array (CHARACTER) of BOOLEAN;

subtype FUNCTION_KEY_NAME is STRING;

FUNCTION_KEY_STATUS_ERROR : exception;

subtype BUTTON_NAME is STRING;

BUTTON_NAME_ERROR : exception;

typo INPUT_EVENT_KIND is
    (NORMAL_KEY_STROKE, FUNCTION_KEY_STROKE, LOCATOR_M0VEMENT,
     BUTTON_DOWN, BUTTON_UP);

INPUT_EVENT_QUEUE_EMPTY : exception;
```


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
procedure OPEN
    (TERMINAL          : in out FILE_TYPE;
     CHANNEL           : in     CHANNEL_NODE_TYPE;
     OPPOSITE_CLIENT   : in     CLIENT_NODE_TYPE;
     SIZE              :    out PHYSICAL_DISPLAY_POSITION;
     DEFAULT_RENDITION :    out RENDITION_TYPE;
     ORIGIN            : in     VIRTUAL_DISPLAY_POSITION := (1,1);
     TIME_LIMIT        : in     CAIS_IO_DURATION         := LONG_IO_DELAY;
     STATUS            : in     STATUS_INDICATOR         := NO_INDICATION);

procedero CLOSE
    (TERMINAL : in out FILE_TYPE;
     STATUS   : in     STATUS_INDICATOR := NO_INDICATION);

function IS_OPEN
    (TERMINAL : in FILE_TYPE;
     STATUS   : in STATUS_INDICATOR := NO_INDICATION)
    return BOOLEAN;

procedure GET_ASSOCIATED_CHANNEL
    (CHANNEL  : in out CHANNEL_NODE_TYPE;
     TERMINAL : in     FILE_TYPE;
     STATUS   : in     STATUS_INDICATOR := NO_INDICATION);

function PHYSICAL_DISPLAY_ORIGIN
    (TERMINAL : in FILE_TYPE;
     STATUS   : in STATUS_INDICATOR := NO_INDICATION)
    return VIRTUAL_DISPLAY_POSITION;

procedure MOVE_PHYSICAL_DISPLAY_ORIGIN
    (TERMINAL : in FILE_TYPE;
     ORIGIN   : in VIRTUAL_DISPLAY_POSITION;
     STATUS   : in STATUS_INDICATOR := NO_INDICATION);

function GRAPHIC_RENDITION_SET_SUPPORTED
    (TERMINAL : in FILE_TYPE;
     SET      : in GRAPHIC_RENDITION_SET;
     STATUS   : in STATUS_INDICATOR := NO_INDICATION)
    return BOOLEAN;

function COLOR_SUPPORTED
    (TERMINAL : in FILE_TYPE;
     COLOR    : in FOREGROUND_COLOR_TYPE;
     STATUS   : in STATUS_INDICATOR := NO_INDICATION)
    return BOOLEAN;

function COLOR_SUPPORTED
    (TERMINAL : in FILE_TYPE;
     COLOR    : in BACKGROUND_COLOR_TYPE;
     STATUS   : in STATUS_INDICATOR :=NO_INDICATION)
    return BOOLEAN;
```


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
procedure BEGIN_VIRTUAL_DISPLAY_BUFFERING
    (TERMINAL : in FILE_ TYPE;
     STATUS   : in STATUS_INDICATOR := NO_INDICATION);

procedure END_VIRTUAL_DISPLAY_BUFFERING
    (TERMINAL : in FILE_TYPE;
     STATUS   : in STATUS_INDICATOR := NO_INDICATION);

procedure REFRESH_VIRTUAL_DISPLAY
    (TERMIMAL : in FILE_TYPE;
     STATUS   : in STATUS_INDICATOR := NO_INDICATION);

procedure CREATE_SCREEN
    (SCREEN            : in out SCREEN_TYPE;
     SIZE              : in     SCREEN_POSITION;
     ACTIVE_RENDITION  : in     RENDITION_TYPE;
     BORDER_ON         : in     BOOLEAN;
     BORDER_RENDITION  : in     RENDITION_TYPE;
     STATUS            : in     STATUS_INDICATOR := NO_INDICATION);

procedure CHANGE_ACTIVE_RENDITION
    (SCREEN            : in SCREEN_TYPE;
     ACTIVE_RENDITION  : in RENDITION_TYPE;
     STATUS            : in STATUS_INDICATOR := NO_INDICATION);

procedure CHANGE_SCREEN_SIZE
    (SCREEN : in SCREEN_TYPE;
     SIZE   : in SCREEN_POSITION;
     STATUS : in STATUS_INDICATOR := NO_INDICATION);

procedure CHANGE_BORDER
    (SCREEN            : in SCREEN_TYPE;
     BORDER_ON         : in BOOLEAN;
     BORDER_RENDITION  : in RENDITION_TYPE;
     STATUS            : in STATUS_INDICATOR := NO_INDICATION);

procedure COPY_SCREEN
    (SCREEN     : in     SCREEN_TYPE;
     NEW_SCREEN : in out SCREEN_TYPE;
     STATUS     : in     STATUS_INDICATOR := NO_INDICATION);

procedure DELETE_SCREEN
    (SCREEN : in out SCREEN_TYPE;
     STATUS : in     STATUS_INDICATOR := NO_INDICATION);

function SIZE
    (SCREEN : in SCREEN_TYPE;
     STATUS : in STATUS_INDICATOR := NO_INDICATION)
    return SCREEN_POSITION;

function ACTIVE_RENDITION
    (SCREEN : in SCREEN_TYPE;
     STATUS : in STATUS_INDICATOR := NO_INDICATION)
    return RENDITION_TYPE;
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
function BORDER_ON
    (SCREEN : in SCREEN_TYPE;
     STATUS : in STATUS_ INDICATOR := NO_INDICATION)
    return BOOLEAN;

function BORDER_RENDITION
    (SCREEN : in SCREEN_TYPE;
     STATUS : in STATUS_INDICATOR := NO_INDICATION)
    return RENDITION_TYPE;

function IS_SAME
    (SCREENl : in SCREEN_TYPE;
     SCREEN2 : in SCREEN_TYPE;
     STATUS  : in STATUS_INDICATOR := NO_INDICATION)
    return BOOLEAN;

procedure PASTE_SCREEN
    (TERMINAL : in FILE_TYPE;
     SCREEN   : in SCREEN_TYPE;
     POSITION : in VIRTUAL_DISPLAY_POSITION;
     STATUS   : in STATUS_INDICATOR := NO_INDICATION);

procedure UNPASTE_SCREEN
    (SCREEN : in SCREEN_TYPE;
     STATUS : in STATUS_INDICATOR := NO_INDICATION);

procedure MOVE_SCREEN
    (SCREEN : in SCREEN_TYPE;
     POSITION : in VIRTUAL_DISPLAY_POSITION;
     STATUS   : in STATUS_INDICATOR := NO_INDICATION);

procedure MOVE_SCREEN_TO_FRONT
    (SCREEN : in SCREEN_TYPE;
     STATUS : in STATUS_INDICATOR := NO_INDICATION);

procedure MOVE_SCREEN_TO_BACK
    (SCREEN : in SCREEN_TYPE;
     STATUS : in STATUS_INDICATOR := NO_INDICATION);

function PASTE_POSITION
    (SCREEN : in SCREEN_TYPE;
     STATUS : in STATUS_INDICATOR := NO_INDICATION)
    return VIRTUAL_DISPLAY_POSITION;

function SCREEN_OCCLUDED
    (SCREEN : in SCREEN_TYPE;
     STATUS : in STATUS_INDICATOR := DO_INDICATION)
    return BOOLEAN;

procedure SET_CURSOR_POSITION
    (SCREEN   : in SCREEN_TYPE;
     POSITION : in SCREEN_POSITION;
     STATUS   : in STATUS_INDICATOR := NO_INDICATION);
```



Source: https://assist.dla.mil -- Downloaded: 2026l0-26T07:54Z
Check the source to verify that this is the currentversion before use.

```
function CURSOR_ POSITION
    (SCREEN   : in SCREEN_TYPE;
     STATUS :   in STATUS_INDICATOR := NO_INDICATION)
    return  SCREEN_POSITION;

procedure  PUT
    (SCREEN : in  SCREEN_TYPE;
     ITEM      :  in  CHARACTER;
     WRAP      :  in  WRAP_TYPE            := OFF;
     STATUS : in  STATUS_INDICATOR   :=  NO_INDICATION);

procedure  PUT
    (SCREEN : in  SCREEN_TYPE;
     ITEM      :  in  STRING;
     WRAP      :  in  WRAP_TYPE            := OFF;
     STATUS : in  STATUS_INDICATOR   :=  NO_INDICATION);

procedure  NEW_LINE
    (SCREEN :   in SCREEN_TYPE;
     STATUS :   in  STATUS_INDICATOR := NO_INDICATION);

procedure  PUT_LINE
    (SCREEN :   in SCREEN_TYPE;
     ITEM      : in STRING;
     WRAP      : in WRAP_TYPE            := OFF;
     STATUS :   in  STATUS_INDICATOR := NO_INDICATION);

procedure INSERT
    (SCREEN :   in SCREEN_TYPE;
     ITEM      : in CHARACTER;
     STATUS :   in  STATUS_INDICATOR := NO_INDICATION);

procedure INSERT
    (SCREEN :   in SCREEN_TYPE;
     ITEM      : in STRING;
     STATUS :   in  STATUS_INDICATOR := NO_INDICATION);

procedure DELETE_CHARACTERS
    (SCREEN :   in SCREEN_TYPE;
     COUNT    :  in  CAIS_POSITIVE  :=  1;
     STATUS :   in  STATUS_INDICATOR := NO_INDICATION);

procedure  INSERT_LINE
    (SCREEN :   in SCREEN_TYPE;
     COUNT    :  in  CAIS_POSITIVE  :=  1;
     SCROLL :   in  SCROLL_TYPE          := UP;
     STATUS :   in  STATUS_INDICATOR := NO_INDICATION);

procedure  DELETE_LINE
    (SCREEN : in  SCREEN_TYPE;
     COUNT : in  CAIS_POSITIVE  :=  1;
     SCROLL : in  SCROLL_TYPE          := DOWN;
     STATUS : in  STATUS_INDICATOR   :=  NO_INDICATION);
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
procedure ERASE_RANGE
    (SCREEN          : in SCREEN_TYPE;
     START_POSITION  : in SCREEN_POSITION;
     END_POSITION    : in SCREEN_POSITION;
     STATUS          : in STATUS_INDICATOR := NO_INDICATION);


procedure CHANGE_RENDITION
    (SCREEN          : in SCREEN_TTPE;
     START_POSITION  : in SCREEN_POSITION;
     END_POSITION    : in SCREEN_POSITION;
     RENDITION       : in RENDITION_TYPE;
     STATUS          : in STATUS_INDICATOR := NO_INDICATION);


function CELL_RENDITION
    (SCREEN : in SCREEN_TYPE;
     STATUS : in STATUS_INDICATOR := NO_INDICATION)
    return RENDITION_TYPE;


procedure READ_STRING
    (SCREEN : in  SCREEN_TYPE;
     ITEM   :     out STRING;
     LAST   :     out CAIS_NATURAL;
     STATUS : in  STATUS_INDICATOR := NO_INDICATION);


procodure SOUND_BELL
    (SCREEN : in SCREEN_TYPE;
     STATUS : in STATUS_INDICATOR := NO_INDICATION);


procedure BEGIN_SCREEN_BUFFERING
    (SCREEN : in SCREEN_TYPE;
     STATUS : in STATUS_INDICATOR := NO_INDICATION);


procedure END_SCREEN_BUFFERING
    (SCREEN : in SCREEN_TYPE;
     STATUS : in STATUS_INDICATOR := NO_INDICATION);


procedure ENABLE_FUNCTION_KEYS
    (TERMINAL : in FILE_TYPE;
     ENABLE   : in BOOLEAN        := TRUE;
     STATUS   : in STATUS_INDICATOR := NO_INDICATION);


function FUNCTION_KEYS_ARE_ENABLED
    (TERMINAL : in FILE_TYPE;
     STATUS   : in STATUS_INDICATOR := NO_INDICATION)
    return BOOLEAN;


function NUMBER_OF_FUNCTION_KEYS
    (TERMINAL : in FILE_TYPE;
     STATUS   : in STATUS_INDICATOR := NO_INDICATION)
    return CAIS_NATURAL;
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
function FUNCTI0N_KEY_ IDENTIFICATION
    (TERMINAL          : in FILE_ TYPE;
     KEY_IDENTIFIER :  in CAIS_POSITIVE;
     STATUS            : in STATUS_INDICATOR  := NO_INDICATION)
    return  FUNCTION_KEY_NAME;

procodure ENABLE_LOCATOR
    (TERMINAL :  in FILE_TYPE;
     ENABLE     : in BOOLEAN              := TRUE;
     STATUS     : in STATUS_INDICATOR  :=  NO_INDICATION);

function LOCATOR_IS_ENABLED
    (TERMINAL :  in FILE_TYPE;
     STATUS     : in STATUS_INDICATOR  := NO_INDICATION)
    return BOOLEAN;

function   LOCATOR_CURSOR_POSITION
    (TERMINAL : in       FILE_TYPE;
     STATUS     :  in       STATUS_INDICATOR  := NO_INDICATION)
    return   VIRTUAL_DISPLAY_POSITION;

procedure GET_SCREEN_CONTAINING_LOCATOR_CURSOR
    (SCREEN     : in out SCREEN_TYPE;
     TERMINAL : in       FILE_TYPE;
     STATUS     :  in       STATUS_INDICATOR  :=  NO_INDICATION);

function LOCATOR_CURSOR_ON_BORDER
    (SCREEN     :  in       SCREEN_TYPE;
     STATUS     :  in       STATUS_INDICATOR  := NO_INDICATION)
    return BOOLEAN;

procedure SET_LOCATOR_CURSOR_POSITION
    (TERMINAL :  in FILE_TYPE;
     POSITION  : in VIRTUAL_DISPLAY_POSITION;
     STATUS     : in STATUS_INDICATOR  :=  NO_INDICATION);

function  NUMBER_OF_BUTTONS
    (TERMINAL :  in FILE_TYPE;
     STATUS     : in STATUS_INDICATOR := NO_INDICATION)
    return CAIS_NATURAL;

function   BUTTON_IDENTIFICATION
    (TERMINAL               : in FILE_TYPE;
     BUTTON_IDENTIFIER : in CAIS_POSITIVE;
     STATUS                 : in STATUS_INDICATOR := NO_INDICATION)
    return BUTTON_NAME;

procedure  READ_INPUT_EVENT
    (TERMINAL                  :  in       FILE_TYPE;
     KIND                      :       out  INPUT_EVENT_KIND;
     NORMAL_KEY                :       out  CHARACTER;
     FUNCTION_KEY              :       out  CAIS_POSITIVE;
```



Source: https://assist.dla.mil -- Downloaded: 2026l0-26T07:54Z
Check the source to verify that this is the currentversion before use.

```
        LOCATOR_POSITION      :     out  VIRTUAL_DISPLAY_POSITION;
        BUTTON_IDENTIFIER     :     out  CAIS_POSITIVE;
        BLOCKING              : in       BOOLEAN              := FALSE;
        TIME_LIM1T            : in       CAIS_IO_DURATION     := LONG_IO_DELAY;
        STATUS                : in       STATUS_INDICATOR     := NO_INDICATION);

    procedure READ_STRING
        (TERMINAL             : in       FILE_TYPE;
        ITEM                  :     out  STRING;
        LAST                  :     out  CAIS_NATURAL;
        TERMINATOR_KIND       :     out  INPUT_EVENT_KIND;
        RECEIVED_TERMINATOR   :     out  CHARACTER;
        VALID_TERMINATORS     : in       TERMINATOR_SET;
        SCREEN                : in       SCREEN_TYPE;
        ECHO                  : in       ECHO_TYPE            := ON;
        WRAP                  : in       WRAP_TYPE            := OFF;
        STATUS                : in       STATUS_INDICATOR     := NO_INDICATION);

    procedure FLUSH_INPUT_EVENT_QUEUE
        (TERMINAL : in FILE_TYPE;
        STATUS    : in STATUS_INDICATOR := NO_INDICATION);

    function END_OF_FILE
        (TERMINAL : in      FILE_TYPE;
        STATUS    : in      STATUS_INDICATOR  := NO_INDICATION)
          return BOOLEAN;

    function INPUT_QUEUE_EMPTY
        (TERMINAL : in      FILE_TYPE;
        STATUS    : in      STATUS_INDICATOR  := NO_INDICATION)
          return BOOLEAN;

private

    type FILE_TYPE in (IMPLEMENTATION_DEFINED);
    -- This type should be defined by the implementer.

    type SCREEN_TYPE is (IMPLEMENTATION_DEFINED);
    -- This type should be defined by the implementor.

    type FOREGROUND_COLOR_TYPE is (IMPLEMENTATION_DEFINED);
    -- This type should be defined by the implementer.

    type BACKGROUND_COLOR_TYPE is (IMPLEMENTATION_DEFINED);
    -- This type should be defined by the implementer.

    BLACK_BACKGROUND     : constant BACKGROUND_COLOR_TYPE := IMPLEMENTATION_DEFINED;
    WHITE_BACKGROUND     : constant BACKGROUND_COLOR_TYPE := IMPLEMENTATION_DEFINED;
    RED_BACKGROUND       : constant BACKGROUND_COLOR_TYPE := IMPLEMENTATION_DEFINED;
    YELLOW_BACKGROUND    : constant BACKGROUND_COLOR_TYPE := IMPLEMENTATION_DEFINED;
    BLUE_BACKGROUND      : constant BACKGROUND_COLOR_TYPE := IMPLEMENTATION_DEFINED;
    GREEN_BACKGROUND     : constant BACKGROUND_COLOR_TYPE := IMPLEMENTATION_DEFINED;
    MAGENTA_BACKGROUND   : constant BACKGROUND_COLOR_TYPE := IMPLEMENTATION_DEFINED;
    CYAN_BACKGROUND      : constant BACKGROUND_COLOR_TYPE := IMPLEMENTATION_DEFINED;
    -- These constants should be defined by the implementer.
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
    BLACK_FOREGROUND     : constant FOREGROUND_COLOR_TYPE := IMPLEMENTATION_DEFINED;
    WHITE_FOREGROUND     : constant FOREGROUND_COLOR_TYPE  := IMPLEMENTATION_DEFINED;
    RED_FOREGROUND       : constant FOREGROUND_COLOR_TYPE  := IMPLEMENTATION_DEFINED;
    YELLOW_FOREGROUND    : constant FOREGROUMD_COLOR_TYPE  := IMPLEMENTATION_DEFINED;
    BLUE_FOREGROUND      : constant FOREGROUND_COLOR_TYPE  := IMPLEMENTATION_DEFINED;
    GREEN_FOREGROUND     : constant FOREGROUND_COLOR_TYPE  := IMPLEMENTATION_DEFINED;
    MAGENTA_FOREGROUND   : constant FOREGROUND_COLOR_TYPE  := IMPLEMENTATION_DEFINED;
    CYAN_FOREGROUND      : constant FOREGROUND_COLOR_TYPE  := IMPLEMENTATION_DEFINED;
    -- Those constants should be defined by the implementer.

end CAIS_TERMINAL_IO;
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

# C. Compatibility with Pre-existing Standards

The material contained in this appendix is a mandatory part of the standard. 1

This appendix defines additional interfaces within the CAIS packages which are provided for compatibility with previous revisions of the CAIS standard. Some or all of the additional interfaces defined within this appendix may be eliminated in future revisions of the CAIS standard. Reliance on any of the interfaces defined within this appendix may result in incompatibilities with future revisions of the CAIS standard. 2

A CAIS implementation must provide all of the interfaces defined in this appendix with the semantics specified. As in the Additional Interfaces paragraphs of the interface semantic descriptions, Ada is used as a conceptual model for describing the semantics of these additional interfaces. This does not imply that these additional interfaces must be implemented exactly as specified, merely that their behavior is equivalent to such an implementation. 3

Each of the sections of this appendix defines the additional compatibility interfaces for a particular CAIS package as designated within the section title. Each of the subsections of a given section contains all of the compatibility additional interfaces that are overloaded versions of a given principal interface. To facilitate determination of the corresponding principal interface, the subsection titles used are the same as those for the section in which the principal interface is described. 4

## 10 Package CAIS_NODE_MANAGEMENT

### 10.1 Opening a node handle

```
procedure OPEN
    (NODE          : in out NODE_TYPE;
     NAME          : in     PATHNAME;
     INTENT        : in     INTENT_SPECIFICATION ;
     TIME_ LIMIT : in     CAIS_DURATION          := LONG_DELAY;
     STATUS        : in     STATUS_INDICATOR := NO_INDICATION)
is
begin
    OPEN (NODE, NAME, (1 => INTENT), TIME_ LIMIT ,  STATUS);
end OPEN;

procedure OPEN
    (NODE          : in out N0DE_TYPE;
     BASE          : in     NODE_TYPE;
     KEY           : in     RELATIONSHIP_KEY;
     RELATION      : in     RELATION_NAME          := DEFAULT_RELATION;
     INTENT        : in     INTENT_SPECIFICATION;
     TIME_LIM1T : in     CAIS_DURATION          := LONG_DELAY;
     STATUS        : in     STATUS_INDICATOR := NO_INDICATION)
is
begin
    OPEN (NODE, BASE, KEY, RELATION, (1 => INTENT), TIME_LIMIT,
          STATUS);
end OPEN;
```



Source: https://assist.dla.mil -- Downloaded: 2026l0-26T07:54Z
Check the source to verify that this is the currentversion before use.

## 10.2 Changing the intent regarding node handle usage

```
procodure CHANGE_INTENT
    (NODE       : in out NODE_TYPE;
     INTENT     : in     INTENT_SPECIFICATION;
     TIME_LIMIT : in     CAIS_DURATION     := LONG_DELAY;
     STATUS     : in     STATUS_INDICATOR  := NO_INDICATION)
is
begin
    CHANGE_INTENT (NODE, (1 => INTENT), TIME_LIMIT, STATUS);
end CHANGE_INTENT;
```

## 10.3 Querying the existence of a node

```
function IS_OBTAINABLE
    (NAME   : in     PATHNAME;
     STATUS : in     STATUS_INDICATOR  := NO_INDICATION)
    return BOOLEAN
is
    NODE   : NODE_TYPE;
    RESULT : BOOLEAN;
begin
    OPEN (NODE, NAME, (1=>NO_ACCESS), LONG_DELAY, STATUS);
    RESULT := IS_OBTAINABLE (NODE, STATUS);
    CLOSE (NODE);
    return RESULT;
end IS_OBTAINABLE;

function IS_OBTAINABLE
    (BASE     : in     NODE_TYPE;
     KEY      : in     RELATIONSHIP_KEY;
     RELATION : in     RELATION_NAME     := DEFAULT_RELATION;
     STATUS   : in     STATUS_INDICATOR  := NO_INDICATION)
    return BOOLEAN
is
    NODE   : NODE_TYPE;
    RESULT : BOOLEAN;
begin
    OPEN (NODE, BASE, KEY, RELATION, (1=>NO_ACCESS), LONG_DELAY, STATUS);
    RESULT := IS_OBTAINABLE (NODE, STATUS);
    CLOSE (NODE);
    return RESULT;
end IS_OBTAINABLE;
```

## 10.4 Querying sameness

```
function IS_SAME
    (NAME1 : in     PATHNAME;
     NAME2 : in     PATHNAME;
     STATUS : in    STATUS_INDICATOR  := NO_INDICATION)
    return BOOLEAN
is
    NODE1, NODE2 : NODE_TYPE;
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
    RESULT          : BOOLEAN;
begin
    OPEN (NODE1, NAME1, (1=> NO_ACCESS), LONG_DELAY, STATUS);
    begin
        OPEN (NODE2, NAME2, (1=>NO_ACCESS), LONG_DELAY, STATUS);
    exception
        when others =>
            CLOSE (NODE1);
            raise;
    end;
    RESULT  := IS_SAME (NODE1, NODE2, STATUS);
    CLOSE (NODE1);
    CLOSE (NODE2);
    return RESULT;
end IS_SAME;
```

## 10.5 Obtaining an open node handle to the parent

```
procedure OPEN_PARENT
    (PARENT       : in out NODE_TYPE;
     NODE         : in     NODE_TYPE;
     INTENT       : in     INTENT_SPECIFICATION;
     TIME_LIMIT : in     CAIS_DURATION       := LONG_DELAY;
     STATUS       : in     STATUS_INDICATOR := NO_INDICATION)
is
begin
    OPEN_PARENT (PARENT, NODE, (l=>INTENT), TIME_LIMIT, STATUS);
end OPEN_PARENT;
```

## 10.6 Copying a node

```
procedure COPY_NODE
    (FROM                    : in     NODE_TYPE;
     TO                      : in     PATHNAME;
     DISCRETIONARY_ACCESS : in     DISCRETIONARY_ACCESS.LIST := ALL_RIGHTS;
     MANDATORY_ACCESS        : in     MANDATORY_ACCESS_LIST  := EMPTY_LIST;
     STATUS                  : in     STATUS_INDICATOR  := NO_INDICATION;
     MEDIUM_RESOURCE         : in     MEDIUM_NODE_TYPE  := DEFAULT_MEDIUM)
is
    TO_BASE :  NODE_TYPE;
begin
    OPEN (TO_BASE, BASE_PATH(TO), (1=>APPEND_RELATIONSHIPS),
          LONG_DELAY, STATUS);
    COPY_NODE (FROM, TO_BASE, LAST_KEY(TO), LAST_RELATION(TO),
               DISCRETIONARY_ACCESS, MANDATORY_ACCESS,
               STATUS,  MEDIUM_RESOURCE);
    CLOSE (TO_BASE);
exception
    when others =>
        CLOSE (TO_BASE);
        raise;
end COPY_MODE;
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

## 10.7 Copying trees

```
procedure COPY_TREE
    (FROM                  : in  NODE_TYPE;
     TO                    : in  PATHNAME;
     DISCRETIONARY_ACCESS  : in  DISCRETIONARY_ACCESS_LIST := ALL_RIGHTS;
     MANDATORY_ACCESS      : in  MANDATORY_ACCESS_LIST := EMPTY_LIST;
     STATUS                : in  STATUS_INDICATOR := NO_INDICATION;
     MEDIUM_RESOURCE       : in  MEDIUM_NODE_TYPE := DEFAULT_MEDIUM)
is
    TO_BASE : NODE_TYPE;
begin
    OPEN (TO_BASE, BASE_PATH(TO), (1=>APPEND_RELATIONSHIPS),
          LONG_DELAY, STATUS);
    COPY_TREE (FROM, TO_BASE, LAST_KEY(TO), LAST_RELATION(TO),
               DISCRETIONARY_ACCESS, MANDATORY_ACCESS,
               STATUS, MEDIUIM_RESOURCE);
    CLOSE (TO_BASE);
exception
    when others =>
        CLOSE (TO_BASE);
        raise;
end COPY_TREE;
```

## 10.8 Renaming the primary relationship of a node

```
procedure RENAME
    (NODE     : in  NODE_TYPE;
     NEW_NAME : in  PATHNAME;
     STATUS   : in  STATUS_INDICATOR := NO_INDICATION)
is
    NEW_BASE : NODE_TYPE;
begin
    OPEN (NEW_BASE, BASE_PATH(NEW_NAME), (1=>APPEND_RELATIONSHIPS),
          LONG_DELAY, STATUS);
    RENAME (NODE, NEW_BASE, LAST_KEY(NEW_NAME),
            LAST_RELATION(NEW_NAME), STATUS);
    CLOSE (NEW_BASE);
exception
    when others =>
        CLOSE (NEW_BASE);
        raise;
end RENAME;
```

## 10.9 Deleting the primary relationship to a node

```
procedure DELETE_NODE
    (NAME   : in  PATHNAME;
     STATUS : in  STATUS_INDICATOR := NO_INDICATION)
is
    NODE : NODE_TYPE;
begin
    OPEN (NODE, NAME, (EXCLUSIVE_WRITE, READ_RELATIONSHIPS),
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
            LONG_DELAY, STATUS);
    DELETE_NODE (NODE, LONG_DELAY, STATUS);
    CLOSE (MODE);
exception
    when others =>
        CLOSE (MODE);
        raise;
end DELETE_NODE;
```

## 10.10 Deleting the primary relationships of a tree

```
procedure DELETE_TREE
    (NAME   : in    PATHNAME;
     STATUS : in    STATUS_INDICATOR := NO_INDICATION)
is
    NODE : NODE_TYPE;
begin
    OPEN (NODE, NAME, (EXCLUSIVE_WRITE,READ_RELATIOHSHIPS),
            LONG_DELAY, STATUS);
    DELETE_TREE (NODE, STATUS);
    CLOSE (NODE);
exception
    when others =>
        CLOSE (MODE);
        raise;
end DELETE_TREE;
```

## 10.11 Creating secondary relationships

```
procedure CREATE_SECONDARY_RELATIONSHIP
    (TARGET_NODE : in    NODE_TYPE;
     NEW_NAME    : in    PATHNAME;
     INHERITABLE : in    BOOLEAN          := FALSE;
     STATUS      : in    STATUS_INDICATOR := NO_INDICATION)
is
    SOURCE_BASE : MODE_TYPE;
begin
    OPEN (SOURCE_BASE, BASE_PATH(NEW_NAME),
            (1=>APPEND_RELATIONSHIPS), LONG_DELAY, STATUS);
  CREATE_SECONDARY_RELATIONSHIP (TARGET_NODE, SOURCE_BASE,
                                 LAST_KEY(NEW_NAME),
                                 LAST_RELATION(NEW_NAME),
                                 INHERITABLE, EMPTY_LIST,
                                 STATUS);
    CLOSE (SOURCE_BASE);
exception
    when others =>
        CLOSE (SOURCE_BASE);
        raise;
end  CREATE_SECOHDARY_RELATIONSHIP;
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

## 10.12 Deleting secondary relationships

```
procedure DELETE_SECONDARY_RELATIONSHIP
    (NAME   : in  PATHNAME;
     STATUS : in  STATUS_INDICATOR := NO_INDICATION)
is
    BASE : NODE_TYPE;
begin
    OPEN (BASE, BASE_PATH(NAME), (1=>WRITE_RELATIONSHIPS),
          LONG_DELAY, STATUS);
    DELETE_SECONDARY_RELATIONSHIP (BASE, LAST_KEY(NAME),
                                   LAST_RELATION(NAME), STATUS);
    CLOSE (BASE);
exception
    when others =>
        CLOSE (BASE);
        raise;
end DELETE_SECONDARY_RELATIONSHIP;
```

## 10.13 Setting inheritance property of a relationship

```
procedure SET_INHERITANCE
    (NAME        : in  PATHNAME;
     INHERITABLE : in  BOOLEAN;
     STATUS      : in  STATUS_INDICATOR := NO_INDICATION)
is
    BASE : NODE_TYPE;
begin
    OPEN (BASE, BASE_PATH(NAME), (1=>WRITE_RELATIONSHIPS),
          LONG_DELAY, STATUS);
    SET_INHERITANCE (BASE, LAST_KEY(NAME), LAST_RELATION(NAME),
                     INHERITABLE, STATUS);
    CLOSE (BASE);
exception
    when others =>
        CLOSE (BASE);
        raise;
end SET_INHERITANCE;
```

## 10.14 Determining if a secondary relationship is inheritable

```
function IS_INHERITABLE
    (NAME   : in  PATHNAME;
     STATUS : in  STATUS_INDICATOR := NO_INDICATION)
    return BOOLEAN
is
    BASE  : NODE_TYPE;
    VALUE : BOOLEAN;
begin
    OPEN (BASE, BASE_PATH(NAME), (1=>READ_RELATIONSHIPS),
          LONG_DELAY, STATUS);
    VALUE := IS_INHERITABLE (BASE, LAST_KEY(NAME),
                             LAST_RELATION(NAME), STATUS);
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
    CLOSE (BASE);
    return VALUE;
exception
    when others =>
        CLOSE (BASE);
        raise;
end IS_INHERITABLE;
```

## 10.15 Setting the current node relationship

```
procedure SET_CURRENT_NODE
    (NAME          : in    PATHNAME;
     TIME_LIMIT : in    CAIS_DURATION      := LONG_DELAY;
     STATUS        : in    STATUS_INDICATON  := NO_INDICATION)
is
    NODE : NODE_TYPE;
begin
    OPEN (NODE, NAME, (l=>NO_ACCESS), TIME_LIMIT, STATUS);
    SET_CURRENT_NODE (NODE, TIME_LIMIT, STATUS);
    CLOSE (MODE);
exception
    when others =>
        CLOSE (NODE);
        raise;
end SET_CURRENT_NODE;
```

## 10.16 Opening anode handle to the current node

```
procedure GET_CURRENT_NODE
    (NODE          : in out NODE_TYPE;
     INTENT        : in    INTENT_SPECIFICATION;
     TIME_LIMIT : in    CAIS_DURATION          := LONG_DELAY;
     STATUS        : in    STATUS_INDICATOR       := NO_INDICATION)
is
begin
    GET_CURRENT_NODE (NODE, (1=>INTENT), TIME_LIMIT, STATUS);
end GET_CURRENT_NODE;
```

## 10.17 Determining the creation time of a node

```
function TIME_CREATED
    (NAME     : in    PATHNAME;
     STATUS : in    STATUS_INDICATOR  := NO_INDICATION)
    return CAIS_CALENDAR.TIME
is
    NODE     : NODE_TYPE;
    RESULT   : CAIS_CALENDAR_TIME;
begin
    OPEN (NODE, NAME, (1=>READ_ATTRIBUTES), LONG_DELAY, STATUS);
    RESULT := TIME_CREATED (NODE, STATUS);
    CLOSE (NODE);
    return RESULT;
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
exception
    when others =>
        CLOSE (MODE);
        raise;
end TIME_CREATED;
```

## 10.18 Determining the last time a relationship was modified

```
function TIME_RELATIONSHIP_WRITTEN
    (NAME   : in     PATHNAME;
     STATUS : in     STATUS_INDICATOR := NO_INDICATION)
    return CAIS_CALENDAR_TIME
is
    MODE   : NODE_TYPE;
    RESULT : CAIS_CALENDAR_TIME;
begin
    OPEN (NODE, NAME, (1=>READ_ATTRIBUTES), LONG_DELAY, STATUS);
    RESULT := TIME_RELATIONSHIP_WRITTEN (NODE, STATUS);
    CLOSE (NODE);
    return RESULT;
exception
    when others =>
        CLOSE (MODE);
        raise;
end TIME_RELATIONSHIP_WRITTEN;
```

## 10.19 Determining the last time that node contents were written

```
function TIME_CONTENTS_WRITTEN
    (NAME   : in     PATHNAME;
     STATUS : in     STATUS_INDICATOR := NO_INDICATION)
    return CAIS_CALENDAR_TIME
is
    MODE   : NODE_TYPE;
    RESULT : CAIS_CALENDAR_TIME;
begin
    OPEN (NODE, NAME, (1=>READ_ATTRIBUTES), LONG_DELAY, STATUS);
    RESULT := TIME_CONTENTS_WRITTEN (NODE, STATUS);
    CLOSE (MODE);
    return RESULT;
exception
    when others =>
        CLOSE (NODE);
        raise;
end TIME_CONTENTS_WRITTEN;
```

## 10.20 Determining the last time an attribute was modified

```
function TIME_ATTRIBUTE_WRITTEN
    (NAME   : in     PATHNAME;
     STATUS : in     STATUS_INDICATOR := NO_INDICATION)
    return CAIS_CALENDAR_TIME
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
is
    NODE     : NODE_TYPE;
    RESULT :  CAIS_CALENDAR.TIME;
begin
    OPEN (NODE, NAME, (l=>READ_ATTRIBUTES), LONG_DELAY STATUS);
    RESULT :=  TIME_ATTRIBUTE_WRITTEN (NODE, STATUS);
    CLOSE (NODE);
    return  RESULT;
exception
    when  others  =>
        CLOSE (MODE);
        raise;
end  TIME_ATTRIBUTE.WRITTEN;
```

## 10.21 Creating an iterator over nodes

```
procedure  CREATE_ITERATOR
    (ITERATOR            :  in  out  NODE_ITERATOR;
     NAME                :  in       PATHNAME;
     KIND                :  in       NODE_KIND_ARRAY             :=  (FILE,  structural);
     KEY                 :  in       RELATIONSHIP_KEY_PATTERN :=  "*";
     RELATION            :  in       RELATION_NAME_PATTERN    :=  DEFAULT_RELATION;
     KIND_OF_RELATION :  in       RELATIONSHIP_KIND        :=  PRIMARY;
     STATUS              :  in       STATUS_INDICATOR         :=  NO_INDICATION)
is
    NODE :  NODE_TYPE;
begin
    OPEN (NODE,  NAME,  (1=>READ_RELATIONSHIPS),  LONG_DELAY_DELAY,  STATUS
    CREATE_ITERATOR (ITERATOR, NODE, KIND, KEY, RELATION,
                     KIND_OF_RELATION,  STATUS);
    CLOSE (NODE);
exception
    when  others  =>
        CLOSE  (NODE);
        raise;
end CREATE_ITERATOR;
```

## 10.22 Getting the next node in a node iteration

```
procedure  GET_NEXT
    (ITERATOR      : in out NODE_ITERATOR;
     NEXT_NODE     : in out NODE_TYPE;
     INTENT        :  in       INTENT_SPECIFICATION;
     TIME_LIMIT : in       CAIS_DURATION          :=  LONG_DELAY;
     STATUS        :  in       STATUS_INDICATOR       :=  NO_INDICATION)
is
begin
    GET_NEXT(ITERATOR,  NEXT_NODE,  (1=>INTENT),  TIME_LIMIT,  STATUS);
and  GET_NEXT;
```


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

# 20 Package CAIS_ATTRIBUTE_MANAGEMENT

## 20.1 Creating node attributes

```
procedure   CREATE_NODE_ATTRIBUTE
     (NAME          : in       PATHNAME;
      ATTRIBUTE : in       ATTRIBUTE_NAME;
      VALUE         : in       LIST_TYPE ;
      STATUS        : in       STATUS_INDICATOR   := NO_INDICATION)
is
    NODE :   NODE_TYPE;
begin
    OPEN (NODE, NAME, (1=>APPEND_ATTRIBUTES), LONG_DELAY, STATUS);
    CREATE_NODE_ATTRIBUTE (NODE, ATTRIBUTE, VALUE, STATUS);
    CLOSE (NODE);
exception
    when others =>
        CLOSE (NODE);
        raise;
end CREATE_NODE_ATTRIBUTE;
```

## 20.2 Creating path attributes

```
procedure CREATE_PATE_ATTRIBUTE
     (NAME          : in       PATHNAME;
      ATTRIBUTE : in       ATTRIBUTE_NAME;
      VALUE         : in       LIST_TYPE;
      STATUS        : in       STATUS_INDICATOR   := NO_INDICATION)
is
    BASE :   NODE_TYPE;
begin
    OPEN (BASE, BASE_PATH(NAME), (1=>WRITE_RELATIONSHIPS),
            LONG_DELAY, STATUS);
    CREATE_PATH_ATTRIBUTE (BASE, LAST_KEY(NAME), LAST_RELATION(NAME),
                                ATTRIBUTE, VALUE, STATUS);
    CLOSE (BASE);
exception
    when others =>
        CLOSE (BASE);
        raise;
end CREATE_PATH_ATTRIBUTE;
```

## 20.3 Deleting node attributes

```
procedure DELETE_NODE_ATTRIBUTE
     (NAME          : in       PATHNAME;
      ATTRIBUTE : in       ATTRIBUTE_NAME;
      STATUS        : in       STATUS_INDICATOR   := NO_INDICATION)
is
    NODE :   NODE_TYPE;
begin
    OPEN (NODE, NAME, (1=WRITE_ATTRIBUTES), LONG_DELAY, STATUS);
```


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
    DELETE_NODE_ATTRIBUTE (NODE, ATTRIBUTE, STATUS);
    CLOSE (MoDE);
exception
    when others =>
        CLOSE (BODE);
        raise;
end DELETE_NODE_ATTRIBUTE;
```

## 20.4 Deleting path attributes

```
procedure DELETE_PATH_ATTRIBUTE
    (NAME      : in  PATHNAME;
     ATTRIBUTE : in  ATTRIBUTE_NAME;
     STATUS    : in  STATUS_INDICATOR := NO_INDICATION)
is
    BASE : NODE_TYPE;
begin
    OPEN (BASE, BASE_PATH(NAME), (1=>WRITE_RELATIONSHIPS),
          LONG_DELAY, STATUS);
    DELETE_PATH_ATTRIBUTE (BASE, LAST_KEY(NAME), LAST_RELATION(NAME),
                           ATTRIBUTE, STATUS);
    CLOSE (BASE);
exception
    when others =>
        CLOSE (BASE);
        raise;
end DELETE_PATH_ATTRIBUTE;
```

## 20.5 Setting node attributes

```
procedure SET_NODE_ATTRIBUTE
    (NAME      : in  PATHNAME;
     ATTRIBUTE : in  ATTRIBUTE_NAME;
     VALUE     : in  LIST_TYPE;
     STATUS    : in  STATUS_INDICATOR := NO_INDICATION)
is
    NODE : NODE_TYPE;
begin
    OPEN (NODE, NAME, (l=>WRITE_ATTRIBUTES), LONG_DELAYS STATUS);
    SET_NODE_ATTRIBUTE (NODE, ATTRIBUTE, VALUE, STATUS);
    CLOSE (NODE);
exception
    when others =>
        CLOSE (NODE);
        raise;
end SET_NODE_ATTRIBUTE;
```

## 20.6 Setting path attributes

```
procedure SET_PATH_ATTRIBUTE
    (NAME      : in  PATHNAME;
     ATTRIBUTE : in  ATTRIBUTE_NAME;
```



Source: https://assist.dla.mil -- Downloaded: 2026l0-26T07:54Z
Check the source to verify that this is the currentversion before use.

```
        VALUE       : in       LIST_TYPE ;
        STATUS      : in       STATUS_INDICATOR   := NO_INDICATION)
    is
        BASE :  NODE_TYPE;
    begin
        OPEN (BASE, BASE_PATH(NAME), (1=>WRITE_RELATIONSHIPS),
                LONG_DELAY, STATUS);
        SET_PATH_ATTRIBUTE (BASE, LAST_KEY(NAME), LAST_RELATION(NAME),
                                ATTRIBUTE, VALUE, STATUS);
        CLOSE (BASE);
    exception
        when others =>
            CLOSE (BASE);
            raise;
    end SET_PATH_ATTRIBUTE;
```

## 20.7 Getting node attributes

```
procedure GET_NODE_ATTRIBUTE
    (NAME       : in       PATHNAME;
     ATTRIBUTE : in       ATTRIBUTE_NAME;
     VALUE      : in out  LIST_TYPE;
     STATUS     : in       STATUS_INDICATOR   := NO_INDICATION)
is
    NODE  : NODE_TYPE;
begin
    OPEN (NODE, NAME, (1=>READ_ATTRIBUTES), LONG_DELAY, STATUS);
    GET_NODE_ATTRIBUTE (NODE, ATTRIBUTE, VALUE, STATUS);
    CLOSE (NODE);
exception
    when others =>
        CLOSE (NODE);
        raise;
end GET_NODE_ATTRIBUTE;
```

## 20.8 Getting path attributes

```
procedure GET_PATH_ATTRIBUTE
    (NAME       : in       PATHNAME;
     ATTRIBUTE : in       ATTRIBUTE_NAME;
     VALUE      : in out LIST_TYPE;
     STATUS     : in       STATUS_INDICATOR   := NO_INDICATION)
is
    BASE :  NODE_TYPE;
begin
    OPEN (BASE, BASE_PATH(NAME), (1=READ_RELATIONSHIPS),
            LONG_DELAY, STATUS);
    GET_PATH_ATTRIBUTE (BASE, LAST_KEY(NAME), LAST_RELATION(NAME),
                            ATTRIBUTE, VALUE, STATUS);
    CLOSE (BASE);
exception
    when others =>
        CLOSE (BASE);
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
        raise;
end GET_PATH_ATTRIBUTE;
```

## 20.9 Creating an iterator over node attributes

```
procedure  CREATE_NODE_ATTRIBUTE_ITERATOR
    (ITERATOR :  in out ATTRIBUTE_ITERATOR;
     NAME      :  in     PATHNAME;
     PATTERN   :  in     ATTRIBUTE_NAME_PATTERN : = "*";
     SCOPE     :  in     ATTRIBUTE_ITERATOR_SCOPE  :=  VISIBLE_ATTRIBUTES;
     STATUS    :  in     STATUS_INDICATOR          := NO_INDICATION)
is
    NODE :  NODE_TYPE;
begin
    OPEN (NODE, NAME, (l=>READ_ATTRIBUTES), LONG_DELAY, STATUS);
    CREATE_NODE_ATTRIBUTE_ITERATOR (ITERATOR, NODE, PATTERN
                                    SCOPE,  STATUS);
    CLOSE (MODE);
exception
    when  others  =>
        CLOSE  (NODE);
        raise;
end   CREATE_NODE_ATTRIBUTE_ITERATOR;
```

## 20.10 Creating an iterator over relationship attributes

```
procedure  CREATE_PATH_ATTRIBUTE_ITERATOR
    (ITERATOR :  in out ATTRIBUTE_ITERATOR;
     NAME      :  in     PATHNAME;
     PATTERN   :  in     ATTRIBUTE_NAME_PATTERN  :=  "*";
     SCOPE     :  in     ATTRIBUTE_ITERATOR_SCOPE  :=  VISIBLE_ATTRIBUTES;
     STATUS    :  in     STATUS_INDICATOR          := NO_INDICATION)
is
    BASE :  NODE_TYPE;
begin
    OPEN (BASE, BASE_PATH(NAME), (1=>READ_RELATIONSHIPS),
            LONG_DELAY, STATUS);
    CREATE_PATH_ATTRIBUTE_ITERATOR (ITERATOR, BASE, LAST_KEY(NAME),
                                    LAST_RELATION(NAME), PATTERN,
                                    SCOPE,  STATUS);
    CLOSE (BASE);
exception
    when  others  =>
        CLOSE (BASE);
        raise;
end CREATE_PATH_ATTRIBUTE_ITERATOR;
```


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

# 30 Package CAIS-ACCESS-CONTROL-MANAGEMENT

## 30.1 Subtypes

```
subtype GRANT_VALUE is CAIS-DEFINITIONS.DISCRETIONARY_ACCESS-LIST;
```

## 30.2 Setting access control

```
procedure SET_GRANTED_RIGHTS
    (NAME          : in    PATHNAME;
     GROUP_NAME : in    PATHNAME;
     GRANT         : in    DISCRETIONARY_ACCESS-LIST;
     STATUS        : in    STATUS_INDICATOR  := NO_INDICATION)
is
    NODE          : NODE_TYPE;
    GROUP_NODE :  GROUP_NODE_TYPE;
begin
    OPEN (NODE, NAME, (l=>CONTROL), LONG-DELAY, STATUS);
    OPEN (GROUP_NODE, GROUP_NAME, (l=>NO_ACCESS), LONG_DELAY, STATUS);
    SET_GRANTED_RIGHTS (NODE, GROUP_NODE, GRANT, STATUS);
    CLOSE (NODE);
    CLOSE (GROUP_NODE);
exception
    when others =>
        CLOSE (NODE);
        CLOSE (GROUP_NODE);
        raise;
end SET-GRANTED_RIGHTS;
```

## 30.3 Deleting access relationships

```
procedure DELETE-GRANTED_RIGHTS
    (NAME          : in    PATHNAME;
     GROUP_NAME : in    PATHNAME;
     STATUS        : in    STATUS_INDICATOR  := NO_INDICATION)
is
    NODE          NODE_TYPE;
    GROUP_NODE :  GROUP-NODE-TYPE;
begin
    OPEN (NODE, NAME, (1=>CONTROL), LONG_DELAY, STATUS);
    OPEN (GROUP_NODE, GROUP_NAME, (1=>NO_ACCESS), LONG_DELAY, STATUS);
    DELETE_GRANTED_RIGHTS (NODE, GROUP_NODE, STATUS);
    CLOSE (NODE);
    CLOSE (GROUP_NODE);
exception
    when others =>
        CLOSE  (NODE);
        CLOSE (GROUP_NODE);
        raise;
end DELETE_GRANTED_RIGHTS;
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

## 30.4 Obtaining the value of a GRANT attribute

```
procedure GET_GRANTED_RIGHTS
    (NAME       : in     PATENAME;
     GROUP_NAME : in     PATENAME;
     GRANT      : in out DISCRETIONARY_ACCESS_LIST;
     STATUS     : in     STATUS_INDICATOR := NO_INDICATION)
is
    NODE       : NODE_TYPE;
    GROUP_NODE : GROUP_NODE_TYPE;
begin
    OPEN (NODE, NAME, (l=>READ_RELATIONSHIPS), LONG_DELAY, STATUS);
    OPEN (GROUP_NODE, GROUP_NAME, (1=NO_ACCESS LONG_DELAY, STATUS);
    GET_GRANTED_RIGHTS (NODE, GROUP_NODE, GRANT, STATUS);
    CLOSE (NODE);
    CLOSE (GROUP_NODE);
exception
    when others =>
        CLOSE (MODE);
        CLOSE (GROUP_NODE);
        raise;
end GET_GRANTED_RIGHTS;
```

## 30.5 Examining access rights

```
function IS_APPROVED
    (OBJECT_NAME : in     PATHNAME;
     ACCESS_RIGET : in    ACCESS_RIGHTS;
     STATUS       : in    STATUS_INDICATOR := NO_INDICATION)
      return BOOLEAN
is
    OBJECT_NODE : NODE_TYPE;
    RESULT      : BOOLEAN;
begin
    OPEN (OBJECT_NODE, OBJECT_NAME, (1=>READ_RELATIONSHIPS),
          LONG_DELAY, STATUS);
    RESULT := IS_APPROVED (OBJECT_NODE, ACCESS_RIGHT, STATUS);
    CLOSE (OBJECT_NODE);
    return RESULT;
exception
    when others =>
        CLOSE (OBJECT_NODE);
        raise;
end IS_APPROVED;
```

## 30.6 Adopting a role

```
procedure ADOPT_ROLE
    (GROUP_NAME : in     PATENAME;
     KEY        : in     RELATIONSHIP_KEY := LATEST_KEY;
     INHERITABLE : in    BOOLEAN          := TRUE;
     STATUS     : in     STATUS_INDICATOR := NO_INDICATION)
is
```


Source: https://assist.dla.mil -- Downloaded: 2026l0-26T07:54Z
Check the source to verify that this is the currentversion before use.

```
    GROUP_NODE  : GROUP_NODE_TYPE;
begin
    OPEN (GROUP_NODE, GROUP_NAME, (1=>NO_ACCESS), LONG_DELAY, STATUS);
    ADOPT_ROLE (GROUP_NODE, KEY, INHERITABLE, STATUS);
    CLOSE (GROUP_NODE);
exception
    when others =>
        CLOSE (GROUP_NODE);
        raise;
end ADOPT_ROLE;
```

# 40 Package CAIS_STRUCTURAL_NODE_MANAGEMENT

## 40.1 Creating structural nodes

```
procedure CREATE_NODE
    (NODE                    : in out STRUCTURAL_NODE_TYPE;
     NAME                    : in     PATHNAME;
     INTENT                  : in     INTENT_ARRAY              := (1=>WRITE);
     ATTRIBUTES              : in     ATTRIBUTE_LIST            := EMPTY_LIST;
     DISCRETIONARY_ACCESS    : in     DISCRETIONARY_ACCESS_LIST := ALL_RIGHTS;
     MANDATORY_ACCESS        : in     MANDATORY_ACCESS_LIST     := EMPTY_LIST;
     KIND                    : in     NODE_KIND_NAME            := "STRUCTURAL";
     STATUS                  : in     STATUS_INDICATOR          := NO_INDICATION;
     MEDIUM_RESOURCE         : in     MEDIUM_NODE_TYPE          := DEFAULT_MEDIUM)
is
    BASE  : NODE_TYPE;
begin
    OPEN (BASE, BASE_PATH(NAME), (1=>APPEND_RELATIONSHIPS),
          LONG_DELAY, STATUS);
    CREATE_NODE (NODE, BASE, LAST_KEY(NAME), LAST_RELATION(NAME),
                 INTENT, ATTRIBUTES, DISCRETIONARY_ACCESS,
                 MANDATORY_ACCESS, KIND, STATUS, MEDIUM_RESOURCE);
    CLOSE (BASE);
exception
    when others =>
        CLOSE (BASE);
        raise;
end CREATE_NODE;

procedure CREATE_NODE
    (BASE                    : in     NODE_TYPE;
     KEY                     : in     RELATIONSHIP_KEY          := LATEST_KEY;
     RELATION                : in     RELATION_NAME             := DEFAULT_RELATION;
     INTENT                  : in     INTENT_ARRAY              := (l=>WRITE);
     ATTRIBUTES              : in     ATTRIBUTE_LIST            := EMPTY_LIST;
     DISCRETIONARY_ACCESS    : in     DISCRETIONARY_ACCESS_LIST := ALL_RIGHTS;
     MANDATORY_ACCESS        : in     MANDATORY_ACCESS_LIST     := EMPTY_LIST;
     KIND                    : in     NODE_KIND_NAME            := "STRUCTURAL";
     STATUS                  : in     STATUS_INDICATOR          := NO_INDICATION;
     MEDIUM_RESOURCE         : in     MEDIUM_NODE_TYPE          := DEFAULT_MEDIUM)
is
   NODE :  STRUCTURAL_NODE_TYPE;
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
begin
    CREATE_NODE (NODE, BASE, KEY, RELATION, INTENT, ATTRIBUTES,
                 DISCRETIONARY_ACCESS, MANDATORY_ACCESS, KIND,
                 STATUS, MEDIUM_RESOURCE);
    CLOSE (NODE);
end CREATE_NODE;

procedure CREATE_NODE
    (NAME                 : in  PATHNAME;
     INTENT               : in  INTENT_ARRAY               := (1=>WRITE);
     ATTRIBUTES           : in  ATTRIBUTE_LIST             := EMPTY_LIST;
     DISCRETIONARY_ACCESS : in  DISCRETIONARY_ACCESS_LIST  := ALL_RIGHTS;
     MANDATORY_ACCESS     : in  MANDATORY_ACCESS_LIST      := EMPTY_LIST;
     KIND                 : in  NODE_KIND_NAME             := "STRUCTURAL";
     STATUS               : in  STATUS_INDICATOR           := NO_INDICATION;
     MEDIUM_RESOURCE      : in  NODE_TYPE                  := DEFAULT_MEDIUM)
is
    NODE : STRUCTURAL_NODE_TYPE;
begin
    CREATE_NODE (NODE, NAME, INTEND ATTRIBUTES,
                 DISCRETIONARY_ACCESS, MANDATORY_ACCESS,
                 KIND, STATUS, MEDIUM_RESOURCE);
    CLOSE (NODE);
end CREATE_NODE;
```

# 50 Package CAIS_PROCESS_MANAGEMENT

## 50.1 Spawning a new process

```
procedure SPAWN_PROCESS
    (NODE                 : in out PROCESS_NODE_TYPE;
     FILE_NODE            : in  EXECUTABLE_IMAGE_NODE_TYPE;
     INTENT               : in  INTENT_SPECIFICATION  := READ_ATTRIBUTES;
     INPUT_PARAMETERS     : in  PARAMETER_LIST        := EMPTY_LIST;
     KEY                  : in  RELATIONSHIP_KEY      := EMPTY_KEY_LIST;
     RELATION             : in  RELATION_NAME         := DEFAULT_RELATION;
     DISCRETIONARY_ACCESS : in  DISCRETIONARY_ACCESS_LIST := ALL_RIGHTS;
     ATTRIBUTES           : in  ATTRIBUTE_LIST        := EMPTY_LIST;
     INPUT_FILE           : in  PATHNAME              := STANDARD_INPUT;
     OUTPUT_FILE          : in  PATHNAME              := STANDARD_OUTPUT;
     ERROR_FILE           : in  PATHNAME              := STANDARD_ERROR;
     ENVIRONMENT_NODE     : in  PATHNAME              := CURRENT_NODE;
     PROCESS_NODE_KIND    : in  NODE_KIND_NAME        := "".
     ENCLOSING_TRANSACTION: in  TRANSACTION_TYPE      := NULL_TRANSACTION;
     STATUS               : in  STATUS_INDICATOR      := NO_INDICATION;
     PRIORITY             : in  PROCESS_PRIORITY      := DEFAULT_PRIORITY;
     PROCESSOR_RESOURCE   : in  PROCESSOR_NODE_TYPE   := DEFAULT_PROCESSOR;
     MEDIUM_RESOURCE      : in  MEDIUM_NODE_TYPE      := DEFAULT_MEDIUM)
is
     CALLER               : PROCESS_NODE_TYPE;
     RELATION_DEFINITION  : RD_TYPE;
     PROCESS_DEFINITION   : NKI_TYPE;
begin
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
        OPEN (CALLER, CURRENT_PROCESS, (1 => NO_ACCESS), STATUS => STATUS);

        DEFINITION_OF_RELATION (RELATION_DEFINITION, CALLER, RELATION, STATUS);

        if PROCESS_NODE_KIND = "" then
            SPAWN_PROCESS
                (NODE, FILE_NODE, (l=>INTENT), INPUT_PARAMETERS, KEY,
                 RELATION_DEFINITION, DISCRETIONARY_ACCESS,
                 ATTRIBUTES, INPUT_FILE, OUTPUT_FILE, ERROR_FILE,
                 ENVIRONMENT_NODE, NULL_NODE,
                 ENCLOSING_TRANSACTION, STATUS, PRIORITY,
                 PROCESSOR_RESOURCE,    MEDIUM_RESOURCE);
        else
            OPEN_NODE_KIND (PROCESS_DEFINITION, PROCESS_NODE_KIND, STATUS);

            SPAWN_PROCESS
                (NODE, FILE_NODE, (1=>INTENT), INPUT_PARAMETERS, KEY,
                 RELATION_DEFINITION, DISCRETIONARY_ACCESS,
                 ATTRIBUTES, INPUT_FILE, OUTPUT_FILE, ERROR_FILE,
                 ENVIRONMENT_NODE, PROCESS_DEFINITION,
                 ENCLOSING_TRANSACTION, STATUS, PRIORITY,
                 PROCESSOR_RESOURCE,    MEDIUM_RESOURCE);
        end if;

        CLOSE (CALLER);
        CLOSE (RELATION_DEFINITION);
        CLOSE (PROCESS_DEFINITION);
    exception
        when others =>
            CLOSE (CALLER);
            CLOSE (RELATION_DEFINITION);
            CLOSE (PROCESS_DEFINITION);
            raise;
    end SPAWN_PROCESS;
```

## 50.2 Invoking a new process

```
procedure INVOKE_PROCESS
    (NODE                : in out PROCESS_NODE_TYPE;
     FILE_HODE           :  in    EXECUTABLE_IMAGE_NODE_TYPE;
     INTENT              :  in    INTENT_SPECIFICATION;
     RESULTS_RETURNED    : in out RESULTS_LIST;
     STATUS              :    out PROCESS_STATUS_KIND;
     INPUT_PARAMETERS    :  in    PARAMETER_LIST          := EMPTY_LIST;
     KEY                 :  in    RELATIONSHIP_KEY        := EMPTY_KEY_LIST;
     RELATION            :  in    RELATION_NAME           := DEFAULT_RELATION;
     DISCRETIONARY_ACCESS:  in    DISCRETIONARY_ACCESS_LIST := ALL_RIGHTS;
     ATTRIBUTES          :  in    ATTRIBUTE_LIST          := EMPTY_LIST;
     INPUT_FILE          :  in    PATHNAME                := STANDARD_INPUT;
     OUTPUT_FILE         :  in    PATHNAME                := STANDARD_OUTPUT;
     ERROR_FILE          :  in    PATHNAME                := STANDARD_ERROR;
     ENVIRONMENT_NODE    :  in    PATHNAME                := CURRENT_NODE;
     TIME_LIMIT          :  in    CAIS_DURATION           := LONG_DELAY;
```


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
            PROCESS_NODE_KIND        : in    NODE_KIND_NAME  := "",
            ENCLOSING_TRANSACTION    : in    TRANSACTION_TYPE := NULL_TRANSACTION;
            INVOKE_PROCESS_STATUS    : in    STATUS_INDICATOR  := NO_INDICATION;
            PRIORITY                 : in    PROCESS_PRIORITY  := DEFAULT_PRIORITY;
            PROCESSOR_RESOURCE       : in    PROCESSOR_NODE_TYPE  := DEFAULT_PROCESSOR;
            MEDIUM_RESOURCE          : in    MEDIUM_NODE_TYPE := DEFAULT_MEDIUM)
        is
            CALLER                   : PROCESS_NODE_TYPE;
            RELATION_DEFINITION      : RD_TYPE;
            PROCESS_DEFINITION       : NKI_TYPE;
        begin

            OPEN (CALLER, CURRENT_PROCESS, (1 => NO_ACCESS),
                    STATUS => INVOKE_PROCESS_STATUS);

            DEFINITION_OF_RELATION (RELATION_DEFINITION, CALLER, RELATION,
                                    INVOKE_PROCESS_STATUS);




    if PROCESS_NODE_KIND = "" then
        INVOKE_PROCESS
            (NODE, FILE_NODE, (l=>INTENT), RESULTS_RETURNED,
             STATUS, INPUT_PARAMETERS, KEY, RELATION_DEFINITION,
             DISCRETIONARY_ACCESS, ATTRIBUTES, INPUT_FILE,
             OUTPUT_FILE, ERROR_FILE, ENVIRONMENT_NODE, TIME_LIMIT,
             NULL_MODE, ENCLOSING_TRANSACTION,
             INVOKE_PROCESS_STATUS, PRIORITY, PROCESSOR_RESOURCE,
             MEDIUM_RESOURCE);
    else
        OPEN_NODE_KIND (PROCESS_DEFINITION, PROCESS_NODE_KIND,
                        INVOKE_PROCESS_STATUS);

        INVOKE_PROCESS
            (NODE, FILE_NODE, (1=>INTENT), RESULTS_RETURNED,
             STATUS, INPUT_PARAMETERS, KEY, RELATION_DEFINITION,
             DISCRETIONARY_ACCESS, ATTRIBUTES, INPUT_FILE,
             OUTPUT_FILE, ERROR_FILE, ENVIRONMENT_NODE, TIME_LIMIT,
             PROCESS_DEFINITION, ENCLOSING_TRANSACTION,
             INVOKE_PROCESS_STATUS, PRIORITY, PROCESSOR_RESOURCE,
             MEDIUM_RESOURCE);
    and if;

    CLOSE (CALLER);
    CLOSE (RELATION_DEFINITION);
    CLOSE (PROCESS_DEFINITION);
exception
    when others =>
        CLOSE (CALLER);
        CLOSE (RELATION_DEFINITION);
        CLOSE (PROCESS_DEFINITION);
        raise;
end INVOKE_PROCESS;
```


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

## 50.3 Awaiting termination of another process

```
procedure AWAIT_PROCESS_COMPLETION
    (NODE          : in     PROCESS_NODE_TYPE;
     TIME_LIMIT : in     CAIS_DURATION      := LONG_DELAY;
     STATUS        : in     STATUS_INDICATOR  := NO_INDICATION)
    renames AWAIT_PROCESS_TERMINATION;

procedure AWAIT_PROCESS_COMPLETION
    (NODE                 : in     PROCESS_NODE_TYPE;
     RESULTS_RETURNED : in out RESULTS_LIST;
     STATUS               :    out  PROCESS_STATUS_KIND;
     TIME_LIMIT           : in     CAIS_DURATION      := LONG_DELAY;
     AWAIT_STATUS         : in     STATUS_INDICATOR  := NO_INDICATION)
    renames AWAIT_PROCESS_TERMINATION;
```

## 50.4 Deleting a job

```
procedure DELETE_JOB
    (NAME   : in     PATHNAME;
     STATUS : in     STATUS_INDICATOR  := NO_INDICATION)
is
    NODE : PROCESS_NODE_TYPE;
begin
    OPEN (NODE, NAME, (EXCLUSIVE_WRITE, READ_RELATIONSHIPS),
          LONG_DELAY, STATUS);
    DELETE_JOB (NODE,STATUS);
exception
    when others =>
        CLOSE (NODE);
        raise;
end DELETE_JOB;
```

## 50.5 Getting results from a process

```
procedure GET_RESULTS
    (NAME                 : in     PATHNAME;
     RESULTS              : in out RESULTS_LIST;
     STATUS               :    out  PROCESS_STATUS_KIND;
     GET_RESULTS_STATUS : in     STATUS_INDICATOR  := NO_INDICATION)
is
    NODE : PROCESS_NODE_TYPE;
begin
    OPEN (NODE, NAME, (1=>READ_ATTRIBUTES), LONG_DELAY, GET_RESULTS_STATUS);
    GET_RESULTS (NODE, RESULTS, GET_RESULTS_STATUS);
    STATUS := CURRENT_STATUS (NODE, GET_RESULTS_STATUS);
    CLOSE (NODE);
exception
    when others =>
        CLOSE (NODE);
        raise;
end GET_RESULTS;
```


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
procedure GET_RESULTS
    (NAME                 : in     PATHNAME;
     RESULTS              : in out RESULTS_ LIST;
     GET_RESULTS_STATUS : in     STATUS_INDICATOR := NO_INDICATION)
is
    NODE : PROCESS_NODE_TYPE;
begin
    OPEN (NODE, NAME, (l=>READ_ATTRIBUTES), LONG_DELAY, GET_RESULTS_STATUS);
    GET_RESULTS (MODE, RESULTS, GET_RESULTS_STATUS);
    CLOSE (NODE);
exception
    when others =>
        CLOSE (MODE);
        raise;
end GET_RESULTS;
```

## 50.6 Determining the status of a process

```
function CURRENT_STATUS
    (NAME   : in     PATHNAME;
     STATUS : in     STATUS_INDICATOR := NO_INDICATION)
    return PROCESS_STATUS_KIND
is
    NODE   : PROCESS_NODE_TYPE;
    RESULT : PROCESS_STATUS_KIND;
begin
    OPEN (NODE, NAME, (1=>READ_ATTRIBUTES), LONG_DELAY, STATUS);
    RESULT := CURRENT_STATUS (NODE, STATUS);
    CLOSE (MODE);
    return RESULT;
exception
    when others =>
        CLOSE (NODE);
        raise;
end CURRENT_STATUS;
```

## 50.7 Aborting a process

```
procedure ABORT_PROCESS
    (NAME    : in     PATHNAME;
     RESULTS : in     RESULTS_STRING;
     STATUS  : in     STATUS_INDICATOR := NO_INDICATION)
is
    NODE : PROCESS_NODE_TYPE;
begin
    OPEN (NODE, NAME,
          (READ_RELATIONSHIPS, WRITE_COMTENTS, WRITE_ATTRIBUTES),
           LONG_DELAY, STATUS);
    ABORT_PROCESS (NODE, RESULTS, STATUS);
    CLOSE (MODE);
exception
    when others =>
        CLOSE (NODE);
```


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
        raise;
end ABORT_PROCESS;


procedure ABORT_PROCESS
    (NODE   : in     PROCESS_NODE_TYPE;
     STATUS : in     STATUS_INDICATOR := NO_INDICATION)
is
begin
    ABORT_PROCESS (NODE, "ABORTED", STATUS);
end ABORT_PROCESS;


procedure ABORT_PROCESS
    (NAME   : in     PATHNAME;
     STATUS : in     STATUS_INDICATOR := NO_INDICATION)
is
    NODE : PROCESS_NODE_TYPE;
begin
    OPEN (NODE, NAME,
          (READ_RELATIONSHIPS, WRITE_CONTENTS, WRITE_ATTRIBUTES),
           LONG_DELAY, STATUS);
    ABORT_PROCESS (NODE, "ABORTED", STATUS);
    CLOSE (NODE);
exception
    when others =>
        CLOSE (NODE);
        raise;
end ABORT_PROCESS;
```

## 50.8 Suspending a process

```
procedure SUSPEND_PROCESS
    (NAME   : in     PATHNAME;
     STATUS : in     STATUS_INDICATOR := NO_INDICATION)
is
    NODE : PROCESS_NODE_TYPE;
begin
    OPEN (NODE, NAME,
          (READ_RELATIONSHIPS, WRITE_ATTRIBUTES, WRITE_CONTENTS),
           LONG_DELAY, STATUS);
    SUSPEND_PROCESS (NODE, STATUS);
    CLOSE (NODE);
exception
    when others =>
        CLOSE (NODE);
        raise;
end SUSPEND_PROCESS;
```

## 50.9 Resuming a process

```
procedure RESUME_PROCESS
    (NAME   : in     PATHNAME;
     STATUS : in     STATUS_INDICATOR := NO_INDICATION)
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
is
    NODE:  PROCESS_NODE_TYPE;
begin
    OPEN (NODE, NAME,
          (READ_RELATIONSHIPS, WRITE_ATTRIBUTES, WRITE_CONTENTS),
           LONG_DELAY, STATUS);
    RESUME_PROCESS (NODE, STATUS);
    CLOSE (NODE);
exception
    when others =>
        CLOSE (MODE);
        raise;
end RESUME_PROCESS;
```

## 50.10 Determining the number of open node handles

```
function OPEN_HANDLE_COUNT
    (NAME     : in      PATHNAME;
     STATUS : in      STATUS_INDICATOR  := NO_INDICATION)
    return CAIS_NATURAL
is
    NODE     : PROCESS_NODE_TYPE;
    RESULT :  CAIS_NATURAL;
begin
    OPEN (NODE, NAME, (1 => READ_ATTRIBUTES), LONG_DELAY, STATUS);
    RESULT  := OPEN_HANDLE_COUNT (NODE, STATUS);
    CLOSE (NODE);
    return RESULT;
exception
    when others =>
        CLOSE (NODE);
        raise;
end OPEN_HANDLE_COUNT;
```

## 50.11 Determining the number of input and output units used

```
function IO_UNIT_COUNT
    (NAME     : in      PATHNAME;
     STATUS : in      STATUS_INDICATOR  := NO_INDICATION)
    return CAIS_NATURAL
is
    NODE     : PROCESS_NODE_TYPE;
    RESULT :  CAIS_NATURAL;
begin
    OPEN (NODE, NAME, (1  => READ_ATTRIBUTES), LONG_DELAY, STATUS);
    RESULT  := IO_UNIT_COUNT (NODE, STATUS);
    CLOSE (NODE);
    return RESULT;
exception
    when others =>
        CLOSE (NODE);
        raise;
end IO_UNIT_COUNT;
```


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

## 50.12 Determining the time of activation

```
function TIME. STARTED
    (NAME      :  in        PATHNAME;
     STATUS : in        STATUS_INDICATOR   := NO_INDICATION)
    return  CAIS_CALENDAR.TIME
is
    NODE      : PROCESS_NODE_TYPE;
    RESULT :   CAIS_CALENDAR.TIME;
begin
    OPEN (NODE, NAME, (1 => READ_ATTRIBUTES), LONG_DELAY, STATUS);
    RESULT   := TIME_STARTED (NODE, STATUS);
    CLOSE (NODE);
    return  RESULT;
exception
    when  others  =>
        CLOSE (NODE);
        raise;
end  TIME_STARTED;
```

## 50.13 Determining the time of termination

```
function TIME_FINISHED
    (NAME      :  in        PATHNAME;
     STATUS : in        STATUS_INDICATOR   := NO_INDICATION)
    return  CAIS_CALENDAR.TIME
is
    NODE      : PROCESS_NODE_TYPE;
    RESULT   :  CAIS_CALENDAR.TIME;
begin
    OPEN (NODE, NAME, (1 => READ_ATTRIBUTES), LONG_DELAY, STATUS);
    RESULT   := TIME_FINISHED (NODE, STATUS);
    CLOSE (NODE);
    return  RESULT;
exception
    when  others  =>
        CLOSE  (NODE);
        raise;
end  TIME_FINISHED;
```

## 50.14 Determining the time a process has been active

```
function   MACHINE_TIME
    (NAME      :  in        PATHNAME;
     STATUS : in        STATUS_INDICATOR   :=  NO_INDICATION)
    return CAIS_DURATION
is
    NODE      : PROCESS_NODE_TYPE;
    RESULT :   CAIS_DURATION;
begin
    OPEN (NODE, NAME, (1 => READ_ATTRIBUTES), LONG_DELAY, STATUS);
    RESULT   := MACHINE_TIME (NODE, STATUS);
    CLOSE  (NODE);
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
    return RESULT;
exception
    when others =>
        CLOSE (NODE);
        raise;
end MACHINE_TIME;
```

## 50.15 Determining the size of a process

```
function PROCESS_SIZE
    (NAME   : in  PATHNAME;
     STATUS : in  STATUS_INDICATOR := NO_INDICATION)
    return CAIS_NATURAL
is
    NODE   : PROCESS_NODE_TYPE;
    RESULT : CAIS_NATURAL;
begin
    OPEN (NODE, NAME, (1 => READ_ATTRIBUTES), LONG_DELAY, STATUS);
    RESULT := PROCESS_SIZE (NODE, STATUS);
    CLOSE (NODE);
    return RESULT;
exception
    when others =>
        CLOSE (NODE);
        raise;
end PROCESS_SIZE;
```

# 60 Package CAIS_IMPORT_EXPORT

## 60.1 Importing a file from the host file system

```
procedure IMPORT_CONTENTS
    (FROM            : in  STRING;
     TO              : in  PATHNAME;
     CHARACTERISTICS : in  LIST_TYPE           := EMPTY_LIST;
     STATUS          : in  STATUS_INDICATOR    := NO_INDICATION)
is
    TO_NODE : SECONDARY_STORAGE_NODE_TYPE;
begin
    OPEN (TO_NODE, TO, (1 => WRITE_CONTENTS), LONG_DELAY, STATUS);
    IMPORT_CONTENTS (FROM, TO_NODE, CHARACTERISTICS, STATUS);
    CLOSE (TO_NODE);
exception
    when others =>
        CLOSE (TO_NODE);
        raise;
end IMPORT_CONTENTS;
```

## 60.2 Exporting a file to the host file system

```
procedure EXPORT_CONTENTS
```


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
    (FROM                : in    PATHNAME;
     TO                  : in    STRING;
     CHARACTERISTICS : in    LIST_TYPE          := EMPTY_LIST;
     STATUS              : in    STATUS_ INDICATOR  := NO_ INDICATION)
is
    FROM_NODE :  SECONDARY_STORAGE_NODE_TYPE;
begin
    OPEN (FROM_NODE, FROM, (READ_ATTRIBUTES, READ_CONTENTS), LONG_DELAY, STATUS);
    EXPORT_CONTENTS (FROM_NODE, TO, CHARACTERISTICS, STATUS);
    CLOSE  (FROM_NODE);
exception
    when others =>
        CLOSE  (FROM_NODE);
        raise;
end EXPORT_CONTENTS;
```

# 70 Package CAIS_FORM_TERMINAL_IO

This package provides the functionality of a form terminal. A form terminal consists of a single device (inasmuch as a programmer is concerned). 1

The scenario for usage of a form terminal has two active agents: a process and a user. Each interaction with the form terminal consists of a three-step sequence. First, the process creates and writes a form to the terminal. Second, the user modifies the form. Third, the process reads the modified form. 2

A form is a two-dimensional matrix of character positions, i.e., places on a form where printable ASCII characters may be displayed. The rows of a form are indexed by positive numbers starting with row one at the top of the display. The columns of a form are indexed by positive numbers starting with column one at the left side of the form. The position identified by row one, column one, is called the start position of the form. The position with the highest row and column index is called the end position of the form. 3

The active position on a form is the position at which the next operation will be performed. The active position is said to advance if (1) the row number of the new position is greater than the row number of the old position or (2) the row number of the new position is the same as the row number of the old position and the new position has a greater column number. Similarly, a position is said to precede the active position if (1) the row number of the position is less than the row number of the active position or (2) the row number of the position is the same as the row number of the active position and the column number of the position is smaller than the column number of the active position. 4

A form is divided into qualified areas. A qualified area identifies a contiguous group of positions that share a common set of characteristics. A qualified area begins at the position designated by an area qualifier and ends either at the end position of the form or at the position preceding the next area qualifier toward the end of the form. Qualification of positions preceding the fist area qualifier on a form is implementation-dependent. The area qualifier at the beginning of an area defies the set of characteristics for that area. Depending on the form, the position of the area qualifier may or may not be considered to be in a qualified area. The characteristics of a qualified area consist of such things as protection (from modification by the user), display renditions (e.g., intensity), and permissible values (e.g., numeric only, alphabetic only). Each position in a qualified area contains a single printable ASCII character. 5



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

## 70.1 Protocol parameters

The IO connection model allows protocol-specific parameters to be determined during the establishment of a channel (see Section 4.5.3 on page 67). There are no protocol-specific parameters for CAIS_FORM_TERMINAL_IO. 1

## 70.2 Types, subtypes, and exceptions

```
type FILE_TYPE is limited private;

type FORM_TYPE is limited private;
```

FILE_TYPE describes the type for file handles. FORM_TYPE describes characteristics of forms. 1

```
FORM_STATUS_ERROR : exception;
```

FORM_STATUS_ERROR is raised if a form already exists (was created) when it should not exist or if a form does not exist (has not been created) when it should exist. 2

```
type CHARACTER_ARRAY is array (CHARACTER) of BOOLEAN;
```

CHARACTER_ARRAY is used to determine the characters that are intercepted due to the characteristics of the underlying operating system and the individual terminal. 3

```
subtype FUNCTION_KEY_NAME is STRING;

FUNCTION_KEY_STATUS_ERROR : exception;
```

FUNCTION_KEY_NAME is used to identify function keys by string representations. FUNCTION_KEY_STATUS_ERROR is raised if a non-existent function key is referenced. 4

```
type TERMINAL_POSITION_TYPE is
   record
      ROW    : CAIS_POSITIVE;
      COLUMN : CAIS_POSITIVE;
   end record;

TERMINAL_POSITION_ERROR : exception;
```

TERMINAL_POSITION_TYPE describes the type of a position on a terminal. TERMINAL_POSITION_ERROR is raised if the active position is inappropriate for the operation. 5

```
type AREA_INTENSITY_KIND is
        (NONE,
         NORMAL,
         HIGH);
```


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
type AREA_PROTECTION_KIND is
        (UNPROTECTED,
         PROTECTED);

type AREA_INPUT_KIND is
        (GRAPHIC_CHARACTERS,
         NUMERICS,
         ALPHABETIC);

type AREA_VALUE_KIND is
        (NO_FILL,
         FILL_WITH_ZEROES ,
         FILL_WITH_SPACES);
```

AREA_INTENSITY_KIND indicates the intensity at which the characters in the area should be displayed; NONE indicates that characters are not displayed. AREA_PROTECTION_KIND specifies whether the user can modify the contents of the area when the form has been activated. AREA_INPUT_KIND specifies the valid characters that may reentered by the user; GRAPHIC_CHARACTERS indicates that any printable character may be entered. AREA_VALUE_KIND indicates the initial value that the area should have when activated; NO_FILL indicates that the value will be specified by a PUT statement. 6

```
subtype PRINTABLE_CHARACTER is CHARACTER range ' ' . . ASCII.TILDE;
```

PRINTABLE_CHARACTER describes the characters that can be output to a terminal. 7

## 70.3 Opening a form terminal file handle

```
procedure OPEN
    (TERMINAL          : in out FILE_TYPE;
     CHANNEL           : in     CHANNEL_NODE_TYPE;
     OPPOSITE_CLIENT   : in     CLIENT_NODE_TYPE;
     TIME_LIMIT        : in     CAIS_IO_DURATION := LONG_IO_DELAY;
     STATUS            : in     STATUS_INDICATOR  := NO_INDICATION);
```

Purpose:

This procedure opens a form file handle for communicating over the channel identified by CHANNEL with the node identified by OPPOSITE_CLIENT. CAIS_FORM_TERMINAL_IO must be an allowed protocol of the CHANNEL node identified by CHANNEL. 1

If the operation concludes successfully, the CONTROLLING_PROTOCOL attribute on the CHANNEL node is set to CAIS_FORM_TERMINAL_IO and the channel is in an ESTABLISHED state. 2



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Parameters:

**TERMINAL** is a file handle, initially closed, to be opened. 3

**CHANNEL** is an open node handle to a CHANNEL node to which both the current process and OPPOSITE. CLIENT are client nodes. 4

**OPPOSITE_CLIENT** is an open node handle to the node to be accessed. 5

**TIME_LIMIT** is the time limit for waiting for the OPEN to complete. 6

**STATUS** is an established status indicator. 7

Exceptions:

**STATUS_ERROR** is raised if TERMINAL is an open file handle [7], if OPPOSITE_CLIENT is not an open node handle [2], if CHANNEL is not an open node handle [2], if the channel denoted by CHANNEL is not in the CONNECTED state [9], or if STATUS is not an established status indicator [3]. 8

**NODE_KIND_ERROR** is raised if the node identified by CHANNEL is not a CHANNEL node [1]. 9

**CLIENT_ERROR** is raised if the current process [1] or the node identified by OPPOSITE_-CLIENT [5] is not a client node of the channel identified by the CHANNEL parameter. 10

**PORT_ERROR** is raised if the DIRECTION attributes of the PORT relationships are missing or have incompatible values [4]. 11

**PROTOCOL_ERROR** is raised if CAIS_FORM_TERMINAL_IO is not an allowed protocol of both PORT relationships of the channel [1], or if the CONTROLLING_PROTOCOL attribute has a value other than NO_PROTOCOL or CAIS_FORM_TERMINAL_IO "[2]. 12

**MODE_ERROR** is raised if the implied mode INOUT_FILE is incompatible with the DIRECTION attribute of the local PORT relationship to the channel identified by CHANNEL as determined by Table XXXIV on page 581 [2]. 13

**USE_ERROR** is raised if the opposite client node of the channel is a SECONDARY_STORAGE or DEVICE node and the implementation can determine that the structure of the content is incompatible with the protocol according to implementation-defined criteria [54], of if the implementation does not support data transfer between the types of clients involved [55], or if another open file handle identifies the same external file and the CAIS implementation does not support the existence of multiple file handles identifying the same external file [41]. Any such restriction must be documented in Appendix F. An implementation is allowed to raise this exception only if it is based on operating system support that does not provide this capability. 14

**TRANSACTION_ERROR** is raised if the operation is included in more than one transaction [1] or if the operation is included in a transaction and the operation modifies a node which is not included in the same transaction [2]. 15

**INTENT_VIOLATION** is raised if the node handle OPPOSITE_CLIENT was not opened with an intent specification including at least the intents required for the implied mode INOUT_FILE as determined by Table XXXIV on page 581 [1]. 16

**OPEN_DENIED** is raised if a remote client process or the opposite client, if it is a process, deleted the the channel, signaling the denial of the connection attempt [1]. 17



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

**TIME_EXPIRED** is raised if the OPEN operation is delayed beyond the specified time limit for reasons other than a lock conflict [1]. 18

**SECURITY_VIOLATION** is raised if the operation represents a violation of mandatory access controls. SECURITY_VIOLATION is raised only if the conditions for other exceptions are not present. 19

Notes:

Closing the node handle OPPOSITE_CLIENT associated with the file handle TERMINAL closes the file handle. 20

## 70.4 Closing a form terminal file handle

```
procedure CLOSE
    (TERMINAL : in out FILE_TYPE;
     STATUS   : in STATUS_INDICATOR := NO_INDICATION);
```

Purpose:

This procedure severs the association between the internal terminal file denoted by the terminal file handle TERMINAL and its associated channel. The terminal file handle becomes closed. Closing an already closed terminal file handle has no effect. 1

The state of the channel associated with TERMINAL becomes CLOSED if presently ESTABLISHED and becomes CONNECTED if already CLOSED. If the opposite client node is a file, the channel becomes immediately CONNECTED. If the channel becomes CONNECTED, the CONTROLLING_ PROTOCOL attribute of the CHANNEL node is set to NO_ PROTOCOL. 2

Parameters:

**TERMINAL** is a terminal file handle to be closed. 3

**STATUS** is an established status indicator. 4

Exceptions:

**TRANSACTION_ERROR** is raised if the operation is included in a transaction and the file handle TERMINAL was not opened inside the transaction. 6

**STATUS_ERROR** is raised if STATUS is not an established status indicator [3]. 6

## 70.5 Determining whether a file handle is open

```
function IS_OPEN
    (TERMINAL : in FILE_TYPE;
     STATUS   : in STATUS_INDICATOR : = NO_INDICATION)
    return BOOLEAN;
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Purpose:

This function returns TRUE if the file handle is open; otherwise, it returns FALSE. 1

Parameters:

**TERMINAL** is a file handle. 2

Exceptions:

**STATUS_ERROR** is raised if STATUS is not an established status indicator [3]. 3

## 70.6 Determining the channel node associated with a file

```
procedure GET_ASSOCIATED_CHANNEL
    (CHANNEL : in out CHANNEL_NODE_TYPE;
     FILE    : in     FILE_TYPE;
     STATUS  : in     STATUS_INDICATOR := NO_INDICATION);
```

Purpose:

This procedure returns a node handle, with intent NO_ACCESS, to the CHANNEL node associated with the file handle FILE. Note that the association was made when the file was initially opened. 1

The last relationship traversed in opening the node handle CHANNEL is the same as that of the CHANNEL node handle provided when the file handle FILE was opened. See subprograms PATH_KEY (Section 5.2.20 on page 119) and PATH_RELATION (Section 5.2.21 on page 120). 2

Parameters:

**CHANNEL** is a node handle to be opened to the associated CHANNEL node. 3

**FILE** is an open file handle identifying the internal file whose associated CHANNEL node is required. 4

**STATUS** is an established status indicator. 5

Exceptions:

**STATUS_ERROR** is raised if FILE is not an open file handle [8], if CHANNEL is open at the time of the call [1] or if STATUS is not an established status indicator [3]. 6

**SECURITY_VIOLATION** is raised if the operation represents a violation of mandatory access controls. SECURITY_VIOLATION is raised only if the conditions for other exceptions are not present. 7

## 70.7 Determining the number of function keys

```
function NUMBER_OF_FUNCTION_KEYS
    (TERMINAL : in FILE_TYPE;
     STATUS   : in STATUS_INDICATOR := NO_INDICATION)
    return CAIS_NATURAL;
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Purpose:

This function returns the number of function keys defined for the terminal associated with the internal file identified by the file handle TERMINAL. 1

Parameters:

**TERMINAL** is an open file handle identifying the internal file associated with the terminal file. 2

**STATUS** is an established status indicator. 3

Exceptions:

**STATUS_ERROR** is raised if TERMINAL is not an open terminal file handle [8] or if STATUS is not an established status indicator [3]. 4

## 70.8 Determining intercepted input characters

```
function INTERCEPTED_INPUT_CHARACTERS
    (TERMINAL : in FILE_TYPE;
     STATUS   : in STATUS_INDICATOR := NO_INDICATION)
    return CHARACTER_ARRAY;
```

Purpose:

This function returns an array of type CHARACTER_ARRAY that indicates the input characters that can never appear in a form activated on the terminal file handle TERMINAL due to characteristics of the underlying system and the individual terminal. A value of FALSE indicates that the input character can appear; a value of TRUE indicates that it cannot appear. 1

Parameters:

TERMINAL is an open file handle identifying the internal file associated with the terminal file. 2

STATUS is an established status indicator. 3

Exceptions:

STATUS_ERROR is raised if TERMINAL is not an open file handle [8] or if STATUS is not an established status indicator [3]. 4

## 70.9 Determining intercepted output characters

```
function INTERCEPTED_OUTPUT_CHARACTERS
    (TERMINAL : in FILE_TYPE;
     STATUS   : in STATUS_INDICATOR := NO_INDICATION)
    return CHARACTER_ARRAY;
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Purpose:

This function returns an array of type CHARACTER_ARRAY that indicates the output characters that can never appear in a form activated on the terminal file handle TERMINAL due to characteristics of the underlying system and the individual terminal. A value of FALSE indicates that the output character can appear; a value of TRUE indicates that it cannot appear. 1

Parameters:

**TERMINAL** is an open file handle identifying the internal file associated with the terminal file. 2

**STATUS** is an established status indicator. 3

Exceptions:

**STATUS_ERROR** is raised if TERMINAL is not an open file handle [8] or if STATUS is not an established status indicator [3]. 4

## 70.10 Creating a form

```
procedure CREATE_FORM
    (FORM                              : in out FORM_TYPE;
     ROWS                              : in     CAIS_POSITIVE;
     COLUMNS                           : in     CAIS_POSITIVE;
     AREA_QUALIFIER_REQUIRES_SPACE     : in     BOOLEAN;
     STATUS                            : in     STATUS_INDICATOR  := NO_INDICATION) ;
```

Purpose:

This procedure creates a form. The form is erased (see Section 70.20 on page 1359).

Parameters:

**FORM** is the form created. 2

**ROWS** is the number of rows in the form. 3

**COLUMNS** is the number of columns in the form. 4

**AREA_QUALIFIER_REQUIRES_SPACE** indicates whether or not an area qualifier requires space on the form. 5

**STATUS** is an established status indicator. 6

Exceptions:

**STATUS_ERROR** is raised if STATUS is not an established status indicator [3].

**FORM_STATUS_ERROR** is raised if the form FORM exists (was created) [2].



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

## 70.11 Deleting a form

```
procedure DELETE_FORM
    (FORM   : in out FORM_TYPE;
     STATUS : in     STATUS_INDICATOR := NO_INDICATION);
```

Purpose:

This procedure deletes a form. The value of its parameter after the call is as if it were never created by the procedure CREATE_FORM (see Section 70.10 on page 1353). Deleting an already deleted form or a form that was never created has no effect.

Parameters:

**FORM** is a form.

**STATUS** is an established status indicator.

Exceptions:

**STATUS_ERROR** is raised if STATUS is not an established status indicator [3].

## 70.12 Copying a form

```
procedure COPY_FORM
    (FROM   : in     FORM_TYPE;
     TO     : in out FORM_TYPE;
     STATUS : in     STATUS_INDICATOR := NO_INDICATION);
```

Purpose:

This procedure copies the value of the form FROM to the form TO.

Parameters:

**FROM** is the form to be copied.

**TO** is the form receiving the value.

**STATUS** is an established status indicator.

Exceptions:

**STATUS_ERROR** is raised if STATUS is not an established status indicator [3].

**FORM_STATUS_ERROR** is raised if FROM has not been previously created by the procedure CREATE_FORM (see Section 70.10 on page 1353) [1].



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Notes:

If the form TO does not exist, it is created with the same parameters with which the form FROM was created. 7

## 70.13 Defining a qualified area

```
procedure DEFINE_QUALIFIED_AREA
    (FORM       : in out FORM_TYPE;
     INTENSITY  : in     AREA_INTENSITY_KIND := NORMAL;
     PROTECTION : in     AREA_PROTECTION_KIND := PROTECTED;
     INPUT      : in     AREA_INPUT_KIND     := GRAPHIC_CHARACTERS;
     VALUE      : in     AREA_VALUE_KIND     := NO_FILL;
     STATUS     : in     STATUS_INDICATOR    := NO_INDICATION);
```

Purpose:

This procedure places an area qualifier with the designated attributes (INTENSITY, PROTECTION, INPUT, VALUE) at the active position of the form FORM. A qualified area consists of the character positions between two area qualifiers. The area is qualified by the area qualifier that precedes the area. A qualified area mayor may not include the position of its area qualifier (see Section 70.28 on page 1364 and Section 70.29 on page 1364). 1

Parameters:

**FORM** is the form on which the qualified area is being defined. 2

**INTENSITY** is the intensity at which the qualified area is to redisplayed. 3

**PROTECTION** is the protection for the qualified area. 4

**INPUT** is the set of permissible input characters for the qualified area. 5

**VALUE** is the initial value of the qualified area. 6

**STATUS** is an established status indicator. 7

Exceptions:

**STATUS_ERROR** is raised if STATUS is not unestablished status indicator [3]. 8

**FORM_STATUS_ERROR** is raised if FORM has not been previously created by the procedure CREATE_FORM (see Section 70.10 on page 1353) [1]. 9

Notes:

The characteristics (intensity, protection, input, value) of positions that precede the fist area qualifier on a form are implementation_defined. 10

## 70.14 Removing an area qualifier

```
procedure REMOVE_AREA_QUALIFIER
    (FORM   : in out FORM_TYPE;
     STATUS : in     STATUS_INDICATOR := NO_INDICATION);
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Purpose:

This procedure removes an area qualifier from the active position of the form. Removing an area qualifier from a position that does not have an area qualifier has no effect. 1

Parameters:

**FORM** is the form from which the qualified area is to be removed. 2

**STATUS** is an established status indicator. 3

Exceptions:

**STATUS_ERROR** is raised if STATUS is not an established status indicator [3]. 4

**FORM_STATUS_ERROR** is raised if FORM has not been previously created by the procedure CREATE_FORM (see Section 70.10 on page 1353) [1]. 5

Notes:

The positions of the area from which the area qualifier is being removed become part of the preceding qualified area. If there is no preceding qualified area the set of characteristics for the positions are implementation_dependent. 6

## 70.15 Changing the active position

```
procedure SET_ACTIVE_POSITION
    (FORM     : in out FORM_TYPE;
     POSITION : in     TERMIMAL_POSITION_TYPE;
     STATUS   : in     STATUS_INDICATOR := NO_INDICATION);
```

Purpose:

This procedure indicates the position on the form that is to become the active position. 1

Parameters:

**FORM** is the form on which to change the active position.

**POSITION** is the new active position on the form.

**STATUS** is an established status indicator.

Exceptions:

**STATUS_ERROR** is raised if STATUS is not an established status indicator [3]. 5

**FORM_STATUS_ERROR** is raised if FORM has not been previously created by the procedure CREATE_FORM (see Section 70.10 on page 1353) [1]. 6

**TERMINAL_POSITION_ERROR** is raised if POSITION does not identify a position in FORM [1]. 7



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

## 70.16 Querying the active position

```
function ACTIVE_POSITION
    (FORM   : in FORM_TYPE;
     STATUS : in STATUS_INDICATOR := NO_INDICATION)
    return TERMINAL_POSITION_TYPE;
```

Purpose:

This function returns the active position of the form FORM.

Parameters:

**FORM** is the form to be queried.

**STATUS** is an established status indicator.

Exceptions:

**STATUS_ERROR** is raised if STATUS is not an estabalished status indicater [3]. 4

**FORM_STATUS_ERROR** is raised if FORM has not been previously created by the procedure CREATE_FORM (see Section 70.10 on page 1353) [1]. 5

## 70.17 Advancing forward to qualified area

```
procedure ADVANCE_TO_QUALIFIED_AREA
    (FORM   : in out FORM_TYPE;
     COUNT  : in     CAIS_POSITIVE := 1;
     STATUS : in     STATUS_INDICATOR := NO_INDICATION);
```

Purpose:

This procedure advances the active position COUNT area qualifiers toward the end of the form. If there are fewer than COUNT area qualifiers between the active position and the end of the FORM the active position is set to the position of the last area qualifier on the form. If there are no area qualifiers between the active position and the end of the FORM the active position is not changed. 1

Parameters:

**FORM** is the form on which the active position is being advanced. 2

**COUNT** is the number of qualified are as the active position is to be advanced. 3

**STATUS** is an established status indicator. 4



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Exceptions:

**STATUS_ERROR** is raised if STATUS is not an established status indicator [3]. 5

**FORM_STATUS_ERROR** is raised if FORM has not been previously created by the procedure CREATE_FORM (see Section 70.10 on page 1353) [1]. 6

## 70.18 Writing to a form

```
procedure PUT
   (FORM   : in out FORM_TYPE;
    ITEM   : in     PRINTABLE_CHARACTER;
    STATUS : in     STATUS_INDICATOR := NO_INDICATION);
```

Purpose:

This procedure places ITEM at the active position of FORM and advances the active position one position toward the end position. If the active position is the end position, the active position is set to the start position. 1

Parameters:

**FORM** is the form being written. 2

**ITEM** is the character to be written to the form. 3

**STATUS** is an established status indicator. 4

Exceptions:

**STATUS_ERROR** is raised if STATUS is not an established status indicator [3]. 5

**FORM_STATUS_ERROR** is raised if FORM has not been previously created by the procedure CREATE_FORM (see Section 70.10 on page 1353) [1]. 6

**USE_ERROR** is raised if the active position contains an area qualifier and AREA_QUALIFIER_REQUIRES_SPACE(FORM) was set to TRUE [66]. 7

Additional Interfaces:

```
procedure PUT
   (FORM   : in out FORM_TYPE;
    ITEM   : in     STRING;
    STATUS : in     STATUS_INDICATOR := NO_INDICATION)
is
begin
   for INDEX in ITEM'FIRST . . ITEM'LAST loop
      PUT (FORM, ITEM(INDEX));
   end loop;
end PUT;
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

## 70.19 Erasing a qualified area

```
procedure ERASE_AREA
    (FORM   : in out FORM_TYPE;
     STATUS : in     STATUS_INDICATOR := NO_INDICATION);
```

Purpose:

This procedure places space characters in all positions of the area in which the active position of the form is located. If the active position is not in a qualified area, this procedure has no effect. 1

Parameters:

**FORM** is the form on which the qualified area is being erased. 2

**STATUS** is an established status indicator. 3

Exceptions:

**STATUS_ERROR** is raised if STATUS is not an established status indicator [3]. 4

**FORM_STATUS_ERROR** is raised if FORM has not been previously created by the procedure CREATE_FORM (see Section 70.10 on page 1353) [1]. 5

## 70.20 Erasing a form

```
procedure ERASE_FORM
    (FORM   : in out FORM_TYPE;
     STATUS : in     STATUS_INDICATOR := NO_INDICATION) ;
```

Purpose:

This procedure removes all area qualifiers and places space characters in all positions of the form. The active position is set to the start position. The form is established as not updated (see Section 70.23 on page 1361). The termination key is established as the normal termination key (a value of zero) (see 70.25 on page 1362). 1

Parameters:

**FORM** is the form to be erased. 2

**STATUS** is an established status indicator. 3

Exceptions:

**STATUS_ERROR** is raised if STATUS is not an established status indicator [3]. 4

**FORM_STATUS_ERROR** is raised if FORM has not been previously created by the procedure CREATE_FORM (see Section 70.10 on page 1353) [1]. 5



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

## 70.21 Activating a form on a terminal

```
procedure ACTIVATE
    (TERMINAL : in     FILE_TYPE;
     FORM     : in out FORM_TYPE;
     STATUS   : in     STATUS_INDICATOR := NO_INDICATION);
```

Purpose:

This procedure activates the form on the terminal file associated with the internal file identified by the file handle TERMINAL. The contents of the terminal file are modified to reflect the contents of the form. When the user of the terminal enters a termination key, the modified contents of the terminal file are copied back to the form and returned. This operation may not result in the modification of protected areas. Qualification oppositions preceding the first area qualifier on a form is implementation_dependent. 1

Parameters:

**TERMINAL** is an open file handle identifying the internal file associated with the terminal file. 2

**FORM** is the form to be activated. 3

**STATUS** is an established status indicator. 4

Exceptions:

**STATUS_ERROR** is raised if TERMINAL is not an open file handle [8] or if STATUS is not an established status indicator [3]. 5

**FORM_STATUS_ERROR** is raised if FORM has not been previously created by the procedure CREATE_FORM (see Section 70.10 on page 1353) [1]. 6

**USE_ERROR** is raised if FORM_SIZE(FORM) is not equal to TERMINAL_SIZE(TERMINAL) [67] or if AREA_QUALIFIER_REQUIRES_SPACE(FORM) is not equal to AREA_QUALIFIER_REQUIRES_SPACE(TERMINAL) [68]. 7

## 70.22 Reading from a form

```
procedure GET
    (FORM   : in out FORM_TYPE;
     ITEM   :    out PRINTABLE_CHARACTER;
     STATUS : in     STATUS_INDICATOR := NO_INDICATION);
```

Purpose:

This procedure reads a character from FORM at the active position and advances the active position one position toward the end position. If the active position is the end position, the active position is set to the start position. An area qualifier on a form on which the area qualifier requires space is read as the space character. 1



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Parameters:

**FORM** is the form to be read.

**ITEM** is the character that was read.

**STATUS** is an established status indicator.

Exceptions:

**STATUS_ERROR** is raised if STATUS is not an established status indicator [3].

**FORM_STATUS_ERROR** is raised if FORM has not been previously created by the procedure CREATE_FORM (see Section 70.10 on page 1353) [1].

Additional Interfaces:

```
procedure GET
    (FORM   : in out FORM_TYPE;
     ITEM   :    out STRING;
     STATUS : in      STATUS_INDICATOR := NO_INDICATION)
is
begin
    for INDEX in ITEM'FIRST . . ITEM'LAST loop
        GET (FORM, ITEM(INDEX));
        end loop;
end GET;
```

## 70.23 Determining changes to a form

```
function IS_FORM_UPDATED
    (FORM   : in FORM_TYPE;
     STATUS : in STATUS_INDICATOR := NO_INDICATION)
    return BOOLEAN;
```

Purpose:

This function returns TRUE if the value of my position on the form was modified during the last activate operation in which the form was used; otherwise it returns FALSE.

Parameters:

**FORM** is the form to be queried.

**STATUS** is unestablished status indicator.


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Exceptions:

**STATUS_ERROR** is raised if STATUS is not an established status indicator [3]. 4

**FORM_STATUS_ERROR** is raised if FORM has not been previously created by the procedure CREATE_FORM (see Section 70,10 on page 1353) [1]. 5

## 70.24 Determining the identification of a function key

```
function FUNCTION_KEY_IDENTIFICATION
    (TERMINAL          : in FILE_TYPE;
     KEY_IDENTIFIER :  in CAIS_POSITIVE;
     STATUS            : in STATUS_INDICATOR : = NO_INDICATION)
    return FUNCTION_KEY_NAME;
```

Purpose:

This function returns the string identification of the function key designated by KEY_IDENTIFIER. 1

Parameters:

**TERMINAL** is an open file handle identifying the internal file associated with the terminal file. 2

**KEY_IDENTIFIER** is the identification number of a function key. 3

**STATUS** is an established status indicator. 4

Exceptions:

**STATUS_ERROR** is raised if TERMINAL is not an open file handle [8] or if STATUS is not an established status indicator [3]. 5

**FUNCTION_KEY_STATUS_ERROR** is raised if the value of KEY_IDENTIFIER is greater than NUMBER_OF_FUNCTION_KEYS(TERMINAL) [1]. 6

## 70.25 Determining the termination key

```
function TERMINATION_KEY
    (FORM     : in FORM_TYPE;
     STATUS :  in STATUS_INDICATOR : = NO_INDICATION)
    return CAIS_NATURAL;
```

Purpose:

This function returns a number that indicates which (implementation_dependent) key terminated the ACTIVATE procedure (see Section 70.21 on page 1360) for the form FORM. A value of zero indicates the normal termination key (e.g., the ENTER key). A positive value indicates a function key. 1



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Parameters:

**FORM** is the form to be queried. 2

**STATUS** is an established status indicator. 3

Exceptions:

**STATUS_ERROR** is raised if STATUS is not an established status indicator [3]. 4

**FORM_STATUS_ERROR** is raised if FORM has not been previously created by the procedure CREATE_FORM (see Section 70.10 on page 1353) [1]. 5

## 70.26 Determining the size of a form

```
function FORM_SIZE
   (FORM   : in FORM_TYPE;
    STATUS : in STATUS_INDICATOR : = NO_INDICATION)
   return TERMINAL_POSITION_TYPE;
```

Purpose:

This function returns the position of the maximum column of the maximum row of the form. 1

Parameters:

**FORM** is the form to be queried. 2

**STATUS** is an established status indicator. 3

Exceptions:

**STATUS_ERROR** is raised if STATUS is not an established status indicator [3]. 4

**FORM_STATUS_ERROR** is raised if FORM has not been previously created by the procedure CREATE_FORM (see Section 70.10 on page 1353) [1]. 5

## 70.27 Determining the size of the terminal

```
function TERMINAL_SIZE
   (TERMINAL : in FILE_TYPE;
    STATUS   : in STATUS_INDICATOR : = NO_INDICATION)
   return TERMINAL_POSITION_TYPE;
```

Purpose:

This function returns the position of the maximum column of the maximum row of the internal file identified by the file handle TERMINAL. 1



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Parameters:

**TERMINAL** is an open file handle identifying the internal file associated with the terminal file. 2

**STATUS** is an established status indicator. 3

Exceptions:

**STATUS_ERROR** is raised if TERMINAL is not an open file handle [8] or if STATUS is not an established status indicator [3]. 4

## 70.28 Determining if the area qualifier requires space in the form

```
function AREA_QUALIFIER_REQUIRES_SPACE
    (FORM   : in FORM_TYPE;
     STATUS : in STATUS_INDICATOR : = NO_INDICATION)
    return BOOLEAN;
```

Purpose:

This function returns TRUE if the area qualifier requires space in the form FORM; otherwise it returns FALSE. 1

Parameters:

**FORM** is the form to be queried. 2

**STATUS** is an established status indicator. 3

Exceptions:

**STATUS_ERROR** is raised if STATUS is not an established status indicator [3]. 4

**FORM_STATUS_ERROR** is raised if FORM has not been previously created by the procedure CREATE_FORM (see Section 70.10 on page 1353) [1]. 5

## 70.29 Determining if the area qualifier requires space on a terminal

```
function AREA_QUALIFIER_REQUIRES_SPACE
    (TERMINAL : in FILE_TYPE;
     STATUS   : in STATUS_INDICATOR : = NO_INDICATION)
    return BOOLEAN;
```

Purpose:

This function returns TRUE if the area qualifier requires space on the internal file identified by the file handle TERMINAL; otherwise it returns FALSE. 1



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Parameters:

**TERMINAL** is an open file handle identifying the internal file associated with the terminal file. 2

**STATUS** is an established status indicator. 3

Exceptions:

**STATUS_ERROR** is raised if TERMINAL is not an open file handle [8] or if STATUS is not an established status indicator [3]. 4

# 80 Package CAIS_PRAGMATICS

```
CAIS_DIRECT_IO_RECORD_SIZE       : constant := CAIS_DIRECT_IO_ELEMENT_SIZE;
DIRECT_IO_RECORD_SIZE            : constant := DIRECT_IO_ELEMENT_SIZE;
CAIS_SEQUENTIAL_IO_RECORD_SIZE : constant := CAIS_SEQUENTIAL_IO_ELEMENT_SIZE;
SEQUENTIAL_IO_RECORD_SIZE        : constant := SEQUENTIAL_IO_ELEMENT_SIZE;
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

# D. Syntax Summary

The material contained in this appendix is not a mandatory part of the standard.

This appendix summarizes the syntax descriptions given throughout this document. Lexical categories not defined here are found in [1815A] Section 2. The notation used is a form of Backus-Naur Form (BNF):

| | |
|---|---|
| Words | identify syntactic categories; |
| [ ] | identify optional items; |
| { } | identify items which may be repeated zero or more times; |
| \| | separates alternatives; and |
| ::= | separates the left-hand and right-hand sides of productions. |

The following definitions are global.

| | | |
|---|---|---|
| identifier | ::= | letter {[ underline] letter-or_digit} |
| graphic_character | ::= | letter<br>\| digit<br>\| special-character<br>\| space-character |
| letter-or_digit | ::= | letter \| digit |
| letter | ::= | upper-case-letter \| lower_case_letter |
| underline | ::= | _ |
| digit | ::= | 0 \| 1 \| 2 \| 3 \| 4 \| 5 \| 6 \| 7 \| 8 \| 9 |
| upper-case_letter | ::= | A \| B \| C \| D \| E \| F \| G \| H \| I \| J \| K \| L \| M \|<br>N \| O \| P \| Q \| R \| S \| T \| U \| V \| W \| X \| Y \| Z |
| lower-case-letter | ::= | a \| b \| c \| d \| e \| f \| g \| h \| i \| j \| k \| l \| m \|<br>n \| o \| p \| q \| r \| s \| t \| u \| v \| w \| x \| y \| z |
| special-character | ::= | " \| # \| & \| ' \| ( \| ) \| * \| + \| , \| - \| . \| / \| : \| ; \| < \| = \| > \| ! \| _ |



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

# 10 Pathname Syntax

The syntax for pathnames is specified as follows:

| | | |
|---|---|---|
| path_name | ::= | relationship_key_designator {path_element}<br>\| path_element {path_element}<br>\| \| |
| path_element | ::= | 'relation_name [(relationship_key_list)]<br>\| .relationship_key_designator |
| relation_name | ::= | identifier |
| relationship_key_list | ::= | relationship_key_designator<br>\| key_value_association<br>{,key_value_association}<br>\| empty_key_list |
| key_value_association | ::= | relationship_key_name =><br>relationship_key_value |
| relationship_key_name | ::= | identifier |
| relationship_key_value | ::= | relationship_key_designator<br>\| null_key |
| relationship_key_designator | ::= | identifier<br>\| [identifier_prefix]# |
| identifier_prefix | ::= | letter {[underline]letter_or_digit}[underline] |
| empty_key_list | ::= | |
| null_key | ::= | |

Note that the relation name DOT must have anon_empty relationship key.

# 20 GRANT Attribute Value Syntax

The syntax for GRANT attribute values is as follows:

| | | |
|---|---|---|
| grant_attribute_value | ::= | ([grant_item{,grant_item}]) |
| grant_item | ::= | ([necessary_right =>]resulting_rights_list) |
| necessary_right | ::= | identifier |
| resulting_rights_list | ::= | identifier \| (identifier{,identifier}) |

# 30 Classification Attribute Value Syntax

The syntax for classification attribute values is as follows:

| | | |
|---|---|---|
| object_classification | ::= | classification |
| subject_classification | ::= | classification |
| classification | ::= | (hierarchical_classification[,non_hierarchical_categories]) |
| hierarchical_classification | ::= | keyword |
| non_hierarchical_categories | ::= | (keyword{,keyword}) |
| keyword | ::= | identifier |



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

# 40 List External Representation Syntax

The syntax for the external representation of lists is as follows:

```
list          ::= named_list
                | unnamed_list
                | empty_list
named_list    ::= (named_item{,named_item})
unnamed_list  ::= (item_value{,item_value})
empty_list    ::= ()
named_item    ::= item_name => item_value
item_value    ::= list
                | quoted_string
                | integer_number
                | float_number
                | identifier
integer_number ::= [ - ] integer_literal
float_number   ::= [ - ] decimal_literal
quoted_string  ::= string_literal
```


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

# 50 Iteration Predicate Syntax

The syntax for iteration predicate values is as follows:

```
predicate           ::=  (expression)
expression          ::=    unary_operation
                         | binary_operation
                         | function_call
                         | item
                         | literal
unary_operation     ::=  (unary_operator, expression)
binary_operation    ::=  (expression, binary_operator, expression)
function_call       ::=     IS_ PRIMARY
                         |  (HAS_EMANATING, expression)
                         |  (IS_DEFINED, expression)
                         | (NODE_ATTRIBUTE, expression)
                         | (RELATIONSHIP_ATTRIBUTE, expression)
                         | (EXTRACT_VALUE, expression [,expression])
item                ::=    list_item | string_item | integer_item
                         | float_item | identifier_item
literal             ::=  (LITERAL, literal_item)
list_item           ::=  unnamed_list | named_list | empty_list
unnamed_list        ::=  (expression {, expression})
named_list          ::=  (identifier => expression
                         {, identifier => expression})
empty_list          ::=  ()
unary_operator      ::=  NOT
binary_operator     ::=    AND | AND_THEN | OR | OR_ELSE | XOR
                         | EQUAL | NOT_EQUAL | MATCHES
                         | GREATER | GREATER_ EQUAL
                           LESS | LESS_ EQUAL
reserved_identifier ::=    unary_operator | binary_operator | EXTRACT_VALUE
                         | HAS_ EMANATING | IS_ DEFINED | IS_PRIMARY | LITERAL
                         | NODE_ ATTRIBUTE | RELATIONSHIP_ ATTRIBUTE
```


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

# 60 External Node Set Syntax

The syntax for the header file of an external node set is as follows:

```
external_node_set   ::= header_section { content_section }

header_section      ::=  (INSTANCE => cais_instance_name,
                          SMALL_FOR_CAIS_DURATION => attribute_value,
                          TRANSMIT_TIME => attribute_value, NODES => node_s,
                          CONTENT_ COUNT => integer)

nodes               ::=  ([ node{ ,node}])
attribute_s         ::=  ([ attribute{ , attribute} ] )
relationship_s      ::=  ([ relationship{ , relationship} ] )

node                ::=  (UID => node_uid, NKD => definition_uid,
                          ATTRIBUTES => attribute_s, RELATIONSHIPS =>
                          relationship_s)
attribute           ::=  (AD => definition_uid, VALUE => attribute_ value)
relationship        ::=  (RD => definition_uid, KIND => relationship_kind,
                          TARGET => target_uid, KEYS => attribute_s,
                          ATTRIBUTES => attributers [ , REVERSE => reverse_link ] )
reverse_link        ::=  (RD => target_definition_uid, KEYS => attribute_s)

content_section     ::=  content_descriptor
                         < text lines representing file node contents >

content _descriptor ::=  (UID => node_uid, NKD => definition_uid,
                          TEXT_LINES => integer)

cais_instance_name  ::=  string

node_uid            ::=  string
definition_uid      ::=  string
target_uid          ::=  string
target_definition_uid ::=  string

attribute_value     ::=  list

relationship_kind   ::=  PRIMARY | SECONDARY
```


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

# 70 ATTRIBUTES_MONITORED Attribute Value Syntax

The syntax for ATTRIBUTES_MONITORED attribute values is as follows:

| | | |
|---|---|---|
| at tributes_monitored_value | ::= | (name => attribute_designator<br>{, name => attribute_designator}) |
| attribute_designator | ::= | AD_unique_identifier<br>\| relationship_at tribute_designat or |
| relationship_attribute_designator | ::= | (RD => RD_unique_identifier, KEYS => key_value_list,<br>AD => AD_unique_identifier) |
| key_value_list | ::= | (key_value_designator {,key_value_designator})<br>I () |
| key_value_designator | ::= | (AD => AD_unique_identifler [, VALUE => key_value ]) |
| name | ::= | identifier |
| key_value | ::= | identifier |
| AD_unique_identifier | ::= | unique_identifier |
| RD_unique_identifier | ::= | unique_identifier |
| unique_identifier | ::= | quoted_string |


Source: https://assist.dla.mil -- Downloaded: 2026l0-26T07:54Z
Check the source to verify that this is the currentversion before use.

# 80 Constraints Attribute Value Syntax

The syntax for constraints attribute values is as follows:

```
constraints_list          ::= ( constraint_item {, constraint_item} )
                            | ()
constraint_item           ::= value_constraint_item
                            | range_constraint_item
                            | pattern_constraint_item
                            | alternative_constraint_item
                            | uniform_list_size_constraint_item
                            | component_constraint_item
                            | naming_constraint_item
value_constraint_item     ::= VALUE => item_value
range_constraint_item     ::= LOWER_BOUND => numeric_item
                            | UPPER_BOUND => numeric_item
patterns_constraint_item  ::= PATTERNS => ( string_item {, string_item} )
alternative_constraint_item ::= ALTERNATIVE =>
                            ( alternative_name => constraints_list
                            {, alternative_name => constraints_list} )
uniform_list_size_constraint_item ::= MINIMUM_SIZE => integer_item
                            | MAXIMUM_SIZE => integer_item
component_constraint_item ::= COMPONENT => constraints_list
                            | COMPONENT =>
                            ( constraints_list {, constraints_list} )
                            | COMPONENT =>
                            ( componentname => constraints_fist
                            {, component_name => constraints_list} )
naming_constraint_item    ::= NAMING => NAMED
                            | NAMING => UNNAMED
numeric_item              ::= integer_item | float_item
alternative_name          ::= identifier
component_name            ::= identifier
```


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

# 90 Status Information List Syntax

The syntax for status information lists is as follows:

```
status_information_list ::= ( PACKAGE => identifier,
                              INTERFACE => identifier
                            [, CONDITION => condition_code]
                            [, SUBCONDITION => subcondition_code]
                            [, PARAMETERS => ( parameter{, parameter} )]
                            {, qualifier} )
qualifier               ::= AMBIGUOUS_NAMES => ( identifier , identifier )
                          | ACCESS_RIGHT => string
                          | ALLOWED_USAGE => relationship_kind
                          | ANCESTOR_DEFINITION => NKD_unique_identifier
                          | ANCESTOR_DEFINITION => RD_unique_identifier
                          | ATTRIBUTE => AD_unique_identifier
                          | ATTRIBUTE_KIND => AKD_unique_identifier
                          | ATTRIBUTES_LIST => list
                          | ATTRIBUTE_NAME => string
                          | BLOCK_SIZE => integer
                          | CHANNEL => CHANNEL_unique_identifier
                          | CHARACTERISTICS => list
                          | CLASSIFICATION_ERROR => classification_error_type
                          | CLASSIFICATION_LABEL => label
                          | CLIENT_NODE => unique_identifier
                          | CHANNEL_STATE => channel_state_type
                          | COMPONENT => component_specification
                          | CONFLICT => constraints_conflict
                          | CONFLICTING_INTENTS => intents_list
                          | CONSTRAINT_VIOLATED => constraint_violated
                          | CONSTRAINT_ITEM => (constraint_item)
                          | CURRENT_LIST => current_linear_list_specification
                          | CURRENT_LIST_1 => current_linear_list_specification
                          | CURRENT_LIST_2 => current_linear_list_specification
                          | DAY => integer
                          | DEADLOCK => TRUE | FALSE
                          | DEFINITION => definition_unique_identifier
                          | DEFINITION_NAME => identifier
                          | DEFINITION_TYPE => definition_type
                          | DEVICE => DEVICE_unique_identifier
                          | DIRECTION => direction_type
                          | END_POSITION => integer
                          | ERROR_ITEM => integer
                          | ERROR_SUB LIST => current_linear_list_specification
                          | EXPECTED_CHANNEL_STATE => channel_state_type
                          | FACTOR => constraints_conflict_factor
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
FILE => FILE_unique_identifier
FILE_NAME => string
GATEWAY => GATEWAY_unique_identifier
GRANT => list
GROUP => GROUP_unique_identifier
IDENTIFIER_TEXT => string
IMPLIED_USAGE => relationship_kind
INPUT_PARAMETER => list
INVALID_ITEM => string
INTENTS => intents_list
INTERPRETATION => interpretation_unique_identifier
INTERPRETATION => RI_unique_identifier
INTERPRETATION_1 => NKI_unique_identifier
INTERPRETATION_2 => RI_unique_identifier
INTERPRETATION_TYPE => interpretation_type
ITEM_KIND => list_item_kind
ITEM_NAME => identifier
ITEM_POSITION => integer
ITEM_VALUE => item_value
KEY => AD_unique_identifier
LIST => list
LIST_1 => list
LIST_2 => list
LIST_KINDS => ( list_kind, list_kind )
LIST_LENGTH => integer
LIST_TEXT => string
MAXIMUM_LINE_LENGTH => integer
MAXIMUM_PAGE_LENGTH => integer
MEDIUM => MEDIUM_unique_identifier
MODE => identifier
MONITOR => MONITOR_unique_identifier
MONTH => integer
NAME => identifier
NAME_TYPE => name_type
NEW_INTENTS => intents
NODE => unique_ identifier
NODE_1 => unique_identifier
NODE_2 => unique_identifier
OTHER_CLIENT_NODE => unique_identifier
OTHER_DIRECTION => direction_type
OTHER_PORT => RD_unique_identifier
PATHNAME => string
PATTERN => string
PORT => RD_unique_identifier
```


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
                                    | PRAGMATIC => identifier
                                    | PREDICATE => list
                                    | PRIORITY => integer
                                    | PROCESS => PROCESS_unique_identifier
                                    | PROCESSOR => PROCESSOR_unique_identifier
                                    | PROPERTY => property_unique_identifier
                                    | PROPERTY_TYPE => property_type
                                    | PROTOCOL => identifier
                                    | RELATION => RD_unique_identifier
                                    | RELATIONSHIP => relationship_specification
                                    | RELATIONSHIP_KEY => string
                                    | RELATIONSHIP_NAME => string
                                    | RELATION_NAME => string
                                    | REQUIRED_ITEM_KIND => list_item_kind
                                    | REQUIRED_NKD => NKD_unique_identifier
                                    | REQUIRED_RD => RD_unique_identifier
                                    | RESOURCE => resource_unique_identifier
                                    | RESOURCE_TYPE => resource_type
                                    | RESTRICTION => node_deletion_restriction
                                    | RESTRICTION => property_deletion_restriction
                                    | RIGHTS_NEEDED => rights_list
                                    | ROLE_TYPE => role_type
                                    | SECONDS => integer
                                    | SOURCE_NODE => unique_identifier
                                    | START_POSITION => integer
                                    | SYNTACTIC_ERROR => syntactic_error_type
                                    | TAPE_STATUS => tape_status_type
                                    | TARGET_NODE => unique_identifier
                                    | UID => unique_identifier
                                    | VALUE => list
                                    | VIEW => VIEW_unique_identifier
                                    | YEAR => integer
access_right                ::=     identifier
AD_unique_identifier        ::=     unique_identifier
base_type_restriction       ::=     constraints_conflict
                                    NONCONFORMING_INSTANCES
channel_state_type          ::=     CONNECTED
                                  | ESTABLISHED
                                  | CLOSED
                                    RETIRED
CHANNEL_unique_identifier   ::=     unique_identifier
classification_error_type   ::=     INVALID_HIERARCHICAL_CLASSIFICATION
                                  | INVALID_NON_HIERARCHICAL_CATEGORY
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
component_identification        ::== ( [ ITEM_POSITION => integer]
                                     [, NAME => identifier]
                                     [, POSITION => identifier] )
component_specification         ::= ( component_identification
                                     {, component_identification} )
condition_code                  ::=   integer
constraint_violated             ::=   BOUNDS
                                    | ENUMERATION_POSITION
                                    | NAMING
                                    | ORDER
                                    | PATTERNS
                                    | SIZE
constraints_conflict            ::=   ATTRIBUTE_BOUNDS
                                    | ATTRIBUTE_DEFAULT
                                    | ATTRIBUTE_DEFAULT_OUTSIDE_RANGE
                                    | ATTRIBUTE_DEFAULT_OUTSIDE_VALUE
                                    | ATTRIBUTE_NAMING
                                    | ATTRIBUTE_PATTERNS
                                    | ATTRIBUTE_SIZE
                                    | ATTRIBUTE_VALUE
                                    | ATTRIBUTE_VALUE_OUTSIDE_RANGE
                                    | RELATION_ALLOWED_USAGE
                                    | RELATION_BOUNDS
                                    | RELATION_TERMINATES
                                    | RELATION_UNIQUE_KEYS
constraints_conflict_factor     ::=   HAS_LOCAL_CONSTRAINT
                                    | HAS_OBTAINABLE_INSTANCES
constraint_item                 ::=   value_constraint_item
                                    | range_constraint_item
                                    | patterns_constraint_item
                                    | naming_constraint_item
                                    | uniform_list_size_constraint_item
current_linear_list_specification ::=   () | ( integer{, integer} )
definition_type                 ::=   NKD | RD | AD | AKD
definition_unique_identifier    ::=   NKD_unique_identifier
                                    | RD_unique_identifier
                                    | AD_unique_identifier
                                    | AKD_unique_identifier
DEVICE_unique_identifier        ::=   unique_identifier
direction_type                  ::=   IN | OUT | IN_ AND_ OUT | IN_OR_OUT
                                    | NO_DIRECTION
FILE_unique_identifier          ::=   unique_identifier
GATEWAY_unique_identifier       ::=   unique_identifier
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
GROUP_unique_identifier      ::=  unique_identifier
intents_list                 ::=  ( intent_specification {, intent_specification} )
intent_specification         ::=  identifier
interpretation_type          ::=  NKI | RI
item_value                   ::=  list | identifier | integer | string | float
key_list                     ::=  ( key_value_specification {, key_value_specification} )
key_value                    ::=  identifier | “”
key_value_specification      ::=  ( KEY => AD_unique_identifier,
                                    VALUE => key_value )
list_item_kind               ::=   LIST | IDENTIFIER | INTEGER | STRING
                                 | FLOAT
list_kind                    ::=  EMPTY | NAMED | UNNAMED
MEDIUM_unique_identifier     ::=  unique_identifier
MONITOR_unique_identifier    ::=  unique_identifier
name_type                    ::=  RELATION | ATTRIBUTE | KEY
NKD_unique_identifier        ::=  unique_identifier
NKI_unique_identifier        ::=  unique_identifier
node_deletion_restriction    ::=   HAS_OBTAINABLE_ATTRIBUTE
                                 | HAS_OBTAINABLE_INSTANCE
                                 |  HAS_OBTAINABLE_SPECIALIZATION
                                 | IN_ADOPTED_VIEW
                                 | IN_MULTIPLE_VIEWS
                                 |  IS_AN_ALTERNATIVE
                                 |  IS_AN_ATTRIBUTE_PROPERTY
                                 | IS_A_COMPONENT
                                 | IS_AN_EMANATES_PROPERTY
                                 |  IS_A_KEY_PROPERTY
                                 |  IS_A_TERMINATES_PROPERTY
                                 |  IS_PREDEFINED
parameter                    ::=  parameter_name = > parameter_value
parameter_name               ::=  identifier
parameter_value              ::=   float | identifier | integer | intents_list
                                 | list | string | unique_identifier
PROCESS_unique_identifier    ::=  unique_identifier
PROCESSOR_unique_identifier  ::=  unique_identifier
property_deletion_restriction ::=  IN_ADOPTED_VIEW
                                 |  IN_MULTIPLE_VIEWS
                                 |  IS_PREDEFINED
                                 |  PROPERTY_INSTANCE
                                 |  RELATION_TERMINATES
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

```
property_type                ::=  ALTERNATIVE
                                | ATTRIBUTE
                                | ATTRIBUTE_KIND
                                | COMPONENT
                                | EMANATES
                                | KEY
                                  TERMINATES
property_unique_identifier   ::=  RD_unique_identifier
                                | AD_unique_identifier
                                | AKD_unique_identifier
RD_unique_identifier         ::=  unique_identifier
relationship_kind            ::=  PRIMARY | SECONDARY
relationship_specification   ::=  ( BASE => unique_identifier,
                                  RELATION => RD_unique_identifier
                                  [, KEY => key_list] )
resource_type                ::=  DEVICE | GATEWAY | MEDIUM | PROCESSOR
resource_unique_ identifier  ::=  DEVICE_unique_identifier
                                | GATEWAY_unique_identifier
                                | MEDIUM_unique_identifier
                                | PROCESSOR_unique_identifier
rights_list                  ::=  ( access_ right {, access_right } )
RI_unique_identifier         ::=  unique_ identifier
role_type                    ::=  IMAGE
                                | USER
subcondition_code            ::=  integer
syntactic_error_type         ::=  COMMA_OR_RIGHT_PARENTHESIS_EXPECTED
                                | ARROW_EXPECTED
                                | ERROR_IN_DECIMAL_LITERAL
                                | ERROR_IN_IDENTIFIER
                                | ERROR_IN_INTEGER_LITERAL
                                | ERROR_ IN_ STRING_ LITERAL
                                | LEFT_PARENTHESIS_EXPECTED
                                | IDENTIFIER_EXPECTED
                                | INTEGER_EXPECTED
                                | RIGHT_PARENTHESIS_EXPECTED
                                | TICK_EXPECTED
                                | UNEXPECTED_SYMBOL
tape_status_type             ::=  OPENED
                                | MOUNT_REQUESTED
                                | MOUNTED
                                | LOADED
                                | CLOSED
unique_identifier            ::=  string
unique_identifier_list       ::=  ( [ unique_identifier{, unique_identifier } ] )
```



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

# E. Discretionary Access Control Conformance Guidelines

The material contained in this appendix is not a mandatory part of the standard. 1

This appendix provides guidelines for meeting discretionary access control requirements at classes Cl, C2, and B3 of DOD 5200.28-STD. These guidelines provide a discretionary access control capability that is natural to the CAIS by taking advantage of the access control mechanisms described in Sections 4.4 and 5.4, and the CAIS node model described in Section 4.3. The access control mechanisms described in Section 5.4 are intended to provide a general access control capability and apply to both secure and non-secure (with respect to DOD 5200.28-STD) implementations. Section 5.4 does not map directly to the various classes of DOD 5200.28-STD; these guidelines provide such a mapping to assist in achieving an implementation-dependent discretionary access control capability for a desired class. 2

## 10 Class C1

> In this class, the TCB shall define and control access between named users and named objects (e.g., files and programs) in the ADP system. The enforcement mechanism (e.g., self/group/public controls, access control lists) shall allow users to specify and control sharing of those objects by named individuals or defined groups or both [DOD5200.28-STD]. 1

In order to meet the discretionary access control requirement at the C1 Class, an implementation must provide the following capabilities: 2

a. User control of objects 3

b. User ability to specify and control access modes that protect these objects 4

c. User ability to specify other individual users or defined groups to which access can be granted 5

Note: At the Cl level a user may have the option to impose discretionary access controls on objects which that user can control. 6

### 10.1 Capability a: user control of objects

The CAIS provides the ability for a user to have control of an object. At object creation, a user can establish initial access control information for the created object and can become the controller of the object. This capability is provided through the parameters of the subtype DISCRETIONARY_-ACCESS_LIST of procedures which create nodes. (For an example, see the procedure CREATE_NODE described in Section 5.5.2 on page 280). Only a user who is the controller of the object can grant, revoke, or modify access rights. 1

### 10.2 Capability b: user ability to specify access modes

The CAIS procedure SET_GRANTED_RIGHTS provides users the ability to specify access rights for a given object (see Section 5.4.10 on page 271). 1



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

## 10.3 Capability c: user ability to specify individuals or groups

Within the CAIS, a group is a set of users that is represented by a GROUP node. GROUP nodes 1
can be used by a CAIS implementation to represent individual users or defined groups of users. Group nodes may have as members individual users or groups represented by other GROUP nodes.

The CAIS associates a role with each GROUP node which is the set of all ACCESS and DENY 2
relationships which emanate from nodes and terminate at the GROUP node or any of the GROUP nodes on its primary path. When a process adopts a role it acquires the access rights to nodes (objects) determined by the ACCESS and DENY relationships which constitute the role.

The CAIS discretionary access control model requires that a “default GROUP node” be associated 3
with each USER node by a secondary DEFAULT_ROLE relationship which emanates from the USER node and terminates at the default GROUP node. A process executing on behalf of a user implicitly adopts the role associated with the user’s default GROUP node. Each USER node can have only one emanating DEFAULT_ROLE relationship. The CAIS discretionary access control model allows more than one user to be a member of the group represented by a default GROUP node.

Groups which are reachable by traversal of PERMANENT_MEMBER and POTENTIAL_MEMBER 4
relationships starting from a given GROUP node are termed that group’s potential members. A process may explicitly adopt additional roles associated with GROUP nodes that are potential members of a GROUP node which is the target of an ADOPTED_ROLE relationship emanating from the process’ PROCESS node.

The CAIS does not provide interfaces to modify GROUP nodes, their attributes, or their relation- 5
ships. Each CAIS implementation must provide interfaces for the creation, modification, or deletion of GROUP nodes and for the creation, deletion, and modification of their relationships. It is also left to each CAIS implementation to provide interfaces for the creation, modification, or deletion of USER nodes and DEFAULT_ROLE relationships. To meet the Cl requirements, control of these interfaces should be limited to the database administrator and described in the Trusted Facility Manual. In order to satisfy the Cl requirement that users be capable of specifying access to named individual users, specific restrictions on the application of these interfaces to the creation of default GROUP nodes should be enforced by the database administrator and described in the Trusted Facility Manual. The restrictions that should be enforced are: (1) each default GROUP node can be the target of at most one DEFAULT_ROLE relationship, and (2) a default GROUP node cannot have emanating PERMANENT. MEMBER or POTENTIAL_ MEMBER relationships. The effect of the first restriction is a guarantee that each default GROUP node represents a group with an individual user as its only member. Such a GROUP node is termed a “single member default GROUP node” and can be used to specify a named individual user. The second restriction prevents processes acting on behalf of one user from explicitly adopting the role associated with the default GROUP node associated with another user.

# 20 Class C2

In this class the TCB shall define and control access between named users and named objects (e.g., 1
files and programs) in the ADP system. The enforcement mechanism (e.g., self/group/public controls, access control lists) shall allow users to specify and control sharing of those objects by named individuals, or defined groups of individuals, or both, and shall provide controls to limit propagation of access rights. The discretionary access control mechanism shall, either by explicit user action or by default, provide that objects are protected from unauthorized access. These



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

access controls shall be capable of including or excluding access to the granularity of a single user. Access permission to an object by users not already possessing access permission shall only be assigned by authorized users [DOD5200.28-STD].

In order to meet the discretionary access control requirements at the C2 Class, an implementation must provide the following capabilities:

a. User control of objects

b. User ability to specify and control access modes that protect these objects

c. User ability to specify individual users or defined groups of individuals

d. User ability to include and exclude object access to the granularity of an individual

e. Access permission to an object must be assigned by an authorized user.

## 20.1 Capability a: user ability to control objects

Capability a is identical to Capability a at the Cl level (see 10.1).

## 20.2 Capability b: user ability to specify and control access modes

Capability b is identical to capability b at the Cl level (see 10.2).

## 20.3 Capability c: user ability to specify individual users or groups of individual users

Capability c is similar to capability c at the Cl level, but has been enhanced. At Class C2, groups (made up of multiple users) specified for the purpose of discretionary access control are required to be made up of individual users. Implementations that wish to comply with the Class C2 should restrict GROUP nodes that are used for discretionary access control to either single member default GROUP nodes or groups made up of single member default GROUP nodes.

PERMANENT_MEMBER and POTENTIAL_MEMBER relationships emanating from a GROUP node and terminating at a single member default GROUP node could be used to construct groups composed of single member default GROUP nodes.

CAIS interfaces do not provide the ability to create PERMANENT_MEMBER and POTENTIAL_-MEMBER relationships. A secure implementation must supply these interfaces and restrict their use. Control of these interfaces should be limited to the database administrator and described in the Trusted Facility Manual.

## 20.4 Capability d: user ability to include or exclude object access to the granularity of an individual

The ability to grant or revoke access control to the granularity of an individual user can be provided to a user by the interface SET_GRANTED_RIGHTS (see Section 5.4.10 on page 271) and by an implementation that enforces single member default GROUP nodes as described in capability c at the



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Cl level (see 20.3). The SET_GRANTED_RIGHTS interface can be used to establish an ACCESS relationship (include access) to an individual user. To revoke (exclude) an access right, the SET_-GRANTED_RIGHTS interface could be used to take away access by changing the GRANT attribute on an ACCESS relationship. Access rights can also be taken away by using the DELETE_GRANTED_-RIGHTS interface to delete ACCESS relationships, (See Section 5.4.11 on page 273).

## 20.5 Capability e: assignment of access control by an authorized user

This requirement is inherent to the CAIS discretionary access control capability described in Section 5.4. A subject must have the access right CONTROL in order to open a node (object) with the intent to manipulate the access rights defined by ACCESS relationships emanating from the node. (See Table X). When a node (object) is created using the CAIS interfaces, an ACCESS relationship is created which emanates from the newly created node and terminates at the default GROUP node of the current user thereby establishing the initial authorization to assign access control information for the object. Thereafter, such authorization may be retained, shared with other groups or relinquished to other groups.

# 30 Class B3

> In this class, the TCB shall define and control access between named users and named objects (e.g., files and programs) in the ADP system. The enforcement mechanism (e.g., access control lists) shall allow users to specify and control sharing of those objects, and shall provide controls to limit propagation of access rights. The “discretionary access control mechanism shall, either by explicit user action or by default, provide that objects are protected from unauthorized access. These access controls shall be capable of specifying, for each named object, a list of named individuals and a list of groups of named individuals with their respective modes of access to that object. Furthermore, for each such named object, it shall be possible to specify a list of named individuals and a list of groups of named individuals for which no access to the object is to be given. Access permission to an object by users not already possessing access permission shall only be assigned by authorized users [DOD-5200.28-STD].

In order to meet the discretionary access control requirements at the B3 Class, an implementation must provide the following capabilities:

a. User control of objects

b. User ability to specify and control access modes that protect these objects

c. Ability to specify for each object a list of individuals and a list of groups of individuals with respective modes of access to that object (Access Control List)

d. User ability to include or exclude object access to the granularity of an individual

e. Access permission to an object must be assigned by an authorized user

f. Ability to specify a list of individuals and a list of groups of individuals for which access to an object will be denied.



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

## 30.1 Capability a: user ability to control objects

Capability a is identical to capability a at the C2 level (See 20.1). 1

## 30.2 Capability b: user ability to specify and control access modes

Capability b is identical to capability b at the C2 level (see 20.2). 1

## 30.3 Capability c: access control list

A CAIS implementation can enforce discretionary access control by using the concept of an Access Control List. Through the use of the SET_GRANTED_RIGHTS interface, a user can specify for each object under the user's control, those access rights to be associated with individual users or a single group made up of individual users. 1

In order for a user to be able to specify individual users and groups of individual users, capability c at the C2 level must be provided (see 20.3). 2

## 30.4 Capability d: user ability to include or exclude object access to the granularity of an individual

Capability d is identical to capability d at the C2 level (see 20.4). 1

## 30.5 Capability e: access permission to an object must be assigned by an authorized user

Capability e is identical to capability e at the C2 level (see 20.5). 1

## 30.6 Capability f: ability to deny access to individuals or groups

The ability to deny access to individual users and groups of individual users can be provided to a user through the interface DENY_ACCESS and by an implementation enforcing single member default GROUP nodes as described in capability c at the Cl level (see 20.3). The procedure DENY_ACCESS can be used to establish a DENY relationship which emanates from an object and terminates at a single member default GROUP node. The establishment of such a DENY relationship prevents access by the individual user whose USER node is associated with the single member default GROUP node. Similarly, access to an object can be denied to a collection of individual users by using the DENY_-ACCESS interface to establish a DENY relationship which emanates from the object and terminates at a GROUP node which is not a single member default GROUP node. DENY relationships can be deleted by the DELETE_DENY_RELATIONSHIP interface (See Section 5.4.14 on page 275). 1

The DENY_ACCESS interface is intended for the specification of users or groups for which no access will be granted to an object. This does not mean that the CAIS access control mechanisms provide the ability to deny individuals or groups selected access rights, but rather represents the ability to deny access altogether. 2


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

# F. Implementation Dependencies

The material contained in this appendix is a mandatory part of the standard.

Reliance on any information provided in this appendix endangers transportability of tools.

This appendix describes those aspects of a CAIS implementation which are implementation-dependent. Some of these aspects are explicitly noted in the CAIS specification and the implementation behavior should be documented herein. Other aspects maybe the result of implementation choices and ambiguity in the CAIS specification; it is recommended that such observed ambiguities be reported to the Ada Joint Program Office as design feedback. (See Section 1.2 on page 2, on application guidance.)

The reference manual of each CAIS implementation must include an appendix (called Appendix F) that describes all implementation-dependent characteristics. The Appendix F for a given implementation must list in particular:

1. Implementation-defined pragmatic limits.
2. Implementation-defined exceptions.
3. Implementation-defined condition codes.
4. Whether and when nodes whose primary relationships have been deleted are actually removed.
5. The effect on existing node iterators of the deletion of the base node, or creation, deletion or conversion of emanating relationships.
6. The effect on existing node iterators of changes to the composite view of the current process.
7. The effect on existing attribute iterators of creation or deletion of attributes or relationships.
8. The effect on existing attribute iterators of changes to the composite view of the current process.
9. The effect on existing node kind instance iterators of the creation or deletion of nodes of the specified node kind, or conversion of nodes to or from the specified node kind.
10. The effect on existing relation instance iterators of the creation or deletion of relationships of the specified relation, or conversion of relationships to or from the specified relation.
11. The interfaces for the creation and deletion of top-level and GROUP nodes as well as the effects of the deletion of such nodes.
12. The interfaces for the creation, modification and deletion of DEFAULT_ROLE relationships, in particular for those emanating from nodes representing the executable image of a program.
13. The effects of alterations of group memberships or of DEFAULT_ROLE relationships on concurrently executing processes.
14. The method of generation of relationship keys which are created with implementation-defined values.
15. The hierarchical classification level set and the non-hierarchical category set (when mandatory security is implemented).



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

16. The criteria that allow an executing process (subject) to establish or alter an access relationship to a GROUP node. For some interfaces, the exception ACCESS_VIOLATION is raised if the executing process (subject) is not allowed to establish or alter an access relationship to the given GROUP node according to these criteria.
17. The possible values of a node's classification as an object or as a subject.
18. Whether reassignment of nodes to medium resources is supported.
19. Restrictions on the reassignment of nodes to medium resources.
20. The method for users to log onto the APSE and the constraints that limit the top-level nodes at which users may log on.
21. The mechanism that controls which executable image is selected for the root process when a user log into the APSE and the execution environment of that process including the following:
    a. The attributes and relationships of the root PROCESS node created at login.
    b. The values of parameters to the CREATE_JOB interface which would be required to use that interface to create a process having the same characteristics as the login process.
    c. The disposition of the process tree, rooted at the root PROCESS node created at login, when logging out.
22. Whether modification of process priorities is supported.
23. The meaning of the values returned by the TIME_STARTED (see Section 6.3.25 on page 556) and TIME_FINISHED (see Section 6.3.26 on page 557) interfaces.
24. implementation-defined data transfer protocols.
25. The method used in the selection of channel NKD's given two PORT RDs.
26. Whether the existence of multiple file handles identifying the same external file is supported.
27. Restrictions related to the resetting of file handles to a specified mode.
28. Whether the existence of multiple terminal handles identifying the same terminal is supported.
29. Other aspects of any implementation explicitly noted as implementation-dependent in the CAIS specification; these must be identified by CAIS Section number and must describe the implementation choices made.



Source: https://assist.dla.mil -- Downloaded: 202610-26T07:54Z
Check the source to verify that this is the current version before use.

# G. List Of Interfaces

Each of the tables contained in this appendix lists the procedures and functions corresponding to the principal CAIS interfaces contained within a given CAIS package. These tables are provided in order to allow the reader ready access to a description of a particular capability in the CAIS. 1

The material contained in this appendix is not a mandatory part of the standard. 2

| Package CAIS_NODE_MANAGEMENT | | |
|---|---|---|
| **Description** | **Interface** | **Page** |
| Opening a node handle | OPEN | 103 |
| Opening a node handle using a unique identifier | OPEN_VIA_UNIQUE_-IDENTIFIER | 106 |
| Closing a node handle | CLOSE | 107 |
| Changing the intent regarding node handle usage | CHANGE_INTENT | 108 |
| Examining the open status of a node handle | IS_OPEN | 109 |
| Querying the intention of a node handle | INTENT | 109 |
| Querying the kind of a node | KIND_OF_NODE | 110 |
| Determining if one node kind is a specialization of another | IS_SUB_KIND | 110 |
| Determining if one relation is a specialization of another | IS_SUB_RELATION | 112 |
| Determining if a node belongs to a node kind | IS_OF_KIND | 113 |
| Determining if a relationship belongs to a relation | IS_OF_RELATION | 114 |
| Querying the number of open file handles on a CLIENT node | OPEN_FILE_HANDLE_COUNT | 116 |
| Obtaining the unique identifier for a node | UNIQUE_IDENTIFICATION | 116 |
| Obtaining a pathname for the unique primary path | PRIMARY_NAME | 117 |
| Obtaining the relationship key list of a primary relationship | PRIMARY_KEY | 118 |
| Obtaining the relation of a primary relationship | PRIMARY_RELATION | 118 |
| Obtaining the relationship key list of the last relationship traversed | PATH_KEY | 119 |
| Obtaining the relation of the last relationship traversed | PATH_RELATION | 120 |
| Obtaining a partial pathname | BASE_PATH | 121 |
| Obtaining the name of the last relation in a pathname | LAST_RELATION | 122 |
| Obtaining the key list of the last relationship in a pathname | LAST_KEY | 122 |



Source: https://assist.dla.mil -- Downloaded: 202640-26T07:54Z
Check the source to verify that this is the current version before use.

| Package CAIS_NODE_MANAGEMENT (Cont.) | | |
|---|---|---|
| **Description** | **Interface** | **Page** |
| Querying the existence of a node | IS_OBTAINABLE | 123 |
| Querying sameness | IS_SAME | 124 |
| Obtaining an index for a node handle | INDEX | 124 |
| Opening a node handle to the parent | OPEN_PARENT | 125 |
| Copying a node | COPY_NODE | 126 |
| Copying trees | COPY_TREE | 129 |
| Renaming a relationship | RENAME | 134 |
| Deleting the primary relationship to a node | DELETE_NODE | 138 |
| Deleting the primary relationships of a tree | DELETE_TREE | 139 |
| Creating secondary relationships | CREATE_SECONDARY_-RELATIONSHIP | 141 |
| Creating bidirectional secondary relationships | CREATE_BIDIRECTIONAL_-SECONDARY_RELATIONSHIP | 143 |
| Deleting secondary relationships | DELETE_SECONDARY_-RELATIONSHIP | 146 |
| Obtaining a corresponding reverse relationship's relation | GET_REVERSE_RELATION | 148 |
| Obtaining a corresponding reverse relationship's key | GET_REVERSE_KEY | 150 |
| Setting the inheritance property of a relationship | SET_INHERITANCE | 151 |
| Determining if a secondary relationship is inheritable | IS_INHERITABLE | 153 |
| Setting the current node relationship | SET_CURRENT_NODE | 154 |
| Opening a node handle to the current node | GET_CURRENT_NODE | 155 |
| Determining the creation time of a node | TIME_CREATED | 155 |
| Determining the last time a relationship was modified | TIME_RELATIONSHIP_-WRITTEN | 156 |
| Determining the last time that node contents were written | TIME_CONTENTS_WRITTEN | 157 |
| Determining the last time an attribute was modified | TIME_ATTRIBUTE_WRITTEN | 157 |
| Obtaining the default iteration predicate | DEFAULT_PREDICATE | 168 |
| Creating an iterator over nodes | CREATE_ITERATOR | 169 |
| Determining node iteration status | MORE | 173 |
| Determining the approximate size of the node iterator | APPROXIMATE_SIZE | 174 |
| Getting the next node in a node iteration | GET_NEXT | 175 |
| Skipping the next node in a node iteration | SKIP_NEXT | 176 |
| Obtaining the path element for the next node in a node iteration | NEXT_NAME | 177 |
| Deleting a node iterator | DELETE_ITERATOR | 178 |
| Creating an iterator over instances of a node kind definition | CREATE_ITERATOR | 178 |
| Determining node kind instance iteration status | MORE | 180 |



Source: https://assist.dla.mil -- Downloaded: 202610-26T07:54Z
Check the source to verify that this is the current version before use.

| Package CAIS_NODE_MANAGEMENT (Cont.) | | |
|---|---|---|
| **Description** | **Interface** | **Page** |
| Determining the approximate size of the node kind instance iterator | APPROXIMATE_SIZE | 181 |
| Getting the next node in a node kind instance iteration | GET_NEXT | 181 |
| Skipping the next node in a node kind instance iteration | SKIP_NEXT | 183 |
| Deleting a node kind instance iterator | DELETE_ITERATOR | 184 |

| Package CAIS_ATTRIBUTE_MANAGEMENT | | |
|---|---|---|
| **Description** | **Interface** | **Page** |
| Creating node attributes | CREATE_NODE_ATTRIBUTE | 186 |
| Creating relationships attributes | CREATE_PATH_ATTRIBUTE | 187 |
| Deleting node attributes | DELETE_NODE_ATTRIBUTE | 189 |
| Deleting relationship attributes | DELETE_PATH_ATTRIBUTE | 190 |
| Setting node attributes | SET_NODE_ATTRIBUTE | 192 |
| Setting relationships attributes | SET_PATH_ATTRIBUTE | 193 |
| Getting node attributes | GET_NODE_ATTRIBUTE | 195 |
| Getting relationship attributes | GET_PATH_ATTRIBUTE | 196 |
| Creating an iterator over node attributes | CREATE_NODE_ATTRIBUTE_-ITERATOR | 246 |
| Creating an iterator over relationship attributes | CREATE_PATH_ATTRIBUTE_-ITERATOR | 247 |
| Determining iteration status | MORE | 250 |
| Determining the approximate size of the iterator | APPROXIMATE_SIZE | 250 |
| Getting an identification for the next attribute | NEXT_NAME | 251 |
| Getting the next attribute value | GET_NEXT_VALUE | 252 |
| Skipping the next attribute in an iteration | SKIP_NEXT | 253 |
| Deleting an attribute iterator | DELETE_ITERATOR | 253 |

| Package CAIS_ATTRIBUTE_MANAGEMENT.CAIS_INTEGER_ATTRIBUTE | | |
|---|---|---|
| **Description** | **Interface** | **Page** |
| Creating integer node attributes | CREATE_NODE_ATTRIBUTE | 198 |
| Creating integer relationship attributes | CREATE_PATH_ATTRIBUTE | 199 |
| Setting integer node attributes | SET_NODE_ATTRIBUTE | 201 |
| Setting integer relationship attributes | SET_PATH_ATTRIBUTE | 203 |
| Getting integer node attributes | NODE_ATTRIBUTE | 204 |
| Getting integer relationship attributes | PATH_ATTRIBUTE | 206 |



Source: https://assist.dla.mil -- Downloaded: 202610-26T07:54Z
Check the source to verify that this is the current version before use.

| Package CAIS_ATTRIBUTE_MANAGEMENT.CAIS_FLOAT_ATTRIBUTE | | |
|---|---|---|
| **Description** | **Interface** | **Page** |
| Creating floating point node attributes | CREATE_NODE_ATTRIBUTE | 207 |
| Creating floating point relationship attributes | CREATE_PATH_ATTRIBUTE | 209 |
| Setting floating point node attributes | SET_NODE_ATTRIBUTE | 211 |
| Setting floating point relationship attributes | SET_PATH_ATTRIBUTE | 212 |
| Getting floating point node attributes | NODE_ATTRIBUTE | 214 |
| Getting floating point relationship attributes | PATH_ATTRIBUTE | 215 |

| Package CAIS_ATTRIBUTE_MANAGEMENT.CAIS_STRING_ATTRIBUTE | | |
|---|---|---|
| **Description** | **Interface** | **Page** |
| Creating string node attributes | CREATE_NODE_ATTRIBUTE | 217 |
| Creating string relationship attributes | CREATE_PATH_ATTRIBUTE | 218 |
| Setting string node attributes | SET_NODE_ATTRIBUTE | 220 |
| Setting string relationship attributes | SET_PATH_ATTRIBUTE | 222 |
| Getting string node attributes | NODE_ATTRIBUTE | 223 |
| Getting string relationship attributes | PATH_ATTRIBUTE | 225 |

| Package CAIS_ATTRIBUTE_MANAGEMENT.CAIS_IDENTIFIER_ATTRIBUTE | | |
|---|---|---|
| **Description** | **Interface** | **Page** |
| Creating identifier node attributes | CREATE_NODE_ATTRIBUTE | 226 |
| Creating identifier relationship attributes | CREATE_PATH_ATTRIBUTE | 227 |
| Setting identifier node attributes | SET_NODE_ATTRIBUTE | 229 |
| Setting identifier relationship attributes | SET_PATH_ATTRIBUTE | 231 |
| Getting identifier node attributes | GET_NODE_ATTRIBUTE | 232 |
| Getting identifier relationship attributes | GET_PATH_ATTRIBUTE | 234 |

| Package CAIS_ATTRIBUTE_MANAGEMENT.CAIS_ENUMERATION_ATTRIBUTE | | |
|---|---|---|
| **Description** | **Interface** | **Page** |
| Creating identifier node attributes | CREATE_NODE_ATTRIBUTE | 236 |
| Creating identifier relationship attributes | CREATE_PATH_ATTRIBUTE | 237 |
| Setting identifier node attributes | SET_NODE_ATTRIBUTE | 239 |
| Setting identifier relationship attributes | SET_PATH_ATTRIBUTE | 240 |
| Getting identifier node attributes | NODE_ATTRIBUTE | 242 |
| Getting identifier relationship attributes | PATH_ATTRIBUTE | 243 |

| Package CAIS_BOOLEAN_ATTRIBUTE | | |
|---|---|---|
| **Description** | **Interface** | **Page** |
| Instantiation of CAIS_ATTRIBUTE_MANAGEMENT.CAIS_ENUMERATION_ATTRIBUTE | See CAIS_ENUMERATION_ATTRIBUTE | 245 |



Source: https://assist.dla.mil -- Downloaded: 202640-26T07:54Z
Check the source to verify that this is the current version before use.

| Package CAIS_ACCESS_CONTROL_MANAGEMENT | | |
|---|---|---|
| **Description** | **Interface** | **Page** |
| Value of all access rights | ALL_RIGHTS | 271 |
| Setting access control | SET_GRANTED_RIGHTS | 271 |
| Deleting ACCESS relationships | DELETE_GRANTED_RIGHTS | 273 |
| Obtaining the value of a GRANT attribute | GET_GRANTED_RIGHTS | 274 |
| Deleting DENY relationships | DELETE_DENY_RELATIONSHIP | 275 |
| Examining access rights | IS_APPROVED | 276 |
| Adopting a role | ADOPT_ROLE | 277 |
| Unlinking an adopted role | UNADOPT_ROLE | 278 |

| Package CAIS_STRUCTURAL_NODE_MANAGEMENT | | |
|---|---|---|
| **Description** | **Interface** | **Page** |
| Opening the predefined STRUCTURAL NKD | STRUCTURAL_NKD | 280 |
| Creating structural nodes | CREATE_NODE | 280 |

| Package CAIS_TRANSLATION_MANAGEMENT | | |
|---|---|---|
| **Description** | **Interface** | **Page** |
| Adopting a view | ADOPT_VIEW | 284 |
| Cancelling a view adoption | UNADOPT_VIEW | 285 |
| Adopting an NKI | ADOPT_INTERPRETATION | 286 |
| Cancelling an NKI adoption | UNADOPT_INTERPRETATION | 288 |
| Opening a handle to an NKD by local name | OPEN_NODE_KIND_-DEFINITION | 289 |
| Opening a handle to an NKI by local name | OPEN_NODE_KIND | 290 |
| Opening a handle to an RD by local name | OPEN_RELATION_DEFINITION | 291 |
| Opening a handle to an RI by local name | OPEN_RELATION | 291 |
| Opening a handle to an AD by local name | OPEN_ATTRIBUTE_-DEFINITION | 292 |
| Opening a handle to an AKD by local name | OPEN_ATTRIBUTE_KIND_-DEFINITION | 293 |
| Determining the name for a definition | NAME_OF_DEFINITION | 294 |
| Determining the NKD for a node | DEFINITION_OF_NODE | 295 |
| Determining the name for a relation | NAME_OF_RELATION | 296 |
| Determining the RD for a relation | DEFINITION_OF_RELATION | 297 |
| Determining the name for a node attribute | NAME_OF_NODE_ATTRIBUTE | 299 |
| Determining the AD for a node attribute | DEFINITION_OF_NODE_-ATTRIBUTE | 300 |
| Determining the name for a relationship attribute | NAME_OF_PATH_ATTRIBUTE | 301 |
| Determining the AD for a relationship attribute | DEFINITION_OF_PATH_-ATTRIBUTE | 302 |



Source: https://assist.dla.mil -- Downloaded: 202640-26T07:54Z
Check the source to verify that this is the current version before use.

| Package CAIS_DEFINITION_MANAGEMENT | | |
|---|---|---|
| **Description** | **Interface** | **Page** |
| Opening the predefined VIEW NKD | VIEW_NKD | 359 |
| Opening the predefined NKI NKD | NODE_KIND_-INTERPRETATION_NKD | 359 |
| Opening the predefined NKD NKD | NODE_KIND_DEFINITION_NKD | 360 |
| Opening the predefined RI NKD | RELATION_-INTERPRETATION_NKD | 360 |
| Opening the predefined RD NKD | RELATION_DEFINITION_NKD | 361 |
| Opening the predefined AD NKD | ATTRIBUTE_DEFINITION_NKD | 361 |
| Opening the predefined named composite list AKD NKD | NAMED_COMPOSITE_LIST_-AKD_NKD | 362 |
| Opening the predefined identifier AKD NKD | IDENTIFIER_ATTRIBUTE_-KIND_DEFINITION_NKD | 362 |
| Creating VIEW nodes | CREATE_VIEW | 363 |
| Deleting the primary relationships of a tree rooted at a VIEW node | DELETE_VIEW | 366 |
| Creating NKDs | CREATE_NODE_KIND | 367 |
| Deleting NKIs and NKDs | DELETE_NODE_KIND | 372 |
| Creating NKIs | IMPORT_NODE_KIND | 373 |
| Renaming NODE_KIND relationships in a view | RENAME_NODE_KIND | 377 |
| Specializing a node kind | SPECIALIZE_NODE_KIND | 378 |
| Generalizing node kinds | GENERALIZE_NODE_KIND | 380 |
| Deleting SPECIALIZATION_OF relationships from NKDs | DELETE_NODE_KIND_-SPECIALIZATION | 382 |
| Creating RDs | CREATE_RELATION | 384 |
| Deleting RIs and RDs | DELETE_RELATION | 389 |
| Creating RIs | IMPORT_RELATION | 391 |
| Renaming RELATION relationships in a view | RENAME_RELATION | 394 |
| Specializing a relation | SPECIALIZE_RELATION | 395 |
| Generalizing relations | GENERALIZE_RELATION | 397 |
| Deleting SPECIALIZATION_OF relationships from RDs | DELETE_RELATION_-SPECIALIZATION | 400 |
| Creating bidirectional RDs | CREATE_BIDIRECTIONAL_-RELATION | 402 |
| Deleting bidirectional RDs | DELETE_BIDIRECTIONAL_-RELATION | 410 |
| Creating ADs | CREATE_ATTRIBUTE_-DEFINITION | 412 |
| Deleting ADs | DELETE_ATTRIBUTE_-DEFINITION | 415 |
| Importing ADs | IMPORT_ATTRIBUTE_-DEFINITION | 417 |



Source: https://assist.dla.mil -- Downloaded: 202640-26T07:54Z
Check the source to verify that this is the current version before use.

| Package CAIS_DEFINITION_MANAGEMENT (Cont.) | | |
|---|---|---|
| **Description** | **Interface** | **Page** |
| Renaming ATTRIBUTE relationships in a view | RENAME_ATTRIBUTE_-DEFINITION | 418 |
| Redefining the AKD for an AD | REDEFINE_ATTRIBUTE_KIND | 419 |
| Creating AKDs | CREATE_ATTRIBUTE_KIND | 421 |
| Creating identifier AKDs | CREATE_IDENTIFIER_-ATTRIBUTE_KIND | 425 |
| Deleting AKDs | DELETE_ATTRIBUTE_KIND | 428 |
| Importing AKDs | IMPORT_ATTRIBUTE_KIND | 430 |
| Renaming ATTRIBUTE_KIND relationships in a view | RENAME_ATTRIBUTE_KIND | 431 |
| Deleting the primary relationships of a tree rooted at a definition node | DELETE_DEFINITION_TREE | 432 |
| Specifying source node kinds for a relation | DEFINE_SOURCE | 434 |
| Deleting source node kinds for a relation | DELETE_SOURCE | 436 |
| Respecifying source node kinds for a relation | NAME_SOURCE | 438 |
| Specifying key attributes for a relation | DEFINE_KEY | 439 |
| Deleting key attributes for a relation | DELETE_KEY | 442 |
| Respecifying key attributes for a relation | NAME_KEY | 443 |
| Specifying target node kinds for a relation | DEFINE_TARGET | 445 |
| Deleting target node kinds for a relation | DELETE_TARGET | 447 |
| Specifying attributes for node kinds and relations | DEFINE_ATTRIBUTE | 448 |
| Deleting attributes for node kinds and relations | DELETE_ATTRIBUTE | 450 |
| Respecifying attributes for node kinds and relations | NAME_ATTRIBUTE | 452 |
| Specifying an attribute kind component | DEFINE_COMPONENT | 453 |
| Deleting an attribute kind component | DELETE_COMPONENT | 455 |
| Specifying a variant attribute alternative | DEFINE_ALTERNATIVE | 457 |
| Deleting a variant attribute alternative | DELETE_ALTERNATIVE | 458 |
| Setting definition node attributes | SET_DEFINITION_ATTRIBUTE | 460 |
| Deleting definition node attributes | DELETE_DEFINITION_-ATTRIBUTE | 462 |
| Setting definition relationship attributes | SET_PROPERTY_ATTRIBUTE | 464 |
| Delete definition relationship attributes | DELETE_PROPERTY_-ATTRIBUTE | 467 |
| Changing the NKD for a node | CONVERT_NODE | 469 |
| Checking conformance of a node to an NKD | CONFORMS | 471 |
| Checking conformance of a relationship to an RD | CONFORMS | 472 |
| Checking conformance of a list to an AD or AKD | CONFORMS | 475 |



Source: https://assist.dla.mil -- Downloaded: 202640-26T07:54Z
Check the source to verify that this is the current version before use.

| Package CAIS_ATTRIBUTE_MONITOR_MANAGEMENT | | |
|---|---|---|
| **Description** | **Interface** | **Page** |
| Opening the predefined MONITOR NKD | MONITOR_NKD | 481 |
| Creating MONITOR nodes | CREATE_MONITOR_NODE | 482 |
| Deleting the primary relationship to a MONITOR node | DELETE_MONITOR_NODE | 489 |
| Associating an event handler process with a MONITOR node | ASSOCIATE_EVENT_-HANDLER_PROCESS | 490 |
| Specifying that a node attribute is to be monitored | MONITOR_NODE_ATTRIBUTE | 491 |
| Specifying that a path attribute is to be monitored | MONITOR_PATH_ATTRIBUTE | 494 |
| Cancelling monitoring of designated attributes | CANCEL_ATTRIBUTE_-MONITORING | 496 |
| Determining a monitored attribute modification | GET_MONITOR_EVENT | 497 |
| Awaiting a monitored attribute modification | AWAIT_MONITOR_EVENT | 499 |

| Package CAIS_RESOURCE_CONTROL | | |
|---|---|---|
| **Description** | **Interface** | **Page** |
| Reassigning a node to a new medium resource | REASSIGN_NODE | 503 |
| Determining the medium resource to which a node is assigned | GET_MEDIUM_RESOURCE | 504 |
| Determining the processor resource to which a process is assigned | GET_PROCESSOR_RESOURCE | 505 |
| Determining the current default medium resource | DEFAULT_MEDIUM | 506 |
| Determining the current default processor resource | DEFAULT_PROCESSOR | 506 |

| Package CAIS_PROCESS_MANAGEMENT | | |
|---|---|---|
| **Description** | **Interface** | **Page** |
| Determining the default priority | DEFAULT_PRIORITY | 524 |
| Creating a dependent PROCESS node | CREATE_PROCESS_NODE | 524 |
| Creating a root PROCESS node | CREATE_JOB_NODE | 530 |
| Activating a process | ACTIVATE_PROCESS | 535 |
| Awaiting termination of another process | AWAIT_PROCESS_-TERMINATION | 536 |
| Activating a process and awaiting its termination | ACTIVATE_AND_AWAIT_-TERMINATION | 537 |
| Spawning a new process | SPAWN_PROCESS | 538 |
| Invoking a new process | INVOKE_PROCESS | 540 |
| Creating and activating a new job | CREATE_JOB | 543 |
| Deleting a job | DELETE_JOB | 545 |



Source: https://assist.dla.mil -- Downloaded: 202640-26T07:54Z
Check the source to verify that this is the current version before use.

| Package CAIS_PROCESS_MANAGEMENT (Cont.) | | |
|---|---|---|
| **Description** | **Interface** | **Page** |
| Appending results | APPEND_RESULTS | 546 |
| Overwriting results | WRITE_RESULTS | 547 |
| Getting results from a process | GET_RESULTS | 547 |
| Determining the status of a process | CURRENT_STATUS | 548 |
| Getting the parameter list | GET_PARAMETERS | 549 |
| Aborting a process | ABORT_PROCESS | 550 |
| Suspending a process | SUSPEND_PROCESS | 551 |
| Resuming a process | RESUME_PROCESS | 552 |
| Determining the lowest specifiable priority | LOWEST_USER_PRIORITY | 553 |
| Determining the highest specifiable priority | HIGHEST_USER_PRIORITY | 553 |
| Determining if the current priority is modifiable | PRIORITIES_ARE_MODIFIABLE | 554 |
| Modifying the current priority | SET_PRIORITY | 554 |
| Determining the number of open node handles | OPEN_HANDLE_COUNT | 555 |
| Determining the number of input and output units used | IO_UNIT_COUNT | 556 |
| Determining the time of activation | TIME_STARTED | 556 |
| Determining the time of termination | TIME_FINISHED | 557 |
| Determining the time a process has been active | MACHINE_TIME | 558 |
| Determining the size of a process | PROCESS_SIZE | 558 |

| Package CAIS_TRANSACTION_MANAGEMENT | | |
|---|---|---|
| **Description** | **Interface** | **Page** |
| Opening a transaction | OPEN_TRANSACTION | 560 |
| Including a node in a transaction | INCLUDE | 561 |
| Including a tree in a transaction | INCLUDE_TREE | 562 |
| Closing a transaction | CLOSE_TRANSACTION | 563 |
| Examining the open status of a transaction | IS_OPEN | 564 |
| Querying sameness for two transactions | IS_SAME | 565 |
| Determining whether a node is included in a transaction | IS_INCLUDED | 565 |

| Package CAIS_FILE_NODE_MANAGEMENT | | |
|---|---|---|
| **Description** | **Interface** | **Page** |
| Opening the predefined SECONDARY_STORAGE NKD | SECONDARY_STORAGE_NKD | 571 |
| Creating secondary storage nodes | CREATE_SECONDARY_-STORAGE_NODE | 571 |



Source: https://assist.dla.mil -- Downloaded: 202640-26T07:54Z
Check the source to verify that this is the current version before use.

| Package CAIS_IO_CONNECTION | | |
|---|---|---|
| **Description** | **Interface** | **Page** |
| Opening the predefined PORT RD | PORT_RD | 589 |
| Creating a channel | CREATE_CHANNEL | 589 |
| Releasing a channel | RELEASE_CHANNEL | 593 |
| Deleting a CHANNEL node | DELETE_CHANNEL_NODE | 594 |
| Waiting for notification of attempted opening | AWAIT_CHANNEL | 595 |
| Opening the opposite client node | OPEN_OPPOSITE_CLIENT | 597 |
| Checking PORT relationship kind compatibility | COMPATIBLE_PORT_KINDS | 598 |
| Setting PORT relationship attributes | SET_PORT_ATTRIBUTE | 599 |

| Package CAIS_IMPORT_EXPORT | | |
|---|---|---|
| **Description** | **Interface** | **Page** |
| Importing a file from the host file system | IMPORT_CONTENTS | 602 |
| Exporting a file to the host file system | EXPORT_CONTENTS | 603 |

| Package CAIS_COMMON_IO | | |
|---|---|---|
| **Description** | **Interface** | **Page** |
| Opening the predefined INTRODUCED_NODE NKD | INTRODUCED_NODE_NKD | 604 |
| Opening the predefined INTRODUCED_NODE_RD | INTRODUCED_NODE_RD | 605 |
| Opening a node set handle | OPEN_NODE_SET | 605 |
| Including a node in a node set | INCLUDE_NODE | 606 |
| Including a tree in a node set | INCLUDE_TREE | 607 |
| Including a structure of nodes in a node set | INCLUDE_STRUCTURE | 609 |
| Transmitting a node set in common external form | TRANSMIT | 610 |
| Closing a node set | CLOSE_NODE_SET | 611 |
| Introducing nodes in common external form | INTRODUCE_NODES | 612 |
| Introducing contents of introduced nodes | INTRODUCE_CONTENTS | 616 |
| Creating VIEW nodes with specific UIDs | CREATE_VIEW_STATING_UID | 618 |
| Creating NKDs with specific UIDs | CREATE_NODE_KIND_-STATING_UID | 619 |
| Creating NKIs with specific UIDs | IMPORT_NODE_KIND_-STATING_UID | 620 |
| Creating RDs with specific UIDs | CREATE_RELATION_STATING_-UID | 621 |
| Creating RIs with specific UIDs | IMPORT_RELATION_STATING_-UID | 622 |



Source: https://assist.dla.mil -- Downloaded: 202640-26T07:54Z
Check the source to verify that this is the current version before use.

| Package CAIS_COMMON_IO (Cont.) | | |
|---|---|---|
| **Description** | **Interface** | **Page** |
| Creating bidirectional RDs with specific UIDs | CREATE_BIDIRECTIONAL_-RELATION_STATING_UID | 623 |
| Creating ADs with specific UIDs | CREATE_ATTRIBUTE_-DEFINITION_STATING_UID | 625 |
| Creating AKDs with specific UIDs | CREATE_ATTRIBUTE_KIND_-STATING_UID | 626 |
| Creating structural nodes with specific UIDs | CREATE_NODE_STATING_UID | 627 |
| Creating a secondary storage node with a specific UID | CREATE_SECONDARY_-STORAGE_NODE_STATING_UID | 628 |

| Package CAIS_FRAME_IO | | |
|---|---|---|
| **Description** | **Interface** | **Page** |
| Opening a frame file handle | OPEN | 633 |
| Closing a frame file handle | CLOSE | 634 |
| Determining whether a frame file handle is open | IS_OPEN | 635 |
| Determining the mode of a frame file handle | MODE | 635 |
| Transmitting a frame | WRITE | 636 |
| Receiving a frame | READ | 637 |
| Resetting a frame file handle | RESET | 638 |
| Determining if all messages have been read | END_OF_FILE | 639 |
| Determining if incoming queue is empty | INCOMING_QUEUE_EMPTY | 640 |
| Determining if outgoing queue is full | OUTGOING_QUEUE_FULL | 641 |
| Determining the channel node associated with a file | GET_ASSOCIATED_CHANNEL | 641 |

| Package CAIS_MAGNETIC_TAPE_IO | | |
|---|---|---|
| **Description** | **Interface** | **Page** |
| Opening a tape drive file handle | OPEN | 646 |
| Closing a tape drive file handle | CLOSE | 647 |
| Determining whether a file handle is open | IS_OPEN | 648 |
| Determining the mode of a magnetic tape drive | MODE | 649 |
| Requesting the mounting of a tape | REQUEST_MOUNT | 649 |
| Loading a tape | LOAD | 650 |
| Unloading a tape | UNLOAD | 651 |
| Requesting the dismounting of a tape | REQUEST_DISMOUNT | 651 |
| Determining the position of the tape | POSITION | 652 |
| Rewinding the tape | REWIND_TAPE | 652 |
| Skipping tape marks | SKIP_TAPE_MARK | 653 |
| Writing a tape mark | WRITE_TAPE_MARK | 654 |



Source: https://assist.dla.mil -- Downloaded: 202640-26T07:54Z
Check the source to verify that this is the current version before use.

| Package CAIS_MAGNETIC_TAPE_IO (Cont.) | | |
|---|---|---|
| **Description** | **Interface** | **Page** |
| Determining the status of a magnetic tape drive | STATUS | 654 |
| Determining the recording method of a magnetic tape | RECORDING_METHOD | 655 |
| Determining whether a write ring is installed | IS_WRITE_RING_INSTALLED | 655 |
| Skipping blocks in a magnetic tape file | SKIP_BLOCK | 656 |
| Reading a block from a magnetic tape file | READ_BLOCK | 657 |
| Writing a block to a magnetic tape file | WRITE_BLOCK | 658 |
| Resetting a magnetic tape file handle | RESET | 659 |
| Determining the channel node associated with a file | GET_ASSOCIATED_CHANNEL | 659 |

| Package CAIS_DIRECT_IO | | |
|---|---|---|
| **Description** | **Interface** | **Page** |
| Creating a direct file | CREATE | 662 |
| Opening a direct file handle | OPEN | 667 |
| Closing a direct file handle | CLOSE | 669 |
| Resetting a direct file handle | RESET | 669 |
| Synchronizing internal file with FILE node contents | SYNCHRONIZE | 670 |
| Determining the channel node associated with a file | GET_ASSOCIATED_CHANNEL | 671 |

| Package CAIS_SEQUENTIAL_IO | | |
|---|---|---|
| **Description** | **Interface** | **Page** |
| Creating a sequential file | CREATE | 673 |
| Opening a sequential file handle | OPEN | 678 |
| Closing a sequential file handle | CLOSE | 680 |
| Resetting a sequential file handle | RESET | 680 |
| Synchronizing internal file with FILE node contents | SYNCHRONIZE | 682 |
| Determining the channel node associated with a file | GET_ASSOCIATED_CHANNEL | 682 |

| Package CAIS_TEXT_IO | | |
|---|---|---|
| **Description** | **Interface** | **Page** |
| Creating a text file | CREATE | 685 |
| Opening a text file handle | OPEN | 690 |
| Closing a text file handle | CLOSE | 692 |
| Resetting a text file handle | RESET | 693 |
| Synchronizing internal file with FILE node contents | SYNCHRONIZE | 694 |
| Determining the channel node associated with a file | GET_ASSOCIATED_CHANNEL | 695 |
| Setting the maximum line length of a text file handle | SET_LINE_LENGTH | 695 |
| Setting the maximum page length of a text file handle | SET_PAGE_LENGTH | 696 |



Source: https://assist.dla.mil -- Downloaded: 202640-26T07:54Z
Check the source to verify that this is the current version before use.

| Package CAIS_TERMINAL_IO | | |
|---|---|---|
| **Description** | **Interface** | **Page** |
| Opening a terminal file handle | OPEN | 704 |
| Closing a terminal file handle | CLOSE | 706 |
| Determining whether a terminal file handle is open | IS_OPEN | 707 |
| Determining the channel node associated with a terminal file handle | GET_ASSOCIATED_CHANNEL | 707 |
| Determining the physical display origin on the virtual display | PHYSICAL_DISPLAY_ORIGIN | 708 |
| Re-orienting the physical display on the virtual display | MOVE_PHYSICAL_DISPLAY_ORIGIN | 709 |
| Determining if a terminal supports a set of graphic renditions | GRAPHIC_RENDITION_SET_SUPPORTED | 709 |
| Determining if a terminal supports a foreground color | COLOR_SUPPORTED | 710 |
| Determining if a terminal supports a background color | COLOR_SUPPORTED | 710 |
| Starting output buffering for a virtual display | BEGIN_VIRTUAL_DISPLAY_BUFFERING | 711 |
| Ending output buffering for a virtual display | END_VIRTUAL_DISPLAY_BUFFERING | 711 |
| Refreshing a virtual display | REFRESH_VIRTUAL_DISPLAY | 712 |
| Creating a screen | CREATE_SCREEN | 712 |
| Changing the active rendition for a screen | CHANGE_ACTIVE_RENDITION | 713 |
| Changing screen size | CHANGE_SCREEN_SIZE | 714 |
| Changing border characteristics | CHANGE_BORDER | 714 |
| Creating a copy of an existing screen | COPY_SCREEN | 715 |
| Deleting a screen | DELETE_SCREEN | 716 |
| Determining the size of a screen | SIZE | 716 |
| Determining the active rendition of a screen | ACTIVE_RENDITION | 716 |
| Determining whether a screen has a border | BORDER_ON | 717 |
| Determining the border rendition of a screen | BORDER_RENDITION | 717 |
| Querying sameness of screens | IS_SAME | 718 |
| Pasting a screen onto the virtual display | PASTE_SCREEN | 719 |
| Removing a screen from a virtual display | UNPASTE_SCREEN | 719 |
| Relocating a screen on a virtual display | MOVE_SCREEN | 720 |
| Move screen to front | MOVE_SCREEN_TO_FRONT | 720 |
| Move screen to back | MOVE_SCREEN_TO_BACK | 721 |
| Determining paste position of a screen | PASTE_POSITION | 721 |
| Determining whether a screen is occluded | SCREEN_OCCLUDED | 722 |
| Moving the cursor position on a screen | SET_CURSOR_POSITION | 722 |
| Determining the position of a cursor in a screen | CURSOR_POSITION | 723 |
| Writing a character to a screen | PUT | 723 |



Source: https://assist.dla.mil -- Downloaded: 202640-26T07:54Z
Check the source to verify that this is the current version before use.

| Package CAIS_TERMINAL_IO (Cont.) | | |
|---|---|---|
| **Description** | **Interface** | **Page** |
| Advancing the cursor to the next line | NEW_LINE | 724 |
| Writing a line of text to a screen | PUT_LINE | 725 |
| Inserting a character into a line | INSERT | 725 |
| Deleting characters from a line | DELETE_CHARACTERS | 726 |
| Inserting blank lines into a screen | INSERT_LINE | 727 |
| Deleting lines from a screen | DELETE_LINE | 727 |
| Erasing a region of a screen | ERASE_RANGE | 728 |
| Changing the rendition for a region of a screen | CHANGE_RENDITION | 729 |
| Determining a cell rendition | CELL_RENDITION | 729 |
| Reading characters from a screen | READ_STRING | 730 |
| Sounding the terminal bell | SOUND_BELL | 731 |
| Starting output buffering for a screen | BEGIN_SCREEN_BUFFERING | 731 |
| Ending output buffering to a screen | END_SCREEN_BUFFERING | 732 |
| Enabling function key usage | ENABLE_FUNCTION_KEYS | 732 |
| Determining function key usage | FUNCTION_KEYS_ARE_-ENABLED | 733 |
| Determining the number of function keys | NUMBER_OF_FUNCTION_KEYS | 733 |
| Determining the identification of a function key | FUNCTION_KEY_-IDENTIFICATION | 734 |
| Enabling and disabling the locator | ENABLE_LOCATOR | 734 |
| Determining locator usage | LOCATOR_IS_ENABLED | 735 |
| Determining the locator cursor position | LOCATOR_CURSOR_POSITION | 735 |
| Determining the screen containing the locator cursor | GET_SCREEN_CONTAINING_-LOCATOR_CURSOR | 736 |
| Determining locator cursor position relative to screen border | LOCATOR_CURSOR_ON_-BORDER | 737 |
| Moving the locator cursor | SET_LOCATOR_CURSOR_-POSITION | 737 |
| Determining the number of locator buttons | NUMBER_OF_BUTTONS | 738 |
| Determining the identification of a locator button | BUTTON_IDENTIFICATION | 738 |
| Reading a single input event | READ_INPUT_EVENT | 739 |
| Reading a string from a virtual keyboard | READ_STRING | 740 |
| Flushing the input event queue | FLUSH_INPUT_EVENT_QUEUE | 742 |
| Determining if all input events have been read | END_OF_FILE | 742 |
| Determining if the input event queue is empty | INPUT_QUEUE_EMPTY | 743 |



Source: https://assist.dla.mil -- Downloaded: 202640-26T07:54Z
Check the source to verify that this is the current version before use.

| Package CAIS_LIST_MANAGEMENT | | |
|---|---|---|
| **Description** | **Interface** | **Page** |
| Copying a list | COPY_LIST | 752 |
| Making a list empty | SET_TO_EMPTY_LIST | 753 |
| Converting from text to list form | CONVERT_TEXT_TO_LIST | 753 |
| Converting a list to its text representation | TEXT_FORM | 754 |
| Determining the equality of two lists | IS_EQUAL | 755 |
| Deleting an item from a linear list | DELETE | 755 |
| Determining the kind of list | KIND_OF_LIST | 756 |
| Determining the kind of list item | KIND_OF_ITEM | 757 |
| Inserting a sequence of items into a linear list | SPLICE | 758 |
| Concatenating two linear lists | CONCATENATE_LISTS | 759 |
| Extracting a sequence of items from a linear list | EXTRACT_LIST | 759 |
| Determining the length of a linear list | NUMBER_OF_ITEMS | 760 |
| Determining the position of the current linear list | POSITION_OF_CURRENT_LIST | 761 |
| Determining whether the current linear list is outermost | CURRENT_LIST_IS_OUTERMOST | 761 |
| Making the next outer linear list current | MAKE_CONTAINING_LIST_CURRENT | 761 |
| Making a nested sublist the current linear list | MAKE_THIS_ITEM_CURRENT | 762 |
| Determining the length of the text form of a list or a list item | TEXT_LENGTH | 763 |
| Determining the name of a named item | GET_ITEM_NAME | 764 |
| Determining the position of a named item | POSITION_BY_NAME | 765 |

| Package CAIS_LIST_MANAGEMENT.CAIS_LIST_ITEM | | |
|---|---|---|
| **Description** | **Interface** | **Page** |
| Extracting a list value from a list item | EXTRACT_VALUE | 766 |
| Replacing a list value in a list item | REPLACE | 767 |
| Inserting a list-valued item into a linear list | INSERT | 768 |
| Locating a list-valued item by value within a linear list | POSITION_BY_VALUE | 770 |

| Package CAIS_LIST_MANAGEMENT.CAIS_IDENTIFIER_ITEM | | |
|---|---|---|
| **Description** | **Interface** | **Page** |
| Copying a token | COPY_TOKEN | 771 |
| Converting an identifier from text to token form | CONVERT_TEXT_TO_TOKEN | 771 |
| Converting an identifier from token to text form | TEXT_FORM | 772 |
| Deleting a token | DELETE_TOKEN | 772 |
| Determining the equality of two identifier tokens | IS_EQUAL | 773 |


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

| Package CAIS_LIST_MANAGEMENT.CAIS_IDENTIFIER_ITEM (Cont.) | | |
|---|---|---|
| **Description** | **Interface** | **Page** |
| Extracting an identifier value from a list item | EXTRACT_VALUE | 773 |
| Replacing an identifier value in a list item | REPLACE | 775 |
| Inserting an identifier-valued item into a linear list | INSERT | 776 |
| Locating an identifier-valued item by value within a linear list | POSITION_BY_VALUE | 777 |
| Determining whether an identifier item is enumerated | IS_ENUMERATED | 778 |
| Determining the enumeration position of an enumerated identifier item | ENUMERATION_POSITION | 779 |

| Package CAIS_LIST_MANAGEMENT.CAIS_ENUMERATION_ITEM | | |
|---|---|---|
| **Description** | **Interface** | **Page** |
| Converting an enumeration value to its canonical text representation | TEXT_FORM | 781 |
| Extracting an enumeration value from a list item | EXTRACTED_VALUE | 781 |
| Replacing an identifier item in a list item with an enumeration value | REPLACE | 782 |
| Inserting an enumerated identifier item into a linear list | INSERT | 784 |
| Locating an identifier item by enumeration value within a linear list | POSITION_BY_VALUE | 785 |

| Package CAIS_BOOLEAN_ITEM | | |
|---|---|---|
| **Description** | **Interface** | **Page** |
| Instantiation of CAIS_LIST_MANAGEMENT. CAIS_ENUMERATION_ITEM | See CAIS_ENUMERATION_ITEM | 786 |

| Package CAIS_LIST_MANAGEMENT.CAIS_INTEGER_ITEM | | |
|---|---|---|
| **Description** | **Interface** | **Page** |
| Converting an integer value to its canonical text representation | TEXT_FORM | 787 |
| Extracting an integer value from a list item | EXTRACTED_VALUE | 788 |
| Replacing an integer value in a list item | REPLACE | 789 |
| Inserting an integer-valued item into a linear list | INSERT | 790 |
| Locating an integer-valued item by value within a linear list | POSITION_BY_VALUE | 791 |


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

| Package CAIS_LIST_MANAGEMENT.CAIS_FLOAT_ITEM | | |
|---|---|---|
| **Description** | **Interface** | **Page** |
| Converting a floating point value to its canonical text form | TEXT_FORM | 793 |
| Extracting a floating point value from a list item | EXTRACTED_VALUE | 793 |
| Replacing a floating point value in a list item | REPLACE | 794 |
| Inserting a floating point-valued item into a linear list | INSERT | 796 |
| Locating a floating point-valued item by value within a linear list | POSITION_BY_VALUE | 797 |

| Package CAIS_LIST_MANAGEMENT.CAIS_STRING_ITEM | | |
|---|---|---|
| **Description** | **Interface** | **Page** |
| Extracting a string value from a list item | EXTRACTED_VALUE | 798 |
| Replacing a string value in a list item | REPLACE | 799 |
| Inserting a string-valued item into a linear list | INSERT | 801 |
| Locating a string-valued item by value within a linear list | POSITION_BY_VALUE | 802 |

| Package CAIS_CALENDAR | | |
|---|---|---|
| **Description** | **Interface** | **Page** |
| Getting the current time | CLOCK | 808 |
| Getting the year part of the time | YEAR | 808 |
| Getting the month part of the time | MONTH | 808 |
| Getting the day part of the time | DAY | 809 |
| Getting the seconds part of the time | SECONDS | 809 |
| Splitting time into its components | SPLIT | 810 |
| Combining components of time | TIME_OF | 811 |
| Adding time and duration | "+" | 811 |
| Subtracting time and duration | "-" | 812 |
| Comparing two values of time | "<" | 812 |

| Package CAIS_STATUS_MANAGEMENT | | |
|---|---|---|
| **Description** | **Interface** | **Page** |
| Creating a status indicator | CREATE_INDICATOR | 857 |
| Setting a status indicator | SET_INDICATOR | 857 |
| Copying a status indicator | COPY_INDICATOR | 858 |
| Deleting a status indicator | DELETE_INDICATOR | 858 |
| Obtaining a status indicator's exception indication | EXCEPTION_INDICATION | 859 |
| Obtaining status information from a status indicator | STATUS_INFORMATION | 859 |


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

| Package CAIS_STATUS_MANAGEMENT (Cont.) | | |
|---|---|---|
| **Description** | **Interface** | **Page** |
| Obtaining a descriptive message from a status indicator | MESSAGE | 860 |
| Determining if a status indicator is established | IS_ESTABLISHED | 860 |

| Package CAIS_FORM_TERMINAL_IO | | |
|---|---|---|
| **Description** | **Interface** | **Page** |
| Opening a form terminal file handle | OPEN | 1332 |
| Closing a form terminal file handle | CLOSE | 1334 |
| Determining whether a file handle is open | IS_OPEN | 1334 |
| Determining the channel node associated with a file | GET_ASSOCIATED_CHANNEL | 1335 |
| Determining the number of function keys | NUMBER_OF_FUNCTION_KEYS | 1335 |
| Determining intercepted input characters | INTERCEPTED_INPUT_-CHARACTERS | 1336 |
| Determining intercepted output characters | INTERCEPTED_OUTPUT_-CHARACTERS | 1336 |
| Creating a form | CREATE_FORM | 1337 |
| Deleting a form | DELETE_FORM | 1338 |
| Copying a form | COPY_FORM | 1338 |
| Defining a qualified area | DEFINE_QUALIFIED_AREA | 1339 |
| Removing an area qualifier | REMOVE_AREA_QUALIFIER | 1339 |
| Changing the active position | SET_ACTIVE_POSITION | 1340 |
| Querying the active position | ACTIVE_POSITION | 1341 |
| Advancing forward to qualified area | ADVANCE_TO_QUALIFIED_-AREA | 1341 |
| Writing to a form | PUT | 1342 |
| Erasing a qualified area | ERASE_AREA | 1343 |
| Erasing a form | ERASE_FORM | 1343 |
| Activating a form on a terminal | ACTIVATE | 1344 |
| Reading from a form | GET | 1344 |
| Determining changes to a form | IS_FORM_UPDATED | 1345 |
| Determining the identification of a function key | FUNCTION_KEY_-IDENTIFICATION | 1346 |
| Determining the termination key | TERMINATION_KEY | 1346 |
| Determining the size of a form | FORM_SIZE | 1347 |
| Determining the size of the terminal | TERMINAL_SIZE | 1347 |
| Determining if the area qualifier requires space in the form | AREA_QUALIFIER_REQUIRES_-SPACE | 1348 |
| Determining if the area qualifier requires space on a terminal | AREA_QUALIFIER_REQUIRES_-SPACE | 1348 |


Source: https://assist.dla.mil -- Downloaded: 2026l0-26T07:54Z
Check the source to verify that this is the currentversion before use.

# Index

## A

abbreviations 7
abort (for a process) 7
abort (for a transaction) 7
ABORT_PROCESS 550
aborted 509
access 8, 62
  checking 8, 62, 262
  control 8, 62, 255
  synchronization 516
  through a generalization 58
  to a node 8, 96, 255
ACCESS relationships 259
access rights 8, 62, 96
  node contents 567
accessible 8, 62
accessing a node 62
ACCESS_ RIGHTS 271
ACCESS_ VIOLATION 92
acronyms 7
activate 8
ACTIVATE 1360
ACTIVATE_AND_AWAIT_TERMINATION 509, 537
ACTIVATE_PROCESS 509, 535
ACTIVE_POSITION 1357,
active rendition 8
ACTIVE_RENDITION 716
active row 8
AD
  operations 358
  predefine name 358
Ada external files 40
Ada Programming Support Environment 8
adopt a role 8, 96
adopt a view 8
adopted interpretation 60
adopted node kind interpretation 9, 355
adopted roles 258
adopted view 60
adopts 258
ADOPT_INTERPRETATION 286
ADOPT_ROLE 277
ADOPT_VIEW 284
ADs 185
  deleting 337
ADVANCE_TO_QUALIFIED_AREA 1357
AFTER_TAPE_MARK 644
AKD 315
  operations 358
  predefined name 358
AKDs
  deleting 337
allocatability relationships 9, 80
allocatable 9
ALLOCATABLE_DEVICE 80
ALLOCATABLE_GATEWAY 80
ALLOCATABLE_MEDIUM 80
ALLOCATABLE_PROCESSOR 80
allowed protocol 576
ALL_ ATTRIBUTES 246
ALL_RIGHTS 271
ALLOWED_PROTOCOLS 575, 582
ALPHABETIC 1348
alternative 329
alternative constraint items 332
alternative of a variant 9
ALTERNATIVE relationships 322
ambiguous formation 9
ambiguous translation 9
ambiguous view 356
AMBIGUOUS_VIEW_ERROR 92, 356
AND_THEN 164
APPEND_CONTENTS 82
APPEND_FILE 632
APPEND_INTENT 569
APPEND_RESULTS 546
applicable naming 9, 355
approved access rights 9, 262
APPROXIMATE_SIZE 169, 174, 181, 250
APSE 1, 8
AREA_INPUT_KIND 1348
AREA_INTENSITY_KIND 1348
AREA_PROTECTION_KIND 1348
AREA_QUALIFIER_REQUIRES_SPACE 1364
AREA_VALUE_KIND 1348
assignment of resources 9
ASSOCIATE_EVENT_HANDLER_ PROCESS 479, 490
AS_ A_ TRANSACTION 518



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

attribute 9
attribute definition node (AD) 10
attribute definition 9, 55
attribute iteration 10, 158
attribute iterator 10, 246
attribute kind definition node (AKD) 10, 43, 55
attribute kind 10, 43
attribute monitor 10, 61
  event 477
  event handler process 478
  event queue 477
attribute names 43, 50
attribute value types 43
attribute value 43
attributes 43
  ALLOWED_PROTOCOLS 575, 582
  ATTRIBUTES_MONITORED 477
  AVAILABLE 78, 82
  CONTROLLING_PROTOCOL 582
  CURRENT_STATUS 478
  DEFAULT_VALUE 334
  DIRECTION 575, 576, 580
  HANDLER_MANDATORY_ACCESS 478
  MAXIMUM_INSTANCES 308
  MESSAGE_LENGTH_LIMIT 582
  MINIMUM_INSTANCES 308
  name 43, 347, 349
  OBJECT_CLASSIFICATION 478
  PATTERNS 163
  STATE 577
  SUBJECT_CLASSIFICATION 478
ATTRIBUTE_DEFINITION 358
ATTRIBUTE_DEFINITION_NKD 361
ATTRIBUTE_ERROR 92
ATTRIBUTE_ITERATOR_SCOPE 246
ATTRIBUTE_KIND_NAME 90
ATTRIBUTE_LIST 90
ATTRIBUTE_ MONITOR 477
ATTRIBUTE_NAME 90
ATTRIBUTE_NAME_PATTERN 246
ATTRIBUTES_MONITORED 477
available 10
AVAILABLE attributes 78
AWAIT_CHANNEL 578, 584, 595
AWAIT_MONITOR_EVENT 478, 481, 499
AWAIT_PROCESS_TERMINATION 536

## B

BACKGROUND_COLOR_TYPE 703
base node 10
BASE_PATH 121
basic node kind definition node 10
basic relation definition node 10
BEGINNING_OF_VOLUME 644
BEGIN_SCREEN_BUFFERING 731
BEGIN_VIRTUAL_DISPLAY_BUFFERING 711
bidirectional relationships 10, 41
BOLD 703
Boolean AKD 322
BORDER_ON 717
BORDER_RENDITION 717
BUTTON_IDENTIFICATION 738
BUTTON_NAME 704
BUTTON_NAME_ERROR 704

## C

CAIS extensions 346
CAIS implementation 11
CAIS instance 11
CAIS interface 11
CAIS_controlled definitions 11, 44
CAIS_controlled instance 11
CAIS_controlled 342
CAIS_determined definition 11
CAIS_determined instance 11
CAIS_determined 345
CAIS_ACCESS_CONTROL_MANAGEMENT 87, 255
CAIS_ATTRIBUTE_MANAGEMENT 87, 185
CAIS_ATTRIBUTE_MONITOR_MANAGEMENT 87, 477
CAIS_BOOLEAN_ITEM 786
CAIS_CALENDAR 807
CAIS_CALENDAR_TIME 71
CAIS_COMMON_IO 567, 604
CAIS_DEFINITION_MANAGEMENT 87, 95, 304
CAIS_DEFINITIONS 87, 88, 95, 185, 280
CAIS_DIRECT_IO 64, 629, 661
CAIS_DURATION 805
CAIS_ENUMERATION_ITEM 780
CAIS_FILE_NODE_MANAGEMENT 567, 571
CAIS_FLOAT_ITEM 792
CAIS_FORM_TERMINAL_IO 1346
CAIS_FRAME_IO 64, 629, 630



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

CAIS_IDENTIFIER_ITEM 771
CAIS_IMPORT_EXPORT 567, 602
CAIS_INTEGER 162, 805
CAIS_INTEGER_ITEM 787
CAIS_IO_CONNECTION 567, 575
CAIS_IO_DEFINITIONS 567, 569
CAIS_IO_DURATION 569
CAIS_LIST_ITEM 766
CAIS_LIST_MANAGEMENT 70, 159, 185, 751
CAIS_MAGNETIC_TAPE_IO 64, 629, 643
CAIS_NATURAL 805
CAIS_NODE_MANAGEMENT 87, 95
CAIS_POSITIVE 805
CAIS_PRAGMATICS 95, 185, 280, 570, 815
CAIS_PROCESS_DEFINITIONS 520
CAIS_PROCESS_MANAGEMENT 478, 521
CAIS_RESOURCE_CONTROL 87, 503
CAIS_SEQUENTIAL_IO 64, 629, 672
CAIS_STANDARD 805
CAIS_STATUS_MANAGEMENT 857
CAIS_STRING_ITEM 798
CAIS_STRUCTURAL_NODE_MANAGEMENT 87, 95, 280
CAIS_TERMINAL_IO 64, 629, 698
CAIS_TEXT_IO 64, 629, 684
CAIS_TRANSACTION_MANAGEMENT 560
CAIS_TRANSLATION_MANAGEMENT 87,95, 284
CANCEL_ATTRIBUTE_MONITORING 477,496
cannot be formed 11
cannot be translated 11
canonical list text representation 11, 749
CAPACITY_ERROR 570, 815
CELL_RENDITION 729
CHANGE_ACTIVE_RENDITION 713
CHANGE_BORDER 714
CHANGE_INTENT 95, 97, 108
CHANGE_RENDITION 729
CHANGE_SCREEN_SIZE 714
channel 11, 64
channel creation 11, 65
channel deletion 11
CHANNEL nodes 65, 66, 68, 575, 577, 578, 582
channel pairs 11, 85
CHANNEL relationships 577
CHANNEL_STATE 569
character cell 12
CHARACTER_ARRAY 1347
classification 256
client nodes 12, 65, 582
client PROCESS node 12, 65
client processes 12, 65
CLIENT_ERROR 570
CLOCK 808
close request 12, 587
CLOSE 95, 107, 1350, 634, 647, 669, 680, 692, 706
closed node handle 12, 87
closed protocol 12
closed transaction handle 12, 518
CLOSED 644
CLOSE_NODE_SET 611
CLOSE_TRANSACTION 517, 563
closing processes 12, 67
closing protocols 67
COLOR_SUPPORTED 710
commit 12
COMMIT_MODIFICATIONS 517
common external form (CEF) 12, 69
common I/O 70
compatibility 69, 578
  checks 69, 578, 579
  DIRECTION attribute 579
  PORT relation 579
  protocol 581
  mode 581
COMPATIBLE_PORT_KINDS 598
completed 12, 509
completed abnormally 12
completed normally 12
component constraint items 333
components 13, 317
composite lists 13, 317
composite view 13, 60, 356
CONCATENATE_LISTS 759
condition code 825
CONDITION_CODE 847
CONFORMANCE_ERROR 92
CONFORMS 471, 472, 475
connected 13
CONNECTED state 577, 583
constraint item 13, 329
constraints list 13, 329
constraints
  stronger 58
  weaker 58



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

content descriptor 13, 71
content section 13, 71
contents 13, 40
of a FILE node 40
of a PROCESS node 40
of a STRUCTURAL node 40
controlling protocol 14, 582
CONTROLLING_PROTOCOL 577, 582
CONVERT_NODE 469
CONVERT_TEXT_TO_LIST 753
CONVERT_TEXT_TO_TOKEN 771
COPY_FORM 1354
COPY_INDICATOR 858
COPY_LIST 752
COPY_NODE 126
COPY_SCREEN 715
COPY_TOKEN 771
COPY_TREE 129
corresponding
definition 347, 349
definition node 14
reverse relationship 14, 41
CREATE 578, 662, 673, 685
created 14
CREATE_ATTRIBUTE_DEFINITION 412
CREATE_ATTRIBUTE_DEFINITION_STATING_-UID 625
CREATE_ATTRIBUTE_KIND 421
CREATE_ATTRIBUTE_KIND_STATING_UID 626
CREATE_BIDIRECTIONAL_SECONDARY_-RELATIONSHIP 143
CREATE_CHANNEL 578, 589
CREATE_FORM 1353
CREATE_IDENTIFIER_ATTWBUTE_KIND 425
CREATE_INDICATOR 857
CREATE_ITERATOR 169, 178
CREATE_JOB 478, 481, 510, 518, 543
CREATE_JOB_NODE 509, 518, 530
CREATE_MONITOR_NODE 477, 479, 482
CREATE_NODE 280
CREATE_NODE_ATTRIBUTE 186, 198, 236, 226, 207, 217
CREATE_NODE_ATTNBUTE_ITERATOR 246
CREATE_NODE_KIND 367
CREATE_NODE_KIND_STATING_UID 619
CREATE_NODE_STATING_UID 627
CREATE_PATH_ATTRIBUTE 187, 199, 209, 218, 227, 237
CREATE_PATH_ATTRIBUTE_ITERATOR 247
CREATE_PROCESS_NODE 509, 518, 524
CREATE_RELATION 384
CREATE_RELATION_STATING_UID 621
CREATE_SCREEN 712
CREATE_SECONDARY_RELATIONSHIP 141
CREATE_SECONDARY_STORAGE_NODE 571
CREATE_SECONDARY_STORAGE_NODE_-STATING_UID 628
CREATE_VIEW 363
CREATE_VIEW_STATING_UID 618
creating channels 65, 67
creating process 14
creating processes 68
current adopted interpretation 14, 355
current job 14
current linear list 14, 748
current node 14
current process 14, 48
current PROCESS node 14
current processor 14
current PROCESSOR node 14, 78
current user 14
current USER node 15
CURRENT_ LIST_ IS_ OUTERMOST 761
CURRENT_JOB 46
CURRENT_NODE 46, 91
CURRENT_PROCESS 91
CURRENT_ STATUS 478, 513, 548
CURRENT_USER 46, 91
cursor position 15
CURSOR_ POSITION 723

D

data stream 15
data transfer protocol 15, 64
database management system 1
DATA_ERROR 570
DAY 809
DAY_DURATION 807
DAY_NUMBER 807
default GROUP node 15, 258
default rendition 15
default roles 258
DEFAULT_MEDIUM 80, 82, 506
DEFAULT_PREDICATE 168
DEFAULT_PRIORITY 524



Source: https://assist.dla.mil -- Downloaded: 2026l0-26T07:54Z
Check the source to verify that this is the currentversion before use.

DEFAULT_PROCESSOR 80, 82, 506
DEFAULT_RELATION 91
DEFINE_ALTERNATIVE 457
DEFINE_ATTRIBUTE 448
DEFINE_COMPONENT 453
DEFINE_KEY 439
DEFINE_QUALIFIED_AREA 1355
DEFINE_SOURCE 434
DEFINE_TARGET 445
definition 15, 55
  terminology 59
definition names 50
definition node 15, 55, 96, 351, 358
DEFINITION_ERROR 92
DEFINITION_NAME 284
DEFINITION_OF_PATH_ATTRIBUTE 302
DEFINITION_OF_NODE 295
DEFINITION_OF_NODE_ATTRIBUTE 300
DEFINITION_OF_RELATION 297
delay 95, 96, 97
DELETE 755
delete request 15, 587
DELETE_ALTERNATIVE 458
DELETE_ATTRIBUTE 450
DELETE_ATTRIBUTE_DEFINITION 415
DELETE_ATTRIBUTE_KIND 428
DELETE_BIDIRECTIONAL_RELATION 410
DELETE_CHANNEL_NODE 578, 594
DELETE_CHARACTERS 726
DELETE_COMPONENT 455
DELETE_DEFINITION_ATTRIBUTE 462
DELETE_DEFINITION_TREE 432
DELETE_DENY_RELATIONSHIP 262, 275
DELETE_FORM 1354
DELETE_GRANTED_RIGHTS 273
DELETE_INDICATOR 858
DELETE_ITERATOR 178, 184, 253
DELETE_JOB 545
DELETE_KEY 442
DELETE_LINE 727
DELETE_MONITOR_NODE 489
DELETE_NODE 138
DELETE_NODE_ATTRIBUTE 189
DELETE_NODE_KIND 372
DELETE_NODE_KIND_SPECIALIZATION 382
DELETE_PATH_ATTRIBUTE 190
DELETE_PROPERTY_ATTRIBUTE 467
DELETE_RELATION 389
DELETE_RELATION_SPECIALIZATION 400
DELETE_SCREEN 716
DELETE_SECONDARY_RELATIONSHIP 146
DELETE_SOURCE 436
DELETE_TARGET 447
DELETE_TOKEN 772
DELETE_TREE 139
DELETE_VIEW 366
DENY 262
denying access 262
denying access rights 62
DENY_ACCESS 262, 275
dependent process 15
dependent PROCESS nodes 46
dependent processes 46
descendant (of a process) 15
descendant (of a transaction) 15
device 15
DEVICE 66, 82
device name 15
DEVICE nodes 16, 65, 67, 69, 77, 78, 80, 575, 582
device resource 16, 76
DEVICE_ERROR 92
direct operation 16
direct scenario 16, 578
DIRECTION attributes 575, 576, 580
discretionary access checking 262
discretionary access control 16, 33, 62
DISCRETIONARY_ACCESS_LIST 90
distributed CAIS implementations 33
DOT 53

## E

ECHO_TYPE 704
effects
  eligible 514
element 16
eligible effects 16, 514, 518
emanating relations 16, 308
EMPTY 751
empty list 16, 745
EMPTY_LIST 751
ENABLE_FUNCTION_KEYS 732
ENABLE_LOCATOR 734
enclosed process 514, 518
enclosing transaction 16, 44, 514, 518
ENCLOSING_TRANSACTION 518



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

END_ERROR 570
END_OF_FILE 639, 742
END_OF_TAPE 644
END_OF_VOLUME 644
END_ SCREEN_BUFFERING 732
END_VIRTUAL_DISPLAY_BUFFERING 711
enumerated identifier AKD 322
enumerated identifier item 746
enumeration AKD 322
enumeration form 16, 746
enumeration item 746
ENUMERATION_POSITION 779
ERASE_AREA 1359
ERASE_FORM 1359
ERASE_ RANGE 728
errors 92
ESTABLISHED state 583
established status indicator 16, 825
event handler process 16, 477, 478
  selection 478
exception indication 825
EXCEPTION_INDICATION 859
EXCEPTION_INDICATION_KIND 845
EXCLUSIVE_READ 97, 516
EXCLUSIVE_WRITE 97, 516
EXECUTABLE_IMAGE_ERROR 520
EXISTING_NODE_ERROR 92
exportation 17, 70
EXPORT_CONTENTS 603
external file 17
external node set 17, 70, 71
EXTRACT_LIST 759
EXTRACT_VALUE 164, 166, 766, 773
EXTRACTED_VALUE 781, 788, 793, 798

## F

FAINT 703
feasible CHANNEL NKD subset 17
file 17
FILE 66
file handles 17, 567
FILE nodes 17, 40, 65, 69, 575, 582
  contents 40
FILE_MODE 632, 644, 662, 673, 685
FILE_TYPE 567, 632, 644, 662, 673, 685, 701, 1347
FILL_WITH_SPACES 1348
FILL_WITH_ZEROES 1348
float AKD 318
FLOAT_ATTRIBUTE_KIND_DEFINITION 358
FLOAT_ITEM_KIND 751
FLUSH_INPUT_EVENT_QUEUE 742
FOREGROUND_COLOR_TYPE 703
FORM_SIZE 1363
FORM_STATUS_ERROR 1347
FORM_TYPE 1347
frame 17
FRAME_ADDRESS 632
FRAME_LENGTH 632
FUNCTION_KEYS_ARE_ENABLED 733
FUNCTION_KEY_IDENTIFICATION 734, 1362
FUNCTION_KEY_NAME 704, 1347
FUNCTION_KEY_STATUS_ERROR 704, 1347

## G

GATEWAY nodes 17,65,69, 77, 78,80,85,575, 582, 583
gateway pairs 17, 85
gateway resource 17, 76
GATEWAY 66, 82
generalization 17, 58
GENERALIZE_NODE_KIND 380
GENERALIZE_RELATION 397
GET 513, 1360
GET_ASSOCIATED_CHANNEL 641,659,671, 682, 695, 707, 1351
GET_CURRENT_NODE 95, 155
GET_GRANTED_RIGHTS 274
GET_ITEM_NAME 764
GET_MEDIUM_RESOURCE 504
GET_MONITOR_EVENT 478, 481, 497
GET_NEXT 95, 169, 175, 181
GET_NEXT_VALUE 252
GET_NODE_ATTRIBUTE 195, 232
GET_PARAMETERS 549
GET_PATH_ATTRIBUTE 196, 234
GET_PROCESSOR_RESOURCE 505
GET_RESULTS 547
GET_REVERSE_KEY 150
GET_REVERSE_RELATION 148
GET_SCREEN_CONTAINING_LOCATOR_-CURSOR 736
granting access rights 62
GRAPHIC_CHARACTERS 1348
GRAPHIC_RENDITION_KIND 703
GRAPHIC_RENDITION_SET 703



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

GRAPHIC_RENDITION_SET_SUPPORTED 709
group 17, 257
group name 17, 257
GROUP nodes 17, 257
GROUP_CODED_RECORDING 644

## H

HANDLER_IMAGE 478
HANDLER_MANDATORY_ACCESS 478
HANDLER_PROCESS 478
HANDLER_PROCESS_OF 478
HAS_EMANATING 166, 180
header section 18, 71
hierarchical classification level 263
HIGH 1348
HIGHEST_USER_PRIORITY 553

## I

identification 50
    of a node 18
    of a relationship 18
    of an attribute 18
identifier AKD 320
identifier equality 18
identifier text form 18, 746
IDENTIFIER_ATTRIBUTE_KIND_DEFINITION 358
IDENTIFIER_ATTRIBUTE_KIND_DEFINITION_-NKD 362
IDENTIFIER_ITEM_KIND 751
IDENTIFIER_TEXT 751
illegal identification 18, 51
implicit inclusion 515
implicit open operation 96
importation 18, 70
IMPORTED_DEFINITION_UID 74
IMPORTED_NAME 74
IMPORTED_RELATIONSHIP_KIND 74
IMPORTED_TARGET_UID 74
IMPORTED_UID 74
importing 74
IMPORT_ATTRIBUTE_DEFINITION 417
IMPORT_ATTRIBUTE_KIND 430
IMPORT_CONTENTS 602
IMPORT_NODE_KIND 373
IMPORT_NODE_KIND_STATING_UID 620
IMPORT_RELATION 391
IMPORT_RELATION_STATING_UID 622
inaccessible 18, 62
include 18
INCLUDE 515, 561
included node 515
included operation 515
INCLUDE_NODE 606
INCLUDE_STRUCTURE 609
INCLUDE_TREE 515, 562, 607
incoming queue 19
INCOMING_QUEUE_EMPTY 640
INDEX 124
INFINITE_IO_DELAY 569
inheritable 19, 42
inheritante
    of constraints 58, 59
    of properties 58, 59
initiate 19
initiated process 19, 43
initiating process 19, 43
INOUT_FILE 632
INOUT_INTENT 569
input and output connection model 19, 65
input and output 2, 33, 64
input event 19
input event queue 19
INPUT_EVENT_KIND 704
INPUT_EVENT_QUEUE_EMPTY 704
INPUT_QUEUE_EMPTY 743
INSERT 725, 768, 776, 784, 790, 796, 801
INSERT_LINE 727
INSERT_COUNT 751
instance (of a definition) 19
instance iteration 19, 158
integer AKD 318
INTEGER_ATTRIBUTE_KIND_DEFINITION 358
INTEGER_ITEM_KIND 751
integration 19, 74
INTENT 88, 95, 109
intents 19, 98
    NO_ACCESS 68, 97
INTENT_ARRAY 88
INTENT_SPECIFICATION 88
INTENT_VIOLATION 92
INTERCEPTED_INPUT_CHARACTERS 1352
INTERCEPTED_OUTPUT_CHARACTERS 1352
interfaces 19, 33
    ABORT_PROCESS 550



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

ACTIVATE 1360
ACTIVATE_AND_AWAIT_TERMINATION 537
ACTIVATE_PROCESS 535
ACTIVE_POSITION 1357
ACTIVE_RENDITION 716
ADOPT_INTERPRETATION 286
ADOPT_ROLE 277
ADOPT_VIEW 284
ADVANCE_TO_QUALIFIED_AREA 1357
ALL_RIGHTS 271
APPEND_RESULTS 546
APPROXIMATE_SIZE 174, 181, 250
AREA_QUALIFIER_REQUIRES_SPACE 1364
ASSOCIATE_EVENT_HANDLER_PROCESS 490
ATTRIBUTE_DEFINITION_NKD 361
AWAIT_CHANNEL 595
AWAIT_MONITOR_EVENT 499
AWAIT_PROCESS_TERMINATION 536
BASE_PATH 121
BEGIN_SCREEN_BUFFERING 731
BEGIN_VIRTUAL_DISPLAY_BUFFERING 711
BORDER_ON 717
BORDER_RENDITION 717
BUTTON_IDENTIFICATION 738
CANCEL_ATTRIBUTE_MONITORING 496
CELL_RENDITION 729
CHANGE_ACTIVE_RENDITION 713
CHANGE_BORDER 714
CHANGE_INTENT 95, 108
CHANGE_RENDITION 729
CHANGE_SCREEN_SIZE 714
CLOCK 808
CLOSE 95, 107, 634, 647, 669, 680, 692, 706, 1350
CLOSE_NODE_SET 611
CLOSE_TRANSACTION 563
COLOR_SUPPORTED 710
COMPATIBLE_PORT_KINDS 598
CONCATENATE_LISTS 759
CONFORMS 471, 472, 475
CONVERT_NODE 469
CONVERT_TEXT_TO_LIST 753
CONVERT_TEXT_TO_TOKEN 771
COPY_FORM 1354
COPY_INDICATOR 858
COPY_LIST 752
COPY_NODE 126
COPY_SCREEN 715
COPY_TOKEN 771
COPY_TREE 129
CREATE 662, 673, 685
CREATE_ATTRIBUTE_DEFINITION 412
CREATE_ATTRIBUTE_DEFINITION_-STATING_UID 625
CREATE_ATTRIBUTE_KIND 421
CREATE_ATTRIBUTE_KIND_STATING_-UID 626
CREATE_BIDIRECTIONAL_RELATION 402
CREATE_BIDIRECTIONAL_RELATION_-STATING_UID 623
CREATE_BIDIRECTIONAL_SECONDARY_-RELATIONSHIP 143
CREATE_CHANNEL 589
CREATE_FORM 1353
CREATE_IDENTIFIER_ATTRIBUTE_KIND 425
CREATE_INDICATOR 857
CREATE_ITERATOR 169, 178
CREATE_JOB 543
CREATE_JOB_NODE 530
CREATE_MONITOR_NODE 482
CREATE_NODE 280
CREATE_NODE_ATTRIBUTE 186, 198,207, 217, 226, 236
CREATE_NODE_ATTRIBUTE_ITERATOR 246
CREATE_NODE_KIND 367
CREATE_NODE_KIND_STATING_UID 619
CREATE_NODE_STATING_UID 627
CREATE_PATH_ATTRIBUTE 187, 199,209, 218, 227, 237
CREATE_PATH_ATTRIBUTE_ITERATORTOR 247
CREATE_PROCESS_NODE 524
CREATE_RELATION 384
CREATE_RELATION_STATING_UID 621
CREATE_SCREEN 712
CREATE_SECONDARY_RELATIONSHIP 141
CREATE_SECONDARY_STORAGE_NODE 571
CREATE_SECONDARY_STORAGE_NODE_-STATING_UID 628



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

CREATE_VIEW 363
CREATE_VIEW_STATING_UID 618
CURRENT_LIST_IS_OUTERMOST 761
CURRENT_STATUS 548
CURSOR_POSITION 723
DAY 809
DEFAULT_MEDIUM 506
DEFAULT_PREDICATE 168
DEFAULT_PRIORITY 524
DEFAULT_PROCESSOR 506
DEFINE_ALTERNATIVE 457
DEFINE_ATTRIBUTE 448
DEFINE_COMPONENT 453
DEFINE_KEY 439
DEFINE_QUALIFIED_AREA 1355
DEFINE_SOURCE 434
DEFINE_TARGET 445
DEFINITION_OF_NODE 295
DEFINITION_OF_NODE_ATTRIBUTE 300
DEFINITION_OF_PATH_ATTRIBUTE 302
DEFINITION_OF_RELATION 297
DELETE 755
DELETE_ALTERNATIVE 458
DELETE_ATTRIBUTE 450
DELETE_ATTRIBUTE_DEFINITION 415
DELETE_ATTRIBUTE_KIND 428
DELETE_BIDIRECTIONAL_RELATION 410
DELETE_CHANNEL_NODE 594
DELETE_CHARACTERS 726
DELETE_COMPONENT 455
DELETE_DEFINITION_ATTRIBUTE 462
DELETE_DEFINITION_TREE 432
DELETE_DENY_RELATIONSHIP 275
DELETE_FORM 1354
DELETE_GRANTED_RIGHTS 273
DELETE_INDICATOR 858
DELETE_ITERATOR 178, 184, 253
DELETE_JOB 545
DELETE_KEY 442
DELETE_LINE 727
DELETE_MONITOR_NODE 489
DELETE_NODE 138
DELETE_NODE_ATTRIBUTE 189
DELETE_NODE_KIND 372
DELETE_NODE_KIND_SPECIALIZATION 382
DELETE_PATH_ATTRIBUTE 190
DELETE_PROPERTY_ATTRIBUTE 467
DELETE_RELATION 389
DELETE_RELATION_SPECIALIZATION 400
DELETE_SCREEN 716
DELETE_SECONDARY_RELATIONSHIP 146
DELETE_SOURCE 436
DELETE_TARGET 447
DELETE_TOKEN 772
DELETE_TREE 139
DELETE_VIEW 366
DENY_ACCESS 275
ENABLE_FUNCTION_KEYS 732
ENABLE_LOCATOR 734
END_OF_FILE 639, 742
END_SCREEN_BUFFERING 732
END_VIRTUAL_DISPLAY_BUFFERING 711
ENUMERATION_POSITION 779
ERASE_AREA 1359
ERASE_FORM 1359
ERASE_RANGE 728
EXCEPTION_INDICATION 859
EXPORT_CONTENTS 603
EXTRACT_LIST 759
EXTRACT_VALUE 766, 766, 773
EXTRACTED_VALUE 781, 788, 793, 798
FLUSH_INPUT_EVENT_ QUEUE 742
FORM_SIZE 1363
FUNCTION_KEY_IDENTIFICATION 734, 1362
FUNCTION_KEYS_ARE_ENABLED 733
GENERALIZE_NODE_KIND 380
GENERALIZE_RELATION 397
GET 1360
GET_ASSOCIATED_CHANNEL 641, 659, 671, 682, 695, 707, 1351
GET_CURRENT_NODE 95, 155
GET_GRANTED_RIGHTS 274
GET_ITEM_NAME 764
GET_MEDIUM_RESOURCE 504
GET_MONITOR_EVENT 497
GET_NEXT 95, 175, 181
GET_NEXT_VALUE 252
GET_NODE_ATTRIBUTE 195, 232
GET_PARAMETERS 549
GET_PATH_ATTRIBUTE 196, 234
GET_PROCESSOR_RESOURCE 505
GET_RESULTS 547



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

GET_REVERSE_KEY 150
GET_REVERSE_RELATION 148
GET_SCREEN_CONTAINING_LOCATOR_-
CURSOR 736
GRAPHIC_RENDITION_SET_SUPPORTED
709
HIGHEST_USER_PRIORITY 553
IDENTIFIER_ATTRIBUTE_KIND_DEFINITION_-
NKD 362
IMPORT_ATTRIBUTE_DEFINITION 417
IMPORT_ATTRIBUTE_KIND 430
IMPORT_CONTENTS 602
IMPORT_NODE_KIND 373
IMPORT_NODE_KIND_STATING_UID 620
IMPORT_RELATION 391
lMPORT_RELATION_STATING_UID 622
INCLUDE 561
INCLUDE_NODE 606
INCLUDE_STRUCTURE 609
INCLUDE_TREE 562, 607
INCOMING_QUEUE_EMPTY 640
INDEX 124
INPUT_QUEUE_EMPTY 743
INSERT 725, 768, 776, 784, 790, 796, 801
INSERT_LINE 727
INTENT 109
INTERCEPTED_INPUT_CHARACTERS 1352
INTERCEPTED_OUTPUT_CHARACTERS
1352
INTRODUCE_CONTENTS 616
INTRODUCE_NODES 612
INTRODUCED_NODE_NKD 604
INTRODUCED_NODE_RD 605
INVOKE_PROCESS 540
IO_UNIT_COUNT 556
IS_APPROVED 276
IS_ENUMERATED 778
IS_EQUAL 755, 773
IS_ESTABLISHED 860
IS_FORM_UPDATED 1361
IS_INCLUDED 565
IS_INHERITABLE 153
IS_OBTAINABLE 123
IS_OF_KIND 113
IS_OF_RELATION 114
IS_OPEN 109, 564, 635, 648, 707, 1350
IS_SAME 124, 565, 718
IS_SUB_KIND 110
IS_SUB_RELATION 112
IS_WRITE_RING_INSTALLED 655
KIND_OF_ITEM 757
KIND_OF_LIST 756
KIND_OF_NODE 110
LAST_KEY 122
LAST_RELATION 122
LOAD 650
LOCATOR_CURSOR_ON_BORDER 737
LOCATOR_CURSOR_POSITION 735
LOCATOR_IS_ENABLED 735
LOWEST_USER_PRIORITY 553
MACHINE_TIME 558
MAKE_CONTAINING_LIST_CURRENT 761
MAKE_THIS_ITEM_CURRENT 762
MESSAGE 860
MODE 635, 649
MONITOR_NKD 481
MONITOR_NODE_ATTRIBUTE 491
MONITOR_PATH_ATTRIBUTE 494
MONTH 808
MORE 173, 180, 250
MOVE_PHYSICAL_DISPLAY_ORIGIN 709
MOVE_ SCREEN 720
MOVE_SCREEN_TO_BACK 721
MOVE_SCREEN_TO_FRONT 720
NAME_ATTRIBUTE 452
NAME_KEY 443
NAME_OF_DEFINITION 294
NAME_OF_NODE_ATTRIBUTE 299
NAME_OF_PATH_ATTRIBUTE 301
NAME_OF_RELATION 296
NAME_SOURCE 438
NAMED_COMPOSITE_LIST_AKD_NKD 362
NEW_LINE 724
NEXT_NAME 177, 251
NODE_ATTRIBUTE 204, 214, 223, 242
NODE_KIND_DEFINITION_NKD 360
NODE_KIND_INTERPRETATION_NKD 359
NUMBER_OF_BUTTONS 738
NUMBER_OF_FUNCTION_KEYS 733, 1351
NUMBER_OF_ITEMS 760
OPEN 95, 103,580, 633,646, 667, 678,690,
704, 1348
OPEN_ATTRIBUTE_DEFINITION 292
OPEN_ATTRIBUTE_KIND_DEFINITION
293
OPEN_FILE_HANDLE_COUNT 116



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

OPEN_HANDLE_COUNT 555
OPEN_NODE_KIND 290
OPEN_NODE_KIND_DEFINITION 289
OPEN_NODE_SET 605
OPEN_OPPOSITE_CLIENT 597
OPEN_PARENT 95, 125
OPEN_RELATION 291
OPEN_RELATION_DEFINITION 291
OPEN_TRANSACTION 560
OPEN_VIA_UNIQUE_IDENTIFIER 95,106
OUTGOING_QUEUE_FULL 641
PASTE_POSITION 721
PASTE_SCREEN 719
PATH_ATTRIBUTE 206, 215, 225, 243
PATH_KEY 119
PATH_RELATION 120
PHYSICAL_DISPLAY_ORIGIN 708
PORT_RD 589
POSITION 652
POSITION_BY_NAME 765
POSITION_BY_VALUE 770,777,785,791, 797, 802
POSITION_OF_CURRENT_LIST 761
PRIMARY_KEY 118
PRIMARY_NAME 117
PRIMARY_RELATION 118
PRIORITIES_ARE_MODIFIABLE 554
PROCESS_SIZE 558
PUT 723, 1358
PU_LINE 725
READ 637
READ_BLOCK 657
READ_INPUT_EVENT 739
READ_STRING 730, 740
REASSIGN_NODE 503
RECORDING_METHOD 655
REDEFINE_ATTRIBUTE_KIND 419
REFRESH_VIRTUAL_DISPLAY 712
RELATION_DEFINITION_NKD 361
RELATION_INTERPRETATION_NKD 360
RELEASE_CHANNEL 593
REMOVE_AREA_QUALIFIER 1355
RENAME 134
RENAME_ATTRIBUTE_DEFINITION 418
RENAME_ATTRIBUTE_KIND 431
RENAME_NODE_KIND 377
RENAME_RELATION 394
REPLACE 767, 775, 782, 789, 794, 799
REQUEST_DISMOUNT 651
REQUEST_MOUNT 649
RESET 638, 659, 669, 680, 693
RESUME_PROCESS 552
REWIND_TAPE 652
SCREEN_OCCLUDED 722
SECONDARY_STORAGE_NKD 571
SECONDS 809
SET_ACTIVE_POSITION 1356
SET_CURRENT_NODE 154
SET_CURSOR_POSITION 722
SET_DEFINITION_ATTRIBUTE 460
SET_GRANTED_RIGHTS 271
SET_INDICATOR 857
SET_INHERITANCE 151
SET_LINE_LENGTH 695
SET_LOCATOR_CURSOR_POSITION 737
SET_NODE_ATTRIBUTE 192, 201, 211, 220, 229, 239
SET_PAGE_LENGTH 696
SET_PATH_ATTRIBUTE 193, 203, 212, 222, 231, 240
SET_PORT_ATTRIBUTE 599
SET_PRIORITY 554
SET_PROPERTY_ATTRIBUTE 464
SET_TO_EMPTY_LIST 753
SIZE 716
SKIP_BLOCK 656
SKIP_NEXT 176, 183, 253
SKIP_TAPE_MARK 653
SOUND_BELL 731
SPAWN_PROCESS 538
SPECIALIZE_NODE_KIND 378
SPECIALIZE_RELATION 395
SPLICE 758
SPLIT 810
STATUS 654
STATUS_INFORMATION 859
STRUCTURAL_NKD 280
SUSPEND_PROCESS 551
SYNCHRONIZE 670, 682, 694
TERMINAL_SIZE 1363
TERMINATION_KEY 1362
TEXT_FORM 754, 772, 781, 787, 793
TEXT_LENGTH 763
TIME_ATTRIBUTE_WRITTEN 157
TIME_CONTENTS_WRITTEN 157
TIME_CREATED 155



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

TIME_FINISHED 557
TIME_OF 811
TIME_RELATIONSHIP_WRITTEN 156
TIME_STARTED 556
TRANSMIT 610
UNADOPT_INTERPRETATION 288
UNADOPT_ROLE 278
UNADOPT_VIEW 285
UNIQUE_IDENTIFICATION 116
UNLOAD 651
UNPASTE_SCREEN 719
VIEW_NKD 359
WRITE 636
WRITE_BLOCK 658
WRITE_RESULTS 547
WRITE_TAPE_MARK 654
YEAR 808
"<" 812
"+" 811
"-" 812
internal files 20, 567
interoperability 1, 20
interpretation 20
adopted 60, 355
node kind 347
relation 349
interpretation node 20, 59, 351, 358
introduced
attributes 20, 74
contents 20
node sets 20, 74
nodes 20, 74
relationships 20, 74
INTRODUCE_CONTENTS 616
INTRODUCE_NODES 612
INTRODUCED_KEY_ATTWBUTES 74
INTRODUCED_NODE 74
INTRODUCED_NODE_ATTNBUTES 74
INTRODUCED_NODE_NKD 604
INTRODUCED_NODE_RD 605
INTRODUCED_RELATIONSHIP_ATTRIBUTES 74
INTRODUCED_RELATIONSHIPS 74
introduction 20, 74
IN_FILE 632, 644
IN_INTENT 569
INVOKE_PROCESS 510, 518, 540
IO_UNIT_COUNT 513, 556
IS_APPROVED 276
IS_DEFINED 166
IS_ENUMERATED 778
IS_EQUAL 755, 773
IS_ESTABLISHED 860
IS_FORM_UPDATED 1361
IS_INCLUDED 565
IS_INHERITABLE 153, 521
IS_OBTAINABLE 123
IS_OF_KIND 113
IS_OF_RELATION 114
IS_OPEN 109, 564, 635, 648, 707, 1350
IS_PRIMARY 168
IS_SAME 124, 565, 718
IS_SUB_KIND 110
IS_SUB_RELATION 112
IS_ WRITE_RING_INSTALLED 655
ITALIC 703
item name 20, 745
item value 20, 745
ITEM_KIND 751
ITEM_KIND_ERROR 752
iteration 20, 158
iteration predicates 20, 158
ITERATION_PREDICATE 159
iterators 20, 158
ITERATOR_ERROR 92

## J

job 21
JOB 46

## K

key value associations 21, 51
key 21
name 349
relationship 41
keyword member 255
keywords 255
KIND_OF_ITEM 757
KIND_OF_LIST 756
KIND_OF_NODE 110

## L

LAST_KEY 122
LAST_RELATION 122
latest key 21, 51
LATEST_KEY 91



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

lattice 21, 305
NKD 58
RD 58
partition 306
partitioning 59
LAYOUT_ERROR 570
line 21
linear list 21, 745
list AKD 323
list item 21, 745
list management 2
lists 21, 317, 745
LIST_ATTRIBUTE_KIND_DEFINITION 358
LIST_ITEM_KIND 751
LIST_KIND 751
LIST_KIND_ERROR 752
LIST_POSITION_ERROR 752
LIST_SIZE 751
LIST_TEXT 751
LIST_TYPE 90
LIST_TYPE 751
LITERAL 167
LOAD 650
LOADED 644
local client nodes 21, 66
local name 21, 59, 347, 349
local PORT relationships 21, 66
LOCATOR_CURSOR_ON_BORDER 737
LOCATOR_CURSOR_POSITION 735
LOCATOR_ IS_ ENABLED 735
LOCATOR_STATUS_ERROR 702
locks 95, 96, 97, 516
LOCK_ERROR 92, 351
LONG_ DELAY 91
LONG_IO_DELAY 569
LOWER_BOUND 318
LOWEST_USER_PRIORITY 553

## M

MACHINE_TIME 513, 558
MAKE_CONTAINING_LIST_CURRENT 761
MAKE_THIS_ITEM_CURRENT 762
mandatory access control 21, 33, 62, 96, 263
MANDATORY_ACCESS_LIST 90
MATCHES 163
maximum cell 21
maximum column 21
maximum row 22
MAXIMUM_FILE_SIZE 570
MEDIUM nodes 22, 77, 78, 80, 82, 503
medium resource 22, 76
message 22
MESSAGE 860
MESSAGE_LENGTH_LIMIT 582
MESSAGE_TEXT 847
migration 22, 336
minimum cell 22
minimum column 22
minimum row 22
MODE 635, 649
MODE parameters 580, 582
MODE_ERROR 570
MONITOR nodes 22, 61, 477
MONITORED_BY 477
MONITORS 477
MONITOR_EVENT_KIND 481
MONITOR_NKD 481
MONITOR_NODE_ATTIUBUTE 477, 491
MONITOR_PATH_ATTRIBUTE 494
MONTH 808
MONTH_NUMBER 807
MORE 169, 173, 180, 250
MOUNTED 644
MOUNT_REQUESTED 644
MOVE_PHYSICAL_DISPLAY_ORIGIN 709
MOVE_SCREEN_TO_BACK 721
MOVE_SCREEN_TO_FRONT 720
MOVE_SCREEN 720
multiple channels 65
multiple keys 51

## N

name 22, 59
ambiguous formation 356
ambiguous translation 356
attribute 347, 349
cannot be formed 357
cannot be translated 357
formation 22, 59
key 349
local 347, 349
relation 347
translation 22, 59
NAMED 751
named composite list AKD 324
named item 22, 745


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

named list 22, 745
NAME_ATTRIBUTE 452
NAME_ERROR 93
NAME_KEY 443
NAME_OF_DEFINITION 294
NAME_OF_NODE_ATTRIBUTE 299
NAME_OF_PATH_ATTRIBUTE 301
NAME_OF_RELATION 296
NAME_SOURCE 438
NAMED_COMPOSITE_LIST_AKD 358
NAMED_COMPOSITE_LIST_AKD_NKD 362
NAMED_LIST_ERROR 752
NAMED_ORDERED_COMPOSITE_LIST_AKD 358
naming 22, 355
  applicable 355
  constraint items 334
  visible 356
native nodes 74
necessary right 22, 259
nested list structure 22, 747
nested sublist 23, 747
NEW_LINE 724
NEXT_NAME 169, 177, 251
NKDs 66, 158
  CHANNEL 67
  deleting 337
  DEVICE 66
  FILE 66
  GATEWAY 66
  operations 358
  predefine name 358
  PROCESS 66
NKI 347
  operations 358
  predefied name 358
NO_ACCESS 68, 97, 124
NO_FILL 1348
NO_INDICATION 845
NO_PROTOCOL 577
node attributes 521, 560
node handle 23
node iteration 23, 158
node iterators 23, 169
node kind 23
node kind definition 23, 55
node kind definition lattice 23
node kind definition node (NKD) 23
node kind instance iterators 23, 178
node kind interpretation 23
node kind interpretation node (NKI) 23, 59
node management 87
node model 1, 23, 33
node set handle 23, 604
node sets 23, 70
nodes 23
  creating 97
  dependent PROCESS 46
  identification 50
  included 514, 515
  non_existing 42
  PROCESS 48
  root PROCESS 46
  system-level 45
  top-level 45
  traversal 54, 96
  unobtainable 42
NODE_ATTRIBUTE 159, 165, 204, 214, 223, 242
NODE_HANDLE_ARRAY 88
NODE_ITERATOR 169
NODE_KIND_ARRAY 169
NODE_KIND_DEFINITION 358
NODE_KIND_DEFINITION_NKD 360
NODE_KIND_ERROR 93
NODE_KIND_INTERPRETATION 358
NODE_KIND_INTERPRETATION_NKD 359
NODE_KIND_NAME 90
NODE_SET_TYPE 604
NODE_TYPE 87, 88, 89, 125
  subtypes 89
non-existing nodes 23, 42
non-hierarchical category 263
non-transparent resource 23, 76
NON_RETURN_TO_ZERO_INVERTED 644
NONE 1348
NORMAL 1348
NOT_IN_VIEW_ERROR 93, 357
null key 23
NULL_NODE 91
NULL_TRANSACTION 518, 560
NUMBER_OF_BUTTONS 738
NUMBER_OF_FUNCTION_KEYS 733, 1351
NUMBER_OF_ITEMS 760
NUMERICS 1348


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

## O

object classification 256
object 24, 62
OBJECT_CLASSIFICATION 478
obtainable 24, 41
open file handle 24, 567
OPEN interface 582, 580
open node handle 24, 87
open node set handle 24
open operation 95
  implicit 96
open screen handle 24
open transaction handle 24, 514
OPEN 95, 103, 578, 633, 646, 667, 678, 690, 704, 1348
OPENED 644
opening protocols 67
OPEN_ATTRIBUTE_DEFINITION 292
OPEN_ATTRIBUTE_KIND_DEFINITION 293
OPEN_DENIED 570
OPEN_FILE_HANDLE_COUNT 116
0PEN_HANDLE_COUNT 555
OPEN_NODE_HANDLE_COUNT 513
OPEN_NODE_KIND 290
OPEN_NODE_KIND_DEFINITION 289
OPEN_NODE_SET 605
OPEN_OPPOSITE_CLIENT 597
OPEN_PARENT 95, 125
OPEN_RELATION 291
OPEN_RELATION_DEFINITION 291
OPEN_TRANSACTION 514, 560
OPEN_VIA_UNIQUE_IDENTIFIER 95, 106
operating system 1, 2
operations 24, 44
  included 514, 515
  on a node 24, 515
operators
  AND_THEN 164
  MATCHES 163
  OR_ELSE 164
opposite CAIS instance 24, 85
opposite channel 24, 85
opposite client node 24, 66
opposite GATEWAY node 24, 85
opposite PORT relationship 24, 85
ordered composite list AKD 324
ORDERED_COMPOSITE_LIST_AKD 358
ordinary identifier AKD 322
OR_ELSE 164
OTHER 644
outgoing queue 24
OUTGOING_QUEUE_FULL 641
OUT_FILE 632, 644
OUT_INTENT 569

## P

package specifications 35
packages 87
  CAIS_ACCESS_CONTROL_MANAGEMENT 87, 255
  CAIS_ATTRIBUTE_MANAGEMENT 87,185
  CAIS_ATTRIBUTE_MONITOR_-MANAGEMENT 87, 477
  CAIS_BOOLEAN_ITEM 786
  CAIS_CALENDAR 807
  CAIS_COMMON_IO 604
  CAIS_DEFINITION_MANAGEMENT 87, 95, 304
  CAIS_DEFINITIONS 87, 88, 95, 185, 280
  CAIS_DIRECT_IO 661
  CAIS_ENUMERATION_ITEM 780
  CAIS_FILE_NODE_MANAGEMENT 571
  CAIS_FLOAT_ITEM 792
  CAIS_FORM_TERMINAL_IO 1346
  CAIS_FRAME_IO 630
  CAIS_IDENTIFIER_ITEM 771
  CAIS_IMPORT_EXPORT 602
  CAIS_INTEGER_ITEM 787
  CAIS_IO_CONNECTION 575
  CAIS_IO_DEFINITIONS 569
  CAIS_LIST_ITEM 766
  CAIS_LIST_MANAGEMENT 159, 185, 751
  CAIS_MAGNETIC_TAPE_IO 643
  CAIS_NODE_MANAGEMENT 87, 95
  CAIS_PRAGMATICS 95, 185, 280, 815
  CAIS_PROCESS_DEFINITIONS 520
  CAIS_PROCESS_MANAGEMENT 478,521
  CAIS_RESOURCE_CONTROL 87, 503
  CAIS_SEQUENTIAL_IO 672
  CAIS_STANDARD 805
  CAIS_STATUS_MANAGEMENT 857
  CAIS_STRING_ITEM 798
  CAIS_STRUCTURAL_NODE_MANAGEMENT 87, 95, 280
  CAIS_TERMINAL_IO 698
  CAIS_TEXT_IO 684



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

CAIS_TRANSACTION_MANAGEMENT 560
CAIS_TRANSLATION_MANAGEMENT 87, 95, 284
general descriptions 33
parameters
MODE 580
INTENT 95
PARAMETERS 513
PARAMETER_LIST 520
parent (of a node) 25
parent (of a transaction) 25
parent nodes 41
parent relationship 25
parent transaction 514
PARENT 41, 46
PASTE_POSITION 721
PASTE_SCREEN 719
pasting order 25
pasting 25
path element 25, 53
path 25, 53
PATHNAME 90
PATHNAME_SYNTAX_ERROR 93
pathnames 25, 50, 51, 53
paths
element 53
name 53
primary 53
PATH_ATTRIBUTE 206, 215, 225, 243
PATH_KEY 119
PATH_RELATION 120
PATTERNS 163
PATTERNS attribute 318, 322
patterns constraint items 331
PHASE_ENCODED 644
physical cursor 25
PHYSICAL_DISPLAY_ORIGIN 708
PHYSICAL_DISPLAY_DIMENSION 701
PHYSICAL_DISPLAY_POSITION 701
PORT 66, 580
PORT relationships 65, 66, 67, 68, 575, 576, 580
PORT_ERROR 570
PORT_RD 589
POSITION 652
POSITION_BY_NAME 765
POSITION_BY_VALUE 770, 777, 785, 791, 797, 802
POSITION_COUNT 751
POSITION_OF_CURRENT_LIST 761
potential member 25, 258
pragmatic 2, 25
predefine
definitions 44
entities 342
relations 46
PREDEFINED_ATTRIBUTE_ERROR 93
PREDEFINED_RELATION_ERROR 93
predicate functions
EXTRACT_VALUE 164
HAS_EMANATING 166
IS_DEFINED 166
IS_PRIMARY 168
NODE_ATTRIBUTE 159
RELATIONSHIP_ATTRIBUTE 165
primary relationship 25, 41
PRIMARY_KEY 118
PRIMARY_NAME 117
PRIMARY_RELATION 118
PRIMARY_RENDITION 703
PRINTABLE_CHARACTER 1348
PRIORITIES_ARE_MODIFIABLE 554
PRIORITY 513
PRIORITY_TYPE 632
PROCESS 66
PROCESS nodes 25, 40, 48, 65, 67, 69, 80, 82, 91, 481, 509, 520, 521, 575, 580, 582
contents 40
process tree 26, 511
processes 25, 43, 509
aborted 510
Ada tasks 43
closing 67
completed 510
created 510
creation 68, 509
current 53
dependent 46
enclosed 514
initiated 43
initiating 43
job 46
model 2
ready 510
root 46
suspended 510
terminated 510



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

PROCESSOR nodes 26, 77, 80, 82, 93, 503
processor resource 26, 76
PROCESS_PRIORITY 520
PROCESS_SIZE 513, 558
PROCESS_STATUS_KIND 520
program 26
PROTECTED 1348
protocol attributes 576
protocol CLOSE interfaces 26, 64
protocol CLOSE operations 26, 64
protocol closing 26, 67
protocol CREATE 64
protocol OPEN interfaces 26, 64
protocol OPEN operations 26, 64
protocol opening 26, 67
protocol packages 26, 64
protocol parameter attributes 26, 582
protocol parameters 26, 576
protocol range attributes 26, 576
protocols 26,64
CAIS_DIRECT_IO 64
CAIS_FRAME_IO 64
CAIS_MAGNETIC_TAPE_IO 64
CAIS_SEQUENTIAL_IO 64
CAIS_TERMINAL_IO 64
CAIS_TEXT_I064
CLOSE operation 64
OPEN operation 64
opening 578
PROTOCOL_ERROR 570
PUT 513, 723, 1358
PUT_LINE 725

## Q

qualifier 833
queue 26
QUEUE_EMPTY 632
QUEUE_FULL 632

## R

range constraint items 331
range 26
RAPID_BLINK 703
RD
operations 358
predefine name 358
RDs 158
PORT 66, 67
deleting 337
READ 637
ready 26
READ_ATTRIBUTES 578
READ_BLOCK 657
READ_INPUT_EVENT 739
READ_RELATIONSHIPS 578
READ_STRING 730, 740
REASSIGN_NODE 503
reconciliation 74
RECORDING_METHOD 655
REDEFINE_ATTRIBUTE_KIND 419
REFRESH_VIRTUAL_DISPLAY 712
relation definition 27, 55
relation definition lattice 27
relation definition node (RD) 27
relation interpretation 27
relation interpretation node (RI) 27, 59
relation names 27, 50, 41, 347
relations 27, 41
predefine 46
relationship attribute 27
relationship keys 27, 51
relationship key designator 27, 51
relationship key list 27, 51
relationship key value 27
relationships 27, 41
ADOPTED_VIEW 351
allocatability 80
ALLOCATABLE_DEVICE 80
ALLOCATABLE_GATEWAY 80
ALLOCATABLE_MEDIUM 80
ALLOCATABLE_PROCESSOR 80
ATTRIBUTE 334
ATTRIBUTE_NAME 347, 349
attributes 576
CHANNEL 577, 578
DEFAULT_ MEDIUM 80, 82
DEFAULT_PROCESSOR 80, 82
DEFINITION 347
EMANATES 308
identification 50
key 41
KEY 334
KEY_NAME 349
PARENT 41
PORT 65, 66, 67, 68, 575, 576, 580
primary 41


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

RELATION_NAME 347
secondary 41
source node 41
target node 41
TERMINATES 308
VIEW 354
RELATIONSHIP_ATTRIBUTE 165
RELATIONSHIP_ERROR 93
RELATIONSHIP_KEY 90
RELATIONSHIP_KEY_PATTERN 169
RELATIONSHIP_KIND 169, 358
RELATION_DEFINITION 358
RELATION_DEFINITION_NKD 361
RELATION_INTERPRETATION 358
RELATION_INTERPRETATION_NKD 360
RELATION_NAME 90
RELATION_NAME_PATTERN 169
RELEASE_CHANNEL 593, 578, 585, 586, 587
remote client process 27, 583
remote interprocess communication 27, 85
REMOVE_AREA_QUALIFIER 1355
RENAME 134
RENAME_ATTNBUTE_KIND 431
RENAME_ATTRIBUTE_DEFINITION 418
RENAME_NODE_KIND 377
RENAME_RELATION 394
rendition 27
RENDITION_TYPE 703
REPLACE 767, 775, 782, 789, 794, 799
reportable event 27, 479
REQUEST_DISMOUNT 651
REQUEST_MOUNT 649
RESET 638, 659, 669, 680, 693
resource control 33
resource group 28
resource model 28, 77
resource nodes 28, 77
RESOURCE nodes 582
resource 28
allocatability 80
RESOURCE_ALLOCATION_ERROR 93
RESOURCE_AVAILABILITY_ERROR 93
RESOURCE_ERROR 815
resulting rights list 28, 259
RESULTS 513
RESULTS_LIST 520
RESULTS_STRING 520
RESUME_PROCESS 552
RETIRED state 588
REVERSE_IMAGE 703
REWIND_TAPE 652
RI 349
operations 358
predefied name 358
role 28, 257
root PROCESS node 28, 46
root processes 46, 521
ROOT_PROCESS 520

## S

screen 28
screen handle 28
SCREEN_DIMENSION 702
SCREEN_NOT_ON_DISPLAY 702
SCREEN_OCCLUDED 722
SCREEN_POSITION 702
SCREEN_POSITION_ERROR 702
SCREEN_TYPE 702
SCROLL_TYPE 703
SEARCH_ERROR 752
secondary relationships 28, 41, 42, 80
DEFAULT_MEDIUM 80
DEFAULT_PROCESSOR 80
secondary storage file 28
SECONDARY_STORAGE_NKD 571
SECONDARY_STORAGE nodes 76
SECONDS 71, 809
security label 255
security level 28, 255, 256
SECURITY_VIOLATION 93
sequential operation 28
server scenario 28, 578
SET_ACTIVE_POSITION 1356
SET_CURRENT_NODE 154
SET_CURSOR_POSITION 722
SET_DEFINITION_ATTNBUTE 460
SET_GRANTED_NGHTS 271
SET_INDICATOR 857
SET_INHERITANCE 151
SET_LINE_LENGTH 695
SET_LOCATOR_CURSOR_POSITION 737
SET_NODE_ATTNBUTE 192, 201, 211, 220, 229, 239
SET_PAGE_LENGTH 696
SET_PATH_ATTRIBUTE 193, 203, 212, 222, 231, 240


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

SET_PORT_ATTRIBUTE 599
SET-PRIORITY 513, 554
SET-PROPERTY-ATTRIBUTE 464
SET-TO-EMPTY-LIST 753
simple attribute kind 28
single keys 51
SIZE 716
SKIP_BLOCK 656
SKIP-NEXT 169, 176, 183, 253
SKIP-TAPE-MARK 653
SLOW_BLINK 703
SMALL_FOR_CAIS_DURATION 71, 569
SOUND_BELL 731
source nodes 29, 41
SPAWN_PROCESS 510, 518, 538
specialization 29, 58, 59
    of predefine definitions 58
SPECIALIZE_NODE_KIND 378
SPECIALIZE_RELATION 395
SPLICE 758
SPLIT 810
STANDARD_ERROR 513, 520
STANDARD_INPUT 513, 520
STANDARD_OUTPUT 513, 520
STATE attribute 577
status indicator 29, 825
status information list 825
status management 2
STATUS 654
STATUS_ERROR 93
STATUS_INDICATOR 845
STATUS_INFORMATION 859
storage unit 29
stream 29
string AKD 318
STRING_ATTRIBUTE_KIND_DEFINITION 358
STRING_ITEM_KIND 751
stronger constraint 29
STRUCTURAL nodes 29, 40, 95, 280
    contents 40
STRUCTURAL_NKD 280
structured attribute kind 29
subject classification 256
subject 29, 62
SUBJECT_CLASSIFICATION 478
sublattice 59
sublist 747
subtransaction 29, 514
suspended 29
SUSPEND_PROCESS 551
synchronization 516
SYNCHRONIZE 670, 682, 694
syntax 35
SYNTAX_ERROR 93, 159, 752
system-level node 29, 45

## T

TAPE_BLOCK 644
TAPE_DRIVE_STATUS_KIND 644
TAPE_NAME 644
TAPE_POSITION_KIND 644
TAPE_RECORDING_METHOD_KIND 644
TAPE_STATUS_ERROR 646
target node 29, 41
tasks 29, 43, 44
TERMINAL_SIZE 1363
TERMINAL_POSITION_ERROR 1347
TERMINAL_POSITION_TYPE 1347
terminated 29, 509
terminating relations 30, 308
TERMINATION_KEY 1362
TERMINATOR_SET 704
TEXT_FORM 754, 772, 781, 787, 793
TEXT_LENGTH 763
TIME 71, 807
TIME_ATTRIBUTE_WRITTEN 157
TIME_CONTENTS_WRITTEN 71, 157
TIME_CREATED 155
TIME_ERROR 807
TIME_EXPIRED 570
TIME_FINISHED 513, 557
TIME_OF 811
TIME_RELATIONSHIP_WRITTEN 156
TIME_STARTED 513, 556
to identify 51
to obtain access 255
token form 30, 746
tokens 747
TOKEN_ERROR 752
TOKEN_TYPE 751
tool sets 1, 30
tools 1, 30
top-level GROUP node 257
top-level nodes 30, 45
track 30
tracking node handles 87


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

transaction handle 30
transactions 30, 44, 97, 514
abort 517
aborted 514
closed 514
closing 517
commit 517
committed 514
created nodes 515
descendants 514
effects 514
handle 514
interfaces 560
locks held 516
nested 514
open 514
opening 514
operations inside and outside 514
parent 514
visibility of effects 516
TRANSACTION_ERROR 93, 516
TRANSACTION_TYPE 560
TRANSMIT 610
transparent resources 30, 76, 82
TRANSPARENT_MEDIUM_RESOURCE node 82
TRANSPARENT_PROCESSOR_RESOURCE node 82
transportability 1, 30
traversal of a node 30, 54, 96
traversal of a relationship 31, 41
tree
operations 96
types
ACCESS_RIGHTS 271
AREA_INPUT_KIND 1348
AREA_INTENSITY_KIND 1348
AREA_PROTECTION_KIND 1348
AREA_VALUE_KIND 1348
ATTRIBUTE_KIND_NAME 90
ATTRIBUTE_LIST 90
ATTRIBUTE_NAME 90
BACKGROUND_COLOR_TYPE 703
BUTTON_NAME 704
CAIS_DURATION 805
CAIS_INTEGER 162, 805
CAIS_IO_DURATION 569
CAIS_NATURAL 805
CAIS_POSITIVE 805
CHANNEL_STATE 569
CHARACTER_ARRAY 1347
CONDITION_CODE 847
DAY_DURATION 807
DAY_NUMBER 807
DEFINITION_NAME 284
DISCRETIONARY_ACCESS_LIST 90
ECHO_TYPE 704
EXCEPTION_INDICATION_KIND 845
FILE_MODE 632, 644, 662, 673, 685
FILE_TYPE 632, 644, 662, 673, 685, 1347
FOREGROUND_COLOR_TYPE 703
FORM_TYPE 1347
FRAME_ADDRESS 632
FRAME_LENGTH 632
FUNCTION_KEY_NAME 704, 1347
GRAPHIC_RENDITION_KIND 703
GRAPHIC_RENDITION_SET 703
IDENTIFIER_TEXT 751
INPUT_EVENT_KIND 704
INSERT_COUNT 751
INTENT 88
INTENT_ARRAY 88
INTENT_SPECIFICATION 88
ITEM_KIND 751
ITERATION_PREDICATE 159
LIST_KIND 751
LIST_SIZE 751
LIST_TEXT 751
LIST_TYPE 90, 751
MANDATORY_ACCESS_LIST 90
MESSAGE_TEXT 847
MONITOR_EVENT_KIND 481
MONTH_NUMBER 807
NODE_HANDLE_ARRAY 88
NODE_ITERATOR 169
NODE_ KIND_ ARRAY 169
NODE_ KIND_INSTANCE_ITERATOR 178
NODE_KIND_NAME 90
NODE_SET_TYPE 604
NODE_TYPE 88, 89
PARAMETER_LIST 520
PATHNAME 90
PHYSICAL_DISPLAY_DIMENSION 701
PHYSICAL_DISPLAY_POSITION 701
POSITION_COUNT 751
PRINTABLE_CHARACTER 1348


Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

PROCESS_PRIORITY 520
PROCESS_STATUS_KIND 520
RELATION_NAME 90
RELATION_NAME_PATTERN 169
RELATIONSHIP_KEY 90
RELATIONSHIP_KEY_PATTERN 169
RELATIONSHIP_KIND 169, 358
RENDITION_TYPE 703
RESULTS_LIST 520
RESULTS_STRING 520
SCREEN_DIMENS1ON 702
SCREEN_POSITION 702
SCREEN_TYPE 702
SCROLL_TYPE 703
STATUS_INDICATOR 845
TAPE_BLOCK 644
TAPE_DRIVE_STATUS_KIND 644
TAPE_NAME 644
TAPE_POSITION_KIND 644
TAPE_RECORDING_METHOD_KIND 644
TERMINAL_POSITION_TYPE 1347
TERMINATOR_SET 704
TIME 807
TOKEN_TYPE 751
TRANSACTION_TYPE 560
UNIQUE_IDENTIFIER 90
UNIQUE_IDENTIFIER_LIST 90
VALUE_TYPE 359
VIRTUAL_DISPLAY_COORDINATE 701
VIRTUAL_DISPLAY_POSITION 701
WRAP_TYPE 704
YEAR_NUMBER 807

## U

UID 54
UID_ERROR 604
unadopt a role 31, 258
UNADOPT_INTERPRETATION 288
UNADOPT_ROLE 278
UNADOPT_VIEW 285
UNBOUNDED_FILE_SIZE 569
undefined token 31, 747
UNDERSCORE 703
unidirectional relationships 31, 41
uniform list AKD 323
uniform list size constraint items 333
uniform lists 31, 317
UNIFORM_ LIST_ ATTRIBUTE_ KIND_-
DEFINITION 358
unique identfier 31, 54
unique key set 31, 310
unique primary path 31, 53
UNIQUE_IDENTIFICATION 116
UNIQUE_IDENTIFIER 90
UNIQUE_IDENTIFIER_LIST 90
UNLOAD 651
unnamed item 31, 745
unnamed list 31, 745
UNNAMED 751
unobtainable 31, 42
UNPASTE_SCREEN 719
unpasting 31
UNPROTECTED 1348
UPPER_BOUND 318
USER 46
user names 31, 46
USER nodes 31, 46, 85
users 31, 45
USE_ERROR 94

## V

value constraint items 329
VALUE_TYPE 359
variant AKD 322
VARIANT_ATTRIBUTE_KIND_DEFINITION
358
view 31, 59, 347
adopted 60
adopted 351
ambiguous 356
composite 60, 351, 356
implicit adoption 354
inheritance 354
operations 351, 355, 358
precedence 351, 354, 355
predefine 351
predefine name 358
user-defined 351
using 355
VIEW 358
VIEW node 32, 35, 358
VIEW_NKD 359
virtual display 32
virtual keyboard 32
virtual locator 32



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

virtual terminal 32
VIRTUAL_DISPLAY_COORDINATE 701
VIRTUAL_DISPLAY_POSITION 701
VIRTUAL_DISPLAY_POSITION_ERROR 701
visibility relationships 32, 80
visible groups 32, 80
visible in a composite view 32, 356
visible in a view 32, 356
visible resource 32
VISIBLE_ATTRIBUTES 246

## W

weaker constraint 32
WRAP_TYPE 704
WRITE 636
WRITE_BLOCK 658
WRITE_RESULTS 547
WRITE_TAPE_MARK 654

## Y

YEAR 808
YEAR_NUMBER 807

“<” 812
“+” 811
“-” 812



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Preparing Activity:
Air Force - 02

(Project MCCR/IPSC 0208)

Custodians:

Army - CR
Navy - EC
Air Force - 10

Review Activities:

Army - CR, AV
Navy - SH, TD
Air Force -17, 02
Other - DC

User Activities:

Army -
Navy - OM, Naval Telecommunications Command
Air Force -
Other - DC, DH, Defense Mapping Agency

Agent: Ada Joint Program Office



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

# Postscript: Submission of Comments

For submission of comments on MIL-STD-1838A, we would appreciate them being sent by ARPANET/ MILNET to the address:

caiscom@ajpo.sei.cmu.edu

If you do not have network access, please send the comments by mail to:

Ada Joint Program Office
Room 3E114, Pentagon
Washington, DC 20301-3081

For mail comments, it will assist us if you are able to send them on 5 l/4-inch double-sided double-density 360 Kbyte floppy diskette formatted for MS-DOS. But even if you can manage this, please also send us a paper copy, in case of problems with reading the diskette.

All comments are sorted and processed mechanically in order to simplify their analysis and to facilitate giving them proper consideration. To aid this process you are kindly requested to precede each comment with a seven-line header.

!Section . . .
!Version MIL-STD-1838A
!Name . . .
!Affiliation . . .
!Date . . .
!Topic . . .
!Rationale . . .

The section line includes the section number and the paragraph number enclosed in parentheses. The name line contains your name. The affiliation line should give your affiliation. The date line should provide the date in ISO standard form (year-month-day). As an example, here is the section line of a comment from a previous version:

!Section 04.06.01(8)

The version line, for comments on the military standard., should only contain “MIL-STD-1838A”. Its purpose is to distinguish comments that refer to different versions.

The topic line should contain a one line summary of the comment. This line is essential, and you are kindly asked to avoid topics such as “Typo” or “Editorial comment” which will not convey any information when printed in a table of contents. As an example of an informative topic line, consider:

!Topic RESOURCE CONTROL

Note also that nothing prevents the topic line from including all the information of a comment, as in the following t epic line:

!Topic Insert: “.. are (implicitly) defined by...”



Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

As a final example here is complete comment:

```
!Section  04.06.01(8)
!Version  MIL-STD-1838A
!Name R. Thall
!Affiliation SofTech, Inc.
!Date  88-05-16
!Topic Change “a non-transparent or transparent” to “any” in line 5.
!Rationale Simpler but equivalent wording.
```

Comments on the technical and/or editorial content of MIL-STD-1838A should be submitted according to the instructions above. The format of MIL-STD-1838A complies with MIL-STD-962B, “Military Standards, Handbooks, and Bulletins, Preparation of.” If you have comments relating to items specified by MIL-STD-962B, please use the following form DD 1426 and submit these comments to the address given on that form.



*U.S. GOVERNMENT PRINTING OFFICE: 1990 704-034/30413

Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

**INSTRUCTIONS:** In a continuing effort to make our standardization documents better, the DoD provides this form for use in submitting comments and suggestions for improvements. All users of military standardization documents are invited to provide suggestions. This form may be detached, folded along the lines indicated, taped along the loose edge *(DO NOT STAPLE)*, and mailed. In block 5, be as specific as possible about particular problem areas such as wording which required interpretation, was too rigid, restrictive, loose, ambiguous, or was incompatible, and give proposed wording changes which would alleviate the problems. Enter in block 6 any remarks not related to a specific paragraph of the document. If block 7 is filled out, an acknowledgement will be mailed to you within 30 days to let you know that your comments were received and are being considered.

*NOTE:* This form may not be used to request copies of documents, nor to request waivers, deviations, or clarification of specification requirements on current contracts. Comments submitted on this form do not constitute or imply authorization to waive any portion of the referenced document*(s)* or to amend contractual requirements.

---

*(Fold along this line)*

*(Fold along this line)*

---

DEPARTMENT OF THE AIR FORCE

NO POSTAGE NECESSARY IF MAILED IN THE UNITED STATES

OFFICIAL BUSINESS
PENALTY FOR PRIVATE USE $300

BUSINESS REPLY MAIL
FIRST CLASS PERMIT NO. 73236 WASHINGTON D. C.
POSTAGE WILL BE PAID BY THE DEPARTMENT OF THE AIR FORCE

ADA JOINT PROGRAM OFFICE (AJPO)
Rm 3E114, The Pentagon
WASHINGTON, D.C. 20301

Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.

# STANDARDIZATION DOCUMENT IMPROVEMENT PROPOSAL

*(See Instructions – Reverse Side)*

| 1. DOCUMENT NUMBER | 2. DOCUMENT TITLE |
|---|---|
| | |

| 3a. NAME OF SUBMITTING ORGANIZATION | 4. TYPE OF ORGANIZATION *(Mark one)* |
|---|---|
| | ☐ VENDOR<br>☐ USER<br>☐ MANUFACTURER<br>☐ OTHER *(Specify)*: _____ |
| b. ADDRESS *(Street, City, State, ZIP Code)* | |
| | |

5. PROBLEM AREAS

a. Paragraph Number and Wording:

b. Recommended Wording:

c. Reason/Rationale for Recommendation:

6. REMARKS

| 7a. NAME OF SUBMITTER *(Last, First, MI)* – Optional | b. WORK TELEPHONE NUMBER *(Include Area Code)* – Optional |
|---|---|
| c. MAILING ADDRESS *(Street, City, State, ZIP Code)* – Optional | 8. DATE OF SUBMISSION *(YYMMDD)* |
| | |

DD FORM 1426 PREVIOUS EDITION IS OBSOLETE.

Source: https://assist.dla.mil -- Downloaded: 2026-10-26T07:54Z
Check the source to verify that this is the current version before use.